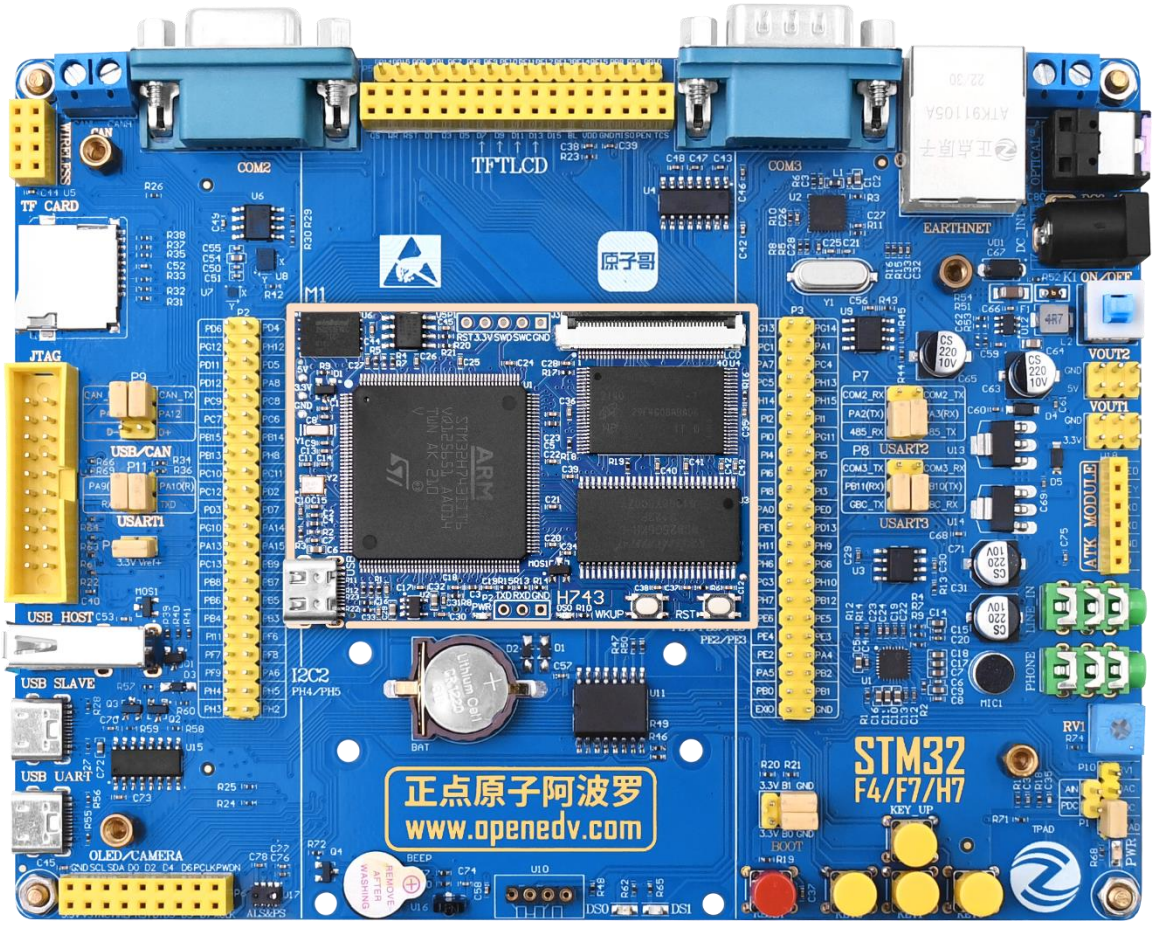
STM32H7 开发指南

V1.2

—正点原子 阿波罗 STM32H743 开发板教程



修订历史：

版本	日期	修改内容
V1.0	2023/05/17	第一次发布
V1.1	2025/08/01	1.第四十七章节由 SPA06-003 气压计实验替换 ST480MC 磁力计实验，第四十八章六轴传感器实验由 QMI8658A 替换原来的 QMI8658A。 2.Nand Flash 芯片新增 FSNS8B004G 3.综合例程 9D 测试和水平仪 APP 也做出相应调整。
V1.2	2026/2/28	1.添加支持多种 SDRAM 声明

正点原子公司名称 ：广州市星翼电子科技有限公司
原子哥在线教学平台：www.yuanzige.com
开源电子网 / 论坛：www.openedv.com/forum.php
正点原子官方网站 ：www.alientek.com
正点原子淘宝店铺 ：https://openedv.taobao.com
正点原子 B 站视频 ：https://space.bilibili.com/394620890

电话：020-38271790 传真：020-36773971

请下载原子哥 APP，数千讲视频免费学习，更快更流畅。
请关注正点原子公众号，资料发布更新我们会通知。

扫码下载“原子哥”APP

扫码关注正点原子公众号

内容简介 33
第一篇 基础篇 35
第一章 本书学习方法 36
1.1 本书学习顺序 36
1.2 本书参考资料 36
1.3 本书编写规范 36
1.4 本书代码规范 37
1.5 例程资源说明 37
1.6 学习资料查找 39
1.7 给初学者的建议 41
第二章 STM32 简介 42
2.1 初识 STM32 42
2.2 STM32H743 资源简介 43
2.3 STM32H743 设计选型 43
2.3.1 STM32 系列 43
2.3.2 STM32 命名 44
2.3.3 STM32 选型 45
2.3.4 STM32 设计 46
第三章 开发环境搭建 50
3.1 常用开发工具简介 50
3.2 MDK 安装 50
3.3 仿真器驱动安装 51
3.4 CH340 USB 虚拟串口驱动安装 51
第四章 STM32 初体验 53
4.1 使用 MDK5 编译例程 53
4.2 使用串口下载程序 56
4.3 使用 DAP 下载与调试程序 60
4.3.1 使用 DAP 下载程序 60
4.3.2 使用 DAP 仿真调试程序 63
4.3.3 仿真调试注意事项 68
4.4 MDK5 使用技巧 68
4.4.1 文本美化 68
4.4.2 语法检测&代码提示 71

4.4.3 代码编辑技巧 72
4.4.4 其他小技巧 76
第五章 STM32 基础知识入门 79
5.1 C 语言基础知识复习 79
5.1.1 位操作 79
5.1.2 define 宏定义 80
5.1.3 ifdef 条件编译 80
5.1.4 extern 外部申明 80
5.1.5 typedef 类型别名 81
5.1.6 结构体 81
5.1.7 指针 82
5.2 寄存器基础知识 83
5.3 STM32H743 系统架构 84
5.3.1 Cortex M7 内核 & 芯片 84
5.3.2 STM32 系统架构 84
5.3.3 存储器映射 87
5.3.4 寄存器映射 90
第六章 新建寄存器版本 MDK 工程 95
6.1 新建寄存器版本 MDK 工程 95
6.1.1 新建工程文件夹 95
6.1.2 新建一个工程框架 97
6.1.3 添加文件 99
6.1.4 魔术棒设置 103
6.1.5 添加 main.c，并编写代码 108
6.2 下载验证 109
第七章 认识 HAL 库 111
7.1 初识 STM32 HAL 库 111
7.1.1 CMSIS 标准 111
7.1.2 HAL 库简介 112
7.1.3 HAL 库能做什么 114
7.2 HAL 库驱动包 114
7.2.1 如何获取 HAL 库固件包 115
7.2.2 STM32Cube 固件包分析 116

7.2.3 CMSIS 文件夹关键文件 118
7.2.4 stdint.h 简介 120
7.3 HAL 库框架结构 120
7.3.1 HAL 库文件夹结构 120
7.3.2 HAL 库文件介绍 121
7.4 如何使用 HAL 库 124
7.4.1 学会用 HAL 库组织开发工具链 124
7.4.2 HAL 库的用户配置文件 125
7.4.3 stm32h7xx_hal.c 文件 127
7.4.4 HAL 库中断处理 132
7.4.5 正点原子对 HAL 库用法的个性化修改 133
7.5 HAL 库使用注意事项 133
第八章 新建 HAL 版本 MDK 工程 135
8.1 新建 HAL 库版本 MDK 工程 135
8.1.1 新建工程文件夹 135
8.1.2 新建一个工程框架 138
8.1.3 添加文件 140
8.1.4 魔术棒设置 146
8.1.5 添加 main.c，并编写代码 151
8.2 下载验证 153
第九章 STM32 启动过程分析 154
9.1 启动模式 154
9.2 启动文件分析 155
9.2.1 启动文件中的一些指令 155
9.2.2 启动文件代码讲解 156
9.2.3 系统启动流程 161
9.3 map 文件分析 163
9.3.1 MDK 编译生成文件简介 163
9.3.2 map 文件分析 164
第十章 STM32CubeMX 简介 168
10.1 STM32CubeMX 的作用 168
10.2 安装 STM32CubeMX 169
10.2.1 安装 JAVA 环境 169

10.2.2 安装 STM32CubeMX 169
10.3 使用 STM32CubeMX 新建工程 172
10.3.1 打开 STM32CubeMX 172
10.3.2 下载和关联的 STM32Cube 固件包 173
10.3.3 新建工程 175
10.4 STM32CubeMX 新建工程使用建议 186
第十一章 STM32 时钟系统 187
11.1 认识时钟树 187
11.1.1 时钟源 188
11.1.2 锁相环 PLL 189
11.1.3 系统时钟 SYSCLK 190
11.1.4 外设内核时钟 192
11.1.5 MCO 时钟输出 193
11.2 如何修改主频 194
11.2.1 STM32H7 时钟系统配置 194
11.2.2 STM32H7 时钟使能和配置 201
第十二章 SYSTEM 文件夹介绍 203
12.1 deley 文件夹代码介绍 203
12.1.1 操作系统支持宏定义及相关函数 204
12.1.2 delay_init 函数 205
12.1.3 delay_us 函数 206
12.1.4 delay_ms 函数 207
12.1.5 HAL 库延时函数 HAL_Delay 208
12.2 sys 文件夹代码介绍 209
12.2.1 Cache 使能函数 209
12.3 usart 文件夹代码介绍 210
12.3.1 printf 函数支持 210
第二篇 入门篇 213
第十三章 跑马灯实验 214
13.1 STM32H7 GPIO 简介 214
13.1.1 GPIO 功能模式 215
13.1.2 GPIO 基本结构分析 215
13.1.3 GPIO 寄存器介绍 220

13.2 硬件设计 223
1. 例程功能 223
2. 硬件资源 224
3. 原理图 224
13.3 程序设计 225
13.3.1 GPIO 的 HAL 库驱动分析 225
13.3.2 程序流程图 227
13.3.3 程序解析 227
13.4 下载验证 229
第十四章 按键输入实验 230
14.1 按键与输入数据寄存器简介 230
14.1.1 独立按键简介 230
14.1.2 GPIO 端口输入数据寄存器（IDR） 230
14.2 硬件设计 231
1. 例程功能 231
2. 硬件资源 231
3. 原理图 231
14.3 程序设计 231
14.3.1 HAL_GPIO_ReadPin 函数 231
14.3.2 程序流程图 232
14.3.3 程序解析 232
14.4 下载验证 236
第十五章 外部中断实验 237
15.1 NVIC 和 EXTI 简介 237
15.1.1 NVIC 简介 237
15.1.2 EXTI 简介 240
15.2 硬件设计 242
1. 例程功能 242
2. 硬件资源 242
3. 原理图 242
15.3 程序设计 242
15.3.1 EXTI 的 HAL 库驱动 242
15.3.2 程序流程图 244

15.3.3 程序解析 244
15.4 下载验证 247
第十六章 串口通信实验 248
16.1 串口简介 248
16.1.1 数据通信的基础概念 248
16.1.2 串口通信协议简介 250
16.1.3 STM32H7 的串口简介 251
16.1.4 GPIO 引脚复用功能 255
16.2 硬件设计 256
1. 例程功能 256
2. 硬件资源 257
3. 原理图 257
16.3 程序设计 257
16.3.1 USART 的 HAL 库驱动 257
16.3.2 程序流程图 261
16.3.3 程序解析 261
16.4 下载验证 266
第十七章 独立看门狗（IWDG）实验 267
17.1 IWDG 简介 267
17.1.1 IWDG 框图 267
17.1.2 IWDG 寄存器 268
17.2 硬件设计 269
1. 例程功能 269
2. 硬件资源 269
3. 原理图 269
17.3 程序设计 269
17.3.1 IWDG 的 HAL 库驱动 269
17.3.2 程序流程图 271
17.3.3 程序解析 271
17.4 下载验证 272
第十八章 窗口门狗（WWDG）实验 273
18.1 WWDG 简介 273
18.1.1 WWDG 框图 273

18.1.2 WWDG 寄存器 275
18.2 硬件设计 276
1. 例程功能 276
2. 硬件资源 276
3. 原理图 277
18.3 程序设计 277
18.3.1 WWDG 的 HAL 库驱动 277
18.3.2 程序流程图 278
18.3.3 程序解析 278
18.4 下载验证 280
第十九章 基本定时器实验 281
19.1 基本定时器简介 281
19.1.1 基本定时器框图 281
19.1.2 TIM6/TIM7 寄存器 283
19.1.3 基本定时器中断应用 284
19.2 硬件设计 285
1. 例程功能 285
2. 硬件资源 285
3. 原理图 285
19.3 程序设计 285
19.3.1 定时器的 HAL 库驱动 285
19.3.2 程序流程图 287
19.3.3 程序解析 287
19.4 下载验证 289
第二十章 通用定时器实验 290
20.1 通用定时器简介 290
20.2 通用定时器中断实验 298
20.2.1 TIM2/TIM3/TIM4/TIM5 寄存器 298
20.2.2 硬件设计 300
20.2.3 程序设计 301
20.2.4 下载验证 302
20.3 通用定时器 PWM 输出实验 303
20.3.1 TIM2/TIM3/TIM4/TIM5 寄存器 304

20.3.2 硬件设计 305
20.3.3 程序设计 306
20.3.4 下载验证 311
20.4 通用定时器输入捕获实验 312
20.4.1 TIM2/TIM3/TIM4/TIM5 寄存器 312
20.4.2 硬件设计 314
20.4.3 程序设计 315
20.4.4 下载验证 321
20.5 通用定时器脉冲计数实验 322
20.5.1 TIM2/TIM3/TIM4/TIM5 寄存器 322
20.5.2 硬件设计 325
20.5.3 程序设计 325
20.5.4 下载验证 330
第二十一章 高级定时器实验 332
21.1 高级定时器简介 332
21.2 高级定时器输出指定个数 PWM 实验 333
21.2.1 TIM8 寄存器 334
21.2.2 硬件设计 336
21.2.3 程序设计 337
21.2.4 下载验证 342
21.3 高级定时器输出比较模式实验 343
21.3.1 TIM1/TIM8 寄存器 343
21.3.2 硬件设计 345
21.3.3 程序设计 345
21.3.4 下载验证 350
21.4 高级定时器互补输出带死区控制实验 351
21.4.1 TIM1/TIM8 寄存器 353
21.4.2 硬件设计 356
21.4.3 程序设计 356
21.4.4 下载验证 361
21.5 高级定时器 PWM 输入模式实验 363
21.5.1 TIM1/TIM8 寄存器 364
21.5.2 硬件设计 366

21.5.3 程序设计 367
21.5.4 下载验证 374
第二十二章 电容触摸按键实验 376
22.1 电容触摸按键简介 376
22.2 硬件设计 377
1. 例程功能 377
2. 硬件资源 377
3. 原理图 377
22.3 程序设计 378
22.3.1 程序流程图 378
22.3.2 程序解析 378
22.4 下载验证 383
第二十三章 OLED 显示实验 384
23.1 OLED 简介 384
23.1.1 硬件驱动接口模式 385
23.1.2 OLED 显存 388
23.2 硬件设计 390
1. 例程功能 390
2. 硬件资源 390
3. 原理图 390
23.3 程序设计 391
23.3.1 程序流程图 391
23.3.2 程序解析 391
23.4 下载验证 400
第二十四章 内存保护（MPU）实验 401
24.1 MPU 简介 401
24.2 硬件设计 405
1. 例程功能 405
2. 硬件资源 405
3. 原理图 405
24.3 程序设计 405
24.3.1 MPU 的 HAL 库驱动 405
24.3.2 程序流程图 407

24.3.3 程序解析 408
24.4 下载验证 411
第二十五章 TFTLCD（MCU 屏）实验 413
25.1 TFTLCD 和 FMC 简介 413
25.1.1 TFTLCD 简介 413
25.1.2 液晶显示控制器 415
25.1.3 FMC 简介 418
25.1.4 FMC 寄存器 424
25.2 硬件设计 427
1. 例程功能 427
2. 硬件资源 427
3. 原理图 427
25.3 程序设计 428
25.3.1 FMC 和 SRAM 的 HAL 库驱动 428
25.3.2 程序流程图 430
25.3.3 程序解析 431
25.4 下载验证 443
第二十六章 SDRAM 实验 444
26.1 SDRAM 简介 444
26.1.1 SDRAM 简介 444
26.1.2 FMC SDRAM 接口简介 452
26.2 硬件设计 459
1. 例程功能 459
2. 硬件资源 459
3. 原理图 459
26.3 程序设计 460
26.3.1 程序流程图 460
26.3.2 程序解析 460
26.4 下载验证 465
第二十七章 LTDC LCD（RGB 屏）实验 467
27.1 RGBLCD&LTDC 简介 467
27.1.1 RGBLCD 简介 467
27.1.2 LTDC 简介 470

27.1.3 DMA2D 简介 482
27.2 硬件设计 486
1. 例程功能 486
2. 硬件资源 486
3. 原理图 487
27.3 程序设计 488
27.3.1 程序流程图 488
27.3.2 程序解析 488
27.4 下载验证 500
第二十八章 USMART 调试组件实验 501
28.1 USMART 调试组件简介 501
28.2 硬件设计 502
1. 例程功能 502
2. 硬件资源 502
28.3 程序设计 502
28.3.1 程序流程图 502
28.3.2 程序解析 503
28.4 下载验证 506
第二十九章 RTC 实时时钟实验 510
29.1 RTC 时钟简介 510
29.1.1 RTC 框图 510
29.1.2 RTC 寄存器 513
29.2 硬件设计 519
1. 例程功能 519
2. 硬件资源 519
3. 原理图 519
29.3 程序设计 519
29.3.1 RTC 的 HAL 库驱动 519
29.3.2 程序流程图 523
29.3.3 程序解析 523
29.4 下载验证 528
第三十章 硬件随机数实验 530
30.1 随机数发生器简介 530

30.1.1 RNG 框图 530
30.1.2 RNG 寄存器 531
30.2 硬件设计 532
1. 例程功能 532
2. 硬件资源 532
3. 原理图 532
30.3 程序设计 532
30.3.1 RNG 的 HAL 库驱动 532
30.3.2 程序流程图 534
30.3.3 程序解析 534
30.4 下载验证 537
第三十一章 低功耗实验 538
31.1 电源控制（PWR）简介 538
31.1.1 电源系统 538
31.1.2 电源监控 540
31.1.3 电源管理 543
31.2 PVD 电压监控实验 545
31.2.1 PWR 寄存器 546
31.2.2 硬件设计 548
31.2.3 程序设计 548
31.2.4 下载验证 551
31.3 睡眠模式实验 552
31.3.1 EXTI 寄存器 552
31.3.2 硬件设计 552
31.3.3 程序设计 553
31.3.4 下载验证 556
31.4 停止模式实验 557
31.4.1 PWR 寄存器 557
31.4.2 硬件设计 558
31.4.3 程序设计 558
31.4.4 下载验证 561
31.5 待机模式实验 562
31.5.1 PWR 寄存器 562

31.5.2 硬件设计 564
31.5.3 程序设计 564
31.5.4 下载验证 567
第三十二章 DMA 实验 568
32.1 DMA 简介 568
32.1.1 DMA 框图 569
32.1.2 DMA 寄存器 571
32.2 硬件设计 574
1. 例程功能 574
2. 硬件资源 574
3. 原理图 574
32.3 程序设计 574
32.3.1 DMA 的 HAL 库驱动 574
32.3.2 程序流程图 577
32.3.3 程序解析 577
32.4 下载验证 580
第三十三章 ADC 实验 582
33.1 ADC 简介 582
33.2 单通道 ADC 采集实验 589
33.2.1 ADC 寄存器 589
33.2.2 硬件设计 593
33.2.3 程序设计 594
33.2.4 下载验证 601
33.3 单通道 ADC 采集（DMA 读取）实验 602
33.3.1 ADC & DMA 寄存器 602
33.3.2 硬件设计 604
33.3.3 程序设计 604
33.3.4 下载验证 610
33.4 多通道 ADC 采集（DMA 读取）实验 611
33.4.1 ADC 寄存器 611
33.4.2 硬件设计 612
33.4.3 程序设计 612
33.4.4 下载验证 618

33.5 单通道 ADC 过采样（26 位分辨率）实验 620
33.5.1 ADC 寄存器 620
33.5.2 硬件设计 621
32.5.3 程序设计 621
33.5.4 下载验证 624
第三十四章 内部温度传感器实验 625
34.1 内部温度传感器简介 625
34.2 硬件设计 625
1. 例程功能 625
2. 硬件资源 625
34.3 程序设计 626
34.3.1 ADC 的 HAL 库驱动 626
34.3.2 程序流程图 626
34.3.3 程序解析 627
34.4 下载验证 630
第三十五章 DAC 实验 631
35.1 DAC 简介 631
35.2 DAC 输出实验 635
35.2.1 DAC 寄存器 635
35.2.2 硬件设计 636
35.2.3 程序设计 637
35.2.4 下载验证 643
35.3 DAC 输出三角波实验 644
35.3.1 DAC 寄存器 644
35.3.2 硬件设计 644
35.3.3 程序设计 644
35.3.4 下载验证 647
35.4 DAC 输出正弦波实验 649
35.4.1 DAC 寄存器 649
35.4.2 硬件设计 649
35.4.3 程序设计 649
35.4.4 下载验证 655
第三十六章 PWM DAC 实验 658

36.1 PWM DAC 技术的实现原理 658
36.2 硬件设计 660
1. 例程功能 660
2. 硬件资源 660
3. 原理图 660
36.3 程序设计 661
36.3.1 程序流程图 662
36.3.2 程序解析 662
36.4 下载验证 665
第三十七章 IIC 实验 667
37.1 IIC 及 24C02 简介 667
37.1.1 IIC 简介 667
37.1.2 24C02 简介 669
37.2 硬件设计 671
1. 例程功能 671
2. 硬件资源 671
3. 原理图 671
37.3 程序设计 671
使用 IIC 传输数据的配置步骤 671
37.3.1 程序流程图 672
37.3.2 程序解析 672
37.4 下载验证 682
第三十八章 IO 扩展实验 684
38.1 PCF8574 简介 684
38.2 硬件设计 686
1. 例程功能 686
2. 硬件资源 686
3. 原理图 687
38.3 程序设计 688
38.3.1 程序流程图 688
38.3.2 程序解析 688
38.4 下载验证 691
第三十九章 光环境传感器实验 693

39.1 AP3216C 简介 693
39.2 硬件设计 695
1. 例程功能 695
2. 硬件资源 695
3. 原理图 696
39.3 程序设计 696
39.3.1 程序流程图 696
39.3.2 程序解析 697
39.4 下载验证 699
第四十章 QSPI 实验 700
40.1 QSPI 及 NOR FLASH 芯片简介 700
40.1.1 QSPI 简介 700
40.1.2 NOR FLASH 芯片简介 708
40.2 硬件设计 710
1. 例程功能 710
2. 硬件资源 710
3. 原理图 710
40.3 程序设计 711
40.3.1 QSPI 的 HAL 库驱动 711
40.3.2 程序流程图 716
40.3.3 程序解析 716
40.4 下载验证 724
第四十一章 485 实验 726
41.1 485 简介 726
41.2 硬件设计 727
1. 例程功能 727
2. 硬件资源 727
3. 原理图 727
41.3 程序设计 728
41.3.1 RS485 的 HAL 库驱动 728
41.3.2 程序流程图 729
41.3.3 程序解析 729
41.4 下载验证 733
第四十二章 FDCAN 通讯实验 735

42.1 CAN/FDCAN 总线简介 735
42.1.1 CAN 简介 735
42.1.2 FDCAN 简介 736
42.1.3 CAN 协议 736
42.1.4 FDCAN 协议 742
42.1.5 消息 RAM 746
42.1.6 位时间特性 750
42.1.7 过滤器设置 751
42.1.8 FDCAN/CAN 收发工程 752
42.1.9 FDCAN 寄存器 753
42.2 硬件设计 756
1. 例程功能 756
2. 硬件资源 757
3. 原理图 757
42.3 程序设计 758
42.3.1 FDCAN 的 HAL 库驱动 758
42.3.2 程序流程图 761
42.3.3 程序解析 761
42.4 下载验证 768
第四十三章 触摸屏实验 771
43.1 触摸屏简介 771
43.1.1 电阻式触摸屏 771
43.1.2 电容式触摸屏 774
43.1.3 触摸控制原理 776
43.2 硬件设计 777
1. 例程功能 777
2. 硬件资源 777
3. 原理图 777
43.3 程序设计 778
43.3.1 HAL 库驱动 778
43.3.2 程序流程图 779
43.3.3 程序解析 779
43.4 下载验证 794
第四十四章 红外遥控实验 796

44.1 红外遥控简介 796
44.1.1 红外遥控技术介绍 796
44.1.2 红外器件特性 796
44.1.3 红外编解码协议介绍 796
44.2 硬件设计 798
1. 例程功能 798
2. 硬件资源 798
3. 原理图 798
44.3 程序设计 799
红外遥控配置步骤 799
44.3.1 程序流程图 800
44.3.2 程序解析 800
44.4 下载验证 805
第四十五章 DS18B20 数字温度传感器实验 807
45.1 DS18B20 及其时序简介 807
45.1.1 DS18B20 简介 807
45.1.2 DS18B20 时序简介 807
45.2 硬件设计 809
1. 例程功能 809
2. 硬件资源 809
3. 原理图 809
45.3 程序设计 810
DS18B20 配置步骤 810
45.3.1 程序流程图 810
45.3.2 程序解析 811
45.4 下载验证 816
第四十六章 DHT11 数字温湿度传感器 817
46.1 DHT11 及时序简介 817
46.1.1 DHT11 简介 817
46.1.2 DHT11 时序简介 817
46.2 硬件设计 819
1. 例程功能 819
2. 硬件资源 819

3. 原理图 819
46.3 程序设计 819
DHT11 配置步骤 820
46.3.1 程序流程图 820
46.3.2 程序解析 821
46.4 下载验证 825
第四十七章 气压计实验 826
47.1 SPA06-003 介绍 826
47.1.1 SPA06-003 简介 826
47.1.2 SPA06-003 工作模式简介 827
47.1.3 SPA06-003 寄存器简介 827
47.1.4 SPA06-003 工作时序简介 833
47.2 硬件设计 833
1. 例程功能 833
2. 硬件资源 834
3. 原理图 834
47.3 程序设计 835
47.3.1 程序流程图 835
47.3.2 程序解析 835
47.4 下载验证 842
第四十八章 QMI8658A 六轴传感器实验 843
48.1 QMI8658A 介绍 843
48.1.1 QMI8658A 简介 843
48.1.2 QMI8658A 寄存器 844
48.2 硬件设计 846
48.2.1 例程功能 846
48.2.2 硬件资源 846
48.2.3 原理图 846
48.3 程序设计 847
48.3.1 程序流程图 847
48.4 下载验证 853
第四十九章 无线通信实验 855
49.1 SPI&NRF24L01 无线模块介绍 855

49.1.1 NRF24L01 简介 855
49.1.2 NRF24L01 工作模式介绍 856
49.1.3 NRF24L01 寄存器 857
49.1.4 SPI 接口简介 860
49.2 硬件设计 862
1. 例程功能 862
2. 硬件资源 862
3. 原理图 862
49.3 程序设计 863
NRF24L01 配置步骤 863
49.3.1 程序流程图 864
49.3.2 程序解析 864
49.4 下载验证 875
第五十章 FLASH 模拟 EEPROM 实验 876
50.1 STM32H743 FLASH 简介 876
50.1.1 闪存的读取 877
50.1.2 闪存的编程和擦除 877
50.1.3 FLASH 寄存器 879
50.2 硬件设计 881
1. 例程功能 881
2. 硬件资源 881
50.3 程序设计 881
50.3.1 FLASH 的 HAL 库驱动 881
50.3.2 程序流程图 882
50.3.3 程序解析 882
50.4 下载验证 887
第五十一章 摄像头实验 888
51.1 OV5640 和 DCMI 简介 888
51.1.1 OV5640 简介 888
51.1.2 STM32H743 DCMI 接口简介 894
51.2 硬件设计 898
1. 例程功能 898
2. 硬件资源 898

3. 原理图 899
51.3 程序设计 899
51.3.1 DCMI 的 HAL 库驱动 899
51.3.2 程序流程图 902
51.3.3 程序解析 902
51.4 下载验证 918
第三篇 提高篇 922
第五十二章 内存管理实验 923
52.1 内存管理简介 923
52.2 硬件设计 924
1. 例程功能 924
2. 硬件资源 924
52.3 程序设计 925
52.3.1 程序流程图 925
52.3.2 程序解析 925
52.4 下载验证 934
第五十三章 SD 卡实验 937
53.1 SD 卡简介 937
53.1.1 SD 物理结构 937
53.1.2 命令和响应 939
53.1.3 卡模式 940
53.1.4 数据模式 942
53.2 SDMMC 简介 944
53.2.1 SDMMC 主要功能及框图 944
53.2.2 SDMMC 的时钟 945
53.2.3 SDMMC 的命令与响应 946
53.2.4 SDMMC 寄存器 948
53.3 SD 卡初始化流程 952
53.3.1 SDIO 模式下的 SD 卡初始化 952
53.3.2 SPI 模式下的 SD 卡初始化 955
53.4 硬件设计 957
1. 例程功能 957
2. 硬件资源 957

3. 原理图 957
53.5 程序设计 958
53.4.1 SD 卡的 HAL 库驱动 958
53.4.2 程序流程图 961
53.4.3 程序解析 961
53.6 下载验证 968
第五十四章 NAND FLASH 实验 970
54.1 NAND FLASH 简介 970
54.1.1 NAND FLASH 简介 970
54.1.2 FTL 简介 976
54.1.3 FMC NAND FLASH 接口简介 979
54.2 硬件设计 982
1. 例程功能 982
2. 硬件资源 982
3. 原理图 983
54.3 程序设计 983
54.3.1 FSNS8B004G 的 HAL 库驱动 983
54.3.1 程序流程图 985
54.3.2 程序解析 985
54.4 下载验证 1000
第五十五章 FATFS 实验 1002
55.1 FATFS 简介 1002
55.2 硬件设计 1006
1. 例程功能 1006
2. 硬件资源 1006
55.3 程序设计 1006
55.3.1 程序流程图 1007
55.3.2 程序解析 1008
55.4 下载验证 1018
第五十六章 汉字显示实验 1019
56.1 汉字显示介绍 1019
56.1.1 字符编码介绍 1019
56.1.2 字库的生成 1023

56.1.3 汉字显示原理 1025
56.2 硬件设计 1026
1. 例程功能 1026
2. 硬件资源 1026
56.3 程序设计 1027
56.3.1 程序流程图 1027
56.3.2 程序解析 1028
56.4 下载验证 1036
第五十七章 图片显示实验 1038
57.1 图片格式介绍 1038
57.1.1 BMP 编码简介 1038
57.1.2 JPEG 编码简介 1038
57.1.3 GIF 编码简介 1039
57.2 硬件设计 1039
1. 例程功能 1039
2. 硬件资源 1040
57.3 程序设计 1040
57.3.1 程序流程图 1040
57.3.2 程序解析 1041
57.4 下载验证 1048
第五十八章 硬件 JPEG 解码实验 1050
58.1 硬件 JPEG 编解码器简介 1050
58.1.1 JPEG 编解码器框图 1050
58.1.2 JPEG 解码器功能 1051
58.1.3 MDMA 简介 1053
58.1.4 YUV 转 RGB 操作 1054
58.1.5 JPEG 编解码器寄存器 1055
58.2 硬件设计 1057
1. 例程功能 1057
2. 硬件资源 1057
58.3 程序设计 1058
58.3.1 硬件 JPEG 解码 JPG/JPEG 的简要步骤 1058
58.3.2 程序流程图 1058

58.3.3 程序解析 1060
58.4 下载验证 1079
第五十九章 照相机实验 1081
59.1 BMP&JPEG 编码简介 1081
59.1.1 BMP 编码简介 1081
59.1.2 JPEG 编码简介 1083
59.2 硬件设计 1084
1. 例程功能 1084
2. 硬件资源 1084
59.3 程序设计 1086
59.3.1 程序流程图 1086
59.3.2 程序解析 1086
59.4 下载验证 1097
第六十章 音乐播放器实验 1099
60.1 WAV&ES8388&SAI 简介 1099
60.1.1 WAV 简介 1099
60.1.2 ES8388 简介 1101
60.1.3 SAI 简介 1105
60.2 硬件设计 1111
1. 例程功能 1111
2. 硬件资源 1111
3. 原理图 1111
60.3 程序设计 1112
60.3.1 程序流程图 1112
60.3.2 程序解析 1113
60.4 下载验证 1131
第六十一章 录音机实验 1133
61.1 SAI 录音简介 1133
61.2 硬件设计 1134
1. 例程功能 1134
2. 硬件资源 1134
3. 原理图 1135
61.3 程序设计 1135

61.3.1 程序流程图 1135
61.3.2 程序解析 1135
61.4 下载验证 1141
第六十二章 SPDIF（光纤音频）实验 1143
62.1 SPDIF 简介 1143
62.1.2.1 SPDIF 寄存器 1148
62.2 硬件设计 1150
1. 例程功能 1150
2. 硬件资源 1150
3. 原理图 1150
62.3 程序设计 1151
具体配置步骤 1151
62.3.1 程序流程图 1152
62.3.2 程序解析 1153
62.4 下载验证 1160
第六十三章 视频播放器实验 1162
63.1 AVI 简介 1162
63.2 硬件设计 1166
1. 例程功能 1166
2. 硬件资源 1167
63.3 程序设计 1167
63.3.1 程序流程图 1167
63.3.2 程序解析 1167
63.4 下载验证 1180
第六十四章 FPU 测试(Julia 分形)实验 1184
64.1 FPU&Julia 分形简介 1184
64.1.1 FPU 简介 1184
64.1.2 Julia 分形简介 1185
64.2 硬件设计 1186
1. 例程功能 1186
2. 硬件资源 1186
64.3 程序设计 1187
64.3.1 程序流程图 1187

64.3.2 程序解析 1187
64.4 下载验证 1191
第六十五章 DSP 测试实验 1193
65.1 DSP 简介与环境搭建 1193
65.1.1 STM32H743 DSP 简介 1193
65.1.2 DSP 库运行环境搭建 1196
65.2 硬件设计 1198
1. 例程功能 1198
2. 硬件资源 1198
65.3 程序设计 1198
65.3.1 DSP BasicMath 测试 1199
65.3.2 DSP FFT 测试 1201
65.4 下载验证 1204
第六十六章 手写识别实验 1206
66.1 手写识别简介 1206
66.2 硬件设计 1207
1. 例程功能 1207
2. 硬件资源 1207
66.3 程序设计 1208
66.3.1 程序流程图 1208
66.3.2 程序解析 1208
66.4 下载验证 1213
第六十七章 T9 拼音输入法实验 1215
67.1 拼音输入法简介 1215
67.2 硬件设计 1215
1. 例程功能 1215
2. 硬件资源 1216
67.3 程序设计 1216
67.3.1 程序流程图 1216
67.3.2 程序解析 1217
67.4 下载验证 1221
第六十八章 串口 IAP 实验 1224
68.1 IAP 简介 1224

68.2 硬件设计 1229
1. 例程功能 1229
2. 硬件资源 1229
68.3 程序设计 1230
APP 程序的生成步骤 1230
68.3.1 程序流程图 1231
68.3.2 程序解析 1231
68.4 下载验证 1236
第六十九章 USB 读卡器(Slave)实验 1238
69.1 USB 简介 1238
69.1.1 USB 简介 1238
69.1.2 STM32H7 的 USB 特性 1240
69.2 硬件设计 1243
1. 例程功能 1243
2. 硬件资源 1243
3. 原理图 1244
69.3 程序设计 1245
69.3.1 程序流程图 1245
69.3.2 程序解析 1246
69.4 下载验证 1255
第七十章 USB 声卡(Slave)实验 1257
70.1 USB 声卡简介 1257
70.2 硬件设计 1257
1. 例程功能 1257
2. 硬件资源 1257
70.3 程序设计 1258
70.3.1 程序流程图 1258
70.3.2 程序解析 1258
70.4 下载验证 1264
第七十一章 USB 虚拟串口(Slave)实验 1266
71.1 USB 虚拟串口简介 1266
71.2 硬件设计 1266
1. 例程功能 1266

2. 硬件资源 1266
71.3 程序设计 1267
71.3.1 程序流程图 1267
71.3.2 程序解析 1267
71.4 下载验证 1274
第七十二章 USB U 盘(Host)实验 1276
72.1 U 盘简介 1276
72.2 硬件设计 1276
1. 例程功能 1276
2. 硬件资源 1276
3. 原理图 1276
72.3 程序设计 1278
72.3.1 程序流程图 1278
72.3.2 程序解析 1278
72.4 下载验证 1286
第七十三章 USB 鼠标键盘(Host)实验 1289
73.1 USB 鼠标键盘简介 1289
73.2 硬件设计 1289
1. 例程功能 1289
2. 硬件资源 1289
73.3 程序设计 1289
73.3.1 程序流程图 1289
73.3.2 程序解析 1290
73.4 下载验证 1298
第七十四章 网络通讯实验 1299
第七十五章 UCOSII 实验 1-任务调度 1300
75.1 嵌入式实时操作系统介绍 1300
75.1.1 裸机系统和多任务系统的区别 1300
75.1.2 UCOSII 介绍 1301
75.1.3 任务定义 1302
75.1.4 任务调度 1303
75.2 硬件设计 1303
1. 例程功能 1303

2. 硬件资源 1303
75.3 程序设计 1303
75.3.1 UCOSII 驱动函数 1303
75.3.2 程序流程图 1306
75.3.3 程序解析 1306
75.4 下载验证 1312
第七十六章 UCOSII 实验 2-信号量和邮箱 1313
76.1 UCOSII 信号量和邮箱简介 1313
76.2 硬件设计 1314
1. 例程功能 1314
2. 硬件资源 1315
76.3 程序设计 1315
76.3.1 UCOSII 相关函数介绍 1315
76.3.2 程序流程图 1319
76.3.3 程序解析 1320
76.4 下载验证 1327
第七十七章 UCOSII 实验 3-消息队列、信号量集和软件定时器 1329
77.1 UCOSII 消息队列、信号量集和软件定时器简介 1329
77.1.1 消息队列 1329
77.1.2 信号量集 1331
77.1.3 软件定时器 1333
77.2 硬件设计 1335
1. 例程功能 1335
2. 硬件资源 1335
77.3 程序设计 1336
77.3.1 UCOSII 相关函数介绍 1336
77.3.2 程序流程图 1341
77.3.3 程序解析 1342
77.4 下载验证 1353
第七十八章 综合测试实验 1355
78.1 综合测试实验简介 1355
78.2 综合测试实验详解 1356
78.2.1 电子图书 1360

78.2.2 数码相框 1362
78.2.3 音乐播放 1363
78.2.4 视频播放 1367
78.2.5 时钟 1368
78.2.6 系统设置 1369
78.2.7 FC 游戏机 1380
78.2.8 记事本 1384
78.2.9 运行器 1386
78.2.10 手写画笔 1387
78.2.11 照相机 1390
78.2.12 录音机 1394
78.2.13 USB 连接 1397
78.2.14 网络通信 1398
78.2.15 无线传书 1403
78.2.16 计算器 1405
78.2.17 二维码 1408
78.2.18 IP 摄像头 1411
78.2.19 人脸识别 1414
78.2.20 9D 测试 1418
78.2.21 水平仪 1419
78.2.22 蜂鸣器 1421
78.2.23 按键测试 1422
78.2.24 LED 测试 1423
78.2.25 拨号 1424
78.2.26 应用中心 1427
78.2.27 电压表 1429

内容简介

本书将由浅入深，带领大家学习 STM32H743 的各个功能，为您开启 STM32H743 的学习之旅。

本书总共分为三篇：

1， 基础篇，主要介绍 STM32H743 的基础入门知识，包括 STM32H743 介绍、软件安装、新建工程、时钟系统介绍、SYSTEM 文件夹介绍等。必须好好学习并掌握；

2， 入门篇，主要介绍 STM32H743 常用外设的使用，包括 GPIO、定时器、AD/DA、DMA、触摸液晶屏等。必须好好学习并掌握；

3， 提高篇，主要介绍 STM32H743 较难一点外设的使用及一些高级例程，包括：SDMMC、

USB、内存管理、文件系统、图片解码、OS 入门等。这些例程难度相对高一点，大家一定要沉下心来，戒骄戒躁，循序渐进的学习。对于一些较难的知识点，我们只需要知道怎么用就可以了，并不需要详细研究具体实现过程，比如文件系统、图片解码、USB 协议栈、OS 实现原理等；

本书为正点原子阿波罗 STM32H743 开发板的配套教程，在开发板配套的光盘里面，有详细原理图以及所有实例的完整代码，这些代码都有详细的注释，所有源码都经过我们严格测试，不会有任何警告和错误，另外，源码有我们生成好的 hex 文件，大家只需要通过仿真器下载到开发板即可看到实验现象，亲自体验实验过程。

本书并没有详细介绍阿波罗 STM32H743 的硬件，硬件部分内容我们单独编写了一个文档，请参考：《阿波罗 V2 硬件参考手册_V1.0》进行学习。

本书不仅非常适合广大学生和电子爱好者学习 STM32H743，其大量的实验以及详细的解说，也是公司产品开发的不二参考。

第一篇 基础篇

万事开头难，如果打好了基础，后面学习就事半功倍了！本篇将详细介绍 STM32H743 学习的基础知识，包括：环境搭建、STM32 入门知识、新建工程、HAL 库介绍、启动过程分析、时钟系统、SYSTEM 文件夹介绍等部分。学好了这些基础知识，在后面的例程学习部分，将会有非常大的帮助，能极大的提高大家的学习效率。

如果您是初学者，建议好好学习本并理解这些知识，手脑并用，不要漏过任何内容，一遍学不会的可以多学几遍，总之这些知识点都要掌握。

如果您已经学过 STM32 了，本篇内容则可以挑选着学习。

本篇将分为如下章节：

1，本书学习方法
2，STM32 简介
3，开发环境搭建
4，STM32 初体验
5，STM32 基础知识入门
6，新建寄存器版本 MDK 工程
7，认识 HAL 库
8，新建 HAL 库版本 MDK 工程
9，STM32 启动过程分析
10，STM32CubeMX 简介
11，STM32 时钟系统
12，SYSTEM 文件夹介绍

第一章 本书学习方法

为了让大家更好的学习和使用本书，本章将给大家介绍一下本书的学习方法，包括：本书的学习顺序、编写规范、代码规范、资料查找、学习建议等内容。

本章将分为如下几个小节：

1.1 本书学习顺序
1.2 本书参考资料
1.3 本书编写规范
1.4 本书代码规范
1.5 例程资源说明
1.6 学习资料查找
1.7 给初学者的建议

1.1 本书学习顺序

为了让大家更好的学习和使用本书，我们做了以下几点考虑：

1，坚持循序渐进的思路编写，从基础到入门，从简单到复杂。

2，将知识进行分类介绍，简化学习过程，包括：基础篇、入门篇、提高篇。

3，将硬件介绍独立成一个文档（《阿波罗 V2 硬件参考手册_V1.0》），本书着重介绍软件知识。

因此，大家在学习本书的时候，我们建议：先通读一遍《阿波罗 V2 硬件参考手册_V1.0》，对开发板的硬件资源有个大概了解，然后从基础篇开始，再到入门篇，最后学习提高篇，循序渐进，逐一攻克。

对初学者来说，尤其要按照以上顺序学习，不要跳跃式学习，因为我们书本的知识都是一环扣一环的，如果前面的知识没学好，后面的知识学起来就会很困难。

对于已经学过 STM32 的朋友来说，就可以跳跃式学习了，当然如有不懂，也得翻阅前面的知识点进行巩固。

1.2 本书参考资料

本书的主要参考资料有以下三份文档：

《STM32H7xx 参考手册_V7（英文版）.pdf》
《STM32H7xx 参考手册_V3（中文版）.pdf》
《STM32H7 编程手册.pdf》

《STM32H7xx 参考手册_V7（英文版）.pdf》是英文版的，该文档详细介绍了 STM32H7 的各种寄存器定义以及外设的使用说明等，是学习 STM32H743 的必备资料。

《STM32H7xx 参考手册_V3（中文版）.pdf》则是针对 H7x3 等系列芯片的中文版本参考手册，对那些英语不是很好的朋友，是一个很好的辅助文档资料（可以对照两本手册学习）。

而《STM32H7 编程手册.pdf》则是对《STM32H7xx 参考手册_V7（英文版）.pdf》的补充，很多关于 Cortex M7 内核的介绍（寄存器等），都可以在这个文档找到答案，该文档同样是 ST 的官方资料，专门针对 ST 的 Cortex M7 产品。结合这三本参考资料，就可以比较好的学习 STM32H7 了。

这几份参考文档都在我们提供的光盘资料里面可以找到，路径：A 盘→8，STM32 参考资料 文件夹下可以找到。

1.3 本书编写规范

本书通过数十个例程，给大家详细介绍 STM32 的所有功能和外设，按难易程度以及知识结构，我们将本书分为三个篇章：基础篇、入门篇和提高篇。

基础篇，共 12 章，主要是一些基础知识介绍，包括开发环境搭建、新建工程、HAL 库介绍、时钟树介绍、SYSTEM 文件夹介绍等，这些章节在结构上没有共性，但是互相有关联，有一个集成的关系在里面，即：必须先学了前面的知识，才好学习后面的知识点。

入门篇和提高篇，共 65 章，详细介绍了 STM32H743 每一个外设的使用方法及驱动代码，并且还介绍了一些非常实用的程序代码（纯软件例程），如：内存管理、文件系统读写、拼音输入法、手写识别、图片解码、IAP 等。这部分内容占了本书的绝大部分篇幅，而且这些章节在结构上都比较有共性，一般分为 4 个部分，如下：

1，外设功能介绍
2，硬件设计
3，程序设计
4，下载验证

外设功能介绍，简单介绍具体章节所要用到的外设功能、框图和寄存器等，让大家对所用外设的功能有一个基本了解，方便后面的程序设计。

硬件设计，包括具体章节的实验具体功能说明、所用到的硬件资源及原理图连接方式，从而知道要做什么？ 需要用到哪些 IO 口？ 是怎么接线的？ 方便程序设计的时候编写驱动代码。

程序设计，一般包括：驱动介绍、程序流程图、关键代码分析、main 函数讲解等三部分。一点点介绍程序代码是怎么来的，注意事项等，从而学会整个代码。

下载验证，属于实践环节，在完成程序设计后，教大家如何下载并验证我们的例程是否正确？ 完成一个闭环过程。

1.4 本书代码规范

为了方便大家编写高质量代码，我们对本书的代码风格进行了统一，详细的代码规范说明文档，见光盘：A 盘→1，入门资料→《嵌入式单片机 C 代码规范与风格.pdf》，初学者务必好好学习一下这个文档。

总结几个规范的关键点：

1，所有函数/变量名字非特殊情况，一般使用小写字母；
2，注释风格使用 doxgen 风格，除屏蔽外，一律使用 /* */ 方式进行注释；
3，TAB 键统一使用 4 个空格对齐，不使用默认的方式进行对齐；
4，每两个函数之间，一般有且只有一个空行；
5，相对独立的程序块之间，使用一个空行隔开；
6，全局变量命名一般用 g_开头，全局指针命名一般用 p_开头；
7，if、for、while、do、case、switch、default 等语句单独占一行，一般无论有多少行执行语句，都要用加括号：{}。

1.5 例程资源说明

阿波罗 STM32H743 开发板提供的标准例程多达 82 个，提供：寄存器和 HAL 库两个版本的代码（本手册以 HAL 库版本例程做为介绍，**我们不再提供寄存器版本文档教程，寄存器版本例程仅供大家参考学习**）。我们提供的这些例程，基本都是原创，拥有非常详细的注释，代码风格统一、循序渐进，非常适合初学者入门。

阿波罗 STM32H743 开发板的例程列表如表 1.5.1 所示：

编号	实验名字	编号	实验名字
1	跑马灯实验	28	QSPI 实验
2	按键输入实验	29	485 实验
3	外部中断实验	30	FDCAN 实验
4	串口通信实验	31	触摸屏实验
5	独立看门狗实验	32	红外遥控实验
6	窗口看门狗实验	33	DS18B20 数字温度传感器实验

7-1	基本定时中断器实验	34	DHT11 数字温湿度传感器实验
8-1	通用定时器中断实验	35	磁力计实验
8-2	通用定时器 PWM 输出实验	36-1	六轴传感器-读取原始数据实验
8-3	通用定时器输入捕获实验	36-2	六轴传感器-姿态解算实验
8-4	通用定时器脉冲计数实验	37	无线通信实验
9-1	高级定时器输出指定个数 PWM 实验	38	FLASH 模拟 EEPROM 实验
9-2	高级定时器输出比较模式实验	39	摄像头实验
9-3	高级定时器互补输出带死区控制实验	40	内存管理实验
9-4	高级定时器 PWM 输入模式实验	41	SD 卡实验
10	电容触摸按键实验	42	NAND FLASH 实验
11	OLED 显示实验	43	FATFS 实验
12	内存保护（MPU）实验	44	汉字显示实验
13	TFTLCD（MCU 屏）实验	45	图片显示实验
14	SDRAM 实验	46	硬件 JPEG 解码实验
15	LTDC LCD（RGB 屏）实验	47	照相机实验
16	USMART 调试实验	48	音乐播放器实验
17	RTC 实时时钟实验	49	录音机实验
18	随机数发生器实验	50	SPDIF（光纤音频）实验
19-1	PVD 电压监控实验	51	视频播放器实验
19-2	睡眠模式实验	52-1	FPU 测试（Julia 分形）实验_开启
19-3	停止模式实验	52-2	FPU 测试（Julia 分形）实验_关闭
19-4	待机模式实验	53	DSP 测试实验
20	DMA 实验	54	手写识别实验
21-1	单通道 ADC 采集实验	55	T9 拼音输入法实验
21-2	单通道 ADC 过采样（DMA 读取）	56	串口 IAP 实验
21-3	多通道 ADC 采集（DMA 读取）实验	57	USB 读卡器（Slave）实验
21-4	单通道 ADC 过采样（26 位分辨率）	58	USB 声卡（Slave）实验
22	内部温度传感器实验	59	USB 虚拟串口（Slave）实验
23-1	DAC 输出实验	60	USB U 盘（Host）实验
23-2	DAC 输出三角波实验	61	USB 鼠标键盘（Host）实验
23-3	DAC 输出正弦波实验	62	网络通信实验
24	PWM DAC 实验	63	UCOSII 实验 1-任务调度
25	IIC 实验	64	UCOSII 实验 2-信号量和邮箱
26	IO 扩展实验	65	UCOSII 实验 3-消息对列、信号量和软件定时器
27	光环境传感器实验		

表 1.5.1 阿波罗 STM32H743 开发板例程表

从上表可以看出，正点原子阿波罗 STM32H743 开发板的例程基本上涵盖了 STM32H743IIT6 芯片的所有内部资源，并且外扩展了很多有价值的例程，比如：FLASH 模拟 EEPROM 实验、USMART 调试实验、UCOSII 实验、内存管理实验、IAP 实验、拼音输入法实验、手写识别实验、综合实验等。

而且从上表可以看出，例程安排是循序渐进的，首先从最基础的跑马灯开始，然后一步步深入，从简单到复杂，有利于大家的学习和掌握。所以，正点原子阿波罗 STM32H743 开发板是非常适合初学者的。当然，对于想深入了解 STM32H743 内部资源的朋友，正点原子阿波罗

STM32H743 开发板也绝对是一个不错的选择。

1.6 学习资料查找

学习资料包括三个方面：

1，ST 官方的学习资料

ST 官方资料有两个网址：www.stmcu.org.cn 和 www.st.com。

www.stmcu.org.cn 是 ST 中文社区，里面的资料全部由 ST 中国区的人负责更新和整理，包含了所有 ST 公司的 MCU 资料，比如：STM32H7 最新的芯片文档（参考手册、数据手册、勘误手册、编程手册等）、软件资源（固件库、配置工具、PC 软件等）、硬件资源（各种官方评估板）等，如图 1.6.1 所示：

图 1.6.1 STM32H7 相关资料（stmcu.org）

www.st.com 是ST官网，ST最新最全的资料，一般都是放在该网站，ST中文社区的资料，一般都是从 ST 官网搬过来的，所以如果你想找最新的 STM32 官方资料，应该在 ST 官网找。对于初学者，一般从 ST 中文社区获取 ST 官方资料就可以了。ST 官网的 STM32H743 资料页面如图 1.6.2 所示（注意：默认是英文语言，需要在网页右上角设置成中文）：

图 1.6.2 STM32H743 官网资料（st.com）

图 1.6.3 STM32H743 官网资料（st.com）

1，STM32H743 的硬件相关资源，在：产品→微控制器与微处理器→STM32 ARM Cortex 32 位微控制器→STM32 高性能单片机→STM32H7 系列→STM32H743/753→工具和软件 路径下面可以找到。

2，STM32H743 的软件相关资源，在：工具与软件→嵌入式软件→微控制器软件→STM32 微控制器软件 路径下面可以找到。

2，正点原子的学习资料

正点原子给大家提供的学习资料，都放在正点原子文档中心，大家可以在文档中心下载所有正点原子最新最全的学习资料，文档中心地址：www.openedv.com/docs/index.html ，如图 1.6.4 所示：

图 1.6.4 正点原子文档中心

在文档中心下面，我们可以找到正点原子所有开发板、模块、产品等的详细资料下载地址。

3，正点原子论坛

正点原子论坛，即开源电子网：http://www.openedv.com/forum.php，该论坛从 2010 年成立至今，已有 10 多年时间，数十万注册用户，STM32 相关帖子数量有 20 多万，每天数百人互动，是一个非常好的 STM32 学习交流平台。

在学习过程中，我们难免会遇到一些问题，有任何问题，大家都可以先去开源电子网搜索一下，基本上你能遇到的问题，我们论坛都有人问过了，所以可以很方便的找到一些参考解决方法。如果实在找不到，你也可以在论坛提问，每天原子哥都会在上面给大家做解答。

不过，大家在论坛发帖的时候，建议先阅读一下提问的智慧，缕清思路，不要乱问一通，提高提问质量，提问的智慧阅读地址：www.openedv.com/thread-7245-1-1.html。

1.7 给初学者的建议

对于学习 STM32，这里我给大家提以下三点建议：

1，准备一款合适的开发板（强烈建议配仿真器）

任何实验，我们都需要验证，最好的验证方式就是在开发板上面实际跑起来，然后通过仿真器仿真调试，查看具体的执行过程。仿真调试可以加深印象，还可以方便的查找 bug，所以学习 STM32 必备：一个开发板 + 一个仿真器。

另外，开发板在精不在多，学好一款，基本上就够用了。

2，两本参考资料，即《STM32H7xx 参考手册_V3（中文版）.pdf》和《STM32H7 编程手册.pdf》。

这两个手册在本章 1.2 节有过介绍，对于我们学习 STM32 和了解 Cortex M7 内核非常有帮助，是学习 STM32 的必备资料，因此初学者尤其要多看这两个手册。

这里建议大家，要多了解一些底层的东西（可结合这两个手册，看看寄存器版本的例程），不要只会使用库，否则，一旦遇到问题，或者换个芯片，就不知道怎么办了。

3，戒骄戒躁，勤思敏行。

学习 STM32 千万不能浮躁，更加不能骄傲，初学者学习 STM32 会遇到很多问题和难点，这个时候千万不能浮躁，不要带情绪，一定要静下心来，缕清思路，逐一攻克。

我就曾经遇到一个问题半个月都没解决的情况，但是这半个月我尝试并掌握了很多解决问题的方法，最终解决问题的时候，其实带来的收获远远大于问题本身。所以不要遇到问题就认怂，就想问别人，问老师，先尝试自己解决一下，比如花个十天半个月去解决一个问题，我相信你也会有很多收获。

学习我们教程的时候，要多思考，多想想为什么要这么写？有没有其他更好的办法？然后，自己去验证，去实践。这里非常重要的一点是要多实践，一定要自己动手写代码，然后再下载到开发板验证，不要只是看看视频，看看例程就算完了，要能做到举一反三，如果自己不实践，不动手写代码，是很难真正学会的。

最后，C 语言是学习 STM32 的必备知识，所以如果 C 语言不过关的朋友，得先好好学习一下 C 语言基础，否则学起来会比较吃力。

第二章 STM32 简介

本章，我们将向大家介绍 STM32 是一个什么东西？有哪些资源？能够做什么？如何选型？等基础知识，让大家对 STM32 有一个大概了解。

本章将分为如下几个小节：

2.1 初识 STM32

2.2 STM32H743 资源简介

2.3 STM32H743 设计选型

2.1 初识 STM32

2007 年 6 月，ST 在北京发布了全球第一款基于 ARM Cortex M3 内核的 32 位通用微控制器芯片：STM32F103，以优异的性能，丰富的资源，超高的性价比，迅速占领市场，从此一鸣惊人，一发不可收拾，截止到 2020 年 6 月，STM32 累计出货量超过 45 亿颗。

战舰开发板使用的 STM32F103ZET6 芯片如图 2.1.1 所示：

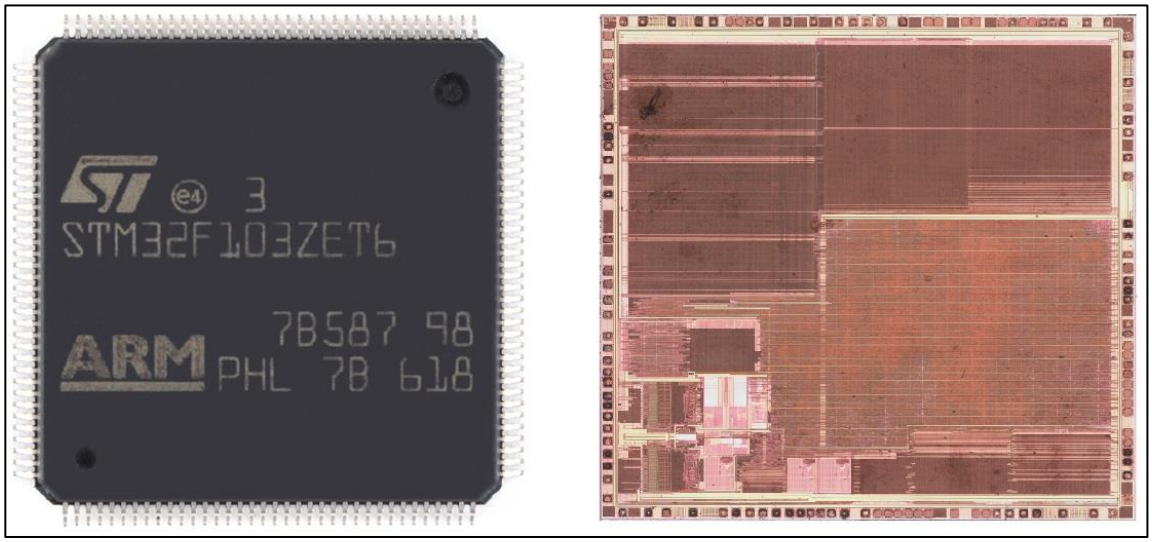


图 2.1.1 STM32F103ZET6 芯片（LQFP144 脚）

上图中，左侧是 STM32F103ZET6 芯片，右侧则是芯片开盖后的图片，即芯片内部视图，可以看到外观看上去平平无奇，但是内部是有很多东西的，需要我们花很多时间和精力去学习掌握。

STM32 的优异性体现在如下几个方面：

1，超低的价格。8 位机的价格，32 位机的性能，是 STM32 最大的优势。

2，超多的外设。STM32 拥有包括：FMC、TIMER、SPI、IIC、USB、CAN、IIS、SDIO、ADC、DAC、RTC、DMA 等众多外设及功能，具有极高的集成度。

3，丰富的型号。STM32 仅 M3 内核就拥有 F100、F101、F102、F103、F105、F107、F207、F217 等 8 个系列上百种型号，具有 QFN、LQFP、BGA 等封装可供选择。同时 STM32 还推出了 STM32L 和 STM32W 等超低功耗和无线应用型的 M3 芯片，另外，ST 还推出了 STM32F4/F7/H7 等更高性能的芯片。

4，优异的实时性能。150 个中断，16 级可编程优先级，并且所有引脚都可以作中断输入。

5，杰出的功耗控制。STM32 各个外设都有自己的独立时钟开关，可以通过关闭相应外设的时钟来降低功耗。

6，极低的开发成本。通过串口即可下载程序，而且相应的仿真器也很便宜，支持 JTAG&SWD 调试接口，最少仅 2 个 IO 口即可实现仿真调试，极大的降低了开发成本。

再来看一个 STM32 与 51 的性能对比如表 2.1.1 所示：

	内核	主频(Mhz)	DMIPS	硬件 FPU
STM32H7	Cortex M7	480	1027	双精度
STM32F7	Cortex M7	216	462	双精度
STM32F4	Cortex M4	168	210	单精度
STM32F1	Cortex M3	72	90	无
STC15	8051	35	35	无

表 2.1.1 STM32 VS 51

这里我们选的 51 是性能比较好的 STC15 系列做为对比，如果换成传统 51，速度会比 STC15 慢 12 倍左右。最强 H7 的 DMIPS 性能约为 STC15 的 30 倍，即便是 STM32F103 也大概有 STC15 性能的 3 倍，由此可见 STM32 的强大，而且最便宜的 STM32F103，价格大概在 5 块多人民币，和 STC15 系列的价格差不多。

简单来说是，价格差不多的情况下，51 能做的，STM32 都能做，51 不能做的，STM32 也能做，因此，越来越多的企业选择使用 STM32 替代 51，所以，大家如果能学会 STM32，找工作的时候，也会有一定的优势。

2.2 STM32H743 资源简介

下面来看看 STM32H743IIT6 具体的内部资源，如表 2.2.1 所示：

STM32H743IIT6 资源					
内核	Cortex M7	基本定时器	2	USART	9
主频	400Mhz	通用定时器	10	FDCAN	2
FLASH	2048KB	高级定时器	2	SDMMC	2
SRAM	1060KB	16 位 ADC	3	FMC	1
封装	LQFP176	ADC 通道数	20	DMA	4
IO 数量	140	12 位 DAC	2	RTC	1
工作电压	3.3V	SPI	6	IIC	4
USB OTG	2	QUADSPI	1	SAI	4
ETH MAC	100M	JPEG Codec	支持	DCMI	支持

表 2.2.1 STM32H743IIT6 内部资源表

由表可知，STM32 内部资源还是非常丰富的，本书将针对这些资源进行详细的使用介绍，并提供丰富的例程，供大家参考学习，相信经过本书的学习，您会对 STM32H743 有一个全面的了解和掌握。

关于 STM32H743 内部资源的详细介绍，请大家参考光盘→A 盘→7，硬件资料→2，芯片资料→ STM32H743IIT6.pdf，该文档即 STM32H743 的数据手册，里面有 STM32H743 详细的资源说明和相关性能参数。

2.3 STM32H743 设计选型

STM32 从 2007 年推出至今，已经有 18 个系列，超过 1000 个型号，为了方便大家选择合适的型号设计产品，本节给大家讲讲 STM32 的设计选型。

2.3.1 STM32 系列

STM32 目前总共有 5 大类，18 个系列，如表 2.3.1.1 所示：

大类	系列	内核	特性
主流级 MCU	G0	Cortex M0+	全新入门级 MCU
	G4	Cortex M4	模数混合型 MCU
	F0	Cortex M0	入门级 MCU
	F1	**Cortex M3**	**基础型 MCU**
	F3	Cortex M4	混合信号 MCU
高性能	F2	Cortex M4	高性能 MCU

MCU	**F4**	**Cortex M4**	**高性能 MCU**
	F7	**Cortex M7**	**高性能 MCU**
	H7	**Cortex M7**	**超高性能 MCU**，部分型号有双核（M7+M4）
超低功耗 MCU	L0	Cortex M0+	超低功耗 MCU
	L1	Cortex M3	超低功耗 MCU
	L4	**Cortex M4**	**超低功耗 MCU**
	L4+	Cortex M4	超低功耗高性能 MCU
	L5	Cortex M33	超低功耗高性能安全 MCU
	U5	Cortex M33	超低功耗高性能大容量安全 MCU
无线 MCU	WB	Cortex M4 Cortex M0+	双核无线 MCU
	WL	Cortex M4	远程无线 MCU
微处理器 MPU	**MP1**	**Cortex A7** **Cortex M4**	**双核超高性能 MPU**

表 2.3.1.1 STM32 系列分类及说明

可以看到，STM32 主要分两大块，MCU 和 MPU，MCU 就是我们常见的 STM32 微控制器，不能跑 Linux，而 MPU 则是 ST 在 19 年才推出的微处理器，可以跑 Linux。本书重点介绍 MCU 产品，正点原子阿波罗 STM32H743 开发板使用的 STM32H743IIT6 属于高性能 MCU 分类里面的超高性能型 H7 系列。

STM32 MCU 提供了包括：基础入门、混合信号、高性能、超低功耗和无线等 5 方面应用的产品型号，我们可以根据自己的实际需要选择合适的 STM32 来设计。比如，我们的产品对性能要求比较高，则可以选择 ST 的高性能 MCU，包括：F2、F4、F7、H7 等 4 个系列的产品；又比如想做超低功耗，则可以选择 ST 的超低功耗 MCU，L 系列的产品。

表 2.3.1.1 中加粗的系列，正点原子都有相应的开发板，包括：主流级 F1、高性能 F4/F7/H7、超低功耗 L4 和微处理器 MP1，大家可以根据自己的需要选择合适的正点原子开发板进行学习。

由于 STM32 系列有很好的兼容性，我们只要能够熟练掌握其中一任何一款 MCU，就可以很方便的学会并使用其他系列的 MCU。比如学好了 STM32F103，再去学 F4/F7/H7 就比较容易学会，由于 STM32F103 系列最早推向市场，资料和教程都是最多的，在市场上的使用也是最为广泛，所以对于没有接触过 STM32 的初学者来说，我们强烈建议先学习 STM32F103，再去学习其他的 STM32 系列。

2.3.2 STM32 命名

STM32 的命名规则如图 2.3.2.1 所示：

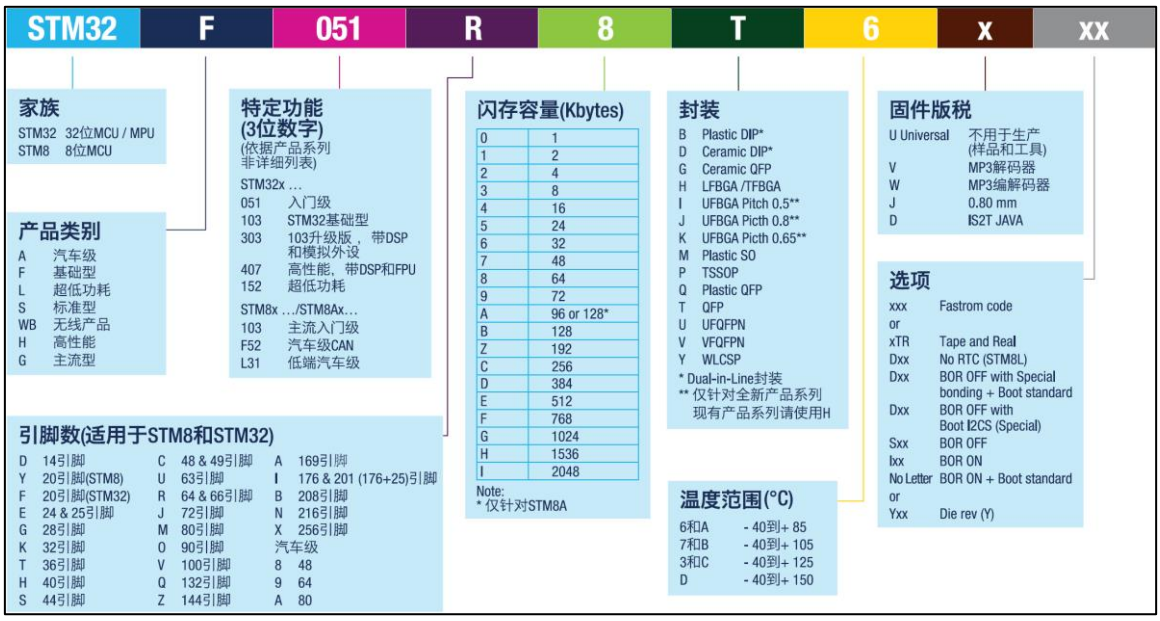


图 2.3.2.1 STM32 MCU 命名规则（摘自 STM32 产品选型手册）

如图所示，STM32 的产品名字里面包含了：家族、类别、特定功能、引脚数、闪存容量、封装、温度范围等重要信息，这些信息可以帮助我们识别和区分 STM32 不同芯片。

下面来看看 STM32H743IIT6 这个型号包含的信息，如表 2.3.2.1 所示：

描述	型号	说明
家族	STM32	ST 公司 32 位 MCU
产品类别	H	高性能
特定功能	743	高性能，带 DSP、FPU、JPEG
引脚数	I	176 脚
闪存容量	I	2048KB FLASH
封装	T	QFP 封装
温度范围	6	-40～+85℃ 工作温度范围（工业级）

表 2.3.2.1 STM32H743IIT6 型号说明

任何 STM32 型号，都可以按图 2.3.2.1 所示命名规则进行区分解读。

2.3.3 STM32 选型

了解了 STM32 的系列和命名以后，我们再进行 STM32 选型就会比较容易了，这里我们只要遵循：**由高到低，由大到小** 的原则，就可以很方便的完成设计选型了。

由高到低原则：在不能评估项目所需性能的时候，可以考虑先选择高性能的 STM32 型号进行开发，比如选择 F4/F7/H7 等，在高性能 STM32 上面完成关键性能（即最需要性能的代码）开发验证，如果满足使用要求，则可以降档，如从 H7→F7→F4→F1，如不满足要求，则可以升档，如从 F4→F7→H7，通过此方法找到最佳性价比的 STM32 系列。

由大到小原则：在不能评估项目所需 FLASH 大小、SRAM 大小、GPIO 数量、定时器等资源需求的时候，可以考虑先选择容量较大的型号进行开发，比如选择 512K/甚至 1M FLASH 的型号进行开发，等到开发完成大部分功能之后，就对项目所需资源有了定论，从而可以根据实际情况进行降档选择（当然极少数情况可能还需要升档），通过此方法，找到最合适的 STM32 型号。

通过这两个原则，我们就可以找到最适合项目使用的 STM32 具体型号了。

整个选型工作大家可以在正点原子开发板上进行验证，一般我们开发板都是选择容量比较大/资源比较多的型号进行设计的，这样可以免去大家自己设计焊接验证板的麻烦，加快项目开发进度。一些资深工程师，对项目要求认识比较深入的话，甚至都不需要验证了，直接就可以选出最合适的型号，这个效率更高。当然这个需要长期积累和多实践，相信只要大家多学习，

多实践，总有一天也能达到这个级别。

2.3.4 STM32 设计

这里我们简单给大家介绍一下 STM32 的原理图设计，上一节我们通过选型原则可以确定项目所需的 STM32 具体型号，但是在选择型号以后，需要先设计原理图，然后再画 PCB、打样、焊接、调试等步骤。这里我们重点介绍如何设计 STM32H743 的原理图。

任何 MCU 部分的原理图设计，其实都遵循：最小系统 + IO 分配 的设计原则。在开始设计原理图之前，我们要通读一遍对 STM32H743 原理图设计非常有用的手册：STM32H743 的数据手册，可以说不看这个数据手册，我们就无法设计 STM32H743 原理图。

数据手册

在设计 STM32H743 原理图的时候，我们需要用到一个非常重要的文档：STM32H743 数据手册，里面对 STM32H743 的 IO 定义和说明有非常详细的描述，是我们设计原理图的基础。正点原子阿波罗 STM32H743 开发板所使用的 STM32H743 芯片数据手册，存放在光盘：A 盘→7，硬件资料→2，芯片资料→ STM32H743IIT6.pdf，接下来我们简单介绍一下如何使用该文档。

STM32H743IIT6.pdf 是最新的英文版（V9）STM32 数据手册，暂时还没有中文版文档。大家可以根据自己的喜欢来选择合适的版本进行阅读，内容上基本大同小异，从准确性全面性的角度来说，看 V9 英文版是最好的。

STM32H743IIT6.pdf 数据手册是针对 STM32H743 系列的数据手册，主要包括 10 个章节，如表 2.3.4.1 所示：

章节	概要说明
介绍	简单说明数据手册作用：介绍高性能 H743 产品的订购信息和机械特性
规格说明	简单介绍 STM32H743 内部所有资源及外设特点
引脚定义	介绍不同封装的引脚分布、引脚定义等，含引脚特性、复用功能、脚位等
存储器映像	介绍 STM32H743 存储空间和外设的地址映射关系
电气特性	介绍 STM32H743 的详细电气特性，包括工作电压、电流、温度、各外设资源的电气性能等
封装特性	介绍了 STM32H743 不同封装的封装机械数据（脚距、长短等）、热特性等
订货代码	和 2.3.2 节内容类似，介绍 STM32 具体型号所代表的意义，方便选型订货
版本历史	介绍数据手册不同版本之间的差异和修订内容

表 2.3.4.1 STM32H743 数据手册各章节内容概要

整个 STM32H743 数据手册，对我们开发学习 STM32 来说都比较重要，因此建议大家可以简单的通读一遍这个文档，以加深印象。对于原理图设计，最重要的莫过于引脚定义这一章节了，只有知道了 STM32 的引脚定义，才能开始设计原理图。

STM32H743IIT6 引脚分布如图 2.3.4.1 所示：

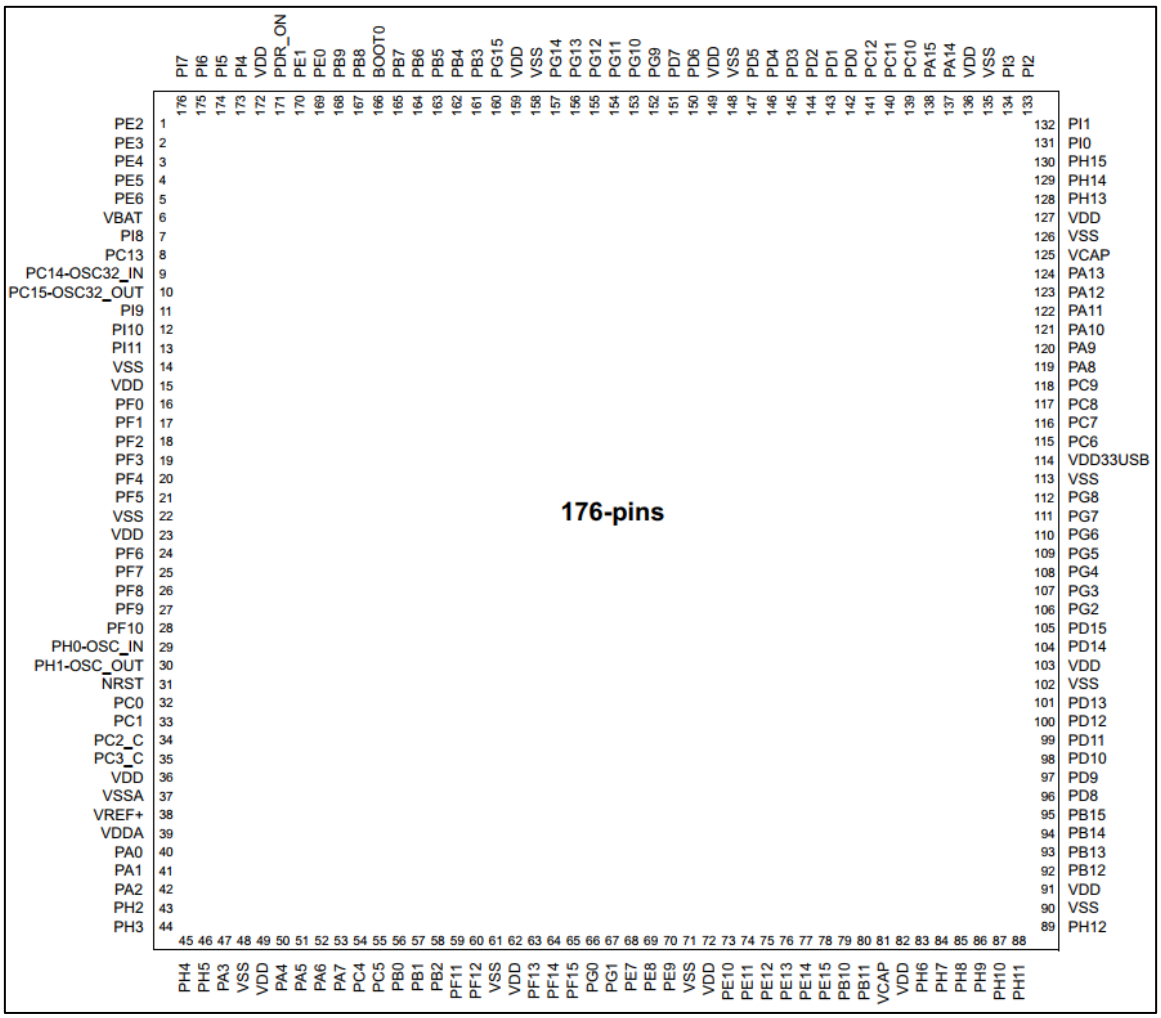


图 2.3.4.1 STM32H743IIT6 芯片引脚分布

STM32H743IIT6 引脚定义如表 2.3.4.2 所示：

Pin/ball name ①								②	③	④		⑤	⑥
LQFP100	TFBGA100	LQFP144	UFBGA169	UFBGA176+25	LQFP176	LQFP208	TFBGA240 +25	Pin name (function after reset)	Pin type	I/O structure	Notes	Alternate functions	Additional functions
1	A3	1	A2	A2	1	1	C3	PE2	I/O	FT_h	-	TRACECLK, SAI1_CK1, SPI4_SCK, SAI1_MCLK_A, SAI4_MCLK_A, QUADSPI_BK1_IO2, SAI4_CK1, ETH_MII_TXD3, FMC_A23, EVENTOUT	-
2	B3	2	B2	A1	2	2	D3	PE3	I/O	FT_h	-	TRACED0, TIM15_BKIN, SAI1_SD_B, SAI4_SD_B, FMC_A19, EVENTOUT	-

表 2.3.4.2 STM32H743IIT6 芯片引脚定义（部分）（摘自 STM32H743 数据手册）

引脚定义表的具体说明如表 2.3.4.3 所示：

序号	名称	说明
①	脚位	对应芯片的引脚，LQFP 使用纯数字表示，BGA 使用字母+数字表示

		这里列出了 8 种封装的脚位描述，根据实际型号选择合适的封装查阅
②	管脚名称	即对应引脚的名字，PE2~3 表示 GPIO 引脚
③	类型	I/O ：表示输入/输出引脚 I ：表示输入引脚 S ：表示电源引脚
④	IO 结构	FT ：表示 5V 兼容的引脚（可以接 5V/3.3V） TT ：表示 5V 不兼容引脚（仅可以接 3.3V） B ：独有的 BOOT0 引脚 RST：内置弱电阻的双向复位引脚 _f ：I2C FM +选项 _a ：模拟选项(由 V_{DDA} 提供) _u ：USB 选项(由 $V_{DD33USB}$ 提供) _h ：高速低压 I/O
⑤	复用功能	该引脚的复用功能
⑥	可选的 复用功能	重定义功能：是指可以通过重映射的复用功能，需设置重映射寄存器

表 2.3.4.3 引脚定义说明

了解了引脚分布和引脚定义以后，我们就可以开始设计 STM32H743 的原理图了。

最小系统

最小系统就是保证 MCU 正常运行的最低要求，一般是指 MCU 的供电、复位、晶振、BOOT 等部分。STM32H743 的最小系统需求如表 2.3.4.4 所示：

类型	引脚名称	说明
电源	VDD/VSS	电源正（VDD）/负（VSS）引脚，给 STM32 供电
	VDDA/VSSA	模拟部分电源正/负引脚，给 STM32 内部模拟部分供电
	VREF+/VREF-	参考电压正/负引脚，给 STM32 内部 ADC/DAC 提供参考电压 100 脚及以上的 STM32H743 型号才有这两个脚
	VBAT	RTC&后备区域供电引脚，给 RTC 和后备区域供电 一般 VBAT 接电池，用于断电维持 RTC 工作 如不需要，直接将 VBAT 接 VDD 即可
复位	NRST	复位引脚，用于复位 STM32，低电平复位
启动	BOOT	启动选择引脚，一般这个引脚接一个下拉电阻即可 其他启动配置说明详见后续分析
晶振	OSC_IN / OSC_OUT	外部 HSE 晶振引脚，用于给 STM32 提供高精度系统时钟 如果使用内部 HSI 能满足使用需求，这两个脚可以不接晶振
	OSC32_IN / OSC32_OUT	外部 LSE 晶振引脚，用于给 STM32 内部 RTC 提供时钟 如果使用内部 LSI 能满足使用需求，这两个脚可以不接晶振
调试	SWCLK/SWDIO	SWD 调试引脚，用于调试 STM32 程序，同时 STM32 还支持 JTAG 调试，不过我们不推荐使用！因为 SWD 省 IO！

表 2.3.4.4 STM32H743 最小系统需求

完成以上引脚设计以后，STM32H743 的最小系统就完成了，关于这些引脚的实际原理图，大家可以参考我们正点原子阿波罗 STM32H743 开发板的原理图。接下来就可以开始进行 IO 分配了。

IO 分配

IO 分配就是在完成最小系统设计以后，根据项目需要对 MCU 的 IO 口进行分配，连接不同的器件，从而实现整体功能。比如：GPIO、IIC、SPI、SDMMC、FMC、USB、中断等。遵循：先分配特定外设 IO，再分配通用 IO，最后微调的原则，见表 2.3.4.5 所示：

分配	外设	说明
特定外设	IIC	IIC 一般用到 2 根线：IIC_SCL 和 IIC_SDA（ST 叫 I2C） 数据手册有 I2C_SCL、I2C_SDA 复用功能的 GPIO 都可选用
	SPI	SPI 用到 4 根线：SPI_CS/MOSI/MISO/SCK 一般 SPI_CS 我们使用通用 GPIO 即可，方便挂多个 SPI 器件 数据手册有 SPI_MOSI/MISO/SCK 复用功能的 GPIO 都可选用
	TIM	根据需要可选：TIM_CH1/2/3/4/ETR/1N/2N/3N/BKIN 等 数据手册有 TIM_CH1/2/3/4/ETR/1N/2N/3N/BKIN 复用功能的 GPIO 都可选用
	USART UART	USART 有 USART_TX/RX/CTS/RTS/CK 信号 UART 仅有 UART_TX/RX 两个信号 一般用到 2 根线：U(S)ART_TX 和 U(S)ART_RX 数据手册有 U(S)ART_TX/RX 复用功能的 GPIO 都可选用
	USB	USB 用到 2 根线：USB_DP 和 USB_DM 数据手册有 USB_DP、USB_DM 复用功能的 GPIO 都可选用
	CAN	CAN 用到 2 根线：CAN_RX 和 CAN_TX 数据手册有 USB_DP、USB_DM 复用功能的 GPIO 都可选用
	ADC	ADC 根据需要可选：ADC_IN0～ADC_IN15 数据手册有 ADC_IN0～ADC_IN15 复用功能的 GPIO 都可选用
	DAC	DAC 根据需要可选：DAC_OUT1 / DAC_OUT2 DAC 固定为：DAC_OUT1 使用 PA4、DAC_OUT2 使用 PA5
	SDMMC	SDMMC 一般用到 6 根线：SDMMC_D0/1/2/3/SCK/CMD 数据手册有 SDMMC_D0/1/2/3/SCK/CMD 复用功能的 GPIO 都可选用
	FMC	根据需要可选：FMC_D0~15/A0~25/ NBL0~1/NE1~4/NCE2~3/ NOE/NWE/NWAIT/CLK 等 数据手册有 FMC_D0~15/A0~25/ NBL0~1/NE1~4/NCE2~3/NOE/ NWE/NWAIT/CLK 复用功能的 GPIO 都可选用
通用	GPIO	在完成特定外设的 IO 分配以后，就可以进行 GPIO 分配了 比如将按键、LED、蜂鸣器等仅需要高低电平读取/输出的外设连接到空闲的普通 GPIO 即可
微调	IO	微调主要包括两部分： 1，当 IO 不够用的时候，通用 GPIO 和特定外设可能要共用 IO 口 2，为了方便布线，可能要调整某些 IO 口的位置 这两点，得根据实际情况进行调整设置，做到：尽可能多的可以同时使用所有功能，尽可能方便布线。

表 2.3.4.5 IO 分配

经过以上几个步骤，我们就可以完成 STM32H743 的原理图设计了。

第三章 开发环境搭建

本章，我们将向大家介绍 STM32 的开发环境搭建，通过本章的学习，我们将了解到有哪些常用的 STM32 开发工具，包括 IDE、调试器、串口工具等。

本章将分为如下几个小节：

3.1 常用开发工具简介
3.2 MDK5 安装
3.3 仿真器驱动安装
3.4 CH340 USB 虚拟串口驱动安装

3.1 常用开发工具简介

我们开发 STM32 需要用到一些开发工具，如：IDE、仿真器、串口调试助手等。常见的工具如表 3.1.1 所示：

工具	名称	说明
集成开发环境	**MDK**	全名：RealView MDK，是 Keil 公司（已被 ARM 收购）的一款集成开发环境，界面美观，简单易用，是 STM32 最常用的集成开发环境
	EWARM	IAR 公司的一款集成开发环境，支持 STM32 开发，对比 MDK，IAR 的使用人数少一些，用惯 IAR 的朋友可以选择这款软件开发 STM32
仿真器	**DAP**	ARM 公司的开源仿真器，可支持 STM32 仿真调试，且带虚拟串口功能。有高速和低速两个版本，具有免驱、速度快、价格便宜等特点
	STLINK	ST 公司自家的仿真器，支持 STM32 和 STM8 仿真调试，目前最常用的是 ST LINK V2，支持全面、稳定、廉价，是其特点
	JLINK	Segger 公司的仿真器，可支持 STM32 仿真调试，具有稳定、高速的特点，就是价格有点贵。
串口调试助手	**XCOM**	正点原子开发的串口调试助手，具有稳定、功能多、使用简单等特点
	SSCOM	丁丁的串口调试助手，具有稳定、小巧、使用简单等特点。

表 3.1.1 常用开发工具

大家可以根据自己的需要和喜好，选择合适的开发工具。表中加粗部分是我们推荐使用的 STM32 开发工具，即 IDE 推荐使用 MDK、仿真器推荐使用 DAP、串口调试助手推荐使用 XCOM，接下来我们介绍这几个软件的安装。

3.2 MDK 安装

注意：MDK 是一款付费集成开发环境，如果大家要商用，请联系 Keil 公司购买，我们这里仅用于教学使用。

MDK5 的安装分为两步：1，安装 MDK5；2，安装器件支持包。

MDK 软件下载地址：https://www.keil.com/download/product，目前最新版本是 MDK5.36。
器件支持包下载地址：https://www.keil.com/dd2/pack，STM32H7 支持包最新版本是 2.7.0。

MDK5.36 和 2.7.0 的 STM32H7 器件支持包我们都已经放在光盘 A 盘了，具体路径为：A 盘→ 6，软件资料→1，软件→1，MDK5，如图 3.2.1 所示：

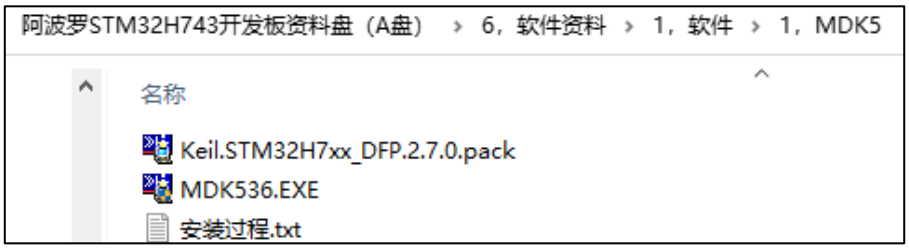


图 3.2.1 MDK5 软件及 STM32H7 器件支持包

MDK5 的安装比较简单，具体安装步骤请参考图 3.2.1 的 **安装过程.txt** 进行安装即可，需要提醒一下大家，在选择安装路径的时候，强烈建议大家将 Pack 的路径和 Core 的路径放在一个位置，比如我们安装在 D 盘（都安装在：D:\MDK5.36 路径下），如图 3.2.2 所示：

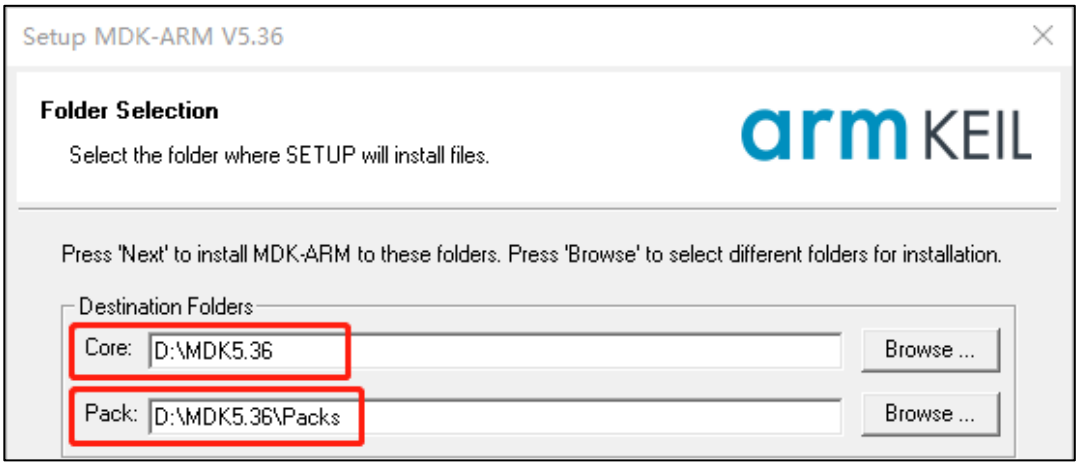


图 3.2.2 设置 Core 和 Pack 安装路径

安装完成后，在我们电脑桌面会显示 MDK5 图标，如图 3.2.3 所示；

图 3.2.3 桌面显示 MDK5 图标

3.3 仿真器驱动安装

STM32 可以通过 DAP、ST LINK、JLINK 等仿真调试器进行程序下载和仿真，我们推荐使用：DAP 仿真器（CMSIS-DAP Debugger），DAP 仿真器在 MDK 下是免驱动的（无需安装驱动），即插即用，非常方便。

正点原子提供了两种规格的 DAP 仿真器：普速版本 DAP（ATK-DAP）和高速版本（ATK-HSDAP），它们的使用方法完全一样，只是高速版本速度更快，大家根据需求选择即可。

如果你用的是 STLINK 仿真器，大家可以参考《ST LINK 调试补充教程 V2.pdf》安装驱动，该文档路径如下：A 盘→ 6，软件资料→1，软件→5，其他软件→ST LINK 驱动及教程。

3.4 CH340 USB 虚拟串口驱动安装

安装 CH340 USB 虚拟串口驱动，以便我们使用电脑通过 USB 和 STM32 进行串口通信。开发板使用的 USB 虚拟串口芯片是 CH340C，其驱动我们已经放在开发板的如下路径：A 盘→6，软件资料→1，软件→ 2，CH340 驱动(USB 串口驱动)，如图 3.4.1 所示：

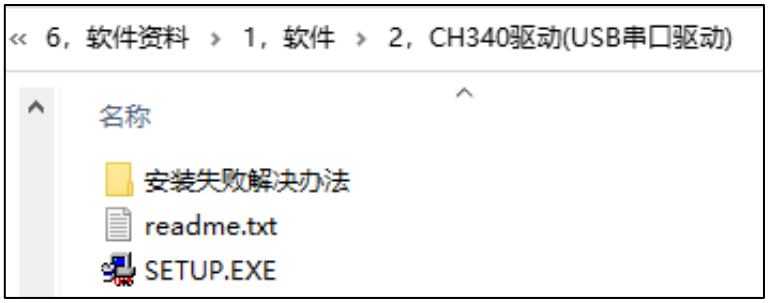


图 3.4.1 CH340 驱动

双击 SETUP.EXE 进行安装，安装完成后，如图 3.4.2 所示：

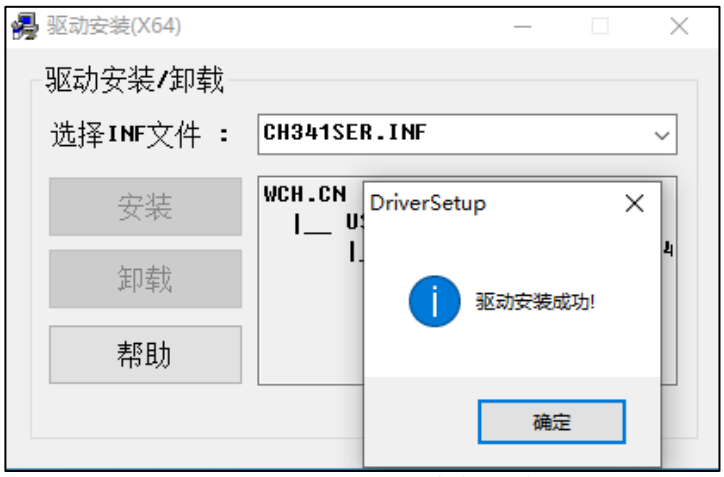


图 3.4.2 CH340 驱动安装成功

在驱动安装成功之后，将开发板的 USB_UART 接口通过 USB 连接到电脑，此时电脑就会自动给其安装驱动了。在安装完成之后，可以在电脑的设备管理器里面找到 USB 串口（如果找不到，则重启下电脑），如图 3.4.3 所示：

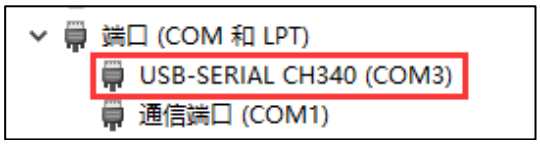


图 3.4.3 设备管理器显示 CH340 USB 虚拟串口

在图 3.4.3 中可以看到，我们的 USB 虚拟串口被识别为 COM3，这里需要注意的是：不同电脑可能不一样，你的可能是 COM4、COM5 等，但是 USB-SERIAL CH340，这个一定是一样的。如果没找到 USB 串口，则有可能是你安装有误，或者系统不兼容。

在安装完 CH340 USB 虚拟串口以后，我们就可以使用串口调试助手，比如 XCOM，和我们的开发板，通过串口进行通信了，这个我们在后续内容再给大家介绍。至此，STM32 的开发环境就搭建完成了。

第四章 STM32 初体验

本章，我们不介绍如何编写代码，而是向大家介绍如何编译、串口下载、仿真器下载、仿真调试开发板例程，体验一下 STM32 的开发流程，并介绍 MDK5 的一些使用技巧，通过本章的学习，将对 STM32 的开发流程和 MDK5 使用有个大概了解，为后续深入学习打好基础。

本章将分为如下几个小节：

4.1 使用 MDK5 编译例程
4.2 使用串口下载程序
4.3 使用 DAP 下载与调试程序
4.4 MDK5 使用技巧

4.1 使用 MDK5 编译例程

我们在编写完代码以后，需要对代码进行编译，编译成功以后，才能下载到开发板进行验证、调试等操作。正点原子阿波罗 STM32H743 开发板标准例程源码路径：A 盘→4，程序源码，如图 4.1.1 所示：

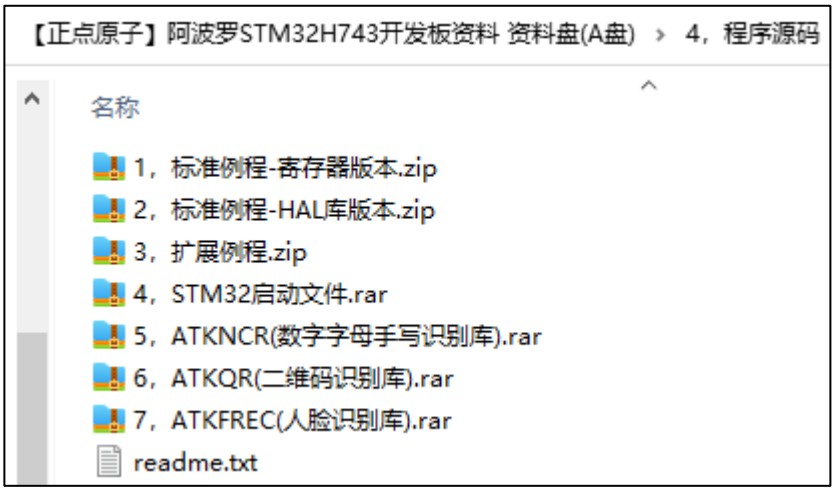


图 4.1.1 正点原子阿波罗 STM32H743 开发板例程源码

这 7 个压缩包说明如表 4.1.1：

压缩包	说明
标准例程-寄存器版本	寄存器版本标准例程，无独立教程，仅供进阶参考
标准例程-HAL 库版本	HAL 库版本标准例程，搭配《阿波罗 STM32H743 开发指南_V1.0.pdf》学习
扩展例程	包含 ucOS、FreeRTOS、emWIN、LVGL 例程搭配对应的开发指南学习
STM32 启动文件	STM32H743 启动文件，用于新建工程
ATKNCR(数字字母手写识别库)	正点原子提供的数字字母手写识别库
ATKQR(二维码识别库)	正点原子提供的二维码识别库
ATKFREC(人脸识别库)	正点原子提供的人脸识别库

表 4.1.1 源码压缩包说明

本书主要针对“标准例程-HAL 库版本”源码进行讲解，所以，这里我们先解压该压缩包，得到 HAL 库版本的标准例程源码如图 4.1.2 所示：

图 4.1.2 标准例程-HAL 库版本

正点原子阿波罗 STM32H743 开发板总共有 65 个实验的标准例程，部分实验有多个例程，比如：通用定时器实验、高级定时器实验、ADC 实验等，实际上我们的总例程数达到了 81 个。这里我们没有把实验 0 统计进来，因为实验 0 主要是新建工程等 MDK 使用基础教学例程。

秉着简单易懂的原则，我们选择：实验 1，跑马灯实验 作为 STM32 入门体验，例程目录如图 4.1.3 所示：

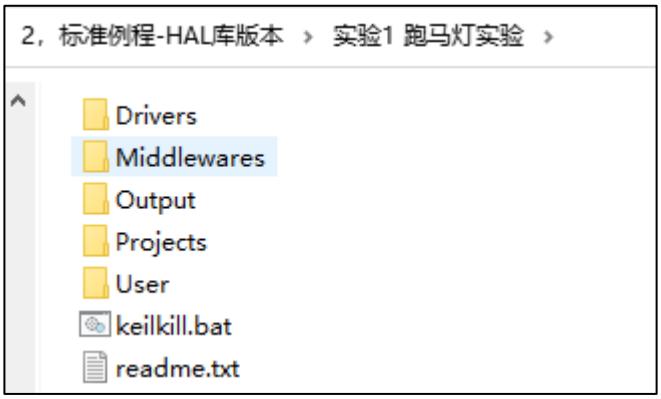


图 4.1.3 跑马灯例程工程目录结构

工程目录下有 5 个文件夹，我们将在后续“新建 HAL 库版本 MDK 工程”这一章给大家详细介绍其作用（其实看文件夹名字的字面意思基本就知道是做啥用的了），这里我们不多说。例程 MDK 工程文件的路径为：Projects→MDK-ARM→ atk_h743.uvprojx，如图 4.1.4 所示：

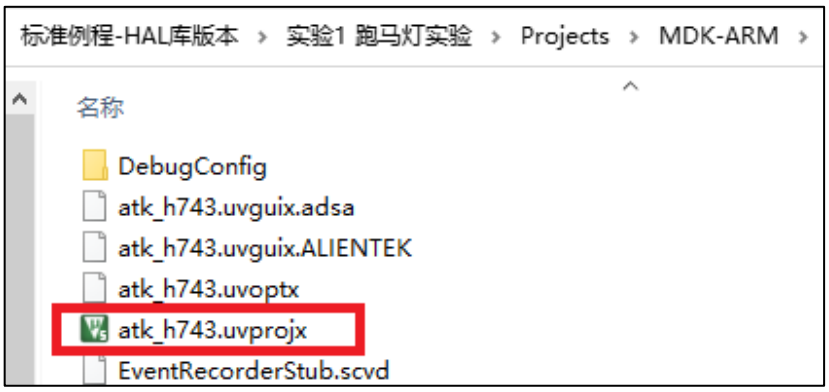


图 4.1.4 跑马灯实验例程 MDK 工程

注意：一定要先安装 MDK5（详见第三章），否则无法打开该工程文件！

双击 atk_h743.uvprojx 打开该工程文件，进入 MDK IDE 界面，如图 4.1.5 所示：

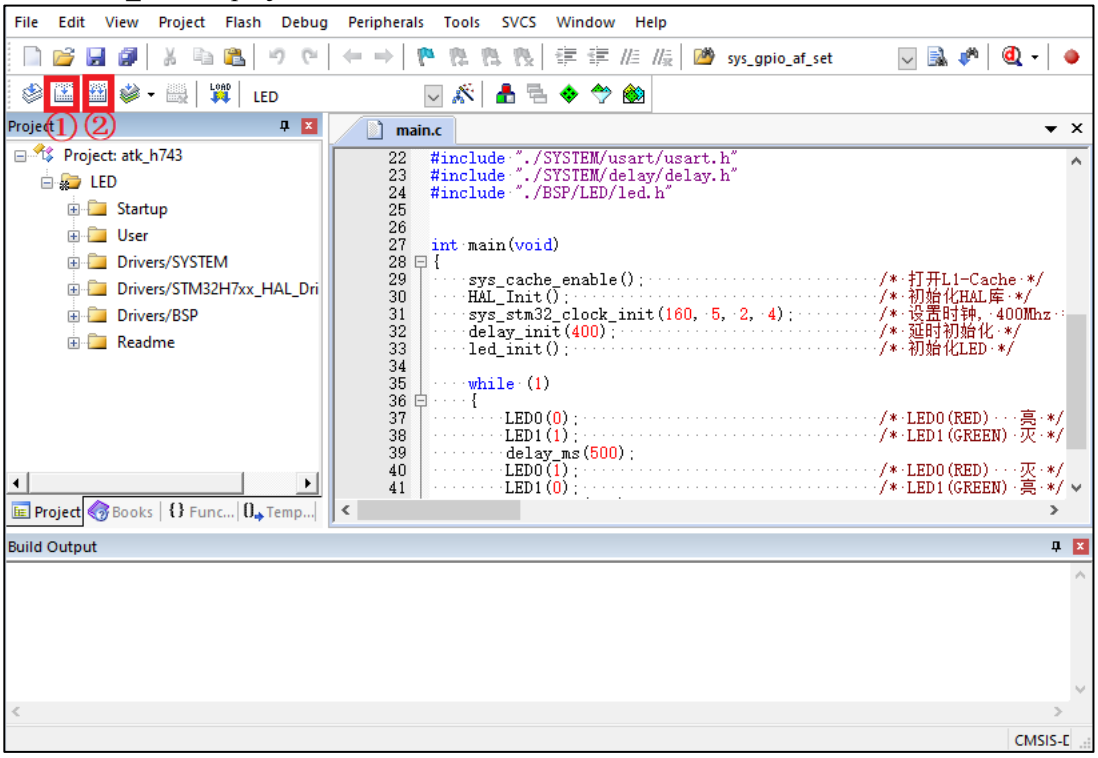


图 4.1.5 MDK 打开跑马灯实验例程

① 是编译按钮，表示编译当前工程项目文件，如果之前已经编译过了，则只会编译有改动的文件。所以一般第一次会比较耗时间，后续因为只编译改动文件，从而大大缩短了编译时间。该按钮可以通过 F7 快捷键进行操作。

② 是重新编译当前工程所有文件按钮，工程代码较多时全部重新编译会耗费比较多的时间，建议少用。

按①处的按钮，编译当前项目，在编译完成后，可以看到如图 4.1.6 所示的编译提示信息：

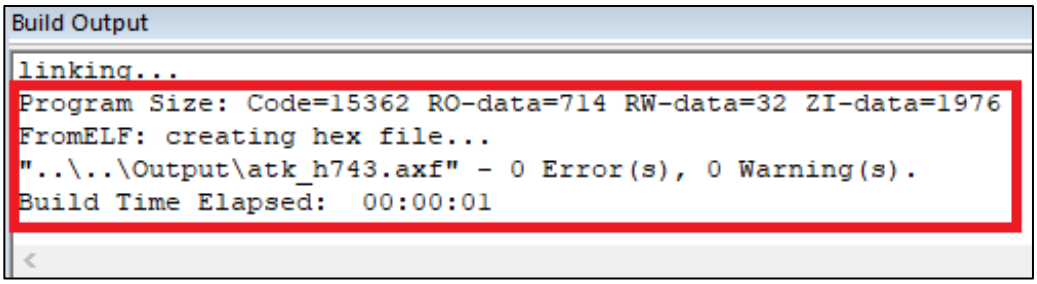


图 4.1.6 编译提示信息

图中：

Code：表示代码大小，占用 15362 字节。

RO-Data：表示只读数据所占的空间大小，一般是指 const 修饰的数据大小。

RW-Data：表示有初值（且非 0）的可读写数据所占的空间大小，它同时占用 FLASH（存放其初始值）和 RAM 空间。

ZI-Data：表示初始化为 0 的可读写数据所占空间大小，它只占用 RAM 空间。

因此图 4.1.6 的提示信息表示：代码总大小（Porgram Size）为：FLASH 占用 16108 字节（Code + RO + RW），SRAM 占用 2008 字节（RW + ZI）；成功创建了 Hex 文件（可执行文件，放在 Output 目录下）；编译 0 错误，0 警告；编译耗时 1 秒钟。

编译完成以后，会生成 Hex 可执行文件，默认输出在 Output 文件夹下，如图 4.1.7 所示：

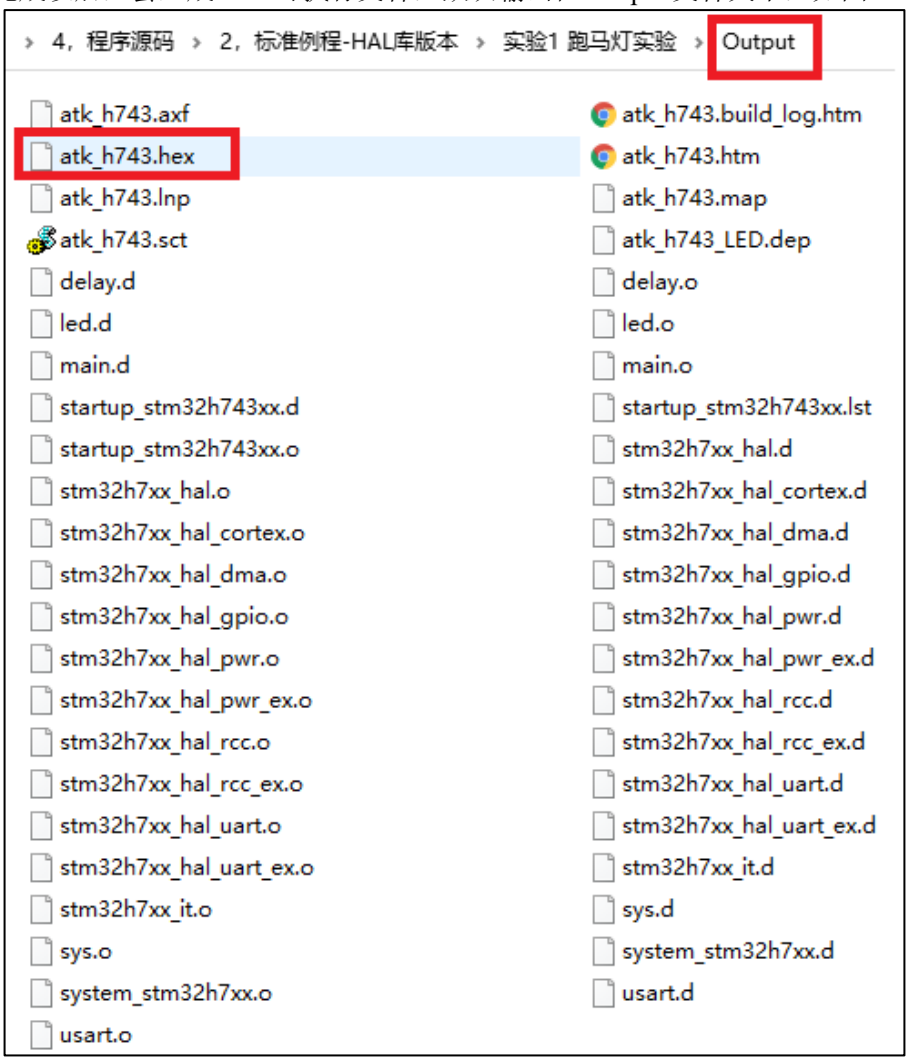


图 4.1.7 Hex 可执行文件

注意：必须编译成功，才会生成 Hex 可执行文件，否则是不会有这个文件的！！

Output 文件夹下除了.hex 文件还有很多其他文件（.axf、.htm、.dep、.lnp、.o、.d、.lst 等），这些文件是编译过程所产生的中间文件，我们将在后续的.map 文件分析给大家详细介绍这些文件的作用。至此，例程编译完成。其他实验例程，大家可以用同样的方法进行编译。

4.2 使用串口下载程序

STM32 常见的下载方式有三种：

下载方式	使用接口	下载条件	优缺点
串口下载	串口 1	1，BOOT0 接 3.3V	优点：仅需一个串口即可通过上位

	PA9/PA10	2，BOOT1 接 GND 3，按复位	机软件下载程序，经济实惠 缺点：速度慢
SWD 下载	SWD 口 PA13/PA14	直接下载 特殊情况需 BOOT0 接 3.3V，按复位	优点：2 个 IO 口就可以下载、仿真、调试代码，高速、高效、实用 缺点：需要一个仿真器
JTAG 下载	JTAG 口 PB3/4/PA13/14/15	直接下载 特殊情况需 BOOT0 接 3.3V，按复位	优点：5 个 IO 口实现下载、仿真、调试代码，实用方便 缺点：需要一个仿真器，占用 IO 多

表 4.2.1 JTAG 和 SWD 下载方式对比

经过以上对比，因此我们推荐使用 **SWD 下载**，强烈推荐大家购买一个仿真器（如 ST LINK、CMSIS DAP 等），可以极大的方便学习和开发。不推荐使用**串口下载**（速度慢、无法仿真和调试）和 **JTAG 下载**（占用 IO 多）。

表 4.2.2 中 BOOT0 和 BOOT1 是 STM32 芯片上面的两个引脚，用于控制 STM32 的启动方式，具体如表 4.2.2 所示：

BOOT0	BOOT1	启动模式	说明
0	X	用户闪存存储器启动	用户闪存储器存启动，也就是 FLASH 启动 正常运行我们自己的程序，就应该用此设置
1	0	系统存储器启动	系统存储器启动，用于串口下载程序
1	1	SRAM 启动（不常用）	SRAM 启动，一般没用

表 4.2.2 STM32 启动模式

BOOT 引脚接 VCC，是从系统存储器启动，进入后直接运行 ST 官方的 Bootloader 程序，就可以通过 ST 官方预设的接口下载程序，如：USART、I2C、SPI、USB-DFU 等。该模式除了可以通过上述的接口下载程序，其实还有一个作用：可用于异常关闭 JTAG/SWD 导致仿真器无法下载程序时的补救措施（在此模式下，下载一个不关闭 JTAG/SWD 接口的程序即可救活）。

接下来，我们将向大家介绍，如何利用串口给 STM32H743（以下简称 STM32）下载代码。

STM32 通过串口 1 实现程序下载， 阿波罗 H743 开发板通过自带的 USB 转串口来实现串口下载。看起来像是 USB 下载（只需一根 USB 线，并不需要串口线）的，实际上，是通过 USB 转成串口，然后再下载的。

下面，我们就一步步教大家如何在实验平台上利用 USB 串口来下载代码。

首先确保开发板接线正确，并成功上电，如图 4.2.1 所示：

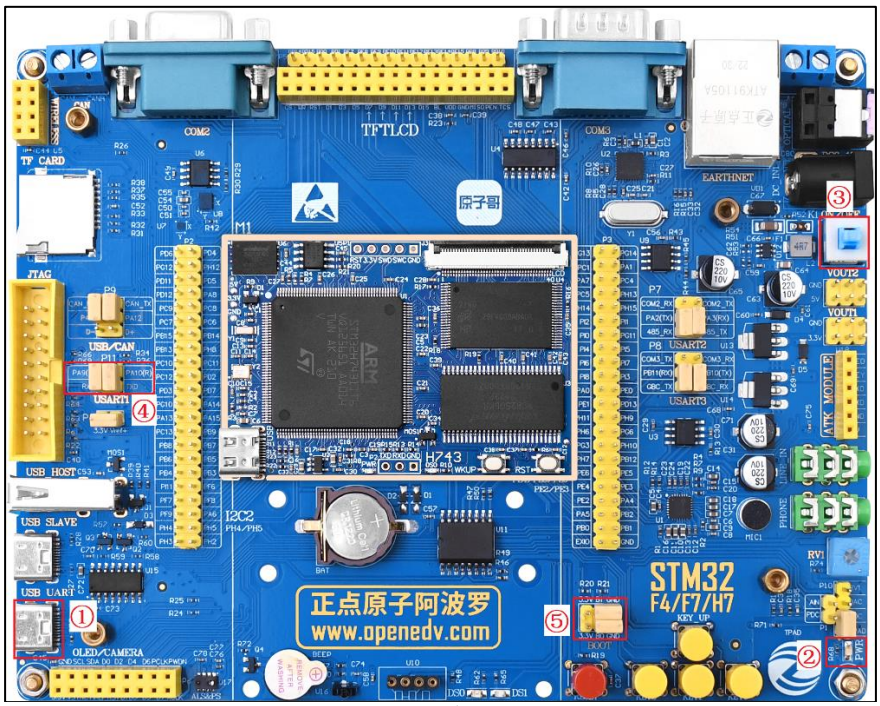


图 4.2.1 开发板设置

a) ①处的 USB_UART 通过 USB 线连接电脑，实现 USB 转串口，同时支持给开发板供电。

b) 确保电源灯②亮起（蓝色），如果不亮检查供电和电源开关是③否按下？

c）确保 RXD 和 PA9（STM32 的 TXD），TXD 和 PA10(STM32 的 RXD)通过跳线帽连接起来（标号④），这样我们就把 CH340C 和 STM32 的串口 1 连接上了。

d）⑤处的 BOOT 设置为 BOOT0（简称 B0）和 BOOT1（简称 B1）都接 GND（一键下载电路自动控制，待会介绍）。

由表 4.2.1 可知，STM32 串口下载的标准方法是四个步骤：

1， 把 B0 接 V3.3；

2， 保持 B1 接 GND；

3， 按一下复位按键；

4， 使用上位机软件下载代码；

通过这几个步骤，我们就可以通过串口下载代码了，下载完成之后，如果没有设置从 0X08000000 开始运行，则代码不会立即运行，此时，你还需要把 B0 接回 GND，然后再按一次复位，才会开始运行你刚刚下载的代码。所以整个过程，你得跳动 2 次跳线帽，还得按 2 次复位，比较繁琐。而我们的一**键下载电路**，则利用串口的 DTR 和 RTS 信号，分别控制 STM32 的复位和 B0，配合正点原子团队研发的上位机软件 ATK-XISP，设置：**DTR 的低电平复位，RTS 高电平进 BootLoader**，这样，B0 和 STM32 的复位，完全可以由下载软件自动控制，从而实现一键下载。一键下载的原理图级别解释详见：阿波罗 STM32H743 硬件参考手册_V1.0.pdf。

串口程序下载需要用到：ATK-XISP 这个上位机软件。这个软件可以实现对 STM32F1 到 STM32H7 等系列芯片的串口编程。ATK-XISP 软件在光盘路径：A 盘→6，软件资料→1，软件→5，其他软件→ STM32 串口下载软件（ATK-XISP），如图 4.2.2 所示：

图 4.2.2 ATK-XISP 串口编程软件

双击打开 ATK-XISP，进行如图 4.2.3 所示的设置：

图 4.2.3 ATK-XISP 设置

① 搜索串口选择 CH340 虚拟的串口（我的是 COM14，不同电脑可能不同，需要根据实际情况选择对应的串口，详见 3.4 节），对于 STM32H7，由于 H7 自带的 bootlaoder 程序对高波特率支持不太好，所以推荐设置波特率为 115200bps，高的波特率将导致极低的下载成功率。

② 选择 4.1 节编译生成的 hex 文件（在 Output 文件夹）。

③ 勾选编程后运行、校验和编程前执行全片擦除三项，可以保证下载代码的正确性，并下载完后自动运行，省去按复位的麻烦。

④ 选择 **DTR 的低电平复位，RTS 高电平进 BootLoader（别选错！）**，以匹配一键下载电路，实现一键下载代码，省去设置 BOOT0、按复位的麻烦。

设置好之后，我们就可以通过按**开始编程（P）**这个按钮，一键下载代码到 STM32 上，下载成功后如图 4.2.4 所示：

图 4.2.4 下载完成

图 4.2.4 中，我们圈出了 ATK-XISP 对一键下载电路的控制过程，先是获取芯片信息，然后对全片擦除，还显示了文件的首地址以及文件大小等。

下载成功后，会有“共写入 xxxxKB，运行成功，共耗时 xxxx 毫秒”的提示，并且从 0X80000000 处开始运行了。此时表示代码下载完成，并已经成功运行了，我们查看开发板就可以看到 DS0（红灯）、DS1（绿灯）开始交替闪烁了，如图 4.2.5 所示：

图 4.2.5 开发板运行代码

至此，STM32 使用串口下载程序完成。其他实验例程，大家可以用同样的方法进行下载。

4.3 使用 DAP 下载与调试程序

上一节我们使用串口给 STM32 下载程序，但是串口下载并不能仿真调试代码，只能下载后观看运行结果，所以在调试代码 bug 的时候，最好还是用仿真器进行下载调试，本节我们将给大家介绍如何使用仿真器给 STM32 下载代码，并调试代码。

这里我们以 DAP 仿真器为例给大家进行讲解，如果你用的是其他仿真器，基本上也是一样的用法，只是选择仿真器的时候，选择对应的型号即可。

DAP 与开发板连接如图 4.3.1 所示：

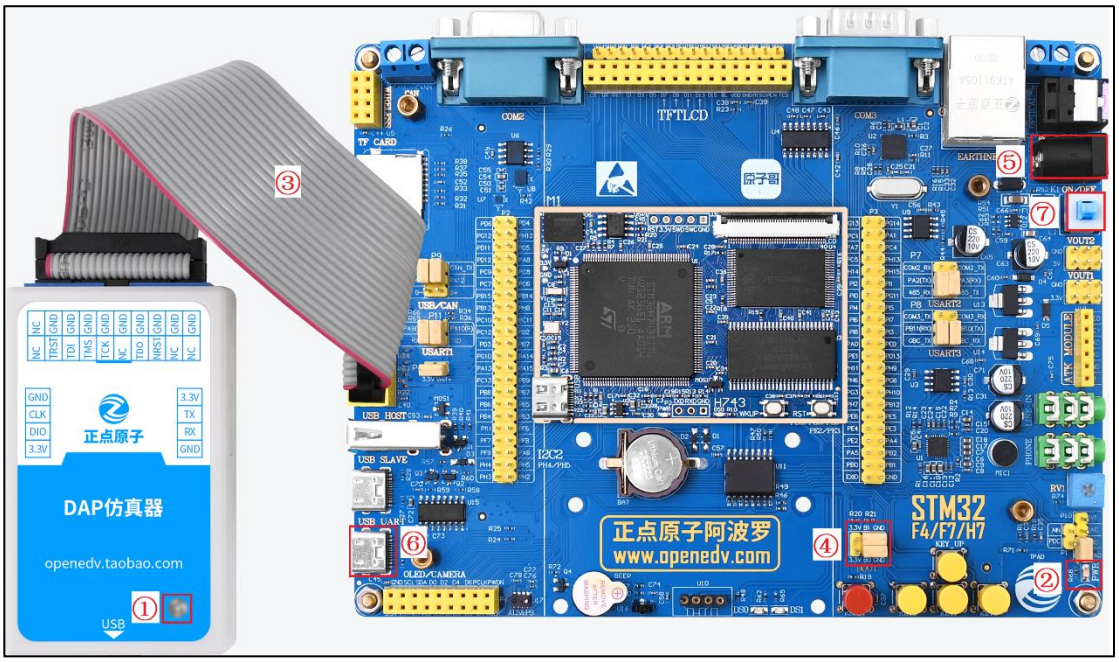


图 4.3.1 DAP 与开发板连接

a) DAP 通过 USB 线连接电脑，且仿真器①处的蓝灯常亮。

b) DAP 通过 20P 灰排线连接开发板，如图③处所示。

c) 确保开发板已经正常供电，蓝色电源灯（②处）亮起，通过⑤处供电，⑥处控制开关。

d) B0，B1 都接 GND，如图④处所示。

4.3.1 使用 DAP 下载程序

在 4.1 节的跑马灯例程 MDK IDE 界面下，点击按钮，打开 Options for Target 选项卡，在 Debug 栏选择仿真工具为 use：CMSIS-DAP Debugger，如图 4.3.1.1 所示：

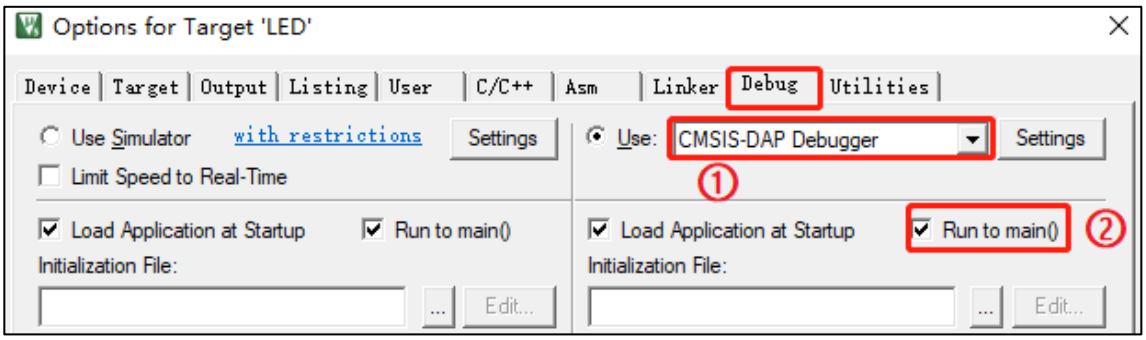


图 4.3.1.1 Debug 选项卡设置

①选择使用 CMSIS-DAP Debugger 仿真器仿真调试代码。如果你使用的是其他仿真器，比如 STLINK、JLINK 等，请在这里选择对应的仿真器型号。

②该选项选中后，只要点击仿真就会直接运行到 main 函数，如果没选择这个选项，则会先执行 startup_stm32h743xx.s 文件的 Reset_Handler，再跳到 main 函数。

然后我们点击 Settings，设置 DAP 的一些参数，如图 4.3.1.2 所示：

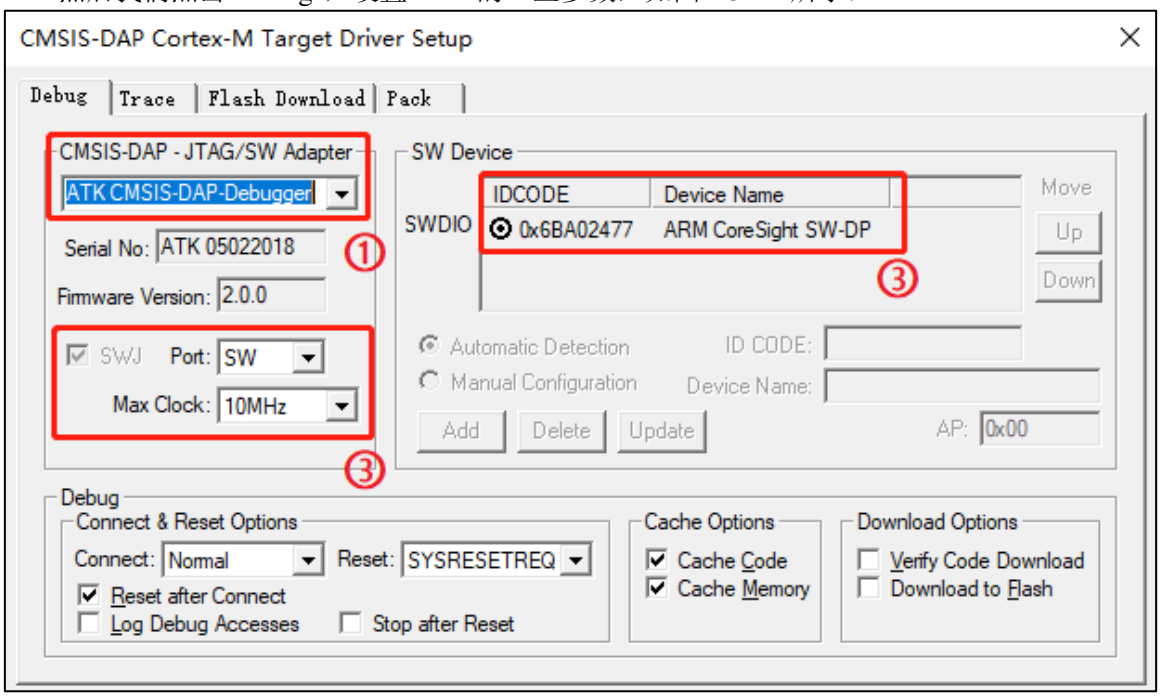


图 4.3.1.2 DAP 仿真器参数设置

①表示 MDK 找到了 ATK CMSIS-DAP 仿真器，如果这里显示为空，则表示没有仿真器被找到，请检查你的电脑是否接了仿真器？并安装了对应的驱动？

②设置接口方式，这里选择 SW（比 JTAG 省 IO），通信速度设置为 10Mhz（实际上大概只有 4M 的速度，MDK 会自动匹配）。

③表示 MDK 通过仿真器的 SW 接口找到了目标芯片，ID 为：0x6BA02477。如果这里显示：No target connected，则表示没找到任何器件，请检查仿真器和开发板连接是否正常？开发板是否供电了？

其他部分使用默认设置，设置完成以后单击“确定”按钮，完成此部分设置，接下来我们还需要在 Utilities 选项卡里面设置下载时的目标编程器，如图 4.3.1.3 所示：

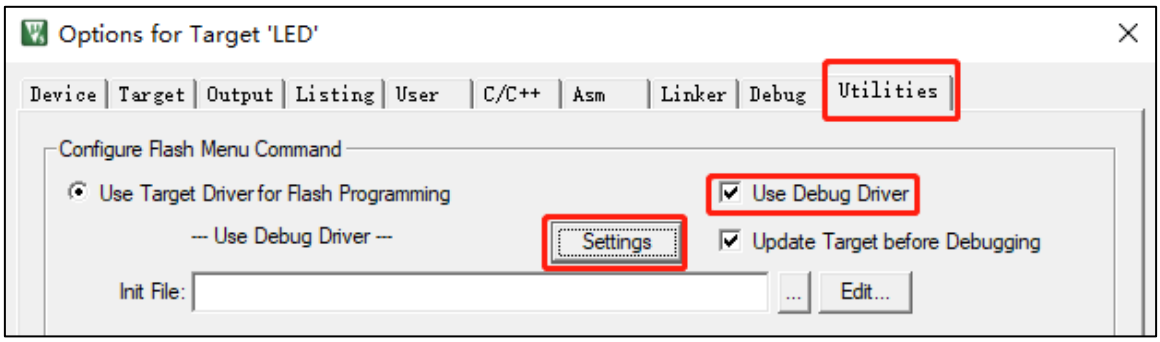


图 4.3.1.3 FLASH 编程器选择

上图中，我们直接勾选 Use Debug Driver，即和调试一样，选择 DAP 来给目标器件的 FLASH 编程，然后点击 Settings，进入 FLASH 算法设置，设置如图 4.3.1.4 所示：

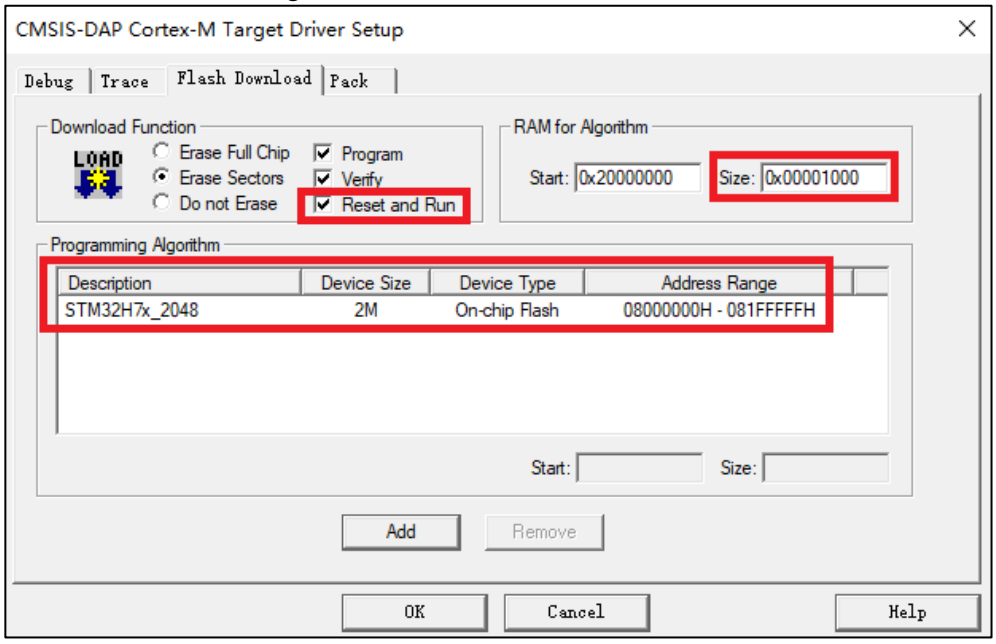


图 4.3.1.4 FLASH 算法设置

这里 MDK5 会根据我们新建工程时选择的目标器件，自动设置 flash 算法。我们使用的是 STM32H743IIT6，FLASH 容量为 2M 字节，所以 Programming Algorithm 里面默认会有 2M 的 STM32H743xx 内部 Flash 下载算法。另外，如果这里没有 flash 算法，大家可以点击 Add 按钮，自行添加即可。最后，选中 Reset and Run 选项，以实现在编程后自动运行，其他默认设置即可。

在设置完之后，点击“确定”，然后再点击“OK”，回到 IDE 界面，编译（可按 F7 快捷键）一下工程，编译完成以后（0 错误，0 警告），我们按 （快捷键：F8）这个按钮，就可以将代码通过仿真器下载到开发板上，在 IDE 下方的 Build Output 窗口就会提示相关信息，如图 4.3.1.5 所示：

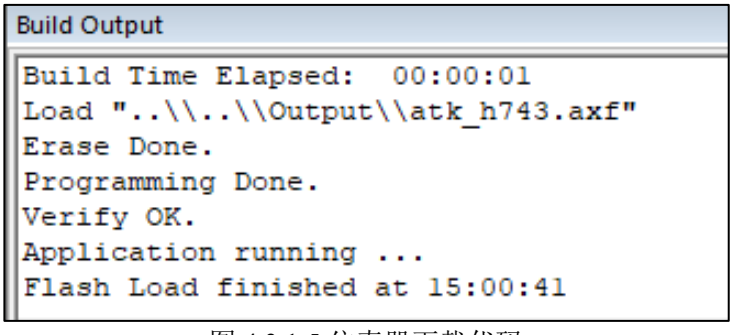


图 4.3.1.5 仿真器下载代码

下载完后，可以看到 LED0 和 LED1 交替闪烁了，说明代码下载成功。

4.3.2 使用 DAP 仿真调试程序

在正常编译完例程以后（0 错误，0 警告），点击： （开始/停止仿真按钮），开始仿真（如果开发板的代码没被更新过，则会先更新代码（即下载代码），再仿真），如图 4.3.2.1 所示：

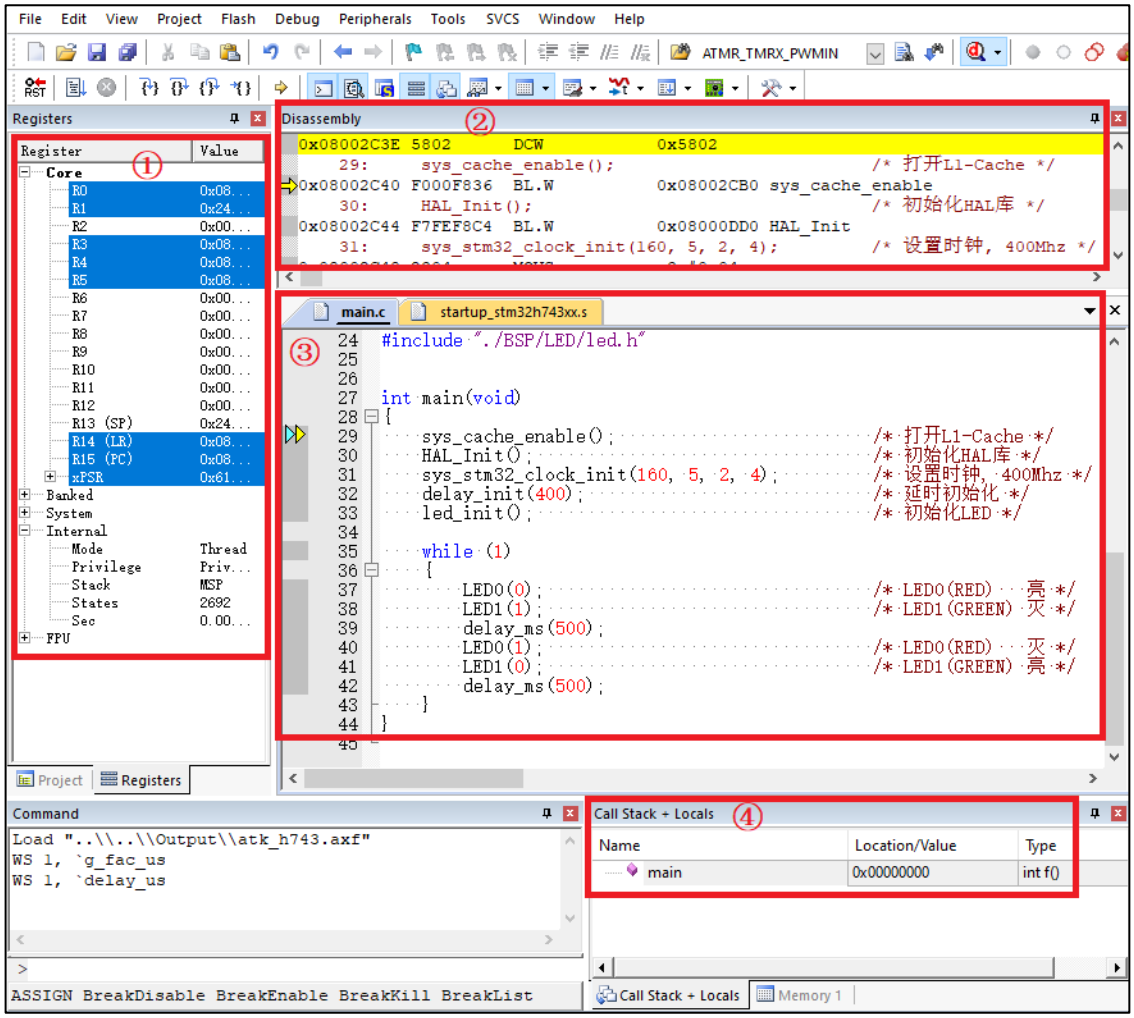


图 4.3.2.1 开始仿真

① Register：寄存器窗口，显示了 Cortex M7 内核寄存器 R0~R15 的值，还显示了内部的线程模式（处理者模式、线程模式）及特权级别（用户级、特权级），并且还显示了当前程序的运行时间（Sec），该选项卡一般用于查看程序运行时间，或者比较高级的 bug 查找（涉及到分析 R0~R14 数据是否异常了）。

② Disassembly：反汇编窗口，将 C 语言代码和汇编对比显示（指令存放地址，指令代码，指令，具体操作），方便从汇编级别查看程序运行状态，同样也属于比较高级别的 bug 查找。

③ 代码窗口，在左侧有黄绿色三角形，黄色的三角形表示将要执行的代码，绿色的三角形表示当前光标所在代码（C 代码 或 当前汇编行代码对应的 C 代码）。一般情况下，这两个三角形是同步的，只有在点击光标查看代码的时候，才可能不同步。

④ Call Stack + Locals：调用关系&局部变量窗口，通过该窗口可以查看函数调用关系，以及函数的局部变量，在仿真调试的时候，是非常有用的。

开始仿真的默认窗口，我们就给大家介绍这几个，实际上还有一些其他的窗口，比如 Watch、Memory、外设寄存器等也是很常用的，可以根据实际使用选择调用合适的窗口来查看对应的数据。

图 4.3.2.1 中，还有一个很重要的工具条：Debug 工具条，其内容和作用如图 4.3.2.2 所示：

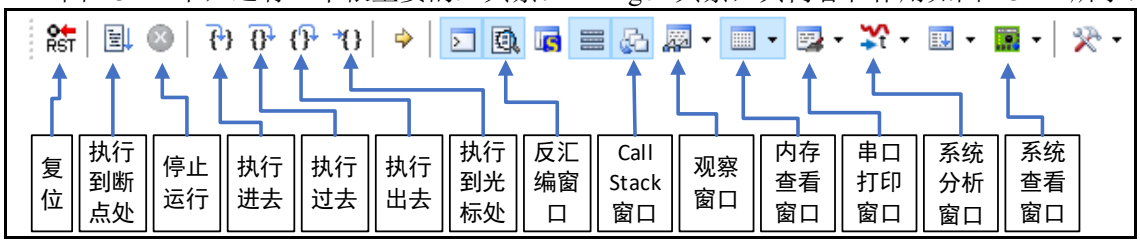


图 4.3.2.2 Debug 工具条

复位：其功能等同于硬件上按复位按钮。相当于实现了一次硬复位。按下该按钮之后，代码会重新从头开始执行。

执行到断点处：该按钮用来快速执行到断点处，有时候你并不需要观看每步是怎么执行的，而是想快速的执行到程序的某个地方看结果，这个按钮就可以实现这样的功能，前提是你在查看的地方设置了断点。

停止运行：此按钮在程序一直执行的时候会变为有效，通过按该按钮，就可以使程序停止下来，进入到单步调试状态。

执行进去：该按钮用来实现执行到某个函数里面去的功能，在没有函数的情况下，是等同于执行过去按钮的。

执行过去：在碰到有函数的地方，通过该按钮就可以单步执行过这个函数，而不进入这个函数单步执行。

执行出去：该按钮是在进入了函数单步调试的时候，有时候可能不必再执行该函数的剩余部分了，通过该按钮就可以一步执行完该函数的余部分，并跳出函数，回到函数被调用的位置。

执行到光标处：该按钮可以迅速的使程序运行到光标处，其实是挺像执行到断点处按钮功能，但是两者是有区别的，断点可以有多个，但是光标所在处只有一个。

反汇编窗口：通过该按钮，就可以查看汇编代码，这对分析程序很有用。

Call STACK 窗口：通过该按钮，显示调用关系&局部变量窗口，显示当前函数的调用关系和局部变量，方便查看，对分析程序非常有用。

观察窗口：MDK5 提供 2 个观察窗口（下拉选择），该按钮按下，会弹出一个显示变量的窗口，输入你所想要观察的变量/表达式，即可查看其值，是很常用的一个调试窗口。

内存查看窗口：MDK5 提供 4 个内存查看窗口（下拉选择），该按钮按下，会弹出一个内存查看窗口，可以在里面输入你要查看的内存地址，然后观察这一片内存的变化情况。是很常用的一个调试窗口

串口打印窗口：MDK5 提供 4 个串口打印窗口（下拉选择），该按钮按下，会弹出一个类似串口调试助手界面的窗口，用来显示从串口打印出来的内容。

系统分析窗口：该图标下面有 6 个选项（下拉选择），我们一般用第一个，也就是逻辑分析窗口(Logic Analyzer)，点击即可调出该窗口，通过 SETUP 按钮新建一些 IO 口，就可以观察这些 IO 口的电平变化情况，以多种形式显示出来，比较直观。

系统查看窗口：该按钮可以提供各种外设寄存器的查看窗口（通过下拉选择），选择对应外设，即可调出该外设的相关寄存器表，并显示这些寄存器的值，方便查看设置的是否正确。

Debug 工具条上的其他几个按钮用的比较少，我们这里就不介绍了。以上介绍的是比较常用的，当然也不是每次都用得着这么多，具体看你程序调试的时候有没有必要观看这些东西，来决定要不要看。

我们在图 4.3.2.1 的基础上：关闭反汇编窗口（Disassembly）、添加观察窗口 1（Watch1）。然后调节一下窗口位置，然后将全局变量：g_fac_us （在 delay.c 里面定义）加入 Watch1 窗口（方法：双击 Enter expression 添加/直接拖动变量 t 到 Watch1 窗口即可），如图 4.3.2.3 所示：

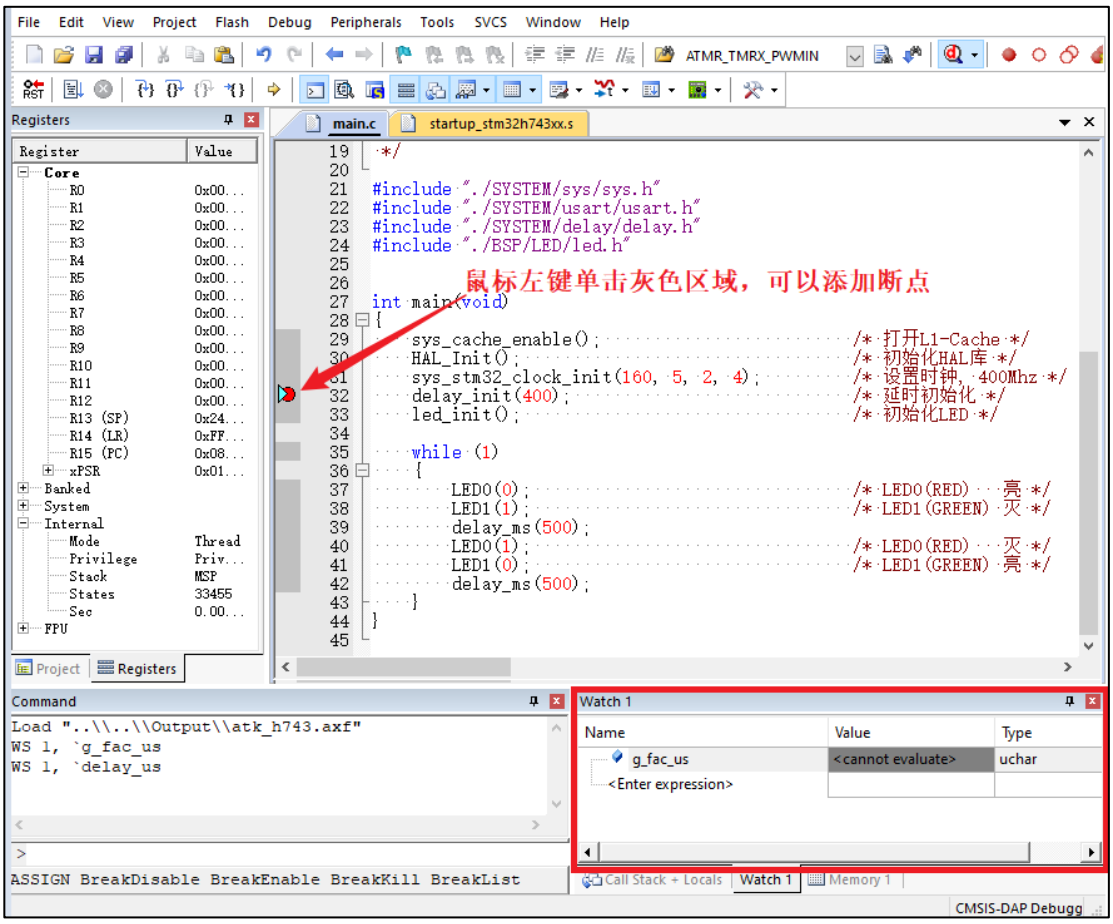


图 4.3.2.3 开始仿真

此时可以看到 Watch1 窗口的 g_fac_us 的值提示：无法计算，我们把鼠标光标放在第 32 行左侧的灰色区域，然后按下鼠标左键，即可放置一个断点（红色的实心点，也可以通过鼠标右键弹出菜单来加入），这样就在 delay_init 函数处放置一个断点，然后点击：，执行到该断点处，然后再点击：，执行进入 delay_init 函数，如图 4.3.2.4 所示：

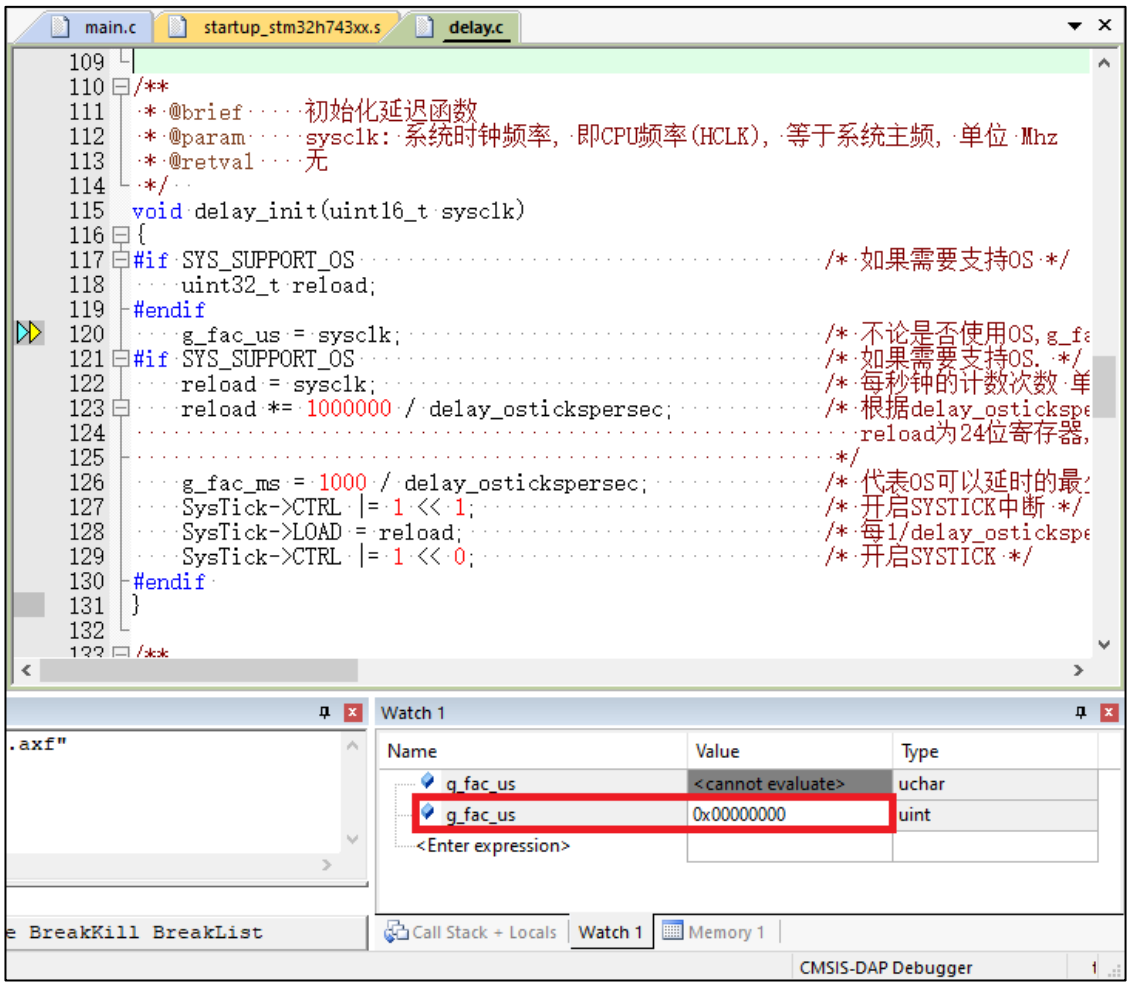


图 4.3.2.4 执行进入 delay_init 函数

此时，可以看到 g_fac_us 的值已经显示出来了，默认是 0（全局变量如果没有赋初值，一般默认都是 0），然后继续单击：{}，单步运行代码到第 131 行，再把鼠标光标放在 sysclk 上停留一会，就可以看到 MDK 自动提示 sysclk（delay_init 的输入参数）的值为：0X0190，即 400，如图 4.3.2.5 所示：

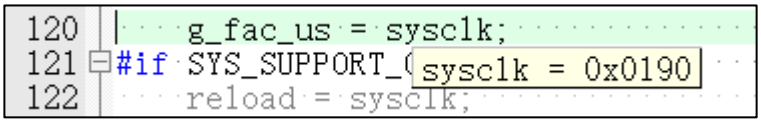


图 4.3.2.5 单步运行到 g_fac_us 赋值

然后就可以在 Watch1 窗口中看到 g_fac_us 的值变成 0X0190 了，如图 4.3.2.6 所示：

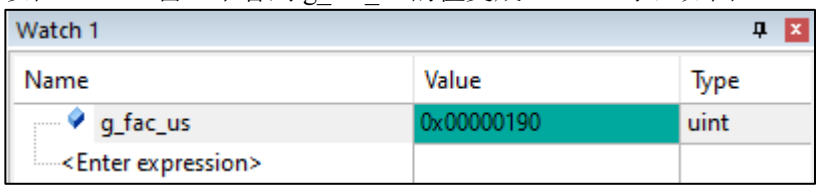


图 4.3.2.6 Watch1 窗口观看 g_fac_us 的值

由此可知，某些全局变量，我们在程序还没运行到其所在文件的时候，MDK 仿真时可能不会显示其值（如提示：cannot evaluate），当我们运行到其所在文件，并实际使用到的时候，此时就会显示其值出来了！

然后，我们再回到 main.c，在第一个 LED1(1)处放置一个断点，运行到断点处，如图 4.3.2.7 所示：

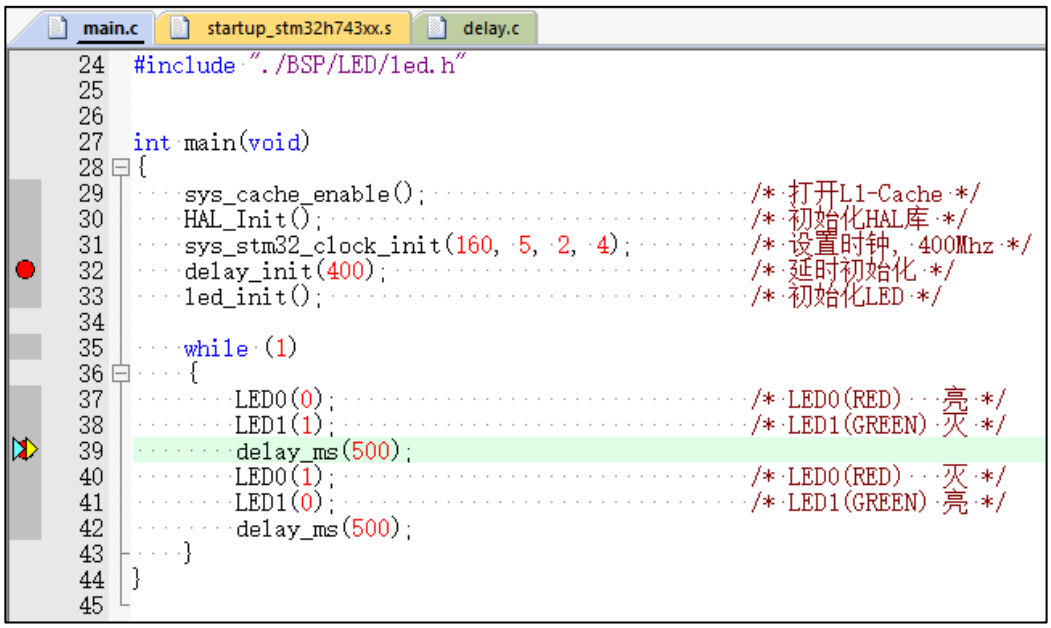


图 4.3.2.7 运行到 LED1(1)代码处

此时，我们点击：{}，执行过这一行代码，就可以看到开发板上的 DS0（红灯）亮起来了，以此继续点击{}，依次可以看到：DS0 灭→DS1 亮→ DS0 亮→ DS1 灭→ DS0 灭→ DS1 亮……，一直循环。这样如果全速运行，就可以看到 DS 0 和 DS 1 交替闪烁。

最后，我们在 delay.c 文件的 delay_us 函数，第十行处设置一个断点，然后运行到断点处，如图 4.3.2.8 所示：

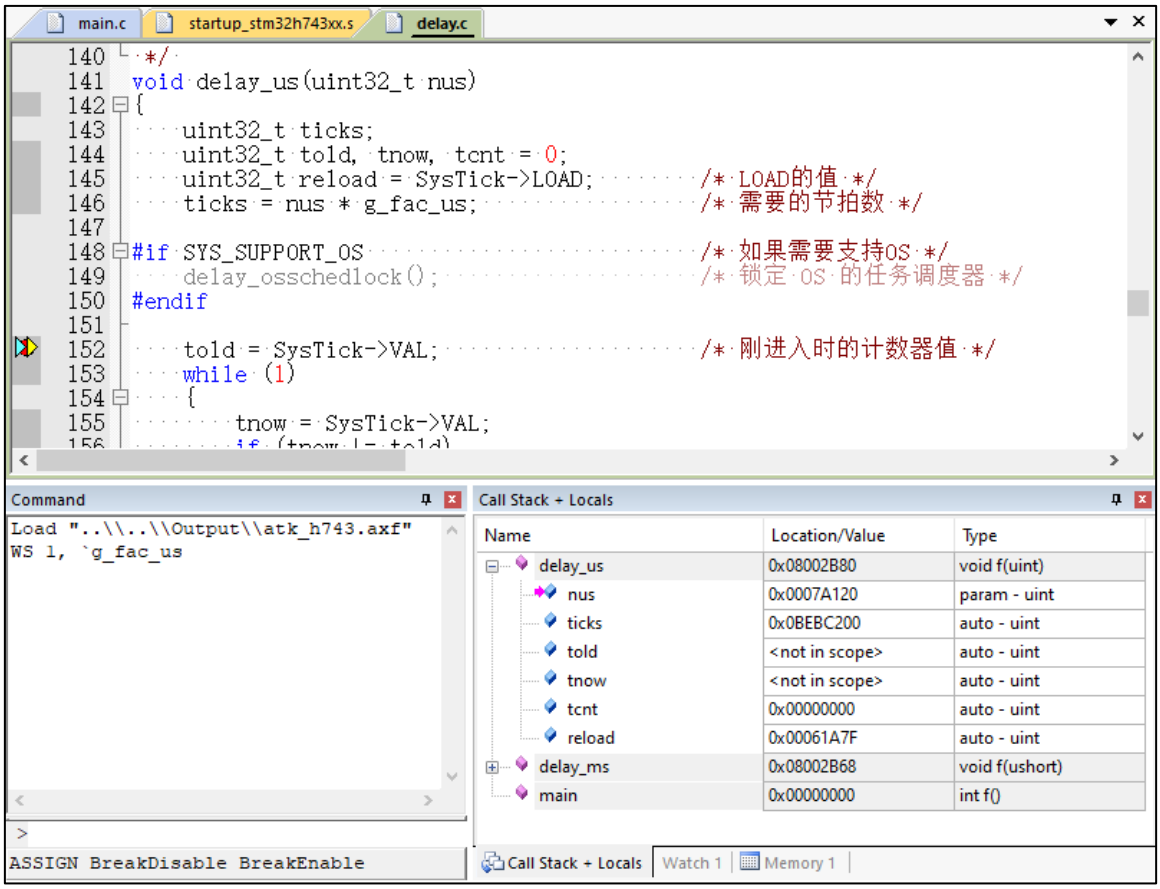


图 4.3.2.8 查看函数调用关系及局部变量

此时，我们可以从 Call Stack＋Locals 窗口看到函数的调用关系，其原则是：**从下往上看，**

即下一个函数调用了上一个函数，因此，其关系为：main 函数调用了 delay_ms 函数，然后 delay_ms 函数调用了 delay_us 函数。这样在一些复杂的代码里面（尤其是第三方代码），可以很容易捋出函数调用关系并查看其局部变量的值，有助于我们分析代码解决问题。

关于 DAP 的仿真调试，我们暂时就讲这么多。

4.3.3 仿真调试注意事项

1、由于 MDK5.23 以后对中文支持不是很好，具体现象是：在仿真的时候，当有断点未清除时点击结束仿真，会出现如图 4.3.3.1 所示的报错：

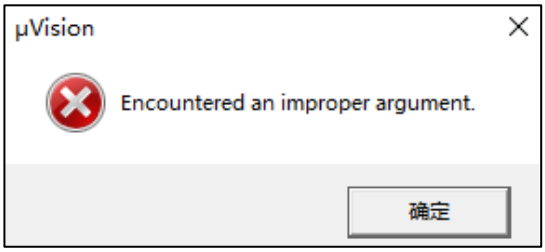


图 4.3.3.1 仿真结束时报错！

此时我们点击确定，是无法关闭 MDK 的，只能到电脑的任务管理器里面强制结束 MDK，才可以将其关闭，比较麻烦。

该错误就是由于 MDK5.23 以后的版本对中文支持不太好导致的，这里提供 2 个解决办法：**a，仿真结束前将所有设置的断点都清除掉**，可以使用 File 工具栏的：按钮，快速清除当前工程的所有断点，然后再结束仿真，就不会报错；**b，将工程路径改浅，并改成全英文路径**（比如，将源码拷贝到：E 盘→Source Code 文件夹下。注意：例程名字一般可以不用改英文，因为只要整个路径不超过 10 个汉字，一般就不会报错了，如果还报错就再减少汉字数量）。通过这两个方法，可以避免仿真结束报错的问题。我们推荐大家使用第二种方法，因为这样就不用每次都全部清除所有断点，下回仿真又得重设的麻烦。

2、关于 STM32 软件仿真，老版本的教程，我们给大家介绍过如何使用 MDK 进行 STM32 软件仿真，由于其限制较多（只支持部分 F1 型号），而且仿真器越来越便宜，硬件仿真更符合实际调试需求，调试效果更好。所以后续我们只介绍硬件仿真，不再推荐大家使用软件仿真了！

3、仿真调试找 bug 是一个软件工程师必备的基本技能。MDK 提供了很多工具和窗口来辅助我们找问题，只要多使用，多练习，肯定就可以把仿真调试学好。这对我们后续的独立开发项目，非常有帮助。因此极力推荐大家多练习使用仿真器查找代码 bug，学会这个基本技能。

4、调试代码不要浅尝辄止，要想尽办法找问题，具体的思路：先根据代码运行的实际现象分析问题，确定最可能出问题的地方，然后在相应的位置放置断点，查看变量，查看寄存器，分析运行状态和预期结果是否一致？从而找到问题原因，解决 bug。特别提醒：一定不要浅尝辄止，很多朋友只跟踪到最上一级函数，就说死机了，不会跟踪进去找问题！所以一定要一层层进入各种函数，越是底层（甚至汇编级别），越好找到问题原因。

4.4 MDK5 使用技巧

本节，我们将向大家介绍 MDK5 软件的一些使用技巧，这些技巧在代码编辑和编写方面会非常有用，希望大家好好掌握，最好实际操作一下，加深印象。

4.4.1 文本美化

文本美化，主要是设置一些关键字、注释、数字等的颜色和字体。如果你刚装 MDK，没进行字体颜色配置，你的界面效果如图 4.4.1.1 所示：

main.c

```
21  #include "./SYSTEM/sys/sys.h"
22  #include "./SYSTEM/delay/delay.h"
23  #include "./SYSTEM/usart/usart.h"
24  #include "./BSP/LED/led.h"
25
26
27  int main(void)
28  {
29      HAL_Init();                             /* 初始化HAL库 */
30      sys_stm32_clock_init(RCC_PLL_MUL9);     /* 设置时钟,72M */
31      delay_init(72);                         /* 初始化延时函数 */
32      led_init();                             /* 配置STM32操作LED相关的寄存器 */
33
34      while(1)
35      {
36          LED0(0);                            /* LED0 亮 */
37          LED1(1);                            /* LED1 灭 */
38          delay_ms(500);
39          LED0(1);                            /* LED0 灭 */
40          LED1(0);                            /* LED1 亮 */
41          delay_ms(500);
42      }
43  }
```

图 4.4.1.1 MDK 默认配色效果

上图是 MDK 默认的设置，可以看到其中的关键字和注释等字体的颜色不是很漂亮，而 MDK 提供了我们自定义字体颜色的功能。我们可以在工具条上点击 (配置对话框)弹出如图 4.4.1.2 所示界面：

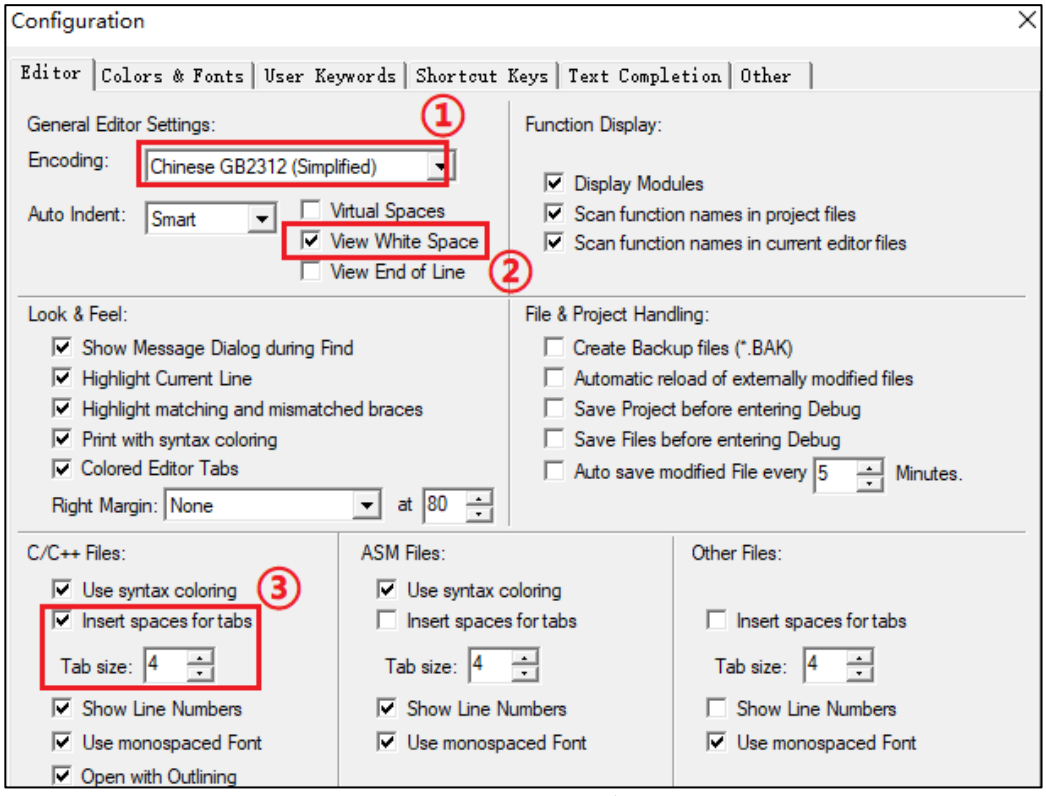


图 4.4.1.2 置对话框

①设置代码编辑器字体使用：Chinese GB2312(Simplified)，以更好的支持中文。

②设置编辑器的空格可见：View White Space，所有空格使用“.”替代，TAB 使用“→”替代，这样可以方便我们对代码进行对齐操作。同时，我们推荐**所有的对齐都用空格**来替代，这样在不同软件之间查看源代码，就不会引起由于 TAB 键大小不一样导致代码不对齐的问题，方便使用不同软件查看和编辑代码。

③设置 C/C++文件，TAB 键的大小为 4 个字符，且字符使用空格替代（Insert spaces for tabs）。这样我们在使用 TAB 键进行代码对齐操作的时候，都会用空格替代，保证不同软件使用代码都可以对齐。

然后，选择：Colors & Fonts 选项卡，在该选项卡内，我们就可以设置自己的代码的字体和颜色了。由于我们使用的是 C 语言，故在 Window 下面选择：C/C++ Editor Files 在右边就可以看到相应的元素了。如图 4.4.1.3 示：

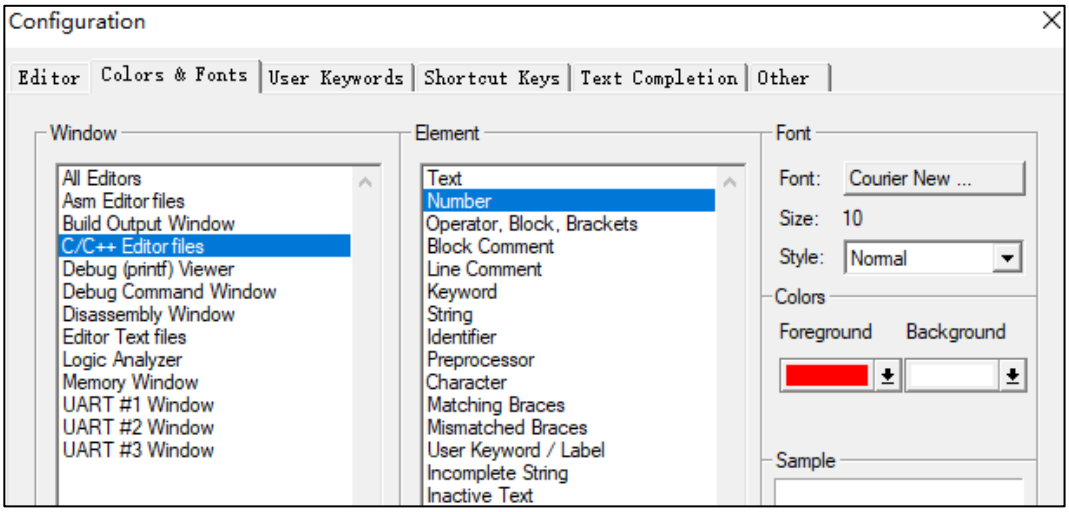


图 4.4.1.3 Colors & Fonts 选项卡

然后点击各个元素（Element）修改为你喜欢的颜色（注意双击，且有时候可能需要设置多次才生效，MDK 的 bug），当然也可以在 Font 栏设置你字体的类型，以及字体的大小等。

然后，点击 User Keywords 选项卡，设置用户定义关键字，以便用户自定义关键字也显示对应的颜色（对应图 4.4.1.3 中的 User Keyword/Lable 颜色）。在 User Keywords 选项卡对话框下面输入你自己定义的关键字，如图 4.4.1.4 示：

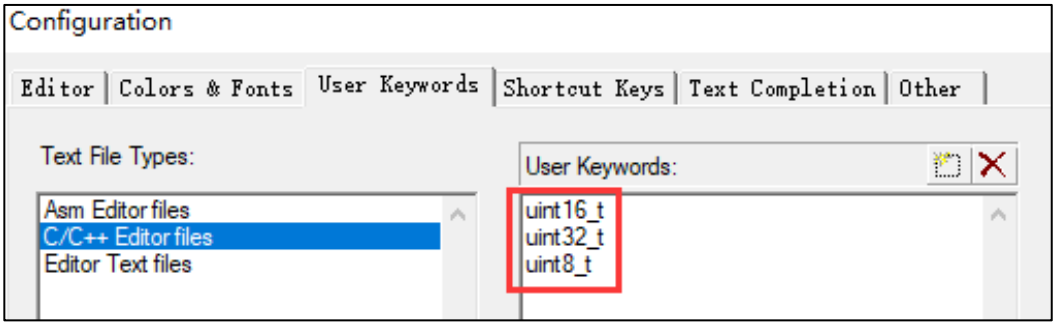


图 4.4.1.4 用户自定义关键字

这里我们设置了 uint8_t、uint16_t 和 uint32_t 等三个用户自定义关键字，相当于 unsigned char、unsigned short 和 unsigned int。如果你还有其他自定义关键字，在这里添加即可。设置成之后，点击 OK，就可以在主界面看到你所修改后的结果，例如我修改后的代码显示效果如图 4.4.1.5 示：

```
#include "./SYSTEM/sys/sys.h"
#include "./SYSTEM/usart/usart.h"
#include "./SYSTEM/delay/delay.h"
#include "./BSP/LED/led.h"


int main(void)
{
    sys_cache_enable();                     /* 打开L1-Cache */
    HAL_Init();                             /* 初始化HAL库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    led_init();                             /* 初始化LED */

    while (1)
    {
        LED0(0);                            /* LED0(RED)  亮 */
        LED1(1);                            /* LED1(GREEN) 灭 */
        delay_ms(500);
        LED0(1);                            /* LED0(RED)  灭 */
        LED1(0);                            /* LED1(GREEN) 亮 */
        delay_ms(500);
    }
}
```

图 4.4.1.5 设置完后显示效果

这就比开始的效果好看一些了。字体大小，则可以直接按住：ctrl+鼠标滚轮，进行放大或者缩小，或者也可以在刚刚的配置界面设置字体大小。

同时，上图中可以看到空白处有很淡的一些……显示，这就是我们勾选了 View White Space 选项后体现出来的效果，可以方便我们对代码进行规范对齐整理。一开始看的时候可能有点不习惯，看多了就习惯了，大家慢慢适应就好了。

其实在这个编辑配置对话框里，还可以对其他很多功能进行设置，比如动态语法检测等，我们将 4.4.2 节介绍。

4.4.2 语法检测&代码提示

MDK4.70 以上的版本，新增了代码提示与动态语法检测功能，使得 MDK 的编辑器越来越好用了，这里我们简单说一下如何设置，同样，点击 🔧，打开配置对话框，选择 Text Completion 选项卡，如图 4.4.2.1 所示：

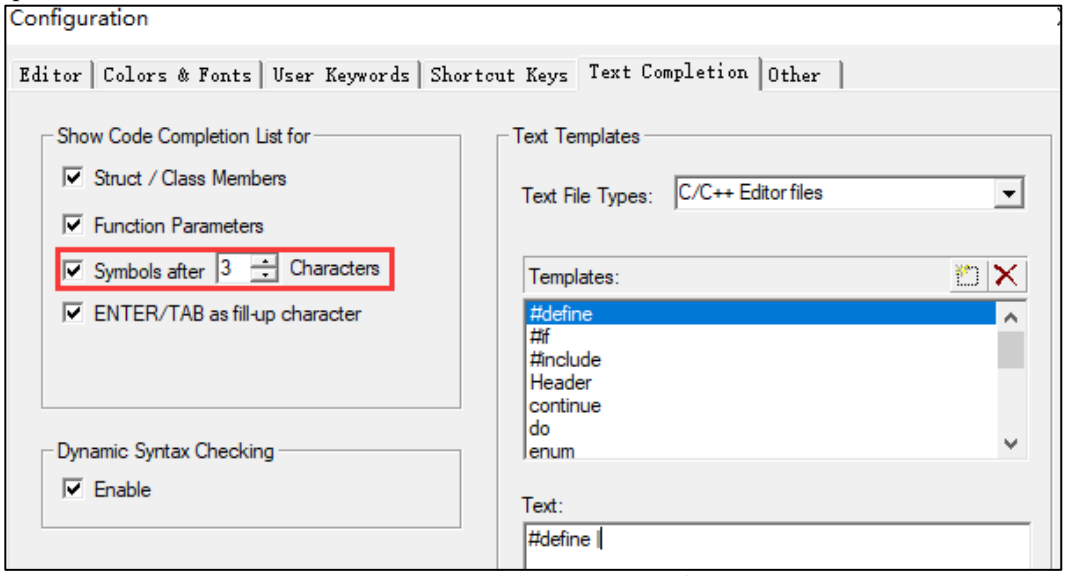


图 4.4.2.1 Text Completion 选项卡设置

Strut / Class Members，用于开启结构体/类成员提示功能。

Function Parameters，用于开启函数参数提示功能。

Symbols after xx characters，用于开启代码提示功能，即在输入多少个字符以后，提示匹配的内容（比如函数名字、结构体名字、变量名字等），这里默认设置 3 个字符以后，就开始提示。如图 4.4.2.2 所示：

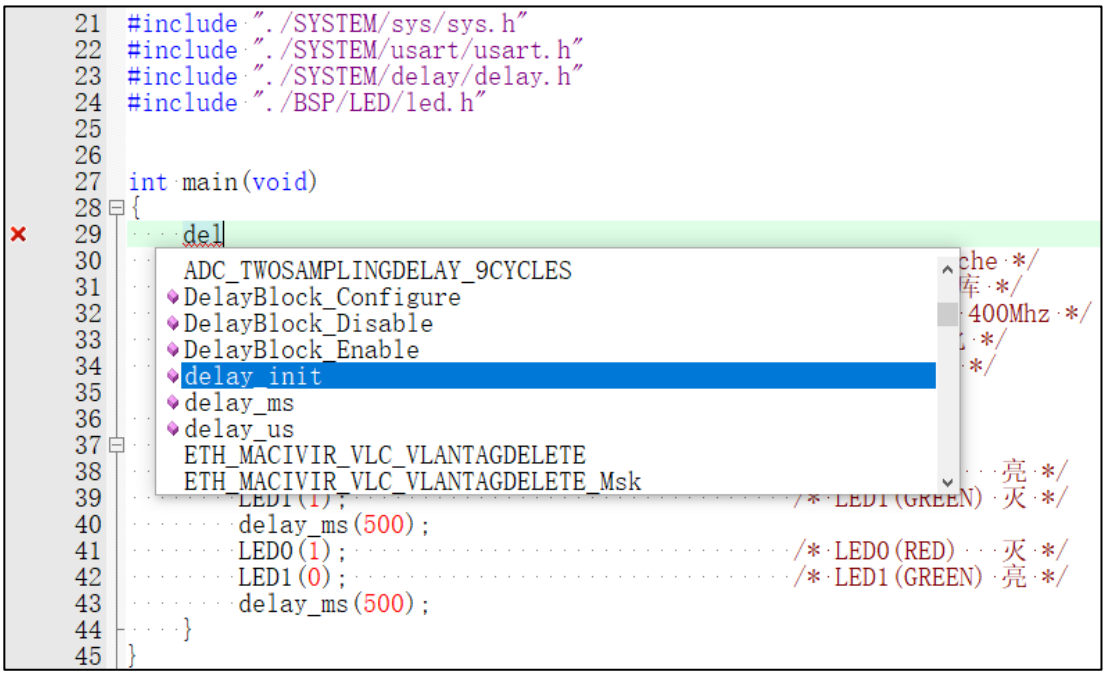


图 4.4.2.2 代码提示

ENTER/TAB as fill-up character，使用回车和 TAB 键填充字符。

Dynamic Syntax Checking，则用于开启动态语法检测，比如编写的代码存在语法错误的时候，会在对应行前面出现✖图标，如出现警告，则会出现⚠图标，将鼠标光标放图标上面，则会提示产生的错误/警告的原因，如图 4.4.2.3 所示：

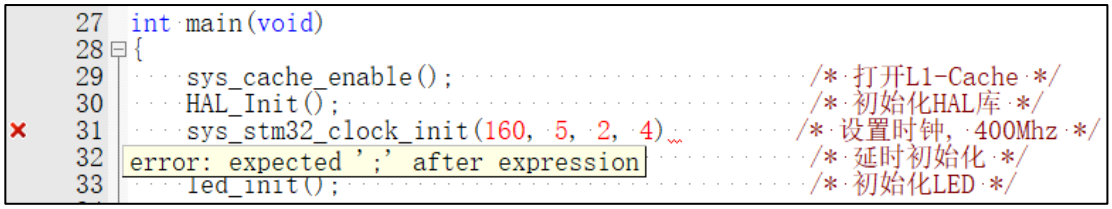


图 4.4.2.3 语法动态检测功能

这几个功能，对我们编写代码很有帮助，可以加快代码编写速度，并且及时发现各种问题。不过这里要提醒大家，语法动态检测这个功能，**有的时候会误报（比如 sys.c 里面，就有误报），大家可以不用理会，只要能编译通过（0 错误，0 警告），这样的语法误报，一般直接忽略即可。**

4.4.3 代码编辑技巧

这里给大家介绍几个我常用的技巧，这些小技巧能给我们的代码编辑带来很大的方便，相信对你的代码编写一定会有所帮助。

1. TAB 键的妙用

首先要介绍的就是 TAB 键的使用，这个键在很多编译器里面都是用来空位的，每按一下移空几个位。如果你是经常编写程序的对这个键一定再熟悉不过了。但是 MDK 的 TAB 键和一般编译器的 TAB 键有不同的地方，和 C++的 TAB 键差不多。MDK 的 TAB 键支持块操作。也就是可以让一片代码整体右移固定的几个位，也可以通过 SHIFT+TAB 键整体左移固定的几个

位。

假设我们前面的串口 1 中断回调函数如图 4.4.3.1 所示：

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
if (huart->Instance == USART_UX)                    /*如果是串口1*/
{
if ((g_usart_rx_sta & 0x8000) == 0)                 /*接收未完成*/
{
if (g_usart_rx_sta & 0x4000)                        /*接收到了0x0d*/
{
if (aRxBuffer[0] != 0x0a)
{
g_usart_rx_sta = 0;                                 /*接收错误,重新开始*/
}
else
{
g_usart_rx_sta |= 0x8000;                           /*接收完成了*/
}
}
else                                                /*还没收到0X0D*/
{
if (aRxBuffer[0] == 0x0d)
g_usart_rx_sta |= 0x4000;
else
{
g_usart_rx_buf[g_usart_rx_sta & 0X3FFF] = aRxBuffer[0];
g_usart_rx_sta++;

if (g_usart_rx_sta > (USART_REC_LEN - 1))
{
g_usart_rx_sta = 0;                                 /*接收数据错误,重新开始接收      */
}
}
}
}
}
}
```

图 4.4.3.1 头大的代码

上图的代码很不规范，这还只是短短的 30 来行代码，如果你的代码有几千行，全部是这个样子，不头大才怪。这时我们就可以通过 TAB 键的妙用来快速修改为比较规范的代码格式。

选中一块然后按 TAB 键，你可以看到整块代码都跟着右移了一定距离，如图 4.4.3.2 所示：

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART_UX)                    /*如果是串口1*/
    {
    if ((g_usart_rx_sta & 0x8000) == 0)                 /*接收未完成*/
    {
    if (g_usart_rx_sta & 0x4000)                        /*接收到了0x0d*/
    {
    if (aRxBuffer[0] != 0x0a)
    {
    g_usart_rx_sta = 0;                                 /*接收错误,重新开始*/
    }
    else
    {
    g_usart_rx_sta |= 0x8000;                           /*接收完成了*/
    }
    }
    else                                                /*还没收到0X0D*/
    {
    if (aRxBuffer[0] == 0x0d)
    g_usart_rx_sta |= 0x4000;
    else
    {
    g_usart_rx_buf[g_usart_rx_sta & 0X3FFF] = aRxBuffer[0];
    g_usart_rx_sta++;

    if (g_usart_rx_sta > (USART_REC_LEN - 1))
    {
    g_usart_rx_sta = 0;                                 /*接收数据错误,重新开始接收      */
    }
    }
    }
    }
    }
}
```

图 4.4.3.2 代码整体偏移

接下来我们就是要多选几次，然后多按几次 TAB 键就可以达到迅速使代码规范化的目的，最终效果如图 4.4.3.3 所示

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if (huart->Instance == USART_UX)                    /*如果是串口1*/
    {
        if ((g_usart_rx_sta & 0x8000) == 0)             /*接收未完成*/
        {
            if (g_usart_rx_sta & 0x4000)                /*接收到了0x0d*/
            {
                if (aRxBuffer[0] != 0x0a)
                {
                    g_usart_rx_sta = 0;                 /*接收错误,重新开始*/
                }
                else
                {
                    g_usart_rx_sta |= 0x8000;           /*接收完成了*/
                }
            }
            else                                        /*还没收到0X0D*/
            {
                if (aRxBuffer[0] == 0x0d)
                    g_usart_rx_sta |= 0x4000;
                else
                {
                    g_usart_rx_buf[g_usart_rx_sta & 0X3FFF] = aRxBuffer[0];
                    g_usart_rx_sta++;

                    if (g_usart_rx_sta > (USART_REC_LEN - 1))
                    {
                        g_usart_rx_sta = 0;             /*接收数据错误,重新开始接收      */
                    }
                }
            }
        }
    }
}
```

图 4.4.3.3 修改后的代码

图 4.4.3.3 中的代码相对于图 4.4.3.1 中的要好看多了，经过这样的整理之后，整个代码一下就变得有条理多了，看起来很舒服。

2. 快速定位函数/变量被定义的地方

上一节，我们介绍了 TAB 键的功能，接下来我们介绍一下如何快速查看一个函数或者变量所定义的地方。

大家在调试代码或编写代码的时候，一定有想看看某个函数是在那个地方定义的，具体里面的内容是怎么样的，也可能想看看某个变量或数组是在哪个地方定义的等。尤其在调试代码或者看别人代码的时候，如果编译器没有快速定位的功能的时候，你只能慢慢的自己找，代码量比较少还好，如果代码量一大，那就郁闷了，有时候要花很久的时间来找这个函数到底在哪里。型号 MDK 提供了这样的快速定位的功能。只要你把光标放到这个函数/变量（xxx）的上面（xxx 为你想要查看的函数或变量的名字），然后右键，弹出如图 4.4.3.4 所示的菜单栏：

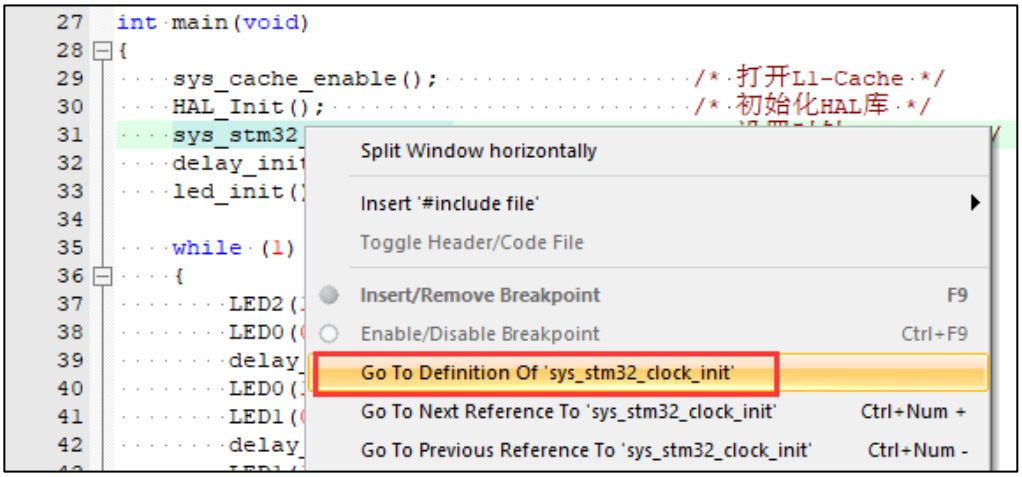


图 4.4.3.4 快速定位

在图 4.4.3.4 中，我们找到 Go to Definition Of‘sys_stm32_clock_init’ 这个地方，然后单击左键就可以快速跳到 sys_stm32_clock_init 函数的定义处（**注意要先在 Options for Target 的 Output 选项卡里面勾选 Browse Information 选项，再编译，再定位，否则无法定位！**）。如图 4.4.3.5 所示：

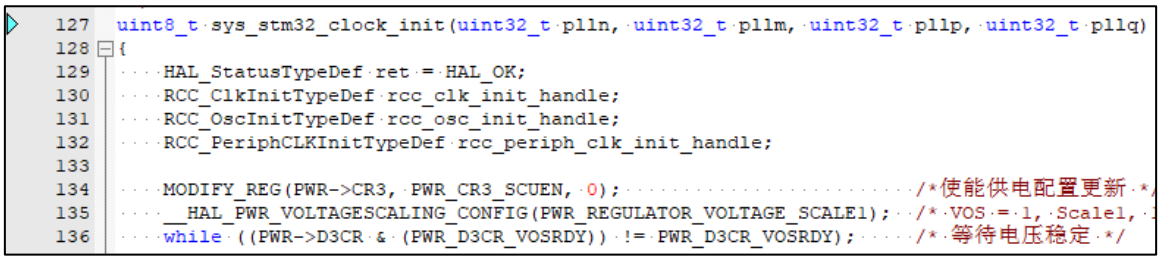


图 4.4.3.5 定位结果

对于变量，我们也可以按这样的操作快速来定位这个变量被定义的地方，大大缩短了你查找代码的时间。

很多时候，我们利用 Go to Definition 看完函数/变量的定义后，又想返回之前的代码继续看，此时我们可以通过 IDE 上的按钮（Back to previous position）快速的返回之前的位置，这个按钮非常好用！

3. 快速注释与快速消注释

接下来，我们介绍一下快速注释与快速消注释的方法。在调试代码的时候，你可能会想注释某一片的代码，来看看执行的情况，MDK 提供了这样的快速注释/消注释块代码的功能。也是通过右键实现的。这个操作比较简单，就是先选中你要注释的代码区，然后右键，选择 Advanced→Comment Selection 就可以了。

以 led_init 函数为例，比如我要注释掉下图中所选中区域的代码，如图 4.4.3.6 所示：

```
28 /**
29  * @brief       初始化LED相关IO口, 并使能时钟
30  * @param       无
31  * @retval      无
32  */
33 void led_init(void)
34 {
35     GPIO_InitTypeDef gpio_init_struct;
36     LED0_GPIO_CLK_ENABLE();                                 /* LED0时钟使能 */
37     LED1_GPIO_CLK_ENABLE();                                 /* LED1时钟使能 */
38 
39     gpio_init_struct.Pin = LED0_GPIO_PIN;                   /* LED0引脚 */
40     gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;            /* 推挽输出 */
41     gpio_init_struct.Pull = GPIO_PULLUP;                    /* 上拉 */
42     
43     gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;          /* 高速 */
44     HAL_GPIO_Init(LED0_GPIO_PORT, &gpio_init_struct);       /* 初始化LED0引脚 */
45 
46     gpio_init_struct.Pin = LED1_GPIO_PIN;                   /* LED1引脚 */
47     HAL_GPIO_Init(LED1_GPIO_PORT, &gpio_init_struct);       /* 初始化LED1引脚 */
48     
49 
50     LED0(1);                                                /* 关闭 LED0 */
51     LED1(1);                                                /* 关闭 LED1 */
52 }
```

图 4.4.3.6 选中要注释的区域

我们只要在选中了之后，选择右键，再选择 Advanced→Comment Selection 就可以把这段代码注释掉了。执行这个操作以后的结果如图 4.4.3.7 所示：

```
28 /**
29  * @brief       初始化LED相关IO口, 并使能时钟
30  * @param       无
31  * @retval      无
32  */
33 void led_init(void)
34 {
35 //    GPIO_InitTypeDef gpio_init_struct;
36 //    LED0_GPIO_CLK_ENABLE();                                 /* LED0时钟使能 */
37 //    LED1_GPIO_CLK_ENABLE();                                 /* LED1时钟使能 */
38 
39 //    gpio_init_struct.Pin = LED0_GPIO_PIN;                   /* LED0引脚 */
40 //    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;            /* 推挽输出 */
41 //    gpio_init_struct.Pull = GPIO_PULLUP;                    /* 上拉 */
42 //    
43 //    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;          /* 高速 */
44 //    HAL_GPIO_Init(LED0_GPIO_PORT, &gpio_init_struct);       /* 初始化LED0引脚 */
45 
46 //    gpio_init_struct.Pin = LED1_GPIO_PIN;                   /* LED1引脚 */
47 //    HAL_GPIO_Init(LED1_GPIO_PORT, &gpio_init_struct);       /* 初始化LED1引脚 */
48 //    
49 
50 //    LED0(1);                                                /* 关闭 LED0 */
51 //    LED1(1);                                                /* 关闭 LED1 */
52 }
```

图 4.4.3.7 注释完毕

这样就快速的注释掉了一片代码，而在某些时候，我们又希望这段注释的代码能快速的取消注释，MDK 也提供了这个功能。与注释类似，先选中被注释掉的地方，然后通过右键→Advanced，不过这里选择的是 Uncomment Selection。

4.4.4 其他小技巧

除了前面介绍的几个比较常用的技巧，这里还介绍几个其他的小技巧，希望能让你的代码编写如虎添翼。

第一个是快速打开头文件。在将光标放到要打开的引用头文件上，然后右键选择 Open

Document“XXX”，就可以快速打开这个文件了（XXX 是你要打开的头文件名字）。如图 4.4.4.1 所示：

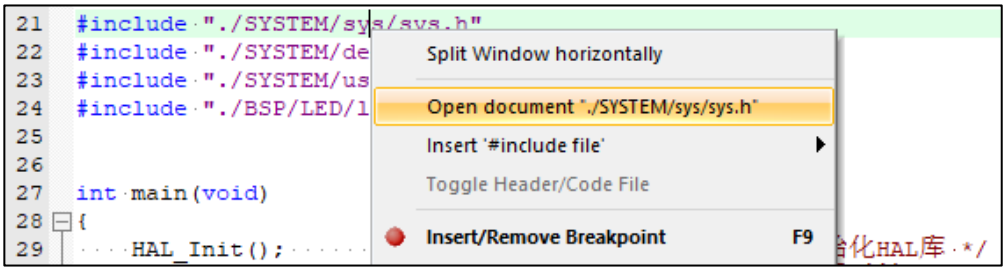


图 4.4.4.1 快速打开头文件

第二个小技巧是查找替换功能。这个和 WORD 等很多文档操作的替换功能是差不多的，在 MDK 里面查找替换的快捷键是“CTRL+H”，只要你按下该按钮就会调出如图 4.4.4.2 所示界面：

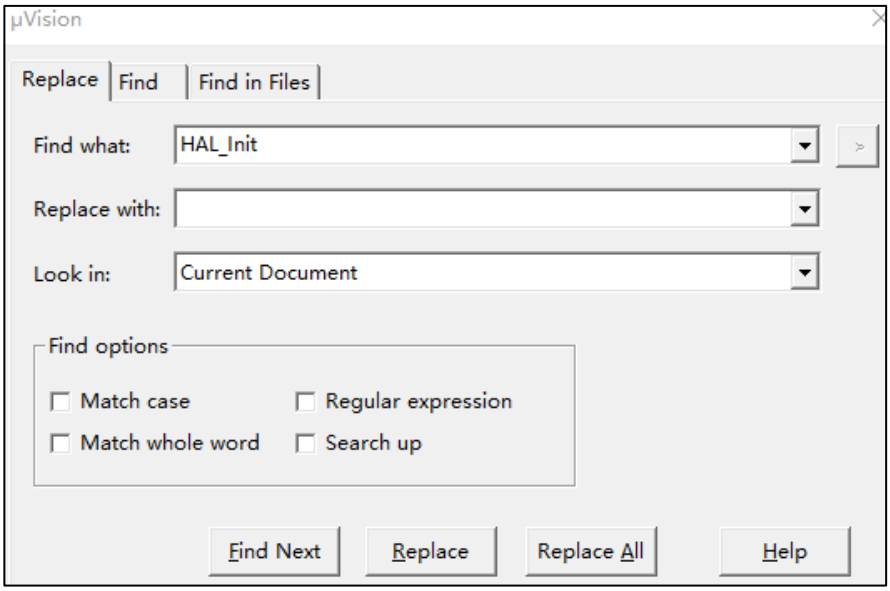


图 4.4.4.2 替换文本

这个替换的功能在有的时候是很有用的，它的用法与其他编辑工具或编译器的差不多，相信各位都不陌生了，这里就不啰嗦了。

第三个小技巧是跨文件查找功能，先双击你要找的函数/变量名（这里以系统时钟初始化函数：sys_stm32_clock_init 为例），然后再点击 IDE 上面的，弹出如图 4.4.4.3 所示对话框：

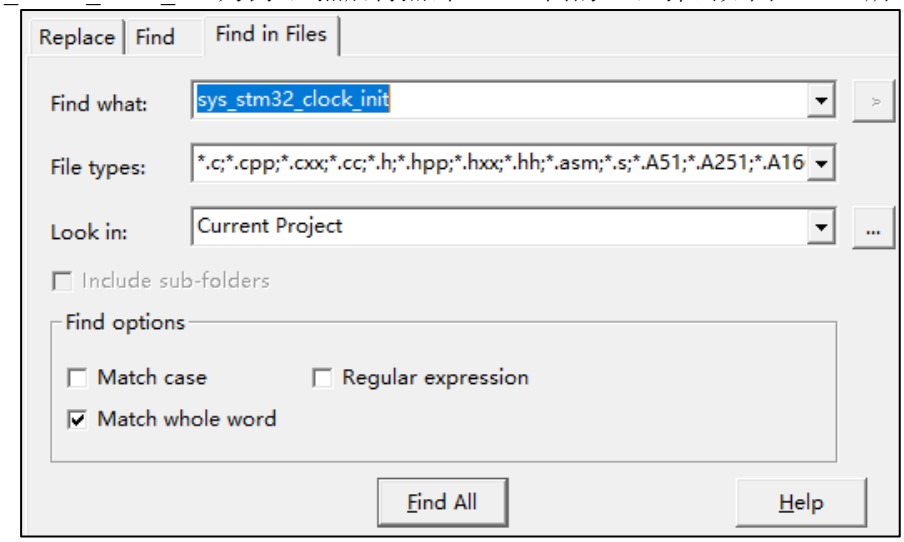


图 4.4.4.3 跨文件查找

点击 Find All，MDK 就会帮你找出所有含有 sys_stm32_clock_init 字段的文件并列出其所在位置，如图 4.4.4.4 所示：

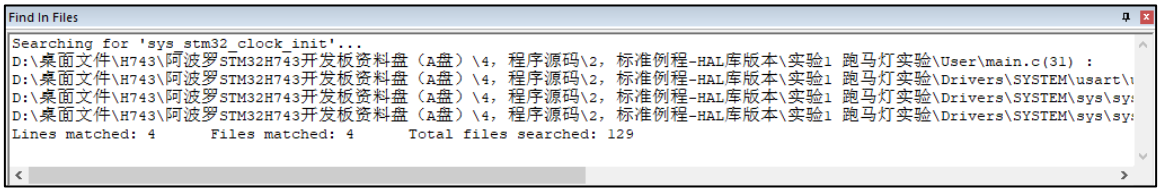


图 4.4.4.4 查找结果

该方法可以很方便的查找各种函数/变量，而且可以限定搜索范围（比如只查找.c 文件和.h 文件等），是非常实用的一个技巧。

第五章 STM32 基础知识入门

本章，我们着重介绍 STM32 的一些基础知识，让大家对 STM32 开发有一个初步的了解，为后面 STM32 的学习做铺垫，方便后面的学习。本章内容大家第一次看的时候可以只了解一个大概，后面需要用到这方面的知识的时候再回过头来仔细看看。

本章将分为如下几个小节：

5.1 C 语言基础知识复习
5.2 寄存器基础知识
5.3 STM32H743 系统架构
5.4 存储器映射
5.5 寄存器映射

5.1 C 语言基础知识复习

本节我们给大家介绍一下 C 语言基础知识，对于 C 语言比较熟练的读者，可以跳过此节，对于基础比较薄弱的读者，建议好好学习一下本节内容。

由于 C 语言博大精深，不可能我们一小节就全讲明白了，所以本节我们只是复习 STM32 开发时常用的几个 C 语言知识点，以便大家的更好的学习并编写 STM32 代码。

5.1.1 位操作

C 语言位操作相信学过 C 语言的人都不陌生了，简而言之，就是对基本类型变量可以在位级别进行操作。这节的内容很多朋友都应该很熟练了，我这里也就点到为止，不深入探讨。下面我们先讲解几种位操作符，然后讲解位操作使用技巧。C 语言支持如下 6 种位操作：

运算符	含义	运算符	含义
&	按位与	~	按位取反
|	按位或	<<	左移
^	按位异或	>>	右移

表 5.1.1.1 六种位操作

这些与或非，取反，异或，右移，左移这些到底怎么回事，这里我们就不多做详细，相信大家学 C 语言的时候都学习过了。如果不懂的话，可以百度一下，非常多的知识讲解这些操作符。下面我们想着重讲解位操作在单片机开发中的一些实用技巧。

1，在不改变其他位的值的状况下，对某几个位进行设值。

这个场景在单片机开发中经常使用，方法就是先对需要设置的位用&操作符进行清零操作，然后用 | 操作符设值。比如我要改变 GPIOA->ODR 的状态,可以先对寄存器的值进行&清零操作：

```
GPIOA->ODR &= 0XFF0F;     /* 将第 4-7 位清 0 */
```

然后再与需要设置的值进行|或运算：

```
GPIOA->ODR |= 0X0040;     /* 设置 bit6 的值为 1，不改变其他位的值 */
```

2，移位操作提高代码的可读性。

移位操作在单片机开发中非常重要，我们来看看下面一行代码：

```
SysTick->CTRL |= 1 << 1;
```

这个操作就是将 CTRL 寄存器的第 1 位（从 0 开始算起）设置为 1，为什么要通过左移而不是直接设置一个固定的值呢？其实这是为了提高代码的可读性以及可重用性。这行代码可以很直观明了的知道，是将第 1 位设置为 1。如果写成：

```
SysTick->CTRL |= 0X0002;
```

这个虽然也能实现同样的效果，但是可读性稍差，而且修改也比较麻烦。

3，~按位取反操作使用技巧

按位取反在设置寄存器的时候经常被使用，常用于清除某一个/某几个位。我们来看看下

面一行代码：

```
SysTick->CTRL &= ~(1 << 0) ;     /* 关闭 SYSTICK */
```

该代码可以解读为 仅设置 CTRL 寄存器的第 0 位（最低位）为 0，其他位的值保持不变。同样我们也不使用按位取反，将代码写成：

```
SysTick->CTRL &= 0XFFFFFFFE;          /* 关闭 SYSTICK */
```

可见前者的可读性，及可维护性都要比后者好很多。

4，^按位异或操作使用技巧

该功能非常适合用于控制某个位翻转，常见的应用场景就是控制 LED 闪烁，如：

```
GPIOB->ODR ^= 1 << 5;
```

执行一次该代码，就会使 PB5 的输出状态翻转一次，如果我们的 LED 接在 PB5 上，就可以看到 LED 闪烁了。

5.1.2 define 宏定义

define 是 C 语言中的预处理命令，它用于宏定义，可以提高源代码的可读性，为编程提供方便。常见的格式：

```
#define          标识符          字符串
```

“标识符”为所定义的宏名。“字符串”可以是常数、表达式、格式串等。例如：

```
#define HSE_VALUE     ((uint32_t)25000000)
```

定义标识符 HSE_VALUE 的值为 25000000。这样我们就可以在代码中直接使用标识符 HSE_VALUE，而不用直接使用常量 25000000，同时也很方便我们修改 HSE_VALUE 的值。

至于 define 宏定义的其他一些知识，比如宏定义带参数这里我们就不多讲解。

5.1.3 ifdef 条件编译

单片机程序开发过程中，经常会遇到一种情况，当满足某条件时对一组语句进行编译，而当条件不满足时则编译另一组语句。条件编译命令最常见的形式为：

```
#ifdef 标识符
    程序段 1
#else
    程序段 2
#endif
```

它的作用是：当标识符已经被定义过(一般是用#define 命令定义)，则对程序段 1 进行编译，否则编译程序段 2。 其中#else 部分也可以没有，即：

```
#ifdef
     程序段 1
#endif
```

条件编译在 MDK 里面是用得很多，在 stm32h7xx_hal_conf.h 这个头文件中经常会看到这样的语句：

```
#ifdef  HAL_GPIO_MODULE_ENABLED
  #include "stm32h7xx_hal_gpio.h"
#endif
```

这段代码的作用是判断宏定义标识符 HAL_GPIO_MODULE_ENABLED 是否被定义，如果被定义了，那么就引入头文件 stm32h7xx_hal_gpio.h。条件编译也是 C 语言的基础知识，这里也就点到为止吧。

5.1.4 extern 外部申明

C 语言中 extern 可以置于变量或者函数前，以表示变量或者函数的定义在别的文件中，提示编译器遇到此变量和函数时在其他模块中寻找其定义。这里面要注意，对于 extern 申明变量可以多次，但定义只有一次。在我们的代码中你会看到看到这样的语句：

```
extern uint16_t g_usart_rx_sta;
```

这个语句是申明 g_usart_rx_sta 变量在其他文件中已经定义了，在这里要使用到。所以，你肯定可以找到在某个地方有变量定义的语句：

```
uint16_t g_usart_rx_sta;
```

extern 的使用比较简单，但是也会经常用到，需要掌握。

5.1.5 typedef 类型别名

typedef 用于为现有类型创建一个新的名字，或称为类型别名，用来简化变量的定义。typedef 在 MDK 用得最多的就是定义结构体的类型别名和枚举类型了。

```
struct _GPIO
{
    __IO uint32_t MODER;
    __IO uint32_t OTYPER;
    …
};
```

定义了一个结构体 GPIO，这样我们定义结构体变量的方式为：

```
struct  _GPIO  gpiox;    /* 定义结构体变量 gpiox */
```

但是这样很繁琐，MDK 中有很多这样的结构体变量需要定义。这里我们可以为结体定义一个别名 GPIO_TypeDef，这样我们就可以在其他地方通过别名 GPIO_TypeDef 来定义结构体变量了，方法如下：

```
typedef struct
{
    __IO uint32_t MODER;
    __IO uint32_t OTYPER;
    …
} GPIO_TypeDef;
```

Typedef 为结构体定义一个别名 GPIO_TypeDef，这样我们可以通过 GPIO_TypeDef 来定义结构体变量：

```
GPIO_TypeDef gpiox;
```

这里的 GPIO_TypeDef 就跟 struct _GPIO 是等同的作用了，但是 GPIO_TypeDef 使用起来方便很多。

5.1.6 结构体

经常很多用户提到，他们对结构体使用不是很熟悉，但是 MDK 中太多地方使用结构体以及结构体指针，这让他们一下子摸不着头脑，学习 STM32 的积极性大大降低，其实结构体并不是那么复杂，这里我们稍微提一下结构体的一些知识，还有一些知识我们会在下面的“寄存器映射”中讲到一些。

声明结构体类型：

```
struct 结构体名
{
    成员列表;
}变量名列表;
```

例如：

```
struct U_TYPE
{
    int BaudRate
    int WordLength;
}usart1, usart2;
```

在结构体申明的时候可以定义变量，也可以申明之后定义，方法是：

```
struct  结构体名字   结构体变量列表 ;
```

例如：

```
struct U_TYPE usart1,usart2;
```

结构体成员变量的引用方法是：

```
结构体变量名字.成员名
```

比如要引用 usart1 的成员 BaudRate，方法是：usart1.BaudRate;结构体指针变量定义也是一样的，跟其他变量没有啥区别。

例如：

```
struct U_TYPE *usart3;    /* 定义结构体指针变量 usart3 */
```

结构体指针成员变量引用方法是通过“->”符号实现，比如要访问 usart3 结构体指针指向的结构体的成员变量 BaudRate,方法是：

```
usart3->BaudRate;
```

上面讲解了结构体和结构体指针的一些知识，其他的什么初始化这里就不多讲解了。讲到这里，有人会问，结构体到底有什么作用呢？为什么要使用结构体呢？下面我们将简单的通过一个实例回答一下这个问题。

在我们单片机程序开发过程中，经常会遇到要初始化一个外设比如串口，它的初始化状态是由几个属性来决定的，比如串口号，波特率，极性，以及模式。对于这种情况，在我们没有学习结构体的时候，我们一般的方法是：

```
void usart_init(uint8_t usartx, uiut32_t BaudRate, uint32_t Parity,
                uint32_t Mode);
```

这种方式是有效的同时在一定场合是可取的。但是试想，如果有一天，我们希望往这个函数里面再传入一个/几个参数，那么势必我们需要修改这个函数的定义，重新加入新的入口参数，随着开发不断的增多，那么是不是我们就要不断的修改函数的定义呢？这是不是给我们开发带来很多的麻烦？那又怎样解决这种情况呢？

我们使用结构体参数，就可以在不改变入口参数的情况下，只需要改变结构体的成员变量，就可以达到改变入口参数的目的。

结构体就是将多个变量组合为一个有机的整体，上面的函数，usartx,BaudRate,Parity, Mode 等这些参数，他们对于串口而言，是一个有机整体，都是来设置串口参数的，所以我们可以将他们通过定义一个结构体来组合在一个。MDK 中是这样定义的：

```
typedef struct
{
    uint32_t BaudRate;
    uint32_t WordLength;
    uint32_t StopBits;
    uint32_t Parity;
    uint32_t Mode;
    uint32_t HwFlowCtl;
    uint32_t OverSampling;
} UART_InitTypeDef;
```

这样，我们在初始化串口的时候入口参数就可以是 USART_InitTypeDef 类型的变量或者指针变量了，于是我们可以改为：

```
void usart_init(UART_InitTypeDef *huart);
```

这样，任何时候，我们只需要修改结构体成员变量，往结构体中间加入新的成员变量，而不需要修改函数定义就可以达到修改入口参数同样的目的了。这样的好处是不用修改任何函数定义就可以达到增加变量的目的。

理解了结构体在这个例子中间的作用吗？在以后的开发过程中，如果你的变量定义过多，如果某几个变量是用来描述某一个对象，你可以考虑将这些变量定义在结构体中，这样也许可以提高你的代码的可读性。

使用结构体组合参数，可以提高代码的可读性，不会觉得变量定义混乱。当然结构体的作用就远远不止这个了，同时，MDK 中用结构体来定义外设也不仅仅只是这个作用，这里我们只是举一个例子，通过最常用的场景，让大家理解结构体的一个作用而已。后面一节我们还会讲解结构体的一些其他知识。

5.1.7 指针

指针是一个值指向地址的变量（或常量），其本质是指向一个地址，从而可以访问一片内存区域。在编写 STM32 代码的时候，或多或少都要用到指针，它可以使不同代码共享同一片内存数据，也可以用作复杂的链接性的数据结构的构建，比如链表，链式二叉树等，而且，有些地方必须使用指针才能实现，比如内存管理等。

申明指针我们一般以 p 开头，如：

```
char * p_str = "This is a test!";
```

这样，我们就申明了一个 p_str 的指针，它指向 This is a test!这个字符串的首地址。我们编写如下代码：

```
int main(void)
{
    uint8_t *p_num = &temp;                    /* 定义指针 p_num，指向 temp 的地址 */
    sys_cache_enable();                                /* 打开 L1-Cache */
    HAL_Init();                                        /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);                /* 设置时钟,400Mhz */
    delay_init(400);                                   /* 延时初始化 */
    usart_init(115200);                                /* 串口初始化为 115200 */
    printf("temp:0X%X\r\n", temp);                     /* 打印 temp 的值 */
    printf("*p_num: 0X %X\r\n", *p_num);               /* 打印*p_num 的值 */
    printf("p_num: 0X %X\r\n", (uint32_t)p_num);       /* 打印 p_num 的值 */
    printf("&p_num: 0X %X\r\n", (uint32_t)&p_num); /* 打印&p_num 的值 */
    while (1);
}
```

此代码的输出结果为：

```
temp:0X88
*p_num: 0X88
p_num: 0X24000BCC
&p_num: 0X24000BC8
```

图 5.1.7.1 输出结果

p_num：是 uint8_t 类型指针，指向 temp 变量的地址，其值等于 temp 变量的地址。

*p_num：取 p_num 指向的地址所存储的值，即 temp 的值。

&p_num：取 p_num 指针的地址，即指针自身的地址。

以上，就是指针的简单使用和基本概念说明，指针的详细知识和使用范例大家可以百度学习，网上有非常多的资料可供参考。指针是 C 语言的精髓，在后续的代码中我们将会大量用到各种指针，大家务必好好学习和了解指针的使用。

5.2 寄存器基础知识

寄存器（Register）是单片机内部一种特殊的内存，它可以实现对单片机各个功能的控制，简单的来说可以把寄存器当成一些控制开关，控制包括内核及外设的各种状态。所以无论是 51 单片机还是 STM32，都需要用寄存器来实现各种控制，以完成不同的功能。

由于寄存器资源非常宝贵，一般都是一个位或者几个位控制一个功能，对于 STM32 来说，其寄存器是 32 位的，一个 32 位的寄存器，可能会有 32 个控制功能，相当于 32 个开关，由于 STM32 的复杂性，它内部有几百个寄存器，所以整体来说 STM32 的寄存器还是比较复杂的。不过，我们不要被其吓到了，实际上 STM32 是由于内部有很多外设，所以导致寄存器很多，实际上我们把它分好类，每个外设也就那么几个或者几十个寄存器，学起来就不难了。

从大方向来区分，STM32 寄存器分为两类，如表 5.2.1 所示：

大类	小类	说明
内核寄存器	内核相关寄存器	包含 R0~R15、xPSR、特殊功能寄存器等
	中断控制寄存器	包含 NVIC 和 SCB 相关寄存器，NVIC 有：ISER、ICER、ISPR、IP 等；SCB 有：VTOR、AIRCR、SCR 等
	SysTick 寄存器	包含 CTRL、LOAD、VAL 和 CALIB 四个寄存器
	内存保护寄存器	可选功能
	调试系统寄存器	ETM、ITM、DWT、IPIU 等相关寄存器
外设寄存器		包含 GPIO、UART、IIC、SPI、TIM、DMA、ADC、DAC、RTC、I/WWDG、PWR、CAN、USB 等各种外设寄存器

表 5.2.1 STM32 寄存器分类

其中，内核寄存器，我们一般只需要关心中断控制寄存器和 SysTick 寄存器即可，其他三大类，我们一般很少直接接触。而外设寄存器，则是学到哪个外设，就了解哪个外设相关寄存

器即可，所以整体来说，我们需要关心的寄存器并不是很多，而且很多都是有共性的，比如 STM32H743IIT6 有 20 个定时器，我们只需要学习了其中一个的相关寄存器，其他 19 个基本都是一样。

说了这么多，给大家举个简单的例子，我们知道寄存器的本质是一个特殊的内存，对于 STM32 来说，以 GPIOB 的 ODR 寄存器为例，其寄存器地址为：0X58020414，所以我们对齐赋值可以写成：

```
*(unsigned int *))(0X58020414) = 0XFFFF;
```

这样我们就完成了对 GPIOB->ODR 寄存器的赋值，全部 0XFFF，表示 GPIOB 所有 IO 口（16 个 IO 口）都输出高电平。对于我们来说，0X58020414 就是一个寄存器的特殊地址，至于它是怎么来的，我们后续再给大家介绍。

虽然上面的代码实现了我们需要的功能，但是从实用的角度来说，这么写肯定是不好的，可读性极差，可维护性也很差，所以一般我们使用结构体来访问，比如改写成这样：

```
GPIOB->ODR = 0XFFFF;
```

这样可读性就比之前的代码好多了，可维护性也相对好一点。至于 GPIOB 结构体怎么来的，我们也会在后续给大家介绍。

5.3 STM32H743 系统架构

STM32H743 是 ST 公司基于 ARM 授权 Cortex M7 内核而设计的一款芯片，而 Cortex M 内核使用的是 ARM v7-M 架构，是为了替代老旧的单片机而量身定做的一个内核，具有低成本、低功耗、实时性好、中断响应快、处理效率高等特点。

5.3.1 Cortex M7 内核 & 芯片

ARM公司提供内核（如Cortex M7，简称CM7，下同）授权，完整的MCU还需要很多其他组件。芯片公司（ST、NXP、TI、GD、华大等）在得到CM7内核授权后，就可以把CM7内核用在自己的硅片设计中，添加：存储器，外设，I/O 以及其它功能块。不同厂家设计出的单片机会有不同的配置，包括存储器容量、类型、外设等都各具特色，因此才会有市面上各种不同应用的ARM芯片。Cortex M7内核和芯片的关系如图5.3.1.1所示：

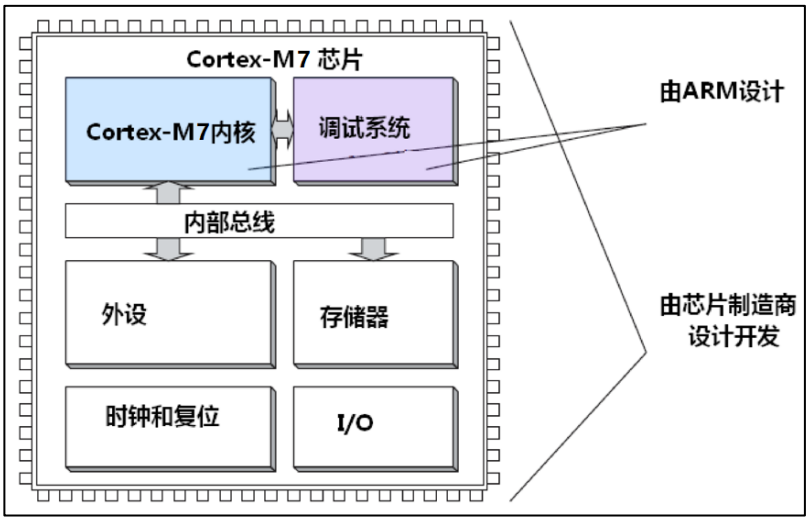


图 5.3.1.1 Cortex M7 内核 & 芯片关系

可以看到，ARM公司提供CM7内核和调试系统，其他的东西（外设（IIC、SPI、UART、TIM等）、存储器（SRAM、FLASH等）、I/O等）由芯片制造商设计开发。这里ST公司就是STM32H743芯片的制造商。

5.3.2 STM32 系统架构

STM32H743IIT6 内部系统结构如图 5.3.2.1：

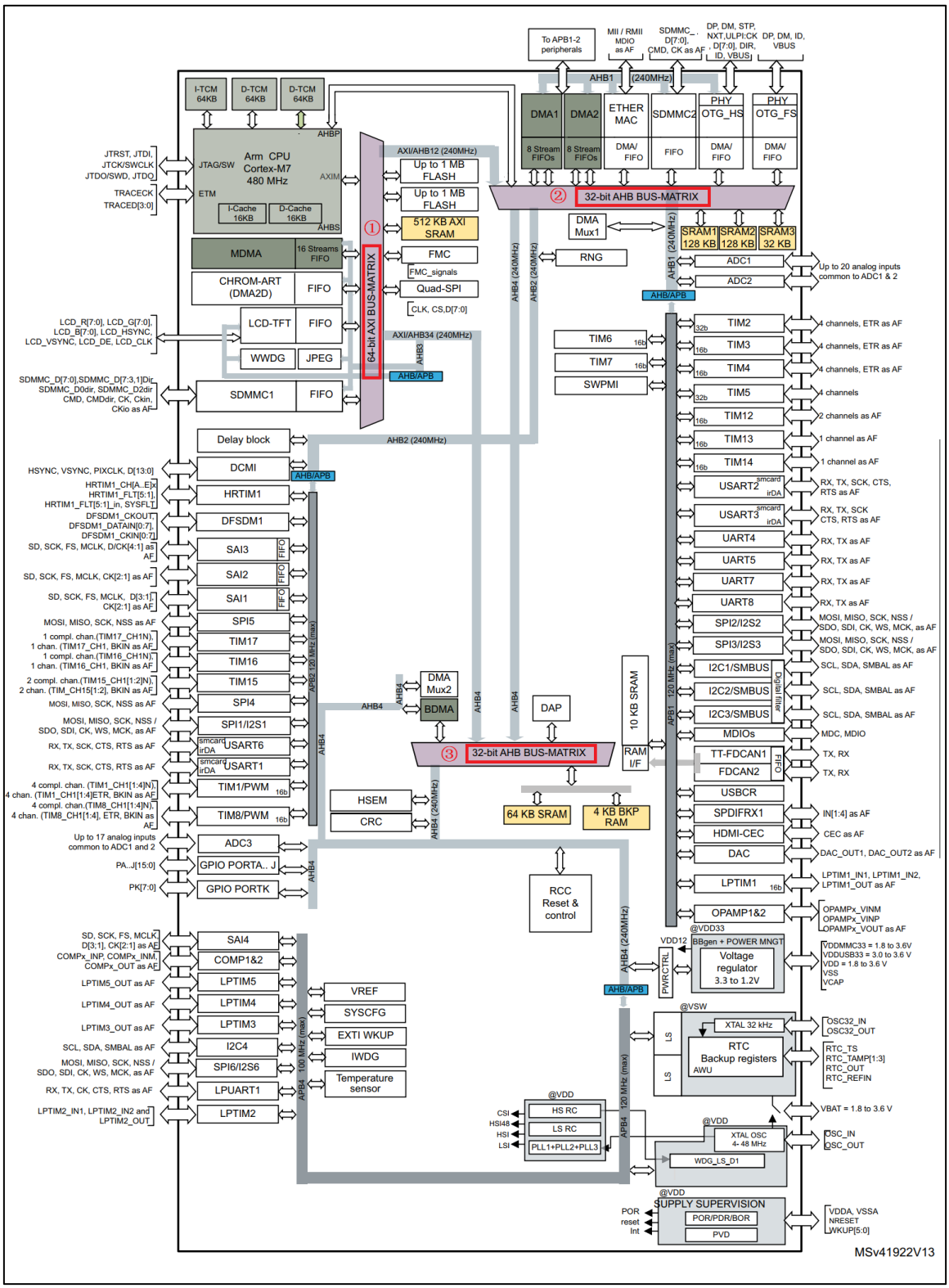


图5.3.2.1 STM32H743系统结构

上图把 STM32H7 的每条总线、总线时钟速度，以及每条总线挂载了那些外设都整理出来，这对我们理解整个 STM32H743 总的框架非常有帮助。

这个图这么复杂，该怎么看？我一般习惯以内核为起点，先看总线，再看总线连接关系，最后再看各个总线分别连接了什么外设，按照这样的顺序查阅，就会清晰很多。

在图 5.3.2.1 中，我们标记了①②③三个标号，①是 64 位的 AXI 总线矩阵，在 D1 域，②

和③都是 32 位的 AHB 总线矩阵，不同的是②在 D2 域，而③是在 D3 域。①②③的关系，我们看下面这张图更能清楚的表达出来，如图 5.3.2.2 所示。

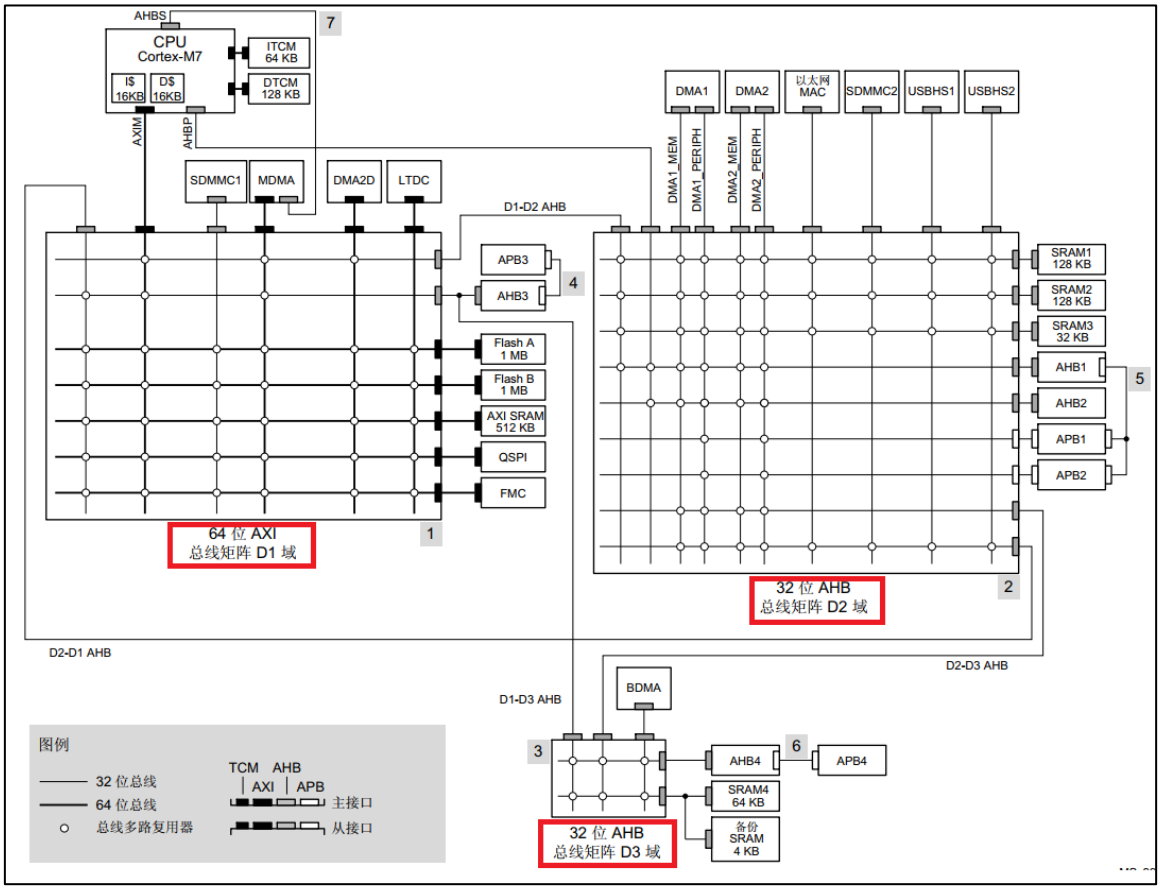


图 5.3.2.2 STM32H743 的总线架构图

从上图中可以看到 STM32H7 的总线架构图比 STM32H7 的系统总架构图简化些，就是把 STM32H7 的系统总架构图中的总线部分提取出来，大家可以把它们结合一起看。

图 5.3.2.2 中的三个域就对应了前面的①②③，其中 D1 域是 AXI 总线，D2 和 D3 都是 AHB 总线的。下面分别介绍一下这三个域：

- **D1 域**

D1 域中的各个外设是挂在 64 位 AXI 总线组成 6*7 的矩阵上。6 个从接口端外接的主控分别是 LTDC、DMA2D、MDMA、SDMMC1、AXIM 和 D2-to-D1 AHB 总线。7 个主接口端外接的从设备分别是 D1-to-D2 AHB 总线、 AHB3 总线、Flash A、Flash B、AXI SRAM 、QSPI 和 FMC。另外 AHB3 再分支出 APB3 总线。

- **D2 域**

D2 域的各个外设是挂在 32 位 AHB 总线组成 10*9 的矩阵上。10 个从接口外接的主控分是 D1-to-D2 AHB 总线、AHBP 总线、DMA1、DMA2、以太网 MAC、SDMMC2、USB HS1 和 USB HS2。9 个主接口外接的从设备分别是 SRAM1、SRAM2、SRAM3、AHB1、AHB2、APB1、APB2、D2-to-D1 AHB 总线和 D2-to-D3 AHB 总线。

- **D3 域**

D3 域的各个外设是挂在 32 位 AHB 总线组成 3*2 的矩阵上。3 个从接口外接的主控分别是 D1-to-D3 AHB 总线、D2-to-D3 AHB 总线和 BDMA。2 个主接口外接的从设备分别是 AHB4、SRAM4 和 备份 SRAM。另外 AHB4 分支出 APB4 总线。

从 STM32H7 的总线架构图里，我们要注意域间总线互连，因为这决定哪些设备（总线）可以相互通信，共有四条域间总线，具体如下：

D2-D1 AHB 总线

该 32 位总线将 D2 域连接到 D1 域中的 AXI 总线矩阵。它使得 D2 域中的总线主设备能够

访问 D1 域中的资源（总线从设备），以及通过 D1-D3 AHB 间接访问 D3 域中的资源（总线从设备）。例如：D2 域里面的 DMA2 访问 D1 域里面的 AXI SRAM。

D1-D2 AHB 总线

该 32 位总线将 D1 域连接到 D2 域 AHB 总线矩阵。它使得 D1 域中的总线主设备能够访问 D2 域中的资源（总线从设备）。例如：D1 域里面的 DMA2D 访问 D2 域里面的 SRAM1。

D1-D3 AHB 总线

该 32 位总线将 D1 域连接到 D3 域 AHB 总线矩阵。它使得 D1 域中的总线主设备能够访问 D3 域中的资源（总线从设备）。例如：D1 域里面的 DMA2D 访问 D3 域里面的 SRAM4。

D2-D3 AHB 总线

该 32 位总线将 D2 域连接到 D3 域 AHB 总线矩阵。它使得 D2 域中的总线主设备能够访问 D3 域中的资源（总线从设备）。例如：D2 域里面的 DMA2 访问 D3 域里面的 SRAM4。

总线结构的内容多且复杂，大家也不要想一次阅读就能掌握，把后面的实验都学习了，不懂就回来查阅（请结合官方原文），把后面的实验学习完，就会越来越清晰了。

5.3.3 存储器映射

STM32是一个32位单片机，他可以很方便的访问4GB以内的存储空间（2^32 = 4GB），因此Cortex M7内核将图5.3.2.1中的所有结构，包括：FLASH、SRAM、外设及相关寄存器等全部组织在同一个4GB的线性地址空间内，我们可以通过C语言来访问这些地址空间，从而操作相关外设（读/写）。数据字节以小端格式（小端模式）存放在存储器中，数据的高字节保存在内存的高地址中，而数据的低字节保存在内存的低地址中。

存储器本身是没有地址信息的，我们对存储器分配地址的过程就叫存储器映射。这个分配一般由芯片厂商做好了，ST将所有的存储器及外设资源都映射在一个4GB的地址空间上（8个块），从而可以通过访问对应的地址，访问具体的外设。其映射关系如图5.3.3.1所示：

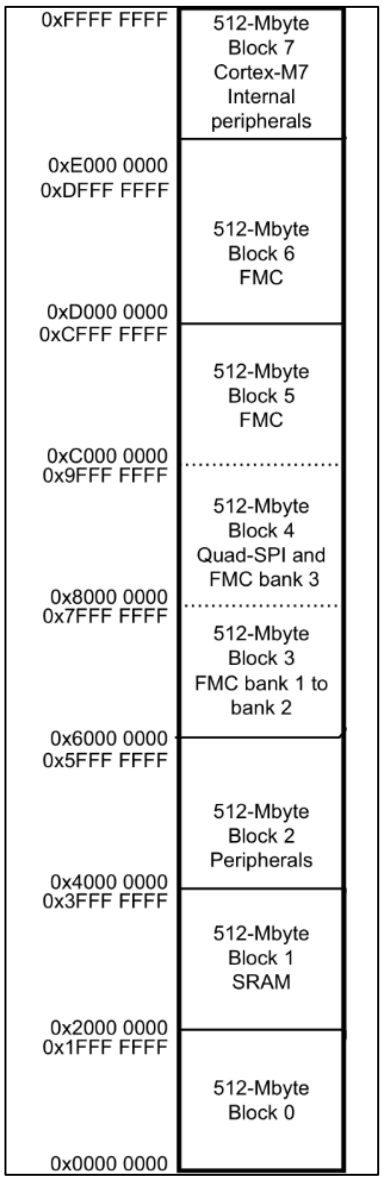


图5.3.3.1 STM32H743存储器映射

存储块功能介绍

ST将4GB空间分成8个块，每个块512MB，如上图所示，从图中我们可以看出有很多保留区域（Reserved），这是因为一般的芯片制造厂家是不可能把4GB空间用完的，同时，为了方便后续型号升级，会将一些空间预留（Reserved）。8个存储块的功能如表5.3.3.1所示：

存储块	功能	地址范围
Block 0	Code	0X0000 0000 ~ 0x1FFF FFFF（512MB）
Block 1	SRAM	0X2000 0000 ~ 0x3FFF FFFF（512MB）
Block 2	外设	0X4000 0000 ~ 0x5FFF FFFF（512MB）
Block 3	FMC Bank1&2	0X6000 0000 ~ 0x7FFF FFFF（512MB）
Block 4	FMC Bank3&4	0X8000 0000 ~ 0x9FFF FFFF（512MB）
Block 5	FMC、Quad-SPI 寄存器	0XA000 0000 ~ 0xBFFF FFFF（512MB）
Block 6	FMC Bank5&6	0XC000 0000 ~ 0xDFFF FFFF（512MB）
Block 7	Cortex M7 内部外设	0XE000 0000 ~ 0xFFFF FFFF（512MB）

表5.3.3.1 STM32 存储块功能及地址范围

这里我们重点挑前面3个存储块给大家介绍。第一个块是Block 0，用于存储代码，即

FLASH空间，其功能划分如表5.3.3.2所示：

存储块	功能	地址范围
Block 0	ITCM RAM，只能被 CPU 和 MDMA 访问，不用经过总线矩阵，属于高速的 RAM	0X0000 0000 ~ 0x0000 FFFF（64KB）
	保留	0X0001 0000 ~ 0x000F FFFF
	系统存储器，用于存储 STM32 出厂固化的 Bootloader 程序，比如用于串口下载代码	0X0010 0000 ~ 0x0010 EDBF
	保留	0X0011 0000 ~ 0x001F FFFF
	FLASH 存储基于 ITCM 总线接口，不支持写操作，即只读。	0X0020 0000 ~ 0x003F FFFF
	保留	0X0040 0000 ~ 0x07FF FFFF
	用户 FLASH，用于存储用户代码（存储区域 1）	0X0800 0000 ~ 0x080F FFFF（1MB）
	用户 FLASH，用于存储用户代码（存储区域 2）	0X0810 0000 ~ 0x081F FFFF（1MB）
	16 个字节用于锁定对应的 OTP 数据块	0X1FFF 0000 ~ 0X1FFF 001F
	保留	0X1FFF 0020 ~ 0X1FFF FFFF

表5.3.3.2 STM32 存储块0的功能划分

可以看到，我们用户FLASH大小是2MB，这是对于我们使用的STM32H743IIT6来说，如果是其他型号， FLASH就可能不一样了，当然，如果ST喜欢，也是可以随时推出更大容量的STM32H743单片机的，因为这里保留了一大块地址空间。DTCM RAM可用于运行指令，也可以存储数据。

第二个块是Block 1，用于存储数据，即SRAM空间，其功能划分如表5.3.3.3所示：

存储块	功能	地址范围
Block 1	DTCM RAM	0x2000 0000 ~ 0x2001 FFFF（128KB）
	AXI SRAM	0x2400 0000 ~ 0x2407 FFFF（512KB）
	SRAM1	0x3000 0000 ~ 0x3001 FFFF（128KB）
	SRAM2	0x3002 0000 ~ 0x3003 FFFF（128KB）
	SRAM3	0x3004 0000 ~ 0x3004 7FFF（32KB）
	SRAM4	0x3800 0000 ~ 0x3800 FFFF（64KB）
	Backup SRAM	0x3880 0000 ~ 0x3880 0FFF（4KB）

表5.3.3.3 STM32 存储块1的功能划分

可以看到STMH743的SRAM分配基本不是连续的地址空间，我们重点注意各块SRAM的特性对比，具体如下：

DTCM RAM，用于数据存取，特点是速度很快，和内核一样。

AXI SRAM，位于D1域，数据带宽是64bit，挂在AXI总线上。

SRAM1，SRAM2和 SRAM3区，位于D2域，数据带宽是32bit，挂在AHB总线上。

SRAM1：用于D2域中的DMA缓冲，也可以当D1域断电后用于运行程序代码。

SRAM2：用于D2域中的DMA缓冲，也可以用于用户数据存取。

SRAM3：主要用于以太网和 USB 的缓冲。

SRAM4，位于D3域，数据带宽是32bit，挂在AHB总线上。可以用于D3域中的DMA缓冲，也可以当D1和D2域进入DStandby待机方式后，继续保存用户数据。

Backup SRAM，位于D3域，数据带宽是32bit，挂在AHB总线上。主要用于系统进入低功耗模式后，继续保存数据（Vbat引脚外接电池）。大家可以参考5.3.2小节，进行理解。

第三个块是Block 2，用于外设访问，STM32内部大部分的外设都是放在这个块里面的。该存储块被分成了APB和AHB两部分，其中APB又被分为APB1，APB2，APB3和APB4。AHB分为AHB1，AHB2，AHB3和AHB4。其功能划分如表5.3.3.4所示：

存储块	功能	地址范围
Block2	APB1 总线外设	0x4000 0000 ~ 0x4000 D3FF

	保留	0x4000 D400 ~ 0x4000 FFFF
	APB2 总线外设	0x4001 0000 ~ 0x4001 77FF
	保留	0x4001 7800 ~ 0x4001 FFFF
	AHB1 总线外设	0x4002 0000 ~ 0x400B FFFF
	保留	0x400C 0000 ~ 0x4801 FFFF
	AHB2 总线外设	0x4802 0000 ~ 0x4802 2BFF
	保留	0x4802 2C00 ~ 0x4FFF FFFF
	APB3 总线外设	0x5000 0000 ~ 0x5000 3FFF
	保留	0x5000 4000 ~ 0x50FF FFFF
	AHB3 总线外设	0x5100 0000 ~ 0x5200 8FFF
	保留	0x5200 9000 ~ 0x57FF FFFF
	APB4 总线外设	0x5800 0000 ~ 0x5800 6BFF
	保留	0x5800 6C00 ~ 0x5801 FFFF
	AHB4 总线外设	0x5802 0000 ~ 0x5802 67FF
	保留	0x5802 6800 ~ 0x5FFF FFFF

表5.3.3.4 STM32 存储块2的功能划分

同样可以看到，各个总线之间，都有预留地址空间，方便后续扩展。关于STM32各个外设具体挂在哪个总线上面，大家可以参考前面的 STM32H743系统结构图进行查找对应。

5.3.4 寄存器映射

给存储器分配地址的过程叫存储器映射，寄存器是一类特殊的存储器，它的每个位都有特定的功能，可以实现对外设/功能的控制，给寄存器的地址命名的过程就叫寄存器映射。

举个简单的例子，大家家里面的纸张就好比通用存储器，用来记录数据是没问题的，但是不会有具体的动作，只能做记录，而你家里面的电灯开关，就好比寄存器了，假设你家有8个灯，就有8个开关（相当于一个8位寄存器），这些开关也可以记录状态，同时还能让电灯点亮/关闭，是会产生具体动作的。为了方便区分和使用，我们会给每个开关命名，比如厨房开关、大厅开关、卧室开关等，给开关命名的过程，就是寄存器映射。

当然STM32内部的寄存器有非常多，远远不止8个开关这么简单，但是原理是差不多的，每个寄存器的每一个位，一般都有特定的作用，涉及到寄存器描述，大家可以参考《STM32H7xx参考手册_V3（中文版）.pdf》相关章节的寄存器描述部分，有详细的描述。

1. 寄存器描述解读

我们以GPIO的ODR寄存器为例，其参考手册的描述如图5.3.4.1所示：

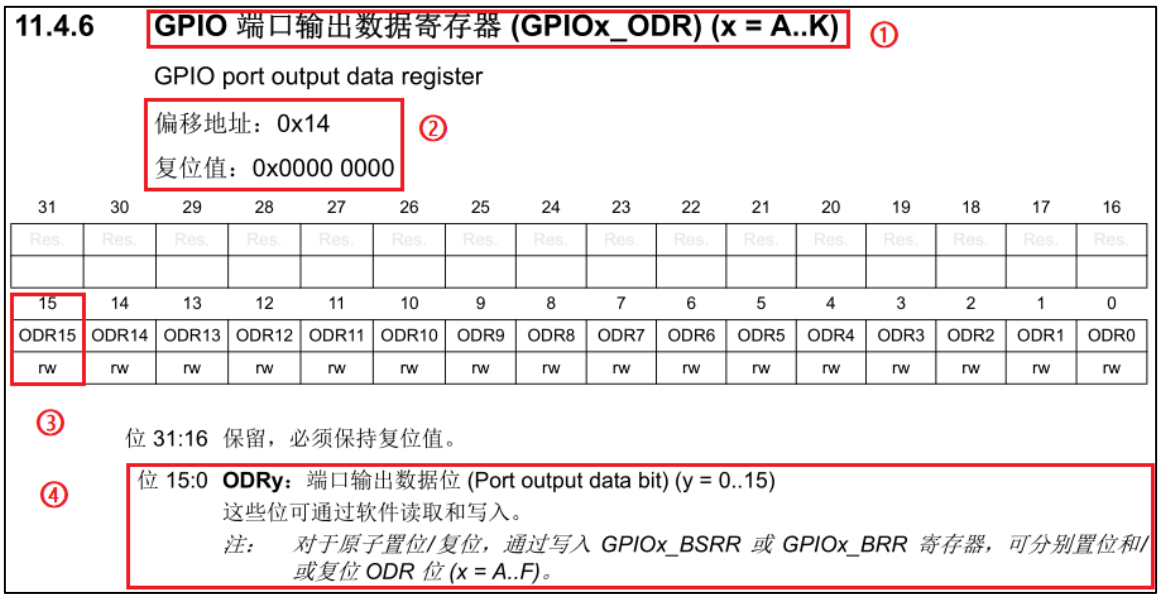


图5.3.4.1 端口输出数据寄存器描述

① 寄存器名字

每个寄存器都有一个对应的名字，以简单表达其作用，并方便记忆，这里GPIOx_ODR表示寄存器英文名，x可以从A~K，说明有11个这样的寄存器。

② 寄存器偏移量及复位值

地址偏移量表示相对该外设基地址的偏移，比如GPIOB，我们由《STM32H7xx参考手册_V3（中文版）.pdf》文档的第100页，可知其外设基地址是：0x58020400。那么GPIOB_ODR寄存器的地址就是：0x58020414。知道了外设基地址和地址偏移量，我们就可以知道任何一个寄存器的实际地址。

复位值表示该寄存器在系统复位后的默认值，可以用于分析外设的默认状态。这里都为0。

③ 寄存器位表

描述寄存器每一个位的作用（共32bit），这里表示ODR寄存器的第15位（bit），位名字为ODR15，rw表示该寄存器可读写（r，可读取；w，可写入）。

④ 位功能描述

描述寄存器每个位的功能，这里表示位0~15，对应ODR0~ODR15，每个位控制一个IO口的输出状态。

其他寄存器描述，参照以上方法解读接口。

2. 寄存器映射举例

从前面的学习我们知道 GPIOB_ODR 寄存器的地址为：0x58020414，假设我们要控制 GPIOB 的 16 个 IO 口都输出 1，则可以写成：

```
*(unsigned int *)(0x58020414) = 0XFFFF;
```

这里我们先要将 0x58020414 强制转换成 unsigned int 类型指针，然后用*对这个指针的值进行设置，从而完成对 GPIOB_ODR 寄存器的写入。

这样写代码功能是没问题，但是可读性和可维护性都很差，使用起来极其不便，因此我们将代码改为：

```
#define GPIOB_ODR       *(unsigned int *)(0x58020414)
GPIOB_ODR = 0XFFFF;
```

这样，我们就定义了一个 GPIOB_ODR 的宏，来替代数值操作，很明显，GPIOB_ODR 的可读性和可维护性，比直接使用数值操作来的直观和方便。这个宏定义过程就可以称之为寄存器的映射。

当然，为了简单，我们只举了一个简单实例，实际上大量寄存器的映射，使用结构体是最方便的方式，stm32h743xx.h 里面使用结构体方式对 STM32H743 的寄存器做了详细映射。

3. 寄存器地址计算

STM32H743大部分外设寄存器地址都是在存储块2上面的，见图5.3.3.1。具体某个寄存器地址，由三个参数决定：1、总线基地址（BUS_BASE_ADDR）；2，外设基于总线基地址的偏移量（PERIPH_OFFSET）；3，寄存器相对外设基地址的偏移量（REG_OFFSET）。可以表示为：

寄存器地址 = BUS_BASE_ADDR + PERIPH_OFFSET + REG_OFFSET

总线基地址（BUS_BASE_ADDR），STM32H743内部的总线基地址如表5.3.4.1所示：

总线	基地址	偏移量
APB1	0X4000 0000	0X0
APB2	0X4001 0000	0X0001 0000
AHB1	0X4002 0000	0X0002 0000
AHB2	0X4802 0000	0X0802 0000
APB3	0X5000 0000	0X1000 0000
AHB3	0X5100 0000	0X1100 0000
APB4	0X5800 0000	0X1800 0000
AHB4	0X5802 0000	0X1802 0000

表5.3.4.1 总线基地址

上表中APB1的基地址，也叫外设基地址，表中的偏移量就是相对于外设基地址的偏移量。

外设基于总线基地址的偏移量（PERIPH_OFFSET），这个不同外设偏移量不一样，我们可以在STM32H743存储器映射图（图5.3.3.1）里面找到具体的偏移量，以GPIO为例，其偏移量如表5.3.4.2所示：

所属总线	外设	基地址	偏移量
AHB4 0X5802 0000	GPIOA	0X5802 0000	0X0
	GPIOB	0X5802 0400	0X0400
	GPIOC	0X5802 0800	0X0800
	GPIOD	0X5802 0C00	0X0C00
	GPIOE	0X5802 1000	0X1000
	GPIOF	0X5802 1400	0X1400
	GPIOG	0X5802 1800	0X1800
	GPIOH	0X5802 1C00	0X1C00
	GPIOI	0X5802 2000	0X2000
	GPIOJ	0X5802 2400	0X2400
	GPIOK	0X5802 2800	0X2800

表5.3.4.2 GPIO外设基地址及相对总线偏移量

上表的偏移量，就是外设基于APB2总线基地址的偏移量（PERIPH_OFFSET）。

知道了外设基地址，再在参考手册里面找到具体某个寄存器相对外设基地址的偏移量就可以知道该寄存器的实际地址了，以GPIOB的相关寄存器为例，如表5.3.4.3所示：

所属总线	所属外设	寄存器	地址	偏移量
AHB4 0X5802 0000	GPIOB 0X5802 0400	GPIOB_MODER	0X5802 0400	0X00
		GPIOB_OTYPER	0X5802 0404	0X04
		GPIOB_OSPEEDR	0X5802 0408	0X08
		GPIOB_PUPDR	0X5802 040C	0X0C
		GPIOB_IDR	0X5802 0410	0X10
		GPIOB_ODR	0X5802 0414	0X14
		GPIOB_BSRR	0X5802 0418	0X18
		GPIOB_LCKR	0X5802 041C	0X1C
		GPIOB_AFRL	0X5802 0420	0X20
		GPIOB_AFRH	0X5802 0424	0X24

表5.3.4.3 GPIOB寄存器相对外设基地址的偏移量

上表的偏移量，就是寄存器基于外设基地址的偏移量（REG_OFFSET）。

因此，我们根据前面的公式，很容易可以计算出GPIOB_ODR的地址：

GPIOB_ODR地址 = AHB4总线基地址 + GPIOB外设偏移量 + 寄存器偏移量

所以得到：GPIOB_ODR 地址 = 0X5802 0000 + 0X0400 + 0X14 = 0X5802 0414

关于寄存器地址计算我们就讲到这里，通过本节的学习，其他寄存器的地址大家都应该可以熟练掌握并计算出来。

4. stm32h743xx.h 寄存器映射说明

STM32H743所有寄存器映射都在stm32h743xx.h里面完成，包括各种基地址定义、结构体定义、外设寄存器映射、寄存器位定义（占了绝大部分）等，整个文件有2万多行，非常庞大。我们没有必要对该文件进行全面分析，因为很多内容都是相似的，我们只需要知道寄存器是如何被映射的，就可以了，至于寄存器位定义这些内容，知道是怎么回事就可以了。

我们还是以GPIO为例进行说明，看看stm32h743xx.h是如何对GPIO的寄存器进行映射的，通过对GPIO寄存器映射，了解stm32h743xx.h的映射规则。

stm32h743xx.h文件主要包含五个部分内容，如表5.3.4.4所示：

文件	主要组成部分	说明
stm32h743xx.h	中断编号定义	定义 IRQn_Type 枚举类型，包含 STM32H743 内部所有中断编号（中断号），方便后续编写代码
	外设寄存器结构体类型定义	以外设为基本单位，使用结构体类型定义对每个外设的所有寄存器进行封装，方便后面的寄存器映射
	寄存器映射	1，定义总线地址和外设基地址 2，使用外设结构体类型定义将外设基地址强制转换成结构体指针，完成寄存器映射
	寄存器位定义	定义外设寄存器每个功能位的位置及掩码
	外设判定	判断某个外设是否合法（即是否存在该外设）

表5.3.4.4 stm32h743xx.h文件主要组成部分

寄存器映射主要涉及到表5.3.4.4中加粗的两个组成部分：外设寄存器结构体类型定义和寄存器映射，总结起来，包括3个步骤：

1、外设寄存器结构体类型定义

2、外设基地址定义

3、寄存器映射（通过将外设基地址强制转换为外设结构体类型指针即可）

以GPIO为例，其寄存器结构体类型定义如下：

```
typedef struct
{
  __IO uint32_t MODER;      /* GPIO_MODER 寄存器，相对外设基地址偏移量：0X00 */
  __IO uint32_t OTYPER;     /* GPIO_OTYPER 寄存器，相对外设基地址偏移量：0X04 */
  __IO uint32_t OSPEEDR;    /* GPIO_OSPEEDR 寄存器，相对外设基地址偏移量：0X08 */
  __IO uint32_t PUPDR;      /* GPIO_PUPDR 寄存器，相对外设基地址偏移量：0X0C */
  __IO uint32_t IDR;        /* GPIO_IDR 寄存器，相对外设基地址偏移量：0X10 */
  __IO uint32_t ODR;        /* GPIO_ODR 寄存器，相对外设基地址偏移量：0X14 */
  __IO uint32_t BSRR;       /* GPIO_BSRR 寄存器，相对外设基地址偏移量：0X18 */
  __IO uint32_t LCKR;       /* GPIO_LCKR 寄存器，相对外设基地址偏移量：0X1C */
  __IO uint32_t AFR[2];     /* GPIO_AFR 寄存器，相对外设基地址偏移量：0X20 */
} GPIO_TypeDef;
```

GPIO外设基地址定义如下：

```
#define PERIPH_BASE           (0x40000000UL)                    /* 外设基地址 */

#define D3_AHB1PERIPH_BASE    (PERIPH_BASE + 0x18020000UL)      /* AHB4 总线基地址 */

#define GPIOA_BASE            (D3_AHB1PERIPH_BASE + 0x0000UL) /* GPIOA 基地址 */
#define GPIOB_BASE            (D3_AHB1PERIPH_BASE + 0x0400UL) /* GPIOB 基地址 */
#define GPIOC_BASE            (D3_AHB1PERIPH_BASE + 0x0800UL) /* GPIOC 基地址 */
#define GPIOD_BASE            (D3_AHB1PERIPH_BASE + 0x0C00UL) /* GPIOD 基地址 */
#define GPIOE_BASE            (D3_AHB1PERIPH_BASE + 0x1000UL) /* GPIOE 基地址 */
#define GPIOF_BASE            (D3_AHB1PERIPH_BASE + 0x1400UL) /* GPIOF 基地址 */
```

```
#define GPIOG_BASE            (D3_AHB1PERIPH_BASE + 0x1800UL) /* GPIOG 基地址 */
#define GPIOH_BASE            (D3_AHB1PERIPH_BASE + 0x1C00UL) /* GPIOH 基地址 */
#define GPIOI_BASE            (D3_AHB1PERIPH_BASE + 0x2000UL) /* GPIOI 基地址 */
#define GPIOJ_BASE            (D3_AHB1PERIPH_BASE + 0x2400UL) /* GPIOJ 基地址 */
#define GPIOK_BASE            (D3_AHB1PERIPH_BASE + 0x2800UL) /* GPIOK 基地址 */
```

GPIO外设寄存器映射定义如下：

```
#define GPIOA           ((GPIO_TypeDef *) GPIOA_BASE)   /* GPIOA 寄存器地址映射 */
#define GPIOB           ((GPIO_TypeDef *) GPIOB_BASE)   /* GPIOB 寄存器地址映射 */
#define GPIOC           ((GPIO_TypeDef *) GPIOC_BASE)   /* GPIOC 寄存器地址映射 */
#define GPIOD           ((GPIO_TypeDef *) GPIOD_BASE)   /* GPIOD 寄存器地址映射 */
#define GPIOE           ((GPIO_TypeDef *) GPIOE_BASE)   /* GPIOE 寄存器地址映射 */
#define GPIOF           ((GPIO_TypeDef *) GPIOF_BASE)   /* GPIOF 寄存器地址映射 */
#define GPIOG           ((GPIO_TypeDef *) GPIOG_BASE)   /* GPIOG 寄存器地址映射 */
#define GPIOH           ((GPIO_TypeDef *) GPIOH_BASE)   /* GPIOH 寄存器地址映射 */
#define GPIOI           ((GPIO_TypeDef *) GPIOI_BASE)   /* GPIOI 寄存器地址映射 */
#define GPIOJ           ((GPIO_TypeDef *) GPIOJ_BASE)   /* GPIOJ 寄存器地址映射 */
#define GPIOK           ((GPIO_TypeDef *) GPIOK_BASE)   /* GPIOK 寄存器地址映射 */
```

以上三部分代码，就完成了STM32H743内部GPIOA~GPIOK的寄存器映射，其原理其实是比较简单的，包括两个核心知识点：1，结构体地址自增；2，地址强制转换；

结构体地址自增，我们第一步就定义了GPIO_TypeDef结构体类型，其成员包括：MODER、OTYPER、OSPEEDR、PUPDR、IDR、ODR、BSRR、LCKR和AFR[2]，每个成员是uint32_t类型，也就是4个字节，这样假设：MODER地址是0的话，OTYPER就是0X04，OSPEEDR就是0X08，PUPDR就是0X0C，以此类推。

地址强制转换，以GPIOB为例，GPIOB外设的基地址为：GPIOB_BASE（0X5802 0400），我们使用GPIO_TypeDef将该地址强制转换为GPIO结构体类型指针：GPIOB，这样GPIOB->MODER的地址就是：GPIOB_BASE（0X5802 0400），GPIOB->OTYPER的地址就是：GPIOB_BASE + 0X04（0X5802 0404），GPIOB->OSPEEDR的地址就是：GPIOB_BASE + 0X08（0X5802 0408），以此类推。

这样我们就使用结构体方式完成了对GPIO寄存器的映射，其他外设的寄存器映射也都是这个方法，这里就不一一介绍了。关于stm32h743xx.h的寄存器映射，我们就介绍到这里。

第六章 新建寄存器版本 MDK 工程

通过前面几章的学习，我们对 STM32 有了个比较清晰的了解，本章我们将讲解新建寄存器库版本 MDK 工程的详细步骤。我们把本章新建好的工程放在光盘里，路径：**4，程序源码\2，标准例程-HAL 库版本\实验 0 基础入门实验\实验 0-2，新建工程实验-寄存器版本**，大家在学习新建工程过程中间遇到一些问题，可以直接打开这个工程，然后对比学习。

本章将分为如下两个小节：

6.1 新建寄存器版本 MDK 工程

6.2 下载验证

6.1 新建寄存器版本 MDK 工程

本节我们将教大家如何新建一个 STM32H743 的 MDK5 工程。为了方便大家参考，我们将本节最终新建好的工程模板存放在 A 盘：4、程序源码\2，标准例程-HAL 库版本\实验 0 基础入门实验\实验 0-2，新建工程实验-寄存器版本，如遇新建工程问题，请打开该实验对比。

整个新建过程比较复杂，我们将其拆分为 5 个步骤进行讲解，请准备大概 2 个小时时间，耐心细致的做完！对你后续的学习非常有帮助！

在新建工程之前，首先我们要做如下准备：

1、 STM32Cube 官方固件包：我们使用的固件包版本是 STM32Cube_FW_H7_V1.9.0，固件包路径：A 盘→8，STM32 参考资料→1，STM32CubeH7 固件包。

2、开发环境搭建：参考本书第三章相关内容。

6.1.1 新建工程文件夹

新建工程文件夹分为 2 个步骤：1，新建工程文件夹；2，拷贝工程相关文件。

1. 新建工程文件夹

首先我们在桌面新建一个工程根目录文件夹，后续的工程文件都将在这个文件夹里建立，我们把这个文件夹重命名为：实验 0-2，新建工程实验-寄存器版本。如图 6.1.1.1 所示：

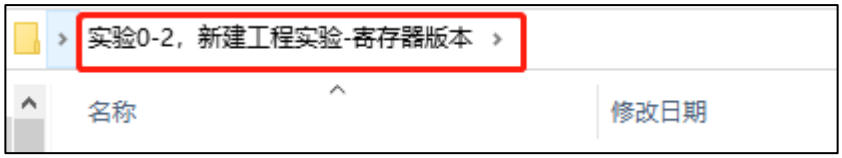


图 6.1.1.1 新建工程根目录文件夹

为了让工程的文件目录结构更加清晰易懂，我们会在工程根目录文件夹下建立以下几个文件夹，每个文件夹名称及其作用如表 6.1.1.1 所示：

名称	作用
Drivers	存放与硬件相关的驱动层文件
Middlewares	存放正点原子提供的中间层组件文件和第三方中间层文件
Output	存放工程编译输出文件
Projects	存放 MDK 工程文件
User	存放 HAL 库用户配置文件、main.c

表 6.1.1.1 工程根目录新建文件夹及其作用

新建完成以后，最后得到我们的工程根目录文件夹如图 6.1.1.2 所示。

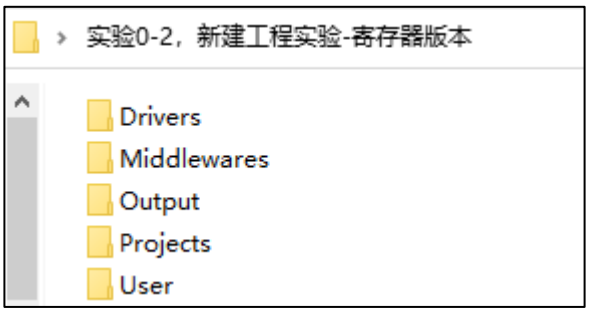


图 6.1.1.2 工程根目录文件夹

工程根目录及其相关文件夹新建好以后，我们需要拷贝一些工程相关文件过来（主要是在 Drivers 文件夹里面），以便等下的新建工程需要。

2. 拷贝工程相关文件

接下来，我们按图 6.1.1.2 的根目录文件夹顺序介绍每个文件夹及其需要拷贝的文件。

Drivers 文件夹

该文件夹用于存放与硬件相关的驱动层文件，一般包括如表 6.1.1.2 所示的三个文件夹：

文件夹名称	作用
BSP	存放开发板板级支持包驱动代码，如各种外设驱动
CMSIS	存放 CMSIS 底层代码，如启动文件（.s 文件）、stm32h7xx.h 等
SYSTEM	存放正点原子系统级核心驱动代码，如 sys.c、delay.c 和 usart.c 等

表 6.1.1.2 Drivers 包含文件夹

BSP 文件夹，用于存放正点原子提供的板级支持包驱动代码，如：LED、按键等。本章我们暂时用不到该文件夹，不过可以先建好备用。

CMSIS 文件夹，用于存放 CMSIS 底层代码（ARM 和 ST 提供），如：启动文件（.s 文件）、stm32h7xx.h 等各种头文件。该文件夹我们可以直接从 STM32CubeH7 固件包（路径：A 盘→8，STM32 参考资料→1，STM32CubeH7 固件包）里面拷贝，不过由于固件包里面的 CMISIS 兼容了太多芯片，导致非常大（300 多 MB），因此我们根据实际情况，对其进行了大幅精简，精简后的 CMSIS 文件夹大小为 2.3MB 左右。精简后的 CMSIS 文件夹大家可以在：A 盘→4，程序源码→1，标准例程-寄存器版本 文件夹里面的任何一个实验的 Drivers 文件夹里面拷贝过来。

SYSTEM 文件夹，用于存放正点原子提供的系统级核心驱动代码，如：sys.c、delay.c 和 usart.c 等，方便大家快速搭建自己的工程。该文件同样可以从：A 盘→4，程序源码→1，标准例程-寄存器版本 文件夹里面的任何一个实验的 Drivers 文件夹里面拷贝过来。

执行完以上操作后，Drivers 文件夹最终结构如图 6.1.1.3 所示：

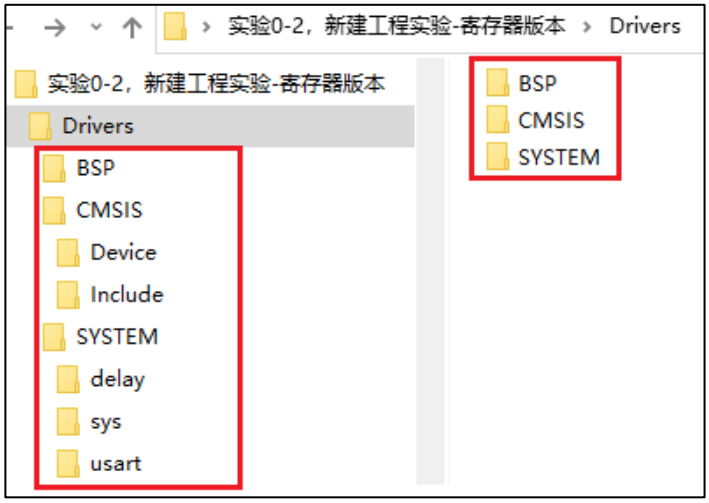


图 6.1.1.3 工程根目录文件夹

Middlewares 文件夹

该文件夹用于存放正点原子和其他第三方提供的中间层代码（组件/Lib 等），如：USMART、MALLOC、TEXT、FATFS、USB、LWIP、各种 OS、各种 GUI 等等。本章我们暂时用不到该文件夹，不过可以先建好备用，后面的实验将会陆续添加各种文件。

Output 文件夹

该文件夹用于存放编译器编译工程输出的中间文件，比如：.hex、.bin、.o 文件等等。这里不需要操作，后面只需要在 MDK 里面设置该文件夹为编译过程中间文件的存放文件夹就行。

Projects 文件夹

该文件夹用于存放编译器（MDK、IAR 等）工程文件，我们主要用 MDK，为了方便区分，我们在该文件夹下新建：MDK-ARM 文件夹，用于存放 MDK 的工程文件，如图 6.1.1.4 所示：

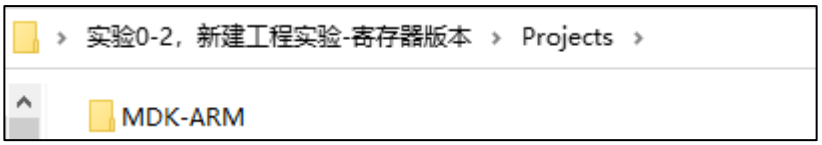


图 6.1.1.4 在 Projects 文件夹下新建 MDK-ARM 文件夹

User 文件夹

该文件夹用于存放用户编写的代码，如：main.c 等。目前还没有任何代码，所以暂时为空即可。

6.1.2 新建一个工程框架

首先，打开 MDK 软件。然后点击 Project→New uVision Project 如图 6.1.2.1 所示：

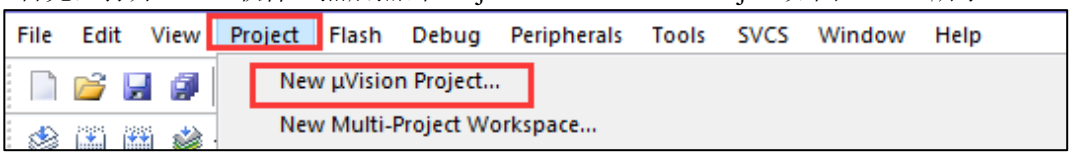


图 6.1.2.1 新建 MDK 工程

然后弹出工程命名和保存的操作窗口，我们将工程文件保存路径设置在上一节新建的工程文件夹内，具体路径为：桌面→实验 0-2，新建工程实验-寄存器版本→Projects→MDK-ARM，工程名字我们取：atk_h743，最后点击保存即可。具体操作窗口如图 6.1.2.2 所示：

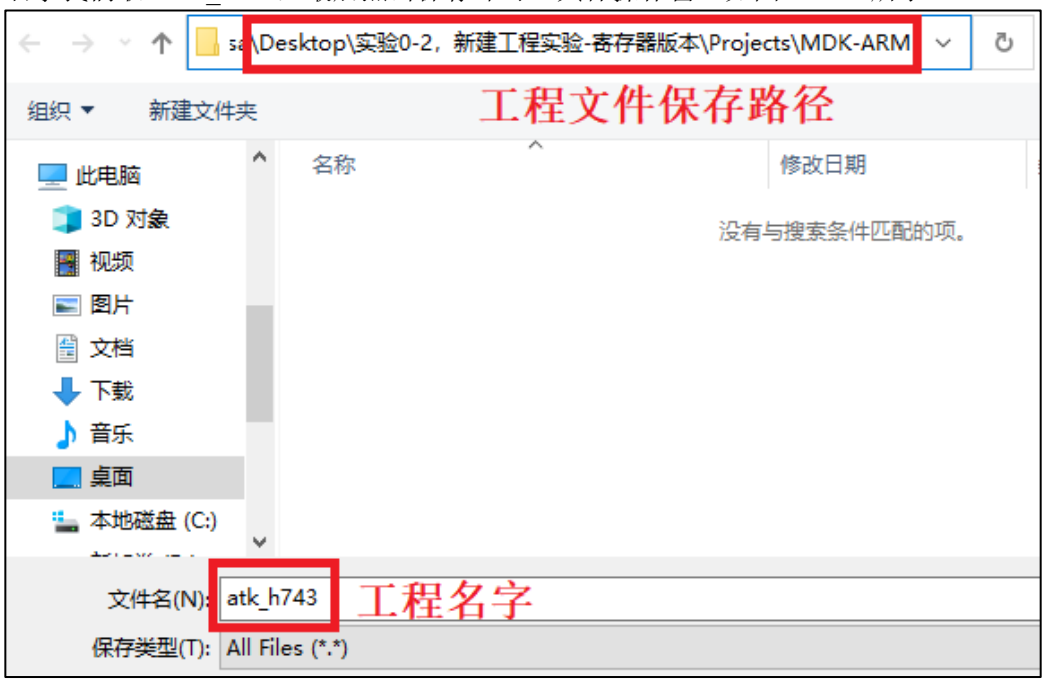


图 6.1.2.2 保存工程界面

之后，弹出器件选择对话框，如图 6.1.2.3 所示。因为正点原子阿波罗 STM32H743 开发板

所使用的 STM32 型号为 STM32H743IIT6，所以我们选择：STMicroelectronics→STM32H7 Series→STM32H743→STM32H743IITx（如果使用的是其他系列的芯片，选择相应的型号就可以了，特别注意：**一定要安装对应的器件 pack** 才会显示这些内容哦!!如果没得选择，请关闭 MDK，然后安装 A 盘：6，软件资料\1，软件\MDK5\ Keil.STM32H7xx_DFP.2.7.0.pack 这个安装包后重试）。

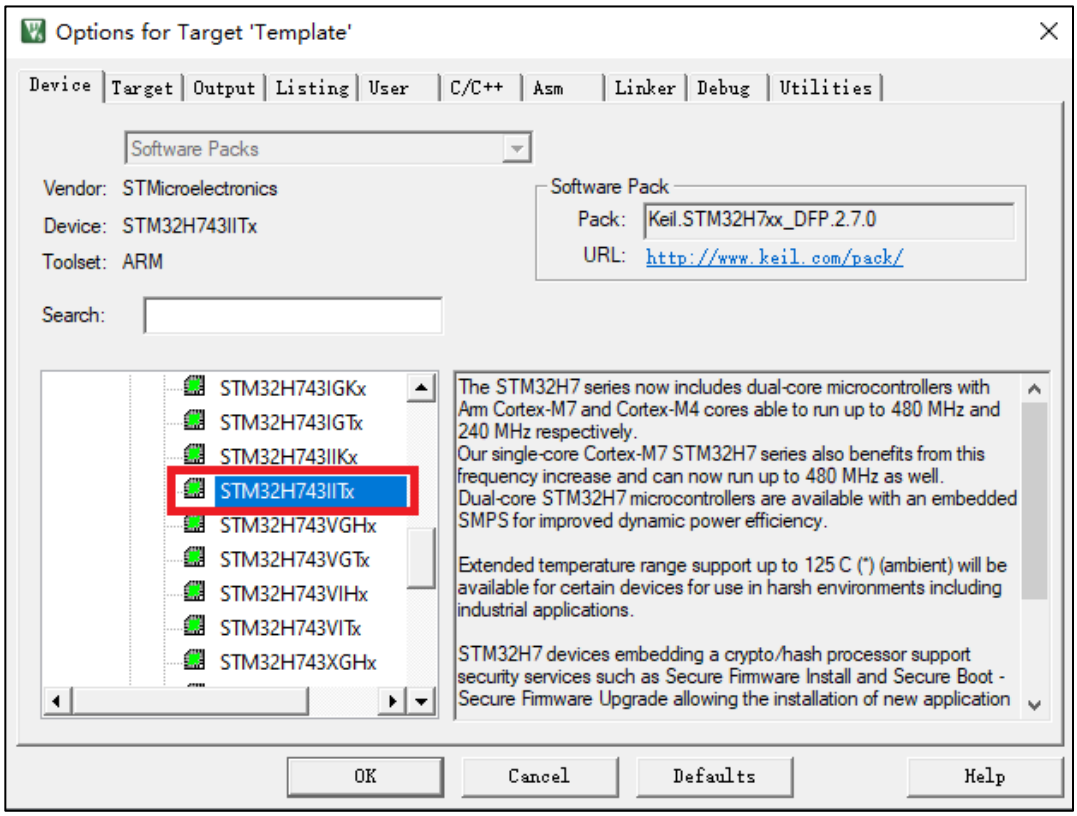


图 6.1.2.3 器件选择界面

点击 OK，MDK 会弹出 Manage Run-Time Environment 对话框，如图 6.1.2.4 所示：

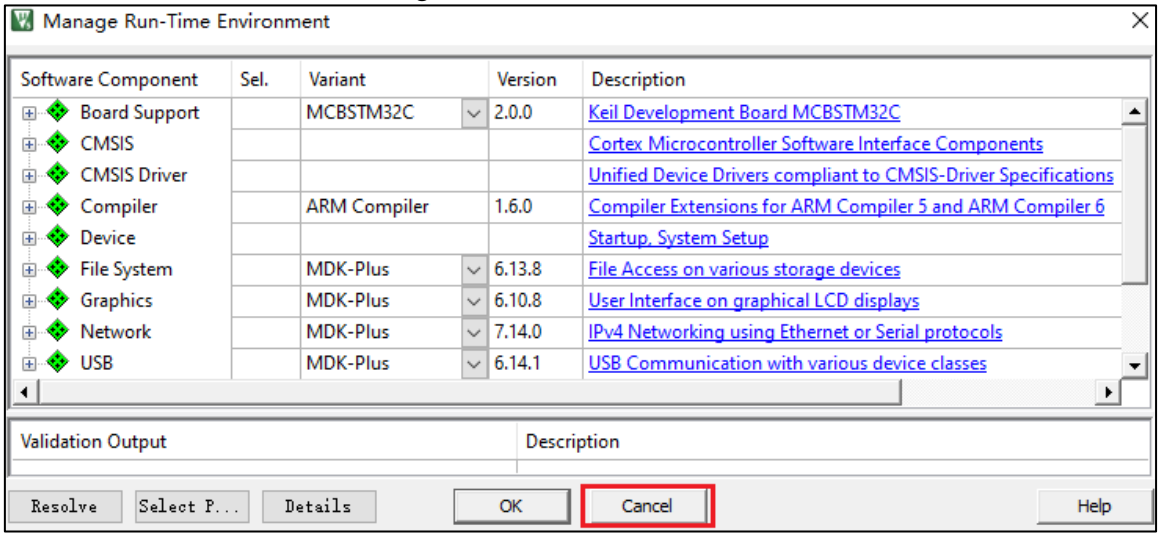


图 6.2.1.4 Manage Run-Time Environment 界面

这是 MDK5 新增的一个功能，在这个界面，我们可以添加自己需要的组件，从而方便构建开发环境，不过这里我们不做介绍。所以在图 6.1.2.4 所示界面，我们直接点击 Cancel，即可，得到如图 6.1.2.5 所示界面：

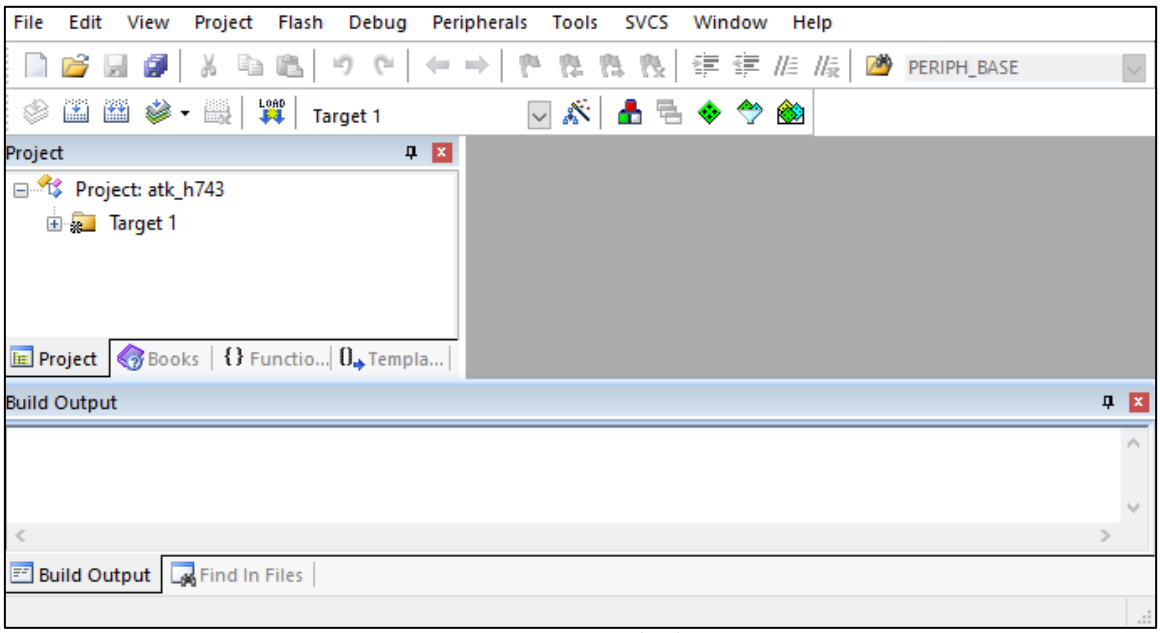


图 6.1.2.5 工程初步建立

此时，我们打开 MDK-ARM 文件夹，会看到 MDK 在该文件夹下自动创建了 3 个文件夹（DebugConfig、Listings 和 Objects），如图 6.1.2.6 所示：

图 6.1.2.6 MDK 新建工程时自动创建的文件夹

这三个文件夹的作用如表 6.1.2.1 所示：

文件夹	作用
DebugConfig	用于存放调试设置信息文件（.dbgconf），不可删除！
Listings	用于存放编译过程产生的链接列表等文件
Objects	用于存放编译过程产生的调试信息、.hex、预览、.lib 文件等

表 6.1.2.1 三个文件夹及其作用

编译过程产生的链接列表、调试信息、预览、lib 等文件，统称为中间文件。为了统一管理，方便使用，我们会把输出在 Listings 和 Objects 文件夹的内容，统一改为输出到 Output 文件夹（通过魔术棒设置），我们先**把 MDK 自动生成的这两个文件夹（Listings 和 Objects）删除**。

至此，我们还只是建了一个框架，还有好几个步骤要做，比如添加文件、魔术棒设置、编写 main.c 等。

6.1.3 添加文件

本节将分 3 个步骤：1，设置工程名和分组名；2，添加启动文件；3，添加 SYSTEM 源码。

1. 设置工程名和分组名

在 Project→Target 上右键，选择 Manage Project Items…（方法一）或在菜单栏点击品字形红绿白图标（方法二）进入工程管理界面，如图 6.1.3.1 所示：

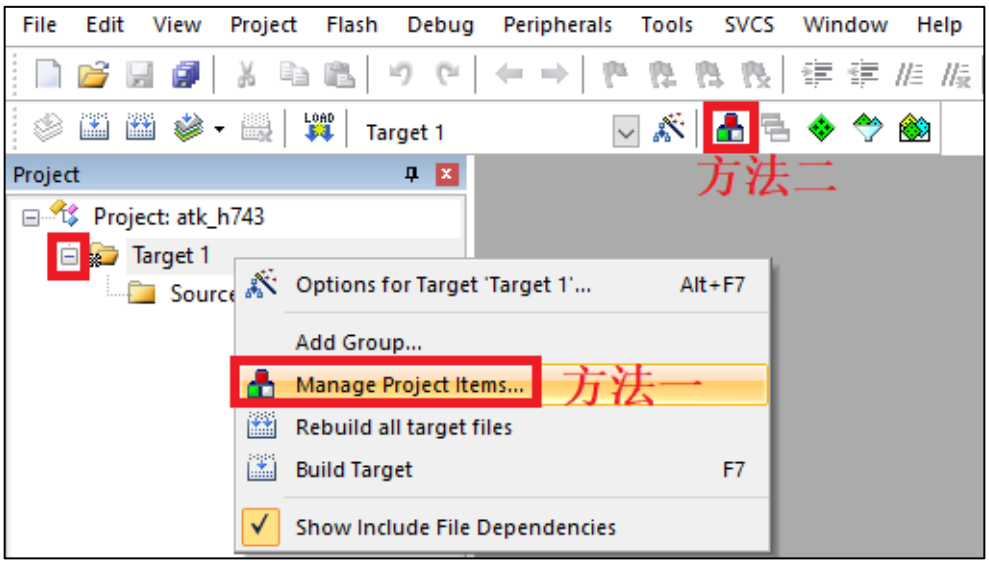


图 6.1.3.1 进入工程管理界面

在工程管理界面，我们可以执行设置工程名字（Project Targets）、分组名字（Groups）以及添加每个分组的文件（Files）等操作。我们设置工程名字为：Template，并设置四个分组：Startup（存放启动文件）、User（存放 main.c 等用户代码）、Drivers/SYSTEM（存放系统级驱动代码）、Readme（存放工程说明文件），如图 6.1.3.2 所示：

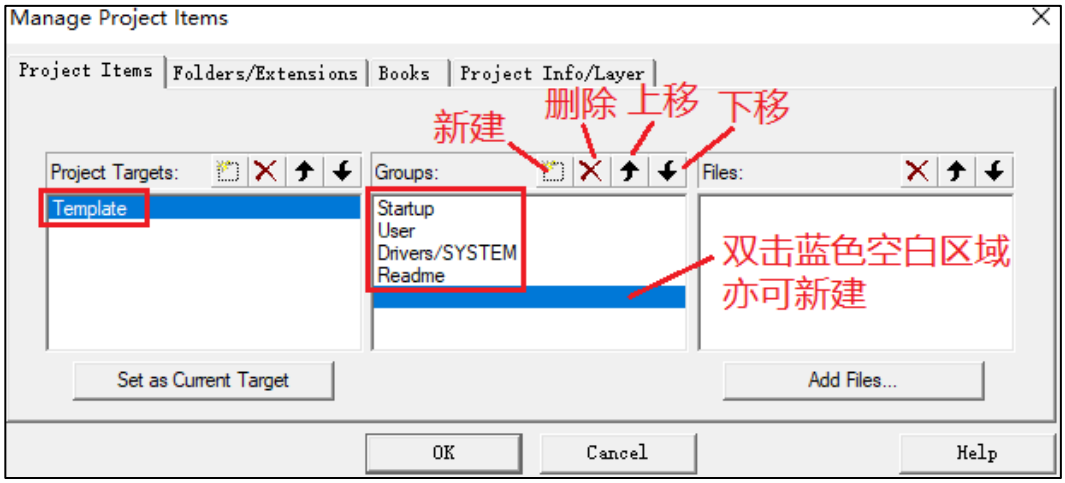


图 6.1.3.2 设置工程名和分组名

设置好之后，我们点击 OK，回到 MDK 主界面，可以看到我们设置的工程名和分组名如图 6.1.3.3 所示：

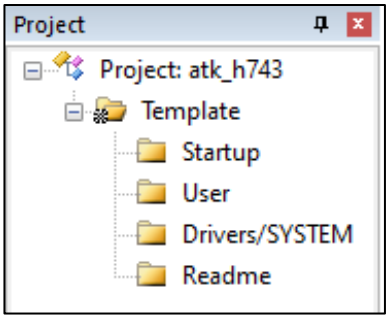


图 6.1.3.3 设置成功

这里我们只是新建了一个简单的工程，并没有添加 BSP、Middlewares 等分组，后面随着工程复杂程度的增加，我们需要一步步添加对应的分组。

注意：为了让工程结构清晰，我们会尽量让 MDK 的工程分组和我们前面新建的工程文件夹对应起来，由于 MDK 分组不支持多级目录，因此我们将路径也带入分组命名里面，以便区分。如：User 分组对应 User 文件夹里面的源码，Drivers/SYSTEM 分组，对应 Drivers/SYSTEM 文件夹里面的源码，Drivers/BSP 分组对应 Drivers/BSP 文件夹里面的源码等。

2. 添加启动文件

启动文件（.s 文件）包含 STM32 的启动代码，其主要作用包括：1、堆栈（SP）的初始化；2、初始化程序计数器（PC）；3、设置向量表异常事件的入口地址；4、调用 main 函数等，是每个工程必不可少的一个文件，我们在本书第九章会有详细介绍。

启动文件由 ST 官方提供，存放在 STM32CubeH7 软件包的：Drivers→CMSIS→Device→ST→STM32H7xx→Source→Templates→arm 文件夹下。因为我们开发板使用的是 STM32H743IIT6，对应的启动文件为：startup_stm32h743xx.s，为了节省空间，在精简版 CMSIS 文件夹里面我们把其他启动文件都删了。而且，为了更好的匹配寄存器版本代码，我们对 startup_stm32h743xx.s 做了 2 处修改：

1，我们用不到编译器的内存管理函数，为节省内存，将 Heap_Size 改成 0，源码如下：

```
;未用到编译器自带的内存管理(malloc,free 等)，设置 Heap_Szie 为 0
Heap_Size       EQU     0x00000000
```

2，寄存器代码不需要调用 SystemInit 函数，因此修改 Reset_Handler 函数，去掉 SystemInit 调用，具体源码如下：

```
Reset_Handler   PROC
                EXPORT  Reset_Handler                   [WEAK]
        ;IMPORT  SystemInit
        ;寄存器代码,不需要在这里调用 SystemInit 函数,故屏蔽掉,库函数版本代码,可以留下
        ;不过需要在外部实现 SystemInit 函数,否则会报错.
        IMPORT  __main
          IF {FPU} != "SoftVFP"
          ;通过 Target 选项卡的 Floating Point Hardware 选项来控制.
          ;如果选择:Not Used,则不编译以下代码(到 ENDIF 结束)
          ;如果选择:Use Single/Double Precision,则编译以下代码
        LDR.W   R0, =0xE000ED88         ; Load address of CPACR register
        LDR     R1, [R0]                ; Read value at CPACR
        ORR     R1,  R1, #(0xF <<20)
        STR     R1, [R0]                ; Wait for store to complete
        ENDIF
        ;LDR     R0, =SystemInit         ;寄存器代码,未用到,屏蔽
        ;BLX     R0                      ;寄存器代码,未用到,屏蔽
        LDR     R0, =__main
        BX      R0
        ENDP
```

这段代码，我们主要加入了开启 STM32H743 硬件 FPU 的代码，以使能 STM32H743 的浮点运算单元，并关闭硬件自动压栈。其中，0xE000ED88 就是协处理器控制寄存器(CPACR)的地址，该寄存器的第 20~23 位用来控制是否支持浮点运算，这里我们全设置为 1，以支持硬件浮点运算。关于 CPACR 寄存器的详细描述，见《STM32H7 编程手册.pdf》第 4.7.1 节。另外，寄存器版本，我们还屏蔽了 SystemInit 函数的调用，如果是 HAL 库版本，可以取消这个函数的注释，并在外部实现 SystemInit 函数。

特别注意：我们在汇编代码里面使能了 FPU，所以在 MDK 里面，我们也要设置使用 FPU，否则可能代码会无法运行，设置方法如下：选择 Options for Target ‘Target1’，打开 Target 选项卡，在 Code Generation 里面，选择 Double Precision，如图 6.1.3.4 所示：

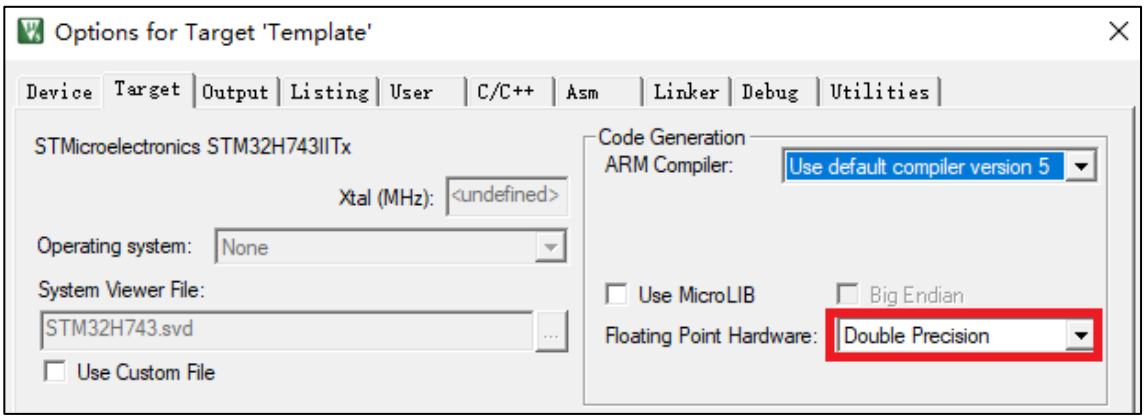


图 6.1.3.4 MDK 开启浮点运算

关于启动文件的说明，我们就介绍这么多，接下来我们看如何添加启动文件到工程里面。我们有两种方法给 MDK 的分组添加文件：1，双击 Project 下的分组名添加。2，进入工程管理界面添加。

这了我们使用方法 1 添加（路径：实验 0-2，新建工程实验-寄存器版本\Drivers\CMSIS\Device\ST\STM32H7xx\Source\Templates\arm），如图 6.1.3.5 所示：

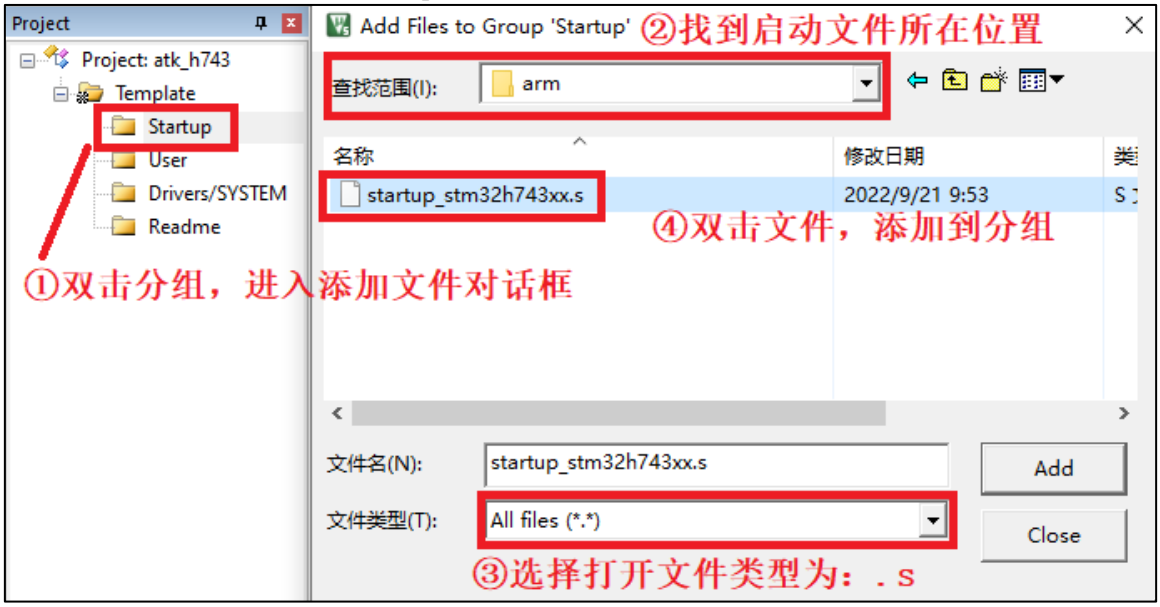


图 6.1.3.5 双击分组添加启动文件（startup_stm32h743xx.s）

上图中，我们也可以点击 Add 按钮进行文件添加。添加完后，点击 Close，完成启动文件添加，得到工程分组如图 6.1.3.6 所示：

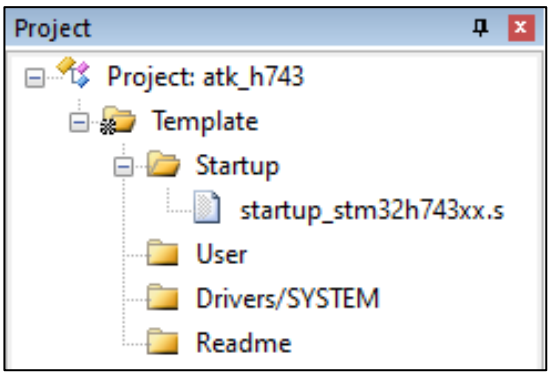


图 6.1.3.6 启动文件添加成功

3. 添加 SYSTEM 源码

这里我们在工程管理界面（方法 2）进行 SYSTEM 源码添加。点击：按钮，进入工程管理界面，选中 Drivers/SYSTEM 分组，然后点击：Add Files，进入文件添加对话框，依次添加 delay.c、sys.c 和 usart.c 到该分组下，如图 6.1.3.6 所示：

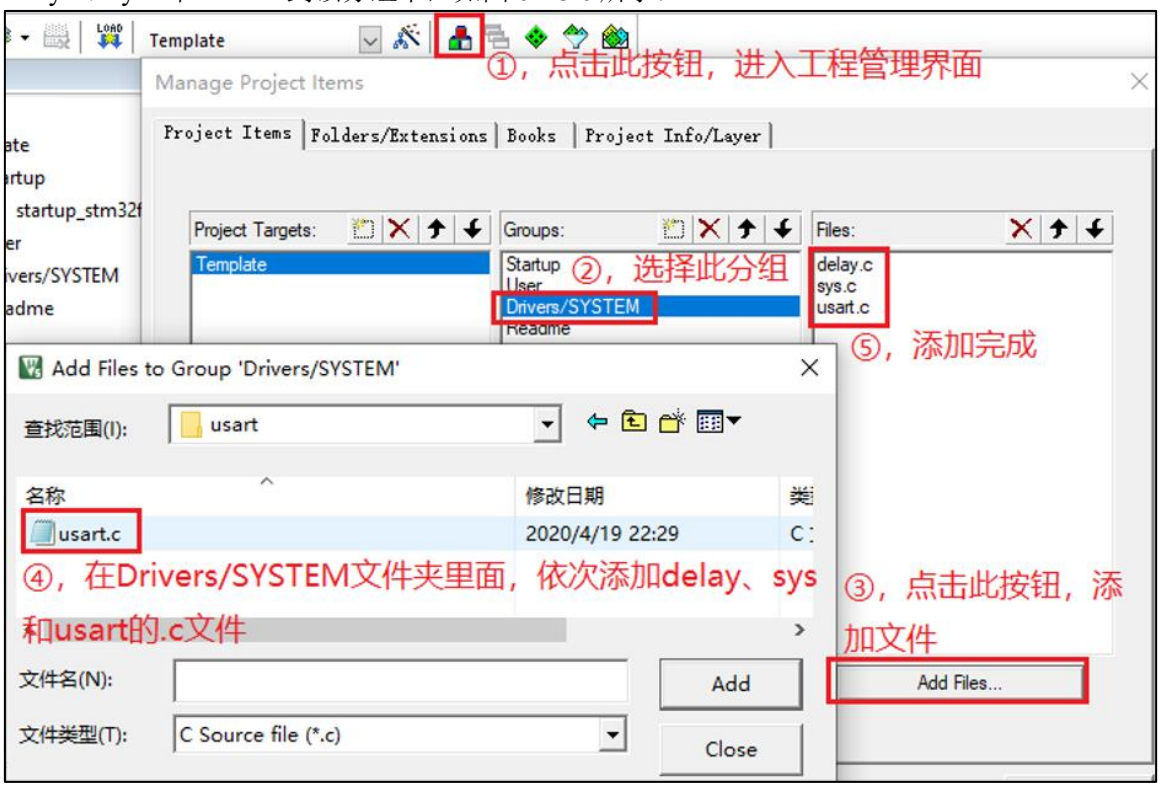


图 6.1.3.6 添加 SYSTEM 源码

注意：这些源码都是在第 6.1.1 小节的第二步拷贝过来的，如果之前没拷贝，是找不到这些源码的。添加完成后，如图 6.1.3.7 所示：

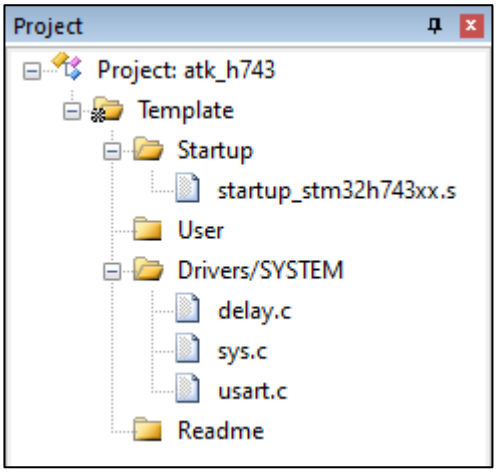


图 6.1.3.7 SYSTEM 源码添加完成

6.1.4 魔术棒设置

为避免编写代码和编译报错，我们需要通过魔术棒对 MDK 工程进行相关设置。在 MDK 主界面，点击：（魔术棒图标，即 Options for Target 按钮），进入工程设置对话框，我们将进行如下几个选项卡的设置。

1. 设置 Target 选项卡

在魔术棒→Target 选项卡里面，我们进行如图 6.1.4.1 所示设置：

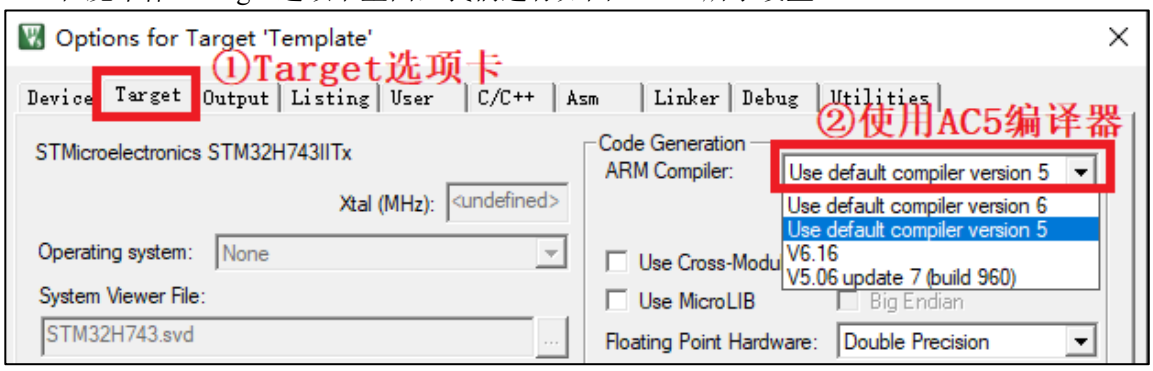


图 6.1.4.1 Target 选项卡设置

上图中，我们选择 ARM Compiler 版本为：Use default compiler version 5（即 AC5 编译器）。

这里我们说明一下 AC5 和 AC6 编译的差异，如表 6.1.4.2 所示：

对比项	AC5	AC6	说明
中文支持	较好	较差	AC6 对中文支持极差，goto definition 无法使用，误报等
代码兼容性	较好	较差	AC6 对某些代码优化可能导致运行异常，需慢慢调试
编译速度	较慢	较快	AC6 编译速度比 AC5 快
语法检查	一般	严格	AC6 语法检查非常严格，代码严谨性较好

表 6.1.4.1 AC5&AC6 简单对比

由于 AC5 对中文支持比较好，且兼容性相对好一点，**为了避免不必要的麻烦，我们推荐大家使用 AC5 编译器**。为了让大家自由选择，我们正点原子的源码，也是支持 AC6 编译器的，不过在选项卡设置上稍有差异，具体差异如表 6.1.4.2 所示：

选项卡	AC5	AC6	说明
Target	选择 AC5 编译器	选择 AC6 编译器	选择对应的编译器
C/C++	Misc Controls 无需设置	Misc Controls 设置： -Wno-invalid-source-encoding	AC6 需设置编译选项以关闭对汉字的错误警告，AC5 则不要

表 6.1.4.2 AC5&AC6 设置差异

2. 设置 Output 选项卡

在魔术棒→Output 选项卡里面，进行如图 6.1.4.2 所示设置：

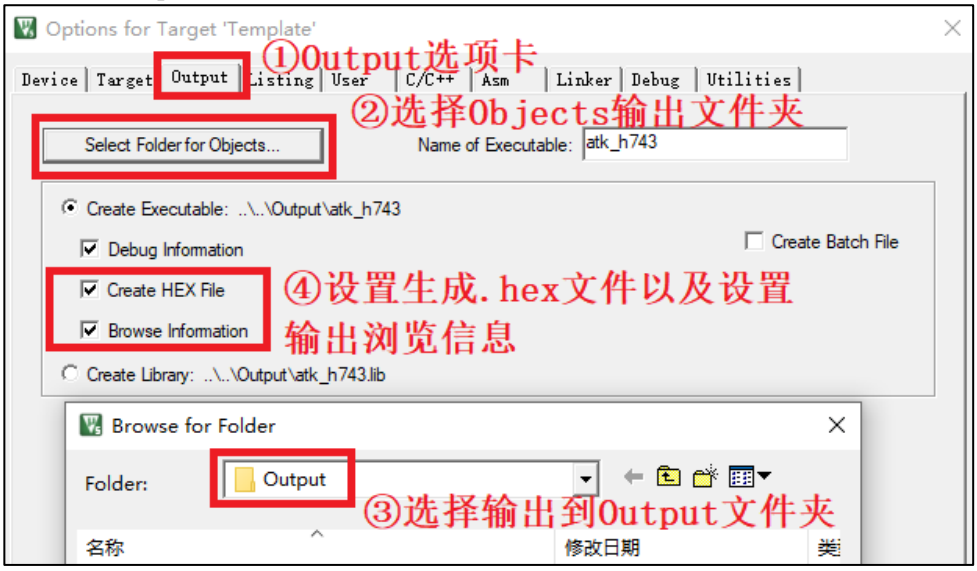


图 6.1.4.2 设置 Output 选项卡

注意，我们勾选：Browse Information，用于输出浏览信息，这样就可以使用 go to definition 查看函数/变量的定义，对我们后续调试代码比较有帮助，如果不需要调试代码，则可以去掉这个勾选，以提高编译速度。

3. 设置 Listing 选项卡

在魔术棒→Listing 选项卡里面，进行如图 6.1.4.3 所示设置：

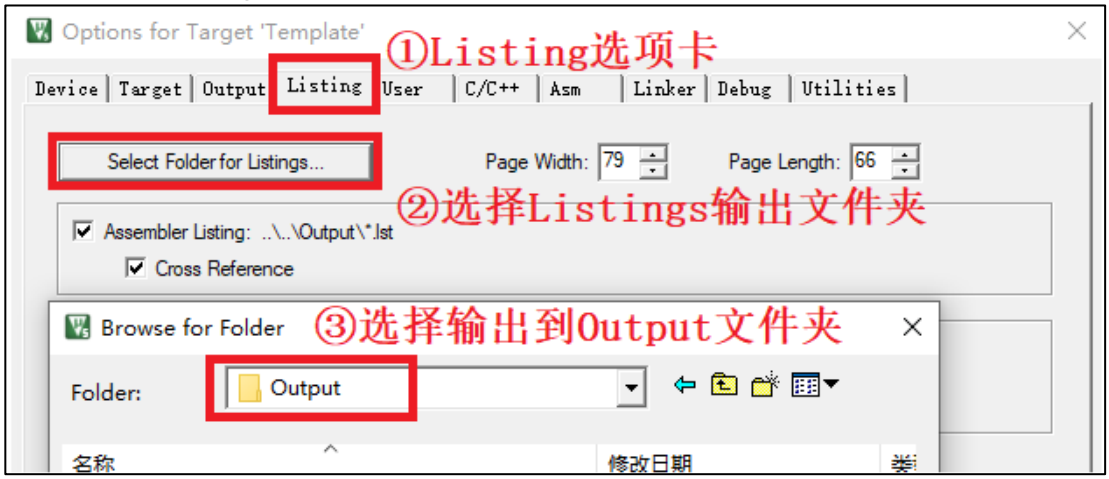


图 6.1.4.3 设置 Listing 选项卡

经过 Output 和 Listing 这两步设置，原来存储在 Objects 和 Listings 文件夹的内容（中间文件）就都改为输出到 Output 文件夹了。

4. 设置 C/C++选项卡

在魔术棒→C/C++选项卡里面，进行如图 6.1.4.4 所示设置：

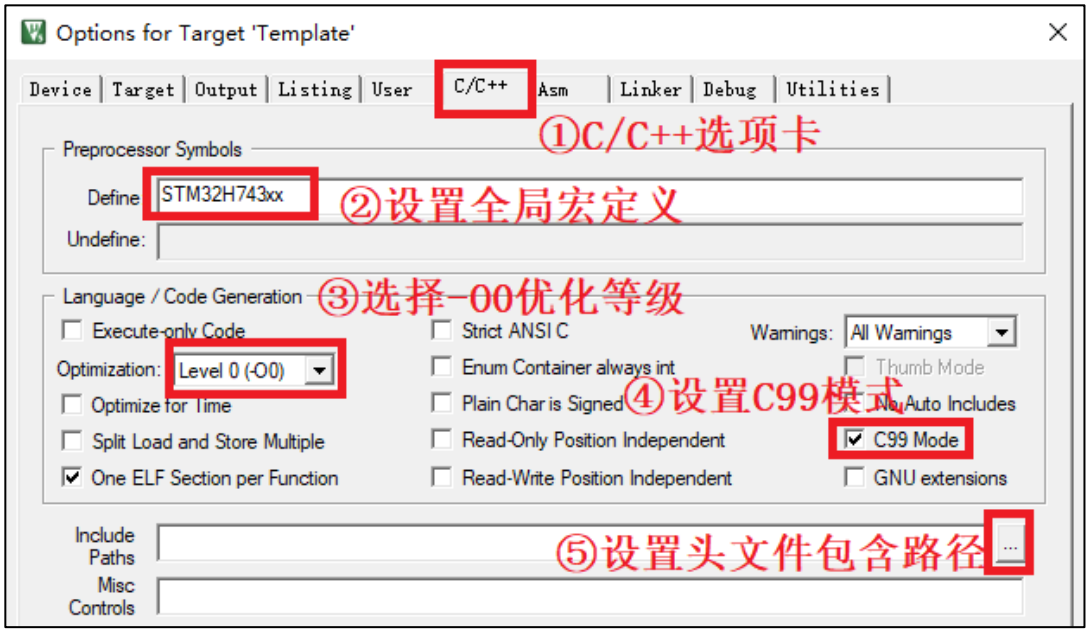


图 6.1.4.4 设置 C/C++选项卡

在②处设置了全局宏定义：STM32H743xx，用于定义所用 STM32 型号，在 stm32h7xx.h 里面会用到该宏定义。

在③处设置了优化等级为-O0，可以得到最好的调试效果，当然为了提高优化效果提升性能并降低代码量，可以设置-O1~-O3，数字越大效果越明显，不过也越容易出问题。注意：当使用 AC6 编译器的时候，这里推荐默认使用-O1 优化。

在④处勾选 C99 模式，即使用 C99 C 语言标准。

在⑤处，我们可以进行头文件包含路径设置，点击此按钮，进行如图 6.1.4.5 所示设置：

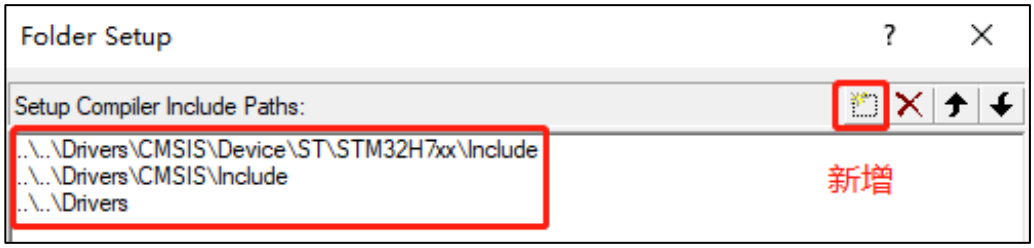


图 6.1.4.5 设置头文件包含路径

上图中我们设置了 4 个头文件包含路径，其中 3 个在 Drivers 文件夹下，一个在 User 文件夹下。为避免频繁设置头文件包含路径，正点原子最新源码的 include 全部使用相对路径，也就是我们只需要在头文件包含路径里面指定一个文件夹，那么该文件夹下的其他文件夹里面的源码，如果全部是使用相对路径，则无需再设置头文件包含路径了，直接在 include 里面就指明了头文件所在。

关于相对路径，这里大家记住 3 点：

1，默认路径就是指 MDK 工程所在的路径，即.uvprojx 文件所在路径（文件夹）

2，“./”表示当前目录（相对当前路径，也可以写做“.\”）

3，“../”表示当前目录的上一层目录（也可以写做“..\”）

举例来说，上图中：..\..\Drivers\CMSIS\Device\ST\STM32H7xx\Include，前面两个“..\”，表示 Drivers 文件夹在当前 MDK 工程所在文件夹（MDK-ARM）的上 2 级目录下，具体解释如图 6.1.4.6 所示：

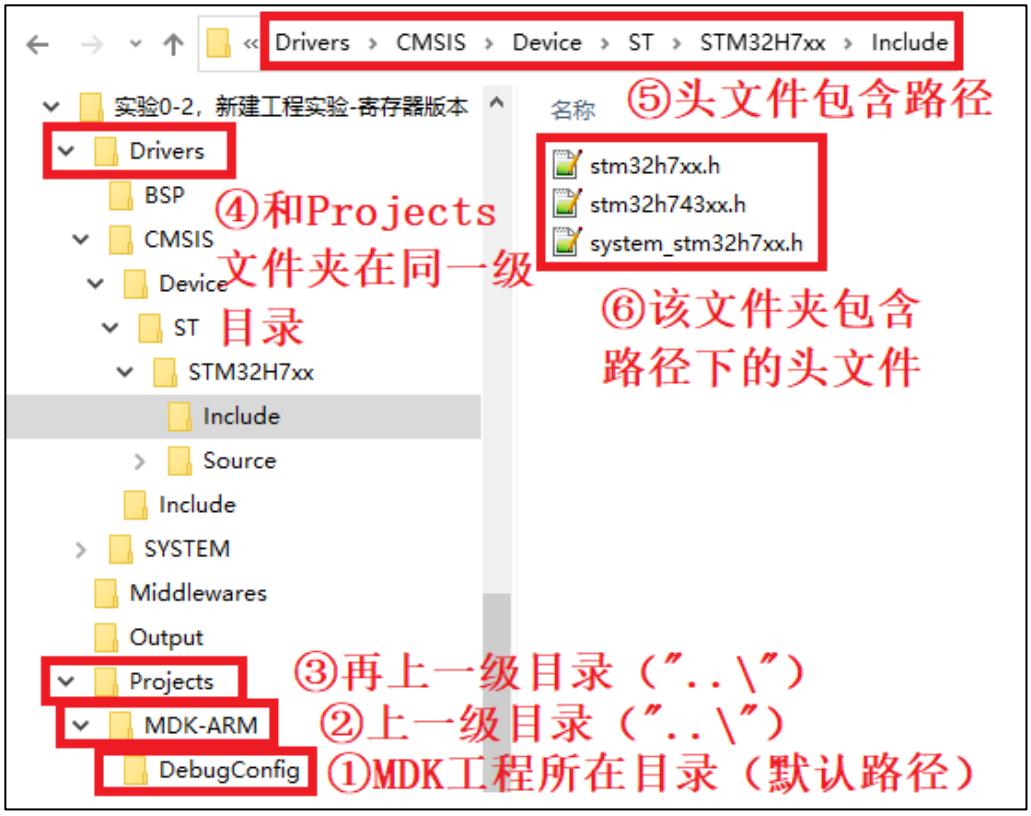


图 6.1.4.6 ..\..\Drivers\CMSIS\Device\ST\STM3H7xx\Include 的解释

上图表示根据头文件包含路径：..\..\Drivers\CMSIS\Device\ST\STM32H7xx\Include，编译器可以找到⑥处所包含的这些头文件，即代码里面可以直接 include 这些头文件使用。

再举个例子，在完成如图 6.1.4.5 所示的头文件包含路径设置以后，我们在代码里面编写：

```
#include "./SYSTEM/sys/sys.h"
```

即表示当前头文件包含路径所指示的 4 个文件夹里面，肯定有某一个文件夹包含了：SYSTEM/sys/sys.h 的路径，实际上就是在 Drivers 文件夹下面，两者结合起来就相当于：

```
#include "../../Drivers/SYSTEM/sys/sys.h"
```

这就是相对路径。它既可以减少头文件包含路径设置（即减少 MDK 配置步骤，免去频繁设置头文件包含路径的麻烦），同时又可以很方便的知道头文件具体在哪个文件夹，因此我们推荐在编写代码的时候使用相对路径。

关于相对路径，我们就介绍这么多，大家搞不明白的可以在网上搜索相关资料学习，也可以在后面的学习，分析我们其他源码，慢慢体会，总之不难，但是好用。

最后，我们如果使用 AC6 编译器，则在图 6.1.4.4 的 Misc Controls 处需要设置：-Wno-invalid-source-encoding，避免中文编码报错，如果使用 AC5 编译器，则不需要该设置!!

5. 设置 Debug 选项卡

在魔术棒→Debug 选项卡里面，进行如图 6.1.4.7 所示设置：

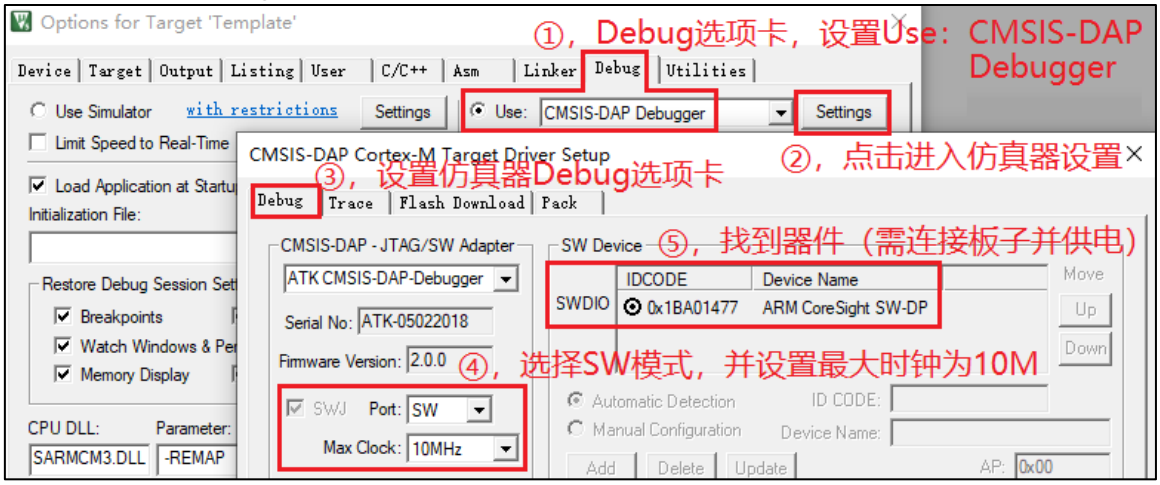


图 6.1.4.7 Debug 选项卡设置

图中，我们选择使用：CMSIS-DAP 仿真器，使用 SW 模式，并设置最大时钟频率为 10Mhz，以得到最高下载速度。当我们将仿真器和开发板连接好，并给开发板供电以后，仿真器就会找到开发板芯片，并在 SW Device 窗口显示芯片的 IDCODE、Device Name 等信息（图中⑤处），当无法找到时，请检查供电和仿真器连接状况。

6. 设置 Utilities 选项卡

在魔术棒→Debug 选项卡里面，进行如图 6.1.4.8 所示设置：

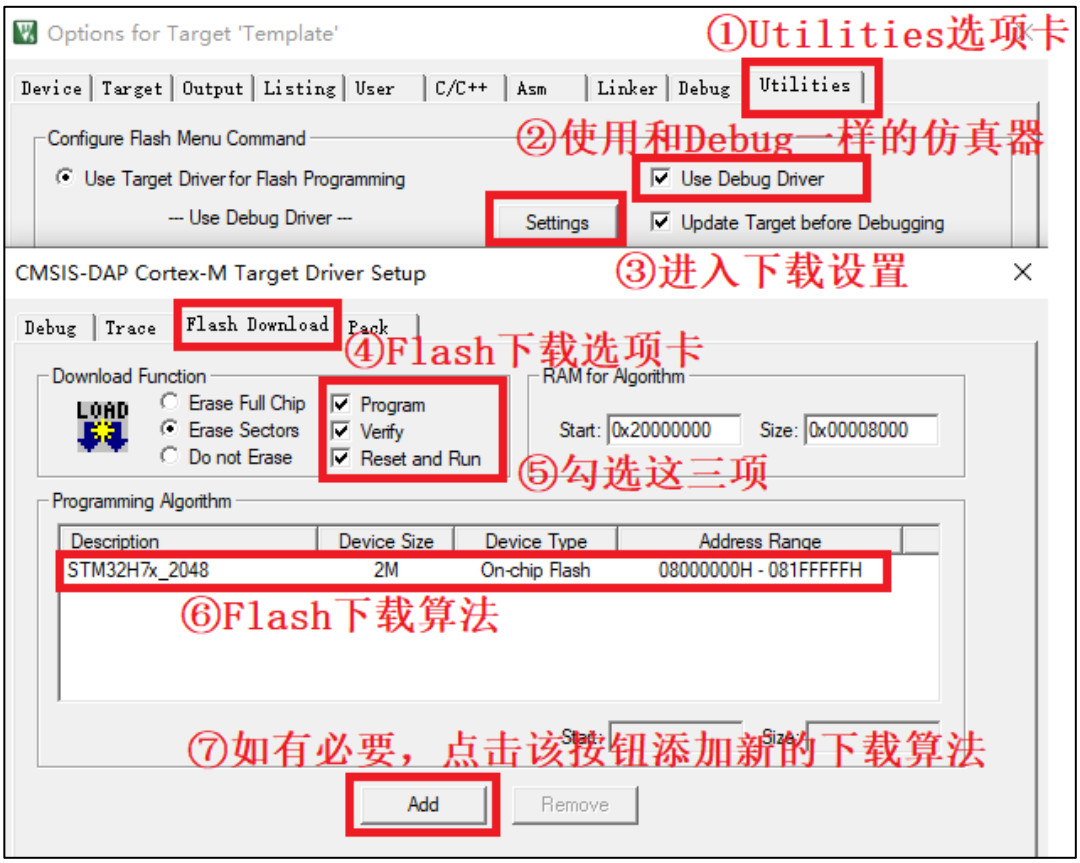


图 6.1.4.8 Utilities 选项卡设置

图中⑥处下载算法，是 MDK 默认添加的，针对 STM32H743 系列产品。一般我们用这个即可，如果⑥处没有下载算法，则点击⑦处按钮添加即可，执行添加一下下载算法即可（名字和⑥处的算法名字一样）。

6.1.5 添加 main.c，并编写代码

在 MDK 主界面，点击：，新建一个 main.c 文件，并保存在 User 文件夹下。然后双击 User 分组，弹出添加文件的对话框，将 User 文件夹下的 main.c 文件添加到 User 分组下。得到如图 6.1.5.1 所示的界面：

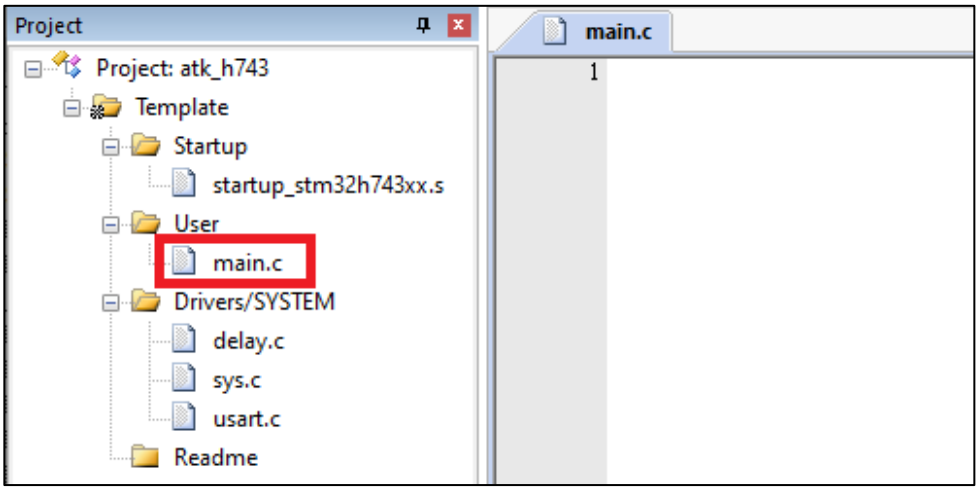


图 6.1.5.1 在 User 分组下加入 main.c 文件

至此，我们就可以开始编写我们自己的代码了。我们在 main.c 文件里面输入如下代码：

```
#include "./SYSTEM/sys/sys.h"
#include "./SYSTEM/usart/usart.h"
```

```
#include "./SYSTEM/delay/delay.h"

int main(void)
{
    uint8_t t = 0;
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(100, 115200);                /* 串口初始化为 115200 */
    while (1)
    {
        printf("t:%d\r\n", t);
        delay_ms(500);
        t++;
    }
}
```

此部分代码，在 A 盘→4，程序源码→2，标准例程-HAL 库版本→ 实验 0 基础入门实验→实验 0-2，新建最工程实验-寄存器版本→User→main.c 里面有，大家可以自己输入，也可以直接拷贝。强烈建议自己输入，以加深对程序的理解和印象!!

注意，这里的 include 就是使用的相对路径，关于相对路径，请参考前面 C/C++选项卡设置章节进行学习。

编写完 main.c 以后，我们点击： （Rebuild）按钮，编译整个工程，编译结果如图 6.1.5.2 所示：

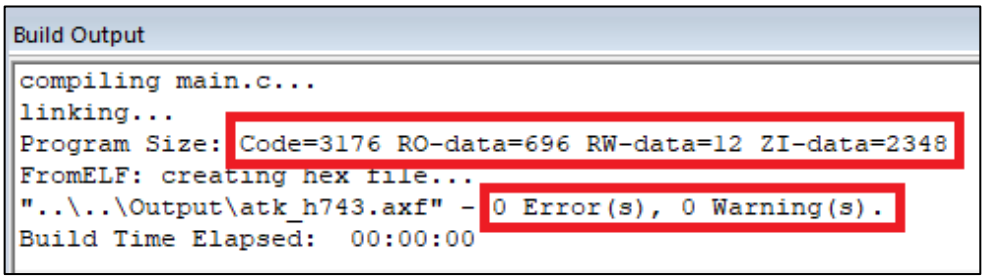


图 6.1.5.2 编译结果

编译结果提示：代码总大小（Porgram Size）为：FLASH 占用 3884 字节（Code + RO + RW），SRAM 占用 2360 字节（RW + ZI）；并成功创建了 Hex 文件（可执行文件，放在 Output 目录下）；编译 0 错误，0 警告。

注意：如果编译提示有错误/警告，请根据提示，从第一个错误/警告开始解决，直到 0 错误 0 警告。如果出错，很有可能是之前的操作存在问题，请对照教程找问题。

另外，我们在 Readme 分组下还没有添加任何文件，由于只是添加一个说明性质的文件（.txt），并不是工程必备文件，因此这里我们就不添加了，开发板光盘的源码我们是有添加的，大家可以去参考一下。

至此，新建寄存器版本 MDK 工程完成。

6.2 下载验证

这里我们继续使用 DAP 仿真器下载，在 MDK 主界面，点击： （下载按钮，也可以按键盘快捷键：F8），就可以将代码下载到开发板，如图 6.2.1 所示：

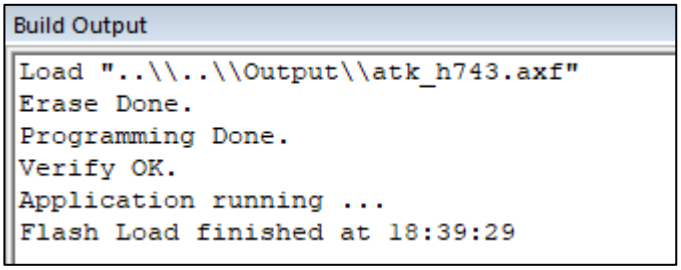


图 6.2.1 下载成功

上图提示：Application running…，则表示代码下载成功，且开始运行。此时，我们打开串

口调试助手，并设置好端口号（COM 号）和波特率（115200），就可以看到打印出来的 t 值，如图 6.2.2 所示：

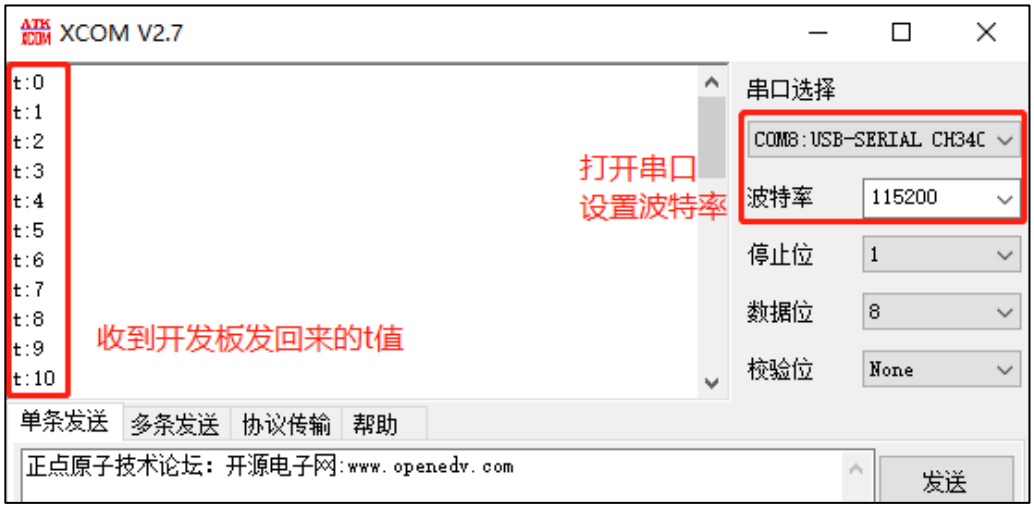


图 6.2.2 收到开发板发回来的数据

说明我们的程序运行正常，下载验证无误。

第七章 认识 HAL 库

HAL，英文全称 Hardware Abstraction Layer，即硬件抽象层。HAL 库是 ST 公司提供的外设驱动代码的驱动库，用户只需要调用库的 API 函数，便可间接配置寄存器。我们要写程序控制 STM32 芯片，其实最终就是控制它的寄存器，HAL 库就为了更方便我们去控制寄存器，从而节约开发时间。

本章将分为如下几个小节：

7.1 初识 STM32 HAL 库
7.2 HAL 库驱动包
7.3 HAL 库框架结构
7.4 如何使用 HAL 库
7.5 HAL 库使用注意事项

7.1 初识 STM32 HAL 库

STM32 开发中常说的 HAL 库开发，指的是利用 HAL 库固件包里封装好的 c 语言编写的驱动文件，来实现对 STM32 内部和外围电器元件的控制的过程。但只有 HAL 库还不能直接驱动一个 STM32 的芯片，其它的组件已经由 ARM 与众多芯片硬件、软件厂商制定的通用的软件开发标准 CMSIS 实现了，本文只简单介绍这个标准，等大家熟悉开发后再研究这个框架。

简单地了解 HAL 库的发展和作用，可以方便学习者确定 HAL 库是否适合作为学习者自己长期开发 STM32 的工具，以降低开发、学习的成本。

7.1.1 CMSIS 标准

根据一些调查研究表明，软件开发已经被嵌入式行业公认为最主要的开发成本,为了降低这个成本，ARM 与 Atmel、IAR、KEIL、SEGGER 和 ST 等诸多芯片和软件工具厂商合作，制定了一个将所有 Cortex 芯片厂商的产品的软件接口标准化的标准 CMSIS（Cortex Microcontroller Software Interface Standard）。下面来看 ARM 官方提供的 CMSIS 规范架构图，如图 7.1.1.1 所示：

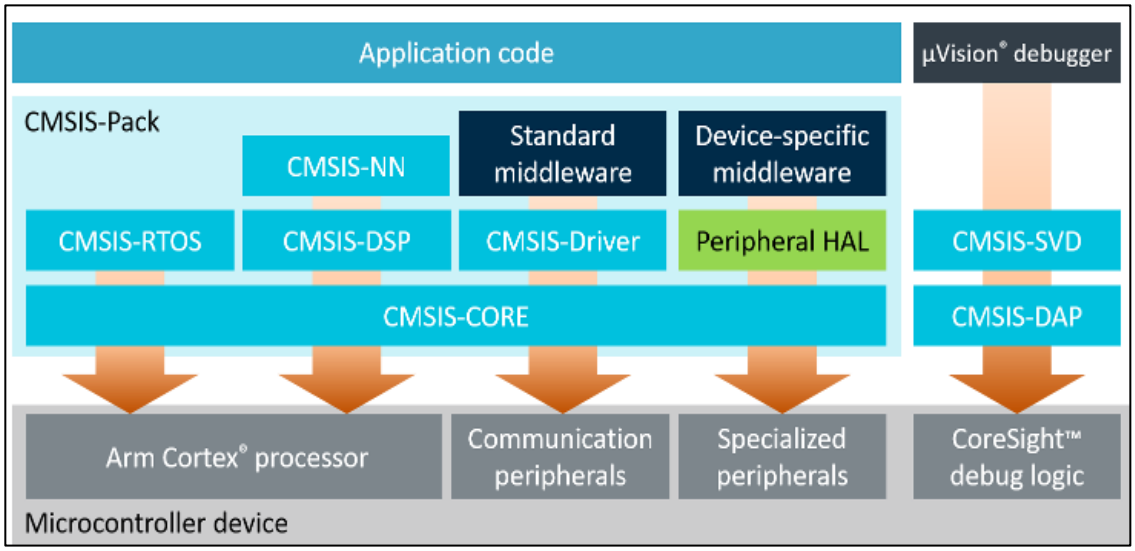


图 7.1.1.1 CorteX 芯片的 CMSIS 分级实现

从图中可以看出这个标准分级明显，从用户程序到内核底层实现做了分层。按照这个分级，HAL 库属于 CMSIS-Pack 中的“Peripheral HAL”层。CMSIS 规定的最主要的 3 个部分为：核内外设访问层（由 ARM 负责实现）、片上外设访问层和外设访问函数（后面两个由芯片厂商

负责实现）。ARM 整合并提供了大量的模版，各厂商根据自己的芯片差异修改模版，这其中包括汇编文件 startup_device.s、system_.h 和 system_.c 这些与初始化和系统相关的函数。

结合 STM32H7 的芯片来说，其 CMSIS 应用程序的简单结构框图，不包括实时操作系统和中间设备等组件，其结构如图 7.1.1.2 所示。

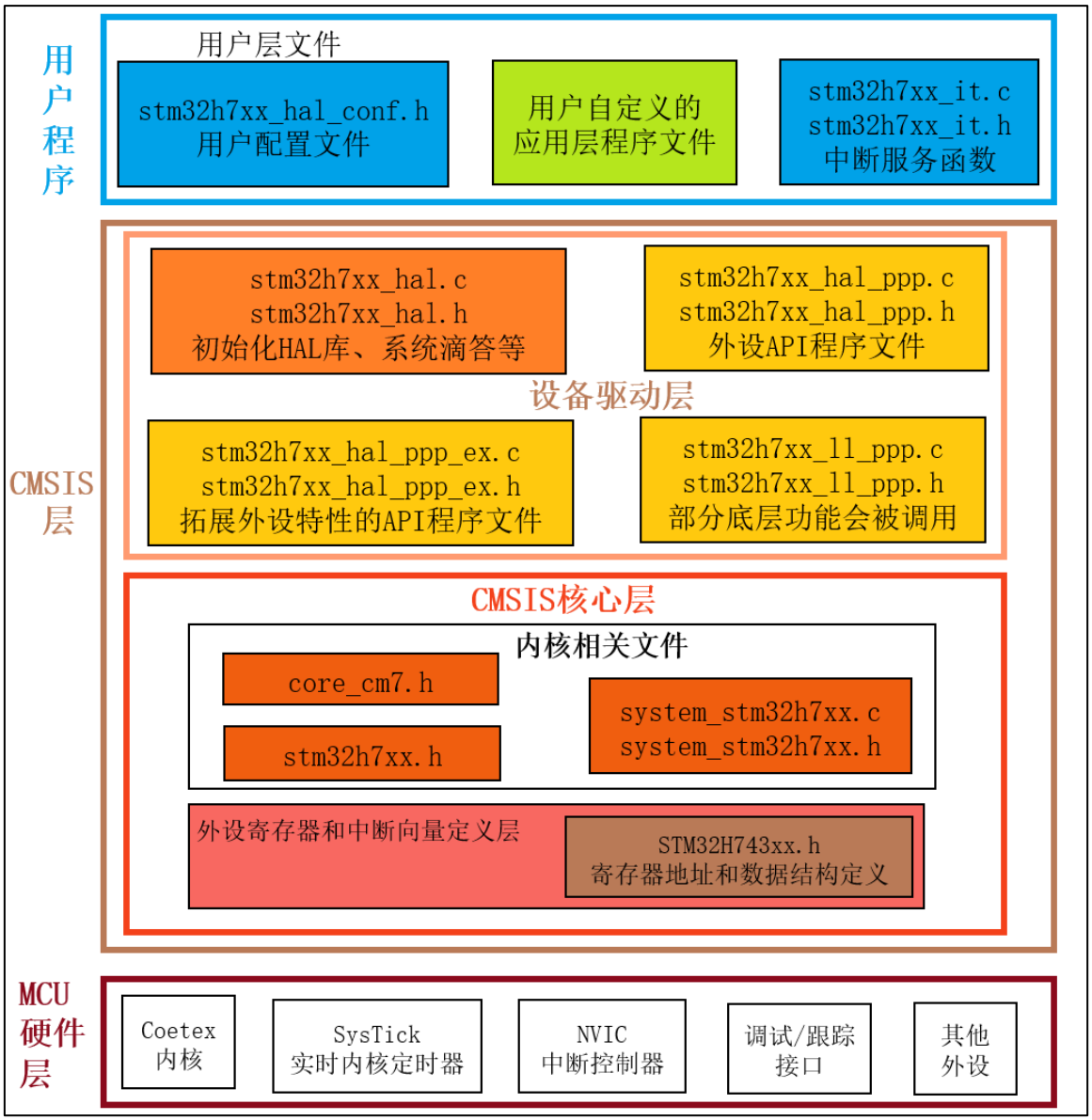


图 7.1.1.2 CMSIS 分级下的 STM32H7 的文件分布

上面的框架是根据我们现在已经学习到的知识回过头来作的一个总结，这里只是作简单介绍，告诉大家它们之间存在一定联系，关于组成这些部分的文件、文件的作用及各文件如何组合、各分层的作用和意义，我们会在今后的学习过程中慢慢学习。

7.1.2 HAL 库简介

库函数的引入，大大降低 STM 主控芯片开发的难度。ST 公司为了方便用户开发 STM32 芯片开发提供了三种库函数，从时间产生顺序是：标准库、HAL 库和 LL 库。目前 ST 已经逐渐暂停对部分标准库的支持，ST 的库函数维护重点对角已经转移到 HAL 库和 LL 库上，下面我们分别为这三种库作一下简单的介绍。

1. 标准外设库（Standard Peripheral Libraries）

标准外设库（Standard Peripherals Library）是对 STM32 芯片的一个完整的封装，包括所有标准器件外设的器件驱动器，是 ST 最早推出的针对 STM 系列主控的库函数。标准库的设计的

初衷是减少用户的程序编写时间，进而降低开发成本。几乎全部使用 C 语言实现并严格按照“Strict ANSI-C”、MISRA-C 2004 等多个 C 语言标准编写。但标准外设库仍然接近于寄存器操作，主要就是将一些基本的寄存器操作封装成了 C 函数。开发者仍需要关注所使用的外设是在哪个总线之上，具体寄存器的配置等底层信息。

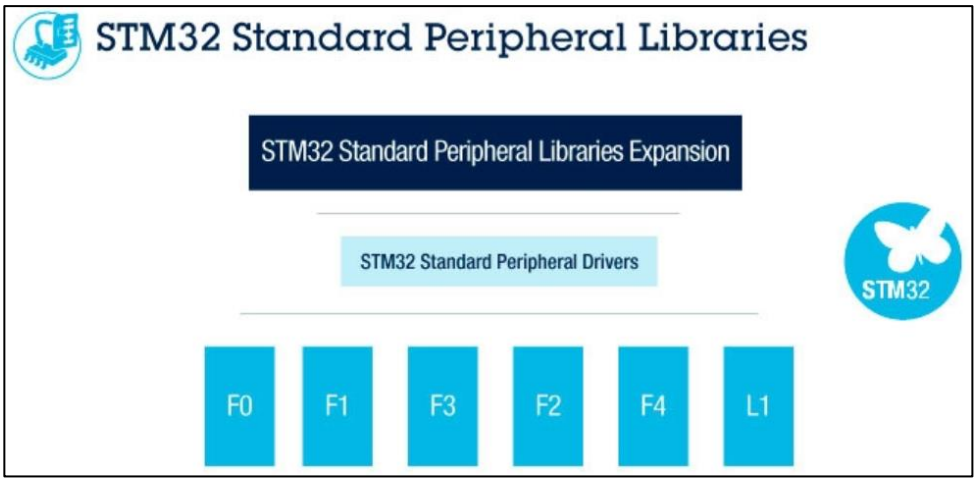


图 7.1.2.1 ST 的标准库函数家族

ST 为各系列提供的标准外设库稍微有些区别。例如，STM32F1x 的库和 STM32F3x 的库在文件结构上就有些不同，此外，在内部的实现上也稍微有些区别，这个在具体使用（移植）时，需要注意一下！但是，不同系列之间的差别并不是很大，而且在设计上是相同的。

STM32 的标准外设库涵盖以下 3 个抽象级别：

- 包含位，位域和寄存器在内的完整的寄存器地址映射。
- 涵盖所有外围功能（具有公共 API 的驱动器）的例程和数据结构的集合。
- 一组包含所有可用外设的示例，其中包含最常用的开发工具的模板项目。

关于更详细的信息，可以参考 ST 的官方文档《STM32 固件库使用手册中文翻译版》，文档中对于标准外设库函数命名、文件结构等都有详细的说明，这里我们就不多介绍了。

值得一提的是由于 STM32 的产品性能及标准库代码的规范和易读性以及例程的全覆盖性，使 STM32 的开发难度大大下降，更多。但 ST 从 L1 以后的芯片 L0、L4 和 F7 等系列就没有再推出相应的标准库支持包了。

2.HAL 库

HAL 是 Hardware Abstraction Layer 的缩写，即硬件抽象层。是 ST 为可以更好的确保跨 STM32 产品的最大可移植性而推出的 MCU 操作库。这种程序设计由于抽离应用程序和硬件底层的操作，更加符合跨平台和多人协作开发的需要。

HAL 库是基于一个非限制性的 BSD 许可协议（Berkeley Software Distribution）而发布的开源代码。ST 制作的中间件堆栈（USB 主机和设备库，STemWin）带有允许轻松重用的许可模式， 只要是在 ST 公司的 MCU 芯片上使用，库中的中间件(USB 主机/设备库，STemWin)协议栈即被允许修改，并可以反复使用。至于基于其它著名的开源解决方案商的中间件（FreeRTOS，FatFs，LwIP 和 PolarSSL）也都具有友好的用户许可条款。

HAL 库是从 ST 公司从自身芯片的整个生产生态出发，为了方便维护而作的一次整合，以改变标准外设库带来各系列芯片操作函数结构差异大、分化大、不利于跨系列移植的情况。相比标准外设库，STM32Cube HAL 库表现出更高的抽象整合水平，HAL 库的 API 集中关注各外设的公共函数功能，这样便于定义一套通用的用户友好的 API 函数接口，从而可以轻松实现从一个 STM32 产品移植到另一个不同的 STM32 系列产品。但由于封闭函数为了适应最大的兼容性，HAL 库的一些代码实际上的执行效率要远低于寄存器操作。但即便如此，HAL 库仍是 ST 未来主推的库。

3.LL 库

LL 库（Low Layer）目前与 HAL 库捆绑发布，它的设计为比 HAL 库更接近于硬件底层的操作，代码更轻量级，代码执行效率更高的库函数组件，可以完全独立于 HAL 库来使用，但

LL 库不匹配复杂的外设，如 USB 等。所以 LL 库并不是每个外设都有对应的完整驱动配置程序。使用 LL 库需要对芯片的功能有一定的认知和了解，它可以：

- 独立使用，该库完全独立实现，可以完全抛开 HAL 库，只用 LL 库编程完成。
- 混合使用，和 HAL 库结合使用。

对于 HAL 库和 LL 库的关系，如图 7.1.2.2 Cube 的软件框架所示，可以看出它们设计为彼此独立的分支，但又同属于 HAL 库体系。

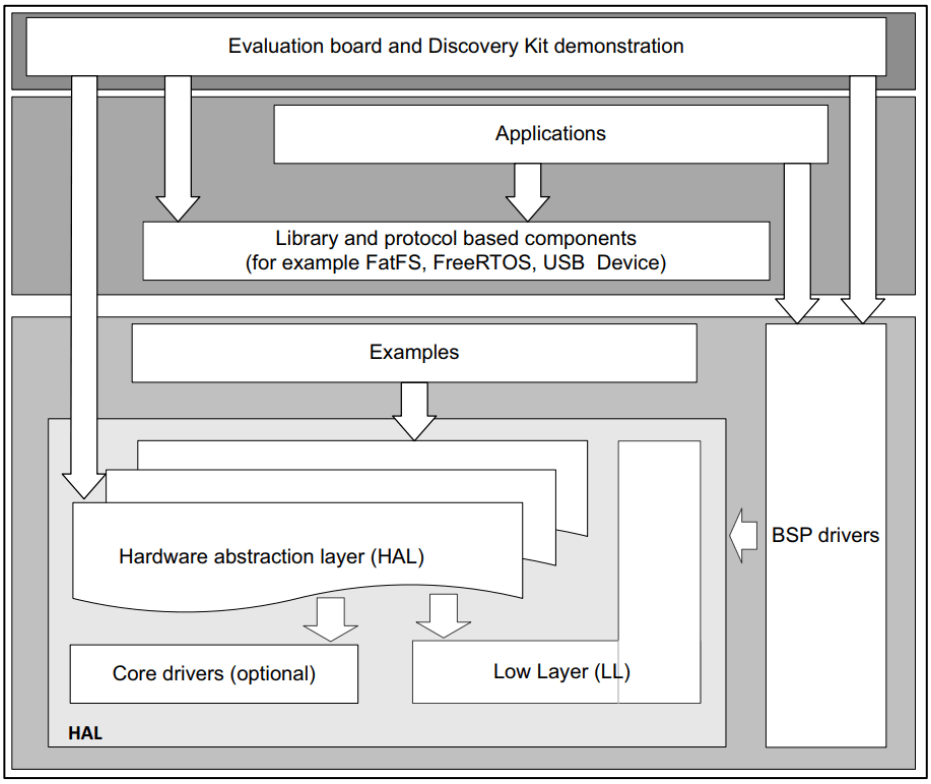


图 7.1.2.2 Cube 的软件框架

通过以上简介我们对目前主流的 STM32 开发库有了一个初步的印象。标准库和 HAL 库、LL 库完全相互独立，HAL 库更倾向于外设通用化，扩展组件中解决芯片差异操作部分；LL 倾向于最简单的寄存器操作，ST 在未来还将重点维护和建设 HAL 库，标准库已经部分停止更新。HAL 库和 LL 库的应用将是未来的一个趋势。

7.1.3 HAL 库能做什么

用过标准库的朋友应该知道，使用标准库可以忽略很多芯片寄存器的细节，根据提供的接口函数快速配置和使用一个 STM32 芯片，使用 HAL 库也是如此。不论何种库，本质都是配置指定寄存器使芯片工作在我们需要的工作模式下。HAL 库在设计的时候会更注重软硬件分离。HAL 库的 API 集中关注各个外设的公共函数功能，便于定义通用性更好、更友好的 API 函数接口，从而具有更好的可移植性。HAL 库写的代码在不同的 STM32 产品上移植，非常方便。

我们需要学会调用 HAL 库的 API 函数，配置对应外设按照我们的要求工作，这就是 HAL 库能做的事。但是无论库封装得多高级，最终还是要通过配置寄存器来实现。所以我们学习 HAL 库的同时，也建议同时学习外设的工作原理和寄存器的配置。只有掌握了原理，才能更好的使用 HAL 库，一旦发生问题也能更快速了定位和解决问题。

HAL 库还可以和 STM32CubeMX（图形化软件配置工具）配套一起使用，开发者可以使用该工具进行可视化配置，并且自动生成配置好的初始化代码，大大的节省开发时间。

7.2 HAL 库驱动包

HAL 库是一系列封装好的驱动函数，本节将从下载渠道、固件包的内容分析及在实际开发中用到的几个文件的详细介绍。

7.2.1 如何获取 HAL 库固件包

HAL 库是 ST 推出的 STM32Cube 软件生态下的一个分支。STM32Cube 是 ST 公司提供的一套免费开发工具和 STM32Cube 固件包，旨在通过减少开发工作、时间和成本来简化开发人员的工作，并且覆盖整个 STM32 产品。它包含两个关键部分：

1、允许用户通过图形化向导来生成 C 语言工程的图形配置工具 STM32CubeMX。可以通过 CubeMX 实现方便地下载各种软件或开发固件包。

2、包括由 STM32Cube 硬件抽象层（HAL），还有一组一致的中间件组件（RTOS、USB、FAT 文件系统、图形、TCP/IP 和以太网），以及一系列完整的例程组成的 STM32Cube 固件包。

ST 提供了多种获取固件包的方法。本节只介绍从 ST 官方网站上直接获取固件库的方法。网页登陆：www.st.com，在打开的页面中依次选择：“Tools & Software”->“Ecosystems”->“STM32Cube”->新页面->选择“Prodcut selector”，具体如下图所示：

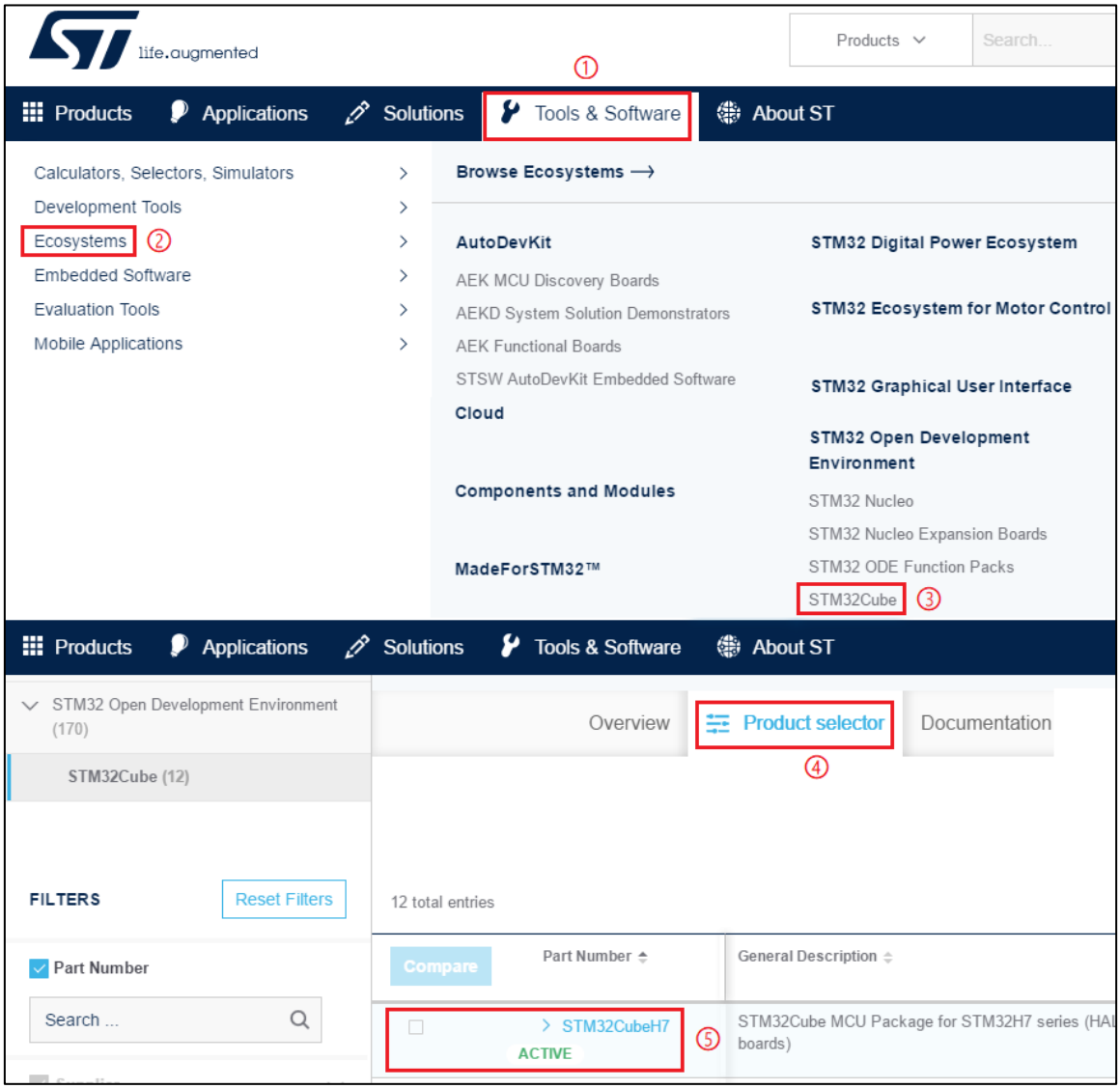


图 7.2.1.1 找到 STM32CubeH7 固件包的下载位置

在展开的页面中选择我们需要和固件，这里展开“STM32CubeH7”即可看到我们需要的 H7 的固件包，按下图操作，在新的窗口中拉到底部，选择适合自己的下载方式，注册帐号即可获取相应的驱动包。

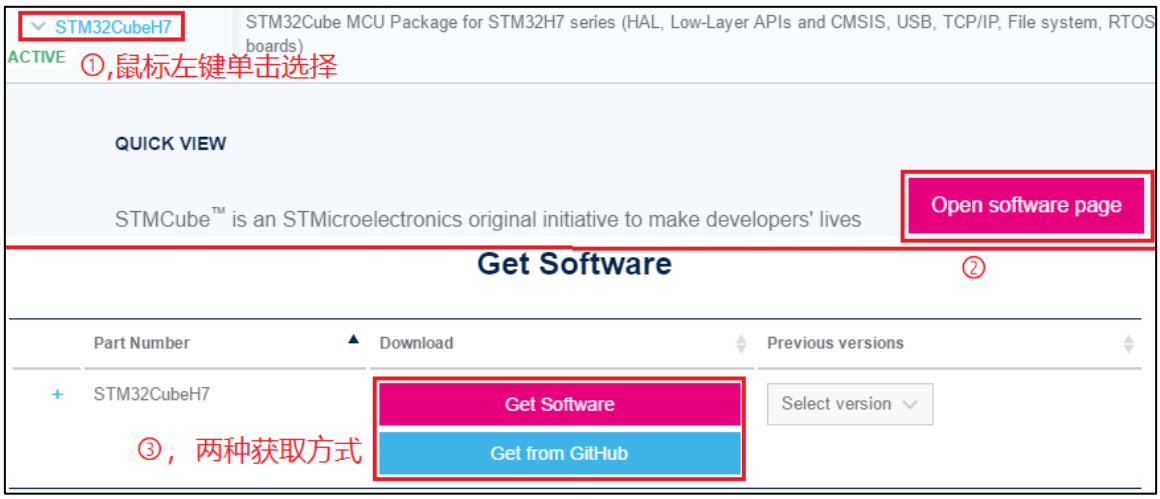


图 7.2.1.2 下载 STM32CubeH7 固件包

STM32Cube 固件包，我们已经给大家下载好并且放到 A 盘→8，STM32 参考资料→1，STM32CubeH7 固件包，当前固件包版本是：STM32Cube_FW_H7_V1.9.0。因为现在是 STM32H743 的学习，所以我们准备好的固件包是 H7 的。大家要根据自己学习的芯片，下载对应的固件包。如果需要最新的固件包，大家按照上述的方法到官网重新获取即可。

7.2.2 STM32Cube 固件包分析

STM32Cube 固件包完全兼容 STM32CubeMX。对于图形配置工具 STM32CubeMX 入门使用，由于需要 STM32F1 基础才能入门使用，所以我们安排在后面第十章给大家讲解。本小节，我们主要讲解 STM32Cube 固件包的结构。

解压 STM32CubeH7 固件包后，我们看看其目录结构，如图 7.2.2.1 所示。

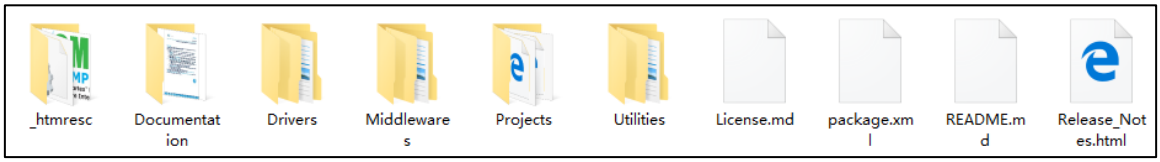


图 7.2.2.1 STM32CubeH7 固件包的目录结构

下面对 STM32CubeH7 固件包进行简要介绍。对于 Documentation 文件夹，里面是一个 STM32CubeH7 英文说明文档，这里我们就不做过多解释。接下来我们通过几个表格依次来介绍一下 STM32CubeH7 中几个关键的文件夹。

（1）Drivers 文件夹

Drivers 文件夹包含 BSP，CMSIS 和 STM32H7xx_HAL_Driver 三个子文件夹。三个子文件夹具体说明请参考下表 7.2.2.1：

Drivers 文件夹	BSP 文件夹	也叫板级支持包，用于适配 ST 官方对应的开发板的硬件驱动程序，每一种开发板对应一个文件夹。例如触摸屏，LCD，SRAM 以及 EEPROM 等板载硬件资源等驱动。这些文件针只匹配特定的开发板使用，不同开发板可能不能直接使用。
	CMSIS 文件夹	顾名思义就是符合 CMSIS 标准的软件抽象层组件相关文件。文件夹内部文件比较多。主要包括 DSP 库(DSP_LIB 文件夹)，Cortex-M 内核及其设备文件（Include 文件夹），微控制器专用头文件/启动代码/专用系统文件等(Device 文件夹)。在新建工程的时候，会使用到这个文件夹内部很多文件。

	STM32H7xx HAL Driver 文件夹	这个文件夹非常重要，它包含了所有的 STM32H7xx 系列 HAL 库头文件和源文件，也就是所有底层硬件抽象层 API 声明和定义。它的作用是屏蔽了复杂的硬件寄存器操作，统一了外设的接口函数。该文件夹包含 Src 和 Inc 两个子文件夹，其中 Src 子文件夹存放的是.c 源文件，Inc 子文件夹存放的是与之对应的.h 头文件。每个.c 源文件对应一个.h 头文件。源文件名称基本遵循 stm32H7xx_hal_ppp.c 定义格式，头文件名称基本遵循 stm32H7xx_hal_ppp.h 定义格式。比如 gpio 相关的 API 的声明和定义在文件 stm32H7xx_hal_gpio.h 和 stm32H7xx_hal_gpio.c 中。该文件夹的文件在我们新建工程章节都会使用到，我们后面会做详细介绍。

表 7.2.2.1 Drivers 文件夹介绍

（2）Middlewares 文件夹

该文件夹下面有 ST 和 Third_Party 2 个子文件夹。ST 文件夹下面存放的是 STM32 相关的一些文件，包括 STemWin 和 USB 库等。Third_Party 文件夹是第三方中间件，这些中间价都是非常成熟的开源解决方案。具体说明请见下表 7.2.2.2：

Middlewares 文件夹	ST 子文件夹	STemWin 文件夹	STemWin 工具包。Segger 提供。
		STM32_USB_Device_Library 文件夹	USB 从机设备支持包。
		STM32_USB_Host_Library 文件夹	USB 主机设备支持包。
	Third_Party 子文件夹	FatFs 文件夹	FAT 文件系统支持包。采用的 FATFS 文件系统。
		FreeRTOS 文件夹	FreeRTOS 实时系统支持包。
		LibJPEG 文件夹	基于 C 语言的 JPEG 图形解码支持包。
		LwIP 文件夹	LwIP 网络通信协议支持包。

表 7.2.2.2 Middlewares 文件夹介绍

（3）Projects 文件夹

该文件夹存放的是 ST 官方的开发板的适配例程，每个文件夹对应一个 ST 官方的 Demo 板，根据型号的不同提供 MDK 和 IAR 等类型的例程。里面有很多实例，读者可以根据自己的需要来作为参考。这里大家注意，每个工程下面都有一个 MDK-ARM 子文件夹，该子文件夹内部会有名称为 Project.uvprojx 的工程文件，我们只需要双击它就可在 MDK 中打开工程。

（4）Utilities 文件夹

该文件夹是一些公用组件，也是主要为 ST 官方的 Demo 板提供的，在我们的例程中使用的不多。有兴趣的同学可以深入研究一下，这里我们不做过多介绍。

（5）其它几个文件

文件夹中还有几个单独的文件，用于声明软件版本或者版权信息，我们使用 ST 的芯片已经默认得到这个软件的版权使用授权，可以简单了解一下各文件的内容，实际项目中我们一般不添加。

License.md：用于声明软件版权信息的文件。

package.xml：描述固件包版本信息的文件。

Release_Notes.html：超文本文件，用浏览器打开可知它是对固件包的补充描述和固件版本

更新的记录说明。

7.2.3 CMSIS 文件夹关键文件

上一节中我们对 STM32cube 固件包的主要目录结构做了分析。这一小节在上一小节的基础上，我们来分析一下 CMSIS 文件夹：由命名可知该文件夹和 7.1.1 小节中提到的 CMSIS 标准是一致的，CMSIS 为软件包的内容制定了标准，包括文件目录的命名和内容构成，CMSIS 版本 5.7.0 的规定软件包目录如表 7.2.3.1 所示：

文件/目录		描述
LICENSE.txt		Apache 2.0 授权的许可文件
ARM.CMSIS.pdsc		描述该 CMSIS 包的文件
Device		基于 Arm Cortex-M 处理器设备的 CMSIS 参考实现
CMSIS 组件	Documentation	这个数据包的描述文档
	Core	CMSIS-Core（Cortex-M）相关文件的用户代码模板,在 ARM.CMSIS.pdsc 中引用
	Core_A	CMSIS-Core（Cortex-A）相关文件的用户代码模板，在 ARM.CMSIS.pdsc 中引用
	DAP	CMSIS-DAP 调试访问端口源代码和参考实现
	Driver	CMSIS 驱动程序外设接口 API 的头文件
	DSP_Lib	CMSIS-DSP 软件库源代码
	NN	CMSIS-NN 软件库源代码
	Include	CMSIS-Core(Cortex-M)和 CMSIS-DSP 需要包括的头文件等。
	Lib	包括 CMSIS 核心（Cortex-M）和 CMSIS-DSP 的文件
	Pack	CMSIS-Pack 示例，包含包含设备支持、板支持和软件组件的软件包示例。
	RTOS	CMSIS-RTOS 版本 1 以及 RTX4 参考实现
	RTOS2	CMSIS-RTOS 版本 2 以及 RTX5 参考实现
	SVD	CMSIS-SVD 样例，规定开发者、制造商、工具制造商的分工和职能
	Utilities	PACK.xsd(CMSIS-Pack 架构文件),PackChk.exe(检查软件包的工具)，CMSIS-SVD.xsd(MSIS-SVD 架构文件),SVDConv.exe(SVD 文件的转换工具)

表 7.2.3.1 CMSIS v5.7.0 的文件夹规范

知道了 CMSIS 规定的组件及其文件目录的大概内容后，我们再来看看 ST 提供的 CMSIS 文件夹，如上节提到的，它的位置是“STM32Cube_FW_H7_V1.9.0\Drivers\CMSIS”。打开文件夹内容如图 7.2.3.1 所示，可以发现它的目录结构完全按照 CMSIS 标准执行，仅仅是作了部分删减。

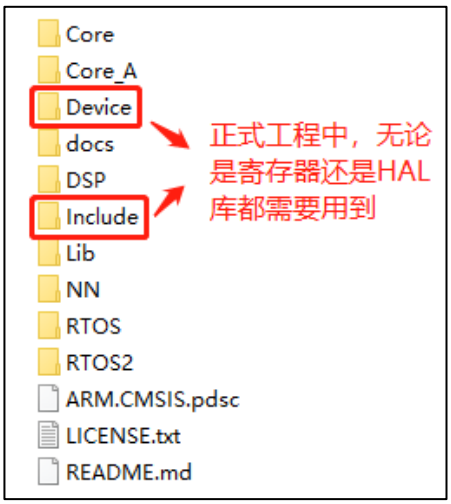


图 7.2.3.1 STM32CubeH7 固件包的 CMSIS 文件夹

CMSIS 文件夹中的 Device 和 Include 这两个文件夹中的文件是我们工程中最常用到的。下面对这两个文件夹作简单的介绍：

（1）Device 文件夹

Device 文件夹关键文件介绍如下表 7.2.3.1 所示：

文件	描述
stm32h7xx.h	该文件内容看起来不多，却非常重要，是所有 stm32h7 系列的顶层头文件。使用 STM32H7 任何型号的芯片，都需要包含这个头文件。该文件包含了很多条件定义和常用的枚举变量类型，与宏定义配合，选择性包含某一特定的 STM32H7 系列芯片的头文件。这个文件使我们在使用 STM32H7 系列的不同型号芯片时，不需要每次都修改工程头文件，只需要修改宏定义并增加特定型号芯片的头文件即可快速选择使用不同类型的 H7 芯片。
stm32h743xx.h	STM32H743 系列芯片通用的片上外设访问层头文件。只要我们使用 STM32H743 系列芯片，都需要包括这个头文件。这个文件的主要作用是定义声明寄存器以及封装内存操作，以结构体和宏定义标识符的形式。
system_stm32h7xx.c system_stm32h7xx.h	主要是声明和定义了系统初始化函数 SystemInit 以及系统时钟更新函数 SystemCoreClockUpdate。SystemInit 函数的作用是进行时钟系统的一些初始化操作以及中断向量表偏移地址设置，但它并没有设置具体的时钟值，这是与标准库的最大区别，在使用标准库的时候，SystemInit 函数会帮我们配置好系统时钟配置相关的各个寄存器。在启动文件 startup_stm32h743xx.s 中会设置系统复位后，直接调用 SystemInit 函数进行系统初始化。SystemCoreClockUpdate 函数是在系统时钟配置进行修改后，调用这个函数来更新全局变量 SystemCoreClock 的值，变量 SystemCoreClock 是一个全局变量，开放这个变量可以方便我们在用户代码中直接使用这个变量来进行一些时钟运算。
startup_stm32h743xx.s	STM32H743 系列芯片的启动文件，每个系列都有与之对应的启动文件。启动文件的作用主要是进行堆栈的初始化，中断向量表以及中断函数定义等。启动文件有一个很重要的作用就是系统复位后引导进入 main 函数。后面会细讲。

表 7.2.3.1 Device 文件夹关键文件介绍

表 7.1.3.1 列出的文件都是正式工程中必须的文件。固件包的 CMSIS 文件包括了所有 STM32H7 芯片型号的文件，而我们只用到 STM32H743 系列，所以只是挑我们用到的系列文件来讲。

（2）Include 文件夹

Include 文件夹存放了符合 CMSIS 标准的 Cortex-M 内核头文件。想要深入学习内核的朋友可以配合内核相关的手册去学习。对于 STM32H7 的工程，我们只要把我们需要的添加到工程即可，需要的头文件有：cmsis_armcc.h、cmsis_armclang.h、cmsis_compiler.h、cmsis_version.h、core_cm7.h 和 mpu_armv7.h。这几个头文件，对比起来，我们会比较多接触的是 core_cm7.h。

core_cm7.h 是内核底层的文件，由 ARM 公司提供，包含一些 AMR 内核指令，如软件复位，开关中断等功能。今后在需要的例程再去讲解其程序，现在之所以提到，是因为它包含了一个重要的头文件 stdint.h。

7.2.4 stdint.h 简介

stdint.h 是从 c99 中引进的一个标准 C 库的文件。在 2000 年 3 月，ANSI 采纳了 C99 标准。ANSI C 被几乎所有广泛使用的编译器（如：MDK、IAR）支持。多数 C 代码是在 ANSI C 基础上写的。任何仅使用标准 C 并且没有和任何硬件有依赖的代码实际上能保证在任何平台上用遵循 C 标准的编译器编译成功。就是说这套标准不依赖硬件，独立于任何硬件，可以跨平台。

stdint.h 可以在 MDK 安装目录下找到，如 MDK5 安装在 C 盘时，可以在路径：C:\Keil_v5\ARM\ARMCC\include 找到。stdint.h 的作用就是提供了类型定义，其部分类型定义代码如下：

```
/* exact-width signed integer types */
typedef   signed          char int8_t;
typedef   signed short    int int16_t;
typedef   signed          int int32_t;
typedef   signed          __INT64 int64_t;

/* exact-width unsigned integer types */
typedef unsigned          char uint8_t;
typedef unsigned short    int uint16_t;
typedef unsigned          int uint32_t;
typedef unsigned          __INT64 uint64_t;
```

在今后的程序，我们都将会使用这些类型，比如：uint32_t（无符号整型）、int16_t 等。

7.3 HAL 库框架结构

这一节我们将简要分析一下 HAL 驱动文件夹下的驱动文件，帮助大家快速认识 HAL 库驱动的构成，旨在帮大家快速认识 HAL 库的函数的一些常用形式，在遇到 HAL 库时能根据名字大致推断该函数的用法，本部分内容不要求大家在学习完本节后完全记住。

7.3.1 HAL 库文件夹结构

HAL 库头文件和源文件在 STM32Cube 固件包的 STM32H7xx_HAL_Driver 文件夹中，打开该文件夹，如图 7.3.1.1 所示。

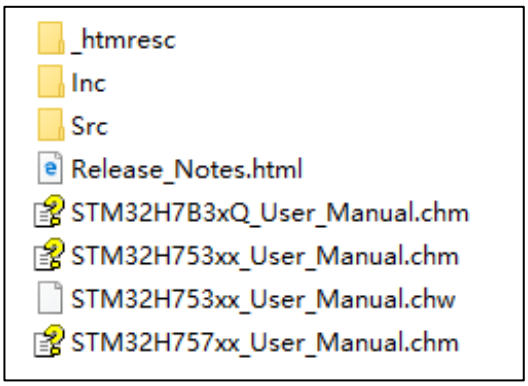


图 7.3.1.1 STM32H7xx_HAL_Driver 文件夹目录结构

STM32H7xx_HAL_Driver 文件夹下的 Src（Source 的简写）文件夹存放是所有外设的驱动程序源码，Inc（Include 的简写）文件夹存放的是对应源码的头文件。Release_Notes.html 是 HAL 库的版本更新信息。最后三个是库的用户手册，这个需要可以去熟悉一下，查阅起来很方便。

打开 Src 和 Inc 文件夹，大家会发现基本都是 stm32h7xx_hal_和 stm32h7xx_ll_开头的.c 和.h 文件。刚学 HAL 库的朋友可能会说，stm32h7xx_hal_开头的是 HAL 库，我能理解。那么 stm32h7xx_ll_开头的文件又是什么？stm32h7xx_ll_开头的文件是前面介绍过的 LL 库的文件。

7.3.2 HAL 库文件介绍

HAL 库关键文件介绍如下表 7.3.2.1 所示，表中 ppp 代表任意外设。

文件	描述
sm32h7xx_hal.c stm32h7xx_hal.h	初始化 HAL 库，（比如 HAL_Init,HAL_DeInit, HAL_Delay 等），主要实现 HLA 库的初始化、系统滴答，HAL 库延时函数、IO 重映射和 DBGMCU 功能等。
stm32h7xx_hal_conf.h HAL 库中本身没有这个文件，可以自行定义，也可以直接使用《Inc》文件夹下 stm32f1xx_hal_conf_template.h 的内容作为参考模版	HAL 的用户配置文件，stm32h7xx_hal.h 引用了这个文件，用来对 HAL 库进行裁剪。由于 Hal 库的很多配置都是通过预编译的条件宏来决定是否使用这一 HAL 库的功能，这也是当前的主流库如 LWIP /FreeRTOS 等的做法，无需修改库函数的源码，通过使能/不使能一些宏来实现库函数的裁剪。
stm32hxx_hal_def.h	通用 HAL 库资源定义，包含 HAL 的通用数据类型定义，声明、枚举，结构体和宏定义。如 HAL 函数操作结果返回值类型 HAL_StatusTypeDef 就是在这个文件中定义的。
stm32h7xx_hal_ppp.c stm32h7xx_hal_ppp.h	外设驱动函数。对于所有的 STM32 该驱动名称都相同，ppp 代表一类外设，包含该外设的操作 API 函数。例如：当 ppp 为 adc 时，那么这个文件就是 stm32h7xx_hal_adc.c/h，可以分别在 Src/Inc 目录下找到。包含该外设的操作 API 函数。其中 stm32h7xx_hal_cortex.c/.h 比较特殊，它是一些 Cortex 内核通用函数声明和定义，例如中断优先级 NVIC 配置，MPU，系统软复位以及 Systick 配置等。
stm32h7xx_hal_ppp_ex.c stm32h7xx_hal_ppp_ex.h	外设特殊功能的 API 文件，作为标准外设驱动的功能补充和扩展。针对部分型号才有的特殊外设作功能扩展，或外设的实现功能与标准方式完全不同的情况下作重新初始化的备用接口。ppp 的含义同标准外设驱动。
stm32h7xx_ll_ppp.c stm32h7xx_ll_ppp..h	LL 库文件，在一些复杂外设中实现底层功能，在部分 stm32h7xx_hal_ppp.c 中被调用。

表 7.3.2.1 HAL 库关键文件介绍

以上是 HAL 库最常见的文件的列表，在 Src/Inc 下面还有 Legacy 文件夹，用于特殊外设的

补充说明。我们的教程中用到的比较少，这里不展开描述。

不止文件命名有一定规则，stm32h7xx_hal_ppp (c/h)中的函数和变量命名也严格按照命名规则，如表 7.3.2.2 所示的命名规则在大部分情况下都是正确的：

文件名	stm32h7xx_hal_ppp (c/h)	stm32h7xx_hal_ppp_ex (c/h)
函数名	HAL_PPP_Function HAL_PPP_FeatureFunction_MODE	HAL_PPPEx_Function HAL_PPPEx_FeatureFunction_MODE
外设句柄	PPP_HandleTypedef	无
初始化参数结构体	PPP_InitTypeDef	PPP_InitTypeDef
枚举类型	HAL_PPP_StructnameTypeDef	无

表 7.3.2.2 HAL 库函数、变量命名规则

对于 HAL 的 API 函数，常见的有以下几种：

- **初始化/反初始化函数**:HAL_PPP_Init()， HAL_PPP_DeInit()
- **外设读写函数**:HAL_PPP_Read()，HAL_PPP_Write()，HAL_PPP_Transmit()，HAL_PPP_Receive()
- **控制函数**:HAL_PPP_Set ()，HAL_PPP_Get ()
- **状态和错误**:HAL_PPP_GetState ()， HAL_PPP_GetError ()

HAL 库封装的很多函数都是通过定义好的结构体将参数一次性传给所需函数，参数也有一定的规律，主要有以下三种：

• 配置和初始化用的结构体

一般为 PPP_InitTypeDef 或 PPP_ ConfTypeDef 的结构体类型，根据外设的寄存器设计成易于理解和记忆的结构体成员。

• 特殊处理的结构体

专为不同外设而设置的，带有“Process”的字样，实现一些特异化的中间处理操作等。

• 外设句柄结构体

HAL 驱动的重要参数，可以同时定义多个句柄结构以支持多外设多模式。HAL 驱动的操作结果也可以通过这个句柄获得。有些 HAL 驱动的头文件中还定义了一些跟这个句柄相关的一些外设操作。如用外设结构体句柄与 HAL 定义的一些宏操作配合，即可实现一些常用的寄存器位操作。

宏定义结构	用途
__HAL_PPP_ENABLE_IT(__HANDLE__, __INTERRUPT__)	使能外设中断
__HAL_PPP_DISABLE_IT(__HANDLE__, __INTERRUPT__)	禁用外设中断
__HAL_PPP_GET_IT (__HANDLE__, __INTERRUPT__)	获取外设的某一中断源
__HAL_PPP_CLEAR_IT (__HANDLE__, __INTERRUPT__)	清除外设中断
__HAL_PPP_GET_FLAG (__HANDLE__, __FLAG__)	获取外设的状态标记
__HAL_PPP_CLEAR_FLAG (__HANDLE__, __FLAG__)	清除外设的状态标记
__HAL_PPP_ENABLE(__HANDLE__)	使能某一外设
__HAL_PPP_DISABLE(__HANDLE__)	禁用某一外设
__HAL_PPP_XXXX (__HANDLE__, __PARAM__)	针对外设的特殊操作
__HAL_PPP_GET_IT_SOURCE (__HANDLE__, __INTERRUPT__)	检查外设的中断源

表 7.3.2.3 HAL 库驱动部分与外设句柄相关的宏

但对于 SYSTICK/NVIC/RCC/FLASH/ GPIO 这些内核外设或共享资源，不使用 PPP_HandleTypedef 这类外设句柄进行控制，如：HAL_GPIO_Init() 只需要初始化的 GPIO 编号和具体的初始化参数。

```
HAL_StatusTypeDef HAL_GPIO_Init (GPIO_TypeDef* GPIOx, GPIO_InitTypeDef *Init)
{
/* GPIO 初始化程序…… */
}
```

最后要分享的是 HAL 库的回调函数，这部分允许用户重定义，并在其中实现用户自定义的功能，也是我们使用 HAL 库的最常用的接口之一：

回调函数	举例
HAL_PPP_MspInit() / _DeInit()	举例: HAL_USART_MspInit() 由 HAL_PPP_Init()这个 API 调用，主要在这个函数中实现外设对应的 GPIO、时钟、DMA 和中断开启的配置和操作。
HAL_PPP_ProcessCpltCallback	举例: HAL_USART_TxCpltCallback 由外设中断或 DMA 中断调用，调用时 API 内部已经实现中断标记的清除的操作，用户只需要专注于自己的软件功能实现即可。
HAL_PPP_ErrorCallback	举例: HAL_USART_ErrorCallback 外设或 DMA 中断中发生的错误，用于作错误处理。

表 7.3.2.4 HAL 库驱动中常用的回调函数接口

至此，我们大概对 HAL 库驱动文件的一些通用格式和命名规则有了初步印象，记住这些规则可以帮助我们快速对 HAL 库的驱动进行归类和判定这些驱动函数的用法。

ST 官方给我们提供了快速查找 API 函数的帮助文档。在路径“STM32Cube_FW_H7_V1.9.0\Drivers\STM32H7xx_HAL_Driver”下有几个 chm 格式的文档，根据我们开发板主控芯片 STM32H743IIT6 我们没有找到直接可用的，但可以查看型号接近的：STM32H753xx_User_Manual.chm。双击打开后，可以看到左边目录下有四个主题，我们来查看 Modules。以外设 GPIO 为例，讲一下怎么使用这个文档。点击 GPIO 外设的主题下的 IO operation functions /functions 看看里面的 API 函数接口描述，如图 7.3.2.1 所示。

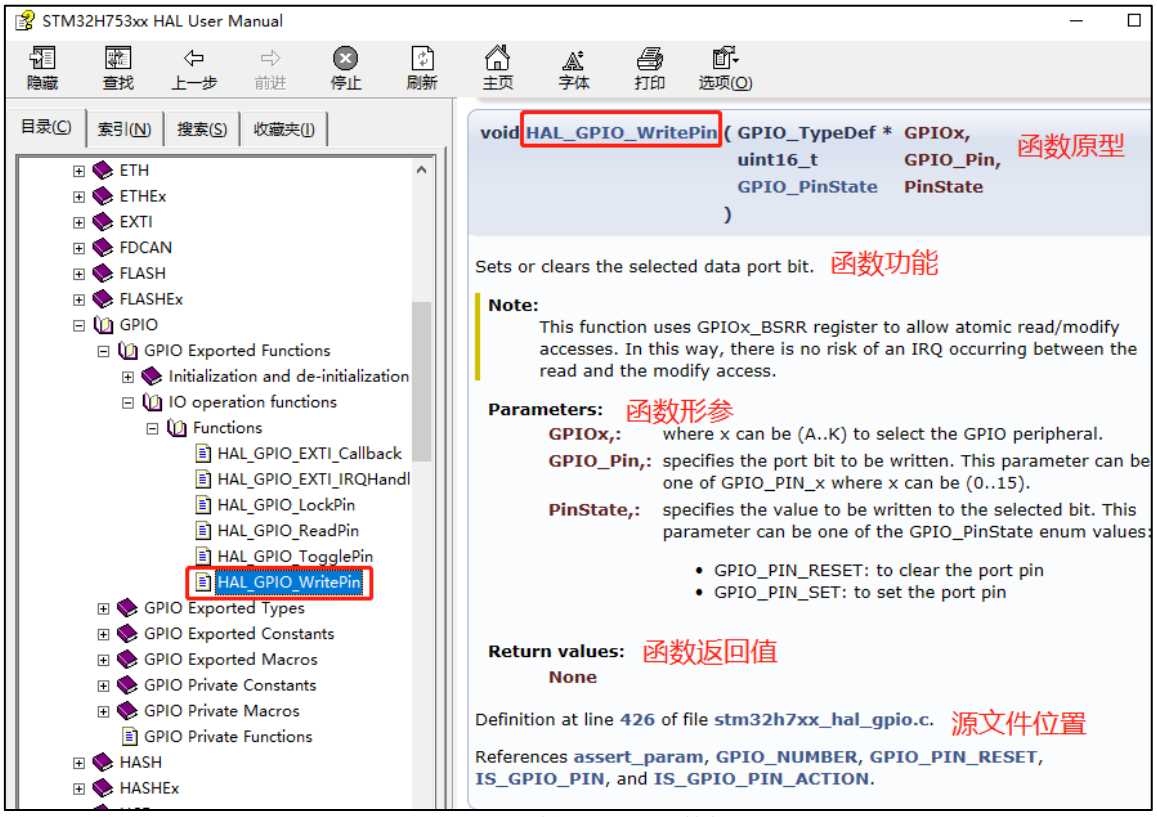


图 7.3.2.1 文档的 API 函数描述

这个文档提供的信息很全，不看源码都可以直接使用它来编写代码，还给我们指示源码位置，非常方便。大家多翻一下其他主题了解一下文档的信息结构，很容易使用。

下面举个例子，比如我们要让 PB4 输出高电平。先看函数功能，HAL_GPIO_WritePin 函数就是我们的 GPIO 口输出设置函数。

函数有三个形参：

第一个形参是 GPIO_TypeDef *GPIOx，形参描述说：x 可以是 A 到 K 之间任何一个，而我们是 PB4 引脚，所以第一个形参确认是 GPIOB。

第二个形参是 uint16_t GPIO_Pin，看形参描述：该参数可以是 GPIO_PIN_x，x 可以 1 到 15，那么我们第二个形参就是 GPIO_PIN_4。

第三个形参是 GPIO_PinState PinState，看形参描述：该参数可以是枚举里的两个数，一个是 GPIO_PIN_RESET：表示该位清零，另一个是 GPIO_PIN_SET：表示设置该位，即置 1，我们要输出 1，所以要置 1 该位，那么我们第三个形参就是 GPIO_PIN_SET。

最后看函数返回值：None，没有返回值。

所以最后得出我们要调用的函数是：

```
HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_SET);
```

文档的使用就讲到这。

7.4 如何使用 HAL 库

我们要先知道 STM32 芯片的某个外设的性能和工作模式，才能借助 HAL 库来帮助我们编程，甚至修改 HAL 库来适配我们的开发项目。HAL 库的 API 虽多，但是查找和使用有规律可循，只要学会其中一个，其他的外设就是类似的，只是添加自己的特性的 API 而已。

7.4.1 学会用 HAL 库组织开发工具链

需要按照芯片使用手册建议的步骤去配置芯片。HAL 库驱动提供了芯片的驱动接口，但我们需要强调一个概念是使用 HAL 库的开发是对芯片功能的开发，而不是开发这个库，也不是由这个库能就直接开发。如果我们对芯片的功能不作了解的话，仍然不知道按照怎样的步骤和寻找哪些可用的接口去实现想要实现的功能。

ST 提供芯片使用手册《STM32H7xx 参考手册_V7（英文版）.pdf》告诉我们使用某一外设功能时如何具体地去操作每一个用到的寄存器的细节，后面我们的例程讲解过程也会结合这个手册来分析配置过程。

嵌入式的软件开发流程总遵循以下步骤：组织工具链、编写代码、生成可执行文件、烧录到芯片、芯片根据内部指令执行我们编程生成的可执行代码。

在 HAL 库学习前期，建议以模仿和操作体验为基础，通过例程来学习如何配置和驱动外设。下面根据我们后续要学习的工程梳理出来的基于 CMSIS 的一个 HAL 库应用程序文件结构，帮助读者学习和体会这些文件的组成意义，如下表 7.4.1.1 所示。

类别	文件名	描述	是否必须
用户程序文件	main.c	存放 main 函数，不一定要在这个文件	否
	main.h	包含头文件、声明等作用，已删除	否
	stm32f7xx_it.c	用户中断服务函数存放文件，不一定放到这个文件，可删除	否
	stm32f7xx_it.h		否
	stm32f7xx_hal_conf.h	用户配置文件	是
	stm32f7xx_hal_msp.c	回调函数存放文件，已删除	否
设备驱动层	sm32h7xx_hal.c	HAL 库的初始化、系统滴答，HAL 库延时函数等功能	是
	stm32h7xx_hal.h		是
	stm32hxx_hal_def.h	通用 HAL 库资源定义	是
	stm32h7xx_hal_ppp.c	外设的操作 API 函数文件	是
	stm32h7xx_hal_ppp.h		是
	stm32h7xx_hal_ppp_ex.c	拓展外设特性的 API 函数文件	是
	stm32h7xx_hal_ppp_ex.h		是
	stm32h7xx_II_ppp.c	LL 库文件，在一些复杂外设中实现底层功能	是
	stm32h7xx_II_ppp..h		是
CMSIS 核心层	stm32h7xx.h	STM32H7 系列的顶层头文件	是
	stm32h743xx.h	STM32H743 系列片上外设头文件	是
	system_stm32h7xx.c	主要存放系统初始化函数 SystemInit	是
	system_stm32h7xx.h		是

	startup_stm32h743xx.s	启动文件，运行到 main 函数前的准备	是
	core_cm7.h	内核寄存器定义，如 Systick、SCB 等	是
	cmsis_armcc.h	内核头文件，一般都不需要去了解	是
	cmsis_armclang.h		
	cmsis_compiler.h		
	cmsis_version.h		
	mpu_armv7.h		

表 7.4.1.1 基于 CMSIS 应用程序文件描述

把这些文件组织起来的方法，我们会在后续章节新建工程中介绍，这只是提前告诉一下大家组成我们需要的编译工具链大概需要哪些文件。

7.4.2 HAL 库的用户配置文件

stm32h7xx_hal_conf.h 用于裁剪 HAL 库和定义一些变量，官方没有直接提供这个文件，但在 STM32Cube_FW_H7_V1.9.0\Drivers\STM32H7xx_HAL_Driver\Inc 这个路径下提供了一个模版文件《stm32h7xx_hal_conf_template.h》，我们可以直接复制这个文件重命名为 stm32h7xx_hal_conf.h，做一些简单的修改即可，也可以从在官方的例程中直接复制过来。我们一般都直接从官方的模板例程中直接复制过来即可。因为我们的芯片是 STM32H743 系列，所以选择的路径是：STM32Cube_FW_H7_V1.9.0\Projects\STM32H743I-EVAL\Templates\Template_Project\Inc。

（1）stm32h7xx_hal_conf.h 文件里面的内容不多，对我们来说最重要的是 HSE_VALUE 这个参数，这个参数表示我们的外部高速晶振的频率。这个参数请务必根据我们板子外部焊接的晶振频率来修改，官方默认是 25M。正点原子阿波罗 STM32H743 开发板外部高速晶振的频率是 25MHZ。代码如下：

```
#if !defined  (HSE_VALUE)
#define HSE_VALUE    ((uint32_t)25000000) /* 外部高速振荡器的值，单位 HZ */
#endif /* HSE_VALUE */
```

（2）还有一个参数就是外部低速晶振频率，这个官方默认是 32.768KHZ，我们开发板的低速晶振也是这个频率，所以不用修改，源码在 135 行。

```
#if !defined  (LSE_VALUE)
  #define LSE_VALUE    ((uint32_t)32768) /* 外部低速振荡器的值，单位 HZ */
#endif /* LSE_VALUE */
```

其他源码都可以不作修改，按照默认的配置即可。下面我们再来了解一下其他程序。

（3）用户配置文件可以用来选择使能何种外设，源码配置在 35 行到 98 行，代码如下。

```
/* ########################## Module Selection ############################## */
/**
  * @brief This is the list of modules to be used in the HAL driver
  */
#define HAL_MODULE_ENABLED
#define HAL_ADC_MODULE_ENABLED
#define HAL_CEC_MODULE_ENABLED
#define HAL_COMP_MODULE_ENABLED
#define HAL_CORTEX_MODULE_ENABLED
...中间省略...
#define HAL_UART_MODULE_ENABLED
#define HAL_USART_MODULE_ENABLED
#define HAL_WWDG_MODULE_ENABLED
```

我们只要屏蔽某个外设的宏，则这个外设的驱动代码机会被屏蔽，从而不可用。比如我们屏蔽 GPIO 外设的宏，源码在 58 行，屏蔽就是把这个宏注释掉，具体如下。

```
#define HAL_GPIO_MODULE_ENABLED
```

然后打开 stm32h7xx_hal_gpio.c 文件，看到第 118 行。

```
#ifdef HAL_GPIO_MODULE_ENABLED
#include "stm32h7xx_hal_gpio.h"
#endif
```

这是一个条件编译符，与#endif 配合使用。这里的要表达的意思是，只要工程中定义了 HAL_GPIO_MODULE_ENABLED 这个宏，#ifdef 到#endif 之间的程序（119 行到 547 行）就会

参与编译，否则不编译。所以只要我们屏蔽了 stm32h7xx_hal_conf.h 文件 53 行的宏，GPIO 的驱动代码就不被编译。也就起到选择使能何种外设的功能，其他外设同理。

可以看官方的示范例程，就是通过屏蔽外设的宏的方法来选择使能何种外设。好处就是编译时间会变短，因为屏蔽了没有用的程序，编译时间自然就短了。正点原子的例程选择另外一中方法，就是工程中只保留需要的 stm32h7xx_hal_ppp.c，不需要的不添加到工程里，这样编译时间就不会太长。

（4）大家看到 stm32h7xx_hal_conf.h 文件的 159 行。

```
#define  TICK_INT_PRIORITY     (15UL) /*!< tick interrupt priority */
```

宏定义 TICK_INT_PRIORITY 是滴答定时器的优先级。这个优先级很重要，因为如果其它的外设驱动程序的延时是通过滴答定时器提供的时间基准，来实现延时的话，又由于实现方式是滴答定时器对寄存器进行计数，所以当我们在其它中断服务程序里调用基于此时间基准的延迟函数 HAL_Delay，那么假如该中断的优先级高于滴答定时器的优先级，就会导致滴答定时器中断服务函数一直得不到运行，从而程序卡死在这里。所以滴答定时器的中断优先级一定要比这些中断高。

请注意这个时间基准可以是滴答定时器提供，也可以是其他的定时器，默认是用滴答定时器。

（5）下面说一下关于断言这个功能，这个功能用来判断函数的形参是否有效，在 HAL 库的 API 里面有用到。这个功能的使能开关代码是一个宏，在源码的 236 行，默认是关闭的，代码如下。

```
/* #define USE_FULL_ASSERT    1U */
```

通过宏 USE_FULL_ASSERT 来选择功能，在源码 413 行到 432，代码如下。

```
/* Exported macro ------------------------------------------------------------*/
#ifdef  USE_FULL_ASSERT
/**
  * @brief  The assert_param macro is used for function's parameters check.
  * @param  expr: If expr is false, it calls assert_failed function
  *         which reports the name of the source file and the source
  *         line number of the call that failed.
  *         If expr is true, it returns no value.
  * @retval None
  */
  #define assert_param(expr) ((expr) ? (void)0U : assert_failed((uint8_t
*)__FILE__, __LINE__))
/* Exported functions ------------------------------------------------------- */
  void assert_failed(uint8_t* file, uint32_t line);
#else
  #define assert_param(expr) ((void)0U)
#endif /* USE_FULL_ASSERT */

#ifdef __cplusplus
}
#endif

#endif /* __STM32H7xx_HAL_CONF_H */
```

也是通过条件编译符来选择对应的功能。当用户自己需要使用断言功能，怎么做呢？首先需要定义宏 USE_FULL_ASSERT 来使能断言功能，即把源码的 236 行的注释去掉即可。然后看到源码 423 行的 assert_failed（）这个函数。其实这个函数是需要我们自己实现的，我们把这个函数定义在正点原子提供的 sys.c 文件里面。后面再跟大家讲 sys.c 文件，现在我们把 assert_failed（）这个函数拿出来给大家先讲，assert_failed（）函数的定义在 sys.c 的 234 行到 239 行，具体如下：

```
#ifdef  USE_FULL_ASSERT

/**
 * @brief       当编译提示出错的时候此函数用来报告错误的文件和所在行
 * @param       file：指向源文件
 * @param       line：指向在文件中的行数
 * @retval      无
```

```
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    while (1)
    {
    }
}
#endif
```

可以看到这个函数里面没有实现如何功能，就是一个什么不做的死循环，具体功能请根据自己的需求去实现。file 是指向源文件的指针，line 是指向源文件的行数。__FILE__是表示源文件名，__LINE__是表示在源文件中的行数。比如我们可以实现打印出这个错误的两个信息等等。

总的来说断言功能就是，在 HAL 库中，如果定义了 USE_FULL_ASSERT 这个宏，那么所有的 HAL 库函数将会检查函数的形参是否正确。如果错误将会调用 assert_failed（）这个函数，这个函数我们默认是个什么事不做的死循环，用户请根据自己的需求设计功能。使用断言功能将会增加了代码量，减慢运行速度等，所以一般只是在调试的时候用，正式发布的软件是不推荐的。

7.4.3 stm32h7xx_hal.c 文件

这个文件内容比较多，包括 HAL 库的初始化、系统滴答、基准电压配置、IO 补偿、低功耗、EXTI 配置等都集合在这个文件里面。下面我们对该文件进行讲解。

1. HAL_Init()函数

源码在 134 行到 180 行，简化函数如下：

```
HAL_StatusTypeDef HAL_Init(void)
{
  /* 设置中断优先级分组 */
  HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_4);

  /* 使用滴答定时器作为时钟基准，配置 1ms 滴答（重置后默认的时钟源为 HSI） */
  if(HAL_InitTick(TICK_INT_PRIORITY) != HAL_OK)
  {
    return HAL_ERROR;
  }

  /* 初始化硬件 */
  HAL_MspInit();

  /* 返回函数状态 */
  return HAL_OK;
}
```

该函数是 HAL 库的初始化函数，在程序中必须优先调用，其主要实现如下功能：

1）设置 NVIC 优先级分组为 4。

2）配置滴答定时器每 1ms 产生一个中断。

3）在这个阶段，系统时钟还没有配置好，因此系统还是默认使用内部高速时钟源 HSI 在跑程序。对于 H7 来说，HSI 的主频是 64MHZ。所以如果用户不配置系统时钟的话，那么系统将会使用 HSI 作为系统时钟源。

4）调用 HAL_MspInit 函数初始化底层硬件，HAL_MspInit 函数在 stm32h7xx_hal.c 文件里面做了弱定义。关于弱定义这个概念，后面会有讲解，现在不理解没关系。正点原子的 HAL 库例程是没有使用到这个函数去初始化底层硬件，而是单独调用需要用到的硬件初始化函数。用户可以根据自己的需求选择是否重新定义该函数来初始化自己的硬件。

注意事项：

为了方便和兼容性，正点原子的 HAL 库例程中的中断优先级分组设置为分组 2，即把源码的 147 行改为如下代码：

```
HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_2);
```

中断优先级分组为 2，也就是 2 位抢占优先级，2 位响应优先级，抢占优先级和响应优先级的值的范围均为 0-3。

2. HAL_DeInit ()函数

源码在 187 行到 222 行，函数如下：

```
HAL_StatusTypeDef HAL_DeInit(void)
{
  /* 复位所有外设 */
  __HAL_RCC_AHB3_FORCE_RESET();
  __HAL_RCC_AHB3_RELEASE_RESET();
  __HAL_RCC_AHB1_FORCE_RESET();
  __HAL_RCC_AHB1_RELEASE_RESET();

  __HAL_RCC_AHB2_FORCE_RESET();
  __HAL_RCC_AHB2_RELEASE_RESET();

  __HAL_RCC_AHB4_FORCE_RESET();
 __HAL_RCC_AHB4_RELEASE_RESET();

  __HAL_RCC_APB3_FORCE_RESET();
  __HAL_RCC_APB3_RELEASE_RESET();

  __HAL_RCC_APB1L_FORCE_RESET();
  __HAL_RCC_APB1L_RELEASE_RESET();

  __HAL_RCC_APB1H_FORCE_RESET();
  __HAL_RCC_APB1H_RELEASE_RESET();

   __HAL_RCC_APB2_FORCE_RESET();
   __HAL_RCC_APB2_RELEASE_RESET();

  __HAL_RCC_APB4_FORCE_RESET();
  __HAL_RCC_APB4_RELEASE_RESET();

  /* 对底层硬件初始化进行复位 */
  HAL_MspDeInit();

  /* 返回函数状态 */
  return HAL_OK;
}
```

该函数取消初始化 HAL 库的公共部分，并且停止 systick，是一个可选的函数。该函数做了以下的事：

1）复位了 AHB1、AHB2、AHB3、AHB4、APB1L、APB1H、APB2、APB3、APB4 的时钟。

2）调用 HAL_MspDeInit 函数，对底层硬件初始化进行复位。HAL_MspDeInit 也在 stm32h7xx_hal.c 文件里面做了弱定义，并且与 HAL_MspInit 函数是一对存在。HAL_MspInit 函数负责对底层硬件初始化，HAL_MspDeInit 函数则是对底层硬件初始化进行复位。这两个函数都是需要用户根据自己的需求去实现功能，也可以不使用。

3. HAL_InitTick ()函数

源码在 262 行到 289 行，简化函数如下：

```
__weak HAL_StatusTypeDef HAL_InitTick(uint32_t TickPriority)
{
  /* uwTickFreq 是个枚举类型，如果检测到 uwTickFreq 为零，则返回 */
  if((uint32_t)uwTickFreq == 0UL)
  {
    return HAL_ERROR;
  }
```

```
    /* 配置滴答定时器 1ms 产生一次中断 */
  if (HAL_SYSTICK_Config(SystemCoreClock /(1000UL / (uint32_t)uwTickFreq))> 0U)
  {
    return HAL_ERROR;
  }
#endif

  /* 配置滴答定时器中断优先级 */
  if (TickPriority < (1UL << __NVIC_PRIO_BITS))
  {
    HAL_NVIC_SetPriority(SysTick_IRQn, TickPriority, 0U);
    uwTickPrio = TickPriority;
  }
  else
  {
    return HAL_ERROR;
  }

  /* 返回函数状态 */
  return HAL_OK;
}
```

该函数用于初始化滴答定时器的时钟基准，主要功能如下：

1）配置滴答定时器 1ms 产生一次中断。

2）配置滴答定时器的中断优先级。

3）该函数是__weak 定义的“弱函数”，用户可以重新定义该函数。

该函数可以通过 HAL_Init()或者 HAL_RCC_ClockConfig()重置时钟。在默认情况下，滴答定时器是时间基准的来源。如果其他中断服务函数调用了 HAL_Delay()，必须小心，滴答定时器中断必须具有比调用了 HAL_Delay()函数的其他中断服务函数的优先级高(数值较低)，否则会导致滴答定时器中断服务函数一直得不到执行，从而卡死在这里。

4. 滴答定时器相关的函数

源码在 327 行到 434 行，相关函数如下：

```
/* 该函数在滴答定时器时钟中断服务函数中被调用，一般滴答定时器 1ms 中断一次，
   所以函数每 1ms 让全局变量 uwTick 计数值加 1 */
__weak void HAL_IncTick(void)
{
  uwTick += (uint32_t)uwTickFreq;
}

/* 获取全局变量 uwTick 当前计算值 */
__weak uint32_t HAL_GetTick(void)
{
  return uwTick;
}

/* 获取滴答时钟优先级 */
uint32_t HAL_GetTickPrio(void)
{
  return uwTickPrio;
}

/* 设置滴答定时器中断频率 */
HAL_StatusTypeDef HAL_SetTickFreq(HAL_TickFreqTypeDef Freq)
{
  HAL_StatusTypeDef status  = HAL_OK;
  HAL_TickFreqTypeDef prevTickFreq;
  assert_param(IS_TICKFREQ(Freq));
  if (uwTickFreq != Freq)
  {
    /* 备份滴答定时器中断频率 */
    prevTickFreq = uwTickFreq;
```

```
    /* 更新被 HAL_InitTick()调用的全局变量 uwTickFreq */
    uwTickFreq = Freq;

    /* 应用新的滴答定时器中断频率  */
    status = HAL_InitTick(uwTickPrio);
    if (status != HAL_OK)
    {
      /* 恢复之前的滴答定时器中断频率 */
      uwTickFreq = prevTickFreq;
    }
  }
  return status;
}

/* 获取滴答定时器中断频率 */
HAL_TickFreqTypeDef HAL_GetTickFreq(void)
{
  return uwTickFreq;
}

/* HAL 库的延时函数，默认延时单位 ms */
__weak void HAL_Delay(uint32_t Delay)
{
  uint32_t tickstart = HAL_GetTick();
  uint32_t wait = Delay;

  /* Add a freq to guarantee minimum wait */
  if (wait < HAL_MAX_DELAY)
  {
    wait += (uint32_t)(uwTickFreq);
  }
  while ((HAL_GetTick() - tickstart) < wait)
  {
  }
}

/* 挂起滴答定时器中断，全局变量 uwTick 计数停止 */
__weak void HAL_SuspendTick(void)
{
  /* 禁止滴答定时器中断 */
  SysTick->CTRL &= ~SysTick_CTRL_TICKINT_Msk;
}

/* 恢复滴答定时器中断，恢复全局变量 uwTick 计数 */
__weak void HAL_ResumeTick(void)
{
  /* 使能滴答定时器中断 */
  SysTick->CTRL  |= SysTick_CTRL_TICKINT_Msk;
}
```

这些函数不是很难，请参照注释理解。注意：如果函数被前缀__weak 定义，则用户可以重新定义该函数。更多的内容可以参考 8.1.5 小节。

5. HAL 库版本相关的函数

源码在 501 行到 589 行，相关函数声明如下：

```
uint32_t HAL_GetHalVersion(void);   /* 获取 HAL 库驱动程序版本 */
uint32_t HAL_GetREVID(void);        /* 获取设备修订标识符 */
uint32_t HAL_GetDEVID(void);        /* 获取设备标识符 */
uint32_t HAL_GetUIDw0(void);        /* 获取唯一设备标识符的第一个字 */
uint32_t HAL_GetUIDw1(void);        /* 获取唯一设备标识符的第二个字 */
uint32_t HAL_GetUIDw2(void);        /* 获取唯一设备标识符的第三个字 */
```

这些函数了解一下就好了，用得不多。

6. 芯片内部电压基准相关函数

源码在 520 行到 589 行，函数声明如下：

```
void HAL_SYSCFG_VREFBUF_VoltageScalingConfig(uint32_t VoltageScaling);
void HAL_SYSCFG_VREFBUF_HighImpedanceConfig(uint32_t Mode);
void HAL_SYSCFG_VREFBUF_TrimmingConfig(uint32_t TrimmingValue);
HAL_StatusTypeDef HAL_SYSCFG_EnableVREFBUF(void);
void HAL_SYSCFG_DisableVREFBUF(void);
```

HAL_SYSCFG_VREFBUF_VoltageScalingConfig 函数用于配置芯片内部电压基准大小，形参有四个值可以选择：

1）当形参为 SYSCFG_VREFBUF_VOLTAGE_SCALE0 时，
电压输出基准为 2.048V，条件是 VDDA >= 2.4V。
2）当形参为 SYSCFG_VREFBUF_VOLTAGE_SCALE1 时，
电压输出基准为 2.5V，条件是 VDDA >= 2.8V。
3）当形参为 SYSCFG_VREFBUF_VOLTAGE_SCALE2 时，
电压输出基准为 1.5V，条件是 VDDA >= 1.8V。
4）当形参为 SYSCFG_VREFBUF_VOLTAGE_SCALE3 时，
电压输出基准为 1.8V，条件是 VDDA >= 2.1V。

HAL_SYSCFG_VREFBUF_HighImpedanceConfig 函数用于配置芯片内部电压是否与 VREF+引脚连接，即是否选择高阻抗模式，有两个形参选择：

1）当形参为 SYSCFG_VREFBUF_HIGH_IMPEDANCE_DISABLE，表示导通。
2）当形参为 SYSCFG_VREFBUF_HIGH_IMPEDANCE_ENABLE，表示高阻抗，即不导通。

HAL_SYSCFG_VREFBUF_TrimmingConfig 函数用于调整校准内部电压基准。

HAL_SYSCFG_EnableVREFBUF 函数用于使能内部电压基准参考。

HAL_SYSCFG_DisableVREFBUF 函数用于禁止内部电压基准参考。

7. 以太网 PHY 接口选择函数

源码在 630 行到 637 行，函数声明如下：

```
void HAL_SYSCFG_ETHInterfaceSelect(uint32_t SYSCFG_ETHInterface);
```

该函数用于以太网 PHY 接口的选择，可以是 MII 或 RMII 接口。

8. HAL_SYSCFG_AnalogSwitchConfig()函数

源码在 630 行到 637 行，函数声明如下：

```
void HAL_SYSCFG_AnalogSwitchConfig(uint32_t SYSCFG_AnalogSwitch ,
                                   uint32_t SYSCFG_SwitchState );
```

当 PA0、PA1、PC2、PC3 引脚复用为 ADC 的时候，还有一组对应的可选引脚 PA0_C、PA1_C、PC2_C、PC3_C。该函数的作用就是切换这些可选的引脚。关于这个不理解，请参考图 7.4.3.1。该函数操作了 SYSCFG_PMCR 寄存器，关于该寄存器请查阅参考手册。

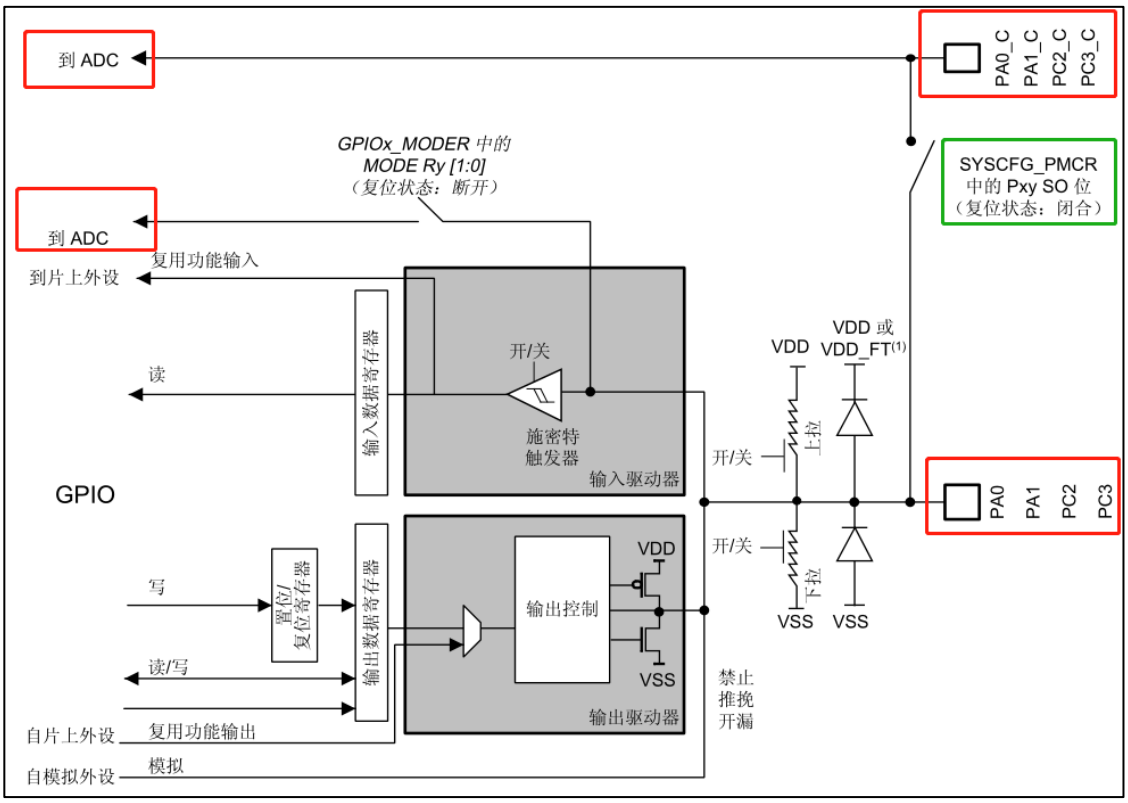


图 7.4.3.1 连接到 ADC 输入的模拟输入

9. Booster 的使能和禁止函数（用于 ADC）

源码在 648 行到 663 行，函数声明如下：

```
void HAL_SYSCFG_EnableBOOST(void);   /* 使能 Booster */
void HAL_SYSCFG_DisableBOOST(void);  /* 禁止 Booster */
```

如果使能 Booster，当供电电压低于 2.7V 时，能够减少模拟开关总的谐波失真。这样就使得模拟开关的性能和供电正常的情况时一样，能够正常工作。

10. HAL_SYSCFG_CM7BootAddConfig()函数

源码在 676 行到 699 行，函数如下：

```
void HAL_SYSCFG_CM7BootAddConfig(uint32_t BootRegister, uint32_t BootAddress)
```

该函数用于配置程序启动模式，BOOT=0 或者 BOOT=1，来选择启动地址。详细的内容请看第九章的 9.1 小节。

11. IO 补偿、低功耗、EXTI 等相关函数

IO 补偿、低功耗、EXTI 等相关函数，这里先不进行讲解了，后面用到再进行说明。

7.4.4 HAL 库中断处理

中断是 STM32 开发的一个很重要的概念，这里我们可以简单地理解为：STM32 暂停了当前手中的事并优先去处理更重要的事务。而这些“更重要的事务”是由软件开发人员在软件中定义的。关于 STM32 中断的概念，我们会在中断例程的讲解再跟大家详细介绍。

由于 HAL 库中断处理的逻辑比较统一，我们将这个处理过程抽象为图 7.4.4.1 所表示的业务逻辑：

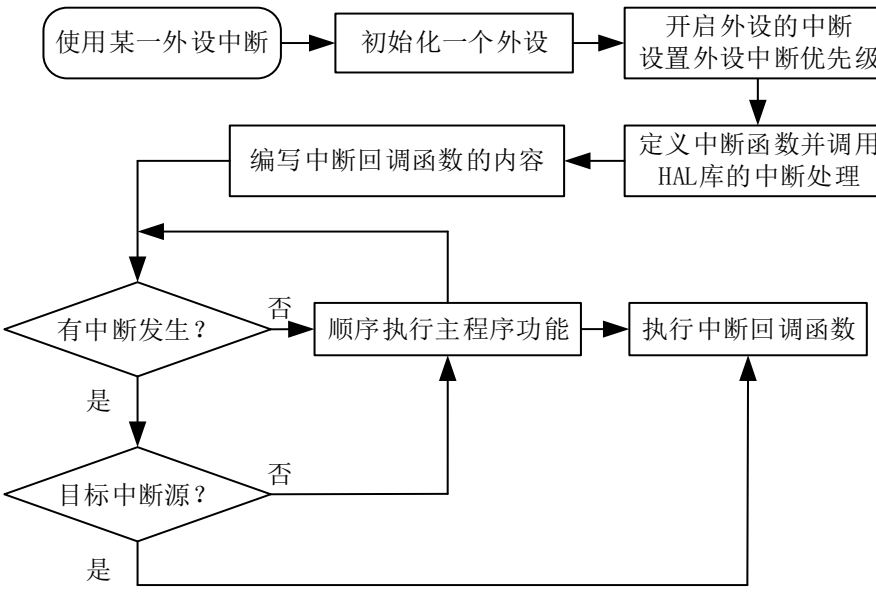


图 7.4.4.1 HAL 驱动中断处理流程

结合以前的 HAL 库文件介绍章节，以上的流程大概就是：设置外设的控制句柄结构体 PPP_HandleType 和初始化 PPP_InitType 结构体的参数，然后调用 HAL 库对应这个驱动的初始化 HAL_PPP_Init()，由于这个 API 中有针对外设初始化细节的接口 Hal_PPP_Mspinit()，我们需要重新实现这个函数并完成外设时钟、IO 等细节差异的设置， 完成各细节处理后，使用 HAL_NVIC_SetPriority()、HAL_NVIC_EnableIRQ()来使能我们的外设中断；定义中断处理函数 PPP_IRQHandler，并在中断函数中调用 HAL_ppp_function_IRQHandler()来判断和处理中断标记； HAL 库中断处理完成后，根据对应中的调用我们需要自定义的中断回调接口 HAL_PPP_ProcessCpltCallback()；如串口接收函数 HAL_UART_RxCpltCallback()，我们在这个函数中实现我们对串口接收数据想做的处理；中断响应处理完成后，stm32 芯片继续顺序执行我们定义的主程序功能，按照以上处理的标准流程完成了一次中断响应。

7.4.5 正点原子对 HAL 库用法的个性化修改

前面按 ST 官方建议的 HAL 库的使用方法给介绍了一个 HAL 库。

1、将中断处理函数独立到每个外设中，便于独立驱动；同类型的外设驱动处理函数不使用 HAL 回调函数接口处理操作而直接在中断函数中处理判断对应中断。

2、我们把原来的中断分组进行了修改，由抢占式无子优先级改为中断分组 2；便于管理同类外设的优先级响应。

3、在很多芯片的初始化过程中，我们使用到了 delay_ms()、delay_us()等函数进行初始化，使用的是 Systick 作的精准延时，而 HAL 库默认也使用 Systick 作延时处理，为解决这种冲突和兼容我们大部分的驱动代码，我们在例程中使用 delay.c 中的延时函数取代 Hal_Delay();取消原来 HAL 库的 Systick 延时设置。

7.5 HAL 库使用注意事项

本小节根据经验跟大家讲述一些关于 HAL 库使用的注意事项，供读者参考。

1、即使我们已经在使用库函数作为开发工具了，我们可以忽略很多芯片的硬件外设使用上的细节，但当发生问题时，我们仍需要回归到芯片使用手册查看当前操作是否违规或缺漏。

2、使用 HAL 库和其它第三方的库开发类似，把我们需要编写的软件和第三方的库分开成相互独立的文件，开发过程中我们尽量不去修改第三方的软件源码，需要修改的部分尽量在自己的代码中实现；这样一旦我们需要更新第三方库时，我们原来编写的功能也能很快地匹配新的库去执行功能。

3、即使 HAL 库目前较以前已经相对更完善了，但它仍无法覆盖我们要想实现的所有细节功能，甚至可能存在错误，我们要有怀疑精神，辩证地去使用好这个工具；如我们在 PWM 一节编码时发现 HAL 库中有个宏定义 TIM_RESET_CAPTUREPOLARITY 括号不匹配导致编译报错，这时我们不得不修改一下 HAL 库的源码了。

4、注意 HAL 库的执行效率。由于 HAL 库的驱动对相同外设大多是可重入的，在执行 HAL 驱动的 API 函数的效率没有直接寄存器操作来得高，如果在对时序要求比较严苛的代码，建议使用简洁的寄存器操作代替。

5、我们在例程中使用 delay.c 中的延时函数取代 Hal_Delay();取消原来 HAL 库的 Systick 延时设置；但这会有一个问题：原来 HAL 库的超时处理机制不再适用，所以对于设置了超时的函数，可能会导致停留在这个函数的处理中，无法按正常的超时退出。

第八章 新建 HAL 版本 MDK 工程

在前面的章节我们介绍了 STM32H7xx 官方固件包的一些知识，本章我们将讲解新建 HAL 库版本的 MDK 工程的详细步骤。我们把本章新建好的工程放在光盘里，路径：**4，程序源码\2，标准例程-HAL 库版本\实验 0 基础入门实验\实验 0-3，新建工程实验-HAL 库版本**，大家在学习新建工程过程中间遇到一些问题，可以直接打开这个工程，然后对比学习。

本章将分为如下三个小节：

8.1 新建 HAL 库版本 MDK 工程

8.2 下载验证

8.1 新建 HAL 库版本 MDK 工程

本节我们将教大家如何新建一个 STM32H743 的 HAL 库版本 MDK5 工程。为了方便大家参考，我们将本节最终新建好的工程模板存放在 A 盘：4、程序源码\2，标准例程-HAL 库版本\实验 0 基础入门实验\实验 0-3，新建工程实验-HAL 库版本，如遇新建工程问题，请打开该实验对比。

整个新建过程比较复杂，我们将其拆分为 5 个步骤进行讲解，请准备大概 2 个小时时间，耐心细致的做完！对你后续的学习非常有帮助！

在新建工程之前，首先我们要做如下准备：

1、 STM32Cube 官方固件包：我们使用的固件包版本是 STM32Cube_FW_H7_V1.9.0，固件包路径：A 盘→8，STM32 参考资料→1，STM32CubeH7 固件包。

2、开发环境搭建：参考本书第三章相关内容。

8.1.1 新建工程文件夹

新建工程文件夹分为 2 个步骤：1，新建工程文件夹；2，拷贝工程相关文件。

1. 新建工程文件夹

首先我们在桌面新建一个工程根目录文件夹，后续的工程文件都将在这个文件夹里建立，我们把这个文件夹重命名为：实验 0-3，新建工程实验-HAL 库版本。如图 8.1.1.1 所示：

图 8.1.1.1 新建工程根目录文件夹

为了让工程的文件目录结构更加清晰易懂，我们会在工程根目录文件夹下建立以下几个文件夹，每个文件夹名称及其作用如表 8.1.1.1 所示：

名称	作用
Drivers	存放与硬件相关的驱动层文件
Middlewares	存放正点原子提供的中间层组件文件和第三方中间层文件
Output	存放工程编译输出文件
Projects	存放 MDK 工程文件
User	存放 HAL 库用户配置文件、main.c

表 8.1.1.1 工程根目录新建文件夹及其作用

新建完成以后，最后得到我们的工程根目录文件夹如图 8.1.1.2 所示。

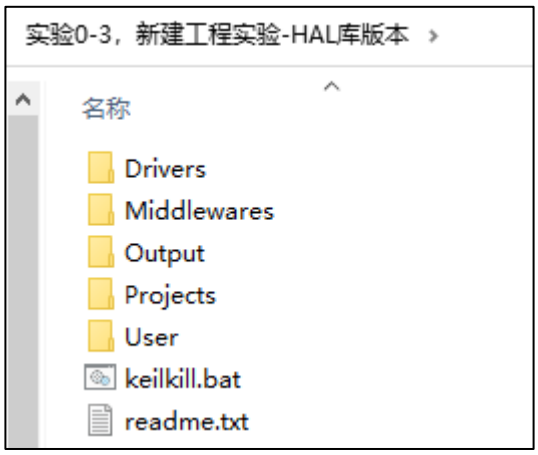


图 8.1.1.2 工程根目录文件夹

另外我们的工程根文件目录下还有一个名为 keilkill.bat 的可执行文件，双击便可执行。其作用是删除编译器编译后的无关文件，减少工程占用的内存，方便打包。还有一个名为 readme 的记事本文件，其作用是介绍本实验的各种信息。

工程根目录及其相关文件夹新建好以后，我们需要拷贝一些工程相关文件过来（主要是在 Drivers 文件夹里面），以便等下的新建工程需要。

2. 拷贝工程相关文件

接下来，我们按图 8.1.1.2 的根目录文件夹顺序介绍每个文件夹及其需要拷贝的文件。

Drivers 文件夹

该文件夹用于存放与硬件相关的驱动层文件，一般包括如表 8.1.1.2 所示的三个文件夹：

文件夹名称	作用
BSP	存放开发板板级支持包驱动代码，如各种外设驱动
CMSIS	存放 CMSIS 底层代码，如启动文件（.s 文件）、stm32h7xx.h 等
STM32H7xx_HAL_Driver	存放 HAL 库驱动代码源文件
SYSTEM	存放正点原子系统级核心驱动代码，如 sys.c、delay.c 和 usart.c

表 8.1.1.2 Drivers 包含文件夹

BSP 文件夹，用于存放正点原子提供的板级支持包驱动代码，如：LED、按键等。本章我们暂时用不到该文件夹，不过可以先建好备用。

CMSIS 文件夹，用于存放 CMSIS 底层代码（ARM 和 ST 提供），如：启动文件（.s 文件）、stm32h7xx.h 等各种头文件。该文件夹我们可以直接从 STM32CubeH7 固件包（路径：A 盘→8，STM32 参考资料→1，STM32CubeH7 固件包）里面拷贝，不过由于固件包里面的 CMISIS 兼容了太多芯片，导致非常大（200 多 MB），因此我们根据实际情况，对其进行了大幅精简，精简后的 CMSIS 文件夹大小为 2.3MB 左右。精简后的 CMSIS 文件夹大家可以在：A 盘→4，程序源码→1，标准例程-HAL 库版本 文件夹里面的任何一个实验的 Drivers 文件夹里面拷贝过来。

SYSTEM 文件夹，用于存放正点原子提供的系统级核心驱动代码，如：sys.c、delay.c 和 usart.c 等，方便大家快速搭建自己的工程。该文件同样可以从：A 盘→4，程序源码→1，标准例程-HAL 库版本 文件夹里面的任何一个实验的 Drivers 文件夹里面拷贝过来。

STM32H7xx_HAL_Driver 文件夹，用于存放 ST 提供的 H7xx HAL 库驱动代码。该文件夹我们可以直接从 STM32CubeH7 固件包里面拷贝。直接拷贝“STM32CubeH7 固件包→Drivers”路径下的“STM32H7xx_HAL_Driver”文件夹到我们工程的 Drivers 下，只保留 Inc 和 Src 文件夹即可。

执行完以上操作后，Drivers 文件夹最终结构如图 8.1.1.3 所示：

图 8.1.1.3 工程根目录下的 Drivers 文件夹

关于工程根目录下的 Drivers 文件操作到这里就完成了，可以说步骤是有点多。在此过程遇到问题的话，请大家多参考我们提供的实验 0-3，新建工程实验-HAL 库版本工程，一步步操作。

Middlewares 文件夹

该文件夹用于存放正点原子和其他第三方提供的中间层代码（组件/Lib 等），如：USMART、MALLOC、TEXT、FATFS、USB、LWIP、各种 OS、各种 GUI 等等。本章我们暂时用不到该文件夹，不过可以先建好备用，后面的实验将会陆续添加各种文件。

Output 文件夹

该文件夹用于存放编译器编译工程输出的中间文件，比如：.hex、.bin、.o 文件等等。这里不需要操作，后面只需要在 MDK 里面设置该文件夹为编译过程中间文件的存放文件夹就行。

Projects 文件夹

该文件夹用于存放编译器（MDK、IAR 等）工程文件，我们主要用 MDK，为了方便区分，我们在该文件夹下新建：MDK-ARM 文件夹，用于存放 MDK 的工程文件，如图 8.1.1.4 所示：

图 8.1.1.4 在 Projects 文件夹下新建 MDK-ARM 文件夹

User 文件夹

User 文件夹用于存放 HAL 库用户配置文件、main.c 文件、中断处理文件。

我们首先从官方固件包里面直接拷贝官方的模板工程下的 HAL 库用户配置文件和中断处理文件到我们的 User 文件夹里。官方的模板工程路径：STM32Cube_FW_H7_V1.9.0\Projects\STM32H743I-EVAL\Templates，打开 Template_Project 文件夹，如图 8.1.1.8 所示。

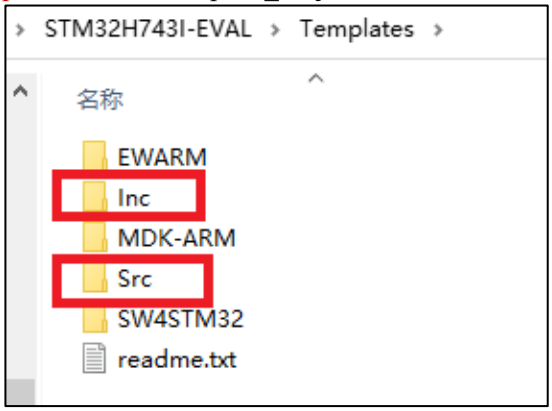


图 8.1.1.8 官方模板工程根目录

我们需要的文件就在 Inc 和 Src 文件夹里面，在这两个文件夹里面找到：stm32h7xx_it.c、stm32h7xx_it.h、stm32h7xx_hal_conf.h 这三个文件，并且拷贝到我们的 User 文件夹下。

main.c 文件我们也是放在 User 文件夹里面的，后面在 MDK 里面教大家新建.c 文件并保存。

8.1.2 新建一个工程框架

首先，打开 MDK 软件。然后点击 Project→New uVision Project 如图 8.1.2.1 所示：

图 8.1.2.1 新建 MDK 工程

然后弹出工程命名和保存的操作窗口，我们将工程文件保存路径设置在上一节新建的工程文件夹内，具体路径为：桌面→实验 0-2，新建工程实验-寄存器版本→Projects→MDK-ARM，工程名字我们取：atk_h743，最后点击保存即可。具体操作窗口如图 8.1.2.2 所示：

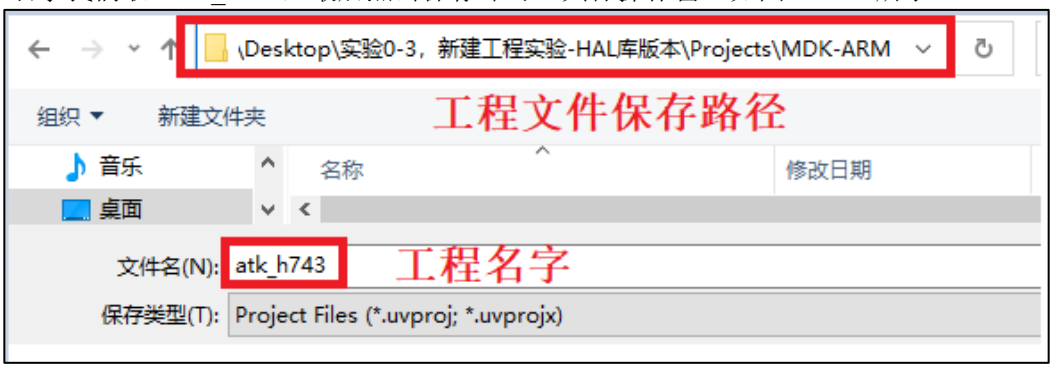


图 8.1.2.2 保存工程界面

之后，弹出器件选择对话框，如图 8.1.2.3 所示。因为阿波罗 STM32H743 开发板所使用的 STM32 型号为 STM32H743IIT6，所以我们选择：STMicroelectronics→STM32H7 Series→STM32H743→STM32H743IITx（如果使用的是其他系列的芯片，选择相应的型号就可以了，特别注意：**一定要安装对应的器件 pack** 才会显示这些内容哦!!如果没得选择，请关闭 MDK，然后安装 A 盘：6，软件资料\1，软件\MDK5\ Keil.STM32H7xx_DFP.2.7.0.pack 这个安装包后重试）。

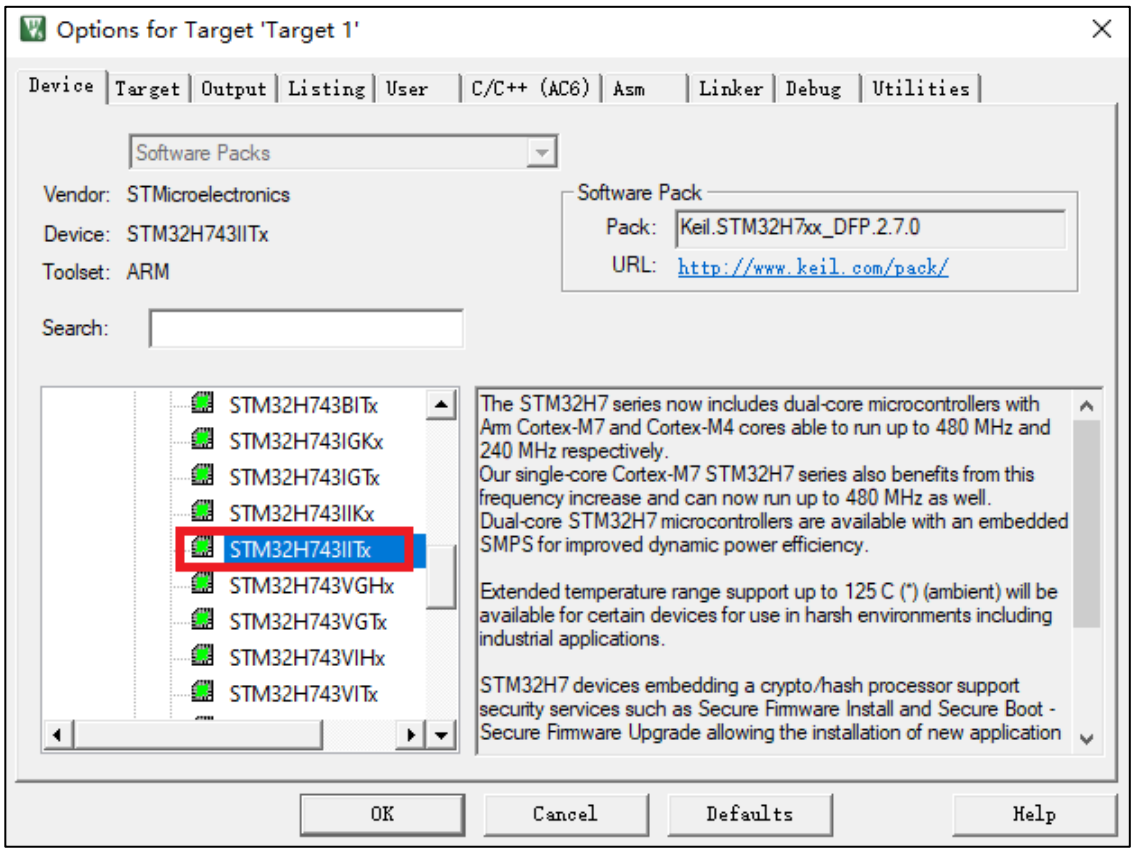


图 8.1.2.3 器件选择界面

点击 OK，MDK 会弹出 Manage Run-Time Environment 对话框，如图 8.1.2.4 所示：

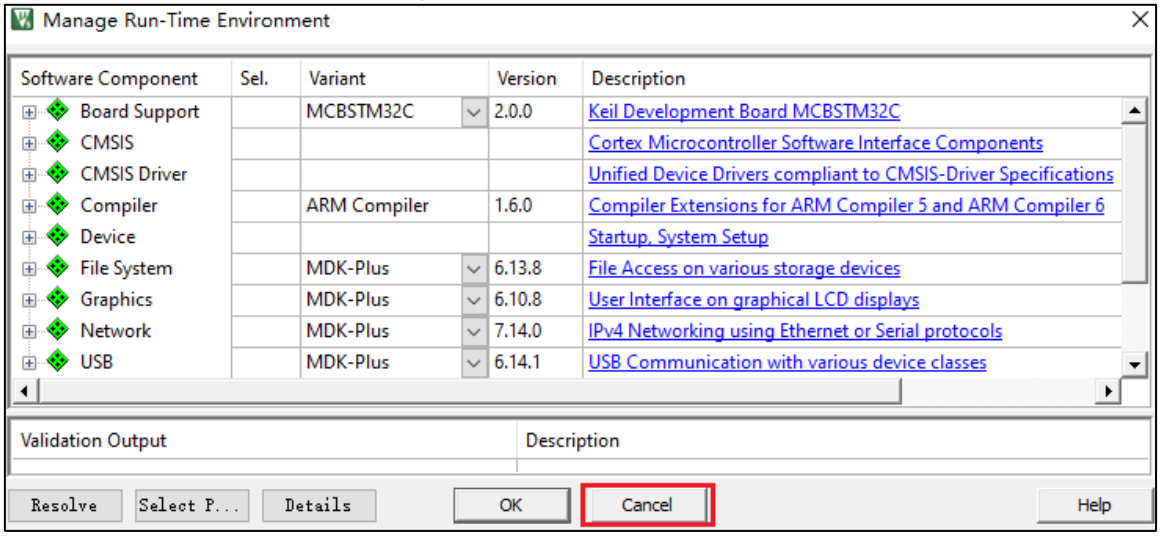


图 8.1.2.4 Manage Run-Time Environment 界面

这是 MDK5 新增的一个功能，在这个界面，我们可以添加自己需要的组件，从而方便构建开发环境，不过这里我们不做介绍。所以在图 8.1.2.4 所示界面，我们直接点击 Cancel，即可，得到如图 8.1.2.5 所示界面：

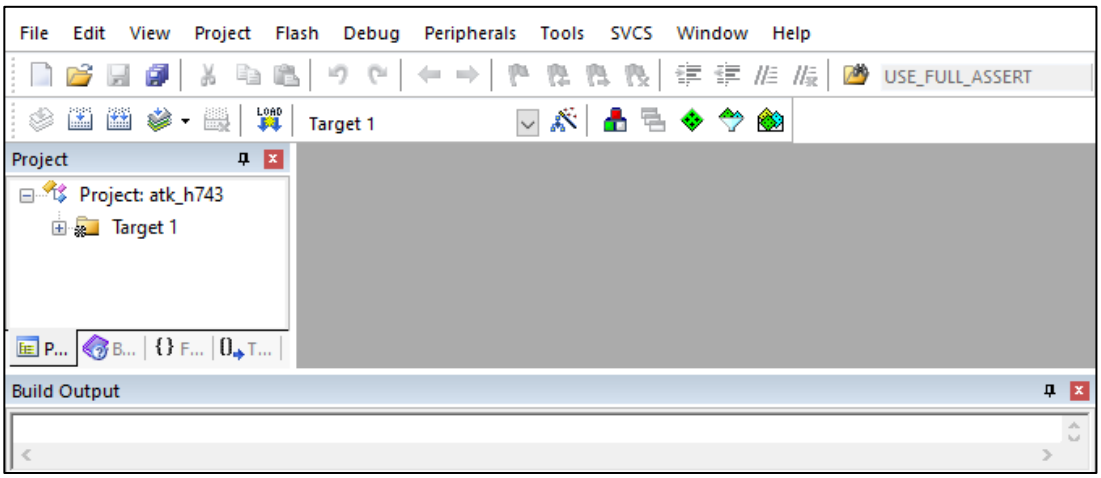


图 8.1.2.5 工程初步建立

此时，我们打开 MDK-ARM 文件夹，会看到 MDK 在该文件夹下自动创建了 3 个文件夹（DebugConfig、Listings 和 Objects），如图 8.1.2.6 所示：

图 8.1.2.6 MDK 新建工程时自动创建的文件夹

这三个文件夹的作用如表 8.1.2.1 所示：

文件夹	作用
DebugConfig	用于存放调试设置信息文件（.dbgconf），不可删除！
Listings	用于存放编译过程产生的链接列表等文件
Objects	用于存放编译过程产生的调试信息、.hex、预览、.lib 文件等

表 8.1.2.1 三个文件夹及其作用

编译过程产生的链接列表、调试信息、预览、lib 等文件，统称为中间文件。为了统一管理，方便使用，我们会把输出在 Listings 和 Objects 文件夹的内容，统一改为输出到 Output 文件夹（通过魔术棒设置），我们先**把 MDK 自动生成的这两个文件夹（Listings 和 Objects）删除**。

至此，我们还只是建了一个框架，还有好几个步骤要做，比如添加文件、魔术棒设置、编写 main.c 等。

8.1.3 添加文件

本节将分 5 个步骤：1，设置工程名和分组名；2，添加启动文件；3，添加 SYSTEM 源码 4，添加 User 源码；5，添加 STM32H7xx_HAL_Driver 源码。

1. 设置工程名和分组名

在 Project→Target 上右键，选择 Manage Project Items…（方法一）或在菜单栏点击品字形红绿白图标（方法二）进入工程管理界面，如图 8.1.3.1 所示：

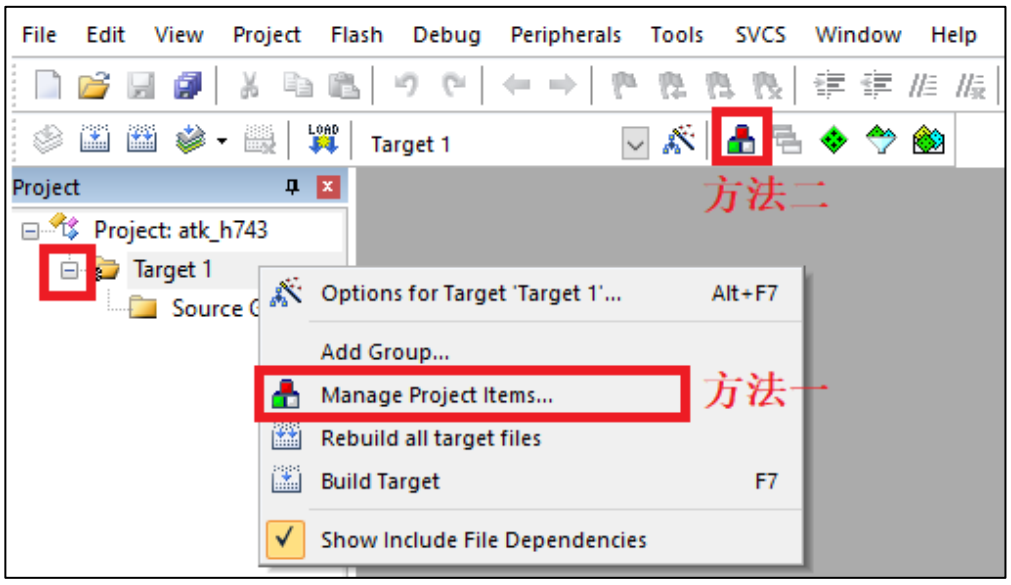


图 8.1.3.1 进入工程管理界面

在工程管理界面，我们可以执行设置工程名字（Project Targets）、分组名字（Groups）以及添加每个分组的文件（Files）等操作。我们设置工程名字为：Template，并设置四个分组：Startup（存放启动文件）、User（存放 main.c 等用户代码）、Drivers/SYSTEM（存放系统级驱动代码）、Readme（存放工程说明文件），如图 8.1.3.2 所示：

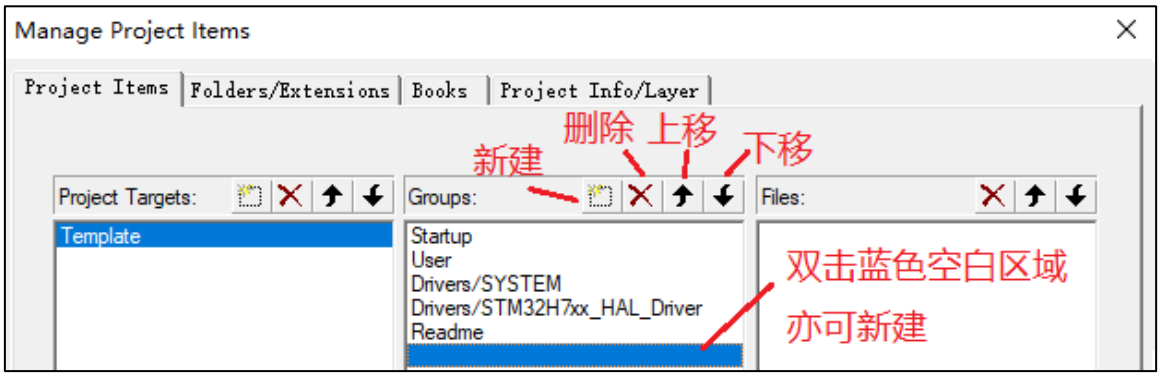


图 8.1.3.2 设置工程名和分组名

设置好之后，我们点击 OK，回到 MDK 主界面，可以看到我们设置的工程名和分组名如图 8.1.3.3 所示：

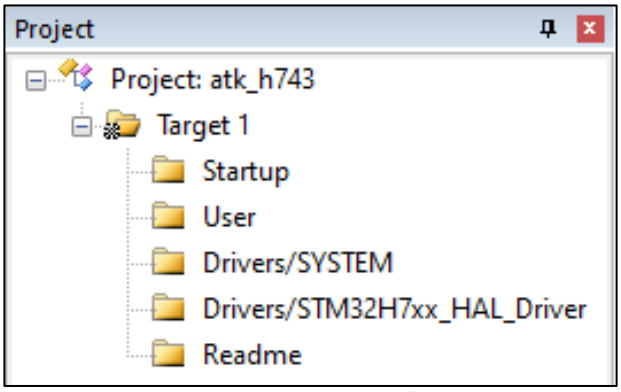


图 8.1.3.3 设置成功

这里我们只是新建了一个简单的工程，并没有添加 BSP、Middlewares 等分组，后面随着工程复杂程度的增加，我们需要一步步添加对应的分组。

注意：为了让工程结构清晰，我们会尽量让 MDK 的工程分组和我们前面新建的工程文件夹对应起来，由于 MDK 分组不支持多级目录，因此我们将路径也带入分组命名里面，以便区

分。如：User 分组对应 User 文件夹里面的源码，Drivers/SYSTEM 分组，对应 Drivers/SYSTEM 文件夹里面的源码，Drivers/BSP 分组对应 Drivers/BSP 文件夹里面的源码等。

2. 添加启动文件

启动文件（.s 文件）包含 STM32 的启动代码，其主要作用包括：1、堆栈（SP）的初始化；2、初始化程序计数器（PC）；3、设置向量表异常事件的入口地址；4、调用 main 函数等，是每个工程必不可少的一个文件，我们在本书第九章会有详细介绍。

启动文件由 ST 官方提供，存放在 STM32CubeH7 软件包的：Drivers→CMSIS→Device→ST→STM32H7xx→Source→Templates→arm 文件夹下。因为我们开发板使用的是 STM32H743IIT6，对应的启动文件为：startup_stm32h743xx.s，为了节省空间，在精简版 CMSIS 文件夹里面我们把其他启动文件都删了。

关于启动文件的说明，我们就介绍这么多，接下来我们看如何添加启动文件到工程里面。我们有两种方法给 MDK 的分组添加文件：1，双击 Project 下的分组名添加。2，进入工程管理界面添加。

这了我们使用方法 1 添加（路径：实验 0-3，新建工程实验-HAL 库版本\Drivers\CMSIS\Device\ST\STM32H7xx\Source\Templates\arm），如图 8.1.3.4 所示：

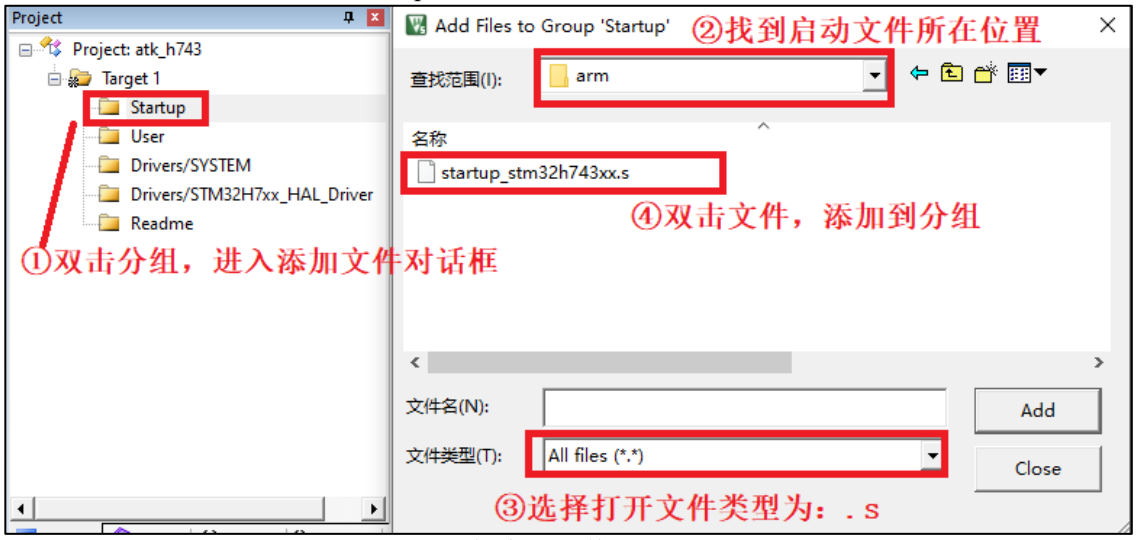


图 8.1.3.4 双击分组添加启动文件（startup_stm32h743xx.s）

上图中，我们也可以点击 Add 按钮进行文件添加。添加完后，点击 Close，完成启动文件添加，得到工程分组如图 8.1.3.5 所示：

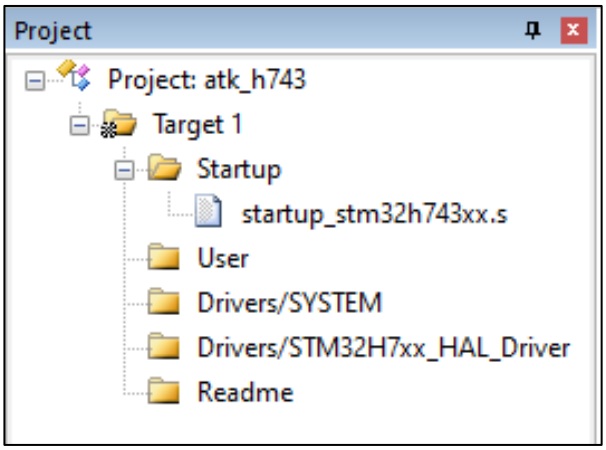


图 8.1.3.5 启动文件添加成功

3. 添加 SYSTEM 源码

这里我们在工程管理界面（方法 2）进行 SYSTEM 源码添加。点击：按钮，进入工程管理界面，选中 Drivers/SYSTEM 分组，然后点击：Add Files，进入文件添加对话框，依次添加 delay.c、sys.c 和 usart.c 到该分组下，如图 8.1.3.6 所示：

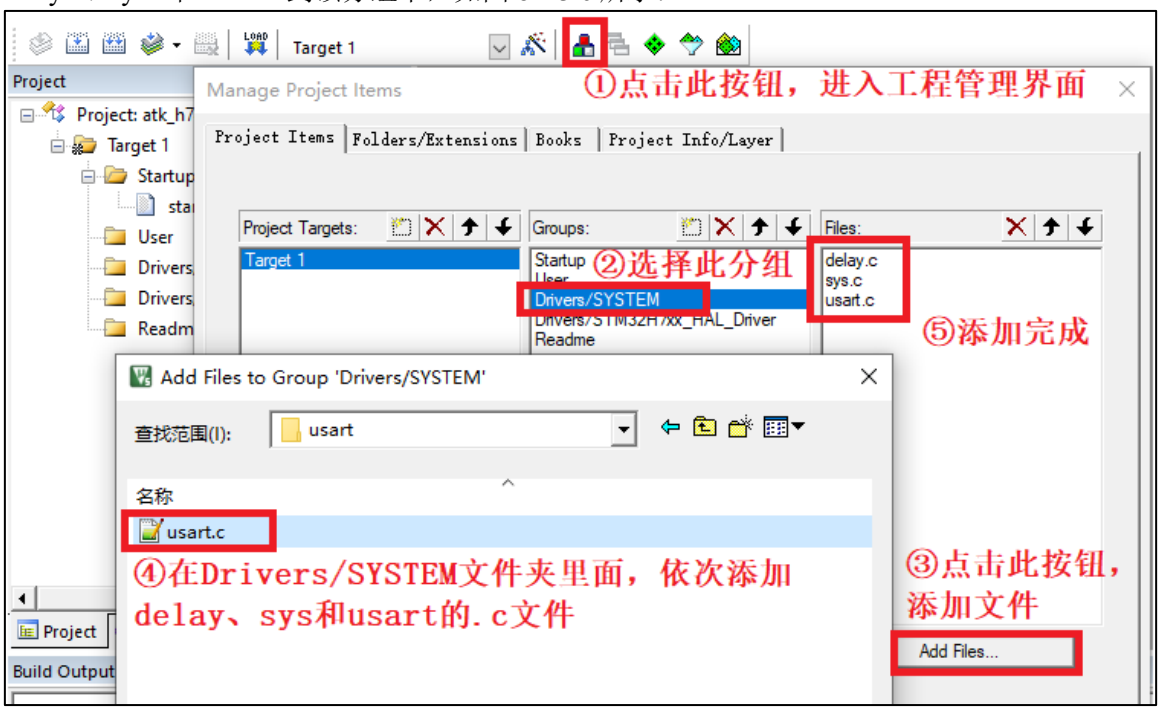


图 8.1.3.6 添加 SYSTEM 源码

注意：这些源码都是在第 8.1.1 小节的第二步拷贝过来的，如果之前没拷贝，是找不到这些源码的。添加完成后，如图 8.1.3.7 所示：

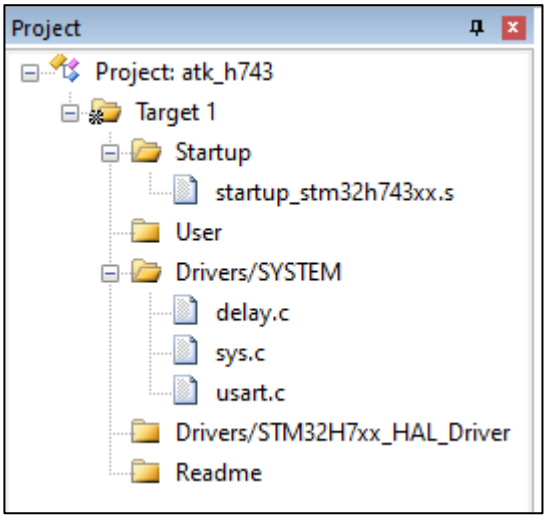


图 8.1.3.7 SYSTEM 源码添加完成

4. 添加 User 源码

这里我们在工程管理界面（方法 2）进行 User 源码添加。点击：按钮，进入工程管理界面，选中 User 分组，然后点击：Add Files，进入文件添加对话框，依次添加 stm32h7xx_it.c 和 system_stm32h7xx.c 到该分组下，如图 8.1.3.8 所示：

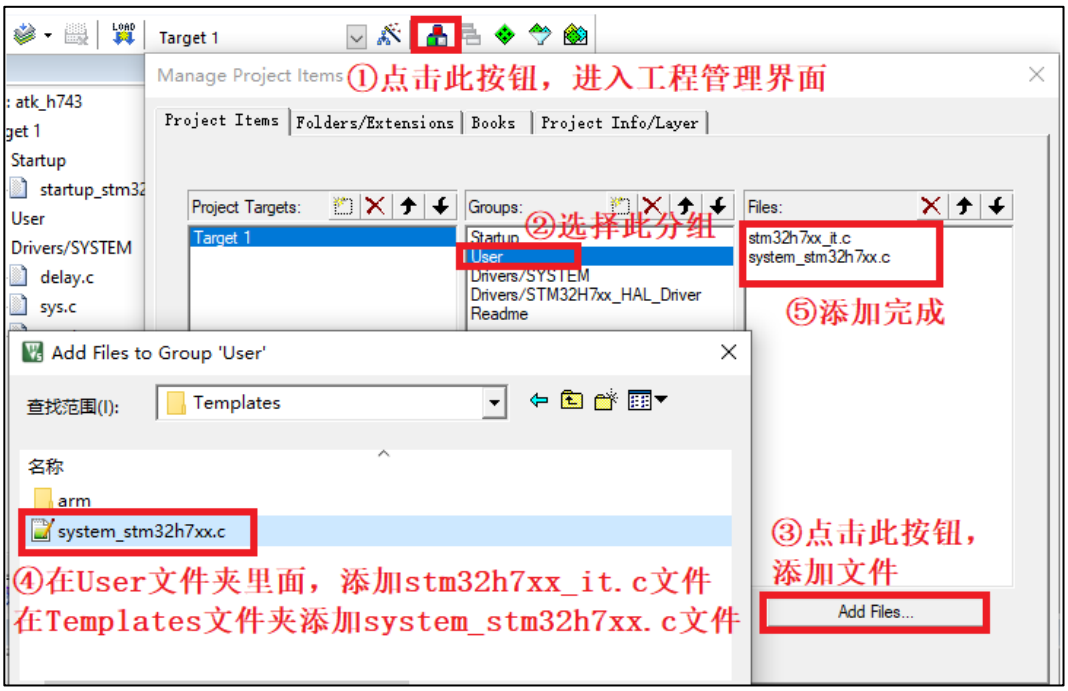


图 8.1.3.8 添加 User 源码

注意：这些源码都是在第 8.1.1 小节的第二步拷贝过来的，如果之前没拷贝，是找不到这些源码的。添加完成后，如图 8.1.3.9 所示：

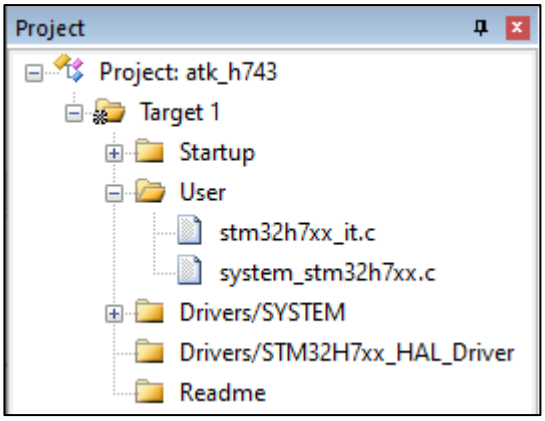


图 8.1.3.9 User 源码添加完成

5. 添加 STM32H7xx_HAL_Driver 源码

接下来我们往 Drivers/STM32H7xx_HAL_Driver 分组里添加文件。点击：按钮，进入工程管理界面，选中 Drivers/STM32H7xx_HAL_Driver 分组，然后点击：Add Files，进入文件添加对话框，依次添加 stm32h7xx_hal.c、stm32h7xx_hal_cortex.c、stm32h7xx_hal_dma.c、stm32h7xx_hal_gpio.c、stm32h7xx_hal_pwr.c、stm32h7xx_hal_pwr_ex.c、stm32h7xx_hal_rcc.c、stm32h7xx_hal_rcc_ex.c、stm32h7xx_hal_uart.c、stm32h7xx_hal_uart_ex.c、stm32h7xx_hal_usart.c 和 stm32h7xx_hal_usart_ex.c 到该分组下，如图 8.1.3.10 所示：

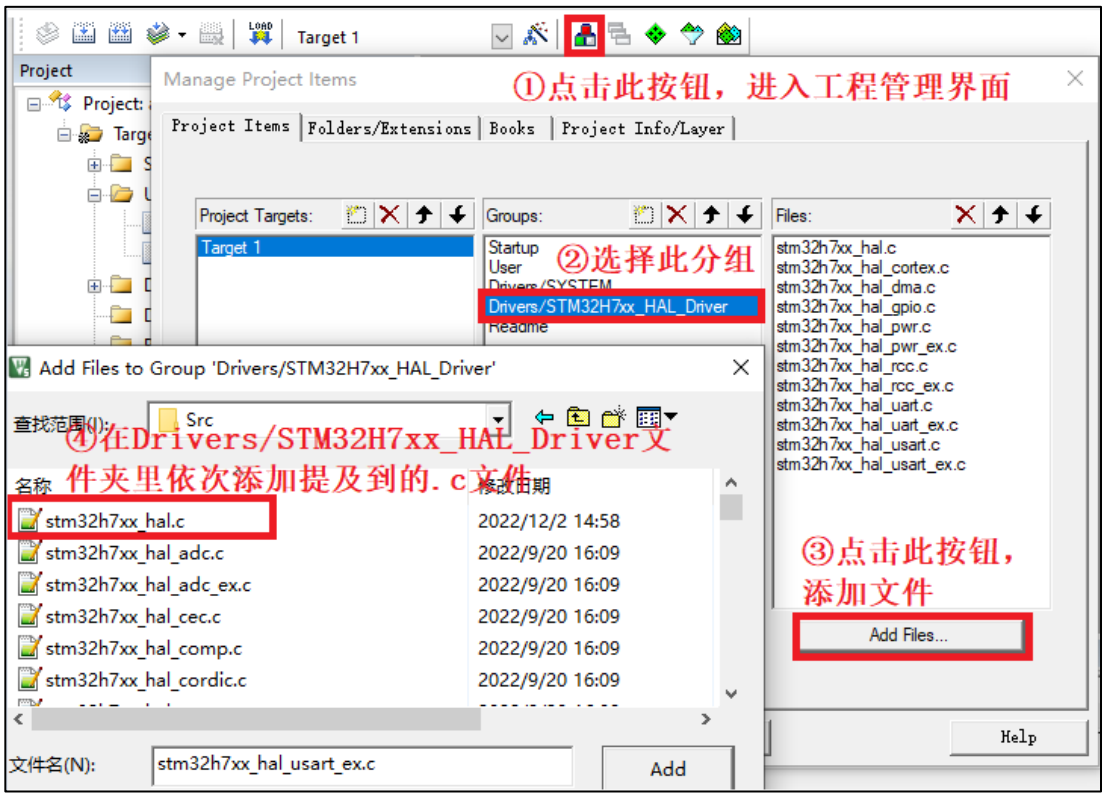


图 8.1.3.10 添加 STM32H7xx_HAL_Driver 源码

添加完成后，如图 8.1.3.11 所示：

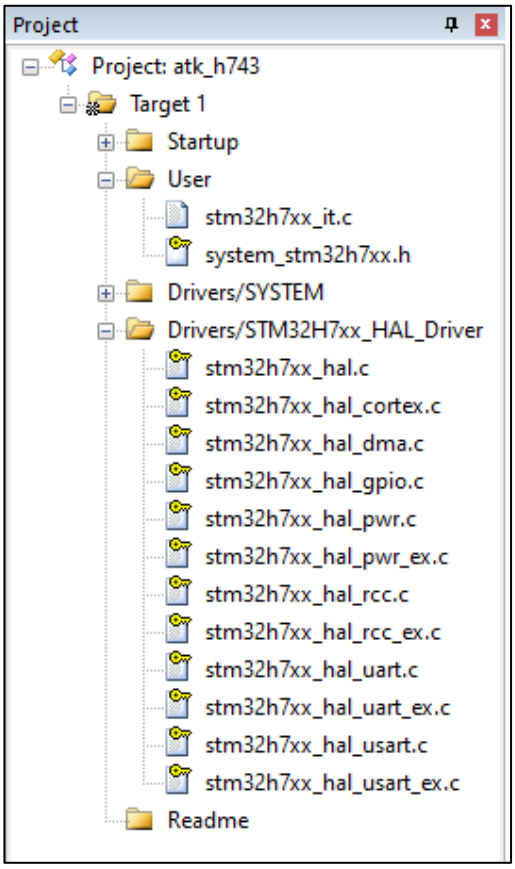


图 8.1.3.11 STM32H7xx_HAL_Driver 源码添加完成

可以看到分组中有些.c 文件有个小钥匙的符号，这是因为官方的固件包的文件设置了只读权限，我们取消只读权限就好了，方法如图 8.1.3.12 所示。

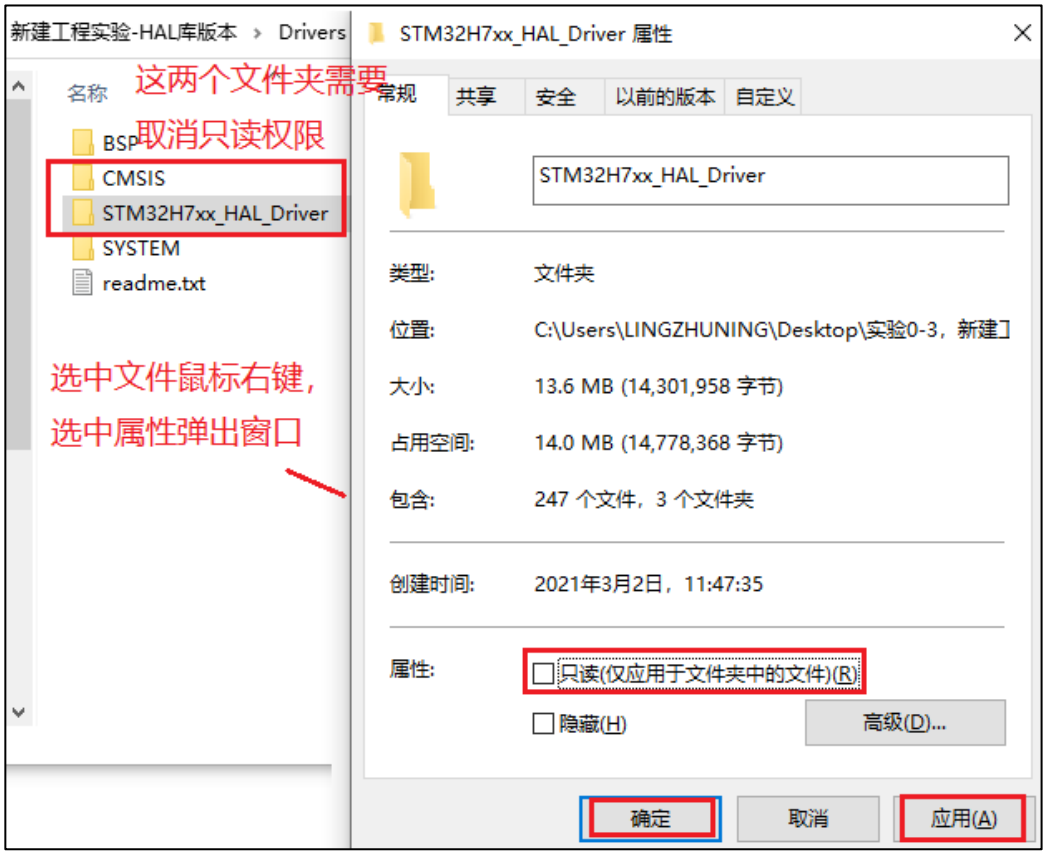


图 8.1.3.12 取消工程文件夹的只读权限

8.1.4 魔术棒设置

为避免编写代码和编译报错，我们需要通过魔术棒对 MDK 工程进行相关设置。在 MDK 主界面，点击：（魔术棒图标，即 Options for Target 按钮），进入工程设置对话框，我们将进行如下几个选项卡的设置。

1. 设置 Target 选项卡

在魔术棒→Target 选项卡里面，我们进行如图 8.1.4.1 所示设置：

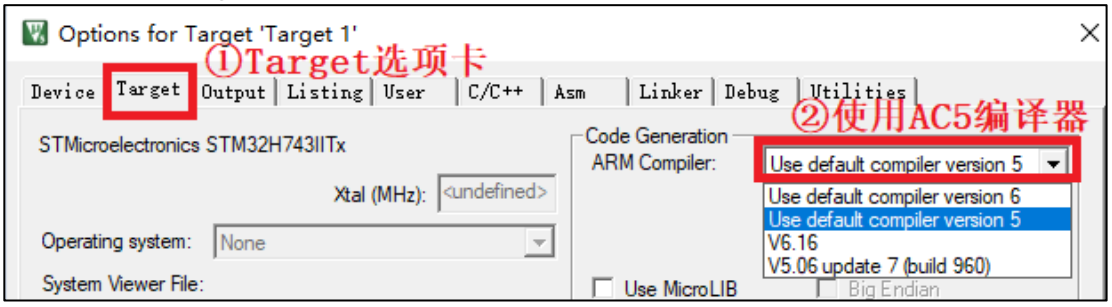


图 8.1.4.1 Target 选项卡设置

上图中，我们选择 ARM Compiler 版本为：Use default compiler version 5（即 AC5 编译器）。这里我们说明一下 AC5 和 AC6 编译的差异，如表 8.1.4.2 所示：

对比项	AC5	AC6	说明
中文支持	较好	较差	AC6 对中文支持极差，goto definition 无法使用，误报等
代码兼容性	较好	较差	AC6 对某些代码优化可能导致运行异常，需慢慢调试

编译速度	较慢	较快	AC6 编译速度比 AC5 快
语法检查	一般	严格	AC6 语法检查非常严格，代码严谨性较好

表 8.1.4.1 AC5&AC6 简单对比

由于 AC5 对中文支持比较好，且兼容性相对好一点，**为了避免不必要的麻烦，我们推荐大家使用 AC5 编译器**。为了让大家自由选择，我们正点原子的源码，也是支持 AC6 编译器的，不过在选项卡设置上稍有差异，具体差异如表 8.1.4.2 所示：

选项卡	AC5	AC6	说明
Target	选择 AC5 编译器	选择 AC6 编译器	选择对应的编译器
C/C++	Misc Controls 无需设置	Misc Controls 设置： -Wno-invalid-source-encoding	AC6 需设置编译选项以关闭对汉字的错误警告，AC5 则不要

表 8.1.4.2 AC5&AC6 设置差异

2. 设置 Output 选项卡

在魔术棒→Output 选项卡里面，进行如图 8.1.4.2 所示设置：

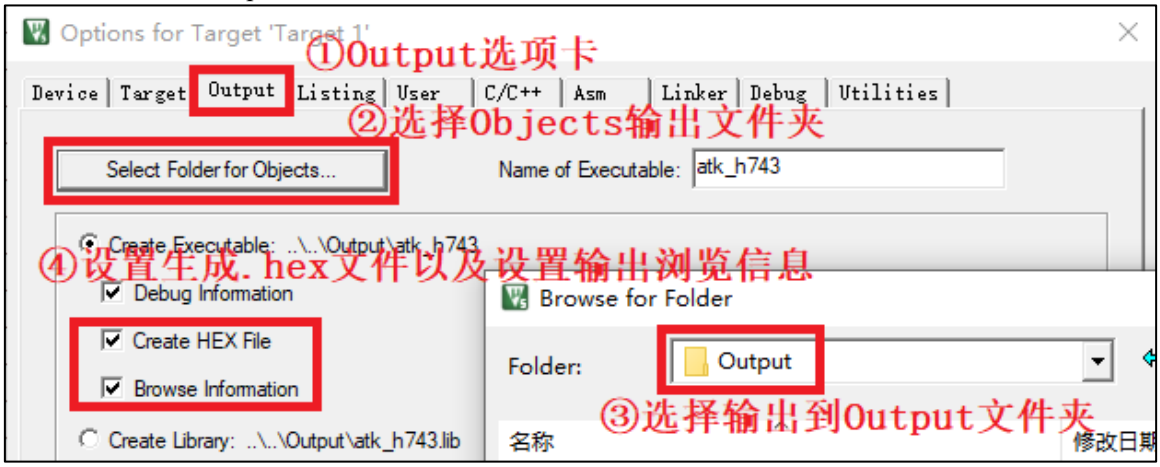


图 8.1.4.2 设置 Output 选项卡

注意，我们勾选：Browse Information，用于输出浏览信息，这样就可以使用 go to definition 查看函数/变量的定义，对我们后续调试代码比较有帮助，如果不需要调试代码，则可以去掉这个勾选，以提高编译速度。

3. 设置 Listing 选项卡

在魔术棒→Listing 选项卡里面，进行如图 8.1.4.3 所示设置：

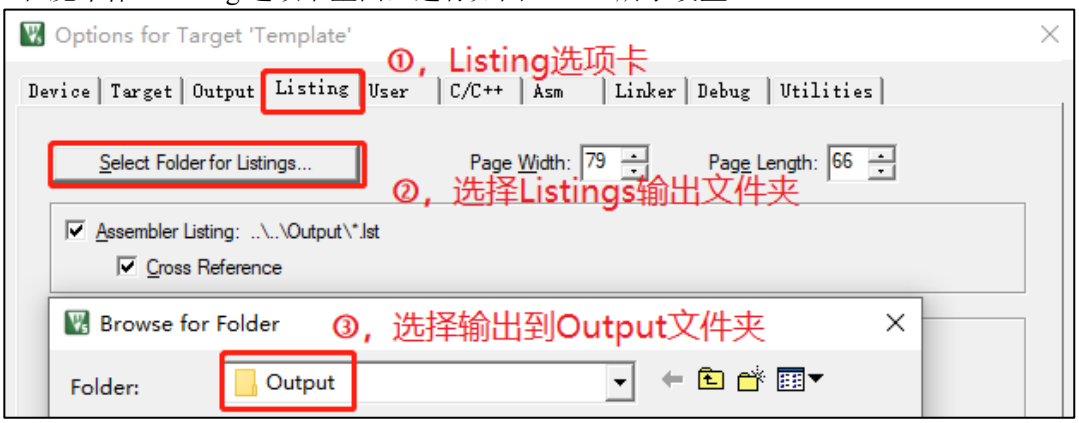


图 8.1.4.3 设置 Listing 选项卡

经过 Output 和 Listing 这两步设置，原来存储在 Objects 和 Listings 文件夹的内容（中间文件）就都改为输出到 Output 文件夹了。

4. 设置 C/C++选项卡

在魔术棒→C/C++选项卡里面，进行如图 8.1.4.4 所示设置：

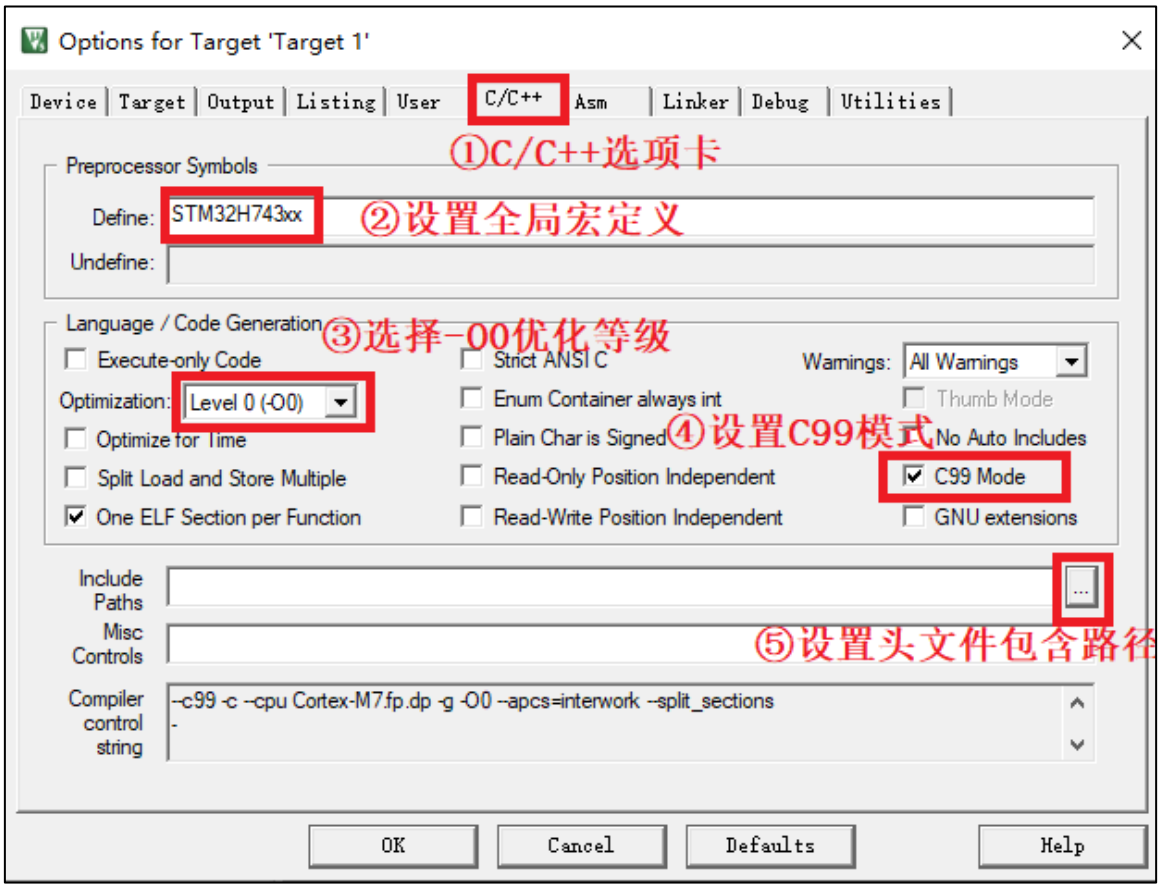


图 8.1.4.4 设置 C/C++选项卡

在②处设置了全局宏定义：STM32H743xx，用于定义所用 STM32 型号，在 stm32h7xx.h 里面会用到该宏定义。

在③处设置了优化等级为-O0，可以得到最好的调试效果，当然为了提高优化效果提升性能并降低代码量，可以设置-O1~-O3，数字越大效果越明显，不过也越容易出问题。注意：当使用 AC6 编译器的时候，这里推荐默认使用-O1 优化。

在④处勾选 C99 模式，即使用 C99 C 语言标准。

在⑤处，我们可以进行头文件包含路径设置，点击此按钮，进行如图 8.1.4.5 所示设置：

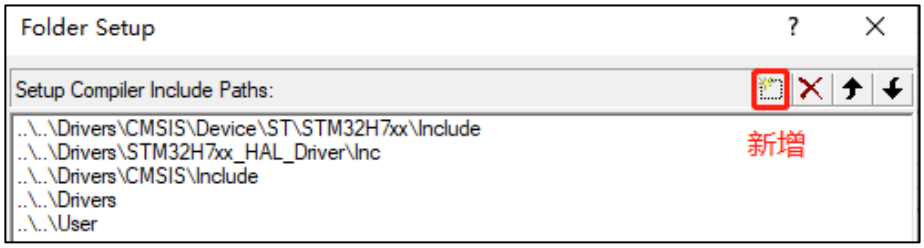


图 8.1.4.5 设置头文件包含路径

上图中我们设置了 5 个头文件包含路径，其中 4 个在 Drivers 文件夹下，一个在 User 文件夹下。为避免频繁设置头文件包含路径，正点原子最新源码的 include 全部使用相对路径，也就是我们只需要在头文件包含路径里面指定一个文件夹，那么该文件夹下的其他文件夹里面的源码，如果全部是使用相对路径，则无需再设置头文件包含路径了，直接在 include 里面就指明了头文件所在。

关于相对路径，这里大家记住 3 点：

1，默认路径就是指 MDK 工程所在的路径，即.uvprojx 文件所在路径（文件夹）

2，“./”表示当前目录（相对当前路径，也可以写做“.\”）

3，“../”表示当前目录的上一层目录（也可以写做“..\”）

举例来说，上图中：..\..\Drivers\CMSIS\Device\ST\STM32H7xx\Include，前面两个“..\”，表示 Drivers 文件夹在当前 MDK 工程所在文件夹（MDK-ARM）的上 2 级目录下，具体解释如图 8.1.4.6 所示：

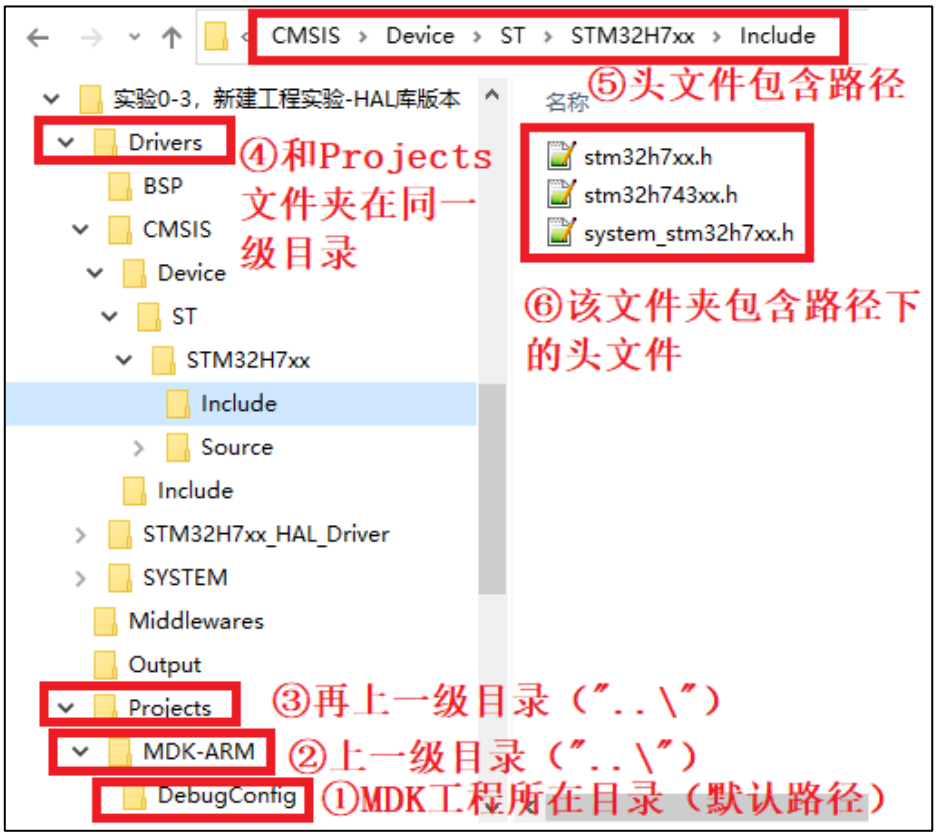


图 8.1.4.6 ..\..\Drivers\CMSIS\Device\ST\STM3H7xx\Include 的解释

上图表示根据头文件包含路径：..\..\Drivers\CMSIS\Device\ST\STM32H7xx\Include，编译器可以找到⑥处所包含的这些头文件，即代码里面可以直接 include 这些头文件使用。

再举个例子，在完成如图 6.1.4.5 所示的头文件包含路径设置以后，我们在代码里面编写：

```
#include "./SYSTEM/sys/sys.h"
```

即表示当前头文件包含路径所指示的 4 个文件夹里面，肯定有某一个文件夹包含了：SYSTEM/sys/sys.h 的路径，实际上就是在 Drivers 文件夹下面，两者结合起来就相当于：

```
#include "../../Drivers/SYSTEM/sys/sys.h"
```

这就是相对路径。它既可以减少头文件包含路径设置（即减少 MDK 配置步骤，免去频繁设置头文件包含路径的麻烦），同时又可以很方便的知道头文件具体在那个文件夹，因此我们推荐在编写代码的时候使用相对路径。

关于相对路径，我们就介绍这么多，大家搞不明白的可以在网上搜索相关资料学习，也可以在后面的学习，分析我们其他源码，慢慢体会，总之不难，但是好用。

最后，我们如果使用 AC6 编译器，则在图 6.1.4.4 的 Misc Controls 处需要设置：-Wno-invalid-source-encoding，避免中文编码报错，如果使用 AC5 编译器，则不需要该设置！！

5. 设置 Debug 选项卡

在魔术棒→Debug 选项卡里面，进行如图 8.1.4.7 所示设置：

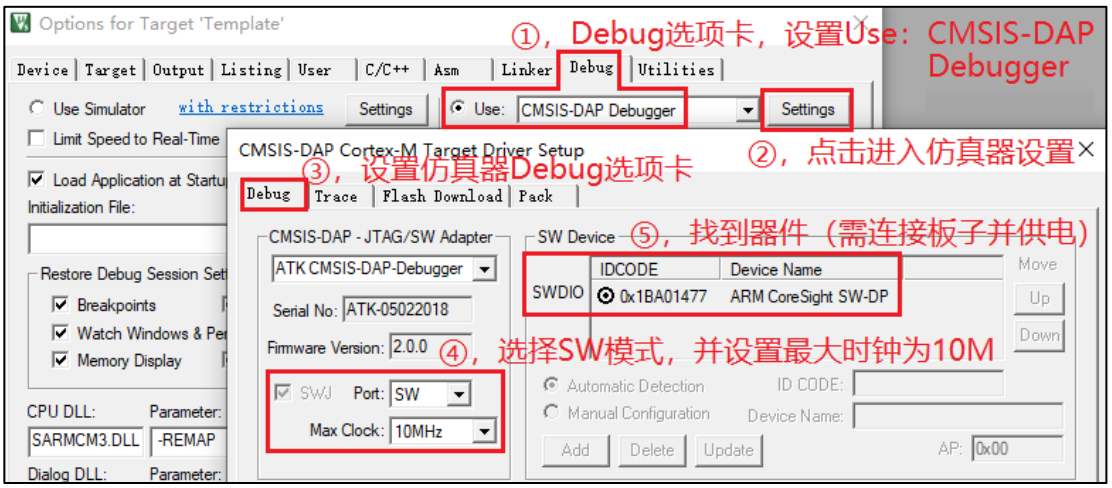


图 8.1.4.7 Debug 选项卡设置

图中，我们选择使用：CMSIS-DAP 仿真器，使用 SW 模式，并设置最大时钟频率为 10Mhz，以得到最高下载速度。当我们将仿真器和开发板连接好，并给开发板供电以后，仿真器就会找到开发板芯片，并在 SW Device 窗口显示芯片的 IDCODE、Device Name 等信息（图中⑤处），当无法找到时，请检查供电和仿真器连接状况。

6. 设置 Utilities 选项卡

在魔术棒→Debug 选项卡里面，进行如图 8.1.4.8 所示设置：

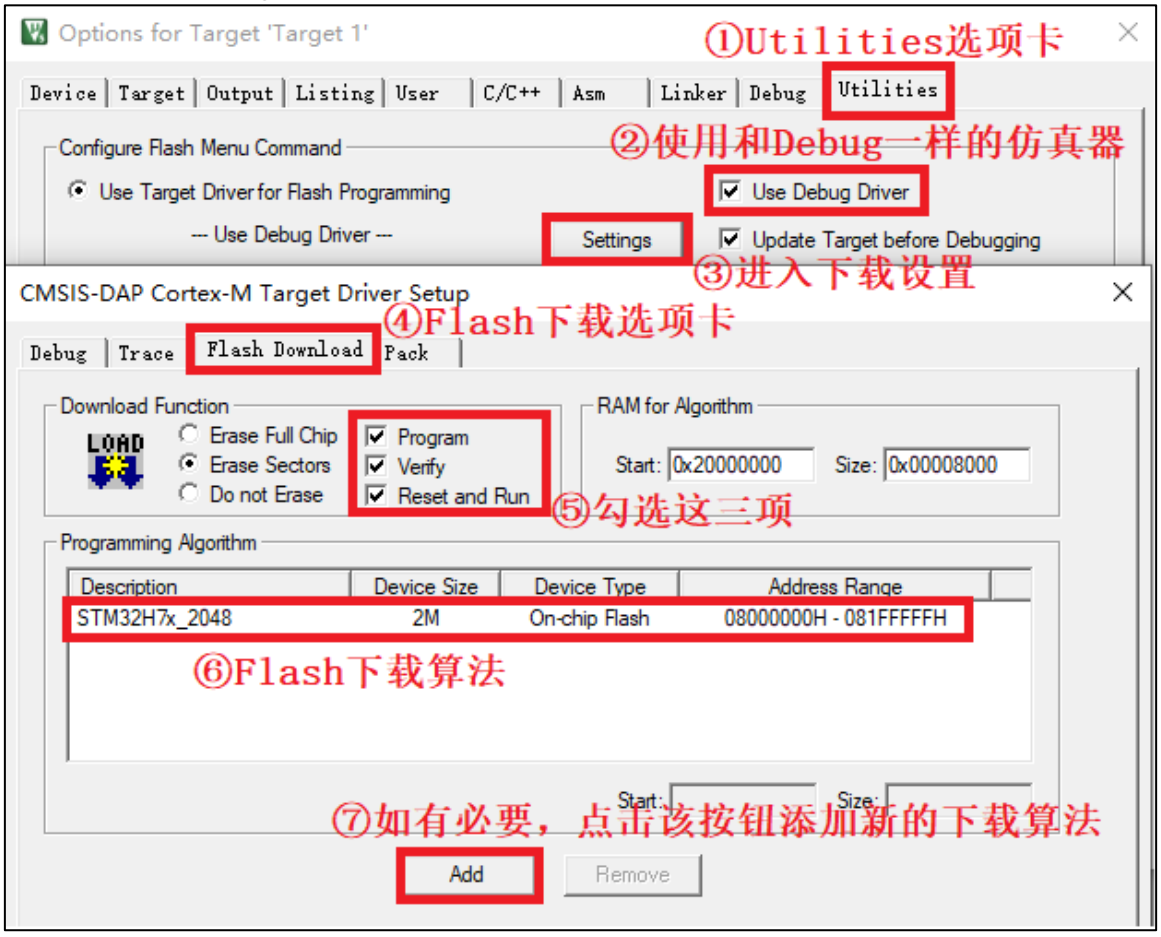


图 8.1.4.8 Utilities 选项卡设置

图中⑥处下载算法 STM32H743，是 MDK 默认添加的，针对 STM32H743 系列产品。一般我们用这个即可，如果⑥处没有下载算法，则点击 Add 按钮，添加下载算法即可（算法名字和⑥处的算法名字一样）。

8.1.5 添加 main.c，并编写代码

在 MDK 主界面，点击：，新建一个 main.c 文件，并保存在 User 文件夹下。然后双击 User 分组，弹出添加文件的对话框，将 User 文件夹下的 main.c 文件添加到 User 分组下。得到如图 8.1.5.1 所示的界面：

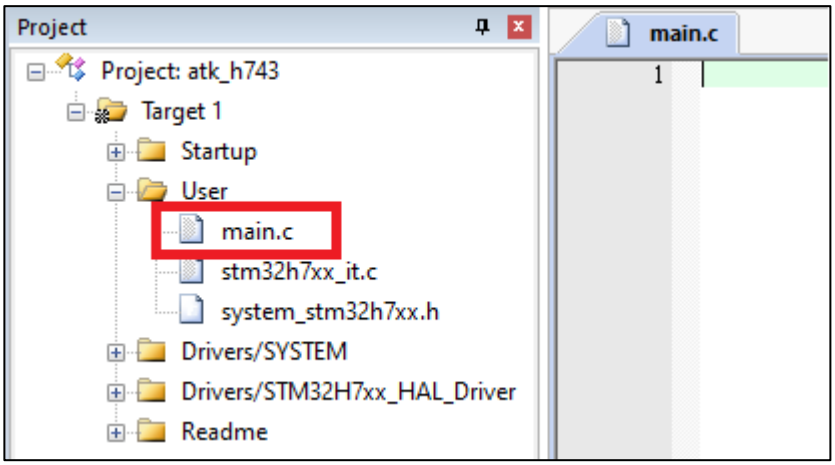


图 8.1.5.1 在 User 分组下加入 main.c 文件

至此，我们就可以开始编写我们自己的代码了。我们在 main.c 文件里面输入如下代码：

```
#include "./SYSTEM/sys/sys.h"
#include "./SYSTEM/usart/usart.h"
#include "./SYSTEM/delay/delay.h"

void led_init(void);                                /* LED 初始化函数声明 */

int main(void)
{
    sys_cache_enable();                             /* 打开 L1-Cache */
    HAL_Init();                                     /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);             /* 设置时钟, 400Mhz */
    delay_init(400);                                /* 延时初始化 */
    led_init();                                     /* LED 初始化 */
    while(1)
    {
        HAL_GPIO_WritePin(GPIOB,GPIO_PIN_0,GPIO_PIN_SET);       /* PB0 置 1 */
        HAL_GPIO_WritePin(GPIOE,GPIO_PIN_1,GPIO_PIN_RESET);     /* PB1 置 0 */
        delay_ms(500);
        HAL_GPIO_WritePin(GPIOE,GPIO_PIN_0,GPIO_PIN_RESET);     /* PB0 置 0 */
        HAL_GPIO_WritePin(GPIOE,GPIO_PIN_1,GPIO_PIN_SET);       /* PB1 置 1 */
        delay_ms(500);
    }
}

/**
 * @brief       初始化 LED 相关 IO 口, 并使能时钟
 * @param       无
 * @retval      无
 */
void led_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;
    __HAL_RCC_GPIOB_CLK_ENABLE(); /* PB0 时钟使能 */
```

```
    __HAL_RCC_GPIOB_CLK_ENABLE(); /* PB1 时钟使能 */

    gpio_init_struct.Pin = GPIO_PIN_0;                          /* LED0 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;                /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;                        /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;         /* 高速 */
    HAL_GPIO_Init(GPIOB, &gpio_init_struct);                    /* 初始化 LED0 引脚 */

    gpio_init_struct.Pin = GPIO_PIN_1;                          /* LED1 引脚 */
    HAL_GPIO_Init(GPIOB, &gpio_init_struct);                    /* 初始化 LED1 引脚 */
}
```

此部分代码，在 A 盘→4，程序源码→1，标准例程-HAL 库版本→ 实验 0 基础入门实验→实验 0-3，新建最工程实验-HAL 库版本→User→main.c 里面有，大家可以自己输入，也可以直接拷贝。强烈建议自己输入，以加深对程序的理解和印象!!

注意，这里的 include 就是使用的相对路径，关于相对路径，请参考前面 C/C++选项卡设置章节进行学习。

编写完 main.c，我们点击：（Rebuild）按钮，编译整个工程，编译结果如下图所示：

```
Build Output
Build started: Project: atk_h743
*** Using Compiler 'V5.06 update 7 (build 960)', folder: 'C:\Keil_v5\ARM\ARMCC\Bin'
Build target 'Template'
compiling stm32h7xx_it.c...
..\..\User\stm32h7xx_it.c(24): error:  #5: cannot open source input file "main.h": No such file or directory
  #include "main.h"
..\..\User\stm32h7xx_it.c: 0 warnings, 1 error
compiling sys.c...
compiling stm32h7xx_hal.c...
"..\..\Output\atk_h743.axf" - 1 Error(s), 0 Warning(s).
Target not created.
Build Time Elapsed:  00:00:00
```

双击

图 8.1.5.2 编译结果

编译结果可以看到 1 个错误，0 个警告。这个错误说找不到 main.h，因为我们也不需要用到 main.h，双击这个错误会弹出下面的 stm32h7xx_it.c 文件对应包含 main.h 的语句。我们只需要把它删除，然后重新编译，如图 8.1.5.3 所示。

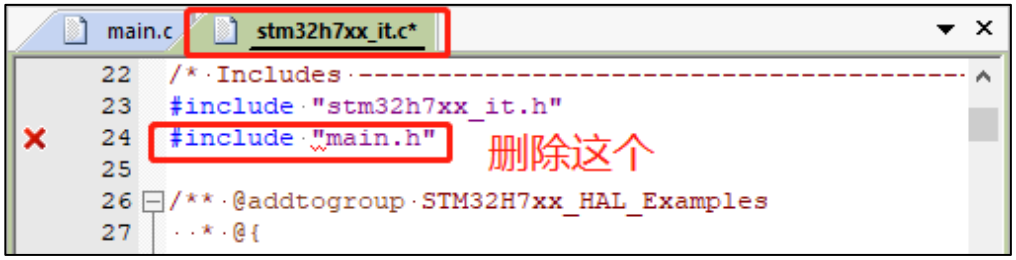


图 8.1.5.3 删除包含 main.h 的语句

编译后发现又有一个警告，警告 HAL_IncTick 函数没有声明，如图 8.1.5.4 所示。

```
Build Output
Build started: Project: atk_h743
*** Using Compiler 'V5.06 update 7 (build 960)', folder: 'C:\Keil_v5\ARM\ARMCC\Bin'
Build target 'Template'
compiling stm32h7xx_it.c...
..\..\User\stm32h7xx_it.c(189): warning:  #223-D: function "HAL_IncTick" declared implicitly
    HAL_IncTick();
..\..\User\stm32h7xx_it.c: 1 warning, 0 errors
linking...
Program Size: Code=11618 RO-data=714 RW-data=32 ZI-data=1976
FromELF: creating hex file...
"..\..\Output\atk_h743.axf" - 0 Error(s), 1 Warning(s).
Build Time Elapsed:  00:00:01
```

图 8.1.5.4 编译报警告

因为这个函数是在 stm32h7xx_hal.c 定义了，并且在 stm32h7xx_hal.h 声明了。我们把 stm32h7xx_hal.h 包含进来即可。这里还有一个原因是整个工程没有包含 stm32h7xx_hal.h 的语句，我们需要用到它，所以在这里把它包含进来。官方的 main.h 是有包含这个头文件的。我们

不用 main.h 文件，我们在 stm32h7xx_it.c 文件刚才删除包含 main.h 的语句的位置，编写包含 stm32h7xx_hal.h 语句，如图 8.1.5.5 所示。

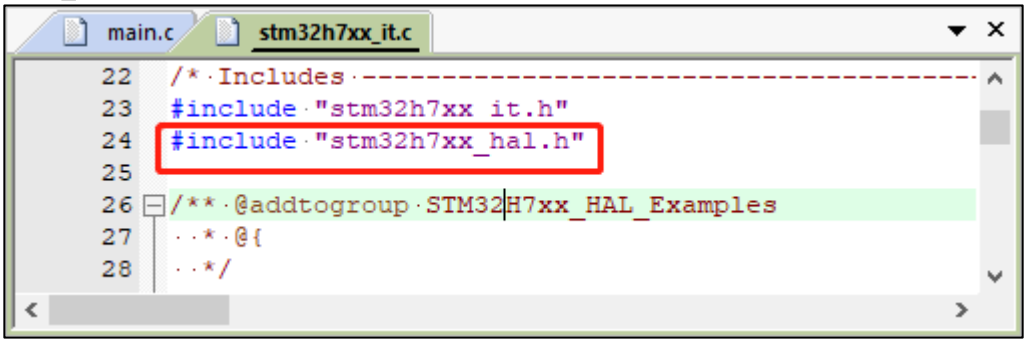


图 8.1.5.5 包含 stm32h7xx_hal.h 头文件到工程

再进行编译就会发现 0 错误 0 警告，结果如图 8.1.5.6 所示。

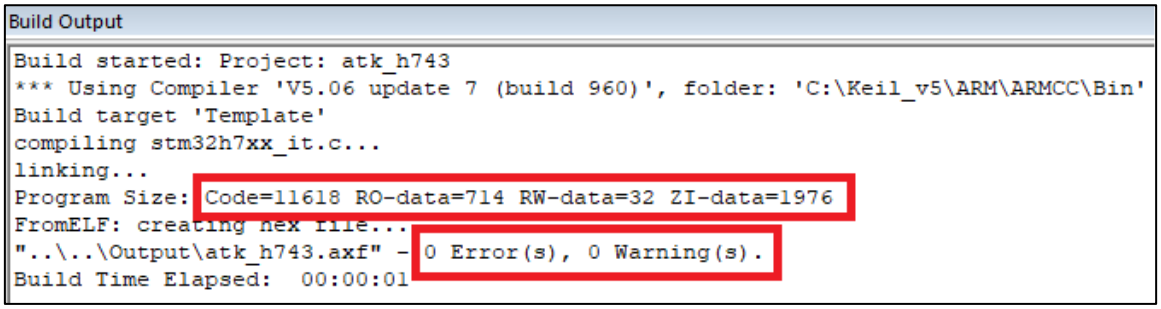


图 8.1.5.6 编译结果

编译结果提示：代码总大小（Porgram Size）为：FLASH 占用 12364 字节（Code + RO + RW），SRAM 占用 2008 字节（RW + ZI）；并成功创建了 Hex 文件（可执行文件，放在 Output 目录下）。

总结：如果编译提示有错误/警告，请根据提示，从第一个错误/警告开始解决，直到 0 错误 0 警告。如果出错，很有可能是之前的操作存在问题，请对照教程找问题。

另外，我们在 Readme 分组下还没有添加任何文件，由于只是添加一个说明性质的文件（.txt），并不是工程必备文件，因此这里我们就不添加了，开发板光盘的源码我们是有添加的，大家可以去参考一下。

至此，新建 HAL 库版本 MDK 工程完成。

8.2 下载验证

这里我们继续使用 DAP 仿真器下载，在 MDK 主界面，点击： （下载按钮，也可以按键盘快捷键：F8），就可以将代码下载到开发板，如图 8.2.1 所示：

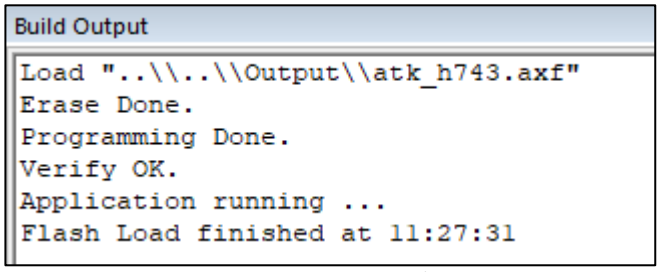


图 8.2.1 下载成功

上图提示：Application running...，则表示代码下载成功，且开始运行。此时，可以看到 LED0 和 LED1 交叉闪烁，类跑马灯。如果有朋友没能下载成功，请看第四章寻找问题，或者直接对照我们提供的实验 0-3，新建工程实验-HAL 库版本工程设置。

第九章 STM32 启动过程分析

本章给大家分析 STM32H7 的启动过程，这里的启动过程是指从 STM32 芯片上电复位执行的第一条指令开始，到执行用户编写的 main 函数这之间的过程。我们编写程序，基本都是用 C 语言编写，并且以 main 函数作为程序的入口。但是事实上，main 函数并非最先执行的，在此之前需要做一些准备工作，准备工作通过启动文件的程序来完成。理解 STM32 启动过程，对今后的学习和分析 STM32 程序有很大的帮助。

注意：学习本章内容之前，请大家最好先阅读由正点原子团队编写的《STM32 启动文件浅析_V1.2.pdf》和《STM32 MAP 文件浅析_V1.1.pdf》这两份文档（路径：A 盘→1，入门资料）。

本章将分为如下几个小节：

9.1 启动模式

9.2 启动文件分析

9.3 map 文件分析

9.1 启动模式

我们知道的复位方式有三种：上电复位，硬件复位和软件复位。当产生复位，并且离开复位状态后，CM7 内核做的第一件事就是读取下列两个 32 位整数的值：

（1）从地址 0x0000 0000 处取出堆栈指针 MSP 的初始值，该值就是栈顶地址。

（2）从地址 0x0000 0004 处取出程序计数器指针 PC 的初始值，该值指向复位后执行的第一条指令。下面用示意图表示，如图 9.1.1 所示。

图 9.1.1 复位序列

上述过程中，内核是从 0x0000 0000 和 0x0000 0004 两个的地址获取堆栈指针 SP 和程序计数器指针 PC。事实上，0x0000 0000 和 0x0000 0004 两个的地址可以被重映射到其他的地址空间。例如：我们将 0x0800 0000 映射到 0x0000 0000，即从内部 FLASH 启动，那么内核会从地址 0x0800 0000 处取出堆栈指针 MSP 的初始值，从地址 0x0800 0004 处取出程序计数器指针 PC 的初始值。CPU 会从 PC 寄存器指向的地址空间取出的第 1 条指令开始执行程序，就是开始执行复位中断服务程序 Reset_Handler。

将 0x0000 0000 和 0x0000 0004 两个的地址重映射到其他地址空间，就是启动模式选择。

对于 STM32H7 的启动模式（也称自举模式），我们看表 9.1.1 进行分析。

启动模式选择		启动地址
BOOT	启动地址选项字节	
0	BOOT_ADD0[15:0]	由用户选项字节 BOOT_ADD0[15:0]决定启动地址，ST 出厂默认的启动地址为:0X0800 0000 的 Flash 地址
1	BOOT_ADD1[15:0]	由用户选项字节 BOOT_ADD1[15:0]决定启动地址，ST 出厂默认的启动地址为:0X1FF0 0000 的系统存储器地址

表 9.1.1 启动模式选择表

注：启动引脚（BOOT）的电平：0：低电平；1：高电平。

由表 9.1.1 可以看到，STM32H7 的启动模式选择需要一个 BOOT 引脚，这个 BOOT 引脚对应了 0 和 1 两个状态，这两个状态都有自己对应的启动地址选项字节，用于选择自己的启动地址。

这里的 BOOT_ADD0[15:0]和 BOOT_ADD1[15:0]对应 32 位地址的高 16 位，也就是说我们只能设置高 16 位对应的地址（低 16 位必须是 0），其设置范围：0X0000 0000～0X3FFF 0000，涵盖了所有 FLASH 地址空间，所有 RAM 地址空间：ITCM、DTCM RAM 和 SRAM，还有 TCM-RAM 地址空间。基本上，可以设置任意地址启动（低 16 位必须是 0），通过 FLASH_BOOT_PRGR 寄存器设置。

在出厂的时候，ST 默认给 BOOT_ADD0 和 BOOT_ADD1 编程为：0X0800 0000 和 0X1FF0 0000 分别对应用户 FLASH 的起始地址和系统存储器地址，用于执行用户代码或者进入 BOOTLOADER 状态。一般情况下我们设置 B00T 引脚为低电平即可，即从 0X0800 0000 的 FLASH 地址启动，执行用户代码。

9.2 启动文件分析

STM32 启动文件由 ST 官方提供，在官方的 STM32Cube 固件包里，对于 STM32H743 系列芯片的启动文件，我们选用的是 startup_stm32h743xx.s 这个文件。启动文件用汇编编写，是系统上电复位后第一个执行的程序。

启动文件主要做了以下工作：

1、初始化堆栈指针 SP = _initial_sp
2、初始化程序计数器指针 PC = Reset_Handler
3、设置堆和栈的大小
4、初始化中断向量表
5、配置外部 SRAM 作为数据存储器（可选）
6、配置系统时钟，通过调用 SystemInit 函数（可选）
7、调用 C 库中的 _main 函数初始化用户堆栈，最终调用 main 函数

9.2.1 启动文件中的一些指令

指令名称	作用
EQU	给数字常量取一个符号名，相当于 C 语言中的 define
AREA	汇编一个新的代码段或者数据段
ALIGN	编译器对指令或者数据的存放地址进行对齐，一般需要跟一个立即数，缺省表示 4 字节对齐。要注意的是，这个不是 ARM 的指令，是编译器的，这里放到一起为了方便。
SPACE	分配内存空间
PRESERVE8	当前文件堆栈需要按照 8 字节对齐
THUMB	表示后面指令兼容 THUMB 指令。在 ARM 以前的指令集中有 16 位的 THUMBM 指令，现在 Cortex-M 系列使用的都是 THUMB-2 指令集，THUMB-2 是 32 位的，兼容 16 位和 32 位的指令，是 THUMB 的超级版。
EXPORT	声明一个标号具有全局属性，可被外部的文件使用
DCD	以字节为单位分配内存，要求 4 字节对齐，并要求初始化这些内存
PROC	定义子程序，与 **ENDP** 成对使用，表示子程序结束
WEAK	弱定义，如果外部文件声明了一个标号，则优先使用外部文件定义的标号，如果外部文件没有定义也不会出错。要注意的是，这个不是 ARM 的指令，是编译器的，这里放到一起为了方便。
IMPORT	声明标号来自外部文件，跟 C 语言中的 extern 关键字类似
LDR	从存储器中加载字到一个存储器中
BLX	跳转到由寄存器给出的地址，并根据寄存器的 LSE 确定处理器的状态，还要把跳转前的下条指令地址保存到 LR
BX	跳转到由寄存器/标号给出的地址，不用返回
B	跳转到一个标号
IF,ELSE,ENDIF	汇编条件分支语句，跟 C 语言的类似

END	到达文件的末尾，文件结束

表 9.2.1.1 启动文件的汇编指令

上表，列举了 STM32 启动文件的一些汇编和编译器指令，关于其他更多的 ARM 汇编指令，我们可以通过 MDK 的索引搜索工具中搜索找到。打开索引搜索工具的方法：MDK->Help->uVision Help，如图 9.2.1.1 所示。

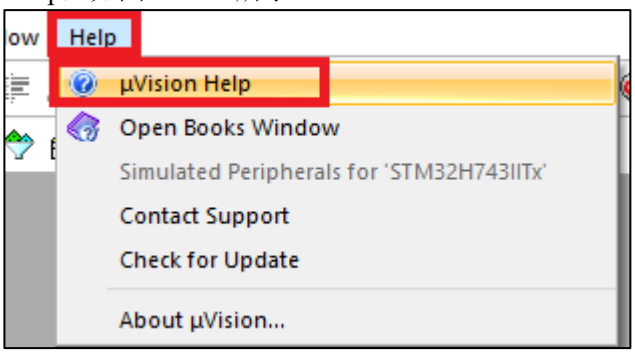


图 9.2.1.1 打开索引搜索工具的方法

打开之后，我们以 **EQU** 为例，演示一下怎么使用，如图 9.2.1.2 所示。

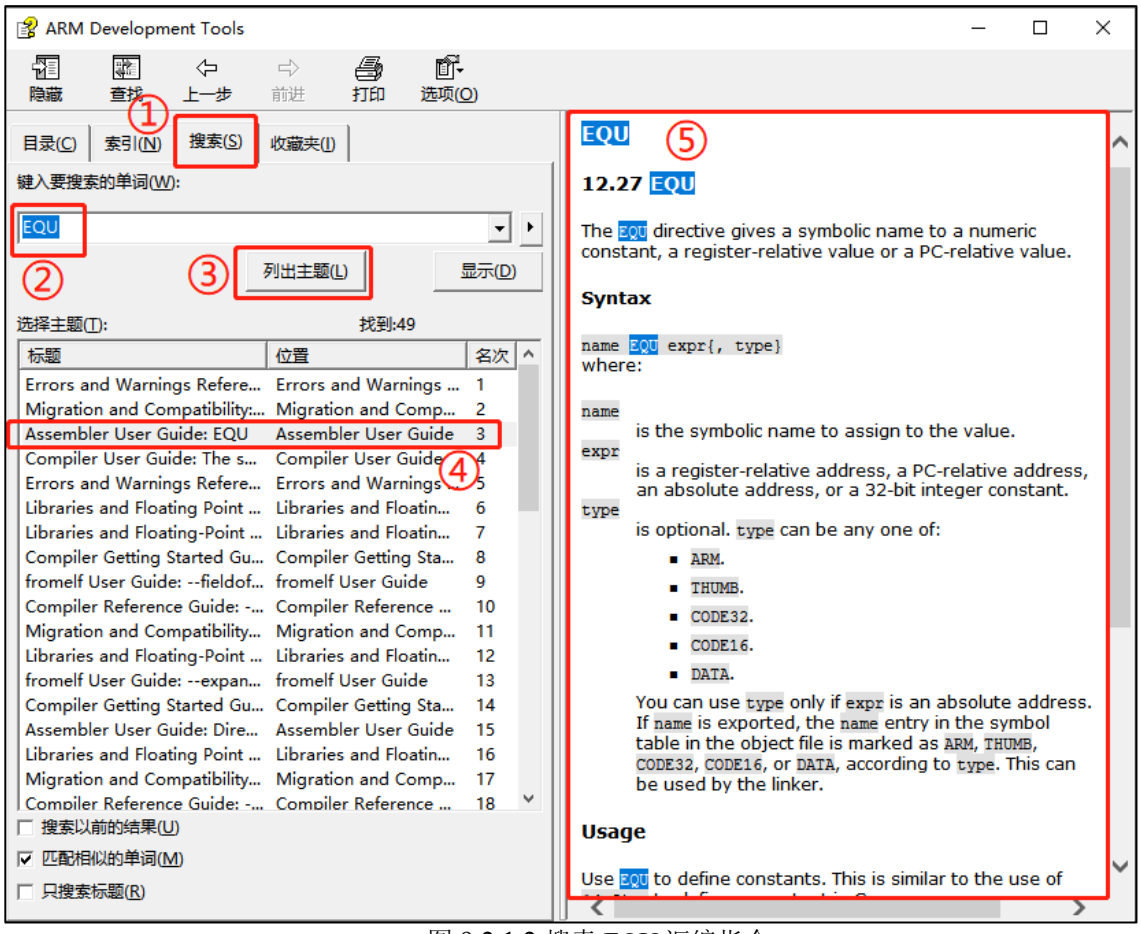


图 9.2.1.2 搜索 EQU 汇编指令

搜索到的标题有很多，我们只需要看 Assembler User Guide 这部分即可。

9.2.2 启动文件代码讲解

（1）栈空间的开辟

栈空间的开辟，源码如图 9.2.2.1 所示：

```
32  Stack_Size      EQU     0x00000800
33
34                  AREA    STACK, NOINIT, READWRITE, ALIGN=3
35  Stack_Mem       SPACE   Stack_Size
36  __initial_sp
```

图 9.2.2.1 栈空间的开辟

源码含义：开辟一段大小为 0x0000 0800（2KB）的栈空间，段名为 STACK，NOINIT 表示不初始化； READWRITE 表示可读可写；ALIGN=3，表示按照 2^3 对齐，即 8 字节对齐。

AREA 汇编一个新的代码段或者数据段。

SPACE 分配内存指令，分配大小为 Stack_Size 字节连续的存储单元给栈空间。

__initial_sp 紧挨着 SPACE 放置，表示栈的结束地址，栈是从高往低生长，所以结束地址就是栈顶地址。

栈主要用于存放局部变量，函数形参等，属于编译器自动分配和释放的内存，栈的大小不能超过内部 SRAM 的大小。如果工程的程序量比较大，定义的局部变量比较多，那么就需要在启动代码中修改栈的大小，即修改 Stack_Size 的值。如果程序出现了莫名其妙的错误，并进入了 HardFault 的时候，你就要考虑下是不是栈空间不够大，溢出了的问题。

（2）堆空间的开辟

堆空间的开辟，源码如图 9.2.2.2 所示：

```
44  Heap_Size       EQU     0x00000200
45
46                  AREA    HEAP, NOINIT, READWRITE, ALIGN=3
47  __heap_base
48  Heap_Mem        SPACE   Heap_Size
49  __heap_limit
```

图 9.2.2.2 堆空间的开辟

源码含义：开辟一段大小为 0x0000 0200（512 字节）的堆空间，段名为 HEAP，不初始化，可读可写，8 字节对齐。

__heap_base 表示堆的起始地址，__heap_limit 表示堆的结束地址。堆和栈的生长方向相反的，堆是由低向高生长，而栈是从高往低生长。

堆主要用于动态内存的分配，像 malloc()、calloc()和 realloc()等函数申请的内存就在堆上面。堆中的内存一般由程序员分配和释放，若程序员不释放，程序结束时可能由操作系统回收。

接下来是 PRESERVE8 和 THUMB 指令两行代码。如图 9.2.2.3 所示。

```
51                  PRESERVE8
52                  THUMB
```

图 9.2.2.3 PRESERVE8 和 THUMB 指令

PRESERVE8：指示编译器按照 8 字节对齐。

THUMB：指示编译器之后的指令为 THUMB 指令。

注意：由于正点原子提供了独立的内存管理实现方式（mymalloc，myfree 等），并不需要使用 C 库的 malloc 和 free 等函数，也就用不到堆空间，因此我们可以设置 Heap_Size 的大小为 0，以节省内存空间。

（3）中断向量表定义（简称：向量表）

为中断向量表定义一个数据段，如图 9.2.2.4 所示：

```
56                  AREA    RESET, DATA, READONLY
57                  EXPORT  __Vectors
58                  EXPORT  __Vectors_End
59                  EXPORT  __Vectors_Size
```

图 9.2.2.4 为中断向量表定义一个数据段

源码含义：定义一个数据段，名字为 RESET, READONLY 表示只读。EXPORT 表示声明一个标号具有全局属性，可被外部的文件使用。这里是声明了__Vectors、__Vectors_End 和

__Vectors_Size 三个标号具有全局性，可被外部的文件使用。

STM32H743 的中断向量表定义代码，如图 9.2.2.5 所示。

```
60  __Vectors       DCD     __initial_sp               ; Top of Stack
61                  DCD     Reset_Handler              ; Reset Handler
62                  DCD     NMI_Handler                ; NMI Handler
63                  DCD     HardFault_Handler          ; Hard Fault Handler
64                  DCD     MemManage_Handler          ; MPU Fault Handler
65                  DCD     BusFault_Handler           ; Bus Fault Handler
66                  DCD     UsageFault_Handler         ; Usage Fault Handler
67                  DCD     0                          ; Reserved
68                  DCD     0                          ; Reserved
69                  DCD     0                          ; Reserved
70                  DCD     0                          ; Reserved
71                  DCD     SVC_Handler                ; SVCall Handler
72                  DCD     DebugMon_Handler           ; Debug Monitor Handler
73                  DCD     0                          ; Reserved
74                  DCD     PendSV_Handler             ; PendSV Handler
75                  DCD     SysTick_Handler            ; SysTick Handler
76
77                  ; External Interrupts
78                  DCD     WWDG_IRQHandler            ; Window WatchDog interrupt (wwdg1_it)
79                  DCD     PVD_AVD_IRQHandler         ; PVD/AVD through EXTI Line detection
80                  ;中间篇幅太长了，省略掉了，代码向量表与STM32H743的向量表对应
81                  DCD     WAKEUP_PIN_IRQHandler      ; Interrupt for all 6 wake-up pins
82
83
84  __Vectors_End
85
86  __Vectors_Size  EQU     __Vectors_End - __Vectors
```

图 9.2.2.5 中断向量表定义代码

__Vectors 为向量表起始地址，__Vectors_End 为向量表结束地址，__Vectors_Size 为向量表大小，__Vectors_Size = __Vectors_End - __Vectors。

DCD：分配一个或者多个以字为单位的内存，以四字节对齐，并要求初始化这些内存。

中断向量表被放置在代码段的最前面。例如：当我们的程序在 FLASH 运行时，那么向量表的起始地址是：0x0800 0000。结合图 9.2.2.5 可以知道，地址 0x0800 0000 存放的是栈顶地址。DCD：以四字节对齐分配内存，也就是下个地址是 0x0800 0004，存放的是 Reset_Handler 中断函数入口地址。

从代码上看，向量表中存放的都是中断服务函数的函数名，所以 C 语言中的函数名对芯片来说实际上就是一个地址。

STM32H743 的中断向量表可以在《STM32H7xx 参考手册_V3（中文版）.pdf》的第 19 章的 19.1.2 小节找到，与中断向量表定义代码是对应的。

（4）复位程序

接下来是定义只读代码段，如图 9.2.2.6 所示：

```
235                     AREA    |.text|, CODE, READONLY
```

图 9.2.2.6 定义只读代码段

定义一个段命为.text，只读的代码段，在 CODE 区。

复位子程序代码，如图 9.2.2.7 所示：

```
238  Reset_Handler    PROC
239                   EXPORT  Reset_Handler                 [WEAK]
240          IMPORT  SystemInit
241          IMPORT  __main
242
243                   LDR     R0, =SystemInit
244                   BLX     R0
245                   LDR     R0, =__main
246                   BX      R0
247                   ENDP
```

图 9.2.2.7 复位子程序代码

利用 PROC、ENDP 这一对伪指令把程序段分为若干个过程，使程序的结构加清晰。

复位子程序是复位后第一个被执行的程序，主要是调用 SystemInit 函数配置系统时钟、还有就是初始化 FSMC/FMC 总线上外挂的 SRAM(可选)。然后在调用 C 库函数__main，最终调用 main 函数去到 C 的世界。

EXPORT 声明复位中断向量 Reset_Handler 为全局属性，这样外部文件就可以调用此复位中断服务。

WEAK：表示弱定义，如果外部文件优先定义了该标号则首先引用外部定义的标号，如果外部文件没有声明也不会出错。这里表示复位子程序可以由用户在其他文件重新实现，这里并不是唯一的。

IMPORT 表示该标号来自外部文件。这里表示 SystemInit 和__main 这两个函数均来自外部的文件。

LDR、BLX、BX 是内核的指令，可在《CM3 权威指南 CnR2》第四章-指令集里面查询到。

LDR 表示从存储器中加载字到一个存储器中。

BLX 表示跳转到由寄存器给出的地址，并根据寄存器的 LSE 确定处理器的状态，还要把跳转前的下条指令地址保存到 LR。

BX 表示跳转到由寄存器/标号给出的地址，不用返回。这里表示切换到__main 地址，最终调用 main 函数，不返回，进入 C 的世界。

（5）中断服务程序

接下来就是中断服务程序了，如图 9.2.2.8 所示：

```
250  ;系统异常中断部分
251  NMI_Handler     PROC
252                  EXPORT  NMI_Handler                [WEAK]
253                  B       .          ;原地跳转（即无限循环）
254                  ENDP
255  HardFault_Handler\
256                  PROC
257                  EXPORT  HardFault_Handler          [WEAK]
258                  B       .
259                  ENDP
260  ;中间代码太长，已经省略掉
261  SysTick_Handler PROC
262                  EXPORT  SysTick_Handler             [WEAK]
263                  B       .
264                  ENDP
265  ;外部中断部分
266  Default_Handler PROC
267
268                  EXPORT  WWDG_IRQHandler             [WEAK]
269                  EXPORT  PVD_AVD_IRQHandler          [WEAK]
270  ;中间代码太长，已经省略掉
271  SAI4_IRQHandler
272  WAKEUP_PIN_IRQHandler
273
274                  B       .
275
276                  ENDP
```

图 9.2.2.8 中断服务程序

可以看到这些中断服务函数都被[WEAK]声明为弱定义函数，如果外部文件声明了一个标号，则优先使用外部文件定义的标号，如果外部文件没有定义也不会出错。

这些中断函数分为系统异常中断和外部中断，外部中断根据不同芯片有所变化。B 指令是跳转到一个标号，这里跳转到一个‘.’，表示无限循环。

在启动文件代码中，已经把我们所有中断的中断服务函数写好了，但都是声明为弱定义，所以真正的中断服务函数需要我们在外部实现。

如果我们开启了某个中断，但是忘记写对应的中断服务程序函数又或者把中断服务函数名写错，那么中断发生时，程序就会跳转到启动文件预先写好的弱定义的中断服务程序中，并且在 B 指令作用下跳转到一个‘.’中，无限循环。

这里的系统异常中断部分是内核的，外部中断部分是外设的。

（6）用户堆栈初始化

ALIGN 指令，如图 9.2.2.9 所示：

```
583                                 ALIGN
```

图 9.2.2.9 ALIGN 指令

ALIGN 表示对指令或者数据的存放地址进行对齐，一般需要跟一个立即数，缺省表示 4 字节对齐。要注意的是，这个不是 ARM 的指令，是编译器的。

接下就是启动文件最后一部分代码，用户堆栈初始化代码，如图 9.2.2.10 所示：

```
585 ····IF ·····:DEF:__MICROLIB
586 ····
587 ····EXPORT ··__initial_sp
588 ····EXPORT ··__heap_base
589 ····EXPORT ··__heap_limit
590 ····
591 ····ELSE
592 ····
593 ····IMPORT ··__use_two_region_memory
594 ····EXPORT ··__user_initial_stackheap
595 ····
596 user_initial_stackheap
597 
598 ····LDR ····R0, ·= ··Heap_Mem
599 ····LDR ····R1, ·=(Stack_Mem ·+ ·Stack_Size)
600 ····LDR ····R2, ·= ·(Heap_Mem ·+ ··Heap_Size)
601 ····LDR ····R3, ·= ·Stack_Mem
602 ····BX ·····LR
603 
604 ····ALIGN
605 
606 ····ENDIF
607 
608 ····END
```

图 9.2.2.10 用户堆栈初始化代码

IF, ELSE, ENDIF 是汇编的条件分支语句。

585 行判断是否定义了__MICROLIB。关于__MICROLIB 这个宏定义，我们是在 KEIL 里面配置，具体方法如图 9.2.2.11 所示。

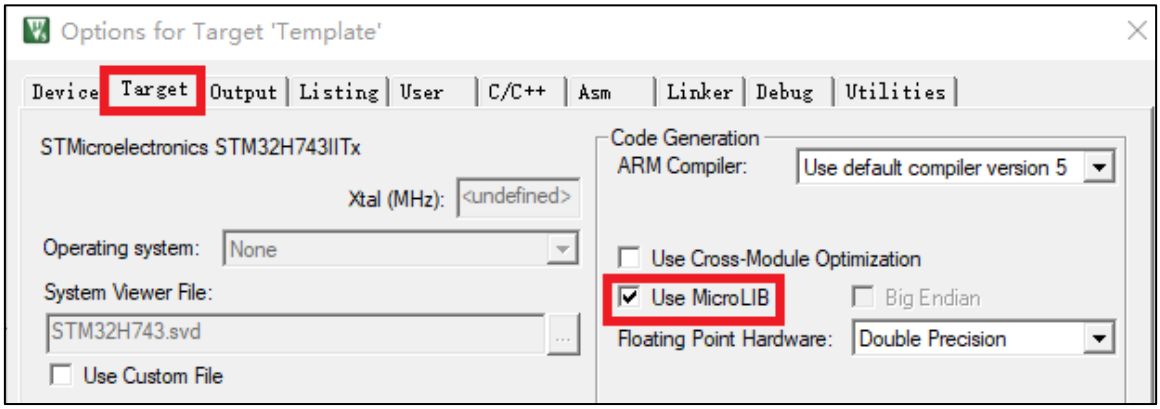


图 9.2.2.11 __MICROLIB 定义方法

勾选了 Use MicroLIB 就代表定义了__MICROLIB 这个宏。

如果定义__MICROLIB，声明__initial_sp、__heap_base 和__heap_limit 这三个标号具有全局属性，可被外部的文件使用。__initial_sp 表示栈顶地址，__heap_base 表示堆起始地址，__heap_limit 表示堆结束地址。

如果没有定义__MICROLIB，实际的情况就是我们没有定义__MICROLIB，所以使用默认的 C 库运行。那么堆栈的初始化由 C 库函数__main 来完成。

IMPORT 声明__use_two_region_memory 标号来自外部文件。

EXPORT 声明__user_initial_stackheap 具有全局属性，可被外部的文件使用。

596 行标号__user_initial_stackheap，表示用户堆栈初始化程序入口。

接下来进行堆栈空间初始化，堆是从低到高生长，栈是从高到低生长，是两个互相独立的数据段，并且不能交叉使用。

598 行保存堆起始地址。

599 行保存栈大小。

600 行保存堆大小。

601 行保存栈顶指针。

602 行跳转到 LR 标号给出的地址，不用返回。

608 行 END 表示到达文件的末尾，文件结束。

Use MicroLIB

MicroLIB 是 MDK 自带的微库，是缺省 C 库的备选库，MicroLIB 进行了高度优化使得其代码变得很小，功能比缺省 C 库少。MicroLIB 是没有源码的，只有库。

关于 MicroLIB 更多知识可以看官方介绍 http://www.keil.com/arm/microlib.asp 。

9.2.3 系统启动流程

我们知道启动模式不同，启动的起始地址是不一样的，下面我们以代码下载到内部 FLASH 的情况举例，即代码从地址 0x0800 0000 开始被执行。

当产生复位，并且离开复位状态后，CM7 内核做的第一件事就是读取下列两个 32 位整数的值：

（1）从地址 0x0800 0000 处取出堆栈指针 MSP 的初始值，该值就是栈顶地址。

（2）从地址 0x0800 0004 处取出程序计数器指针 PC 的初始值，该值指向中断服务程序 Reset_Handler。下面用示意图表示，如图 9.2.3.1 所示。

图 9.2.3.1 复位序列

我们看看阿波罗 STM32H743 开发板 HAL 库例程的实验 1 跑马灯实验中，取出的 MPS 和 PC 的值是多少，方法如图 9.2.3.2 所示。

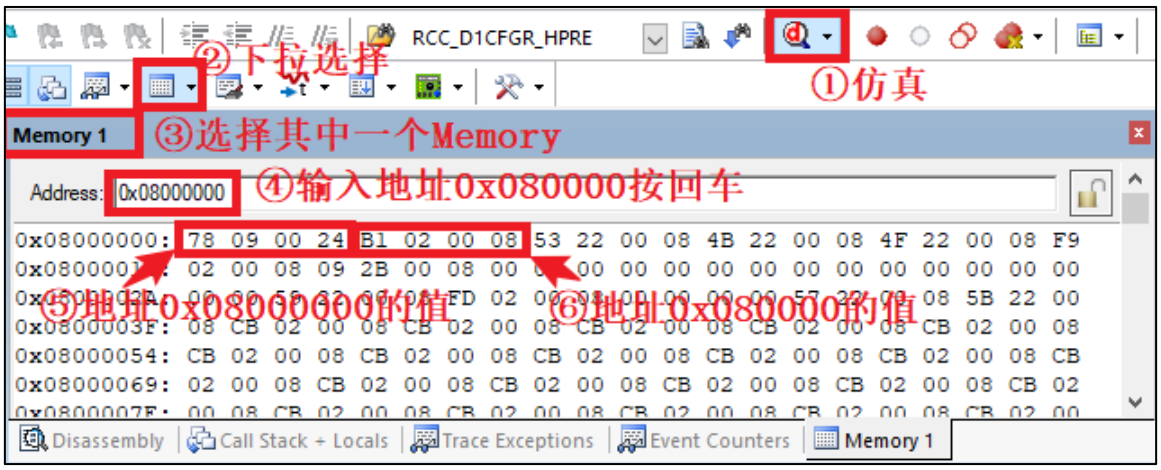


图 9.2.3.2 取出的 MPS 和 PC 的值

由图 9.2.3.2 可以知道地址 0x0800 0000 的值是 0x2400 0978，地址 0x0800 0004 的值是 0x0800 02B1，即堆栈指针 SP =0x2400 0978，程序计数器指针 PC = 0x0800 02B1（即复位中断服务程序 Reset_Handler 的入口地址），即。因为 CM7 内核是小端模式，所以倒着读。

请注意，这与传统的 ARM 架构不同——其实也和绝大多数的其它单片机不同。传统的 ARM 架构总是从 0 地址开始执行第一条指令。它们的 0 地址处总是一条跳转指令。而在 CM3

内核中，0 地址处提供 MSP 的初始值，然后就是向量表（向量表在以后还可以被移至其它位置）。向量表中的数值是 32 位的地址，而不是跳转指令。向量表的第一个条目指向复位后应执行的第一条指令，就是 Reset_Handler 这个函数。下面继续以正点原子阿波罗 STM32H743 开发板 HAL 库例程的实验 1 跑马灯实验为例，代码从地址 0x0800 0000 开始被执行，讲解一下系统启动，初始化堆栈、MSP 和 PC 后的内存情况。

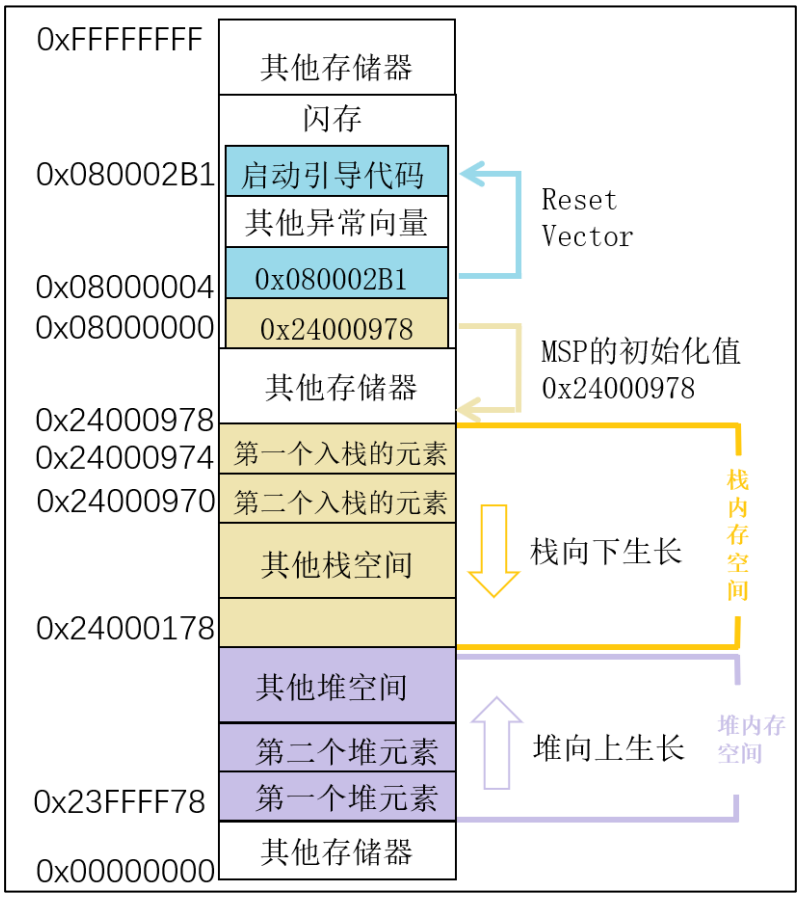


图 9.2.3.3 初始化堆栈、MSP 和 PC 后的内存情况

因为 CM3 使用的是向下生长的满栈，所以 MSP 的初始值必须是堆栈内存的末地址加 1。举例来说，如果你的栈区域在 0x2400 0178 - 0x2400 0974 （2KB 大小）之间，那么 MSP 的初始值就必须是 0x2400 0978。

向量表跟随在 MSP 的初始值之后——也就是第 2 个表目。

R15 是程序计数器，在汇编代码中，可以使用名字“PC”来访问它。ARM 规定：PC 最低两位并不表示真实地址，最低位 LSB 用于表示是 ARM 指令（0）还是 Thumb 指令（1），因为 CM3 主要执行 Thumb 指令，所以这些指令的最低位都是 1（都是奇数）。因为 CM3 内部使用了指令流水线，读 PC 时返回的值是当前指令的地址+4。比如说：

```
0x1000 :  MOV R0,  PC ;  R0 = 0x1004
```

如果向 PC 写数据，就会引起一次程序的分支（但是不更新 LR 寄存器）。CM3 中的指令至少是半字对齐的，所以 PC 的 LSB 总是读回 0。然而，在分支时，无论是直接写 PC 的值还是使用分支指令，都必须保证加载到 PC 的数值是奇数（即 LSB=1），表明是在 Thumb 状态下执行。倘若写了 0，则视为转入 ARM 模式，CM3 将产生一个 fault 异常。

正因为上述原因，图 9.2.3.3 中使用 0x0800 02B1 来表达地址 0x0800 02B0。当 0x0800 02B1 处的指令得到执行后，就正式开始了程序的执行（即去到 C 的世界）。所以在此之前初始化 MSP 是必需的，因为可能第 1 条指令还没执行就会被 NMI 或是其它 fault 打断。MSP 初始化好后就已经为它们的服务例程准备好了堆栈。

STM32 启动文件分析就给大家介绍到这里，更多内容请看 **《STM32 启动文件浅析_V1.1》** 。

9.3 map 文件分析

9.3.1 MDK 编译生成文件简介

MDK 编译工程，会生成一些中间文件（如.o、.axf、.map 等），最终生成 hex 文件，以便下载到 MCU 上面执行，以阿波罗 STM32H743 开发板 HAL 库例程的实验 1 跑马灯实验为例（其他开发板类似），编译过程产生的所有文件，都存放在 OBJ 文件夹下，如图 9.3.1.1 所示：

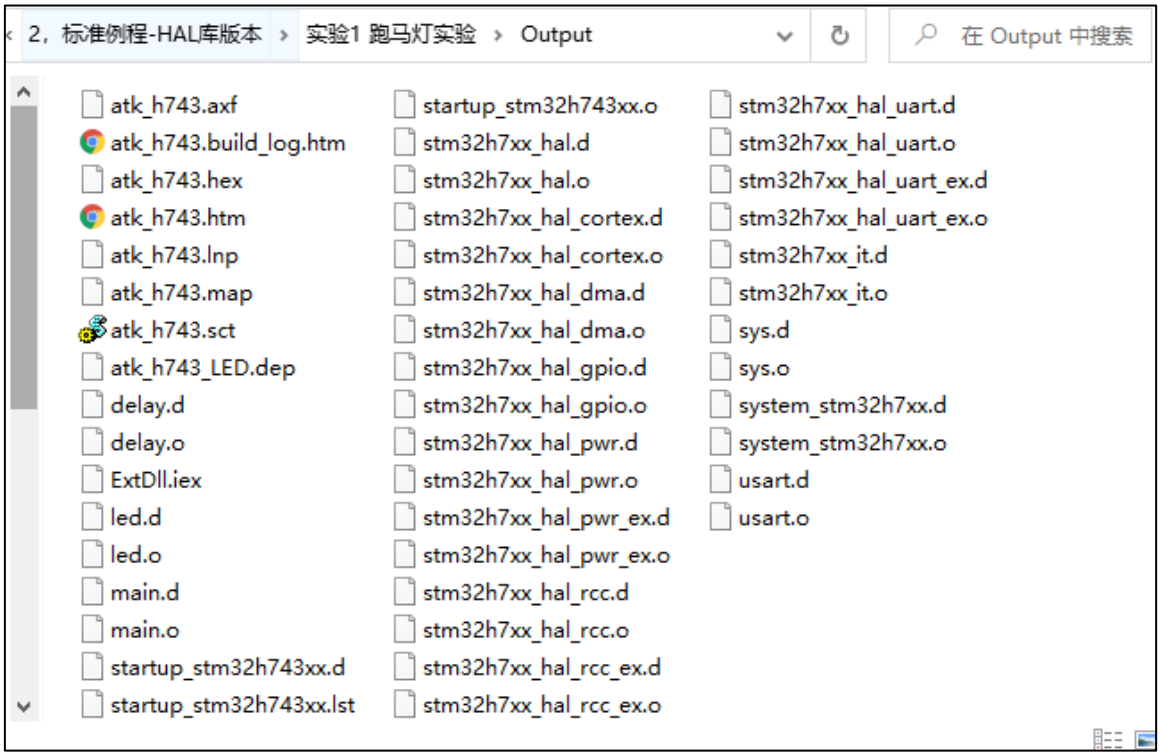


图 9.3.1.1 MDK 编译过程生成的文件

可以看到，这里总共生成了 46 个文件，共 11 个类型，分别是：.axf、.crf、.d、.dep、.hex、.lnp、.lst、.o、.htm、bulild_log.htm 和.map。46 个文件看着不是很多，但是随着工程的增大，这些文件也会越来越多，大项目编译一次，可以生成几百甚至上千个这种文件，不过文件类型基本就是上面这些。

对于 MDK 工程来说，基本上任何工程在编译过程中都会有这 11 类文件，常见的 MDK 编译过程生产文件类型如表 9.3.1.1 所示：

文件类型	说明
.o	可重定向 1 对象文件，每个源文件（.c/.s 等）编译都会生成一个.o 文件
.axf	由 ARMCC 编译生产的可执行对象文件，不可重定向 2 （绝对地址） 多个.o 文件链接生成.axf 文件，我们在仿真的时候，需要用到该文件
.hex	Intel Hex 格式文件，可用于下载到 MCU，.hex 文件由.axf 文件转换而来
.crf	交叉引用文件，包含浏览信息（定义、标识符、引用）
.d	由 ARMCC/GCC 编译生产的依赖文件（.o 文件所对应的依赖文件） 每个.o 文件，都有一个对应的.d 文件
.dep	整个工程的依赖文件
.lnp	MDK 生成的链接输入文件，用于命令输入
.lst	C 语言或汇编编译器生成的列表文件
.htm	链接生成的列表文件
.build_log.htm	最近一次编译工程时的日志记录文件
.map	连接器生成的列表文件/MAP 文件，**该文件对我们非常有用**

表 9.3.1.1 常见的中间文件类型说明

注 1，可重定向是指该文件包含数据/代码，但是并没有指定地址，它的地址可由后续链接的时候进行指定。

注 2，不可重定向是指该文件所包含的数据/代码都已经指定地址了，不能再改变。

9.3.2 map 文件分析

.map 文件是编译器链接时生成的一个文件，它主要包含了交叉链接信息。通过.map 文件，我们可以知道整个工程的函数调用关系、FLASH 和 RAM 占用情况及其详细汇总信息，能具体到单个源文件（.c/.s）的占用情况，根据这些信息，我们可以对代码进行优化。.map 文件可以分为以下 5 个组成部分：

1， 程序段交叉引用关系（Section Cross References）
2， 删除映像未使用的程序段（Removing Unused input sections from the image）
3， 映像符号表（Image Symbol Table）
4， 映像内存分布图（Memory Map of the image）
5， 映像组件大小（Image component sizes）

9.3.2.1 map 文件的 MDK 设置

要生成 map 文件，我们需要在 MDK 的魔术棒→Listing 选项卡里面，进行相关设置，如图 9.3.2.1.1 所示：

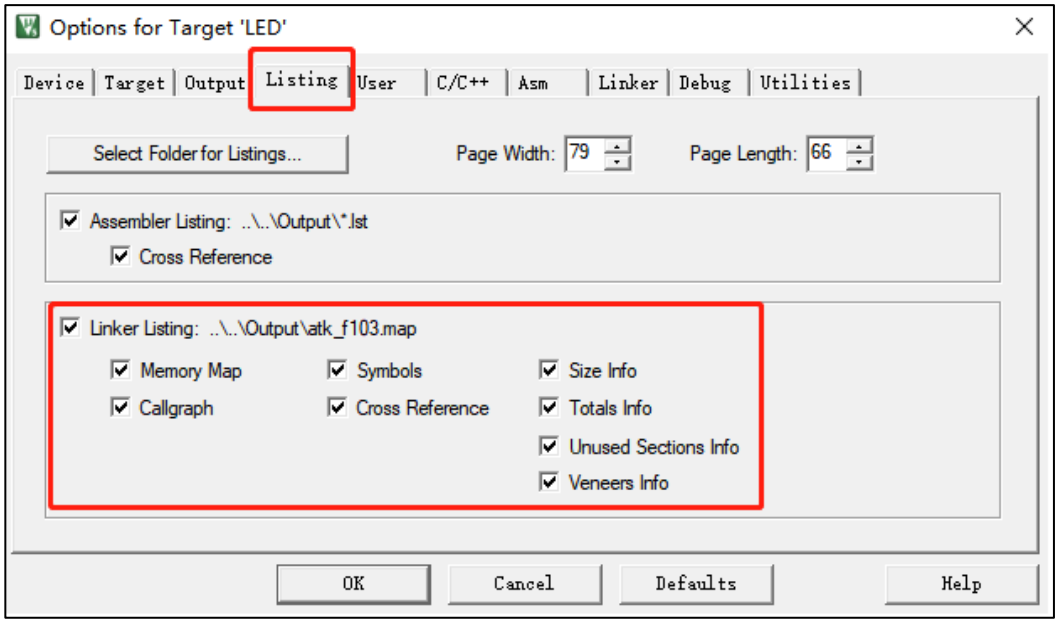


图 9.3.2.1.1 .map 文件生成设置

图 9.3.2.1.1 中红框框出的部分就是我们需要设置的，默认情况下，MDK 这部分设置就是全勾选的，如果我们想取消掉一些信息的输出，则取消相关勾选即可（一般不建议）。

如图 9.3.2.1.1 设置好 MDK 以后，全编译当前工程，当编译完成后（无错误），就会生成.map 文件。在 MDK 里面打开.map 文件的方法如图 9.3.2.1.2 所示：

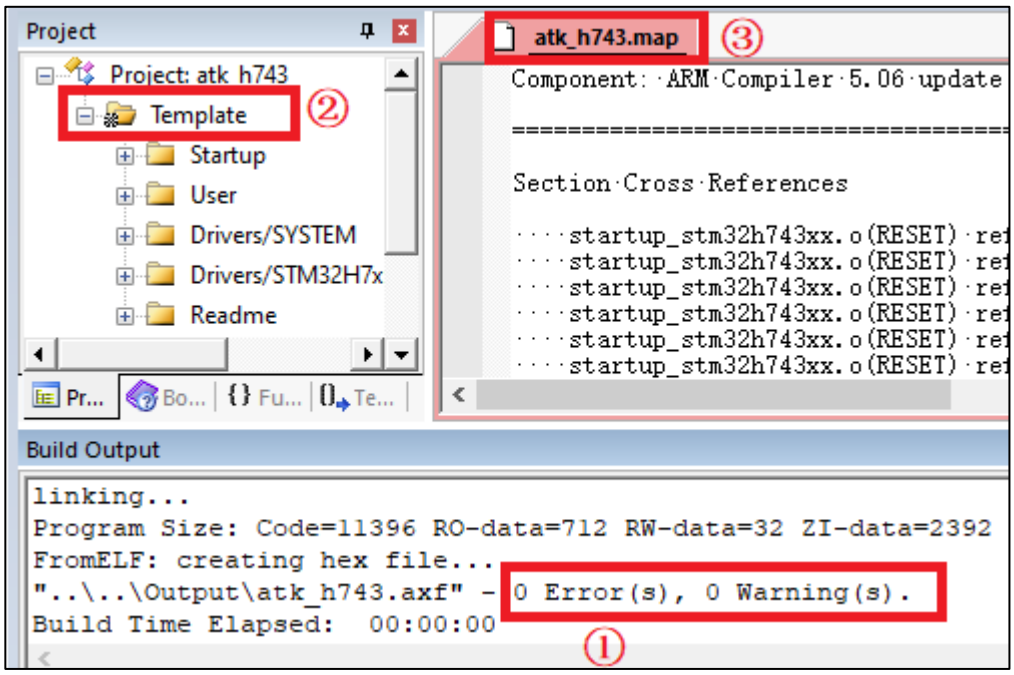


图 9.3.2.1.2 打开.map 文件

①先确保工程编译成功（无错误）。
②双击 LED，打开.map 文件。
③map 文件打开成功。

9.3.2.2 map 文件的基础概念

为了更好的分析 map 文件，我们先对需要用到的一些基础概念进行一个简单介绍，相关概念如下：

- Section：描述映像文件的代码或数据块，我们简称程序段
- RO：Read Only 的缩写，包括只读数据（RO data）和代码（RO code）两部分内容，占用 FLASH 空间
- RW：Read Write 的缩写，包含可读写数据（RW data，有初值，且不为 0），占用 FLASH（存储初值）和 RAM（读写操作）
- ZI：Zero initialized 的缩写，包含初始化为 0 的数据（ZI data），占用 RAM 空间
- .text：相当于 RO code
- .constdata：相当于 RO data
- .bss：相当于 ZI data
- .data：相当于 RW data

9.3.2.3 map 文件的组成部分说明

我们前面说 map 文件分为 5 个部分组成，下面以阿波罗 STM32H743 开发板 HAL 库例程的实验 1 跑马灯实验为例，简要讲解一下。

1. 程序段交叉引用关系（S S ection Cross References s ）

这部分内容描述了各个文件（.c/.s 等）之间函数（程序段）的调用关系，举个例子如图 9.3.2.3.1 所示：

```
startup_stm32h743xx.o(.text) refers to startup_stm32h743xx.o(STACK) for Stack_Mem
main.o(i.main) refers to sys.o(i.sys_cache_enable) for sys_cache_enable
main.o(i.main) refers to stm32h7xx_hal.o(i.HAL_Init) for HAL_Init
main.o(i.main) refers to sys.o(i.sys_stm32_clock_init) for sys_stm32_clock_init
main.o(i.main) refers to delay.o(i.delay_init) for delay_init
```

图 9.3.2.3.1 程序段交叉引用关系图

上图中，框出部分：main.o(i.main) refers to sys.o(i.sys_stm32_clock_init) for sys_stm32_

clock_init 表示：main.c 文件中的 main 函数，调用了 sys.c 中的 sys_stm32_clock_init 函数。其中：i.main 表示 main 函数的入口地址，同理 i. sys_stm32_clock_init 表示 sys_stm32_clock_init 函数的入口地址。

2. 删除映像未使用的程序段（Removing Unused input sections from the image）

这部分内容描述了工程中由于未被调用而被删除的冗余程序段（函数/数据），如图 9.3.2.3.2 所示：

```
==============================================================================

Removing Unused input sections from the image.

//省略部分内容
    Removing usart.o(i._ttywrch), (4 bytes).
    Removing usart.o(i.fputc), (28 bytes).
    Removing usart.o(i.usart_init), (56 bytes).
//省略部分内容

307 unused section(s) (total 35274 bytes) removed from the image.

==============================================================================
```

图 9.3.2.3.2 删除未用到的程序段

上图中，列出了所有被移除的程序段，比如 usart.c 里面的 usart_init 函数就被移除了，因为该例程没用到 usart_init 函数。

另外，在最后还有一个统计信息：307 unused section(s) (total 35274 bytes) removed from the image. 表示总共移除了 307 个程序段（函数/数据），大小为 35274 字节。即给我们的 MCU 节省了 35274 字节的程序空间。

为了更好的节省空间，我们一般在 MDK→魔术棒→C/C++选项卡里面勾选：One ELF Section per Function，如图 9.3.2.3.3 所示：

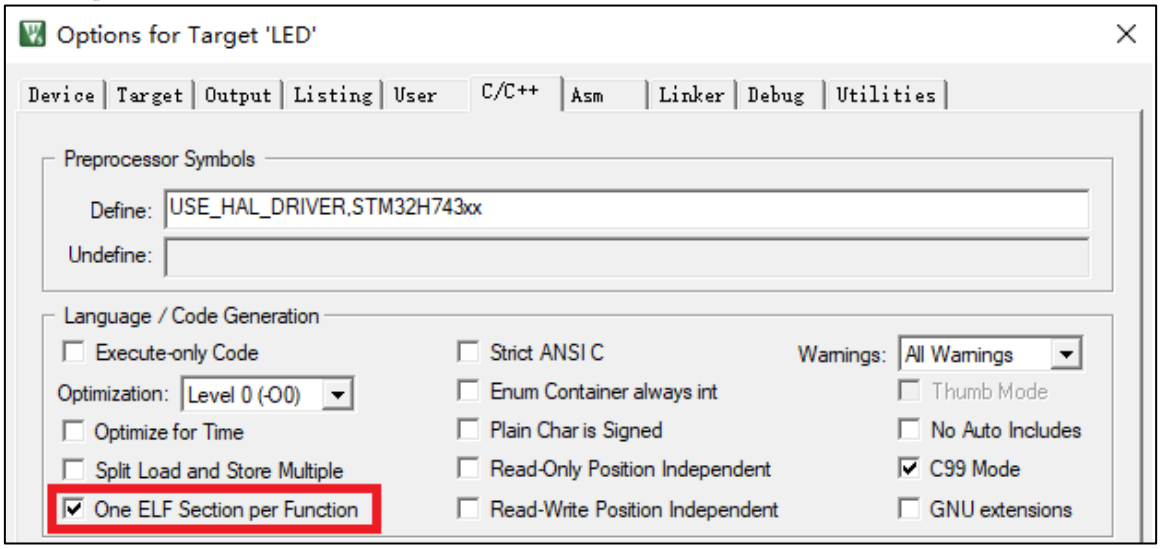


图 9.3.2.3.3 MDK 勾选 One ELF Section per Function

3. 映像符号表（Image Symbol Table）

映像符号表（Image Symbol Table）描述了被引用的各个符号（程序段/数据）在存储器中的存储地址、类型、大小等信息。映像符号表分为两类：本地符号（Local Symbols）和全局符号（Global Symbols）。

本地符号（Local Symbols）记录了用 static 声明的全局变量地址和大小，c 文件中函数的地址和用 static 声明的函数代码大小，汇编文件中的标号地址（作用域：限本文件）。

全局符号（Global Symbols）记录了全局变量的地址和大小，C 文件中函数的地址及其代码大小，汇编文件中的标号地址（作用域：全工程）。

4. 映像内存分布图（Memory Map of the image）

映像文件分为加载域（Load Region）和运行域（Execution Region），一个加载域必须有至少一个运行域（可以有多个运行域），而一个程序又可以有多个加载域。加载域为映像程序的实际存储区域，而运行域则是 MCU 上电后的运行状态。加载域和运行域的简化关系（这里仅表示一个加载域的情况）图，如图 9.3.2.3.4 所示：

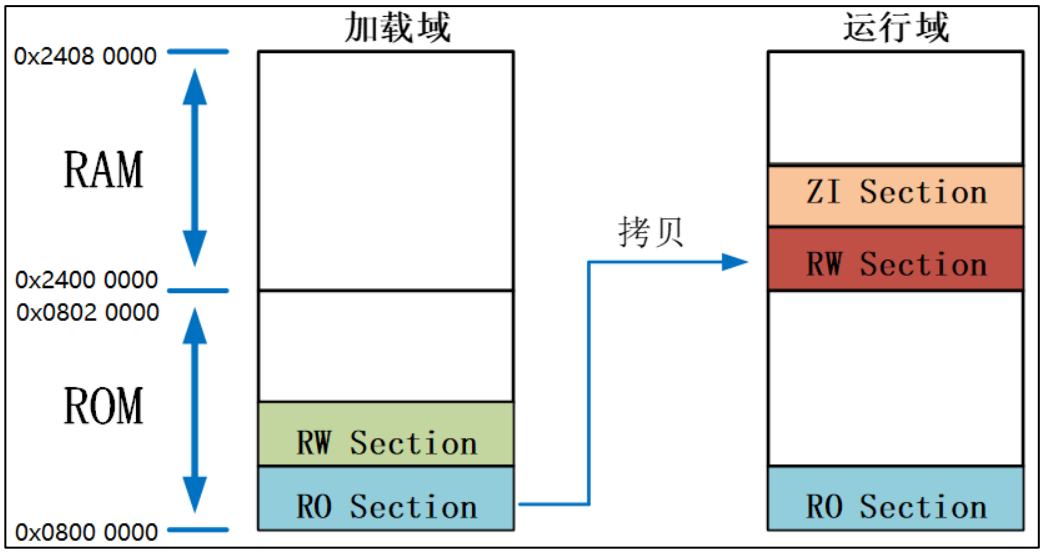


图 9.3.2.3.4 加载域运行域关系

由图可知，RW 区也是存放在 ROM（FLASH）里面的，在执行 main 函数之前，RW（有初值且不为 0 的变量）数据会被拷贝到 RAM 区，同时还会在 RAM 里面创建 ZI 区（初始化为 0 的变量）。

5. 映像组件大小（Image component sizes）

映像组件大小（Image component sizes）给出了整个映像所有代码（.o）占用空间的汇总信息。

由于篇幅较长，更多内容请大家查阅 **《STM32 MAP 文件浅析_V1.1.pdf》**文档的内容。

第十章 STM32CubeMX 简介

STM32CubeMX 是由 ST 公司开发的图形化代码自动生成工具，能够快速生成初始化代码，如 GPIO、时钟树、中间件等，使用户专注于业务代码的开发。现在 ST 主推 HAL 库，经典的标准外设库已经停止维护了，新产品也只提供 HAL 库的代码，因此，我们学习 HAL 库是更加有优势的，由于 HAL 库具有低耦合、通用、抽象了硬件层，使得开发者无需太过关注硬件驱动的实现，使得开发更加的简单快速，更容易维护，因此被越来越多的产品所使用。

本章将分为如下几个小节：

10.1 STM32CubeMX 的作用
10.2 安装 STM32CubeMX
10.3 使用 STM32CubeMX 新建工程

10.1 STM32CubeMX 的作用

STM32CubeMX 具有如下特性：

① 直观的选择 MCU 型号，可指定系列、封装、外设数量等条件
② 微控制器图形化配置
③ 自动处理引脚冲突
④ 动态设置时钟树，生成系统时钟配置代码
⑤ 可以动态设置外围和中间件模式和初始化
⑥ 功耗预测
⑦ C 代码工程生成器覆盖了 STM32 微控制器初始化编译软件，如 IAR，KEIL，GCC
⑧ 可以独立使用或者作为 Eclipse 插件使用
⑨ 可作为 ST 的固件包、芯片手册等的下载引擎

这里特别说明一下 STM32CubeMX 和 STM32Cube 固件库的关系，STM32CubeMX 图形工具配置生成的代码，是基于 STM32Cube 固件库的，并且可以在图形工具中直接下载 STM32Cube 固件库。也就是说，我们使用 STM32CubeMX 配置出来的初始化代码，兼容 STM32Cube 库，例如硬件抽象层代码就是使用的 STM32 的 HAL 库。不同系列的 STM32 芯片，会有不同系列的 STM32Cube 库，而 STM32CubeMX 图形工具只有一个，所以开发不同的 STM32 系列芯片，选择不同系列的 STM32Cube 库即可，它们的关系如下图所示：

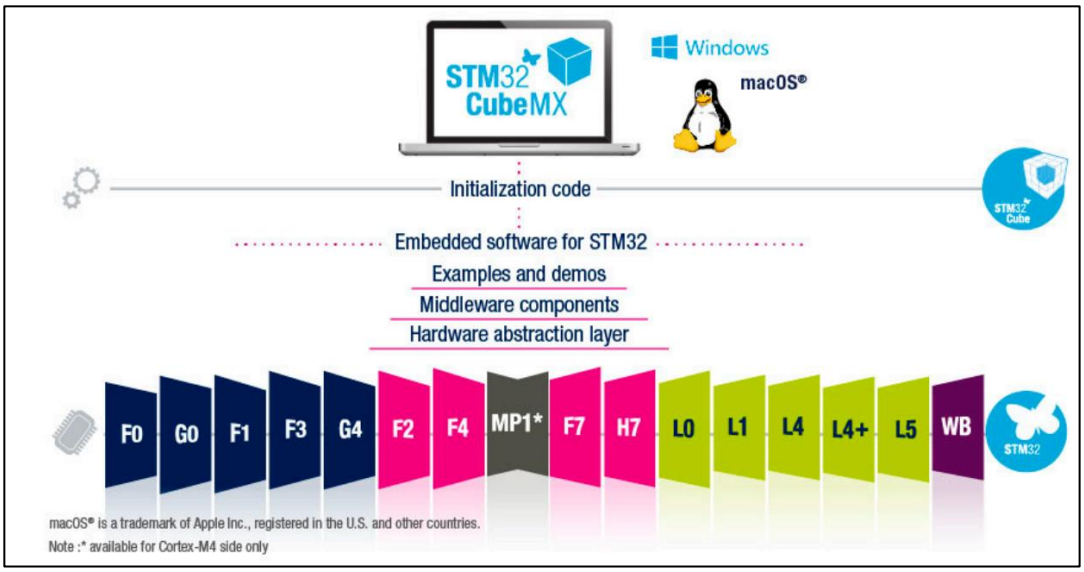


图 10.1.1 STM32CubeMX 和 STM32Cube 固件库的关系

当然，自动生成的驱动代码我们不去仔细专研其原理的话，对学习的提升很有限，而且在出现 BUG 的时候难以快速定位解决，因此我们也要了解其背后的原理。

10.2 安装 STM32CubeMX

STM32CubeMX 运行环境搭建包含两个部分，首先安装 Java 环境，再安装 STM32CubeMX。

10.2.1 安装 JAVA 环境

安装 Java 运行环境，大家可以到 Java 官网 www.java.com 下载最新的 Java 软件，也可以直接从光盘资料获取安装包，目录如下：A 盘→6，软件资料→1，软件→3、STM32CubeMX→Java 安装包，Java 安装包文件下有 x64 和 x86 两个文件夹，分别是 64 位和 32 位的电脑的安装包，大家根据自己电脑的位数选择即可。比如 64 位电脑选择 x64 文件夹的 jre-8u301-windows-x64.exe 安装包，并根据提示安装即可。安装完成之后提示界面如下图 10.2.1.1 所示。

图 10.2.1.1 Java 安装成功提示界面

安装完 Java 运行环境之后，为了检测是否正常安装，我们可以打开 Windows 的命令输入框，输入：java –version 命令，如果显示 Java 版本信息，则安装成功。提示信息如下图 10.2.1.2：

图 10.2.1.2 查看 Java 版本

10.2.2 安装 STM32CubeMX

在安装了 Java 运行环境之后，接下来我们安装 STM32CubeMX 图形化工具。该软件可以直接从光盘资料获取，目录如下：A 盘→6，软件资料→1，软件→3、STM32CubeMX，也可以直接从 ST 官方下载，下载网址为：https://www.st.com/en/development-tools/stm32cubemx.html。

接下来我们直接双击 SetupSTM32CubeMX-6.3.0.exe，安装步骤如下。

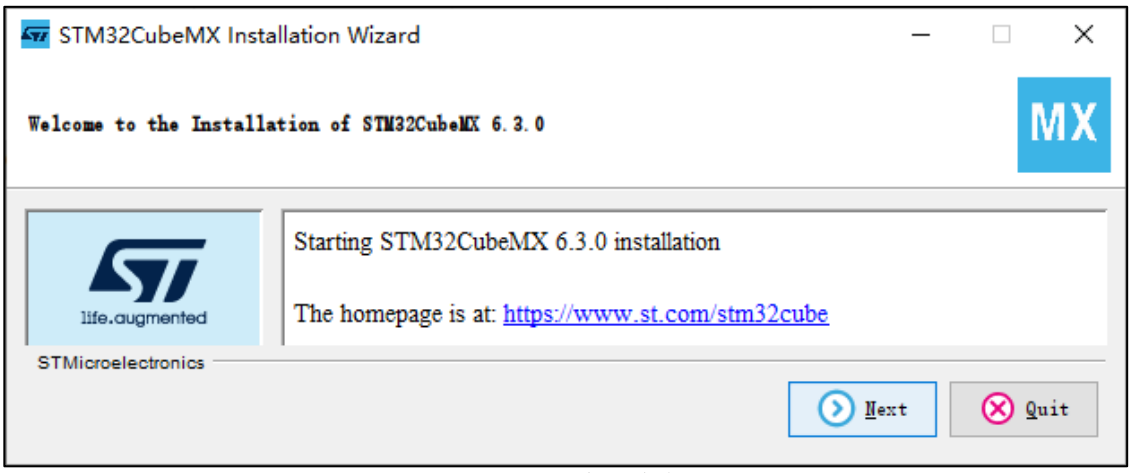


图 10.2.2.1 启动安装

图 10.2.2.2 接受本许可协议

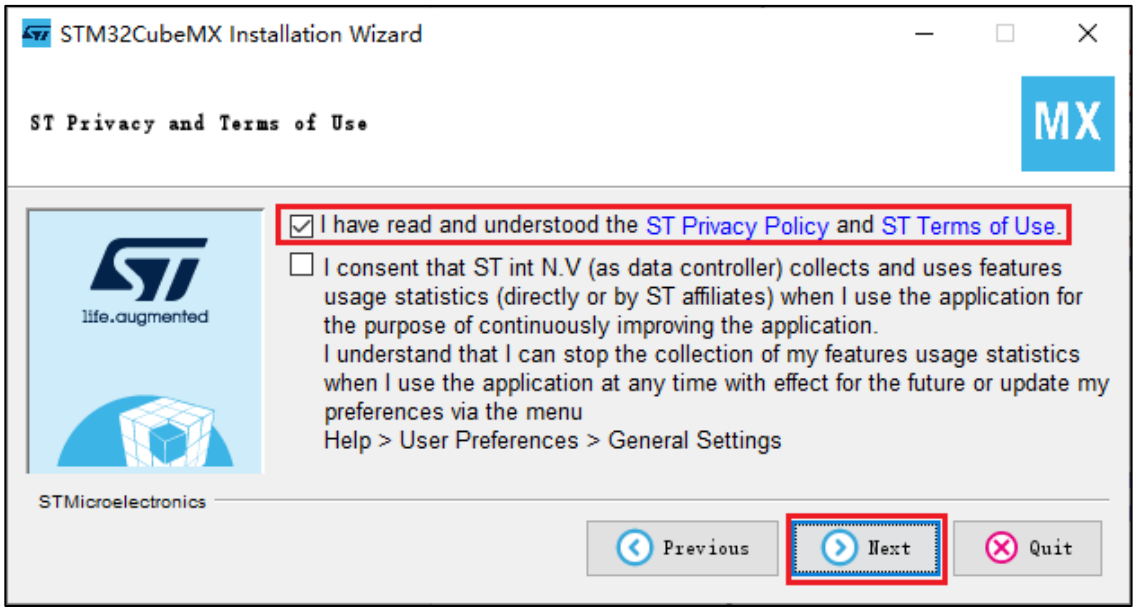


图 10.2.2.3 勾选第一项即可

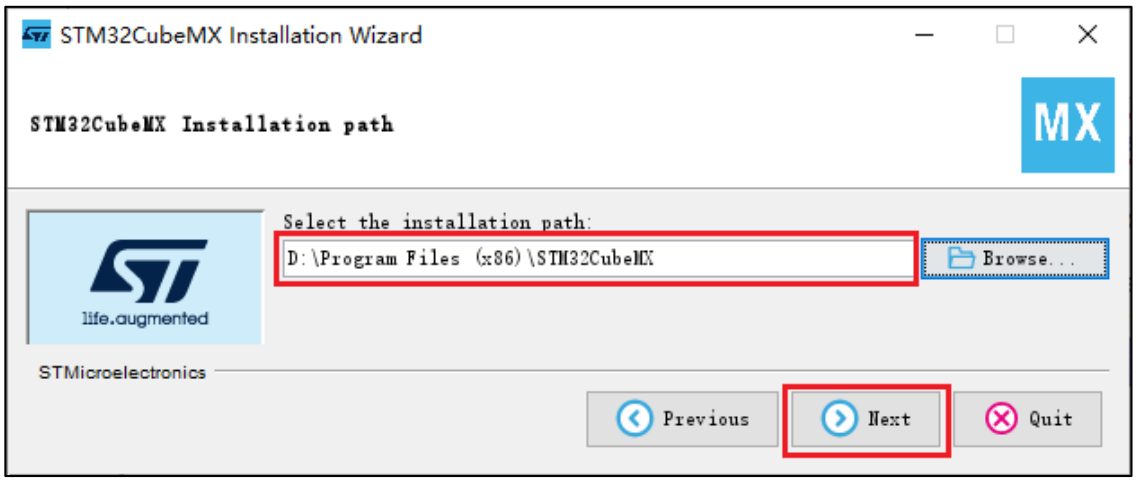


图 10.2.2.4 指定安装路径

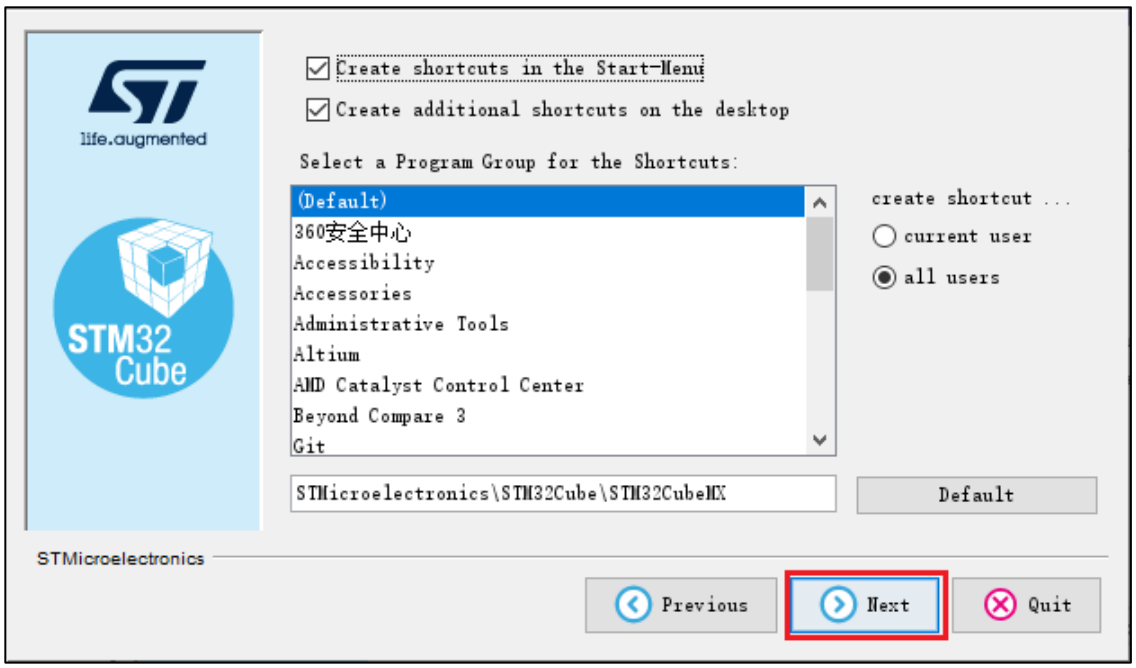


图 10.2.2.5 创建快捷方式

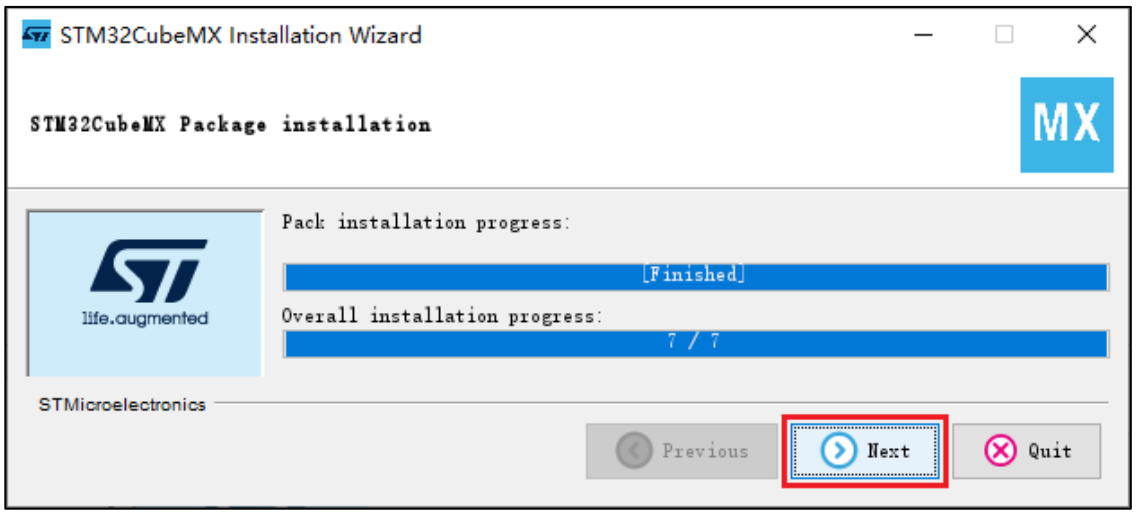


图 10.2.2.6 安装进度提示

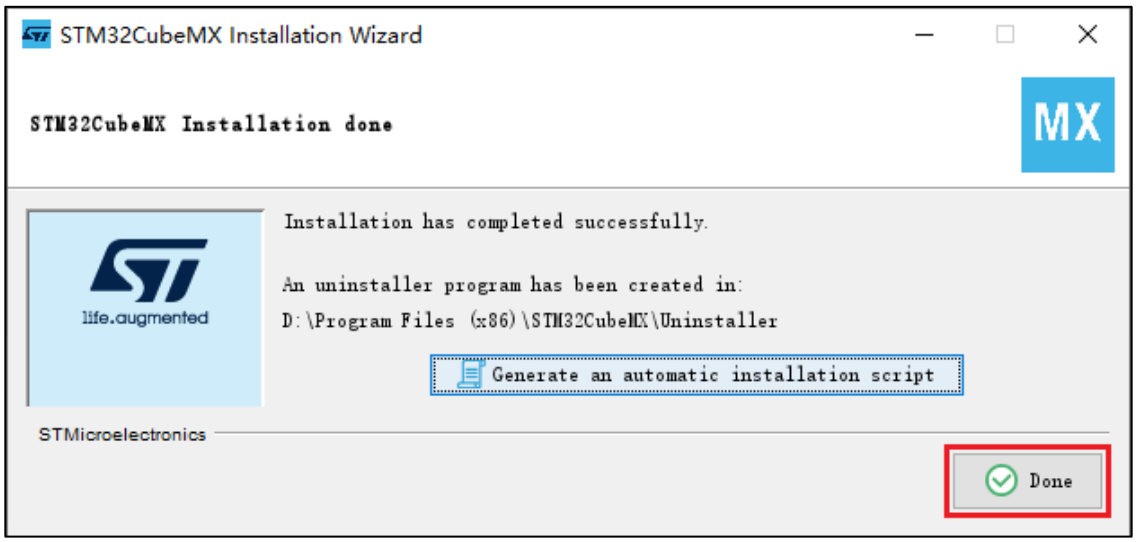


图 10.2.2.7 完成安装

10.3 使用 STM32CubeMX 新建工程

大多数情况下，我们都只使用 STM32CubeMX 来生成工程的时钟系统初始化代码以及外设的初始化代码，因为用户控制逻辑代码是无法在 STM32CubeMX 中完成的，需要用户自己根据需求来实现。

10.3.1 打开 STM32CubeMX

双击桌面 STM32CubeMX 快捷方式图标，如图 10.3.1.1 所示。

图 10.3.1.1 CubeMX 快捷方式

打开后 CubeMX 主界面如图 10.3.1.2 所示。

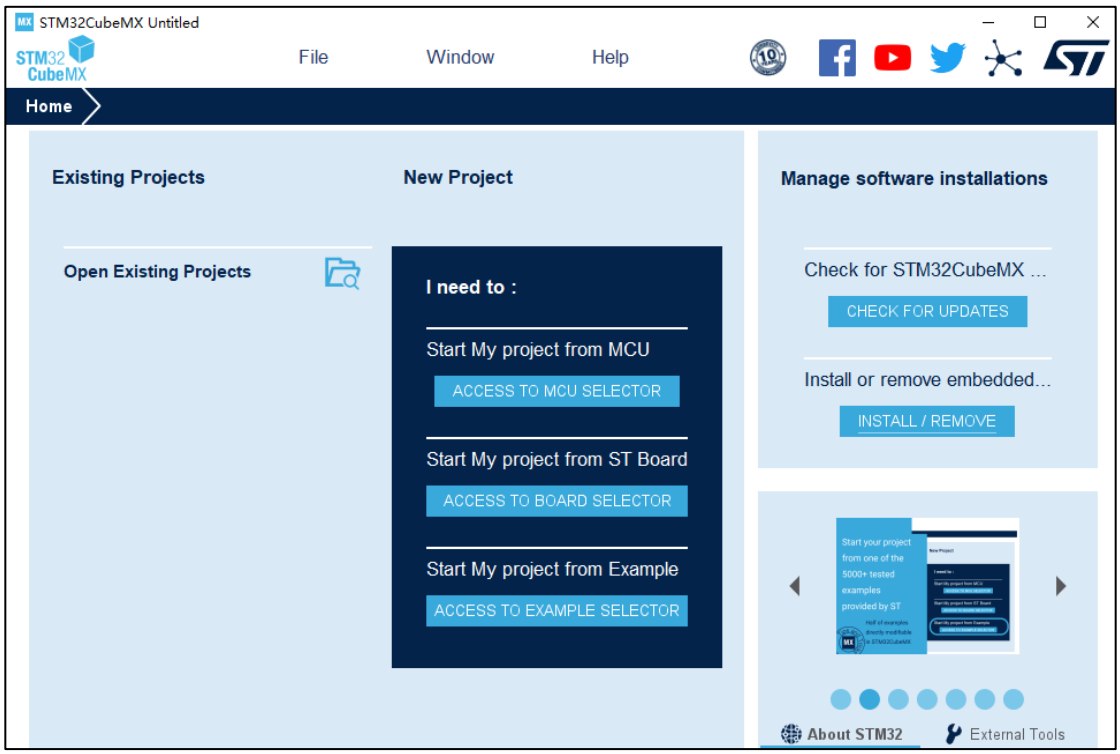


图 10.3.1.2 CubeMX 主界面

10.3.2 下载和关联的 STM32Cube 固件包

我们知道 STM32CubeMX 图形工具只有一种， STM32Cube 固件包却有多种，需要选择我们工程对应的固件包。

为了方便，新建工程前，我们先来下载和关联 STM32Cube 固件包，点击 Help->Manage embedded software packages，如图 10.3.2.1 所示。

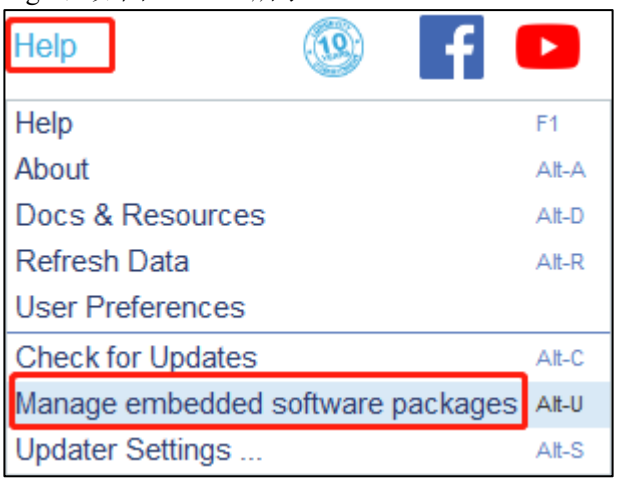


图 10.3.2.1 管理固件包

然后弹出管理界面，在该窗口找到 STM32H7 列表选项，勾选 1.9.0 版本。这里选择 1.9.0 版本是因为我们的光盘的固件包是这个版本的。关联 STM32Cube 固件包有两个方法，如图 10.3.2.2 所示.

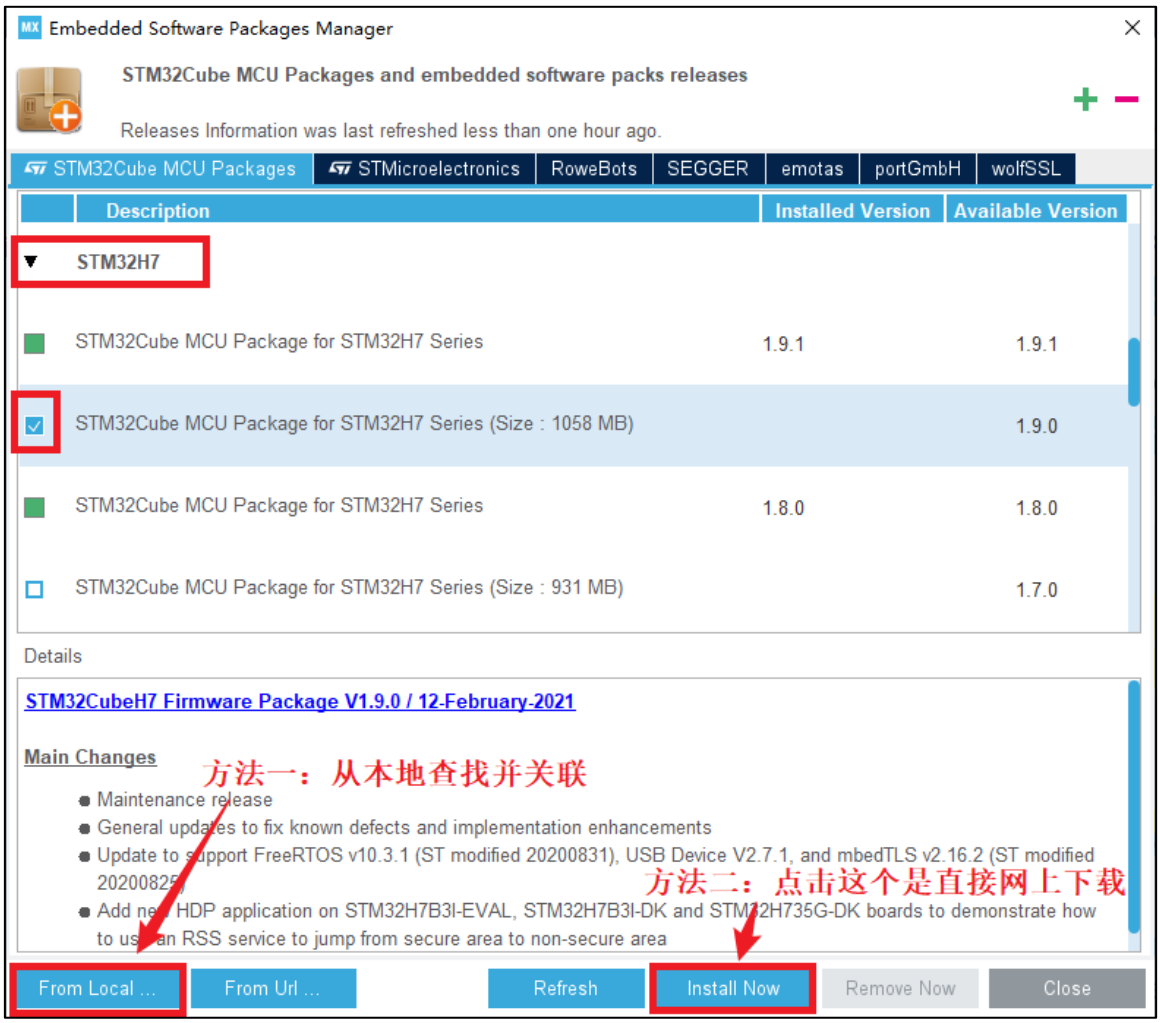


图 10.3.2.2 下载和关联 STM32Cube 固件包

方法一：点击后，弹出下面的窗口，然后选择光盘中的对应的固件包，注意这里是压缩包的形式，如图 10.3.2.3 所示。

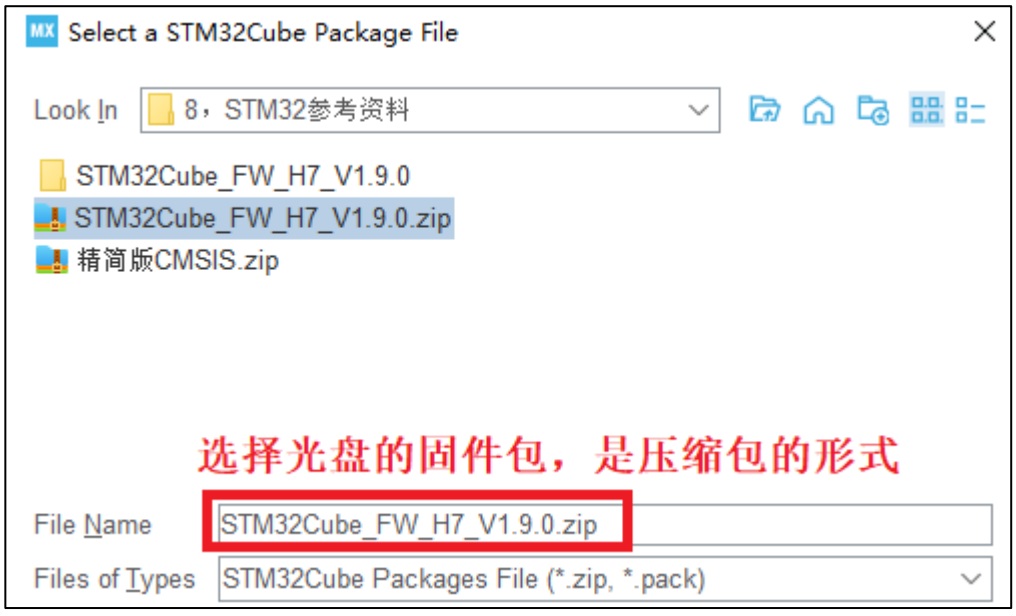


图 10.3.2.3 关联本地 STM32Cube 固件包

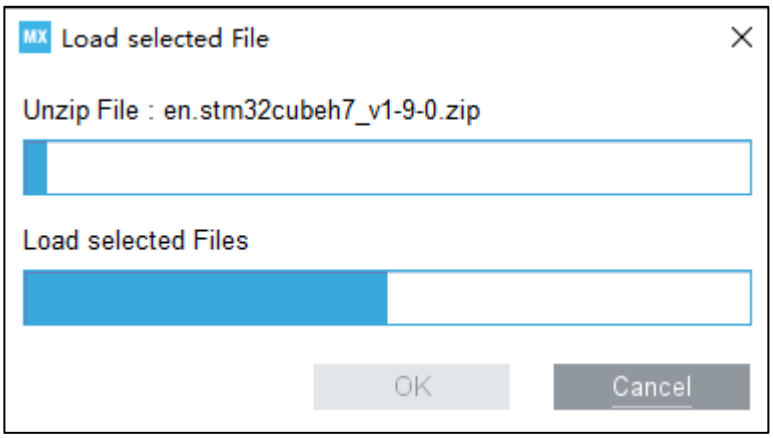


图 10.3.2.4 等待关联

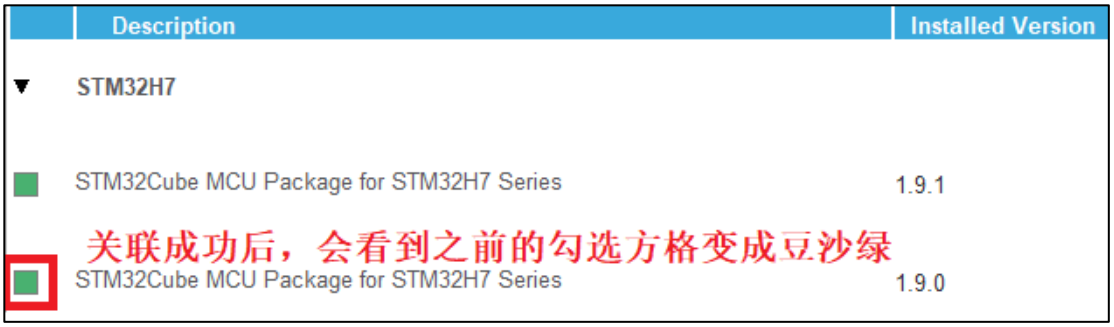


图 10.3.2.5 关联成功

关联好固件包我们就可以开始新建工程了。

方法二：下载好之后，会自动关联，所以不需要多讲什么。

10.3.3 新建工程

使用 STM32CubeMX 配置工程的一般步骤为：

1，工程初步建立

2，HSE 和 LSE 时钟源设置

3，时钟系统（时钟树）配置

4，GPIO 功能引脚配置

5，Cortex-M7 内核基本配置（限定项）

6，生成工程源码

7，用户程序

接下来将按照这 7 个步骤，依次教大家使用 STM32CubeMX 工具生成一个完整的工程。

1 工程初步建立

方法一：依次点击“File”，“New Project”即可建新工程。如果之前打开过的话，左侧最近打开的过程一列会有打开的工程列表，直接点击这些工程也可以打开。

方法二：直接点击 ACCESS TO MCU SELECTOR。

具体操作如图 10.3.3.1 所示。

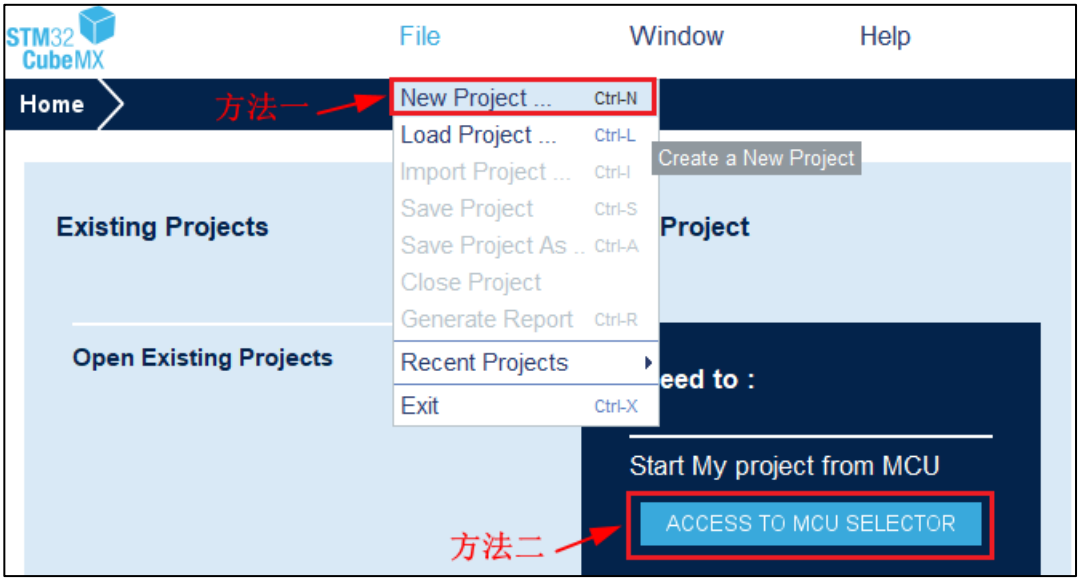


图 10.3.3.1 新建工程

点击新建工程后，第一次可能会联网下载一些的文件，可能等待时间比较长，可以直接选择取消即可。

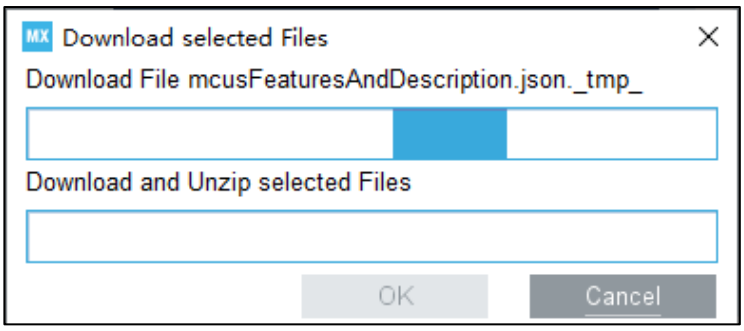


图 10.3.3.2 启动时联网更新检测

之后都可以进入芯片选型界面，如图 10.3.3.3 所示。

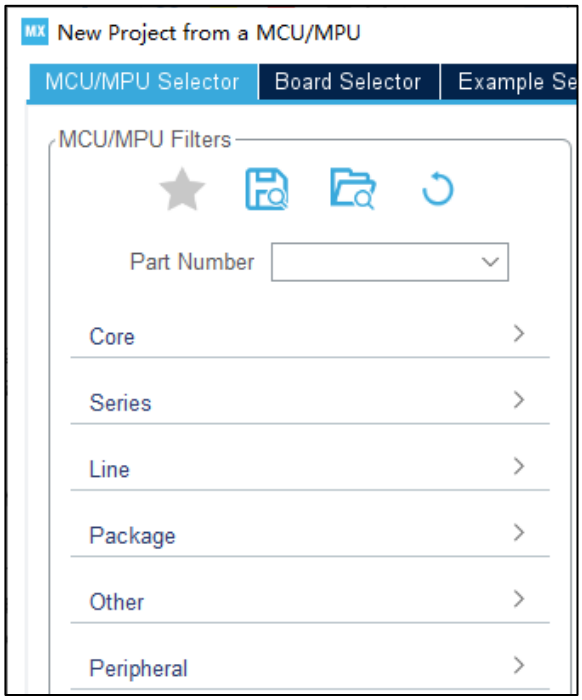


图 10.3.3.3 芯片选型界面

选择具体的芯片型号，如图 10.3.3.4 所示。

图 10.3.3.4 选择具体的芯片型号

选择了芯片型号后，弹出主设计界面，如图 10.3.3.5 所示。

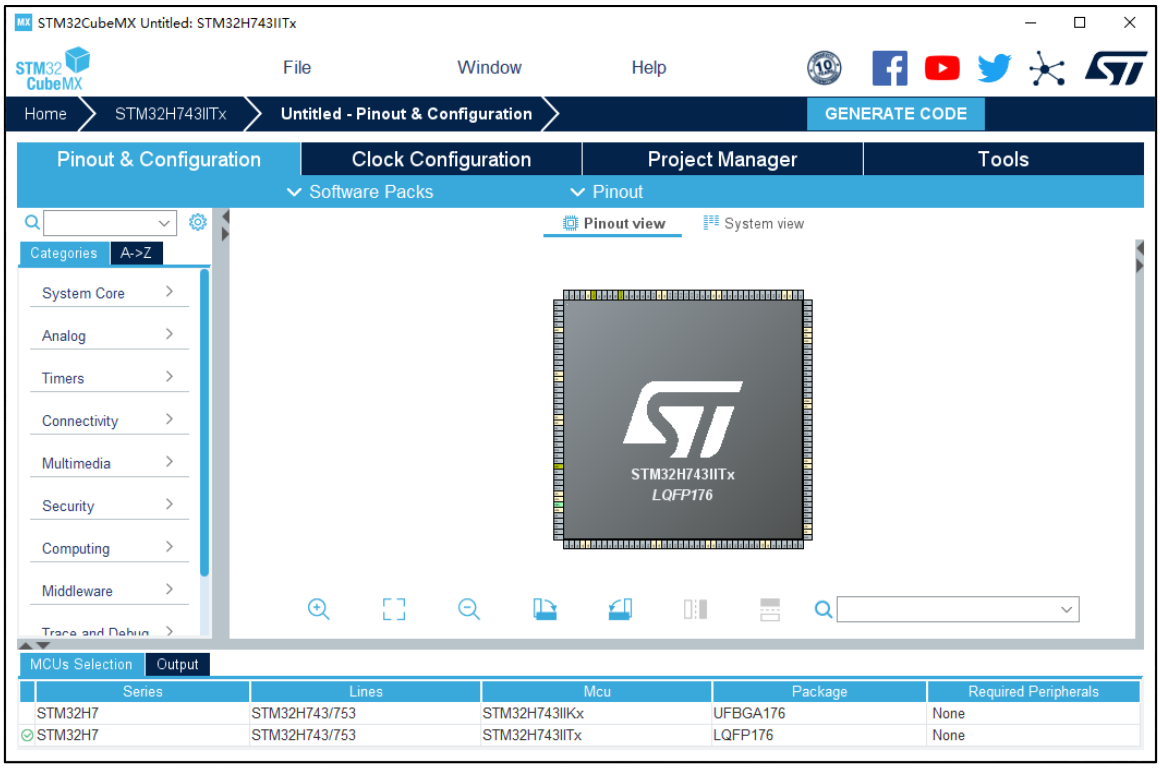


图 10.3.3.5 主设计界面

2 HSE 和 LSE 时钟源设置

进入工程主设计界面后，首先设置时钟源 HSE 和 LSE。如图 10.3.3.6 所示。

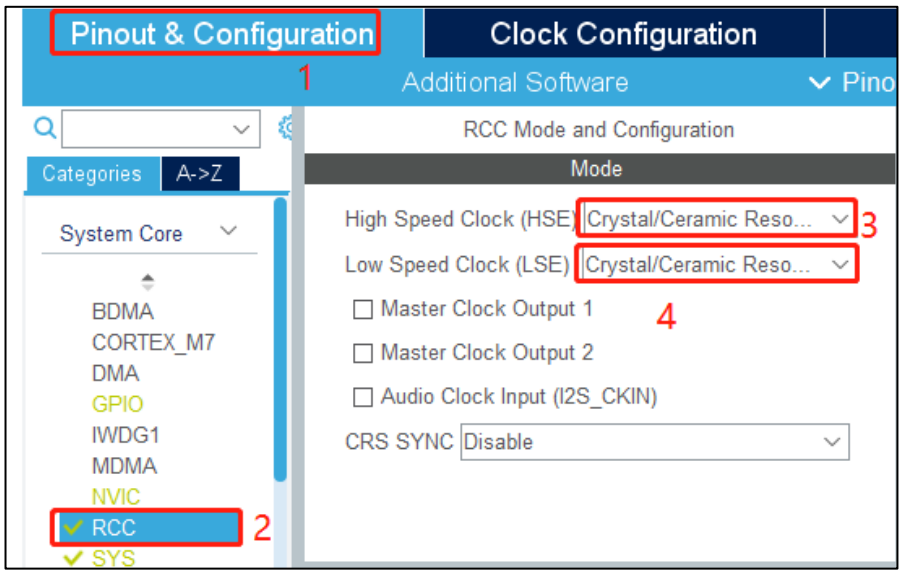


图 10.3.3.6 设置时钟源 HSE 和 LSE

图 10.3.3.6 中的标号 3 和 4，我们都选择了 Crystal/Ceramic Resonator，表示外部晶振作为它们的时钟源。我们开发板的外部高速晶振和外部低速晶振分别是：25MHZ 和 32.768KHZ。所以 HSE 时钟频率就是 25MHZ，LSE 时钟频率就是 32.768KHZ。

选项 Master Clock Output 1 用来选择是否使能 MCO1 引脚时钟输出，选项 Master Clock Output 2 用来选择是否使能 MCO2 引脚时钟输出，最后一个选项 Audio Clock Input（I2S_CKIN）用来选择是否从 I2S_CKIN(PC9)输入 I2S 时钟。这里大家要注意，因为选项 Master Clock Output 2 和选项 Audio Clock Input（I2S_CKIN）都是使用的 PC9 引脚，所以如果我们使能了其中一个，那么另一个选项会自动显示为红色，也就是不允许配置，这就是 STM32CubeMX 的自动冲突检测功能。

3 时钟系统（时钟树）配置

点击 Clock Configuration 选项卡即可进入时钟系统配置栏，如下图 10.3.3.7 所示：

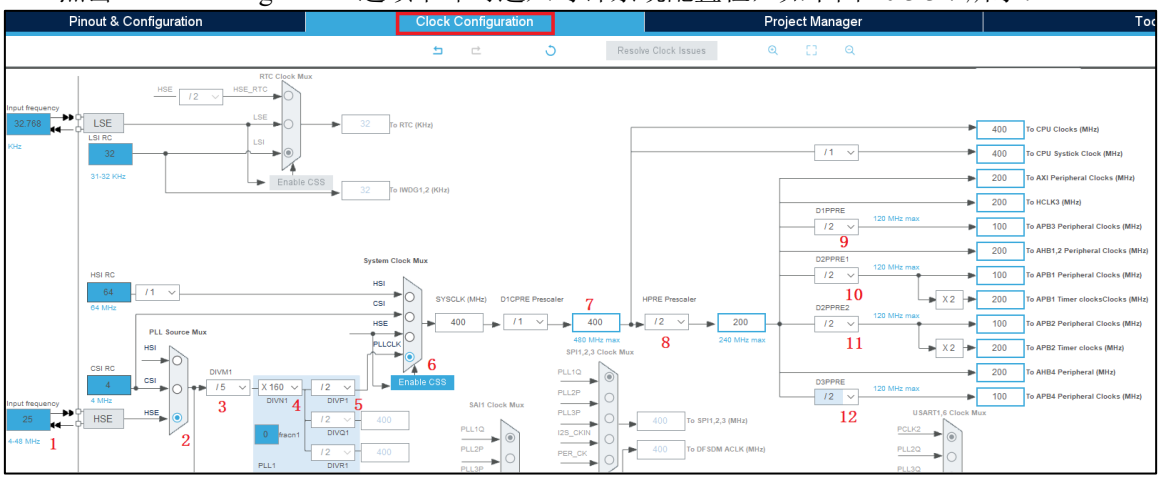


图 10.3.3.7 时钟系统配置栏

进入 Clock Configuration 配置栏之后可以看到，界面展现一个完整的 STM32H7 时钟系统框图。从这个时钟树配置图可以看出，配置的主要是外部晶振大小，分频系数，倍频系数以及选择器。在我们配置的工程中，时钟值会动态更新，如果某个时钟值在配置过程中超过允许值，那么相应的选项框会红色提示。

这里，我们将配置一个以 HSE 为时钟源，配置 PLL1 相关参数，然后系统时钟选择 PLLCLK 为时钟源，最终配置系统时钟为 400MHz 的过程。同时，还配置了 AHB，APB1，APB2、APB3、APB4 和 Systick 的相关分频系数。由于图片比较大，我们把主要的配置部分分

两部分来讲解，第一部分是配置系统时钟，第二部分是配置 SYSTICK、AHB、APB1、APB2、APB3 和 APB4 的分频系数。首先我们来看看第一部分配置如下图 10.3.3.8 所示：

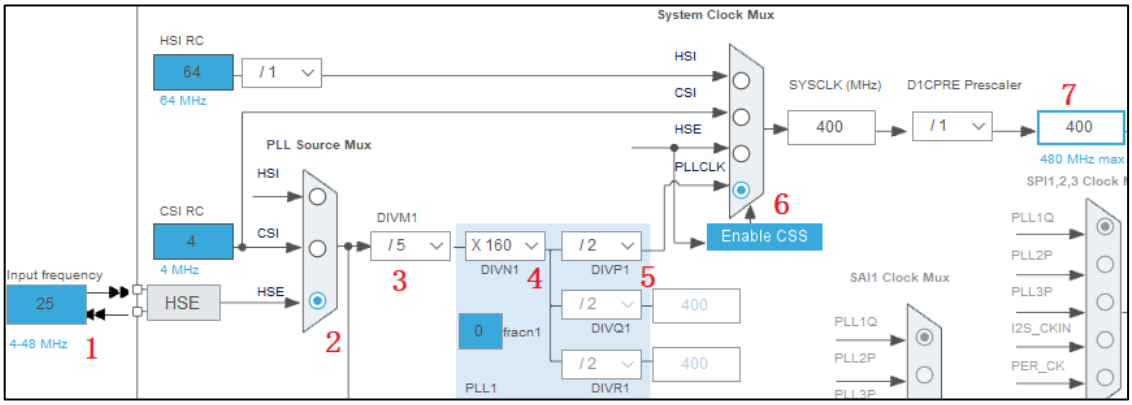


图 10.3.3.8 系统时钟配置图

我们把系统时钟配置分为七个步骤，分别用标号 1~7 表示，详细过程为：

1、时钟源参数设置：我们选择 HSE 为时钟源，所以我们要根据硬件实际的高速晶振频率（这里我们是 25MHZ）填写。

2、时钟源选择：我们配置选择器选择 HSE 即可。

3、PLL1 分频系数 M 配置。分频系数 M 我们设置为 5。

4、PLL1 倍频系数 N 配置。倍频系数 N 我们设置为 160。

5、PLL1 分频系数 P 配置。分频系数 P 我们配置为 2。

6、系统时钟时钟源选择：PLL,HSI 还是 HSE。我们选择 PLL，选择器选择 PLLCLK 即可。

7、经过上面配置以后此时 SYSCLK=400Mhz。

经过上面的 7 个步骤，就会生成标准的 400MHz 系统时钟。接下来我们只需要配置 AHB、APB1、APB2、APB3、APB4 和 Systick 的分频系数，就可以实现 sys.c 文件中的 sys_stm32_clock_init 函数配置的部分时钟系统。配置如下图 10.3.3.9 所示：

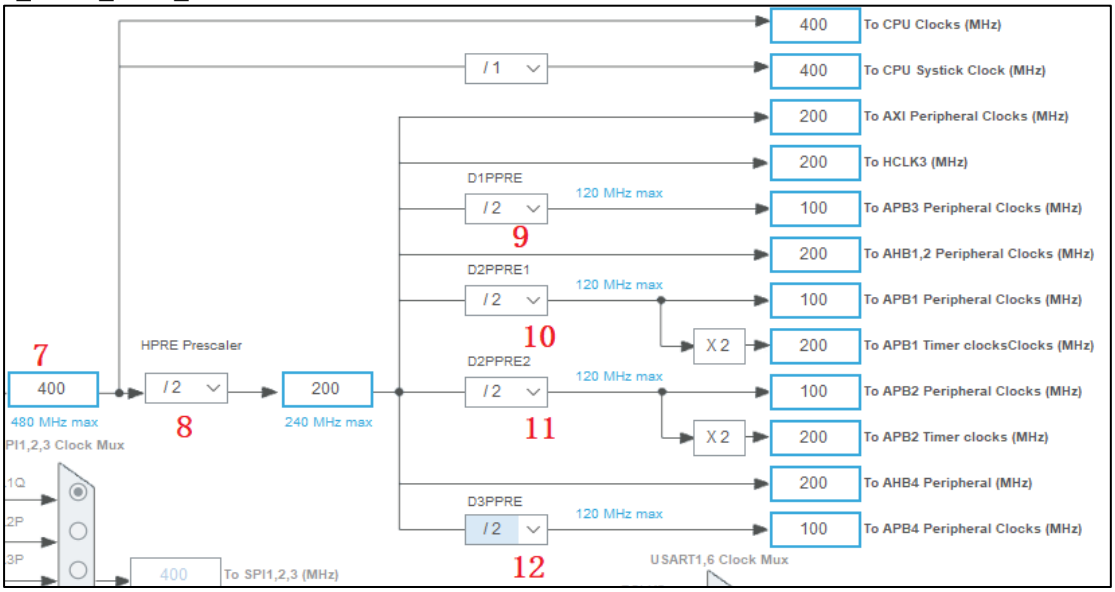


图 10.3.3.9 AHB、APB1、APB2、APB3 和 APB4 总线时钟配置

AHB、APB1、APB2、APB3 和 APB4 总线时钟以及 Systick 时钟的最终来源都是系统时钟 SYSCLK。其中 AHB 总线时钟 HCLK 是由 SYSCLK 经过 AHB 预分频器之后的来，如果我们要设置 HCLK 为 200MHz，那么我们只需要配置图中标号 8 的地方为 2 即可。得到 HCLK 之后，接下来我们将在图标号 9~12 处同样的方法依次配置 APB3、APB1、APB2 和 APB4 分频系数分别为 2，2、2 和 2 即可。注意！systick 固定为 400MHz，配置完成之后，那么 HCLK=200MHZ，Systic=400MHz，PCLK1=100MHz，PCLK2=100MHz，PCLK3=100Mhz，

PCLK4=100MHz，这和我们使用 sys_stm32_clock_init 函数配置的时钟是一样的。

4 GPIO 功能引脚配置

本小节，我们讲解怎么使用 STM32CubeMX 工具配置 STM32H7 的 GPIO 口。阿波罗 STM32H743 开发板的 PB0 和 PB1 引脚连接 LED 灯，我们来学习配置这两个 IO 口的相关参数。这里我们回到 STM32CubeMX 的 Pinout&Configuration 选项，在搜索栏输入 PB0 后回车，可以在引脚图中显示位置，如下图 10.3.3.10 所示：

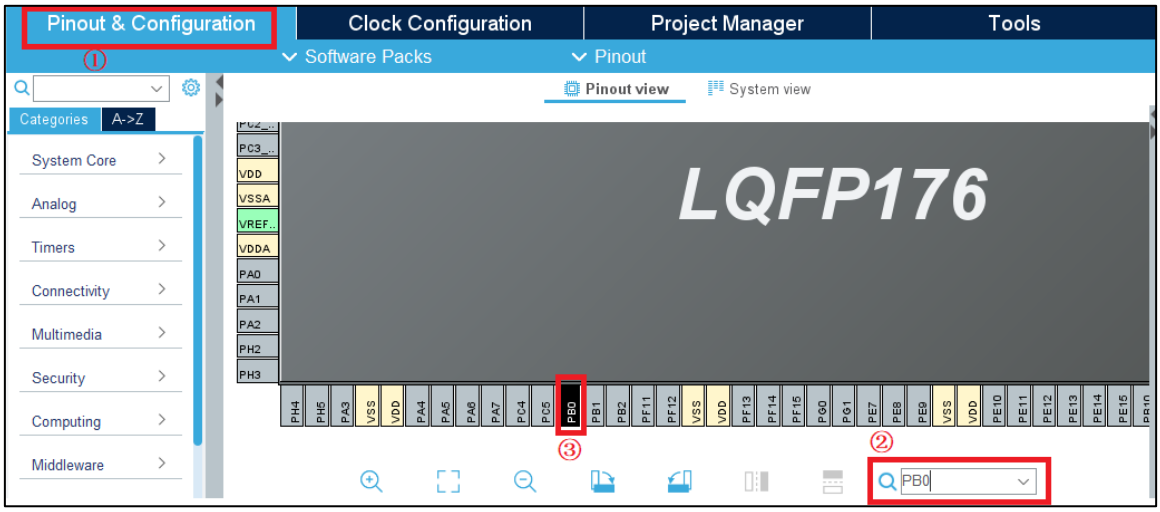


图 10.3.3.10 搜索引脚位置

接下来，我们在图 10.3.3.11 引脚图中点击 PB0，在弹出的下拉菜单中，选择 IO 口的功能为 GPIO_Output。操作方法如下图 10.3.3.11 所示：

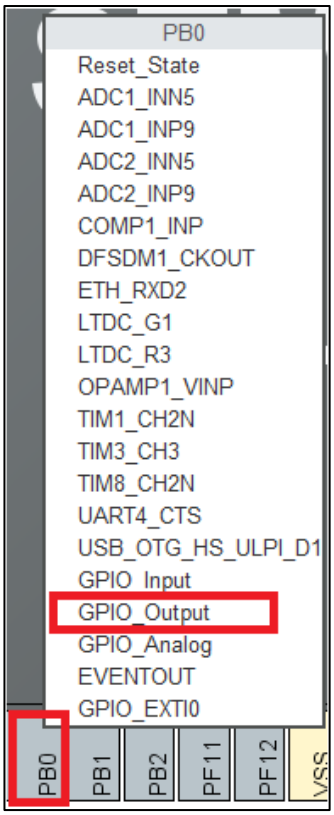


图 10.3.3.11 配置 GPIO 模式

同样的方法，我们配置 PB1 选择功能为 GPIO_Oput 即可。设置好即可看到引脚从灰色变成绿色，标识该管脚已经启用。这里我们需要说明一下，如果我们要配置 IO 口为外部中断引

脚或者其他复用功能，我们选择相应的选项即可。配置完 IO 口功能之后，还要配置 IO 口的速度，上下拉等参数。这些参数我们通过 System Core 下的 GPIO 选项进行配置，如图 10.3.3.12 所示。

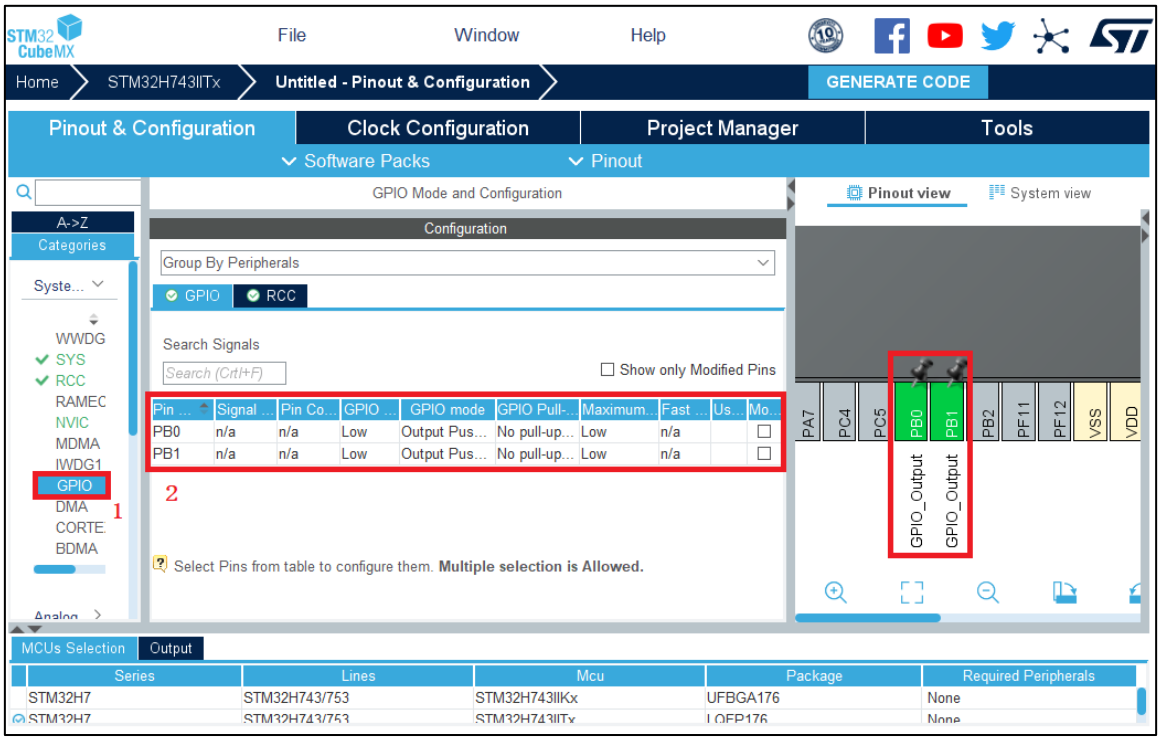


图 10.3.3.12 GPIO 选项

我们先配置 PB0，PB1 配置方法一样的。点击图 10.3.3.12 的 2 号框里面的 PB0，配置如图 10.3.3.13 所示。

图 10.3.3.13 配置 GPIO 口详细参数

GPIO output level 是 IO 的初始值，为了开始让 LED 灯熄灭，我们设置初始值输出高电平。

GPIO mode 默认是推挽输出，不需要更改。

GPIO Pull-up/Pull-down 默认是不上下拉，我们改为上拉。

Maximum output speed 输出速度配置，默认是低速，我们设置为中速（可以不改）。

User Label 用户符号，我们可以给 PB0 起一个别的名字。

PB1 也是按照这样的方法配置即可。

5 Cortex-M7 内核基本配置

这里我们主要配置 Cortex-M7 内核相关的参数。我们依次点击 Cortex_M7 进入配置界面，操作过程如下图 10.3.3.14 所示：

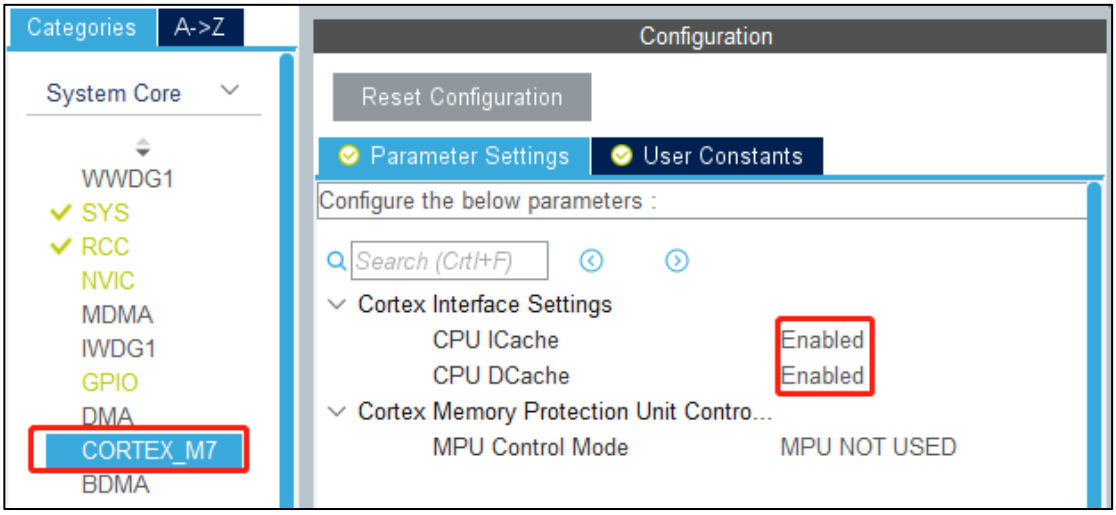


图 10.3.3.14 Cotex_M7 配置

该界面一共有两个配置栏目。第一个配置栏目 Cortex Interface Settings 下面有两个配置项：

1) CPU ICache：使能 I-Cache。

2) CPU DCache::：使能 D-Cache。

上面这 2 个参数是 CM7 内核相关配置。第二个配置栏目 Cortex Memory Protection Unit，是用来配置内存保护单元 MPU，在我们后面的实验会讲解 MPU 配置。

6 生成工程源码

接下来我们学习怎么设置生成一个工程，Project Manager-> Project 选项用来配置工程的选项，我们了解一下里面的信息。

Project Name：工程名称，填入工程名称（半角，不能有中文字符）

Project Location：工程保存路径，点击 Browse 选择保存的位置（半角，不能有中文字符）

Toolchain Folder Location:工具链文件夹位置，默认即可。

Application Structure：应用的结构，选择 Basic（基础），不勾选 Do not generate the main(), 因为我们要其生成 main 函数。

Toolchain/IDE:工具链/集成开发环境，我们使用 Keil，因此选择 MDK-ARM，Min Version 选择 V5.32（最新）。

Linker Settings 链接器设置：

Minimum Heap Size 最小堆大小，默认（大工程需按需调整）。

Minimum Stack Size 最小栈大小，默认（大工程需按需调整）。

MCU and Firmware Package 是 MCU 及固件包设置：

MCU Reference：目标 MCU 系列名称。

Firmware Package Name and Version ：固件包名称及版本。

勾选 Use Default Firmware Location，文本框里面的路径就是固件包的存储地址，我们使用默认地址即可。（这里因为我有两个版本的固件包，所以它默认使用最新的，这个关系不大，就用新的）。 最后工程配置，如图 10.3.3.15 所示。

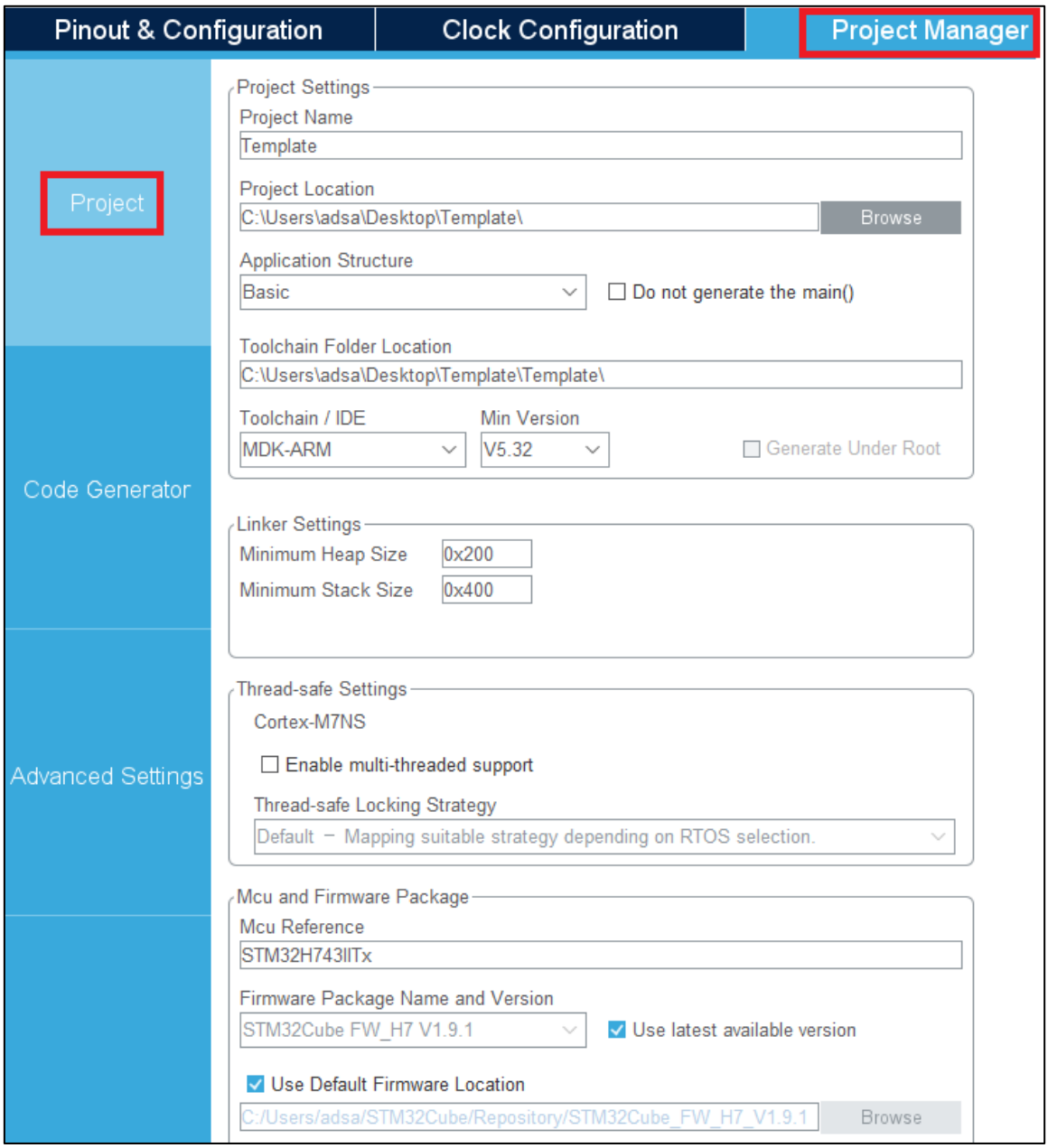


图 10.3.3.15 工程配置

打开 Project Manager-> Code Generator 选项，Generated files 生成文件选项，勾选 Generate peripheral initialization as a pair of '.c/.h' files per peripheral，勾选这个选项的话将会将每个外设单独分开成一组.c、.h 文件，使得代码结构更加的清晰，如图 10.3.3.16 所示。

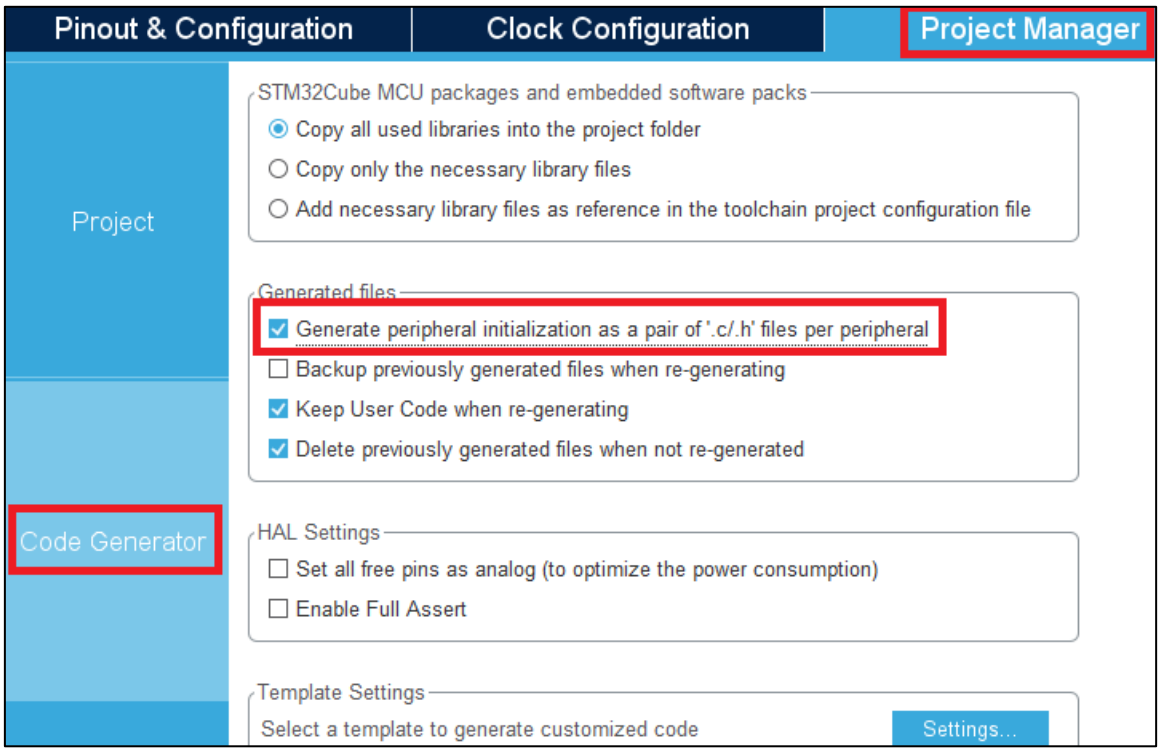


图 10.3.3.16 代码生成器设置

至此工程最基础配置就已经完成，点击蓝色按钮(SENERATE CODE)就可以生成工程。

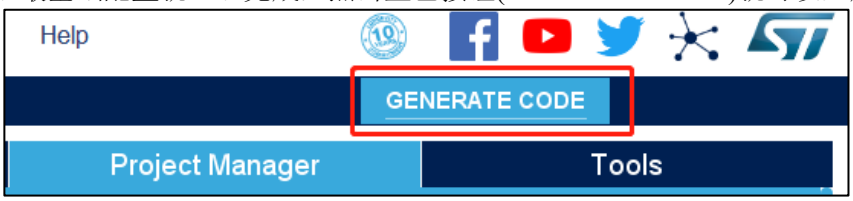


图 10.3.3.17 生成工程

在弹出来的窗口中点击 Open Project 就打开 MDK 工程。

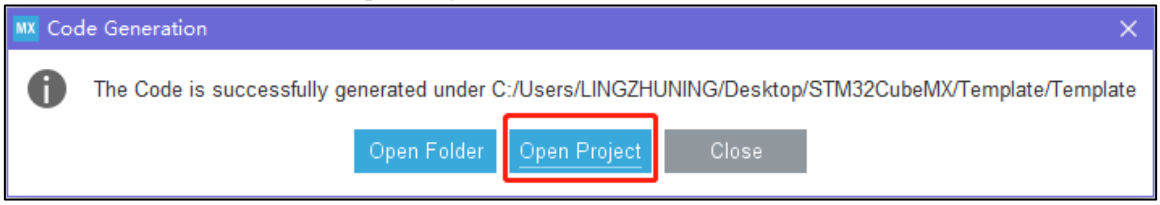


图 10.3.3.18 打开工程

完整的 STM32H7 工程就已经生成完成。生成后的工程目录结构如下图 10.3.3.19 所示：

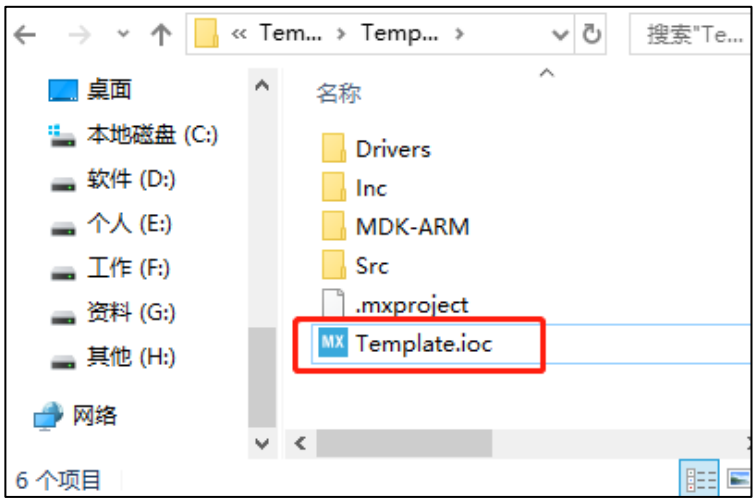


图 10.3.3.19 STM32CubeMX 生成的工程目录结构

Drivers 文件夹存放的是 HAL 库文件和 CMSIS 相关文件。

Inc 文件夹存放的是工程必须的部分头文件。

MDK-ARM 下面存放的是 MDK 工程文件。

Src 文件夹下面存放的是工程必须的部分源文件。

Template.ioc 是 STM32CubeMX 工程文件，双击该文件就会在 STM32CubeMX 中打开。

7 用户程序

在编写用户程序之前，首先我们打开生成的工程模板进行编译，发现没有任何错误和警告。

接下来我们看看生成的工程模板的 main 函数，这里我们删掉了源码注释，关键源码如下：

```
int main(void)
{
  SCB_EnableICache();
  SCB_EnableDCache();
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  while (1)
  {
  }
}
```

大家需要注意，STM32CubeMX 生成的 main.c 文件中，有很多地方有“/* USER CODE BEGIN X */”和“/* USER CODE END X */”格式的注释，我们在这些注释的 BEGIN 和 END 之间编写代码，那么重新生成工程之后，这些代码会保留而不会被覆盖。

我们编写一个跑马灯的用户程序，程序具体如下：

```
/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
  SCB_EnableICache();
  SCB_EnableDCache();
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();
  while (1)
  {
    /* USER CODE BEGIN 3 */
    HAL_GPIO_WritePin(GPIOB,GPIO_PIN_1,GPIO_PIN_SET);          /* PB1 置 1 */
    HAL_GPIO_WritePin(GPIOB,GPIO_PIN_0,GPIO_PIN_RESET);        /* PB0 置 0 */
    HAL_Delay(500);
    HAL_GPIO_WritePin(GPIOB,GPIO_PIN_1,GPIO_PIN_RESET);        /* PB1 置 0 */
```

```
    HAL_GPIO_WritePin(GPIOB,GPIO_PIN_0,GPIO_PIN_SET);          /* PB0 置 1 */
    HAL_Delay(500);
  }
  /* USER CODE END 3 */
}
```

编写好程序后，编译没有任何警告和错误。可以直接下载程序阿波罗 STM32H743 开发板中（使用 DAP 下载，请注意配置 MDK）。

本小节使用 STM32CubeMX 新建的工程模板在我们光盘目录：**“4，程序源码\标准例程-库函数版本\实验 0-4 Template 工程模板-使用 STM32CubeMX 配置 ”**中有存放，大家在编写用户代码过程中可以参考该工程的 main.c 文件。

10.4 STM32CubeMX 新建工程使用建议

① 使用 CubeMX 的环境搭建工程，工程文件夹路径、文件名不要带任何中文及中文字符，否则会遇到各种报错；

② 本书以新建工程-HAL 库版本为基准来展开，不对 CubeMX 的使用过多讲解。使用 CubeMX 可以帮助我们快速搭建工程，使用户专注于应用开发，但 STM32 的开发与硬件密切相关，对 STM32 开发来说，抛开底层只专注做应用并不实际，毕竟无法使用一套通用设计来满足不同用户的需求；

③ 关于新建 CubeMX 的工程路径中有中文的情况的解决：

如果我们配置的 CubeMX 工程路径里面有中文可能会报以下的错误：

```
Build Output
linking...
LED_TEST\LED_TEST.sct(7): error: L6236E: No section matches selector - no section to be FIRST/LAST.
Not enough information to list image symbols.
Not enough information to list load addresses in the image map.
Finished: 2 information, 0 warning and 1 error messages.
"LED_TEST\LED_TEST.axf" - 1 Error(s), 0 Warning(s).
Target not created.
```

图 10.4.1 直接编译报错

造成错误的原因是 CubeMX 对中文的支持不友好，且生成的 MDK 工程默认通过工程中的 CMSIS 那个绿色的控件选择启动文件而不是直接添加启动文件(startup_xxx.s)到我们的工程中，而有中文路径时就会找不到，有两个解决办法：

1、用 CubeMX 生成的工程不要放置在包含中文路径的文件夹下；

2、添加启动文件到我们的工程中，我们新建一个 Application/MDK-ARM 分组，把 startup_stm32h743xx.s 添加到这个分组，如图 10.4.2 所示：

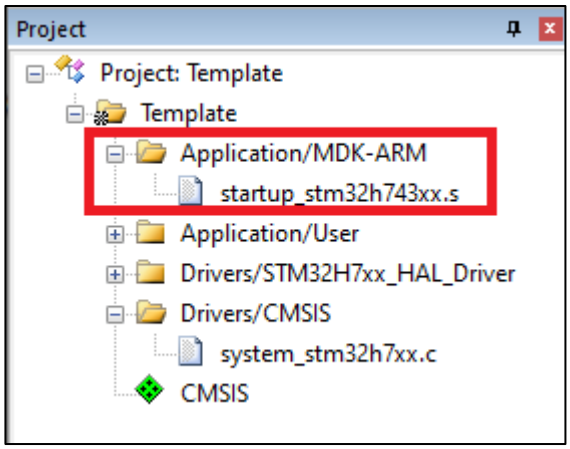


图 10.4.2 STM32CubeMX 生成的工程目录结构

④ 关于配置的文件 CubeMX 工程(.ioc 后缀)名字有中文的情况，我们建议重新新建工程或者把生成的工程文件重命名为英文。因为带中文的 CubeMX 工程生成的 MDK 的 Output 目录有中文，MDK 也会报错，尽管可以重新设置 MDK 工程的 Output 目录和添加③所描述步骤的启动文件，使本次编译通过，但下次重新用 CubeMX 生成工程时，仍旧需要重复修改配置。

第十一章 STM32 时钟系统

STM32H7 时钟系统的知识在《STM32H7xx 参考手册_V7（英文版）.pdf》第八章复位和时钟控制章节有较详细的讲解。这里我们对 STM32H7 的整体架构作一个简单的介绍，帮助大家更全面、系统地认识 STM32H7 系统的主控结构。了解时钟系统在整个 STM32 系统的贯穿和驱动作用，学会设置 STM32 的系统时钟。

本章将分为如下几个小节：

11.1 认识时钟树

11.2 如何修改主频

11.1 认识时钟树

数字电路的知识告诉我们：任意复杂的电路控制系统都可以经由门电路组成的组合电路实现。回顾第五章的知识点，我们知道 STM32 内部也是由多种多样的电路模块组合在一起实现的。当一个电路越复杂，在达到正确的输出结果前，它可能因为延时会有一些短暂的中间状态，而这些中间状态有时会导致输出结果会有一个短暂的错误，这叫做电路中的“毛刺现象”，如果电路需要运行得足够快，那么这些错误状态会被其它电路作为输入采样，最终形成一系列的系统错误。为了解决这个问题，在单片机系统中，设计时以时序电路控制替代纯粹的组合电路，在每一级输出结果前对各个信号进行采样，从而使得电路中某些信号即使出现延时也可以保证各个信号的同步，可以避免电路中发生的“毛刺现象”，达到精确控制输出的效果。

由于时序电路的重要性，因此在 MCU 设计时就设计了专门用于控制时序的电路，在芯片设计中称为时钟树设计。由此设计出来的时钟，可以精确控制我们的单片机系统，这也是我们这节要展开分析的时钟分析。为什么是时钟树而不是时钟呢？一个 MCU 越复杂，时钟系统也会相应地变得复杂，如 STM32H7 的时钟系统比较复杂，不像简单的 51 单片机一个系统时钟就可以解决一切。对于 STM32H7 系列的芯片，正常工作的主频可以达到 400MHz，但并不是所有外设都需要系统时钟这么高的频率，比如看门狗以及 RTC 只需要几十 KHz 的时钟即可。同一个电路，时钟越快功耗越大，同时抗电磁干扰能力也会越弱，所以对于较为复杂的 MCU 一般都是采取多时钟源的方法来解决这些问题。

STM32 内部非常复杂，外设也非常多，为了更低的功耗，STM32 默认不开启这些外设功能。用户可以根据自己的需要决定 STM32 芯片要使用哪些外设，从而再开启该外设的时钟，否则该外设不工作。

下面来看一下 STM32H7 时钟系统图。STM32H7 时钟系统图看起来有点多且复杂，但是分开几部分来理解其实没有那么难了。

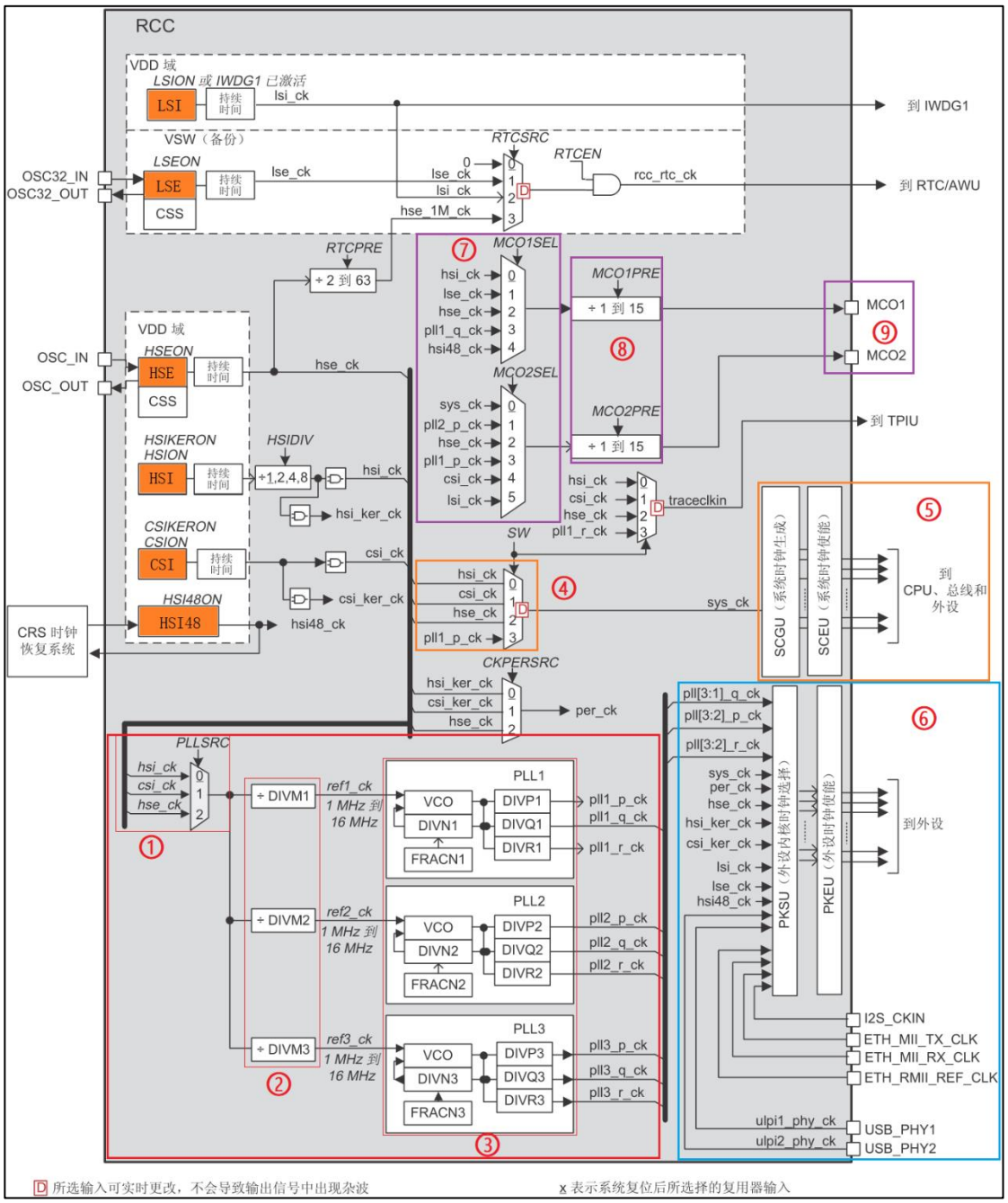


图 11.1.1 STM32H7 时钟系统图

11.1.1 时钟源

对于 STM32H7 有六个时钟源可供使用，可分为外部时钟源和内部时钟源。具体如图 11.1.1 所示，在 STM32H7 的时钟系统图里，我们把这六个时钟源标记成橘黄色。

（1）2 个外部时钟源：

- 高速外部振荡器 (HSE)

支持 4 MHz 到 48 MHz 频率范围内的晶振。

- 低速外部振荡器 (LSE)

一般是 32.768 kHz 晶振，可用于看门狗和 RTC 实时时钟。

2 个外部时钟源都是芯片外部晶振产生的时钟频率，故而都有精度高的优点。

（2）4 个内部时钟源：

- 高速内部振荡器 (HSI)

频率 64MHz 的高速 RC 振荡器，可用于系统时钟。

- 48 MHz RC 振荡器 (HSI48)

频率 48MHz，用于给特定的外设提供时钟，比如配合 CRS 可以作为 USB 的时钟源。

- 低功耗内部振荡器 (CSI)

频率约是 4MHz，主要用于低功耗。

- 低速内部振荡器 (LSI)

频率约 32 kHz 的低速 RC 振荡器，可用于 RTC 实时时钟、独立看门狗和自动唤醒。

11.1.2 锁相环 PLL

STM32H7 时钟系统图中看到红色框的部分，可以知道 STM32H7 有三个锁相环，分别是：PLL1、PLL2、PLL3。而且它们的时钟源只有一个，一起共用，但是进入到锁相环的时钟源分别经过三个不同的预分频器，这样我们就可以让锁相环有不同的输入时钟频率。

下面看到图 11.1.1 的 STM32H7 时钟系统图标号①②③这三个组成部分：

- **① 锁相环时钟源选择器**

锁相环时钟源有三个：hsi_ck、csi_ck、hse_ck。我们一般选择 hse_ck，即来自外部高速晶振，正点原子阿波罗 STM32H743 外部高速晶振为 25MHZ。

- **② PLL1\PLL2\PLL3 时钟源预分频器**

DIVM1、DIVM2 和 DIVM3 分别是 PLL1、PLL2 和 PLL3 输入时钟的预分频系数，取值范围都是：1~63，请根据实际需要设置。

- **③ PLL1\PLL2\PLL3 锁相环**

PLL1、PLL2 和 PLL3 锁相环都是一个输入，三个输出。hsi_ck、csi_ck、hse_ck 三个时钟源中的一个经过 DIVM 预分频器处理作为锁相环的输入时钟源。PLL1 三个输出分别是：pll1_p_ck、pll1_q_ck 和 pll1_r_ck，PLL2 和 PLL3 同理。

下面以 PLL1 为例介绍一下锁相环内部框图，PLL2 和 PLL3 同理。

DIVN1 是 PLL1 中 VCO 的倍频系数，其取值范围是：4~512。

DIVP1 是 pll1_p_ck 的预分频系数，取值范围是：2、4、6…128（必须是偶数）。

DIVQ1 是 pll1_q_ck 的预分频系数，取值范围是：1~128。

DIVR1 是 pll1_r_ck 的预分频系数，取值范围是：1~128。

FRACN1 是 PLL1 中 VCO 的倍频系数的小数部分，它和 DIVN1 一起组成 PLL1 的倍频系数，但是我们一般情况下都是不需要用到小数倍频的，所以小数部分我们不讨论。

这里以 pll1_p_ck 为例，简单介绍下 PLL 输出频率的计算公式（时钟 PLL 输入频率为 hse_ck）：

$$pll1_p_ck = \frac{\left(\frac{hse_ck}{DIVM1}\right) * DIVN1}{DIVP1}$$

假设外部晶振为 25Mhz，需要得到 400Mhz 的 pll1_p_ck 频率，则可以设置：DIVM1=5，DIVN1=160，DIVP1=2 即可。其他分频输出（pll1_q_ck/pll1_r_ck）计算方法与上式相似。我们把它们对应到 STM32CubeMX 工具的时钟系统图上理解就很直观了，如图 11.1.2.1 所示。

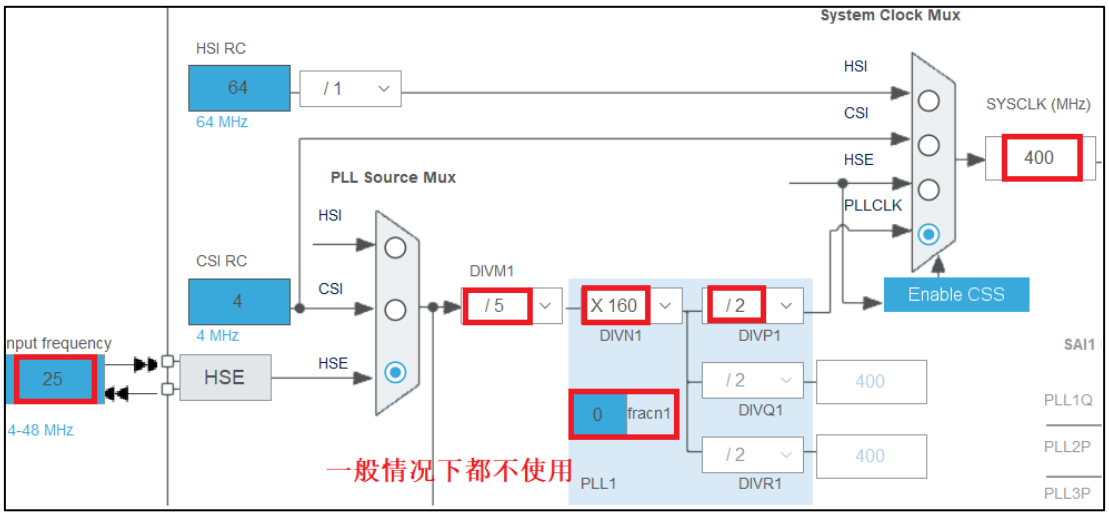


图 11.1.2.1 STM32CubeMX 对应的设置

关于时钟系统，大家如果看数据手册不太理解，结合 STM32CubeMX 的时钟系统配置界面理解会有很好的效果。

11.1.3 系统时钟 SYSCLK

下面看到图 11.1.1 的 STM32H7 时钟系统图标号④⑤两个组成部分：

- **④ sys_ck 时钟源选择器**

sys_ck（即系统时钟）的时钟源有四个：hsi_ck、csi_ck、hse_ck 和 pll1_p_ck，系统默认是选择 hsi_ck（64Mhz）作为时钟源的。当我们把 PLL1 配置好并且选择 pll1_p_ck 作为系统时钟的时钟源，则系统时钟可以得到 400MHZ 的时钟频率，从而得到最高性能。

- **⑤ 系统时钟生成和使能单元**

SCGU（System Clock Generation Unit，系统时钟生成单元），用于将 sys_ck 分成各种时钟频率，供给 CPU、SysTick，以及各条总线使用，从而有了许多的时钟名称。这里大家对以下的总线及其相对应的时钟名称有印象即可，比如：在 AHB1~4 总线上的时钟就是 rcc_hclk1~4，在 APB1~4 总线上的时钟就是 rcc_pclk1~4。这 8 条总线和对应的 8 个时钟在后面会经常提及。

SCEU（System Clock Eable Unit，系统时钟使能单元），用于使能各个外设、总线等时钟，是一个时钟开关。

接下来，我们介绍系统时钟生成单元图（非常重要），如图 11.1.3.1 所示；

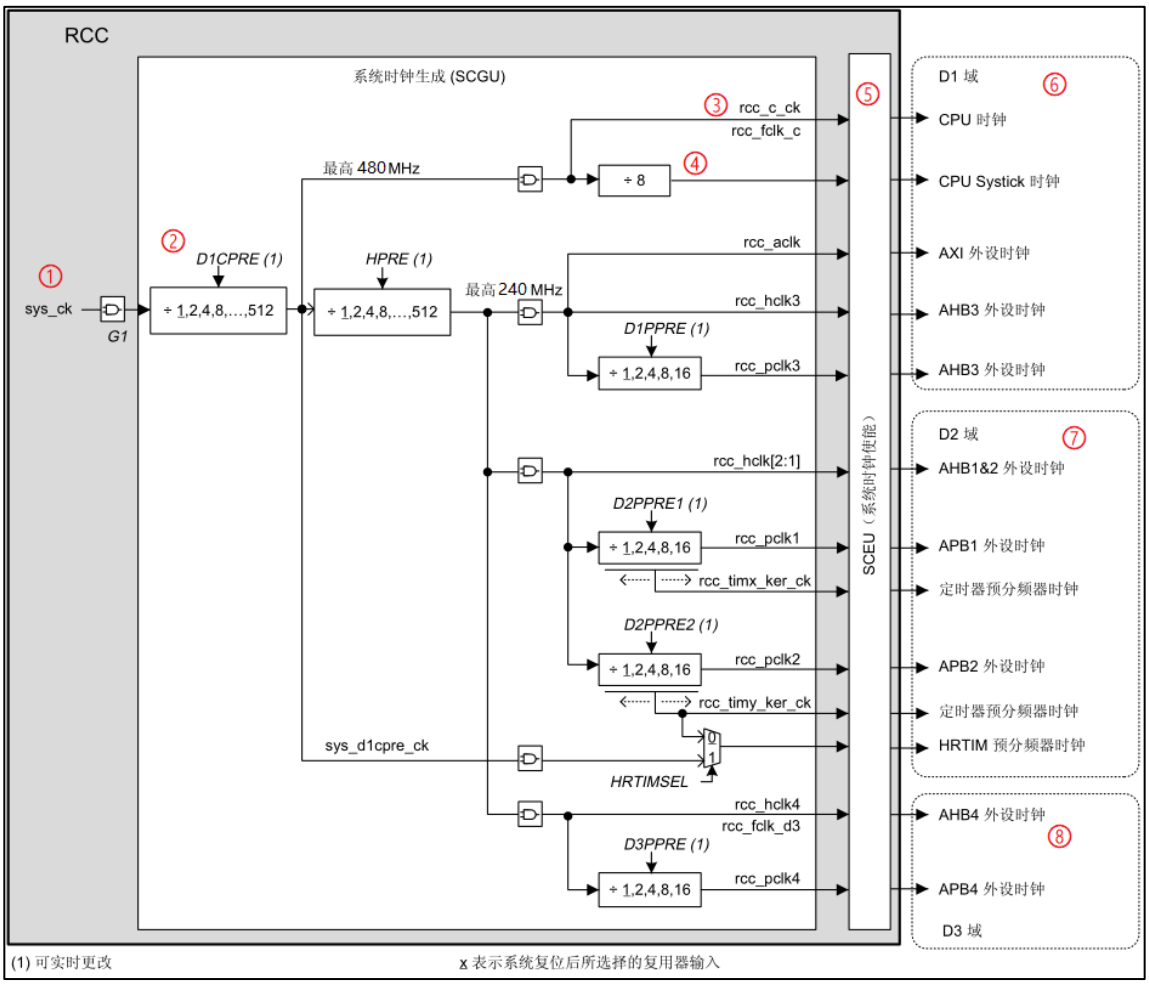


图 11.1.3.1 STM32H743 系统时钟生成图

上图主要列出了 STM32H743 系统时钟的生成原理，包括 CPU 时钟、SysTick 时钟、AXI 时钟、AHB1~4 总线和 APB1~4 总线时钟等，这些时钟对整个系统运行来说非常重要，所以该图必须要看懂。

图中，D1、D2 和 D3 域是 ST 为了支持动态能效管理，所设计的 3 个独立的电源域，每个域都能独立开启/关闭。系统时钟由 SCGU 产生，然后经过 SCEU 做开关，最终输出到各个时钟域（D1、D2 和 D3），从而能够控制和访问各类外设，保证系统的正常运行。

我们挑选出 8 个重要的地方进行介绍（图 11.1.3.1 中标出的①~⑧）：

① SCGU 输入时钟（sys_ck），该时钟我们一般选择来自 pll1_p_ck，频率为 400Mz。

② sys_d1cpre_ck 时钟的分频系数，取值范围为 1~512，通过 RCC_ D1CFGR 寄存器的 D1CPRE[3:0]位设置，我们一般设置为 1 分频，以得到最高的 sys_d1cpre_ck 频率，400Mhz。

③ CPU 时钟（rcc_c_ck、rcc_fclk_c），CPU 时钟是直接来自 sys_d1cpre_ck，没有分频器，频率为 400Mhz。

④ SysTick 时钟预分频器（可以选择 1 分频或者 8 分频），在例程中，我们选择 1 分频，因此 SysTick 的时钟源直接来自 sys_d1cpre_ck，频率为 400Mhz。

⑤ 系统时钟使能单元（SCEU），它能够对 D1、D2 和 D3 域内的所有外设时钟进行开/关控制，所以在使用外设的时候，必须设置 SCEU，使能其时钟，否则外设无法使用。这里所谓的 SCEU 设置，实际上就是通过 AHB1ENR 之类的寄存器设置的。

⑥ D1 域，是高性能处理域，包含 CPU、SysTick、AXI、AHB3 和 APB3 等部分。CPU 可以从 TCM 和 Cache 中提取紧急的或优先级较高的用户程序，在 400M 的主频下执行，确保实现最快速响应。

⑦ D2 域，外设接口域，包含大部分外设，包括：AHB1、AHB2、APB1 和 APB2 等时钟部分。**注意：定时器的时钟都是在 D2 域进行控制，而且当 D2PPRE1 或 D2PPRE2 的分频系**

数不为 1 的时候，定时器的时钟频率为 rcc_pclk1 或 rcc_pclk2 的 2 倍。

⑧D3 域，低功耗处理域，包括 AHB4 和 APB4 等时钟部分。此部分中的 ADC 可以在整个系统深度休眠时仍然进行数据处理。在电池驱动的情况下，D3 可以保证在低功耗条件下仍然进行必要的数据处理工作。

我们可以把 STM32H743 系统时钟生成图对应到 STM32CubeMX 工具上理解，如图 11.1.3.2 所示。

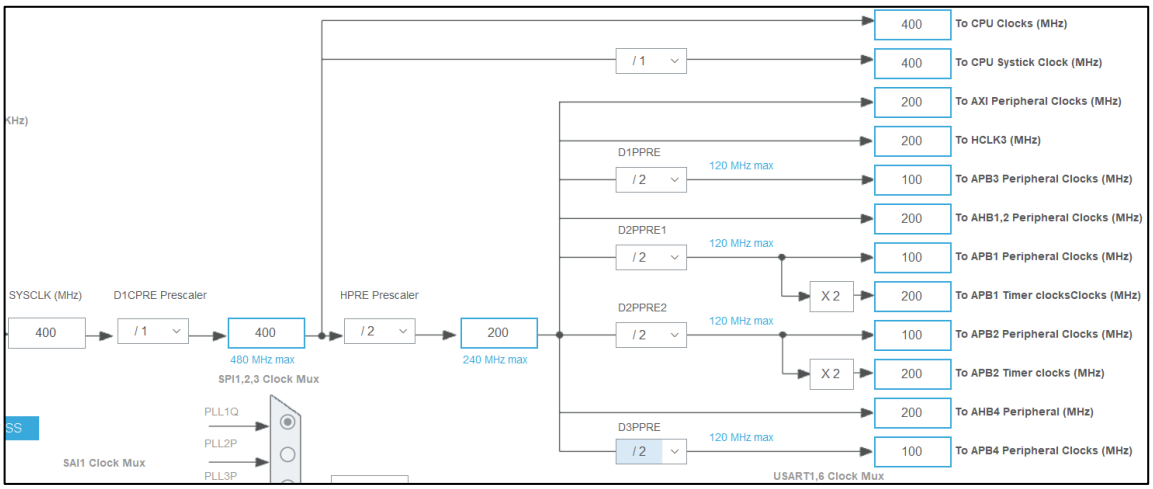


图 11.1.3.2 对应 STM32CubeMX 配置图

11.1.4 外设内核时钟

下面看到图 11.1.1 的 STM32H7 时钟系统图标号⑥部分：

- **⑥外设内核时钟选择和使能单元**

PKSU（Peripheral Kernel clock Selection Unit，外设内核时钟选择单元），用于选择部分外设的内核时钟源，STM32H7 内部，很多外设都可以自由选择内核时钟来源，从而提高使用的灵活性。

PKEU（Peripheral Kernel clock Enable Unit，外设内核时钟使能单元），用于使能部分外设的内核时钟，从而控制外设内核时钟的开/关。

接下来我们看看外设时钟使能框图，如图 11.1.4.1 所示：

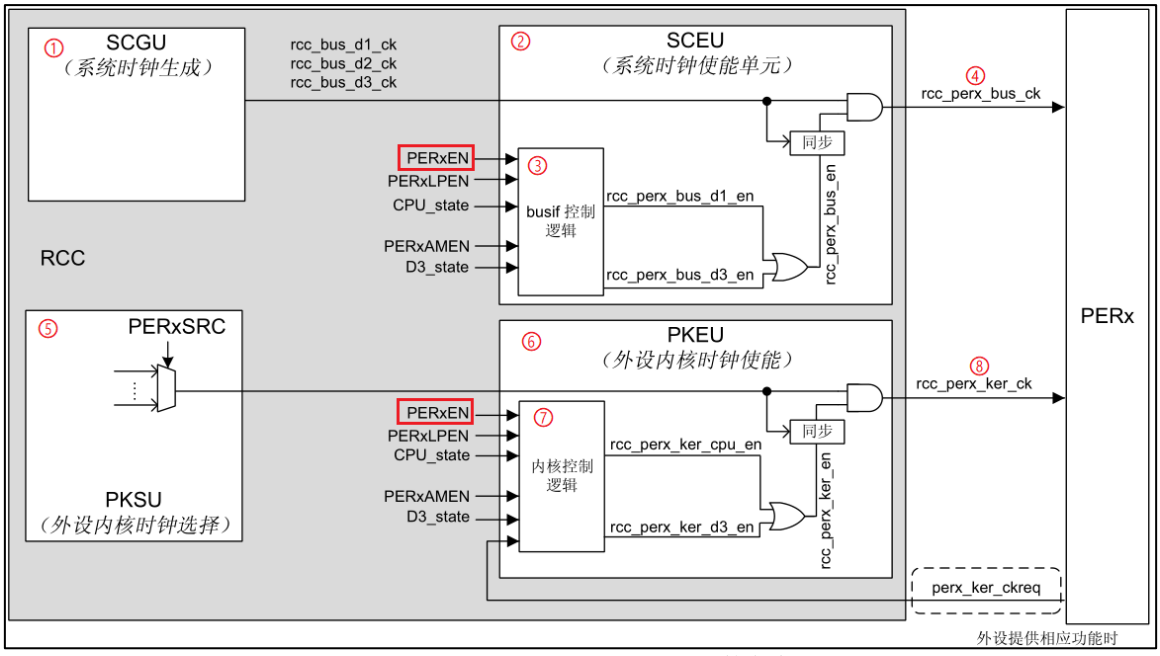


图 11.1.4.1 STM32H743 外设时钟使能框图

图中，PERx（perx）代表外设 x（x 是 STM32H7 内部的外设，比如 USART、TIM、SPI 等），通过此图，我们可以可以知道 STM32H7 内部外设时钟的由来。

我们挑选出 8 个重要的地方进行介绍（图 11.1.4.1 中标出的①~⑧）：

①系统时钟生成单元（SCGU），此部分在图 11.1.3.1 我们进行了详细介绍，这里通过 SCGU 产生了三大总线时钟：rcc_bus_d1_ck、rcc_bus_d2_ck、rcc_bus_d3_ck，这些时钟实际上是指 rcc_hclk1~4、rcc_pclk1~4 等时钟。

② 系统时钟使能单元（SCEU），此部分将 SCGU 生成的时钟进行使能/禁止操作，最终控制是否输出时钟（rcc_perx_bus_ck）给外设。

③ 总线接口时钟控制逻辑（busif Control Logic），用于控制 SCEU 是否输出时钟给外设，它有很多控制信号，其中我们常用的是 PERxEN（图中红框框出部分），通过这个位的设置就可以控制具体外设的时钟开/关。

④ 外设总线时钟（rcc_perx_bus_ck），该时钟主要用于访问外设寄存器，对外设进行设置。任何外设都必须使能该时钟才可以正常使用。

⑤ 外设内核时钟选择单元（PKSU，即：Peripheral Kernel clock Selection Unit），用于选择某个外设的内核时钟来源选择。

⑥ 外设内核时钟使能单元（PKEU，即：Peripheral Kernel clock Enable Unit），此部分将⑤处选择的外设内核时钟进行使能/禁止操作，最终控制是否输出内核时钟（rcc_perx_ker_ck）给外设。

⑦ 内核时钟控制逻辑（Kernel Control Logic），用于控制 PKEU 是否输出内核时钟给外设，它有很多控制信号，其中我们常用的是 PERxEN（图中红圈圈出部分），通过这个位的设置就可以控制具体外设内核时钟的开/关。注意：③和⑦，都可以通过 PERxEN 控制使能。

⑧ 外设内核时钟（rcc_perx_ker_ck），该时钟用于驱动外设产生时序，如波特率、时钟脉冲等。大部分外设都需要用到 rcc_perx_ker_ck，比如串口、SPI、IIC、FMC、SAI、LTDC 和 CAN 等，但是，并不是所有的外设都需要用内核时钟来产生时序，比如：OPAMP、DMA 等。

因此，大部分外设的使用，需要同时用到外设总线时钟（rcc_perx_bus_ck）和外设内核时钟（rcc_perx_ker_ck），一般情况下，这两个时钟都是由 PERxEN 控制使能。至于哪些外设有内核时钟，请参考：《STM32H7xx 参考手册_V7（英文版）.pdf》352 页，Table 59。

SCEU 和 PKEU，一个用于控制外设的访问时钟（访问寄存器），一个用于控制外设的内核时钟（生成控制时序，如波特率等）。但是，并不是所有的外设都需要用到 PKEU，因为有些外设并不需要生成时序，没有所谓的外设内核时钟，比如 DMA、OPAMP 等，这些外设只需要在 SCEU 进行使能即可。

11.1.5 MCO 时钟输出

MCO 时钟输出，其作用是为外部器件提供时钟，STM32H743 有两个 MCO。下面看到 STM32H7 时钟系统图标号⑦⑧⑨三个部分：

- **⑦MCO1\MCO2 时钟源选择器**

MCO1（外部器件的输出时钟 1）时钟源有五个：hsi_ck、lse_ck、hse_ck、pll1_q_ck 和 hsi48_ck。

MCO2（外部器件的输出时钟 2）时钟源有六个：sys_ck、pll2_p_ck、hse_ck、pll1_p_ck、csi_ck 和 lsi_ck。

- **⑧MCO1\MCO2 时钟分频器**

MCO1PRE 是 MCO1 的预分频器，取值范围：1 到 15。

MCO2PRE 是 MCO2 的预分频器，取值范围：1 到 15。

- **⑨MCO1\MCO2 时钟输出引脚**

MCO1、MCO2 两个时钟输出引脚给外部器件提供时钟源（分别由 PA8 和 PC9 复用功能实现），每个引脚可以选择一个时钟源，通过 RCC 时钟配置寄存器 (RCC_CFGR)进行配置。

关于 STM32H7xx 时钟的详细介绍，请参考《STM32H7xx 参考手册_V7（英文版）.pdf》第 8.5 节，有不明白的地方，可以对照手册仔细研究。具体哪个外设是连接在哪个时钟总线上，以及对应的时钟总线最高主频是多少，请参考 STM32H743IIT6.pdf 数据手册（路径：A 盘→7，

硬件资料→2，芯片资料→ STM32H743IIT6.pdf）。

通过以上内容的介绍，我们知道 STM32H743 的时钟设计的比较复杂，各个时钟基本都是可控的，任何外设都有对应的时钟控制开关，这样的设计，对降低功耗是非常有用的，不用的外设不开启时钟，就可以大大降低其功耗。

11.2 如何修改主频

STM32H743 默认的情况下（比如：串口 IAP 时或者是未初始化时钟时），使用的是内部 64M 的 HSI 作为时钟源，所以不需要外部晶振也可以下载和运行代码的。

下面我们来讲解如何让 STM32H743 芯片在 400MHZ 的频率下工作，这是官方推荐使用的最高的稳定时钟频率。正点原子的阿波罗 STM32H743 开发板的外部高速晶振的频率是 25MHZ，我们就是在这个晶振频率的基础上，通过各种倍频和分频得到 400MHZ 的系统工作频率。

11.2.1 STM32H7 时钟系统配置

下面我们将分几步给大家讲解 STM32H7 时钟系统配置过程，这部分内容很重要，请大家认真阅读。

第 1 步：配置 HSE_VALUE

讲解 stm32h7xx_hal_conf.h 文件的时候，我们知道需要宏定义 HSE_VALUE 匹配我们实际硬件的高速晶振频率（这里是 25MHZ），代码如下：

```
#if !defined  (HSE_VALUE)
#define HSE_VALUE    ((uint32_t)25000000) /* 外部高速振荡器的值，单位 HZ */
#endif /* HSE_VALUE */
```

第 2 步：调用 SystemInit 函数

我们介绍启动文件的时候就知道，在系统启动之后，程序会先执行 SystemInit 函数，进行系统一些初始化配置。启动代码调用 SystemInit 函数如下：

```
Reset_Handler    PROC
                 EXPORT  Reset_Handler                    [WEAK]
        IMPORT  SystemInit
        IMPORT  __main

                 LDR     R0, =SystemInit
                 BLX     R0
                 LDR     R0, =__main
                 BX      R0
                 ENDP
```

下面我们来看看 system_stm32h7xx.c 文件下定义的 SystemInit 程序，源码在 174 行到 302 行，简化函数如下。

```
void SystemInit (void)
{
  /* FPU 设置------------------------------------------------------------*/
  #if (__FPU_PRESENT == 1) && (__FPU_USED == 1)
    SCB->CPACR |= ((3UL << (10*2))|(3UL << (11*2)));/* 设置 CP10 和 CP11 为全访问 */
  #endif
  /* 复位 RCC 时钟配置为默认配置 */
  /* 打开 HSION 位 */
  RCC->CR |= RCC_CR_HSION;
  RCC->CFGR = 0x00000000;  /* 复位 CFGR 寄存器 */
  /* 复位 HSEON, CSSON , CSION,RC48ON, CSIKERON PLL1ON, PLL2ON and PLL3ON 位 */
  RCC->CR &= 0xEAF6ED7FU;
  RCC->D1CFGR = 0x00000000;         /* 复位 D1CFGR 寄存器 */
  RCC->D2CFGR = 0x00000000;         /* 复位 D2CFGR 寄存器 */
  RCC->D3CFGR = 0x00000000;         /* 复位 D3CFGR 寄存器 */
  RCC->PLLCKSELR = 0x00000000;      /* 复位 PLLCKSELR 寄存器 */
```

```
  RCC->PLLCFGR = 0x00000000;          /* 复位 PLLCFGR 寄存器 */
  RCC->PLL1DIVR = 0x00000000;         /* 复位 PLL1DIVR 寄存器 */
  RCC->PLL1FRACR = 0x00000000;        /* 复位 PLL1FRACR 寄存器 */
  RCC->PLL2DIVR = 0x00000000;         /* 复位 PLL2DIVR 寄存器 */
  RCC->PLL2FRACR = 0x00000000;        /* 复位 PLL2FRACR 寄存器 */
  RCC->PLL3DIVR = 0x00000000;         /* 复位 PLL3DIVR 寄存器 */
  RCC->PLL3FRACR = 0x00000000;        /* 复位 PLL3FRACR 寄存器 */
  RCC->CR &= 0xFFFBFFFFU;             /* 清除 CR 寄存器的 HSEBYP 位 */
  RCC->CIER = 0x00000000;             /* 关闭所有的中断 */

  /* 双核 CM7 或单核线 */
  if((DBGMCU->IDCODE & 0xFFFF0000U) < 0x20000000U)
  {
    /* if stm32h7 revY*/
   /* Change  the switch matrix read issuing capability
      to 1 for the AXI SRAM target (Target 7) */
    *((__IO uint32_t*)0x51008108) = 0x000000001U;
  }

  /* Configure the Vector Table location add offset address for cortex-M7 ---*/
#ifdef VECT_TAB_SRAM
  SCB->VTOR = D1_AXISRAM_BASE  | VECT_TAB_OFFSET;
#else
  SCB->VTOR = FLASH_BANK1_BASE | VECT_TAB_OFFSET;
#endif
}
```

从上面代码可以看出，SystemInit 主要做了如下三个方面工作：

1) 设置 FPU

2) 复位 RCC 时钟配置为默认复位值（默认开启 HSI）

3) 配置中断向量表地址

HAL 库的 SystemInit 函数并没有像标准库的 SystemInit 函数一样进行时钟的初始化配置。HAL 库的 SystemInit 函数除了打开 HSI 之外，没有任何时钟相关配置，所以使用 HAL 库，我们必须编写自己的时钟配置函数。

第 3 步：在 main 函数里调用用户编写的时钟设置函数

我们打开 HAL 库例程实验 1 跑马灯实验，看看我们在工程目录 Drivers\SYSTEM 分组下面定义的 sys.c 文件中的时钟设置函数 sys_stm32_clock_init 的内容：

```
/**
 * @brief       时钟设置函数
 * @param       plln: PLL1 倍频系数(PLL 倍频), 取值范围: 4~512.
 * @param       pllm: PLL1 预分频系数(进 PLL 之前的分频), 取值范围: 2~63.
 * @param       pllp: PLL1 的 p 分频系数(PLL 之后的分频), 分频后作为系统时钟, 取值范围:
 *                    2~128.(且必 须是 2 的倍数)
 * @param       pllq: PLL1 的 q 分频系数(PLL 之后的分频), 取值范围: 1~128.
 * @note
 *
 *              Fvco: VCO 频率
 *              Fsys: 系统时钟频率, 也是 PLL1 的 p 分频输出时钟频率
 *              Fq:   PLL1 的 q 分频输出时钟频率
 *              Fs:   PLL 输入时钟频率, 可以是 HSI, CSI, HSE 等.
 *              Fvco = Fs * (plln / pllm);
 *              Fsys = Fvco / pllp = Fs * (plln / (pllm * pllp));
 *              Fq   = Fvco / pllq = Fs * (plln / (pllm * pllq));
 *
 *              外部晶振为 25M 的时候,推荐值:plln=160, pllm = 5, pllp = 2, pllq = 4.
 *              得到:Fvco = 25 * (160 / 5) = 800Mhz
 *                   Fsys = pll1_p_ck = 800 / 2 = 400Mhz
 *                   Fq   = pll1_q_ck = 800 / 4 = 200Mhz
 *
```

```
 *              H743 默认需要配置的频率如下:
 *              CPU 频率(rcc_c_ck) = sys_d1cpre_ck = 400Mhz
 *              rcc_aclk = rcc_hclk3 = 200Mhz
 *              AHB1/2/3/4(rcc_hclk1/2/3/4) = 200Mhz
 *              APB1/2/3/4(rcc_pclk1/2/3/4) = 100Mhz
 *              pll2_p_ck = (25 / 25) * 440 / 2) = 220Mhz
 *              pll2_r_ck = FMC 时钟频率 = ((25 / 25) * 440 / 2) = 220Mhz
 *
 * @retval     错误代码: 0, 成功; 1, 错误;
 */
uint8_t sys_stm32_clock_init(uint32_t plln, uint32_t pllm,
                             uint32_t pllp, uint32_t pllq)
{
    HAL_StatusTypeDef ret = HAL_OK;
    RCC_ClkInitTypeDef rcc_clk_init_handle;
    RCC_OscInitTypeDef rcc_osc_init_handle;

    MODIFY_REG(PWR->CR3, PWR_CR3_SCUEN, 0); /* 使能供电配置更新 */
    /* VOS = 1, Scale1, 1.2V 内核电压,FLASH 访问可以得到最高性能 */
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
    while ((PWR->D3CR & (PWR_D3CR_VOSRDY)) != PWR_D3CR_VOSRDY);/* 等待电压稳定 */

    /* 使能 HSE，并选择 HSE 作为 PLL 时钟源，配置 PLL1，开启 USB 时钟 */
    rcc_osc_init_handle.OscillatorType = RCC_OSCILLATORTYPE_HSE
                                       | RCC_OSCILLATORTYPE_HSI48;
    rcc_osc_init_handle.HSEState = RCC_HSE_ON;
    rcc_osc_init_handle.HSIState = RCC_HSI_OFF;
    rcc_osc_init_handle.CSIState = RCC_CSI_OFF;
    rcc_osc_init_handle.HSI48State = RCC_HSI48_ON;
    rcc_osc_init_handle.PLL.PLLState = RCC_PLL_ON;
    rcc_osc_init_handle.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    rcc_osc_init_handle.PLL.PLLN = plln;
    rcc_osc_init_handle.PLL.PLLM = pllm;
    rcc_osc_init_handle.PLL.PLLP = pllp;
    rcc_osc_init_handle.PLL.PLLQ = pllq;
    rcc_osc_init_handle.PLL.PLLVCOSEL = RCC_PLL1VCOWIDE;
    rcc_osc_init_handle.PLL.PLLRGE = RCC_PLL1VCIRANGE_2;
    rcc_osc_init_handle.PLL.PLLFRACN = 0;
    ret = HAL_RCC_OscConfig(&rcc_osc_init_handle);
    if(ret != HAL_OK)
    {
        return 1;
    }

    /*
     *  选择 PLL 的输出作为系统时钟
     *  配置 RCC_CLOCKTYPE_SYSCLK 系统时钟,400M
     *  配置 RCC_CLOCKTYPE_HCLK 时钟,200Mhz,对应 AHB1，AHB2，AHB3 和 AHB4 总线
     *  配置 RCC_CLOCKTYPE_PCLK1 时钟,100Mhz,对应 APB1 总线
     *  配置 RCC_CLOCKTYPE_PCLK2 时钟,100Mhz,对应 APB2 总线
     *  配置 RCC_CLOCKTYPE_D1PCLK1 时钟,100Mhz,对应 APB3 总线
     *  配置 RCC_CLOCKTYPE_D3PCLK1 时钟,100Mhz,对应 APB4 总线
     */

    rcc_clk_init_handle.ClockType = (RCC_CLOCKTYPE_SYSCLK \
                                   | RCC_CLOCKTYPE_HCLK \
                                   | RCC_CLOCKTYPE_PCLK1 \
                                   | RCC_CLOCKTYPE_PCLK2 \
                                   | RCC_CLOCKTYPE_D1PCLK1 \
                                   | RCC_CLOCKTYPE_D3PCLK1);

    rcc_clk_init_handle.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    rcc_clk_init_handle.SYSCLKDivider = RCC_SYSCLK_DIV1;
    rcc_clk_init_handle.AHBCLKDivider = RCC_HCLK_DIV2;
```

```
    rcc_clk_init_handle.APB1CLKDivider = RCC_APB1_DIV2;
    rcc_clk_init_handle.APB2CLKDivider = RCC_APB2_DIV2;
    rcc_clk_init_handle.APB3CLKDivider = RCC_APB3_DIV2;
    rcc_clk_init_handle.APB4CLKDivider = RCC_APB4_DIV2;
    ret = HAL_RCC_ClockConfig(&rcc_clk_init_handle, FLASH_LATENCY_2);
    if(ret != HAL_OK)
    {
        return 1;
    }

    HAL_PWREx_EnableUSBVoltageDetector();       /* 使能 USB 电压电平检测器 */
    __HAL_RCC_CSI_ENABLE() ;                    /* 使能 CSI 时钟 */
    __HAL_RCC_SYSCFG_CLK_ENABLE() ;             /* 使能 SYSCFG 时钟 */
    HAL_EnableCompensationCell();               /* 使能 IO 补偿单元 */
    return 0;
}
```

函数 sys_stm32_clock_init 就是用户的时钟系统配置函数，除了配置 PLL 相关参数确定 SYSCLK 值之外，还配置了 AHB、APB1、APB2、APB3 和 APB4 总线时钟的预分频系数，也就是确定了 rcc_hclk1~4 和 rcc_pclk1~4 等时钟频率。我们首先来看看使用 HAL 库配置 STM32H7 时钟系统的一般步骤：

1) 设置调压器输出电压级别：调用函数__HAL_PWR_VOLTAGESCALING_CONFIG()。

2) 配置时钟源相关参数：调用函数 HAL_RCC_OscConfig()。

3) 选择系统时钟 SYSCLK 的时钟来源以及配置 AHB、APB1、APB2、APB3 和 APB4 总线时钟的预分频系数，调用 HAL_RCC_ClockConfig()函数。

4) 使能其他一些时钟和 IO 补偿单元。

下面我们详细讲解这个 4 个步骤。

步骤 1：调压器输出电压级别 VOS，它是由 PWR 控制寄存器 D3CR 的位 15:14 来确定的：

```
位 15:14 VOS[1:0]
00:保留（默认选择电压调节 3）
01:电压调节 3（默认）
10:电压调节 2
11:电压调节 1
```

电压调节级别数值越小工作频率越高，所以如果我们要配置 H743 的主频为 400MHz，那么我们需要配置调压器输出电压调节级别 VOS 为级别 1，详细的表格参考在步骤 3 会讲解。所以函数 sys_stm32_clock_init 中步骤 1 源码如下：

```
/* VOS = 1, Scale1, 1.2V 内核电压,FLASH 访问可以得到最高性能 */
__HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);
```

步骤 2：配置时钟源和锁相环 PLL1 的相关参数，使能并选择 HSE 作为 PLL 时钟源，我们调用的函数为 HAL_RCC_OscConfig()，该函数在 HAL 库头文件 stm32h7xx_hal_rcc.h 中声明，在文件 stm32h7xx_hal_rcc.c 中定义。首先我们来看看该函数声明：

```
HAL_StatusTypeDef HAL_RCC_OscConfig(RCC_OscInitTypeDef *RCC_OscInitStruct);
```

该函数只有一个形参，就是 RCC_OscInitTypeDef 结构体类型指针。接下来我们看看 RCC_OscInitTypeDef 结构体的定义：

```
typedef struct
{
  uint32_t OscillatorType;              /* 需要选择配置的振荡器类型 */
  uint32_t HSEState;                    /* HSE 状态 */
  uint32_t LSEState;                    /* LSE 状态 */
  uint32_t HSIState;                    /* HSI 状态 */
  uint32_t HSICalibrationValue;         /* HSI 校准值 */
  uint32_t LSIState;                    /* LSI 状态 */
  uint32_t HSI48State;                  /* HSI48 的状态 */
  uint32_t CSIState;                    /* CSI 状态 */
  uint32_t CSICalibrationValue;         /* CSI 校准值 */
  RCC_PLLInitTypeDef PLL;               /* PLL 配置 */
}RCC_OscInitTypeDef;
```

该结构体前面几个参数主要是用来选择配置的振荡器类型。比如我们要开启 HSE，那么我们会设置 OscillatorType 的值为 RCC_OSCILLATORTYPE_HSE，然后设置 HSEState 的值为 RCC_HSE_ON 开启 HSE。对于其他时钟源：HSI、LSI、LSE、CSI 和 HSI48，配置方法类似，这里我们还开启了 HSI48 的时钟。

RCC_OscInitTypeDef 这个结构体还有一个很重要的成员变量是 PLL，它是结构体 RCC_PLLInitTypeDef 类型。它的作用是配置 PLL 相关参数，我们来看看它的定义：

```
typedef struct
{
  uint32_t PLLState;        /* PLL1 状态 */
  uint32_t PLLSource;       /* PLL1 时钟源 */
  uint32_t PLLM;            /* PLL1 分频系数 M */
  uint32_t PLLN;            /* PLL1 倍频系数 N */
  uint32_t PLLP;            /* PLL1 分频系数 P */
  uint32_t PLLQ;            /* PLL1 分频系数 Q */
  uint32_t PLLR;            /* PLL1 分频系数 R */
  uint32_t PLLRGE;          /* PLL1 时钟输入范围 */
  uint32_t PLLVCOSEL;       /* PLL1 时钟输出范围 */
  uint32_t PLLFRACN;        /* PLL1 VCO 乘数因子的小数部分 */
}RCC_PLLInitTypeDef;
```

从 RCC_PLLInitTypeDef 结构体的定义很容易看出该结构体主要用来设置 PLL 时钟源以及相关分频倍频参数。这个结构体定义的相关内容请结合图 11.1.1 时钟树中红色框部分内容一起理解。

接下来看看系统时钟初始化函数 sys_stm32_clock_init 中的配置内容：

```
/* 使能 HSE，并选择 HSE 作为 PLL 时钟源，配置 PLL1，开启 USB 时钟 */
rcc_osc_init_handle.OscillatorType = RCC_OSCILLATORTYPE_HSE |
                                     RCC_OSCILLATORTYPE_HSI48;
rcc_osc_init_handle.HSEState = RCC_HSE_ON;              /* 打开 HSE */
rcc_osc_init_handle.HSIState = RCC_HSI_OFF;             /* 关闭 HSI */
rcc_osc_init_handle.CSIState = RCC_CSI_OFF;             /* 关闭 CSI */
rcc_osc_init_handle.HSI48State = RCC_HSI48_ON;          /* 打开 HSI48 */
rcc_osc_init_handle.PLL.PLLState = RCC_PLL_ON;          /* 打开 PLL */
rcc_osc_init_handle.PLL.PLLSource = RCC_PLLSOURCE_HSE; /* 设置 PLL 时钟源为 HSE */
rcc_osc_init_handle.PLL.PLLN = plln;
rcc_osc_init_handle.PLL.PLLM = pllm;
rcc_osc_init_handle.PLL.PLLP = pllp;
rcc_osc_init_handle.PLL.PLLQ = pllq;
rcc_osc_init_handle.PLL.PLLVCOSEL = RCC_PLL1VCOWIDE;
rcc_osc_init_handle.PLL.PLLRGE = RCC_PLL1VCIRANGE_2;
rcc_osc_init_handle.PLL.PLLFRACN = 0;
ret = HAL_RCC_OscConfig(&rcc_osc_init_handle);
```

通过函数的该段程序，我们开启了 HSE 时钟源，同时选择 PLL 时钟源为 HSE，然后把 sys_stm32_clock_init 的 4 个形式参数直接设置作为 PLL 的参数 N、M、P 和 Q 的值，这样就达到了设置 PLL 时钟源相关参数的目的。另外我们还开启了 HSI48 时钟，为 USB 相关实验做准备。

设置好时钟源和 PLL1 的相关参数后，就完成了步骤 2 的内容。

步骤 3：选择系统时钟 SYSCLK 的时钟来源以及配置 AHB、APB1、APB2、APB3 和 APB4 总线时钟的预分频系数，用函数 HAL_RCC_ClockConfig()，声明如下：

```
HAL_StatusTypeDef HAL_RCC_ClockConfig(RCC_ClkInitTypeDef *RCC_ClkInitStruct,
                                      uint32_t FLatency);
```

该函数有两个形参，第一个形参 RCC_ClkInitStruct 是 RCC_ClkInitTypeDef 结构体类型指针变量，用于设置 SYSCLK 时钟源以及 SYSCLK、AHB、APB1、APB2、APB3 和 APB4 的分频系数。第二个形参 FLatency 用于设置 FLASH 延迟。

RCC_ClkInitTypeDef 结构体类型定义比较简单，我们来看看其定义：

```
typedef struct
{
  uint32_t ClockType;               /* 要配置的时钟 */
  uint32_t SYSCLKSource;            /* 系统时钟源 */
```

```
  uint32_t SYSCLKDivider;          /* SYSCLK 分频系数 */
  uint32_t AHBCLKDivider;          /* AHB 分频系数 */
  uint32_t APB3CLKDivider;         /* APB3 分频系数 */
  uint32_t APB1CLKDivider;         /* APB1 分频系数 */
  uint32_t APB2CLKDivider;         /* APB2 分频系数 */
  uint32_t APB4CLKDivider;         /* APB4 分频系数 */
}RCC_ClkInitTypeDef;
```

我们在 sys_stm32_clock_init 函数中的实际应用配置内容如下：

```
/*
 *  选择 PLL 的输出作为系统时钟
 *  配置 RCC_CLOCKTYPE_SYSCLK 系统时钟,400M
 *  配置 RCC_CLOCKTYPE_HCLK 时钟,200Mhz,对应 AHB1，AHB2，AHB3 和 AHB4 总线
 *  配置 RCC_CLOCKTYPE_PCLK1 时钟,100Mhz,对应 APB1 总线
 *  配置 RCC_CLOCKTYPE_PCLK2 时钟,100Mhz,对应 APB2 总线
 *  配置 RCC_CLOCKTYPE_D1PCLK1 时钟,100Mhz,对应 APB3 总线
 *  配置 RCC_CLOCKTYPE_D3PCLK1 时钟,100Mhz,对应 APB4 总线
 */

rcc_clk_init_handle.ClockType = (RCC_CLOCKTYPE_SYSCLK \
                                 | RCC_CLOCKTYPE_HCLK \
                                 | RCC_CLOCKTYPE_PCLK1 \
                                 | RCC_CLOCKTYPE_PCLK2 \
                                 | RCC_CLOCKTYPE_D1PCLK1 \
                                 | RCC_CLOCKTYPE_D3PCLK1);

rcc_clk_init_handle.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
rcc_clk_init_handle.SYSCLKDivider = RCC_SYSCLK_DIV1;
rcc_clk_init_handle.AHBCLKDivider = RCC_HCLK_DIV2;
rcc_clk_init_handle.APB1CLKDivider = RCC_APB1_DIV2;
rcc_clk_init_handle.APB2CLKDivider = RCC_APB2_DIV2;
rcc_clk_init_handle.APB3CLKDivider = RCC_APB3_DIV2;
rcc_clk_init_handle.APB4CLKDivider = RCC_APB4_DIV2;
ret = HAL_RCC_ClockConfig(&rcc_clk_init_handle, FLASH_LATENCY_4);
```

sys_stm32_clock_init 函数中的 RCC_ClkInitTypeDef 结构体配置内容：

第一个成员 ClockType 配置表示我们要配置到的时钟，把所以配置到的时钟的宏都赋值给该成员，这里我们包含了六个宏定义。

第二个成员 SYSCLKSource 配置选择系统时钟源为 PLL。

第三个成员 SYSCLKDivider 配置 SYSCLK 分频系数为 1。

第四个成员 AHBCLKDivider 配置 AHB 总线时钟预分频系数为 2。

第五个成员 APB1CLKDivider 配置 APB1 总线时钟预分频系数为 2。

第六个成员 APB2CLKDivider 配置 APB2 总线时钟预分频系数为 2。

第七个成员 APB3CLKDivider 配置 APB3 总线时钟预分频系数为 2。

第八个成员 APB4CLKDivider 配置 APB4 总线时钟预分频系数为 2。

根据我们在 mian 函数中调用 sys_stm32_clock_init(160, 5, 2, 4)时设置的形参数值，代入 PLL 输出频率的计算公式可得：

$$pll1_p_ck = \frac{\left(\frac{hse_ck}{DIVM1}\right)*DIVN1}{DIVP1} = \frac{\left(\frac{hse_ck}{M}\right)*N}{P} = \frac{\left(\frac{25MHZ}{5}\right)*160}{2} = 400MHZ$$

因为我们选择系统时钟源为 PLL（pll1_p_ck），所以系统时钟 SYSCLK=400MHz。再结合各个总线时钟的预分频系数，总结一下调用函数 sys_stm32_clock_init(160, 5, 2, 4)之后，关键的时钟频率值如下：（HCLK 代表 AHB1~4 总线上的时钟）

```
sys_ck(系统时钟)                          = 400MHz
sys_ck 分频后的时钟（sys_d1cpre_ck）       = 400MHz
AHB 总线时钟（HCLK= sys_d1cpre_ck /2）     = 200MHz
APB1 总线时钟（rcc_pclk1=HCLK/2）          = 100MHz
APB2 总线时钟（rcc_pclk2=HCLK/2）          = 100MHz
APB3 总线时钟 (rcc_pclk3=HCLK/2)           = 100MHz
APB4 总线时钟 (rcc_pclk4=HCLK/2)           = 100MHz
```

最后我们来看看函数 HAL_RCC_ClockConfig 第二个入口参数 FLatency 的含义，对于 STM32H7 系列，FLASH 延迟配置参数值是通过下表来确定的：

Number of wait states (LATENCY)	Programming delay (WRHIGHFREQ)	AXI Interface clock frequency vs V_{CORE} range			
		VOS3 range 0.95 V - 1.05 V	VOS2 range 1.05 V - 1.15 V	VOS1 range 1.15 V - 1.26 V	VOS0 range 1.26 V - 1.40 V
0 WS (1 FLASH clock cycle)	00	[0;45 MHz]	[0;55 MHz]	[0;70 MHz]	[0;70 MHz]
1 WS (2 FLASH clock cycles)	01	]45 MHz;90 MHz]	]55 MHz;110 MHz]	]70 MHz;140 MHz]	]70 MHz;140 MHz]
2 WS (3 FLASH clock cycles)	01	]90 MHz;135 MHz]	]110 MHz;165 MHz]	]140 MHz;185 MHz]	]140 MHz;185 MHz]
	10	-	-	]185 MHz;210 MHz]	]185 MHz;210 MHz]
3 WS (4 FLASH clock cycles)	10	]135 MHz;180 MHz]	]165 MHz;225 MHz]	]210 MHz;225 MHz]	]210 MHz;225 MHz]
4 WS (5 FLASH clock cycles)	10	]180 MHz;225 MHz]	225 MHz	-	]225 MHz;240 MHz]

表 11.2.1.1 FLASH 推荐的等待状态和编程延迟数

上表中的时钟频率为 ACLK(即 AXI Interface clock)，ACLK=HCLK=200MHz（我们现在实际是 200MHZ），上表中，当调压器设置为 VOS1 级别的时候(我们设置的就是 VOS1 级别)，我们的 ACLK=HCLK=200MHz，那么 FLASH 延迟等待周期要为 2WS，也就是总共 3 个 FLASH 时钟周期才访问一次 FLASH。下面我们看看我们在 sys_stm32_clock_init 中调用函数 HAL_RCC_ClockConfig 的时候，第二个形参设置值：

```
ret = HAL_RCC_ClockConfig(&rcc_clk_init_handle, FLASH_LATENCY_2);
```

从上可以看出，我们设置值为 FLASH_LATENCY_2，也就是每次访问 FLASH 需要延迟 2WS，总共要 3 个 FLASH 周期。

最后，再给大家用 STM32CubeMX 工具的时钟配置界面，标记一下：sys_ck、sys_d1cpre_ck、rcc_hclk1~4 以及 rcc_pclk1~4 对应的位置，方便大家更容易理解。如下图：

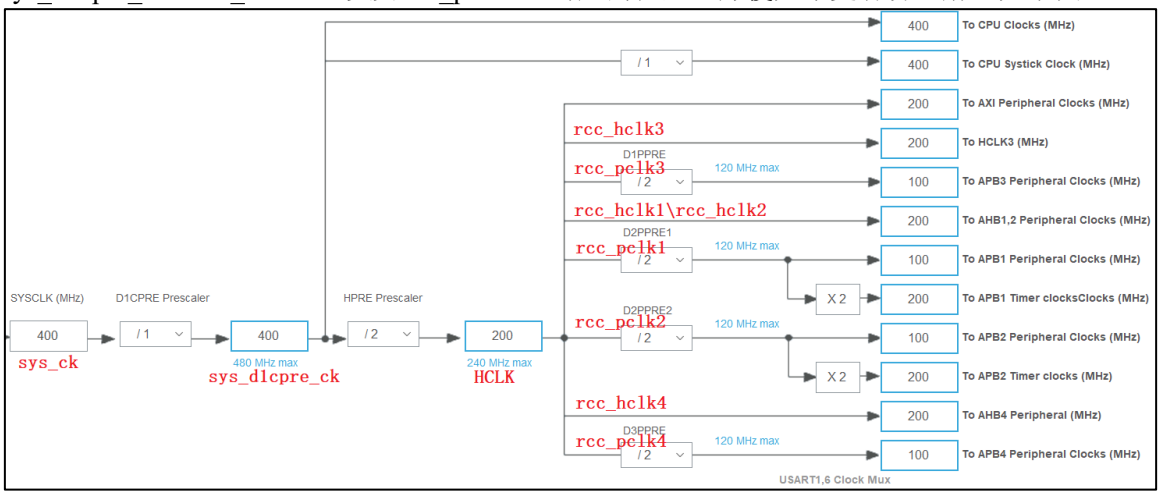


图 11.2.1.1 STM32CubeMX 时钟配置界面标记图

由上图可知道：

rcc_hclk1~4 分别对应的是 AHB1，AHB2，AHB3 和 AHB4 总线的时钟。

rcc_pclk1~4 分别对应的是 APB1，APB2，APB3 和 APB4 总线的时钟。

步骤 4: 使能其他一些时钟和 IO 补偿单元，我们直接看代码：

```
HAL_PWREx_EnableUSBVoltageDetector();       /* 使能 USB 电压电平检测器 */
__HAL_RCC_CSI_ENABLE();                     /* 使能 CSI 时钟 */
__HAL_RCC_SYSCFG_CLK_ENABLE() ;             /* 使能 SYSCFG 时钟 */
HAL_EnableCompensationCell();               /* 使能 IO 补偿单元 */
```

后面的函数就比较简单，都是一些使能函数。其中使能 IO 补偿单元功能，对于要求 IO 翻转速度较快的应用，一般都建议使能 IO 补偿单元功能。使能 CSI 时钟，为 IO 补偿单元提供时钟。

时钟系统配置相关知识就给大家讲解到这里。

11.2.2 STM32H7 时钟使能和配置

上一节我们讲解了时钟系统配置步骤。在配置好时钟系统之后，如果我们要使用某些外设，例如 GPIO，ADC 等，我们还要使能这些外设时钟。这里大家必须注意，如果在使用外设之前没有使能外设时钟，这个外设是不可能正常运行的。STM32 的外设时钟使能是在 RCC 相关寄存器中配置的。因为 RCC 相关寄存器非常多，有兴趣的同学可以直接打开《STM32H7xx 参考手册_V7（英文版）.pdf》 8.7 小节查看所有 RCC 相关寄存器的配置。接下来我们来讲解通过 STM32H7 的 HAL 库使能外设时钟的方法。

在 STM32H7 的 HAL 库中，外设时钟使能操作都是在 RCC 相关固件库文件头文件 stm32h7xx_hal_rcc.h 定义的。大家打开 stm32h7xx_hal_rcc.h 头文件可以看到文件中除了少数几个函数声明之外大部分都是宏定义标识符。外设时钟使能在 HAL 库中都是通过宏定义标识符来实现的。首先，我们来看看 GPIOA 的外设时钟使能宏定义标识符：

```
#define __HAL_RCC_GPIOA_CLK_ENABLE()   do { \
                        __IO uint32_t tmpreg; \
                        SET_BIT(RCC->AHB4ENR, RCC_AHB4ENR_GPIOAEN);\
                        /* Delay after an RCC peripheral clock enabling */ \
                        tmpreg = READ_BIT(RCC->AHB4ENR,RCC_AHB4ENR_GPIOAEN);\
                        UNUSED(tmpreg); \
                      } while(0)
```

这段代码主要是定义了一个宏定义标识符__HAL_RCC_GPIOA_CLK_ENABLE()，它的核心操作是通过下面这行代码实现的：

```
SET_BIT(RCC->AHB4ENR, RCC_AHB4ENR_GPIOAEN);
```

这行代码的作用是，设置寄存器 RCC->AHB4ENR 的相关位为 1，至于是哪个位，是由宏定义标识符 RCC_AHB4ENR_GPIOAEN 的值决定的，而它的值为：

```
#define RCC_AHB4ENR_GPIOAEN_Pos           (0U)
#define RCC_AHB4ENR_GPIOAEN_Msk           (0x1UL << RCC_AHB4ENR_GPIOAEN_Pos)
#define RCC_AHB4ENR_GPIOAEN               RCC_AHB4ENR_GPIOAEN_Msk
```

上面三行代码很容易计算出来 RCC_AHB4ENR_GPIOAEN= 0x00000001，因此上面代码的作用是设置寄存器 RCC->AHB4ENR 寄存器的最低位为 1。我们可以从 STM32H7 的参考手册中搜索 AHB4ENR 寄存器定义，最低位的作用是用来使用 GPIOA 时钟。AHB4ENR 寄存器的位 0 描述如下：

```
位 0 GPIOAEN：GPIOA 外设时钟使能
        由软件置 1 和清零
        0：禁止 IO 端口 A 时钟（复位后的默认值）
        1：使能 IO 端口 A 时钟
```

那么我们只需要在我们的用户程序中调用宏定义标识符就可以实现 GPIOA 时钟使能。使用方法为：

```
__HAL_RCC_GPIOA_CLK_ENABLE();      /* 使能 GPIOA 时钟 */
```

对于其他外设，同样都是在 stm32h7xx_hal_rcc.h 头文件中定义，大家只需要找到相关宏定义标识符即可，这里我们列出几个常用使能外设时钟的宏定义标识符使用方法：

```
__HAL_RCC_DMA1_CLK_ENABLE();       /* 使能 DMA1 时钟 */
__HAL_RCC_USART2_CLK_ENABLE();     /* 使能串口 2 时钟 */
__HAL_RCC_TIM1_CLK_ENABLE();       /* 使能 TIM1 时钟 */
```

我们使用外设的时候需要使能外设时钟，如果我们不需要使用某个外设，同样我们可以禁止某个外设时钟。禁止外设时钟使用方法和使能外设时钟非常类似，同样是头文件中定义的宏定义标识符。我们同样以 GPIOA 为例，宏定义标识符为：

```
#define __HAL_RCC_GPIOA_CLK_DISABLE() (RCC->AHB4ENR) &= ~ (RCC_AHB4ENR_GPIOAEN)
```

同样，宏定义标识符__HAL_RCC_GPIOA_CLK_DISABLE()的作用是设置 RCC->AHB4ENR 寄存器的最低位为 0，也就是禁止 GPIOA 时钟。具体使用方法我们这里就不

做过多讲解，我们这里同样列出几个常用的禁止外设时钟的宏定义标识符使用方法：

```
__HAL_RCC_DMA1_CLK_DISABLE();       /* 禁止 DMA1 时钟 */
__HAL_RCC_USART2_CLK_DISABLE();     /* 禁止串口 2 时钟 */
__HAL_RCC_TIM1_CLK_DISABLE();       /* 禁止 TIM1 时钟 */
```

关于 STM32H7 的外设时钟使能和禁止方法我们就给大家讲解到这里。

第十二章 SYSTEM 文件夹介绍

SYSTEM 文件夹里面的代码由正点原子提供，是 STM32H7xx 系列的底层核心驱动函数，可以用在 STM32H7xx 系列的各个型号上面，方便大家快速构建自己的工程。

SYSTEM 文件夹下包含了 delay、sys、usart 等三个文件夹。分别包含了 delay.c、sys.c、usart.c 及其头文件。通过这 3 个 c 文件，可以快速的给任何一款 STM32H7 构建最基本的框架，使用起来是很方便的。

本章，我们将向大家介绍这些代码，通过这章的学习，大家将了解到这些代码的由来，也希望大家可以灵活使用 SYSTEM 文件夹提供的函数，来快速构建工程，并实际应用到自己的项目中去。

本章将分为如下几个小节：

12.1 deley 文件夹代码介绍

12.2 sys 文件夹代码介绍

12.3 usart 文件夹代码介绍

12.1 deley 文件夹代码介绍

delay 文件夹内包含了 delay.c 和 delay.h 两个文件，这两个文件用来实现系统的延时功能，其中包含 7 个函数：

```
void delay_osschedlock(void);
void delay_osschedunlock(void);
void delay_ostimedly(uint32_t ticks);
void SysTick_Handler(void);
void delay_init(uint16_t sysclk);
void delay_us(uint32_t nus);
void delay_ms(uint16_t nms);
```

前面 4 个函数，仅在支持操作系统（OS）的时候，需要用到，而后面 3 个函数，则不论是否支持 OS 都需要用到。

在介绍这些函数之前，我们先了解一下编程思想：CM7 内核和 CM3/CM4 内核一样，内部都包含了一个 SysTick 定时器，SysTick 是一个 24 位的向下递减的计数定时器，当计数值减到 0 时，将从 RELOAD 寄存器中自动重装载定时初值。只要不把它在 SysTick 控制及状态寄存器中的使能位清除，就永不停息。SysTick 在《STM32H7xx 参考手册_V7（英文版）.pdf》里面基本没有介绍，其详细介绍，请参阅《STM32H7 编程手册.pdf》第 212 页，4.4 节。我们就是利用 STM32 的内部 SysTick 来实现延时的，这样既不占用中断，也不占用系统定时器。

这里我们将介绍的是正点原子提供的最新版本的延时函数，该版本的延时函数支持在任意操作系统（OS）下面使用，它可以和操作系统共用 SysTick 定时器。

这里，我们以 UCOSII 为例，介绍如何实现操作系统和我们的 delay 函数共用 SysTick 定时器。首先，我们简单介绍下 UCOSII 的时钟：ucos 运行需要一个系统时钟节拍（类似“心跳”），而这个节拍是固定的（由 OS_TICKS_PER_SEC 宏定义设置），比如要求 1ms 一次（即可设置：OS_TICKS_PER_SEC=1000），在 STM32 上面，一般是由 SysTick 来提供这个节拍，也就是 SysTick 要设置为 1ms 中断一次，为 ucos 提供时钟节拍，而且这个时钟一般是不能被打断的（否则就不准了）。

因为在 ucos 下 systick 不能再被随意更改，如果我们还想利用 systick 来做 delay_us 或者 delay_ms 的延时，就必须想点办法了，这里我们利用的是**时钟摘取法**。以 delay_us 为例，比如 delay_us（50），在刚进入 delay_us 的时候先计算好这段延时需要等待的 systick 计数次数，这里为 50*400（假设系统时钟为 400Mhz，因为 systick 的频率等于系统时钟频率，那么 systick 每增加 1，就是 1/400us），然后我们就一直统计 systick 的计数变化，直到这个值变化了 50*400，一旦检测到变化达到或者超过这个值，就说明延时 50us 时间到了。这样，我们只是抓取 SysTick 计数器的变化，并不需要修改 SysTick 的任何状态，完全不影响 SysTick 作为 UCOS 时钟节拍的功能，这就是实现 delay 和操作系统共用 SysTick 定时器的原理。

下面我们开始介绍这几个函数。

12.1.1 操作系统支持宏定义及相关函数

当需要 delay_ms 和 delay_us 支持操作系统（OS）的时候，我们需要用到 3 个宏定义和 4 个函数，宏定义及函数代码如下：

```
/*
 *  当 delay_us/delay_ms 需要支持 OS 的时候需要三个与 OS 相关的宏定义和函数来支持
 *  首先是 3 个宏定义:
 *      delay_osrunning     :用于表示 OS 当前是否正在运行,以决定是否可以使用相关函数
 *      delay_ostickspersec:用于表示 OS 设定的时钟节拍,delay_init
 *                          将根据这个参数来初始化 systick
 *      delay_osintnesting :用于表示 OS 中断嵌套级别,因为中断里面不可以调度,
 *                          delay_ms 使用该参数来决定如何运行
 *  然后是 3 个函数:
 *      delay_osschedlock  :用于锁定 OS 任务调度,禁止调度
 *      delay_osschedunlock:用于解锁 OS 任务调度,重新开启调度
 *      delay_ostimedly     :用于 OS 延时,可以引起任务调度.
 *
 *  本例程仅作 UCOSII 的支持,其他 OS,请自行参考着移植
 */
/* 支持 UCOSII */
#define delay_osrunning    OSRunning    /* OS 是否运行标记,0,不运行;1,在运行 */
#define delay_ostickspersec OS_TICKS_PER_SEC   /* OS 时钟节拍,即每秒调度次数 */
#define delay_osintnesting  OSIntNesting       /* 中断嵌套级别,即中断嵌套次数 */

/**
 * @brief      us 级延时时,关闭任务调度(防止打断 us 级延迟)
 * @param      无
 * @retval     无
 */
void delay_osschedlock(void)
{
    OSSchedLock();                  /* UCOSII 的方式,禁止调度，防止打断 us 延时 */
}

/**
 * @brief      us 级延时时,恢复任务调度
 * @param      无
 * @retval     无
 */
void delay_osschedunlock(void)
{
    OSSchedUnlock();                /* UCOSII 的方式,恢复调度 */
}

/**
 * @brief      us 级延时时,恢复任务调度
 * @param      ticks: 延时的节拍数
 * @retval     无
 */
void delay_ostimedly(uint32_t ticks)
{
    OSTimeDly(ticks);   /* UCOSII 延时 */
}

/**
 * @brief      systick 中断服务函数,使用 OS 时用到
 * @param      ticks: 延时的节拍数
 * @retval     无
 */
```

```
void SysTick_Handler(void)
{
    if (delay_osrunning == OS_TRUE)     /* OS 开始跑了,才执行正常的调度处理 */
    {
        OS_CPU_SysTickHandler();        /* 调用 uC/OS-II 的 SysTick 中断服务函数 */
    }
    HAL_IncTick();
}
#endif
```

以上代码，仅支持 UCOSII，不过，对于其他 OS 的支持，也只需要对以上代码进行简单修改即可实现。

支持 OS 需要用到的三个宏定义（以 UCOSII 为例）即：

```
#define delay_osrunning    OSRunning          /* OS 是否运行标记,0,不运行;1,在运行 */
#define delay_ostickspersec OS_TICKS_PER_SEC  /* OS 时钟节拍,即每秒调度次数 */
#define delay_osintnesting  OSIntNesting      /* 中断嵌套级别,即中断嵌套次数 */
```

宏定义：delay_osrunning，用于标记 OS 是否正在运行，当 OS 已经开始运行时，该宏定义值为 1，当 OS 还未运行时，该宏定义值为 0。

宏定义：delay_ ostickspersec，用于表示 OS 的时钟节拍，即 OS 每秒钟任务调度次数。

宏定义：delay_ osintnesting，用于表示 OS 中断嵌套级别，即中断嵌套次数，每进入一个中断，该值加 1，每退出一个中断，该值减 1。

支持 OS 需要用到的 4 个函数，即：

函数：delay_osschedlock，用于 delay_us 延时，作用是禁止 OS 进行调度，以防打断 us 级延时，导致延时时间不准。

函数：delay_osschedunlock，同样用于 delay_us 延时，作用是在延时结束后恢复 OS 的调度，继续正常的 OS 任务调度。

函数：delay_ostimedly，则是调用 OS 自带的延时函数，实现延时。该函数的参数为时钟节拍数。

函数：SysTick_Handler，则是 systick 的中断服务函数，该函数为 OS 提供时钟节拍，同时可以引起任务调度。

以上就是 delay_ms 和 delay_us 支持操作系统时，需要实现的 3 个宏定义和 4 个函数。

12.1.2 delay_init 函数

该函数用来初始化重要参数：g_fac_us；同时把 SysTick 的时钟源选择为外部时钟，如果需要支持操作系统（OS），只需要在 sys.h 里面，设置 SYS_SUPPORT_OS 宏的值为 1 即可，然后，该函数会根据 delay_ostickspersec 宏的设置，来配置 SysTick 的中断时间，并开启 SysTick 中断。具体代码如下：

```
/**
 * @brief       初始化延迟函数
 * @param       sysclk: 系统时钟频率, 即 CPU 频率(HCLK), 等于系统主频, 单位 Mhz
 * @retval      无
 */
void delay_init(uint16_t sysclk)
{
#if SYS_SUPPORT_OS                  /* 如果需要支持 OS */
    uint32_t reload;
#endif
    g_fac_us = sysclk;              /* 不论是否使用 OS,g_fac_us 都需要使用 */
#if SYS_SUPPORT_OS                  /* 如果需要支持 OS. */
    reload = sysclk;                /* 每秒钟的计数次数 单位为 M */
    reload *= 1000000 / delay_ostickspersec;
    /* 根据 delay_ostickspersec 设定溢出时间, reload 为 24 位寄存器,最大值:16777216*/
    g_fac_ms = 1000 / delay_ostickspersec;      /* 代表 OS 可以延时的最少单位 */
    SysTick->CTRL |= 1 << 1;                    /* 开启 SYSTICK 中断 */
    SysTick->LOAD = reload;         /* 每 1/delay_ostickspersec 秒中断一次 */
    SysTick->CTRL |= 1 << 0;                    /* 开启 SYSTICK */
```

```
#endif
}
```

可以看到，delay_init 函数使用了条件编译，来选择不同的初始化过程，如果不使用 OS 的时候，只是设置一下 SysTick 的时钟源以及确定 fac_us 值。而如果使用 OS 的时候，则会进行一些不同的配置，这里的条件编译是根据 SYS_SUPPORT_OS 这个宏来确定的，该宏在 sys.h 里面定义。

SysTick 是 MDK 定义了的一个结构体（在 core_m7.h 里面），里面包含 CTRL、LOAD、VAL、CALIB 等 4 个寄存器。

SysTick->CTRL 的各位定义如图 12.1.2.1 所示：

位段	名称	类型	复位值	描述
16	COUNTFLAG	R	0	如果在上次读取本寄存器后，SysTick 已经数到了 0，则该位为 1。如果读取该位，该位将自动清零
2	CLKSOURCE	R/W	0	0=外部时钟源(STCLK) 1=内核时钟(FCLK)
1	TICKINT	R/W	0	1=SysTick 倒数到 0 时产生 SysTick 异常请求 0=数到 0 时无动作
0	ENABLE	R/W	0	SysTick 定时器的使能位

图 12.1.2.1 SysTick->CTRL 寄存器各位定义

SysTick-> LOAD 的定义如图 12.1.2.2 所示：

位段	名称	类型	复位值	描述
23:0	RELOAD	R/W	0	当倒数至零时，将被重装载的值

图 12.1.2.2 SysTick->LOAD 寄存器各位定义

SysTick-> VAL 的定义如图 12.1.2.3 所示：

位段	名称	类型	复位值	描述
23:0	CURRENT	R/Wc	0	读取时返回当前倒计数的值，写它则使之清零，同时还会清除在 SysTick 控制及状态寄存器中的 COUNTFLAG 标志

图 12.1.2.3 SysTick->VAL 寄存器各位定义

SysTick-> CALIB 不常用，在这里我们也用不到，故不介绍了。

HAL_SYSTICK_CLKSourceConfig(SYSTICK_CLKSOURCE_HCLK);这句代码把 SysTick 的时钟选择为内核时钟，这里需要注意的是：SysTick 的时钟源自 HCLK，假设我们外部晶振为 25M，然后倍频到 400MHZ，那么 SysTick 的时钟即为 400Mhz，也就是 SysTick 的计数器 VAL 每减 1，就代表时间过了 1/400us。

在不使用 OS 的时候：g_fac_us，为 us 延时的基数，也就是延时 1us，Systick 定时器需要走过的时钟周期数。

当使用 OS 的时候，g_fac_us，还是 us 延时的基数，不过这个值不会被写到 SysTick->LOAD 寄存器来实现延时，而是通过时钟摘取的办法实现的（前面已经介绍了）。

12.1.3 delay_us 函数

该函数用来延时指定的 us，其参数 nus 为要延时的微秒数。实现函数如下：

```
/**
 * @brief       延时 nus
 * @note        无论是否使用 OS, 都是用时钟摘取法来做 us 延时
 * @param       nus: 要延时的 us 数
 * @note        nus 取值范围: 0~(2^32 / fac_us) (fac_us 一般等于系统主频, 自行套入计算)
 * @retval      无
 */
```

```
void delay_us(uint32_t nus)
{
    uint32_t ticks;
    uint32_t told, tnow, tcnt = 0;
    uint32_t reload = SysTick->LOAD;          /* LOAD 的值 */
    ticks = nus * g_fac_us;                   /* 需要的节拍数 */

#if SYS_SUPPORT_OS                            /* 如果需要支持 OS */
    delay_osschedlock();                      /* 锁定 OS 的任务调度器 */
#endif

    told = SysTick->VAL;                      /* 刚进入时的计数器值 */
    while (1)
    {
        tnow = SysTick->VAL;
        if (tnow != told)
        {
            if (tnow < told)
            {
                tcnt += told - tnow;
               /* 这里注意一下 SYSTICK 是一个递减的计数器就可以了 */
            }
            else
            {
                tcnt += reload - tnow + told;
            }
            told = tnow;
            if (tcnt >= ticks)
            {
                break;                        /* 时间超过/等于要延迟的时间,则退出 */
            }
        }
    }

#if SYS_SUPPORT_OS                            /* 如果需要支持 OS */
    delay_osschedunlock();                    /* 恢复 OS 的任务调度器 */
#endif
}
```

这里就是利用了我们前面提到的时钟摘取法，ticks 是延时 nus 需要等待的 SysTick 计数次数（也就是延时时间），told 用于记录最近一次的 SysTick->VAL 值，然后 tnow 则是当前的 SysTick->VAL 值，通过他们的对比累加，实现 SysTick 计数次数的统计，统计值存放在 tcnt 里面，然后通过对比 tcnt 和 ticks，来判断延时是否到达，从而达到不修改 SysTick 实现 nus 的延时。对于使用 OS 的时候，delay_us 的实现函数和不使用 OS 的时候方法类似，都是使用的时钟摘取法，只不过使用 delay_osschedlock 和 delay_osschedunlock 两个函数，用于调度上锁和解锁，这是为了防止 OS 在 delay_us 的时候打断延时，可能导致的延时不准，所以我们利用这两个函数来实现免打断，从而保证延时精度。

上面的 delay_osschedlock 和 delay_osschedunlock 是 OS 提供的两个函数，用于调度上锁和解锁，这里为了防止 OS 在 delay_us 的时候打断延时，可能导致的延时不准，所以我们利用这两个函数来实现免打断，从而保证延时精度！同时，此时的 delay_us，可以实现最长 2^32/g_fac_us。

12.1.4 delay_ms 函数

该函数是用来延时指定的 ms 的，其参数 nms 为要延时的毫秒数。实现函数如下：

```
/**
 * @brief     延时 nms
 * @param     nms: 要延时的 ms 数 (0< nms <= (2^32 / fac_us / 1000))
              (fac_us 一般等于系统主频, 自行套入计算)
```

```
 * @retval     无
 */
void delay_ms(uint16_t nms)
{
#if SYS_SUPPORT_OS  /* 如果需要支持 OS, 则根据情况调用 os 延时以释放 CPU */
    if (delay_osrunning && delay_osintnesting == 0)
    /* 如果 OS 已经在跑了,并且不是在中断里面(中断里面不能任务调度) */
    {
        if (nms >= g_fac_ms)    /* 延时的时间大于 OS 的最少时间周期 */
        {
            delay_ostimedly(nms / g_fac_ms);            /* OS 延时 */
        }

        nms %= g_fac_ms;       /* OS 已经无法提供这么小的延时了,采用普通方式延时 */
    }
#endif

    delay_us((uint32_t)(nms * 1000));                   /* 普通方式延时 */
}
```

该函数其实就是多次调用 delay_us 函数，来实现毫秒级延时的。我们做了一些处理，使得调用 delay_us 函数的次数减少，这样时间会更加精准。

该函数中，delay_osrunning 是 OS 正在运行的标志，delay_osintnesting 则是 OS 中断嵌套次数，必须 delay_osrunning 为真，且 delay_osintnesting 为 0 的时候，才可以调用 OS 自带的延时函数进行延时（可以进行任务调度），delay_ostimedly 函数就是利用 OS 自带的延时函数，实现任务级延时的，其参数代表延时的时钟节拍数（假设 delay_ostickspersec=200，那么 delay_ostimedly (1)，就代表延时 5ms）。

当 OS 还未运行的时候，我们的 delay_ms 就是直接由 delay_us 实现的，OS 下的 delay_us 可以实现很长的延时（达到 10 秒）而不溢出！，所以放心的使用 delay_us 来实现 delay_ms，不过由于 delay_us 的时候，任务调度被上锁了，所以还是建议不要用 delay_us 来延时很长的时间，否则影响整个系统的性能。

当 OS 运行的时候，我们的 delay_ms 函数将先判断延时时长是否大于等于 1 个 OS 时钟节拍（fac_ms）,当大于这个值的时候，我们就通过调用 OS 的延时函数来实现（此时任务可以调度），不足 1 个时钟节拍的时候，直接调用 delay_us 函数实现（此时任务无法调度）。

12.1.5 HAL 库延时函数 HAL_Delay

前面我们在 7.4.2 章节介绍 stm32h7xx_hal.c 文件时，已经讲解过 Systick 实现延时相关函数。实际上，HAL 库提供的延时函数，只能实现简单的毫秒级别延时，没有实现 us 级别延时。我看看 HAL 库的 HAL_Delay 函数原定义：

```
/* HAL 库的延时函数,默认延时单位 ms */
__weak void HAL_Delay(uint32_t Delay)
{
  uint32_t tickstart = HAL_GetTick();
  uint32_t wait = Delay;

  /* Add a freq to guarantee minimum wait */
  if (wait < HAL_MAX_DELAY)
  {
    wait += (uint32_t)(uwTickFreq);
  }

  while ((HAL_GetTick() - tickstart) < wait)
  {
  }
}
```

HAL 库实现延时功能非常简单，首先定义了一个 32 位全局变量 uwTick，在 Systick 中断服务函数 SysTick_Handler 中通过调用 HAL_IncTick 实现 uwTick 值不断增加，也就是每隔 1ms 增加 uwTickFreq，而 uwTickFreq 默认是 1。而 HAL_Delay 函数在进入函数之后先记录当前

uwTick 的值，然后不断在循环中读取 uwTick 当前值，进行减运算，得出的就是延时的毫秒数，整个逻辑非常简单也非常清晰。

但是，HAL 库的延时函数有一个局限性，在中断服务函数中使用 HAL_Delay 会引起混乱(虽然一般禁止在中断中使用延时函数)，因为它是通过中断方式实现，而 Systick 的中断优先级是最低的，所以在中断中运行 HAL_Delay 会导致延时出现严重误差。所以一般情况下，推荐大家使用正点原子提供的延时函数库。

HAL 库的 ms 级别的延时函数__weak void HAL_Delay(uint32_t Delay)；它是弱定义函数，所以用户可以自己重新定义该函数。例如：我们在 deley.c 文件可以这样重新定义该函数：

```
/**
 * @brief       HAL 库内部函数用到的延时
 * @note        HAL 库的延时默认用 Systick，如果我们没有开 Systick 的中断会导致调用
                这个延时后无法退出
 * @param       Delay : 要延时的毫秒数
 * @retval      无
 */ void HAL_Delay(uint32_t Delay)
{
     delay_ms(Delay);
}
```

12.2 sys 文件夹代码介绍

sys 文件夹内包含了 sys.c 和 sys.h 两个文件，主要实现下面的几个函数，以及一些汇编函数。

```
/* 设置中断向量表偏移量 */
void sys_nvic_set_vector_table(uint32_t baseaddr, uint32_t offset);
void sys_cache_enable(void);                    /* 使能 STM32H7 的 L1-Cahce */
uint8_t sys_stm32_clock_init(uint32_t plln, uint32_t pllm, uint32_t pllp,
                             uint32_t pllq);          /* 配置系统时钟 */
/* 以下为汇编函数 */
void sys_wfi_set(void);                 /* 执行 WFI 指令 */
void sys_intx_disable(void);            /* 关闭所有中断 */
void sys_intx_enable(void);             /* 开启所有中断 */
void sys_msr_msp(uint32_t addr);        /* 设置栈顶地址 */
```

sys_nvic_set_vector_table 函数用于设置中断向量表偏移地址。sys_stm32_clock_ini 函数的讲解请参考 11.2.1 小节 STM32H7 时钟系统配置章节内容。

接下来我们重点看一下 sys_cache_enable 函数。

12.2.1 Cache 使能函数

STM32H7 自带了指令 Cache（I Cache）和数据 Cache（D Cache），使用 I/D Cache 可以缓存指令/数据，提高 CPU 访问指令/数据的速度，从而大大提高 MCU 的性能。不过，MCU 在复位后，I/D Cache 默认都是关闭的，为了提高性能，我们需要开启 I/D Cache，在 sys.c 里面，我们提供了如下函数：

```
/**
 * @brief       使能 STM32H7 的 L1-Cache, 同时开启 D cache 的强制透写
 * @param       无
 * @retval      无
 */
void sys_cache_enable(void)
{
    SCB_EnableICache();    /* 使能 I-Cache,函数在 core_cm7.h 里面定义 */
    SCB_EnableDCache();    /* 使能 D-Cache,函数在 core_cm7.h 里面定义 */
    SCB->CACR |= 1 << 2; /* 强制 D-Cache 透写,如不开启透写,实际使用中可能遇到各种问题 */
}
```

该函数，通过调用 SCB_EnableICache 和 SCB_EnableDCache 这两个函数来使能 I Cache 和 D Cache。不过，在使能 D Cache 之后，SRAM 里面的数据有可能会被缓存在 Cache 里面，此时如果有 DMA 之类的外设访问这个 SRAM 里面的数据，就有可能和 Cache 里面数据不同步，

导致数据出错，为了防止这种问题，保证数据的一致性，我们设置了 D Cache 的强制透写功能（Write Through），这样 CPU 每次操作 Cache 里面的数据，同时也会更新到 SRAM 里面，保证 D Cache 和 SRAM 里面数据一致。关于 Cache 的详细介绍，请参考《STM32F7 Cache Oveview》和《Level 1 cache on STM32F7 Series》（见光盘：8，STM32 参考资料 文件夹）。注意：F7 和 H7 这部分知识是通用的，所以参考 F7 的这两份文档即可，H7 的这两个资料暂时没出来。

这里 SCB_EnableICache 和 SCB_EnableDCache 这两个函数，是在 core_cm7.h 里面定义的，我们直接调用即可，另外，core_cm7.h 里面还提供了以下五个常用函数：

1，SCB_DisableICache 函数，用于关闭 I Cache。

2，SCB_DisableDCache 函数，用于关闭 D Cache。

3，SCB_InvalidateDCache 函数，用于丢弃 D Cache 当前数据，重新从 SRAM 获取数据。

4，SCB_CleanDCache 函数，用于将 D Cache 数据回写到 SRAM 里面，同步数据。

5，SCB_CleanInvalidateDCache 函数，用于回写数据到 SRAM，并重新获取 D Cache 数据。

在 Cache_Enable 函数里面，我们直接开启了 D Cache 的透写模式，这样带来的好处就是可以保证 D Cache 和 SRAM 里面数据的一致性，坏处就是会损失一定的性能（每次都要回写数据），如果大家想自己控制 D Cache 数据的回写，以获得最佳性能，则可以关闭 D Cache 透写模式，并在适当的时候，调用 SCB_CleanDCache、SCB_InvalidateDCache 和 SCB_CleanInvalidateDCache 等函数，这对程序员的要求非常高，程序员必须清楚什么时候该回写，什么时候该更新 D Cache！如果能力不够，还是建议开启 D Cache 的透写，以免引起各种莫名其妙的问题。

12.3 usart 文件夹代码介绍

该文件夹下面有 usart.c 和 usarts.h 两个文件。在我们的工程使用串口 1 和串口调试助手来实现调试功能，可以把单片机的信息通过串口助手显示到电脑屏幕。串口相关知识，**我们将在第十六章讲解串口实验的时候给大家详细讲解。**本节我们只给大家讲解比较独立的 printf 函数支持相关的知识。

12.3.1 printf 函数支持

在我们学习 C 语言时，可以通过 printf 函数把需要的参数显示到屏幕上，可以做一些简单的调试信息，但对于单片机来说，如果想实现类似的功能来用 printf 辅助调试的话，是否有办法呢？有，这就是这一节要讲的内容。

标准库下的 printf 为调试属性的函数，如果直接使用，会使单片机进入半主机模式（semihosting），这是一种调试模式，直接下载代码后出现程序无法运行，但是在 Debug 模式下程序可能正常工作的情况。半主机是 ARM 目标的一种机制，用于将输入/输出请求从应用程序代码通信到运行调试器的主机。例如，此机制可用于允许 C 库中的函数（如 printf()和 scanf()）使用主机的屏幕和键盘，而不是在目标系统上设置屏幕和键盘。这很有用，因为开发硬件通常不具有最终系统的所有输入和输出设备，如屏幕、键盘等。半主机是通过一组定义好的软件指令（如 SVC）SVC 指令（以前称为 SWI 指令）来实现的，这些指令通过程序控制生成异常。应用程序调用相应的半主机调用，然后调试代理处理该异常。调试代理（这里的调试代理是仿真器）提供与主机之间的必需通信。也就是说使用半主机模式必须使用仿真器调试。

如果想在独立环境下运行调试功能的函数，我们这里是 printf ，printf 对字符 ch 处理后写入文件 f，最后使用 fputc 将文件 f 输出到显示设备。对于 PC 端的设备，fputc 通过复杂的源码，最终把字符显示到屏幕上。那我们需要做的，就是把 printf 调用的 fputc 函数重新实现，重定向 fputc 的输出，同时避免进入半主模式。

要避免半主机模式，现在主要有两种方式：一是使用 MicroLib，即微库；另一种方法是确保 ARM 应用程序中没有链接 MicroLib 的半主机相关函数，我们要取消 ARM 的半主机工作模式，这可以通过代码实现。

先说微库，ARM 的 C 微库 MicroLib 是为嵌入式设备开发的一套类似于标准 C 接口函数的精简代码库，用于替代默认 C 库，是专门针对专业嵌入式应用开发而设计的，特别适合那些对存储空间有特别要求的嵌入式应用程序，这些程序一般不在操作系统下运行。使用微库编写程序要注意其与默认 C 库之间存在的一些差异，如 main()函数不能声明带参数，也无须返回；不

支持 stdio，除了无缓冲的 stdin、stdout 和 syderr; 微库不支持操作系统函数；微库不支持可选的单或两区存储模式；微库只提供分离的堆和栈两区存储模式等等，它裁减了很多函数，而且还有很多东西不支持。如果原来用标准库可以跑，选择 MicroLib 后却突然不行了，是很常见的。与标准的 C 库不一样，微库重新实现了 printf，使用微库的情况下就不会进入半主机模式了。Keil 下使用微库的方法很简单，在“Target”下勾选“Use MicroLib”即可。

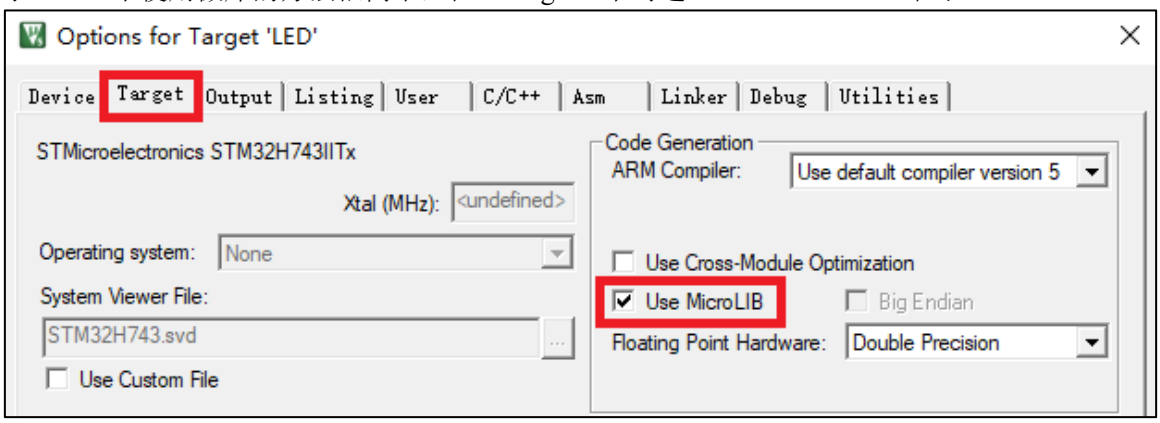


图 12.3.1.1 MDK 工程下使用微库的方法

在 keil5 中，不管是否使用半主机模式，使用 printf,scanf,fopen,fread 等都需要自己填充底层函数，以 printf 为例，需要补充定义 fputc，启用微库后，在我们初始化和使能串口 1 之后，我们只需要重新实现 fputc 的功能即可将每个传给 fputc 函数的字符 ch 重定向到串口 1，如果这时接上串口调试助手的话，可以看到串口的数据。实现的代码如下：

```
/* 重定义 fputc 函数, printf 函数最终会通过调用 fputc 输出字符串到串口 */
int fputc(int ch, FILE *f)
{
    while ((USART_UX->ISR & 0X40) == 0);    /* 等待上一个字符发送完成 */

    USART_UX->TDR = (uint8_t)ch;            /* 将要发送的字符 ch 写入到 DR 寄存器 */
    return ch;
}
```

上面说到了微库的一些限制，使用时注意某些函数与标准库的区别就不会影响到我们代码的正常功能。如果不想使用微库，那就要用到我们提到的第二种方法：取消 ARM 的半主机工作模式；只需在代码中添加不使用半主机的声明即可，对于 AC5 和 AC6 编译器版本，声明半主机的语法不同，为了同时兼容这两种语法，我们在利用编译器自带的宏__ARMCC_VERSION 判定编译器版本，并根据版本不同选择不同的语法声明不使用半主机模式，具体代码如下：

```
#if (__ARMCC_VERSION >= 6010050)            /* 使用 AC6 编译器时 */
__asm(".global __use_no_semihosting\n\t"); /* 声明不使用半主机模式 */
__asm(".global __ARM_use_no_argv \n\t");    /* AC6 下需要声明 main 函数为无参数格式，否
                                               则部分例程可能出现半主机模式 */

#else
/* 使用 AC5 编译器时, 要在这里定义__FILE 和 不使用半主机模式 */
#pragma import(__use_no_semihosting)

/* 解决 HAL 库使用时, 某些情况可能报错的 bug */
struct __FILE
{
    int handle;
    /* Whatever you require here. If the only file you are using is */
    /* standard output using printf() for debugging, no file handling */
    /* is required. */
};

#endif
```

使用的上面的代码，Keil 的编译器就不会把标准库的这部分函数链接到我们的代码里。如

果用到原来半主机模式下的调试函数，需要重新实现它的一些依赖函数接口，对于 printf 函数需要实现的接口，我们的代码中将它们实现如下：

```
/* 不使用半主机模式，至少需要重定义_ttywrch\_sys_exit\_sys_command_string 函数,
    以同时兼容 AC6 和 AC5 模式 */
int _ttywrch(int ch)
{
    ch = ch;
    return ch;
}
/* 定义_sys_exit()以避免使用半主机模式 */
void _sys_exit(int x)
{
    x = x;
}
char *_sys_command_string(char *cmd, int len)
{
    return NULL;
}
/* FILE 在 stdio.h 里面定义. */
FILE __stdout;
```

fputc 的重定向和之前一样，重定向到串口 1 即可，如果硬件资源允许，读者有特殊需求，也可以重定向到 LCD 或者其它串口。

```
   /* 重定义 fputc 函数, printf 函数最终会通过调用 fputc 输出字符串到串口 */
   int fputc(int ch, FILE *f)
   {
       while ((USART_UX->SR & 0X40) == 0); /* 等待上一个字符发送完成 */
       USART_UX->DR = (uint8_t)ch;         /* 将要发送的字符 ch 写入到 DR 寄存器 */
       return ch;
   }
```

第二篇 入门篇

功夫不负有心人，相信学习至此你已经掌握了基础篇介绍的知识。我们希望通过前面的章节你已经掌握了 STM32 开发的工具和方法。本篇我们将和大家一起来学习 STM32 的一些基础外设，这些外设实际项目中经常会用到，希望大家认真学习和掌握，以便将来更好、更快的完成实际项目开发。

本篇将采取一章一实例的方式，介绍 STM32 常用外设的使用，通过本篇的学习，我们将带领大家进入 STM32 的精彩世界。

第十三章 跑马灯实验

本章将通过一个经典的跑马灯程序，带大家开启 STM32H743 之旅。通过本章的学习，我们将了解到 STM32H743 的 IO 口作为输出使用的方法。我们将通过代码控制开发板上的 LED 灯：DS0 和 DS1 交替闪烁，实现类似跑马灯的效果。

本章分为如下 4 个小节：

13.1 STM32H7 GPIO 简介
13.2 硬件设计
13.3 程序设计
13.4 下载验证

13.1 STM32H7 GPIO 简介

GPIO 是控制或者采集外部器件的信息的外设，既负责输入输出。它按组分配，每组 16 个 IO 口，组数视芯片而定。STM32H743IIT6 芯片是 176 脚的芯片，它的 IO 口总共有 140 个，底板上总共引出了 110 个 IO，剩下的 30 个 IO 主要用在在晶振、SDRAM、RGBLCD 等常用外设上面，不太适合再引出来做其他用。IO 分组为 8 组，分别是 GPIOA-GPIOH。这里 GPIOA-GPIOH，16*8=128 个 IO 口。

这里重点说一下 STM32H743 的 IO 电平兼容性问题，STM32H743 的绝大部分 IO 口，都兼容 5V，至于到底哪些是兼容 5V 的，请看 STM32H743IIT6.pdf 的数据手册（注意是数据手册，不是中文参考手册），见表：Table 8 STM32H743xI/G pin/ball definition，凡是有 FT 标志的，都是兼容 5V 电平的 IO 口，可以直接接 5V 的外设（注意：如果引脚设置的是模拟输入模式，则不能接 5V！），凡是不是 FT 标志的，大家就不要接 5V 了，可能烧坏 MCU。对 Table 8 符号的描述如下：

Table 8. Legend/abbreviations used in the pinout table

Name	Abbreviation	Definition
Pin name	Unless otherwise specified in brackets below the pin name, the pin function during and after reset is the same as the actual pin name	
Pin type	S	Supply pin
	I	Input only pin
	I/O	Input / output pin
	ANA	Analog-only Input
I/O structure	FT	5 V tolerant I/O
	TT	3.3 V tolerant I/O
	B	Dedicated BOOT0 pin
	RST	Bidirectional reset pin with embedded weak pull-up resistor
	Option for TT and FT I/Os	
	_f	I2C FM+ option
	_a	analog option (supplied by V_{DDA})
	_u	USB option (supplied by $V_{DD33USB}$)
	_h	High-speed low-voltage I/O
Notes	Unless otherwise specified by a note, all I/Os are set as floating inputs during and after reset.	

表 13.1.1 IO 口属性缩写符号表

13.1.1 GPIO 功能模式

GPIO 有八种工作模式，分别是：

1、输入浮空
2、输入上拉
3、输入下拉
4、模拟
5、开漏输出
6、推挽输出
7、开漏式复用功能
8、推挽式复用功能

13.1.2 GPIO 基本结构分析

我们知道了 GPIO 有八种工作模式，具体这些模式是怎么实现的？下面我们通过 GPIO 的基本结构图来分别进行详细分析，先看看总的框图，如图 13.1.2.1 所示。

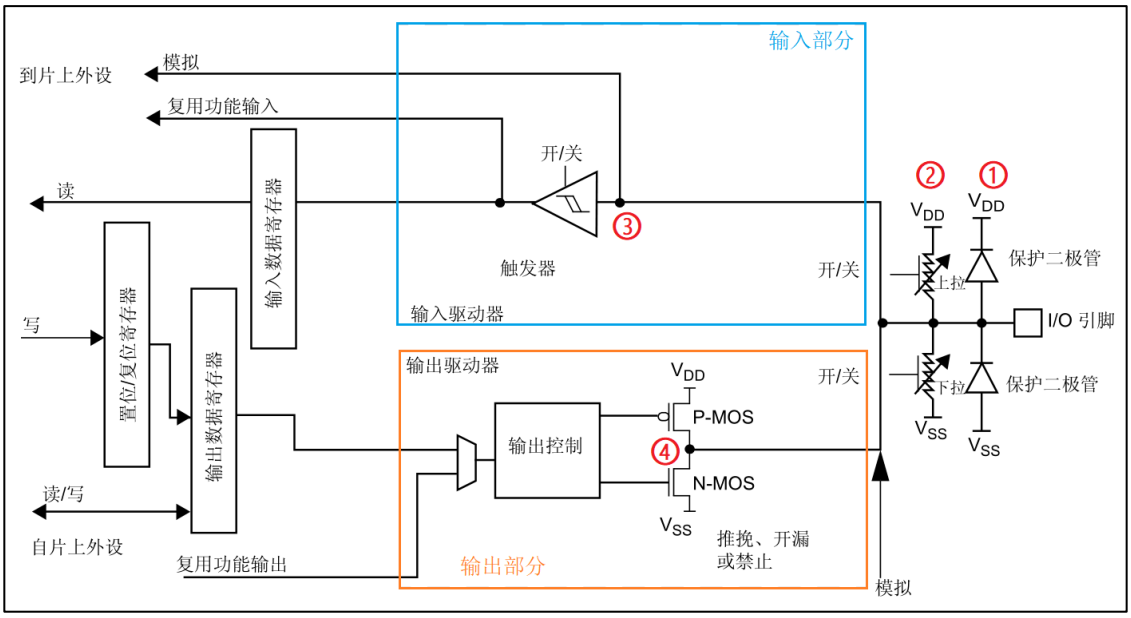


图 13.1.2.1 GPIO 的基本结构图

如上图所示，可以看到右边只有 I/O 引脚，这个 I/O 引脚就是我们可以看到的芯片实物的引脚，其他部分都是 GPIO 的内部结构。

① 保护二极管

保护二极管共有两个，用于保护引脚外部过高或过低的电压输入。当引脚输入电压高于 VDD 时，上面的二极管导通，当引脚输入电压低于 VSS 时，下面的二极管导通，从而使输入芯片内部的电压处于比较稳定的值。虽然有二极管的保护，但这样的保护却很有限，大电压大电流的接入很容易烧坏芯片。所以在实际的设计中我们要考虑设计引脚的保护电路。

② 上拉、下拉电阻

它们阻值大概在 30~50K 欧之间，可以通过上、下两个对应的开关控制，这两个开关由寄存器控制。当引脚外部的器件没有干扰引脚的电压时，即没有外部的上、下拉电压，引脚的电平由引脚内部上、下拉决定，开启内部上拉电阻工作，引脚电平为高，开启内部下拉电阻工作，则引脚电平为低。同样，如果内部上、下拉电阻都不开启，这种情况就是我们所说的浮空模式。浮空模式下，引脚的电平是不可确定的。引脚的电平可以由外部的上、下拉电平决定。需要注意的是，STM32 的内部上拉是一种“弱上拉”，这样的上拉电流很弱，如果有要求大电流还是得外部上拉。

③ 施密特触发器

对于标准施密特触发器，当输入电压高于正向阈值电压，输出为高；当输入电压低于负向

阈值电压，输出为低；当输入在正负向阈值电压之间，输出不改变，也就是说输出由高电准位翻转为低电准位，或是由低电准位翻转为高电准位对应的阈值电压是不同的。只有当输入电压发生足够的变化时，输出才会变化，因此将这种元件命名为触发器。这种双阈值动作被称为迟滞现象，表明施密特触发器有记忆性。从本质上来说，施密特触发器是一种双稳态多谐振荡器。

施密特触发器可作为波形整形电路，能将模拟信号波形整形为数字电路能够处理的方波波形，而且由于施密特触发器具有滞回特性，所以可用于抗干扰，其应用包括在开回路配置中用于抗扰，以及在闭回路正回授/负回授配置中用于实现多谐振荡器。

下面看看比较器跟施密特触发器的作用的比较，就清楚的知道施密特触发器对外部输入信号具有一定抗干扰能力，如图 13.1.2.2 所示。

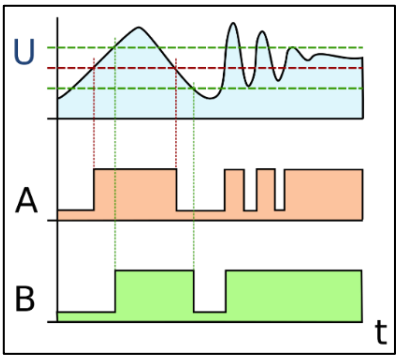


图 13.1.2.2 比较器的（A）和施密特触发器（B）作用比较

④ P-MOS 管和 N-MOS 管

这个结构控制 GPIO 的开漏输出和推挽输出两种模式。开漏输出：输出端相当于三极管的集电极，要得到高电平状态需要上拉电阻才行。推挽输出：这两只对称的 MOS 管每次只有一只导通，所以导通损耗小、效率高。输出既可以向负载灌电流，也可以从负载拉电流。推拉式输出既能提高电路的负载能力，又能提高开关速度。

上面我们对 GPIO 的基本结构图中的关键器件做了介绍，下面分别介绍 GPIO 八种工作模式对应结构图的工作情况。

- **1、输入浮空**

输入浮空模式：上拉/下拉电阻为断开状态，施密特触发器打开，输出被禁止。输入浮空模式下，IO 口的电平完全是由外部电路决定。如果 IO 引脚没有连接其他的设备，那么检测其输入电平是不确定的。该模式可以用于按键检测，RX1 等。

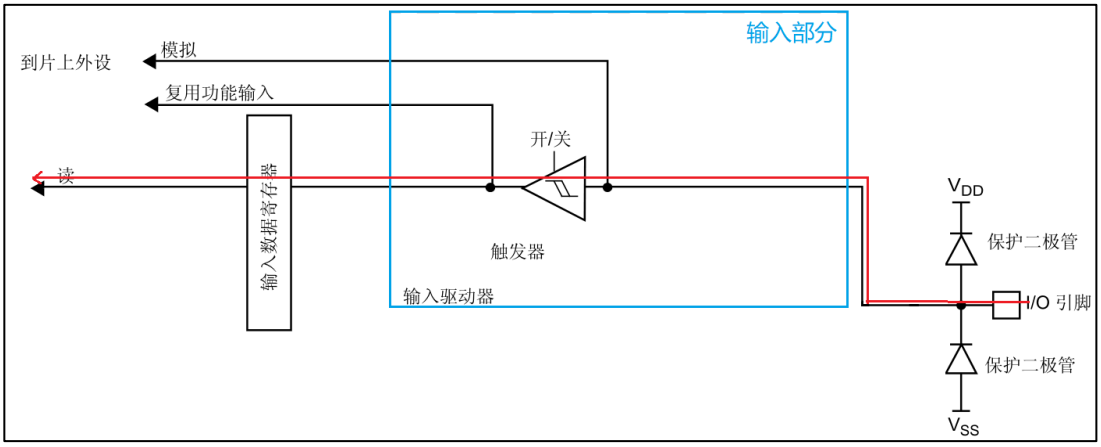


图 13.1.2.3 输入浮空模式

- **2、输入上拉**

输入上拉模式：上拉电阻导通，施密特触发器打开，输出被禁止。在需要外部上拉电阻的时候，可以使用内部上拉电阻，这样可以节省一个外部电阻，但是内部上拉电阻的阻值较大，所以只是“弱上拉”，不适合做电流型驱动。

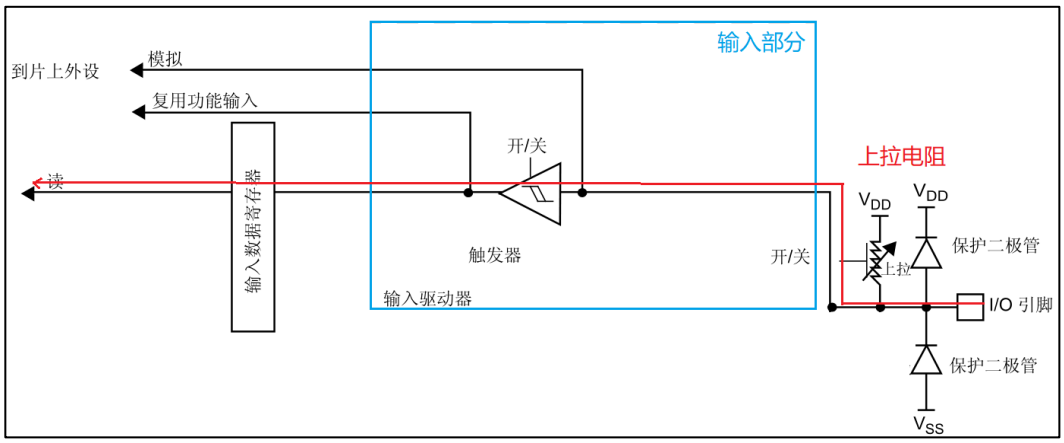


图 13.1.2.4 输入上拉模式

- **3、输入下拉**

输入下拉模式：下拉电阻导通，施密特触发器打开，输出被禁止。在需要外部下拉电阻的时候，可以使用内部下拉电阻，这样可以节省一个外部电阻，但是内部下拉电阻的阻值较大，所以不适合做电流型驱动。

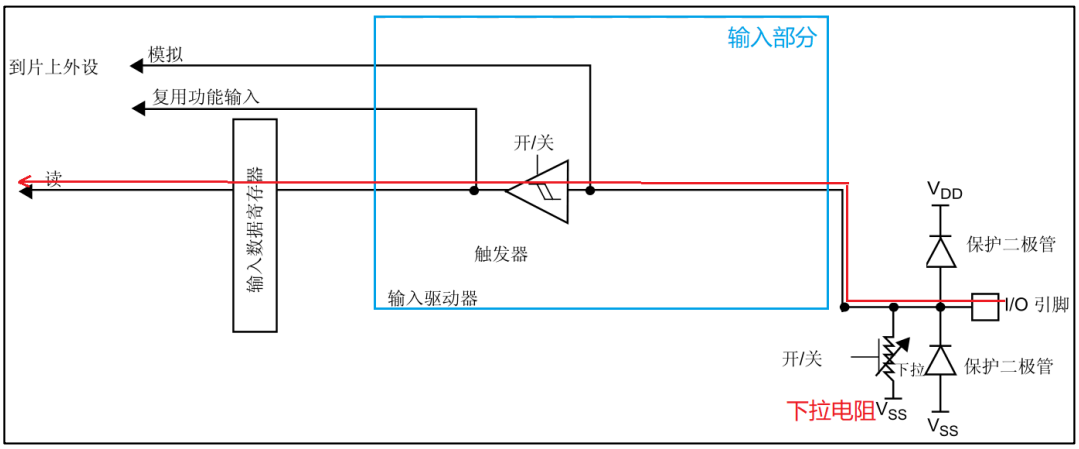


图 13.1.2.5 输入下拉模式

- **4、模拟功能**

模拟功能：上下拉电阻断开，施密特触发器关闭，双 MOS 管也关闭。该模式用于 ADC 采集或者 DAC 输出，或者低功耗下省电。

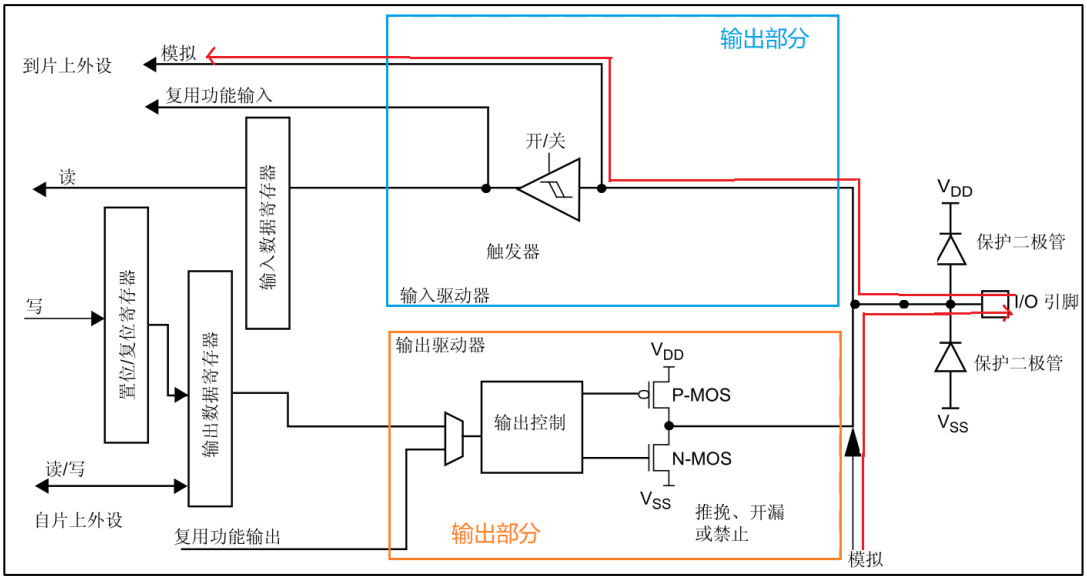


图 13.1.2.6 模拟功能

- **5、开漏输出**

开漏输出模式：STM32 的开漏输出模式是数字电路输出的一种，从结果上看它只能输出低电平 Vss 或者高阻态。根据《STM32H7xx 参考手册_V3（中文版）.pdf》第 451 页关于“GPIO 输出配置”的描述，我们可以推知开漏模式下输出电路大致是这样工作的：

- P-MOS 被“输出控制”控制在截止状态，因此 IO 的状态取决于 N-MOS 的导通状况。
- 只有 N-MOS 还受控制于输出寄存器，“输出控制器”对输入信号进行了逻辑非的操作。
- IO 到输入电路的采样电路仍被打开，且可以选择是否使用上下拉电阻。

根据参考手册的描述，我们替换了“输出控制”部分，作出了如图 13.1.2.7 的开漏模式下的简化等效图，图中①的输入对应的②的输出是我们最关心的开漏输出的结果。简化后的图 13.1.2.7 能更好地表示开漏输出模式的输出关系。

开漏输出的具体的理解描述如下：

开漏模式下，P-MOS 管是一直截止的，所以 P-MOS 管的栅极一直接 V_{SS}。如果输出数据寄存器设置为 0 时，经过“输出控制”的逻辑非操作后，输出逻辑 1 到 N-MOS 管的栅极，这时 N-MOS 管就会导通，使得 I/O 引脚接到 V_{SS}，即输出低电平。

如果输出数据寄存器设置为 1 时，经过“输出控制器”的逻辑非操作后，输出逻辑 0 到 N-MOS 管的栅极，这时 N-MOS 管就会截止。因为 P-MOS 管是一直截止的，使得 I/O 引脚呈现高阻态，即不输出低电平，也不输出高电平。因此要 I/O 引脚输出高电平就必须接上拉电阻。这时可以接内部上拉电阻，或者接一个外部上拉电阻。由于内部上拉电阻的阻值较大，所以只是“弱上拉”。需要大电流驱动，请接外部的上拉电阻。此外，上拉电阻具有线与特性，即如果有很多开漏模式的引脚连在一起的时候，只有当所有引脚都输出高阻态，电平才为 1，只要有其中一个为低电平时，就等于接地，使得整条线路都为低电平 0。我们的 IIC 通信（IIC_SDA）就用到这个原理。

另外在开漏输出模式下，施密特触发器是打开的，所以 IO 口引脚的电平状态会被采集到输入数据寄存器中，如果对输入数据寄存器进行读访问可以得到 IO 口的状态。也就是说开漏输出模式下，我们可以对 IO 口进行读数据。

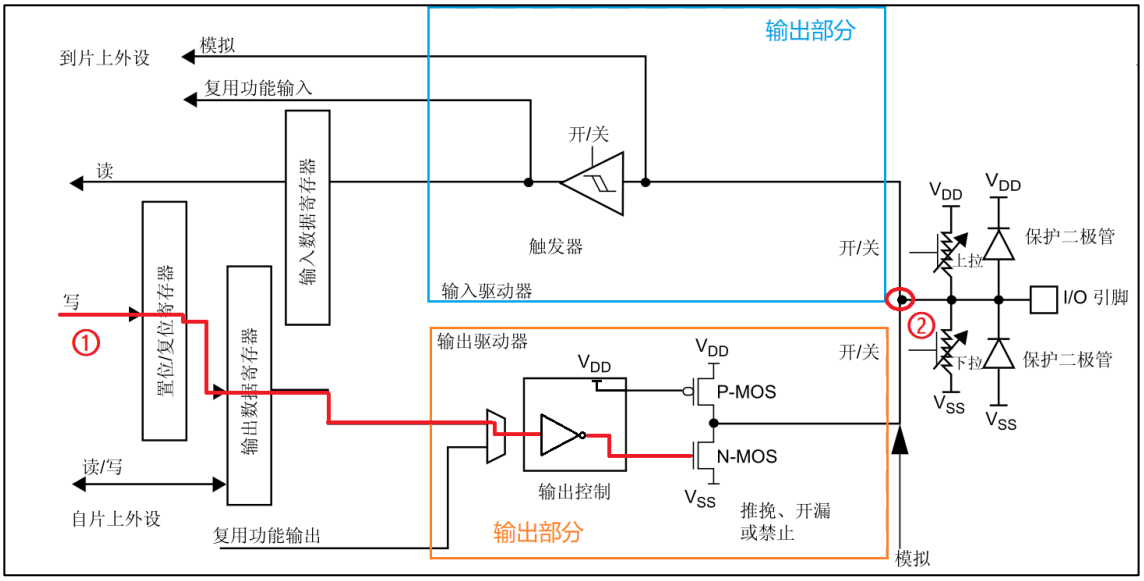


图 13.1.2.7 开漏输出模式

- **6、推挽输出**

推挽输出模式：STM32 的推挽输出模式，从结果上看它会输出低电平 VSS 或者高电平 VDD。推挽输出跟开漏输出不同的是，推挽输出模式 P-MOS 管和 N-MOS 管都用上。同样地，我们根据参考手册推挽模式下的输出描述，列出等效原理如图 13.1.2.8 所示，根据手册描述可以把“输出控制”简单地等效为一个非门。

推挽输出的具体的理解描述如下：

如果输出数据寄存器设置为 0 时，经过“输出控制”的逻辑非操作后，输出逻辑 1 到 P-

MOS 管的栅极，这时 P-MOS 管就会截止，同时也会输出逻辑 1 到 N-MOS 管的栅极，这时 N-MOS 管就会导通，使得 I/O 引脚接到 V_{SS}，即输出低电平。

如果输出数据寄存器设置为 1 时，经过“输出控制”的逻辑非操作后，输出逻辑 0 到 N-MOS 管的栅极，这时 N-MOS 管就会截止，同时也会输出逻辑 0 到 P-MOS 管的栅极，这时 P-MOS 管就会导通，使得 I/O 引脚接到 V_{DD}，即输出高电平。

上面的描述可以知道，推挽输出模式下，P-MOS 管和 N-MOS 管同一时间只能有一个 MOS 管是导通的。当引脚高低电平切换时，两个管子轮流导通，一个负责灌电流，一个负责拉电流，使其负载能力和开关速度都有很大的提高。

另外在推挽输出模式下，施密特触发器也是打开的，我们可以读取 IO 口的电平状态。

由于推挽输出模式输出高电平时，是直接连接 VDD，所以驱动能力较强，可以做电流型驱动，驱动电流最大可达 25mA。该模式也是最常用的输出模式。

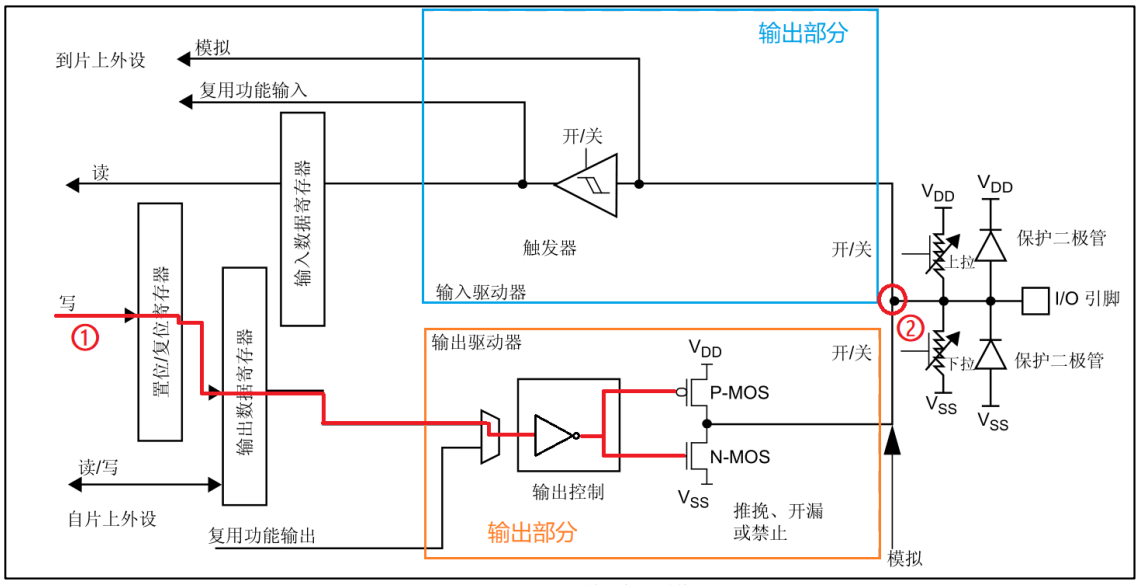


图 13.1.2.8 推挽输出模式

● 7、开漏式复用功能

开漏式复用功能：一个 IO 口可以是通用的 IO 口功能，还可以是其他外设的特殊功能引脚，这就是 IO 口的复用功能。一个 IO 口可以是多个外设的功能引脚，我们需要选择作为其中一个外设的功能引脚。当选择复用功能时，引脚的状态是由对应的外设控制，而不是输出数据寄存器。除了复用功能外，其他的结构分析请参考开漏输出模式。

另外在开漏式复用功能模式下，施密特触发器也是打开的，我们可以读取 IO 口的电平状态，同时外设可以读取 IO 口的信息。

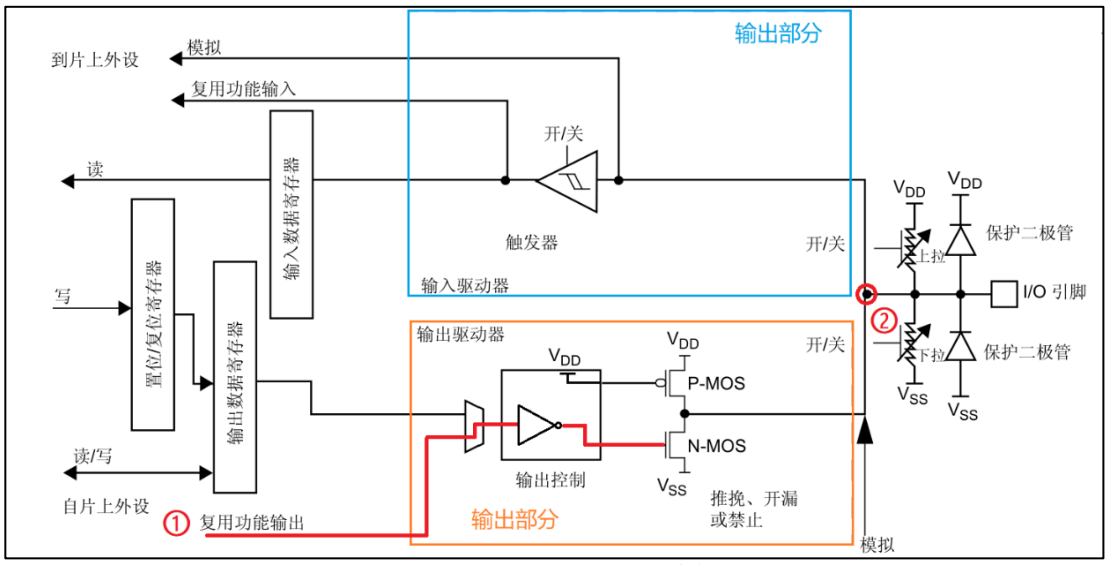


图 13.1.2.9 开漏式复用功能

- **8、推挽式复用功能**

推挽式复用功能：复用功能介绍请查看开漏式复用功能，结构分析请参考推挽输出模式，这里不再赘述。

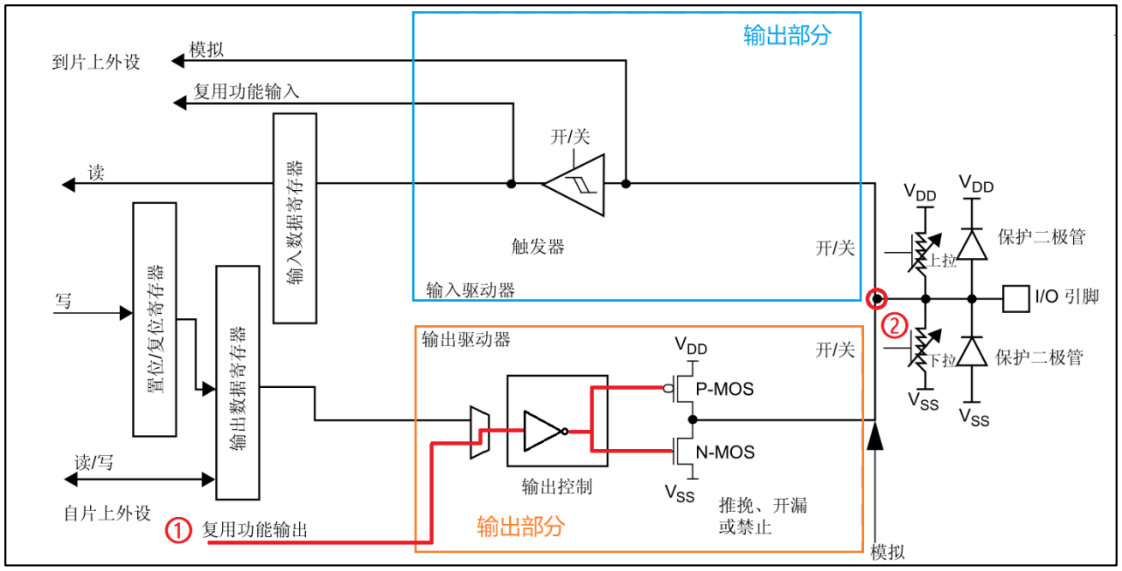


图 13.1.2.10 推挽式复用功能

13.1.3 GPIO 寄存器介绍

STM32H7 每组（这里是 A~H）通用 GPIO 口有 10 个 32 位寄存器控制，包括：
4 个 32 位配置寄存器（MODER、OTYPER、OSPEEDR 和 PUPDR）
2 个 32 位数据寄存器（IDR 和 ODR）
1 个 32 位置位/复位寄存器 (BSRR)
1 个 32 位锁定寄存器 (LCKR)
2 个 32 位复用功能选择寄存器（AFRH 和 AFRL）

下面我们将带大家理解本章用到的寄存器，没有介绍到的寄存器后面用到会继续介绍。这里主要是带大家学会怎么理解这些寄存器的方法，其他寄存器理解方法是一样的。因为寄存器太多不可能一个个列出来讲，以后基本就是只会把重要的寄存器拿出来讲述，希望大家尽快培养自己查看手册的能力。下面先看 GPIO 的 4 个 32 位配置寄存器：

- **GPIO 端口模式寄存器 (GPIOx_MODER) (x =A..K)**

该寄存器是 GPIO 口模式控制寄存器，用于控制 GPIOx（STM32H7 最多有 11 组 IO，用大写字母表示，即 x=A/B/C/D/E/F/G/H/I/J/K，下同）的工作模式，寄存器描述如图 13.1.3.1 所示。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 2y+1:2y **MODERy[1:0]**：端口 x 配置位 (Port x configuration bits) (y = 0..15)
这些位通过软件写入，用于配置 I/O 模式。
00：输入模式（复位状态）
01：通用输出模式
10：复用功能模式
11：模拟模式

图 13.1.3.1 MODER 寄存器描述

每组 GPIO 下有 16 个 IO 口，该寄存器共 32 位，每 2 个位控制 1 个 IO。我们看看这个寄存器的复位值，然后用复位值举例说明一下这样的配置值代表什么意思。比如 GPIOA 的复位值是 0xABFF FFFF，低 16 位都是 1，也就是 PA0~PA7 默认都是模拟模式。高 16 位的值是 0xABFF，也就是 PA8~PA12 默认是模拟模式，PA13\PA14\PA15 则默认是复用功能模式。而 GPIOB 的复位值是 0xFFFF FEBF，只有 PB3 默认是复用功能模式，其他默认都是模拟模式。这四个默认是复用功能模式的 IO 口都是 JTAG 功能对应的 IO 口。

- **GPIO 端口输出类型寄存器 (GPIOx_OTYPER) (x = A..K)**

该寄存器用于控制 GPIOx 的输出类型，寄存器描述如图 13.1.3.2 所示。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 15:0 **OTy**：端口 x 配置位 (Port x configuration bits) (y = 0..15)
这些位通过软件写入，用于配置 I/O 输出类型。
0：推挽输出（复位状态）
1：开漏输出

图 13.1.3.2 GPIOx OTYPER 寄存器描述

该寄存器仅用于输出模式，在输入模式（MODER[1:0]=00/11 时）下不起作用。该寄存器低 16 位有效，每一个位控制一个 IO 口，复位后，该寄存器值均为 0，也就是在输出模式下 IO 口默认为推挽输出。

- **GPIO 端口输出速度寄存器 (GPIOx_OSPEEDR) (x = A..K)**

该寄存器用于控制 GPIOx 的输出速度，寄存器描述如图 13.1.3.3 所示。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEEDR15 [1:0]		OSPEEDR14 [1:0]		OSPEEDR13 [1:0]		OSPEEDR12 [1:0]		OSPEEDR11 [1:0]		OSPEEDR10 [1:0]		OSPEEDR9 [1:0]		OSPEEDR8 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEEDR7 [1:0]		OSPEEDR6 [1:0]		OSPEEDR5 [1:0]		OSPEEDR4 [1:0]		OSPEEDR3 [1:0]		OSPEEDR2 [1:0]		OSPEEDR1 [1:0]		OSPEEDR0 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 2y+1:2y **OSPEEDRy[1:0]**：端口 x 配置位 (Port x configuration bits) (y = 0..15)

这些位通过软件写入，用于配置 I/O 输出速度。

00：低速
01：中速
10：高速
11：超高速

注：　有关 OSPEEDRy 位以及 V_{DD} 范围和外部负载的值，请参见产品数据手册。

图 13.1.3.3 GPIOx OSPEEDR 寄存器描述

该寄存器仅用于输出模式，在输入模式（MODER[1:0]=00/11 时）下不起作用。该寄存器共 32 位，每两个位控制一个 IO 口。

- **GPIO 端口上拉/下拉寄存器 (GPIOx_PUPDR) (x = A..K)**

该寄存器用于控制 GPIOx 的上拉/下拉，寄存器描述如图 13.1.3.4 所示。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PUPDR15[1:0]		PUPDR14[1:0]		PUPDR13[1:0]		PUPDR12[1:0]		PUPDR11[1:0]		PUPDR10[1:0]		PUPDR9[1:0]		PUPDR8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PUPDR7[1:0]		PUPDR6[1:0]		PUPDR5[1:0]		PUPDR4[1:0]		PUPDR3[1:0]		PUPDR2[1:0]		PUPDR1[1:0]		PUPDR0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 2y+1:2y **PUPDRy[1:0]**：端口 x 配置位 (Port x configuration bits) (y = 0..15)

这些位通过软件写入，用于配置 I/O 上拉或下拉。

00：无上拉或下拉　　10：下拉
01：上拉　　11：保留

图 13.1.3.4 GPIOx PUPDR 寄存器描述

该寄存器每两个位控制一个 IO 口，用于设置上下拉，复位后，该寄存器值一般为 0，即无上拉或下拉。

上面这 4 个配置寄存器就是用来配置 GPIO 的相关模式和状态，它们通过不同的配置组合方法，就决定我们所说的 8 种工作模式。下面，我们来列表阐述，如表 13.1.3.1 所示。

GPIO 工作模式	模式寄存器 MODER[0:1]	输出类型寄存器 OTYPER	输出速度寄存器 OSPEEDR[0:1]	上拉/下拉寄存器 PUPDR[0:1]
输入浮空	00-输入模式	无效	无效	00-无上拉或下拉
输入上拉				01-上拉
输入下拉				10-下拉
模拟功能	11-模拟模式			00-无上拉或下拉
开漏输出	01-通用输出	1-开漏输出	00-低速 01-中速 10-高速 11-超高速	00-无上拉或下拉 01-上拉 10-下拉 11-保留
推挽输出		0-推挽输出		
开漏式复用功能	10-复用功能	1-开漏输出		
推挽式复用功能		0-推挽输出		

表 13.1.3.1 4 个配置寄存器组合下的 8 种工作模式

因为本章需要 GPIO 作为输出口使用，所以我们再来看看端口输出数据寄存器。

- **端口输出数据寄存器（ODR）**

该寄存器用于控制 GPIOx 的输出高电平或者低电平，寄存器描述如图 13.1.3.5 所示。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ODR15	ODR14	ODR13	ODR12	ODR11	ODR10	ODR9	ODR8	ODR7	ODR6	ODR5	ODR4	ODR3	ODR2	ODR1	ODR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 15:0 **ODRy**：端口输出数据位 (Port output data bit) (y = 0..15)
这些位可通过软件读取和写入。
注： 对于原子置位/复位，通过写入 GPIOx_BSRR 或 GPIOx_BRR 寄存器，可分别置位和/或复位 ODR 位 (x = A..F)。

图 13.1.3.5 GPIOx ODR 寄存器描述

该寄存器低 16 位有效，分别对应每一组 GPIO 的 16 个引脚。当 CPU 写访问该寄存器，如果对应的某位写 0(ODRy=0)，则表示设置该 IO 口输出的是低电平，如果写 1(ODRy=1)，则表示设置该 IO 口输出的是高电平，y=0~15。

除了 ODR 寄存器，还有一个寄存器也是用于控制 GPIO 输出的，它就是 BSRR 寄存器。

- **端口置位/复位寄存器（BSRR）**

该寄存器也用于控制 GPIOx 的输出高电平或者低电平，寄存器描述如图 13.1.3.6 所示。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

位 31:16 **BRy**：端口 x 复位位 y (Port x reset bit y) (y = 0..15)
这些位为只写。读取这些位可返回值 0x0000。
0：不会对相应的 ODRx 位执行任何操作
1：复位相应的 ODRx 位
注： 如果同时对 BSx 和 BRx 置位，则 BSx 的优先级更高。

位 15:0 **BSy**：端口 x 置位位 y (Port x set bit y) (y= 0..15)
这些位为只写。读取这些位可返回值 0x0000。
0：不会对相应的 ODRx 位执行任何操作
1：置位相应的 ODRx 位

图 13.1.3.6 GPIOx BSRR 寄存器描述

为什么有了 ODR 寄存器，还要这个 BDRR 寄存器呢？我们先看看 BSRR 的寄存器描述，首先 BSRR 是只写权限，而 ODR 是可读可写权限。BSRR 寄存器 32 位有效，对于低 16 位（0-15），我们往相应的位写 1(BSy=1)，那么对应的 IO 口会输出高电平，往相应的位写 0(BSy=0)，对 IO 口没有任何影响，高 16 位（16-31）作用刚好相反，对相应的位写 1(BRy=1)会输出低电平，写 0(BRy=0)没有任何影响，y=0~15。

也就是说，对于 BSRR 寄存器，你写 0 的话，对 IO 口电平是没有任何影响的。我们要设置某个 IO 口电平，只需要相关位设置为 1 即可。而 ODR 寄存器，我们要设置某个 IO 口电平，我们首先需要读出来 ODR 寄存器的值，然后对整个 ODR 寄存器重新赋值来达到设置某个或者某些 IO 口的目的，而 BSRR 寄存器，我们就不需要先读，而是直接设置即可，这在多任务实时操作系统中作用很大。BSRR 寄存器还有一个好处，就是 BSRR 寄存器改变引脚状态的时候，不会被中断打断，而 ODR 寄存器有被中断打断的风险。

13.2 硬件设计

1. 例程功能

LED灯：LED0（红）和LED1（绿）每过500ms一次交替闪烁，实现类似跑马灯的效果。

2. 硬件资源

1）LED 灯
LED0 – PB1
LED1 – PB0

3. 原理图

本章用到的硬件用到板载 LED 灯：LED0 和 LED1。电路在开发板上已经连接好，所以在硬件上不需要动任何东西，直接下载代码就可以测试使用。其连接原理图如图 13.2.1 所示：

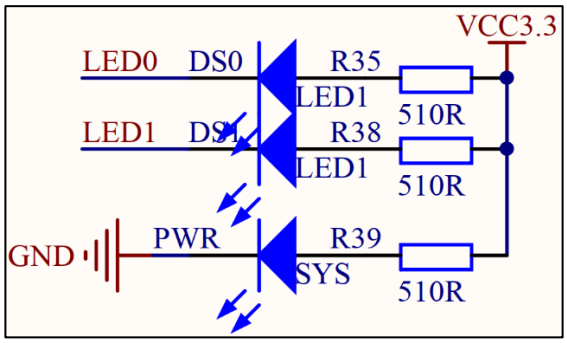


图 13.2.1 LED 与 STM32H743 连接原理图

下面，给大家介绍一下 LED 灯的压降和额定电流，供大家设计硬件参考。

直插超亮发光二极管压降，主要有三种颜色，然而三种发光二极管的压降都不相同，具体压降参考值如下：

红色发光二极管的压降为 2.0V-2.2V。
黄色发光二极管的压降为 1.8V-2.0V。
绿色发光二极管的压降为 3.0V-3.2V。
正常发光时的额定电流约为 20mA。

贴片 LED 压降：

红色的压降为 1.82-1.88V，电流 5-8mA。
绿色的压降为 1.75-1.82V，电流 3-5mA。
橙色的压降为 1.7-1.8V，电流 3-5mA。
蓝色的压降为 3.1-3.3V，电流 8-10mA。
白色的压降为 3-3.2V，电流 10-15mA。

LED 发光颜色的不同，需要的驱动电压也有差异。我们使用的 LED0 和 LED1 是红绿两个颜色的贴片灯珠，它的正常工作驱动电流典型值约为 25mA/1.8V。而 STM32 的 GPIO 的输出电流也大概是 25mA/3.3V，所以可以直接用 STM32 的 IO 驱动这种 LED。但 STM32 对总的输出电流有限制，为了经过 LED 的电流更稳定，我们一般不用 STM32 的 IO 直接输出电流驱动 LED，而是通过外接电阻的方式限制灌入 STM32 的电流。如我们使用红灯时灌到 IO 口的最大电流约为（3.3-1.8）V/510R≈2.94mA。

符号	描述	最大值(1)	单位
I_{VDD}	经过V_{DD}/V_{DDA}电源线的总电流(供应电流)(1)	150	mA
I_{VSS}	经过V_{SS}地线的总电流(流出电流)(1)	150	
I_{IO}	任意I/O和控制引脚上的输出灌电流	25	
	任意I/O和控制引脚上的输出电流	-25	
$I_{INJ(PIN)}$(2)(3)	NRST引脚的注入电流	+/-5	
	HSE的OSC_IN引脚和LSE的OSC_IN引脚的注入电流	+/-5	
	其他引脚的注入电流(4)	+/-5	
$\Sigma I_{INJ(PIN)}$(2)	所有I/O和控制引脚上的总注入电流(4)	+/-25	

图 13.2.2 STM32 IO 的电气参数

13.3 程序设计

了解了 GPIO 的结构原理和寄存器，还有我们的实验功能，下面开始设计程序。

13.3.1 GPIO 的 HAL 库驱动分析

HAL 库中关于 GPIO 的驱动程序在 stm32h7xx_hal_gpio.c 文件以及其对应的头文件。

1. HAL_GPIO_Init 函数

要使用一个外设我们首先要对它进行初始化，所以我们先看外设 GPIO 的初始化函数。其声明如下：

```
void  HAL_GPIO_Init(GPIO_TypeDef  *GPIOx, GPIO_InitTypeDef *GPIO_Init);
```

- **函数描述：**

  用于配置 GPIO 功能模式，还可以设置 EXTI 功能。

- **函数形参：**

  形参 1 是端口号，可以有以下的选择：

```
#define GPIOA                ((GPIO_TypeDef *) GPIOA_BASE)
#define GPIOB                ((GPIO_TypeDef *) GPIOB_BASE)
#define GPIOC                ((GPIO_TypeDef *) GPIOC_BASE)
#define GPIOD                ((GPIO_TypeDef *) GPIOD_BASE)
#define GPIOE                ((GPIO_TypeDef *) GPIOE_BASE)
#define GPIOF                ((GPIO_TypeDef *) GPIOF_BASE)
#define GPIOG                ((GPIO_TypeDef *) GPIOG_BASE)
#define GPIOH                ((GPIO_TypeDef *) GPIOH_BASE)
#define GPIOI                ((GPIO_TypeDef *) GPIOI_BASE)
#define GPIOJ                ((GPIO_TypeDef *) GPIOJ_BASE)
#define GPIOK                ((GPIO_TypeDef *) GPIOK_BASE)
```

这是库里面的选择项，实际上我们的芯片只能从 GPIOA~GPIOH，因为我们只有 8 组 IO 口。

形参 2 是 GPIO_InitTypeDef 类型的结构体指针变量，其定义如下：

```
typedef struct
{
  uint32_t Pin;         /* 引脚号 */
  uint32_t Mode;        /* 模式设置 */
  uint32_t Pull;        /* 上拉下拉设置 */
  uint32_t Speed;       /* 速度设置 */
  uint32_t Alternate;   /* 复用功能 */
} GPIO_InitTypeDef;
```

该结构体很重要，下面对每个成员介绍一下。

成员 Pin 表示引脚号，范围：GPIO_PIN_0 到 GPIO_PIN_15，另外还有 GPIO_PIN_All 和 GPIO_PIN_MASK 可选。

成员 Mode 是 GPIO 的模式选择，有以下选择项：

```
#define  GPIO_MODE_INPUT                  MODE_INPUT              /* 输入模式 */
#define  GPIO_MODE_OUTPUT_PP              (MODE_OUTPUT|OUTPUT_PP) /* 推挽输出 */
#define  GPIO_MODE_OUTPUT_OD              (MODE_OUTPUT|OUTPUT_OD) /* 开漏输出 */
#define  GPIO_MODE_AF_PP                  (MODE_AF|OUTPUT_PP)   /* 推挽式复用 */
#define  GPIO_MODE_AF_OD                  (MODE_AF|OUTPUT_OD)   /* 开漏式复用 */
#define  GPIO_MODE_ANALOG                  MODE_ANALOG          /* 模拟模式 */
#define  GPIO_MODE_IT_RISING              (MODE_INPUT|EXTI_IT|TRIGGER_RISING)
                                          /* 外部中断，上升沿触发检测 */
#define  GPIO_MODE_IT_FALLING             (MODE_INPUT|EXTI_IT|TRIGGER_FALLING)
                                          /* 外部中断，下降沿触发检测 */
#define  GPIO_MODE_IT_RISING_FALLING      /* 外部中断，上升和下降双沿触发检测 */
(MODE_INPUT|EXTI_IT|TRIGGER_RISING|TRIGGER_FALLING)
#define  GPIO_MODE_EVT_RISING             (MODE_INPUT|EXTI_EVT|TRIGGER_RISING)
                                          /* 外部事件，上升沿触发检测 */
#define  GPIO_MODE_EVT_FALLING            (MODE_INPUT|EXTI_EVT|TRIGGER_FALLING)
                                          /* 外部事件，下降沿触发检测 */
#define  GPIO_MODE_EVT_RISING_FALLING     /* 外部事件，上升和下降双沿触发检测 */
```

```
(MODE_INPUT|EXTI_EVT|TRIGGER_RISING|TRIGGER_FALLING)
```

成员 Pull 用于配置上下拉电阻，有以下选择项：

```
#define  GPIO_NOPULL         (0x00000000U)      /* 无上下拉 */
#define  GPIO_PULLUP         (0x00000001U)      /* 上拉 */
#define  GPIO_PULLDOWN       (0x00000002U)      /* 下拉 */
```

成员 Speed 用于配置 GPIO 的速度，有以下选择项：

```
#define  GPIO_SPEED_FREQ_LOW          (0x00000000U)     /* 低速 */
#define  GPIO_SPEED_FREQ_MEDIUM       (0x00000001U)     /* 中速 */
#define  GPIO_SPEED_FREQ_HIGH         (0x00000002U)     /* 快速 */
#define  GPIO_SPEED_FREQ_VERY_HIGH    (0x00000003U)     /* 高速 */
```

成员 Alternate 用于配置具体的复用功能，不同的 GPIO 口可以复用的功能不同，具体可参考数据手册《STM32H743IIT6.pdf》。复用功能的选择在 stm32h7xx_hal_gpio_ex.h 文件里进行了定义，后面具体用到了，我们在进行讲解。

- **函数返回值：**

  无

- **注意事项：**

  HAL 库的 EXTI 外部中断的设置功能整合到此函数里面，而不是单独独立一个文件。这个我们到外部中断实验再细讲。

2. HAL_GPIO_WritePin 函数

HAL_GPIO_WritePin 函数是 GPIO 口的写引脚函数。其声明如下：

```
void HAL_GPIO_WritePin(GPIO_TypeDef *GPIOx,
                       uint16_t GPIO_Pin, GPIO_PinState PinState);
```

- **函数描述：**

  用于设置引脚输出高电平或者低电平，通过 BSRR 寄存器复位或者置位操作。

- **函数形参：**

  形参 1 是端口号，可以选择范围：GPIOA~GPIOK。

  形参 2 是引脚号，可以选择范围：GPIO_PIN_0 到 GPIO_PIN_15。

  形参 3 是要设置输出的状态，是枚举型有两个选择：GPIO_PIN_SET 表示高电平，GPIO_PIN_RESET 表示低电平。

- **函数返回值：**

  无

3. HAL_GPIO_TogglePin 函数

HAL_GPIO_TogglePin 函数是 GPIO 口的电平翻转函数。其声明如下：

```
void HAL_GPIO_TogglePin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
```

- **函数描述：**

  用于设置引脚的电平翻转，也是通过 BSRR 寄存器复位或者置位操作。

- **函数形参：**

  形参 1 是端口号，可以选择范围：GPIOA~GPIOK。

  形参 2 是引脚号，可以选择范围：GPIO_PIN_0 到 GPIO_PIN_15。

- **函数返回值：**

  无

本实验我们用到上面三个函数，其他的 API 函数后面用到再进行讲解。

GPIO 输出配置步骤

1）使能对应 GPIO 时钟

STM32 在使用任何外设之前，我们都要先使能其时钟（下同）。本实验用到 PB0 和 PB1 两个 IO 口，因此需要先使能 GPIOB 的时钟，代码如下:

```
    __HAL_RCC_GPIOB_CLK_ENABLE();
```

2）设置对应 GPIO 工作模式（推挽输出）

本实验 GPIO 使用推挽输出模式，控制 LED 亮灭，通过函数 HAL_GPIO_Init 设置实现。

3）控制 GPIO 引脚输出高低电平

在配置好 GPIO 工作模式后，我们就可以通过 HAL_GPIO_WritePin 函数控制 GPIO 引脚输

出高低电平，从而控制 LED 的亮灭了。

13.3.2 程序流程图

程序流程图能帮助我们更好的理解一个工程的功能和实现的过程，对学习和设计工程有很好的主导作用。

下面看看本实验的程序流程图：

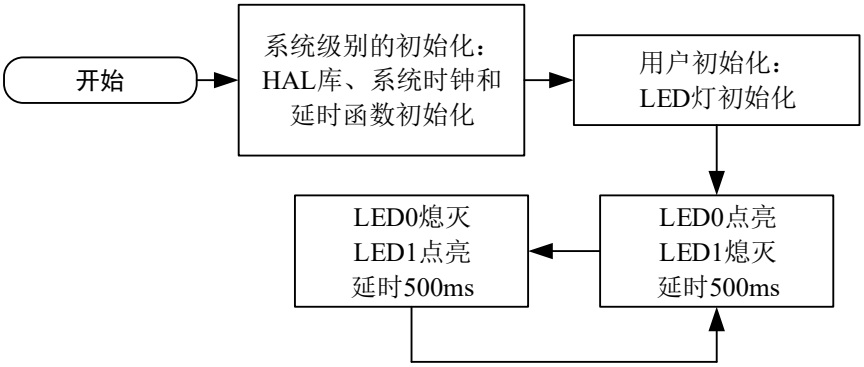


图 13.3.2.1 跑马灯实验程序流程图

13.3.3 程序解析

1. led 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。LED 驱动源码包括两个文件：led.c 和 led.h。

下面我们先解析 led.h 的程序，我们把它分两部分功能进行讲解。

- **LED 灯引脚宏定义**

由硬件设计小节，我们知道 LED 灯在硬件上分别连接到 PB0 和 PB1，再结合 HAL 库，我们做了下面的引脚定义。

```
/* LED0 引脚定义 */
#define LED0_GPIO_PORT                  GPIOB
#define LED0_GPIO_PIN                   GPIO_PIN_1
#define LED0_GPIO_CLK_ENABLE()          do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)
/* LED1 引脚定义 */
#define LED1_GPIO_PORT                  GPIOB
#define LED1_GPIO_PIN                   GPIO_PIN_0
#define LED1_GPIO_CLK_ENABLE()          do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)
```

这样的好处是进一步隔离底层函数操作，移植更加方便，函数命名更贴近实际的开发板。比如：当我们看到 LED0_GPIO_PORT 这个宏定义，我们就知道这是灯 LED0 的端口号；看到 LED0_GPIO_PIN 这个宏定义，就知道这是灯 LED0 的引脚号；看到 LED0_GPIO_CLK_ENABLE 这个宏定义，就知道这是灯 LED0 的时钟使能函数。大家后面学习时间长了就会慢慢熟悉这样的命名方式。

特别注意：这里的时钟使能函数宏定义，使用了 do{ }while(0)结构，是为了避免在某些使用场景出错的问题（下同），详见《嵌入式单片机 C 代码规范与风格》第六章第 2 点。

__HAL_RCC_GPIOx_CLK_ENABLE 函数是 HAL 库的 IO 口时钟使能函数，x=A 到 K。

- **LED 灯操作函数宏定义**

为了后续对 LED 灯进行便捷的操作，我们为 LED 灯操作函数做了下面的定义。

```
/* LED 端口定义 */
#define LED0(x)    do{ x ? \
                       HAL_GPIO_WritePin(LED0_GPIO_PORT,LED0_GPIO_PIN, GPIO_PIN_SET) : \
                       HAL_GPIO_WritePin(LED0_GPIO_PORT,LED0_GPIO_PIN, GPIO_PIN_RESET); \
                   }while(0)       /* LED0 翻转 */
#define LED1(x)    do{ x ? \
```

```
            HAL_GPIO_WritePin(LED1_GPIO_PORT,LED1_GPIO_PIN, GPIO_PIN_SET) : \
            HAL_GPIO_WritePin(LED1_GPIO_PORT,LED1_GPIO_PIN, GPIO_PIN_RESET); \
                    }while(0)       /* LED1 翻转 */
/* LED 电平翻转定义 */
#define LED0_TOGGLE()     do{ HAL_GPIO_TogglePin(LED0_GPIO_PORT,
                              LED0_GPIO_PIN); }while(0)    /* LED0 = !LED0 */
#define LED1_TOGGLE()     do{ HAL_GPIO_TogglePin(LED1_GPIO_PORT,
                              LED1_GPIO_PIN); }while(0)    /* LED1 = !LED1 */
```

LED0 和 LED1 这两个宏定义，分别是控制 LED0 和 LED1 的亮灭。例如：对于宏定义标识符 LED0(x)，它的值是通过条件运算符来确定：当 x=0 时，宏定义的值为 HAL_GPIO_WritePin(LED0_GPIO_PORT, LED0_GPIO_PIN, GPIO_PIN_RESET)，也就是设置 PB1 输出低电平，当 n!=0 时，宏定义的值为 HAL_GPIO_WritePin (LED0_GPIO_PORT, LED0_GPIO_PIN, GPIO_PIN_SET)，也就是设置 PB1 输出高电平。所以如果要设置 LED0 输出低电平，那么调用宏定义 LED0(0)即可，如果要设置 LED0 输出高电平，调用宏定义 LED0(1)即可。宏定义 LED1(x)和 LED0(x)同理。

因为 STM32H7 不支持位带操作，所以这里我们并没有像 F1/F4 一样通过位带操作来实现 IO 口输出输入电平控制。

LED0_TOGGLE 和 LED1_TOGGLE 这两个宏定义，分别是控制 LED0 和 LED1 的翻转。这里利用 HAL_GPIO_TogglePin 函数实现 IO 口输出电平取反操作。

下面我们再解析 led.c 的程序，这里只有一个函数 led_init，这是 LED 灯的初始化函数，其定义如下：

```
/**
 * @brief       初始化 LED 相关 IO 口, 并使能时钟
 * @param       无
 * @retval      无
 */
void led_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;
    LED0_GPIO_CLK_ENABLE(); /* LED0 时钟使能 */
    LED1_GPIO_CLK_ENABLE(); /* LED1 时钟使能 */

    gpio_init_struct.Pin = LED0_GPIO_PIN;              /* LED0 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;       /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;               /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_MEDIUM;   /* 中速 */
    HAL_GPIO_Init(LED0_GPIO_PORT, &gpio_init_struct);  /* 初始化 LED0 引脚 */

    gpio_init_struct.Pin = LED1_GPIO_PIN;              /* LED1 引脚 */
    HAL_GPIO_Init(LED1_GPIO_PORT, &gpio_init_struct); /* 初始化 LED1 引脚 */

    LED0(1);     /* 关闭 LED0 */
    LED1(1);     /* 关闭 LED1 */
}
```

对 LED 灯的两个引脚都设置为中速上拉的推挽输出。最后关闭两个 LED 灯，防止没有操作就亮了。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    led_init();                             /* 初始化 LED */
```

```
    while (1)
    {
        LED0(0);                            /* LED0(RED)   亮 */
        LED1(1);                            /* LED1(GREEN) 灭 */
        delay_ms(500);
        LED0(1);                            /* LED0(RED)   灭 */
        LED1(0);                            /* LED1(GREEN) 亮 */
        delay_ms(500);
    }
}
```

首先是调用系统级别的初始化：sys_cache_enable 函数使能 I-Cache 和 D-Cache，然后初始化 HAL 库、系统时钟和延时函数。接下来，调用 led_init 来初始化 LED 灯。最后在无限循环里面实现 LED0 和 LED1 间隔 500ms 交替闪烁一次。

13.4 下载验证

我们先来看看编译结果，如图 13.4.1 所示。

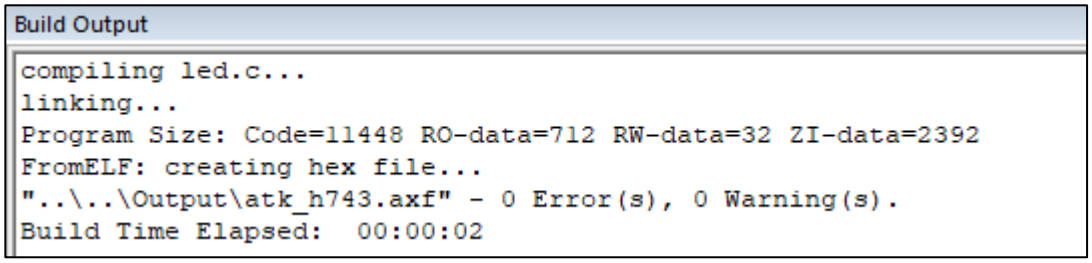


图 13.4.1 编译结果

可以看到没有 0 错误，0 警告。从编译信息可以看出，我们的代码占用 FLASH 大小为：12160 字节（11448+712），所用的 SRAM 大小为：2424 个字节（32+2392）。这里我们解释一下，编译结果里面的几个数据的意义：

Code：表示程序所占用 FLASH 的大小（FLASH）。

RO-data：即 Read Only-data，表示程序定义的常量（FLASH）。

RW-data：即 Read Write-data，表示已被初始化的变量（SRAM）

ZI-data：即 Zero Init-data，表示未被初始化的变量(SRAM)

有了这个就可以知道你当前使用的 flash 和 sram 大小了，所以，一定要注意的是程序的大小不是.hex 文件的大小，而是编译后的 Code 和 RO-data 之和。

接下来，大家就可以下载验证了。这里我们使用 DAP 仿真器下载（也可以通过其他仿真器下载，**如果是 JLINK，必须是 V9 或者以上版本，才可以支持 STM32H743!!** 下同）。

下载完之后，可以看到 LED 灯的 LED0（红）和 LED1（绿）轮流亮。

至此，我们的跑马灯实验的学习就结束了，本章介绍了 STM32H743 的 IO 口的使用及注意事项，是后面学习的基础，希望大家好好理解。

第十四章 按键输入实验

上一章，我们介绍了 STM32H7 的 IO 口作为输出的使用。本章，我们将向大家介绍如何使用 STM32H7 的 IO 口作为输入。我们将利用板载的 4 个按键，来控制板载的两个 LED 灯亮灭。通过本章的学习，我们将了解到 STM32H7 的 IO 口作为输入的使用方法。

本章分为如下几个小节：

14.1 按键与输入数据寄存器

14.2 硬件设计

14.3 程序设计

14.4 下载验证

14.1 按键与输入数据寄存器简介

14.1.1 独立按键简介

几乎每个开发板都会板载有独立按键，因为按键用处很多。常态下，独立按键是断开的，按下的时候才闭合。每个独立按键会单独占用一个 IO 口，通过 IO 口的高低电平判断按键的状态。但是按键在闭合和断开的时候，都存在抖动现象，即按键在闭合时不会马上就稳定的连接，断开时也不会马上断开。这是机械触点，无法避免。独立按键抖动波形图如下：

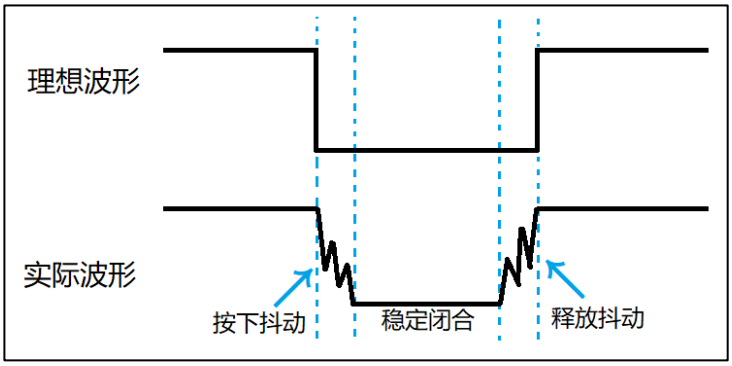


图 14.1.1.1 独立按键抖动波形图

图中的按下抖动和释放抖动的时间一般为 5~10ms 如果在抖动阶段采样，其不稳定状态可能出现一次按键动作被认为是多次按下的情况。为了避免抖动可能带来的误操作，我们要做的措施就是给按键消抖（即采样稳定闭合阶段）。消抖方法分为硬件消抖和软件消抖，我们常用软件的方法消抖。

软件消抖：方法很多，我们例程中使用最简单的延时消抖。检测到按键按下后，一般进行 10ms 延时，用于跳过抖动的时间段，如果消抖效果不好可以调整这个 10ms 延时，因为不同类型的按键抖动时间可能有偏差。待延时过后再检测按键状态，如果没有按下，那我们就判断这是抖动或者干扰造成的；如果还是按下，那么我们就认为这是按键真的按下了。对按键释放的判断同理。

硬件消抖：利用 RC 电路的电容充放电特性来对抖动产生的电压毛刺进行平滑出来，从而实现消抖，但是成本会更高一点，本着能省则省的原则，我们推荐使用软件消抖即可。

14.1.2 GPIO 端口输入数据寄存器（IDR）

本实验我们将会用到 GPIO 端口输入数据寄存器，下面来介绍一下。

该寄存器用于存储 GPIOx 的输入状态，它连接到施密特触发器上，IO 口外部的电平信号经过触发器后，模拟信号就被转化成 0 和 1 这样的数字信号，并存储到该寄存器中。寄存器描述如图 14.1.2.1 所示。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
IDR15	IDR14	IDR13	IDR12	IDR11	IDR10	IDR9	IDR8	IDR7	IDR6	IDR5	IDR4	IDR3	IDR2	IDR1	IDR0
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

位 15:0 **IDRy**：端口输入数据位 (Port input data bit) (y = 0..15)
这些位为只读。它们包含相应 I/O 端口的输入值。

图 14.1.2.1 GPIOx IDR 寄存器描述

该寄存器低 16 位有效，分别对应每一组 GPIO 的 16 个引脚。当 CPU 访问该寄存器，如果对应的某位为 0(IDRy=0)，则说明该 IO 口输入的是低电平，如果是 1(IDRy=1)，则表示输入的是高电平，y=0~15。

14.2 硬件设计

1. 例程功能

通过开发板上的四个独立按键控制 LED 灯：KEY0 控制 LED0/LED1 同时翻转，KEY1 控制 LED1 翻转，KEY2 控制 LED0 翻转，KEY_UP 控制 LED0/LED1 互斥点亮。

2. 硬件资源

1）LED 灯
LED0 - PB1
LED1 - PB0
2）独立按键
KEY0 - PH3
KEY1 - PH2
KEY2 - PC13
KEY_UP - PA0(程序中的宏名:WK_UP)

3. 原理图

独立按键硬件部分的原理图，如图 14.2.1 所示：

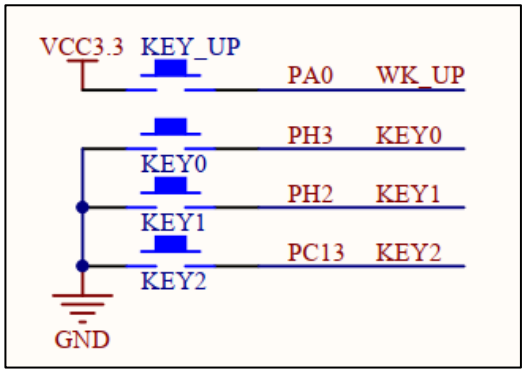


图 14.2.1 独立按键与 STM32H7 连接原理图

这里需要注意的是：KEY0、KEY1 和 KEY2 是低电平有效的，而 KEY_UP 是高电平有效的，并且外部都没有上下拉电阻，所以，需要在 STM32H743 内部设置上下拉。

14.3 程序设计

14.3.1 HAL_GPIO_ReadPin 函数

HAL_GPIO_ReadPin 函数是 GPIO 口的读引脚函数。其声明如下：

```
GPIO_PinState HAL_GPIO_ReadPin(GPIO_TypeDef *GPIOx, uint16_t GPIO_Pin);
```

- **函数描述：**
  用于读取 GPIO 引脚状态，通过 IDR 寄存器读取。
- **函数形参：**
  形参 1 是端口号，可以选择范围：GPIOA~GPIOK。
  形参 2 是引脚号，可以选择范围：GPIO_PIN_0 到 GPIO_PIN_15。
- **函数返回值：**
  引脚状态值 0 或者 1

GPIO 输入配置步骤

1）使能对应 GPIO 时钟

本实验用到 PA0 和 PH2/PH3/PC13 这四个 IO 口，因此需要使能 GPIOA、GPIOC 和 GPIOH 的时钟，代码如下:

```
__HAL_RCC_GPIOA_CLK_ENABLE();
__HAL_RCC_GPIOC_CLK_ENABLE();
__HAL_RCC_GPIOH_CLK_ENABLE();
```

2）设置对应 GPIO 工作模式（上拉/下拉输入）

本实验 GPIO 使用输入模式（带上拉/下拉），从而可以读取 IO 口的状态，实现按键检测，GPIO 模式通过函数 HAL_GPIO_Init 设置实现。

3）读取 GPIO 引脚高低电平

在配置好 GPIO 工作模式后，我们就可以通过 HAL_GPIO_ReadPin 函数读取 GPIO 引脚的高低电平，从而实现按键检测了。

14.3.2 程序流程图

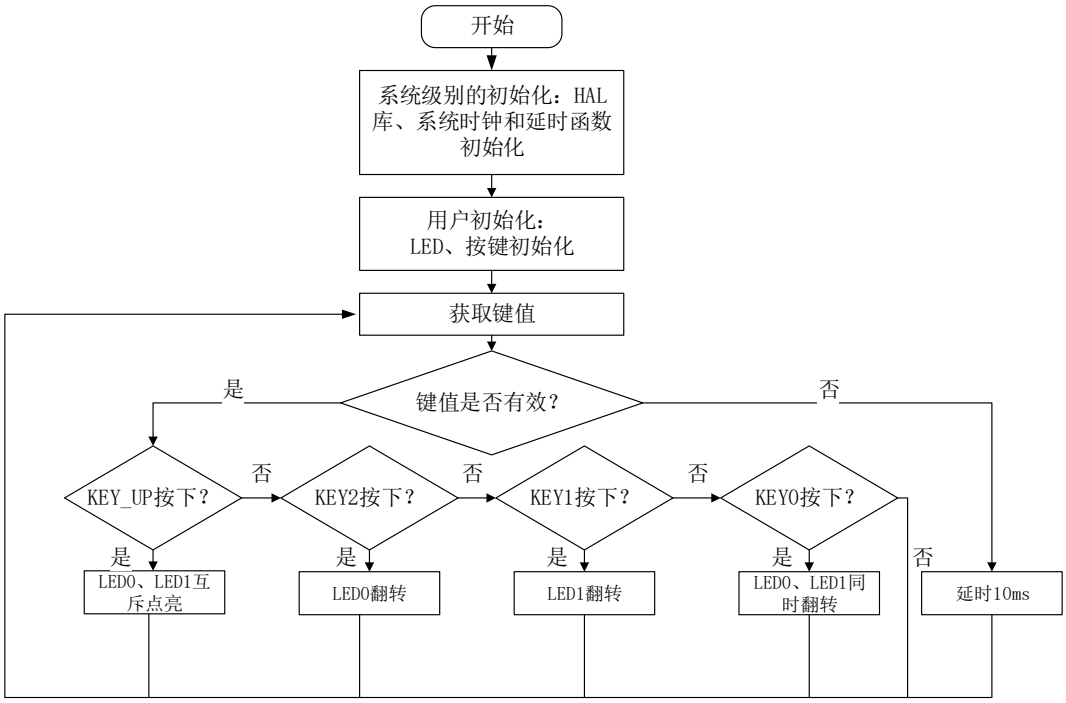


图 14.3.2.1 按键输入实验程序流程图

14.3.3 程序解析

1. 按键驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。按键（KEY）

驱动源码包括两个文件：key.c 和 key.h。

下面我们先解析 key.h 的程序，我们把它分两部分功能进行讲解。

- **按键引脚定义**

由硬件设计小节，我们知道 KEY0、KEY1、KEY2 和 KEY_UP 分别来连接到 PH3、PH2、PC3 和 PA0 上，我们做了下面的引脚定义。

```
/* 引脚 定义 */
#define KEY0_GPIO_PORT                  GPIOH
#define KEY0_GPIO_PIN                   GPIO_PIN_3
#define KEY0_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOH_CLK_ENABLE(); }while(0)    /* PH 口时钟使能 */

#define KEY1_GPIO_PORT                  GPIOH
#define KEY1_GPIO_PIN                   GPIO_PIN_2
#define KEY1_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOH_CLK_ENABLE(); }while(0)    /* PH 口时钟使能 */

#define KEY2_GPIO_PORT                  GPIOC
#define KEY2_GPIO_PIN                   GPIO_PIN_13
#define KEY2_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOC_CLK_ENABLE(); }while(0)    /* PC 口时钟使能 */

#define WKUP_GPIO_PORT                  GPIOA
#define WKUP_GPIO_PIN                   GPIO_PIN_0
#define WKUP_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)    /* PA 口时钟使能 */
```

- **按键操作函数定义**

为了后续对按键进行便捷的操作，我们为按键操作函数做了下面的定义。

```
#define KEY0  HAL_GPIO_ReadPin(KEY0_GPIO_PORT, KEY0_GPIO_PIN) /* 读取 KEY0 引脚 */
#define KEY1  HAL_GPIO_ReadPin(KEY1_GPIO_PORT, KEY1_GPIO_PIN) /* 读取 KEY1 引脚 */
#define KEY2  HAL_GPIO_ReadPin(KEY2_GPIO_PORT, KEY2_GPIO_PIN) /* 读取 KEY2 引脚 */
#define WK_UP HAL_GPIO_ReadPin(WKUP_GPIO_PORT, WKUP_GPIO_PIN) /* 读取 WKUP 引脚 */

#define KEY0_PRES    1              /* KEY0 按下 */
#define KEY1_PRES    2              /* KEY1 按下 */
#define KEY2_PRES    3              /* KEY2 按下 */
#define WKUP_PRES    4              /* KEY_UP 按下(即 WK_UP) */
```

KEY0、KEY1、KEY2 和 WK_UP 分别是读取对应按键状态的宏定义。用 HAL_GPIO_ReadPin 函数实现，该函数的返回值就是 IO 口的状态，返回值是枚举类型，取值 0 或者 1。

KEY0_PRES、KEY1_PRES、KEY2_PRES 和 WKUP_PRES 则是按键对应的四个键值宏定义标识符。

下面我们再解析 key.c 的程序，这里有两个函数，先看按键初始化函数，其定义如下：

```
/**
 * @brief       按键初始化函数
 * @param       无
 * @retval      无
 */
void key_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;                  /* GPIO 配置参数存储变量 */
    KEY0_GPIO_CLK_ENABLE();                             /* KEY0 时钟使能 */
    KEY1_GPIO_CLK_ENABLE();                             /* KEY1 时钟使能 */
    KEY2_GPIO_CLK_ENABLE();                             /* KEY2 时钟使能 */
    WKUP_GPIO_CLK_ENABLE();                             /* WKUP 时钟使能 */

    gpio_init_struct.Pin = KEY0_GPIO_PIN;               /* KEY0 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_INPUT;            /* 输入 */
    gpio_init_struct.Pull = GPIO_PULLUP;                /* 上拉 */
```

```
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;    /* 高速 */
    HAL_GPIO_Init(KEY0_GPIO_PORT, &gpio_init_struct);
    /* KEY0 引脚模式设置,上拉输入 */

    gpio_init_struct.Pin = KEY1_GPIO_PIN;              /* KEY1 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_INPUT;           /* 输入 */
    gpio_init_struct.Pull = GPIO_PULLUP;               /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;     /* 高速 */
    HAL_GPIO_Init(KEY1_GPIO_PORT, &gpio_init_struct);
    /* KEY1 引脚模式设置,上拉输入 */

    gpio_init_struct.Pin = KEY2_GPIO_PIN;              /* KEY2 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_INPUT;           /* 输入 */
    gpio_init_struct.Pull = GPIO_PULLUP;               /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;     /* 高速 */
   HAL_GPIO_Init(KEY2_GPIO_PORT, &gpio_init_struct);
   /* KEY2 引脚模式设置,上拉输入 */

    gpio_init_struct.Pin = WKUP_GPIO_PIN;              /* WKUP 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_INPUT;           /* 输入 */
    gpio_init_struct.Pull = GPIO_PULLDOWN;             /* 下拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;     /* 高速 */
   HAL_GPIO_Init(WKUP_GPIO_PORT, &gpio_init_struct);
   /* WKUP 引脚模式设置,下拉输入 */
}
```

这里需要注意的是：KEY0 和 KEY1 是低电平有效的（即一端接地），所以我们要设置为内部上拉，而 KEY_UP 是高电平有效的（即一端接电源），所以我们要设置为内部下拉。

另一个函数是按键扫描函数，其定义如下：

```
/**
 * @brief       按键扫描函数
 * @note        该函数有响应优先级(同时按下多个按键): WK_UP > KEY2 > KEY1 > KEY0!!
 * @param       mode:0 / 1, 具体含义如下:
 *   @arg       0,  不支持连续按(当按键按下不放时, 只有第一次调用会返回键值,
 *                  必须松开以后, 再次按下才会返回其他键值)
 *   @arg       1,  支持连续按(当按键按下不放时, 每次调用该函数都会返回键值)
 * @retval      键值, 定义如下:
 *              KEY0_PRES, 1, KEY0 按下
 *              KEY1_PRES, 2, KEY1 按下
 *              KEY2_PRES, 3, KEY2 按下
 *              WKUP_PRES, 4, WKUP 按下
 */
uint8_t key_scan(uint8_t mode)
{
    static uint8_t key_up = 1;  /* 按键按松开标志 */
    uint8_t keyval = 0;
    if (mode) key_up = 1;       /* 支持连按 */
    if (key_up && (KEY0 == 0 || KEY1 == 0 || KEY2 == 0 || WK_UP == 1))
    {/* 按键松开标志为 1, 且有任意一个按键按下了 */
        delay_ms(10);           /* 去抖动 */
        key_up = 0;

        if (KEY0 == 0)  keyval = KEY0_PRES;
        if (KEY1 == 0)  keyval = KEY1_PRES;
        if (KEY2 == 0)  keyval = KEY2_PRES;
        if (WK_UP == 1) keyval = WKUP_PRES;
    }
    else if (KEY0 == 1 && KEY1 == 1 && KEY2 == 1 && WK_UP == 0)
    {/* 没有任何按键按下, 标记按键松开 */
        key_up = 1;
    }
```

```
    return keyval;                  /* 返回键值 */
}
```

key_scan 函数用于扫描这 4 个 IO 口是否有按键按下。key_scan 函数，支持两种扫描方式，通过 mode 参数来设置。

当 mode 为 0 的时候，key_scan 函数将不支持连续按，扫描某个按键，该按键按下之后必须要松开，才能第二次触发，否则不会再响应这个按键，这样的好处就是可以防止按一次多次触发，而坏处就是在需要长按的时候比较不合适。

当 mode 为 1 的时候，key_scan 函数将支持连续按，如果某个按键一直按下，则会一直返回这个按键的键值，这样可以方便的实现长按检测。

有了 mode 这个参数，大家就可以根据自己的需要，选择不同的方式。这里要提醒大家，因为该函数里面有 static 变量，所以该函数不是一个可重入函数，在有 OS 的情况下，这个大家要留意下。可以看到该函数的消抖延时是 10ms。同时还有一点要注意的是，该函数的按键扫描是有优先级的，最优先的是 KEY_UP，第二优先的是 KEY1，最后是按键 KEY0。该函数有返回值，如果有按键按下，则返回非 0 值，如果没有或者按键不正确，则返回 0。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t key;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    led_init();                             /* 初始化 LED */
    key_init();                             /* 初始化按键 */
    LED0(0);                                /* 先点亮红灯 */

    while (1)
    {
        key = key_scan(0);                  /* 得到键值 */
        if(key)
        {
            switch (key)
            {
                case WKUP_PRES:              /* 控制 LED0, LED1 互斥点亮 */
                    LED1_TOGGLE();           /* LED1 状态取反 */
                    if(HAL_GPIO_ReadPin(LED1_GPIO_PORT, LED1_GPIO_PIN) == 1)
                    {
                      LED0(0);
                    }
                    else LED0(1);
                    break;

                case KEY0_PRES:              /* 同时控制 LED0, LED1 翻转 */
                    LED0_TOGGLE();           /* LED0 状态取反 */
                    LED1_TOGGLE();           /* LED1 状态取反 */
                    break;

                case KEY1_PRES:              /* 控制 LED1(GREEN)翻转 */
                    LED1_TOGGLE();           /* LED1 状态取反 */
                    break;

                case KEY2_PRES:              /* 控制 LED0(RED)翻转 */
                    LED0_TOGGLE();           /* LED0 状态取反 */
                    break;
            }
```

```
        }
        else
        {
            delay_ms(10);
        }
    }
}
```

首先是调用系统级别的初始化：sys_cache_enable 函数使能 I-Cache 和 D-Cache，然后初始化 HAL 库、系统时钟和延时函数。接下来，调用 led_init 来初始化 LED 灯，调用 key_init 函数初始化独立按键。最后在无限循环里面扫描获取键值，接着用键值判断哪个按键按下，如果有按键按下则翻转相应的灯，如果没有按键按下则延时 10ms。

14.4 下载验证

在下载好程序后，我们可以按 KEY0、KEY1、KEY2 和 KEY_UP 来看看 LED 灯的变化，是否和我们预期的结果一致？

至此，我们的本章的学习就结束了。本章学习了 STM32H743 的 IO 作为输入的使用方法，同时巩固了前面的学习。

第十五章 外部中断实验

在前面几章的学习中，我们掌握了 STM32H7 的 IO 口最基本的操作。本章我们将介绍如何把 STM32H7 的 IO 口作为外部中断输入来使用，在本章中，我们将以中断的方式，实现我们在第十五章所实现的功能。

本章分为如下几个小节：

15.1 STM32H7 NVIC 和外部中断简介
15.2 硬件设计
15.3 程序设计
15.4 下载验证

15.1 NVIC 和 EXTI 简介

15.1.1 NVIC 简介

什么是 NVIC？NVIC 即嵌套向量中断控制器，全称 Nested vectored interrupt controller。它是内核的器件，所以它的更多描述可以看内核有关的资料。M3/M4/M7 内核都是支持 256 个中断，其中包含了 16 个系统中断和 240 个外部中断，并且具有 256 级的可编程中断设置。然而芯片厂商一般不会把内核的这些资源全部用完，如 STM32H743 的系统中断有 10 个，外部中断有 150 个。这个在讲解启动文件的时候有提到过，可以回顾一下。下面我们看看系统中断部分：

信号	优先级	NVIC 位置	缩略语	说明	偏移地址
-	-	-	-	保留	0x0000 0000
-	-3	-	Reset	复位	0x0000 0004
-	-2	-	NMI	不可屏蔽中断。 RCC 时钟安全系统 (CSS) 连接到 NMI 向量。	0x0000 0008
-	-1	-	HardFault	所有类型的错误	0x0000 000C
-	0	-	MemManage	存储器管理	0x0000 0010
-	1	-	BusFault	预取故障，存储器访问故障	0x0000 0014
-	2	-	UsageFault	未定义的指令或非法状态	0x0000 0018
-	-	-	-	保留	0x0000 001C- 0x0000 002B
-	3	-	SVCall	通过 SWI 指令调用的系统服务	0x0000 002C
-	4	-	DebugMonitor	调试监控器	0x0000 0030
-	-	-	-	保留	0x0000 0034
-	5	-	PendSV	可挂起的系统服务	0x0000 0038
-	6	-	SysTick	系统节拍定时器	0x0000 003C

表 15.1.1.1 中断向量表-系统中断部分

关于 150 个外部中断部分在《STM32H7xx 参考手册_V3（中文版）.pdf》的 19.1.2 小节有详细的列表，这里就不列出来了。STM32H743 的中断向量表在 stm32h743xx.h 文件中被定义。

15.1.1.1 NVIC 寄存器

NVIC 相关的寄存器定义可以在 core_cm7.h 文件中找到，还可以在《STM32H7 编程手册.pdf》的 4.2 小节找到其描述。我们直接通过程序的定义来分析 NVIC 相关的寄存器，其定义

如下：

```
typedef struct
{
  __IOM uint32_t ISER[8U];                /* 中断使能寄存器 */
        uint32_t RESERVED0[24U];
  __IOM uint32_t ICER[8U];                /* 中断除能寄存器 */
        uint32_t RSERVED1[24U];
  __IOM uint32_t ISPR[8U];                /* 中断使能挂起寄存器 */
        uint32_t RESERVED2[24U];
  __IOM uint32_t ICPR[8U];                /* 中断解挂寄存器 */
        uint32_t RESERVED3[24U];
  __IOM uint32_t IABR[8U];                /* 中断有效位寄存器 */
        uint32_t RESERVED4[56U];
  __IOM uint8_t  IP[240U];                /* 中断优先级寄存器（8Bit 位宽） */
        uint32_t RESERVED5[644U];
  __OM  uint32_t STIR;                    /* 中断触发中断寄存器 */
}  NVIC_Type;
```

STM32H743 的中断在这些寄存器的控制下有序的执行的。只有了解这些中断寄存器，才能方便的使用 STM32H743 的中断。下面重点介绍这几个寄存器：

ISER[8]：ISER 全称是：Interrupt Set Enable Registers，这是一个中断使能寄存器组。上面说了 CM7 内核支持 256 个中断，这里用 8 个 32 位寄存器来控制，每个位控制一个中断。但是 STM32H743 的可屏蔽中断最多只有 150 个，所以对我们来说，有用的就是四个（ISER[0~4]），总共可以表示 160 个中断。而 STM32H743 只用了其中的 150 个。ISER[0]的 bit0~31 分别对应中断 0~31；ISER[1]的 bit0~31 对应中断 32~63；其他以此类推，这样总共 150 个中断就可以分别对应上了。你要使能某个中断，必须设置相应的 ISER 位为 1，使该中断被使能（这里仅仅是使能，还要配合中断分组、屏蔽、IO 口映射等设置才算是一个完整的中断设置）。具体每一位对应哪个中断，请参考 STM32H743xx.h 里面的第 48 行到 200 行，共 150 个。

ICER[8]：全称是：Interrupt Clear Enable Registers，是一个中断除能寄存器组。该寄存器组与 ISER 的作用恰好相反，是用来清除某个中断的使能的。其对应位的功能，也和 ICER 一样。这里要专门设置一个 ICER 来清除中断位，而不是向 ISER 写 0 来清除，是因为 NVIC 的这些寄存器都是写 1 有效的，写 0 是无效的。

ISPR[8]：全称是：Interrupt Set Pending Registers，是一个中断使能挂起控制寄存器组。每个位对应的中断和 ISER 是一样的。通过置 1，可以将正在进行的中断挂起，而执行同级或更高级别的中断。写 0 是无效的。

ICPR[8]：全称是：Interrupt Clear Pending Registers，是一个中断解挂控制寄存器组。其作用与 ISPR 相反，对应位也和 ISER 是一样的。通过设置 1，可以将挂起的中断解挂。写 0 无效。

IABR[8]：全称是：Interrupt Active Bit Registers，是一个中断激活标志位寄存器组。对应位所代表的中断和 ISER 一样，如果为 1，则表示该位所对应的中断正在被执行。这是一个只读寄存器，通过它可以知道当前在执行的中断是哪一个。在中断执行完了由硬件自动清零。

IP[240]：全称是：Interrupt Priority Registers，是一个中断优先级控制的寄存器组。这个寄存器组相当重要！STM32H743 的中断分组与这个寄存器组密切相关。IP 寄存器组由 240 个 8bit 的寄存器组成，每个可屏蔽中断占用 8bit，这样总共可以表示 240 个可屏蔽中断。而 STM32H743 只用到了其中的 150 个。IP[149]~IP[0]分别对应中断 149~0。而每个可屏蔽中断占用的 8bit 并没有全部使用，而是只用了高 4 位。这 4 位，又分为抢占优先级和子优先级。抢占优先级在前，子优先级在后。而这两个优先级各占几个位又要根据 SCB->AIRCR 中的中断分组设置来决定。关于中断优先级控制的寄存器组我们下面详细讲解。

15.1.1.2 中断优先级

STM32 中的中断优先级可以分为：抢占式优先级和响应优先级，响应优先级也称子优先级，每个中断源都需要被指定这两种优先级。抢占式优先级和响应优先级的区别：

抢占优先级：抢占优先级高的中断可以打断正在执行的抢占优先级低的中断。

响应优先级：抢占优先级相同，响应优先级高的中断不能打断响应优先级低的中断。

还有一种情况就是当两个或者多个中断的抢占式优先级和响应优先级相同时，那么就遵循

自然优先级，看中断向量表的中断排序，数值越小，优先级越高。

在 NVIC 中由寄存器 NVIC_IPR0-NVIC_IPR59 共 60 个寄存器控制中断优先级，每个寄存器的每 8 位又分为一组，可以分 4 组，所以就有了 240 组宽度为 8bit 的中断优先级控制寄存器，原则上每个外部中断可配置的优先级为 0~255，数值越小，优先级越高。但是实际上 M3 /M4 /M7 芯片为了精简设计，只使用了高四位[7:4]，低四位取零，这样以至于最多只有 16 级中断嵌套，即 2^4=16。

对于 NVCI 的中断优先级分组：STM32H743 将中断分为 5 个组，组 0~4。该分组的设置是由 SCB->AIRCR 寄存器的 bit10~8 来定义的。具体的分配关系如表 15.1.1.2.1 所示：

优先级分组	AIRCR[10：8]	bit[7：4]分配情况	分配结果
0	111	0：4	0 位抢占优先级，4 位响应优先级
1	110	1：3	1 位抢占优先级，3 位响应优先级
2	101	2：2	2 位抢占优先级，2 位响应优先级
3	100	3：1	3 位抢占优先级，1 位响应优先级
4	011	4：0	4 位抢占优先级，0 位响应优先级

表 15.1.1.2.1 AIRCR 中断分组设置表

通过这个表，我们就可以清楚的看到组 0~4 对应的配置关系，例如优先级分组设置为 3，那么此时所有的 150 个中断，每个中断的中断优先寄存器的高四位中的最高 3 位是抢占优先级，低 1 位是响应优先级。每个中断，你可以设置抢占优先级为 0~7，响应优先级为 1 或 0。抢占优先级的级别高于响应优先级。而数值越小所代表的优先级就越高。

结合实例说明一下：假定设置中断优先级分组为 2，然后设置中断 3(RTC_WKUP 中断)的抢占优先级为 2，响应优先级为 1。中断 6（外部中断 0）的抢占优先级为 3，响应优先级为 0。中断 7（外部中断 1）的抢占优先级为 2，响应优先级为 0。那么这 3 个中断的优先级顺序为：中断 7>中断 3>中断 6。

上面例子中的中断 3 和中断 7 都可以打断中断 6 的中断。而中断 7 和中断 3 却不可以相互打断！

15.1.1.3 NVIC 相关函数

ST 公司把 core_cm7.h 文件的 NVIC 相关函数封装到 stm32h7xx_hal_cortex.c 文件中，下面列出我们较为常用的函数进行，想了解更多其他的函数请自行查阅。

1. HAL_NVIC_SetPriorityGrouping 函数

HAL_NVIC_SetPriorityGrouping 是设置中断优先级分组函数。其声明如下：

```
void HAL_NVIC_SetPriorityGrouping(uint32_t PriorityGroup);
```

- **函数描述：**

  用于设置中断优先级分组。

- **函数形参：**

  形参 1 是中断优先级分组号，可以选择范围：NVIC_PRIORITYGROUP_0 到 NVIC_PRIORITYGROUP_4（共 5 组）。

- **函数返回值：**无

- **注意事项：**

  这个函数在一个工程里基本只调用一次，而且是在程序 HAL 库初始化函数里面已经被调用，后续就不会再调用了。因为当后续调用设置成不同的中断优先级分组时，有可能造成前面设置好的抢占优先级和响应优先级不匹配。

2. HAL_NVIC_SetPriority 函数

HAL_NVIC_SetPriority 是设置中断优先级函数。其声明如下：

```
void HAL_NVIC_SetPriority(IRQn_Type IRQn, uint32_t PreemptPriority,
                          uint32_t SubPriority);
```

- **函数描述：**

  用于设置中断的抢占优先级和响应优先级（子优先级）。

- **函数形参：**

  形参 1 是中断号，可以选择范围：IRQn_Type 定义的枚举类型，定义在 stm32h743xx.h。

形参 2 是抢占优先级，可以选择范围：0 到 15。

形参 3 是响应优先级，可以选择范围：0 到 15。

- **函数返回值：**无

3. HAL_NVIC_EnableIRQ 函数

HAL_NVIC_EnableIRQ 是中断使能函数。其声明如下：

```
void HAL_NVIC_EnableIRQ(IRQn_Type IRQn);
```

- **函数描述：**

用于使能中断。

- **函数形参：**

形参 1 是中断号，可以选择范围：IRQn_Type 定义的枚举类型，定义在 stm32h743xx.h。

- **函数返回值：**无

4. HAL_NVIC_DisableIRQ 函数

HAL_NVIC_DisableIRQ 是中断除能函数。其声明如下：

```
void HAL_NVIC_disableIRQ(IRQn_Type IRQn);
```

- **函数描述：**

用于中断除能。

- **函数形参：**无形参
- **函数返回值：**无

5. HAL_NVIC_SystemReset 函数

HAL_NVIC_SystemReset 是系统复位函数。其声明如下：

```
void HAL_NVIC_SystemReset(void);
```

- **函数描述：**

用于软件复位系统。

- **函数形参：**无形参
- **函数返回值：**无

其他的 NVIC 函数用得较少，我们就不一一列出来了。NVIC 的介绍就到这，下面介绍外部中断。

15.1.2 EXTI 简介

EXTI 即是外部中断和事件控制器，它由三个部分组成：APB 接口访问的寄存器模块、事件输入触发模块和屏蔽模块。其中寄存器块包含了所有 EXTI 寄存器，事件输入触发模块提供事件输入边沿触发逻辑，屏蔽模块为不同的唤醒、中断和事件输出及其屏蔽功能提供事件分配。EXTI 的功能框图如下图所示：

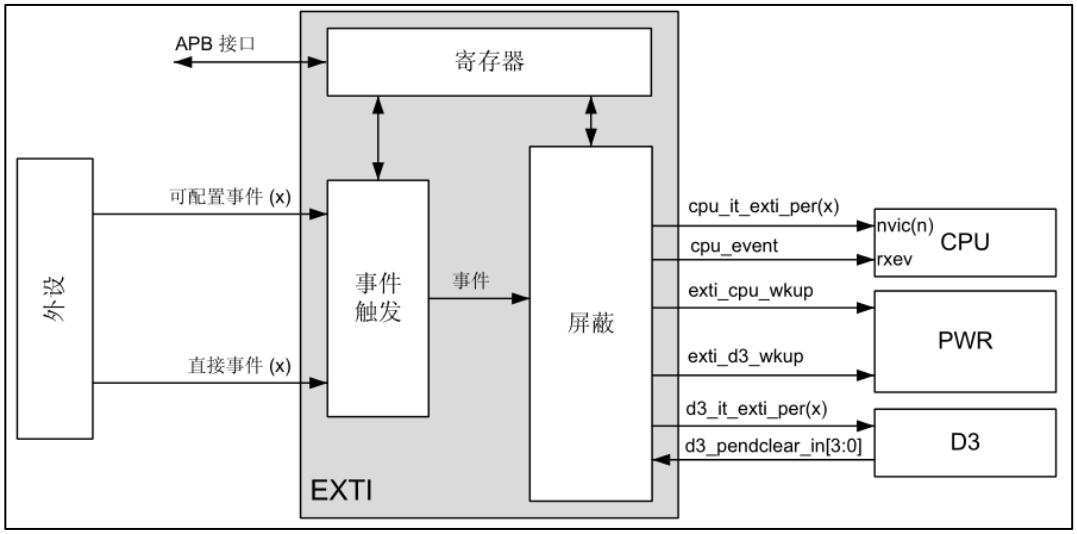


图 15.1.2.1 EXTI 功能框图

由外设对 EXTI 产生的事件分为可配置事件和直接事件。

可配置事件是来自能够生成脉冲的 IO 或外设的信号，这类事件具有多种特性：可选择的

有效触发边沿、中断挂起状态寄存器位、单独的中断和事件生成屏蔽、支持软件触发、可配置系统 D3 域唤醒事件。

直接事件是来自其它外设的中断和唤醒源，需要在外设中清除，这类事件具备特性：固定上升沿有效触发、EXTI 中无中断挂起状态寄存器位、单独的中断和实际生产屏蔽、不支持软件触发、直接系统 D3 域唤醒事件包含一个 D3 域挂起屏蔽和状态寄存器。

EXTI 支持 88 个中断/事件请求，即包括可配置事件和直接事件。每个中断设有状态位，每个中断/事件都有独立的屏蔽设置，其中 26 个可配置的输入事件。前 23 个中断/事件如下表：

事件输入	源	事件输入类型	唤醒目标	连接至 NVIC
0 - 15	EXTI[15:0]	可配置	任意	有
16	PVD 和 AVD(1)	可配置	仅限 CPU	有
17	RTC 闹钟	可配置	仅限 CPU	有
18	RTC 入侵、RTC 时间戳、RCC LSECSS(2)	可配置	仅限 CPU	有
19	RTC 唤醒定时器	可配置	任意	有
20	COMP1	可配置	任意	有
21	COMP2	可配置	任意	有
22	I2C1 唤醒	直接	仅限 CPU	有

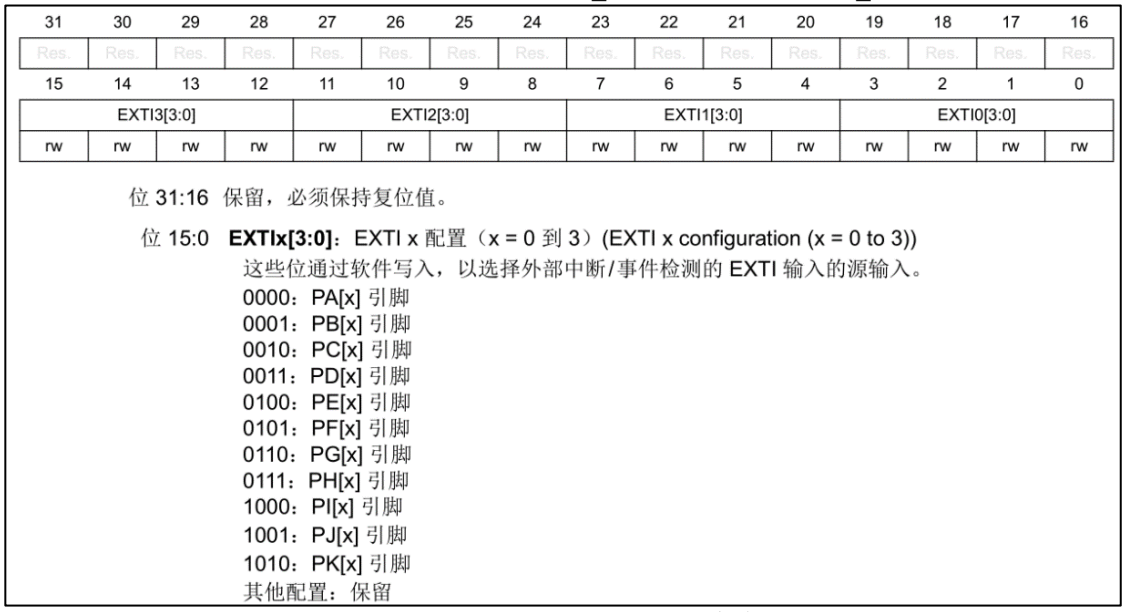
表 15.1.2.1 EXTI 事件输入映射

由于表格太长，其余的请大家参考《STM32H7xx 参考手册_V3（中文版）.pdf》20.4 小节的 EXTI 事件输入映射表格。

从上表可以看出，普通的 GPIO 口事件输入，属于可配置事件，唤醒目标为任意，比如 CPU NVIC 中断控制器，或者连接至 D3 低功耗域，产生唤醒中断。

事件输入 0-15 对应外部 IO 口的输入中断，一共是 16 个外部中断线。STM32H7 供 IO 口使用的中断线只有 16 个，但是 STM32H7 的 IO 口却远远不止 16 个，那么 STM32H7 是怎么把 16 个中断线和 IO 口一一对应起来的呢？于是 STM32 就这样设计，GPIO 的引脚 GPIOx.0 ~ GPIOx.15(x=A，B，C，D，E，F，G，H，I，J，K)分别对应中断线 0~15。这样每个中断线对应了最多 11 个 IO 口，以线 0 为例：它对应了 GPIOA.0、GPIOB.0、GPIOC.0、GPIOD.0、GPIOE.0、GPIOF.0、GPIOG.0、GPIOH.0、GPIOI.0、GPIOJ.0、GPIOK.0。而中断线每次只能连接到 1 个 IO 口上，这样就需要通过配置来决定对应的中断线配置到哪个 GPIO 上了。

GPIO 和中断线映射关系是在寄存器 SYSCFG_EXTICR1 ~ SYSCFG_EXTICR4 中配置的。



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EXTI3[3:0]				EXTI2[3:0]				EXTI1[3:0]				EXTI0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:16 保留，必须保持复位值。

位 15:0 **EXTIx[3:0]**：EXTI x 配置（x = 0 到 3）(EXTI x configuration (x = 0 to 3))
这些位通过软件写入，以选择外部中断/事件检测的 EXTI 输入的源输入。
0000：PA[x] 引脚
0001：PB[x] 引脚
0010：PC[x] 引脚
0011：PD[x] 引脚
0100：PE[x] 引脚
0101：PF[x] 引脚
0110：PG[x] 引脚
0111：PH[x] 引脚
1000：PI[x] 引脚
1001：PJ[x] 引脚
1010：PK[x] 引脚
其他配置：保留

图 15.1.2.2 SYSCFG_EXTICR1 寄存器

SYSCFG_EXTICR1 寄存器配置 EXTI0 到 EXTI3 线，包含的外部中断的引脚包括 PAx 到 PKx，x=0 到 3。SYSCFG_EXTICR2 寄存器配置 EXTI4 到 EXTI7 线，包含的外部中断的引脚包括 PAx 到 PGx，x=4 到 7，SYSCFG_EXTICR2 寄存器请打开参考手册查看（这里没有截图出来）。SYSCFG_EXTICR3 和 SYSCFG_EXTICR4 以此类推。

另外要注意的是，我们配置 SYSCFG 相关寄存器前，还需要打开 SYSCFG 时钟。

15.2 硬件设计

1. 例程功能

通过外部中断的方式让开发板上的四个独立按键控制 LED 灯：KEY0 控制 LED0/LED1 同时翻转，KEY1 控制 LED1 翻转，KEY2 控制 LED0 翻转，KEY_UP 控制 LED0/LED1 互斥点亮。

2. 硬件资源

1）LED 灯

LED0 – PB1
LED1 – PB0

2）独立按键

KEY0 – PH3
KEY1 – PH2
KEY2 – PC13
KEY_UP – PA0

3. 原理图

独立按键硬件部分的原理图，如图 15.2.1 所示：

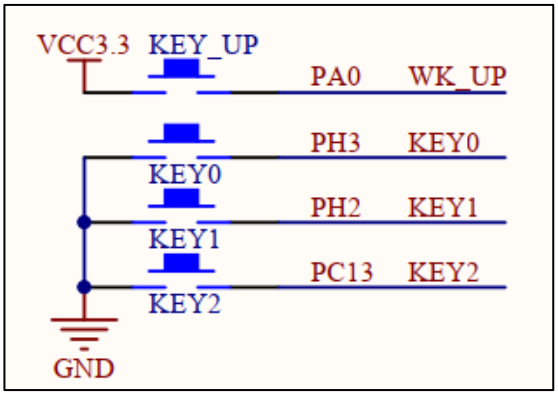


图 15.2.1 独立按键与 STM32H7 连接原理图

这里需要注意的是：KEY0、KEY1 和 KEY2 设计为采样到按键另一端的低电平为有效，并且外部都没有上下拉电阻，所以需要在 STM32H743 内部设置上下拉以设置空闲电平。

15.3 程序设计

15.3.1 EXTI 的 HAL 库驱动

前面讲解 HAL_GPIO_Init 函数的时候有提到过：HAL 库的 EXTI 外部中断的设置功能整合到 HAL_GPIO_Init 函数里面，而不是单独独立一个文件。所以我们的外部中断的初始化函数也是用 HAL_GPIO_Init 函数。

既然是要用到外部中断，所以我们的 GPIO 的模式要从下面的三个模式中选中一个：

```
#define  GPIO_MODE_IT_RISING           (0x11110000U)/* 外部中断，上升沿触发检测 */
#define  GPIO_MODE_IT_FALLING          (0x11210000U)/* 外部中断，下降沿触发检测 */
/* 外部中断，上升和下降双沿触发检测 */
#define  GPIO_MODE_IT_RISING_FALLING   (0x11310000U)
```

程序设计按键为按下触发中断，KEY0、KEY1 和 KEY2 是低电平有效，所以我们要选择下降沿触发检测，而 KEY_UP 是高电平有效，所以我们要选择上升沿触发。

EXTI 外部中断配置步骤

1） 使能 IO 口时钟。

本实验用到的 GPIO 和按键输入实验是一样的，因此 GPIO 时钟使能也是一样的，请参考上一章代码。

2） 设置 IO 口模式，触发条件，开启 SYSCFG 时钟，设置 IO 口与中断线的映射关系。

这些步骤 HAL 库全部封装在 HAL_GPIO_Init 函数里面，我们只需要设置好对应的参数，再调用 HAL_GPIO_Init 函数即可完成配置。

3）配置中断优先级（NVIC），并使能中断。

配置好 GPIO 模式以后，我们需要设置中断优先级和使能中断，中断优先级我们使用 HAL_NVIC_SetPriority 函数设置，中断使能我们使用 HAL_NVIC_EnableIRQ 函数设置。

4） 编写中断服务函数。

每开启一个中断，就必须编写其对应的中断服务函数，否则将导致死机（CPU 将找不到中断服务函数）。中断服务函数接口厂家已经在 startup_stm32h743xx.s 中做好了。STM32H7 的 IO 口外部中断函数只有 7 个，分别为：

```
void EXTI0_IRQHandler();
void EXTI1_IRQHandler();
void EXTI2_IRQHandler();
void EXTI3_IRQHandler();
void EXTI4_IRQHandler();
void EXTI9_5_IRQHandler();
void EXTI15_10_IRQHandler();
```

中断线 0-4，每个中断线对应一个中断函数，中断线 5-9 共用中断函数 EXTI9_5_IRQHandler，中断线 10-15 共用中断函数 EXTI15_10_IRQHandler。一般情况下，我们可以把中断控制逻辑直接编写在中断服务函数中，但是 HAL 库把中断处理过程进行了简单封装，请看下面步骤 5 讲解。

5）编写中断处理回调函数 HAL_GPIO_EXTI_Callback

HAL 库为了用户使用方便，提供了一个中断通用入口函数 HAL_GPIO_EXTI_IRQHandler，在该函数内部直接调用回调函数 HAL_GPIO_EXTI_Callback。

我们先来看一下 HAL_GPIO_EXTI_IRQHandler 函数定义：

```
void HAL_GPIO_EXTI_IRQHandler(uint16_t GPIO_Pin)
{
  if(__HAL_GPIO_EXTI_GET_IT(GPIO_Pin) != 0x00U)
  {
    __HAL_GPIO_EXTI_CLEAR_IT(GPIO_Pin);     /* 清中断标志位 */
    HAL_GPIO_EXTI_Callback(GPIO_Pin);       /* 外部中断回调函数 */
  }
}
```

该函数实现的作用非常简单，通过入口参数 GPIO_Pin 判断中断来自哪个 IO 口，然后清除相应的中断标志位，最后调用回调函数 HAL_GPIO_EXTI_Callback()实现控制逻辑。在所有的外部中断服务函数中直接调用外部中断共用处理函数 HAL_GPIO_EXTI_IRQHandler，然后在回调函数 HAL_GPIO_EXTI_Callback 中通过判断中断是来自哪个 IO 口编写相应的中断服务控制逻辑。

因此我们可以在 HAL_GPIO_EXTI_Callback 里面实现控制逻辑编写，详见本实验源码。

15.3.2 程序流程图

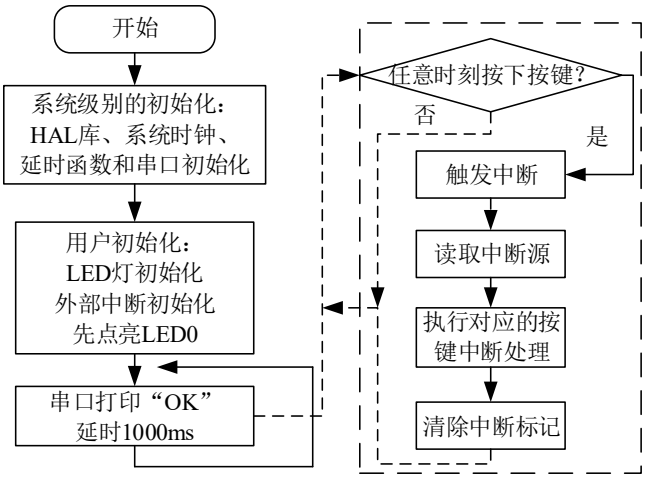


图 15.3.2.1 外部中断实验程序流程图

主程序初始外设，在按键初始化时初始化按键的采样边缘。

15.3.3 程序解析

1. 外部中断驱动代码

下面我们先解析 exti.h 的程序。

- **外部中断引脚定义**

由硬件设计小节，我们知道 KEY0、KEY1、KEY2 和 KEY_UP 分别来连接到 PH3、PH2、PC13 和 PA0 上，我们做了下面的引脚定义。

```
/* 引脚 和 中断编号 & 中断服务函数 定义 */
#define KEY0_INT_GPIO_PORT              GPIOH
#define KEY0_INT_GPIO_PIN               GPIO_PIN_3
#define KEY0_INT_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOH_CLK_ENABLE(); }while(0)    /* PH 口时钟使能 */
#define KEY0_INT_IRQn                   EXTI3_IRQn
#define KEY0_INT_IRQHandler             EXTI3_IRQHandler

#define KEY1_INT_GPIO_PORT              GPIOH
#define KEY1_INT_GPIO_PIN               GPIO_PIN_2
#define KEY1_INT_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOH_CLK_ENABLE(); }while(0)    /* PH 口时钟使能 */
#define KEY1_INT_IRQn                   EXTI2_IRQn
#define KEY1_INT_IRQHandler             EXTI2_IRQHandler

#define KEY2_INT_GPIO_PORT              GPIOC
#define KEY2_INT_GPIO_PIN               GPIO_PIN_13
#define KEY2_INT_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOC_CLK_ENABLE(); }while(0)    /* PC 口时钟使能 */
#define KEY2_INT_IRQn                   EXTI15_10_IRQn
#define KEY2_INT_IRQHandler             EXTI15_10_IRQHandler

#define WKUP_INT_GPIO_PORT              GPIOA
#define WKUP_INT_GPIO_PIN               GPIO_PIN_0
#define WKUP_INT_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)    /* PA 口时钟使能 */
#define WKUP_INT_IRQn                   EXTI0_IRQn
#define WKUP_INT_IRQHandler             EXTI0_IRQHandler
```

KEY0、KEY1、KEY2 和 KEY_UP 分别来连接到 PH3、PH2、PC13 和 PA0，即对应了 EXTI3、EXTI2、EXTI15_10_IRQn 和 EXTI0 这四条外部中断线。这里需要注意的是 EXTI0 到 EXTI4 都是有单独的中断向量，EXTI5 到 EXTI9 是公用 EXTI9_5_IRQn，EXTI10 到 EXTI15 是

公用 EXTI15_10_IRQn。

下面我们再解析 exti.c 的程序，先看外部中断初始化函数，其定义如下：

```
/**
 * @brief       外部中断初始化程序
 * @param       无
 * @retval      无
 */
void extix_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;

    key_init();
    gpio_init_struct.Pin = KEY0_INT_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_IT_FALLING;              /* 下降沿触发 */
    gpio_init_struct.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(KEY0_INT_GPIO_PORT, &gpio_init_struct);
    /* KEY0 配置为下降沿触发中断 */

    gpio_init_struct.Pin = KEY1_INT_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_IT_FALLING;              /* 下降沿触发 */
    gpio_init_struct.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(KEY1_INT_GPIO_PORT, &gpio_init_struct);
    /* KEY1 配置为下降沿触发中断 */

    gpio_init_struct.Pin = KEY2_INT_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_IT_FALLING;              /* 下降沿触发 */
    gpio_init_struct.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(KEY2_INT_GPIO_PORT, &gpio_init_struct);
    /* KEY2 配置为下降沿触发中断 */

    gpio_init_struct.Pin = WKUP_INT_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_IT_RISING;               /* 上升沿触发 */
    gpio_init_struct.Pull = GPIO_PULLDOWN;
    HAL_GPIO_Init(WKUP_GPIO_PORT, &gpio_init_struct);
    /* WKUP 配置为上升沿触发中断 */

    HAL_NVIC_SetPriority(KEY0_INT_IRQn, 0, 2);          /* 抢占 0，子优先级 2 */
    HAL_NVIC_EnableIRQ(KEY0_INT_IRQn);                  /* 使能中断线 3 */

    HAL_NVIC_SetPriority(KEY1_INT_IRQn, 1, 2);          /* 抢占 1，子优先级 2 */
    HAL_NVIC_EnableIRQ(KEY1_INT_IRQn);                  /* 使能中断线 2 */

    HAL_NVIC_SetPriority(KEY2_INT_IRQn, 2, 2);          /* 抢占 2，子优先级 2 */
    HAL_NVIC_EnableIRQ(KEY2_INT_IRQn);                  /* 使能中断线 13 */

    HAL_NVIC_SetPriority(WKUP_INT_IRQn, 3, 2);          /* 抢占 3，子优先级 2 */
    HAL_NVIC_EnableIRQ(WKUP_INT_IRQn);                  /* 使能中断线 0 */
}
```

外部中断初始化函数主要做了两件事情，先是调用 IO 口初始化函数 HAL_GPIO_Init 来初始化 IO 口，然后设置中断优先级并使能中断线。

4 个外部中断服务函数，其定义如下：

```
/**
 * @brief       外部中断服务程序
 * @param       无
 * @retval      无
 */
void KEY0_INT_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(KEY0_INT_GPIO_PIN);
    /* 调用中断处理公用函数 清除 KEY0 所在中断线 的中断标志位 */
```

```
    __HAL_GPIO_EXTI_CLEAR_IT(KEY0_INT_GPIO_PIN);
    /* HAL 库默认先清中断再处理回调，退出时再清一次中断，避免按键抖动误触发 */
}

void KEY1_INT_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(KEY1_INT_GPIO_PIN);
    /* 调用中断处理公用函数 清除 KEY1 所在中断线 的中断标志位，中断下半部在
    HAL_GPIO_EXTI_Callback 执行 */
    __HAL_GPIO_EXTI_CLEAR_IT(KEY1_INT_GPIO_PIN);
    /* HAL 库默认先清中断再处理回调，退出时再清一次中断，避免按键抖动误触发 */
}

void KEY2_INT_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(KEY2_INT_GPIO_PIN);
     /* 调用中断处理公用函数 清除 KEY2 所在中断线 的中断标志位，
        中断下半部在 HAL_GPIO_EXTI_Callback 执行 */
    __HAL_GPIO_EXTI_CLEAR_IT(KEY2_INT_GPIO_PIN);
    /* HAL 库默认先清中断再处理回调，退出时再清一次中断，避免按键抖动误触发 */
}

void WKUP_INT_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(WKUP_INT_GPIO_PIN);
    /* 调用中断处理公用函数 清除 KEY_UP 所在中断线 的中断标志位，
        中断下半部在 HAL_GPIO_EXTI_Callback 执行 */
    __HAL_GPIO_EXTI_CLEAR_IT(WKUP_INT_GPIO_PIN);
    /* HAL 库默认先清中断再处理回调，退出时再清一次中断，避免按键抖动误触发 */
}
```

所有的外部中断服务函数里都只调用了同样一个函数 HAL_GPIO_EXTI_IRQHandler，该函数是外部中断共用入口函数，函数内部会进行中断标志位清零， 并且调用中断处理共用回调函数 HAL_GPIO_EXTI_Callback。但是它们的形参不同，我们的回调函数也是根据形参去判断是哪个 IO 口的外部中断线被触发。

外部中断回调函数，其定义如下：

```
/**
 * @brief       外部中断回调函数
 * @param       GPIO_Pin: 中断引脚号
 * @note        在 HAL 库中所有的外部中断服务函数都会调用此函数
 * @retval      无
 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    delay_ms(20);       /* 消抖 */
    switch(GPIO_Pin)
    {
        case KEY0_INT_GPIO_PIN:
            if (KEY0 == 0)
            {
                LED1_TOGGLE();  /* LED1 状态取反 */
                LED0_TOGGLE();  /* LED0 状态取反 */
            }
            break;

        case KEY1_INT_GPIO_PIN:
            if (KEY1 == 0)
            {
                LED1_TOGGLE();  /* LED1 状态取反 */
            }
            break;

        case KEY2_INT_GPIO_PIN:
```

```
            if (KEY2 == 0)
            {
                LED0_TOGGLE();  /* LED0 状态取反 */
            }
            break;

        case WKUP_INT_GPIO_PIN:
            if (WK_UP == 1)
            {
                 LED1_TOGGLE(); /* LED1 状态取反 */
                 if(HAL_GPIO_ReadPin(LED1_GPIO_PORT, LED1_GPIO_PIN) == 1)
                 {
                   LED0(0);
                 }
                 else LED0(1);
            }
            break;

        default : break;
    }
}
```

外部中断回调函数 HAL_GPIO_EXTI_Callback 是用来编写真正的外部中断控制逻辑。该函数有一个形参就是 IO 引脚号。所以我们在该函数内部，一般通过判断 IO 引脚号来确定中断是来自哪个 IO 口，也就是哪个中断线，然后编写相应的控制逻辑。所以在该函数内部，我们通过 switch 语句判断 IO 口来源，例如是来自 GPIO_PIN_0，那么一定是来自 PA0，因为中断线一次只能连接一个 IO 口，而四个 IO 口中引脚号为 0 的 IO 口只有 PA0，所以中断线 0 一定是连接 PA0，也就是外部中断由 PA0 触发。其他的引脚号的逻辑类似。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 初始化 USART */
    led_init();                             /* 初始化 LED */
    extix_init();                           /* 初始化外部中断输入 */
    LED0(0);                                /* 先点亮红灯 */

    while (1)
    {
        printf("OK\r\n");
        delay_ms(1000);
    }
}
```

首先是调用系统级别的初始化：sys_cache_enable 函数使能 I-Cache 和 D-Cache，然后初始化 HAL 库、系统时钟和延时函数。接下来，调用 led_init 来初始化 LED 灯，调用 extix_init 函数初始化外部中断，点亮红灯。最后在无限循环里面执行 1000ms 的延时函数。逻辑控制代码都在中断回调函数中完成。

15.4 下载验证

在下载好程序后，我们可以按 KEY0、KEY1、KEY2 和 KEY_UP 来看看 LED 灯的变化，是否和我们预期的结果一致？

至此，我们的本章的学习就结束了。本章学习了 STM32H743 外部中断的使用方法。

第十六章 串口通信实验

本章我们将学习 STM32H7 的串口，教大家如何使用 STM32H7 的串口来发送和接收数据。本章将实现如下功能：STM32H7 通过串口和上位机的对话，STM32H7 在收到上位机发过来的字符串后，原原本本的返回给上位机。

本章分为如下几个小节：

16.1 串口简介
16.2 硬件设计
16.3 程序设计
16.4 下载验证

16.1 串口简介

学习串口前，我们先来了解一下数据通信的一些基础概念。

16.1.1 数据通信的基础概念

在单片机的应用中，数据通信是必不可少的一部分，比如：单片机和上位机、单片机和外围器件之间，它们都有数据通信的需求。由于设备之间的电气特性、传输速率、可靠性要求各不相同，于是就有了各种通信类型、通信协议，我们常见的有：USART、IIC、SPI、CAN、USB 等。下面，我们先来学习数据通信的一些基础概念。

1. 数据通信方式

按数据通信方式分类，可分为串行通信和并行通信两种。串行和并行的对比如下图所示：

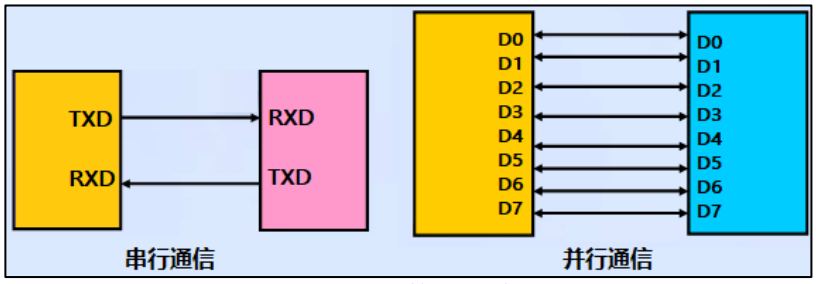


图 16.1.1.1 数据传输方式

串行通信的基本特征是数据逐位顺序依次传输，优点是传输线少成本低，抗干扰能力强可用于远距离传输，缺点就是传输速率低。

而并行通信是数据各位可以通过多条线同时传输，优点是传输速率高，缺点就是线多成本就高了，抗干扰能力差因而适用于短距离、高速率的通信。

2. 数据传输方向

根据数据传输方向，通信又可分为全双工、半双工和单工通信。全双工、半双工和单工通信的比较如下图所示：

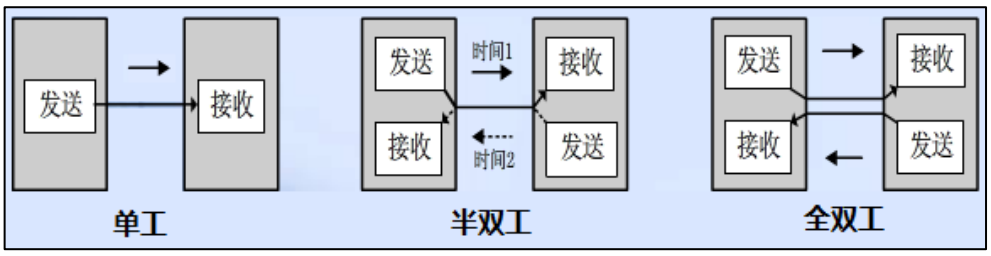


图 16.1.1.2 数据传输方式

单工是指数据传输仅能沿一个方向，不能实现反方向传输。

半双工是指数据传输可以沿着两个方向，但是需要分时进行。

全双工是指数据可以同时进行双向传输。

这里注意全双工和半双工通信的区别：半双工通信是共用一条线路实现双向通信，而全双工是利用两条线路，一条用于发送数据，另一条用于接收数据。

3. 数据同步方式

根据数据同步方式，通信又可分为同步通信和异步通信。同步通信和异步通信比较如下图所示：

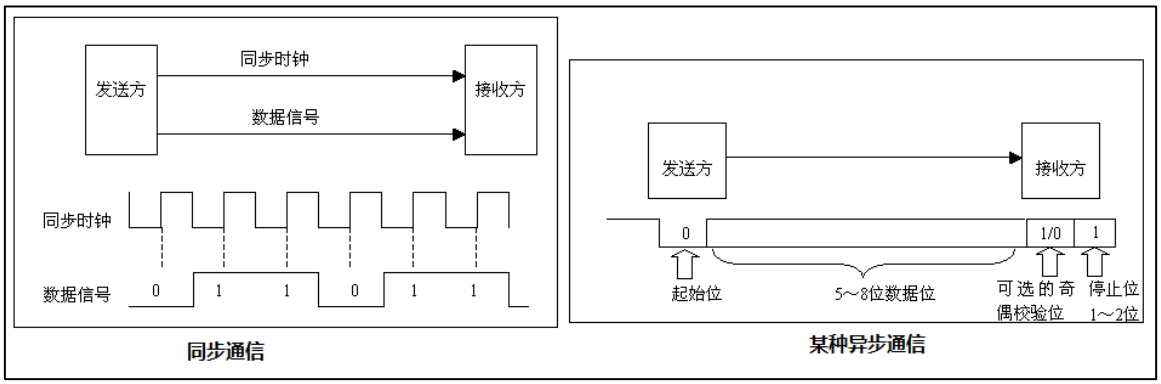


图 16.1.1.3 数据同步方式

同步通信要求通信双方共用同一时钟信号，在总线上保持统一的时序和周期完成信息传输。优点：可以实现高速率、大容量的数据传输，以及点对多点传输。缺点：要求发送时钟和接收时钟保持严格同步，收发双方时钟允许的误差较小，同时硬件复杂。

异步通信不需要时钟信号，而是在数据信号中加入开始位和停止位等一些同步信号，以便使接收端能够正确地将每一个字符接收下来，某些通信中还需要双方约定传输速率。优点：没有时钟信号硬件简单，双方时钟可允许一定误差。缺点：通信速率较低，只适用点对点传输。

4. 通信速率

在数字通信系统中，通信速率（传输速率）指数据在信道中传输的速度，它分为两种：传信率和传码率。

传信率：每秒钟传输的信息量，即每秒钟传输的二进制位数，单位为 bit/s（即比特每秒），因而又称为比特率。

传码率：每秒钟传输的码元个数，单位为 Baud（即波特每秒），因而又称为波特率。

比特率和波特率这两个概念又常常被人们混淆。比特率很好理解，我们来看看波特率，波特率被传输的是码元，码元是信号被调制后的概念，每个码元都可以表示一定 bit 的数据信息量。举个例子，在 TTL 电平标准的通信中，用 0V 表示逻辑 0，5V 表示逻辑 1，这时候这个码元就可以表示两种状态。如果电平信号 0V、2V、4V 和 6V 分别表示二进制数 00、01、10、11，这时候每一个码元就可以表示四种状态。

由上述可以看出，码元携带一定的比特信息，所以比特率和波特率也是有一定的关系的。

比特率和波特率的关系可以用以下式子表示：

$$比特率 = 波特率 * \log_2 M$$

其中 M 表示码元承载的信息量。我们也可以理解 M 为码元的进制数。

举个例子：波特率为 100 Baud，即每秒传输 100 个码元，如果码元采用十六进制编码（即 M=2，代入上述式子），那么这时候的比特率就是 400 bit/s。如果码元采用二进制编码（即 M=2，代入上述式子），那么这时候的比特率就是 100 bit/s。

可以看出采用二进制的时候，波特率和比特率数值上相等。但是这里要注意，它们的相等只是数值相等，其意义上不同，看波特率和波特率单位就知道。由于我们的所用的数字系统都是二进制的，所以有部分人久而久之就直接把波特率和比特率混淆了。

16.1.2 串口通信协议简介

串口通信是一种设备间常用的串行通信方式，串口按位（bit）发送和接收字节。尽管比特字节（byte）的串行通信慢，但是串口可以在使用一根线发送数据的同时用另一根线接收数据。**串口通信协议**是指规定了数据包的内容，内容包含了起始位、主体数据、校验位及停止位，双方需要约定一致的数据包格式才能正常收发数据的有关规范。在串口通信中，常用的协议包括 RS-232、RS-422 和 RS-485 等。

随着科技的发展，RS-232 在工业上还有广泛的使用，但是在商业技术上，已经慢慢的使用 USB 转串口取代了 RS-232 串口。我们只需要在电路中添加一个 USB 转串口芯片，就可以实现 USB 通信协议和标准 UART 串行通信协议的转换，而我们开发板上的 USB 转串口芯片是 CH340C 这个芯片。关于 USB 转串口芯片的原理图请看 16.2 小节。

下面我们来学习串口通信协议，这里主要学习串口通信的协议层。

串口通信的数据包由发送设备的 TXD 接口传输到接收设备的 RXD 接口。在串口通信的协议层中，规定了数据包的内容，它由启始位、主体数据、校验位以及停止位组成，通讯双方的数据包格式要约定一致才能正常收发数据，其组成如图 16.1.2.1 所示。

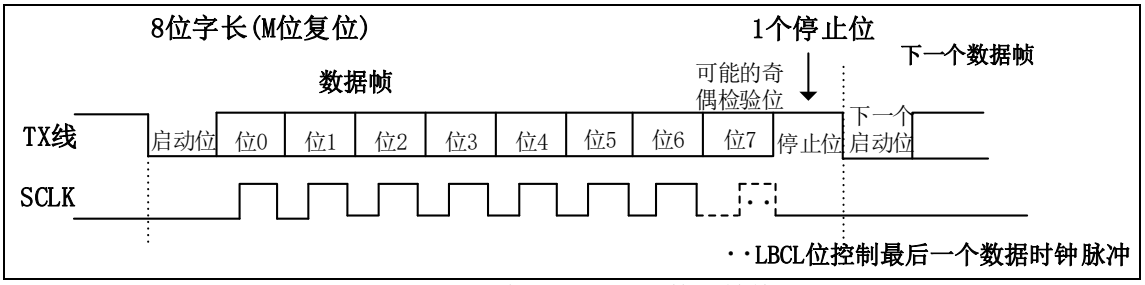


图 16.1.2.1 串口通信协议数据帧格式

串口通信协议数据包组成可以分为波特率和数据帧格式两部分。

1. 波特率

本章主要讲解的是串口异步通信，异步通信是不需要时钟信号的，但是这里需要我们约定好两个设备的波特率。波特率表示每秒钟传送的码元符号的个数，所以它决定了数据帧里面每一个位的时间长度。两个要通信的设备的波特率一定要设置相同，我们常见的波特率是 4800、9600、115200 等。

2. 数据帧格式

数据帧格式需要我们提前约定好，串口通信的数据帧包括起始位、停止位、有效数据位以及校验位。

- **起始位和停止位**

串口通信的一个数据帧是从起始位开始，直到停止位。数据帧中的起始位是由一个逻辑 0 的数据位表示，而数据帧的停止位可以是 0.5、1、1.5 或 2 个逻辑 1 的数据位表示，只要双方约定一致即可。

- **有效数据位**

数据帧的起始位之后，就接着是数据位，也称有效数据位，这就是我们真正需要的数据，有效数据位通常会被约定为 5、6、7 或者 8 个位长。有效数据位是低位（LSB）在前，高位（MSB）在后。

- **校验位**

校验位可以认为是一个特殊的数据位。校验位一般用来判断接收的数据位有无错误，检验方法有：奇检验、偶检验、0 检验、1 检验以及无检验。下面分别介绍一下：

奇校验是指有效数据为和校验位中“1”的个数为奇数，比如一个 8 位长的有效数据为：10101001，总共有 4 个“1”，为达到奇校验效果，校验位设置为“1”，最后传输的数据是 8 位的有效数据加上 1 位的校验位总共 9 位。

偶校验与奇校验要求刚好相反，要求帧数据和校验位中“1”的个数为偶数，比如数据帧：11001010，此时数据帧“1”的个数为 4 个，所以偶校验位为“0”。

0 校验是指不管有效数据中的内容是什么，校验位总为“0”，1 校验是校验位总为“1”。

无校验是指数据帧中不包含校验位。

我们一般是使用无校验的情况。

16.1.3 STM32H7 的串口简介

STM32H743 的串口资源相当丰富，功能也相当强劲。STM32H743IIT6 最多可提供 9 路串口（8 路普通串口和 1 路低功耗串口），支持 8/16 倍过采样、支持自动波特率检测、支持 Modbus 通信、支持同步单线通信和半双工单线通讯、支持 LIN、支持调制解调器操作、智能卡协议和 IrDA SIR ENDEC 规范、具有 DMA 等。

STM32H7 的 8 路普通串口分为两种：USART（即通用同步异步收发器）和 UART（即通用异步收发器）。UART 是在 USART 基础上裁剪掉了同步通信功能，只剩异步通信功能。简单区分同步和异步就看通信时需不需要对外提供时钟输出，我们平时用串口通信基本都是异步通信。

STM32H7 有四个 USART 和四个 UART，其中 USART1 和 USART6 的时钟源可以来于 APB2/PLL2_Q/PLL3_Q/HSI/CSI/LSE 时钟，其最大频率为 120MHz，其他六个串口的时钟源可以来于 APB1/PLL2_Q/PLL3_Q/HSI/CSI/LSE 时钟，其最大频率为 120MHz。

STM32 的串口输出的是 TTL 电平信号，如果需要 RS-232 标准的信号可使用 MAX3232 芯片进行转换，而本实验我们是通过 USB 转串口芯片来与电脑的上位机进行通信。

16.1.3.1 USART 框图

下面先来学习 USART 框图，通过 USART 框图引出 USART 的相关知识，从而有了一个很好的整体掌握，对之后的编程也会有一个清晰的思路。

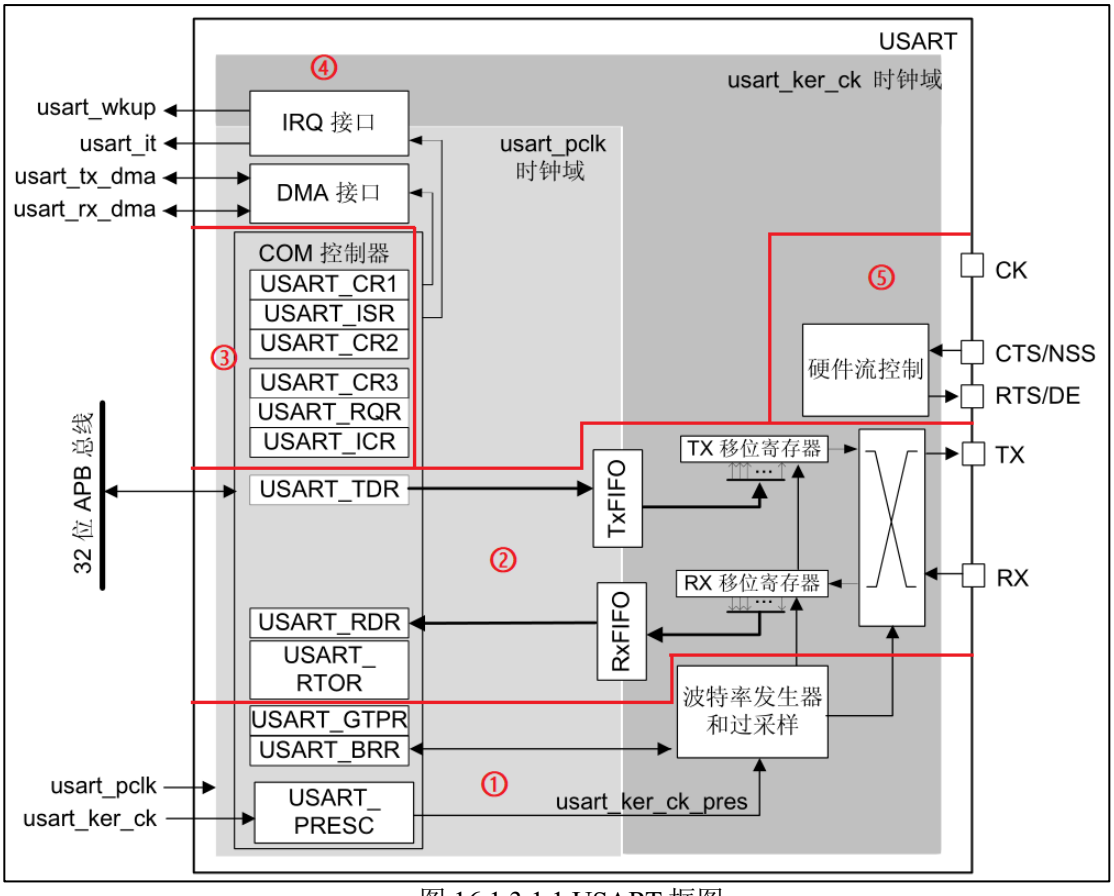


图 16.1.3.1.1 USART 框图

为了方便大家理解，我们把整个框图分成几个部分来介绍。

① 时钟与波特率

这部分的主要功能就是为 USART 提供时钟以及配置波特率。

在 USART 框图中，可以看到有两个时钟域，usart_pclk 时钟域和 usart_ker_ck 内核时钟域。usart_pclk 是外设总线时钟，需要访问 USART 寄存器时，该信号必须有效。usart_ker_ck 是 USART 时钟源，独立于 usart_pclk，由 RCC 提供。因此，即使 usart_ker_ck 时钟停止，也可以连续对 USART 寄存器进行读/写操作。

波特率，即每秒钟传输的码元个数，在二进制系统中（串口的数据帧就是二进制的形式），波特率与波特率的数值相等，所以我们今后在把串口波特率理解为每秒钟传输的二进制位数。波特率的计算公式分为 16 倍过采样和 8 倍过采样：

在 16 倍过采样的情况下，波特率通过以下公式得出：

$$baud = \frac{\text{usart_ker_ckpres}}{\text{USARTDIV}}$$

在 8 倍过采样的情况下，波特率通过以下公式得出：

$$baud = \frac{2 * \text{usart_ker_ckpres}}{\text{USARTDIV}}$$

usart_ker_ckpres 为 USART 的工作时钟，在 usart_ker_ck 不分频的情况下，时钟频率最高为 120MHz。USARTDIV 是一个无符号定点数，我们可以通过上述的式子得到，因为一般我们知道的是 baud 和 usart_ker_ck 的时钟。得到 USARTDIV 的值后，下一步就是要把该值设置到波特率寄存器 USART1->BRR 中，以完成波特率的设置。波特率的设置还得根据 OVER8 的值来确定格式。

OVER8 是 USART_CR1 寄存器的位 15，当 OVER8 = 0 时（即 16 倍过采样），BRR = USARTDIV；当 OVER8 = 1 时，BRR[2:0] = USARTDIV[3:0]，这里右移 1 位，BRR[3]必须保持清零，BRR[15:4] = USARTDIV[15:4]。无论是 16 倍采样还是 8 倍过采样，USARTDIV 都必须大于或等于 0d16。

下面举个例子说明：

当在 16 倍过采样时需要得到 115200 的波特率，usart_ker_ckpre = 100MHZ，那么可得：

$$115200 = \frac{100000000}{\text{USARTDIV}}$$

得到 USARTDIV = 868，可解得 BRR = USARTDIV = 868d = 0364h，那么需要设置 USART_BRR 的值为 0x364。这里的 USARTDIV 是有余数的，我们用四舍五入进行取整，这样会导致波特率会有所偏差，而这样的小误差是可以被允许的。8 倍过采样计算方法类似。

② 收发数据

USART 双向通信需要的两个引脚：

TX：发送数据输出引脚。

RX：接收数据输入引脚。

USART_TDR 是 USART 发送数据寄存器，要发送什么数据，往这个寄存器里写即可，低 9 位有效。

USART_RDR 是 USART 接收数据寄存器，要接收什么数据，读这个寄存器里写即可，低 9 位有效。

USART_TDR 和 USART_RDR 的第 9 位是否有效，通过 USART 控制寄存器 1(USART_CR1)的 M 位（M0：位 12，M1：位 28）设置：

7 位字符长度：M[1:0] = “10”

8 位字符长度：M[1:0] = “00”

9 位字符长度：M[1:0] = “01”

我们基本都是使用 8 位数据字长。

③ 控制寄存器

我们可以通过控制寄存器控制 USART 数据的发送、数据接收、各种通信模式的设置、中断、DMA 模式还有唤醒单元等。具体在后面讲解 USART 寄存器的时候细讲。

④ DMA 和中断功能

USART 支持 DMA 传输，可以实现高速数据传输，具体我们会在 DMA 实验中为大家讲解。在 USART 通信过程中，中断可由不同事件生成，同时支持 USART 模块生成唤醒中断。常用的中断比如：发送数据寄存器为空、发送 FIFO 未满、发送完成、接收 FIFO 非空、接收 FIFO 已满等。

⑤ **USART 信号引脚**

在 RS232 硬件流控制模式下需要以下两个引脚：

CTS（清除以发送）：发送器在发送下一帧数据之前会检测 CTS 引脚，如果为低电平，表示可以发送数据，如果为高电平则在发送完当前数据帧之后停止发送。

RTS（请求以发送）：如果为低电平，则该信号用于指示 USART 已准备好接收数据。

在 RS485 硬件控制模式下需要下面这个引脚：

DE（驱动器使能）：该信号用于激活外部收发器的发送模式。

在同步主/从模式和智能卡模式下需要以下引脚：

CK：该引脚在同步主模式和智能卡模式下用作时钟输出，在同步从模式下用作时钟输入。

NSS：该引脚在同步从模式下用作从器件选择输入。

这些引脚我们暂时都没有用到，就给大家简单提一下。

16.1.3.2 USART 寄存器

STM32H743 的串口使用起来还是蛮简单的，只要你开启了串口时钟，并设置相应 IO 口的模式，然后配置一下波特率，数据位长度，奇偶校验位等信息，就可以使用了。下面，我们就简单介绍下这几个与串口基本配置直接相关的寄存器。

（1）串口时钟使能

串口作为 STM32H743 的一个外设，其时钟由外设时钟使能寄存器控制，这里我们使用的串口 1 是在 APB2ENR 寄存器的第 4 位。注意：除了串口 1 和串口 6 的时钟使能在 APB2ENR 寄存器，其他串口的时钟使能位都在 APB1LENR 和 APB4ENR 寄存器。

（2）串口波特率设置

每个串口都有一个自己独立的波特率寄存器 USART_BRR，通过设置该寄存器就可以达到配置不同波特率的目的。波特率设置介绍过，请回顾。

（3）串口控制

STM32H743 的每个串口都有 3 个控制寄存器 USART_CR1~3，串口的很多配置都是通过这 3 个寄存器来设置的。USART_CR1 寄存器的描述如图 16.1.3.2.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RXFFIE	TXFEIE	FIFOEN	M1	EOBIE	RTOIE	DEAT[4:0]					DEDT[4:0]				
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	CMIE	MME	M0	WAKE	PCE	PS	PEIE	TXEIE/TXFNFIE	TCIE	RXNEIE/RXFNEIE	IDLEIE	TE	RE	UESM	UE
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 16.1.3.2.1 USART_CR1 寄存器

该寄存器我们只介绍本节需要用到的一些位：M[1:0]位(位 28 和 12)，用于设置字长，我们一般设置为：00 表示 1 个起始位，8 个数据位，n 个停止位（n 的个数，由 USART_CR2 的[13:12]位控制）。OVER8 为过采样模式设置位，我们一般设置位 0，即 16 倍过采样已获得更好的容错性；UE 为串口使能位，通过该位置 1，以使能串口；PCE 为校验使能位，设置为 0，则禁止校验，否则使能校验；PS 为校验位选择位，设置为 0 则为偶校验，否则为奇校验；TXEIE 为发送缓冲区空中断使能位，设置该位为 1，当 USART_ISR 中的 TXE 位为 1 时，将产生串口中断；TCIE 为发送完成中断使能位，设置该位为 1，当 USART_ISR 中的 TC 位为 1 时，将产生串口中断；RXNEIE 为接收缓冲区非空中断使能，设置该位为 1，当 USART_ISR 中的 ORE 或者 RXNE 位为 1 时，将产生串口中断；TE 为发送使能位，设置为 1，将开启串口的发送功能；RE 为接收使能位，用法同 TE。

其他位的设置，大家可以参考《STM32H7xx 参考手册_V7（英文版）.pdf》 。

（4）数据发送与接收

与 STM32F1 和 F4 不同，STM32H7 的串口发送和接收由两个不同的寄存器组成。发送数据是 USART_TDR 寄存器，接收数据是 USART_RDR 寄存器，USART_TDR 寄存器描述如图 16.1.3.2.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	TDR[8:0]								
							rw	rw	rw	rw	rw	rw	rw	rw	rw

位 8:0 **TDR[8:0]**：发送数据值 (Transmit data value)

包含要发送的数据字符。

USART_TDR 寄存器在内部总线和输出移位寄存器之间提供了并行接口（请参见*图 562*）。

在使能奇偶校验位的情况下（USART_CR1 寄存器中的 PCE 位被置 1）进行发送时，由于 MSB 的写入值（位 7 或位 8，具体取决于数据长度）会被奇偶校验位所取代，因此该值不起任何作用。

注：　只能在 TXE/TXFNF=“1”时写入此寄存器。

图 16.1.3.2.2 USART_TDR 寄存器

可以看出，USART_TDR 虽然是一个 32 位寄存器，但是只用了低 9 位（DR[8:0]），其他都是保留，TDR[8:0]为串口数据，具体多少位，由前面介绍的 M[1:0]决定（一般是 8 位数据）。

当我们需要发送数据的时候，往 USART_TDR 寄存器写入你想要发送的数据，就可以通过串口发送出去了。而当有串口数据接收到，需要读取出来的时候，我们则必须读取 USART_RDR 寄存器，USART_RDR 寄存器各位描述同 USART_TDR 是完全一样的，只是一个用来接收，一个用来发送。

当使能校验位(USART_CR1 中 PCE 位被置位)进行发送时，写到 MSB 的值(根据数据的长度不同，MSB 是第 7 位或者第 8 位)会被后来的校验位取代。

当使能校验位进行接收时，读到的 MSB 位是接收到的校验位。

（5）串口状态

串口状态通过状态寄存器 USART_ISR 读取。USART_ISR 的各位描述如图 16.1.3.2.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	TXFT	RXFT	TCBGT	RXFF	TXFE	RE ACK	TE ACK	WUF	RWU	SBKF	CMF	BUSY
				r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABRF	ABRE	UDR	EOBF	RTOF	CTS	CTSIF	LBDF	TXE/TX FNF	TC	RXNE/ RXFNE	IDLE	ORE	NE	FE	PE
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

图 16.1.3.2.3 USART_ISR 寄存器

这里我们关注一下两个位，第 5、6 位 RXNE 和 TC。

RXNE（读数据寄存器非空），当该位被置 1 的时候，就是提示已经有数据被接收到了，并且可以读出来了。这时候我们要做的就是尽快去读取 USART_RDR，通过读 USART_RDR 可以将该位清零，也可以向该位写 0，直接清除。

TC（发送完成），当该位被置位的时候，表示 USART_TDR 内的数据已经被发送完成了。如果设置了这个位的中断，则会产生中断。该位也有两种清零方式：

1）读 USART_ISR，写 USART_TDR。

2）向 ICR 寄存器的 TCCF 位写 1。

通过以上一些寄存器的操作再加上 IO 口的配置，我们就可以达到串口最基本的配置了，关于串口更详细的介绍，请参考《STM32H7xx 参考手册_V7（英文版）.pdf》通用同步异步收发器这一章。

16.1.4 GPIO 引脚复用功能

我们知道芯片有许多外设，而引脚的资源是很有限的，为了解决这个问题，方法就是引脚复用，这样使得引脚除了作为普通的 IO 口之外，还会与一些外设关联起来，作为第二功能使用，而且一个引脚不单单只有一种复用功能，而是拥有多个第二功能，但是一次只允许一个外设的复用功能，以确保共用同一个 IO 引脚的外设之间不会产生冲突。

下面我们把之前没讲解的 GPIO 复用功能寄存器讲解一下。

- **GPIO 复用功能寄存器（AFRH 和 AFRL）**

复用功能寄存器有 2 个，都是 32 位有效的寄存器，分高位（AFRH）和低位（AFRL）。复用器采用 16 路复用功能输入 AF0~AF15，通过 GPIOx_AFRL（引脚 0~7）、GPIOx_AFRH（引脚 8~15）寄存器对复用功能输入进行配置，每四位控制 1 路复用。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AFR7[3:0]				AFR6[3:0]				AFR5[3:0]				AFR4[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AFR3[3:0]				AFR2[3:0]				AFR1[3:0]				AFR0[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:0 **AFRy[3:0]**：端口 x 引脚 y 的复用功能选择 (Alternate function selection for port x pin y) (y = 0..7)

这些位通过软件写入，用于配置复用功能 I/O。

AFSELy 选择：

0000: AF0	0100: AF4	1000: AF8	1100: AF12
0001: AF1	0101: AF5	1001: AF9	1101: AF13
0010: AF2	0110: AF6	1010: AF10	1110: AF14
0011: AF3	0111: AF7	1011: AF11	1111: AF15

图 16.1.4.1 AFRL 寄存器

AFRL 寄存器配置引脚 0~7 复用功能，AFRH 寄存器配置引脚 8~15 复用功能。

IO 口并不是想复用什么功能都可以，是有规定的，每个 IO 引脚的复用可以通过查阅数据手册《STM32H743IIT6.pdf》，我们看看 Table 10. Port A alternate functions，其他的端口请自行查阅。

Port		AF0	AF1	AF2	AF3	AF4	AF5	AF6	AF7	AF8	AF9	AF10	AF11	AF12	AF13	AF14	AF15
		SYS	TIM1/2/16/17/LPTIM1/HRTIM1	SAI1/TIM3/4/5/12/HRTIM1	LPUART/TIM8/LPTIM2/3/4/5/HRTIM1/DFSDM1	I2C1/2/3/4/USART1/TIM15/LPTIM2/DFSDM1/CEC	SPI1/2/3/4/5/6/CEC	SPI2/3/SAI1/3/I2C4/UART4/DFSDM1	SPI2/3/6/USART1/2/3/6/UART7/SDMMC1	SPI6/SAI2/4/UART4/5/8/LPUART/SDMMC1/SPDIFRX1	SAI4/FDCAN1/2/TIM13/14/QUADSPI/FMC/SDMMC2/LCD/SPDIFRX1	SAI2/4/TIM8/QUADSPI/SDMMC2/OTG1_HS/OTG2_FS/LCD	I2C4/UART7/SWPMI1/TIM1/8/DFSDM1/SDMMC2/MDIOS/ETH	TIM1/8/FMC/SDMMC1/MDIOS/OTG1_FS/LCD	TIM1/DCMI/LCD/COMP	UART5/LCD	SYS
Port A	PA0	-	TIM2_CH1/TIM2_ETR	TIM5_CH1	TIM8_ETR	TIM15_BKIN	-	-	USART2_CTS/USART2_NSS	UART4_TX	SDMMC2_CMD	SAI2_SD_B	ETH_MII_CRS	-	-	-	EVENT-OUT
	PA1	-	TIM2_CH2	TIM5_CH2	LPTIM3_OUT	TIM15_CH1N	-	-	USART2_RTS/USART2_DE	UART4_RX	QUADSPI_BK1_IO3	SAI2_MCLK_B	ETH_MII_RX_CLK/ETH_RMII_REF_CLK	-	-	LCD_R2	EVENT-OUT
	PA2	-	TIM2_CH3	TIM5_CH3	LPTIM4_OUT	TIM15_CH1	-	-	USART2_TX	SAI2_SCK_B	-	-	ETH_MDIO	MDIOS_MDIO	-	LCD_R1	EVENT-OUT
	PA3	-	TIM2_CH4	TIM5_CH4	LPTIM5_OUT	TIM15_CH2	-	-	USART2_RX	-	LCD_B2	OTG_HS_ULPI_D0	ETH_MII_COL	-	-	LCD_B5	EVENT-OUT
	PA4	D1PWREN	-	TIM5_ETR	-	-	SPI1_NSS/I2S1_WS	SPI3_NSS/I2S3_WS	USART2_CK	SPI6_NSS	-	-	-	OTG_HS_SOF	DCMI_HSYNC	LCD_VSYNC	EVENT-OUT
	PA5	D2PWREN	TIM2_CH1/TIM2_ETR	-	TIM8_CH1N	-	SPI1_SCK/I2S1_CK	-	-	SPI6_SCK	-	OTG_HS_ULPI_CK	-	-	-	LCD_R4	EVENT-OUT
	PA6	-	TIM1_BKIN	TIM3_CH1	TIM8_BKIN	-	SPI1_MISO/I2S1_SDI	-	-	SPI6_MISO	TIM13_CH1	TIM8_BKIN_COMP12	MDIOS_MDC	TIM1_BKIN_COMP12	DCMI_PIXCLK	LCD_G2	EVENT-OUT
	PA7	-	TIM1_CH1N	TIM3_CH2	TIM8_CH1N	-	SPI1_MOSI/I2S1_SDO	-	-	SPI6_MOSI	TIM14_CH1	-	ETH_MII_RX_DV/ETH_RMII_CRS_DV	FMC_SDNWE	-	-	EVENT-OUT
	PA8	MCO1	TIM1_CH1	HRTIM_CHB2	TIM8_BKIN2	I2C3_SCL	-	-	USART1_CK	-	-	OTG_FS_SOF	UART7_RX	TIM8_BKIN2_COMP12	LCD_B3	LCD_R6	EVENT-OUT
	PA9	-	TIM1_CH2	HRTIM_CHC1	LPUART1_TX	I2C3_SMBA	SPI2_SCK/I2S2_CK	-	USART1_TX	-	-	-	-	-	DCMI_D0	LCD_R5	EVENT-OUT
	PA10	-	TIM1_CH3	HRTIM_CHC2	LPUART1_RX	-	-	-	USART1_RX	-	-	OTG_FS_ID	MDIOS_MDIO	LCD_B4	DCMI_D1	LCD_B1	EVENT-OUT
	PA11	-	TIM1_CH4	HRTIM_CHD1	LPUART1_CTS	-	SPI2_NSS/I2S2_WS	UART4_RX	USART1_CTS/USART1_NSS	-	FDCAN1_RX	OTG_FS_DM	-	-	-	LCD_R4	EVENT-OUT
	PA12	-	TIM1_ETR	HRTIM_CHD2	LPUART1_RTS/LPUART1_DE	-	SPI2_SCK/I2S2_CK	UART4_TX	USART1_RTS/USART1_DE	SAI2_FS_B	FDCAN1_TX	OTG_FS_DP	-	-	-	LCD_R5	EVENT-OUT

表 16.1.4.1 Port A 引脚复用

我们圈出来 PA9 和 PA10 对应 AF7 这列，因为我们的串口 1 就是用到这两个 IO 口，配置复用功能，使得 PA9 用做串口 1 的发送引脚 TX，PA10 则用做串口 1 的接收引脚 RX。

我们还需要学会在 HAL 库中寻找这些复用的宏定义，在 **stm32h7xx_hal_gpio_ex.h** 文件中可以找到。从表 16.1.4.1 我们知道 PA9 和 PA10 都是在复用器的 AF7 这一路中，所以我们在 HAL 库中就找 AF7 的宏定义，其定义如下：

```
/**
  * @brief    AF 7 selection
  */
#define GPIO_AF7_SPI2    ((uint8_t)0x07) /* SPI2 Alternate Function mapping */
#define GPIO_AF7_SPI3    ((uint8_t)0x07) /* SPI3 Alternate Function mapping */
#define GPIO_AF7_SPI6    ((uint8_t)0x07) /* SPI6 Alternate Function mapping */
#define GPIO_AF7_USART1  ((uint8_t)0x07) /* USART1 Alternate Function mapping */
#define GPIO_AF7_USART2  ((uint8_t)0x07) /* USART2 Alternate Function mapping */
#define GPIO_AF7_USART3  ((uint8_t)0x07) /* USART3 Alternate Function mapping */
#define GPIO_AF7_USART6  ((uint8_t)0x07) /* USART6 Alternate Function mapping */
#define GPIO_AF7_UART7   ((uint8_t)0x07) /* UART7 Alternate Function mapping */
#define GPIO_AF7_DFSDM1  ((uint8_t)0x07) /* DFSDM Alternate Function mapping */
#define GPIO_AF7_SDMMC1  ((uint8_t)0x07) /* SDMMC1 Alternate Function mapping */
```

细心的朋友可以看出，这些宏定义的值都是一样的，即都是(uint8_t)0x07，而宏名只是为了区分是哪个外设而已。因为我们的外设是串口 1，所以就很容易选择到我们复用的功能要用到的宏定义，就是 GPIO_AF7_USART1。具体的场景应用请看我们的串口 1 的初始化源码。

16.2 硬件设计

1. 例程功能

LED0 闪烁，提示程序在运行。 STM32 通过串口 1 和上位机对话，STM32 在收到上位机发过来的字符串（以回车换行结束）后，会返回给上位机。同时每隔一定时间，通过串口 1 输出一段信息到电脑。

2. 硬件资源

1）LED 灯

LED0 - PB1

2）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340C 上面)，需要跳线帽连接。

3. 原理图

USB 转串口硬件部分的原理图，如图 16.2.1 所示：

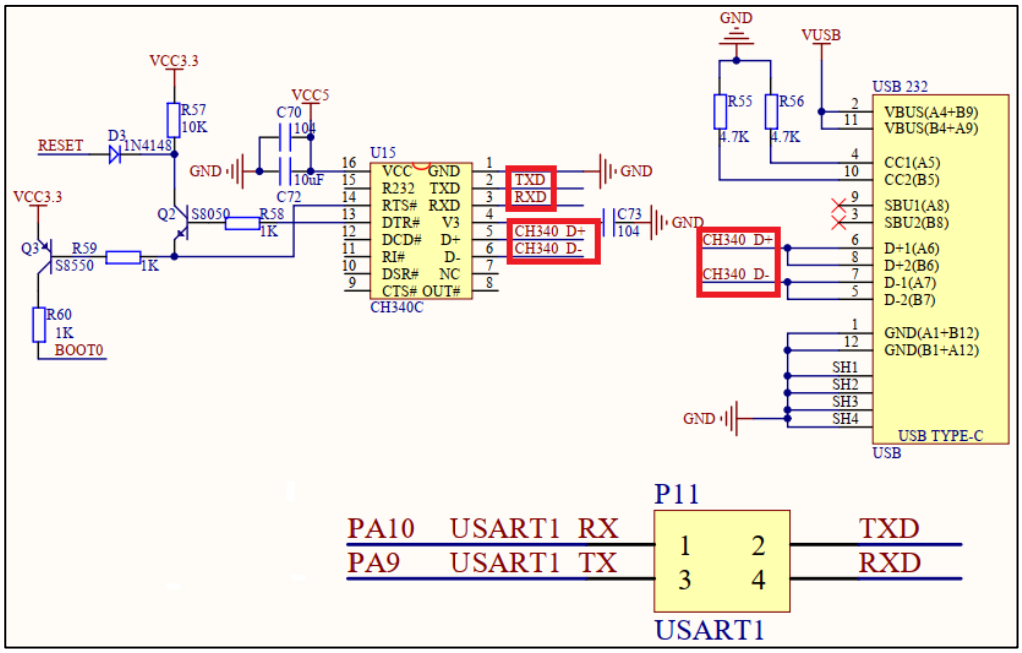


图 16.2.1 USB 转串口原理图

这里需要注意的是：上图中的红色的接线引脚和蓝色的接线引脚需要用跳线帽连接起来，否则串口的信号线跟 USB 接口是没有连接的。这里我们要把 P11 的 RXD 和 PA9 用跳线帽连接，以及 TXD 和 PA10 也用跳线帽连接。如图 16.2.2 所示：

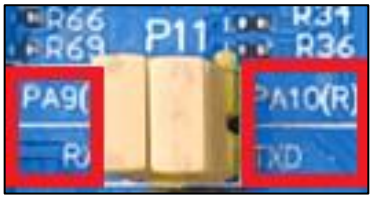


图 16.2.2 跳线帽连接

16.3 程序设计

16.3.1 USART 的 HAL 库驱动

HAL 库中关于串口的驱动程序比较多，我们主要先来学习本章需要用到的，其余的后续用到再讲解。因为我们现在只是用到异步收发器功能，所以我们现在只需要 stm32h7xx_hal_uart.c 文件（及其头文件）的驱动代码，stm32h7xx_hal_usart.c 是通用同步异步收发器，暂时没有用到，可以暂时不看。用到一个外设第一个函数就应该是其初始化函数。

1. HAL_UART_Init 函数

要使用一个外设首先要对它进行初始化，所以先看串口的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_UART_Init(UART_HandleTypeDef *huart);
```

- **函数描述：**

用于初始化异步模式的收发器。

- **函数形参：**

形参 1 是串口的句柄，UART_HandleTypeDef 结构体类型，其定义如下：

```
typedef struct
{
  USART_TypeDef                 *Instance;              /* UART 寄存器基地址 */
  UART_InitTypeDef               Init;                  /* UART 通信参数 */
  UART_AdvFeatureInitTypeDef     AdvancedInit;          /* UART 高级功能配置结构体 */
  const uint8_t                 *pTxBuffPtr;            /* 指向 UART 发送缓冲区 */
  uint16_t                       TxXferSize;            /* UART 发送数据的大小 */
  __IO uint16_t                  TxXferCount;           /* UART 发送数据的个数 */
  uint8_t                       *pRxBuffPtr;            /* 指向 UART 接收缓冲区 */
  uint16_t                       RxXferSize;            /* UART 接收数据大小 */
  __IO uint16_t                  RxXferCount;           /* UART 接收数据的个数 */
  uint16_t                       Mask;                  /* UART 数据接收寄存器掩码 */
  uint32_t                       FifoMode;              /* 指定是否使用 FIFO 模式 */
  uint16_t                       NbRxDataToProcess;     /* 接收执行期间要处理的数据数*/
  uint16_t                       NbTxDataToProcess;     /* 发送执行期间要处理的数据数*/
  __IO HAL_UART_RxTypeTypeDef    ReceptionType;         /* 持续接收类型 */
  void (*RxISR)(struct __UART_HandleTypeDef *huart);/* 处理器上的函数指针*/
  void (*TxISR)(struct __UART_HandleTypeDef *huart);/* 处理器上的函数指针*/
  DMA_HandleTypeDef              *hdmatx;               /* UART 发送参数设置（DMA） */
  DMA_HandleTypeDef              *hdmarx;               /* UART 接收参数设置（DMA） */
  HAL_LockTypeDef                Lock;                  /* 锁定对象 */
  __IO HAL_UART_StateTypeDef     gState;                /* UART 发送状态结构体 */
  __IO HAL_UART_StateTypeDef     RxState;               /* UART 接收状态结构体 */
  __IO uint32_t                  ErrorCode;             /* UART 操作错误信息 */
}UART_HandleTypeDef;
```

1）Instance：指向 UART 寄存器基地址。实际上这个基地址 HAL 库已经定义好了，可以选择范围：USART1~USART3、USART6、UART4、UART5、UART7、UART8。

2）Init：UART 初始化结构体，用于配置通讯参数，如波特率、数据位数、停止位等等。下面我们再详细讲解这个结构体。

3）AdvancedInit：用于配置高级功能，如自动波特率，MSB 先行等。

4）pTxBuffPtr，TxXferSize，TxXferCount：分别是指向发送数据缓冲区的指针，发送数据的大小，发送数据的个数。

5）pRxBuffPtr，RxXferSize，RxXferCount：分别是指向接收数据缓冲区的指针，接受数据的大小，接收数据的个数；

6）Mask：UART 数据接收寄存器的掩码，用于存放数据的校验位。

7）hdmatx，hdmarx：配置串口发送接收数据的 DMA 具体参数。

8）Lock：对资源操作增加操作锁保护，可选 HAL_UNLOCKED 或者 HAL_LOCKED 两个参数。如果 gState 的值等于 HAL_UART_STATE_RESET，则认为串口未被初始化，此时，分配锁资源，并且调用 HAL_UART_MspInit 函数来对串口的 GPIO 和时钟进行初始化。

9）gState，RxState：分别是 UART 的发送状态、工作状态的结构体和 UART 接受状态的结构体。HAL_UART_StateTypeDef 是一个枚举类型，列出串口在工作过程中的状态值，有些值只适用于 gState，如 HAL_UART_STATE_BUSY。

10）ErrorCode：串口错误操作信息。主要用于存放串口操作的错误信息。

下面，我们来了解 UART_InitTypeDef 这个结构体类型，该结构体用于配置 UART 的各个通信参数，包括波特率，停止位等，具体说明如下：

```
typedef struct
{
  uint32_t BaudRate;            /* 波特率 */
  uint32_t WordLength;          /* 字长 */
  uint32_t StopBits;            /* 停止位 */
  uint32_t Parity;              /* 校验位 */
  uint32_t Mode;                /* UART 模式 */
  uint32_t HwFlowCtl;           /* 硬件流设置 */
```

```
  uint32_t OverSampling;        /* 过采样设置 */
  uint32_t OneBitSampling;      /* 采样位方法选择 */
  uint32_t ClockPrescaler;      /* 时钟源的预分频值 */
}UART_InitTypeDef
```

1）BaudRate：波特率设置。一般设置为 2400、9600、19200、115200。
2）WordLength：数据帧字长，可选 8 位或 9 位。这里我们设置为 8 位字长数据格式。
3）StopBits：停止位设置，可选 0.5 个、1 个、1.5 个和 2 个停止位，一般我们选择 1 个停止位。
4）Parity：奇偶校验控制选择，我们设定为无奇偶校验位。
5）Mode：UART 模式选择，可以设置为只收模式，只发模式，或者收发模式。这里我们设置为全双工收发模式。
6）HwFlowCtl：硬件流控制选择，我们设置为无硬件流控制。
7）OverSampling：过采样选择，选择 8 倍过采样或者 16 过采样，一般选择 16 过采样。
8）OneBitSampling：一个采样位方法使能，0：三个采样位方法；1：一个采样位方法。
9）ClockPrescaler：时钟分频系数，默认选择不分频。

- **函数返回值：**

HAL_StatusTypeDef 枚举类型的值，有 4 个，分别是 HAL_OK 表示成功，HAL_ERROR 表示错误，HAL_BUSY 表示忙碌，HAL_TIMEOUT 超时。后续遇到该结构体也是一样的。

2. HAL_UART_Receive_IT 函数

HAL_UART_Receive_IT 函数是开启串口接收中断函数。其声明如下：

```
HAL_StatusTypeDef HAL_UART_Receive_IT(UART_HandleTypeDef *huart,
                                      uint8_t *pData, uint16_t Size);
```

- **函数描述：**

用于开启以中断的方式接收指定字节。数据接收在中断处理函数里面实现。

- **函数形参：**

形参 1 是 UART_HandleTypeDef 结构体指针类型的串口句柄。
形参 2 是要接收的数据地址。
形参 3 是要接收的数据大小，以字节为单位。

- **函数返回值：**

HAL_StatusTypeDef 枚举类型的值。

3. HAL_UART_IRQHandler 函数

HAL_UART_IRQHandler 函数是 HAL 库中断处理公共函数。其声明如下：

```
void HAL_UART_IRQHandler(UART_HandleTypeDef *huart);
```

- **函数描述：**

该函数是 HAL 库中断处理公共函数，在串口中断服务函数中被调用。

- **函数形参：**

形参 1 是 UART_HandleTypeDef 结构体指针类型的串口句柄。

- **函数返回值：**无
- **注意事项：**

该函数是 HAL 库已经定义好，用户一般不能随意修改。，如果用户要在中断中实现自己的逻辑代码，可以直接在函数 HAL_UART_IRQHandler 的前面或者后面添加新代码，也可以直接在 HAL_UART_IRQHandler 调用的各种回调函数里面执行，这些回调都是弱定义的，方便用户直接在其它文件里面重定义。串口回调函数主要有下面几个：

```
__weak void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart)
__weak void HAL_UART_TxHalfCpltCallback(UART_HandleTypeDef *huart)
__weak void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
__weak void HAL_UART_RxHalfCpltCallback(UART_HandleTypeDef *huart)
__weak void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)
__weak void HAL_UART_AbortCpltCallback(UART_HandleTypeDef *huart)
__weak void HAL_UART_AbortTransmitCpltCallback(UART_HandleTypeDef *huart)
__weak void HAL_UART_AbortReceiveCpltCallback(UART_HandleTypeDef *huart)
```

本实验我们用到的是接收回调函数 HAL_UART_RxCpltCallback，就是在接收回调函数里面编写我们的接收逻辑代码，具体请参考实验源码。

串口通信配置步骤

1) 串口参数初始化（波特率、字长、奇偶校验等），并使能串口。

HAL 库通过调用串口初始化函数 HAL_UART_Init 完成对串口参数初始化，详见例程源码。

注意：该函数会调用：HAL_UART_MspInit 函数来完成对串口底层的初始化，包括：串口及 GPIO 时钟使能、GPIO 模式设置、中断设置等。

2）使能串口和 GPIO 口时钟

本实验用到 USART1 串口，使用 PA9 和 PA10 作为串口的 TX 和 RX 脚，因此需要先使能 USART1 和 GPIOA 时钟。参考代码如下：

```
__HAL_RCC_USART1_CLK_ENABLE();            /* 使能 USART1 时钟 */
__HAL_RCC_GPIOA_CLK_ENABLE();             /* 使能 GPIOA 时钟 */
```

3）GPIO 模式设置（速度、上下拉、复用功能等）

GPIO 模式设置通过调用 HAL_GPIO_Init 函数实现，详见本例程源码。

4）开启串口相关中断，配置串口中断优先级

本实验我们使用串口中断来接收数据。我们使用 HAL_UART_Receive_IT 函数开启串口中断接收，并设置接收 buffer 及其长度。通过 HAL_NVIC_EnableIRQ 函数使能串口中断，通过 HAL_NVIC_SetPriority 函数设置中断优先级。

5）编写中断服务函数

串口 1 中断服务函数为：USART1_IRQHandler，当发生中断的时候，程序就会执行中断服务函数。HAL 库为了使用方便，提供了一个串口中断通用处理函数 HAL_UART_IRQHandler，该函数在串口接收完数据后，又会调用回调函数 HAL_UART_RxCpltCallback ，用于给用户处理串口接收到的数据。

因此我们需要在 HAL_UART_RxCpltCallback 函数实现数据接收处理，详见本例程源码。

6）串口数据接收和发送

最后我们可以通过读写 USART_DR 寄存器，完成串口数据的接收和发送，HAL 库也给我们提供了：HAL_UART_Receive 和 HAL_UART_Transmit 两个函数用于串口数据的接收和发送。

大家可以根据实际情况选择使用那种方式来收发串口数据。

16.3.2 程序流程图

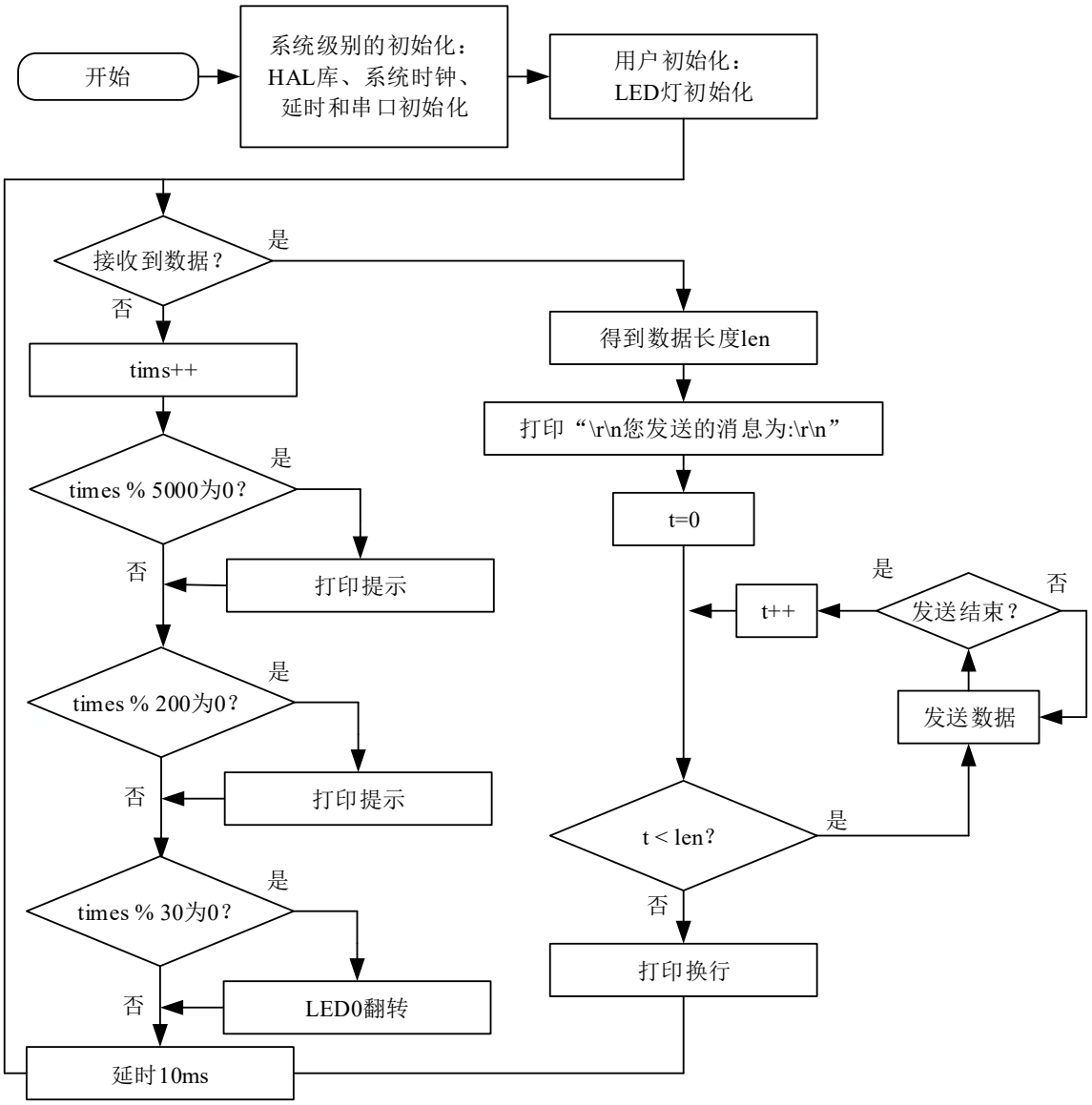


图 16.3.2.1 串口通信实验程序流程图

16.3.3 程序解析

1. 串口 1 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。串口 1（USART1）驱动源码包括两个文件：usart.c 和 usart.h。

下面我们先解析 usart.h 的程序。

- **串口 1 引脚定义**

由硬件设计小节，我们知道 PA9 和 PA10 分别被复用为串口 1 的发送和接收引脚，我们做了下面的引脚定义。

```
/* 串口 1 的 GPIO */
#define USART_TX_GPIO_PORT              GPIOA
#define USART_TX_GPIO_PIN               GPIO_PIN_9
#define USART_TX_GPIO_AF                GPIO_AF7_USART1
/* 发送引脚时钟使能 */
#define USART_TX_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)
```

```
#define USART_RX_GPIO_PORT              GPIOA
#define USART_RX_GPIO_PIN               GPIO_PIN_10
#define USART_RX_GPIO_AF                GPIO_AF7_USART1
/* 接收引脚时钟使能 */
#define USART_RX_GPIO_CLK_ENABLE() do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)

#define USART_UX                        USART1
#define USART_UX_IRQn                   USART1_IRQn
#define USART_UX_IRQHandler             USART1_IRQHandler
/* USART1 时钟使能 */
#define USART_UX_CLK_ENABLE()       do{ __HAL_RCC_USART1_CLK_ENABLE(); }while(0)
```

PA9 和 PA10 都在复用器的 AF7 这路信号线中，所以都复用为 GPIO_AF7_USART1，前面有提过。USART1_IRQn 也就是我们中断向量表的 37 号中断，USART1_IRQHandler 是串口 1 的中断服务函数。每个串口都有自己的中断函数，但是我们最终都是通过回调函数去实现逻辑代码，当然我们亦可在中断函数里实现逻辑代码。

另外我们还定义了三个宏，具体如下：

```
#define USART_REC_LEN   200             /* 定义最大接收字节数 200 */
#define USART_EN_RX     1               /* 使能（1）/禁止（0）串口 1 接收 */
#define RXBUFFERSIZE    1               /* 缓存大小 */
```

可以看到 USART_REC_LEN 表示最大接收字节数，这里定义的是 200 个字节，后续如果有需求要发送更大的数据包，可以改大这个值，这里不改太大，是避免浪费太多内存。USART_EN_RX 则是用于使能串口 1 的接收数据。RXBUFFERSIZE 是缓冲大小。

下面我们再解析 usart.c 的程序，先看串口 1 的初始化函数，其定义如下：

```
/**
 * @brief       串口 x 初始化函数
 * @param       baudrate: 波特率，根据自己需要设置波特率值
 * @note        注意：必须设置正确的时钟源，否则串口波特率就会设置异常.
 *              这里的 USART 的时钟源在 sys_stm32_clock_init()函数中已经设置过了.
 * @retval      无
 */
void usart_init(uint32_t baudrate)
{
    g_uart1_handle.Instance = USART_UX;                    /* USART1 */
    g_uart1_handle.Init.BaudRate = baudrate;               /* 波特率 */
    g_uart1_handle.Init.WordLength = UART_WORDLENGTH_8B;   /* 字长为 8 位数据格式 */
    g_uart1_handle.Init.StopBits = UART_STOPBITS_1;        /* 一个停止位 */
    g_uart1_handle.Init.Parity = UART_PARITY_NONE;         /* 无奇偶校验位 */
    g_uart1_handle.Init.HwFlowCtl = UART_HWCONTROL_NONE;   /* 无硬件流控 */
    g_uart1_handle.Init.Mode = UART_MODE_TX_RX;            /* 收发模式 */
   HAL_UART_Init(&g_uart1_handle);  /* HAL_UART_Init()会使能 UART1 */

   /*该函数会开启接收中断：标志位 UART_IT_RXNE，并且设置接收缓冲以及接收缓冲接收最大数据量*/
    HAL_UART_Receive_IT(&g_uart1_handle, (uint8_t *)g_rx_buffer, RXBUFFERSIZE);
}
```

g_uart1_handle 是结构体 UART_HandleTypeDef 类型的全局变量，UART_HandleTypeDef 结构体成员的含义请回到前面回顾。波特率我们直接赋值给 g_uart1_handle.Init.BaudRate 这个成员，可以看出很方便。需要注意的是，最后一行代码调用函数 HAL_UART_Receive_IT，作用是开启接收中断，同时设置接收的缓存区以及接收的数据量。

上面的初始化函数只是串口初始化的其中一部分，我们还有一部分初始化需要 HAL_UART_MspInit 函数去完成。HAL_UART_MspInit 是 HAL 库定义的弱定义函数，这里我们做重定义以实现我们的初始化需求。HAL_UART_MspInit 函数在 HAL_UART_Init 函数中会被调用，其定义如下：

```
/**
 * @brief       UART 底层初始化函数
 * @param       huart: UART 句柄类型指针
 * @note        此函数会被 HAL_UART_Init()调用
```

```
 *              完成时钟使能，引脚配置，中断配置
 * @retval      无
 */
void HAL_UART_MspInit(UART_HandleTypeDef *huart)
{
    GPIO_InitTypeDef gpio_init_struct;
    if(huart->Instance == USART_UX)          /* 如果是串口 1，进行串口 1 MSP 初始化 */
    {
        USART_UX_CLK_ENABLE();               /* USART1 时钟使能 */
        USART_TX_GPIO_CLK_ENABLE();          /* 发送引脚时钟使能 */
        USART_RX_GPIO_CLK_ENABLE();          /* 接收引脚时钟使能 */

        gpio_init_struct.Pin = USART_TX_GPIO_PIN;              /* TX 引脚 */
        gpio_init_struct.Mode = GPIO_MODE_AF_PP;               /* 复用推挽 */
        gpio_init_struct.Pull = GPIO_PULLUP;                   /* 上拉 */
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;         /* 高速 */
        gpio_init_struct.Alternate = USART_TX_GPIO_AF;         /* 复用为 USART1 */
        HAL_GPIO_Init(USART_TX_GPIO_PORT, &gpio_init_struct);/* 初始化发送引脚 */

        gpio_init_struct.Pin = USART_RX_GPIO_PIN;              /* RX 引脚 */
        gpio_init_struct.Alternate = USART_RX_GPIO_AF;         /* 复用为 USART1 */
        HAL_GPIO_Init(USART_RX_GPIO_PORT, &gpio_init_struct);/* 初始化接收引脚 */
#if USART_EN_RX
        HAL_NVIC_EnableIRQ(USART_UX_IRQn);          /* 使能 USART1 中断通道 */
        HAL_NVIC_SetPriority(USART_UX_IRQn, 3, 3); /* 抢占优先级 3，子优先级 3 */
#endif
    }
}
```

怎么来理解这个函数呢？该函数主要实现底层的初始化，事实上这个函数的代码还可以直接放到 usart_init 函数里面，但是 HAL 库为了代码的功能分层初始化，定义这个函数方便用户使用。所以我们也按照 HAL 库的这个结构来初始化外设。这个函数首先是调用 if(huart->Instance == USART_UX)判断是要初始化哪个串口，因为每个串口初始化都会调用 HAL_UART_MspInit 这个函数，所以需要判断是哪个串口要初始化才做相应的处理。只能说 HAL 库这样的结构机制有好处，自然也有坏处。

首先就是使能串口以及 PA9 和 PA10 的时钟，PA9 和 PA10 需要用做复用功能，复用功能模式有两个选择：GPIO_MODE_AF_PP 推挽式复用和 GPIO_MODE_AF_OD 开漏式复用，我们选择推挽式复用。选择了推挽式复用，我们还需要指定复用到哪个外设，通过赋值给 Alternate 这个结构体成员。然后就是调用 HAL_GPIO_Init 函数进行 IO 口的初始化。

最后因为我们用到串口中断，所以还需要中断相关的配置。HAL_NVIC_EnableIRQ 函数使能串口 1 复用通道。HAL_NVIC_SetPriority 函数配置串口中断的抢占优先级以及响应优先级。

串口初始化由上述两个函数完成，下面就该讲到串口中断服务函数了，其定义如下：

```
/**
 * @brief       串口 X 中断服务函数
 * @param       无
 * @retval      无
 */
void USART_UX_IRQHandler(void)
{
#if SYS_SUPPORT_OS                          /* 使用 OS */
    OSIntEnter();
#endif

    HAL_UART_IRQHandler(&g_uart1_handle); /* 调用 HAL 库中断处理公用函数 */

#if SYS_SUPPORT_OS                      /* 使用 OS */
    OSIntExit();
#endif
}
```

在中断服务函数里主要就调用 HAL 库的串口中断公共处理函数 HAL_UART_IRQHandler()，然后该函数内部再调用相关的中断回调函数，我们这里用到的是串口接收完成中断回调函数 HAL_UART_RxCpltCallback 来处理用户的逻辑代码。

下面主要来看看 HAL_UART_RxCpltCallback 函数的定义：

```
/**
 * @brief       Rx 传输回调函数
 * @param       huart: UART 句柄类型指针
 * @retval      无
 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{
    if(huart->Instance == USART_UX)             /* 如果是串口 1 */
    {
        if((g_usart_rx_sta & 0x8000) == 0)      /* 接收未完成 */
        {
            if(g_usart_rx_sta & 0x4000)         /* 接收到了 0x0d（即回车键）*/
            {
                if(aRxBuffer[0] != 0x0a)        /* 接收到的不是 0x0a（即不是换行键）*/
                {
                    g_usart_rx_sta = 0;         /* 接收错误,重新开始 */
                }
                else                            /* 接收到的是 0x0a（即换行键）*/
                {
                    g_usart_rx_sta |= 0x8000; /* 接收完成了 */
                }
            }
            else                                /* 还没收到 0X0D（即回车键） */
            {
                if (g_rx_buffer[0] == 0x0d)
                {
                    g_usart_rx_sta |= 0x4000;
                }
                else
                {
                    g_usart_rx_buf[g_usart_rx_sta & 0X3FFF] = g_rx_buffer[0] ;
                    g_usart_rx_sta++;
                    if(g_usart_rx_sta > (USART_REC_LEN - 1))
                    {
                        g_usart_rx_sta = 0;          /* 接收数据错误,重新开始接收 */
                    }
                }
            }
        }
        HAL_UART_Receive_IT(&g_uart1_handle, (uint8_t *)g_rx_buffer,
                            RXBUFFERSIZE);
    }
}
```

因为我们设置了串口句柄成员变量 RxXferSize 为 1，那么每当串口 1 接收到一个字符后触发接收完成中断，便会在中断服务函数中引导执行该回调函数。当串口接受到一个字符后，它会保存在缓存 g_rx_buffer 中，由于我们设置了缓存大小为 1，而且 RxXferSize=1，所以每次接收一个字符，会直接保存到 RxXferSize[0]中，我们直接通过读取 RxXferSize[0]的值就是本次接收到的字符。这里我们设计了一个小小的接收协议：通过这个函数，配合一个数组 g_usart_rx_buf，一个接收状态寄存器 g_usart_rx_sta（此寄存器其实就是一个全局变量，由作者自行添加。由于它起到类似寄存器的功能，这里暂且称之为寄存器）实现对串口数据的接收管理。数组 g_usart_rx_buf 的大小由 USART_REC_LEN 定义，也就是一次接收的数据最大不能超过 USART_REC_LEN 个字节。g_usart_rx_sta 是一个接收状态寄存器其各的定义如表 16.3.3.1 所示：

g_usart_rx_sta		
bit15	bit14	bit13~0

接收完成标志	接收到 0X0D 标志	接收到的有效字节个数

表 16.3.3.1 接收状态寄存器位定义表

设计思路如下：

当接收到从电脑发过来的数据，把接收到的数据保存在数组 g_usart_rx_buf 中，同时在接收状态寄存器（g_usart_rx_sta）中计数接收到的有效数据个数，当收到回车（回车的表示由 2 个字节组成：0X0D 和 0X0A）的第一个字节 0X0D 时，计数器将不再增加，等待 0X0A 的到来，而如果 0X0A 没有来到，则认为这次接收失败，重新开始下一次接收。如果顺利接收到 0X0A，则标记 g_usart_rx_sta 的第 15 位，这样完成一次接收，并等待该位被其他程序清除，从而开始下一次的接收，而如果迟迟没有收到 0X0D，那么在接收数据超过 USART_REC_LEN 的时候，则会丢弃前面的数据，重新接收。

学到这里大家会发现，HAL 库定义的串口中断逻辑确实非常复杂，并且因为处理过程繁琐所以效率不高。这里我们需要说明的是，在中断服务函数中，大家也可以不用调用 HAL_UART_IRQHandler 函数，而是直接编写自己的中断服务函数。串口实验我们之所以遵循 HAL 库写法， 是为了让大家对 HAL 库有一个更清晰的理解。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t len;
    uint16_t times = 0;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    led_init();                             /* 初始化 LED */
    while (1)
    {
        if (g_usart_rx_sta & 0x8000)        /* 接收到了数据 */
        {
            len = g_usart_rx_sta & 0x3fff; /* 得到此次接收到的数据长度 */
            printf("\r\n 您发送的消息为:\r\n");

            /* 发送接收到的数据 */

          HAL_UART_Transmit(&uartx_handler,(uint8_t*)g_usart_rx_buf,len,1000);
            /* 等待发送结束 */
            while(__HAL_UART_GET_FLAG(&uartx_handler,UART_FLAG_TC)!=SET);
            printf("\r\n\r\n"); /* 插入换行 */
            g_usart_rx_sta = 0;
        }
        else
        {
            times++;
            if (times % 5000 == 0)
            {
                printf("\r\n 正点原子 STM32 开发板 串口实验\r\n");
                printf("正点原子@ALIENTEK\r\n\r\n\r\n");
            }
            if (times % 200 == 0) printf("请输入数据,以回车键结束\r\n");
            if (times % 30  == 0) LED0_TOGGLE(); /* 闪烁 LED,提示系统正在运行. */
            delay_ms(10);
        }
    }
}
```

我们主要看无限循环里面的逻辑：首先判断全局变量 g_usart_rx_sta 的最高位是否为 1，如果为 1 的话，那么代表前一次数据接收已经完成，接下来就是把我们自定义接收缓冲的数据发送到串口，在上位机显示。这里比较重点的两条语句是：第一条是调用 HAL 串口发送函数 HAL_UART_Transmit 来发送一段字符到串口。第二条是我们发送一个字节之后之后，要检测这个数据是否已经被发送完成了。如果全局变量 g_usart_rx_sta 的最高位为 0，则执行一段时间往上位机发送提示字符，以及让 LED0 每隔一段时间翻转，提示系统正在运行。

16.4 下载验证

在下载好程序后，可以看到板子上的 LED0 开始闪烁，说明程序已经在跑了。串口调试助手，我们用 XCOM V2.7，该软件在光盘有提供，且无需安装，直接可以运行，但是需要你的电脑安装有.NET Framework 4.0(WIN 自带了)或以上版本的环境才可以，该软件的详细介绍请看：http://www.openedv.com/posts/list/22994.htm 这个帖子。

接着我们打开 XCOM V2.7，设置串口为开发板的 USB 转串口（CH340 虚拟串口，得根据你自己的电脑选择，我的电脑是 COM8，另外，请注意：波特率是 115200）。因为我们在程序上面设置了必须输入回车，串口才认可接收到的数据，所以必须在发送数据后再发送一个回车符，这里 XCOM 提供的发送方法是通过勾选发送新行实现，只要勾选了这个选项，每次发送数据后，XCOM 都会自动多发一个回车(0X0D+0X0A)。设置好了发送新行，我们再在发送区输入你想要发送的文字，然后单击发送，可以看到如图 16.4.1 所示信息：

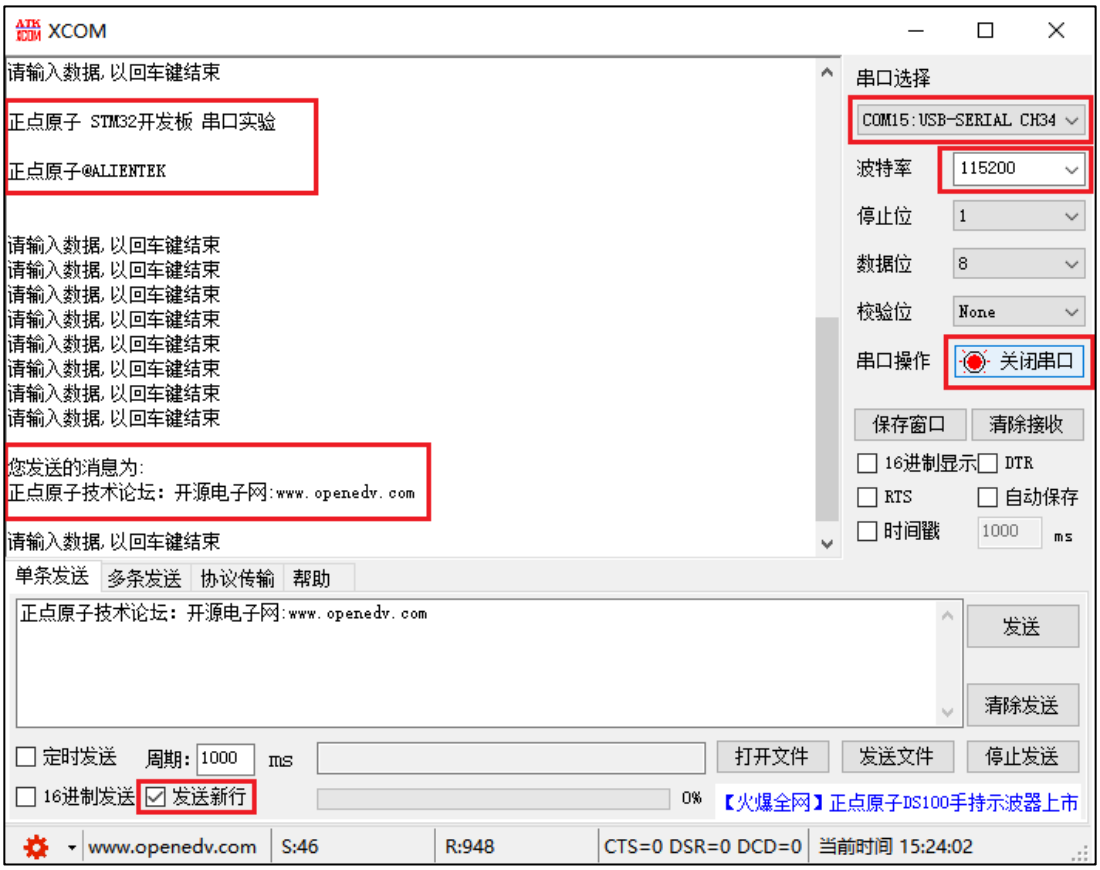


图 16.4.1 串口助手

可以看到，我们发送的消息被发送回来了。大家可以试试，如果不发送回车（取消发送新行），在输入内容之后，直接按发送是什么结果，大家测试一下吧。

第十七章 独立看门狗（IWDG）实验

本章我们学习如何使用 STM32H7 的独立看门狗（以下简称 IWDG）。STM32H7 内部自带了 2 个看门狗：独立看门狗（IWDG）和窗口看门狗（WWDG）。这一章我们只介绍独立看门狗，窗口看门狗将在下一章介绍。在本章中，我们将通过按键 KEY_UP 来喂狗，然后通过 LED0 提示复位状态。

本章分为如下几个小节：

17.1 IWDG 简介
17.2 硬件设计
17.3 程序设计
17.4 下载验证

17.1 IWDG 简介

独立看门狗本质上是一个定时器，这个定时器有一个输出端，可以输出复位信号。该定时器是一个 12 位的递减计数器，当计数器的值减到 0 的时候，就会产生一个复位信号。如果在计数没减到 0 之前，重置计数器的值的话，那么就不会产生复位信号，这个动作我们称为喂狗。看门狗功能由 VDD 电压域供电，在停止模式和待机模式下仍然可以工作。

17.1.1 IWDG 框图

下面先来学习 IWDG 框图，通过学习 IWDG 框图会有一个很好的整体掌握，同时对之后的编程也会有一个清晰的思路。

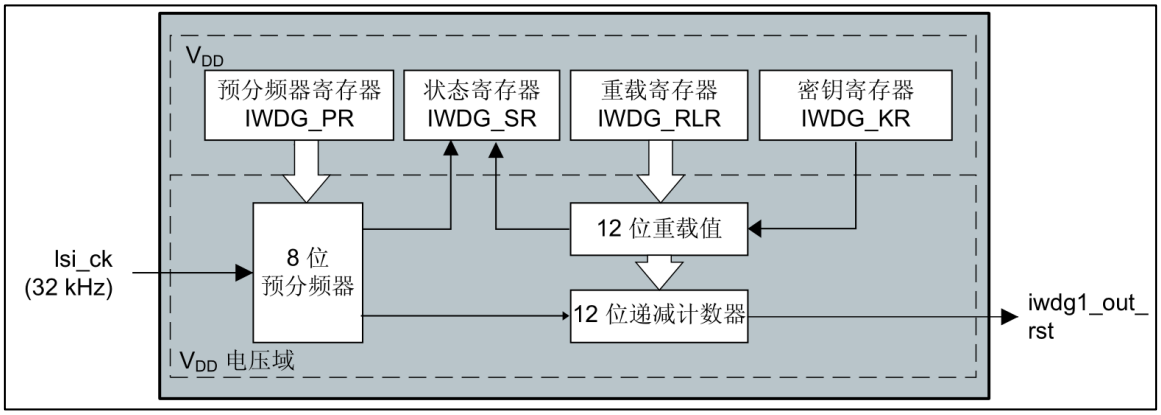


图 17.1.1.1 IWDG 框图

从 IWDG 框图整体认知就是，IWDG 有一个输入（时钟 lsi_ck），经过一个 8 位的可编程预分频器提供时钟给一个 12 位递减计数器，满足条件就会输出一个复位信号（iwdg1_out_rst）。IWDG 内部输入/输出信号如下表：

信号名称	信号类型	说明
lsi_ck	数字信号	LSI 时钟
iwdg1_out_rst	数字信号	IWDG1 复位信号输出

表 17.1.1.1 IWDG 内部输入/输出信号

STM32H743 的独立看门狗由内部专门的 32Khz 低速时钟（lsi_ck）驱动，即使主时钟发生故障，它也仍然有效。这里需要注意独立看门狗的时钟是一个内部 RC 时钟，所以并不是准确的 32Khz，而是在 17~47Khz 之间的一个可变化的时钟，只是我们在估算的时候，以 32Khz 的频率来计算，看门狗对时间的要求不是很精确，所以，时钟有些偏差，都是可以接受的。

17.1.2 IWDG 寄存器

IWDG 的框图很简单，用到的寄存器也不多。我们主要用到其中 3 个寄存器：

- **键寄存器（IWDG_KR）**

键寄存器可以看作是独立看门狗的控制寄存器，其描述如图 17.1.2.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

位 31:16 保留，必须保持复位值。

位 15:0 **KEY[15:0]**：键值 (Key value)（只能写，读为 0x0000）
必须每隔一段时间便通过软件对这些位写入键值 0xAAAA，否则当计数器计数到 0 时，看门狗会产生复位。
写入键值 0x5555 可使能对 IWDG_PR、IWDG_RLR 和 IWDG_WINR 寄存器的访问（请参见 *第 45.3.7 节：寄存器访问保护*）
写入键值 0xCCCC 可启动看门狗（选中硬件看门狗选项的情况除外）

图 17.1.2.1 WDG_KR 寄存器

在键寄存器(IWDG_KR)中写入 0xCCCC，开始启用独立看门狗；此时计数器开始从其复位值 0xFFF 递减计数。当计数器计数到末尾 0x000 时，会产生一个复位信号(IWDG_RESET)。无论何时，只要键寄存器 IWDG_KR 中被写入 0xAAAA， IWDG_RLR 中的值就会被重新加载到计数器中从而避免产生看门狗复位 。

IWDG_PR 和 IWDG_RLR 寄存器具有写保护功能。要修改这两个寄存器的值，必须先向 IWDG_KR 寄存器中写入 0x5555。将其他值写入这个寄存器将会打乱操作顺序，寄存器将重新被保护。重装载操作(即写入 0xAAAA)也会启动写保护功能。

- **预分频寄存器（IWDG_PR）**

预分频寄存器描述如图 17.1.2.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PR[2:0]		
													rw	rw	rw

位 2:0 **PR[2:0]**：预分频系数 (Prescaler divider)
这些位受写访问保护，请参见 *第 45.3.7 节：寄存器访问保护*。通过软件设置这些位来选择计数器时钟的预分频因子。若要更改预分频器的分频系数，*状态寄存器 (IWDG_SR)* 的 PVU 位必须为 0。
000：4 分频
001：8 分频
010：16 分频
011：32 分频
100：64 分频
101：128 分频
110：256 分频
111：256 分频

注： 读取该寄存器会返回 V_{DD} 电压域的预分频器值。如果正在对该寄存器执行写操作，则读取的值可能不是最新的/有效的。因此，只有在状态寄存器 (IWDG_SR) 中的 PVU 位为 0 时，从寄存器读取的值才有效。

图 17.1.2.2 IWDG_ PR 寄存器

该寄存器用来设置看门狗时钟（LSI）的分频系数，最低为 4 分频，最高位 256 分频，该寄存器是一个 32 位的寄存器，但是我们只用了最低 3 位，其他都是保留位。

- **重载寄存器（IWDG_ RLR）**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	RL[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 11:0 **RL[11:0]**：看门狗计数器重载值 (Watchdog counter reload value)

这些位受写访问保护，请参见*寄存器访问保护*。这个值由软件设置，每次对*键寄存器 (IWDG_KR)* 写入值 0xAAAA 时，这个值就会重装载到看门狗计数器中。之后，看门狗计数器便从该装载的值开始递减计数。超时周期由该值和时钟预分频器共同决定。有关超时信息，请参见数据手册。

若要更改重载值，*状态寄存器 (IWDG_SR)* 中的 RVU 位必须为 0。

注：　读取该寄存器会返回 V_{DD} 电压域的重载值。如果正在对该寄存器执行写操作，则读取的值可能不是最新的/有效的。因此，只有在状态寄存器 (IWDG_SR) 中的 RVU 位为 0 时，从寄存器读取的值才有效。

图 17.1.2.3 IWDG_ RLR 寄存器

该寄存器用来保存重装载到计数器中的值。该寄存器也是一个 32 位寄存器，只有低 12 位是有效的。

17.2 硬件设计

1. 例程功能

在配置看门狗后，LED0 将常亮，如果 KEY_UP 按键按下，就喂狗，只要 KEY_UP 不停的按，看门狗就一直不会产生复位，保持 LED0 的常亮，一旦超过看门狗定溢出时间（Tout）还没按，那么将会导致程序重启，这将导致 LED0 熄灭一次。

2. 硬件资源

1）LED 灯
　LED0：LED0 - PB1
2）独立按键
　WK_UP - PA0。
3）独立看门狗

3. 原理图

独立看门狗实验的核心是在 STM32H743 内部进行，并不需要外部电路。但是考虑到指示当前状态和喂狗等操作，我们需要 2 个 IO 口，一个用来输入喂狗信号，另外一个用来指示程序是否重启。喂狗我们采用板上的 KEY_UP 键来操作，而程序重启，则是通过 LED0 来指示的。

17.3 程序设计

17.3.1 IWDG 的 HAL 库驱动

IWDG 在 HAL 库中的驱动代码在 stm32h7xx_hal_iwdg.c 文件（及其头文件）中。

1. HAL_IWDG_Init 函数

IWDG 的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_IWDG_Init(IWDG_HandleTypeDef *hiwdg);
```

- **函数描述：**
  用于初始化 IWDG。
- **函数形参：**
  形参 1 是 IWDG 句柄，IWDG_HandleTypeDef 结构体类型，其定义如下：

```
typedef struct
{
```

```
    IWDG_TypeDef                 *Instance;   /* IWDG 寄存器基地址 */
    IWDG_InitTypeDef             Init;        /* IWDG 初始化参数 */
}IWDG_HandleTypeDef;
```

1）Instance：指向 IWDG 寄存器基地址。

2）Init：IWDG 初始化结构体，用于配置计数器的相关参数。

IWDG_InitTypeDef 这个结构体类型定义如下：

```
typedef struct
{
  uint32_t Prescaler;   /* 预分频系数 */
  uint32_t Reload;      /* 重装载值 */
  uint32_t Window;      /* 窗口值 */
} IWDG_InitTypeDef;
```

1）Prescaler：预分频系数，IWDG_PRESCALER_4 到 IWDG_PRESCALER_256。

2）Reload：重装载值，范围：0 到 0x0FFF。

3）Window：窗口值。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

2. HAL_IWDG_Refresh 函数

HAL_IWDG_Refresh 函数是独立看门狗的喂狗函数。其声明如下：

```
HAL_StatusTypeDef HAL_IWDG_Refresh(IWDG_HandleTypeDef *hiwdg);
```

- **函数描述：**

  用于把重装载寄存器的值重载到计数器中，喂狗，防止 IWDG 复位。

- **函数形参：**

  形参 1 是 IWDG_HandleTypeDef 结构体指针类型的 IWDG 句柄。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

独立看门狗配置步骤

1）取消寄存器写保护，设置看门狗预分频系数和重装载值

首先我们必须取消 IWDG_PR 和 IWDG_RLR 寄存器的写保护，这样才可以设置寄存器 IWDG_PR 和 IWDG_RLR 的值。取消写保护和设置预分频系数以及重装载值在 HAL 库中是通过函数 HAL_IWDG_Init 实现的。

通过该函数设置看门狗的分频系数，和重装载的值。看门狗的喂狗时间（也就是看门狗溢出时间）的计算方式为：

$$Tout = ((4 \times 2^{prer}) \times rlr) / 32$$

其中 Tout 为看门狗溢出时间（单位为 ms）。

prer 为看门狗时钟预分频值（IWDG_PR 值），范围为 0~7。

rlr 为看门狗的重装载值（IWDG_RLR 的值）。

比如我们设定 prer 值为 4（4 代表的是 64 分频，HAL 库中可以使用宏定义标识符 IWDG_PRESCALER_64），rlr 值为 500，那么就可以得到 Tout=64×500/32=1000ms，这样，看门狗的溢出时间就是 1s，只要你在一秒钟之内，有一次写入 0XAAAA 到 IWDG_KR，就不会导致看门狗复位（当然写入多次也是可以的）。这里需要提醒大家的是，看门狗的时钟不是准确的 32Khz，所以在喂狗的时候，最好不要太晚了，否则，有可能发生看门狗复位。

2）重载计数值喂狗（向 IWDG_KR 写入 0XAAAA）

在 HAL 中重载计数值的函数是 HAL_IWDG_Refresh，该函数的作用是把值 0xAAAA 写入到 IWDG_KR 寄存器，从而触发计数器重载，即实现独立看门狗的喂狗操作。

3) 启动看门狗(向 IWDG_KR 写入 0XCCCC)

HAL 库函数里面启动独立看门狗是通过宏定义标识符来实现的：

```
#define __HAL_IWDG_START(__HANDLE__)
        WRITE_REG((__HANDLE__)->Instance->KR, IWDG_KEY_ENABLE);
```

我们只需要调用宏定义标识符__HAL_IWDG_START 即可实现看门狗使能。实际上，当我们调用了看门狗初始化函数 HAL_IWDG_Init 之后，在内部已经调用了该宏启动看门狗。

17.3.2 程序流程图

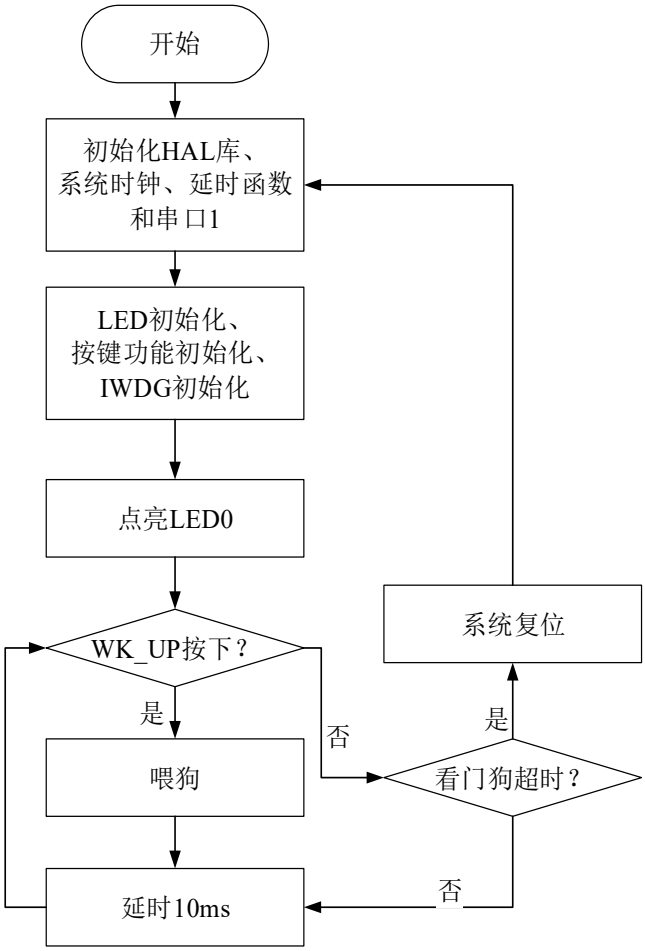


图 17.3.2.1 独立看门狗实验程序流程图

17.3.3 程序解析

1. IWDG 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。独立看门狗（IWDG）驱动源码包括两个文件：wdg.c 和 wdg.h。

wdg.h 头文件只有函数的声明，就不解释了。下面我们直接解析 wdg.c 的程序，先看 IWDG 的初始化函数，其定义如下：

```
/**
 * @brief       初始化独立看门狗
 * @param       prer: IWDG_PRESCALER_4~IWDG_PRESCALER_256,对应 4~256 分频
 *   @arg       分频因子 = 4 * 2^prer. 但最大值只能是 256!
 * @param       rlr: 自动重装载值,0~0XFFF.
 * @note        时间计算(大概):Tout=((4 * 2^prer) * rlr) / 32 (ms).
 * @retval      无
 */
void iwdg_init(uint8_t prer, uint16_t rlr)
{
    g_iwdg_handle.Instance = IWDG1;
    g_iwdg_handle.Init.Prescaler = prer;                    /* 设置 IWDG 分频系数 */
    g_iwdg_handle.Init.Reload = rlr;                        /* 重装载 */
    g_iwdg_handle.Init.Window = IWDG_WINDOW_DISABLE;        /* 关闭窗口功能 */
    HAL_IWDG_Init(&g_iwdg_handle);
```

```
}
```

iwdg_init 是独立看门狗初始化函数，主要设置预分频数和重装载寄存器的值。通过这两个寄存器，就可以大概知道看门狗复位的时间周期为多少了。

```
/**
 * @brief       喂独立看门狗
 * @param       无
 * @retval      无
 */
void iwdg_feed(void)
{
    HAL_IWDG_Refresh(&g_iwdg_handle); /* 重装载计数器 */
}
```

iwdg_feed 函数用来喂狗，在该函数内部只需调用 HAL 库函数 HAL_IWDG_Refresh。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);/* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    led_init();                             /* 初始化 LED */
    key_init();                             /* 初始化按键 */
    delay_ms(100);                          /* 延时 100ms 再初始化看门狗,LED0 的变化可见 */
    iwdg_init(IWDG_PRESCALER_64, 500); /* 预分频数为 64,重载值为 500,溢出时间为 1s */
    LED0(0);                                /* 点亮 LED0(红灯) */
    while (1)
    {
        if (key_scan(0) == WKUP_PRES)   /* 如果 WK_UP 按下,则喂狗 */
        {
            iwdg_feed();                    /* 喂狗 */
        }
        delay_ms(10);
    }
}
```

在 main 函数里，先初始化系统和用户的外设代码，然后先点亮 LED0，在无限循环里开始获取按键的键值，并判断是不是按键 WK_UP 按下，是的话就喂狗，不是则延时 10ms，继续上述操作。当 1 秒钟后都没测到按键 WK_UP 按下，IWDG 就会产生一次复位信号，系统复位，可以看到 LED0 因系统复位熄灭一次，再亮。反之，当按下按键 WK_UP 后，1 秒内再按下按键 WK_UP，就会及时喂狗，结果就是系统不会复位，LED0 也就不会闪烁。

iwdg_init(IWDG_PRESCALER_64, 500);这个语句有必要跟大家说明一下，这里的第一个形参直接使用 HAL 库自定义的 IWDG_PRESCALER_64，即预分频系数为 64，重装载值是 500，所以可由公式得到 Tout=64×500/32=1000ms，即溢出时间就是 1s。只要你在一秒钟之内，有一次写 0XAAAA 到 IWDG_KR，就不会导致看门狗复位（当然写入多次也是可以的）。这里需要提醒大家的是，看门狗的时钟不是准确的 32Khz，所以在喂狗的时候，最好不要太晚了，否则，有可能发生看门狗复位。

17.4 下载验证

下载代码后，可以看到 LED0 不停的闪烁，证明系统在不停的复位，否则 LED0 常亮。这时我们试试不停的按 KEY_UP 按键，可以看到 LED0 就常亮了，不会再闪烁。说明我们的实验是成功的。

第十八章 窗口门狗（WWDG）实验

本章我们学习如何使用 STM32H7 的另外一个看门狗，窗口看门狗（以下简称 WWDG）。我们将使用窗口看门狗的中断功能来喂狗，通过 LED0 和 LED1 提示程序的运行状态。

本章分为如下几个小节：

18.1 WWDG 简介
18.2 硬件设计
18.3 程序设计
18.4 下载验证

18.1 WWDG 简介

窗口看门狗（WWDG）通常被用来监测由外部干扰或不可预见的逻辑条件造成的应用程序背离正常的运行序列而产生的软件故障。窗口看门狗跟独立看门狗一样，也是一个递减计数器，不同的是它们的复位条件不一样。窗口看门狗产生复位信号有两个条件：

1） 当递减计数器的数值从 0x40 减到 0x3F 时（T6 位跳变到 0）。

2） 当喂狗的时候如果计数器的值大于 W[6:0]时，此数值在 WWDG_CFR 寄存器定义。

上述的两个条件详细解释是，当计数器的值减到 0X40 时还不喂狗的话，到下一个计数就会产生复位，这个值称为窗口的下限值，是固定的值，不能改变。这个跟独立看门狗类似，不同的是窗口看门狗的计数器的值在减到某一个数之前喂狗的话也会产生复位，这个值叫窗口的上限，上限值 W[6:0]由用户设置。窗口看门狗计数器的上限值和下限值就是窗口的含义，喂狗也必须在窗口之内，否则就会复位。

18.1.1 WWDG 框图

下面先来学习 WWDG 框图，通过学习 WWDG 框图会有一个很好的整体掌握，同时对之后的编程也会有一个清晰的思路。

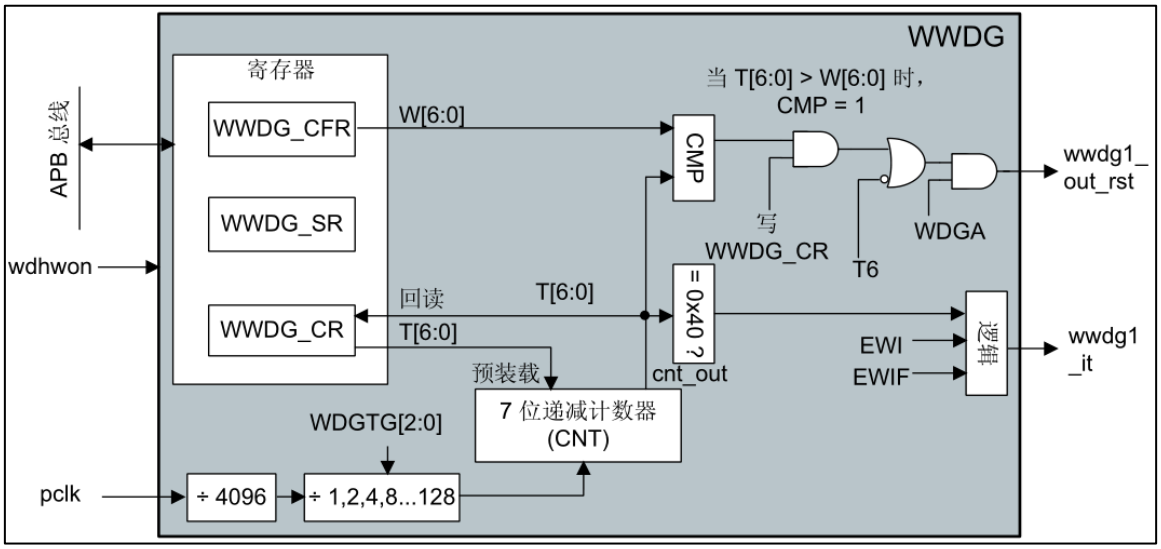


图 18.1.1.1 WWDG 框图

从 WWDG 框图整体认知就是，WWDG 有一个输入（时钟 pclk），经过一个除 4096 的分频器，再经过一个分频系数可选（1、2、4、8…128）的可编程预分频器提供时钟给一个 7 位递减计数器，这里有两个输出信号。具体如下表：

信号名称	信号类型	说明
pclk	数字信号	APB 总线时钟（PCLK3=100Mhz）
wwdg1_out_rst	数字信号	WWDG1 复位信号输出

wwdg1_it	数字信号	WWDG1 中断输出

表 18.1.1.1 WWDG 内部输入/输出信号

这里需要注意的就是 pclk 时钟是由 PCLK3 提供，由前面十一章的内容知道，我们的例程在系统时钟设置函数中把 PCLK3 的时钟频率设置为 100Mhz。

结合寄存器分析窗口看门狗的上限值和下限值。W[6:0] 是 WWDG_CFR 寄存器的低 7 位，用于与递减计数器比较的窗口值，也就是我们说的上限值，由用户设置。0x40 就是下限值，递减计数器达到这个值就会产生复位。T6 位就是 WWDG_CR 寄存器的位 6，即递减计数器 T[6:0]的最高位。他们的关系可以用图 18.1.1.2 来说明：

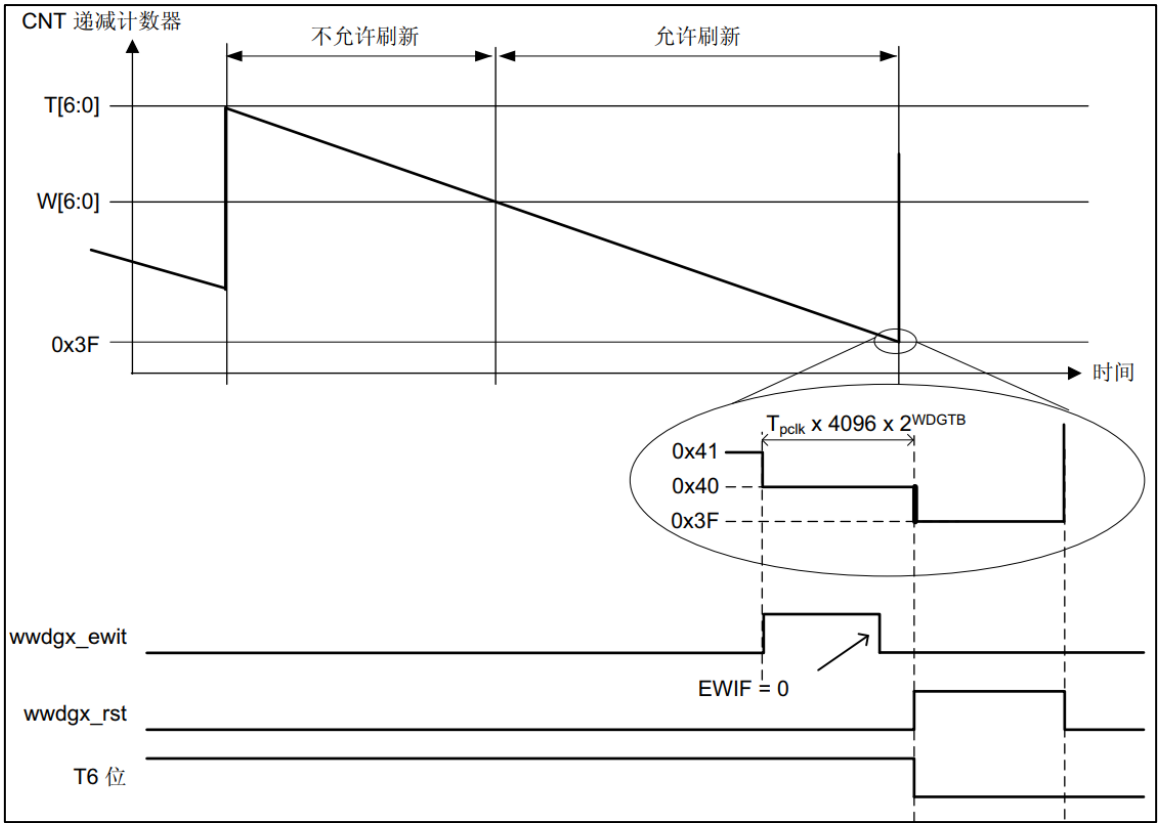


图 18.1.1.2 窗口看门狗工作示意图

图 18.1.1.2 可以看出，递减计数器的值递减过程中，当 T[6:0]>W[6:0]是不允许刷新 T[6:0]的值，即不允许喂狗，否则会产生复位。只有在 W[6:0]<T[6:0]< 0x3F 这个时间可以喂狗，这就是喂狗的窗口时间。当 T[6:0]=0x3F，即 T6 位为 0 这一刻，也会产生复位。

上限值 W[6:0]是由用户自己设置，但是一定要确保大于 0X40，否则就不存在上图的窗口了，下限值 0x40 是固定的，不可修改。

知道了窗口看门狗的工作原理，下面学习如何计算窗口看门狗的超时公式：

$$Twwdg= T_{PCLK3} \times 4096 \times 2^{\wedge WDGTB[2:0]} \times (T[5:0]+1)$$

其中：

T_{wwdg}：WWDG 超时时间（单位为 ms）

T_{PCLK3}：APB3 以 ms 为单位的时钟间隔（即单位为 1 /KHz）

WDGTB[2:0]：WWDG 的预分频系数，0~7

T[5:0]：窗口看门狗的计数器低 6 位

根据以上公式，假设 PCLK3=100Mhz，那么可以得到最小-最大超时时间表如下表所示：

WDGTB	最小超时(us)T[5:0]=0x00	最大超时(ms)T[5:0]=0x3F
0	40.96	2.62
1	81.92	5.24
2	163.84	10.48
3	372.68	20.96

4	655.36	41.94
5	1310.72	83.886
6	2621.44	167.772
7	5242.88	335.544

表 18.1.1.2 100M 时钟下窗口看门狗的最小最大超时表

18.1.2 WWDG 寄存器

WWDG 只有 3 个寄存器，具体如下：

- **控制寄存器（WWDG_CR）**

窗口看门狗的控制寄存器描述如图 18.1.2.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WDGA	T[6:0]						
								rs	rw						

位 7 **WDGA**：激活位 (Activation bit)
此位由软件置 1，只有复位后才由硬件清零。当 WDGA = 1 时，看门狗可产生复位。
0：禁止看门狗
1：使能看门狗

位 6:0 **T[6:0]**：7 位计数器 (7-bit counter)（MSB 到 LSB）
这些位用来存储看门狗计数器的值，每隔 (4096 x $2^{WDGTB[2:0]}$) PCLK 个周期递减一次。当它从 0x40 递减到 0x3F（T6 清零）时会产生复位。

图 18.1.2.1 WWDG_CR 寄存器

该寄存器只有低八位有效，其中 T[6：0]用来存储看门狗的计数器的值，随时更新的，每隔（4096×2^ WDGTB[2:0]）PCLK 个周期减 1。当该计数器的值从 0X40 变为 0X3F 的时候，将产生看门狗复位。

WDGA 位则是看门狗的激活位，该位由软件置 1，启动看门狗，并且一定要注意的是该位一旦设置，就只能在硬件复位后才能清零了。

- **配置寄存器（WWDG_CFR）**

配置寄存器描述如图 18.1.2.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	WDGTB[2:0]			Res.	EWI	Res.	Res.	W[6:0]						
		rw	rw	rw		rs			rw	rw	rw	rw	rw	rw	rw

位 31:14 保留，必须保持复位值。

位 13:11 **WDGTB[2:0]**：定时器时基 (Timer base)
可按如下方式修改预分频器的时基：
000：CK 计数器时钟 (PCLK div 4096) 分频器 1
001：CK 计数器时钟 (PCLK div 4096) 分频器 2
010：CK 计数器时钟 (PCLK div 4096) 分频器 4
011：CK 计数器时钟 (PCLK div 4096) 分频器 8
100：CK 计数器时钟 (PCLK div 4096) 分频器 16
101：CK 计数器时钟 (PCLK div 4096) 分频器 32
110：CK 计数器时钟 (PCLK div 4096) 分频器 64
111：CK 计数器时钟 (PCLK div 4096) 分频器 128

位 10 保留，必须保持复位值。

位 9 **EWI**：提前唤醒中断 (Early wakeup interrupt)
置 1 后，只要计数器值达到 0x40 就会产生中断。此中断只有在复位后才由硬件清零。

位 8:7 保留，必须保持复位值。

位 6:0 **W[6:0]**：7 位窗口值 (7-bit window value)
这些位包含用于与递减计数器进行比较的窗口值。

图 18.1.2.2 WWDG_CFR 寄存器

该寄存器中的 EWI 位是提前唤醒中断，如果该位置 1，当递减计数器等于 0x40 时产生提前唤醒中断，我们就可以及时喂狗以避免 WWDG 复位。因此，我们一般都会用该位来设置中断，当窗口看门狗的计数器值减到 0X40 的时候，如果该位设置，并开启了中断，则会产生中断，我们可以在中断里面向 WWDG_CR 重新写入计数器的值，来达到喂狗的目的。注意这里在进入中断后，必须在不大于 1 个窗口看门狗计数周期的时间（在 pclk3 频率为 100M 且 WDGTB[2:0]为 0 的条件下，该时间为 40.96us）内重新写 WWDG_CR，否则，看门狗将产生复位！

- **状态寄存器（WWDG_SR）**

该寄存器用来记录当前是否有提前唤醒的标志。该寄存器仅有位 0 有效，其他都是保留位。当计数器值达到 0x40 时，此位由硬件置 1。它必须通过软件写 0 来清除。对此位写 1 无效。即使中断未被使能，在计数器的值达到 0X40 的时候，此位也会被置 1。

18.2 硬件设计

1. 例程功能

先点亮 LED0 延时 300ms 后，初始化窗口看门狗，进入死循环，关闭 LED0。然后等待窗口看门狗中断的到来，在中断里面，喂狗，并执行 LED1 的翻转操作。我们将通过 LED0 来指示 STM32H7 是否被复位了，如果被复位了就会点亮 300ms。LED1 用来指示中断喂狗，每次中断喂狗翻转一次。

2. 硬件资源

1）LED 灯
LED0 – PB1
LED1 – PB0
2）窗口看门狗

3. 原理图

窗口看门狗属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们通过 LED0 和 LED1 来指示 STM32H743 的复位情况和窗口看门狗的喂狗情况。

18.3 程序设计

18.3.1 WWDG 的 HAL 库驱动

WWDG 在 HAL 库中的驱动代码在 stm32h7xx_hal_wwdg.c 文件（及其头文件）中。

1. HAL_WWDG_Init 函数

WWDG 的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_WWDG_Init(WWDG_HandleTypeDef *hwwdg);
```

- **函数描述：**

  用于初始化 WWDG。

- **函数形参：**

  形参 1 是 WWDG 句柄，WWDG_HandleTypeDef 结构体类型，其定义如下：

```
typedef struct
{
  WWDG_TypeDef                *Instance;  /* WWDG 寄存器基地址 */
  WWDG_InitTypeDef             Init;      /* WWDG 初始化参数 */
}WWDG_HandleTypeDef;
```

1）Instance：指向 WWDG 寄存器基地址。

2）Init：WWDG 初始化结构体，用于配置计数器的相关参数。

WWDG_InitTypeDef 这个结构体类型定义如下：

```
typedef struct
{
  uint32_t Prescaler;   /* 预分频系数 */
  uint32_t Window;      /* 窗口值 */
  uint32_t Counter;     /* 计数器值 */
  uint32_t EWIMode;     /* 提前唤醒中断使能 */
} WWDG_InitTypeDef;
```

1）Prescaler：预分频系数，WWDG_PRESCALER_1 到 WWDG_PRESCALER_128。

2）Window：窗口值，即上限值。

3）Counter：计数器值，用于保存要设置计数器的值。

4）EWIMode：提前唤醒中断使能。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

2. HAL_WWDG_Refresh 函数

HAL_WWDG_Refresh 函数是窗口看门狗的喂狗函数。其声明如下：

```
HAL_StatusTypeDef HAL_WWDG_Refresh(WWDG_HandleTypeDef *hwwdg);
```

- **函数描述：**

  该函数实际就是往 CR 寄存器重写 Counter 这个预先保存的计数器值。

- **函数形参：**

  形参 1 是 WWDG_HandleTypeDef 结构体指针类型的 WWDG 句柄。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

窗口看门狗配置步骤

1）使能 WWDG 时钟

WWDG 不同于 IWDG，IWDG 有自己独立的 32Khz 时钟。而 WWDG 使用的是 PCLK3 的时钟，需要先使能时钟。方法是:

```
__HAL_RCC_WWDG_CLK_ENABLE();
```

2）设置窗口值,分频数和计数器初始值

在 HAL 库中，这三个值都是通过函数 HAL_WWDG_Init 来设置的，详见本例程源码。

3）开启 WWDG

通过设置 WWDG_CR 寄存器的 WDGA(bit7)位为 1 来实现开启窗口看门狗，同样是在 HAL_WWDG_Init 函数里面实现。

4）使能中断通道并配置优先级（如果开启了 WWDG 中断）

WWDG 的中断也是通过：HAL_NVIC_EnableIRQ 函数使能，通过 HAL_NVIC_SetPriority 函数设置优先级。

HAL 库同样为看门狗提供了 MSP 回调函数 HAL_WWDG_MspInit，一般情况下，步骤 1 和步骤 4 的步骤，我们均放在该回调函数中。

5）编写中断服务函数

在最后，还是要编写窗口看门狗的中断服务函数，通过该函数来喂狗，喂狗要快，否则当窗口看门狗计数器值减到 0X3F 的时候，就会引起软复位了。在中断服务函数里面也要将状态寄存器的 EWIF 位清空。

窗口看门狗中断服务函数为：WWDG_IRQHandler，喂狗函数为：HAL_WWDG_Refresh。

6）重写窗口看门狗唤醒中断处理回调函数 HAL_WWDG_EarlyWakeupCallback

HAL 库定义了一个中断处理共用函数 HAL_WWDG_IRQHandler，我们在 WWDG 中断服务函数中会调用该函数。同时该函数会调用回调函数 HAL_WWDG_EarlyWakeupCallback，提前唤醒中断逻辑（喂狗、闪灯）我们写在回调函数 HAL_WWDG_EarlyWakeupCallback 中。

18.3.2 程序流程图

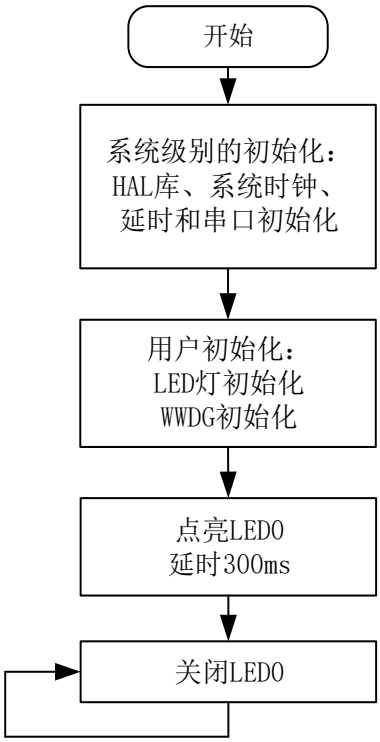


图 18.3.2.1 窗口看门狗实验程序流程图

18.3.3 程序解析

1. WWDG 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。窗口看门狗（WWDG）驱动源码包括两个文件：wdg.c 和 wdg.h。

wdg.h 头文件只有函数的声明，就不解释了。下面我们直接解析 wdg.c 的程序，先看

WWDG 的初始化函数，其定义如下：

```
/**
 * @brief       初始化窗口看门狗
 * @param       tr: T[6:0],计数器值
 * @param       wr: W[6:0],窗口值
 * @param       fprer:分频系数（WDGTB）,范围:WWDG_PRESCALER_1~WWDG_PRESCALER_128,
 *               Fwwdg=PCLK3/(4096*2^fprer). 一般 PCLK3=100Mhz
 *               溢出时间=(4096*2^fprer)*(tr-0X3F)/PCLK3
 *               假设 fprer=4,tr=7f,PCLK3=100Mhz
 *               则溢出时间=4096*16*64/100Mhz=41.94ms
 * @retval      无
 */
void wwdg_init(uint8_t tr, uint8_t wr, uint32_t fprer)
{
    g_wwdg_handle.Instance = WWDG1;
    g_wwdg_handle.Init.Prescaler = fprer;          /* 设置分频系数 */
    g_wwdg_handle.Init.Window = wr;                /* 设置窗口值 */
    g_wwdg_handle.Init.Counter = tr;               /* 设置计数器值 */
    g_wwdg_handle.Init.EWIMode = WWDG_EWI_ENABLE; /* 使能窗口看门狗提前唤醒中断 */
    HAL_WWDG_Init(&g_wwdg_handle);                 /* 初始化 WWDG */
}
```

wwdg_init 是独立看门狗初始化函数，主要设置预分频数、窗口值和计数器的值，以及选择是否使能窗口看门狗提前唤醒中断。

因为用到中断，我们用 HAL_WWDG_MspInit 函数来编写窗口看门狗中断的初始化代码。当然大家也可以 HAL_WWDG_MspInit 函数的代码放到 wwdg_init 函数里面。这个初始化框架就是 HAL 库的特点。

```
/**
 * @brief       WWDG MSP 回调
 * @param       WWDG 句柄
 * @note        此函数会被 HAL_WWDG_Init()调用
 * @retval      无
 */
void HAL_WWDG_MspInit(WWDG_HandleTypeDef *hwwdg)
{
    __HAL_RCC_WWDG_CLK_ENABLE();              /* 使能窗口看门狗时钟 */

    HAL_NVIC_SetPriority(WWDG_IRQn,2,3);     /* 抢占优先级 2，子优先级为 3 */
    HAL_NVIC_EnableIRQ(WWDG_IRQn);            /* 使能窗口看门狗中断 */
}
```

HAL_WWDG_MspInit 函数会被 HAL_WWDG_Init 函数调用。该函数使能窗口看门狗的时钟，并设置窗口看门狗中断的抢占优先级为 2，响应优先级为 3。

```
/**
 * @brief       窗口看门狗中断服务程序
 * @param       无
 * @retval      无
 */
void WWDG_IRQHandler(void)
{
    HAL_WWDG_IRQHandler(&g_wwdg_handle);
}
```

WWDG_IRQHandler 函数是窗口看门狗中断服务函数，而这个函数实际上就是调用 HAL 库的中断处理函数 HAL_WWDG_IRQHandler。逻辑程序在下面的这个回调函数中：

```
/**
 * @brief       窗口看门狗喂狗提醒中断服务回调函数
 * @param       hwwdg 句柄
 * @note        此函数会被 HAL_WWDG_IRQHandler()调用
 * @retval      无
 */
void HAL_WWDG_EarlyWakeupCallback(WWDG_HandleTypeDef *hwwdg)
```

```
{
    HAL_WWDG_Refresh(&g_wwdg_handle);/* 更新窗口看门狗值 */
    LED1_TOGGLE();  /* LED1 闪烁 */
}
```

在回调函数内部调用 HAL_WWDG_Refresh 函数喂狗，并翻转 LED1。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);/* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    led_init();                             /* 初始化 LED */
    LED0(0);                                /* 点亮 LED0(红灯) */
    delay_ms(300);                          /* 延时 300ms 再初始化看门狗,LED0 的变化可见 */
    /* 计数器值为 7f,窗口寄存器为 5f,分频数为 16 */
    wwdg_init(0X7F, 0X5F, WWDG_PRESCALER_16);
    while (1)
    {
        LED0(1);    /* 关闭红灯 */
    }
}
```

在 main 函数里，先初始化系统和用户的外设代码，然后先点亮 LED0，延时 300ms 后，初始化窗口看门狗，进入死循环，关闭 LED0。

调用 wwdg_init(0X7F, 0X5F, WWDG_PRESCALER_16)这个语句，就设置计数器值为 7f，窗口寄存器为 5f，分频数为 16，然后可由前面的公式得到 Twwdg=4096×16×64/100000=41.94ms，即超时时间就是 41.94ms。41.94ms 进中断喂狗一次，并翻转 LED1。

18.4 下载验证

下载代码后，可以看到 LED0 亮了一下就熄灭，紧接着 LED1 开始不停的闪烁。每秒钟闪烁 23 次左右，说明程序在中断不停的喂狗，和我们预期的一致。

第十九章 基本定时器实验

STM32H743 有众多的定时器，其中包括 2 个基本定时器（TIM6 和 TIM7）、10 个通用定时器（TIM2~TIM5，TIM12~ TIM17）、2 个高级控制定时器（TIM1 和 TIM8）和 5 个低功耗定时器（LPTIM1~LPTIM5）。这些定时器彼此完全独立，不共享任何资源。本章我们学习如何使用 STM32H743 的基本定时器中断。我们将使用 TIM6 的定时器中断来控制 LED1 的翻转，在主函数用 LED0 的翻转来提示程序正在运行。

本章分为如下几个小节：

19.1 基本定时器简介
19.2 硬件设计
19.3 程序设计
19.4 下载验证

19.1 基本定时器简介

STM32H743 有两个基本定时器 TIM6 和 TIM7，它们的功能完全相同，资源是完全独立的，可以同时使用。其主要特性如下：16 位自动重载递增计数器，16 位可编程预分频器，预分频系数 1~65536，用于对计数器时钟频率进行分频，还可以触发 DAC 的同步电路，以及生成中断/DMA 请求。

19.1.1 基本定时器框图

下面先来学习基本定时器框图，通过学习基本定时器框图会有一个很好的整体掌握，同时对之后的编程也会有一个清晰的思路。

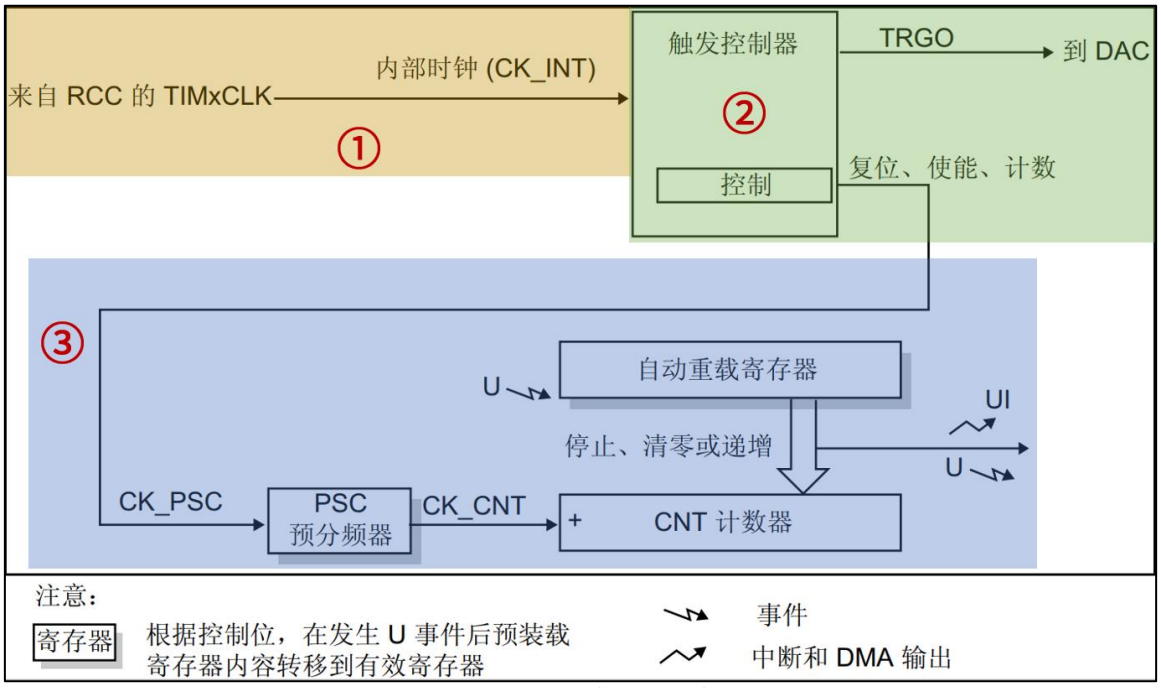


图 19.1.1.1 基本定时器框图

① 时钟源

定时器的核心就是计算器，要实现计算功能，首先要给它一个时钟源。基本定时器时钟挂载在 APB1 总线，所以它的时钟来自于 APB1 总线，但是基本定时器时钟不是直接由 APB1 总线直接提供，而是先经过一个倍频器。在 TIMPRE 位默认设置为 0 的情况下，当 APB1 的 D2PPRE1 预分频器预的分频系数为 1 时，这个倍频器系数就为 1，即定时器的时钟源频率等于

APB1 总线时钟频率；当 APB1 的 D2PPRE1 预分频器的预分频系数≥2 分频时，这个倍频器系数就为 2，即定时器的时钟源频率等于 APB1 总线时钟频率的两倍。我们在时钟设置函数 sys_stm32_clock_init 已经设置 APB1 总线时钟频率为 100MHz，D2PPRE1 预分频器的预分频系数系数为 2，所以定时器时钟源频率为 200Mhz。

我们还可以通过 RCC 时钟配置寄存器(RCC_CFGR)的 TIMPRE 位设置所有定时器的时钟频率，但是基本上我们都是不做修改，默认值为 0 即可。具体可以参考《STM32H7xx 参考手册_V3（中文版）.pdf》第 290 页，表 47。

① 控制器

控制器除了控制定时器复位、使能、计数等功能之外，还可以用于触发 DAC 转换。

② 时基单元

时基单元包括：计数器寄存器(TIMx_CNT)、预分频器寄存器(TIMx_PSC)、自动重载寄存器(TIMx_ARR) 。基本定时器的这三个寄存器都是 16 位有效数字，即可设置值范围是 0~65535。

时基单元中的预分频器 PSC，它有一个输入和一个输出。输入 CK_PSC 来源于控制器部分，实际上就是来自于内部时钟（CK_INT），即 2 倍的 APB1 总线时钟频率（200MHz）。输出 CK_CNT 是分频后的时钟，它是计数器实际的计数时钟，通过设置预分频器寄存器(TIMx_PSC)的值可以得到不同频率 CK_CNT，计算公式如下：

$$f_{CK_CNT} = f_{CK_PSC} / (PSC[15:0] + 1)$$

上式中，PSC[15:0]是写入预分频器寄存器(TIMx_PSC)的值。

另外：预分频器寄存器(TIMx_PSC)可以在运行过程中修改它的数值，新的预分频数值将在下一个更新事件时起作用。因为更新事件发生时，会把 TIMx_PSC 寄存器值更新到其影子寄存器中，这才会起作用。

什么是影子寄存器？从框图上看，可以看到图 19.1.1.1 中的预分频器 PSC 后面有一个影子，自动重载寄存器也有个影子，这就表示这些寄存器有影子寄存器。影子寄存器是一个实际起作用的寄存器，不可直接访问。举个例子：我们可以把预分频系数写入预分频器寄存器(TIMx_PSC)，但是预分频器寄存器只是起到缓存数据的作用，只有等到更新事件发生时，预分频器寄存器的值才会被自动写入其影子寄存器中，这时才真正起作用。

自动重载寄存器及其影子寄存器的作用和上述同理。不同点在于自动重载寄存器是否具有缓冲作用还受到 ARPE 位的控制，当该位置 0 时，ARR 寄存器不进行缓冲，我们写入新的 ARR 值时，该值会马上被写入 ARR 影子寄存器中，从而直接生效；当该位置 1 时，ARR 寄存器进行缓冲，我们写入新的 ARR 值时，该值不会马上被写入 ARR 影子寄存器中，而是要等到更新事件发生才会被写入 ARR 影子寄存器，这时才生效。预分频器寄存器则没有这样相关的控制位，这就是它们不同点。

值得注意的是，更新事件的产生有两种情况，一是由软件产生，将 TIMx_EGR 寄存器的位 UG 置 1，产生更新事件后，硬件会自动将 UG 位清零。二是由硬件产生，满足以下条件即可：计数器的值等于自动重装载寄存器影子寄存器的值。下面来讨论一下硬件更新事件。

基本定时器的计数器（CNT）是一个递增的计数器，当寄存器（TIMx_CR1）的 CEN 位置 1，即使能定时器，每来一个 CK_CNT 脉冲，TIMx_CNT 的值就会递增加 1。当 TIMx_CNT 值与 TIMx_ARR 的设定值相等时，TIMx_CNT 的值就会被自动清零并且会生成更新事件（如果开启相应的功能，就会产生 DMA 请求、产生中断信号或者触发 DAC 同步电路），然后下一个 CK_CNT 脉冲到来，TIMx_CNT 的值就会递增加 1，如此循环。在此过程中，TIMx_CNT 等于 TIMx_ARR 时，我们称之为定时器溢出，因为是递增计数，故而又称为定时器上溢。定时器溢出就伴随着更新事件的发生。

由上述可知，我们只要设置预分频寄存器和自动重载寄存器的值就可以控制定时器更新事件发生的时间。自动重载寄存器(TIMx_ARR)是用于存放一个与计数器作比较的值，当计数器的值等于自动重载寄存器的值时就会生成更新事件，硬件自动置位相关更新事件的标志位，如：更新中断标志位。

下面举个例子：如何设置预分频寄存器和自动重载寄存器的值，得到我们想要的更新事件周期？比如我们需要一个 500ms 周期的定时器更新中断，一般思路是先设置预分频寄存器，然后才是自动重载寄存器。考虑到我们设置的 CK_INT 频率为 200MHz，我们把预分频系数设置为 20000，即写入预分频寄存器的值为 19999，那么 f_{CK_CNT}=200MHz/20000=10KHz。这样得到

计数器的计数频率为 10KHz，即计数器 1 秒钟可以计 10000 个数。我们需要 500ms 周期的更新中断，所以我们要让计数器计 5000 个数就能满足要求，即需要设置自动重载寄存器的值为 4999，另外还要把定时器更新中断使能位 UIE 置 1，CEN 位也要置 1。

19.1.2 TIM6/TIM7 寄存器

下面介绍 TIM6/TIM7 的几个重要的寄存器，具体如下：

- **控制寄存器 1（TIMx_CR1）**

TIM6/TIM7 的控制寄存器 1 描述如图 19.1.2.1 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	Res	Res	Res	UIF RE-MAP	Res	Res	Res	ARPE	Res	Res	Res	OPM	URS	UDIS	CEN
				rw				rw				rw	rw	rw	rw

图 19.1.2.1 TIMx_CR1 寄存器

该寄存器，我们需要注意的是：位 0（CEN）用于使能或者禁止计数器，该位置 1 计数器开始工作，置 0 则停止。还有位 7（APRE）用于控制自动重载寄存器 ARR 是否具有缓冲作用，如果 ARPE 位置 1，ARR 起缓冲作用，即只有在更新事件发生时才会把 ARR 的值写入其影子寄存器里；如果 ARPE 位置 0，那么修改自动重载寄存器的值时，该值会马上被写入其影子寄存器中，从而立即生效。

- **DMA/中断使能寄存器（TIMx_DIER）**

TIM6/TIM7 的 DMA/中断使能寄存器描述如图 19.1.2.2 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	Res	Res	Res	Res	Res	Res	UDE	Res	Res	Res	Res	Res	Res	Res	UIE
							rw								rw

位 8 **UDE**：更新 DMA 请求使能 (Update DMA request enable)
0：禁止更新 DMA 请求。
1：使能更新 DMA 请求。
位 0 **UIE**：更新中断使能 (Update interrupt enable)
0：禁止更新中断。
1：使能更新中断。

图 19.1.2.2 TIMx_DIER 寄存器

该寄存器位 0（UIE）用于使能或者禁止更新中断，因为本实验我们用到中断，所以该位需要置 1。位 8（UDE）用于使能或者禁止更新 DMA 请求，我们暂且用不到，置 0 即可。

- **状态寄存器（TIMx_SR）**

TIM6/TIM7 的状态寄存器描述如图 19.1.2.3 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	UIF
															rc_w0

位 0 **UIF**：更新中断标志 (Update interrupt flag)
该位在发生更新事件时通过硬件置 1。但需要通过软件清零。
0：未发生更新。
1：更新中断挂起。该位在以下情况下更新寄存器时由硬件置 1：
– 重复计数器值上溢或下溢并且 TIMx_CR1 寄存器中的 UDIS=0 时。
– 如果 TIMx_CR1 寄存器中 URS = 0 且 UDIS = 0，通过软件使用 TIMx_EGR 寄存器中的 UG 位重新初始化 CNT 时。

图 19.1.2.3 TIMx_SR 寄存器

该寄存器位 0（UIF）是中断更新的标志位，当发生中断时由硬件置 1，然后就会执行中断服务函数，需要软件去清零，所以我们必须在中断服务函数里把该位清零。如果中断到来后，不把该位清零，那么系统就会一直进入中断服务函数，这显然不是我们想要的。

- **计数器寄存器（TIMx_CNT）**

TIM6/TIM7 的计数器寄存器描述如图 19.1.2.4 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UIF CPY	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31 **UIFCPY**：UIF 副本 (UIF Copy)
该位是 TIMx_ISR 寄存器中 UIF 位的只读副本。如果 TIMx_CR1 中的 UIFREMAP 位复位，则位 31 保留，读为 0。

位 30:16 保留，必须保持复位值。

位 15:0 **CNT[15:0]**：计数器值 (Counter value)

图 19.1.2.4 TIMx_CNT 寄存器

该寄存器位[15:0]就是计数器的实时的计数值。

- **预分频寄存器（TIMx_PSC）**

TIM6/TIM7 的预分频寄存器描述如图 19.1.2.5 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSC[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 15:0 **PSC[15:0]**：预分频器值 (Prescaler value)
计数器时钟频率 CK_CNT 等于 f_{CK_PSC} / (PSC[15:0] + 1)。
PSC 包含在每次发生更新事件时要装载到实际预分频器寄存器的值。
（包括当计数器通过 TIMx_EGR 寄存器的 UG 位或在“复位模式”下通过触发控制器清零时）。

图 19.1.2.5 TIMx_PSC 寄存器

该寄存器是 TIM6/TIM7 的预分频寄存器，比如我们要 20000 分频，就往该寄存器写入 19999。注意这是 16 位的寄存器，写入的数值范围：0 到 65535，分频系数范围：1 到 65536。

- **自动重载寄存器（TIMx_ARR）**

TIM6/TIM7 的自动重载寄存器描述如图 19.1.2.6 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 15:0 **ARR[15:0]**：预分频器值 (Prescaler value)
ARR 为要装载到实际自动重载寄存器的值。
有关 ARR 更新和行为的详细信息，请参见*第 1690 页的第 42.3.1 节：时基单元*。
当自动重载值为空时，计数器不工作。

图 19.1.2.6 TIMx_ARR 寄存器

该寄存器可以由 APRE 位设置是否进行缓冲。计数器的值会和 ARR 寄存器影子寄存器进行比较，当两者相等，定时器就会溢出，从而发生更新事件，如果打开更新中断，还会发生更新中断。

19.1.3 基本定时器中断应用

本实验，我们主要配置定时器产生周期性溢出，从而在定时器更新中断中做周期性的操作，如周期性翻转 LED 灯。假设计数器计数模式为递增计数模式，那么实现周期性更新中断原理示意图如下所示：

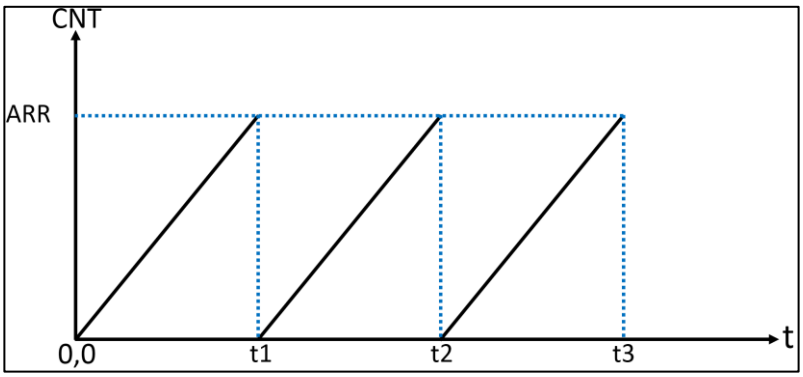


图 19.1.3.1 定时器更新中断原理示意图

如图 19.1.3.1 所示，CNT 计数器从 0 开始计数，当 CNT 的值和 ARR 相等时（t1），产生一个更新中断，然后 CNT 复位（清零），然后继续递增计数，依次循环。图中的 t1、t2、t3 就是定时器更新中断产生的时刻。

通过修改 ARR 的值，可以改变定时时间。另外，通过修改 PSC 的值，使用不同的计数频率（改变图中 CNT 的斜率），也可以改变定时时间。

19.2 硬件设计

1. 例程功能

LED0 用来指示程序运行，每 200ms 翻转一次。我们在更新中断中，将 LED1 的状态取反。LED1 用于指示定时器发生更新事件的频率，500ms 取反一次。

2. 硬件资源

1）LED 灯

LED0：LED0 - PB1

LED1：LED1 - PB0

2）定时器 6

3. 原理图

定时器属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们通过 LED1 来指示 STM32H743 的定时器进入中断的频率。

19.3 程序设计

19.3.1 定时器的 HAL 库驱动

定时器在 HAL 库中的驱动代码在 stm32h7xx_hal_tim.c 和 stm32h7xx_hal_tim_ex.c 文件（以及它们的头文件）中。

1. HAL_TIM_Base_Init 函数

定时器的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_TIM_Base_Init(TIM_HandleTypeDef *htim);
```

- **函数描述：**

  用于初始化定时器。

- **函数形参：**

  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量（亦称定时器句柄），结构体定义如下：

```
typedef struct
{
  TIM_TypeDef                 *Instance;     /* 外设寄存器基地址 */
  TIM_Base_InitTypeDef        Init;          /* 定时器初始化结构体*/
```

```
  HAL_TIM_ActiveChannel        Channel;      /* 定时器通道 */
  DMA_HandleTypeDef            *hdma[7];     /* DMA 管理结构体 */
  HAL_LockTypeDef              Lock;         /* 锁定资源 */
  __IO HAL_TIM_StateTypeDef    State;        /* 定时器状态 */
  __IO HAL_TIM_ChannelStateTypeDef     ChannelState[6];  /* 通道运行状态 */
  __IO HAL_TIM_ChannelStateTypeDef     ChannelNState[4]; /* 互补通道运行状态 */
  __IO HAL_TIM_DMABurstStateTypeDef    DMABurstState;    /* DMA 突发操作状态 */
}TIM_HandleTypeDef;
```

1）Instance：指向定时器寄存器基地址。
2）Init：定时器初始化结构体，用于配置定时器的相关参数。
3）Channel：定时器的通道选择，基本定时器没有该功能。
4）hdma[7]：用于配置定时器的 DMA 请求。
5）Lock：ADC 锁资源。
6）State：定时器工作状态。

我们主要看 TIM_Base_InitTypeDef 这个结构体类型定义：

```
typedef struct
{
  uint32_t Prescaler;              /* 预分频系数 */
  uint32_t CounterMode;            /* 计数模式 */
  uint32_t Period;                 /* 自动重载值 ARR */
  uint32_t ClockDivision;          /* 时钟分频因子 */
  uint32_t RepetitionCounter;      /* 重复计数器 */
  uint32_t AutoReloadPreload;      /* 自动重载预装载使能 */
} TIM_Base_InitTypeDef;
```

1）Prescaler：预分频系数，即写入预分频寄存器的值，范围 0 到 65535。
2）CounterMode：计数器计数模式，这里基本定时器只能向上计数。
3）Period：自动重载值，即写入自动重载寄存器的值，范围 0 到 65535。
4）ClockDivision：时钟分频因子，也就是定时器时钟频率 CK_INT 与数字滤波器所使用的采样时钟之间的分频比。
5）RepetitionCounter：设置重复计数器寄存器的值，用在高级定时器中。
6）AutoReloadPreload：自动重载预装载使能，即控制寄存器 1 (TIMx_CR1)的 ARPE 位。

- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

2. HAL_TIM_Base_Start_IT 函数

HAL_TIM_Base_Start_IT 函数是更新定时器中断和使能定时器的函数。其声明如下：

```
HAL_StatusTypeDef HAL_TIM_Base_Start_IT(TIM_HandleTypeDef *htim);
```

- **函数描述：**
  该函数调用了__HAL_TIM_ENABLE_IT 和__HAL_TIM_ENABLE 两个函数宏定义，分别是更新定时器中断和使能定时器的宏定义。
- **函数形参：**
  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量，即定时器句柄。
- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。
- **注意事项：**
  下面分别列出单独使能/关闭定时器中断和使能/关闭定时器方法：

```
__HAL_TIM_ENABLE_IT(htim, TIM_IT_UPDATE);      /* 使能句柄指定的定时器更新中断 */
__HAL_TIM_DISABLE_IT (htim, TIM_IT_UPDATE);    /* 关闭句柄指定的定时器更新中断 */
__HAL_TIM_ENABLE(htim);                        /* 使能句柄 htim 指定的定时器 */
__HAL_TIM_DISABLE(htim);                       /* 关闭句柄 htim 指定的定时器 */
```

定时器中断配置步骤

1）开启定时器时钟

HAL 中定时器使能是通过宏定义标识符来实现对相关寄存器操作的，方法如下：

```
    __HAL_RCC_TIMx_CLK_ENABLE();  /* x=1~17 */
```

2）初始化定时器参数,设置自动重装值，分频系数，计数方式等

定时器的初始化参数是通过定时器初始化函数 HAL_TIM_Base_Init 实现的。

注意：该函数会调用：HAL_TIM_Base_MspInit 函数，我们可以通过后者存放定时器时钟和中断等初始化的代码。

3）使能定时器更新中断，开启定时器计数，配置定时器中断优先级

通过 HAL_TIM_Base_Start_IT 函数使能定时器更新中断和开启定时器计数。

通过 HAL_NVIC_EnableIRQ 函数使能定时器中断，通过 HAL_NVIC_SetPriority 函数设置中断优先级。

4）编写中断服务函数

定时器中断服务函数为：TIMx_IRQHandler 等，当发生中断的时候，程序就会执行中断服务函数。HAL 库提供了一个定时器中断公共处理函数 HAL_TIM_IRQHandler，该函数又会调用 HAL_TIM_PeriodElapsedCallback 等一些回调函数，需要用户根据中断类型选择重定义对应的中断回调函数来处理中断程序。

19.3.2 程序流程图

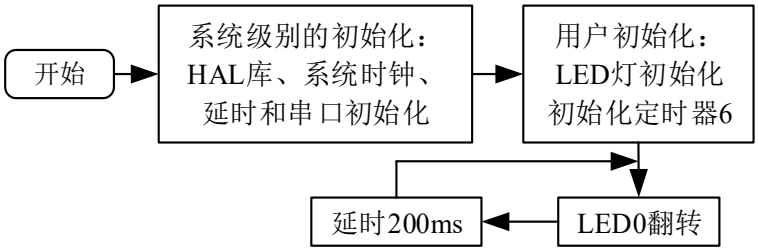


图 19.3.2.1 基本定时器中断实验程序流程图

程序开始先进行一系列初始化，然后在 main 中让 LED0 每过 200ms 翻转一次，用于指示系统代码正在运行。LED1 的翻转将在定时器更新中断里进行，请看程序解析。

19.3.3 程序解析

1. 基本定时器中断驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。基本定时器驱动源码包括两个文件：btim.c 和 btim.h。

首先看 btim.h 头文件的几个宏定义：

```
/* TIMX 中断定义
 * 默认是针对 TIM6\TIM7.
 * 注意: 通过修改这 4 个宏定义,可以支持 TIM1~TIM17 任意一个定时器.
 */
#define BTIM_TIMX_INT                   TIM6
#define BTIM_TIMX_INT_IRQn              TIM6_DAC_IRQn
#define BTIM_TIMX_INT_IRQHandler        TIM6_DAC_IRQHandler
/* TIM6 时钟使能 */
#define BTIM_TIMX_INT_CLK_ENABLE()      do{ __HAL_RCC_TIM6_CLK_ENABLE();}while(0)
```

通过修改这 4 个宏定义，可以支持 TIM1~TIM17 任意一个定时器，本实验中没有用到 BTIM_TIMX_INT_IRQn 这个宏定义。

下面我们解析 btim.c 的程序，先看定时器的初始化函数，其定义如下：

```
/**
 * @brief     基本定时器 TIMX 定时中断初始化函数
 * @note
 *            基本定时器的时钟来自 APB1,当 D2PPRE1≥2 分频的时候
 *            基本定时器的时钟为 APB1 时钟的 2 倍, 而 APB1 为 100M, 所以定时器时钟 = 200Mhz
 *            定时器溢出时间计算方法: Tout = ((arr + 1) * (psc + 1)) / Ft us.
 *            Ft=定时器工作频率,单位:Mhz
 *
```

```
 * @param       arr: 自动重装值。
 * @param       psc: 时钟预分频数
 * @retval      无
 */
void btim_timx_int_init(uint16_t arr, uint16_t psc)
{
    g_timx_handle.Instance = BTIM_TIMX_INT;                 /* 定时器 x */
    g_timx_handle.Init.Prescaler = psc;                     /* 预分频 */
    g_timx_handle.Init.CounterMode = TIM_COUNTERMODE_UP;    /* 递增计数器 */
    g_timx_handle.Init.Period = arr;                        /* 自动装载值 */
    HAL_TIM_Base_Init(&g_timx_handle);
    HAL_TIM_Base_Start_IT(&g_timx_handle);    /* 使能定时器 x 和定时器 x 更新中断 */
}
```

btim_timx_int_init 函数用来初始化定时器，我们可以通过修改宏定义 BTIM_TIMX_INT 来初始化 TIM1~TIM17 中的任意一个，本章我们是初始化基本定时器 6。该函数的 2 个形参：arr 设置自动重载寄存器(TIMx_ARR)，psc 设置预分频器寄存器(TIMx_PSC)。HAL_TIM_Base_Init 函数初始化定时器后，再调用 HAL_TIM_Base_Start_IT 函数使能定时器和定时器更新中断。

因为我们在 sys_stm32_clock_init 函数里面已经初始化 APB1 的时钟为 HCLK 的 2 分频，所以 APB1 的时钟为 100M。当 APB1 的时钟分频数为 1 的时候，TIM2~7 以及 TIM12~14 的时钟为 APB1 的时钟，而如果 APB1 的时钟分频数不为 1，那么 TIM2~7 以及 TIM12~14 的时钟频率将为 APB1 时钟的两倍。因此，TIM6 的时钟为 200M，再根据我们设置的 arr 和 psc 的值，就可以得到定时器的溢出时间计算公式：

$$Tout = ((arr+1) * (psc+1))/Tclk$$

其中：

Tout：定时器溢出时间（单位为 s）。

Tclk：定时器的时钟源频率（单位为 MHz）。

arr：自动重装寄存器（TIMx_ARR）的值。

psc：预分频器寄存器（TIMx_PSC）的值

下面，再看到 HAL_TIM_Base_MspInit 函数，其定义如下：

```
/**
 * @brief       定时器底层驱动，开启时钟，设置中断优先级
                此函数会被 HAL_TIM_Base_Init()函数调用
 * @param       htim:定时器句柄指针
 * @retval      无
 */
void HAL_TIM_Base_MspInit(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == BTIM_TIMX_INT)
    {
        BTIM_TIMX_INT_CLK_ENABLE();                         /* 使能 TIMx 时钟 */
        HAL_NVIC_SetPriority(BTIM_TIMX_INT_IRQn, 1, 3);/* 抢占 1，子优先级 3 */
        HAL_NVIC_EnableIRQ(BTIM_TIMX_INT_IRQn);             /* 开启 ITMx 中断 */
    }
}
```

HAL_TIM_Base_MspInit 函数用于存放 GPIO、NVIC 和时钟相关的代码，这里首先判断定时器的寄存器基地址，满足条件后，首先设置使能定时器的时钟，然后设置定时器中断的抢占优先级为 1，响应优先级为 3，最后开启定时器中断。这里没有用到 IO 引脚，所以不用初始化 GPIO。

接着是定时器中断服务函数的定义，这里用的是宏名，其定义如下：

```
/**
 * @brief       定时器中断服务函数
 * @param       无
 * @retval      无
 */
void BTIM_TIMX_INT_IRQHandler(void)
{
    HAL_TIM_IRQHandler(&g_timx_handle);
```

```
}
```

这个函数实际上调用 HAL 库的定时器中断公共处理函数 HAL_TIM_IRQHandler。HAL 库的中断公共处理函数，会根据中断标志位调用各个中断回调函数，中断要处理的逻辑代码就写到这个回调函数中。比如这里我们使用到的是更新中断，定义的更新中断回调函数如下：

```
/**
 * @brief       定时器更新中断回调函数
 * @param       htim:定时器句柄指针
 * @retval      无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == BTIM_TIMX_INT)
    {
        LED1_TOGGLE();
    }
}
```

更新中断回调函数是所有定时器公用的，所以我们就需要在更新中断回调函数中对定时器寄存器基地址进行判断，只有符合对应定时器发生的更新中断，才能进行相应的处理，从而避免多个定时器同时使用到更新中断，导致更新中断代码的逻辑混乱。这里我们使用定时器 6 的更新中断，所以进入更新中断回调函数后，先判断是不是定时器 6 的寄存器基地址，当然这里使用宏的形式，BTIM_TIMX_INT 的原型就是 TIM6，执行的逻辑代码是翻转 LED1。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    led_init();                             /* 初始化 LED */
    btim_timx_int_init(5000 - 1,20000 - 1);/* 10Khz 的计数频率，计数 5K 次为 500ms */

    while (1)
    {
        LED0_TOGGLE();
        delay_ms(200);
    }
}
```

在 main 函数里，先初始化系统和用户的外设初始化代码，然后在 wilhe(1)里每 200ms 翻转一次 LED0。由前面的内容知道，定时器 6 的时钟频率为 200MHz，而调用 btim_timx_int_init 初始化函数之后，就相当于写入预分频寄存器的值为 19999，写入自动重载寄存器的值为 4999。由公式得：

$$Tout = ((4999+1)*(19999+1))/200000000 = 0.5s = 500ms$$

19.4 下载验证

下载代码后，可以看到 LED0 不停闪烁（每 400ms 一个周期），而 LED1 也是不停的闪烁，但是闪烁时间较 LED0 慢（每 1s 一个周期）。

第二十章 通用定时器实验

本章我们主要来学习通用定时器，上一章我们知道 STM32H743 有 10 个通用定时器（TIM2~TIM5，TIM12~ TIM17）。我们将通过四个实验来学习通用定时器的各个功能，分别是通用定时器中断实验、通用定时器 PWM 输出实验、通用定时器输入捕获实验和通用定时器脉冲计数实验。

本章分为如下几个小节：

20.1 通用定时器简介
20.2 通用定时器中断实验
20.3 通用定时器 PWM 输出实验
20.4 通用定时器输入捕获实验
20.5 通用定时器脉冲计数实验

20.1 通用定时器简介

STM32H743 的通用定时器有 10 个之多，其基本特性也是不尽相同，为了更好的区别各个定时器的特性，我们列了一个表格，如下所示：

定时器类型	定时器	计数器位数	计数模式	预分频系数（整数）	产生 DMA 请求	捕获/比较通道	互补输出	最大接口时钟（MHz）	最大定时器（MHz）
基本定时器	TIM6 TIM7	16	递增	1~65536	可以	0	无	120	240
通用定时器	TIM2 TIM5	32	递增、递减、中心对齐	1~65536	可以	4	无	120	240
	TIM3 TIM4	16	递增、递减、中心对齐	1~65536	可以	4	无	120	240
	TIM12	16	递增	1~65536	不可以	2	无	120	240
	TIM13 TIM14	16	递增	1~65536	不可以	1	无	120	240
	TIM15	16	递增	1~65536	可以	2	有	120	240
	TIM16 TIM17	16	递增	1~65536	可以	1	有	120	240
高级定时器	TIM1 TIM8	16	递增、递减、中心对齐	1~65536	可以	4	有	120	240

表 20.1.1 定时器基本特性表

注：该表摘自《STM32H743IIT6.pdf》，原表在该文档的 3.28 小节（第 42 页）。

由上表知道：除了 TIM2 和 TIM5 是 32 位的计数器，其他定时器是 16 位的。通用定时器和高级定时器其实也就是在基本定时器的基础上，添加了一些其他功能，如：输入捕获、输出比较、输出 PWM 和单脉冲模式等。而通用定时器数量较多，其特性也有一些的差异，但是基本原理都一样。

通用定时器框图

下面我们以 TIM2/TIM3/TIM4/TIM5 的框图为例来学习通用定时器框图，其他通用定时器

的框图会有差异，因为内容比较多，大家学习了这里的框图再看 ST 官方手册其他的定时器框图就会比较容易理解。通过学习通用定时器框图会有一个很好的整体掌握，同时对之后的编程也会有一个清晰的思路。

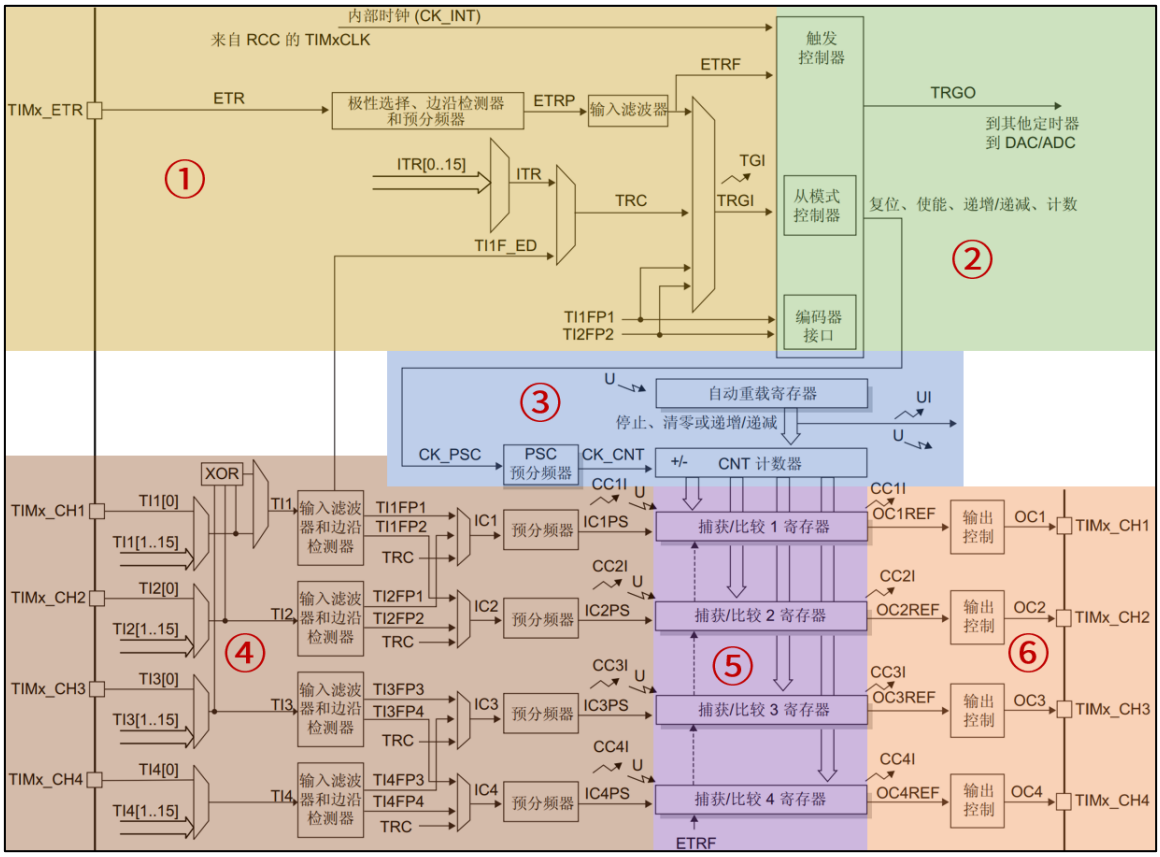


图 20.1.1 通用定时器框图

如上图，通用定时器的框图比基本定时器的框图复杂许多，为了方便介绍，我们将其分成六个部分讲解：

① 时钟源

通用定时器时钟可以选择下面四类时钟源之一：

1）内部时钟(CK_INT)

2）外部时钟模式 1：外部输入引脚(TIx)，x=1，2（即只能来自于通道 1 或者通道 2）

3）外部时钟模式 2：外部触发输入(ETR)

4）内部触发输入(ITRx)：使用一个定时器作为另一定时器的预分频器

通用定时器时钟源的设置方法如下表所示：

定时器时钟类型	设置方法
内部时钟(CK_INT)	设置 TIMx_SMCR 的 SMS=0000
外部时钟模式 1：外部输入引脚(TIx)	设置 TIMx_SMCR 的 SMS=1111
外部时钟模式 2：外部触发输入(ETR)	设置 TIMx_SMCR 的 ECE=1
内部触发输入(ITRx)	设置可参考《STM32H7xx 参考手册_V3（中文版）.pdf》39.3.19 小节

表 20.1.2 通用定时器时钟源设置方法

内部时钟（CK_INT）

STM32H7 系列的 TIM2/TIM3/TIM4/TIM5/ TIM6/TIM7/ TIM12/ TIM13/ TIM14 都是挂载在 APB1 总线上，这些定时器的内部时钟(CK_INT)实际上来自于 APB1 总线提供的时钟。但是这些定时器时钟不是由 APB1 总线直接提供，而是要先经过一个倍频器。在 HAL 库版本例程源码的 sys.c 文件中，系统时钟初始化函数 sys_stm32_clock_init 已经设置 APB1 总线时钟频率为 100MHz，D2PPRE1 预分频器的预分频系数为 2，所以这些定时器时钟源频率为 200MHz。因

为在 TIMPRE 位默认设置为 0 的情况下，当 APB1 的 D2PPRE1 预分频器的预分频系数≥2 分频时，挂载在 APB1 总线上的定时器时钟频率是该总线时钟频率的两倍。这个和基本定时器一样，可回顾基本定时器这部分内容。

APB2 总线上挂载的通用定时器 TIM15/TIM16/TIM17，以及高级定时器 TIM1 和 TIM8，它们的情况是上面的描述是一样的，不同点是：定时器挂载的总线变成了 APB2，需要看的预分频器变成了 D2PPRE2。在系统时钟初始化函数 sys_stm32_clock_init 已经设置 APB2 总线时钟频率为 100MHz，D2PPRE2 预分频器的预分频系数为 2，所以上述的定时器时钟源频率为 200MHz。具体可以参考《STM32H7xx 参考手册_V3（中文版）.pdf》第 290 页，表 47。

外部时钟模式 1（TI1、TI2）

外部时钟模式 1 这类时钟源，顾名思义时钟信号来自芯片外部。时钟源进入定时器的流程如下：外部时钟源信号→IO→TIMx_CH1（或者 TIMx_CH2），这里需要注意的是：外部时钟模式 1 下，时钟源信号只能从 CH1 或者 CH2 输入到定时器，CH3 和 CH4 都是不可以的。从 IO 到 TIMx_CH1（或者 TIMx_CH2），就需要我们配置 IO 的复用功能，才能使 IO 和定时器通道相连通。

时钟源信号来到定时器 CH1 或 CH2 后，需要经过什么“关卡”才能到达计数器作为计数的时钟频率的，下面通过外部时钟模式 1 框图给大家解答。

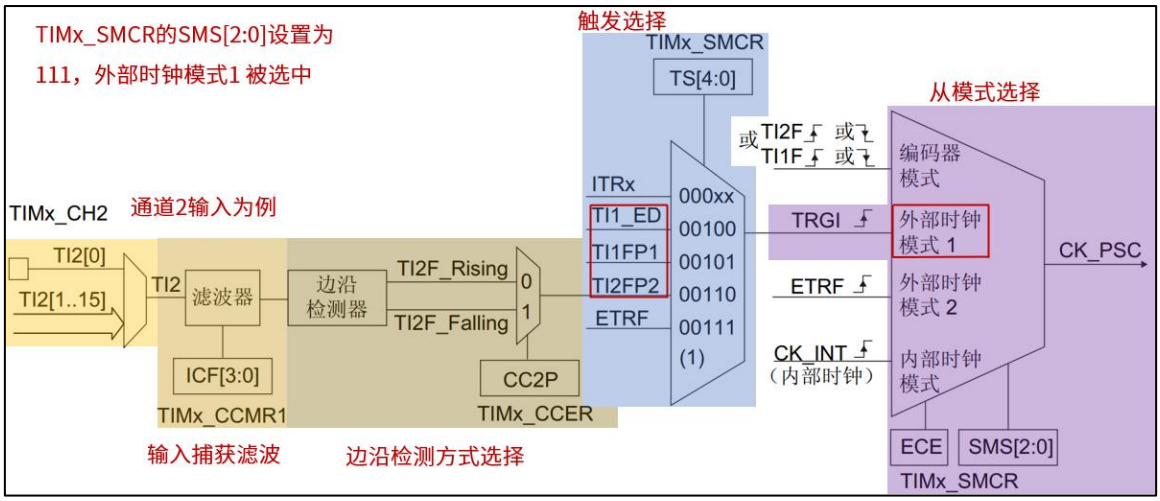


图 20.1.2 外部时钟模式 1 框图

图 20.1.2 中是以 CH2（通道 2）为例的，时钟源信号到达 CH2 后，那么这里我们把这个时钟源信号用 TI2 表示，因为它只是个信号，来到定时器内部，那我们就按定时器内部的信号来命名，所谓入乡随俗。

TI2 首先经过一个滤波器，由 ICF[3:0]位来设置滤波方式，也可以设置不使用滤波器。

接着经过边沿检测器，由 CC2P 位来设置检测的边沿，可以上升沿或者下降沿检测。

然后经过触发输入选择器，由 TS[4:0]位来选择 TRGI（触发输入信号）的来源。可以看到图 20.1.2 中框出了 TI1F_ED、TI1FP1 和 TI2FP2 三个触发输入信号（TRGI）。TI1F_ED 表示来自于 CH1，并且没有经过边沿检测器过滤的信号，所以它是 CH1 的双边沿信号，即上升沿或者下降沿都是有效的。TI1FP1 表示来自 CH1 并经过边沿检测器后的信号，可以是上升沿或者下降沿。TI2FP2 表示来自 CH2 并经过边沿检测器后的信号，可以是上升沿或者下降沿。这里以 CH2 为例，那只能选择 TI2FP2。如果是 CH1 为例，那就可以选择 TI1F_ED 或者 TI1FP1。

最后经过从模式选择器，由 ECE 位和 SMS[2:0]位来选择定时器的时钟源。这里我们介绍的是外部时钟模式 1，所以 ECE 位置 0，SMS[2:0] = 111 即可。CK_PSC 需要经过定时器的预分频器分频后，最终就能到达计数器进行计数了。

外部时钟模式 2（ETR）

外部时钟模式 2，顾名思义时钟信号来自芯片外部。时钟源进入定时器的流程如下：外部时钟源信号→IO→TIMx_ETR。从 IO 到 TIMx_ETR，就需要我们配置 IO 的复用功能，才能使 IO 和定时器相连通。

时钟源信号来到定时器 TIMx_ETR 后，需要经过什么“关卡”才能到达计数器作为计数的

时钟频率的，下面通过外部时钟模式 2 框图给大家解答。

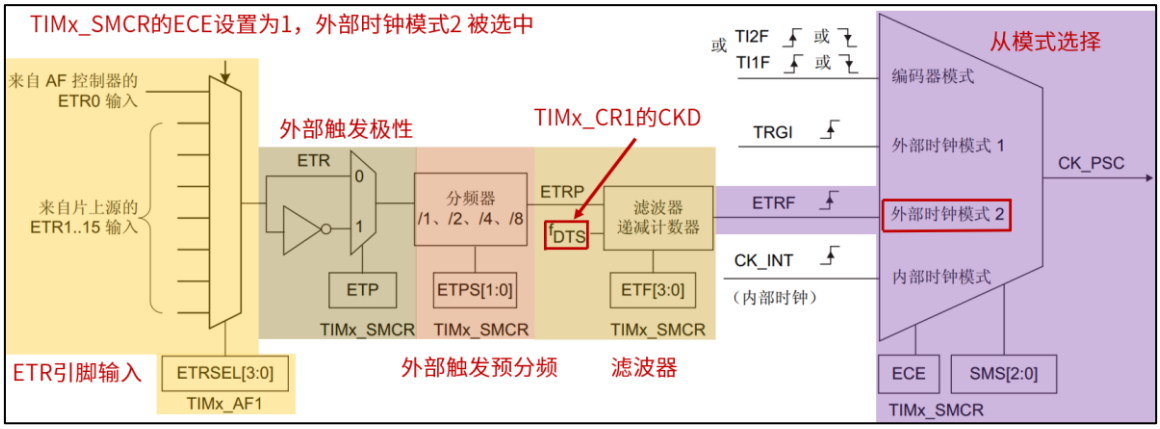


图 20.1.3 外部时钟模式 2 框图

图 20.1.3 中，首先配置 TIMx_AF1 寄存器中的 ETRSEL[3:0]位选择正确的 ETR 源（内部或外部，这里我们选择外部 IO 作为时钟源，所以 ETRSEL[3:0]位设置为 0000 即可）。注意：不是所有通用定时器都有这个寄存器的，这个需要查看参考手册。因为外部时钟模式 2 的时钟信号是来自 IO 外部的，ETRSEL[3:0]位默认就是 0000，所以我们可以直接不用设置这个寄存器。

接着经过外部触发极性选择器，由 ETP 位来设置上升沿有效还是下降沿有效，选择下降沿有效的话，信号会经过反相器。

然后经过外部触发预分频器，由 ETPS[1:0]位来设置预分频系数，系数范围：1、2、4、8。

紧接着经过滤波器器，由 ETF[3:0]位来设置滤波方式，也可以设置不使用滤波器。fDTS 由 TIMx_CR1 寄存器的 CKD 位设置。

最后经过从模式选择器，由 ECE 位和 SMS[2:0]位来选择定时器的时钟源。这里我们介绍的是外部时钟模式 2，直接把 ECE 位置 1 即可。CK_PSC 需要经过定时器的预分频器分频后，最终就能到达计数器进行计数了。

内部触发输入（ITRx）

内部触发输入是使用一个定时器作为另一个定时器的预分频器，即实现定时器的级联。下面以 TIM3 作为 TIM2 的预分频器为例，给大家介绍。

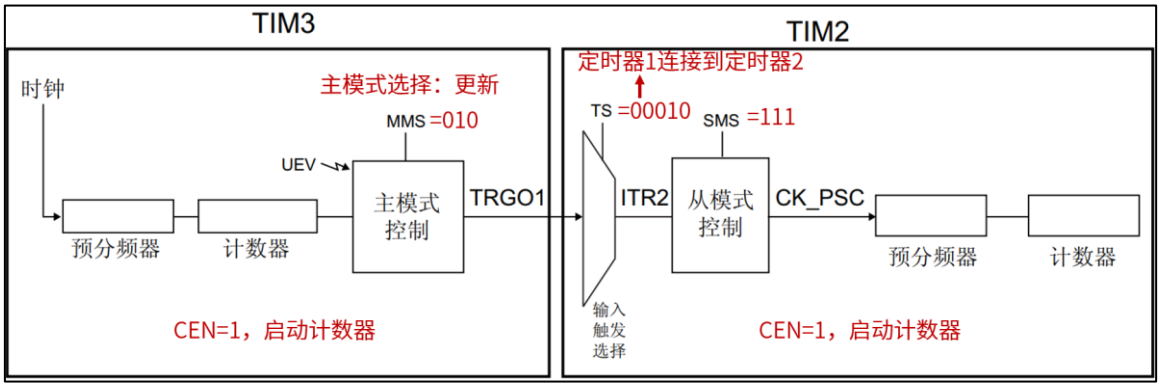


图 20.1.4 TIM3 作为 TIM2 的预分频器框图

上图中，TIM3 作为 TIM2 的预分频器，需要完成的配置步骤如下：

1，TIM3_CR2 寄存器的 MMS[2:0]位设置为 010，即 TIM3 的主模式选择为更新（选择更新事件作为触发输出 （TRGO））。

2，TIM2_SMCR 寄存器的 TS[4:0]位设置为 00010，即使用 ITR2 作为内部触发。TS[4:0]位用于配置触发选择，除了 ITR2，还有其他的选择，详细描述如下图所示：

位 6:4 **TS[4:0]**：触发选择 (Trigger selection)（请参见 TS[4:3] 的位 21:20）
此位域可选择将要用于同步计数器的触发输入。
00000：内部触发 0 (ITR0)
00001：内部触发 1 (ITR1)
00010：内部触发 2 (ITR2)
00011：内部触发 3 (ITR3)
00100：TI1 边沿检测器 (TI1F_ED)
00101：滤波后的定时器输入 1 (TI1FP1)
00110：滤波后的定时器输入 2 (TI2FP2)
00111：外部触发输入 (ETRF)
01000：内部触发 4 (ITR4)
01001：内部触发 5 (ITR5)
01010：内部触发 6 (ITR6)
01011：内部触发 7 (ITR7)
01100：内部触发 8 (ITR8)
其它：保留
有关各定时器 ITRx 含义的详细信息，请参见*第 1538 页的表 317：TIMx 内部触发连接*。

图 20.1.5 触发选择

上图中的触发选择中，我们在讲解外部时钟模式1的时候说过TI1F_ED、TI1FP1和TI2FP2，以及外部时钟模式 2 讲的 ETRF，它们都是属于外部的，其余的都是内部触发了。那么这内部触发都代表什么意思呢？大家打开《STM32H7xx参考手册_V3（中文版）.pdf》的1538页，就可以找下面这个表。

从 TIM	TIM2	TIM3	TIM4	TIM5
ITR0	TIM1	TIM1	TIM1	TIM1
ITR1	TIM8	TIM2	TIM2	TIM8
ITR2	TIM3	TIM15	TIM3	TIM3
ITR3	TIM4	TIM4	TIM8	TIM4
ITR4	ETH PPS	ETH PPS	-	-
ITR5	USB1 OTG_HS_SOF	-	-	-
ITR6	USB2 OTG_FS_SOF	-	-	fdcan1_soc
ITR7	-	-	-	USB1 OTG_HS_SOF
ITR8	-	-	-	USB2 OTG_FS_SOF

表 20.1.3 TIMx 内部触发连接

在步骤 2 中，TS[4:0]位设置为 00010，使用 ITR2 作为内部触发，这个 ITR2 什么意思？由表 20.1.3 可以知道，当从模式定时器为 TIM2 时，ITR2 表示主模式定时器就是 TIM3。这里只是 TIM2~5 的内部触发连接情况，其他定时器请查看参考手册的相应章节。

3，TIM2_SMCR 寄存器的 SMS[2:0]位设置为 111，即从模式控制器选择外部时钟模式 1。

4，TIM3 和 TIM2 的 CEN 位都要置 1，即启动计数器。

定时器的时钟源这部分内容是非常重要的，因为这计数器工作的基础。虽然定时器有四类时钟源之多，但是我们最常用的还是内部时钟。

② 控制器

控制器包括：从模式控制器、编码器接口和触发控制器（TRGO）。从模式控制器可以控制计数器复位、启动、递增/递减、计数。编码器接口针对编码器计数。触发控制器用来提供触发信号给别的外设，比如为其它定时器提供时钟或者为 DAC/ADC 的触发转换提供信号。

③ 时基单元

时基单元包括：计数器寄存器(TIMx_CNT)、预分频器寄存器(TIMx_PSC)、自动重载寄存器(TIMx_ARR)。这部分内容和基本定时器基本一样的，大家可以参考基本定时器的介绍。

不同点是：通用定时器的计数模式有三种：递增计数模式、递减计数模式和中心对齐模式；TIM2 和 TIM5 的计数器是 32 位的。递增计数模式在讲解基本定时器的时候已经讲过了，那么对应到递减计数模式就很好理解了。就是来了一个计数脉冲，计数器就减 1，直到计数器寄存器的值减到 0，减到 0 时定时器溢出，由于是递减计数，故而称为定时器下溢，定时器溢出就会伴随着更新事件的发生。然后计数器又从自动重载寄存器影子寄存器的值开始继续递减计数，如此循环。最后是中心对齐模式，字面上不太好理解。该模式下，计数器先从 0 开始递增计数，直到计数器的值等于自动重载寄存器影子寄存器的值减 1 时，定时器上溢，同时生成更新事件，然后从自动重载寄存器影子寄存器的值开始递减计算，直到计数值等于 1 时，定时器下溢，同时生成更新事件，然后又从 0 开始递增计数，依此循环。每次定时器上溢或下溢都会生成更新事件。计数器的计数模式的设置请参考 TIMx_CR1 寄存器的位 CMS 和位 DIR。

下面通过一张图给大家展示定时器工作在不同计数模式下，更新事件发生的情况。

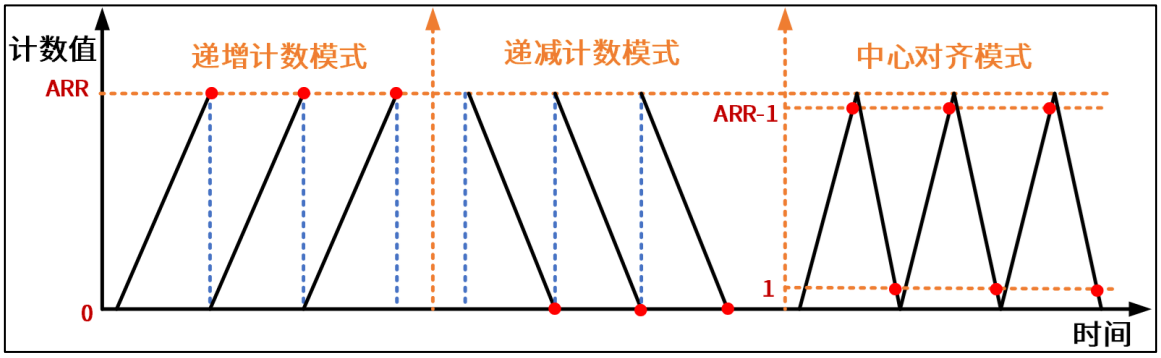


图 20.1.6 更新事件发生条件

上图中，纵轴表示计数器的计数值，横轴表示时间，ARR 表示自动重载寄存器的值，小红点就是更新事件发生的时间点。举个例子，递增计数模式下，当计数值等于 ARR 时，计数器的值被复位为 0，定时器溢出，并伴随着更新事件的发生，后面继续递增计数。递减计数模式和中心对齐模式请参考前面的描述。

上表的描述属于硬件更新事件发生条件，我们还可以通过 UG 位产生软件更新事件。

关于影子寄存器和定时器溢出时间计算公式等内容可以参考基本定时器的相关内容。

④ 输入捕获

图 20.1.1 中的第④部分是输入捕获，一般应用是要和第⑤部分一起完成测量功能。TIMx_CH1~ TIMx_CH4 表示定时器的 4 个通道，这 4 个通道都是可以独立工作的。IO 端口通过复用功能与这些通道相连。配置好 IO 端口的复用功能后，将需要测量的信号输入到相应的 IO 端口，输入捕获部分可以对输入的信号的上升沿，下降沿或者双边沿进行捕获，常见的测量有：测量输入信号的脉冲宽度、测量 PWM 输入信号的频率和占空比等。后续有相应的实验。

下面简单说一下测量高电平脉冲宽度的工作原理，方便大家的理解：一般先要设置输入捕获的边沿检测极性，如：我们设置上升沿检测，那么当检测到上升沿时，定时器会把计数器 CNT 的值锁存到相应的捕获/比较寄存器 TIMx_CCRy 里，y=1~4。然后我们再设置边沿检测为下降沿检测，当检测到下降沿时，定时器会把计数器 CNT 的值再次锁存到相应的捕获/比较寄存器 TIMx_CCRy 里。最后，我们将前后两次锁存的 CNT 的值相减，就可以算出高电平脉冲期间内计数器的计数个数，再根据定时器的计数频率就可以计算出这个高电平脉冲的时间。如果要测量的高电平脉宽时间长度超过定时器的溢出时间周期，就会发生溢出，这时候我们还需要做定时器溢出的额外处理。低电平脉冲捕获同理。

上面的描述是第④部分输入捕获整体上的一个应用情况，下面我们来看第④部分的细节。当需要测量的信号进入通道后，需要经过哪些“关卡”？我们用图 20.1.7 给大家讲解。

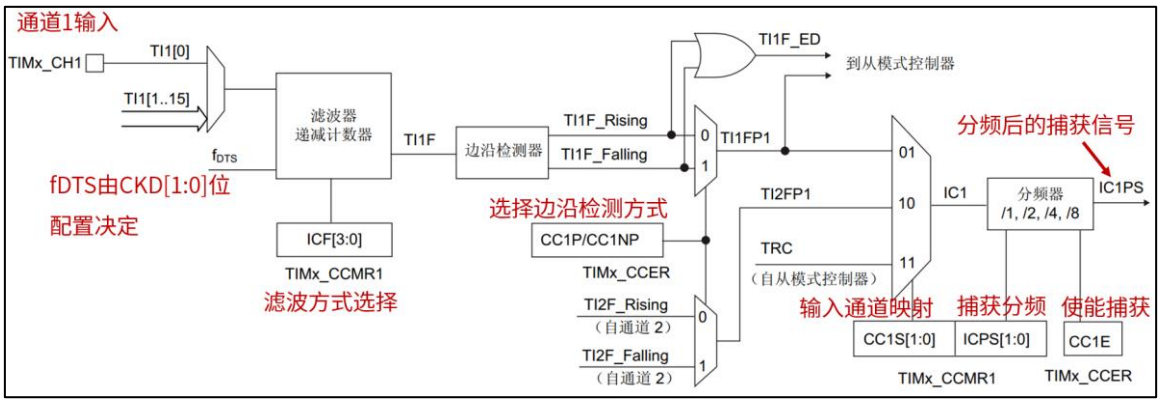


图 20.1.7 通道 1 输入阶段

图 20.1.7 是图 20.1.1 第④部分通道 1 的“放大版”，这里是以通道 1 输入捕获为例进行介绍，其他通道同理。

待测量信号到达 TIMx_CH1 后，那么这里我们把这个待测量信号用 TI1 表示，原因在讲解外部时钟模式 1 的时候说过，所谓“入乡随俗”。

TI1 首先经过一个滤波器，由 ICF[3:0]位来设置滤波方式，也可以设置不使用滤波器。fDTS 由 TIMx_CR1 寄存器的 CKD 位设置。

接着经过边沿检测器，由 CC1P 位来设置检测的边沿，可以上升沿或者下降沿检测。CC1NP 是配置互补通道的边沿检测的，在高级定时器才有，通用定时器没有。

然后经过输入捕获映射选择器，由 CC1S[1:0]位来选择把 IC1 映射到 TI1、TI2 还是 TRC。这里我们的待测量信号从通道 1 进来，所以选择 IC1 映射到 TI1 上即可。

紧接着经过输入捕获 1 预分频器，由 ICPS[1:0]位来设置预分频系数，范围：1、2、4、8。

最后需要把 CC1E 位置 1，使能输入捕获，IC1PS 就是分频后的捕获信号。这个信号将会到达图 20.1.1 的第⑤部分。

下面我们接着看图 20.1.1 的第⑤部分的“放大版”，如下图所示：

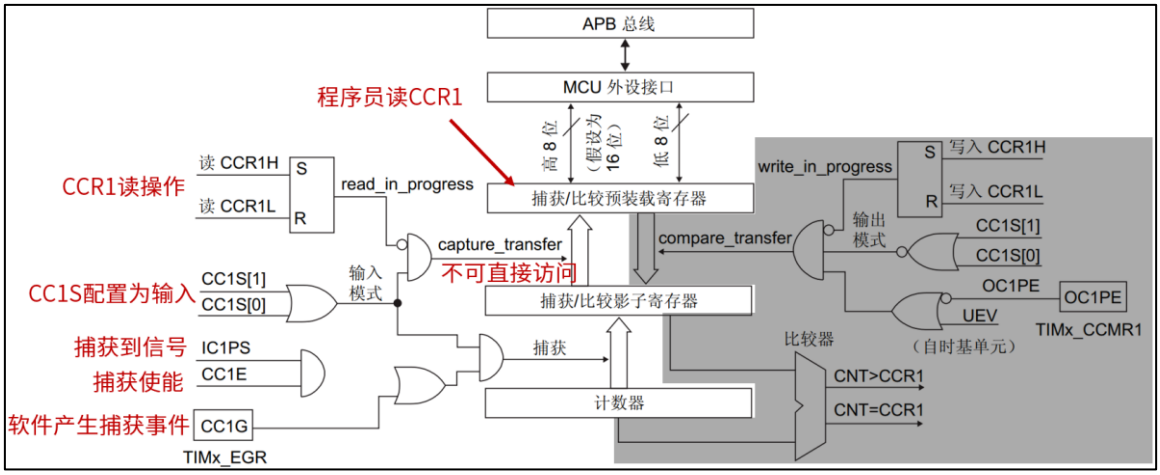


图 20.1.8 捕获/比较通道 1 主电路（输入捕获功能部分）

图 20.1.8 中，灰色阴影部分是输出比较功能部分，讲到第⑥部分输出比较的时候再介绍。左边没有阴影部分就是输入捕获功能部分了。

首先看到捕获/比较预装载寄存器，我们以通道 1 为例，那么它就是 CCR1 寄存器，通道 2、通道 3、通道 4 就分别对应 CCR2、CCR3、CCR4。在图 20.1.1 中就可以看到 CCR1~4 是有影子寄存器的，所以这里就可以看到图 20.1.8 中有捕获/比较影子寄存器，该寄存器不可直接访问。

图 20.1.8 左下角的 CC1G 位可以产生软件捕获事件，那么硬件捕获事件如何产生的？这里我们还是以通道 1 输入为例，CC1S[1:0] = 01，即 IC1 映射到 TI1 上；CC1E 位置 1，使能输入捕获；比如不滤波、不分频，ICF[3:0] = 00，ICPS[1:0] = 00；比如检测上升沿，CC1P 位置 0；接着就是等待测量信号的上升沿到来。当上升沿到来时，IC1PS 信号就会触发输入捕获事件发

生，计数器的值就会被锁存到捕获/比较影子寄存器里。当 CCR1 寄存器没有被进行读操作的时候，捕获/比较影子寄存器里的值就会锁存到 CCR1 寄存器中，那么程序员就可以读取 CCR1 寄存器，得到计数器的计数值。检测下降沿同理。

⑤ 输入捕获和输出比较公用部分

该部分需要结合第④部分或者第⑥部分共同完成相应功能。

⑥ 输出比较

图 20.1.1 中的第⑥部分是输出比较，一般应用是要和第⑤部分一起完成定时器输出功能。TIMx_CH1~ TIMx_CH4 表示定时器的 4 个通道，这 4 个通道都是可以独立工作的。IO 端口通过复用功能与这些通道相连。

下面我们按照输出信号产生过程顺序给大家介绍定时器如何实现输出功能的？首先看到第⑤部分的“放大版”图，如下图所示：

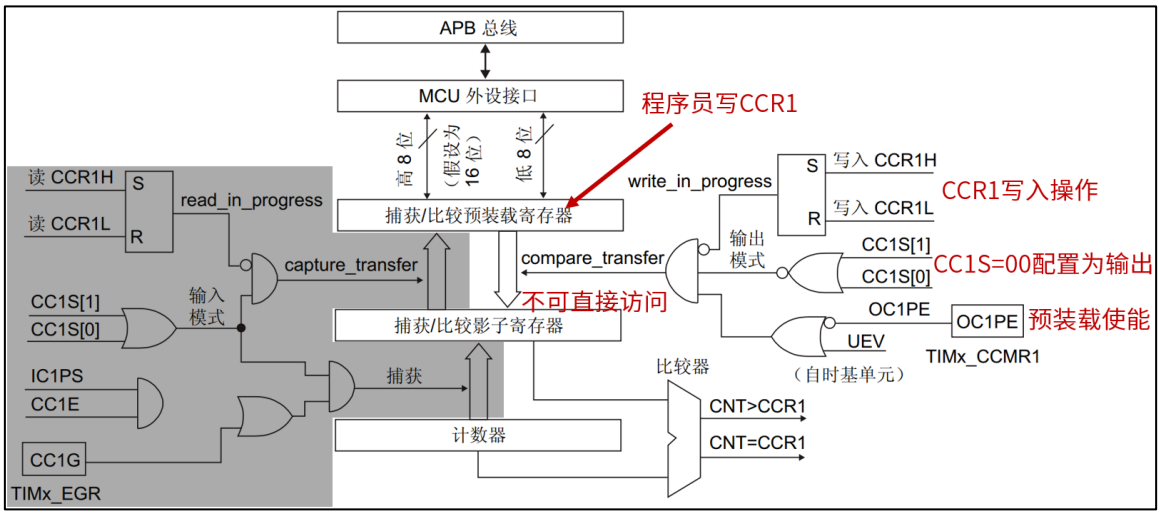


图 20.1.9 捕获/比较通道 1 主电路（输出比较功能部分）

图 20.1.9 中，灰色阴影部分是输入捕获功能部分，前面已经讲过。这里我们看到右边没有阴影部分就是输出比较功能部分了。下面以通道 1 输出比较功能为例给大家介绍定时器如何实现输出功能的。

首先程序员写 CCR1 寄存器，即写入比较值。这个比较值需要转移到对应的捕获/比较影子寄存器后才会真正生效。什么条件下才能转移？图 20.1.9 中可以看到 compare_transfer 旁边的与门，需要满足三个条件：CCR1 不在写入操作期间、CC1S[1:0] = 0 配置为输出、OC1PE 位置 0（或者 OC1PE 位置 1，并且需要发生更新事件，这个更新事件可以软件产生或者硬件产生）。

当 CCR1 寄存器的值转移到其影子寄存器后，新的值就会和计数器的值进行比较，它们的比较结果将会通过第⑥部分影响定时器的输出。

下面来看看第⑥部分通道 1 的“放大版”，如下图所示：

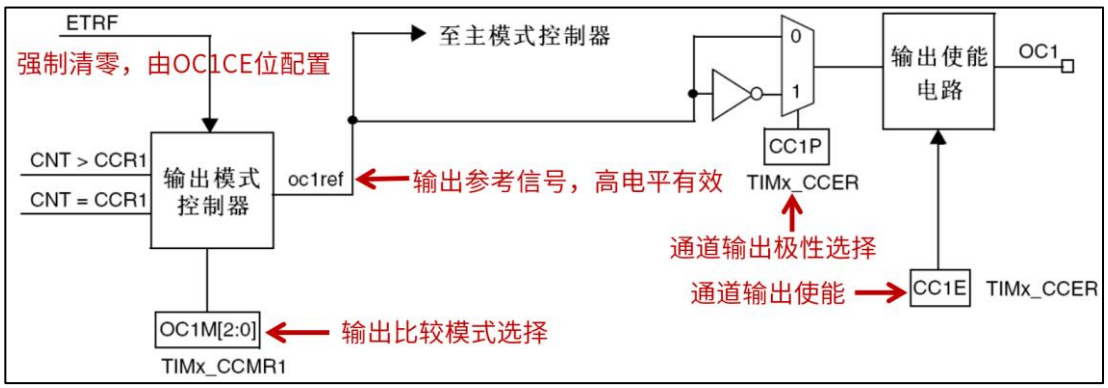


图 20.1.10 通道 1 输出阶段

上图中，可以看到输出模式控制器，由 OC1M[3:0]位配置输出比较模式，该位的描述请参

考《STM32H7xx 参考手册_V3（中文版）.pdf》相关定时器章节的 TIMx_CCMR1 寄存器。H7 系列有 14 种输出比较模式之多，后面用到再来介绍。

oc1ref 是输出参考信号，高电平有效，为高电平时称之为有效电平，为低电平时称之为无效电平。它的高低电平受到三个方面的影响：OC1M[3:0]位配置的输出比较模式、第⑤部分比较器的比较结果、还有就是 OC1CE 位配置的 ETRF 信号。ETRF 信号可以将 Oc1ref 电平强制清零，该信号来自 IO 外部。

一般来说，当计数器的值和捕获/比较寄存器的值相等时，输出参考信号 oc1ref 的极性就会根据我们选择的输出比较模式而改变。如果开启了比较中断，还会发生比较中断。

CC1P 位用于选择通道输出极性。

CC1E 位置 1 使能通道输出。

OC1 信号就会从 TIMx_CH1 输出到 IO 端口，再到 IO 外部。

下面分别通过四个实验来学习通用定时器的功能。

20.2 通用定时器中断实验

本小节我们来学习通用定时器中断实验，以定时器 3 中断为例，首先来了解相关的寄存器。

20.2.1 TIM2/TIM3/TIM4/TIM5 寄存器

下面介绍 TIM2/TIM3/TIM4/TIM5 的几个与定时器中断相关且重要的寄存器，其他的通用定时器的寄存器会有一些差异，请大家自行对比《STM32H7xx 参考手册_V3（中文版）.pdf》相关章节。下面先来看 TIMx_CR1 寄存器的一些位描述，具体如下：

- **控制寄存器 1（TIMx_CR1）**

TIM2/TIM3/TIM4/TIM5 的控制寄存器 1 描述如图 20.2.1.1 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	UIF RE-MAP	Res.	CKD[1:0]		ARPE	CMS		DIR	OPM	URS	UDIS	CEN
				rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 7 **ARPE**：自动重载预装载使能 (Auto-reload preload enable)
0：TIMx_ARR 寄存器不进行缓冲　　1：TIMx_ARR 寄存器进行缓冲
位 6:5 **CMS**：中心对齐模式选择 (Center-aligned mode selection)
00：边沿对齐模式。计数器根据方向位 (DIR) 递增计数或递减计数。
01：中心对齐模式 1。计数器交替进行递增计数和递减计数。仅当计数器递减计数时，配置为输出的通道（TIMx_CCMRx 寄存器中的 CxS=00）的输出比较中断标志才置 1。
10：中心对齐模式 2。计数器交替进行递增计数和递减计数。仅当计数器递增计数时，配置为输出的通道（TIMx_CCMRx 寄存器中的 CxS=00）的输出比较中断标志才置 1。
11：中心对齐模式 3。计数器交替进行递增计数和递减计数。当计数器递增计数或递减计数时，配置为输出的通道（TIMx_CCMRx 寄存器中的 CxS=00）的输出比较中断标志都会置 1。
注：　只要计数器处于使能状态 (CEN=1)，就不得从边沿对齐模式切换为中心对齐模式。
位 4 **DIR**：方向 (Direction)
0：计数器递增计数　　1：计数器递减计数
注：　当定时器配置为中心对齐模式或编码器模式时，该位为只读状态。
位 0 **CEN**：计数器使能 (Counter enable)
0：禁止计数器　　1：使能计数器
注：　只有事先通过软件将 CEN 位置 1，才可以使用外部时钟、门控模式和编码器模式。而触发模式可通过硬件自动将 CEN 位置 1。
在单脉冲模式下，当发生更新事件时会自动将 CEN 位清零。

图 20.2.1.1 TIMx_CR1 寄存器

上图中我们只列出了本实验需要用的一些位，其中：位 7（ARPE）用于控制自动重载寄存器是否进行缓冲，如果 ARPE 位置 1，ARR 起缓冲作用，即只有在更新事件发生时才会把 ARR 的值写入其影子寄存器里；如果 ARPE 位置 0，那么修改自动重载寄存器的值时，该值会马上被写入其影子寄存器中，从而立即生效。

CMS[1:0]位，用于设置边沿对齐模式还是中心对齐模式，本实验我们使用边沿对齐模式，所以设置为 00 即可。

DIR 位，用于控制定时器的计数方向，我们使用递增计数模式，所以设置 DIR 位为 0。

CEN 位，用于使能计数器的工作，必须要设置该位为 1，计数器才会开始计数。

- **从模式控制寄存器（TIMx_SMCR）**

TIM2/TIM3/TIM4/TIM5 的从模式控制寄存器描述如图 20.2.1.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS[4:3]		Res.	Res.	Res.	SMS[3]
										rw	rw				rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETP	ECE	ETPS[1:0]		ETF[3:0]				MSM	TS[2:0]			Res.	SMS[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw

图 20.2.1.2 TIMx_SMCR 寄存器

该寄存器的 SMS[3:0]位，用于从模式选择，其实就是选择计数器输入时钟的来源。比如通用定时器中断实验我们设置 SMS[3:0]=0000，禁止从模式，这样 PSC 预分频器的时钟就直接来自内部时钟（CK_INT），按照我们例程 sys_stm32_clock_init 函数的配置，频率为 200MHz（APB1 总线时钟频率的 2 倍）。

- **TIMx DMA/中断使能寄存器 (TIMx_DIER)**

TIM2/TIM3/TIM4/TIM5 的 DMA/中断使能寄存器描述如图 20.2.1.3 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	TDE	Res.	CC4DE	CC3DE	CC2DE	CC1DE	UDE	Res.	TIE	Res.	CC4IE	CC3IE	CC2IE	CC1IE	UIE
	rw		rw	rw	rw	rw	rw		rw		rw	rw	rw	rw	rw

位 0 **UIE**：更新中断使能 (Update interrupt enable)
0：禁止更新中断。
1：使能更新中断。

图 20.2.1.3 TIMx_DIER 寄存器

该寄存器用于使能/失能触发 DMA 请求、捕获/比较中断以及更新中断。本实验只用到更新中断，所以把位 0（UIE）置 1 即可。

- **状态寄存器（TIMx_SR）**

TIM2/TIM3/TIM4/TIM5 的状态寄存器描述如图 20.2.1.4 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	CC4OF	CC3OF	CC2OF	CC1OF	Res.	Res.	TIF	Res.	CC4IF	CC3IF	CC2IF	CC1IF	UIF
			rc_w0	rc_w0	rc_w0	rc_w0			rc_w0		rc_w0	rc_w0	rc_w0	rc_w0	rc_w0

图 20.2.1.4 TIMx_SR 寄存器

该寄存器都是一些中断标志位，比如更新中断标志位、捕获/比较中断标志位等。在通用定时器中断实验我们用到更新中断标志位，当定时器更新中断到来后，位 0（UIF）会由硬件置 1，我们需要在中断服务函数里面把该位清零。

- **计数器寄存器（TIMx_CNT）**

TIM2/TIM3/TIM4/TIM5 的计数器寄存器描述如图 20.2.1.5 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CNT[31]或UIFCPY	CNT[30:16]（取决于定时器）														
rw or r	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CNT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31 值取决于 TIMx_CR1 中的 IUFREMAP。
如果 UIFREMAP = 0
CNT[31]：计数器值的最高有效位（对于 TIM2 和 TIM5）(Most significant bit of counter value (on TIM2 and TIM5))
其它定时器上保留
如果 UIFREMAP = 1
UIFCPY：UIF 副本 (UIF Copy)
该位是 TIMx_ISR 寄存器中 UIF 位的只读副本

位 30:16 **CNT[30:16]**：计数器值的最高有效部分（对于 TIM2 和 TIM5）(Most significant part counter value (on TIM2 and TIM5))

位 15:0 **CNT[15:0]**：计数器值的最低有效部分 (Least significant part of counter value)

图 20.2.1.5 TIMx_CNT 寄存器

因为定时器 2 和定时器 5 的计数器是 32 位的，所以当用到这两个定时器的时候，TIMx_CNT 寄存器的 32 位都是用做计数器寄存器，其他定时器的就跟基本定时器一样，低 16 位有效。可以直接写该寄存器设置计数的初始值，也可以读取该寄存器获取计数器值。

- **预分频寄存器（TIMx_PSC）**

定时器的预分频寄存器都是 16 位的，即写入该寄存器的数值范围是 0 到 65535，表示 1 到 65536 分频。比如我们要 20000 分频，就往该寄存器写入 19999。

- **自动重载寄存器（TIMx_ARR）**

TIM2/TIM3/TIM4/TIM5 的自动重载寄存器描述如图 20.2.1.6 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ARR[31:16]（取决于定时器）															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ARR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:16 **ARR[31:16]**：自动重载值的高 16 位（对于 TIM2 和 TIM5）(High auto-reload value (on TIM2 and TIM5))

位 15:0 **ARR[15:0]**：自动重载值的低 16 位 (Low Auto-reload value)
ARR 为要装载到有效自动重载寄存器的值。
有关 ARR 更新和行为的更多详细信息，请参见*第 1492 页的第 39.3.1 节：时基单元*。
当自动重载值为空时，计数器不工作。

图 20.2.1.6 TIMx_ARR 寄存器

在 H7 系列中，TIM2 和 TIM5 的自动重载寄存器是 32 位的，其他定时器的自动重载寄存器是低 16 位有效。该寄存器可以由 APRE 位设置是否进行缓冲。计数器的值会和自动重装寄存器影子寄存器进行比较，当两者相等，定时器就会溢出，从而发生更新事件，如果打开了更新中断，还会发生更新中断。

20.2.2 硬件设计

1. 例程功能

LED0 用来指示程序正在运行，200ms 翻转一次。LED1 在定时器中断中翻转，500ms 进入中断一次。

2. 硬件资源

1）LED 灯

LED0：LED0 - PB1

LED1：LED1 - PB0

2）定时器 3

3. 原理图

定时器属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们通过 LED1 来指示 STM32H743 的定时器进入中断的频率，LED0 则指示程序的运行状态。

20.2.3 程序设计

本实验的相关 HAL 库驱动以及实验配置步骤请参考基本定时器相关内容，基本一样。不同点是基本定时器只能是递增计数模式，通用定时器可以递增计数模式、递减计数模式和中心对齐模式。

20.2.3.1 程序流程图

下面看看本实验的程序流程图，main 函数中并没有对 LED1 的操作，我们把对 LED1 的操作放到定时器的中断中进行处理：

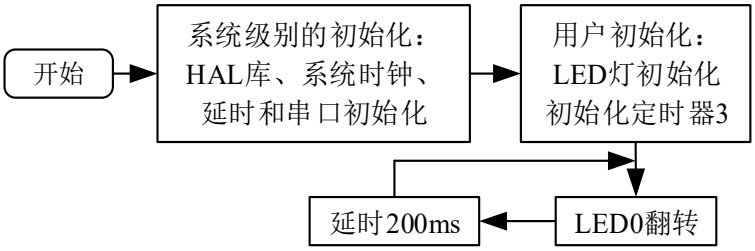


图 20.2.3.1.1 通用定时器中断实验程序流程图

20.2.3.2 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。通用定时器驱动源码包括两个文件：gtim.c 和 gtim.h。

首先看 gtim.h 头文件的几个宏定义：

```
/* TIMX 中断定义 */

#define GTIM_TIMX_INT                   TIM3
#define GTIM_TIMX_INT_IRQn              TIM3_IRQn
#define GTIM_TIMX_INT_IRQHandler        TIM3_IRQHandler
/* TIM3 时钟使能 */
#define GTIM_TIMX_INT_CLK_ENABLE()      do{ __HAL_RCC_TIM3_CLK_ENABLE();}while(0)
```

通过修改这 4 个宏定义，可以支持 TIM1~TIM17 任意一个定时器。

下面再来看一下 gtim.c 文件的代码，主要包括两个函数，先来看看通用定时器的初始化函数，其定义如下：

```
/**
 * @brief    通用定时器 TIMX 定时中断初始化函数
 * @note
 *           通用定时器的时钟来自 APB1,当 D2PPRE1≥2 分频的时候
 *           通用定时器的时钟为 APB1 时钟的 2 倍, 而 APB1 为 100M, 所以定时器时钟 = 200Mhz
 *           定时器溢出时间计算方法: Tout = ((arr + 1) * (psc + 1)) / Ft us.
 *           Ft=定时器工作频率,单位:Mhz
 * @param    arr: 自动重装值。
 * @param    psc: 时钟预分频数
```

```
 * @retval  无
 */
void gtim_timx_int_init(uint16_t arr, uint16_t psc)
{
    GTIM_TIMX_INT_CLK_ENABLE();                                 /* 使能 TIMx 时钟 */

    g_timx_handle.Instance = GTIM_TIMX_INT;                     /* 通用定时器 x */
    g_timx_handle.Init.Prescaler = psc;                         /* 预分频系数 */
    g_timx_handle.Init.CounterMode = TIM_COUNTERMODE_UP;        /* 递增计数模式 */
    g_timx_handle.Init.Period = arr;                            /* 自动装载值 */
    HAL_TIM_Base_Init(&g_timx_handle);

   /* 设置中断优先级，抢占优先级 1，子优先级 3 */
    HAL_NVIC_SetPriority(GTIM_TIMX_INT_IRQn, 1, 3);
    HAL_NVIC_EnableIRQ(GTIM_TIMX_INT_IRQn);    /* 开启 ITMx 中断 */

    HAL_TIM_Base_Start_IT(&g_timx_handle);     /* 使能定时器 x 和定时器 x 更新中断 */
}
```

这里配置的参数和基本定时器中断实验的是一样的，只是这里没有使用到 HAL 库的 HAL_TIM_Base_MspInit 函数来存放 NVIC 和使能时钟的代码，而是全部存放到 gtim_timx_int_init 函数里。在一个项目中，用到多个定时器时，建议大家使用这种方式来处理代码，这样方便代码的管理。

下面再来看看定时器中断服务函数，其定义如下：

```
/**
 * @brief       定时器中断服务函数
 * @param       无
 * @retval      无
 */
void GTIM_TIMX_INT_IRQHandler(void)
{
    /* 以下代码没有使用定时器 HAL 库共用处理函数来处理，而是直接通过判断中断标志位的方式 */
    if(__HAL_TIM_GET_FLAG(&g_timx_handle, TIM_FLAG_UPDATE) != RESET)
    {
        LED1_TOGGLE();
       /* 清除定时器溢出中断标志位 */
        __HAL_TIM_CLEAR_IT(&g_timx_handle, TIM_IT_UPDATE);
    }
}
```

可以看到，这里我们没有使用 HAL 库的定时器公共处理函数来处理中断部分的代码，而是通过自行判断中断标志位的方式来处理。只不过获取标志位的方式还是使用 HAL 库的函数宏__HAL_TIM_GET_FLAG()，大家也可以直接使用寄存器的方式来操作。

通过__HAL_TIM_GET_FLAG()获取中断标志位并判断是否了中断，然后处理中断程序，最后通过__HAL_TIM_CLEAR_IT()将中断标志位清零，这样就完成了一次对中断的处理。这样的方式来处理中断，也是大家学习 HAL 库需要掌握的。在一个项目中，用到多个定时器相关中断时，建议大家使用这种方式来处理代码，这样方便代码的管理。

20.2.4 下载验证

下载代码后，可以看到 LED0 不停闪烁（每 400ms 一个周期），而 LED1 也是不停的闪烁，但是闪烁时间较 LED0 慢（每 1s 一个周期）。

20.3 通用定时器 PWM 输出实验

本小节我们来学习使用通用定时器的 PWM 输出模式。

脉冲宽度调制(PWM)，是英文“Pulse Width Modulation”的缩写，简称脉宽调制，是利用微处理器的数字输出来对模拟电路进行控制的一种非常有效的技术。我们可以让定时器产生 PWM，在计数器频率固定时，PWM 频率或者周期由自动重载寄存器（TIMx_ARR）的值决定，其占空比由捕获/比较寄存器（TIMx_CCRx）的值决定。PWM 产生原理示意图如下图所示：

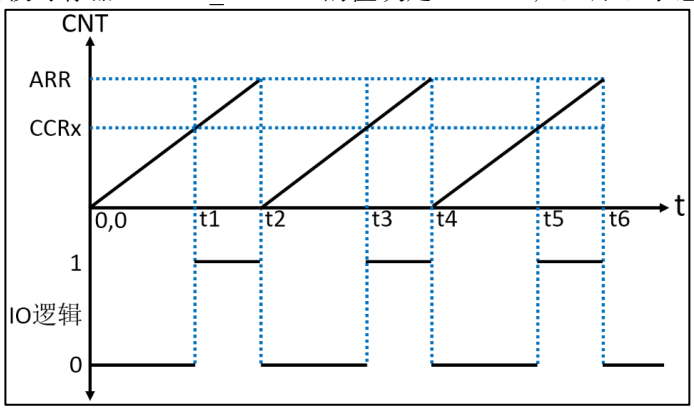


图 20.3.1 PWM 生成示意图

上图中，定时器工作在递增计数模式，纵轴是计数器的计数值 CNT，横轴表示时。当 CNT<CCRx 时，IO 输出低电平（逻辑 0）；当 CNT>=CCRx 时，IO 输出高电平（逻辑 1）；当 CNT=ARR 时，定时器溢出，CNT 的值被清零，然后继续递增，依次循环。在这个循环中，改变 CCRx 的值，就可以改变 PWM 的占空比，改变 ARR 的值，就可以改变 PWM 的频率，这就是 PWM 输出的原理。

定时器产生 PWM 的方式有许多种，下面我们以边沿对齐模式（即递增计数模式/递减计数模式）为例，PWM 模式 1 或者 PWM 模式 2 产生 PWM 的示意图，如下图所示：

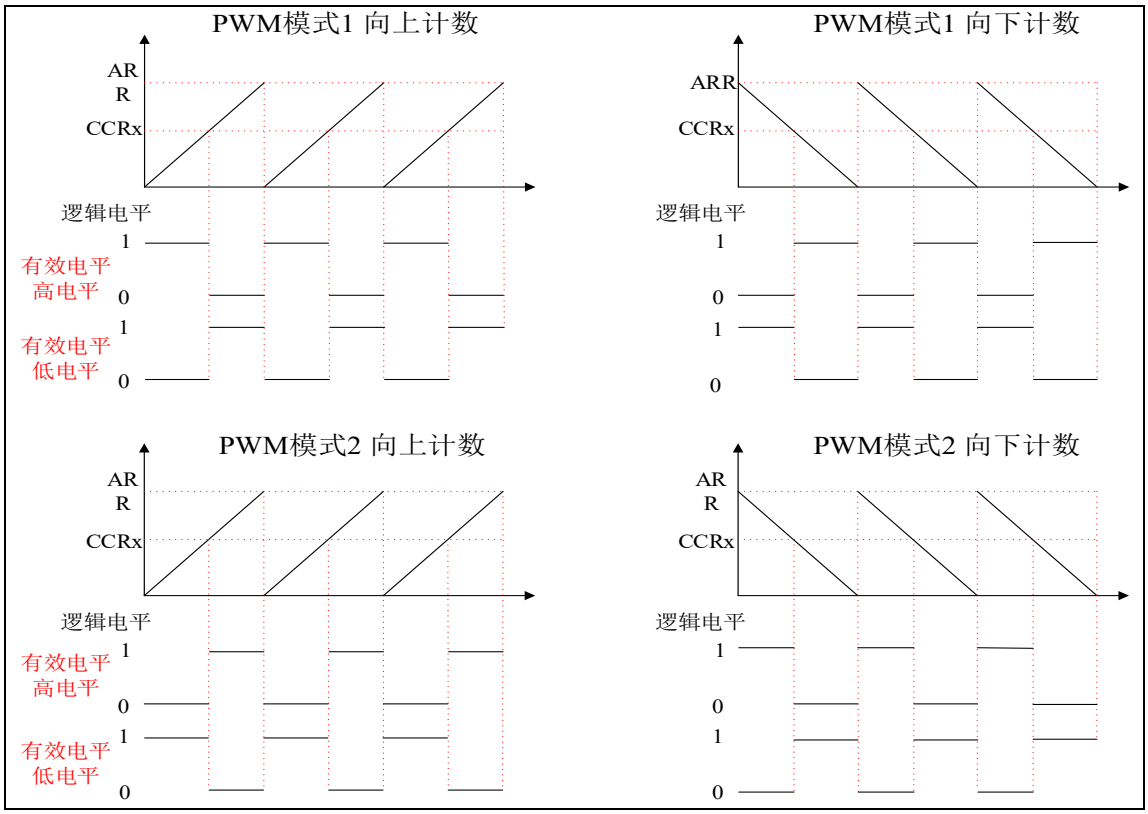


图 20.3.2 产生 PWM 示意图

STM32H743 的定时器除了 TIM6 和 TIM7 外，其他的定时器都可以产生 PWM 输出。其中

高级定时器 TIM1 和 TIM8 可以同时产生多达 7 路的 PWM 输出。而通用定时器也能同时产生多达 4 路的 PWM 输出！本实验我们以 TIM3 的 CH4 产生一路 PWM 输出为例进行学习。

20.3.1 TIM2/TIM3/TIM4/TIM5 寄存器

要使 STM32H743 的通用定时器 TIMx 产生 PWM 输出，除了上一小节介绍的寄存器外，我们还会用到另外 3 个寄存器，来控制 PWM 输出。这三个寄存器分别是：捕获/比较模式寄存器（TIMx_CCMR1/2）、捕获/比较使能寄存器（TIMx_CCER）、捕获/比较寄存器（TIMx_CCR1~4）。接下来我们简单介绍一下这三个寄存器。

- **捕获/比较模式寄存器 1/2（TIMx_CCMR1/2）**

TIM2/TIM3/TIM4/TIM5 的捕获/比较模式寄存器（TIMx_CCMR1/2），该寄存器一般有 2 个：TIMx _CCMR1 和 TIMx _CCMR2。TIMx_CCMR1 控制 CH1 和 CH2，而 TIMx_CCMR2 控制 CH3 和 CH4。TIMx_CCMR2 寄存器描述如图 20.3.1.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC4M[3]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC3M[3]
							Res.								Res.
							rw								rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC4CE	OC4M[2:0]			OC4PE	OC4FE	CC4S[1:0]		OC3CE	OC3M[2:0]			OC3PE	OC3FE	CC3S[1:0]	
IC4F[3:0]				IC4PSC[1:0]				IC3F[3:0]				IC3PSC[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 20.3.1.1 TIMx_CCMR2 寄存器

该寄存器的有些位在不同模式下，功能不一样，我们现在用到输出比较，先看输出比较部分，输入捕获在后面的实验再讲解。关于该寄存器的详细说明，请参考《STM32H7xx 参考手册_V3（中文版）.pdf》第 1542 页，39.4.7 节。比如我们要让 TIM3 的 CH4 输出 PWM 波，该寄存器的模式设置位 OC4M[3:0]就是对应着通道 4 的模式设置。总共可以配置 14 种模式，我们使用的是 PWM 模式 1 或者 PWM 模式 2，所以位 OC4M[3:0]设置为 0110 或者 0111。这两种 PWM 模式的区别就是输出有效电平的极性相反。位 OC4PE 控制输出比较通道 4 的预装载使能，实际就是控制 CCR4 寄存器是否进行缓冲。因为 CCR4 寄存器也是有影子寄存器的，影子寄存器才是真正起作用的寄存器。CC4S[1:0]用于设置通道 4 的方向（输入/输出）默认设置为 0，就是设置通道作为输出使用。

- **捕获/比较使能寄存器（TIMx_ CCER）**

TIM2/TIM3/TIM4/TIM5 的捕获/比较使能寄存器，该寄存器控制着各个输入输出通道的开关和极性。TIMx_CCER 寄存器描述如图 20.3.1.2 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CC4NP	Res.	CC4P	CC4E	CC3NP	Res.	CC3P	CC3E	CC2NP	Res.	CC2P	CC2E	CC1NP	Res.	CC1P	CC1E
rw		rw	rw	rw		rw	rw	rw		rw	rw	rw		rw	rw

图 20.3.1.2 TIMx_CCER 寄存器

该寄存器比较简单，要让 TIM3 的 CH4 输出 PWM 波，这里我们要使能 CC4E 位，该位是通道 4 输入/输出使能位，要想 PWM 从 IO 口输出，这个位必须设置为 1。CC4P 位是设置通道 4 的输出极性，我们默认设置 0。

- **捕获/比较寄存器 1/2/3/4（TIMx_ CCR1/2/3/4）**

捕获/比较寄存器（TIMx_ CCR1/2/3/4），该寄存器总共有 4 个，分别对应 4 个通道 CH1~CH4。我们使用的是通道 4，所以来看看 TIMx_ CCR4 寄存器描述，如图 20.3.1.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CCR4[31:16]（取决于定时器）															
rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR4[15:0]															
rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r

位 31:16 **CCR4[31:16]**：捕获/比较 4 值的高 16 位（对于 TIM2 和 TIM5）(High Capture/Compare 4 value (on TIM2 and TIM5))

位 15:0 **CCR4[15:0]**：捕获/比较 4 值的低 16 位 (Low Capture/Compare 4 value)

1. 如果 CC4 通道配置为输出（CC4S 位）：
CCR4 是捕获/比较寄存器 4 的预装载值。
如果没有通过 TIMx_CCMR2 寄存器中的 OC4PE 位来使能预装载功能，则该值立刻生效；否则只在发生更新事件时生效（拷贝到有效的捕获/比较寄存器 4）。
有效捕获/比较寄存器中包含要与计数器 TIMx_CNT 进行比较并在 OC4 输出上发出信号的值。
2. 如果 CC4 通道配置为输入（TIMx_CCMR4 寄存器中的 CC4S 位）：
CCR4 为上一个输入捕获 4 事件 (IC4) 发生时的计数器值。只能读取 TIMx_CCR4 寄存器，无法对其进行编程。

图 20.3.1.3 TIMx_ CCR4 寄存器

在输出模式下，捕获/比较寄存器影子寄存器的值与 CNT 的值比较，根据比较结果产生相应动作，利用这点，我们通过修改这个寄存器的值，就可以控制 PWM 的占空比了。注意，对于 TIM2 和 TIM5 来说，该寄存器是 32 位有效的，对其他定时器来说，则是 16 位有效位。

- **断路和死区寄存器（TIMx_ BDTR）**

STMH7 系列，通用定时器 TIM15、TIM16 和 TIM17，以及高级定时器 TIM1 和 TIM8，除了配置上述的寄存器之外，还需要配置：断路和死区寄存器（TIMx_BDTR），该寄存器各位描述如图 20.3.1.4 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	BK2P	BK2E	BK2F[3:0]				BKF[3:0]			
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MOE	AOE	BKP	BKE	OSSR	OSSI	LOCK[1:0]		DTG[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 15 **MOE**：主输出使能 (Main output enable)

只要断路输入（BRK 或 BRK2）为有效状态，此位便由硬件异步清零。此位由软件置 1，也可根据 AOE 位状态自动置 1。此位仅对配置为输出的通道有效。

0：响应断路事件（2 个）。禁止 OC 和 OCN 输出。

响应断路事件或向 MOE 写入 0 时：OC 和 OCN 输出被禁止或被强制为空闲状态，具体取决于 OSSI 位。

1：如果 OC 和 OCN 输出的相应使能位（TIMx_CCER 寄存器中的 CCxE 和 CCxNE 位）均置 1，则使能 OC 和 OCN 输出。

图 20.3.1.4 TIMx_ BDTR 寄存器

要想上述定时器的 PWM 正常输出，则必须设置 MOE 位为 1，否则不会有输出。该寄存器的其他位我们这里就不详细介绍了，讲到高级定时器的时候再介绍其它位。

20.3.2 硬件设计

1. 例程功能

使用 TIM3 通道 4（由 PB1 复用）输出 PWM， PB1 引脚连接了 LED0，从而实现 PWM 输出控制 LED0 亮度。

2. 硬件资源

1）LED 灯

LED0: LED0 - PB1

2）定时器 3 输出通道 4（对应 PB1）

3. 原理图

定时器属于 STM32H73 的内部资源，只需要软件设置好即可正常工作。我们通过 LED0 来间接指示定时器的 PWM 输出情况。

20.3.3 程序设计

20.3.3.1 定时器的 HAL 库驱动

定时器在 HAL 库中的驱动代码在前面介绍基本定时器已经介绍了部分，这里我们再介绍几个本实验用到的函数。

1. HAL_TIM_PWM_Init 函数

定时器 PWM 输出基础工作参数初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_TIM_PWM_Init(TIM_HandleTypeDef *htim);
```

- **函数描述：**

  用于初始化定时器的基础工作参数，即初始化 TIM_HandleTypeDef 结构体成员。

- **函数形参：**

  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量，基本定时器的时候已经介绍。

- **函数返回值：**HAL_StatusTypeDef 枚举类型的值。

- **注意事项：**

该函数实现的功能以及使用方法和 HAL_TIM_Base_Init 类似，作用都是初始化定时器的 ARR 和 PSC 等参数。为什么 HAL 库要提供这个函数而不直接让我们使用 HAL_TIM_Base_Init 函数呢？这是因为 HAL 库为定时器的针对 PWM 输出定义了单独的 MSP 回调函数 HAL_TIM_PWM_MspInit，所以当我们调用 HAL_TIM_PWM_Init 进行 PWM 初始化之后，该函数内部会调用 MSP 回调函数 HAL_TIM_PWM_MspInit。当我们使用 HAL_TIM_Base_Init 初始化定时器参数的时候，它内部调用的回调函数是 HAL_TIM_Base_MspInit，这里大家注意区分。

2. HAL_TIM_PWM_ConfigChannel 函数

定时器 PWM 模式通道配置函数。其声明如下：

```
HAL_StatusTypeDef HAL_TIM_PWM_ConfigChannel(TIM_HandleTypeDef *htim,
                                            TIM_OC_InitTypeDef *sConfig, uint32_t Channel);
```

- **函数描述：**

  该函数用于设置定时器的 PWM 输出模式及通道等参数。

- **函数形参：**

  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量，用于配置定时器基本参数。

  形参 2 是 TIM_OC_InitTypeDef 结构体类型指针变量，用于配置定时器输出比较参数。

  下面重点来了解一下 TIM_OC_InitTypeDef 结构体指针类型，其定义如下：

```
typedef struct
{
  uint32_t OCMode;          /* 输出比较模式选择，寄存器的时候说过了，共 14 种模式 */
  uint32_t Pulse;           /* 设置比较值*/
  uint32_t OCPolarity;      /* 设置输出比较极性 */
  uint32_t OCNPolarity;     /* 设置互补输出比较极性 */
  uint32_t OCFastMode;      /* 使能或失能输出比较快速模式 */
  uint32_t OCIdleState;     /* 选择空闲状态下的非工作状态（OC1 输出） */
  uint32_t OCNIdleState;    /* 设置空闲状态下的非工作状态（OC1N 输出） */
} TIM_OC_InitTypeDef;
```

我们重点关注前三个结构体成员。成员变量 OCMode 用来设置模式，这里我们设置为 PWM 模式 1。成员变量 Pulse 用来设置捕获比较值。成员变量 TIM_OCPolarity 用来设置输出极性。其他成员 TIM_OutputNState，TIM_OCNPolarity，TIM_OCIdleState 和 TIM_OCNIdleState 后面用到再介绍。

形参 3 是定时器通道，范围：TIM_CHANNEL_1~6，比如定时器 3 只有 4 个通道，那选择范围就只有 TIM_CHANNEL_1~4，所以要根据具体情况选择。

- **函数返回值：**HAL_StatusTypeDef 枚举类型的值。

3. HAL_TIM_PWM_Start 函数

定时器 PWM 输出启动函数，其声明如下：

```
HAL_StatusTypeDef HAL_TIM_PWM_Start(TIM_HandleTypeDef *htim, uint32_t Channel);
```

- **函数描述：**

  用于使能通道输出和启动计数器，即启动 PWM 输出。

- **函数形参：**

  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量。

  形参 2 是定时器通道，范围：TIM_CHANNEL_1 到 TIM_CHANNEL_6。

- **函数返回值：**HAL_StatusTypeDef 枚举类型的值。
- **注意事项：**

  HAL 库提供了单独使能定时器输出通道的函数，其声明如下：

```
void TIM_CCxChannelCmd(TIM_TypeDef *TIMx, uint32_t Channel,
                                          uint32_t ChannelState);
```

HAL_TIM_PWM_Start 函数内部也是调用了该函数。

4. HAL_TIM_ConfigClockSource 函数

配置定时器时钟源函数，其声明如下：

```
HAL_StatusTypeDef HAL_TIM_ConfigClockSource(TIM_HandleTypeDef *htim,
                                      TIM_ClockConfigTypeDef *sClockSourceConfig);
```

- **函数描述：**

  用于配置定时器时钟源。

- **函数形参：**

  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量。

  形参 2 是 TIM_ClockConfigTypeDef 结构体类型指针变量，用于配置定时器时钟源参数。

  TIM_ClockConfigTypeDef 定义如下：

```
typedef struct
{
  uint32_t ClockSource;         /* 时钟源 */
  uint32_t ClockPolarity;       /* 时钟极性 */
  uint32_t ClockPrescaler;      /* 定时器预分频器 */
  uint32_t ClockFilter;         /* 时钟过滤器 */
} TIM_ClockConfigTypeDef;
```

- **函数返回值：**HAL_StatusTypeDef 枚举类型的值。
- **注意事项：**

  该函数主要配置 TIMx_SMCR 寄存器。默认情况下，定时器的时钟源是内部时钟。本实验就是使用内部时钟的，所以我们不用对时钟源进行初始化，默认即可。这里只是让大家知道有这个函数可以设定时器的时钟源。比如用 HAL_TIM_ConfigClockSource 初始化选择内部时钟，方法如下：

```
TIM_HandleTypeDef timx_handle;          /* 定时器 x 句柄 */
TIM_ClockConfigTypeDef sClockSourceConfig = {0};
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;     /* 选择内部时钟 */
HAL_TIM_ConfigClockSource(&timx_handle, &sClockSourceConfig);
```

后面的定时器初始化凡是用到内部时钟我们都没有去进行初始化，系统默认即可。

定时器 PWM 输出模式配置步骤

1）开启 TIMx 和相应通道输出的 GPIO 时钟，配置该 IO 口的复用功能输出

首先开启 TIMx 的时钟，然后配置 GPIO 为复用功能输出。本实验我们默认用到定时器 3 通道 4，对应 IO 是 PB1，它们的时钟开启方法如下：

```
__HAL_RCC_TIM3_CLK_ENABLE();                /* 使能定时器 3 */
__HAL_RCC_GPIOB_CLK_ENABLE();               /* 开启 GPIOB 时钟 */
```

IO 口复用功能是通过函数 HAL_GPIO_Init 来配置的。

2）初始化 TIMx,设置 TIMx 的 ARR 和 PSC 等参数

使用定时器的 PWM 输出功能时，通过 HAL_TIM_PWM_Init 函数初始化定时器 ARR 和 PSC 等参数。

注意：该函数会调用：HAL_TIM_PWM_MspInit 函数，我们可以通过后者存放定时器和 GPIO 时钟使能、GPIO 初始化、中断使能以及优先级设置等代码。

3）设置定时器为 PWM 模式，输出比较极性，比较值等参数

在 HAL 库中，通过 HAL_TIM_PWM_ConfigChannel 函数来设置定时器为 PWM1 模式或者 PWM2 模式，根据需求设置输出比较的极性，设置比较值（控制占空比）等。

4）使能 TIMx，使能 TIMx 的 CHy 输出

在 HAL 库中，通过调用 HAL_TIM_PWM_Start 函数来使能 TIMx 的某个通道输出 PWM。

5）修改 TIM3_CCR4 来控制占空比

在经过以上设置之后，PWM 其实已经开始输出了，只是其占空比和频率都是固定的，而我们可以通过修改比较值来控制 PWM 的输出占空比。HAL 库中提供一个修改占空比的宏定义：

```
__HAL_TIM_SET_COMPARE (__HANDLE__, __CHANNEL__, __COMPARE__)
```

__HANDLE__是 TIM_HandleTypeDef 结构体类型指针变量，__CHANNEL__对应 PWM 的输出通道，__COMPARE__则是要写到捕获/比较寄存器（TIMx_ CCR1/2/3/4）的值。实际上该宏定义最终还是往对应的捕获/比较寄存器写入比较值来控制 PWM 波的占空比。如下解析：

比如我们要修改定时器 3 通道 4 的输出比较值（控制占空比），寄存器操作方法：

```
TIM3->CCR4 = ledrpwmval; /* ledrpwmval 是比较值，并且动态变化的，
                            所以我们要周期性调用这条语句，已达到及时修改 PWM 的占空比 */
```

__HAL_TIM_SET_COMPARE 这个宏定义函数最终也是调用这个寄存器操作的，所以说我们使用 HAL 库的函数其实就是间接操作寄存器的。

20.3.3.2 程序流程图

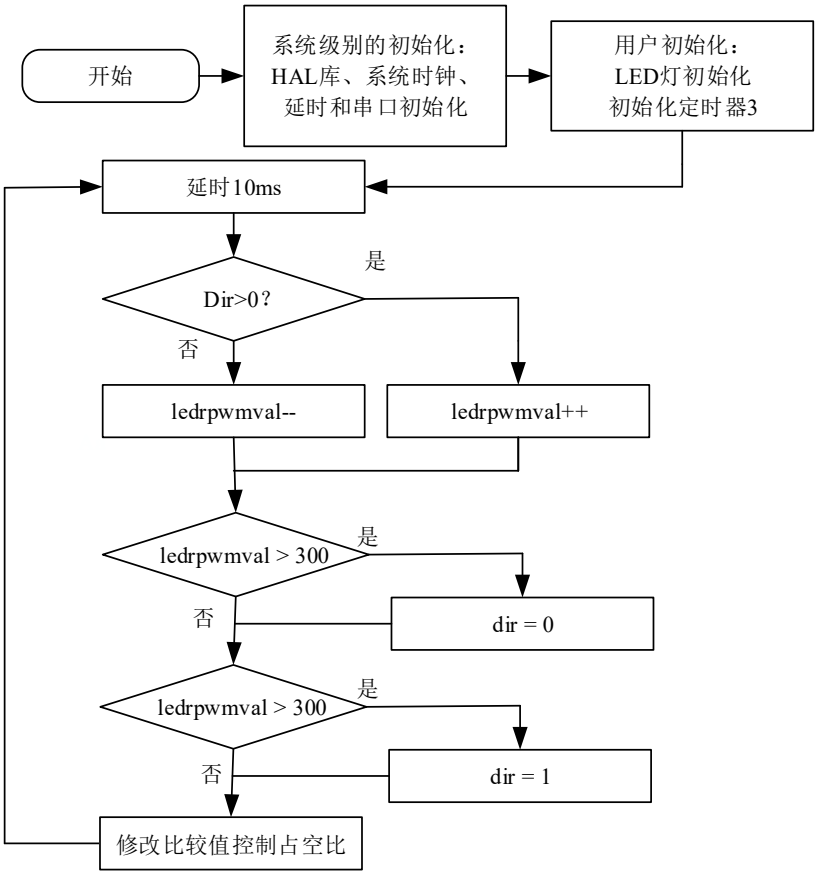


图 20.3.3.2.1 通用定时器 PWM 输出实验程序流程图

20.3.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考本实验对应源码。通用定时器驱动源码包括两个文件：gtim.c 和 gtim.h。

首先看 gtim.h 头文件的几个宏定义：

```
/* TIMX PWM 输出定义
 * 这里输出的 PWM 控制 LED0(RED)的亮度
 * 默认是针对 TIM2~TIM5, TIM12~TIM17.
 */
#define GTIM_TIMX_PWM_CHY_GPIO_PORT      GPIOB
#define GTIM_TIMX_PWM_CHY_GPIO_PIN       GPIO_PIN_1
#define GTIM_TIMX_PWM_CHY_GPIO_AF        GPIO_AF2_TIM3 /* AF 功能选择 */
#define GTIM_TIMX_PWM_CHY_GPIO_CLK_ENABLE()
        do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)    /* PB 口时钟使能 */

#define GTIM_TIMX_PWM                    TIM3
#define GTIM_TIMX_PWM_CHY                TIM_CHANNEL_4 /* 通道 Y,  1<= Y <=4 */
#define GTIM_TIMX_PWM_CHY_CLK_ENABLE()
        do{ __HAL_RCC_TIM3_CLK_ENABLE(); }while(0)      /* TIM3 时钟使能 */
```

可以把上面的宏定义分成两部分，第一部分是定时器 3 输出通道 4 对应的 IO 口的宏定义。第二部分则是定时器 3 输出通道 4 的相应宏定义，这里的宏定义是定时器 3 通道 4 输出 PWM 控制 LED0 的相关宏定义。

gtim.h 头文件就添加了这部分的程序，下面看 gtim.c 的程序，首先是通用定时器 PWM 输出初始化函数。

```
/**
 * @brief    通用定时器 TIMX 通道 Y PWM 输出 初始化函数（使用 PWM 模式 1）
 * @note
 *           通用定时器的时钟来自 APB1,当 D2PPRE1≥2 分频的时候
 *           通用定时器的时钟为 APB1 时钟的 2 倍, 而 APB1 为 100M, 所以定时器时钟 = 200Mhz
 *           定时器溢出时间计算方法: Tout = ((arr + 1) * (psc + 1)) / Ft us.
 *           Ft=定时器工作频率,单位:Mhz
 *
 * @param    arr: 自动重装值。
 * @param    psc: 时钟预分频数
 * @retval   无
 */
void gtim_timx_pwm_chy_init(uint16_t arr,uint16_t psc)
{
    TIM_OC_InitTypeDef timx_oc_pwm_chy = {0};                   /* 定时器输出句柄 */
    g_timx_pwm_chy_handle.Instance = GTIM_TIMX_PWM;             /* 定时器 x */
    g_timx_pwm_chy_handle.Init.Prescaler = psc;                 /* 定时器分频 */
    g_timx_pwm_chy_handle.Init.CounterMode=TIM_COUNTERMODE_UP;/* 向上计数模式 */
    g_timx_pwm_chy_handle.Init.Period = arr;                     /* 自动重装载值 */
    HAL_TIM_PWM_Init(&g_timx_pwm_chy_handle);                    /* 初始化 PWM */

    g_timx_oc_pwm_chy_handle.OCMode = TIM_OCMODE_PWM1;          /* 模式选择 PWM1 */
    /* 设置比较值,此值用来确定占空比，这里默认比较值为自动重装载值的一半,即占空比为 50% */
    g_timx_oc_pwm_chy_handle.Pulse = arr/2;
    g_timx_oc_pwm_chy_handle.OCPolarity=TIM_OCPOLARITY_LOW;/* 输出比较极性为低 */
    HAL_TIM_PWM_ConfigChannel(&g_timx_pwm_chy_handle,&g_timx_oc_pwm_chy_handle,
                              GTIM_TIMX_PWM_CHY);/* 配置 TIMx 通道 y */
    HAL_TIM_PWM_Start(&g_timx_pwm_chy_handle,GTIM_TIMX_PWM_CHY);
    /* 开启 PWM 通道 1 */
}
```

HAL_TIM_PWM_Init 初始化 TIM3 并设置 TIM3 的 ARR 和 PSC 等参数，其次通过调用函数 HAL_TIM_PWM_ConfigChannel 设置 TIM3_CH4 的 PWM 模式以及比较值等参数，最后通过调用函数 HAL_TIM_PWM_Start 来使能 TIM3 以及使能 PWM 通道 TIM3_CH4 输出。

本实验我们使用 PWM 的 MSP 初始化回调函数 HAL_TIM_PWM_MspInit 来存放时钟、

GPIO 的初始化代码，其定义如下：

```
/**
 * @brief       定时器底层驱动，时钟使能，引脚配置
                此函数会被 HAL_TIM_PWM_Init()调用
 * @param       htim:定时器句柄
 * @retval      无
 */
void HAL_TIM_PWM_MspInit(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == GTIM_TIMX_INT)
    {
        GPIO_InitTypeDef gpio_init_struct;
        GTIM_TIMX_PWM_CHY_GPIO_CLK_ENABLE();           /* 开启通道 y 的 GPIO 时钟 */
        GTIM_TIMX_PWM_CHY_CLK_ENABLE();

        gpio_init_struct.Pin = GTIM_TIMX_PWM_CHY_GPIO_PIN;  /* 通道 y 的 GPIO 口 */
        gpio_init_struct.Mode = GPIO_MODE_AF_PP;            /* 复用推完输出 */
        gpio_init_struct.Pull = GPIO_PULLUP;                /* 上拉 */
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;      /* 高速 */
        /* 定时器 x 通道 y 的 GPIO 口复用 */
        gpio_init_struct.Alternate = GTIM_TIMX_PWM_CHY_GPIO_AF;
        HAL_GPIO_Init(GTIM_TIMX_PWM_CHY_GPIO_PORT, &gpio_init_struct);
    }
}
```

该函数首先判断定时器寄存器基地址，符合条件后，开启对应的 GPIO 时钟和定时器时钟，并且初始化 GPIO。上面是使用 HAL 库标准的做法，我们亦可把 HAL_TIM_PWM_MspInit 函数里面的代码直接放到 gtim_timx_pwm_chy_init 函数里。这样做的好处是当一个项目中用到多个定时器时，代码的移植性、可读性好，方便管理。

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint16_t ledrpwmval = 0;
    uint8_t dir = 1;
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    led_init();                             /* 初始化 LED */
    /* 200M/200=1M 的计数频率，自动重装载为 500，那么 PWM 频率为 1M/500=2kHZ */
    gtim_timx_pwm_chy_init(500 - 1, 200 - 1);
    while (1)
    {
        delay_ms(10);

        if (dir)ledrpwmval++;               /* dir==1 ledrpwmval 递增 */
        else ledrpwmval--;                  /* dir==0 ledrpwmval 递减 */

        if (ledrpwmval > 300)dir = 0;       /* ledrpwmval 到达 300 后，方向为递减 */
        if (ledrpwmval == 0)dir = 1;        /* ledrpwmval 递减到 0 后，方向改为递增 */

        /* 修改比较值控制占空比 */
        __HAL_TIM_SET_COMPARE(&g_timx_pwm_chy_handle, GTIM_TIMX_PWM_CHY,
                              ledrpwmval);
    }
}
```

本小节开头我们就说 PWM 频率由自动重载寄存器（TIMx_ARR）的值决定，其占空比则由捕获/比较寄存器（TIMx_CCRx）的值决定。下面结合实际看看具体怎么计算：

定时器 3 的时钟源频率为 2 倍 APB1 总线时钟频率，即频率为 200MHz，而调用 gtim_timx_pwm_chy_init 初始化函数之后，就相当于写入预分频寄存器的值为 199，写入自动

重载寄存器的值为 499。基本定时器讲的定时器溢出公式由公式得：

Tout= ((arr+1)*(psc+1))/Tclk= ((499+1)*(199+1))/200000000=0.0005s

再由频率是周期的倒数关系得到 PWM 的频率为 2000Hz。

占空比怎么计算的呢？结合图 20.3.1，我们分两种情况分析，输出比较极性为低和输出比较极性为高，它们的情况正好相反。因为在 main 函数中的比较值是动态变化的，不利于我们计算占空比，我们假设比较值固定为 200，在本实验中可以调用如下语句得到。

```
__HAL_TIM_SET_COMPARE(&g_timx_pwm_chy_handle, GTIM_TIMX_PWM_CHY, 200);
```

因为 LED0 是低电平有效，所以我们在 gtim_timx_pwm_chy_init 函数中设置了输出比较极性为低，那么当比较值固定为 200 时，占空比 = (arr+1) – CCR4 / (arr+1) = 500-200/500=60%。其中 arr 是写入自动重载寄存器（TIMx_ARR）的值，CCR4 就是写入捕获/比较寄存器 4（TIMx_CCR4）的值。这里我们还需要提醒一下，占空比是指在一个周期内，高电平时间相对于总时间所占的比例。

另外一种情况：设置了输出比较极性为高，那么当比较值固定为 200 时，占空比 = CCR4 / (arr+1) = 200/500=40%。可以看到输出比较极性为低和输出比较极性为高的占空比正好反过来。

在这里，我们使用 DS100 示波器进行验证，效果图如图 20.3.3.3.1 所示：

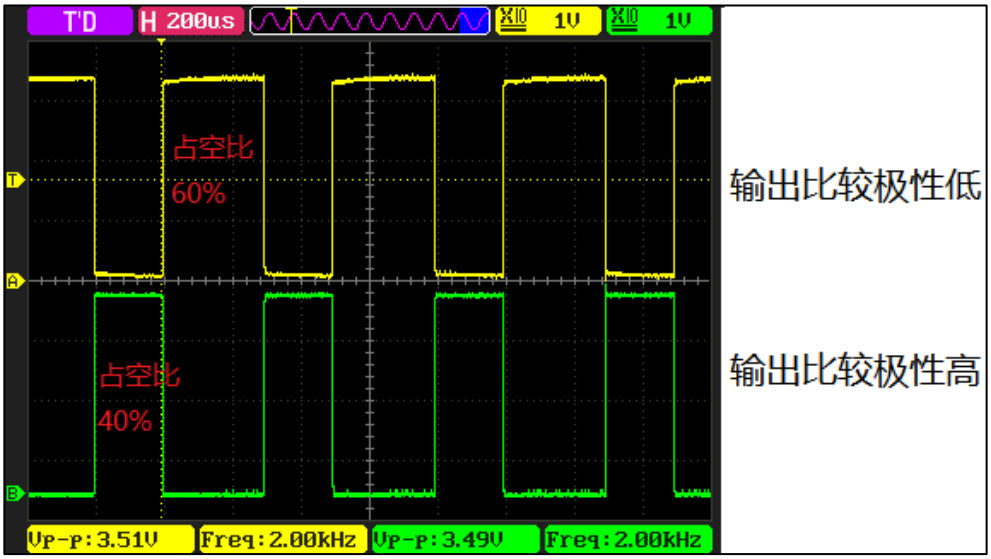


图 20.3.3.3.1 验证效果图

这里把输出比较极性低和输出比较极性高的 PWM 波形都显示出来了。本实验默认设置 PWM 模式 1、输出比较极性低，当 CCR1 寄存器的值设置为 200 时，对应的 PWM 波形如上图黄色的波形图。如果把输出比较极性设置为高，对应的波形图就是绿色的波形图了。

大家感兴趣也可以自行用示波器进行验证。

20.3.4 下载验证

下载代码后，定时器 3 通道 4 输出 PWM 信号到 PB1 口。可以看到 LED0 不停的由暗变到亮，然后又从亮变到暗。

20.4 通用定时器输入捕获实验

本小节我们来学习使用通用定时器的输入捕获模式。

输入捕获模式可以用来测量脉冲宽度或者测量频率。我们以测量脉宽为例，用一个简图来说明输入捕获脉宽测量原理，如图 20.4.1 所示：

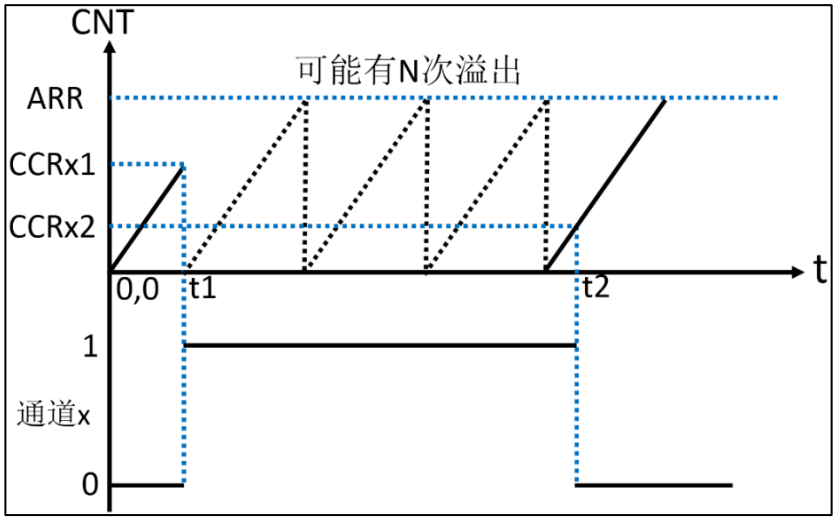


图 20.4.1 输入捕获脉宽测量原理

图 20.4.1 中，t1 到 t2 的时间段，就是我们需要测量的高电平时间。测量方法如下：假如定时器工作在递增计数模式，首先设置定时器通道 x 为上升沿捕获，这样在 t1 时刻上升沿到来时，就会发生捕获事件。这里我们还会打开捕获中断，所以捕获事件发生就意味着捕获中断也会发生。在捕获中断里将计数器值清零，并设置通道 x 为下降沿捕获，这样 t2 时刻下降沿到来时，就会发生捕获事件和捕获中断。捕获事件发生时，计数器的值会被锁存到捕获/比较寄存器中（比如通道 1 对应的是 CCR1 寄存器）。那么在捕获中断里，我们读取捕获/比较寄存器就可以获取到高电平脉冲时间内，计数器计数的个数，从而可以算出高电平脉冲的时间。这里是假设定时器没有溢出为前提的。

实际上，t1 到 t2 时间段，定时器可能会产生 N 次溢出，这就需要我们对定时器溢出做相应的处理，防止高电平太长，导致测量出错。在 t1 到 t2 时间段，假设定时器溢出 N 次，那么高电平脉冲时间内，计数器计数的个数计算方法为：N*（ARR+1）+ CCRx2，CCRx2 表示 t2 时间点，捕获/比较寄存器的值。经过计算得到高电平脉宽时间内计数器计数个数后，用这个个数乘以计数器的计数周期，就可得到高电平持续的时间。就是输入捕获测量高电平脉宽时间的整个过程。

STM32H743 的定时器除了 TIM6 和 TIM7，其他定时器都有输入捕获功能。输入捕获，简单的说就是通过检测 TIMx_Chy 输入的信号边沿，在信号边沿发生跳变（比如上升沿/下降沿）时，会发生捕获事件，将当前定时器的值（TIMx_CNT）锁存到对应通道的捕获/比较寄存器（TIMx_CCRy）里，完成一次捕获。同时还可以配置捕获事件发生时是否触发捕获中断/DMA。另外还要考虑测量的过程中是否可能发生定时器溢出，如果可能溢出，还要做溢出处理。

20.4.1 TIM2/TIM3/TIM4/TIM5 寄存器

通用定时器输入捕获实验需要用到的寄存器有：TIMx_ARR、TIMx_PSC、TIMx_CCMR1、TIMx_CCER、TIMx_DIER、TIMx_CR1、TIMx_CCR1 这些寄存器在前面的章节都有提到，在这里只需针对性的介绍。

- **捕获/比较模式寄存器 1/2（TIMx_CCMR1/2）**

该寄存器我们在 PWM 输出实验时讲解了作为输出功能的配置，现在重点学习输入捕获模式的配置。因为本实验我们用到定时器 5 通道 1 输入，所以我们要看 TIMx_CCMR1 寄存器，其描述如图 20.4.1.1 所示：

这些通道可用于输入（捕获模式）或输出（比较模式）模式。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。对于任一给定位，OCxx 用于说明通道配置为输出时该位对应的功能，ICxx 则用于说明通道配置为输入时该位对应的功能。因此，必须注意同一个位在输入阶段和输出阶段具有不同的含义。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC2M[3]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC1M[3]
							Res.								Res.
							rw								rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2CE	OC2M[2:0]			OC2PE	OC2FE	CC2S[1:0]		OC1CE	OC1M[2:0]			OC1PE	OC1FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]				IC1F[3:0]				IC1PSC[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 7:4 **IC1F**：输入捕获 1 滤波器 (Input capture 1 filter)

此位域可定义 TI1 输入的采样频率和适用于 TI1 的数字滤波器带宽。数字滤波器由事件计数器组成，每 N 个连续事件才视为一个有效输出边沿：

0000：无滤波器，按 f_{DTS} 频率进行采样
0001：$f_{SAMPLING}=f_{CK_INT}$，N=2
0010：$f_{SAMPLING}=f_{CK_INT}$，N=4
0011：$f_{SAMPLING}=f_{CK_INT}$，N=8
0100：$f_{SAMPLING}=f_{DTS}/2$，N=6
0101：$f_{SAMPLING}=f_{DTS}/2$，N=8
0110：$f_{SAMPLING}=f_{DTS}/4$，N=6
0111：$f_{SAMPLING}=f_{DTS}/4$，N=8
1000：$f_{SAMPLING}=f_{DTS}/8$，N=6
1001：$f_{SAMPLING}=f_{DTS}/8$，N=8
1010：$f_{SAMPLING}=f_{DTS}/16$，N=5
1011：$f_{SAMPLING}=f_{DTS}/16$，N=6
1100：$f_{SAMPLING}=f_{DTS}/16$，N=8
1101：$f_{SAMPLING}=f_{DTS}/32$，N=5
1110：$f_{SAMPLING}=f_{DTS}/32$，N=6
1111：$f_{SAMPLING}=f_{DTS}/32$，N=8

位 3:2 **IC1PSC**：输入捕获 1 预分频器 (Input capture 1 prescaler)

此位域定义 CC1 输入 (IC1) 的预分频比。只要 CC1E=0（TIMx_CCER 寄存器），预分频器便立即复位。

00：无预分频器，捕获输入上每检测到一个边沿便执行捕获
01：每发生 2 个事件便执行一次捕获
10：每发生 4 个事件便执行一次捕获
11：每发生 8 个事件便执行一次捕获

位 1:0 **CC1S**：捕获/比较 1 选择 (Capture/Compare 1 Selection)

此位域定义通道方向（输入/输出）以及所使用的输入。

00：CC1 通道配置为输出
01：CC1 通道配置为输入，IC1 映射到 TI1 上
10：CC1 通道配置为输入，IC1 映射到 TI2 上
11：CC1 通道配置为输入，IC1 映射到 TRC 上。此模式仅在通过 TS 位（TIMx_SMCR 寄存器）选择内部触发输入时有效

注：　仅当通道关闭时（TIMx_CCER 中的 CC1E = 0），才可向 CC1S 位写入数据。

图 20.4.1.1 TIMx_CCMR1 寄存器

该寄存器在输入模式和输出模式下，功能是不一样，所以需要看准不同模式的描述，请打开手册查看。TIMx_CCMR1 寄存器对应于通道 1 和通道 2 的设置，CCMR2 寄存器对应通道 3 和通道 4。如：TIMx_CCMR1 寄存器位[7：0]和位 16 用于捕获/比较通道 1 的控制，而位[15：8]和位 24 则用于捕获/比较通道 2 的控制。我们用到定时器 5 通道 1 输入，所以需要配置 TIMx_CCMR1 的位[7:0]和位 16。

其中 CC1S[1:0]，这两个位用于 CCR1 的通道配置，这里我们设置 IC1S[1:0]=01，也就是配置 IC1 映射在 TI1 上。

输入捕获 1 预分频器 IC1PSC[1:0]，这个比较好理解。我们是 1 次边沿就触发 1 次捕获，所以选择 00 就行了。

输入捕获 1 滤波器 IC1F[3:0]，这个用来设置输入采样频率和数字滤波器长度。其中，f_{CK_INT} 是定时器时钟源频率，按照例程的配置为 200Mhz，而 f_{DTS} 则是根据 TIMx_CR1 的 CKD[1:0]的设置来确定的，如果 CKD[1:0]设置为 00，那么 $f_{DTS}=f_{CK_INT}$。N 值采样次数，举个简单的例子：假设 IC1F[3:0]=0011，并设置 IC1 映射到 TI1 上。表示以 f_{CK_INT} 为采样频率，当连续 8 次都是采样到 TI1 为高电平或者低电平，滤波器才输出一个有效输出边沿。当 8 次采样中有高有低，那就保持原来的输出，这样可以滤除高频干扰信号，从而达到滤波的效果。这里，

我们不做滤波处理，所以设置 IC1F[3:0]=0000。

- **捕获/比较使能寄存器（TIMx_ CCER）**

TIM2/TIM3/TIM4/TIM5 的捕获/比较使能寄存器，该寄存器控制着各个输入输出通道的开关和极性。TIMx_CCER 寄存器描述如图 20.4.1.2 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CC4NP	Res.	CC4P	CC4E	CC3NP	Res.	CC3P	CC3E	CC2NP	Res.	CC2P	CC2E	CC1NP	Res.	CC1P	CC1E
rw		rw	rw	rw		rw	rw	rw		rw	rw	rw		rw	rw

位 1 **CC1P**：*捕获/比较 1 输出极性 (Capture/Compare 1 output polarity)*。

CC1 通道配置为输出：

0：OC1 高电平有效

1：OC1 低电平有效

CC1 通道配置为输入：CC1NP/CC1P 位可针对触发或捕获操作选择 TI1FP1 和 TI2FP1 的极性。

00：未反相/上升沿触发

电路对 TIxFP1 上升沿敏感（在复位模式、外部时钟模式或触发模式下执行捕获或触发操作），TIxFP1 未反相（在门控模式或编码器模式下执行触发操作）。

01：反相/下降沿触发

电路对 TIxFP1 下降沿敏感（在复位模式、外部时钟模式或触发模式下执行捕获或触发操作），TIxFP1 反相（在门控模式或编码器模式下执行触发操作）。

10：保留，不使用此配置。

11：未反相/上升沿和下降沿均触发

电路对 TIxFP1 上升沿和下降沿都敏感（在复位模式、外部时钟模式或触发模式下执行捕获或触发操作），TIxFP1 未反相（在门控模式下执行触发操作）。编码器模式下不得使用此配置。

位 0 **CC1E**：*捕获/比较 1 输出使能 (Capture/Compare 1 output enable)*。

CC1 通道配置为输出：

0：关闭——OC1 未激活

1：开启——在相应输出引脚上输出 OC1 信号

CC1 通道配置为输入：此位决定了是否可以实际将计数器值捕获到输入捕获/比较寄存器 1 (TIMx_CCR1) 中。

0：禁止捕获

1：使能捕获

图 20.4.1.2 TIMx_CCER 寄存器

我们要用到这个寄存器的最低 2 位，CC1E 和 CC1P 位。要使能输入捕获，必须设置 CC1E=1，而 CC1P 则根据自己的需要来配置。我们这里是保留默认设置值 0，即高电平触发捕获。

接下来我们再看看 DMA/中断使能寄存器：TIMx_DIER，该寄存器的各位描述见图 20.2.1.3（在 20.2.1 小节）。本小节，我们需要用到中断来处理捕获数据，所以必须开启通道 1 的捕获比较中断，即 CC1IE 设置为 1。同时我们还需要在定时器溢出中断中累计定时器溢出的次数，所以还需要使能定时器的更新中断，即 UIE 置 1。

控制寄存器：TIMx_CR1，我们只用到了它的最低位，也就是用来使能定时器的。

最后再来看看捕获/比较寄存器 1：TIMx_CCR1，该寄存器用来存储发生捕获事件时，TIMx_CNT 的值，我们从 TIMx_CCR1 就可以读出通道 1 捕获事件发生时刻的 TIMx_CNT 值，通过两次捕获（一次上升沿捕获，一次下降沿捕获）的差值，就可以计算出高电平脉冲的宽度（注意，对于高电平脉宽太长的情况，还要计算定时器溢出的次数）。

20.4.2 硬件设计

1. 例程功能

1、使用 TIM5_CH1 来做输入捕获，捕获 PA0 上的高电平脉宽，并将脉宽时间通过串口打印出来，通过按 WK_UP 按键，模拟输入高电平，例程中能测试的最长高电平脉宽时间为：4194303 us。

2、LED0 闪烁指示程序运行。

2. 硬件资源

1）LED 灯
LED : LED0 - PB1
2） 独立按键：WK_UP - PA0
3）定时器 5，使用 TIM5 通道 1，将 PA0 复用为 TIM5_CH1。

3. 原理图

定时器属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们借助 WK_UP 做输入脉冲源并通过串口上位机来监测定时器输入捕获的情况。

20.4.3 程序设计

20.4.3.1 定时器的 HAL 库驱动

定时器在 HAL 库中的驱动代码在前面已经介绍了部分，这里我们再介绍几个本实验用到的函数。

1. HAL_TIM_IC_Init 函数

定时器的输入捕获模式初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_TIM_IC_Init(TIM_HandleTypeDef *htim);
```

- **函数描述：**
  用于初始化定时器的输入捕获模式。
- **函数形参：**
  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量，介绍基本定时器的时候已经介绍。
- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。
- **注意事项：**
  与 PWM 输出实验一样，当使用定时器做输入捕获功能时，在 HAL 库中并不使用定时器初始化函数 HAL_TIM_Base_Init 来实现，而是使用输入捕获特定的定时器初始化函数 HAL_TIM_IC_Init。该函数内部还会调用输入捕获初始化回调函数 HAL_TIM_IC_MspInit 来初始化输入通道对应的 GPIO（复用），以及输入捕获相关的配置。

2. HAL_TIM_IC_ConfigChannel 函数

定时器的输入捕获通道设置初始化函数。其声明如下：

```
HAL_StatusTypeDef HAL_TIM_IC_ConfigChannel(TIM_HandleTypeDef *htim,
                                           TIM_IC_InitTypeDef *sConfig, uint32_t Channel);
```

- **函数描述：**
  该函数用于设置定时器的输入捕获通道。
- **函数形参：**
  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量，用于配置定时器基本参数。
  形参 2 是 TIM_IC_InitTypeDef 结构体类型指针变量，用于配置定时器的输入捕获参数。
  重点了解一下 TIM_IC_InitTypeDef 结构体指针类型，其定义如下：

```
typedef struct
{
  uint32_t ICPolarity;      /* 输入捕获触发方式选择，比如上升、下降和双边沿捕获 */
  uint32_t ICSelection;     /* 输入捕获选择，用于设置映射关系 */
  uint32_t ICPrescaler;     /* 输入捕获分频系数 */
  uint32_t ICFilter;        /* 输入捕获滤波器设置 */
} TIM_IC_InitTypeDef;
```

该结构体成员我们现在介绍一下。成员变量 ICPolarity 用来设置输入信号的有效捕获极性，取值范围为：TIM_ICPOLARITY_RISING(上升沿捕获)，TIM_ICPOLARITY_FALLING（下降沿捕获）和 TIM_ICPOLARITY_BOTHEDGE（双边沿捕获）。成员变量 ICSelection 用来设置映射关系，我们配置 IC1 直接映射在 TI1 上，选择 TIM_ICSELECTION_DIRECTTI（另外还有两

个输入通道 TIM_ICSELECTION_INDIRECTTI 和 TIM_ICSELECTION_TRC）。成员变量 ICPrescaler 用来设置输入捕获分频系数，可以设置为 TIM_ICPSC_DIV1（不分频），TIM_ICPSC_DIV2（2 分频），TIM_ICPSC_DIV4（4 分频）以及 TIM_ICPSC_DIV8（8 分频），本实验需要设置为不分频，所以选值为 TIM_ICPSC_DIV1。成员变量 ICFilter 用来设置滤波器长度，这里我们不使用滤波器，所以设置为 0。

形参 3 是定时器通道，范围：TIM_CHANNEL_1 到 TIM_CHANNEL_6，比如定时器 5 只有 4 个通道，那选择范围只有 TIM_CHANNEL_1 到 TIM_CHANNEL_4，就具体情况选择。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

3. HAL_TIM_IC_Start_IT 函数

启动定时器输入捕获模式函数，其声明如下：

```
HAL_StatusTypeDef HAL_TIM_IC_Start_IT(TIM_HandleTypeDef *htim,
                                      uint32_t Channel);
```

- **函数描述：**

  用于启动定时器的输入捕获模式，且开启输入捕获中断。

- **函数形参：**

  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量。

  形参 2 是定时器通道，范围：TIM_CHANNEL_1 到 TIM_CHANNEL_6。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

- **注意事项：**

  如果我们不需要开启输入捕获中断，只是开启输入捕获功能，HAL 库函数为：

```
HAL_StatusTypeDef HAL_TIM_IC_Start(TIM_HandleTypeDef *htim, uint32_t Channel);
```

定时器输入捕获模式配置步骤

1）开启 TIMx 和相应输入通道的 GPIO 时钟，配置该 IO 口的复用功能输入

首先开启 TIMx 的时钟，然后配置 GPIO 为复用功能输出。本实验我们默认用到定时器 5 通道 1，对应 IO 是 PA0，它们的时钟开启方法如下：

```
__HAL_RCC_TIM5_CLK_ENABLE();                /* 使能定时器 5 */
__HAL_RCC_GPIOA_CLK_ENABLE();               /* 开启 GPIOA 时钟 */
```

IO 口复用功能是通过函数 HAL_GPIO_Init 来配置的。

2）初始化 TIMx，设置 TIMx 的 ARR 和 PSC 等参数

使用定时器的输入捕获功能时，我们调用的是 HAL_TIM_IC_Init 函数来初始化定时器 ARR 和 PSC 等参数。

注意：该函数会调用：HAL_TIM_IC_MspInit 函数，我们可以通过后者存放定时器和 GPIO 时钟使能、GPIO 初始化、中断使能以及优先级设置等代码。

3）设置定时器为输入捕获模式，开启输入捕获

在 HAL 库中，定时器的输入捕获模式是通过 HAL_TIM_IC_ConfigChannel 函数来设置定时器某个通道为输入捕获通道，包括映射关系，输入滤波和输入分频等。

4）使能定时器更新中断，开启捕获功能以及捕获中断，配置定时器中断优先级

通过__HAL_TIM_ENABLE_IT 函数使能定时器更新中断。

通过 HAL_TIM_IC_Start_IT 函数使能定时器并开启捕获功能以及捕获中断。

通过 HAL_NVIC_EnableIRQ 函数使能定时器中断。

通过 HAL_NVIC_SetPriority 函数设置中断优先级。

因为我们要捕获的是高电平信号的脉宽，所以，第一次捕获是上升沿，第二次捕获是下降沿，必须在捕获上升沿之后，设置捕获边沿为下降沿，同时，如果脉宽比较长，那么定时器就会溢出，对溢出必须做处理，否则结果就不准了。

5）编写中断服务函数

定时器中断服务函数为：TIMx_IRQHandler 等，当发生中断的时候，程序就会执行中断服务函数。HAL 库提供了一个定时器中断公共处理函数 HAL_TIM_IRQHandler，该函数会根据

中断类型调用相关的中断回调函数。用户根据自己的需要重定义这些中断回调函数来处理中断程序。本实验我们除了用到更新（溢出）中断回调函数 HAL_TIM_PeriodElapsedCallback 之外，还要用到捕获中断回调函数 HAL_TIM_IC_CaptureCallback。详见本实验例程源码。

20.4.3.2 程序流程图

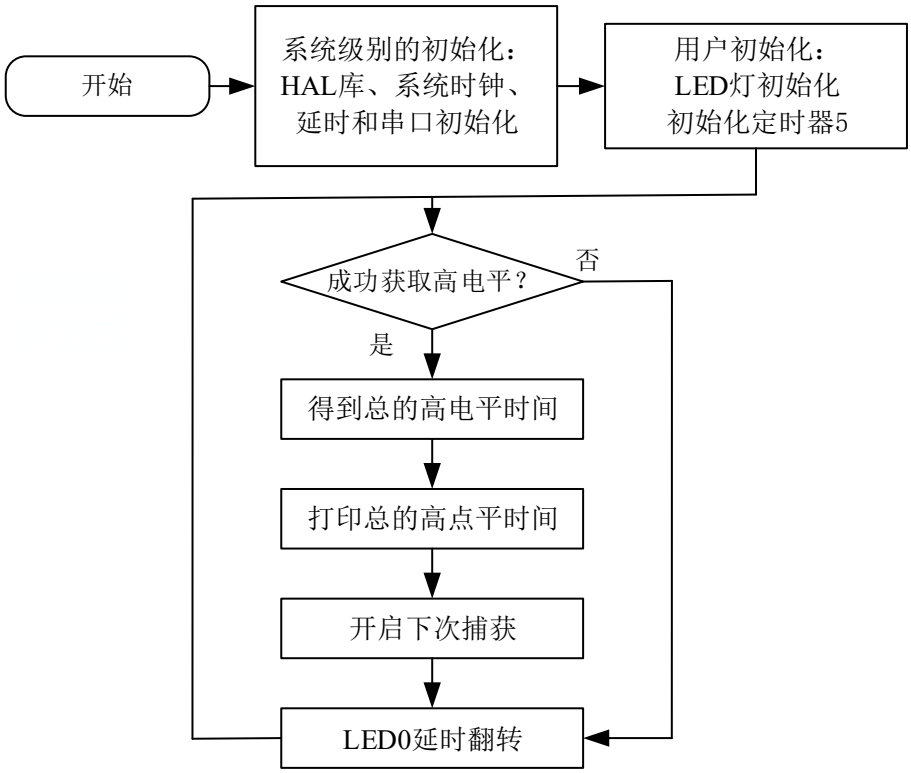


图 20.4.3.2.1 通用定时器输入捕获实验程序流程图

20.4.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。通用定时器驱动源码包括两个文件：gtim.c 和 gtim.h。

首先看 gtim.h 头文件的几个宏定义：

```
/* TIMX 输入捕获定义
 * 这里的输入捕获使用定时器 TIM5_CH1,捕获 WK_UP 按键的输入
 * 注意:默认用的 PA0,设置的是下拉输入!如果改其他 IO,对应的上下拉方式/AF 功能等也得改!
 */
#define GTIM_TIMX_CAP_CHY_GPIO_PORT            GPIOA
#define GTIM_TIMX_CAP_CHY_GPIO_PIN             GPIO_PIN_0
#define GTIM_TIMX_CAP_CHY_GPIO_AF              GPIO_AF2_TIM5   /* AF 功能选择 */
#define GTIM_TIMX_CAP_CHY_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)   /* PA 口时钟使能 */

#define GTIM_TIMX_CAP                       TIM5
#define GTIM_TIMX_CAP_IRQn                  TIM5_IRQn
#define GTIM_TIMX_CAP_IRQHandler            TIM5_IRQHandler
#define GTIM_TIMX_CAP_CHY                   TIM_CHANNEL_1/* 通道 Y,  1<= Y <=4 */
#define GTIM_TIMX_CAP_CHY_CLK_ENABLE()
                do{ __HAL_RCC_TIM5_CLK_ENABLE(); }while(0)    /* TIMX 时钟使能 */
```

可以把上面的宏定义分成两部分，第一部分是定时器 5 输入通道 1 对应的 IO 口的宏定义，第二部分则是定时器 5 输入通道 1 的相应宏定义。

gtim.h 头文件就添加了这部分的程序，下面看 gtim.c 的程序，首先是通用定时器输入捕获初始化函数，其定义如下：

```
/**
 * @brief    通用定时器 TIMX 通道 Y 输入捕获 初始化函数
 * @note
 *           通用定时器的时钟来自 APB1,当 D2PPRE1≥2 分频的时候
 *           通用定时器的时钟为 APB1 时钟的 2 倍, 而 APB1 为 100M, 所以定时器时钟 = 200Mhz
 *           定时器溢出时间计算方法: Tout = ((arr + 1) * (psc + 1)) / Ft us.
 *           Ft=定时器工作频率,单位:Mhz
 *
 * @param    arr: 自动重装值
 * @param    psc: 时钟预分频数
 * @retval   无
 */
void gtim_timx_cap_chy_init(uint16_t arr, uint16_t psc)
{
    TIM_IC_InitTypeDef timx_ic_cap_chy = {0};

    g_timx_cap_chy_handle.Instance = GTIM_TIMX_CAP;                 /* 定时器 5 */
    g_timx_cap_chy_handle.Init.Prescaler = psc;                     /* 定时器分频 */
    g_timx_cap_chy_handle.Init.CounterMode=TIM_COUNTERMODE_UP;/* 递增计数模式 */
    g_timx_cap_chy_handle.Init.Period = arr;                        /* 自动重装载值 */
    HAL_TIM_IC_Init(&g_timx_cap_chy_handle);

    timx_ic_cap_chy.ICPolarity = TIM_ICPOLARITY_RISING;             /* 上升沿捕获 */
    timx_ic_cap_chy.ICSelection = TIM_ICSELECTION_DIRECTTI;     /* 映射到 TI1 上 */
    timx_ic_cap_chy.ICPrescaler = TIM_ICPSC_DIV1;       /* 配置输入分频，不分频 */
    timx_ic_cap_chy.ICFilter = 0;                       /* 配置输入滤波器，不滤波 */
    HAL_TIM_IC_ConfigChannel(&g_timx_cap_chy_handle, &timx_ic_cap_chy,
                             GTIM_TIMX_CAP_CHY);/* 配置 TIM5 通道 1 */

    __HAL_TIM_ENABLE_IT(&g_timx_cap_chy_handle, TIM_IT_UPDATE);/* 使能更新中断 */
    /* 使能通道输入以及使能捕获中断*/
    HAL_TIM_IC_Start_IT(&g_timx_cap_chy_handle, GTIM_TIMX_CAP_CHY);
}
```

HAL_TIM_IC_Init 初始化定时器的基础工作参数，如：ARR 和 PSC 等，第二部分则是调用 HAL_TIM_IC_ConfigChannel 函数配置输入捕获通道映射关系，滤波和分频等。最后是使能更新中断和使能通道输入以及定时器捕获中断。通道对应的 IO、时钟开启和 NVIC 的初始化都在 HAL_TIM_IC_MspInit 函数里编写，其定义如下：

```
/**
 * @brief       通用定时器输入捕获初始化接口
                HAL 库调用的接口，用于配置不同的输入捕获
 * @param       htim:定时器句柄
 * @note        此函数会被 HAL_TIM_IC_Init()调用
 * @retval      无
 */
void HAL_TIM_IC_MspInit(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == GTIM_TIMX_CAP)                    /* 输入通道捕获 */
    {
       GPIO_InitTypeDef gpio_init_struct;
       GTIM_TIMX_CAP_CHY_CLK_ENABLE();                      /* 使能 TIMx 时钟 */
       GTIM_TIMX_CAP_CHY_GPIO_CLK_ENABLE();                 /* 开启捕获 IO 的时钟 */

       gpio_init_struct.Pin = GTIM_TIMX_CAP_CHY_GPIO_PIN;/* 输入捕获的 GPIO 口 */
       gpio_init_struct.Mode = GPIO_MODE_AF_PP;             /* 复用推挽 */
       gpio_init_struct.Pull = GPIO_PULLDOWN;               /* 下拉 */
       gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;/* 高速 */
       gpio_init_struct.Alternate = GTIM_TIMX_CAP_CHY_GPIO_AF; /* 复用 */
       HAL_GPIO_Init(GTIM_TIMX_CAP_CHY_GPIO_PORT, &gpio_init_struct);

       HAL_NVIC_SetPriority(GTIM_TIMX_CAP_IRQn, 1, 3);     /* 抢占 1，子优先级 3 */
```

```
        HAL_NVIC_EnableIRQ(GTIM_TIMX_CAP_IRQn);             /* 开启 ITMx 中断 */
    }
}
```

该函数调用 HAL_GPIO_Init 函数初始化定时器输入通道对应的 IO，并且开启 GPIO 时钟，使能定时器。其中要注意 IO 口复用功能的选择一定要选对了。最后配置中断抢占优先级和响应优先级，以及打开定时器中断。

通过上面的两个函数输入捕获的初始化就完成了，下面先来介绍两个变量。

```
/* 输入捕获状态(g_timxchy_cap_sta)
 * [7]  :0,没有成功的捕获;1,成功捕获到一次.
 * [6]  :0,还没捕获到高电平;1,已经捕获到高电平了.
 * [5:0]:捕获高电平后溢出的次数,最多溢出 63 次,所以最长捕获值= 63*65536+65535 = 4194303
 *       注意:为了通用,我们默认 ARR 和 CCRy 都是 16 位寄存器,对于 32 位的定时器(如:TIM5),
 *       也只按 16 位使用
 *       按 1us 的计数频率,最长溢出时间为:4194303 us, 约 4.19 秒
 *      (说明一下：正常 32 位定时器来说,1us 计数器加 1,溢出时间:4294 秒)
 */
uint8_t g_timxchy_cap_sta = 0;      /* 输入捕获状态 */
uint16_t g_timxchy_cap_val =0 ;     /* 输入捕获值 */
```

这两个变量用于辅助实现高电平捕获。其中 g_timxchy_cap_sta，是用来记录捕获状态，（这个变量，我们把它当成一个寄存器那样来使用）。对其各位赋予状态含义，描述如下表所示：

g_timxchy_cap_sta		
bit7	bit6	bit5~0
捕获完成标志	捕获到高电平标志	捕获高电平后定时器溢出的次数

表 20.4.3.3.1 g_timxchy_cap_sta 各位描述

变量 g_timxchy_cap_sta 的位[5:0]是用于记录捕获高电平定时器溢出次数，总共 6 位，所以最多可以记录溢出的次数为 2 的 6 次方减一次，即 63 次。

变量 g_timxchy_cap_val，则用来记录捕获到下降沿的时候，TIM5_CNT 寄存器的值。

下面开始看中断服务函数的逻辑程序，HAL_TIM_IRQHandler 函数会调用下面两个回调函数，我们的逻辑代码就是放在回调函数里，函数定义如下：

```
/**
 * @brief       定时器输入捕获中断处理回调函数
 * @param       htim:定时器句柄指针
 * @note        该函数在 HAL_TIM_IRQHandler 中会被调用
 * @retval      无
 */
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if ((g_timxchy_cap_sta & 0X80) == 0)      /* 还没成功捕获 */
    {
        if (g_timxchy_cap_sta & 0X40)         /* 捕获到一个下降沿 */
        {
            g_timxchy_cap_sta |= 0X80;        /* 标记成功捕获到一次高电平脉宽 */
            g_timxchy_cap_val =
            HAL_TIM_ReadCapturedValue(&g_timx_cap_chy_handle,
                                      GTIM_TIMX_CAP_CHY);    /* 获取当前的捕获值 */
            TIM_RESET_CAPTUREPOLARITY(&g_timx_cap_chy_handle,
                                      GTIM_TIMX_CAP_CHY);    /* 一定要先清除原来的设置 */
            TIM_SET_CAPTUREPOLARITY(&g_timx_cap_chy_handle, GTIM_TIMX_CAP_CHY,
                                    TIM_ICPOLARITY_RISING);/* 配置 TIM5 通道 1 上升沿捕获 */
        }
        else                                              /* 还未开始,第一次捕获上升沿 */
        {
            g_timxchy_cap_sta = 0;                        /* 清空 */
            g_timxchy_cap_val = 0;
            g_timxchy_cap_sta |= 0X40;                    /* 标记捕获到了上升沿 */
            __HAL_TIM_DISABLE(&g_timx_cap_chy_handle);    /* 关闭定时器 5 */
```

```
            __HAL_TIM_SET_COUNTER(&g_timx_cap_chy_handle,0);/* 计数器清零 */
            TIM_RESET_CAPTUREPOLARITY(&g_timx_cap_chy_handle,
                                      GTIM_TIMX_CAP_CHY); /* 一定要先清除原来的设置！！ */
            TIM_SET_CAPTUREPOLARITY(&g_timx_cap_chy_handle, GTIM_TIMX_CAP_CHY,
                    TIM_ICPOLARITY_FALLING);     /* 定时器 5 通道 1 设置为下降沿捕获 */
            __HAL_TIM_ENABLE(&g_timx_cap_chy_handle);    /* 使能定时器 5 */
        }
    }
}

/**
 * @brief       定时器更新中断回调函数
 * @param       htim:定时器句柄指针
 * @note        此函数会被定时器中断函数共同调用的
 * @retval      无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == GTIM_TIMX_CAP)
    {
        if ((g_timxchy_cap_sta & 0X80) == 0)            /* 还没成功捕获 */
        {
            if (g_timxchy_cap_sta & 0X40)               /* 已经捕获到高电平了 */
            {
                if ((g_timxchy_cap_sta & 0X3F) == 0X3F)/* 高电平太长了 */
                {
                    TIM_RESET_CAPTUREPOLARITY(&g_timx_cap_chy_handle,
                                              GTIM_TIMX_CAP_CHY);  /* 一定要先清除原来的设置 */
                    /* 配置 TIM5 通道 1 上升沿捕获 */
                    TIM_SET_CAPTUREPOLARITY(&g_timx_cap_chy_handle,
                                            GTIM_TIMX_CAP_CHY, TIM_ICPOLARITY_RISING);
                    g_timxchy_cap_sta |= 0X80;          /* 标记成功捕获了一次 */
                    g_timxchy_cap_val = 0XFFFF;
                }
                else                                    /* 累计定时器溢出次数 */
                {
                    g_timxchy_cap_sta++;
                }
            }
        }
    }
}
```

现在我们来介绍一下，捕获高电平脉宽的思路：首先，设置 TIM5_CH1 捕获上升沿，然后等待上升沿中断到来，当捕获到上升沿中断，此时如果 g_timxchy_cap_sta 的第 6 位为 0，则表示还没有捕获到新的上升沿，就先把 g_timxchy_cap_sta、g_timxchy_cap_val 和 TIM5_CNT 寄存器等清零，然后再设置 g_timxchy_cap_sta 的第 6 位为 1，标记捕获到高电平，最后设置为下降沿捕获，等待下降沿到来。如果等待下降沿到来期间，定时器发生了溢出，就用 g_timxchy_cap_sta 变量对溢出次数进行计数，当最大溢出次数来到的时候，就强制标记捕获完成，并配置定时器通道上升沿捕获。当下降沿到来的时候，先设置 g_timxchy_cap_sta 的第 7 位为 1，标记成功捕获一次高电平，然后读取此时的定时器值到 g_timxchy_cap_val 里面，最后设置为上升沿捕获，回到初始状态。

这样，我们就完成一次高电平捕获了，只要 g_timxchy_cap_sta 的第 7 位一直为 1，那么就不会进行第二次捕获，我们在 main 函数处理完捕获数据后，将 g_timxchy_cap_sta 置零，就可以开启第二次捕获。

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint32_t temp = 0;
    uint8_t t = 0;
```

```
    sys_cache_enable();                                 /* 打开 L1-Cache */
    HAL_Init();                                         /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);                 /* 设置时钟, 400Mhz */
    delay_init(400);                                    /* 延时初始化 */
    usart_init(115200);                                 /* 串口初始化为 115200 */
    led_init();                                         /* 初始化 LED */
    key_init();                                         /* 初始化按键 */
    gtim_timx_cap_chy_init(0XFFFF, 200 - 1);            /* 以 1Mhz 的频率计数 捕获 */

    while (1)
    {
        if (g_timxchy_cap_sta & 0X80)                   /* 成功捕获到了一次高电平 */
        {
            temp = g_timxchy_cap_sta & 0X3F;
            temp *= 65536;                              /* 溢出时间总和 */
            temp += g_timxchy_cap_val;                  /* 得到总的高电平时间 */
            printf("HIGH:%d us\r\n", temp);             /* 打印总的高点平时间 */
            g_timxchy_cap_sta = 0;                      /* 开启下一次捕获*/
        }
        t++;
        if (t > 20)                 /* 200ms 进入一次 */
        {
            t = 0;
            LED0_TOGGLE();          /* LED0 闪烁 ,提示程序运行 */
        }
        delay_ms(10);
    }
}
```

先看 gtim_timx_cap_chy_init(0XFFFF, 200 - 1)这个语句，这两个形参分别设置自动重载寄存器的值为 65535，以及预分频器寄存器的值为 199。定时器 5 是 32 位的计数器，为了通用性，我们只使用 16 位，所以计数器设置为 65535。预分频系数，我们设置为 200 分频，定时器 5 的时钟频率是 2 倍的 APB1 总线时钟频率，即 200MHz，可以得到计数器的计数频率是 1MHz，即 1us 计数一次，所以我们的捕获时间精度是 1us。这里可以知道定时器的溢出时间是 65536us。

while（1）无限循环通过判断 g_timxchy_cap_sta 的第 7 位，来获知有没有成功捕获到一次高电平，如果成功捕获，先计算总的高电平时间，再通过串口传输到电脑。

20.4.4 下载验证

下载代码后，可以看到 LED0 在闪烁，说明程序已经正常在跑了，我们再打开串口调试助手，选择对应的串口端口，我这边是 COM15，然后按 KEY_UP 按键，可以看到串口打印的高电平持续时间，如图 20.4.4.1 所示：

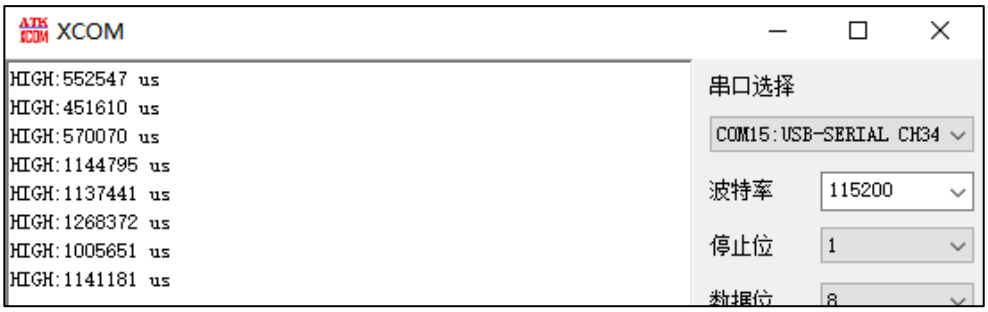


图 20.4.4.1 打印捕获到的高电平时间

20.5 通用定时器脉冲计数实验

前面我们介绍了通用定时器的四类时钟源，本小节我们来学习使用通用定时器的外部时钟模式 1 这类时钟源。

前面的三个通用定时器实验的时钟源都是来自内部时钟 (CK_INT)，本实验我们将使用外部时钟模式 1：外部输入引脚 (TIx)作为定时器的时钟源。关于这个外部输入引脚(TIx)，我们使用 WK_UP 按键按下产生的高电平脉冲作为定时器的计数器时钟，每按下一次按键产生一次高电平脉冲，计数器加一。

下面通过框图给大家展示本实验用到定时器内部哪些资源，如下图所示：

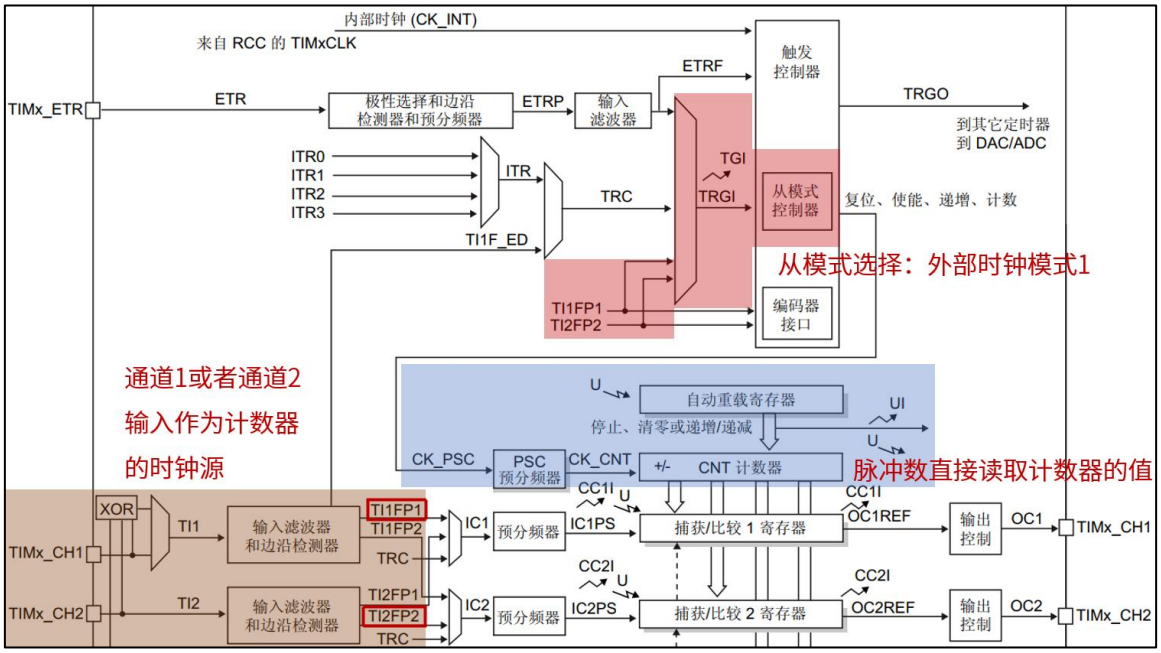


图 20.5.1 脉冲计数实验原理

前面介绍过，外部时钟模式 1 的外部输入引脚只能是通道 1 或者通道 2 对应的 IO，通道 3 或者通道 4 是不可以的。以通道 1 输入为例，外部时钟源信号通过通道 1 输入后，接下来我们用 TI1 表示该信号。TI1 分别要经过滤波器、边沿检测器后，来到 TI1FP1，被触发输入选择器选择为触发源，接着来到从模式控制器。从模式选择为外部时钟模式 1，这时候外部时钟源信号就会到达时基单元的预分频器，后面就是经过分频后就作为计数器的计数时钟了。这个过程的描述，大家亦可参考前面介绍外部时钟模式 1 的描述。因为前面已经介绍过，这里只是简单讲一下。

如果大家想时钟源信号的上升沿和下降沿，计数器都计数，可以选择 TI1F_ED 作为触发输入选择器的触发源。

假设计数器工作在递增计数模式，那么每来一个选择的边沿，计数器就加一。最后，外部时钟源信号的边沿计数个数会保存计数器寄存器中，我们只需要直接读取 CNT 的值即可。这里是没有考虑定时器溢出的情况，如果定时器溢出还需要对溢出进行处理。比如开启更新中断，定时器溢出后，在更新中断里，对溢出次数进行记录，然后用溢出次数乘以溢出一次计数的个数，再加上 CNT 现在的值，就可以得到总的计数个数了。在例程中，我们也是这样处理的。

下面开始讲解本实验用到的寄存器配置情况。

20.5.1 TIM2/TIM3/TIM4/TIM5 寄存器

通用定时器脉冲计数实验需要用到的寄存器有：TIMx_ARR、TIMx_PSC、TIMx_CCMR1、TIMx_CCER、TIMx_DIER、TIMx_CR1、TIMx_EGR 这些寄存器在前面的章节都有提到，在这里只需针对性的介绍。

- **捕获/比较模式寄存器 1/2（TIMx_CCMR1/2）**

  该寄存器我们在 PWM 输出实验时讲解了他作为输出功能的配置，在输入捕获实验学习了

输入捕获模式的配置，本小节我们的外部信号也同样要作为输入信号给定时器作为时钟源，所以我们要看输入捕获模式定时器对应功能。WK_UP 按键（PA0）对应着定时器 2 的通道 1，这个可以在《STM32H743IIT6.pdf》数据手册 68 页找到。接下来我们开始配置 TIMx_CCMR1 寄存器，其描述如图 20.5.1.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC2M[3]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC1M[3]
							Res.								Res.
							rw								rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2CE	OC2M[2:0]			OC2PE	OC2FE	CC2S[1:0]		OC1CE	OC1M[2:0]			OC1PE	OC1FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]				IC1F[3:0]				IC1PSC[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 7:4 **IC1F**：输入捕获 1 滤波器 (Input capture 1 filter)

此位域可定义 TI1 输入的采样频率和适用于 TI1 的数字滤波器带宽。数字滤波器由事件计数器组成，每 N 个连续事件才视为一个有效输出边沿：

0000：无滤波器，按 f_{DTS} 频率进行采样

0001：$f_{SAMPLING}=f_{CK_INT}$，N=2　0110：$f_{SAMPLING}=f_{DTS}/4$，N=6　1011：$f_{SAMPLING}=f_{DTS}/16$，N=6

0010：$f_{SAMPLING}=f_{CK_INT}$，N=4　0111：$f_{SAMPLING}=f_{DTS}/4$，N=8　1100：$f_{SAMPLING}=f_{DTS}/16$，N=8

0011：$f_{SAMPLING}=f_{CK_INT}$，N=8　1000：$f_{SAMPLING}=f_{DTS}/8$，N=6　1101：$f_{SAMPLING}=f_{DTS}/32$，N=5

0100：$f_{SAMPLING}=f_{DTS}/2$，N=6　1001：$f_{SAMPLING}=f_{DTS}/8$，N=8　1110：$f_{SAMPLING}=f_{DTS}/32$，N=6

0101：$f_{SAMPLING}=f_{DTS}/2$，N=8　1010：$f_{SAMPLING}=f_{DTS}/16$，N=5　1111：$f_{SAMPLING}=f_{DTS}/32$，N=8

位 3:2 **IC1PSC**：输入捕获 1 预分频器 (Input capture 1 prescaler)

此位域定义 CC1 输入 (IC1) 的预分频比。只要 CC1E=0（TIMx_CCER 寄存器），预分频器便立即复位。

00：无预分频器，捕获输入上每检测到一个边沿便执行捕获　10：每发生 4 个事件便执行一次捕获

01：每发生 2 个事件便执行一次捕获　11：每发生 8 个事件便执行一次捕获

位 1:0 **CC1S**：捕获/比较 1 选择 (Capture/Compare 1 Selection)

此位域定义通道方向（输入/输出）以及所使用的输入。

00：CC1 通道配置为输出

01：CC1 通道配置为输入，IC1 映射到 TI1 上

10：CC1 通道配置为输入，IC1 映射到 TI2 上

11：CC1 通道配置为输入，IC1 映射到 TRC 上。此模式仅在通过 TS 位（TIMx_SMCR 寄存器）选择内部触发输入时有效

注：　仅当通道关闭时（TIMx_CCER 中的 CC1E = 0），才可向 CC1S 位写入数据。

图 20.5.1.1 TIMx_CCMR1 寄存器

用到定时器 2 的通道 1，所以要配置 TIM2_CCMR1 寄存器的位[7：0]，其中 CC1S[1:0]，这两个位用于 CCR1 的通道配置，这里我们设置 IC1S[1:0]=01，也就是配置 IC1 映射在 TI1 上，即 CCR1 对应 TIMx_CH1。

输入捕获 1 预分频器 IC1PSC[1:0]，我们是 1 次高电平脉冲就触发 1 次计数，所以不用分频选择 00 即可。

输入捕获 1 滤波器 IC1F[3:0]，这个用来设置输入采样频率和数字滤波器长度，关于滤波长度的介绍请看上一个实验。这里，我们不做滤波处理，所以设置 IC1F[3:0]=0000，只要采集到上升沿，就触发捕获。

- **捕获/比较使能寄存器（TIMx_ CCER）**

TIM2/TIM3/TIM4/TIM5 的捕获/比较使能寄存器，该寄存器控制着各个输入输出通道的开关和极性。TIMx_CCER 寄存器描述如图 20.5.1.2 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CC4NP	Res.	CC4P	CC4E	CC3NP	Res.	CC3P	CC3E	CC2NP	Res.	CC2P	CC2E	CC1NP	Res.	CC1P	CC1E
rw		rw	rw	rw		rw	rw	rw		rw	rw	rw		rw	rw

图 20.5.1.2 TIMx_CCER 寄存器

我们要用到这个寄存器的最低 2 位，CC1E 和 CC1P 位。要使能输入捕获，必须设置 CC1E=1，而 CC1P 则根据自己的需要来配置。我们这里是保留默认设置值 0，即高电平触发捕获。

- **从模式控制寄存器（TIMx_ SMCR）**

TIM2/TIM3/TIM4/TIM5 的从模式控制寄存器，该寄存器用于配置从模式，以及定时器的

触发源相关的设置。TIMx_SMCR 寄存器描述如图 20.5.1.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS[4:3]		Res.	Res.	Res.	SMS[3]
										rw	rw				rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETP	ECE	ETPS[1:0]		ETF[3:0]				MSM	TS[2:0]			Res.	SMS[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw

位 6:4 **TS[4:0]**：触发选择 (Trigger selection)（请参见 TS[4:3] 的位 21:20）

此位域可选择将要用于同步计数器的触发输入。

00000：内部触发 0 (ITR0)
00001：内部触发 1 (ITR1)
00010：内部触发 2 (ITR2)
00011：内部触发 3 (ITR3)
00100：TI1 边沿检测器 (TI1F_ED)
00101：滤波后的定时器输入 1 (TI1FP1)
00110：滤波后的定时器输入 2 (TI2FP2)
00111：外部触发输入 (ETRF)
01000：内部触发 4 (ITR4)
01001：内部触发 5 (ITR5)
01010：内部触发 6 (ITR6)
01011：内部触发 7 (ITR7)
01100：内部触发 8 (ITR8)
其它：保留

有关各定时器 ITRx 含义的详细信息，请参见*第 1538 页的表 317：TIMx 内部触发连接。*

注：　这些位只能在未使用的情况下（例如，SMS=000　时）进行更改，以避免转换时出现错误的边沿检测。

位 2:0 **SMS**：从模式选择 (Slave mode selection)

选择外部信号时，触发信号 (TRGI) 的有效边沿与外部输入上所选的极性相关（请参见输入控制寄存器和控制寄存器说明）。

0000：禁止从模式——如果 CEN = “1”，预分频器时钟直接由内部时钟提供。
0001：编码器模式 1——计数器根据 TI2FP2 电平在 TI1FP1 边沿递增/递减计数。
0010：编码器模式 2——计数器根据 TI1FP1 电平在 TI2FP2 边沿递增/递减计数。
0011：编码器模式 3——计数器在 TI1FP1 和 TI2FP2 的边沿计数，计数的方向取决于另外一个输入的电平。
0100：复位模式——在出现所选触发输入 (TRGI) 上升沿时，重新初始化计数器并生成一个寄存器更新事件。
0101：门控模式——触发输入 (TRGI) 为高电平时使能计数器时钟。只要触发输入变为低电平，计数器立即停止计数（但不复位）。计数器的启动和停止都被控制。
0110：触发模式——触发信号 TRGI 出现上升沿时启动计数器（但不复位）。只控制计数器的启动。
0111：外部时钟模式 1——由所选触发信号 (TRGI) 的上升沿提供计数器时钟。
1000：组合复位 + 触发模式——在出现所选触发输入 (TRGI) 上升沿时，重新初始化计数器，生成一个寄存器更新事件并启动计数器。

图 20.5.1.3 TIMx_SMCR 寄存器

因为我们要让外部引脚脉冲信号作为定时器的时钟源，所以位[2:0]和位 16 组合的 SMS[0:3]，我们设置的值是 0111，即外部时钟模式 1。位[6:4] 和位[21:20]组合的 TS[0:5]是触发选择设置，TIMx_CH1 对应 TI1FP1，TIMx_CH2 则对应 TI2FP2，我们是定时器通道 1，所以需要配置的值为 00101。ETF[3:0]和 ETPS[1:0]分别是外部触发滤波器和外部触发预分频器，我们没有用到。

接下来我们再看看 DMA/中断使能寄存器：TIMx_DIER，该寄存器的各位描述见图 20.2.1.3（在 20.2.1 小节）。本实验，我们需要用到定时器更新中断，在中断服务函数中累加定时器溢出的次数，所以需要使能定时器的更新中断，即 UIE 置 1。

控制寄存器 1：TIMx_CR1，我们只用到了它的最低位，也就是用来使能定时器的。

20.5.2 硬件设计

1. 例程功能

使用 TIM2_CH1 做输入捕获，我们将捕获 PA0 上的高电平脉宽，并对脉宽进行计数，通过串口打印出来。大家可以通过按 WK_UP 按键，输入高电平脉冲，通过按 KEY0 重设当前计数。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED：LED0 - PB1
2）独立按键：
KEY0 - PH3
WK_UP - PA0
3）定时器 2，使用 TIM2 通道 1，PA0 复用为 TIM2_CH1。

3. 原理图

定时器属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们借助 WK_UP 做输入脉冲源，捕获 PA0 上的高电平脉宽，然后对脉宽进行计数并通过串口上位机打印出来。还可以通过按 KEY0 重设当前计数。

20.5.3 程序设计

20.5.3.1 定时器的 HAL 库驱动

定时器在 HAL 库中的驱动代码在前面已经介绍了部分，这里我们针对定时器从模式介绍 HAL_TIM_SlaveConfigSynchronization 函数。

1. HAL_TIM_SlaveConfigSynchronization 函数

该函数是 HAL_TIM_SlaveConfigSynchro 函数的宏定义，真正的函数定义是后者，其定义如下：

```
HAL_StatusTypeDef HAL_TIM_SlaveConfigSynchro(TIM_HandleTypeDef *htim,
                                             TIM_SlaveConfigTypeDef *sSlaveConfig);
```

- **函数描述：**
  该函数用于配置定时器的从模式选择、输入触发源等参数。
- **函数形参：**
  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量，用于配置定时器基本参数。
  形参 2 是 TIM_SlaveConfigTypeDef 结构体类型指针变量，用于配置定时器的从模式。
  重点了解一下 TIM_SlaveConfigTypeDef 结构体指针类型，其定义如下：

```
typedef struct
{
  uint32_t SlaveMode;           /* 从模式选择 */
  uint32_t InputTrigger;        /* 输入触发源选择 */
  uint32_t TriggerPolarity;     /* 输入触发极性 */
  uint32_t TriggerPrescaler;    /* 输入触发预分频 */
  uint32_t TriggerFilter;       /* 输入滤波器设置 */
} TIM_SlaveConfigTypeDef;
```

- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

定时器从模式脉冲计数配置步骤

1）开启 TIMx 和输入通道的 GPIO 时钟，配置该 IO 口的复用功能输入

首先开启 TIMx 的时钟，然后配置 GPIO 为复用功能输出。本实验我们默认用到定时器 2 通道 1，对应 IO 是 PA0，它们的时钟开启方法如下：

```
__HAL_RCC_TIM2_CLK_ENABLE();                /* 使能定时器 2 */
__HAL_RCC_GPIOA_CLK_ENABLE();               /* 开启 GPIOA 时钟 */
```

IO 口复用功能是通过函数 HAL_GPIO_Init 来配置的。

2）初始化 TIMx，设置 TIMx 的 ARR 和 PSC 等参数

使用定时器的输入捕获功能时，我们调用的是 HAL_TIM_IC_Init 函数来初始化定时器 ARR 和 PSC 等参数。

注意：该函数会调用：HAL_TIM_IC_MspInit 函数，我们可以通过后者存放定时器和 GPIO 时钟使能、GPIO 初始化、中断使能和优先级设置等代码。

3）选择从模式：外部触发模式 1

TIMx_SMCR 寄存器控制着定时器的从模式，包括模式选择，触发源选择、极性和输入预分频等。HAL 库是通过 HAL_TIM_SlaveConfigSynchro 函数来初始化定时器从模式配置参数的。

4）设置定时器为输入捕获模式，开启输入捕获

在 HAL 库中，定时器的输入捕获模式是通过 HAL_TIM_IC_ConfigChannel 函数来设置定时器某个通道为输入捕获通道，包括映射关系，输入滤波和输入分频等。

5）使能定时器更新中断，开启捕获功能，配置定时器中断优先级

通过__HAL_TIM_ENABLE_IT 函数使能定时器更新中断。

通过 HAL_TIM_IC_Start 函数使能定时器并开启捕获功能。

通过 HAL_NVIC_EnableIRQ 函数使能定时器中断。

通过 HAL_NVIC_SetPriority 函数设置中断优先级。

6）编写中断服务函数

定时器中断服务函数为：TIMx_IRQHandler 等，当发生中断的时候，程序就会执行中断服务函数。HAL 库提供了一个定时器中断公共处理函数 HAL_TIM_IRQHandler，该函数会根据中断类型调用相关的中断回调函数。用户根据自己的需要重定义这些中断回调函数来处理中断程序。本实验我们不使用 HAL 库的中断回调机制，而是把中断程序写在定时器中断服务函数里。详见本实验例程源码。

20.5.3.2 程序流程图

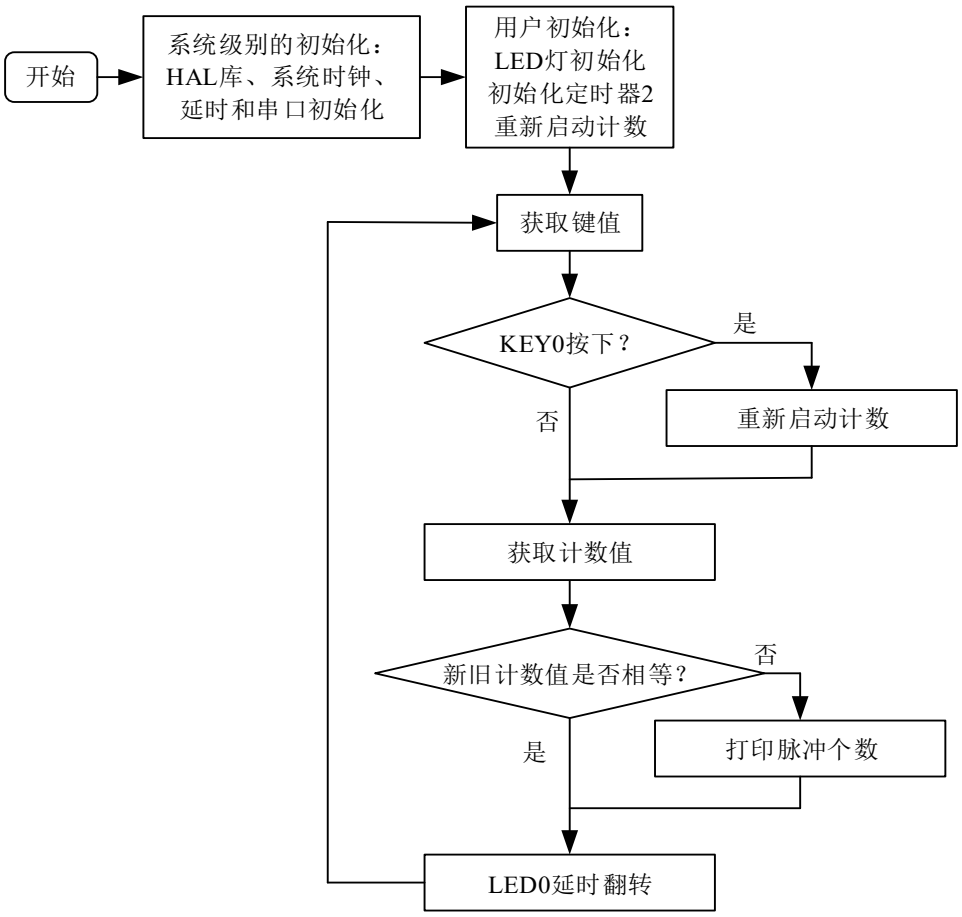


图 20.5.3.2.1 通用定时器脉冲计数实验程序流程图

20.5.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。通用定时器驱动源码包括两个文件：gtim.c 和 gtim.h。

首先看 gtim.h 头文件的几个宏定义：

```
/* TIMX 输入计数定义
 * 这里的输入计数使用定时器 TIM2_CH1,捕获 WK_UP 按键的输入
 * 默认是针对 TIM2~TIM5, TIM12~TIM17.只有 CH1 和 CH2 通道可以用做输入计数,CH3/CH4 不支持!
 * 特别要注意:默认用的 PA0,设置的是下拉输入!如果改其他 IO,对应的上下拉方式/AF 功能等也得改!
 */
#define GTIM_TIMX_CNT_CHY_GPIO_PORT      GPIOA
#define GTIM_TIMX_CNT_CHY_GPIO_PIN       GPIO_PIN_0
#define GTIM_TIMX_CNT_CHY_GPIO_AF        GPIO_AF1_TIM2          /* AF 功能选择 */
#define GTIM_TIMX_CNT_CHY_GPIO_CLK_ENABLE()
               do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)     /* PA 口时钟使能 */

#define GTIM_TIMX_CNT                     TIM2
#define GTIM_TIMX_CNT_IRQn                TIM2_IRQn
#define GTIM_TIMX_CNT_IRQHandler          TIM2_IRQHandler
#define GTIM_TIMX_CNT_CHY                 TIM_CHANNEL_1 /* 通道 Y,  1<= Y <=2 */
#define GTIM_TIMX_CNT_CHY_CLK_ENABLE()
              do{ __HAL_RCC_TIM2_CLK_ENABLE(); }while(0)/* TIMX 时钟使能 */
```

可以把上面的宏定义分成两部分，第一部分是定时器 2 输入通道 1 对应的 IO 口的宏定义，第二部分则是定时器 2 输入通道 1 的相应宏定义。需要注意的点是：只有 CH1 和 CH2 通道可以用做输入计数，CH3/CH4 不支持!

gtim.h 头文件就添加了这部分的程序，下面看 gtim.c 的程序，首先是通用定时器脉冲计数初始化函数，其定义如下：

```
/**
 * @brief     通用定时器 TIMX 通道 Y 脉冲计数 初始化函数
 * @note
 *            本函数选择通用定时器的时钟选择：外部时钟源模式 1(SMS[2:0] = 111)
 *            计数器时钟源就来自 TIMX_CH1/CH2，可以实现外部脉冲计数(脉冲接入 CH1/CH2)
 *
 *            时钟分频数 = psc，一般设置为 0，表示每一个时钟都会计数一次，以提高精度.
 *            通过读取 CNT 和溢出次数，经过简单计算，可以得到当前的计数值，从而实现脉冲计数
 *
 * @param     psc: 预分频系数
 * @retval    无
 */
void gtim_timx_cnt_chy_init(uint16_t psc)
{
    GPIO_InitTypeDef gpio_init_struct;
    TIM_SlaveConfigTypeDef tim_slave_config = {0};
    GTIM_TIMX_CNT_CHY_CLK_ENABLE();                             /* 使能 TIMx 时钟 */
    GTIM_TIMX_CNT_CHY_GPIO_CLK_ENABLE();                        /* 开启 GPIOA 时钟 */

    g_timx_cnt_chy_handle.Instance = GTIM_TIMX_CNT;             /* 定时器 x */
    g_timx_cnt_chy_handle.Init.Prescaler = psc;                 /* 定时器分频 */
    g_timx_cnt_chy_handle.Init.CounterMode=TIM_COUNTERMODE_UP;/* 递增计数模式 */
    g_timx_cnt_chy_handle.Init.Period = 65535;                  /* 自动重装载值 */
    HAL_TIM_IC_Init(&g_timx_cnt_chy_handle);

    gpio_init_struct.Pin = GTIM_TIMX_CNT_CHY_GPIO_PIN;  /* 输入捕获的 GPIO 口 */
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;            /* 复用推挽 */
    gpio_init_struct.Pull = GPIO_PULLDOWN;              /* 下拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH; /* 高速 */
    gpio_init_struct.Alternate = GTIM_TIMX_CNT_CHY_GPIO_AF; /* 复用 */
    HAL_GPIO_Init(GTIM_TIMX_CNT_CHY_GPIO_PORT, &gpio_init_struct);

    /* 从模式：外部触发模式 1 */
    tim_slave_config.SlaveMode = TIM_SLAVEMODE_EXTERNAL1;/* 外部触发模式 1 */
    tim_slave_config.InputTrigger = TIM_TS_TI1FP1;/* TI1FP1 作为触发输入源 */
    tim_slave_config.TriggerPolarity = TIM_TRIGGERPOLARITY_RISING;/* 上升沿 */
    tim_slave_config.TriggerPrescaler = TIM_TRIGGERPRESCALER_DIV1;/* 不分频 */
    tim_slave_config.TriggerFilter = 0x0;           /* 滤波：本例中不需要任何滤波 */
    HAL_TIM_SlaveConfigSynchronization(&g_timx_cnt_chy_handle,
                                       &tim_slave_config);
   /* 设置中断优先级，抢占优先级 1，子优先级 3 */
    HAL_NVIC_SetPriority(GTIM_TIMX_CNT_IRQn, 1, 3);
    HAL_NVIC_EnableIRQ(GTIM_TIMX_CNT_IRQn);             /* 开启 ITMx 中断 */

    __HAL_TIM_ENABLE_IT(&g_timx_cnt_chy_handle, TIM_IT_UPDATE);/* 使能更新中断 */
    HAL_TIM_IC_Start(&g_timx_cnt_chy_handle,GTIM_TIMX_CNT_CHY);/* 使能通道输入 */
}
```

gtim_timx_cnt_chy_init 函数包含了输入通道对应 IO 的初始代码、NVIC、使能时钟、定时器基础工作参数和从模式配置的所有代码。下面来看看该函数的代码内容。

第一部分使能定时器和 GPIO 的时钟。

第二部分调用 HAL_TIM_IC_Init 函数初始化定时器的基础工作参数，如：ARR 和 PSC 等。

第三部分是定时器输入通道对应的 IO 的初始化。

第四部分调用 HAL_TIM_SlaveConfigSynchronization 函数配置从模式选择、输入捕获通道映射关系、捕获边沿和滤波等。

第五部分是 NVIC 的初始化，配置抢占优先级、响应优先级和开启 NVIC 定时器中断。

最后是使能更新中断和使能通道输入。

我们在通用定时器输入捕获实验使用了 HAL_TIM_IC_MspInit 函数，为了方便代码的管理和移植性等，这里就不再使用这个函数了。当一个项目使用到多个定时器时，也同样建议大家不要使用 HAL 库的回调机制，真的不方便。至于前面为什么要用 HAL_TIM_IC_MspInit 函数，只是为了让大家知道 HAL 库回调机制的使用方法。大家可以根据自己的情况权衡利弊，决定是否使用 HAL 库的回调机制。

下面先看中断服务函数，在基本定时器中断实验中，我们知道中断逻辑程序的逻辑代码是放在更新中断回调函数里面的，这是 HAL 库回调机制标准的做法。因为我们在通用定时器输入捕获实验中使用过 HAL_TIM_PeriodElapsedCallback 更新中断回调函数，所以本实验我们不使用 HAL 库这套回调机制，而是直接将中断处理写在定时器中断服务函数中，定时器中断服务函数定义如下：

```
/**
 * @brief       通用定时器 TIMX 脉冲计数 更新中断服务函数
 * @param       无
 * @retval      无
 */
void GTIM_TIMX_CNT_IRQHandler(void)
{
    /* 以下代码没有使用定时器 HAL 库共用处理函数来处理，而是直接通过判断中断标志位的方式 */
    if(__HAL_TIM_GET_FLAG(&g_timx_cnt_chy_handle, TIM_FLAG_UPDATE) != RESET)
    {
        g_timxchy_cnt_ofcnt++;          /* 累计溢出次数 */
    }

    __HAL_TIM_CLEAR_IT(&g_timx_cnt_chy_handle, TIM_IT_UPDATE);
}
```

在函数中，使用__HAL_TIM_GET_FLAG 函数宏获取更新中断标志位，然后判断是否发生更新中断，如果发生了更新中断，表示脉冲计数的个数等于 ARR 寄存器的值，那么我们让 g_timxchy_cnt_ofcnt 变量++，累计定时器溢出次数。最后调用__HAL_TIM_CLEAR_IT 函数宏清除更新中断标志位。这样就完成一次对更新中断的处理。

再来介绍两个自定义的功能函数，第一个是获取当前计数值函数，其定义如下：

```
/**
 * @brief       通用定时器 TIMX 通道 Y 获取当前计数值
 * @param       无
 * @retval      当前计数值
 */
uint32_t gtim_timx_cnt_chy_get_count(void)
{
    uint32_t count = 0;
    count = g_timxchy_cnt_ofcnt * 65536;          /* 计算溢出次数对应的计数值 */
    count += __HAL_TIM_GET_COUNTER(&g_timx_cnt_chy_handle);/* 加上当前 CNT 的值 */
    return count;
}
```

该函数先是计算定时器溢出次数对应的计数个数，因为定时器每溢出一次的计数个数是 65536，所以这里用 g_timxchy_cnt_ofcnt 乘以 65536，就可以得到溢出计数的个数，前提是 ARR 寄存器的值得设置为 65535。然后再加上定时器计数器当前的值，计数器当前的值通过调用__HAL_TIM_GET_COUNTER 函数宏可以获取。函数返回值是脉冲计数的总个数。

第二个自定义功能函数是重启计数器函数，其定义如下：

```
/**
 * @brief       通用定时器 TIMX 通道 Y 重启计数器
 * @param       无
 * @retval      无
 */
void gtim_timx_cnt_chy_restart(void)
{
    __HAL_TIM_DISABLE(&g_timx_cnt_chy_handle);          /* 关闭定时器 TIMX */
    g_timxchy_cnt_ofcnt = 0;                            /* 累加器清零 */
    __HAL_TIM_SET_COUNTER(&g_timx_cnt_chy_handle, 0);   /* 计数器清零 */
```

```
    __HAL_TIM_ENABLE(&g_timx_cnt_chy_handle);           /* 使能定时器 TIMX */
}
```

该函数先关闭定时器，然后做清零操作，包括：记录溢出次数全局变量 g_timxchy_cnt_ofcnt 和定时器计数器的值，最后使能定时器重新计数。

通用定时器脉冲计数实验的整体驱动和逻辑程序还算比较容易理解，下面看看 main.c 里面编写的代码：

```
int main(void)
{
    uint32_t curcnt = 0;
    uint32_t oldcnt = 0;
    uint8_t key = 0;
    uint8_t t = 0;
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    led_init();                             /* 初始化 LED */
    key_init();                             /* 初始化按键 */
    gtim_timx_cnt_chy_init(0);              /* 定时器计数初始化, 不分频 */
    gtim_timx_cnt_chy_restart();            /* 重启计数 */
    while (1)
    {
        key = key_scan(0);                  /* 扫描按键 */
        if (key == KEY0_PRES)               /* KEY0 按键按下,重启计数 */
        {
            gtim_timx_cnt_chy_restart();        /* 重新启动计数 */
        }
        curcnt = gtim_timx_cnt_chy_get_count(); /* 获取计数值 */
        if (oldcnt != curcnt)
        {
            oldcnt = curcnt;
            printf("CNT:%d\r\n", oldcnt);       /* 打印脉冲个数 */
        }
        t++;
        if (t > 20)             /* 200ms 进入一次 */
        {
            t = 0;
            LED0_TOGGLE();      /* LED0 闪烁 ,提示程序运行 */
        }
        delay_ms(10);
    }
}
```

调用定时器初始化函数 gtim_timx_cnt_chy_init(0)，形参是 0，表示设置预分频器寄存器的值为 0，即表示不分频。如果形参设置为 1，就是 2 分频，这种情况，要按按键 WK_UP 两次才会计数一次，大家不妨试试。该函数内部已经设置自动重载寄存器的值为 65535，所以在不分频的情况下，定时器发生一次更新中断，表示脉冲计数了 65536 次。

20.5.4 下载验证

下载代码后，可以看到 LED0 在闪烁，说明程序已经正常在跑了，我们再打开串口调试助手，然后每按 KEY_UP 按键一次，就可以看到串口打印的高电平脉冲次数。如果按 KEY0 按键，就会重设当前计数，从 0 开始计数，如图 20.5.4.1 所示：

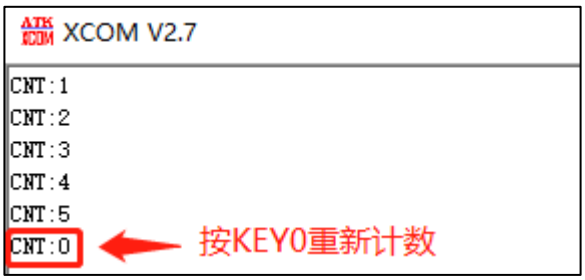


图 20.5.4.1 打印高电平脉冲次数

第二十一章 高级定时器实验

本章我们主要来学习高级定时器，STM32H743 有 2 个高级定时器（TIM1 和 TIM8）。我们将通过四个实验来学习高级定时器的各个功能，分别是高级定时器输出指定个数 PWM 实验、高级定时器输出比较模式实验、高级定时器互补输出带死区控制实验和高级定时器 PWM 输入模式实验。

本章分为如下几个小节：

21.1 高级定时器简介
21.2 高级定时器输出指定个数 PWM 实验
21.3 高级定时器输出比较模式实验
21.4 高级定时器互补输出带死区控制实验
21.5 高级定时器 PWM 输入模式实验

21.1 高级定时器简介

高级定时器的框图和通用定时器框图很类似，只是添加了其它的一些功能，如：重复计数器、带死区控制的互补输出通道、断路输入等。这些功能在高级定时器框图的位置如下：

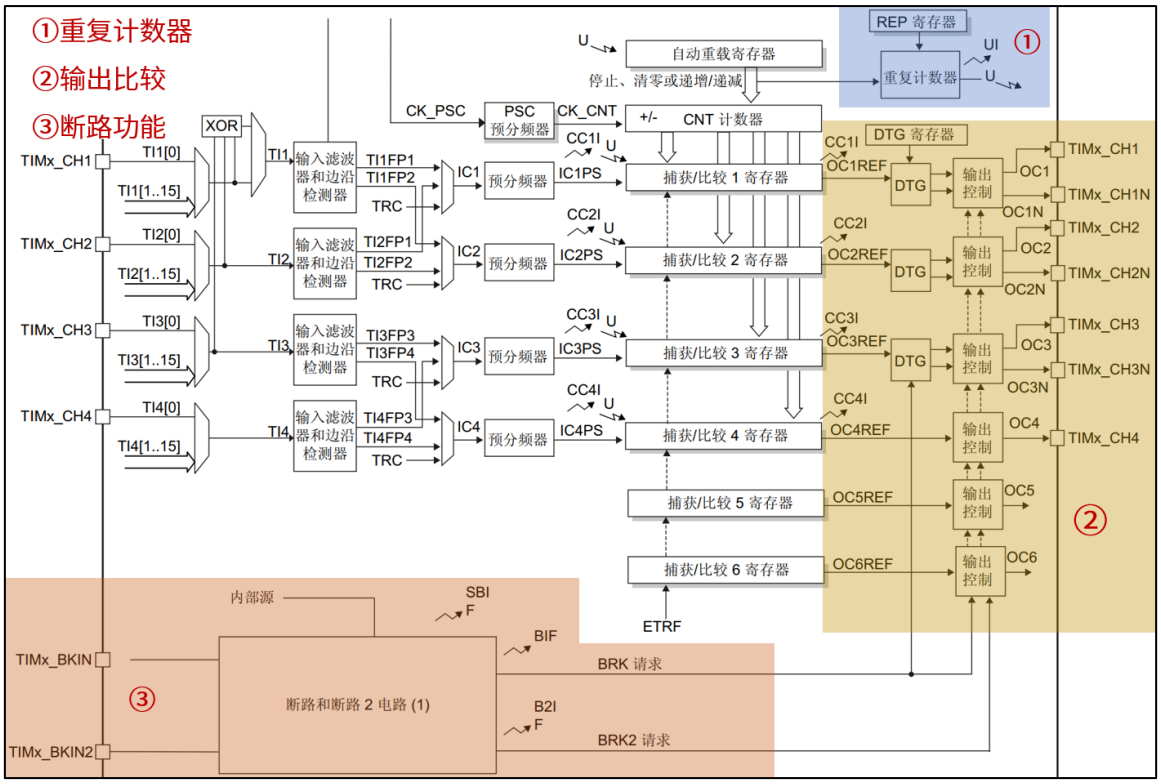


图 21.1.1 高级定时器框图

上图中，框出来三个部分，这是和通用定时器不同的地方，下面来分别介绍它们。

① 重复计数器

在 H7 系列中，除了高级定时器 TIM1 和 TIM8 之外，通用定时器 TIM15、TIM16 和 TIM17 也有重复计数器。通过查看手册可以知道 TIM15、TIM16 和 TIM17 虽为通用定时器，但是高级定时器有的功能，它们也基本都有，所以它们是很特殊的。

下面来介绍一下重复计数器有什么作用？在学习基本定时器和通用定时器的时候，我们知道定时器发生上溢或者下溢时，会直接生成更新事件。但是有重复计数器的定时器并不完全是这样的，定时器每次发生上溢或下溢时，重复计数器的值会减一，当重复计数器的值为 0 时，再发生一次上溢或者下溢才会生成定时器更新事件。如果我们设置重复计数器寄存器 RCR 的

值为 N，那么更新事件将在定时器发生 N+1 次上溢或下溢时发生。

这里需要注意的是重复计数器寄存器是具有影子寄存器的，所以 RCR 寄存器只是起缓冲的作用。RCR 寄存器的值会在更新事件发生时，被转移至其影子寄存器中，从而真正生效。

重复计数器的特性，在控制生成 PWM 信号时很有用，后面会有相应的实验。

② 输出比较

高级定时器输出比较部分和通用定时器相比，多了带死区控制的互补输出功能。图 21.1.1 第②部分的 TIMx_CH1N、TIMx_CH2N 和 TIMx_CH3N 分别是定时器通道 1、通道 2 和通道 3 的互补输出通道，通道 4 是没有互补输出通道的。DTG 是死区发生器，死区时间由 DTG[7:0]位来配置如果不使用互补通道和死区时间控制，那么高级定时器 TIM1/8，以及 TIM15/16/17 和其它通用定时器的输出比较部分使用方法基本一样，只是要注意 MOE 位得置 1 定时器才能输出。

如果使用互补通道，那么就有一定的区别了，具体我们在高级定时器互补输出带死区控制实验小节再来介绍。

③ 断路功能

断路功能也称刹车功能，一般用于电机控制的刹车。H7 系列有两个断路通道。断路 1 通道可以收集系统级故障（时钟失效和奇偶校验错误等）和应用故障（来自输入引脚：TIMx_BKIN 或者 TIMx_BKIN2，还可以来自内置比较器），可以在死区持续时间后将输出强制为预定义的电平（有效或无效）。断路 2 通道只包含应用故障，能够将输出强制为无效状态。系统复位后，断路功能默认被禁止，MOE 位为低。

使能断路功能的方法：将 TIMx_BDTR 的位 BKE（或 BK2E）置 1。断路输入引脚 TIMx_BKIN 或者 TIMx_BKIN2 的输入有效电平可通过 TIMx_BDTR 寄存器的位 BKP（或 BK2P）设置。

使能刹车功能后：由 TIMx_BDTR 的 MOE、OSSI、OSSR 位，TIMx_CR2 的 OISx、OISxN 位，TIMx_CCER 的 CCxE、CCxNE 位控制 OCx 和 OCxN 输出状态。无论何时，OCx 和 OCxN 输出都不能同时处在有效电平。

当发生断路输入后，会怎么样？

1，MOE 位被异步地清零，OCx 和 OCxN 为无效、空闲或复位状态(由 OSSI 位选择)。

2，OCx 和 OCxN 的状态：由相关控制位状态决定，当使用互补输出时：根据情况自动控制输出电平，参考《STM32H7xx 参考手册_V3（中文版）.pdf》手册 1465 页的表 313.具有断路功能的互补通道 Ocx 和 OcxN 的输出控制位。

3，BIF 位置 1，如果使能了 BIE 位，还会产生刹车中断；如果使能了 TDE 位，会产生 DMA 请求。

4，如果 AOE 位置 1，在下一个 更新事件 UEV 时，MOE 位被自动置 1。

高级定时器框图部分就简单介绍到这里，下面通过实际的实验来学习高级定时器。

21.2 高级定时器输出指定个数 PWM 实验

要实现定时器输出指定个数 PWM，只需要掌握下面几点内容：

第一，如果大家还不清楚定时器是如何输出 PWM 的，请回顾通用定时器 PWM 输出实验的内容，这部分的知识是一样的。但是需要注意的是：本实验，我们需要把 MOE 位置 1，这样定时器 8 的通道才能输出。

第二，要清楚重复计数器特性，设置重复计数器寄存器 RCR 的值为 N，那么更新事件将在定时器发生 N+1 次上溢或下溢时发生。换句话来说就是，想要指定输出 N 个 PWM，只需要把 N-1 写入 RCR 寄存器。因为在边沿对齐模式下，定时器溢出周期对应着 PWM 周期，我们只要在更新事件发生时，停止输出 PWM 就行。

第三，为了保证定时器输出指定个数的 PWM 后，定时器马上停止继续输出，我们使能更新中断，并在定时器中断里关闭计数器。

原理部分我们就讲到这里，下面直接开始寄存器的介绍。

21.2.1 TIM8 寄存器

本实验用到定时器 8，在《STM32H7xx 参考手册_V3（中文版）.pdf》手册的 38.4 小节（1447 页）可以找到 TIM8 的寄存器描述。这里我们将介绍 TIM8 寄存器里我们用到的几个重要的寄存器，具体如下：

- **控制寄存器 1（TIM8_CR1）**

TIM8 的控制寄存器 1 描述如图 21.2.1.1 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	UIFRE MAP	Res.	CKD[1:0]		ARPE	CMS[1:0]		DIR	OPM	URS	UDIS	CEN
				rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 7 **ARPE**：自动重载预装载使能 (Auto-reload preload enable)
0：TIMx_ARR 寄存器不进行缓冲
1：TIMx ARR 寄存器进行缓冲
位 0 **CEN**：计数器使能 (Counter enable)
0：禁止计数器
1：使能计数器
注： 只有事先通过软件将 CEN 位置 1，才可以使用外部时钟、门控模式和编码器模式。而触发模式可通过硬件自动将 CEN 位置 1。

图 21.2.1.1 TIM8_CR1 寄存器

上图中我们只列出了本章需要用的一些位，其中：位 7（ARPE）用于控制自动重载寄存器是否具有缓冲作用，在基本定时器的时候已经讲过，请回顾。在本实验中我们把该位要置 1，这样就算改变 ARR 寄存器的值，该值也不会马上生效，而是等待之前设置的 PWM 完整输出后（发生更新事件）才生效。

CEN 位，用于使能计数器的工作，必须要设置该位为 1，才可以开始计数。

- **捕获/比较模式寄存器 1（TIM8_CCMR1）**

TIM8 的捕获/比较模式寄存器 1，（TIMx_CCMR1/2），该寄存器一般有 2 个：TIMx_CCMR1 和 TIMx _CCMR2。TIMx_CCMR1 控制 CH1 和 CH2，而 TIMx_CCMR2 控制 CH3 和 CH4。TIM8_CCMR1 寄存器描述如图 21.2.1.2 所示：

这些通道可用于输入（捕获模式）或输出（比较模式）模式。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。对于任一给定位，OCxx 用于说明通道配置为输出时该位对应的功能，ICxx 则用于说明通道配置为输入时该位对应的功能。因此，必须注意同一个位在输入阶段和输出阶段具有不同的含义。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC2M[3]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC1M[3]
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
							rw								rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2 CE	OC2M[2:0]			OC2 PE	OC2 FE	CC2S[1:0]		OC1 CE	OC1M[2:0]			OC1 PE	OC1 FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]				IC1F[3:0]				IC1PSC[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 21.2.1.2 TIM8_CCMR1 寄存器

该寄存器的有些位在不同模式下，功能不一样，我们前面已经说过。比如我们要让 TIM8 的 CH1 输出 PWM 波为例，该寄存器的模式设置位 OC1M[2:0]就是对应着通道 1 的模式设置，此部分由 3 位组成。TIM8 总共可以配置成 8 种模式，我们使用的是 PWM 模式，所以这 3 位必须设置为 110 或者 111，分别对应 PWM 模式 1 和 PWM 模式 2。这两种 PWM 模式的区别就是输出有效电平的极性相反，这里我们设置为 PWM 模式 1。位 3 OC1PE 是输出比较通道 1 的预装使能，该位需要置 1，另外 CC1S[1:0]用于设置通道 1 的方向（输入/输出）默认设置为 0，就是设置通道作为输出使用。

- **捕获/比较使能寄存器（TIM8_ CCER）**

TIM8 的捕获/比较使能寄存器，该寄存器控制着各个输入输出通道的开关。TIM8_CCER

寄存器描述如图 21.2.1.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CC6P	CC6E	Res.	Res.	CC5P	CC5E
										rw	rw			rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CC4NP	Res.	CC4P	CC4E	CC3NP	CC3NE	CC3P	CC3E	CC2NP	CC2NE	CC2P	CC2E	CC1NP	CC1NE	CC1P	CC1E
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 21.2.1.3 TIM8_CCER 寄存器

该寄存器比较简单，要让 TIM8 的 CH1 输出 PWM 波，这里我们要使能 CC1E 位，该位是通道 1 输入/输出使能位，要想 PWM 从 IO 口输出，这个位必须设置为 1。CC1P 位是设置通道 1 的输出极性，我们默认设置 0，即 OC1 高电平有效。

- **事件产生寄存器（TIM8_ EGR）**

TIM8 的事件产生寄存器，该寄存器作用是让用户用软件方式产生各类事件。TIM8_EGR 寄存器描述如图 21.2.1.4 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	B2G	BG	TG	COMG	CC4G	CC3G	CC2G	CC1G	UG
							w	w	w	w	w	w	w	w	w

图 21.2.1.4 TIM8_EGR 寄存器

UG 位是更新事件的控制位，作用和定时器溢出时产生的更新事件一样，区别是这里是通过软件产生的，而定时器溢出是硬件自己完成的。只有开启了更新中断，这两种方式都可以产更新中断。本实验用到该位去产生软件更新器事件，在需要的时候把 UG 位置 1 即可，会由硬件自动清零。

- **重复计数器寄存器（TIM8_ RCR）**

重复计数器寄存器用于设置重复计数器值，因为它具有影子寄存器，所以它本身只是起缓冲作用。当更新事件发生时，该寄存器的值会转移到其影子寄存器中，从而真正起作用。TIM8_ RCR 寄存器描述如图 21.2.1.5 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
REP[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 15:0 **REP[15:0]**：重复计数器值 (Repetition Counter value)

使能预装载寄存器时，用户可通过这些位设置比较寄存器的更新频率（即，从预装载寄存器向有效寄存器周期性传输数据）；使能更新中断时，也可设置更新中断的生成速率。

与 REP_CNT 相关的减计数器每次计数到 0 时，都将生成一个更新事件并且计数器从 REP 值重新开始计数。由于只有生成重复更新事件 U_RC 时，REP_CNT 才会重载 REP 值，因此在生成下一重复更新事件之前，无论向 TIMx_RCR 寄存器写入何值都无影响。

这意味着 PWM 模式下 (REP+1) 相当于：

边沿对齐模式下的 PWM 周期数。

中心对齐模式下的 PWM 半周期数。

图 21.2.1.5 TIM8_ RCR 寄存器

该寄存器的 REP[15:0]位是低 8 位有效，即最大值 255。因为这个寄存器只是起缓冲作用，如果大家对该寄存器写入值后，想要立即生效，可以通过对 UG 位写 1，产生软件更新事件。

- **捕获/比较寄存器 1/2（TIM8_ CCR1/2/3/4）**

捕获/比较寄存器（TIM8_ CCR1/2/3/4），该寄存器总共有 4 个，对应 TIM8 的 4 个通道 CH1~CH4。我们使用的是通道 1，所以来看看 TIM8_ CCR1 寄存器描述如图 21.2.1.6 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR1[15:0]															
rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r

位 15:0 **CCR1[15:0]**：捕获/比较 1 值 (Capture/Compare 1 value)

如果通道 CC1 配置为输出：CCR1 为要装载到有效捕获/比较 1 寄存器的值（预装载值）。
如果没有通过 TIMx_CCMR1 寄存器中的 OC1PE 位来使能预装载功能，则该值立刻生效；否则只在发生更新事件时生效（拷贝到有效的捕获/比较寄存器 1）。
有效捕获/比较寄存器中包含要与计数器 TIMx_CNT 进行比较并在 OC1 输出上发出信号的值。

如果通道 CC1 配置为输入：CR1 为上一个输入捕获 1 事件 (IC1) 发生时的计数器值。只能读取 TIMx_CCR1 寄存器，无法对其进行编程。

图 21.2.1.6 TIM8_ CCR1 寄存器

在输出模式下，捕获/比较寄存器影子寄存器的值与 CNT 的值比较，根据比较结果产生相应动作，利用这点，我们通过修改这个寄存器的值，就可以控制 PWM 的占空比了。

- **断路和死区寄存器（TIM8_ BDTR）**

高级定时器 TIM1/8 在输出模式下，还需要配置：断路和死区寄存器（TIMx_BDTR），该寄存器各位描述如图 21.2.1.7 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	BK2P	BK2E	BK2F[3:0]				BKF[3:0]			
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MOE	AOE	BKP	BKE	OSSR	OSSI	LOCK[1:0]		DTG[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 15 **MOE**：主输出使能 (Main output enable)

只要断路输入（BRK 或 BRK2）为有效状态，此位便由硬件异步清零。此位由软件置 1，也可根据 AOE 位状态自动置 1。此位仅对配置为输出的通道有效。
0：响应断路事件（2 个）。禁止 OC 和 OCN 输出。
响应断路事件或向 MOE 写入 0 时：OC 和 OCN 输出被禁止或被强制为空闲状态，具体取决于 OSSI 位。
1：如果 OC 和 OCN 输出的相应使能位（TIMx_CCER 寄存器中的 CCxE 和 CCxNE 位）均置 1，则使能 OC 和 OCN 输出。
有关详细信息，请参见 OC/OCN 使能说明（*第 38.4.9 节：TIM1/TIM8 捕获/比较使能寄存器 (TIMx_CCER)*）。

图 21.2.1.7 TIMx_ BDTR 寄存器

本实验，我们只需要关注该寄存器的位 15（MOE），要想高级定时器的 PWM 正常输出，则必须设置 MOE 位为 1，否则不会有输出。

21.2.2 硬件设计

1. 例程功能

通过 TIM8_CH1（由 PC6 复用）输出 PWM，为了指示 PWM 的输出情况，我们用杜邦线连接 PC6 和 PB0，来实现 PWM 输出控制 LED1 的亮灭。上电默认输出 5 个 PWM 控制 LED1 亮灭五次。之后按一下按键 KEY0，就会输出 5 个 PWM 控制 LED1 亮灭五次。

2. 硬件资源

1）LED 灯
LED1 - PB0
2) 独立按键
KEY0 - PH3
3）定时器 8，使用 TIM8 通道 1，由 PC6 复用。

3. 原理图

定时器属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们通过 LED1 来指示 STM32H743 的定时器的 PWM 输出情况，同时还可以用按键 KEY0 进行控制。

21.2.3 程序设计

本实验用到的 HAL 库函数介绍请回顾通用定时器 PWM 输出实验。下面介绍一下定时器输出指定个数 PWM 的配置步骤。

定时器输出指定个数 PWM 配置步骤

1）开启 TIMx 和通道输出的 GPIO 时钟，配置该 IO 口的复用功能输出

首先开启 TIMx 的时钟，然后配置 GPIO 为复用功能输出。本实验我们默认用到定时器 8 通道 1，对应 IO 是 PC6，它们的时钟开启方法如下：

```
__HAL_RCC_TIM8_CLK_ENABLE();                /* 使能定时器 8 */
__HAL_RCC_GPIOC_CLK_ENABLE();               /* 开启 GPIOE 时钟 */
```

IO 口复用功能是通过函数 HAL_GPIO_Init 来配置的。

2）初始化 TIMx,设置 TIMx 的 ARR 和 PSC 等参数

使用定时器的 PWM 模式功能时，我们调用的是 HAL_TIM_PWM_Init 函数来初始化定时器 ARR 和 PSC 等参数。

注意：该函数会调用：HAL_TIM_PWM_MspInit 函数，我们可以通过后者存放定时器和 GPIO 时钟使能、GPIO 初始化、中断使能以及优先级设置等代码。

3）设置定时器为 PWM 模式，输出比较极性，比较值等参数

在 HAL 库中，通过 HAL_TIM_PWM_ConfigChannel 函数来设置定时器为 PWM1 模式或者 PWM2 模式，根据需求设置输出比较的极性，设置比较值（控制占空比）等。

本实验我们设置 TIM8 的通道 1 为 PWM1 模式，因为我们的 LED1（连接 PC6 和 PB0）是低电平亮，而我们希望输出最后一个 PWM 波的时候，LED1 就灭，所以我们设置输出比较极性为高。捕获/比较寄存器的值（即比较值）设置为自动重装载值的一半，即 PWM 占空比为 50%。

4）使能定时器更新中断，开启定时器并输出 PWM，配置定时器中断优先级

通过__HAL_TIM_ENABLE_IT 函数使能定时器更新中断。

通过 HAL_TIM_PWM_Start 函数使能定时器并开启输出 PWM。

通过 HAL_NVIC_EnableIRQ 函数使能定时器中断。

通过 HAL_NVIC_SetPriority 函数设置中断优先级。

5）编写中断服务函数

定时器中断服务函数为：TIMx_IRQHandler 等，当发生中断的时候，程序就会执行中断服务函数。HAL 库提供了一个定时器中断公共处理函数 HAL_TIM_IRQHandler，该函数会根据中断类型调用相关的中断回调函数。用户根据自己的需要重定义这些中断回调函数来处理中断程序。本实验我们不使用 HAL 库的中断回调机制，而是把中断程序写在定时器中断服务函数里。详见本章例程源码。

21.2.3.1 程序流程图

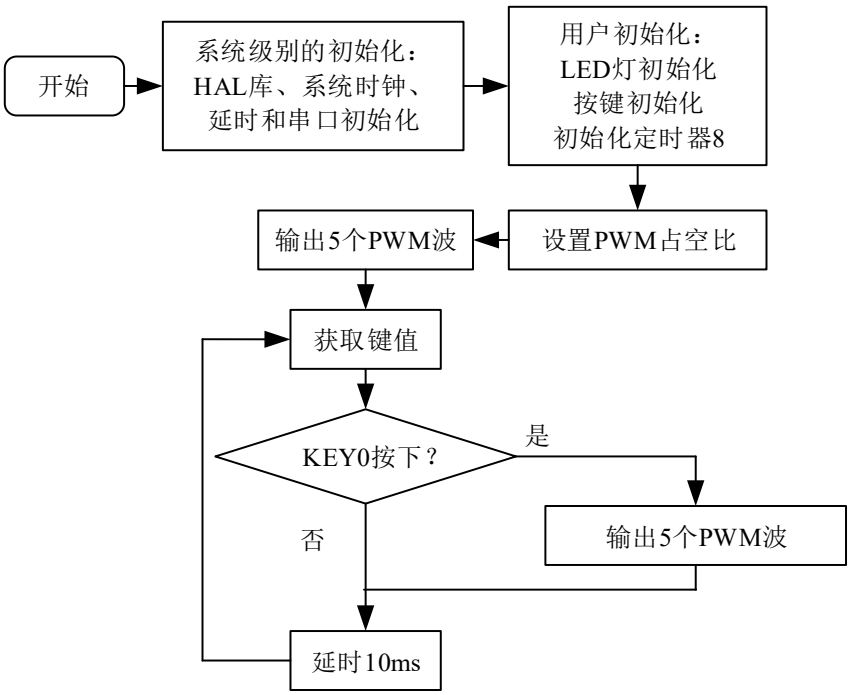


图 21.2.3.2.1 高级定时器输出指定个数 PWM 实验程序流程图

21.2.3.2 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。本实验使用到 TIM8，所以相关的 BSP 驱动代码还是存放到 atim.c 和 atim.h 文件中。

首先看 atim.h 头文件的几个宏定义：

```
/* TIMX 输出指定个数 PWM 定义
 * 这里输出的 PWM 通过 PC6(TIM8_CH1)输出，我们用杜邦线连接 PC6 和 PB0，
   然后在程序里面将 PB0 设置成浮空输入就可以 看到 TIM8_CH1 控制 LED1(GREEN)的亮灭，
   亮灭一次表示一个 PWM 波
 * 默认使用的是 TIM8_CH1.
 */
#define ATIM_TIMX_NPWM_CHY_GPIO_PORT            GPIOC
#define ATIM_TIMX_NPWM_CHY_GPIO_PIN             GPIO_PIN_6
#define ATIM_TIMX_NPWM_CHY_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOC_CLK_ENABLE(); }while(0)    /* PC 口时钟使能 */
#define ATIM_TIMX_NPWM_CHY_GPIO_AF              GPIO_AF3_TIM8

#define ATIM_TIMX_NPWM                          TIM8
#define ATIM_TIMX_NPWM_IRQn                     TIM8_UP_TIM13_IRQn
#define ATIM_TIMX_NPWM_IRQHandler               TIM8_UP_TIM13_IRQHandler
#define ATIM_TIMX_NPWM_CHY                      TIM_CHANNEL_1
/* 通道 Y,  1<= Y <=4 */
#define ATIM_TIMX_NPWM_CHY_CCRX                 TIM8->CCR1
/* 通道 Y 的输出比较寄存器 */
#define ATIM_TIMX_NPWM_CHY_CLK_ENABLE()
do{ __HAL_RCC_TIM8_CLK_ENABLE(); }while(0)   /* TIM8 时钟使能 */
```

可以把上面的宏定义分成两部分，第一部分是定时器 8 输入通道 1 对应的 IO 口的宏定义，第二部分则是定时器 8 输入通道 1 的相应宏定义。

下面看 atim.c 的程序，首先是输出指定个数 PWM 初始化函数，其定义如下：

```
/**
 * @brief 高级定时器 TIMX 通道 Y 输出指定个数 PWM 初始化函数
 * @note
```

```
 *          高级定时器的时钟来自 APB2, 当 D2PPRE2≥2 分频的时候,高级定时器的时钟为 APB2 时钟
 *          的 2 倍, 而 APB2 为 100Mhz,所以定时器时钟为 200MHz
 *          定时器溢出时间计算方法: Tout = ((arr + 1) * (psc + 1)) / Ft us.
 *          Ft=定时器工作频率,单位:Mhz
 *
 * @param  arr: 自动重装值
 * @param  psc: 预分频系数
 * @retval 无
 */
void atim_timx_npwm_chy_init(uint16_t arr, uint16_t psc)
{
    GPIO_InitTypeDef gpio_init_struct;
    TIM_OC_InitTypeDef timx_oc_npwm_chy = {0};  /* 定时器输出 */
    ATIM_TIMX_NPWM_CHY_GPIO_CLK_ENABLE();       /* TIMX 通道 IO 口时钟使能 */
    ATIM_TIMX_NPWM_CHY_CLK_ENABLE();            /* TIMX 时钟使能 */

    g_timx_npwm_chy_handle.Instance = ATIM_TIMX_NPWM;          /* 定时器 x */
    g_timx_npwm_chy_handle.Init.Prescaler = psc;               /* 定时器分频 */
    g_timx_npwm_chy_handle.Init.CounterMode = TIM_COUNTERMODE_UP;/*递增计数模式*/
    g_timx_npwm_chy_handle.Init.Period = arr;                  /* 自动重装载值 */
    g_timx_npwm_chy_handle.Init.AutoReloadPreload =
              TIM_AUTORELOAD_PRELOAD_ENABLE;           /* 使能 TIMx_ARR 进行缓冲 */
    g_timx_npwm_chy_handle.Init.RepetitionCounter = 0; /* 重复计数器初始值 */
    HAL_TIM_PWM_Init(&g_timx_npwm_chy_handle);         /* 初始化 PWM */

    gpio_init_struct.Pin = ATIM_TIMX_NPWM_CHY_GPIO_PIN;/* 开启通道 y 的 GPIO 口 */
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;           /* 复用推完输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;               /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;     /* 高速 */
    gpio_init_struct.Alternate = ATIM_TIMX_NPWM_CHY_GPIO_AF; /* 端口复用 */
    HAL_GPIO_Init(ATIM_TIMX_NPWM_CHY_GPIO_PORT, &gpio_init_struct);

    timx_oc_npwm_chy.OCMode = TIM_OCMODE_PWM1;  /* 模式选择 PWM 1 */
    timx_oc_npwm_chy.Pulse = arr / 2;           /* 设置比较值,此值用来确定占空比 */
    /* 这里默认设置比较值为自动重装载值的一半,即占空比为 50% */
    timx_oc_npwm_chy.OCPolarity = TIM_OCPOLARITY_HIGH;  /* 输出比较极性为高 */
    HAL_TIM_PWM_ConfigChannel(&g_timx_npwm_chy_handle, &timx_oc_npwm_chy,
    ATIM_TIMX_NPWM_CHY);                                /* 配置 TIMx 通道 y */

    /* 设置中断优先级, 抢占优先级 1, 子优先级 3 */
    HAL_NVIC_SetPriority(ATIM_TIMX_NPWM_IRQn, 1, 3);
    HAL_NVIC_EnableIRQ(ATIM_TIMX_NPWM_IRQn);            /* 开启 ITMx 中断 */

    /* 允许更新中断 */
    __HAL_TIM_ENABLE_IT(&g_timx_npwm_chy_handle, TIM_IT_UPDATE);
    /* 开启对应 PWM 通道 */
    HAL_TIM_PWM_Start(&g_timx_npwm_chy_handle, ATIM_TIMX_NPWM_CHY);
}
```

atim_timx_npwm_chy_init 函数包含了输出通道对应 IO 的初始代码、NVIC、使能时钟、定时器基础工作参数和输出模式配置的所有代码。下面来看看该函数的代码内容。

第一部分使能定时器和 GPIO 的时钟。

第二部分调用 HAL_TIM_PWM_Init 函数初始化定时器基础工作参数，如：ARR 和 PSC 等。

第三部分是定时器输出通道对应的 IO 的初始化。

第四部分调用 HAL_TIM_PWM_ConfigChannel 设置 PWM 模式以及比较值等参数。

第五部分是 NVIC 的初始化，配置抢占优先级、响应优先级和开启 NVIC 定时器中断。

最后是使能更新中断和使能通道输出。

为了方便代码的管理和移植性等，这里就没有使用 HAL_TIM_PWM_MspInit 这个函数来存放使能时钟、GPIO、NVIC 相关的代码，而是全部存放在 atim_timx_npwm_chy_init 函数中。

下面我们看设置 PWM 个数的函数，其定义如下：

```
/* g_npwm_remain 表示当前还剩下多少个脉冲要发送
 * 每次最多发送 256 个脉冲
 */
static uint32_t g_npwm_remain = 0;

/**
 * @brief       高级定时器 TIMX NPWM 设置 PWM 个数
 * @param       npwm: PWM 的个数, 1~2^32 次方个
 * @retval      无
 */
void atim_timx_npwm_chy_set(uint32_t npwm)
{
    if (npwm == 0) return;
    g_npwm_remain = npwm;                           /* 保存脉冲个数 */
    HAL_TIM_GenerateEvent(&g_timx_npwm_chy_handle, TIM_EVENTSOURCE_UPDATE);
    /* 产生一次更新事件,在中断里面处理脉冲输出 */
    __HAL_TIM_ENABLE(&g_timx_npwm_chy_handle);  /* 使能定时器 TIMX */
}
```

我们要输出多少个周期的 PWM 就用这个函数来设置。该函数作用是把我们设置输出的 PWM 个数的值赋值给静态全局变量 g_npwm_remain，该变量会在更新中断服务函数回调函数中发挥作用。最后对 TIMx_EGR 寄存器 UG 位写 1，产生一次更新事件，并使能定时器。

下面来介绍定时器中断服务函数，其定义如下：

```
/**
 * @brief       高级定时器 TIMX NPWM 中断服务函数
 * @param       无
 * @retval      无
 */
void ATIM_TIMX_NPWM_IRQHandler(void)
{
    uint16_t npwm = 0;

    /* 以下代码没有使用定时器 HAL 库共用处理函数来处理，而是直接通过判断中断标志位的方式 */
    if (__HAL_TIM_GET_FLAG(&g_timx_npwm_chy_handle, TIM_FLAG_UPDATE) != RESET)
    {
        if (g_npwm_remain >= 256)             /* 还有大于 256 个脉冲需要发送 */
        {
            g_npwm_remain = g_npwm_remain - 256;
            npwm = 256;
        }
        else if (g_npwm_remain % 256)         /* 还有位数（不到 256）个脉冲要发送 */
        {
            npwm = g_npwm_remain % 256;
            g_npwm_remain = 0;                /* 没有脉冲了 */
        }

        if (npwm) /* 有脉冲要发送 */
        {
            /* 设置重复计数寄存器值为 npwm-1, 即 npwm 个脉冲 */
            ATIM_TIMX_NPWM->RCR = npwm - 1;
            /* 产生一次更新事件,在中断里面处理脉冲输出 */
            HAL_TIM_GenerateEvent(&g_timx_npwm_chy_handle,
                                  TIM_EVENTSOURCE_UPDATE);
            /* 使能定时器 TIMX */
            __HAL_TIM_ENABLE(&g_timx_npwm_chy_handle);
        }
        else
        {
            /* 关闭定时器 TIMX，使用 HAL Disable 会清除 PWM 通道信息，此处不用 */
            ATIM_TIMX_NPWM->CR1 &= ~(1 << 0);
        }
        /* 清除定时器溢出中断标志位 */
```

```
        __HAL_TIM_CLEAR_IT(&g_timx_npwm_chy_handle, TIM_IT_UPDATE);
    }
}
```

这里我们没有使用 HAL 库的中断回调机制，而是像寄存器操作一样，直接通过判断中断标志位处理中断。通过__HAL_TIM_GET_FLAG 函数宏判断是否发生更新中断，然后进行更新中断的代码处理，最后通过__HAL_TIM_CLEAR_IT 函数宏清除更新中断标志位。

因为重复计数器寄存器 (TIM8_RCR)是 8 位有效的，所以在定时器中断服务函数中首先对全局变量 g_npwm_remain（即我们要输出的 PWM 个数）进行判断，是否大于 256，如果大于 256，那就得分次写入重复计数器寄存器。写入重复计数寄存器后，需要产生软件更新事件把 RCR 寄存器的值更新到 RCR 影子寄存器中，最后一定不要忘记清除定时器更新中断标志位。

在 main 函数里面编写如下代码：

```
int main(void)
{
    uint8_t key = 0;
    uint8_t t = 0;
    GPIO_InitTypeDef gpio_init_struct;

    sys_cache_enable();                         /* 打开 L1-Cache */
    HAL_Init();                                 /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);         /* 设置时钟, 400Mhz */
    delay_init(400);                            /* 延时初始化 */
    usart_init(115200);                         /* 初始化 USART */
    led_init();                                 /* 初始化 LED */
    key_init();                                 /* 初始化按键 */
    atim_timx_npwm_chy_init(10000-1,20000-1);/* 10Khz 的计数频率,1Hz 的 PWM 频率. */

    gpio_init_struct.Pin = LED1_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_INPUT;
    gpio_init_struct.Pull = GPIO_PULLUP;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
    HAL_GPIO_Init(LED1_GPIO_PORT, &gpio_init_struct);

    ATIM_TIMX_NPWM_CHY_CCRX = 5000;
    /* 设置 PWM 占空比,50%,这样可以控制每一个 PWM 周期,LED1(GREEN)
      有一半时间是亮的,一半时间是灭的,LED1 亮灭一次,表示一个 PWM 波 */
    atim_timx_npwm_chy_set(5);    /* 输出 5 个 PWM 波(控制 LED1(GREEN)闪烁 5 次) */
    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)    /* KEY0 按下 */
        {
            atim_timx_npwm_chy_set(5);
            /* 输出 5 个 PWM 波(控制 TIM8_CH1, 即 PC6 输出 5 个脉冲) */
        }
        t++;
        delay_ms(10);
        if (t > 50)             /* 控制 LED0 闪烁, 提示程序运行状态 */
        {
            t = 0;
            LED0_TOGGLE();
        }
    }
}
```

先看 atim_timx_npwm_chy_init(10000 - 1, 20000 - 1);这个语句，这两个形参分别设置自动重载寄存器的值为 9999，以及预分频器寄存器的值为 19999。按照 sys_stm32_clock_init 函数的配置，定时器 8 的时钟频率为 2 倍的 APB2 总线时钟频率，即 200MHz，可以得到计数器的计数频率是 10KHz。自动重载寄存器的值决定的是 PWM 周期或频率（请回顾 20.3 小节的内容），计数器计 10000 个数所用的时间是 PWM 的周期。在边沿对齐模式下，定时器的溢出周期等于 PWM 的周期。根据定时器溢出时间计算公式，可得：

Tout= ((arr+1)*(psc+1))/Tclk= ((9999+1)*(19999+1))/200000000=1s

再由频率是周期的倒数关系得到 PWM 的频率为 1Hz。

占空比则由捕获/比较寄存器（TIMx_CCRx）的值决定，这里就是由 TIM8_CCR1 寄存器决定。初始化定时器 8 时我们设置通道输出比较极性为高，ATIM_TIMX_NPWM_CHY_CCRX = 5000，就设置了占空比为 50%。因为我们的 LED 灯是低电平点亮，所以正占空比期间 LED 灯熄灭，负占空比期间 LED 灯亮。

21.2.4 下载验证

下载代码后，可以看到 LED1 亮灭五次，然后我们每按一下按键 KEY0，LED1 都会亮灭五次。下面我们使用正点原子 DS100 手持数字示波器，把 PC6 引脚的波形截获，具体如下：

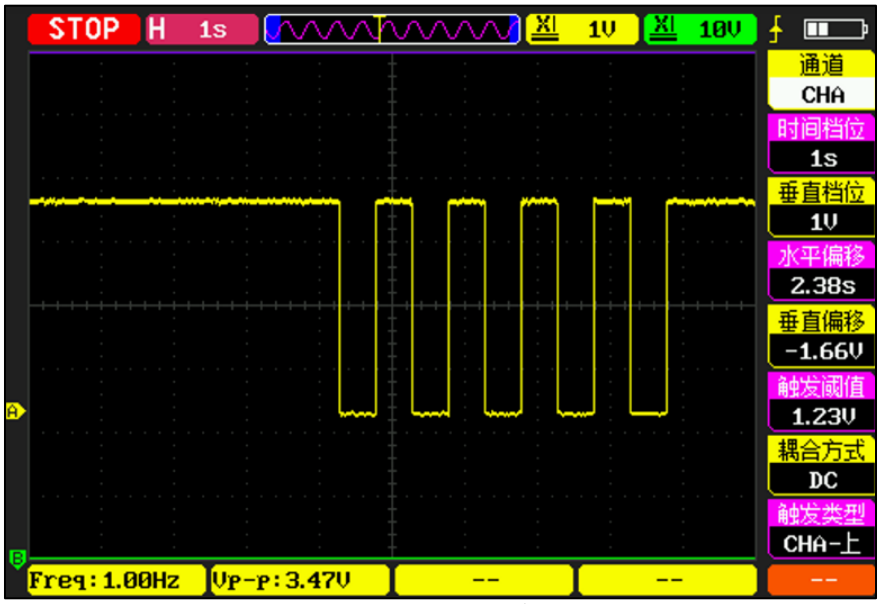


图 21.2.4 PC6 引脚波形图

由 LED 的原理图可以知道，PC6 引脚输出低电平 LED1 亮、输出高电平 LED1 灭。图 21.2.4 中，从左往右看，可以知道，LED1 一开始是熄灭的，然后经过 5 次亮灭，最后就是一直保持熄灭的状态。PWM 频率是 1Hz，占空比 50%，大家可以自行测量。

21.3 高级定时器输出比较模式实验

本小节我们来学习使用高级定时器输出比较模式下翻转功能，通过定时器 4 个通道分别输出 4 个 50%占空比、不同相位的 PWM。

输出比较模式下翻转功能作用是：当计数器的值等于捕获/比较寄存器影子寄存器的值时，OC1REF 发生翻转，进而控制通道输出（OCx）翻转。通过翻转功能实现输出 PWM 的具体原理如下：PWM 频率由自动重载寄存器（TIMx_ARR）的值决定，在这个过程中，只要自动重载寄存器的值不变，那么 PWM 占空比就固定为 50%。我们可以通过捕获/比较寄存器（TIMx_CCRx）的值改变 PWM 的相位。生成 PWM 的原理如图 21.3.1 所示：

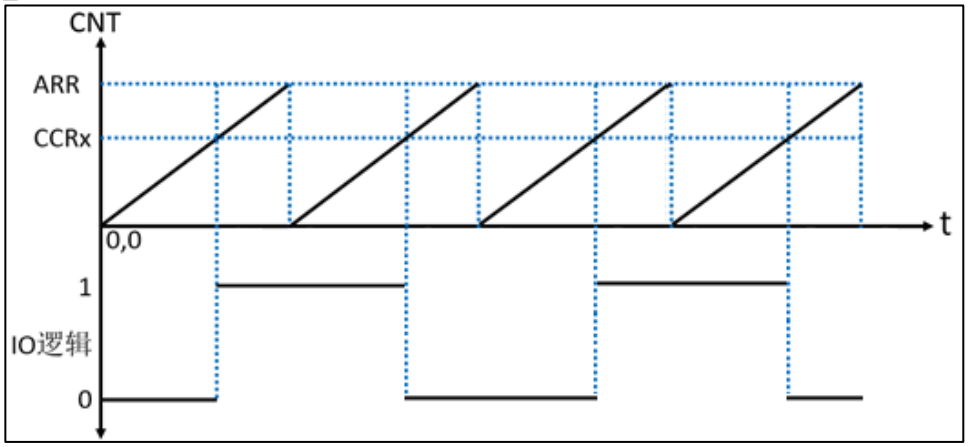


图 21.3.1 翻转功能输出 PWM 原理示意图

本实验就是根据图 21.3.1 的原理来设计的，具体实验是：我们设置固定的 ARR 值为 999，那么 PWM 占空比固定为 50%，通过改变 4 个通道的捕获/比较寄存器（TIMx_CCRx）的值使得每个通道输出的 PWM 的相位都不一样，注意捕获/比较寄存器的值设置范围是：0 ~ ARR。比如：TIMx_CCR1=250-1，TIMx_CCR2=500-1，TIMx_CCR3=750-1，TIMx_CCR4=1000-1，那么可以得到通道 1~通道 4 输出的 PWM 的相位分别是：25%、50%、75%、100%。翻转功能输出的 PWM 周期，这里用 T 表示，其计算公式如下：

$$T = 2 * (arr + 1) * ((psc + 1) / Tclk)$$

其中：

T：翻转功能输出的 PWM 周期（单位为 s）。

Tclk：定时器的时钟源频率（单位为 MHz）。

arr：自动重装寄存器（TIMx_ARR）的值。

psc：预分频器寄存器（TIMx_PSC）的值

21.3.1 TIM1/TIM8 寄存器

高级定时器输出比较模式除了用到定时器的时基单元：计数器寄存器(TIMx_CNT)、预分频器寄存器(TIMx_PSC)、自动重载寄存器(TIMx_ARR) 之外。主要还用到以下这些寄存器：

- **控制寄存器 1（TIMx_CR1）**

TIM1/TIM8 的控制寄存器 1 描述如图 21.3.1.1 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	UIFRE MAP	Res.	CKD[1:0]		ARPE	CMS[1:0]		DIR	OPM	URS	UDIS	CEN
				rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 7 **ARPE**：自动重载预装载使能 (Auto-reload preload enable)
0：TIMx_ARR 寄存器不进行缓冲
1：TIMx_ARR 寄存器进行缓冲

位 0 **CEN**：计数器使能 (Counter enable)
0：禁止计数器
1：使能计数器

图 21.3.1.1 TIMx_CR1 寄存器

上图中我们只列出了本实验需要用的一些位，其中：位 7（ARPE）用于控制自动重载寄存器是否具有缓冲作用，在基本定时器的时候已经讲过，请回顾。本实验中，我们把该位置 1。

CEN 位，用于使能计数器的工作，必须要设置该位为 1，才可以开始计数。

其它位保持复位值即可。

- **捕获/比较模式寄存器 1/2（TIMx_CCMR1/2）**

TIM1/TIM8 的捕获/比较模式寄存器（TIMx_CCMR1/2），该寄存器一般有 2 个：TIMx_CCMR1 和 TIMx _CCMR2。TIMx_CCMR1 控制 CH1 和 CH2，而 TIMx_CCMR2 控制 CH3 和 CH4。TIMx_CCMR1 寄存器描述如图 21.3.1.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC2M[3]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC1M[3]
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
							rw								rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2 CE	OC2M[2:0]			OC2 PE	OC2 FE	CC2S[1:0]		OC1 CE	OC1M[2:0]			OC1 PE	OC1 FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]				IC1F[3:0]				IC1PSC[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 21.3.1.2 TIMx_CCMR1 寄存器

该寄存器的有些位在不同模式下，功能不一样，我们现在用到输出比较模式。关于该寄存器的详细说明，请参考《STM32H7xx 参考手册_V3（中文版）.pdf》第 1458 页，38.4.7 节。

本实验我们用到了定时器 8 输出比较的 4 个通道，所以我们需要配置 TIM8_CCMR1 和 TIM8_CCMR2。以 TIM8_CCMR1 寄存器为例，模式设置位 OC1M[2:0]就是对应着通道 1 的模式设置，此部分由 4 位组成，总共可以配置 8 种模式，我们使用的是翻转功能，所以这 4 位必须设置为 011。通道 2 也是如此，将位 OC2M[2:0] 设置为 011。通道 3 和通道 4 就要设置 TIM8_CCMR2 寄存器的位 OC3M[2:0]和位 OC4M[2:0]。除此之外，我们还要设置输出比较的预装载使能位，如通道 1 对应输出比较的预装载使能位 OC1PE 置 1，其他通道也要把相应位置 1。

- **捕获/比较使能寄存器（TIMx_ CCER）**

TIM1/TIM8 的捕获/比较使能寄存器，该寄存器控制着各个输入输出通道的开关和极性。TIMx_CCER 寄存器描述如图 21.3.1.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CC6P	CC6E	Res.	Res.	CC5P	CC5E
										rw	rw			rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CC4NP	Res.	CC4P	CC4E	CC3NP	CC3NE	CC3P	CC3E	CC2NP	CC2NE	CC2P	CC2E	CC1NP	CC1NE	CC1P	CC1E
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 21.3.1.3 TIMx_CCER 寄存器

该寄存器比较简单，要让 TIM8 的 4 个通道都输出，我们需要把对应的捕获/比较 1 输出使能位置 1。通道 1 到通道 4 的使能位分别是：CC1E、CC2E、CC3E、CC4E，我们把这 4 个位置 1，使能通道输出。

- **捕获/比较寄存器 1/2/3/4（TIMx_ CCR1/2/3/4）**

捕获/比较寄存器（TIMx_ CCR1/2/3/4），该寄存器总共有 4 个，对应 4 个通道 CH1~CH4。本实验 4 个通道都要使用到，以通道 1 对应的 TIMx_ CCR1 寄存器为例，其描述如下图所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR1[15:0]															
rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r

位 15:0 **CCR1[15:0]**：捕获/比较 1 值 (Capture/Compare 1 value)

如果通道 CC1 配置为输出：CCR1 为要装载到有效捕获/比较 1 寄存器的值（预装载值）。

如果没有通过 TIMx_CCMR1 寄存器中的 OC1PE 位来使能预装载功能，则该值立刻生效；

否则只在发生更新事件时生效（拷贝到有效的捕获/比较寄存器 1）。

有效捕获/比较寄存器中包含要与计数器 TIMx_CNT 进行比较并在 OC1 输出上发出信号的值。

图 21.3.1.4 TIMx_ CCR1 寄存器

在本实验中，我们通过改变 TIMx_ CCR1/2/3/4 寄存器的值来改变 4 个通道输出的 PWM 的相位。

- **断路和死区寄存器（TIMx_ BDTR）**

本实验用的是高级定时器，所以我们还需要配置：断路和死区寄存器（TIMx_BDTR），该寄存器各位描述如图 21.3.1.5 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	BK2P	BK2E	BK2F[3:0]				BKF[3:0]			
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MOE	AOE	BKP	BKE	OSSR	OSSI	LOCK[1:0]		DTG[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 15 **MOE**：主输出使能 (Main output enable)

只要断路输入（BRK 或 BRK2）为有效状态，此位便由硬件异步清零。此位由软件置 1，也可根据 AOE 位状态自动置 1。此位仅对配置为输出的通道有效。

0：响应断路事件（2 个）。禁止 OC 和 OCN 输出。

响应断路事件或向 MOE 写入 0 时：OC 和 OCN 输出被禁止或被强制为空闲状态，具体取决于 OSSI 位。

1：如果 OC 和 OCN 输出的相应使能位（TIMx_CCER 寄存器中的 CCxE 和 CCxNE 位）均置 1，则使能 OC 和 OCN 输出。

有关详细信息，请参见 OC/OCN 使能说明（*第 38.4.9 节：TIM1/TIM8 捕获/比较使能寄存器 (TIMx_CCER)*）。

图 21.3.1.5 TIMx_ BDTR 寄存器

该寄存器，我们只需要关注位 15（MOE），要想高级定时器的通道正常输出，则必须设置 MOE 位为 1，否则不会有输出。

21.3.2 硬件设计

1. 例程功能

使用输出比较模式的翻转功能，通过定时器 8 的 4 路通道输出占空比固定为 50%、相位分别是 25%、50%、75%和 100%的 PWM。

2. 硬件资源

1）LED 灯

LED：LED0 - PB1

2）PC6 复用为 TIM8_CH1

PC7 复用为 TIM8_CH2

PC8 复用为 TIM8_CH3

PC9 复用为 TIM8_CH4

3. 原理图

定时器属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们需要通过示波器观察 PC6、PC7、PC8 和 PC9 引脚 PWM 输出的情况。

21.3.3 程序设计

21.3.3.1 定时器的 HAL 库驱动

定时器在 HAL 库中的驱动代码在前面已经介绍了部分，请回顾，这里我们再介绍几个本实验用到的函数。

1. HAL_TIM_OC_Init 函数

定时器的输出比较模式初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_TIM_OC_Init(TIM_HandleTypeDef *htim);
```

- **函数描述：**
  用于初始化定时器的输出比较模式。
- **函数形参：**
  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量，基本定时器的时候已经介绍。
- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

2. HAL_TIM_OC_ConfigChannel 函数

定时器的输出比较通道设置初始化函数。其声明如下：

```
HAL_StatusTypeDef HAL_TIM_OC_ConfigChannel(TIM_HandleTypeDef *htim,
                                           TIM_OC_InitTypeDef *sConfig, uint32_t Channel);
```

- **函数描述：**
  该函数用于初始化定时器的输出比较通道。
- **函数形参：**
  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量，用于配置定时器基本参数。
  形参 2 是 TIM_OC_InitTypeDef 结构体类型指针变量，用于配置定时器的输出比较参数。在通用定时器 PWM 输出实验已经介绍过 TIM_OC_InitTypeDef 结构体指针类型。
  形参 3 是定时器通道，范围：TIM_CHANNEL_1 到 TIM_CHANNEL_6。
- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

3. HAL_TIM_OC_Start 函数

定时器的输出比较启动函数，其声明如下：

```
HAL_StatusTypeDef HAL_TIM_OC_Start(TIM_HandleTypeDef *htim, uint32_t Channel);
```

- **函数描述：**
  用于启动定时器的输出比较模式。
- **函数形参：**
  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量。
  形参 2 是定时器通道，范围：TIM_CHANNEL_1 到 TIM_CHANNEL_6。
- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。
- **注意事项：**
  HAL 库也同样提供了单独使能定时器的输出通道函数，函数为：

```
void TIM_CCxChannelCmd(TIM_TypeDef *TIMx, uint32_t Channel,
                                          uint32_t ChannelState);
```

HAL_TIM_OC_Start 函数内部也调用了该函数。

定时器输出比较模式配置步骤

1）开启 TIMx 和通道输出的 GPIO 时钟，配置该 IO 口的复用功能输出

首先开启 TIMx 的时钟，然后配置 GPIO 为复用功能输出。本实验我们默认用到定时器 8 通道 1、2、3、4，对应 IO 是 PC6\PC7\PC8\PC9，它们的时钟开启方法如下：

```
__HAL_RCC_TIM8_CLK_ENABLE();                /* 使能定时器 8 */
__HAL_RCC_GPIOC_CLK_ENABLE();               /* 开启 GPIOC 时钟 */
```

IO 口复用功能是通过函数 HAL_GPIO_Init 来配置的。

2）初始化 TIMx,设置 TIMx 的 ARR 和 PSC 等参数

使用定时器的输出比较模式时，我们调用的是 HAL_TIM_OC_Init 函数来初始化定时器 ARR 和 PSC 等参数。

注意：该函数会调用 HAL_TIM_OC_MspInit 函数，我们可以通过后者存放定时器和 GPIO 时钟使能、GPIO 初始化、中断使能以及优先级设置等代码。

3）设置定时器为输出比较模式，输出比较极性，输出比较值、翻转功能等参数

在 HAL 库中，通过 HAL_TIM_OC_ConfigChannel 函数来设置定时器为输出比较模式，根据需求设置输出比较的极性，设置输出比较值、翻转功能等。

最后我们通过__HAL_TIM_ENABLE_OCxPRELOAD 函数使能通道的预装载。

4）开启定时器并输出 PWM

通过 HAL_TIM_OC_Start 函数使能定时器并开启输出。

21.3.3.2 程序流程图

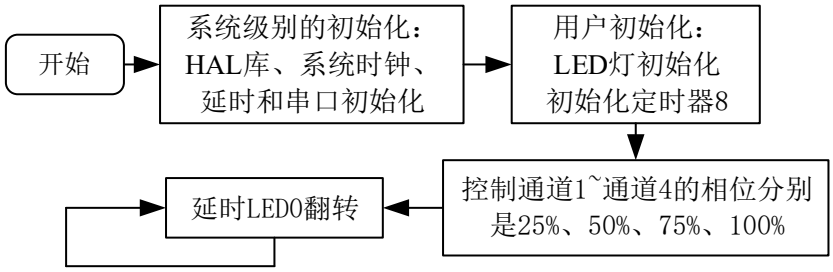


图 21.3.3.2.1 高级定时器输出比较模式实验程序流程图

21.3.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。高级定时器驱动源码包括两个文件：atim.c 和 atim.h。

首先看 atim.h 头文件的几个宏定义：

```
/* TIMX 输出比较模式 定义
 * 这里通过 TIM8 的输出比较模式,控制 PC6,PC7,PC8,PC9 输出 4 路 PWM,占空比 50%,并且每一路
   PWM 之间的相位差为 25% 修改 CCRx 可以修改相位.
 * 默认是针对 TIM1/TIM8
 */
#define ATIM_TIMX_COMP_CH1_GPIO_PORT            GPIOC
#define ATIM_TIMX_COMP_CH1_GPIO_PIN             GPIO_PIN_6
#define ATIM_TIMX_COMP_CH1_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOC_CLK_ENABLE(); }while(0)   /* PC 口时钟使能 */

#define ATIM_TIMX_COMP_CH2_GPIO_PORT            GPIOC
#define ATIM_TIMX_COMP_CH2_GPIO_PIN             GPIO_PIN_7
#define ATIM_TIMX_COMP_CH2_GPIO_CLK_ENABLE()
              do{ __HAL_RCC_GPIOC_CLK_ENABLE(); }while(0)     /* PC 口时钟使能 */

#define ATIM_TIMX_COMP_CH3_GPIO_PORT            GPIOC
#define ATIM_TIMX_COMP_CH3_GPIO_PIN             GPIO_PIN_8
#define ATIM_TIMX_COMP_CH3_GPIO_CLK_ENABLE()
              do{ __HAL_RCC_GPIOC_CLK_ENABLE(); }while(0)     /* PC 口时钟使能 */

#define ATIM_TIMX_COMP_CH4_GPIO_PORT            GPIOC
#define ATIM_TIMX_COMP_CH4_GPIO_PIN             GPIO_PIN_9
#define ATIM_TIMX_COMP_CH4_GPIO_CLK_ENABLE()
              do{ __HAL_RCC_GPIOC_CLK_ENABLE(); }while(0)     /* PC 口时钟使能 */

#define ATIM_TIMX_COMP_GPIO_AF    GPIO_AF3_TIM8        /* AF 功能选择 */
#define ATIM_TIMX_COMP            TIM8
#define ATIM_TIMX_COMP_CH1_CCRX   ATIM_TIMX_COMP->CCR1 /* 通道 1 的输出比较寄存器 */
#define ATIM_TIMX_COMP_CH2_CCRX   ATIM_TIMX_COMP->CCR2 /* 通道 2 的输出比较寄存器 */
#define ATIM_TIMX_COMP_CH3_CCRX   ATIM_TIMX_COMP->CCR3 /* 通道 3 的输出比较寄存器 */
#define ATIM_TIMX_COMP_CH4_CCRX   ATIM_TIMX_COMP->CCR4 /* 通道 4 的输出比较寄存器 */
#define ATIM_TIMX_COMP_CLK_ENABLE()
          do{ __HAL_RCC_TIM8_CLK_ENABLE(); }while(0)    /* TIM8 时钟使能 */
```

可以把上面的宏定义分成两部分，第一部分是定时器 8 输出通道 1~通道 4 对应的 IO 口的宏定义。第二部分则是定时器 8 的相应宏定义。

下面来看到 atim.c 文件的程序，首先是高级定时器输出比较模式初始化函数，其定义如下：

```
/**
 * @brief     高级定时器 TIMX 输出比较模式 初始化函数（使用输出比较模式）
 * @note
 *            配置高级定时器 TIMX 4 路输出比较模式 PWM 输出,实现 50%占空比,不同相位控制
 *            注意,本例程输出比较模式,每 2 个计数周期才能完成一个 PWM 输出,因此输出频率减半
 *            另外,我们还可以开启中断在中断里面修改 CCRx,从而实现不同频率/不同相位的控制
 *            但是我们不推荐这么使用,因为这可能导致非常频繁的中断,从而占用大量 CPU 资源
 *
 *            高级定时器的时钟来自 APB2,当 D2PPRE2≥2 分频的时候
 *            高级定时器的时钟为 APB2 时钟的 2 倍, 而 APB2 为 100M, 所以定时器时钟 = 200Mhz
 *            定时器溢出时间计算方法: Tout = ((arr + 1) * (psc + 1)) / Ft us.
 *            Ft=定时器工作频率,单位:Mhz
 * @param     arr: 自动重装值。
 * @param     psc: 时钟预分频数
 * @retval    无
 */
void atim_timx_comp_pwm_init(uint16_t arr, uint16_t psc)
{
    TIM_OC_InitTypeDef tim_oc_handle = {0};
    g_timx_comp_pwm_handle.Instance = ATIM_TIMX_COMP;       /* 定时器 1 */
    g_timx_comp_pwm_handle.Init.Prescaler = psc  ;          /* 定时器分频 */
    g_timx_comp_pwm_handle.Init.CounterMode=TIM_COUNTERMODE_UP;/* 递增计数模式*/
    g_timx_comp_pwm_handle.Init.Period = arr;               /* 自动重装载值 */
    g_timx_comp_pwm_handle.Init.AutoReloadPreload =
                TIM_AUTORELOAD_PRELOAD_ENABLE;       /* 使能 TIMx_ARR 进行缓冲 */
    HAL_TIM_OC_Init(&g_timx_comp_pwm_handle);       /* 输出比较模式初始化 */

    tim_oc_handle.OCMode = TIM_OCMODE_TOGGLE;        /* 比较输出模式翻转功能 */
    tim_oc_handle.Pulse = 250-1;                     /* 设置输出比较寄存器的值 */
    tim_oc_handle.OCPolarity = TIM_OCPOLARITY_HIGH; /* 输出比较极性为高 */ */
    HAL_TIM_OC_ConfigChannel(&g_timx_comp_pwm_handle, &tim_oc_handle,
                             TIM_CHANNEL_1); /* 初始化定时器的输出比较通道 1 */
   /* CCR1 寄存器预装载使能 */
    __HAL_TIM_ENABLE_OCxPRELOAD(&g_timx_comp_pwm_handle, TIM_CHANNEL_1);

    tim_oc_handle.Pulse = 500-1;
    HAL_TIM_OC_ConfigChannel(&g_timx_comp_pwm_handle, &tim_oc_handle,
                             TIM_CHANNEL_2); /* 初始化定时器的输出比较通道 2 */
   /* CCR2 寄存器预装载使能 */
    __HAL_TIM_ENABLE_OCxPRELOAD(&g_timx_comp_pwm_handle, TIM_CHANNEL_2);

    tim_oc_handle.Pulse = 750-1;
    HAL_TIM_OC_ConfigChannel(&g_timx_comp_pwm_handle, &tim_oc_handle,
                             TIM_CHANNEL_3); /* 初始化定时器的输出比较通道 3 */
   /* CCR3 寄存器预装载使能 */
    __HAL_TIM_ENABLE_OCxPRELOAD(&g_timx_comp_pwm_handle, TIM_CHANNEL_3);
    tim_oc_handle.Pulse = 1000-1;
    HAL_TIM_OC_ConfigChannel(&g_timx_comp_pwm_handle, &tim_oc_handle,
                             TIM_CHANNEL_4); /* 初始化定时器的输出比较通道 4 */
   /* CCR4 寄存器预装载使能 */
    __HAL_TIM_ENABLE_OCxPRELOAD(&g_timx_comp_pwm_handle, TIM_CHANNEL_4);

    HAL_TIM_OC_Start(&g_timx_comp_pwm_handle,TIM_CHANNEL_1);
    HAL_TIM_OC_Start(&g_timx_comp_pwm_handle,TIM_CHANNEL_2);
    HAL_TIM_OC_Start(&g_timx_comp_pwm_handle,TIM_CHANNEL_3);
    HAL_TIM_OC_Start(&g_timx_comp_pwm_handle,TIM_CHANNEL_4);
}
```

在 atim_timx_comp_pwm_init 函数中，首先调用 HAL_TIM_OC_Init 函数初始化定时器的 ARR 和 PSC 等参数。然后通过调用函数 HAL_TIM_OC_ConfigChannel 设置通道 1~通道 4 的工作参数，包括：输出比较模式功能、输出比较寄存器的值，输出极性等。接着调用 __HAL_TIM_ENABLE_OCxPRELOAD 函数宏使能 CCR1/2/3/4 寄存器的预装载。最后通过调

用函数 HAL_TIM_OC_Start 来使能 TIM1 通道 1~通道 4 输出。

HAL_TIM_OC_Init 函数会调用 HAL_TIM_OC_MspInit 回调函数，我们把使能定时器和通道对应的 IO 时钟、IO 初始化的代码存放到该函数里，其定义如下：

```
/**
 * @brief       定时器输出比较底层驱动
                HAL 库调用的接口，用于配置不同的输入捕获
 * @param       htim:定时器句柄
 * @note        此函数会被 HAL_TIM_OC_Init()调用
 * @retval      无
 */
void HAL_TIM_OC_MspInit(TIM_HandleTypeDef *htim)
{
    if(htim->Instance == ATIM_TIMX_COMP)
    {
        GPIO_InitTypeDef gpio_init_struct;
        ATIM_TIMX_COMP_CLK_ENABLE();
        ATIM_TIMX_COMP_CH1_GPIO_CLK_ENABLE();
        ATIM_TIMX_COMP_CH2_GPIO_CLK_ENABLE();
        ATIM_TIMX_COMP_CH3_GPIO_CLK_ENABLE();
        ATIM_TIMX_COMP_CH4_GPIO_CLK_ENABLE();

        gpio_init_struct.Pin = ATIM_TIMX_COMP_CH1_GPIO_PIN;
        gpio_init_struct.Mode = GPIO_MODE_AF_PP;
        gpio_init_struct.Pull = GPIO_NOPULL;
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
        gpio_init_struct.Alternate = ATIM_TIMX_COMP_GPIO_AF;
        HAL_GPIO_Init(ATIM_TIMX_COMP_CH1_GPIO_PORT, &gpio_init_struct);

        gpio_init_struct.Pin = ATIM_TIMX_COMP_CH2_GPIO_PIN;
        HAL_GPIO_Init(ATIM_TIMX_COMP_CH2_GPIO_PORT, &gpio_init_struct);

        gpio_init_struct.Pin = ATIM_TIMX_COMP_CH3_GPIO_PIN;
        HAL_GPIO_Init(ATIM_TIMX_COMP_CH3_GPIO_PORT, &gpio_init_struct);

        gpio_init_struct.Pin = ATIM_TIMX_COMP_CH4_GPIO_PIN;
        HAL_GPIO_Init(ATIM_TIMX_COMP_CH4_GPIO_PORT, &gpio_init_struct);
    }
}
```

该函数主要是使能定时器和通道对应的 IO 时钟，初始化 IO 口。要注意 IO 口的复用功能一定要选对了。

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t t = 0;
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    led_init();                             /* 初始化 LED */
    atim_timx_comp_pwm_init(1000 - 1, 200 - 1);  /* 1Mhz 的计数频率 1Khz 的周期 */
    /* atim_timx_comp_pwm_init 已经设置过输出比较寄存器，这是寄存器操作的方法 */
    ATIM_TIMX_COMP_CH1_CCRX = 250 - 1;      /* 通道 1 相位 25% */
    ATIM_TIMX_COMP_CH2_CCRX = 500 - 1;      /* 通道 2 相位 50% */
    ATIM_TIMX_COMP_CH3_CCRX = 750 - 1;      /* 通道 3 相位 75% */
    ATIM_TIMX_COMP_CH4_CCRX = 1000 - 1;     /* 通道 4 相位 100% */
    while (1)
    {
        t++;
        delay_ms(10);
        if (t >= 50)
        {
```

```
            LED0_TOGGLE();   /* LED0(RED)闪烁 */
            t = 0;
        }
    }
}
```

本小节开头我们讲解了输出比较模式翻转功能如何产生 PWM，下面结合程序一起计算出 PWM 的周期，频率等参数。

定时器 8 时钟源的时钟频率为 2 倍的 APB2 总线时钟频率，即 200MHz，而调用 atim_timx_comp_pwm_init(1000 - 1, 200 - 1)初始化函数之后，就相当于写入预分频寄存器的值为 199，写入自动重载寄存器的值为 999。将这些参数代入本小节介绍的翻转功能输出的 PWM 周期计算公式，可得：

T = 2*(arr+1)*((psc+1)/ Tclk) = 2*(999+1)*((199+1)/ 200000000) = 0.002s

由上述式子得到 PWM 周期为 2ms，频率为 500Hz。ARR 值为固定为 1000，所以占空比则固定为 50%。定时器 1 通道 1~通道 4 输出的 PWM 波的相位分别是：25%、50%、75%、100%。

21.3.4 下载验证

下载代码后，可以看到 LED0 在闪烁，说明程序已经正常在跑了。我们需要借助示波器观察 PC6、 PC7、 PC8 和 PC9 引脚 PWM 输出的情况，我们已经把示波器的显示内容截图出来，如图 21.3.4.1 所示：

图 21.3.4.1 相位为 25%、50%、75%、100%的 PWM 波

图 21.3.4.1 中，由上到下分别是引脚 PC6、PC7、PC8 和 PC9 输出的 PWM，即分别对应的是 TIM8_CH1、TIM8_CH2、TIM8_CH3 和 TIM8_CH4 输出的相位为 25%、50%、75%和 100%的 PWM。大家可以把其中一个通道的捕获/比较寄存器的值设置为 0，那么就可以得到 PWM 初相位的波形，即相位为 0%。

21.4 高级定时器互补输出带死区控制实验

本小节我们来学习使用高级定时器的互补输出带死区控制功能。对于刚接触这个知识的朋友可能会问：什么是互补输出？还带死区控制？What？下面给大家简单说一下。

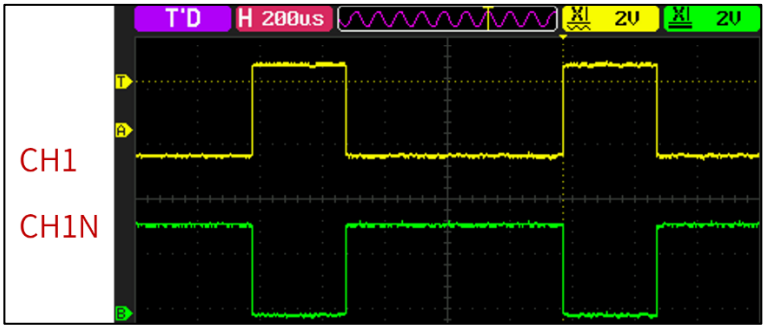


图 21.4.1 互补输出

上图中，CH1 输出黄色的 PWM，它的互补通道 CH1N 输出绿色的 PWM。通过对比，可以知道这两个 PWM 刚好是反过来的，CH1 的 PWM 为高电平期间，CH1N 的 PWM 则是低电平，反之亦然，这就是互补输出。

下面来看一下什么是带死区控制的互补输出？

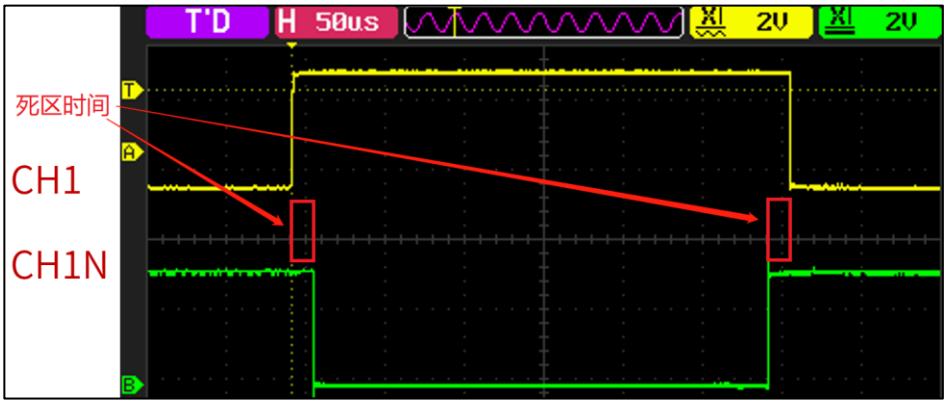


图 21.4.2 带死区控制的互补输出

上图中，CH1 输出的 PWM 和 CH1N 输出的 PWM 在高低电平转换间，插入了一段时间才实现互补输出。这段时间称为死区时间，可以通过 DTG[7:0]位配置控制死区时间的长度，后面会详细讲解如何配置死区时间。上图中，箭头指出的两段死区时间的长度是一样的，因为都是由同一个死区发生器产生。

理解了互补输出和带死区控制的互补输出，下面来看一下带死区控制的互补输出有什么用？带死区控制的互补输出经常被用于控制电机的 H 桥中，下面给大家画了一个 H 桥的简图：

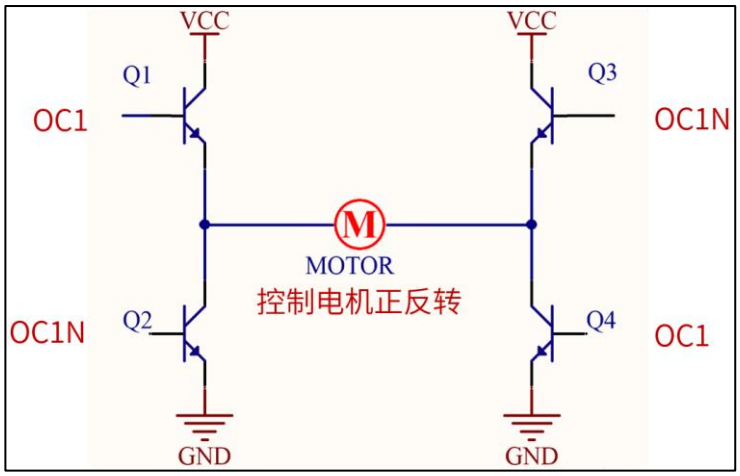


图 21.4.3 H 桥简图

图 21.4.3 是 H 桥的简图，实际控制电机正反转的 H 桥会根据复杂些，而且更多的是使用 MOS 管，这里只是为了解释带死区控制的互补输出在 H 桥中的控制逻辑原理，大家理解原理就行。上图的 H 桥搭建全部使用的是 NPN，并且导通逻辑都是基极为高电平时导通。如果 Q1 和 Q4 三极管导通，那么电机的电流方向是从左到右（假设电机正转）；如果 Q2 和 Q3 三极管导通，那么电机的电流方向是从右到左（假设电机反转）。上述就是 H 桥控制电机正反转的逻辑原理。但是同一侧的三极管是不可以同时导通的，否则会短路，比如：Q1 和 Q2 同时导通或者 Q3 和 Q4 同时导通，这都是不可取的。

下面大家想一下图 21.4.1 的 OC1（CH1）和 OC1N（CH1N）输出的 PWM 输入到图 21.4.3 的 H 桥中，会怎样？按理来说应该是 OC1N 输出高电平的时候，OC1 输出就是低电平，刚好 Q2 和 Q3 导通，电机的电流方向是从右到左（假设电机反转）；反之，OC1 输出高电平的时候，OC1N 输出就是低电平，刚好 Q1 和 Q4 导通，电机的电流方向是从左到右（假设电机正转），这似乎已经完美解决电机正反转问题了。实际上，元器件是有延迟特性的，比如：控制信号从 OC1 传导至电机，是要经过一定的时间的，复杂的 H 桥电路更是如此。由于元器件特性，就会导致直接使用互补输出信号驱动 H 桥时存在短路现象。为了避免这种情况，于是就有了带死区控制的互补输出来驱动 H 桥电路。如图 21.4.2 的死区时间就是为了解决元器件延迟特性的。用户必须根据与输出相连接的器件及其特性（电平转换器的固有延迟、开关器件产生的延迟）来调整死区时间。

死区时间计算

下面来看一下定时器的死区时间是怎么计算并设置的？死区时间是由 TIMx_CR1 寄存器的 CKD[1:0]位和 TIMx_BDTR 寄存器的 DTG[7:0]位来设置，如下图所示：

TIMx_CR1

CKD[1:0]：时钟分频 (Clock division)

此位域指示定时器时钟 (CK_INT) 频率与死区发生器以及数字滤波器（ETR、TIx）所使用的死区及采样时钟 (t_{DTS}) 之间的分频比，

00：$t_{DTS}=t_{CK_INT}$

01：$t_{DTS}=2*t_{CK_INT}$

10：$t_{DTS}=4*t_{CK_INT}$

11：保留，不要设置成此值

TIMx_BDTR

DTG[7:0]：配置死区发生器 (Dead-time generator setup)

此位域定义插入到互补输出之间的死区持续时间。DT 与该持续时间相对应。

DTG[7:5]=0xx => DT=DTG[7:0]x t_{dtg}，其中 $t_{dtg}=t_{DTS}$。

DTG[7:5]=10x => DT=(64+DTG[5:0])xt_{dtg}，其中 $T_{dtg}=2xt_{DTS}$。

DTG[7:5]=110 => DT=(32+DTG[4:0])xt_{dtg}，其中 $T_{dtg}=8xt_{DTS}$。

DTG[7:5]=111 => DT=(32+DTG[4:0])xt_{dtg}，其中 $T_{dtg}=16xt_{DTS}$。

示例：如果 T_{DTS}=125ns (8MHz)，则可能的死区值为：

0 到 15875 ns（步长为 125 ns）

16 us 到 31750 ns（步长为 250 ns）

32 us 到 63us（步长为 1 us）

64 us 到 126 us（步长为 2 us）

图 21.4.4 CKD[1:0]和 DTG[7:0]位

死区时间计算分三步走：

第一步：通过 CKD[1:0]位确定 tDTS。根据 CKD[1:0]位的描述，可以得到下面的式子：

$$tDTS = \frac{2\text{^}CKD[1:0]}{Tclk}$$

其中：

CKD[1:0]：CKD[1:0]位设置的值。

Tclk：定时器的时钟源频率（单位为 MHz）。

假设定时器时钟源频率是 200MHz，我们设置 CKD[1:0]位的值为 2，代入上面的式子可得：

$$tDTS = \frac{2^{CKD[1:0]}}{Tclk} = \frac{2^2}{200000000} = \frac{4}{200000000} = 20ns$$

通过上式可得 tDTS 约等于 20ns，本实验例程中我们也是这样设置的。

第二步：根据 DTG[7:5]选择计算公式。

第三步：代入选择的公式计算。

下面给大家举个例子，假设定时器时钟源频率是 200MHz，我们设置 CKD[1:0]位的值为 2，DTG[7:0]位的值为 250。从上面的例子知道 CKD[1:0]位的值为 2，得到的 tDTS=20ns。下面来看一下 DTG[7:0]位的值为 250，应该选择 DTG[7:0]位描述中哪条公式？250 的二进制数为 11111010，即 DTG[7:5]为 111，所以选择第四条公式：DT=(32+ DTG[4:0]) * t_{dtg}，其中 t_{dtg} = 16 * tDTS。可以看到手册上的式子符号大小写乱乱的，这里大小写不敏感。由手册的公式可以得到 DT = (32+ DTG[4:0]) * 16 * tDTS = (32+ 26) * 16 * 20ns = 18560ns = 18.56us，即死区时间为 18.56us。死区时间计算方法就给大家介绍到这里。

关于互补输出和死区插入的更多内容请看《STM32H7xx 参考手册_V3（中文版）.pdf》手册的 38.3.15 小节（1423 页），下面我们介绍相关的寄存器。

21.4.1 TIM1/TIM8 寄存器

高级定时器互补输出带死区控制除了用到定时器的时基单元：计数器寄存器(TIMx_CNT)、预分频器寄存器(TIMx_PSC)、自动重载寄存器(TIMx_ARR) 之外。主要还用到以下这些寄存器：

- **控制寄存器 1（TIMx_CR1）**

TIM1/TIM8 的控制寄存器 1 描述如图 21.4.1.1 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	UIFRE MAP	Res.	CKD[1:0]		ARPE	CMS[1:0]		DIR	OPM	URS	UDIS	CEN
				rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 9:8 **CKD[1:0]**：时钟分频 (Clock division)
此位域指示定时器时钟 (CK_INT) 频率与死区发生器以及数字滤波器（ETR、TIx）所使用的死区及采样时钟 (t_{DTS}) 之间的分频比，
00：$t_{DTS}=t_{CK_INT}$
01：$t_{DTS}=2*t_{CK_INT}$
10：$t_{DTS}=4*t_{CK_INT}$
11：保留，不要设置成此值

位 7 **ARPE**：自动重载预装载使能 (Auto-reload preload enable)
0：TIMx_ARR 寄存器不进行缓冲
1：TIMx_ARR 寄存器进行缓冲

位 0 **CEN**：计数器使能 (Counter enable)
0：禁止计数器
1：使能计数器
注：只有事先通过软件将 CEN 位置 1，才可以使用外部时钟、门控模式和编码器模式。而触发模式可通过硬件自动将 CEN 位置 1。

图 21.4.1.1 TIMx_CR1 寄存器

上图中我们只列出了本实验需要用的一些位，其中：位 7（APRE）用于控制自动重载寄存器是否进行缓冲，在基本定时器的时候已经讲过，请回顾。本实验中，我们把该位置 1。

CKD[1:0]位指示定时器时钟（CK_INT）频率与死区发生器以及数字滤波器（ETR、TIx）所使用的死区及采样时钟（tDTS）之间的分频比。我们设置 CKD[1:0]位为 10，结合定时器时钟源频率是 2 倍 APB2 总线时钟频率，即 200MHz，可以得到 tDTS=20ns。

CEN 位，用于使能计数器的工作，必须要设置该位为 1，才可以开始计数。

- **捕获/比较模式寄存器 1/2（TIMx_CCMR1/2）**

TIM1/TIM8 的捕获/比较模式寄存器（TIMx_CCMR1/2），该寄存器一般有 2 个：TIMx_CCMR1 和 TIMx _CCMR2。TIMx_CCMR1 控制 CH1 和 CH2，而 TIMx_CCMR2 控制 CH3 和 CH4。TIMx_CCMR1 寄存器描述如图 21.4.1.2 所示：

这些通道可用于输入（捕获模式）或输出（比较模式）模式。通道方向通过配置相应的 CCxS 位进行定义。此寄存器的所有其它位在输入模式和输出模式下的功能均不同。对于任一给定位，OCxx 用于说明通道配置为输出时该位对应的功能，ICxx 则用于说明通道配置为输入时该位对应的功能。因此，必须注意同一个位在输入阶段和输出阶段具有不同的含义。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC2M[3]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC1M[3]
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
							rw								rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2 CE	OC2M[2:0]			OC2 PE	OC2 FE	CC2S[1:0]		OC1 CE	OC1M[2:0]			OC1 PE	OC1 FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]				IC1F[3:0]				IC1PSC[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 21.4.1.2 TIMx_CCMR1 寄存器

该寄存器的有些位在不同模式下，功能不一样，我们现在用到输出比较模式。关于该寄存器的详细说明，请参考《STM32H7xx 参考手册_V3（中文版）.pdf》第 1458 页，38.4.7 节。

本实验我们用到了定时器 1 输出比较的通道 1，所以我们需要配置 TIM1_CCMR1 模式设置位 OC1M[3:0]，我们使用的是 PWM 模式 1，所以这 4 位必须设置为 0110。

- **捕获/比较使能寄存器（TIMx_ CCER）**

TIM1/TIM8 的捕获/比较使能寄存器，该寄存器控制着各个输入输出通道的开关和极性。TIMx_CCER 寄存器描述如图 21.4.1.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CC6P	CC6E	Res.	Res.	CC5P	CC5E
										rw	rw			rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CC4NP	Res.	CC4P	CC4E	CC3NP	CC3NE	CC3P	CC3E	CC2NP	CC2NE	CC2P	CC2E	CC1NP	CC1NE	CC1P	CC1E
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 21.4.1.3 TIMx_CCER 寄存器

该寄存器比较简单，要让 TIM1 的通道 1 输出，我们需要把对应的捕获/比较 1 输出使能位 CC1E 置 1。因为本实验中，我们需要实现互补输出，所以还需要把 CC1NE 位置 1，使能互补通道输出。CC1P 和 CC1NP 分别是通道 1 输出和通道 1 互补输出的极性设置位。这里我们把 CC1P 和 CC1NP 位都置 1，即输出极性为低，就可以得到互补的 PWM。

- **捕获/比较寄存器 1/2/3/4（TIMx_CCR1/2/3/4）**

捕获/比较寄存器（TIMx_ CCR1/2/3/4），该寄存器总共有 4 个，对应 4 个通道 CH1~CH4。我们使用的是通道 1，所以来看看 TIMx_ CCR1 寄存器描述如图 21.4.1.4 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR1[15:0]															
rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r

位 15:0 **CCR1[15:0]**：捕获/比较 1 值 (Capture/Compare 1 value)

如果通道 CC1 配置为输出：CCR1 为要装载到有效捕获/比较 1 寄存器的值（预装载值）。如果没有通过 TIMx_CCMR1 寄存器中的 OC1PE 位来使能预装载功能，则该值立刻生效；否则只在发生更新事件时生效（拷贝到有效的捕获/比较寄存器 1）。
有效捕获/比较寄存器中包含要与计数器 TIMx_CNT 进行比较并在 OC1 输出上发出信号的值。

如果通道 CC1 配置为输入：CR1 为上一个输入捕获 1 事件 (IC1) 发生时的计数器值。只能读取 TIMx_CCR1 寄存器，无法对其进行编程。

图 21.4.1.4 TIMx_ CCR1 寄存器

对于 TIM1 和 TIM8 来说，该寄存器 16 位有效位，本实验中可以通过改变该寄存器的值来改变 PWM 的占空比。

- **断路和死区寄存器（TIMx_ BDTR）**

TIM1/TIM8 断路和死区寄存器，该寄存器各位描述如图 21.4.1.5 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	BK2P	BK2E	BK2F[3:0]				BKF[3:0]			
						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MOE	AOE	BKP	BKE	OSSR	OSSI	LOCK[1:0]		DTG[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

注：　由于可以根据 LOCK 配置锁定位 BK2P、BK2E、BK2F[3:0]、BKF[3:0]、AOE、BKP、BKE、OSSI、OSSR 和 DTG[7:0] 的写操作，因此必须在第一次对 TIMx_BDTR 寄存器执行写访问时对这些位进行配置。

位 14 **AOE**：自动输出使能 (Automatic output enable)

0：MOE 只能由软件置 1

1：MOE 可由软件置 1，也可在发生下一更新事件时自动置 1（如果断路输入 BRK 和 BRK2 均无效）

注：　只要编程了 LOCK（TIMx_BDTR 寄存器中的 LOCK 位）级别 1，此位即无法修改。

位 13 **BKP**：断路极性 (Break polarity)

0：断路输入 BRK 为低电平有效

1：断路输入 BRK 为高电平有效

注：　只要编程了 LOCK（TIMx_BDTR 寄存器中的 LOCK 位）级别 1，此位即无法修改。

注：　对该位执行任何写操作后，都需要经过 1 个 APB 时钟周期的延迟才生效。

位 12 **BKE**：断路使能 (Break enable)

该位可使能完整的断路保护（包括连接到 bk_acth 的所有源和相应的 BKIN 源，如*图 381：断路和断路 2 电路概述*所示）。

0：禁止断路功能

1：使能断路功能

注：　编程了 LOCK（TIMx_BDTR 寄存器中的 LOCK 位）级别 1 后，此位即无法修改。

注：　对该位执行任何写操作后，都需要经过 1 个 APB 时钟周期的延迟才生效。

位 7:0 **DTG[7:0]**：配置死区发生器 (Dead-time generator setup)

此位域定义插入到互补输出之间的死区持续时间。DT 与该持续时间相对应。

DTG[7:5]=0xx => DT=DTG[7:0]x t_{dtg}，其中 $t_{dtg}=t_{DTS}$。

DTG[7:5]=10x => DT=(64+DTG[5:0])xt_{dtg}，其中 $T_{dtg}=2xt_{DTS}$。

DTG[7:5]=110 => DT=(32+DTG[4:0])xt_{dtg}，其中 $T_{dtg}=8xt_{DTS}$。

DTG[7:5]=111 => DT=(32+DTG[4:0])xt_{dtg}，其中 $T_{dtg}=16xt_{DTS}$。

示例：如果 T_{DTS}=125ns (8MHz)，则可能的死区值为：

0 到 15875 ns（步长为 125 ns）

16 us 到 31750 ns（步长为 250 ns）

32 us 到 63us（步长为 1 us）

64 us 到 126 us（步长为 2 us）

注：　只要编程了 LOCK（TIMx_BDTR 寄存器中的 LOCK 位）级别 1、2 或 3，此位域即无法修改。

图 21.4.1.5 TIMx_ BDTR 寄存器

该寄存器控制定时器的断路和死区控制的功能。我们先看断路控制，H7 系列的高级定时器 TIM1/TIM8 有两个断路通道，本实验我们用到断路 1（断路输入引脚为 PE15）。要使用断路 1 就要使能位 BKE，置 1 即可。

位 BKP 选择断路输入信号有效电平。本实验中，我们选择低电平有效，即 BKP 置 0。

位 AOE 是自动输出使能位，如果使能 AOE 位，那么在我们输入刹车信号后再断开了刹车信号，互补的 PWM 会自动恢复输出，如果失能 AOE 位，那么在输入刹车信号后再断开了刹车信号，互补的 PWM 就不会恢复输出，而是一直保持刹车信号输入时的状态。为了方便观察，我们使能该位，即置 1。

BKF[3:0]是断路输入信号的滤波器，这里我们不需要用滤波器，所以这 4 个位置 0。

要使用断路 2 功能可以参考断路 1 的配置即可。

位 MOE 是使能主输出，想要高级定时器的通道正常输出，则必须设置 MOE 位为 1。

最后是 DTG[7:0]位，用于设置死区时间，前面已经教过大家怎么设置了。这里以我们例程的设置为例，CKD[1:0] 设置为 10，定时器时钟源频率是 200MHz，所以 tDTS = 20ns。

本例程的 DTG[7:0]位的值设置为十进制 100，即二进制数 0110 0100。DTG[7:5]=011，符合第一条式子：DT=DTG[7:0] * t_{dtg}，其中 t_{dtg} = tDTS。DT 是死区时间，可以得到 DT = 100*20 ns = 2us。到后面下载验证小节，我们通过示波器验证一下这个死区时间计算的理论值和实际值是否一样。

21.4.2 硬件设计

1. 例程功能

1，利用 TIM1_CH1(PE9)输出 70%占空比的 PWM，它的互补输出通道(PE8)则是输出 30%占空比的 PWM。

2，刹车功能，当给刹车输入引脚(PE15)输入低电平时，进行刹车，即 PE9 和 PE8 停止输出 PWM。

3，LED0 闪烁指示程序运行。

2. 硬件资源

1）LED 灯

LED：LED0 - PB1

2）定时器 1

TIM1 正常输出通道 PE9

TIM1 互补输出通道 PE8

TIM1 刹车输入通道 PE15

3. 原理图

定时器属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们需要通过示波器观察 PE9 和 PE8 引脚 PWM 输出的情况。还可以通过给 PE15 引脚接入低电平进行刹车。

21.4.3 程序设计

21.4.3.1 定时器的 HAL 库驱动

定时器在 HAL 库中的驱动代码在前面已经介绍了部分，这里我们再介绍几个本实验用到的函数。

1. HAL_TIMEx_ConfigBreakDeadTime 函数

定时器的断路和死区时间配置初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_TIMEx_ConfigBreakDeadTime(TIM_HandleTypeDef *htim,
                                                TIM_BreakDeadTimeConfigTypeDef
                                                    *sBreakDeadTimeConfig);
```

- **函数描述：**

  用于初始化定时器的断路（即刹车）和死区时间。

- **函数形参：**

  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量，基本定时器的时候已经介绍。

  形参 2 是 TIM_BreakDeadTimeConfigTypeDef 结构体类型指针变量，用于配置断路和死区参数，其定义如下：

```
typedef struct
{
  uint32_t OffStateRunMode;         /* 运行模式下的关闭状态选择 */
  uint32_t OffStateIDLEMode;        /* 空闲模式下的关闭状态选择 */
  uint32_t LockLevel;               /* 寄存器锁定配置 */
  uint32_t DeadTime;                /* 死区时间设置 */
  uint32_t BreakState;              /* 断路（即刹车）输入使能控制 */
  uint32_t BreakPolarity;           /* 断路输入极性 */
  uint32_t BreakFilter;             /* 断路输入滤波器 */
```

```
  uint32_t Break2State;              /* 断路 2 输入使能控制 */
  uint32_t Break2Polarity;           /* 断路 2 输入极性 */
  uint32_t Break2Filter;             /* 断路 2 输入滤波器 */
  uint32_t AutomaticOutput;          /* 自动恢复输出使能控制 */
} TIM_BreakDeadTimeConfigTypeDef;
```

- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

2. HAL_TIMEx_PWMN_Start 函数

定时器的互补输出启动函数。其声明如下：

```
HAL_StatusTypeDef HAL_TIMEx_PWMN_Start(TIM_HandleTypeDef *htim,
                                       uint32_t Channel);
```

- **函数描述：**
  该函数用于启动定时器的互补输出。
- **函数形参：**
  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量，用于配置定时器基本参数。
  形参 2 是定时器通道，范围：TIM_CHANNEL_1 到 TIM_CHANNEL_6。
- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

定时器互补输出带死区控制配置步骤

1）开启 TIMx 和通道输出以及刹车输入的 GPIO 时钟，配置该 IO 口的复用功能输出

首先开启 TIMx 的时钟，然后配置 GPIO 为复用功能输出。本实验我们默认用到定时器 1 通道 1，对应 IO 是 PE9，互补输出通道引脚是 PE8，刹车输入引脚是 PE15，它们的时钟开启方法如下：

```
__HAL_RCC_TIM1_CLK_ENABLE();                /* 使能定时器 1 */
__HAL_RCC_GPIOE_CLK_ENABLE();               /* 开启 GPIOE 时钟 */
```

IO 口复用功能是通过函数 HAL_GPIO_Init 来配置的。

2）初始化 TIMx，设置 TIMx 的 ARR 和 PSC 等参数

这里我们要使用定时器的 PWM 模式功能，所以调用的是 HAL_TIM_PWM_Init 函数来初始化定时器 ARR 和 PSC 等参数。注意：本实验要使用该函数配置 TIMx_CR1 寄存器的 CKD[1:0] 位，从而确定 t DTS，方便后续设置死区时间。

注意：该函数会调用：HAL_TIM_PWM_MspInit 函数，但是为不跟前面的实验共用该回调函数，提高独立性，我们就直接在 atim_timx_cplm_pwm_init 函数中，使能定时器时钟和 GPIO 时钟，初始化通道对应 IO 引脚等。

3）设置定时器为 PWM 模式，输出比较极性，互补输出极性等参数

通过 HAL_TIM_PWM_ConfigChannel 函数来设置定时器为 PWM1 模式，根据需求设置 OCy 输出极性和 OCyN 互补输出极性等。

4）设置死区参数

通过 HAL_TIMEx_ConfigBreakDeadTime 函数来设置死区参数，比如：设置死区时间、运行模式的关闭输出状态、空闲模式的关闭输出状态、刹车输入有效信号极性、刹车输入信号滤波设置、是否使用断路 2 和是否允许刹车后自动恢复输出等。

5）启动 Ocy 输出以及 OCyN 互补输出

通过 HAL_TIM_PWM_Start 函数启动 OCy 输出，通过 HAL_TIMEx_PWMN_Start 函数启动启动 OCyN 互补输出。

21.4.3.2 程序流程图

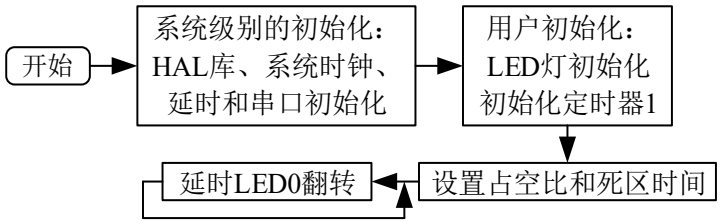


图 21.4.3.2.1 高级定时器互补输出带死区控制实验

21.4.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。高级定时器驱动源码包括两个文件：atim.c 和 atim.h。

首先看 atim.h 头文件的几个宏定义：

```
/* TIMX 互补输出模式 定义
 * 这里设置互补输出相关硬件配置, CHY 即正常输出, CHYN 即互补输出
 * 修改 CCRx 可以修改占空比.
 * 默认是针对 TIM1/TIM8
 */
/* 输出通道引脚 */
#define ATIM_TIMX_CPLM_CHY_GPIO_PORT            GPIOE
#define ATIM_TIMX_CPLM_CHY_GPIO_PIN             GPIO_PIN_9
#define ATIM_TIMX_CPLM_CHY_GPIO_AF              GPIO_AF1_TIM1   /* AF 功能选择 */
#define ATIM_TIMX_CPLM_CHY_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOE_CLK_ENABLE(); }while(0)    /* PE 口时钟使能 */

/* 互补输出通道引脚 */
#define ATIM_TIMX_CPLM_CHYN_GPIO_PORT           GPIOE
#define ATIM_TIMX_CPLM_CHYN_GPIO_PIN            GPIO_PIN_8
#define ATIM_TIMX_CPLM_CHYN_GPIO_AF             GPIO_AF1_TIM1   /* AF 功能选择 */
#define ATIM_TIMX_CPLM_CHYN_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOE_CLK_ENABLE(); }while(0)    /* PE 口时钟使能 */

/* 刹车输入引脚 */
#define ATIM_TIMX_CPLM_BKIN_GPIO_PORT           GPIOE
#define ATIM_TIMX_CPLM_BKIN_GPIO_PIN            GPIO_PIN_15
#define ATIM_TIMX_CPLM_BKIN_GPIO_AF             GPIO_AF1_TIM1   /* AF 功能选择 */
#define ATIM_TIMX_CPLM_BKIN_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOE_CLK_ENABLE(); }while(0)    /* PE 口时钟使能 */

/* 互补输出使用的定时器 */
#define ATIM_TIMX_CPLM                          TIM1
#define ATIM_TIMX_CPLM_CHY                      TIM_CHANNEL_1
#define ATIM_TIMX_CPLM_CHY_CCRY                 ATIM_TIMX_CPLM->CCR1
#define ATIM_TIMX_CPLM_CLK_ENABLE()
                  do{ __HAL_RCC_TIM1_CLK_ENABLE(); }while(0) /* TIM1 时钟使能 */
```

可以把上面的宏定义分成两部分，第一部分包括是定时器 1 输出、互补输出和刹车输入通道对应的 IO 口的宏定义，第二部分则是定时器 1 的相应宏定义。

下面来看 atim.c 文件的程序，首先是高级定时器互补输出初始化函数，其定义如下：

```
/**
 * @brief   高级定时器 TIMX 互补输出 初始化函数（使用 PWM 模式 1）
 * @note
 *          配置高级定时器 TIMX 互补输出, 一路 OCy 一路 OCyN, 并且可以设置死区时间
 *
 *          高级定时器的时钟来自 APB2,当 D2PPRE2≥2 分频的时候
 *          高级定时器的时钟为 APB2 时钟的 2 倍, 而 APB2 为 100M, 所以定时器时钟 = 200Mhz
 *          定时器溢出时间计算方法: Tout = ((arr + 1) * (psc + 1)) / Ft us.
```

```
 *           Ft=定时器工作频率,单位:Mhz
 * @param    arr: 自动重装值。
 * @param    psc: 时钟预分频数
 * @retval   无
 */
void atim_timx_cplm_pwm_init(uint16_t arr, uint16_t psc)
{
    GPIO_InitTypeDef gpio_init_struct = {0};
    TIM_OC_InitTypeDef tim_oc_cplm_pwm = {0};

    ATIM_TIMX_CPLM_CLK_ENABLE();                /* TIMx 时钟使能 */
    ATIM_TIMX_CPLM_CHY_GPIO_CLK_ENABLE();    /* 通道 X 对应 IO 口时钟使能 */
    ATIM_TIMX_CPLM_CHYN_GPIO_CLK_ENABLE();   /* 通道 X 互补通道对应 IO 口时钟使能 */
    ATIM_TIMX_CPLM_BKIN_GPIO_CLK_ENABLE();   /* 通道 X 刹车输入对应 IO 口时钟使能 */

    gpio_init_struct.Pin = ATIM_TIMX_CPLM_BKIN_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;
    gpio_init_struct.Pull = GPIO_PULLUP;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;
    gpio_init_struct.Alternate = ATIM_TIMX_CPLM_CHY_GPIO_AF; /* 端口复用 */
    HAL_GPIO_Init(ATIM_TIMX_CPLM_BKIN_GPIO_PORT, &gpio_init_struct);

    gpio_init_struct.Pin = ATIM_TIMX_CPLM_CHY_GPIO_PIN;
    gpio_init_struct.Alternate = ATIM_TIMX_CPLM_CHY_GPIO_AF; /* 端口复用 */
    HAL_GPIO_Init(ATIM_TIMX_CPLM_CHY_GPIO_PORT, &gpio_init_struct);

    gpio_init_struct.Pin = ATIM_TIMX_CPLM_CHYN_GPIO_PIN;
    gpio_init_struct.Alternate = ATIM_TIMX_CPLM_CHY_GPIO_AF; /* 端口复用 */
    HAL_GPIO_Init(ATIM_TIMX_CPLM_CHYN_GPIO_PORT, &gpio_init_struct);

    g_timx_cplm_pwm_handle.Instance = ATIM_TIMX_CPLM;         /* 定时器 x */
    g_timx_cplm_pwm_handle.Init.Prescaler = psc;              /* 预分频系数 */
    g_timx_cplm_pwm_handle.Init.CounterMode = TIM_COUNTERMODE_UP;/* 向上计数 */
    g_timx_cplm_pwm_handle.Init.Period = arr;                 /* 自动重装载值 */
    g_timx_cplm_pwm_handle.Init.ClockDivision= TIM_CLOCKDIVISION_DIV4;
    /* CKD[1:0] = 10, tDTS = 4 * tCK_INT = Ft / 4 = 50Mhz*/
    g_timx_cplm_pwm_handle.Init.AutoReloadPreload =
                        TIM_AUTORELOAD_PRELOAD_ENABLE;/* 使能影子寄存器 TIMx_ARR */
    HAL_TIM_PWM_Init(&g_timx_cplm_pwm_handle) ;

    tim_oc_cplm_pwm.OCMode = TIM_OCMODE_PWM1;              /* PWM 模式 1 */
    tim_oc_cplm_pwm.OCPolarity = TIM_OCPOLARITY_LOW;       /* OCy 低电平有效 */
    tim_oc_cplm_pwm.OCNPolarity = TIM_OCNPOLARITY_LOW;     /* OCyN 低电平有效 */
    tim_oc_cplm_pwm.OCIdleState = TIM_OCIDLESTATE_SET;     /* 当 MOE=0，OCx=1 */
    tim_oc_cplm_pwm.OCNIdleState = TIM_OCNIDLESTATE_SET;   /* 当 MOE=0，OCxN=1 */
    HAL_TIM_PWM_ConfigChannel(&g_timx_cplm_pwm_handle, &tim_oc_cplm_pwm,
                              ATIM_TIMX_CPLM_CHY);

    /* 设置死区参数，开启死区中断 */
    g_sbreak_dead_time_config.OffStateRunMode = TIM_OSSR_DISABLE;

   /* 运行模式的关闭输出状态 */
    g_sbreak_dead_time_config.OffStateIDLEMode = TIM_OSSI_DISABLE;

   /* 空闲模式的关闭输出状态 */
    g_sbreak_dead_time_config.LockLevel = TIM_LOCKLEVEL_OFF;

   /* 不用寄存器锁功能 */
    g_sbreak_dead_time_config.BreakState = TIM_BREAK_ENABLE;

   /* 使能刹车输入 */
```

```
    g_sbreak_dead_time_config.BreakPolarity = TIM_BREAKPOLARITY_LOW;
    /* 刹车输入有效信号极性为低 */
    g_sbreak_dead_time_config.BreakFilter = 0;
    /* 刹车输入信号滤波设置 */
    g_sbreak_dead_time_config.Break2State = TIM_BREAK2_DISABLE;
    /* 不使用断路 2 */
    g_sbreak_dead_time_config.AutomaticOutput = TIM_AUTOMATICOUTPUT_ENABLE;
    /* 使能 AOE 位，允许刹车结束后自动恢复输出 */
    HAL_TIMEx_ConfigBreakDeadTime(&g_timx_cplm_pwm_handle,
                                  &g_sbreak_dead_time_config);
    /* OCy 输出使能 */
    HAL_TIM_PWM_Start(&g_timx_cplm_pwm_handle, ATIM_TIMX_CPLM_CHY);

   /* OCyN 输出使能 */
    HAL_TIMEx_PWMN_Start(&g_timx_cplm_pwm_handle, ATIM_TIMX_CPLM_CHY);
}
```

在 atim_timx_cplm_pwm_init 函数中，没有使用 HAL 库的 MSP 回调，而是把相关的初始化都写到该函数里面。

第一部分，使能定时器和相关通道对应的 GPIO 时钟，以及初始化相关 IO 引脚。

第二部分，通过 HAL_TIM_PWM_Init 函数初始化定时器的 ARR 和 PSC 等参数。

第三部分，通过 HAL_TIM_PWM_ConfigChannel 函数设置 PWM 模式 1、输出极性，以及输出空闲状态等。

第四部分，通过 HAL_TIMEx_ConfigBreakDeadTime 函数配置断路功能。

最后一定记得要调用 HAL_TIM_PWM_Start 函数和 HAL_TIMEx_PWMN_Start 函数启动通道输出和互补通道输出。

为了方便，我们还定义了设置输出比较值和死区时间的函数，其定义如下：

```
/**
 * @brief       定时器 TIMX 设置输出比较值 & 死区时间
 * @param       ccr: 输出比较值
 * @param       dtg: 死区时间
 *   @arg       dtg[7:5]=0xx 时, 死区时间 = dtg[7:0] * tDTS
 *   @arg       dtg[7:5]=10x 时, 死区时间 = (64 + dtg[5:0]) * 2  * tDTS
 *   @arg       dtg[7:5]=110 时, 死区时间 = (32 + dtg[4:0]) * 8  * tDTS
 *   @arg       dtg[7:5]=111 时, 死区时间 = (32 + dtg[4:0]) * 16 * tDTS
 *   @note      tDTS = 1 / (Ft /  CKD[1:0]) = 1 / 50M = 20ns
 * @retval      无
 */
void atim_timx_cplm_pwm_set(uint16_t ccr, uint8_t dtg)
{
    g_sbreak_dead_time_Config.DeadTime = dtg;       /* 死区时间设置 */
    HAL_TIMEx_ConfigBreakDeadTime(&g_timx_cplm_pwm_handle,
                                  &g_sbreak_dead_time_Config);/* 重设死区时间 */
    __HAL_TIM_MOE_ENABLE(&g_timx_cplm_pwm_handle);  /* MOE=1,使能主输出 */
    ATIM_TIMX_CPLM_CHY_CCRY = ccr;                  /* 设置比较寄存器 */
}
```

通过重新调用 HAL_TIMEx_ConfigBreakDeadTime 函数设置死区时间，注意这里的 g_sbreak_dead_time_config 是全局结构体变量，在 atim_timx_cplm_pwm_init 函数已经初始化其他结构体成员了，这里只是对 DeadTime 成员（死区时间）配置。死区时间的计算方法前面已经讲解过，这里只要把要设置的 DTG[7:0]值，通过 dtg 形参赋值给 DeadTime 结构体成员就行。另外一个形参是 ccr，用于设置捕获/比较寄存器的值，即控制 PWM 的占空比。

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t t = 0;
    sys_cache_enable();                         /* 打开 L1-Cache */
    HAL_Init();                                 /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);         /* 设置时钟, 400Mhz */
    delay_init(400);                            /* 延时初始化 */
```

```
    usart_init(115200);                             /* 串口初始化为 115200 */
    led_init();                                     /* 初始化 LED */
    atim_timx_cplm_pwm_init(1000 - 1, 20 - 1); /* 10Mhz 的计数频率 10Khz 的周期 */
    atim_timx_cplm_pwm_set(300, 100);               /* 占空比:7:3,死区时间 100*tDTS */
    while (1)
    {
        t++;
        delay_ms(10);
        if (t >= 50)
        {
            LED0_TOGGLE();  /* LED0(RED)闪烁 */
            t = 0;
        }
    }
}
```

先看 atim_timx_cplm_pwm_init(1000 - 1, 20 - 1)这个语句，这两个形参分别设置自动重载寄存器的值为 999，以及定时器预分频器寄存器的值为 19。先看预分频系数，我们设置为 20 分频，定时器 1 的时钟源频率是 2 倍的 APB2 总线时钟频率，即 200MHz，可以得到计数器的计数频率是 10MHz，即每 0.1us 计数一次。再到自动重载寄存器的值为 999 决定的是 PWM 的频率（周期），可以得到 PWM 的周期为(999+1)*0.1us = 100us。边沿对齐模式下，使用 PWM 模式 1 或者 PWM 模式 2，得到的 PWM 周期是定时器溢出时间。这里的 100us，也可以直接通过定时器溢出时间计算公式 Tout= ((arr+1)*(psc+1))/Tclk 得到。

调用 atim_timx_cplm_pwm_set(300, 100) 这个语句，相当于设置捕获/比较寄存器的值为 300，DTG[7:0]的值为 100。通过计算可以得到 PWM 的占空比为 70%，死区时间为 2us。根据 PWM 生成原理分析，再结合图 21.3.2 产生 PWM 示意图，以及我们在 atim_timx_cplm_pwm_init 函数配置 PWM 模式 1、OCy 输出极性为低，占空比的计算很简单，可以由(1000-300)/1000 得到。关于死区时间的计算方法，前面已经讲解过，这里以 DTG[7:0]的值为 100 为例，再来讲解一遍计算过程。由前面讲解的内容知道，我们例程配置 CKD[1:0]位的值为 2，可以得到 tDTS= 20ns。基于这个前提，通过改变 DTG[7:0]的值，可以得到不同的死区时间。这里我们配置 DTG[7:0]的值为 100，即二进制数 0110 0100，符合第一种情况 dtg[7:5]=0xx 时，死区时间 DT = DTG [7:0] * tDTS。可以得到死区时间 DT = 100*20 ns = 2us。

下面我们下载到开发板验证一下。

21.4.4 下载验证

下载代码后，可以看到 LED0 在闪烁，说明程序已经正常在跑了。我们需要借助示波器观察 PE9 正常输出和 PE8 互补输出 PWM 的情况，示波器显示截图如图 21.4.4.1 所示：

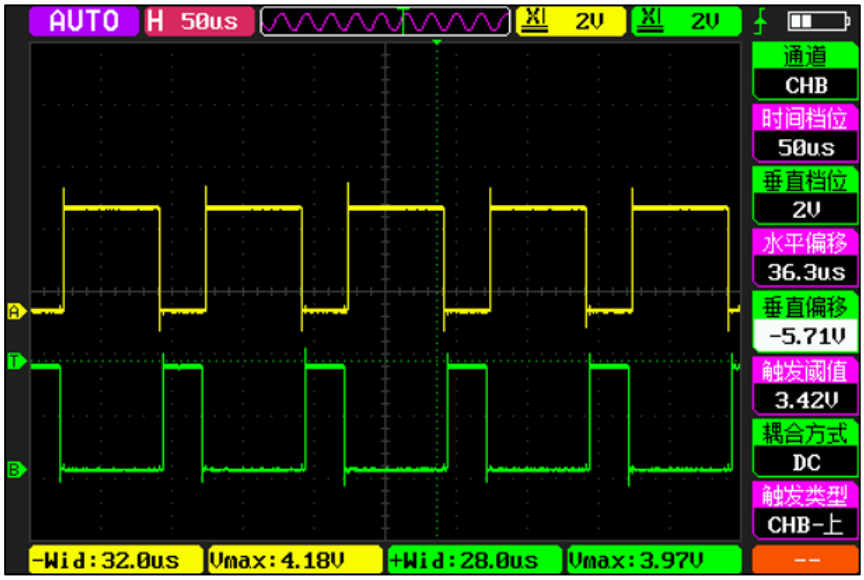


图 21.4.4.1 PE9 正常输出和 PE8 互补输出 PWM 的情况

图 21.4.4.1 中的由上到下分别是 PE9 正常输出 70%占空比的 PWM 波和 PE8 互补输出 30%占空比的 PWM 波。正常输出的 PWM 波的正脉宽减去互补输出的 PWM 的负脉宽的值除以 2 就是死区时间，也可以是互补输出的 PWM 的正脉宽减去正常输出的 PWM 波的负脉宽的值除以 2。我们使用第一种方法得到：死区时间 =（32 - 28）/2 us= 2us。与我们理论计算得到的值 2us 一样。

要是不相信，我们再举个例子，我们把调用的函数改为 atim_timx_cplm_pwm_set(300, 250)，即配置 DTG[7:0]的值为 250，这个例子的计算过程在本实验前面死区时间计算的内容讲过，这里就不再赘述。经过计算得到死区时间 DT =18.56us。修改好后，示波器显示截图如下图所示：

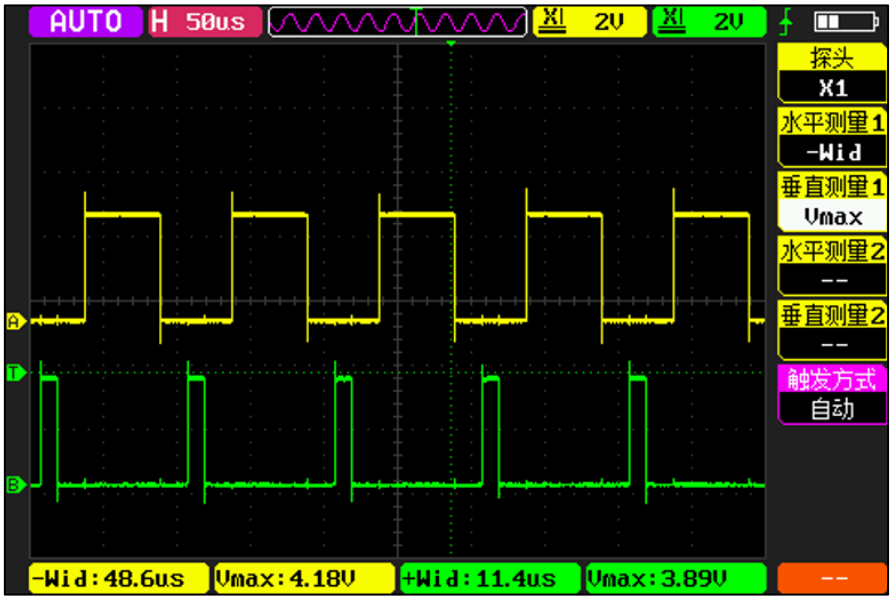


图 21.4.4.2 修改程序后 PE9 正常输出和 PE8 互补输出 PWM 的情况

由图 21.4.4.2 可得到，死区时间 DT =（48.6 - 11.4）/2 us= 18.6us。与我们理论计算得到的值 18.56us 也是差不多的，误差在正常范围。由此证明我们的死区时间设置是没有问题。

刹车功能验证：当给刹车输入引脚（PE15）接入低电平（这里直接用杜邦线把 PE15 连接到 GND）时，就会进行刹车，MOE 位被硬件清零。由《STM32H7xx 参考手册_V3（中文版）.pdf》第 1465 页 表 313 可以知道刹车信号输入后，如果存在时钟，经过死区后 OCx=OISx 且 OCxN=OISxN。在 atim_timx_cplm_pwm_init 函数中，我们设置当 MOE=0 时，OCx=1、OCxN=1，即 PE9 和 PE8 都是输出高电平。下面通过示波器来验证一下，如图 21.4.4.3 所示：

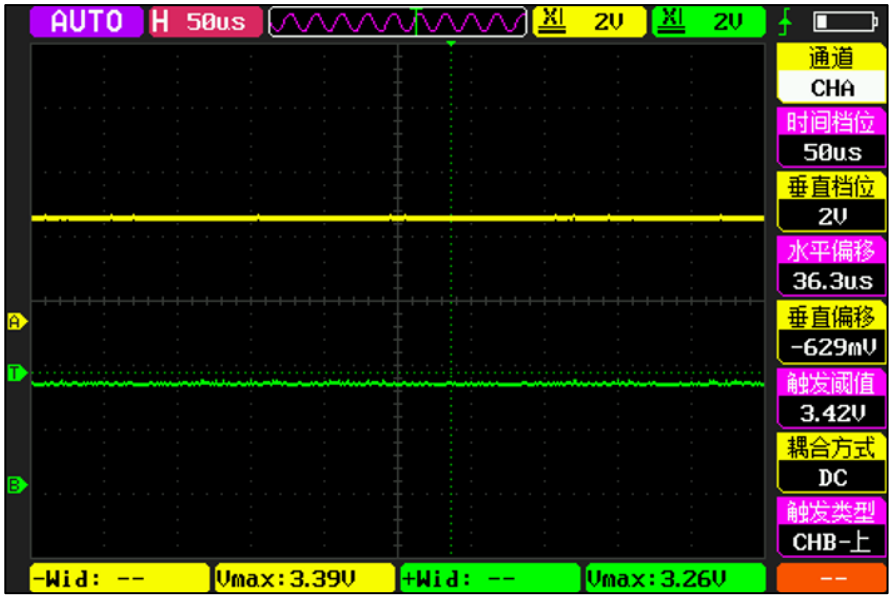


图 21.4.4.3 刹车后的输出情况

从上图可以看到 PE9 和 PE8 输出的都是高电平，符合我们预期的设置。

另外因为我们使能了 AOE 位（即把该位置 1），如果刹车输入为无效极性时，MOE 位在发生下一个更新事件时自动置 1，恢复运行模式（即继续输出 PWM）。因此当停止给 PE15 接入低电平（拔掉之前连接的杜邦线），PWM 会自动恢复输出。

21.5 高级定时器 PWM 输入模式实验

本小节我们来学习使用高级定时器 PWM 输入模式，此模式是输入捕获模式的一个特例。PWM 输入模式经常被应用于测量 PWM 脉宽和频率。PWM 输入模式在《STM32H7xx 参考手册_V3（中文版）.pdf》手册 1415 页有详细的文字描述。下面我们结合这些文字，配合高级定时器框图给大家介绍 PWM 输入的工作原理。

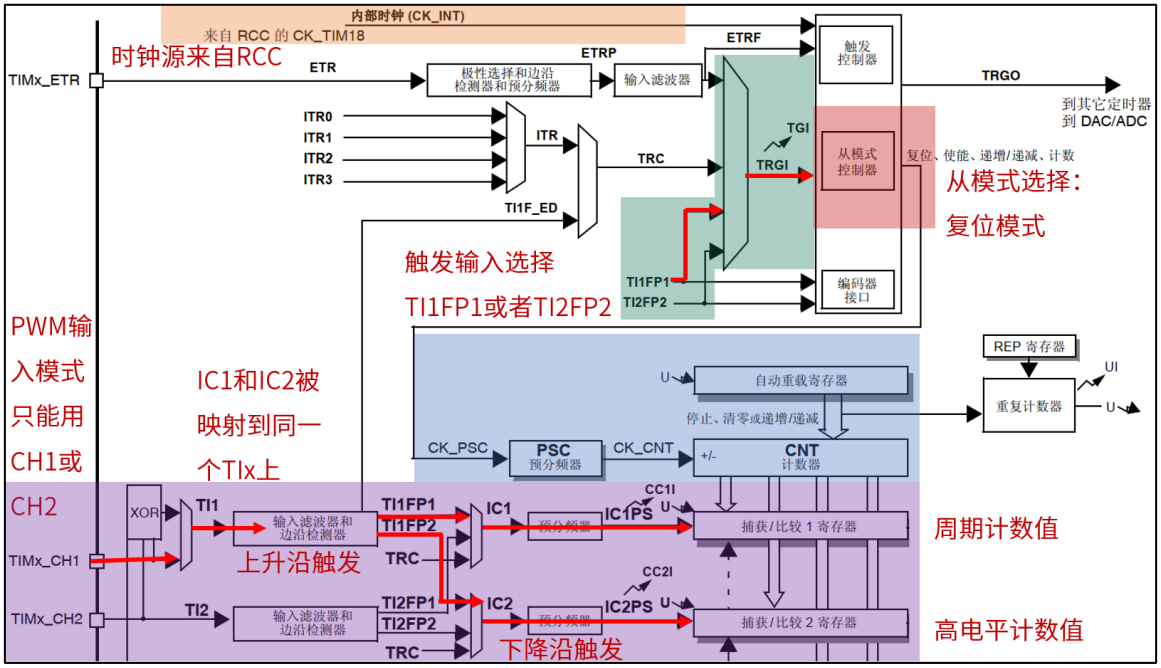


图 21.5.1 PWM 输入模式工作原理示意图

第一，确定定时器时钟源。本实验中我们使用内部时钟（CK_INT），H7 系列高级定时器挂载在 APB2 总线上，按照 sys_stm32_clock_init 函数的配置，定时器时钟频率为 2 倍 APB2 总线时钟频率，即 200MHz。计数器的计数频率确定了测量的精度。

第二，确定 PWM 输入的通道。PWM 输入模式下测量 PWM，PWM 信号输入只能从通道 1（CH1）或者通道 2（CH2）输入。

第三，确定 IC1 和 IC2 的捕获边沿。这里以通道 1（CH1）输入 PWM 为例，一般我们习惯设置 IC1 捕获边沿为上升沿捕获，IC2 捕获边沿为下降沿捕获。

第四，选择触发输入信号（TRGI）。这里也是以通道 1（CH1）输入 PWM 为例，那么我们就应该选择 TI1FP1 为触发输入信号。如果是通道 2（CH2）输入 PWM，那就选择 TI2FP2 为触发输入信号。可以看到这里并没有对应通道 3（CH3）或者通道 4（CH4）的触发输入信号，所以我们只选择通道 1 或者通道 2 作为 PWM 输入的通道。

第五，从模式选择：复位模式。复位模式的作用是：在出现所选触发输入 (TRGI) 上升沿时，重新初始化计数器并生成一个寄存器更新事件。

第六，读取一个 PWM 周期内计数器的计数个数，以及高电平期间的计数个数，再结合计数器的计数周期（即计一个数的时间），最终通过计算得到输入的 PWM 周期和占空比等参数。以通道 1（CH1）输入 PWM，设置 IC1 捕获边沿为上升沿捕获，IC2 捕获边沿为下降沿捕获为例，那么 CCR1 寄存器的值+1 就是 PWM 周期内计数器的计数个数，CCR2 寄存器的值+1 就是 PWM 高电平期间计数器的计数个数。通过这两个值就可以计算出 PWM 的周期或者占空比等参数。

再举个例子，以通道 1（CH1）输入 PWM，设置 IC1 捕获边沿为下降沿捕获，IC2 捕获边

沿为上升沿捕获为例，那么 CCR1 寄存器的值+1 依然是 PWM 周期内计数器的计数个数，但是 CCR2 寄存器的值+1 就是 PWM 低电平期间计数器的计数个数。通过这两个得到的参数依然可以计算出 PWM 的其它参数。这个大家了解一下就可以了，一般我们使用第六介绍的例子。

通过上面的描述，如果大家还不理解，下面我们结合 PWM 输入模式时序来分析一下。PWM 输入模式时序图如图 21.5.2 所示：

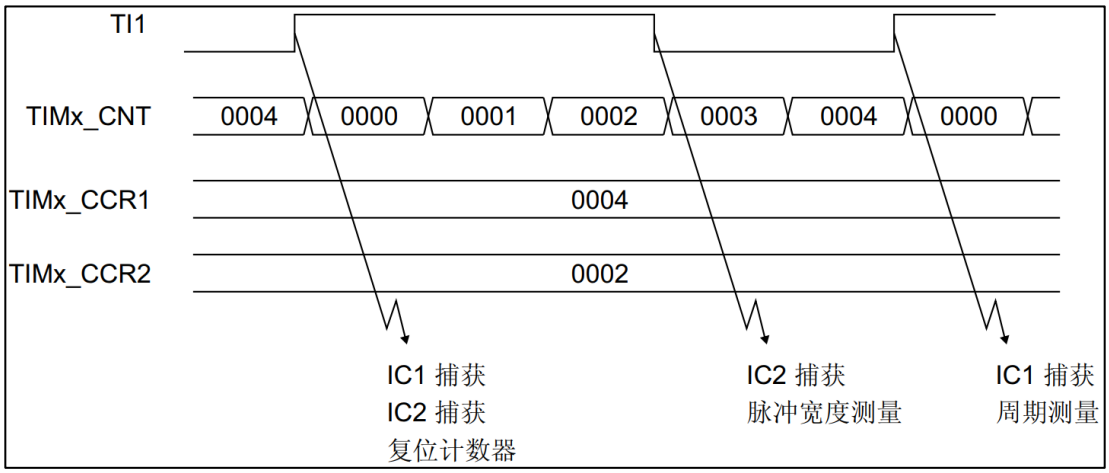


图 21.5.2 PWM 输入模式时序图

图 21.5.2 是以通道 1（CH1）输入 PWM，设置 IC1 捕获边沿为上升沿捕获，IC2 捕获边沿为下降沿捕获为例的 PWM 输入模式时序图。

从时序图可以看出，计数器的计数模式是递增计数模式。从左边开始看，当 TI1 来了上升沿时，计数器的值被复位为 0（原因是从模式选择为复位模式），IC1 和 IC2 都发生捕获事件。然后计数器的值计数到 2 的时候，IC2 发生了下降沿捕获，捕获事件会导致这时候的计数器的值被锁存到 CCR2 寄存器中，该值+1 就是高电平期间计数器的计数个数。最后计数器的值计数到 4 的时候，IC1 发生了上升沿捕获，捕获事件会导致这时候的计数器的值被锁存到 CCR1 寄存器中，该值+1 就是 PWM 周期内计数器的计数个数。

假设计数器的计数频率是 200MHz，那我们就可以计算出这个 PWM 的周期、频率和占空比等参数了。下面就以这个为例给大家计算一下。由计数器的计数频率为 200MHz，可以得到计数器计一个数的时间是 5ns（即测量的精度是 5ns）。知道了测量精度，再来计算 PWM 的周期，PWM 周期 =(4+1)*(1/200000000) = 25ns，那么 PWM 的频率就是 48MHz。占空比 = (2+1)/(4+1) =3/5（即占空比为 60%）。

21.5.1 TIM1/TIM8 寄存器

高级定时器 PWM 输入模式实验除了用到定时器的时基单元：计数器寄存器(TIMx_CNT)、预分频器寄存器(TIMx_PSC)、自动重载寄存器(TIMx_ARR) 之外。主要还用到这些寄存器：

- **从模式控制寄存器（TIMx_SMCR）**

  TIM1/TIM8 的从模式控制寄存器描述如图 21.5.1.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TS[4:3]		Res.	Res.	Res.	SMS[3]
										rw	rw				rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ETP	ECE	ETPS[1:0]		ETF[3:0]				MSM	TS[2:0]			Res.	SMS[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw

位 6:4 **TS[2:0]**：触发选择 (Trigger selection)

.....

00101：滤波后的定时器输入 1 (TI1FP1)
00110：滤波后的定时器输入 2 (TI2FP2)
00111：外部触发输入 (ETRF)
其它值：保留

位 2:0 **SMS**：从模式选择 (Slave mode selection)

选择外部信号时，触发信号 (TRGI) 的有效边沿与外部输入上所选的极性相关（请参见输入控制寄存器和控制寄存器说明）。

.....

0100：复位模式——在出现所选触发输入 (TRGI) 上升沿时，重新初始化计数器并生成一个寄存器更新事件。

图 21.5.1.1 TIMx_SMCR 寄存器

该寄存器的 SMS[2:0]位，用于从模式选择。比如本实验中我们要用复位模式，所以设置 SMS[2:0]=100。TS[2:0]位是触发选择，我们设置为 TI1FP1，即 TS[2:0]=101。

- **捕获/比较模式寄存器 1/2（TIMx_CCMR1/2）**

TIM1/TIM8 的捕获/比较模式寄存器（TIMx_CCMR1/2），该寄存器一般有 2 个：TIMx_CCMR1 和 TIMx _CCMR2。TIMx_CCMR1 控制 CH1 和 CH2，而 TIMx_CCMR2 控制 CH3 和 CH4。TIMx_CCMR1 寄存器描述如图 21.5.1.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC2M[3]	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OC1M[3]
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
							rw								rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OC2 CE	OC2M[2:0]			OC2 PE	OC2 FE	CC2S[1:0]		OC1 CE	OC1M[2:0]			OC1 PE	OC1 FE	CC1S[1:0]	
IC2F[3:0]				IC2PSC[1:0]				IC1F[3:0]				IC1PSC[1:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 21.5.1.2 TIMx_CCMR1 寄存器

该寄存器的有些位在不同模式下，功能不一样，我们现在用到输入捕获模式。

关于该寄存器的详细说明，请参考《STM32H7xx 参考手册_V3（中文版）.pdf》第 1458 页，38.4.7 节。

本实验我们通过定时器 1 通道 1 输入 PWM 信号，所以 IC1 和 IC2 都映射到 TI1 上。配置 CC1S[1:0]=01、CC2S [1:0]=10，其他位不用设置，默认为 0 即可。

- **捕获/比较使能寄存器（TIMx_ CCER）**

TIM1/TIM8 的捕获/比较使能寄存器，该寄存器控制着各个输入输出通道的开关和极性。TIMx_CCER 寄存器描述如图 21.5.1.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CC6P	CC6E	Res.	Res.	CC5P	CC5E
										rw	rw			rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CC4NP	Res.	CC4P	CC4E	CC3NP	CC3NE	CC3P	CC3E	CC2NP	CC2NE	CC2P	CC2E	CC1NP	CC1NE	CC1P	CC1E
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 21.5.1.3 TIMx_CCER 寄存器

IC1 捕获上升沿，所以 CC1P 位置 0，即捕获发生在 IC1 的上升沿。IC2 捕获下降沿，所以 CC2P 位置 1，即捕获发生在 IC1 的下降沿。设置好捕获边沿后，还需要使能这两个通道捕获，即 CC1E 和 CC2E 位置 1。

- **捕获/比较寄存器 1/2/3/4（TIMx_CCR1/2/3/4）**

捕获/比较寄存器（TIMx_ CCR1/2/3/4），该寄存器总共有 4 个，对应 4 个通道 CH1~CH4。我们使用的是通道 1，所以来看看 TIMx_ CCR1 寄存器描述如图 21.5.1.4 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CCR1[15:0]															
rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r	rw/r

位 15:0 **CCR1[15:0]**：捕获/比较 1 值 (Capture/Compare 1 value)

如果通道 CC1 配置为输出：CCR1 为要装载到有效捕获/比较 1 寄存器的值（预装载值）。如果没有通过 TIMx_CCMR1 寄存器中的 OC1PE 位来使能预装载功能，则该值立刻生效；否则只在发生更新事件时生效（拷贝到有效的捕获/比较寄存器 1）。
有效捕获/比较寄存器中包含要与计数器 TIMx_CNT 进行比较并在 OC1 输出上发出信号的值。

如果通道 CC1 配置为输入：CR1 为上一个输入捕获 1 事件 (IC1) 发生时的计数器值。只能读取 TIMx_CCR1 寄存器，无法对其进行编程。

图 21.5.1.4 TIMx_ CCR1 寄存器

本实验中，CCR1 寄存器用于获取 PWM 周期内计数器的计数个数。CCR2 寄存器用于获取 PWM 高电平期间计数器的计数个数。

- **DMA/中断使能寄存器（TIMx_DIER）**

DMA/中断使能寄存器描述如图 21.5.1.5 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	TDE	COMDE	CC4DE	CC3DE	CC2DE	CC1DE	UDE	BIE	TIE	COMIE	CC4IE	CC3IE	CC2IE	CC1IE	UIE
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 1 **CC1IE**：捕获/比较 1 中断使能 (Capture/Compare 1 interrupt enable)
0：禁止 CC1 中断
1：使能 CC1 中断

位 0 **UIE**：更新中断使能 (Update interrupt enable)
0：禁止更新中断
1：使能更新中断

图 21.5.1.5 TIMx_DIER 寄存器

该寄存器位 0（UIE）用于使能或者禁止更新中断，因为本实验我们用到更新中断，所以该位需要置 1。位 1（CC1IE）用于使能或者禁止捕获/比较 1 中断，我们用到捕获中断，所以该位需要置 1。

21.5.2 硬件设计

1. 例程功能

首先通过 TIM3_CH4(PB1)输出 PWM 波。然后把 PB1 输出的 PWM 波用杜邦线接入 PC6（定时器 8 通道 1），最后通过串口打印 PWM 波的脉宽和频率等信息。LED1 闪烁来提示程序正在运行。

2. 硬件资源

1）LED 灯
LED : LED1 - PB0
2）定时器 3 通道 4（PB1）输出 PWM 波
定时器 8 通道 1（PC6）输入 PWM 波

3. 原理图

定时器属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们把 PB1 引脚输出的 PWM 波用杜邦线接入 PC6 引脚，然后通过电脑串口上位机软件观察打印出来的信息。

21.5.3 程序设计

定时器 PWM 输入模式实验用到的 HAL 库中的驱动代码在前面实验都有介绍过了。 我们在程序解析再详细讲解应用到的函数，下面介绍一下高级定时器 PWM 输入模式的配置步骤。

高级定时器 PWM 输入模式配置步骤

1）开启 TIMx 和输入通道的 GPIO 时钟，配置该 IO 口的复用功能输入

首先开启 TIMx 的时钟，然后配置 GPIO 为复用功能输出。本实验我们默认用到定时器 8 通道 1，对应 IO 是 PC6，它们的时钟开启方法如下：

```
__HAL_RCC_TIM8_CLK_ENABLE();                /* 使能定时器 8 */
__HAL_RCC_GPIOC_CLK_ENABLE();               /* 开启 GPIOC 时钟 */
```

IO 口复用功能是通过函数 HAL_GPIO_Init 来配置的。

2）初始化 TIMx，设置 TIMx 的 ARR 和 PSC 等参数

使用定时器的输入捕获功能时，我们调用的是 HAL_TIM_IC_Init 函数来初始化定时器 ARR 和 PSC 等参数。

注意：该函数会调用：HAL_TIM_IC_MspInit 函数，但是为不跟前面的实验共用该回调函数，提高独立性，我们就直接在 atim_timx_pwmin_chy_init 函数中，使能定时器时钟和 GPIO 时钟，初始化通道对应 IO 引脚等。

3）从模式配置，IT1 触发更新

通过 HAL_TIM_SlaveConfigSynchronization 函数，配置从模式：复位模式、定时器输入触发源、边缘检测、是否滤波等。

4）设置 IC1 捕获相关参数

通过 HAL_TIM_IC_ConfigChannel 函数来设置定时器捕获通道 1 的工作方式，包括边缘检测极性、映射关系，输入滤波和输入分频等。

5）设置 IC2 捕获相关参数

通过 HAL_TIM_IC_ConfigChannel 函数来设置定时器捕获通道 2 的工作方式，包括边缘检测极性、映射关系，输入滤波和输入分频等。

6）使能定时器更新中断，开启捕获功能，配置定时器中断优先级

通过__HAL_TIM_ENABLE_IT 函数使能定时器更新中断。

通过 HAL_TIM_IC_Start_IT 函数使能定时器并开启通道 1 或者通道 2 的捕获功能，使能捕获中断。

通过 HAL_NVIC_EnableIRQ 函数使能定时器中断。

通过 HAL_NVIC_SetPriority 函数设置中断优先级。

7）编写中断服务函数

H7 系列的 TIM1 和 TIM8 有独立的输入捕获中断服务函数，分别是 TIM1_CC_IRQHandler 和 TIM8_CC_IRQHandler，其他定时器则没有，所以如果是 TIM1 和 TIM8 可以直接使用输入捕获中断服务函数来处理输入捕获中断。在使用 TIM8 的时候，如果要考虑定时器 8 溢出，可以重定义更新中断服务函数 TIM8_UP_IRQHandler。如果使用 HAL 库的中断回调机制，可以在相关中断服务函数中直接调用定时器中断公共处理函数 HAL_TIM_IRQHandler，然后我们直接重定义相关的中断回调函数来编写中断程序即可。本实验为了兼容性，我们自定义一个中断处理函数 atim_timx_pwmin_chy_process，里面包含了捕获中断和更新中断的处理，具体看源码。

21.5.3.1 程序流程图

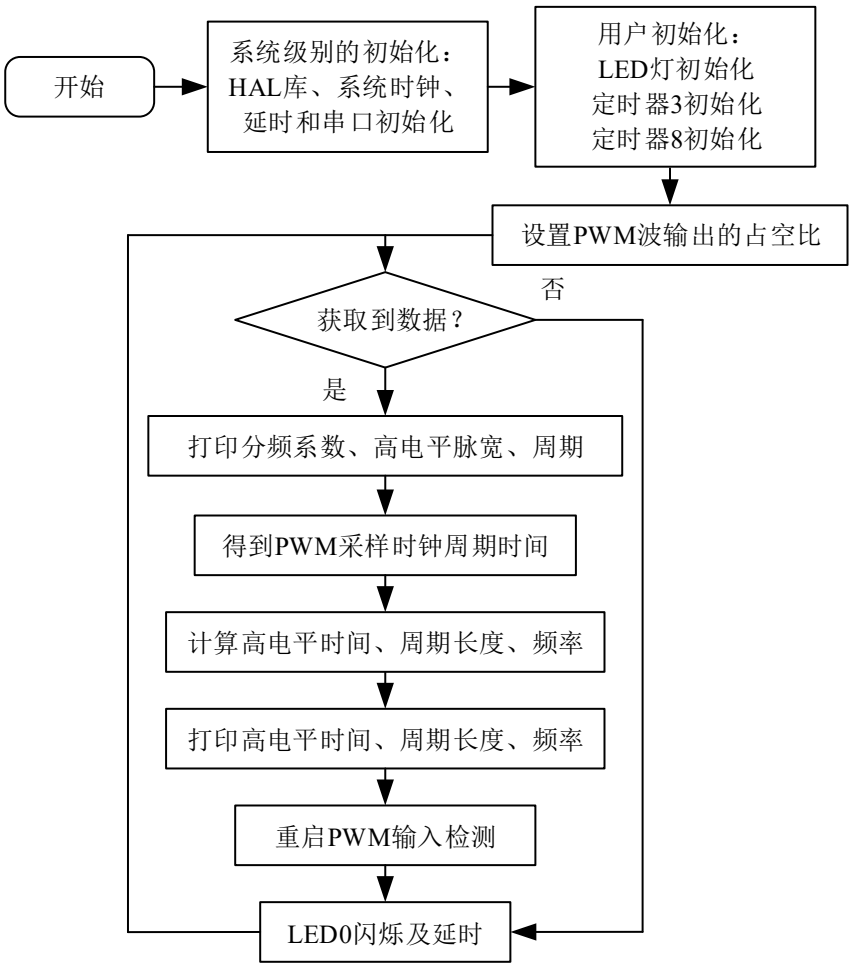


图 21.5.3.1.1 高级定时器 PWM 输入模式实验程序流程图

21.5.3.2 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。高级定时器驱动源码包括两个文件：atim.c 和 atim.h。

首先看 atim.h 头文件的几个宏定义：

```
/* TIMX PWM 输入模式 定义
 * 这里的输入捕获使用定时器 TIM1_CH1,捕获 WK_UP 按键的输入
 */
#define ATIM_TIMX_PWMIN_CHY_GPIO_PORT          GPIOC
#define ATIM_TIMX_PWMIN_CHY_GPIO_PIN           GPIO_PIN_6
#define ATIM_TIMX_PWMIN_CHY_GPIO_AF            GPIO_AF3_TIM8   /* AF 功能选择 */
#define ATIM_TIMX_PWMIN_CHY_GPIO_CLK_ENABLE()
               do{ __HAL_RCC_GPIOC_CLK_ENABLE(); }while(0)    /* PC 口时钟使能 */

#define ATIM_TIMX_PWMIN                  TIM8
#define ATIM_TIMX_PWMIN_IRQn             TIM8_UP_TIM13_IRQn
#define ATIM_TIMX_PWMIN_IRQHandler       TIM1_UP_TIM13_IRQHandler
#define ATIM_TIMX_PWMIN_CHY              TIM_CHANNEL_1    /* 通道 Y,  1<= Y <=2*/
#define ATIM_TIMX_PWMIN_CHY_CLK_ENABLE()
              do{ __HAL_RCC_TIM8_CLK_ENABLE();}while(0)/* TIMX 时钟使能 */

/* TIM1 / TIM8 有独立的捕获中断服务函数,需要单独定义,对于 TIM2~5/TIM12/TIM15 等,
     则不需要以下定义 */
#define ATIM_TIMX_PWMIN_CC_IRQn                 TIM8_CC_IRQn
```

```
#define ATIM_TIMX_PWMIN_CC_IRQHandler           TIM8_CC_IRQHandler
```

可以把上面的宏定义分成三部分，第一部分包括是定时器 8 通道 1 对应的 IO 口的宏定义，第二部分则是定时器 8 的相应宏定义，另外针对 TIM1/ TIM8 有独立的捕获中断服务函数，需要单独定义。

下面看 atim.c 的程序，首先是高级定时器 PWM 输入模式初始化函数，其定义如下：

```
/**
 * @brief  定时器 TIMX 通道 Y PWM 输入模式 初始化函数
 * @note
 *       高级定时器的时钟来自 APB2,当 D2PPRE1≥2 分频的时候
 *       高级定时器的时钟为 APB2 时钟的 2 倍, 而 APB2 为 100M, 所以定时器时钟 = 200Mhz
 *       定时器溢出时间计算方法: Tout = ((arr + 1) * (psc + 1)) / Ft us.
 *       Ft=定时器工作频率,单位:Mhz
 *
 * @param  无
 * @retval 无
 */
void atim_timx_pwmin_chy_init(void)
{
    GPIO_InitTypeDef gpio_init_struct = {0};
    TIM_SlaveConfigTypeDef slave_config = {0};
    TIM_IC_InitTypeDef tim_ic_pwmin_chy = {0};

    ATIM_TIMX_PWMIN_CHY_CLK_ENABLE();
    ATIM_TIMX_PWMIN_CHY_GPIO_CLK_ENABLE();      /* TIMX 通道 IO 口时钟使能 */

    gpio_init_struct.Pin = ATIM_TIMX_PWMIN_CHY_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;
    gpio_init_struct.Pull = GPIO_PULLDOWN;
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH ;
    gpio_init_struct.Alternate = ATIM_TIMX_PWMIN_CHY_GPIO_AF;
    HAL_GPIO_Init(ATIM_TIMX_PWMIN_CHY_GPIO_PORT, &gpio_init_struct);

    g_timx_pwmin_chy_handle.Instance = ATIM_TIMX_PWMIN;     /* 定时器 x */
    g_timx_pwmin_chy_handle.Init.Prescaler = 0;             /* 定时器预分频系数 */
    g_timx_pwmin_chy_handle.Init.CounterMode = TIM_COUNTERMODE_UP; /* 递增计数 */
    g_timx_pwmin_chy_handle.Init.Period = 65535;            /* 自动重装载值 */
    HAL_TIM_IC_Init(&g_timx_pwmin_chy_handle);

    /* 从模式配置,IT1 触发更新 */
    slave_config.SlaveMode = TIM_SLAVEMODE_RESET;  /* 从模式: 复位模式 */
    slave_config.InputTrigger = TIM_TS_TI1FP1;     /* 定时器输入触发源: TI1FP1 */
    slave_config.TriggerPolarity = TIM_INPUTCHANNELPOLARITY_RISING;/*上升沿检测*/
    slave_config.TriggerFilter = 0;                                /* 不滤波 */
    HAL_TIM_SlaveConfigSynchronization(&g_timx_pwmin_chy_handle,&slave_config);

    /* IC1 捕获: 上升沿触发 TI1FP1 */
    tim_ic_pwmin_chy.ICPolarity=TIM_INPUTCHANNELPOLARITY_RISING;/* 上升沿检测 */
    tim_ic_pwmin_chy.ICSelection=TIM_ICSELECTION_DIRECTTI;/* IC1 映射到 TI1 上 */
    tim_ic_pwmin_chy.ICPrescaler = TIM_ICPSC_DIV1;          /* 不分频 */
    tim_ic_pwmin_chy.ICFilter = 0;                          /* 不滤波 */
    HAL_TIM_IC_ConfigChannel(&g_timx_pwmin_chy_handle, &tim_ic_pwmin_chy,
                             TIM_CHANNEL_1);

    /* IC2 捕获: 下降沿触发 TI1FP2 */
    tim_ic_pwmin_chy.ICPolarity = TIM_INPUTCHANNELPOLARITY_FALLING;/*下降沿检测*/
    tim_ic_pwmin_chy.ICSelection=TIM_ICSELECTION_INDIRECTTI;/*IC2 映射到 TI1 上*/
    HAL_TIM_IC_ConfigChannel(&g_timx_pwmin_chy_handle, &tim_ic_pwmin_chy,
                             TIM_CHANNEL_2);
   /* 设置中断优先级, 抢占优先级 1, 子优先级 3 */
    HAL_NVIC_SetPriority(ATIM_TIMX_PWMIN_IRQn, 1, 3);
    HAL_NVIC_EnableIRQ(ATIM_TIMX_PWMIN_IRQn);           /* 开启 ITMx 中断 */
```

```
    /* TIM1/TIM8 有独立的输入捕获中断服务函数 */
    if (ATIM_TIMX_PWMIN == TIM1 || ATIM_TIMX_PWMIN == TIM8)
    {
        /* 设置中断优先级，抢占优先级 1，子优先级 3 */
        HAL_NVIC_SetPriority(ATIM_TIMX_PWMIN_CC_IRQn, 1, 3);
        HAL_NVIC_EnableIRQ(ATIM_TIMX_PWMIN_CC_IRQn);  /* 开启 ITMx 中断 */
    }

    __HAL_TIM_ENABLE_IT(&g_timx_pwmin_chy_handle, TIM_IT_UPDATE);
    HAL_TIM_IC_Start_IT(&g_timx_pwmin_chy_handle, TIM_CHANNEL_1);
    HAL_TIM_IC_Start_IT(&g_timx_pwmin_chy_handle, TIM_CHANNEL_2);
}
```

在 atim_timx_pwmin_chy_init 函数中，没有使用 HAL 库的 MSP 回调，而是把相关的初始化都写到该函数里面。

第一部分，使能定时器和相关通道对应的 GPIO 时钟，以及初始化相关 IO 引脚。

第二部分，通过 HAL_TIM_IC_Init 函数初始化定时器的 ARR 和 PSC 等参数。

第三部分，通过 HAL_TIM_SlaveConfigSynchronization 函数配置从模式，复位模式等。

第四部分，通过 HAL_TIM_IC_ConfigChannel 函数分别配置 IC1 和 IC2。

第五部分，配置 NVIC，如使能定时器中断，配置抢占优先级和响应优先级。

最后，通过调用 HAL_TIM_IC_Start_IT 函数和__HAL_TIM_ENABLE_IT 函数宏使能捕获中断和更新中断，并且使能定时器。

为了方便，我们定义了重新启动捕获函数，其定义如下：

```
/**
 * @brief       定时器 TIMX PWM 输入模式 重新启动捕获
 * @param       无
 * @retval      无
 */
void atim_timx_pwmin_chy_restart(void)
{
    sys_intx_disable();                                 /* 关闭中断 */
    g_timxchy_pwmin_sta = 0;                            /* 清零状态,重新开始检测 */
    g_timxchy_pwmin_psc = 0;                            /* 分频系数清零 */
    /* 以最大的计数频率采集,以得到最好的精度 */
    __HAL_TIM_SET_PRESCALER(&g_timx_pwmin_chy_handle, 0);
    __HAL_TIM_SET_COUNTER(&g_timx_pwmin_chy_handle, 0);         /* 计数器清零 */

    __HAL_TIM_ENABLE_IT(&g_timx_pwmin_chy_handle, TIM_IT_CC1);/* 使能捕获中断 */
    __HAL_TIM_ENABLE_IT(&g_timx_pwmin_chy_handle, TIM_IT_UPDATE);/*使能更新中断*/
    __HAL_TIM_ENABLE(&g_timx_pwmin_chy_handle);         /* 使能定时器 TIMX */

    /* 清零捕获/比较 1 中断标志 */
    __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC1);
    /* 清零捕获/比较 2 中断标志 */
    __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC2);
    /* 清零更新中断标志 */
    __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_UPDATE);
    sys_intx_enable();                                  /* 打开中断 */
}
```

该函数首先关闭所有中断，然后把一些状态标志位清零、设置定时器预分频系数、计数器值、使能相关中断、以及清除相关中断标志位，最后才允许被中断。

最后要介绍的是中断服务函数，在定时器 8 的输入捕获中断服务函数 TIM8_CC_IRQHandler 和更新中断服务函数 TIM8_UP_IRQHandler 里面都是直接调用 atim_timx_pwmin_chy_process 函数。输入捕获中断服务函数和更新中断服务函数都是用到宏定义的，这三个函数定义如下：

```
/**
 * @brief       定时器 TIMX 更新/溢出 中断服务函数
 * @note        TIM1/TIM8 的这个函数仅用于更新/溢出中断服务,捕获在另外一个函数!
 *              其他普通定时器则更新/溢出/捕获,都在这个函数里面处理!
```

```
 * @param       无
 * @retval      无
 */
void ATIM_TIMX_PWMIN_IRQHandler(void)
{
    atim_timx_pwmin_chy_process();
}

/**
 * @brief       定时器 TIMX 输入捕获 中断服务函数
 * @note        仅 TIM1/TIM8 有这个函数,其他普通定时器没有这个中断服务函数!
 * @param       无
 * @retval      无
 */
void ATIM_TIMX_PWMIN_CC_IRQHandler(void)
{
    atim_timx_pwmin_chy_process();
}

/**
 * @brief       定时器 TIMX 通道 Y PWM 输入模式 中断处理函数
 * @note
 *              因为 TIM1/TIM8 等有多个中断服务函数,而 TIM2~5/TIM12/TIM15 等
 *              普通定时器只有 1 个中断服务函数,为了更好的兼容,我们对中断处理统一放到
 *              atim_timx_pwin_chy_process 函数里面进行处理
 *
 * @param       无
 * @retval      无
 */
static void atim_timx_pwmin_chy_process(void)
{
    static uint8_t sflag = 0;                /* 启动 PWMIN 输入检测标志 */

    if (g_timxchy_pwmin_sta)
    {
        g_timxchy_pwmin_psc = 0;
       /* 清零捕获/比较 1 中断标志 */
       __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC1);
       /* 清零捕获/比较 2 中断标志 */
       __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC2);
       /* 清零更新中断标志 */
       __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_UPDATE);
       __HAL_TIM_SET_COUNTER(&g_timx_pwmin_chy_handle, 0);       /* 计数器清零 */
       return ;
    }

    /* 如果发生了更新中断 */
    if (__HAL_TIM_GET_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_UPDATE))
    {
        /* 清除更新中断标记 */
        __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_UPDATE);
        /* 没有发生周期捕获中断,且捕获未完成 */
        if (__HAL_TIM_GET_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC1) == 0)
        {
            sflag = 0;
            if (g_timxchy_pwmin_psc == 0)    /* 从 0 到 1 */
            {
                g_timxchy_pwmin_psc ++;
            }
            else
            {
                if (g_timxchy_pwmin_psc == 65535) /* 已经最大了,可能是无输入状态 */
                {
```

```
                    g_timxchy_pwmin_psc = 0;       /* 重新恢复不分频 */
                }
                else if (g_timxchy_pwmin_psc > 32767)/* 不能倍增了 */
                {
                    g_timxchy_pwmin_psc = 65535;     /* 直接等于最大分频系数 */
                }
                else
                {
                    g_timxchy_pwmin_psc += g_timxchy_pwmin_psc; /* 倍增 */
                }
            }
            __HAL_TIM_SET_PRESCALER(&g_timx_pwmin_chy_handle,
                                    g_timxchy_pwmin_psc); /* 设置定时器预分频系数 */
            __HAL_TIM_SET_COUNTER(&g_timx_pwmin_chy_handle, 0);  /* 计数器清零 */
            /* 清零捕获/比较 1 中断标志 */
            __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC1);
            /* 清零捕获/比较 2 中断标志 */
            __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC2);
            /* 清零更新中断标志 */
            __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_UPDATE);
            return ;
        }
    }
    if (sflag == 0)    /* 第一次采集到捕获中断 */
    {
        /* 检测到了第一次周期捕获中断 */
        if (__HAL_TIM_GET_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC1))
        {
            sflag = 1;/* 标记第一次周期已经捕获, 第二次周期捕获可以开始了 */
        }
        /* 清零捕获/比较 1 中断标志 */
        __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC1);
        /* 清零捕获/比较 2 中断标志 */
        __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC2);
        /* 清零更新中断标志 */
        __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_UPDATE);
        return ;                    /* 完成此次操作 */
    }

    if (g_timxchy_pwmin_sta == 0)/* 还没有成功捕获 */
    {
        /* 检测到了周期捕获中断 */
        if (__HAL_TIM_GET_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC1))
        {
            g_timxchy_pwmin_hval = HAL_TIM_ReadCapturedValue(
              &g_timx_pwmin_chy_handle, TIM_CHANNEL_2) + 1; /* 高定平脉宽捕获值 */
            g_timxchy_pwmin_cval = HAL_TIM_ReadCapturedValue(
              &g_timx_pwmin_chy_handle, TIM_CHANNEL_1) + 1; /* 周期捕获值 */

            /* 高电平脉宽必定小于周期长度 */
            if (g_timxchy_pwmin_hval < g_timxchy_pwmin_cval)
            {
               g_timxchy_pwmin_sta = 1;       /* 标记捕获成功 */
               g_timxchy_pwmin_psc=ATIM_TIMX_PWMIN->PSC;/* 获取 PWM 输入分频系数 */
               if (g_timxchy_pwmin_psc == 0) /* 分频系数为 0 的时候, 修正读取数据 */
                {
                    g_timxchy_pwmin_hval++;       /* 修正系数为 1, 加 1 */
                    g_timxchy_pwmin_cval++;       /* 修正系数为 1, 加 1 */
                }
                sflag = 0;
                /* 每次捕获 PWM 输入成功后, 停止捕获,避免频繁中断影响系统正常代码运行 */
                ATIM_TIMX_PWMIN->CR1   &= ~(1 << 0);         /* 关闭定时器 TIMX */
```

```
                /* 关闭通道 1 捕获中断 */
                __HAL_TIM_DISABLE_IT(&g_timx_pwmin_chy_handle, TIM_IT_CC1);
                /* 关闭通道 2 捕获中断 */
                __HAL_TIM_DISABLE_IT(&g_timx_pwmin_chy_handle, TIM_IT_CC2);
                /* 关闭更新中断 */
                __HAL_TIM_DISABLE_IT(&g_timx_pwmin_chy_handle, TIM_IT_UPDATE);
                /* 清零捕获/比较 1 中断标志 */
                __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC1);
                /* 清零捕获/比较 2 中断标志 */
                __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC2);
                /* 清零更新中断标志 */
                __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle,TIM_FLAG_UPDATE);
            }
            else
            {
                atim_timx_pwmin_chy_restart();
            }
        }
    }
    /* 清零捕获/比较 1 中断标志 */
    __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC1);
    /* 清零捕获/比较 2 中断标志 */
    __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_CC2);
    /* 清零更新中断标志 */
    __HAL_TIM_CLEAR_FLAG(&g_timx_pwmin_chy_handle, TIM_FLAG_UPDATE);

}
```

atim_timx_pwmin_chy_process 函数包含了捕获中断程序和更新中断程序的处理。如果发生了更新中断（即定时器溢出），证明超出定时器量程，这里会加大预分频系数，以得到更大的量程。量程变大了，那么测量的精度就会降低，所谓鱼和熊掌不可兼得。代码中的“if (sflag == 0) /* 第一次采集到捕获中断 */”这个程序段，表示第一次采集到捕获中断。这时候相当于第一次捕获到上升沿，我们只是把 sflag 标志位置 1，然后清除所有中断标志位，等待下次的捕获中断发生。如果再次发生捕获中断，就会来到“if (g_timxchy_pwmin_sta == 0) /* 还没有成功捕获 */”程序段。通过 HAL_TIM_ReadCapturedValue 函数获取 CCR1 和 CCR2 寄存器的值，把这个获取到的寄存器值+1 才是对应的计数器计数个数。如果预分频系数为 0 的时候，还要把这两个寄存器的值再+1，这样计算的结果更准确。其它的代码细节请大家自行查看源码，有详细的注释。

下面介绍一下待测试的 PWM 怎么得到。在实验 8-2 通用定时器 PWM 输出实验我们已经编写了 PWM 输出的程序，这些宏定义如下：

```
/* TIMX PWM 输出定义 */
#define GTIM_TIMX_PWM_CHY_GPIO_PORT          GPIOB
#define GTIM_TIMX_PWM_CHY_GPIO_PIN           GPIO_PIN_1
#define GTIM_TIMX_PWM_CHY_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)   /* PB 口时钟使能 */
#define GTIM_TIMX_PWM_CHY_GPIO_AF            GPIO_AF2_TIM3
/* 端口复用到 TIM14 */

#define GTIM_TIMX_PWM                  TIM3            /* TIM3 */
#define GTIM_TIMX_PWM_CHY              TIM_CHANNEL_4   /* 通道 Y,  1<= Y <=4 */
#define GTIM_TIMX_PWM_CHY_CCRX         TIM3->CCR4      /* 通道 Y 的输出比较寄存器 */
#define GTIM_TIMX_PWM_CHY_CLK_ENABLE()
                do{ __HAL_RCC_TIM3_CLK_ENABLE(); }while(0)   /* TIM3 时钟使能 */
```

添加好这些宏定义后，然后在 main 函数调用 PWM 输出初始化函数就可以使得 PB1 输出 PWM 波。然后我们用杜邦线把 PB1 和 PC6 连接起来。这样 PB1 输出的 PWM 就可以输入到 PC6（定时器 8 通道 1）进行测量。

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t t = 0;
```

```
    double ht, ct, f, tpsc;
    sys_cache_enable();                             /* 打开 L1-Cache */
    HAL_Init();                                     /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);             /* 设置时钟, 400Mhz */
    delay_init(400);                                /* 延时初始化 */
    usart_init(115200);                             /* 串口初始化为 115200 */
    led_init();                                     /* 初始化 LED */
    gtim_timx_pwm_chy_init(10 - 1, 200 - 1);        /* 10Mhz 的计数频率, 100Khz PWM */
    atim_timx_pwmin_chy_init();                     /* 初始化 PWM 输入捕获 */
    GTIM_TIMX_PWM_CHY_CCRX = 2;                     /* 低电平宽度 2,高电平宽度 8 */

    while (1)
    {
        delay_ms(10);
        t++;

        if (t >= 20)    /* 每 200ms 输出一次结果,并闪烁 LED0,提示程序运行 */
        {
            if (g_timxchy_pwmin_sta)    /* 捕获了一次数据 */
            {
                printf("\r\n");         /* 输出空,另起一行 */
                printf("PWM PSC :%d\r\n",g_timxchy_pwmin_psc);/* 打印分频系数 */
                printf("PWM Hight:%d\r\n",g_timxchy_pwmin_hval);/*打印高电平脉宽*/
                printf("PWM Cycle:%d\r\n", g_timxchy_pwmin_cval);/* 打印周期 */
                /* 得到 PWM 采样时钟周期时间 */
                tpsc = ((double)g_timxchy_pwmin_psc + 1)/200;
                ht = g_timxchy_pwmin_hval * tpsc;       /* 计算高电平时间 */
                ct = g_timxchy_pwmin_cval * tpsc;       /* 计算周期长度 */
                f = (1 / ct) * 1000000;                 /* 计算频率 */
                printf("PWM Hight time:%.3fus\r\n", ht);/* 打印高电平脉宽长度 */
                printf("PWM Cycle time:%.3fus\r\n", ct);/* 打印周期时间长度 */
                printf("PWM Frequency :%.3fHz\r\n", f); /* 打印频率 */

                atim_timx_pwmin_chy_restart(); /* 重启 PWM 输入检测 */
            }

            LED1_TOGGLE();  /* LED1(GREEN)闪烁 */
            t = 0;
        }
    }
}
```

先看 gtim_timx_pwm_chy_init(10 - 1, 200 - 1)这个语句，这两个形参分别设置自动重载寄存器的值为 9，以及定时器预分频寄存器的值为 199。先看预分频系数，我们设置为 200 分频，定时器 3 的时钟频率是 2 倍的 APB1 总线时钟频率，即 200MHz，可以得到计数器的计数频率是 1MHz，即 1us 计数一次。再到自动重载寄存器的值为 9 决定的是 PWM 波的频率（周期），可以得到 PWM 的周期为 10*1us=10us。然后通过 GTIM_TIMX_PWM_CHY_CCRX=2 这个语句设置占空比，低电平宽度 2，总的周期宽度是 10，所以高电平宽度 8。即产生的 PWM 波周期为 10us，频率为 100KHz，占空比为 80%。下载验证的时候验证一下捕获到的与输出的是否一致。

atim_timx_pwmin_chy_init 这个语句，就初始化 PWM 输入捕获。然后在无限循环中每 200ms 判断是否 g_timxchy_pwmin_sta 标志变量，是否捕获到数据，捕获到就打印和计数相关信息。

下面我们下载到开发板验证一下。

21.5.4 下载验证

下载代码后，可以看到 LED1 在闪烁，说明程序已经正常在跑了，我们再打开串口调试助手，选择对应的串口端口。然后用杜邦线把 P3 排针的 PB1 引脚连接到 P2 排针的 PC6 引脚，

就可以看到串口助手不断打印 PWM 波的信息，如图 21.5.4.1 所示：

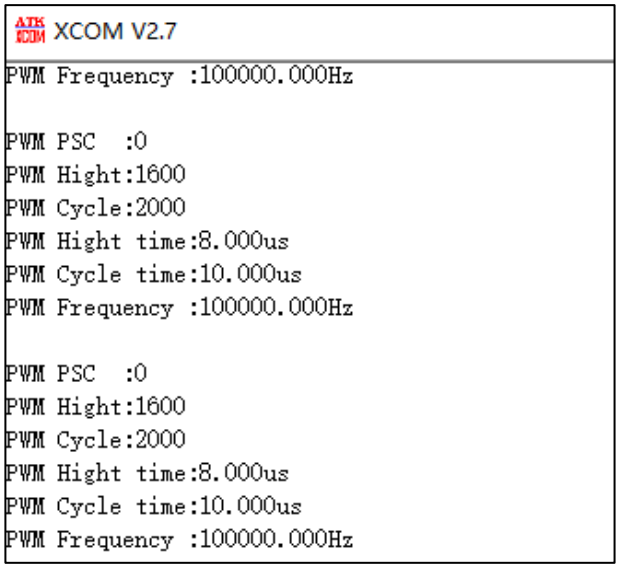


图 21.5.4.1 打印高电平脉冲次数

可以看到打印出来的 PWM 波信息为：周期是 10us，频率是 100KHz，占空比是 80%，和我们的预想结果一样。

大家可以通过 gtim_timx_pwm_chy_init 函数的形参设置其他参数的 PWM 波，以及 GTIM_TIMX_PWM_CHY_CCRX 设置占空比。这里的测试的 PWM 波有一定的范围，不是全范围的 PWM 都可以进行准确的测试，大家可以进行验证。

第二十二章 电容触摸按键实验

上一章，我们介绍了 STM32H743 的输入捕获功能及其使用。这一章，我们将向大家介绍如何通过输入捕获功能，来做一个电容触摸按键。在本章中，我们将用 TIM2 的通道 1（ PA5）来做输入捕获，并实现一个简单的电容触摸按键，通过该按键控制 DS1 的亮灭。从本章分为如 下几个部分：

22.1 电容触摸按键简介
22.2 硬件设计
22.3 程序设计
22.4 下载验证

22.1 电容触摸按键简介

触摸按键相对于传统的机械按键有寿命长、占用空间少、易于操作等诸多优点。大家看看如今的手机，触摸屏、触摸按键大行其道，而传统的机械按键，正在逐步从手机上面消失。本章，我们将给大家介绍一种简单的触摸按键：电容式触摸按键。我们将利用阿波罗 STM32 开发板上的触摸按键（ TPAD），来实现对 DS1 的亮灭控制。这里 TPAD 其实就是阿波罗 STM32 开发板上的一小块覆铜区域，实现原理如图 22.1.1 所示：

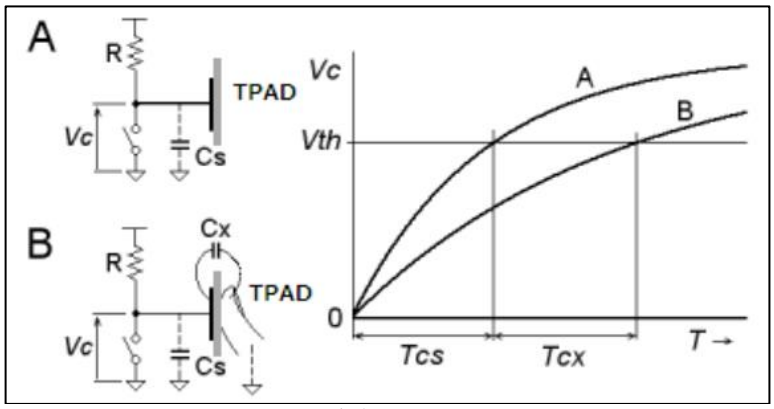


图 22.1.1 电容触摸按键原理

这里我们使用的是检测电容充放电时间的方法来判断是否有触摸，图中 R 是外接的电容充电电阻， Cs 是没有触摸按下时 TPAD 与 PCB 之间的杂散电容。而 Cx 则是有手指按下的时候，手指与 TPAD 之间形成的电容。图中的开关是电容放电开关（实际使用时，由 STM32H743 的 IO 代替）。

先用开关将 Cs（或 Cs+Cx）上的电放尽，然后断开开关，让 R 给 Cs（或 Cs+Cx）充电，当没有手指触摸的时候， Cs 的充电曲线如图中的 A 曲线。而当有手指触摸的时候，手指和 TPAD 之间引入了新的电容 Cx，此时 Cs+Cx 的充电曲线如图中的 B 曲线。从上图可以看出，A、 B 两种情况下， Vc 达到 Vth 的时间分别为 Tcs 和 Tcs+Tcx。其中，除了 Cs 和 Cx 我们需要计算，其他都是已知的，根据电容充放电公式：

$$V_C = V0 \times (1 - e^{(-\frac{t}{RC})})$$

其中 Vc 为电容电压，V0 为充电电压，R 为充电电阻，C 为电容容值，e 为自然底数， t 为充电时间。根据这个公式，我们就可以计算出 Cs 和 Cx。利用这个公式，我们还可以把阿波罗开发板作为一个简单的电容计，直接可以测电容容量了，有兴趣的朋友可以捣鼓下。

在本章中，其实我们只要能够区分 Tcs 和 Tcs+Tcx，就已经可以实现触摸检测了，当充电时间在 Tcs 附近，就可以认为没有触摸，而当充电时间大于 Tcs+ Tcx 时，就认为有触摸按下（ Tcx 为检测阀值）。

本章，我们使用 PA5(TIM2_CH1)来检测 TPAD 是否有触摸，在每次检测之前，我们先

配置 PA5 为推挽输出，将电容 Cs（或 Cs+Cx）放电，然后配置 PA5 为复用功能浮空输入，利用外部上拉电阻给电容 Cs(Cs+Cx)充电，同时开启 TIM2_CH1 的输入捕获，检测上升沿，当检测到上升沿的时候，就认为电容充电完成了，完成一次捕获检测。

在 MCU 每次复位重启的时候，我们执行一次捕获检测（可以认为没触摸），记录此时的值，记为 tpad_default_val，作为判断的依据。在后续的捕获检测，我们就通过与 tpad_default_val 的对比，来判断是不是有触摸发生。

关于输入捕获的配置，在上一章我们已经有详细介绍了，这里我们就不再介绍。至此，电容触摸按键的原理介绍完毕。

22.2 硬件设计

1. 例程功能

1，利用开发板板载的电容触摸按键(右下角白色 LOGO，即 TPAD)，通过 TIM2_CH1（PA5）对电容触摸按键的检测，实现对 DS1 的控制, 下载本代码后，我们通过按压开发板右下角的 TPAD 按钮，就可以控制 DS1 的亮灭了。

2，LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯

LED0：LED0 - PB1

LED1：LED1 - PB0

2）TPAD 电容触摸按键（右下角 LOGO，即 TPAD，连接在 PA5）

3）定时器 2（TIM2），TIM2 的通道 1（TIM2_CH1，连接在 PA5 上面）

3. 原理图

我们需要通过 TIM2_CH1（PA5）采集 TPAD 的信号，所以本实验需要用跳线帽短接多功能端口（P10）的 TPAD 和 ADC，以实现 TPAD 连接到 PA5，连接如图 22.2.1 所示：

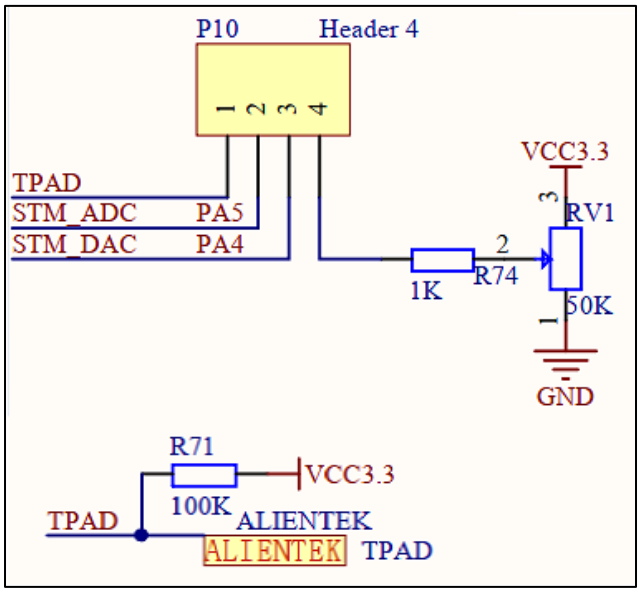


图 22.2.1 TPAD 与 STM32H743 连接原理图

硬件设置（用跳线帽短接多功能端口的 ADC 和 TPAD 即可）好之后，下面我们开始软件设计。

22.3 程序设计

我们在基本定时器一节已经学习过定时器的输入捕获功能，这里我们可以类似地，用定时器 2 来实现对 TPAD 引脚上的电平状态的捕获功能。本实验用到的 HAL 库驱动请回顾基本定时器实验的介绍。下面直接从程序流程图开始介绍。

22.3.1 程序流程图

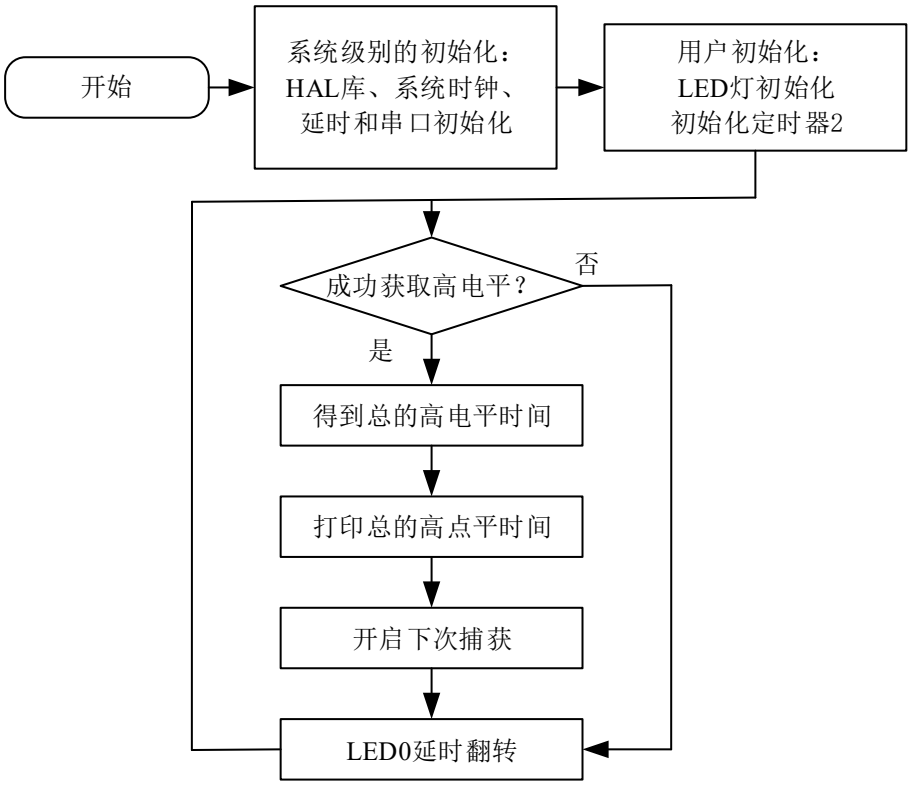


图 22.3.1.1 电容触摸按键实验程序流程图

22.3.2 程序解析

TPAD 可以看作《实验 8-3 通用定时器输入捕获实验》 的一个应用案例，相关的 HAL 库函数在定时器章节我们已经介绍过了，所以这里就不重复介绍了，大家回过头复习一下即可。

1. TPAD 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码，TPAD 的驱动主要包括两个文件：tpad.c 和 tpad.h。

首先看 tpad.h 头文件的几个宏定义：

```
/* TPAD 引脚 及 定时器 定义 */
/* 我们使用定时器的输入捕获功能，对 TPAD 进行检测
 * 这里的输入捕获使用定时器 TIM2_CH1，捕获 TPAD 按键的输入
 */
#define TPAD_GPIO_PORT          GPIOA
#define TPAD_GPIO_PIN           GPIO_PIN_5
#define TPAD_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)     /* PA 口时钟使能 */
#define TPAD_GPIO_AF        GPIO_AF1_TIM2       /* 端口复用为 TIM2 通道 1 */

#define TPAD_TIMX_CAP           TIM2
#define TPAD_TIMX_CAP_CHY       TIM_CHANNEL_1 /* 通道 Y，  1<= Y <=4 */
#define TPAD_TIMX_CAP_CHY_CCRX  TIM2->CCR1    /* 通道 Y 的捕获/比较寄存器 */
```

```
#define TPAD_TIMX_CAP_CHY_CLK_ENABLE()
    do{ __HAL_RCC_TIM2_CLK_ENABLE(); }while(0)       /* TIM2 时钟使能 */
```

PA5 是定时器 2 的 PWM 通道 1，如果我们使用其他定时器和它们对应的捕获通道的其它 IO，我们只需要修改上面的宏即可。

利用前面描述的触摸按键的原理上电时检测 TPAD 上的电容的充放电时间，并以此为基准，每次需要重新检测 TPAD 时，通过比较充放电的时长来检测当前是否有按下，所以我们需要用定时器的输入捕获来监测 TPAD 上低电平的时间编写 tpad_timx_cap_init()函数如下：

```
/**
 * @brief       触摸按键输入捕获设置
 * @param       arr     : 自动重装值
 * @param       psc     : 时钟预分频数
 * @retval      无
 */
static void tpad_timx_cap_init(uint16_t arr, uint16_t psc)
{
    GPIO_InitTypeDef gpio_init_struct;
    TPAD_GPIO_CLK_ENABLE();                         /* TPAD 引脚 时钟使能 */
    TPAD_TIMX_CAP_CHY_CLK_ENABLE();                 /* 定时器 时钟使能 */

    gpio_init_struct.Pin = TPAD_GPIO_PIN;           /* 输入捕获的 GPIO 口 */
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;        /* 复用推挽 */
    gpio_init_struct.Pull = GPIO_NOPULL;            /* 不带上下拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;      /* 概述 */
    gpio_init_struct.Alternate = TPAD_GPIO_AF;          /* PA5 复用为 TIM2_CH1 */
    HAL_GPIO_Init(TPAD_GPIO_PORT, &gpio_init_struct);   /* TPAD 引脚复用 */

    g_timx_cap_chy_handle.Instance = TPAD_TIMX_CAP;     /* 定时器 2 */
    g_timx_cap_chy_handle.Init.Prescaler = psc;         /* 定时器分频 */
    g_timx_cap_chy_handle.Init.CounterMode = TIM_COUNTERMODE_UP;/*向上计数模式 */
    g_timx_cap_chy_handle.Init.Period = arr;            /* 自动重装载值 */
    /* 时钟分频因子 */
    g_timx_cap_chy_handle.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    HAL_TIM_IC_Init(&g_timx_cap_chy_handle);

    g_timx_ic_cap_chy_handle.ICPolarity = TIM_ICPOLARITY_RISING; /* 上升沿捕获 */
    /* 映射到 TI1 上 */
    g_timx_ic_cap_chy_handle.ICSelection = TIM_ICSELECTION_DIRECTTI;

    g_timx_ic_cap_chy_handle.ICPrescaler=TIM_ICPSC_DIV1;/* 配置输入分频，不分频 */
    g_timx_ic_cap_chy_handle.ICFilter = 0;      /* 配置输入滤波器，不滤波*/
    HAL_TIM_IC_ConfigChannel(&g_timx_cap_chy_handle, &g_timx_ic_cap_chy_handle,
                             TPAD_TIMX_CAP_CHY);        /* 配置 TIM2 通道 1 */
    /* 使能输入捕获和定时器 */
    HAL_TIM_IC_Start(&g_timx_cap_chy_handle, TPAD_TIMX_CAP_CHY);
}
```

这和我们《实验 8-3 通用定时器输入捕获实验》的代码基本一样，原理就不重复解释了，接下来我们通过控制变量法，每次先给 TPAD 放电（STM32 输出低电平）相同时间，然后释放，监测 VCC 每次给 TPAD 的充电时间，由此可以得到一个充电时间，操作的代码如下：

```
/**
 * @brief       复位 TPAD
 * @note        我们将 TPAD 按键看做是一个电容，当手指按下/不按下时容值有变化
 *              该函数将 GPIO 设置成推挽输出，然后输出 0，进行放电，然后再设置
 *              GPIO 为浮空输入，等待外部大电阻慢慢充电
 * @param       无
 * @retval      无
 */
static void tpad_reset(void)
{
    GPIO_InitTypeDef gpio_init_struct;
```

```
    gpio_init_struct.Pin = TPAD_GPIO_PIN;           /* 输入捕获的 GPIO 口 */
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;    /* 复用推挽 */
    gpio_init_struct.Pull = GPIO_PULLDOWN;          /* 下拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH; /* 高速 */
    HAL_GPIO_Init(TPAD_GPIO_PORT, &gpio_init_struct);

    /* TPAD 引脚输出 0, 放电 */
    HAL_GPIO_WritePin(TPAD_GPIO_PORT, TPAD_GPIO_PIN, GPIO_PIN_RESET);
    delay_ms(5);

    g_timx_cap_chy_handle.Instance->SR = 0;          /* 清除标记 */
    g_timx_cap_chy_handle.Instance->CNT = 0;         /* 归零 */

    gpio_init_struct.Pin = TPAD_GPIO_PIN;            /* 输入捕获的 GPIO 口 */
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;         /* 复用推挽 */
    gpio_init_struct.Pull = GPIO_NOPULL;             /* 不带上下拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;   /* 高速 */
    gpio_init_struct.Alternate = TPAD_GPIO_AF;       /* PA5 复用为 TIM2 通道 1 */
    HAL_GPIO_Init(TPAD_GPIO_PORT,&gpio_init_struct);/* TPAD 引脚浮空输入 */
}

/**
 * @brief       得到定时器捕获值
 * @note        如果超时, 则直接返回定时器的计数值
 *              我们定义超时时间为: TPAD_ARR_MAX_VAL - 500
 * @param       无
 * @retval      捕获值/计数值（超时的情况下返回）
 */
static uint16_t tpad_get_val(void)
{
    uint32_t flag = (TPAD_TIMX_CAP_CHY== TIM_CHANNEL_1)?TIM_FLAG_CC1:\
                    (TPAD_TIMX_CAP_CHY== TIM_CHANNEL_2)?TIM_FLAG_CC2:\
                    (TPAD_TIMX_CAP_CHY== TIM_CHANNEL_3)?TIM_FLAG_CC3:\
                                                        TIM_FLAG_CC4;
    tpad_reset();
    while (__HAL_TIM_GET_FLAG(&g_timx_cap_chy_handle ,flag) == RESET)
    {/* 等待通道 CHY 捕获上升沿 */
        if (g_timx_cap_chy_handle.Instance->CNT > TPAD_ARR_MAX_VAL - 500)
        {
          return g_timx_cap_chy_handle.Instance->CNT;/* 超时了,直接返回 CNT 的值 */
        }
    }
    return TPAD_TIMX_CAP_CHY_CCRX;                    /* 返回捕获/比较值 */
}

/**
 * @brief       读取 n 次, 取最大值
 * @param       n       : 连续获取的次数
 * @retval      n 次读数里面读到的最大读数值
 */
static uint16_t tpad_get_maxval(uint8_t n)
{
    uint16_t temp = 0;
    uint16_t maxval = 0;
    while (n--)
    {
        temp = tpad_get_val();  /* 得到一次值 */
        if (temp > maxval)
        {
            maxval = temp;
        }
    }
    return maxval;
```

```
}
```

得到充电时间后，接下来我们要做的就是获取没有按下 TPAD 时的充电时间，并把它作为基准来确认后续有无按下操作，我们定义全局变量 g_tpad_default_val 来保存这个值，通过多次平均的滤波算法来减小误差，编写的初始化函数 tpad_init 代码如下。

```
/**
 * @brief       初始化触摸按键
 * @param       psc      : 分频系数(值越小, 越灵敏, 最小值为: 1)
 * @retval      0, 初始化成功; 1, 初始化失败;
 */
uint8_t tpad_init(uint16_t psc)
{
    uint16_t buf[10];
    uint16_t temp;
    uint8_t j, i;
    /* 以 Ft / (psc - 1) Mhz 的频率计数 @Ft = 定时器工作频率 */
    tpad_timx_cap_init(TPAD_ARR_MAX_VAL, psc - 1);

    for (i = 0; i < 10; i++)                            /* 连续读取 10 次 */
    {
        buf[i] = tpad_get_val();
        delay_ms(10);
    }

    for (i = 0; i < 9; i++)                             /* 排序 */
    {
        for (j = i + 1; j < 10; j++)
        {
            if (buf[i] > buf[j])                        /* 升序排列 */
            {
                temp = buf[i];
                buf[i] = buf[j];
                buf[j] = temp;
            }
        }
    }

    temp = 0;
    for (i = 2; i < 8; i++)     /* 取中间的 6 个数据进行平均 */
    {
        temp += buf[i];
    }

    g_tpad_default_val = temp / 6;
    printf("g_tpad_default_val:%d\r\n", g_tpad_default_val);

    if (g_tpad_default_val > (uint16_t)TPAD_ARR_MAX_VAL / 2)
    {
        return 1;               /* 初始化遇到超过 TPAD_ARR_MAX_VAL/2 的数值,不正常! */
    }
    return 0;
}
```

函数 tpad_init 用于初始化输入捕获，并获取默认的 TPAD 值。该函数有一个参数，用来传递分频系数，其实是为了配置 tpad_timx_cap_init 的计数周期。在该函数中连续 10 次读取 TPAD 值，将这些值升序排列后取中间 6 个值再做平均（这样做的目的是尽量减少误差），并赋值给 g_tpad_default_val，用于后续触摸判断的标准。

得到初始值后，我们需编写一个按键扫描函数，以方便在需要监控 TPAD 的地方调用，代码如下：

```
/**
 * @brief       扫描触摸按键
 * @param       mode : 扫描模式
 *   @arg       0, 不支持连续触发(按下一次必须松开才能按下一次);
```

```
 *   @arg        1, 支持连续触发(可以一直按下)
 * @retval      0, 没有按下; 1, 有按下;
 */
uint8_t tpad_scan(uint8_t mode)
{
    static uint8_t keyen = 0;       /* 0, 可以开始检测;  > 0, 还不能开始检测; */
    uint8_t res = 0;
    uint8_t sample = 3;             /* 默认采样次数为 3 次 */
    uint16_t rval;
    if (mode)
    {
        sample = 6;                 /* 支持连按的时候，设置采样次数为 6 次 */
        keyen = 0;                  /* 支持连按，每次调用该函数都可以检测 */
    }
    rval = tpad_get_maxval(sample);
    if (rval > (g_tpad_default_val + TPAD_GATE_VAL))
    {/* 大于 tpad_default_val + TPAD_GATE_VAL,有效 */
        if (keyen == 0)
        {
            res = 1;                /* keyen==0, 有效 */
        }
        keyen = 3;                  /* 至少要再过 3 次之后才能按键有效 */
    }
    if (keyen)keyen--;
    return res;
}
```

该函数用于扫描 TPAD 是否有触摸，该函数的参数 mode，用于设置是否支持连续触发。返回值如果是 0，说明没有触摸，如果是 1，则说明有触摸。该函数同样包含了一个静态变量，用于检测控制，类似第十四章按键扫描实验的 key_scan 函数。所以该函数同样是不可重入的。

TPAD 函数到此就编写完了，接下来我们通过 main 函数编写测试代码来验证一下 TPAD 的逻辑是否正确。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t t = 1;
    sys_cache_enable();
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4); /* 设置时钟,400Mhz */
    delay_init(400);                        /* 延时初始化 */
    led_init();                             /* 初始化 LED */
    tpad_init(2);                           /* 初始化触摸按键 */

    while(1)
    {
        if (tpad_scan(0))    /* 成功捕获到了一次上升沿(此函数执行时间至少 15ms) */
        {
            LED1_TOGGLE();  /* LED1 翻转 */
        }
        t++;
        if (t == 15)
        {
            t = 0;
            LED0_TOGGLE();  /* LED0 翻转 */
        }
        delay_ms(10);
    }
}
```

初始化了必要的外设后， 我们通过循环来实现我们的代码操作。我们在扫描函数中定义

了电容按键触摸发生后的状态，通过判断返回值来判断是否符合按下的条件，如果按下我们就翻转一次 LED1。LED0 通过累计延时次数的方法，既能保证扫描的频率，又能达到定时翻转的目的。

22.4 下载验证

下载代码后，可以看到 LED0 不停闪烁（每 300ms 闪烁一次），用手指按下电容按键时，LED1 的状态发生改变（亮灭交替一次）。这里确保开发板上的 TPAD 和 ADC 的跳线帽要连接好，否则测试就不准了，如果下载代码前没有连接好，请连接后复位重新测试即可。

第二十三章 OLED 显示实验

本章我们来学习使用 OLED 液晶显示屏，在开发板上我们预留了 OLED 模块接口，需要准备一个 OLED 显示模块。下面我们一起来点亮 OLED，并实现 ASCII 字符的显示。

本章分为如下几个小节：

23.1 OLED 简介
23.2 硬件设计
23.3 程序设计
23.4 下载验证

23.1 OLED 简介

OLED，即有机发光二极管（Organic Light-Emitting Diode），又称为有机电激光显示（Organic Electroluminesence Display，OELD）。OLED 由于同时具备自发光，不需背光源、对比度高、厚度薄、视角广、反应速度快、可用于挠曲性面板、使用温度范围广、构造及制程较简单等优异之特性，被认为是下一代的平面显示器新兴应用技术。

LCD 都需要背光，而 OLED 不需要，因为它是自发光的。这样同样的显示，OLED 效果要来得好一些。以目前的技术，OLED 的尺寸还难以大型化，但是分辨率确可以做到很高。在本章中，我们使用的是 ALINETEK 的 OLED 显示模块，该模块有以下特点：

1）模块有单色和双色两种可选，单色为纯蓝色，而双色则为黄蓝双色。
2）尺寸小，显示尺寸为 0.96 寸，而模块的尺寸仅为 27mm*26mm 大小。
3）高分辨率，该模块的分辨率为 128*64。
4）多种接口方式，该模块提供了总共 4 种接口包括：6800、8080 两种并行接口方式、4 线 SPI 接口方式以及 IIC 接口方式（只需要 2 根线就可以控制 OLED 了！）。
5）不需要高压，直接接 3.3V 就可以工作了。

这里要提醒大家的是，该模块不和 5.0V 接口兼容，所以请大家在使用的时候一定要小心，别直接接到 5V 的系统上去，否则可能烧坏模块。以下 4 种模式通过模块的 BS1 和 BS2 设置，BS1 和 BS2 的设置与模块接口模式的关系如表 23.1.1 所示：

接口方式	4 线 SPI	IIC	8 位 6800	8 位 8080
BS1	0	1	0	1
BS2	0	0	1	1

表 23.1.1 OLED 模块接口方式设置表

表 23.1.1 中：“1”代表接 VCC，而“0”代表接 GND。

该模块的外观图如图 23.1.1 所示：

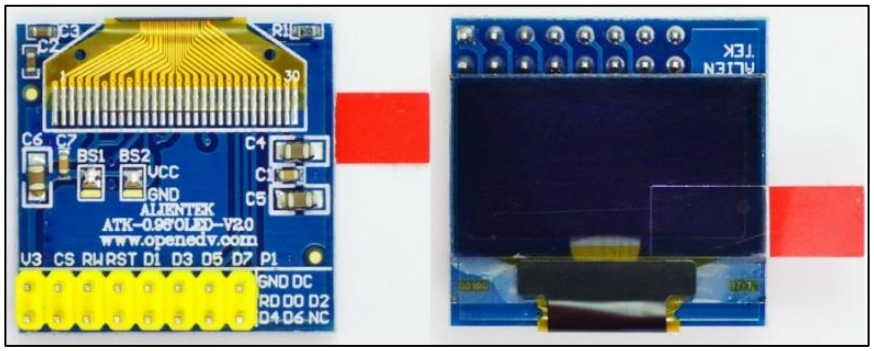


图 23.1.1 正点原子 OLED 模块外观图

正点原子 OLED 模块默认设置是：BS1 和 BS2 接 VCC，即使用 8080 并口方式，如果你想要设置为其他模式，则需要在 OLED 的背面，用烙铁修改 BS1 和 BS2 的设置。

模块的原理图如图 23.1.2 所示：

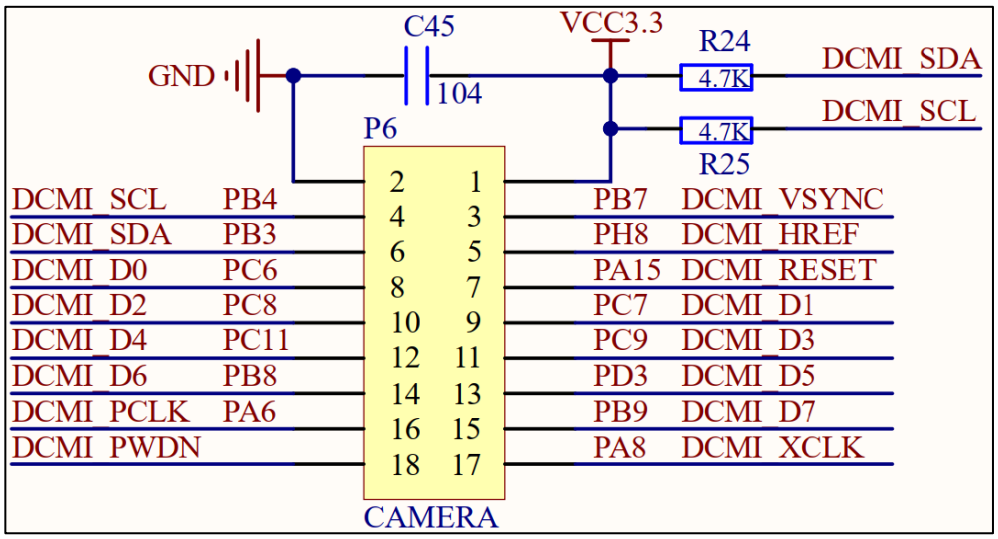


图 23.1.2 正点原子 OLED 模块原理图

该模块采用 8*2 的 2.54 排针与外部连接，总共有 16 个管脚，在 16 条线中，我们只用了 15 条，有一个是悬空的。15 条线中，电源和地线占了 2 条，还剩下 13 条信号线。在不同模式下，我们需要的信号线数量是不同的，在 8080 模式下，需要全部 13 条，而在 IIC 模式下，仅需要 2 条线就够了！这其中有一条是共同的，那就是复位线 RST（RES），RST 上的低电平，将导致 OLED 复位，在每次初始化之前，都应该复位一下 OLED 模块。

正点原子 OLED 模块的控制器是 SSD1306，本章，我们将学习如何通过 STM32H743 来控制该模块显示字符和数字，本章的实例代码将可以支持两种方式与 OLED 模块连接，一种是 8080 的并口方式，另外一种是 4 线 SPI 方式。

实际使用过程我们也通常只选用其中的一种来实现硬件上的连接，我们会分别介绍这两种模式，读者可以选择性阅读。

23.1.1 硬件驱动接口模式

1. 8080 并口模式

首先我们介绍一下模块的 8080 并行接口，8080 并行接口的发明者是 INTEL，该总线也被广泛应用于各类液晶显示器，正点原子 OLED 模块也提供了这种接口，使得 MCU 可以快速的访问 OLED。正点原子 OLED 模块的 8080 接口方式需要如下一些信号线：

CS：OLED 片选信号。

WR：向 OLED 写入数据。

RD：从 OLED 读取数据。

D[7：0]：8 位双向数据线。

RST(RES)：硬复位 OLED。

DC：命令/数据标志（0，读写命令；1，读写数据）。

模块的 8080 并口写的过程为：先根据要写入的数据的类型，设置 DC 为高（数据）/低（命令），设置 WR 起始电平为高，然后拉低片选，选中 SSD1306，接着我们在整个读时序上保持 RD 为高电平，然后：

拉低 WR 的电平准备写入数据，向数据线（D[7：0]）上输入要写的信息；

拉高 WR，这样得到一个 WR 的上升沿，在这个上升沿，使数据写入到 SSD1306 里面；

SSD1306 的 8080 并口写时序图如图 23.1.1.1 所示：

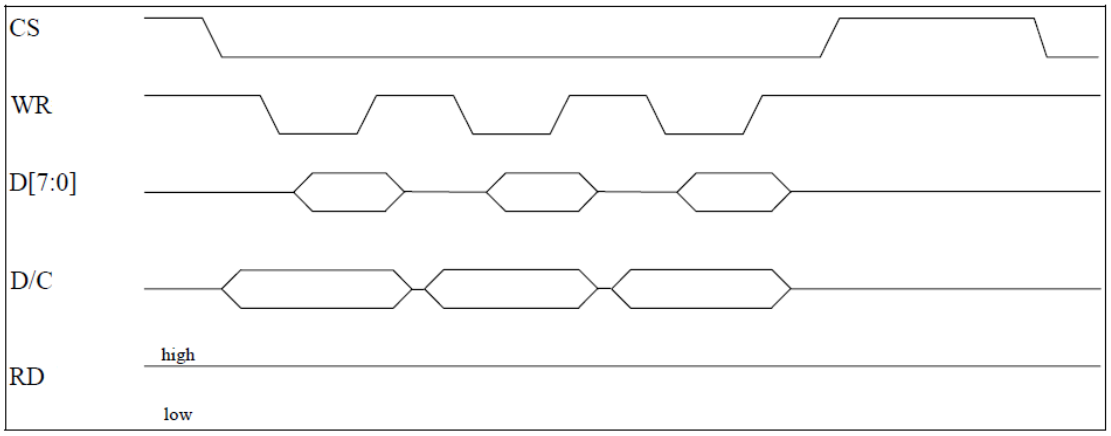


图 23.1.1.1 8080 并口写时序图

模块的 8080 并口读的过程为：先根据要写入的数据的类型，设置 DC 为高（数据）/低（命令），设置 RD 起始电平为高，然后拉低片选 CS 信号，选中 SSD1306，接着我们在整个读时序上保持 WR 为高电平，然后类似写时序，同样的：

在 RD 的上升沿， 使数据锁存到数据线（D[7：0]）上；

SSD1306 的 8080 并口读时序图如图 23.1.1.2 所示：

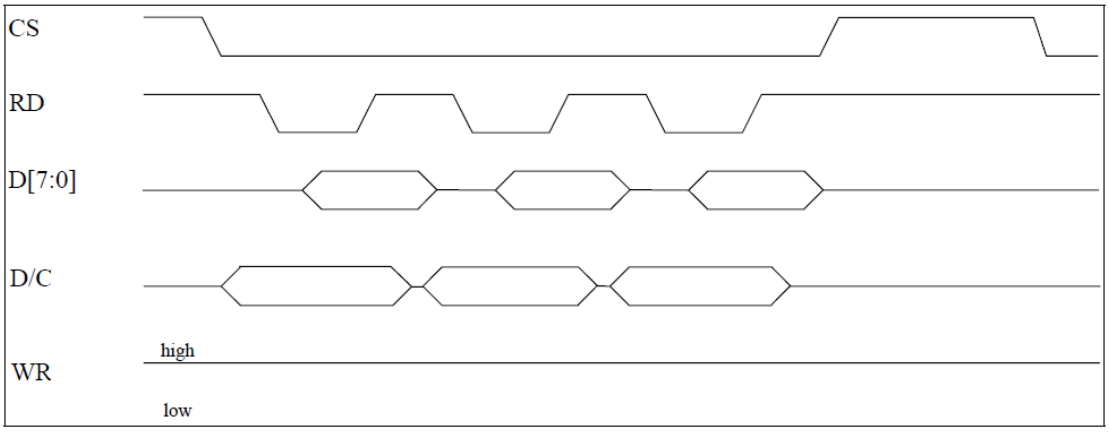


图 23.1.1.2 8080 并口读时序图

SSD1306 的 8080 接口方式下，控制脚的信号状态所对应的功能如表 23.1.1.1：

功能	RD	WR	CS	DC
写命令	H	↑	L	L
读状态	↑	H	L	L
写数据	H	↑	L	H
读数据	↑	H	L	H

表 23.1.1.1 控制脚信号状态功能表

在 8080 方式下读数据操作的时候，我们有时候（例如读显存的时候）需要一个假读命（Dummy Read），以使得微控制器的操作频率和显存的操作频率相匹配。在读取真正的数据之前，由一个的假读的过程。这里的假读，其实就是第一个读到的字节丢弃不要，从第二个开始，才是我们真正要读的数据。

一个典型的读显存的时序图，如图 23.1.1.3 所示：

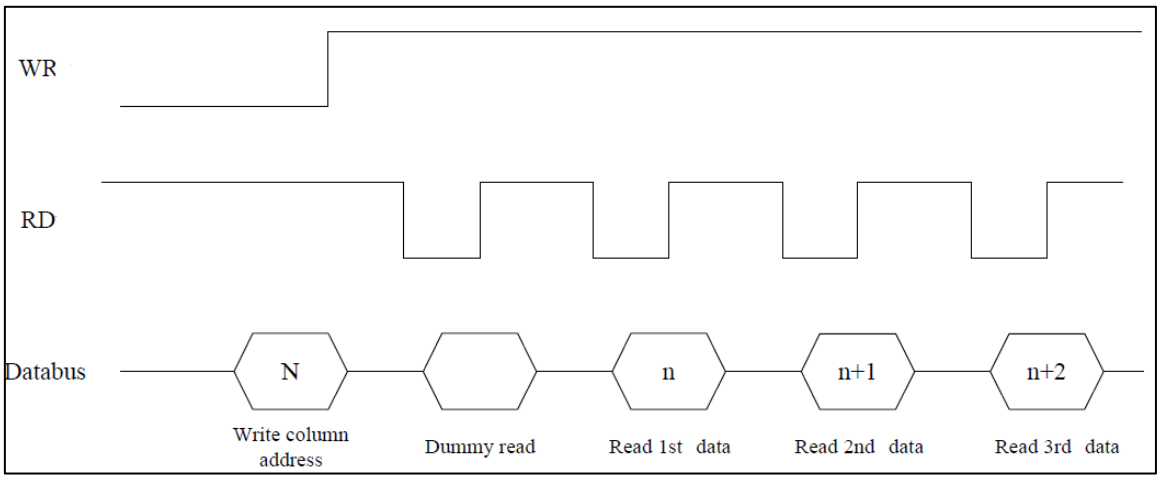


图 23.1.1.3 读显存时序图

可以看到，在发送了列地址之后，开始读数据，第一个是 Dummy Read，也就是假读，我们从第二个开始，才算是真正有效的数据。并行接口模式就介绍到这里。

2. SPI 模式

我们的代码同时兼容 SPI 方式的驱动，如果你使用的是这种驱动方式，则应该把代码中的宏 OLED_MODE 设置为：

```
#define OLED_MODE       0    /* 0: 4 线串行模式 */
```

我们接下来介绍一下 4 线串行（SPI）方式，4 先串口模式使用的信号线有如下几条：

CS：OLED 片选信号。

RST(RES)：硬复位 OLED。

DC：命令/数据标志（0，读写命令；1，读写数据）。

SCLK：串行时钟线。在 4 线串行模式下，D0 信号线作为串行时钟线 SCLK。

SDIN：串行数据线。在 4 线串行模式下，D1 信号线作为串行数据线 SDIN。

模块的 D2 需要悬空，其他引脚可以接到 GND。在 4 线串行模式下，只能往模块写数据而不能读数据。

在 4 线 SPI 模式下，每个数据长度均为 8 位，在 SCLK 的上升沿，数据从 SDIN 移入到 SSD1306，并且是高位在前的。DC 线还是用作命令/数据的标志线。在 4 线 SPI 模式下，写操作的时序如图 23.1.1.4 所示：

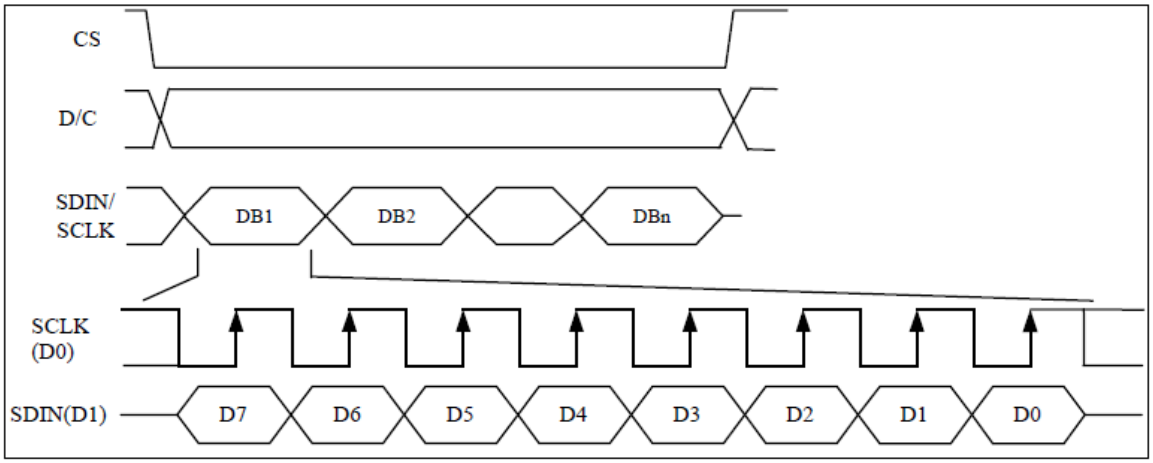


图 23.1.1.4 4 线 SPI 写操作时序图

4 线串行模式就为大家介绍到这里。其他还有几种模式，在 SSD1306 的数据手册上都有详细的介绍，我们把资料放到“开发板资料 A 盘->7，硬件资料\3，液晶资料\0.96 寸 OLED-SSD1306\SSD1306-Revision 1.1 (Charge Pump).pdf”，如果要使用这些方式，请大家参考该手册并自行实现相应的功能代码。

23.1.2 OLED 显存

接下来，我们介绍一下模块的显存，SSD1306 的显存总共为 128*64bit 大小，SSD1306 将这些显存分为了 8 页，不使用显存对应的行列的重映射，其对应关系如表 23.1.2.1 所示：

列（COM0~63）	行（COL0~127）						
	SEG0	SEG1	SEG2	……	SEG125	SEG126	SEG127
	PAGE0						
	PAGE1						
	PAGE2						
	PAGE3						
	PAGE4						
	PAGE5						
	PAGE6						
	PAGE7						

表 23.1.2.1 SSD1306 显存与屏幕对应关系表

可以看出，SSD1306 的每页包含了 128 个字节，总共 8 页，这样刚好是 128*64 的点阵大小。当 GRAM 的写入模式为页模式时，需要设置低字节起始的列地址（0x00~0x0F）和高字节的起始列地址（0x10~0x1F），芯片手册中给出了写入 GRAM 与显示的对应关系，写入列地址在写完一字节后自动按列增长，如图 23.1.2.2 所示：

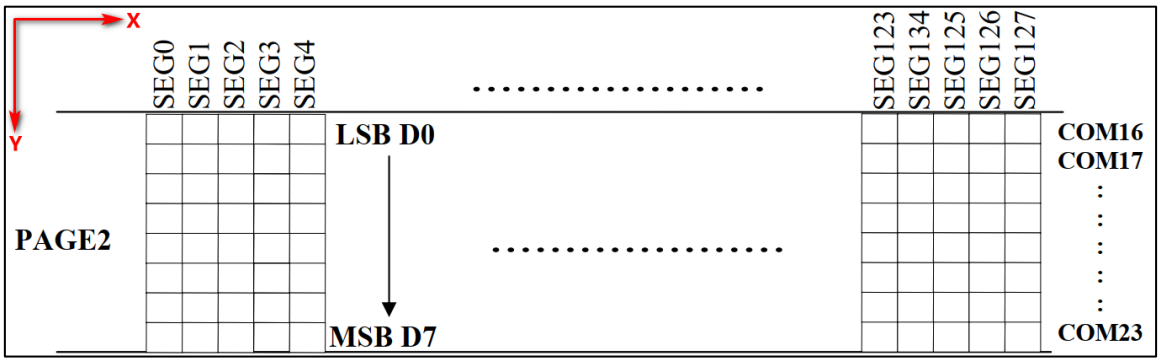


图 23.1.2.2 SSD1306 页 2 显存写入字节与屏幕坐标的关系

因为每次写入都是按字节写入的，这就存在一个问题，如果我们使用只写方式操作模块，那么，每次要写 8 个点，这样，我们在画点的时候，就必须把要设置的点所在的字节的每个位都搞清楚当前的状态（0/1？），否则写入的数据就会覆盖掉之前的状态，结果就是有些不需要显示的点，显示出来了，或者该显示的没有显示了。这个问题在能读的模式下，我们可以先读出来要写入的那个字节，得到当前状况，在修改了要改写的位之后再写进 GRAM，这样就不会影响到之前的状况了。但是这样需要能读 GRAM，对于 4 线 SPI 模式/IIC 模式，模块是不支持读的，而且读→改→写的方式速度也比较慢。

所以我们采用的办法是在 STM32H743 的内部建立一个虚拟的 OLED 的 GRAM（共 128*8=1024 个字节），在每次修改的时候，只是修改 STM32H743 上的 GRAM（实际上就是 SRAM），在修改完了之后，一次性把 STM32H743 上的 GRAM 写入到 OLED 的 GRAM。当然这个方法也有坏处，一个是对于那些 SRAM 很小的单片机（比如 51 系列）不太友好，另一个是每次都写入全屏，屏幕刷新率会变低。

SSD1306 的命令比较多，这里我们仅介绍几个比较常用的命令，这些命令如下表所示：

序号	指令	各位描述								命令	说明
	HEX	D7	D6	D5	D4	D3	D2	D1	D0		
0	81	1	0	0	0	0	0	0	1	设置对比度	A的值越大屏幕越亮，A的范围从0X00~0XFF
	A[7:0]	A7	A6	A5	A4	A3	A2	A1	A0		
1	AE/AF	1	0	1	0	1	1	1	X0	设置显示开关	X0=0，关闭显示；X0=1，开启显示；
2	8D	1	0	0	0	1	1	0	1	电荷泵设置	A2=0，关闭电荷泵 A2=1,开启电荷泵
	A[7:0]	*	*	0	1	0	A2	0	0		
3	B0~B7	1	0	1	1	0	X2	X1	X0	设置页地址	X[2:0]=0~7对应页0~7
4	00~0F	0	0	0	0	X3	X2	X1	X0	设置列地址低四位	设置8位起始列地址的低四位
5	10~1F	0	0	0	0	X3	X2	X1	X0	设置列地址高四位	设置8位起始列地址的高四位

表 23.1.2.3 SSD1306 常用命令表

第 0 个命令为 0X81，用于设置对比度的，这个命令包含了两个字节，第一个 0X81 为命令，随后发送的一个字节为要设置的对比度的值。这个值设置得越大屏幕就越亮。

第 1 个命令为 0XAE/0XAF。0XAE 为关闭显示命令；0XAF 为开启显示命令。

第 2 个命令为 0X8D，该指令也包含 2 个字节，第一个为命令字，第二个为设置值，第二个字节的 BIT2 表示电荷泵的开关状态，该位为 1，则开启电荷泵，为 0 则关闭。在模块初始化的时候，这个必须要开启，否则是看不到屏幕显示的。

第 3 个命令为 0XB0~B7，该命令用于设置页地址，其低三位的值对应着 GRAM 的页地址。

第 4 个指令为 0X00~0X0F，该指令用于设置显示时的起始列地址低四位。

第 6 个指令为 0X10~0X1F，该指令用于设置显示时的起始列地址高四位。

其他命令，我们就不在这里一一介绍了，大家可以参考 SSD1306 datasheet 的第 28 页。从这页开始，对 SSD1306 的指令有详细的介绍。

最后，我们再来介绍一下 OLED 模块的初始化过程，SSD1306 的典型初始化框图如图 23.1.2.4 所示：

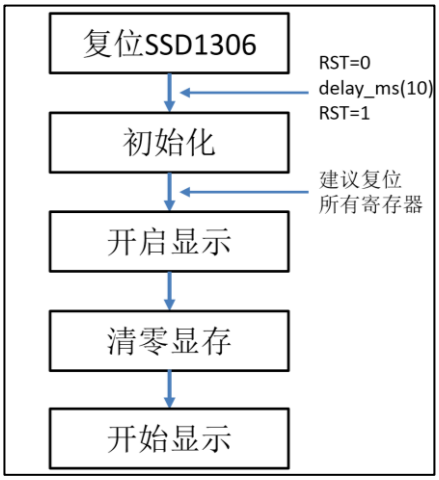


图 23.1.2.4 SSD1306 初始化框图

驱动 IC 的初始化代码，我们直接使用厂家推荐的设置就可以了，只要对细节部分进行一些修改，使其满足我们自己的要求即可，其他不需要变动。

OLED 的介绍就到此为止，我们重点向大家介绍了正点原子 OLED 模块的相关知识，接下来我们将使用这个模块来显示字符和数字。通过以上介绍，我们可以得出 OLED 显示需要的相关设置步骤如下：

1）设置 STM32H743 与 OLED 模块相连接的 IO。

这一步，先将我们与 OLED 模块相连的 IO 口设置为输出，具体使用哪些 IO 口，这里需要根据连接电路以及 OLED 模块所设置的通讯模式来确定。这些将在硬件设计部分向大家介绍。

2）初始化 OLED 模块。

其实这里就是上面的初始化框图的内容，通过对 OLED 相关寄存器的初始化，来启动

OLED 的显示。为后续显示字符和数字做准备。

3）通过函数将字符和数字显示到 OLED 模块上。

这里就是通过我们设计的程序，将要显示的字符送到 OLED 模块就可以了，这些函数将在软件设计部分向大家介绍。

通过以上三步，我们就可以使用正点原子 OLED 模块来显示字符和数字了，在后面我们还将会给大家介绍显示汉字的方法。这一部分就先介绍到这里。

23.2 硬件设计

1. 例程功能

使用 8080 并口模式驱动或者使用 4 线 SPI 串口模式，驱动 OLED 模块，不停的显示 ASCII 码和码值。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯

LED : LED0 - PB1

2）正点原子 0.96 寸 OLED 模块，在硬件上，OLED 与开发板的 IO 口对应关系如下：

开发板 IO	OLED 模块 IO	STM32 芯片对应 GPIO
VCC3.3	VCC3.3	
OV_VSYNC	OLED_CS	PB7
OV_HREF	OLED_RW	PH8
OV_RST	OLED_RST	PA15
OV_D1	OLED_D1	PB8
OV_D3	OLED_D3	PC11
OV_D5	OLED_D5	PC8
OV_D7	OLED_D7	PC6
OV_SCL	OLED_DC	PB4
OV_SDA	OLED_RD	PB3
OV_D0	OLED_D0	PB9
OV_D2	OLED_D2	PD3
OV_D4	OLED_D4	PC9
OV_D6	OLED_D6	PC7
OV_PCLK	悬空	悬空

表 23.2.1 OLED 模块与开发板连接示意图

3. 原理图

OLED 模块的原理图在前面已有详细说明了，这里我们介绍 OLED 模块与我们开发板的连接，开发板上有一个 OLED/CAMERA 的接口（P6 接口）可以和正点原子 OLED 模块直接对插（靠左插！），连接如图 23.2.1 所示：

图 23.3.1 OLED 模块与开发板连接示意图

这些线的连接，开发板的内部已经连接好了，我们只需要将 OLED 模块插上去就好了，注

意，这里的 OLED_D[7:0]因为不是接的连续的 IO，所以得用拼凑的方式去组合一下，后续会介绍。

23.3 程序设计

OLED 只是用到 HAL 库中 GPIO 外设的驱动代码，在前面跑马灯实验已经介绍了。

23.3.1 程序流程图

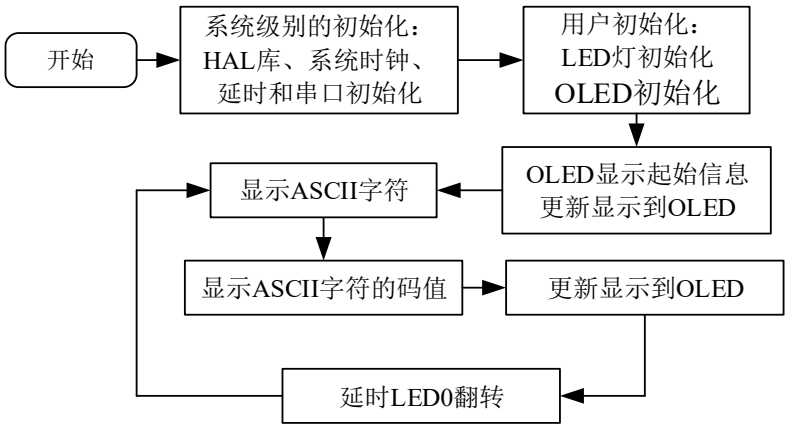


图 23.3.1.1 OLED 实验程序流程图

23.3.2 程序解析

1. OLED 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。OLED 驱动源码包括三个文件：oled.c、oled.h 和 oledfont.h。oledfont.h 头文件存放的是 ASCII 字符集，oled.h 存放的是引脚接口宏定义和函数声明等，oled.c 则是驱动代码。

首先看 oledfont.h 头文件的 ASCII 字符集内容：

```
/* 常用 ASCII 表
 * 偏移量 32
 * ASCII 字符集: !"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]
                ^_`abcdefghijklmnopqrstuvwxyz{|}~
 * PC2LCD2002 取模方式设置：阴码+逐列式+顺向+C51 格式
 * 总共：3 个字符集（12*12、16*16 和 24*24），用户可以自行新增其他分辨率的字符集。
 * 每个字符所占用的字节数为:(size/8+((size%8)?1:0))*(size/2),
   其中 size:是字库生成时的点阵大小(12/16/24...)
 */
/* 12*12 ASCII 字符集点阵 */
const unsigned char oled_asc2_1206[95][12]={ ...这里省略字符集库... };
/* 16*16 ASCII 字符集点阵 */
const unsigned char oled_asc2_1608[95][16]={ ...这里省略字符集库... };
/* 24*24 ASICII 字符集点阵 */
const unsigned char oled_asc2_2412[95][36]={ ...这里省略字符集库... };
```

该头文件中包含三个大小不同的 ASCII 字符集点阵，其中包括：12*12 ASCII 字符集点阵、16*16 ASCII 字符集点阵、24*24 ASICII 字符集点阵。每个字符集点阵都包含 95 个常用的 ASCII 字符集，从空格符开始，分别为： !"#$%&'()*+,-0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~。

上面的 ASCII 字符集，我们可以使用一个款很好的字符提取软件来制作获取。字符提取软件为：ATK_XFONT，该软件可以提供各种字符，包括汉字（字体和大小都可以自己设置）阵提取，且取模方式可以设置好几种，常用的取模方式，该软件都支持。该软件还支持图形模式，

也就是用户可以自己定义图片的大小，然后画图，根据所画的图形再生成点阵数据，这功能在制作图标或图片的时候很有用。

该软件的界面如图 23.3.2.1 所示：

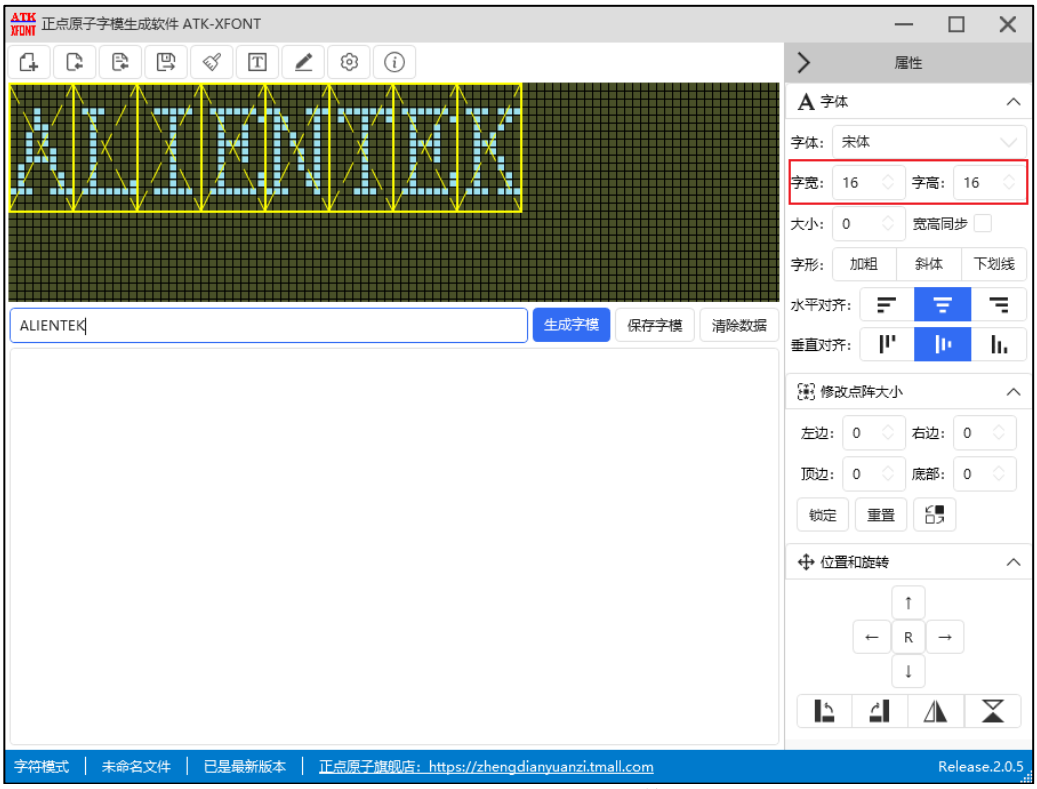


图 23.3.2.1 ATK_XFONT 软件界面

然后我们选择设置，在设置里面设置取模方式如图 23.3.2.2 所示：

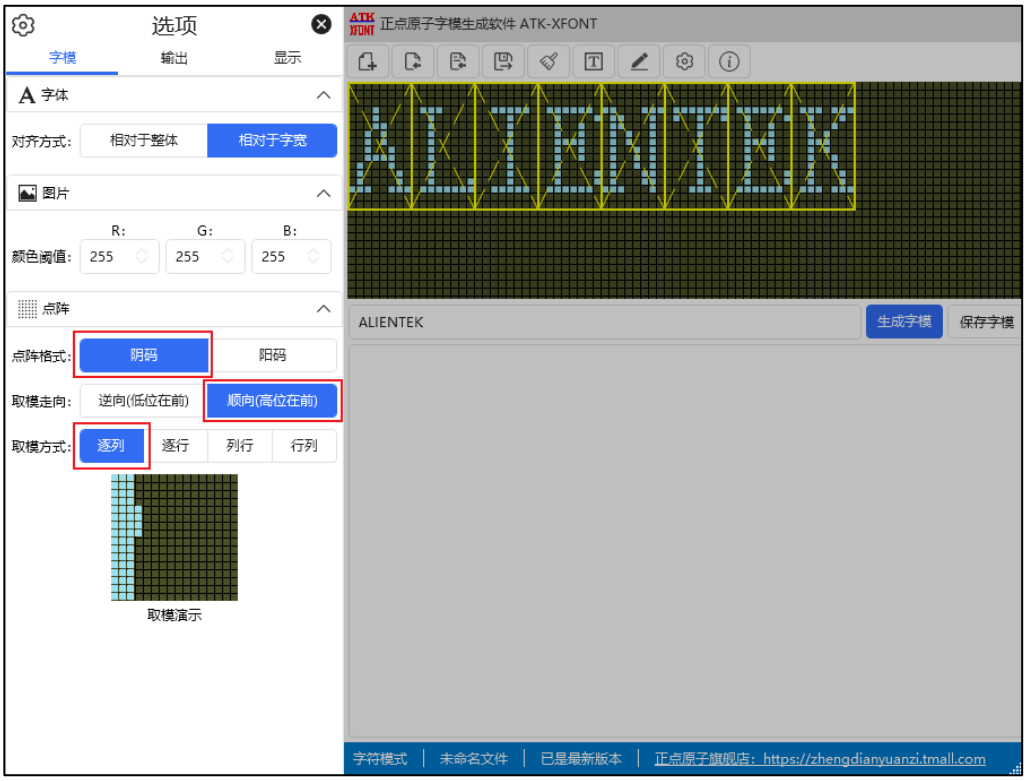


图 23.3.2.2 设置取模方式

上图设置的取模方式，在右上角的取模说明里面有，即：从第一列开始向下每取 8 个点作

为一个字节，如果最后不足 8 个点就补满 8 位。取模顺序是从高到低，即第一个点作为最高位。如*-------取为 10000000。其实就是按如图 23.3.2.3 所示的这种方式：

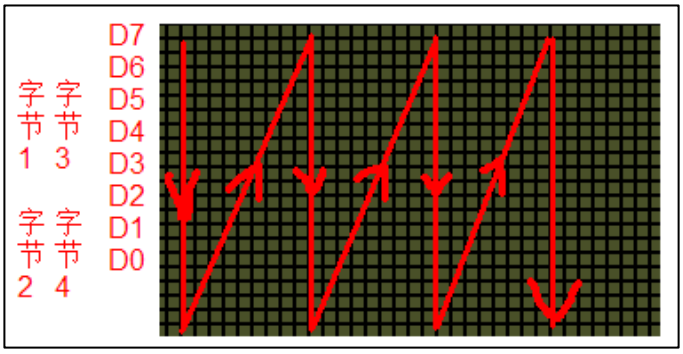


图 23.3.2.3 取模方式图解

从上到下，从左到右，高位在前。我们按这样的取模方式，然后把 ASCII 字符集按 12*6 大小、16*8 和 24*12 大小取模出来（对应汉字大小为 12*12、16*16 和 24*24，字符的只有汉字的一半大！），每个 12*6 的字符占用 12 个字节，每个 16*8 的字符占用 16 个字节，每个 24*12 的字符占用 36 个字节。

oled.c 和 oled.h 文件的代码可以帮助显示我们制作好的字符集。我们还是先看 oled.h 文件的宏定义，首先是 OLED 模式设置宏定义：

```
/* OLED 模式设置
 * 0: 4 线串行模式    （模块的 BS1，BS2 均接 GND）
 * 1: 并行 8080 模式  （模块的 BS1，BS2 均接 VCC）
 */
#define OLED_MODE       1    /* 默认使用 8080 并口模式 */
```

通过宏定义 OLED_MODE 来决定使用 4 线串行模式（0）还是并行 8080 模式（1），默认使用 8080 并口模式。

关于 OLED 80 并口模式和 SPI 模式的引脚定义就不列出来了，请看源码。

还有两个关于向 OLED 写入选择命令或者数据的宏定义，后面讲的 oled_wr_byte 函数用到。

```
/* 命令/数据 定义 */
#define OLED_CMD        0       /* 写命令 */
#define OLED_DATA       1       /* 写数据 */
```

最后就是 oled.c 文件的驱动源码介绍。先是 OLED(SSD1306)的初始化函数，其定义如下：

```
/**
 * @brief       初始化 OLED(SSD1306)
 * @param       无
 * @retval      无
 */
void oled_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;

#if OLED_MODE==1                                        /* 使用 8080 并口模式 */
    __HAL_RCC_GPIOA_CLK_ENABLE();                       /* 使能 PORTA 时钟 */
    __HAL_RCC_GPIOB_CLK_ENABLE();                       /* 使能 PORTB 时钟 */
    __HAL_RCC_GPIOC_CLK_ENABLE();                       /* 使能 PORTC 时钟 */
    __HAL_RCC_GPIOD_CLK_ENABLE();                       /* 使能 PORTD 时钟 */
    __HAL_RCC_GPIOH_CLK_ENABLE();                       /* 使能 PORTH 时钟 */

    /* PA15 设置 */
    gpio_init_struct.Pin = GPIO_PIN_15;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;        /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;                /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;      /* 高速 */
    HAL_GPIO_Init(GPIOA, &gpio_init_struct);

    /* PB3, PB4, PB7, PB8, PB9 设置 */
```

```
    gpio_init_struct.Pin = GPIO_PIN_3 | GPIO_PIN_4 | GPIO_PIN_7 | GPIO_PIN_8 |
                           GPIO_PIN_9;
    HAL_GPIO_Init(GPIOB, &gpio_init_struct);

    /* PC6, PC7, PC8, PC9, PC11 设置*/
    gpio_init_struct.Pin = GPIO_PIN_6 | GPIO_PIN_7 | GPIO_PIN_8 | GPIO_PIN_9 |
                           GPIO_PIN_11;
    HAL_GPIO_Init(GPIOC, &gpio_init_struct);

     /* PD3 设置 */
    gpio_init_struct.Pin = GPIO_PIN_3;
    HAL_GPIO_Init(GPIOD, &gpio_init_struct);

    /* PH8 设置 */
    gpio_init_struct.Pin = GPIO_PIN_8;
    HAL_GPIO_Init(GPIOH, &gpio_init_struct);

    OLED_WR(1);
    OLED_RD(1);

#else                /* 使用 4 线 SPI 串口模式 */

    OLED_SPI_RST_CLK_ENABLE();
    OLED_SPI_CS_CLK_ENABLE();
    OLED_SPI_RS_CLK_ENABLE();
    OLED_SPI_SCLK_CLK_ENABLE();
    OLED_SPI_SDIN_CLK_ENABLE();

    gpio_init_struct.Pin = OLED_SPI_RST_PIN;              /* RST 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;          /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;                  /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_MEDIUM;      /* 中速 */
    HAL_GPIO_Init(OLED_SPI_RST_PORT, &gpio_init_struct);  /* RST 引脚模式设置 */

    gpio_init_struct.Pin = OLED_SPI_CS_PIN;               /* CS 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;          /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;                  /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_MEDIUM;      /* 中速 */
    HAL_GPIO_Init(OLED_SPI_CS_PORT, &gpio_init_struct);   /* CS 引脚模式设置 */

    gpio_init_struct.Pin = OLED_SPI_RS_PIN;               /* RS 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;          /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;                  /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_MEDIUM;      /* 中速 */
    HAL_GPIO_Init(OLED_SPI_RS_PORT, &gpio_init_struct);   /* RS 引脚模式设置 */

    gpio_init_struct.Pin = OLED_SPI_SCLK_PIN;             /* SCLK 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;          /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;                  /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_MEDIUM;      /* 中速 */
    HAL_GPIO_Init(OLED_SPI_SCLK_PORT,&gpio_init_struct);  /* SCLK 引脚模式设置 */

    gpio_init_struct.Pin = OLED_SPI_SDIN_PIN;             /* SDIN 引脚模式设置 */
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;          /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;                  /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_MEDIUM;      /* 中速 */
    HAL_GPIO_Init(OLED_SPI_SDIN_PORT, &gpio_init_struct); /* SDIN 引脚模式设置 */

    OLED_SDIN(1);
    OLED_SCLK(1);
#endif
    OLED_CS(1);
```

```
    OLED_RS(1);

    OLED_RST(0);
    delay_ms(100);
    OLED_RST(1);

    oled_wr_byte(0xAE, OLED_CMD);    /* 关闭显示 */
    oled_wr_byte(0xD5, OLED_CMD);    /* 设置时钟分频因子,震荡频率 */
    oled_wr_byte(80, OLED_CMD);      /* [3:0],分频因子;[7:4],震荡频率 */
    oled_wr_byte(0xA8, OLED_CMD);    /* 设置驱动路数 */
    oled_wr_byte(0X3F, OLED_CMD);    /* 默认 0X3F(1/64) */
    oled_wr_byte(0xD3, OLED_CMD);    /* 设置显示偏移 */
    oled_wr_byte(0X00, OLED_CMD);    /* 默认为 0 */

    oled_wr_byte(0x40, OLED_CMD);    /* 设置显示开始行 [5:0],行数. */

    oled_wr_byte(0x8D, OLED_CMD);    /* 电荷泵设置 */
    oled_wr_byte(0x14, OLED_CMD);    /* bit2，开启/关闭 */
    oled_wr_byte(0x20, OLED_CMD);    /* 设置内存地址模式 */
    oled_wr_byte(0x02, OLED_CMD);    /* [1:0],00，列地址模式;01，行地址模式;10,页地址
                                        模式;默认 10; */
    oled_wr_byte(0xA1, OLED_CMD);    /* 段重定义设置,bit0:0,0->0;1,0->127; */
    oled_wr_byte(0xC8, OLED_CMD);    /* 设置 COM 扫描方向;bit3:0,普通模式;1,重定义模式
                                        COM[N-1]->COM0;N:驱动路数 */
    oled_wr_byte(0xDA, OLED_CMD);    /* 设置 COM 硬件引脚配置 */
    oled_wr_byte(0x12, OLED_CMD);    /* [5:4]配置 */

    oled_wr_byte(0x81, OLED_CMD);    /* 对比度设置 */
    oled_wr_byte(0xEF, OLED_CMD);    /* 1~255;默认 0X7F (亮度设置,越大越亮) */
    oled_wr_byte(0xD9, OLED_CMD);    /* 设置预充电周期 */
    oled_wr_byte(0xf1, OLED_CMD);    /* [3:0],PHASE 1;[7:4],PHASE 2; */
    oled_wr_byte(0xDB, OLED_CMD);    /* 设置 VCOMH 电压倍率 */
    oled_wr_byte(0x30, OLED_CMD);
    /* [6:4] 000,0.65*vcc;001,0.77*vcc;011,0.83*vcc; */

    oled_wr_byte(0xA4, OLED_CMD);  /* 全局显示开启;bit0:1,开启;0,关闭;(白屏/黑屏) */
    oled_wr_byte(0xA6, OLED_CMD);  /* 设置显示方式;bit0:1,反相显示;0,正常显示 */
    oled_wr_byte(0xAF, OLED_CMD);  /* 开启显示 */
    oled_clear();
}
```

该函数的结构比较简单，开始是对 GPIO 口的初始化，这里我们用了宏定义 OLED_MODE 来决定要设置的 IO 口，后面的就是一些初始化序列了，我们按照厂家提供的资料来做就可以。值得注意一点的是，因为 OLED 是无背光的，在初始化之后，我们把显存都清空了，所以我们在屏幕上是看不到任何内容的，就像没通电一样，不要以为这就是初始化失败，要写入数据模块才会显示的。

接着，要介绍的是 oled_refresh_gram 更新显存到 OLED 函数，该函数的作用是把我们在程序中定义的二维数组 g_oled_gram 的值一次性刷新到 OLED 的显存 GRAM 中。我们在 oled.c 文件开头定义了如下一个二维数组：

```
/*
 * OLED 的显存
 * 每个字节表示 8 个像素， 128,表示有 128 列， 8 表示有 64 行， 高位表示高行数.
 * 比如:g_oled_gram[0][0],包含了第一列,第 1~8 行的数据. g_oled_gram[0][0].0,即表示坐标
   (0,0)
 * 类似的: g_oled_gram[1][0].1,表示坐标(1,1), g_oled_gram[10][1].2,表示坐标(10,10),
 * 存放格式如下(高位表示高行数).
 * [0]0 1 2 3 ... 127
 * [1]0 1 2 3 ... 127
 * [2]0 1 2 3 ... 127
 * [3]0 1 2 3 ... 127
```

```
 * [4]0 1 2 3 ... 127
 * [5]0 1 2 3 ... 127
 * [6]0 1 2 3 ... 127
 * [7]0 1 2 3 ... 127
 */
static uint8_t g_oled_gram[128][8];
```

该数组值与 OLED 显存 GRAM 值一一对应。在操作的时候我们只需要先修改该数组的值，然后再通过调用 oled_refresh_gram 函数把数组的值一次性刷新到 OLED 的 GRAM 上即可。oled_refresh_gram 函数定义如下：

```
/**
 * @brief           更新显存到 OLED
 * @param           无
 * @retval          无
 */
void oled_refresh_gram(void)
{
    uint8_t i, n;
    for (i = 0; i < 8; i++)
    {
        oled_wr_byte (0xb0 + i, OLED_CMD); /* 设置页地址（0~7） */
        oled_wr_byte (0x00, OLED_CMD);     /* 设置显示位置—列低地址 */
        oled_wr_byte (0x10, OLED_CMD);     /* 设置显示位置—列高地址 */
        for (n = 0; n < 128; n++)
        {
            oled_wr_byte(g_oled_gram[n][i], OLED_DATA);
        }
    }
}
```

oled_refresh_gram 函数先设置页地址，然后写入列地址（也就是纵坐标），然后从 0 开始写入 128 个字节，写满该页，最后循环把 8 页的内容都写入，就实现了整个从 STM32 显存到 OLED 显存的拷贝。

oled_refresh_gram 函数还调用了 oled_wr_byte 这个函数，也就是我们接着要介绍的函数：该函数和硬件相关，8080 并口模式下该函数定义如下：

```
/**
 * @brief           向 OLED 写入一个字节
 * @param           data: 要输出的数据
 * @param           cmd: 数据/命令标志 0,表示命令;1,表示数据;
 * @retval          无
 */
static void oled_wr_byte(uint8_t data, uint8_t cmd)
{
    oled_data_out(data);
    OLED_RS(cmd);
    OLED_CS(0);
    OLED_WR(0);
    OLED_WR(1);
    OLED_CS(1);
    OLED_RS(1);
}
```

8080 并口模式下的 oled_wr_byte 函数还调用 oled_data_out 函数，其定义如下：

```
/**
 * @brief           通过拼凑的方法向 OLED 输出一个 8 位数据
 * @param           data: 要输出的数据
 * @retval          无
 */
static void oled_data_out(uint8_t data)
{
    uint16_t dat = data & 0X0F;
    GPIOC->ODR &= ~(0XF << 6);          /* 清空 6~9 */
    GPIOC->ODR |= dat << 6;             /* D[3:0]-->PC[9:6] */
```

```
    GPIOC->ODR &= ~(0X1 << 11);                 /* 清空 11 */
    GPIOC->ODR |= ((data >> 4) & 0x01) << 11;   /* D4 */

    GPIOD->ODR &= ~(0X1 << 3);                  /* 清空 3 */
    GPIOD->ODR |= ((data >> 5) & 0x01) << 3;    /* D5 */

    GPIOB->ODR &= ~(0X3<<8);                    /* 清空 8,9 */
    GPIOB->ODR |= ((data >> 6) & 0x01) << 8;    /* D6 */
    GPIOB->ODR |= ((data >> 7) & 0x01) << 9;    /* D7 */
}
```

oled_data_out 函数的处理方法，就是我们前面说的，因为 OLED 的 D0~D7 不是接的连续 IO，所以必须将数据，拆分到各个 IO，以实现一次完整的数据传输，该函数就是根据我们 OLED_D[7:0]具体连接的 IO，对数据进行拆分，然后输出给对应位的各个 IO，实现并口数据输出。这种方式会降低并口速度，但是我们 OLED 模块，是单色的，数据量不是很大，所以这种方式也不会造成视觉上的影响，大家可以放心使用，但是如果是 TFTLCD，就不推荐了。

SPI 模式下该函数定义如下：

```
/**
 * @brief           向 OLED 写入一个字节
 * @param           data: 要输出的数据
 * @param           cmd: 数据/命令标志 0,表示命令;1,表示数据;
 * @retval          无
 */
static void oled_wr_byte(uint8_t data, uint8_t cmd)
{
    uint8_t i;
    OLED_RS(cmd);    /* 写命令 */
    OLED_CS(0);
    for (i = 0; i < 8; i++)
    {
        OLED_SCLK(0);
        if (data & 0x80)
        {
            OLED_SDIN(1);
        }
        else
        {
            OLED_SDIN(0);
        }
        OLED_SCLK(1);
        data <<= 1;
    }
    OLED_CS(1);
    OLED_RS(1);
}
```

两种模式下的 oled_wr_byte 函数形参是一样的。第一个形参 data 就是要写的内容。第二个形参 cmd 是通过选用 OLED_CMD 和 OLED_DATA 两个宏定义的其中一个，控制选择写命令还是写数据。两种模式下的 oled_wr_byte 函数的时序操作就是根据上面我们对 8080 接口以及 4 线 SPI 接口的时序来编写的。

g_oled_gram [128][8]二维数组中的 128 代表列数（x 坐标），而 8 代表的是页，每页又包含 8 行，总共 64 行（y 坐标），从高到低对应行数从小到大，如表 23.3.2.1 所示：

页数	x坐标 / y坐标	0	1	2	3	……	127
PAGE0	0	G[0][0].b0	G[1][0].b0	G[2][0].b0	G[3][0].b0		G[127][0].b0
	1	G[0][0].b1	G[1][0].b1	G[2][0].b1	G[3][0].b1		G[127][0].b1
	2	G[0][0].b2	G[1][0].b2	G[2][0].b2	G[3][0].b2		G[127][0].b2
	3	G[0][0].b3	G[1][0].b3	G[2][0].b3	G[3][0].b3		G[127][0].b3
	4	G[0][0].b4	G[1][0].b4	G[2][0].b4	G[3][0].b4		G[127][0].b4
	5	G[0][0].b5	G[1][0].b5	G[2][0].b5	G[3][0].b5		G[127][0].b5
	6	G[0][0].b6	G[1][0].b6	G[2][0].b6	G[3][0].b6	……	G[127][0].b6
	7	G[0][0].b7	G[1][0].b7	G[2][0].b7	G[3][0].b7		G[127][0].b7
PAGE1	8	G[0][1].b0	G[1][1].b0	G[2][1].b0	G[3][1].b0		G[127][1].b0
	9	G[0][1].b1	G[1][1].b1	G[2][1].b1	G[3][1].b1		G[127][1].b1
……	……	……					……
PAGE7	61	G[0][7].b5	G[1][7].b5	G[2][7].b5	G[3][7].b5		G[127][7].b5
	62	G[0][7].b6	G[1][7].b6	G[2][7].b6	G[3][7].b6		G[127][7].b6
	63	G[0][7].b7	G[1][7].b7	G[2][7].b7	G[3][7].b7		G[127][7].b7

表 23.3.2.1 OLED_GRAM 和 OLED 屏坐标对应关系

上表中 G 代表 OLED_GRAM，G[0][0]就表示 OLED_GRAM[0][0]。比如，我们要在 x=3，y=9 这个点写入 1，则可以用这个句子实现：

```
OLED_GRAM[3][1]|=1<<1;
```

一个通用的在点（x，y）置 1 表达式为：

```
OLED_GRAM[x][y/8]|=1<<(y%8);
```

其中 x 的范围为：0~127；y 的范围为：0~63。

因此，我们可以得出接下来介绍的这个比较重要的函数：OLED 画点函数，其定义如下：

```
/**
 * @brief       OLED 画点
 * @param       x   : 0~127
 * @param       y   : 0~63
 * @param       dot: 1 填充 0,清空
 * @retval      无
 */
void oled_draw_point(uint8_t x, uint8_t y, uint8_t dot)
{
    uint8_t pos, bx, temp = 0;
    if (x > 127 || y > 63) return;  /* 超出范围了 */

    pos = y / 8;    /* 计算 GRAM 里面的 y 坐标所在的字节, 每个字节可以存储 8 个行坐标 */

    bx = y % 8;     /* 取余数,方便计算 y 在对应字节里面的位置,及行(y)位置 */
    temp = 1 << bx;/* 高位表示高行号, 得到 y 对应的 bit 位置,将该 bit 先置 1 */

    if (dot)    /* 画实心点 */
    {
        g_oled_gram[x][pos] |= temp;
    }
    else        /* 画空点,即不显示 */
    {
        g_oled_gram[x][pos] &= ~temp;
    }
}
```

该函数有 3 个形参，前两个是横纵坐标，第三个为要写入 1 还是 0。该函数实现了我们在 OLED 模块上任意位置画点的功能。

前面我们知道取模方式是：从上到下，从左到右，高位在前。下面根据取模的方式来编写显示字符 oled_show_char 函数，其定义如下：

```
/**
 * @brief       在指定位置显示一个字符,包括部分字符
 * @param       x   : 0~127
```

```
 * @param       y    : 0~63
 * @param       size: 选择字体 12/16/24
 * @param       mode: 0,反白显示;1,正常显示
 * @retval      无
 */
void oled_show_char(uint8_t x,uint8_t y,uint8_t chr,uint8_t size,uint8_t mode)
{
    uint8_t temp, t, t1;
    uint8_t y0 = y;
    uint8_t *pfont = 0;
    /* 得到字体一个字符对应点阵集所占的字节数 */
    uint8_t csize = (size / 8 + ((size % 8) ? 1 : 0)) * (size / 2);
    chr = chr - ' ';     /* 得到偏移后的值,因为字库是从空格开始存储的,第一个字符是空格 */

    if (size == 12)      /* 调用 1206 字体 */
    {
        pfont = (uint8_t *)oled_asc2_1206[chr];
    }
    else if (size == 16)/* 调用 1608 字体 */
    {
        pfont = (uint8_t *)oled_asc2_1608[chr];
    }
    else if (size == 24)/* 调用 2412 字体 */
    {
        pfont = (uint8_t *)oled_asc2_2412[chr];
    }
    else                 /* 没有的字库 */
    {
        return;
    }

    for (t = 0; t < csize; t++)
    {
        temp = pfont[t];
        for (t1 = 0; t1 < 8; t1++)
        {
            if (temp & 0x80)oled_draw_point(x, y, mode);
            else oled_draw_point(x, y, !mode);

            temp <<= 1;
            y++;

            if ((y - y0) == size)
            {
                y = y0;
                x++;
                break;
            }
        }
    }
}
```

该函数为字符以及字符串显示的核心部分，函数中 chr = chr - ' ';这句是要得到在字符点阵数据里面的实际地址，因为我们的取模是从空格键开始的，例如 oled_asc2_1206 [0][0]，代表的是空格符开始的点阵码。在接下来的代码，我们也是按照从上到下(先 y++)，从左到右（再 x++）的取模方式来编写的，先得到最高位，然后判断是写 1 还是 0，画点；接着读第二位，如此循环，直到一个字符的点阵全部取完为止。这其中涉及到列地址和行地址的自增，根据取模方式来理解，就不难了。

oled.c 的内容比较多，其他的函数请大家自行理解，下面开始 main.c 文件的介绍。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t t = 0;
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    led_init();                             /* 初始化 LED */
    oled_init();                            /* 初始化 OLED */
    oled_show_string(0, 0, "ALIENTEK", 24);
    oled_show_string(0, 24, "0.96' OLED TEST", 16);
    oled_show_string(0, 52, "ASCII:", 12);
    oled_show_string(64, 52, "CODE:", 12);
    oled_refresh_gram();    /* 更新显示到 OLED */
   t = ' ';
    while (1)
    {
        oled_show_char(36, 52, t, 12, 1);  /* 显示 ASCII 字符 */
        oled_show_num(94, 52, t, 3, 12);   /* 显示 ASCII 字符的码值 */
        oled_refresh_gram();               /* 更新显示到 OLED */
        t++;
        if (t > '~')t = ' ';
        delay_ms(500);
        LED0_TOGGLE();  /* LED0 闪烁 */
    }
}
```

main.c 主要功能就是在 OLED 上显示一些实验信息字符，然后从空格键开始不停的循环显示 ASCII 字符集，并显示该字符的 ASCII 值。最后 LED0 闪烁提示程序正在运行。

23.4 下载验证

下载代码后，LED0 不停的闪烁，提示程序已经在运行了。同时 OLED 模块显示 ASCII 字符集等信息，如图 23.4.1 所示：

图 23.4.1 OLED 显示效果

OLED 显示了三种尺寸的字符：24*12（正点原子）、16*8（0.96’ OLED TEST）和 12*6（剩下的内容）。说明我们的实验是成功的，实现了三种不同尺寸 ASCII 字符的显示，在最后一行不停的显示 ASCII 字符以及其码值。

通过这一章的学习，我们学会了正点原子 OLED 模块的使用，在调试代码的时候，又多了一种显示信息的途径，在以后的程序编写中，大家可以好好利用。

第二十四章 内存保护（MPU）实验

STM32 的 Cortex M4（STM32F3/F4 系列）和 Cortex M7（STM32H7/H7 系列）系列的产品，都带有内存保护单元（memory protection unit），简称：MPU。使用 MPU 可以设置不同存储区域的存储器访问特性（如只支持特权访问或全访问）和存储器属性（如可缓存、可共享），从而提高嵌入式系统的健壮性，使系统更加安全。接下来，我们将以 STM32H743 为例，给大家介绍 STM32H7 内存保护单元（MPU）的使用。

本章分为如下几个小节：

24.1 MPU 简介
24.2 硬件设计
24.3 程序设计
24.4 下载验证

24.1 MPU 简介

MPU，即内存保护单元，可以设置不同存储区域的存储器访问特性（如只支持特权访问或全访问）和存储器属性（如可缓存、可缓冲、可共享），对存储器（主要是内存和外设）提供保护，从而提高系统可靠性：

1，阻止用户应用程序破坏操作系统使用的数据。
2，阻止一个任务访问其他任务的数据区，从而隔离任务。
3，可以把关键数据区域设置为只读，从根本上解决被破坏的可能。
4，检测意外的存储访问，如堆栈溢出，数组越界等。
5，将 SRAM 或者 RAM 空间定义为不可执行（用不执行，XN），防止代码注入攻击。

注意，MPU 不仅可以保护内存区域（SRAM 区），还可以保护外设区（比如 FMC）。我们可以通过 MPU 设置存储器的访问权限，当存储器访问和 MPU 定义的访问权限相冲突的时候，则访问会被阻止，并且触发一次错误异常（一般是 MemManage 异常）。然后，在异常处理的时候，就可以确定系统是否应该复位或者执行其他操作。

STM32H7 的 MPU 提供多达 16 个可编程保护区域（region），每个区域（region）都有自己的可编程起始地址、大小及设置。MPU 功能必须开启才会有效，默认条件下，MPU 是关闭的，所以，我们要向使用 MPU，必须先打开 MPU 才行。

16 个可编程保护区域（region），一般来说是足够使用的了，如果觉得不够，每个区域（region）还可以被进一步划分为更小的子区域（sub region），另外，还允许启用一个背景区域（即没有 MPU 设置的其他所有地址空间），背景区域只允许特权访问。在启用 MPU 后，就不得再访问定义之外的地址区间，也不得访问未经授权的区域（region），否则，将以“访问违例”处理，触发 MemManage 异常。

此外，MPU 定义的区域（region）还可以相互交迭。如果某块内存落在多个区域（region）中，则访问属性和权限将由编号最大的 region 来决定。比如，若 2 号 region 与 5 号 region 交迭，则交迭的部分受 5 号 region 控制。

MPU 设置是由 CTRL、RNR、RBAR 和 RASR 等寄存器控制的，接下来，我们分别介绍一下这几个寄存器。

首先是 MPU 控制寄存器（CTRL），该寄存器只有最低三位有效，其描述如表 24.1.1 所示：

位段	名称	类型	复位值	描述
2	PRIVDEFENA	RW	0	是否为特权级打开缺省存储器映射（即背景 region）。 1=特权级下打开背景 region 0=不打开背景 region。任何访问违例以及对 region 外地址区的访问都将引起 fault
1	HFNMIENA	RW	0	1=在 NMI 和硬 fault 服务例程中不强制除能 MPU 0=在 NMI 和硬 fault 服务例程中强制除能 MPU
0	ENABLE	RW	0	使能 MPU

表 24.1.1 MPU_CTRL 寄存器各位描述

PRIVDEFENA 位用于设置是否开启背景区域（region），通过设置该位为 1，可以在没有建立任何 region 就使能 MPU 的情况下，依然允许特权级程序访问所有地址，而只有用户级程序被卡死。但是，如果设置了其它的 region（最多 16 个 region）并使能 MPU，则背景 region 与这些 region 重合的部分，就要受各 region 的限制。HFNMIENA 位用于控制是否在 NMI 和硬件 fault 中断服务例程中禁止 MPU，一般设置为 0 即可。ENABLE 位，则用于控制是否使能 MPU，我们一般在 MPU 配置完）以后，才对其进行使能，从而开启 MPU。

接下来，介绍 MPU 区域编号寄存器（RNR），该寄存器只有低 8 位有效，其描述如表 24.1.2 所示：

位段	名称	类型	复位值	描述
7:0	REGION	RW	-	选择下一个要配置的 region。因为只支持 16 个 region，所以事实上只有[3:0]有意义

表 24.1.2 MPU_RNR 寄存器各位描述

在配置任何一个区域（region）之前，必须先在 MPU 内选中这个区域，我们可以通过将区域编号写入 MPU_RNR 寄存器来完成这个操作。该寄存器只有低 8 位有效，不过由于 STM32H7 最多只支持 16 个区域，所以，实际上只有最低 4 位有效（0~15）。在配置完区域编号以后，我们就可以对区域属性进行设置了。

接下来，介绍 MPU 基地址寄存器（RBAR），该寄存器各位描述如表 24.1.3 所示：

位段	名称	类型	复位值	描述
31:N	ADDR	RW	-	Region 基址字段。N 取决于 region 容量，以使**基址在数值上能被容量整除**。在 MPU region 属性及容量寄存器中有个 SZENABLE 位段，它决定 ADDR 中有多少个位被采用。
4	VALID	RW	-	决定是否理会写入REGION字段的值 1=MPU region号寄存器被REGION覆盖 0=MPU region号寄存器的值保持不变
3:0	REGION	RW	-	MPU region 覆写位段

表 24.1.3 MPU_RBAR 寄存器各位描述

注意，表中 ADDR 字段所设置的基址必须对齐到区域（region）容量的边界。例如，我们定义某个 region 的容量是 64KB（通过 RASR 寄存器设置），那么它的基址（ADDR）就必须能被 64KB 整除，比如 0X0001 0000、0X0002 0000、0X0003 0000 等（低 16 位全为 0）。

VALID 用于控制 REGION 段（bit[3:0]）的数据是否有效，如果 VALID=1，则 REGION 段的区域编号将覆盖 MPU_RNR 寄存器所设置的区域编号，否则将使用 MPU_RNR 所设置的区域编号。我们一般设置 VALID 为 0，这样 MPU_RBAR 寄存器的低 5 位就没有用到。

特别注意，表 24.1.3 中的 N 值最少也是 5，所以，基址必须是 32 的倍数，从而可以知道我们设置 region 的容量，必须是 32 字节的倍数。

最后，我们介绍 MPU 区域属性和容量寄存器（RASR），该寄存器各位描述如表 24.1.4 所

示：

位段	长度	名称	功能
31:29	3	-	保留
28	1	XN	1=此区禁止取指 0=此区允许取指
27	1	-	保留
26:24	3	AP	访问许可
23:22	2	—	保留
21:19	3	TEX	类型扩展
18	1	S	Sharable（可否共享） 1=可共享 0=不可共享
17	1	C	Cacheable（可否缓存） 1=可缓存 0=不可缓存
16	1	B	Buffable（可否缓冲） 1=可缓冲 0=不可缓冲
15:8	8	SRD	子region除能位段。每设置SRD的一个位，就会除能与之对应的一个子region。容量大于128字节的region都被划分成8个容量相同的子region。容量小于等于128字节的region不能再分。更多信息，请参见对子Region的论述。
7:6	2	-	保留
5:1	5	REGIONSIZE	Region容量，单位是字节。容量为1<<(REGIONSIZE+1)，但是最小容量为32字节
0	1	SZENABLE	1=使能此region　　0=除能此region

表 24.1.4 MPU_RASR 寄存器各位描述

XN 位，用于控制是否允许从此区域取指，如果 XN=1，说明禁止从区域取指，如果强行取指，将产生一个 MemManage 异常。如果设置 XN=0，则允许取指。

AP 位，由 3 个位（bit[26:24]）组成，用于控制数据的访问权限（访问许可），控制关系如表 24.1.5 所示：

值	特权级下的许可	用户级下的许可	典型用法
0b000	禁止访问	禁止访问	禁止访问
0b001	RW	禁止访问	只支持特权访问
0b010	RW	RO	禁止用户程序执行写操作
0b011	RW	RW	全访问
0b100	n/a	n/a	n/a
0b101	RO	禁止访问	仅支持特权读
0b110	RO	RO	只读
0b111	RO	RO	只读

表 24.1.5 不同 AP 设置及其访问权限

TEX、S、C 和 B 等位，对应着存储系统中比较高级的概念，可以通过对这些位段的编程，来支持多样的内存管理模型，这些位组合的详细功能如表 24.1.6 所示：

TEX	C	B	描述	存储器类型	可否共享
000	0	0	强序（严格按照顺序执行）	强序	可以
000	0	1	共享的设备（可以写缓冲）	设备	可以
000	1	0	片外或片内的"写通"型内存，非写分配	普通	S位决定
000	1	1	片外或片内的"写回"型内存，非写分配	普通	S位决定
001	0	0	片外或片内的"不可缓存"型内存	普通	S位决定
001	0	1	n/a	n/a	n/a
001	1	0	由具体实现定义	n/a	n/a
001	1	1	片外或片内的"写回"型，带读和写的分配	普通	S位决定
010	1	x	共享不可的设备	设备	总是不可
010	0	1	n/a	n/a	n/a
010	1	x	n/a	n/a	n/a
1BB	A	A	带缓存的内存。BB=适用于片外内存，AA=适用于片内内存	普通	S位决定

表 24.1.6 TEX、S、C 和 B 对存储器类型的定义

有些情况下，内部和外部内存可能需要不同的缓存策略，次数需要设置 TEX 的第二位为 1，这样 TEX[1:0]的定义就会变为外部策略表（表 24.1.6 种表示为 BB），而 C 和 B 位则会变为内部策略表（表 24.1.6 种表示为 AA）。缓存策略的定义（AA 和 BB）如表 24.1.7 所示：

存储器属性编码 (AA and BB)	高速缓存策略
00	不可共享
01	写回，读写均有分配
10	写通，无写分配
11	写回，无写分配

表 24.1.7 TEX 最高位为 1 时内外缓存策略编码

S 位用于控制存储器的共享特性，设置 S=1，则二级存储器不可以缓存（Cache），如果设置 S=0，则可以缓存（Cache），一般我们设置该位为 0 即可。

C 位用于控制存储器的缓存特性，也就是是否可以 Cache，STM32H7 自带 Cache，如果我们想要某个存储器可以被 Cache，则必须设置 C=1。此位需要根据具体的需要设置。

B 位用于控制存储器的缓冲特性，设置 B=1，则二级存储器可以缓冲，即写回模式，设置 B=0，则二级存储器不可以缓冲，即写通模式。此位需根据具体的需要进行设置。

SRD[15:8]，这 8 个位用于控制子区域（sub region）使能。前面提到，STM32H7 的 MPU 最多支持 8 个 region，有时候可能不够用，通过子区域的概念，可以将每个 region 的内部进一步划分成更小的块，这就是 sub region，每个 sub region 可以独立地使能或除能（相当于可以部分地使能一个 region）。

sub region 的使用必须满足：

1，每个 region 必须 8 等分，每份是一个 sub region，其属性与主 region 完全相同。

2，可以被分为 8 个 sub region 的 region，其大小必须大于等于 256 字节。

SRD 中的 8 个位，每个位控制一个 sub region 是否被除能。如 SRD.4=0，则 4 号 sub region 被除能。如果某个 sub region 被除能，且其对应的地址范围又没有落在其它 region 中，则对该区的访问将引发 fault。

REGIONSIZE[5:1]，这 5 个位用于控制 region 的容量（大小），计算关系如下：

$$rsize = 2 \wedge (REGIONSIZE + 1)$$

rsize 即 region 的容量，必须大于等于 32 字节，即 REGIONSIZE 必须大于等于 4。region 的容量范围为：32B~4GB，根据实际需要进行设置。

SZENABLE 位，用于设置 region 的使能。该位一般最后设置，设置为 1，则启用此 region，

使能 MPU 保护。

至此，关于 MPU 的简介就介绍完了，关于 MPU 更详细的说明，请参考：《STM32H7 编程手册.pdf》 、《STM32 MPU 说明》和《Cortex M3 权威指南(中文)》第 14 章。

24.2 硬件设计

1. 例程功能

本实验使用 STM32H7 自带的 MPU 功能，对一个特定的内存空间（数组，地址：0X20002000）进行写访问保护。开机时，串口调试助手显示：MPU closed，表示默认是没有写保护的。按 KEY0 可以往数组里面写数据，按 KEY1，可以读取数组里面的数据。按 KEY_UP 则开启 MPU 保护，此时，如果再按 KEY0 往数组写数据，就会引起 MemManage 错误，进入 MemManage_Handler 中断服务函数，此时 LED1 点亮，同时打印错误信息，最后软件复位，系统重启。LED0 用于提示程序正在运行，所有信息都是通过串口 1 输出，需要用串口调试助手查看。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
LED1：LED1 - PB0
2）独立按键
KEY0 - PH3
KEY1 - PH2
KEY2 - PC13
WK_UP - PA0
3）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)

3. 原理图

MPU 属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们借助按键和 LED 灯验证 MPU 工作是否正常，然后通过电脑串口上位机软件观察打印出来的信息。

24.3 程序设计

24.3.1 MPU 的 HAL 库驱动

MPU 在 HAL 库中的驱动代码在 stm32h7xx_hal_cortex.c 文件（及其头文件）中。

1. HAL_MPU_ConfigRegion 函数

MPU 的初始化函数，其声明如下：

```
void HAL_MPU_ConfigRegion(MPU_Region_InitTypeDef *MPU_Init);
```

- **函数描述：**
  用于配置 MPU。
- **函数形参：**
  形参 1 是 MPU_Region_InitTypeDef 结构体类型指针变量，其定义如下：

```
typedef struct
{
  uint8_t                 Enable;                /* 区域使能/禁止 */
  uint8_t                 Number;                /* 区域编号 */
  uint32_t                BaseAddress;           /* 配置区域基地址 */
  uint8_t                 Size;                  /* 区域容量 */
  uint8_t                 SubRegionDisable;      /* 子 region 除能位段设置 */
  uint8_t                 TypeExtField;          /* 类型扩展级别 */
  uint8_t                 AccessPermission;      /* 设置访问权限 */
  uint8_t                 DisableExec;           /* 允许/禁止取指 */
```

```
  uint8_t                 IsShareable;          /* 禁止/允许共享 */
  uint8_t                 IsCacheable;          /* 禁止/允许缓存 */
  uint8_t                 IsBufferable;         /* 禁止/允许缓冲 */
}MPU_Region_InitTypeDef;
```

该结构体成员变量很多，每个成员变量的含义我们都在上面定义中有注释。这里大家注意，除了 BaseAddress 和 Number 两个成员变量是分别用来配置 MPU->RBAR 和 MPU->RNR 寄存器之外，其他成员变量都是用来配置 MPU->RASR 寄存器相关位，大家如果对这些配置项不理解的话，可以直接对照我们前面讲解的寄存器 MPU->RASR 各个位含义来理解。

- **函数返回值：**

  无

2. HAL_MPU_Enable 函数

HAL_MPU_Enable 函数是 MPU 使能函数。其声明如下：

```
void HAL_MPU_Enable(uint32_t MPU_Control);
```

- **函数描述：**

  该函数用于使能 MPU，形参一般使用 MPU_PRIVILEGED_DEFAULT。

- **函数形参：**

  形参 1 是 uint32_t 类型变量，共有四个选择，对应设置如表 24.1.1 的 MPU_CTRL 寄存器，四个选择参数设置情况如下：

```
#define  MPU_HFNMI_PRIVDEF_NONE       ((uint32_t)0x00000000)
```

该参数设置 MPU 的 CTL 控制寄存器的 PRIVDEFENA 位为 0，即禁止了背景区，访问任何未使能 MPU 的区域均会造成内存异常 MemFault。

此参数设置 MPU 的 CTL 控制寄存器的 HFNMIENA 位为 0，即 NMI 不可屏蔽中断服务程序和硬件异常中断服务程序执行期间会强制关闭 MPU。

```
#define  MPU_HARDFAULT_NMI            ((uint32_t)0x00000002)
```

该参数设置 MPU 的 CTL 控制寄存器的 PRIVDEFENA 位为 0，即禁止了背景区，访问任何未使能 MPU 的区域均会造成内存异常 MemFault。

此参数设置 MPU 的 CTL 控制寄存器的 HFNMIENA 位为 1，即 NMI 不可屏蔽中断服务程序和硬件异常中断服务程序执行期间会继续开启 MPU。

```
#define  MPU_PRIVILEGED_DEFAULT       ((uint32_t)0x00000004)
```

该参数设置 MPU 的 CTL 控制寄存器的 PRIVDEFENA 位为 1，即使能了背景区，特权级下可以正常访问任何未使能 MPU 的区域。

此参数设置 MPU 的 CTL 控制寄存器的 HFNMIENA 位为 0，即 NMI 不可屏蔽中断服务程序和硬件异常中断服务程序执行期间会强制关闭 MPU。

```
#define  MPU_HFNMI_PRIVDEF            ((uint32_t)0x00000006)
```

该参数设置 MPU 的 CTL 控制寄存器的 PRIVDEFENA 位为 1，即使能了背景区，特权级下可以正常访问任何未使能 MPU 的区域。

此参数设置 MPU 的 CTL 控制寄存器的 HFNMIENA 位为 1，即 NMI 不可屏蔽中断服务程序和硬件异常中断服务程序执行期间会继续开启 MPU。

- **函数返回值：**

  无

3. HAL_MPU_Disable 函数

HAL_MPU_Disable 函数是 MPU 失能函数。其声明如下：

```
void HAL_MPU_Disable (void);
```

- **函数描述：**

  该函数用于禁止 MPU，我们配置 MPU 前需要先调用该函数禁止 MPU，才能进行配置。

- **函数形参：**

  无

- **函数返回值：**

  无

MPU 配置步骤

1）禁止 MPU 以及 MemManage 中断

调用 HAL_MPU_Disable 函数禁止 MPU 以及 MemManage 中断。

2）配置某个区域的 MPU 保护参数

在进行 MPU 配置之前我们必须先通过 MPU_RNR 区域编号寄存器用来选择下一个要配置的区域，然后通过配置 MPU_RBAR 基地址寄存器来配置基地址，最后通过区域属性和容量寄存器 RASR 来配置区域相关属性和参数。这些过程都可以通过调用 HAL_MPU_ConfigRegion 函数来完成。除了 BaseAddress 和 Number 两个成员变量是分别用来配置 MPU->RBAR 和 MPU->RNR 寄存器之外，其他成员变量都是用来配置 MPU->RASR 寄存器相关位，大家如果对这些配置项不理解的话，可以直接对照我们前面讲解的寄存器 MPU->RASR 各个位含义来理解。

3）使能 MPU 以及 MemManage 中断

调用 HAL_MPU_Enable 函数禁止 MPU 以及 MemManage 中断。

24.3.2 程序流程图

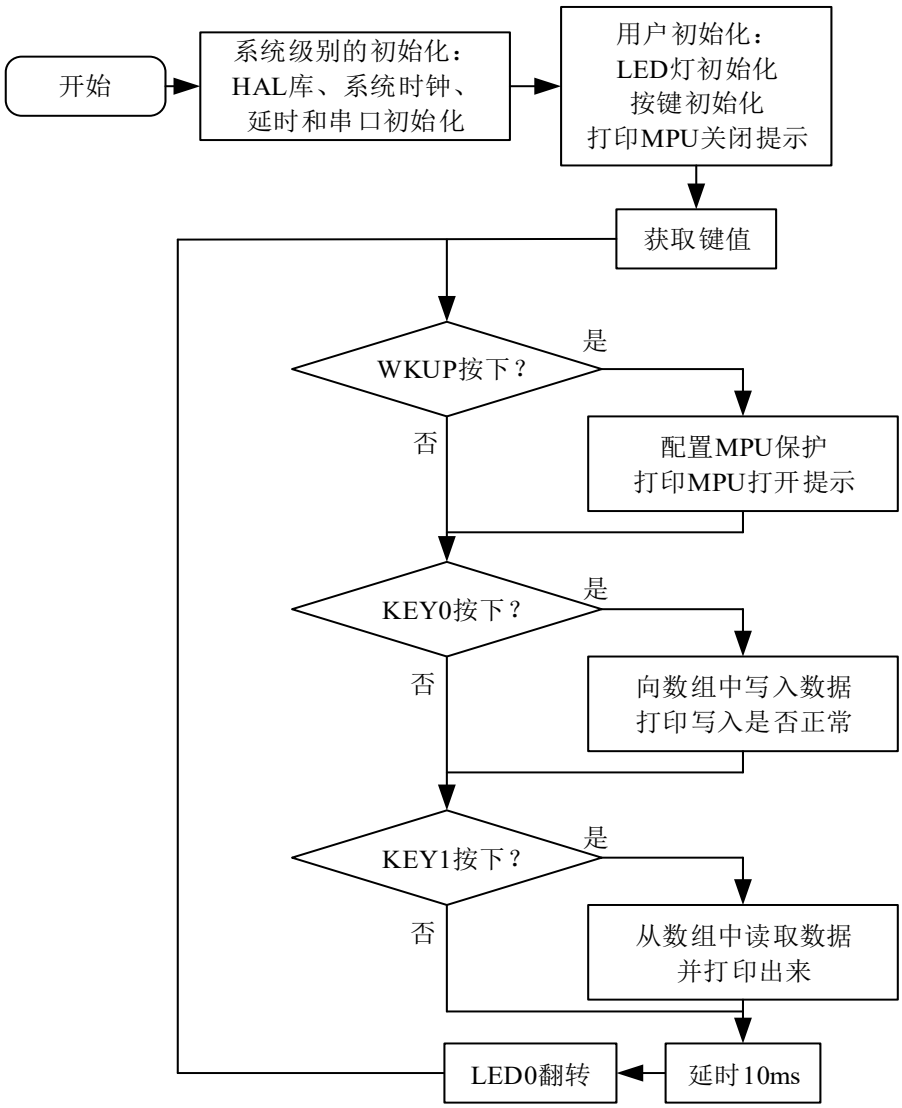


图 24.3.2.1 内存保护（MPU）实验程序流程图

24.3.3 程序解析

1. MPU 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。MPU 驱动源码包括两个文件：mpu.c 和 mpu.h。

mpu.h 头文件只有函数声明，就不看了，所以我们直接看 mpu.c 文件，先看设置某个区域的 MPU 保护函数：

```
/**
 * @brief       设置某个区域的 MPU 保护
 * @param       baseaddr: MPU 保护区域的基址(首地址)
 * @param       size:MPU 保护区域的大小(必须是 32 的倍数,单位为字节)
 * @param       rnum:MPU 保护区编号,范围:0~7,最大支持 8 个保护区域
 * @param       de:禁止指令访问;0,允许指令访问;1,禁止指令访问
 * @param       ap:访问权限,访问关系如下:
 *   @arg       0,无访问（特权&用户都不可访问）
 *   @arg       1,仅支持特权读写访问
 *   @arg       2,禁止用户写访问（特权可读写访问）
 *   @arg       3,全访问（特权&用户都可访问）
 *   @arg       4,无法预测(禁止设置为 4!!!)
 *   @arg       5,仅支持特权读访问
 *   @arg       6,只读（特权&用户都不可以写）
 *   @note      详见:STM32H7 编程手册.pdf,4.6.6 节,Table 91.
 * @param       sen:是否允许共用;0,不允许;1,允许
 * @param       cen:是否允许 cache;0,不允许;1,允许
 * @param       ben:是否允许缓冲;0,不允许;1,允许
 * @retval      0, 成功; 1, 错误;
 */
uint8_t mpu_set_protection(uint32_t baseaddr, uint32_t size, uint32_t rnum,
                           uint8_t de, uint8_t ap, uint8_t sen, uint8_t cen, uint8_t ben)
{
    MPU_Region_InitTypeDef mpu_region_init_handle;     /* MPU 初始化句柄 */
    HAL_MPU_Disable();              /* 配置 MPU 之前先关闭 MPU,配置完成以后在使能 MPU */

    mpu_region_init_handle.Enable = MPU_REGION_ENABLE;/* 使能该保护区域 */
    mpu_region_init_handle.Number = rnum;             /* 设置保护区域 */
    mpu_region_init_handle.BaseAddress = baseaddr;    /* 设置基址 */
    mpu_region_init_handle.Size = size;               /* 设置保护区域大小 */
    mpu_region_init_handle.SubRegionDisable = 0X00;   /* 禁止子区域 */
    mpu_region_init_handle.TypeExtField = MPU_TEX_LEVEL0;
    /* 设置类型扩展域为 level0 */
    mpu_region_init_handle.AccessPermission = ap;     /* 设置访问权限 */
    mpu_region_init_handle.DisableExec = de;          /* 是否允许指令访问 */
    mpu_region_init_handle.IsShareable = sen;         /* 是否允许共用 */
    mpu_region_init_handle.IsCacheable = cen;         /* 是否允许 cache */
    mpu_region_init_handle.IsBufferable = ben;        /* 是否允许缓冲 */
    HAL_MPU_ConfigRegion(&mpu_region_init_handle);    /* 配置 MPU */
    HAL_MPU_Enable(MPU_PRIVILEGED_DEFAULT);           /* 开启 MPU */
    return 0;
}
```

该函数使用 HAL_MPU_ConfigRegion 函数对 MPU_Region_InitTypeDef 结构体变量进行配置。注意：配置之前我们要调用 HAL_MPU_Disable 函数禁止 MPU 才能进行配置，最后再调用 HAL_MPU_Enable 函数开启 MPU。

接着，要介绍的是 mpu_memory_protection 函数，其定义如下：

```
/**
 * @brief       设置需要保护的存储块
 * @param       无
```

```
 * @note        必须对部分存储区域进行 MPU 保护,否则可能导致程序运行异常
 *              比如 MCU 屏不显示,摄像头采集数据出错等等问题...
 */
void mpu_memory_protection(void)
{
   /* 保护整个 DTCM,共 128K 字节 */
    mpu_set_protection( 0x20000000,                 /* 基地址 */
                        MPU_REGION_SIZE_128KB,      /* 长度 */
                        MPU_REGION_NUMBER1, 0,      /* NUMER1,允许指令访问 */
                        MPU_REGION_FULL_ACCESS,     /* 全访问 */
                        MPU_ACCESS_NOT_SHAREABLE,   /* 禁止共用 */
                        MPU_ACCESS_CACHEABLE,       /* 允许 cache */
                        MPU_ACCESS_BUFFERABLE);     /* 允许缓冲 */

    /* 保护整个 AXI SRAM,共 512K 字节 */
    mpu_set_protection( 0x24000000,                 /* 基地址 */
                        MPU_REGION_SIZE_512KB,      /* 长度 */
                        MPU_REGION_NUMBER2, 0,      /* NUMER2,允许指令访问 */
                        MPU_REGION_FULL_ACCESS,     /* 全访问 */
                        MPU_ACCESS_SHAREABLE,       /* 允许共用 */
                        MPU_ACCESS_CACHEABLE,       /* 允许 cache */
                        MPU_ACCESS_NOT_BUFFERABLE); /* 禁止缓冲 */

    /* 保护整个 SRAM1~SRAM3,共 512K 字节 */
    mpu_set_protection( 0x30000000,                 /* 基地址 */
                        MPU_REGION_SIZE_512KB,      /* 长度 */
                        MPU_REGION_NUMBER3, 0,      /* NUMER3,允许指令访问 */
                        MPU_REGION_FULL_ACCESS,     /* 全访问 */
                        MPU_ACCESS_NOT_SHAREABLE,   /* 禁止共用 */
                        MPU_ACCESS_CACHEABLE,       /* 允许 cache */
                        MPU_ACCESS_BUFFERABLE);     /* 允许缓冲 */

    /* 保护整个 SRAM4,共 64K 字节 */
    mpu_set_protection( 0x38000000,                 /* 基地址 */
                        MPU_REGION_SIZE_64KB,       /* 长度 */
                        MPU_REGION_NUMBER4, 0,      /* NUMER4,允许指令访问 */
                        MPU_REGION_FULL_ACCESS,     /* 全访问 */
                        MPU_ACCESS_NOT_SHAREABLE,   /* 禁止共用 */
                        MPU_ACCESS_CACHEABLE,       /* 允许 cache */
                        MPU_ACCESS_BUFFERABLE);     /* 允许缓冲 */

    /* 保护 MCU LCD 屏所在的 FMC 区域,共 64M 字节 */
    mpu_set_protection( 0x60000000,                 /* 基地址 */
                        MPU_REGION_SIZE_64MB,       /* 长度 */
                        MPU_REGION_NUMBER5, 0,      /* NUMER5,允许指令访问 */
                        MPU_REGION_FULL_ACCESS,     /* 全访问 */
                        MPU_ACCESS_NOT_SHAREABLE,   /* 禁止共用 */
                        MPU_ACCESS_NOT_CACHEABLE,   /* 禁止 cache */
                        MPU_ACCESS_NOT_BUFFERABLE); /* 禁止缓冲 */

    /* 保护 SDRAM 区域,共 32M 字节 */
    mpu_set_protection( 0xC0000000,                 /* 基地址 */
                        MPU_REGION_SIZE_32MB,       /* 长度 */
                        MPU_REGION_NUMBER6, 0,      /* NUMER6,允许指令访问 */
                        MPU_REGION_FULL_ACCESS,     /* 全访问 */
                        MPU_ACCESS_NOT_SHAREABLE,   /* 禁止共用 */
                        MPU_ACCESS_CACHEABLE,       /* 允许 cache */
                        MPU_ACCESS_BUFFERABLE);     /* 允许缓冲 */
```

```
    /* 保护整个 NAND FLASH 区域,共 256M 字节 */
    mpu_set_protection( 0x80000000,                  /* 基地址 */
                        MPU_REGION_SIZE_256MB,       /* 长度 */
                        MPU_REGION_NUMBER7, 1,       /* NUMER7,禁止指令访问 */
                        MPU_REGION_FULL_ACCESS,      /* 全访问 */
                        MPU_ACCESS_NOT_SHAREABLE,    /* 禁止共用 */
                        MPU_ACCESS_NOT_CACHEABLE,    /* 禁止 cache */
                        MPU_ACCESS_NOT_BUFFERABLE); /* 禁止缓冲 */
}
```

mpu_memory_protection 函数，用于设置整个代码里面，我们需要保护的存储块，这里我们对 7 个存储块（使用了 7 个区域（region））进行了保护：

1，从 0x20000000 地址开始的 128KB DTCM 地址空间，允许指令访问，禁止共用，允许 cache，允许缓冲。

2，从 0x24000000 地址开始的 512KB AXI SRAM 地址空间，允许指令访问，允许共用，允许 cache，禁止缓冲。

3，从 0x30000000 地址开始的 512KB SRAM1~SRAM3 地址空间，允许指令访问，禁止共用，允许 cache，允许缓冲。

4，从 0x38000000 地址开始的 64KB SRAM4 地址空间，允许指令访问，禁止共用，允许 cache，允许缓冲。

5，从 0x60000000 地址开始的 64MB 地址空间，允许指令访问，禁止共用，禁止 cache，禁止缓冲，保护 MCU LCD 屏的访问地址区间，如不进行设置，可能导致 MCU LCD 白屏。

6，从 0xC0000000 地址开始的 32MB 地址空间，即 SDRAM 的地址范围，允许指令访问，禁止共用，允许 cache，允许缓冲。

7，从 0x80000000 地址开始的 256MB 地址空间，即 NAND FLASH 区域，禁止指令访问，禁止共用，禁止 cache，禁止缓冲。

这七个地址空间的保护设置，可以提高代码的稳定性（其实就是减少使用 cache 导致的各种莫名奇妙的问题），请大家不要随意改动。此函数在本例程没有用到，不过我们在后续代码都会用到。

最后，MemManage_Handler 函数，用于处理产生 MemManage 错误的中断服务函数，在该函数里面点亮了 LED1，并输出一些串口信息，对系统进行软复位，以便观察本例程的实验结果。函数定义如下：

```
/**
 * @brief       MemManage 错误处理中断
 * @note        进入此中断以后,将无法恢复程序运行!!
 * @param       无
 * @retval      无
 */
void MemManage_Handler(void)
{
    LED1(0);                                /* 点亮 LED1(GREEN LED) */
    printf("Mem Access Error!!\r\n");       /* 输出错误信息 */
    delay_ms(1000);
    printf("Soft Reseting...\r\n");         /* 提示软件重启 */
    delay_ms(1000);
    NVIC_SystemReset();                     /* 软复位 */
}
```

mpu.c 的内容介绍就到此，下面开始 main.c 文件的介绍。

2. main.c 代码

在 main.c 文件代码如下：

```
uint8_t mpudata[128] __attribute__((at(0X20002000)));   /* 定义一个数组 */

int main(void)
{
    uint8_t key = 0;
```

```
    uint8_t t = 0;
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    led_init();                             /* 初始化 LED */
    key_init();                             /* 初始化按键 */
    printf("\r\nMPU closed!\r\n");          /* 提示 MPU 关闭 */

    while (1)
    {
        key = key_scan(0);

        if (key == WKUP_PRES)               /* 使能 MPU 保护数组 mpudata */
        {
            mpu_set_protection(0X20002000, MPU_REGION_SIZE_128B,
                  MPU_REGION_NUMBER0, 0, MPU_REGION_PRIV_RO_URO,
                  MPU_ACCESS_NOT_SHAREABLE, MPU_ACCESS_NOT_CACHEABLE,
                  MPU_ACCESS_BUFFERABLE); /* 只读,禁止共用,禁止 catch,允许缓冲 */
            printf("MPU open!\r\n");        /* 提示 MPU 打开 */
        }
        /* 向数组中写入数据，如果开启了 MPU 保护的话会进入内存访问错误！ */
        else if (key == KEY0_PRES)
        {
            printf("Start Writing data...\r\n");
            sprintf((char *)mpudata, "MPU test array %d", t);
            printf("Data Write finshed!\r\n");
        }
        /* 从数组中读取数据，不管有没有开启 MPU 保护都不会进入内存访问错误！ */
        else if (key == KEY1_PRES)
        {
            printf("Array data is:%s\r\n", mpudata);
        }
        else
        {
            delay_ms(10);
        }
        t++;
        if ((t % 50) == 0) LED0_TOGGLE();    /* LED0 取反 */
    }
}
```

在 main 函数前面，我们定义了一个 128 字节大小的数组：mpudata，其首地址为 0X20002000，默认情况下，MPU 保护关闭，可以对该数组进行读写访问。当我们按下 KEY_UP 按键的时候，通过 mpu_set_protection 函数，对其 0X20002000 为起始地址，大小为 128 字节的内存空间进行保护，仅支持特权读访问，此时如果再按 KEY0，对数组进行写入操作，则会引起 MemManage 访问异常，进入 MemManage_Handler 中断服务函数，执行相关操作。

其他的代码比较简单，这里就不多做说明了，在整个代码编译通过之后，我们就可以开始下载验证了。

24.4 下载验证

下载代码后，LED0 不停的闪烁，提示程序已经在运行了。然后，打开串口调试助手（XCOM V2.7），设置串口为开发板的 USB 转串口（CH340 虚拟串口，得根据你自己的电脑选择，我的电脑是 COM8，另外，请注意：波特率是 115200），可以看到如图 24.4.1 所示信息（如果没有提示信息，请先按复位）：

图 24.4.1 复位后串口调试助手收到的信息

从图 24.4.1 可以看出，此时串口助手提示：MPU Closed，即 MPU 保护是关闭的，我们可以按 KEY0 往数组里面写入数据，按 KEY1，可以读取刚刚写入的数据，按 KEY_UP，则开启 MPU 保护，提示：MPU open!，此时，如果再按 KEY0，往数组里面写数据的话，则会引起 MemManage 访问异常，进入 MemManage_Handler 中断服务函数，点亮 LED1，并提示：Mem Access Error!!，并在 1 秒钟以后，重启系统（软复位），如图 24.4.2 所示：

图 24.4.2 串口调试助手显示运行结果

整个过程，验证了我们代码的正确性，通过 MPU 实现了对特定内存的写保护功能。通过 MPU，我们可以提高系统的可靠性，使代码更加安全的运行。

第二十五章 TFTLCD（MCU 屏）实验

前面我们介绍了 OLED 模块及其显示，但是该模块只能显示单色/双色，不能显示彩色，而且尺寸也较小。本章我们将介绍正点原子的 TFT LCD 模块（MCU 屏），该模块采用 TFTLCD 面板，可以显示 16 位色的真彩图片。在本章中，我们将使用开发板底板上的 TFTLCD 接口（仅支持 MCU 屏，本章仅介绍 MCU 屏的使用），来点亮 TFTLCD，并实现 ASCII 字符和彩色的显示等功能，并在串口打印 LCD 控制器 ID，同时在 LCD 上面显示。

本章分为如下几个小节：

25.1 TFTLCD 和 FMC 简介
25.2 硬件设计
25.3 程序设计
25.4 下载验证

25.1 TFTLCD 和 FMC 简介

本章我们将通过 STM32H743 的 FMC 接口来控制 TFTLCD 的显示，所以本节分为两个部分，分别介绍 TFTLCD 和 FMC。

25.1.1 TFTLCD 简介

液晶显示器，即 Liquid Crystal Display，利用了液晶导电后透光性可变的特性，配合显示器光源、彩色滤光片和电压控制等工艺，最终可以在液晶阵列上显示彩色的图像。目前液晶显示技术以 TN、STN、TFT 三种技术为主，TFT-LCD 即采用了 TFT（Thin Film Transistor）技术的液晶显示器，也叫薄膜晶体管液晶显示器。

TFT-LCD 与无源 TN-LCD、STN-LCD 的简单矩阵不同的是，它在液晶显示屏的每一个象素上都设置有一个薄膜晶体管（TFT），可有效地克服非选通时的串扰，使显示液晶屏的静态特性与扫描线数无关，因此大大提高了图像质量。TFT 式显示器具有很多优点：高响应度，高亮度，高对比度等等。TFT 式屏幕的显示效果非常出色，广泛应用于手机屏幕、笔记本电脑和台式机显示器上。

由于液晶本身不会发光，加上液晶本身的特性等原因，使得液晶屏的成像角受限，我们从屏幕的的一侧可能无法看清液晶的显示内容。液晶显示器的成像角的大小也是评估一个液晶显示器优劣的指标，目前，规格较好的液晶显示器成像角一般在 120°~160°之间。

正点原子 TFT-LCD 模块(MCU 屏)有如下特点：

1，2.8’/3.5’/4.3’/7’ 等 4 种大小的屏幕可选。
2，320×240 的分辨率（3.5’分辨率为:320*480，4.3’和 7’分辨率为：800*480）。
3，16 位真彩显示。
4，自带触摸屏，可以用来作为控制输入。

本章，我们以正点原子 2.8 寸（此处的寸是代表英寸，下同）的 TFT-LCD 模块为例介绍，（其他尺寸的 LCD 可参考具体的 LCD 型号的资料，也比较类似），该模块支持 65K 色显示，显示分辨率为 320×240，接口为 16 位的 8080 并口，自带触摸功能。

该模块的外观图如图 25.1.1.1 所示：

图 25.1.1.1 正点原子 2.8 寸 TFTLCD 外观图

模块原理图如图 25.1.1.2 所示：

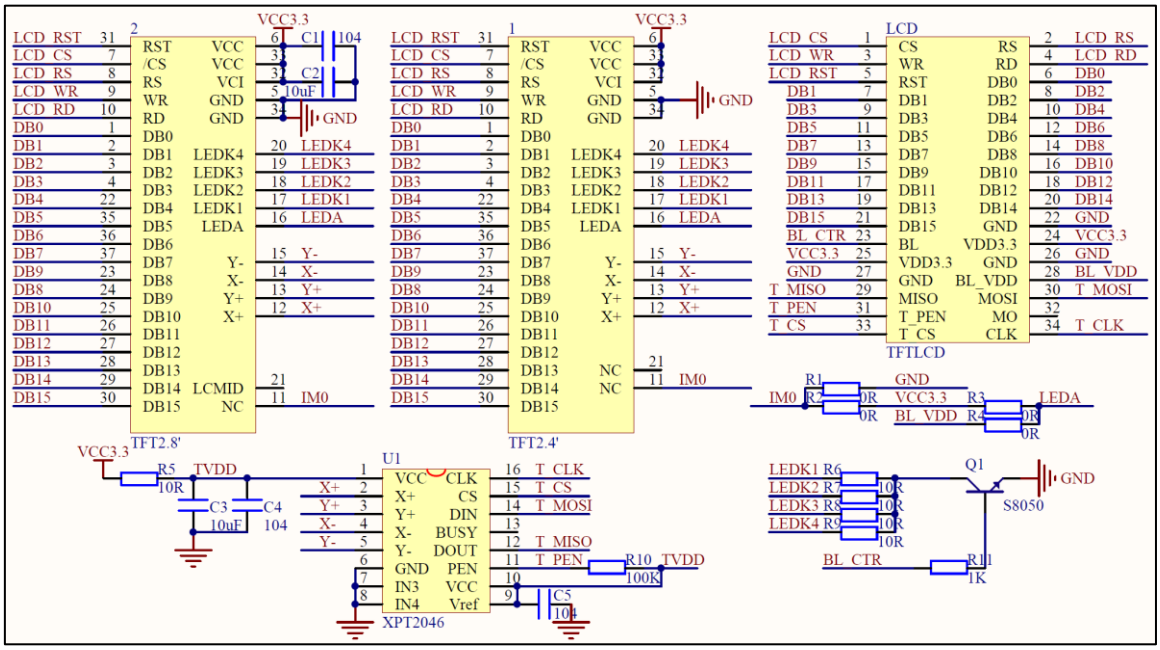


图 25.1.1.2 正点原子 TFTLCD 模块原理图

TFTLCD 模块采用 2*17 的 2.54 公排针与外部连接，即图中 TFT_LCD 部分。从图 25.1.1.2 可以看出，正点原子 TFTLCD 模块采用 16 位的并方式与外部连接。图 25.1.1.2 还列出了触摸控制的接口，但触摸控制是在显示的基础上叠加的一个控制功能，不配置也不会对显示造成影响，我们放到以后的章节再介绍触摸的用法。该模块与显示功能有关的信号线如表 25.1.1.1：

名称	功能
CS	TFT-LCD 片选信号。
WR	向 TFT-LCD 写入数据。
RD	从 TFT-LCD 读取数据。
DB[15:0]	16 位双向数据线。
RST	硬复位 TFT-LCD。
RS	命令/数据标志（0，读写命令；1，读写数据）。
BL	背光控制。

表 25.1.1.1 TFT-LCD 接口信号线

上述的接口线实际是对应到液晶显示控制器上的，这个芯片位于液晶屏的下方，所以我们从外观图上看不到。控制 LCD 显示的过程，就是按其显示驱动芯片的时序，把色彩和位置信息正确地写入对应的寄存器。

25.1.2 液晶显示控制器

正点原子提供 2.8/3.5/4.3/7 寸等 4 种不同尺寸和分辨率的 TFTLCD 模块，其驱动芯片为：ILI9341/ST7789/NT35310/NT35510/SSD1963 等(具体的型号，大家可以通过下载本章实验代码，通过串口或者 LCD 显示查看)，这里我们仅以 ILI9341 控制器为例进行介绍，其他的控制基本都类似，我们就不详细阐述了。

ILI9341 液晶控制器自带显存，可配置支持 8/9/16/18 位的总线中的一种，可以通过 3/4 线串行协议或 8080 并口驱动。正点原子的 TFTLCD 模块上的电路配置为 8080 并口方式，其显存总大小为 172800（240*320*18/8），即 18 位模式（26 万色）下的显存量。在 16 位模式下，ILI9341 采用 RGB565 格式存储颜色数据，此时 ILI9341 的 18 位显存与 MCU 的 16 位数据线以及 RGB565 的对应关系如图 25.1.2.1 所示：

9341 显存	B17	B16	B15	B14	B13	B12	B11	B10	B9	B8	B7	B6	B5	B4	B3	B2	B1	B0
RGB565 GRAM	R[4]	R[3]	R[2]	R[1]	R[0]	NC	G[5]	G[4]	G[3]	G[2]	G[1]	G[0]	B[4]	B[3]	B[2]	B[1]	B[0]	NC
9341数据线	DB15	DB14	DB13	DB12	DB11		DB10	DB9	DB8	DB7	DB6	DB5	DB4	DB3	DB2	DB1	DB0	
MCU 数据线	D15	D14	D13	D12	D11		D10	D9	D8	D7	D6	D5	D4	D3	D2	D1	D0	

图 25.1.2.1 16 位数据与显存对应关系图

从图中可以看出，ILI9341 在 16 位模式下面，18 位显存的 B0 和 B12 并没有用到，对外的数据线使用 DB0-DB15 连接 MCU 的 D0-D15 实现 16 位颜色的传输（使用 8080 MCU 16bit I 型接口，详见 9341 数据手册 7.1.1 节）。

这样 MCU 的 16 位数据，最低 5 位代表蓝色，中间 6 位为绿色，最高 5 位为红色。数值越大，表示该颜色越深。另外，特别注意 ILI9341 所有的指令都是 8 位的（高 8 位无效），且参数除了读写 GRAM 的时候是 16 位，其他操作参数，都是 8 位的。

知道了屏幕的显色信息后，我们如何驱动它呢？OLED 的章节我们已经描述过 8080 方式操作的时序，我们通过《ILI9341_DS.pdf》来加深一下在 8080 并口方式下如何操作这个芯片。

以写周期为例，8080 方式下的操作时序如图 25.1.2.2 所示。

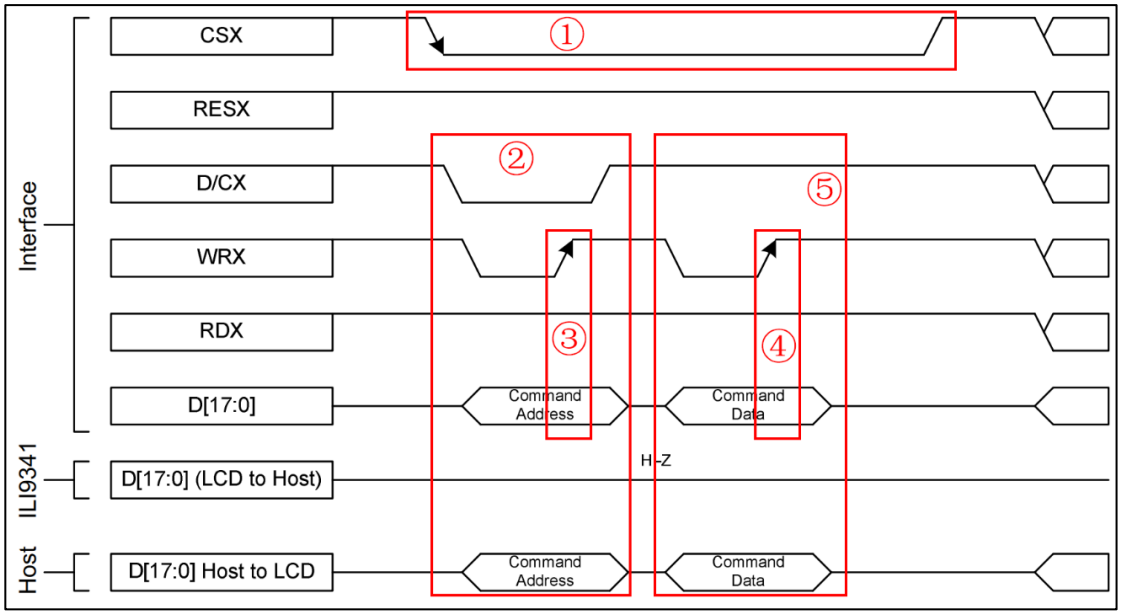


图 25.1.2.2 8080 方式下对液晶控制器的写操作

上图中的各个控制线与我们在表 25.1.1.1 提到的命名有些许差异，因为我们在原理图时往往为了方便自己记忆会对命名进行微调，为了方便读者对照，我们把图 25.1.2.2 中列出的引脚引脚与我们的 TFTLCD 模块的的对应关系再列出，如表 25.1.2.1 所示。

TFTLCD 名称	ILI9341	功能
CS	CSX	TFT-LCD 片选信号。
WR	WRX	向 TFT-LCD 写入数据。
RD	RDX	从 TFT-LCD 读取数据。
D[15:0]	D[15:0]	16 位双向数据线。

RS	D/CX	命令/数据标志（0，读写命令；1，读写数据）。

表 25.1.2.1 TFT-LCD 引脚与液晶控制器的对应关系

这下我们再来分析一下图 25.1.2.2 所示的写操作的时序，控制液晶的主机，在整个写周期内需要控制片选 CSX 拉低（标注为①），之后对其它的控制线的电平才有效。在标号②表示的这个写命令周期中，D/CX 被位低（参考 ILI9341 的引脚定义），同时把命令码通过数据线 D[17:0]（我们实际只用了 16 个引脚）按位编码。注意到③处，需要数据线在入电平拉高后再操持一段时间以便数据被正确采样。

图 25.1.2.2 中⑤表示写数据操作，与前面描述的写命令操作只有 D/CX 的操作不同，读者们可以尝试自己分析一下。更多的关于 ILI9341 的读写操作时序则参考《ILI9341_DS.pdf》。

通过前述的时序分析，我们知道了对于 ILI9341 来说，控制命令有命令码、数据码之分，接下来，我们介绍一下 ILI9341 的几个重要命令。因为 ILI9341 的命令很多，我们这里就不全部介绍了，有兴趣的大家可以找到 ILI9341 的 datasheet 看看。里面对这些命令有详细的介绍。我们将介绍：0xD3，0x36，0x2A，0x2B，0x2C，0x2E 等 6 条指令。

指令 0xD3，是读 ID4 指令，用于读取 LCD 控制器的 ID，该指令如表 25.1.2.2 所示：

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	1	1	0	1	0	0	1	1	D3H
参数 1	1	↑	1	XX	X	X	X	X	X	X	X	X	X
参数 2	1	↑	1	XX	0	0	0	0	0	0	0	0	00H
参数 3	1	↑	1	XX	1	0	0	1	0	0	1	1	93H
参数 4	1	↑	1	XX	0	1	0	0	0	0	0	1	41H

表 25.1.2.2 0xD3 指令描述

从上表可以看出，0xD3 指令后面跟了 4 个参数，最后 2 个参数，读出来是 0x93 和 0x41，刚好是我们控制器 ILI9341 的数字部分，从而，通过该指令，即可判别所用的 LCD 驱动器是什么型号，这样，我们的代码，就可以根据控制器的型号去执行对应驱动 IC 的初始化代码，从而兼容不同驱动 IC 的屏，使得一个代码支持多款 LCD。

接下来看指令：0x36，这是存储访问控制指令，可以控制 ILI9341 存储器的读写方向，简单的说，就是在连续写 GRAM 的时候，可以控制 GRAM 指针的增长方向，从而控制显示方式（读 GRAM 也是一样）。该指令如表 25.1.2.3 所示：

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	1	0	1	1	0	36H
参数	1	1	↑	XX	MY	MX	MV	ML	BGR	MH	0	0	0

表 25.1.2.3 0x36 指令描述

从上表可以看出，0x36 指令后面，紧跟一个参数，这里主要关注：MY、MX、MV 这三个位，通过这三个位的设置，我们可以控制整个 ILI9341 的全部扫描方向，如表 25.1.2.4 所示：

控制位			效果
MY	MX	MV	LCD 扫描方向（GRAM 自增方式）
0	0	0	从左到右,从上到下
1	0	0	从左到右,从下到上
0	1	0	从右到左,从上到下
1	1	0	从右到左,从下到上
0	0	1	从上到下,从左到右
0	1	1	从上到下,从右到左
1	0	1	从下到上,从左到右
1	1	1	从下到上,从右到左

表 25.1.2.4 MY、MX、MV 设置与 LCD 扫描方向关系表

这样，我们在利用 ILI9341 显示内容的时候，就有很大灵活性了，比如显示 BMP 图片，BMP 解码数据，就是从图片的左下角开始，慢慢显示到右上角，如果设置 LCD 扫描方向为从

左到右，从下到上，那么我们只需要设置一次坐标，然后就不停的往 LCD 填充颜色数据即可，这样可以大大提高显示速度。

实验中，我们默认使用从左到右，从上到下的扫描方式。

接下来看指令：0x2A，这是列地址设置指令，在从左到右，从上到下的扫描方式（默认下面，该指令用于设置横坐标（x 坐标），该指令如表 25.1.2.5 所示：

顺序	控制			各位描述								HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	0	1	0	2AH
参数 1	1	1	↑	XX	SC15	SC14	SC13	SC12	SC11	SC10	SC9	SC8	SC
参数 2	1	1	↑	XX	SC7	SC6	SC5	SC4	SC3	SC2	SC1	SC0	
参数 3	1	1	↑	XX	EC15	EC14	EC13	EC12	EC11	EC10	EC9	EC8	EC
参数 4	1	1	↑	XX	EC7	EC6	EC5	EC4	EC3	EC2	EC1	EC0	

表 25.1.2.5 0x2A 指令描述

在默认扫描方式时，该指令用于设置 x 坐标，该指令带有 4 个参数，实际上是 2 个坐标值：SC 和 EC，即列地址的起始值和结束值，SC 必须小于等于 EC，且 0≤SC/EC≤239。一般在设置 x 坐标的时候，我们只需要带 2 个参数即可，也就是设置 SC 即可，因为如果 EC 没有变化，我们只需要设置一次即可（在初始化 ILI9341 的时候设置），从而提高速度。

与 0X2A 指令类似，指令：0X2B，是页地址设置指令，在从左到右，从上到下的扫描方式（默认）下面，该指令用于设置纵坐标（y 坐标）。该指令如表 25.1.2.6 所示：

顺序	控制			各位描述								HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	0	1	0	2BH
参数 1	1	1	↑	XX	SP15	SP14	SP13	SP12	SP11	SP10	SP9	SP8	SP
参数 2	1	1	↑	XX	SP7	SP6	SP5	SP4	SP3	SP2	SP1	SP0	
参数 3	1	1	↑	XX	EP15	EP14	EP13	EP12	EP11	EP10	EP9	EP8	EP
参数 4	1	1	↑	XX	EP7	EP6	EP5	EP4	EP3	EP2	EP1	EP0	

表 25.1.2.6 0X2B 指令描述

在默认扫描方式时，该指令用于设置 y 坐标，该指令带有 4 个参数，实际上是 2 个坐标值：SP 和 EP，即页地址的起始值和结束值，SP 必须小于等于 EP，且 0≤SP/EP≤319。一般在设置 y 坐标的时候，我们只需要带 2 个参数即可，也就是设置 SP 即可，因为如果 EP 没有变化，我们只需要设置一次即可（在初始化 ILI9341 的时候设置），从而提高速度。

接下来看指令：0X2C，该指令是写 GRAM 指令，在发送该指令之后，我们便可以往 LCD 的 GRAM 里面写入颜色数据了，该指令支持连续写，指令描述如表 25.1.2.7 所示。

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D8	D7	D6	D5	D4	D3	D2	D1	D0	
指令	0	1	↑	XX	0	0	1	0	1	1	0	0	2CH
参数 1	1	1	↑	D1[15：0]									XX
……	1	1	↑	D2[15：0]									XX
参数 n	1	1	↑	Dn[15：0]									XX

表 25.1.2.7 0X2C 指令描述

由表 25.1.2.7 可知，在收到指令 0X2C 之后，数据有效位宽变为 16 位，我们可以连续写入 LCD GRAM 值，而 GRAM 的地址将根据 MY/MX/MV 设置的扫描方向进行自增。例如：假设设置的是从左到右，从上到下的扫描方式，那么设置好起始坐标（通过 SC，SP 设置）后，每写入一个颜色值，GRAM 地址将会自动自增 1（SC++），如果碰到 EC，则回到 SC，同时 SP++，一直到坐标：EC，EP 结束，其间无需再次设置的坐标，从而大大提高写入速度。

最后，来看看指令：0X2E，该指令是读 GRAM 指令，用于读取 ILI9341 的显存（GRAM），该指令在 ILI9341 的数据手册上面的描述是有误的，真实的输出情况如表 25.1.2.8 所示：

顺序	控制			各位描述									HEX
	RS	RD	WR	D15~D11	D10~D8	D7	D6	D5	D4	D3	D2	D1	D0	

指令	0	1	↑	XX				0	0	1	0	1	1	1	0	2EH
参数 1	1	↑	1	XX												dummy
参数 2	1	↑	1	R1[4:0]		XX		G1[5:0]						XX		R1G1
参数 3	1	↑	1	B1[4:0]		XX		R2[4:0]						XX		B1R2
参数 4	1	↑	1	G2[5:0]			XX	B2[4:0]						XX		G2B2
参数 5	1	↑	1	R3[4:0]		XX		G3[5:0]						XX		R3G3
参数 N	1	↑	1	按以上规律输出												

表 25.1.2.8 0X2E 指令描述

该指令用于读取 GRAM，如表 25.1.2.8 所示，ILI9341 在收到该指令后，第一次输出的是 dummy 数据，也就是无效的数据，第二次开始，读取到的才是有效的 GRAM 数据（从坐标：SC，SP 开始），输出规律为：每个颜色分量占 8 个位，一次输出 2 个颜色分量。比如：第一次输出是 R1G1，随后的规律为：B1R2→G2B2→R3G3→B3R4→G4B4→R5G5...以此类推。如果我们只需要读取一个点的颜色值，那么只需要接收到参数 3 即可，如果要连续读取（利用 GRAM 地址自增，方法同上），那么就按照上述规律去接收颜色数据。

以上，就是操作 ILI9341 常用的几个指令，通过这几个指令，我们便可以很好的控制 ILI9341 显示我们所要显示的内容了。

25.1.3 FMC 简介

STM32H743xx 系列芯片都带有 FMC 接口，即可变存储控制器，能够与同步或异步存储器、SDRAM 存储器和 NAND FLASH 等连接，STM32H743 的 FMC 接口支持包括 SRAM、SDRAM、NAND FLASH、NOR FLASH 和 PSRAM 等存储器。FMC 的框图如图 25.1.3.1 所示：

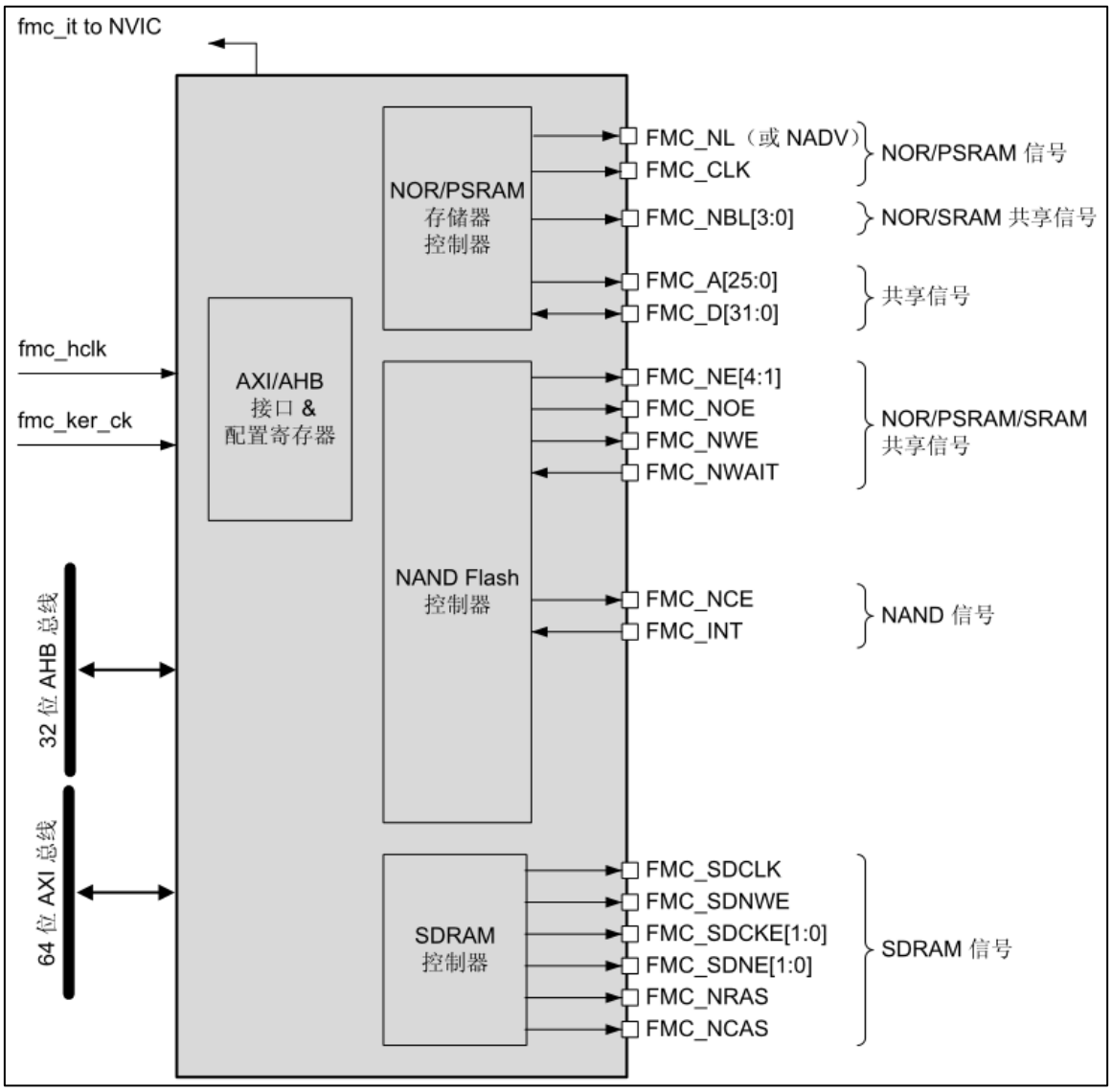


图 25.1.3.1 FMC 框图

从上图我们可以看出，STM32H743 的 FMC 将外部设备分为 3 类：NOR/PSRAM 设备、NAND 设备和 SDRAM 设备。他们共用地址数据总线等信号，他们具有不同的 CS 以区分不同的设备，比如本章我们用到的 TFTLCD 就是用的 FMC_NE1 做片选，其实就是将 TFTLCD 当成 SRAM 来控制。

图中的 fmc_hclk 时钟来自 AHB3，例程设置为 200Mhz，该时钟用于寄存器访问。而 fmc_ker_ck 时钟来自 RCC_D1CCIPR 寄存器 FMCSEL[1:0]的设置，如图 25.1.3.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	CKPERSEL[1:0] (1)		Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDMMCSEL (1)
		rw													rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	QSPISEL[1:0] (1)		Res.	Res.	FMCSEL[1:0] (1)	
										rw				rw	

位 1:0 **FMCSEL[1:0]**：FMC 内核时钟源选择 (FMC kernel clock source selection)

00：将 **rcc_hclk3** 时钟选作内核外设时钟（复位后的默认值）

01：将 **pll1_q_ck** 时钟选作内核外设时钟

10：将 **pll2_r_ck** 时钟选作内核外设时钟

11：将 **per_ck** 时钟选作内核外设时钟

图 25.1.3.2 RCC_D1CCIPR 寄存器各位描述

在 sys.c 文件夹的 sys_stm32_clock_init 时钟设置初始化函数中，我们通过配置 HAL_RCCEx_PeriphCLKConfig 函数设置了 RCC_D1CCIPR 寄存器的 FMCSEL[1:0]位为 10，即选择 pll2_r_ck 作为 fmc_ker_ck 时钟，为 200Mhz。所以，fmc_ker_ck=200Mhz。

另外，需要注意：图 25.1.2.1 中的 32bit AHB 总线仅用于访问 FMC 的寄存器，而 64bit AXI 总线则用于访问相关存储器。因此访问 FMC 寄存器和访问外部存储器，是通过不同的总线访问的。

这里我们介绍下为什么可以把 TFTLCD 当成 SRAM 设备用：首先我们了解下外部 SRAM 的连接，外部 SRAM 的控制一般有：地址线（如 A0~A18）、数据线（如 D0~D15）、写信号（WE）、读信号（OE）、片选信号（CS），如果 SRAM 支持字节控制，那么还有 UB/LB 信号。而 TFTLCD 的信号我们在 25.1.1 节有介绍，包括：RS、D0~D15、WR、RD、CS、RST 和 BL 等，其中真正在操作 LCD 的时候需要用到的就只有：RS、D0~D15、WR、RD 和 CS。其操作时序和 SRAM 的控制完全类似，唯一不同就是 TFTLCD 有 RS 信号，但是没有地址信号。

TFTLCD 通过 RS 信号来决定传送的数据是数据还是命令，本质上可以理解为一个地址信号，比如我们把 RS 接在 A0 上面，那么当 FMC 控制器写地址 0 的时候，会使得 A0 变为 0，对 TFTLCD 来说，就是写命令。而 FMC 写地址 1 的时候，A0 将会变为 1，对 TFTLCD 来说，就是写数据了。这样，就把数据和命令区分开了，他们其实就是对应 SRAM 操作的两个连续地址。当然 RS 也可以接在其他地址线上，我们的开发板把 RS 连接在 A19 上面的。

STM32H743 的 FMC 支持 8/16/32 位数据宽度，我们这里用到的 LCD 是 16 位宽度的，所以在设置的时候，选择 16 位宽就 OK 了。我们再来看看 FMC 的外部设备地址映像，STM32H743 的 FMC 将外部存储器划分为 6 个固定大小为 256M 字节的存储区域，如图 25.1.3.3 所示：

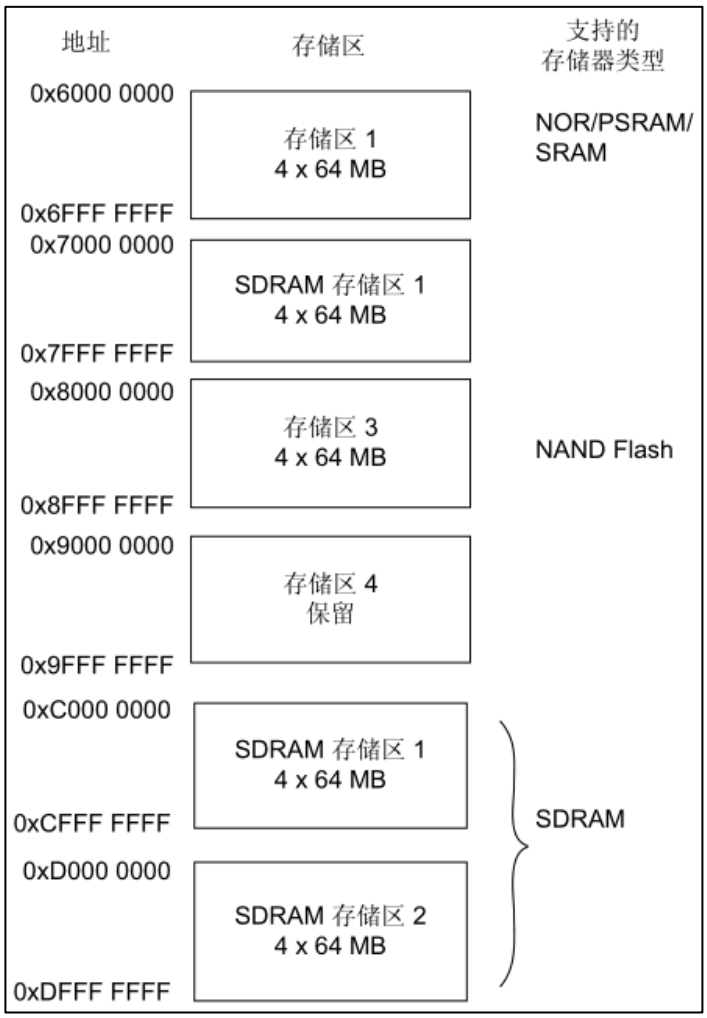


图 25.1.3.3 FMC 存储块地址映像

从上图可以看出，FMC 总共管理 1.5GB 空间，拥有 6 个存储块（Bank），每个存储块 256MB 空间。本章，我们把 TFTLCD 当成 SRAM 设备来使用，所以用到的是存储块 1。下面我们仅讨论存储块 1 的相关配置，其他块的配置，请参考《STM32H7xx 参考手册_V3（中文版）.pdf》第 22 章（690 页）的相关介绍。

STM32H743 的 FMC 存储块 1（Bank1）被分为 4 个区，每个区管理 64M 字节空间，每个区都有独立的寄存器对所连接的存储器进行配置。Bank1 的 256M 字节空间由 28 根地址线（ADDR[27:0]）寻址。

这里 ADDR 是内部 AXI 地址总线，其中 ADDR[25:0]来自外部存储器地址 FMC_A[25:0]，而 ADDR[26:27]对 4 个区进行寻址。如表 25.1.3.1 所示：

Bank1 所选区	片选信号	地址范围	ADDR	
			[27:26]	[25:0]
第 1 区	FMC_NE1	0X6000,0000~63FF,FFFF	00	FMC_A[25:0]
第 2 区	FMC_NE2	0X6400,0000~67FF,FFFF	01	
第 3 区	FMC_NE3	0X6800,0000~6BFF,FFFF	10	
第 4 区	FMC_NE4	0X6C00,0000~6FFF,FFFF	11	

表 25.1.3.1 Bank1 存储区选择表

ADDR[25:0]位包含外部存储器的地址，由于 ADDR 为字节地址，而存储器按字寻址，所以，根据存储器数据宽度的不同，实际上向存储器发送的地址也有所不同，如表 25.1.3.2 所示：

存储器宽度	向存储器发出的数据地址	最大存储器容量（位）
8 位	ADDR[25:0]	64 MB x 8 = 512 Mb
16 位	ADDR[25:1] >> 1	64 MB/2 x 16 = 512 Mb
32 位	ADDR[25:2] >> 2	64 MB/4 x 32 = 512 Mb

表 25.1.3.2 NOR/PSRAM 外部存储器地址

因此，FMC 内部 ADDR 与存储器寻址地址的实际对应关系就是：

当接的是 32 位宽度存储器的时候：ADDR[25:2]→ FMC_A [23:0]。

当接的是 16 位宽度存储器的时候：ADDR[25:1]→ FMC_A [24:0]。

当接的是 8 位宽度存储器的时候：ADDR[25:0]→ FMC_A [25:0]。

不论外部接 8 位/16 位/32 位宽设备，FMC_A[0]永远接在外部设备地址 A[0]。 这里，TFTLCD 使用的是 16 位数据宽度，所以 ADDR[0]并没有用到，只有 ADDR[25:1]是有效的，对应关系变为：ADDR[25:1]→ FMC_A[24:0]，相当于右移了一位，这里请大家特别留意。另外，ADDR[27:26]的设置，是不需要我们干预的，比如：当你选择使用 Bank1 的第一个区，即使用 FMC_NE1 来连接外部设备的时候，即对应了 ADDR[27:26]=00，我们要做的就是配置对应第 1 区的寄存器组，来适应外部设备。STM32H743 的 FMC 各 Bank 配置寄存器如表 25.1.3.3 所示：

内部控制器	存储块	管理的地址范围	支持的设备类型	配置寄存器
NOR FLASH/PSRAM 控制器	Bank1	0X6000,0000~ 0X6FFF,FFFF	SRAM/ROM NOR FLASH PSRAM	FMC_BCR1/2/3/4 FMC_BTR1/2/2/3 FMC_BWTR1/2/3/4
SDRAM 控制器	Bank2	0X7000,0000~ 0X7FFF,FFFF	SDRAM	同 Bank5/Bank6
NAND FLASH 控制器	Bank3	0X8000,0000~ 0X8FFF,FFFF	NAND FLASH	FMC_PCR/FMC_SR FMC_PMEM/FMC_PATT FMC_ECCR
保留	Bank4	0X9000,0000~ 0X9FFF,FFFF	保留	保留
SDRAM 控制器	Bank5	0XC000,0000~ 0XCFFF,FFFF	SDRAM	FMC_SDCR1/2 FMC_SDTR1/2 FMC_SDCMR FMC_SDRTR FMC_SDSR
	Bank6	0XD000,0000~ 0XDFFF,FFFF	SDRAM	

表 25.1.3.3 FMC 各 Bank 配置寄存器表

对于 NOR FLASH 控制器，主要是通过 FMC_BCRx、FMC_BTRx 和 FMC_BWTRx 寄存器设置（其中 x=1~4，对应 4 个区）。通过这 3 个寄存器，可以设置 FMC 访问外部存储器的时序参数，拓宽了可选用的外部存储器的速度范围。FMC 的 NOR FLASH 控制器支持同步和异步突发两种访问方式。选用同步突发访问方式时，FMC 将 fmc_ker_ck 时钟(FMC 内核时钟)分频后，发送给外部存储器作为同步时钟信号 FMC_CLK。此时需要的设置的时间参数有 2 个：

1，fmc_ker_ck 与 FMC_CLK 的分频系数(CLKDIV)，可以为 2～16 分频；

2，同步突发访问中获得第 1 个数据所需要的等待延迟(DATLAT)。

对于异步突发访问方式，FMC 主要设置 3 个时间参数：地址建立时间(ADDSET)、数据建立时间(DATAST)和地址保持时间(ADDHLD)。FMC 综合了 SRAM、PSRAM 和 NOR Flash 产品的信号特点，定义了 4 种不同的异步时序模型。选用不同的时序模型时，需要设置不同的时序参数，如表 25.1.3.4 所列：

时序模型		简单描述	时间参数
异步	Mode1	SRAM/CRAM 时序	DATAST、ADDSET
	ModeA	SRAM/CRAM OE 选通型时序	DATAST、ADDSET

	Mode2/B	NOR FLASH 时序	DATAST、ADDSET
	ModeC	NOR FLASH OE 选通型时序	DATAST、ADDSET
	ModeD	延长地址保持时间的异步时序	DATAST、ADDSET、ADDHLD
同步突发		根据同步时钟 FMC_CK 读取多个顺序单元的数据	CLKDIV、DATLAT

表 25.1.3.4 NOR FLASH/PSRAM 控制器支持的时序模型

在实际扩展时，根据选用存储器的特征确定时序模型，从而确定各时间参数与存储器读/写周期参数指标之间的计算关系；利用该计算关系和存储芯片数据手册中给定的参数指标，可计算出 FMC 所需要的各时间参数，从而对时间参数寄存器进行合理的配置。

本章，我们使用异步模式 A（ModeA）方式来控制 TFTLCD，模式 A 的读操作时序如图 25.1.3.4 所示：

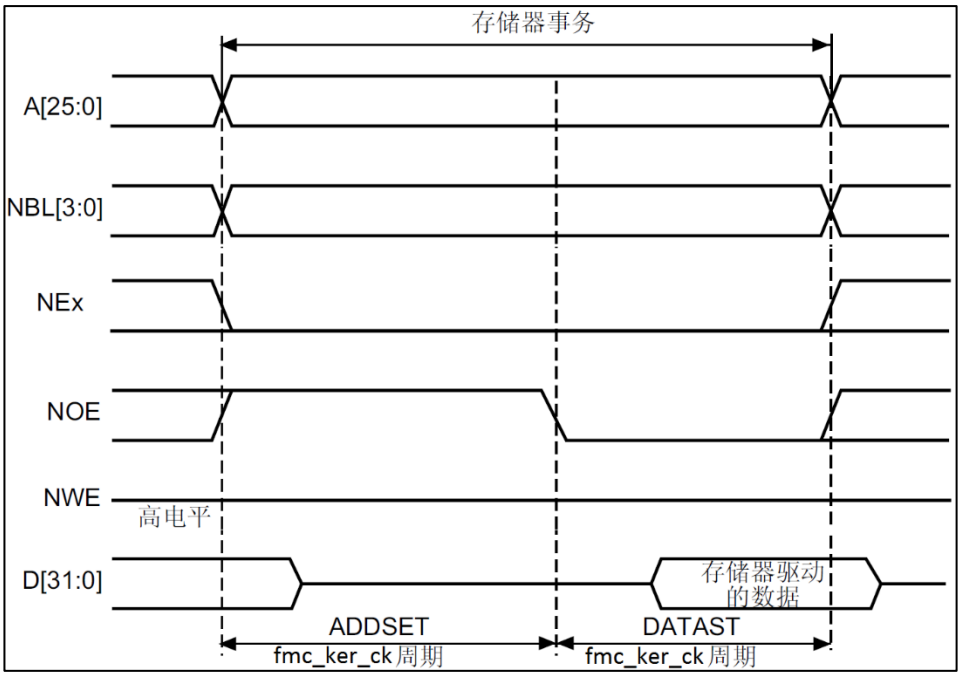


图 25.1.3.4 模式 A 读操作时序图

模式 A 支持独立的读写时序控制，这个对我们驱动 TFTLCD 来说非常有用，因为 TFTLCD 在读的时候，一般比较慢，而在写的时候可以比较快，如果读写用一样的时序，那么只能以读的时序为基准，从而导致写的速度变慢，或者在读数据的时候，重新配置 FMC 的延时，在读操作完成的时候，再配置回写的时序，这样虽然也不会降低写的速度，但是频繁配置，比较麻烦。而如果有独立的读写时序控制，那么我们只要初始化的时候配置好，之后就不用再配置，既可以满足速度要求，又不需要频繁改配置。

模式 A 的写操作时序如图 25.1.3.5 所示：

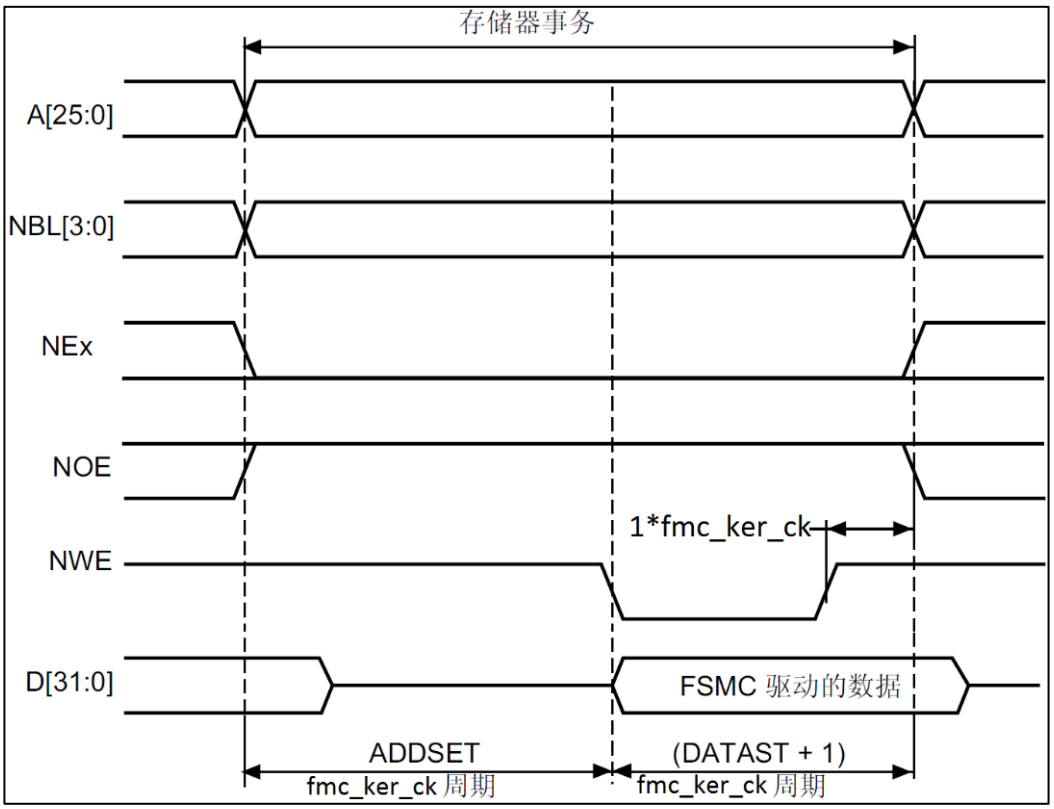


图 25.1.3.5 模式 A 写操作时序

图 25.1.2.4 和图 25.1.2.5 中的 ADDSET 与 DATAST，是通过不同的寄存器设置的，接下来我们讲解一下 Bank1 的几个控制寄存器。

25.1.4 FMC 寄存器

- **NOR/PSRAM 控制寄存器 1/2/3/4（FMC_BCR1/2/3/4）**

SRAM/NOR 闪存片选控制寄存器:FMC_BCRx（x=1~4），该寄存器描述如图 25.1.4.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FMCEN	Res.	Res.	Res.	Res.	Res.	BMAP[1:0]		Res.	Res.	WFDIS	CCLKEN	CBURSTRW	CPSIZE[2:0]			ASYNCWAIT	EXTMOD	WAITEN	WREN	WAITCFG	Res.	WAITPOL	BURSTEN	Res.	FACCEN	MWID		MTYP		MUXEN	MBKEN
rw						rw				rw	rw	rw	rw			rw	rw	rw	rw	rw		rw	rw		rw	rw	rw	rw	rw	rw	rw

图 25.1.4.1 FMC_BCRx 寄存器各位描述（部分）

该寄存器我们在本章用到的设置有：FMCEN、EXTMOD、WREN、MWID[1:0]、MTYP[1:0]和 MBKEN 这几个设置，我们将逐个介绍。

FMCEN：FMC 使能位。我们要使用 FMC 驱动 TFTLCD 就必须设置该位为 1，不过只有 FMC_BCR1 的 FMCEN 位有效，FMC_BCR2~4 的 FMCEN 位无效，统一由 FMC_BCR1 的 FMCEN 位控制。

EXTMOD：扩展模式使能位，也就是是否允许读写不同的时序，很明显，我们本章需要读写不同的时序，故该位需要设置为 1。

WREN：写使能位。我们需要向 TFTLCD 写数据，故该位必须设置为 1。

MWID[1:0]：存储器数据总线宽度。00，表示 8 位数据模式；01 表示 16 位数据模式；10 表示 32 位数据模式；11 保留。我们的 TFTLCD 是 16 位数据线，所以设置 WMID[1:0]=01。

MTYP[1:0]：存储器类型。00 表示 SRAM；01 表示 PSRAM；10 表示 NOR FLASH/OneNAND FLASH;11 保留。前面提到，我们把 TFTLCD 当成 SRAM 用，所以需要设置 MTYP[1:0]=00。

MBKEN：存储块使能位。我们需要用到该存储块控制 TFTLCD，所以要使能该存储块。

- **SRAM/NOR-Flash 片选时序寄存器 1/2/3/4 (FMC_BTR1/2/3/4)**

SRAM/NOR 闪存片选时序寄存器：FMC_BTRx（x=1~4），该寄存器描述如图 25.1.4.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	ACCMOD		DATLAT				CLKDIV				BUSTURN			
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATAST								ADDHLD				ADDSET			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 25.1.4.2 FMC_BTRx 寄存器各位描述

这个寄存器包含了每个存储器块的控制信息，可以用于 SRAM 和 NOR 闪存存储器等。如果 FMC_BCRx 寄存器中设置了 EXTMOD 位，则有两个时序寄存器分别对应读(本寄存器)和写操作(FMC_BWTRx 寄存器)。因为我们要求读写分开时序控制，所以 EXTMOD 是使能了的，也就是本寄存器是读操作时序寄存器，控制读操作的相关时序。本章我们要用到的设置有：ACCMOD、DATAST 和 ADDSET 这三个设置。

ACCMOD[1:0]：访问模式。00 表示访问模式 A；01 表示访问模式 B；10 表示访问模式 C；11 表示访问模式 D，本章我们用到模式 A，故设置为 00。

DATAST[7:0]：数据保持时间。0 为保留设置，其他设置则代表保持时间为: DATAST 个 fmc_ker_ck 时钟周期，最大为 255 个。对 ILI9341 来说，其实就是 RD 低电平持续时间，一般为 355ns。而一个 fmc_ker_ck 时钟周期为 4.5ns 左右（1/220Mhz），为了兼容其他屏，我们这里设置 DATAST 为 78，也就是 78 个 fmc_ker_ck 周期，时间大约是 351ns（略超，但不影响使用）。

ADDSET[3:0]：地址建立时间。其建立时间为：ADDSET 个 fmc_ker_ck 周期，最大为 15 个。对 ILI9341 来说，这里相当于 RD 高电平持续时间，为 90ns，我们设置 ADDSET 为最大 15，即 15*4.3=67.5ns（略超，但不影响使用）。

- **SRAM/NOR-Flash 写入时序寄存器 1/2/3/4 (FMC_BWTR1/2/3/4)**

SRAM/NOR 闪写时序寄存器：FMC_BWTRx（x=1~4），该寄存器描述如图 25.1.4.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	ACCMOD		Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BUSTURN			
		rw	rw									rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATAST								ADDHLD				ADDSET[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 25.1.4.3 FMC_BWTRx 寄存器各位描述

该寄存器在本章用作写操作时序控制寄存器，需要用到的设置同样是：ACCMOD、DATAST 和 ADDSET 这三个设置。这三个设置的方法同 FMC_BTRx 一模一样，只是这里对应的是写操作的时序，ACCMOD 设置同 FMC_BTRx 一模一样，同样是选择模式 A，另外 DATAST 和 ADDSET 则对应 WR 的低电平和高电平持续时间，对 ILI9341 来说，这两个时间只需要 15ns 就够了，比读操作快得多。所以我们这里设置 DATAST 为 3，即 3 个 fmc_ker_ck 周期，时间约为 13.5ns。然后 ADDSET 设置为 3，也是 13.5ns（略超，但不影响使用）。

至此，我们对 STM32H743 的 FMC 介绍就差不多了，关于 FMC 的详细介绍，请大家参考《STM32H7xx 参考手册_V7（英文版）.pdf》第 21 章。通过以上两个小节的了解，我们可以开始写 LCD 的驱动代码了。注意：在 MDK 的寄存器定义里面，并没有定义 FMC_BCRx、FMC_BTRx、FMC_BWTRx 等这个单独的寄存器，而是将他们进行了一些组合。

FMC_BCRx 和 FMC_BTRx，组合成 BTCR[8]寄存器组，他们的对应关系如下：

BTCR[0]对应 FMC_BCR1，BTCR[1]对应 FMC_BTR1

BTCR[2]对应 FMC_BCR2，BTCR[3]对应 FMC_BTR2

BTCR[4]对应 FMC_BCR3，BTCR[5]对应 FMC_BTR3

BTCR[6]对应 FMC_BCR4，BTCR[7]对应 FMC_BTR4

FMC_BWTRx 则组合成 BWTR[7]，他们的对应关系如下：

BWTR[0]对应 FMC_BWTR1，BWTR[2]对应 FMC_BWTR2，
BWTR[4]对应 FMC_BWTR3，BWTR[6]对应 FMC_BWTR4，
BWTR[1]、BWTR[3]和 BWTR[5]保留，没有用到。

通过对 FMC 相关的寄存器的描述，大家对 FMC 的原理有了一定的认识，如果还不熟悉的朋友，请一定要搜索网络资料理解 FMC 的原理。

一般 TFTLCD 模块的使用流程如图 25.1.1.5：

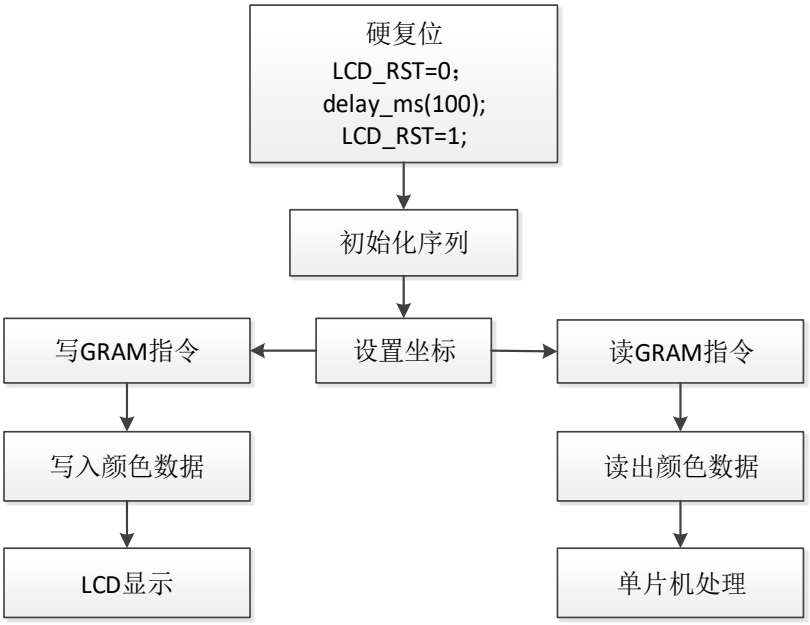


图 25.1.1.5 TFTLCD 使用流程

任何 LCD，使用流程都可以简单的用以上流程图表示。其中硬复位和初始化序列，只需要执行一次即可。而画点流程就是：设置坐标→写 GRAM 指令→写入颜色数据，然后在 LCD 上面，我们就可以看到对应的点显示我们写入的颜色了。读点流程为：设置坐标→读 GRAM 指令→读取颜色数据，这样就可以获取到对应点的颜色数据了。

以上只是最简单的操作，也是最常用的操作，有了这些操作，一般就可以正常使用 TFTLCD 了。接下来我们将该模块用来来显示字符和数字，通过以上介绍，我们可以得出 TFTLCD 显示需要的相关设置步骤如下：

1）设置 STM32H743 与 TFTLCD 模块相连接的 IO。

这一步，先将我们与 TFTLCD 模块相连的 IO 口进行初始化，以便驱动 LCD，这里我们用到的是 FMC。

2）初始化 TFTLCD 模块。

即图 25.1.1.5 的初始化序列，这里我们没有硬复位 LCD，因为开发板的 LCD 接口将 TFTLCD 的 RST 同 STM32H743 的 RESET 连接在一起了，只要按下开发板的 RESET 键，就会对 LCD 进行硬复位。初始化序列，就是向 LCD 控制器写入一系列的设置值（比如伽马校准），这些初始化序列一般 LCD 供应商会提供给客户，我们直接使用这些序列即可，不需要深入研究。在初始化之后，LCD 才可以正常使用。

3）通过函数将字符和数字显示到 TFTLCD 模块上。

这一步则通过图 25.1.1.5 左侧的流程，即：设置坐标→写 GRAM 指令→写 GRAM 来实现，但是这个步骤，只是一个点的处理，我们要显示字符/数字，就必须要多次使用这个步骤，从而达到显示字符/数字的目的，所以需要设计一个函数来实现数字/字符的显示，之后调用该函数，就可以实现数字/字符的显示了。

25.2 硬件设计

1. 例程功能

使用开发板的 MCU 屏接口连接正点原子 TFTLCD 模块(仅限 MCU 屏模块)，实现 TFTLCD 模块的显示。通过把 LCD 模块插入底板上的 TFTLCD 模块接口，按下复位之后，就可以看到 LCD 模块不停的显示一些信息并不断切换底色。同时该实验会显示 LCD 驱动器的 ID，并且会在串口打印（按复位一次，打印一次）。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯

LED ： LED0 – PB1

2）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(仅限 MCU 屏，16 位 8080 并口驱动)

3. 原理图

TFTLCD 模块的电路见图 25.2.3.1，而开发板的 LCD 接口和正点原子 TFTLCD 模块直接可以对插，开发板上的 LCD 接口如图 25.2.3.1 所示：

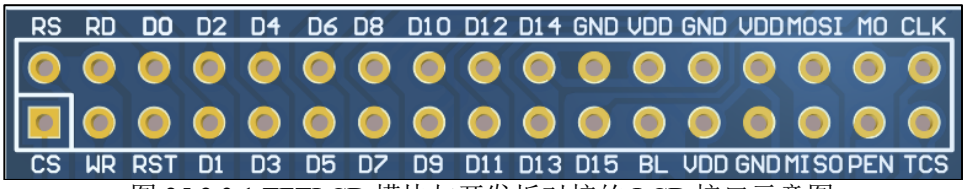


图 25.2.3.1 TFTLCD 模块与开发板对接的 LCD 接口示意图

TFTLCD 模块与开发板的连接原理图如图 25.2.3.2 所示：

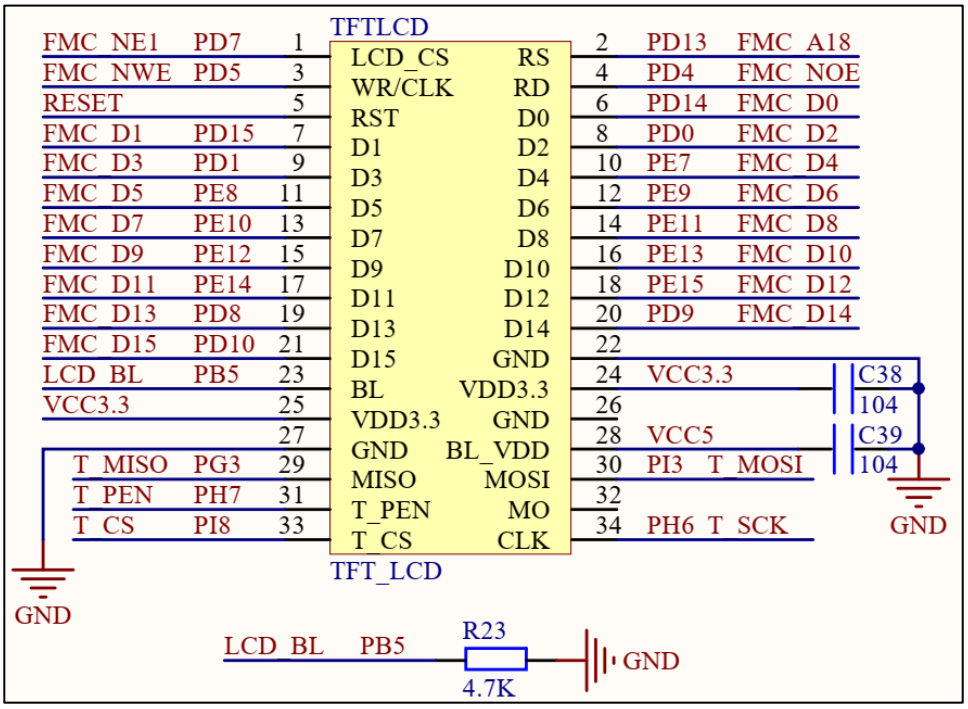


图 25.2.3.2 TFTLCD 模块与开发板的连接原理图

在硬件上，TFTLCD 模块与开发板的 IO 口对应关系如下：

LCD_BL(背光控制)对应 PB5；

LCD_CS 对应 PD7，即 FMC_NE1；

LCD _RS 对应 PD13，即 FMC_A18；

LCD _WR 对应 PD5，即 FMC_NEW；

LCD _RD 对应 PD4，即 FMC_NOE；
LCD _D[15:0]则直接连接在 FMC_D15~FMC_D0；
这些线的连接，开发板的内部已经连接好了，我们只需要将 TFTLCD 模块插上去就好了。

25.3 程序设计

25.3.1 FMC 和 SRAM 的 HAL 库驱动

SRAM 和 FMC 在 HAL 库中的驱动代码在 stm32h7xx_ii_fmc.c/stm32h7xx_hal_sram.c 以及 stm32h7xx_ii_fmc.h/stm32h7xx_hal_sram.h 中。

1. HAL_SRAM_Init 函数

SRAM 的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_SRAM_Init(SRAM_HandleTypeDef *hsram,
       FMC_NORSRAM_TimingTypeDef *Timing, FMC_NORSRAM_TimingTypeDef *ExtTiming);
```

- **函数描述：**

  用于初始化 SRAM，注意这个函数不限制一定是 SRAM，只要时序类似，均可使用。前面说过，这里我们把 LCD 当作 SRAM 使用，因为他们时序类似。

- **函数形参：**

  形参 1 是 SRAM_HandleTypeDef 结构体类型指针变量，其定义如下：

```
typedef struct
{
  FMC_NORSRAM_TypeDef             *Instance;    /* 寄存器基地址 */
  FMC_NORSRAM_EXTENDED_TypeDef    *Extended;    /* 扩展模式寄存器基地址 */
  FMC_NORSRAM_InitTypeDef          Init;        /* SRAM 设备控制配置结构体 */
  HAL_LockTypeDef                  Lock;        /* SRAM 锁定对象 */
  __IO HAL_SRAM_StateTypeDef       State;       /* SRAM 设备访问状态 */
  MDMA_HandleTypeDef              *hmdma;       /* 指针 DMA 处理配置 */
}SRAM_HandleTypeDef;
```

1）Instance：指向 FMC 寄存器基地址。本实验我们使用异步模式 A（ModeA）方式来控制 TFTLCD，使用的存储块是 Bank1，所以寄存器基地址 Instance 我们直接写 FMC_Bank1_R 即可，因为 HAL 库定义好了宏定义 FMC_NORSRAM_DEVICE，也就是如果是 SRAM 设备，直接填写这个宏定义标识符即可。

2）Extended：指向 FMC 扩展模式寄存器基地址，因为我们要配置的读写时序是不一样的，前面讲的 FMC_BCRx 寄存器的 EXTMOD 位我们会配置为 1 允许读写不同的时序，所以还要指定写操作时序寄存器地址，也就是通过参数 Extended 来指定的，这里设置为 FMC_Bank1E_R 即可，同样 HAL 库定义了 FMC_NORSRAM_EXTENDED_DEVICE，直接填写这个宏定义标识符即可。

3）Init：用于配置 FMC 外接 SRAM 或者相同时序设备时的基本参数，是我们接触最多的参数。

4）Lock：用于配置锁状态。

5）State：SRAM 设备访问状态。

6）hmdma：用于配置关联 MAMA 句柄。

其中成员变量 Init 是 FMC_NORSRAM_InitTypeDef 结构体指针类型，该变量才是真正用来设置 SRAM 控制接口参数的。下面详细了解这个结构体定义：

```
typedef struct
{
  uint32_t NSBank;              /* 存储区块号 */
  uint32_t DataAddressMux;      /* 地址/数据复用使能 */
  uint32_t MemoryType;          /* 存储器类型 */
  uint32_t MemoryDataWidth;     /* 存储器数据宽度 */
  uint32_t BurstAccessMode;     /* 使能或者禁止突发模式 */
  uint32_t WaitSignalPolarity;  /* 设置等待信号的极性 */
  uint32_t WaitSignalActive;    /* 等待状态之前或等待状态期间 */
  uint32_t WriteOperation;      /* 存储器写使能 */
  uint32_t WaitSignal;          /* 使能或者禁止通过等待信号来插入等待状态 */
```

```
  uint32_t ExtendedMode;        /* 使能或者禁止使能扩展模式 */
  uint32_t AsynchronousWait;    /* 用于异步传输期间，使能或者禁止等待信号 */
  uint32_t WriteBurst;          /* 用于使能或者禁止异步的写突发操作 */
  uint32_t ContinuousClock;     /* 使能或者禁止 FMC 时钟输出到外部存储设备 */
  uint32_t WriteFifo;           /* 使能或者禁止写 FIFO */
  uint32_t PageSize;            /* 设置页大小 */
}FMC_NORSRAM_InitTypeDef;
```

NSBank 用来指定使用到的存储块区号，前面讲过，我们是使用的存储块区号 1，所以选择值为 FMC_NORSRAM_BANK1。

DataAddressMux 用来设置是否使能地址/数据复用,该变量仅对 NOR/PSRAM 有效，所以这里我们选择不使能地址/数据复用值 FMC_DATA_ADDRESS_MUX_DISABLE 即可。

MemoryType 用来设置存储器类型，这里我们把 LCD 当 SRAM 使用，所以设置为 FMC_MEMORY_TYPE_SRAM 即可。

MemoryDataWidth 用来设置存储器数据总线宽度，可选 8 位还是 16 位，这里我们选择 16 位数据宽度 FMC_NORSRAM_MEM_BUS_WIDTH_16。

WriteOperation 用来设置存储器写使能，也就是是否允许写入。毫无疑问我们会进行存储器写操作，所以这里设置为 FMC_WRITE_OPERATION_ENABLE。

ExtendedMode 用来设置是否使能扩展模式，也就是是否允许读写使用不同时序，前面讲解过本实验读写采用不同时序，所以设置值为使能值 FMC_EXTENDED_MODE_ENABLE。

ContinuousClock 用来设置启用/禁止 FMC 时钟输出到外部存储设备，这里仅当使用 FMC_BCR1 寄存器的时候需要启用，启用值为 FMC_CONTINUOUS_CLOCK_SYNC_ASYNC。

其他参数 WriteBurst，BurstAccessMode，WaitSignalPolarity，WaitSignalActive，WaitSignal，AsynchronousWait 等是用在突发访问和异步时序情况下，这里我们不做过多讲解。

形参 2 Timing 和**形参 3** ExtTiming 都是 FMC_NORSRAM_TimingTypeDef 结构体类型指针变量，其定义如下：

```
typedef struct
{
  uint32_t AddressSetupTime;        /* 地址建立时间 */
  uint32_t AddressHoldTime;         /* 地址保持时间 */
  uint32_t DataSetupTime;           /* 数据建立时间 */
  uint32_t BusTurnAroundDuration;   /* 总线周转阶段的持续时间 */
  uint32_t CLKDivision;             /* CLK 时钟输出信号的周期 */
  uint32_t DataLatency;             /* 同步突发 NOR FLASH 的数据延迟 */
  uint32_t AccessMode;              /* 异步模式配置 */
}FMC_NORSRAM_TimingTypeDef;
```

对于本实验，读速度比写速度慢得多，因此读写时序不一样，所以对于 Timing 和 ExtTiming 要设置了不同的值，其中 Timing 设置写时序参数，ExtTiming 设置读时序参数。

下面解析一下结构体的成员变量：

AddressSetupTime 用来设置地址建立时间。

AddressHoldTime 用来设置地址保持时间。

DataSetupTime 用来设置数据建立时间。

BusTurnAroundDuration 用来配置总线周转阶段的持续时间。

CLKDivision 用来配置 CLK 时钟输出信号的周期，以 HCLK 周期数表示。

DataLatency 用来设置同步突发 NOR FLASH 的数据延迟。

AccessMode 用来设置异步模式，取值范围为 FMC_ACCESS_MODE_A、FMC_ACCESS_MODE_B,、FMC_ACCESS_MODE_C 和 FMC_ACCESS_MODE_D，这里我们用是异步模式 A，所以取值为 FMC_ACCESS_MODE_A。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

- **注意事项：**

  和其他外设一样，HAL 库也提供了 SRAM 的初始化 MSP 回调函数，函数声明如下：

```
void HAL_SRAM_MspInit(SRAM_HandleTypeDef *hsram) ;
```

2. FMC_NORSRAM_Extended_Timing_Init 函数

FMC_NORSRAM_Extended_Timing_Init 函数是初始化扩展时序模式函数。其声明如下：

```
HAL_StatusTypeDef  FMC_NORSRAM_Extended_Timing_Init(
     FMC_NORSRAM_EXTENDED_TypeDef *Device, FMC_NORSRAM_TimingTypeDef *Timing,
     uint32_t Bank, uint32_t ExtendedMode);
```

- **函数描述：**
  该函数用于初始化扩展时序模式。
- **函数形参：**
  形参 1 是 FMC_NORSRAM_EXTENDED_TypeDef 结构体类型指针变量，扩展模式寄存器基地址选择。
  形参 2 是 FMC_NORSRAM_TimingTypeDef 结构体类型指针变量，可以是读或者写时序结构体。
  形参 3 是储存区块号。
  形参 4 是使能或者禁止扩展模式。
- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。
- **注意事项：**
  该函数我们用于重新配置写或者读时序。

FMC 驱动 LCD 显示配置步骤

1）使能 FMC 和相关 GPIO 时钟，并设置好 GPIO 工作模式

我们通过 FMC 控制 LCD，所以先需要使能 FMC 以及相关 GPIO 口的时钟，并设置好 GPIO 的工作模式。

2）设置 FMC 参数

这里我们需要设置 FMC 的相关访问参数（数据位宽、访问时序、工作模式等），以匹配液晶驱动 IC，这里我们通过 HAL_SRAM_Init 函数完成 FMC 参数配置，详见本例程源码。

3）初始化 LCD

由于我们例程兼容了很多种液晶驱动 IC，所以先要读取对应 IC 的驱动型号，然后根据不同的 IC 型号来调用不同的初始化函数，完成对 LCD 的初始化。

注意：这些初始化函数里面的代码，都是由 LCD 厂家提供，一般不需要改动，也不需要深究，我们直接照抄即可。

4）实现 LCD 画点&读点函数

在初始化 LCD 完成以后，我们就可以控制 LCD 显示了，而最核心的一个函数，就是画点和读点函数，只要实现这两个函数，后续的各种 LCD 操作函数，都可以基于这两个函数实现。

5）实现其他 LCD 操作函数

在完成画点和读点两个最基础的 LCD 操作函数以后，我们就可以基于这两个函数实现各种 LCD 操作函数了，比如画线、画矩形、显示字符、显示字符串、显示数字等，如果不够用还可以根据自己需要来添加。详见本例程源码。

25.3.2 程序流程图

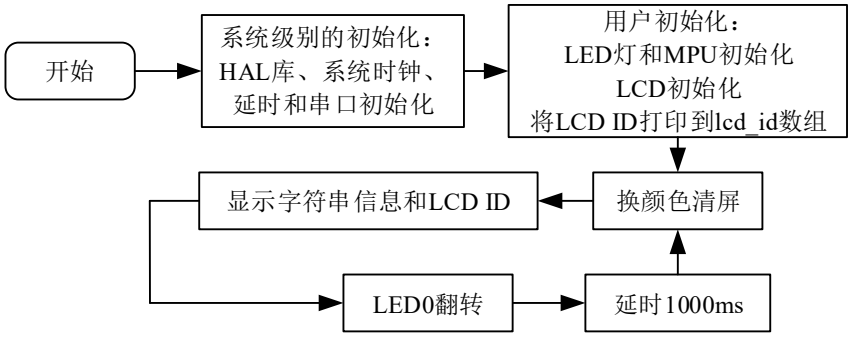


图 25.3.2.1 TFTLCD（MCU 屏）实验程序流程图

25.3.3 程序解析

1. LCD 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。LCD 驱动源码包括四个文件：lcd.c、lcd.h、lcd_ex.c 和 lcdfont.h。

lcd.c 和 lcd.h 文件是驱动函数和引脚接口宏定义以及函数声明等。lcd_ex.c 存放各个 LCD 驱动 IC 的寄存器初始化部分代码，是 lcd.c 文件的补充文件，起到简化 lcd.c 文件的作用。lcdfont.h 头文件存放了 4 种字体大小不一样的 ASCII 字符集（12*12、16*16、24*24 和 32*32）。这个跟 oledfont.h 头文件一样的，只是这里多了 32*32 的 ASCII 字符集，制作方法请回顾 OLED 实验。

下面我们还是先介绍 lcd.h 文件，首先是 LCD 的引脚定义：

```
/* LCD RST/WR/RD/BL/CS/RS 引脚 定义
 * LCD_D0~D15,由于引脚太多,就不在这里定义了,直接在 lcd_init 里面修改.所以在移植的时候,除了
    改这 6 个 IO 口, 还得改 LCD_Init 里面的 D0~D15 所在的 IO 口.
 */
#define LCD_WR_GPIO_PORT                GPIOD
#define LCD_WR_GPIO_PIN                 GPIO_PIN_5
#define LCD_WR_GPIO_CLK_ENABLE()
           do{ __HAL_RCC_GPIOD_CLK_ENABLE(); }while(0) /* 所在 IO 口时钟使能 */

#define LCD_RD_GPIO_PORT                GPIOD
#define LCD_RD_GPIO_PIN                 GPIO_PIN_4
#define LCD_RD_GPIO_CLK_ENABLE()
           do{ __HAL_RCC_GPIOD_CLK_ENABLE(); }while(0) /* 所在 IO 口时钟使能 */

#define LCD_BL_GPIO_PORT                GPIOB
#define LCD_BL_GPIO_PIN                 GPIO_PIN_5
#define LCD_BL_GPIO_CLK_ENABLE()
       do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)      /* 背光所在 IO 口时钟使能 */

/* LCD_CS(需要根据 LCD_FMC_NEX 设置正确的 IO 口) 和
   LCD_RS(需要根据 LCD_FMC_AX 设置正确的 IO 口) 引脚 定义 */
#define LCD_CS_GPIO_PORT                GPIOD
#define LCD_CS_GPIO_PIN                 GPIO_PIN_7
#define LCD_CS_GPIO_CLK_ENABLE()
           do{ __HAL_RCC_GPIOD_CLK_ENABLE(); }while(0) /* 所在 IO 口时钟使能 */

#define LCD_RS_GPIO_PORT                GPIOD
#define LCD_RS_GPIO_PIN                 GPIO_PIN_13
#define LCD_RS_GPIO_CLK_ENABLE()
           do{ __HAL_RCC_GPIOD_CLK_ENABLE(); }while(0) /* 所在 IO 口时钟使能 */

/* FMC 相关参数 定义
 * 注意: 我们默认是通过 FMC 块 1 来连接 LCD, 块 1 有 4 个片选: FMC_NE1~4
 *
 * 修改 LCD_FMC_NEX, 对应的 LCD_CS_GPIO 相关设置也得改
 * 修改 LCD_FMC_AX , 对应的 LCD_RS_GPIO 相关设置也得改
 */
#define LCD_FMC_NEX         1       /* 使用 FMC_NE1 接 LCD_CS,取值范围只能是: 1~4 */
#define LCD_FMC_AX          18      /* 使用 FMC_A18 接 LCD_RS,取值范围是: 0 ~ 25 */
```

第一部分的宏定义是 LCD RST/WR/RD/BL/CS/RS 引脚定义，需要注意的是：LCD 的 RST 引脚和系统复位脚连接在一起，所以不用单独使用一个 IO 口（节省一个 IO 口）。

第二部分的宏定义是 LCD_FMC_NEX 和 LCD_FMC_AX，这两个宏定义是为计算 LCD 的基地址 LCD_BASE 而定义的。其中 LCD_FMC_NEX 是 FMC 的存储区块号，取值范围只能是: 1~4，因为 LCD 的驱动用到块 1，所以我们默认取值为 1。而 LCD_FMC_AX 是对应 LCD_RS 引脚的 IO 口的复用功能，开发板上 LCD RS 引脚硬件连接 PD13，PD13 要复用为 FMC_A18 引脚。所以 LCD_FMC_AX 定义为 18，18 对应着 FMC_A18，取值范围是: 0 ~ 25。

下面介绍我们在 lcd.h 里面定义的一个重要的结构体：

```
/* LCD 重要参数集 */
typedef struct
{
    uint16_t width;         /* LCD 宽度 */
    uint16_t height;        /* LCD 高度 */
    uint16_t id;            /* LCD ID */
    uint8_t dir;            /* 横屏还是竖屏控制：0，竖屏；1，横屏。 */
    uint16_t wramcmd;       /* 开始写 gram 指令 */
    uint16_t setxcmd;       /* 设置 x 坐标指令 */
    uint16_t setycmd;       /* 设置 y 坐标指令 */
} _lcd_dev;

/* LCD 参数 */
extern _lcd_dev lcddev; /* 管理 LCD 重要参数 */

/* LCD 的画笔颜色和背景色 */
extern uint32_t  g_point_color;     /* 默认红色 */
extern uint32_t  g_back_color;      /* 背景颜色.默认为白色 */
```

该结构体用于保存一些 LCD 重要参数信息，比如 LCD 的长宽、LCD ID（驱动 IC 型号）、LCD 横竖屏状态等，这个结构体虽然占用了十几个字节的内存，但是却可以让我们的驱动函数支持不同尺寸的 LCD，同时可以实现 LCD 横竖屏切换等重要功能，所以还是利大于弊的。最后声明_lcd_dev 结构体类型变量 lcddev，lcddev 在 lcd.c 中定义。

紧接着就是 g_point_color 和 g_back_color 变量的声明，它们也是在 lcd.c 中被定义。g_point_color 变量用于保存 LCD 的画笔颜色，g_back_color 则是保存 LCD 的背景色。

下面是 LCD 背光控制 IO 口的宏定义：

```
/* LCD 背光控制 */
#define LCD_BL(x)   do{ x ? \
      HAL_GPIO_WritePin(LCD_BL_GPIO_PORT, LCD_BL_GPIO_PIN, GPIO_PIN_SET) : \
      HAL_GPIO_WritePin(LCD_BL_GPIO_PORT, LCD_BL_GPIO_PIN, GPIO_PIN_RESET);\
      }while(0)
```

我们知道 TFTLCD 的 RS 接在 FMC 的 A18（即 PD13）上面，CS 接在 FMC_NE1（即 PD7）上，并且是 16 位数据总线。我们使用的是 FMC 存储器 1 的第 1 区，所以我们定义如下 LCD 操作结构体：

```
/* LCD 地址结构体 */
typedef struct
{
    volatile uint16_t LCD_REG;
    volatile uint16_t LCD_RAM;
} LCD_TypeDef;

/* LCD_BASE 的详细解算方法:
 * 我们一般使用 FMC 的块 1(BANK1)来驱动 TFTLCD 液晶屏(MCU 屏), 块 1 地址范围总大小为 256MB,
 * 均分成 4 块:
 * 存储块 1(FMC_NE1)地址范围: 0X6000 0000 ~ 0X63FF FFFF
 * 存储块 2(FMC_NE2)地址范围: 0X6400 0000 ~ 0X67FF FFFF
 * 存储块 3(FMC_NE3)地址范围: 0X6800 0000 ~ 0X6BFF FFFF
 * 存储块 4(FMC_NE4)地址范围: 0X6C00 0000 ~ 0X6FFF FFFF
 *
 * 我们需要根据硬件连接方式选择合适的片选(连接 LCD_CS)和地址线(连接 LCD_RS)
 * H743 开发板使用 FMC_NE1 连接 LCD_CS, FMC_A18 连接 LCD_RS ,16 位数据线,计算方法如下:
 * 首先 FMC_NE1 的基地址为: 0x6000 0000;
   NEX 的基址为(x=1/2/3/4): 0x6000 0000 + (0x400 0000 * (x - 1))
 * FMC_A18 对应地址值: 2^18 * 2 = 0x80000;
   FMC_Ay 对应的地址为(y = 0 ~ 25): 2^y * 2
 *
 * LCD->LCD_REG,对应 LCD_RS = 0(LCD 寄存器);
   LCD->LCD_RAM,对应 LCD_RS = 1(LCD 数据)
```

```
 * 则 LCD->LCD_RAM 的地址为:  0x6000 0000 + 2^18 * 2 = 0x6008 0000
 *    LCD->LCD_REG 的地址可以为 LCD->LCD_RAM 之外的任意地址.
 * 由于我们使用结构体管理 LCD_REG 和 LCD_RAM(REG 在前,RAM 在后,均为 16 位数据宽度)
 * 因此 结构体的基地址(LCD_BASE) = LCD_RAM - 2 = 0x6008 0000 -2
 *
 * 更加通用的计算公式为((片选脚 FSMC_NEX)X=1/2/3/4, (RS 接地址线 FSMC_Ay)y=0~25):
 * LCD_BASE = (0x6000 0000 + (0x400 0000 * (x - 1))) | (2^y * 2 -2)
 * 等效于(使用移位操作)
 * LCD_BASE =(0x6000 0000 + (0x400 0000 * (x - 1))) | ((1 << y) * 2 -2)
 */
#define LCD_BASE        (uint32_t)((0X60000000 + (0X4000000 * (LCD_FMC_NEX -
                                  1))) | (((1 << LCD_FMC_AX) * 2) -2))
#define LCD             ((LCD_TypeDef *) LCD_BASE)
```

其中 LCD_BASE，必须根据我们外部电路的连接来确定，即根据前面介绍的 LCD_FMC_NEX 和 LCD_FMC_AX 宏定义的值来确定。我们使用 Bank1.sector1，把宏定义 LCD_FMC_NEX 的值代入(0X60000000 + (0X4000000 * (LCD_FMC_NEX - 1)))得到的就是存储区的基地址，即 0x60000000。而 FMC_A18 对应地址值为(((1 << LCD_FMC_AX) * 2) -2))，代入 LCD_FMC_AX 的值得到 0x0007 FFFE，所以 LCD_BASE 表示的地址值为 0x60000000 | 0x0007 FFFE。

上面是直接代入我们给出的公式进行计算，但是很多朋友不太明白 FMC_A18 对应地址值怎么来的。下面我们来解析一下，以 FMC_A18 为例，0x0007 FFFE 转换成二进制就是：0111 1111 1111 1111 1110。从表 24.1.2.2 知道 16 位数据时，地址右移一位对齐，那么实际对应到地址引脚的时候，就是：[A18:A0]=011 1111 1111 1111 1111，此时 A18 是 0，但是如果 16 位地址再加 1（注意：对应到 8 位地址是加 2，即 0x0007 FFFE +0X02），那么：[A18:A0]=100 0000 0000 0000 0000，这时 A18 就是 1 了，即实现了对 RS 的 0 和 1 的控制。

定义 LCD 的时候，我们将 LCD_BASE 这个地址强制转换为 LCD_TypeDef 结构体地址，那么可以得到 LCD->LCD_REG 的地址就是 0x6007 FFFE，地址右移一位对齐后对应 A18 的状态为 0(即 RS=0)，而 LCD->LCD_RAM 的地址就是 0x6010 0000（结构体地址自增），地址右移一位对齐后对应 A18 的状态为 1（即 RS=1）。

所以，有了这个定义，当我们要往 LCD 写命令/数据的时候，可以这样写：

```
    LCD->LCD_REG = CMD;     /* 写命令 */
    LCD->LCD_RAM = DATA;    /* 写数据 */
```

而读的时候反过来操作就可以了，如下所示：

```
    CMD = LCD->LCD_REG;     /* 读 LCD 寄存器 */
    DATA = LCD->LCD_RAM;    /* 读 LCD 数据 */
```

这其中，CS、WR、RD 和 IO 口方向都是由 FMC 硬件自动控制，不需要我们手动设置了。

最后是一些其他的宏定义，包括 LCD 扫描方向和颜色，以及 SSD1963 相关配置参数。

下面开始对 lcd.c 文件介绍，先看 LCD 初始化函数，其定义如下：

```
/**
 * @brief       初始化 LCD
 * @note        该初始化函数可以初始化各种型号的 LCD(详见本.c 文件最前面的描述)
 *
 * @param       无
 * @retval      无
 */
void lcd_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;
    FMC_NORSRAM_TimingTypeDef fmc_read_handle;
    FMC_NORSRAM_TimingTypeDef fmc_write_handle;

    LCD_CS_GPIO_CLK_ENABLE();    /* LCD_CS 脚时钟使能 */
    LCD_WR_GPIO_CLK_ENABLE();    /* LCD_WR 脚时钟使能 */
    LCD_RD_GPIO_CLK_ENABLE();    /* LCD_RD 脚时钟使能 */
    LCD_RS_GPIO_CLK_ENABLE();    /* LCD_RS 脚时钟使能 */
    LCD_BL_GPIO_CLK_ENABLE();    /* LCD_BL 脚时钟使能 */
```

```
    gpio_init_struct.Pin = LCD_CS_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;                /* 推挽复用 */
    gpio_init_struct.Pull = GPIO_PULLUP;                    /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;     /* 高速 */
    gpio_init_struct.Alternate = GPIO_AF12_FMC;             /* 复用为 FMC */
    HAL_GPIO_Init(LCD_CS_GPIO_PORT, &gpio_init_struct);     /* 初始化 LCD_CS 引脚 */

    gpio_init_struct.Pin = LCD_WR_GPIO_PIN;
    HAL_GPIO_Init(LCD_WR_GPIO_PORT, &gpio_init_struct);     /* 初始化 LCD_WR 引脚 */

    gpio_init_struct.Pin = LCD_RD_GPIO_PIN;
    HAL_GPIO_Init(LCD_RD_GPIO_PORT, &gpio_init_struct);     /* 初始化 LCD_RD 引脚 */

    gpio_init_struct.Pin = LCD_RS_GPIO_PIN;
    HAL_GPIO_Init(LCD_RS_GPIO_PORT, &gpio_init_struct);     /* 初始化 LCD_RS 引脚 */

    gpio_init_struct.Pin = LCD_BL_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;            /* 推挽输出 */
    HAL_GPIO_Init(LCD_BL_GPIO_PORT, &gpio_init_struct);/* LCD_BL 引脚模式设置 */
    lcd_mpu_config();                                      /* 使能 MPU 保护 LCD 区域 */
    g_sram_handle.Instance = FMC_NORSRAM_DEVICE;
    g_sram_handle.Extended = FMC_NORSRAM_EXTENDED_DEVICE;

    g_sram_handle.Init.NSBank = FMC_NORSRAM_BANK1;      /* 使用 NE1 */
   /* 不复用数据线 */
    g_sram_handle.Init.DataAddressMux = FMC_DATA_ADDRESS_MUX_DISABLE;
    g_sram_handle.Init.MemoryType = FMC_MEMORY_TYPE_SRAM;    /* SRAM */
   /* 16 位数据宽度 */
    g_sram_handle.Init.MemoryDataWidth = FMC_NORSRAM_MEM_BUS_WIDTH_16;
   /* 是否使能突发访问,仅对同步突发存储器有效,此处未用到 */
    g_sram_handle.Init.BurstAccessMode = FMC_BURST_ACCESS_MODE_DISABLE;
   /* 等待信号的极性,仅在突发模式访问下有用 */
    g_sram_handle.Init.WaitSignalPolarity = FMC_WAIT_SIGNAL_POLARITY_LOW;
   /* 存储器是在等待周期之前的一个时钟周期还是等待周期期间使能 NWAIT */
    g_sram_handle.Init.WaitSignalActive = FMC_WAIT_TIMING_BEFORE_WS;
   /* 存储器写使能 */
    g_sram_handle.Init.WriteOperation = FMC_WRITE_OPERATION_ENABLE;
   /* 等待使能位,此处未用到 */
    g_sram_handle.Init.WaitSignal = FMC_WAIT_SIGNAL_DISABLE;
   /* 读写使用不同的时序 */
    g_sram_handle.Init.ExtendedMode = FMC_EXTENDED_MODE_ENABLE;
   /* 是否使能同步传输模式下的等待信号,此处未用到 */
    g_sram_handle.Init.AsynchronousWait = FMC_ASYNCHRONOUS_WAIT_DISABLE;
    g_sram_handle.Init.WriteBurst = FMC_WRITE_BURST_DISABLE;    /* 禁止突发写 */
    g_sram_handle.Init.ContinuousClock = FMC_CONTINUOUS_CLOCK_SYNC_ASYNC;

     /* FMC 读时序控制寄存器 */
     /* 地址建立时间(ADDSET)为 15 个 fmc_ker_ck 1/220M=4.5ns*15=67.5ns */
     fmc_read_handle.AddressSetupTime = 0x0F;
     fmc_read_handle.AddressHoldTime = 0x00;
     fmc_read_handle.DataSetupTime = 0x4E;
     /* 数据保存时间(DATAST)为 78 个 fmc_ker_ck=4.5*78=351ns */
     /* 因为液晶驱动 IC 的读数据的时候,速度不能太快,尤其是个别芯片 */
     fmc_read_handle.AccessMode = FMC_ACCESS_MODE_A;    /* 模式 A */
     /* FMC 写时序控制寄存器 */
     fmc_write_handle.AddressSetupTime = 0x0F;
     /* 地址建立时间(ADDSET)为 15 个 fmc_ker_ck=67.5ns */
     fmc_write_handle.AddressHoldTime = 0x00;
     /* 数据保存时间(DATAST)为 15 个 fmc_ker_ck=67.5ns */
     fmc_write_handle.DataSetupTime = 0x0F;
     fmc_write_handle.AccessMode = FMC_ACCESS_MODE_A;  /* 模式 A */
     HAL_SRAM_Init(&g_sram_handle, &fmc_read_handle, &fmc_write_handle);
```

```
    delay_ms(50);

    /* 尝试 9341 ID 的读取 */
    lcd_wr_regno(0XD3);
    lcddev.id = lcd_rd_data(); /* dummy read */
    lcddev.id = lcd_rd_data(); /* 读到 0X00 */
    lcddev.id = lcd_rd_data(); /* 读取 0X93 */
    lcddev.id <<= 8;
    lcddev.id |= lcd_rd_data();/* 读取 0X41 */

    if (lcddev.id != 0X9341)        /* 不是 9341 , 尝试看看是不是 ST7789 */
    {
        lcd_wr_regno(0X04);
        lcddev.id = lcd_rd_data();          /* dummy read */
        lcddev.id = lcd_rd_data();          /* 读到 0X85 */
        lcddev.id = lcd_rd_data();          /* 读取 0X85 */
        lcddev.id <<= 8;
        lcddev.id |= lcd_rd_data();         /* 读取 0X52 */

        if (lcddev.id == 0X8552)            /* 将 8552 的 ID 转换成 7789 */
        {
            lcddev.id = 0x7789;
        }

        if (lcddev.id != 0x7789)            /* 也不是 ST7789, 尝试是不是 NT35310 */
        {
            lcd_wr_regno(0XD4);
            lcddev.id = lcd_rd_data();      /* dummy read */
            lcddev.id = lcd_rd_data();      /* 读回 0X01 */
            lcddev.id = lcd_rd_data();      /* 读回 0X53 */
            lcddev.id <<= 8;
            lcddev.id |= lcd_rd_data();     /* 这里读回 0X10 */

            if (lcddev.id != 0x5310)  /* 也不是 NT35310,尝试看看是不是 ST7796 */
                {
                    lcd_wr_regno(0XD3);
                    lcddev.id = lcd_rd_data();  /* dummy read */
                    lcddev.id = lcd_rd_data();  /* 读到 0X00 */
                    lcddev.id = lcd_rd_data();  /* 读取 0X77 */
                    lcddev.id <<= 8;
                    lcddev.id |= lcd_rd_data(); /* 读取 0X96 */

                    if (lcddev.id != 0x7796)
                    /* 也不是 ST7796,尝试看看是不是 NT35510 */
                    {
                        /* 发送密钥（厂家提供） */
                        lcd_write_reg(0xF000, 0x0055);
                        lcd_write_reg(0xF001, 0x00AA);
                        lcd_write_reg(0xF002, 0x0052);
                        lcd_write_reg(0xF003, 0x0008);
                        lcd_write_reg(0xF004, 0x0001);

                        lcd_wr_regno(0xC500);       /* 读取 ID 低八位 */
                        lcddev.id = lcd_rd_data();  /* 读回 0x80 */
                        lcddev.id <<= 8;

                        lcd_wr_regno(0xC501);       /* 读取 ID 高八位 */
                        lcddev.id |= lcd_rd_data(); /* 读回 0x00 */
                        delay_ms(5);                /* 等待 5ms, 因为 0XC501 指令对
                                                    1963 来说就是软件复位指令, 等待
                                                    5ms 让 1963 复位完成再操作 */
```

```
                    if (lcddev.id != 0x5510)
                    /* 也不是 NT5510,尝试看看是不是 ILI9806 */
                    {
                        lcd_wr_regno(0XD3);
                        lcddev.id = lcd_rd_data();  /* dummy read */
                        lcddev.id = lcd_rd_data();  /* 读回 0X00 */
                        lcddev.id = lcd_rd_data();  /* 读回 0X98 */
                        lcddev.id <<= 8;
                        lcddev.id |= lcd_rd_data(); /* 读回 0X06 */

                        if (lcddev.id != 0x9806)
                        /* 也不是 ILI9806,尝试看看是不是 SSD1963 */
                        {
                            lcd_wr_regno(0xA1);
                            lcddev.id = lcd_rd_data();
                            lcddev.id = lcd_rd_data();  /* 读回 0x57 */
                            lcddev.id <<= 8;
                            lcddev.id |= lcd_rd_data(); /* 读回 0x61 */

                            if (lcddev.id == 0x5761)lcddev.id = 0x1963;
                            /* SSD1963 读回的 ID 是 5761H,我们强制设置为 1963 */
                        }
                    }
                }
            }
        }
    }

    /* 特别注意, 如果在 main 函数里面屏蔽串口 1 初始化, 则会卡死在 printf
     * 里面(卡死在 f_putc 函数), 所以, 必须初始化串口 1, 或者屏蔽掉下面
     * 这行 printf 语句 !!!!!!!
     */
    printf("LCD ID:%x\r\n", lcddev.id); /* 打印 LCD ID */

    if (lcddev.id == 0X7789)
    {
        lcd_ex_st7789_reginit();        /* 执行 ST7789 初始化 */
    }
    else if (lcddev.id == 0X9341)
    {
        lcd_ex_ili9341_reginit();       /* 执行 ILI9341 初始化 */
    }
    else if (lcddev.id == 0x5310)
    {
        lcd_ex_nt35310_reginit();       /* 执行 NT35310 初始化 */
    }
    else if (lcddev.id == 0x7796)
    {
        lcd_ex_st7796_reginit();        /* 执行 ST7796 初始化 */
    }

    else if (lcddev.id == 0x5510)
    {
        lcd_ex_nt35510_reginit();       /* 执行 NT35510 初始化 */
    }
    else if (lcddev.id == 0x9806)
    {
        lcd_ex_ili9806_reginit();       /* 执行 ILI9806 初始化 */
    }

    else if (lcddev.id == 0X1963)
    {
        lcd_ex_ssd1963_reginit();     /* 执行 SSD1963 初始化 */
```

```
        lcd_ssd_backlight_set(100);  /* 背光设置为最亮 */
    }

    /* 初始化完成以后,提速 */
    if (lcddev.id == 0x9341 || lcddev.id == 0x7789)
    {/* 如果是这几个 IC, 则设置 WR 时序为最快 */
        /* 重新配置写时序控制寄存器的时序 */
        fmc_write_handle.AddressSetupTime = 5;
        fmc_write_handle.DataSetupTime = 5;
        FMC_NORSRAM_Extended_Timing_Init(g_sram_handle.Extended,
                                &fmc_write_handle, g_sram_handle.Init.NSBank,
                                    g_sram_handle.Init.ExtendedMode);
    }
    if (lcddev.id == 0x1963 )  /* 如果是这几个 IC, 则设置 WR 时序为最快 */
    {
       /* 重新配置写时序控制寄存器的时序 */
       fmc_write_handle.AddressSetupTime = 3;
       /* 地址建立时间(ADDSET)为 3 个 fmc_ker_ck = 13.5 ns */
       fmc_write_handle.DataSetupTime = 3;
       /* 数据保存时间(DATAST)为 3 个 fmc_ker_ck = 13.5 ns */
       FMC_NORSRAM_Extended_Timing_Init(g_sram_handle.Extended,
                                &fmc_write_handle, g_sram_handle.Init.NSBank,
                                  g_sram_handle.Init.ExtendedMode);
        }

if (lcddev.id == 0x5310 || lcddev.id == 0x5510|| lcddev.id == 0x7796
     ||lcddev.id == 0x9806)
    {/* 如果是这几个 IC, 则设置 WR 时序为最快 */
        /* 重新配置写时序控制寄存器的时序 */
        fmc_write_handle.AddressSetupTime = 3;
        fmc_write_handle.DataSetupTime = 3;
        FMC_NORSRAM_Extended_Timing_Init(g_sram_handle.Extended,
                                  &fmc_write_handle,g_sram_handle.Init.NSBank,
                                      g_sram_handle.Init.ExtendedMode);
    }

    lcd_display_dir(0); /* 默认为竖屏 */
    LCD_BL(1);          /* 点亮背光 */
    lcd_clear(WHITE);
}
```

该函数先对 FMC 相关 IO 进行初始化，然后使用 HAL_SRAM_Init 函数初始化 FMC 控制器，同时我们使用 HAL_SRAM_MspInit 回调函数来初始化相应的 IO 口，最后读取 LCD 控制器的型号，根据控制 IC 的型号执行不同的初始化代码，这样提高了整个程序的通用性。为了简化 lcd.c 的初始化程序，不同控制 IC 的芯片对应的初始化程序（如：lcd_ex_st7789_reginit()、lcd_ex_ili9341_reginit()等）我们放在 lcd_ex.c 文件中，这些初始化代码完成对 LCD 寄存器的初始化，由 LCD 厂家提供，一般是不需要做任何修改的，我们直接调用就可以了。

下面是 6 个简单，但是很重要的函数：

```
/**
 * @brief           LCD 写数据
 * @param           data: 要写入的数据
 * @retval          无
 */
void lcd_wr_data(volatile uint16_t data)
{
    data = data;            /* 使用-O2 优化的时候,必须插入的延时 */
    LCD->LCD_RAM = data;
}

/**
 * @brief           LCD 写寄存器编号/地址函数
 * @param           regno: 寄存器编号/地址
```

```
 * @retval          无
 */
void lcd_wr_regno(volatile uint16_t regno)
{
    regno = regno;          /* 使用-O2 优化的时候,必须插入的延时 */
    LCD->LCD_REG = regno;   /* 写入要写的寄存器序号 */
}

/**
 * @brief           LCD 写寄存器
 * @param           regno:寄存器编号/地址
 * @param           data:要写入的数据
 * @retval          无
 */
void lcd_write_reg(uint16_t regno, uint16_t data)
{
    LCD->LCD_REG = regno;    /* 写入要写的寄存器序号 */
    LCD->LCD_RAM = data;     /* 写入数据 */
}

/**
 * @brief       LCD 延时函数,仅用于部分在 mdk -O1 时间优化时需要设置的地方
 * @param       i:延时的数值
 * @retval      无
 */
static void lcd_opt_delay(uint32_t i)
{
    while (i--);
}

/**
 * @brief           LCD 读数据
 * @param           无
 * @retval          读取到的数据
 */
static uint16_t lcd_rd_data(void)
{
   volatile uint16_t ram;  /* 防止被优化 */
    lcd_opt_delay(2);
    ram = LCD->LCD_RAM;
    return ram;
}

/**
 * @brief       准备写 GRAM
 * @param       无
 * @retval      无
 */
void lcd_write_ram_prepare(void)
{
    LCD->LCD_REG = lcddev.wramcmd;
}
```

因为 FMC 自动控制了 WR/RD/CS 等这些信号，所以这 6 个函数实现起来都非常简单，我们就不多说，注意，上面有几个函数，我们添加了一些对 MDK –O2 优化的支持，去掉的话，在-O2 优化的时候会出问题。这些函数实现功能见函数前面的备注，通过这几个简单函数的组合，我们就可以对 LCD 进行各种操作了。

下面要介绍的函数是坐标设置函数，该函数代码如下：

```
/**
 * @brief       设置光标位置(对 RGB 屏无效)
 * @param       x,y: 坐标
 * @retval      无
```

```
 */
void lcd_set_cursor(uint16_t x, uint16_t y)
{
    if (lcddev.id == 0X1963)
    {
        if (lcddev.dir == 0)        /* 竖屏模式, x 坐标需要变换 */
        {
            x = lcddev.width - 1 - x;
            lcd_wr_regno(lcddev.setxcmd);
            lcd_wr_data(0);
            lcd_wr_data(0);
            lcd_wr_data(x >> 8);
            lcd_wr_data(x & 0XFF);
        }
        else                        /* 横屏模式 */
        {
            lcd_wr_regno(lcddev.setxcmd);
            lcd_wr_data(x >> 8);
            lcd_wr_data(x & 0XFF);
            lcd_wr_data((lcddev.width - 1) >> 8);
            lcd_wr_data((lcddev.width - 1) & 0XFF);
        }

        lcd_wr_regno(lcddev.setycmd);
        lcd_wr_data(y >> 8);
        lcd_wr_data(y & 0XFF);
        lcd_wr_data((lcddev.height - 1) >> 8);
        lcd_wr_data((lcddev.height - 1) & 0XFF);
    }
    else if (lcddev.id == 0X5510)
    {
        lcd_wr_regno(lcddev.setxcmd);
        lcd_wr_data(x >> 8);
        lcd_wr_regno(lcddev.setxcmd + 1);
        lcd_wr_data(x & 0XFF);
        lcd_wr_regno(lcddev.setycmd);
        lcd_wr_data(y >> 8);
        lcd_wr_regno(lcddev.setycmd + 1);
        lcd_wr_data(y & 0XFF);
    }
    else    /* 9341/5310/7789/7796/9806 等 设置坐标 */
    {
        lcd_wr_regno(lcddev.setxcmd);
        lcd_wr_data(x >> 8);
        lcd_wr_data(x & 0XFF);
        lcd_wr_regno(lcddev.setycmd);
        lcd_wr_data(y >> 8);
        lcd_wr_data(y & 0XFF);
    }
}
```

该函数实现将 LCD 的当前操作点设置到指定坐标(x,y)。因为 9341/5310/7789/7796/9806 等的设置有些不太一样，所以进行了区别对待。

接下来介绍画点函数，其定义如下：

```
/**
 * @brief       画点
 * @param       x,y: 坐标
 * @param       color: 点的颜色(32 位颜色,方便兼容 LTDC)
 * @retval      无
 */
void lcd_draw_point(uint16_t x, uint16_t y, uint32_t color)
{
    lcd_set_cursor(x, y);           /* 设置光标位置 */
    lcd_write_ram_prepare();    /* 开始写入 GRAM */
    LCD->LCD_RAM = color;
```

```
}
```

该函数实现比较简单，就是先设置坐标，然后往坐标写颜色。lcd_draw_point 函数虽然简单，但是至关重要，其他几乎所有上层函数，都是通过调用这个函数实现的。

下面介绍读点函数，用于读取 LCD 的 GRAM，这里说明一下，为什么 OLED 模块没做读 GRAM 的函数，而这里做了。因为 OLED 模块是单色的，所需要全部 GRAM 也就 1K 个字节，而 TFTLCD 模块为彩色的，点数也比 OLED 模块多很多，以 16 位色计算，一款 320×240 的液晶，需要 320×240×2 个字节来存储颜色值，也就是也需要 150K 字节，这对任何一款单片机来说，都不是一个小数目了。而且我们在图形叠加的时候，可以先读回原来的值，然后写入新的值，在完成叠加后，我们又恢复原来的值。这样在做一些简单菜单的时候，是很有用的。这里我们读取 TFTLCD 模块数据的函数为 lcd_read_point，该函数直接返回读到的 GRAM 值。该函数使用之前要先设置读取的 GRAM 地址，通过 lcd_set_cursor 函数来实现。lcd_read_point 的代码如下：

```
/**
 * @brief       读取个某点的颜色值
 * @param       x,y:坐标
 * @retval      此点的颜色(32 位颜色,方便兼容 LTDC)
 */
uint32_t lcd_read_point(uint16_t x, uint16_t y)
{
    uint16_t r = 0, g = 0, b = 0;

    if (x >= lcddev.width || y >= lcddev.height)return 0;/* 超过了范围,直接返回 */
    lcd_set_cursor(x, y);       /* 设置坐标 */

    if (lcddev.id == 0X5510)
    {
        lcd_wr_regno(0X2E00);   /* 5510 发送读 GRAM 指令 */
    }
    else
    {
        lcd_wr_regno(0X2E);
        /* 9341/5310/1963/7789/7796/9806 等发送读 GRAM 指令 */
    }
    r = lcd_rd_data();          /* 假读(dummy read) */

    if (lcddev.id == 0X1963)return r;   /* 1963 直接读就可以 */
    r = lcd_rd_data();                  /* 实际坐标颜色 */
    if (lcddev.id == 0x7796)    /* 7796 一次读取一个像素值 */
    {
        return r;
    }

    /* 9341/5310/5510/7789/9806 要分 2 次读出 */
    b = lcd_rd_data();
    /* 对于 9341/5310/5510/7789/9806,第一次读取的是 RG 的值,R 在前,G 在后,各占 8 位 */
    g = r & 0XFF;
    g <<= 8;
    /* ILI9341/NT35310/NT35510/ST7789/ILI9806 需要公式转换一下 */
    return (((r >> 11) << 11) | ((g >> 10) << 5) | (b >> 11));
}
```

在 lcd_read_point 函数中，因为我们的代码不止支持一种 LCD 驱动器，所以，我们根据不同的 LCD 驱动器（(lcddev.id）型号，执行不同的操作，以实现对各个驱动器兼容，提高函数的通用性。

第十个要介绍的是字符显示函数 lcd_show_char，该函数同前面 OLED 模块的字符显示函数差不多，但是这里的字符显示函数多了 1 个功能，就是可以以叠加方式显示，或者以非叠加方式显示。叠加方式显示多用于在显示的图片上再显示字符。非叠加方式一般用于普通的显示。该函数实现代码如下：

```
/**
```

```
 * @brief       在指定位置显示一个字符
 * @param       x,y  : 坐标
 * @param       chr  : 要显示的字符:" "--->"~"
 * @param       size : 字体大小 12/16/24/32
 * @param       mode : 叠加方式(1); 非叠加方式(0);
 * @param       color: 字体颜色
 * @retval      无
 */
void lcd_show_char(uint16_t x, uint16_t y, char chr, uint8_t size,
                   uint8_t mode, uint16_t color)
{
    uint8_t temp, t1, t;
    uint16_t y0 = y;
    uint8_t csize = 0;
    uint8_t *pfont = 0;
    /* 得到字体一个字符对应点阵集所占的字节数 */
    csize = (size / 8 + ((size % 8) ? 1 : 0)) * (size / 2);
    /* 得到偏移后的值（ASCII 字库是从空格开始取模，所以-' '就是对应字符的字库） */
    chr = chr - ' ';

    switch (size)
    {
        case 12:
            pfont = (uint8_t *)asc2_1206[chr];  /* 调用 1206 字体 */
            break;

        case 16:
            pfont = (uint8_t *)asc2_1608[chr];  /* 调用 1608 字体 */
            break;

        case 24:
            pfont = (uint8_t *)asc2_2412[chr];  /* 调用 2412 字体 */
            break;

        case 32:
            pfont = (uint8_t *)asc2_3216[chr];  /* 调用 3216 字体 */
            break;

        default:
            return ;
    }

    for (t = 0; t < csize; t++)
    {
        temp = pfont[t];                /* 获取字符的点阵数据 */
        for (t1 = 0; t1 < 8; t1++)    /* 一个字节 8 个点 */
        {
            if (temp & 0x80)            /* 有效点,需要显示 */
            {
                lcd_draw_point(x, y, color);        /* 画点出来,要显示这个点 */
            }
            else if (mode == 0)                       /* 无效点,不显示 */
            {
              /* 画背景色,相当于这个点不显示(注意背景色由全局变量控制) */
                lcd_draw_point(x, y, g_back_color);
            }
            temp <<= 1; /* 移位, 以便获取下一个位的状态 */
            y++;
            if (y >= lcddev.height)return;  /* 超区域了 */
            if ((y - y0) == size)             /* 显示完一列了? */
            {
                y = y0; /* y 坐标复位 */
                x++;    /* x 坐标递增 */
```

```
                if (x >= lcddev.width)return;    /* x 坐标超区域了 */
                break;
            }
        }
    }
}
```

在 lcd_show_char 函数里面，我们用到了四个字符集点阵数据数组 asc2_1206、asc2_1608、asc2_2412 和 asc2_3216。

lcd.c 的函数比价多，其他的函数请大家自行查看源码，都有详细的注释。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t x = 0;
    uint8_t lcd_id[12];

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    lcd_init();                             /* 初始化 LCD */

    g_point_color = RED;
    sprintf((char *)lcd_id,"LCD ID:%04X",lcddev.id);/*将 LCD ID 打印到 lcd_id 数组*/

    while (1)
    {
        switch (x)
        {
            case 0:     lcd_clear(WHITE);   break;
            case 1:     lcd_clear(BLACK);   break;
            case 2:     lcd_clear(BLUE);    break;
            case 3:     lcd_clear(RED);     break;
            case 4:     lcd_clear(MAGENTA); break;
            case 5:     lcd_clear(GREEN);   break;
            case 6:     lcd_clear(CYAN);    break;
            case 7:     lcd_clear(YELLOW);  break;
            case 8:     lcd_clear(BRRED);   break;
            case 9:     lcd_clear(GRAY);    break;
            case 10:    lcd_clear(LGRAY);   break;
            case 11:    lcd_clear(BROWN);   break;
        }

        lcd_show_string(10, 40, 240, 32, 32, "STM32", RED);
        lcd_show_string(10, 80, 240, 24, 24, "TFTLCD TEST", RED);
        lcd_show_string(10, 110, 240, 16, 16, "ATOM@ALIENTEK", RED);
        lcd_show_string(10, 130, 240, 16, 16, (char*)lcd_id, RED);
       /* 显示 LCD ID*/

        x++;
        if (x == 12)x = 0;

        LED0_TOGGLE();  /* 红灯闪烁 */
        delay_ms(1000);
    }
}
```

main 函数功能主要是显示一些固定的字符，字体大小包括 32*16、24*12、16*8 和 12*6 四

种，同时显示 LCD 驱动 IC 的型号，然后不停的切换背景颜色，每 1s 切换一次。而 LED0 也会不停的闪烁，指示程序已经在运行了。其中我们用到一个 sprintf 的函数，该函数用法同 printf，只是 sprintf 把打印内容输出到指定的内存区间上，sprintf 的详细用法，请百度学习。

特别注意：

usart_init 函数，不能去掉，因为在 LCD_Init 函数里面调用了 printf，所以一旦去掉这个初始化，就会死机！实际上，只要你的代码有用到 printf，就必须初始化串口，否则都会死机，即停在 usart.c 里面的 fputc 函数，出不来。

25.4 下载验证

下载代码后，LED0 不停的闪烁，提示程序已经在运行了。同时可以看到 TFTLCD 模块的显示背景色不停切换，如图 25.4.1 所示：

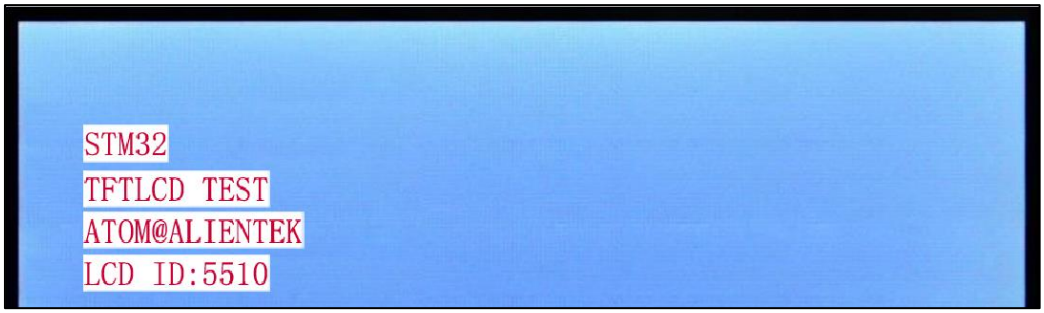


图 25.4.1 TFTLCD 显示效果图

此外，为了让大家能直观的了解 LCD 屏的扫描方式，我们额外编写了两个 main.c 文件（main1.c 和 main2.c，放到 User 文件夹中），方便大家编译下载，观察现象。

使用方法：关闭工程后，先把原实验中的 main.c 改成其他名字，然后把 main1.c 重命名为 main.c，双击 keilkill.bat 清理编译的中间文件，最后打开工程重新编译下载，就可以观察实验现象。观察了 main1.c，可以再观察 main2.c，main2.c 文件的操作方法类似。这两个 main.c 文件的程序非常简单，这里就不讲解，具体请看源码。

第二十六章 SDRAM 实验

STM32H743IIT6 自带了 1060K 字节的 SRAM，对一般应用来说，已经足够了，不过在一些对内存要求高的场合，STM32H743 自带的这些内存就不够用了。比如使用 LTDC 驱动 RGB 屏、跑算法或者跑 GUI 等，就可能不太够用。因此，阿波罗 STM32H743 开发板板载了一颗 32M 字节容量的 SDRAM 芯片，支持 W9825G6KH 和 EM63A165TS-6G 等多种型号。这些型号在功能参数、容量及管脚定义上完全一致，可直接互换使用，能够满足大内存应用需求。

本章以 W9825G6KH 为例介绍驱动流程，其他兼容型号的驱动流程大同小异。接下来我们将完成对 W9825G6KH 的访问控制，并测试其容量。本章主要包含以下内容：

26.1 SDRAM 简介
26.2 硬件设计
26.3 程序设计
26.4 下载验证

26.1 SDRAM 简介

本章我们将通过经过 STM32H743 的 FMC 接口，来驱动 W9825G6KH 这颗芯片，本节我们将介绍 SDRAM 相关知识点，包括，1，SDRAM 简介；2，FMC SDRAM 接口简介。

26.1.1 SDRAM 简介

SDRAM，英文名是： Synchronous Dynamic Random Access Memory，即同步动态随机存储器，相较于 SRAM（静态存储器），SDRAM 具有：容量大和价格便宜的特点。 STM32H743 支持 SDRAM，因此，我们可以外挂 SDRAM，从而大大降低外扩内存的成本。

阿波罗板载的 SDRAM 型号为：W9825G6KH，其内部结构框图如图 26.1.1.1 所示：

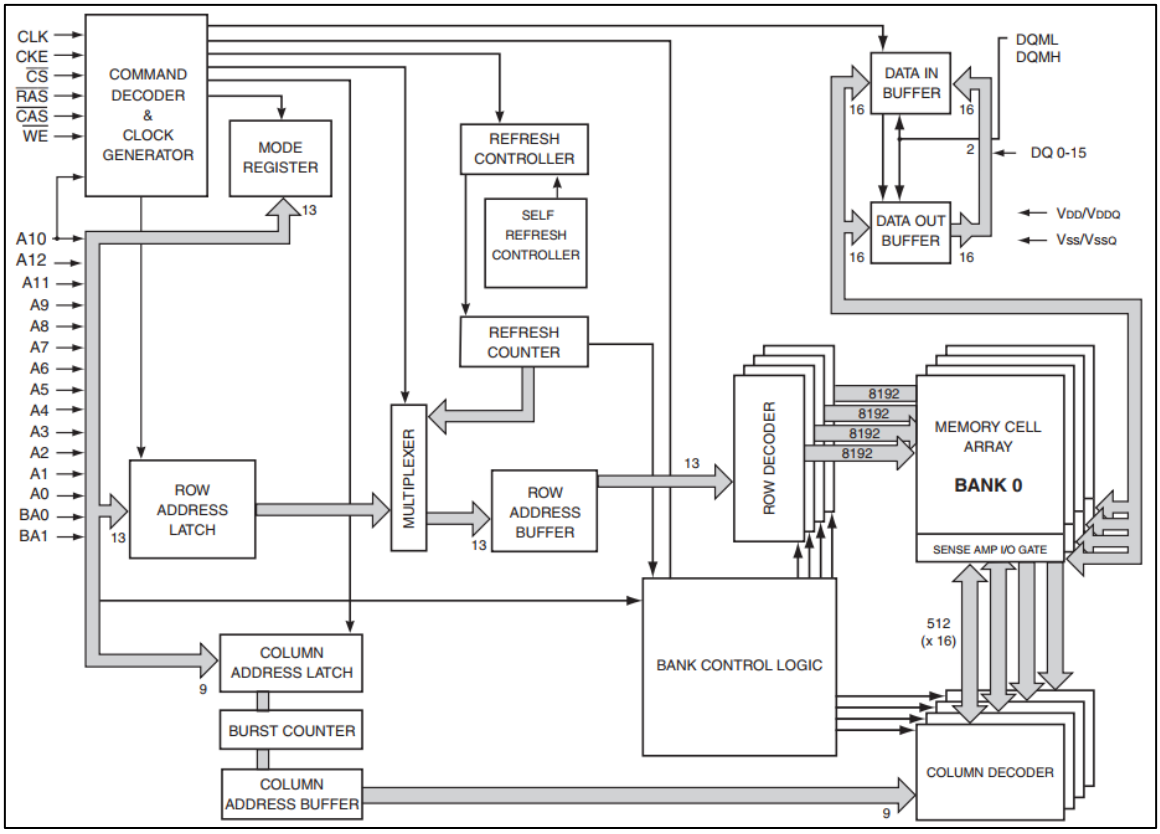


图 26.1.1.1 W9825G6KH 内部结构框图

接下来，我们结合图 26.1.1.1，对 SDRAM 的几个重要知识点进行介绍。

（1） SDRAM 信号线

SDRAM 信号线如表 26.1.1.1 所示：

信号线	说明
CLK	时钟信号，在该时钟的上升沿采集输入信号
CKE	时钟使能，禁止时钟时，SDRAM 会进入自刷新模式
CS#	片选信号，低电平有效
RAS#	行地址选通信号，低电平时，表示行地址
CAS#	列地址选通信号，低电平时，表示列地址
WE#	写使能信号，低电平有效
A0~A12	地址线（行/列）
BA0，BA1	BANK 地址线
DQ0~15	数据线
DQML，DQMH	数据掩码，表示 DQ 的有效部分

表 26.1.1.1 SDRAM 信号线

（2） 存储单元

SDRAM 的存储单元（称之为：BANK）是以阵列的形式排列的，如图 26.1.1.2 所示：

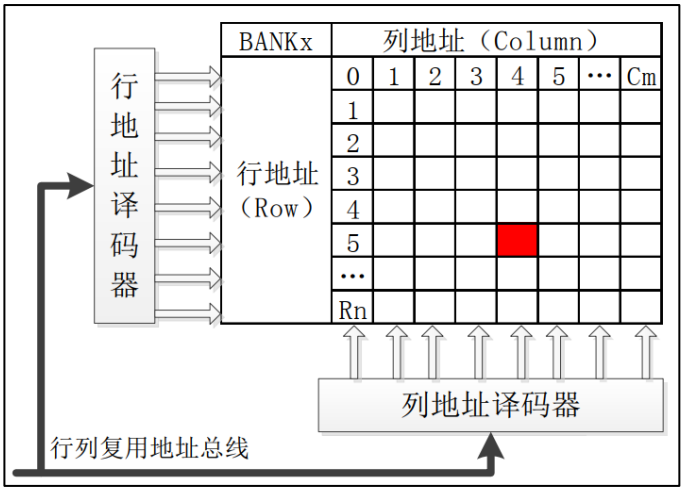


图 26.1.1.2 SDRAM BANK 结构示意图

对于这个存储阵列，我们可以将其看成是一个表格，只需要给定行地址和列地址，就可以确定其唯一位置，这就是 SDRAM 寻址的基本原理。而一个 SDRAM 芯片内部，一般又有 4 个这样的存储单元（BANK），所以，在 SDRAM 内部寻址的时候，先指定 BANK 号和行地址，然后再指定列地址，就可以查找到目标地址。

SDRAM 的存储结构示意图，如图 26.1.1.3 所示，寻址的时候，首先 RAS 信号为低电平，选通行地址，地址线 A0~A12 所表示的地址，会被传输并锁存到行地址译码器里面，最为行地址，同时 BANK 地址线上面的 BS0，BS1 所表示的 BANK 地址，也会被锁存，选中对应的 BANK，然后， CAS 信号为低电平，选通列地址，地址线 A0~A12 所表示的地址，会被传输并锁存到列地址译码器里面，作为列地址，这样，就完成了一次寻址。

W9825G6KH 的存储结构为：行地址：8192 个；列地址：512 个； BANK 数： 4 个；位宽：16 位；这样，整个芯片的容量为： 8192*512*4*16=32M 字节。

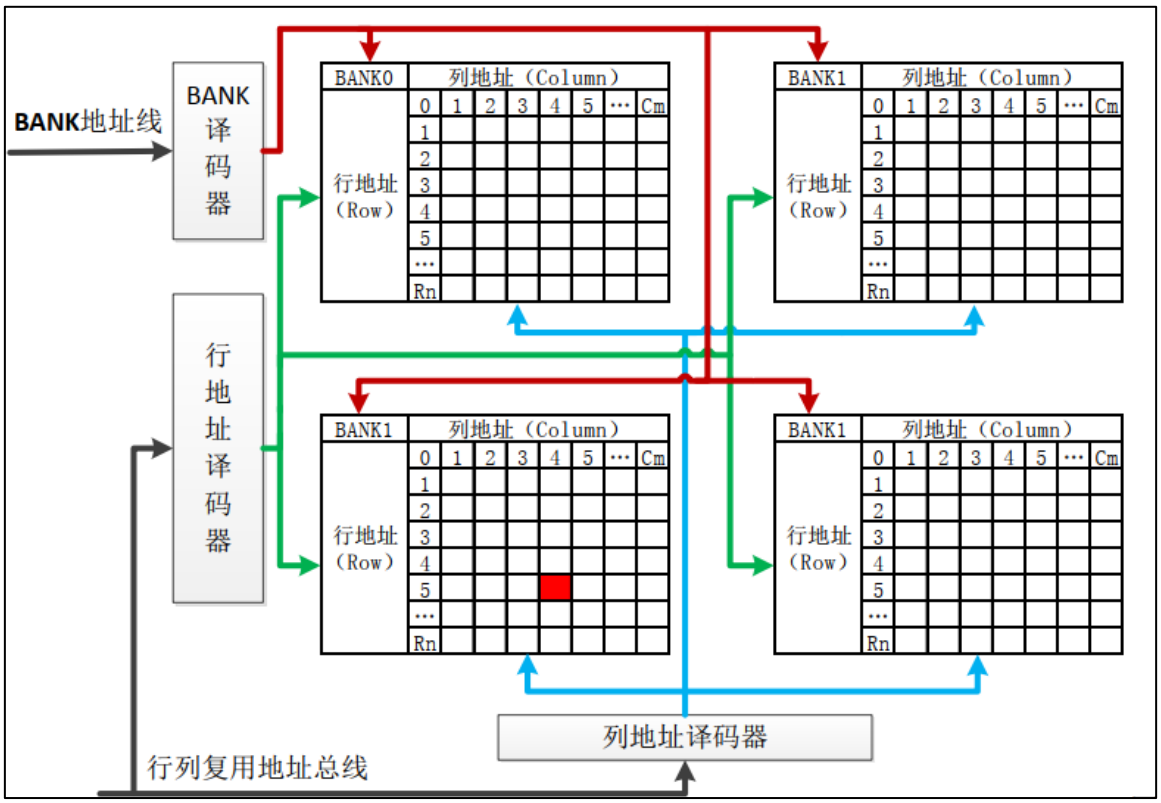


图 26.1.1.3 SDRAM 存储结构图

（3） 数据传输

在完成寻址以后，数据线 DQ0~DQ15 上面的数据会通过图 26.1.1.1 中所示的数据控制逻辑写入（或读出）存储阵列。

特别注意：因为 SDRAM 的位宽，可以达到 32 位，也就是最多有 32 条数据线，在实际使用的时候，我们可能会以：8 位、16 位、24 位和 32 位等宽度来读写数据，这样的话，并不是每条数据线，都会被使用到，未被用到的数据线上面的数据，必须被忽略，这个时候就需要用到数据掩码（ DQM）线来控制了，每一个数据掩码线，对应 8 个位的数据，低电平表示对应数据位有效，高电平表示对应数据位无效。

以 W9825G6KH 为例，假设以 8 位数据访问， 我们只需要 DQ0~DQ7 的数据，而 DQ8~DQ15 的数据需要忽略，此时，我们只需要设置 LDQM 为低电平， UDQM 为高电平，就可以了。

（4） 控制命令

SDRAM的驱动需要用到一些命令，我们列出几个常用的命令给大家做讲解，如表26.1.1.2所示：

命令	CS	RAS	CAS	WE	DQM	ADDR	DQ
NO-Operation	L	H	H	H	X	X	X
Active	L	L	H	H	X	Bank/Row	X
Read	L	H	L	H	L/H	Bank/Col	DATA
Write	L	H	L	L	L/H	Bank/Col	DATA
Precharge	L	L	H	L	X	A10=H/L	X
Refresh	L	L	L	H	X	X	X
Mode Register Set	L	L	L	L	X	MODE	X
Burst Stop	L	H	H	L	X	X	DATA

表 26.1.1.2 SDRAM 控制命令

1， NO-Operation

NO-Operation，即空操作命令，用于选中 SDRAM，防止 SDRAM 接受错误的命令，为接下来的命令发送做准备。

2， Active

Active，即激活命令，该命令必须在读写操作之前被发送，用于设置所需要的Bank和行地址（同时设置这 2 个地址），Bank 地址由 BS0，BS1（也写作 BA0，BA1，下同）指定，行地址由 A0~A12 指定，时序图如图 26.1.1.4 所示：

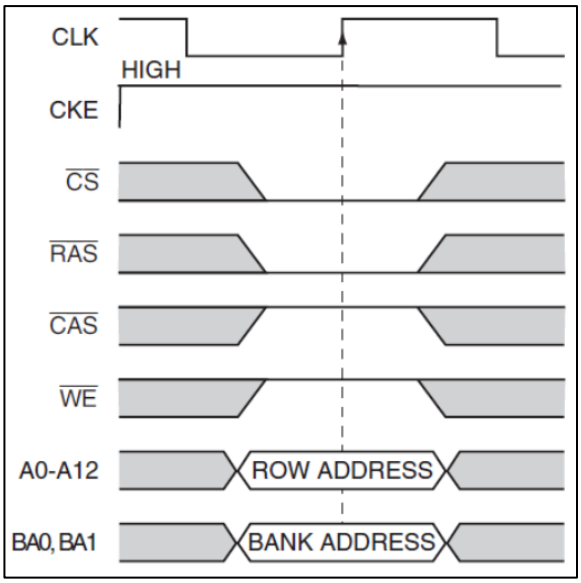


图 26.1.1.4 激活命令时序图

3， Read/Write

Read/Write，即读/写命令，在发送完激活命令后，再发送列地址就可以完成对 SDRAM 的寻址，并进行读写操作了，读/写命令和列地址的发送，是通过一次传输完成的，如图 26.1.1.5 所示：

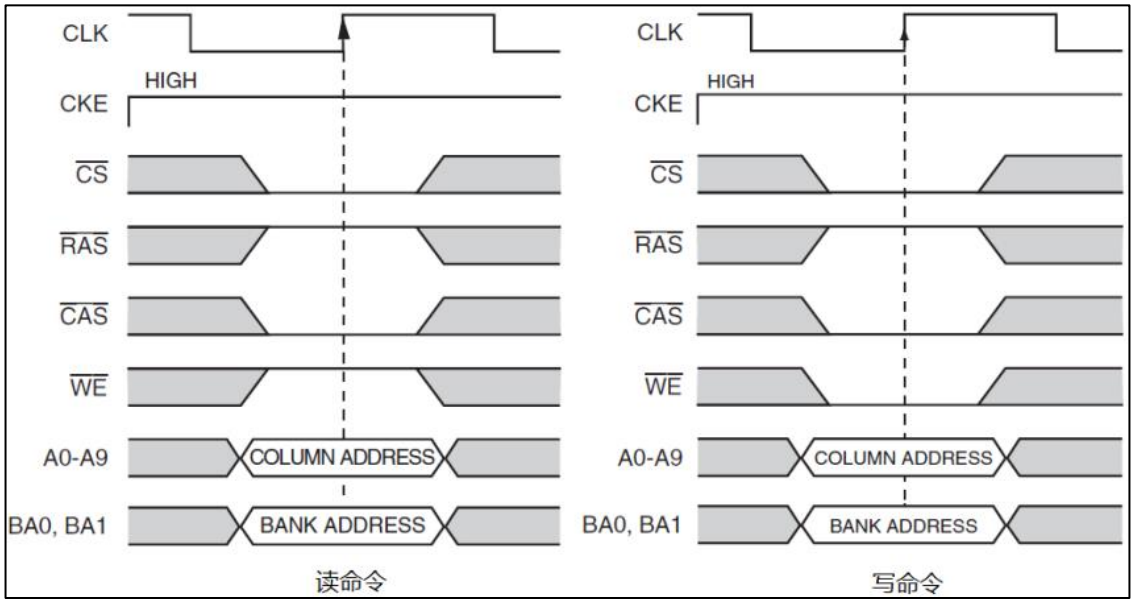


图 26.1.1.5 读/写命令时序图

列地址由 A0~A9 指定，WE 信号控制读/写命令，高电平表示读命令，低电平表示写命令，各条信号线的状态，在 CLK 的上升沿被锁存到芯片内部。

4， Percharge

Percharge，即预充电指令，用于关闭 Bank 中所打开的行地址。由于 SDRAM 的寻址具有独占性，所以在进行完读写操作后，如果要对同一 Bank 的另一行进行寻址，就要将原来有效（打开）的行关闭，重新发送行/列地址。Bank 关闭现有行，准备打开新行的操作就叫做预充电（Percharge）。

预充电命令时序，如图 26.1.1.6 所示：

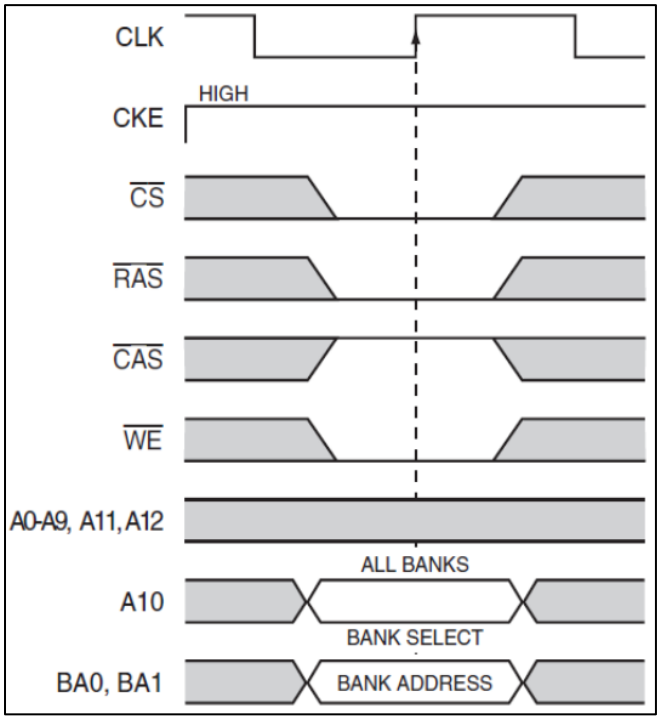


图 26.1.1.6 预充电命令时序图

预充电命令可以通过独立的命令发送，也可以在每次发送读/写命令的时候，使用地址线 A10，来设置自动预充电。在发送读/写命令的时候，当 A10=1，则使能所有 Bank 的预充电，在读/写操作完成后，自动进行预充电。这样，下次读/写操作之前，就不需要再发预充电命令了，从而提高读/写速度。

5， Refresh

Refresh，即刷新命令，用于刷新一行数据。SDRAM 里面存储的数据，需要不断的进行刷新操作才能保留住，因此刷新命令对于 SDRAM 来说，尤为重要。预充电命令和刷新命令，都可以实现对 SDRAM 数据的刷新，不过预充电仅对当前打开的行有效（仅刷新当前行），而刷新命令，则可以依次对所有的行进行刷新操作。

总共有两种刷新操作：自动刷新（ Auto Refresh）和自我刷新（ Self Refresh），在发送 Refresh 命令时，如果 CKE 有效（高电平），则使用自动刷新模式，否则使用自我刷新模式。不论是何种刷新方式，都不需要外部提供行地址信息，因为这是一个内部的自动操作。

自动刷新： SDRAM 内部有一个行地址生成器（也称刷新计数器）用来自动的依次生成要刷新的行地址。由于刷新是针对一行中的所有存储体进行，所以无需列寻址。刷新涉及到所有 Bank，因此在刷新过程中，所有 Bank 都停止工作，而每次刷新所占用的时间为 9 个时钟周期（ PC133 标准），之后就可进入正常的工作状态，也就是说在这 9 个时钟期间内，所有工作指令只能等待而无法执行。刷新操作必须不停的执行，完成一次所有行的刷新所需要的时间，称为刷新周期，一般为 64ms。显然，刷新操作肯定会对 SDRAM 的性能造成影响，但这是没办法的事情，也是 DRAM 相对于 SRAM（静态内存，无需刷新仍能保留数据）取得成本优势的同时所付出的代价。

自我刷新：主要用于休眠模式低功耗状态下的数据保存，在发出自动刷新命令时，将 CKE 置于无效状态（低电平），就进入了自我刷新模式，此时不再依靠系统时钟工作，而是根据内部的时钟进行刷新操作。在自我刷新期间除了 CKE 之外的所有外部信号都是无效的（无需外部提供刷新指令），只有重新使 CKE 有效（高电平）才能退出自刷新模式并进入正常操作状态。

6， Mode Register Set

Mode Register Set，即设置模式寄存器。 SDRAM 芯片内部有一个逻辑控制单元，控制单元的相关参数由模式寄存器提供，我们通过设置模式寄存器命令，来完成对模式寄存器的设置，

这个命令在每次对 SDRAM 进行初始化的时候，都需要用到。

发送该命令时，通过地址线来传输模式寄存器的值，W9825G6KH 的模式寄存器描述如图 26.1.1.7 所示：

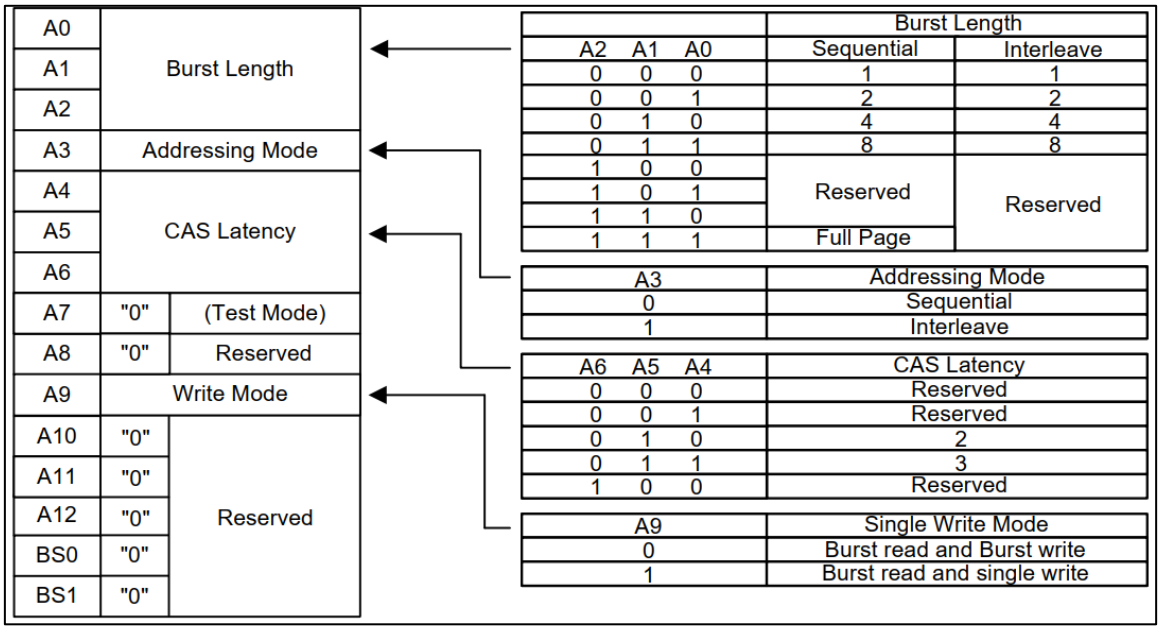


图 26.1.1.7 W9825G6KH 的模式寄存器

由图可知，模式寄存器的配置分为几个部分：

Burst Length，即突发长度（简称 BL），通过 A0~A2 设置， 是指在同一行中相邻的存储单元连续进行数据传输的方式，连续传输所涉及到存储单元（列）的数量就是突发长度。

前面我们说的读/写操作，都是一次对一个存储单元进行寻址，如果要连续读/写就还要对当前存储单元的下一个单元进行寻址，也就是要不断的发送列地址与读/写命令（行地址不变，所以不用再对行寻址）。虽然由于读/写延迟相同可以让数据的传输在 I/O 端是连续的，但它占用了大量的内存控制资源，在数据进行连续传输时无法输入新的命令，效率很低。

为此，人们开发了突发传输技术，只要指定起始列地址与突发长度，内存就会依次地自动对后面相应数量的存储单元进行读/写操作而不再需要控制器连续地提供列地址。 这样，除了第一个数据的传输需要若干个周期外，其后每个数据只需一个周期的即可获得。

非突发连续读取模式：不采用突发传输而是依次单独寻址，此时可等效于 BL=1。虽然可以让数据是连续的传输，但每次都要发送列地址与命令信息，控制资源占用极大。突发连续读取模式：只要指定起始列地址与突发长度，寻址与数据的读取自动进行，而只要控制好两段突发读取命令的间隔周期（与 BL 相同）即可做到连续的突发传输。 至于 BL 的数值，也是不能随便设或在数据进行传输前临时决定，而是在初始化的时候，通过模式寄存器设置命令，进行设置。 目前可用的选项是 1、 2、 4、 8、全页（ Full Page），常见的设定是 4 和 8。若传输长度小于突发长度，则需要发送 Burst Stop（停止突发）命令，结束突发传输。

Addressing Mode，即突发访问的地址模式，通过 A3 设置，可以设置为： Sequential（顺序）或 Interleave（交错）。顺序方式，地址连续访问，而交错模式则地址是乱序的，一般选择连续模式。

CAS Latency，即列地址选通延迟（简称 CL）。在读命令（同时发送列地址）发送完之后，需要等待几个时钟周期， DQ 数据线上的数据，才会有效，这个延迟时间，就叫 CL，一般设置为 2/3 个时钟周期，如图 26.1.1.8 所示：

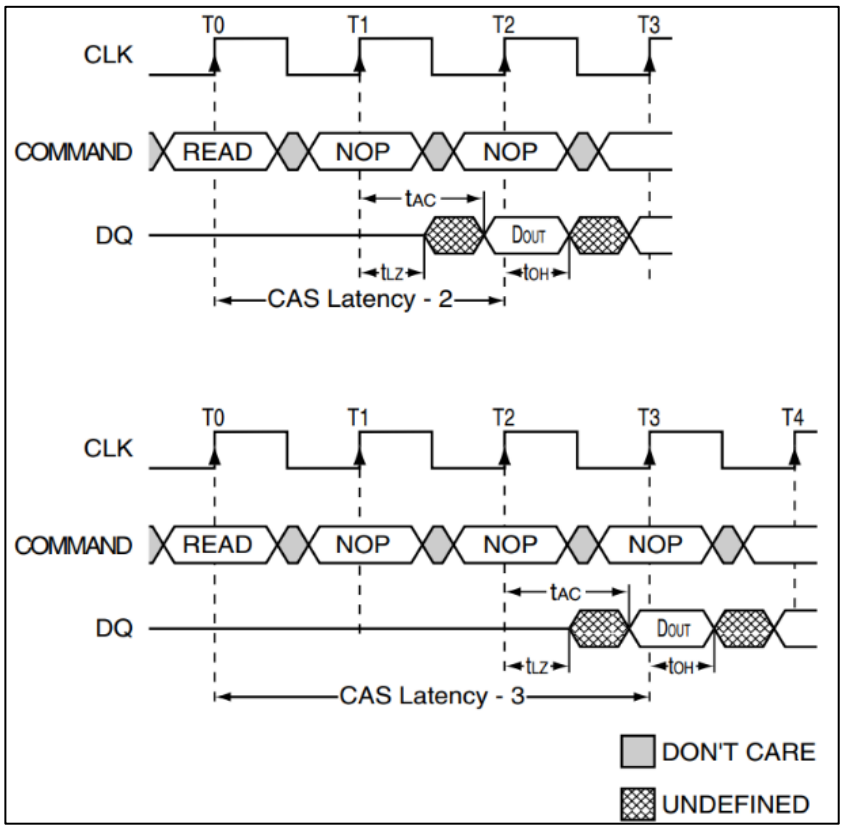


图 26.1.1.8 CAS 延迟（2/3）

特别注意：列地址选通延迟（ CL），仅在读命令的时候有效果，在写命令的时候，并不需要这个延迟。

Write Mode，即写模式，用于设置单次写的模式，可以选择突发写入或者单次写入。

（5） 初始化

SDRAM 上电后，必须进行初始化，才可以正常使用。SDRAM 初始化时序图如图 26.1.1.9 所示：

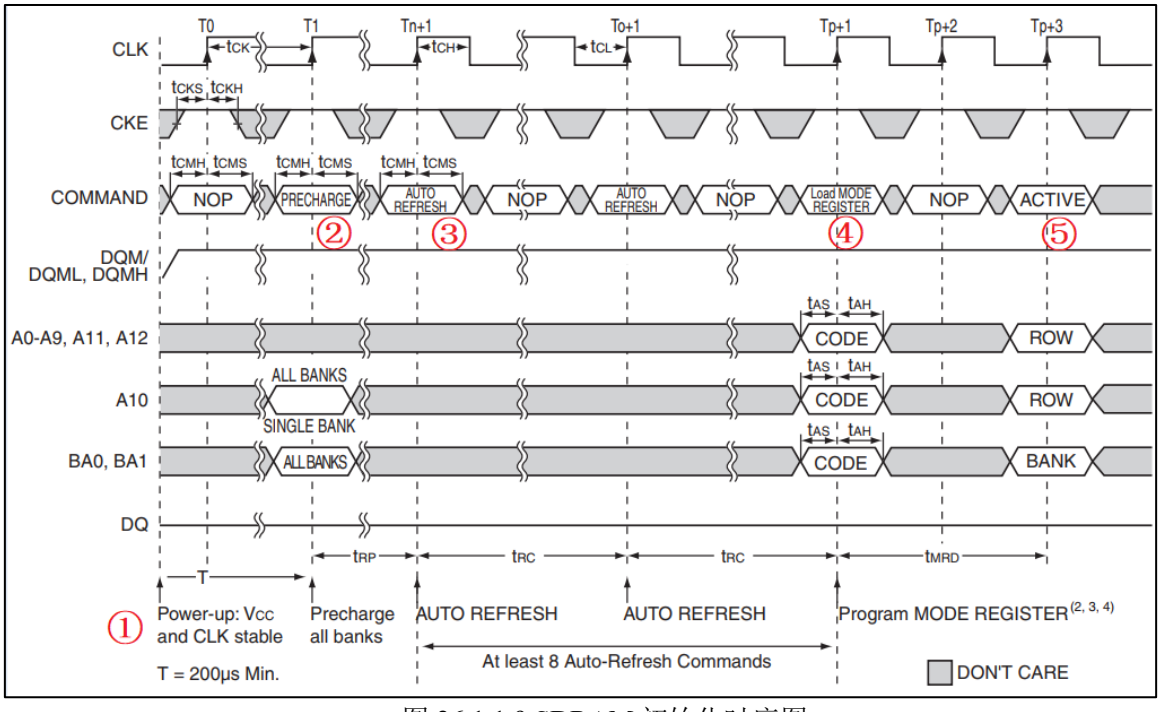


图 26.1.1.9 SDRAM 初始化时序图

初始化过程分为五步：

①上电

此步，给 SDRAM 供电，使能 CLK 时钟，并发送 NOP（No Operation 命令），注意，上电后，要等待最少 200us，再发送其他指令。

②发送预充电命令

第二步，就是发送预充电命令，给所有 Bank 预充电。

③发送自动刷新命令

这一步，至少要发送 8 次自刷新命令，每一个自刷新命令之间的间隔时间为 tRC。

④设置模式寄存器

这一步，发送模式寄存器的值，配置 SDRAM 的工作参数。配置完成后，需要等待 tMRD(也叫 tRSC)，使模式寄存器的配置生效，才能发送其他命令。

⑤完成

经过前面四步的操作，SDRAM 的初始化就完成了，接下来就可以发送激活命令和读/写命令，进行数据的读/写了。

这里提到的 tRC、tMRD 和 tRSC 见 SDRAM 的芯片数据手册。

（6） 写操作

在完成对 SDRAM 的初始化后，我们就可以对 SDRAM 进行读写操作了，首先，我们来看写操作，时序图如图 26.1.1.10 所示：

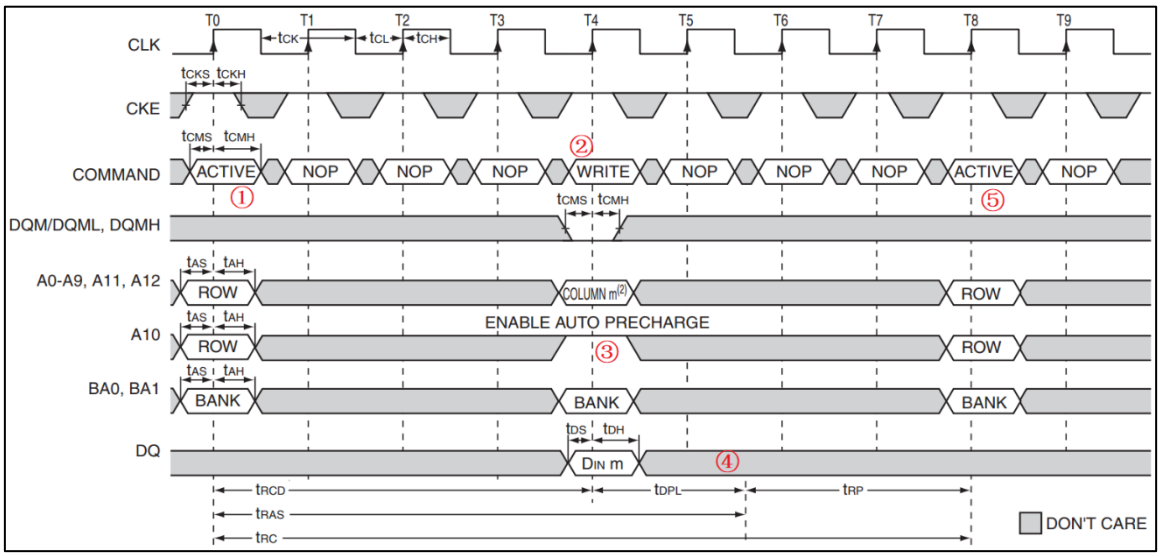


图 26.1.1.10 SDRAM 写时序图（自动预充电）

SDRAM 的写流程如下：

① 发送激活命令

此命令同时设置行地址和 BANK 地址，发送该命令后，需要等待 tRCD 时间，才可以发送写命令。

② 发送写命令

在发送完激活命令，并等待 tRCD 后，发送写命令，该命令同时设置列地址，完成对 SDRAM 的寻址。同时，将数据通过 DQ 数据线，存入 SDRAM。

③ 使能自动预充电

在发送写命令的同时，拉高 A10 地址线，使能自动预充电，以提高读写效率。

④ 执行预充电

预充电在发送激活命令的 tRAS 时间后启动，并且需要等待 tRP 时间，来完成。

⑤ 完成一次数据写入

最后，发送第二个激活命令，启动下一次数据传输。这样，就完成了一次数据的写入。

（7） 读操作

前面介绍了 SDRAM 的写操作，接下来我们看读操作时序，如图 26.1.1.11 所示：

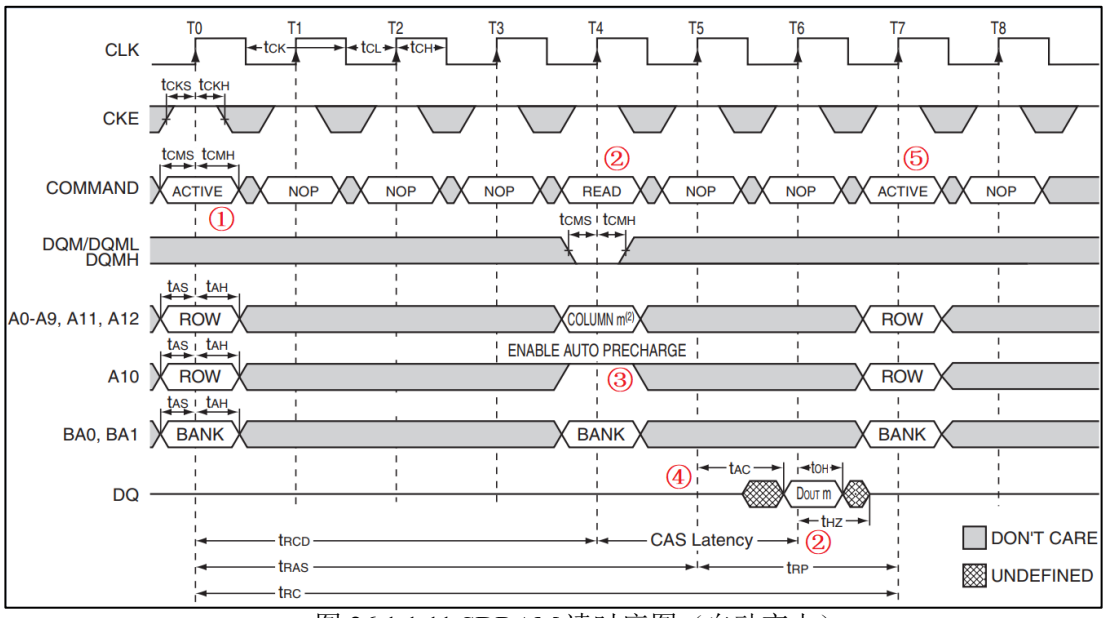


图 26.1.1.11 SDRAM 读时序图（自动充电）

SDRAM 的读流程如下：

① 发送激活命令

此命令同时设置行地址和 BANK 地址，发送该命令后，需要等待 tRCD 时间，才可以发送读命令。

② 发送写命令

在发送完激活命令，并等待 tRCD 后，发送读命令，该命令同时设置列地址，完成对 SDRAM 的寻址。读操作还有一个 CL 延迟（CAS Latency），所以需要等待给定的 CL 延迟（2 个或 3 个 CLK）后，再从 DQ 数据线上读取数据。

③ 使能自动预充电

在发送读命令的同时，拉高 A10 地址线，使能自动预充电，以提高读写效率。

④ 执行预充电

预充电在发送激活命令的 tRAS 时间后启动，并且需要等待 tRP 时间来完成。

⑤ 完成一次数据写入

最后，发送第二个激活命令，启动下一次数据传输。这样就完成了一次数据的读取。

SDRAM 的简介，就给大家介绍到这里，以上，tRCD、tRAS 和 tRP 等时间参数，见 SDRAM 的数据手册，且在后续配置 FMC 的时候需要用到。

26.1.2 FMC SDRAM 接口简介

在上一章，我们对 STM32H743 的 FMC 接口进行了讲解，并利用 FMC 接口来驱动 MCU 屏，本章，我们将介绍如何利用 FMC 接口驱动 SDRAM。STM32H743FMC 接口的 SDRAM 控制器，具有如下特点：

- 两个 SDRAM 存储区域，可独立配置
- 支持 8 位、16 位和 32 位数据总线宽度
- 支持 13 位行地址，11 位列地址，4 个内部存储区域：4*16M*32bit(256MB)、4*16M*16bit（128MB）、4*16M*8bit（64MB）
- 支持字、半字和字节访问
- 自动进行行和存储区域边界管理
- 多存储区域乒乓访问
- 可编程时序参数
- 支持自动刷新操作，可编程刷新速率
- 自刷新模式
- 读 FIFO 可缓存，支持 6 行*32 位深度（6*14 位地址标记）

通过 26.1.1 小节的介绍，我们对 SDRAM 已经有了一个比较深入的了解，包括接线、命令、初始化流程和读写流程，接下来，我们介绍一些配置 FMC SDRAM 控制器需要用到的几个寄存器。

首先，我们介绍 SDRAM 的控制寄存器：FMC_SDCRx，x=1/2，该寄存器各位描述如图 26.1.2.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15
Res.	RPIPE[1:0]		RBURST	SDCLK[1:0]		WP	CAS[1:0]		NB	MWID[1:0]		NR[1:0]		NC51:0]		Res.
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	

图 26.1.2.1 FMC_SDCRx 寄存器各位描述

该寄存器只有低 15 位有效，且都需要进行配置：

NC：这两个位定义列地址的位数（00~11，表示 8~11 位），W9825G6KH 有 9 位列地址，所以，这里应该设置为 01。

NR：这两个位定义行地址的位数（00~10，表示 11~13 位），W9825G6KH 有 13 位行地址，所以，这里设置为 10。

MWID：这两个位定义存储器数据总线宽度（ 00~10，表示 8~32 位），W9825G6KH 数据位宽为 16 位，所以这里设置为 01。

NB：该位用于设置 SDRAM 内部存储区域（BANK）数量（0=2 个，1=4 个）， W9825G6KH 内部有 4 个 BANK，所以，这里设置为：1。

CAS：这两个位可设置 SDRAM 的 CAS 延迟， 按存储器时钟周期计（ 01~11，表示 1~3 个）。W9825G6KH 可以设置为 2，也可以设置为 3，我们设置为 11。

WP：该位用于写保护设置（0=写使能，1=写保护），我们需要用到写操作，所以这里设置为 1 即可。

SDCLK：这两个位用于配置 SDRAM 的时钟（10=pll2_r_ck/2，11=pll2_r_ck/3），需要在禁止 SDRAM 时钟的前提下配置。 W9825G6KH 最快可以到 200M（@CL=3），为了较快的速度，我们设置为 10。

RBURST：此位用于使能突发读模式（0=禁止，1=使能）。这里我们设置为 1，使能突发读。

RPIPE： 这两个位可定义在 CAS 延迟后延后多少个 pll2_r_ck 时钟周期读取数据（00~10，表示 0~2 个）。这里，我们设置为 00 即可。

接下来，我们介绍 SDRAM 的时序寄存器： FMC_SDTRx，x=1/2，该寄存器各位描述如图 26.1.2.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	TRCD				TRP				TWR			
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TRC				TRAS				TXSR				TMRD			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 26.1.2.2 FMC_SDTRx 寄存器各位描述

该寄存器用于控制 SDRAM 的时序，非常重要，接下来我们分别介绍各个参数：

TMRD：这四个位定义加载模式寄存器命令和激活或刷新命令之间的延迟，这个参数就是 SDRAM 数据手册里面的 tMRD 或 tRSC 参数，W9825G6KH 的 tRSC 值为 2 个时钟，所以我们设置为 1 即可（2 个时钟周期，这里的时钟周期是指 SDRAM 的时钟周期，下同）。

TXSR：这四个位定义从发出自刷新命令到发出激活命令之间的延迟。W9825G6KH 的这个时间为 72ns，我们设置一个 SDRAM 的时钟频率为 110M，一个周期为 9.1ns，设置 TXSR 为

7，即 8 个时钟周期即可。

TRAS：这四个位用于设置自刷新周期。W9825G6KH 的自刷新周期为 60ns，我们设置 TRAS 为 6，即 7 个时钟周期即可。

TRC：这四个位定义刷新命令和激活命令之间的延迟，以及两个相邻刷新命令之间的延迟。W9825G6KH 的这个时间同样是 60ns，我们设置 TRC 为 6，即 7 个时钟周期即可。

TWR： 这四个位定义写命令和预充电命令之间的延迟。W9825G6KH 的这个时间为 2 个时钟周期，所以，我们设置 TWR=1 即可。

TRP： 这四个位定义预充电命令与其它命令之间的延迟。W9825G6KH 的这个时间为 15ns，所以，我们设置 TRP=1 ，即 2 个时钟周期（18.2ns）。

TRCD：这四个位定义激活命令与读/写命令之间的延迟。W9825G6KH 的这个时间为 15ns，所以，我们设置 TRP=1，即 2 个时钟周期（18.2ns）。

接下来，我们介绍 SDRAM 的命令模式寄存器：FMC_SDCMR，该寄存器各位描述如图 26.1.2.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MRD						
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MRD							NRFS				CTB1	CTB2	MODE		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 26.1.2.3 FMC_SDCMR 寄存器各位描述

该寄存器用于发送控制 SDRAM 的命令，以及 SDRAM 控制器的工作模式时序，非常重要。接下来我们分别介绍各个参数：

MODE： 这三个位定义发送到 SDRAM 存储器的命令。000：正常模式；**001：时钟配置使能；010：预充电所有存储区；011：自刷新命令；100：配置模式寄存器；**101：自刷新命令；110：掉电命令； 111：保留。加粗部分的命令，我们配置的时候需要用到。

CTB2/CTB1：这两个位用于指定命令所发送的目标存储器，因为 SDRAM 控制器可以外挂 2 个 SDRAM，发送命令的时候，需要通过 CTB1/CTB2 指定命令发送给哪个存储器。我们使用的是第一个存储器（SDNE0），所以设置 CTB1 即可。

NRFS：这四个位定义在 MODE=011 时，所发出的连续自刷新命令的个数。 0000~1110，表示 1~15 个自刷新命令。W9825G6KH 在初始化的时候，至少需要连续发送 8 个自刷新命令。

MRD：这十三个位，定义 SDRAM 模式寄存器的内容（通过地址线发送），在 MODE=100 的时候，需要配置。

接下来，我们介绍 SDRAM 的刷新定时器寄存器：FMC_SDRTR，该寄存器各位描述如图 26.1.2.4 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	REIE	COUNT													CRE
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	w

图 26.1.2.4 FMC_SDRTR 寄存器各位描述

该寄存器通过配置刷新定时器计数值来设置刷新循环之间的刷新速率，按 SDRAM 的时钟周期计数。计算公式为：

刷新速率 =（COUNT + 1）* SDRAM 频率时钟

COUNT =（SDRAM 刷新周期/行数）- 20

我们以 W9825G6KH 为例讲解计算过程，W9825G6KH 的刷新周期为 64ms，行数为 8192 行，所以刷新频率为：

刷新频率 = 64ms / 8192 = 7.81us

而 SDRAM 时钟频率=220Mhz/2=110Mhz(9.09ns)，所以 COUNT 的值为：

COUNT = 7.81us / 9.09ns ≈ 859

而如果 SDRAM 在接受读请求后，出现内部刷新请求，则必须将刷新速率增加 20 个 SDRAM 时钟周期，以获得充足的余量，所以，实际设计的 COUNT 值应该是：COUNT-20=839。所以，我们设置 FMC_SDRTR 的 COUNT=839，就可以完成对该寄存器的配置。

至此，FMC SDRAM 部分的寄存器就介绍完了，关于 FMC SDRAM 的控制器的详细介绍，请大家参考《STM32H7xx 参考手册_V3（中文版）》第 22.9 节。通过以上两个小节的了解，我们可以开始写 SDRAM 的驱动代码了。不过，MDK 并没有将寄存器定义成 FMC_SDCR1/2 的形式，而是定义成： FMC_SDCR[0]/[1]，对应的就是 FMC_SDCR1/2，其他几个寄存器类似，使用的时候注意一下。

阿波罗 STM32H743 核心板板载的 W9825G6KH 芯片挂在 FMC SDRAM 的控制器 1 上面(SDNE0)，其原理图如图 26.1.2.5 所示：

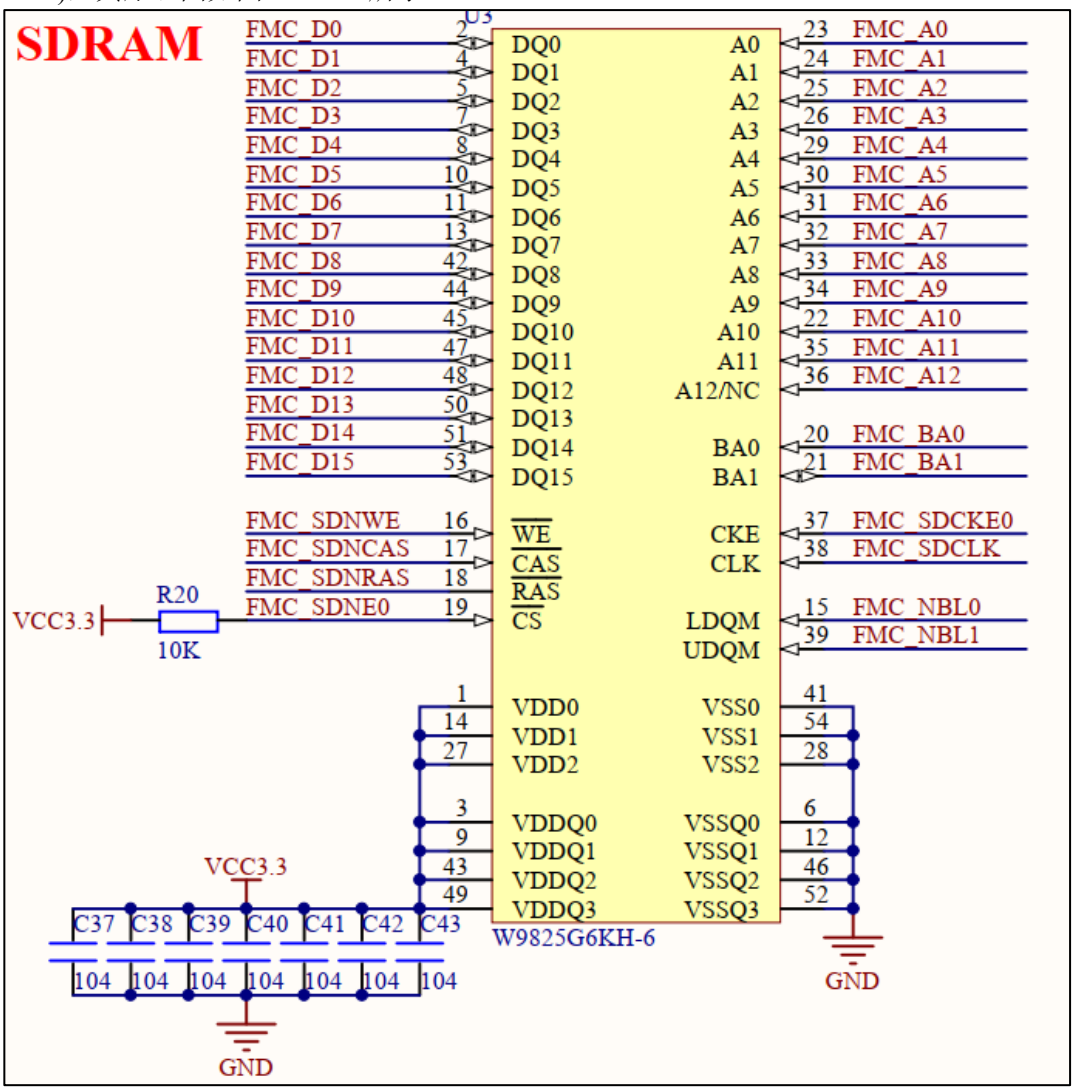


图 26.1.2.5 W9825G6KH 原理图

从原理图可以看出，W9825G6KH 同 STM32H743 的连接关系：

A[0:12]接 FMC_A[0:12]
BA[0:1]接 FMC_BA[0:1]
D[0:15]接 FMC_D[0:15]
CKE 接 FMC_SDCKE0
CLK 接 FMC_SDCLK
UDQM 接 FMC_NBL1
LDQM 接 FMC_NBL0
WE 接 FMC_SDNWE

CAS 接 FMC_SDNCAS
RAS 接 FMC_SDNRAS
CS 接 FMC_SDNE0

最后，我们来看看使用 HAL 库实现对 W9825G6KH 的驱动，需要对 FMC 进行哪些配置。对于 SDRAM 配置，我们要新引入的 HAL 库文件为 stm32f4xx_hal_sdram.c 和 stm32f4xx_hal_sdram.h。具体步骤如下：

1） 使能 FMC 时钟，并配置 FMC 相关的 IO 及其时钟使能。

要使用 FMC，当然首先得开启其时钟。然后需要把 FMC_D0~15， FMCA0~12 等相关 IO 口，全部配置为复用输出，并使能各 IO 组的时钟。使能时钟和初始化 IO 口方法前面已经多次讲解，这里就不累赘，请参考实验代码。

2） 初始化 SDRAM 控制参数和时间参数，也就是设置寄存器 FMC_SDCR1 和 FMC_SDTR1。

寄存器 FMC_SDCR1 用来设置 SDRAM 的相关控制参数，比如地址线宽度、 CAS 延迟、SDRAM 时钟等。 设置该寄存器的 HAL 库函数为 FMC_SDRAM_Init，声明如下：

```
HAL_StatusTypeDef FMC_SDRAM_Init(FMC_SDRAM_TypeDef *Device,
                                 FMC_SDRAM_InitTypeDef *Init);
```

寄存器 FMC_SDTR1 用来设置 SDRAM 时间相关参数，比如自刷新时间、恢复延迟、预充电延迟等。设置该寄存器的 HAL 库函数为 FMC_SDRAM_Timing_Init 函数，声明如下：

```
HAL_StatusTypeDef FMC_SDRAM_Timing_Init(FMC_SDRAM_TypeDef *Device,
                                        FMC_SDRAM_TimingTypeDef *Timing,
                                        uint32_t Bank);
```

实际上，HAL 库还提供了共同设置 SDRAM 控制参数和时间参数函数 HAL_SDRAM_Init，该函数会在内部会依次调用函数 FMC_SDRAM_Init 和 FMC_SDRAM_Timing_Init 进行 SDRAM 控制参数初始化和时间参数初始化，所以这里我们着重讲解函数 FMC_SDRAM_Init，声明如下：

```
HAL_StatusTypeDef  FMC_SDRAM_Init(FMC_SDRAM_TypeDef *Device,
                                  FMC_SDRAM_InitTypeDef *Init);
```

在讲解该函数之前首先我们要说明一点，和其他外设初始化一样，HAL 库同样提供了 SDRAM 的 MSP 初始化回调函数,函数为 HAL_SDRAM_MspInit ，该函数声明为：

```
void HAL_SDRAM_MspInit(SDRAM_HandleTypeDef *hsdram);
```

SDRAM 初始化函数内部会调用该回调函数，用来进行 MCU 相关初始化。对于 MSP 函数这里我们就不做过多讲解。接下来我们继续讲解 SDRAM 初始化函数 HAL_SDRAM_Init，该函数有两个入口参数，一个入口参数是 hsdram，该参数是 SDRAM_HandleTypeDef 结构体指针类型 ,用来设置 SDRAM 的控制参数 ， 另一个入口参数是 Timing ,该参数是 FMC_SDRAM_TimingTypeDef结构体指针类型，用来设置 SDRAM 的时间相关参数。我们先看看结构体 SDRAM_HandleTypeDef，定义如下。

```
typedef struct
{
  FMC_SDRAM_TypeDef              *Instance;
  FMC_SDRAM_InitTypeDef          Init;
  __IO HAL_SDRAM_StateTypeDef    State;
  HAL_LockTypeDef                Lock;
  MDMA_HandleTypeDef             *hdma;
} SDRAM_HandleTypeDef;
```

该结构体有 5 个成员变量，第一个成员变量用来设置 BANK 寄存器基地址，这个我们根据其入口参数有效范围即可找到，这里我们设置为 FMC_SDRAM_DEVICE 即可。第三个和第四个成员变量是 HAL 库使用的一些状态标识参数，最后一个成员变量 hdma 是与 DMA 相关，这里暂不讲解。接下来我们重点看看第二个成员变量 Init，它是真正的初始化结构体类型变量，结构体 FMC_SDRAM_InitTypeDef 定义如下：

```
typedef struct
{
  uint32_t SDBank;
  uint32_t ColumnBitsNumber;
  uint32_t RowBitsNumber;
  uint32_t MemoryDataWidth;
```

```
  uint32_t InternalBankNumber;
  uint32_t CASLatency;
  uint32_t WriteProtection;
  uint32_t SDClockPeriod;
  uint32_t ReadBurst;
  uint32_t ReadPipeDelay;
} FMC_SDRAM_InitTypeDef;
```

成员变量 SDBank 用来设置是使用的 SDRAM 的第几个 BANK，前面说过 SDRAM 有两个独立的 BANK，取值为 FMC_SDRAM_BANK1 或者 FMC_SDRAM_BANK2，我们使用的是 SDRAM 的 BANK1，所以设置为 FMC_SDRAM_BANK1 即可。

其他成员变量都是用来配置 FMC_SDCRx 控制寄存器的相应位的值。

ColumnBitsNumber 用来设置列地址数量，也就是 FMC_SDCRx 寄存器的 NC 位。

RowBitsNumber 用来设置行地址数量，也就 FMC_SDCRx 寄存器的 NR 位。

MemoryDataWidth 用来设置存储器数据总线宽度，也就是 FMC_SDCRx 的 MWID 位。

InternalBankNumber 用来设置 SDRAM 内部存储区域（BANK）数量，也就是 FMC_SDCRx 的 NB 位。

CASLatency 用来设置 SDRAM 的 CAS 延迟。也就是 FMC_SDCRx 的 CAS 位。

WriteProtection 用来设置写保护，也就是 FMC_SDCRx 的 WP。

SDClockPeriod 用来设置 SDRAM 的时钟，也就是 FMC_SDCRx 的 SDCLK 位。

ReadBurst 用来设置使能突发读模式，也就是 FMC_SDCRx 的 RBURST 位。

ReadPipeDelay 用来设置在 CAS 延迟后延后多少个 HCLK 时钟周期读取数据，也就是 FMC_SDCRx 的 RPIPE 位。

讲解完 HAL_SDRAM_Init 函数的第一个入口参数 hsdram，接下来我们看看第二个入口参数 Timing，它是 FMC_SDRAM_TimingTypeDef 结构体指针类型，该结构体主要用来设置寄存器 FMC_SDTRx 的值。该结构体定义如下：

```
typedef struct
{
  uint32_t LoadToActiveDelay;
  uint32_t ExitSelfRefreshDelay;
  uint32_t SelfRefreshTime;
  uint32_t RowCycleDelay;
  uint32_t WriteRecoveryTime;
  uint32_t RPDelay;
  uint32_t RCDDelay;
} FMC_SDRAM_TimingTypeDef;
```

该结构体一共有 7 个成员变量，这些成员变量都是时间参数，每个参数与寄存器的四个位对应，取值范围均为 1-16。

成员变量 LoadToActiveDelay 用来设置加载模式寄存器命令和激活或刷新命令之间的延迟，对应寄存器 FMC_SDTRx 的 TMRD 位。

ExitSelfRefreshDelay 用来设置从发出自刷新命令到发出激活命令之间的延迟，对应 TXSR 位。

SelfRefreshTime 用来设置自刷新周期，对应 TRAS 位。

RowCycleDelay 用来设置刷新命令和激活命令之间的延迟以及两个相邻刷新命令之间的延迟，对应 TRC 位。

WriteRecoveryTime 用来设置写命令和预充电命令之间的延迟，对应 TWR 位。

RPDelay 用来设置预充电命令与其它命令之间的延迟，对应位 TRP。

RCDDelay 用来设置激活命令与读/写命令之间的延迟，对应位 TRCD。

函数 HAL_SDRAM_Init 的使用范例如下：

```
void sdram_init(void)
{
    FMC_SDRAM_TimingTypeDef sdram_timing_handle;

    g_sdram_handle.Instance = FMC_SDRAM_DEVICE;     /* SDRAM 在 BANK5,6 */
    g_sdram_handle.Init.SDBank = FMC_SDRAM_BANK1;   /* 第一个 SDRAM BANK */
    g_sdram_handle.Init.ColumnBitsNumber=FMC_SDRAM_COLUMN_BITS_NUM_9;/*列数量 */
```

```
    g_sdram_handle.Init.RowBitsNumber = FMC_SDRAM_ROW_BITS_NUM_13;/* 行数量 */
    /* 数据宽度为 16 位 */
    g_sdram_handle.Init.MemoryDataWidth = FMC_SDRAM_MEM_BUS_WIDTH_16;

    /* 一共 4 个 BANK */
    g_sdram_handle.Init.InternalBankNumber = FMC_SDRAM_INTERN_BANKS_NUM_4;
    g_sdram_handle.Init.CASLatency = FMC_SDRAM_CAS_LATENCY_3;  /* CAS 为 3 */
    /* 失能写保护 */
    g_sdram_handle.Init.WriteProtection = FMC_SDRAM_WRITE_PROTECTION_DISABLE;
    /* SDRAM 时钟=pll2_r_ck/2=220M/2=110M=9.1ns */
    g_sdram_handle.Init.SDClockPeriod = FMC_SDRAM_CLOCK_PERIOD_2;
    g_sdram_handle.Init.ReadBurst = FMC_SDRAM_RBURST_ENABLE;      /* 使能突发 */
    g_sdram_handle.Init.ReadPipeDelay = FMC_SDRAM_RPIPE_DELAY_1; /* 读通道延时 */

    /* 加载模式寄存器到激活时间的延迟为 2 个时钟周期 */
    sdram_timing_handle.LoadToActiveDelay = 2;
    sdram_timing_handle.ExitSelfRefreshDelay = 8;/* 退出自刷新延迟为 8 个时钟周期 */
    sdram_timing_handle.SelfRefreshTime = 7;      /* 自刷新时间为 7 个时钟周期 *
    sdram_timing_handle.RowCycleDelay = 7;        /* 行循环延迟为 7 个时钟周期 */
    sdram_timing_handle.WriteRecoveryTime = 2;    /* 恢复延迟为 2 个时钟周期 */
    sdram_timing_handle.RPDelay = 2;              /* 行预充电延迟为 2 个时钟周期 */
    sdram_timing_handle.RCDDelay = 2;             /* 行到列延迟为 2 个时钟周期 */
    HAL_SDRAM_Init(&g_sdram_handle, &sdram_timing_handle);
    sdram_initialization_sequence(&g_sdram_handle);/* 发送 SDRAM 初始化序列 */
    /**
     * 刷新频率计数器(以 SDCLK 频率计数),计算方法:
     * COUNT=SDRAM 刷新周期/行数-20=SDRAM 刷新周期(us)*SDCLK 频率(Mhz)/行数
     * 我们使用的 SDRAM 刷新周期为 64ms,SDCLK=220/2=110Mhz,行数为 8192(2^13).
     * 所以,COUNT=64*1000*110/8192-20=839
     */
    HAL_SDRAM_ProgramRefreshRate(&g_sdram_handle, 839);      /* 设置刷新频率 */
}
```

3） 发送 SDRAM 初始化序列。

这里根据前面提到的 SDRAM 初始化步骤，对 SDRAM 进行初始化，首先使能时钟配置，然后等待至少 200us，对所有 BANK 进行预充电，执行自刷新命令等，最后配置模式寄存器。完成对 SDRAM 的初始化。 发送初始化系列主要是向 SRAM 存储区发送命令， HAL 库提供了发送命令函数为：

```
HAL_StatusTypeDef HAL_SDRAM_SendCommand(SDRAM_HandleTypeDef *hsdram,
                        FMC_SDRAM_CommandTypeDef *Command, uint32_t Timeout);
```

该函数的第一个入口参数 hsdram 是 SDRAM 句柄，第三个参数是发送命令 Timeout 时间，这两个参数都比较好理解，接下来我们着重讲解第二个入口参数 Command,该参数是 FMC_SDRAM_CommandTypeDef 结构体指针类型，该结构体定义如下：

```
typedef struct
{
  uint32_t CommandMode;
  uint32_t CommandTarget;
  uint32_t AutoRefreshNumber;
  uint32_t ModeRegisterDefinition;
} FMC_SDRAM_CommandTypeDef;
```

成员变量 CommandMode 用来设置命令类型，我们前面讲解过，一共有 7 种命令类型，包括时钟配置使能命令 FMC_SDRAM_CMD_CLK_ENABLE， 自刷新命令 FMC_SDRAM_CMD_AUTOREFRESH_MODE 等，这里我们就不一一讲解。

CommandTarget 用来设置目标 SDRAM 存储区域，因为 SDRAM 控制器可以外挂 2 个 SDRAM，发送命令的时候，需要指定命令发送给哪个存储器取值范围为：FMC_SDRAM_CMD_TARGET_BANK1，FMC_SDRAM_CMD_TARGET_BANK2 和 FMC_SDRAM_CMD_TARGET_BANK1_2。

AutoRefreshNumber 用来设置自刷新次数；

ModeRegisterDefinition 用来设置 SDRAM 模式寄存器的内容。了解了向 SRAM 存储区发

送命令方法，那么发送 SDRAM 初始化序列也就是发送命令到 SRAM 存储区，这就变得非常简单了。后面我们会给大家列出发送 SDRAM 初始化序列方法。

4） 设置刷新频率，也就是设置寄存器 FMC_ SDRTR 参数。

HAL 库提供的设置刷新频率函数为：

```
HAL_StatusTypeDef HAL_SDRAM_ProgramRefreshRate(SDRAM_HandleTypeDef *hsdram,
                                               uint32_t RefreshRate);
```

该函数入口参数比较简单，这里我们不做过多讲解。

通过以上几个步骤，我们就完成了 FMC 的配置，可以访问 W9825G6KH 了。

26.2 硬件设计

1. 例程功能

本章实验功能简介：开机后，显示提示信息，然后按下 KEY0 按键，即测试外部 SDRAM 容量大小并显示在 LCD 上。按下 KEY1 按键，即显示预存在外部 SDRAM 的数据。DS0 指示程序运行状态。

2. 硬件资源

本实验用到的硬件资源有：

1） 指示灯 DS0
2） KEY0 和 KEY1 按键
3） 串口 1
4） 外部 SDRAM（W9825G6KH）

3. 原理图

我们已经介绍过（W9825G6KH 与 STM32H743 的各 IO 对应关系，请参考光盘原理图），接下来我们开始软件设计。

26.3 程序设计

26.3.1 程序流程图

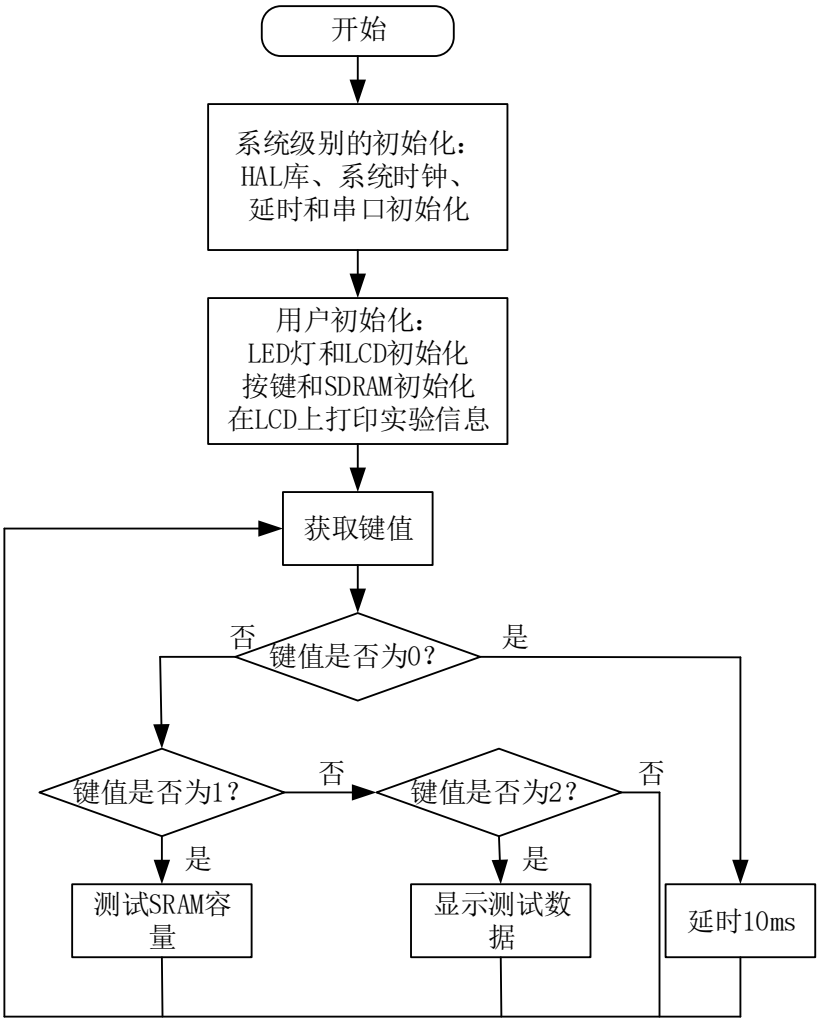


图 26.3.1.1 SDRAM 实验程序流程图

26.3.2 程序解析

1. SDRAM 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。SDRAM 驱动源码包括两个文件：sdram.c 和 sdram.h。

sdram.h 头文件只有函数声明，就不解释了。下面我们直接解析 sdram.c 的程序，该函数的实现代码如下：

```
/**
 * @brief       初始化 SDRAM
 * @param       无
 * @retval      无
 */
void sdram_init(void)
{
    FMC_SDRAM_TimingTypeDef sdram_timing;

    g_sdram_handle.Instance = FMC_SDRAM_DEVICE;         /* SDRAM 在 BANK5,6 */
    g_sdram_handle.Init.SDBank = FMC_SDRAM_BANK1;       /* 第一个 SDRAM BANK */
```

```
    g_sdram_handle.Init.ColumnBitsNumber=FMC_SDRAM_COLUMN_BITS_NUM_9;/* 列数量 */
    g_sdram_handle.Init.RowBitsNumber = FMC_SDRAM_ROW_BITS_NUM_13;    /* 行数量 */
    g_sdram_handle.Init.MemoryDataWidth = FMC_SDRAM_MEM_BUS_WIDTH_16;
    /* 数据宽度为 16 位 */
    g_sdram_handle.Init.InternalBankNumber = FMC_SDRAM_INTERN_BANKS_NUM_4;
    /* 一共 4 个 BANK */
    g_sdram_handle.Init.CASLatency = FMC_SDRAM_CAS_LATENCY_2;     /* CAS 为 2 */
    g_sdram_handle.Init.WriteProtection = FMC_SDRAM_WRITE_PROTECTION_DISABLE;
    /* 失能写保护 */
    g_sdram_handle.Init.SDClockPeriod = FMC_SDRAM_CLOCK_PERIOD_2;
    /* SDRAM 时钟=pll2_r_ck/2=220M/2=110M=9.1ns */
    g_sdram_handle.Init.ReadBurst = FMC_SDRAM_RBURST_ENABLE;      /* 使能突发 */
    g_sdram_handle.Init.ReadPipeDelay = FMC_SDRAM_RPIPE_DELAY_1; /* 读通道延时 */
    sdram_timing.LoadToActiveDelay=2;/* 加载模式寄存器到激活时间的延迟为 2 个时钟周期 */
    sdram_timing.ExitSelfRefreshDelay = 8; /* 退出自刷新延迟为 8 个时钟周期 */
    sdram_timing.SelfRefreshTime = 7;      /* 自刷新时间为 7 个时钟周期 */
    sdram_timing.RowCycleDelay = 7;        /* 行循环延迟为 7 个时钟周期 */
    sdram_timing.WriteRecoveryTime = 2;    /* 恢复延迟为 2 个时钟周期 */
    sdram_timing.RPDelay = 2;              /* 行预充电延迟为 2 个时钟周期 */
    sdram_timing.RCDDelay = 2;             /* 行到列延迟为 2 个时钟周期 */
    HAL_SDRAM_Init(&g_sdram_handle, &sdram_timing);
    sdram_initialization_sequence();       /* 发送 SDRAM 初始化序列 */
    /**
     * 刷新频率计数器(以 SDCLK 频率计数),计算方法:
     * COUNT=SDRAM 刷新周期/行数-20=SDRAM 刷新周期(us)*SDCLK 频率(Mhz)/行数
     * 我们使用的 SDRAM 刷新周期为 64ms,SDCLK=220/2=110Mhz,行数为 8192(2^13).
     * 所以,COUNT=64*1000*110/8192-20=839
     */
    HAL_SDRAM_ProgramRefreshRate(&g_sdram_handle, 839);       /* 设置刷新频率 */
}
/**
 * @brief       发送 SDRAM 初始化序列
 * @param       无
 * @retval      无
 */
void sdram_initialization_sequence(void)
{
    uint32_t temp = 0;
    /* SDRAM 控制器初始化完成以后还需要按照如下顺序初始化 SDRAM */
    sdram_send_cmd(0, FMC_SDRAM_CMD_CLK_ENABLE, 1, 0); /* 时钟配置使能 */
    delay_us(500);                                     /* 至少延时 200us */
    sdram_send_cmd(0, FMC_SDRAM_CMD_PALL, 1, 0);       /* 对所有存储区预充电 */
    sdram_send_cmd(0, FMC_SDRAM_CMD_AUTOREFRESH_MODE, 8, 0);  /* 设置自刷新次数 */
    /* 配置模式寄存器,SDRAM 的 bit0~bit2 为指定突发访问的长度,
     * bit3 为指定突发访问的类型,bit4~bit6 为 CAS 值,bit7 和 bit8 为运行模式
     * bit9 为指定的写突发模式,bit10 和 bit11 位保留位 */
    temp = (uint32_t)SDRAM_MODEREG_BURST_LENGTH_1  |
    /* 设置突发长度:1(可以是 1/2/4/8) */
    SDRAM_MODEREG_BURST_TYPE_SEQUENTIAL | /* 设置突发类型:连续(可以是连续/交错) */
    SDRAM_MODEREG_CAS_LATENCY_2         | /* 设置 CAS 值:2(可以是 2/3) */
    SDRAM_MODEREG_OPERATING_MODE_STANDARD|/* 设置操作模式:0,标准模式 */
    SDRAM_MODEREG_WRITEBURST_MODE_SINGLE; /* 设置突发写模式:1,单点访问 */
    sdram_send_cmd(0, FMC_SDRAM_CMD_LOAD_MODE,1,temp);/* 设置 SDRAM 的模式寄存器 */
}

/**
 * @brief       SDRAM 底层驱动,引脚配置,时钟使能
 * @note        此函数会被 HAL_SDRAM_Init()调用
 * @param       hsdram:SDRAM 句柄
 * @retval      无
```

```
 */
void HAL_SDRAM_MspInit(SDRAM_HandleTypeDef *hsdram)
{
    GPIO_InitTypeDef gpio_init_struct;

    __HAL_RCC_FMC_CLK_ENABLE();                 /* 使能 FMC 时钟 */
    __HAL_RCC_GPIOC_CLK_ENABLE();               /* 使能 GPIOC 时钟 */
    __HAL_RCC_GPIOD_CLK_ENABLE();               /* 使能 GPIOD 时钟 */
    __HAL_RCC_GPIOE_CLK_ENABLE();               /* 使能 GPIOE 时钟 */
    __HAL_RCC_GPIOF_CLK_ENABLE();               /* 使能 GPIOF 时钟 */
    __HAL_RCC_GPIOG_CLK_ENABLE();               /* 使能 GPIOG 时钟 */

    gpio_init_struct.Pin = GPIO_PIN_0|GPIO_PIN_2|GPIO_PIN_3;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;    /* 推挽复用 */
    gpio_init_struct.Pull = GPIO_PULLUP;        /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_HIGH;   /* 高速 */
    gpio_init_struct.Alternate = GPIO_AF12_FMC; /* 复用为 FMC */
    HAL_GPIO_Init(GPIOC, &gpio_init_struct);    /* 初始化 PC0,2,3 */

    gpio_init_struct.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_8 | GPIO_PIN_9 |
    GPIO_PIN_10 | GPIO_PIN_14 | GPIO_PIN_15;
    HAL_GPIO_Init(GPIOD, &gpio_init_struct);     /* 初始化 PD0,1,8,9,10,14,15 */

    gpio_init_struct.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_7 | GPIO_PIN_8 |
    GPIO_PIN_9 | GPIO_PIN_10 | GPIO_PIN_11 | GPIO_PIN_12 | GPIO_PIN_13 |
    GPIO_PIN_14 | GPIO_PIN_15;
    HAL_GPIO_Init(GPIOE, &gpio_init_struct);
    /* 初始化 PE0,1,7,8,9,10,11,12,13,14,15 */

    gpio_init_struct.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3 |
    GPIO_PIN_4 | GPIO_PIN_5 | GPIO_PIN_11 | GPIO_PIN_12 | GPIO_PIN_13 |
    GPIO_PIN_14 | GPIO_PIN_15;
    HAL_GPIO_Init(GPIOF, &gpio_init_struct);
    /* 初始化 PF0,1,2,3,4,5,11,12,13,14,15 */

    gpio_init_struct.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_4 |
    GPIO_PIN_5 | GPIO_PIN_8 | GPIO_PIN_15;
    HAL_GPIO_Init(GPIOG, &gpio_init_struct);     /* 初始化 PG0,1,2,4,5,8,15 */
}

/**
 * @brief       向 SDRAM 发送命令
 * @param       bankx:0,向 BANK5 上面的 SDRAM 发送指令
 * @param             1,向 BANK6 上面的 SDRAM 发送指令
 * @param       cmd:指令(0,正常模式/1,时钟配置使能/2,预充电所有存储区/3,自动刷新/
                        4,加载模式寄存器/5,自刷新/6,掉电)
 * @param       refresh:自刷新次数
 * @param       返回值:0,正常;1,失败.
 * @retval      模式寄存器的定义
 */
uint8_t sdram_send_cmd(uint8_t bankx, uint8_t cmd,
                       uint8_t refresh, uint16_t regval)
{
    uint32_t target_bank = 0;
    FMC_SDRAM_CommandTypeDef command;

    if (bankx == 0)
    {
        target_bank = FMC_SDRAM_CMD_TARGET_BANK1;
    }
    else if (bankx == 1)
    {
        target_bank = FMC_SDRAM_CMD_TARGET_BANK2;
```

```
    }

    command.CommandMode = cmd;                  /* 命令 */
    command.CommandTarget = target_bank;        /* 目标 SDRAM 存储区域 */
    command.AutoRefreshNumber = refresh;        /* 自刷新次数 */
    command.ModeRegisterDefinition = regval;    /* 要写入模式寄存器的值 */

    if (HAL_SDRAM_SendCommand(&g_sdram_handle, &command, 0X1000) == HAL_OK)
    {/* 向 SDRAM 发送命令 */
        return 0;
    }
    else return 1;
}

/**
 * @brief       在指定地址(addr+Bank5_SDRAM_ADDR)开始,连续写入 n 个字节
 * @param       pbuf      : 字节指针
 * @param       writeaddr : 要写入的地址
 * @param       n         : 要写入的字节数
 * @retval      无
*/
void fmc_sdram_write_buffer(uint8_t *pbuf, uint32_t writeaddr, uint32_t n)
{
    for (;n != 0;n--)
    {
        *(volatile uint8_t*)(BANK5_SDRAM_ADDR + writeaddr) = *pbuf;
        writeaddr ++;
        pbuf++;
    }
}

/**
 * @brief       在指定地址((addr+Bank5_SDRAM_ADDR))开始,连续读出 n 个字节.
 * @param       pbuf      : 字节指针
 * @param       readaddr  : 要读出的起始地址
 * @param       n         : 要写入的字节数
 * @retval      无
*/
void fmc_sdram_read_buffer(uint8_t *pbuf, uint32_t readaddr, uint32_t n)
{
    for (;n != 0;n--)
    {
        *pbuf++ = *(volatile uint8_t*)(BANK5_SDRAM_ADDR + readaddr);
        readaddr ++;
    }
}
```

此部分代码包含 6 个函数，sdram_init 函数用于初始化 FMC/SDRAM 配置，发送 SDRAM 初始化序列和设置刷新时间等，完全就是根据我们前面所说的步骤来实现的；函数 HAL_SDRAM_MspInit 是 SDRAM 的 MSP 初始化回调函数，用来初始化 IO 口和使能时钟；函数 sdram_initialization_sequence 是单独的用来发送 SRAM 初始化序列函数，在初始化函数 sdram_init 内部有调用该函数。sdram_send_cmd 函数用于给 SDRAM 发送命令，在初始化的时候需要用到；fmc_sdram_writebuffer 和 fmc_sdram_readbuffer 这两个函数分别用于在外部 SDRAM 的指定地址写入和读取指定长度的数据（字节数），一般用不到。

这里需要注意的是：当位宽为 16 位的时候，HADDR 右移一位同地址对齐，但是 addr 我们这里却没有加 2，而是加 1，是因为我们这里用的数据位宽是 8 位，通过 FMC_NBL1 和 FMC_NBL0 来控制高低字节位，所以地址在这里是可以只加 1 的。

2. main.c 代码

在 main 函数之前，我们添加了 sdram_test 这个函数，代码如下：

```
/**
 * @brief       SDRAM 内存测试
 * @param       x,y:起点坐标
 * @retval      无
*/
void sdram_test(uint16_t x, uint16_t y)
{
    uint32_t i = 0;
    uint32_t temp = 0;
    uint32_t sval = 0;     /* 在地址 0 读到的数据 */
    lcd_show_string(x, y,180, y + 16, 16,"Ex Memory Test:    0KB ", RED);

    /* 每隔 16K 字节,写入一个数据,总共写入 2048 个数据,刚好是 32M 字节 */
    for (i = 0;i < 32 *1024 * 1024;i += 16 * 1024)
    {
        *(volatile uint32_t*)(BANK5_SDRAM_ADDR + i) = temp;
        temp++;
    }
    /* 依次读出之前写入的数据,进行校验 */
    for (i = 0;i < 32 * 1024 * 1024;i += 16 * 1024)
    {
        temp =*(volatile uint32_t*)(BANK5_SDRAM_ADDR + i);
        if (i == 0)
        sval = temp;

        else if (temp <= sval)break;    /* 后面读出的数据一定要比第一次读到的数据大 */
        /* 显示内存容量 */
        lcd_show_num(x + 15 * 8,y,(uint16_t)(temp - sval + 1) * 16, 5, 16, 0);
        /* 打印 SDRAM 容量 */
        printf("SDRAM Capacity:%dKB\r\n",(uint16_t)(temp-sval + 1) * 16);
    }
}
```

该函数用于测试外部 SRAM 的容量大小，并显示其容量。

main 函数代码如下：

```
int main(void)
{
    uint8_t key;
    uint8_t i = 0;
    uint32_t ts = 0;
    sys_cache_enable();
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟,400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 初始化 USART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化 KEY */
    sdram_init();                           /* 初始化 SDRAM */

    lcd_show_string(30, 50, 200, 16, 16,"STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16,"SDRAM TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16,"ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16,"KEY0:Test SDRAM", RED);
    lcd_show_string(30, 130, 200, 16, 16,"KEY1:Test DATA", RED);

    for(ts = 0;ts < 250000; ts++)
    {
        testsdram[ ts ]= ts;                /* 预存测试数据 */
    }
    while (1)
    {
        key = key_scan(0);                  /* 不支持连按 */
```

```
        if (key == KEY0_PRES)
        {
          sdram_test(30, 170);                  /* 测试 SDRAM 容量 */
        }
        else if(key == KEY1_PRES)               /* 打印预存测试数据 */
        {
            for(ts = 0;ts < 250000;ts++)
            {
                lcd_show_num(30, 190,testsdram[ts], 6, 16, 0); /* 显示测试数据 */
                printf("testsdram[%d]:%d\r\n", ts, testsdram[ts]);
            }

        else delay_ms(10);
        i++;
        if (i == 20)
        {
            i = 0;
        }
        LED0_TOGGLE();                          /* DS0 闪烁 */
    }
}
```

此段代码，我们定义了一个超大数组 testsdram，我们指定该数组定义在外部 sdram 起始地址（__attribute__((at(0XC0000000)))），该数组用来测试外部 SDRAM 数据的读写。注意该数组的定义方法，是我们推荐的使用外部 SDRAM 的方法。

另外，sdram_test 函数和 main 函数，我们都加入了 printf 输出结果，对于没有 MCU 屏模块的朋友来说，可以打开串口调试助手，观看实验结果，软件部分就给大家介绍到这里。

26.4 下载验证

在代码编译成功之后，我们通过下载代码到阿波罗 STM32 开发板上，得到如图 26.4.1 所示：

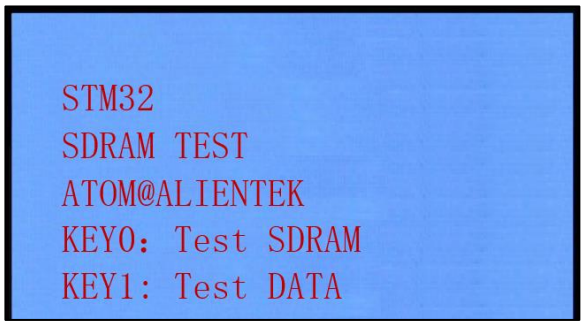


图 26.4.1 程序运行效果图

此时，我们按下 KEY0，就可以在 LCD 上看到内存测试的画面，同样，按下 KEY1，就可以看到 LCD 显示存放在数组 testsdram 里面的测试数据，如图 26.4.2 所示：

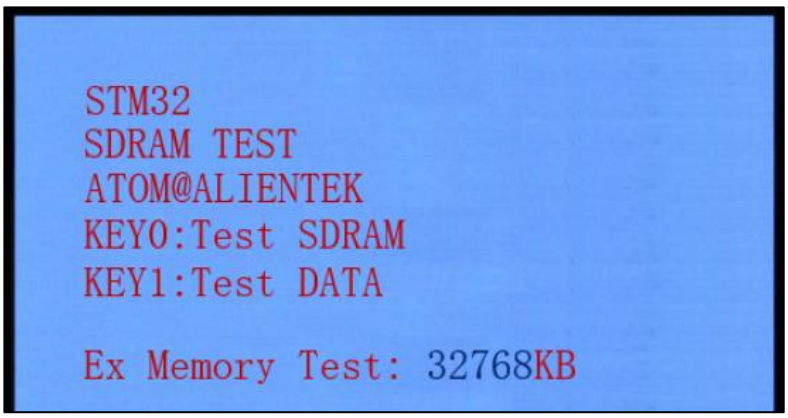


图 26.4.2 外部 SDRAM 测试界面

对于没有 MCU 屏模块的朋友，我们可以用串口来检查测试结果，如图 26.4.3 所示:

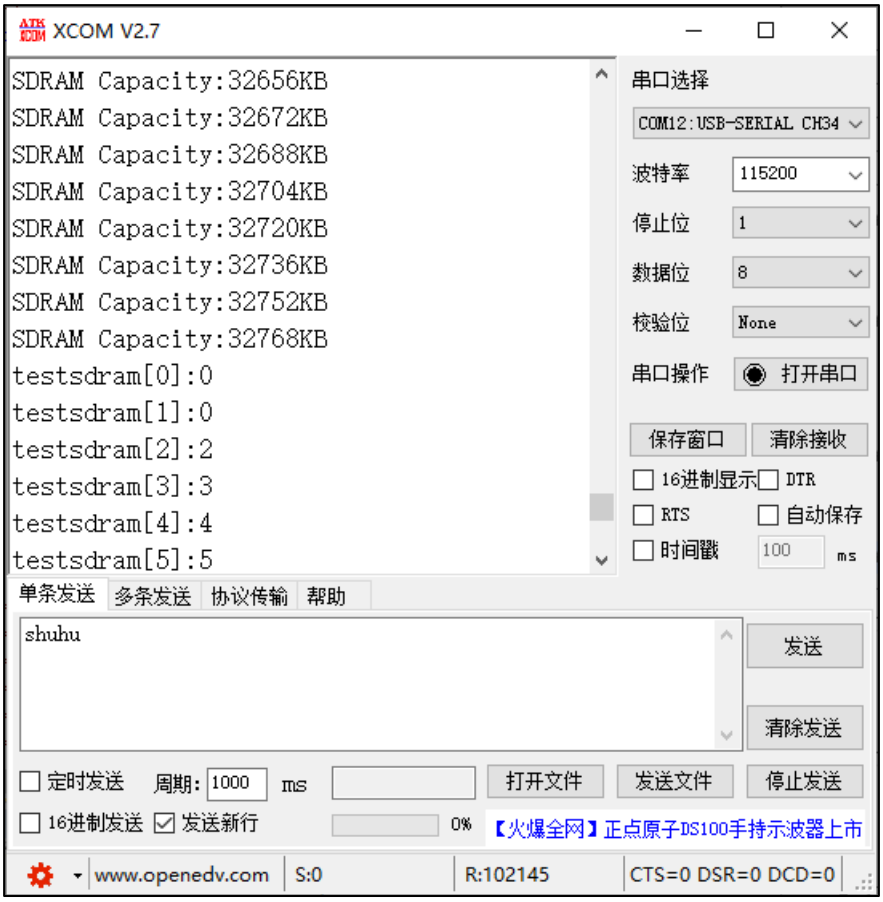


图 26.4.3 串口观看测试结果

第二十七章 LTDC LCD（RGB 屏）实验

在前面，我们介绍了 TFTLCD 模块（MCU 屏）的使用，但是高分辨率的屏幕（超过 800*480），一般都没有 MCU 屏接口，而是使用 RGB 接口的，这种接口的屏，就需要用到 STM32H743 的 LTDC 来驱动了。在本章中，我们将使用阿波罗 STM32H743 开发板核心板上的 LCD 接口（仅支持 RGB 屏，本章介绍 RGB 屏的使用），来点亮 LCD，并实现 ASCII 字符和彩色的显示等功能，并在串口打印 LCD ID，同时在 LCD 上面显示。

本章分为如下几个小节：

27.1 RGBLCD&LTDC 简介
27.2 硬件设计
27.3 程序设计
27.4 下载验证

27.1 RGBLCD&LTDC 简介

本章我们将通过 STM32H743 的 LTDC 接口来驱动 RGBLCD 的显示，另外，STM32H743 的 LTDC 还有 DMA2D 图形加速，我们也顺带进行介绍。本节分为三个部分，分别介绍 RGBLCD、LTDC 和 DMA2D。

27.1.1 RGBLCD 简介

在第 25 章，我们已经介绍过 TFTLCD 液晶了，实际上 RGBLCD 也是 TFTLCD，只是接口不同而已。接下来我们简单介绍一些 RGBLCD 的驱动。

（1） RGBLCD 的信号线

RGBLCD 的信号线如表 27.1.1.1 所示：

信号线	说明
R[0:7]	红色数据线，一般为 8 位
G[0:7]	绿色数据线，一般为 8 位
B[0:7]	蓝色数据线，一般为 8 位
DE	数据使能线
VS	垂直同步信号线
HS	水平同步信号线
DCLK	像素时钟信号线

表 27.1.1.1 RGBLCD 信号线

一般的 RGB 屏都有如表 27.1.1.1 所示的信号线，有 24 根颜色数据线（RGB 各 8 根，即 RGB888 格式），这样可以表示最多 1600W 色，DE、VS、HS 和 DCLK，用于控制数据传输。

（2） RGBLCD 的驱动模式

RGB 屏一般有 2 种驱动模式：DE 模式和 HV 模式。DE 模式使用 DE 信号来确定有效数据（DE 为高/低时，数据有效），而 HV 模式，则需要行同步和场同步，来表示扫描的行和列。

DE 模式和 HV 模式的行扫描时序图（以 800*480 的 LCD 面板为例），如图 27.1.1.1 所示：

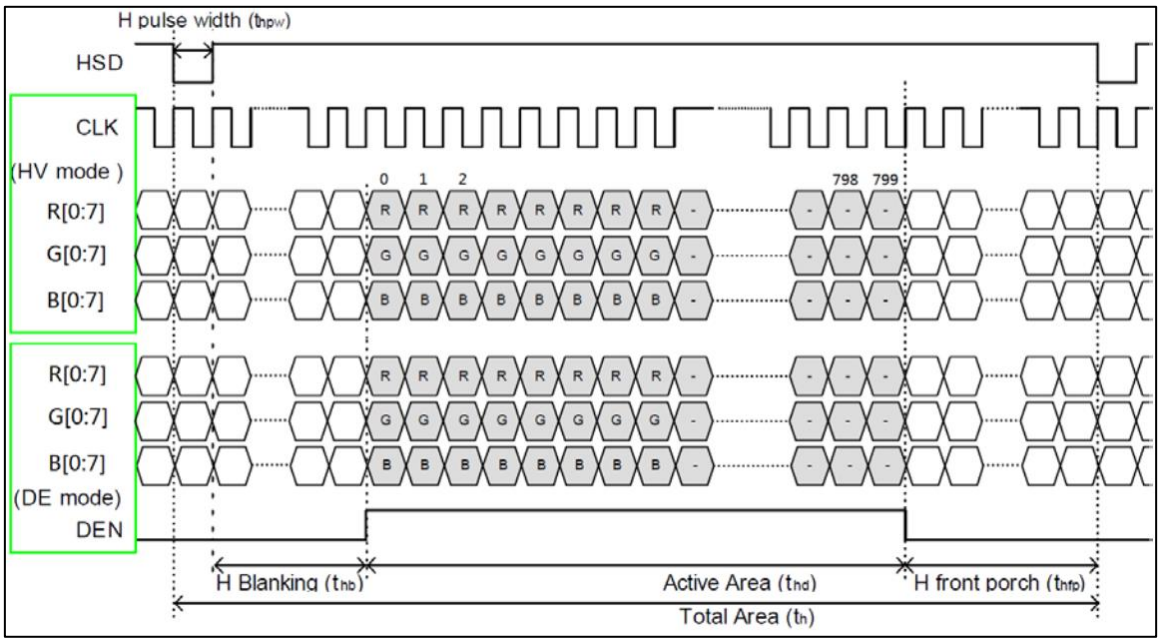


图 27.1.1.1 DE/HV 模式行扫描时序图

从图中可以看出，DE 和 HV 模式，时序基本一样，DEN 模式需要提供 DE 信号（DEN），而 HV 模式，则无需 DE 信号。图中的 HSD 即 HS 信号，用于行同步，注意：在 DE 模式下面，是可以不用 HS 信号的，即不接 HS 信号，液晶照样可以正常工作。

图中的 thpw 为水平同步有效信号脉宽，用于表示一行数据的开始；thb 为水平后廊，表示从水平有效信号开始，到有效数据输出之间的像素时钟个数；thfp 为水平前廊，表示一行数据结束后，到下一个水平同步信号开始之前的像素时钟个数；这几个时间非常重要，在配置 LTDC 的时候，需要根据 LCD 的数据手册，进行正确的设置。

图 27.1.1.1 仅是一行数据的扫描，输出 800 个像素点数据，而液晶面板总共有 480 行，这就还需要一个垂直扫描时序图，如图 27.1.1.2 所示：

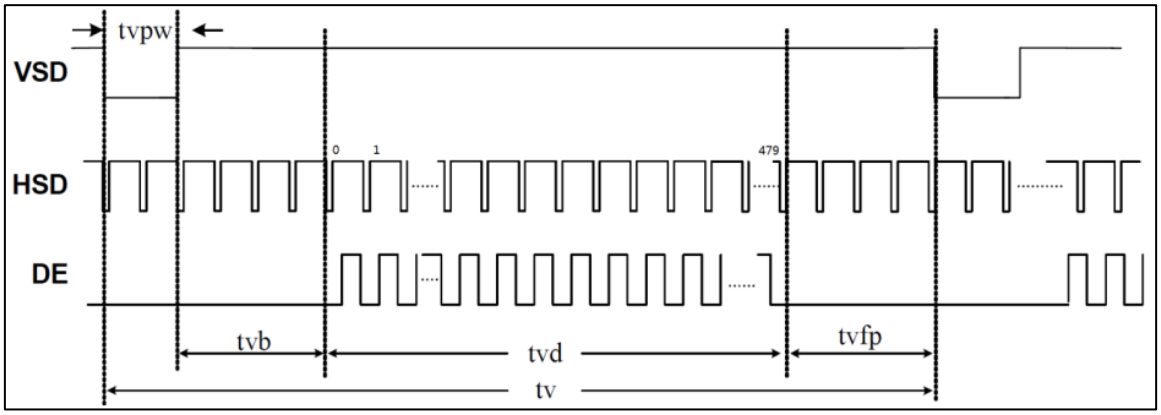


图 27.1.1.2 垂直扫描时序图

图中的 VSD 就是垂直同步信号，HSD 就是水平同步信号，DE 为数据使能信号。如图可知，一个垂直扫描，刚好就是 480 个有效的 DE 脉冲信号，每一个 DE 时钟周期，扫描一行，总共扫描 480 行，完成一帧数据的显示。这就是 800*480 的 LCD 面板扫描时序，其他分辨率的 LCD 面板，时序类似。

图中的 tvpw 为垂直同步有效信号脉宽，用于表示一帧数据的开始；tvb 为垂直后廊，表示垂直同步信号以后的无效行数，tvfp 为垂直前廊，表示一帧数据输出结束后，到下一个垂直同步信号开始之前的无效行数；这几个时间同样在配置 LTDC 的时候，需要进行设置。

（3） 正点原子 RGBLCD 模块

正点原子目前提供大家五款 RGBLCD 模块：ATK-4342（4.3 寸，480*272）、ATK-4384（4.3 寸，800*480） ATK-7084（7 寸，800*480）、ATK-7016（7 寸，1024*600）和 ATK-1018

（10.1 寸，1280*800），这里我们以 ATK-7084 为例，给大家介绍。该模块的接口原理图如图 27.1.1.3 所示：

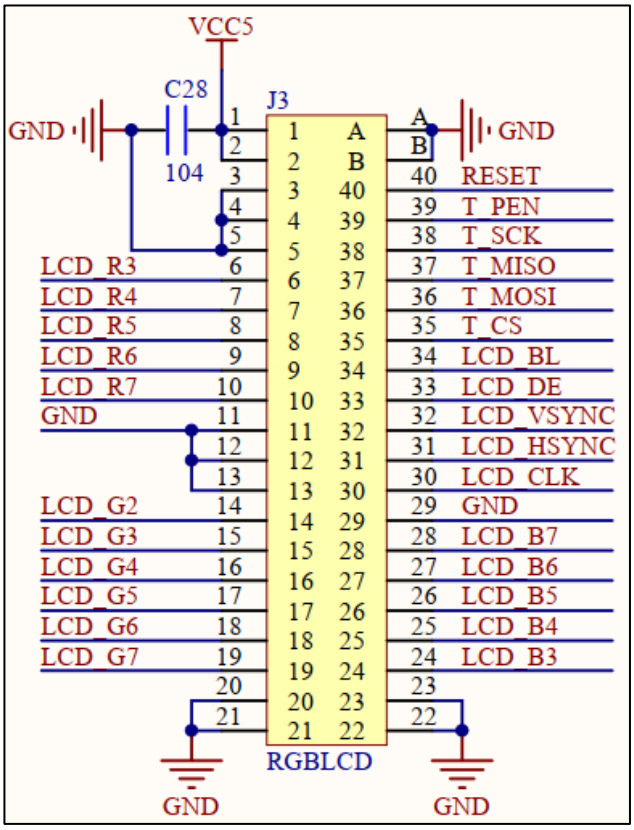


图 27.1.1.3 RGBLCD 模块对外接口原理图

图中 J3 就是对外接口，是一个 40PIN 的 FPC 座（0.5mm 间距），通过 FPC 线，可以连接到阿波罗 STM32H743 开发板的核心板上面，从而实现和 STM32H743 的连接。该接口十分完善，采用 RGB888 格式，并支持 DE&HV 模式，还支持触摸屏（电阻/电容）和背光控制。右侧的几个电阻，并不是都焊接的，而是可以用户自己选择。默认情况， R1 和 R6 焊接，设置 LCD_LR 和 LCD_UD，控制 LCD 的扫描方向，是从左到右，从上到下（横屏看）。而 LCD_R7/G7/B7 则用来设置 LCD 的 ID，由于 RGBLCD 没有读写寄存器，也就没有所谓的 ID，这里我们通过在模块上面，控制 R7/G7/B7 的上/下拉，来自定义 LCD 模块的 ID，帮助 MCU 判断当前 LCD 面板的分辨率和相关参数，以提高程序兼容性。这几个位的设置关系如表 27.1.1.2 所示：

M2(B7)	M1(G7)	M0(R7)	RGBLCD 模块参数
0	0	0	4.3 寸， 480*272 分辨率（RGB）
0	0	1	7 寸， 800*480 分辨率（RGB）
0	1	0	7 寸， 1024*600 分辨率（RGB）
0	1	1	7 寸， 1280*800 分辨率（RGB）
1	0	0	4.3 寸， 800*480 分辨率（RGB）
1	0	1	10.1 寸， 1280*800 分辨率（RGB）
X	X	X	暂时未用到

表 27.1.1.2 正点原子 RGBLCD 模块 ID 对应关系

这样我们在程序里面，读取 LCD_R7/G7/B7，得到 M0:M2 的值，从而判断 RGBLCD 模块的型号，并执行不同的配置，即可实现不同 LCD 模块的兼容。

RGBLCD 我们就给大家介绍到这里，接下来，我们介绍 LTDC。

27.1.2 LTDC 简介

STM32H743xx 系列芯片都带有 TFT LCD 控制器，即 LTDC，通过这个 LTDC，STM32H743 可以直接外接 RGBLCD 屏，实现液晶驱动。STM32H743 的 LTDC 具有如下特点：

- 24 位 RGB 并行像素输出：每像素 8 位数据（RGB888）
- 2 个带有专用 FIFO 的显示层（FIFO 深度 64x32 位）
- 支持查色表 (CLUT)，每层高达 256 种颜色（256x24 位）
- 可针对不同显示面板编辑时序
- 可编程背景色
- 可编程 HSync、VSync 和数据使能（DE）信号的极性
- 每层有多达 8 种颜色格式可供选择：ARG8888、RGB8888、RGB565、ARGB1555、ARGB4444、L8（8 位 Luminance 或 CLUT）、AL44（4 位 alpha+4 位 luminance）和 AL88（8 位 alpha+8 位 luminance）
- 每通道的低位采用伪随机抖动输出（红色、绿色、蓝色的抖动宽度为 2 位）
- 使用 alpha 值（每像素或常数）在两层之间灵活混合
- 色键（透明颜色）
- 可编程窗口位置和大小
- 支持薄膜晶体管（TFT）彩色显示器
- AHB 主接口支持 16 个字的突发
- 高达 4 个可编程中断事件

LTDC 控制器主要包含：信号线、图像处理单元、AHB 接口、配置和状态寄存器以及时钟部分，其框图如图 27.1.2.1 所示：

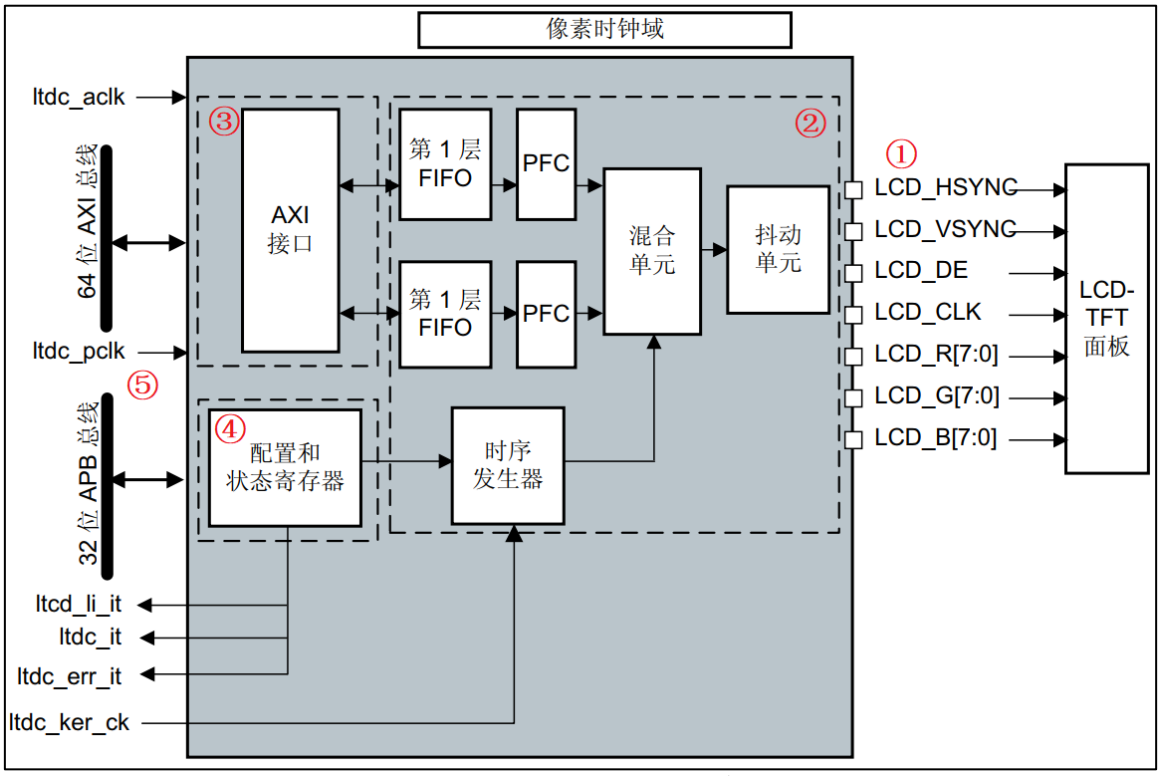


图 27.1.2.1 LTDC 控制器框图

① 信号线

这里就包含了我们前面提到的 RGBLCD 驱动所需要的所有信号线，这些信号线通过 STM32H743 核心板板载的 LCD 接口引出，其信号线说明和 IO 连接关系，见表 27.1.2.1 所示：

LTDC 信号线	对应 IO	说明
LCD_CLK	PG7	像素时钟输出

LCD_HSYNC	PI10	水平同步
LCD_VSYNC	PI9	垂直同步
LCD_DE	PF10	数据使能
LCD_R[7:3]	PG6、PH12、PH11、PH10、PH9	红色数据线，LCD_R[2:0]未用到
LCD_G[7:2]	PI2、PI1、PI0、PH15、PH14、PH13	绿色数据线，LCD_G[1:0]未用到
LCD_B[7:3]	PI7、PI6、PI5、PI4、PG11	蓝色数据线，LCD_B[2:0]未用到

表 27.1.2.1 LTDC 信号线及 IO 连接关系说明

LTDC 总共有 24 位数据线，支持 RGB888 格式，但是我们为了节省 IO，并提高图片显示速度，使用 RGB565 颜色格式，这样的话，只需 16 个 IO 口，当使用 RGB565 格式的时候，LCD 面板的数据线，必须连接到 LTDC 数据线的 MSB，即：LTDC 的 LCD_R[7:3]接 RGBLCD 的 R[7:3]，LTDC 的 LCD_G[7:2]接 RGBLCD 的 G[7:2]，LTDC 的 LCD_B[7:3]接 RGBLCD 的 B[7:3]，这样，RGB 数据线分别是 5:6:5，即 RGB565 格式。表中对应 IO 就是我们 STM32H743 核心板上面，LCD 接口所连接的 IO。

② 图像处理单元

此部分先从 AHB 接口获取显存中的图像数据，然后经过层 FIFO（有 2 个，对应 2 个层）缓存，每个层 FIFO 具有 64*32 位存储深度，然后经过像素格式转换器（PFC），把从层的所选输入像素格式转换为 ARGB8888 格式，再通过混合单元，把两层数据合并，混合得到单层要显示的数据，最后经过抖动单元处理（可选）后，输出给 LCD 显示。

这里的 ARGB8888，即带 8 位透明通道，即最高 8 位为透明通道参数，表示透明度，值越大，则约不透明，值越小，越透明。比如 A=255 时，表示完全不透明，而 A=0 时，表示完全透明。RGB888 就表示 R、G、B 各 8 位，可表示的颜色深度为 1600W 色。

STM32H743 的 LTDC 总共有三个层：背景层、第一层和第二层，其中，背景层只可以是纯色（即单色），而第一层和第二层都可以用来显示信息，混合单元会将三个层混合起来，进行显示，显示关系如图 27.1.2.2 所示：

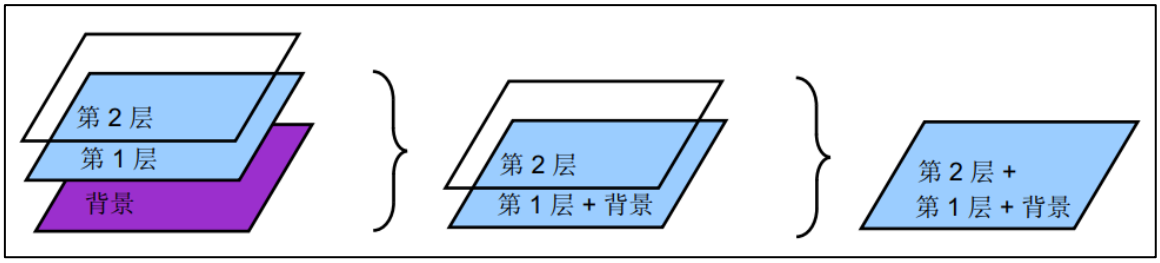


图 27.1.2.2 三个层混合关系

从图中可以看出，第二层位于最顶端，背景层位于最低端，混合单元首先将第一层与背景层进行混合，随后，第二层与第一层和第二层的混合颜色结果再次混合，完成混合后，送给 LCD 显示。

③ AHB 接口

由于 LTDC 驱动 RGBLCD 的时候，需要有很多内存来做显存，比如一个 800*480 的屏幕，按一般的 16 位 RGB565 模式，一个像素需要 2 个字节的内存，总共需要：800*480*2 =768K 字节内存，STM32 内部是没有这么多内存的，所以必须借助外部 SDRAM，而 SDRAM 是挂在 AHB 总线上的，LTDC 的 AHB 接口，就是用来将显存数据，从 SDRAM 存储器传输到 FIFO 里面。

④ 配置和状态寄存器

此部分包含了 LTDC 的各种配置寄存器以及状态寄存器，用于控制整个 LTDC 的工作参数，主要有：各信号的有效电平、垂直/水平同步时间参数、像素格式、数据使能等等。LTDC 的同步时序（HV 模式）控制框图，如图 27.1.2.3 所示：

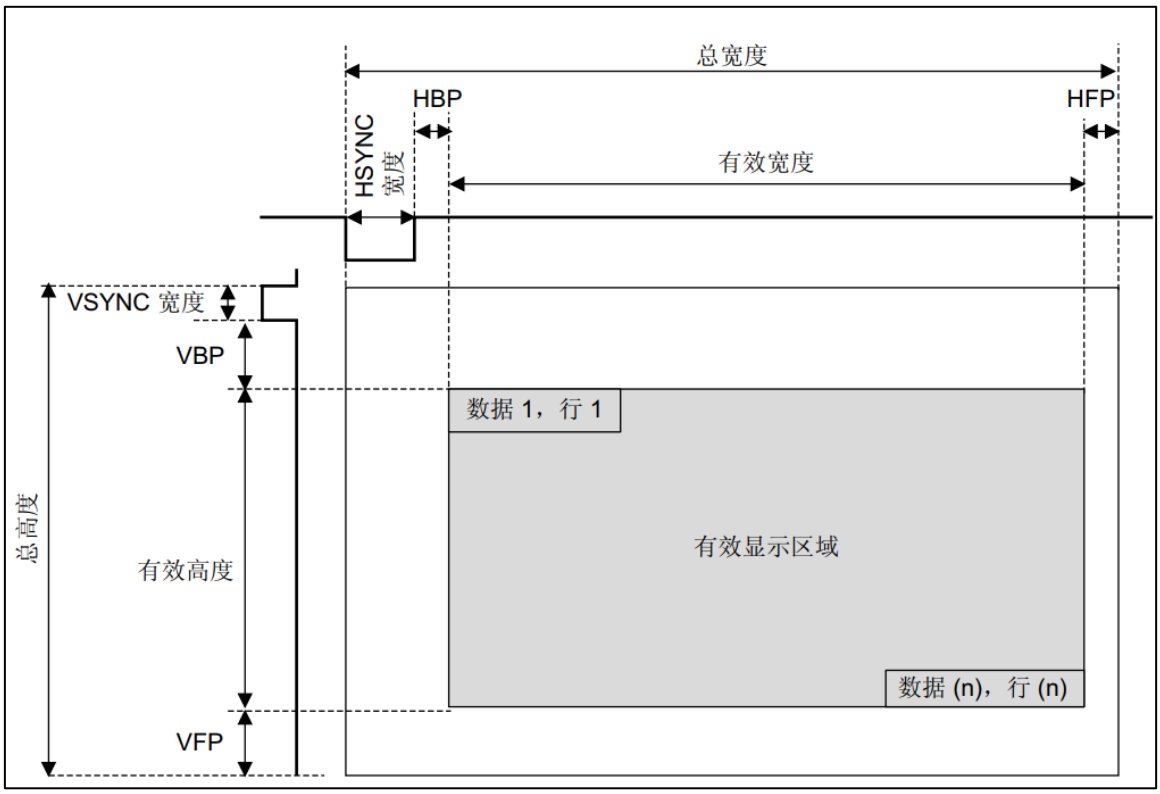


图 27.1.2.3 LTDC 同步时序框图

图中有效显示区域，就是我们 RGBLCD 面板的显示范围（即分辨率），有效宽度*有效高度，就是 LCD 的分辨率。另外，这里还有的参数包括：HSYNC 的宽度（HSW）、VSYNC 的宽度（VSW）、HBP、HFP、VBP 和 VFP 等，这些参数的说明，见表 27.1.2.2 所示：

参数	说明
HSW(horizontal sync width)	水平同步脉宽，单位为相素时钟（CLK）个数
VSW(vertical sync width)	垂直同步脉宽，单位为行周期个数
HBP(horizontal back porch)	水平后廊，表示水平同步信号开始到行有效数据开始之间的相素时钟（CLK）个数
HFP(horizontal front porch)	水平前廊，表示行有效数据结束到下一个水平有效信号开始之前的相素时钟（CLK）个数
VBP(vertical back porch)	垂直后廊，表示垂直同步信号后，无效行的个数
VFP(vertical front porch)	垂直前廊，表示一帧数据输出结束后，到下一个垂直同步信号开始之前的无效行数

表 27.1.2.2 LTDC 驱动时序参数

如果 RGBLCD 使用的是 DE 模式， LTDC 也只需要设置表 27.1.2.2 所示的参数，然后 LTDC 会根据这些设置，自动控制 DE 信号。这些参数通过相关寄存器来配置，接下来，我们介绍一下 LTDC 的一些相关寄存器。

首先，我们来看 LTDC 全局控制寄存器：LTDC_GCR，该寄存器各位描述如图 27.1.2.4 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
HSPOL	VSPOL	DEPOL	PCPOL	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DEN
rw	rw	rw	rw												rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	DRW[2:0]			Res.	DGW[2:0]			Res.	DBW[2:0]			Res.	Res.	Res.	LTDCEN
	r	r	r		r	r	r		r	r	r				rw

图 27.1.2.4 LTDC_GCR 寄存器各位描述

该寄存器我们在本章用到的设置有： LTDCEN、 PCPOL、 DEPOL、VSPOL 和 HSPOL 这几个设置，我们将逐个介绍。

LTDCEN： TFT LCD 控制器使能位，也就是 LTDC 的开关，该位需要设置为 1。

PCPOL：像素时钟极性。控制像素时钟的极性，根据 LCD 面板的特性来设置，我们所用的 LCD 一般设置为 0 即可，表示低电平有效。

DEPOL：数据使能极性。控制 DE 信号的极性，根据 LCD 面板的特性来设置，我们所用的 LCD 一般设置为 0 即可，表示低电平有效。

VSPOL：垂直同步极性。控制 VSYNC 信号的极性，根据 LCD 面板的特性来设置，我们所用的 LCD 一般设置为 0 即可，表示低电平有效。

HSPOL：水平同步极性。控制 HSYNC 信号的极性，根据 LCD 面板的特性来设置，我们所用的 LCD 一般设置为 0 即可，表示低电平有效。

接下来，我们看看 LTDC 同步大小配置寄存器： LTDC_SSCR，该寄存器各位描述如图 27.1.2.5 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	HSW[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	VSH[10:0]										
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.2.5 LTDC_SSCR 寄存器各位描述

该寄存器用于设置垂直同步高度（VSH）和水平同步宽度（HSW），其中：

VSH：表示垂直同步高度（以水平扫描行为单位），表示垂直同步脉宽减 1，即 VSW-1。

HSW：表示水平同步宽度（以像素时钟为单位），表示水平同步脉宽减 1，即 HSW-1.

接下来，我们看看 LTDC 后沿配置寄存器：LTDC_BPCR，该寄存器各位描述如图 27.1.2.6 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	AHBP[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	AVBP[10:0]										
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.2.6 LTDC_BPCR 寄存器各位描述

该寄存器我们需要配置 AVBP 和 AHBP：

AVBP：累加垂直后沿（以水平扫描行为单位），表示：VSW+VBP-1（见表 27.1.2.2）

AHBP：累加水平后沿（以像素时钟为单位），表示 HSW+HBP-1（见表 27.1.2.2，下同）

接下来，我们看看 LTDC 有效宽度配置寄存器：LTDC_AWCR，该寄存器各位描述如图 27.1.2.7 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved				AAW											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved				AAH											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.2.7 LTDC_AWCR 寄存器各位描述

该寄存器我们需要配置 AAH 和 AAW：

AAH：累加有效高度（以水平扫描行为单位），表示：VSW+VBP+有效高度-1。

AAW：累加有效宽度（以像素时钟为单位），表示：HSW+HBP+有效宽度-1。

这里所说的有效高度和有效宽度，是指 LCD 面板的宽度和高度（构成分辨率，下同）

接下来，我们看看 LTDC 总宽度配置寄存器：LTDC_TWCR，该寄存器各位描述如图 27.1.2.8 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	TOTALW[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
16	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	TOTALH[10:0]										
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.2.8 LTDC_TWCR 寄存器各位描述

该寄存器我们需要配置 TOTALH 和 TOTALW：

TOTALH：总高度（以水平扫描行为单位），表示：VSW+VBP+有效高度+VFP-1.

TOTALW：总宽度（以像素时钟为单位），表示：HSW+HBP+有效宽度+HFP-1.

接下来，我们看看 LTDC 背景色配置寄存器：LTDC_BCCR，该寄存器各位描述如图 27.1.2.9 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BCRED[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BCGREEN[7:0]								BCBLUE[7:0]							
rw								rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.2.9 LTDC_BCCR 寄存器各位描述

该寄存器定义背景层的颜色（RGB888），通过低 24 位配置，我们一般设置为全 0 即可。

接下来，我们看看 LTDC 的层颜色帧缓冲区地址寄存器：LTDC_LxCFBAR(x=1/2)，该寄存器各位描述如图 27.1.2.10 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CFBADD[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CFBADD[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.2.10 LTDC_LxCFBAR 寄存器各位描述

该寄存器用来定义一层显存的起始地址。STM32H743 的 LTDC 支持 2 个层，所以总共两个寄存器，分别设置层 1 和层 2 的显存起始地址。

接下来，我们看看 LTDC 的层像素格式配置寄存器：LTDC_LxPFCR(x=1/2)，该寄存器各位描述如图 27.1.2.11 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PF[2:0]		
													rw	rw	rw

图 27.1.2.11 LTDC_LxPFCR 寄存器各位描述

该寄存器只有最低 3 位有效，用于设置层颜色的像素格式：000：ARGB8888；001：RGB 888；010：RGB565；011：ARGB1555；100：ARGB4444；101：L8（8 位 Luminance）;110：AL44(4 位 Alpha，4 位 Luminance)；111：AL88（8 位 Alpha，8 位 Luminance）。我们一般使用 RGB565 格式，即该寄存器设置为：010 即可。

接下来，我们看看 LTDC 的层恒定 Alpha 配置寄存器：LTDC_LxCACR(x=1/2)，该寄存器各位描述如图 27.1.2.12 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CONSTA[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.2.12 LTDC_LxCACR 寄存器各位描述

该寄存器低 8 位（CONSTA）有效，这些位配置混合时使用的恒定 Alpha。恒定 Alpha 由硬件实现 255 分频。关于这个恒定 Alpha 的使用，我们将在介绍 LTDC_LxBFCR 寄存器的时候进行讲解。

接下来，我们看看 LTDC 的层默认颜色配置寄存器：LTDC_LxDCCR(x=1/2)，该寄存器各位描述如图 27.1.2.13 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DCALPHA[7:0]								DCRED[7:0]							
rw								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DCGREEN[7:0]								DCBLUE[7:0]							
rw								rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.2.13 LTDC_LxDCCR 寄存器各位描述

该寄存器定义采用 ARGB8888 格式的层的默认颜色。默认颜色在定义的层窗口外使用或在层禁止时使用。一般情况下，用不到，所以该寄存器一般设置为 0 即可。

接下来，我们看看 LTDC 的层混合系数配置寄存器：LTDC_LxBFCR(x=1/2)，该寄存器各位描述如图 27.1.2.14 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	BF1[2:0]			Res.	Res.	Res.	Res.	Res.	BF2[2:0]		
					rw	rw	rw						rw	rw	rw

图 27.1.2.14 LTDC_LxBFCR 寄存器各位描述

该寄存器用于定义混合系数：BF1 和 BF2。BF1=100 的时候，使用恒定的 Alpha 混合系数（由 LTDC_LxCACR 寄存器设置恒定 Alpha 值），BF1=110 的时候，使用像素 Alpha*恒定 Alpha。像素 Alpha 即 ARGB 格式像素的 A 值（Alpha 值），仅限 ARGB 颜色格式时使用。在 RGB565 格式下，我们设置 BF1=100 即可。BF2 同 BF1 类似，BF2=101 的时候，使用恒定的 Alpha 混合系数，BF2=111 的时候，使用像素 Alpha*恒定 Alpha。在 RGB565 格式下，我们设置 BF2=101

即可。

通用的混合公式为：

$$BC = BF1 * C + BF2 * Cs$$

其中： BC=混合后的颜色； BF1=混合系数 1； C=当前层颜色，即我们写入层显存的颜色值；BF2=混合系数 2； Cs=底层混合后的颜色，对于层 1 来说， Cs=背景层的颜色，对于层 2 来说，Cs=背景层和层 1 混合后的颜色。

以使用恒定的 Alpha 值，并仅使能第一层为例，给大家讲解一下混色的计算方式。恒定 Alpha 的值由 LTDC_LxCACR 寄存器设置，恒定 Alpha=LTDC_LxCACR 设置值 /255 。 假设：LTDC_LxCACR=240； C=128； Cs（背景色） =48；那么恒定 Alpha=240/255=0.94，则：

$$BC = 0.94 * 128 + (1 - 0.94) * 48 = 123$$

则混合后，颜色值变成了 123。另外，需要注意的是： BF1 和 BF2 的恒定 Alpha 值互补，他们之和为 1，且 BF1 使用的是恒定 Alpha 值， BF2 使用的是互补值。一般情况下，我们设置 LTDC_LxCACR 的值为 255，这样，在使用恒定 Alpha 值的时候，可以得到 BC=C，即混合后的颜色，就是显存里面的颜色（不进行混色）。

LTDC 的层支持窗口设置功能，通过 LTDC_LxWHPCR 和 LTDC_LxWVPCR 这两个寄存器设置，可以调整显示区域的大小，如图 27.1.2.15 所示：

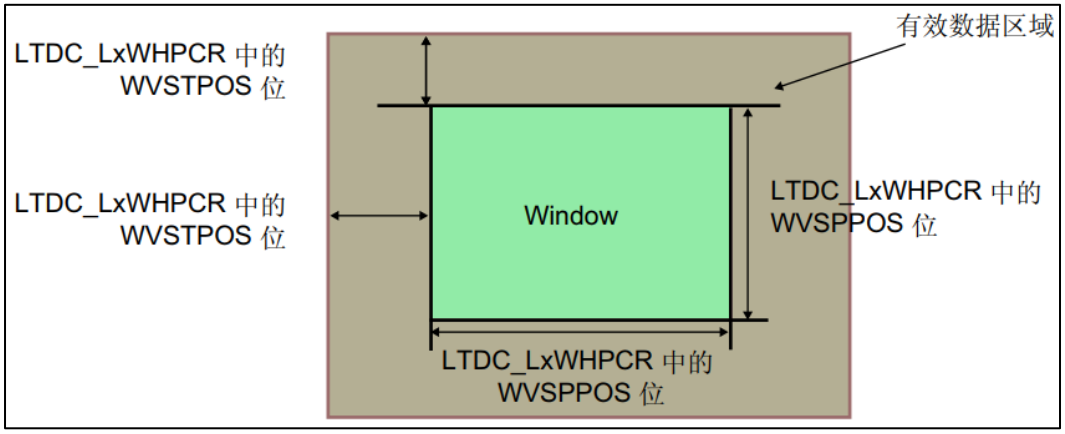


图 27.1.2.15 LTDC 层窗口设置关系图

上图中，层中的第一个和最后一个可见像素通过配置 LTDC_LxWVPCR 寄存器中的 WHSTPOS[11:0]和 WHSPPOS[11:0]进行设置，配置完成后，即可确定窗口的大小。

接下来，我们来介绍这两个寄存器，首先是 LTDC 的层窗口水平位置配置寄存器：

LTDC_LxWHPCR(x=1/2)，该寄存器各位描述如图 27.1.2.16 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	WHSPPOS[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	WHSTPOS[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.2.16 LTDC_LxWHPCR 寄存器各位描述

该寄存器定义第 1 层或第 2 层窗口的水平位置（第一个和最后一个像素），其中：

WHSTPOS：窗口水平起始位置，定义层窗口的一行的第一个可见像素，见图 27.1.2.15。

WHSPPOS：窗口水平停止位置，定义层窗口的一行的最后一个可见像素，见图 27.1.2.15。

然后，我们介绍 LTDC 的层窗口垂直位置配置寄存器： LTDC_LxWVPCR（x=1/2），该寄存器各位描述如图 27.1.2.17 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	WVSPPOS[10:0]										
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	WVSTPOS[10:0]										
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.2.17 LTDC_LxWVPCR 寄存器各位描述

该寄存器定义第 1 层或第 2 层窗口的垂直位置（第一行或最后一行），其中：

WVSTPOS：窗口垂直起始位置，定义层窗口的第一个可见行，见图 27.1.2.15。

WVSPPOS：窗口垂直停止位置，定义层窗口的最后一个可见行，见图 27.1.2.15。

接下来，我们看看 LTDC 的层颜色帧缓冲区长度寄存器：LTDC_LxCFBLR（x=1/2），该寄存器各位描述如图 27.1.2.18 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	CFBP[12:0]												
			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	CFBLL[12:0]												
			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.2.18 LTDC_LxCFBLR 寄存器各位描述

该寄存器定义颜色帧缓冲区的行长和行间距。其中：

CFBLL：这些位定义一行像素的长度（以字节为单位） +3。行长的计算方法为：有效宽度*每像素的字节数+3。比如， LCD 面板的分辨率为 800*480，有效宽度为 800，采用 RGB 565 格式，那么 CFBLL 需要设置为： 800*2+3=1603。

CFBP：这些位定义从像素某行的起始处到下一行的起始处的增量（以字节为单位）。这个设置，其实同样是一行像素的长度，对于 800*480 的 LCD 面板，RGB565 格式，设置 CFBP 为：800*2=1600 即可。

最后，我们看看 LTDC 的层颜色帧缓冲区行数寄存器：LTDC_LxCFBLNR（x=1/2），该寄存器各位描述如图 27.1.2.19 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	CFBLNBR[10:0]										
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.2.19 LTDC_LxCFBLNR 寄存器各位描述

该寄存器定义颜色帧缓冲区中的行数。 CFBLNBR 用于定义帧缓冲区行数，比如， LCD 面板的分辨率为 800*480，那么帧缓冲区的行数为 480 行，则设置 CFBLNBR=480 即可。

至此，LTDC 相关的寄存器，基本就介绍完了，通过这些寄存器的配置，我们就可以完成对 LTDC 的初始化，控制 LCD 显示了。关于 LTDC 的详细介绍，和寄存器描述，请看《STM32H7xx 参考手册_V3（中文版）.pdf》第 32 章。

⑤ 时钟域

LTDC 有三个时钟域： AXI 时钟域（ltdc_aclk）、 APB3 时钟域（ltdc_plck）和像素时钟域（ltdc_ker_ck），时钟域用于驱动：接口读取存储器的数据到 FIFO 里面，APB3 时钟域用于配置寄存器，像素时钟域则用于生成 LCD 接口信号（LCD_HSYNC、LCD_VSYNC 和 LCD_CLK 等），这些时钟信号需根据 LCD 面板的要求进行配置。

接下来，我们重点介绍下 LCD_CLK 的配置过程。LCD_CLK 的时钟源来自 ltdc_ker_ck，而 ltdc_ker_ck 则来自 PLL3 的 R 分频，如图 27.1.2.20 所示：

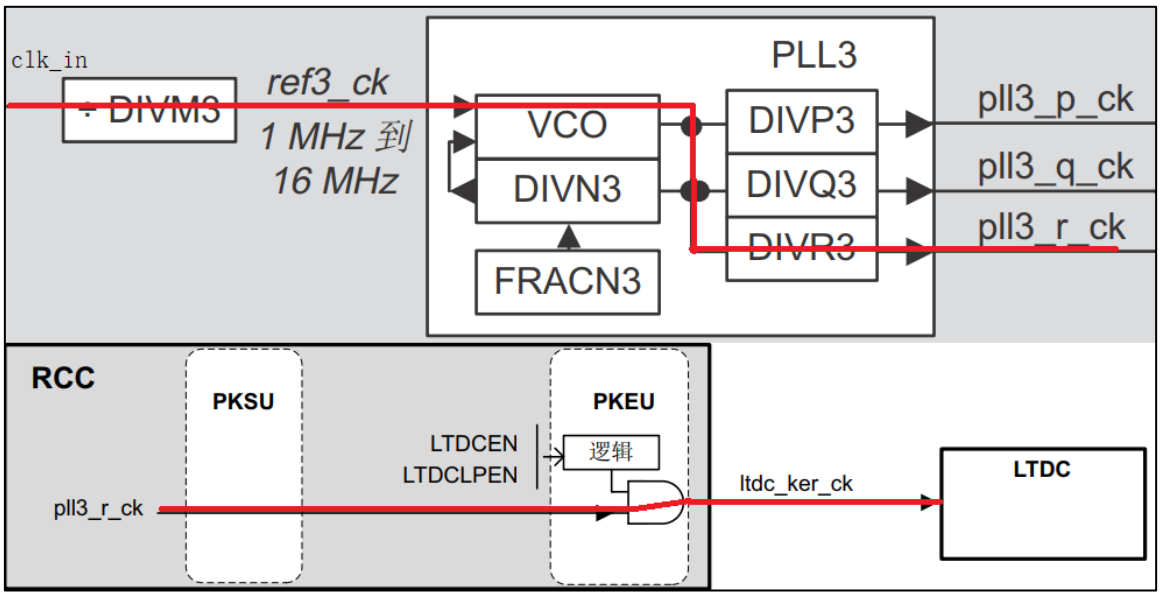


图 27.1.2.20 ltdc_ker_ck 时钟图

由图可知，ltdc_ker_ck 的来源为 clk_in，一般为外部晶振（假定外部晶振作为系统时钟源），经过 DIVM3 分频器分频，输入锁相环进行倍频（DIVN3）和分频（DIVR3），输出 pll3_r_ck，然后通过 PKSU 和 PKEU 得到 ltdc_ker_ck，最终输出到 LTDC，产生 LCD_CK，驱动液晶面板。接下来，我们简单介绍下配置 LCD_CLK 需要用到的一些寄存器。

首先是 PLL 时钟选择寄存器：RCC_PLLCKSELR，该寄存器的各位描述如图 27.1.2.21 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	DIVM3[5:0]						Res.	Res.	DIVM2[5:4]	
						rw								rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIVM2[3:0]				Res.	Res.	DIVM1[5:0]						Res.	Res.	PLLSRC[1:0]	
rw						rw								rw	

图 27.1.2.21 RCC_PLLCKSELR 寄存器各位描述

其中，PLLSRC[1:0]用于选择所有 PLL 的输入时钟源，也就是图 27.1.2.20 中 clk_in 的来源，我们在 SYSTEM 文件夹介绍的时候，就已经设置了这两个位，设置 PLLSRC[1:0]=2，即选择 PLL 输入时钟源为 HSE（hse_ck，外部晶振频率为 25MHz）。

DIVM3[5:0]用于设置 PLL3 输入时钟的分频系数，我们一般设置为 25，这样就可以得到 PLL3 的输入时钟频率为 25MHz/25=1MHz。

接下来，我们看 PLL3 分频配置寄存器（RCC_PLL3DIVR），该寄存器各位描述如图 27.1.2.22 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	DIVR3[6:0]							Res.	DIVQ3[6:0]						
	rw								rw						
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIVP3[6:0]								DIVN3[8:0]							
rw								rw							

图 27.1.2.22 RCC_PLL3DIVR 寄存器各位描述

这个寄存器主要对 PLL3 倍频器的：N、P、Q 和 R 等参数进行配置，他们的设置关系（假定使用外部 HSE 作为时钟源）为：

$$f(XCO\ clock) = f(hse) \times (DIVN3[8:0] / DIVM3[5:0])$$
$$f(PLL3_P) = f(VCO\ clock) / DIVP3[6:0]$$
$$f(PLL3_Q) = f(VCO\ clock) / DIVQ3[6:0]$$
$$f(PLL3_R) = f(VCO\ clock) / DIVR3[6:0]$$

f（hse）为我们外部晶振的频率，DIVM3 就是 PLL3 的 M 分频因子，DIVN3 为 PLL3 的倍

频数，取值范围为：3~511（表示 4~512 倍频）；DIVP3、DIVQ3 和 DIVR3 为 PLL3 的 P、Q 和 R 分频系数，取值范围为：2~128；阿波罗 STM32H743 核心板所用的 HSE 晶振频率为 25Mhz，一般我们设置 PLLM3 为 25，那么输入 PLL3 的输入时钟频率就是 1Mhz，然后可得：

f(ltdc_ker_ck) = 1Mhz * DIVN3[8:0] / DIVR3[6:0]

LCD_CLK 的时钟频率就等于 ltdc_ker_ck，因此 LCD_CLK 时钟频率计算公式就是上式。以群创 AT070TN92 面板为例，查其数据手册，可知 DCLK 的频率典型值为： 33.3Mhz，我们需要设置：DIVN3[8:0]=300，DIVR3[6:0]=9，DIVM3[5:0]=25，得到

f(LCD_CLK) = 1Mhz * 300 / 9 = 33.33Mhz

为了让 PLL3 正常工作，我们还需要配置 RCC_PLLCFGR 和 RCC_CR。RCC_PLLCFGR 寄存器各位描述如图 27.1.2.23 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	DIVR3EN	DIVQ3EN	DIVP3EN	DIVR2EN	DIVQ2EN	DIVP2EN	DIVR1EN	DIVQ1EN	DIVP1EN
							rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	PLL3RGE[1:0]		PLL3VCOSEL	PLL3FRACEN	PLL2RGE[1:0]		PLL2VCOSEL	PLL2FRACEN	PLL1RGE[1:0]		PLL1VCOSEL	PLL1FRACEN
				rw		rw	rw	rw		rw	rw	rw		rw	rw

图 27.1.2.23 RCC_PLLCFGR 寄存器各位描述

因为 ltdc_ker_ck 时钟来自 PLL3 的 R 分频，因此 RCC_PLLCFGR 寄存器我们只需要关心 PLL3FRACEN、PLL3VCOSEL、PLL3RGE[1:0]和 DIVR3EN 这些位。接下来我们分别介绍：

PLL3FRACEN 位，用于设置是否使能分数倍频器，本章我们不需要分数倍频，因此设置该位为 0 即可。

PLL3VCOSEL 位，用于选择 VCO 的范围。当 VCO 的范围在 192~836MHz 之间时，设置该位为 0；当 VCO 范围在 150~420MHz 之间时，设置该位为 1；本章我们设置该位为 0，使用 192~836MHz 的倍频范围。

PLL3RGE[1:0]位，用于设置 PLL3 输入频率范围。当输入频率范围在 1~2MHz 之间时，设置这两个位为 0 即可，本章输入范围为 1MHz（25M/25），因此设置这两个位为 0 即可。

DIVR3EN 位，用于使能 PLL3 的 R 分频输出。本章需要用到的 R 分频输出，因此必须设置该位为 1。

最后，我们看看 RCC 控制寄存器：RCC_CR，该寄存器各位描述如图 27.1.2.24 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	PLL3RDY	PLL3ON	PLL2RDY	PLL2ON	PLL1RDY	PLL1ON	Res.	Res.	Res.	Res.	HSECSSON	HSEBYP	HSERDY	HSEON
		r	rw	r	rw	r	rw					rs	rw	r	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
D2CKRDY	D1CKRDY	HSI48RDY	HSI48ON	Res.	Res.	CSIKERON	CSIRDY	CSION	Res.	HSIDIVF	HSIDIV[1:0]		HSIRDY	HSIKERON	HSION
r	r	r	rw			rw	r	rw		r	rw	rw	r	rw	rw

图 27.1.2.24 RCC_CR 寄存器各位描述

该寄存器我们只关心 PLL3ON 和 PLL3RDY 两个位：

PLL3ON 位，用于使能 PLL3，我们需要用到 PLL3 的 R 分频来产生 LCD_CLK 时钟，因此必须使能 PLL3，该位需要设置为 1.

PLL3RDY 位，用于查询 PLL3 锁相环是否锁定了，如果锁定了，该位为 1，否则该位为 0。我们在开启 PLL3 之后，需要查询该位来判断 PLL3 是否准备好（锁定）。

最后，我们来看看实现 LTDC 驱动 RGBLCD，需要对 LTDC 进行哪些配置。LTDC 相关 HAL 库操作分布在函数 stm32h7xx_hal_ltdc.c 和 stm32h7xx_hal_ltdc_ex.c 以及他们对应的头文件

中。操作步骤如下：

1） 使能 LTDC 时钟，并配置 LTDC 相关的 IO 及其时钟使能。

要使用 LTDC，当然首先得开启其时钟。然后需要把 LCD_R/G/B 数据线、LCD_HSYNC 和 LCD_VSYNC 等相关 IO 口，全部配置为复用输出，并使能各 IO 组的时钟。GPIO 配置这里我们就不做讲解，LTDC 时钟使能方法为：

```
__HAL_RCC_LTDC_CLK_ENABLE();                    /* 使能 LTDC 时钟 */
```

2） 设置 LTDC_CLK 时钟。

此步需要配置 LTDC 的像素时钟，根据 LTDC 的面板参数进行设置，LTDC_CLK 由 PLLSAI 进行配置，前面我们已经讲解非常详细，配置使用到的 HAL 库函数为：

```
HAL_StatusTypeDef HAL_RCCEx_PeriphCLKConfig(RCC_PeriphCLKInitTypeDef
                                            *PeriphClkInit)
```

该函数是 HAL 库提供的用来配置扩展外设时钟通用函数。LTDC_CLK 前面讲解过，它来自 PLL3R，可知，我们需要配置 PLL3M，PLL3N 和 PLL3R 等参数，具体使用实例如下：

```
uint8_t ltdc_clk_set(uint32_t pll3m, uint32_t pll3n, uint32_t pll3r)
{
    RCC_PeriphCLKInitTypeDef periphclk_initure;
    /* LTDC 输出像素时钟，需要根据自己所使用的 LTDC 数据手册来配置！ */
    periphclk_initure.PeriphClockSelection=RCC_PERIPHCLK_LTDC;  /* LTDC 时钟 */
    periphclk_initure.PLL3.PLL3M = pll3m;
    periphclk_initure.PLL3.PLL3N = pll3n;
    periphclk_initure.PLL3.PLL3P = 2;
    periphclk_initure.PLL3.PLL3Q = 2;
    periphclk_initure.PLL3.PLL3R = pll3r;
    if(HAL_RCCEx_PeriphCLKConfig(&periphclk_initure) == HAL_OK)/* 配置像素时钟 */
    {
        return 0;   /* 成功 */
    }
    else return 1;  /* 失败 */
}
```

3）设置 RGBLCD 的相关参数，并使能 LTDC。

这一步，我们需要完成对 LTDC 面板参数的配置，包括：LTDC 使能、时钟极性、HSW、VSW、HBP、HFP、VBP 和 VFP 等（见表 27.1.2.2），通过 LTDC_GCR、LTDC_SSCR、LTDC_BPCR、LTDC_AWCR 和 LTDC_TWCR 等寄存器配置。HAL 库配置 LTDC 参数并使能 LTDC 的函数为：

```
HAL_StatusTypeDef HAL_LTDC_Init(LTDC_HandleTypeDef *hltdc);
```

该函数只有一个入口参数就是 hltdc，为 LTDC_HandleTypeDef 结构体指针类型。接下来我们看看 LTDC_HandleTypeDef 结构体定义如下：

```
typedef struct
{
  LTDC_TypeDef              *Instance;
  LTDC_InitTypeDef           Init;
  LTDC_LayerCfgTypeDef       LayerCfg[MAX_LAYER];
  HAL_LockTypeDef            Lock;
  __IO HAL_LTDC_StateTypeDef State;
  __IO uint32_t              ErrorCode;
}LTDC_HandleTypeDef;
```

该结构体有 6 个成员变量。成员变量 Lock 和 State，他们是 HAL 库用来标识一些状态过程的变量，这里就不做过多讲解。Instance 变量是 LTDC_TypeDef 结构体指针类型，和其他初始化结构体成员变量 Instance 一样都是用来设置配置寄存器的基地址，这在 HAL 库中已经通过宏定义定好了，设置值为 LTDC 即可。成员变量 LayerCfg 是一个数组，它是用来保存 LTDC 层配置参数，下一步我们会讲解。最后我们重点看看成员变量 Init，它是真正用来初始化 LTDC 的结构体变量，结构体 LTDC_InitTypeDef 定义如下:

```
typedef struct
{
  uint32_t            HSPolarity;             /* 水平同步极性 */
  uint32_t            VSPolarity;             /* 垂直同步极性 */
  uint32_t            DEPolarity;             /* 数据使能极性 */
```

```
  uint32_t            PCPolarity;             /* 像素时钟极性 */
  uint32_t            HorizontalSync;         /* 水平同步宽度 */
  uint32_t            VerticalSync;           /* 垂直同步宽度 */
  uint32_t            AccumulatedHBP;         /* 水平同步后沿宽度 */
  uint32_t            AccumulatedVBP;         /* 垂直同步后沿高度 */
  uint32_t            AccumulatedActiveW;     /* 累加有效宽度 */
  uint32_t            AccumulatedActiveH;     /* 累加有效高度 */
  uint32_t            TotalWidth;             /* 总宽度 */
  uint32_t            TotalHeigh;             /* 总宽度 */
  LTDC_ColorTypeDef Backcolor;                /* 屏幕背景层颜色 */
} LTDC_InitTypeDef;
```

这些参数含义我们都在结构体成员变量之后注释了，具体含义大家可以参考前面第四点配置和状态寄存器讲解。

```
    LTDC_HandleTypeDef  g_ltdc_handle;
    /* LTDC 配置 */
    g_ltdc_handle.Instance = LTDC;
    g_ltdc_handle.Init.HSPolarity = LTDC_HSPOLARITY_AL; /* 水平同步极性 */
    g_ltdc_handle.Init.VSPolarity = LTDC_VSPOLARITY_AL; /* 垂直同步极性 */
    g_ltdc_handle.Init.DEPolarity = LTDC_DEPOLARITY_AL; /* 数据使能极性 */
    g_ltdc_handle.Init.HorizontalSync = lcdltdc.hsw - 1;/* 水平同步宽度 */
    g_ltdc_handle.Init.VerticalSync = lcdltdc.vsw - 1;  /* 垂直同步宽度 */
    g_ltdc_handle.Init.AccumulatedHBP = lcdltdc.hsw + lcdltdc.hbp - 1;
    /* 水平同步后沿宽度 */
    g_ltdc_handle.Init.AccumulatedVBP = lcdltdc.vsw + lcdltdc.vbp - 1;
    /* 垂直同步后沿高度 */
    g_ltdc_handle.Init.AccumulatedActiveW = lcdltdc.hsw + lcdltdc.hbp +
                                            lcdltdc.pwidth - 1; /* 有效宽度 */
    g_ltdc_handle.Init.AccumulatedActiveH = lcdltdc.vsw + lcdltdc.vbp +
                                            lcdltdc.pheight - 1;/* 有效高度 */
    g_ltdc_handle.Init.TotalWidth = lcdltdc.hsw + lcdltdc.hbp + lcdltdc.pwidth
                                    + lcdltdc.hfp - 1;          /* 总宽度 */
    g_ltdc_handle.Init.TotalHeigh = lcdltdc.vsw + lcdltdc.vbp + lcdltdc.pheight
                                    + lcdltdc.vfp - 1;          /* 总高度 */
    g_ltdc_handle.Init.Backcolor.Red = 0;           /* 屏幕背景层红色部分 */
    g_ltdc_handle.Init.Backcolor.Green = 0;         /* 屏幕背景层绿色部分 */
    g_ltdc_handle.Init.Backcolor.Blue = 0;          /* 屏幕背景色蓝色部分 */
    HAL_LTDC_Init(&g_ltdc_handle);
```

和其他外设或接口初始化一样，HAL 同样提供了 LTDC 初始化 MSP 回调函数，HAL_LTDC_MspInit，该函数一般用来使能时钟和初始化 IO 口等于 MCU 相关操作：

```
void HAL_LTDC_MspInit(LTDC_HandleTypeDef* hltdc)
```

4）设置 LTDC 层参数。

此步，我们需要设置 LTDC 某一层的相关参数，包括：帧缓存首地址、颜色格式、混合系数和层默认颜色等。通过 LTDC_LxCFBAR、LTDC_LxPFCR、LTDC_LxCACR、LTDC_LxDCCR 和 LTDC_LxBFCR 等寄存器配置。HAL 库提供的 LTDC 层参数配置函数为：

```
HAL_StatusTypeDef HAL_LTDC_ConfigLayer(LTDC_HandleTypeDef *hltdc,
                       LTDC_LayerCfgTypeDef *pLayerCfg, uint32_t LayerIdx)
```

基于篇幅考虑，该函数具体的入口参数定义这里我们就不做过多讲解，具体使用方法请参考 27.3 小节函数讲解以及实验工程。

5）设置 LTDC 层窗口，并使能层。

这一步，完成对 LTDC 某个层的显示窗口设置（一般设置为整层显示，不开窗），通过 LTDC_LxWHPCR、LTDC_LxWVPCR、LTDC_LxCFBLR 和 LTDC_LxCFBLNR 等寄存器配置。层使能通过配置 LTDC_LxCR 寄存器的最低位实现，使能层以后，RGBLCD 就可以正常工作了。HAL 库提供的 LTDC 层窗口配置函数为：

```
HAL_StatusTypeDef HAL_LTDC_SetWindowSize(LTDC_HandleTypeDef *hltdc,
           uint32_t XSize, uint32_t YSize, uint32_t LayerIdx)/* 层窗口尺寸配置 */
```

```
HAL_StatusTypeDef HAL_LTDC_SetWindowPosition(LTDC_HandleTypeDef *hltdc,
                uint32_t X0, uint32_t Y0, uint32_t LayerIdx)  /* 层窗口位置配置 */
```

这两个函数使用方法非常简单，这里我们就不累赘了。

通过以上几个步骤，我们就完成了 LTDC 的配置，可以控制 RGBLCD 显示了。LTDC 我们就给大家介绍到这里，接下来，我们介绍 DMA2D。

27.1.3 DMA2D 简介

为了提高 STM32H743 的图像处理能力，ST 公司设计了一个专用于图像处理的专业 DMA：Chrom-Art Accelerator™，即：DMA2D，通过 DMA2D 对图像进行填充和搬运，可以完全不用 CPU 干预，从而提高效率，减轻 CPU 负担。它可以执行下列操作：

- 用特定颜色填充目标图像的一部分或全部（可用于快速单色填充）
- 将源图像的一部分（或全部）复制到目标图像的一部分（或全部）中（可用于快速图像填充）
- 通过像素格式转换将源图像的一部分（或全部）复制到目标图像的一部分（或全部）中
- 将像素格式不同的两个源图像部分和/ 或全部混合，再将结果复制到颜色格式不同的部分或整个目标图像中。

DMA2D 有四种工作模式，通过 DMA2D_CR 寄存器的 MODE[1:0]位选择工作模式：

1， 寄存器到存储器
2， 存储器到存储器
3， 存储器到存储器并执行 PFC
4， 存储器到存储器并执行 PFC 和混合

本章，我们仅介绍前两种工作模式，后两种工作模式，请大家参考《STM32H7xx 参考手册_V3(中文版).pdf》第 17 章。

寄存器到存储器

寄存器到存储器模式用于以预定义颜色填充用户自定义区域，也就是可以实现快速的单色填充显示，比如清屏操作。

在该模式下：颜色格式在 DMA2D_OPFCCR 中设置， DMA2D 不从任何源获取数据，它只是将 DMA2D_OCOLR 寄存器中定义的颜色写入通过 DMA2D_OMAR 寻址以及 DMA2D_NLR 和 DMA2D_OOR 定义的区域。

存储器到存储器

该模式下， DMA2D 不执行任何图形数据转换。前景层输入 FIFO 充当缓冲区，数据从 DMA2D_FGMAR 中定义的源存储单元传输到 DMA2D_OMAR 寻址的目标存储单元，可用于快速图像填充。 DMA2D_FGPFCCR 寄存器的 CM[3:0]位中编程的颜色模式决定输入和输出的每像素位数。对于要传输的区域大小，源区域大小由 DMA2D_NLR 和 DMA2D_FGOR 寄存器定义，目标区域大小则由 DMA2D_NLR 和 DMA2D_OOR 寄存器定义。

以上两个工作模式， LTDC 在层帧缓存里面的开窗关系都一样，如图 27.1.3.1 所示：

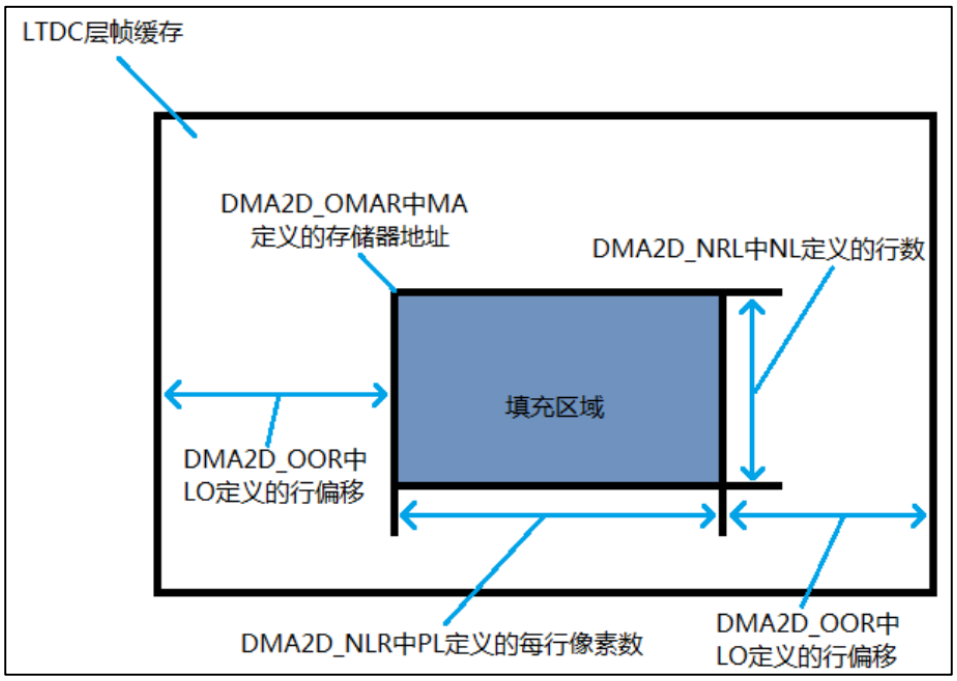


图 27.1.3.1 层帧缓冲开窗示意图

窗口显示区域的显存首地址由 DMA2D_OMAR 寄存器指定，窗口宽度和高度由 DMA2D_NRL 寄存器的 PL 和 NL 指定，行偏移（确定下一行的起始地址）由 DMA2D_OOR 寄存器指定，经过这三个寄存器的配置，就可以确定窗口的显示位置和大小。

在寄存器到存储器模式下，在开窗完成后， DMA2D 可以将 DMA2D_OCOLR 指定的颜色，自动填充到开窗区域，完成单色填充。

在存储器到存储器模式下，需要完成两个开窗：前景层和显示层，完成配置后，图像数据从前景层拷贝到显示层（仅限窗口范围内），从而显示到 LCD 上面。显示层的开窗，如图 27.1.3.1 所示，而前景层的开窗，则和图 27.1.3.1 所示相似，只是 DMA2D_OMAR 寄存器变成了 DMA2D_FGMAR， DMA2D_OOR 寄存器变成了 DMA2D_FGOR， DMA2D_NRL 则两个层共用，然后就可以完成对前景层的开窗，确定好两个窗口后， DMA2D 就将前景层窗口内的数据，拷贝到显示层窗口，完成快速图像填充。

接下来，我们介绍一下 DMA2D 的一些相关寄存器。

首先，我们来看 DMA2D 控制寄存器：DMA2D_CR，该寄存器各位描述如图 27.1.3.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved														MODE	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved		CEIE	CTCIE	CAEIE	TWIE	TCIE	TEIE	Reserved					ABORT	SUSP	START
		rw	rw	rw	rw	rw	rw						rs	rw	rs

图 27.1.3.2 DMA2D_CR 寄存器各位描述

该寄存器，我们主要关心 MODE 和 START 这两个设置，其中：

MODE[1:0]：表示 DMA2D 的工作模式， 00：存储器到存储器模式； 01：存储器到存储器模式并执行 PFC； 10：存储器到存储器并执行混合； 11，寄存器到存储器模式；

本章我们需要用到的设置为： 00 或者 11。

START：该位控制 DMA2D 的启动，在配置完成后，设置该位为 1，启动 DMA2D 传输。

接下来，我们介绍 DMA2D 输出 PFC 控制寄存器： DMA2D_OPFCCR，该寄存器各位描述如图 27.1.3.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RBS	AI	Res.	Res.	Res.	Res.
										rw	rw				
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CM[2:0]		
													rw	rw	rw

图 27.1.3.3 DMA2D_OPFCCR 寄存器各位描述

该寄存器用于设置寄存器到存储器模式下的颜色格式，只有最低 3 位有效（ CM[2:0]），表示的颜色格式有： 000， ARGB8888； 001： RGB888； 010： RGB565； 011： ARGB1555；100：ARGB1444。我们一般使用的是 RGB565 格式，所以设置 CM[2:0]=010 即可。

同样的，还有前景层 PFC 控制寄存器： DMA2D_FGPFCCR，该寄存器各位描述如图 27.1.3.4 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ALPHA[7:0]								Res.	Res.	RBS	AI	CSS[1:0]		AM[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CS[7:0]								Res.	Res.	START	CCM	CM[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw			rc_w1	rw	rw	rw	rw	rw

图 27.1.3.4 DMA2D_FGPFCCR 寄存器各位描述

该寄存器，我们只关心最低 4 位： CM[3:0]，用于设置存储器到存储器模式下的颜色格式，这四个位表示的颜色格式为： 0000： ARGB8888； 0001： RGB888； 0010： RGB565； 0011：ARGB1555；0100： ARGB4444； 0101： L8； 0110： AL44； 0111： AL88； 1000： L4；1001： A8； 1010： A4；

我们一般使用 RGB565 格式，所以设置 CM[3:0]=0010 即可。

接下来，我们介绍 DMA2D 输出偏移寄存器： DMA2D_OOR，该寄存器各位描述如图 27.1.3.5 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	LO[13:0]													
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.3.5 DMA2D_OOR 寄存器各位描述

该寄存器仅最低 14 位有效（LO[13:0]），用于设置输出行偏移，作用于显示层，以像素为单位表示。此值用于生成地址。行偏移将添加到各行末尾，用于确定下一行的起始地址，参见图 27.1.3.1。同样的，还有前景层偏移寄存器： DMA2D_FGOR，该寄存器同 DMA2D_OOR 一样，也是低 14 位有效，用于控制前景层的行偏移，也是用于生成地址，添加到各行末尾，从而确定下一行的起始地址。

接下来，我们介绍 DMA2D 输出存储器地址寄存器 ： DMA2D_OMAR，该寄存器各位描述如图 27.1.3.6 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MA[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.3.6 DMA2D_OMAR 寄存器各位描述

该寄存器设置由 MA[31:0]设置输出存储器地址，也就是输出 FIFO 所存储的数据地址，该

地址需要根据开窗的起始坐标来进行设置。以 800*480 的 LCD 屏为例，行长度为 800 像素，假定帧缓存数组为： g_ltdc_framebuf，我们设置窗口的起始地址为： sx（<800）， sy（<480），颜色格式为 RGB565，每个像素 2 个字节，那么 MA 的设置值应该为：

MA[31:0] = g_framebuf + 2 * (800 * sy + sx)

同样的，还有前景层偏移寄存器： DMA2D_ FGMAR，该寄存器同 DMA2D_OMAR 一样，不过是用于控制前景层的存储器地址，计算方法同 DMA2D_OMAR。

接下来，我们介绍 DMA2D 行数寄存器： DMA2D_NLR，该寄存器各位描述如图 27.1.3.7 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved		PL[13:0]													
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NL[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.3.7 DMA2D_NLR 寄存器各位描述

该寄存器用于控制每行的像素和行数，该寄存器的设置对前景层和显示层均有效，通过该寄存器的配置，就可以设置开窗的大小。其中：

NL[15: 0]：设置待传输区域的行数，用于确定窗口的高度。

PL[13: 0]：设置待传输区域的每行像素数，用于确定窗口的宽度。

接下来，我们介绍 DMA2D 输出颜色寄存器： DMA2D_OCOLR，该寄存器各位描述如图 27.1.3.8 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ALPHA[7:0]								RED[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
GREEN[7:0]								BLUE[7:0]							
RED[4:0]				GREEN[5:0]				BLUE[4:0]							
A	RED[4:0]				GREEN[4:0]			BLUE[4:0]							
ALPHA[3:0]				RED[3:0]				GREEN[3:0]				BLUE[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 27.1.3.8 DMA2D_OCOLR 寄存器各位描述

该寄存器用于配置在寄存器到存储器模式下，填充时所用的颜色值，该寄存器是一个 32 位寄存器，可以支持 ARGB8888 格式，也可以支持 RGB565 格式。我们一般使用 RGB565 格式，比如要填充红色，那么直接设置 DMA2D_OCOLR=0XF800 就可以了。

接下来，我们介绍 DMA2D 中断状态寄存器： DMA2D_ISR，该寄存器各位描述如图 27.1.3.9 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved										CEIF	CTCIF	CAEIF	TWIF	TCIF	TEIF
										r	r	r	r	r	r

图 27.1.3.9 DMA2D_ISR 寄存器各位描述

该寄存器表示了 DMA2D 的各种状态标识，这里我们只关心 TCIF 位，表示 DMA2D 的传输完成中断标志。当 DMA2D 传输操作完成（仅限数据传输）时此位置 1，表示可以开始下一次 DMA2D 传输了。

另外，还有一个 DMA2D 中断标志清零寄存器： DMA2D_IFCR，用于清除 DMA2D_ISR 寄存器对应位的标志。通过向该寄存器的第 1 位（CTCIF）写 1，可以用于清除 DMA2D_ISR

寄存器的 TCIF 位标志。

最后，我们来看看利用 DMA2D 完成颜色填充，需要哪些步骤。这里需要说明一下，使用官方提供的 HAL 库 DMA2D 相关库函数进行颜色填充效率极为低下，大量时间浪费在函数的入栈出栈以及过程处理，所以在项目开发中一般都不会使用 DMA2D 库函数进行颜色填充，包括 ST 官方提供的 STEMWIN 实例关于 DMA2D 部分，均采用的寄存器操作。 具体操作步骤如下：

1） 使能 DMA2D 时钟，并先停止 DMA2D。

要使用 DMA2D，先得开启其时钟。然后 DMA2D 在配置其相关参数的时候，需要先停 DMA2D 传输。 使能 DMA2D 时钟和停止 DMA2D 方法为

```
__HAL_RCC_DMA2D_CLK_ENABLE();      /* 使能 DM2D 时钟 */
DMA2D->CR &= ~(DMA2D_CR_START);    /* 先停止 DMA2D */
```

2） 设置 DMA2D 工作模式。

通过 DMA2D_CR 寄存器，配置 DMA2D 的工作模式。我们用了寄存器到存储器模式和存储器到存储器这两个模式。

寄存器到存储器模式设置：

```
DMA2D->CR = DMA2D_R2M;             /* 寄存器到存储器模式 */
```

寄存器到存储器模式设置：

```
DMA2D->CR = DMA2D_M2M;             /* 存储器到存储器模式 */
```

3） 设置 DMA2D 的相关参数。

这一步，我们需要设置：颜色格式、输出窗口、输出存储器地址、前景层地址（仅存储器到存储器模式需要设置）、颜色寄存器（仅 寄存器到存储器模式需要设置）等，由：DMA2D_OPFCCR、 DMA2D_FGPFCCR、 DMA2D_OOR、 DMA2D_FGOR 、 DMA2D_OMAR、DMA2D_FGMAR 和 DMA2D_NLR 等寄存器进行配置。 具体配置过程请参考实验源码。

4）启动 DMA2D 传输。

通过 DMA2D_CR 寄存器配置开启 DMA2D 传输，实现图像数据的拷贝填充，方法为：

```
DMA2D->CR |= DMA2D_CR_START;               /* 启动 DMA2D */
```

5） 等待 DMA2D 传输完成，清除相关标识。

最后，在传输过程中，不要再次设置 DMA2D，否则会打乱显示，所以一般在启动 DMA2D 后，需要等待 DMA2D 传输完成（判断 DMA2D_ISR），在传输完成后，清除传输完成标识（设置 DMA2D_IFCR），以便启动下一次 DMA2D 传输。 方法为：

```
while((DMA2D->ISR & (DMA2D_FLAG_TC)) == 0)   /* 等待传输完成 */
DMA2D->IFCR |= DMA2D_FLAG_TC;                /* 清除传输完成标志 */
```

通过以上几个步骤，我们就完成了 DMA2D 填充， DMA2D 的简介，我们就介绍完了，详细的介绍请大家参考《STM32H7xx 参考手册_V3(中文版).pdf》第 18 章。

27.2 硬件设计

1. 例程功能

本实验利用 STM32H743 的 LTDC 接口来驱动 RGB 屏，RGBLCD 模块的接口在核心板上，通过 40P 的 FPC 排线连接 RGBLCD 模块，实现 RGBLCD 模块的驱动和显示，下载成功后，按下复位之后，就可以看到 RGBLCD 模块不停的显示一些信息并不断切换底色。同时，屏幕上会显示 LCD 的 ID。

2. 硬件资源

1）LED 灯

LED0.：LED0 - PB1

2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)

3）正点原子 4.3 寸/7 寸/10.1 寸 RGBLCD 模块

4）外部 SDRAM（支持 W9825G6KH 和 EM63A165TS-6G 等多种型号）

3. 原理图

这里 RGBLCD 接口在 STM32H743 核心板上，原理图如图 27.2.1 所示

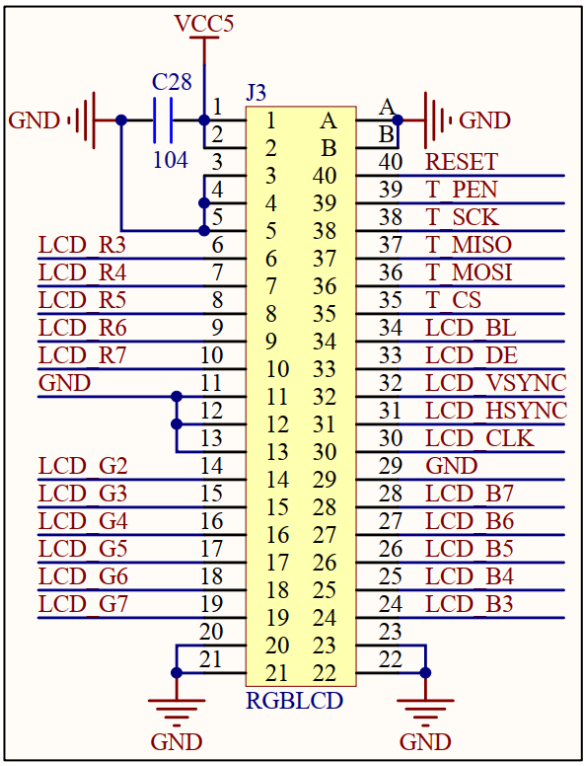


图 27.2.1 RGBLCD 接口原理图

图中 RGBLCD 接口的接线关系见表 27.1.2.1.这些线的连接，阿波罗 STM32H743 核心板的内部已经连接好了，我们只需将 RGBLCD 模块通过 40PIN 的 FPC 线连接这个 RGBLCD 接口即可。实物连接（7 寸 RGBLCD 模块）如图 27.2.2 所示：

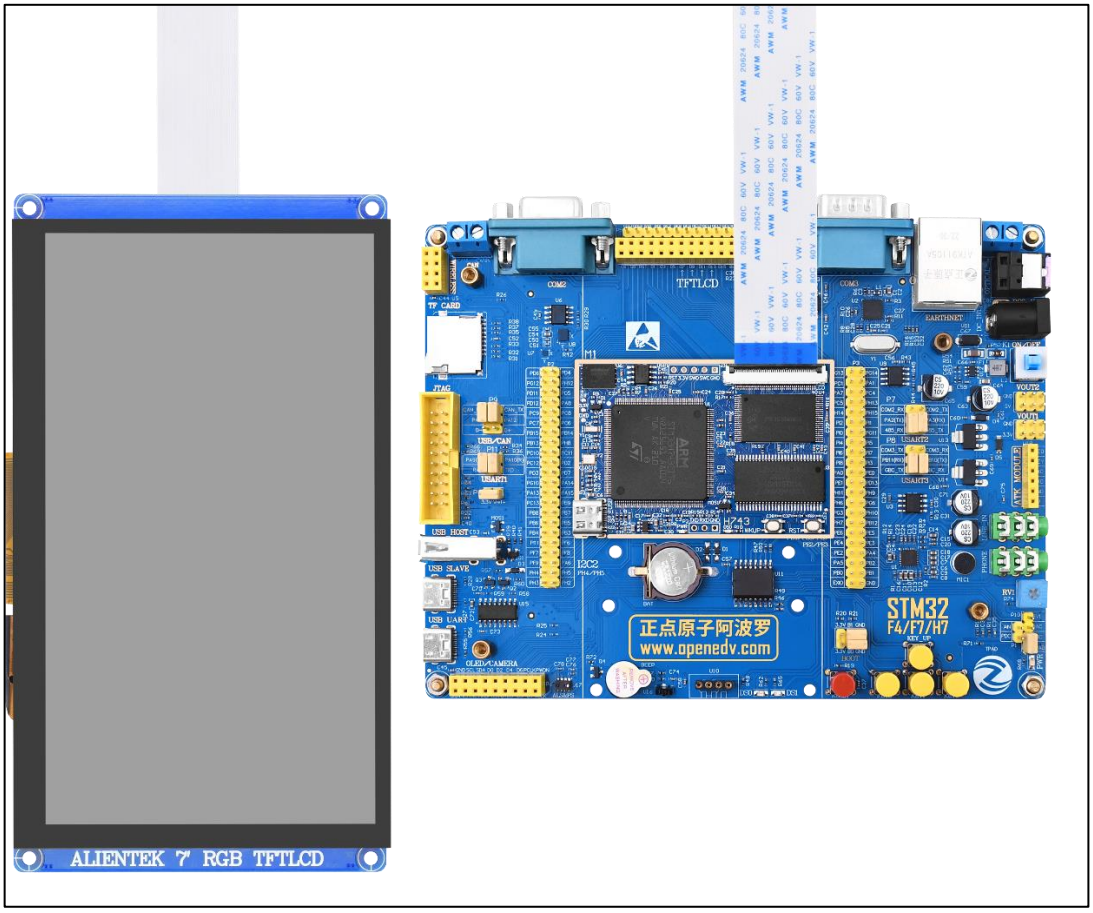


图 27.2.2 RGBLCD 与开发板连接实物图

27.3 程序设计

27.3.1 程序流程图

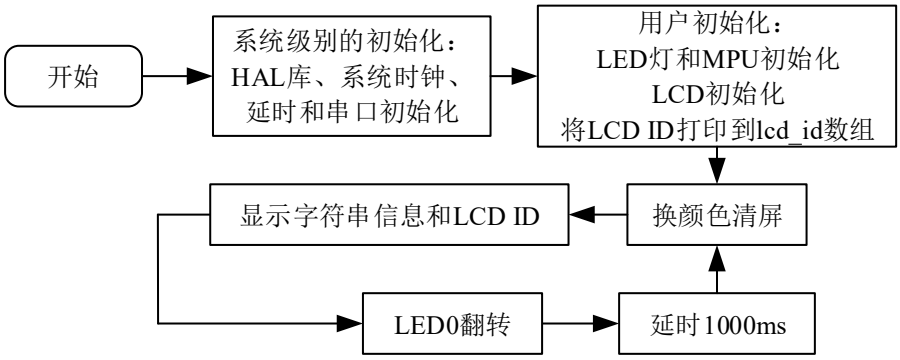


图 27.3.1.1 TFTLCD（RGB 屏）实验程序流程图

27.3.2 程序解析

1. LTDC 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。ltdc.c 代码比较多，这里就不一一贴出来了，只针对几个重要的函数进行讲解。

本实验，我们用到 LTDC 驱动 RGBLCD，通过前面的介绍，我们知道不同的 RGB 屏，驱动参数有一些差异，为了方便兼容不同的 RGBLCD，我们定义如下 LTDC 参数结构体（在

ltdc.h 里面定义）

```
/* LCD LTDC 重要参数集 */
typedef struct
{
    uint32_t pwidth;        /* LTDC 面板的宽度,固定参数,
                               不随显示方向改变,如果为 0,说明没有任何 RGB 屏接入 */
    uint32_t pheight;       /* LTDC 面板的高度,固定参数,不随显示方向改变 */
    uint16_t hsw;           /* 水平同步宽度 */
    uint16_t vsw;           /* 垂直同步宽度 */
    uint16_t hbp;           /* 水平后廊 */
    uint16_t vbp;           /* 垂直后廊 */
    uint16_t hfp;           /* 水平前廊 */
    uint16_t vfp;           /* 垂直前廊  */
    uint8_t activelayer;    /* 当前层编号:0/1 */
    uint8_t dir;            /* 0,竖屏;1,横屏; */
    uint16_t width;         /* LTDC 宽度 */
    uint16_t height;        /* LTDC 高度 */
    uint32_t pixsize;       /* 每个像素所占字节数 */
}_ltdc_dev;
```

该结构体用于保存一些 RGBLCD 重要参数信息，比如 LTDC 面板的长宽、水平后廊和垂直后廊等参数。这个结构体虽然占用了几十个字节的内存，但是却可以让我们的驱动函数支持不同尺寸的 LTDC，同时可以实现 LTDC 横竖屏切换等重要功能，所以还是利大于弊的。

接下来，我们来看两个很重要的数组：

```
/* 根据不同的颜色格式,定义帧缓存数组 */
#if LTDC_PIXFORMAT == LTDC_PIXFORMAT_ARGB8888 || LTDC_PIXFORMAT ==
LTDC_PIXFORMAT_RGB888
    uint32_t ltdc_lcd_framebuf[1280][800]
                        __attribute__((at(LTDC_FRAME_BUF_ADDR)));
                        /* 定义最大屏分辨率时,LTDC 所需的帧缓存数组大小*/
#else
    uint16_t ltdc_lcd_framebuf[1280][800]
                __attribute__((at(LTDC_FRAME_BUF_ADDR)));
                /* 定义最大屏分辨率时,LTDC 所需的帧缓存数组大小 */
#endif
```

其中， ltdc_lcd_framebuf 的大小是 LTDC 一帧图像的显存大小， STM32H7 的 LTDC 最大可以支持 1280*800 的 RGB 屏，该数组根据我们选择的颜色格式（ ARGB8888/RGB565），自动确定数组类型。另外，我们采用 __attribute__ 关键字，将数组的地址定向到 LCD_FRAME_BUF_ADDR，它在 ltdc.h 里面定义，其值为： 0XC0000000，也就是 SDRAM 的首地址。这样，我们就把 ltdc_lcd_framebuf 数组定义到了 SDRAM 的首地址，大小为 800*1280*2 字节（ RGB565 格式时）。

而 g_ltdc_framebuf 则是 LTDC 的帧缓存数组指针， LTDC 支持 2 个层，所以数组大小为 2 。该指针为 32 位类型，必须指向对应的数组，才可以正常使用。在实际使用的时候，我们编写代码：

```
uint32_t *g_ltdc_framebuf[2];/* LTDC LCD 帧缓存数组指针,必须指向对应大小的内存区域 */
```

就将 LTDC 第一层的帧缓存，指向了 ltdc_lcd_framebuf 数组。往 ltdc_lcd_framebuf 里面写入不同的数据，就可以修改 RGBLCD 上面显示的内容。

接下来看画点函数：ltdc_draw_point，该函数代码如下：

```
/**
 * @brief       LTDC 画点函数
 * @param       x,y         : 写入坐标
 * @param       color       : 颜色值
 * @retval      无
 */
void ltdc_draw_point(uint16_t x, uint16_t y, uint32_t color)
{
#if LTDC_PIXFORMAT == LTDC_PIXFORMAT_ARGB8888 || LTDC_PIXFORMAT
                    == LTDC_PIXFORMAT_RGB888
```

```
    if (lcdltdc.dir)            /* 横屏 */
    {
        *(uint32_t*)((uint32_t)g_ltdc_framebuf[lcdltdc.activelayer] +
            lcdltdc.pixsize * (lcdltdc.pwidth * y + x)) = color;
    }
    else                        /* 竖屏 */
    {
        *(uint32_t*)((uint32_t)g_ltdc_framebuf[lcdltdc.activelayer] +
      lcdltdc.pixsize * (lcdltdc.pwidth *(lcdltdc.pheight - x- 1) + y))=color;
    }
#else
    if (lcdltdc.dir)            /* 横屏 */
    {
        *(uint16_t*)((uint32_t)g_ltdc_framebuf[lcdltdc.activelayer] +
            lcdltdc.pixsize * (lcdltdc.pwidth * y + x)) = color;
    }
    else                        /* 竖屏 */
    {
        *(uint16_t*)((uint32_t)g_ltdc_framebuf[lcdltdc.activelayer] +
    lcdltdc.pixsize * (lcdltdc.pwidth * (lcdltdc.pheight - x - 1) + y))=color;
    }
#endif
}
```

该函数实现往 RGBLCD 上面画点的功能，根据 LTDC_PIXFORMAT 定义的颜色格式以及横竖屏状态，执行不同的操作。 RGBLCD 的画点，实际上就是往指定坐标的显存里面写数据，以 7 寸 800*480 的屏幕， RGB565 格式，竖屏模式为例，画某个点对应到屏幕上面的关系如图 27.3.1 所示：

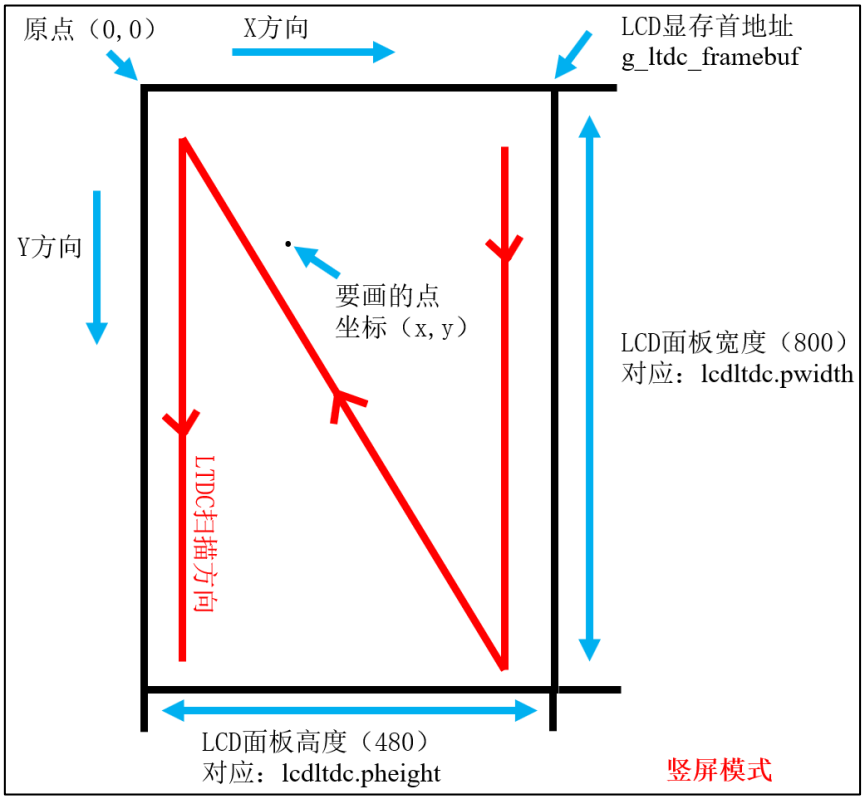


图 27.3.1 画点与 LCD 显存对应关系

注意图中的 LTDC 扫描方向（LTDC 在显存 g_ltdc_framebuf 里面读取 GRAM 数据的顺序也是这个方向），是从上到下，从右到左，而竖屏的时候，原点在左上角，所以有一个变换的过程，经过变换后的画点函数为：

```
*(uint16_t*)((uint32_t)g_ltdc_framebuf[lcdltdc.activelayer] + lcdltdc.pixsize *
            (lcdltdc.pwidth * (lcdltdc.pheight - x - 1) + y)) = color;
```

其中 g_ ltdc_framebuf，就是层帧缓冲的首地址； lcdltdc.activelayer 表示层编号： 0/1 代表

第 1/2 层； lcdltdc.pixsize 表示每个像素的字节数，对于 RGB565，它的值为 2； lcdltdc.pwidth 和 lcdltdc.pheight 为 LTDC 面板的宽度和高度， lcdltdc.pwidth=800， lcdltdc.pheight=480； x，y 就是要写入显存的坐标（也就是显示在 LTDC 上面的坐标）； color 为要写入的颜色值。

有画点函数，就有读点函数， LTDC 的读点函数代码如下：

```
/**
 * @brief       LTDC 读点函数
 * @param       x,y         : 读取点的坐标
 * @retval      返回值:颜色值
 */
uint32_t ltdc_read_point(uint16_t x, uint16_t y)
{
#if LTDC_PIXFORMAT == LTDC_PIXFORMAT_ARGB8888 || LTDC_PIXFORMAT
                      ==  LTDC_PIXFORMAT_RGB888
    if (lcdltdc.dir)    /* 横屏 */
    {
        return *(uint32_t*)((uint32_t)g_ltdc_framebuf[lcdltdc.activelayer] +
                lcdltdc.pixsize * (lcdltdc.pwidth * y + x));
    }
    else                /* 竖屏 */
    {
        return *(uint32_t*)((uint32_t)g_ltdc_framebuf[lcdltdc.activelayer] +
            lcdltdc.pixsize * (lcdltdc.pwidth * (lcdltdc.pheight - x- 1) + y));
    }
#else
    if (lcdltdc.dir)    /* 横屏 */
    {
        return *(uint16_t*)((uint32_t)g_ltdc_framebuf[lcdltdc.activelayer] +
                lcdltdc.pixsize * (lcdltdc.pwidth * y + x));
    }
    else                    /* 竖屏 */
    {
        return *(uint16_t*)((uint32_t)g_ltdc_framebuf[lcdltdc.activelayer] +
            lcdltdc.pixsize * (lcdltdc.pwidth * (lcdltdc.pheight - x -1) + y));
    }
#endif
}
```

画点函数和读点函数十分类似，只是过程反过来了而已，坐标的计算，也是在 g_ltdc_framebuf 数组内，根据坐标计算偏移量，完全和读点函数一模一样。

第三个介绍的函数是 LTDC 单色填充函数： ltdc_fill，该函数使用了 DMA2D 操作，使得填充速度大大加快，该函数代码如下：

```
/**
 * @brief       LTDC 填充矩形,DMA2D 填充
 * @note        有时候需要频繁的调用填充函数，所以为了速度，填充函数采用寄存器版本，
 * @note        不过下面有对应的库函数版本的代码。
 * @param       (sx,sy),(ex,ey)   : 填充矩形对角坐标,区域大小为:(ex-sx+1)*(ey-sy+1)
 * @param       color             : 要填充的颜色
 * @retval      无
 */
void ltdc_fill(uint16_t sx, uint16_t sy, uint16_t ex,
                            uint16_t ey, uint32_t color)
{
    uint32_t psx, psy, pex, pey;    /* 以 LCD 面板为基准的坐标系,不随横竖屏变化而变化 */
    uint32_t timeout = 0;
    uint16_t offline;
    uint32_t addr;
    /* 坐标系转换 */
    if (lcdltdc.dir)                /* 横屏 */
    {
        psx = sx;
        psy = sy;
        pex = ex;
```

```
        pey = ey;
    }
    else                                /* 竖屏 */
    {
        if (ex >= lcdltdc.pheight)
        {
           ex = lcdltdc.pheight - 1;  /* 限制范围 */
        }
        if (sx >= lcdltdc.pheight)
        {
          sx = lcdltdc.pheight - 1;  /* 限制范围 */
        }

        psx = sy;
        psy = lcdltdc.pheight - ex - 1;
        pex = ey;
        pey = lcdltdc.pheight - sx - 1;
    }
    offline = lcdltdc.pwidth - (pex - psx + 1);
    addr = ((uint32_t)g_ltdc_framebuf[lcdltdc.activelayer]
                          + lcdltdc.pixsize *(lcdltdc.pwidth * psy + psx));

    __HAL_RCC_DMA2D_CLK_ENABLE();      /* 使能 DM2D 时钟 */
    DMA2D->CR &= ~(DMA2D_CR_START);    /* 先停止 DMA2D */
    DMA2D->CR = DMA2D_R2M;             /* 寄存器到存储器模式 */
    DMA2D->OPFCCR = LCD_PIXFORMAT;     /* 设置颜色格式 */
    DMA2D->OOR = offline;              /* 设置行偏移  */

    DMA2D->OMAR = addr;                /* 输出存储器地址 */
    DMA2D->NLR = (pey - psy + 1) | ((pex - psx + 1) << 16);  /* 设定行数寄存器 */
    DMA2D->OCOLR = color;              /* 设定输出颜色寄存器 */
    DMA2D->CR |= DMA2D_CR_START;       /* 启动 DMA2D */
    while((DMA2D->ISR & (DMA2D_FLAG_TC)) == 0)     /* 等待传输完成 */
    {
        timeout++;
        if(timeout > 0X1FFFFF)break;     /* 超时退出 */
    }
    DMA2D->IFCR |= DMA2D_FLAG_TC;        /* 清除传输完成标志 */
}
```

该函数使用 DMA2D 完成矩形色块的填充，其操作步骤，就是按 27.1.3 节最后的介绍来进行的，我们这了就不多说了，详见 27.1.3 节。另外，还有一个 LTDC 彩色填充函数，也是采用的 DMA2D 填充，函数名为 ltdc_color_fill，该函数代码同 ltdc_fill 非常接近，我们这里就不介绍了，请大家参考本例程源码。

第四个介绍的函数是清屏函数： ltdc_clear，该函数代码如下：

```
/**
 * @brief       LTCD 清屏
 * @param       color     : 颜色值
 * @retval      无
 */
void ltdc_clear(uint32_t color)
{
    ltdc_fill(0, 0, lcdltdc.width - 1, lcdltdc.height - 1, color);
}
```

该函数代码非常简单，清屏操作调用了我们前面介绍的 ltdc_fill 函数，采用 DMA2D 完成对 LTDC 的清屏，提高了清屏速度。

第五个介绍的函数是 LTDC_CLK 频率设置函数： ltdc_clk_set，该函数代码如下：

```
/**
 * @brief       LTDC 时钟(Fdclk)设置函数
 * @param       pll3n     : PLL3 倍频系数(PLL 倍频),           取值范围:4~512.
 * @param       pll3m     : PLL3 预分频系数(进 PLL 之前的分频), 取值范围:2~63.
```

```
 * @param       pll3r     : PLL3 的 r 分频系数(PLL 之后的分频),    取值范围:1~128.
 *
 * @note        Fvco = Fs * (pll3n / pll3m);
 *              Fr = Fvco / pll3r = Fs * (pll3n / (pll3m * pll3r));
 *              Fdclk = Fr;
 *              其中:
 *              Fvco: VCO 频率
 *              Fr: PLL3 的 r 分频输出时钟频率
 *              Fs: PLL3 输入时钟频率,可以是 HSI,CSI,HSE 等.
 *
 *              LTDC,我们一般设置 pll3m=25,pll3n=300,这样,可以得到 Fvco=300Mhz
 *              然后,只需要通过修改 pll3r, 来匹配不同的液晶屏时序即可.

 *              假设:外部晶振为 25M,pllm =25 的时候,Fs=25Mhzpllm 分频后频率为 1Mhz.
 *              例如: 要得到 33M 的 LTDC 时钟, 则可以设置: pll3n = 300, pllm = 25,
                                                          pll3r = 9
 *              Fdclk= ((25 / 25) * 300) / 9 = 33 Mhz
 * @retval      0, 成功;
 *              其他, 失败;
 */
uint8_t ltdc_clk_set(uint32_t pll3n, uint32_t pll3m, uint32_t pll3r)
{
    RCC_PeriphCLKInitTypeDef periphclk_initure;

    /* LTDC 输出像素时钟，需要根据自己所使用的 LCD 数据手册来配置！ */
    periphclk_initure.PeriphClockSelection=RCC_PERIPHCLK_LTDC;    /* LTDC 时钟 */
    periphclk_initure.PLL3.PLL3M = pll3m;
    periphclk_initure.PLL3.PLL3N = pll3n;
    periphclk_initure.PLL3.PLL3P = 2;
    periphclk_initure.PLL3.PLL3Q = 2;
    periphclk_initure.PLL3.PLL3R = pll3r;
    if(HAL_RCCEx_PeriphCLKConfig(&periphclk_initure) == HAL_OK)/* 配置像素时钟 */
    {
        return 0;    /* 成功 */
    }
    else return 1;  /* 失败 */
}
```

该函数完成对 PLL3 的配置，最终控制输出 LTDC_CLK 的频率， LTDC_CLK 的频率设置方法，我们在 27.1.2 节进行了介绍，请大家参考前面的介绍进行学习。

第六个介绍的函数是 LTDC 层参数设置函数： ltdc_layer_parameter_config，该函数代码如下：

```
/**
 * @brief       LTDC,基本参数设置.
 * @note        此函数,必须在 LTDC_Layer_Window_Config 之前设置.
 * @param       layerx:层值,0/1.
 * @param       bufaddr:层颜色帧缓存起始地址
 * @param       pixformat:颜色格式.0,ARGB8888;1,RGB888;2,RGB565;3,
                                  ARGB1555;4,ARGB4444;5,L8;6;AL44;7;AL88
 * @param       alpha:层颜色 Alpha 值,0,全透明;255,不透明
 * @param       alpha0:默认颜色 Alpha 值,0,全透明;255,不透明
 * @param       bfac1:混合系数 1,4(100),恒定的 Alpha;6(101),像素 Alpha*恒定 Alpha
 * @param       bfac2:混合系数 2,5(101),恒定的 Alpha;7(111),像素 Alpha*恒定 Alpha
 * @param       bkcolor:层默认颜色,32 位,低 24 位有效,RGB888 格式
 * @retval      返回值:无
*/
void ltdc_layer_parameter_config(uint8_t layerx, uint32_t bufaddr, uint8_t
                                 pixformat, uint8_t alpha, uint8_t alpha0,
                                 uint8_t bfac1, uint8_t bfac2, uint32_t bkcolor)
{
    LTDC_LayerCfgTypeDef playercfg;
```

```
    playercfg.WindowX0 = 0;                 /* 窗口起始 X 坐标 */
    playercfg.WindowY0 = 0;                 /* 窗口起始 Y 坐标 */
    playercfg.WindowX1 = lcdltdc.pwidth;    /* 窗口终止 X 坐标 */
    playercfg.WindowY1 = lcdltdc.pheight;   /* 窗口终止 Y 坐标 */
    playercfg.PixelFormat = pixformat;      /* 像素格式 */
    playercfg.Alpha = alpha;                /* Alpha 值设置，0~255,255 为完全不透明 */
    playercfg.Alpha0 = alpha0;              /* 默认 Alpha 值 */
    playercfg.BlendingFactor1 = (uint32_t)bfac1 << 8;       /* 设置层混合系数 */
    playercfg.BlendingFactor2 = (uint32_t)bfac2 << 8;       /* 设置层混合系数 */
    playercfg.FBStartAdress = bufaddr;              /* 设置层颜色帧缓存起始地址 */
    playercfg.ImageWidth = lcdltdc.pwidth;          /* 设置颜色帧缓冲区的宽度 */
    playercfg.ImageHeight = lcdltdc.pheight;        /* 设置颜色帧缓冲区的高度 */
    /* 背景颜色红色部分 */
    playercfg.Backcolor.Red = (uint8_t)(bkcolor & 0X00FF0000) >> 16;
    /* 背景颜色绿色部分 */
    playercfg.Backcolor.Green = (uint8_t)(bkcolor & 0X0000FF00) >> 8;
    /* 背景颜色蓝色部分 */
    playercfg.Backcolor.Blue = (uint8_t)bkcolor & 0X000000FF;
    HAL_LTDC_ConfigLayer(&g_ltdc_handle, &playercfg, layerx);/* 设置所选中的层 */
}
```

该函数中主要调用 HAL 库函数 HAL_LTDC_ConfigLayer 设置 LTDC 层的基本参数，包括：层帧缓冲区首地址、颜色格式、 Alpha 值、混合系数和层默认颜色等，这些参数都需要根据大家的实际需要来进行设置。

第七个介绍的函数是 LTDC 层窗口设置函数： ltdc_layer_window_config，该函数代码如下：

```
/**
 * @brief       LTDC 层窗口设置, 窗口以 LCD 面板坐标系为基准
 * @note        此函数必须在 ltdc_layer_parameter_config 之后再设置.
                另外,当设置的窗口值不等于面板的尺寸时,GRAM 的操作(读/写点函数),
                也要根据窗口的宽高来进行修改,否则显示不正常(本例程就未做修改).
 * @param       layerx      : 0,第一层; 1,第二层;
 * @param       sx, sy      : 起始坐标
 * @param       width,height: 宽度和高度
 * @retval      无
 */
void ltdc_layer_window_config(uint8_t layerx, uint16_t sx, uint16_t sy,
                                          uint16_t width, uint16_t height)
{
 HAL_LTDC_SetWindowPosition(&g_ltdc_handle, sx, sy, layerx);/* 设置窗口的位置 */
 HAL_LTDC_SetWindowSize(&g_ltdc_handle, width, height, layerx);/* 设置窗口大小 */
}
```

该函数依次调用 HAL 库 LTDC 窗口位置设置函数 HAL_LTDC_SetWindowPositio 和窗口大小设置函数 HAL_LTDC_SetWindowSizel 来控制 LTDC 在某一层（ 1/2）上面的开窗操作，这个我们在 27.1.2 节也介绍过了，请参考前面的内容进行学习。这里我们一般设置层窗口为整个 LCD 的分辨率，也就是不进行开窗操作。注意：此函数必须在 ltdc_layer_parameter_config 之后再设置。另外， **当设置的窗口值不等于面板的尺寸时，对层 GRAM 的操作(读/写点函数)，也要根据层窗口的宽高来进行修改，否则显示不正常（ 本例程就未做修改）。**

第八个介绍的函数是 LTDC LCD ID 获取函数： ltdc_panelid_read，该函数代码如下：

```
/**
 * @brief       LTDC 读取面板 ID
 * @note        利用 LCD RGB 线的最高位(R7,G7,B7)来识别面板 ID
 *              PG6 = R7(M0); PI2 = G7(M1); PI7 = B7(M2);
 *              M2:M1:M0
 *              0 :0 :0     4.3 寸 480*272  RGB 屏,ID = 0X4342
 *              0 :0 :1     7   寸 800*480  RGB 屏,ID = 0X7084
 *              0 :1 :0     7   寸 1024*600 RGB 屏,ID = 0X7016
 *              0 :1 :1     7   寸 1280*800 RGB 屏,ID = 0X7018
 *              1 :0 :0     4.3 寸 800*480  RGB 屏,ID = 0X4384
 *              1 :0 :1     10.1 寸 1280*800 RGB 屏,ID = 0X1018
```

```
 * @param       无
 * @retval      0, 非法;
 *              其他, LCD ID
 */
uint16_t ltdc_panelid_read(void)
{
    uint8_t idx = 0;

    GPIO_InitTypeDef gpio_init_struct;
    __HAL_RCC_GPIOG_CLK_ENABLE();                       /* 使能 GPIOG 时钟 */
    __HAL_RCC_GPIOI_CLK_ENABLE();                       /* 使能 GPIOI 时钟 */

    gpio_init_struct.Pin = GPIO_PIN_6;                  /* PG6 */
    gpio_init_struct.Mode = GPIO_MODE_INPUT;            /* 输入 */
    gpio_init_struct.Pull = GPIO_PULLUP;                /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_HIGH;           /* 高速 */
    HAL_GPIO_Init(GPIOG, &gpio_init_struct);            /* 初始化 */

    gpio_init_struct.Pin = GPIO_PIN_2 | GPIO_PIN_7;     /* PI2,7 */
    HAL_GPIO_Init(GPIOI, &gpio_init_struct);            /* 初始化 */

    delay_us(10);
    idx = (uint8_t)HAL_GPIO_ReadPin(GPIOG, GPIO_PIN_6);      /* 读取 M0 */
    idx|= (uint8_t)HAL_GPIO_ReadPin(GPIOI, GPIO_PIN_2) << 1;/* 读取 M1 */
    idx|= (uint8_t)HAL_GPIO_ReadPin(GPIOI, GPIO_PIN_7) << 2;/* 读取 M2 */
    switch (idx)
    {
        case 0 : return 0X4342;    /* 4.3 寸屏,480*272 分辨率 */
        case 1 : return 0X7084;    /* 7 寸屏,800*480 分辨率 */
        case 2 : return 0X7016;    /* 7 寸屏,1024*600 分辨率 */
        case 3 : return 0X7018;    /* 7 寸屏,1280*800 分辨率 */
        case 4 : return 0X4384;    /* 4.3 寸屏,800*480 分辨率 */
        case 5 : return 0X1018;    /* 10.1 寸屏,1280*800 分辨率 */
        default : return 0;
    }
}
```

因为 RGBLCD 屏并没有读的功能，所以，一般情况，外接 RGB 屏的时候， MCU 是无法获取屏幕的任何信息的。但是 正点原子 在 RGBLCD 模块上面，利用数据线（ R7/G7/B7）做了一个巧妙的设计，可以让 MCU 读取到 RGBLCD 模块的 ID，从而执行不同的初始化，实现对不同分辨率的 RGBLCD 模块的兼容。详细原理见：本章 27.1.1 节，第（3）部分： 正点原子 RGBLCD 模块的说明。

ltdc_panelid_read 函数，就是用这样的方法来读取 M[2:0]的值，并将结果（转换成屏型号了）返回给上一层。

最后要介绍的函数是 LTDC 初始化函数： ltdc_init，该函数的简化代码如下：

```
/**
 * @brief       LTDC 初始化函数
 * @param       无
 * @retval      无
 */
void ltdc_init(void)
{
    uint16_t lcdid = 0;

    lcdid = ltdc_panelid_read();                /* 读取 LCD 面板 ID */
    if(lcdid == 0X4342)
    {
        lcdltdc.pwidth = 480;                   /* 面板宽度,单位:像素 */
        lcdltdc.pheight = 272;                  /* 面板高度,单位:像素 */
        lcdltdc.hsw = 1;                        /* 水平同步宽度 */
        lcdltdc.vsw = 1;                        /* 垂直同步宽度 */
```

```
        lcdltdc.hbp = 40;                       /* 水平后廊 */
        lcdltdc.vbp = 8;                        /* 垂直后廊 */
        lcdltdc.hfp = 5;                        /* 水平前廊 */
        lcdltdc.vfp = 8;                        /* 垂直前廊 */
        ltdc_clk_set(300, 25, 33);              /* 设置像素时钟 9Mhz */
        /* 其他参数待定. */
    }
    else if(lcdid == 0X7084)
    {
        lcdltdc.pwidth = 800;                       /* 面板宽度,单位:像素 */
        lcdltdc.pheight = 480;                      /* 面板高度,单位:像素 */
        lcdltdc.hsw = 1;                            /* 水平同步宽度 */
        lcdltdc.vsw = 1;                            /* 垂直同步宽度 */
        lcdltdc.hbp = 46;                           /* 水平后廊 */
        lcdltdc.vbp = 23;                           /* 垂直后廊 */
        lcdltdc.hfp = 210;                          /* 水平前廊 */
        lcdltdc.vfp = 22;                           /* 垂直前廊 */
        ltdc_clk_set(300, 25, 9);
        /* 设置像素时钟 33M(如果开双显,需要降低 DCLK 到:
           18.75Mhz  300/4/4,才会比较好) */
    }
    else if(lcdid == 0X7016)
    {
        lcdltdc.pwidth = 1024;                      /* 面板宽度,单位:像素 */
        lcdltdc.pheight = 600;                      /* 面板高度,单位:像素 */
        lcdltdc.hsw = 20;                           /* 水平同步宽度 */
        lcdltdc.vsw = 3;                            /* 垂直同步宽度 */
        lcdltdc.hbp = 140;                          /* 水平后廊 */
        lcdltdc.vbp = 20;                           /* 垂直后廊 */
        lcdltdc.hfp = 160;                          /* 水平前廊 */
        lcdltdc.vfp = 12;                           /* 垂直前廊 */
        ltdc_clk_set(300, 25, 6);                   /* 设置像素时钟  40Mhz  */
        /* 其他参数待定.*/
    }
    else if(lcdid == 0X7018)
    {
        lcdltdc.pwidth = 1280;                  /* 面板宽度,单位:像素 */
        lcdltdc.pheight = 800;                  /* 面板高度,单位:像素 */
        /* 其他参数待定.*/
    }
    else if(lcdid == 0X4384)
    {
        lcdltdc.pwidth = 800;                   /* 面板宽度,单位:像素 */
        lcdltdc.pheight = 480;                  /* 面板高度,单位:像素 */
        lcdltdc.hbp = 88;                       /* 水平后廊 */
        lcdltdc.hfp = 40;                       /* 水平前廊 */
        lcdltdc.hsw = 48;                       /* 水平同步宽度 */
        lcdltdc.vbp = 32;                       /* 垂直后廊 */
        lcdltdc.vfp = 13;                       /* 垂直前廊 */
        lcdltdc.vsw = 3;                        /* 垂直同步宽度 */
        ltdc_clk_set(300, 25, 9);               /* 设置像素时钟 33M */
        /* 其他参数待定. */
    }
    else if(lcdid == 0X1018)                /* 10.1 寸 1280*800 RGB 屏 */
    {
        lcdltdc.pwidth = 1280;              /* 面板宽度,单位:像素 */
        lcdltdc.pheight = 800;              /* 面板高度,单位:像素 */
        lcdltdc.hbp = 140;                  /* 水平后廊 */
        lcdltdc.hfp = 10;                   /* 水平前廊 */
        lcdltdc.hsw = 10;                   /* 水平同步宽度 */
```

```
        lcdltdc.vbp = 10;                   /* 垂直后廊 */
        lcdltdc.vfp = 10;                   /* 垂直前廊 */
        lcdltdc.vsw = 3;                    /* 垂直同步宽度 */
        ltdc_clk_set(300, 25, 6);           /* 设置像素时钟  45Mhz */
    }
    lcddev.width = lcdltdc.pwidth;
    lcddev.height = lcdltdc.pheight;

#if LTDC_PIXFORMAT == LTDC_PIXFORMAT_ARGB8888 || LTDC_PIXFORMAT
                    == LTDc_PIXFORMAT_RGB888
    g_ltdc_framebuf[0] = (uint32_t*) &ltdc_lcd_framebuf;
    lcdltdc.pixsize = 4;                    /* 每个像素占 4 个字节 */
#else
    lcdltdc.pixsize = 2;                    /* 每个像素占 2 个字节 */
    g_ltdc_framebuf[0] = (uint32_t*)&ltdc_lcd_framebuf;
#endif
    /* LTDC 配置 */
    g_ltdc_handle.Instance = LTDC;
    g_ltdc_handle.Init.HSPolarity = LTDC_HSPOLARITY_AL;      /* 水平同步极性 */
    g_ltdc_handle.Init.VSPolarity = LTDC_VSPOLARITY_AL;      /* 垂直同步极性 */
    g_ltdc_handle.Init.DEPolarity = LTDC_DEPOLARITY_AL;      /* 数据使能极性 */
    g_ltdc_handle.State = HAL_LTDC_STATE_RESET;

    if (lcdid == 0X1018)
    {
        g_ltdc_handle.Init.PCPolarity = LTDC_PCPOLARITY_IIPC;/* 像素时钟极性 */
    }
    else
    {
        g_ltdc_handle.Init.PCPolarity = LTDC_PCPOLARITY_IPC; /* 像素时钟极性 */
    }
    g_ltdc_handle.Init.HorizontalSync = lcdltdc.hsw - 1;     /* 水平同步宽度 */
    g_ltdc_handle.Init.VerticalSync = lcdltdc.vsw - 1;       /* 垂直同步宽度 */
    g_ltdc_handle.Init.AccumulatedHBP = lcdltdc.hsw + lcdltdc.hbp - 1;
                                                        /* 水平同步后沿宽度 */
    g_ltdc_handle.Init.AccumulatedVBP = lcdltdc.vsw + lcdltdc.vbp - 1;
                                                        /* 垂直同步后沿高度 */
    g_ltdc_handle.Init.AccumulatedActiveW = lcdltdc.hsw + lcdltdc.hbp +
                                            lcdltdc.pwidth - 1; /* 有效宽度 */
    g_ltdc_handle.Init.AccumulatedActiveH = lcdltdc.vsw + lcdltdc.vbp +
                                            lcdltdc.pheight - 1;/* 有效高度 */
    g_ltdc_handle.Init.TotalWidth = lcdltdc.hsw + lcdltdc.hbp + lcdltdc.pwidth
                                            + lcdltdc.hfp - 1;  /* 总宽度 */
    g_ltdc_handle.Init.TotalHeigh = lcdltdc.vsw + lcdltdc.vbp + lcdltdc.pheight
                                            + lcdltdc.vfp - 1;  /* 总高度 */
    g_ltdc_handle.Init.Backcolor.Red = 0;           /* 屏幕背景层红色部分 */
    g_ltdc_handle.Init.Backcolor.Green = 0;         /* 屏幕背景层绿色部分 */
    g_ltdc_handle.Init.Backcolor.Blue = 0;          /* 屏幕背景色蓝色部分 */
    HAL_LTDC_Init(&g_ltdc_handle);
    /* 层配置 */
    ltdc_layer_parameter_config(0,(uint32_t)g_ltdc_framebuf[0], LTDC_PIXFORMAT,
                                255, 0, 6, 7, 0X000000);/* 层参数配置 */
    ltdc_layer_window_config(0, 0, 0,lcdltdc.pwidth, lcdltdc.pheight);
    /* 层窗口配置,以 LCD 面板坐标系为基准,不要随便修改! */
    ltdc_display_dir(0);            /* 默认竖屏 */
    ltdc_select_layer(0);           /* 选择第 1 层 */
    LTDC_BL(1);                     /* 点亮背光 */
    ltdc_clear(0XFFFFFFFF);         /* 清屏 */
}
```

ltdc_init 的初始化步骤，是按照 27.1.2 节最后介绍的步骤来进行的， 该函数先读取 RGBLCD 的 ID， 然后根据不同的 RGBLCD 型号，执行不同的面板参数初始化， 然后调用

HAL_LTDC_Init 函数来设置 RGBLCD 的相关参数并使能 LTDC， 最后配置层参数和层窗口完成对 LTDC 的初始化。注意，代码里面的 lcdltdc.hsw、 lcdltdc.vsw、 lcdltdc.hbp 等参数的值，均是来自对应 RGBLCD 屏的数据手册，其中 lcdid=0X7084 的配置参数，来自：AT070TN92.pdf。

以上，就是 ltdc 驱动部分的代码，因为阿波罗 STM32H743 开发板还有 MCU 屏接口，为了同时兼容 MCU 屏和 RGB 屏，我们对第 25 章介绍的 lcd.c 部分代码做了小改，添加对 RGB 屏的支持，由于篇幅所限，这里我们只挑几个重点的函数给大家介绍下。

首先读点函数，改为了：

```
/**
 * @brief       读取个某点的颜色值
 * @param       x,y:坐标
 * @retval      此点的颜色(32 位颜色,方便兼容 LTDC)
 */
uint32_t lcd_read_point(uint16_t x, uint16_t y)
{
    uint16_t r = 0, g = 0, b = 0;

    if (x >= lcddev.width || y >= lcddev.height)
    {
        return 0;                       /* 超过了范围,直接返回 */
    }
    if(lcdltdc.pwidth != 0)         /* 如果是 RGB 屏 */
    {
        return ltdc_read_point(x, y);
    }
    ……/* 省略部分代码 */
}
```

当 lcdltdc.pwidth!=0 的时候，说明接入的是 RGB 屏，所以调用 ltdc_read_point 函数，实现读点操作，其他情况，说明是 MCU 屏，执行 MCU 屏的读点操作（代码省略）。

然后是画点函数，改为了：

```
/**
 * @brief       画点
 * @param       x,y: 坐标
 * @param       color: 点的颜色(32 位颜色,方便兼容 LTDC)
 * @retval      无
 */
void lcd_draw_point(uint16_t x, uint16_t y, uint32_t color)
{
    if(lcdltdc.pwidth != 0)         /* 如果是 RGB 屏 */
    {
        ltdc_draw_point(x, y, color);
    }
    else
    {
       ……/* 省略部分代码 */
    }
}
```

当 lcdltdc.pwidth!=0 的时候，说明接入的是 RGB 屏，所以调用 ltdc_draw_point 函数，实现画点操作，其他情况，说明是 MCU 屏，执行 MCU 屏的画点操作（代码省略）。

最后，是 LCD 初始化函数，改为：

```
/**
 * @brief       初始化 LCD
 * @note        该初始化函数可以初始化各种型号的 LCD(详见本.c 文件最前面的描述)
 * @param       无
 * @retval      无
 */
void lcd_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;
```

```
    FMC_NORSRAM_TimingTypeDef fmc_read_handle;
    FMC_NORSRAM_TimingTypeDef fmc_write_handle;

    lcddev.id = ltdc_panelid_read();  /* 检查是否有 RGB 屏接入 */
    if (lcddev.id != 0)
    {
        ltdc_init();                        /* ID 非零,说明有 RGB 屏接入 */
    }
    else
    ……/* 省略部分代码 */
}
```

首先通过 ltdc_panelid_read 函数，读取 RGBLCD 模块的 ID 值，如果合法，则说明接入了 RGB 屏，调用 ltdc_init 函数，完成对 LTDC 的初始化。否则的话，执行 MCU 屏的初始化。

在 lcd.c 里面，其他还有一些函数进行了兼容 RGB 屏的修改，这里我们就不一一列举了，请大家参考本例程源码。在完成修改后，我们的例程就可以同时兼容 MCU 屏和 RGB 屏了，且 RGB 屏的优先级较高。

2. main.c 代码

main 函数代码如下：

```
int main(void)
{
    uint8_t x = 0;
    uint8_t lcd_id[12];
    sys_cache_enable();
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4); /* 设置时钟,400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 初始化 USART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    key_init();                             /* 初始化 KEY */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */

    /* 将 LCD ID 打印到 lcd_id 数组 */
    sprintf((char *)lcd_id, "LCD ID:%04X", lcddev.id);

    while (1)
    {
        switch (x)
        {
            case 0:lcd_clear(WHITE); break;
            case 1: lcd_clear(BLACK); break;
            ……/* 此处省略部分代码 */
            case 11: lcd_clear(BROWN); break;
        }

        lcd_show_string(10, 40, 240, 32, 32, "STM32", RED);
        lcd_show_string(10, 80, 240, 24, 24, "LTDC TEST", RED);
        lcd_show_string(10, 110, 240, 16, 16, "ATOM@ALIENTEK", RED);
        lcd_show_string(10, 130, 240, 16, 16, (char *)lcd_id, RED);
       /* 显示 LCD ID */

        x++;
        if (x == 12) x = 0;
        LED0_TOGGLE();              /*红灯闪烁*/
        delay_ms(1000);
    }
}
```

该部分代码与第 25 章几乎一摸一样， 显示一些固定的字符，字体大小包括 32*16、24*12、

16*8 和 12*6 等四种，同时显示 LCD 的型号，然后不停的切换背景颜色，每 1s 切换一次。而 LED0 也会不停的闪烁，指示程序已经在运行了。 其中我们用到一个 sprintf 的函数，该函数用法同 printf，只是 sprintf 把打印内容输出到指定的内存区间上， sprintf 的详细用法，请百度。

另外**特别注意**： usart_init 函数，不能去掉，因为在 lcd_init 函数里面调用了 printf，所以一旦你去掉这个初始化，就会死机了!实际上，只要你的代码有用到 printf，就必须初始化串口，否则都会死机，即停在 usart.c 里面的 fputc 函数，出不来。

在编译通过之后，我们开始下载验证代码

27.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。同时，可以看到 RGBLCD 模块的显示如图 27.4.1 所示：

图 27.4.1 RGBLCD 显示效果图

我们可以看到屏幕的背景是不停切换的，同时 DS0 不停的闪烁，证明我们的代码被正确的执行了，达到了我们预期的目的。 最后，需要注意的是：本例程兼容 MCU 屏，所以，当插入 MCU 屏的时候（不插 RGB 屏），也可以显示同样的结果。

第二十八章 USMART 调试组件实验

本章，我们将向大家介绍一个十分重要的辅助调试工具：USMART 调试组件。该组件由正点原子开发提供，功能类似 linux 的 shell（RTT 的 finsh 也属于此类）。USMART 最主要的功能就是通过串口调用单片机里面的函数，并执行，对我们调试代码是很有帮助的。

本章分为如下几个小节：

28.1 USMART 调试组件简介
28.2 硬件设计
28.3 程序设计
28.4 下载验证

28.1 USMART 调试组件简介

USMART 是由正点原子开发的一个灵巧的串口调试互交组件，通过它你可以通过串口助手调用程序里面的任何函数，并执行。因此，你可以随意更改函数的输入参数（支持数字（10/16 进制，支持负数)、字符串、函数入口地址等作为参数），单个函数最多支持 10 个输入参数，并支持函数返回值显示，目前最新版本为 V3.5。

USMART 的特点如下：

1， 可以调用绝大部分用户直接编写的函数。
2， 资源占用极少（最少情况：FLASH:4K；SRAM:72B）。
3， 支持参数类型多（数字（包含 10/16 进制，支持负数）、字符串、函数指针等）。
4， 支持函数返回值显示。
5， 支持参数及返回值格式设置。
6， 支持函数执行时间计算（V3.1 及以后的版本新特性）。
7， 支持 AC6 编译器。

有了 USMART，你可以轻易的修改函数参数、查看函数运行结果，从而快速解决问题。比如你调试一个摄像头模块，需要修改其中的几个参数来得到最佳的效果，普通的做法：写函数→修改参数→下载→看结果→不满意→修改参数→下载→看结果→不满意….不停的循环，直到满意为止。这样做很麻烦不说，单片机也是有寿命的啊，老这样不停的刷，很折寿的。而利用 USMART，则只需要在串口调试助手里面输入函数及参数，然后直接串口发送给单片机，就执行了一次参数调整，不满意的话，你在串口调试助手修改参数在发送就可以了，直到你满意为止。这样，修改参数十分方便，不需要编译、不需要下载、不会让单片机折寿。

USMART 支持的参数类型基本满足任何调试了，支持的类型有：10 或者 16 进制数字、字符串指针（如果该参数是用作参数返回的话，可能会有问题！）、函数指针等。因此绝大部分函数，可以直接被 USMART 调用，对于不能直接调用的，你只需要重写一个函数，把影响调用的参数去掉即可，这个重写后的函数，即可以被 USMART 调用了。

USMART 的实现流程简单概括就是：第一步，添加需要调用的函数（在 usmart_config.c 里面的 usmart_nametab 数组里面添加）；第二步，初始化串口；第三步，初始化 USMART（通过 usmart_init 函数实现）；第四步，轮询 usmart_scan 函数，处理串口数据。

经过以上简单介绍，我们对 USMART 有了个大概了解，接下来我们来简单介绍下 USMART 组件的移植。USMART 组件总共包含 8 文件如图 28.1.1 所示：

图 28.1.1 USMART 组件代码

USMART 每个文件的作用如表 28.1.1 所示：

USMART 文件	说明
usmart.c	USMART 核心文件，用于处理命令以及对外交互
usmart.h	USMART 核心文件头文件，定义结构体，宏定义、函数声明等
usmart_str.c	USMART 字符串处理文件，用于字符串转换、参数获取等
usmart_str.h	USMART 字符串处理头文件，用于函数声明
usmart_port.c	USMART 移植文件，用于 USMART 移植
usmart_port.h	USMART 移植头文件，定义用户配置参数、宏定义、函数声明等
usmart_config.c	USMART 函数管理文件，用于添加用户需要 USMART 管理的函数
readme.txt	USMART 介绍文件，用于说明 USMART 版本和功能

表 28.1.1 USMART 文件介绍

经过以上简单介绍，我们对 USMART 有了个大概了解，接下来我们将在下一小节给大家介绍 USMART 组件的移植和使用。

28.2 硬件设计

1. 例程功能

本实验通过 usmart 调用单片机里面的函数，实现对 LCD 显示和 LED 以及延时的控制。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
LED1：LED1 - PB0
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)

28.3 程序设计

28.3.1 程序流程图

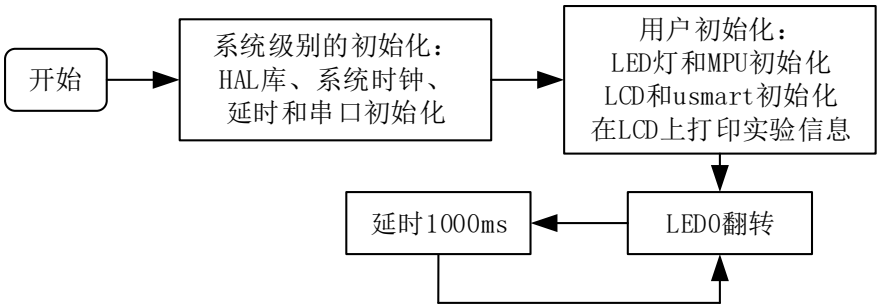


图 28.3.1.1 USMART 调试组件实验程序流程图

28.3.2 程序解析

1. USMART 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。USMART 驱动源码包括七个文件：usmart.h、usmart_port.c、usmart_port.h、usmart_str.c、usmart.c、usmart_config.c 和 usmart_str.h。

要使用 USMART，我们先得进行代码移植，USMART 的移植非常简单，我们只需要修改 usmart_port.c 里面的 5 个函数即可完成移植。

第一个函数，USMART 输入数据流获取函数，该函数的实现代码如下：

```
/**
 * @brief       获取输入数据流(字符串)
 * @note        USMART 通过解析该函数返回的字符串以获取函数名及参数等信息
 * @param       无
 * @retval
 *   @arg       0,  没有接收到数据
 *   @arg       其他,数据流首地址(不能是 0)
 */
char *usmart_get_input_string(void)
{
    uint8_t len;
    char *pbuf = 0;
    if (g_usart_rx_sta & 0x8000)            /* 串口接收完成? */
    {
        len = g_usart_rx_sta & 0x3fff;      /* 得到此次接收到的数据长度 */
        g_usart_rx_buf[len] = '\0';         /* 在末尾加入结束符. */
        pbuf = (char*)g_usart_rx_buf;
        g_usart_rx_sta = 0;                 /* 开启下一次接收 */
    }
    return pbuf;
}
```

该函数通过 SYSTEM 文件夹默认的串口接收来实现输入数据流获取。SYSTEM 文件夹里面的串口接收函数，最大可以一次接收 200 字节，用于从串口接收函数名和参数等。大家如果在其他平台移植，请参考 SYSTEM 文件夹串口接收的实现方式进行移植（详细接收原理请参考：串口通信实验）。

第二个是 usmart_timx_init 函数，该函数的实现代码如下：

```
/**
 * @brief       定时器初始化函数
 * @param       arr:自动重装载值
 * @param       psc:定时器分频系数
 * @retval      无
 */
void usmart_timx_init(uint16_t arr, uint16_t psc)
{
    USMART_TIMX_CLK_ENABLE();
    g_timx_usmart_handle.Instance = USMART_TIMX;/* 通用定时器 4 */
    g_timx_usmart_handle.Init.Prescaler = psc;  /* 分频系数 */
    g_timx_usmart_handle.Init.CounterMode = TIM_COUNTERMODE_UP;  /* 向上计数器 */
    g_timx_usmart_handle.Init.Period = arr;     /* 自动装载值 */
    g_timx_usmart_handle.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    HAL_TIM_Base_Init(&g_timx_usmart_handle);
    HAL_TIM_Base_Start_IT(&g_timx_usmart_handle);/* 使能定时器和定时器中断 */
    HAL_NVIC_SetPriority(USMART_TIMX_IRQn,3,3);  /* 抢占优先级 3，子优先级 3 */
    HAL_NVIC_EnableIRQ(USMART_TIMX_IRQn);        /* 开启 TIM 中断 */
}
```

该函数只有在：USMART_ENTIMX_SCAN 值为 1 时，才需要实现，用于定时器初始化，利用定时器完成对 usmart_scan 函数的周期性调用，并实现函数运行时间计时（runtime）功能。该函数里面的：USMART_TIMX_CLK_ENABLE、USMART_TIMX 和 USMART_TIMX_IRQn

等定时器相关宏定义我们在 usmart_prot.h 里面定义，方便大家修改。本实验我们用的 TIM4。

注意：usmart_timx_init 函数是在 usmart.c 里面，被 usmart_init 调用，usmart_init 源码如下：

```
/**
 * @brief       初始化 USMART
 * @param       tclk: 定时器的工作频率(单位:Mhz)
 * @retval      无
 */
void usmart_init(uint16_t tclk)
{
#if USMART_ENTIMX_SCAN == 1
    usmart_timx_init(1000, tclk * 100 - 1);
#endif
    usmart_dev.sptype = 1;      /* 十六进制显示参数 */
}
```

该函数有一个参数 tclk，就是用于定时器初始化。注意：这里的 tclk 是指所选 TIM 的时钟频率，而非系统主频！这里我们用的 TIM4，对于 STM32H743 来说，TIM4 的时钟源来自 2 倍的 APB1，频率为：200Mhz。

另外，USMART_ENTIMX_SCAN 是在 usmart_port.h 里面定义的一个是否使能定时器中断扫描的宏定义。如果为 1，就初始化定时器中断，并在中断里面调用 usmart_scan 函数。如果为 0，那么需要用户需要自行间隔一定时间（100ms 左右为宜）调用一次 usmart_scan 函数，以实现串口数据处理。**注意：如果要使用函数执行时间统计功能（runtime 1），则必须设置 USMART_ENTIMX_SCAN 为 1。另外，为了让统计时间精确到 0.1ms，定时器的计数时钟频率必须设置为 10Khz，否则时间就不是 0.1ms 了。**

第三和第四个函数仅用于服务 USMART 的函数执行时间统计功能（串口指令：runtime 1），分别是：usmart_timx_reset_time 和 usmart_timx_get_time，这两个函数代码如下：

```
/**
 * @brief       复位 runtime
 * @note        需要根据所移植到的 MCU 的定时器参数进行修改
 * @param       无
 * @retval      无
 */
void usmart_timx_reset_time(void)
{
    __HAL_TIM_CLEAR_FLAG(&g_timx_usmart_handle,TIM_FLAG_UPDATE);/* 清除中断标志*/
    __HAL_TIM_SET_AUTORELOAD(&g_timx_usmart_handle,0XFFFF);/*重载值设置最大*/
    __HAL_TIM_SET_COUNTER(&g_timx_usmart_handle, 0);        /* 清定时器 CNT */
    usmart_dev.runtime = 0;
}

/**
 * @brief       获得 runtime 时间
 * @note        需要根据所移植到的 MCU 的定时器参数进行修改
 * @param       无
 * @retval      执行时间,单位:0.1ms,最大延时时间为定时器 CNT 值的 2 倍*0.1ms
 */
uint32_t usmart_timx_get_time(void)
{
    /* 在运行期间,产生了定时器溢出 */
    if (__HAL_TIM_GET_FLAG(&g_timx_usmart_handle, TIM_FLAG_UPDATE) == SET)
    {
        usmart_dev.runtime += 0XFFFF;
    }
    usmart_dev.runtime += __HAL_TIM_GET_COUNTER(&g_timx_usmart_handle);
    return usmart_dev.runtime;      /* 返回计数值 */
}
```

usmart_timx_reset_time 函数在每次 USMART 调用函数之前执行，清除定时器的计数器，然后在函数执行完之后，调用 usmart_timx_get_time 获取计数器值，从而得到整个函数的运行时间。由于 usmart 调用的函数，都是在中断里面执行的，所以我们不太方便再用定时器的中断

功能来实现定时器溢出统计，因此，USMART 的函数执行时间统计功能，最多可以统计定时器溢出 1 次的时间，对 STM32H743 的定时器 4，该定时器是 16 位的，最大计数是 65535，而由于我们定时器设置的是 0.1ms 一个计时周期（10Khz），所以最长计时时间是：

65535 * 2 * 0.1ms = 13.1 秒

也就是说，如果函数执行时间超过 13.1 秒，那么计时将不准确。

最后一个是 USMART_TIMX_IRQHandler 函数，该函数的实现代码如下：

```
/**
 * @brief       USMART 定时器中断服务函数
 * @param       无
 * @retval      无
 */
void USMART_TIMX_IRQHandler(void)
{
    /* 溢出中断 */
    if(__HAL_TIM_GET_IT_SOURCE(&g_timx_usmart_handle,TIM_IT_UPDATE)==SET)
    {
        usmart_dev.scan();                                      /* usmart 扫描 */
        __HAL_TIM_SET_COUNTER(&g_timx_usmart_handle, 0);;       /* 清定时器 CNT */
        __HAL_TIM_SET_AUTORELOAD(&g_timx_usmart_handle, 100);/* 恢复原来的设置 */
    }
    __HAL_TIM_CLEAR_IT(&g_timx_usmart_handle, TIM_IT_UPDATE);/* 清除中断标志位 */
}
```

该函数是定时器 TIMX 的中断服务函数，也是一个宏定义函数，同样是在 usmart_port.h 里面定义，方便大家修改。该函数主要用于周期性调用 usmart 扫描函数（实际函数：usmart_scan），完成对输入数据流的处理。同时，清除定时器的 CNT 值，并设置自动重装载值。

完成这几个函数的移植，就可以使用 USMART 了。不过，需要注意的是，usmart 同外部的互交，一般是通过 usmart_dev 结构体实现，所以 usmart_init 和 usmart_scan 的调用分别是通过：usmart_dev.init 和 usmart_dev.scan 实现的。

另外我们还需要在 usmart_config.c 文件里面添加想要被 USMART 调用的函数。打开 usmart_config.c 文件，如图 28.3.2.1 所示：

```
/*********************************************************************************************/
/* 用户配置区
 * 这下面要包含所用到的函数所申明的头文件(用户自己添加)
 */

#include "./SYSTEM/sys/sys.h"
#include "./SYSTEM/delay/delay.h"
#include "./BSP/LCD/lcd.h"

extern void led_set(uint8_t sta);
extern void test_fun(void(*ledset)(uint8_t), uint8_t sta);

/* 函数名列表初始化(用户自己添加)
 * 用户直接在这里输入要执行的函数名及其查找串
 */
struct _m_usmart_nametab usmart_nametab[] =
{
#if USMART_USE_WRFUNS == 1      /* 如果使能了读写操作 */
    (void *)read_addr, "uint32_t read_addr(uint32_t addr)",
    (void *)write_addr, "void write_addr(uint32_t addr,uint32_t val)",
#endif
    (void *)delay_ms, "void delay_ms(uint16_t nms)",
    (void *)delay_us, "void delay_us(uint32_t nus)",
```

函数所在头文件添加区

用户函数添加区

图 28.3.2.1 添加需要被 USMART 调用的函数

这里的添加函数很简单，只要把函数所在头文件添加进来，并把函数名按上图所示的方式增加即可，默认我们添加了两个函数：delay_ms 和 delay_us。另外，read_addr 和 write_addr 属于 usmart 自带的函数，用于读写指定地址的数据，通过配置 USMART_USE_WRFUNS 宏定义，可以使能或者禁止这两个函数。

这里我们根据自己的需要按上图的格式添加其他函数，usmart_config.c 文件中我们已经添加了 LCD 的相关函数，大家可以查看本实验的 usmart_config.c 文件，并可在串口助手中调用。

具体的调用方法 28.4 小节有具体讲解。

2. main.c 代码

在 main 函数之前，我们添加了 led_set 和 test_fun 两个函数，代码如下：

```
/* LED 状态设置函数 */
void led_set(uint8_t sta)
{
    LED1(sta);
}

/* 函数参数调用测试函数 */
void test_fun(void(*ledset)(uint8_t), uint8_t sta)
{
    ledset(sta);
}
```

led_set 函数，用于设置 LED1 的状态，而第二个函数 test_fun 则是测试 USMART 对函数参数的支持的，test_fun 的第一个参数是函数，在 USMART 里面也是可以被调用的。

main 函数代码如下：

```
int main(void)
{
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    lcd_show_string(30,50,200,16,16,"STM32", RED);
    lcd_show_string(30,70,200,16,16,"USMART TEST", RED);
    lcd_show_string(30,90,200,16,16,"ATOM@ALIENTEK", RED);
    while(1)
    {
        LED0_TOGGLE();  /* LED0(RED) 闪烁 */
        delay_ms(500);
    }
}
```

此部分代码功能如下：经过一系列初始化，显示使用信息后，就是在无限循环中 LED0 翻转延时，并等待串口数据。

28.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。同时，屏幕上显示了一些字符（就是主函数里面要显示的字符）。

我们打开串口调试助手 XCOM，选择正确的串口号→多条发送→勾选发送新行（即发送回车键）选项，然后发送 list 指令，即可打印所有 usmart 可调用函数。如下图所示：

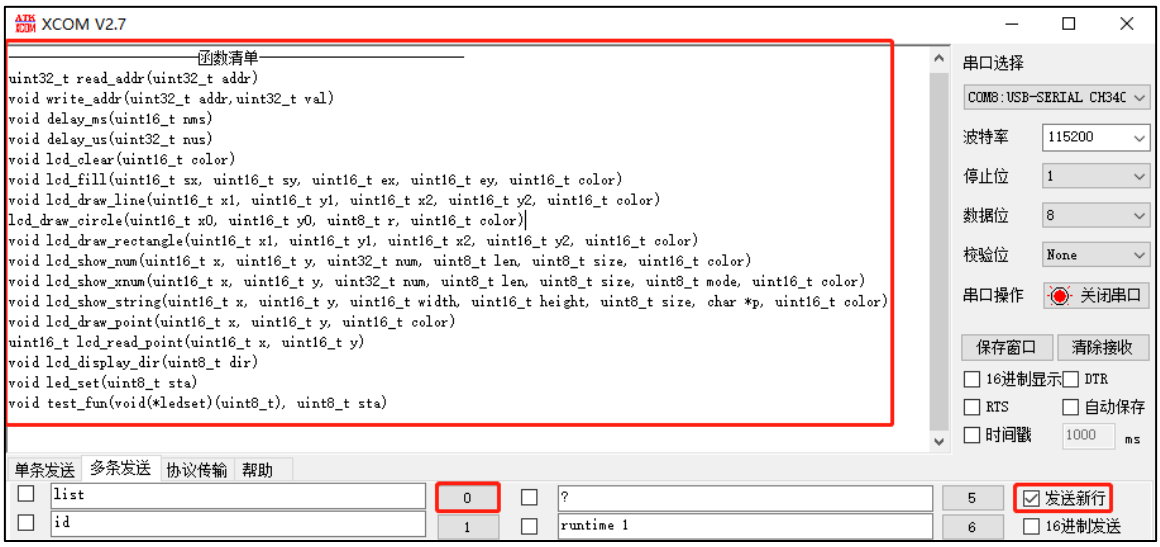


图 28.4.1 驱动串口调试助手

上图中 list、id、help、hex、dec、？和 runtime 都属于 usmart 自带的系统命令，点击后方的数字按钮，即可发送对应的指令。下面我们简单介绍下这几个命令：

list，该命令用于打印所有 usmart 可调用函数。发送该命令后，串口将受到所有能被 usmart 调用得到函数，如图 28.4.1 所示。

id，该指令用于获取各个函数的入口地址。比如前面写的 test_fun 函数，就有一个函数参数，我们需要先通过 id 指令，获取 led_set 函数的 id（即入口地址），然后将这个 id 作为函数参数，传递给 test_fun。

help（或者‘?’也可以），发送该指令后，串口将打印 usmart 使用的帮助信息。

hex 和 dec，这两个指令可以带参数，也可以不带参数。当不带参数的时候，hex 和 dec 分别用于设置串口显示数据格式为 16 进制/10 进制。当带参数的时候，hex 和 dec 就执行进制转换，比如输入：hex 1234，串口将打印：HEX:0X4D2，也就是将 1234 转换为 16 进制打印出来。又比如输入：dec 0X1234，串口将打印：DEC:4660，就是将 0X1234 转换为 10 进制打印出来。

runtime 指令，用于函数执行时间统计功能的开启和关闭，发送：runtime 1，可以开启函数执行时间统计功能；发送：runtime 0，可以关闭函数执行时间统计功能。函数执行时间统计功能，默认是关闭的。

大家可以亲自体验下这几个系统指令，不过要注意，所有的指令都是大小写敏感的，不要写错哦。

接下来，我们将介绍如何调用 list 所打印的这些函数，先来看一个简单的 delay_ms 的调用，我们分别输入 delay_ms(1000)和 delay_ms(0x3E8)，如图 28.4.2 所示：

图 28.4.2 串口调用 delay_ms 函数

从上图可以看出，delay_ms(1000)和 delay_ms(0x3E8)的调用结果是一样的，都是延时 1000ms，因为 usmart 默认设置的是 hex 显示，所以看到串口打印的参数都是 16 进制格式的，大家可以通过发送 dec 指令切换为十进制显示。另外，由于 USMART 对调用函数的参数大小写不敏感，所以参数写成：0X3E8 或者 0x3e8 都是正确的。另外，发送：runtime 1，开启运行时间统计功能，从测试结果看，USMART 的函数运行时间统计功能，是相当准确的。

我们再看另外一个函数，lcd_show_string 函数，该函数用于显示字符串，我们通过串口输入：lcd_show_string(30,200,200,16,16," This is a test for usmart!!", 0xF800)，如图 28.4.3 所示：

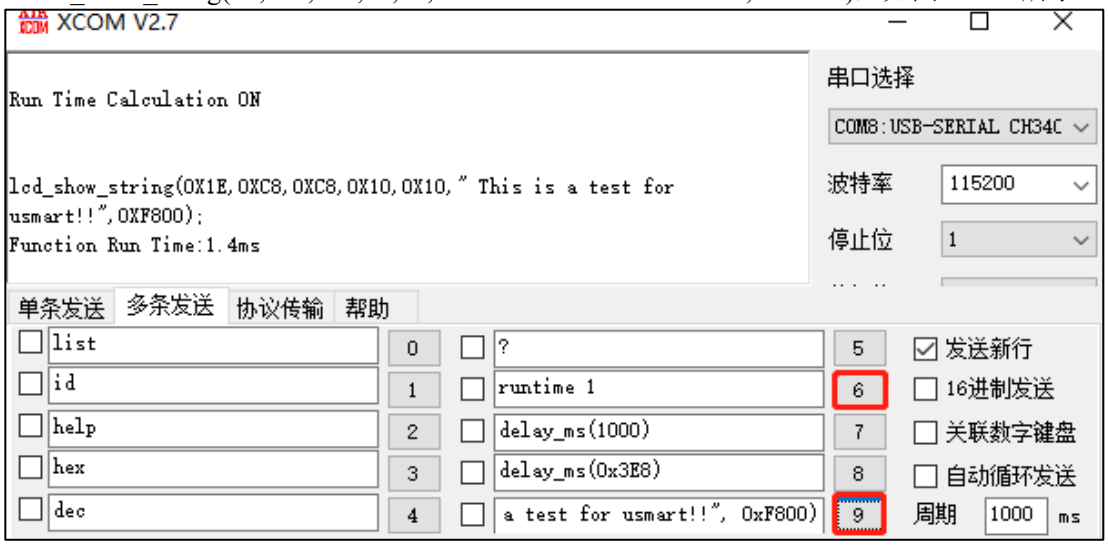


图 28.4.3 串口调用 lcd_show_string 函数

该函数用于在指定区域，显示指定字符串，发送给开发板后，我们可以看到 LCD 在我们指定的地方显示了：This is a test for usmart!! 这个字符串。

其他函数的调用，也都是一样的方法，这里我们就不多介绍了，最后说一下带有函数参数的函数的调用。我们将 led_set 函数作为 test_fun 的参数，通过在 test_fun 里面调用 led_set 函数，实现对 LED1 的控制。前面说过，我们要调用带有函数参数的函数，就必须先得到函数参数的入口地址（id），通过输入 id 指令，我们可以得到 led_set 的函数入口地址是：0X0800321D，所

以，我们在串口输入：test_fun(0X0800321D,0)，就可以控制 LED1 亮了。如图 28.4.4 所示：

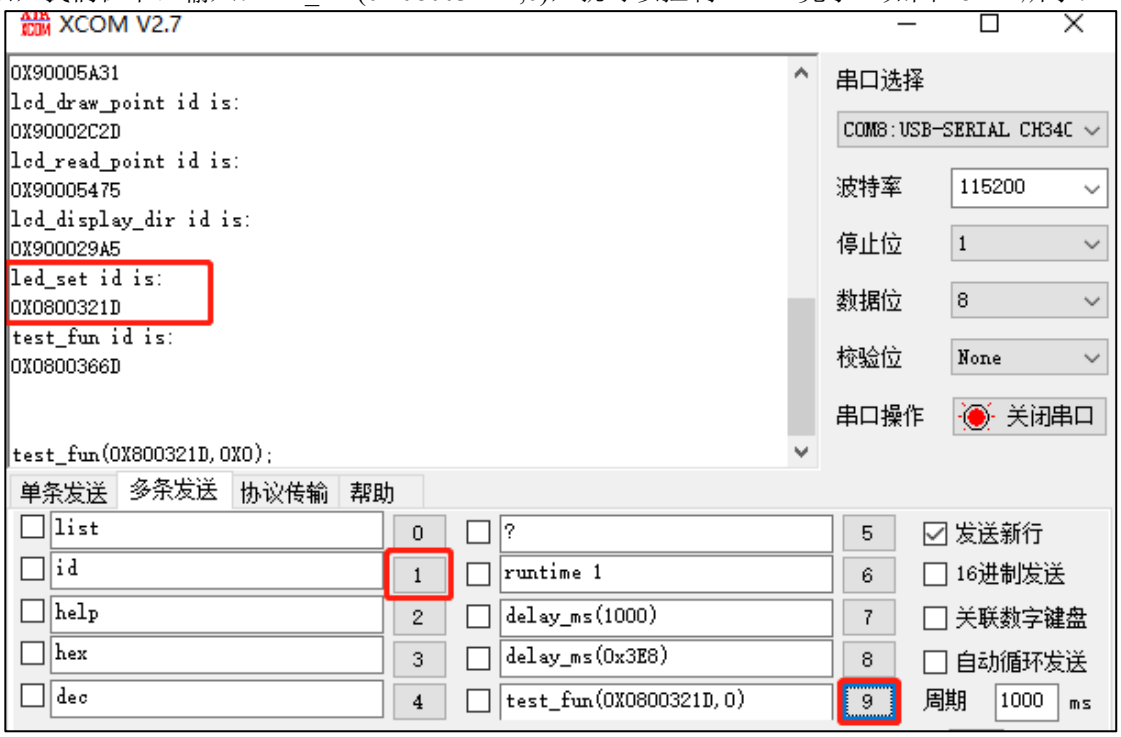


图 28.4.4 串口调用 test_fun 函数

在开发板上，我们可以看到，收到串口发送的 test_fun(0X0800321D,0)后，开发板的 LED1 亮了，然后大家可以通过发送 test_fun(0X0800321D,1)，来关闭 LED1。说明我们成功的通过 test_fun 函数调用 led_set，实现了对 LED1 的控制。也就验证了 USMART 对函数参数的支持。

USMART 调试组件的使用，就为大家介绍到这里。USMART 是一个非常不错的调试组件，希望大家能学会使用，可以达到事半功倍的效果。

第二十九章 RTC 实时时钟实验

本章，我们将介绍 STM32H743 的内部实时时钟（RTC）。我们将使用 LCD 模块来显示日期和时间，实现一个简单的实时时钟，并可以设置闹铃，另外还将介绍 BKP 的使用。

本章分为如下几个小节：

29.1 RTC 时钟简介
29.2 硬件设计
29.3 程序设计
29.4 下载验证

29.1 RTC 时钟简介

STM32H743 的实时时钟（RTC）相对于 STM32F1 来说，改进了不少，带了日历功能了。RTC 是一个独立的 BCD 定时器/计数器。RTC 提供一个日历时钟（包含年月日时分秒信息）、两个可编程闹钟（ALARM A 和 ALARM B）中断，以及一个具有中断功能的周期性可编程唤醒标志。RTC 还包含用于管理低功耗模式的自动唤醒单元。

两个 32 位寄存器（TR 和 DR）包含二进码十进数格式 (BCD) 的秒、分钟、小时（12 或 24 小时制）、星期、日期、月份和年份。此外，还可提供二进制格式的亚秒值。

STM32H743 的 RTC 可以自动将月份的天数补偿为 28、29（闰年）、30 和 31 天。并且还可以进行夏令时补偿。

RTC 模块和时钟配置是在后备区域，即在系统复位或从待机模式唤醒后 RTC 的设置和时间维持不变，只要后备区域供电正常，那么 RTC 将可以一直运行。但是在系统复位后，会自动禁止访问后备寄存器和 RTC，以防止对后备区域(BKP)的意外写操作。所以在要设置时间之前， 先要取消备份区域（BKP）写保护。

29.1.1 RTC 框图

下面先来学习 RTC 框图，通过学习 RTC 框图会有一个很好的整体掌握，同时对之后的编程也会有一个清晰的思路。RTC 的框图，如图 29.1.1 所示：

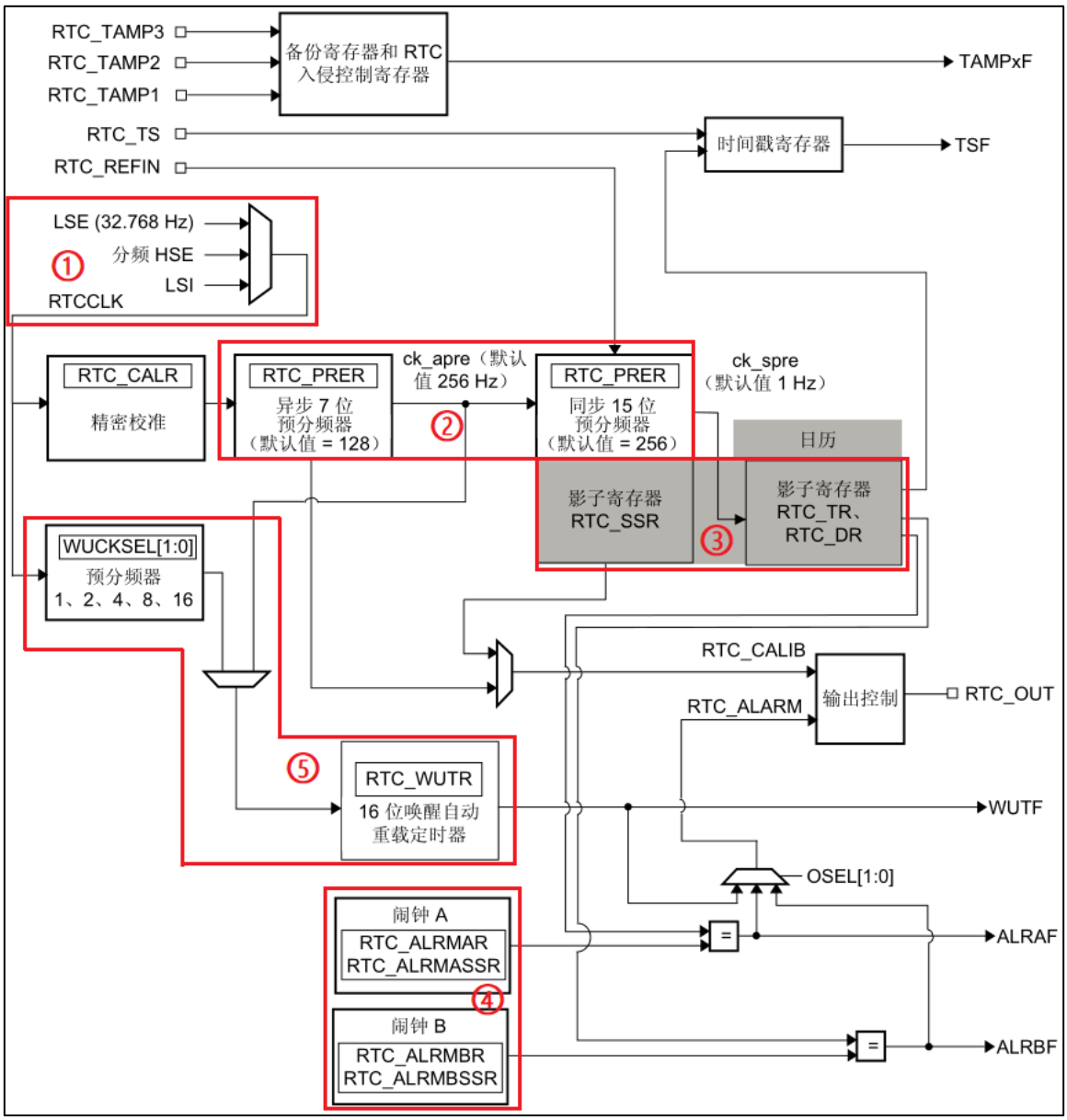


图 29.1.1 RTC 框图

我们把 RTC 框图分成以下几个部分讲解：

① 时钟源

STM32H743 的 RTC 时钟源（RTCCLK）通过时钟控制器，可以从 LSE 时钟、LSI 时钟以及 HSE 时钟三者中选择其一（通过设置 RCC_BDCR 寄存器选择）。一般我们选择 LSE，即外部 32.768Khz 晶振作为时钟源(RTCCLK)。外部晶振具有精度高的优点。LSI 是 STM32 芯片内部的低速 RC 振荡器，频率约 32 kHz，缺点是精度较低，可能会有温漂，所以一般不建议使用。比如当没有外部低速晶振（32.768Khz）的时候，分频后的 HSE 可以作为备选使用的时钟源。

② 预分频器

预分配器（RTC_PRER）分为 2 个部分：一个通过 RTC_PRER 寄存器的 PREDIV_A 位配置的 7 位异步预分频器。另一个通过 RTC_PRER 寄存器的 PREDIV_S 位配置的 15 位同步预分频器。

经过 7 位异步预分频器出来的时钟 ck_apre 可作为 RTC_SSR 亚秒递减计数器（RTC_SSR）的时钟，ck_apre 时钟频率的计算公式如下：

$$Fck_apre = \frac{Frtcclk}{(PREDIV_A + 1)}$$

当 RTC_SSR 寄存器递减到 0 的时候，会使用 PREDIV_S 的值重新装载 PREDIV_S。而 PREDIV_S 一般为 255，这样，我们得到亚秒时间的精度是：1/256 秒，即 3.9ms 左右，有了这个亚秒寄存器 RTC_SSR，就可以得到更加精确的时间数据。

经过 15 位同步预分频器出来的时钟 ck_spre 可以用于更新日历，也可以用作 16 位唤醒自动重载定时器的时基，ck_apre 时钟频率的计算公式如下：

$$Fck_spre = \frac{Frtcclk}{(PREDIV_S + 1) * (PREDIV_A + 1)}$$

PREDIV_A 和 PREDIV_S 分别为 RTC 的异步和同步分频器，使用两个预分频器时，我们推荐设置 7 位异步预分频器（PREDIV_A）的值较大，以最大程度降低功耗。例如：本实验我们的外部低速晶振的频率 32.768KHZ 经过 7 位异步预分频器后，再经过 15 位同步预分频器，要得到 1HZ 频率的时钟用于更新日历。通过计算知道，32.768KHZ 的时钟要经过 32768 分频，才能得到 1Hz 的 ck_spre。于是我们只需要设置：PREDIV_A=0X7F，即 128 分频；PREDIV_S=0XFF，即 256 分频，即可得到 1Hz 的 Fck_spre，PREDIV_A 的值我们也是往尽量大的原则，以最大程度降低功耗。

③ 时间和日期相关寄存器

该部分包括三个影子寄存器，RTC_SSR（亚秒）、RTC_TR（时间）、RTC_DR（日期）。实时时钟一般表示为：时/分/秒/亚秒。RTC_TR 寄存器用于存储时/分/秒时间数据，可读可写（即可设置或者获取时间）。RTC_DR 寄存器用于存储日期数据，包括年/月/日/星期，可读可写（即可设置或者获取日期）。RTC_SSR 寄存器用于存储亚秒级的时间，这样我们可以获取更加精确的时间数据。

这三个影子寄存器可以通过与 PCLK4（APB4 时钟）同步的影子寄存器来访问，这些时间和日期寄存器也可以直接访问，这样可避免等待同步的持续时间。

每隔 2 个 RTCCLK 周期，当前日历值便会复制到影子寄存器，并置位 RTC_ISR 寄存器的 RSF 位。我们可以读取 RTC_TR 和 RTC_DR 来得到当前时间和日期信息，不过需要注意的是：时间和日期都是以 BCD 码的格式存储的，读出来要转换一下，才可以得到十进制的数据。

④ 可编程闹钟

STM32H743 提供两个可编程闹钟：闹钟 A（ALARM_A）和闹钟 B（ALARM_B）。通过 RTC_CR 寄存器的 ALRAE 和 ALRBE 位置 1 来使能闹钟。当亚秒、秒、分、小时、日期分别与闹钟寄存器 RTC_ALRMASSR/RTC_ALRMAR 和 RTC_ALRMBSSR/RTC_ALRMBR 中的值匹配时，则可以产生闹钟（需要适当配置）。本章我们将利用闹钟 A 产生闹铃，即设置 RTC_ALRMASSR 和 RTC_ALRMAR 即可。

⑤ 周期性自动唤醒

STM32H743 的 RTC 不带秒钟中断了，但是多了一个周期性自动唤醒功能。周期性唤醒功能，由一个 16 位可编程自动重载递减计数器（RTC_WUTR）生成，可用于周期性中断/唤醒。

我们可以通过 RTC_CR 寄存器中的 WUTE 位设置使能此唤醒功能。

唤醒定时器的时钟输入可以是：2、4、8 或 16 分频的 RTC 时钟(RTCCLK)，也可以是 ck_spre 时钟（一般为 1Hz）。

当选择 RTCCLK(假定 LSE 是：32.768 kHz)作为输入时钟时，可配置的唤醒中断周期介于 122us（因为 RTCCLK/2 时，RTC_WUTR 不能设置为 0）和 32 s 之间，分辨率最低为：61us。

当选择 ck_spre（1Hz）作为输入时钟时，可得到的唤醒时间为 1s 到 36h 左右，分辨率为 1 秒。并且这个 1s~36h 的可编程时间范围分为两部分：

当 WUCKSEL[2:1]=10 时为：1s 到 18h。

当 WUCKSEL[2:1]=11 时约为：18h 到 36h。

在后一种情况下，会将 2^16 添加到 16 位计数器当前值（即扩展到 17 位，相当于最高位用 WUCKSEL [1]代替）。

初始化完成后，定时器开始递减计数。在低功耗模式下使能唤醒功能时，递减计数保持有效。此外，当计数器计数到 0 时，RTC_ISR 寄存器的 WUTF 标志会置 1，并且唤醒寄存器会使用其重载值（RTC_WUTR 寄存器值）动重载，之后必须用软件清零 WUTF 标志。

通过将 RTC_CR 寄存器中的 WUTIE 位置 1 来使能周期性唤醒中断时，可以使 STM32H743 退出低功耗模式。系统复位以及低功耗模式（睡眠、停机和待机）对唤醒定时器没有任何影响，

它仍然可以正常工作，故唤醒定时器，可以用于周期性唤醒 STM32H743。

29.1.2 RTC 寄存器

接下来，我们介绍本实验我们要用到的 RTC 寄存器。

- **RTC 时间寄存器（RTC_TR）**

RTC 时间寄存器描述如图 29.1.2.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PM	HT[1:0]		HU[3:0]			
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	MNT[2:0]			MNU[3:0]				Res.	ST[2:0]			SU[3:0]			
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

位 22 **PM**：AM/PM 符号 (AM/PM notation)
0：AM 或 24 小时制
1：PM

位 21:20 **HT[1:0]**：小时的十位（BCD 格式）(Hour tens in BCD format)

位 19:16 **HU[3:0]**：小时的个位（BCD 格式）(Hour units in BCD format)

位 14:12 **MNT[2:0]**：分钟的十位（BCD 格式）(Minute tens in BCD format)

位 11:8 **MNU[3:0]**：分钟的个位（BCD 格式）(Minute units in BCD format)

位 6:4 **ST[2:0]**：秒的十位（BCD 格式）(Second tens in BCD format)

位 3:0 **SU[3:0]**：秒的个位（BCD 格式）(Second units in BCD format)

图 29.1.2.1 RTC_TR 寄存器

该寄存器是 RTC 的时间寄存器，可读可写，对该寄存器写，可以设置时间，对该寄存器读，可以获取当前的时间，此外该寄存器受到寄存器写保护，通过 RTC 写保护寄存器(RTC_WPR)设置，后面会讲解到RTC_WPR寄存器。需要注意的是：本寄存器存储的数据都是BCD格式的，读取之后需要进行转换，方可得到十进制的时分秒等数据。

- **RTC 日期寄存器（RTC_DR）**

RTC 日期寄存器描述如图 29.1.2.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	YT[3:0]				YU[3:0]			
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WDU[2:0]			MT	MU[3:0]				Res.	Res.	DT[1:0]		DU[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

位 23:20 **YT[3:0]**：年份的十位（BCD 格式）(Year tens in BCD format)

位 19:16 **YU[3:0]**：年份的个位（BCD 格式）(Year units in BCD format)

位 15:13 **WDU[2:0]**：星期几的个位 (Week day units)
000：禁止
001：星期一
...
111：星期日

位 12 **MT**：月份的十位（BCD 格式）(Month tens in BCD format)

位 11:8 **MU**：月份的个位（BCD 格式）(Month units in BCD format)

位 5:4 **DT[1:0]**：日期的十位（BCD 格式）(Date tens in BCD format)

位 3:0 **DU[3:0]**：日期的个位（BCD 格式）(Date units in BCD format)

图 29.1.2.2 RTC_DR 寄存器

该寄存器是 RTC 的日期寄存器，可读可写，对该寄存器写，可以设置日期，对该寄存器读，可以获取当前的日期，同样该寄存器也受到寄存器写保护，存储的数据也都是 BCD 格式的。

- **RTC 亚秒寄存器（RTC_SSR）**

RTC 亚秒寄存器描述如图 29.1.2.3 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SS[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

位 15:0 **SS**：亚秒值 (Sub seconds value)
SS[15:0] 是同步预分频器计数器的值。此亚秒值可根据以下公式得出：
亚秒值 = (PREDIV_S - SS) / (PREDIV_S + 1)
注： 仅当执行平移操作之后，SS 才能大于 PREDIV_S。在这种情况下，正确的时间/日期比 RTC_TR/RTC_DR 所指示的时间/日期慢一秒钟。

图 29.1.2.3 RTC_SSR 寄存器

该寄存器可用于获取更加精确的 RTC 时间。不过，在本章没有用到，如果需要精确时间的地方，大家可以使用该寄存器。

- **RTC 控制寄存器（RTC_CR）**

RTC 控制寄存器描述如图 29.1.2.4 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	ITSE	COE	OSEL[1:0]		POL	COSEL	BKP	SUB1H	ADD1H
							rw	rw	rw	rw	rw	rw	rw	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TSIE	WUTIE	ALRBIE	ALRAIE	TSE	WUTE	ALRBE	ALRAE	Res.	FMT	BYPS HAD	REFCKON	TSEDGE	WUCKSEL[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

图 29.1.2.4 RTC_CR 寄存器

该寄存器重点介绍几个要用到的位：WUTIE 是唤醒定时器中断使能位，ALRAIE 是闹钟 A 中断使能位，本章用到这两个使能位，都设置为 1 即可。WUTE 和 ALRAE 分别是唤醒定时器和闹钟 A 使能位，同样都设置为 1，开启。FMT 为小时格式选择位，我们设置为 0，选择 24 小时制。WUCKSEL[2:0]，用于唤醒时钟选择，这个前面已经有介绍了，我们这里就不多说了。

- **RTC 初始化和状态寄存器（RTC_ISR）**

RTC 初始化和状态寄存器描述如图 29.1.2.5 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ITSF	RECALPF
														rc_w0	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TAMP3F	TAMP2F	TAMP1F	TSOVF	TSF	WUTF	ALRBF	ALRAF	INIT	INITF	RSF	INITS	SHPF	WUTWF	ALRBWF	ALRAWF
rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	rw	r	rc_w0	r	r	r	r	r

图 29.1.2.5 RTC_ISR 寄存器

该寄存器中，WUTF、ALRBF 和 ALRAF，分别是唤醒定时器、闹钟 B 和闹钟 A 的中断标志位，当对应事件产生时，这些标志位被置 1，如果设置了中断，则会进入中断服务函数，这些位通过软件写 0 清除。

INIT 为初始化模式控制位，要初始化 RTC 时，必须先设置 INIT=1。

INITF 为初始化标志位，当设置 INIT 为 1 以后，要等待 INITF 为 1，才可以更新时间、日期和预分频寄存器等。

RSF 位为寄存器同步标志，仅在该位为 1 时，表示日历影子寄存器已同步，可以正确读取 RTC_TR/RTC_TR 寄存器的值了。

WUTWF、ALRBWF 和 ALRAWF 分别是唤醒定时器、闹钟 B 和闹钟 A 的写标志，只有在这些位为 1 的时候，才可以更新对应的内容。比如：要设置闹钟 A 的 ALRMAR 和 ALRMASSR，则必须先等待 ALRAWF 为 1，才可以设置。

- **RTC 预分频寄存器（RTC_PRER）**

RTC 预分频寄存器描述如图 29.1.2.6 所示：

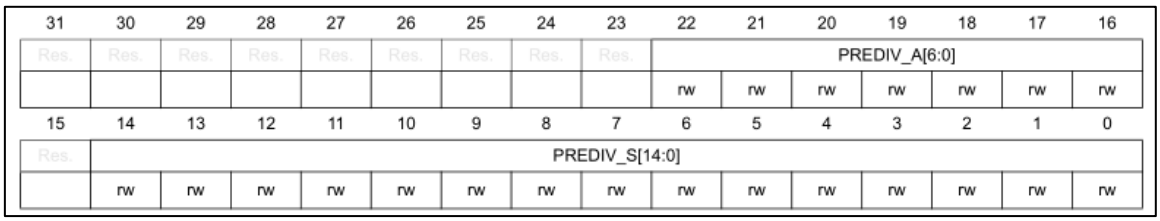


图 29.1.2.6 RTC_PRER 寄存器

该寄存器用于 RTC 的分频，我们在之前也有讲过，这里就不多说了。该寄存器的配置，必须在初始化模式（INITF=1）下，才可以进行。

- **RTC 唤醒寄存器（RTC_WUTR）**

RTC 唤醒寄存器描述如图 29.1.2.7 所示：

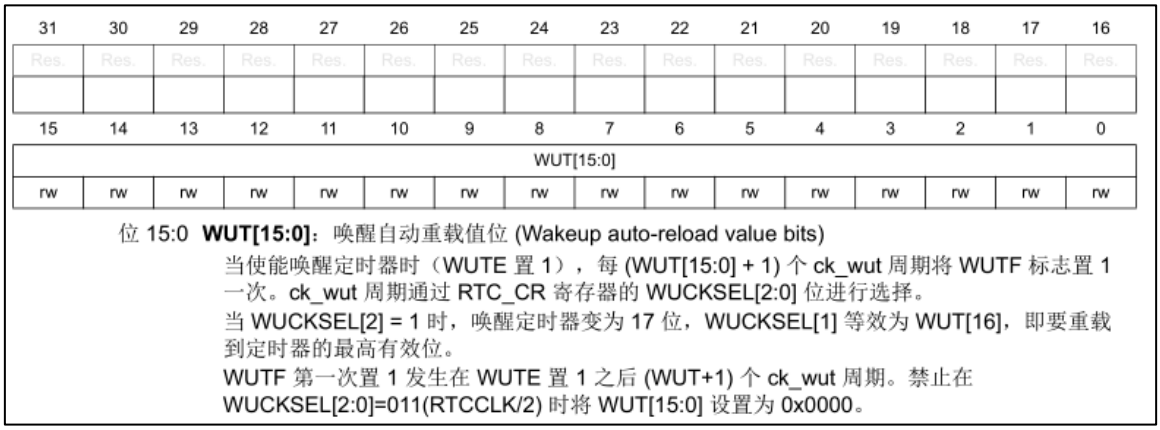


图 29.1.2.7 RTC_WUTR 寄存器

该寄存器用于设置自动唤醒重装载值，可用于设置唤醒周期。该寄存器的配置，必须等待 RTC_ISR 的 WUTWF 为 1 才可以进行。

- **RTC 闹钟 A 寄存器（RTC_ALRMAR）**

RTC 闹钟 A 寄存器描述如图 29.1.2.8 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MSK4	WDSEL	DT[1:0]		DU[3:0]				MSK3	PM	HT[1:0]		HU[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MSK2	MNT[2:0]			MNU[3:0]				MSK1	ST[2:0]			SU[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31 **MSK4**：闹钟 A 日期掩码 (Alarm A date mask)
0：如果日期/日匹配，则闹钟 A 置 1　　1：在闹钟 A 比较中，日期/日无关
位 30 **WDSEL**：星期几选择 (Week day selection)
0：DU[3:0] 代表日期的个位　　1：DU[3:0] 代表星期几 DT[1:0] 为无关位。
位 29:28 **DT[1:0]**：日期的十位（BCD 格式）(Date tens in BCD format)。
位 27:24 **DU[3:0]**：日期的个位或日（BCD 格式）(Date units or day in BCD format)。
位 23 **MSK3**：闹钟 A 小时掩码 (Alarm A hours mask)
0：如果小时匹配，则闹钟 A 置 1　　1：在闹钟 A 比较中，小时无关
位 22 **PM**：AM/PM 符号 (AM/PM notation)
0：AM 或 24 小时制　　1：PM
位 21:20 **HT[1:0]**：小时的十位（BCD 格式）(Hour tens in BCD format)。
位 19:16 **HU[3:0]**：小时的个位（BCD 格式）(Hour units in BCD format)。
位 15 **MSK2**：闹钟 A 分钟掩码 (Alarm A minutes mask)
0：如果分钟匹配，则闹钟 A 置 1　　1：在闹钟 A 比较中，分钟无关
位 14:12 **MNT[2:0]**：分钟的十位（BCD 格式）(Minute tens in BCD format)。
位 11:8 **MNU[3:0]**：分钟的个位（BCD 格式）(Minute units in BCD format)。
位 7 **MSK1**：闹钟 A 秒掩码 (Alarm A seconds mask)
0：如果秒匹配，则闹钟 A 置 1　　1：在闹钟 A 比较中，秒无关
位 6:4 **ST[2:0]**：秒的十位（BCD 格式）(Second tens in BCD format)。
位 3:0 **SU[3:0]**：秒的个位（BCD 格式）(Second units in BCD format)。

图 29.1.2.8 RTC_WUTR 寄存器

该寄存器用于设置闹铃 A，当 WDSEL 选择 1 时，使用星期制闹铃，本章我们选择星期制闹铃。该寄存器的配置，必须等待 RTC_ISR 的 ALRAWF 为 1 才可以进行。另外，还有 RTC_ALRMASSR 寄存器，该寄存器我们这里就不再介绍了，大家参考手册。

- **RTC 写保护寄存器（RTC_WPR）**

RTC 写保护寄存器描述如图 29.1.2.9 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	KEY							
								w	w	w	w	w	w	w	w

位 31:8 保留，必须保持复位值。
位 7:0 **KEY**：写保护关键字 (Write protection key)
可通过软件对该字节执行写操作。
读取该字节时，始终返回 0x00。
有关如何解锁 RTC 寄存器写保护的介绍，请参见 *RTC 寄存器写保护*。

图 29.1.2.9 RTC_WPR 寄存器

RTC 写保护寄存器：RTC_WPR，该寄存器比较简单，低八位有效。上电后，所有 RTC 寄存器都受到写保护（RTC_ISR[13:8]、RTC_TAFCR 和 RTC_BKPxR 除外），必须依次写入：0XCA、0X53 两关键字到 RTC_WPR 寄存器，才可以解锁。写一个错误的关键字将再次激活 RTC 的寄存器写保护。

- **RTC 备份寄存器（RTC_BKPxR）**

RTC 备份寄存器描述如图 29.1.2.10 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BKP[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BKP[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	w	rw	rw

位 31:0 **BKP[31:0]**

应用可向/从这些寄存器写入/读取数据。

当 VDD 关闭时，这些寄存器由 V_{BAT} 供电，因而系统复位时，这些寄存器不会复位，并且当器件在低功耗模式下工作时，寄存器的内容仍然有效。

发生入侵检测事件时该寄存器会被复位，并且只要 TAMPxF=1，该寄存器就一直保持复位。

图 29.1.2.10 RTC_BKPxR 寄存器

该寄存器组总共有 32 个，每个寄存器是 32 位的，可以存储 128 个字节的用户数据，这些寄存器在备份域中实现，可在 VDD 电源关闭时通过 VBAT 保持上电状态。备份寄存器不会在系统复位或电源复位时复位，也不会在 MCU 从待机模式唤醒时复位。

复位后，对 RTC 和 RTC 备份寄存器的写访问被禁止，执行以下操作可以使能对 RTC 及 RTC 备份寄存器的写访问：

1）电源控制寄存器(PWR_CR)的 DBP 位来使能对 RTC 及 RTC 备份寄存器的访问。

2）往 RTC_WPR 写入 0XCA、0X53 解锁序列（先写 0XCA，再写 0X53）。

我们可以用 BKP 来存储一些重要的数据，相当于一个 EEPROM，不过这个 EEPROM 并不是真正的 EEPROM，而是需要电池来维持它的数据。

- **备份区域控制（RCC_BDCR）**

备份区域控制寄存器描述如图 29.1.2.11 所示：

访问：0 ≤ 等待周期 ≤ 7，按字、半字和字节访问。连续访问该寄存器时，则插入等待周期。

系统复位后，RCC_BDCR 寄存器受写保护。要修改此寄存器，*PWR 控制寄存器 1 (PWR_CR1)* 中的 DBP 位必须置 “1”。只有备份域复位后，RCC_BDCR 位才能复位（请参见*第 8.4.6 节：备份域复位*）。其他内部复位或外部复位对这些位不会有任何影响。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BDRST
															rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTCEN	Res.	Res.	Res.	Res.	Res.	RTCSEL[1:0]		Res.	LSECSSD	LSECSSON	LSEDRV[1:0]		LSEBYP	LSERDY	LSEON
rw						rwo			r	rs	rw		rw	r	rw

位 16 **BDRST**：备份域软件复位 (Backup domain software reset)

由软件置 1 和复位。

0：复位未激活（备份域复位后的默认值）

1：复位整个 VSW 域

位 15 **RTCEN**：RTC 时钟使能 (RTC clock enable)

由软件置 1 和复位。

0：禁止 **rtc_ck** 时钟（备份域复位后的默认值）

1：使能 **rtc_ck** 时钟

位 9:8 **RTCSEL[1:0]**：RTC 时钟源选择 (RTC clock source selection)

由软件置 1，用于为 RTC 选择时钟源。这些位只能写入一次（在 LSE 上检测到故障的情况除外）。这些位必须在使能 LSECSSON 之前写入。BDRST 位可用于复位这些位，然后该位可再次写入。

如果将 HSE 选作 RTC 时钟：在系统处于停止模式或在发生引脚复位 (NRST) 时，该时钟将丢失。

00：无时钟（备份域复位后的默认值）

01：将 LSE 时钟用作 RTC 时钟

10：将 LSI 时钟用作 RTC 时钟

11：将以 RTCPRE 值进行分频的 HSE 时钟用作 RTC 时钟

位 4:3 **LSEDRV[1:0]**：LSE 振荡器驱动能力 (LSE oscillator driving capability)

由软件置 1，用于选择 LSE 振荡器的驱动能力。

00：最低驱动（备份域复位后的默认值）

01：中低驱动

10：中高驱动

11：最高驱动

位 2 **LSEBYP**：LSE 振荡器旁路 (LSE oscillator bypass)

由软件置 1 和复位，用于在调试模式下旁路振荡器。在 LSE 使能（通过 LSEON）或就绪（LSERDY = “1”）时，不能写入该位

0：不旁路 LSE 振荡器（备份域复位后的默认值）

1：旁路 LSE 振荡器

位 1 **LSERDY**：LSE 振荡器就绪 (LSE oscillator ready)

由硬件置 1 和复位，用于指示 LSE 已稳定。将 LSEON 设为“0”后，该位需要经过 6 个 **lse_ck** 时钟周期才能变为低电平。

0：LSE 振荡器未就绪（备份域复位后的默认值）

1：LSE 振荡器已就绪

位 0 **LSEON**：LSE 振荡器使能 (LSE oscillator enabled)

由软件置 1 和复位。

0：LSE 振荡器关闭（备份域复位后的默认值）

1：LSE 振荡器开启

图 29.1.2.11 RCC_BDCR 寄存器

RTC 的时钟源选择及使能设置都是通过这个寄存器来实现的，所以我们在 RTC 操作之前先要通过这个寄存器选择 RTC 的时钟源，然后才能开始其他的操作。

29.2 硬件设计

1. 例程功能

本实验通过 LCD 显示 RTC 时间，并可以通过 usmart 设置 RTC 时间，从而调节时间，或设置 RTC 闹钟，还可以写入或者读取 RTC 后备区域 SRAM。LED1 每两秒闪烁一次，表示进入 WAKE UP 中断。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 : LED0 – PB1
LED1 : LED1 – PB0
2） 串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3） RTC(实时时钟)
4）正点原子 2.8/3.5/4.3/7 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)

3. 原理图

RTC 属于 STM32H743 内部资源，通过软件设置好就可以了。不过 RTC 不能断电，否则数据就丢失了，我们如果想让时间在断电后还可以继续走，那么必须确保开发板的电池有电。

29.3 程序设计

29.3.1 RTC 的 HAL 库驱动

RTC 在 HAL 库中的驱动代码在 stm32h7xx_hal_rtc.c 文件（及其头文件）中。下面介绍几个重要的 RTC 函数，其他没有介绍的请看源码。

1. HAL_RTC_Init 函数

RTC 的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_RTC_Init(RTC_HandleTypeDef *hrtc);
```

- **函数描述：**
  用于初始化 RTC。
- **函数形参：**
  形参 1 是 RTC_HandleTypeDef 结构体类型指针变量，其定义如下：

```
typedef struct
{
  RTC_TypeDef                 *Instance;    /* 寄存器基地址 */
  RTC_InitTypeDef              Init;        /* RTC 配置结构体 */
  HAL_LockTypeDef              Lock;        /* RTC 锁定对象 */
  __IO HAL_RTCStateTypeDef     State;       /* RTC 设备访问状态 */
}RTC_HandleTypeDef;
```

1）Instance：指向 RTC 寄存器基地址。
2）Init：是真正的 RTC 初始化结构体，其结构体类型 RTC_InitTypeDef 定义如下：

```
typedef struct
{
  uint32_t HourFormat;          /* 小时格式 */
  uint32_t AsynchPrediv;        /* 异步预分频系数 */
  uint32_t SynchPrediv;         /* 同步预分频系数 */
  uint32_t OutPut;              /* 选择连接到 RTC_ALARM 输出的标志 */
  uint32_t OutPutRemap;         /* 指定 RTC 输出的映射 */
  uint32_t OutPutPolarity;      /* 设置 RTC_ALARM 的输出极性 */
  uint32_t OutPutType;          /* 设置 RTC_ALARM 的输出类型为开漏输出还是推挽输出 */
}RTC_InitTypeDef;
```

HourFormat 用来设置小时格式，可以是 12 小时制或者 24 小时制，这两个选项的宏定义分

别为 RTC_HOURFORMAT_12 和 RTC_HOURFORMAT_24。

AsynchPrediv 用来设置 RTC 的异步预分频系数，也就是设置 RTC_PRER 寄存器的 PREDIV_A 相关位，因为异步预分频系数是 7 位，所以最大值为 0x7F，不能超过这个值。

SynchPrediv 用来设置 RTC 的同步预分频系数，也就是设置 RTC_PRER 寄存器的 PREDIV_S 相关位，因为同步预分频系数也是 15 位，所以最大值为 0x7FFF，不能超过这个值。

OutPut 用来选择要连接到 RTC_ALARM 输出的标志，取值为：RTC_OUTPUT_DISABLE（禁止输出），RTC_OUTPUT_ALARMA（使能闹钟 A 输出），RTC_OUTPUT_ALARMB（使能闹钟 B 输出）和 RTC_OUTPUT_WAKEUP（使能唤醒输出）。

OutPutRemap 用于设置指定 RTC 输出的映射，即配置 RTC_OR 寄存器。

OutPutPolarity 用来设置 RTC_ALARM 的输出极性，与 Output 成员变量配合使用，取值为 RTC_OUTPUT_POLARITY_HIGH（高电平）或 RTC_OUTPUT_POLARITY_LOW（低电平）。

OutPutType 用来设置 RTC_ALARM 的输出类型为开漏输出（RTC_OUTPUT_TYPE_OPENDRAIN）还是推挽输出（RTC_OUTPUT_TYPE_PUSHPULL），与成员变量 OutPut 和 OutPutPolarity 配合使用。

3）Lock：用于配置锁状态。

4）State：RTC 设备访问状态。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

2. HAL_RTC_SetTime 函数

HAL_RTC_SetTime 是设置 RTC 的时间函数。其声明如下：

```
HAL_StatusTypeDef HAL_RTC_SetTime(RTC_HandleTypeDef *hrtc,
                                  RTC_TimeTypeDef *sTime, uint32_t Format);
```

- **函数描述：**

  该函数用于设置 RTC 的时间，即设置时间寄存器 RTC_TR 的相关位的值。

- **函数形参：**

  形参 1 是 RTC_HandleTypeDef 结构体类型指针变量，即 RTC 的句柄。

  形参 2 是 RTC_TimeTypeDef 结构体类型指针变量，定义如下：

```
typedef struct
{
  uint8_t Hours;
  uint8_t Minutes;
  uint8_t Seconds;
  uint8_t TimeFormat;
  uint32_t SubSeconds;
  uint32_t SecondFraction;
  uint32_t DayLightSaving;
  uint32_t StoreOperation;
}RTC_TimeTypeDef;
```

前面四个成员变量就比较好理解了，分别用来设置 RTC 时间参数的小时，分钟，秒钟，以及 AM/PM 符号，大家参考前面讲解的 RTC_TR 的位描述即可。SubSeconds 用来读取保存亚秒寄存器 RTC_SSR 的值，SecondFraction 用来读取保存同步预分频系数的值，也就是 RTC_PRER 的位 0~14，DayLightSaving 用来设置日历时间增加 1 小时，减少 1 小时，还是不变。StoreOperation 用户可对此变量设置以记录是否已对夏令时进行更改。

形参 3 是 uint32_t 类型变量，用来设置输入的时间格式为 BIN 格式还是 BCD 格式，可选值为 RTC_FORMAT_BIN 或者 RTC_FORMAT_BCD。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

3. HAL_RTC_SetDate 函数

HAL_RTC_SetDate 是设置 RTC 的日期函数。其声明如下：

```
HAL_StatusTypeDef HAL_RTC_SetDate(RTC_HandleTypeDef *hrtc,
                                  RTC_DateTypeDef *sDate, uint32_t Format);
```

- **函数描述：**

  该函数用于设置 RTC 的日期，即设置日期寄存器 RTC_DR 的相关位的值。

- **函数形参：**

  形参 1 是 RTC_HandleTypeDef 结构体类型指针变量，即 RTC 的句柄。

  形参 2 是 RTC_DateTypeDef 结构体类型指针变量，定义如下：

```
typedef struct
{
  uint8_t WeekDay;      /* 星期 */
  uint8_t Month;        /* 月份 */
  uint8_t Date;         /* 日期 */
  uint8_t Year;         /* 年份 */
}RTC_DateTypeDef;
```

结构体一共四个成员变量，这四个成员变量分别对应星期、月份、日期和年份，对应的是 RTC_DR 寄存器。

形参 3 是 uint32_t 类型变量，用来设置输入的时间格式为 BIN 格式还是 BCD 格式，可选值为 RTC_FORMAT_BIN 或者 RTC_FORMAT_BCD。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

4. HAL_RTC_GetTime 函数

HAL_RTC_GetTime 是获取当前 RTC 时间函数。其声明如下：

```
HAL_StatusTypeDef HAL_RTC_GetTime(RTC_HandleTypeDef *hrtc,
                                  RTC_TimeTypeDef *sTime, uint32_t Format);
```

- **函数描述：**

  该函数用于获取当前 RTC 时间，即读时间寄存器 RTC_TR 的相关位的值。

- **函数形参：**

  形参 1 是 RTC_HandleTypeDef 结构体类型指针变量，即 RTC 的句柄。

  形参 2 是 RTC_TimeTypeDef 结构体类型指针变量，对应的是 RTC_TR 寄存器。

  形参 3 是 uint32_t 类型变量，用来设置获取的时间格式为 BIN 格式还是 BCD 格式，可选值为 RTC_FORMAT_BIN 或者 RTC_FORMAT_BCD。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

5. HAL_RTC_SetDate 函数

HAL_RTC_SetDate 是获取当前 RTC 日期函数。其声明如下：

```
HAL_StatusTypeDef HAL_RTC_SetDate (RTC_HandleTypeDef *hrtc,
                                   RTC_DateTypeDef *sDate, uint32_t Format);
```

- **函数描述：**

  该函数用于获取当前 RTC 日期，即读时间寄存器 RTC_DR 的相关位的值。

- **函数形参：**

  形参 1 是 RTC_HandleTypeDef 结构体类型指针变量，即 RTC 的句柄。

  形参 2 是 RTC_DateTypeDef 结构体类型指针变量，对应的是 RTC_DR 寄存器。

  形参 3 是 uint32_t 类型变量，用来设置获取的时间格式为 BIN 格式还是 BCD 格式，可选值为 RTC_FORMAT_BIN 或者 RTC_FORMAT_BCD。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

6. HAL_RTC_SetAlarm_IT 函数

HAL_RTC_SetAlarm_IT 是设置闹钟并开启闹钟中断的函数。其声明如下：

```
HAL_StatusTypeDef HAL_RTC_SetAlarm_IT(RTC_HandleTypeDef *hrtc,
                                      RTC_AlarmTypeDef *sAlarm, uint32_t Format);
```

- **函数描述：**

  该函数用于设置闹钟并开启闹钟中断。

- **函数形参：**

  形参 1 是 RTC_HandleTypeDef 结构体类型指针变量，即 RTC 的句柄。

  形参 2 是 RTC_AlarmTypeDef 结构体类型指针变量。

  形参 3 是 uint32_t 类型变量，用来设置获取的时间格式为 BIN 格式还是 BCD 格式，可选

值为 RTC_FORMAT_BIN 或者 RTC_FORMAT_BCD。

重点介绍 RTC_AlarmTypeDef 结构体指针类型，结构体定义如下：

```
typedef struct
{
  RTC_TimeTypeDef AlarmTime;
  uint32_t AlarmMask;
  uint32_t AlarmSubSecondMask;
  uint32_t AlarmDateWeekDaySel;
  uint8_t AlarmDateWeekDay;
  uint32_t Alarm;
}RTC_AlarmTypeDef;
```

AlarmTime 用来设置闹钟时间，是 RTC_TimeTypeDef 结构体类型，该结构体前面我们已经讲解过各个成员变量含义。

AlarmMask 用来设置闹钟时间掩码，也就是在我们第一个参数设置的时间中（包括后面参数 RTC_AlarmDateWeekDay 设置的星期几/哪一天），哪些是无关的。比如我们设置闹钟时间为每天的 10 点 10 分 10 秒，那么我们可以选择值 RTC_AlarmMask_DateWeekDay，也就是我们不关心是星期几/每月哪一天。这里我们选择为 RTC_AlarmMask_None，也就是精确匹配时间，所有的时分秒以及星期几/(或者每月哪一天)都要精确匹配。

AlarmSubSecondMask 和 AlarmMask 作用类似，只不过该变量是用来设置亚秒的。

AlarmDateWeekDaySel 用来选择是闹钟是按日期还是按星期。比如我们选择 RTC_AlarmDateWeekDaySel_WeekDay 那么闹钟就是按星期。如果我们选择 RTC_AlarmDateWeekDaySel_Date 那么闹钟就是按日期。这与后面第四个参数是有关联的，我们在后面第四个参数讲解。

AlarmDateWeekDay 用来设置闹钟的日期或者星期几。比如我们第三个参数 RTC_AlarmDateWeekDaySel 设置了值为 RTC_AlarmDateWeekDaySel_WeekDay，也就是按星期，那么参数 RTC_AlarmDateWeekDay 的取值范围就为星期一到星期天，也就是 RTC_Weekday_Monday 到 RTC_Weekday_Sunday。如果第三个参数 RTC_AlarmDateWeekDaySel 设置值为 RTC_AlarmDateWeekDaySel_Date，那么它的取值范围就为日期值，0~31。

Alarm 用来设置是闹钟 A 还是闹钟 B，这个很好理解。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

RTC 配置步骤

1）使能 RTC 时钟，并使能 RTC 及 RTC 后备寄存器写访问

复位后，后备区域存在写保护，RTC 属于后备区域，所以我们需要取消后备区域写保护。同时，我们需要使能 RTC 时钟，通过 RCC_APB4ENR 寄存器中的 RTCAPBEN 位设置。而取消后备区域写保护则是通过 PWR_CR1 寄存器中的 DBP 位去设置。HAL 库设置方法为：

```
__HAL_RCC_RTC_CLK_ENABLE();   /* 使能 RTC 时钟 */
HAL_PWR_EnableBkUpAccess();   /* 取消备份区域写保护 */
```

2）开启外部低速振荡器 LSE，选择 RTC 时钟，并使能

调用 HAL_RCC_OscConfig 函数配置开启 LSE。

调用 HAL_RCCEx_PeriphCLKConfig 函数选择 RTC 时钟源。

使能 RTC 函数为：__HAL_RCC_RTC_ENABLE。

3）初始化 RTC，设置 RTC 的分频，以及配置 RTC 参数

在 HAL 中，通过 HAL_RTC_Init 函数配置 RTC 分频系数，以及 RTC 的工作参数。

注意：该函数会调用：HAL_RTC_MspInit 函数来完成对 RTC 的底层初始化，包括：RTC 时钟使能、时钟源选择等。

4）设置 RTC 的时间

调用 HAL_RTC_SetTime 函数设置 RTC 时间，该函数实际设置时间寄存器 RTC_TR 的相关位的值。

5）设置 RTC 的日期

调用 HAL_RTC_SetDate 函数设置 RTC 的日期，该函数实际设置日期寄存器 RTC_DR 的相

关位的值。

6）获取 RTC 当前日期和时间

调用 HAL_RTC_GetTime 函数获取当前 RTC 时间，该函数实际读取 RTC_TR 寄存器，然后将值存放到相应的结构体中。

调用 HAL_RTC_GetDate 函数获取当前 RTC 日期，该函数实际读取 RTC_DR 寄存器，然后将值存放到相应的结构体中。

通过以上 6 个步骤，我们就完成了对 RTC 的配置，RTC 即可正常工作，而且这些操作不是每次上电都必须执行的，可以视情况而定。当然，我们还可以唤醒中断、闹钟等，这些将在后面介绍。

29.3.2 程序流程图

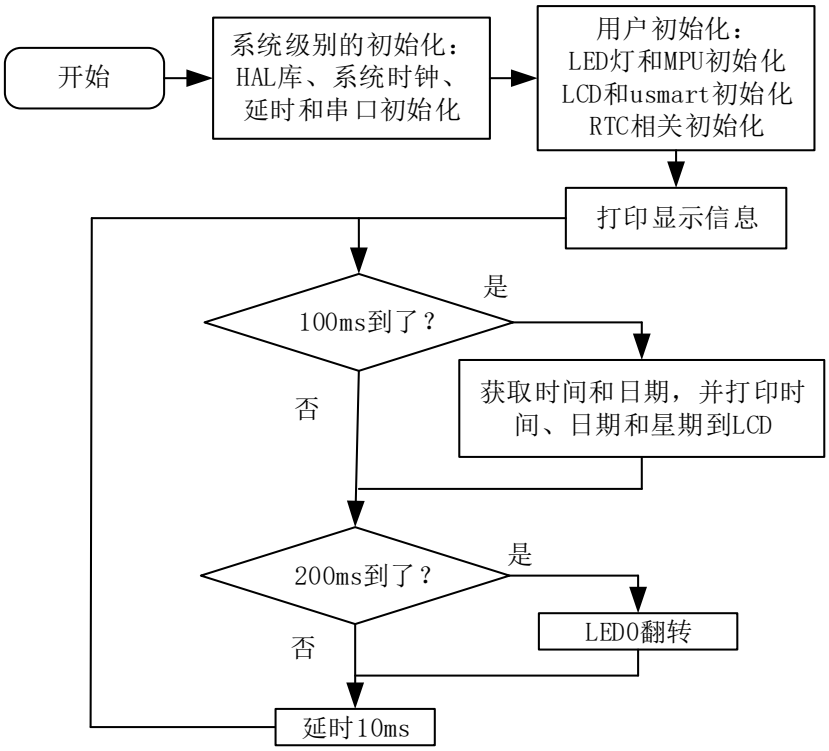


图 29.3.2.1 RTC 实时时钟实验程序流程图

29.3.3 程序解析

1. RTC 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。RTC 驱动源码包括两个文件：rtc.c 和 rtc.h。

由于篇幅所限，rtc.c 中的代码，我们不全部贴出了，只针对几个重要的函数进行介绍。

rtc.h 头文件只有函数的声明，下面我们直接介绍 rtc.c 的程序，先看 RTC 的初始化函数，其定义如下：

```
/**
 * @brief       RTC 初始化
 * @note
 *              默认尝试使用 LSE,当 LSE 启动失败后,切换为 LSI.
 *              通过 BKP 寄存器 0 的值,可以判断 RTC 使用的是 LSE/LSI:
 *              当 BKP0==0X5050 时,使用的是 LSE
 *              当 BKP0==0X5051 时,使用的是 LSI
 *              注意:切换 LSI/LSE 将导致时间/日期丢失,切换后需重新设置.
 *
```

```
 * @param       无
 * @retval      0,成功
 *              1,进入初始化模式失败
 */
uint8_t rtc_init(void)
{
    /* 检查是不是第一次配置时钟 */
    uint16_t bkpflag = 0;

    g_rtc_handle.Instance = RTC;
    g_rtc_handle.Init.HourFormat = RTC_HOURFORMAT_24;/* RTC 设置为 24 小时格式 */
    g_rtc_handle.Init.AsynchPrediv = 0X7F;          /* RTC 异步分频系数(1~0X7F) */
    g_rtc_handle.Init.SynchPrediv = 0XFF;           /* RTC 同步分频系数(0~7FFF) */
    g_rtc_handle.Init.OutPut = RTC_OUTPUT_DISABLE;
    g_rtc_handle.Init.OutPutPolarity = RTC_OUTPUT_POLARITY_HIGH;
    g_rtc_handle.Init.OutPutType = RTC_OUTPUT_TYPE_OPENDRAIN;
    bkpflag = rtc_read_bkr(0);  /* 读取 BKP0 的值 */
    if (HAL_RTC_Init(&g_rtc_handle) != HAL_OK)
    {
        return 1;
    }

    if ((bkpflag != 0X5050) && (bkpflag != 0x5051))/* 之前未初始化过 重新配置 */
    {
        rtc_set_time(6, 59, 56, RTC_HOURFORMAT12_AM);  /* 设置时间*/
        rtc_set_date(20, 4, 22, 3);                     /* 设置日期 */
    }

    return 0;
}
```

该函数用来初始化 RTC 配置以及日期和时钟，但是只在第一次的时候设置时间，以后如果重新上电/复位都不会再进行时间设置了（前提是备份电池有电）。在第一次配置的时候，我们是按照上面介绍的 RTC 初始化步骤调用函数 HAL_RTC_Init 来实现的。

我们通过读取 BKP 寄存器 0 的值来判断是否需要进行时间的设置，对 BKP 寄存器 0 的写操作是在 HAL_RTC_MspInit 回调函数中实现，下面会讲。第一次未对 RTC 进行初始化 BKP 寄存器 0 的值非 0x5050 非 0x5051，当进行 RTC 初始化时，BKP 寄存器 0 的值就是 0x5050 或 0x5051，所以以上代码操作确保时间只会设置一次，复位时不会重新设置时间。电池正常供电时，我们设置的时间不会因复位或者断电而丢失。

读取后备寄存器的函数其实还是调用 HAL 库提供的函数接口，写后备寄存器函数同样也是。这两个函数如下：

```
uint32_t HAL_RTCEx_BKUPRead(RTC_HandleTypeDef *hrtc, uint32_t BackupRegister);
void HAL_RTCEx_BKUPWrite(RTC_HandleTypeDef *hrtc, uint32_t BackupRegister,
                         uint32_t Data);
```

这两个函数的使用方法就非常简单，分别用来读和写 BKR 寄存器的值。这里我们只是略微点到为止，详看例程源码。

这里设置时间和日期，分别是通过 rtc_set_time 和 rtc_set_date 函数来实现的，这两个函数实际就是调用库函数里面的 HAL_RTC_SetTime 函数和 HAL_RTC_SetDate 函数来实现，这里我们之所以要写两个这样的函数，目的是为了我们的 USMART 来调用，方便直接通过 USMART 来设置时间和日期。rtc_set_time 和 rtc_set_date 实现十分简单，这里讲解了，详看源码。

接下来，我们用 HAL_RTC_MspInit 函数来编写 RTC 时钟配置等代码，其定义如下：

```
void HAL_RTC_MspInit(RTC_HandleTypeDef *hrtc)
{
    uint16_t retry = 200;
    RCC_OscInitTypeDef rcc_osc_init_handle;
    RCC_PeriphCLKInitTypeDef rcc_periphclk_init_handle;

    __HAL_RCC_RTC_CLK_ENABLE();     /* 使能 RTC 时钟 */
    HAL_PWR_EnableBkUpAccess();     /* 取消备份区域写保护 */
    __HAL_RCC_RTC_ENABLE();         /* 使能 RTC */
```

```
    /* 使用寄存器的方式去检测 LSE 是否可以正常工作 */
    RCC->BDCR |= 1 << 0;    /* 开启外部低速振荡器 LSE */

    while (retry && ((RCC->BDCR & 0X02) == 0))  /* 等待 LSE 准备好 */
    {
        retry--;
        delay_ms(5);
    }

    if (retry == 0)     /* LSE 起振失败，使用 LSI */
    {   /* 选择要配置的振荡器 */
        rcc_osc_init_handle.OscillatorType = RCC_OSCILLATORTYPE_LSI;
        rcc_osc_init_handle.LSEState = RCC_LSI_ON;       /* LSI 状态：开启 */
        rcc_osc_init_handle.PLL.PLLState = RCC_PLL_NONE;/* PLL 无配置 */
        HAL_RCC_OscConfig(&rcc_osc_init_handle);

        /* 选择要配置的外设 RTC */
        rcc_periphclk_init_handle.PeriphClockSelection = RCC_PERIPHCLK_RTC;
        /* RTC 时钟源选择 LSI */
        rcc_periphclk_init_handle.RTCClockSelection = RCC_RTCCLKSOURCE_LSI;
        HAL_RCCEx_PeriphCLKConfig(&rcc_periphclk_init_handle);
        rtc_write_bkr(0, 0X5051);
    }
    else
    {
        rcc_osc_init_handle.OscillatorType = RCC_OSCILLATORTYPE_LSE;
        rcc_osc_init_handle.PLL.PLLState = RCC_PLL_NONE;   /* PLL 不配置 */
        rcc_osc_init_handle.LSEState = RCC_LSE_ON;         /* LSE 状态：开启 */
        HAL_RCC_OscConfig(&rcc_osc_init_handle);

        rcc_periphclk_init_handle.PeriphClockSelection = RCC_PERIPHCLK_RTC;
        rcc_periphclk_init_handle.RTCClockSelection = RCC_RTCCLKSOURCE_LSE;
        HAL_RCCEx_PeriphCLKConfig(&rcc_periphclk_init_handle);
        rtc_write_bkr(0, 0X5050);
    }
}
```

接着，我们介绍一下 rtc_set_alarma 函数，其定义如下：

```
/**
 * @breif       设置闹钟时间(按星期闹铃,24 小时制)
 * @param       week         : 星期几(1~7)
 * @param       hour,min,sec: 小时,分钟,秒钟
 * @retval      无
 */
void rtc_set_alarma(uint8_t week, uint8_t hour, uint8_t min, uint8_t sec)
{
    RTC_AlarmTypeDef rtc_alarm_handle;

    rtc_alarm_handle.AlarmTime.Hours = hour;           /* 小时 */
    rtc_alarm_handle.AlarmTime.Minutes = min;          /* 分钟 */
    rtc_alarm_handle.AlarmTime.Seconds = sec;          /* 秒 */
    rtc_alarm_handle.AlarmTime.SubSeconds = 0;
    rtc_alarm_handle.AlarmTime.TimeFormat = RTC_HOURFORMAT12_AM;

    rtc_alarm_handle.AlarmMask = RTC_ALARMMASK_NONE;   /* 精确匹配星期，时分秒 */
    rtc_alarm_handle.AlarmSubSecondMask = RTC_ALARMSUBSECONDMASK_NONE;
    /* 按星期 */
    rtc_alarm_handle.AlarmDateWeekDaySel=RTC_ALARMDATEWEEKDAYSEL_WEEKDAY;
    rtc_alarm_handle.AlarmDateWeekDay = week;              /* 星期 */
    rtc_alarm_handle.Alarm = RTC_ALARM_A;                  /* 闹钟 A */
    HAL_RTC_SetAlarm_IT(&g_rtc_handle, &rtc_alarm_handle, RTC_FORMAT_BIN);

    HAL_NVIC_SetPriority(RTC_Alarm_IRQn, 1, 2);    /* 抢占优先级 1,子优先级 2 */
```

```
    HAL_NVIC_EnableIRQ(RTC_Alarm_IRQn);
}
```

该函数用于设置闹钟 A，也就是设置 ALRMAR 和 ALRMASSR 寄存器的值，来设置闹钟时间，这里使用 HAL 库的 HAL_RTC_SetAlarm_IT 函数来初始化闹钟相关寄存器，前面已经介绍过这个函数，请回顾。

调用函数 rtc_set_wakeup 设置闹钟 A 的参数之后，最后，开启闹钟 A 中断（连接在外部中断线 17），并设置中断分组。当 RTC 的时间和闹钟 A 设置的时间完全匹配时，将产生闹钟中断。

接着，我们介绍一下 rtc_set_wakeup 函数，该函数代码如下：

```
/**
 * @breif       周期性唤醒定时器设置
 * @param       wksel
 *   @arg       RTC_WAKEUPCLOCK_RTCCLK_DIV16       ((uint32_t)0x00000000)
 *   @arg       RTC_WAKEUPCLOCK_RTCCLK_DIV8        ((uint32_t)0x00000001)
 *   @arg       RTC_WAKEUPCLOCK_RTCCLK_DIV4        ((uint32_t)0x00000002)
 *   @arg       RTC_WAKEUPCLOCK_RTCCLK_DIV2        ((uint32_t)0x00000003)
 *   @arg       RTC_WAKEUPCLOCK_CK_SPRE_16BITS     ((uint32_t)0x00000004)
 *   @arg       RTC_WAKEUPCLOCK_CK_SPRE_17BITS     ((uint32_t)0x00000006)
 * @note        000,RTC/16;001,RTC/8;010,RTC/4;011,RTC/2;
 * @note        注意:RTC 就是 RTC 的时钟频率,即 RTCCLK!
 * @param       cnt: 自动重装载值.减到 0,产生中断.
 * @retval      无
 */
void rtc_set_wakeup(uint8_t wksel, uint16_t cnt)
{
    /* 清除 RTC WAKE UP 的标志 */
    __HAL_RTC_WAKEUPTIMER_CLEAR_FLAG(&g_rtc_handle, RTC_FLAG_WUTF);
    HAL_RTCEx_SetWakeUpTimer_IT(&g_rtc_handle,cnt,wksel);/* 设置重装载值和时钟 */
    HAL_NVIC_SetPriority(RTC_WKUP_IRQn, 2, 2);  /* 抢占优先级 2,子优先级 2 */
    HAL_NVIC_EnableIRQ(RTC_WKUP_IRQn);
}
```

该函数用于设置 RTC 周期性唤醒定时器，实现周期性唤醒中断，连接在外部中断线 19。该函数调用的是 HAL 库函数 HAL_RTCEx_SetWakeUpTimer_IT 实现的。

有了中断设置函数，就必定有中断服务函数，同时 HAL 库会开放中断处理回调函数。接下来看这两个中断的中断服务函数和中断处理回调函数，代码如下：

```
/**
 * @breif       RTC 闹钟中断服务函数
 * @param       无
 * @retval      无
 */
void RTC_Alarm_IRQHandler(void)
{
    HAL_RTC_AlarmIRQHandler(&g_rtc_handle);
}

/**
 * @breif       RTC 闹钟 A 中断处理回调函数
 * @param       hrtc:RTC 句柄
 * @retval      无
 */
void HAL_RTC_AlarmAEventCallback(RTC_HandleTypeDef *hrtc)
{
    printf("ALARM A!\r\n");
}

/**
 * @breif       RTC WAKE UP 中断服务函数
 * @param       无
 * @retval      无
 */
```

```
void RTC_WKUP_IRQHandler(void)
{
    HAL_RTCEx_WakeUpTimerIRQHandler(&g_rtc_handle);
}

/**
 * @breif       RTC WAKE UP 中断处理处理回调函数
 * @param       hrtc:RTC 句柄
 * @retval      无
 */
void HAL_RTCEx_WakeUpTimerEventCallback(RTC_HandleTypeDef *hrtc)
{
    LED1_TOGGLE();
}
```

其中，RTC_Alarm_IRQHandler 函数用于闹钟中断，其中断控制逻辑写在中断回调函数 HAL_RTC_AlarmAEventCallback 中，每当闹钟 A 闹铃时，会从串口打印一个字符串“ALARM A!”。RTC_WKUP_IRQHandler 函数用于 RTC 自动唤醒定时器中断，其中断控制逻辑写在中断回调函数 HAL_RTCEx_WakeUpTimerEventCallback 中，可以通过观察 LED1 的状态来查看 RTC 自动唤醒中断的情况。

rtc.c 的其他程序，这里就不再介绍了，请大家直接看源码。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t hour, min, sec, ampm;
    uint8_t year, month, date, week;
    uint8_t tbuf[40];
    uint8_t t = 0;
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    rtc_init();                             /* 初始化 RTC */
    /* 配置 WAKE UP 中断,1 秒钟中断一次 */
    rtc_set_wakeup(RTC_WAKEUPCLOCK_CK_SPRE_16BITS, 0);
    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "RTC TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    while (1)
    {
        t++;
        if ((t % 10) == 0)  /* 每 100ms 更新一次显示数据 */
        {
            rtc_get_time(&hour, &min, &sec, &ampm);
            sprintf((char *)tbuf, "Time:%02d:%02d:%02d", hour, min, sec);
            lcd_show_string(30, 130, 210, 16, 16, (char*)tbuf, RED);
            rtc_get_date(&year, &month, &date, &week);
            sprintf((char *)tbuf, "Date:20%02d-%02d-%02d", year, month, date);
            lcd_show_string(30, 150, 210, 16, 16, (char*)tbuf, RED);
            sprintf((char *)tbuf, "Week:%d", week);
            lcd_show_string(30, 170, 210, 16, 16, (char*)tbuf, RED);
        }
        if ((t % 20) == 0)
        {
```

```
            LED0_TOGGLE();  /* 每 200ms,翻转一次 LED0 */
        }
        delay_ms(10);
    }
}
```

我们通过 rtc_set_wakeup(RTC_WAKEUPCLOCK_CK_SPRE_16BITS,0)设置 RTC 周期性自动唤醒周期为 1 秒钟。然后，在无限循环中每 100ms 读取 RTC 的时间和日期（一次），并显示在 LCD 上面。每 200ms，翻转一次 LED0。

为方便 RTC 相关函数的调用验证，在 usmart_config.c 里面，修改了 usmart_nametab 如下：

```
/* 函数名列表初始化(用户自己添加)
 * 用户直接在这里输入要执行的函数名及其查找串
 */
struct _m_usmart_nametab usmart_nametab[] =
{
#if USMART_USE_WRFUNS == 1      /* 如果使能了读写操作 */
    (void *)read_addr, "uint32_t read_addr(uint32_t addr)",
    (void *)write_addr, "void write_addr(uint32_t addr,uint32_t val)",
#endif
    (void *)delay_ms, "void delay_ms(uint16_t nms)",
    (void *)delay_us, "void delay_us(uint32_t nus)",

    (void *)rtc_read_bkr, "uint32_t rtc_read_bkr(uint32_t bkrx)",
    (void *)rtc_write_bkr, "void rtc_write_bkr(uint32_t bkrx, uint32_t data)",
    (void *)rtc_set_time, "uint8_t rtc_set_time(uint8_t hour, uint8_t min,
                                                uint8_t sec, uint8_t ampm)",
    (void *)rtc_set_date, "uint8_t rtc_set_date(uint8_t year, uint8_t month,
                                                uint8_t date, uint8_t week)",

    (void *)rtc_set_wakeup, "void rtc_set_wakeup(uint8_t wksel, uint16_t cnt)",
    (void *)rtc_get_week, "uint8_t rtc_get_week(uint16_t year, uint8_t month,
                                                uint8_t day)",
    (void *)rtc_set_alarma, "void rtc_set_alarma(uint8_t week, uint8_t hour,
                                                 uint8_t min, uint8_t sec)",
};
```

将 RTC 的一些相关函数加入了 usmart，这样通过串口就可以直接设置 RTC 时间、日期、闹钟 A 和周期性唤醒等操作。 至此，RTC 实时时钟的软件设计就完成了，接下来就让我们来检验一下，我们的程序是否正确。

29.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了，同时 LED1 每两秒闪烁一次，说明周期性唤醒中断工作正常。然后，可以看到 LCD 模块开始显示时间，实际显示效果如图 29.4.1 所示：

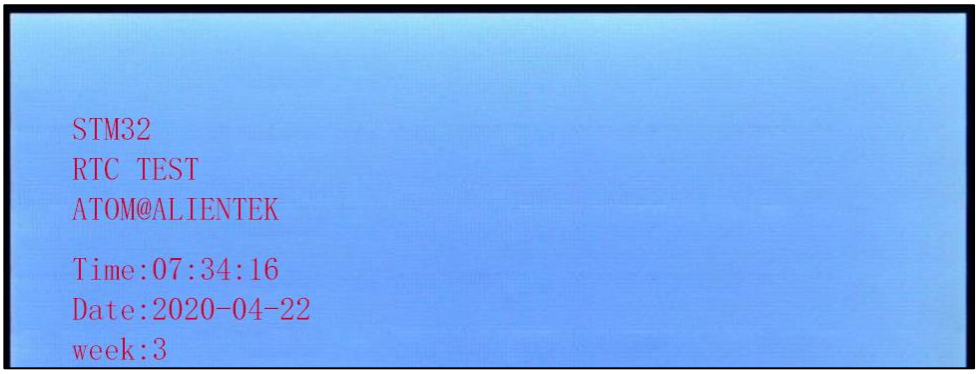


图 29.4.1 RTC 实验测试图

如果时间和日期不正确，可以利用上一章介绍的 usmart 工具，通过串口来设置，并且可以设置闹钟时间等，如图 29.4.2 所示：

图 29.4.2 通过 USMART 设置时间和日期并测试闹钟 A

按照图中编号 6、7、8 顺序，设置闹钟 A、设置日期、设置时间。然后等待我们设置的时间到来后，串口打印 ALARM A!这个字符串，证明我们的闹钟 A 程序正常运行了！

第三十章 硬件随机数实验

本章，我们将介绍 STM32H743 的硬件随机数发生器。我们使用 KEY0 按键来获取硬件随机数，并且将获取到的随机数值显示在 LCD 上面。同时，使用 LED0 指示程序运行状态。

本章分为如下几个小节：

30.1 随机数发生器简介
30.2 硬件设计
30.3 程序设计
30.4 下载验证

30.1 随机数发生器简介

STM32H743 自带了硬件随机数发生器（RNG），RNG 处理器是一个以连续模拟噪声为基础的随机数发生器，在主机读数时提供一个 32 位的随机数。

30.1.1 RNG 框图

下面先来学习 RNG 框图，通过学习 RNG 框图会有一个很好的整体掌握，同时对之后的编程也会有一个清晰的思路。STM32H743 的随机数发生器框图如图 30.1.1.1 所示：

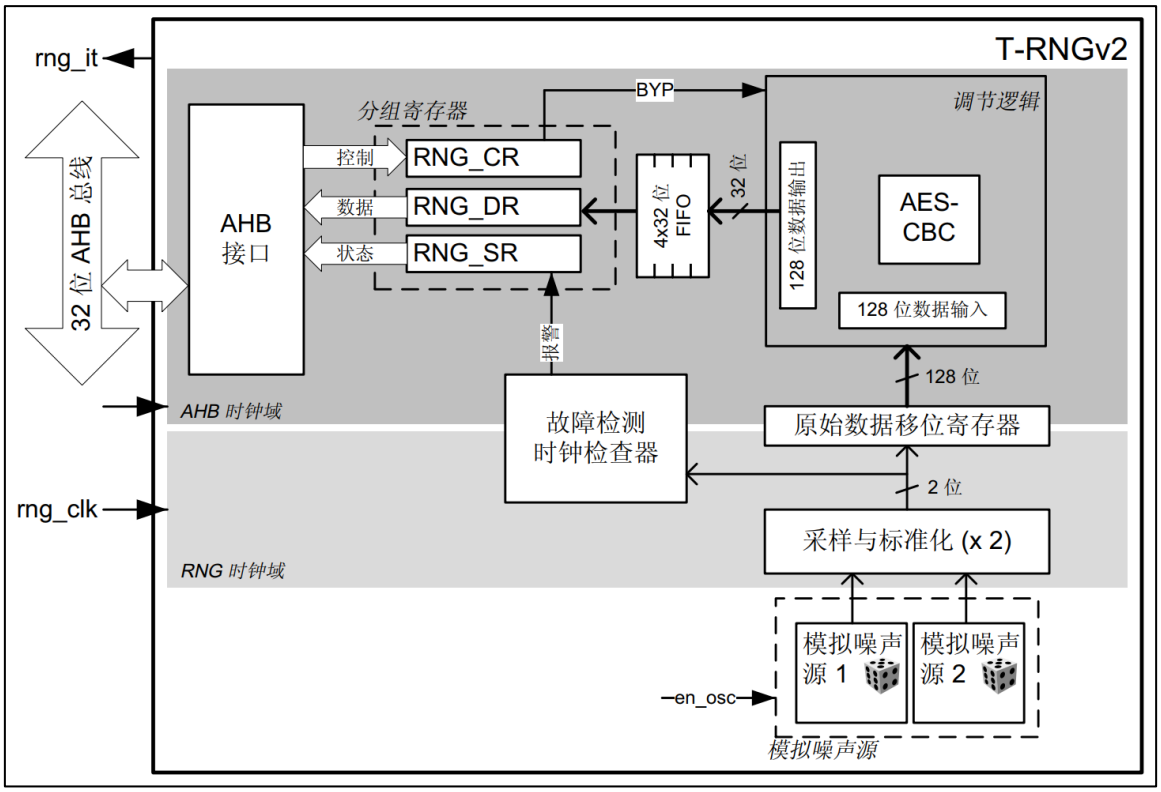


图 30.1.1.1 随机数发生器(RNG)框图

随机数发生器有 2 个时钟域：AHB 时钟域和 RNG 时钟域。AHB 时钟域的时钟来自 AHB2，用于访问相关寄存器等，通过 AHB2ENR 的 RNGEN 位（bit6）使能。

RNG 时钟域时钟为 rng_clk，rng_clk 由 D2CCIP2R 寄存器的 RNGSEL[1:0]位选择具体的时钟源，具体选择关系为：0，hsi8_ck；1，pll1_q_ck；2，lse_ck；3，lsi_ck。我们一般选择 pll1_q_ck 作为 rng_clk 的时钟源（实验中调用 sys_stm32_clock_init 函数时设置了 pll1_q_ck 为 200MHZ），因此设置 RNGSEL[1:0]=1 即可。AHB2ENR 和 D2CCIP2R 这两个寄存器我们就不做介绍了，大家参考手册。

从 RNG 框图整体上知道，RNG 有两个输入和一个输出。具体如下表：

信号名称	信号类型	说明
rng_it	数字输出	RNG 全局中断请求
rng_hclk2	数字输入	AHB2 时钟
rng_clk	数字输入	RNG 专用时钟，与 rng_hclk2 异步

表 30.1.1.1 RNG 内部输入/输出信号

STM32H743 的随机数发生器（RNG）采用模拟电路实现，由内部两个模拟噪声源产生种子，经过采样和归一化处理，再经过线性移位寄存器和判断逻辑，最终输出到 RNG_DR，生成 32 位随机数。

每个模拟噪声源由 3 个环形振荡器组成，振荡器产生的输出经过异或运算产生种子，经过采样归一化处理后，输出到 RNG 内部的线性移位寄存器。采样频率由 rng_clk 时钟提供，因此，随机数质量与 HCLK 频率无关。当将大量种子引入线性移位寄存器后，经过判断逻辑，最终输出到数据寄存器 (RNG_DR)。

同时，系统会监视模拟种子和专用时钟 rng_clk，当种子上出现异常序列，或 rng_clk 时钟频率过低时，可以由 RNG_SR 寄存器的对应位读取到，如果设置了中断，则在检测到错误时，还可以产生中断。

30.1.2 RNG 寄存器

- **RNG 控制寄存器（RNG_CR）**

RNG 控制寄存器描述如图 30.1.2.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BYP	CED	Res.	IE	RNGEN	Res.	Res.
									rw	rw		rw	rw		

位 2 **RNGEN**：真随机数发生器使能 (True random number generator enable)

0：禁止真随机数发生器。模拟噪声源关闭，并会对由 RNG 时钟提供时钟信号的逻辑系统进行门控。

1：使能真随机数发生器。

图 30.1.2.1 RNG_CR 寄存器

该寄存器我们只关心 RNGEN 位，该位用于使能随机数发生器，所以必须设置为 1。

- **RNG 状态寄存器（RNG_SR）**

RNG 状态寄存器描述如图 30.1.2.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEIS	CEIS	Res.	Res.	SECS	CECS	DRDY
									rc_w0	rc_w0			r	r	r

位 0 **DRDY**：数据就绪 (Data ready)

0：RNG_DR 寄存器尚未有效，无可用随机数据

1：RNG_DR 寄存器包含有效随机数据

在输出缓冲区变空之后（读取 RNG_DR 寄存器之后），在生成新的随机值之前，该位返回“0”。

注：　当禁用外设时（RNG_CR 寄存器中的 RNGEN = “0”），可以将 DRDY 位置 1。

如果 RNG_CR 寄存器中的 IE = “1”，则在 DRDY = “1”时生成中断。

图 30.1.2.2 RNG_SR 寄存器

该寄存器我们仅关心最低位（DRDY 位），该位用于表示 RNG_DR 寄存器包含的随机数数据是否有效，如果该位为 1，则说明 RNG_DR 的数据是有效的，可以读取出来了。读 RNG_DR 后，该位自动清零。

- **RNG 数据寄存器（RNG_DR）**

RNG 数据寄存器描述如图 30.1.2.3 所示：

RNG_DR 寄存器是只读寄存器，在读取时提供 32 位随机数值。读取该寄存器后，如果输出 FIFO 为空，则该寄存器将在 216 个 AHB 时钟周期之后提供一个新的随机值。

当 DRDY = “1” 时，即使 RNGEN = “0”，该寄存器的内容也是有效的。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RNDATA[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RNDATA[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

位 31:0 **RNDATA[31:0]**：随机数据 (Random data)
DRDY = “1” 时有效的 32 位随机数据。DRDY = “0” 时，RNDATA 值为 0。

图 30.1.2.3 RNG_DR 寄存器

RNG_DR 寄存器是只读寄存器，我们可以读取该寄存器获得 32 位随机数值。此寄存器在最多 216 个 AHB 时钟周期后，又可以提供新的随机数值。

30.2 硬件设计

1. 例程功能

本实验使用 STM32H743 自带的硬件随机数生成器（RNG），获取随机数，并显示在 LCD 屏幕上。按 KEY0 可以获取一次随机数。同时程序自动获取 0~9 范围内的随机数，显示在屏幕上。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED : LED0 - PB1
2）独立按键 KEY0 - PH3
3）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4） 正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
5） RNG（硬件随机数生成器）

3. 原理图

RNG 属于 STM32H743 内部资源，通过软件设置好就可以了。本实验通过配合按键获取随机数和通过 LCD 显示。

30.3 程序设计

30.3.1 RNG 的 HAL 库驱动

RNG 在 HAL 库中的驱动代码在 stm32h7xx_hal_rng.c 文件（及其头文件）中。

1. HAL_RNG_Init 函数

RNG 的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_RNG_Init(RNG_HandleTypeDef *hrng);
```

- **函数描述：**
  用于初始化 RNG。
- **函数形参：**
  形参 1 是 RNG_HandleTypeDef 结构体类型指针变量，其定义如下：

```
typedef struct
{
```

```
  RNG_TypeDef                  *Instance;       /* RNG 基地址 */
  RNG_InitTypeDef               Init;           /* RNG 初始化配置结构体 */
  HAL_LockTypeDef               Lock;           /* RNG 锁设置 */
  __IO HAL_RNG_StateTypeDef     State;          /* RNG 设备访问状态 */
  __IO  uint32_t                ErrorCode;      /* RNG 错误代码 */
  uint32_t                      RandomNumber;   /* RNG 最后生成的随机数 */
} RNG_HandleTypeDef;
```

1）Instance：指向 RNG 寄存器基地址。

2）Init：是的 RNG 初始化结构体，其结构体类型 RTC_InitTypeDef 定义如下：

```
typedef struct
{
  uint32_t            ClockErrorDetection;         /* CED 时钟错误检测 */
} RNG_InitTypeDef;
```

3）Lock：用于配置锁状态。

4）State：RNG 设备访问状态。

5）ErrorCode ：RNG 错误代码

6）RandomNumber ：该变量存储 RNG 最后生成的随机数

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

2. HAL_RNG_GenerateRandomNumber 函数

HAL_RNG_GenerateRandomNumber 是 RNG 生成随机数函数。其声明如下：

```
HAL_StatusTypeDef HAL_RNG_GenerateRandomNumber(RNG_HandleTypeDef *hrng,
                                               uint32_t *random32bit);
```

- **函数描述：**

  该函数用于 RNG 生成随机数。

- **函数形参：**

  形参 1 是 RNG_HandleTypeDef 结构体类型指针变量，即 RNG 的句柄。

  形参 2 是 uint32_t 类型指针变量，随机 32 位指针，生成随机变量。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

RNG 配置步骤

1）使能随机数发生器时钟

调用__HAL_RCC_RNG_CLK_ENABLE 函数使能随机数发生器时钟，实际是通过设置 AHB2ENR 寄存器的相关位。

2）初始化(使能)随机数发生器

通过调用 HAL_RNG_Init 函数初始化随机数发生器，然后自行调用 RNG 的 MSP 回调函数，并使能随机数发生器。

当我们使用 HAL_RNG_Init 之后，在该函数内部，会调用 RNG 的 MSP 回调函数。回调函数中一般编写与 MCU 相关的外设时钟初始化以及 NVIC 配置。

3）判断 DRDY 位，读取随机数值

经过前面两个步骤，我们就可以读取随机数值了，不过每次读取之前，必须先判断 RNG_SR 寄存器的 DRDY 位，如果该位为 1，则可以读取 RNG_DR 得到随机数值，如果不为 1，则需要等待。在 HAL 库中，通过调用 HAL_RNG_GenerateRandomNumber 函数判断 DRDY 位并读取随机数值。

通过以上 3 个步骤的设置，我们就可以使用 STM32H7 的随机数发生器（RNG）了。

30.3.2 程序流程图

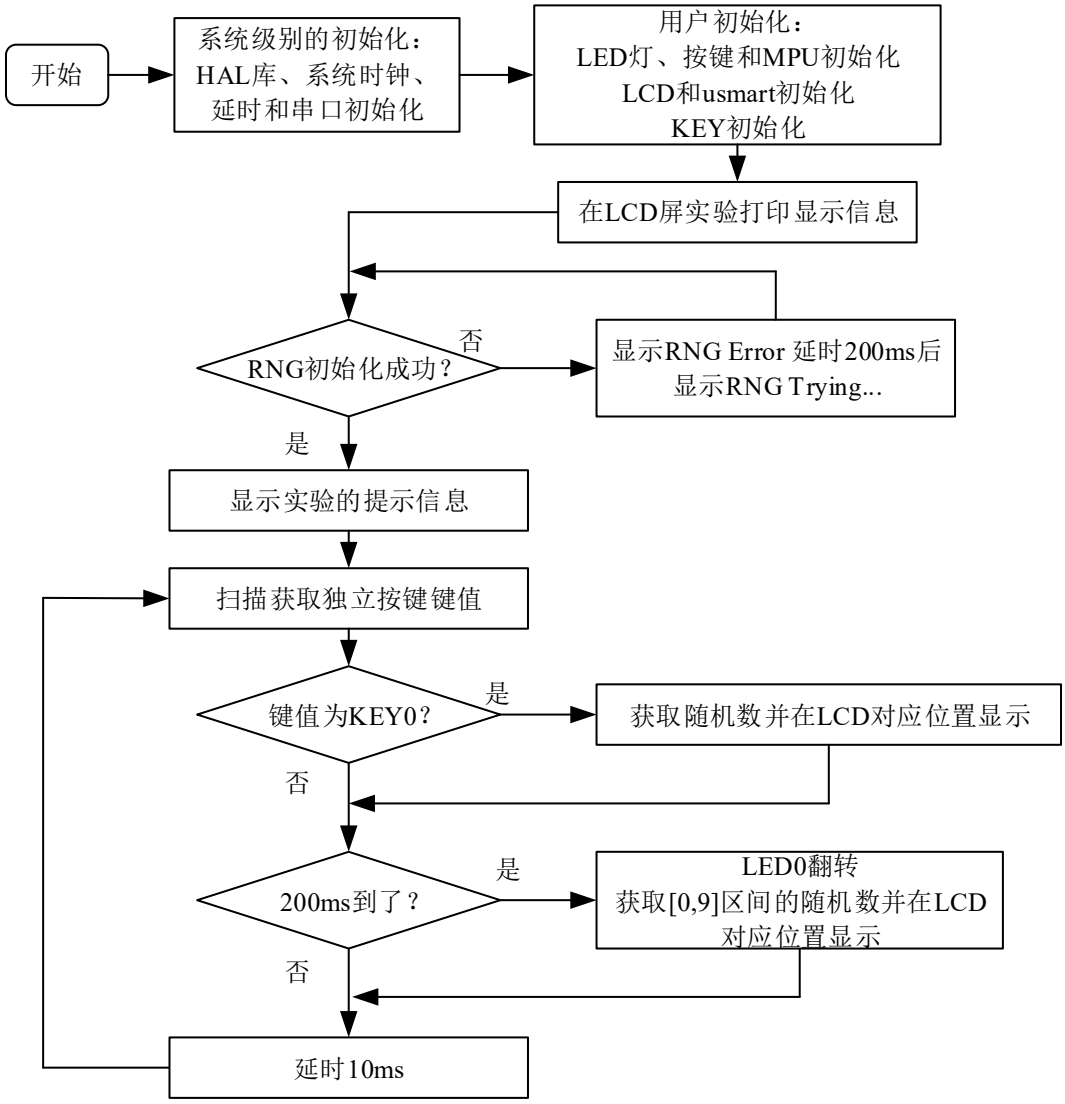


图 30.3.2.1 硬件随机数实验程序流程图

30.3.3 程序解析

1. RNG 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。RNG 驱动源码包括两个文件：rng.c 和 rng.h。

rng.h 头文件只有函数的声明，下面我们直接介绍 rng.c 的程序，先看 RNG 的初始化函数，其定义如下：

```
/**
 * @brief       初始化 RNG
 * @param       无
 * @retval      0,成功;1,失败
 */
uint8_t rng_init(void)
{
    uint16_t retry = 0;

    rng_handle.Instance = RNG;
    HAL_RNG_DeInit(&rng_handle);
```

```
    HAL_RNG_Init(&rng_handle); /* 初始化 RNG */
    while (__HAL_RNG_GET_FLAG(&rng_handle, RNG_FLAG_DRDY) == RESET
    && retry < 10000)           /* 等待 RNG 准备就绪 */
    {
        retry++;
        delay_us(10);
    }
    if (retry >= 10000)
    {
        return 1; /* 随机数产生器工作不正常 */
    }
    return 0;
}
```

rng_init 函数中调用 HAL_RNG_Init 函数初始化 RNG，然后等待 RNG 初始化完成，同时判断初始化过程是否超时。

我们用 HAL_RTC_MspInit 回调函数来编写 RNG 时钟配置等代码，其定义如下：

```
/**
 * @brief       RNG 底层驱动，时钟源设置和使能
 * @note        此函数会被 HAL_RNG_Init()调用
 * @param       hrng:RNG 句柄
 * @retval      无
 */
void HAL_RNG_MspInit(RNG_HandleTypeDef *hrng)
{
   RCC_PeriphCLKInitTypeDef rcc_periph_clk_init;
   /* 设置 RNG 时钟源，选择 pll1_q_ck 作为 RNG 时钟源(200Mhz) */
   rcc_periph_clk_init.PeriphClockSelection = RCC_PERIPHCLK_RNG;
   /* 设置 RNG 时钟源 */
   rcc_periph_clk_init.RngClockSelection = RCC_RNGCLKSOURCE_PLL;
   /*RNG 时钟源选择 PLL*/
   HAL_RCCEx_PeriphCLKConfig(&rcc_periph_clk_init);
   __HAL_RCC_RNG_CLK_ENABLE();/* 使能 RNG 时钟 */
}
```

最后两个是我们获取随机数函数，它们的定义如下：

```
/**
 * @brief       得到随机数
 * @param       无
 * @retval      获取 到的随机数(32bit)
 */
uint32_t rng_get_random_num(void)
{
    uint32_t randomnum;
    HAL_RNG_GenerateRandomNumber(&g_rng_handle, &randomnum);
    return randomnum;
}

/**
 * @brief       得到某个范围内的随机数
 * @param       min,max: 最小,最大值.
 * @retval      得到的随机数(rval),满足:min<=rval<=max
 */
int rng_get_random_range(int min, int max)
{
    uint32_t randomnum;
    HAL_RNG_GenerateRandomNumber(&g_rng_handle, &randomnum);
    return randomnum % (max - min + 1) + min;
}
```

其中 rng_get_random_num 用于读取随机数值，而 rng_get_random_range 用于读取一个特定范围内的随机数，实际上它们都是通过调用的函数 HAL_RNG_GenerateRandomNumber 来实现的。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint32_t random;
    uint8_t t = 0, key;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "RNG TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    while (rng_init())  /* 初始化随机数发生器 */
    {
        lcd_show_string(30, 130, 200, 16, 16, "  RNG Error! ", RED);
        delay_ms(200);
        lcd_show_string(30, 130, 200, 16, 16, "RNG Trying...", RED);
    }

    lcd_show_string(30, 110, 200, 16, 16, "RNG Ready!   ", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEY0:Get Random Num", RED);
    lcd_show_string(30, 150, 200, 16, 16, "Random Num:", RED);
    lcd_show_string(30, 180, 200, 16, 16, "Random Num[0-9]:", RED);

    while (1)
    {
        key = key_scan(0);

        if (key == KEY0_PRES)
        {
            random = rng_get_random_num();
            lcd_show_num(30 + 8 * 11, 150, random, 10, 16, BLUE);
        }

        if ((t % 20) == 0)
        {
            LED0_TOGGLE();                          /* 每 200ms,翻转一次 LED0 */
            random = rng_get_random_range(0, 9);    /* 取[0,9]区间的随机数 */
            lcd_show_num(30 + 8 * 16, 180, random, 1, 16, BLUE);/* 显示随机数 */
        }

        delay_ms(10);
        t++;
    }
}
```

该部分代码也比较简单，在所有外设初始化成功后，进入死循环，等待按键按下，如果KEY0按下，则调用rng_get_random_num函数，读取随机数值，并将读到的随机数显示在LCD上面。每隔 200ms 获取一次区间[0,9]的随机数，并实时显示在液晶上。同时 LED0，周期性闪烁，400ms 闪烁一次。

最后，为了方便测试，我们将rng_get_random_num和rng_get_random_range加入USMART，

修改 usmart_nametab 如下：

```
struct _m_usmart_nametab usmart_nametab[] =
{
#if USMART_USE_WRFUNS == 1      /* 如果使能了读写操作 */
    (void *)read_addr, "uint32_t read_addr(uint32_t addr)",
    (void *)write_addr, "void write_addr(uint32_t addr,uint32_t val)",
#endif
    (void *)delay_ms, "void delay_ms(uint16_t nms)",
    (void *)delay_us, "void delay_us(uint32_t nus)",

    (void *)rng_get_random_num, "uint32_t rng_get_random_num(void)",
    (void *)rng_get_random_range, "int rng_get_random_range(int min,int max)",
};
```

这样，我们便可以在串口输入我们想要调用的函数（rng_get_random_range 或 rng_get_random_num）进行测试。

30.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。然后我们按下 KEY0，就可以在屏幕上看到获取到的随机数。同时，就算不按 KEY0，程序也会自动的获取 0~9 区间的随机数显示在 LCD 上面。实验结果如图 30.4.1 所示：

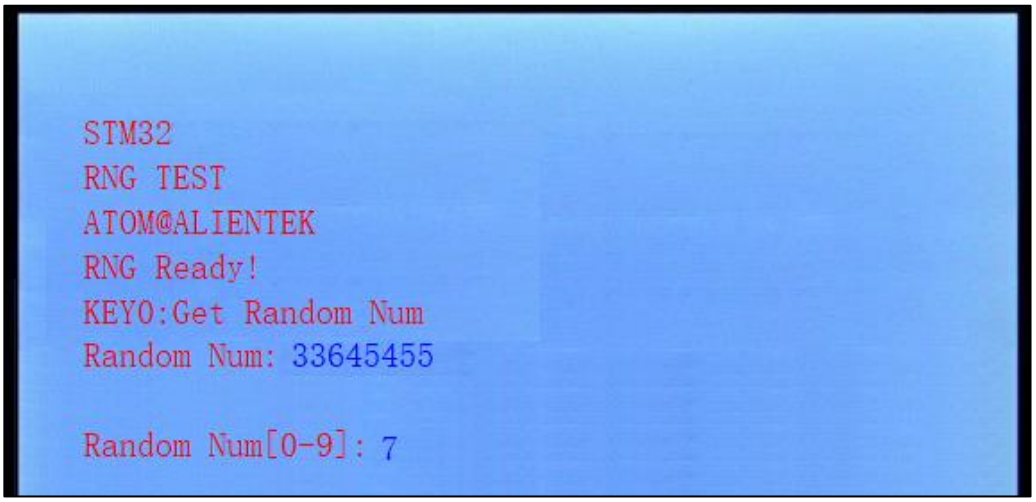


图 30.4.1 获取随机数成功

然后，我们还可以在串口调试助手里面调用 rng_get_random_num 和 rng_get_random_range 函数，测试这两个函数的功能，大家可以自行测试。

第三十一章 低功耗实验

本章，我们将介绍 STM32H743 的电源控制（PWR），并实现低功耗模式相关功能。我们将通过四个实验来学习并实现低功耗相关功能，分别是 PVD 电压监控实验、睡眠模式实验、停止模式实验和待机模式实验。

本章分为如下几个小节：

31.1 电源控制（PWR）简介
31.2 PVD 电压监控实验
31.3 睡眠模式实验
31.4 停止模式实验
31.5 待机模式实验

31.1 电源控制（PWR）简介

电源控制部分（PWR）概述了不同电源域的电源架构以及电源配置控制器。PWR 的内容比较多，我们把它们的主要特性概括为以下 3 点：

电源系统：USB 稳压器、内核域(V_{CORE})、VDD 域、备份域、模拟域（V_{DDA}）。

电源监控：POR/PDR 监控器、BOR 监控器、PVD 监控器、AVD 监控器、VBAT 阈值、温度阈值。

电源管理：VBAT 电池充电、工作模式、电压调节控制、低功耗模式。

下面将分别对这 3 个特性进行简单介绍。

31.1.1 电源系统

为了方便对电源系统进行管理，设计者把 STM32 的内核和外设等器件根据功能划分了不同的电源区域，具体如图 31.1.1.1 所示。

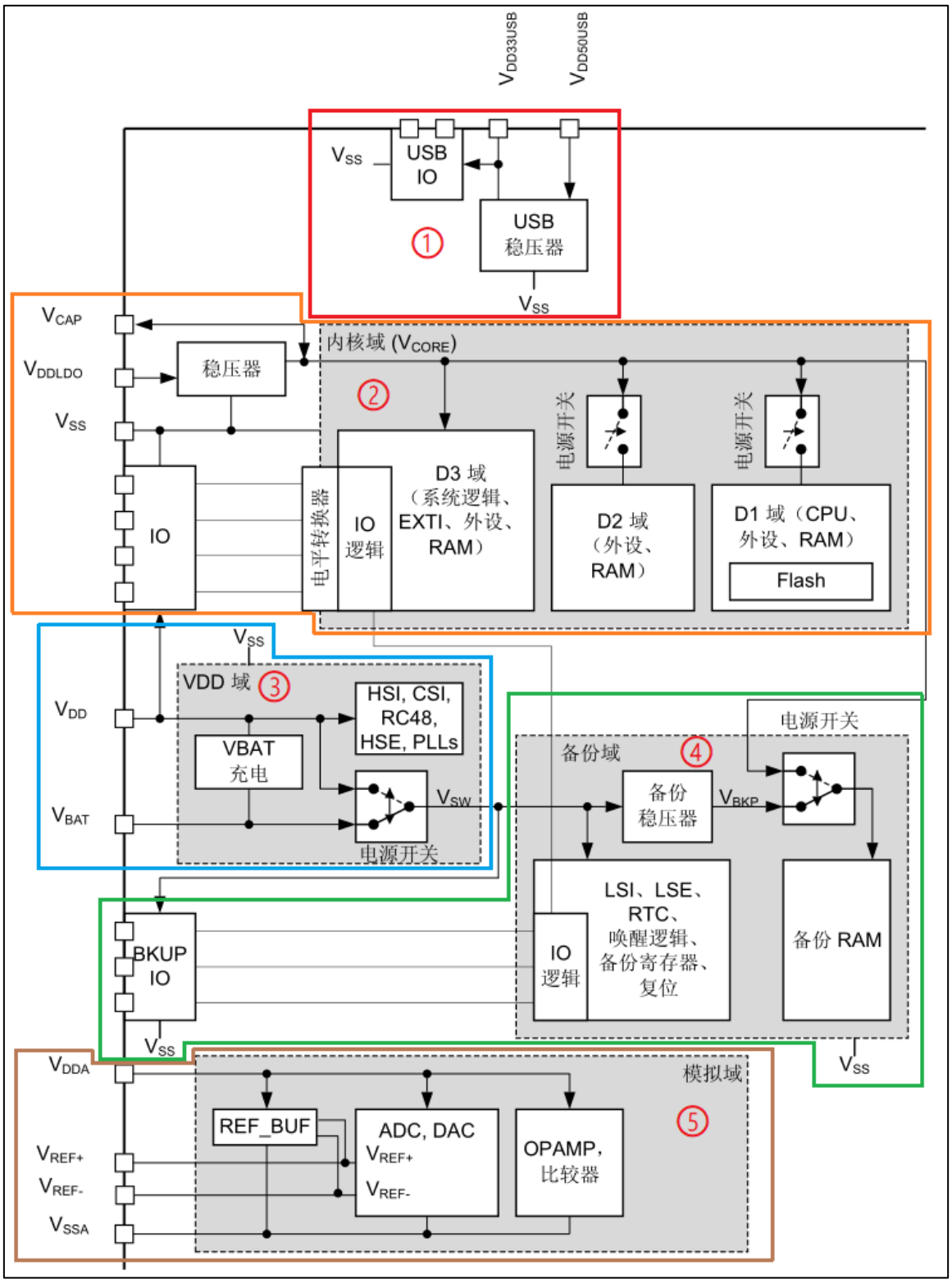


图 31.1.1.1 电源概述框图

在电源概述框图中我们划分了 5 个区域①②③④⑤，分别是 USB 稳压器域、内核域、VDD 域、备份域和模拟域。下面分别进行简单介绍：

① USB 稳压器域

V_{SS} 是所有电源和模拟稳压器的公共地。

$V_{DD50USB}$ 为 USB 稳压器供电的外部电源。

$V_{DD33USB}$ 为 USB 接口供电的 USB 稳压器供电输出。

1）当 USB 稳压器使能时，$V_{DD33USB}$ 由内部 USB 稳压器提供。

2）当 USB 稳压器禁止时，$V_{DD33USB}$ 由独立的外部电源输入提供。

② 内核域(V_{CORE})

V_{DDLDO} 是稳压器供电的外部电源。

V_{CAP} 数字内核域电源，该电源独立于所有其它电源：

1）当稳压器使能时，VCORE 由内部稳压器提供。

2）当稳压器禁止时，VCORE 由外部电源通过 VCAP 引脚提供。

V_{CORE} 内核域电源可通过稳压器或者外部电源（V_{CAP}）供电。V_{CORE} 为除备份域和待机电路以外的所有数字电路供电。V_{CORE} 域分为 3 个部分：

1、D1 域（CPU (Cortex-M7)、外设、RAM 和 Flash）。

2、D2 域（外设、RAM）。

3、D3 域（系统逻辑、EXTI、低功耗外设、RAM 和 IO 逻辑）。

当发生系统复位时，稳压器使能并为 V_{CORE} 供电。稳压器的输出电压为 1.2V（M4/M7），如果是 M3，则是 1.8V。稳压器提供三种不同的工作模式：主 (MR) 模式、低功耗模式 (LP) 或关闭模式。这些模式将根据系统工作模式（运行、停止和待机）进行使用，详细如下：

1、运行模式：稳压器工作在主模式，并为 V_{CORE} 域（内核、存储器和数字外设）提供全功率。稳压器输出电源可通过软件调节为不同电压级别（VOS1、VOS2 和 VOS3），这些级别通过 PWR D3 域控制寄存器(PWR_D3CR) 中的 VOS 位配置。

2、停止模式：V_{CORE} 域的所有时钟都被关闭，相应的外设都停止了工作，但稳压器还会为 V_{CORE} 供电以保存内核寄存器和内部存储器（SRAM）的内容。稳压器模式通过 PWR 控制寄存器 1 (PWR_CR1) 中的 SVOS 和 LPDS 位选择。如果选择了 SVOS3 电压调节，则可以选择主模式或低功耗模式；如果选择 SVOS4 和 SVOS5 调节，则只能选择低功耗模式。由于 SVOS4 和 SVOS5 调节的电压级别低，因此可进一步降低停止模式的功耗。

3、待机模式：

稳压器关闭且 V_{CORE} 域掉电。除待机电路和备份域外，内核寄存器和内部存储器（SRAM）的内容都将丢失。

③ VDD 域

V_{DD} 为 I/O 和系统模拟模块（如复位、电源管理和时钟）供电的外部电源。

V_{BAT} 是后备电源（来源于开发板上 3V 的纽扣电池），当 V_{DD} 不存在时（开发板断电），V_{BAT} 为备份域供电。

④备份域

备份域的电源自来 VSW，VSW 来自 VDD 域的 VDD 或者 VBAT。在备份域中，包含 LSI、LSE、RTC、唤醒逻辑、备份寄存器、复位以及备份 SRAM 等器件。

⑤模拟域

V_{DDA} 为 ADC、DAC、OPAMP、比较器和电压参考缓冲器供电的外部模拟电源。该电源独立于所有其它电源。

V_{SSA} 是独立的模拟和参考电压地。

V_{REF+} 是 ADC 和 DAC 的外部参考电压。

1）当电压参考缓冲器使能时，VREF+ 和 VREF- 由内部电压参考缓冲器提供。

2）当电压参考缓冲器禁止时，VREF+ 由独立的外部参考电源提供。

31.1.2 电源监控

电源监控的部分我们主要关注 PVD 监控器，实验 19-1 就是根据该监控器进行的，此外还需要知道上电复位(POR)/掉电复位(PDR)和欠压复位(BOR)。其他部分的内容请大家查看《STM32H7xx 参考手册_V7（英文版）.pdf》第 6.5 节（266 页）。

- **上电复位(POR)/掉电复位(PDR)**

上电时，当 V_{DD} 低于指定 V_{POR} 阈值时，系统无需外部复位电路便会保持复位模式。一旦 V_{DD} 电源电压高于 V_{POR} 阈值，系统便会退出复位状态。掉电时，当 V_{DD} 低于指定 V_{PDR} 阈值时，系统就会保持复位模式。如图 31.1.2.1 所示，pwr_por_rst 为上电复位信号。

注意：POR 与 PDR 的复位电压阈值是固定的，V_{POR} 阈值（典型值）为 1.72V，V_{PDR} 阈值（典型值）为 1.68V。

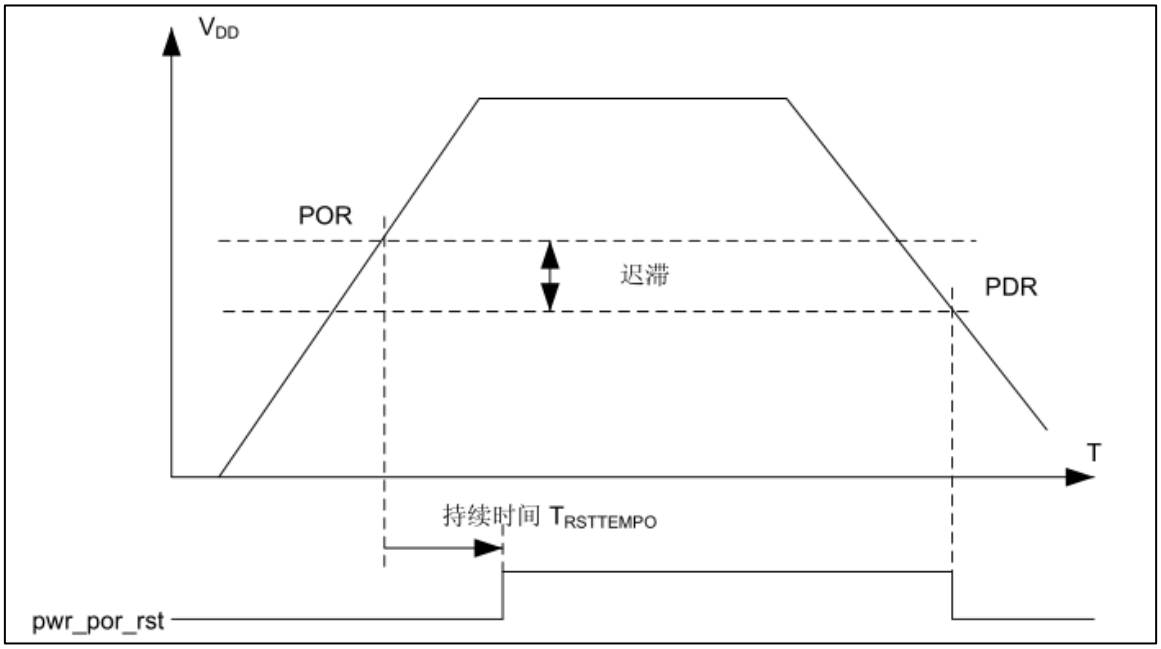


图 31.1.2.1 上电复位/掉电复位波形

- **欠压复位(BOR)**

上电期间，欠压复位(BOR)将使系统保持复位状态，直到 V_{DD} 电源电压达到指定的 V_{BOR} 阈值。V_{BOR} 阈值通过系统选项字节（某些寄存器的 BOR_LEV 位）进行配置。默认情况下，BOR 关闭。可选择以下可编程 V_{BOR} 阈值：

Symbol	Parameter	Conditions	Min	Typ	Max	Unit
$t_{RSTTEMPO}$(1)	Reset temporization after BOR0 released	-	-	377	-	µs
$V_{BOR0/POR/PDR}$	Brown-out reset threshold 0 (V_{POR}/V_{PDR} thresholds)	Rising edge(1)	1.62	1.67	1.71	
		Falling edge	1.58	1.62	1.68	
V_{BOR1}	Brown-out reset threshold 1	Rising edge	2.04	2.10	2.15	
		Falling edge	1.95	2.00	2.06	
V_{BOR2}	Brown-out reset threshold 2	Rising edge	2.34	2.41	2.47	
		Falling edge	2.25	2.31	2.37	
V_{BOR3}	Brown-out reset threshold 3	Rising edge	2.63	2.70	2.78	
		Falling edge	2.54	2.61	2.68	

表 31.1.2.1 BOR 欠压阀值等级

该表截取于《STM32H743IIT6.pdf》手册的 214 页。

欠压复位的描述波形图如图 31.1.2.2 所示，pwr_bor_rst 为欠压复位信号。

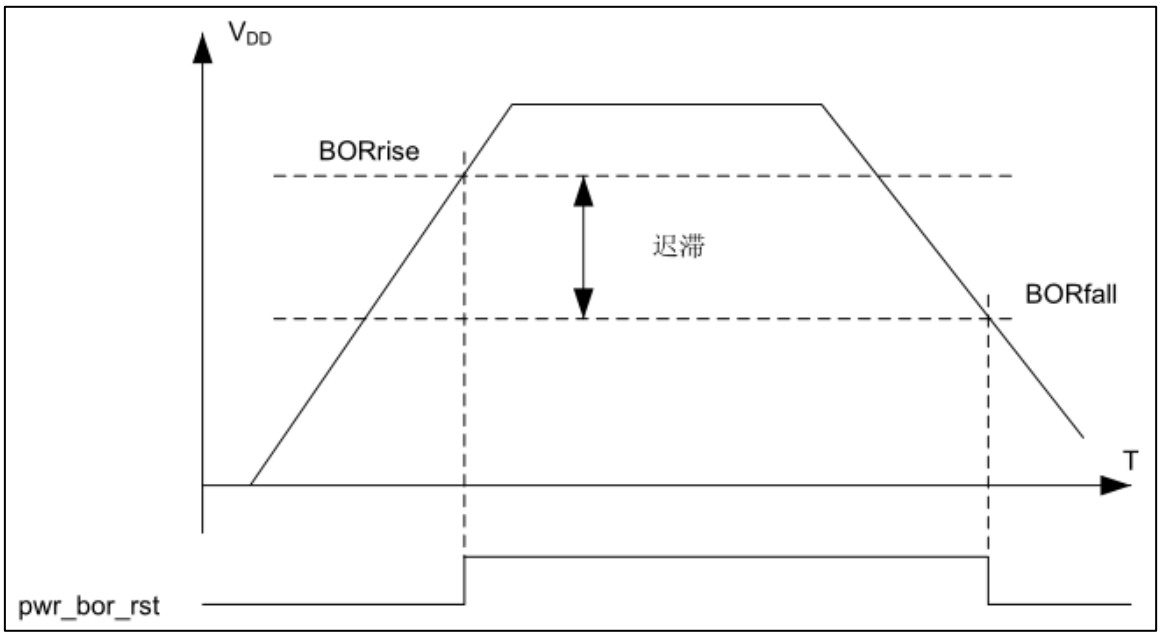


图 31.1.2.2 欠压复位波形

- **可编程电压检测器(PVD)**

上面介绍的 POR、PDR 以及 BOR 功能都是设置电压阈值与外部供电电压 V_{DD} 比较，当 V_{DD} 低于设置的电压阈值时，就会直接进入复位状态，防止电压不足导致的误操作。

下面介绍可编程电压检测器（PVD），它可以实时监视 V_{DD} 的电压，方法是将 V_{DD} 与 PWR 控制寄存器 1（PWR_CR1）中的 PLS[2:0]位所选的 V_{PVD} 阈值进行比较。当检测到电压低于 V_{PVD} 阈值时，如果使能 EXTI16 线中断，即使能 PVD & AVD 中断（可参考表 15.1.2.1 知道 EXTI16 线内部连接 PVD 中断事件），可以产生 PVD 中断，具体取决于 EXTI16 线配置为检测上升还是下降沿，然后在复位前，在中断服务程序中执行紧急关闭系统等任务。PVD 阀值检测波形，如图 31.1.2.1 所示。

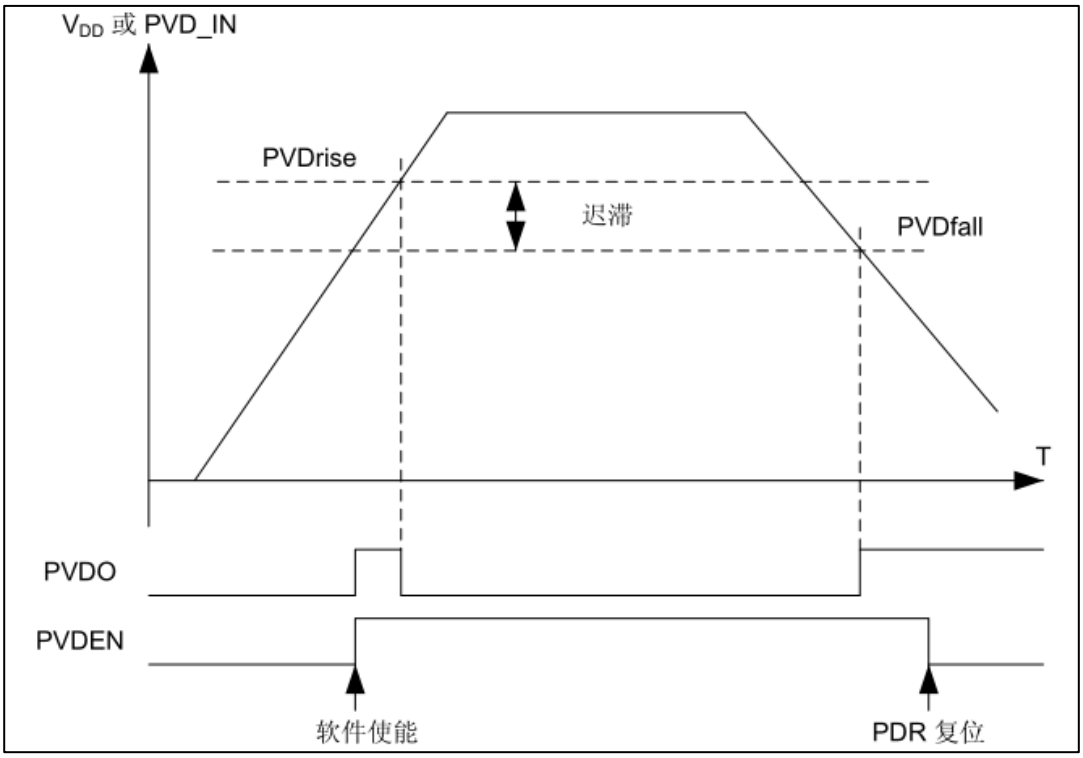


图 31.1.2.3 PVD 检测波形

PVD 阀值有 8 个等级，有上升沿和下降沿的区别，分别就是图 31.1.2.3 中 PVDrise 电压为

上升沿阀值，PVDfall 为下降沿阀值，具体如表 31.1.2.2 所示。

Symbol	Parameter	Conditions	Min	Typ	Max	Unit
V_{PVD0}	Programmable Voltage Detector threshold 0	Rising edge	1.90	1.96	2.01	
		Falling edge	1.81	1.86	1.91	
V_{PVD1}	Programmable Voltage Detector threshold 1	Rising edge	2.05	2.10	2.16	
		Falling edge	1.96	2.01	2.06	
V_{PVD2}	Programmable Voltage Detector threshold 2	Rising edge	2.19	2.26	2.32	
		Falling edge	2.10	2.15	2.21	
V_{PVD3}	Programmable Voltage Detector threshold 3	Rising edge	2.35	2.41	2.47	
		Falling edge	2.25	2.31	2.37	
V_{PVD4}	Programmable Voltage Detector threshold 4	Rising edge	2.49	2.56	2.62	
		Falling edge	2.39	2.45	2.51	
V_{PVD5}	Programmable Voltage Detector threshold 5	Rising edge	2.64	2.71	2.78	
		Falling edge	2.55	2.61	2.68	
V_{PVD6}	Programmable Voltage Detector threshold 6	Rising edge	2.78	2.86	2.94	
		Falling edge in Run mode	2.69	2.76	2.83	

表 31.1.2.2 PVD 阀值等级

该表截取于《STM32H743IIT6.pdf》手册的 214 页。

表 31.1.2.2 中只有 7 个 PVD 阀值等级，最后一个就是 PVD_IN 引脚上的外部电压级别（与内部 V_{REFINT} 值相比较）。

31.1.3 电源管理

电源管理的部分我们主要关注低功耗模式，其他部分的内容请自行查看手册。

很多单片机都有低功耗模式，STM32 也不例外。STM32H743 提供了 6 种低功耗模式，以达到不同层次的降低功耗的目的，这六种模式如下：

1、CSleep（CPU 时钟停止，所有的外设仍可以运行）；

2、CStop（CPU 时钟停止，大多数 CPU 子系统时钟停止，所以大部分外设也停止工作）；

3、DStop（域总线矩阵时钟停止，D1、D2 域进入停止模式）

4、停止（系统时钟停止，D3 域进入停止模式，D1 和 D2 域进入停止或者待机模式）

5、DStandby（域掉电，D1/D2/D3 进入待机模式）

6、待机（系统掉电，达到最低功耗）

在这六种低功耗模式中，最低功耗的是待机模式，在此模式下，最低只需要 2uA 左右的电流。停止模式是次低功耗的，其典型的电流消耗在 170uA 左右。最后就是睡眠模式了。用户可以根据最低电源消耗，最快速启动时间和可用的唤醒源等条件，选定一个最佳的低功耗模式。

下面是低功耗模式汇总介绍，如下表所示。

系统	域	CPU	进入	唤醒	系统时钟	系统时钟	域总线矩阵时钟	外设时钟	CPU 时钟	调压器	域电源
运行	DRUN(1)	CRun	-	-	开启	开启	开启	开启	开启	开启	开启
		CSleep	WFI 或从 ISR 或 WFE 返回	任何中断或事件				开启 / 关闭(2)	关闭		
	DStop(3)	CStop	SLEEPDEEP 位 + WFI 或者从 ISR 或 WFE 返回	任何 EXTI 中断或事件			关闭	开启 / 关闭(4)			开启
	DStandby(3)										关闭
Stop(5)	DStop(3)		SLEEPDEEP 位 + WFI 或者从 ISR 或 WFE 返回或者清除唤醒源(6)		开启 / 关闭(7)	关闭		关闭			开启
	DStandby(3)										关闭
待机(8)	DStandby(3)		所有 PDDS_Dn 位 + SLEEPDEEP 位 + WFI 或者从 ISR 或 WFE 返回或者清除唤醒源(6)	WKUP 引脚上升沿或下降沿、RTC 闹钟（闹钟 A 或闹钟 B）、RTC 唤醒事件、RTC 入侵事件、RTC 时间戳事件、NRST 引脚外部复位、IWDG 复位	关闭					关闭	

表 31.1.3.1 低功耗模式汇总

下面对睡眠模式、停止模式和待机模式的进入及退出方法进行介绍。

1、睡眠模式

进入睡眠模式，CPU 时钟关闭，但是其他所有的外设仍可以运行，所以任何中断或事件都可以唤醒睡眠模式。有两种方式进入睡眠模式，这两种方式进入的睡眠模式唤醒的方法不同，分别是 WFI(wait for interrupt)和 WFE(wait for event)，即由等待“中断”唤醒和由“事件”唤醒。下面我们看看睡眠模式进入及退出方法：

睡眠模式	说明
进入模式	WFI（等待中断）或 WFE（等待事件），条件为： 1、CM7 系统控制寄存器中的 SLEEPDEEP 位置 0 2、没有中断（针对 WFI）和事件（针对 WFE）挂起
	从 ISR 返回，条件为： 1、CM7 系统控制寄存器中的 SLEEPDEEP 位置 0，及 SLEEPONEXIT 位置 1 2、没有中断和事件挂起
退出模式	如果使用 WFI 或从 ISR 返回进入：在 NVIC 中使能的任何中断都可以 如果使用 WFE 进入且 SEVONPEND = 0：任何事件都可以 如果使用 WFE 进入且 SEVONPEND = 1：任何中断（即使在 NVIC 中禁止时）
唤醒延迟	无延时

表 31.1.3.2 睡眠模式进入及退出方法

2、停止模式

进入停止模式，所有的时钟都关闭，于是所有的外设也停止了工作。但是内核域的 V_{CORE} 电源是没有关闭的，所以内核的寄存器和内存信息都保留下来，等待重新开启时钟就可以从上次停止的地方继续执行程序。停止模式可以被任何一个外部中断(EXTI)或事件唤醒。在停止模式中可以选择稳压器的工作方式为主模式或者低功耗模式。下面我们看看停止模式进入及退出方法：

停止模式	说明

进入模式	WFI（等待中断）或 WFE（等待事件），条件为： 1、 CM7 系统控制寄存器中的 SLEEPDEEP 位置 1 2、没有中断（针对 WFI）和事件（针对 WFE）挂起 3、所有 CPU EXTI 唤醒源清除
	从 ISR 返回，条件为： 1、CM7 系统控制寄存器中的 SLEEPDEEP 位置 1，及 SLEEPONEXIT 位置 1 2、没有中断和事件挂起 3、所有 CPU EXTI 唤醒源清除
退出模式	如果使用 WFI 或从 ISR 返回进入：在 NVIC 中使能的任意 EXTI 线中断 如果使用 WFE 进入且 SEVONPEND = 0：EXTI 事件 如果使用 WFE 进入且 SEVONPEND = 1：EXTI 中断（即使在 NVIC 中禁止）
唤醒延迟	1、为达到 VOS3（默认电压）而需要的稳压器稳定时间。 2、系统时钟重启延时（HSI/CSI 重启延时） 3、EXTI 和 RCC 唤醒同步

表 31.1.3.3 停止模式进入及退出方法

3、待机模式

待机模式可实现最低功耗。该模式是在 CM7 深睡眠模式时关闭稳压器（V_{COER} 关闭），PLL、HSI、CSI、HSI48 和 HSE 振荡器也被断电。除备份域（RTC 寄存器、RTC 备份寄存器和备份 SRAM）和待机电路中的寄存器外，SRAM 和寄存器内容都将丢失。不过如果我们使能了备份区域（备份 SRAM、RTC、LSE），那么待机模式下的功耗，将达到 6uA 左右。下面我们看看待机模式进入及退出方法：

待机模式	说明
进入模式	WFI（等待中断）或 WFE（等待事件），条件为： 1、CM7 系统控制寄存器中的 SLEEPDEEP 位置 1 2、没有中断（针对 WFI）和事件（针对 WFE）挂起 3、所有 WKUPF 位清零 4、RUN_D3 位清 5、PDDS_D1、PDDS_D2、PDDS_D3 全设置为 1
	从 ISR 返回，条件为： 1、CM7 系统控制寄存器中的 SLEEPDEEP 和 SLEEPONEXIT 位置 1 2、没有中断挂起 3、所有 WKUPF 位清零 4、RUN_D3 位清零 5、PDDS_D1、PDDS_D2、PDDS_D3 全设置为 1
退出模式	WKUP 脚的上升沿或下降、RTC 闹钟（闹钟 A 或闹钟 B）、RTC 唤醒事件、入侵事件、时间戳事件、硬件复位、IWDG 复位
唤醒延迟	系统复位阶段

表 31.1.3.4 待机模式进入及退出方法

从待机模式唤醒后的代码执行等同于复位后的执行(采样启动模式引脚，读取复位向量等)。在进入待机模式后，除了复位引脚、RTC_TAMP1/2/3 引脚（PC13/PI8/PC1）（如果针对入侵、时间戳、RTC 闹钟输出或 RTC 时钟校准输出进行了配置）和 WK_UP（PA0/PA2/PC1/PC13/PI8/PI11）（如果使能了）等引脚外，其他所有 IO 引脚都将处于高阻态。

下面开始本章的四个实验的介绍。

31.2 PVD 电压监控实验

本小节我们来学习 PVD 电压监控实验，该部分的知识点内容请回顾 31.1.2 电源监控。我们直接从寄存器介绍开始。

31.2.1 PWR 寄存器

本实验用到 PWR 的部分寄存器，在《STM32H7xx 参考手册_V3（中文版）.pdf》手册的 6.8 小节（247 页）可以找到 PWR 的寄存器描述。这里我们只介绍 PVD 电压监控实验我们用到的 PWR 的控制寄存器 1（PWR_CR1），还有就是我们要用到 EXTI16 线中断，所以还要配置 EXTI 相关的寄存器，具体如下：

- **PWR 控制寄存器 1（PWR_CR1）**

PWR 控制寄存器 1 描述如图 31.2.1.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALS		AVDEN
													rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SVOS		Res.	Res.	Res.	Res.	FLPS	DBP	PLS			PVDE	Res.	Res.	Res.	LPDS
rw	rw					rw	rw	rw	rw	rw	rw				rw

位 7:5 **PLS**：可编程电压检测器的电平选择 (Programmable voltage detector level selection)
这些位用于选择通过 PVD 检测的电压阈值。
000：1.95 V
001：2.1 V
010：2.25 V
011：2.4 V
100：2.55 V
101：2.7 V
110：2.85 V
111：PVD_IN 引脚上的外部电压级别（与内部 V_{REFINT} 值相比较）。
注： 有关详细信息，请参见产品数据手册的“电气特性”部分。

位 4 **PVDE**：可编程电压检测器使能 (Programmable voltage detector enable)
0：禁止可编程电压检测器。
1：使能可编程电压检测器。

图 31.2.1.1 PWR_CR1 寄存器（部分）

位[7:5] PLS 用于设置 PVD 检测的电压阀值，即前面我们介绍 PVD 的 8 个等级阀值选择。位 4 PVDE 位，用于使能或者禁止 PVD 检测，显然我们要使能 PVD 检测，该位置 1。

- **EXTI 中断屏蔽寄存器（EXTI_CPUIMR1）**

EXTI 中断屏蔽寄存器描述如图 31.2.1.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MR31	MR30	MR29	MR28	MR27	MR26	MR25	MR24	MR23	MR22	MR21	MR20	MR19	MR18	MR17	MR16
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:22 **MRx**：直接事件输入 x 上的 CPU 中断屏蔽 (CPU interrupt Mask on Direct Event input x)(1)
0：屏蔽来自 x 线的中断请求
1：取消屏蔽来自线 x 的中断请求

位 21:0 **MRx**：可配置事件输入 x 上的 CPU 中断屏蔽 (CPU interrupt Mask on Configurable Event input x) (2)
0：屏蔽来自 x 线的中断请求
1：取消屏蔽来自线 x 的中断请求

图 31.2.1.2 EXTI_CPUIMR1 寄存器

我们要使用到 EXTI16 线中断，所以 MR16 位要置 1，即取消屏蔽 EXTI16 线的中断请求。

- **EXTI 上升沿触发选择寄存器（EXTI_RTSR1）**

EXTI 上升沿触发选择寄存器描述如图 31.2.1.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR21	TR20	TR19	TR18	TR17	TR16
										rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:22 保留，必须保持复位值。

位 21:0 **TRx**：可配置事件输入 x 的上升沿触发事件配置位 (Rising trigger event configuration bit of Configurable Event input x)(1)

0：禁止输入线上升沿触发（事件和中断）
1：允许输入线上升沿触发（事件和中断）

图 31.2.1.3 EXTI_RTSR1 寄存器

我们要使用到 EXTI16 线中断，所以 TR16 位要置 1。

- **EXTI 下降沿触发选择寄存器（EXTI_FTSR1）**

EXTI 下降沿触发选择寄存器描述如图 31.2.1.4 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TR21	TR20	TR19	TR18	TR17	TR16
										rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TR15	TR14	TR13	TR12	TR11	TR10	TR9	TR8	TR7	TR6	TR5	TR4	TR3	TR2	TR1	TR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:22 保留，必须保持复位值。

位 21:0 **TRx**：可配置事件输入 x 的下降沿触发事件配置位 (Falling trigger event configuration bit of Configurable Event input x)(1)

0：禁止输入线下降沿触发（事件和中断）
1：允许输入线下降沿触发（事件和中断）。

图 31.2.1.4 EXTI_FTSR1 寄存器

我们要使用到 EXTI16 线中断，所以 TR16 位要置 1。

- **EXTI 挂起寄存器（EXTI_CPUPR1）**

EXTI 挂起寄存器描述如图 31.2.1.5 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PR21	PR20	PR19	PR18	PR17	PR16
										rc1	rc1	rc1	rc1	rc1	rc1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PR15	PR14	PR13	PR12	PR11	PR10	PR9	PR8	PR7	PR6	PR5	PR4	PR3	PR2	PR1	PR0
rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1	rc1

位 31:22 保留，必须保持复位值。

位 21:0 **PRx**：可配置事件输入 x 挂起位 (Configurable event inputs x Pending bit)

0：未发生触发请求
1：发生了选择的触发请求
当在外部中断线上发生了选择的边沿事件，该位被置“1”。向此位写入 1 可将其清除，也可以通过更改边沿检测器的敏感性将其清除。

图 31.2.1.5 EXTI_CPUPR1 寄存器

EXTI 挂起寄存器 EXTI_CPUPR1 管理的是 EXTI0 线到 EXTI21 线的中断标志位。在 PVD 中断服务函数里面，我们记得要对 PR16 位写 1，来清除 EXTI16 线的中断标志。

31.2.2 硬件设计

1. 例程功能

开发板供电正常的话，LCD 屏会显示"PVD Voltage OK!"。当供电电压过低，则会通过 PVD 中断服务函数将 LED1 点亮；当供电电压正常后，会在 PVD 中断服务函数将 LED1 熄灭。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
LED1：LED1 - PB0
2）PVD(可编程电压监测器)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)

3. 原理图

PVD 属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们通过 LED1 和 LCD 来指示进入 PVD 中断的情况。

31.2.3 程序设计

31.2.3.1 PWR 的 HAL 库驱动

PWR 在 HAL 库中的驱动代码在 stm32h7xx_hal_pwr.c 文件（及其头文件）中。

1. HAL_PWR_ConfigPVD 函数

PVD 的初始化函数，其声明如下：

```
void HAL_PWR_ConfigPVD (PWR_PVDTypeDef *sConfigPVD);
```

- **函数描述：**
  用于初始化 PWR。
- **函数形参：**
  形参 1 是 PWR_PVDTypeDef 结构体类型指针变量，其定义如下：

```
typedef struct
{
  uint32_t PVDLevel;    /* 指定 PVD 检测级别 */
  uint32_t Mode;        /* 指定 PVD 的 EXTI 检测模式 */
}PWR_PVDTypeDef;
```

1）PVDLevel：指向 PVD 检测级别，对应 PWR_CR1 寄存器的 PLS 位的设置，取值范围 PWR_PVDLEVEL_0 到 PWR_PVDLEVEL_7，共八个级别。
2）Mode：指定 PVD 的 EXTI 边沿检测模式。

- **函数返回值：**
  无

PVD 电压监控配置步骤

1）配置 PVD

调用 HAL_PWR_ConfigPVD 函数配置 PVD，包括检测电压级别、使用中断线的什么边沿缘触发等。

2）使能 PVD 检测，配置 PVD/AVD 中断优先级，开启 PVD/AVD 中断

通过 HAL_PWR_EnablePVD 函数使能 PVD 检测。
通过 HAL_NVIC_EnableIRQ 函数使能 PVD/AVD 中断。
通过 HAL_NVIC_SetPriority 函数设置中断优先级。

3）编写中断服务函数

PVD/AVD 中断服务函数为：PVD_AVD_IRQHandler，当发生中断的时候，程序就会执行中断服务函数。HAL 库有专门的 PVD/AVD 中断处理函数，我们只需要在 PVD/AVD 中断服务函数里面调用 HAL_PWR_PVD_IRQHandler 函数，然后逻辑代码在 PVD/AVD 中断服务回调函数 HAL_PWR_PVDCallback 中编写，详见本例程源码。

31.2.3.2 程序流程图

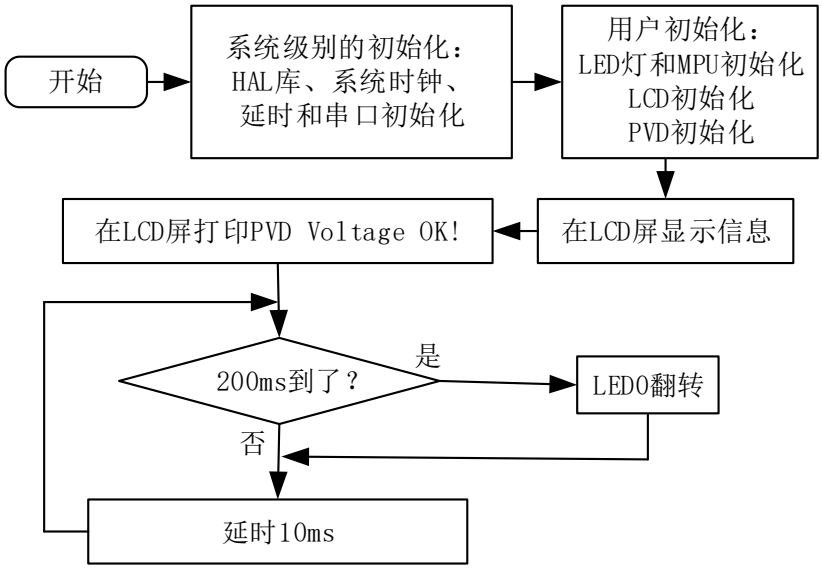


图 31.2.3.2.1 PVD 电压监控实验程序流程图

31.2.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。PWR 源码包括两个文件：pwr.c 和 pwr.h。该章节有四个实验，每一个实验的代码都是在上一个实验后面追加。

pwr.h 头文件只有函数声明，下面直接开始介绍 pwr.c 的程序，首先是 PVD 初始化函数。

```
/**
 * @brief       初始化 PVD 电压监视器
 * @param       pls: 电压等级
 *   @arg       PWR_PVDLEVEL_0,1.95V;   PWR_PVDLEVEL_1,2.1V
 *   @arg       PWR_PVDLEVEL_2,2.25V;   PWR_PVDLEVEL_3,2.4V;
 *   @arg       PWR_PVDLEVEL_4,2.55V;   PWR_PVDLEVEL_5,2.7V;
 *   @arg       PWR_PVDLEVEL_6,2.85V;   PWR_PVDLEVEL_7,使用 PVD_IN 脚上的电压(与
                                                          Vrefint 比较)
 * @retval          无
 */
void pwr_pvd_init(uint32_t pls)
{
    PWR_PVDTypeDef pvd_handle = {0};

    HAL_PWR_EnablePVD();                        /* 使能 PVD 时钟 */

    pvd_handle.PVDLevel = pls;                  /* 检测电压级别 */
    /* 使用中断线的上升沿和下降沿双边缘触发 */
    pvd_handle.Mode = PWR_PVD_MODE_IT_RISING_FALLING;
    HAL_PWR_ConfigPVD(&pvd_handle);

    HAL_NVIC_SetPriority(PVD_AVD_IRQn, 3 ,3);
    HAL_NVIC_EnableIRQ(PVD_AVD_IRQn);
    HAL_PWR_EnablePVD();                        /* 使能 PVD 检测 */
}
```

这里需要注意的就是 PVD 中断线选择的是上升沿和下降沿双边沿触发，其他的内容前面

已经讲过。

下面介绍的是 PVD/AVD 中断服务函数及其回调函数，函数定义如下：

```
/**
 * @brief       PVD/AVD 中断服务函数
 * @param       无
 * @retval      无
 */
void PVD_AVD_IRQHandler(void)
{
    HAL_PWR_PVD_IRQHandler();
}

/**
 * @brief       PVD/AVD 中断服务回调函数
 * @param       无
 * @retval      无
 */
void HAL_PWR_PVDCallback(void)
{
    if (__HAL_PWR_GET_FLAG(PWR_FLAG_PVDO))    /* 电压比 PLS 所选电压还低 */
    {
        /* LCD 显示电压低 */
        lcd_show_string(30, 130, 200, 16, 16, "PVD Low Voltage!", RED);
        LED1(0);    /* 点亮绿灯，表明电压低了 */
    }
    else
    {
        /* LCD 显示电压正常 */
        lcd_show_string(30, 130, 200, 16, 16, "PVD Voltage OK! ", BLUE);
        LED1(1);    /* 灭掉绿灯 */
    }
}
```

HAL_PWR_PVDCallback 回调函数中首先是判断 V_{DD} 电压是否比 PLS 所选电压还低，是的话，就在 LCD 显示 PVD Low Voltage!并且点亮 LED1，否则，在 LCD 显示 PVD Voltage OK!并且关闭 LED1。

在 main 函数里面编写如下代码：

```
int main(void)
{
    uint8_t t = 0;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟，400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    pwr_pvd_init(PWR_PVDLEVEL_6);           /* PVD 2.85V 检测 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "PVD TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    /* 默认 LCD 显示电压正常 */
    lcd_show_string(30, 110, 200, 16, 16, "PVD Voltage OK! ", BLUE);

    while (1)
    {
        if ((t % 20) == 0)
        {
```

```
            LED0_TOGGLE();   /* 每 200ms,翻转一次 LED0 */
        }

        delay_ms(10);
        t++;
    }
}
```

这里我们选择 PVD 的检测电压阀值为 2.9V，其他的代码很好理解，最后下载验证一下。

31.2.4 下载验证

下载代码后，默认 LCD 屏会显示"PVD Voltage OK!"，当供电电压过低，则 LED1 会点亮，并且 LCD 屏会显示 PVD Low Voltage!。当开发板供电正常，LED1 会熄灭，LCD 屏会继续显示"PVD Voltage OK!"。

31.3 睡眠模式实验

本小节我们来学习睡眠模式实验，该部分的知识点内容请回顾 31.1.23 电源管理。我们直接从寄存器介绍开始。

31.3.1 EXTI 寄存器

本实验我们用到外部中断来唤醒睡眠模式。我们用到 WFI 指令进入睡眠模式，这个后面会讲，进入睡眠模式后，使用外部中断唤醒。进入外部中断后，EXTI_CPUIMR1 寄存器的值会自动清零，我们需要对对应的外部中断线位置 1，取消屏蔽，相当于其他中断的中断标志位进入中断后硬件自动置 1，需要手动清零。

- **EXTI 中断屏蔽寄存器（EXTI_CPUIMR1）**

EXTI 中断屏蔽寄存器描述如图 31.3.1.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MR31	MR30	MR29	MR28	MR27	MR26	MR25	MR24	MR23	MR22	MR21	MR20	MR19	MR18	MR17	MR16
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MR15	MR14	MR13	MR12	MR11	MR10	MR9	MR8	MR7	MR6	MR5	MR4	MR3	MR2	MR1	MR0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:22 **MRx**：直接事件输入 x 上的 CPU 中断屏蔽 (CPU interrupt Mask on Direct Event input x)(1)
0：屏蔽来自 x 线的中断请求
1：取消屏蔽来自线 x 的中断请求

位 21:0 **MRx**：可配置事件输入 x 上的 CPU 中断屏蔽 (CPU interrupt Mask on Configurable Event input x) (2)
0：屏蔽来自 x 线的中断请求
1：取消屏蔽来自线 x 的中断请求

图 31.3.1.1 EXTI_CPUIMR1 寄存器

实验中我们使用 WK_UP（PA0）唤醒，即 EXTI0 线中断，所以在外部中断服务函数要把 MR0 位要置 1。

31.3.2 硬件设计

1. 例程功能

LED0 闪烁，表明代码正在运行。按下按键 KEY0 后，LED1 点亮，提示进入睡眠模式，此时 LED0 不再闪烁，说明已经进入睡眠模式。按下按键 WK_UP 后，LED1 熄灭，提示退出睡眠模式，此时 LED0 继续闪烁，说明已经退出睡眠模式。

2. 硬件资源

1）LED 灯
LED0 : LED0 - PB1
LED1 : LED1 - PB0
2）独立按键 KEY0 - PH3，WK_UP - PA0
3）电源管理(低功耗模式 - 睡眠模式)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)

3. 原理图

PWR 属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们通过 KEY0 让 CPU 进入睡眠模式，再通过 WK_UP 触发 EXTI 中断来唤醒 CPU。LED0 指示程序是否执行，LED1 指示 CPU 是否进入睡眠模式。

31.3.3 程序设计

31.3.3.1 PWR 的 HAL 库驱动

1. HAL_PWR_EnterSLEEPMode 函数

进入睡眠模式函数，其声明如下：

```
void HAL_PWR_EnterSLEEPMode (uint32_t Regulator, uint8_t SLEEPEntry);
```

- **函数描述：**

  用于设置 CPU 进入睡眠模式。

- **函数形参：**

  形参 1 指定稳压器的状态。有两个选择，PWR_MAINREGULATOR_ON 表示稳压器处于主模式，PWR_LOWPOWERREGULATOR_ON 表示稳压器处于低功耗模式。对应的是 PWR_CR1 寄存器的 LPDS 位的设置（该形参在该函数中没有实质用处）。

  形参 2 指定进入睡眠模式的方式。有两个选择，PWR_SLEEPENTRY _WFI 表示使用 WFI 指令，PWR_SLEEPENTRY_WFE 表示使用 WFE 指令。

- **函数返回值：**

  无

睡眠模式配置步骤

1）配置唤醒睡眠模式的方式

这里我们用外部中断的方式唤醒睡眠模式，所以这里需要配置一个外部中断功能，我们用 WK_UP 按键作为中断触发源，接下来就是配置 PA0（连接按键 WK_UP）。

通过__HAL_RCC_GPIOA_CLK_ENABLE 函数使能 GPIOA 的时钟。

通过 HAL_GPIO_Init 函数配置 PA0 为上升沿触发检测的外部中断模式，开启下拉电阻等。

通过 HAL_NVIC_EnableIRQ 函数使能 EXTI0 中断。

通过 HAL_NVIC_SetPriority 函数设置中断优先级。

编写 EXTI0_IRQHandle 中断函数，在中断服务函数中调用 HAL_GPIO_EXTI_IRQHandler。

最后编写 HAL_GPIO_EXTI_Callback 回调函数。由于前面已经介绍过外部中断的配置步骤，这里就介绍到这里，详见本例程源码。

2）进入 CPU 睡眠模式

通过 HAL_SuspendTick 函数暂停滴答时钟，防止通过滴答时钟中断唤醒。

通过 HAL_PWR_EnterSLEEPMode 函数进入睡眠模式。

3）通过按下按键触发外部中断唤醒睡眠模式

在本实验中，通过按下 KEY0 按键进入睡眠模式，然后通过按下 WK_UP 按键触发外部中断唤醒睡眠模式。

31.3.3.2 程序流程图

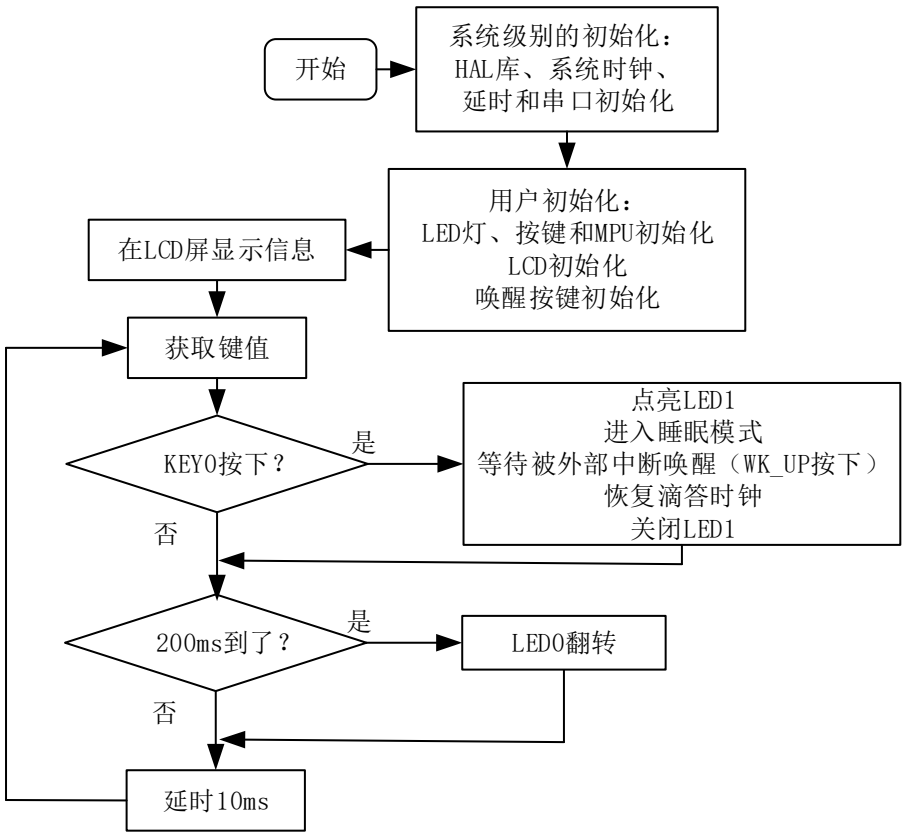


图 31.3.3.2.1 睡眠模式实验程序流程图

31.3.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。PWR 驱动源码包括两个文件：pwr.c 和 pwr.h。

首先看 pwr.h 头文件的几个宏定义：

```
/* PWR WKUP 按键 引脚和中断 定义
 * 我们通过 WK_UP 按键唤醒 MCU， 因此必须定义这个按键及其对应的中断服务函数
 */
#define PWR_WKUP_GPIO_PORT          GPIOA
#define PWR_WKUP_GPIO_PIN           GPIO_PIN_0
#define PWR_WKUP_GPIO_CLK_ENABLE()  do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)

#define PWR_WKUP_INT_IRQn           EXTI0_IRQn
#define PWR_WKUP_INT_IRQHandler     EXTI0_IRQHandler
```

这些定义是 WK_UP 按键的相关宏定义，以及其对应的外部中断线 0 的相关定义。

pwr.h 头文件就介绍这部分的程序，下面是 pwr.c 文件，先看低功耗模式下的按键初始化函数，其定义如下：

```
/**
 * @brief       低功耗模式下的按键初始化(用于唤醒睡眠模式/停止模式)
 * @param       无
 * @retval      无
 */
void pwr_wkup_key_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;

    PWR_WKUP_GPIO_CLK_ENABLE();      /* WKUP 时钟使能 */
```

```
    gpio_init_struct.Pin = PWR_WKUP_GPIO_PIN;                /* WKUP 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_IT_RISING;             /* 中断,上升沿 */
    gpio_init_struct.Pull = GPIO_PULLDOWN;                   /* 下拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;           /* 高速 */
    HAL_GPIO_Init(PWR_WKUP_GPIO_PORT, &gpio_init_struct);  /* WKUP 引脚初始化 */

    HAL_NVIC_SetPriority(PWR_WKUP_INT_IRQn,2,2);    /* 抢占优先级 2，子优先级 2 */
    HAL_NVIC_EnableIRQ(PWR_WKUP_INT_IRQn);
}
```

该函数初始化 WK_UP 按键（PA0），并设置上升沿触发的外部中断线 0，最后设置中断优先级并使能外部中断线 0。

下面介绍的是进入 CPU 睡眠模式函数，其定义如下：

```
/**
 * @brief       进入 CPU 睡眠模式
 * @param       无
 * @retval      无
 */
void pwr_enter_sleep(void)
{
    HAL_SuspendTick();  /* 暂停滴答时钟，防止通过滴答时钟中断唤醒 */
    /* 进入睡眠模式 */
    HAL_PWR_EnterSLEEPMode(PWR_MAINREGULATOR_ON, PWR_SLEEPENTRY_WFI);
}
```

首先是调用 HAL_SuspendTick 函数，暂停滴答时钟，防止睡眠模式被滴答时钟中断唤醒。然后调用 HAL_PWR_EnterSLEEPMode 函数使用 WFI 指令进入睡眠模式。

下面介绍的是 WK_UP 按键外部中断服务函数及其回调函数，函数定义如下：

```
/**
 * @brief       WK_UP 按键 外部中断服务程序
 * @param       无
 * @retval      无
 */
void PWR_WKUP_INT_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(PWR_WKUP_GPIO_PIN);
}

/**
 * @brief       外部中断回调函数
 * @param       GPIO_Pin:中断线引脚
 * @note        此函数会被 PWR_WKUP_INT_IRQHandler()调用
 * @retval      无
 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == PWR_WKUP_GPIO_PIN)
    {
        /* HAL_GPIO_EXTI_IRQHandler()函数已经为我们清除了中断标志位，
          所以我们进了回调函数可以不做任何事 */
    }
}
```

在 WK_UP 按键外部中断服务函数中我们调用 HAL 库的 HAL_GPIO_EXTI_IRQHandler 函数来处理外部中断。该函数会调用__HAL_GPIO_EXTI_CLEAR_IT 函数取消屏蔽对应的外部中断线位，这里是 EXTI_CPUIMR1 寄存器相应位，还有其他寄存器控制其他外部中断线。我们只是唤醒睡眠模式而已，不需要其他的逻辑程序，所以 HAL_GPIO_EXTI_Callback 回调函数可以什么都不用做，甚至也可以不重新定义这个回调函数（屏蔽该回调函数也可以）。

最后在 main.c 里面编写如下代码：

```
int main(void)
{
```

```
    uint8_t t = 0, key = 0;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    pwr_wkup_key_init();                    /* 唤醒按键初始化 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "SLEEP TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Enter SLEEP MODE", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:Exit SLEEP MODE", RED);

    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)
        {
            LED1(0);                /* 点亮绿灯,提示进入睡眠模式 */
            pwr_enter_sleep();      /* 进入睡眠模式 */
            HAL_ResumeTick();       /* 恢复滴答时钟 */
            LED1(1);                /* 关闭绿灯,提示退出睡眠模式 */
        }
        if ((t % 20) == 0)
        {
            LED0_TOGGLE();          /* 每 200ms,翻转一次 LED0 */
        }
        delay_ms(10);
        t++;
    }
}
```

该部分程序，功能就是按下 KEY0 后，点亮 LED1、暂停滴答时钟并进入睡眠模式。然后一直等待外部中断唤醒，当按下按键 WK_UP，就触发外部中断，睡眠模式就被唤醒，然后继续执行后面的程序，恢复滴答时钟，关闭 LED1 等。

31.3.4 下载验证

下载代码后，LED0 闪烁，表明代码正在运行。按下按键 KEY0 后，LED1 点亮，提示进入睡眠模式，此时 LED0 不再闪烁，说明已经进入睡眠模式。按下按键 WK_UP 后，LED1 熄灭，提示退出睡眠模式，此时 LED0 继续闪烁，说明已经退出睡眠模式。

31.4 停止模式实验

本小节我们来学习停止模式实验，该部分的知识点内容请回顾 31.1.2.3 电源管理。我们直接从寄存器介绍开始。

31.4.1 PWR 寄存器

本实验我们用到外部中断来唤醒停止模式。我们用到 WFI 指令进入停止模式，这个后面会讲，进入停止模式后，使用外部中断唤醒。外部中断部分内容参照睡眠模式即可，都是共用同样的配置。

下面主要介绍 PWR_CR1 和 PWR_CPUCR 寄存器相关位。

- **PWR 控制寄存器 1（PWR_CR1）**

PWR 的控制寄存器 1 描述如图 31.4.1.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALS		AVDEN
													rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SVOS		Res.	Res.	Res.	Res.	FLPS	DBP	PLS			PVDE	Res.	Res.	Res.	LPDS
rw	rw					rw	rw	rw	rw	rw	rw				rw

位 0 **LPDS**：电压调节为 SVOS3 的低功耗深度睡眠模式 (Low-power Deepsleep with SVOS3)（无论此位的设置为何，SVOS4 和 SVOS5 始终使用低功耗模式）
0：当选择电压调节为 SVOS3 的停止模式时，稳压器处于主模式 (MR)
1：当选择电压调节为 SVOS3 的停止模式时，稳压器处于低功耗模式 (LPR)

图 31.4.1.1 PWR_CR1 寄存器

进入停止模式，我们设置稳压器为低功耗模式，等待中断唤醒，所以位 LPDS 置 1。

- **PWR CPU 控制寄存器（PWR_CPUCR）**

PWR CPU 控制寄存器描述如图 31.4.1.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	RUN_D3	Res.	CSSF	SBF_D2	SBF_D1	SBF	SOPFF	Res.	Res.	PDDS_D3	PDDS_D2	PDDS_D1
				rw		rw	r	r	r	r			rw	rw	rw

位 2 **PDDS_D3**：系统 D3 域掉电深度睡眠 (System D3 domain Power Down Deepsleep)
此位可为系统 D3 域定义深度睡眠模式。
0：在 D3 域进入深度睡眠时保持停止模式。
1：在 D3 域进入深度睡眠时允许待机模式。

位 1 **PDDS_D2**：D2 域掉电深度睡眠 (D2 domain Power Down Deepsleep)
此位可为 D2 域定义深度睡眠模式。
0：在 D2 域进入深度睡眠时保持 DStop 模式。
1：在 D2 域进入深度睡眠时允许 DStandby 模式。

位 0 **PDDS_D1**：D1 域掉电深度睡眠选择 (D1 domain Power Down Deepsleep selection)
此位可为 D1 域定义深度睡眠模式。
0：在 D1 域进入深度睡眠时保持 DStop 模式。
1：在 D1 域进入深度睡眠时允许 DStandby 模式。

图 31.4.1.2 PWR_CPUCR 寄存器

位 PDDS_D1~ PDDS_D3 都设置为 0， 保持 D1/D2/D3 进入深度睡眠后，进入停止模式。在设置该寄存器前，我们要使能系统控制寄存器 SCR 的位 2（SLEEPDEEP），该位置 1，该寄存器在 M7 内核手册。然后在停止模式后，关闭 SLEEPDEEP 位。

31.4.2 硬件设计

1. 例程功能

LED0 闪烁，表明代码正在运行。按下按键 KEY0 后，LED1 点亮，提示进入停止模式，此时 LED0 不再闪烁，说明已经进入停止模式。按下按键 WK_UP 后，LED1 熄灭，提示退出停止模式，此时 LED0 继续闪烁，说明已经退出停止模式。

2. 硬件资源

1）LED 灯
LED0 : LED0 - PB1
LED1 : LED1 - PB0
2）独立按键
KEY0 - PH3
WK_UP - PA0
3）电源管理(低功耗模式 - 停止模式)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)

3. 原理图

PWR 属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们通过 KEY0 让 CPU 进入停止模式，再通过 WK_UP 触发 EXTI 中断来唤醒 CPU。LED0 指示程序是否执行，LED1 指示 CPU 是否进入停止模式。

31.4.3 程序设计

31.4.3.1 PWR 的 HAL 库驱动

1. HAL_PWR_EnterSTOPMode 函数

进入停止模式函数，其声明如下：

```
void HAL_PWR_EnterSTOPMode (uint32_t Regulator, uint8_t STOPEntry);
```

- **函数描述：**

用于设置 CPU 进入停止模式。

- **函数形参：**

形参 1 指定稳压器在停止模式下的状态。有两个选择，PWR_MAINREGULATOR_ON 表示稳压器处于主模式，PWR_LOWPOWERREGULATOR_ON 表示稳压器处于低功耗模式。对应的是 PWR_CR1 寄存器的 LPDS 位的设置。

形参 2 指定用 WFI 还是 WFE 指令进入停止模式。有两个选择，PWR_STOPENTRY_WFI 表示使用 WFI 指令，PWR_STOPENTRY_WFE 表示使用 WFE 指令。我们选择前者，不了解这两种指令的区别，可以回看 31.1.3 小节的知识。

- **函数返回值：**

无

停止模式配置步骤

1）配置唤醒停止模式的方式

这里我们用外部中断的方式唤醒停止模式，所以这里需要配置一个外部中断功能，我们用 WK_UP 按键作为中断触发源，接下来就是配置 PA0（连接按键 WK_UP）。

通过__HAL_RCC_GPIOA_CLK_ENABLE 函数使能 GPIOA 的时钟。
通过 HAL_GPIO_Init 函数配置 PA0 为上升沿触发检测的外部中断模式，开启下拉电阻等。
通过 HAL_NVIC_EnableIRQ 函数使能 EXTI0 中断。
通过 HAL_NVIC_SetPriority 函数设置中断优先级。

编写 EXTI0_IRQHandle 中断函数，在中断服务函数中调用 HAL_GPIO_EXTI_IRQHandler。

最后编写 HAL_GPIO_EXTI_Callback 回调函数。由于前面已经介绍过外部中断的配置步骤，这里就介绍到这里，详见本例程源码。

2）进入 CPU 停止模式

通过 sys_stm32_clock_init 函数降频。降低性能前，应先减小系统频率，再更改电压调节。

通过 HAL_PWR_EnterSTOPMode 函数进入停止模式。

3）通过按下按键触发外部中断唤醒睡眠模式

在本实验中，通过按下 KEY0 按键进入停止模式，然后通过按下 WK_UP 按键触发外部中断唤醒停止模式。

31.4.3.2 程序流程图

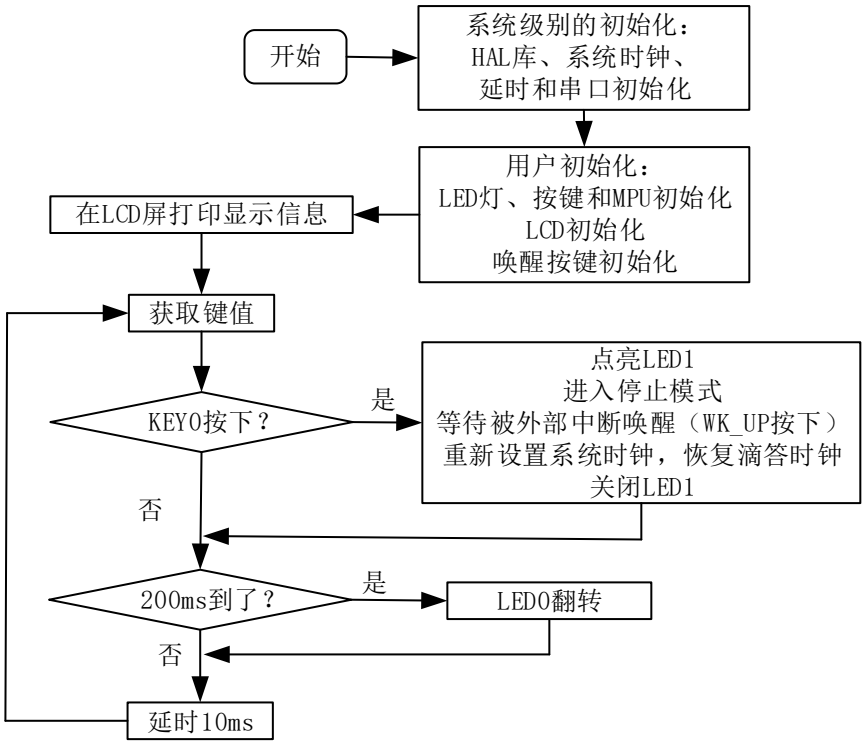


图 31.4.3.2.1 停止模式实验程序流程图

31.4.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。PWR 驱动源码包括两个文件：pwr.c 和 pwr.h。

首先看 pwr.h 头文件，因为我们还是用到 WK_UP 对应的外部中断线来唤醒停止模式的 CPU，pwr.h 头文件的 WK_UP 按键对应的宏定义我们也是用到的，上个实验已经讲过，这里不再赘述。下面是 pwr.c 文件，WK_UP 按键的相关函数我们还是用上个实验的，我们主要介绍进入停止模式函数，其定义如下：

```
/**
 * @brief       进入停止模式
 * @param       无
 * @retval      无
 */
void pwr_enter_stop(void)
{
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟,400Mhz，降频 */
    /* 当 SVOS3 进入停止模式时，设置稳压器为低功耗模式，等待中断唤醒 */
    HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
}
```

该函数首先是调用 sys_stm32_clock_init 函数重新设置系统时钟频率，降频到 400MHZ。然后调用 HAL_PWR_EnterSTOPMode 函数进入停止模式，形参 1 选择 PWR_LOWPOWERREGULATOR_ON 表示设置稳压器为低功耗模式，形参 2 则是选择 WFI 指令。这里我们不再需要像睡眠模式实验一样要暂停滴答时钟，因为滴答时钟中断无法唤醒停止模式，只有外部中断可以唤醒。

最后在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t t = 0;
    uint8_t key = 0;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    pwr_wkup_key_init();                    /* 唤醒按键初始化 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "STOP TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Enter STOP MODE", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:Exit STOP MODE", RED);

    while (1)
    {
        key = key_scan(0);

        if (key == KEY0_PRES)
        {
            LED1(0);            /* 点亮绿灯,提示进入停止模式 */
            pwr_enter_stop();   /* 进入停止模式 */
            /* 从停止模式唤醒, 需要重新设置系统时钟, 400Mhz */
            sys_stm32_clock_init(160, 5, 2, 4);
            lcd_init();                             /* 初始化 LCD */
            lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
            lcd_show_string(30, 70, 200, 16, 16, "STOP TEST", RED);
            lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
            lcd_show_string(30, 110, 200, 16, 16, "KEY0:Enter STOP MODE", RED);
            lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:Exit STOP MODE", RED);

            LED1(1);            /* 关闭绿灯,提示退出停止模式 */
        }

        if ((t % 20) == 0)
        {
            LED0_TOGGLE();      /* 每 200ms,翻转一次 LED0 */
        }

        delay_ms(10);
        t++;
    }
}
```

该部分程序，功能就是按下 KEY0 后，点亮 LED1、暂停滴答时钟并进入停止模式。然后一直等待外部中断唤醒，当按下按键 WK_UP，就触发外部中断，停止模式就被唤醒，然后继续执行后面的程序，重新设置系统时钟 400MHZ，关闭 LED1 等。

31.4.4 下载验证

下载代码后，LED0 闪烁，表明代码正在运行。按下按键 KEY0 后，LED1 点亮，提示进入停止模式，此时 LED0 不再闪烁，说明已经进入停止模式。按下按键 WK_UP 后，LED1 熄灭，提示退出停止模式，此时 LED0 继续闪烁，说明已经退出停止模式。

31.5 待机模式实验

本小节我们来学习待机模式实验，该部分的知识点内容请回顾 31.1.3 电源管理。我们直接从寄存器介绍开始。

31.5.1 PWR 寄存器

本实验是先对相关的电源控制寄存器配置待机模式的参数，然后通过 WFI 指令进入待机模式，使用特定唤醒源 WKUP 引脚来唤醒待机模式。

下面主要介绍本实验用到的几个寄存器。

- **PWR 唤醒使能和极性寄存器（PWR_WKUPEPR）**

PWR 唤醒使能和极性寄存器描述如图 31.5.1.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	WKUPPUPD6		WKUPPUPD5		WKUPPUPD4		WKUPPUPD3		WKUPPUPD2		WKUPPUPD1	
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	WKUPP6	WKUPP5	WKUPP4	WKUPP3	WKUPP2	WKUPP1	Res.	Res.	WKUPEN6	WKUPEN5	WKUPEN4	WKUPEN3	WKUPEN2	WKUPEN1
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

位 27:16 **WKUPPUPD[truncate(n/2)-7)**：WKUP(truncate(n/2)-7) 唤醒引脚拉取配置 (Wakeup pin pull configuration for WKUP(truncate(n/2)-7))

这些引脚用于定义 WKUPEN(truncate(n/2)-7) = 1 时所使用的 I/O 引脚上拉和下拉配置。相关的 GPIO 端口上拉和下拉配置应设置为相同值或“00”。

唤醒引脚的上拉和下拉配置保持待机模式下的配置。

00：无上拉　　01：上拉　　10：下拉　　11：保留

位 13:8 **WKUPPn-7**：WKUPn-7 唤醒引脚极性位 (Wakeup pin polarity bit for PA0)

这些位定义用于 WKUPn-7 外部唤醒引脚事件检测的极性。

0：在高电平（上升沿）检测　　1：在低电平（下降沿）检测

位 5:0 **WKUPENn+1**：使能唤醒引脚 WKUPn+1 (Enable Wakeup Pin WKUPn+1)

各个位由软件置 1 和清零。

0：WKUPn+1 引脚上的事件不会把系统从待机模式唤醒。

1：WKUPn+1 引脚上的上升沿或下降沿将系统从待机模式唤醒。

注：　如果 WKUPn+1 引脚电平在 WKUPPn+1 选择上升沿时已为高电平，或者在 WKUPPn+1 选择下降沿时已为低电平，则当 WKUPn+1 引脚使能时（将 WKUPENn+1 位置 1），会检测到额外的唤醒事件。

图 31.5.1.1 PWR_WKUPEPR 寄存器

本章，我们使用 PA0 的上升沿来唤醒 MCU，PA0 对应的 WKUP 源为：WKUP1，因此，对于 PWR_WKUPEPR 寄存器，我们需要设置的位如下：

1，设置 WKUPEN1 位为 1，使能 WKUP1 的唤醒功能。

2，设置 WKUPPP1 位为 0，选择上升沿唤醒。

3，设置 WKUPPUPD1[1:0]位为 2，选择使用下拉电阻，以保持 WKUP1 脚的低电平状态。

根据这三个步骤，设置好 PWR_WKUPEPR 寄存器，就可以设置 PA0 引脚的上升沿唤醒 MCU 了。

- **PWR 唤醒清除寄存器（PWR_WKUPCR）**

PWR 唤醒清除寄存器描述如图 31.5.1.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WKUPC6	WKUPC5	WKUPC4	WKUPC3	WKUPC2	WKUPC1
										rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

位 5:0 **WKUPCn+1**：将 WKUPn+1 唤醒引脚标志清零 (Clear Wakeup pin flag for WKUPn+1)。
此位始终读为 0
0：无影响
1：写 1 会将 WKUPFn+1 唤醒引脚标志清零（此位通过硬件清零）

图 31.5.1.2 PWR_WKUPCR 寄存器

该寄存器我们只关心 WKUPC1 位，设置 WKUPC1 为 1，可以清除 PA0 的唤醒标志位。

- **PWR CPU 控制寄存器（PWR_CPUCR）**

PWR CPU 控制寄存器描述如图 31.5.1.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	RUN_D3	Res.	CSSF	SBF_D2	SBF_D1	SBF	SOPFF	Res.	Res.	PDDS_D3	PDDS_D2	PDDS_D1
				rw		rw	r	r	r	r			rw	rw	rw

位 2 **PDDS_D3**：系统 D3 域掉电深度睡眠 (System D3 domain Power Down Deepsleep)
此位可为系统 D3 域定义深度睡眠模式。
0：在 D3 域进入深度睡眠时保持停止模式。
1：在 D3 域进入深度睡眠时允许待机模式。

位 1 **PDDS_D2**：D2 域掉电深度睡眠 (D2 domain Power Down Deepsleep)
此位可为 D2 域定义深度睡眠模式。
0：在 D2 域进入深度睡眠时保持 DStop 模式。
1：在 D2 域进入深度睡眠时允许 DStandby 模式。

位 0 **PDDS_D1**：D1 域掉电深度睡眠选择 (D1 domain Power Down Deepsleep selection)
此位可为 D1 域定义深度睡眠模式。
0：在 D1 域进入深度睡眠时保持 DStop 模式。
1：在 D1 域进入深度睡眠时允许 DStandby 模式。

图 31.5.1.3 PWR_CPUCR 寄存器

该寄存器我们只需要关心最低 3 个位，分别用于设置在 D1 域、D2 域和 D3 域进入深度睡眠模式时，允许待机模式，已达到最低功耗的目的。所以这三个位都要设置为 1。

- **系统控制寄存器（SCB_SCR）**

系统控制寄存器描述如图 31.5.1.4 所示：

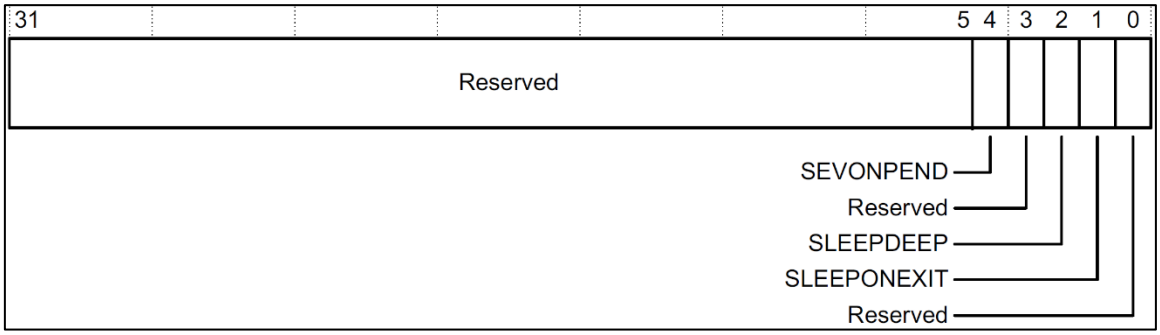


图 31.5.1.4 SCB_SCR 寄存器

该寄存器我们只关心：SLEEPDEEP 位，要进入待机模式，我们必须设置该位为 1。

31.5.2 硬件设计

1. 例程功能

LED0 闪烁，表明代码正在运行。按下按键 KEY0 后，进入待机模式。待机模式下大部分引脚处于高阻态，所以说这时候 LED0 会熄灭，TFTLCD 屏熄灭。按下按键 WK_UP 后，退出待机模式（相当于复位操作），程序重新执行，LED0 继续闪烁，TFTLCD 屏点亮。

2. 硬件资源

1）LED 灯
LED0 : LED0 - PB1
2）独立按键
KEY0 - PH3
WK_UP - PA0
3）电源管理(低功耗模式 - 待机模式)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)

3. 原理图

PWR 属于 STM32H743 的内部资源，只需要软件设置好即可正常工作。我们通过 KEY0 让 CPU 进入待机模式，再通过 WK_UP 上升沿触发唤醒 CPU。LED0 指示程序是否执行。

31.5.3 程序设计

31.5.3.1 PWR 的 HAL 库驱动

1. HAL_PWR_EnableWakeUpPin 函数

使能唤醒引脚函数，其声明如下：

```
void HAL_PWR_EnableWakeUpPin (uint32_t WakeUpPinPolarity);
```

- **函数描述：**
  用于使能唤醒引脚，实际上该函数设置 PWR 唤醒使能和极性寄存器（PWR_WKUPEPR）的位[5:0]和位[13:8]。
- **函数形参：**
  形参 1 取值范围：PWR_WAKEUP_PIN1_HIGH~ PWR_WAKEUP_PIN6_HIGH（等同于 PWR_WAKEUP_PIN1~PWR_WAKEUP_PIN6、PWR_WAKEUP_PIN1_LOW~ PWR_WAKEUP_PIN6_LOW。
- **函数返回值：**无
- **注意事项：**
  禁止某个唤醒引脚使用的函数如下：

```
void HAL_PWR_DisableWakeUpPin (uint32_t WakeUpPinPolarity);
```

2. HAL_PWR_EnterSTANDBYMode 函数

进入待机模式函数，其声明如下：

```
void HAL_PWR_EnterSTANDBYMode (void);
```

- **函数描述：**
  用于使 CPU 进入待机模式，进入待机模式，首先要设置 SLEEPDEEP 位，接着我们通过 PWR_CR 设置 PDDS 位，使得 CPU 进入深度睡眠时进入待机模式，最后执行 WFI 指令开始进入待机模式，并等待 WK_UP 唤醒的到来。
- **函数形参：**无
- **函数返回值：**无

待机模式配置步骤

1）进入 CPU 待机模式

在进入待机模式之前我们需要做一些准备，比如：关闭 RTC 相关中断、清除 RTC 相关中断标志位等一些操作，只是防止 RTC 中断唤醒。这里就不细讲，详见本例程源码。

通过__HAL_PWR_CLEAR_FLAG 函数清除唤醒标志位。

通过 HAL_PWR_EnableWakeUpPin 函数使能 PA0 的唤醒功能。

通过 HAL_PWR_EnterSTANDBYMode 函数进入待机模式。

2）通过按下 WKUP 引脚上升沿触发唤醒待机模式

在本实验中，通过按下 KEY0 按键进入待机模式，然后通过按下 WK_UP 按键（特定唤醒源）触发唤醒待机模式。

31.5.3.2 程序流程图

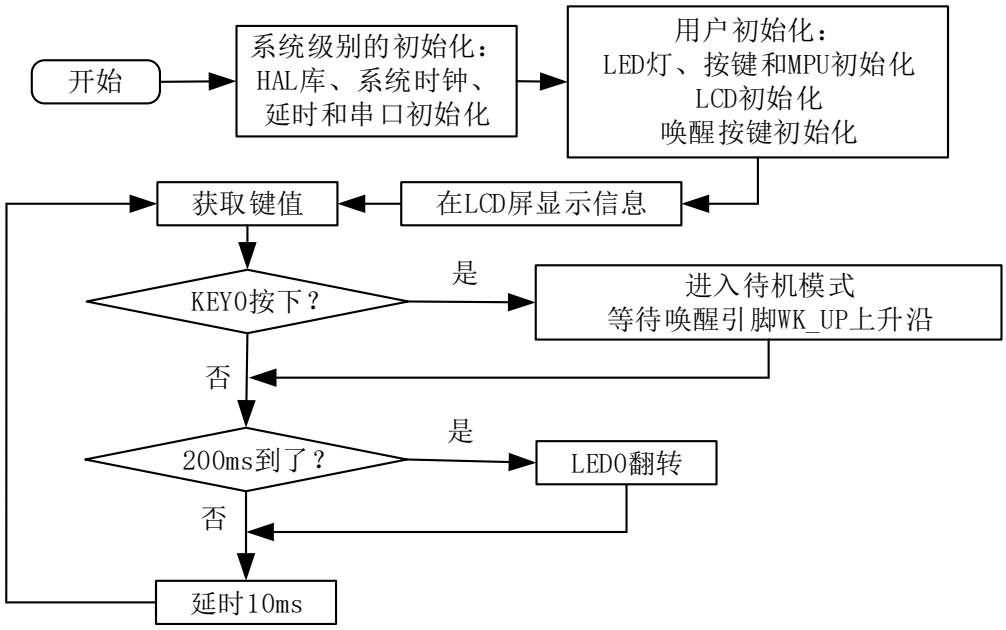


图 31.5.3.2.1 待机模式实验程序流程图

31.5.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。PWR 驱动源码包括两个文件：pwr.c 和 pwr.h。

pwr.h 头文件上的宏定义我们是没有用到的，这里我们使用的是特定唤醒源，与外部中断无关。下面是 pwr.c 文件，我们主要介绍进入待机模式函数，其定义如下：

```
/**
 * @brief       进入待机模式
 * @param       无
 * @retval      无
 */
void pwr_enter_standby(void)
{
    __HAL_RCC_AHB1_FORCE_RESET();                   /* 复位所有 IO 口 */
    while (WKUP_KD);/* 等待 WK_UP 按键松开(在有 RTC 中断时,必须等 WK_UP 松开再进入待机) */

    HAL_PWR_DisableWakeUpPin(PWR_WAKEUP_PIN1);      /* 禁止唤醒 */
    __HAL_RCC_BACKUPRESET_FORCE();                  /* 复位备份区域 */
    HAL_PWR_EnableBkUpAccess();                     /* 后备区域访问使能 */

    __HAL_PWR_CLEAR_FLAG(PWR_FLAG_WKUP1);
```

```
    __HAL_RTC_WRITEPROTECTION_DISABLE(&g_rtc_handle);    /* 关闭 RTC 写保护 */

    /* 关闭 RTC 相关中断，可能在 RTC 实验打开了 */
    __HAL_RTC_WAKEUPTIMER_DISABLE_IT(&g_rtc_handle, RTC_IT_WUT);
    __HAL_RTC_TIMESTAMP_DISABLE_IT(&g_rtc_handle, RTC_IT_TS);
    __HAL_RTC_ALARM_DISABLE_IT(&g_rtc_handle, RTC_IT_ALRA | RTC_IT_ALRB);

    /* 清除 RTC 相关中断标志位 */
    __HAL_RTC_ALARM_CLEAR_FLAG(&g_rtc_handle, RTC_FLAG_ALRAF | RTC_FLAG_ALRBF);
    __HAL_RTC_TIMESTAMP_CLEAR_FLAG(&g_rtc_handle, RTC_FLAG_TSF);
    __HAL_RTC_WAKEUPTIMER_CLEAR_FLAG(&g_rtc_handle, RTC_FLAG_WUTF);

    __HAL_RCC_BACKUPRESET_RELEASE();                     /* 备份区域复位结束 */
    __HAL_RTC_WRITEPROTECTION_ENABLE(&g_rtc_handle); /* 使能 RTC 写保护 */
    __HAL_PWR_CLEAR_FLAG(PWR_FLAG_WKUP1);               /* 清除 Wake_UP 标志 */

    HAL_PWR_EnableWakeUpPin(PWR_WAKEUP_PIN1);       /* 使能 KEY_UP 引脚的唤醒功能 */
    PWR->WKUPEPR &= ~(3 << 16);                     /* 清除 WKUPPUPD 原来的设置 */
    PWR->WKUPEPR |= 2 << 16;                        /* WKUPPUPD=10,PA0 下拉 */
    PWR->WKUPCR |= 0X3F << 0;                       /* 清除 WKUP0~5 唤醒标志 */
    HAL_PWR_EnterSTANDBYMode();                     /* 进入待机模式 */
}
```

该函数首先是关闭 RTC 相关中断，清除 RTC 相关中断标志位。然后使能 WKUP 引脚上升沿来唤醒待机模式。最后调用 HAL_PWR_EnterSTANDBYMode 函数进入待机模式。

最后在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t t = 0;
    uint8_t key = 0;
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    pwr_wkup_key_init();                    /* 唤醒按键初始化 */
    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "STANDBY TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Enter STANDBY MODE", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:Exit STANDBY MODE", RED);

    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)
        {
            pwr_enter_standby();    /* 进入待机模式 */
            /* 从待机模式唤醒相当于系统重启(复位), 因此不会执行到这里 */
        }
        if ((t % 20) == 0)
        {
            LED0_TOGGLE();          /* 每 200ms,翻转一次 LED0 */
        }
        delay_ms(10);
        t++;
    }
}
```

该部分程序，经过一系列初始化后，判断到 KEY0 按下就调用 pwr_enter_standby 函数进入待机模式，然后等待按下 WK_UP 按键进行唤醒。注意待机模式唤醒后，系统会进行复位。

31.5.4 下载验证

下载代码后，LED0 闪烁，表明代码正在运行。按下按键 KEY0 后，TFTLCD 屏熄灭，此时 LED0 不再闪烁，说明已经进入待机模式。按下按键 WK_UP 后，TFTLCD 屏点亮，此时 LED0 继续闪烁，说明系统从待机模式中唤醒相当于复位。

第三十二章 DMA 实验

本章，我们将介绍 STM32H743 的 DMA。我们将利用 DMA 来实现串口数据传送，并在 LCD 模块上显示当前的传送进度。

本章分为如下几个小节：

32.1 DMA 简介
32.2 硬件设计
32.3 程序设计
32.4 下载验证

32.1 DMA 简介

DMA，全称为：Direct Memory Access，即直接存储器访问。DMA 传输方式无需 CPU 直接控制传输，也没有中断处理方式那样保留现场和恢复现场的过程，通过硬件为 RAM 与 I/O 设备开辟一条直接传送数据的通路，能使 CPU 的效率大为提高。

STM32H743 内部有 4 个 DMA 控制器，它们的连接关系请参考前面第十一章的图 11.3.2 STM32H7 的总线架构图。图中可以看到每个域都拥有各自的 DMA 控制器，D1 域有 1 个高速主 DMA（MDMA）；D2 域有 2 个双口 DMA，DMA1 和 DMA2；D3 域有 1 个基本 DMA（BDMA）。

1、MDMA 用于实现：内存→内存、内存→外设、外设→内存之间的高速数据传输，MDMA 支持 AXI 和 AHBS 总线之间的数据传输。注意：仅 MDMA 支持 AHBS 的访问（可访问 DTCM 和 ITCM），其他 DMA 都无法访问 AHBS 总线。

2、双口 DMA 同样可以实现：内存→内存、内存→外设、外设→内存之间的高速数据传输，双口 DMA 支持 AHB 外设之间的数据传输，具有独立的 FIFO，支持不同位宽的数据传输。

3、BDMA 用于实现：内存→内存、内存→外设、外设→内存、外设到外设之间的高速数据传输。

本章我们用到的是双口 DMA，因此接下来，我们重点介绍双口 DMA。

STM32H743 内部有 2 个双口 DMA 控制器（DMA1 和 DMA2），共 16 个数据流（每个控制器 8 个），每一个双口 DMA 控制器都用于管理一个或多个外设的存储器访问请求。每个数据流可以有多达 115 个通道（或称请求）。每个数据流通道都有一个仲裁器，用于处理 DMA 请求间的优先级。

STM32H743 的双口 DMA 有以下一些特性：

① 双 AHB 主总线架构，一个用于存储器访问，另一个用于外设访问

② 仅支持 32 位访问的 AHB 从编程接口

③ 每个 DMA 控制器有 8 个数据流，每个数据流有多达 115 个通道（或称请求）

④ 每个数据流有单独的四级 32 位先进先出存储器缓冲区(FIFO)，可用于 FIFO 模式或直接模式

⑤ 通过硬件可以将每个数据流配置为：

1，支持外设到存储器、存储器到外设和存储器到存储器传输的常规通道

2，支持在存储器方双缓冲的双缓冲区通道

⑥ 8 个数据流中的每一个都连接到专用硬件 DMA 通道（请求）

⑦ DMA 数据流请求之间的优先级可用软件编程（4 个级别：非常高、高、中、低），在软件优先级相同的情况下可以通过硬件决定优先级（例如，请求 0 的优先级高于请求 1）

⑧ 每个数据流也支持通过软件触发存储器到存储器的传输

⑨ 每个数据流的通道请求可以由 DMA 请求复用器 1（DMAMUX1）选择，可供选择的通道请求数多达 115 个。此选择可由软件配置，允许多个外设启动 DMA 请求

⑩ 要传输的数据项的数目可以由 DMA 控制器或外设管理：

1，DMA 流控制器：要传输的数据项的数目是 1 到 65535，可用软件编程

2，外设流控制器：要传输的数据项的数目未知并由源或目标外设控制，这些外设通过硬件发出传输结束的信号

⑪ 独立的源和目标传输宽度（字节、半字、字）：源和目标的数据宽度不相等时，DMA 自动封装/解封必要的传输数据来优化带宽。这个特性仅在 FIFO 模式下可用。

⑫ 对源和目标的增量或非增量寻址

⑬ 支持 4 个、8 个和 16 个节拍的增量突发传输。突发增量的大小可由软件配置，通常等于外设 FIFO 大小的一半

⑭ 每个数据流都支持循环缓冲区管理

⑮ 5 个事件标志（DMA 半传输、DMA 传输完成、DMA 传输错误、DMA FIFO 错误、直接模式错误），进行逻辑或运算，从而产生每个数据流的单个中断请求

32.1.1 DMA 框图

STM32H743 有两个双口 DMA 控制器，DMA1 和 DMA2，本章，我们仅针对 DMA2 进行介绍。下面先来学习双口 DMA 控制器框图，通过学习 DMA 控制器框图会有一个很好的整体掌握，同时对之后的编程也会有一个清晰的思路。STM32H743 的双口 MDA 控制器框图如图 32.1.1.1 所示：

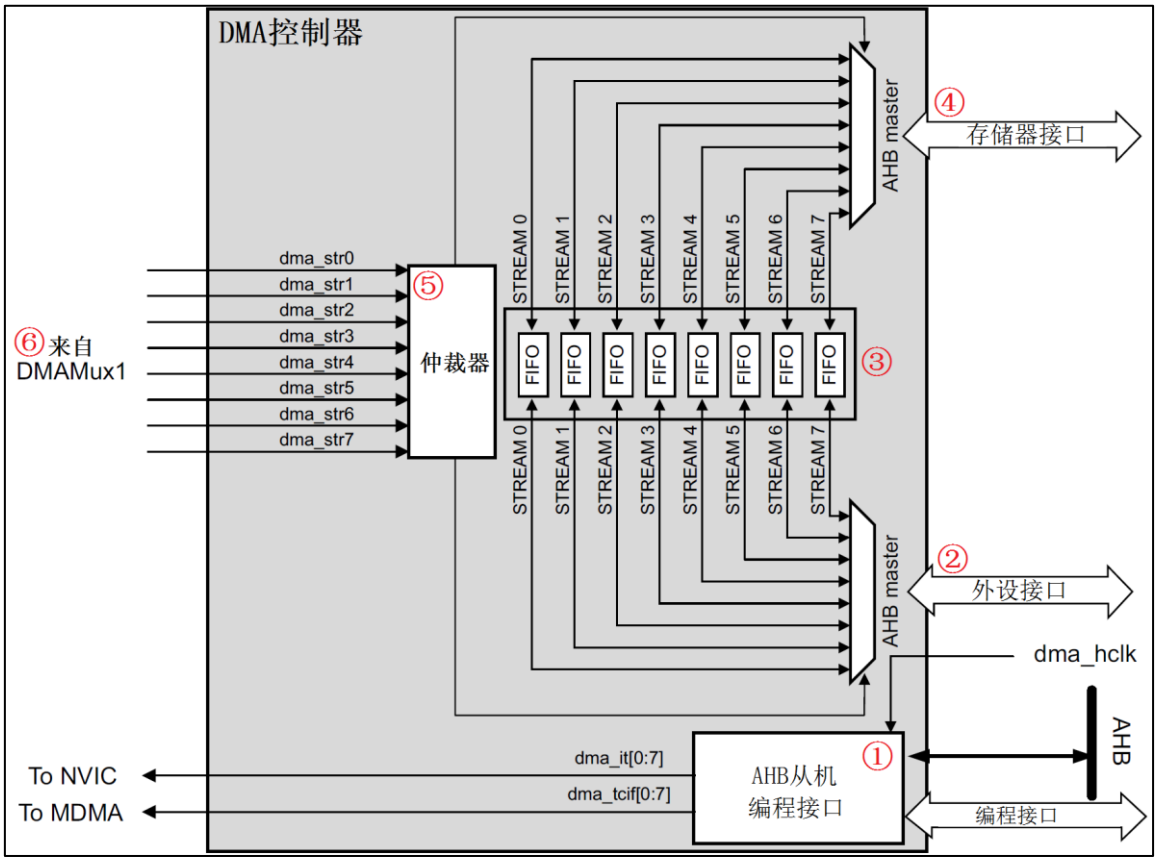


图 32.1.1.1 DMA 控制器框图

图中，我们标记了 6 处位置，起作用分别是：

①，DMA 控制器的从机编程接口，通过该接口可以对 DMA 的相关控制寄存器进行设置，从而配置 DMA，实现不同的功能。同时，该接口可以输出 dma_it[0:7]的中断信号到 NVIC，以及 dma_tcif[0:7]的信号到 MDMA。

②，DMA 控制器的外设接口，用于访问相关外设，特别的，当外设接口设置的访问地址是内存地址的时候，DMA 就可以工作在内存到内存模式了。

③，DMA 控制器的 FIFO 区，每个数据流（总共 8 个数据流）都有一个独立的 FIFO，可以实现存储器接口到外设接口之间的数据长度非对齐传输。

④，DMA 控制器的存储器接口，用于访问外部存储器，特别的当存储器地址是外设地址

的时候，可以实现类似外设到外设的传输效果。

⑤，DMA 控制器的仲裁器，用于仲裁数据流 0~7 的请求优先级，保证数据有序传输。

⑥，这是 DMA 控制器数据流 0~7 的通道请求信号，由 DMAMUX1 的选择，每个数据流有多达 115 个通道请求可以选择。我们必须根据实际需求来选择对应的通道请求。

这里重点介绍一下 DMAMUX1，其全称是 DMA 请求复用器 1，用于管理 DMA1 和 DMA2 的通道请求，而 DMAMUX2 则用来管理 BDMA 的通道请求，这里重点介绍 DMAMUX1。

DMAMUX1 总共有 16 个通道，其中通道 0~7 对应 DMA1 的数据流 0~7，通道 8~15 对应 DMA2 的数据流 0~7。DMAMUX 框图如图 32.1.1.2 所示：

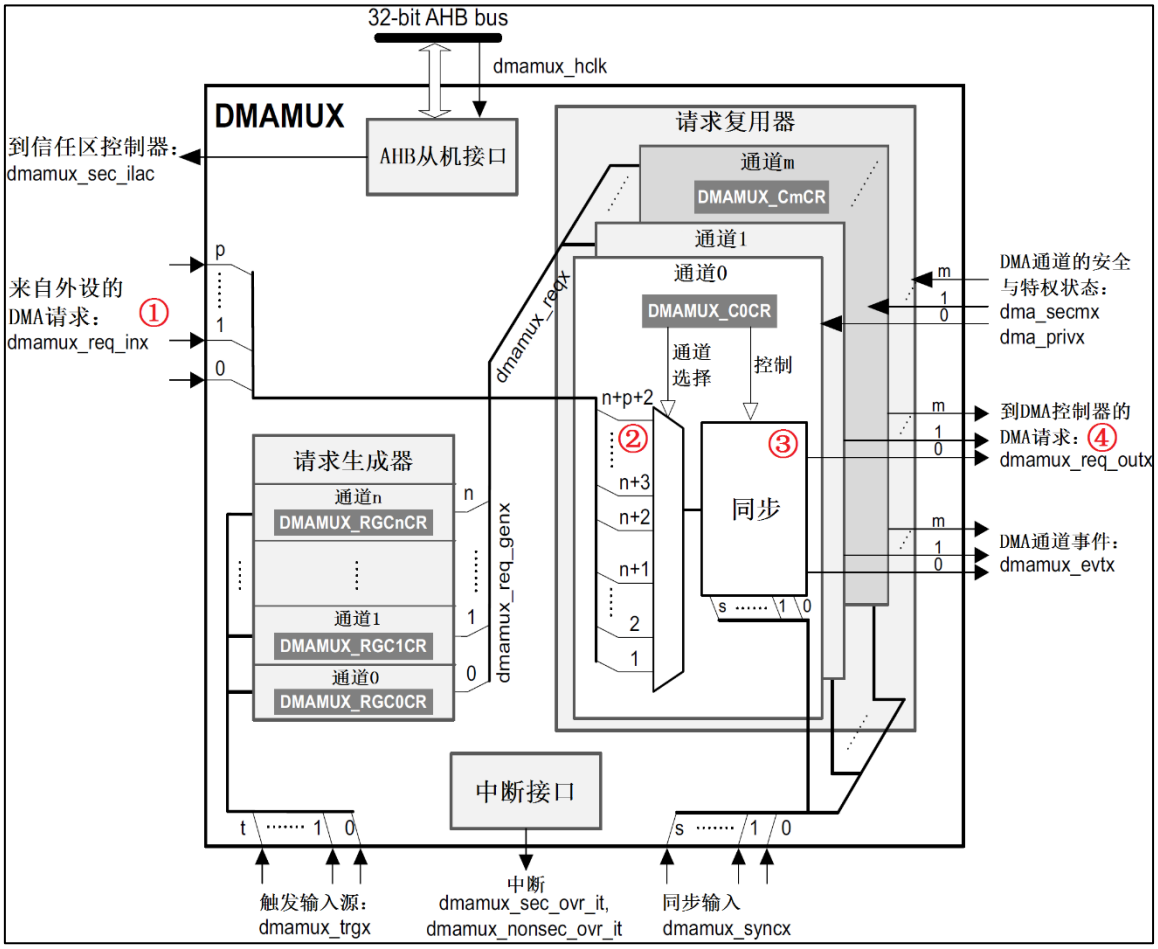


图 32.1.1.2 DMAMUX 控制器框图

整个 DMAMUX 的功能比较复杂，包括：选择具体的 DMA 请求通道（源）、同步控制、请求生成、请求计数和中断等，本章我们只用到其最简单的应用：选择具体的 DMA 请求通道，即通过 DMAMUX1 选择 DMA1/DMA2 的数据流通道（请求通道）。为了方便说明，我们在图中标出了几个关键点：

①，外设请求，即 STM32H7 芯片内部外设的请求，这部分总共有 p+1 个外设请求，对于 DMAMUX1 来说，总共有 107 个（p=106）。具体的对应关系详见：《STM32H7xx 参考手册_V7（英文版）.pdf》第 694 页，Table 120。

②，通道选择，通过 DMAMUX_CmCR（对 DMAMUX1 来说，m=0~15，该寄存器也写作：DMAMUX_CCRm，下同）寄存器的 DMAREQ_ID[7:0]位来选择 DMA 的具体请求通道，总共有 115 个通道，其中①处有 107 个，还有 8 个通道来自 DMAMUX 内部的请求生成器，我们一般用①处的 107 个请求通道。

③，同步控制，用于同步 DMA 请求，最终输出给 DMA 控制器，我们本章不使用同步控制，因此可以关闭同步（通过 DMAMUX_CmCR 寄存器的 SE 位设置）。

④，输出到 DMA 控制器的请求信号，对 DMAMUX1 来说总共有 16 个输出请求信号（m=15），分别对应 DMA1 和 DMA2 的数据流 0~7。

因此，我们一般只需要通过设置 DMAMUX1 通道 m 的 DMAMUX1_CmCR 寄存器来选择输入通道，即可完成 DMA1/DMA2 某个数据流的 DMA 输入请求通道选择。

举例来说，假设我们要设置 DMA2 数据流 3 的 DMA 输入请求通道为：串口 7 的 TX（UART7_TX），则：

1， DMA2 数据流 3 对应 DMAMUX1 的通道 11（从 0 开始算起）。

2， 串口 7 的 TX 对应的外设 DMA 请求编号为：80（查《STM32H7xx 参考手册_V7（英文版）.pdf》第 696 页 Table 121）。

因此，我们只需要设置 DMAMUX1_CCR11 的 DMAREQ_ID[7:0]位为 80 即可完成 DMA2 数据流 3 的请求来自 UART7_TX 的设置。

DMAMUX1 的 DMA 请求资源分配表，如表 32.1.1.1 所示：

DMA请求复用输入编号	资源（源）	DMA请求复用输入编号	资源（源）	DMA请求复用输入编号	资源（源）
1	dmamux1_req_gen0	40	SPI2_TX	79	UART7_RX
2	dmamux1_req_gen1	41	USART1_RX	80	UART7_TX
3	dmamux1_req_gen2	42	USART1_TX	81	UART8_RX
4	dmamux1_req_gen3	43	USART2_RX	82	UART8_TX
5	dmamux1_req_gen4	44	USART2_TX	83	SPI4_RX
6	dmamux1_req_gen5	45	USART3_RX	84	SPI4_TX
7	dmamux1_req_gen6	46	USART3_TX	85	SPI5_RX
8	dmamux1_req_gen7	47	TIM8_CH1	86	SPI5_TX
9	ADC1	48	TIM8_CH2	87	SAI1_A
.....					
33	I2C1_RX	72	USART6_TX	111	TIM17_CH1
34	I2C1_TX	73	I2C3_RX	112	TIM17_UP
35	I2C2_RX	74	I2C3_TX	113	SAI3_A
36	I2C2_TX	75	DCMI	114	SAI3_B
37	SPI1_RX	76	CRYP_IN	115	ADC3
38	SPI1_TX	77	CRYP_OUT	-	-
39	SPI2_RX	78	HASH_IN	-	-

表 32.1.1.1 DMAMUX1 DMA 请求资源分配表（部分）

由表可知，DMAMUX1 的 DMA 请求输入总共有 115 个，表中我们省略了一部分内容，详见：《STM32H7xx 参考手册_V7（英文版）》第 696 页 Table 120 和 Table 121。

由于 DMA1/DMA2 数据流 0~7 的 DMA 请求全部是来自 DMAMUX1 的设置（16 个通道），DMAMUX1 的所有通道都可以独立设置，因此，对于 STM32H7 来说，DMA1/DMA2 的任何一个数据流的请求都可以来自 115 个通道的任意一个，因此相对于 STM32 其他系列来说，STM32H7 的 DMA 配置更加灵活，无需固定通道对应固定外设请求，可以随意设置。

32.1.2 DMA 寄存器

- **DMAMUX1 请求线复用器通道 x 配置寄存器（DMAMUX1_CxCR），（x = 0 到 15）**

  DMAMUX1 请求线复用器通道 x 配置寄存器描述如图 32.1.2.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	SYNC_ID[4:0]					NBREQ[4:0]					SPOL[1:0]		SE
			rw					rw					rw		rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	ege	SOIE	DMAREQ_ID[7:0]							
						rw	rw	rw							

位 7:0 **DMAREQ_ID[7:0]**：DMA 请求标识 (DMA request identification)
选择输入 DMA 请求。（请参见表：DMAMUX - 复用器输入到资源的分配）

图 32.1.2.1 DMAMUX1_CxCR 寄存器各位描述

本章，该寄存器我们只需要关心 DMAREQ_ID[7:0]这 8 个位，其他位全部用不到（设置为 0 即可），DMAREQ_ID[7:0]用于选择输出通道 x 的 DMA 输入请求，我们需要通过这 8 个位选择 DMA1/DMA2 数据流的请求源。请求列表见表 32.1.1（完整的表见《STM32H7xx 参考手册_V7（英文版）》第 696 页 Table 120 和 Table 121）。

- **DMA 中断状态寄存器（DMA_LISR 和 DMA_HISR）**

DMA 中断状态寄存器，该寄存器总共有 2 个：DMA_LISR 和 DMA_HISR，每个寄存器管理 4 数据流（总共 8 个），DMA_LISR 寄存器用于管理数据流 0~3，而 DMA_HISR 用于管理数据流 4~7。这两个寄存器各位描述都完全一模一样，只是管理的数据流不一样。

这里我们仅以 DMA_LISR 寄存器为例进行介绍，DMA_LISR 各位描述如图 32.1.2.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	TCIF3	HTIF3	TEIF3	DMEIF3	Res.	FEIF3	TCIF2	HTIF2	TEIF2	DMEIF2	Res.	FEIF2
r	r	r	r	r	r	r	r		r	r	r	r	r		r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	TCIF1	HTIF1	TEIF1	DMEIF1	Res.	FEIF1	TCIF0	HTIF0	TEIF0	DMEIF0	Res.	FEIF0
r	r	r	r	r	r	r	r		r	r	r	r	r		r

位 27、21、11、5 **TCIFx**：数据流 x 传输完成中断标志 (Stream x transfer complete interrupt flag) (x = 3..0)
此位将由硬件置 1，由软件清零，软件只需将 1 写入 DMA_LIFCR 寄存器的相应位。
0：数据流 x 上无传输完成事件
1：数据流 x 上发生传输完成事件

位 26、20、10、4 **HTIFx**：数据流 x 半传输中断标志 (Stream x half transfer interrupt flag) (x=3..0)
此位将由硬件置 1，由软件清零，软件只需将 1 写入 DMA_LIFCR 寄存器的相应位。
0：数据流 x 上无半传输事件
1：数据流 x 上发生半传输事件

位 25、19、9、3 **TEIFx**：数据流 x 传输错误中断标志 (Stream x transfer error interrupt flag) (x=3..0)
此位将由硬件置 1，由软件清零，软件只需将 1 写入 DMA_LIFCR 寄存器的相应位。
0：数据流 x 上无传输错误
1：数据流 x 上发生传输错误

位 24、18、8、2 **DMEIFx**：数据流 x 直接模式错误中断标志 (Stream x direct mode error interrupt flag) (x=3..0)
此位将由硬件置 1，由软件清零，软件只需将 1 写入 DMA_LIFCR 寄存器的相应位。
0：数据流 x 上无直接模式错误
1：数据流 x 上发生直接模式错误

位 22、16、6、0 **FEIFx**：数据流 x FIFO 错误中断标志 (Stream x FIFO error interrupt flag) (x=3..0)
此位将由硬件置 1，由软件清零，软件只需将 1 写入 DMA_LIFCR 寄存器的相应位。
0：数据流 x 上无 FIFO 错误事件
1：数据流 x 上发生 FIFO 错误事件

图 32.1.2.2 DMA_LISR 寄存器

如果开启了 DMA_LISR 中这些位对应的中断，则在达到条件后就会跳到中断服务函数里面去，即使没开启，我们也可以通过查询这些位来获得当前 DMA 传输的状态。这里我们常用的是 TCIFx 位，即数据流 x 的 DMA 传输完成与否标志。注意此寄存器为只读寄存器，所以在这些位被置位之后，只能通过其他的操作来清除。DMA_HISR 寄存器各位描述通 DMA_LISR 寄存器各位描述完全一样，只是对应数据流 4~7，这里我们就不列出来了。

- **DMA 中断标志清除寄存器（DMA_LIFCR 和 DMA_HIFCR）**

DMA 中断标志清除寄存器, 该寄存器同样有 2 个：DMA_LIFCR 和 DMA_HIFCR，同样是每个寄存器控制 4 个数据流，DMA_LIFCR 寄存器用于管理数据流 0~3，而 DMA_ HIFCR 用于

管理数据流 4~7。这两个寄存器各位描述都完全一模一样，只是管理的数据流不一样。

这里，我们仅以 DMA_LIFCR 寄存器为例进行介绍，DMA_LIFCR 各位描述如图 32.1.2.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	CTCIF3	CHTIF3	CTEIF3	CDMEIF3	Res.	CFEIF3	CTCIF2	CHTIF2	CTEIF2	CDMEIF2	Res.	CFEIF2
				w	w	w	w		w	w	w	w	w		w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	CTCIF1	CHTIF1	CTEIF1	CDMEIF1	Res.	CFEIF1	CTCIF0	CHTIF0	CTEIF0	CDMEIF0	Res.	CFEIF0
				w	w	w	w		w	w	w	w	w		w

位 31:28、15:12 保留，必须保持复位值。

位 27、21、11、5 **CTCIFx**：数据流 x 传输完成中断标志清零 (Stream x clear transfer complete interrupt flag) (x = 3..0)
将 1 写入此位时，DMA_LISR 寄存器中相应的 TCIFx 标志将清零

位 26、20、10、4 **CHTIFx**：数据流 x 半传输中断标志清零 (Stream x clear half transfer interrupt flag) (x = 3..0)
将 1 写入此位时，DMA_LISR 寄存器中相应的 HTIFx 标志将清零

位 25、19、9、3 **CTEIFx**：数据流 x 传输错误中断标志清零 (Stream x clear transfer error interrupt flag) (x = 3..0)
将 1 写入此位时，DMA_LISR 寄存器中相应的 TEIFx 标志将清零

位 24、18、8、2 **CDMEIFx**：数据流 x 直接模式错误中断标志清零 (Stream x clear direct mode error interrupt flag) (x = 3..0)
将 1 写入此位时，DMA_LISR 寄存器中相应的 DMEIFx 标志将清零

位 23、17、7、1 保留，必须保持复位值。

位 22、16、6、0 **CFEIFx**：数据流 x FIFO 错误中断标志清零 (Stream x clear FIFO error interrupt flag) (x = 3..0)
将 1 写入此位时，DMA_LISR 寄存器中相应的 CFEIFx 标志将清零

图 32.1.2.3 DMA_LIFCR 寄存器

DMA_LIFCR 的各位就是用来清除 DMA_LISR 的对应位的，通过写 1 清除。在 DMA_LISR 被置位后，我们必须通过向该位寄存器对应的位写入 1 来清除。DMA_HIFCR 的使用同 DMA_LIFCR 类似，这里就不做介绍了。

第四个是 DMA 数据流 x 配置寄存器（DMA_SxCR）（x=0~7，下同）。该寄存器的我们在这里就不贴出来了，见《STM32H7xx 参考手册_V3（中文版）.pdf》第 565 页 15.5.5 小节。该寄存器控制着 DMA 的很多相关信息，包括数据宽度、外设及存储器的宽度、优先级、增量模式、传输方向、中断允许、使能等都是通过该寄存器来设置的。所以 DMA_ SxCR 是 DMA 传输的核心控制寄存器。

第五个是 DMA 数据流 x 数据项数寄存器（DMA_SxNDTR）。这个寄存器控制 DMA 数据流 x 的每次传输所要传输的数据量。其设置范围为 0~65535。并且该寄存器的值会随着传输的进行而减少，当该寄存器的值为 0 的时候就代表此次数据传输已经全部发送完成了。所以可以通过这个寄存器的值来知道当前 DMA 传输的进度。特别注意，这里是数据项数目，而不是指的字节数。比如设置数据位宽为 16 位，那么传输一次（一个项）就是 2 个字节。

第六个是 DMA 数据流 x 的外设地址寄存器（DMA_SxPAR）。该寄存器用来存储 STM32H743 外设的地址，比如我们使用串口 1，那么该寄存器必须写入 0x40011028（其实就是&USART1_TDR）。如果使用其他外设，就修改成相应外设的地址就行了。

最后一个是 DMA 数据流 x 的存储器地址寄存器，由于 STM32H743 的 DMA 支持双缓存，所以存储器地址寄存器有两个：DMA_SxM0AR 和 DMA_SxM1AR，其中 DMA_SxM1AR 仅在双缓冲模式下，才有效。本章我们没用到双缓冲模式，所以存储器地址寄存器就是：DMA_SxM0AR，该寄存器和 DMA_CPARx 差不多，但是是用来放存储器的地址的。比如我们使用 SendBuf[7800]数组来做存储器，那么我们在 DMA_SxM0AR 中写入&SendBuff 就可以了。

32.2 硬件设计

1. 例程功能

每按下按键 KEY0，串口 1 就会以 DMA 方式发送数据，同时在 LCD 上面显示传送进度。打开串口调试助手，可以收到 DMA 发送的内容。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 : LED0 - PB1
2） 独立按键 KEY0 - PH3
3） 串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
5）DMA

3. 原理图

DMA 属于 STM32H743 内部资源，通过软件设置好就可以了。本实验通过按键 KEY0 触发使用 DMA 的方式向串口发送数据，LCD 显示传送进度，通过串口上位机可以看到传输的内容。

32.3 程序设计

32.3.1 DMA 的 HAL 库驱动

DMA 在 HAL 库中的驱动代码在 stm32h7xx_hal_dma.c 文件（及其头文件）中。

1. HAL_DMA_Init 函数

DMA 的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_DMA_Init(DMA_HandleTypeDef *hdma);
```

- **函数描述：**
  用于初始化 DMA1，DMA2 和 BDMA。
- **函数形参：**
  形参 1 是 DMA_HandleTypeDef 结构体类型指针变量，其定义如下：

```
typedef struct __DMA_HandleTypeDef
{
  void                          *Instance;    /* 寄存器基地址 */
  DMA_InitTypeDef                Init;        /* DAM 通信参数 */
  HAL_LockTypeDef                Lock;        /* DMA 锁对象 */
  __IO HAL_DMA_StateTypeDef      State;       /* DMA 传输状态 */
  void                          *Parent;      /* 父对象状态，HAL 库处理的中间变量 */
  void (*XferCpltCallback)( struct __DMA_HandleTypeDef *hdma);/*DMA 传输完成回调*/
  void (* XferHalfCpltCallback)( struct __DMA_HandleTypeDef * hdma);
  /* DMA 一半传输完成回调 */
  void (* XferM1CpltCallback)( struct __DMA_HandleTypeDef * hdma);
  /* DMA 传输完整的 Memory1 回调 */
  void (* XferM1HalfCpltCallback)( struct __DMA_HandleTypeDef * hdma);
  /* DMA 传输半完全内存回调 */
  void (* XferErrorCallback)( struct __DMA_HandleTypeDef * hdma);
  /* DMA 传输错误回调 */
  void (* XferAbortCallback)( struct __DMA_HandleTypeDef * hdma);
  /* DMA 传输中止回调 */
 __IO uint32_t                   ErrorCode;              /* DMA 存取错误代码 */
 uint32_t                        StreamBaseAddress;      /* DMA 数据流基地址 */
 uint32_t                        StreamIndex;            /* DMA 数据流索引号 */
 DMAMUX_Channel_TypeDef         *DMAmuxChannel;          /* DMAMUX 信道基础地址 */
 DMAMUX_ChannelStatus_TypeDef *DMAmuxChannelStatus;     /* DMAMUX 通道状态基础地址*/
```

```
  uint32_t                      DMAmuxChannelStatusMask; /* DMAMUX 通道状态掩码 */
  DMAMUX_RequestGen_TypeDef     *DMAmuxRequestGen;/* DMAMUX 请求生成器基地地址 */
  DMAMUX_RequestGenStatus_TypeDef  *DMAmuxRequestGenStatus;
                                                  /* DMAMUX 请求生成器状态地址 */
  uint32_t             DMAmuxRequestGenStatusMask;/* DMAMUX 请求生成器状态掩码 */
}DMA_HandleTypeDef;
```

这个结构体内容比较多，上面已注释中文翻译，下面列出几个成员说明一下。

Instance：是用来设置寄存器基地址，例如要设置为 DMA2 的数据流 7，那么取值为 DMA2_Stream7。

Parent：是 HAL 库处理中间变量，用来指向 DMA 通道外设句柄。

StreamBaseAddress 和 StreamIndex 是数据流基地址和索引号，这个是 HAL 库处理的时候会自动计算，用户无需设置。

其他成员变量是 HAL 库处理过程状态标识变量，这里就不做过多讲解。

接下来我们重点介绍 Init，它是 DMA_InitTypeDef 结构体类型变量，该结构体定义如下：

```
typedef struct
{
  uint32_t Request;          /* 请求设置，设置是哪个外设请求的 */
  uint32_t Direction;        /* 传输方向，例如存储器到外设 DMA_MEMORY_TO_PERIPH */
  uint32_t PeriphInc;        /* 外设（非）增量模式，非增量模式 DMA_PINC_DISABLE */
  uint32_t MemInc;           /* 存储器（非）增量模式，增量模式 DMA_MINC_ENABLE */
  uint32_t PeriphDataAlignment;/* 外设数据大小：8/16/32 位 */
  uint32_t MemDataAlignment;    /* 存储器数据大小：8/16/32 位 */
  uint32_t Mode;                /* 模式：外设流控模式/循环模式/普通模式 */
  uint32_t Priority;            /* DMA 优先级：低/中/高/非常高 */
  uint32_t FIFOMode;            /* FIFO 模式开启或者禁止 */
  uint32_t FIFOThreshold;       /* FIFO 阈值选择 */
  uint32_t MemBurst;            /* 存储器突发模式：单次/4 个节拍/8 个节拍/16 个节拍 */
  uint32_t PeriphBurst;         /* 外设突发模式：单次/4 个节拍/8 个节拍/16 个节拍 */
}DMA_InitTypeDef;
```

该结构体成员变量非常多，但是每个成员变量配置的基本都是 DMA_SxCR 寄存器和 DMA_SxFCR 寄存器的相应位。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

以 DMA 的方式传输串口数据配置步骤

1）使能 DMA 时钟

DMA 的时钟使能是通过 AHB1ENR 寄存器来控制的，这里我们要先使能时钟，才可以配置 DMA 相关寄存器。HAL 库方法为：

```
__HAL_RCC_DMA1_CLK_ENABLE(); /* DMA1 时钟使能 */
__HAL_RCC_DMA2_CLK_ENABLE(); /* DMA2 时钟使能 */
```

2）初始化 DMA

调用 HAL_DMA_Init 函数初始化 DMA 的相关参数，包括配置通道，外设地址，存储器地址，传输数据量等。

HAL 库为了处理各类外设的 DMA 请求，在调用相关函数之前，需要调用一个宏定义标识符，来连接 DMA 和外设句柄。例如要使用串口 DMA 发送，所以方式为：

```
__HAL_LINKDMA(&g_uart1_handle, hdmatx, g_dma_handle);
```

其中 g_uart1_handle 是串口初始化句柄，我们在 usart.c 中定义过了。g_dma_handle 是 DMA 初始化句柄。hdmatx 是外设句柄结构体的成员变量，在这里实际就是 g_uart1_handle 的成员变量。在 HAL 库中，任何一个可以使用 DMA 的外设，它的初始化结构体句柄都会有一个 DMA_HandleTypeDef 指针类型的成员变量，是 HAL 库用来做相关指向的。hdmatx 就是 DMA_HandleTypeDef 结构体指针类型。

这句话的含义就是把 g_uart1_handle 句柄的成员变量 hdmatx 和 DMA 句柄 g_dma_handle 连接起来，是纯软件处理，没有任何硬件操作。

这里我们就点到为止，如果大家要详细了解 HAL 库指向关系，请查看本实验宏定义标识符__HAL_LINKDMA 的定义和调用方法，就会很清楚了。

3）使能串口的 DMA 发送，启动传输

串口 1 的 DMA 发送实际是串口控制寄存器 CR3 的位 7 来控制的，在 HAL 库中操作该寄存器来使能串口 DMA 发送的函数为 HAL_UART_Transmit_DMA。

这里大家需要注意，调用该函数后会开启相应的 DMA 中断，对于本章实验，我们是通过查询的方法获取数据传输状态，所以并没有做中断相关处理，也没有编写中断服务函数。

HAL 库还提供了对串口的 DMA 发送的停止，暂停，继续等操作函数：

```
HAL_StatusTypeDef HAL_UART_DMAStop(UART_HandleTypeDef *huart);     /* 停止 */
HAL_StatusTypeDef HAL_UART_DMAPause(UART_HandleTypeDef *huart);    /* 暂停 */
HAL_StatusTypeDef HAL_UART_DMAResume(UART_HandleTypeDef *huart);   /* 恢复 */
```

4）查询 DMA 传输状态

在 DMA 传输过程中，我们要查询 DMA 传输通道的状态，使用的方法是：

```
__HAL_DMA_GET_FLAG(&g_dma_handle, DMA_FLAG_TCIF3_7);
```

获取当前传输剩余数据量：

```
__HAL_DMA_GET_COUNTER(&g_dma_handle);
```

同样，我们也可以设置对应的 DMA 数据流传输的数据量大小，函数为：

```
__HAL_DMA_SET_COUNTER (&g_dma_handle, 1000);
```

DMA 相关的库函数我们就讲解到这里，大家可以查看固件库中文手册详细了解。

5）DMA 中断使用方法

DMA 中断对于每个数据流都有一个中断服务函数，比如 DMA2_Stream7 的中断服务函数为 DMA2_Stream7_IRQHandler。HAL 库提供了通用 DMA 中断处理函数 HAL_DMA_IRQHandler，在该函数内部，会对 DMA 传输状态进行分析，然后调用相应的中断处理回调函数：

```
void HAL_UART_TxCpltCallback(UART_HandleTypeDef *huart);     /* 发送完成回调函数 */
void HAL_UART_TxHalfCpltCallback(UART_HandleTypeDef *huart);/* 发送一半回调函数 */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart);     /* 接收完成回调函数 */
void HAL_UART_RxHalfCpltCallback(UART_HandleTypeDef *huart);/* 接收一半回调函数 */
void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart);      /* 传输出错回调函数 */
```

32.3.2 程序流程图

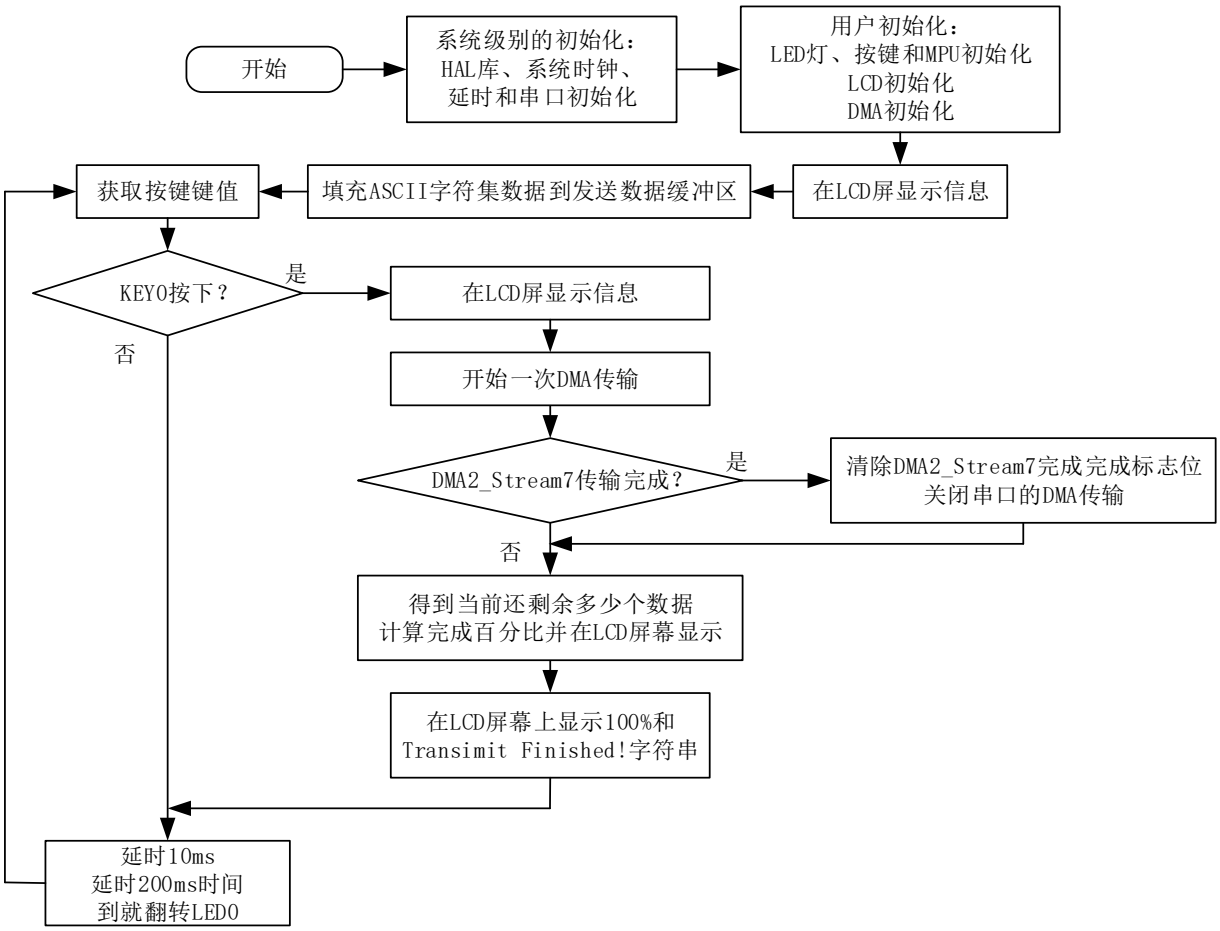


图 32.3.2.1 DMA 实验程序流程图

32.3.3 程序解析

1. DMA 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。DMA 驱动源码包括两个文件：dma.c 和 dma.h。

dma.h 头文件只有函数的声明，下面我们直接介绍 dma.c 的程序，下面是 DMA 的初始化函数，其定义如下：

```
/**
 * @brief   串口 TX DMA 初始化函数
 * @note    这里的传输形式是固定的, 这点要根据不同的情况来修改
 *          从存储器 -> 外设模式/8 位数据宽度/存储器增量模式
 * @param   dma_stream_handle : DMA 数据流,DMA1_Stream0~7/DMA2_Stream0~7
 * @retval 无
 */
void dma_init(DMA_Stream_TypeDef *dma_stream_handle)
{
    /* 得到当前 stream 是属于 DMA2 还是 DMA1 */
    if ((uint32_t)dma_stream_handle > (uint32_t)DMA2)
    {
        __HAL_RCC_DMA2_CLK_ENABLE();          /* DMA2 时钟使能 */
    }
    else
    {
```

```
        __HAL_RCC_DMA1_CLK_ENABLE();          /* DMA1 时钟使能 */
    }
    /* 将 DMA 与 USART1 联系起来(发送 DMA) */
    __HAL_LINKDMA(&g_uart1_handle,hdmatx, g_dma_handle);

    /* Tx DMA 配置 */
   g_dma_handle.Instance = dma_stream_handle;            /* 数据流选择 */
   g_dma_handle.Init.Request = DMA_REQUEST_USART1_TX;    /* DMA 通道选择 */
   g_dma_handle.Init.Direction = DMA_MEMORY_TO_PERIPH;   /* 存储器到外设 */
   g_dma_handle.Init.PeriphInc = DMA_PINC_DISABLE;       /* 外设非增量模式 */
   g_dma_handle.Init.MemInc = DMA_MINC_ENABLE;           /* 存储器增量模式 */
   g_dma_handle.Init.PeriphDataAlignment=DMA_PDATAALIGN_BYTE;/*外设数据长度:8 位*/
   g_dma_handle.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE/*存储器数据长度:8 位*/
   g_dma_handle.Init.Mode = DMA_NORMAL;                 /* 外设流控模式 */
   g_dma_handle.Init.Priority = DMA_PRIORITY_MEDIUM;    /* 中等优先级 */
   g_dma_handle.Init.FIFOMode = DMA_FIFOMODE_DISABLE;   /* 关闭 FIFO 模式 */
   g_dma_handle.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_FULL; /* FIFO 阈值配置 */
   g_dma_handle.Init.MemBurst = DMA_MBURST_SINGLE;    /* 存储器突发单次传输 */
    g_dma_handle.Init.PeriphBurst = DMA_PBURST_SINGLE;/* 外设突发单次传输 */
    HAL_DMA_DeInit(&g_dma_handle);
    HAL_DMA_Init(&g_dma_handle);
}
```

该函数是一个通用的 DMA 配置函数，DMA1/DMA2 的所有通道，都可以利用该函数配置，不过有些固定参数可能要适当修改（比如位宽，传输方向等）。该函数在外部只能修改 DMA 数据流编号和通道号，更多的其他设置只能在该函数内部修改。对照前面的配置步骤的详细讲解来分析这部分代码即可。

2. main.c 代码

在 main.c 里面编写如下代码：

```
/* 要循环发送的字符串 */
const uint8_t TEXT_TO_SEND[] = {"正点原子 STM32 DMA 串口实验"};
#define SEND_BUF_SIZE  (sizeof(TEXT_TO_SEND) + 2) * 200    /* 发送数据长度 */
uint8_t g_sendbuf[SEND_BUF_SIZE];                          /* 发送数据缓冲区 */

int main(void)
{
    uint8_t  key = 0;
    uint16_t i, k;
    uint16_t len;
    uint8_t  mask = 0;
    float pro = 0;                              /* 进度 */

    sys_cache_enable();                         /* 打开 L1-Cache */
    HAL_Init();                                 /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);         /* 设置时钟, 400Mhz */
    delay_init(400);                            /* 延时初始化 */
    usart_init(115200);                         /* 串口初始化为 115200 */
    mpu_memory_protection();                    /* 保护相关存储区域 */
    led_init();                                 /* 初始化 LED */
    sdram_init();                               /* 初始化 SDRAM */
    lcd_init();                                 /* 初始化 LCD */
    key_init();                                 /* 初始化按键 */
    dma_init(DMA2_Stream7);                     /* 初始化 DMA */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "DMA TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Start", RED);
```

```
    len = sizeof(TEXT_TO_SEND);
    k = 0;

    for (i = 0; i < SEND_BUF_SIZE; i++) /* 填充 ASCII 字符集数据 */
    {
        if (k >= len)    /* 入换行符 */
        {
            if (mask)
            {
                g_sendbuf[i] = 0x0a;
                k = 0;
            }
            else
            {
                g_sendbuf[i] = 0x0d;
                mask++;
            }
        }
        else      /* 复制 TEXT_TO_SEND 语句 */
        {
            mask = 0;
            g_sendbuf[i] = TEXT_TO_SEND[k];
            k++;
        }
    }
    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)    /* KEY0 按下 */
        {
            printf("\r\nDMA DATA:\r\n");
            lcd_show_string(30, 130, 200, 16, 16, "Start Transimit....", BLUE);
            lcd_show_string(30, 150, 200, 16, 16, "    %", BLUE);/* 显示百分号 */
             /* 开始一次 DMA 传输！ */
            HAL_UART_Transmit_DMA(&g_uart1_handle,g_sendbuf,SEND_BUF_SIZE);

            /* 等待 DMA 传输完成，此时我们来做另外一些事情，比如点灯
             * 实际应用中，传输数据期间，可以执行另外的任务 */
            while (1)
            {
              /* 等待 DMA2_Steam7 传输完成 */
                if (__HAL_DMA_GET_FLAG(&g_dma_handle, DMA_FLAG_TCIF3_7))
                {
                 /* 清除 DMA2_Steam7 传输完成标志 */
                    __HAL_DMA_CLEAR_FLAG(&g_dma_handle, DMA_FLAG_TCIF3_7);
                    HAL_UART_DMAStop(&g_uart1_handle);/* 传输完成以后关闭串口 DMA */
                    break;
                }
                /*得到当前还剩余多少个数据*/
                pro =__HAL_DMA_GET_COUNTER(&g_dma_handle);
                len = SEND_BUF_SIZE;       /* 总长度 */
                pro = 1 - (pro / len);     /* 得到百分比 */
                pro *= 100;                /* 扩大 100 倍 */
                lcd_show_num(30, 150, pro, 3, 16, BLUE);
            }
            lcd_show_num(30, 150, 100, 3, 16, BLUE);     /* 显示 100% */
            /* 提示传送完成 */
            lcd_show_string(30, 130, 200, 16, 16, "Transimit Finished!", BLUE);
        }

        i++;
        delay_ms(10);

        if (i == 20)
```

```
        {
            LED0_TOGGLE();   /* LED0 闪烁,提示系统正在运行 */
            i = 0;
        }
    }
}
```

main 函数的流程大致是：先初始化发送数据缓冲区 g_sendbuf 的值，然后通过 KEY0 开启串口 DMA 发送，在发送过程中，通过__HAL_DMA_GET_COUNTER(&g_dma_handle)获取当前还剩余的数据量来计算传输百分比，最后在传输结束之后清除相应标志位，提示已经传输完成。

32.4 下载验证

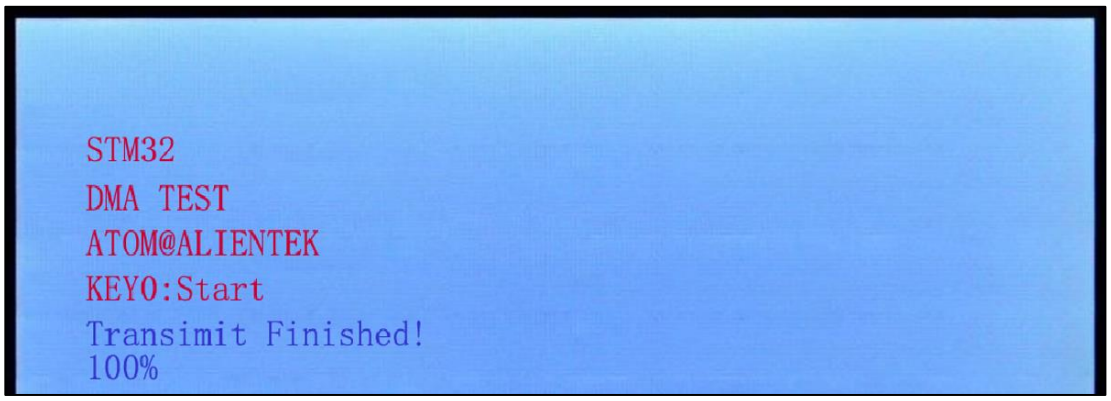
将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 32.4.1 所示：

图 32.4.1 DMA 实验测试图

我们打开串口调试助手，然后按 KEY0，可以看到串口显示如图 32.4.2 所示的内容：

图 32.4.2 串口收到的数据内容

可以看到串口收到了开发板发送过来的数据，同时可以看到 TFTLCD 上显示了进度等信息，如图 32.4.3 所示：



32.4.3 DMA 串口数据传输中

至此，我们整个 DMA 实验就结束了，希望大家通过本章的学习，掌握 STM32H743 的 DMA 使用。DMA 是个非常好的功能，它不但能减轻 CPU 负担，还能提高数据传输速度，合理的应用 DMA，往往能让你的程序设计变得简单。

第三十三章 ADC 实验

本章，我们将介绍 STM32H743 的 ADC（Analog-to-digital converters，模数转换器）功能。我们通过四个实验来学习 ADC，分别是单通道 ADC 采集实验、单通道 ADC 采集（DMA 读取）实验、多通道 ADC 采集（DMA 读取）实验和单通道 ADC 过采样（26 位分辨率）实验。

本章分为如下几个小节：

33.1 ADC 简介
33.2 单通道 ADC 采集实验
33.3 单通道 ADC 采集（DMA 读取）实验
33.4 多通道 ADC 采集（DMA 读取）实验
33.5 单通道 ADC 过采样（26 位分辨率）实验

33.1 ADC 简介

STM32H743xx 系列有 3 个 ADC，都可以独立工作，其中 ADC1 和 ADC2 还可以组成双重模式（提高采样率），ADC3 比较独立，我们在下一章会有详细的讲解。

STM32H743 的 ADC 主要特性我们可以总结为以下几条：

1、可配置 16 位、14 位、12 位、10 位或 8 位分辨率，降低分辨率可以缩短转换时间，转换时间越短，可以做到的采样率就越高。

2、每个 ADC 支持多达 20 个的采集通道，其中有 6 路快速通道和 14 路慢速通道，慢速和快速的区别主要是支持的最高采样率不同，慢速通道要比快速通道低。这些通道的 A/D 转换可以单次、连续、扫描或间断模式执行。

3、ADC 的结果可以左对齐或右对齐方式存储在 32 位数据寄存器中。

4、ADC 具有五条专用的内部通道，内部参考电压 (V_{REFINT}) 连接到 ADC3_INP/INN19，内部温度传感器 (V_{SENSE}) 连接到 ADC3_INP/INN18 和 V_{BAT} 监测通道 (V_{BAT} /4) 连接到 ADC3_INP/INN17，这三个都是连接到 ADC3。另外 DAC 内部通道 1 连接到 ADC2_INP/INN16，DAC 内部通道 2 连接到 ADC2_INP/INN17。

5、支持过采样，最高可以达到 26 位采样率。

6、每个 ADC 支持三路模拟看门狗。

7、支持单独输入和差分输入（可按通道进行编程）。

8、ADC 输入范围：$V_{REF-} \leqslant V_{IN} \leqslant V_{REF+}$。由 V_{REF-}、V_{REF+}、V_{DDA} 和 V_{SSA} 这四个外部引脚决定。一般我们把 V_{SSA} 和 V_{REF-} 接地，把 V_{REF+} 和 V_{DDA} 接到 3.3V，所以得到 ADC 的输入电压范围是：0~3.3V。注意不要接超出这个范围的电压进来，否则容易烧坏芯片。

9、自校准（偏移校准和线性度校准）。

10、最多 4 条注入转换序列，16 条常规转换序列。

上面我们列出的一些特性都是 ADC 重要的特性，其它特性请查看参考手册。

下面来介绍 ADC（仅限 ADC1 或 ADC2）的框图：

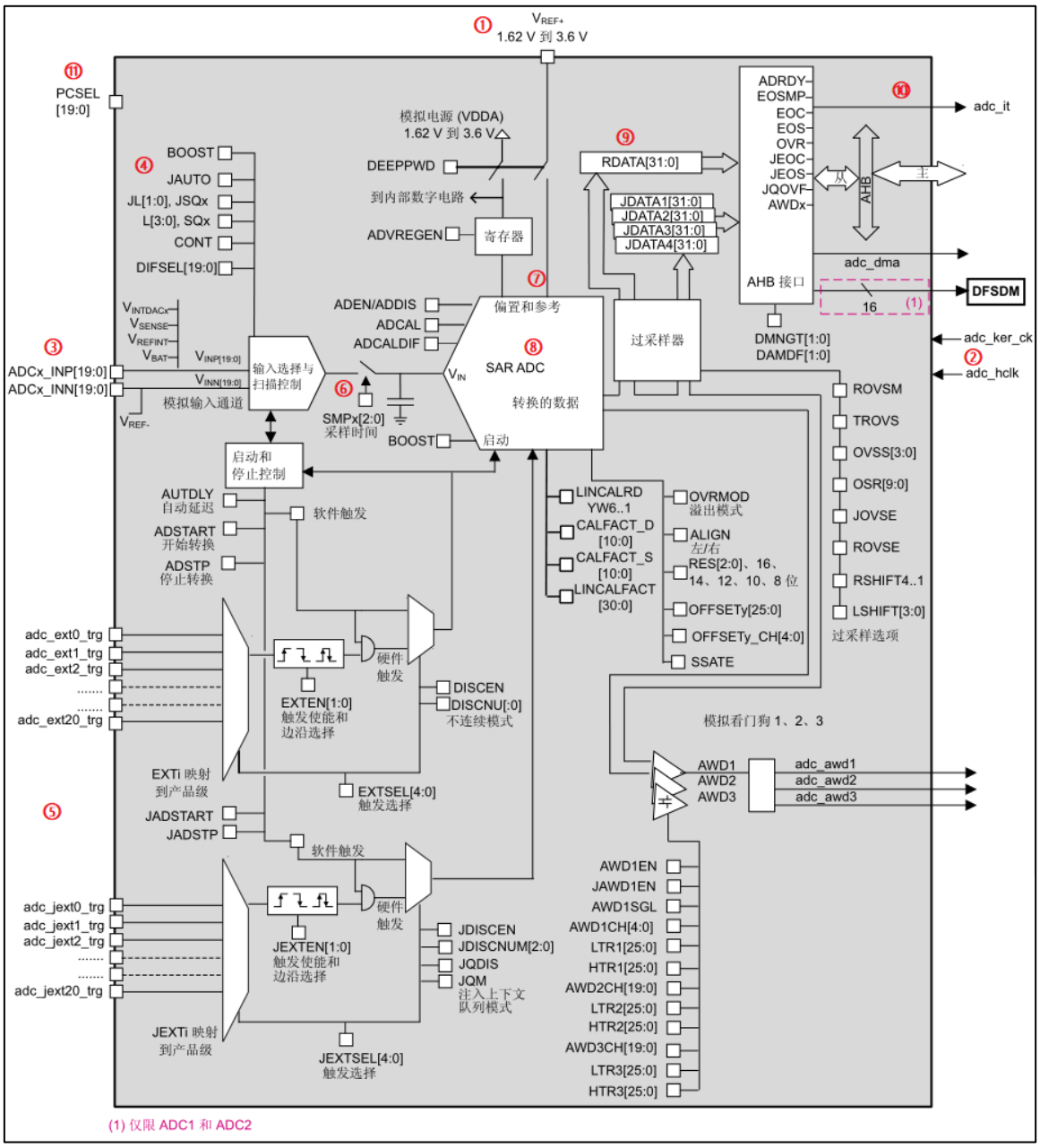


图 33.1.1 ADC 框图

图中，我们标记了 11 处位置，分别如下：

① V_{REF+}电压

V_{REF+} 是正模拟参考电压输入，选择范围是 1.62V~3.6V，开发板上我们一般给 V_{REF+}接入的电压时 3.3V，所以得到开发板上的 ADC 测量范围是 0~3.3V。

② ADC 的双时钟域架构

ADC 有两种时钟源可以选择，分别是：

（1）adc_hclk（属于同步时钟），来自 AHB 总线的系统时钟，ADC1 和 ADC2 处在 200MHZ 的 AHB1 总线时钟。可以通过 ADCx_CCR 寄存器的 CKMODE[1：0]位来选择不同分频的 AHB1 总线时钟，有以下的四种情况：

CKMODE[1：0]=00，这是异步时钟模式选择的配置，适用于下面要讲的 adc_ker_ck 时钟。

CKMODE[1：0]=01，adc_hclk/1（同步时钟模式）

CKMODE[1：0]=10，adc_hclk/2（同步时钟模式）

CKMODE[1：0]=11，adc_hclk/4（同步时钟模式）

比如我们选择 4 分频的 adc_hclk，得到的频率是 50MHZ，但是数据手册限制 ADC 时钟频

率最高是 36MHz，说明这样配置就超频了，这是不可行的，因为超频误差会比较大。我们可以降低 AHB 总线时钟，但是这样会影响我们其它外设的性能，所以为了系统能达到最优的性能，我们一般不会选择 adc_hclk 作为 ADC 的时钟源，于是我们选择下面要说的这种时钟源。

（2）adc_ker_ck（属于异步时钟），可以通过 RCC_D3CCIPR 寄存器的 ADCSEL [1：0]位来选择不同的时钟源，前提是前面提到的 CKMODE[1：0]=00。ADC 异步时钟模式下可以选择以下的时钟源：

ADCSEL [1：0]=00，pll2_p_ck 作为 ADC 时钟源（复位后的默认值）

ADCSEL [1：0]=01，pll3_r_ck 作为 ADC 时钟源

ADCSEL [1：0]=10，per_ck 作为 ADC 时钟源

实际的例程中我们选择 per_ck 作为 ADC 时钟源，而 per_ck 时钟可为 hse_ck、hsi_ker_ck 或 csi_ker_ck，通过 RCC_D1CCIPR 寄存器的 CKPERSEL 位选择，默认选择 hsi_ker_ck 作为 per_ck 的时钟源。hsi_ker_ck 时钟源就是来自频率为 64MHz 的高速内部 RC 振荡器（HSI）。

选择了 adc_ker_ck 时钟源作为 ADC 的时钟，则可以通过 ADCx_CCR 寄存器的 PRESC[3:0]位进行分频，可以是 1、2、4、6、8、10、12、16、32、64、128、256 这 12 种分频系数。

上面的分析请结合下面的 ADC 时钟方案图理解。

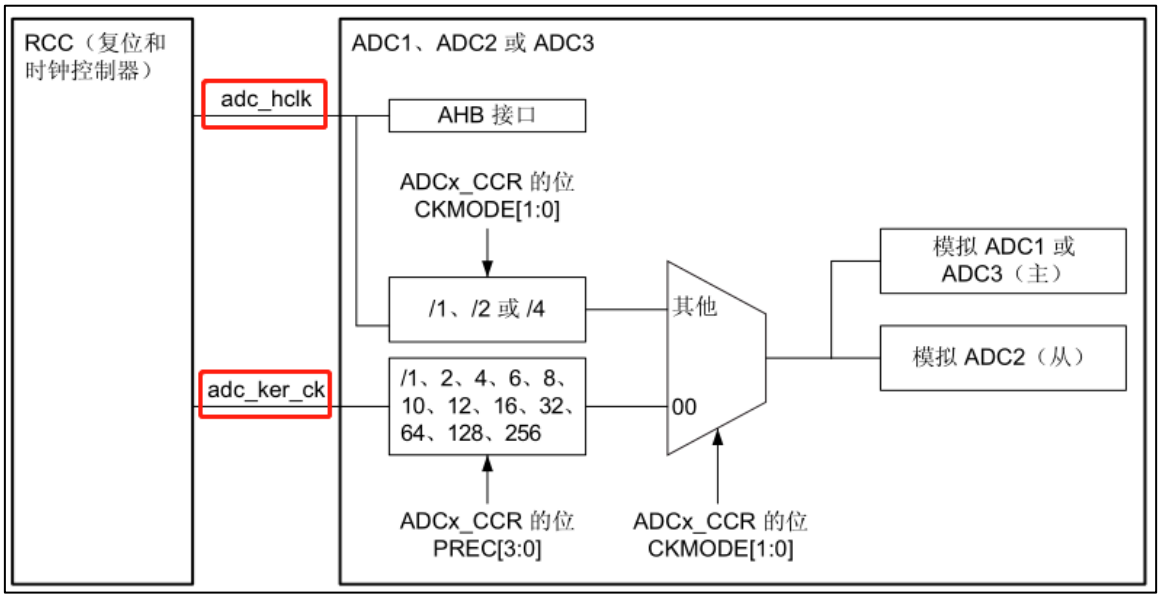


图 33.1.2 ADC 时钟方案

③ 输入通道

ADC 总共有 20 个输入通道。注意：STM32H7 的 ADC 支持单端/差分转换，由寄存器 ADCx_DIFSEL（x=1~3）控制，该寄存器默认是 0（单端模式），配置为 1（则为差分模式）。因为 H743 的 ADC 支持差分通道输入，因此有 ADCx_INP[19:0]和 ADCx_INN[19:0]两组通道。其中，INP 是差分正向输入，INN 是差分反向输入。ADC_INP[0:5]和 ADC_INN[0:5]是快速模拟输入。ADC_INP[6:19]和 ADC_INN[6:19]是慢速模拟输入。如果我们使用单端输入，则只有 ADCx_INP[19:0]有效，ADCx_INN[19:0]在内部自动接 V_{SSA}。

ADC 连接 5 路内部模拟输入，分别是：

（1）内部参考电压 (V_{REFINT}) 连接到 ADC3_INP/INN19。

（2）内部温度传感器 (V_{SENSE}) 连接到 ADC3_INP/INN18。

（3）V_{BAT} 监测通道 ($V_{BAT}/4$) 连接到 ADC3_INP/INN17。

（4）DAC 内部通道 1，连接到 ADC2_INP/INN16。

（5）DAC 内部通道 2，连接到 ADC2_INP/INN17。

为了方便大家查询 ADC 通道和 IO 的对应关系，给大家整理了表 33.1.1。大多数情况，我们都是使用单端模式。前面也说了单端模式下，ADCx_INN[19:0]在内部自动接 V_{SSA}，所以表 33.1.1 中的通道 0~通道 19 指的是 ADCx_INP[19:0]。

ADC1	IO	ADC2	IO	ADC3	IO

通道 0	PA0_C	通道 0	PA0_C	通道 0	PC2_C
通道 1	PA1_C	通道 1	PA1_C	通道 1	PC3_C
通道 2	PF11	通道 2	PF13	通道 2	PF9
通道 3	PA6	通道 3	PA6	通道 3	PF7
通道 4	PC4	通道 4	PC4	通道 4	PF5
通道 5	PB1	通道 5	PB1	通道 5	PF3
通道 6	PF12	通道 6	PF14	通道 6	PF10
通道 7	PA7	通道 7	PA7	通道 7	PF8
通道 8	PC5	通道 8	PC5	通道 8	PF6
通道 9	PB0	通道 9	PB0	通道 9	PF4
通道 10	PC0	通道 10	PC0	通道 10	PC0
通道 11	PC1	通道 11	PC1	通道 11	PC1
通道 12	PC2	通道 12	PC2	通道 12	PC2
通道 13	PC3	通道 13	PC3	通道 13	PH2
通道 14	PA2	通道 14	PA2	通道 14	PH3
通道 15	PA3	通道 15	PA3	通道 15	PH4
通道 16	PA0	通道 16	DAC 内部通道 1	通道 16	PH5
通道 17	PA1	通道 17	DAC 内部通道 2	通道 17	备用电源
通道 18	PA4	通道 18	PA4	通道 18	内部温度传感器
通道 19	PA5	通道 19	PA5	通道 19	内部参考电压

表 33.1.1 ADC 通道表

④ 转换序列

可以将转换分为两组：常规转换组和注入转换组。常规转换组最多允许 16 个通道进行转换。注入转换组最多允许 4 个通道进行转换。

如何理解常规转换组和注入转换组？常规转换组相当于你正常运行的程序，而注入转换组就相当于中断。在你程序正常执行的时候，中断是可以打断你的执行的，获得优先执行的权利。所以注入转换组可以打断常规转换组的转换，获得优先转换的权利，在注入转换组转换完成后，常规转换组才得以继续转换。

为了便于理解，请看常规转换组和注入转换组的转换优先级对比图，如图 33.1.3 所示：

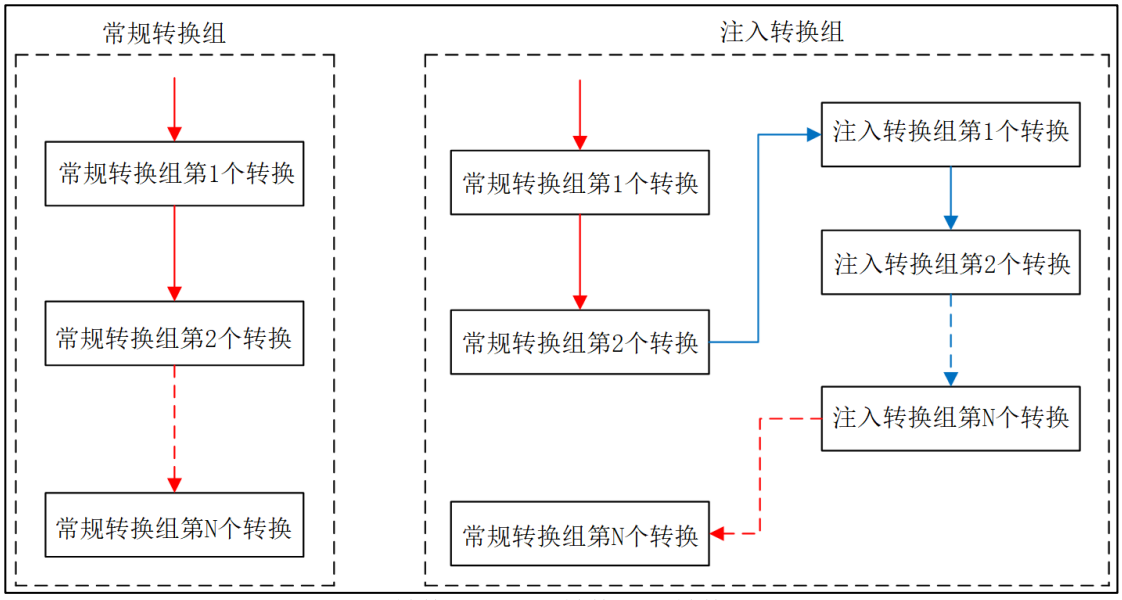


图 33.1.3 常规转换组和注入转换组的转换优先级对比图

常规转换组最多允许 16 个通道进行转换，注入转换组最多允许 4 个通道进行转换，那么转换的顺序怎么设置的？我们把这个转换顺序分别称为常规序列和注入序列。

（1）常规序列

常规序列在 ADCx_SQRy 寄存器中设置，每个 ADC 都有 4 个 SQR 寄存器，比如 ADC1 的 SQR 寄存器有 ADC1_SQR1~ADC1_SQR4。这四个寄存器怎么来设置常规序列的呢？下面通过表 33.1.2 给大家说明。

常规序列寄存器控制关系汇总			
寄存器	寄存器位	功能	取值
SQR1	SQL [3 : 0]	设置规则序列要转换的通道数	0~15
	SQ1[4 : 0]	设置第 1 个转换的通道	输入通道 0~19
	SQ2 [4 : 0]	设置第 2 个转换的通道	输入通道 0~19
	SQ3 [4 : 0]	设置第 3 个转换的通道	输入通道 0~19
	SQ4 [4 : 0]	设置第 4 个转换的通道	输入通道 0~19
SQR2	SQ5 [4 : 0]	设置第 5 个转换的通道	输入通道 0~19
	SQ6 [4 : 0]	设置第 6 个转换的通道	输入通道 0~19
	SQ7 [4 : 0]	设置第 7 个转换的通道	输入通道 0~19
	SQ8 [4 : 0]	设置第 8 个转换的通道	输入通道 0~19
	SQ9 [4 : 0]	设置第 9 个转换的通道	输入通道 0~19
SQR3	SQ10 [4 : 0]	设置第 10 个转换的通道	输入通道 0~19
	SQ11 [4 : 0]	设置第 11 个转换的通道	输入通道 0~19
	SQ12 [4 : 0]	设置第 12 个转换的通道	输入通道 0~19
	SQ13 [4 : 0]	设置第 13 个转换的通道	输入通道 0~19
	SQ14 [4 : 0]	设置第 14 个转换的通道	输入通道 0~19
SQR3	SQ15 [4 : 0]	设置第 15 个转换的通道	输入通道 0~19
	SQ16 [4 : 0]	设置第 16 个转换的通道	输入通道 0~19

表 33.1.2 常规序列寄存器控制关系汇总表

从上表可以知道，当我们想设置 ADC 的某个输入通道在常规序列的第 1 个转换，只需要把相应的输入通道号的值写入 SQR1 寄存器中的 SQ1[4:0]位即可。例如想让输入通道 5 先进行转换，那么就可以把 5 这个数值写入 SQ1[4:0]位。如果还想让输入通道 8 在第 2 个转换，那么就可以把 8 这个数值写入 SQ2[4:0]位。最后还要设置你的这个规则序列的输入通道个数，只需把输入通道个数写入 SQR1 的 SQL[3:0]位。注意：写入 0 到 SQL[3:0]位，表示这个常规序列有 1 个输入通道的意思，而不是 0 个输入通道。

（2）注入序列

注入序列和常规序列差不多，决定的是注入转换组的顺序。注入组最大允许 4 个通道输入，它的注入序列由 JSQR 寄存器配置。注入序列寄存器 JSQR 控制关系如表 33.1.3 所示：

注入序列寄存器控制关系汇总			
寄存器	寄存器位	功能	取值
JSQR	JSQ1 [4 : 0]	设置第 1 个转换的通道	输入通道 0~19
	JSQ2 [4 : 0]	设置第 2 个转换的通道	输入通道 0~19
	JSQ3 [4 : 0]	设置第 3 个转换的通道	输入通道 0~19
	JSQ4 [4 : 0]	设置第 4 个转换的通道	输入通道 0~19
	JL [1 : 0]	设置注入序列要转换的通道数	0~3

表 33.1.3 注入序列寄存器控制关系汇总表

注入序列的长度写入 JL [1 : 0]位，范围是 0~3。注意：写入 0 表示这个注入序列有一个输入通道，而不是 0 个输入通道。

⑤ 触发源

ADC 的触发转换有两种方法：分别是通过软件或外部事件（也就是硬件）触发转换。

我们先来看看通过软件触发转换的方法，常规通道由 ADCx_CR 寄存器的 ADSTART 位触发，注入通道由 ADCx_CR 寄存器的 JADSTART 位触发。方法是：通过对 ADCx_CR 寄存器的 ADSTART（JADSTART）位写 1 开始转换，转换结束由硬件清零该位，这个控制 ADC 转换的方式非常简单。

另一种就是通过外部事件触发转换的方法，如定时器和输入引脚触发等等，具体请看

《STM32H7xx 参考手册_V3（中文版）.pdf》的 825 页和 826 页的表 192 和表 193。外部事件触发转换可分为：常规通道的外部触发和注入通道的外部触发两种。

adc_ext_trg[20:0]，对应的就是常规通道的外部触发，共有 21 路。

adc_jext_trg[20:0]，对应的就是注入通道的外部触发，共有 21 路。

如果选择硬件触发，则需要选择相应的硬件触发事件和触发边沿等，然后由外部硬件事件来触发 ADC 的采集（外部事件触发配置 ADSTART 位为 1）。

硬件触发事件由 ADCx_CFGR 寄存器的 EXTSEL[4:0]和 ADCx_JSQR 寄存器的 JEXTSEL[4:0]位来选择，分别是常规转换组和注入转换组的触发源选择。而触发边沿是通过 ADCx_CFGR 寄存器的 EXTEN[1:0]和 ADCx_JSQR 寄存器的 JEXTEN[1:0]位来选择。

⑥ **转换时间**

STM32H7 的 ADC 总转换时间的计算公式如下：

T_{CONV} = 采样时间（T_{SMPL}） + 逐次逼近时间（T_{SAR}）

采样时间（T_{SMPL}）可通过 ADCx_SMPR1 和 ADCx_SMPR2 寄存器中的 SMP[2:0]位编程，ADC_SMPR1 控制的是通道 0~9，ADC_SMPR2 控制的是通道 10~19。所有通道都可以通过编程来控制使用不同的采样时间，可选采样时间值如下：

- SMP = 000：1.5 个 ADC 时钟周期
- SMP = 001：2.5 个 ADC 时钟周期
- SMP = 010：8.5 个 ADC 时钟周期
- SMP = 011：16.5 个 ADC 时钟周期
- SMP = 100：32.5 个 ADC 时钟周期
- SMP = 101：64.5 个 ADC 时钟周期
- SMP = 110：387.5 个 ADC 时钟周期
- SMP = 111：810.5 个 ADC 时钟周期

逐次逼近时间（T_{SAR}）是由分辨率决定的，分辨率通过对 ADCx_CFGR 寄存器的 RES[1:0]位进行编程，可将分辨率配置为 16 位、14 位、12 位、10 位、8 位。而逐次逼近时间和分辨率的对应关系如下表所示：

RES	T_{SAR} ADC 时钟周期	T_{SAR} (ns), F_{ADC}=24 MHz	T_{ADC}（ADC 时钟周期）（采样时间 = 1.5 个 ADC 时钟周期）	T_{ADC} (ns), F_{ADC}=24 MHz
16	8.5 个 ADC 时钟周期	354.2	10 个 ADC 时钟周期	416.7
14	7.5 个 ADC 时钟周期	312.5	9 个 ADC 时钟周期	375
12	6.5 个 ADC 时钟周期	270.8	8 个 ADC 时钟周期	333.3
10	5.5 个 ADC 时钟周期	229.2	7 个 ADC 时钟周期	291.7
8	4.5 个 ADC 时钟周期	187.5	6 个 ADC 时钟周期	250.0

表 33.1.4 T_{SAR} 与分辨率的对应关系

举个例子，我们配置 SMP = 111，即设置最大采样周期，然后采用 16 位分辨率，那么得到：

T_{CONV} = 810.5 个 ADC 时钟周期 + 8.5 个 ADC 时钟周期 = 819 个 ADC 时钟周期

表格中，F_{ADC} 的频率是 24MHZ，我们的例程中 ADC 的时钟源是 64MHZ 的 hsi_ker_ck 经过 2 分频得到，即 32MHZ。我们就以 F_{ADC} 的频率为 32MHZ 来举例，可得到：

T_{CONV} = 819 个 ADC 时钟周期 = = 25.6us

⑦ **参考电压**

选择参考电压，我们可以设置参考电压来自外部的 V_{ref+}，也可以设置参考电压来自内部的稳压器。

⑧ **ADC 的核心**

ADC 的核心是一个 16 位的逐次逼近型 ADC 转换器，它根据我们设置好的参考电压、输入通道、启动条件等，执行模数转换。

⑨ 数据寄存器

常规转换组的转换结果会存放到 ADCx_DR 寄存器的 RDATA[31:0]位中，注入转换组的转换结果会存放到 ADCx_JDRy 寄存器的 JDATA1~4[31:0]位中。如果是使用双重模式，常规通道的数据则是存放在 ADC_CDR 寄存器。转换结果 CPU 可以通过 AHB 总线读取，同时也可以产生相关中断（adc_it）。

常规数据寄存器（ADCx_DR）（x=1~4）

常规数据寄存器 ADC_DR 是一个 32 位的寄存器。因为 ADC 的最大分辨率是 16 位，如果使用过采样，则分辨率可达 26 位，所以允许数据对齐方式。由 ADC_CFGR2 寄存器的 OVSS[3:0]位和 LSHIFT[3:0]位设置数据对齐方式。

细心的朋友可能发现，常规转换组最多有 16 个输入通道，而 ADC 常规数据寄存器只有一个，如果一个常规转换组用到好几个通道，数据怎么读取？如果使用多通道转换，那么这些通道的数据也会存放在 DR 里面，按照常规转换组的顺序，上一个通道转换的数据，会被下一个通道转换的数据覆盖掉，所以当通道转换完成后要及时把数据取走。比较常用的方法是使用 DMA 模式。当常规转换组的通道转换结束时，就会产生 DMA 请求，这样就可以及时把转换的数据搬运到用户指定的目的地址存放。如果没有使用 DMA 传输，可以通过判断 ADC_ISR 寄存器相关位来得到当前 ADC 的转换状态，从而进行控制。

注入数据寄存器（ADCx_JDRy）（x=1~3）（y=1~4）

每个 ADC 注入数据寄存器有 4 个，注入转换组最多有 4 个输入通道，刚好每个通道都有自己对应的数据寄存器。ADC_JDRx 寄存器是 32 位的，低 16 位有效，高 16 位保留，数据也同样需要选择对齐方式。也是由 ADC_CFGR2 寄存器的 OVSS[3:0]位和 LSHIFT[3:0]位设置数据对齐方式。

通用常则数据寄存器 ADC_CDR

常规数据寄存器 ADC_DR 仅适用于独立模式下，常规转换组的转换数据存储，而通用常规数据寄存器 ADC_CDR 适用于双重模式下，常规转换组的转换数据存储。在双重模式下，一般会配合 DMA 来传输数据。

⑩ 中断

对于每个 ADC 都可在下列情况下产生中断：

中断事件	事件标志	使能控制位
ADC 就绪	ADRDY	ADRDYIE
结束常规组的转换	EOC	EOCIE
常规组的转换序列结束	EOS	EOSIE
注入组的转换结束	JEOC	JEOCIE
注入组的转换序列结束	JEOS	JEOSIE
模拟看门狗 1 状态位置 1	AWD1	AWD1IE
模拟看门狗 2 状态位置 1	AWD2	AWD2IE
模拟看门狗 3 状态位置 1	AWD3	AWD3IE
采样阶段结束	EOSMP	EOSMPIE
上溢	OVR	OVRIE
注入上下文队列溢出	JQOVF	JQOVFIE

表 33.1.5 每个 ADC 的 ADC 中断

在表 33.1.5 中，前面 5 个中断都很好理解，我们从模拟看门狗中断介绍。

模拟看门狗中断发生条件：首先通过 ADCx_LTR 和 ADCx_HTR 寄存器设置低阈值和高阈值，然后开启了模拟看门狗中断后，当被 ADC 转换的模拟电压低于低阈值或者高于高阈值时，就会产生中断。例如我们设置高阈值是 3.0V，那么模拟电压超过 3.0V 的时候，就会产生模拟看门狗中断，低阈值的情况类似。

采样阶段结束：如果位 EOSMP 被硬件置 1，则说明采样阶段结束（仅限常规转换），然后可以通过对 EOSMP 位写 1 来清零该位。如果 EOSMPIE 位置 1，可以产生采样阶段结束中断。

上溢：如果常规转换后的数据未在新转换数据可用之前（由 CPU 或 DMA）读取，会由溢出标志（OVR）指示缓冲区溢出事件。如果 OVRIE 位置 1，可以产生一个溢出中断。

此外，我们还要知道常规组和注入组的转换结束后，除了产生中断外，还可以产生 DMA 请求，把转换好的数据存储在内存里面，防止读取不及时数据被覆盖。

⑪ 通道预选控制信号

通道预选控制信号，用于将 ADC 某个通道连接到对应的 GPIO 上。PCSEL[19:0]每个位对应一个通道，总共 20 个通道。这一点和以前的 STM32 系列不一样，在使用的时候，需要特别注意。

⑫ 单次转换模式和连续转换模式

单次转换模式和连续转换模式在框图中是没有标号，为了更好地学习后续的内容，这里简单给大家讲讲。

（1）单次转换模式

通过将 ADC_CFGR 寄存器的 CONT 位置 0 选择单次转换模式。该模式下，ADC 只执行一次转换。单次转换由 ADC_CR 寄存器的 ADSTART 位（只适用于常规转换组）或者 JADSTART 位（只适用于注入转换组）启动，也可以通过外部事件触发启动（适用于常规转换组或注入转换组），并且触发外部事件之前，ADSTART 位或 JADSTART 位必须置 1。

在常规序列中，每次转换完成后：转换数据存储在 32 位 ADCx_DR 寄存器中、EOC（常规转换结束）标志置 1、EOCIE 位置 1 时将产生中断。

在注入序列中，每次转换完成后：转换数据存储在四个 32 位 ADC_JDR1 寄存器的其中一个寄存器中、JEOC（注入转换结束）标志置 1、JEOCIE 位置 1 时将产生中断。

常规序列完成后：EOS（常规序列结束）标志置 1、EOSIE 位置 1 时将产生中断。

注入序列完成后：JEOS（注入序列结束）标志置 1、JEOSIE 位置 1 时将产生中断。

随后，ADC 会停止工作，直至发生新的外部常规或注入触发，或者将 ADSTART 或 JADSTART 位再次置 1。

（2）连续转换模式

通过将 ADC_CFGR 寄存器的 CONT 位置 1 选择连续转换模式。该模式仅适用于常规转换组。在连续转换模式下，如果发生软件或者硬件常规触发事件，ADC 会将常规转换组的所有通道执行一次，随后会自动重启并持续执行序列的每个转换。CONT 位为 1 时，可通过外部触发或将 ADCx_CR 寄存器中的 ADSTART 位置 1 来启动此模式。

在常规序列中，每次转换完成后：转换数据存储在 32 位 ADCx_DR 寄存器中、EOC（转换结束）标志置 1、EOCIE 位置 1 时将产生中断。

转换序列完成后：EOS（序列结束）标志置 1、EOSIE 位置 1 时将产生中断。

随后，会立即重启新序列，ADC 会继续重复执行转换序列。

注意：注入通道不能连续转换，唯一例外的是，在连续转换模式下（使用 JAUTO 位）注入通道配置为在常规通道后的自动转换。

到这里，我们基本上介绍了 ADC 的大多数基础的知识点，其它知识后面用到会继续补充，如果还有不懂的内容，请参考《STM32H7xx 参考手册_V3（中文版）.pdf》的第 25 章。

33.2 单通道 ADC 采集实验

本实验我们来学习单通道 ADC 采集实验。本实验使用常规转换组单通道的单次转换模式，并且通过软件触发，即通过对 ADCx_CR 寄存器的 ADSTART 位写 1 启动转换。下面先带大家来了解本实验要配置的寄存器。

33.2.1 ADC 寄存器

这里，我们只介绍本实验用到的寄存器的关键位，其它寄存器后续用到会继续介绍。

- **ADCx 通用控制寄存器（ADCx_CCR）**

ADCx 通用控制寄存器描述如图 33.2.1.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	VBATEN	VSENSEEN	VREFEN	PRESC[3:0]				CKMODE[1:0]	
							rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DAMDF[1:0]		Res.	Res.	DELAY[3:0]				Res.	Res.	Res.	DUAL[4:0]				
rw	rw			rw	rw	rw	rw				rw	rw	rw	rw	rw

位 21:18 **PRESC[3:0]**：ADC 预分频系数 (ADC prescaler)

这些位由软件置 1 和清零，用于选择 ADC 的时钟频率。该时钟为所有 ADC 所共用。

0000：输入 ADC 时钟未分频
0001：输入 ADC 时钟 2 分频
0010：输入 ADC 时钟 4 分频
0011：输入 ADC 时钟 6 分频
0100：输入 ADC 时钟 8 分频
0101：输入 ADC 时钟 10 分频
0110：输入 ADC 时钟 12 分频
0111：输入 ADC 时钟 16 分频
1000：输入 ADC 时钟 32 分频
1001：输入 ADC 时钟 64 分频
1010：输入 ADC 时钟 128 分频
1011：输入 ADC 时钟 256 分频
其它：保留

注：　仅当 ADC 已禁止时（ADCAL=0、JADSTART=0、ADSTART=0、ADSTP=0、ADDIS=0 且 ADEN=0），才允许通过软件对这些位执行写操作。仅当 CKMODE[1:0] = 0b00 时，才会应用 ADC 预分频值。

图 33.2.1.1 ADCx_CCR 寄存器

该寄存器本章只需要用到 PRESC[3:0]这四个位，用于设置 ADC 时钟的预分频系数（即对 adc_ker_ck 的分频系数），由上图可以得到这四个位域的值的含义可以表示为设置 2^PRESC[3:0]分频。

adc_ker_ck 的时钟源由 RCC_D3CCIPR 寄存器的 ADCSEL[1:0]位配置。本章的实验我们都设置 ADCSEL[1:0]=2，即选择 per_ck 作为时钟源，而 per_ck 又由 RCC_D1CCIPR 寄存器的 CKPERSEL[1:0]位选择，默认为 0，即选择 hsi_ker_ck（64MHz）作为 per_ck。因此：

adc_ker_ck=per_ck=hsi_ker_ck=64MHz。

又由于 ADC 的输入时钟频率不能大于 36M，所以，我们需要设置 PRESC[3:0]=1，即可得到 ADC 输入时钟频率为：

adc_ker_ck/2^PRESC[3:0]=64/2=32MHz。

- **ADCx 控制寄存器（ADCx_CR）**

ADCx 控制寄存器描述如图 33.2.1.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADCAL	ADCALDIF	DEEPPWD	ADVREGEN	LINCALRDYW6	LINCALRDYW5	LINCALRDYW4	LINCALRDYW3	LINCALRDYW2	LINCALRDYW1	Res.	Res.	Res.	Res.	Res.	ADCALLIN
rs	rw	rw	rw	rw	rw	rw	rw	rw	rw						rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	BOOST	Res.	Res.	JADSTP	ADSTP	JADSTART	ADSTART	ADDIS	ADEN
							rw			rs	rs	rs	rs	rs	rs

图 33.2.1.2 ADCx_CR 寄存器

该寄存器我们用到多个位，这里就不全部列出来讲解了，而是抽出几个重要的位进行针对性的介绍，详细的介绍请参考《STM32H7xx 参考手册_V3（中文版）.pdf》第 25.5.3 节，881 页。

ADEN 位用于使能 ADC 转换器。需要设置该位为 1，ADC 才可以正常工作。

ADSTART 位用于启动 ADC 常规通道的转换序列。当使用硬件触发时（EXTEN[1:0]!=0），设置该位为 1，必须在相应的硬件触发事件产生时，才会启动 ADC 转换。而当不使用硬件触发时（EXTEN[1:0]=0），设置该位为 1 则可以立即启动 ADC 转换。

BOOST 位用于设置是否使用 BOOST 模式。当 BOOST=0 时，ADC 输入时钟必须小于

20MHz；当 BOOST=1 时，ADC 输入时钟必须大于 20MHz。因为我们设置的 ADC 输入时钟频率为 32MHz，因此该位必须设置为 1。

ADCALLIN 位用于设置线性 ADC 校准。设置该位为 1，可以设置 ADC 的校准模式为线性校准。

ADCAL 位用与控制/读取 ADC 校准状态。设置该位为 1 时，可以启动 ADC 校准，等校准完成以后，硬件会自动清零该位。因此在设置改位为 1 以后，通过判断该位是否变为 0，即可判断校准是否完成。

- **ADCx 配置寄存器（ADCx_CFGR）**

ADCx 配置寄存器描述如图 33.2.1.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
JQDIS	AWD1CH[4:0]					JAUTO	JAWD1 EN	AWD1 EN	AWD1S GL	JQM	JDISC EN	DISCNUM[2:0]			DISC EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	AUT DLY	CONT	OVR MOD	EXTEN[1:0]		EXTSEL[4:0]					RES[2:0]			DMNGT[1:0]	
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 33.2.1.3 ADCx_CFGR 寄存器

RES[2:0]位用于设置 ADC 转换的分辨率：0 表示 16 位；1 表示 14 位；2 表示 12 位；3 表示 10 位；4 表示 8 位；其它值：保留。本实验使用 16 位分辨率，因此设置 RES[2:0]=0 即可。

EXTEN[1:0]位用于设置常规通道的外部触发方式和极性。本实验使用软件触发，因此设置 EXTEN[1:0]=00 即可，即禁止外部触发。

OVRMOD 位用于设置是否使能覆写功能。当设置该位为 0 时，如果上一次转换的数据未及时读取，新的转换结果将被丢弃；当设置该位为 1 时，如果上一次转换的数据未及时读取，将会被新的结果覆盖。本实验该位设置为 1。

CONT 位用于设置转换模式。当 CONT=0 时，表示单次转换模式；当 CONT=1 时，表示连续转换模式。本实验该位设置为 0。

- **ADCx 配置寄存器 2（ADCx_CFGR2）**

ADCx 配置寄存器 2 描述如图 33.2.1.4 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LSHIFT[3:0]				Res.	Res.	OSR[9:0]									
rw	rw	rw	rw			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	RSHIF T4	RSHIF T3	RSHIF T2	RSHIF T1	ROVSM	TROVS	OVSS[3:0]				Res.	Res.	Res.	JOVSE	ROVSE
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw				rw	rw

图 33.2.1.4 ADCx_CFGR2 寄存器

OSR[9:0]位用于设置 ADC 的过采样率。OSR[9:0]=0~1023，表示 1x~1024x 过采样。本实验不使用过采样，设置 OSR[9:0]=0 即可。

LSHIFT[3:0]位用于设置输出结果的左移位数，0~15 表示左移 0~15 位。本实验使用右对齐，因此设置 LSHIFT[3:0]=0 即可。

- **ADCx 常规序列寄存器 1（ADCx_SQR1）**

ADCx 常规序列寄存器 1 描述如图 33.2.1.5 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	SQ4[4:0]					Res.	SQ3[4:0]					Res.	SQ2[4]
			rw	rw	rw	rw	rw		rw	rw	rw	rw	rw		rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ2[3:0]				Res.	SQ1[4:0]					Res.	Res.	L[3:0]			
rw	rw	rw	rw		rw	rw	rw	rw	rw			rw	rw	rw	rw

图 33.2.1.5 ADCx_SQR1 寄存器

L[3:0]位用于存储常规序列的长度，取值范围：0~15，表示常规序列长度为：1~16。本实验只用到 1 个通道，L[3:0]=0 即可。

SQ1[4:0]~SQ4[4:0]同于设置常规序列中第 1~4 个转换通道，第 5~16 个转换通道的设置请查看 ADCx_SQR2 和 ADCx_SQR4 寄存器。设置过程非常简单，忘记了请参考前面给大家整理出来的常规序列寄存器控制关系汇总表。

- **ADCx 采样时间寄存器 2（ADCx_SMPR2）**

ADCx 采样时间寄存器 2 描述如图 33.2.1.6 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	SMP19[2:0]			SMP18[2:0]			SMP17[2:0]			SMP16[2:0]			SMP15[2:1]	
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMP15[0]	SMP14[2:0]			SMP13[2:0]			SMP12[2:0]			SMP11[2:0]			SMP10[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 29:0 **SMPx[2:0]**：通道 x 采样时间选择 (Channel x sampling time selection)

通过软件写入这些位可分别为各个通道选择采样时间。在采样周期期间，通道选择位必须保持不变。

000：1.5 个 ADC 时钟周期
001：2.5 个 ADC 时钟周期
010：8.5 个 ADC 时钟周期
011：16.5 个 ADC 时钟周期
100：32.5 个 ADC 时钟周期
101：64.5 个 ADC 时钟周期
110：387.5 个 ADC 时钟周期
111：810.5 个 ADC 时钟周期

注：　仅当 ADSTART=0 且 JADSTART=0 时（这可确保当前未进行任何转换），才允许通过软件对这些位执行写操作。

图 33.2.1.6 ADCx_SMPR2 寄存器

该寄存器用于设置 ADC 通道 10~19 的采样时间，而 ADCx_SMPR1 设置 ADC 通道 0~9 的采样时间。STM32H7 的 ADC 总转换时间的计算方法前面已经介绍过了。建议采样时间尽量长一点，以获得较高的准确度，但是这样会降低 ADC 的转换速率，所以大家在实际应用中自行结合自身情况设置。

- **ADCx 通道预选寄存器（ADCx_ PCSEL）**

ADCx 通道预选寄存器描述如图 33.2.1.7 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PCSEL19	PCSEL18	PCSEL17	PCSEL16
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PCSEL 15	PCSEL 14	PCSEL 13	PCSEL 12	PCSEL 11	PCSEL 10	PCSEL9	PCSEL8	PCSEL7	PCSEL6	PCSEL5	PCSEL4	PCSEL3	PCSEL2	PCSEL1	PCSEL0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 19:0 PCSELx：通道 x ($V_{INP[i]}$) 预选 (Channel x (VINP[i]) pre selection)

这些位由软件写入，用于预选 IO 实例中要转换的输入通道。

0：未预选通道 x (Vinp x) 进行转换，该通道的 ADC 转换结果显示的结果不正确。
1：预选输入通道 x (Vinp x) 进行转换

注：　仅当 ADSTART=0 且 JADSTART=0 时（这可确保当前未进行任何转换），才允许通过软件对这些位执行写操作。

图 33.2.1.7 ADCx_ PCSEL 寄存器

该寄存器用于控制 ADC 具体某个输入通道和对应 IO 的连接，相当于在 ADC 输入和 IO 之间，加了一个开关，想要正常使用某个通道，则必须设置对应的 PCSELy 位为 1（y=0~19），否则无法得到对应 IO 口的正常电压。注意：在 STM32H7 之前的其它 STM32 芯片上面，是没有该寄存器的。该寄存器的存在，有利于 ADC 和 IO 的隔离。

举个简单的例子，在 STM32H7 上面，即便是 ADC 通道对应的 IO 口，只要不使用 ADC 功能（PCSEL 不设置为 1），那么该 IO 口就可以兼容 5V，但是在 STM32H7 之前的其它 STM32

芯片上面，ADC 所在的 IO 口，都不能做 5V 兼容。

- **ADCx 常规数据寄存器（ADCx_ DR）**

ADCx 常规数据寄存器描述如图 33.2.1.8 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RDATA[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RDATA[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

位 31:0 **RDATA[31:0]**：已转换常规数据 (Regular Data converted)
这些位为只读。其中包含上一转换常规通道的转换结果。数据有左对齐和右对齐两种方式，如*第 25.3.27 节：数据管理*所述。

图 33.2.1.8 ADCx_ DR 寄存器

常规序列中的 AD 转化结果都将被存在这个寄存器里面，我们读取该寄存器，即可得到 ADC 转换后的结果，而注入通道的转换结果被保存在 ADCx_JDRy（y=1~4）里面。

- **ADCx 中断和状态寄存器（ADCx_ ISR）**

ADCx 中断和状态寄存器描述如图 33.2.1.9 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	JQOVF	AWD3	AWD2	AWD1	JEOS	JEOC	OVR	EOS	EOC	EOSMP	ADRDY
					rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

位 2 **EOC**：转换结束标志 (End of conversion flag)
当通道的每次常规转换结束，新数据出现在 ADCx_DR 寄存器时，会通过硬件将该位置 1。通过软件向该位写入 1，或读取 ADCx_DR 寄存器都可将该位清零。
0：常规通道转换未完成（或标志事件已通过软件确认并清零）
1：常规通道转换已完成

图 33.2.1.9 ADCx_ ISR 寄存器

该寄存器保存了 ADC 转换时的各种状态。本实验我们通过 EOC 位的状态来判断 ADC 转换是否完成，如果查询到 EOC 位被硬件置 1，就可以从 ADC_DR 寄存器中读取转换结果，否则需要等待转换完成。

至此，本章要用到的 ADC 相关寄存器全部介绍完毕了，对于未介绍的部分，请大家参考《STM32H7xx 参考手册_V7（英文版）.pdf》第 25 章相关内容。

33.2.2 硬件设计

1. 例程功能

使用 ADC1 采集通道 19（PA5）上面的电压，在 LCD 模块上面显示 ADC 转换值以及换算成电压后的电压值。使用杜邦线将 P10 的 ADC 和 RV1 连接，使得 PA5 连接到电位器上，然后将 ADC 采集到的数据和转换后的电压值在 TFTLCD 屏中显示。用户可以通过调节电位器的旋钮改变电压值。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0: LED0 - PB1
2）串口 1（PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面）
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块（包括 MCU 屏和 RGB 屏,都支持）

4）ADC1 ：通道 19 - PA5

3. 原理图

ADC 属于 STM32H743 内部资源，实际上我们只需要软件设置就可以正常工作，另外还需要将待测量的电压源连接到 ADC 通道上，以便 ADC 测量。本实验，我们通过 ADC1 的通道 19（PA5）来采集外部电压值，开发板有一个电位器，可调节的电压范围是：0~3.3V。我们可以通过杜邦线将 PA5 与电位器连接，如下图所示：

图 33.2.2.1 PA5 与电位器示意图

使用杜邦线将 P10 的 ADC 和 RV1 连接好后，并下载程序后，就可以用螺丝刀调节电位器变换多种电压值进行测试。

有的朋友可能还想测试其它地方的电压值，我们只需要 1 根杜邦线，一端接到 P10 的 ADC 排针上，另外一端就接你要测试的电压点。**一定要保证测试点的电压在 0~3.3V 的电压范围，否则可能烧坏我们的 ADC，甚至是整个主控芯片。**

33.2.3 程序设计

33.2.3.1 ADC 的 HAL 库驱动

ADC 在 HAL 库中的驱动代码在 stm32h7xx_hal_adc.c 和 stm32h7xx_hal_adc_ex.c 文件（及其头文件）中。

1. HAL_ADC_Init 函数

ADC 的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_ADC_Init(ADC_HandleTypeDef *hadc);
```

- **函数描述：**

  用于初始化 ADC。

- **函数形参：**

  形参 1 是 ADC_HandleTypeDef 结构体类型指针变量，其定义如下：

```
typedef struct
{
  ADC_TypeDef                   *Instance;      /* ADC 寄存器基地址 */
  ADC_InitTypeDef                Init;          /* ADC 参数初始化结构体变量 */
  DMA_HandleTypeDef             *DMA_Handle;    /* DMA 配置结构体 */
  HAL_LockTypeDef                Lock;          /* ADC 锁定对象 */
  __IO uint32_t                  State;         /* ADC 工作状态 */
  __IO uint32_t                  ErrorCode;     /* ADC 错误代码 */
  ADC_InjectionConfigTypeDef InjectionConfig ; /* ADC 注入通道配置结构，用于配置注入
                                                  通道的转换顺序，数据格式等 */
}ADC_HandleTypeDef;
```

该结构体定义和其他外设比较类似，我们着重看第二个成员变量 Init 含义，它是结构体 ADC_InitTypeDef 类型，结构体 ADC_InitTypeDef 定义为：

```
typedef struct {
 uint32_t ClockPrescaler;                /* 设置预分频系数，即 PRESC[3:0]位 */
 uint32_t Resolution;                    /* 配置 ADC 的分辨率 */
 uint32_t ScanConvMode;                  /* 扫描模式 */
 uint32_t EOCSelection;                  /* 转换完成标志位 */
```

```
  FunctionalState LowPowerAutoWait;          /* 低功耗自动延时 */
  FunctionalState ContinuousConvMode;        /* 设置单次转换模式还是连续转换模式 */
  uint32_t NbrOfConversion;                  /* 设置转换通道数目，赋值范围是 1~16 */
  FunctionalState DiscontinuousConvMode;     /* 设置常规转换组不连续模式 */
  uint32_t NbrOfDiscConversion;              /* 常规转换组不连续模式转换通道的数目 */
  uint32_t ExternalTrigConv;                 /* ADC 外部触发源选择*/
  uint32_t ExternalTrigConvEdge;             /* ADC 外部触发极性*/
  uint32_t ConversionDataManagement;         /* 数据管理 */
  uint32_t Overrun;                          /* 发生溢出时，进行的操作 */
  uint32_t LeftBitShift;                     /* 数据左移几位 */
  FunctionalState OversamplingMode;          /* 过采样模式 */
  ADC_OversamplingTypeDef Oversampling;      /* 过采样的参数配置 */
} ADC_InitTypeDef;
```

1) ClockPrescaler：ADC 预分频系数选择，可选的分频系数为 1、2、4、6、8、10、12、16、32、64、128、256。ADC 最大时钟配置为 36MHZ。
2) Resolution：配置 ADC 的分辨率，可选的分辨率有 16 位、12 位、10 位和 8 位。分辨率越高，转换数据精度越高，转换时间也越长；反之分辨率越低，转换数据精度越低，转换时间也越短。
3) ScanConvMode：配置是否使用扫描。如果是单通道转换使用 ADC_SCAN_DISABLE，如果是多通道转换使用 ADC_SCAN_ENABLE。
4) EOCSelection：可选参数为 ADC_EOC_SINGLE_CONV 和 ADC_EOC_SEQ_CONV，指定转换结束时是否产生 EOS 中断或事件标志。
5) LowPowerAutoWait：配置是否使用低功耗自动延迟等待模式，可选参数为 ENABLE 和 DISABLE，当使能时，仅当一组内所有之前的数据已处理完毕时，才开始新的转换，适用于低频应用。该模式仅用于 ADC 的轮询模式，不可用于 DMA 以及中断。
6) ContinuousConvMode：可选参数为 ENABLE 和 DISABLE，配置自动连续转换还是单次转换。使用 ENABLE 配置为使能自动连续转换；使用 DISABLE 配置为单次转换，转换一次后停止需要手动控制才重新启动转换。
7) NbrOfConversion：设置常规转换通道数目，范围是：1~16。
8) DiscontinuousConvMode：配置是否使用不连续的采样模式，比如要转换的通道有 1、2、5、7、8、9，那么第一次触发会进行通道 1 与通道 2，下次触发就是转换通道 5 与通道 7，这样不连续的转换，依次类推。此参数只有将 ScanConvMode 使能，还有 ContinuousConvMode 失能的情况下才有效，不可同时使能。
9) NbrOfDiscConversion：不连续采样通道数。
10) ExternalTrigConv：外部触发方式的选择，如果使用软件触发，那么外部触发会关闭。
11) ExternalTrigConvEdge：外部触发极性选择，如果使用外部触发，可以选择触发的极性，可选有禁止触发检测、上升沿触发检测、下降沿触发检测以及上升沿和下降沿均可触发检测。
12) ConversionDataManagement： 指定 ADC 转换后的数据处理方式。可以选择 DMA 管理传输数据、数据存储在数据寄存器中或者是传输到 DFSDM 寄存器中。
13) Overrun：当有新的数据溢出时，可以选择覆盖写入或者是丢弃新的数据。
14) LeftBitShift：数据左移位数，最多可支持左移 15 位。
15) OversamplingMode：是否使用过采样模式。
16) Oversampling：配置过采样模式的参数。

- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

2. HAL_ADCEx_Calibration_Start 函数

ADC 的自校准函数，其声明如下：

```
HAL_StatusTypeDef HAL_ADCEx_Calibration_Start(ADC_HandleTypeDef *hadc,
                                              uint32_t CalibrationMode, uint32_t SingleDiff);
```

- **函数描述：**
  首先调用 HAL_ADC_Init 函数配置了相关的功能后，再调用此函数进行 ADC 自校准功能。
- **函数形参：**

形参 1 是 ADC_HandleTypeDef 结构体类型指针变量。

形参 2 是校准模式选择，有以下两种：

1）ADC_CALIB_OFFSET 表示只运行偏移校准而不运行线性度校准。

2）ADC_CALIB_OFFSET_LINEARITY 表示同时运行偏移校准和线性度校准。

形参 3 是单端或差分模式选择，有以下两种：

1）ADC_SINGLE_ENDED 表示单端输入模式。

2）ADC_DIFFERENTIAL_ENDED 表示差分输入模式。

- **函数返回值：**

HAL_StatusTypeDef 枚举类型的值。

3. HAL_ADC_ConfigChannel 函数

ADC 通道配置函数，其声明如下：

```
HAL_StatusTypeDef HAL_ADC_ConfigChannel(ADC_HandleTypeDef *hadc,
                                        ADC_ChannelConfTypeDef *sConfig);
```

- **函数描述：**

调用了 HAL_ADC_Init 函数配置了相关的功能后，就可以调用此函数配置 ADC 具体通道。

- **函数形参：**

形参 1 是 ADC_HandleTypeDef 结构体类型指针变量。

形参 2 是 ADC_ChannelConfTypeDef 结构体类型指针变量，用于配置 ADC 采样时间，使用的通道号，单端或者差分方式的配置等。该结构体定义如下：

```
typedef struct {
  uint32_t Channel;                         /* ADC 转换通道*/
  uint32_t Rank;                            /* ADC 转换顺序 */
  uint32_t SamplingTime;                    /* ADC 采样周期 */
  uint32_t SingleDiff;                      /* 输入信号线的类型*/
  uint32_t OffsetNumber;                    /* 采用偏移量的通道 */
  uint32_t Offset;                          /* 偏移量 */
  FunctionalState OffsetRightShift;         /* 数据右移位数*/
  FunctionalState OffsetSignedSaturation;   /* 转换数据格式为有符号位数据 */
} ADC_ChannelConfTypeDef;
```

1) Channel：ADC 转换通道，范围：0~19。

2) Rank：在常规转换中的常规组的转换顺序，可以选择 1~16。

3) SamplingTime：ADC 的采样周期，最大 810.5 个 ADC 时钟周期，要求尽量大以减少误差。

4) SingleDiff：选择通道单端输入还是差分输入。

5) OffsetNumber：选择使用偏移量的通道。

6) Offset：定义要从原始数据减去的偏移量。根据 ADC 的分辨率不同，支持的最大偏移量也不同，例如分辨率是 16bit，，最大的偏移量为 0xFFFF。

7) OffsetRightShift：采样值进行右移的位数。

8) OffsetSignedSaturation：是否使能 ADC 采样值的最高位为符号位。

- **函数返回值：**

HAL_StatusTypeDef 枚举类型的值。

4. HAL_ADC_Start 函数

ADC 转换启动函数，其声明如下：

```
HAL_StatusTypeDef HAL_ADC_Start(ADC_HandleTypeDef *hadc);
```

- **函数描述：**

当配置好 ADC 的基础的功能后，就调用此函数启动 ADC。

- **函数形参：**

形参 1 是 ADC_HandleTypeDef 结构体类型指针变量。

- **函数返回值：**

HAL_StatusTypeDef 枚举类型的值。

5. HAL_ADC_PollForConversion 函数

等待 ADC 常规组转换完成函数，其声明如下：

```
HAL_StatusTypeDef HAL_ADC_PollForConversion(ADC_HandleTypeDef *hadc,
                                            uint32_t Timeout);
```

- **函数描述：**

一般先调用 HAL_ADC_Start 函数启动转换，再调用该函数等待转换完成，然后再调用 HAL_ADC_GetValue 函数来获取当前的转换值。

- **函数形参：**

形参 1 是 ADC_HandleTypeDef 结构体类型指针变量。

形参 2 是等待转换的等待时间，单位是毫秒（ms）。

- **函数返回值：**

HAL_StatusTypeDef 枚举类型的值。

6. HAL_ADC_GetValue 函数

获取常规组 ADC 转换值函数，其声明如下：

```
uint32_t HAL_ADC_GetValue(ADC_HandleTypeDef *hadc);
```

- **函数描述：**

一般先调用 HAL_ADC_Start 函数启动转换，再调用 HAL_ADC_PollForConversion 函数等待转换完成，然后再调用 HAL_ADC_GetValue 函数来获取当前的转换值。

- **函数形参：**ADC_HandleTypeDef 结构体类型指针变量。
- **函数返回值：**当前的转换值，uint32_t 类型数据。

单通道 ADC 采集配置步骤

1）开启 ADCx 和 ADC 通道对应的 IO 时钟，并配置该 IO 为模拟功能

首先开启 ADCx 的时钟，然后配置 GPIO 为模拟模式。本实验我们默认用到 ADC1 通道 19，对应 IO 是 PA5，它们的时钟开启方法如下：

```
__HAL_RCC_ADC12_CLK_ENABLE ();              /* 使能 ADC1 & ADC2 时钟 */
__HAL_RCC_GPIOA_CLK_ENABLE();               /* 开启 GPIOA 时钟 */
```

2）初始化 ADCx，配置其工作参数

通过 HAL_ADC_Init 函数来设置 ADCx 时钟分频系数、分辨率、模式、扫描方式、对齐方式等信息。

注意：该函数会调用：HAL_ADC_MspInit 回调函数来存放 ADC 及 GPIO 时钟使能、GPIO 初始化等代码。

3）配置 ADC 通道并启动 AD 转换器

在 HAL 库中，通过 HAL_ADC_ConfigChannel 函数来设置配置 ADC 的通道，根据需求设置通道、序列、采样时间和校准配置单端输入模式或差分输入模式等。

配置好 ADC 通道之后，通过 HAL_ADC_Start 函数启动 AD 转换器。

4）读取 ADC 值

这里选择查询方式读取，在读取 ADC 值之前需要调用 HAL_ADC_PollForConversion 等待上一次转换结束。然后就可以通过 HAL_ADC_GetValue 来读取 ADC 值。

33.2.3.2 程序流程图

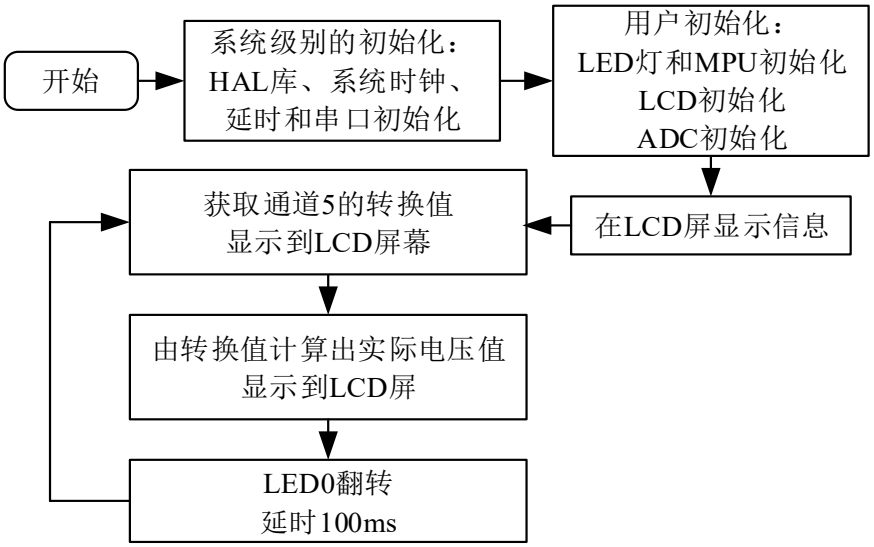


图 33.2.3.2.1 单通道 ADC 采集实验程序流程图

33.2.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。ADC 驱动源码包括两个文件：adc.c 和 adc.h。

adc.h 文件针对 ADC 及通道引脚定义了一些宏定义，具体如下：

```
/* ADC 及引脚 定义 */
#define ADC_ADCX_CHY_GPIO_PORT              GPIOA
#define ADC_ADCX_CHY_GPIO_PIN               GPIO_PIN_5
#define ADC_ADCX_CHY_GPIO_CLK_ENABLE()
        do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)    /* PA 口时钟使能 */

#define ADC_ADCX                            ADC1
#define ADC_ADCX_CHY                        ADC_CHANNEL_19    /* 通道 Y,  0 <= Y <= 19 */
#define ADC_ADCX_CHY_CLK_ENABLE()
        do{ __HAL_RCC_ADC12_CLK_ENABLE(); }while(0)    /* ADC1 & ADC2 时钟使能 */
```

ADC 的通道与引脚的对应关系在 《STM32H743IIT6.pdf》 数据手册可以查到，我们这里使用 ADC1 的通道 19，在数据手册中的表格为：

PA5	I/O	TT_ha	-	D2PWREN, TIM2_CH1/TIM2_ETR, TIM8_CH1N, SPI1_SCK/I2S1_CK, SPI6_SCK, OTG_HS_ULPI_CK, LCD_R4, EVENTOUT	ADC12_INN18, ADC12_INP19, DAC1_OUT2

表 33.2.3.3.1 ADC1 通道 5 对应引脚查看表

下面直接开始介绍 adc.c 的程序，首先是 ADC 初始化函数。

```
/**
 * @brief    ADC 初始化函数
 * @note     本函数支持 ADC1/ADC2 任意通道,但是不支持 ADC3
 *           我们使用 16 位精度, ADC 采样时钟=32M, 转换时间为:采样周期 + 8.5 个 ADC 周期
 *           设置最大采样周期: 810.5, 则转换时间 = 819 个 ADC 周期 = 25.6us
 * @param    无
 * @retval   无
 */
void adc_init(void)
```

```
{
    g_adc_handle.Instance = ADC_ADCX;
    g_adc_handle.Init.ClockPrescaler = ADC_CLOCK_SYNC_PCLK_DIV2;
    /* 2 分频，ADCCLK=PER_CK/2=64/2=32MHZ */
    g_adc_handle.Init.Resolution = ADC_RESOLUTION_16B;    /* 16 位模式 */
    g_adc_handle.Init.ScanConvMode = DISABLE;             /* 非扫描模式 */
    g_adc_handle.Init.EOCSelection = ADC_EOC_SINGLE_CONV;/* 关闭 EOC 中断 */
    g_adc_handle.Init.LowPowerAutoWait = DISABLE;         /* 自动低功耗关闭 */
    g_adc_handle.Init.ContinuousConvMode = DISABLE;       /* 关闭连续转换 */
    g_adc_handle.Init.NbrOfConversion = 1;
    /* 1 个转换在规则序列中 也就是只转换规则序列 1 */
    g_adc_handle.Init.DiscontinuousConvMode = DISABLE;  /* 禁止不连续采样模式 */
    g_adc_handle.Init.NbrOfDiscConversion = 0;          /* 不连续采样通道数为 0 */
    g_adc_handle.Init.ExternalTrigConv = ADC_SOFTWARE_START; /* 软件触发 */
    g_adc_handle.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
    /* 使用软件触发 */
    g_adc_handle.Init.Overrun = ADC_OVR_DATA_OVERWRITTEN;
    /* 有新的数据的死后直接覆盖掉旧数据 */
    g_adc_handle.Init.OversamplingMode = DISABLE;       /* 过采样关闭 */
    g_adc_handle.Init.ConversionDataManagement = ADC_CONVERSIONDATA_DR;
    /* 规则通道的数据仅仅保存在 DR 寄存器里面 */
    HAL_ADC_Init(&g_adc_handle);          /* 初始化  */

    HAL_ADCEx_Calibration_Start(&g_adc_handle, ADC_CALIB_OFFSET,
                                ADC_SINGLE_ENDED); /* ADC 校准 */
}
```

该函数主要调用了两个 HAL 库函数，HAL_ADC_Init 函数配置了选择哪个 ADC、数据对齐方式、是否使用扫描模式等参数，HAL_ADCEx_Calibration_Start 函数用于校准 ADC。另外 HAL_ADC_Init 函数会调用它的 MSP 回调函数 HAL_ADC_MspInit，该函数用来存放使能 ADC 和通道对应 IO 的时钟和初始化 IO 口等代码，其定义如下：

```
/**
 * @brief       ADC 底层驱动，引脚配置，时钟使能
 * @note        此函数会被 HAL_ADC_Init()调用
 * @param       hadc:ADC 句柄
 * @retval      无
 */
void HAL_ADC_MspInit(ADC_HandleTypeDef *hadc)
{
    GPIO_InitTypeDef gpio_init_struct;

    ADC_ADCX_CHY_CLK_ENABLE();                  /* 使能 ADC1/2 时钟 */
    ADC_ADCX_CHY_GPIO_CLK_ENABLE();             /* 开启 ADC 通道 IO 引脚时钟 */
    __HAL_RCC_ADC_CONFIG(RCC_ADCCLKSOURCE_CLKP); /* ADC 外设时钟选择 */

    gpio_init_struct.Pin = ADC_ADCX_CHY_GPIO_PIN;/* ADC 通道 IO 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_ANALOG;    /* 模拟 */
    gpio_init_struct.Pull = GPIO_NOPULL;         /* 不带上下拉 */
    HAL_GPIO_Init(ADC_ADCX_CHY_GPIO_PORT, &gpio_init_struct);
}
```

可以看到在 HAL_ADC_MspInit 函数中，我们除了使能 ADC 和通道对应 IO 时钟、初始化 IO 外，还配置了 ADC 的时钟源来自于 per_ck。

接下来要介绍的是 adc_get_result 函数，其定义如下：

```
/**
 * @brief       获得 ADC 转换后的结果
 * @param       ch: 通道值 0~19，取值范围为：ADC_CHANNEL_0~ADC_CHANNEL_19
 * @retval      返回值:转换结果
 */
uint32_t adc_get_result(uint32_t ch)
{
    ADC_ChannelConfTypeDef adc_ch_conf;
```

```
    adc_ch_conf.Channel = ch;                                   /* 通道 */
    adc_ch_conf.Rank = ADC_REGULAR_RANK_1;                      /* 1 个序列 */
    adc_ch_conf.SamplingTime = ADC_SAMPLETIME_810CYCLES_5;/* 采样时间*/
    adc_ch_conf.SingleDiff = ADC_SINGLE_ENDED;                  /* 单边采集 */
    adc_ch_conf.OffsetNumber = ADC_OFFSET_NONE;                 /* 不使用偏移量的通道 */
    adc_ch_conf.Offset = 0;                                     /* 偏移量为 0 */
    HAL_ADC_ConfigChannel(&g_adc_handle ,& adc_ch_conf);        /* 通道配置 */

    HAL_ADC_Start(&g_adc_handle);                               /* 开启 ADC */
    HAL_ADC_PollForConversion(&g_adc_handle, 10);               /* 轮询转换 */
    return HAL_ADC_GetValue(&g_adc_handle);  /* 返回最近一次 ADC1 常规组的转换结果 */
}
```

该函数先是调用 HAL_ADC_ConfigChannel 函数选择 ADC 通道、设置转换序列号和采样时间等，接着调用 HAL_ADC_Start 启动转换，然后调用 HAL_ADC_PollForConversion 函数等待转换完成，最后调用 HAL_ADC_GetValue 函数获取转换结果。

下面介绍的是获取 ADC 某通道的转换多次后的平均值函数，函数定义如下：

```
/**
 * @brief       获取通道 ch 的转换值，取 times 次,然后平均
 * @param       ch      : 通道号, 0~19
 * @param       times   : 获取次数
 * @retval      通道 ch 的 times 次转换结果平均值
 */
uint32_t adc_get_result_average(uint32_t ch, uint8_t times)
{
    uint32_t temp_val = 0;
    uint8_t t;
    for (t = 0; t < times; t++) /* 获取 times 次数据 */
    {
        temp_val += adc_get_result(ch);
        delay_ms(5);
    }
    return temp_val / times;     /* 返回平均值 */
}
```

该函数用于多次获取 ADC 值，取平均，用来提高准确度。

最后在 main 函数里面编写如下代码：

```
int main(void)
{
    uint16_t adcx;
    float temp;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    adc_init();                             /* 初始化 ADC */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "ADC TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 130, 200, 16, 16, "ADC1_CH19_VAL:", BLUE);
    /* 先在固定位置显示小数点 */
    lcd_show_string(30, 150, 200, 16, 16, "ADC1_CH19_VOL:0.000V", BLUE);
    while (1)
```

```
    {
        /* 获取通道 19 的转换值，20 次取平均 */
        adcx = adc_get_result_average(ADC_ADCX_CHY, 20);
        lcd_show_xnum(142, 130, adcx, 5,16,0,BLUE);/* 显示 ADC 采样后的原始值 */
        /* 获取计算后的带小数的实际电压值，比如 3.1111 */
        temp = (float)adcx * (3.3 / 65536);
        adcx = temp;            /* 赋值整数部分给 adcx 变量，因为 adcx 为 u16 整形 */
        /* 显示电压值的整数部分，3.1111 的话，这里就是显示 3 */
        lcd_show_xnum(142, 150, adcx, 1, 16, 0, BLUE);

        temp -= adcx;/* 把已经显示的整数部分去掉，留下小数部分，比如 3.1111-3=0.1111 */
        temp *= 1000;/*小数部分乘以 1000，如：0.1111 就转换为 111.1，相当于保留三位小数*/
        /* 显示小数部分（前面转换为了整形显示），这里显示的就是 111. */
        lcd_show_xnum(158, 150, temp, 3, 16, 0X80, BLUE);

        LED0_TOGGLE();
        delay_ms(250);
    }
}
```

此部分代码，我们在 TFTLCD 模块上显示一些提示信息后，将每隔 100ms 读取一次 ADC 通道 19 的值，并显示读到的 ADC 值（数字量），以及其转换成模拟量后的电压值。同时控制 LED0 闪烁，以提示程序正在运行。ADC 值的显示简单介绍一下：首先我们在液晶固定位置显示了小数点，然后后面计算步骤中，先计算出整数部分在小数点前面显示，然后计算出小数部分，在小数点后面显示。这样就在液晶上面显示转换结果的整数和小数部分。

33.2.4 下载验证

下载代码后，可以看到 LCD 显示如图 33.2.4.1 所示：

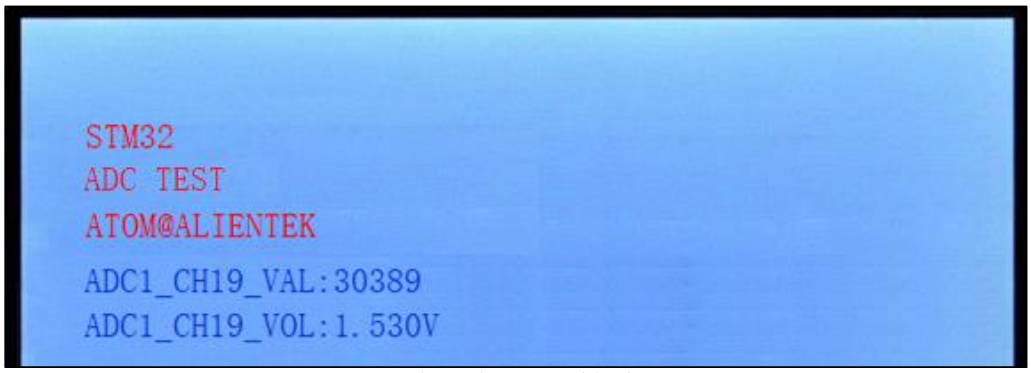


图 33.2.4.1 单通道 ADC 采集实验测试图

上图中，我们使用杜邦线将 P10 的 ADC 和 RV1 连接，使得 PA5 连接到电位器上，测试的是电位器的电压，并可以通过螺丝刀调节电位器改变电压值，范围：0~3.3V。LED0 闪烁，提示程序运行。

大家也可以用杜邦线将 ADC 排针接到其它待测量的电压点，看看测量到的电压值是否准确？**但是要注意：一定要保证测试点的电压在 0~3.3V 的电压范围，否则可能烧坏我们的 ADC，甚至是整个主控芯片。**

33.3 单通道 ADC 采集（DMA 读取）实验

本实验我们来学习单通道 ADC 采集（DMA 读取）实验。本实验使用常规转换组单通道的连续转换模式，并且通过软件触发，即通过对 ADCx_CR 寄存器的 ADSTART 位写 1 启动转换。由于使用连续转换模式，所以使用 DMA 读取转换结果的方式。下面先带大家来了解本实验要配置的寄存器。

33.3.1 ADC & DMA 寄存器

本实验我们很多的设置和单通道 ADC 采集实验是一样的，所以下面介绍寄存器的时候我们不会继续全部都介绍，而是针对性选择与单通道 ADC 采集实验不同设置的 ADCx_CFGR 寄存器进行介绍，其他的配置基本一样的。另外因为我们用到 DMA 读取数据，所以还会介绍如何配置相关 DMA 的寄存器。

- **ADCx 配置寄存器（ADCx_CFGR）**

ADCx 配置寄存器描述如图 33.3.1.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
JQDIS	AWD1CH[4:0]					JAUTO	JAWD1 EN	AWD1 EN	AWD1S GL	JQM	JDISC EN	DISCNUM[2:0]			DISC EN
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	AUT DLY	CONT	OVR MOD	EXTEN[1:0]		EXTSEL[4:0]					RES[2:0]			DMNGT[1:0]	
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 13 **CONT**：常规转换的单次/连续转换模式 (Single / continuous conversion mode for regular conversions)
此位由软件置 1 和清零。该位置 1 时，常规转换将持续进行，直到该位清零。
0：单次转换模式
1：连续转换模式
注：　不能同时使能不连续模式和连续模式：禁止同时将 DISCEN 和 CONT 位置 1。
仅当 ADSTART=0 时（这可确保当前未进行任何常规转换），才允许通过软件对此位执行写操作。
如果使能了双重模式（ADCx_CCR 寄存器的 DAMDF 位不等于零），则从 ADC 的 CONT 位不再可写，其内容与主 ADC 的 CONT 位相等。

位 1:0 **DMNGT[1:0]**：数据管理配置 (Data Management configuration)
此位由软件置 1 和清零，用于选择 ADC 接口输出数据的管理方式。
00：常规转换数据仅存储在 DR 中
01：选择 DMA 单次模式
10：选择 DFSDM 模式。
11：选择 DMA 循环模式
注：　仅当 ADSTART=0 且 JADSTART=0 时（这可确保当前未进行任何转换），才允许通过软件对此位执行写操作。
在双重 ADC 模式下，此位无意义，会由 ADCx_CCR 寄存器的控制位 DAMDF 代替。

图 33.3.1.1 ADCx_CFGR 寄存器

ADCx_CFGR 寄存器中我们主要跟前面设置不同的有两个位，分别如下：

CONT 位用于设置单次转换模式还是连续转换模式，本实验我们使用连续转换模式，所以 CONT 位置 1 即可。

DMNGT[1:0]位用于数据管理配置。单通道 ADC 采集实验我们是默认设置为 00：常规转换数据仅存储在 DR 中，然后通过软件去 DR 数据寄存器读取。本实验我们要设置为 01：选择 DMA 单次模式，这样启动一次 DMA 传输，DMA 就会自动读取一次数据。

这里介绍 ADCx_CFGR 寄存器的这两个位，其它请参考上一个实验的配置。下面介绍 DMA 一些比较重要的寄存器配置。

- **DMA 数据流 x 外设地址寄存器（DMA_SxPAR）**

DMA 数据流 x 外设地址寄存器描述如图 33.3.1.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PAR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PAR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:0 **PAR[31:0]**：外设地址 (Peripheral address)
读/写数据的外设数据寄存器的基址。
这些位受到写保护，只有 DMA_SxCR 寄存器中的 EN 为“0”时才可以写入。

图 33.3.1.2 DMA_SxPAR 寄存器

该寄存器用于存放数据流 x 的外设地址。本实验，我们需要通过 DMA 读取 ADC1 转换后存放在 ADC1 常规数据寄存器 (ADC1_DR) 的结果数据。所以我们需要给 DMA_SxPAR 寄存器写入 ADC1_DR 寄存器的地址。这样配置后，DMA 就会从 ADC1_DR 寄存器的地址读取 ADC 的转换后的数据到某个内存空间。这个内存空间地址需要我们通过 DMA_SxM0AR 寄存器来设置，比如定义一个变量，把这个变量的地址值写入该寄存器。

注意：DMA_SxPAR 寄存器受到写保护，只有 DMA_SxCR 寄存器中的 EN 为“0”时才可以写入，即先要禁止数据流传输才可以写入。

- **DMA 数据流 x 存储器 0 地址寄存器（DMA_SxM0AR）**

DMA 数据流 x 存储器 0 地址寄存器描述如图 33.3.1.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
M0A[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M0A[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:0 **M0A[31:0]**：存储器 0 地址 (Memory 0 address)
读/写数据的存储区 0 的基址。
这些位受到写保护，只有在以下情况下才可以写入：
– 禁止数据流（DMA_SxCR 寄存器中的位 EN=“0”）或
– 使能数据流（DMA_SxCR 寄存器中的 EN=“1”）并且 DMA_SxCR 寄存器中的位 CT =“1”（在双缓冲区模式下）。

图 33.3.1.3 DMA_SxM0AR 寄存器

该寄存器存放的是 DMA 数据流 x 存储器 0 地址。如果用到双缓冲区模式我们还需要用到 DMA_SxM1AR 寄存器，本实验我们是用不到的。

- **DMA 数据流 x 数据项数寄存器（DMA_SxNDTR）**

DMA 数据流 x 数据项数寄存器描述如图 33.3.1.4 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NDT[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 15:0 **NDT[15:0]**：要传输的数据项数目 (Number of data items to transfer)
要传输的数据项数目（0 到 65535）。只有在禁止数据流时，才能向此寄存器执行写操作。使能数据流后，此寄存器为只读，用于指示要传输的剩余数据项数。每次 DMA 传输后，此寄存器将递减。
传输完成后，此寄存器保持为零（数据流处于正常模式时），或者在以下情况下自动以先前编程的值重载：
– 以循环模式配置数据流时
– 通过将 EN 位置“1”来重新使能数据流时
如果该寄存器的值为零，则即使使能数据流，也无法完成任何事务。

图 33.3.1.4 DMA_SxNDTR

该寄存器控制 DMA 数据流 x 的每次传输所要传输的数据量。其设置范围为 0~65535。并且该寄存器的值会随着传输的进行而减少，当该寄存器的值为 0 的时候就代表此次数据传输已经全部发送完成了。所以可以通过这个寄存器的值来知道当前 DMA 传输的进度。特别注意，这里是数据项数目，而不是指的字节数。比如设置数据位宽为 16 位，那么传输一次（一个项）就是 2 个字节。

其它的 DMA 寄存器我们就不一一介绍了，请大家看着寄存器源码对照手册理解，都不难。

33.3.2 硬件设计

1. 例程功能

使用 ADC1 采集（DMA 读取）通道 19（PA5）上面的电压，在 LCD 模块上面显示 ADC 转换值以及换算成电压后的电压值。使用杜邦线将 P10 的 ADC 和 RV1 连接，使得 PA5 连接到电位器上，然后将 ADC 采集到的数据和转换后的电压值在 TFTLCD 屏中显示。用户可以通过调节电位器的旋钮改变电压值。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0: LED0 - PB1
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）ADC1 ： 通道 19 - PA5
5） DMA(DMA1 数据流 7 DMA 请求源 9)

3. 原理图

ADC 和 DMA 属于 STM32H743 内部资源，实际上我们只需要软件设置就可以正常工作，另外还需要将待测量的电压源连接到 ADC 通道上，以便 ADC 测量。本实验，我们通过 ADC1 的通道 19（PA5）来采集外部电压值，并通过 DMA 来读取，开发板有一个电位器，可调节的电压范围是：0~3.3V。我们可以通过杜邦线将 PA5 与电位器连接，如下图所示：

图 33.3.2.1 连接实物图

使用杜邦线将 P10 的 ADC 和 RV1 连接好后，并下载程序后，就可以用螺丝刀调节电位器变换多种电压值进行测试。

有的朋友可能还想测试其他地方的电压值，我们只需要 1 根杜邦线，一端接到 P10 的 ADC 排针上，另外一端就接你要测试的电压点。**一定要保证测试点的电压在 0~3.3V 的电压范围，否则可能烧坏我们的 ADC，甚至是整个主控芯片。**

33.3.3 程序设计

33.3.3.1 ADC & DMA 的 HAL 库驱动

单通道 ADC 采集实验已经介绍本实验要用到的 ADC 的 HAL 库 API 函数，这里要介绍的是 HAL_DMA_Start_IT 和 HAL_ADC_Start_DMA 函数。

1. HAL_DMA_Start_IT 函数

启动使用中断的 DMA 传输函数，其声明如下：

```
HAL_StatusTypeDef HAL_DMA_Start_IT(DMA_HandleTypeDef *hdma,
                uint32_t SrcAddress, uint32_t DstAddress, uint32_t DataLength);
```

- **函数描述：**
  用于启动使用中断的 DMA 传输，DMA1，DMA2 和 BDMA 都是用的这个函数。
- **函数形参：**
  形参 1 是 DMA_HandleTypeDef 结构体类型指针变量。
  形参 2 是 DMA 传输的源地址。
  形参 3 是 DMA 传输的目的地址。
  形参 4 是要传输的数据项数目。
- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

2. HAL_ADC_Start_DMA 函数

启动 ADC（DMA 传输）方式函数，其声明如下：

```
HAL_StatusTypeDef HAL_ADC_Start_DMA(ADC_HandleTypeDef *hadc,
                                    uint32_t *pData, uint32_t Length);
```

- **函数描述：**
  用于启动 ADC（DMA 传输方式）函数。
- **函数形参：**
  形参 1 是 ADC_HandleTypeDef 结构体类型指针变量。
  形参 2 是 ADC 采样数据传输的目的地址。
  形参 3 是要传输的数据项数目。
- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

单通道 ADC 采集（DMA 读取）配置步骤

1）开启 ADCx 和 ADC 通道对应的 IO 时钟，并配置该 IO 为模拟功能

首先开启 ADCx 的时钟，然后配置 GPIO 为模拟模式。本实验我们默认用到 ADC1 通道 19，对应 IO 是 PA5，它们的时钟开启方法如下：

```
__HAL_RCC_ADC12_CLK_ENABLE ();              /* 使能 ADC1 & ADC2 时钟 */
__HAL_RCC_GPIOA_CLK_ENABLE();               /* 开启 GPIOA 时钟 */
```

2）初始化 ADCx，配置其工作参数

通过 HAL_ADC_Init 函数来设置 ADCx 时钟分频系数、分辨率、模式、扫描方式、对齐方式等信息。

注意：该函数会调用：HAL_ADC_MspInit 回调函数来存放 ADC 及 GPIO 时钟使能、GPIO 初始化等代码。我们也可以不存放在这个函数里，本实验就没用到这个 MSP 回调函数。

3）配置 ADC 通道

在 HAL 库中，通过 HAL_ADC_ConfigChannel 函数来设置配置 ADC 的通道，根据需求设置通道、序列、采样时间和校准配置单端输入模式或差分输入模式等。

4）初始化 DMA

通过 HAL_DMA_Init 函数初始化 DMA，包括配置通道，外设地址，存储器地址，传输数据量等。

HAL 库为了处理各类外设的 DMA 请求，在调用相关函数之前，需要调用一个宏定义标识符，来连接 DMA 和外设句柄。这个宏定义为__HAL_LINKDMA。

5）使能 DMA 对应数据流中断，配置 DMA 中断优先级并开启中断，启动 ADC 和 DMA

通过 HAL_ADC_Start_DMA 函数开启 ADC 转换，通过 DMA 传输结果。
通过 HAL_DMA_Start_IT 函数启动 DMA 读取，使能 DMA 中断。
通过 HAL_NVIC_EnableIRQ 函数使能 DMA 数据流中断。
通过 HAL_NVIC_SetPriority 函数设置中断优先级。

6）编写中断服务函数

DMA 中断对于每个数据流几乎都有一个中断服务函数，比如 DMA1_Stream7 的中断服务函数为 DMA1_Stream7_IRQHandler。简单的做法就是在，对应的中断服务函数里面，通过判断相关的中断标志位的方式，完成中断逻辑代码，最后清楚该中断标志位，本实验的做法就是如此。

还可以通过调用 HAL 库提供的 DMA 中断公用处理函数 HAL_DMA_IRQHandler，然后重新定义相关的中断回调处理函数。

33.3.3.2 程序流程图

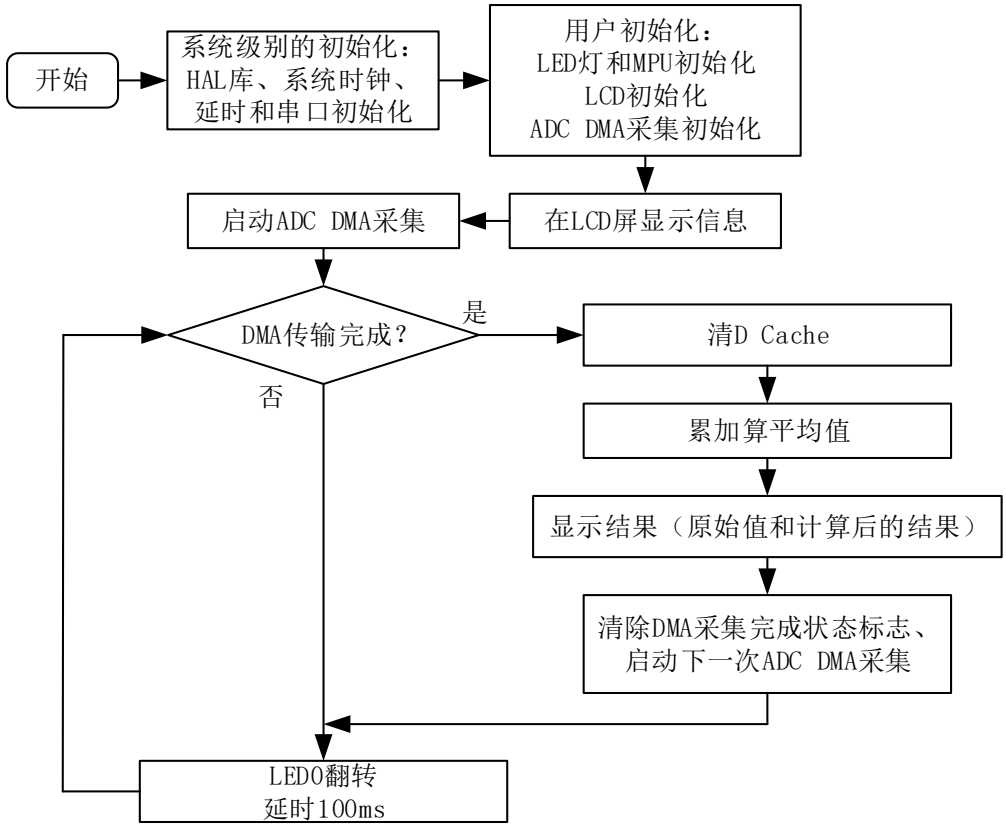


图 33.3.3.2.1 单通道 ADC 采集（DMA 读取）实验程序流程图

33.3.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。ADC 驱动源码包括两个文件：adc.c 和 adc.h。

由于本实验用到 DMA，所以在 adc.h 头文件定义了以下一些宏定义：

```
/* ADC 单通道/多通道 DMA 采集 DMA 数据流相关 定义
 * 注意：这里我们的通道还是使用上面的定义 */
#define ADC_ADCX_DMASx                  DMA1_Stream7
#define ADC_ADCX_DMASx_REQ              DMA_REQUEST_ADC1     /* ADC1_DMA 请求源 */
#define ADC_ADCX_DMASx_IRQn             DMA1_Stream7_IRQn
#define ADC_ADCX_DMASx_IRQHandler       DMA1_Stream7_IRQHandler

/*判断 DMA1 Stream7 传输完成标志，这是一个假函数形式，不能当函数使用，只能用在 if 等语句里*/
#define ADC_ADCX_DMASx_IS_TC()
        ( __HAL_DMA_GET_FLAG(&g_dma_adc_handle, DMA_FLAG_TCIF3_7) )

/* 清除 DMA1 Stream7 传输完成标志 */
#define ADC_ADCX_DMASx_CLR_TC()
       do{ __HAL_DMA_CLEAR_FLAG(&g_dma_adc_handle, DMA_FLAG_TCIF3_7); }while(0)
```

下面给大家介绍 adc.c 文件里面的函数，首先是 ADC DMA 读取初始化函数。

```
/**
 * @brief       ADC DMA 读取 初始化函数
 * @param       par         : 外设地址
 * @param       mar         : 存储器地址
 * @retval      无
 */
void adc_dma_init(uint32_t par, uint32_t mar)
{
    GPIO_InitTypeDef gpio_init_struct;
    ADC_ChannelConfTypeDef adc_ch_conf = {0};

    ADC_ADCX_CHY_GPIO_CLK_ENABLE();                 /* 开启 ADC 通道 IO 引脚时钟 */
    ADC_ADCX_CHY_CLK_ENABLE();                      /* 使能 ADC1/2 时钟 */

    /* 得到当前 stream 是属于 DMA2 还是 DMA1 */
    if ((uint32_t)ADC_ADCX_DMASx > (uint32_t)DMA2)
    {
        __HAL_RCC_DMA2_CLK_ENABLE();                /* DMA2 时钟使能 */
    }
    else
    {
        __HAL_RCC_DMA1_CLK_ENABLE();                /* DMA1 时钟使能 */
    }

    __HAL_RCC_ADC_CONFIG(RCC_ADCCLKSOURCE_CLKP); /* ADC 外设时钟选择 */

    /* 初始化 ADC 采集通道对应的 IO 引脚 */
    gpio_init_struct.Pin = ADC_ADCX_CHY_GPIO_PIN;       /* ADC 通道 IO 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_ANALOG;           /* 模拟 */
    HAL_GPIO_Init(ADC_ADCX_CHY_GPIO_PORT, &gpio_init_struct);

    /* 初始化 DMA */
    g_dma_adc_handle.Instance = ADC_ADCX_DMASx;             /* 使用 DMA1 Stream7 */
    g_dma_adc_handle.Init.Request = ADC_ADCX_DMASx_REQ;     /* 请求选择*/
    g_dma_adc_handle.Init.Direction=DMA_PERIPH_TO_MEMORY;/* 从外设到存储器模式 */
    g_dma_adc_handle.Init.PeriphInc = DMA_PINC_DISABLE;     /* 外设非增量模式 */
    g_dma_adc_handle.Init.MemInc = DMA_MINC_ENABLE;         /* 存储器增量模式 */
    g_dma_adc_handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;
    /* 外设数据长度:16 位 */
    /* 存储器数据长度:16 位 */
    g_dma_adc_handle.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;
    g_dma_adc_handle.Init.Mode = DMA_NORMAL;        /* 非循环模式（即使用普通模式） */
    g_dma_adc_handle.Init.Priority = DMA_PRIORITY_MEDIUM;   /* 中等优先级 */
    g_dma_adc_handle.Init.FIFOMode = DMA_FIFOMODE_DISABLE;  /* 禁止 FIFO*/
    HAL_DMA_Init(&g_dma_adc_handle);                        /* 初始化 DMA */

    /* 将 DMA 句柄与 ADC 句柄关联起来 */
    __HAL_LINKDMA(&g_adc_dma_handle, DMA_Handle, g_dma_adc_handle);

    /* 初始化 ADC */
    g_adc_dma_handle.Instance = ADC_ADCX;           /* 选择哪个 ADC */
    /* 输入时钟 2 分频,即 adc_ker_ck=per_ck/2=32Mhz */
    g_adc_dma_handle.Init.ClockPrescaler = ADC_CLOCK_ASYNC_DIV2;
    g_adc_dma_handle.Init.Resolution = ADC_RESOLUTION_16B;  /* 16 位模式 */
    g_adc_dma_handle.Init.ScanConvMode = DISABLE;           /* 非扫描模式 */
    g_adc_dma_handle.Init.EOCSelection = ADC_EOC_SINGLE_CONV;/* 关闭 EOC 中断 */
    g_adc_dma_handle.Init.LowPowerAutoWait = DISABLE;       /* 自动低功耗关闭 */
    g_adc_dma_handle.Init.ContinuousConvMode = ENABLE;      /* 使能连续转换模式 */
    g_adc_dma_handle.Init.NbrOfConversion = 1;
    /* 赋值范围是 1~16，这里用到 1 个通道 */
    /* 禁止常规转换组不连续采样模式 */
```

```
    g_adc_dma_handle.Init.DiscontinuousConvMode = DISABLE;
    /* 配置不连续采样模式的通道数，禁止常规转换组不连续采样模式后，此参数忽略 */
    g_adc_dma_handle.Init.NbrOfDiscConversion = 0;
    g_adc_dma_handle.Init.ExternalTrigConv = ADC_SOFTWARE_START;
    /* 采用软件触发 */
    /* 采用软件触发的话，此位忽略 */
    g_adc_dma_handle.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
    g_adc_dma_handle.Init.ConversionDataManagement =
                    ADC_CONVERSIONDATA_DMA_ONESHOT;     /* DMA 单次传输 ADC 数据 */
    /* 有新的数据后直接覆盖掉旧数据 */
    g_adc_dma_handle.Init.Overrun = ADC_OVR_DATA_OVERWRITTEN;
    /* 设置 ADC 转换结果的左移位数 */
    g_adc_dma_handle.Init.LeftBitShift = ADC_LEFTBITSHIFT_NONE;
    g_adc_dma_handle.Init.OversamplingMode = DISABLE;    /* 过采样关闭 */
    HAL_ADC_Init(&g_adc_dma_handle);                     /* 初始化 */

    HAL_ADCEx_Calibration_Start(&g_adc_dma_handle, ADC_CALIB_OFFSET,
                                ADC_SINGLE_ENDED);       /* ADC 校准 */

    /* 配置 ADC 通道 */
    adc_ch_conf.Channel = ADC_ADCX_CHY;            /* 配置使用的 ADC 通道 */
    adc_ch_conf.Rank = ADC_REGULAR_RANK_1;         /* 采样序列里的第 1 个 */
    /* 采样周期为 810.5 个时钟周期 */
    adc_ch_conf.SamplingTime = ADC_SAMPLETIME_810CYCLES_5;
    adc_ch_conf.SingleDiff = ADC_SINGLE_ENDED ;    /* 单端输入 */
    adc_ch_conf.OffsetNumber = ADC_OFFSET_NONE;    /* 无偏移 */
    adc_ch_conf.Offset = 0;                        /* 无偏移的情况下，此参数忽略 */
    adc_ch_conf.OffsetRightShift = DISABLE;        /* 禁止右移 */
    adc_ch_conf.OffsetSignedSaturation = DISABLE; /* 禁止有符号饱和 */
    HAL_ADC_ConfigChannel(&g_adc_dma_handle, &adc_ch_conf);  /* 配置 ADC 通道 */

    /* 配置 DMA 数据流请求中断优先级 */
    HAL_NVIC_SetPriority(ADC_ADCX_DMASx_IRQn, 3, 3);
    HAL_NVIC_EnableIRQ(ADC_ADCX_DMASx_IRQn);

    HAL_DMA_Start_IT(&g_dma_adc_handle, par, mar, 0); /* 启动 DMA，并开启中断 */
    HAL_ADC_Start_DMA(&g_adc_dma_handle, &mar, 0);/* 开启 ADC，通过 DMA 传输结果 */
}
```

adc_dma_init 函数包含了输出通道对应 IO 的初始代码、NVIC、使能时钟、ADC 时钟预分频系数、ADC 工作参数和 ADC 通道配置等代码。下面来看看该函数的代码内容。

第一部分使能 ADC、DMA 和 GPIO 的时钟。

第二部分选择 ADC 的时钟源为 per_ck，per_ck 默认选择 hsi_ker_ck 作为时钟源，即 ADC 的时钟源为 64MHZ 的 hsi_ker_ck。

第三部分是设置 ADC 采集通道对应 IO 引脚工作模式。

第四部分初始化 DMA，并通过__HAL_LINKDMA 宏定义将 DMA 相关的配置关联到 ADC 的句柄中。

第五部分是初始化 ADC，并校准 ADC。

第六部分是配置 ADC 通道。

第七部分是配置 DMA 数据流请求中断优先级，并使能该中断。

第八部分是启动 DMA 并开启 DMA 中断，以及启动 ADC 并通过 DMA 传输转换结果。

为了方便代码的管理和移植性等，这里就没有使用 HAL_ADC_MspInit 这个函数来存放使能时钟、GPIO、NVIC 相关的代码，而是全部存放在 adc_dma_init 函数中。

接下来给大家介绍使能一次 ADC DMA 传输函数，其定义如下：

```
/**
 * @brief     使能一次 ADC DMA 传输
 * @note      该函数用寄存器来操作，防止用 HAL 库操作对其他参数有修改，也是为了兼容后续实验
 * @param     ndtr: DMA 传输的次数
```

```
 * @retval    无
 */
void adc_dma_enable(uint16_t ndtr)
{
    ADC_ADCX->CR &= ~(1 << 0);              /* 先关闭 ADC */

    ADC_ADCX_DMASx->CR &= ~(1 << 0);        /* 关闭 DMA 传输 */
    while (ADC_ADCX_DMASx->CR & 0X1);       /* 确保 DMA 可以被设置 */
    ADC_ADCX_DMASx->NDTR = ndtr;            /* 要传输的数据项数目 */
    ADC_ADCX_DMASx->CR |= 1 << 0;           /* 开启 DMA 传输 */
    ADC_ADCX->CR |= 1 << 0;                 /* 重新启动 ADC */
    ADC_ADCX->CR |= 1 << 2;                 /* 启动常规转换通道 */
}
```

该函数我们使用寄存器来操作，防止用 HAL 库相关宏操作会对其它参数进行修改，同时也是为了兼容后面的实验。HAL_DMA_Start_IT 函数已经配置好了 DMA 传输的源地址和目标地址，本函数只需要调用 ADC_ADCX_DMASx->NDTR = ndtr;语句给 DMA_SxNDTR 寄存器写入要传输的数据项数目，然后启动 DMA 就可以传输了。

下面介绍的是 ADC DMA 采集中断服务函数，函数定义如下：

```
/**
 * @brief       ADC DMA 采集中断服务函数
 * @param       无
 * @retval      无
 */
void ADC_ADCX_DMASx_IRQHandler(void)
{
    if (ADC_ADCX_DMASx_IS_TC())
    {
        g_adc_dma_sta = 1;              /* 标记 DMA 传输完成 */
        ADC_ADCX_DMASx_CLR_TC();        /* 清除 DMA1 数据流 7 传输完成中断 */
    }
}
```

在该函数里，通过判断 DMA 传输完成标志位是否是 1，是 1 就给 g_adc_dma_sta 变量赋值为 1，标记 DMA 传输完成，最后清除 DMA 的传输完成标志位。

最后在 main.c 里面编写如下代码：

```
#define ADC_DMA_BUF_SIZE        50          /* ADC DMA 采集 BUF 大小 */
uint16_t g_adc_dma_buf[ADC_DMA_BUF_SIZE];/* ADC DMA BUF */
extern uint8_t g_adc_dma_sta;               /* DMA 传输状态标志, 0,未完成; 1, 已完成 */

int main(void)
{
    uint16_t i;
    uint16_t adcx;
    uint32_t sum;
    float temp;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    /* 初始化 ADC DMA 采集 */
    adc_dma_init((uint32_t)&ADC1->DR, (uint32_t)&g_adc_dma_buf);
    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "ADC DMA TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 130, 200, 16, 16, "ADC1_CH19_VAL:", BLUE);
```

```
    /* 先在固定位置显示小数点 */
    lcd_show_string(30, 150, 200, 16, 16, "ADC1_CH19_VOL:0.000V", BLUE);
    adc_dma_enable(ADC_DMA_BUF_SIZE);    /* 启动 ADC DMA 采集 */
    while (1)
    {
        if (g_adc_dma_sta == 1)
        {
            /* 清 D Cache */
            SCB_InvalidateDCache();
            /* 计算 DMA 采集到的 ADC 数据的平均值 */
            sum = 0;
            for (i=0; i<ADC_DMA_BUF_SIZE; i++)    /* 累加 */
            {
                sum += g_adc_dma_buf[i];
            }
            adcx= sum / ADC_DMA_BUF_SIZE;    /* 取平均值 */

            /* 显示结果 */
            lcd_show_xnum(142, 110, adcx, 5,16,0,BLUE);/* 显示 ADC 采样的原始值*/
            /* 获取计算后的带小数的实际电压值，比如 3.1111 */
            temp = (float)adcx * (3.3 / 65536);
            adcx = temp;  /* 赋值整数部分给 adcx 变量，因为 adcx 为 u16 整形 */
            /* 显示电压值的整数部分，3.1111 的话，这里就是显示 3 */
            lcd_show_xnum(142, 150, adcx, 1, 16, 0, BLUE);
            /* 把已经显示的整数部分去掉，留下小数部分，比如 3.1111-3=0.1111 */
            temp -= adcx;
            /* 小数部分乘以 1000，例如：0.1111 就转换为 111.1，相当于保留三位小数。 */
            temp *= 1000;
            /* 显示小数部分（前面转换为了整形显示），这里显示的就是 111. */
            lcd_show_xnum(158, 150, temp, 3, 16, 0X80, BLUE);
            g_adc_dma_sta = 0;                    /* 清除 DMA 采集完成状态标志 */
            adc_dma_enable(ADC_DMA_BUF_SIZE); /* 启动下一次 ADC DMA 采集 */
        }
        LED0_TOGGLE();
        delay_ms(100);
    }
}
```

此部分代码，和单通道 ADC 采集实验十分相似，只是这里使能了 DMA 传输数据，DMA 传输的数据存放在 g_adc_dma_buf 数组里，这里我们对数组的数据取平均值，减少误差。在 LCD 屏显示结果的处理和单通道 ADC 采集实验一样。首先我们在液晶固定位置显示了小数点，然后后面计算步骤中，先计算出整数部分在小数点前面显示，然后计算出小数部分，在小数点后面显示。这样就在液晶上面显示转换结果的整数和小数部分。

33.3.4 下载验证

下载代码后，可以看到 LCD 显示如图 33.3.4.1 所示：

图 33.3.4.1 单通道 ADC 采集（DMA 读取）实验测试图

这里的实验效果和单通道 ADC 采集实验是一样的，我们使用杜邦线将 P10 的 ADC 和 RV1 连接，使得 PA5 连接到电位器上，测试的是电位器的电压，并可以通过螺丝刀调节电位器改变

电压值，范围：0~3.3V。LED0 闪烁，提示程序运行。

大家也可以用杜邦线将 ADC 排针接到其它待测量的电压点，看看测量到的电压值是否准确？**但是要注意：一定要保证测试点的电压在 0~3.3V 的电压范围，否则可能烧坏我们的 ADC，甚至是整个主控芯片。**

33.4 多通道 ADC 采集（DMA 读取）实验

本实验我们来学习多通道 ADC 采集（DMA 读取）实验。本实验使用常规转换组多通道的连续转换模式，并且通过软件触发，即通过对 ADCx_CR 寄存器的 ADSTART 位写 1 启动转换。由于使用连续转换模式，所以使用 DMA 读取转换结果的方式。下面先带大家来了解本实验要配置的寄存器。

33.4.1 ADC 寄存器

本实验我们很多的设置和单通道 ADC 采集（DMA 读取）实验是一样的，所以下面介绍寄存器的时候我们不会继续全部都介绍，而是针对性选择与单通道 ADC 采集（DMA 读取）实验不同设置的 ADCx_SQR 寄存器进行介绍，其他的配置基本一样的。另外我们用到 DMA 读取数据，配置上和单通道 ADC 采集（DMA 读取）实验是一样的。

ADCx 常规序列寄存器有四个（ADCx_SQR1~ ADCx_SQR4），具体怎么配置，需要看我们用多少个通道，比如本实验我们使用 6 个通道同时采集 ADC 数据，具体配置如下：

- **ADCx 常规序列寄存器 1（ADCx_SQR1）**

ADCx 常规序列寄存器 1 描述如图 33.4.1.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	SQ4[4:0]					Res.	SQ3[4:0]					Res.	SQ2[4]
			rw	rw	rw	rw	rw		rw	rw	rw	rw	rw		rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SQ2[3:0]				Res.	SQ1[4:0]					Res.	Res.	L[3:0]			
rw	rw	rw	rw		rw	rw	rw	rw	rw			rw	rw	rw	rw

位 28:24 **SQ4[4:0]**：常规序列中的第四次转换 (4th conversion in regular sequence)
通过软件写入这些位，并将通道编号 (0..19) 分配为常规转换序列中的第四次转换。

位 22:18 **SQ3[4:0]**：常规序列中的第三次转换 (3th conversion in regular sequence)
通过软件写入这些位，并将通道编号 (0..19) 分配为常规转换序列中的第三次转换。
注：仅当 ADSTART=0 时（这可确保当前未进行任何常规转换），才允许通过软件对这些位执行写操作。

位 16:12 **SQ2[4:0]**：常规序列中的第二次转换 (2th conversion in regular sequence)
通过软件写入这些位，并将通道编号 (0..19) 分配为常规转换序列中的第二次转换。
注：仅当 ADSTART=0 时（这可确保当前未进行任何常规转换），才允许通过软件对这些位执行写操作。

位 10:6 **SQ1[4:0]**：常规序列中的第一次转换 (1th conversion in regular sequence)
通过软件写入这些位，并将通道编号 (0..19) 分配为常规转换序列中的第一次转换。
注：仅当 ADSTART=0 时（这可确保当前未进行任何常规转换），才允许通过软件对这些位执行写操作。

位 3:0 **L[3:0]**：常规通道序列长度 (Regular channel sequence length)
通过软件写入这些位可定义常规通道转换序列中的转换总数。
0000：1 次转换
0001：2 次转换
...
1111：16 次转换
注：仅当 ADSTART=0 时（这可确保当前未进行任何常规转换），才允许通过软件对这些位执行写操作。

图 33.4.1.1 ADCx_SQR1 寄存器

L[3:0]位用于设置常规序列的长度，取值范围：0~15，表示常规序列长度为 1~16。本实验使用到 6 个通道，所以设置这几个位的值为 5 即可。

SQ1[4:0]~SQ4[4:0]位设置常规组序列的第 1~4 个转换编号，第 5~16 个转换编号的设置请查看 ADCx_SQR2 和 ADCx_SQR4 寄存器。设置过程非常简单，忘记了请参考前面给大家整理

出来的常规序列寄存器控制关系汇总表。

下面我们来看看本实验是怎么设置的：SQ1[4:0]位赋值为 14、SQ2[4:0]位赋值为 15、SQ3[4:0]位赋值为 16、SQ4[4:0]位赋值为 17、SQ5[4:0]位赋值为 18、SQ6[4:0]位赋值为 19，即常规序列 1 到 6 分别对应的通道是 14 到 19。其中 SQ5[4:0]位和 SQ6[4:0]位是在 ADCx_SQR2 寄存器中配置。

33.4.2 硬件设计

1. 例程功能

使用 ADC1 采集（DMA 读取）通道 14\15\16\17\18\19 的电压，在 LCD 模块上面显示对应的 ADC 转换值以及换算成电压后的电压值。可以使用杜邦线连接 PA0\PA1\PA2\PA3\PA4\PA5 到你想测量的电压源（0~3.3V），然后通过 TFTLCD 显示的电压值。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 : LED0 - PB1
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）ADC1 ：通道 14 - PA2、通道 15 - PA3、通道 16 - PA0、
通道 17 - PA1、通道 18 - PA4、通道 19 - PA5
5）DMA（DMA1 数据流 7 DMA 请求源 9）

3. 原理图

ADC 和 DMA 属于 STM32H743 内部资源，实际上我们只需要软件设置就可以正常工作，另外还需要将待测量的电压源连接到 ADC 通道上，以便 ADC 测量。本实验，我们通过 ADC1 的通道 14\15\16\17\18\19 来采集外部电压值，并通过 DMA 来读取。

33.4.3 程序设计

33.4.3.1 ADC 的 HAL 库驱动

本实验用到的 ADC 的 HAL 库 API 函数前面都介绍过，具体调用情况请看程序解析部分。下面介绍多通道 ADC 采集（DMA 读取）配置步骤。

多通道 ADC 采集（DMA 读取）配置步骤

1）开启 ADCx 和 ADC 通道对应的 IO 时钟，并配置该 IO 为模拟功能

首先开启 ADCx 的时钟，然后配置 GPIO 为模拟模式。本实验我们默认用到 ADC1 通道 14、15、16、17、18、19，对应 IO 是 PA2、PA3、PA0、PA1、PA4 和 PA5，它们的时钟开启方法如下：

```
__HAL_RCC_ADC12_CLK_ENABLE();            /* 使能 ADC1 & ADC2 时钟 */
__HAL_RCC_GPIOA_CLK_ENABLE();            /* 开启 GPIOA 时钟 */
```

2）初始化 ADCx，配置其工作参数

通过 HAL_ADC_Init 函数来设置 ADCx 时钟分频系数、分辨率、模式、扫描方式、对齐方式等信息。

注意：该函数会调用：HAL_ADC_MspInit 回调函数来存放 ADC 及 GPIO 时钟使能、GPIO 初始化等代码。我们也可以不存放在这个函数里，本实验就没用到这个 MSP 回调函数。

3）配置 ADC 通道

在 HAL 库中，通过 HAL_ADC_ConfigChannel 函数来设置配置 ADC 的通道，根据需求设

置通道、序列、采样时间和校准配置单端输入模式或差分输入模式等。这里配置多通道输出，需要多次调用该函数。

4）初始化 DMA

通过 HAL_DMA_Init 函数初始化 DMA，包括配置通道，外设地址，存储器地址，传输数据量等。

HAL 库为了处理各类外设的 DMA 请求，在调用相关函数之前，需要调用一个宏定义标识符，来连接 DMA 和外设句柄。这个宏定义为__HAL_LINKDMA。

5）使能 DMA 对应数据流中断，配置 DMA 中断优先级并开启中断，启动 ADC 和 DMA

通过 HAL_ADC_Start_DMA 函数开启 ADC 转换，通过 DMA 传输结果。

通过 HAL_DMA_Start_IT 函数启动 DMA 读取，使能 DMA 中断。

通过 HAL_NVIC_EnableIRQ 函数使能 DMA 数据流中断。

通过 HAL_NVIC_SetPriority 函数设置中断优先级。

6）编写中断服务函数

DMA 中断对于每个数据流都有一个中断服务函数，比如 DMA1_Stream7 的中断服务函数为 DMA1_Stream7_IRQHandler。简单的做法就是在，对应的中断服务函数里面，通过判断相关的中断标志位的方式，完成中断逻辑代码，最后清楚该中断标志位，本实验的做法就是如此。

还可以通过调用 HAL 库提供的 DMA 中断公用处理函数 HAL_DMA_IRQHandler，然后定重新义相关的中断回调处理函数。

33.4.3.2 程序流程图

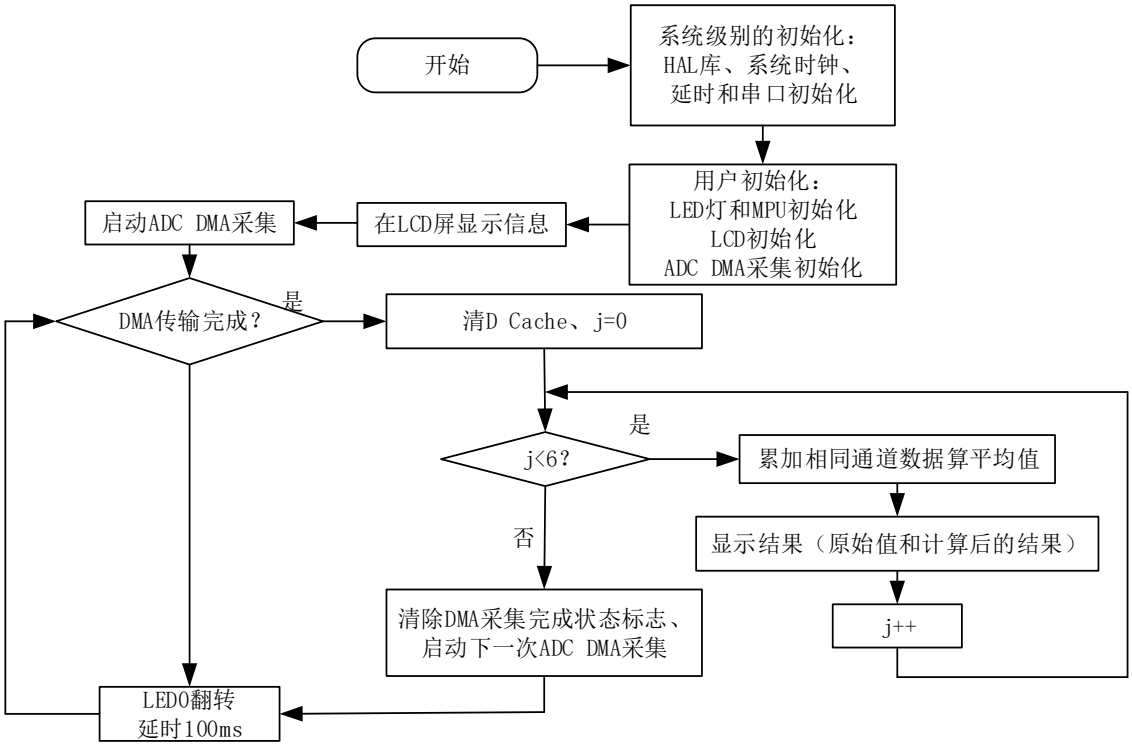


图 33.4.3.2.1 多通道 ADC 采集（DMA 读取）实验程序流程图

33.4.3.3 程序解析

在本实验中 adc.h 头文件只是添加了一些函数声明，下面开始介绍 adc.c 的函数，本实验只增加了一个函数，ADC 的 N 通道(6 通道) DMA 读取初始化函数，其定义如下：

```
/**
 * @brief    ADC N 通道(6 通道) DMA 读取 初始化函数
 * @note     另外,由于本函数用到了 6 个通道, 宏定义会比较多内容, 因此,本函数就不采用宏定义
            的方式来修改通道了,直接在本函数里面修改, 这里我们默认使用 PA0~PA5 这 6 个通道.
```

```
 *           注意: 本函数还是使用 ADC_ADCX(默认=ADC1) 和 ADC_ADCX_DMASx
             (默认=DMA1_Stream7) 及其相关定义不要乱修改 adc.h 里面的这两部分内容, 必须在
             理解原理的基础上进行修改, 否则可能导致无法正常使用.
 * @param    par          : 外设地址
 * @param    mar          : 存储器地址
 * @retval   无
 */
void adc_nch_dma_init(uint32_t par, uint32_t mar)
{
    GPIO_InitTypeDef gpio_init_struct;
    ADC_ChannelConfTypeDef adc_ch_conf = {0};

    __HAL_RCC_GPIOA_CLK_ENABLE();                /* 开启 GPIOA 引脚时钟 */
    ADC_ADCX_CHY_CLK_ENABLE();                   /* 使能 ADC1/2 时钟 */

    /* 得到当前 stream 是属于 DMA2 还是 DMA1 */
    if ((uint32_t)ADC_ADCX_DMASx > (uint32_t)DMA2)
    {
        __HAL_RCC_DMA2_CLK_ENABLE();             /* DMA2 时钟使能 */
    }
    else
    {
        __HAL_RCC_DMA1_CLK_ENABLE();             /* DMA1 时钟使能 */
    }

    __HAL_RCC_ADC_CONFIG(RCC_ADCCLKSOURCE_CLKP); /* ADC 外设时钟选择 */

    /* 初始化 ADC 多通道对应的 GPIO
     * PA0-ADC_CHANNEL_16、PA1-ADC_CHANNEL_17、PA2-ADC_CHANNEL_14
     * PA3-ADC_CHANNEL_15、PA4-ADC_CHANNEL_18、PA5-ADC_CHANNEL_19
     */
    gpio_init_struct.Pin = GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2 | GPIO_PIN_3
                           | GPIO_PIN_4 | GPIO_PIN_5; /* GPIOA0~5 */
    gpio_init_struct.Mode = GPIO_MODE_ANALOG;          /* 模拟 */
    HAL_GPIO_Init(GPIOA, &gpio_init_struct);

    /* 初始化 DMA */
    g_dma_nch_adc_handle.Instance = ADC_ADCX_DMASx;    /* 使用 DMA1 Stream7 */
    /* 请求选择 DMA_REQUEST_ADC1 */
    g_dma_nch_adc_handle.Init.Request = ADC_ADCX_DMASx_REQ;
    /* 传外设到存储器模式 */
    g_dma_nch_adc_handle.Init.Direction = DMA_PERIPH_TO_MEMORY;
    /* 外设非增量模式 */
    g_dma_nch_adc_handle.Init.PeriphInc = DMA_PINC_DISABLE;
    g_dma_nch_adc_handle.Init.MemInc = DMA_MINC_ENABLE;     /* 存储器增量模式 */
    /* 外设数据长度:16 位 */
    g_dma_nch_adc_handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;
    /* 存储器数据长度:16 位 */
    g_dma_nch_adc_handle.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;
    g_dma_nch_adc_handle.Init.Mode = DMA_NORMAL;  /* 非循环模式(即使用普通模式) */
    g_dma_nch_adc_handle.Init.Priority = DMA_PRIORITY_MEDIUM; /* 中等优先级 */
    g_dma_nch_adc_handle.Init.FIFOMode = DMA_FIFOMODE_DISABLE;/* 禁止 FIFO*/
    HAL_DMA_Init(&g_dma_nch_adc_handle);                      /* 初始化 DMA */

    /* 将 DMA 与 adc 联系起来 */
    __HAL_LINKDMA(&g_adc_nch_dma_handle, DMA_Handle, g_dma_nch_adc_handle);

    /* 初始化 ADC */
    g_adc_nch_dma_handle.Instance = ADC_ADCX;          /* 选择哪个 ADC */
    /* 输入时钟 2 分频,即 adc_ker_ck=per_ck/2=32Mhz */
    g_adc_nch_dma_handle.Init.ClockPrescaler = ADC_CLOCK_ASYNC_DIV2;
```

```
    g_adc_nch_dma_handle.Init.Resolution = ADC_RESOLUTION_16B; /* 16 位模式  */
    g_adc_nch_dma_handle.Init.ScanConvMode = ADC_SCAN_ENABLE;  /* 扫描模式 */
    g_adc_nch_dma_handle.Init.EOCSelection=ADC_EOC_SINGLE_CONV;/*关闭 EOC 中断 */
    g_adc_nch_dma_handle.Init.LowPowerAutoWait = DISABLE;  /* 自动低功耗关闭 */
    g_adc_nch_dma_handle.Init.ContinuousConvMode = ENABLE; /* 使能连续转换模式 */
    /* 赋值范围是 1~16，本实验用到 6 个通道 */
    g_adc_nch_dma_handle.Init.NbrOfConversion = 6;
    /* 禁止常规转换组不连续采样模式 */
    g_adc_nch_dma_handle.Init.DiscontinuousConvMode = DISABLE;
    /* 配置不连续采样模式的通道数，禁止常规转换组不连续采样模式后，此参数忽略 */
    g_adc_nch_dma_handle.Init.NbrOfDiscConversion = 0;
    /* 采用软件触发 */
    g_adc_nch_dma_handle.Init.ExternalTrigConv = ADC_SOFTWARE_START;
    g_adc_nch_dma_handle.Init.ExternalTrigConvEdge =
              ADC_EXTERNALTRIGCONVEDGE_NONE;     /* 采用软件触发的话，此位忽略 */
    g_adc_nch_dma_handle.Init.ConversionDataManagement =
              ADC_CONVERSIONDATA_DMA_ONESHOT;    /* DMA 单次传输 ADC 数据 */
    /* 有新的数据后直接覆盖掉旧数据 */
    g_adc_nch_dma_handle.Init.Overrun = ADC_OVR_DATA_OVERWRITTEN;
    /* 设置 ADC 转换结果的左移位数 */
    g_adc_nch_dma_handle.Init.LeftBitShift = ADC_LEFTBITSHIFT_NONE;
    g_adc_nch_dma_handle.Init.OversamplingMode = DISABLE;    /* 过采样关闭 */
    HAL_ADC_Init(&g_adc_nch_dma_handle);                     /* 初始化 */

    HAL_ADCEx_Calibration_Start(&g_adc_nch_dma_handle, ADC_CALIB_OFFSET,
                                ADC_SINGLE_ENDED);          /* ADC 校准 */

    /* 配置 ADC 通道 */
    adc_ch_conf.Channel = ADC_CHANNEL_14;          /* 配置使用的 ADC 通道 */
    adc_ch_conf.Rank = ADC_REGULAR_RANK_1;         /* 采样序列里的第 1 个 */
    /* 采样周期为 810.5 个时钟周期 */
    adc_ch_conf.SamplingTime = ADC_SAMPLETIME_810CYCLES_5;
    adc_ch_conf.SingleDiff = ADC_SINGLE_ENDED ;    /* 单端输入 */
    adc_ch_conf.OffsetNumber = ADC_OFFSET_NONE;    /* 无偏移 */
    adc_ch_conf.Offset = 0;                        /* 无偏移的情况下，此参数忽略 */
    adc_ch_conf.OffsetRightShift = DISABLE;        /* 禁止右移 */
    adc_ch_conf.OffsetSignedSaturation = DISABLE; /* 禁止有符号饱和 */
    HAL_ADC_ConfigChannel(&g_adc_nch_dma_handle, &adc_ch_conf);/* 配置 ADC 通道 */

    adc_ch_conf.Channel = ADC_CHANNEL_15;          /* 配置使用的 ADC 通道 */
    adc_ch_conf.Rank = ADC_REGULAR_RANK_2;         /* 采样序列里的第 2 个 */
    HAL_ADC_ConfigChannel(&g_adc_nch_dma_handle, &adc_ch_conf);/* 配置 ADC 通道 */

    adc_ch_conf.Channel = ADC_CHANNEL_16;          /* 配置使用的 ADC 通道 */
    adc_ch_conf.Rank = ADC_REGULAR_RANK_3;         /* 采样序列里的第 3 个 */
    HAL_ADC_ConfigChannel(&g_adc_nch_dma_handle, &adc_ch_conf);/* 配置 ADC 通道 */

    adc_ch_conf.Channel = ADC_CHANNEL_17;          /* 配置使用的 ADC 通道 */
    adc_ch_conf.Rank = ADC_REGULAR_RANK_4;         /* 采样序列里的第 4 个 */
    HAL_ADC_ConfigChannel(&g_adc_nch_dma_handle, &adc_ch_conf);/* 配置 ADC 通道 */

    adc_ch_conf.Channel = ADC_CHANNEL_18;          /* 配置使用的 ADC 通道 */
    adc_ch_conf.Rank = ADC_REGULAR_RANK_5;         /* 采样序列里的第 5 个 */
    HAL_ADC_ConfigChannel(&g_adc_nch_dma_handle, &adc_ch_conf);/* 配置 ADC 通道 */

    adc_ch_conf.Channel = ADC_CHANNEL_19;          /* 配置使用的 ADC 通道 */
    adc_ch_conf.Rank = ADC_REGULAR_RANK_6;         /* 采样序列里的第 6 个 */
    HAL_ADC_ConfigChannel(&g_adc_nch_dma_handle, &adc_ch_conf);/* 配置 ADC 通道 */

    /* 配置 DMA 数据流请求中断优先级 */
```

```
    HAL_NVIC_SetPriority(ADC_ADCX_DMASx_IRQn, 3, 3);
    HAL_NVIC_EnableIRQ(ADC_ADCX_DMASx_IRQn);
    /* 启动 DMA，并开启中断 */
    HAL_DMA_Start_IT(&g_dma_nch_adc_handle, par, mar, 0);
    /* 开启 ADC，通过 DMA 传输结果 */
    HAL_ADC_Start_DMA(&g_adc_nch_dma_handle, &mar, 0);
}
```

adc_nch_dma_init 函数包含了输出通道对应 IO 的初始代码、NVIC、使能时钟、ADC 时钟预分频系数、ADC 工作参数和 ADC 通道配置等代码。大部分代码和单通道 ADC 采集(DMA 读取)实验一样，下面来看看该函数的代码内容。

第一部分使能 ADC、DMA 和 GPIO 的时钟。

第二部分选择 ADC 的时钟源为 per_ck，per_ck 默认选择 hsi_ker_ck 作为时钟源，即 ADC 的时钟源为 64MHZ 的 hsi_ker_ck。

第三部分是设置 ADC 采集通道对应 IO 引脚工作模式，这里用到 6 个通道。

第四部分初始化 DMA，并通过__HAL_LINKDMA 宏定义将 DMA 相关的配置关联到 ADC 的句柄中。

第五部分是初始化 ADC，并校准 ADC。

第六部分是配置 ADC 通道，这里有 6 个通道需要配置。

第七部分是配置 DMA 数据流请求中断优先级，并使能该中断。

第八部分是启动 DMA 并开启 DMA 中断，以及启动 ADC 并通过 DMA 传输转换结果。

为了方便代码的管理和移植性等，这里就没有使用 HAL_ADC_MspInit 这个函数来存放使能时钟、GPIO、NVIC 相关的代码，而是全部存放在 adc_nch_dma_init 函数中。

最后在 main.c 里面编写如下代码：

```
#define ADC_DMA_BUF_SIZE  50*6 /* ADC DMA 采集 BUF 大小，应等于 ADC 通道数的整数倍 */
uint16_t g_adc_dma_buf[ADC_DMA_BUF_SIZE]; /* ADC DMA BUF */

extern uint8_t g_adc_dma_sta;       /* DMA 传输状态标志，0,未完成；1，已完成 */

int main(void)
{
    uint16_t i,j;
    uint16_t adcx;
    uint32_t sum;
    float temp;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟，400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    /* 初始化 ADC DMA 采集 */
    adc_nch_dma_init((uint32_t)&ADC1->DR, (uint32_t)&g_adc_dma_buf);

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "ADC DMA TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 130, 200, 12, 12, "ADC1_CH14_VAL:", BLUE);
    /* 先在固定位置显示小数点 */
    lcd_show_string(30, 142, 200, 12, 12, "ADC1_CH14_VOL:0.000V", BLUE);

    lcd_show_string(30, 160, 200, 12, 12, "ADC1_CH15_VAL:", BLUE);
    /* 先在固定位置显示小数点 */
    lcd_show_string(30, 172, 200, 12, 12, "ADC1_CH15_VOL:0.000V", BLUE);
```

```
    lcd_show_string(30, 190, 200, 12, 12, "ADC1_CH16_VAL:", BLUE);
    /* 先在固定位置显示小数点 */
    lcd_show_string(30, 202, 200, 12, 12, "ADC1_CH16_VOL:0.000V", BLUE);

    lcd_show_string(30, 220, 200, 12, 12, "ADC1_CH17_VAL:", BLUE);
    /* 先在固定位置显示小数点 */
    lcd_show_string(30, 232, 200, 12, 12, "ADC1_CH17_VOL:0.000V", BLUE);

    lcd_show_string(30, 250, 200, 12, 12, "ADC1_CH18_VAL:", BLUE);
    /* 先在固定位置显示小数点 */
    lcd_show_string(30, 262, 200, 12, 12, "ADC1_CH18_VOL:0.000V", BLUE);

    lcd_show_string(30, 280, 200, 12, 12, "ADC1_CH19_VAL:", BLUE);
    /* 先在固定位置显示小数点 */
    lcd_show_string(30, 292, 200, 12, 12, "ADC1_CH19_VOL:0.000V", BLUE);

    adc_dma_enable(ADC_DMA_BUF_SIZE);    /* 启动 ADC DMA 采集 */
    while (1)
    {
        if (g_adc_dma_sta == 1)
        {
            /* 清除 D Cache 数据 */
            SCB_InvalidateDCache();
            /* 循环显示通道 14~通道 19 的结果 */
            for(j = 0; j < 6; j++)  /* 遍历 6 个通道 */
            {
                sum = 0; /* 清零 */
                /* 每个通道采集了 50 次数据,进行 50 次累加 */
                for (i = 0; i < ADC_DMA_BUF_SIZE / 6; i++)
                {
                    sum += g_adc_dma_buf[(6 * i) + j];/* 相同通道的转换数据累加 */
                }
                adcx = sum / (ADC_DMA_BUF_SIZE / 6);  /* 取平均值 */

                /* 显示结果 */
                /* 显示 ADC 采样后的原始值 */
                lcd_show_xnum(114, 130 + (j * 30), adcx, 5, 12, 0, BLUE);
                /* 获取计算后的带小数的实际电压值，比如 3.1111 */
                temp = (float)adcx * (3.3 / 65536);
                adcx = temp;     /* 赋值整数部分给 adcx 变量，因为 adcx 为 u16 整形 */
                /* 显示电压值的整数部分，3.1111 的话，这里就是显示 3 */
                lcd_show_xnum(114, 142 + (j * 30), adcx, 1, 12, 0, BLUE);
                /* 把已经显示的整数部分去掉，留下小数部分，比如 3.1111-3=0.1111 */
                temp -= adcx;
                /* 小数部分乘以 1000，例如：0.1111 就转换为 111.1，相当于保留三位小数 */
                temp *= 1000;
                /* 显示小数部分（前面转换为了整形显示），这里显示的就是 111. */
                lcd_show_xnum(126, 142 + (j * 30), temp, 3, 12, 0X80, BLUE);
            }
            g_adc_dma_sta = 0;                  /* 清除 DMA 采集完成状态标志 */
            adc_dma_enable(ADC_DMA_BUF_SIZE);  /* 启动下一次 ADC DMA 采集 */
        }
        LED0_TOGGLE();
        delay_ms(100);
    }
}
```

这里使用了 DMA 传输数据，DMA 传输的数据存放在 g_adc_dma_buf 数组里，该数组的大小是 50 * 6。本实验用到 6 个通道，每个通道使用 50 个 uint16_t 大小的空间存放 ADC 的结果。输入通道 14 的转换数据存放在 g_adc_dma_buf[0]到 g_adc_dma_buf[49]，输入通道 15 的转换数据存放在 g_adc_dma_buf[50]到 g_adc_dma_buf[99]，后面的以此类推。然后对数组的每个通道的数据取平均值，减少误差。最后在 LCD 屏上显示 ADC 的转换值和换算成电压后的电压值。

33.4.4 下载验证

下载代码后，LED0 闪烁，提示程序运行。可以看到 LCD 显示如图 33.4.4.1 所示：

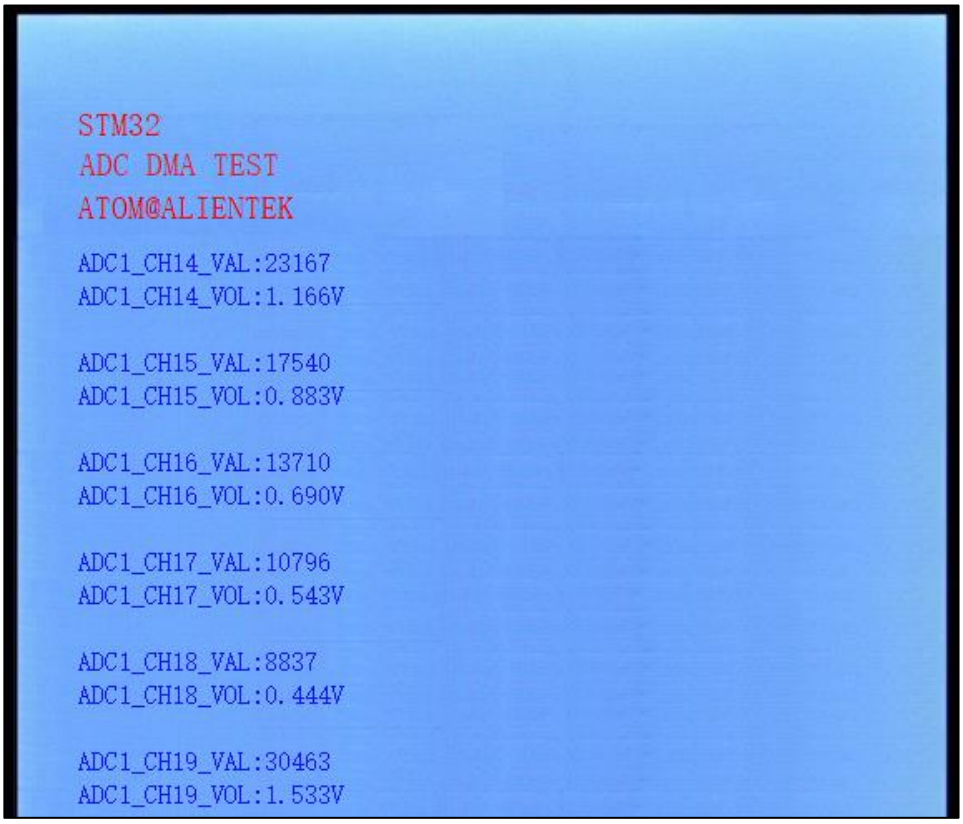


图 33.4.4.1 多通道 ADC 采集（DMA 读取）实验测试图

使用 ADC1 采集（DMA 读取）通道 14\15\16\17\18\19 的电压，在 LCD 模块上面显示对应的 ADC 转换值以及换算成电压后的电压值。可以使用杜邦线连接 PA0\PA1\PA2\PA3\PA4\PA5 到你想测量的电压源（0~3.3V）。

这 6 个通道对应引出来的引脚 PA0\PA1\PA2\PA3\PA4\PA5 在开发板上的位置，如下图所示：

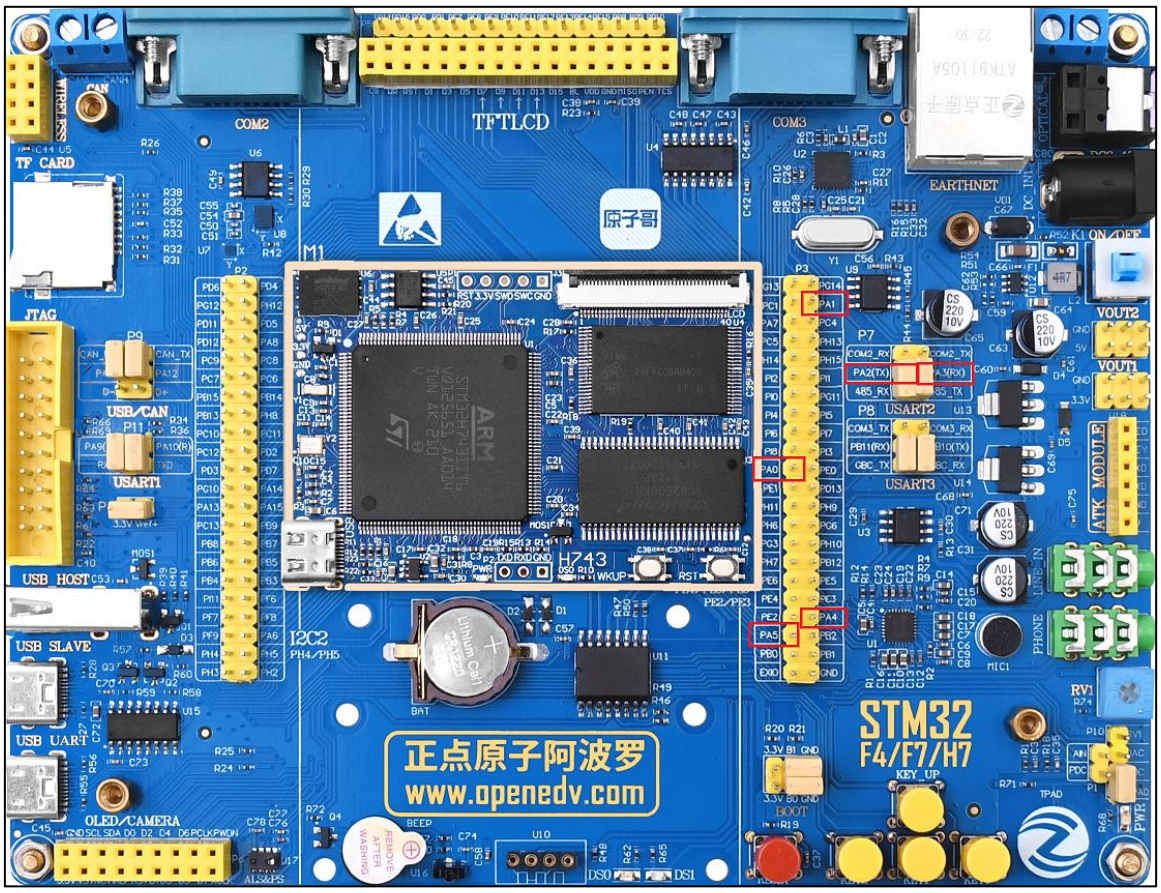


图 33.4.4.2 ADC1 的通道 14\15\16\17\18\19 引脚在开发板位置示意图

这六个通道可以同时测量不同测试点的电压，只需要用杜邦线分别接到不同的电压测试点即可。**注意：一定要保证测试点的电压在 0~3.3V 的电压范围，否则可能烧坏我们的 ADC，甚至是整个主控芯片。**

33.5 单通道 ADC 过采样（26 位分辨率）实验

本实验我们来学习单通道 ADC 过采样（26 位分辨率）实验。本实验使用常规转换组单通道的单次转换模式，并且通过软件触发，即通过对 ADCx_CR 寄存器的 ADSTART 位写 1 启动转换。下面先带大家来了解本实验要配置的寄存器。

33.5.1 ADC 寄存器

本实验很多配置和单通道 ADC 采集实验是一样的，下面只介绍 ADCx_CFGR2 寄存器。

- **ADCx 配置寄存器 2（ADCx_CFGR2）**

ADCx 配置寄存器 2 描述如图 33.5.1.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LSHIFT[3:0]				Res.	Res.	OSR[9:0]									
rw	rw	rw	rw			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	RSHIFT4	RSHIFT3	RSHIFT2	RSHIFT1	ROVSM	TROVS	OVSS[3:0]				Res.	Res.	Res.	JOVSE	ROVSE
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw				rw	rw

位 25:16 **OSR[9:0]**：过采样率 (Oversampling ratio)
此位域由软件置 1 和清零，用于定义过采样率。
0：1x（不进行过采样）
1：2x
2：3x
...
1023：1024x
注： 仅当 ADSTART=0 时（这可确保当前未进行任何转换），才允许通过软件对此位执行写操作。

位 10 **ROVSM**：常规过采样模式 (Regular Oversampling mode)
此位由软件置 1 和清零，用于选择常规过采样模式。
0：连续模式：如果触发注入转换，过采样会暂时停止，并会在注入序列完成后继续（注入序列过程中会保留过采样缓冲区）
1：恢复模式：如果触发注入转换，当前过采样会中止，并会在注入序列完成后从头开始进行过采样（注入序列开始后，过采样缓冲区会清零）
注： 仅当 ADSTART=0 时（这可确保当前未进行任何转换），才允许通过软件对此位执行写操作。

位 9 **TROVS**：已触发常规过采样 (Triggered Regular Oversampling)
此位由软件置 1 和清零，以使能已触发过采样
0：会在触发后连续完成某一通道的所有过采样转换
1：某一通道的每个过采样转换都需要重新触发
注： 仅当 ADSTART=0 时（这可确保当前未进行任何转换），才允许通过软件对此位执行写操作。

位 8:5 **OVSS[3:0]**：过采样右移 (Oversampling right shift)
此位域由软件置 1 和清零，用于定义应用到原始过采样结果的右移位数。
0000：不右移
0001：右移 1 位
0010：右移 2 位
0011：右移 3 位
0100：右移 4 位
0101：右移 5 位
0110：右移 6 位
0111：右移 7 位
1000：右移 8 位
1001：右移 9 位
1010：右移 10 位
1011：右移 11 位
其他代码保留
注： 仅当 ADSTART=0 时（这可确保当前未进行任何转换），才允许通过软件对这些位执行写操作。

位 0 **ROVSE**：常规过采样使能 (Regular Oversampling Enable)
此位由软件置 1 和清零，用于使能常规过采样。
0：禁止常规过采样
1：使能常规过采样
注： 仅当 ADSTART=0 且 JADSTART=0 时（这可确保当前未进行任何转换），才允许通过软件对此位执行写操作

图 33.5.1.1 ADCx_CFGR2 寄存器

OSR[9:0]位用于设置 ADC 的过采样率。OSR[9:0]=0~1023，表示 1x~1024x 过采样。本实验使用过采样，并且是 26 位分辨率，所以设置 OSR[9:0] = 1023。

ROVSM 位用于设置常规过采样模式，默认为 0 即可，即连续模式。

TROVS 位用于设置已触发常规过采样，默认为 0 即可，即会在触发后连续完成某一通道的所有过采样转换。

OVSS[3:0]位用于设置过采样结果右移位数。
ROVSE 位是常规过采样使能位，置 1 使能常规过采样。

33.5.2 硬件设计

1. 例程功能

ADC1 过采样(26 位分辨率)采集通道 19（PA5）上面的电压，在 LCD 模块上面显示对应的 ADC 转换值以及换算成电压后的电压值。可以使用杜邦线连接 PA5 到你想测量的电压源（0~3.3V），然后通过 TFTLCD 显示的电压值。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 : LED0 - PB1
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）ADC1 ： 通道 19 - PA5

3. 原理图

ADC 属于 STM32H743 内部资源，实际上我们只需要软件设置就可以正常工作，另外还需要将待测量的电压源连接到 ADC 通道上，以便 ADC 测量。本实验，我们通过 ADC1 过采样(26 位分辨率)通道 19（PA5）来采集外部电压值，开发板有一个电位器，可调节的电压范围是：0~3.3V，可以通过杜邦线将 PA5 与电位器连接，从而测量电位器的电压。

32.5.3 程序设计

32.5.3.1 ADC 的 HAL 库驱动

本实验用到的 ADC 的 HAL 库 API 函数前面都介绍过，具体调用情况请看程序解析部分。

单通道 ADC 过采样（26 位分辨率）配置步骤

1）开启 ADCx 和 ADC 通道对应的 IO 时钟，并配置该 IO 为模拟功能

首先开启 ADCx 的时钟，然后配置 GPIO 为模拟模式。本实验我们默认用到 ADC1 通道 19，对应 IO 是 PA5，它们的时钟开启方法如下：

```
__HAL_RCC_ADC12_CLK_ENABLE ();            /* 使能 ADC1 & ADC2 时钟 */
__HAL_RCC_GPIOA_CLK_ENABLE();             /* 开启 GPIOA 时钟 */
```

2）初始化 ADCx，配置其工作参数

通过 HAL_ADC_Init 函数来设置 ADCx 时钟分频系数、分辨率、模式、扫描方式、对齐方式、开启过采样等信息。

注意：该函数会调用：HAL_ADC_MspInit 回调函数来存放 ADC 及 GPIO 时钟使能、GPIO 初始化等代码。我们也可以不存放在这个函数里，本实验就没用到这个 MSP 回调函数。

3）配置 ADC 通道并启动 AD 转换器

在 HAL 库中，通过 HAL_ADC_ConfigChannel 函数来设置配置 ADC 的通道，根据需求设置通道、序列、采样时间和校准配置单端输入模式或差分输入模式等。

配置好 ADC 通道之后，通过 HAL_ADC_Start 函数启动 AD 转换器。

4）读取 ADC 值

这里选择查询方式读取，在读取 ADC 值之前需要调用 HAL_ADC_PollForConversion 等待上一次转换结束。然后就可以通过 HAL_ADC_GetValue 来读取 ADC 值。

33.5.3.2 程序流程图

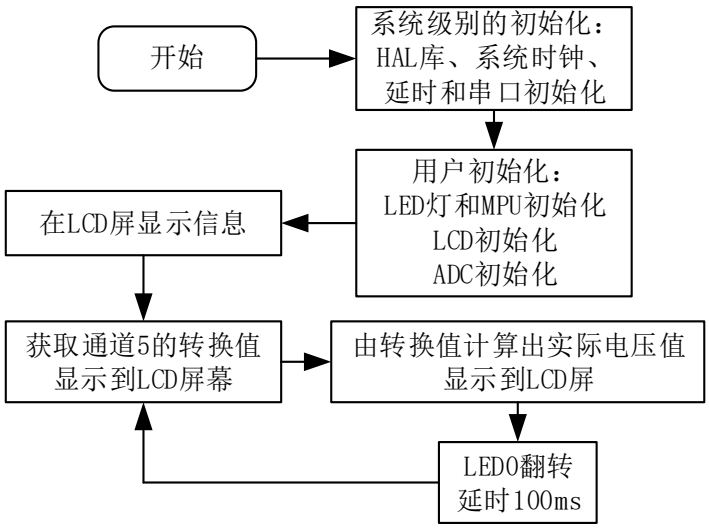


图 33.5.3.2.1 单通道 ADC 过采样（26 位分辨率）实验程序流程图

33.5.3.3 程序解析

adc.h 文件只是添加了函数声明，下面来介绍 adc.c 文件的 adc_oversample_init 函数。

```
/**
 * @brief   ADC 过采样 初始化函数
 * @note    本函数可以控制 ADC 过采样范围从 1x ~ 1024x, 得到最高 26 位分辨率的 AD 转换结果
 * @param   osr : 过采样倍率, 0 ~ 1023, 表示:1x ~ 1024x 过采样倍率
 * @param   ovss: 过采样右移位数, 0~11, 表示右移 0 位~11 位.
 * @note    过采样后, ADC 的转换时间相应的会慢 osr 倍.
 * @retval  无
 */
void adc_oversample_init(uint32_t osr, uint32_t ovss)
{
    GPIO_InitTypeDef gpio_init_struct;

    ADC_ADCX_CHY_GPIO_CLK_ENABLE();                 /* 开启 ADC 通道 IO 引脚时钟 */
    ADC_ADCX_CHY_CLK_ENABLE();                      /* 使能 ADC1/2 时钟 */
    __HAL_RCC_ADC_CONFIG(RCC_ADCCLKSOURCE_CLKP);    /* ADC 外设时钟选择 */

    gpio_init_struct.Pin = ADC_ADCX_CHY_GPIO_PIN; /* ADC 通道 IO 引脚 */
    gpio_init_struct.Mode = GPIO_MODE_ANALOG;     /* 模拟 */
    HAL_GPIO_Init(ADC_ADCX_CHY_GPIO_PORT, &gpio_init_struct);

    g_adc_handle.Instance = ADC_ADCX;               /* 选择哪个 ADC */
    /* 输入时钟 2 分频,即 adc_ker_ck=per_ck/2=32Mhz */
    g_adc_handle.Init.ClockPrescaler = ADC_CLOCK_ASYNC_DIV2;
    g_adc_handle.Init.Resolution = ADC_RESOLUTION_16B;    /* 16 位模式 */
    g_adc_handle.Init.ScanConvMode = ADC_SCAN_DISABLE;    /* 非扫描模式 */
    g_adc_handle.Init.EOCSelection = ADC_EOC_SINGLE_CONV;/* 关闭 EOC 中断 */
    g_adc_handle.Init.LowPowerAutoWait = DISABLE;         /* 自动低功耗关闭 */
    g_adc_handle.Init.ContinuousConvMode = DISABLE;       /* 关闭连续转换 */
    g_adc_handle.Init.NbrOfConversion = 1;/* 赋值范围是 1~16，本实验用到 1 个通道 */
    /* 禁止常规转换组不连续采样模式 */
    g_adc_handle.Init.DiscontinuousConvMode = DISABLE;
    /* 配置不连续采样模式的通道数，禁止常规转换组不连续采样模式后，此参数忽略 */
    g_adc_handle.Init.NbrOfDiscConversion = 0;
    g_adc_handle.Init.ExternalTrigConv = ADC_SOFTWARE_START;     /* 软件触发 */
    /* 采用软件触发的话，此位忽略 */
```

```
    g_adc_handle.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
    /* 常规通道的数据仅仅保存在 DR 寄存器里面 */
    g_adc_handle.Init.ConversionDataManagement = ADC_CONVERSIONDATA_DR;
    /* 有新的数据后直接覆盖掉旧数据 */
    g_adc_handle.Init.Overrun = ADC_OVR_DATA_OVERWRITTEN;
    /* 设置 ADC 转换结果的左移位数 */
    g_adc_handle.Init.LeftBitShift = ADC_LEFTBITSHIFT_NONE;
    g_adc_handle.Init.OversamplingMode = ENABLE;         /* 开启过采样 */
    g_adc_handle.Init.Oversampling.Ratio = osr;          /* 设置 osr+1 倍过采样 */
    g_adc_handle.Init.Oversampling.RightBitShift = ovss;/* 数据右移 ovss  bit */
    g_adc_handle.Init.Oversampling.TriggeredMode =
       ADC_TRIGGEREDMODE_SINGLE_TRIGGER;  /* 会在触发后连续完成通道的所有过采样转换 */
    g_adc_handle.Init.Oversampling.OversamplingStopReset =
       ADC_REGOVERSAMPLING_CONTINUED_MODE;/* ROVSE=1, 使能常规过采样 */
    HAL_ADC_Init(&g_adc_handle);           /* 初始化 */

    HAL_ADCEx_Calibration_Start(&g_adc_handle, ADC_CALIB_OFFSET,
                      ADC_SINGLE_ENDED);  /* ADC 校准 */
}
```

adc_oversample_init 函数大概可以分为下面几个部分的功能：

第一部分使能 ADC 和 GPIO 的时钟。

第二部分选择 ADC 的时钟源为 per_ck，per_ck 默认选择 hsi_ker_ck 作为时钟源，即 ADC 的时钟源为 64MHZ 的 hsi_ker_ck。

第三部分是设置 ADC 采集通道对应 IO 引脚工作模式。

第四部分初始化 ADC，并校准 ADC。在这里，开启过采样、还配置过采样率等。

最后在 main.c 里面编写如下代码：

```
int main(void)
{
    uint32_t adcx;
    float temp;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */

    /* 初始化 ADC, 1024x 过采样, 不移位
     * 26 位 ADC 分辨率最大值为:67108864, 实际上由于分辨率太高 ,低位值已经不准确
     * 一般我们可以设置 ovss=4, 缩小 16 倍, 即 22 位分辨率, 低位值会相对稳定一些.
     * 这里我们为了演示 26 位过采样 ADC 转换效果, 把分辨率调到最大, 26 位,并且不移位.
     */
    adc_oversample_init(1024 - 1, ADC_RIGHTBITSHIFT_NONE);

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "ADC OverSample TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    lcd_show_string(30, 110, 200, 16, 16, "ADC1_CH19_VAL:", BLUE);
    /* 先在固定位置显示小数点 */
    lcd_show_string(30, 130, 200, 16, 16, "ADC1_CH19_VOL:0.000V", BLUE);

    while (1)
    {
        /* 获取通道 19 的转换值，10 次取平均 */
        adcx = adc_get_result_average(ADC_ADCX_CHY, 10);
```

```
        lcd_show_xnum(142, 110, adcx, 8, 16,0,BLUE);/* 显示 ADC 采样后的原始值 */
        /* 获取计算后的带小数的实际电压值，比如 3.1111 */
        temp = (float)adcx * (3.3 / 67108864);
        adcx = temp;        /* 赋值整数部分给 adcx 变量，因为 adcx 为整形 */
        /* 显示电压值的整数部分，3.1111 的话，这里就是显示 3 */
        lcd_show_xnum(142, 130, adcx, 1, 16, 0, BLUE);

        temp -= adcx;/* 把已经显示的整数部分去掉，留下小数部分，比如 3.1111-3=0.1111 */
        temp *= 1000;/*小数部分乘以 1000，如：0.1111 就转换为 111.1，相当于保留三位小数*/
        /* 显示小数部分（前面转换为了整形显示），这里显示的就是 111. */
        lcd_show_xnum(158, 130, temp, 3, 16, 0X80, BLUE);

        LED0_TOGGLE();
        delay_ms(200);
    }
}
```

此部分代码，我们在 TFTLCD 模块上显示一些提示信息后，将每隔 100ms 读取一次 ADC 通道 5 的值，并显示读到的 ADC 值（数字量），以及其转换成模拟量后的电压值。同时控制 LED0 闪烁，以提示程序正在运行。

33.5.4 下载验证

下载代码后，LED0 闪烁，提示程序运行。可以看到 LCD 显示如图 33.5.4.1 所示：

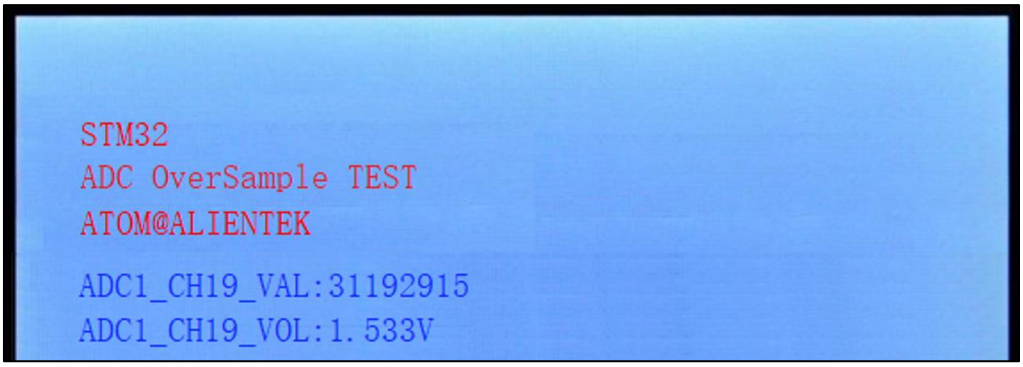


图 33.5.4.1 单通道 ADC 过采样（26 位分辨率）实验测试图

上图中，我们使用杜邦线将 P10 的 ADC 和 RV1 连接，使得 PA5 连接到电位器上，测试的是电位器的电压，并可以通过螺丝刀调节电位器改变电压值，范围：0~3.3V。LED0 闪烁，提示程序运行。

大家也可以用杜邦线将 ADC 排针接到其它待测量的电压点，看看测量到的电压值是否准确？**但是要注意：一定要保证测试点的电压在 0~3.3V 的电压范围，否则可能烧坏我们的 ADC，甚至是整个主控芯片。**

第三十四章 内部温度传感器实验

本章，我们将介绍 STM32H743 的内部温度传感器并使用它来读取温度值，然后在 LCD 模块上显示出来。

本章分为如下几个小节：

34.1 内部温度传感器简介
34.2 硬件设计
34.3 程序设计
34.4 下载验证

34.1 内部温度传感器简介

STM32H743 有一个内部的温度传感器，可以用来测量 CPU 及周围的温度(TA)。对于 STM32H7 系列来说，该温度传感器在内部和 ADC3_INP18 输入通道相连接，此通道把传感器输出的电压转换成数字值。 STM32H743 的内部温度传感器支持的温度范围为：-40~125 度。精度为±3℃左右。

STM32H743 内部温度传感器的使用很简单，只要设置一下内部 ADC，并激活其内部温度传感器通道就差不多了。关于 ADC 的设置，我们在上一章已经进行了详细的介绍，这里就不再多说。接下来我们介绍一下和温度传感器设置相关的两个地方。

第一个地方，我们要使用 STM32H743 的内部温度传感器，必须先激活 ADC 的内部通道，这里通过 ADC3_COMMON_CCR 的 VSENSEEN 位（bit23）设置。设置该位为 1 则启用内部温度传感器。

第二个地方，STM32H743 的内部温度传感器固定的连接在 ADC3 的通道 18 上，所以，我们在设置好 ADC3 之后只要读取通道 18 的值，就是温度传感器返回来的电压值了。根据这个值，我们就可以计算出当前温度。计算公式如下：

$$T(℃)=\frac{70}{TS_CAL2-TS_CAL1}\times(TS_DATA - TS_CAL1)+30$$

上式中：

TS_CAL1 是温度传感器在 30℃时的校准值，固定保存在芯片内部的：0X1FF1 E820 ~ 0X1FF1 E821 这两个地址（16 位）。

TS_CAL2 是温度传感器在 110℃时的校准值，固定保存在芯片内部的：0X1FF1 E840 ~ 0X1FF1 E841 这两个地址（16 位）。

TS_DATA：ADC3 通道 18 读取到的当前温度传感器转换值。

利用以上公式，我们就可以方便的计算出当前温度传感器的温度了。

34.2 硬件设计

1. 例程功能

通过 ADC3 的通道 18 读取 STM32H7 内部温度传感器的电压值，并将其转换为温度值，显示在 TFTLCD 屏上。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 : LED0 - PB1
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）ADC3 通道 18
5）内部温度传感器

34.3 程序设计

34.3.1 ADC 的 HAL 库驱动

本实验用到的 ADC 的 HAL 库 API 函数前面都介绍过，具体调用情况请看程序解析部分。下面介绍读取内部温度传感器 ADC 值的配置步骤。

读取内部温度传感器 ADC 值配置步骤

1）开启 ADC 时钟

通过__HAL_RCC_ADC3_CLK_ENABLE 函数开启 ADC3 的时钟。

2）设置 ADC3，开启内部温度传感器

调用 HAL_ADC_Init 函数来设置 ADC3 时钟分频系数、分辨率、模式、扫描方式、对齐方式等信息。

注意：该函数会调用：HAL_ADC_MspInit 回调函数来完成对 ADC 底层的初始化，包括：ADC3 时钟使能、ADC3 时钟源的选择等。

3）配置 ADC 通道并启动 AD 转换器

调用 HAL_ADC_ConfigChannel()函数配置 ADC3 通道 18，根据需求设置通道、序列、采样时间和校准配置单端输入模式或差分输入模式等。然后通过 HAL_ADC_Start 函数启动 AD 转换器。

4）读取 ADC 值，计算温度

这里选择查询方式读取，在读取 ADC 值之前需要调用 HAL_ADC_PollForConversion 等待上一次转换结束。然后就可以通过 HAL_ADC_GetValue 来读取 ADC 值。最后根据上面介绍的公式计算出温度传感器的温度值。

34.3.2 程序流程图

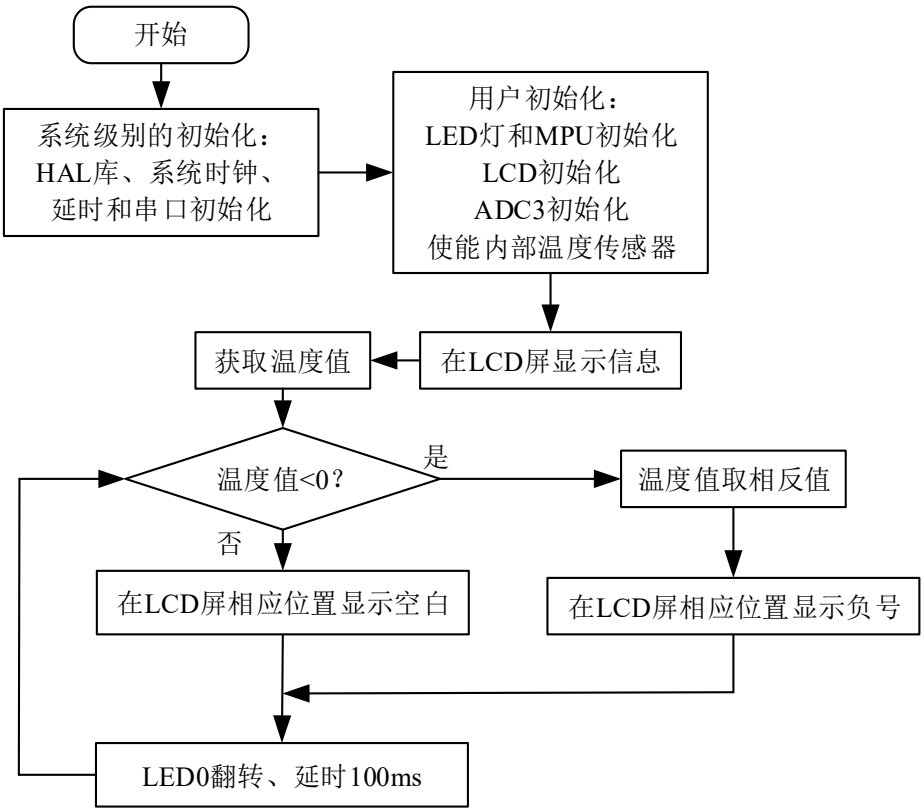


图 34.3.2.1 内部温度传感器实验程序流程图

34.3.3 程序解析

1. adc3 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。ADC3 驱动源码包括两个文件：adc3.c 和 adc3.h。

adc3.h 头文件只有一个宏定义和一些函数的声明，该宏定义如下：

```
#define ADC3_TEMPSENSOR_CHX                ADC_CHANNEL_TEMPSENSOR
```

ADC_CHANNEL_TEMPSENSOR 就是 ADC3 通道 18 连接内部温度传感器的通道 18 宏定义。我们在定义为 ADC3_TEMPSENSOR_CHX，可以让大家更容易理解这个宏定义的含义。

下面我们直接介绍 adc3.c 的程序，首先是 ADC3 初始化函数，其定义如下：

```
/**
 * @brief     ADC3 初始化函数
 * @note      本函数专用于支持 ADC3, 和 ADC1/2 区分开来, 方便使用
 *            我们使用 16 位精度, ADC 采样时钟=32M, 转换时间为:采样周期 + 8.5 个 ADC 周期
 *            设置最大采样周期: 810.5, 则转换时间 = 819 个 ADC 周期 = 25.6us
 * @param     无
 * @retval    无
 */
void adc3_init(void)
{
    g_adc3_handle.Instance = ADC3;                      /* 选择哪个 ADC */
    /* 输入时钟 2 分频,即 adc_ker_ck=per_ck/2=32Mhz */
    g_adc3_handle.Init.ClockPrescaler = ADC_CLOCK_ASYNC_DIV2;
    g_adc3_handle.Init.Resolution = ADC_RESOLUTION_16B;       /* 16 位模式  */
    g_adc3_handle.Init.ScanConvMode = ADC_SCAN_DISABLE;       /* 非扫描模式 */
    g_adc3_handle.Init.EOCSelection = ADC_EOC_SINGLE_CONV;    /* 关闭 EOC 中断 */
    g_adc3_handle.Init.LowPowerAutoWait = DISABLE;            /* 自动低功耗关闭 */
    g_adc3_handle.Init.ContinuousConvMode = DISABLE;          /* 关闭连续转换模式 */
    g_adc3_handle.Init.NbrOfConversion = 1;/* 赋值范围是 1~16，本实验用到 1 个通道 */
    /* 禁止常规转换组不连续采样模式 */
    g_adc3_handle.Init.DiscontinuousConvMode = DISABLE;
    /* 配置不连续采样模式的通道数，禁止常规转换组不连续采样模式后，此参数忽略 */
    g_adc3_handle.Init.NbrOfDiscConversion = 0;
    g_adc3_handle.Init.ExternalTrigConv = ADC_SOFTWARE_START;    /* 软件触发 */
    /* 采用软件触发的话，此位忽略 */
    g_adc3_handle.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
    /* 常规通道的数据仅仅保存在 DR 寄存器里面 */
    g_adc3_handle.Init.ConversionDataManagement = ADC_CONVERSIONDATA_DR;
    /* 有新的数据后直接覆盖掉旧数据 */
    g_adc3_handle.Init.Overrun = ADC_OVR_DATA_OVERWRITTEN;
    /* 设置 ADC 转换结果的左移位数 */
    g_adc3_handle.Init.LeftBitShift = ADC_LEFTBITSHIFT_NONE;
    g_adc3_handle.Init.OversamplingMode = DISABLE;        /* 关闭过采样 */
    HAL_ADC_Init(&g_adc3_handle);                         /* 初始化 */

    HAL_ADCEx_Calibration_Start(&g_adc3_handle, ADC_CALIB_OFFSET,
                                ADC_SINGLE_ENDED); /* ADC 校准 */
}
```

该函数主要调用了两个 HAL 库函数，HAL_ADC_Init 函数配置了选择哪个 ADC、数据对齐方式、是否使用扫描模式等参数，HAL_ADCEx_Calibration_Start 函数用于校准 ADC。另外 HAL_ADC_Init 函数会调用它的 MSP 回调函数 HAL_ADC_MspInit，该函数用来存放使能 ADC 和配置选择 ADC 时钟源等代码，其定义如下：

```
/**
 * @brief       ADC 底层驱动，引脚配置，时钟使能
 * @note        此函数会被 HAL_ADC_Init()调用
 * @param       hadc:ADC 句柄
 * @retval      无
```

```
 */
void HAL_ADC_MspInit(ADC_HandleTypeDef* hadc)
{
    RCC_PeriphCLKInitTypeDef rcc_periph_clk_struct = {0};
    __HAL_RCC_ADC3_CLK_ENABLE();                /* 使能 ADC3 时钟 */
    rcc_periph_clk_struct.PeriphClockSelection = RCC_PERIPHCLK_ADC;
    rcc_periph_clk_struct.AdcClockSelection = RCC_ADCCLKSOURCE_CLKP;
    HAL_RCCEx_PeriphCLKConfig(&rcc_periph_clk_struct);
}
```

下面是获得 ADC 转换后的结果函数，其定义如下：

```
/**
 * @brief       获得 ADC 转换后的结果
 * @param       hadc:ADC 句柄
 * @param       ch: 通道号, ADC_CHANNEL_0~ADC_CHANNEL_19
 * @retval      转换结果
 */
uint32_t adc3_get_result(ADC_HandleTypeDef adc_handle, uint32_t ch)
{
    ADC_ChannelConfTypeDef adc_ch_conf = {0};

    adc_ch_conf.Channel = ch;                                   /* 通道 */
    adc_ch_conf.Rank = ADC_REGULAR_RANK_1;                      /* 序列 */
    adc_ch_conf.SamplingTime = ADC_SAMPLETIME_810CYCLES_5;/* 采样时间 */
    adc_ch_conf.SingleDiff = ADC_SINGLE_ENDED;                  /* 单边采集 */
    adc_ch_conf.OffsetNumber = ADC_OFFSET_NONE;                 /* 不使用偏移量的通道 */
    adc_ch_conf.Offset=0;                                       /* 偏移量为 0 */
    HAL_ADC_ConfigChannel(&adc_handle, &adc_ch_conf);           /* 通道配置 */
    HAL_ADC_Start(&adc_handle);                                 /* 开启 ADC */
    HAL_ADC_PollForConversion(&adc_handle, 10);                 /* 轮询转换 */
    return HAL_ADC_GetValue(&adc_handle);    /* 返回最近一次 ADC 规则组的转换结果 */
}
```

该函数先调用 HAL_ADC_ConfigChannel 函数设置通道的转换序列和采样时间等，再调用 HAL_ADC_Start 函数开启 ADC，接着调用 HAL_ADC_PollForConversion 函数等待转换完成，最后调用 HAL_ADC_GetValue 函数获取转换后的当前结果。

下面介绍的是获取 ADC 某通道的转换多次后的平均值函数，函数定义如下：

```
/**
 * @brief       获取通道 ch 的转换值，取 times 次,然后平均
 * @param       adc_handle : ADC 句柄
 * @param       ch       : 通道号, 0~19
 * @param       times    : 获取次数
 * @retval      通道 ch 的 times 次转换结果平均值
 */
uint32_t adc3_get_result_average(ADC_HandleTypeDef adc_handle,
                                 uint32_t ch, uint8_t times)
{
    uint32_t temp_val = 0;
    uint8_t t;
    for (t = 0; t < times; t++)    /* 获取 times 次数据 */
    {
        temp_val += adc3_get_result(adc_handle, ch);
        delay_ms(5);
    }
    return temp_val / times;       /* 返回平均值 */
}
```

该函数用于多次获取 ADC 值，累加再取平均值，以提高准确度。

最后一个函数是获取内部温度传感器温度值函数，函数定义如下：

```
/**
 * @brief       获取内部温度传感器温度值
 * @param       无
 * @retval      温度值(扩大了 100 倍,单位:℃.)
```

```
 */
short adc3_get_temperature(void)
{
    uint32_t adcx;
    short result;
    double temperature;
    float temp = 0;
    uint16_t ts_cal1, ts_cal2;

    ts_cal1 = *(volatile uint16_t *)(0X1FF1E820);        /* 获取 TS_CAL1 */
    ts_cal2 = *(volatile uint16_t *)(0X1FF1E840);        /* 获取 TS_CAL2 */
    temp = (float)((110.0 - 30.0) / (ts_cal2 - ts_cal1));/* 获取比例因子 */

    /* 读取内部温度传感器通道,10 次取平均 */
    adcx = adc3_get_result_average(adc3_handle, ADC3_TEMPSENSOR_CHX, 10);
    temperature = (float)(temp * (adcx - ts_cal1) + 30); /* 计算温度 */

    result = temperature *= 100;      /* 扩大 100 倍. */
    return result;
}
```

该函数根据 ADC3 通道 18 采集温度传感器返回来的电压值代入 34.1 小节介绍的公式来计算出当前温度值。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    short temp;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    adc3_init();                            /* 初始化 ADC3(使能内部温度传感器) */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "Temperature TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 120, 200, 16, 16, "TEMPERATE: 00.00C", BLUE);

    while (1)
    {
      temp = adc3_get_temperature(); /* 得到温度值 */

      if (temp < 0)
      {
         temp = -temp;
         lcd_show_string(30 + 10 * 8, 120, 16, 16, 16, "-", BLUE);/* 显示负号 */
      }
      else
      {
         lcd_show_string(30 + 10 * 8, 120, 16, 16, 16, " ", BLUE); /* 无符号 */
      }
       /* 显示整数部分 */
       lcd_show_xnum(30 + 11 * 8, 120, temp / 100, 2, 16, 0, BLUE);
       /* 显示小数部分 */
```

```
        lcd_show_xnum(30 + 14 * 8, 120, temp % 100, 2, 16, 0X80, BLUE);

        LED0_TOGGLE();  /* LED0 闪烁,提示程序运行 */
        delay_ms(250);
    }
}
```

该部分的代码逻辑很简单，先是得到温度值，再根据温度值判断正负值，来显示温度符号，再显示整数和小数部分。

34.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 34.4.1 所示：

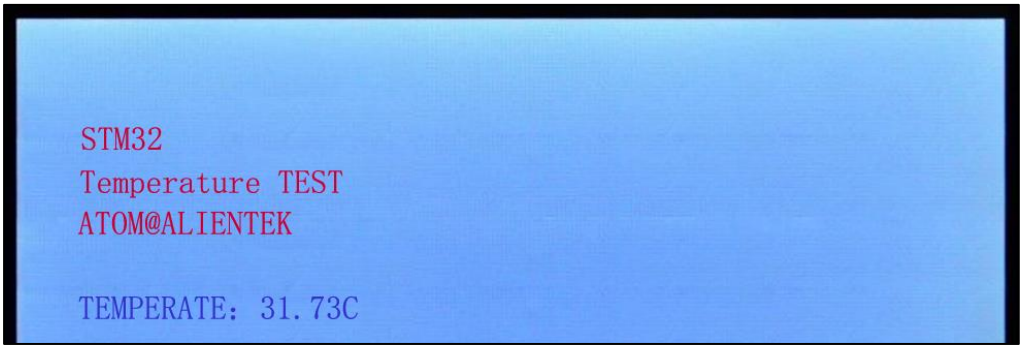


图 34.4.1 内部温度传感器实验测试图

大家可以看看你的温度值与实际是否相符合（因为芯片会发热，所以一般会比实际温度偏高）？

第三十五章 DAC 实验

本章，我们将介绍 STM32H743 的 DAC（Digital -to- analog converters，数模转换器）功能。我们通过三个实验来学习 DAC，分别是 DAC 输出实验、DAC 输出三角波实验和 DAC 输出正弦波实验。

本章分为如下几个小节：

35.1 DAC 简介
35.2 DAC 输出实验
35.3 DAC 输出三角波实验
35.4 DAC 输出正弦波实验

35.1 DAC 简介

STM32H743 的 DAC 模块（数字/模拟转换模块）是 12 位数字输入，电压输出型的 DAC。DAC 可以配置为 8 位或 12 位模式，也可以与 DMA 控制器配合使用。DAC 工作在 12 位模式时，数据可以设置成左对齐或右对齐。DAC 模块有 2 个输出通道，每个通道都有单独的转换器。在双 DAC 模式下，2 个通道可以独立地进行转换，也可以同时进行转换并同步地更新 2 个通道的输出。DAC 可以通过引脚输入参考电压 Vref+（通 ADC 共用）以获得更精确的转换结果。

STM32H743 的 DAC 模块主要特点有：

① 2 个 DAC 转换器：每个转换器对应 1 个输出通道
② 8 位或者 12 位单调输出
③ 12 位模式下数据左对齐或者右对齐
④同步更新功能
⑤ 噪声波形生成
⑥三角波形生成
⑦ 双 DAC 双通道同时或者分别转换
⑧每个通道都有 DMA 功能

DAC 通道框图如图 35.1.1 所示：

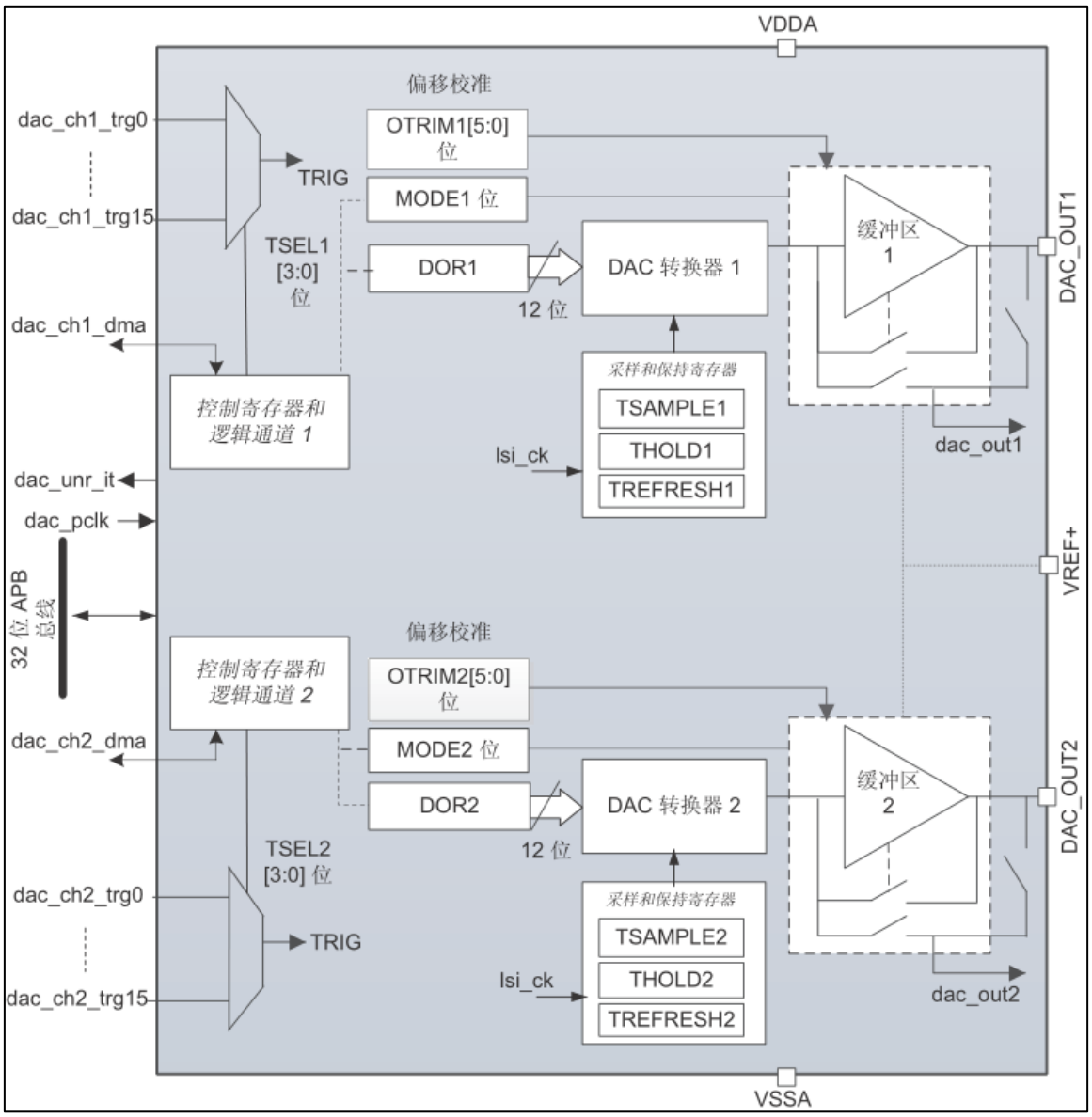


图 35.1.1 DAC 通道框图

图中 VDDA 和 VSSA 为 DAC 模块模拟部分的供电，而 VREF+则是 DAC 模块的参考电压。DAC_OUT1/2 就是 DAC 的两个输出通道了（对应 PA4 或者 PA5 引脚）。ADC 的这些输入/输出引脚信息如下表所示：

引脚名称	信号类型	说明
V_{REF+}	正模拟参考电压输入	DAC 高/正参考电压，$V_{REF+} \leq V_{DDAmax}$
VDDA	模拟电源输入	模拟电源
VSSA	模拟电源地输入	模拟电源地
DAC_OUTx	模拟输出信号	DAC 通道 x 模拟输出，x=1/2

表 35.1.1 DAC 输入/输出引脚

除了上表列出的输入/输出引脚，DAC 通道框图还有一些内部输入/输出信号，具体如下表所示：

内部信号名称	信号类型	说明
dac_ch1_dma	双向	DACx 通道 1 DMA 请求
dac_ch2_dma	双向	DACx 通道 2 DMA 请求
dac_ch1_trg[0:15]	输入	DACx 通道 1
dac_ch2_trg[0:15]	输入	DACx 通道 2

dac_unr_it	输出	DACx 下溢中断
dac_pclk	输入	DACx 外设时钟
dac_out1	模拟输出	片上外设的 DACx 通道 1 输出
dac_out2	模拟输出	片上外设的 DACx 通道 2 输出

表 35.1.2 DAC 内部输入/输出信号

注意：表中的 dac_pclk 即 DAC 的时钟源，DA 转换和寄存器访问都是靠这个时钟，该时钟来自 APB1，通过 sys_stm32_clock_init 函数配置之后，为 100MHz。

从图 35.1.1 可以看出，DAC 输出是受 DORx（x=1/2，下同）寄存器直接控制的，但是我们不能直接往 DORx 寄存器写入数据，而是通过 DHRx 间接的传给 DORx 寄存器，从而实现对 DAC 输出的控制。

前面我们提到，STM32H743 的 DAC 支持 8/12 位模式，8 位模式的时候是固定的右对齐的，而 12 位模式又可以设置左对齐/右对齐。DAC 单通道模式下的数据寄存器对齐方式，总共有 3 种情况，如下图所示：

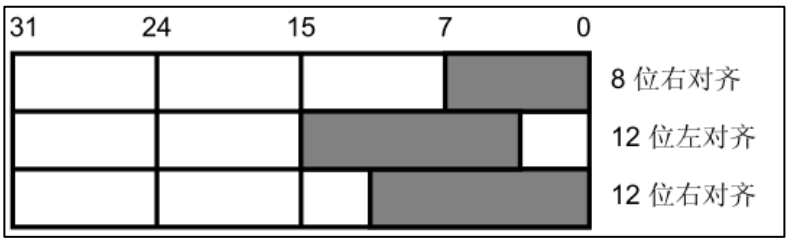


图 35.1.2 DAC 单通道模式下的数据寄存器对齐方式

① 8 位数据右对齐：用户将数据写入 DAC_DHR8Rx[7:0]位（实际存入 DHRx[11:4]位）。

② 12 位数据左对齐：用户将数据写入 DAC_DHR12Lx[15:4]位（实际存入 DHRx[11:0]位）。

③ 12 位数据右对齐：用户将数据写入 DAC_DHR12Rx[11:0]位（实际存入 DHRx[11:0]位）。

我们本章实验中使用的都是单通道模式下的 DAC 通道 1，采用 12 位右对齐格式，所以采用第③种情况。另外 DAC 还具有双通道转换功能。

对于 DAC 双通道（可用时），也有三种可能的方式，如下图所示：

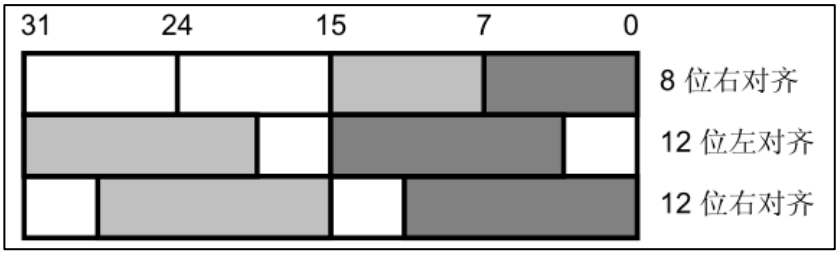


图 35.1.3 DAC 双通道模式下的数据寄存器对齐方式

①8 位数据右对齐：用户将 DAC 通道 1 的数据写入 DAC_DHR8RD[7:0]位（实际存入 DHR1 [11:4]位），将 DAC 通道 2 的数据写入 DAC_DHR8RD[15:8]位（实际存入 DHR2 [11:4]位）。

②12 位数据左对齐：用户将 DAC 通道 1 的数据写入 DAC_DHR12LD [15:4]位（实际存入 DHR1[11:0]位），将 DAC 通道 2 的数据写入 DAC_DHR12LD [31:20]位（实际存入 DHR2[11:0]位）。

③12 位数据右对齐：用户将 DAC 通道 1 的数据写入 DAC_DHR12RD [11:0]位（实际存入 DHR1[11:0]位），将 DAC 通道 2 的数据写入 DAC_DHR12RD [27:16]位（实际存入 DHR2[11:0]位）。

DAC 可以通过软件或者硬件触发转换，通过配置 TENx 控制位来决定。

如果没有选中硬件触发（寄存器 DAC_CR1 的 TENx 位置 0），存入寄存器 DAC_DHRx 的数据会在 1 个 APB1 时钟周期后自动传至寄存器 DAC_DORx。如果选中硬件触发（寄存器 DAC_CR1 的 TENx 位置 1），数据传输在触发发生以后 3 个 APB1 时钟周期后完成。 一旦数据从 DAC_DHRx 寄存器装入 DAC_DORx 寄存器，在经过时间 $t_{SETTLING}$ 之后，输出即有效，这段时间的长短依电源电压和模拟输出负载的不同会有所变化。我们可以从《STM32H743IIT6.pdf》

数据手册查到 $t_{SETTLING}$ 的典型值为 1.7us，最大是 2us，所以 DAC 的转换速度最快是 588K 左右。

不使用硬件触发（TEN=0），其转换的时间框图如图 35.1.4 所示：

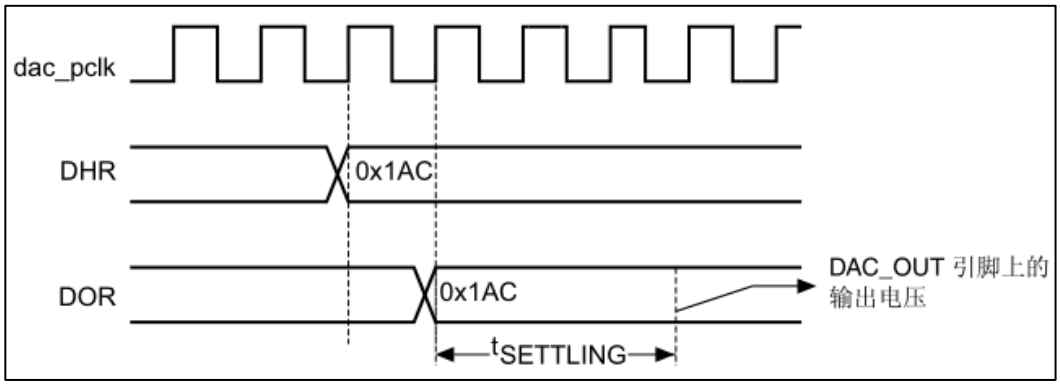


图 35.1.4 TEN=0 时 DAC 模块转换时间框图

当 DAC 的参考电压为 VREF+的时候，DAC 的输出电压是线性的从 0~VREF+，12 位模式下 DAC 输出电压与 VREF +以及 DORx 的计算公式如下：

DACx 输出电压 = VREF *（DORx/4096）

如果使用硬件触发（TEN=1），可通过外部事件（定时计数器、外部中断线）触发 DAC 转换。由 TSELx[3:0]控制位来决定选择 16 个触发事件中的一个来触发转换。这 16 个触发事件如下表所示：

源	类型	TSELx[3:0]
SWTRIG	软件控制位	0000
TIM1_TRGO	片上定时器的内部信号	0001
TIM2_TRGO	片上定时器的内部信号	0010
TIM4_TRGO	片上定时器的内部信号	0011
TIM5_TRGO	片上定时器的内部信号	0100
TIM6_TRGO	片上定时器的内部信号	0101
TIM7_TRGO	片上定时器的内部信号	0110
TIM8_TRGO	片上定时器的内部信号	0111
TIM15_TRGO	片上定时器的内部信号	1000
HRTIM1_DACTRG1	片上定时器的内部信号	1001
HRTIM1_DACTRG2	片上定时器的内部信号	1010
LPTIM1_OUT	片上定时器的内部信号	1011
LPTIM2_OUT	片上定时器的内部信号	1100
EXTI9	外部引脚	1101
保留	-	1110
保留	-	1111

表 35.1.3 DAC 触发选择

原表见《STM32H7xx 参考手册_V3（中文版）.pdf》第 923 页表 208。

每个 DAC 通道都有 DMA 功能，两个 DMA 通道分别用于处理两个 DAC 通道的 DMA 请求。如果 DMAENx 位置 1 时，如果发生外部触发（而不是软件触发），就会产生一个 DMA 请求，然后 DAC_DHRx 寄存器的数据被转移到 DAC_NORx 寄存器。

35.2 DAC 输出实验

本实验我们来学习 DAC 输出实验。

35.2.1 DAC 寄存器

下面，我们介绍要实现 DAC 的通道 1 输出，需要用到的一些 DAC 寄存器。

- **DACx 控制寄存器（DACx_CR）（x=1 或 2）**

DACx 控制寄存器描述如图 35.2.1.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	CEN2	DMAUDRIE2	DMAEN2	MAMP2[3:0]				WAVE2[1:0]		TSEL23	TSEL22	TSEL21	TSEL20	TEN2	EN2
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	CEN1	DMAUDRIE1	DMAEN1	MAMP1[3:0]				WAVE1[1:0]		TSEL13	TSEL12	TSEL11	TSEL10	TEN1	EN1
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 14 **CEN1**：DAC 通道 1 校准使能 (DAC Channel 1 calibration enable)
此位由软件置 1 和清零，用于使能/禁止 DAC 通道 1 校准，仅当 DACx_CR 中的位 EN1=0 时才可写入（仅当 DAC 通道已禁止时才可以进入/退出校准模式），否则会忽略写操作。
0：DAC 通道 1 处于正常运行模式
1：DAC 通道 1 处于校准模式

位 12 **DMAEN1**：DAC 通道 1 DMA 使能 (DAC channel1 DMA enable)
此位由软件置 1 和清零。
0：禁止 DAC 通道 1 DMA 模式
1：使能 DAC 通道 1 DMA 模式

位 7:6 **WAVE1[1:0]**：DAC 通道 1 噪声/三角波生成使能 (DAC channel1 noise/triangle wave generation enable)
这些位将由软件置 1 和清零。
00：禁止生成波
01：使能生成噪声波
1x：使能生成三角波
只在位 TEN1 = 1（使能 DAC 通道 1 触发）时使用。

位 5:2 **TSEL1[3:0]**：DAC 通道 1 触发器选择 (DAC channel1 trigger selection)
这些位用于选择 DAC 通道 1 的外部触发事件。
有关触发配置和映射的详细信息，请参见触发选择表。
注：　只在位 TEN1 = 1（使能 DAC 通道 1 触发）时使用。

位 1 **TEN1**：DAC 通道 1 触发使能 (DAC channel1 trigger enable)
此位由软件置 1 和清零，以使能/禁止 DAC 通道 1 触发。
0：禁止 DAC 通道 1 触发，写入 DACx_DHR1 寄存器的数据在一个 dac_pclk 时钟周期之后转移到 DACx_DOR1 寄存器
1：使能 DAC 通道 1 触发，DACx_DHR1 寄存器的数据在三个 dac_pclk 时钟周期之后转移到 DACx_DOR1 寄存器
注：　如果选择软件触发，DACx_DHR1 寄存器的内容只需一个 dac_pclk 时钟周期即可转移到 DACx_DOR1 寄存器。

位 0 **EN1**：DAC 通道 1 使能 (DAC channel1 enable)
此位由软件置 1 和清零，以使能/禁止 DAC 通道 1 。
0：禁止 DAC 通道 1
1：使能 DAC 通道 1

图 35.2.1.1 DACx_CR 寄存器

DACx_CR 寄存器的低 16 位用于控制通道 1，高 16 位用于控制通道 2，下面介绍本实验需要设置的一些位：

EN1 位用于使能/禁止 DAC 通道 1，本实验用到 DAC1 通道 1，所以该位 EN1 置 1。

TEN1 位用于 DAC 通道 1 的触发使能，本实验不使用硬件触发，所以该位置 0。写入 DHR1 的值会在 1 个 APB1 周期后传送到 DOR1，然后输出到 PA4 口上。

TSEL[3:0]位用于选择 DAC 通道 1 的触发方式，本实验使用软件触发，所以该位域置 0。具体的设置关系详见表 35.1.3 DAC 触发选择。

WAVE[1:0]位用于控制 DAC 通道 1 的噪声/波形输出功能，默认设置为 00，不使能噪声/波

形输出。

DMAEN1 位用于控制 DAC 通道 1 的 DMA 使能，本实验不使能，设置该位为 0 即可。

CEN1 位用于控制 DAC 通道 1 的输出缓冲校准使能，本实验不使用校准功能（默认有一个出厂校准值，我们使用默认的校准值即可），设置该位为 0 即可。

- **DACx 模式控制寄存器（DACx_ MCR）**

DACx 模式控制寄存器描述如图 35.2.1.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MODE2[2:0]		
													rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MODE1[2:0]		
													rw		

位 2:0 **MODE1[2:0]**：DAC 通道 1 模式 (DAC Channel 1 mode)

仅当 DAC 已禁止且不处于校准模式时（DACx_CR 寄存器中的位 EN1 = 0 且位 CEN1 = 0），才可写入这些位。如果 EN1=1 或 CEN1 =1，则会忽略写操作。

这些位可由软件置 1 和清零，用于选择 DAC 通道 1 模式。

– DAC 通道 1 处于正常模式

000：DAC 通道 1 连接到外部引脚且使能了缓冲器

001：DAC 通道 1 连接到外部引脚以及片上外设且使能了缓冲器

010：DAC 通道 1 连接到外部引脚且禁止了缓冲器

011：DAC 通道 1 连接到片上外设且禁止了缓冲器

– DAC 通道 1 处于采样和保持模式

100：DAC 通道 1 连接到外部引脚且使能了缓冲器

101：DAC 通道 1 连接到外部引脚以及片上外设且使能了缓冲器

110：DAC 通道 1 连接到外部引脚以及片上外设且禁止了缓冲器

111：DAC 通道 1 连接到片上外设且禁止了缓冲器

注：　仅可在 EN1 = 0 时修改该寄存器。

图 35.2.1.2 DACx_ MCR 寄存器

该寄存器我们只关心 MODE1[2:0]，这三个位用于设置 DAC 通道 1 的工作模式，本实验使用普通模式，且使用输出缓冲，设置 MODE1[2:0]=0 即可。MODE2[2:0] 设置通道 2 的工作模式，本实验没用到。

- **DACx 通道 1 12 位右对齐数据保持寄存器（DACx_ DHR12R1）**

DACx 通道 1 12 位右对齐数据保持寄存器描述如图 35.2.1.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	DACC1DHR[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:12 保留，必须保持复位值。

位 11:0 **DACC1DHR[11:0]**：DAC 通道 1 12 位右对齐数据 (DAC channel1 12-bit right-aligned data)

这些位通过软件写入，用于指定 DAC 通道 1 的 12 位数据。

图 35.2.1.3 DACx_ DHR12R1 寄存器

该寄存器用来设置 DAC 输出，通过写入 12 位数据到该寄存器，就可以在 DAC 输出通道 1（PA4）得到我们所要的结果。

35.2.2 硬件设计

1. 例程功能

使用 KEY1/KEY_UP 两个按键，控制 STM32 内部 DAC 的通道 1 输出电压大小，然后通过

ADC1 的通道 19 采集 DAC 输出的电压，在 LCD 模块上面显示 ADC 采集到的电压值以及 DAC 的设定输出电压值等信息。也可以通过 usmart 调用 dac_set_voltage 函数，来直接设置 DAC 输出电压。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 : LED0 - PB1
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）独立按键：KEY1 - PH2、WK_UP - PA0
5）ADC1：通道 19 - PA5
6）DAC1：通道 1 - PA4

3. 原理图

我们来看看原理图上 ADC1 通道 19（PA5）和 DAC1 通道 1（PA4）引出来的引脚，如下图所示：

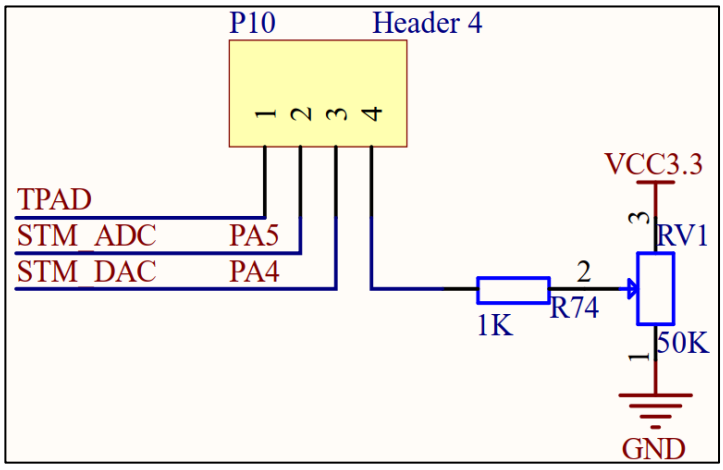


图 35.2.2.1 ADC 和 DAC 在开发板上的连接关系原理图

P10 是多功能端口，我们只需要通过跳线帽连接 P10 的 ADC 和 DAC，就可以使得 ADC1 通道 19（PA5）和 DAC1 通道 1（PA4）连接起来。对应的硬件连接如图 35.2.2.2 所示：

图 35.2.2.2 硬件连接示意图

35.2.3 程序设计

35.2.3.1 DAC 的 HAL 库驱动

DAC 在 HAL 库中的驱动代码在 stm32h7xx_hal_dac.c 和 stm32h7xx_hal_dac_ex.c 文件（及其头文件）中。

1. HAL_DAC_Init 函数

DAC 的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_DAC_Init(DAC_HandleTypeDef *hdac);
```

- **函数描述：**
  用于初始化 DAC。
- **函数形参：**
  形参 1 是 DAC_HandleTypeDef 结构体类型指针变量，其定义如下：

```
typedef struct
{
  DAC_TypeDef                 *Instance;         /* DAC 寄存器基地址 */
  __IO HAL_DAC_StateTypeDef   State;            /* DAC 工作状态 */
  HAL_LockTypeDef             Lock;             /* DAC 锁定对象 */
  DMA_HandleTypeDef           *DMA_Handle1;     /* 通道 1 的 DMA 处理句柄指针 */
  DMA_HandleTypeDef           *DMA_Handle2;     /* 通道 2 的 DMA 处理句柄指针 */
  __IO uint32_t                ErrorCode;       /* DAC 错误代码 */
} DAC_HandleTypeDef;
```

从该结构体看到该函数并没有设置任何 DAC 相关寄存器，即没有对 DAC 进行任何配置，它只是 HAL 库提供用来在软件上初始化 DAC，为后面 HAL 库操作 DAC 做好准备。

- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。
- **注意事项：**
  DAC 的 MSP 初始化函数 HAL_DAC_MspInit，该函数声明如下：

```
void HAL_DAC_MspInit(DAC_HandleTypeDef* hdac);
```

2. HAL_DAC_ConfigChannel 函数

DAC 的通道参数初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_DAC_ConfigChannel(DAC_HandleTypeDef *hdac,
                                        DAC_ChannelConfTypeDef *sConfig, uint32_t Channel);
```

- **函数描述：**
  该函数用来配置 DAC 通道的触发类型以及输出缓冲。
- **函数形参：**
  形参 1 是 DAC_HandleTypeDef 结构体类型指针变量。
  形参 2 是 DAC_ChannelConfTypeDef 结构体类型指针变量，其定义如下：

```
typedef struct
{
  uint32_t DAC_SampleAndHold;           /* 设置是否使能低功耗模式，即采样和保持模式 */
  uint32_t DAC_Trigger;                 /* DAC 触发源的选择 */
  uint32_t DAC_OutputBuffer;            /* 启用或者禁用 DAC 通道输出缓冲区 */
  uint32_t DAC_ConnectOnChipPeripheral;/* 指定 DAC 输出是否连接到片上外设 */
  uint32_t DAC_UserTrimming;          /* 设置 DAC 的校准方式，采用出厂模式还是用户模式 */
  uint32_t DAC_TrimmingValue;         /* 设置用户校准模式的偏移值 */
  DAC_SampleAndHoldConfTypeDef  DAC_SampleAndHoldConfig; /* 设置采样保持具体参数 */
} DAC_ChannelConfTypeDef;
```

形参 3 用于选择要配置的通道，可选择 DAC_CHANNEL_1 或者 DAC_CHANNEL_2。

- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

3. HAL_DAC_Start 函数

使能启动 DAC 转换通道函数，其声明如下：

```
HAL_StatusTypeDef HAL_DAC_Start(DAC_HandleTypeDef *hdac, uint32_t Channel);
```

- **函数描述：**
  使能启动 DAC 转换通道。
- **函数形参：**
  形参 1 是 DAC_HandleTypeDef 结构体类型指针变量。
  形参 2 用于选择要启动的通道，可选择 DAC_CHANNEL_1 或者 DAC_CHANNEL_2。
- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

4. HAL_DAC_SetValue 函数

DAC 的通道输出值函数，其声明如下：

```
HAL_StatusTypeDef HAL_DAC_SetValue(DAC_HandleTypeDef *hdac, uint32_t Channel,
                                   uint32_t Alignment, uint32_t Data);
```

- **函数描述：**
  配置 DAC 的通道输出值。
- **函数形参：**
  形参 1 是 DAC_HandleTypeDef 结构体类型指针变量。
  形参 2 用于选择要输出的通道，可选择 DAC_CHANNEL_1 或者 DAC_CHANNEL_2。
  形参 3 用于指定数据对齐方式。
  形参 4 设置要加载到选定数据保存寄存器中的数据。
- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

5. HAL_DAC_GetValue 函数

DAC 读取通道输出值函数，其声明如下：

```
uint32_t HAL_DAC_GetValue(DAC_HandleTypeDef *hdac, uint32_t Channel);
```

- **函数描述：**
  获取所选 DAC 通道的最后一个数据输出值。
- **函数形参：**
  形参 1 是 DAC_HandleTypeDef 结构体类型指针变量。
  形参 2 用于选择要读取的通道，可选择 DAC_CHANNEL_1 或者 DAC_CHANNEL_2。
- **函数返回值：**
  获取到的输出值。

DAC 输出配置步骤

1）开启 DACx 和 DAC 通道对应的 IO 时钟，并配置该 IO 为模拟功能

首先开启 DACx 的时钟，然后配置 GPIO 为模拟模式。本实验我们默认用到 DAC1 通道 1，对应 IO 是 PA4，它们的时钟开启方法如下：

```
__HAL_RCC_DAC12_CLK_ENABLE ();          /* 使能 DAC1 时钟 */
__HAL_RCC_GPIOA_CLK_ENABLE();           /* 开启 GPIOA 时钟 */
```

2）初始化 DACx

通过 HAL_DAC_Init 函数来设置需要初始化的 DAC。该函数并没有设置任何 DAC 相关寄存器，也就是说没有对 DAC 进行任何配置，它只是 HAL 库提供用来在软件上初始化 DAC。

注意：该函数会调用 HAL_DAC_MspInit 函数来存放 DAC 和对应通道的 IO 时钟使能和初始化 IO 等代码。

3）配置 DAC 通道并启动 DA 转换器

在 HAL 库中，通过 HAL_DAC_ConfigChannel 函数来配置 DAC 的通道，根据需求设置触发类型以及输出缓冲。

配置好 DAC 通道之后，通过 HAL_DAC_Start 函数启动 DA 转换器。

4）设置 DAC 的输出值

通过 HAL_DAC_SetValue 函数设置 DAC 的输出值。

35.2.3.2 程序流程图

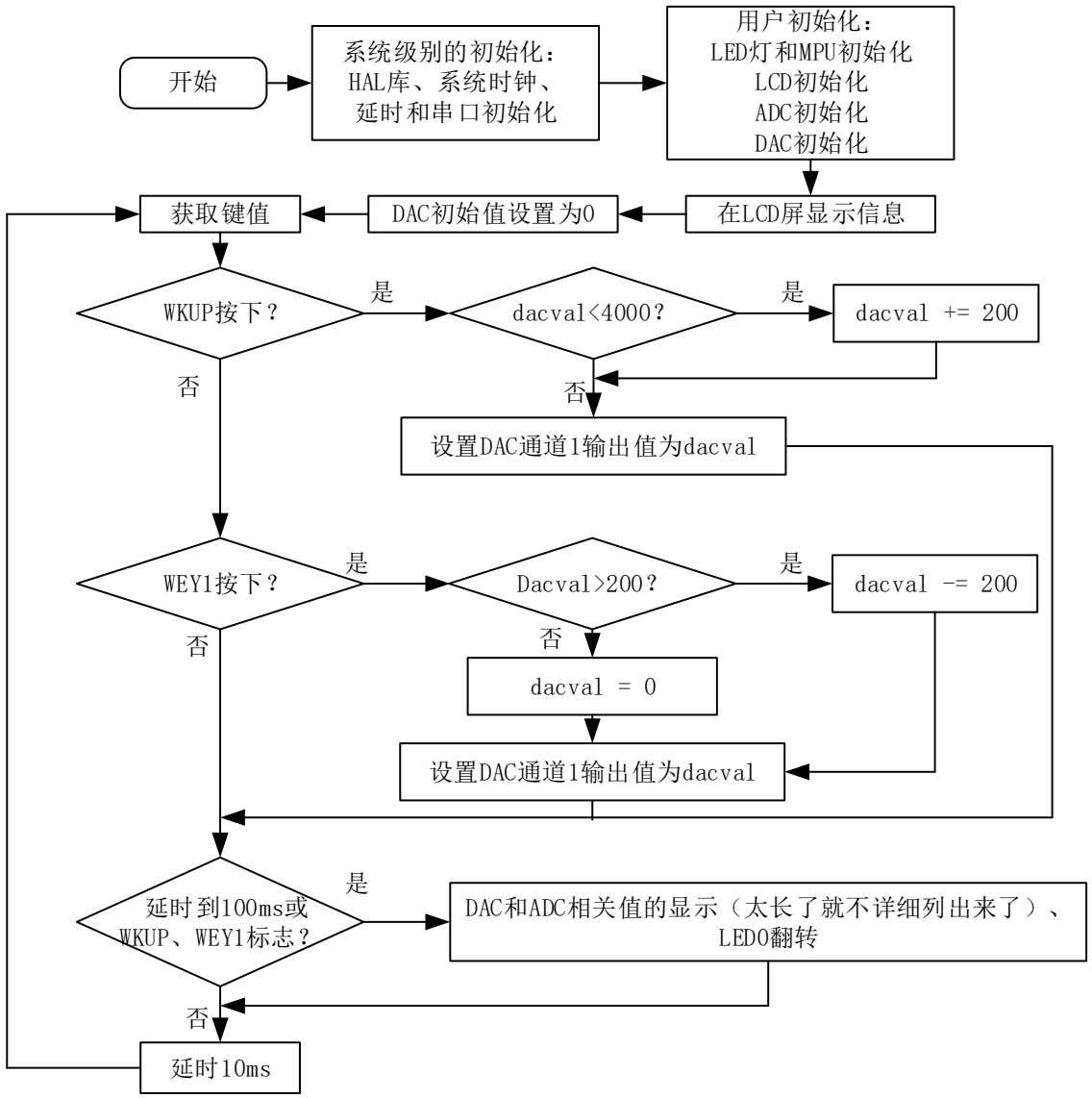


图 35.2.3.2.1 DAC 输出实验程序流程图

35.2.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。DAC 驱动源码包括两个文件：dac.c 和 dac.h。

dac.h 文件只有一些声明，下面直接开始介绍 dac.c 的程序，首先是 DAC 初始化函数。

```
/**
 * @brief   DAC 初始化函数
 * @note    本函数支持 DAC1_OUT1/2 通道初始化
 *     DAC 的输入时钟来自 APB1, 时钟频率=100Mhz=10ns
 *     DAC 在输出 buffer 关闭的时候, 输出建立时间: tSETTLING = 2us (H743 数据手册有写)
 *     因此 DAC 输出的最高速度约为:500Khz, 以 10 个点为一个周期, 最大能输出 50Khz 左右的波形
 *
 * @param       outx: 要初始化的通道. 1,通道 1; 2,通道 2
 * @retval      无
 */
void dac_init(uint8_t outx)
{
```

```
    DAC_ChannelConfTypeDef dac_ch_conf;      /* DAC 通道配置结构体 */
    GPIO_InitTypeDef gpio_init_struct;
    __HAL_RCC_DAC12_CLK_ENABLE(); /* 使能 DAC12 时钟，本芯片只有 DAC1 */
    __HAL_RCC_GPIOA_CLK_ENABLE(); /* 使能 DAC OUT1/2 的 IO 口时钟*/

   /* STM32 单片机，总是 PA4=DAC1_OUT1，PA5=DAC1_OUT2 */
    gpio_init_struct.Pin = (outx==1)? GPIO_PIN_4 : GPIO_PIN_5;
    gpio_init_struct.Mode = GPIO_MODE_ANALOG;       /* 模拟 */
    HAL_GPIO_Init(GPIOA, &gpio_init_struct);

    g_dac_handle.Instance = DAC1;                   /* DAC1 寄存器基地址 */
    HAL_DAC_Init(&g_dac_handle);                    /* 初始化 DAC */

    dac_ch_conf.DAC_Trigger = DAC_TRIGGER_NONE;     /* 不使用触发功能 */
    dac_ch_conf.DAC_OutputBuffer=DAC_OUTPUTBUFFER_DISABLE;/* DAC1 输出缓冲关闭 */
    switch(outx)
    {
        case 1 :
            /* DAC 通道 1 配置 */
            HAL_DAC_ConfigChannel(&g_dac_handle, &dac_ch_conf, DAC_CHANNEL_1);
            HAL_DAC_Start(&g_dac_handle, DAC_CHANNEL_1);  /* 开启 DAC 通道 2 */
            break;
        case 2 :
            /* DAC 通道 1 配置 */
            HAL_DAC_ConfigChannel(&g_dac_handle, &dac_ch_conf, DAC_CHANNEL_2);
            HAL_DAC_Start(&g_dac_handle, DAC_CHANNEL_2);  /* 开启 DAC 通道 2 */
            break;
        default : break;
    }
}
```

该函数主要调用 HAL_DAC_Init 和 HAL_DAC_ConfigChannel 函数初始化 DAC，并调用 HAL_DAC_Start 函数使能 DAC 通道。HAL_DAC_Init 函数会调用 HAL_DAC_MspInit 回调函数，该函数用于存放 DAC 和对应通道的 IO 时钟使能和初始化 IO 等代码。本实验为了让 dac_init 函数支持 DAC 的 OUT1/2 两个通道的初始化，就没有用到该函数。

下面是设置 DAC 通道 1/2 输出电压函数，其定义如下：

```
/**
 * @brief       设置通道 1/2 输出电压
 * @param       outx: 1,通道 1; 2,通道 2
 * @param       vol : 0~3300,代表 0~3.3V
 * @retval      无
 */
void dac_set_voltage(uint8_t outx, uint16_t vol)
{
    double temp = vol;
    temp /= 1000;
    temp = temp * 4096 / 3.3;
    if (temp >= 4096)temp = 4095;   /* 如果值大于等于 4096，则取 4095 */
    if (outx == 1)   /* 通道 1 */
    {
        /* 12 位右对齐数据格式设置 DAC 值 */
        HAL_DAC_SetValue(&g_dac_handle, DAC_CHANNEL_1, DAC_ALIGN_12B_R, temp);
    }
    else             /* 通道 2 */
    {
        /* 12 位右对齐数据格式设置 DAC 值 */
        HAL_DAC_SetValue(&g_dac_handle, DAC_CHANNEL_2, DAC_ALIGN_12B_R, temp);
    }
}
```

该函数实际就是将电压值转换为 DAC 输入值，形参 1 用于设置通道，形参 2 设置要输出的电压值，设置的范围：0~3300，代表 0~3.3V。

最后在 main 函数里面编写如下代码：

```
int main(void)
{
    uint32_t adcx;
    float temp;
    uint8_t t = 0;
    uint16_t dacval = 0;
    uint8_t key;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    adc_init();                             /* 初始化 ADC */
    dac_init(1);                            /* 初始化 DAC1_OUT1 通道 */
    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "DAC TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "WK_UP:+  KEY1:-", RED);

    lcd_show_string(30, 150, 200, 16, 16, "DAC VAL:", BLUE);
    lcd_show_string(30, 170, 200, 16, 16, "DAC VOL:0.000V", BLUE);
    lcd_show_string(30, 190, 200, 16, 16, "ADC VOL:0.000V", BLUE);
    /* 初始值为 0 */
    HAL_DAC_SetValue(&g_dac_handle, DAC_CHANNEL_1, DAC_ALIGN_12B_R, 0);
    while (1)
    {
        t++;
        key = key_scan(0);          /* 按键扫描 */
        if (key == WKUP_PRES)
        {
            if (dacval < 4000)dacval += 200;
            HAL_DAC_SetValue(&g_dac_handle, DAC_CHANNEL_1, DAC_ALIGN_12B_R,
                             dacval);/* 输出增大 200 */
        }
        else if (key == KEY1_PRES)
        {
            if (dacval > 200)dacval -= 200;
            else dacval = 0;
            HAL_DAC_SetValue(&g_dac_handle, DAC_CHANNEL_1, DAC_ALIGN_12B_R,
                             dacval);/* 输出减少 200 */
        }
        /* WKUP/KEY1 按下了,或者定时时间到了 */
        if (t == 10 || key == KEY1_PRES || key == WKUP_PRES)
        {
            /* 读取前面设置 DAC1_OUT1 的值 */
            adcx = HAL_DAC_GetValue(&g_dac_handle, DAC_CHANNEL_1);
            lcd_show_xnum(94, 150, adcx, 4, 16, 0, BLUE);/* 显示 DAC 寄存器值 */
            temp = (float)adcx * (3.3 / 4096);          /* 得到 DAC 电压值 */
            adcx = temp;
            lcd_show_xnum(94, 170, temp, 1, 16, 0, BLUE);/* 显示电压值整数部分 */
            temp -= adcx;
            temp *= 1000;
            lcd_show_xnum(110, 170, temp,3,16,0X80,BLUE);/*显示电压值的小数部分*/
            /* 得到 ADC 通道 19 的转换结果 */
```

```
            adcx=adc_get_result_average(ADC_ADCX_CHY, 10);
            temp = (float)adcx*(3.3 / 65536);/* 得到 ADC 电压值(adc 是 16bit 的) */
            adcx = temp;
            lcd_show_xnum(94, 190, temp, 1, 16, 0, BLUE); /* 显示电压值整数部分 */
            temp -= adcx;
            temp *= 1000;
            lcd_show_xnum(110, 190,temp, 3, 16,0X80,BLUE);/*显示电压值的小数部分*/
            LED0_TOGGLE();  /* LED0 闪烁 */
            t = 0;
        }
        delay_ms(10);
    }
}
```

此部分代码，我们通过KEY_UP（WKUP按键）和KEY1（也就是上下键）来实现对DAC输出的幅值控制。按下 KEY_UP 增加，按 KEY1 减小。同时在 LCD 上面显示 DHR12R1 寄存器的值、DAC 设置输出电压以及 ADC 采集到的 DAC 输出电压。

35.2.4 下载验证

下载代码后，可以看到LED0不停的闪烁，提示程序已经在运行了。LCD显示如下图所示：

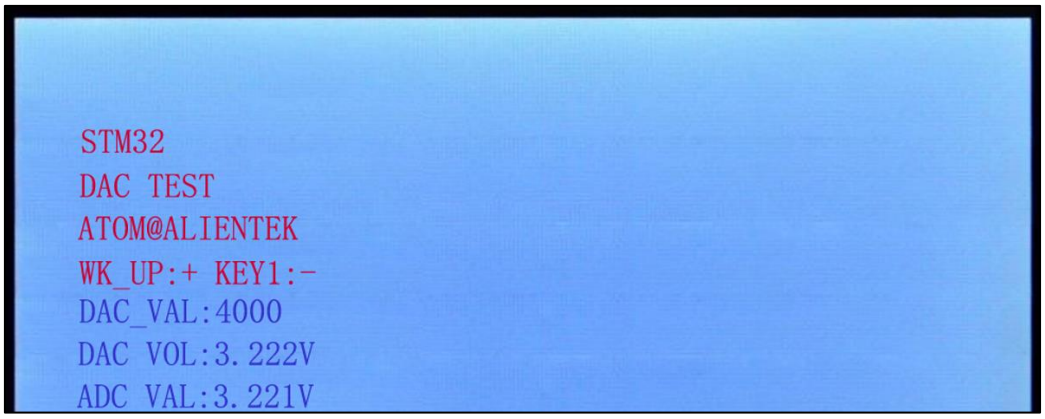


图 35.2.4.1 DAC 输出实验测试图

验证试验前记得先通过跳线帽连接 P10 的 ADC 和 DAC，然后我们可以通过按 WK_UP 按键，增加 DAC 输出的电压，这时 ADC 采集到的电压也会增大，通过按 KEY1 减小 DAC 输出的电压，这时 ADC 采集到的电压也会减小。

除此之外，我们还可以通过usmart调用dac_set_voltage函数，来直接设置DAC输出电压。

35.3 DAC 输出三角波实验

本实验我们来学习使用如何让 DAC 输出三角波，DAC 初始化部分还是用 DAC 输出实验的，所以做本实验的前提是先学习 DAC 输出实验。

35.3.1 DAC 寄存器

本实验用到的寄存器在 DAC 输出实验都有介绍。

35.3.2 硬件设计

1. 例程功能

使用 DAC 输出三角波，通过 KEY0/KEY1 两个按键，控制 DAC1 的通道 1 输出两种三角波，需要通过示波器接 PA4 进行观察。也可以通过 usmart 调用 dac_triangular_wave 函数，来控制输出哪种三角波。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED : LED0 - PB1
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7 寸 LCD 模块(包括 MCU 屏和 RGB 屏，都支持)
4）独立按键：KEY0 - PH3、KEY1 - PH2
5）DAC1：通道 1 - PA4

3. 原理图

我们只需要把示波器的探头接到 DAC1 通道 1（PA4）引脚，就可以在示波器上显示 DAC 输出的波形。PA4 在 P10 多功能端口的 DAC 标志排针已经引出，硬件连接如图 35.3.2.1 所示：

图 35.3.2.1 硬件连接示意图

35.3.3 程序设计

本实验用到的 DAC 的 HAL 库 API 函数前面都介绍过，具体调用情况请看程序解析部分。下面介绍 DAC 输出三角波的配置步骤。

DAC 输出三角波配置步骤

1）开启 DACx 和 DAC 通道对应的 IO 时钟，并配置该 IO 为模拟功能

首先开启 DACx 的时钟，然后配置 GPIO 为模拟模式。本实验我们默认用到 DAC1 通道 1，对应 IO 是 PA4，它们的时钟开启方法如下：

```
__HAL_RCC_DAC12_CLK_ENABLE ();          /* 使能 DAC1 时钟 */
__HAL_RCC_GPIOA_CLK_ENABLE();           /* 开启 GPIOA 时钟 */
```

2）初始化 DACx

通过 HAL_DAC_Init 函数来设置需要初始化的 DAC。该函数并没有设置任何 DAC 相关寄

存器，也就是说没有对 DAC 进行任何配置，它只是 HAL 库提供用来在软件上初始化 DAC。

注意：该函数会调用 HAL_DAC_MspInit 函数来存放 DAC 和对应通道的 IO 时钟使能和初始化 IO 等代码。

3）配置 DAC 通道并启动 DA 转换器

在 HAL 库中，通过 HAL_DAC_ConfigChannel 函数来设置配置 DAC 的通道，根据需求设置触发类型以及输出缓冲。

配置好 DAC 通道之后，通过 HAL_DAC_Start 函数启动 DA 转换器。

4）设置 DAC 的输出值

通过 HAL_DAC_SetValue 函数设置 DAC 的输出值。这里我们根据三角波的特性，创建了 dac_triangular_wave 函数用于控制输出三角波。

35.3.3.1 程序流程图

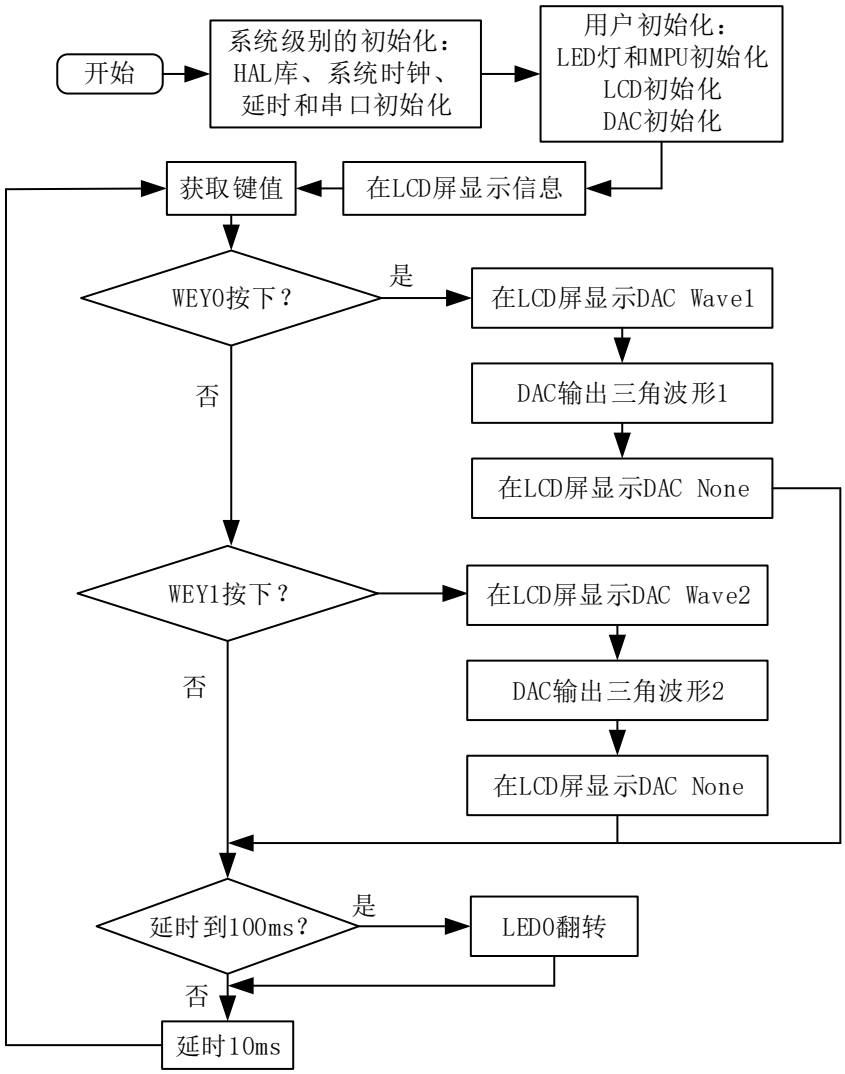


图 35.3.3.1.1 DAC 输出三角波实验程序流程图

35.3.3.2 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。DAC 驱动源码包括两个文件：dac.c 和 dac.h。

dac.h 文件只有一些声明，下面直接开始介绍 dac.c 的程序，本实验的 DAC 初始化我们还是用到 dac_init 函数，就添加了一个设置 DAC_OUT1 输出三角波函数，其定义如下：

```
/**
```

```
 * @brief    设置 DAC_OUT1 输出三角波
 * @note     输出频率 ≈ 1000 / (dt * samples) Khz, 不过在 dt 较小的时候,比如
             小于 5us 时, 由于 delay_us 本身就不准了(调用函数,计算等都需要时间,延时很小
             的时候,这些时间会影响到延时), 频率会偏小.
 *
 * @param    maxval : 最大值(0 < maxval < 4096), (maxval + 1)必须大于等于 samples/2
 * @param    dt     : 每个采样点的延时时间(单位: us)
 * @param    samples: 采样点的个数, samples 必须小于等于(maxval + 1) * 2 ,
                      且 maxval 不能等于 0
 * @param    n      : 输出波形个数,0~65535
 * @retval   无
 */
void dac_triangular_wave(uint16_t maxval,
                         uint16_t dt, uint16_t samples, uint16_t n)
{
    uint16_t i, j;
    float incval;           /* 递增量 */
    float curval;           /* 当前值 */

    if((maxval + 1) <= samples)return ;         /* 数据不合法 */
    incval = (maxval + 1) / (samples / 2);      /* 计算递增量 */
    for(j = 0; j < n; j++)
    {
        curval = 0;
        /* 先输出 0 */
        HAL_DAC_SetValue(&g_dac_handle,DAC_CHANNEL_1,DAC_ALIGN_12B_R,curval);
        for(i = 0; i < (samples / 2); i++)/* 输出上升沿 */
        {
          curval  +=  incval;               /* 新的输出值 */
          /* 用寄存器操作波形会更稳定 */
          HAL_DAC_SetValue(&g_dac_handle,DAC_CHANNEL_1,DAC_ALIGN_12B_R,curval);
          delay_us(dt);
        }
        for(i = 0; i < (samples / 2); i++)/* 输出下降沿 */
        {
          curval  -=  incval;               /* 新的输出值 */
          /* 用寄存器操作波形会更稳定 */
          HAL_DAC_SetValue(&g_dac_handle,DAC_CHANNEL_1,DAC_ALIGN_12B_R,curval);
          delay_us(dt);
        }
    }
}
```

该函数用于设置 DAC 通道 1 输出三角波，输出频率 ≈ 1000 / (dt * samples) Khz，形参意义在源码已经有详细注释。该函数中，我们使用 HAL_DAC_SetValue 函数来设置 DAC 的输出值，这样得到的三角波在示波器上可以看到。如果有跳动现象（不平稳），是正常的，因为调用函数，计算等都需要时间，这样就会导致输出的波形是不太稳定的。越高性能的 MCU，得到的波形会越稳定。而且用 HAL 库函数操作效率没有直接操作寄存器高，所以可以像寄存器版本实验一样，直接操作 DHR12R1 寄存器，得到的波形会相对稳定些。

由于使用 HAL 库的函数，CPU 花费的时间会更长（因为指令变多了），在时间精度要求比较高的应用，就不适合用 HAL 库函数来操作了，这一点希望大家明白。所以学 STM32 不是说只要会 HAL 库就可以了，对寄存器也是需要有一定的理解，最好是熟悉。这里用 HAL 库操作只是为了演示怎么使用 HAL 库的相关函数。

最后在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t t = 0;
    uint8_t key;
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
```

```
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    dac_init(1);                            /* 初始化 DAC1_OUT1 通道 */
    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "DAC Triangular WAVE TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Wave1  KEY1:Wave2", RED);
    lcd_show_string(30, 130, 200, 16, 16, "DAC None", BLUE); /* 提示无输出 */
    while (1)
    {
        t++;
        key = key_scan(0);              /* 按键扫描 */
        if (key == KEY0_PRES)           /* 高采样率 , 约 0.1Khz 波形 */
        {
            lcd_show_string(30, 130, 200, 16, 16, "DAC Wave1 ", BLUE);
            /* 幅值 4095, 采样点间隔 5us, 2000 个采样点, 100 个波形 */
            dac_triangular_wave(4095, 5, 2000, 100);
            lcd_show_string(30, 130, 200, 16, 16, "DAC None  ", BLUE);
        }
        else if (key == KEY1_PRES)      /* 低采样率 , 约 0.1Khz 波形 */
        {
            lcd_show_string(30, 130, 200, 16, 16, "DAC Wave2 ", BLUE);
            /* 幅值 4095, 采样点间隔 500us, 20 个采样点, 100 个波形 */
            dac_triangular_wave(4095, 500, 20, 100);
            lcd_show_string(30, 130, 200, 16, 16, "DAC None  ", BLUE);
        }
        if (t == 10)                    /* 定时时间到了 */
        {
            LED0_TOGGLE();              /* LED0 闪烁 */
            t = 0;
        }
        delay_ms(10);
    }
}
```

该部分代码功能是，按下 KEY0 后，DAC 输出三角波 1，按下 KEY1 后，DAC 输出三角波 2，将 dac_triangular_wave 的形参代入公式：输出频率 ≈ 1000 / (dt * samples) KHz，得到三角波 1 和三角波 2 的频率都是 0.1KHz。

35.3.4 下载验证

下载代码后，可以看到 LED0 不停的闪烁，提示程序已经在运行了，如图 35.3.4.1 所示：

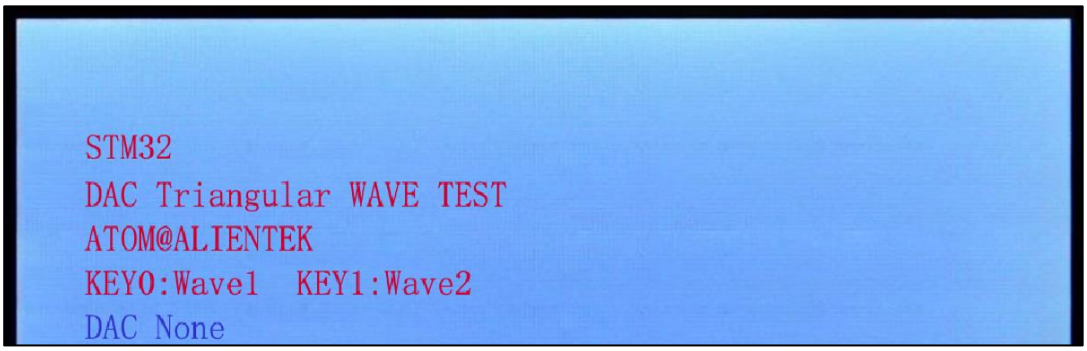


图 35.3.4.1 DAC 输出三角波实验测试图

没有按下任何按键之前，LCD 屏显示 DAC None，当按下 KEY0 后，DAC 输出三角波 1，LCD 屏显示 DAC Wave1，三角波 1 输出完成后 LCD 屏继续显示 DAC None，当按下 KEY1 后，DAC 输出三角波 2，LCD 屏显示 DAC Wave2，三角波 2 输出完成后 LCD 屏继续显示 DAC None。

其中三角波 1 和三角波 2 在示波器的显示情况如下图所示：

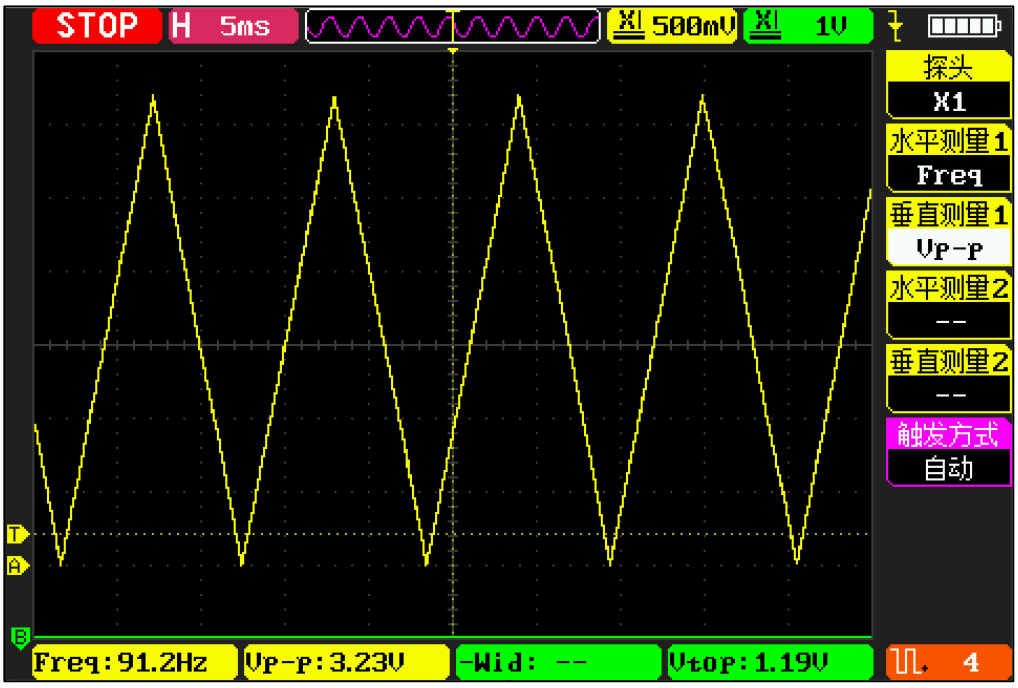


图 35.3.4.2 DAC 输出的三角波 1

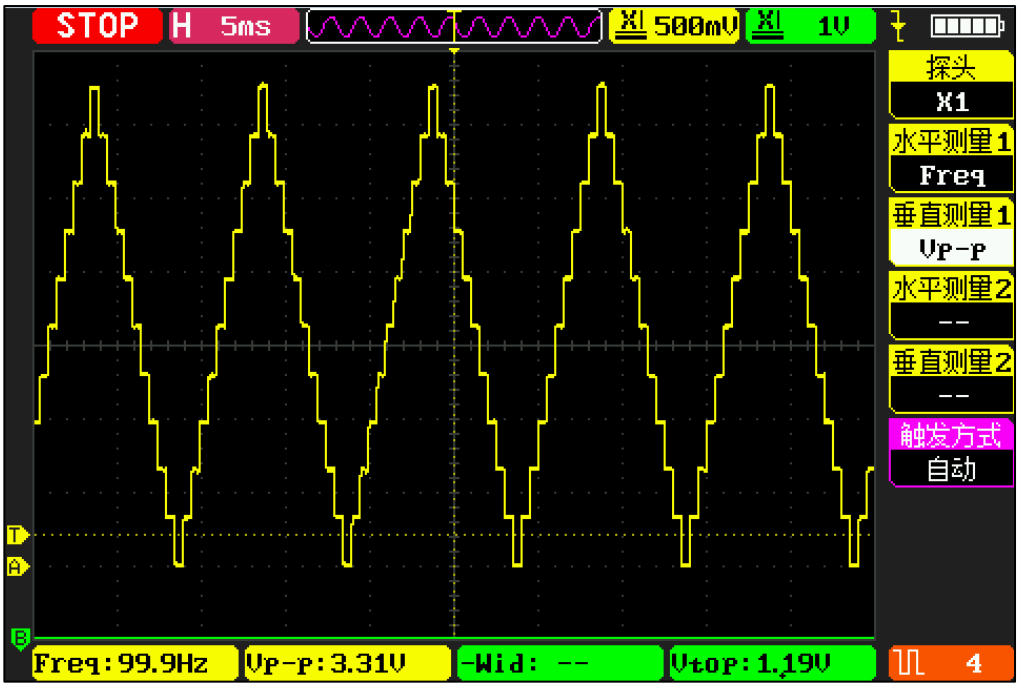


图 35.3.4.3 DAC 输出的三角波 2

由上面两张测试图可以知道，三角波 1 的频率是 91.2Hz，三角波 2 的频率是 99.9Hz，基本都接近我们算出来的结果 0.1KHz。三角波 1 频率的误差较大，在介绍 dac_triangular_wave 函数时也说了原因，加上三角波 1 的采样率比较高，所以误差就会比较大。

35.4 DAC 输出正弦波实验

本实验我们来学习使用如何让 DAC 输出正弦波。实验将用定时器 7 来触发 DAC 进行转换输出正弦波，以 DMA 传输数据的方式。

35.4.1 DAC 寄存器

本实验用到的寄存器在前面的实验都有介绍。

35.4.2 硬件设计

1. 例程功能

使用 DAC 输出正弦波，通过 KEY0/KEY1 两个按键，控制 DAC1 的通道 1 输出两种正弦波，需要通过示波器接 PA4 进行观察。TFTLCD 显示 DAC 转换值、电压值和 ADC 的电压值。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0： LED0 - PB1
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）独立按键：KEY0 - PH3、KEY1 - PH2
5）ADC1：通道 19 - PA5
6）DAC1：通道 1 - PA4
7）DMA(DMA2 数据流 6 DMA 请求源 7)
8）定时器 7

3. 原理图

我们只需要把示波器的探头接到 DAC1 通道 1（PA4）引脚，就可以在示波器上显示 DAC 输出的波形。PA4 在 P10 多功能端口的 DAC 标志排针已经引出，硬件连接如图 35.4.2.1 所示：

图 35.4.2.1 硬件连接示意图

35.4.3 程序设计

35.4.3.1 DAC 的 HAL 库驱动

本实验用到的 HAL 库 API 函数前面大都介绍过，下面将介绍本实验用到且没有介绍过的。

1. HAL_DAC_Start_DMA 函数

启动 DAC 使用 DMA 方式传输函数，其声明如下：

```
HAL_StatusTypeDef HAL_DAC_Start_DMA(DAC_HandleTypeDef *hdac, uint32_t Channel,
                                    uint32_t *pData, uint32_t Length, uint32_t Alignment);
```

- **函数描述：**

用于启动 DAC 使用 DMA 的方式。

- **函数形参：**

  形参 1 是 DAC_HandleTypeDef 结构体类型指针变量。

  形参 2 用于选择要启动的通道，可选择 DAC_CHANNEL_1 或者 DAC_CHANNEL_2。

  形参 3 是使用 DAC 输出数据缓冲区的指针。

  形参 4 是 DAC 输出数据的长度。

  形参 5 是指定 DAC 通道的数据对齐方式，有：DAC_ALIGN_8B_R（8 位右对齐）、DAC_ALIGN_12B_L（12 位左对齐）和 DAC_ALIGN_12B_R（12 位右对齐）三种方式。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

2. HAL_DAC_Stop_DMA 函数

停止 DAC 的 DMA 方式函数，其声明如下：

```
HAL_StatusTypeDef HAL_DAC_Stop_DMA(DAC_HandleTypeDef *hdac, uint32_t Channel);
```

- **函数描述：**

  用于停止 DAC 的 DMA 方式。

- **函数形参：**

  形参 1 是 DAC_HandleTypeDef 结构体类型指针变量。

  形参 2 用于选择要启动的通道，可选择 DAC_CHANNEL_1 或者 DAC_CHANNEL_2。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

3. HAL_TIMEx_MasterConfigSynchronization 函数

配置主模式下的定时器触发输出选择函数，其声明如下：

```
HAL_StatusTypeDef HAL_TIMEx_MasterConfigSynchronization(
              TIM_HandleTypeDef *htim, TIM_MasterConfigTypeDef *sMasterConfig);
```

- **函数描述：**

  用于配置主模式下的定时器触发输出选择。

- **函数形参：**

  形参 1 是 TIM_HandleTypeDef 结构体类型指针变量。

  形参 2 是 TIM_MasterConfigTypeDef 结构体类型指针变量，用于配置定时器工作在主/从模式，以及触发输出(TRGO 和 TRGO2)的选择。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

DAC 输出正弦波配置步骤

1）开启 DACx 和 DAC 通道对应的 IO 时钟，并配置该 IO 为模拟功能

首先开启 DACx 的时钟，然后配置 GPIO 为模拟模式。本实验我们默认用到 DAC1 通道 1，对应 IO 是 PA4，它们的时钟开启方法如下：

```
__HAL_RCC_DAC12_CLK_ENABLE ();          /* 使能 DAC1 时钟 */
__HAL_RCC_GPIOA_CLK_ENABLE();           /* 开启 GPIOA 时钟 */
```

2）初始化 DACx

通过 HAL_DAC_Init 函数来设置需要初始化的 DAC。该函数并没有设置任何 DAC 相关寄存器，也就是说没有对 DAC 进行任何配置，它只是 HAL 库提供用来在软件上初始化 DAC。

注意：该函数会调用 HAL_DAC_MspInit 函数来存放 DAC 和对应通道的 IO 时钟使能和初始化 IO 等代码。

3）配置 DAC 通道

在 HAL 库中，通过 HAL_DAC_ConfigChannel 函数来设置配置 DAC 的通道，根据需求设置触发类型以及输出缓冲等。

4）配置 DMA 并关联 DAC

通过 HAL_DMA_Init 函数初始化 DMA，包括配置通道，外设地址，存储器地址，传输数据量等。

HAL 库为了处理各类外设的 DMA 请求，在调用相关函数之前，需要调用一个宏定义标识符，来连接 DMA 和外设句柄。这个宏定义为__HAL_LINKDMA。

5）配置定时器控制触发 DAC

通过 HAL_TIM_Base_Init 函数设置定时器溢出频率。

通过 HAL_TIMEx_MasterConfigSynchronization 函数配置定时器溢出事件用于触发。

通过 HAL_TIM_Base_Start 函数启动计数。

6）启动 DAC 转换并以 DMA 方式传输数据

通过 HAL_DAC_Stop_DMA 函数先停止之前的 DMA 传输以及 DAC 输出。

再通过 HAL_DAC_Start_DMA 函数启动 DMA 传输以及 DAC 输出。

35.4.3.2 程序流程图

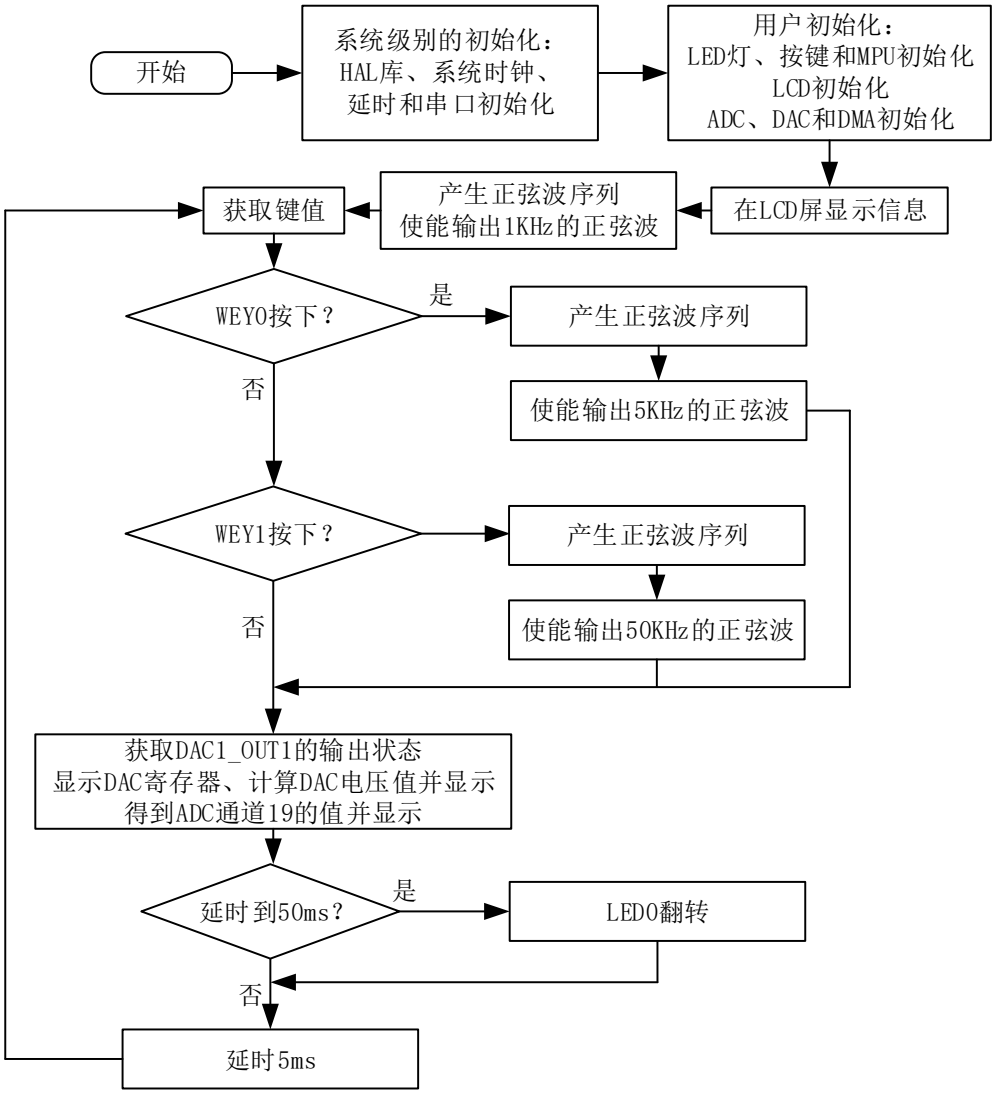


图 35.4.3.2.1 DAC 输出正弦波实验程序流程图

35.4.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。DAC 驱动源码包括两个文件：dac.c 和 dac.h。

dac.h 文件只有一些声明，下面直接开始介绍 dac.c 的程序，本实验的 DAC 以及 DMA 的初始化，我们用到 dac_dma_wave_init 函数，其定义如下：

```
/**
```

```
 * @brief  DAC DMA 输出正弦波初始化函数
 * @note DAC 的输入时钟来自 APB1, 时钟频率=100Mhz=10ns
 *       DAC 在输出 buffer 关闭的时候, 输出建立时间: tSETTLING = 2us (H743 数据手册有写)
 *       DAC 输出的最高速度约为:500Khz,以 10 个点为一个周期, 最大能输出 50Khz 左右的波形
 * @param    无
 * @retval   无
 */
void dac_dma_wave_init(void)
{
    DAC_ChannelConfTypeDef dac_ch_conf={0};

    GPIO_InitTypeDef gpio_init_struct;

    __HAL_RCC_GPIOA_CLK_ENABLE();                /* DAC 通道引脚端口时钟使能 */
    __HAL_RCC_DAC12_CLK_ENABLE();                /* DAC 外设时钟使能 */
    __HAL_RCC_DMA2_CLK_ENABLE();                 /* DMA 时钟使能 */

    gpio_init_struct.Pin = GPIO_PIN_4;                    /* PA4 */
    gpio_init_struct.Mode = GPIO_MODE_ANALOG;             /* 模拟 */
    HAL_GPIO_Init(GPIOA, &gpio_init_struct);              /* 初始化 DAC 引脚 */

    g_dma_dac_handle.Instance = DMA2_Stream6;             /* 使用的 DAM2 Stream6 */
    g_dma_dac_handle.Init.Request = DMA_REQUEST_DAC1_CH1;/* DAC 触发 DMA 传输 */
    /* 存储器到外设模式 */
    g_dma_dac_handle.Init.Direction = DMA_MEMORY_TO_PERIPH;
    g_dma_dac_handle.Init.PeriphInc = DMA_PINC_DISABLE;     /* 外设地址禁止自增 */
    g_dma_dac_handle.Init.MemInc = DMA_MINC_ENABLE;         /* 存储器地址自增 */
    /* 外设数据长度:16 位 */
    g_dma_dac_handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_HALFWORD;
    /* 存储器数据长度:16 位 */
    g_dma_dac_handle.Init.MemDataAlignment = DMA_MDATAALIGN_HALFWORD;
    g_dma_dac_handle.Init.Mode = DMA_CIRCULAR;              /* 循环模式 */
    g_dma_dac_handle.Init.Priority = DMA_PRIORITY_MEDIUM;   /* 中等优先级 */
    g_dma_dac_handle.Init.FIFOMode = DMA_FIFOMODE_DISABLE; /* 不使用 FIFO */
    HAL_DMA_Init(&g_dma_dac_handle);                        /* 初始化 DMA */

    /* DMA 句柄与 DAC 句柄关联 */
    __HAL_LINKDMA(&g_dac_dma_handle, DMA_Handle1, g_dma_dac_handle);

    g_dac_dma_handle.Instance = DAC1;                     /* 选择哪个 DAC */
    HAL_DAC_Init(&g_dac_dma_handle);                      /* DAC 初始化 */

    /* 关闭采样保持模式，这个模式主要用于低功耗 */
    dac_ch_conf.DAC_SampleAndHold = DAC_SAMPLEANDHOLD_DISABLE;
    dac_ch_conf.DAC_Trigger = DAC_TRIGGER_T7_TRGO;          /* 采用定时器 7 触发 */
    dac_ch_conf.DAC_OutputBuffer = DAC_OUTPUTBUFFER_ENABLE;/* 使能输出缓冲 */
    /* 不将 DAC 连接到片上外设 */
    dac_ch_conf.DAC_ConnectOnChipPeripheral = DAC_CHIPCONNECT_DISABLE;
    dac_ch_conf.DAC_UserTrimming = DAC_TRIMMING_FACTORY;   /* 使用出厂校准 */
    /* DAC 通道输出配置 */
    HAL_DAC_ConfigChannel(&g_dac_dma_handle, &dac_ch_conf, DAC_CHANNEL_1);
}
```

该函数用于初始化 DAC 用 DMA 的方式输出正弦波。本函数用到的 API 函数在前面都介绍过，请结合前面介绍过的相关内容来理解源码。这里值得注意的是我们是采用定时器 7 触发 DAC 进行转换输出的。

下面介绍 DAC DMA 使能波形输出函数，其定义如下：

```
/**
 * @brief       DAC DMA 使能波形输出
 * @note        TIM7 的输入时钟频率(f)来自 APB1, f = 100M * 2 = 200Mhz.
 *              DAC 触发频率 ftrgo = f / ((psc + 1) * (arr + 1))
```

```
 *              波形频率 = ftrgo / ndtr;
 * @param       ndtr        : DMA 通道单次传输数据量
 * @param       arr         : TIM7 的自动重装载值
 * @param       psc         : TIM7 的分频系数
 * @retval      无
 */
void dac_dma_wave_enable(uint16_t ndtr, uint16_t arr, uint16_t psc)
{
    TIM_HandleTypeDef tim7_handle = {0};
    TIM_MasterConfigTypeDef master_config={0};

    __HAL_RCC_TIM7_CLK_ENABLE();                                /* TIM7 时钟使能 */

    tim7_handle.Instance = TIM7;                                /* 选择定时器 7 */
    tim7_handle.Init.Prescaler = psc;                           /* 分频系数 */
    tim7_handle.Init.CounterMode = TIM_COUNTERMODE_UP;          /* 递增计数 */
    tim7_handle.Init.Period = arr;                              /* 重装载值 */
    /* 自动重装 */
    tim7_handle.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
    HAL_TIM_Base_Init(&tim7_handle);                            /* 初始化定时器 7 */
    master_config.MasterOutputTrigger = TIM_TRGO_UPDATE;
    master_config.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
    /* 配置 TIM7 TRGO */
    HAL_TIMEx_MasterConfigSynchronization(&tim7_handle, &master_config);
    HAL_TIM_Base_Start(&tim7_handle);                           /* 使能定时器 7 */
    HAL_DAC_Stop_DMA(&g_dac_dma_handle, DAC_CHANNEL_1); /* 先停止之前的传输 */
    HAL_DAC_Start_DMA(&g_dac_dma_handle, DAC_CHANNEL_1,
                      (uint32_t *)g_dac_sin_buf, ndtr, DAC_ALIGN_12B_R);
}
```

该函数用于使能波形输出，利用定时器 7 的更新事件来触发 DAC 转换输出。使能定时器 7 的时钟后，调用 HAL_TIMEx_MasterConfigSynchronization 函数配置 TIM7 选择更新事件作为触发输出 (TRGO)，然后调用 HAL_DAC_Stop_DMA 函数停止 DAC 转换以及 DMA 传输，最后再调用 HAL_DAC_Start_DMA 函数重新配置并启动 DAC 和 DMA。

最后在 main.c 里面编写如下代码：

```
uint16_t g_dac_sin_buf[4096];     /* 发送数据缓冲区 */

/**
 * @brief       产生正弦波序列
 * @note        需保证: maxval > samples/2
 * @param       maxval : 最大值(0 < maxval < 2048)
 * @param       samples: 采样点的个数
 * @retval      无
 */
void dac_creat_sin_buf(uint16_t maxval, uint16_t samples)
{
    uint8_t i;
    float inc = (2 * 3.1415962) / samples;/* 计算增量（一个周期 DAC_SIN_BUF 个点）*/
    float outdata = 0;
    for (i = 0; i < samples; i++)
    {
        /* 计算以 dots 个点为周期的每个点的值，放大 maxval 倍，并偏移到正数区域 */
        outdata = maxval * (1 + sin(inc * i));
        if (outdata > 4095) outdata = 4095;      /* 上限限定 */
        //printf("%f\r\n",outdata);
        g_dac_sin_buf[i] = outdata;
    }
}

/**
 * @brief       通过 USMART 设置正弦波输出参数,方便修改输出频率.
 * @param       arr : TIM7 的自动重装载值
```

```
 * @param       psc : TIM7 的分频系数
 * @retval      无
 */
void dac_dma_sin_set(uint16_t arr, uint16_t psc)
{
    dac_dma_wave_enable(100, arr, psc);
}

int main(void)
{
    uint16_t adcx;
    float temp;
    uint8_t t = 0;
    uint8_t key;
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    adc_init();                             /* 初始化 ADC */
    dac_dma_wave_init();                    /* 初始化 DAC 通道 1 DMA 波形输出 */
    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "DAC DMA Sine WAVE TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:5Khz  KEY1:50Khz", RED);
    lcd_show_string(30, 130, 200, 16, 16, "DAC VAL:", BLUE);
    lcd_show_string(30, 150, 200, 16, 16, "DAC VOL:0.000V", BLUE);
    lcd_show_string(30, 170, 200, 16, 16, "ADC VOL:0.000V", BLUE);
    dac_creat_sin_buf(2048, 100);
    dac_dma_wave_enable(100, 100 - 1, 20 - 1);
    while (1)
    {
        t++;
        key = key_scan(0);  /* 按键扫描 */
        if (key == KEY0_PRES)               /* 高采样率 , 约 5Khz 波形 */
        {
            dac_creat_sin_buf(2048, 100);  /* 产生正弦波函序列 */
            /* 500Khz 触发频率, 100 个点, 得到最高 5KHz 的正弦波. */
            dac_dma_wave_enable(100, 20 - 1, 20 - 1);
        }
        else if (key == KEY1_PRES)          /* 低采样率 , 约 50Khz 波形 */
        {
            dac_creat_sin_buf(2048, 10);    /* 产生正弦波函序列 */
            /* 500Khz 触发频率, 10 个点, 可以得到最高 50KHz 的正弦波. */
            dac_dma_wave_enable(10, 20 - 1, 20 - 1);
        }
        adcx = DAC1->DHR12R1;               /* 获取 DAC1_OUT1 的输出状态 */
        lcd_show_xnum(94, 130, adcx, 4, 16, 0, BLUE);/* 显示 DAC 寄存器值 */
        temp = (float)adcx * (3.3 / 4096);          /* 得到 DAC 电压值 */
        adcx = temp;
        lcd_show_xnum(94, 150, temp, 1, 16, 0, BLUE);/* 显示电压值整数部分 */
        temp -= adcx;
        temp *= 1000;
        lcd_show_xnum(110, 150, temp, 3, 16, 0X80, BLUE);/* 显示电压值的小数部分 */
        adcx = adc_get_result_average(ADC_ADCX_CHY,20);
        /* 得到 ADC 通道 19 的转换结果 */
```

```
        temp = (float)adcx * (3.3 / 65536);  /* 得到 ADC 电压值(adc 是 16bit 的) */
        adcx = temp;
        lcd_show_xnum(94, 170, temp, 1, 16, 0, BLUE); /* 显示电压值整数部分 */
        temp -= adcx;
        temp *= 1000;
        lcd_show_xnum(110, 170, temp, 3, 16, 0X80, BLUE);/* 显示电压值的小数部分 */
        if (t == 40)         /* 定时时间到了 */
        {
            LED0_TOGGLE();  /* LED0 闪烁 */
            t = 0;
        }
        delay_ms(5);
    }
}
```

adc_init 函数初始化 ADC1，用于测量 DAC 通道 1 的电压值。

dac_dma_wave_init 函数初始化 DAC 通道 1，并指定 DMA 搬运的数据的开始地址和目标地址。dac_creat_sin_buf 函数用于产生正弦波序列，并保存在 g_dac_sin_buf 数组中，供给 DAC 转换。在进入 wilhe(1)循环之前，dac_dma_wave_enable 函数默认配置 DAC 的采样点个数时 100，并配置定时器 7 的溢出频率为 100KHz。这样就可以输出 1KHz 的正弦波。下面给大家解释一下为什么是输出 1KHz 的正弦波？

定时器 7 的溢出频率为 100KHz，不记得怎么计算的朋友，请回顾基本定时器的相关内容，这里直接把公式列出：

$$Tout = ((arr + 1) * (psc + 1)) / Tclk$$

看到 dac_dma_wave_enable(100, 100 - 1, 20 - 1);这个语句，第二个形参是自动重装载值，第三个形参是分频系数，那么代入公式，可得：

$$Tout = ((arr + 1) * (psc + 1)) / Tclk = ((99+1)*(19+1))/ 200MHZ= 0.00001s$$

得到定时器的更新事件周期是 0.00001 秒，即更新事件频率为 100KHz，也就得到 DAC 输出触发频率为 100KHz。

再结合总一个正弦波共有 100 个采样点，就可以得到正弦波的频率为 100KHz/100 = 1KHz。

知道了正弦波的频率怎么来的，下面代码中，按下按键 KEY0，得到 5KHz 的正弦波，按下按键 KEY1，得到 50KHz 的正弦波，计算方法都一样的。

dac_dma_sin_set 函数可以通过 USMART 设置正弦波输出参数，方便修改输出频率。

35.4.4 下载验证

下载代码后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示如图 35.4.4.1 所示：

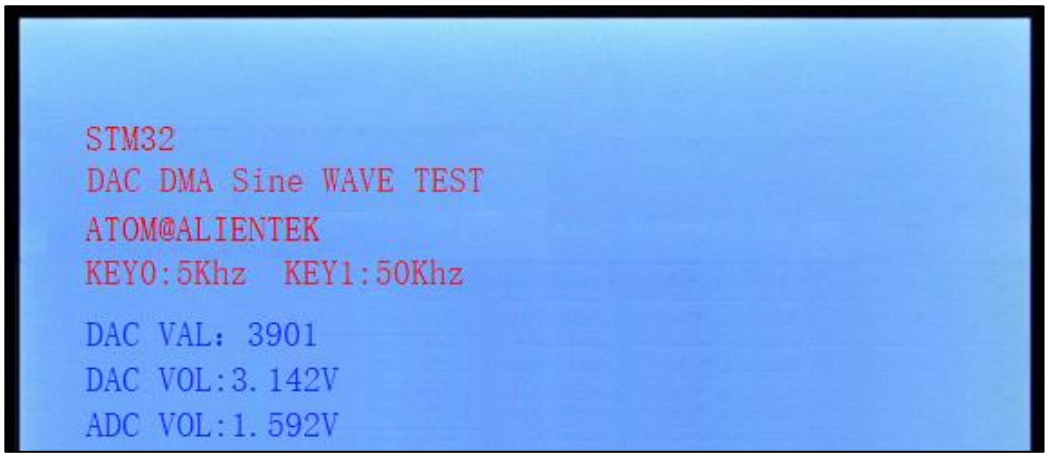


图 35.4.4.1 DAC 输出正弦波实验测试图

上图是将跳线帽连接多功能端口 P10 的 ADC 和 DAC 两个排针，可以看到 ADC VOL 的值随着 DAC 的输出变化而变化，即 ADC 采集到的值是不停变化的。由于变化太快了，这样看不出采集到值形成什么波形，下面我们借用示波器来进行观察，首先将探头接到 DAC 的排针上。

没有按下任何按键之前，默认是输出 1KHz（100 个采样点）的正弦波，如下图所示：

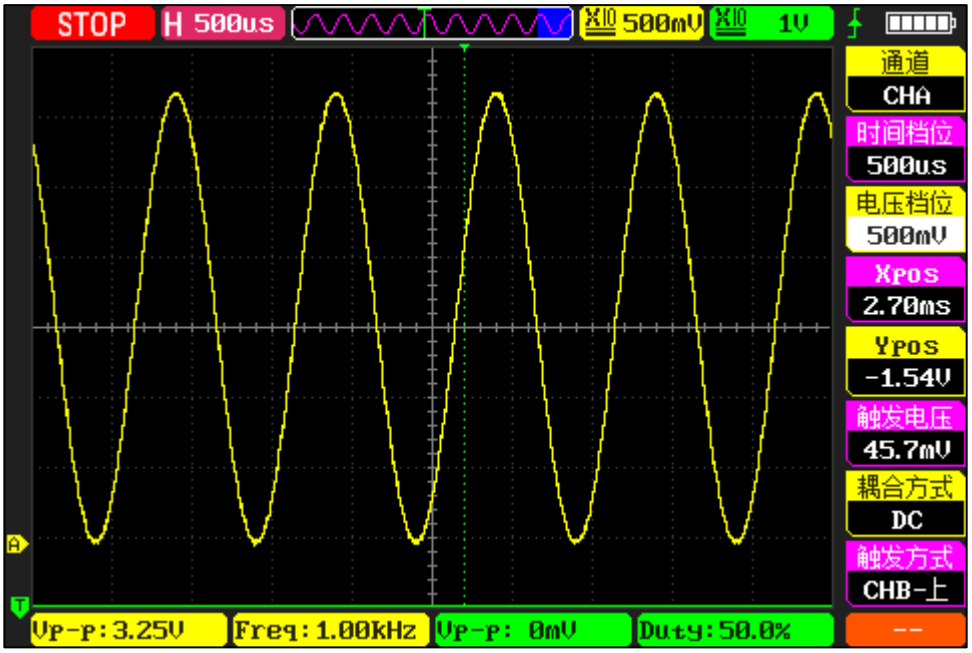


图 35.3.4.2 默认 DAC 输出的的正弦波

当按下 KEY0 后，DAC 输出 5KHz（100 个采样点）的正弦波，如下图所示：

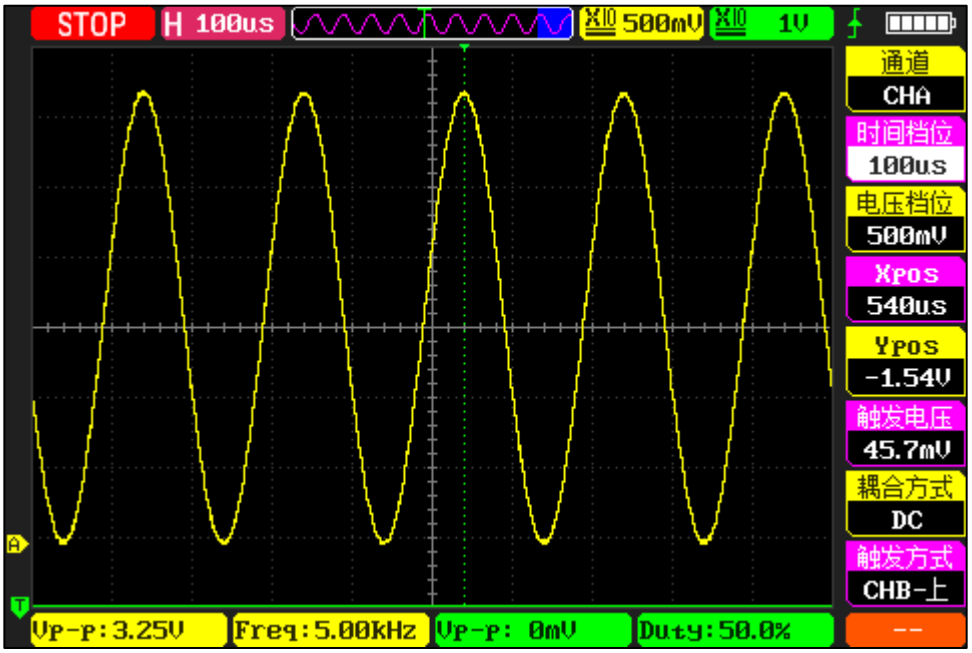


图 35.3.4.3 按下 KEY0，DAC 输出的的正弦波

当按下 KEY1 后，DAC 输出 30KHz（10 个采样点）的正弦波，如下图所示：

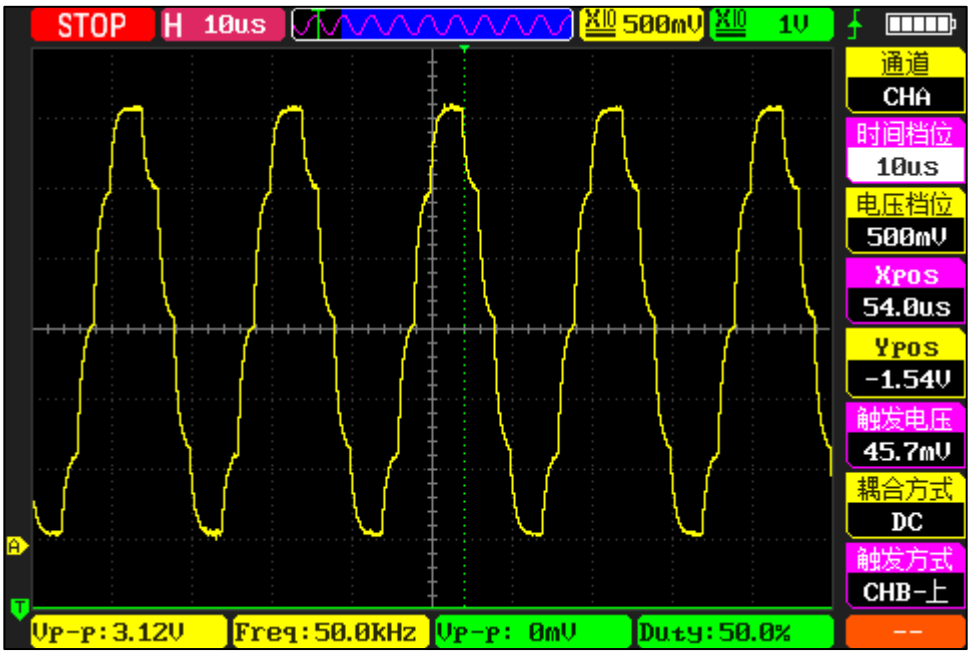


图 35.3.4.4 按下 KEY1，DAC 输出的的正弦波

第三十六章 PWM DAC 实验

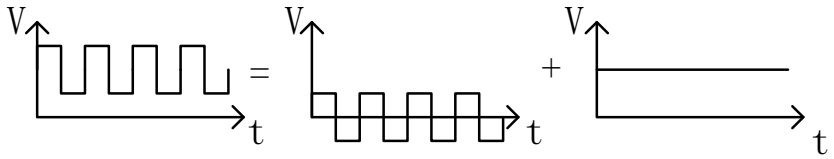
前面的章节，我们介绍了 STM32H743 自带 DAC 模块的使用，但有时候可能两个 DAC 不够用，此时，我们可以通过 PWM+RC 滤波来实现一个 PWM DAC。本章我们将向大家介绍如何使用 STM32H743 的 PWM 来设计一个 DAC。我们将使用按键（或 USMART）控制 STM32H743 的 PWM 输出，从而控制 PWM DAC 的输出电压，通过 ADC1 的通道 19 采集 PWM DAC 的输出电压，并在 LCD 模块上面显示 ADC 获取到的电压值以及 PWM DAC 的设定输出电压值等信息。本章将分为如下几个部分：

36.1 PWM DAC 技术的实现原理
36.2 硬件设计
36.3 程序设计
36.4 下载验证

36.1 PWM DAC 技术的实现原理

从前面的学习我们知道 DAC 是根据我们的源电压，按指定的 8 位、12 位、16 位等精度对源电压进行分割，其输出按最小精度 LSB（即 $1/2^8$、$1/2^{12}$、$1/2^{16}$ 等）的倍数输出，即得到我们需要的 DAC 电压。最后，我们得到的 DAC 的电压为直流有效信号。

我们来分析一下 PWM 波形的特性。从电做功的角度，可以把一个 PWM 波等效成一个“总有效值为 0”的交流波形和一个直流的电信号的叠加，直流部分的特性可根据占空比的改变而改变，这符合 DAC 的特性，如图 36.1.1.1 所示。



（a）PWM 波形　（b）有效值为0的PWM波形（c）直流有效值波形

图 36.1.1.1 PWM 波形的等效

上面的等效原理我们从图形上就很容易等效出来，下面我们简单介绍一下这个现象的数学原理，我们知道对于一个典型的 PWM 波型，它的输出波形和时间的关系如图 36.1.1.2 所示。

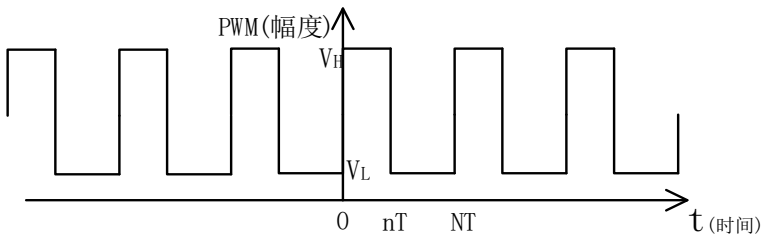


图 36.1.1.2 PWM 波形的时域表示

对于上面的 PWM 波形，下面根据高数和信号与系统课程的知识我们作一个简单的推导，感兴趣的同学可以查阅对应的知识，如果不感兴趣，直接跳过推导过程看最后的结论即可。

我们可以把 PWM 波形用分段函数①表示出来。占空比可以用②的表达式来表示。

$$f(t)=\begin{cases}V_H, & kNT \le x < kNT+nT \\ V_L, & kNT+nT \le x \le kNT+NT\end{cases} \quad ①$$

$$p=\frac{n}{N} \quad ②$$

PWM 是一个周期信号，我们令周期为 NT，由傅里叶变换的知识可知任意周期信号都可按频域展开，我们把分段表达式按频域展开。得到如③的表达式，④⑤⑥为展开系数：

$$f(t)=A_0+\sum_{k=1}^{\infty}\left[A_n\cos\left(\frac{2k\pi t}{NT}\right)+B_n\sin\left(\frac{2k\pi t}{NT}\right)\right] \quad ③$$

$$A_0 = \frac{1}{2NT}\int_{-NT}^{NT} f(t)\,dt \quad ④$$
$$A_k = \frac{1}{2NT}\int_{-NT}^{NT} f(t)\cos\left(\frac{2k\pi t}{NT}\right)dt \quad ⑤$$
$$B_k = \frac{1}{2NT}\int_{-NT}^{NT} f(t)\sin\left(\frac{2k\pi t}{NT}\right)dt \quad ⑥$$

我们知道PWM的幅度为V_H-V_L，占空比为p，代入公式③~⑥求对应的积分，可以求出f(t)的展开系数分别如下：

$$A_0 = (V_H - V_L) * p \quad ⑦$$
$$A_k = (V_H - V_L) \times \frac{1}{k\pi}[\sin(k\pi p) - \sin\left(2k\pi(1-\frac{p}{2})\right)] \quad ⑧$$
$$B_k = 0 \quad ⑨$$

根据展开得到的频率参数，由此可得出 PWM 信号及其占空比在时域上的表达式。

$$f(t) = (V_H - V_L) * p + \sum_{k=1}^{\infty}[(V_H - V_L) \times \frac{1}{k\pi}\left[\sin(k\pi p) - \sin\left(2k\pi\left(1-\frac{p}{2}\right)\right)\right] \times \cos\left(\frac{2k\pi t}{NT}\right) \quad ⑩$$

公式⑩正好验证了图 36.1.1.1 的 PWM 等效原理。由此我们可知 PWM 的输出波形为一个与占空比有关的直流等效信号，同时伴有多个不同频率的信号的叠加。如果能把这些频率信号尽可能过滤掉，那么我们通过调整 PWM 的占空比即可方便实现我们需要的 DAC 结果，即 **V_{DAC}=(V_H-V_L)*p**。

分辨率也是 DAC 一个重要的参数，它可以表示 DAC 输出的最小精度。存在两个主要误差源影响 PWM 方式 DAC 分辨率。首先，PWM 信号的占空比只能表示有限的分辨率。这是因为 STM32 的 PWM 的占空比是输出比较寄存器 CCRx 与 TIMx_CNT 进行比较的结果，而 CCRx 在 STM32H7 系列中大部分是 16 位的（TIM2 和 TIM5 是 32 位的）。那么很显然地，用 PWM 实现的 DAC 分辨率就与 TIMx_CNT 有关，即定时器的时钟频率越高则 CCRx 可以设置的值越多，分辨率相应地越高。定时器最高时钟是 200Mhz，定时器的频率越高，DAC 的速度越慢。

第二个误差源是 PWM 信号中不期望的谐波分量产生的峰值。前面 PWM 的频域展开公式⑩说明 PWM 信号需要通过滤波器才能输出一个纹波较小的直流信号，但实际上对于简单设计的滤波器对交流信号的过滤能力是有限的，所以输出信号还会带有一定的交流成份。

根据公式⑧，将 k=1 代入我们可以算出 PWM 的一次谐波幅度：

$$A_1 = (V_H - V_L) \times \frac{1}{\pi}\left[\sin(\pi p) - \sin\left(2\pi\left(1-\frac{p}{2}\right)\right)\right] = \frac{2}{\pi}(V_H - V_L)\sin(\pi p - \pi) \quad ⑪$$

当 sin(πp - π) = 1 时滤波器需要达到衰减峰值，可知 PWM 占空比为 50%时，一次谐波的幅度最大。为了减少这个基波的影响，我们希望滤波器在这个最大幅度下也能把基波的交流影响衰减到 1/2LSB 以下，即后外围滤波器至少需要满足以下条件才能避免 DAC 输出干扰过大：

$$A_{k=1} \times A_{filite} \le \frac{1}{2} \times \frac{(V_H - V_L)}{2^Y} \quad ⑫$$

根据公式⑪⑫可知Aflite $\le \frac{\pi}{2^{Y+2}}$=$20 \times \log(\frac{\pi}{2^{Y+2}})$，当 DAC 为 12 位精度时，代入 Y=12 可知我们设计的滤波器需要衰减 74dB 以上，当为 8 位精度时，衰减需要达到 50dB。

我们知道一阶 RC 电路截止频率计算公式为：

$$f_c = \frac{1}{2\pi RC}$$

把电容等效成一个电阻，对于一阶分压时电压的等效衰减的表达式可以是：

$$V_{out} = V_{IN}(\frac{1}{\sqrt{(2\pi f_c CR)^2+1^2}})$$

根据以上公式就能很好地设计一个满足我们需要的滤波器参数了。为了实现低成本的 RC 电路，我们使用两个一阶 RC 电路串联起来作为滤波器。

我们以 200Mhz 的频率为例，8 位分辨率的时候，PWM 频率为 200M/256=781.25Khz。如果是 1 阶 RC 滤波，则要求截止频率为 2.66Khz，如果是 2 阶 RC 滤波，则要求截止频率为 33.62Khz。

36.2 硬件设计

1. 例程功能

我们将设计一个 8 位的 DAC，使用按键（或 USMART）控制 STM32H743 的 PWM 输出（占空比），从而控制 PWM DAC 的输出电压。为了得知 PWM 的输出电压，通过 ADC1 的通道 19 采集 PWM DAC 的输出电压，并在 LCD 模块上面显示 ADC 获取到的电压值以及 PWM DAC 的设定输出电压值等信息。

2. 硬件资源

1）LED 灯
LED0 - PB1
2）独立按键
KEY0 - PH3
KEY_UP - PA0
3）PWM 输出通道
我们的例程中使用 TIM15 的通道 2，对应 IO 是 PA3
4）设计一个对 PWM 输出的 DAC 信号进行滤波的滤波电路
5）ADC
ADC1 通道 19 - PA5

3. 原理图

根据前面分析的原理，在硬件设计上就比较简单了，PWM 可以由 STM32 定时器输出，我们只需要在外围增加一个滤波电路即可。我们使用的是二阶 RC 滤波电路，所以电路设计如下：

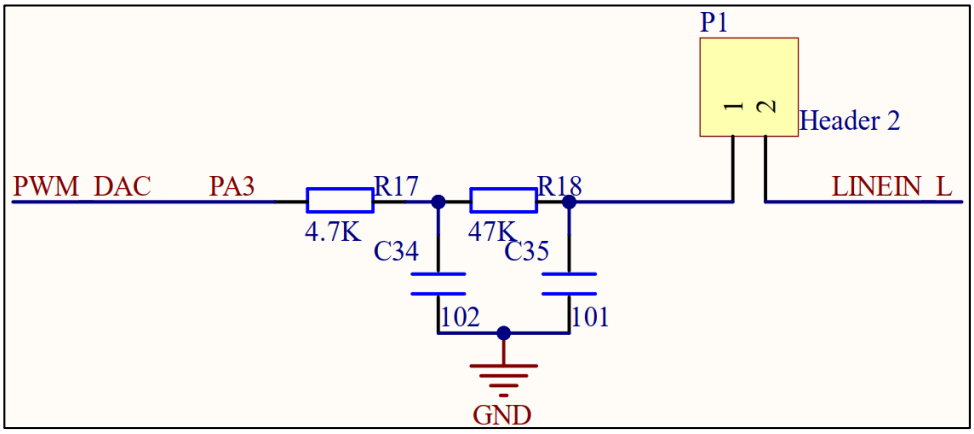


图 36.2.1 PWM DAC 连接原理设计

根据我们的设计，输出 8 位 DAC 时，经过二阶滤波后 DAC 输出的交流信号大概的衰减可以达到 50dB，可见我们设计是符合要求的。

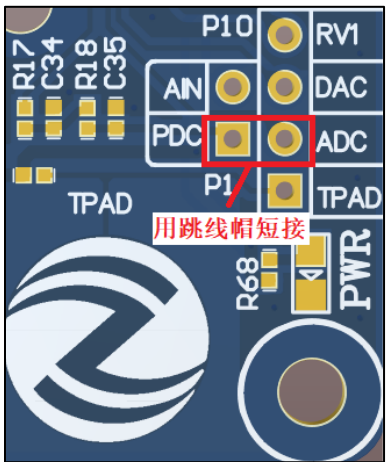


图 36.2.2 硬件连接示意图

我们把 PWM 的输出引到排针 P10 上，它在开发板上与 ADC 相邻。我们需要使用 ADC 功能，来测量我们的 PWM 输出的 DAC 值，所以用跳线帽把 PDC 和 ADC 连接到一起即可。

36.3 程序设计

前面学习了用 PWM 实现 DAC 的原理，下面我们将结合之前学习的知识，实战一下 PWM DAC 的实现。

36.3.1 程序流程图

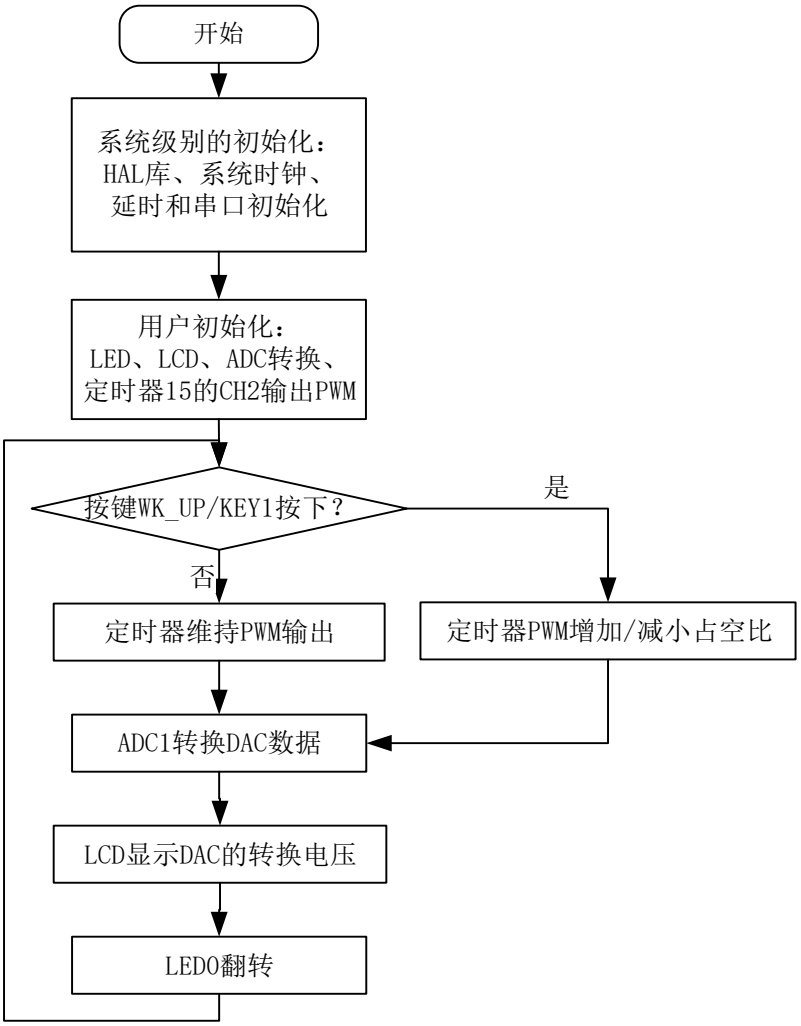


图 36.3.1.1 PWM DAC 实验程序流程图

36.3.2 程序解析

1. PWM DAC 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。**PWM DAC** 驱动源码包括两个文件：pwmdac.c 和 pwmdac.h。

为方便修改，我们在 pwmdac.h 中使用宏定义 PWM 的输出控制外设，它们列出如下：

```
/* PWMDAC 默认是使用 PA3，对应的定时器为 TIM15_CH2，如果你要修改成其他 IO 输出，则相应
 * 的定时器及通道也要进行修改. 请根据实际情况进行修改.
 */
#define PWMDAC_GPIO_PORT                GPIOA
#define PWMDAC_GPIO_PIN                 GPIO_PIN_3
#define PWMDAC_GPIO_CLK_ENABLE()
              do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)/* PA 口时钟使能 */
#define PWMDAC_GPIO_AFTIMX              GPIO_AF4_TIM15

#define PWMDAC_TIMX                     TIM15
#define PWMDAC_TIMX_CHY                 TIM_CHANNEL_2     /* 通道 Y,  1<= Y <=4 */

#define PWMDAC_TIMX_CCRX                PWMDAC_TIMX->CCR2
/* 通道 Y 的输出比较寄存器 */
#define PWMDAC_TIMX_CLK_ENABLE()
```

```
    do{ __HAL_RCC_TIM15_CLK_ENABLE(); }while(0)/* TIM15 时钟使能 */
```

我们使用 pwmdac_init 来进行定时器 15 及其 PWM 输出通道的设置的输出使能，下面我们具体来看一下该函数的实现：

```
/**
 * @brief       PWM DAC 初始化, 实际上就是初始化定时器
 * @note
 *              定时器的时钟来自 APB1 / APB2, 当 APB1 / APB2 分频时, 定时器频率自动翻倍
 *              所以, 一般情况下, 我们所有定时器的频率, 都是 200Mhz（APB 时钟 = 100Mhz）
 *              定时器溢出时间计算方法: Tout = ((arr + 1) * (psc + 1)) / Ft us.
 *              Ft = 定时器工作频率, 单位: Mhz
 *
 * @param       arr: 自动重装值。
 * @param       psc: 时钟预分频数
 * @retval      无
 */
void pwmdac_init(uint16_t arr, uint16_t psc)
{
    g_tim15_handle.Instance = PWMDAC_TIMX;                  /* 定时器 15 */
    g_tim15_handle.Init.Prescaler = psc;                    /* 定时器分频 */
    g_tim15_handle.Init.CounterMode = TIM_COUNTERMODE_UP;   /* 向上计数模式 */
    g_tim15_handle.Init.Period = arr;                       /* 自动重装载值 */
    HAL_TIM_PWM_Init(&g_tim15_handle);                      /* 初始化 PWM */

    g_tim15_ch2handle.OCMode = TIM_OCMODE_PWM1;             /* CH1/2 PWM 模式 1 */
    /* 设置比较值,此值用来确定占空比，默认比较值为自动重装载值的一半，即占空比为 50% */
    g_tim15_ch2handle.Pulse = arr / 2;
    g_tim15_ch2handle.OCPolarity = TIM_OCPOLARITY_HIGH;     /* 输出比较极性为高 */
    /* 配置 TIM15 通道 2 */
    HAL_TIM_PWM_ConfigChannel(&g_tim15_handle, &g_tim15_ch2handle,
                                                PWMDAC_TIMX_CHY);

    HAL_TIM_PWM_Start(&g_tim15_handle, PWMDAC_TIMX_CHY);    /* 开启 PWM 通道 2 */
}
```

调用时 HAL_TIM_PWM_Init 会调用回调接口 HAL_TIM_PWM_MspInit，同样地，我们把用到的 DAC 引脚的初始化在回调函数里完成，代码如下：

```
/**
 * @brief       定时器底层驱动，时钟使能，引脚配置
 * @note
 *              此函数会被 HAL_TIM_PWM_Init()调用
 * @param       htim:定时器句柄
 * @retval      无
 */
void HAL_TIM_PWM_MspInit(TIM_HandleTypeDef *htim)
{
    GPIO_InitTypeDef gpio_init_struct;

    if (htim->Instance == PWMDAC_TIMX)
    {
        PWMDAC_TIMX_CLK_ENABLE();           /* 使能定时器 */
        PWMDAC_GPIO_CLK_ENABLE();           /* PWM DAC GPIO 时钟使能 */

        gpio_init_struct.Pin = PWMDAC_GPIO_PIN;
        gpio_init_struct.Mode = GPIO_MODE_AF_PP;
        gpio_init_struct.Pull = GPIO_PULLUP;
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_LOW;
        gpio_init_struct.Alternate = PWMDAC_GPIO_AFTIMX;
        HAL_GPIO_Init(PWMDAC_GPIO_PORT, &gpio_init_struct);
        /* TIMX PWM CHY 引脚模式设置 */
    }
}
```

我们需要通过重设占空比来调整 DAC 的输出有效值，为了方便设置电压，需要把目标电

压作为我们的形参，我们把参数放大 100 倍设成整型而不使用浮点数来加快运算。根据需要的比例来调整输出比较值来达到重设占空比的目的，所以设计函数如下：

```
/**
 * @brief       设置 PWM DAC 输出电压
 * @param       vol : 0~3300,代表 0~3.3V
 * @retval      无
 */
void pwmdac_set_voltage(uint16_t vol)
{
    float temp = vol;

    temp /= 100;                /* 缩小 100 倍，得到实际电压值 */
    temp = temp * 256 / 3.3f;   /* 将电压转换成 PWM 占空比 */
    /* 设置新的占空比 */
    __HAL_TIM_SET_COMPARE(&g_tim15_handle, PWMDAC_TIMX_CHY, temp);
}
```

本部分用定时器输出 PWM 的知识可以参考我们定时器 PWM 关联的章节，我们需要关心的就是 PWM 的精度设计的部分的知识。

2. main.c 代码

在 main.c 文件代码如下：

```
extern TIM_HandleTypeDef g_tim15_handle;

int main(void)
{
    uint16_t adcx;
    float temp;
    uint8_t t = 0;
    uint8_t key;
    uint16_t pwmval = 0;
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟,400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 初始化 USART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    adc_init();                             /* 初始化 ADC */
    pwmdac_init(256 - 1, 1);    /* PWM DAC 初始化，Fpwm = 200 / 256 =781.25Khz */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "PWM DAC TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY_UP:+  KEY1:-", RED);
    lcd_show_string(30, 130, 200, 16, 16, "PWM VAL:", BLUE);
    /* 先在固定位置显示小数点 */
    lcd_show_string(30, 150, 200, 16, 16, "DAC VOL:0.000V", BLUE);
    lcd_show_string(30, 170, 200, 16, 16, "ADC VOL:0.000V", BLUE);

    /* 输出新的 PWM 占空比 */
    __HAL_TIM_SET_COMPARE(&g_tim15_handle, PWMDAC_TIMX_CHY, pwmval);

    while (1)
    {
        t++;
        key = key_scan(0);          /* 按键扫描 */
```

```
        if (key == WKUP_PRES)        /* PWM 占空比调高 */
        {
            if (pwmval < 250)        /* 范围限定 */
            {
                pwmval += 10;
            }
            /* 输出新的 PWM 占空比 */
            __HAL_TIM_SET_COMPARE(&g_tim15_handle, PWMDAC_TIMX_CHY, pwmval);
        }
        else if (key == KEY1_PRES)   /* PWM 占空比调低 */
        {
            if (pwmval > 10)         /* 范围限定 */
            {
                pwmval -= 10;
            }
            else
            {
                pwmval = 0;
            }
             /* 输出新的 PWM 占空比 */
             __HAL_TIM_SET_COMPARE(&g_tim15_handle, PWMDAC_TIMX_CHY, pwmval);
        }

        if (t == 10 || key == KEY1_PRES || key == WKUP_PRES)
        {    /* WKUP / KEY1 按下了, 或者定时时间到了 */
            adcx = __HAL_TIM_GET_COMPARE(&g_tim15_handle,PWMDAC_TIMX_CHY);
            /* PWM DAC 定时器输出比较值 */
            lcd_show_xnum(94, 130, adcx, 3, 16, 0, BLUE);
            /* 显示 CCRX 寄存器值 */

            temp = (float)adcx * (3.3 / 256);              /* 得到 DAC 电压值 */
            adcx = temp;
            lcd_show_xnum(94, 150, temp, 1, 16, 0, BLUE); /* 显示电压值整数部分 */
            temp -= adcx;
            temp *= 1000;
            /* 显示电压值的小数部分 */
            lcd_show_xnum(110, 150, temp, 3, 16, 0X80, BLUE);

            /* 得到 ADC1 通道 19 的转换结果 */
            adcx = adc_get_result_average(ADC_ADCX_CHY, 10);

            temp = (float)adcx*(3.3 / 65536);/* 得到 ADC 电压值(adc 是 16bit 的) */
            adcx = temp;
            lcd_show_xnum(94, 170, temp, 1, 16, 0, BLUE); /* 显示电压值整数部分 */

            temp -= adcx;
            temp *= 1000;
            lcd_show_xnum(110, 170, temp, 3, 16, 0X80, BLUE);
            /* 显示电压值的小数部分 */

            LED0_TOGGLE();  /* LED0 闪烁 */
            t = 0;
        }
        delay_ms(5);
    }
}
```

main 函数初始化了 LED 和 LCD 用于显示效果，初始化按键和 ADC 用于修改 ADC 的占空比，辅助显示 ADC。

36.4 下载验证

下载代码后，LED0 不停的闪烁，提示程序已经在运行了。我们按下 KEY_UP/KEY1 调整

PWM 的占空比，可以看到输出电压随 PWM VAL 发生变化，也可以把 PA3 和 PWM DAC 的最终输出接到示波器观察滤波后的区别。

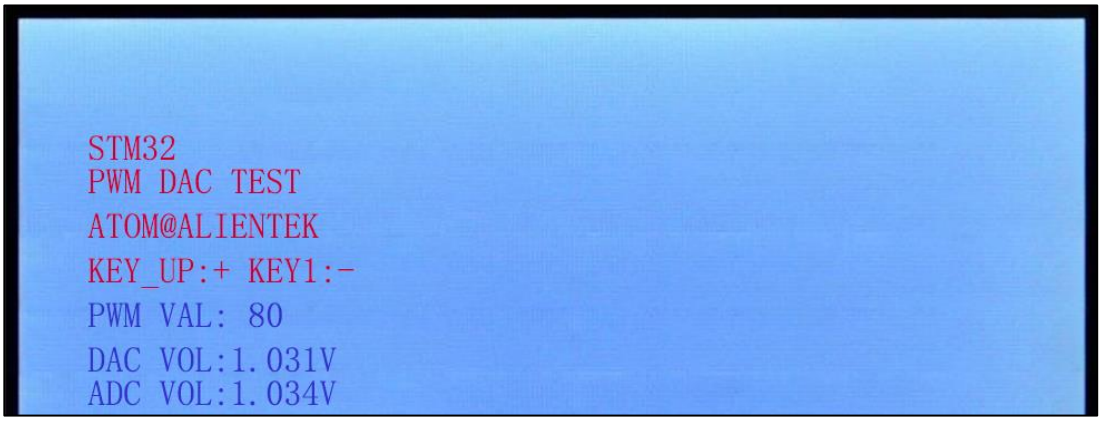


图 36.4.1 TFTLCD 显示效果图

注意：因为 PWM_DAC 和 USART2 共用了 PA3 引脚，所以在本例程不能使用串口 2 的接收功能，否则可能会影响 PWM 转换结果！！！

PWM 实现 DAC 的知识我们就讲解到这里，大家可以试试其它精度或者更换定时器输出通道测试。

第三十七章 IIC 实验

本章，我们将介绍如何使用 STM32H743 的普通 IO 口模拟 IIC 时序，并实现和 24C02 之间的双向通信，并把结果显示在 TFTLCD 模块上

本章分为如下几个小节：

37.1 IIC 及 24C02 简介

37.2 硬件设计

37.3 程序设计

37.4 下载验证

37.1 IIC 及 24C02 简介

37.1.1 IIC 简介

IIC(Inter-Integrated Circuit)总线是一种由 PHILIPS 公司开发的两线式串行总线，用于连接微控制器以及其外围设备。它是由数据线 SDA 和时钟 SCL 构成的串行总线，可发送和接收数据，在 CPU 与被控 IC 之间、IC 与 IC 之间进行双向传送。

IIC 总线有如下特点：

①总线由数据线 SDA 和时钟线 SCL 构成的串行总线，数据线用来传输数据，时钟线用来同步数据收发。

②总线上每一个器件都有一个唯一的地址识别，所以我们只需要知道器件的地址，根据时序就可以实现微控制器与器件之间的通信。

③数据线 SDA 和时钟线 SCL 都是双向线路，都通过一个电流源或上拉电阻连接到正的电压，所以当总线空闲的时候，这两条线路都是高电平。

④总线上数据的传输速率在标准模式下可达 100kbit/s 在快速模式下可达 400kbit/s 在高速模式下可达 3.4Mbit/s。

⑤总线支持设备连接。在使用 IIC 通信总线时，可以有多个具备 IIC 通信能力的设备挂载在上面，同时支持多个主机和多个从机，连接到总线的接口数量只由总线电容 400pF 的限制决定。IIC 总线挂载多个器件的示意图，如下图所示：

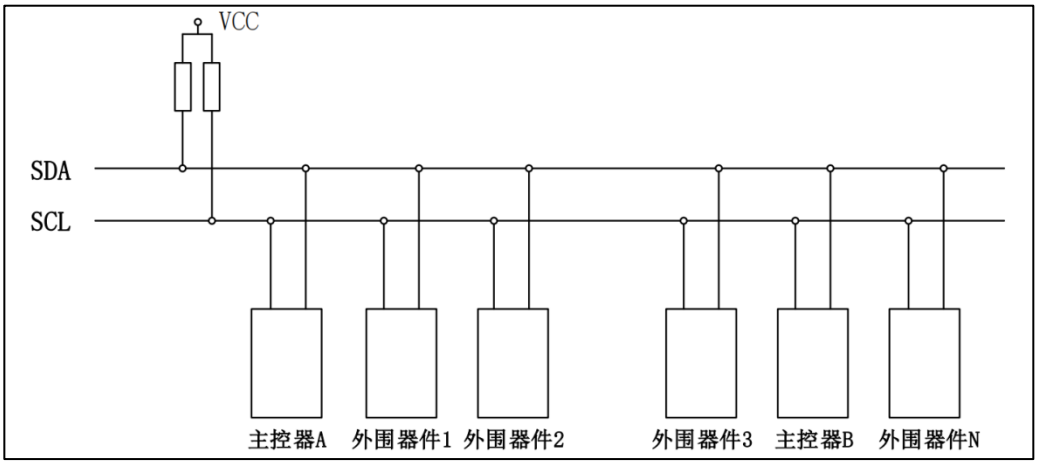


图 37.1.1.1 IIC 总线挂载多个器件

下面来学习 IIC 总线协议，IIC 总线时序图如下所示：

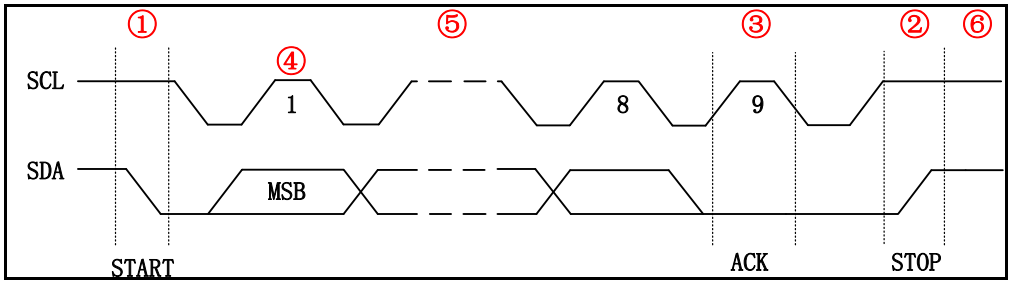


图 37.1.1.2 IIC 总线时序图

为了便于大家更好的了解 IIC 协议，我们从起始信号、停止信号、应答信号、数据有效性、数据传输以及空闲状态等 6 个方面讲解，大家需要对应图 37.1.1.2 的标号来理解。

① 起始信号

当 SCL 为高电平期间，SDA 由高到低的跳变。起始信号是一种电平跳变时序信号，而不是一个电平信号。该信号由主机发出，在起始信号产生后，总线就会处于被占用状态，准备数据传输。

② 停止信号

当 SCL 为高电平期间，SDA 由低到高的跳变。停止信号也是一种电平跳变时序信号，而不是一个电平信号。该信号由主机发出，在停止信号发出后，总线就会处于空闲状态。

③ 应答信号

发送器每发送一个字节，就在时钟脉冲 9 期间释放数据线，由接收器反馈一个应答信号。应答信号为低电平时，规定为有效应答位（ACK 简称应答位），表示接收器已经成功地接收了该字节。应答信号为高电平时，规定为非应答位（NACK），一般表示接收器接收该字节没有成功。

观察上图标号③就可以发现，有效应答的要求是从机在第 9 个时钟脉冲之前的低电平期间将 SDA 线拉低，并且确保在该时钟的高电平期间为稳定的低电平。如果接收器是主机，则在它收到最后一个字节后，发送一个 NACK 信号，以通知被控发送器结束数据发送，并释放 SDA 线，以便主机接收器发送一个停止信号。

④ 数据有效性

IIC 总线进行数据传送时，时钟信号为高电平期间，数据线上的数据必须保持稳定，只有在时钟线上的信号为低电平期间，数据线上的高电平或低电平状态才允许变化。数据在 SCL 的上升沿到来之前就需准备好。并在下降沿到来之前必须稳定。

⑤ 数据传输

在 IIC 总线上传送的每一位数据都有一个时钟脉冲相对应（或同步控制），即在 SCL 串行时钟的配合下，在 SDA 上逐位地串行传送每一位数据。数据位的传输是边沿触发。

⑥ 空闲状态

IIC 总线的 SDA 和 SCL 两条信号线同时处于高电平时，规定为总线的空闲状态。此时各个器件的输出级场效应管均处在截止状态，即释放总线，由两条信号线各自的上拉电阻把电平拉高。

了解这些知识后，下面介绍一下 IIC 的基本的读写通讯过程，包括主机写数据到从机即写操作，主机到从机读取数据即读操作。下面先看一下写操作通讯过程图，见图 37.1.1.3 所示：

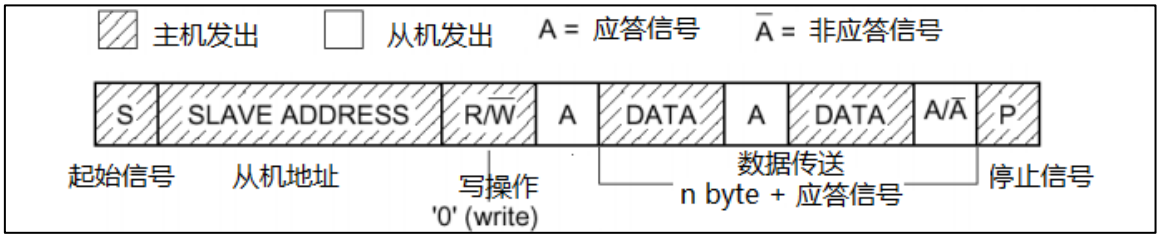


图 37.1.1.3 写操作通讯过程图

主机首先在 IIC 总线上发送起始信号，那么这时总线上的从机都会等待接收由主机发出的数据。主机接着发送从机地址+0(写操作)组成的 8bit 数据，所有从机接收到该 8bit 数据后，自行检验是否是自己的设备的地址，假如是自己的设备地址，那么从机就会发出应答信号。主机

在总线上接收到有应答信号后，才能继续向从机发送数据。注意：IIC 总线上传送的数据信号是广义的，既包括地址信号，又包括真正的数据信号。

接着讲解一下 IIC 总线的读操作过程，先看一下读操作通讯过程图，见图 37.1.1.4 所示：

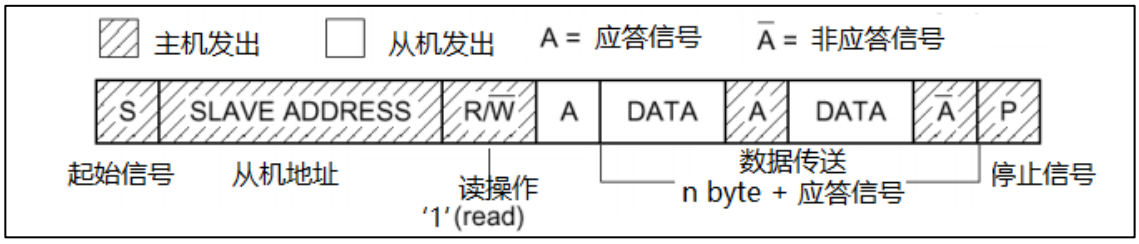


图 37.1.1.4 读操作通讯过程图

主机向从机读取数据的操作，一开始的操作与写操作有点相似，观察两个图也可以发现，都是由主机发出起始信号，接着发送从机地址+1(读操作)组成的 8bit 数据，从机接收到数据验证是否是自身的地址。 那么在验证是自己的设备地址后，从机就会发出应答信号，并向主机返回 8bit 数据，发送完之后从机就会等待主机的应答信号。假如主机一直返回应答信号，那么从机可以一直发送数据，也就是图中的（n byte + 应答信号）情况，直到主机发出非应答信号，从机才会停止发送数据。

24C02 的数据传输时序是基于 IIC 总线传输时序，下面讲解一下 24C02 的数据传输时序。

37.1.2 24C02 简介

24C02 是一个 2K bit 的串行 EEPROM 存储器，内部含有 256 个字节。在 24C02 里面还有一个 8 字节的页写缓冲器。该设备的通信方式 IIC，通过其 SCL 和 SDA 与其他设备通信，芯片的引脚图如图 37.1.2.1 所示。

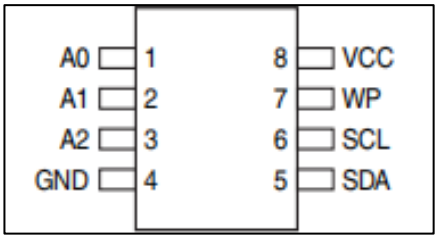


图 37.1.2.1 24C02 引脚图

上图中有一个 WP，这个是写保护引脚，接高电平只读，接地允许读和写，我们的板子设计是把该引脚接地。每一个设备都有自己的设备地址，24C02 也不例外，但是 24C02 的设备地址是包括不可编程部分和可编程部分，可编程部分是根据上图的硬件引脚 A0、A1 和 A2 所决定。设备地址最后一位用于设置数据的传输方向，即读操作/写操作，0 是写操作，1 是读操作，具体格式如下图 37.1.2.2 所示：

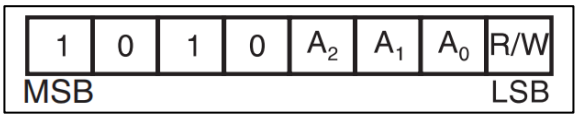


图 37.1.2.2 24C02 设备地址格式图

根据我们的板子设计，A0、A1 和 A2 均接地处理，所以 24C02 设备的读操作地址为：0xA1；写操作地址为：0xA0。

在前面已经说过 IIC 总线的基本读写操作，那么我们就可以基于 IIC 总线的时序的上，理解 24C02 的数据传输时序。

下面把实验中到的数据传输时序讲解一下，分别是对 24C02 的写时序和读时序。24C02 写时序图见图 37.1.2.3 所示。

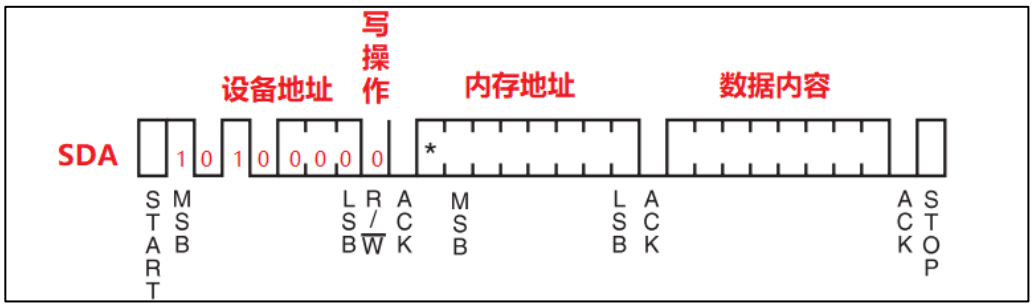


图 37.1.2.3 24C02 写时序图

上图展示的主机向 24C02 写操作时序图，主机在 IIC 总线发送第 1 个字节的数据为 24C02 的设备地址 0xA0，用于寻找总线上找到 24C02，在获得 24C02 的应答信号之后，继续发送第 2 个字节数据，该字节数据是 24C02 的内存地址，再等到 24C02 的应答信号，主机继续发送第 3 字节数据，这里的数据即是写入在第 2 字节内存地址的数据。主机完成写操作后，可以发出停止信号，终止数据传输。

上面的写操作只能单字节写入到 24C02，效率比较低，所以 24C02 有页写入时序，大大提高了写入效率，下面看一下 24C02 页写时序图，图 37.1.2.4 所示。

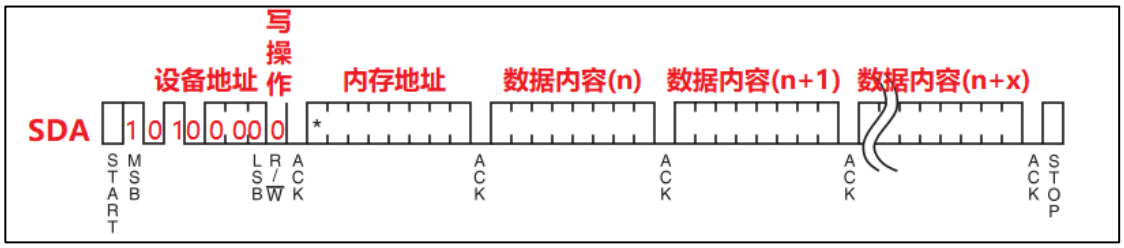


图 37.1.2.4 24C02 页写时序

在单字节写时序时，每次写入数据时都需要先写入设备的内存地址才能实现，在页写时序中，只需要告诉 24C02 第一个内存地址 1，后面数据会按照顺序写入到内存地址 2，内存地址 3 等，大大节省了通信时间，提高了时效性。因为 24C02 每次只能写 8bit 数据，所以它的页大小也就是 1 字节。页写时序的操作方式跟上面的单字节写时序差不多，所以不作过多解释了。参考以上说明去理解页写时序。

说完两种写入方式之后，下面看一下图 37.1.2.5 关于 24C02 的读时序。

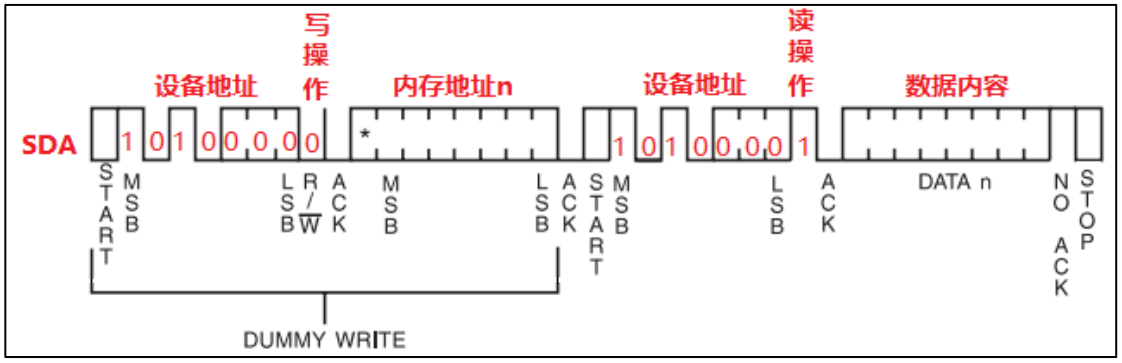


图 37.1.2.5 24C02 读时序图

24C02 读取数据的过程是一个复合的时序，其中包含写时序和读时序。先看第一个通信过程，这里是写时序，起始信号产生后，主机发送 24C02 设备地址 0xA0，获取从机应答信号后，接着发送需要读取的内存地址；在读时序中，起始信号产生后，主机发送 24C02 设备地址 0xA1,获取从机应答信号后，接着从机返回刚刚在写时序中内存地址的数据，以字节为单位传输在总线上，假如主机获取数据后返回的是应答信号，那么从机会一直传输数据，当主机发出的是非应答信号并以停止信号发出为结束，从机就会结束传输。

以上的时序的发生基于软件 IIC 的实现，不用硬件 IIC 实现，虽然 STM32H743 带有 IIC 总线接口，但是 ST 把硬件 IIC 设计得非常复杂，所以使用起来很不方便，所以我们采用软件模拟。

37.2 硬件设计

1. 例程功能

每按下 KEY1，MCU 通过 IIC 总线向 24C02 写入数据，通过按下 KEY0 来控制 24C02 读取数据。同时在 LCD 上面显示相关信息。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 : LED0 - PB1
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）独立按键
KEY0 - PH3
KEY1 - PH2
5）24C02
IIC_SCL - PH4
IIC_SDA - PH5

3. 原理图

我们主要来看看 24C02 和开发板的连接，如下图所示：

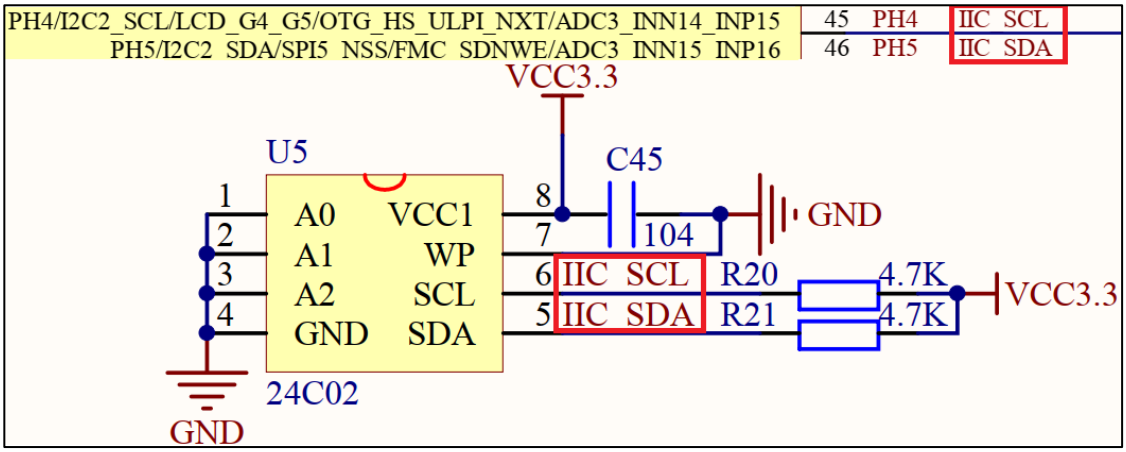


图 37.2.1 24C02 与开发板连接示意图

24C02 的 SCL 和 SDA 分别连接在 STM32 的 PH4 和 PH5 上。本实验通过软件模拟 IIC 信号建立起与 24C02 的通信，进行数据发送与接收，使用按键 KEY0 和 KEY1 去触发，LCD 屏幕进行显示。

37.3 程序设计

IIC 实验中使用的是软件模拟 IIC，所以用到的是 HAL 中 GPIO 相关函数，前面也有介绍到，这里就不做展开了。下面介绍一下使用 IIC 传输数据的配置步骤：

使用 IIC 传输数据的配置步骤

1）使能 IIC 的 SCL 和 SDA 对应的 GPIO 时钟。

本实验中 IIC 使用的 SCL 和 SDA 分别是 PH4 和 PH5，因此需要先使能 GPIOH 的时钟，代码如下：

```
__HAL_RCC_GPIOH_CLK_ENABLE();
```

2）设置对应 GPIO 工作模式（SCL 推挽输出 SDA 开漏输出）

SDA 线的 GPIO 模式使用开漏输出模式（硬件已接外部上拉电阻，也可以用内部的上拉电

阻），而 SCL 线的 GPIO 模式使用推挽输出模式，通过函数 HAL_GPIO_Init 设置实现。

3）参考 IIC 总线协议，编写信号函数（起始信号，停止信号，应答信号）

起始信号：SCL 为高电平时，SDA 由高电平向低电平跳变。

停止信号：SCL 为高电平时，SDA 由低电平向高电平跳变。

应答信号：接收到 IC 数据后，向 IC 发出特定的低电平脉冲表示已接收到数据。

4）编写 IIC 的读写函数

通过参考时序图，在一个时钟周期内发送 1bit 数据或者读取 1bit 数据。读写函数均以一字节数据进行操作。

有了读和写函数，我们就可以对外设进行驱动了。

37.3.1 程序流程图

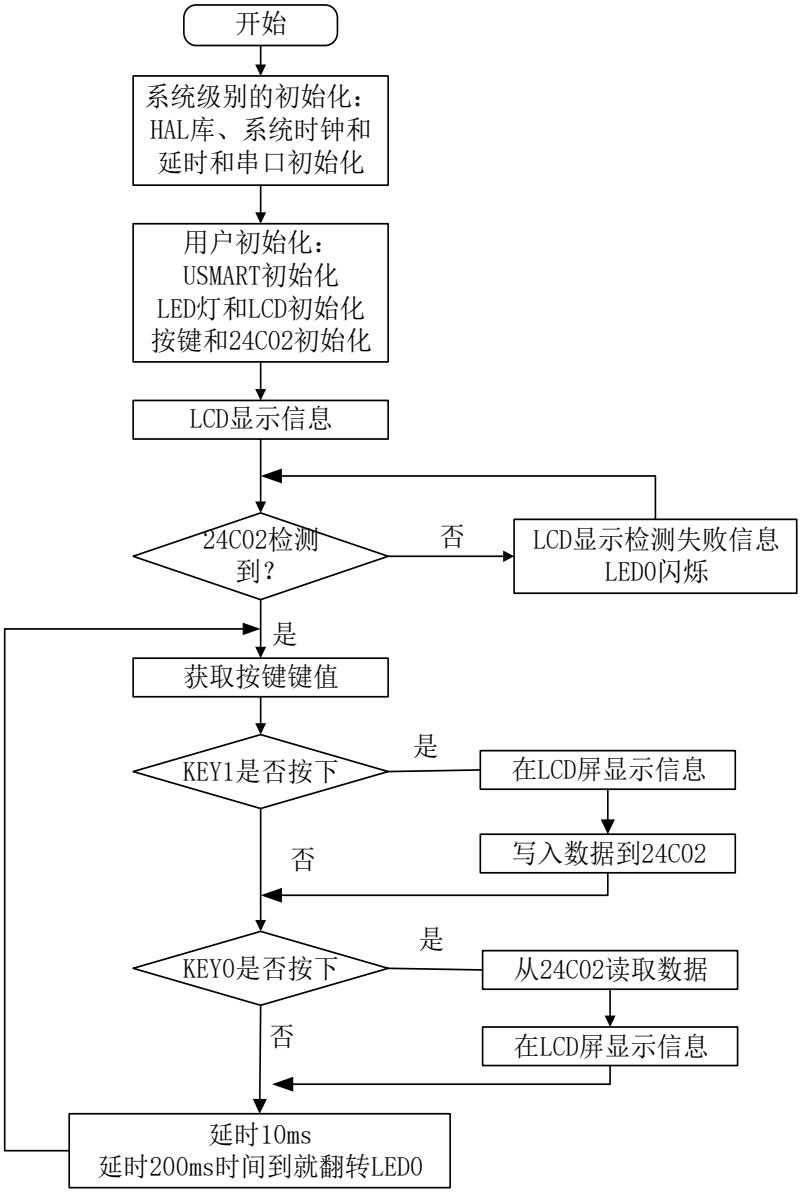


图 37.3.1.1 IIC 实验程序流程图

37.3.2 程序解析

本实验中，我们通过 GPIO 使用软件来模拟 IIC，所以不需要用到 HAL 库的 IIC 驱动源码。在工程文件中，我们新增了 myiic.c 存放 iic 底层驱动代码，24cxx.c 文件夹存放 24C02 驱动。

1. IIC 底层驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。IIC 驱动源码包括两个文件：myiic.c 和 myiic.h。

下面我们直接介绍 IIC 相关的程序，首先先介绍 myiic.h 文件，其定义如下：

```
/* 引脚 定义 */
#define IIC_SCL_GPIO_PORT           GPIOH
#define IIC_SCL_GPIO_PIN            GPIO_PIN_4
#define IIC_SCL_GPIO_CLK_ENABLE()   do{ __HAL_RCC_GPIOH_CLK_ENABLE(); }while(0)

#define IIC_SDA_GPIO_PORT           GPIOH
#define IIC_SDA_GPIO_PIN            GPIO_PIN_5
#define IIC_SDA_GPIO_CLK_ENABLE()   do{ __HAL_RCC_GPIOH_CLK_ENABLE(); }while(0)

/* IO 操作 */
#define IIC_SCL(x)     do{ x ? \
       HAL_GPIO_WritePin(IIC_SCL_GPIO_PORT, IIC_SCL_GPIO_PIN, GPIO_PIN_SET) : \
       HAL_GPIO_WritePin(IIC_SCL_GPIO_PORT, IIC_SCL_GPIO_PIN, GPIO_PIN_RESET);
\}while(0)       /* SCL */

#define IIC_SDA(x)     do{ x ? \
       HAL_GPIO_WritePin(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN, GPIO_PIN_SET) : \
       HAL_GPIO_WritePin(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN, GPIO_PIN_RESET);
\}while(0)       /* SDA */
/* 读取 SDA */
#define IIC_READ_SDA     HAL_GPIO_ReadPin(IIC_SDA_GPIO_PORT, IIC_SDA_GPIO_PIN)
```

我们通过宏定义标识符的方式去定义 SCL 和 SDA 两个引脚，同时通过宏定义的方式定义了 IIC_SCL() 和 IIC_SDA()设置这两个管脚可以输出 0 或者 1，主要还是通过 HAL 库的 GPIO 操作函数实现的。另外方便在 iic 操作函数中调用读取 SDA 管脚的数据，这里直接宏定义 IIC_READ_SDA 实现，在后面 iic 模拟信号实现中会频繁调用。

接下来我们看一下 myiic.c 代码中的初始化函数，代码如下：

```
/**
 * @brief       初始化 IIC
 * @param       无
 * @retval      无
 */
void iic_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;
    IIC_SCL_GPIO_CLK_ENABLE();      /* SCL 引脚时钟使能 */
    IIC_SDA_GPIO_CLK_ENABLE();      /* SDA 引脚时钟使能 */

    gpio_init_struct.Pin = IIC_SCL_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;            /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;                    /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;     /* 快速 */
    HAL_GPIO_Init(IIC_SCL_GPIO_PORT, &gpio_init_struct);    /* SCL */

    gpio_init_struct.Pin = IIC_SDA_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_OD;            /* 开漏输出 */
    HAL_GPIO_Init(IIC_SDA_GPIO_PORT, &gpio_init_struct);    /* SDA */
    /* SDA 引脚模式设置,开漏输出,上拉, 这样就不用再设置 IO 方向了, 开漏输出的时候(=1),
       也可以读取外部信号的高低电平 */

    iic_stop();     /* 停止总线上所有设备 */
}
```

在 iic_init 函数中主要工作就是对于 GPIO 的初始化，用于 iic 通信，不过这里需要注意的一点是 SDA 线的 GPIO 模式要使用开漏模式，特别注意：假如是 STM32F103 必须外接上拉电阻！

接下来介绍在上面已经在文字上说明过的 IIC 模拟信号：起始信号、停止信号、应答信号，

下面以代码方法实现，大家可以对着图去看代码，有利于理解。

```
/**
 * @brief       IIC 延时函数,用于控制 IIC 读写速度
 * @param       无
 * @retval      无
 */
static void iic_delay(void)
{
    delay_us(2);     /* 2us 的延时, 读写速度在 250Khz 以内 */
}

/**
 * @brief       产生 IIC 起始信号
 * @param       无
 * @retval      无
 */
void iic_start(void)
{
    IIC_SDA(1);
    IIC_SCL(1);
    iic_delay();
    IIC_SDA(0);      /* START 信号: 当 SCL 为高时, SDA 从高变成低, 表示起始信号 */
    iic_delay();
    IIC_SCL(0);      /* 钳住 I2C 总线，准备发送或接收数据 */
    iic_delay();
}

/**
 * @brief       产生 IIC 停止信号
 * @param       无
 * @retval      无
 */
void iic_stop(void)
{
    IIC_SDA(0);      /* STOP 信号: 当 SCL 为高时, SDA 从低变成高, 表示停止信号 */
    iic_delay();
    IIC_SCL(1);
    iic_delay();
    IIC_SDA(1);      /* 发送 I2C 总线结束信号 */
    iic_delay();
}
```

在这里首先定义一个 iic_delay 函数，目的就是控制 IIC 的读写速度，通过示波器检测读写速度在 250KHz 内，所以一秒钟传送 500Kb 数据，换算一下即一个 bit 位需要 2us，在这个延时时间内可以让器件进行获得一个稳定性的数据采集。

为了大家更加清晰了解代码实现的过程，下面单独把起始信号和停止信号从 iic 总线时序图中抽取出来，如图 37.3.2.1 所示：

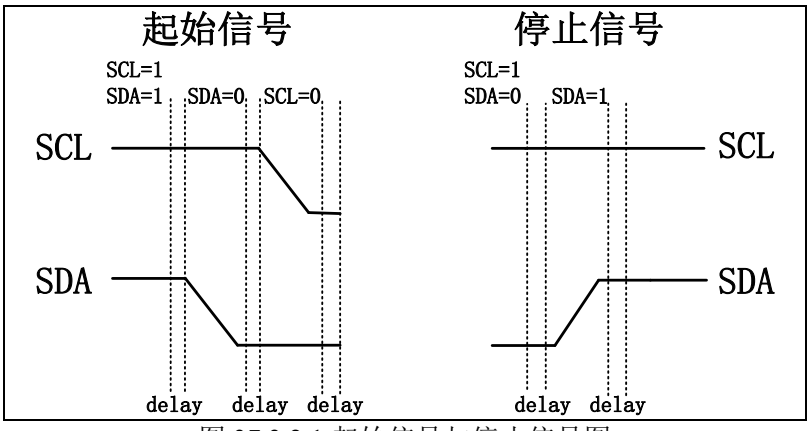


图 37.3.2.1 起始信号与停止信号图

iic_start 函数中，通过调用 myiic.h 中通过宏定义好的可以输出高低电平的 SCL 和 SDA 来模拟 iic 总线中起始信号的发送，在 SCL 时钟线为高电平的时候，SDA 数据线从高电平状态转化到低电平状态，最后拉低时钟线，准备发送或者接收数据。

iic_stop 函数中，也是按着模拟 iic 总线中停止信号的逻辑，在 SCL 时钟线为高电平的时候，SDA 数据线从低电平状态转化到高电平状态。

接下来讲解一下 iic 的发送函数，其定义如下：

```
/**
 * @brief       IIC 发送一个字节
 * @param       data: 要发送的数据
 * @retval      无
 */
void iic_send_byte(uint8_t data)
{
    uint8_t t;

    for (t = 0; t < 8; t++)
    {
        IIC_SDA((data & 0x80) >> 7);     /* 高位先发送 */
        iic_delay();
        IIC_SCL(1);
        iic_delay();
        IIC_SCL(0);
        data <<= 1;      /* 左移 1 位,用于下一次发送 */
    }
    IIC_SDA(1);          /* 发送完成, 主机释放 SDA 线 */
}
```

在 iic 的发送函数 iic_send_byte 中，我们把需要发送的数据作为形参，形参大小为 1 个字节。在 iic 总线传输中，一个时钟信号就发送一个 bit，所以该函数需要循环八次，模拟八个时钟信号，才能把形参的 8 个位数据都发送出去。这里使用的是形参 data 和 0x80 与运算的方式，判断其最高位的逻辑值，假如为 1 即需要控制 SDA 输出高电平，否则为 0 控制 SDA 输出低电平。为了更好说明，数据发送的过程，单独拿出数据传输时序图，见图 37.3.2.2。

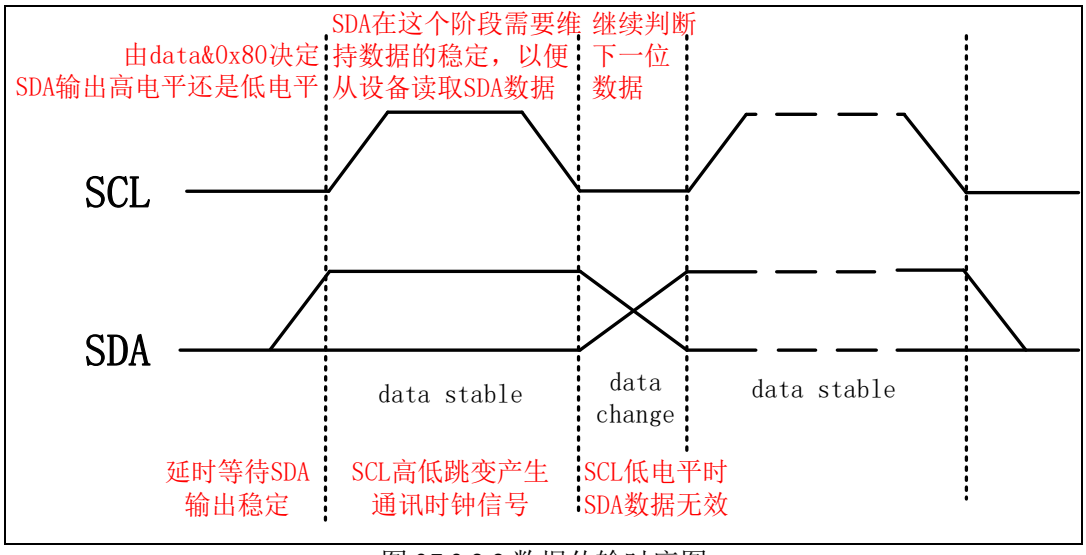


图 37.3.2.2 数据传输时序图

通过上图就可以很清楚了解数据传输时的细节，经过第一步的 SDA 高低电平的确定后，接着需要延时，确保 SDA 输出的电平稳定，在 SCL 保持高电平期间，SDA 线上的数据是有效的，此过程也是需要延时，使得从设备能够采集到有效的电平。然后准备下一位的数据，所以这里需要的是把 data 左移一位，等待下一个时钟的到来，从设备进行读取。把上述的操作重复 8 次就可以把 data 的 8 个位数据发送完毕，循环结束后，把 SDA 线拉高，等待接收从设备发送过来的应答信号。

接着讲解一下 iic 的读取函数 iic_read_byte，它的定义如下：

```
/**
 * @brief       IIC 读取一个字节
 * @param       ack:  ack=1 时，发送 ack; ack=0 时，发送 nack
 * @retval      接收到的数据
 */
uint8_t iic_read_byte(uint8_t ack)
{
    uint8_t i, receive = 0;

    for (i = 0; i < 8; i++ )     /* 接收 1 个字节数据 */
    {
        receive <<= 1;  /* 高位先输出,所以先收到的数据位要左移 */
        IIC_SCL(1);
        iic_delay();

        if (IIC_READ_SDA)
        {
            receive++;
        }

        IIC_SCL(0);
        iic_delay();
    }
    if (!ack)
    {
        iic_nack();     /* 发送 nACK */
    }
    else
    {
        iic_ack();      /* 发送 ACK */
    }

    return receive;
}
```

iic_read_byte 函数具体实现的方式跟 iic_send_byte 函数有所不同。首先可以明确的是时钟信号是通过主机发出的，而且接收到的数据大小为 1 字节，但是 IIC 传输的单位是 bit，所以就需要执行 8 次循环，才能把一字节数据接收完整。

具体实现过程：首先需要一个变量 receive 存放接收到的数据，在每一次循环开始前都需要对 receive 进行左移 1 位操作，那么 receive 的 bit0 位每一次赋值前都是空的，用来存放最新接收到的数据位，然后在 SCL 线进行高低电平切换时输出 IIC 时钟，在 SCL 高电平期间加入延时，确保有足够的时间能让数据发送并进行处理，使用宏定义 IIC_READ_SDA 就可以判断读取到的高低电平，假如 SDA 为高电平，那么 receive++即在 bit0 置 1，否则不做处理即保持原来的 0 状态。当 SCL 线拉低后，需要加入延时，便于从机切换 SDA 线输出数据。在 8 次循环结束后，我们就获得了 8bit 数据，把它作为返回值返回，然而按照时序图，作为主机就需要发送应答或者非应答信号，去回复从机。

上面提及到应答信号和非应答信号是在读时序中发生的，此外在写时序中也存在有一个信号响应，当发送完 8bit 数据后，这里是一个等待从机应答信号的操作，这里我们也定义了，下面看一下它们的定义：

```
/**
 * @brief       等待应答信号到来
 * @param       无
 * @retval      1，接收应答失败
 *              0，接收应答成功
 */
uint8_t iic_wait_ack(void)
{
    uint8_t waittime = 0;
    uint8_t rack = 0;

    IIC_SDA(1);     /* 主机释放 SDA 线(此时外部器件可以拉低 SDA 线) */
```

```
    iic_delay();
    IIC_SCL(1);      /* SCL=1, 此时从机可以返回 ACK */
    iic_delay();

    while (IIC_READ_SDA)     /* 等待应答 */
    {
        waittime++;

        if (waittime > 250)
        {
            iic_stop();
            rack = 1;
            break;
        }
    }

    IIC_SCL(0);      /* SCL=0, 结束 ACK 检查 */
    iic_delay();
    return rack;
}

/**
 * @brief       产生 ACK 应答
 * @param       无
 * @retval      无
 */
void iic_ack(void)
{
    IIC_SDA(0);      /* SCL 0 -> 1  时 SDA = 0,表示应答 */
    iic_delay();
    IIC_SCL(1);      /* 产生一个时钟 */
    iic_delay();
    IIC_SCL(0);
    iic_delay();
    IIC_SDA(1);      /* 主机释放 SDA 线 */
    iic_delay();
}

/**
 * @brief       不产生 ACK 应答
 * @param       无
 * @retval      无
 */
void iic_nack(void)
{
    IIC_SDA(1);      /* SCL 0 -> 1  时 SDA = 1,表示不应答 */
    iic_delay();
    IIC_SCL(1);      /* 产生一个时钟 */
    iic_delay();
    IIC_SCL(0);
    iic_delay();
}
```

首先先讲解一下 iic_wait_ack 函数，该函数主要用在写时序中，当启动起始信号，发送完 8bit 数据到从机时，我们就需要等待以及处理接收从机发送过来的响应信号或者非响应信号，一般就是在 iic_send_byte 函数后面调用。

具体实现：首先先释放 SDA，把电平拉高，延时等待从机操作 SDA 线，然后主机拉高时钟线并延时，确保有充足的时间让主机接收到从机发出的 SDA 信号，这里使用的是 IIC_READ_SDA 宏定义去读取，根据 IIC 协议，主机读取 SDA 的值为低电平，就表示“应答信号”；读到 SDA 的值为高电平，就表示“非应答信号”。在这个等待读取的过程中加入了超时判断，假如超过这个时间没有接收到数据，那么主机直接发出停止信号，跳出循环，返回等于 1 的变量。在正常等待到应答信号后，主机会把 SCL 时钟线拉低并延时，返回是否接收到应

答信号。

当主机作为接收端时，调用 iic_read_byte 函数之后，按照 iic 通信协议，需要给从机返回应答或者是非应答信号，这里就是用到了 iic_ack 和 iic_nack 函数。

具体实现：从上面的说明已经知道了 SDA 为低电平即应答信号，高电平即非应答信号，那么还是老规矩，首先先根据返回“应答”或者“非应答”两种情况拉低或者拉高 SDA，并延时等待 SDA 电平稳定，然后主机拉高 SCL 线并延时，确保从机能有足够时间去接收 SDA 线上的电平信号。然后主机拉低时钟线并延时，完成这一位数据的传送。最后把 SDA 拉高，呈高阻态，方便后续通信用到。

2. 24C02 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。24CXX 驱动源码包括两个文件：24cxx.c 和 24cxx.h。

在上一小节已经对 IIC 协议中的需要用到的信号都用函数封装好了，那么现在就要定义符合 24C02 时序的函数。为了使代码功能更加健全，所以在 24cxx.h 中宏定义了不同容量大小的 24C 系列型号，具体定义如下：

```
#define AT24C01     127
#define AT24C02     255
#define AT24C04     511
#define AT24C08     1023
#define AT24C16     2047
#define AT24C32     4095
#define AT24C64     8191
#define AT24C128    16383
#define AT24C256    32767

/* 开发板使用的是 24c02，所以定义 EE_TYPE 为 AT24C02 */
#define EE_TYPE     AT24C02
```

在 24cxx.c 文件中，读/写操作函数对于不同容量大小的 24Cxx 芯片都有相对应的代码块解决处理。下面先看一下 at24cxx_write_one_byte 函数，实现在 AT24Cxx 芯片指定地址写入一个数据，代码如下：

```
/**
 * @brief       在 AT24CXX 指定地址写入一个数据
 * @param       addr: 写入数据的目的地址
 * @param       data: 要写入的数据
 * @retval      无
 */
void at24cxx_write_one_byte(uint16_t addr, uint8_t data)
{
    /* 原理说明见:at24cxx_read_one_byte 函数, 本函数完全类似 */
    iic_start();                    /* 发送起始信号 */

    if (EE_TYPE > AT24C16)          /* 24C16 以上的型号, 分 2 个字节发送地址 */
    {
        iic_send_byte(0XA0);        /* 发送写命令, IIC 规定最低位是 0, 表示写入 */
        iic_wait_ack();             /* 每次发送完一个字节,都要等待 ACK */
        iic_send_byte(addr >> 8);   /* 发送高字节地址 */
    }
    else
    {   /* 发送器件 0XA0 + 高位 a8/a9/a10 地址,写数据 */
        iic_send_byte(0XA0 + ((addr >> 8) << 1));
    }

    iic_wait_ack();             /* 每次发送完一个字节,都要等待 ACK */
    iic_send_byte(addr % 256);  /* 发送低位地址 */
    iic_wait_ack();             /* 等待 ACK, 此时地址发送完成了 */

    /* 因为写数据的时候,不需要进入接收模式了,所以这里不用重新发送起始信号了 */
```

```
    iic_send_byte(data);            /* 发送 1 字节 */
    iic_wait_ack();                 /* 等待 ACK */
    iic_stop();                     /* 产生一个停止条件 */
    delay_ms(10);           /* 注意: EEPROM 写入比较慢,必须等到 10ms 后再写下一个字节 */
}
```

该函数的操作流程跟前面已经分析过的 24C02 单字节写时序一样，首先调用 iic_start 函数产生起始信号，然后调用 iic_send_byte 函数发送第 1 个字节数据设备地址，等待 24Cxx 设备返回应答信号；收到应答信号后，继续发送第 2 个 1 字节数据内存地址 addr；等待接收应答后，最后发送第 3 个字节数据写入内存地址的数据 data，24Cxx 设备接收完数据，返回应答信号，主机调用 iic_stop 函数产生停止信号终止数据传输，最终需要延时 10ms，等待 EEPROM 写入完毕。

我们的函数兼容 24Cxx 系列多种容量，就在发送设备地址处做了处理，这里说一下为什么需要这样子设计。大家请看一下 24Cxx 芯片内存组织表，见表 37.3.2.1 所示。

芯片	页数	每页字节数	总的字节数	字节寻址地址线数量
AT24C01A	16	8	128	7
AT24C02	32	8	256	8
AT24C04	32	16	512	9
AT24C08A	64	16	1024	10
AT24C16A	128	16	2048	11
AT24C32	128	32	4096	12
AT24C64A	256	32	8192	13

表 37.3.2.1 24Cxx 芯片内存组织表

主机发送的设备地址和内存地址共同确定了要写入的地方，这里分析一下 24C16 的使用的是 iic_send_byte(0XA0+((addr>>8)<<1))和 iic_send_byte(addr % 256)确定写入位置，由于它内存大小一共 2048 字节，所以只需要定义 11 个寻址地址线，2048 = 2^11。主机下发读写命令的时候带了 3 位，后面再跟 1 个字节（8 位）的地址，正好 11 位，就不需要再发后续的地址字节了。

而容量大于 24C16 的芯片，需要单独发送 2 个字节（甚至更多）的地址，如 24C32，它的大小为 4096，需要 12 个寻址地址线支持，4096 = 2^12。24C16 是 2 个字节刚刚好，而它需要三个字节才能确定写入的位置。24C32 芯片规定设备写地址 0xA0/读地址 0xA1，后面接着发送 8 位高地址，最后才发送 8 位低地址。与函数里面的操作是一致。

接下来看一下 at24cxx_read_one_byte 函数，其定义如下：

```
/**
 * @brief       在 AT24CXX 指定地址读出一个数据
 * @param       addr: 开始读数的地址
 * @retval      读到的数据
 */
uint8_t at24cxx_read_one_byte(uint16_t addr)
{
    uint8_t temp = 0;
    iic_start();                    /* 发送起始信号 */

    /* 根据不同的 24CXX 型号, 发送高位地址
     * 1, 24C16 以上的型号, 分 2 个字节发送地址
     * 2, 24C16 及以下的型号, 分 1 个低字节地址 + 占用器件地址的 bit1~bit3 位 用于表示高位
        地址, 最多 11 位地址
     *    对于 24C01/02, 其器件地址格式(8bit)为: 1  0  1  0  A2  A1  A0  R/W
     *    对于 24C04,    其器件地址格式(8bit)为: 1  0  1  0  A2  A1  a8  R/W
     *    对于 24C08,    其器件地址格式(8bit)为: 1  0  1  0  A2  a9  a8  R/W
     *    对于 24C16,    其器件地址格式(8bit)为: 1  0  1  0  a10 a9  a8  R/W
     *    R/W       : 读/写控制位 0,表示写; 1,表示读;
     *    A0/A1/A2  : 对应器件的 1,2,3 引脚(只有 24C01/02/04/8 有这些脚)
     *    a8/a9/a10: 对应存储整列的高位地址, 11bit 地址最多可以表示 2048 个位置,可以寻址
        24C16 及以内的型号
     */
```

```
    if (EE_TYPE > AT24C16)          /* 24C16 以上的型号, 分 2 个字节发送地址 */
    {
        iic_send_byte(0XA0);       /* 发送写命令, IIC 规定最低位是 0, 表示写入 */
        iic_wait_ack();            /* 每次发送完一个字节,都要等待 ACK */
        iic_send_byte(addr >> 8);/* 发送高字节地址 */
    }
    else
    {
       /* 发送器件 0XA0 + 高位 a8/a9/a10 地址,写数据 */
        iic_send_byte(0XA0 + ((addr >> 8) << 1));
    }

    iic_wait_ack();                 /* 每次发送完一个字节,都要等待 ACK */
    iic_send_byte(addr % 256);      /* 发送低位地址 */
    iic_wait_ack();                 /* 等待 ACK, 此时地址发送完成了 */

    iic_start();                    /* 重新发送起始信号 */
    iic_send_byte(0XA1);            /* 进入接收模式, IIC 规定最低位是 0, 表示读取 */
    iic_wait_ack();                 /* 每次发送完一个字节,都要等待 ACK */
    temp = iic_read_byte(0);        /* 接收一个字节数据 */
    iic_stop();                     /* 产生一个停止条件 */
    return temp;
}
```

这里的函数的实现跟前面第 37.1.2 小节 24C02 数据传输中的读时序一致，主机首先调用 iic_start 函数产生起始信号，然后调用 iic_send_byte 函数发送第 1 个字节数据设备写地址，使用 iic_wait_ack 函数等待 24Cxx 设备返回应答信号；收到应答信号后，继续发送第 2 个 1 字节数据内存地址 addr；等待接收应答后，重新调用 iic_start 函数产生起始信号，这一次的设备方向改变了，调用 iic_send_byte 函数发送设备读地址，然后使用 iic_wait_ack 函数去等待设备返回应答信号，同时使用 iic_read_byte 去读取从从机发出来的数据。由于 iic_read_byte 函数的形参是 0，所以在获取完 1 个字节的数据后，主机发送非应答信号，停止数据传输，最终调用 iic_stop 函数产生停止信号，返回从从机 addr 中读取到的数据。

为了方便检测 24Cxx 芯片是否正常工作，在这里也定义了一个检测函数，代码如下：

```
/**
 * @brief       检查 AT24CXX 是否正常
 * @note        检测原理: 在器件的末地址写如 0X55, 然后再读取, 如果读取值为 0X55
 *              则表示检测正常. 否则,则表示检测失败.
 *
 * @param       无
 * @retval      检测结果
 *                  0: 检测成功
 *                  1: 检测失败
 */
uint8_t at24cxx_check(void)
{
    uint8_t temp;
    uint16_t addr = EE_TYPE;
    temp = at24cxx_read_one_byte(addr); /* 避免每次开机都写 AT24CXX */

    if (temp == 0X55)   /* 读取数据正常 */
    {
        return 0;
    }
    else    /* 排除第一次初始化的情况 */
    {
        at24cxx_write_one_byte(addr, 0X55); /* 先写入数据 */
        temp = at24cxx_read_one_byte(255);  /* 再读取数据 */
        if (temp == 0X55)return 0;
    }
    return 1;
}
```

学到这个地方相信大家，对于这个操作并不陌生了，在前面的 RTC 实验也有相似的操作，可以翻回去看看。这里利用的是 EEPROM 芯片掉电不丢失的特性，在第一次写入了某个值之后，再去读一下是否写入成功，这种方式去检测芯片是否正常工作。

此外方便多字节写入和读取，还定义了在指定地址读取指定个数的函数以及在指令地址写入指定个数的函数，代码如下：

```
/**
 * @brief           在 AT24CXX 里面的指定地址开始读出指定个数的数据
 * @param           addr    : 开始读出的地址 对 24c02 为 0~255
 * @param           pbuf    : 数据数组首地址
 * @param           datalen : 要读出数据的个数
 * @retval          无
 */
void at24cxx_read(uint16_t addr, uint8_t *pbuf, uint16_t datalen)
{
    while (datalen--)
    {
        *pbuf++ = at24cxx_read_one_byte(addr++);
    }
}

/**
 * @brief           在 AT24CXX 里面的指定地址开始写入指定个数的数据
 * @param           addr    : 开始写入的地址 对 24c02 为 0~255
 * @param           pbuf    : 数据数组首地址
 * @param           datalen : 要写入数据的个数
 * @retval          无
 */
void at24cxx_write(uint16_t addr, uint8_t *pbuf, uint16_t datalen)
{
    while (datalen--)
    {
        at24cxx_write_one_byte(addr, *pbuf);
        addr++;
        pbuf++;
    }
}
```

对于这两个函数都是调用前面的单字节操作函数去实现的，利用 for 循环，连续调用单字节操作函数去实现，这里就不多讲。

3. main.c 代码

在 main.c 里面编写如下代码：

```
/* 要写入到 24c02 的字符串数组 */
const uint8_t g_text_buf[] = {"STM32 IIC TEST"};
#define TEXT_SIZE       sizeof(g_text_buf)  /* TEXT 字符串长度 */

int main(void)
{
    uint8_t key;
    uint16_t i = 0;
    uint8_t datatemp[TEXT_SIZE];

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
```

```
    sdram_init();                               /* 初始化 SDRAM */
    lcd_init();                                 /* 初始化 LCD */
    key_init();                                 /* 初始化按键 */
    at24cxx_init();                             /* 初始化 24CXX */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "IIC TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    /* 显示提示信息 */
    lcd_show_string(30, 110, 200, 16, 16, "KEY1:Write  KEY0:Read", RED);

    while (at24cxx_check()) /* 检测不到 24c02 */
    {
        lcd_show_string(30, 130, 200, 16, 16, "24C02 Check Failed!", RED);
        delay_ms(500);
        lcd_show_string(30, 130, 200, 16, 16, "Please Check!      ", RED);
        delay_ms(500);
        LED0_TOGGLE();      /* 红灯闪烁 */
    }

    lcd_show_string(30, 130, 200, 16, 16, "24C02 Ready!", RED);

    while (1)
    {
        key = key_scan(0);

        if (key == KEY1_PRES)    /* KEY1 按下,写入 24C02 */
        {
            lcd_fill(0, 150, 239, 319, WHITE);  /* 清除半屏 */
            lcd_show_string(30,150,200,16,16,"Start Write 24C02....", BLUE);
            at24cxx_write(0, (uint8_t *)g_text_buf, TEXT_SIZE);
            /* 提示传送完成 */
            lcd_show_string(30,150,200,16,16,"24C02 Write Finished!", BLUE);
        }

        if (key == KEY0_PRES)    /* KEY0 按下,读取字符串并显示 */
        {
            lcd_show_string(30,150,200,16,16,"Start Read 24C02.... ", BLUE);
            at24cxx_read(0, datatemp, TEXT_SIZE);
            /* 提示传送完成 */
            lcd_show_string(30,150,200,16,16, "The Data Readed Is:  ", BLUE);
            /* 显示读到的字符串 */
            lcd_show_string(30, 170, 200, 16, 16, (char *)datatemp, BLUE);
        }
        i++;
        if (i == 20)
        {
            LED0_TOGGLE();  /* 红灯闪烁 */
            i = 0;
        }
        delay_ms(10);
    }
}
```

main 函数的流程大致是：在 main 函数外部定义要写入 24C02 的字符串数组 g_text_buf。在完成系统级和用户级初始化工作后，检测 24c02 是否存在，然后通过 KEY0 去读取 0 地址存放得数据并把数据显示在 LCD 上；另外还可以通过 KEY1 去 0 地址处写入 g_text_buf 数据并在 LCD 界面中显示传输中，完成后并显示“24C02 Write Finished!”。

37.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了，先按下 KEY1 写入数据，然后再按 KEY0 读取数据，最终 LCD 显示的内容如图 37.4.1 所示：

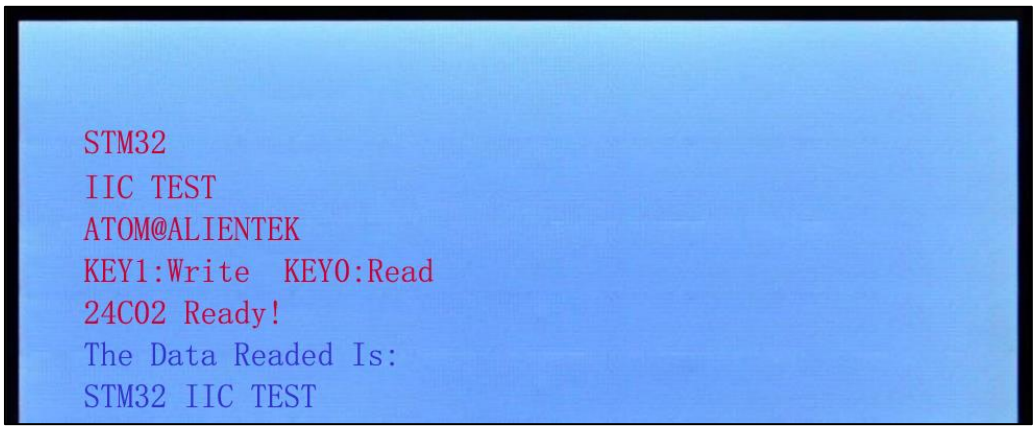


图 37.4.1 IIC 实验程序运行效果图

假如大家需要验证 24C02 的自检函数，可以用跟杜邦线把 PH4 和 PH5 短接，看是否能看到报错。

该实验还支持 USMART，在这里我们可以方便测试 24C02 的读写功能，可以操作 24C02 的任意地址，不过在 0~255 这个范围，读写测试图如图 37.4.2 所示。

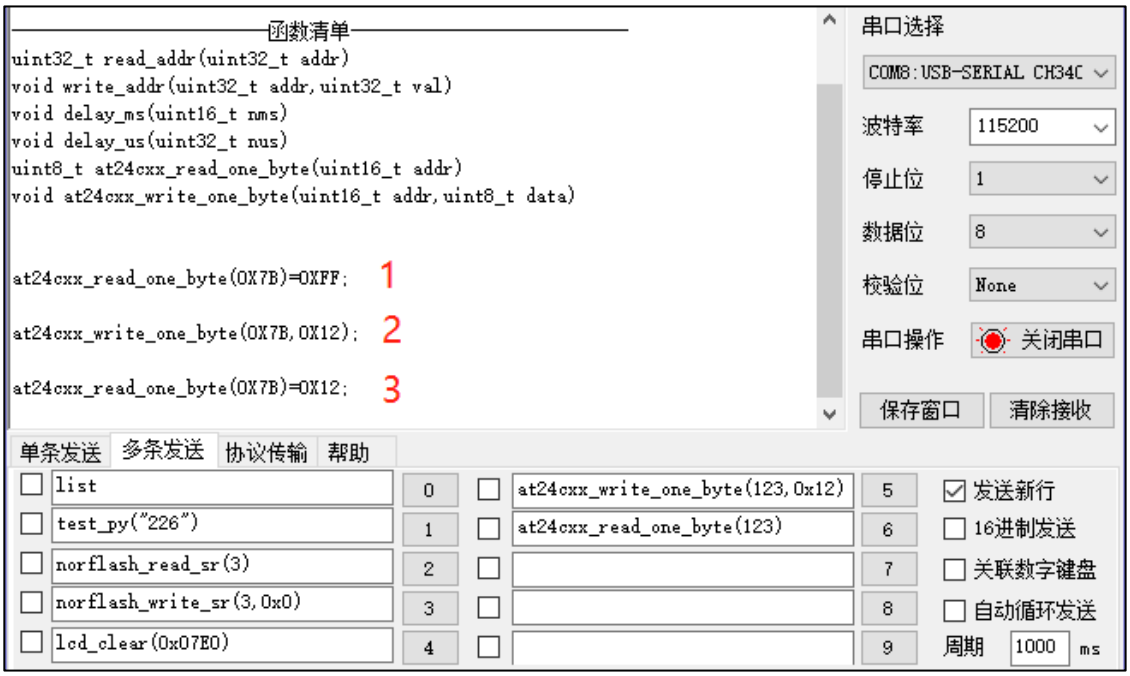


图 37.4.2 24C02 读写测试图

图中，我们首先调用 at24cxx_read_one_byte 函数在 123 地址处读取数据，获取到的值为 0XFF。然后通过调用 at24cxx_write_one_byte 函数在 123 地址处写入的值为 0x12，最后继续调用 at24cxx_read_one_byte 函数在 123 地址处读取数据，获取到的值为 0x12，表明实验成功。

至此，我们整个 IIC 实验就结束了，本章内容比较多，需要大家花多点时间去理解，一定要自己去用一下 IIC 通信协议。市面上很多器件都是具有 IIC 通信接口的，可以尝试去驱动它们，这样才能学以致用。

第三十八章 IO 扩展实验

上一章，我们介绍了 IIC 驱动 24C02，本章我们将向大家介绍如何使用 IIC 来扩展 IO 口。在本章中，我们将使用 STM32H743 的普通 IO 口模拟 IIC 时序，来驱动 PCF8574T，达到扩展 IO 口的目的。

本章分为如下几个小节：

38.1 PCF8574 简介
38.2 硬件设计
38.3 程序设计
38.4 下载验证

38.1 PCF8574 简介

PCF8574 是飞利浦公司推出的一款带 IIC 总线，可使大多数 MCU 实现远程 I/O 扩展。该器件包含一个 8 位准双向口和一个 IIC 总线接口。PCF8574 电流消耗很低且口输出锁存具有大电流驱动能力可直接驱动 LED，它还带有一条中断接线（INT）可与 MCU 的中断逻辑相连，通过 INT 发送中断信号，远端 I/O 口不必经过 IIC 总线通信就可通知 MCU 是否有数据从端口输入，这意味着 PCF8574 可以作为一个单被控器。

PCF8574 有如下特性：

- 支持 2.5~6.0V 操作电压
- 低备用电流（≤10uA）
- 支持开漏中断输出
- 支持 IIC 总线扩展 8 路 I/O 口
- 口输出锁存具有大电流驱动能力可直接驱动 LED
- 通过 3 个硬件地址引脚可寻址 8 个器件（F8574A 可多达 16 个）

1.引脚说明

PCF8574 的引脚说明如表 38.1.1 所示：

标号	管脚 SO16	描述
A0	1	地址输入 0
A1	2	地址输入 1
A2	3	地址输入 2
P0	4	准双向 I/O 口 0
P1	5	准双向 I/O 口 1
P2	6	准双向 I/O 口 2
P3	7	准双向 I/O 口 3
V_{SS}	8	地
P4	9	准双向 I/O 口 4
P5	10	准双向 I/O 口 5
P6	11	准双向 I/O 口 6
P7	12	准双向 I/O 口 7
$\overline{INT}$	13	中断输入（低电平有效）
SCL	14	串行时钟线
SDA	15	串行数据线
V_{DD}	16	电源

表 38.1.1 PCF8574 引脚说明

我们使用的 PCF8574 采用 SO16 封装，总共 16 个脚，其中包括：8 个准双向 IO 口（P0~P7）、3 个地址线（A0~A2）、SCL、SDA、INT、VDD 和 VSS。每个 PCF8574T 只需要最少 2 个 IO

口，就可以扩展 8 路 IO，且支持一个 IIC 总线上挂最多 8 个 PCF8574T，这样通过 2 个 IO，最多可以扩展 64 个 IO 口，在 MCU 的 IO 不够用的时候，PCF8574 是一个非常不错的 IO 扩展方案。

2.寻址

一个 IIC 总线上，最多可以挂 8 个 PCF8574T（通过 A0~A2 寻址），PCF8574T 的从机地址格式如图 38.1.1 所示：

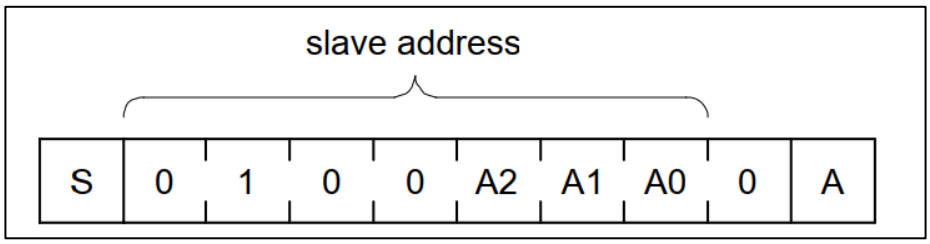


图 38.1.1 PCF8574T 从机地址格式

图中的 S 代表 IIC 的 start 信号（启动信号）；A 代表 PCF8574T 发出的应答信号；A0~A2 为 PCF8574T 的寻址信息，我们开发板上 A0~A2 都是接 GND 的，所以，PCF8574T 的地址为：0X40（左移了一位）；R/W 为读/写控制位，R/W=0 的时候，表示写数据到 PCF8574T，输出到 P0~P7 口，R/W=1 的时候，表示读取 PCF8574T 的数据，获取 P0~P7 的 IO 口状态。

关于 IIC 协议的介绍，请参考上一章节。

3.写数据（输出）

PCF8574T 的写数据时序如图 38.1.2 所示：

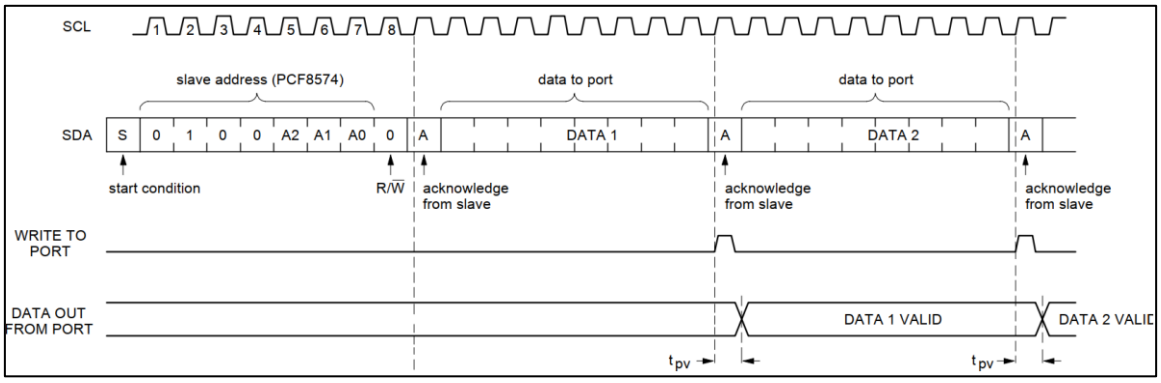


图 38.1.2 PCF8574 写数据时序

由图可知，PCF8574 的数据写入非常简单，首先发送 PCF8574T 的从机地址+写信号（R/W=0），然后等待 PCF8574 的应答信号，在应答成功后，发送数据（DATA1）给 PCF8574T 就可以了，发送完数据，会受到 PCF8574 的应答信号，在发送应答信号的同时，PCF8574T 会将接收到的数据（DATA1）输出到 P0~P7 上面（对应关系见上图）。注意：图中的 WRITE TO PORT 信号是 PCF8574T 内部自己产生的，它每次发送应答的同时产生，用于将刚刚接收到的数据，输出到 P0~P7 上，此信号不需要 MCU 发送。

4.读数据（输入）

PCF8574T 的读数据时序如图 38.1.3 所示：

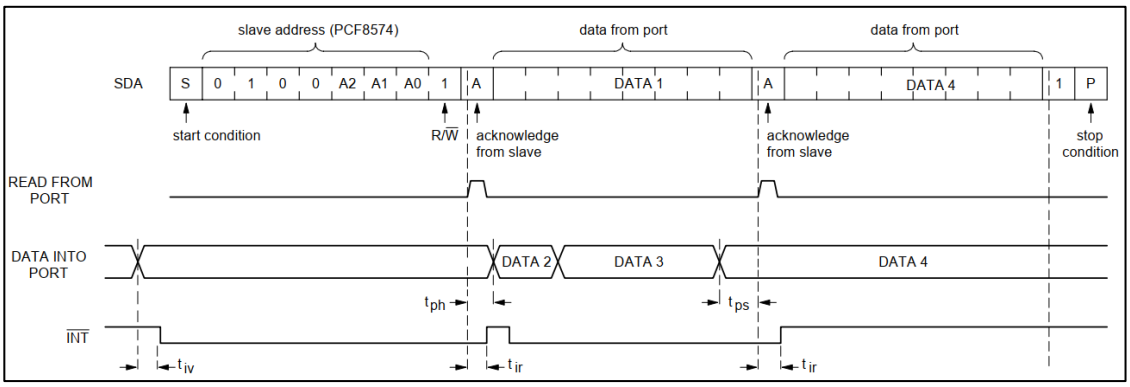


图 38.1.3 PCF8574 读数据时序

PCF8574T 的读数据流程：首先发送 PCF8574T 的从机地址+读信号（R/W=1），然后等待 PCF8574T 应答（注意：PCF8574T 在发送应答的同时，会锁存 P0~P7 的数据），然后读取 P0~P7 的数据。数据读取支持连续读取，在最后的时候发送 STOP 信号，即可完成读数据操作。

这里需要注意的是：PCF8574T 的数据锁存（READ FROM PORT），发生在发送应答信号的时候，之后，P0~P7 发送的数据变化（比如图中的 DATA2 和 DATA3），将不会读取进来，直到下一个应答信号进行锁存。

5.中断

PCF8574T 带有中断输出脚，它可以连接到 MCU 的中断输入引脚上。在输入模式中（IO 口输出高电平，即可做输入使用），输入的上升沿或下降沿都可以产生中断，在 tiv 时间之后 INT 有效。特别注意：一旦中断有效后，必须对 PCF8574T 进行一次读取/写入操作，复位中断，才可以输出下一次中断，否则中断将一直保持（无法输出下一次输入信号变化所产生的中断）。

关于 PCF8574 的简介，我们就介绍到这里。

38.2 硬件设计

1. 例程功能

开机的时候先检测 PCF8574T 是否存在，然后在主循环里面检测 KEY0 按键和 PCF8574T 的中断信号，当 KEY0 按下时，控制 PCF8574T 的 P0 口输出，从而控制蜂鸣器（连接在 P0 口）的开关；当检测到 PCF8574T 的中断信号时，读取 EXIO（连接在 PCF8574T 的 P4 口）的状态，当 EXIO=0（即 P4=0）的时候，控制 LED1 的翻转。同时，LCD 模块上显示相关信息，并用 DS0 提示程序正在运行。另外，本例程将 PCF8574T 的相关控制函数加入 USMART 控制，我们也可以通过 USMART 控制/读取 PCF8574T。

2. 硬件资源

1）LED 灯
LED0 : LED0 - PB1
LED1 : LED1 - PB0
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）独立按键
KEY0 - PH3
5）PCF8574（通过 IIC 接口连接）
IIC_SCL - PH4
IIC_SDA - PH5
6）蜂鸣器

3. 原理图

我们主要来看看 PCF8574T 和开发板的连接，如下图所示：

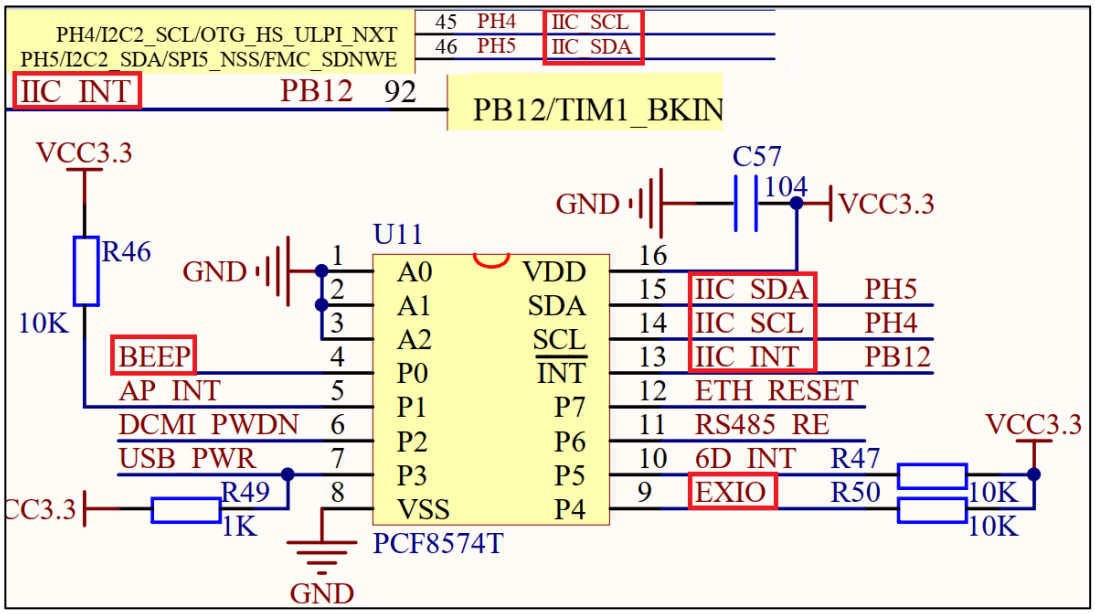


图 38.2.1 PCF8574T 与开发板连接原理图

由图可知，蜂鸣器控制信号 BEEP 连接在 PCF8574T 的 P0 脚上，EXIO 连接在 P4 脚上，其他还连接了一些外设（比如网络复位脚、摄像头、USB、485 等），我们将在对应章节进行介绍，这里就不多说了。这里需要注意的是：IIC_INT 脚，同 1WIRE_DQ 共用了 PB12，在使用的时候。他们只能分时复用，不能同时使用。

38.3 程序设计

38.3.1 程序流程图

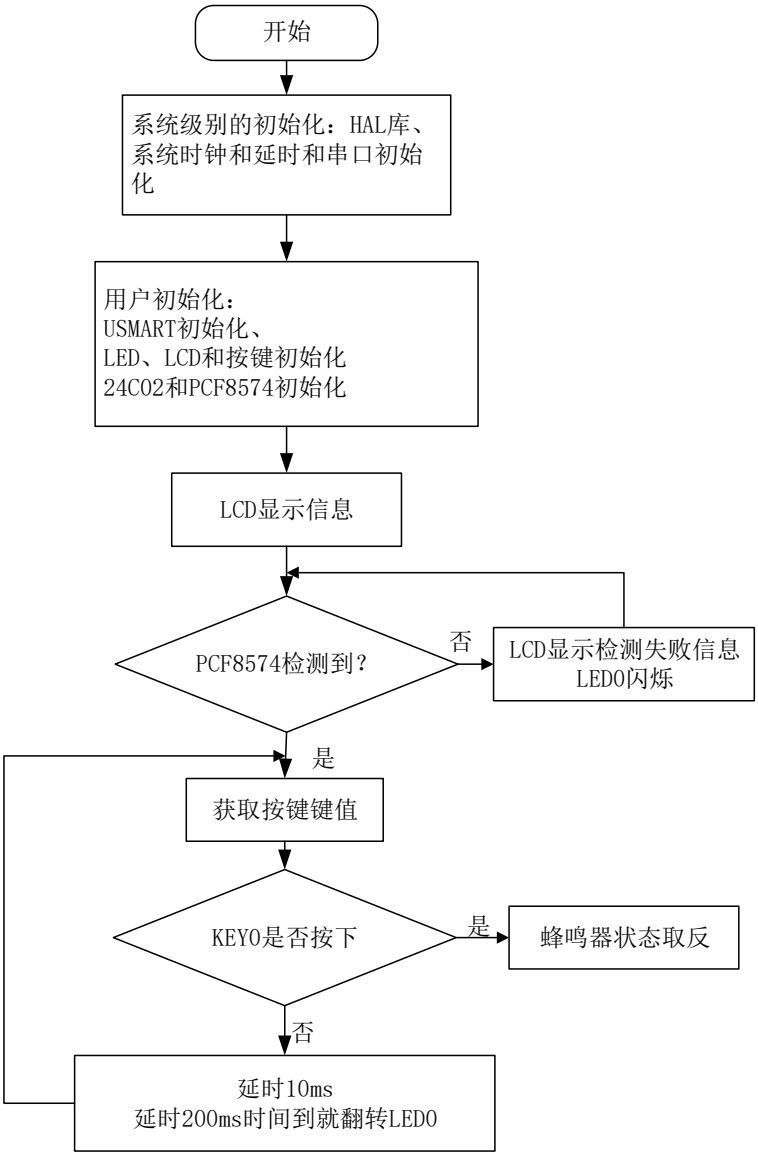


图 38.3.1.1 IO 扩展实验程序流程图

38.3.2 程序解析

1. PCF 8574 底层驱动代码

由于 PCF8574 要使用到 IIC 接口，所以这里我们保留了上一章 IIC 相关源码。同时还增加了 PCF8574 相关的驱动代码文件 pcf8574.c 源文件和 pcf8574.h 头文件。

下面我们直接介绍 PCF8574 相关的程序，首先先介绍 pcf8574.c 文件，其定义如下：

```
/**
 * @brief       初始化 PCF8574
 * @param       无
 * @retval      0, 成功;
                1, 失败;
 */
uint8_t pcf8574_init(void)
```

```
{
    uint8_t temp = 0;
    GPIO_InitTypeDef gpio_init_struct;
    PCF8574_GPIO_CLK_ENABLE();              /* 使能 GPIOB 时钟 */

    gpio_init_struct.Pin = PCF8574_GPIO_PIN;                /* PB12 */
    gpio_init_struct.Mode = GPIO_MODE_INPUT;                /* 输入 */
    gpio_init_struct.Pull = GPIO_PULLUP;                    /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_HIGH;               /* 高速 */
    HAL_GPIO_Init(PCF8574_GPIO_PORT, &gpio_init_struct);    /* 初始化 */
    iic_init();                                             /* IIC 初始化 */

    /* 检查 PCF8574 是否在位 */
    iic_start();
    iic_send_byte(PCF8574_ADDR);      /* 写地址 */
    temp = iic_wait_ack();  /* 等待应答,通过判断是否有 ACK 应答,来判断 PCF8574 的状态 */
    iic_stop();                       /* 产生一个停止条件 */
    pcf8574_write_byte(0XFF);         /* 默认情况下所有 IO 输出高电平 */
    return temp;
}

/**
 * @brief       读取 PCF8574 的 8 位 IO 值
 * @param       无
 * @retval      返回值:读到的数据
 */
uint8_t pcf8574_read_byte(void)
{
    uint8_t temp = 0;

    iic_start();
    iic_send_byte(PCF8574_ADDR | 0X01);    /* 进入接收模式 */
    iic_wait_ack();
    temp = iic_read_byte(0);
    iic_stop();                            /* 产生一个停止条件 */

    return temp;
}

/**
 * @brief       向 PCF8574 写入 8 位 IO 值
 * @param       data:要写入的数据
 * @retval      无
 */
void pcf8574_write_byte(uint8_t data)
{
    iic_start();
    iic_send_byte(PCF8574_ADDR | 0X00);    /* 发送器件地址 0X40,写数据 */
    iic_wait_ack();
    iic_send_byte(data);            /* 发送字节 */
    iic_wait_ack();
    iic_stop();                     /* 产生一个停止条件  */
    delay_ms(10);
}

/**
 * @brief       设置 PCF8574 某个 IO 的高低电平
 * @param       bit:要设置的 IO 编号,0~7
 * @param       sta:IO 的状态;0 或 1
 * @retval      无
 */
void pcf8574_write_bit(uint8_t bit, uint8_t sta)
```

```
{
    uint8_t data;
    data = pcf8574_read_byte();        /* 先读出原来的设置 */
    if (sta == 0)
    {
        data &= ~(1 << bit);
    }
    else
    {
        data |= 1 << bit;
    }
    pcf8574_write_byte(data);    /* 写入新的数据 */
}

/**
 * @brief       读取 PCF8574 的某个 IO 的值
 * @param       bit: 要读取的 IO 编号,0~7
 * @retval      返回值:此 IO 的值,0 或 1
 */
uint8_t pcf8574_read_bit(uint8_t bit)
{
    uint8_t data;
    data = pcf8574_read_byte(); /* 先读取这个 8 位 IO 的值  */
    if (data & (1 << bit))return 1;
    else return 0;
}
```

该部分为 PCF8574 的驱动代码，其中 IIC 相关函数，直接用的上一章 myiic.c 里面提供的相关函数，这里不做介绍。

这里总共有 5 个函数：pcf8574_init 函数用于初始化并检测 pcf8574，这里我们初始化 PB12 为上拉输入，以检测 PCF8574T 的中断输出信号，另外，在函数里面，我们通过检查 PCF8574 的应答信号来确认 PCF8574 是否正常（在位）；pcf8574_read_byte 和 pcf8574_write_byte 这两个函数用于读取/写入 PCF8574，从而读取/控制 P0~P7；最后，pcf8574_write_bit 和 pcf8574_read_bit 函数用于控制或者读取 PCF8574 的单个 IO。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t key;
    uint16_t i = 0;
    uint8_t beepsta = 1;
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟,400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 初始化 USART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    usmart_init(200);                       /* 初始化 USMART */
    led_init();                             /* 初始化 LED */
    key_init();                             /* 初始化按键 */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "PCF8574 TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:BEEP ON/OFF", RED);
    lcd_show_string(30, 130, 200, 16, 16, "EXIO:DS1 ON/OFF", RED);

    while (pcf8574_init()) /* 检测不到 PCF8574 */
```

```
    {
        lcd_show_string(30, 170, 200, 16, 16, "PCF8574 Check Failed!", RED);
        delay_ms(500);
        lcd_show_string(30, 170, 200, 16, 16, "Please Check!      ", RED);
        delay_ms(500);
        LED0_TOGGLE();      /* 红灯闪烁 */
    }

    lcd_show_string(30, 170, 200, 16, 16, "PCF8574 Ready!", RED);

    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)       /* KEY0 按下,读取字符串并显示 */
        {
            beepsta =! beepsta;     /* 蜂鸣器状态取反 */
            pcf8574_write_bit(BEEP_IO, beepsta);    /* 控制蜂鸣器 */
        }

        if (PCF8574_INT == 0)       /* PCF8574 的中断低电平有效 */
        {
            key = pcf8574_read_bit(EX_IO);
            /* 读取 EXIO 状态,同时清除 PCF8574 的中断输出(INT 恢复高电平) */
            if(key == 0)
            {
                LED1_TOGGLE();      /* LED1 状态取反 */
            }
        }
        i++;
        delay_ms(10);
        if (i == 20)
        {
            LED0_TOGGLE();        /* 红灯闪烁 */
            i = 0;
        }
    }
}
```

main 函数的流程大致是：通过 KEY0 按键可以控制蜂鸣器的开关，另外，在 while 循环里面，会不停的检测 PCF8574 的中断引脚，是否有输出中断，如果有，就读取 EXIO 的状态（读操作会复位中断，以便检测下一个中断），当 EXIO=0 的时候控制 DS1 开关。同时，在 LCD 模块上面显示相关信息，并用 DS0 指示程序运行状态。

38.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了，最终 LCD 显示的内容如图 38.4.1 所示：

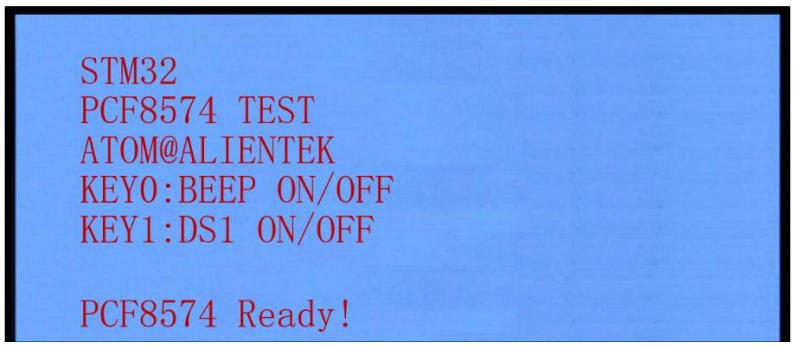


图 38.4.1 IO 扩展实验程序运行效果图

屏幕提示 PCF8574 Ready!，表示 PCF8574 已经准备好，同时 DS0 会不停的闪烁，提示程序正在运行。此时，我们按按键 KEY0，就可以控制蜂鸣器的开和关。也可以用一根杜邦线，

连接 EXIO（在 P3 排针的最左下角）和 GND（短接一次 GND，改变一次 DS1 的状态），就可以控制 DS1 的开和关。

此实验支持 USMART 控制，通过 pcf8574_read_byte、pcf8574_write_byte、pcf8574_read_bit 和 pcf8574_write_bit 这四个函数可以方便测试 PCF8574。

第三十九章 光环境传感器实验

上一章，我们介绍了 IIC 驱动 PCF8574T，本章我们将向大家介绍如何使用 IIC 来驱动光环境传感器。在本章中，我们将使用 STM32H743 的普通 IO 口模拟 IIC 时序，来驱动 AP3216C，从而检测环境光强度（ALS）、接近距离（PS）和红外线强度（IR）等环境参数。

本章分为如下几个小节：

39.1 光敏传感器简介
39.2 硬件设计
39.3 程序设计
39.4 下载验证

39.1 AP3216C 简介

AP3216C 是敦南科技推出的一款三合一环境传感器， 它包含了：数字环境光传感器（ALS）、接近传感器（PS）和一个红外 LED（IR）。该芯片通过 IIC 接口和 MCU 连接，并支持中断（INT）输出。AP3216C 的特点如下：

- IIC 接口，支持高达 400KHz 通信速率
- 支持多种工作模式（ALS、PS+IR、ALS+PS+IR 等）
- 内置温度补偿电路
- 工作温度支持-30~80℃
- 环境光传感器具有 16 位分辨率
- 接近传感器具有 10 位分辨率
- 红外传感器具有 10 位分辨率
- 超小封装（4.1*2.4*1.35mm）

因为以上一些特性，AP3216C 被广泛应用于智能手机上面，用来检测光强度（自动背光控制），和接近开关控制（听筒靠近耳朵，手机自动灭屏功能）。AP3216C 的框图如图 39.1.1 所示：

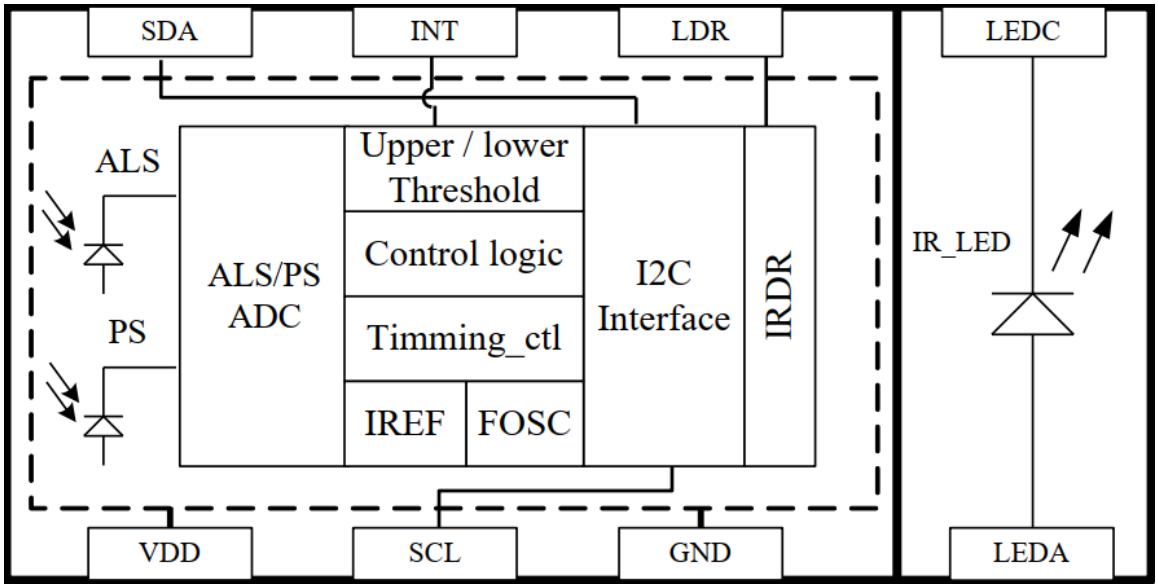


图 39.1.1 AP3216C 框图

1.引脚说明

AP3216C 的引脚说明如表 39.1.1 所示：

引脚编号	标号	说明
1	VDD	电源，接 3.3V

2	SCL	IIC 时钟信号，开漏
3	GND	地线
4	LEDA	LED 阳极，接 3.3V
5	LEDC	LED 阴极，一般连接 LDR
6	LDR	LED 驱动输出脚，一般接 LEDC
7	INT	中断输出脚
8	SDA	IIC 数据信号，开漏

表 39.1.1 AP3216C 引脚说明

AP3216C 和我们的 MCU 只需要连接 SCL、SDA 和 INT，就可以实现驱动。其 SCL 和 SDA 同 24C02 共用，连接在 PH4 和 PH5 上，INT 脚连接在 PCF8574 的 P1 上，见图 38.2.1。关于 IIC 协议的介绍，请参考 IIC 实验。

2.写寄存器

AP3216C 的写寄存器时序如图 39.1.2 所示：

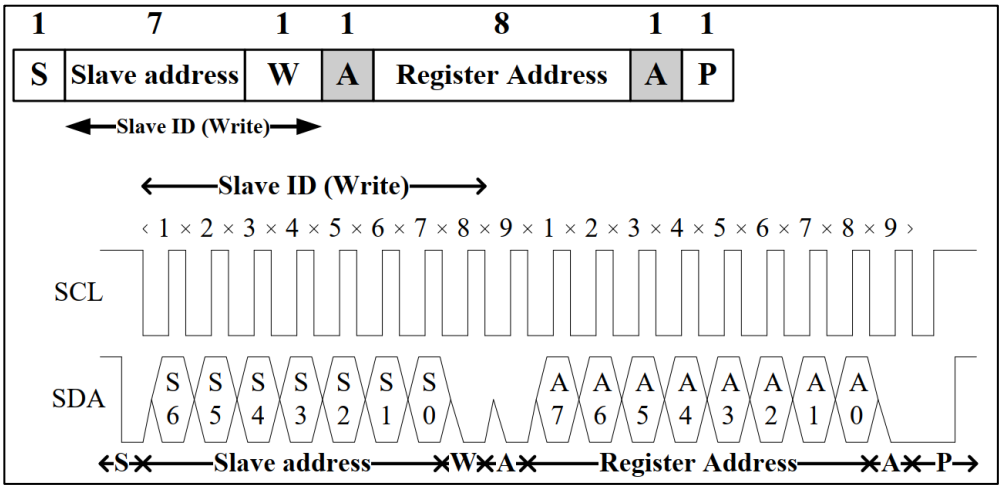


图 39.1.2 AP3216C 写寄存器时序

图中，先发送 AP3216C 的地址（7 位，0X1E，左移一位后为：0X3C），最低位 W=0 表示写数据，随后发送 8 位寄存器地址，最后发送 8 位寄存器值。其中：S，表示 IIC 起始信号；W，表示读/写标志位（W=0 表示写，W=1 表示读）；A，表示应答信号；P，表示 IIC 停止信号。

3.读寄存器

AP3216C 的读寄存器时序如图 39.1.3 所示：

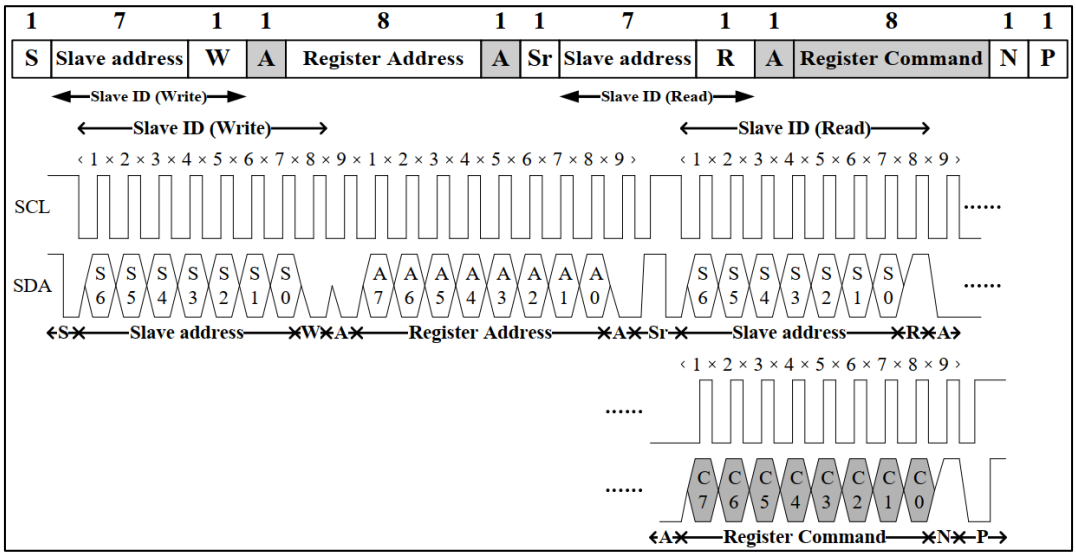


图 39.1.3 AP3216C 读寄存器时序

图中，同样是先发送 7 位地址+写操作，然后再发送寄存器地址，随后，重新发送起始信号（Sr），再次发送 7 位地址+读操作，然后读取寄存器值。其中：Sr，表示重新发送 IIC 起始信号；N，表示不对 AP3216C 进行应答；其他简写同上。

4.寄存器描述

AP3216C 有一些列寄存器，由这些寄存器来控制 AP3216C 的工作模式，以及中断配置和数据输出等。这里我们仅介绍我们在本章需要用到的一些寄存器，其他寄存器的描述和说明，请大家参考 AP3216C 的数据手册。

本章需要用到 AP3216C 的寄存器如表 39.1.2 所示：

地址	有效位	指令	说明
0X00	2:0	系统模式	000：掉电模式（默认） 001：ALS 功能激活 010：PS+IR 功能激活 011：ALS+PS+IR 功能激活 100：软复位 101：ALS 单次模式 110：PS+IR 单次模式 111：ALS+PS+IR 单次模式
0X0A	7	IR 低位数据	0：IR&PS 数据有效；1：无效
	1:0		IR 最低 2 位数据
0X0B	7:0	IR 高位数据	IR 高 8 位数据
0X0C	7:0	ALS 低位数据	ALS 低 8 位数据
0X0D	7:0	ALS 高位数据	ALS 高 8 位数据
0X0E	7	PS 低位数据	0：物体在远离；1：物体在靠近
	6		0：IR 数据有效；1：IR 数据无效
	3:0		PS 最低 4 位数据
0X0F	7	PS 高位数据	0：物体在远离；1：物体在靠近
	6		0：IR 数据有效；1：IR 数据无效
	5:0		PS 高 6 位数据

表 39.1.2 AP3216C 相关寄存器及其说明

上表中，0X00 是一个系统模式控制寄存器，主要在初始化的时候配置，初始化的时候，我们先设置其值为 100，实行一次软复位，随后设置其值为 011，开启 ALS+PS+IR 检测功能。

剩下的 6 个寄存器，为数据寄存器，输出 AP3216C 内部三个传感器所检测到的数据（ADC 值），描述如表所示，这里需要注意的是：读取间隔至少要大于 112.5ms，因为，AP3216C 内部完成一次 ALS+PS+IR 的数据转换，需要 112.5ms 的时间。

AP3216C 的简介，我们就介绍到这里，关于该芯片的详细说明，请大家参考其数据手册。

39.2 硬件设计

1. 例程功能

开机的时候先检测 AP3216C 是否存在，如检测不到 AP3216C，则在 LCD 屏幕上面显示报错信息。如果检测到 AP3216C，则显示正常，并在主循环里面，循环读取 ALS+PS+IR 的传感器数据，并显示在 LCD 屏幕上面。同时，DS0 闪烁，提示程序正在运行。另外，本例程将 AP3216C 的读写操作函数加入 USMART 控制，我们也可以通过 USMART 对 AP3216C 进行控制。

2. 硬件资源

1）LED 灯

LED0 - PB1

2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)

3）正点原子 2.8/3.5/4.3/7 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）AP3216C（通过 IIC 接口连接）
IIC_SCL －PH4
IIC_SDA －PH5

3. 原理图

我们主要来看看光环境传感器和开发板的连接，如下图所示：

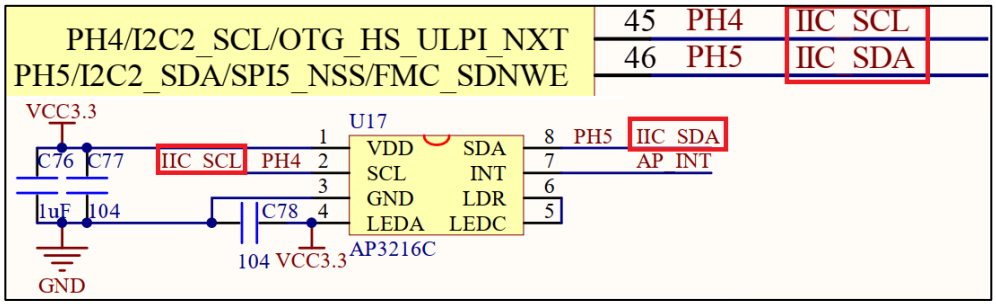


图 39.2.1 光敏传感器与开发板连接示意图

这里需要说明一下：AP3216C 的 AP_INT 脚，是连接在 PCF8574T 的 P1 脚上的（见图 38.2.1），如果大家想要用 AP3216C 的中断输出功能，则必须初始化 PCF8574T，并监控 PCF8574T 的中断引脚，然后在发现有中断输入的时候，读取 PCF8574T，判断 P1 脚是否有低电平出现，从而检测 AP3216C 的中断。本章，我们并没有用到 AP3216C 的中断功能，所以，不需要配置 PCF8574T。

39.3 程序设计

39.3.1 程序流程图

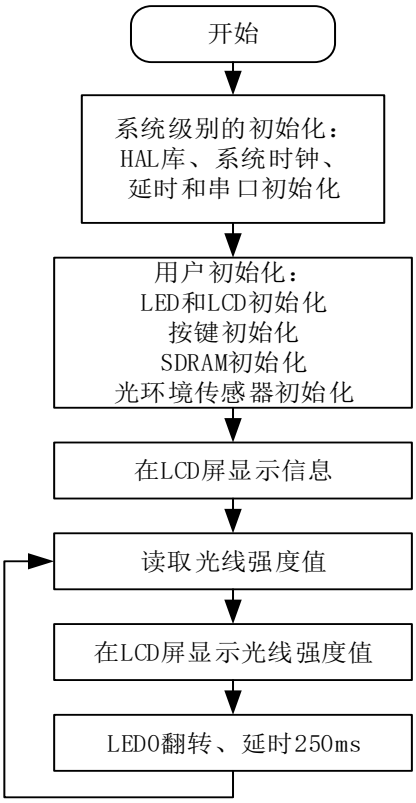


图 39.3.1.1 光敏传感器实验程序流程图

39.3.2 程序解析

1. AP3216C 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。AP3216C 驱动源码包括两个文件：ap3216c.c 和 ap3216.h。

ap3216c.h 头文件代码非常简单，主要是函数声明以及宏定义标识符，这里大家只需要注意，宏定义标识符 AP3216C_ADDR 配置的是器件 AP3216C 的 IIC 地址。

下面介绍 ap3216c.c 的函数，首先是光环境传感器初始化函数，其定义如下：

```
/**
 * @brief       初始化 AP3216C
 * @param       无
 * @retval      0, 成功;
                1, 失败;
 */
uint8_t ap3216c_init(void)
{
    uint8_t temp = 0;
    iic_init();                             /* 初始化 IIC */
    ap3216c_write_one_byte(0x00, 0X04); /* 复位 AP3216C */
    delay_ms(50);                           /* AP33216C 复位至少 10ms */
    ap3216c_write_one_byte(0x00, 0X03); /* 开启 ALS、PS+IR    */
    temp = ap3216c_read_one_byte(0X00); /* 读取刚刚写进去的 0X03 */
    if (temp == 0X03)return 0;              /* AP3216C 正常 */
    else return 1;                          /* AP3216C 失败 */
}
/**
 * @brief       读取 AP3216C 的数据
 * @note        读取原始数据，包括 ALS,PS 和 IR
 *              如果同时打开 ALS,IR+PS 的话两次数据读取的时间间隔要大于 112.5ms
 * @param       ir      : IR 传感器值
 * @param       ps      : PS 传感器值
 * @param       als     : ALS 传感器值
 * @retval      无
 */
void ap3216c_read_data(uint16_t *ir, uint16_t *ps, uint16_t *als)
{
    uint8_t buf[6];
    uint8_t i;

    for (i = 0;i < 6;i++)
    {
       buf[i] = ap3216c_read_one_byte(0X0A + i);/* 循环读取所有传感器数据 */
    }
    if (buf[0] & 0X80) * ir = 0;                /* IR_OF 位为 1,则数据无效 */
    /* 读取 IR 传感器的数据 */
    else *ir =((uint16_t)buf[1] << 2) | (buf[0] & 0X03);
    *als = ((uint16_t)buf[3] << 8) | buf[2];    /* 读取 ALS 传感器的数据 */
    if (buf[4] & 0x40) * ps = 0;                /* IR_OF 位为 1,则数据无效 */
    else *ps = ((uint16_t)(buf[5] & 0X3F) << 4) | (buf[4] & 0X0F);
    /* 读取 PS 传感器的数据 */
}
/**
 * @brief       AP3216C 写入一个字节
 * @param       reg     : 寄存器地址
 * @param       data    : 要写入的数据
 * @retval      0, 成功;
                1, 失败;
 */
```

```
uint8_t ap3216c_write_one_byte(uint8_t reg, uint8_t data)
{
    iic_start();
    iic_send_byte(AP3216C_ADDR | 0X00);     /* 发送器件地址+写命令 */
    if (iic_wait_ack())                     /* 等待应答 */
    {
        iic_stop();
        return 1;
    }
    iic_send_byte(reg);         /* 写寄存器地址 */
    iic_wait_ack();             /* 等待应答 */
    iic_send_byte(data);        /* 发送数据 */
    if (iic_wait_ack())         /* 等待 ACK */
    {
        iic_stop();
        return 1;
    }
    iic_stop();
    return 0;
}
/**
 * @brief       AP3216C 读取一个字节
 * @param       reg     : 寄存器地址
 * @retval      读到的数据
 */
uint8_t ap3216c_read_one_byte(uint8_t reg)
{
    uint8_t res;

    iic_start();
    iic_send_byte(AP3216C_ADDR | 0X00);     /* 发送器件地址+写命令 */
    iic_wait_ack();                         /* 等待应答 */
    iic_send_byte(reg);                     /* 写寄存器地址 */
    iic_wait_ack();                         /* 等待应答 */
    iic_start();
    iic_send_byte(AP3216C_ADDR | 0X01);     /* 发送器件地址+读命令 */
    iic_wait_ack();                         /* 等待应答 */
    res = iic_read_byte(0);                 /* 读数据,发送 nACK */
    iic_stop();                             /* 产生一个停止条件 */

    return res;
}
```

该部分为 AP3216C 的驱动代码，其中的 IIC 相关函数，直接是用第三十七章 myiic.c 里面提供的相关函数，这里不做介绍。

这里总共有 4 个函数：ap3216c_init 函数用于初始化并检测 AP3216C，先设置 AP3216C 软复位，随后设置其工作在 ALS+PS+IR 模式，通过对系统模式寄存器的读写操作，来判断 AP3216C 是否正常（在位）；ap3216c_write_one_byte 和 ap3216c_read_one_byte 这两个函数实现 AP3216C 的寄存器写入和读取功能；ap3216c_read_data 函数，则用于读取 ALS+PS+IR 传感器的数据，一般我们只需要调用该函数获取数据即可。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint16_t ir, als, ps;
    sys_cache_enable();                     /* 打开 L1-cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4); /* 设置时钟,400Mhz */
    delay_init(400);                        /* 延时初始化 */
```

```
    usart_init(115200);                 /* 初始化 USART */
    mpu_memory_protection();            /* 保护相关存储区域 */
    usmart_init(200);                   /* 初始化 USMART */
    led_init();                         /* 初始化 LED */
    sdram_init();                       /* 初始化 SDRAM */
    lcd_init();                         /* 初始化 LCD */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "AP3216C TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    while (ap3216c_init())  /* 检测不到 AP3216C */
    {
        lcd_show_string(30, 130, 200, 16, 16, "AP3216C Check Failed!", RED);
        delay_ms(500);
        lcd_show_string(30, 130, 200, 16, 16, "Please Check!      ", RED);
        delay_ms(500);
        LED0_TOGGLE();      /* 红灯闪烁 */
    }
    lcd_show_string(30, 130, 200, 16, 16, "AP3216C Ready!", RED);
    lcd_show_string(30, 160, 200, 16, 16, " IR:", RED);
    lcd_show_string(30, 180, 200, 16, 16, " PS:", RED);
    lcd_show_string(30, 200, 200, 16, 16, "ALS:", RED);
    while (1)
    {
        ap3216c_read_data(&ir, &ps, &als);              /* 读取数据  */
        lcd_show_num(30 + 32, 160, ir, 5, 16, BLUE);    /* 显示 IR 数据 */
        lcd_show_num(30 + 32, 180, ps, 5, 16, BLUE);    /* 显示 PS 数据 */
        lcd_show_num(30 + 32, 200, als, 5, 16, BLUE);   /* 显示 ALS 数据  */

        LED0_TOGGLE();                                  /* 提示系统正在运行 */
        delay_ms(120);
    }
}
```

该部分的代码逻辑很简单，初始化各个外设之后，进入死循环，通过调用 ap3216c_read_data 函数，读取 ALS+PS+IR 的数据，并显示在 LCD 上面。同时，DS0 闪烁，提示程序正在运行。这里我们延时 120ms 读取一次，确保 ALS+PS+IR 的转换全部完成，以保证数据正常。

39.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 39.4.1 所示：

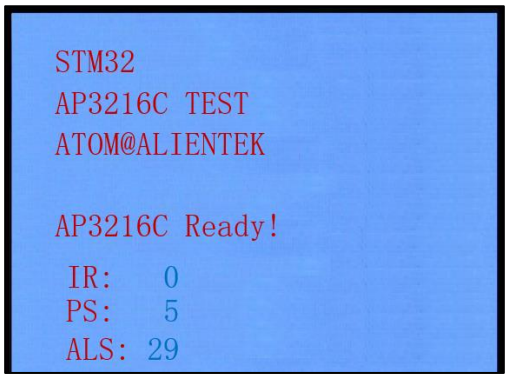


图 39.4.1 光环境传感器实验测试图

我们可以用手遮挡/靠近 AP3216C 传感器，可以看到三个传感器的数据变化，说明我们的代码是工作正常的。

第四十章 QSPI 实验

本章，我们将介绍 STM32H743 的 QSPI 功能，并使用 STM32H743 自带的 QSPI 来实现对外部 NOR FLASH 的读写，并将结果显示在 LCD 模块上。

本章分为如下几个小节：

40.1 QSPI 及 NOR FLASH 芯片简介
40.2 硬件设计
40.3 程序设计
40.4 下载验证

40.1 QSPI 及 NOR FLASH 芯片简介

40.1.1 QSPI 简介

QSPI 是 Quad SPI 的缩写，是 Motorola 公司推出 SPI 接口后的一种扩展接口，较 SPI 应用更为广泛。在 SPI 协议的基础上，Motorola 公司对其功能进行了增加，增加了队列传输机制，推出了队列串行外围接口协议（即 QSPI 协议）。QSPI 是一种专用的通信接口，连接单、双或四（条数据线）SPI FLASH 存储器。STM32H7 具有 QSPI 接口，支持如下三种工作模式：

1、间接模式：使用 QSPI 寄存器执行全部操作。

2、状态轮询模式：周期性读取外部 FLASH 状态寄存器，当标志位置 1 时会产生中断（如擦除或烧写完成，产生中断）。

3、内存映射模式：外部 FLASH 映射到微控制器地址空间，从而系统将其视作内部存储器。

STM32H7 的 QSPI 接口具有如下特点：

- 支持三种工作模式：间接模式、状态轮询模式和内存映射模式。
- 支持双闪存模式，可以并行访问两个 FLASH，可同时发送/接收 8 位数据。
- 支持 SDR（单倍率速率）和 DDR（双倍率速率）模式。
- 针对间接模式和内存映射模式，完全可编程操作码。
- 针对间接模式和内存映射模式，完全可编程帧格式。
- 集成 FIFO，用于发送和接收。
- 允许 8、16 和 32 位数据访问。
- 具有适用于间接模式操作的 DMA 通道。
- 在达到 FIFO 阈值、超时、操作完成以及发生访问错误时产生中断。

40.1.1.1 QSPI 框图

STM32H7 的 QSPI 单闪存模式的功能框图如图 40.1.1.1.1 所示：

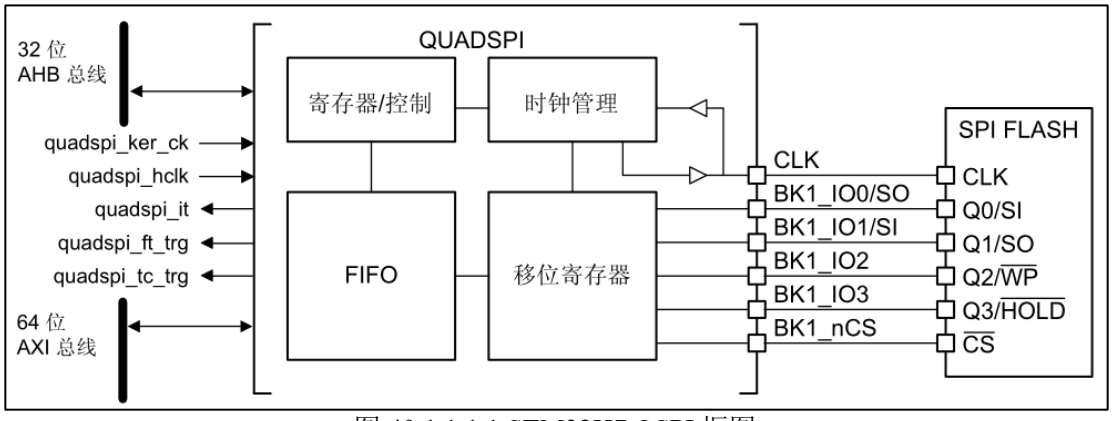


图 40.1.1.1.1 STM32H7 QSPI 框图

上图左边可以看到 QSPI 连接到 64 位 AXI 总线以及 32 位 AHB 总线上，此外还有 5 条 QSPI

的内部信号，如下表所示：

信号名称	信号类型	说明
quadspi_ker_ck	数字输入	QUADSPI 内核时钟
quadspi_hclk	数字输入	QUADSPI 寄存器接口时钟
quadspi_it	数字输出	QUADSPI 全局中断
quadspi_ft_trg	数字输出	MDMA 的 QUADSPI FIFO 阈值触发信号
quadspi_tc_trg	数字输出	MDMA 的 QUADSPI 传输完成触发信号

表 40.1.1.1.1 QSPI 内部信号

quadspi_ker_ck，用于通讯过程的时钟。可以选择的时钟源有：HCLK3（即 AHB3）、PLL1Q、PLL2R 和 PER_CK，实验中我们选择 HCLK3。经过 sys_stm32_clock_init 函数的配置，HCLK3 时钟频率为 200MHZ。quadspi_ker_ck 还需要经过一个分频器出来的时钟频率才作为 QSPI 的实际使用的时钟频率，该分频器的分频系数由 QUADSPI_CR 寄存器的 PRESCALER[7:0]位设置，范围是：0~255。

quadspi_hclk，用于操作 QUADSPI 寄存器的时钟。时钟源来自 HCLK3，同样是经过 sys_stm32_clock_init 函数的配置，配置后 HCLK3 时钟频率为 200MHZ。

quadspi_it，中断请求信号线。在达到 FIFO 阈值、超时、操作完成以及发送访问错误时产生中断。

quadsqp_ft_trg，达到 FIFO 阈值时触发 MDMA 请求。

quadspi_tc_trg，操作完成时触发 MDMA 请求

上图中间部分就是 QUADSPI 内核。

我们重点看看上图右边的 QSPI 接口引脚，通过 6 根线与 SPI FLASH 芯片连接，包括：4 根数据线（IO0~3）、1 根时钟线（CLK）和 1 根片选线（nCS），具体如下表所示：

引脚名称	信号类型	说明
CLK	数字输出	时钟，由主机产生，用于通讯数据同步，决定通信速率
BK1_IO0/SO	数字输入/输出	在双线/四线模式中为双向 IO，单线模式中为串行输出
BK1_IO1/SI	数字输入/输出	在双线/四线模式中为双向 IO，单线模式中为串行输入
BK1_IO2	数字输入/输出	在四线模式中为双向 IO
BK1_IO3	数字输入/输出	在四线模式中为双向 IO
BK1_nCS	数字输出	片选输出（低电平有效）

表 40.1.1.1.2 QSPI 接口引脚

我们知道普通的 SPI 通信一般只有一根数据线（MOSI/MISO，发送/接收用），而 QSPI 则具有 4 根数据线，所以 QSPI 的速率至少是普通 SPI 的 4 倍，可以大大提高通信速率。如果使用双闪存模式，同时访问两个 Quad-SPI Flash，速度可以再翻一番。我们开发板板载了一个 Quad-SPI Flash，所以我们使用单闪存模式，双闪存模式就不具体介绍了，感兴趣请自行查看手册。

接下来，我们给大家简单介绍一下 STM32H7 QSPI 接口的的几个重要知识点。

40.1.1.2 QSPI 命令序列

QSPI 通过命令与 FLASH 通信，每条命令包括：指令、地址、交替字节、空指令和数据这五个阶段，任一阶段均可通过配置 QUADSPI_CCR 寄存器相关字段跳过，但至少要包含指令、地址、交替字节或数据阶段之一。

nCS 在每条指令开始前下降，在每条指令完成后再次上升。QSPI 四线模式下的读命令时序，如图 40.1.1.2.1 所示：

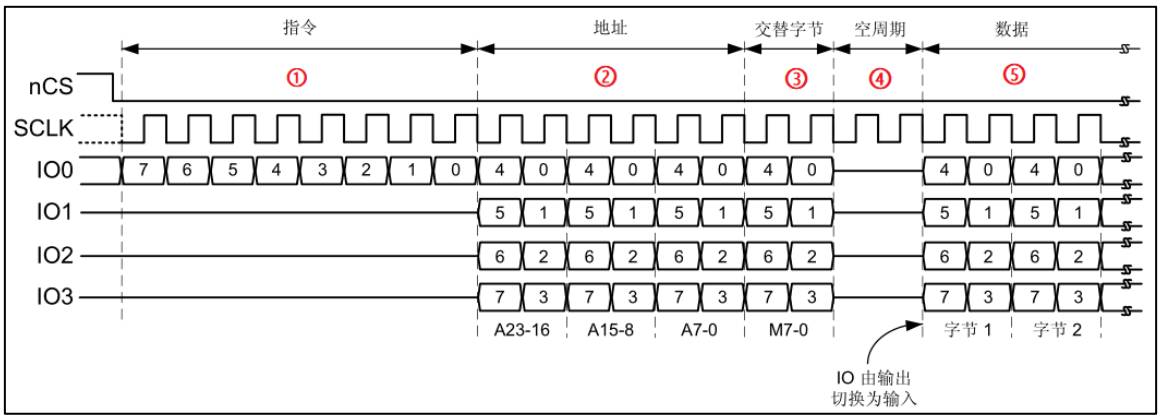


图 40.1.1.2.1 四线模式下 QSPI 读命令时序

从上图可以看出一次 QSPI 传输的 5 个阶段，接下来我们分别介绍。

① 指令阶段

此阶段通过 QUADSPI_CCR[7:0]寄存器的 INSTRUCTION 字段指定一个 8 位指令发送到 FLASH。注意，指令阶段，一般是通过 IO0 单线发送，但是也可以配置为双线/四线发送指令，可以通过 QUADSPI_CCR[9:8]寄存器的 IMODE[1:0]这两个位进行配置，如 IMODE[1:0]=00，则表示无需发送指令。

② 地址阶段

此阶段可以发送 1~4 字节地址给 FLASH 芯片，指示要操作的地址。地址字节长度由 QUADSPI_CCR[13:12]寄存器的 ADSIZE[1:0]字段指定，0~3 表示 1~4 字节地址长度。在间接模式和轮询模式下，待发送的地址由 QUADSPI_AR 寄存器指定。地址阶段同样可以以单线/双线/四线模式发送，通过 QUADSPI_CCR[11:10]寄存器的 ADMODE[1:0]这两个位进行配置，如 ADMODE [1:0]=00，则表示无需发送地址。

③ 交替字节（复用字节）阶段

此阶段可以发送 1~4 字节数据给 FLASH 芯片，一般用于控制操作模式。待发送的交替字节数由 QUADSPI_CCR[17:16]寄存器的 ABSIZE[1:0]位配置。待发送的数据由 QUADSPI_ABR 寄存器中指定。交替字节同样可以以单线/双线/四线模式发送，通过 QUADSPI_CCR[15:14]寄存器的 ABMODE[1:0]这两个位配置，ABMODE[1:0]=00，则跳过交替字节阶段。

④ 空指令周期阶段

在空指令周期阶段，在给定的 1~31 个周期内不发送或接收任何数据，目的是当采用更高的时钟频率时，给 FLASH 芯片留出准备数据阶段的时间。这一阶段中给定的周期数由 QUADSPI_CCR[22:18]寄存器的 DCYC[4:0]位配置。若 DCYC 为零，则跳过空指令周期阶段，命令序列直接进入下一个阶段。

⑤ 数据阶段

此阶段可以从 FLASH 读取/写入任意字节数量的数据。在间接模式和自动轮询模式下，待发送/接收的字节数由 QUADSPI_DLR 寄存器指定。在间接写入模式下，发送到 FLASH 的数据必须写入 QUADSPI_DR 寄存器。在间接读取模式下，通过读取 QUADSPI_DR 寄存器获得从 FLASH 接收的数据。数据阶段同样可以以单线/双线/四线模式发送，通过 QUADSPI_CCR[25:24]寄存器的 DMODE [1:0]这两个位进行配置，如 DMODE [1:0]=00，则表示无数据。

以上就是 QSPI 数据传输的 5 个阶段，其中交替字节阶段我们一般用不到，可以省略（通过设置 ABMODE[1:0]=00）。

另外说明一下，QUADSPI 信号接口协议模式包括：单线 SPI 模式、双线 SPI 模式、四线 SPI 模式、SDR 模式、DDR 模式和双闪存模式。这些模式请大家自行参考官方手册。

40.1.1.3 QSPI 三种功能模式

前面已经提到过 QSPI 的三种功能模式：间接模式、状态标志轮询模式和内存映射模式。下面对这三个功能模式分别简单介绍。

① 间接模式

在间接模式下，通过写入 QUADSPI 寄存器来触发命令，通过读写数据寄存器来传输数据。

当 FMODE=00 (QUADSPI_CCR[27:26])时，QUADSPI 处于间接写入模式，在数据阶段，将数据写入数据寄存器(QUADSPI_DR)，即可写入数据到 FLASH。

当 FMODE=01 时，QUADSPI 处于间接读取模式，在数据阶段，读取 QUADSPI_DR 寄存器，即可读取 FLASH 里面的数据。

读/写字节数由数据长度寄存器(QUADSPI_DLR)指定。当 QUADSPI_DLR=0xFFFFFFFF 时，则数据长度视为未定义，QUADSPI 将持续传输数据，直到到达 FLASH 结尾（FLASH 容量由 QUADSPI_DCR[20:16]寄存器的 FSIZE[4:0]位定义）。如果不传输任何数据，则 DMODE[1:0](QUADSPI_CCR[25:24])应设置为 00。

当发送或接收的字节数（数据量）达到编程设定值时，如果 TCIE=1，则 TCF 置 1 并产生中断。在数据量不确定的情况下，将根据 FSIZE[4:0]定义的 FLASH 大小，在达到外部 SPI FLASH 的限制时，TCF 置 1。

在间接模式下有三种触发命令启动的方式，分别是：

（1）当不需要发送地址（ADMODE[1:0]==00）和数据（DMODE[1:0]==00）时，对 INSTRUCTION[7:0]（QUADSPI_CCR[7:0]）执行写入操作。

（2）当需要发送地址（ADMODE[1:0]!=00），但不需要发送数据（DMODE[1:0]==00），对 ADDRESS[31:0]（QUADSPI_AR）执行写入操作。

（3）当需要发送地址（ADMODE[1:0]!=00）和数据（DMODE[1:0]!=00）时，对 DATA[31:0]（QUADSPI_DR）执行写入操作。

如果命令启动，BUSY 位（QUADSPI_SR 的第 5 位）将自动置 1。

② 状态标志轮询模式

将 FMODE 字段(QUADSPI_CCR[27:26]) 设置为 10，使能状态标志轮询模式。在此模式下，将发送编程的帧并周期性检索数据。每帧中读取的最大数据量为 4 字节。如果 QUADSPI_DLR 请求更多的数据，则忽略多余部分并仅读取 4 个字节。在 QUADSPI_PISR 寄存器指定周期性。

在检索到状态数据后，可在内部进行处理，以达到以下目的：

（1）将状态匹配标志位置 1，如果使能，还将产生中断.

（2）自动停止周期性检索状态字节。

接收到的值可通过存储于 QUADSPI_PSMKR 寄存器中的值来进行屏蔽，并与存储在 QUADSPI_PSMAR 寄存器中的值进行或运算或与运算。

若是存在匹配，则状态匹配标志置 1，并且在使能了中断的情况下还将产生中断；如果 AMPS 位置 1，则 QUADSPI 自动停止。

在任何情况下，最新的检索值都在 QUADSPI_DR 中可用。

① 内存映射模式

在配置为内存映射模式时，外部 FLASH 器件被视作内部存储器，只是存在访问延迟。在该模式下，仅允许对外部 FLASH 执行读取操作。将 QUADSPI_CCR 寄存器中的 FMODE 设置为 11 可进入内存映射模式。当主器件访问存储器映射空间时，将发送已编程的指令和帧。另外数据长度寄存器（QUADSPI_DLR）在内存映射模式中无意义。

QUADSPI 外设若没有正确配置并使能，禁止访问 QUADSPI Flash 的存储区域。即使 FLASH 容量更大，寻址空间也无法超过 256MB。如果访问的地址超出 FSIZE 定义的范围但仍在 256MB 范围内，则生成总线错误。此错误的影响具体取决于尝试进行访问的总线主器件：

（1）如果为 Cortex® CPU，则会在使能总线故障时发生总线故障异常，在禁止总线故障时发生硬性故障(hard fault) 异常。

（2）如果为 DMA，则生成 DMA 传输错误，并自动禁用相应的 DMA 通道。

内存映射模式支持字节、半字和字访问类型，并支持芯片内执(XIP)操作，QUADSPI 接受下一个微控制器访问并提前加载后面地址中的字节。如果之后访问的是连续地址，由于值已经预取，访问将更快完成。

默认情况下，即便在很长时间内不访问 FLASH，QUADSPI 也不会停止预取操作，之前的读取操作将保持激活状态并且 nCS 保持低电平。由于 nCS 保持低电平时，FLASH 功耗增加，应用程序可能会激活超时计数器(TCEN = 1, QUADSPI_CR 的位 3)。从而在 FIFO 中写满预取的

数据后，若在 TIMEOUT[15:0] (QUADSPI_LPTR) 个周期的时长内没有访问，则释放 nCS。BUSY 在第一个存储器映射访问发生时变为高电平。由于进行预取操作，BUSY 在发生超时、中止或外设禁止前不会下降。

40.1.1.4 QSPI FLASH 配置

SPI FLASH 芯片的相关参数通过器件配置寄存器 (QUADSPI_DCR) 来进行设置。寄存器 QUADSPI_DCR[20:16]的 FSIZE[4:0]这 5 个位，用于指定外部存储器的大小，计算公式为：

Fcap = 2^[FSIZE+1]

FSIZE+1 是对 Flash 寻址所需的地址位数。Fcap 表示 FLASH 的容量，单位为字节，在间接模式下，最高支持 4GB（使用 32 位进行寻址）容量的 FLASH 芯片。但是在内存映射模式下的可寻址空间限制为 256MB。

QSPI 连续执行两条命令时，它在两条命令之间将片选信号 (nCS) 置为高电平默认仅一个 CLK 周期。某些 FLASH 需要命令之间的时间更长，可以通过寄存器 QUADSPI_DCR[10:8]的 CSHT[2:0]（选高电平时间）这 3 个位设置高电平时长：0~7 表示 1~8 个时钟周期（最大为 8）。

时钟模式，用于指定在 nCS 为高电平时，CLK 的时钟极性。通过寄存器 QUADSPI_DCR[0]的 CKMODE 位指定：当 CKMODE=0 时，CLK 在 nCS 为高电平期间保持低电平，称之为模式 0；当 CKMODE=1 时，CLK 在 nCS 为高电平期间保持高电平，称之为模式 3。

40.1.1.5 QUADSPI 寄存器

- **QUADSPI 控制寄存器（QUADSPI_CR）**

  QUADSPI 控制寄存器描述如图 40.1.1.5.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PRESCALER								PMM	APMS	Res.	TOIE	SMIE	FTIE	TCIE	TEIE
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	FTHRES					FSEL	DFM	Res.	SSHIFT	TCEN	Res.	ABORT	EN
			rw	rw	rw	rw	rw	rw	rw		rw	rw		rw	w1s

位 31:24 **PRESCALER[7:0]**：时钟预分频器 (Clock prescaler)
该字段定义基于 quadspi_ker_ck 时钟生成 CLK 所用的分频系数（值 + 1）。
0：$F_{CLK} = F_{quadspi_ker_ck}$，quadspi_ker_ck 时钟直接用作 QUADSPI CLK（预分频器被旁路）
1：$F_{CLK} = F_{quadspi_ker_ck}/2$ … 2：$F_{CLK} = F_{quadspi_ker_ck}/3$ 255：$F_{CLK} = F_{quadspi_ker_ck}/256$
对于奇数时钟分频系数，CLK 的占空比并非 50%。时钟信号的低电平持续时间比高电平持续时间多一个周期
仅可在 BUSY = 0 时修改该字段。

位 12:8 **FTHRES[4:0]**：FIFO 阈值级别 (FIFO threshold level)
定义在间接模式下 FIFO 中将导致 FIFO 阈值标志（FTF，QUADSPI_SR[2]）置 1 的字节数阈值。
在间接写入模式下 (FMODE = 00)：
0：如果 FIFO 中存在 1 个或更多空闲字节可供写入，则 FTF 置 1
1：如果 FIFO 中存在 2 个或更多空闲字节可供写入，则 FTF 置 1
…
31：如果 FIFO 中存在 32 个空闲字节可供写入，则 FTF 置 1
在间接读取模式下 (FMODE = 01)：
0：如果 FIFO 中存在 1 个或更多有效字节可供读取，则 FTF 置 1
1：如果 FIFO 中存在 2 个或更多有效字节可供读取，则 FTF 置 1
…
31：如果 FIFO 中存在 32 个有效字节可供读取，则 FTF 置 1

位 7 **FSEL**：FLASH 选择 (FLASH memory selection)
该位选择单闪存模式 (DFM = 0) 下要寻址的 FLASH。 0：选择 FLASH 1 1：选择 FLASH 2
仅可在 BUSY = 0 时修改该位。 在 DFM = 1 时忽略该位。

位 6 **DFM**：双闪存模式 (Dual-flash mode)
该位激活双闪存模式，同时使用两个外部 FLASH 以将吞吐量和容量扩大一倍。
0：禁止双闪存模式 1：使能双闪存模式。 仅可在 BUSY = 0 时修改该位。

位 4 **SSHIFT**：采样移位 (Sample shift)
默认情况下，QUADSPI 在 FLASH 驱动数据后过半个 CLK 周期开始采集数据。使用该
位，可考虑外部信号延迟，推迟数据采集。 0：不发生移位 1：移位半个周期
在 DDR 模式下 (DDRM = 1)，固件必须确保 SSHIFT = 0。
仅可在 BUSY = 0 时修改该字段。

位 1 **ABORT**：中止请求 (Abort request)
该位中止执行中的命令序列。在中止完成时自动复位。该位可停止当前的传输。
在轮询模式或内存映射模式下，该位也用以复位 APM 位或 DM 位。 0：不请求中止 1：请求中止

位 0 **EN**：使能 (Enable) 使能 QUADSPI。 0：禁止 QUADSPI 1：使能 QUADSPI

图 40.1.1.5.1 QUADSPI_CR 寄存器

该寄存器我们只关心需要用到的一些位（下同），首先是 PRESCALER[7:0]，用于设置 AHB 时钟预分频器：0~255，表示 0~256 分频。我们使用的 W25Q256 最大支持 133Mhz 的时钟，这里我们设置 PRESCALER=1，即 2 分频，得到 QSPI 时钟为 100Mhz（200/2）。

FTHRES[4:0]，用于设置 FIFO 阈值，范围为 0~31，表示 FIFO 的阈值为 1~32 字节。

FSEL 位，用于选择 FLASH，我们的 W25Q256 连接在 STM32H7 的 QSPI BK1 上面，所以设置此位为 0 即可。

DFM 位，用于设置双闪存模式，我们用的是单闪存模式，所以设置此位为 0 即可。

SSHIFT 位，用于设置采样移位，默认情况下，QSPI 接口在 FLASH 驱动数据后过半个 CLK 周期开始采集数据。使用该位，可考虑外部信号延迟，推迟数据采集。我们一般设置此位为 1，移位半个周期采集，确保数据稳定。

ABORT 位，用于终止 QSPI 的当前传输，设置为 1 即可终止当前传输，在读写 FLASH 数据的时候，可能会用到。

EN 位，用于控制 QSPI 的使能，我们需要用到 QSPI 接口，所以必须设置此位为 1。

- **QUADSPI 器件配置寄存器（QUADSPI_ DCR）**

QUADSPI 器件配置寄存器描述如图 40.1.1.5.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FSIZE				
											rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	CSHT			Res.	Res.	Res.	Res.	Res.	Res.	Res.	CK-MODE
					rw	rw	rw								rw

位 20:16 **FSIZE[4:0]**：FLASH 大小 (FLASH memory size)

该字段使用下面的公式定义了外部存储器的大小：

FLASH 中的字节数 = $2^{[FSIZE+1]}$

FSIZE+1 是对 FLASH 寻址所需的地址位数。在间接模式下，FLASH 容量最高可达 4GB（使用 32 位进行寻址），但在内存映射模式下的可寻址空间限制为 256MB。

如果 DFM = 1，FSIZE 表示两个 FLASH 容量的总和。

仅可在 BUSY = 0 时修改该字段。

位 10:8 **CSHT[2:0]**：片选高电平时间 (Chip select high time)

CSHT+1 定义片选 (nCS) 在发送至 FLASH 的命令之间必须保持高电平的最少 CLK 周期数。

0：nCS 在 FLASH 命令之间保持高电平至少 1 个周期

1：nCS 在 FLASH 命令之间保持高电平至少 2 个周期

...

7：nCS 在 FLASH 命令之间保持高电平至少 8 个周期

仅可在 BUSY = 0 时修改该字段。

位 0 **CKMODE**：模式 0/模式 3 (Mode 0 / mode 3)

该位指示 CLK 在命令之间（nCS = 1 时）的电平。

0：nCS 为高电平（片选释放）时，CLK 必须保持低电平。这称为模式 0。

1：nCS 为高电平（片选释放）时，CLK 必须保持高电平。这称为模式 3。

仅可在 BUSY = 0 时修改该字段。

图 40.1.1.5.2 QUADSPI_ DCR 寄存器

该寄存器可以设置 FLASH 芯片的容量（FSIZE）、片选高电平时间（CSHT）和时钟模式（CKMODE）等，这些位的设置说明见前面的 40.1.1.4 小节有详细讲解。

- **QUADSPI 状态寄存器（QUADSPI_ SR）**

QUADSPI 状态寄存器描述如图 40.1.1.5.3 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	FLEVEL[5:0]						Res.	Res.	BUSY	TOF	SMF	FTF	TCF	TEF
		r	r	r	r	r	r			r	r	r	r	r	r

位 13:8 **FLEVEL[5:0]**：FIFO 级别 (FIFO level)

该字段给出 FIFO 中的有效字节数。FIFO 为空时 FLEVEL = 0，写满时 FLEVEL = 32。在内存映射模式和自动状态轮询模式下，FLEVEL 为零。

位 5 **BUSY**：忙 (Busy)

操作进行时，该位置 1。在对 FLASH 的操作完成并且 FIFO 为空时，该位自动清零。

位 4 **TOF**：超时标志 (Timeout flag)

发生超时时该位置 1。向 CTOF 写入 1 可将该位清零。

位 3 **SMF**：状态匹配标志 (Status match flag)

在自动轮询模式下，若未屏蔽的接收数据与匹配寄存器 (QUADSPI_PSMAR) 中的对应位相匹配，则该位置 1。向 CSMF 写入 1 可将该位清零。

位 2 **FTF**：FIFO 阈值标志 (FIFO threshold flag)

在间接模式下，若达到 FIFO 阈值，或从 FLASH 读取完成后，FIFO 中留有数据时，该位置 1。只要阈值条件不再为“真”，该位就自动清零。

在自动轮询模式下，每次读取状态寄存器时，该位即置 1；读取数据寄存器时，该位清零。

位 1 **TCF**：传输完成标志 (Transfer complete flag)

在间接模式下，当传输的数据数量达到编程设定值，或在任何模式下传输中止时，该位置 1。向 CTCF 写入 1 时，该位清零。

位 0 **TEF**：传输错误标志 (Transfer error flag)

在间接模式下访问无效地址时，该位置 1。向 CTEF 写入 1 可将该位清零。

图 40.1.1.5.3 QUADSPI_ SR 寄存器

BUSY 位，指示操作是否忙。当该位为 1 时，表示 QSPI 正在执行操作。在操作完成或者 FIFO 为空的时候，该位自动清零。

FTF 位，表示 FIFO 是否到达阈值。在间接模式下，若达到 FIFO 阈值，或从 FLASH 读取完成后，FIFO 中留有数据时，该位置 1。只要阈值条件不再为“真”，该位就自动清零。

TCF 位，表示传输是否完成。在间接模式下，当传输的数据数量达到编程设定值，或在任何模式下传输中止时，该位置 1。向 QUADSPI_FCR 寄存器的 CTCF 位写 1，可以清零此位。

- **QUADSPI 标志清零寄存器（QUADSPI_ FCR）**

QUADSPI 标志清零寄存器描述如图 40.1.1.5.4 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CTOF	CSMF	Res.	CTCF	CTEF
											w1o	w1o		w1o	w1o

位 4 **CTOF**：清除超时标志 (Clear timeout flag)
写入 1 可将 QUADSPI_SR 寄存器中的 TOF 标志清零
位 3 **CSMF**：清除状态匹配标志 (Clear status match flag)
写入 1 可将 QUADSPI SR 寄存器中的 SMF 标志清零
位 1 **CTCF**：清除传输完成标志 (Clear transfer complete flag)
写入 1 可将 QUADSPI_SR 寄存器中的 TCF 标志清零
位 0 **CTEF**：清除传输错误标志 (Clear Transfer error flag)
写入 1 可将 QUADSPI_SR 寄存器中的 TEF 标志清零

图 40.1.1.5.4 QUADSPI_ FCR 寄存器

该寄存器，我们一般只用到 CTCF 位，用于清除 QSPI 的传输完成标志。

- **QUADSPI 通信配置寄存器（QUADSPI_ CCR）**

QUADSPI 通信配置寄存器描述如图 40.1.1.5.5 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DDRM	DHHC	FRCM	SIOO	FMODE[1:0]		DMODE		Res.	DCYC[4:0]					ABSIZE	
rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABMODE		ADSIZE		ADMODE		IMODE		INSTRUCTION[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 40.1.1.5.5 QUADSPI_ CCR 寄存器

DDRM 位，用于设置双倍率模式（DDR），我们没用到双倍率模式，所以设置此位为 0。

SIOO 位，用于设置指令是否只发送一次，我们需要每次都发送指令，所以设置此位为 0。

FMODE[1:0]，这两个位用于设置功能模式：00，间接写入模式；01，间接读取模式；10，自动轮询模式；11，内存映射模式；我们使用间接模式，所以此位根据需要设置为 00/01。

DMODE[1:0]，这两个位用于设置数据模式：00，无数据；01，单线传输数据；10，双线传输数据；11，四线传输数据；我们一般设置为 00/11。

DCYC[4:0]，这 5 个位用于设置空指令周期数，可以控制空指令阶段的持续时间，设置范围为：0~31。设置为 0，则表示没有空指令周期。

ABMODE[1:0]，这两个位用于设置交替字节模式，我们一般设置为 0，表示无交替字节。

ADMODE[1:0]，这两个位用于设置地址模式：00，无地址；01，单线传输地址；10，双线传输地址；11，四线传输地址；我们一般设置为 00/11。

IMODE[1:0]，这两个位用于设置指令模式：00，无指令；01，单线传输指令；10，双线传输指令；11，四线传输指令；我们一般设置为 00/11。

INSTRUCTION[7:0]，这 8 个位用于设置将要发送给 FLASH 的指令。

注意，以上这些位的配置，都必须在 QUADSPI_SR 寄存器的 BUSY 位为 0 时才可配置。

接下来，我们看 QSPI 数据长度寄存器：QUADSPI_DLR，该寄存器为一个 32 位寄存器，

可以设置的数据长度范围为：0~0XFFFFFFFF，当 QUADSPI_DLR!=0XFFFFFFFF 时，表示传输的字节长度（+1）；当 QUADSPI_DLR==0XFFFFFFFF 时，表示不限传输长度，直到到达由 FSIZE 定义的 FLASH 结尾。

接下来，我们看 QSPI 地址寄存器：QUADSPI_AR，该寄存器为一个 32 位寄存器，用于指定发送到 FLASH 的地址。

接下来，我们看 QSPI 数据寄存器：QUADSPI_DR，该寄存器为一个 32 位寄存器，用于指定与外部 SPI FLASH 设备交换的数据。该寄存器支持字、半字和字节访问。

在间接写入模式下，写入该寄存器的数据在数据阶段发送到 FLASH，在此之前则存储于 FIFO，如果 FIFO 满了，则暂停写入，直到 FIFO 具有足够的空间接受要写入的数据才继续。

在间接模式下，读取该寄存器可获得（通过 FIFO）已从 FLASH 接收的数据。如果 FIFO 所含字节数比读取操作要求的字节数少，且 BUSY=1，则暂停读取，直到足够的数据出现或传输完成才继续。

40.1.2 NOR FLASH 芯片简介

NOR FLASH 芯片有很多种芯片型号，在我们的 norflash.h 头文件中有定义芯片 ID 的宏定义，对应的就是不同型号的 NOR FLASH 芯片，比如有：W25Q128、W25Q256、BY25Q128、NM25Q128，它们是来自不同的厂商的同种规格的 NOR FLASH 芯片，内存空间都是 128M 字，即 16M 字节。它们的很多参数、操作都是一样的，所以我们的实验都是兼容它们的。

由于这么多的芯片我们就不一一进行介绍了，就拿其中一款型号进行介绍即可，其他的型号都是类似的。

W25Q256 是华邦公司生产的一颗容量为 32M 字节的串行 FLASH 芯片，它将 32M 的容量分为 512 个块（Block），每个块大小为 64K 字节，每个块又分为 16 个扇区（Sector），每个扇区 4K 个字节。W25Q256 的最小擦除单位为一个扇区，也就是每次必须擦除 4K 个字节。这样我们需要给 W25Q256 开辟一个至少 4K 的缓存区，这样对 SRAM 要求比较高，要求芯片必须有 4K 以上 SRAM 才能很好的操作。

W25Q256 的擦写周期多达 10W 次，具有 20 年的数据保存期限，支持电压为 2.7~3.6V，W25Q256 支持标准的 SPI，还支持双输出/四输出 SPI 和 QPI（QPI 即 QSPI），最高时钟频率可达 104Mhz（双输出时相当于 208Mhz，四输出时相当于 416M），本实验我们将使用 STM32H7 的 QSPI 接口来实现对 W25Q256 的驱动。

接下来，我们介绍一下本实验驱动 W25Q256 需要用到的一些指令，如表 40.1.2.1 所示：

输入/输出数据		字节 1	字节 2	字节 3	字节 4	字节 5	字节 6	字节 7
时钟数	SPI 模式	0-7	8-15	16-23	24-31	32-39	40-47	48-55
	QPI 模式	0,1	2,3	4,5	6,7	8,9	10,11	12,13
W25X_ReadStatusReg1		0X05	S7-S0					
W25X_ReadStatusReg2		0X35	S15-S8					
W25X_ReadStatusReg3		0X15	S23-S16					
W25X_WriteStatusReg1		0X01	S7-S0					
W25X_WriteStatusReg2		0X31	S15-S8					
W25X_WriteStatusReg3		0X11	S23-S16					
W25X_ManufactDeviceID		0X90	Dummy	Dummy	0X00	MF7-MF0	ID7-ID0	
W25X_EnterQPIMode		0X38						
W25X_Enable4ByteAddr		0XB7						
W25X_SetReadParam		0XC0	P7-P0					
W25X_WriteEnable		0X06						
W25X_FastReadData		0X0B	A31-A24	A23-A16	A15-A8	A7-A0	Dummy 1	D7-D02

W25X_PageProgram	0X02	A31-A24	A23-A16	A15-A8	A7-A0	D7-D02	D7-D02
W25X_SectorErase	0X20	A31-A24	A23-A16	A15-A8	A7-A0		
W25X_ChipErase	0XC7						

表 40.1.2.1 W25Q256 指令

上表列出了本章我们驱动 W25Q256 所需要用到的所有指令和对应的参数，注意 SPI 模式和 QPI 模式下时钟数的区别，可知 QPI 模式比 SPI 模式所需要的时钟数少的多，所以速度也快得多。接下来我们简单介绍一下这些指令。

首先，前面 6 个指令，是用来读取/写入状态寄存器 1~3 的。在读取的时候，读取 S23~S0 的数据，在写入的时候，写入 S23~S0。而 S23~S0 则由三部分组成：S23~S16，S15~S8，S7~S0 即状态寄存器 3、2、1，如表 40.1.2.2 所示：

状态寄存器 3	S23	S22	S21	S20	S19	S18	S17	S16
位说明	HOLD/RST	DRV1	DRV0			WPS	ADP	ADS
状态寄存器 2	S15	S14	S13	S12	S11	S10	S9	S8
位说明	SUS	CMP	LB3	LB2	LB1		QE	SRP1
状态寄存器 1	S7	S6	S5	S4	S3	S2	S1	S0
位说明	SRP0	TB	BP3	BP2	BP1	BP0		BUSY

表 40.1.2.2 W25Q256 状态寄存器

这三个状态寄存器，我们只关心我们需要用到的一些位：ADS、QE 和 BUSY 位。其他位的说明，请看 W25Q256 的数据手册。

ADS 位，表示 W25Q256 当前的地址模式，是一个只读位，当 ADS=0 的时候，表示当前是 3 字节地址模式，当 ADS=1 的时候，表示当前是 4 字节地址模式，我们需要使用 4 字节地址模式，所以在读取到该位为 0 的时候，必须通过 W25X_Enable4ByteAddr 指令，设置为 4 字节地址模式。

QE 位，用于使能 4 线模式（Quad），此位可读可写，并且是可以保存的（掉电后可以继续保持上一次的值）。在本章，我们需要用到 4 线模式，所以在读到该位为 0 的时候，必须通过 W25X_WriteStatusReg2 指令设置此位为 1，表示使能 4 线模式。

BUSY 位，用于表示擦除/编程操作是否正在进行，当擦除/编程操作正在进行时，此位为 1，此时 W25Q256 不接受任何指令，当擦除/编程操作完成时，此位为 0。此位为只读位，我们在执行某些操作的时候，必须等待此位为 0。

W25X_ManufactDeviceID 指令，用于读取 W25Q256 的 ID，可以用于判断 W25Q256 是否正常。对于 W25Q256 来说：MF[7:0]=0XEF，ID[7:0]=0X18。

W25X_EnterQPIMode 指令，用于设置 W25Q256 进入 QPI 模式。上电时，W25Q256 默认是 SPI 模式，我们需要通过该指令设置其进入 QPI 模式。注意：在发送该指令之前，必须先设置状态寄存器 2 的 QE 位为 1！！

W25X_Enable4ByteAddr 指令，用于设置 W25Q256 进入 4 字节地址模式。当读取到 ADS 位为 0 的时候，我们必须通过此指令将 W25Q256 设置为 4 字节地址模式，否则将只能访问 16MB 的地址空间。

W25X_SetReadParam 指令，可以用于设置读参数控制位 P[5:4]，这两个位的描述如表 40.1.2.3 所示：

P5 – P4	DUMMY CLOCKS	MAXIMUM READ FREQ.	MAXIMUM READ FREQ. (A[1:0]=0,0)	MAXIMUM READ FREQ. (A[1:0]=0,0 VCC=3.0V~3.6V)
0 0	2	33MHz	33MHz	40MHz
0 1	4	55MHz	80MHz	80MHz
1 0	6	80MHz	80MHz	104MHz
1 1	8	80MHz	80MHz	104MHz

表 40.1.2.3 W25Q256 读参数控制位

为了让 W25Q256 可以工作在最大频率下，我们这里设置 P[5:4]=11，即可工作在 104Mhz 的时钟频率下。此时，读取数据时的 dummy 时钟个数为 8 个（参见 W25X_FastReadData 指令）。

W25X_WriteEnable 指令，用于设置 W25Q256 写使能。在执行擦除、编程、写状态寄存器等操作之前，都必须通过该指令，设置 W25Q256 写使能，否则无法写入。

W25X_FastReadData 指令，用于读取 FLASH 数据，在发送完该指令以后，就可以读取 W25Q256 的数据了。该指令发送完成后，我们可以持续读取 FLASH 里面的数据，只要不停的给时钟，就可以不停的读取数据。

W25X_PageProgram 指令，用于编程 FLASH（写入数据到 FLASH），该指令发送完成后，最多可以一次写入 256 字节到 W25Q256，超过 256 字节则需要多次发送该指令。

W25X_SectorErase 指令，用于擦除一个扇区（4KB）的数据。因为 FLASH 具有只可以写 0，不可以写 1 的特性，所以在写入数据的时候，一般需要先擦除（归 1），再写。W25Q256 的最小擦除单位为一个扇区（4KB）。该指令在写入数据的时候，经常要有用。

W25X_ChipErase 指令，用于全片擦除 W25Q256。

为了在程序上方便使用，我们把 FLASH 芯片的常用指令编码定义为宏定义的形式，存放在 norflash.h 文件中。

40.2 硬件设计

1. 例程功能

通过 KEY1 按键来控制 norflash 的写入，通过按键 KEY0 来控制 norflash 的读取。并在 LCD 模块上面显示相关信息。我们还可以通过 USMART 控制读取 norflash 的 ID、擦除某个扇区或整片擦除。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）独立按键：KEY0 - PH3、KEY1 - PH2
5）QSPI(PB2/PB6/PF6/PF7/PF8/PF9)
6）norflash(QSPI FLASH 芯片，连接在 QSPI 接口上)

3. 原理图

板载的 QSPI FLASH 芯片与 STM32H743 的连接关系，如下图所示：

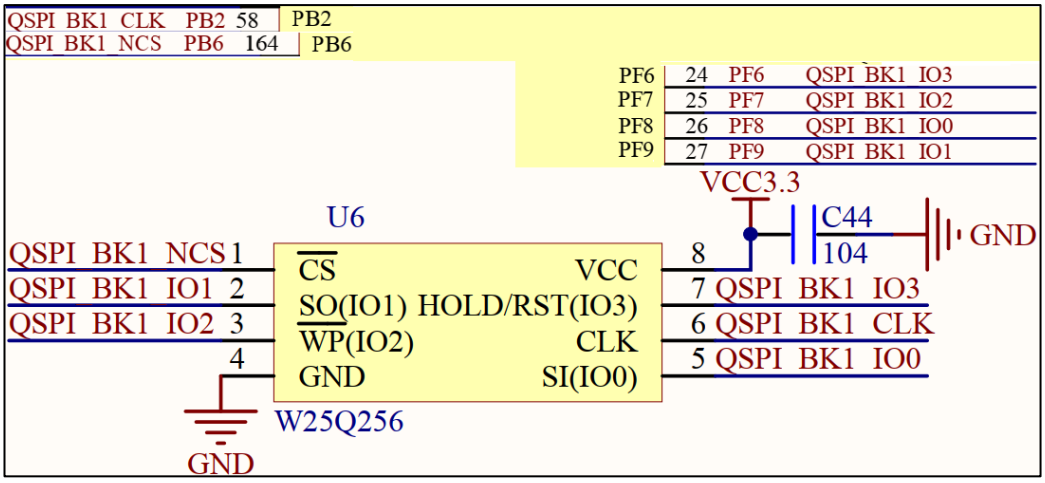


图 40.2.1 QSPI FLASH 芯片与 STM32H743 连接示意图

本实验支持多种型号的QSPI FLASH芯片，比如：BY25Q128/NM25Q128/W25Q128等等，具体请看 norflash.h 文件的宏定义，程序上只需要稍微修改一下，后面讲解程序的时候会说到。

40.3 程序设计

40.3.1 QSPI 的 HAL 库驱动

QSPI 在 HAL 库中的驱动代码在 stm32h7xx_hal_qspi.c 文件（及其头文件）中。

1. HAL_QSPI_Init 函数

QSPI 的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_QSPI_Init(QSPI_HandleTypeDef *hqspi);
```

- **函数描述：**

  用于初始化 QSPI。

- **函数形参：**

  形参 1 是 QSPI_HandleTypeDef 结构体类型指针变量，其定义如下：

```
typedef struct
{
  QUADSPI_TypeDef              *Instance;        /* QSPI 寄存器基址 */
  QSPI_InitTypeDef              Init;            /* QSPI 参数配置结构体 */
  uint8_t                      *pTxBuffPtr;      /* 要发送数据的地址 */
  __IO uint16_t                 TxXferSize;      /* 要发送数据的大小 */
  __IO uint16_t                 TxXferCount;     /* 剩余要发送数据的个数 */
  uint8_t                      *pRxBuffPtr;      /* 要接收数据的地址 */
  __IO uint16_t                 RxXferSize;      /* 要接收数据的大小 */
  __IO uint16_t                 RxXferCount;     /* 剩余要接收数据的个数 */
  DMA_HandleTypeDef            * hmdma;          /* DMA 配置结构体 */
  __IO HAL_LockTypeDef          Lock;            /* 锁对象 */
  __IO HAL_QSPI_StateTypeDef    State;           /* QSPI 通信状态 */
  __IO uint32_t                 ErrorCode;       /* 错误代码 */
  uint32_t                      Timeout;         /* 配置 QSPI 内存访问超时时间 */
}QSPI_HandleTypeDef;
```

1) Instance：用于设置 QSPI 寄存器基地址，设置为 QUADSPI 即可，这个官方已经为我们做好了宏定义。

2) Init：用于设置 QSPI 的相关参数，QSPI_InitTypeDef 结构体下面再进行详细讲解。

3) pTxBuffPtr、TxXferSize 和 TxXferCount：分别用于设置 QSPI 发送缓冲指针、发送数据量和发送剩余数据量。

4) pRxBuffPtr、RxXferSize 和 RxXferCount：分别用于设置接收缓冲指针、接收数据量和接收剩余数据量。

5) hmdma：用于配置相关的 DMA 参数。

6) Lock：用于分配锁资源，可选择 HAL_UNLOCKED 或者是 HAL_LOCKED 两个参数。

7) State：用于存放通讯过程中的工作状态。

8) ErrorCode：通过该参数，用户可以了解到 QSPI 通讯过程中通信失败的原因。

9) Timeout：用于设置超时时间。QSPI 访问时间一旦超出 Timeout 这个变量值，那么 ErrorCode 成员变量就会被赋值为 HAL_QSPI_ERROR_TIMEOUT，表示操作超时。

下面重点来了解 QSPI_InitTypeDef 结构体的内容，其定义如下：

```
typedef struct
{
  uint32_t ClockPrescaler;             /* 时钟预分频系数 */
  uint32_t FifoThreshold;              /* 设置 FIFO 阈值级别 */
  uint32_t SampleShifting;             /* 设置采样移位 */
  uint32_t FlashSize;                  /* 设置 FLASH 大小 */
  uint32_t ChipSelectHighTime;         /* 设置片选高电平时间 */
  uint32_t ClockMode;                  /* 设置时钟模式 */
  uint32_t FlashID;                    /* 闪存 ID，第一片还是第二片 */
  uint32_t DualFlash;                  /* 双闪存模式设置 */
```

```
}QSPI_InitTypeDef;
```

1) ClockPrescaler：用于设置预分频系数，对应 QUADSPI_CR 寄存器的 PRESCALER[7:0]位，取值范围是 0~255。仅可在 BUSY = 0 时修改该字段。
2) FifoThreshold：用于设置 FIFO 阈值级别，可设置范围为 0~31，对应 QUADSPI_CR 寄存器的 FTHRES[4:0]位。
3) SampleShifting：用于设置采样移位，对应 QUADSPI_CR 寄存器的 SSHIFT 位。使用该位是考虑到外部信号延迟时，推迟数据采样。可以取值 QSPI_SAMPLE_SHIFTING_NONE（即 0）：不发生移位；QSPI_SAMPLE_SHIFTING_HALFCYCLE（即 1）：移位半个周期。在 DDR 模式下 (DDRM = 1)，固件必须确保 SSHIFT = 0。
4) FlashSize：用于设置 FLASH 大小，对应 QUADSPI_ DCR 寄存器的 FSIZE[4:0]位，可设置的范围是：0 到 31 之间的整数。FLASH 中的字节数= $2^{[FSIZE+1]}$。在间接模式下，FLASH 容量最高可达 4GB（使用 32 位进行寻址），但在内存映射模式下的可寻址空间限制为 256MB。
5) ChipSelectHighTime：用于设置片选高电平时间，取值范围：QSPI_CS_HIGH_TIME_1_CYCLE～QSPI_CS_HIGH_TIME_8_CYCLE，表示 1~8 个周期，对应 QUADSPI_DCR 寄存器的 CSHT[2:0]位。CSHT+1 定义片选 (nCS) 在发送至 Flash 的命令之间必须保持高电平的最少 CLK 周期数。
6) ClockMode：用于设置时钟模式，对应 QUADSPI_DCR 寄存器 CKMODE 位，指示 CLK 在命令之间（nCS = 1 时）的电平，可以选择的参数是：QSPI_CLOCK_MODE_0（表示模式 0）或者 QSPI_CLOCK_MODE_3（表示模式 3）。模式 0 是：nCS 为高电平（片选释放）时，CLK 必须保持低电平。模式 3 是：nCS 为高电平（片选释放）时，CLK 必须保持高电平。
7) FlashID：用于选择 Flash1 或者 Flash2，单闪存模式下选择 QSPI_FLASH_ID_1（表示 Flash1）。
8) DualFlash：用于使能双闪存模式，QSPI_DUALFLASH_DISABLE：禁止双闪存模式；QSPI_DUALFLASH_ENABLE：使能双闪存模式。对应 QUADSPI_CR 寄存器 DFM 位。

- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。
- **注意事项：**
  QSPI 的 MSP 初始化函数 HAL_QSPI_MspInit，该函数声明如下：

```
void HAL_QSPI_MspInit(QSPI_HandleTypeDef *hqspi);
```

2. HAL_QSPI_Command 函数

QSPI 设置命令配置函数，其声明如下：

```
HAL_StatusTypeDef HAL_QSPI_Command(QSPI_HandleTypeDef *hqspi,
                                   QSPI_CommandTypeDef *cmd, uint32_t Timeout);
```

- **函数描述：**
  该函数用来配置 QSPI 命令。
- **函数形参：**
  形参 1 是 QSPI_HandleTypeDef 结构体类型指针变量。
  形参 2 是 QSPI_CommandTypeDef 结构体类型指针变量，其定义如下：

```
typedef struct
{
  uint32_t Instruction;            /* 指令 */
  uint32_t Address;                /* 地址 */
  uint32_t AlternateBytes;         /* 交替字节 */
  uint32_t AddressSize;            /* 地址长度 */
  uint32_t AlternateBytesSize;     /* 交替字节大小 */
  uint32_t DummyCycles;            /* 控指令周期数 */
  uint32_t InstructionMode;        /* 指令模式 */
  uint32_t AddressMode;            /* 地址模式 */
  uint32_t AlternateByteMode;      /* 交替字节模式 */
  uint32_t DataMode;               /* 数据模式 */
  uint32_t NbData;                 /* 数据长度 */
  uint32_t DdrMode;                /* 指定地址、备用字节和数据阶段的双数据速率模式 */
  uint32_t DdrHoldHalfCycle;       /* 指定 DDR 模式下数据保持的周期 */
  uint32_t SIOOMode;               /* 指定发送指令仅一次模式 */
```

```
}QSPI_CommandTypeDef;
```

1) Instruction：设置通信指令，指定要发送到外部 QSPI 设备的指令，指令表定义在 norflash.h 里。
2) Address：指定要发送到外部 QSPI 设备的地址，BUSY = 0 或 FMODE = 11（内存映射模式）时，将忽略写入该字段。在双闪存模式下，由于地址始终为偶地址，ADDRESS[0] 自动保持为“0”。
3) AlternateBytes：指定要在地址后立即发送到外部 QSPI 设备的可选数据。
4) AddressSize：定义地址长度，可以是 8 位，16 位，24 位或者 32 位。
5) AlternateBytesSize：定义交替字节长度，可以是 8 位，16 位，24 位或者 32 位。
6) DummyCycles：定义空指令阶段持续周期，SDR 和 DDR 模式下，指定 CLK 周期数(0~31)。
7) InstructionMode：用于指定指令阶段模式，如下四种：
   QSPI_INSTRUCTION_NONE：无指令；
   QSPI_INSTRUCTION_1_LINE：单线传输指令；
   QSPI_INSTRUCTION_2_LINES：双线传输指令；
   QSPI_INSTRUCTION_4_LINES：四线传输指令。
8) AddressMode：指定地址模式，如下四种：
   QSPI_ADDRESS_NONE：无地址；QSPI_ADDRESS_1_LINE：单线传输地址；
   QSPI_ADDRESS_2_LINES：双线传输地址；QSPI_ADDRESS_4_LINES：四线传输地址。
9) AlternateByteMode：指定交替字节模式，如下四种：
   QSPI_ALTERNATE_BYTES_NONE：无交替字节；
   QSPI_ALTERNATE_BYTES_1_LINE：单线传输交替字节；
   QSPI_ALTERNATE_BYTES_2_LINES：双线传输交替字节；
   QSPI_ALTERNATE_BYTES_4_LINES：四线传输交替字节。
10) DataMode：指定数据模式，如下四种：
   QSPI_DATA_NONE：无数据；
   QSPI_DATA_1_LINE：单线传输数据；
   QSPI_DATA_2_LINES：双线传输数据；
   QSPI_DATA_4_LINES：四线传输数据。
11) NbData：用于设置数据长度，在间接模式和状态轮询模式下待检索的数据数量（值 + 1）。对状态轮询模式应使用不大于 3 的值（表示 4 字节）。
12) DdrMode：为地址、交替字节和数据阶段设置 DDR 模式，可以选择的值是：
   QSPI_DDR_MODE_DISABLE：禁止 DDR 模式；
   QSPI_DDR_MODE_ENABLE：使能 DDR 模式。
13) DdrHoldHalfCycle：用于设置 DDR 模式下数据输出延迟 1/4 个 QUADSPI 输出时钟周期，可选值如下：
   QSPI_DDR_HHC_ANALOG_DELAY：使用模拟延迟来延迟数据输出；
   QSPI_DDR_HHC_HALF_CLK_DELAY：数据输出延迟 1/4 个 QUADSPI 输出时钟周期。
14) SIOOMode：设置是否开启仅发送指令一次模式，可选值如下：
   QSPI_SIOO_INST_EVERY_CMD：在每个事务中发送指令；
   QSPI_SIOO_INST_ONLY_FIRST_CMD：仅为第一条命令发送指令。

形参 3 用于设置超时时间。

- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

3. HAL_QSPI_Receive 函数

QSPI 接收数据函数，其声明如下：

```
HAL_StatusTypeDef HAL_QSPI_Receive(QSPI_HandleTypeDef *hqspi,
                                   uint8_t *pData, uint32_t Timeout);
```

- **函数描述：**
  该函数用来接收数据。
- **函数形参：**

形参 1 是 QSPI_HandleTypeDef 结构体类型指针变量。
形参 2 是 uint8_t 类型指针变量，存放接收数据缓冲区指针。
形参 3 设置操作超时时间。

- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

4. HAL_QSPI_Transmit 函数

QSPI 发送数据函数，其声明如下：

```
HAL_StatusTypeDef HAL_QSPI_Transmit (QSPI_HandleTypeDef *hqspi,
                                     uint8_t *pData, uint32_t Timeout);
```

- **函数描述：**
  该函数用来发送数据。
- **函数形参：**
  形参 1 是 QSPI_HandleTypeDef 结构体类型指针变量。
  形参 2 是 uint8_t 类型指针变量，存放发送数据缓冲区指针。
  形参 3 设置操作超时时间。
- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

QSPI 初始化配置步骤

1）开启 QSPI 接口和相关 IO 的时钟，设置 IO 口的复用功能。

要使用 QSPI，肯定要先开启其时钟（由 AHB3ENR 控制），然后根据我们使用的 QSPI IO 口，开启对应 IO 口的时钟，并初始化相关 IO 口的复用功能（选择 QSPI 复用功能）。

QSPI 时钟使能方法为：

```
__HAL_RCC_QSPI_CLK_ENABLE();          /* 使能 QSPI 时钟 */
```

这里大家要注意，和其他外设处理方法一样，HAL 库提供了 QSPI 的初始化回调函数 HAL_QSPI_MspInit，一般用来编写与 MCU 相关的初始化操作。时钟使能和 IO 口初始化一般在回调函数中编写。

2）设置 QSPI 相关参数。

此部分需要设置两个寄存器：QUADSPI_CR 和 QUADSPI_DCR，控制 QSPI 的时钟、片选参数、FLASH 容量和时钟模式等参数，设定 SPI FLASH 的工作条件。最后，使能 QSPI，完成对 QSPI 的初始化。HAL 库中设置 QSPI 相关参数函数为 HAL_QSPI_Init，该函数声明为：

```
HAL_StatusTypeDef HAL_QSPI_Init(QSPI_HandleTypeDef *hqspi);
```

QSPI_HandleTypeDef 结构体这些成员变量是用来配置 QUADSPI_CR 寄存器和 QUADSPI_DCR 寄存器相应位，大家可以结合这两个寄存器的位定义和结构体定义来理解。

对于 HAL_QSPI_Init 函数使用范例请参考后面 40.3 软件设置部分程序源码。

QSPI 发送命令步骤

1）等待 QSPI 空闲。

在 QSPI 发送命令前，必须先等待 QSPI 空闲，通过判断 QUADSPI_SR 寄存器的 BUSY 位为 0，来确定。

2）设置命令参数。

此部分主要是通过通信配置寄存器（QUADSPI_CCR）设置，将 QSPI 配置为：每次都发送指令、间接写模式，根据具体需要设置：指令、地址、空周期和数据等的传输位宽等信息。如果需要发送地址，则配置地址寄存器（QUADSPI_AR）。

在配置完成以后，即可启动发送。如果不需要传输数据，则需要等待命令发送完成（等待 QUADSPI_SR 寄存器的 TCF 位为 1）。

在 HAL 库中上述两个步骤是通过函数 HAL_QSPI_Command 来实现，该函数声明为：

```
HAL_StatusTypeDef HAL_QSPI_Command(QSPI_HandleTypeDef *hqspi,
                                   QSPI_CommandTypeDef *cmd, uint32_t Timeout);
```

QSPI 读数据步骤

1）设置数据传输长度。

通过设置数据长度寄存器（QUADSPI_DLR），配置需要传输的字节数。

2）设置 QSPI 工作模式并设置地址。

因为要读取数据，所以，设置 QUADSPI_CCR 寄存器的 FMODE[1:0]位为 01，工作在间接读取模式。然后，通过地址寄存器（QUADSPI_AR），设置我们将要读取的数据的首地址。

3）读取数据。

在发送完地址以后，就可以读取数据了，不过要等待数据准备好，通过判断 QUADSPI_SR 寄存器的 FTF 和 TCF 位，当这两个位任意一个位为 1 的时候，我们就可以读取 QUADSPI_DR 寄存器来获取从 FLASH 读到的数据。

最后，在所有数据接收完成以后，终止传输（ABORT），清除传输完成标志位（TCF）。

HAL 库中，读取数据是通过函数 HAL_QSPI_Receive 来实现的，该函数声明为：

```
HAL_StatusTypeDef HAL_QSPI_Receive(QSPI_HandleTypeDef *hqspi,
                                   uint8_t *pData, uint32_t Timeout);
```

在调用该函数读取数据之前，我们会先调用上个步骤讲解的函数 HAL_QSPI_Command 来指定读取数据的存放空间。

QSPI 写数据步骤

1）设置数据传输长度。

通过设置数据长度寄存器（QUADSPI_DLR），配置需要传输的字节数。

2）设置 QSPI 工作模式并设置地址。

因为要读取数据，所以，设置 QUADSPI_CCR 寄存器的 FMODE[1:0]位为 00，工作在间接写入模式。然后，通过地址寄存器（QUADSPI_AR），设置我们将要写入的数据的首地址。

3）写入数据。

在发送完地址以后，就可以写入数据了，不过要等待 FIFO 不满，通过判断 QUADSPI_SR 寄存器的 FTF 位，当这个位为 1 的时候，表示 FIFO 可以写入数据，此时往 QUADSPI_DR 写入需要发送的数据，就可以实现写入数据到 FLASH。

最后，在所有数据写入完成以后，终止传输（ABORT），清除传输完成标志位（TCF）。

在 HAL 库中，QSPI 发送数据是通过函数 HAL_QSPI_Transmit 来实现的，该函数声明为:

```
HAL_StatusTypeDef HAL_QSPI_Transmit (QSPI_HandleTypeDef *hqspi,
                                     uint8_t *pData, uint32_t Timeout);
```

同理，在调用该函数发送数据之前，我们会先调用 HAL_QSPI_Command 函数来指定要写入数据的存储地址信息。

FLASH 芯片初始化步骤

1）使能 QPI 模式。

因为我们是通过 QSPI 访问 W25Q256 的，所以先设置 W25Q256 工作在 QPI 模式下。通过 FLASH_EnterQPIMode 指令控制。注意：在该指令发送之前，必须先使能 W25Q256 的 QE 位。

2）设置 4 字节地址模式。

W25Q256 上电后，一般默认是 3 字节地址模式，我们需要通 FLASH_Enable4ByteAddr 指令，设置其为四字节地址模式，否则只能访问 16MB 的地址空间。

3）设置读参数。

这一步，我们通过 FLASH_SetReadParam 指令，将 P[5:4]设置为 11，以支持最高速度访问 W25Q256（8 个 dummy，104M 时钟频率）。

40.3.2 程序流程图

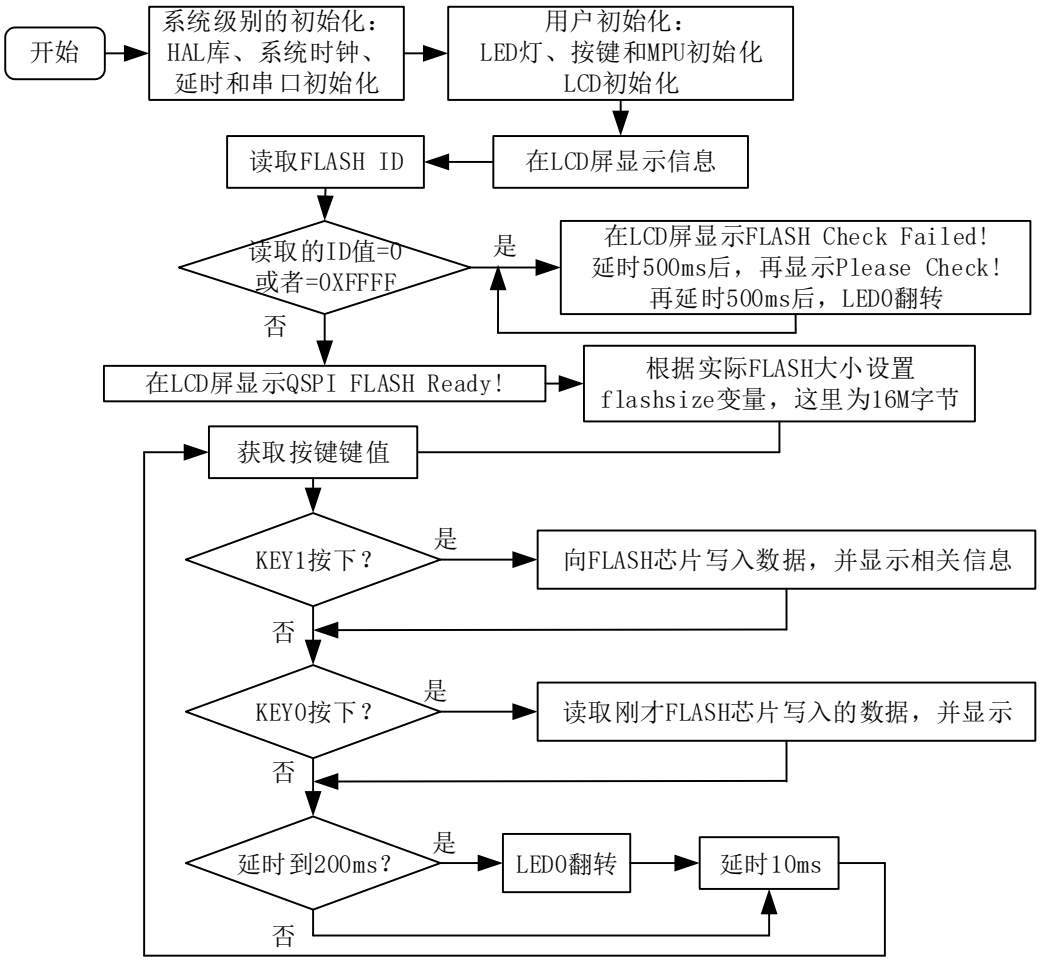


图 40.3.2.1 QSPI 实验程序流程图

40.3.3 程序解析

1. QSPI 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。QSPI 驱动源码包括两个文件：qspi.c 和 qspi.h。

qspi.h 头文件对 QSPI 相关引脚做了宏定义，该宏定义如下：

```
/* QSPI 相关 引脚 定义 */
#define QSPI_BK1_CLK_GPIO_PORT          GPIOB
#define QSPI_BK1_CLK_GPIO_PIN           GPIO_PIN_2
#define QSPI_BK1_CLK_GPIO_AF            GPIO_AF9_QUADSPI
#define QSPI_BK1_CLK_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOB_CLK_ENABLE; }while(0)    /* PB 口时钟使能 */

#define QSPI_BK1_NCS_GPIO_PORT          GPIOB
#define QSPI_BK1_NCS_GPIO_PIN           GPIO_PIN_6
#define QSPI_BK1_NCS_GPIO_AF            GPIO_AF10_QUADSPI
#define QSPI_BK1_NCS_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOB_CLK_ENABLE; }while(0)    /* PB 口时钟使能 */

#define QSPI_BK1_IO0_GPIO_PORT          GPIOF
#define QSPI_BK1_IO0_GPIO_PIN           GPIO_PIN_6
#define QSPI_BK1_IO0_GPIO_AF            GPIO_AF9_QUADSPI
#define QSPI_BK1_IO0_GPIO_CLK_ENABLE()
```

```
                do{ __HAL_RCC_GPIOF_CLK_ENABLE; }while(0)   /* PF 口时钟使能 */

#define QSPI_BK1_IO1_GPIO_PORT          GPIOF
#define QSPI_BK1_IO1_GPIO_PIN           GPIO_PIN_7
#define QSPI_BK1_IO1_GPIO_AF            GPIO_AF9_QUADSPI
#define QSPI_BK1_IO1_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOF_CLK_ENABLE; }while(0)   /* PF 口时钟使能 */

#define QSPI_BK1_IO2_GPIO_PORT          GPIOF
#define QSPI_BK1_IO2_GPIO_PIN           GPIO_PIN_8
#define QSPI_BK1_IO2_GPIO_AF            GPIO_AF9_QUADSPI
#define QSPI_BK1_IO2_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOF_CLK_ENABLE; }while(0)   /* PF 口时钟使能 */

#define QSPI_BK1_IO3_GPIO_PORT          GPIOF
#define QSPI_BK1_IO3_GPIO_PIN           GPIO_PIN_9
#define QSPI_BK1_IO3_GPIO_AF            GPIO_AF9_QUADSPI
#define QSPI_BK1_IO3_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOF_CLK_ENABLE; }while(0)   /* PF 口时钟使能 */
```

注意这 6 个 GPIO 都是用到复用功能，对应引脚的复用功能情况请看《STM32H743IIT6.pdf》数据手册 64 页之后的端口复用功能表格。

下面我们开始介绍 qspi.c 的程序，首先是 QSPI 接口初始化函数，其定义如下：

```
/**
 * @brief       初始化 QSPI 接口
 * @param       无
 * @retval      0, 成功; 1, 失败.
 */
uint8_t qspi_init(void)
{
    g_qspi_handle.Instance = QUADSPI;       /* QSPI */
    /* QPSI 分频比，W25Q256 最大频率为 133M，所以此处应该为 2，QSPI 频率就为
       200/(1+1)=100MHZ */
    g_qspi_handle.Init.ClockPrescaler = 1;
    g_qspi_handle.Init.FifoThreshold = 4;  /* FIFO 阈值为 4 个字节 */
    /* 采样移位半个周期(DDR 模式下,必须设置为 0) */
    g_qspi_handle.Init.SampleShifting = QSPI_SAMPLE_SHIFTING_HALFCYCLE;
    /* SPI FLASH 大小，W25Q256 大小为 32M 字节,2^25，所以取权值 25-1=24 */
    g_qspi_handle.Init.FlashSize = POSITION_VAL(0X2000000)-1;
    /* 片选高电平时间为 3 个时钟(10*3=30ns),即手册里面的 tSHSL 参数 */
    g_qspi_handle.Init.ChipSelectHighTime = QSPI_CS_HIGH_TIME_3_CYCLE;
    g_qspi_handle.Init.ClockMode = QSPI_CLOCK_MODE_3;       /* 模式 3 */
    g_qspi_handle.Init.FlashID = QSPI_FLASH_ID_1;           /* 第一片 flash */
    g_qspi_handle.Init.DualFlash = QSPI_DUALFLASH_DISABLE;  /* 禁止双闪存模式 */
    if (HAL_QSPI_Init(&g_qspi_handle) == HAL_OK)
    {
        return 0;       /* QSPI 初始化成功 */
    }
    else
    {
        return 1;
    }
}
```

这里我们需要注意的是，QSPI 的时钟源在 sys_stm32_clock_init 函数中已经选择了 HCLK3（我们设置为 200MHZ），这里就不需要再选择时钟源了，只需要选择预分频系数就可以确定 QSPI 的时钟频率。时钟模式选择模式 3，在未进行任何操作时 CLK 升至高电平。我们用单闪存模式，FlashID 要选择 QSPI_FLASH_ID_1。

我们用 HAL_QSPI_MspInit 函数来编写 QSPI 时钟和 IO 配置等代码，其定义如下：

```
/**
 * @brief       QSPI 底层驱动,引脚配置，时钟使能
 * @param       hqspi:QSPI 句柄
 * @note        此函数会被 HAL_QSPI_Init()调用
```

```
 * @retval      0, 成功; 1, 失败.
 */
void HAL_QSPI_MspInit(QSPI_HandleTypeDef *hqspi)
{
    GPIO_InitTypeDef gpio_init_struct;

    __HAL_RCC_QSPI_CLK_ENABLE();      /* 使能 QSPI 时钟 */
    QSPI_BK1_CLK_GPIO_CLK_ENABLE();   /* CLK 时钟使能 */
    QSPI_BK1_NCS_GPIO_CLK_ENABLE();   /* NCS 时钟使能 */
    QSPI_BK1_IO0_GPIO_CLK_ENABLE();   /* IO0 时钟使能 */
    QSPI_BK1_IO1_GPIO_CLK_ENABLE();   /* IO1 时钟使能 */
    QSPI_BK1_IO2_GPIO_CLK_ENABLE();   /* IO2 时钟使能 */
    QSPI_BK1_IO3_GPIO_CLK_ENABLE();   /* IO3 时钟使能 */

    gpio_init_struct.Pin = QSPI_BK1_NCS_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;                /* 复用 */
    gpio_init_struct.Pull = GPIO_PULLUP;                    /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;     /* 高速 */
    gpio_init_struct.Alternate = GPIO_AF10_QUADSPI;         /* 复用为 QSPI */
    /* 初始化 QSPI_BK1_NCS 引脚 */
    HAL_GPIO_Init(QSPI_BK1_NCS_GPIO_PORT, &gpio_init_struct);

    gpio_init_struct.Pin = QSPI_BK1_CLK_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;                /* 复用 */
    gpio_init_struct.Pull = GPIO_PULLUP;                    /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;     /* 高速 */
    gpio_init_struct.Alternate = GPIO_AF9_QUADSPI;          /* 复用为 QSPI */
    /* 初始化 QSPI_BK1_CLK 引脚 */
    HAL_GPIO_Init(QSPI_BK1_CLK_GPIO_PORT, &gpio_init_struct);

    gpio_init_struct.Pin = QSPI_BK1_IO0_GPIO_PIN;
    /* 初始化 QSPI_BK1_IO0 引脚 */
    HAL_GPIO_Init(QSPI_BK1_IO0_GPIO_PORT, &gpio_init_struct);

    gpio_init_struct.Pin = QSPI_BK1_IO1_GPIO_PIN;
    /* 初始化 QSPI_BK1_IO1 引脚 */
    HAL_GPIO_Init(QSPI_BK1_IO1_GPIO_PORT, &gpio_init_struct);

    gpio_init_struct.Pin = QSPI_BK1_IO2_GPIO_PIN;
    /* 初始化 QSPI_BK1_IO2 引脚 */
    HAL_GPIO_Init(QSPI_BK1_IO2_GPIO_PORT, &gpio_init_struct);

    gpio_init_struct.Pin = QSPI_BK1_IO3_GPIO_PIN;
    /* 初始化 QSPI_BK1_IO3 引脚 */
    HAL_GPIO_Init(QSPI_BK1_IO3_GPIO_PORT, &gpio_init_struct);
}
```

这里初始化的 6 个引脚全部都要配置为复用功能模式，以及使能 QSPI 和相应 IO 时钟。

接下来介绍 QSPI 发送命令函数，其定义如下：

```
/**
 * @brief       QSPI 发送命令
 * @param       cmd : 要发送的指令
 * @param       addr: 发送到的目的地址
 * @param       mode: 模式,详细位定义如下:
 *  @arg mode[1:0]: 指令模式;00,无指令;01,单线传输指令;10,双线传输指令;11,四线传输指令.
 *  @arg mode[3:2]: 地址模式;00,无地址;01,单线传输地址;10,双线传输地址;11,四线传输地址.
 *  @arg mode[5:4]: 地址长度;00,8 位地址;01,16 位地址; 10,24 位地址;   11,32 位地址.
 *  @arg mode[7:6]: 数据模式;00,无数据; 01,单线传输数据;10,双线传输数据;11,四线传输数据.
 * @param       dmcycle: 空指令周期数
 * @retval      无
 */
void qspi_send_cmd(uint8_t cmd, uint32_t addr, uint8_t mode, uint8_t dmcycle)
```

```
{
    QSPI_CommandTypeDef qspi_command_handle;

    qspi_command_handle.Instruction = cmd;          /* 指令 */
    qspi_command_handle.Address = addr;             /* 地址 */
    qspi_command_handle.DummyCycles = dmcycle;      /* 设置空指令周期数 */

    if(((mode >> 0) & 0x03) == 0)
    qspi_command_handle.InstructionMode = QSPI_INSTRUCTION_NONE;   /* 指令模式 */
    else if(((mode >> 0) & 0x03) == 1)
    qspi_command_handle.InstructionMode = QSPI_INSTRUCTION_1_LINE; /* 指令模式 */
    else if(((mode >> 0) & 0x03) == 2)
    qspi_command_handle.InstructionMode = QSPI_INSTRUCTION_2_LINES;/* 指令模式 */
    else if(((mode >> 0) & 0x03) == 3)
    qspi_command_handle.InstructionMode = QSPI_INSTRUCTION_4_LINES;/* 指令模式 */

    if(((mode >> 2) & 0x03) == 0)
    qspi_command_handle.AddressMode = QSPI_ADDRESS_NONE;     /* 地址模式 */
    else if(((mode >> 2) & 0x03) == 1)
    qspi_command_handle.AddressMode = QSPI_ADDRESS_1_LINE;   /* 地址模式 */
    else if(((mode >> 2) & 0x03) == 2)
    qspi_command_handle.AddressMode = QSPI_ADDRESS_2_LINES;  /* 地址模式 */
    else if(((mode >> 2) & 0x03) == 3)
    qspi_command_handle.AddressMode = QSPI_ADDRESS_4_LINES;  /* 地址模式 */

    if(((mode >> 4)&0x03) == 0)
    qspi_command_handle.AddressSize = QSPI_ADDRESS_8_BITS;   /* 地址长度 */
    else if(((mode >> 4) & 0x03) == 1)
    qspi_command_handle.AddressSize = QSPI_ADDRESS_16_BITS;  /* 地址长度 */
    else if(((mode >> 4) & 0x03) == 2)
    qspi_command_handle.AddressSize = QSPI_ADDRESS_24_BITS;  /* 地址长度 */
    else if(((mode >> 4) & 0x03) == 3)
    qspi_command_handle.AddressSize = QSPI_ADDRESS_32_BITS;  /* 地址长度 */

    if(((mode >> 6) & 0x03) == 0)
    qspi_command_handle.DataMode=QSPI_DATA_NONE;             /* 数据模式 */
    else if(((mode >> 6) & 0x03) == 1)
    qspi_command_handle.DataMode = QSPI_DATA_1_LINE;         /* 数据模式 */
    else if(((mode >> 6) & 0x03) == 2)
    qspi_command_handle.DataMode = QSPI_DATA_2_LINES;        /* 数据模式 */
    else if(((mode >> 6) & 0x03) == 3)
    qspi_command_handle.DataMode = QSPI_DATA_4_LINES;        /* 数据模式 */

   qspi_command_handle.SIOOMode = QSPI_SIOO_INST_EVERY_CMD; /* 每次都发送指令 */
   qspi_command_handle.AlternateByteMode=QSPI_ALTERNATE_BYTES_NONE;/*无交替字节*/
   qspi_command_handle.DdrMode = QSPI_DDR_MODE_DISABLE;     /* 关闭 DDR 模式 */
   qspi_command_handle.DdrHoldHalfCycle = QSPI_DDR_HHC_ANALOG_DELAY;

    HAL_QSPI_Command(&g_qspi_handle, &qspi_command_handle, 5000);
}
```

该函数主要就是配置 QSPI_CommandTypeDef 结构体的参数，并调用 HAL_QSPI_Command 函数配置发送命令，是一个重要的基础函数。

接下来介绍的是 QSPI 接收函数，其定义如下：

```
/**
 * @brief           QSPI 接收指定长度的数据
 * @param           buf     : 接收数据缓冲区首地址
 * @param           datalen : 要传输的数据长度
 * @retval          0, 成功; 其他, 错误代码.
 */
uint8_t qspi_receive(uint8_t *buf, uint32_t datalen)
{
```

```
    g_qspi_handle.Instance->DLR = datalen - 1;    /* 配置数据长度 */
    if (HAL_QSPI_Receive(&g_qspi_handle, buf, 5000) == HAL_OK)
    {
        return 0;
    }
    else
    {
        return 1;
    }
}
```

该函数首先把要接收数据的长度赋值到 QUADSPI 数据长度寄存器(QUADSPI_DLR)中，然后通过调用 HAL_QSPI_Receive 函数接收数据。

接下来介绍的是 QSPI 发送函数，其定义如下：

```
/**
 * @brief       QSPI 发送指定长度的数据
 * @param       buf     : 发送数据缓冲区首地址
 * @param       datalen : 要传输的数据长度
 * @retval      0, 成功; 其他, 错误代码.
 */
uint8_t qspi_transmit(uint8_t *buf, uint32_t datalen)
{
    g_qspi_handle.Instance->DLR = datalen - 1; /* 配置数据长度 */
    if (HAL_QSPI_Transmit(&g_qspi_handle, buf, 5000) == HAL_OK)
    {
        return 0;
    }
    else
    {
        return 1;
    }
}
```

该函数首先把要发送数据的长度赋值到 QUADSPI 数据长度寄存器(QUADSPI_DLR)中，然后通过调用 HAL_QSPI_Transmit 函数发送数据。

最后要介绍的一个函数是等待状态标志函数，其定义如下：

```
/**
 * @brief           等待状态标志
 * @param           flag : 需要等待的标志位
 * @param           sta  : 需要等待的状态
 * @param           wtime: 等待时间
 * @retval          0, 等待成功; 1, 等待失败.
 */
uint8_t qspi_wait_flag(uint32_t flag, uint8_t sta, uint32_t wtime)
{
    uint8_t flagsta = 0;
    while (wtime)
    {
        flagsta = (QUADSPI->SR & flag) ? 1 : 0; /* 获取状态标志 */

        if (flagsta == sta)break;

        wtime--;
    }
    if (wtime)return 0;
    else return 1;
}
```

该函数可以设置一段时钟等待 QUADSPI 状态寄存器(QUADSPI_SR)的任意位为 0，或者为 1。然后通过返回值，判断等待是否成功，0 表示等待成功；1 表示等待失败。

2. norflash 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。NORFLASH 驱

动源码包括两个文件：norflash.c 和 norflash.h。

下面介绍 norflash.c 文件几个重要的函数，首先是 NOR FLASH 初始化函数，其定义如下：

```
/**
 * @brief       初始化 SPI NOR FLASH
 * @param       无
 * @retval      无
 */
void norflash_init(void)
{
    uint8_t temp;
    qspi_init();                            /* 初始化 QSPI */
    norflash_qe_enable();                   /* 使能 QSPI 模式 */
    g_norflash_type = norflash_read_id();/* 读取 FLASH ID */

    if (g_norflash_type == W25Q256 || g_norflash_type == BY25Q256)
    {/* SPI FLASH 为 25Q256, 必须使能 4 字节地址模式 */
        temp = norflash_read_sr(3); /* 读取状态寄存器 3，判断地址模式 */
        if ((temp & 0X01) == 0)     /* 如果不是 4 字节地址模式,则进入 4 字节地址模式 */
        {
            norflash_write_enable();    /* 写使能 */
            /* QPI,使能 4 字节地址指令,地址为 0,无数据_8 位地址_无地址_4 线传输指令,
             无空周期,0 个字节数据 */
          qspi_send_cmd(FLASH_Enable4ByteAddr,0,(0<<6)|(0<<4)|(0<<2)|(3<<0),0);
        }

        norflash_write_enable();    /* 写使能 */
        /* QPI,设置读参数指令,地址为 0,4 线传数据_8 位地址_无地址_4 线传输指令,
          无空周期,1 个字节数据 */
        qspi_send_cmd(FLASH_SetReadParam,0,(3<<6)|(0<<4)|(0<<2)|(3<<0),0);
        temp = 3 << 4;              /* 设置 P4&P5=11,8 个 dummy clocks,104M */
        qspi_transmit(&temp, 1);    /* 发送 1 个字节 */
    }
}
```

该函数用于初始化 NOR FLASH，首先调用 qspi_init 函数，初始化 STM32H743 的 QSPI 接口。然后退出 QPI 模式（避免芯片之前进入这个模式，导致下载失败），使能 FLASH 的 QE 位，使能 IO2/IO3。最后读取 FLASH ID，如果 SPI FLASH 为 W25Q256，还必须使能 4 字节地址模式。调用本函数在初始化完成以后，我们便可以通过 QSPI 接口读写 NOR FLASH 的数据了。

接下来介绍读取 SPI FLASH 函数，其定义如下：

```
/**
 * @brief       读取 SPI FLASH,仅支持 QSPI 模式
 * @note        在指定地址开始读取指定长度的数据
 * @param       pbuf    : 数据存储区
 * @param       addr    : 开始读取的地址(最大 32bit)
 * @param       datalen : 要读取的字节数(最大 65535)
 * @retval      无
 */
void norflash_read(uint8_t *pbuf, uint32_t addr, uint16_t datalen)
{
    /* QPI,快速读数据,地址为 ReadAddr,4 线传输数据_32 位地址_
     4 线传输地址_4 线传输指令,8 空周期,NumByteToRead 个数据 */
    qspi_send_cmd(FLASH_FastReadData, addr,(3<<6)|(3<<4)|(3<<2)|(3 << 0), 8);
    qspi_receive(pbuf, datalen);
}
```

该函数用于从 NOR FLASH 的指定地址读出指定长度的数据，由于 NOR FLASH 支持以任意地址（但是不能超过 NOR FLASH 的地址范围）开始读取数据，所以，这个代码相对来说就比较简单了，通过 qspi_send_cmd 函数，发送 FLASH_FastReadQuad 指令，并发送读数据首地址（addr），然后通过 qspi_receive 函数循环读取数据，存放在 pbuf 里面。

接下来，我们介绍写入 NOR FLASH 函数，其定义如下：

```
/**
```

```
 * @brief       写 SPI FLASH
 * @note        在指定地址开始写入指定长度的数据 , 该函数带擦除操作!
 *              SPI FLASH 一般是: 256 个字节为一个 Page, 4Kbytes 为一个 Sector,
                16 个扇区为 1 个 Block
 *              擦除的最小单位为 Sector.
 * @param       pbuf    : 数据存储区
 * @param       addr    : 开始写入的地址(最大 32bit)
 * @param       datalen : 要写入的字节数(最大 65535)
 * @retval      无
 */
uint8_t g_norflash_buf[4096];   /* 扇区缓存 */

void norflash_write(uint8_t *pbuf, uint32_t addr, uint16_t datalen)
{
    uint32_t secpos;
    uint16_t secoff;
    uint16_t secremain;
    uint16_t i;
    uint8_t *norflash_buf;

    norflash_buf = g_norflash_buf;
    secpos = addr / 4096;       /* 扇区地址 */
    secoff = addr % 4096;       /* 在扇区内的偏移 */
    secremain = 4096 - secoff; /* 扇区剩余空间大小 */

    //printf("ad:%X,nb:%X\r\n", addr, datalen); /* 测试用 */
    if (datalen <= secremain)
    {
        secremain = datalen;    /* 不大于 4096 个字节 */
    }

    while (1)
    {
        norflash_read(norflash_buf, secpos * 4096, 4096);/* 读出整个扇区的内容 */
        for (i = 0; i < secremain; i++)   /* 校验数据 */
        {
            if (norflash_buf[secoff + i] != 0XFF)
            {
                break;          /* 需要擦除, 直接退出 for 循环 */
            }
        }

        if (i < secremain)    /* 需要擦除 */
        {
            norflash_erase_sector(secpos);      /* 擦除这个扇区 */
            for (i = 0; i < secremain; i++)     /* 复制 */
            {
                norflash_buf[i + secoff] = pbuf[i];
            }
            /* 写入整个扇区 */
            norflash_write_nocheck(norflash_buf, secpos * 4096, 4096);
        }
        else            /* 写已经擦除了的,直接写入扇区剩余区间. */
        {
            norflash_write_nocheck(pbuf, addr, secremain);  /* 直接写扇区 */
        }

        if (datalen == secremain)
        {
            break;      /* 写入结束了 */
        }
        else            /* 写入未结束 */
```

```
        {
            secpos++;                   /* 扇区地址增 1 */
            secoff = 0;                 /* 偏移位置为 0 */

            pbuf += secremain;          /* 指针偏移 */
            addr += secremain;          /* 写地址偏移 */
            datalen -= secremain;       /* 字节数递减 */

            if (datalen > 4096)
            {
                secremain = 4096;       /* 下一个扇区还是写不完 */
            }
            else
            {
                secremain = datalen;    /* 下一个扇区可以写完了 */
            }
        }
    }
}
```

该函数可以在 NOR FLASH 的任意地址开始写入任意长度（必须不超过 NOR FLASH 的容量）的数据。我们这里简单介绍一下思路：先获得首地址（addr）所在的扇区，并计算在扇区内的偏移，然后判断要写入的数据长度是否超过本扇区所剩下的长度，如果不超过，再先看看是否要擦除，如果不要，则直接写入数据即可，如果要则读出整个扇区，在偏移处开始写入指定长度的数据，然后擦除这个扇区，再一次性写入。当所需要写入的数据长度超过一个扇区的长度的时候，我们先按照前面的步骤把扇区剩余部分写完，再在新扇区内执行同样的操作，如此循环，直到写入结束。这里我们还定义了一个 g_norflash_buf 的全局数组，用于擦除时缓存扇区内的数据。

norflash.c 文件我们就介绍这三个函数，其他请大家自行查阅。

3. main.c 代码

在 main.c 里面编写如下代码：

```
/* 要写入到 FLASH 的字符串数组 */
const uint8_t g_text_buf[] = {"STM32H7 QSPI TEST"};
#define TEXT_SIZE       sizeof(g_text_buf)  /* TEXT 字符串长度 */

int main(void)
{
    uint8_t key;
    uint16_t i = 0;
    uint8_t datatemp[TEXT_SIZE+2];
    uint8_t rectemp[TEXT_SIZE+2];
    uint32_t flashsize;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    norflash_init();                        /* 初始化 norflash */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "QSPI TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
```

```
    /* 显示提示信息 */
    lcd_show_string(30, 110, 200, 16, 16, "KEY1:Write  KEY0:Read", RED);

    while (norflash_read_id()!=g_norflash_type) /* 检测不到 FLASH 芯片 */
    {
        lcd_show_string(30, 130, 200, 16, 16, "FLASH Check Failed!", RED);
        delay_ms(500);
        lcd_show_string(30, 130, 200, 16, 16, "Please Check!      ", RED);
        delay_ms(500);
        LED0_TOGGLE();      /* LED0 闪烁 */
    }

    lcd_show_string(30, 130, 200, 16, 16, "QSPI FLASH Ready!", BLUE);
    flashsize = 32 * 1024 * 1024;   /* FLASH 大小为 32M 字节 */

    while (1)
    {
        key = key_scan(0);
        if (key == KEY1_PRES)   /* KEY1 按下,写入 */
        {
            lcd_fill(0, 150, 239, 319, WHITE);  /* 清除半屏 */
            lcd_show_string(30,150,200,16,16,"Start Write FLASH....", BLUE);
            sprintf((char *)datatemp, "%s%d", (char *)g_text_buf, i);
            /* 从倒数第 200 个地址处开始,写入 SIZE 长度的数据 */
            norflash_write((uint8_t *) datatemp, flashsize - 200, TEXT_SIZE+2);
            /* 提示传送完成 */
            lcd_show_string(30,150,200,16,16,"FLASH Write Finished!", BLUE);
            printf("datatemp:%s  \r\n", datatemp);
        }

        if (key == KEY0_PRES)   /* KEY0 按下,读取字符串并显示 */
        {
            lcd_show_string(30,150,200,16,16,"Start Read FLASH.... ", BLUE);
            /* 从倒数第 200 个地址处开始,读出 SIZE 个字节 */
            norflash_read((uint8_t*)rectemp, flashsize - 200, TEXT_SIZE+2);
            /* 提示传送完成 */
            lcd_show_string(30,150,200,16,16,"The Data Readed Is:   ", BLUE);
            /* 显示读到的字符串 */
            lcd_show_string(30,170,200,16,16,(char *)rectemp, BLUE);
            printf("rectemp:%s  \r\n", rectemp);
        }
        i++;
        if (i == 20)
        {
            LED0_TOGGLE();      /* LED0 闪烁 */
            i = 0;
        }
        delay_ms(10);
    }
}
```

这部分代码和 IIC 实验那部分代码大同小异，我们就不多说了，实现的功能就和 IIC 差不多，不过此次写入和读出的是 SPI FLASH，而不是 EEPROM。最后，我们将 norflash_read_id、norflash_erase_chip 和 norflash_erase_sector 等函数加入 USMART 控制，这样，我们就可以通过串口调试助手，操作 W25Q256，方便大家测试。

40.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 40.4.1 所示：

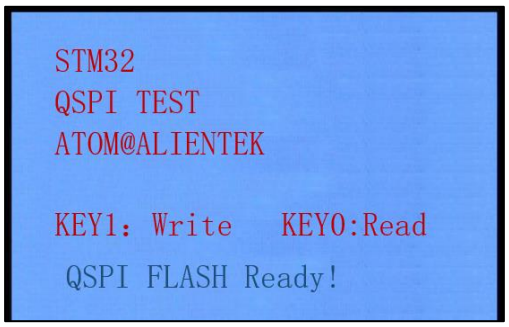


图 40.4.1 QSPI 实验程序运行效果图

通过先按 KEY1 按键写入数据，然后按 KEY0 读取数据，得到如图 40.4.2 所示：

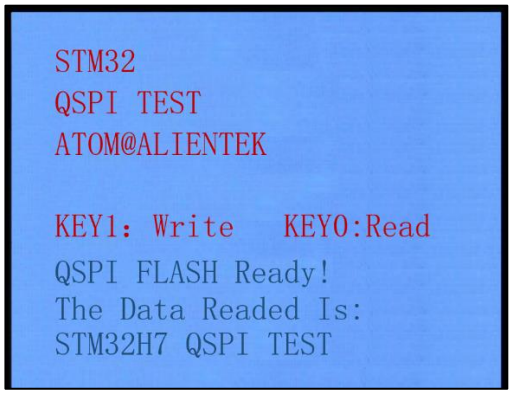


图 40.4.2 操作后的显示效果图

程序在开机的时候会检测 NOR FLASH 是否存在，如果不存在则会在 LCD 模块上显示错误信息，同时 LED0 慢闪。

第四十一章 485 实验

本章我们将向大家介绍如何使用 STM32H743 的串口实现 485 通信（半双工）。在本章中，我们将使用 STM32H743 的串口 2 来实现两块开发板之间的 485 通信，并将结果显示在 TFTLCD 模块上。

本章分为如下几个部分：

41.1 485 简介
41.2 硬件设计
41.3 程序设计
41.4 下载验证

41.1 485 简介

485（一般称作 RS485/EIA-485）隶属于 OSI 模型物理层，是串行通讯的一种。电气特性规定为 2 线，半双工，多点通信的类型。它的电气特性和 RS-232 大不一样。用缆线两端的电压差值来表示传递信号。RS485 仅仅规定了接受端和发送端的电气特性。它没有规定或推荐任何数据协议。

RS485 的特点包括：

1、接口电平低，不易损坏芯片。RS485 的电气特性：逻辑“1”以两线间的电压差为+(2~6)V 表示；逻辑“0”以两线间的电压差为-(2~6)V 表示。接口信号电平比 RS232 降低了，不易损坏接口电路的芯片，且该电平与 TTL 电平兼容，可方便与 TTL 电路连接。

2、传输速率高。10 米时，RS485 的数据最高传输速率可达 35Mbps，在 1200m 时，传输速度可达 100Kbps。

3、抗干扰能力强。RS485 接口是采用平衡驱动器和差分接收器的组合，抗共模干扰能力增强，即抗噪声干扰性好。

4、传输距离远，支持节点多。RS485 总线最长可以传输 1200m 左右，更远的距离则需要中继传输设备支持但这时（速率≤100Kbps）才能稳定传输，一般最大支持 32 个节点，如果使用特制的 485 芯片，可以达到 128 个或者 256 个节点，最大的可以支持到 400 个节点。

RS485 推荐使用在点对点网络中，比如：线型，总线型网络等，而不能是星型，环型网络。理想情况下 RS485 需要 2 个终端匹配电阻，其阻值要求等于传输电缆的特性阻抗（一般为 120Ω）。没有特性阻抗的话，当所有的设备都静止或者没有能量的时候就会产生噪声，而且线移需要双端的电压差。没有终接电阻的话，会使得较快速的发送端产生多个数据信号的边缘，导致数据传输出错。485 推荐的一主多从连接方式如图 41.1.1 所示：

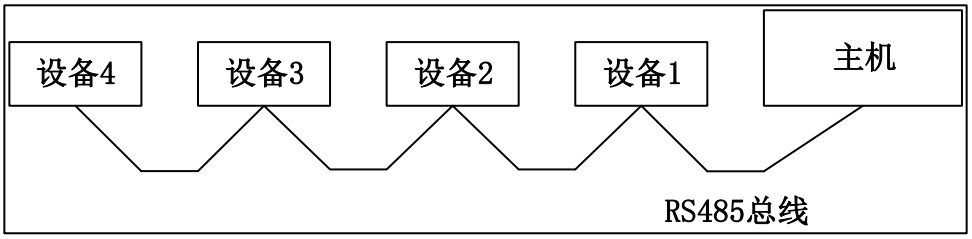


图 41.1.1 RS485 连接

在上面的连接中，如果需要添加匹配电阻，我们一般在总线的起止端加入，也就是主机和设备 4 上面各加一个 120Ω 的匹配电阻。

由于 RS485 具有传输距离远、传输速度快、支持节点多和抗干扰能力更强等特点，所以 RS485 有很广泛的应用。实际多设备时收发器有范围为-7V 到+12V 的共模电压，为了稳定传输，也有使用 3 线的布线方式，即在原有的 A、B 两线上多增加一条地线。（4 线制只能实现点对点的全双工通讯方式，这种也叫 RS422，由于布线的难度和通讯局限，相对使用得比较少）。

TP8485E/SP3485 可作为 RS485 的收发器，该芯片支持 3.3V~5.5V 供电，最大传输速度可达 250Kbps，支持多达 256 个节点(单位负载为 1/8 的条件下)，并且支持输出短路保护。该芯片

的框图如图 41.1.2 所示：

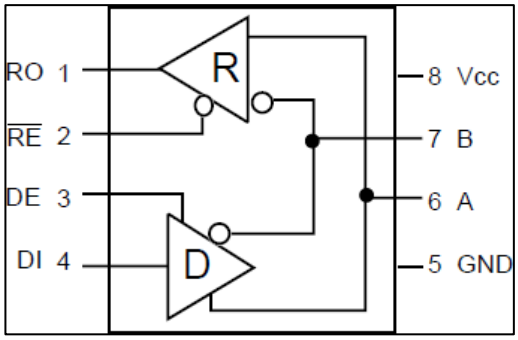


图 41.1.2 TP8485E/SP3485 框图

图中 A、B 总线接口，用于连接 485 总线。RO 是接收输出端，DI 是发送数据收入端，RE 是接收使能信号（低电平有效），DE 是发送使能信号（高电平有效）。

41.2 硬件设计

1. 例程功能

经过前面的学习我们知道实际的 RS485 仍是串行通讯的一种电平传输方式，那么我们实际通讯时可以使用串口进行实际数据的收发处理，使用 485 转换芯片将串口信号转换为 485 的电平信号进行传输，本章，我们只需要配置好串口 2，就可以实现正常的 485 通信了，串口 2 的配置和串口 1 基本类似，只是串口 2 的时钟来自 APB1，最大频率为 100Mhz。

本章将实现这样的功能：通过连接两个阿波罗 STM32H743 的 RS485 接口，然后由 KEY0 控制发送，当按下一个开发板的 KEY0 的时候，就发送 5 个数据给另外一个开发板，并在两个开发板上分别显示发送的值和接收到的值。

2. 硬件资源

1）LED 灯
LED0 – PB1
2）USART2，用于实际的 485 信号串行通讯。
3）独立按键 KEY0 – PH3
4）正点原子 2.8/3.5/4.3/7 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
5）RS485 收发芯片 TP8485/SP3485
6）开发板两块（485 半双式模式无法自收发，我们需要用两个开发板或者 USB 转 485 调试器+串口助手来帮助我们完成测试，大家根据自己的实际条件选择）

3. 原理图

根据我们需要实现的程序功能，我们设计电路原理如下：

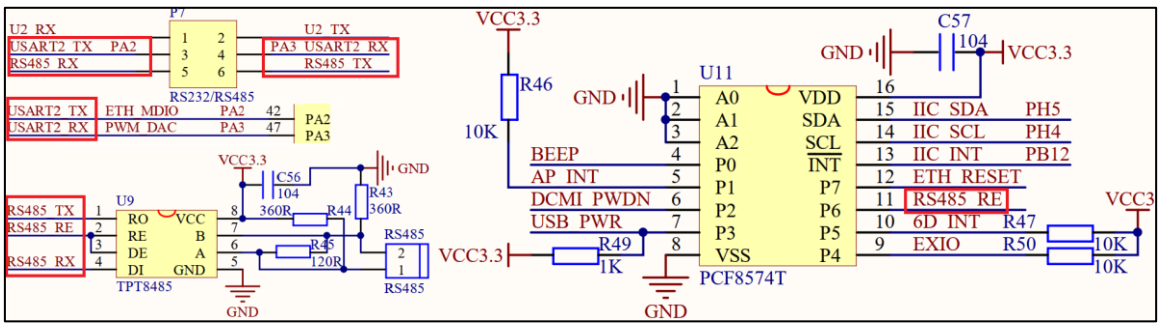


图 41.2.2 RS485 连接原理设计

从上图可以看出：开发板的串口 2 通过 P7 端口设置，连接到 TP8485。注意：RS485_RE 信号，是连接在 PCF8574T 的 P6 脚上的，并没有直接连接到 MCU，需要通过 IIC 总线控制

PCF8574T，从而实现对 RS485_RE 的控制。RS485_RE 控制 TP8485 的收发，当 RS485_RE=0 的时候，为接收模式；当 RS485_RE=1 的时候，为发送模式。

另外，PA2，PA3 和 ETH_MDIO 和 PWM_DAC 有共用 IO，所以在使用的时候需要分时复用，不能同时使用。

图中的 R43 和 R44 是两个偏置电阻，用来保证总线空闲时，A、B 之间的电压差都会大于 200mV（逻辑 1）。从而避免总线空闲时因 A、B 压差不稳定，可能出现的乱码。

最后，我们用 2 根导线将两个开发板 RS485 端子的 A 和 A，B 和 B 连接起来。这里注意不要接反了，接反了会导致通讯异常!!

41.3 程序设计

41.3.1 RS485 的 HAL 库驱动

由于 485 实际上是串口通讯，我们参照串口实验一节使用类似的 HAL 库驱动即可，在这里分析一下 RS485 配置步骤。

RS485 配置步骤

1）使能串口和 GPIO 口时钟

本实验用到 USART2 串口，使用 PA2 和 PA3 作为串口的 TX 和 RX 脚，因此需要先使能 USART2 和 GPIOA 时钟。参考代码如下：

```
__HAL_RCC_USART2_CLK_ENABLE();            /* 使能 USART2 时钟 */
__HAL_RCC_GPIOA_CLK_ENABLE();             /* 使能 GPIOA 时钟 */
```

2) 串口参数初始化（波特率、字长、奇偶校验等）

HAL 库通过调用串口初始化函数 HAL_UART_Init 完成对串口参数初始化，详见例程源码。

该函数通常会调用：HAL_UART_MspInit 函数来完成对串口底层的初始化，包括：串口及 GPIO 时钟使能、GPIO 模式设置、中断设置等。但是本实验避免与 USART1 冲突，所以把串口底层初始化没有放在 HAL_UART_MspInit 函数里。

3）GPIO 模式设置（速度，上下拉，复用功能等）

GPIO 模式设置通过调用 HAL_GPIO_Init 函数实现，详见本例程源码。

4）开启串口相关中断，配置串口中断优先级

本实验我们使用串口中断来接收数据。我们使用 HAL_UART_Receive_IT 函数开启串口中断接收，并设置接收 buffer 及其长度。通过 HAL_NVIC_EnableIRQ 函数使能串口中断，通过 HAL_NVIC_SetPriority 函数设置中断优先级。

5）编写中断服务函数

串口 2 中断服务函数为：USART2_IRQHandler，当发生中断的时候，程序就会执行中断服务函数，在这里就可以对接收到的数据进行处理，详见本例程源码。

6）串口数据接收和发送

最后我们可以通过读写 USART_DR 寄存器，完成串口数据的接收和发送，HAL 库也给我们提供了：HAL_UART_Receive 和 HAL_UART_Transmit 两个函数用于串口数据的接收和发送。

大家可以根据实际情况选择使用哪种方式来收发串口数据。

41.3.2 程序流程图

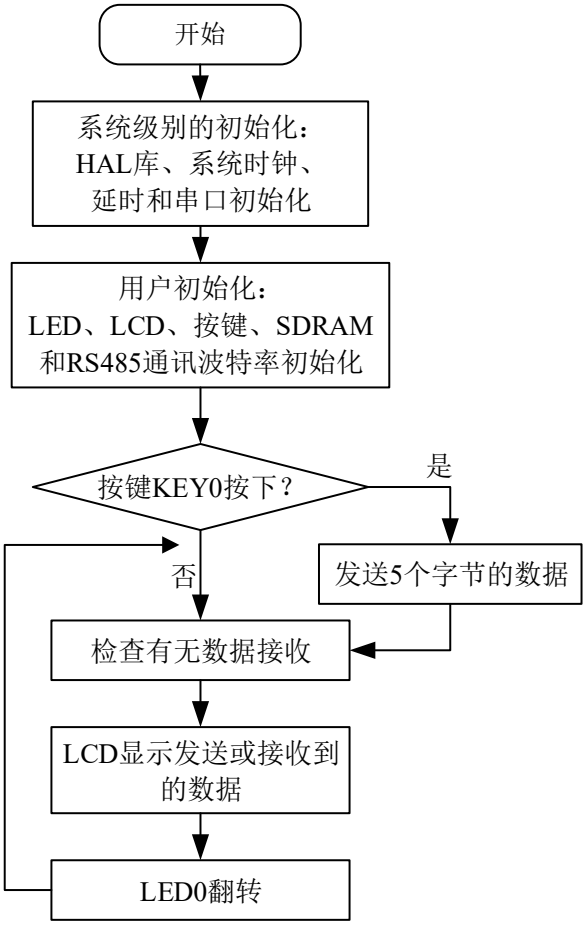


图 41.3.2.1 RS485 实验程序流程图

41.3.3 程序解析

1. RS485 驱动

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。RS485 驱动相关源码包括两个文件：rs485.c 和 rs485.h。

为方便修改，我们在 rs485.h 中使用宏定义 485 相关的控制引脚和串口编号，如果需要使用其它的引脚或者串口，修改宏和串口的定义即可，它们在 rs485.h 中定义，它们列出如下：

```
/* RS485 引脚 和 串口 定义 */
#define RS485_TX_GPIO_PORT                GPIOA
#define RS485_TX_GPIO_PIN                 GPIO_PIN_2
#define RS485_TX_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0) /* PA 口时钟使能 */

#define RS485_RX_GPIO_PORT                GPIOA
#define RS485_RX_GPIO_PIN                 GPIO_PIN_3
#define RS485_RX_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0) /* PA 口时钟使能 */

#define RS485_UX                          USART2
#define RS485_UX_IRQn                     USART2_IRQn
#define RS485_UX_IRQHandler               USART2_IRQHandler
#define RS485_UX_CLK_ENABLE()
               do{ __HAL_RCC_USART2_CLK_ENABLE(); }while(0) /* USART2 时钟使能 */
#define RS485_UX_CLK_ENABLE()      do{ __HAL_RCC_USART2_CLK_ENABLE();}while(0)
```

```
/* 如果想串口中断接收，设置 EN_USART2_RX 为 1，否则设置为 0 */
#define EN_USART2_RX                1          /* 0,不接收;1,接收. */
#define RS485_REC_LEN               64         /* 定义最大接收字节数 64 */
```

1）rs485_init 函数

rs485_init 的配置与串口类似，也需要设置波特率等参数，另外还需要配置收发模式的驱动引脚，我们的程序设计如下：

```
/**
 * @brief       RS485 初始化函数
 * @note        该函数主要是初始化串口
 * @param       bound:波特率
 * @retval      无
 */
void rs485_init(uint32_t bound)
{
    /* GPIO 端口设置 */
    GPIO_InitTypeDef gpio_init_struct;

    pcf8574_init();                 /* 初始化 PCF8574，用于控制 RE 脚 */
    RS485_TX_GPIO_CLK_ENABLE();     /* 使能 GPIOA 时钟 */
    RS485_UX_CLK_ENABLE();          /* 使能 USART2 时钟 */

    gpio_init_struct.Pin = RS485_TX_GPIO_PIN | RS485_RX_GPIO_PIN; /* PA2,3 */
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;              /* 复用推挽 */
    gpio_init_struct.Pull = GPIO_PULLUP;                  /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_HIGH;             /* 高速 */
    gpio_init_struct.Alternate = GPIO_AF7_USART2;         /* 复用为 USART2 */
    HAL_GPIO_Init(RS485_TX_GPIO_PORT,&gpio_init_struct);/* 初始化 PA2,3 */

    /* USART 初始化设置 */
    g_rs458_handle.Instance = RS485_UX;                     /* USART2 */
    g_rs458_handle.Init.BaudRate = bound;                   /* 波特率 */
    g_rs458_handle.Init.WordLength = UART_WORDLENGTH_8B;    /* 字长为 8 位数据格式 */
    g_rs458_handle.Init.StopBits = UART_STOPBITS_1;         /* 一个停止位 */
    g_rs458_handle.Init.Parity = UART_PARITY_NONE;          /* 无奇偶校验位 */
    g_rs458_handle.Init.HwFlowCtl = UART_HWCONTROL_NONE;    /* 无硬件流控 */
    g_rs458_handle.Init.Mode = UART_MODE_TX_RX;             /* 收发模式 */
    HAL_UART_Init(&g_rs458_handle);             /* HAL_UART_Init()会使能 USART2 */

    __HAL_UART_DISABLE_IT(&g_rs458_handle, UART_IT_TC);

#if EN_USART2_RX
    __HAL_UART_ENABLE_IT(&g_rs458_handle, UART_IT_RXNE);/* 开启接收中断 */
    HAL_NVIC_EnableIRQ(RS485_UX_IRQn);           /* 使能 USART1 中断 */
    HAL_NVIC_SetPriority(RS485_UX_IRQn, 3, 3);   /* 抢占优先级 3，子优先级 3 */
#endif

    rs485_tx_set(0);                             /* 设置为接收模式 */
}
```

可以看到代码基本跟串口的配置一样，只是多了收发控制引脚的配置。

2）发送函数

发送函数用于输出 485 信号到 485 总线上，我的默认的 485 方式一般空闲时为接收状态，只有发送数据时我们才控制 485 芯片进入发送状态，发送完成后马上回到空闲接收状态，这样可以保证操作过程中 485 的数据丢失最小。我们实现的发送函数如下：

```
/**
 * @brief       RS485 发送 len 个字节
 * @param       buf:发送区首地址
 * @param       len:发送的字节数(为了和本代码的接收匹配,这里建议不要超过 64 个字节)
 * @retval      无
 */
```

```
void rs485_send_data(uint8_t *buf, uint8_t len)
{
    rs485_tx_set(1);                                    /* 设置为发送模式 */
    HAL_UART_Transmit(&g_rs458_handle, buf, len, 1000);/* 串口 2 发送数据 */
    g_RS485_rx_cnt = 0;
    rs485_tx_set(0);                  /* 设置为接收模式 */
}
```

3）485 接收中断函数

RS485 的接收就与串口中断一样了，不过要注意空闲时要切换回接收状态，否则会收不到数据。我们定义了一个全局的缓冲区 g_RS485_rx_buf 进行接收测试，通过串口中断接收数据，编写的接收代码如下：

```
/* 接收缓存区 */
uint8_t g_RS485_rx_buf[64];           /* 接收缓冲,最大 64 个字节. */
/* 接收到的数据长度 */
uint8_t g_RS485_rx_cnt = 0;

void USART2_IRQHandler(void)
{
    uint8_t res;

    if ((__HAL_UART_GET_FLAG(&g_rs458_handle,
                             UART_FLAG_RXNE) != RESET)) /* 接收中断 */
    {
       HAL_UART_Receive(&g_rs458_handle, &res, 1, 1000);
       if (g_RS485_rx_cnt < 64)
       {
          g_RS485_rx_buf[g_RS485_rx_cnt] = res;        /* 记录接收到的值 */
          g_RS485_rx_cnt++;                            /* 接收数据增加 1  */
       }
    }
}
```

4）485 查询接收数据函数

该函数用于查询 485 总线上接收到的数据，主要实现的逻辑是：一开始进入函数时，先记录下当前接收计数器的值，再来一个延时去判断接收是否结束（即该期间有无接收到数据），假如说接收计数器的值没有改变，就证明接收结束，我们就可以把当前接收缓冲区传递出去。函数实现如下：

```
void rs485_receive_data(uint8_t *buf, uint8_t *len)
{
    uint8_t rxlen = g_RS485_rx_cnt;
    uint8_t i = 0;
    *len = 0;          /* 默认为 0 */
    delay_ms(10);      /* 等待 10ms,连续超过 10ms 没有接收到一个数据,则认为接收结束 */

    if (rxlen == g_RS485_rx_cnt && rxlen) /* 接收到了数据,且接收完成了 */
    {
        for (i = 0; i < rxlen; i++)
        {
            buf[i] = g_RS485_rx_buf[i];
        }
        *len = g_RS485_rx_cnt;     /* 记录本次数据长度 */
        g_RS485_rx_cnt = 0;        /* 清零 */
    }
}
```

RS485 的代码就讲到这里，基本是串口的知识，大家不明白的配置可以翻看之前串口章节的知识。

2. main.c 代码

在 main.c 中编写如下代码：

```
int main(void)
```

```
{
    uint8_t key;
    uint8_t i = 0, t = 0;
    uint8_t cnt = 0;
    uint8_t rs485buf[5];
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟,400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 初始化 USART */
    usmart_dev.init(200);                   /* 初始化 USMART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    rs485_init(9600);                       /* 初始化 RS485 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "RS485 TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Send", RED);

    lcd_show_string(30, 150, 200, 16, 16, "Count:", BLUE);
    lcd_show_string(30, 170, 200, 16, 16, "Send Data:", BLUE);
    lcd_show_string(30, 210, 200, 16, 16, "Receive Data:", BLUE);

    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)                   /* KEY0 按下,发送一次数据 */
        {
            for(i = 0;i < 5; i++)
            {
                rs485buf[i] = cnt + i;          /* 填充发送缓冲区 */
                lcd_show_num(30 + i * 32, 190,
                             rs485buf[i], 3, 16, 0X80);  /* 显示数据 */
            }
                rs485_send_data(rs485buf, 5);   /* 发送 5 个字节 */
        }
        rs485_receive_data(rs485buf, &key);
        if (key)                                /* 接收到有数据 */
        {
            if (key > 5)
            {
                key = 5;                        /* 最大是 5 个数据. */
            }
            for (i = 0; i < key; i++)
            {
                lcd_show_num(30 + i * 32, 230,
                             rs485buf[i], 3, 16, 0X80); /* 显示数据 */
            }
        }
        t++;
        delay_ms(10);
        if (t == 20)
        {
            LED0_TOGGLE(); /* 提示系统正在运行 */
            t = 0;
            cnt++;
            lcd_show_num(30 + 48, 150, cnt, 3, 16, 0X80);  /* 显示数据 */
        }
    }
```

```
}
```

我们是通过按键控制数据的发送。在此部分代码中，cnt 是一个累加数，一旦 KEY0 按下，就以这个数位基准连续发送 5 个数据。当 485 总线收到数据得时候，就将收到的数据直接显示在 LCD 屏幕上。

41.4 下载验证

在代码编译成功之后，我们通过下载代码到正点原子阿波罗 STM32H743 开发板上（注意要 2 个开发板都下载这个代码哦），得到如图 41.4.1 所示：

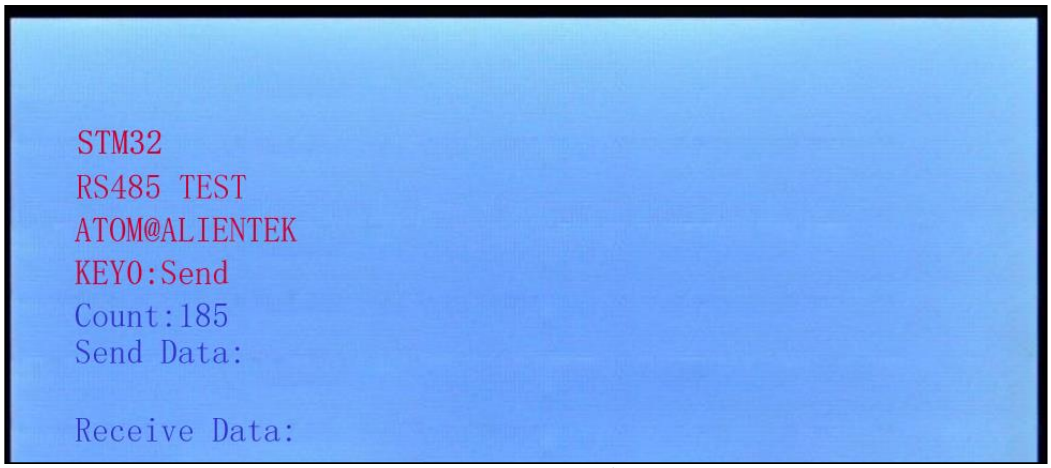


图 41.4.1 程序运行效果图

伴随 DS0 的不停闪烁，提示程序在运行。此时，我们按下 KEY0 就可以在另外一个开发板上面收到这个开发板发送的数据了。如图 41.4.2 和图 41.4.3 所示：

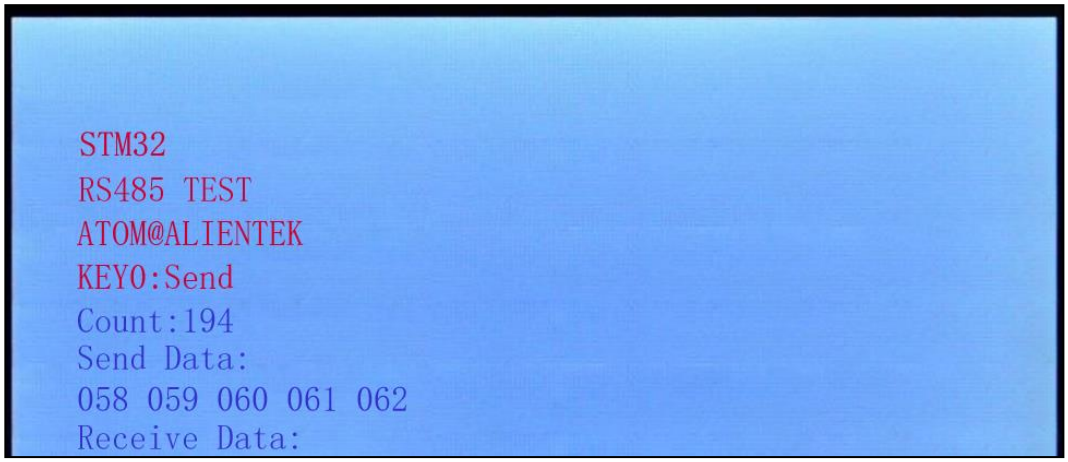


图 41.4.2 发送 RS485 数据的开发板界面

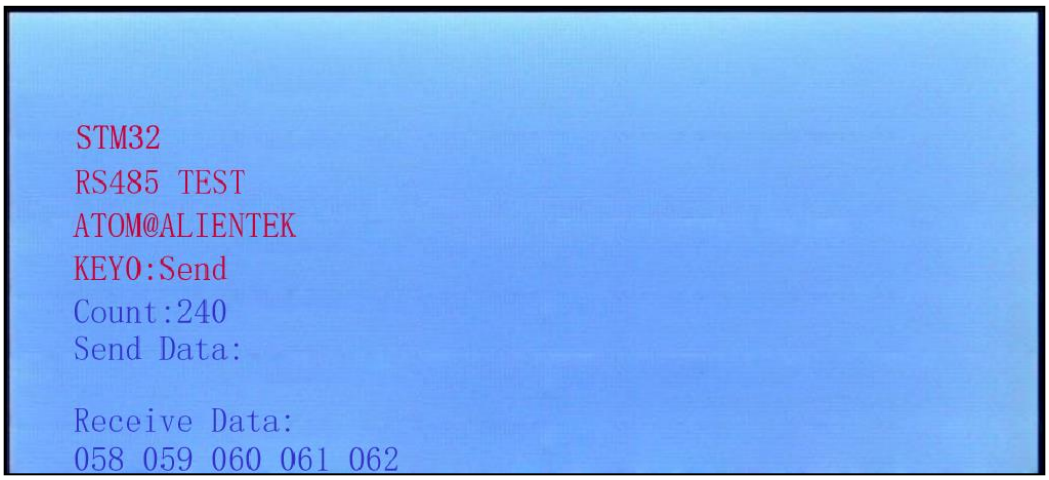


图 41.4.3 接收 RS485 数据的开发板

图 41.4.2 来自开发板 A，发送了 5 个数据，图 41.4.3 来自开发板 B，接收到了来自开发板 A 的 5 个数据。

本章介绍的 485 总线时通过串口控制收发的，我们只需要将 P7 的跳线帽稍作改变(将 PA2/PA3 连接 COM2_RX/COM2_TX)，该实验就变成了一个 RS232 串口通信实验了，通过对接两个开发板的 RS232 接口，即可得到同样的实验现象，不过 RS232 不需要使能脚，有兴趣的读者可以实验一下。

另外，利用 USMART 测试的部分，我们这里就不做介绍了，大家可自行验证下。

第四十二章 FDCAN 通讯实验

本章我们将向大家介绍如何使用 STM32H743 自带的 FDCAN 控制器，相比于 STM32 以前自带的 CAN，FDCAN 性能更为强大，FDCAN 每帧数据最大可以到 64 字节，FDCAN 速度远超 CAN 的 1Mbps，能够达到 5Mbps 甚至更高。本章我们就来实现两个开发板之间的 FDCAN 通讯，并将结果显示在 LCD 模块上。本章分为如下几个部分：

42.1 CAN/FDCAN 简介
42.2 硬件设计
42.3 程序设计
42.4 下载验证

42.1 CAN/FDCAN 总线简介

42.1.1 CAN 简介

CAN 是 Controller Area Network 的缩写（以下称为 CAN），是 ISO 国际标准化的串行通信协议。在当前的汽车产业中，出于对安全性、舒适性、方便性、低公害、低成本的要求，各种各样的电子控制系统被开发了出来。由于这些系统之间通信所用的数据类型及对可靠性的要求不尽相同，由多条总线构成的情况很多，线束的数量也随之增加。为适应“减少线束的数量”、“通过多个 LAN，进行大量数据的高速通信”的需要，1986 年德国电气商博世公司开发出面向汽车的 CAN 通信协议。此后，CAN 通过 ISO11898 及 ISO11519 进行了标准化，现在在欧洲已是汽车网络的标准协议。

现在，CAN 的高性能和可靠性已被认同，并被广泛地应用于工业自动化、船舶、医疗设备、工业设备等方面。现场总线是当今自动化领域技术发展的热点之一，被誉为自动化领域的计算机局域网。它的出现为分布式控制系统实现各节点之间实时、可靠的数据通信提供了强有力的技术支持。

CAN 协议具有一下特点：

① **多主控制。**在总线空闲时，所有单元都可以发送消息（多主控制），而两个以上的单元同时开始发送消息时，根据标识符（Identifier以下称为ID）决定优先级。ID并不是表示发送的目的地址，而是表示访问总线的消息的优先级。两个以上的单元同时开始发送消息时，对各消息ID的每个位进行逐个仲裁比较。仲裁获胜（被判定为优先级最高）的单元可继续发送消息，仲裁失利的单元则立刻停止发送而进行接收工作。

② **系统的柔软性。**与总线相连的单元没有类似于“地址”的信息。因此在总线上增加单元时，连接在总线上的其它单元的软硬件及应用层都不需要改变。

③ **通信速度较快，通信距离远。**最高1Mbps（距离小于40M），最远可达10KM（速率低于5Kbps）。

④ **具有错误检测、错误通知和错误恢复功能。**所有单元都可以检测错误（错误检测功能），检测出错误的单元会立即同时通知其他所有单元（错误通知功能），正在发送消息的单元一旦检测出错误，会强制结束当前的发送。强制结束发送的单元会不断反复地重新发送此消息直到成功发送为止（错误恢复功能）。

⑤ **故障封闭功能。**CAN可以判断出错误的类型是总线上暂时的数据错误（如外部噪声等）还是持续的数据错误（如单元内部故障、驱动器故障、断线等）。由此功能，当总线上发生持续数据错误时，可将引起此故障的单元从总线上隔离出去。

⑥ **连接节点多。**CAN总线是可同时连接多个单元的总线。可连接的单元总数理论上是没有限制的。但实际上可连接的单元数受总线上的时间延迟及电气负载的限制。降低通信速度，可连接的单元数增加；提高通信速度，则可连接的单元数减少。

正是因为 CAN 协议的这些特点，使得 CAN 特别适合工业过程监控设备的互连，因此，越来越受到工业界的重视，并已公认为最有前途的现场总线之一。

CAN 协议经过 ISO 标准化后有两个标准：ISO11898 标准（高速 CAN）和 ISO11519-2 标

准（低速 CAN）。其中 ISO11898 是针对通信速率为 125Kbps~1Mbps 的高速通信标准，而 ISO11519-2 是针对通信速率为 125Kbps 以下的低速通信标准。

42.1.2 FDCAN 简介

CAN 最高只能到 1Mbps，但是随着汽车技术的发展，人们对于更高的数据传输速度需求越来越强烈。再加上 CAN 与竞争对手产品的巨大差异，比如 FlexRay 最高可以到 10Mbps，2011 年 Bosch 发布了 CAN-FD（也就是 STM32H743 自带的 FDCAN），CAN-FD 继承了传统 CAN 的主要特性，CAN-FD 对带宽和数据长度做了优化，是下一代汽车总线。CAN-FD 相比 CAN 主要有以下几个优势：

① 、更高的传输速率

CAN-FD 可以以更高的速率传输数据，仲裁段（ID 和 ACK）的速率和 CAN 一样最高 1Mbps，保持不变，这样可以保证总线健壮可靠。但是 CAN-FD 帧数据段可以 5Mbps 甚至更高！这个技术叫做数据段波特率可变，CAN-FD 中 FD 的全称就是 Flexible Data-Rate。

② 、更长的数据帧

CAN 的一帧只能发送 8 字节数据，而 CAN-FD 一帧最高可以发送 64 字节的数据，大大的提高了数据传输效率，以前需要分几帧传输的报文，现在一帧就可以传输了。

CAN-FD 向下兼容 CAN，如果要想使用 CAN-FD 功能的话都需要哪些条件呢？

①、所使用的控制器或者 MCU 支持 CAN-FD 协议，比如 STM32H743 的 FDCAN 就支持 CAN-FD 协议。

②、所使用的 CAN 收发器芯片支持 CAN-FD，阿波罗 STM32H743 开发板上的 CAN 收发器 SIT1042T 或者 TPT1051V，这两款收发器均支持 CAN-FD 协议。因此，在 STM32H743 开发板上，我们将使用 FDCAN 协议，其最高仲裁段速率为 1Mbps，数据段速率至少为 5Mbit/s，每个数据帧最大可达 64 字节。

本章，我们使用 STM32H743 中的 FDCAN 功能实现通信，通信速率为 500Kbps，并将数据段速率设置为 5Mbit/s（仅 FDCAN 模式支持）。我们使用的是 ISO11898 标准，本章将首先讲解 CAN，然后再深入讲解 CAN-FD 相关内容，CAN 的物理层特征如图 42.1.2.1 所示：

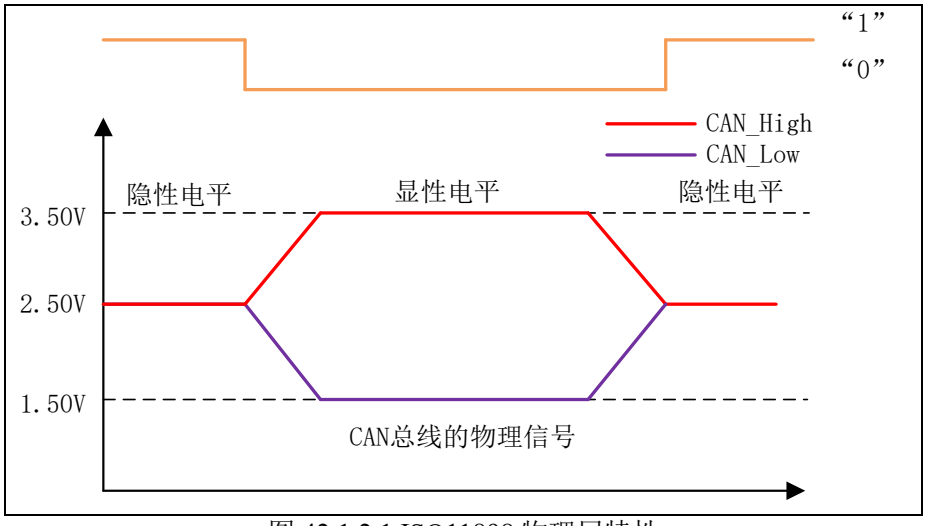


图 42.1.2.1 ISO11898 物理层特性

从该特性可以看出，显性电平对应逻辑 0，CAN_High 和 CAN_Low 之差为 2V 左右。而隐性电平对应逻辑 1，CAN_High 和 CAN_Low 之差为 0V。在总线上显性电平具有优先权，只要有一个单元输出显性电平，总线上即为显性电平。而隐性电平则具有包容的意味，只有所有的单元都输出隐性电平，总线上才为隐性电平（显性电平比隐性电平更强）。另外，在 CAN 总线的起止端都有一个 120Ω 的终端电阻，来做阻抗匹配，以减少回波反射。

42.1.3 CAN 协议

CAN 协议是通过以下 5 种类型的帧进行的：

- 数据帧
- 遥控帧
- 错误帧
- 过载帧
- 间隔帧

另外，数据帧和遥控帧有标准格式和扩展格式两种格式。标准格式有 11 个位的标识符（ID），扩展格式有 29 个位的 ID。各种帧的用途如表 42.1.3.1 所示：

帧类型	帧用途
数据帧	用于发送单元向接收单元传送数据的帧
遥控帧	用于接收单元向具有相同ID的发送单元请求数据的帧
错误帧	用于当检测出错误时向其它单元通知错误的帧
过载帧	用于接收单元通知其尚未做好接收准备的帧
间隔帧	用于将数据帧及遥控帧与前面的帧分离开来的帧

表 42.1.3.1 CAN 协议各种帧及其用途

由于篇幅所限，我们这里仅对数据帧进行详细介绍，数据帧一般由 7 个段构成，即：

帧起始。表示数据帧开始的段。
仲裁段。表示该帧优先级的段。
控制段。表示数据的字节数及保留位的段。
数据段。数据的内容，一帧可发送 0~8 个字节的数据。
CRC 段。检查帧的传输错误的段。
ACK 段。表示确认正常接收的段。
帧结束。表示数据帧结束的段。

数据帧的构成如图 42.1.3.1 所示：

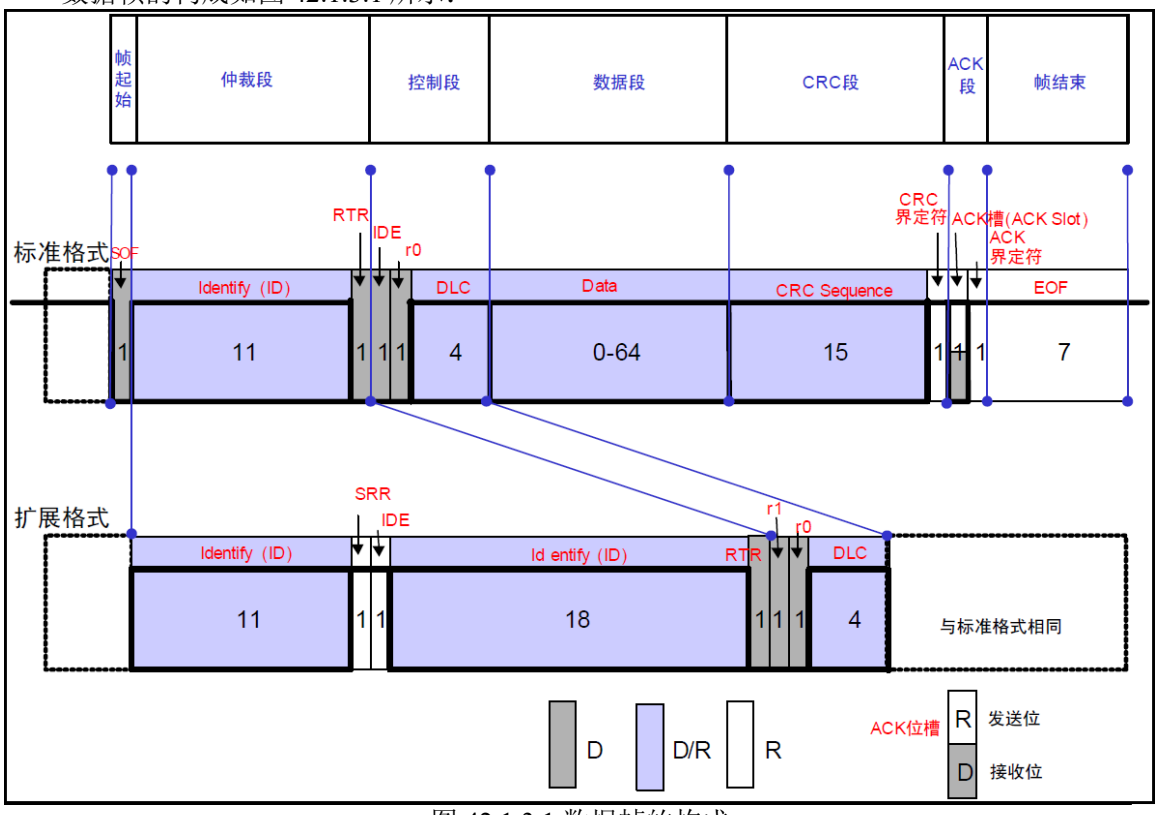


图 42.1.3.1 数据帧的构成

图中 D 表示显性电平，R 表示隐性电平（下同）。

帧起始，这个比较简单，标准帧和扩展帧都是由 1 个位的显性电平表示帧起始。

仲裁段，表示数据优先级的段，标准帧和扩展帧格式在本段有所区别，如图 42.1.3.2 所示：

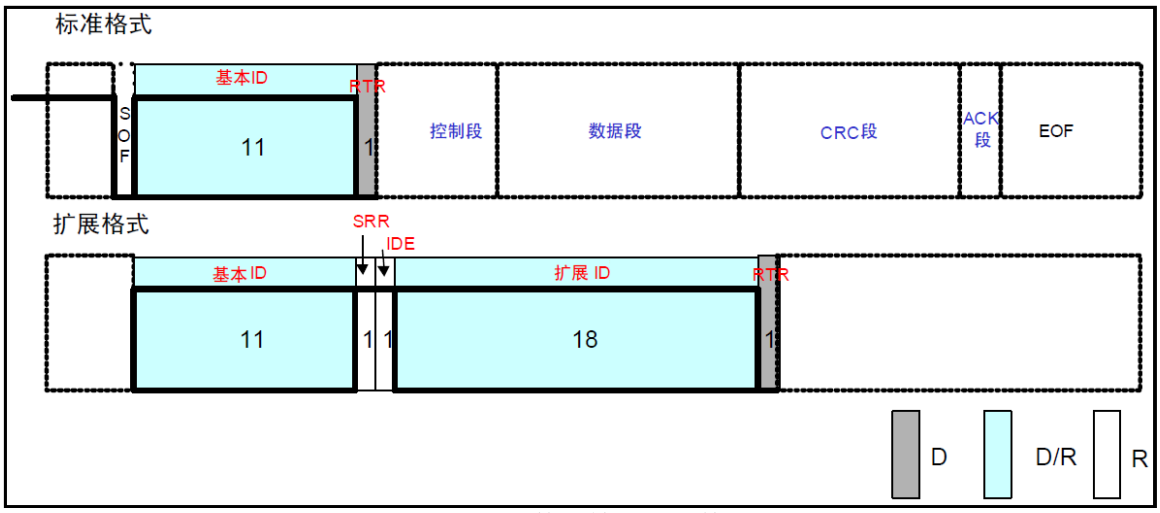


图 42.1.3.2 数据帧仲裁段构成

标准格式的 ID 有 11 个位。禁止高 7 位都为隐性（禁止设定：ID=1111111XXXX）。扩展格式的 ID 有 29 个位。基本 ID 从 ID28 到 ID18，扩展 ID 由 ID17 到 ID0 表示。基本 ID 和标准格式的 ID 相同。禁止高 7 位都为隐性（禁止设定：基本 ID=1111111XXXX）。

其中 RTR 位用于标识是否是远程帧（0，数据帧；1，远程帧），IDE 位为标识符选择位（0，使用标准标识符；1，使用扩展标识符），SRR 位为代替远程请求位，为隐性位，它代替了标准帧中的 RTR 位。

控制段，由 6 个位构成，表示数据段的字节数。标准帧和扩展帧的控制段稍有不同，如图 42.1.3.3 所示：

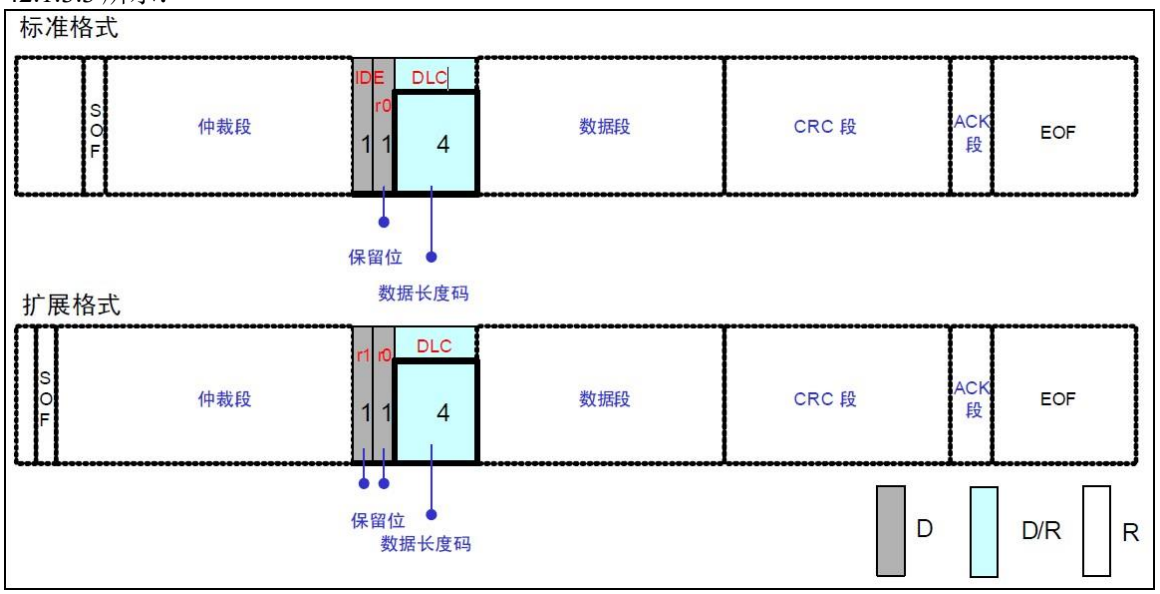


图 42.1.3.3 数据帧控制段构成

上图中，r0 和 r1 为保留位，必须全部以显性电平发送，但是接收端可以接收显性、隐性及任意组合的电平。DLC 段为数据长度表示段，高位在前，DLC 段有效值为 0~8，但是接收方接收到 9~15 的时候并不认为是错误。

数据段，该段可包含 0~8 个字节的数据。从最高位（MSB）开始输出，标准帧和扩展帧在这个段的定义都是一样的。如图图 42.1.3.4 所示：

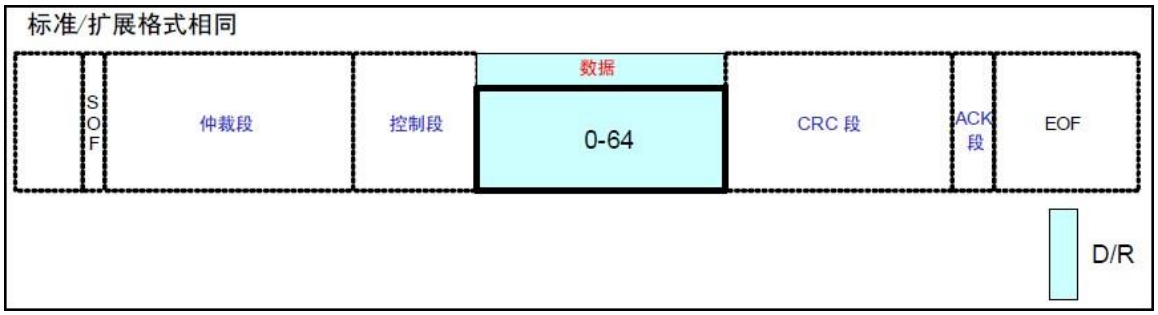


图 42.1.3.4 数据帧数据段构成

CRC 段，该段用于检查帧传输错误。由 15 个位的 CRC 顺序和 1 个位的 CRC 界定符（用于分隔的位）组成，标准帧和扩展帧在这个段的格式也是相同的。如图 42.1.3.5 所示：

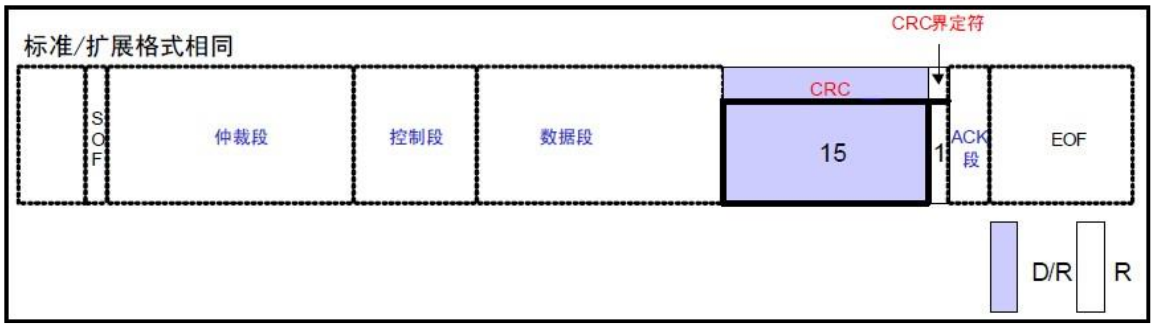


图 42.1.3.5 数据帧 CRC 段构成

此段 CRC 的值计算范围包括：帧起始、仲裁段、控制段、数据段。接收方以同样的算法计算 CRC 值并进行比较，不一致时会通报错误。

ACK 段，此段用来确认是否正常接收。由 ACK 槽(ACKSlot)和 ACK 界定符 2 个位组成。标准帧和扩展帧在这个段的格式也是相同的。如图 42.1.3.6 所示：

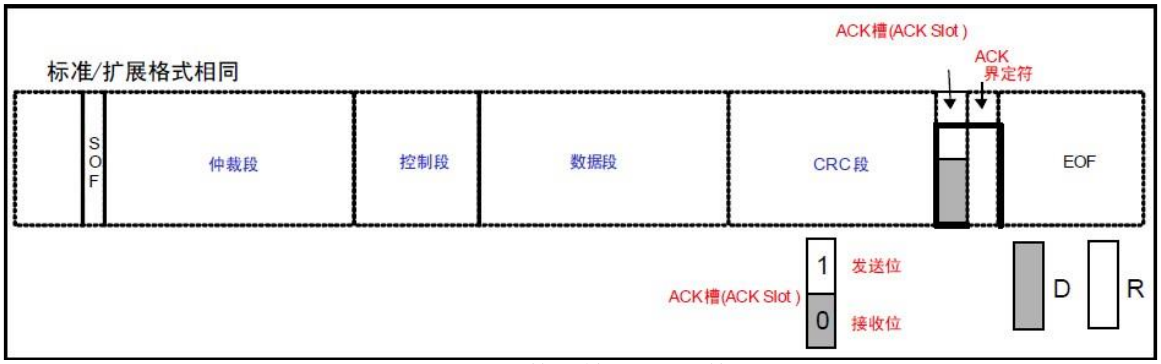


图 42.1.3.6 数据帧 CRC 段构成

发送单元的ACK，发送2个位的隐性位，而接收到正确消息的单元在ACK槽（ACKSlot）发送显性位，通知发送单元正常接收结束，这个过程叫发送 ACK/返回 ACK。发送 ACK 的是在既不处于总线关闭态也不处于休眠态的所有接收单元中，接收到正常消息的单元（发送单元不发送 ACK）。所谓正常消息是指不含填充错误、格式错误、CRC 错误的消息。

帧结束，这个段也比较简单，标准帧和扩展帧在这个段格式一样，由 7 个位的隐性位组成。

至此，数据帧的 7 个段就介绍完了，其他帧的介绍，请大家参考光盘的《CAN 入门书.pdf》相关章节。接下来，我们再来看看 CAN 的位时序。

由发送单元在非同步的情况下发送的每秒钟的位数称为位速率。一个位可分为 4 段。

- 同步段（SS）
- 传播时间段（PTS）
- 相位缓冲段 1（PBS1）
- 相位缓冲段 2（PBS2）

这些段又由可称为 Time Quantum（以下称为 Tq）的最小时间单位构成。

1 位分为 4 个段，每个段又由若干个 Tq 构成，这称为位时序。

1 位由多少个 Tq 构成、每个段又由多少个 Tq 构成等，可以任意设定位时序。通过设定位时序，多个单元可同时采样，也可任意设定采样点。各段的作用和 Tq 数如表 42.1.3.2 所示：

段名称	段的作用	Tq 数	
同步段 （SS: Synchronization Segment）	多个连接在总线上的单元通过此段实现时序调整，同步进行接收和发送的工作。由隐性电平到显性电平的边沿或由显性电平到隐性电平边沿最好出现在此段中。	1Tq	8～25Tq
传播时间段 （PTS: Propagation Time Segment）	用于吸收网络上的物理延迟的段。 所谓的网络的物理延迟指发送单元的输出延迟、总线上信号的传播延迟、接收单元的输入延迟。 这个段的时间为以上各延迟时间的和的两倍。	1～8Tq	
相位缓冲段 1 (PBS1: Phase Buffer Segment 1)	当信号边沿不能被包含于 SS 段中时，可在此段进行补偿。 由于各单元以各自独立的时钟工作，细微的时钟误差会累积起来，PBS 段可用于吸收此误差。 通过对相位缓冲段加减 SJW 吸收误差。 SJW 加大后允许误差加大，但通信速度下降。	1～8Tq	
相位缓冲段 2 (PBS2: Phase Buffer Segment 2)		2～8Tq	
再同步补偿宽度 (SJW: reSynchronization Jump Width)	因时钟频率偏差、传送延迟等，各单元有同步误差。SJW 为补偿此误差的最大值。	1～4Tq	

表 42.1.3.2 一个位各段及其作用

1 个位的构成如图 42.1.3.7 所示：

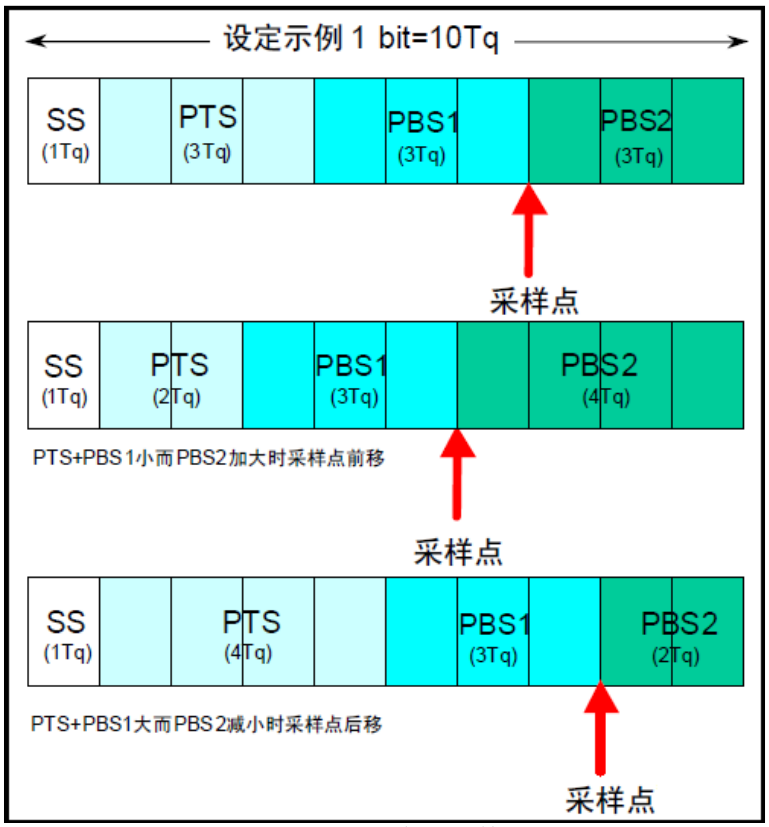


图 42.1.3.7 一个位的构成

上图的采样点，是指读取总线电平，并将读到的电平作为位值的点。位置在 PBS1 结束处。根据这个位时序，我们就可以计算 CAN 通信的波特率了。具体计算方法，我们等下再介绍，前面提到的 CAN 协议具有仲裁功能，下面我们来看看是如何实现的。

在总线空闲态，最先开始发送消息的单元获得发送权。当多个单元同时开始发送时，各发送单元从仲裁段的第一位开始进行仲裁。连续输出显性电平最多的单元可继续发送。实现过程，如图 42.1.3.8 所示：

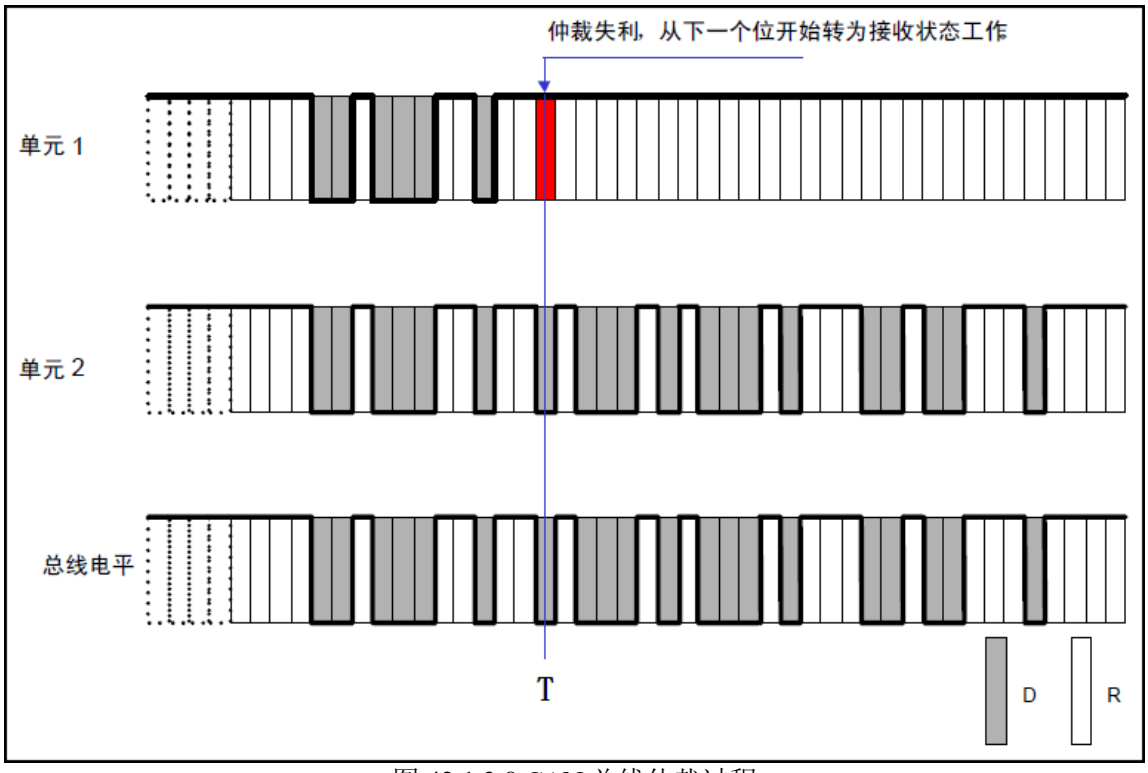


图 42.1.3.8 CAN 总线仲裁过程

上图中，单元 1 和单元 2 同时开始向总线发送数据，开始部分他们的数据格式是一样的，故无法区分优先级，直到 T 时刻，单元 1 输出隐性电平，而单元 2 输出显性电平，此时单元 1 仲裁失利，立刻转入接收状态工作，不再与单元 2 竞争，而单元 2 则顺利获得总线使用权，继续发送自己的数据。这就实现了仲裁，让连续发送显性电平多的单元获得总线使用权。

通过以上介绍，我们对 CAN 总线有了个大概了解（详细介绍参考光盘的：《CAN 入门书.pdf》），接下来我们介绍下 FDCAN 协议。

42.1.4 FDCAN 协议

FDCAN 发送的数据包由以下三个部分组成：

- 第一仲裁段 first arbitration phase,下图左蓝色部分
- 数据段 data phase,下图中红色部分
- 第二仲裁段 second arbitration phase,下图右蓝色部分

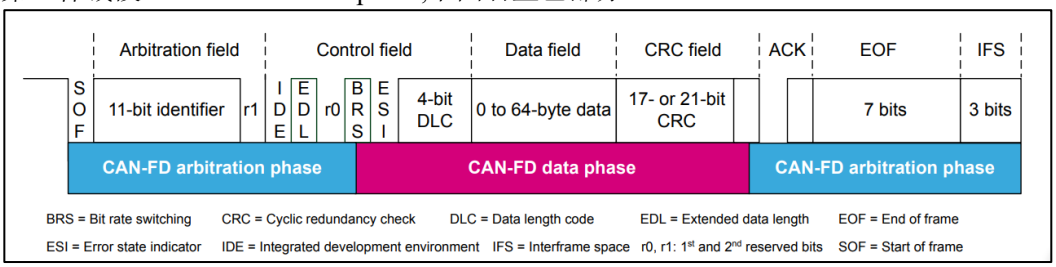


图 42.1.4.1 标准的 FDCAN 数据帧

由上图，可以看出 FDCAN 协议与传统的 CAN 协议非常相似。下面简要概述一个标准的 FDCAN 数据帧的构成，其中包括：

1、第一个仲裁阶段包含以下内容：

- 帧起始 SOF，与传统 CAN 协议一样由 1 个位的显性电平表示帧起始。
- ID 号和其他位，表示消息的目的 (提供或请求数据)以及速度和格式配置

2、数据段包含以下内容：

- 数据长度码 (DLC)，指示消息包含多少字节，该位有 4bit，因此有 16 种不同的数据

长度可供选择，分别为 0~8、12、16、20、24、32、48 和 64，这些固定的字节数选择可以用来确定数据包中消息的长度。注意传统 CAN 模式下最大支持 8 字节。FDCAN 模式下最大支持 64 字节。

- 用户希望发的数据。
- 循环冗余序列校验（CRC）, 通过以下方式确保数据传输的完整性：17 级多项式用于对 16 字节内的有效载荷进行 CRC 校验，21 级多项式用于对 16 字节与 64 字节之间的有效载荷进行校验。
- 显性位。

3、第二个仲裁阶段包含以下内容：

- 总线上其他节点发送的确认接收器（ACK）（如果至少有一个已成功接收到消息）；
- 帧结束（EOF），在 IFS 期间不发送消息, 目的是将当前帧与下一帧分开。

与标准 FDCAN 帧格式相比，29 位扩展帧的格式在标准帧的第一个仲裁阶段的 IDE 位之后会添加一个长度为 18 位的标识符。下图展示了扩展帧的格式。

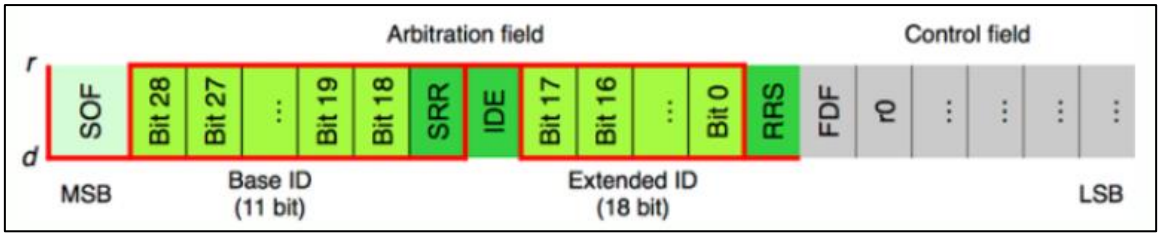


图 42.1.4.2 FDCAN 扩展帧格式

扩展帧我们这里不多做介绍，下面我们以标准帧为例，来讲解下传统 CAN2.0 与 FDCAN 的区别：

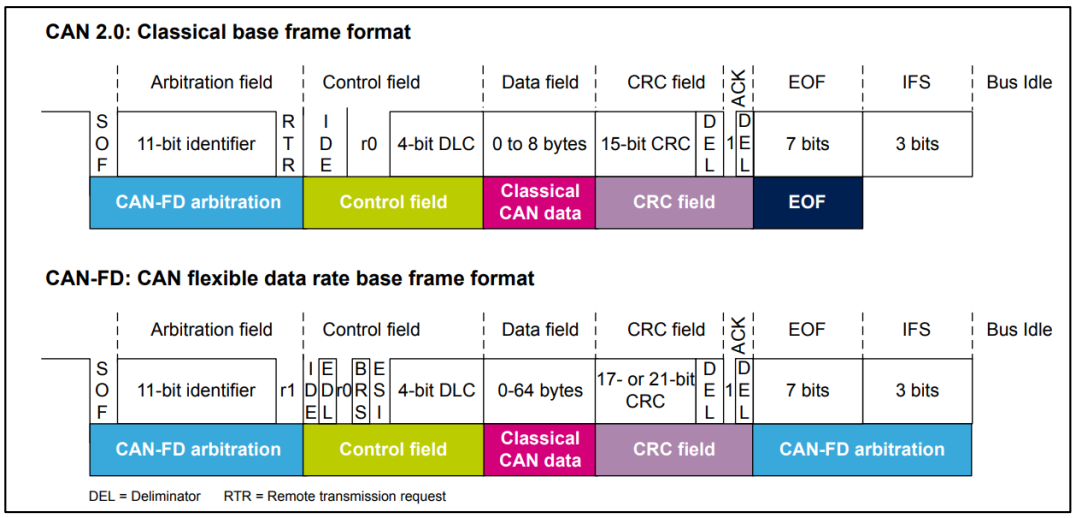


图 42.1.4.3 CAN 和 FDCAN 的区别

CAN 2.0 中 RTR 为显性(dominant, 逻辑 0)表示数据帧，为隐性(recessive，逻辑 1)表示远程帧。

FDCAN 仅支持数据帧，RTR 始终为显性数据帧，可以理解为保留不用。

IDE 位保持不变，用于区分标准帧还是扩展帧。FDCAN 在控制字段中新增了 3 个位：

- EDL，Extend data length，扩展数据长度位，隐性(逻辑 1)表示 FDCAN 帧，显性(逻辑 0，R0)表示 CAN2.0 帧
- BRS，Bit rate switching，位速率切换，指是否切换高速率传输，如从 500K 切换到 2M。
- ESI，Error state indicator，错误状态指示器，指示节点处于 error-active 模式还是 error-passive 模式。

DLC（Data Length Code）是数据长度代码的缩写，用于指示数据包中数据的长度。在 CAN 2.0 和 FDCAN 中，DLC 处于相同的位置，并且均为 4bit 长度，但 CAN2.0 和 FDCAN 的

最大数据有效载荷不同，如下表所示：

DLC (Dec)	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
CAN 2.0	0	1	2	3	4	5	6	7	8	8	8	8	8	8	8	8
CAN-FD	0	1	2	3	4	5	6	7	8	12	16	20	24	32	48	64

表 42.1.4.1 CAN 和 FDCAN 数据长度

由表 42.1.4.1 可知 FDCAN 的有效载荷从 CAN 2.0 的最大 8 字节提升到最大 64 字节从而改善了网络带宽，减少了对多包处理的需求。因此，通过增加 CRC 字段的位数以增强消息完整性成为必要措施：

- 有效载荷在 16 字节及以内，CRC 以 17-bit 编码；
- 有效载荷在 20 字节及以上，CRC 以 21-bit 编码。

下表汇总了 FDCAN 和 CAN2.0 之间的主要差异。相较于 CAN2.0，FDCAN 的主要改进特点在于其增加了数据有效载荷，并且通过可用的 BRS、EDL 和 ESI 位等功能，实现了更高的传输速率。

Features	CAN 2.0	CAN-FD
Compatibility	Does not support CAN-FD	Supports CAN 2.0 A/B
Maximum bit rate (Mbit/s)	Frame bitrate: up to 1	Arbitration bitrate: up to 1s Data bitrate: up to 8
DLC field (4 bits) code	Coded in 0 to 8	Coded in 0 to 64
Maximum data bytes in one message	8 bytes of data	64 bytes of data
BRS support	No	Yes
EDL support	No	Yes
ASI support	No	Yes
CRC bits check codes	Bits not included in CRC calculation	Bits included in CRC calculation
Remote frame support	Yes	No

表 42.1.4.2 CAN2.0 和 FDCAN 的主要差异

介绍完 CAN2.0 与 FDCAN 的区别后，接下来我们将重点介绍 STM32 的 FDCAN 外设的特点。

STM32H743 自带的是 FDCAN，它支持 CAN 协议 2.0A、2.0B 和 CAN-FD V1.0。它的设计目标是，以最小的 CPU 负荷来高效处理大量收到的报文。它也支持报文发送的优先级要求(优先级特性可软件配置)。对于安全紧要的应用，FDCAN 提供所有支持时间触发通信模式所需的硬件功能。

STM32H743 的 FDCAN 的主要特点有：

- 符合 CAN 协议版本 2.0A/B 和 ISO 11898-1
- 最多支持 64 个字节数据
- 支持错误记录
- 更好的接收过滤水平
- 支持 AUTOSAR 和 J1939
- 两个可配置的接收 FIFO
- 支持回环模式
- 仲裁段位速率最高 1Mbit/s，数据段位速率最高 8Mbit/s

在 STM32H743IIT6 中，带有 2 个 FDCAN 控制器，而我们本章只用了一个 FDCAN，即 FDCAN1。FDCAN 的框图如图 42.1.3.9 所示：

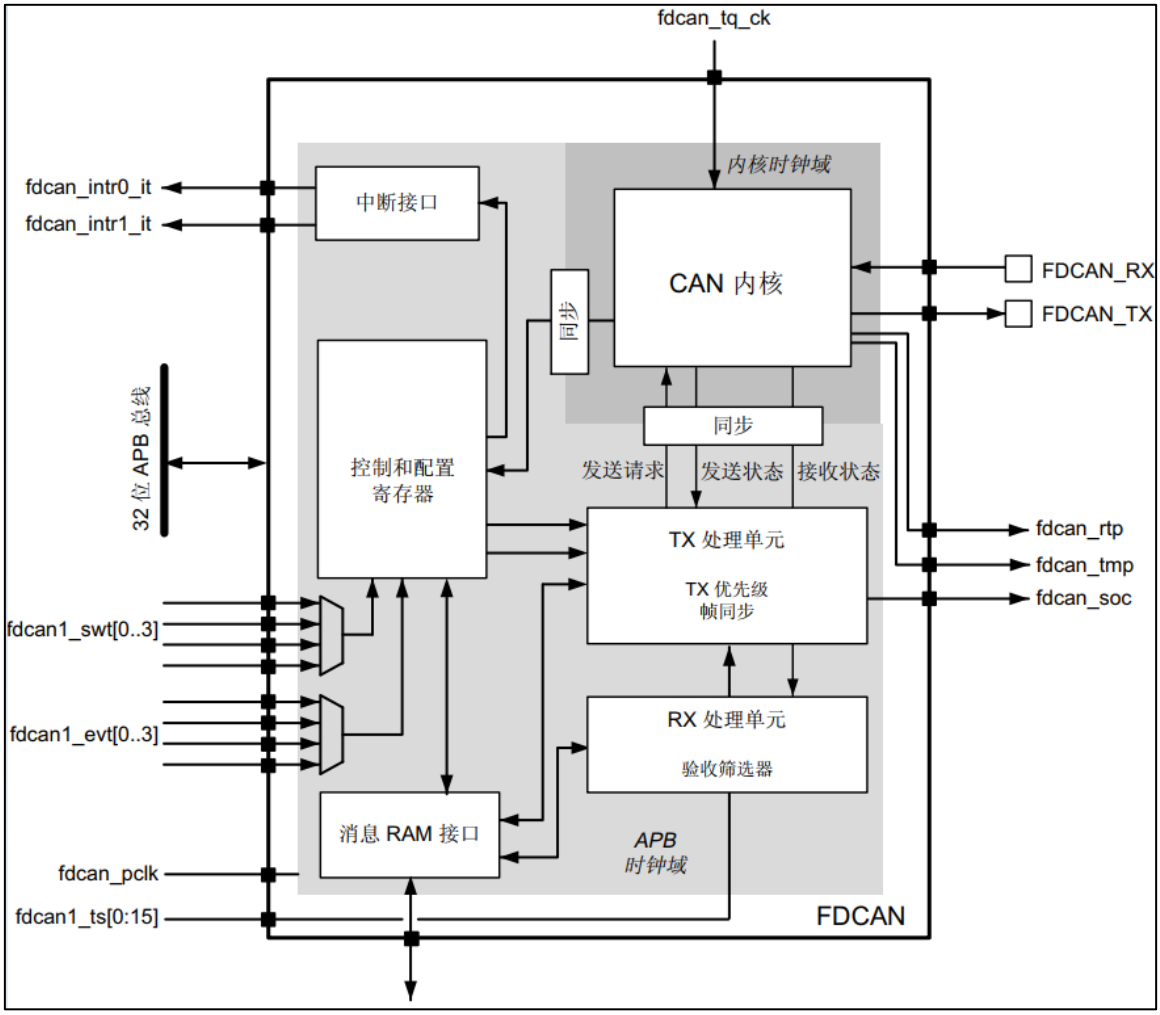


图 42.1.3.9 FDCAN 框图

① 双中断线

从图中可以看出，FDCAN 提供了两个中断线：fdcan_intr0_it 和 fdcan_intr1_it。可以通过寄存器 FDCAN_ILE 的 EINT0 和 EINT1 这两个位来使能或者关闭这两个中断。

② CAN 内核

CAN Core 包含协议控制器和收发移位寄存器，它支持 ISO 11898-1:2015 的所有协议功能，支持 11 位和 29 位 ID。

③ 同步

Sync 同步单元用于同步 APB 时钟信号和 CAN 内核时钟。

④ 发送处理

TX Handler 负责将消息 RAM 中的数据发送到 CAN 内核，最多可以给发送单元配置 32 个发送 buffer。

⑤ 接收处理

RX Handler 负责将 CAN 内核的数据传输到外部消息 RAM 中，RX Handler 支持两个接收 FIFO，每个 FIFO 可以配置 64 个专用的 buffer。

⑥ APB 接口

连接 FDCAN 到 APB 总线上。

⑦ 消息 RAM 接口

下一小节详细讲解消息 RAM 的接口。

42.1.5 消息 RAM

消息 RAM 是 FDCAN 的核心部分，消息 RAM 就是一段 10KB 的内存，在 HAL 库中提供了函数 FDCAN_CalcultateRamBlockAddress 来设置这段内存，这段内存分成了不同的区域，不同的区域代表的含义不同。STM32H743 的 FDCAN 在消息 RAM 实现了过滤器、接收 FIFO、接收 buffer、发送事件和发送 buffer。消息 RAM 是 FDCAN1 和 FDCAN2 共享的，消息 RAM 的内存分配如图 42.1.5.1 所示：

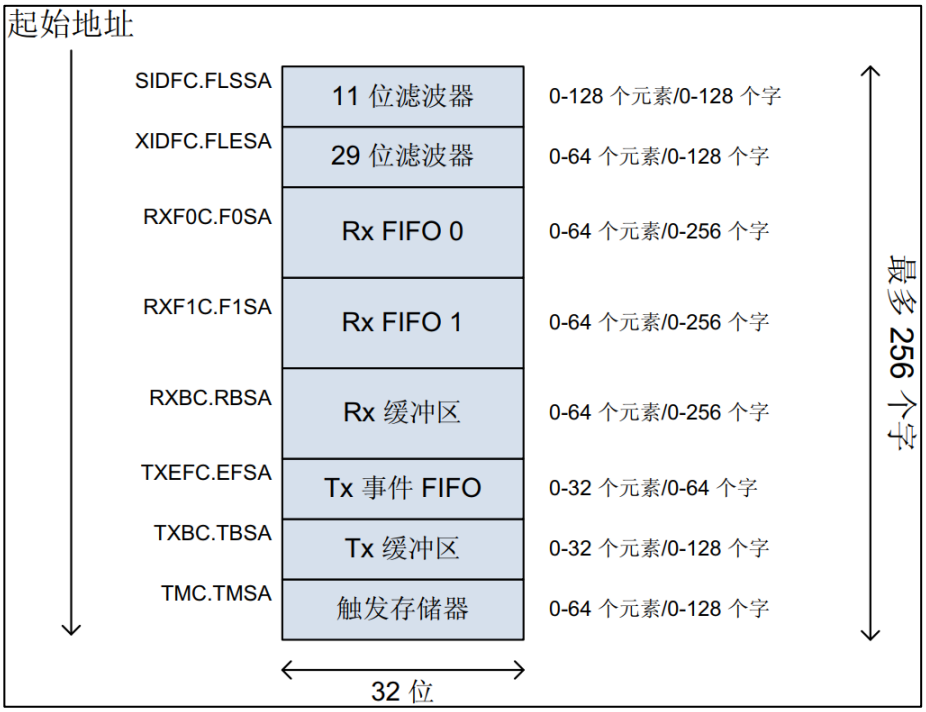


图 42.1.5.1 消息 RAM

消息 RAM 中一个元素的大小是 32 位，也就是 4 字节。因此 10K 的 RAM 一共有 10*1024/4=2560 个元素，也就是 2560 字，就是图 42.1.5.1 中的 Max 2560 Words。我们来看一下这 2560 个字都是怎么被分配的：

SIDFC.FLSSA：这个区域用来存放 11 位过滤 ID，后面“0-128 elements/0-128 words”中的“0-128 elements”表示这个区域占用了 128 个字，“128 words”表示 SIDFC.FLSSA 这个区域一共有 128 个字，下面的其他区域同理。在以前的 STM32 中（如 STM32F4/F7），过滤是通过寄存器来配置的，比如设置过滤 ID 等，在 STM32H743 中是通过设置 SIDFC.FLSSA 和 XIDFC.FLESA 这两个区域。

XIDFC.FLESA:这个区域用来存放 29 位过滤 ID，这个区域用了 64 字，一共有 128 个字。

RXF0C.F0SA:接收 FIFO0，这个区域用了 64 个字，一共有 1152 个字。

RXF1C.F1SA:接收 FIFO1，这个区域用了 64 个字，一共有 1152 个字。

RXBC.RBSA:接收 buffer，这个区域用了 64 个字，一共有 1152 个字。

TXEFC.EFSA:发送事件 FIFO，这个区域用了 32 个字，一共有 64 个字。

TXBC.TBSA:发送 buffer，这个区域用了 32 个字，一共有 576 个字。

TMC.TMSA:触发内存，这个区域用了 64 个字，一共有 128 个字。

接下来我们详细的分析一下消息 RAM 中这些区域的具体含义：

1、SIDFC.FLSSA

SIDFC.FLSSA 是标准消息 ID（11 位）滤波区域，一共有 128 个元素，每个元素的具体 bit 含义如下图 42.1.5.2 所示：

位	31	24	23	16	15	8	7	0
S0	SFT[1:0]	SFEC[2:0]	SFID1[10:0]		Res.		SFID2[10:0]	

图 42.1.5.2 SIDFC.FLSSA

SFT[1:0](bit31:30):这两位用来表示过滤类型，一共有四种：

00 范围过滤，过滤到 SFID1 到 SFID2 中的所有信息。
01 双过滤，过滤到 SFID1 和 SFID2 中的所有信息。
10 传统位过滤，SFID1 是过滤值，SFID2 是掩码。
11 关闭过滤机制。

SFEC[2:0](bit29:27):标准过滤配置

000 关闭过滤机制。
001 当过滤匹配以后存储在 Rx FIFO0 中。
010 当过滤匹配以后存储在 Rx FIFO1 中。
011 当过滤匹配以后丢弃。
100 当过滤匹配以后设置优先级。
101 当过滤匹配以后存储在 Rx FIFO0 中并设置优先级。
110 当过滤匹配以后存储在 Rx FIFO1 中并设置优先级。
111 存储到 Rx buffer 中或者作为 debug 消息，FDCAN_SFT[1:0]忽略。

SFID1[10:0](bit26:16):标准过滤 ID1，第一个要过滤的 ID。

SFID2[15:10](bit15:10):标准过滤 ID2，此位根据 SFEC 的配置不同而不同，当

SFEC=001~110 的时候此位域表示标准过滤 ID。
SFEC=111 的时候此位域表示 Rx buffer 或者 debug 消息。

SFID2[10:9](bit10:9):设置接收到的消息是存放在 Rx buffer 中还是处理为 debug 消息的消息 A、B 或 C。

00 将消息存储在 Rx buffer 中。
01 debug 消息 A。
10 debug 消息 B。
11 debug 消息 C。

SFID2[8:6](bit8:6):确定是否将滤波器事件引脚作为外部接口。

SFID2[5:0](bit5:0):定义当匹配以后存储消息的区域相对于 Rx buffer 的开始地址 RXBC.RBSA 的偏移。

2、XIDFC.FLESA

XIDFC.FLESA 是扩展消息 ID（29 位）滤波区域，一共有 64 个元素，每个元素的具体 bit 含义如下图 42.1.5.3 所示：

位	31		24	23 16	15 8	7 0
F0	EFEC[2:0]			EFID1[28:0]		
F1	EFTI[1:0]	Res.		EFID2[28:0]		

图 42.1.5.3 XIDFC.FLESA

F0 EFEC[2:0](bit31:29):扩展过滤配置

000 关闭过滤机制
001 当过滤匹配以后存储在 Rx FIFO0 中。
010 当过滤匹配以后存储在 Rx FIFO1 中。
011 当过滤匹配以后丢弃。
100 当过滤匹配以后设置优先级。
101 当过滤匹配以后存储在 Rx FIFO0 中并设置优先级。
110 当过滤匹配以后存储在 Rx FIFO1 中并设置优先级。
111 存储到 Rx buffer 中，EFT[1:0]忽略。

F0 EFID1[28:0](bit28:0):扩展过滤 ID1。

F1 EFTI[1:0](bit31:30):扩展过滤类型

00 范围过滤，过滤到 EF1ID 到 EF2ID 中的所有信息。
01 双过滤，过滤到 EF1ID 和 EF2ID 中的信息。

10 传统位过滤，EF1ID 是过滤值，EF2ID 是掩码。

11 范围过滤，过滤到 ED1ID 到 ED2ID 中的所有信息，没有使用 FDCAN_XIDAM 的掩码。

F1 EFID2[10:9](bit10:9):设置接收到的消息是存放在 Rx buffer 中还是处理为 debug 消息的消息 A、B 或 C。

00 将消息存储在 Rx Buffer 中。

01 debug 消息 A。

10 debug 消息 B。

11 debug 消息 C。

F1 EFID2[8:6](bit8:6):确定是否将滤波器事件引脚作为外部接口。

F1 EFID2[5:0](bit5:0):定义当匹配以后存储消息的区域相对于 Rx buffer 的开始地址 RXBC.RBSA 的偏移。

3、RXF0C.F0SA、RXF1C.F1SA 和 RXBC.RBSA

这三个分别为 Rx FIFO0、Rx FIFO1 和 Rx buffer，这三个的 bit 含义都是一样的，见图 42.1.5.4 所示：

位	31			24	23				16	15	8	7	0
R0	ESI	XTD	RTR	ID[28:0]									
R1	ANMF	FIDX[6:0]			Res.	FDF	BRS	DLC[3:0]		RXTS[15:0]			
R2	DB3[7:0]				DB2[7:0]					DB1[7:0]		DB0[7:0]	
R3	DB7[7:0]				DB6[7:0]					DB5[7:0]		DB4[7:0]	
⋮	⋮				⋮					⋮			
Rn	DBm[7:0]				DBm-1[7:0]					DBm-2[7:0]		DBm-3[7:0]	

图 42.1.5.4 Rx buffer 和 FIFO

R0 ESI(bit31):错误状态标志

0 传输节点显性错误

1 传输节点隐性错误

R0 XTD(bit30):标记是标准 ID 还是扩展 ID

0 11 位标准 ID

1 29 位扩展 ID

R0 RTR(bit29):遥控传输请求，标记当前帧是数据帧还是遥控帧

0 接收到的帧是数据帧

1 接收到的帧是遥控帧

R0 ID[28:0](bit28:0):帧 ID，根据 XTD 位决定是 11 位 ID 还是 29 位 ID。

R1 ANMF(bit31):0 接收到的帧和 FIDX 中的过滤帧匹配

1 接收到的帧和 Rx 滤波器中的任何元素都不匹配

R1 FIDX[6:0](bit30:24):过滤器索引，0~127，表示和 Rx 滤波器中的哪个元素匹配。

R1 FDF(bit21):帧格式

0 标准帧格式

FDCAN 帧格式

R1 BRS(bit20):比特率切换

0 接收到的帧没有比特率切换

1 接收到的帧有比特率切换

R1 DLC[3:0](bit):数据长度。

0-8 传统 CAN+CAN FD，接收帧长度为 0~8 字节。

9-15 传统 CAN，接收帧长度为 8 字节。

9-15 CAN FD 模式的话表示接收到的长度 12/16/20/24/32/48/64 字节。

R1 RXTS[15:0](bit15:0):接收时间戳。

R2-Rn:接收到的数据

4、RXBC.RBSA

这个域是 Tx Buffer，一共有 32 个元素，每个元素的具体 bit 含义如下图 42.1.5.5 所示：

位	31			24	23					16	15	8	7	0
T0	ESI	XTD	RTR		ID[28:0]									
T1	MM[7:0]				EFC	Res.	FDF	BPS	DLC[3:0]		Res.			
T2	DB3[7:0]				DB2[7:0]						DB1[7:0]		DB0[7:0]	
T3	DB7[7:0]				DB6[7:0]						DB5[7:0]		DB4[7:0]	
⋮	⋮				⋮						⋮			
Tn	DBm[7:0]				DBm-1[7:0]						DBm-2[7:0]		DBm-3[7:0]	

图 42.1.5.5 Tx Buffer

T0 ESI(bit31):错误状态标志

0 CAN FD 帧中取决于被动错误标志。

1 CAN FD 帧中隐形错误。

T0 XTD(bit30):标记是标准 ID 还是扩展 ID

0 11 位标准 ID

1 29 位扩展 ID

T0 RTR(bit29):遥控传输请求，标记当前帧是数据帧还是遥控帧。

0 发送的帧是数据帧

1 发送的帧是遥控帧

T0 ID[28:0](bit28:0):帧 ID，根据 XTD 位决定是 11 位还是 29 位。

T1 MM[7:0](bit31:24):消息掩码，配置 Tx buffer 的时候由 CPU 写入。

T1 EFC(bit23):事件 FIFO 控制

0 不存储发送事件

1 存储发送事件

T1 FDF(bit21):帧格式

0 标准帧格式

1 FDCAN 帧格式

T1 BRS(bit20):比特率切换

0 发送的帧没有比特率切换

1 发送的帧带有比特率切换

T1 DLC[19:16](bit):数据长度。

0-8 传统 CAN+CAN FD，接收帧长度为 0~8 字节。

9-15 传统 CAN，接收帧长度为 8 字节。

9-15 CAN FD 模式的话表示接收到的长度 12/16/20/24/32/48/64 字节。

T2-Tn:发送的数据。

5、TXEFC.EFSA

这个域是 Tx event FIFO，一共是 32 个元素，每个元素的具体含义如下图 42.1.5.6 所示：

位	31			24	23					16	15	8	7	0
E0	ESI	XTD	RTR		ID[28:0]									
E1	MM[7:0]				ET[1:0]		EDL	BRS	DLC[3:0]		TXTS[15:0]			

图 42.1.5.6 Tx event FIFO

E0 ESI(bit31):错误状态标志。

0 传输节点显性错误

1 传输节点隐性错误

E0 XTD(bit30):标记是标准 ID 还是扩展 ID。

0 11 位标准 ID
1 29 位扩展 ID

E0 RTR(bit29):遥控传输请求，标记当前帧是数据帧还是遥控帧。
0 发送的帧是数据帧
1 发送的帧是遥控帧

E0 ID[28:0](bit28:0):帧 ID，根据 XTD 位决定是 11 位 ID 还是 29 位 ID。

E1 MM[7:0](bit31:24):消息掩码，从 Tx buffer 拷贝到 Tx Event FIFO 中。

E1 EFC(bit23:22):事件类型。
00 保留。
01 发送事件。
10 发送取消。
11 保留。

E1 EDL(bit21):扩展数据长度。
0 标准帧格式
1 FDCAN 帧格式

E1 BRS(bit20):比特率切换。
0 发送的帧没有比特率切换
1 发送的帧带有比特率切换

E1 DLC[19:16](bit):数据长度。
0-8 传统 CAN+CAN FD，长度为 0~8 字节。
9-15 长度为 8 字节。

E1 TXTS[15:0](bit15:0):时间戳。

可以看出消息 RAM 和寄存器一样的，都可以设置相应的 bit 来完成不同的功能。关于消息 RAM 中各个域就讲到这里，后面设置滤波器、发送和接收数据都跟消息 RAM 的各个域有关。关于消息 RAM 的详细请参考《STM32H7xx 参考手册_V3（中文版）》的 55.3.2 节（2442 页）。

42.1.6 位时间特性

接下来，我们简单看看 STM32H743 的 FDCAN 位时间特性，因为 FDCAN 的波特率就是通过位时间来设置的。STM32H743 的 FDCAN 位时间有 3 段：同步段（SYNC_SEG）、时间段 1（BS1）和时间段 2（BS2）。STM32H743 的同步段长度为 1 个时间单元 t_q，BS1 段可以设置为 1~16 个时间单元 t_q，BS2 段可以设置 1~8 个时间单元 t_q。STM32H743 的 CAN 位时序如图 42.1.6.1 所示：

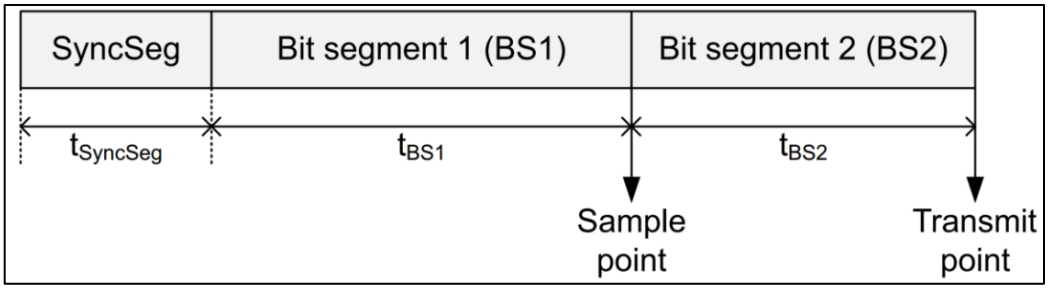


图 42.1.6.1 STM32H743 CAN 时序

FDCAN 的波特率计算公式如下：

baudrate = 1/bit time。

bit time = $t_{SyncSeg}+t_{BS1}+t_{BS2}$。

对于标称位时间

t_q = (FDCAN_NBTP.NBRP[8:0]+1) * $t_{fdcan_tq_clk}$

$t_{SyncSeg}$ = 1 * t_q

t_{BS1} = t_q * (FDCAN_NBTP.NTSEG1[7:0]+1)

t_{BS2} = t_q * (FDCAN_NBTP.NTSEG2[6:0]+1)

对于数据位时间

t_q = (FDCAN_NBTP.NBRP[4:0]+1) * $t_{fdcan_tq_clk}$

$t_{SyncSeg}$ = 1 * t_q

t_{BS1} = t_q * (FDCAN_DBTP.DTSEG1[4:0]+1)

t_{BS2} = t_q * (FDCAN_DBTP.DTSEG2[3:0]+1)

上面波特率计算公式有标称位时间和数据位时间两种，我们这里使用标称位时间，波特率计算公式可以简化为：

$$baudrate = \frac{f_{fdcan_tq_ck}}{(FDCAN_NBTP.NTSEG1[7:0] + FDCAN_NBTP.NTSEG2[6:0] + 3) * (FDCAN_NBTP.NBRP[8:0] + 1)}$$

当使用可变位速率的 FDCAN 模式才需要用到数据位时间的计算，计算公式可以简化为：

$$baudrate = \frac{f_{fdcan_tq_ck}}{(FDCAN_DBTP.DTSEG1[4:0] + FDCAN_DBTP.DTSEG2[3:0] + 3) * (FDCAN_DBTP.DBRP[4:0] + 1)}$$

我们只需要确定：$f_{fdcan_tq_ck}$、FDCAN_NBTP.NTSEG1[7:0]、FDCAN_NBTP.NTSEG2[6:0]和 FDCAN_NBTP.NBRP[8:0]、FDCAN_DBTP.DTSEG1[4:0]、FDCAN_DBTP.DTSEG2[3:0]和 FDCAN_DBTP.DBRP[4:0]的值，就可以确定 CAN 的波特率和数据段位速率是多少了。

其中，$f_{fdcan_tq_ck}$ 是 CAN 的时间片时钟，用于生成位时序，其来源如图 42.1.6.2：

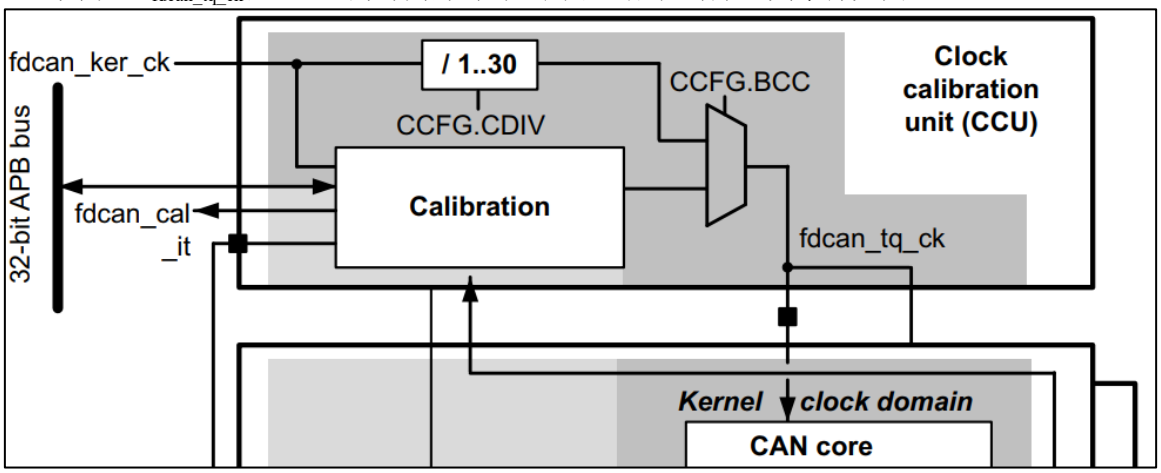


图 42.1.6.2 CAN 时间片时钟来源图

图中 fdcan_ker_ck 是 CAN 内核时钟，该时钟经过 CCU 处理后，生成 fdcan_tq_ck 时钟，输入 CAN 内核，作为 CAN 时间片时钟，用于生成位时序。

fdcan_ker_ck 由 RCC_D2CCIP1R 寄存器的 FDCANSEL[1:0]位选择时钟来源：00，hse_ck；01，pll1_q_ck；10，pll2_q_ck；11，保留。我们设置 FDCANSEF[1:0]的值为 01，选择 pll1_q_ck（一般为 200MHz）作为 fdcan_ker_ck 的时钟源。

fdcan_pclk 是 CAN 的外设时钟，用于访问 CAN 相关寄存器等等。FDCAN 是挂在 APB1 上面的，所以 fdcan_pclk 的时钟就等于 APB1 时钟，一般为 100MHz。

fdcan_tq_ck 是 CAN 的时间片时钟，由 fdcan_ker_ck 分频而来，其频率必须满足该条件：fdcan_tq_ck≤fdcan_pclk，当 fdcan_ker_ck=200MHz 时，至少需设置 CCFG.CDIV 为 2 分频。

FDCAN_NBTP.NTSEG1[7:0]、FDCAN_NBTP.NTSEG2[6:0]和 FDCAN_NBTP.NBRP[8:0]等三个参数由寄存器 FDCAN_NBTP 设置，假设我们设置：fdcan_tq_ck=50Mhz，NTSEG1=31，NTSEG2=8，NBRP=10，则可以得到 500kbps 的 CAN 通信波特率。

42.1.7 过滤器设置

FDCAN 提供了过滤器设置，通过过滤器可以设置允许接收哪些 ID 的消息。前面讲解消息 RAM 的时候详细的讲过 SIDFC.FLSSA 和 XIDFC.FLSSA 这两个域，分别为标准 ID 过滤器和扩展 ID 过滤器。我们以标准 ID 过滤器为例讲解如何设置 FDCAN 的过滤器。

标准 ID 过滤器有三种过滤模式：

1、 指定范围过滤

通过 SIDFC.FLSSA 的 SFID1 和 SFID2 来设置需要过滤的 ID 范围，其中 SFID2 的值要大于 SFID1，这样只有 ID 值在 SFID1~SFID2 之内的消息才能被接收到。比如我们现在要设置只接收 ID 在 0X123~0X321 范围内的消息，使用标准滤波器 n，SIDFC.FLSSAn(n=0~128)的各个位设置如下：

```
SIDFC.FLSSAn.SFT=0              /* 范围滤波 */
SIDFC.FLSSAn.SFEC=1             /* 如果滤波匹配成功的话将消息保存到 Rx FIFO 中 */
SIDFC.FLSSAn.SFID1=0x123        /* ID1 */
SIDFC.FLSSAn.SFID2=0x321        /* ID2 */
```

2、指定 ID 过滤

我们也可以设置只接收指定的一个或者两个 ID 的消息，如果只接收指定的一个 ID 消息的话 SFID1=SFID2.比如我们要设置只接收 ID 为 0X123 的消息，设置如下：

```
SIDFC.FLSSAn.SFT=1              /* 范围滤波 */
SIDFC.FLSSAn.SFEC=1             /* 如果滤波匹配成功的话将消息保存到 Rx FIFO 中 */
SIDFC.FLSSAn.SFID1=0x123        /* ID1 */
SIDFC.FLSSAn.SFID2=0x123        /* ID2 */
```

3、传统的位过滤

第三种过滤模式就是以前 STM32 的 CAN 上存在的过滤模式，在屏蔽位模式下，过滤消息 ID 和过滤掩码一起工作决定接收哪些消息，其中 SFID1 为过滤的消息 ID，SFID2 为过滤掩码。

举个简单的例子，我们设置过滤器 SIDFC.FLSSAn 工作在：传统过滤模式，然后设置如下：

```
SIDFC.FLSSAn.SFT=2              /* 传统位过滤 */
SIDFC.FLSSAn.SFEC=1             /* 如果滤波匹配成功的话将消息保存到 Rx FIFO 中 */
SIDFC.FLSSAn.SFID1=0xFF00       /* ID1 */
SIDFC.FLSSAn.SFID2=0xF000       /* 掩码 */
```

其中 SFID1 是我们期望接收到的消息 ID，我们希望最好接收到 ID=0XFF00 的消息。SFID2 的 0XF000 规定了我们必须关心的 ID，也就是接收到的消息 ID 其位[15:12]必须和 SFID1 中的位[15:12]完全一样，其他的位不关心。也即是说接收到的消息 ID 必须是 0XFFxx 这样的才算正确（x 表示不关心）。

关于滤波设置的详细接收请参考《STM32H7xx 参考手册_V3（中文版）》的 55.3.2 节中的滤波小节（2444 页）。

42.1.8 FDCAN/CAN 收发工程

1、CAN 接收流程

CAN 接收到消息以后存放到 FIFO0 或者 FIFO1 中，至于是放到 FIFO0 还是 FIFO1 中，我们在初始化 CAN 的时候会设置。如果消息满足过滤器设置的话消息就存放到 FIFO 中，当 FIFO 满了以后就不能再存任何消息了，除非有消息从 FIFO 中读出。读出消息就是直接从 FIFO 读数据，比如现在使用的是 FIFO0，那就直接读取 RXF0C.F0SAn（n 为索引号 0~64），消息的组成方式前面讲解 RXF0C.F0SA 的时候就详细的说过了。比如 HAL 库中从 FIFO 中读取消息的函数 HAL_FDCAN_GetRxMessage 处理方式如下：

```
GetIndex+=((hfdcan->Instance->RXF0S&FDCAN_RXF0S_F0GI>> FDCAN_RXF0S_F0GI_Pos);
/* 计算索引 */
RxAddress=(uint32_t *)(hfdcan->msgRam.RxFIFO0SA+(GetIndex*
  hfdcan->Init.RxFifo0ElmtSize * 4U));    /* 要读取的 FIFO 地址 */
```

上面代码我们已经得到了消息的 FIFO 地址，接下来就是按照 RXF0C.F0SA 的格式来恢复数据了，处理方法如下：

```
pRxHeader->IdType = *RxAddress & FDCAN_ELEMENT_MASK_XTD;
pRxHeader->Identifier = ((*RxAddress & FDCAN_ELEMENT_MASK_STDID) >> 18);
pRxHeader->RxFrameType = (*RxAddress & FDCAN_ELEMENT_MASK_RTR);
pRxHeader->ErrorStateIndicator = (*RxAddress & FDCAN_ELEMENT_MASK_ESI);
……
/* 读取消息数据 */
pData = (uint8_t *)RxAddress;
for (ByteCounter=0;ByteCounter<DLCtoBytes[pRxHeader->DataLength >> 16];
ByteCounter++)
{
  pRxData[ByteCounter] = pData[ByteCounter];
```

```
}
```

关于 CAN 的接收详细过程，可以参考 HAL 库中的函数 HAL_FDCAN_GetRxMessage。

2、CAN 发送流程

CAN 的发送流程和 CAN 的接收流程刚好相反，我们需要将消息按照 TXBC.TBSA 格式构建好，然后写入到发送 FIFO（buffer）中，写进去以后设置 TXBAR 寄存器的指定位为 1 来请求传输。HAL 库中函数 HAL_FDCAN_AddMessageToTxFifoQ 来完成此工作，处理部分如下：

```
PutIndex=((hfdcan->Instance->TXFQS&FDCAN_TXFQS_TFQPI)
    >>FDCAN_TXFQS_TFQPI_Pos);
/* 获取索引 */
FDCAN_CopyMessageToRAM(hfdcan, pTxHeader, pTxData, PutIndex);
/* 拷贝消息到 FIFO */
hfdcan->Instance->TXBAR = ((uint32_t)1 << PutIndex);
```

42.1.9 FDCAN 寄存器

接下来，我们介绍一下本章需要用到的一些比较重要的寄存器。

- **FDCAN 的控制寄存器（FDCAN_CCCR）**

FDCAN 的控制寄存器（FDCAN_CCCR）各位描述如图 42.1.9.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NISO	TXP	EFBI	PHXD	Res.	Res.	BRSE	FDOE	TEST	DAR	MON	CSR	CSA	ASM	CCE	INIT
rw	rw	rw	rw			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 42.1.9.1 寄存器 FDCAN_CCCR 各位描述

该寄存器的详细描述，请参考《STM32H7xx 参考手册_V3（中文版）.pdf》55.4.6 节，这里我们仅介绍几个重要的位：

INIT 位，该位用来控制初始化请求。软件对该位清 0，可使 CAN 从初始化模式进入正常工作模式。软件对该位置 1 可使 CAN 从正常工作模式进入初始化模式。所以我们在 CAN 初始化的时候，先要设置该位为 1，然后进行初始化，之后再设置该位为 0，让 CAN 进入正常工作模式。

FDOE 位，此位用来设置是否使用 CAN FD 功能，如果使用 CAN FD 功能的话就将此位置 1，此位清零的话就关闭 CAN FD 功能。

DAR 位，此位用来设置是否使能自动重传机制，如果要使用自动重传的话就将此位置 0，此位置 1 就关闭自动重传，经过作者测试，如果 FDCAN 要工作在传统 CAN 模式的话一定要关闭自动重传机制，否则通信会失败!!!

MON 位，此位设置是否使能总线监视模式，此位置 1 的话就会使能总线监视模式，此位清零关闭总线监视模式。总线监视模式用来检测 CAN 总线的通信状况。

TEST 位，此位用来设置 FDCAN 工作在正常模式还是测试模式，如果要工作在正常模式的话此位清零，如果要使用测试模式的话就将此位置 1。

- **FDCAN 标称位时序寄存器（FDCAN_NBTP）**

该寄存器用于设置分频、t_{BS1}、t_{BS2} 以及 t_{SJW} 等非常重要的参数，直接决定了 CAN 的波特率，该寄存器各位描述如图 42.1.9.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
NSJW[6:0]							NBRP[8:0]								
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NTSEG1[7:0]								Res.	TSEG2[6:0]						
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:25 **NSJW**：标称（重新）同步跳转宽度 (Nominal (Re)Synchronization Jump Width)
有效值为 1 到 127。实际上，硬件会将该值解析为编程值加 1。
这些位受写保护 (P)，仅当 CCCR 寄存器的位 1 [CCE] 和位 0 [INIT] 置“1”时，才能进行写访问。

位 24:16 **NBRP**：比特率预分频器 (Bit Rate Prescaler)
生成位时间片时将振荡器频率除以的值。位时间为该时间片的倍数。有效值为 0 到 511。实际上，硬件会将该值解析为编程值加 1。
这些位受写保护 (P)，仅当 CCCR 寄存器的位 1 [CCE] 和位 0 [INIT] 置“1”时，才能进行写访问。

位 15:8 **NTSEG1**：采样点之前的标称时间段 (Nominal time segment before sample point)
有效值为 1 到 255。实际上，硬件会将该值解析为编程值加 1。
这些位受写保护 (P)，仅当 CCCR 寄存器的位 1 [CCE] 和位 0 [INIT] 置“1”时，才能进行写访问。

位 7 保留

位 6:0 **TSEG2**：采样点之后的标称时间段 (Nominal time segment after sample point)
有效值为 1 到 127。实际上，硬件会将该值解析为编程值加 1。

图 42.1.9.2 寄存器 FDCAN_NBTP 各位描述

- **FDCAN 测试寄存器（FDCAN_TEST）**

FDCAN 测试寄存器（FDCAN_TEST），该寄存器各位描述如图 42.1.9.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RX	TX[1:0]		LBCK	Res.	Res.	Res.	Res.
r	r	r	r	r	r	r	r	rw	rw	rw	rw	r	r	r	r

位 31:8 保留

位 7 **RX**：接收引脚 (Receive Pin)
监视发送引脚 FDCANx_RX 的实际值
0：CAN 总线为显性（FDCANx_RX = “0”）
1：CAN 总线为隐性（FDCANx_RX = “1”）

位 6:5 **TX**：发送引脚控制 (Control of Transmit Pin)
00：复位值，FDCANx_TX 由 CAN 内核控制，会在 CAN 位时间结束时更新
01：可在引脚 FDCANx_TX 监控采样点
10：引脚 FDCANx_TX 上为显性（“0”）电平
11：引脚 FDCANx_TX 上为隐性（“1”）电平

位 4 **LBCK**：环回模式（Loop Back mode）
0：复位值，禁止环回模式
1：使能环回模式（请参见*测试模式*）

位 3:0 保留

图 42.1.9.3 FDCAN 测试模式寄存器

该寄存器我们重点介绍一个 LBCK 位，此位用来使能回测模式，此位置 1 的时候就会使能 FDCAN 的回测模式，一共有两种回环模式：外部回环模式和内部回环模式。

外部回环模式

要进入外部回环模式需要将 FDCAN_CCCR 的 TEST 和 FDCAN_TEST 的 LBCK 这两个位

置 1。在外部回环模式下，FDCAN 把发送的报文当做接收的报文并保存（如果可以通过接收过滤）在接收 FIFO 里。也就是说外部回环模式是一个自发自收的模式，如图 42.1.9.4 所示：

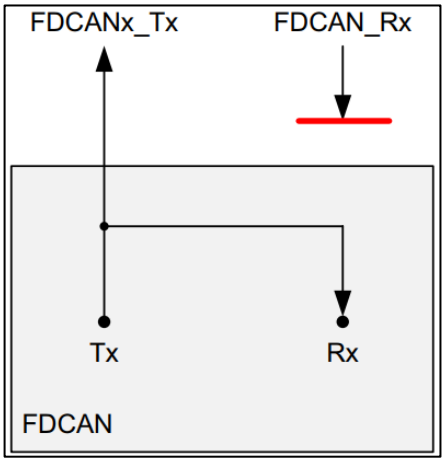


图 42.1.9.4 FDCAN 外部回环模式

外部回环模式提供了硬件自测。为了避免外部的影响，在回环模式下 FDCAN 内核忽略确认错误（在数据/远程帧的确认位时刻，不检测是否有显性位）。在回环模式下，FDCAN 在内部把 Tx 输出回馈到 Rx 输入上，而完全忽略 Rx 引脚的实际状态。发送的报文可以在 Tx 引脚上检测到。这里的外部回环模式就是以前 STM32 的回环模式。

内部回环模式

如果要使用内部回环模式的话，需要将 FDCAN_CCCR 的 TEST、FDCAN_TEST 的 LBCK 和 FDCAN_CCCR 的 MON 这三个位都置 1。内部回环模式如图 42.1.9.5 所示：

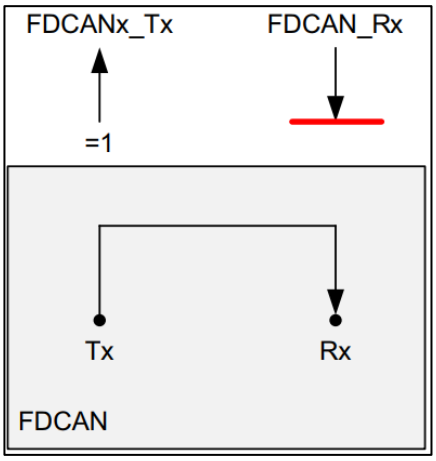


图 42.1.9.5 内部回环模式

从图 42.1.9.5 可以看出，内部回环模式下 Tx 引脚输出显性电平，因此测试的时候 Tx 引脚不会输出报文。外部回环模式下 Tx 是会输出报文的。这个模式可以用于热机自测，测试的时候不会影响到 CAN 总线上的正常数据收发。

- **FDCAN 发送 buffer 请求寄存器(FDCAN_TXBAR)**

  该寄存器各位描述如图 42.1.9.6 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
AR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
AR[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:0 **AR**：添加请求 (Add Request)

每个发送缓冲区都有自己的添加请求位。写入“1”会将相应的添加请求位置 1；写入“0”没有影响。这样，主机通过对 TXBAR 进行一次写操作便可为多个发送缓冲区设置发送请求。仅会为通过 TXBC 配置的发送缓冲区设置 TXBAR 位。如果没有运行发送扫描，这些位会立即复位，否则在发送扫描过程完成之前，这些位保持置 1 状态。

0：未添加发送请求

1：已添加发送请求

图 42.1.9.6 寄存器 FDCAN_TXBAR 各位描述

此寄存器用来设置 FDCAN 的哪个发送 buffer 可以发送数据，FDCAN 有 32 个发送 buffer，我们发送数据的时候就需要将所要发送的数据拷贝到这些 buffer 中，拷贝完成以后就需要标记此 buffer 可以发送。

- **FDCAN 接收 buffer 元素大小寄存器(FDCAN_RXESC)**

FDCAN 接收 buffer 元素大小寄存器(FDCAN_RXESC)，该寄存器各位描述如图 42.1.9.7 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	RBDS[2:0]			Res.	F1DS[2:0]			Res.	F0DS[2:0]		
					r	r	r		r	r	r		r	r	r

位 31:11 保留

位 10:8 **RBDS**：接收缓冲区数据字段大小 (Rx Buffer Data Field Size)

000：8 字节数据字段

001：12 字节数据字段

010：16 字节数据字段

011：20 字节数据字段

100：24 字节数据字段

101：32 字节数据字段

110：48 字节数据字段

111：64 字节数据字段

图 42.1.9.7 寄存器 FDCAN_RXESC 各位描述

该寄存器用来设置接收 buffer 的数据元素大小，其实就是设置 FDCAN 接收帧数据大小，因为本章没有使用 CAN FD 功能，因此一帧数据最大只能是 8 个字节。如果使用 CAN FD 功能的话最大就可以设置到 64 个字节。

- **FDCAN 发送 buffer 元素大小符寄存器(FDCAN_TXESC)**

该寄存器和上面的 FDCAN_RXESC 一样，只不过该寄存器要设置的是发送帧数据大小。

42.2 硬件设计

1. 例程功能

本章包括两个实验，分别为《传统 CAN 模式实验》和《FDCAN 模式实验》。这两个实验的功能和现象基本相同，唯一的区别在于使用的通信模式不同。在实验中，通过 KEY_UP 按键（即 WK_UP 按键）选择 CAN 的工作模式（正常模式/回环模式），然后通过 KEY0 控制数据发送，接着查询是否有数据接收到，假如接收到数据，就将接收到的数据显示在 LCD 模块上。如果是回环模式，我们不需要 2 个开发板。如果是正常模式，我们就需要 2 个阿波罗开发板，

并且将他们的 CAN 接口对接起来，然后一个开发板发送数据，另外一个开发板将接收到的数据显示在 LCD 模块上。

2. 硬件资源

1） LED 灯
LED0 - PB1
2） 独立按键
KEY0 - PH3
KEY_UP - PA0
3） TFTLCD 模块
4） STM32 自带 CAN 控制器
5） CAN 收发芯片 SIT1042T/TPT1051V

3. 原理图

STM32 有 CAN 的控制器，但要实现 CAN 通讯的差分电平，我们还需要借助外围电路来实现，根据我们需要实现的程序功能，我们设计电路原理如下：

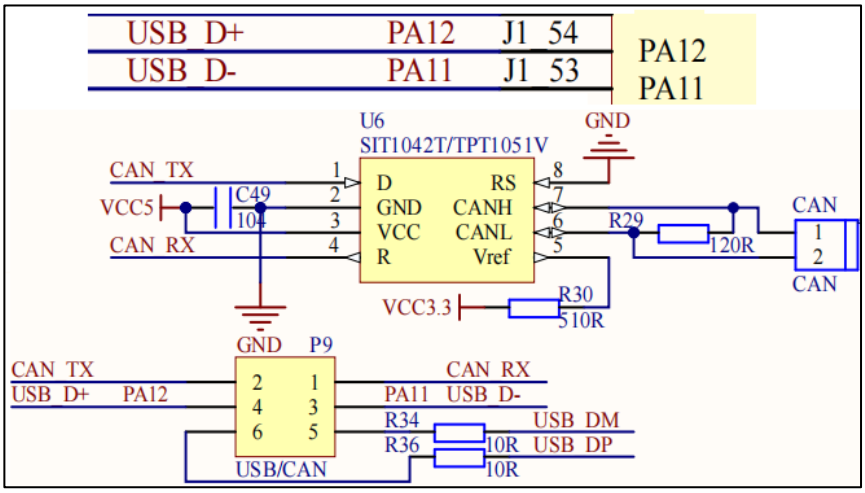


图 42.2.3.1 CAN 连接原理设计

从上图可以看出：STM32H7 的 CAN 通过 P9 的设置，连接到 SIT1042T/TPT1051V 收发芯片（支持 FDCAN），然后通过接线端子（CAN）同外部的 CAN 总线连接。图中可以看出，在阿波罗 STM32H743 开发板上面是带有 120Ω 的终端电阻的，如果我们的开发板不是作为 CAN 的终端的话，需要把这个电阻去掉，以免影响通信。另外，需要注意：CAN 和 USB 共用了 PA11 和 PA12，所以他们不能同时使用。

这里还要注意，我们要设置好开发板上 P9 排针的连接，通过跳线帽将 PA11 和 PA12 分别连接到 CAN_RX 和 CAN_TX 上面，如图 42.2.3.2 所示。

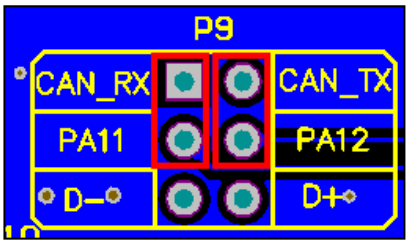


图 42.2.3.2 CAN 实验需要跳线连接的位置

最后，我们用 2 根导线将两个开发板 CAN 端子的 CAN_L 和 CAN_L，CAN_H 和 CAN_H 连接起来。这里注意不要接反了（CAN_L 接 CAN_H），接反了会导致通讯异常!!

42.3 程序设计

42.3.1 FDCAN 的 HAL 库驱动

CAN 在 HAL 库中的驱动代码在 stm32h7xx_hal_fdcan.c 文件（及其头文件）中。

1. HAL_FDCAN_Init 函数

要使用一个外设首先要对它进行初始化，所以先看 FDCAN 的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_FDCAN_Init(FDCAN_HandleTypeDef *hfdcan);
```

- **函数描述：**

  用于 FDCAN 控制器的初始化。

- **函数形参：**

  形参 1 是 FDCAN 的控制句柄，结构体类型是 FDCAN_HandleTypeDef，其定义如下：

```
typedef struct
{
  FDCAN_GlobalTypeDef          *Instance;        /* FDCAN 控制寄存器基地址 */
  TT_TypeDef                   *ttcan;           /* TT 寄存器基地址 */
  FDCAN_InitTypeDef             Init;            /* 初始化所需参数 */
  FDCAN_MsgRamAddressTypeDef    msgRam;          /* 消息内存块 */
  uint32_t                      LatestTxFifoQRequest;/* 缓冲区索引 */
  __IO HAL_FDCAN_StateTypeDef   State;           /* CAN 通讯状态 */
  HAL_LockTypeDef               Lock;            /* 锁定对象 */
  __IO uint32_t                 ErrorCode;       /* CAN 通讯结果编码 */
} FDCAN_HandleTypeDef;
```

该结构体除了 State，Lock，和 ErrorCode 三个 HAL 库处理状态过程变量之外，只有四个成员变量需要我们外部设置。

1）Instance：指向 FDCAN 寄存器基地址。这里我们使用 FDCAN1，设置为 FDCAN1 即可。

2）ttcan：ttcan 寄存器基地址，是跟时间触发有关的，本章我们用不到。

3）msgRam：这个用来保存消息 RAM 的地址。

4）Init：它是 FDCAN_InitTypeDef 结构体类型，该结构体定义为：

```
typedef struct
{
  uint32_t FrameFormat;                  /* 帧格式 */
  uint32_t Mode;                         /* FDcan 操作模式 */
  FunctionalState AutoRetransmission;    /* 禁止/使能自动重传模式 */
  FunctionalState TransmitPause;         /* 禁止/使能传输暂停功能 */
  FunctionalState ProtocolException;     /* 禁止/使能协议异常处理 */
  uint32_t NominalPrescaler;             /* 指定振荡器频率的值 */
  uint32_t NominalSyncJumpWidth;         /* 指定 FDCAN 最大时间量 */
  uint32_t NominalTimeSeg1;              /* 比特段 1 中的时间量子数 */
  uint32_t NominalTimeSeg2;              /* 比特段 2 中的时间量子数 */
  uint32_t DataPrescaler;                /* 指定振荡器频率的值 */
  uint32_t DataSyncJumpWidth;            /* 指定 FDCAN 最大时间量 */
  uint32_t DataTimeSeg1;                 /* 比特段 1 中的时间量子数 */
  uint32_t DataTimeSeg2;                 /* 比特段 2 中的时间量子数 */
  uint32_t MessageRAMOffset;             /* 消息 RAM 起始地址 */
  uint32_t StdFiltersNbr;                /* 标准消息 ID 筛选器的数目 */
  uint32_t ExtFiltersNbr;                /* 扩展消息 ID 筛选器的数目 */
  uint32_t RxFifo0ElmtsNbr;              /* Rx FIFO0 元素的个数 */
  uint32_t RxFifo0ElmtSize;              /* Rx FIFO0 元素的数据字段大小 */
  uint32_t RxFifo1ElmtsNbr;              /* Rx FIFO0 元素的个数 */
  uint32_t RxFifo1ElmtSize;              /* Rx FIFO0 元素的数据字段大小 */
  uint32_t RxBuffersNbr;                 /* 专用 Rx 缓冲区元素的数量 */
  uint32_t RxBufferSize;                 /* 专用 Rx 缓冲区元素的大小 */
  uint32_t TxEventsNbr;                  /* Tx Event FIFO 元素个数 */
  uint32_t TxBuffersNbr;                 /* 专用 Tx 缓冲区的数量 */
  uint32_t TxFifoQueueElmtsNbr;          /* Tx FIFO 的 Tx 缓冲区数量 */
```

```
  uint32_t TxFifoQueueMode;                /* Tx FIFO 队列模式选择 */
  uint32_t TxElmtSize;                     /* Tx 元素中的数据字段大小 */
} FDCAN_InitTypeDef;
```

这个结构体看起来成员变量比较多，实际上参数可以分为两类。前面 16 个参数是用来设置完成 FDCAN 的通用设置，比如 FrameFormat 用来设置 FDCAN 是工作在 CAN FD 模式还是传统的 CAN 模式。Mode 是设置 FDCAN 工作模式的参数，本实验需要用到内部回环模式（FDCAN_MODE_INTERNAL_LOOPBACK，数值 3）和常规模式（FDCAN_MODE_NORMAL 数值是 0）。

其他设置波特率相关的参数 NominalPrescaler，NominalSyncJumpWidth，NominalTimeSeg1 和 NominalTimeSeg2 分别用来设置波特率分频器，重新同步跳跃宽度以及时间段 1 和时间段 2 占用的时间单元数。注意：NominaPrescaler，NominalTimeSeg1 和 NominalTimeSeg2 这三个参数分别对应寄存器：FDCAN_NBTP 的 NTSEG1[7:0]、NTSEG2[6:0]和 NBRP[8:0]的值加 1，即：

NominalPrescler = NBRP[8:0] + 1
NominalTimeSeg1 = NTSEG1[7:0] + 1
NominalTimeSeg2 = NTSEG2[6:0] + 1

因此，CAN 通信波特率的计算公式可以简化为：

$$baudrate = \frac{f_{\text{fdcan_tq_ck}}}{(NominalTimeSeg1 + NominalTimeSeg2 + 1) \times NominalPrescaler}$$

与数据段通信速率相关的参数 DataPrescaler，DataSyncJumpWidth，DataTimeSeg1 和 DataTimeSeg2 分别用来设置数据段速率分频器，重新同步跳跃宽度以及时间段 1 和时间段 2 占用的时间单元数。注意：DataTimeSeg1 和 DataTimeSeg2，DataPrescaler 这三个参数分别对应寄存器：FDCAN_DBTP 的 DTSEG1[4:0]、DTSEG2[3:0]和 DBRP[4:0]的值加 1，即：

DataPrescaler= DBRP[4:0] + 1
DataTimeSeg1= DTSEG1[4:0] + 1
DataTimeSeg2= DTSEG2[3:0]+ 1

因此，FDCAN 的数据段通信速率的计算公式可以简化为：

$$baudrate = \frac{f_{fdcan_tq_ck}}{(DataTimeSeg1 + DataTimeSeg2 + 1) \times DataPrescaler}$$

后面 Tx 和 Rx 开头的成员变量用来设置 FDCAN 的收发 FIFO，重点是 RxFifo0ElmtSize 和 TxElmtSize 这两个成员变量，这两个成员变量设置收发 FIFO 的元素大小，也就是收发一帧数据大小，我们用的是传统 CAN 模式，因此最大只能是 8 字节，当我们使用的是 FDCAN 模式时，最大能到 64 字节。

- **函数返回值：**

HAL_StatusTypeDef 枚举类型的值，有 4 个，分别是 HAL_OK 表示成功，HAL_ERROR 表示错误，HAL_BUSY 表示忙碌，HAL_TIMEOUT 为超时。

调用初始化函数之后，同样我们需要重定义 HAL_FDCAN_MspInit 来初始化跟底层硬件相关的配置，我们后面编写初始化函数时用到。

2. HAL_FDCAN_ConfigFilter 函数

FDCAN 的接收过滤器是属于硬件，可以根据软件的设置，在接收报文的时候，可以过滤出符合过滤器配置条件的报文 ID，大大节省了 CPU 的开销，过滤器配置函数定义如下：

```
HAL_StatusTypeDef HAL_FDCAN_ConfigFilter(FDCAN_HandleTypeDef *hfdcan,
                                         FDCAN_FilterTypeDef
                                         *sFilterConfig)
```

- **函数描述：**

用于配置 FDCAN 的接收过滤器。

- **函数形参：**

形参 1 是 FDCAN 的控制句柄指针，初始化函数已经介绍过它的结构了，这里不重复了。

形参 2 是过滤器的结构体，这个是根据 STM32 的 FDCAN 过滤器模式设置的一些配置参数，它的结构如下：

```
typedef struct
{
  uint32_t IdType;                  /* 过滤器标识符类型 */
```

```
  uint32_t FilterIndex;             /* 指定要初始化的筛选器 */
  uint32_t FilterType;              /* 过滤类型*/
  uint32_t FilterConfig;            /* 过滤配置 */
  uint32_t FilterID1;               /* 筛选器标识 1 */
  uint32_t FilterID2;               /* 筛选器标识 2 */
  uint32_t RxBufferIndex;           /* Rx 缓冲区的索引*/
  uint32_t IsCalibrationMsg;        /* 校准信息 */
} FDCAN_FilterTypeDef;
```

结构体一共有 8 个成员变量，第一个参数 IdType 设置 ID 类型，有两种：FDCAN_STANDARD_ID 和 FDCAN_EXTENDED_ID。本实验中使用 FDCAN_STANDARD_ID，即标准 ID。第 2 个至第 6 个是用来设置过滤器类型和过滤器的 32 位 id 以及 32 位 msak id。

- **函数返回值：**

  我们只关注 HAL_OK 的情况。

3. HAL_FDCAN_AddMessageToTxFifoQ 函数

使能 CAN 控制器以接入总线进行数据收发处理。

```
HAL_StatusTypeDef HAL_FDCAN_AddMessageToTxFifoQ(FDCAN_HandleTypeDef *hfdcan,
                                                FDCAN_TxHeaderTypeDef *pTxHeader,uint8_t *pTxData)
```

- **函数描述：**

  按需要配置完 FDCAN 相关参数后，使能 FDCAN 控制器进行发送和接收消息。

- **函数形参：**

  形参 1 为 hfdcan，指定要发送数据的 FDCAN 句柄。

  形参 2 为 FDCAN_TxHeaderTypeDef 类型，这是个结构体，结构体描述如下：

```
typedef struct
{
  uint32_t Identifier;                /* 标识符 */
  uint32_t IdType;                    /* 标识符类型 */
  uint32_t TxFrameType;               /* 要传输的消息帧类型 */
  uint32_t DataLength;                /* 数据长度 */
  uint32_t ErrorStateIndicator;       /* 错误状态指示器 */
  uint32_t BitRateSwitch;             /* 传输的 Tx 帧带/不带比特率交换 */
  uint32_t FDFormat;                  /* Tx 帧将以经典方式还是 FD 格式传输 */
  uint32_t TxEventFifoControl;        /* 事件 FIFO 控制 */
  uint32_t MessageMarker;             /* 消息标记 */
} FDCAN_TxHeaderTypeDef;
```

我们来看一下其中一些重要的成员变量：

Identifier 是用来设置消息 ID。

IdType 用来设置 ID 类型，有 FDCAN_STANDARD_ID 和 FDCAN_EXTENDED_ID 两种可选。

TxFrameType 设置要发送的帧类型，有 FDCAN_DATA_FRAME 和 FDCAN_REMOTE_FRAME 这两种类型。

DataLength 设置要发送的数据长度。

- **函数返回值：**

  我们只关注 HAL_OK 的情况。

FDCAN 的初始化配置步骤

1) **FDCAN 参数初始化（工作模式、波特率等）**

HAL 库通过调用 FDCAN 初始化函数 HAL_FDCAN_Init 完成对 FDCAN 参数初始化，详见例程源码。

注意：该函数会调用：HAL_FDCAN_MspInit 函数来完成对 FDCAN 底层的初始化，包括：FDCAN 以及 GPIO 时钟使能、GPIO 模式设置、中断设置等。

2) **开启 FDCAN 和对应管脚时钟，配置 FDCAN_TX 和 FDCAN_RX 的复用功能输出**

首先开启 FDCAN 的时钟，然后配置 FDCAN 相关引脚为复用功能。本实验中 FDCAN_TX 对应的是 PA12，FDCAN_RX 对应的是 PA11。他们的时钟开启方法如下：

```
__HAL_RCC_FDCAN_CLK_ENABLE();              /* 使能 FDCAN1 */
__HAL_RCC_GPIOA_CLK_ENABLE();              /* 开启 GPIOA 时钟 */
```

IO 口复用功能是通过函数 HAL_GPIO_Init 来配置的。

3) **设置滤波器**

HAL 库通过调用 HAL_FDCAN_ConfigFilter 完成 FDCAN 的滤波器相关参数初始化。

4) **FDCAN 数据接收和发送**

通过调用 HAL_FDCAN_AddMessageToTxFifoQ 函数进行发送消息。

通过调用 HAL_FDCAN_GetRxMessage 函数进行接收数据。

至此，FDCAN 就可以开始正常工作了。如果用到中断，就还需要进行中断相关的配置。本实验也提供 FDCAN 接收中断，详看例程源码，这里就不作介绍了。

42.3.2 程序流程图

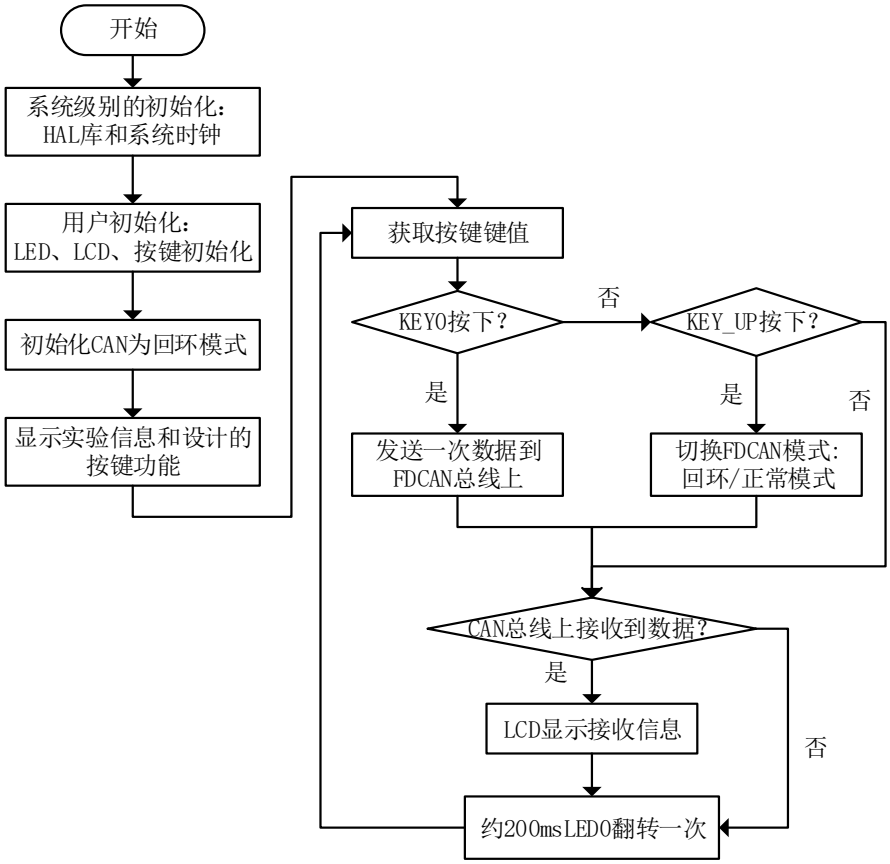


图 42.3.2.1 FDCAN 通讯实验程序流程图

42.3.3 程序解析

我们要使用 LED、LCD、按键这些功能，直接复制上一个 485 实验的代码，把 rs485 的代码从工程中移除，并在 Drivers/BSP 目录下新建一个 FDCAN 文件夹，与之前一样，新建 fdcan.c/fdcan.h 文件并把它们加入到工程中。

42.3.3.1 传统 CAN 模式实验解析

1. fdcan.c 函数

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。FDCAN 驱动相关源码包括两个文件：fdcan.c 和 fdcan.h。

我们根据《STM32H7xx 参考手册_V3（中文版）.pdf》第 56 章的内容，我们利用前面介绍

的 HAL 库函数来配置 FDCAN 的接收时钟及模式等参数，配置过滤器以使能硬件自动过滤功能，最后使能 FDCAN 以开始 FDCAN 控制器的工作，编写 FDCAN 初始化函数。

```
/**
 * @brief   FDCAN 初始化，波特率为 500KBit/s
 * @note    配置 FDCAN 的时钟源为 PLL1Q=200Mhz
 * @param   presc:分频值，取值范围 1~512
 *          tsjw:重新同步跳跃时间单元.范围:1~128
 *          ntsg1:取值范围 2~256
 *          ntsg2:取值范围 2~128
 *          mode: FDCAN_MODE_NORMAL,普通模式;FDCAN_MODE_EXTERNAL_LOOPBACK,回环模式;
 *          mode:CAN_MODE_NORMAL,普通模式;CAN_MODE_LOOPBACK,回环模式;
 * @retval 返回值:0,初始化 OK;
 *              其他,初始化失败;
 */
uint8_t fdcan_init(uint16_t presc, uint8_t tsjw, uint16_t ntsg1,
                                   uint8_t ntsg2, uint32_t mode)
{
    FDCAN_FilterTypeDef fdcan_filterconfig;
    HAL_FDCAN_DeInit(&g_fdcanx_handle);                     /* 先清除以前的设置 */
    g_fdcanx_handle.Instance = FDCAN1;
    g_fdcanx_handle.Init.FrameFormat = FDCAN_FRAME_CLASSIC;/* 传统模式 */
    g_fdcanx_handle.Init.Mode = mode;                       /* 模式设置  */
    g_fdcanx_handle.Init.AutoRetransmission=DISABLE;
    /* 关闭自动重传！传统模式下一定要关闭！！！  */
    g_fdcanx_handle.Init.TransmitPause = DISABLE;       /* 关闭传输暂停 */
    g_fdcanx_handle.Init.ProtocolException = DISABLE;   /* 关闭协议异常处理 */
    g_fdcanx_handle.Init.NominalPrescaler = presc;      /* 分频系数 */
    g_fdcanx_handle.Init.NominalSyncJumpWidth = tsjw;   /* 重新同步跳跃宽度 */
    g_fdcanx_handle.Init.NominalTimeSeg1 = ntsg1;       /* tsg1 范围：2~256 */
    g_fdcanx_handle.Init.NominalTimeSeg2 = ntsg2;       /* tsg2 范围：2~128 */
    g_fdcanx_handle.Init.MessageRAMOffset = 0;          /* 信息 RAM 偏移 */
    g_fdcanx_handle.Init.StdFiltersNbr = 0;             /* 标准信息 ID 滤波器编号 */
    g_fdcanx_handle.Init.ExtFiltersNbr = 0;             /* 扩展信息 ID 滤波器编号 */
    g_fdcanx_handle.Init.RxFifo0ElmtsNbr = 1;           /* 接收 FIFO0 元素编号 */
    g_fdcanx_handle.Init.RxFifo0ElmtSize = FDCAN_DATA_BYTES_8;
    /* 接收 FIFO0 元素大小：8 字节 */
    g_fdcanx_handle.Init.RxBuffersNbr = 0;              /* 接收 FIFO0 元素编号 */
    g_fdcanx_handle.Init.TxEventsNbr = 0;               /* 发送事件编号 */
    g_fdcanx_handle.Init.TxBuffersNbr = 0;              /* 发送缓冲编号 */
    g_fdcanx_handle.Init.TxFifoQueueElmtsNbr = 1;       /* 发送 FIFO 序列元素编号 */
    g_fdcanx_handle.Init.TxFifoQueueMode = FDCAN_TX_FIFO_OPERATION;
    /* 发送 FIFO 序列模式 */
    g_fdcanx_handle.Init.TxElmtSize = FDCAN_DATA_BYTES_8;     /* 发送大小:8 字节 */

    if (HAL_FDCAN_Init(&g_fdcanx_handle) != HAL_OK)
    {
        return 1;    /* 初始化 */
    }
    fdcan_filterconfig.IdType = FDCAN_STANDARD_ID;              /* 标准 ID */
    fdcan_filterconfig.FilterIndex = 0;                         /* 滤波器索引 */
    fdcan_filterconfig.FilterType = FDCAN_FILTER_MASK;          /* 滤波器类型 */
    fdcan_filterconfig.FilterConfig = FDCAN_FILTER_TO_RXFIFO0;
    /* 过滤器 0 关联到 FIFO0 */
    fdcan_filterconfig.FilterID1 = 0x0000;                      /* 32 位 ID */
    fdcan_filterconfig.FilterID2 = 0x0000;
    /* 如果 FDCAN 配置为传统模式的话，这里是 32 位掩码 */

    /* 过滤器配置 */
    if (HAL_FDCAN_ConfigFilter(&g_fdcanx_handle,&fdcan_filterconfig)!= HAL_OK)
    {
```

```
        return 2;                                               /* 滤波器初始化 */
    }

    HAL_FDCAN_Start(&g_fdcanx_handle);                          /* 开启 FDCAN */
    HAL_FDCAN_ActivateNotification(&g_fdcanx_handle,
                                   FDCAN_IT_RX_FIFO0_NEW_MESSAGE, 0);

    return 0;
}
```

调用 HAL_FDCAN_Init 后会调用 HAL_FDCAN_MspInit，我们重定义这个函数，在函数中初始化我们用于控制 FDCAN 的收发引脚：

```
void HAL_FDCAN_MspInit(FDCAN_HandleTypeDef *hfdcan)
{
   __HAL_RCC_FDCAN_FORCE_RESET();                   /* 复位 FDCAN1 */
   __HAL_RCC_FDCAN_RELEASE_RESET();                 /* 停止复位 */
}
```

初始化函数就编写完成了，我们要设置它的工作波特率为 500Kbps，设置工作模式为回环模式，参照各个参数的意义，我们最后用以下的配置来完成初始化设置：

```
    fdcan_init(10, 8, 31, 8, FDCAN_MODE_INTERNAL_LOOPBACK);
```

要与其它的 FDCAN 节点设备通讯，我们需要编写 FDCAN 相关的收发函数。首先是发送函数，发送报文是有 ID，所以我们在发送时需要设定 ID，故需要设计一个形参为 ID 号，我们利用 HAL 库的发送函数封装一个更方便我们使用的函数，代码如下：

```
/**
 * @brief       can 发送一组数据(固定格式:ID 为 0X12,标准帧,数据帧)
 * @param       len:数据长度(最大为 8)
 * @param       msg:数据指针,最大为 8 个字节.
 * @retval      返回值:0,成功;
 * @retval      其他,失败;
 */
uint8_t fdcan_send_msg(uint8_t *msg, uint32_t len)
{
    g_fdcanx_txheade.Identifier = 0x12;                         /* 32 位 ID */
    g_fdcanx_txheade.IdType = FDCAN_STANDARD_ID;                /* 标准 ID */
    g_fdcanx_txheade.TxFrameType = FDCAN_DATA_FRAME;            /* 使用标准帧 */
    g_fdcanx_txheade.DataLength = len;                          /* 数据长度 */
    g_fdcanx_txheade.ErrorStateIndicator = FDCAN_ESI_ACTIVE;
    g_fdcanx_txheade.BitRateSwitch = FDCAN_BRS_OFF;             /* 关闭速率切换 */
    g_fdcanx_txheade.FDFormat = FDCAN_CLASSIC_CAN;              /* 传统的 CAN 模式 */
    g_fdcanx_txheade.TxEventFifoControl = FDCAN_NO_TX_EVENTS; /* 无法送事件 */
    g_fdcanx_txheade.MessageMarker = 0;

    if (HAL_FDCAN_AddMessageToTxFifoQ(&g_fdcanx_handle, &g_fdcanx_txheade,
        msg) != HAL_OK) /* 发送消息 */
    {
        return 1;
    }
    return 0;
}
```

在 FDCAN 初始化时，我们对于过滤器的配置是不过滤任何报文 ID，也就是说可以接收全部报文。但是我们可以编写接收函数时，使用软件的方式过滤报文 ID，通过形参来跟接收到的报文 ID 进行匹配。接收函数代码具体如下：

```
/**
 * @brief       can 口接收数据查询
 * @param       buf:数据缓存区
 * @retval      返回值:0,无数据被收到;
 * @retval      其他,接收的数据长度;
 */
uint8_t fdcan_receive_msg(uint8_t *buf)
{
```

```
    if (HAL_FDCAN_GetRxMessage(&g_fdcanx_handle, FDCAN_RX_FIFO0,
        &g_fdcanx_rxheade, buf) != HAL_OK)    /* 接收数据 */
    {
        return 0;
    }
    return g_fdcanx_rxheade.DataLength>>16;
}
```

最后，我们可以把 fdcan_send_msg 函数加到 USMART 接口中，就可以方便地用串口来调试 CAN 接口了。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t key;
    uint8_t i = 0, t = 0;
    uint8_t cnt = 0;
    uint8_t canbuf[8];
    uint8_t rxlen = 0;
    uint8_t res;
    uint8_t mode = 1; /* CAN 工作模式: 0,普通模式; 1,环回模式 */

    sys_cache_enable();                         /* 打开 L1-Cache */
    HAL_Init();                                 /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);         /* 设置时钟,400Mhz */
    delay_init(400);                            /* 延时初始化 */
    usart_init(115200);                         /* 初始化 USART */
    usmart_dev.init(200);                       /* 初始化 USMART */
    mpu_memory_protection();                    /* 保护相关存储区域 */
    led_init();                                 /* 初始化 LED */
    sdram_init();                               /* 初始化 SDRAM */
    lcd_init();                                 /* 初始化 LCD */
    key_init();                                 /* 初始化按键 */
    /* CAN 初始化, 环回模式, 波特率 500Kbps */
    fdcan_init(10, 8, 31, 8, FDCAN_MODE_INTERNAL_LOOPBACK);

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "FDCAN TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 130, 200, 16, 16, "LoopBack Mode", RED);
    lcd_show_string(30, 150, 200, 16, 16, "KEY0:Send KEK_UP:Mode", RED);
    lcd_show_string(30, 170, 200, 16, 16, "Count:", BLUE);
    lcd_show_string(30, 190, 200, 16, 16, "Send Data:", BLUE);
    lcd_show_string(30, 250, 200, 16, 16, "Receive Data:", BLUE);

    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)           /* KEY0 按下,发送一次数据 */
        {
            for (i = 0; i < 8; i++)
            {
                canbuf[i] = cnt + i;    /* 填充发送缓冲区 */
                if (i < 4)
                {
                    /* 显示数据 */
                    lcd_show_xnum(30 + i * 32, 210, canbuf[i],
                                  3, 16, 0x80, BLUE);
                }
                else
                {
                    /* 显示数据 */
```

```
                lcd_show_xnum(30 + (i - 4) * 32, 230,
                              canbuf[i], 3, 16, 0x80, BLUE);
            }
        }
        res = fdcan_send_msg(canbuf, FDCAN_DLC_BYTES_8); /* 发送 8 个字节 */
        if (res)
        {
            /* 提示发送失败 */
            lcd_show_string(30 + 80, 190, 200, 16, 16, "Failed", BLUE);
        }
        else
        {
            /* 提示发送成功 */
            lcd_show_string(30 + 80, 190, 200, 16, 16, "OK    ", BLUE);
        }
    }
    else if (key == WKUP_PRES) /* WK_UP 按下, 改变 CAN 的工作模式 */
    {
        mode = !mode;
        if (mode == 0)
        {
            /* CAN 初始化, 普通(0)/回环(1)模式, 波特率 500Kbps */
            fdcan_init(10, 8, 31, 8, FDCAN_MODE_NORMAL);
        }
        else if (mode == 1)
        {
            /* CAN 初始化, 普通(0)/回环(1)模式, 波特率 500Kbps */
            fdcan_init(10, 8, 31, 8, FDCAN_MODE_LOOPBACK);
        }

        if (mode == 0)  /* 普通模式, 需要 2 个开发板 */
        {
            lcd_show_string(30, 130, 200, 16, 16, "Nnormal Mode ", RED);
        }
        else            /* 回环模式,一个开发板就可以测试了. */
        {
            lcd_show_string(30, 130, 200, 16, 16, "LoopBack Mode", RED);
        }
    }

    rxlen = fdcan_receive_msg(canbuf);  /* CAN ID = 0x12, 接收数据查询 */

    if (rxlen)                                  /* 接收到有数据 */
    {
        lcd_fill(30, 270, 160, 310, WHITE); /* 清除之前的显示 */

        for (i = 0; i < rxlen; i++)
        {
            if (i < 4)
            {
                lcd_show_xnum(30+i*32, 270, canbuf[i],
                              3, 16, 0X80, BLUE); /* 显示数据 */
            }
            else
            {
                lcd_show_xnum(30+(i-4)*32,290,canbuf[i],
                              3,16,0X80,BLUE);    /* 显示数据 */
            }
        }
    }

    t++;
    delay_ms(10);
    if (t == 20)
```

```
        {
            LED0_TOGGLE();                  /* 提示系统正在运行 */
            t = 0;
            cnt++;
            lcd_show_xnum(30 + 48, 150, cnt, 3, 16, 0X80, BLUE);  /* 显示数据 */
        }
    }
}
```

main 函数的执行过程与程序流程图是一致的，这里需要注意：在选择正常模式的情况下，要使两个开发板通信成功，必须保持一致的波特率。其它细节，参考光盘资料中的源码即可。

42.3.3.2 FDCAN 模式实验解析

该实验与传统 CAN 模式实验相似，我们将重点讲解两者在配置上的差异及对应的代码实现。

1、fdcan.c 函数

在本实验中，我们将 FDCAN 的帧格式设置成位速率可变的 FDCAN 模式，如下：

```
/**
 * @brief       FDCAN 初始化
 * @param       presc    : 分频值，取值范围 1~512;
 * @param       tsjw     : 重新同步跳跃时间单元.范围:1~128;
 * @param       ntsg1    : 时间段 1 的时间单元.取值范围 2~256;
 * @param       ntsg2    : 时间段 2 的时间单元.取值范围 2~128;
 * @param       mode     : FDCAN_MODE_NORMAL，普通模式;
                           FDCAN_MODE_INTERNAL_LOOPBACK，内部回环模式;
                           FDCAN_MODE_EXTERNAL_LOOPBACK，外部回环模式;
                           FDCAN_MODE_RESTRICTED_OPERATION，限制操作模式
                           FDCAN_MODE_BUS_MONITORING，总线监控模式
 * @note        以上 5 个参数，除了模式选择其余的参数在函数内部会减 1，所以，
                任何一个参数都不能等于 0
 *              FDCAN 其输入时钟频率为 Fpclk1 = 20Mhz
 *              波特率 = Fpclk1 / ((ntsg1 + ntsg2 + 1) * presc);
 *              我们设置 fdcan_init(10, 8, 31, 8, 1)，则 CAN 波特率为:
 *              20M / ((31 + 8 + 1) * 10) = 500Kbps
 * @retval      0,  初始化成功; 其他，初始化失败;
 */
uint8_t fdcan_init(uint16_t presc, uint8_t tsjw, uint16_t ntsg1, uint8_t ntsg2,
uint32_t mode)
{
    FDCAN_FilterTypeDef fdcan_filterconfig;

    HAL_FDCAN_DeInit(&g_fdcanx_handle);                     /* 先清除以前的设置 */
    g_fdcanx_handle.Instance = FDCAN1;
    g_fdcanx_handle.Init.FrameFormat = FDCAN_FRAME_FD_BRS;/*位速率变换 FDCAN 模式*/
    g_fdcanx_handle.Init.Mode = mode;                       /* 模式设置  */
    g_fdcanx_handle.Init.AutoRetransmission = ENABLE;
    /* 使能自动重传！传统模式下一定要关闭！！！ */
    g_fdcanx_handle.Init.TransmitPause = ENABLE;            /* 使能传输暂停 */
    g_fdcanx_handle.Init.ProtocolException = DISABLE;       /* 关闭协议异常处理 */
    /* FDCAN 中仲裁段位速率最高 1Mbit/s，数据段位速率最高 8Mbit/s */
    /* 数据段通信速率（仅 FDCAN 模式需配置）= 20M / (1 + dseg1 + dseg2)
                                        = 20M / (2 + 1 + 1) = 5 Mbit/s */
    g_fdcanx_handle.Init.DataPrescaler = 10;     /* 数据段分频系数范围:1~32  */
    g_fdcanx_handle.Init.DataSyncJumpWidth = 16;/* 数据段重新同步跳跃宽度 1~16 */
    g_fdcanx_handle.Init.DataTimeSeg1 = 2;       /* 数据段 dsg1 范围:1~32  5 */
    g_fdcanx_handle.Init.DataTimeSeg2 = 1;       /* 数据段 dsg2 范围:1~16  1 */

    /* 仲裁段通信速率（FDCAN 与传统 CAN 均需配置） = 20M / (1 + ntsg1 + ntsg2)
```

```
    = 20M / (31 + 8 + 1) = 500Kbit/s */
    g_fdcanx_handle.Init.NominalPrescaler = presc;       /* 分频系数 */
    g_fdcanx_handle.Init.NominalSyncJumpWidth = tsjw;    /* 重新同步跳跃宽度 */
    g_fdcanx_handle.Init.NominalTimeSeg1 = ntsg1;        /* tsg1 范围: 2~256 */
    g_fdcanx_handle.Init.NominalTimeSeg2 = ntsg2;        /* tsg2 范围: 2~128 */

    g_fdcanx_handle.Init.MessageRAMOffset = 0;          /* 信息 RAM 偏移 */
    g_fdcanx_handle.Init.StdFiltersNbr = 28;            /* 标准信息 ID 滤波器编号 */
    g_fdcanx_handle.Init.ExtFiltersNbr = 8;             /* 扩展信息 ID 滤波器编号 */
    g_fdcanx_handle.Init.RxFifo0ElmtsNbr = 1;           /* 接收 FIFO0 元素编号 */
    g_fdcanx_handle.Init.RxFifo0ElmtSize = FDCAN_DATA_BYTES_8;
    /* 接收 FIFO0 元素大小: 8 字节 */
    g_fdcanx_handle.Init.RxBuffersNbr = 0;              /* 接收 FIFO0 元素编号 */
    g_fdcanx_handle.Init.TxEventsNbr = 0;               /* 发送事件编号 */
    g_fdcanx_handle.Init.TxBuffersNbr = 0;              /* 发送缓冲编号 */
    g_fdcanx_handle.Init.TxFifoQueueElmtsNbr = 1;       /* 发送 FIFO 序列元素编号 */
    g_fdcanx_handle.Init.TxFifoQueueMode = FDCAN_TX_FIFO_OPERATION;
    /* 发送 FIFO 序列模式 */
    g_fdcanx_handle.Init.TxElmtSize = FDCAN_DATA_BYTES_8; /* 发送大小:8 字节 */

    if (HAL_FDCAN_Init(&g_fdcanx_handle) != HAL_OK)
    {
        return 1;    /* 初始化 */
    }

    /* 配置 CAN 过滤器 */
    fdcan_filterconfig.IdType = FDCAN_STANDARD_ID;      /* 标准 ID */
    fdcan_filterconfig.FilterIndex = 0;                 /* 滤波器索引 */
    fdcan_filterconfig.FilterType = FDCAN_FILTER_MASK; /* 滤波器类型 */
    fdcan_filterconfig.FilterConfig = FDCAN_FILTER_TO_RXFIFO0;
    /* 过滤器 0 关联到 FIFO0 */
    fdcan_filterconfig.FilterID1 = 0x0000;              /* 32 位 ID */
    fdcan_filterconfig.FilterID2 = 0x0000;
    /* 如果 FDCAN 配置为传统模式的话，这里是 32 位掩码 */

    /* 过滤器配置 */
    if (HAL_FDCAN_ConfigFilter(&g_fdcanx_handle, &fdcan_filterconfig)
    != HAL_OK)
    {
        return 2;                                       /* 滤波器初始化 */
    }
    /* 配置全局过滤器,拒收所有不匹配的标准帧或扩展帧 */
    if (HAL_FDCAN_ConfigGlobalFilter(&g_fdcanx_handle, FDCAN_REJECT,
    FDCAN_REJECT, FDCAN_FILTER_REMOTE, FDCAN_FILTER_REMOTE) != HAL_OK)
    {
        return 3;
    }
    /* 启动 CAN 外围设备 */
    if (HAL_FDCAN_Start(&g_fdcanx_handle) != HAL_OK)
    {
        return 4;
    }

    HAL_FDCAN_ActivateNotification(&g_fdcanx_handle,
    FDCAN_IT_RX_FIFO0_NEW_MESSAGE, 0);

    return 0;
}
```

fdcan_init 该函数需要特别注意的几个点，首先，需要将位速率转变为 FDCAN 模式，设置为 FDCAN_FRAME_FD_BRS。其次，需要通过设置以下几个成员变量来设置 FDCAN 的位速率：

- g_fdcanx_handler.Init.DataPrescaler = 10;
- g_fdcanx_handler.Init.DataSyncJumpWidth = 16;
- g_fdcanx_handler.Init.DataTimeSeg1 = 2;
- g_fdcanx_handler.Init.DataTimeSeg2 = 1;

并且也要通过设置代码中的几个成员变量来设置 FDCAN 的通信波特率：

本实验中我们设置通信波特率为 500Kbps，位速率设置为 5Mbit/s，关于计算公式详见第 42.1.6 小节。

过滤器的配置方面我们需要注意本实验我们使用的是位过滤模式（详见 42.1.7 小节）。并且 FilterID2 设置的是 0x0000，也就是支持接收任何 ID 发过来的数据。

与传统 CAN 模式相比，本实验的 fdcan_init 函数的主要区别在于使用了可变位速率的 FDCAN 模式，而其他配置则基本保持一致。

接下来是 FDCAN 的发送和接收函数。其中，FDCAN 的接收函数与传统 CAN 模式相同，仅有的区别在于传统 CAN 模式最大只能发送和接收 8 字节的数据，而 FDCAN 模式则可发送和接收最大 64 字节的数据。不过，在本实验中，我们均使用了 8 字节的数据，因此 FDCAN 的接收函数没有做出任何修改。而发送函数则有些许区别，下面我们来看一下 FDCAN 发送函数的区别说明。

```
/**
 * @brief       FDCAN 发送一组数据
 * @note        发送格式固定为: 标准 ID, 数据帧
 * @param       len     :数据长度,取值范围: FDCAN_DLC_BYTES_0 ~ FDCAN_DLC_BYTES_64
 * @param       msg     :数据指针,最大为 64 个字节
 * @retval      发送状态 0, 成功; 1, 失败;
 */
uint8_t fdcan_send_msg(uint8_t *msg, uint32_t len)
{
    g_fdcanx_txheade.Identifier = 0x12;                 /* 32 位 ID */
    g_fdcanx_txheade.IdType = FDCAN_STANDARD_ID;        /* 标准 ID */
    g_fdcanx_txheade.TxFrameType = FDCAN_DATA_FRAME;    /* 使用标准帧 */
    g_fdcanx_txheade.DataLength = len;                  /* 数据长度 */
    g_fdcanx_txheade.ErrorStateIndicator = FDCAN_ESI_ACTIVE;
    g_fdcanx_txheade.BitRateSwitch = FDCAN_BRS_ON;      /* 开启速率切换 */
    g_fdcanx_txheade.FDFormat = FDCAN_FD_CAN;           /* FDCAN 模式发送 */
    g_fdcanx_txheade.TxEventFifoControl = FDCAN_NO_TX_EVENTS;/* 无发送事件 */
    g_fdcanx_txheade.MessageMarker = 0;

    if (HAL_FDCAN_AddMessageToTxFifoQ(&g_fdcanx_handle, &g_fdcanx_txheade,
     msg) != HAL_OK) /* 发送消息 */
    {
        return 1;
    }

    return 0;
}
```

在 FDCAN 的发送函数中，与传统 CAN 模式相比较，不同之处在于开启了速率切换功能，以及将所发送的数据帧设置为位速率可变 FDCAN 数据帧格式。

以上是传统 CAN 模式与 FDCAN 模式的配置区别，其他配置基本保持一致。在 main.c 中的实现也是类似的，因此我们在此不再赘述。接下来，我们将直接观察两个实验的下载验证结果。

42.4 下载验证

本章包含两个实验，实验现象是完全相同的（仅模式以及速度有区别），所以我们在此统一进行讲解。在代码编译成功之后，我们通过下载代码到开发板上，得到如图 42.4.1 所示：

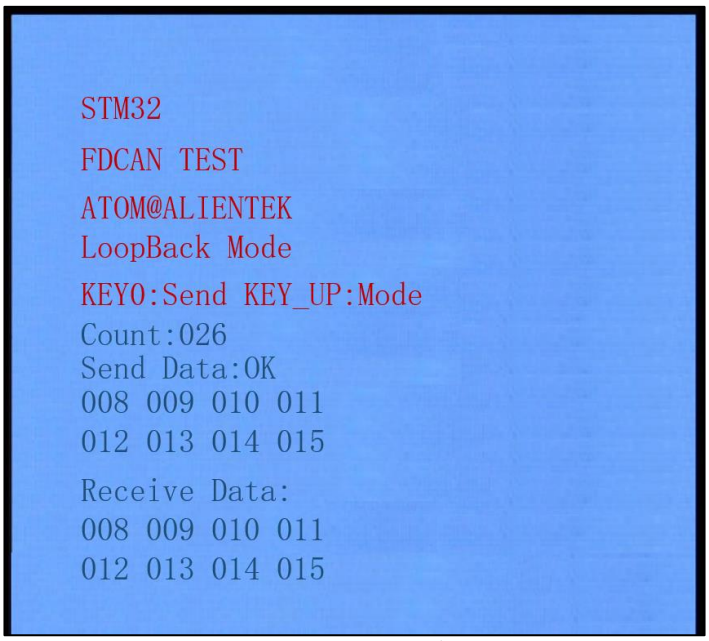


图 42.4.1 程序运行效果图

伴随 LED0 的不停闪烁，提示程序在运行。默认设置的回环模式，按下 KEY0 就可以在 LCD 模块上面看到自发自收的数据（如上图所示），如果我们选择正常模式（KEY_UP 按键切换），就必须连接两个开发板的 CAN 接口，然后就可以互发数据了。如图 42.4.2 和图 42.4.3 所示：

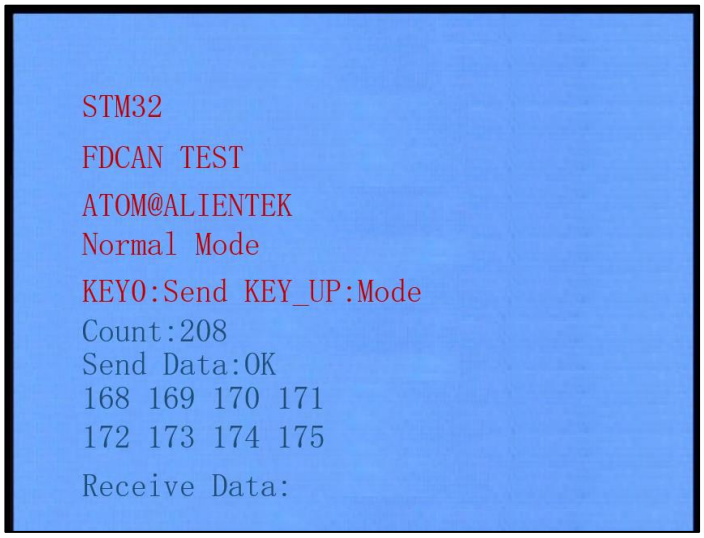


图 42.4.2 CAN 正常模式发送数据

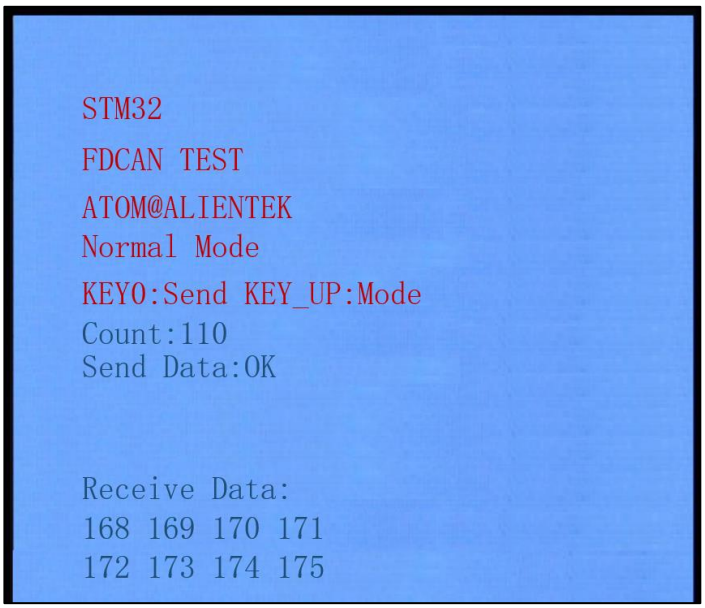


图 42.4.3 CAN 正常模式接收数据

图 42.4.2 来自开发板 A，发送了 8 个数据，图 42.4.3 来自开发板 B，收到了来自开发板 A 的个数据。另外，利用 USMART 测试的部分，我们这里就不做介绍了，大家可自行验证下。

第四十三章 触摸屏实验

本章，我们将介绍如何使用 STM32H743 来驱动触摸屏，STM32H743 本身并没有触摸屏控制器，但是它支持触摸屏，可以通过外接带触摸屏的 LCD 模块（比如正点原子 TFTLCD 模块），来实现触摸屏控制。在本章中，我们将向大家介绍 STM32 控制正点原子 TFTLCD 模块（包括电阻触摸与电容触摸），实现触摸屏驱动，最终实现一个手写板的功能。

本章分为如下几个小节：

43.1 触摸屏简介
43.2 硬件设计
43.3 程序设计
43.4 下载验证

43.1 触摸屏简介

触摸屏是在显示屏的基础上，在屏幕或屏幕上方分布一层与屏幕大小相近的传感器形成的组合器件。触摸和显示功能由软件控制，可以独立也可以组合实现，用户可以通过侦测传感器的触点再配合相应的软件实现触摸效果。目前最常用的触摸屏有两种：电阻式触摸屏与电容式触摸屏。下面，我们来分别介绍。

43.1.1 电阻式触摸屏

正点原子 2.4/2.8/3.5 寸 TFTLCD 模块自带的触摸屏都属于电阻式触摸屏，下面简单介绍下电阻式触摸屏的原理。

电阻触摸屏的主要部分是一块与显示器表面非常贴合的电阻薄膜屏，这是一种多层的复合薄膜，具体结构如下图 43.1.1.1 所示。

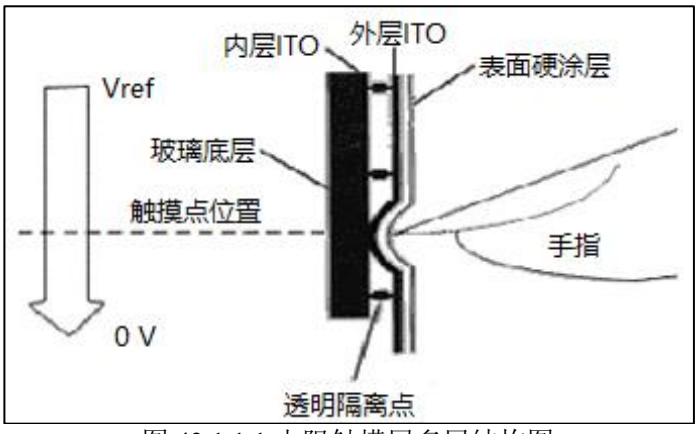


图 43.1.1.1 电阻触摸屏多层结构图

表面硬涂层起保护作用，主要是一层外表面硬化处理、光滑防擦的塑料层。玻璃底层用于支撑上面的结构，主要是玻璃或者塑料平板。透明隔离点用来分离开外层 ITO 和内层 ITO。ITO 层是触摸屏关键结构，是涂有铟锡金属氧化物的导电层。还有一个结构上图没有标出，就是 PET 层。PET 层是聚酯薄膜，处于外层 ITO 和表面硬涂层之间，很薄很有弹性，触摸时向下弯曲，使得 ITO 层接触。

当手指触摸屏幕时，两个 ITO 层在触摸点位置就有接触，电阻发生变化，在 X 和 Y 两个方向上产生电信号，然后送到触摸屏控制器，具体情况如下图 43.1.1.2 所示。触摸屏控制器侦测到这一接触并计算出 X 和 Y 方向上的 AD 值，简单来讲，电阻触摸屏将触摸点（X，Y）的物理位置转换为代表 X 坐标和 Y 坐标的电压值。单片机与触摸屏控制器进行通信获取到 AD 值，通过一定比例关系运算，获得 X 和 Y 轴坐标值。

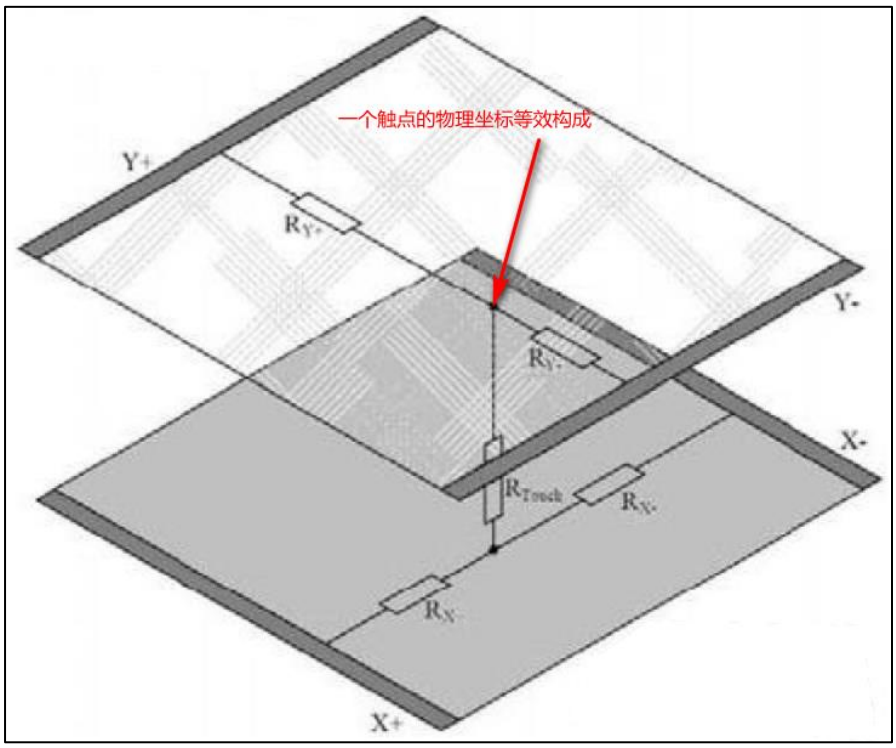


图 43.1.1.2 电阻式触摸屏的触点坐标结构

电阻触摸屏的优点：精度高、价格便宜、抗干扰能力强、稳定性好。

电阻触摸屏的缺点：容易被划伤、透光性不太好、不支持多点触摸。

从以上介绍可知，触摸屏都需要一个 AD 转换器，一般来说是需要一个控制器的。正点原子 TFTLCD 模块选择的是四线电阻式触摸屏，这种触摸屏的控制芯片有很多，包括：ADS7543、ADS7846、TSC2046、XPT2046 和 HR2046 等。这几款芯片的驱动基本上是一样的，也就是你只要写出了 XPT2046 的驱动，这个驱动对其他几个芯片也是有效的。而且封装也有一样的，完全 PIN-TO-PIN 兼容。所以在替换起来，很方便。

正点原子 TFTLCD 模块自带的触摸屏控制芯片为 XPT2046 或 HR2046。这里以 XPT2046 作为介绍。XPT2046 是一款 4 导线制触摸屏控制器，使用的是 SPI 通信接口，内含 12 位分辨率 125KHz 转换速率逐步逼近型 A/D 转换器。XPT2046 支持从 1.5V 到 5.25V 的低电压 I/O 接口。XPT2046 能通过执行两次 A/D 转换（一次获取 X 位置，一次获取 Y 位置）查出被按的屏幕位置，除此之外，还可以测量加在触摸屏上的压力。内部自带 2.5V 参考电压可以作为辅助输入、温度测量和电池监测模式之用，电池监测的电压范围可以从 0V 到 6V。XPT2046 片内集成有一个温度传感器。在 2.7V 的典型工作状态下，关闭参考电压，功耗可小于 0.75mW。

XPT2046 的驱动方法也是很简单，主要看懂 XPT2046 通信时序图，如下图 43.1.1.3 所示：

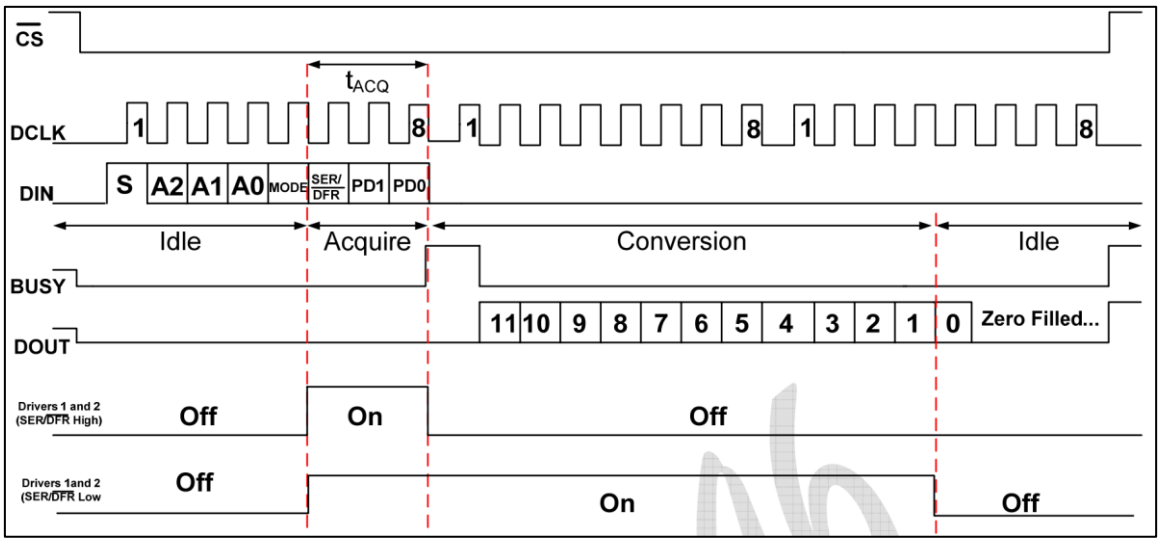


图 43.1.1.3 XPT2046 通信时序图

依照时序图，就可以很好写出这个通信代码，上图具体过程：拉低片选，选中器件→发送命令字→清除 BUSY→读取 16 位数据（高 12 位数据有效即转换的 AD 值）→拉高片选，结束操作。这里的难点就是需要搞清楚命令字该发送什么？只要搞清楚发送什么数值，就可以获取到 AD 值。命令字的详情如下图 43.1.1.4 所示：

位 7 (MSB)	位 6	位 5	位 4	位 3	位 2	位 1	位 0 (LSB)
S	A2	A1	A0	MODE	SER/$\overline{DFR}$	PD1	PD0

位	名称	功能描述
7	S	开始位。为 1 表示一个新的控制字节到来，为 0 则忽略 PIN 引脚上数据
6-4	A2-A0	通道选择位。参见表 1 和表 2
3	MODE	12 位/8 位转换分辨率选择位。为 1 选择 8 位为转换分辨率，为 0 选择 12 位分辨率
2	SER/$\overline{DFR}$	单端输入方式/ 差分输入方式选择位。为 1 是单端输入方式，为 0 是差分输入方式
1-0	PD1-PD0	低功率模式选择位。若为11，器件总处于供电状态;若为00，器件在变换之间处于低功率模式

图 43.1.1.4 命令字详情图

位 7，开始位，置 1 即可。位 3，为了提供精度，MODE 位清 0 选择 12 位分辨率。位 2，是进行工作模式选择，为了达到最佳性能，首选差分工作模式即该位清 0 即可。位 1-0 是功耗相关的，直接清 0 即可。而位 6-4 的值要取决于工作模式，在确定了差分功能模式后，通道选择位也确定了，如图 43.1.1.5 所示。

A2	A1	A0	+REF	-REF	YN	XP	YP	Y-位置	X-位置	Z_1-位置	Z_2-位置	驱动
0	0	1	YP	YN		+IN		测量				YP，YN
0	1	1	YP	XN		+IN				测量		YP，XN
1	0	0	YP	XN	+IN						测量	YP，XN
1	0	1	XP	XN			+IN		测量			XP，XN

图 43.1.1.5 差分模式输入配置图（SER/DFR=0）

从上图，就可以知道：当我们需要检测 Y 轴位置时，A2A1A0 赋值为 001；检测 X 轴位置时，A2A1A0 赋值为 101。结合前面对其他位的赋值，在 X，Y 方向与屏幕相同的情况下，命令字 0xD0 就是读取 X 坐标 AD 值，0x90 就是读取 Y 坐标的 AD 值。假如 X，Y 方向与屏幕相反，0x90 就是读取 X 坐标的 AD 值，而 0xD0 就是读取 Y 坐标的 AD 值。

关于这个芯片其他的功能，也可以参考芯片的 datasheet。

电阻式触摸屏就介绍到这里。

43.1.2 电容式触摸屏

现在几乎所有智能手机，包括平板电脑都是采用电容屏作为触摸屏，电容屏是利用人体感应进行触点检测控制，不需要直接接触或只需要轻微接触，通过检测感应电流来定位触摸坐标。正点原子 4.3/7 寸 TFTLCD 模块自带的触摸屏采用的是电容式触摸屏，下面简单介绍下电容式触摸屏的原理。

电容式触摸屏主要分为两种：

1、表面电容式电容触摸屏。

表面电容式触摸屏技术是利用 ITO(铟锡氧化物，是一种透明的导电材料)导电膜，通过电场感应方式感测屏幕表面的触摸行为进行。但是表面电容式触摸屏有一些局限性，它只能识别一个手指或者一次触摸。

2、投射式电容触摸屏。

投射电容式触摸屏是传感器利用触摸屏电极发射出静电场线。一般用于投射电容传感技术的电容类型有两种：自我电容和交互电容。

自我电容又称绝对电容，是最广为采用的一种方法，自我电容通常是指扫描电极与地构成的电容。在玻璃表面有用 ITO 制成的横向与纵向的扫描电极，这些电极和地之间就构成一个电容的两极。当用手或触摸笔触摸的时候就会并联一个电容到电路中去，从而使在该条扫描线上的总体的电容量有所改变。在扫描的时候，控制 IC 依次扫描纵向和横向电极，并根据扫描前后的电容变化来确定触摸点坐标位置。笔记本电脑触摸输入板就是采用的这种方式，笔记本电脑的输入板采用 X*Y 的传感电极阵列形成一个传感格子，当手指靠近触摸输入板时，在手指和传感电极之间产生一个小量电荷。采用特定的运算法则处理来自行、列传感器的信号来确定手指的位置。

交互电容又叫做跨越电容，它是在玻璃表面的横向和纵向的 ITO 电极的交叉处形成电容。交互电容的扫描方式就是扫描每个交叉处的电容变化，来判定触摸点的位置。当触摸的时候就会影响到相邻电极的耦合，从而改变交叉处的电容量，交互电容的扫面方法可以侦测到每个交叉点的电容值和触摸后电容变化，因而它需要的扫描时间与自我电容的扫描方式相比要长一些，需要扫描检测 X*Y 根电极。目前智能手机/平板电脑等的触摸屏，都是采用交互电容技术。

正点原子所选择的电容触摸屏，也是采用的是投射式电容屏（交互电容类型），所以后面仅以投射式电容屏作为介绍。

投射式电容触摸屏采用纵横两列电极组成感应矩阵，来感应触摸。以两个交叉的电极矩阵，即：X 轴电极和 Y 轴电极，来检测每一格感应单元的电容变化，如图 43.1.2.1 所示：

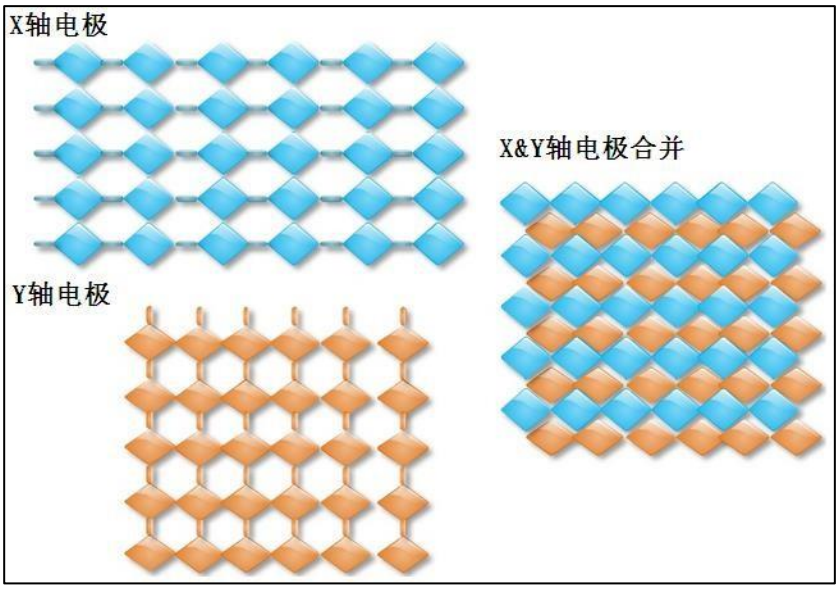


图43.1.2.1 投射式电容屏电极矩阵示意图

示意图中的电极，实际是透明的，这里是为了方便大家理解。图中，X、Y 轴的透明电极

电容屏的精度、分辨率与 X、Y 轴的通道数有关，通道数越多，精度越高。以上就是电容触摸屏的基本原理，接下来看看电容触摸屏的优缺点：

电容触摸屏的优点：手感好、无需校准、支持多点触摸、透光性好。

电容触摸屏的缺点：成本高、精度不高、抗干扰能力差。

这里特别提醒大家电容触摸屏对工作环境的要求是比较高的，在潮湿、多尘、高低温环境下面，都是不适合使用电容屏的。

电容触摸屏一般都需要一个驱动 IC 来检测电容触摸，正点原子的电容触摸屏使用的是 IIC 接口输出触摸数据的触摸芯片。正点原子 7'TFTLCD 模块的电容触摸屏，采用的是 15*10 的驱动结构（10 个感应通道，15 个驱动通道），采用的是 GT911/FT5206 作为驱动 IC。正点原子 4.3'TFTLCD 模块采用的驱动 IC 是：GT9xxx（GT9147/GT917S/GT911/GT1151/GT9271）,不同型号感应通道和驱动通道数量都不一样，详看数据手册，但是这些驱动 IC 驱动方式都类似，这里我们以 GT9147 为例给大家做介绍，其他的大家参考着学习即可。

GT9147 与 MCU 通过 4 根线连接：SDA、SCL、RST 和 INT。GT9147 的 IIC 地址，可以是 0X14 或者 0X5D，当复位结束后的 5ms 内，如果 INT 是高电平，则使用 0X14 作为地址，否则使用 0X5D 作为地址，具体的设置过程，请看：GT9147 数据手册.pdf 这个文档。本章我们使用 0X14 作为器件地址（不含最低位，换算成读写命令则是读：0X29，写：0X28），接下来，介绍一下 GT9147 的几个重要的寄存器。

1，控制命令寄存器（0X8040）

该寄存器可以写入不同值，实现不同的控制，我们一般使用 0 和 2 这两个值，写入 2，即可软复位 GT9147。在硬复位之后，一般要往该寄存器写 2，实行软复位。然后，写入 0，即可正常读取坐标数据（并且会结束软复位）。

2，配置寄存器组（0X8047~0X8100）

这里共 186 个寄存器，用于配置 GT9147 的各个参数，这些配置一般由厂家提供给我们（一个数组），所以我们只需要将厂家给我们的配置，写入到这些寄存器里面，即可完成 GT9147 的配置。由于 GT9147 可以保存配置信息（可写入内部 FLASH，从而不需要每次上电都更新配置），我们有几点注意的地方提醒大家：1，0X8047 寄存器用于指示配置文件版本号，程序写入的版本号，必须大于等于 GT9147 本地保存的版本号，才可以更新配置。2，0X80FF 寄存器用于存储校验和，使得 0X8047~0X80FF 之间所有数据之和为 0。3，0X8100 用于控制是否将配置保存在本地，写 0，则不保存配置，写 1 则保存配置。

3，产品 ID 寄存器（0X8140~0X8143）

这里总共由 4 个寄存器组成，用于保存产品 ID，对于 GT9147，这 4 个寄存器读出来就是：9，1，4，7 四个字符（ASCII 码格式）。因此，我们可以通过这 4 个寄存器的值，来判断驱动 IC 的型号，以便执行不同的初始化。

4，状态寄存器（0X814E）

该寄存器各位描述如表表 43.1.2.1 所示：

寄存器	bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
0X814E	buffer 状态	大点	接近有效	按键	有效触点个数			

表 43.1.2.1 状态寄存器各位描述

这里，我们仅关心最高位和最低 4 位，最高位用于表示 buffer 状态，如果有数据（坐标/按键），buffer 就会是 1，最低 4 位用于表示有效触点的个数，范围是：0~5，0，表示没有触摸，5 表示有 5 点触摸。最后，该寄存器在每次读取后，如果 bit7 有效，则必须写 0，清除这个位，否则不会输出下一次数据!! 这个要特别注意!!!

5，坐标数据寄存器（共 30 个）

这里共分成 5 组（5 个点），每组 6 个寄存器存储数据，以触点 1 的坐标数据寄存器组为例，如表 43.1.2.2 所示：

寄存器	bit7~0	寄存器	bit7~0
0X8150	触点 1 x 坐标低 8 位	0X8151	触点 1 x 坐标高 8 位
0X8152	触点 1 y 坐标低 8 位	0X8153	触点 1 y 坐标高 8 位

0X8154	触点 1 触摸尺寸低 8 位	0X8155	触点 1 触摸尺寸高 8 位

表 43.1.2.2 触点 1 坐标寄存器组描述

我们一般只用到触点的 x，y 坐标，所以只需要读取 0X8150~0X8153 的数据，组合即可得到触点坐标。其他 4 组分别是：0X8158、0X8160、0X8168 和 0X8170 等开头的 16 个寄存器组成，分别针对触点 2~4 的坐标。同样 GT9147 也支持寄存器地址自增，我们只需要发送寄存器组的首地址，然后连续读取即可，GT9147 会自动地址自增，从而提高读取速度。

GT9147 相关寄存器的介绍就介绍到这里，更详细的资料，请参考：GT9147 编程指南.pdf 这个文档。

GT9147 只需要经过简单的初始化就可以正常使用了，初始化流程：硬复位→延时 10ms→结束硬复位→设置 IIC 地址→延时 100ms→软复位→更新配置（需要时）→结束软复位。此时 GT9147 即可正常使用了。然后，我们不停的查询 0X814E 寄存器，判断是否有有效触点，如果有，则读取坐标数据寄存器，得到触点坐标。特别注意，如果 0X814E 读到的值最高位为 1，就必须对该位写 0，否则无法读到下一次坐标数据。

电容式触摸屏部分，就介绍到这里。

43.1.3 触摸控制原理

前面已经简单地介绍了电阻屏和电容屏的原理，并且知道了不同类型的触摸屏其实是屏幕+触摸传感器组成。那么这里就会有两组相互独立的参数：屏幕坐标和触摸坐标。要实现触摸功能，就是要把触摸点和屏幕坐标对应起来。

我们以 LCD 显示屏为例，我们知道屏幕的扫描方向是可以编程设定的，而触摸点，在触摸传感器安装好后，AD 值的变化向方向则是固定的，我们以最常见的屏幕坐标方向：先从左到右，再从上到下扫描为例，此时，屏幕坐标和触点 AD 的坐标有类似的规律：从坐标原点出发，水平方向屏幕坐标增加时，AD 值的 X 方向也增加；屏幕坐标的 Y 方向坐标增加，AD 值的 Y 方向也增加；坐标减少时对应的关系也类似，可以用图 43.1.3.1 的示意图来表示这种关系：

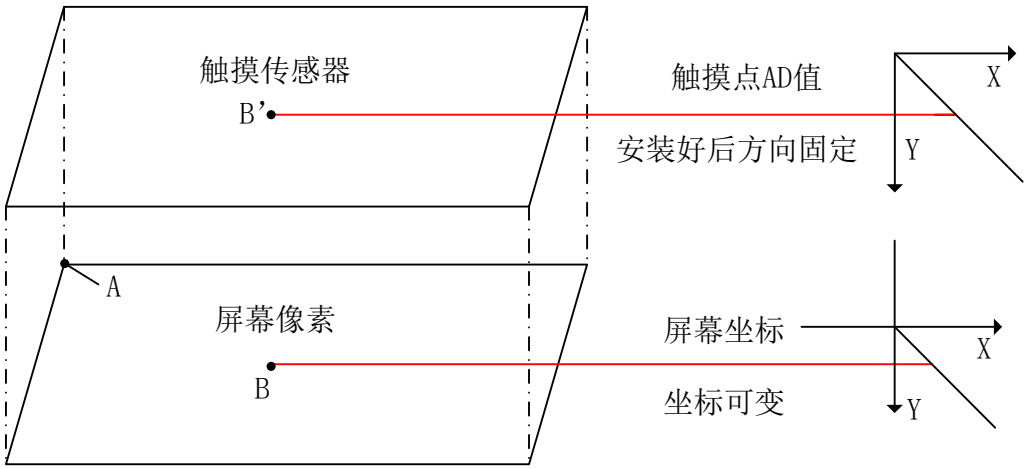


图 43.1.3.1 屏幕坐标和触摸坐标的一种对应关系

这里再来引入两个概念，物理坐标和逻辑坐标。物理坐标指触摸屏上点的实际位置，通常以液晶上点的个数来度量。逻辑坐标指这点被触摸时 A/D 转换后的坐标值。仍以图 43.1.3.1 为例，我们假定液晶最左上角为坐标轴原点 A，在液晶上任取一点 B（实际人手比像素点大得多，一次按下会有多个触点，此处取十字线交叉中心），B 在 X 方向与 A 相距 100 个点，在 Y 方向与 A 距离 200 个点，则这点的物理坐标 B 为（100，200）。如果我们触摸这一点时得到的 X 向 A/D 转换值为 200，Y 向 A/D 转换值为 400，则这点的逻辑坐标 B’为（200，400）。

需要特别说明的是，正点原子的电容屏的参数已经在出厂时由厂家调好，所以无需进行校准，而且可以直接读到转换后的触点坐标；对于电阻屏，请大家理解并熟记物理坐标和逻辑坐标逻辑上的对应关系，我们后面编程需要用到。

43.2 硬件设计

1. 例程功能

正点原子的触摸屏种类很多，并且设计了规格相对统一的接口。根据屏幕的种类不同，设置了相应的硬件 ID（正点原子自编 ID），可以通过软件判断触摸屏的种类。

本章实验功能简介：开机的时候先初始化 LCD，读取 LCD ID，随后，根据 LCD ID 判断是电阻触摸屏还是电容触摸屏，如果是电阻触摸屏，则先读取 24C02 的数据判断触摸屏是否已经校准过，如果没有校准，则执行校准程序，校准过后再进入电阻触摸屏测试程序，如果已经校准了，就直接进入电阻触摸屏测试程序。

如果是 4.3 寸电容触摸屏，则执行 GT9xxx 的初始化代码；如果是 7 寸电容触摸屏（仅支持新款 7 寸屏，使用 SSD1963+FT5206 方案），则执行 FT5206 的初始化代码，在初始化电容触摸屏完成后，进入电容触摸屏测试程序（电容触摸屏无需校准!!）。

电阻触摸屏测试程序和电容触摸屏测试程序基本一样，只是电容触摸屏支持最多 5 点同时触摸，电阻触摸屏只支持一点触摸，其他一模一样。测试界面的右上角会有一个清空的操作区域（RST），点击这个地方就会将输入全部清除，恢复白板状态。使用电阻触摸屏的时候，可以通过按 KEY0 来实现强制触摸屏校准，只要按下 KEY0 就会进入强制校准程序。

2. 硬件资源

1）LED 灯
LED：LED0 - PB1
2）独立按键
KEY0 - PH3
3）EEPROM AT24C02
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
5）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)

3. 原理图

LCD 模块的触摸屏（电阻触摸屏）总共有 5 根线与 STM32H743 连接，连接电路图如下图所示：

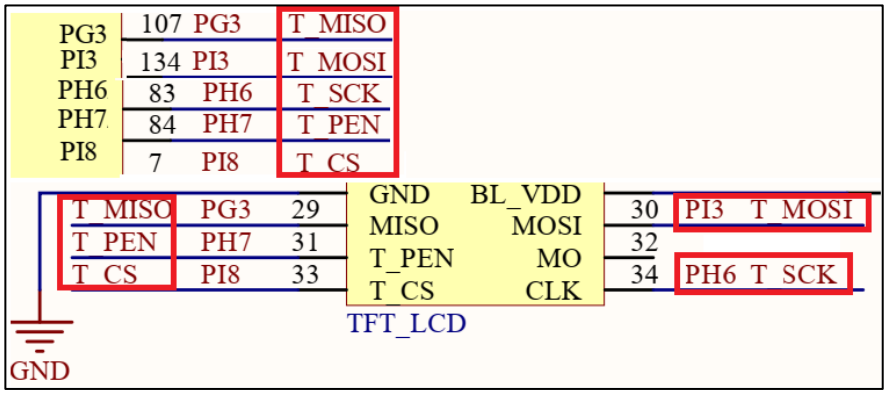


图 43.2.1 触摸屏与 STM32H743 的连接图

从图中可以看出，T_MISO、T_PEN、T_CS、T_MOSI 和 T_SCK 分别连接在 STM32H743 的：PG3、PH7、PI8、PI3 和 PH6 上。

如果是电容式触摸屏，我们的接口和电阻式触摸屏一样，只是没有用到五根线了，而是四根线，分别是：T_PEN(CT_INT)、T_CS(CT_RST)、T_SCK(CT_SCL)和 T_MOSI(CT_SDA)。其中：CT_INT、CT_RST、CT_SCL 和 CT_SDA 分别是 GT9147/FT5206 的：中断输出信号、复位信号，IIC 的 SCL 和 SDA 信号。这里，我们用查询的方式读取 GT9147/FT5206 的数据，对于 FT5206 没有用到中断信号（CT_INT），所以同 STM32H743 的连接，最少只需要 3 根线即

可，不过 GT9147 还需要用到 CT_INT 做 IIC 地址设定，所以需要 4 根线连接。

43.3 程序设计

43.3.1 HAL 库驱动

触摸芯片我们使用到的是 IIC 和 SPI 的驱动，这部分的时序分析可以参考之前 IIC/SPI 的章节，我们直接使用的是软件模拟的方式，所以只需要使用 HAL 库的驱动的 GPIO 操作部分。

触摸 IC 初始化步骤

1）初始化通信接口与其 IO（使能时钟、配置 GPIO 工作模式）

触摸 IC 用到的 GPIO 口，主要是 PG3、PH6、PH7、PI3 和 PI8，因为都是用软件模拟的方式，因此在这里我们只需使能 GPIOG、GPIOH 和 GPIOI 时钟即可。参考代码如下：

```
__HAL_RCC_GPIOG_CLK_ENABLE();            /* 使能 GPIOG 时钟 */
__HAL_RCC_GPIOH_CLK_ENABLE();            /* 使能 GPIOH 时钟 */
__HAL_RCC_GPIOI_CLK_ENABLE();            /* 使能 GPIOI 时钟 */
```

GPIO 模式设置通过调用 HAL_GPIO_Init 函数实现，详见本例程源码。

2）编写通信协议基础读写函数

通过参考时序图，在 IIC 驱动或 SPI 驱动基础上，编写基础读写函数。读写函数均以一字节数据进行操作。

3）参考触摸 IC 时序图，编写触摸 IC 读写驱动函数

根据触摸 IC 的读写时序进行编写触摸 IC 的读写函数，详见本例程源码。

4）编写坐标获取函数（电阻触摸屏和电容触摸屏）

查阅数据手册获得命令词(电阻触摸屏)/寄存器(电容触摸屏)，通过读写函数获取坐标数据，详见本例程源码。

43.3.2 程序流程图

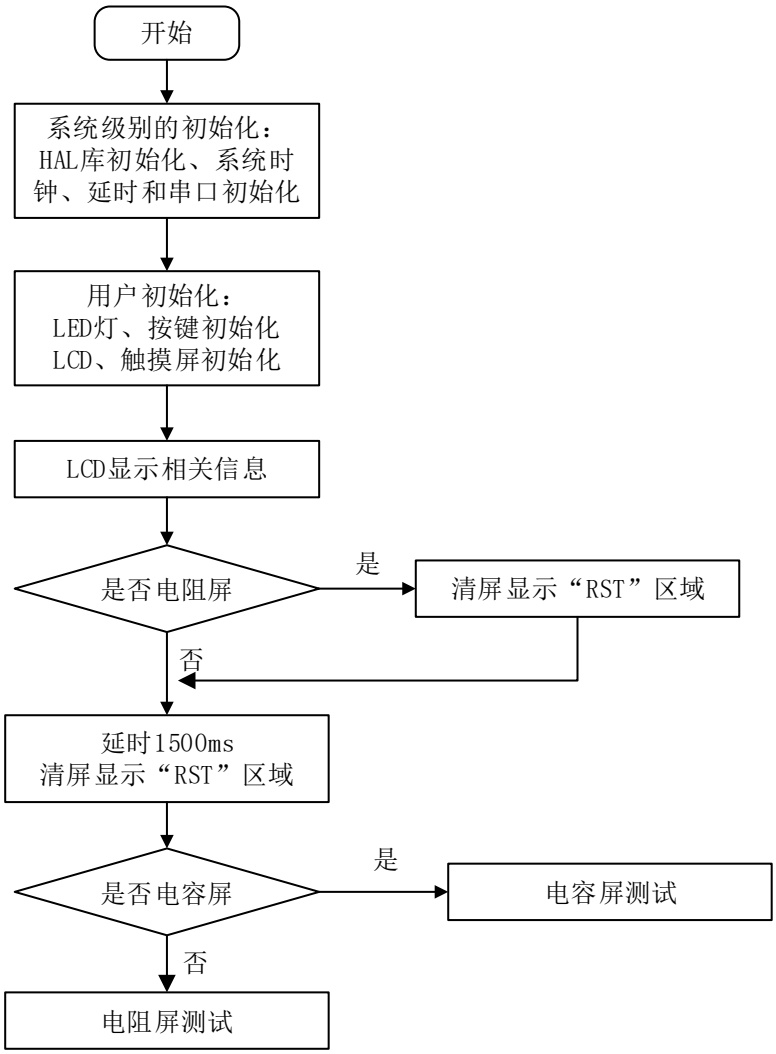


图 43.3.2.1 触摸屏实验流程图

43.3.3 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。TOUCH 驱动源码包括八个文件：ctiic.c、ctiic.h 、ft5206.c、ft5206.h、gt9xxx.c、gt9xxx.h、touch.c 和 touch.h。

由于正点原子的 TFTLCD 的型号很多，触摸控制这部分驱动代码根据不同屏幕搭载的触摸芯片驱动而有不同，在我们的屏幕上使用的是 LCD ID 来帮助软件上区分。为了解决多种驱动芯片的问题，我们设计了 touch.c/touch.h 这两个文件统一管理各类型的驱动。不同的驱动芯片类型可以在 touch.c 中集中添加，并通过 touch.c 中的接口统一调用，不同的触摸芯片各自编写独立的.c/.h 文件，需要时被 touch.c 调用。电阻触摸屏相关代码也在 touch.c 中实现。

1. 触摸管理驱动代码

因为需要支持的触摸驱动比较多，为了方便管理和添加新的驱动，我们用 touch.c 文件来统一管理这些触摸驱动，然后针对各类触摸芯片编写独立的驱动。为了方便管理触摸，我们在 touch.h 中定义一个用于管理触摸信息的结构体类型，具体代码如下：

```
/* 触摸屏控制器 */
typedef struct
{
    uint8_t (*init)(void);          /* 初始化触摸屏控制器 */
```

```
    uint8_t (*scan)(uint8_t);    /* 扫描触摸屏.0,屏幕扫描;1,物理坐标; */
    void (*adjust)(void);        /* 触摸屏校准 */
    uint16_t x[CT_MAX_TOUCH];    /* 当前坐标 */
    uint16_t y[CT_MAX_TOUCH];    /* 电容屏有最多 10 组坐标,电阻屏则用 x[0],y[0]代表:此次
                                 扫描时,触屏的坐标,用 x[9],y[9]存储第一次按下时的坐标 */

    uint16_t sta;                /* 笔的状态
                                  * b15:按下 1/松开 0;
                                  * b14:0,没有按键按下;1,有按键按下.
                                  * b13~b10:保留
                                  * b9~b0:电容触摸屏按下的点数(0,表示未按下,1 表示按下)
                                    */

    /* 5 点校准触摸屏校准参数(电容屏不需要校准) */
    float xfac;                  /* 5 点校准法 x 方向比例因子 */
    float yfac;                  /* 5 点校准法 y 方向比例因子 */
    short xc;                    /* 中心 X 坐标物理值(AD 值) */
    short yc;                    /* 中心 Y 坐标物理值(AD 值) */

    /* 新增的参数,当触摸屏的左右上下完全颠倒时需要用到.
     * b0:  0, 竖屏(适合左右为 X 坐标,上下为 Y 坐标的 TP)
     *      1, 横屏(适合左右为 Y 坐标,上下为 X 坐标的 TP)
     * b1~6:    保留.
     * b7:  0, 电阻屏
     *      1, 电容屏
     */
    uint8_t touchtype;
} _m_tp_dev;

extern _m_tp_dev tp_dev;         /* 触屏控制器在 touch.c 里面定义 */
```

这里我们定义了函数指针，只要把相对应的触摸芯片的函数指针赋值给它，就可以通过这个通用接口很方便调用不同芯片的函数接口。正点原子不同的触摸屏区别如下：

1、在使用 4.3 寸屏、10.1 寸屏电容屏时，使用的是汇顶科技的 GT9xxx 系列触摸屏驱动 IC，这是一个 IIC 接口的驱动芯片，我们要编写 gt9xxx 系列芯片的初始化程序，并编写一个坐标扫描程序，这里我们先预留这两个接口分别为 gt9xxx_init()和 gt9xxx_scan()，在 gt9xxx.c 文件中再专门实现这两个驱动，标记使用的为电容屏；

2、类似地，在使用 SSD1963 7 寸屏、7 寸 800*480/1024*600 RGB 屏时，我们的屏幕搭载的触摸驱动芯片是 ft5206/GT911，FT5206 触摸 IC 预留这两个接口分别为 ft5206_init()和 ft5206_scan()，在 ft5206.c 文件中再专门实现这两个驱动，标记使用的为电容屏;GT911 也是调用 gtxxx_init()和 gt9xxx_scan()接口。

3、当为其它 ID 时，默认为电阻屏，而电阻屏默认使用的是 SPI 接口的 XPT2046 芯片。由于电阻屏存在线性误差，所以在使用前需要进行校准，这也是为什么在前面的结构体类型中存在关于校准参数的成员。为了避免每次都要进行校准的麻烦，所以会使用 AT24C02 来存储校准成功后的数据。如何进行校准也会在后面进行讲解。作为电阻屏，它也有一个扫描坐标函数即 tp_scan()。

(*init)(void)这个结构体函数指针，默认指向 tp_init 的，而在 tp_init 里对触摸屏进行初始化并对(*scan)(uint8_t)函数指针根据触摸芯片类型重新做了指向。在这里简单看一下 touch.c 的触摸屏初始化函数 tp_init，其代码如下：

```
/**
 * @brief       触摸屏初始化
 * @param       无
 * @retval      0,没有进行校准
 *              1,进行过校准
 */
uint8_t tp_init(void)
{
```

```
GPIO_InitTypeDef gpio_init_struct;

tp_dev.touchtype = 0;                           /* 默认设置(电阻屏 & 竖屏) */
tp_dev.touchtype |= lcddev.dir & 0X01;          /* 根据 LCD 判定是横屏还是竖屏 */

if (lcddev.id == 0x7796)
/* 3.5 寸屏有两种，一种屏幕 ID 为 0x5510 带电阻触摸屏，一种屏幕 ID
   为 0x7796 带 GT 型号的电容触摸屏 */
{
    if (gt9xxx_init() == 0)
    /* 初始化 GT 系列触摸屏成功,即当前 3.5 寸屏为电容触摸屏 */
    {
        tp_dev.scan = gt9xxx_scan;  /* 扫描函数指向 GT9147 触摸屏扫描 */
        tp_dev.touchtype |= 0X80;   /* 电容屏 */
        return 0;
    }
}

if (lcddev.id == 0X5510 || lcddev.id == 0X4342 || lcddev.id == 0X1018 ||
    lcddev.id == 0X4384 || lcddev.id == 0X9806)
{   /* 电容触摸屏,4.3 寸/10.1 寸屏 */
    gt9xxx_init();
    tp_dev.scan = gt9xxx_scan;  /* 扫描函数指向 GT9147 触摸屏扫描 */
    tp_dev.touchtype |= 0X80;   /* 电容屏 */
    return 0;
}
else if (lcddev.id == 0X1963 || lcddev.id == 0X7084 || lcddev.id == 0X7016)
{/* 触摸 IC 是 FT 系列的就执行 ft5206_init 函数以及使用 ft5206_scan 扫描函数 */
    if (!ft5206_init())
    {
        tp_dev.scan = ft5206_scan; /* 扫描函数指向 FT5206 触摸屏扫描 */
    }
    else
    {
        gt9xxx_init();
        tp_dev.scan = gt9xxx_scan; /* 扫描函数指向 GT9147 触摸屏扫描 */
    }
    tp_dev.touchtype |= 0X80;       /* 电容屏 */
    return 0;
}
else
{
    T_PEN_GPIO_CLK_ENABLE();        /* T_PEN 脚时钟使能 */
    T_CS_GPIO_CLK_ENABLE();         /* T_CS 脚时钟使能 */
    T_MISO_GPIO_CLK_ENABLE();       /* T_MISO 脚时钟使能 */
    T_MOSI_GPIO_CLK_ENABLE();       /* T_MOSI 脚时钟使能 */
    T_CLK_GPIO_CLK_ENABLE();        /* T_CLK 脚时钟使能 */

    gpio_init_struct.Pin = T_PEN_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_INPUT;            /* 输入 */
    gpio_init_struct.Pull = GPIO_PULLUP;                /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;/* 高速 */
    HAL_GPIO_Init(T_PEN_GPIO_PORT, &gpio_init_struct); /* 初始化 T_PEN 引脚 */

    gpio_init_struct.Pin = T_MISO_GPIO_PIN;
    HAL_GPIO_Init(T_MISO_GPIO_PORT,&gpio_init_struct);/* 初始化 T_MISO 引脚 */

    gpio_init_struct.Pin = T_MOSI_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;        /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;                /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;/* 高速 */
    HAL_GPIO_Init(T_MOSI_GPIO_PORT,&gpio_init_struct);/* 初始化 T_MOSI 引脚 */
```

```
        gpio_init_struct.Pin = T_CLK_GPIO_PIN;
        HAL_GPIO_Init(T_CLK_GPIO_PORT, &gpio_init_struct); /* 初始化 T_CLK 引脚 */

        gpio_init_struct.Pin = T_CS_GPIO_PIN;
        HAL_GPIO_Init(T_CS_GPIO_PORT, &gpio_init_struct); /* 初始化 T_CS 引脚 */

        tp_read_xy(&tp_dev.x[0], &tp_dev.y[0]);       /* 第一次读取初始化 */
        at24cxx_init();                               /* 初始化 24CXX */

        if (tp_get_adjust_data())
        {
            return 0;               /* 已经校准 */
        }
        else                        /* 未校准? */
        {
            lcd_clear(WHITE);       /* 清屏 */
            tp_adjust();            /* 屏幕校准 */
            tp_save_adjust_data();
        }
        tp_get_adjust_data();
    }
    return 1;
}
```

正点原子的电容屏在出厂时已经由厂家校对好参数了，而电阻屏由于工艺和每个屏的线性有所差异，我们需要先对其进行“校准”，我们在下一点补充说明它的实现。

通过上面的触摸初始化后，我们就可以读取相关的触点信息用于显示编程了，注意到上面还有很多个函数还没实现，比如读取坐标和校准，我们在接下来的代码中将它补充完整。

2. 电阻屏触摸函数

前面我们介绍过了电阻式触摸屏的原理，由于电阻屏的驱动代码都比较类似，我们决定把电阻屏的驱动函数直接添加在 touch.c/touch.h 中实现。

在 touch.c 的初始化函数 tp_init 中，对使用到的 SPI 接口 IO 进行了初始化。接下来介绍一下获取触摸点在屏幕上坐标的算法：先获取逻辑坐标（AD 值），再转换成屏幕坐标。

如何获取逻辑坐标（AD 值），在前面已经分析过了，所以这里我们看一下 tp_read_ad()函数接口：

```
/**
 * @brief       SPI 读数据
 * @note        从触摸屏 IC 读取 adc 值
 * @param       cmd: 指令
 * @retval      读取到的数据,ADC 值(12bit)
 */
static uint16_t tp_read_ad(uint8_t cmd)
{
    uint8_t count = 0;
    uint16_t num = 0;
    T_CLK(0);               /* 先拉低时钟 */
    T_MOSI(0);              /* 拉低数据线 */
    T_CS(0);                /* 选中触摸屏 IC */
    tp_write_byte(cmd);     /* 发送命令字 */
    delay_us(6);            /* ADS7846 的转换时间最长为 6us */
    T_CLK(0);
    delay_us(1);
    T_CLK(1);               /* 给 1 个时钟，清除 BUSY */
    delay_us(1);
    T_CLK(0);

    for (count = 0; count < 16; count++)    /* 读出 16 位数据,只有高 12 位有效 */
    {
```

```
        num <<= 1;
        T_CLK(0);           /* 下降沿有效 */
        delay_us(1);
        T_CLK(1);
        if (T_MISO)num++;
    }

    num >>= 4;          /* 只有高 12 位有效. */
    T_CS(1);            /* 释放片选 */
    return num;
}
```

这里我们使用的是软件模拟 SPI，遵照时序编写 SPI 读函数接口。而发送命令字是通过写函数 tp_write_byte 来实现，详看源码。

一次读取的误差会很大，我们采用平均值滤波的方法，多次读取数据并丢弃波动最大的最大和最小值，取余下的平均值。具体可以查看 tp_read_xoy 函数内部实现。

```
/* 电阻触摸驱动芯片 数据采集 滤波用参数 */
#define TP_READ_TIMES   5       /* 读取次数 */
#define TP_LOST_VAL     1       /* 丢弃值 */

/**
 * @brief       读取一个坐标值(x 或者 y)
 * @note        连续读取 TP_READ_TIMES 次数据,对这些数据升序排列,
 *              然后去掉最低和最高 TP_LOST_VAL 个数, 取平均值
 *              设置时需满足: TP_READ_TIMES > 2*TP_LOST_VAL 的条件
 *
 * @param       cmd : 指令
 *   @arg       0XD0: 读取 X 轴坐标(@竖屏状态,横屏状态和 Y 对调.)
 *   @arg       0X90: 读取 Y 轴坐标(@竖屏状态,横屏状态和 X 对调.)
 *
 * @retval      读取到的数据(滤波后的), ADC 值(12bit)
 */
static uint16_t tp_read_xoy(uint8_t cmd)
{
    uint16_t i, j;
    uint16_t buf[TP_READ_TIMES];
    uint16_t sum = 0;
    uint16_t temp;

    for (i = 0; i < TP_READ_TIMES; i++)     /* 先读取 TP_READ_TIMES 次数据 */
    {
        buf[i] = tp_read_ad(cmd);
    }

    for (i = 0; i < TP_READ_TIMES - 1; i++) /* 对数据进行排序 */
    {
        for (j = i + 1; j < TP_READ_TIMES; j++)
        {
            if (buf[i] > buf[j])    /* 升序排列 */
            {
                temp = buf[i];
                buf[i] = buf[j];
                buf[j] = temp;
            }
        }
    }

    sum = 0;
    for (i = TP_LOST_VAL; i < TP_READ_TIMES - TP_LOST_VAL; i++)
    {   /* 去掉两端的丢弃值 */
        sum += buf[i];  /* 累加去掉丢弃值以后的数据. */
    }
```

```
    temp = sum / (TP_READ_TIMES - 2 * TP_LOST_VAL); /* 取平均值 */
    return temp;
}
```

有了前述代码，我们就可以通过 tp_read_xoy(uint8_t cmd)接口调取需要的 x 或者 y 坐标的 AD 值了。这里我们加上横屏或者竖屏的处理代码，编写一个可以通过指针一次得到 x 和 y 的两个 AD 值的接口，代码如下：

```
/**
 * @brief       读取 x, y 坐标
 * @param       x,y: 读取到的坐标值
 * @retval      无
 */
static void tp_read_xy(uint16_t *x, uint16_t *y)
{
    uint16_t xval, yval;

    if (tp_dev.touchtype & 0X01)        /* X,Y 方向与屏幕相反 */
    {
        xval = tp_read_xoy(0X90);       /* 读取 X 轴坐标 AD 值, 并进行方向变换 */
        yval = tp_read_xoy(0XD0);       /* 读取 Y 轴坐标 AD 值 */
    }
    else                                /* X,Y 方向与屏幕相同 */
    {
        xval = tp_read_xoy(0XD0);       /* 读取 X 轴坐标 AD 值 */
        yval = tp_read_xoy(0X90);       /* 读取 Y 轴坐标 AD 值 */
    }

    *x = xval;
    *y = yval;
}
```

为了进一步保证参数的精度，我们连续读两次触摸数据并取平均值作为最后的触摸参数，并对这两次滤波值平均后再传给目标存储区，由于 AD 的精度为 12 位，故该函数读取坐标的值 0~4095，tp_read_xy2 的代码如下：

```
/* 连续两次读取 X,Y 坐标的数据误差最大允许值 */
#define TP_ERR_RANGE    50      /* 误差范围 */

/**
 * @brief       连续读取 2 次触摸 IC 数据, 并滤波
 * @note        连续 2 次读取触摸屏 IC,且这两次的偏差不能超过 ERR_RANGE,满足
 *              条件,则认为读数正确,否则读数错误.该函数能大大提高准确度.
 *
 * @param       x,y: 读取到的坐标值
 * @retval      0, 失败; 1, 成功;
 */
static uint8_t tp_read_xy2(uint16_t *x, uint16_t *y)
{
    uint16_t x1, y1;
    uint16_t x2, y2;

    tp_read_xy(&x1, &y1);   /* 读取第一次数据 */
    tp_read_xy(&x2, &y2);   /* 读取第二次数据 */

   /* 前后两次采样在+-TP_ERR_RANGE 内 */
   if (((x2 <= x1 && x1<x2+TP_ERR_RANGE)||(x1 <= x2 && x2<x1+TP_ERR_RANGE))&&
       ((y2 <= y1 && y1<y2+TP_ERR_RANGE)||(y1 <= y2 && y2<y1+TP_ERR_RANGE)))
    {
        *x = (x1 + x2) / 2;
        *y = (y1 + y2) / 2;
        return 1;
    }

    return 0;
```

```
}
```

根据以上的流程，可以得到电阻屏触摸点的比较精确的 AD 信息。每次触摸屏幕时会对应一组 X、Y 的 AD 值，由于坐标的 AD 值是在 X、Y 方向都是线性的，很容易想到要把触摸信息的 AD 值和屏幕坐标联系起来，这里需要编写一个坐标转换函数，前面在编写初始化接口时讲到的校准函数这时候就派上用场了。

从前面的知识我们就知道触摸屏的 AD 的 X_{AD}、Y_{AD} 可以构成一个逻辑平面，LCD 屏的屏幕坐标 X、Y 也是一个逻辑平面，由于存在误差，这两个平面并不重合，校准的作用就是要将逻辑平面映射到物理平面上，即得到触点在液晶屏上的位置坐标。校准算法的中心思想也就是要建立这样一个映射函数现有的校准算法大多是基于线性校准，即首先假定物理平面和逻辑平面之间的误差是线性误差，由旋转和偏移形成。

常用的电阻式触摸屏校正方法有两点校准法和三点校准法。本文这里介绍的是结合了不同的电阻式触摸屏校正法的优化算法：五点校正法。其中主要的原理是使用 4 点校正法的比例运算以及三点校正法的基准点运算。五点校正法优势在于可以更加精确的计算出 X 和 Y 方向的比例缩放系数，同时提供了中心基准点，对于一些线性电阻系数比较差电阻式触摸屏有很好的校正功能。校正相关的变量主要有：

- x［5］，y［5］五点定位的物理坐标（LCD 坐标）
- xl［5］，yl［5］五点定位的逻辑坐标（触摸 AD 值）
- KX，KY 横纵方向伸缩系数
- XLC，YLC 中心基点逻辑坐标
- XC，YC 中心基点物理坐标（数值采用 LCD 显示屏的物理长宽分辨率的一半）x［5］，y［5］五点定位的物理坐标是已知的，其中 4 点分别设置在 LCD 的角落，一点设置在 LCD 正中心，作为基准校正点，校正关键点和距离布局如图 43.3.3.1 所示。

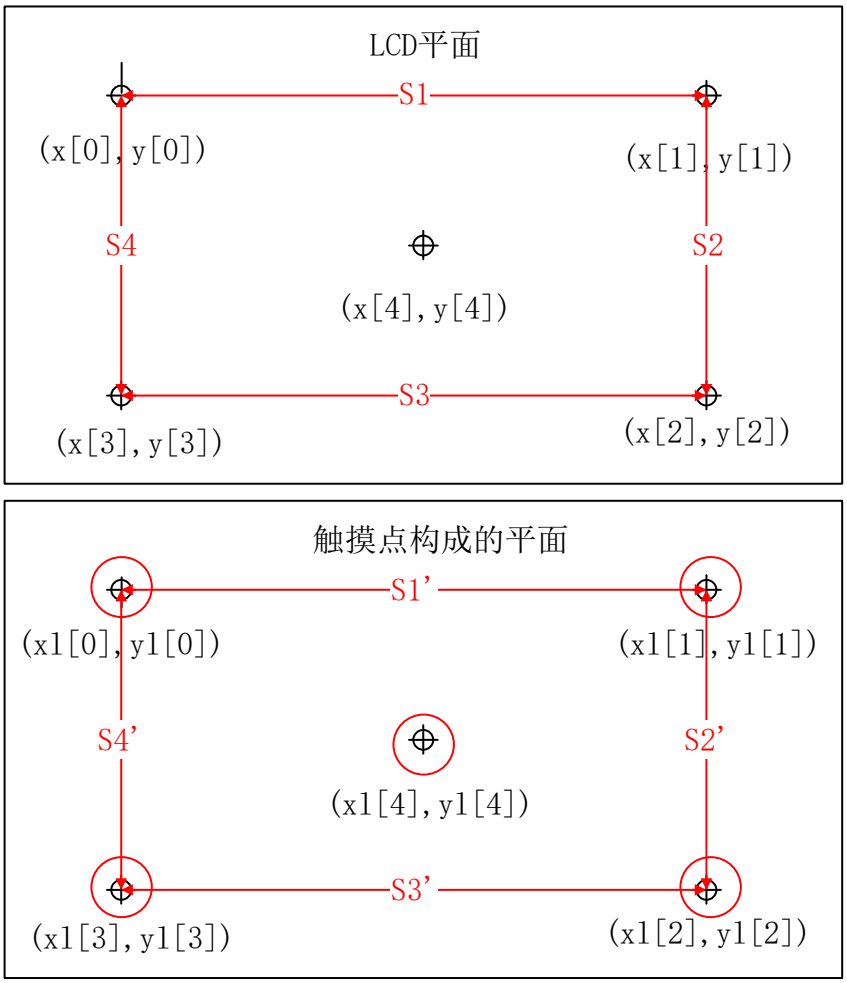


图 43.3.3.1 电阻屏五点校准法的参考点设定

校正步骤如下：

1. 通过先后点击 LCD 的 4 个角落的校正点，获取 4 个角落的逻辑坐标值。

2. 计算屏幕坐标和四点间距：

```
S1 = x[1] - x[0]
S3 = x[2] - x[3]
S2 = y[2] - y[1]
S4 = y[3] - y[0]
```

一般取点可以人为的设定 S1=S3 和 S2=S4，以方便运算。

计算逻辑坐标的四点“间距”，由于实际触点肯定会存在误差，所以触摸点会落在实际设定点的更大范围内，在图 43.3.1 中，设定点为五个点⊕，但实际采样时触点有时会落在稍大的外圈范围，图中用红色的圆圈标注了，所以有必要设定一个误差范围：

```
S1' = xl[1] - xl[0]
S3' = xl[2] - xl[3]
S2' = yl[2] - yl[1]
S4' = yl[3] - yl[0]
```

由于触点的误差，对于逻辑点 S1’和 S3’则大概率不会相等，同样的，S2’和 S4’也很难取到相等的点，那么为了简化计算，我们强制以（S1’+S3’）/2 的线长作一个矩形一边，以（S2’+S4’）/2 为矩形另一边，这样构建的矩形在误差范围是可以接受的，也方便计算，于是得到 X 和 Y 方向的近似缩放系数：

```
KX = (S1' + S3') / 2 / S1
KY = (S2' + S4') / 2 / S2
```

3. 点击 LCD 正中心，获取中心点的逻辑坐标，作为校正的基准点。这里也同样的需要限制误差，之后可以得到一个中心点的 AD 值坐标(xl[4],yl[4])，这个点的 AD 值我们就作为我们对比的基准点，即 xl[4]=XLC，yl[4]=YLC；

4. 完成以上步骤则校正完成。下次点击触摸屏的时候获取的逻辑值 XL 和 YL，便可以按下以公式转换为物理坐标：

```
X = (XL - XLC) / KX + XC
Y = (YL - YLC) / KY + YC
```

最后一步的转换公式可能不好理解，大家换个角度，如果我们求到的缩放比例是正确的，在取新的触摸的时候，这个触摸点的逻辑坐标和物理坐标的转换，必然与中心点在两方向上的缩放比例相等，用中学数学直线斜率相等的情况，变换便可得到上述公式。

通过上述得到校准参数后，在以后的使用中，我们把所有得到的物理坐标都按照这个关系式来计算，得到的就是触摸点的屏幕坐标。为了省去每次都需要校准的麻烦，我们保存这些参数到 AT24Cxx 的指定扇区地址，这样只要校准一次就可以重复使用这些参数了。

根据上面的原理，我们设计的校准函数 tp_adjust 如下：

```
/**
 * @brief       触摸屏校准代码
 * @note        使用五点校准法(具体原理请百度)
 *              本函数得到 x 轴/y 轴比例因子 xfac/yfac 及物理中心坐标值(xc,yc)等 4 个参数
 *              我们规定:物理坐标即 AD 采集到的坐标值,范围是 0~4095.
 *                       逻辑坐标即 LCD 屏幕的坐标, 范围为 LCD 屏幕的分辨率.
 *
 * @param       无
 * @retval      无
 */
void tp_adjust(void)
{
    uint16_t pxy[5][2];         /* 物理坐标缓存值 */
    uint8_t  cnt = 0;
    short s1, s2, s3, s4;       /* 4 个点的坐标差值 */
    double px, py;              /* X,Y 轴物理坐标比例,用于判定是否校准成功 */
    uint16_t outtime = 0;
    cnt = 0;

    lcd_clear(WHITE);           /* 清屏 */
```

```
    lcd_show_string(40, 40, 160, 100, 16, TP_REMIND_MSG_TBL,RED);/*显示提示信息*/
    tp_draw_touch_point(20, 20, RED);       /* 画点 1 */
    tp_dev.sta = 0;                         /* 消除触发信号 */

    while (1)                               /* 如果连续 10 秒钟没有按下,则自动退出 */
    {
        tp_dev.scan(1);                     /* 扫描物理坐标 */
        if ((tp_dev.sta & 0xc000) == TP_CATH_PRES)
        {   /* 按键按下了一次(此时按键松开了.) */
            outtime = 0;
            tp_dev.sta &= ~TP_CATH_PRES;        /* 标记按键已经被处理过了. */
            pxy[cnt][0] = tp_dev.x[0];          /* 保存 X 物理坐标 */
            pxy[cnt][1] = tp_dev.y[0];          /* 保存 Y 物理坐标 */
            cnt++;

            switch (cnt)
            {
                case 1:
                    tp_draw_touch_point(20, 20, WHITE);             /* 清点 1 */
                    tp_draw_touch_point(lcddev.width - 20, 20, RED);/* 画点 2 */
                    break;

                case 2:
                    tp_draw_touch_point(lcddev.width - 20, 20,WHITE);/* 清点 2 */
                    tp_draw_touch_point(20, lcddev.height - 20, RED);/* 画点 3 */
                    break;

                case 3:
                    tp_draw_touch_point(20, lcddev.height-20, WHITE);/* 清点 3*/
                    /* 画点 4 */
                    tp_draw_touch_point(lcddev.width-20,lcddev.height-20, RED);
                    break;

                case 4:
                    lcd_clear(WHITE); /* 画第五个点了, 直接清屏 */
                    /* 画点 5 */
                    tp_draw_touch_point(lcddev.width / 2,lcddev.height/2,RED);
                    break;

                case 5:                 /* 全部 5 个点已经得到 */
                    s1 = pxy[1][0] - pxy[0][0];
                    /* 第 2 个点和第 1 个点的 X 轴物理坐标差值(AD 值) */
                    s3 = pxy[3][0] - pxy[2][0];
                    /* 第 4 个点和第 3 个点的 X 轴物理坐标差值(AD 值) */
                    s2 = pxy[3][1] - pxy[1][1];
                    /* 第 4 个点和第 2 个点的 Y 轴物理坐标差值(AD 值) */
                    s4 = pxy[2][1] - pxy[0][1];
                    /* 第 3 个点和第 1 个点的 Y 轴物理坐标差值(AD 值) */

                    px = (double)s1 / s3;       /* X 轴比例因子 */
                    py = (double)s2 / s4;       /* Y 轴比例因子 */

                    if (px < 0)px = -px;        /* 负数改正数 */
                    if (py < 0)py = -py;        /* 负数改正数 */

                    if (px < 0.95 || px > 1.05 || py < 0.95 || py > 1.05 ||
                        abs(s1)>4095||abs(s2)>4095||abs(s3)>4095||abs(s4)>4095||
                        abs(s1)==0 ||abs(s2)==0||abs(s3)==0||abs(s4)==0)
                    {/* 比例不合格, 差值大于坐标范围或等于 0, 重绘校准图形 */
                    cnt = 0;
                    /* 清除点 5 */
                    tp_draw_touch_point(lcddev.width/2,lcddev.height/2, WHITE);
```

```
                    tp_draw_touch_point(20, 20, RED); /* 重新画点 1 */
                    tp_adjust_info_show(pxy, px, py); /* 显示当前信息,方便找问题 */
                    continue;
                    }

                    tp_dev.xfac = (float)(s1 + s3) / (2 * (lcddev.width - 40));
                    tp_dev.yfac = (float)(s2 + s4) /(2 * (lcddev.height - 40));
                    tp_dev.xc = pxy[4][0];      /* X 轴,物理中心坐标 */
                    tp_dev.yc = pxy[4][1];      /* Y 轴,物理中心坐标 */
                    lcd_clear(WHITE);           /* 清屏 */
                    lcd_show_string(35, 110, lcddev.width, lcddev.height, 16,
                                    "Touch Screen Adjust OK!", BLUE); /* 校正完成 */
                    delay_ms(1000);
                    tp_save_adjust_data();
                    lcd_clear(WHITE);  /* 清屏 */
                    return;         /* 校正完成 */
                }
            }

            delay_ms(10);
            outtime++;
            if (outtime > 1000)
            {
                tp_get_adjust_data();
                break;
            }
        }
    }
```

注意该函数里面多次使用了 lcddev.width 和 lcddev.height，用于坐标设置，故在程序调用前需要预先初始化 LCD 得到 LCD 的一些屏幕信息，主要是为了兼容不同尺寸的 LCD（比如 320*240、480*320 和 800*480 的屏都可以兼容）。

有了校准参数后，由于我们需要频繁地进行屏幕坐标和物理坐标的转换，我们为电阻屏增加一个 tp_scan(uint8_t mode)用于转换，为了实际使用上更灵活，我们使这个参数支持物理坐标和屏幕坐标，设计的函数如下：

```
/**
 * @brief       触摸按键扫描
 * @param       mode: 坐标模式
 *   @arg       0, 屏幕坐标;
 *   @arg       1, 物理坐标(校准等特殊场合用)
 *
 * @retval      0, 触屏无触摸; 1, 触屏有触摸;
 */
uint8_t tp_scan(uint8_t mode)
{
    if (T_PEN == 0)     /* 有按键按下 */
    {
        if (mode)       /* 读取物理坐标, 无需转换 */
        {
            tp_read_xy2(&tp_dev.x[0], &tp_dev.y[0]);
        }
        else if(tp_read_xy2(&tp_dev.x[0], &tp_dev.y[0]))/* 读取屏幕坐标, 需要转换*/
        {   /* 将 X 轴 物理坐标转换成逻辑坐标(即对应 LCD 屏幕上面的 X 坐标值) */
            tp_dev.x[0] = (signed short)(tp_dev.x[0] - tp_dev.xc)
                          / tp_dev.xfac + lcddev.width / 2;
            /* 将 Y 轴 物理坐标转换成逻辑坐标(即对应 LCD 屏幕上面的 Y 坐标值) */
            tp_dev.y[0] = (signed short)(tp_dev.y[0] - tp_dev.yc)
                                    / tp_dev.yfac + lcddev.height / 2;
        }

        if ((tp_dev.sta & TP_PRES_DOWN) == 0)    /* 之前没有被按下 */
        {
```

```
            tp_dev.sta = TP_PRES_DOWN | TP_CATH_PRES;  /* 按键按下 */
            tp_dev.x[CT_MAX_TOUCH - 1] = tp_dev.x[0];  /* 记录第一次按下时的坐标 */
            tp_dev.y[CT_MAX_TOUCH - 1] = tp_dev.y[0];
        }
    }
    else
    {
        if (tp_dev.sta & TP_PRES_DOWN)       /* 之前是被按下的 */
        {
            tp_dev.sta &= ~TP_PRES_DOWN;     /* 标记按键松开 */
        }
        else      /* 之前就没有被按下 */
        {
            tp_dev.x[CT_MAX_TOUCH - 1] = 0;
            tp_dev.y[CT_MAX_TOUCH - 1] = 0;
            tp_dev.x[0] = 0xffff;
            tp_dev.y[0] = 0xffff;
        }
    }

    return tp_dev.sta & TP_PRES_DOWN; /* 返回当前的触屏状态 */
}
```

要进行电阻触摸屏的触摸扫描，只要调取 tp_scan()函数，就能灵活地得到触摸坐标。电阻屏的触摸就讲到这里。

3. 电容屏触摸驱动代码

电容触摸芯片使用的是 IIC 接口。IIC 接口部分代码，我们可以参考 myiic.c 和 myiic.h 的代码，为了使代码独立，我们在“TOUCH”文件夹下也是采用软件模拟 IIC 的方式实现 ctiic.c 和 ctiic.h，这样 IO 的使用更灵活，这里部分参考 IIC 章节的知识就可以了，这里不重复介绍了。

电容触摸芯片除了 IIC 接口相关引脚 CT_SCL 和 CT_SDA，还有 CT_INT 和 CT_RST，接口图如图 43.3.3.2 所示。

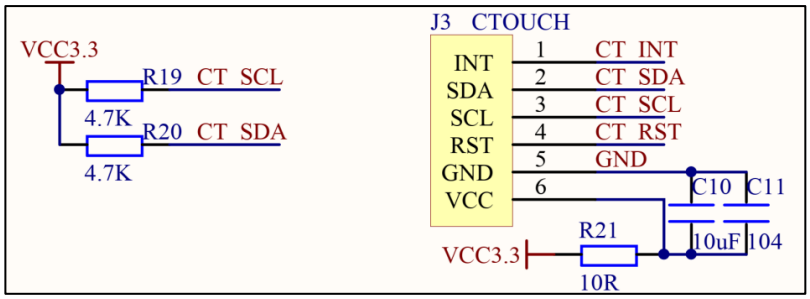


图 43.3.3.1 电容触摸芯片接口图

gt9xxx_init 的实现也比较简单，实现 CT_INT 和 CT_RST 引脚初始化和调用 ct_iic_init 函数实现对 CT_SDA 和 CT_SCL 初始化。由于电容触摸屏在设计时是根据屏幕进行参数设计的，参数已经保存在芯片内部。所以在初始化后，就可以参考手册推荐的 IIC 时序从相对应的坐标数据寄存器中把对应的 XY 坐标数据读出来，再通过数据整理转成 LCD 坐标。

与电阻屏不同的是，我们是通过 IIC 读取状态寄存器的值并非引脚电平。而 gt9xxx 系列是支持中断或轮询方式得到触摸状态，本实验使用的是轮询方式：

1、按照读时序，先读取寄存器 0x814E，若当前 buffer（buffer status 为 1）数据准备好，则依据有效触点个数到相对应的坐标数据地址处进行坐标数据读取。

2、若在 1 中发现 buffer 数据（buffer status 为 0）未准备好，则等待 1ms 再进行读取。

这样，gt9xxx_scan()函数的实现如下：这样，gt9xxx_scan()函数的实现如下：

```
/* GT9XXX 10 个触摸点(最多) 对应的寄存器表 */
const uint16_t GT9XXX_TPX_TBL[10] =
{
 GT9XXX_TP1_REG, GT9XXX_TP2_REG, GT9XXX_TP3_REG, GT9XXX_TP4_REG,
 GT9XXX_TP5_REG, GT9XXX_TP6_REG, GT9XXX_TP7_REG, GT9XXX_TP8_REG,
```

```
 GT9XXX_TP9_REG, GT9XXX_TP10_REG,
};
/**
 * @brief       扫描触摸屏(采用查询方式)
 * @param       mode : 电容屏未用到次参数, 为了兼容电阻屏
 * @retval      当前触屏状态
 *   @arg       0, 触屏无触摸;
 *   @arg       1, 触屏有触摸;
 */
uint8_t gt9xxx_scan(uint8_t mode)
{
    uint8_t buf[4];
    uint8_t i = 0;
    uint8_t res = 0;
    uint16_t temp;
    uint16_t tempsta;
    static uint8_t t = 0;    /* 控制查询间隔,从而降低 CPU 占用率 */
    t++;

    if ((t % 10) == 0 || t < 10)
    {    /* 空闲时,每进入 10 次 CTP_Scan 函数才检测 1 次,从而节省 CPU 使用率 */
        gt9xxx_rd_reg(GT9XXX_GSTID_REG, &mode, 1);  /* 读取触摸点的状态 */

        if ((mode & 0X80) && ((mode & 0XF) <= g_gt_tnum))
        {
            i = 0;
            gt9xxx_wr_reg(GT9XXX_GSTID_REG, &i, 1); /* 清标志 */
        }

        if ((mode & 0XF) && ((mode & 0XF) <= g_gt_tnum))
        {
          /* 将点的个数转换为 1 的位数,匹配 tp_dev.sta 定义 */
            temp = 0XFFFF << (mode & 0XF);
            tempsta = tp_dev.sta;                   /* 保存当前的 tp_dev.sta 值 */
            tp_dev.sta = (~temp) | TP_PRES_DOWN | TP_CATH_PRES;
            tp_dev.x[g_gt_tnum - 1] = tp_dev.x[0]; /* 保存触点 0 的数据 */
            tp_dev.y[g_gt_tnum - 1] = tp_dev.y[0];

            for (i = 0; i < g_gt_tnum; i++)
            {
                if (tp_dev.sta & (1 << i))  /* 触摸有效? */
                {
                    gt9xxx_rd_reg(GT9XXX_TPX_TBL[i], buf, 4); /* 读取 XY 坐标值 */
                    if (lcddev.id == 0X5510)            /* 4.3 寸 800*480 MCU 屏 */
                    {
                        if (tp_dev.touchtype & 0X01)    /* 横屏 */
                        {
                            tp_dev.x[i] = lcddev.width - (((uint16_t)buf[3]
                                          << 8) + buf[2]);
                            tp_dev.y[i] = ((uint16_t)buf[1] << 8) + buf[0];

                        }
                        else
                        {
                            tp_dev.x[i] = ((uint16_t)buf[1] << 8) + buf[0];
                            tp_dev.y[i] = ((uint16_t)buf[3] << 8) + buf[2];
                        }
                    }
                    else    /* 其他型号 */
                    {
                        if (tp_dev.touchtype & 0X01)     /* 横屏 */
                        {
                            tp_dev.x[i] = (((uint16_t)buf[1] << 8) + buf[0]);
                            tp_dev.y[i] = (((uint16_t)buf[3] << 8) + buf[2]);
```

```
                        }
                        else
                        {
                            tp_dev.x[i]=lcddev.width-
                                        (((uint16_t)buf[3]<<8)+buf[2]);
                            tp_dev.y[i] = ((uint16_t)buf[1] << 8) + buf[0];
                        }
                    }
                //printf("x[%d]:%d,y[%d]:%d\r\n",i,tp_dev.x[i],i,tp_dev.y[i]);
                }
            }

            res = 1;

            if (tp_dev.x[0] > lcddev.width || tp_dev.y[0] > lcddev.height)
        {   /* 非法数据(坐标超出了) */
                if((mode&0XF) > 1)/*有其他点有数据,则复第二个触点的数据到第一个触点 */
                {
                    tp_dev.x[0] = tp_dev.x[1];
                    tp_dev.y[0] = tp_dev.y[1];
                    t = 0;  /* 触发一次,则会最少连续监测 10 次,从而提高命中率 */
                }
                else        /* 非法数据,则忽略此次数据(还原原来的) */
                {
                    tp_dev.x[0] = tp_dev.x[g_gt_tnum - 1];
                    tp_dev.y[0] = tp_dev.y[g_gt_tnum - 1];
                    mode = 0X80;
                    tp_dev.sta = tempsta;   /* 恢复 tp_dev.sta */
                }
            }
            else
            {
                t = 0;      /* 触发一次,则会最少连续监测 10 次,从而提高命中率 */
            }
        }
    }

    if ((mode & 0X8F) == 0X80)                  /* 无触摸点按下 */
    {
        if (tp_dev.sta & TP_PRES_DOWN)          /* 之前是被按下的 */
        {
            tp_dev.sta &= ~TP_PRES_DOWN;        /* 标记按键松开 */
        }
        else    /* 之前就没有被按下 */
        {
            tp_dev.x[0] = 0xffff;
            tp_dev.y[0] = 0xffff;
            tp_dev.sta &= 0XE000;               /* 清除点有效标记 */
        }
    }

    if (t > 240)t = 10;                         /* 重新从 10 开始计数 */
    return res;
}
```

大家可以打开 gt9xxx 芯片对应的编程手册，对照时序，即可理解上述的实现过程，只是程序中为了匹配多种屏幕和横屏显示，添加了一些代码。

电容屏驱动 ft5206.c/ft5206.h 的驱动实现与 gt9xxx 的实现类似，大家参考本例程源码即可。

电容屏的触摸实验代码讲解到这里。

4. main 函数和测试代码

在 main.c 里面编程如下代码：

```
void rtp_test(void)
```

```
{
    uint8_t key;
    uint8_t i = 0;

    while (1)
    {
        key = key_scan(0);
        tp_dev.scan(0);

        if (tp_dev.sta & TP_PRES_DOWN)  /* 触摸屏被按下 */
        {
            if (tp_dev.x[0] < lcddev.width && tp_dev.y[0] < lcddev.height)
            {
                if (tp_dev.x[0] > (lcddev.width - 24) && tp_dev.y[0] < 16)
                {
                    load_draw_dialog(); /* 清除 */
                }
                else
                {
                    tp_draw_big_point(tp_dev.x[0], tp_dev.y[0], RED); /* 画点 */
                }
            }
        }
        else
        {
            delay_ms(10);         /* 没有按键按下的时候 */
        }

        if (key == KEY0_PRES)   /* KEY0 按下,则执行校准程序 */
        {
            lcd_clear(WHITE);   /* 清屏 */
            tp_adjust();        /* 屏幕校准 */
            tp_save_adjust_data();
            load_draw_dialog();
        }

        i++;
        if (i % 20 == 0)LED0_TOGGLE();
    }
}
/* 10 个触控点的颜色(电容触摸屏用) */
const uint16_t POINT_COLOR_TBL[10] = {RED, GREEN, BLUE, BROWN, YELLOW, MAGENTA,
                                      CYAN, LIGHTBLUE, BRRED, GRAY};

void ctp_test(void)
{
    uint8_t t = 0;
    uint8_t i = 0;
    uint16_t lastpos[10][2];        /* 最后一次的数据 */
    uint8_t maxp = 5;

   if (lcddev.id == 0X1018)maxp = 10;

    while (1)
    {
        tp_dev.scan(0);

        for (t = 0; t < maxp; t++)
        {
            if ((tp_dev.sta) & (1 << t))
            {   /* 坐标在屏幕范围内 */
                if (tp_dev.x[t] < lcddev.width && tp_dev.y[t] < lcddev.height)
                {
                    if (lastpos[t][0] == 0XFFFF)
                    {
```

```
                        lastpos[t][0] = tp_dev.x[t];
                        lastpos[t][1] = tp_dev.y[t];
                    }

                    lcd_draw_bline(lastpos[t][0], lastpos[t][1], tp_dev.x[t],
                                   tp_dev.y[t], 2, POINT_COLOR_TBL[t]); /* 画线 */
                    lastpos[t][0] = tp_dev.x[t];
                    lastpos[t][1] = tp_dev.y[t];

                    if (tp_dev.x[t] > (lcddev.width - 24) && tp_dev.y[t] < 20)
                    {
                        load_draw_dialog();/* 清除 */
                    }
                }
            }
            else
            {
                lastpos[t][0] = 0XFFFF;
            }
        }

        delay_ms(5);
        i++;

        if (i % 20 == 0)LED0_TOGGLE();
    }
}

int main(void)
{
    sys_cache_enable();                         /* 打开 L1-Cache */
    HAL_Init();                                 /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);         /* 设置时钟, 400Mhz */
    delay_init(400);                            /* 延时初始化 */
    usart_init(115200);                         /* 串口初始化为 115200 */
    mpu_memory_protection();                    /* 保护相关存储区域 */
    led_init();                                 /* 初始化 LED */
    sdram_init();                               /* 初始化 SDRAM */
    lcd_init();                                 /* 初始化 LCD */
    key_init();                                 /* 初始化按键 */
    tp_dev.init();                              /* 触摸屏初始化 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "TOUCH TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    if (tp_dev.touchtype != 0XFF)
    {   /* 电阻屏才显示 */
        lcd_show_string(30, 110, 200, 16, 16, "Press KEY0 to Adjust", RED);
    }

    delay_ms(1500);
    load_draw_dialog();

    if (tp_dev.touchtype & 0X80)
    {
        ctp_test(); /* 电容屏测试 */
    }
    else
    {
        rtp_test(); /* 电阻屏测试 */
    }
}
```

上面没有把 main.c 全部代码列出来，只是列出重要函数，这里简单介绍一下这三个函数。

rtp_test，该函数用于电阻触摸屏的测试，该函数代码比较简单，就是扫描按键和触摸屏，如果触摸屏有按下，则在触摸屏上面划线，如果按中“RST”区域，则执行清屏。如果按键 KEY0 按下，则执行触摸屏校准。

ctp_test，该函数用于电容触摸屏的测试，由于我们采用 tp_dev.sta 来标记当前按下的触摸屏点数，所以判断是否有电容触摸屏按下，也就是判断 tp_dev.sta 的最低 5 位，如果有数据，则画线，如果没数据则忽略，且 5 个点画线的颜色各不一样，方便区分。另外，电容触摸屏不需要校准，所以没有校准程序。

main 函数，则比较简单，初始化相关外设，然后根据触摸屏类型，去选择执行 ctp_test 还是 rtp_test。

软件部分就介绍到这里，接下来看看下载验证。

43.4 下载验证

在代码编译成功之后，我们通过下载代码到开发板上，电阻触摸屏测试如图 43.4.1 所示界面：

图43.4.1 电阻触摸屏测试程序运行效果

图中我们在电阻屏上画了一些内容，右上角的 RST 可以用来清屏，点击该区域，即可清屏重画。另外，按 KEY0 可以进入校准模式，如果发现触摸屏不准，则可以按 KEY0，进入校准， 重新校准一下，即可正常使用。

如果是电容触摸屏，测试界面如图 43.4.2 所示：

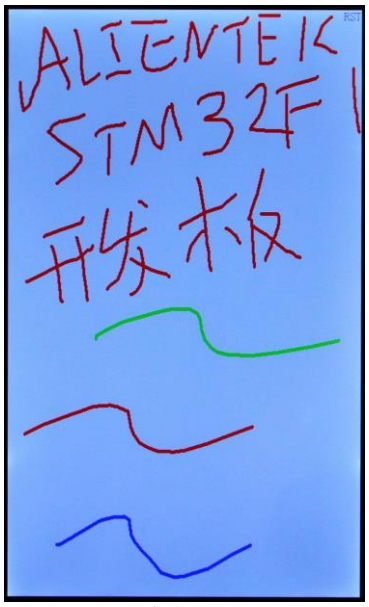


图 43.4.2 电容触摸屏测试界面

图中，同样输入了一些内容。电容屏支持多点触摸，每个点的颜色都不一样，图中的波浪线就是三点触摸画出来的，最多可以 5 点触摸。按右上角的 RST 标志，可以清屏。电容屏无需校准，所以按 KEY0 无效。KEY0 校准仅对电阻屏有效。

第四十四章 红外遥控实验

本章，我们将介绍 STM32 对红外遥控器的信号解码。STM32 板子上标配的红外接收头和一个小巧的红外遥控器。我们将利用 STM32 的输入捕获功能，解码开发板标配的红外遥控器的编码信号，并将编码后的键值在 LCD 模块中显示出来。

本章分为如下几个小节：

44.1 红外遥控简介
44.2 硬件设计
44.3 程序设计
44.4 下载验证

44.1 红外遥控简介

44.1.1 红外遥控技术介绍

红外遥控是一种无线、非接触控制技术，具有抗干扰能力强，信息传输可靠，功耗低，成本低，易实现等显著优点，被诸多电子设备特别是家用电器广泛采用，并越来越多的应用到计算机系统中。 由于红外线遥控不具有像无线电遥控那样穿过障碍物去控制被控对象的能力，所以，在设计红外线遥控器时，不必要像无线电遥控器那样，每套(发射器和接收器)要有不同的遥控频率或编码(否则，就会隔墙控制或干扰邻居的家用电器)，所以同类产品的红外线遥控器，可以有相同的遥控频率或编码，而不会出现遥控信号“串门”的情况。这对于大批量生产以及在家用电器上普及红外线遥控提供了极大的方便。由于红外线为不可见光，因此对环境影响很小，再由红外光波动波长远小于无线电波的波长，所以红外线遥控不会影响其他家用电器，也不会影响临近的无线电设备。

44.1.2 红外器件特性

红外遥控的情景中，必定会有一个红外发射端和红外接收端。在本实验中，正点原子的红外遥控器作为红外发射端，红外接收端就是板载的红外接收器，实物图可以查看 44.2.3 小节原理图部分。要使两者通信成功，收/发红外波长与载波频率需一致，在这里波长就是 940nm，载波频率就是 38kHz。

红外发射管也是属于二极管类，红外发射电路通常使用三极管控制红外发射器的导通或者截至，在导通的时候，红外发射管会发射出红外光，反之，就不会发射出红外光。虽然我们用肉眼看不到红外光，但是我们借助手机摄像头就能看到红外光。但是红外接收管的特性是当接收到红外载波信号时，OUT 引脚输出低电平；假如没有接收到红外载波信号时，OUT 引脚输出高电平。

红外载波信号其实就是由一个个红外载波周期组成。在频率为 38KHz 下，红外载波周期约等于 26.3us（1s / 38KHz ≈ 26.3us）。在一个红外载波发射周期里，发射红外光时间 8.77us 和不发射红外光 17.53us，发射红外光的占空比一般为 1/3。相对的，整个周期内不发射红外光，就是载波不发射周期。在红外遥控器内已经把载波和不载波信号处理好，我们需要做的就是识别遥控器按键发射出的信号，信号也是遵循某种协议的。

44.1.3 红外编解码协议介绍

红外遥控的编码方式目前广泛使用的是：PWM（脉冲宽度调制）的 NEC 协议和 Philips PPM（脉冲位置调制）的 RC-5 协议的。开发板配套的遥控器使用的是 NEC 协议，其特征如下：

1、8 位地址和 8 位指令长度；
2、地址和命令 2 次传输（确保可靠性）；
3、PWM 脉冲位置调制，以发射红外载波的占空比代表“0”和“1”；

4、载波频率为 38Khz；

5、位时间为 1.125ms 或 2.25ms；

在 NEC 协议中，如何为协议中的数据‘0’或者‘1’？这里分开红外接收器和红外发射器。

红外发射器：发送协议数据‘0’ = 发射载波信号 560us + 不发射载波信号 560us

发送协议数据‘1’ = 发射载波信号 560us + 不发射载波信号 1680us

红外发射器的位定义如下图所示：

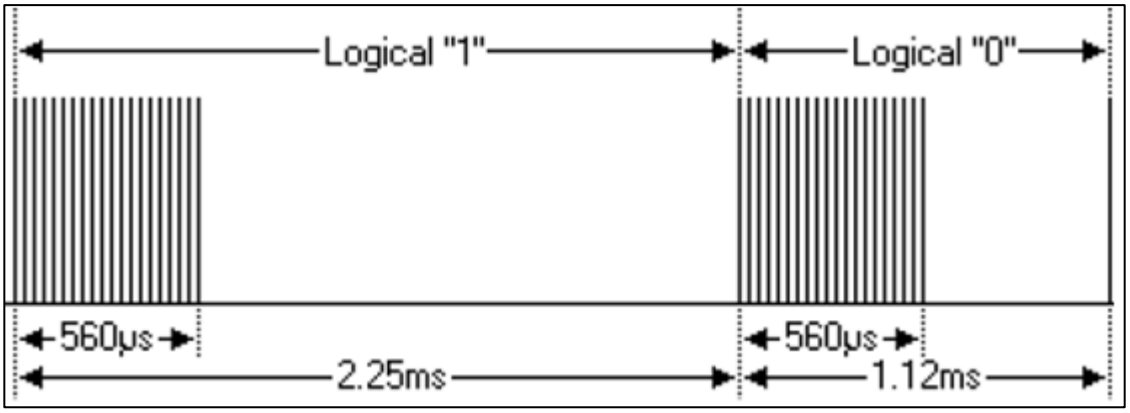


图 44.1.3.1 红外发射器位定义图

红外接收器：接收到协议数据‘0’ = 560us 低电平 + 560us 高电平

接收到协议数据‘1’ = 560us 低电平 + 1680us 高电平

红外接收器的位定义如下图所示：

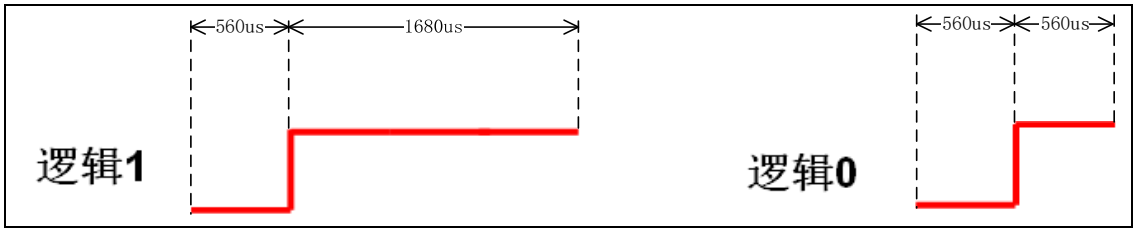


图 44.1.3.2 红外接收器位定义图

NEC 遥控指令的数据格式为：同步码头、地址码、地址反码、控制码、控制反码。同步码由一个 9ms 的低电平和一个 4.5ms 的高电平组成，地址码、地址反码、控制码、控制反码均是 8 位数据格式。按照低位在前，高位在后的顺序发送。采用反码是为了增加传输的可靠性（可用于校验）。

我们遥控器的按键▽按下时，从红外接收头端收到的波形如图 44.1.1 所示：

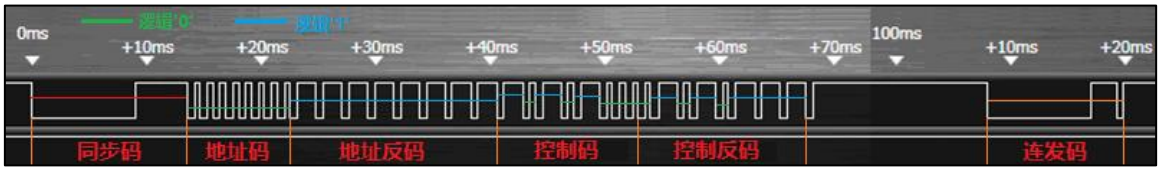


图 44.1.3.3 按键▽所对应的红外波形

从上图中可以看到，其地址码为 0，控制码为 21（正确解码后 00010101）。可以看到在 100ms 之后，我们还收到了几个脉冲，这是 NEC 码规定的连发码(由 9ms 低电平+2.25ms 高电平+0.56ms 低电平+97.94ms 高电平组成)，如果在一帧数据发送完毕之后，按键仍然没有放开，则发射重复码，即连发码可以通过统计连发码的次数来标记按键按下的长短/次数。

第二十一章我们曾经介绍过利用输入捕获来测量高电平的脉宽，本章解码红外遥控信号，刚好可以利用输入捕获的这个功能来实现遥控解码。关于输入捕获的介绍，请参考第二十一章的内容。

44.2 硬件设计

1. 例程功能

本实验开机在 LCD 上显示一些信息之后，即进入等待红外触发，如果接收到正确的红外信号，则解码，并在 LCD 上显示键值和所代表的意义，以及按键次数等信息。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
2）红外接收头
REMOTE_IN - PA8
3）正点原子红外遥控器
4）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
5）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)

3. 原理图

红外遥控接收头与 STM32 的连接关系，如下图所示：

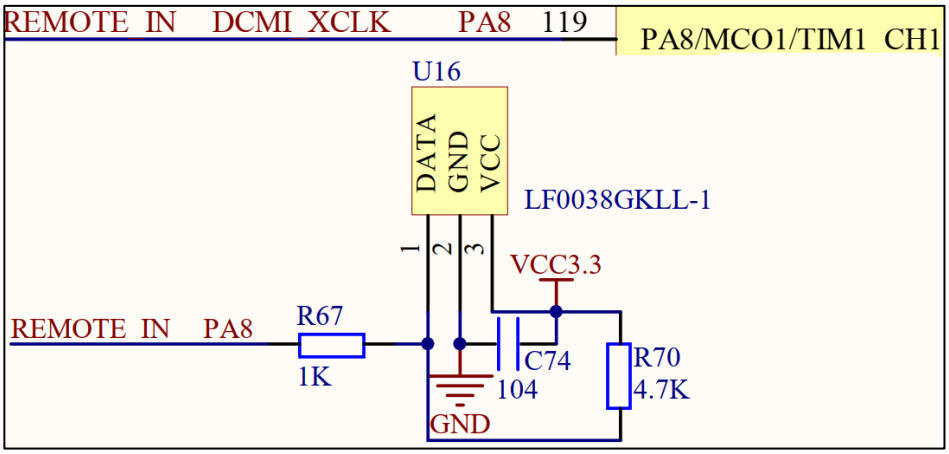


图 44.2.1 红外遥控接收头与 STM32 的连接电路图

红外遥控接收头连接在 STM32 的 PA8（TIM1_CH1）上。硬件上不需要变动，只要程序将 TIM1_CH1 设计为输入捕获，然后将收到的脉冲信号解码就可以了。不过需要注意：REMOTE_IN 和 DCMI_XCLK 共用了 PA8，所以他们不可以同时使用。

开发板配套的红外遥控器外观如图 44.2.1 所示：

图 44.2.2 红外遥控器

开发板上接收红外遥控器信号的红外管外观如图 44.2.3 所示。使用时需要遥控器有红外管的一端对准开发板上的红外管才能正确收到信号。

图 44.2.3 开发板上的红外接收管位置

44.3 程序设计

由于红外遥控实验采用的是定时器的输入捕获功能，所以这里大家就需要往前面定时器章节输入捕获实验中重温一下输入捕获功能的配置。下面我们也把红外遥控的配置步骤讲解一下。

红外遥控配置步骤

1）初始化 TIMx，设置 TIMx 的 ARR 和 PSC 等参数

HAL 库通过调用定时器输入捕获初始化函数 HAL_TIM_IC_Init 完成对定时器参数初始化。

注意：该函数会调用：HAL_TIM_IC_MspInit 函数来完成对定时器底层以及其输入通道 IO 的初始化，包括：定时器及 GPIO 时钟使能、GPIO 模式设置、中断设置等。

2）开启 TIMx 和输入通道的 GPIO 时钟，配置该 IO 口的复用功能输入

首先开启 TIMx 的时钟，然后配置 GPIO 为复用功能输出。本实验我们默认用到定时器 1 通道 1，对应 IO 是 PA8，它们的时钟开启方法如下：

```
__HAL_RCC_TIM1_CLK_ENABLE();                /* 使能定时器 1 */
__HAL_RCC_GPIOA_CLK_ENABLE();               /* 开启 GPIOA 时钟 */
```

IO 口复用功能是通过函数 HAL_GPIO_Init 来配置的。

3）设置 TIMx_CHy 的输入捕获模式，开启输入捕获

在 HAL 库中，定时器的输入捕获模式是通过 HAL_TIM_IC_ConfigChannel 函数来设置定时器某个通道为输入捕获通道，包括映射关系，输入滤波和输入分频等。

4）使能定时器更新中断，开启捕获功能以及捕获中断，配置定时器中断优先级

通过__HAL_TIM_ENABLE_IT 函数使能定时器更新中断。

通过 HAL_TIM_IC_Start_IT 函数使能定时器并开启捕获功能以及捕获中断。

通过 HAL_NVIC_EnableIRQ 函数使能定时器中断。

通过 HAL_NVIC_SetPriority 函数设置中断优先级。

5）编写中断服务函数

定时器 1 中断服务函数为：TIM1_IRQHandler，当发生中断的时候，程序就会执行中断服

务函数。HAL 库为了使用方便，提供了一个定时器中断通用处理函数 HAL_TIM_IRQHandler，该函数会调用一些定时器相关的回调函数，用于给用户处理定时器中断到了之后，需要处理的程序。本实验我们除了用到更新（溢出）中断回调函数 HAL_TIM_PeriodElapsedCallback 之外，还要用到捕获中断回调函数 HAL_TIM_IC_CaptureCallback。详见本例程源码。

44.3.1 程序流程图

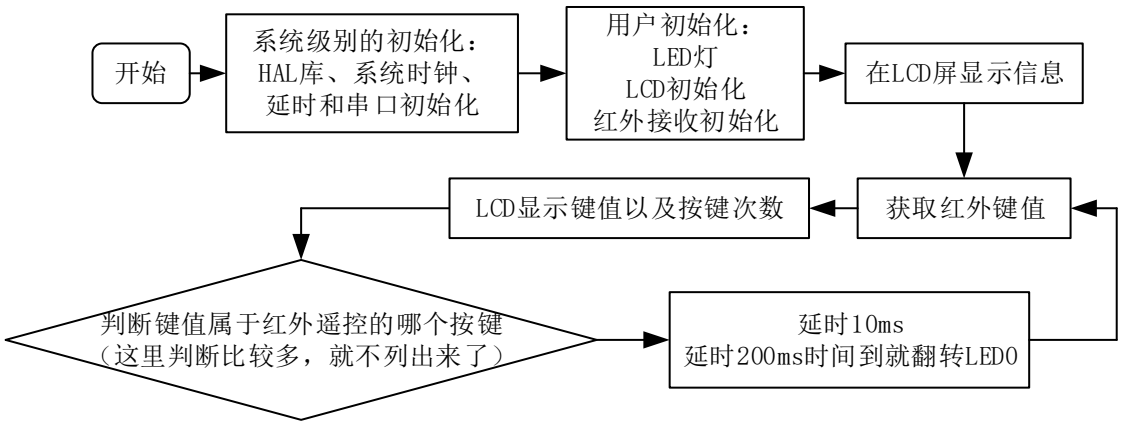


图 44.3.2.1 红外遥控实验程序流程图

44.3.2 程序解析

1. REMOTE 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。REMOTE 驱动源码包括两个文件：remote.c 和 remote.h。 remote.h 和前面定时器输入捕获功能的.h 头文件代码相似，这里就不介绍了，详见本例程源码。

下面我们直接介绍 remote.c 的程序，下面是与红外遥控初始化相关的函数，其定义如下：

```
TIM_HandleTypeDef g_tim_handle;       /* 定时器 1 句柄 */
/**
 * @brief       红外遥控初始化
 * @note        设置 IO 以及定时器的输入捕获
 * @param       无
 * @retval      无
 */
void remote_init(void)
{
    TIM_IC_InitTypeDef tim_ic_init_handle;

    g_tim_handle.Instance = REMOTE_IN_TIMX;     /* 通用定时器 1 */
    g_tim_handle.Init.Prescaler = 200-1;        /* 预分频器,1M 的计数频率,1us 加 1 */
    g_tim_handle.Init.CounterMode = TIM_COUNTERMODE_UP;    /* 向上计数器 */
    g_tim_handle.Init.Period = 10000;                      /* 自动装载值 */
    g_tim_handle.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
    HAL_TIM_IC_Init(&g_tim_handle);

    /* 初始化 TIM1 输入捕获参数 */
    tim_ic_init_handle.ICPolarity = TIM_ICPOLARITY_RISING;      /* 上升沿捕获 */
    tim_ic_init_handle.ICSelection = TIM_ICSELECTION_DIRECTTI; /* 映射到 TI1 上 */
    tim_ic_init_handle.ICPrescaler = TIM_ICPSC_DIV1;            /* 不分频 */
    tim_ic_init_handle.ICFilter = 0x03;                 /* 8 个定时器时钟周期滤波 */
    HAL_TIM_IC_ConfigChannel(&g_tim_handle, &tim_ic_init_handle,
                             REMOTE_IN_TIMX_CHY);           /* 配置 TIM1 通道 1 */
   HAL_TIM_IC_Start_IT(&g_tim_handle, REMOTE_IN_TIMX_CHY);/* 开始捕获 TIM 通道 */
    __HAL_TIM_ENABLE_IT(&g_tim_handle, TIM_IT_UPDATE);     /* 使能更新中断 */
```

```
}

/**
 * @brief       定时器 1 底层驱动，时钟使能，引脚配置
 * @param       htim:定时器句柄
 * @note        此函数会被 HAL_TIM_IC_Init()调用
 * @retval      无
 */
void HAL_TIM_IC_MspInit(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == REMOTE_IN_TIMX)
    {
    GPIO_InitTypeDef gpio_init_struct;

    REMOTE_IN_GPIO_CLK_ENABLE();             /* 红外接入引脚 GPIO 时钟使能 */
    REMOTE_IN_TIMX_CHY_CLK_ENABLE();         /* 定时器时钟使能 */

    gpio_init_struct.Pin = REMOTE_IN_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;              /* 复用推挽 */
    gpio_init_struct.Pull = GPIO_PULLUP;                  /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;   /* 高速 */
    gpio_init_struct.Alternate = REMOTE_IN_GPIO_AF;       /* 复用为 TIM1 通道 1 */
    HAL_GPIO_Init(REMOTE_IN_GPIO_PORT,&gpio_init_struct);/* 初始化 TIM 通道引脚 */

   /* 设置中断优先级，抢占优先级 1，子优先级 3 */
    HAL_NVIC_SetPriority(REMOTE_IN_TIMX_IRQn, 1, 2);
    HAL_NVIC_EnableIRQ(REMOTE_IN_TIMX_IRQn);              /* 开启 TIM1 中断 */

    /* 如果是通用定时器 ,不需要设置 CC 中断 设置中断优先级，抢占优先级 1，子优先级 2 */
    HAL_NVIC_SetPriority(REMOTE_IN_TIMX_CC_IRQn, 1, 3);
    HAL_NVIC_EnableIRQ(REMOTE_IN_TIMX_CC_IRQn);           /* 开启 TIM CC 中断 */
    }
}
```

remote_init 函数主要是对红外遥控使用到的定时器 1 和定时器通道 1 进行相关配置，关于定时器 1 通道 1 的 IO 放在回调函数 HAL_TIM_IC_MspInit 中初始化。

在 remote_init 函数中，通过调用 HAL_TIM_IC_Init 函数初始化定时器的 ARR 和 PSC 等参数；通过调用 HAL_TIM_IC_ConfigChannel 函数配置映射关系，滤波和分频等；最后调用 HAL_TIM_IC_Start_IT 和__HAL_TIM_ENABLE_IT 分别使能捕获通道和使能定时器中断。

在 HAL_TIM_IC_MspInit 函数中主要通过 HAL_GPIO_Init 函数对定时器输入通道的 GPIO 口进行配置，最后还需要设置中断抢占优先级和响应优先级。

通过上面两个函数的配置后，定时器的输入捕获已经初始化完成，接下来我们还需要作一些接收处理，下面先介绍下面这三个变量。

```
/* 遥控器接收状态
 * [7]  : 收到了引导码标志
 * [6]  : 得到了一个按键的所有信息
 * [5]  : 保留
 * [4]  : 标记上升沿是否已经被捕获
 * [3:0]: 溢出计时器
 */
uint8_t g_remote_sta = 0;
uint32_t g_remote_data = 0;  /* 红外接收到的数据 */
uint8_t  g_remote_cnt = 0;   /* 按键按下的次数 */
```

这三个变量用于辅助实现高电平的捕获。其中 g_remote_sta 是用来记录捕获状态，这个变量，我们把它当成一个寄存器来使用。对其各位进行定义，描述如下表所示：

g_remote_sta				
bit7	bit6	bit5	bit4	Bit3~0
收到引导码	得到一个按键所有信息	保留	标记上升沿是否已经被捕获	溢出计时器

表 44.3.2.1 g_remote_sta 各位描述

变量 g_remote_data 用于存放红外接收到的数据，而 g_remote_cnt 是存放按键按下的次数。

下面开始看中断服务函数里面的逻辑程序，HAL_TIM_IRQHandler 函数会调用下面两个回调函数，我们的逻辑代码就是放在回调函数里，函数的定义如下：

```
/**
 * @brief       定时器输入捕获中断回调函数
 * @param       htim:定时器句柄
 * @retval      无
 */
void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == REMOTE_IN_TIMX)
    {
        uint16_t dval;  /* 下降沿时计数器的值 */

        if (RDATA)      /* 上升沿捕获 */
        {   /* 一定要先清除原来的设置 */
            __HAL_TIM_SET_CAPTUREPOLARITY(&g_tim_handle,
                    REMOTE_IN_TIMX_CHY,TIM_INPUTCHANNELPOLARITY_FALLING)
            /* 设置为下降沿捕获 */
            __HAL_TIM_SET_COUNTER(&g_tim_handle, 0);  /* 清空定时器值 */
            g_remote_sta |= 0X10;                     /* 标记上升沿已经被捕获 */
        }
        else            /* 下降沿捕获 */
        {   /* 读取 CCR1 也可以清 CC1IF 标志位 */
           dval = HAL_TIM_ReadCapturedValue(&g_tim_handle, REMOTE_IN_TIMX_CHY);
            /* 一定要先清除原来的设置 */
            TIM_SET_CAPTUREPOLARITY(&g_tim1_handle, REMOTE_IN_TIMX_CHY,
                   TIM_ICPOLARITY_RISING);    /* 配置 TIM1 通道 1 上升沿捕获 */

            if (g_remote_sta & 0X10)          /* 完成一次高电平捕获 */
            {
                if (g_remote_sta & 0X80)      /* 接收到了引导码 */
                {
                    if (dval > 300 && dval < 800)       /* 560 为标准值,560us */
                    {
                        g_remote_data <<= 1;            /* 左移一位. */
                        g_remote_data |= 0;             /* 接收到 0 */
                    }
                    else if(dval > 1400 && dval < 1800)/* 1680 为标准值,1680us */
                    {
                        g_remote_data <<= 1;            /* 左移一位*/
                        g_remote_data |= 1;             /* 接收到 1 */
                    }
                    else if (dval > 2200 && dval < 2600)
                    {   /* 得到按键键值增加的信息 2250 为标准值 2.25ms */
                        g_remote_cnt++;          /* 按键次数增加 1 次 */
                        g_remote_sta &= 0XF0;    /* 清空计时器 */
                    }
                }
                else if (dval > 4200 && dval < 4700)     /* 4500 为标准值 4.5ms */
                {
                    g_remote_sta |= 1 << 7;    /* 标记成功接收到了引导码 */
                    g_remote_cnt = 0;          /* 清除按键次数计数器 */
                }
            }
            g_remote_sta &= ~(1<<4);
        }
    }
}
```

现在我们来介绍一下，捕获高电平脉宽的思路：首先，设置 TIM1_CH1 捕获上升沿，然后等待上升沿中断到来，当捕获到上升沿中断，设置该通道为下降沿捕获，清除 TIM1_CNT 寄

存器的值，最后把 g_remote_sta 的位 4 置 1，表示已经捕获到高电平，等待下降沿到来。当下降沿到来的时候，读取此时定时器计数器的值到 dval 中并设置该通道为上升沿捕获，然后判断 dval 的值属于哪个类型（引导码，数据 0，数据 1 或者重发码），相对应就把 g_remote_sta 相关位进行调整。例如，一开始识别为引导码的情况，就需要把 g_remote_sta 第 7 位置 1。当检测到重复码，就把按键次数增量存放在 g_remote_cnt 变量中。

```
/**
 * @brief       定时器更新中断回调函数
 * @param       htim:定时器句柄
 * @retval      无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == REMOTE_IN_TIMX)
    {
        if (g_remote_sta & 0x80)        /* 上次有数据被接收到了 */
        {
            g_remote_sta &= ~0X10;      /* 取消上升沿已经被捕获标记 */

            if ((g_remote_sta & 0X0F) == 0X00)
            {
                g_remote_sta |= 1 << 6;     /* 标记已经完成一次按键的键值信息采集 */
            }

            if ((g_remote_sta & 0X0F) < 14)
            {
                g_remote_sta++;
            }
            else
            {
                g_remote_sta &= ~(1 << 7);    /* 清空引导标识 */
                g_remote_sta &= 0XF0;         /* 清空计数器 */
            }
        }
    }
}
```

定时器更新中断回调函数主要是对标志位进行管理。在函数内通过 g_remote_sta 标志的判断，主要思路就是：在接收到引导码的前提下，对 g_remote_sta 状态进行判断并在符合条件下进行运算，这里主要就做了两件事：标记完成一次按键信息采集和是否松开按键（即没有接收到数据）。当完成一次按键信息采集时，g_remote_data 已经存放了控制反码、控制码、地址反码、地址码。那为啥可以检测是否可以松开按键？是因为接收到重发码的情况下会清空计数器，所以说当我们松开按键接收不到重发码时，溢出中断次数增多最终会导致 g_remote_sta&0x0f 值大于 14，进而就可以把引导码，计数器清空，便于下一次的接收。

```
/**
 * @brief       处理红外按键(类似按键扫描)
 * @param       无
 * @retval      0 , 没有任何按键按下
 *              其他, 按下的按键键值
 */
uint8_t remote_scan(void)
{
    uint8_t sta = 0;
    uint8_t t1, t2;

    if (g_remote_sta & (1 << 6))                /* 得到一个按键的所有信息了 */
    {
        t1 = g_remote_data>>24;                 /* 得到地址码 */
        t2 = (g_remote_data >> 16) & 0xff;      /* 得到地址反码 */

        if ((t1 == (uint8_t)~t2) && t1 == REMOTE_ID)
        {   /* 检验遥控识别码(ID)及地址 */
```

```
            t1 = (g_remote_data >> 8;
            t2 = g_remote_data;

            if (t1 == (uint8_t)~t2)
            {
                sta = t1;           /* 键值正确 */
            }
        }

        if ((sta == 0) || ((g_remote_sta & 0X80) == 0))
       {   /* 按键数据错误/遥控已经没有按下了 */
            g_remote_sta &= ~(1 << 6);      /* 清除接收到有效按键标识 */
            g_remote_cnt = 0;               /* 清除按键次数计数器 */
        }
    }

    return sta;
}
```

remote_scan 函数是用来扫描解码结果的，相当于我们的按键扫描，输入捕获解码的红外数据，通过该函数传送给其他程序。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t key;
    uint8_t t = 0;
    char *str = 0;

    sys_cache_enable();                         /* 打开 L1-Cache */
    HAL_Init();                                 /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);         /* 设置时钟, 400Mhz */
    delay_init(400);                            /* 延时初始化 */
    usart_init(115200);                         /* 串口初始化为 115200 */
    mpu_memory_protection();                    /* 保护相关存储区域 */
    led_init();                                 /* 初始化 LED */
    key_init();                                 /* 初始化 KEY */
    sdram_init();                               /* 初始化 SDRAM */
    lcd_init();                                 /* 初始化 LCD */
    remote_init();                              /* 红外接收初始化 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "REMOTE TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEYVAL:", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEYCNT:", RED);
    lcd_show_string(30, 150, 200, 16, 16, "SYMBOL:", RED);

    while (1)
    {
        key = remote_scan();
        if (key)
        {
            lcd_show_num(86, 110, key, 3, 16, BLUE);          /* 显示键值 */
            lcd_show_num(86, 130, g_remote_cnt, 3, 16, BLUE); /* 显示按键次数 */

            switch (key)
            {
                case 0:     str = "ERROR";          break;
                case 69:    str = "POWER";          break;
                case 70:    str = "UP";             break;
                case 64:    str = "PLAY";           break;
```

```
                case 71:    str = "ALIENTEK";       break;
                case 67:    str = "RIGHT";          break;
                case 68:    str = "LEFT";           break;
                case 7:     str = "VOL-";           break;
                case 21:    str = "DOWN";           break;
                case 9:     str = "VOL+";           break;
                case 22:    str = "1";              break;
                case 25:    str = "2";              break;
                case 13:    str = "3";              break;
                case 12:    str = "4";              break;
                case 24:    str = "5";              break;
                case 94:    str = "6";              break;
                case 8:     str = "7";              break;
                case 28:    str = "8";              break;
                case 90:    str = "9";              break;
                case 66:    str = "0";              break;
                case 74:    str = "DELETE";         break;
            }
            lcd_fill(86, 150, 116 + 8 * 8, 170 + 16, WHITE);  /* 清楚之前的显示 */
            lcd_show_string(86, 150, 200, 16, 16, str, BLUE); /* 显示 SYMBOL */
        }
        else
        {
            delay_ms(10);
        }

        t++;
        if (t == 20)
        {
            t = 0;
            LED0_TOGGLE();  /* LED0 闪烁 */
        }
    }
}
```

main 函数代码比较简单，主要是通过 remote_scan 函数获得红外遥控输入的数据（控制码），然后显示在 LCD 上面。正点原子红外遥控器按键对应的控制码图如下图所示。

图 44.3.2.1 红外遥控器按键对应的控制码图（十六进制数）

特别注意：上图中的控制码数值是十六进制的，而我们代码中使用的是十进制的表示方式。此外，正点原子红外遥控器的地址码是 0。

44.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 44.4.1 所示：

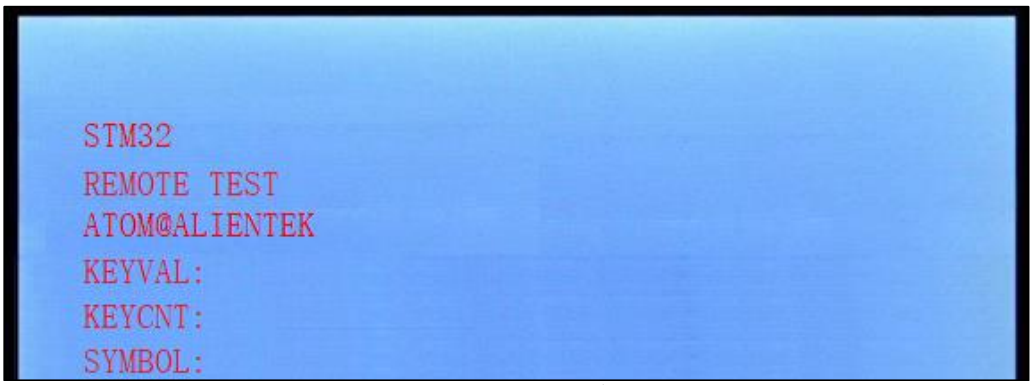


图 44.4.1 程序运行效果图

此时我们通过遥控器按下不同的按键，则可以看到 LCD 上显示了不同按键的键值以及按键次数和对应的遥控器上的符号。如图 44.4.2 所示：

图 44.4.2 解码成功

第四十五章 DS18B20 数字温度传感器实验

本章，我们将介绍 STM32 如何读取外部温度传感器的温度，来得到较为准确的环境温度。我们将学习单总线技术，通过它来实现 STM32 和外部温度传感器 DS18B20 的通信，并把从温度传感器得到的温度显示在 LCD 上。

本章分为如下几个小节：

45.1 DS18B20 及其时序简介
45.2 硬件设计
45.3 程序设计
45.4 下载验证

45.1 DS18B20 及其时序简介

45.1.1 DS18B20 简介

DS18B20 是由 DALLAS 半导体公司推出的一种“单总线”接口的温度传感器。与传统的热敏电阻等测温元件相比，它是一种新型的体积小、适用电压宽、与微处理器接口简单的数字化温度传感器。单总线结构具有简洁且经济的特点，可使用户轻松地组建传感器网络，从而为测量系统的构建引入全新的概念，测试温度范围为-55~+125℃，精度为±0.5℃。现场温度直接以单总线的数字方式传输，大大提高了系统的抗干扰性。它能直接读出被测温度，并且可根据实际要求通过简单的编程实现 9~12 位的数字值读数方式。它工作在 3~5.5V 的电压范围，采用多种封装形式，从而使系统设置灵活、方便，设定分辨率以及用户设定的报警温度存储在 EEPROM 中，掉电后依然保存。其内部结构如图 45.1.1 所示：

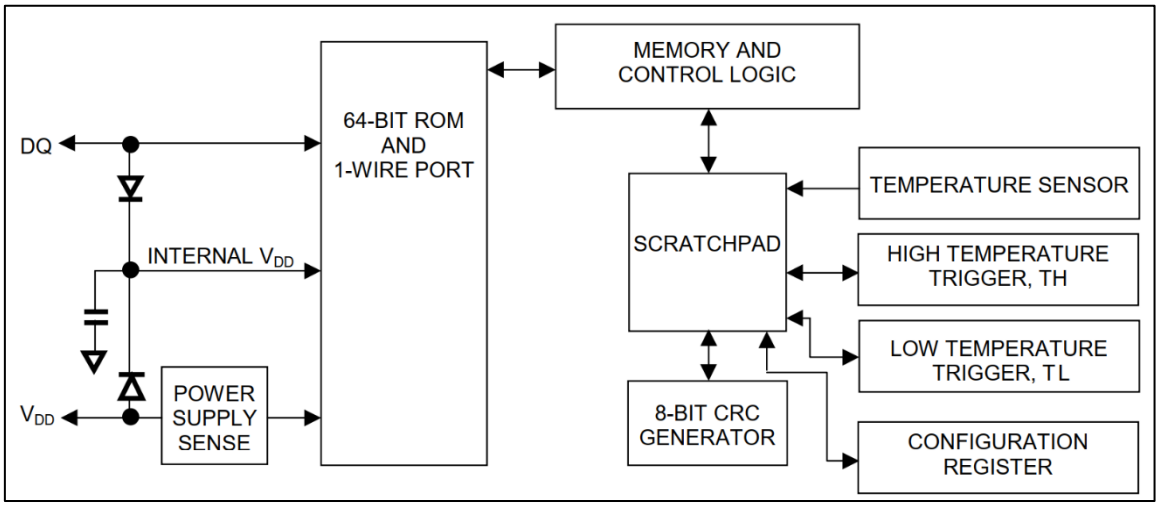


图 45.1.1.1 DS18B20 内部结构图

ROM 中的 64 位序列号是出厂前被标记好的，它可以看作使该 DS18B20 的地址序列码，每个 DS18B20 的 64 位序列号均不相同。64 位 ROM 的排列是：前 8 位是产品家族码，接着 48 位是 DS18B20 的序列号，最后 8 位是前面 56 位的循环冗余校验码(CRC=X8+X5+X4+1）。ROM 作用是使每一个 DS18B20 都各不相同，这样设计可以允许一根总线上挂载多个 DS18B20 模块同时工作且不会引起冲突。

45.1.2 DS18B20 时序简介

所有单总线器件要求采用严格的信号时序，以保证数据的完整性。DS18B20 共有 6 种信号类型：复位脉冲、应答脉冲、写 0、写 1、读 0 和读 1。所有这些信号，除了应答脉冲以外，都是由主机发出同步信号。并且发送所有的命令和数据都是字节的低位在前。这里我们简单介绍

这几个信号的时序。

1） 复位脉冲和应答脉冲

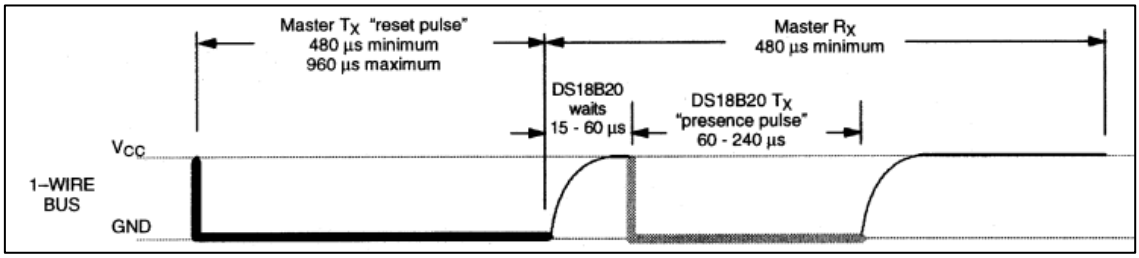


图 45.1.2.1 复位脉冲和应答脉冲时序图

单总线上的所有通信都是以初始化序列开始。主机输出低电平，保持低电平时间至少要在480us，以产生复位脉冲。接着主机释放总线，4.7K 的上拉电阻将单总线拉高，延时时间要在15~60us，并进入接收模式（Rx）。接着 DS18B20 拉低总线 60~240us，以产生低电平应答脉冲。

2） 写时序

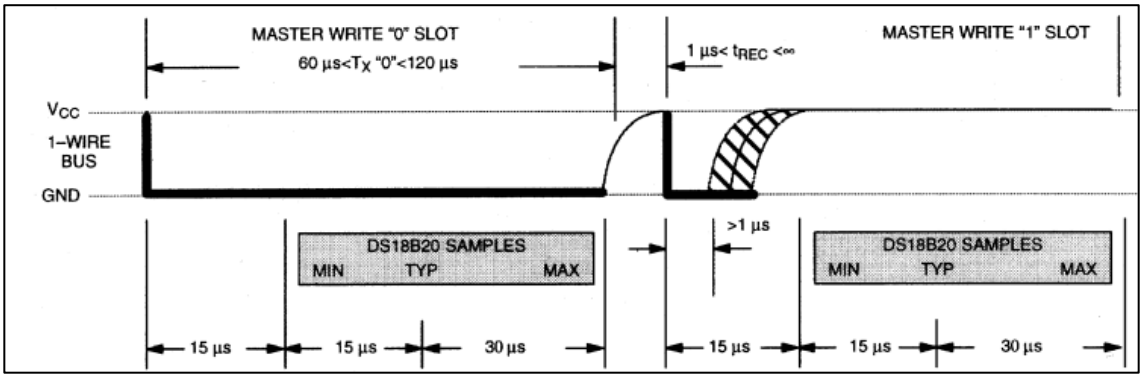


图 45.1.2.2 写时序图

写时序包括写 0 时序和写 1 时序。所有写时序至少需要 60us，且在两次独立的写时序之间至少需要 1us 的恢复时间，两种写时序均起始于主机拉低总线。写 1 时序：主机输出低电平，延时 2us，然后释放总线，延时 60us。写 0 时序：主机输出低电平，延时 60us，然后释放总线延时 2us。

3） 读时序

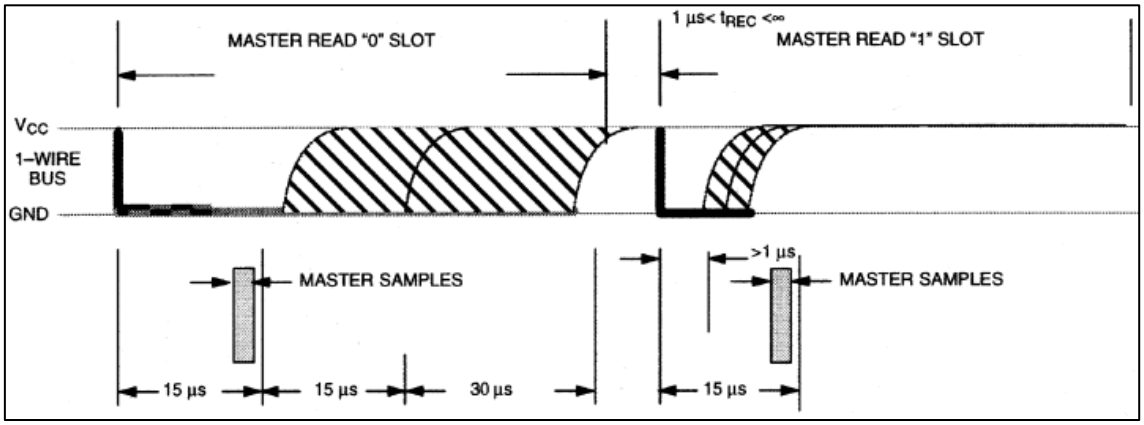


图 45.1.2.3 读时序图

单总线器件仅在主机发出读时序时，才向主机传输数据，所以，在主机发出读数据命令后，必须马上产生读时序，以便从机能够传输数据。所有读时序至少需要 60us，且在 2 次独立的读时序之间至少需要 1us 的恢复时间。每个读时序都由主机发起，至少拉低总线 1us。主机在读时序期间必须释放总线，并且在时序起始后的 15us 之内采样总线状态。典型的读时序过程为：主机输出低电平延时 2us，然后主机转入输入模式延时 12us，然后读取单总线当前的电平，然后延时 50us。

在了解单总线时序之后，我们来看一下 DS18B20 的典型温度读取过程，DS18B20 的典型温度读取过程为：复位→发 SKIP ROM（0xCC）→发开始转换命令（0x44）→延时→复位→发送 SKIP ROM 命令（0xCC）→发送存储器命令（0xBE）→连续读取两个字节数据（即温度）→结束。

45.2 硬件设计

1. 例程功能

本实验开机的时候先检测是否有 DS18B20 存在，如果没有，则提示错误。只有在检测到 DS18B20 之后才开始读取温度并显示在 LCD 上，如果发现了 DS18B20，则程序每隔 100ms 左右读取一次数据，并把温度显示在 LCD 上。 LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 – PB1
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）PCF8574T
5）DS18B20 数字温度传感器(接在 PB12 上)

3. 原理图

DS18B20 接口与 STM32 的连接关系，如下图所示：

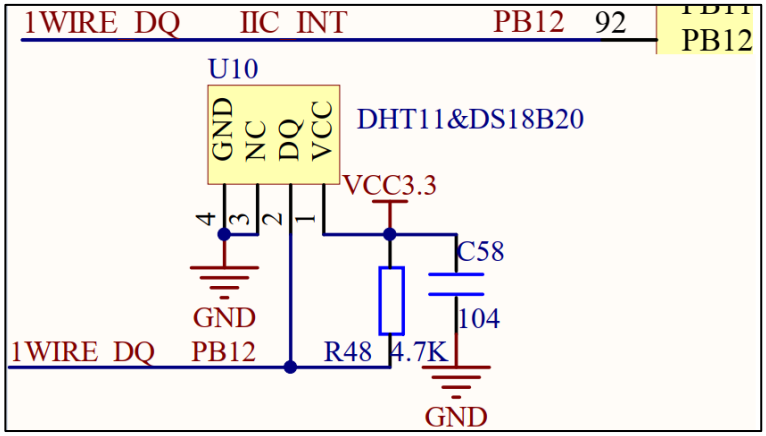


图 45.2.1 DS18B20 接口与 STM32 的连接电路图

从上图可以看出，我们使用的是 STM32 的 PB12 来连接 U10 的 DQ 引脚，图中 U10 为 DHT11（数字温湿度传感器）和 DS18B20 共用的一个接口，DHT11 我们将在下一章介绍。

DS18B20 只用到 U10 的 3 个引脚（U10 的 1、2 和 3 脚），将 DS18B20 传感器插入到这个上面就可以通过 STM32 来读取 DS18B20 的温度了。连接示意图如图 45.2.2 所示：

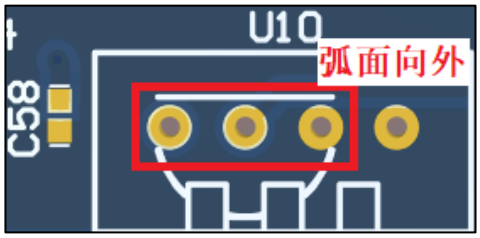


图 45.2.2 DS18B20 连接示意图

从上图可以看出，DS18B20 的平面部分（有字的那面）应该朝内，而曲面部分朝外。然后插入如图所示的三个孔内。

45.3 程序设计

DS18B20 实验中使用的是单总线协议，用到的是 HAL 中 GPIO 相关函数，前面也有介绍到，这里就不做展开了。下面介绍一下如何驱动 DS18B20。

DS18B20 配置步骤

1）使能 DS18B20 数据线对应的 GPIO 时钟。

本实验中 DS18B20 的数据线引脚是 PB12，因此需要先使能 GPIOB 的时钟，代码如下：

```
__HAL_RCC_GPIOB_CLK_ENABLE();       /* PB 口时钟使能 */
```

2）设置对应 GPIO 工作模式（开漏输出）

本实验 GPIO 使用开漏输出模式，通过函数 HAL_GPIO_Init 设置实现。

3）参考单总线协议，编写信号函数（复位脉冲、应答脉冲、写 0/1、读 0/1）

复位脉冲：主机发出低电平，保持低电平时间至少 480us。

应答脉冲：DS18B20 拉低总线 60~240us，以产生低电平应答信号。

写 1 信号：主机输出低电平，延时 2us，然后释放总线，延时 60us。

写 0 信号：主机输出低电平，延时 60us，然后释放总线，延时 2us。

读 0/1 信号：主机输出低电平延时 2us，然后主机转入输入模式延时 12us，然后读取单总线当前的电平，然后延时 50us。

4）编写 DS18B20 的读和写函数

基于写 1bit 数据和读 1bit 数据的基础上，编写 DS18B20 写 1 字节和读 1 字节函数。

5）编写 DS18B20 获取温度函数

参考 DS18B20 典型温度读取过程，编写获取温度函数。

45.3.1 程序流程图

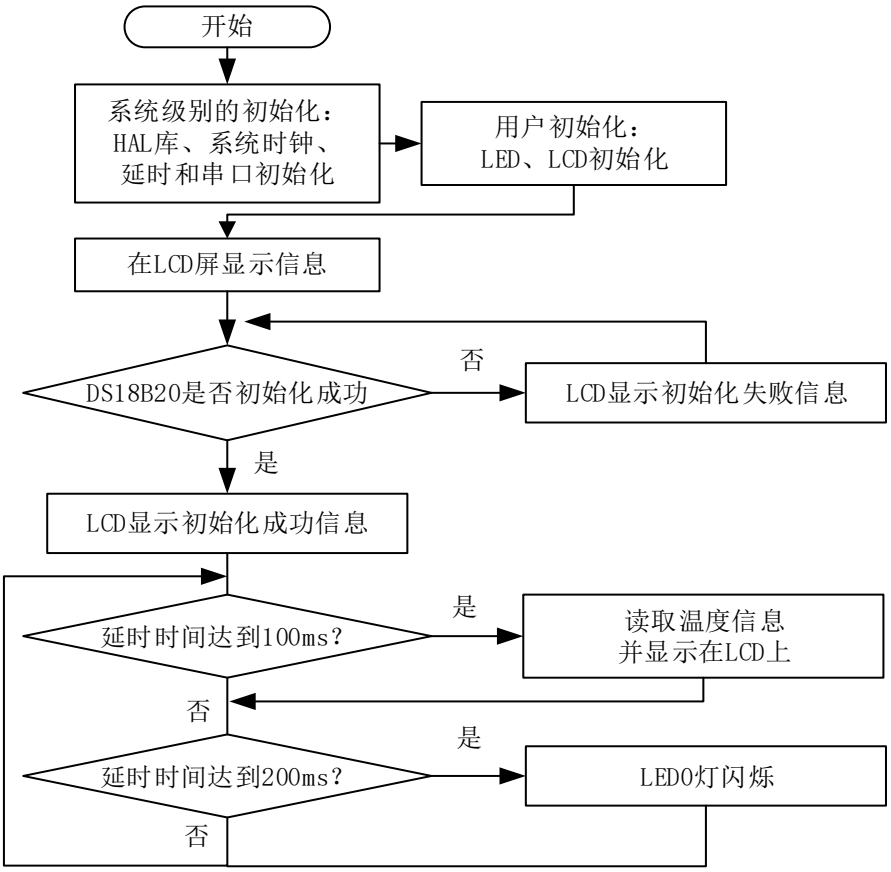


图 45.3.1.1 DS18B20 实验程序流程图

45.3.2 程序解析

1.DS18B20 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。DS18B20 驱动源码包括两个文件：ds18b20.c 和 ds18b20.h。

首先我们先看一下 ds18b20.h 头文件里面的内容，其定义如下：

```
/* DS18B20 引脚 定义 */
#define DS18B20_DQ_GPIO_PORT                    GPIOB
#define DS18B20_DQ_GPIO_PIN                     GPIO_PIN_12
#define DS18B20_DQ_GPIO_CLK_ENABLE()
            do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)    /* PB 口时钟使能 */

/* IO 操作函数 */
#define DS18B20_DQ_OUT(x)    do{ x ? \
 HAL_GPIO_WritePin(DS18B20_DQ_GPIO_PORT, DS18B20_DQ_GPIO_PIN, GPIO_PIN_SET) : \
 HAL_GPIO_WritePin(DS18B20_DQ_GPIO_PORT, DS18B20_DQ_GPIO_PIN, GPIO_PIN_RESET);
\ }while(0)   /* 数据端口输出 */
#define DS18B20_DQ_IN      HAL_GPIO_ReadPin(DS18B20_DQ_GPIO_PORT,
                                            DS18B20_DQ_GPIO_PIN)      /* 数据端口输入 */
```

在 ds18b20.h 的操作跟 IIC 实验代码很类似，主要对用到 GPIO 口进行宏定义，以及宏定义 IO 操作函数，方便时序函数调用。

下面我们直接介绍 ds18b20.c 的程序，首先介绍的是 DS18B20 传感器的初始化函数，其定义如下：

```
/**
 * @brief       初始化 DS18B20 的 IO 口  DQ  同时检测 DS18B20 的存在
 * @param       无
 * @retval      0, 正常
 *              1, 不存在/不正常
 */
uint8_t ds18b20_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;

    DS18B20_DQ_GPIO_CLK_ENABLE();    /* 开启 DQ 引脚时钟 */

    gpio_init_struct.Pin = DS18B20_DQ_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_OD;     /* 开漏输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;             /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;   /* 高速 */
    /* 初始化 DS18B20_DQ 引脚 */
    HAL_GPIO_Init(DS18B20_DQ_GPIO_PORT, &gpio_init_struct);
    /* DS18B20_DQ 引脚模式设置,开漏输出,上拉, 这样就不用再设置 IO 方向了,
       开漏输出的时候(=1), 也可以读取外部信号的高低电平 */
    ds18b20_reset();
    return ds18b20_check();
}
```

在 ds18b20 的初始化函数中，主要对用到的 GPIO 口进行初始化，同时在函数最后调用复位函数和自检函数，这两个函数在后面会解释到。

下面介绍的是复位 DS18B20 函数和等待 DS18B20 的回应函数，它们的定义如下：

```
/**
 * @brief       复位 DS18B20
 * @param       data: 要写入的数据
 * @retval      无
 */
static void ds18b20_reset(void)
{
    DS18B20_DQ_OUT(0);    /* 拉低 DQ,复位 */
    delay_us(750);        /* 拉低 750us */
```

```
    DS18B20_DQ_OUT(1);    /* DQ=1, 释放复位 */
    delay_us(15);         /* 延迟 15US */
}

/**
 * @brief       等待 DS18B20 的回应
 * @param       无
 * @retval      0, DS18B20 正常
 *              1, DS18B20 异常/不存在
 */
uint8_t ds18b20_check(void)
{
    uint8_t retry = 0;
    uint8_t rval = 0;

    while (DS18B20_DQ_IN && retry < 200)     /* 等待 DQ 变低, 等待 200us */
    {
        retry++;
        delay_us(1);
    }

    if (retry >= 200)
    {
        rval = 1;
    }
    else
    {
        retry = 0;
        while (!DS18B20_DQ_IN && retry < 240)    /* 等待 DQ 变高, 等待 240us */
        {
            retry++;
            delay_us(1);
        }
        if (retry >= 240) rval = 1;
    }
    return rval;
}
```

以上两个函数分别代表着前面所说的复位脉冲与应答信号，大家可以对比前面的时序图进行理解。由于复位脉冲比较简单，所以这里不做展开。现在看一下应答信号函数，函数主要是对于 DS18B20 传感器的回应信号进行检测，对此判断其是否存在。函数的实现也是依据时序图进行逻辑判断，例如当主机发送了复位信号之后，按照时序，DS18B20 会拉低数据线 60~240us，同时主机接收最小时间为 480us，我们就依据这两个硬性条件进行判断，首先需要设置一个时限等待 DS18B20 响应，后面也设置一个时限等待 DS18B20 释放数据线拉高，满足这两个条件即 DS18B20 成功响应。

下面介绍的是写函数，其定义如下：

```
/**
 * @brief       写一个字节到 DS18B20
 * @param       data: 要写入的字节
 * @retval      无
 */
static void ds18b20_write_byte(uint8_t data)
{
    uint8_t j;
    for (j = 1; j <= 8; j++)
    {
        if (data & 0x01)
        {
            DS18B20_DQ_OUT(0);  /*  Write 1 */
            delay_us(2);
            DS18B20_DQ_OUT(1);
            delay_us(60);
        }
```

```
        else
        {
            DS18B20_DQ_OUT(0);  /*  Write 0 */
            delay_us(60);
            DS18B20_DQ_OUT(1);
            delay_us(2);
        }
        data >>= 1;             /* 右移,获取高一位数据 */
    }
}
```

通过形参决定是写 1 还是写 0，按照前面对写时序的分析，我们可以很清晰知道写函数的逻辑处理。

有写函数肯定就有读函数，下面介绍的是读函数，其定义如下：

```
/**
 * @brief       从 DS18B20 读取一个位
 * @param       无
 * @retval      读取到的位值：0 / 1
 */
static uint8_t ds18b20_read_bit(void)
{
    uint8_t data = 0;
    DS18B20_DQ_OUT(0);
    delay_us(2);
    DS18B20_DQ_OUT(1);
    delay_us(12);

    if (DS18B20_DQ_IN)
    {
        data = 1;
    }
    delay_us(50);
    return data;
}

/**
 * @brief       从 DS18B20 读取一个字节
 * @param       无
 * @retval      读到的数据
 */
static uint8_t ds18b20_read_byte(void)
{
    uint8_t i, b, data = 0;
    for (i = 0; i < 8; i++)
    {
        b = ds18b20_read_bit();/* DS18B20 先输出低位数据 ,高位数据后输出 */
        data |= b << i;        /* 填充 data 的每一位 */
    }
    return data;
}
```

在这里 ds18b20_read_bit 函数从 DS18B20 处读取 1 位数据，在前面已经对读时序也进行了详细的分析，所以这里也不展开解释了。

下面介绍读取温度函数，其定义如下：

```
/**
 * @brief       开始温度转换
 * @param       无
 * @retval      无
 */
static void ds18b20_start(void)
{
    ds18b20_reset();
    ds18b20_check();
    ds18b20_write_byte(0xcc);   /*  skip rom */
    ds18b20_write_byte(0x44);   /*  convert */
```

```
}
/**
 * @brief       从 ds18b20 得到温度值(精度：0.1C)
 * @param       无
 * @retval      温度值 （-550~1250）
 * @note        返回的温度值放大了 10 倍.
 *              实际使用的时候,要除以 10 才是实际温度.
 */
short ds18b20_get_temperature(void)
{
    uint8_t flag = 1;           /* 默认温度为正数 */
    uint8_t TL, TH;
    short temp;

    ds18b20_start();            /*  ds1820 start convert */
    ds18b20_reset();
    ds18b20_check();
    ds18b20_write_byte(0xcc);   /*  skip rom */
    ds18b20_write_byte(0xbe);   /*  convert */
    TL = ds18b20_read_byte();   /*  LSB */
    TH = ds18b20_read_byte();   /*  MSB */

    if (TH > 7)
    {   /* 温度为负，查看 DS18B20 的温度表示法与计算机存储正负数据的原理一致：
                正数补码为寄存器存储的数据自身，负数补码为寄存器存储值按位取反后+1
                所以我们直接取它实际的负数部分，但负数的补码为取反后加一，但考虑到低位可能+1
            后有进位和代码冗余，
                我们这里先暂时没有作+1 的处理，这里需要留意  */
        TH = ~TH;
        TL = ~TL;
        flag = 0;   /* 温度为负 */
    }

    temp = TH;      /* 获得高八位 */
    temp <<= 8;
    temp += TL;     /* 获得底八位 */

    /* 转换成实际温度 */
    if (flag == 0)
    {   /* 将温度转换成负温度，这里的+1 参考前面的说明 */
        temp = (double)(temp+1) * 0.625;
        temp = -temp;
    }
    else
    {
        temp = (double)temp * 0.625;
    }
    return temp;
}
```

在这里简单介绍一下上面用到的 RAM 指令：

跳过 ROM(0xCC)，该指令只适合总线只有一个节点，它通过允许总线上的主机不提供 64 位 ROM 序列号而直接访问 RAM，节省了操作时间。

温度转换(0x44)，启动 DS18B20 进行温度转换，结果存入内部 RAM。

读寄存器(0xBE)，读寄存器 9 个字节内容，该指令从 RAM 的第一个字节（字节 0）开始读取，直到九个字节（字节 8，CRC 值）被读出为止。如果不需要读出所有字节的内容，那么主机可以在任何时候发出复位信号以中止读操作。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
```

```
{
    uint8_t t = 0;
    short temperature;

    sys_cache_enable();                             /* 打开 L1-Cache */
    HAL_Init();                                     /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);             /* 设置时钟, 400Mhz */
    delay_init(400);                                /* 延时初始化 */
    usart_init(115200);                             /* 串口初始化为 115200 */
    mpu_memory_protection();                        /* 保护相关存储区域 */
    led_init();                                     /* 初始化 LED */
    sdram_init();                                   /* 初始化 SDRAM */
    lcd_init();                                     /* 初始化 LCD */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "DS18B20 TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    while (ds18b20_init())                  /* DS18B20 初始化 */
    {
        lcd_show_string(30, 110, 200, 16, 16, "DS18B20 Error", RED);
        delay_ms(200);
        lcd_fill(30, 110, 239, 130 + 16, WHITE);
        delay_ms(200);
    }

    lcd_show_string(30, 110, 200, 16, 16, "DS18B20 OK", RED);
    lcd_show_string(30, 130, 200, 16, 16, "Temp:    . C", BLUE);

    while (1)
    {
        if (t % 10 == 0)    /* 每 100ms 读取一次 */
        {
            temperature = ds18b20_get_temperature();
            if (temperature < 0)
            {
                lcd_show_char(30 + 40, 130, '-', 16, 0, BLUE);  /* 显示负号 */
                temperature = -temperature; /* 转为正数 */
            }
            else
            {
                lcd_show_char(30 + 40, 130, ' ', 16, 0, BLUE);  /* 去掉负号 */
            }
            /* 显示正数部分 */
            lcd_show_num(30 + 40 + 8, 130, temperature / 10, 2, 16, BLUE);
            /* 显示小数部分 */
            lcd_show_num(30 + 40 + 32, 130, temperature % 10, 1, 16, BLUE);
        }

        delay_ms(10);
        t++;

        if (t == 20)
        {
            t = 0;
            LED0_TOGGLE();  /* LED0 闪烁 */
        }
    }
}
```

主函数代码比较简单，一系列硬件初始化后，在循环中调用 ds18b20_get_temperature 函数获取温度值，然后显示在 LCD 上。

45.4 下载验证

假定 DS18B20 传感器已经接上去正确的位置，将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示当前的温度值的内容如图 45.4.1 所示：

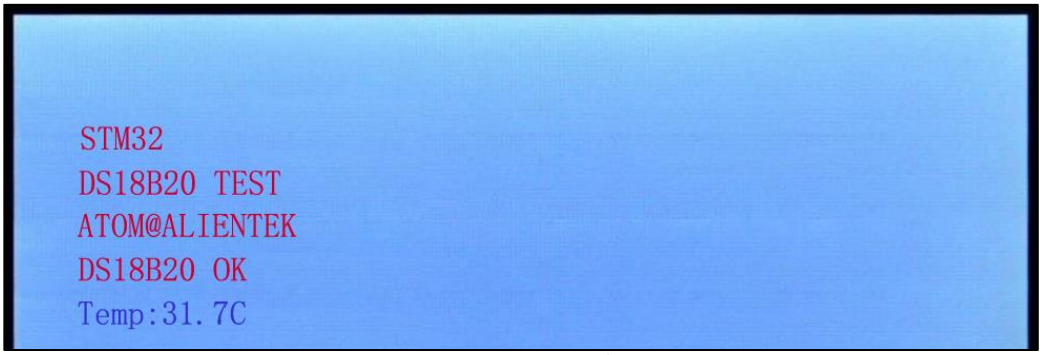


图 45.4.1 程序运行效果图

该程序还可以读取并显示负温度值，具备零下温度条件可以测试一下。

第四十六章 DHT11 数字温湿度传感器

本章，我们将介绍数字温湿度传感器 DHT11 的使用，与前一章的温度传感器相比，该传感器不但能测温度，还能测湿度。我们将学习如何获取 DHT11 传感器的温湿度数据，并把数据显示在 LCD 上。

本章分为如下几个小节：

46.1 DHT11 及其时序简介
46.2 硬件设计
46.3 程序设计
46.4 下载验证

46.1 DHT11 及时序简介

46.1.1 DHT11 简介

DHT11 是一款温湿度一体化的数字传感器。该传感器包括一个电阻式测湿元件和一个 NTC 测温元件，并与一个高性能 8 位单片机相连接。通过单片机等微处理器简单的电路连接就能够实时的采集本地湿度和温度。DHT11 与单片机之间能采用简单的单总线进行通信，仅仅需要一个 I/O 口。传感器内部湿度和温度数据 40Bit 的数据一次性传给单片机，数据采用校验和方式进行校验，有效的保证数据传输的准确性。DHT11 功耗很低，5V 电源电压下，工作平均最大电流 0.5mA。

DHT11 的技术参数如下：

- 工作电压范围：3.3V ~ 5.5V
- 工作电流：平均 0.5mA
- 输出：单总线数字信号
- 测量范围：湿度 5 ~ 95%RH，温度-20 ~ 60℃
- 精度：湿度±5%，温度±2℃
- 分辨率：湿度 1%，温度 0.1℃

DHT11 的管脚排列如图 46.1.1 所示：

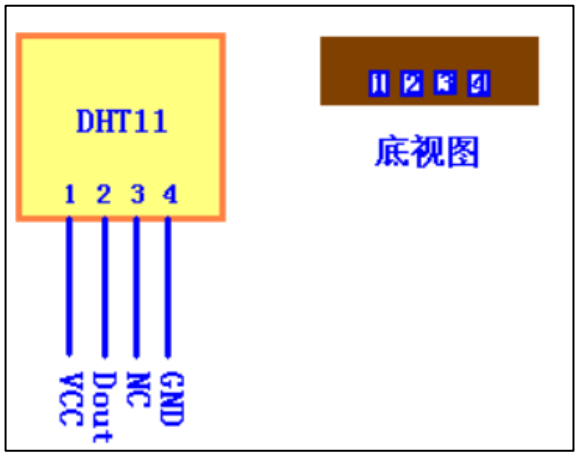


图 46.1.1.1 DHT11 管脚排列图

46.1.2 DHT11 时序简介

虽然 DHT11 与 DS18B20 类似，都是单总线访问，但是 DHT11 的访问，相对 DS18B20 来说简单很多。下面我们先来看看 DHT11 的数据结构。

DHT11 数字温湿度传感器采用单总线数据格式。即，单个数据引脚端口完成输入输出双向传输。其数据包由 5byte(40bit)组成。数据分小数部分和整数部分，一次完整的数据传输为 40bit，高位先处。DHT11 的数据格式为：8bit 湿度整数数据+8bit 湿度小数数据+8bit 温度整数数据+8bit 温度小数部分+8bit 校验和。其中校验和数据为前面四个字节相加。

传感器数据输出的是未编码的二进制数据。数据（湿度、温度、整数、小数）之间应该分开处理。例如，某次从 DHT11 读到的数据如图 46.1.2.1 所示：

byte4	byte3	byte2	byte1	byte0
00101101	00000000	00011100	00000000	01001001
整数	小数	整数	小数	校验和
湿度		温度		校验和

图 46.1.2.1 某次读取到 DHT11 数据

由以上数据就可得到湿度和温度的值，计算方法：

湿度 = byte4 . byte3 = 45.0(%RH)

温度 = byte2 . byte1 = 28.0(℃)

校验 = byte4 + byte3 + byte2 + byte1 = 73 (= 湿度 + 温度) （校验正确）

可以看出，DHT11 的数据格式十分简单的，DHT11 和 MCU 的一次通信最大为 34ms 左右，建议主机连续读取时间间隔不要小于 2s。

下面，我们介绍一下 DHT11 的传输时序。DHT11 的数据发送流程如图 46.1.2.2 所示：

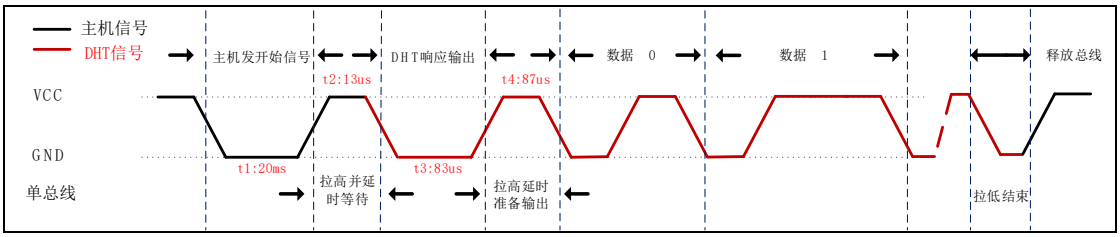


图 46.1.2.2 DHT11 数据发送流程图

首先主机发送开始信号，即：拉低数据线，保持 t1（至少 18ms）时间，然后拉高数据线 t2（10~35us）时间，然后读取 DHT11 的响应，正常的话，DHT11 会拉低数据线，保持 t3（78~88us）时间，作为响应信号，然后 DHT11 拉高数据线，保持 t4（80~92us）时间后，开始输出数据。

DHT11 输出数字‘0’时序如图 46.1.2.3 所示：

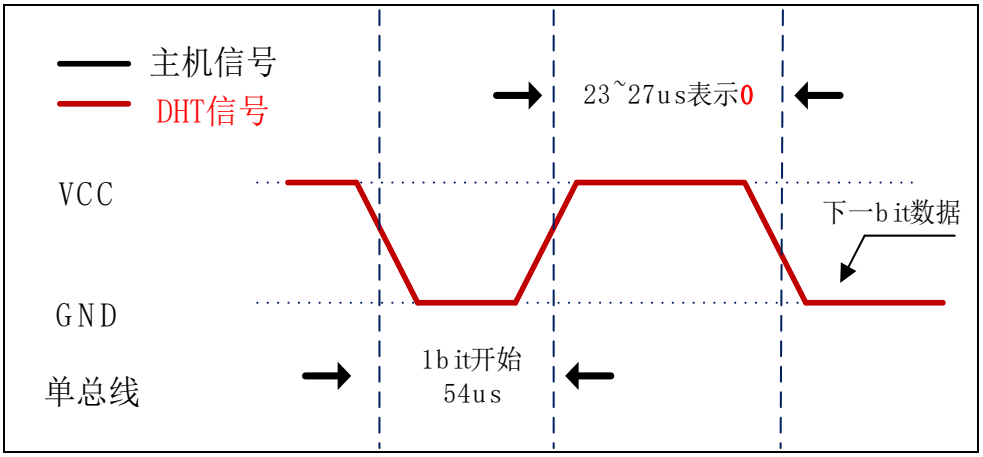


图 46.1.2.3 DHT11 数字‘0’时序图

DHT11 输出数字‘1’的时序如图 46.1.2.4 所示：

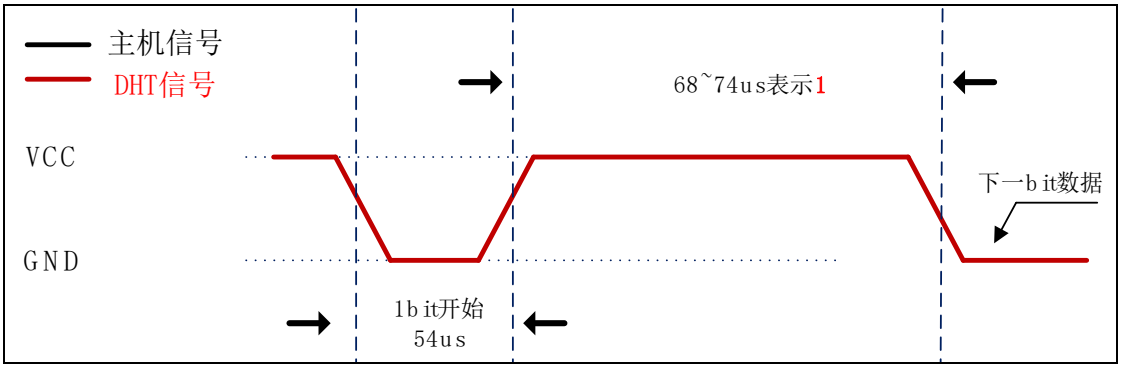


图 46.1.2.4 DHT11 输出数字‘1’时序图

DHT11 输出数字‘0’和‘1’时序，一开始都是 DHT11 拉低数据线 54us，后面拉高数据线保持的时间就不一样，数字‘0’就是 23~27us，而数字‘1’就是 68~74us。

通过以上了解，我们就可以通过 STM32H743 来实现对 DHT11 的读取了。DHT11 的介绍就到这里，更详细的介绍，请参考 DHT11 的数据手册。

46.2 硬件设计

1. 例程功能

本实验开机的时候先检测是否有 DHT11 存在，如果没有，则提示错误。只有在检测到 DHT11 之后才开始读取温湿度值，并显示在 LCD 上，如果发现了 DHT11，则程序每隔 100ms 左右读取一次数据，并把温湿度显示在 LCD 上。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0 - PB1
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）DHT11 传感器(接在 PB12 上)

3. 原理图

DHT11 接口与 STM32 的连接关系跟上一章节中 DS18B20 和 STM32 的关系是一样的，使用到的 GPIO 口是 PB12。这里原理图就不列出来了，可以翻看上一章节原理图。

DHT11 和 DS18B20 的接口是共用一个的，不过 DHT11 有 4 条腿，需要把 U10 的 4 个接口都用上，将 DHT11 传感器插入到这个上面就可以通过 STM32 来读取温湿度值了。连接示意图如图 46.2.1 所示：

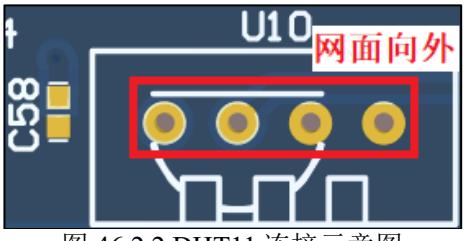


图 46.2.2 DHT11 连接示意图

这里要注意，将 DHT11 贴有字的一面朝内，而有很多孔的一面(网面)朝外，然后插入如图所示的四个孔内就可以了。

46.3 程序设计

DHT11 实验中使用的是单总线协议，用到的是 HAL 中 GPIO 相关函数，前面也有介绍到，

这里就不做展开了。下面介绍一下如何驱动 DHT11。

DHT11 配置步骤

1）使能 DHT11 数据线对应的 GPIO 时钟。

本实验中 DHT11 的数据线引脚是 PB12，因此需要先使能 GPIOB 的时钟，代码如下：

```
__HAL_RCC_GPIOB_CLK_ENABLE();       /* PB 口时钟使能 */
```

2）设置对应 GPIO 工作模式（开漏输出）

本实验 GPIO 使用开漏输出模式，通过函数 HAL_GPIO_Init 设置实现。

3）参考单总线协议，编写信号代码（复位脉冲、应答脉冲、读 0/1）

复位脉冲：拉低数据线，保持至少 18ms 时间，然后拉高数据线 10~35us 时间。

应答脉冲：DHT11 拉低数据线，保持 78~88us 时间。

读 0/1 信号：DHT11 拉低数据线延时 54us，然后拉高数据线延时一定时间，主机通过判断高电平时间得到 0 或者 1。

4）编写 DHT11 的读函数

基于读 1bit 数据的基础上，编写 DHT11 读 1 字节函数。

5）编写 DHT11 获取温度函数

参考 DHT11 典型温湿度读取过程，编写获取温湿度函数。

46.3.1 程序流程图

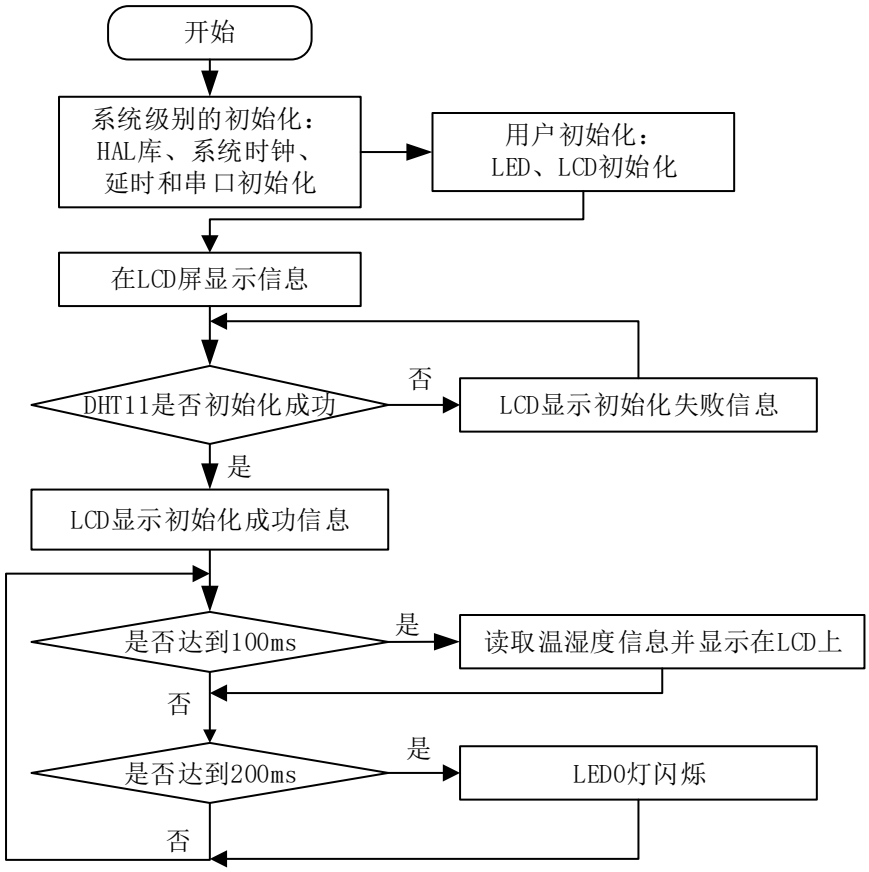


图 46.3.1.1 DHT11 实验程序流程图

46.3.2 程序解析

1.DHT11 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。DHT11 驱动源码包括两个文件：dht11.c 和 dht11.h。

首先我们先看一下 dht11.h 头文件里面的内容，其定义如下：

```
/* DHT11 引脚 定义 */
#define DHT11_DQ_GPIO_PORT                  GPIOB
#define DHT11_DQ_GPIO_PIN                   GPIO_PIN_12
#define DHT11_DQ_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)    /* PB 口时钟使能 */
/* IO 操作函数 */
#define DHT11_DQ_OUT(x)     do{ x ? \
 HAL_GPIO_WritePin(DHT11_DQ_GPIO_PORT, DHT11_DQ_GPIO_PIN, GPIO_PIN_SET) : \
 HAL_GPIO_WritePin(DHT11_DQ_GPIO_PORT, DHT11_DQ_GPIO_PIN, GPIO_PIN_RESET); \
                                  }while(0)           /* 数据端口输出 */
/* 数据端口输入 */
#define DHT11_DQ_IN    HAL_GPIO_ReadPin(DHT11_DQ_GPIO_PORT, DHT11_DQ_GPIO_PIN)
```

对 DHT11 的相关引脚以及 IO 操作进行宏定义，方便程序中调用。

下面我们直接介绍 dht11.c 的程序，首先先介绍一下 DHT11 传感器的初始化函数，其定义如下：

```
/**
 * @brief       初始化 DHT11 的 IO 口 DQ 同时检测 DHT11 的存在
 * @param       无
 * @retval      0, 正常
 *              1, 不存在/不正常
 */
uint8_t dht11_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;

    DHT11_DQ_GPIO_CLK_ENABLE();      /* 开启 DQ 引脚时钟 */

    gpio_init_struct.Pin = DHT11_DQ_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_OD;           /* 开漏输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;                   /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;    /* 高速 */
    HAL_GPIO_Init(DHT11_DQ_GPIO_PORT, &gpio_init_struct);  /* 初始化 DQ 引脚 */
   /* DHT11_DQ 引脚模式设置,开漏输出,上拉, 这样就不用再设置 IO 方向了, 开漏输出的时候(=1),
      也可以读取外部信号的高低电平 */

    dht11_reset();
    return dht11_check();
}
```

在 DHT11 的初始化函数中，主要对用到的 GPIO 口进行初始化，同时在函数最后调用复位函数和自检函数，这两个函数在后面会解释到。

下面介绍的是复位 DHT11 函数和等待 DHT11 的回应函数，它们的定义如下：

```
/**
 * @brief       复位 DHT11
 * @param       data: 要写入的数据
 * @retval      无
 */
static void dht11_reset(void)
{
    DHT11_DQ_OUT(0);        /* 拉低 DQ */
    delay_ms(20);           /* 拉低至少 18ms */
    DHT11_DQ_OUT(1);        /* DQ=1 */
    delay_us(30);           /* 主机拉高 10~35us */
```

```
}

/**
 * @brief       等待 DHT11 的回应
 * @param       无
 * @retval      0, DHT11 正常
 *              1, DHT11 异常/不存在
 */
uint8_t dht11_check(void)
{
    uint8_t retry = 0;
    uint8_t rval = 0;

    while (DHT11_DQ_IN && retry < 100)      /* DHT11 会拉低 83us */
    {
        retry++;
        delay_us(1);
    }
    if (retry >= 100)
    {
        rval = 1;
    }
    else
    {
        retry = 0;
        while (!DHT11_DQ_IN && retry < 100)     /* DHT11 拉低后会再次拉高 87us */
        {
            retry++;
            delay_us(1);
        }
        if (retry >= 100) rval = 1;
   }

    return rval;
}
```

以上两个函数分别代表着前面所说的复位脉冲与应答信号，大家可以对比前面的时序图进行理解。那么在上一章 DS18B20 的实验中，也对复位脉冲以及应答信号进行了详细的解释，大家也可以对比理解。

DHT11 与 DS18B20 有所不同，DHT11 是不需要写函数，只需要读函数即可，下面介绍从 DHT11 读取一个位函数和从 DHT11 读取一个字节函数，它们的定义如下：

```
/**
 * @brief       从 DHT11 读取一个位
 * @param       无
 * @retval      读取到的位值：0 / 1
 */
uint8_t dht11_read_bit(void)
{
    uint8_t retry = 0;

    while (DHT11_DQ_IN && retry < 100) /* 等待变为低电平 */
    {
        retry++;
        delay_us(1);
   }

   retry = 0;

    while (!DHT11_DQ_IN && retry < 100)     /* 等待变高电平 */
    {
        retry++;
        delay_us(1);
   }
```

```
    delay_us(40);           /* 等待 40us */

    if (DHT11_DQ_IN)        /* 根据引脚状态返回 bit */
    {
        return 1;
    }
    else
    {
        return 0;
    }
}

/**
 * @brief       从 DHT11 读取一个字节
 * @param       无
 * @retval      读到的数据
 */
static uint8_t dht11_read_byte(void)
{
    uint8_t i, data = 0;

    for (i = 0; i < 8; i++)             /* 循环读取 8 位数据 */
    {
        data <<= 1;                     /* 高位数据先输出，先左移一位 */
        data |= dht11_read_bit();       /* 读取 1bit 数据 */
    }

    return data;
}
```

在这里 dht11_read_bit 函数从 DHT11 处读取 1 位数据，大家可以对照前面的读时序图进行分析，读数字 0 和 1 的不同，在于高电平的持续时间，所以这个作为判断的依据。dht11_read_byte 函数就是调用一字节读取函数进行实现。

下面介绍读取温湿度函数，其定义如下：

```
/**
 * @brief       从 DHT11 读取一次数据
 * @param       temp: 温度值(范围:-20~60°)
 * @param       humi: 湿度值(范围:5%~95%)
 * @retval      0, 正常.
 *              1, 失败
 */
uint8_t dht11_read_data(uint8_t *temp, uint8_t *humi)
{
   uint8_t buf[5];
   uint8_t i;

    dht11_reset();

    if (dht11_check() == 0)
    {
        for (i = 0; i < 5; i++)      /* 读取 40 位数据 */
        {
            buf[i] = dht11_read_byte();
        }

        if ((buf[0] + buf[1] + buf[2] + buf[3]) == buf[4])
        {
            *humi = buf[0];
            *temp = buf[2];
        }
    }
    else
    {
        return 1;
```

```
    }

    return 0;
}
```

读取温湿度函数也是根据时序图进行实现的，在发送复位信号以及应答信号产生后，即可以读取 5Byte 数据进行处理，校验成功即读取数据有效成功。

2. main.c 代码

在 main.c 里面编写如下代码：

```
int main(void)
{
    uint8_t t = 0;
    uint8_t temperature;
    uint8_t humidity;

    sys_cache_enable();                             /* 打开 L1-Cache */
    HAL_Init();                                     /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);             /* 设置时钟, 400Mhz */
    delay_init(400);                                /* 延时初始化 */
    usart_init(115200);                             /* 串口初始化为 115200 */
    mpu_memory_protection();                        /* 保护相关存储区域 */
    led_init();                                     /* 初始化 LED */
    sdram_init();                                   /* 初始化 SDRAM */
    lcd_init();                                     /* 初始化 LCD */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "DHT11 TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    while (dht11_init())                            /* DHT11 初始化 */
    {
        lcd_show_string(30, 110, 200, 16, 16, "DHT11 Error", RED);
        delay_ms(200);
        lcd_fill(30, 110, 239, 130 + 16, WHITE);
        delay_ms(200);
    }

    lcd_show_string(30, 110, 200, 16, 16, "DHT11 OK", RED);
    lcd_show_string(30, 130, 200, 16, 16, "Temp:  C", BLUE);
    lcd_show_string(30, 150, 200, 16, 16, "Humi:  %", BLUE);

    while (1)
    {
        if (t % 10 == 0)    /* 每 100ms 读取一次 */
        {
            dht11_read_data(&temperature, &humidity);           /* 读取温湿度值 */
            lcd_show_num(30 + 40, 130, temperature, 2, 16, BLUE);/* 显示温度 */
            lcd_show_num(30 + 40, 150, humidity, 2, 16, BLUE);   /* 显示湿度 */
        }

        delay_ms(10);
        t++;
        if (t == 20)
        {
            t = 0;
            LED0_TOGGLE();  /* LED0 闪烁 */
        }
    }
}
```

主函数代码比较简单，一系列硬件初始化后，如果 DHT11 初始化成功，那么在循环中调用 dht11_get_temperature 函数获取温湿度值，每隔 100ms 读取数据并显示在 LCD 上。

46.4 下载验证

假定 DHT11 传感器已经接上去正确的位置，将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示当前的温度值的内容如图 46.4.1 所示：

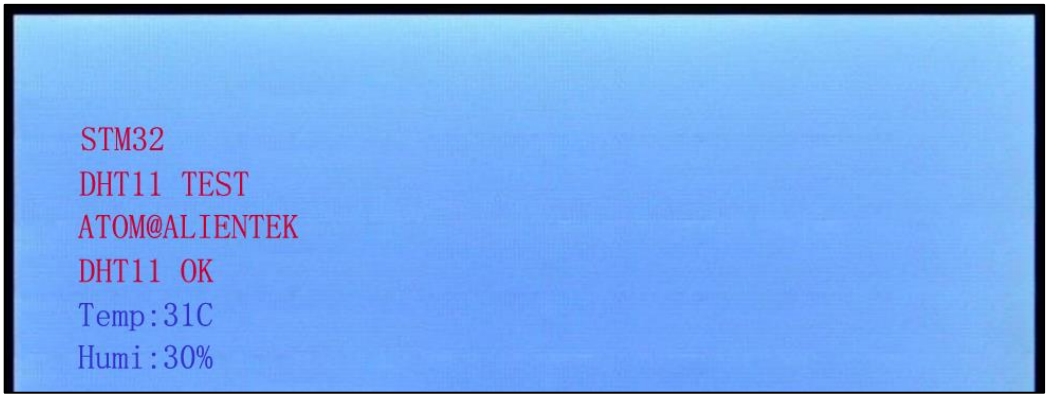


图 46.4.1 程序运行效果图

至此，本章实验结束。大家可以将本章通过 DHT11 读取到的温度值，和前一章的通过 DS18B20 读取到的温度值对比一下，看看哪个更准确？

第四十七章 气压计实验

阿波罗 STM32H743 开发板自带了 SPA06-003 传感器。本章，我们将介绍如何使用气压计 SPA06-003，将读取到的数据转换为气压值显示到 LCD。

本章分为如下几个小节：

47.1 SPA06-003 介绍
47.2 硬件设计
47.3 程序设计
47.4 下载验证

47.1 SPA06-003 介绍

47.1.1 SPA06-003 简介

SPA06-003 是一款由歌尔 (Goertek)公司推出的小型化、高精度、低功耗的数字气压传感器，同时集成了压力与温度测量功能。其压力传感元件采用电容式传感原理，即使在温度变化条件下也能确保高精度测量。紧凑的封装设计使其成为移动设备和可穿戴应用的理想选择。

该传感器内部集成的信号处理器能够将压力和温度传感元件的输出转换为 24 位数字结果。每个 SPA06-003 压力传感器在出厂前均经过单独校准，并内置了校准系数。用户可在应用中调用这些系数，将原始测量值转换为真实的压力和温度数值。

SPA06-003 内置 FIFO（先进先出）存储器，可存储最新的 32 组测量数据。利用此 FIFO 功能，主处理器能够在更长的间隔时间内保持睡眠状态，仅在需要时唤醒读取批量数据，从而显著降低系统整体功耗。传感器测量数据及校准系数均可通过标准的串行 I2C 或 SPI 接口访问。

SPA06-003 主要特点如下：

- 压力范围：300hPa~1100hPa
- 温度范围：-40℃~85℃
- 电源电压：1.7V~3.6V(VDD)，1.08V~3.6V(VDDIO)
- 封装：LGA 封装，带金属盖
- 相对精度：典型±0.03hPa，等效至±0.25 米
- 绝对精度：类型±0.3hPa

下面是传感器 3D 结构图：

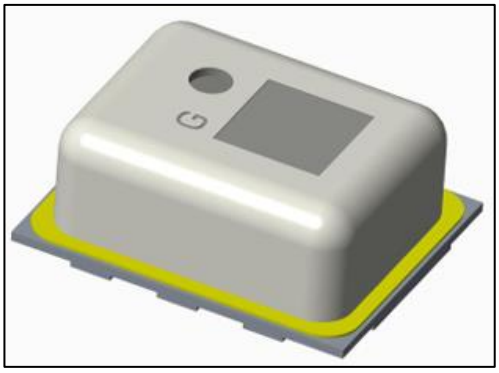


图 47.1.1.2 SPA06-003 传感器 3D 结构图

在气压检测方面，SPA06-003 传感器采用电容式 MEMS 膜片。气压变化导致膜片形变，改变上下电极间距，通过测量电容值变化计算压力值。而在温度检测方面，又了集成温度传感器（ASIC 或外部 MEMS），用于补偿气压测量的温漂。

SPA06-003 传感器，通过气压、温度传感元件采集数据，然后将原始模拟信号转换为 24 位数字量，存入 PRS_Bn（气压）和 TMP_Tn（温度）寄存器。 另外，值得一提的是该传感器同样支持 SPI 协议，感兴趣的读者可以自己研究，我们主要针对 IIC 协议进行讲解。

每一个 IIC 设备都有自己的设备地址，SPA06-003 也不例外，SPA06-003 的设备地址是包括不可编程部分和可编程部分，可编程部分是根据硬件引脚 SDO 所决定。设备地址最后一位用于设置数据的传输方向，即读操作/写操作，0 是写操作，1 是读操作，具体格式如下图 47.1.1.3 所示：

图 47.1.1.3 SPA06-003 设备地址格式图

本实验将 SDO 引脚接地，那么设备地址（不包含 R/W 位）为 0x76，上图中的“X”位需要置零。

47.1.2 SPA06-003 工作模式简介

接下来我们分别介绍 SPA06-003 三种工作模式：待机模式(Standby Mode)、指令模式(Command Mode)、后台模式(Background Mode)。

1.待机模式(Standby Mode)

当 SPA06-003 设置为该模式时，上电或复位后的默认模式，不执行任何测量。此时，所有寄存器和补偿系数均可访问。

2.指令模式(Command Mode)

当 SPA06-003 设置为指令模式后，跟据选定精度执行单次温度或压力测量。测量完成后传感器自动返回待机模式，结果存储于数据寄存器中。

3.后台模式(Background Mode)

当 SPA06-003 设置为后台模式后，根据配置的精度和速率连续执行压力/温度测量。其中，压力测量后立即进行温度测量。支持使用 FIFO（先进先出队列） 存储最近 32 次测量结果，最大限度减少主机访问次数以降低功耗。

47.1.3 SPA06-003 寄存器简介

在这里简单介绍一下 SPA06-003 比较重要的寄存器。

（1） 压力数据寄存器 (PRS_Bn)

存储 24 位压力测量值（二进制补码格式）。如果 FIFO 启用，则包含 FIFO 中的压力或温度结果。对 PRS_B2 的描述如下图所示：

7	6	5	4	3	2	1	0
PRS23	PRS22	PRS21	PRS20	PRS19	PRS18	PRS17	PRS16
r							

Field	Bits	Type	Description
PRS[23:16]	7:0	r	MSB of 24 bit 2´s complement pressure data.

图 47.1.3.1 SPA06-003 的 PRS_B2 位

PRS_B2(0x00)用于存储压力值的高 8 位(bit23~bit16)。对 PRS_B1 的描述如下图所示：

7	6	5	4	3	2	1	0
PRS15	PRS14	PRS13	PRS12	PRS11	PRS10	PRS9	PRS8-
r							

Field	Bits	Type	Description
PRS[15:8]	7:0	r	LSB of 24 bit 2´s complement pressure data.

图 47.1.3.2 SPA06-003 的 PRS_B1 位

PRS_B1(0x01)用于存储压力值的中 8 位(bit15~bit8)。对 PRS_B0 的描述如下图所示：

7	6	5	4	3	2	1	0
PRS7	PRS6	PRS5	PRS4	PRS3	PRS2	PRS1	PRS0
r							

Field	Bits	Type	Description
PRS[7:0]	7:0	r	XLSB of 24 bit 2´s complement pressure data.

图 47.1.3.3 SPA06-003 的 PRS_B0 位

PRS_B0(0x02)用于存储压力值的低 8 位(bit7~bit0)。

该寄存器访问类型为只读类型，复位值为 0x00。

（2） 温度数据寄存器 (TMP_Tn)

存储 24 位温度测量值（二进制补码格式）。除非 FIFO 启用，否则存储温度结果。对 TMP _B2 的描述如下图所示：

7	6	5	4	3	2	1	0
TMP23	TMP22	TMP21	TMP20	TMP19	TMP18	TMP17	TMP16
r							

Field	Bits	Type	Description
TMP[23:16]	7:0	r	MSB of 24 bit 2´s complement temperature data.

图 47.1.3.4 SPA06-003 的 TMP _B2 位

TMP _B2(0x03)用于存储温度值的高 8 位(bit23~bit16)。对 TMP _B1 的描述如下图所示：

7	6	5	4	3	2	1	0
TMP15	TMP14	TMP13	TMP12	TMP11	TMP10	TMP9	TMP8
r							

Field	Bits	Type	Description
TMP[15:8]	7:0	r	LSB of 24 bit 2´s complement temperature data.

图 47.1.3.5 SPA06-003 的 TMP _B1 位

TMP _B1(0x04)用于存储温度值的中 8 位(bit15~bit8)。对 TMP _B0 的描述如下图所示：

7	6	5	4	3	2	1	0
TMP7	TMP6	TMP5	TMP4	TMP3	TMP2	TMP1	TMP0
r							

Field	Bits	Type	Description
TMP[7:0]	7:0	r	XLSB of 24 bit 2´s complement temperature data.

图 47.1.3.6 SPA06-003 的 TMP _B0 位

TMP _B0(0x05)用于存储温度值的低 8 位(bit7~bit0)。

该寄存器访问类型为只读类型，复位值为 0x00。

（3） 压力配置寄存器 (PRS_CFG))，地址：0x06。

该寄存器用于配置压力测量速率（PM_RATE）和分辨率（PM_PRC，即过采样率）。对 PRS_CFG 的描述如下图所示：

7	6	5	4	3	2	1	0
-PM_RATE[3:0]				PM_PRC[3:0]			
-	rw			rw			

Field	Bits	Type	Description
PM_RATE[3:0]	7:4	rw	Pressure measurement rate: 0000 - 1 measurements pr. sec. 0001 - 2 measurements pr. sec. 0010 - 4 measurements pr. sec. 0011 - 8 measurements pr. sec. 0100 - 16 measurements pr. sec. 0101 - 32 measurements pr. sec. 0110 - 64 measurements pr. sec. 0111 - 128 measurements pr. sec. 1000 - 25/16 sample/sec 1001 - 25/8 sample/sec 1010 - 25/4 sample/sec 1011 - 25/2 sample/sec 1100 - 25 sample/sec 1101 - 50 sample/sec 1110 - 100 sample/sec 1111 - 200 sample/sec Applicable for measurements in Background mode only
PM_PRC[3:0]	3:0	rw	Pressure oversampling rate: 0000 - Single. (Low Precision) 0001 - 2 times (Low Power). 0010 - 4 times. 0011 - 8 times. 0100 *)- 16 times (Standard). 0101 *) - 32 times. 0110 *) - 64 times (High Precision). 0111 *) - 128 times. 1xxx – Reserved

图 47.1.3.7 SPA06-003 的 PRS_CFG 压力配置寄存器

该寄存器的 bit7~bit4 为读写位，用于设置压力测量速率，但仅后台模式有效。bit3~bit0 为读写位，用于设置压力过采样率。

（4） 温度配置寄存器 (TMP_CFG))，地址：0x07。

该寄存器用于配置温度测量速率（TMP_RATE）和分辨率（TMP_PRC）。对 TMP _CFG 的描述如下图所示：

7	6	5	4	3	2	1	0
TMP_RATE[3:0]				-TMP_PRC[3:0]			
rw		rw		-		rw	

Field	Bits	Type	Description
TMP_RATE[3:0]	7:4	rw	Temperature measurement rate: 0000 - 1 measurement pr. sec. 0001 - 2 measurements pr. sec. 0010 - 4 measurements pr. sec. 0011 - 8 measurements pr. sec. 0100 - 16 measurements pr. sec. 0101 - 32 measurements pr. sec. 0110 - 64 measurements pr. sec. 0111 - 128 measurements pr. sec. 1001 - 25/8 sample/sec 1010 - 25/4 sample/sec 1011 - 25/2 sample/sec 1100 - 25 sample/sec 1101 - 50 sample/sec 1110 - 100 sample/sec 1111 - 200 sample/sec Applicable for measurements in Background mode only
TMP_PRC[3:0]	3:0	rw	Temperature oversampling (precision): 0000 - single. (Default) - Measurement time 3.6 ms. Note: Following are optional, and may not be relevant: 0001 - 2 times. 0010 - 4 times. 0011 - 8 times. 0100 - 16 times. 0101 - 32 times. 0110 - 64 times.. 0111 - 128 times. 1xxx – Reserved

图 47.1.3.8 SPA06-003 的 TMP _CFG 温度配置寄存器

该寄存器的 bit7~bit4 为读写位，用于设置温度测量速率，但仅后台模式有效。bit3~bit0 为读写位，用于设置温度过采样率。

（5） 传感器工作模式与状态寄存器(MEAS_CFG))，地址：0x08。

该寄存器用于设置测量模式，并指示传感器状态。对 MEAS _CFG 的描述如下图所示：

7	6	5	4	3	2	1	0
COEF_RDY	SENSOR_RDY	TMP_RDY	PRS_RDY	TMP_EXT	MEAS_CTR		
r	r	r	r		rw		

Field	Bits	Type	Description
COEF_RDY	7	r	Coefficients will be read to the Coefficients Registers after start- up: 0 - Coefficients are not available yet. 1 - Coefficients are available.
SENSOR_RDY	6	r	The pressure sensor is running through self-initialization after start-up. 0 - Sensor initialization not complete 1 - Sensor initialization complete It is recommend not to start measurements until the sensor has completed the self-initialization.
TMP_RDY	5	r	Temperature measurement ready 1 - New temperature measurement is ready. Cleared when temperature measurement is read.
PRS_RDY	4	r	Pressure measurement ready 1 - New pressure measurement is ready. Cleared when procurement measurement is read.
TMP_EXT	3		0 - Internal temperature sensor (of ASIC) recommended 1- External temperature sensor (of pressure sensor MEMS element if any)
MEAS_CTRL	2:0	rw	Set measurement mode and type: Standby Mode 000 - Idle / Stop background measurement Command Mode 001 - Pressure measurement 010 - Temperature measurement 011 - na. 100 - na. Background Mode 101 - Continuous pressure measurement 110 - Continuous temperature measurement 111 - Continuous pressure and temperature measurement

图 47.1.3.9 SPA06-003 的传感器工作模式与状态寄存器

该寄存器的 bit7 为只读位，用于系数就绪标志(为 1 时系数可读)。bit6 为只读位，压力传感器在启动后通过自初始化运行(1=完成)，建议在传感器完成自初始化之前不要开始测量。bit5 为只读位，温度测量就绪(读取后清零)。bit4 为只读位，压力测量就绪(读取后清零)。bit3 为读写位，用于温度传感器选择(0=内部 ASIC，1=外部 MEMS)。bit2~bit0 为读写位，用于设置设置工作模式，分别为：

- 000: 待机模式（停止后台测量）
- 001: 指令模式（单次压力测量）
- 010: 指令模式（单次温度测量）
- 101: 后台模式（连续压力测量）
- 110: 后台模式（连续温度测量）
- 111: 后台模式（连续压力+温度测量）

（6） 中断与 FIFO 配置寄存器(CFG_REG)，地址：0x09。

该寄存器用于配置中断行为、数据偏移和 FIFO 使能。对 CFG_REG 的描述如下图所示：

7	6	5	4	3	2	1	0
INT_HL	INT_FIFO	INT_TMP	INT_PRS	T_SHIFT	P_SHIFT	FIFO_EN	SPI_MODE
rw	rw	rw	rw	rw	rw	rw	rw

Field	Bits	Type	Description
INT_HL	7	rw	Interrupt (on SDO pin) active level: 0 - Active low. 1 - Active high.
INT_FIFO	6	rw	Generate interrupt when the FIFO is full: 0 - Disable. 1 - Enable.
INT_TMP	5	rw	Generate interrupt when a temperature measurement is ready: 0 - Disable. 1 - Enable.
INT_PRS	4	rw	Generate interrupt when a pressure measurement is ready: 0 - Disable. 1 - Enable.
T_SHIFT	3	rw	Temperature result bit-shift 0 - no shift. 1 - shift result right in data register. Note: Must be set to '1' when the oversampling rate is >8 times.
P_SHIFT	2	rw	Pressure result bit-shift 0 - no shift. 1 - shift result right in data register. Note: Must be set to '1' when the oversampling rate is >8 times.
FIFO_EN	1	rw	Enable the FIFO: 0 - Disable. 1 - Enable.
SPI_MODE	0	rw	Set SPI mode: 0-4-wire interface. 1-3-wire interface.

图 47.1.3.10 SPA06-003 的中断与 FIFO 配置寄存器

该寄存器的 bit7 为读写位，中断有效电平（0=低电平，1=高电平）。bit6 为读写位，FIFO 满中断使能（1=启用）。bit5 为读写位，温度就绪中断使能（1=启用）。bit4 为读写位，压力就绪中断使能（1=启用）。bit3 为读写位，温度结果右移使能（过采样>8 次时必须置 1）。bit2 为读写位，压力结果右移使能（过采样>8 次时必须置 1）。bit1 为读写位，FIFO 使能(1=启用)。bit0 为读写位，SPI 接口模式（0=4 线 SPI，1=3 线 SPI）。

（7） 产品与版本 ID 寄存器 (ID)，地址：0x0D。

该寄存器用于读取产品 ID 和修订版本。对 ID 的描述如下图所示：

7	6	5	4	3	2	1	0
REV_ID				PROD_ID			
r				r			

Field	Bits	Type	Description
REV_ID	7:4	r	Revision ID
PROD_ID	3:0	r	Product ID

图 47.1.3.11 SPA06-003 的产品与版本 ID 寄存器

该寄存器的 bit7~bit4 为只读位，存储修订版本 ID。bit3~bit0 为只读位，存储产品 ID。

47.1.4 SPA06-003 工作时序简介

在阿波罗 STM32H743 开发板上，SDO 是接 GND 的，所以 SPA06-003 的设备地址是 0x76（不含最低位）。在前面我们已经介绍过 IIC 总线的基本读写，所以我们只介绍 SPA06-003 的数据传输时序。

下面把实验中用到的数据时序讲解一下，分别是对 SPA06-003 的写时序和读时序。SPA06-003 写寄存器时序图如图 47.1.4.1 所示：

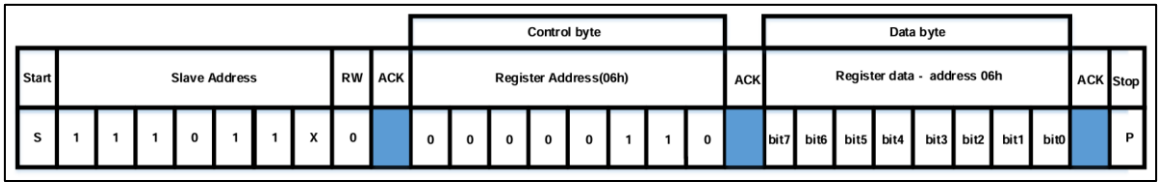


图 47.1.4.1 SPA06-003 写时序图

上图展示的是主机向 SPA06-003 写操作时序图，该过程的写入操作如下：首先发送写模式下的从机地址（RW='0'），此时从机地址为 111011X0（其中'X'由 SDO 引脚状态决定）。随后主机循环发送寄存器地址与寄存器数据配对序列。本次操作以停止条件结束。

说完寄存器写入方式之后，下面来看一下图 47.1.4.2 关于 SPA06-003 的读寄存器时序。

图 47.1.4.2 SPA06-003 读寄存器时序图

SPA06-003 读取寄存器的操作流程如下：首先以写模式发送寄存器地址（从机地址 111011X0），随后必须产生停止条件或重复起始条件。接着以读模式（RW='1'）访问从机地址 111011X1，此后从机将自动递增寄存器地址并持续发送数据，直至主机返回无应答信号（NOACKM）并发出停止条件。

47.2 硬件设计

1. 例程功能

本实验开机的时候先检测是否有 SPA06-003 存在，如果没有，则提示错误。在检测到 SPA06-003 之后（必须进入一次校准，不然数据不是准确的数据，初始化中已经加入校准函数）

就会不断的读取 SPA06-003 的数据，同时在 LCD 上面显示相关信息。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED0 – PB1
2）独立按键
KEY0 – PH3
KEY_UP –PH2
3）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
5）气压计 SPA06-003

3. 原理图

我们主要来看看 SPA06-003 和开发板的连接，如图 47.2.1 所示：

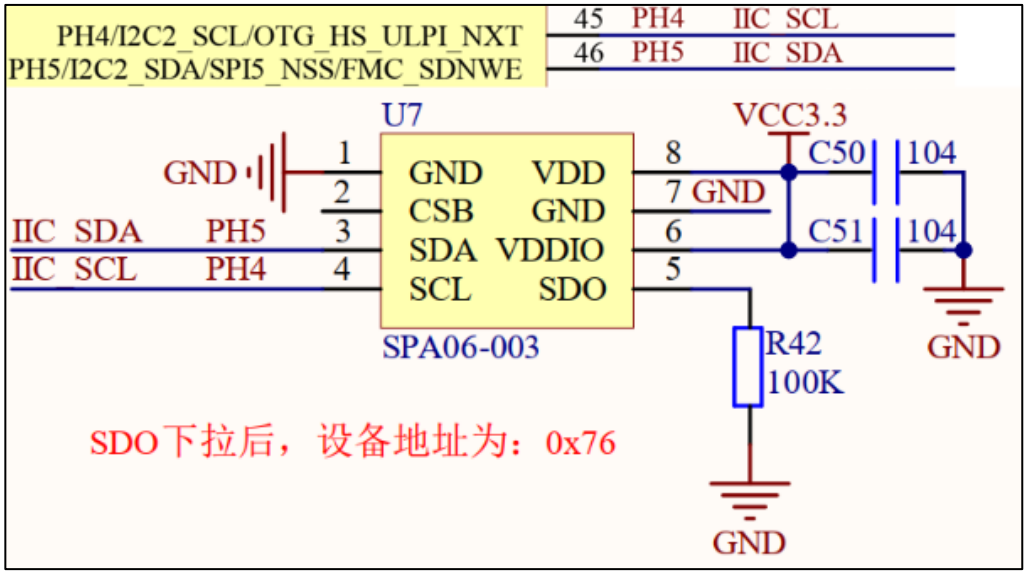


图 47.2.1 SPA06-003 连接原理

SPA06-003 的 SCL 和 SDA 分别连接在 STM32 的 PH4 和 PH5 上。本实验通过软件模拟 IIC 信号建立起与 SPA06-003 的通信，进行数据发送与接收，LCD 屏幕进行显示。

47.3 程序设计

47.3.1 程序流程图

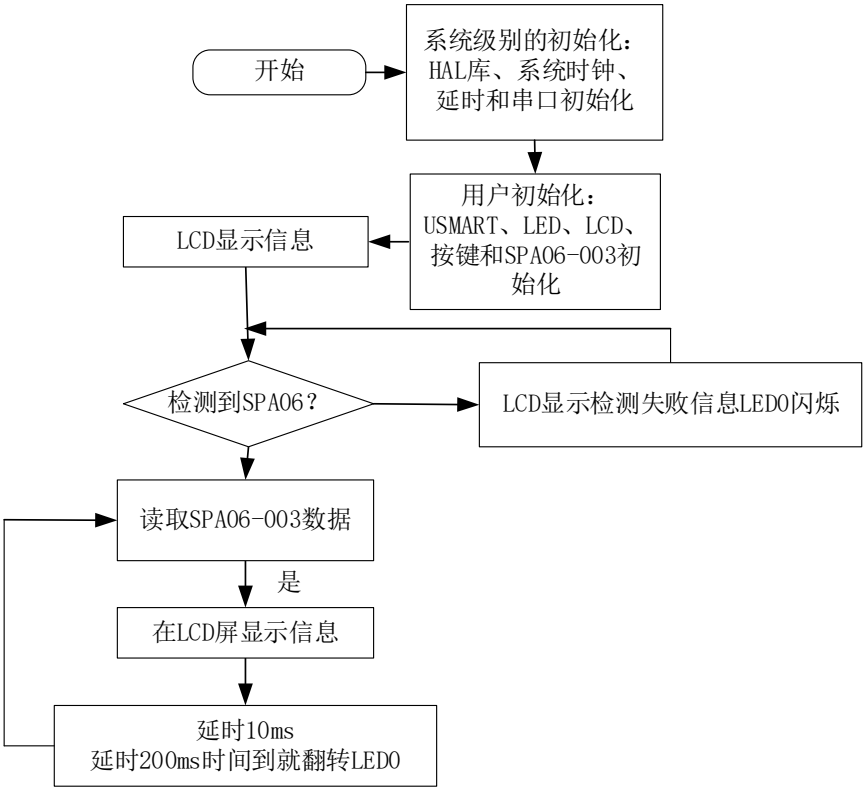


图 47.3.1.1 气压计实验程序流程图

47.3.2 程序解析

本实验中，我们通过 GPIO 来模拟 IIC，所以不需要在 Drivers/STM32F4xx_HAL_Driver 分组下添加 IIC 的 HAL 库文件支持。实验过程中，我们复制 IIC 实验工程，重命名为气压计实验，新增 SPA06.c 文件存放 SPA06-003 驱动。

1.SPA06-003 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。SPA06-003 驱动源码包括两个文件：SPA06.c 和 SPA06.h。

在 IIC 实验中已经对 IIC 协议中的需要用到的信号都用函数封装好了，那么现在就要定义符合 SPA06-003 时序的函数。为了使代码功能更加健全，所以在 SPA06-003.h 中宏定义了 SPA06-003 的相关寄存器，其定义如下：

```
/* 器件地址 */
/* SDO = 0 地址为：0X76, SDO = 1 地址为：0X77 */
#define SPA06_I2C_ADDR                  (0x76)

/* 寄存器地址 */
#define SPA06_DEFAULT_CHIP_ID           (0x11)

#define SPA06_PRESSURE_MSB_REG          (0x00)  /* Pressure MSB Register */
```

```
#define SPA06_PRESSURE_LSB_REG        (0x01) /* Pressure LSB Register */
#define SPA06_PRESSURE_XLSB_REG       (0x02) /* Pressure XLSB Register */
#define SPA06_TEMPERATURE_MSB_REG     (0x03) /* Temperature MSB Reg */
#define SPA06_TEMPERATURE_LSB_REG     (0x04) /* Temperature LSB Reg */
#define SPA06_TEMPERATURE_XLSB_REG    (0x05) /* Temperature XLSB Reg */
#define SPA06_PRESSURE_CFG_REG        (0x06) /* Pressure configuration Reg */
#define SPA06_TEMPERATURE_CFG_REG     (0x07) /* Temperature configuration Reg */
#define SPA06_MODE_CFG_REG            (0x08) /* Mode and Status Configuration */
#define SPA06_INT_FIFO_CFG_REG        (0x09) /* Interrupt and FIFO Configuration
#define SPA06_INT_STATUS_REG          (0x0A) /* Interrupt Status Reg */
#define SPA06_FIFO_STATUS_REG         (0x0B) /* FIFO Status Reg */
#define SPA06_RST_REG                 (0x0C) /* Softreset Register */
#define SPA06_CHIP_ID                 (0x0D) /* Chip ID Register */
#define SPA06_COEFFICIENT_CALIB_REG   (0x10) /* Coeffcient calibraion Register*/

#define SPA06_CALIB_COEFFICIENT_LENGTH  (18)
#define SPA06_DATA_FRAME_SIZE           (6)

#define SPA06_CONTINUOUS_MODE           (0x07)

#define TEMPERATURE_INTERNAL_SENSOR     (0)
#define TEMPERATURE_EXTERNAL_SENSOR     (1)

/* 测量次数 times / S */
#define SPA06_MWASURE_1                 (0x00)
#define SPA06_MWASURE_2                 (0x01)
#define SPA06_MWASURE_4                 (0x02)
#define SPA06_MWASURE_8                 (0x03)
#define SPA06_MWASURE_16                (0x04)
#define SPA06_MWASURE_32                (0x05)
#define SPA06_MWASURE_64                (0x06)
#define SPA06_MWASURE_128               (0x07)

/* 过采样率 */
#define SPA06_OVERSAMP_1                (0x00)
#define SPA06_OVERSAMP_2                (0x01)
#define SPA06_OVERSAMP_4                (0x02)
#define SPA06_OVERSAMP_8                (0x03)
#define SPA06_OVERSAMP_16               (0x04)
#define SPA06_OVERSAMP_32               (0x05)
#define SPA06_OVERSAMP_64               (0x06)
#define SPA06_OVERSAMP_128              (0x07)

/*****************************************************************************
************/
/* 传感器类型 */
typedef enum
{
    PRESURE_SENSOR,
    TEMPERATURE_SENSOR
}spa06Sensor_e;

/* 用于存储校准数据 */
typedef struct
{
    int16_t c0;
    int16_t c1;
    int32_t c00;
    int32_t c10;
    int16_t c01;
    int16_t c11;
    int16_t c20;
    int16_t c21;
    int16_t c30;
} spa06_calibcoeff_t;
```

```
/* 用于存储温度、气压值、海拔高度 */
typedef struct
{
    int32_t praw;    /* 气压原始值 */
    int32_t traw;    /* 温度原始值 */
    float   pcomp;   /* 计算后的气压值 hpa */
    float   tcomp;   /* 计算后的温度值 ℃ */
    float   asl;     /* 通过气压值转换后的海拔高度 m */
} spa06_result_t;
```

下面先看一下 SPA06-003_read_nbytes 函数，实现从 SPA06-003 芯片指定地址读取 N 字节数据，代码如下：

```
/**
 * @brief   从 SPA06 读取 N 字节数据
 * @param   reg:         寄存器地址
 * @param   date:        数据存储 buf
 * @param   len:         数据长度
 * @retval  读出结果
 * @retval  0, 操作成功
 *          其他, 操作失败
 */
int spa06_read_nbytes(uint8_t reg, uint8_t *date, uint8_t len)
{
    uint8_t i;

    iic_start();
    iic_send_byte((SPA06_I2C_ADDR << 1) | 0x00);
    if (iic_wait_ack() != 0)
    {
        iic_stop();
        return 1;
    }
    iic_send_byte(reg);
    if (iic_wait_ack() != 0)
    {
        iic_stop();
        return 1;
    }

    iic_start();
    iic_send_byte((SPA06_I2C_ADDR << 1) | 0x01);
    if (iic_wait_ack() != 0)
    {
        iic_stop();
        return 1;
    }
    for (i = 0; i < len; i++)
    {
        date[i] = iic_read_byte((i == (len - 1)) ? 0 : 1);
    }
    iic_stop();

    return 0;
}
```

该函数首先调用 iic_start 函数产生起始信号，然后调用 iic_send_byte 函数发送第 1 个字节数据设备写地址，SPA06_I2C_ADDR 是设备地址，左移 1 位（因为 I2C 地址是 7 位，左移后最低位是 R/W 位），然后或上 0x00 表示写操作，并等待从机应答，如果失败，停止并返回 1；收到应答信号后，继续发送第 2 个 1 字节数据寄存器地址 reg，然后等待从机应答，如果失败，停止并返回 1；然后发送第 3 个 1 字节数据设备读地址再或上 0x01 表示读操作，等待从机应答，如果失败，停止并返回 1；最后 SPA06 循环读取 len 个字节的数据，读取的字节存入 date[i]，读取完成后，发送停止条件，返回 0 表示成功。

```
/**
 * @brief   读取校准数据（用于计算气压值和温度值）
 * @param   无
 * @retval  无
 */
void spa06_get_calib_param(void)
{
    uint8_t buffer[SPA06_CALIB_COEFFICIENT_LENGTH] = {0};

    spa06_read_nbytes(SPA06_COEFFICIENT_CALIB_REG,
                      buffer,
                      SPA06_CALIB_COEFFICIENT_LENGTH);

    spa06calib.c0 = (int16_t)buffer[0]<<4 | buffer[1]>>4;
    spa06calib.c0 = (spa06calib.c0 & 0x0800) ?
                    (spa06calib.c0 | 0xF000) :
                    spa06calib.c0;

    spa06calib.c1 = (int16_t)(buffer[1] & 0x0F)<<8 | buffer[2];
    spa06calib.c1 = (spa06calib.c1 & 0x0800) ?
                    (spa06calib.c1 | 0xF000) :
                    spa06calib.c1;

    spa06calib.c00 = (int32_t)buffer[3]<<12 |
                     (int32_t)buffer[4]<<4 |
                     (int32_t)buffer[5]>>4;
    spa06calib.c00 = (spa06calib.c00 & 0x080000) ?
                     (spa06calib.c00 | 0xFFF00000) :
                     spa06calib.c00;

    spa06calib.c10 = (int32_t)(buffer[5] & 0x0F)<<16 |
                     (int32_t)buffer[6]<<8 |
                     (int32_t)buffer[7];
    spa06calib.c10 = (spa06calib.c10 & 0x080000) ?
                     (spa06calib.c10 | 0xFFF00000) :
                     spa06calib.c10;

    spa06calib.c01 = (int16_t)buffer[8]<<8 | buffer[9];
    spa06calib.c11 = (int16_t)buffer[10]<<8 | buffer[11];
    spa06calib.c20 = (int16_t)buffer[12]<<8 | buffer[13];
    spa06calib.c21 = (int16_t)buffer[14]<<8 | buffer[15];
    spa06calib.c30 = (int16_t)buffer[16]<<8 | buffer[17];
}

/**
 * @brief   设置对应传感器的测量速率以及过采样率
 * @param   sensor:          传感器类型
 * @param   measureRate:     测量速率
 * @param   oversamplRate:   过采样率
 * @retval  无
 */
void spa06_rateset(spa06Sensor_e sensor, uint8_t measureRate, uint8_t
oversamplRate)
{
    uint8_t reg;

    if (sensor == PRESURE_SENSOR)           /* 压力传感器 */
    {
        kp = scale_factor[oversamplRate];
        spa06_write_byte(SPA06_PRESSURE_CFG_REG,
                         measureRate<<4 | oversamplRate);

        if (oversamplRate > SPA06_OVERSAMP_8)
        {
            spa06_read_nbytes(SPA06_INT_FIFO_CFG_REG, &reg, 1);
```

```
            spa06_write_byte(SPA06_INT_FIFO_CFG_REG, reg | 0x04);
        }
    }
    else if (sensor == TEMPERATURE_SENSOR)  /* 温度传感器 */
    {
        kt = scale_factor[oversamplRate];
        spa06_write_byte(SPA06_TEMPERATURE_CFG_REG,
                         measureRate<<4 | oversamplRate | 0x80);
        if (oversamplRate > SPA06_OVERSAMP_8)
        {
            spa06_read_nbytes(SPA06_INT_FIFO_CFG_REG, &reg, 1);
            spa06_write_byte(SPA06_INT_FIFO_CFG_REG, reg | 0x08);
        }
    }
}

/**
 * @brief   获取温度值（℃）
 * @param   temperature:    原始的温度数据
 * @retval  计算后 最终的补偿温度值
 */
float spa06_get_temperature(int32_t temperature)
{
    float fTCompensate;
    float fTsc;

    fTsc = temperature / (float)kt;
    fTCompensate =  spa06calib.c0 * 0.5 + spa06calib.c1 * fTsc;
    return fTCompensate;
}

/**
 * @brief   获取气压值（Pa）
 * @param   pressure:       原始压力数据
 * @param   temperature:    原始的温度数据
 * @retval  计算后 最终的补偿气压值
 */
float spa06_get_pressure(int32_t pressure, int32_t temperature)
{
    float fTsc, fPsc;
    float qua2, qua3;
    float fPCompensate;

    fTsc = temperature / (float)kt;
    fPsc = pressure / (float)kp;
    qua2 = spa06calib.c10 + fPsc * (spa06calib.c20 + fPsc* spa06calib.c30);
    qua3 = fTsc * fPsc * (spa06calib.c11 + fPsc * spa06calib.c21);
    /* qua3 = 0.9f *fTsc * fPsc * (spa06Calib.c11 + fPsc * spa06Calib.c21); */

    fPCompensate = spa06calib.c00 + fPsc * qua2 + fTsc * spa06calib.c01 + qua3;
    return fPCompensate;
}

/**
 * @brief   以 m 为单位,将压力转换为海平面以上高度(ASL)
 * @param   pressure:   经过计算后的压力数据,注意气压值的单位需为 hpa
 * @retval  海拔高度（m）
 */
float spa06_pressure_to_asl(float pressure)
{
    if(pressure)
    {
        return 44330.f * (powf((1015.7f / pressure), 0.190295f) - 1.0f);
    }
    else
```

```
    {
        return 0;
    }
}

/**
 * @brief    获取 spa06 数据
 * @param    spa06_result_t *p_res: 该结构体用于存放获取到的温度、压力、海拔高度数据
 * @retval   无
 */
void spa06_get_data(spa06_result_t *p_res)
{
    uint8_t data[SPA06_DATA_FRAME_SIZE];

    /* 读取寄存器中的气压值和温度值 */
    spa06_read_nbytes(SPA06_PRESSURE_MSB_REG, data, SPA06_DATA_FRAME_SIZE);
    /* 得到气压数据 */
    p_res->praw = (int32_t)data[0] << 16 |
                  (int32_t)data[1] << 8 |
                  (int32_t)data[2];
    p_res->praw = (p_res->praw & 0x800000) ?
                  (0xFF000000 | p_res->praw) :
                  p_res->praw;
    /* 得到温度数据 */
    p_res->traw = (int32_t)data[3] << 16 |
                  (int32_t)data[4] << 8 |
                  (int32_t)data[5];
    p_res->traw = (p_res->traw & 0x800000) ? (0xFF000000 | p_res->traw) :
p_res->traw;

    /* 获取计算后的补偿温度值和气压值 */
    p_res->tcomp = spa06_get_temperature(p_res->traw); /* 单位℃ */
    /* 单位 hPa */
    p_res->pcomp = spa06_get_pressure(p_res->praw, p_res->traw) / 100;
    p_res->asl = spa06_pressure_to_asl(p_res->pcomp);  /* 转换成海拔高度 */
}
```

整个代码模块完成了 SPA06 传感器的校准参数读取、配置、数据读取和补偿计算，以及海拔高度的转换。

2. main.c 代码

我们读取到的 z，y，z 轴数据是原始数据，此时的数据还不能直接用作方位角的计算。最后，我们打开 main.c：

```
/**
 * @brief       显示气压值
 * @param       x, y : 坐标
 * @param       angle: 气压值
 * @retval      无
 */
void user_show_pressure(uint16_t x, uint16_t y, float press)
{
    char buf[20];

    sprintf(buf,"Pressure:%.2fhPa", press);          /* 格式化输出 */
    lcd_fill(x, y, x + 160, y + 16, WHITE);
    lcd_show_string(x, y, 160, 16, 16, buf, BLUE);  /* 显示字符串 */
}

/**
 * @brief       显示温度
 * @param       x, y : 坐标
 * @param       temp : 温度
```

```
 * @retval      无
 */
void user_show_temprate(uint16_t x, uint16_t y, float temp)
{
    char buf[20];

    sprintf(buf,"Temp:%.2fC", temp);                /* 格式化输出 */
    lcd_fill(x, y, x + 160, y + 16, WHITE);
    lcd_show_string(x, y, 160, 16, 16, buf, BLUE);  /* 显示字符串 */
}

/**
 * @brief       显示海拔高度
 * @param       x, y : 坐标
 * @param       alt : 海拔高度
 * @retval      无
 */
void user_show_altitude(uint16_t x, uint16_t y, float alt)
{
    char buf[20];

    sprintf(buf,"Altitude:%.2fm", alt);             /* 格式化输出 */
    lcd_fill(x, y, x + 160, y + 16, WHITE);
    lcd_show_string(x, y, 160, 16, 16, buf, BLUE);  /* 显示字符串 */
}

int main(void)
{
    uint8_t t = 0;

    spa06_result_t spa06_result;

    HAL_Init();                                     /* 初始化 HAL 库 */
    sys_stm32_clock_init(360, 25, 2, 8);            /* 设置时钟,180Mhz */
    delay_init(180);                                /* 延时初始化 */
    usart_init(115200);                             /* 初始化 USART */
    usmart_dev.init(90);                            /* 初始化 USMART */
    led_init();                                     /* 初始化 LED */
    key_init();                                     /* 初始化按键 */
    sdram_init();                                   /* 初始化 SDRAM */
    lcd_init();                                     /* 初始化 LCD */

    while (spa06_init())                            /* 初始化 SPA06 */
    {
        lcd_show_string(30, 110, 200, 16, 16, "SPA06 Error", RED);
        delay_ms(200);
        lcd_fill(30, 110, 239, 130 + 16, WHITE);
        delay_ms(200);
    }

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "SPA06 TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    while (1)
    {
        delay_ms(10);
        t++;

        if (t == 20)    /* 0.2 秒左右更新一次数值 */
        {
            spa06_get_data(&spa06_result);          /* 执行一次约 150ms */
            user_show_pressure(30, 110, spa06_result.pcomp);/* 显示气压值 */
```

```
            user_show_temprate(30, 130, spa06_result.tcomp);/* 显示温度 */
            user_show_altitude(30, 150, spa06_result.asl);  /* 显示海拔高度 */

            t = 0;
            LED0_TOGGLE();                                  /* LED0 闪烁 */
        }
    }
}
```

main 函数，则比较简单，初始化相关外设，在循环中不断获取气压计计数据，并在 LCD 上显示数据。

47.4 下载验证

在代码编译成功后，我们通过下载代码到开发板上，可以看到 LED0 不停闪烁，提示程序已经在运行了。LCD 显示气压计的气压值，温度值，以及海拔高度，如图 47.4.1 所示：

图 47.4.1 程序运行效果图

第四十八章 QMI8658A 六轴传感器实验

本章，我们将介绍一款高性价比六轴（三轴角速度（陀螺仪）+三轴加速度）传感器：QMI8658A，该传感器可广泛用于四轴、平衡车和空中鼠标等设计，性价比极高，具有非常广泛的应用范围。正点原子阿波罗 H743 开发板自带了 QMI8658A 传感器。本章我们将使用 STM32H743IIT6 来驱动 QMI8658A，读取其原始数据，并利用 MPL 库实现姿态解算，结合匿名四轴上位机软件和 LCD 显示，教大家如何使用这款功能强大的六轴传感器。

本章分为如下几个小节：

48.1 QMI8658A 介绍
48.2 硬件设计
48.3 软件设计
48.4 下载验证

48.1 QMI8658A 介绍

48.1.1 QMI8658A 简介

QMI8658A 是一款针对大众市场，高性价比的六轴传感器，具有：功耗低、噪声低、封装小的特点。

QMI8658A 内部集成有 3 轴陀螺仪和 3 轴加速度计；可以通过 I3C、I2C 和 3 线或 4 线 SPI 的接口和单片机进行数据交互，传输速率可达 400kHZ/s。陀螺仪的角速度测量范围最高达±2048°/s，具有良好的动态响应特性。加速度计的测量范围最大为±16g（g 为重力加速度），静态测量精度高。广泛应用于智能手机、游戏控制器、遥控器和汽车安全系统等场景。

QMI8658A 的特点包括：

- 陀螺仪 13 mdps/Hz 的低噪声，低延迟，宽带宽
- 150μg/Hz 的加速度计噪声
- 自带一个温度数字传感器
- 可编程数字滤波器
- 集成计步器，轻拍，任意运动，移动，显著运动检测
- 自带 1536 字节 FIFO 可用于缓冲传感器数据，降低系统功耗
- 超小封装尺寸：2.5×3.0×0.86mm(LGA)

QMI8658A 传感器的检测轴如图 48.1.1.1 所示：

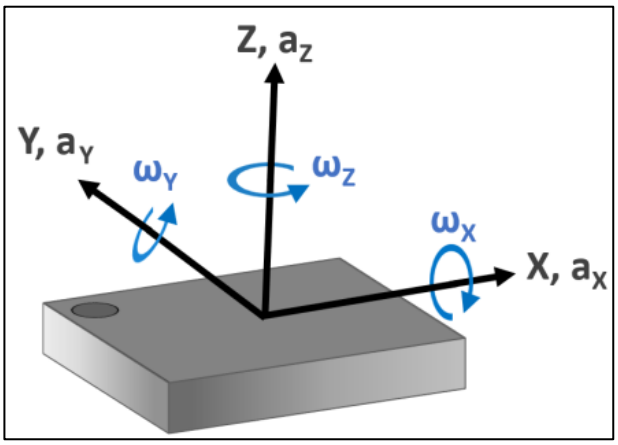


图 48.1.1.1QMI8658A 检测轴及其方向

QMI8658A 的内部框图如图 48.1.1.2 所示：

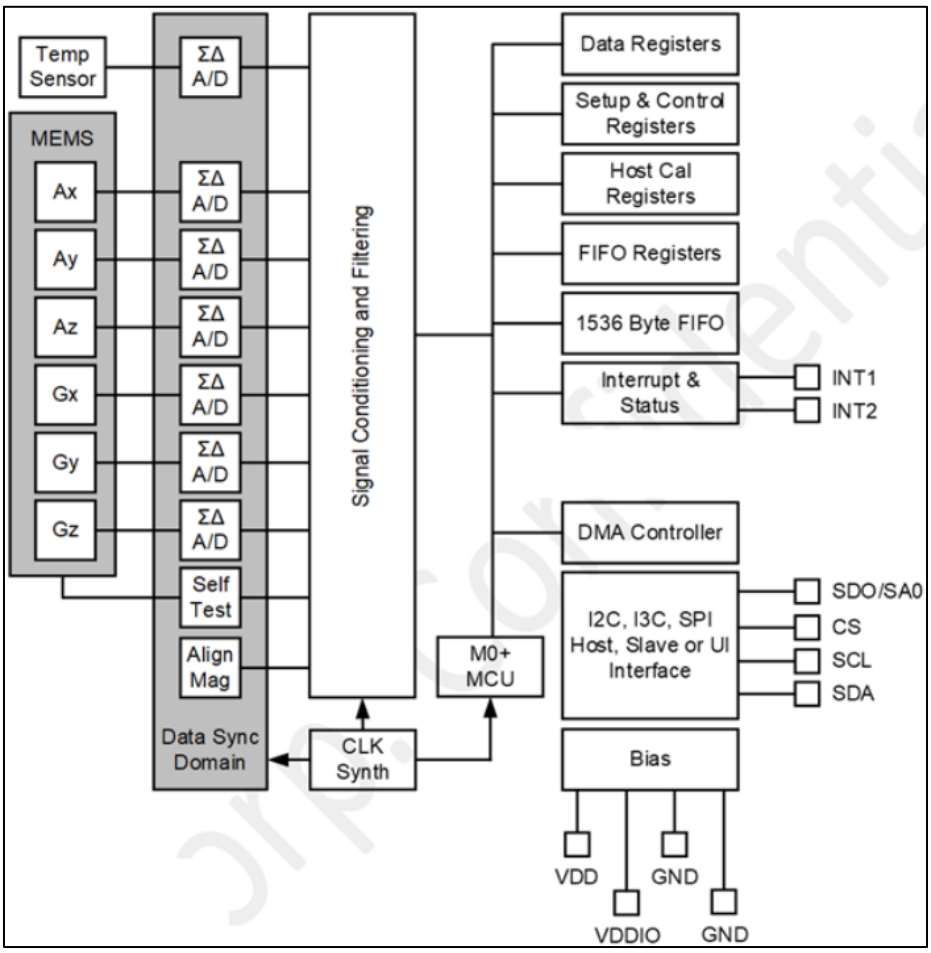


图 48.1.1.2 QMI8658A 框图

其中，SCL 和 SDA 可以连接 MCU 的 I2C、I3C 和 SPI 接口，MCU 通过这些接口来控制 QMI8658A，我们例程中用的就是 IIC 接口。另外，SA0 是从 IIC 接口（接 MCU）的地址控制引脚，该引脚控制 IIC 地址的最低位。如果接 GND，则 QMI8658A 的 IIC 地址是：0X6B，如果接 VDD，则是 0X6A，注意：这里的地址是不包含数据传输的最低位的（最低位用来表示读写）!!

在 STM32 开发板上，SA0 是接 VCC 的，所以 QMI8658A 的 IIC 地址是 0X6A（不含最低位），IIC 通信的时序我们在之前已经介绍过（第三十七章，IIC 实验），这里就不再细说了。

48.1.2 QMI8658A 寄存器

在这里简单介绍一下本实验用到的 QMI8658A 比较重要的寄存器。

- **陀螺仪配置寄存器（Gyroscope Settings）**

该寄存器地址为：0X04，各位描述如表 48.1.2.1 所示：

寄存器（HEX）	BIT7	BIT6	BIT5	BIT4	BIT3	BIT2	BIT1	BIT0
04	gST	gFS[2:0]			gODR[3:0]			

表 48.1.2.1 陀螺仪配置寄存器

该寄存器我们关心 gFS[2:0]和 gODR[3:0]这些位，gFS[2:0]用于设置陀螺仪的满量程范围：000，±16dps；001，±32dps；010，±64dps；011，±128dps；100，±256dps；101，±512dps；110，±1024dps；111，±2048dps；我们一般设置为 011，即±128dps，因为陀螺仪的 ADC 为 16 位分辨率，所以得到灵敏度为：65536/256=256LSB(dps)。

gODR[3:0]位用于设置陀螺仪输出数据速率，我们例程中设置为 0100，即约 500Hz 的速率。

- **加速度传感器配置寄存器（Accelerometer Settings）**

该寄存器地址为：0X03，各位描述如下表所示：

寄存器（HEX）	BIT7	BIT6	BIT5	BIT4	BIT3	BIT2	BIT1	BIT0
03	aST	aFS[2:0]			aODR[3:0]			

表 48.1.2.2 加速度传感器配置寄存器

该寄存器我们关心 aFS[2:0]和 aODR[3:0]这些位，aFS[2:0]用于设置加速度计的满量程范围：000，±2g；001，±4g；010，±8g；011，±16g；我们一般设置为 011，即±16g，因为加速度传感器的 ADC 也是 16 位分辨率，所以得到灵敏度为：65536/32=2048LSB/g。

aODR[3:0]位用于设置加速度计输出数据速率，我们例程中设置为 0100，即约 500Hz 的速率。

- **传感器数据处理寄存器（Sensor Data Processing Settings）**

该寄存器的地址为：0X06，各位描述如表 48.1.2.3 所示：

寄存器（HEX）	BIT7	BIT6	BIT5	BIT4	BIT3	BIT2	BIT1	BIT0
06	Reserved	gLPF_MODE[1:0]		gLPF_EN	Reserved	aLPF_MODE[1:0]		aLPF_EN

表 48.1.2.3 传感器数据处理寄存器

这里，我们主要关心数字低通滤波器（LPF_MODE）的设置位，即：gLPF_MODE[1:0]和 aLPF_MODE[1:0]，陀螺仪和加速度传感器分别根据这两个位的配置进行过滤。首先看陀螺仪 gLPF_MODE[1:0]不同配置对应的过滤情况如表 48.1.2.4 所示：

gLPF_MODE[1:0]	噪声 BW（Hz）
00	ODR 的 2.66%
01	ODR 的 3.63%
10	ODR 的 5.39%
11	ODR 的 13.37%

表 48.1.2.4 gLPF_MODE 配置表

再看加速度 aLPF_MODE[1:0]对应的过滤情况如表 48.1.2.5 所示：

aLPF_MODE[1:0]	噪声 BW（Hz）
00	ODR 的 2.66%
01	ODR 的 3.63%
10	ODR 的 5.39%
11	ODR 的 13.37%

表 48.1.2.5 aLPF_MODE 配置表

在例程中，我们将加速度传感器和陀螺仪的低通过滤器模式选择位：gLPF_MODE[1:0]和 aLPF_MODE[1:0]都设置为 11，即 ODR 的 13.37%。

- **温度传感器数据输出寄存器（Temp Sensor Output）**

该寄存器地址为：0X33-0X34，各位描述如表 48.1.2.6 所示：

寄存器（HEX）	BIT7	BIT6	BIT5	BIT4	BIT3	BIT2	BIT1	BIT0
33	TEMP_H							

表 48.1.2.6 温度传感器数据输出寄存器

这个寄存器可以读取温度传感器的值，可以通过读取 0X33（高 8 位）和 0X34（低 8 位）寄存器得到，温度换算公式为：

T= TEMP_H + (TEMP_L/256)

其中，T 为计算得到的温度值，单位为℃，TEMP_H(L)为从 0X33 和 0X34 读到的温度传感器值。

接下来，加速度传感器数据输出寄存器，也有 6 个，地址为：0X35~0X3A，通过读取这 6 个寄存器，就可以读到加速度传感器 x/y/z 轴的值，比如读 x 轴的数据，可以通过读取 0X35（高 8 位）和 0X36（低 8 位）寄存器得到，其他轴以此类推。

同样，我们看看陀螺仪数据输出寄存器，总共由 6 个寄存器组成，地址为：0X3B~0X40，通过读取 0X3B（高 8 位）和 0X3C（低 8 位）寄存器得到，其他轴以此类推。

关于 QMI8658A 的基础介绍，我们就介绍到这。QMI8658A 的详细资料和相关寄存器介绍，请参考光盘：7，硬件资料→2，芯片资料→ QMI8658A.pdf 这个文档，供大家参考学习。

48.2 硬件设计

48.2.1 例程功能

本实验采用 STM32H743 的 2 个普通 IO 连接 QMI8658A（IIC），本章实验功能简介：程序先初始化 QMI8658A，然后利用 imu 姿态解算的计算，最后，在死循环里面不停读取：温度传感器、加速度传感器、陀螺仪、姿态解算后的欧拉角等数据，通过串口上报给上位机（温度不上报），利用上位机软件（ANO_TC 匿名科创地面站 v4.exe），可以实时显示 QMI8658A 的传感器状态曲线，并显示 3D 姿态，可以通过 KEY0 按键开启/关闭数据上传功能。同时，在 LCD 模块上面显示温度和欧拉角等信息。LED 来指示程序正在运行。

48.2.2 硬件资源

1）LED 灯
LED0 – PB1
2）独立按键
KEY0 – PH3
KEY_UP –PH2
3）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
5）气压计 QMI8658A

48.2.3 原理图

QMI8658A 原理图，如下图所示。

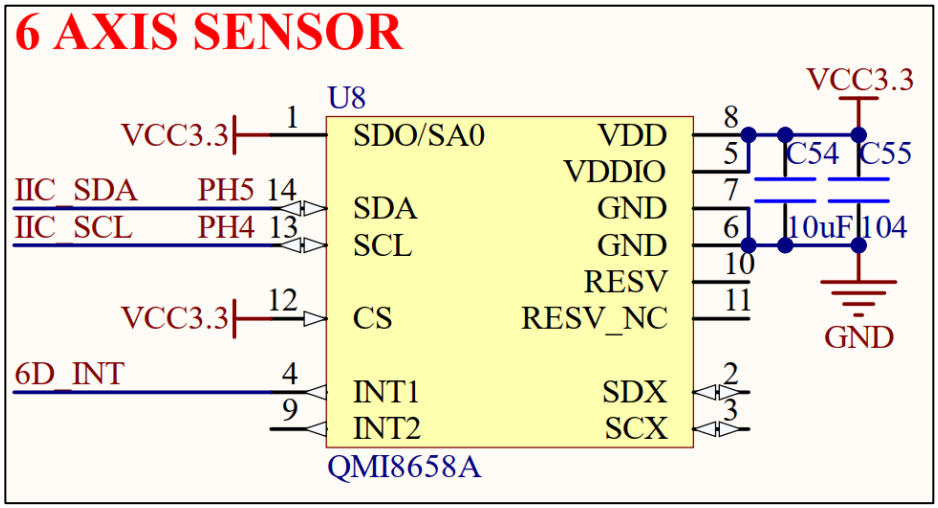


图 48.2.3.1 QMI8658A 原理图

这里说明一下，QMI8658A 的 QMA_INT 脚是连接在 PCF8574T 器件的 P5 脚上，如果大家要使用 QMI8658A 的中断输出功能，必须先初始化 PCF8574T 器件并配置 P5 为输入功能，监

测 PCF8574T 中断引脚是否有中断产生。若发现有中断产生，则判断是否是 P5 导致的，从而检测到 QMI8658A 的中断。在本章中，并没有用到 QMI8658A 中断功能，所以没有对 PCF8574T 器件的 P5 做设置。

48.3 程序设计

48.3.1 程序流程图

程序流程图能帮助我们更好的理解一个工程的功能和实现的过程，对学习和设计工程有很好的主导作用。下面看看本实验的程序流程图：

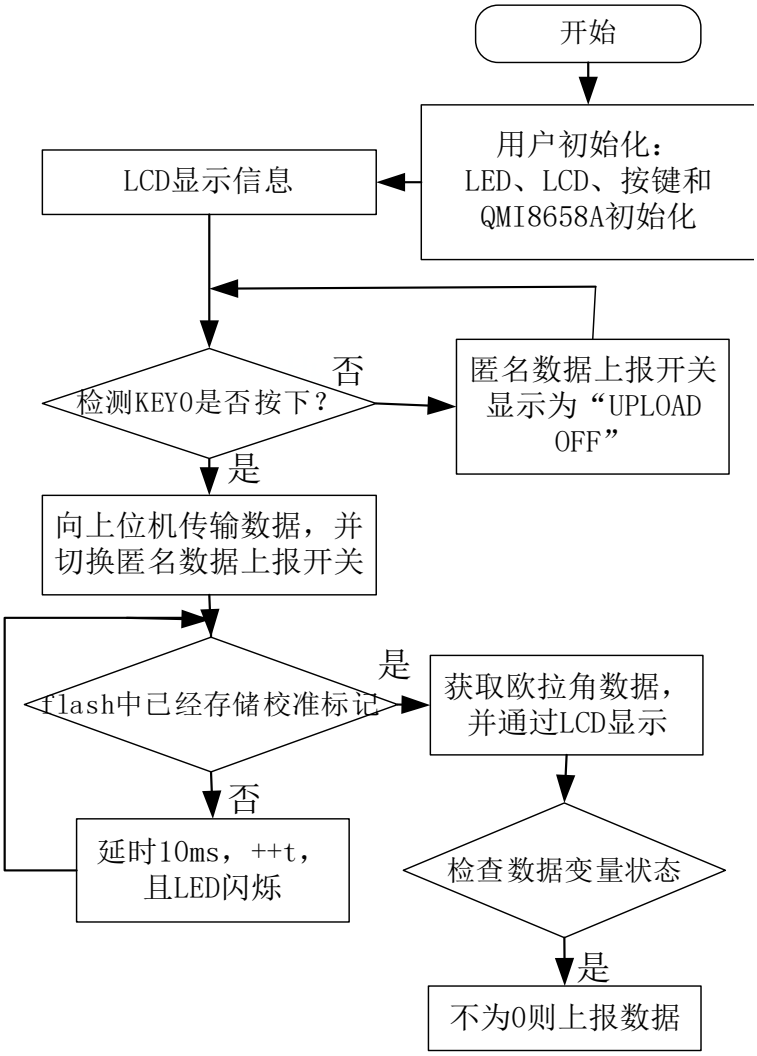


图 48.3.1.1 QMI8658A 实验程序流程图

1. QMI8658A 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。QMI8658A 驱动源码包括两个文件：qmi8658.c 和 qmi8658.h。

qmi8658.h 的文件是一些函数的定义，我们直接看 qmi8658.c 文件。

下面来看一下 qmi8658_read_nbytes 函数，实现 QMI8658A 芯片指定地址读取 N 字节数据，代码如下：

```
/**
 * @brief   从 QMI8568 读取 N 字节数据
 * @param   reg:        寄存器地址
 * @param   date:       数据存储 buf
 * @param   len:        数据长度
 * @retval  读出结果
 * @retval  0, 操作成功
 *          其他, 操作失败
 */
int qmi8568_read_nbytes(uint8_t reg, uint8_t *date, uint8_t len)
{
    uint8_t i;

    iic_start();
    iic_send_byte((QMI8658_ADDR << 1) | 0x00);
    if (iic_wait_ack() != 0)
    {
        iic_stop();
        return 1;
    }
    iic_send_byte(reg);
    if (iic_wait_ack() != 0)
    {
        iic_stop();
        return 1;
    }

    iic_start();
    iic_send_byte((QMI8658_ADDR << 1) | 0x01);
    if (iic_wait_ack() != 0)
    {
        iic_stop();
        return 1;
    }
    for (i = 0; i < len; i++)
    {
        date[i] = iic_read_byte((i == (len - 1)) ? 0 : 1);
    }
    iic_stop();

    return 0;
}
```

该函数首先调用 iic_start 函数产生起始信号，然后调用 iic_send_byte 函数发送第 1 个字节数据设备写地址，等待 QMI8658A 设备返回应答信号；收到应答信号后，继续发送第 2 个 1 字节数据内存地址 reg；等待接收应答后，发送第 3 个 1 字节数据设备读地址；最后 QMI8658A 最先收到的是状态字节（一个字节），随后是读取的数据（数据都是两个字节），返回应答信号，主机调用 iic_stop 函数产生停止信号终止数据传输，等待 QMI8658A 读取完毕。

接下来看一下 qmi8658_write_nbytes 函数，实现从 QMI8658A 芯片指定地址写入数据，其定义如下：

```
/**
 * @brief   QMI8568 写入 N 字节数据
 * @param   reg:        寄存器地址
 * @param   data:       写入数据
 * @param   len:        数据长度
 * @retval  写入结果
 * @arg     0:          成功
 * @arg     1:          失败
 */
uint8_t qmi8658_write_nbytes(uint8_t reg, uint8_t* data, uint8_t len)
{
    uint8_t i;
```

```
    iic_start();
    iic_send_byte((QMI8658_ADDR << 1) | 0x00);
    if (iic_wait_ack() != 0)
    {
        iic_stop();
        return 1;
    }
    iic_send_byte(reg);
    if (iic_wait_ack() != 0)
    {
        iic_stop();
        return 1;
    }
    for (i = 0; i < len; i++)
    {
        iic_send_byte(data[i]);
        if (iic_wait_ack() != 0)
        {
            iic_stop();
            return 1;
        }
    }
    iic_stop();

    return 0;
}
```

该函数首先调用 iic_start 函数产生起始信号，然后调用 iic_send_byte 函数发送第 1 个字节数据设备写地址，等待 QMI8658A 设备返回应答信号；收到应答信号后，继续发送第 2 个 1 字节写入数据内存地址 reg；等待接收应答后，发送第 3 个 1 字节数据设备写地址；QMI8658A 设备写完数据，返回应答信号，主机调用 iic_stop 函数产生停止信号终止数据传输，等待 QMI8658A 写入完毕。

最后，我们介绍一下读取数据函数，代码如下：

```
/**
 * @brief   获取传感器温度
 * @param   无
 * @retval  温度值,单位为℃
 */
float qmi8658_get_temperature(void)
{
    short temp = 0;
    float temp_f = 0;
    uint8_t buf[2];

    qmi8568_read_nbytes(Register_Tempearture_L, buf, 2);     /* 读取温度数据 */
    temp = ((short)buf[1] << 8) | buf[0];
    temp_f = (float)temp / 256.0f;
    return temp_f;
}



/**
 * @brief   判断数据更新后，在读取补偿后 QMI8658 陀螺仪和加速度的数据(推荐使用)
 * @param   acc  : 加速度计 X,Y,Z 缓存区;
 * @param   gyro : 陀螺仪 X,Y,Z 缓存区;
 * @retval  无
 */
void qmi8658_read_xyz(float *acc, float *gyro)
{
    unsigned char status = 0;
    unsigned char data_ready = 0;
    int retry = 0;
```

```
    while (retry++ < 3)
    {
#if defined(QMI8658_SYNC_SAMPLE_MODE)
        qmi8568_read_nbytes(Register_StatusInt, &status, 1);

        if (status & 0x01)
        {
            delay_us(12);       /* delay 12us <=500Hz */
        }
        if ((status & 0x01) || (status & 0x03))
        {
            data_ready = 1;
            break;
        }
#else
        /* 检查加速度计和陀螺仪数据是否可用 */
        qmi8568_read_nbytes(Register_Status0, &status, 1);
        if (status & 0x03)
        {
            data_ready = 1;
            break;
        }
#endif
    }
    if (data_ready)
    {
        qmi8658_read_sensor_data(acc, gyro);

        g_imu.imu[0] = acc[0];
        g_imu.imu[1] = acc[1];
        g_imu.imu[2] = acc[2];
        g_imu.imu[3] = gyro[0];
        g_imu.imu[4] = gyro[1];
        g_imu.imu[5] = gyro[2];
    }
    else
    {
        acc[0] = g_imu.imu[0];
        acc[1] = g_imu.imu[1];
        acc[2] = g_imu.imu[2];
        gyro[0] = g_imu.imu[3];
        gyro[1] = g_imu.imu[4];
        gyro[2] = g_imu.imu[5];
//      printf("数据还未准备好\r\n");     /* 调试使用 */
    }
}
```

qmi8658_get_temperature 函数：读取温度数据。

qmi8658_read_sensor_data 函数：读取补偿后加速度计和陀螺仪的 x，y，z 轴原始数据。

下面看一下 QMI8658A 的初始化函数，其定义如下：

```
/**
 * @brief   初始化 QMI8658
 * @param   无
 * @retval  初始化结果
 * @arg     0: 成功
 * @arg     1: 失败
 */
uint8_t qmi8658_init(void)
{
    iic_init();

    qmi8658_reset();                                    /* 复位传感器 */

    if(qmi8658_check_whoami())                          /* 检查设备 ID 是否正确 */
    {
```

```
        return 1;
    }
    if(qmi8658_calibration())
    {
        return 1;
    }
    printf("校准成功！！\r\n");

    qmi8658_write_one_byte(Register_Ctrl1, 0x60);    /* I2C 驱动 */
    qmi8658_write_one_byte(Register_Ctrl7, 0x00);    /* 关闭陀螺仪、加速度计 */

    qmi8658_config_reg(0);
    /* 配置陀螺仪和加速度计的量程和数据输出速率等参数 */
    qmi8658_enablesensors(g_imu.cfg.ensensors);      /* 使能陀螺仪、加速度计 */

    return 0;
}
```

在初始化函数中，我们主要对该模块的加速度、陀螺仪，还有温度传感器的使能和设置。

2. main.c 代码

```
/**
 * @brief    通过串口 1 上报结算后的姿态数据给电脑
 * @param    roll  : 横滚角.单位 0.1 度。 -9000 -> 9000 对应 -90.00  ->  90.00 度
 * @param    pitch : 俯仰角.单位 0.1 度。-18000 -> 18000 对应 -180.00 -> 180.00 度
 * @param    yaw   : 航向角.单位为 0.1 度 -18000 -> 18000  对应 -180.00 -> 180.00 度
 * @param    prs   : 气压计高度,单位:cm
 * @param    fly_mode : 飞行模式
 * @param    armed    : 锁定状态
 * @retval   无
 */
void usart1_report_imu(short roll, short pitch, short yaw, int prs, uint8_t
fly_mode, uint8_t armed)
{
    uint8_t tbuf[12];

    tbuf[0] = (roll >> 8) & 0XFF;
    tbuf[1] = roll & 0XFF;
    tbuf[2] = (pitch >> 8) & 0XFF;
    tbuf[3] = pitch & 0XFF;
    tbuf[4] = (yaw >> 8) & 0XFF;
    tbuf[5] = yaw & 0XFF;
    tbuf[6] = (prs >> 24) & 0XFF;
    tbuf[7] = (prs >> 16) & 0XFF;
    tbuf[8] = (prs >> 8) & 0XFF;
    tbuf[9] = prs & 0XFF;
    tbuf[10] = fly_mode;
    tbuf[11] = armed;
    usart1_niming_report(0X01, tbuf, 12);    /* 状态帧,0X01 */
}

int main(void)
{
    uint8_t t = 0;
    uint8_t key;
    uint8_t report = 0;
    float temperature;
    float gyro[3];
    float accel[3];
    float euler_angle[3] = {0, 0, 0};

    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(360, 25, 2, 8);/* 配置时钟，600MHz */
```

```
    delay_init(180);                        /* 初始化延时 */
    usart_init(500000);                     /* 初始化串口 */
    led_init();                             /* 初始化 LED */
    key_init();                             /* 初始化按键 */
    hyperram_init();                        /* 初始化 HyperRAM */
    lcd_init();                             /* 初始化 LCD */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "QMI8658A TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:UPLOAD ON/OFF", RED);

    while (qmi8658_init() != 0)             /* 初始化 QMI8658A */
    {
        lcd_show_string(30, 130, 200, 16, 16, "QMI8658A Error!", RED);
        delay_ms(500);
        lcd_show_string(30, 130, 200, 16, 16, "Please Check!  ", RED);
        delay_ms(500);
        LED0_TOGGLE();
    }

    lcd_show_string(30, 130, 200, 16, 16, "QMI8658A Ready!", RED);
    lcd_show_string(30, 150, 200, 16, 16, "UPLOAD OFF", BLUE);
    lcd_show_string(30, 170, 200, 16, 16, "Temp :    . C", BLUE);
    lcd_show_string(30, 190, 200, 16, 16, "Pitch:    . C", BLUE);
    lcd_show_string(30, 210, 200, 16, 16, "Roll :    . C", BLUE);
    lcd_show_string(30, 230, 200, 16, 16, "Yaw  :    . C", BLUE);

    while (1)
    {
        qmi8658_read_xyz(accel, gyro);
        key = key_scan(0);
        if (key == KEY0_PRES)
        {
            /* 切换匿名数据上报开关 */
            report = !report;
            if (report == 0)
            {
                lcd_show_string(30, 150, 200, 16, 16, "UPLOAD OFF", BLUE);
            }
            else
            {
                lcd_show_string(30, 150, 200, 16, 16, "UPLOAD ON ", BLUE);
            }
        }

        /* 获取并显示温度 */
        show_data(30, 170, qmi8658_get_temperature());

        /* 获取并显示欧拉角 */
        if (g_imu_init)
        {
            imu_get_eulerian_angles(accel, gyro, euler_angle, IMU_DELTA_T);
            show_data(30, 190, euler_angle[0]);
            show_data(30, 210, euler_angle[1]);
            show_data(30, 230, euler_angle[2]);

            if (report != 0)
            {
                /* 上报匿名状态帧 */
                qmi8658_send_data(accel[0], accel[1], accel[2], gyro[0],
                gyro[1], gyro[2]);  /* 发送加速度+陀螺仪原始数据 */
                usart1_report_imu((int)(euler_angle[1] * 100),
                (int)(euler_angle[0] * 100), (int)(euler_angle[2] * 100),
```

```
                    0, 0, 0);           /* Pitch 和 Roll 角位置调换 */
            }
        }
        if (++t == 100)
        {
            t = 0;
            LED0_TOGGLE();
        }

        delay_ms(10);
    }
}
```

此部分代码除了 main 函数，还有几个函数，用于上报数据给上位机软件，可以通过上位机软件显示六轴传感器波形，以及 3D 姿态显示，有助于更好的调试 QMI8658A。上位机软件使用 ANO_TC 匿名科创地面站 v4.exe，该软件在：开发板光盘→6，软件资料→软件→5，其他软件→匿名地面站文件夹里可以找到，该软件的使用方法，见该文件夹下的：飞控通信协议 v1.3-0720.pdf，这里我们不做介绍。其中，usart1_niming_report 函数用于将数据打包、计算校验和，然后上报给匿名地面站上位机软件。qmi8658_send_data 函数用于上报加速度和陀螺仪的原始数据，可用于波形显示传感器数据，通过传感器帧（0X02）发送。而 uasrt1_report_imu 函数，则用于发送匿名状态帧，可以实时 3D 显示 QMI8658A 的姿态，传感器数据等，通过状态帧（0X01）发送。

下面我们来看 main 函数，利用上位机软件（ANO_TC 匿名科创地面站 v4.exe），可以实时显示 QMI8658A 的传感器状态曲线，并显示 3D 姿态，可以通过 KEY0 按键开启/关闭数据上传功能。同时，在 LCD 模块上面显示温度和欧拉角等信息。

48.4 下载验证

将程序下载到开发板后，可以看到 LCD 显示的内容如图 48.4.1 所示：

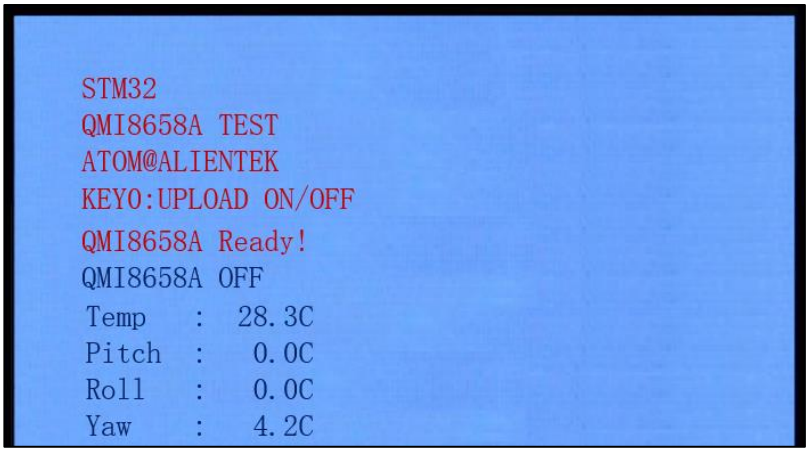


图 48.4.1 程序运行效果

屏幕显示了 QMI8658A 的温度、俯仰角（Pitch）、横滚角（Roll）和航向角（Yaw）的数值。然后，我们可以晃动开发板，看看各角度的变化。

另外，模块默认是关闭状态，通过按下 KEY0 可以开启或关闭数据上报，在开启状态下，我们可以打开：ANO_TC 匿名科创地面站 v4.exe 这个软件，接收 STM32H743 上传的数据，从而图形化显示飞行姿态，如图 48.4.2 所示：

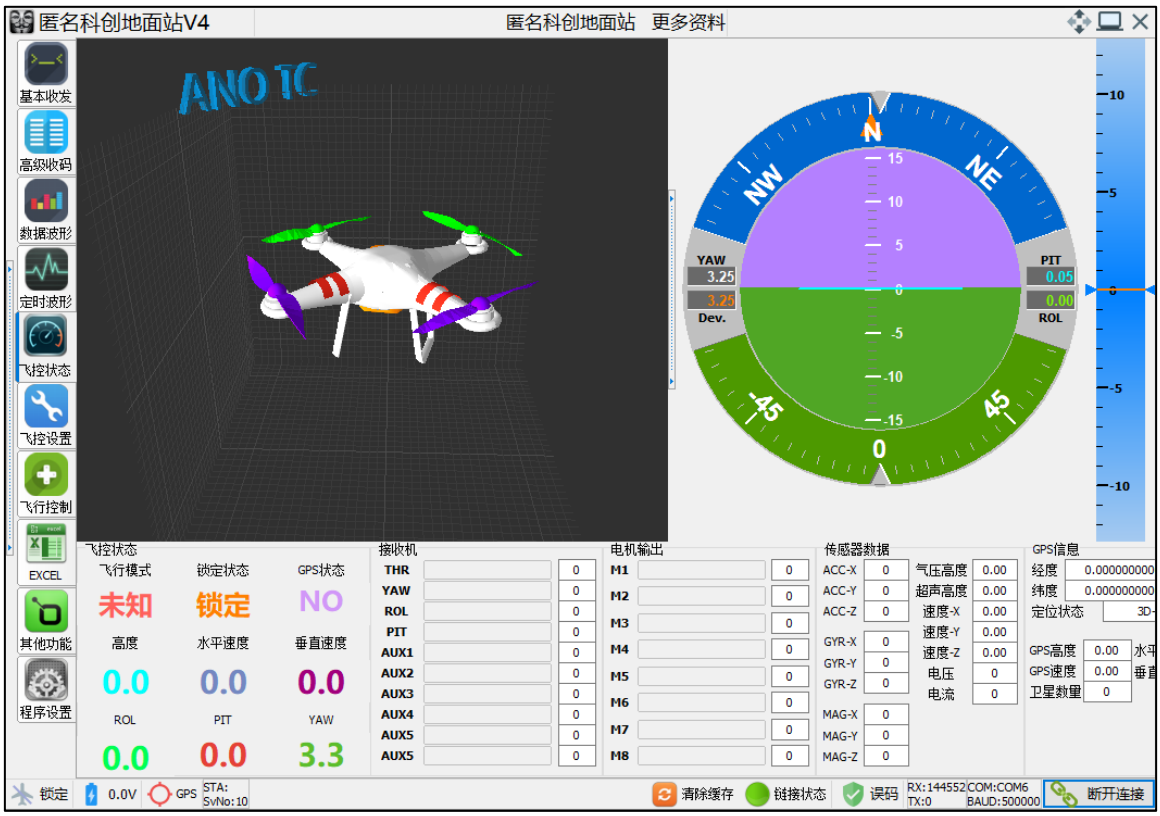


图 48.4.2 飞控状态显示

第四十九章 无线通信实验

本章，我们将介绍如何使用 2.4G 无线模块 NRF24L01 实现无线通信。将使用两块 STM32 开发板，一块用于发送，一块用于接收，从而实现无线数据传输，并把数据显示在 LCD 上。本章分为如下几个小节：

49.1 SPI&NRF24L01 无线模块介绍
49.2 硬件设计
49.3 程序设计
49.4 下载验证

49.1 SPI&NRF24L01 无线模块介绍

49.1.1 NRF24L01 简介

NRF24L01 无线模块，采用的芯片是 NRF24L01+。该芯片是由 NORDIC 公司生产，并且集成 NORDIC 自家的 Enhance ShortBurst 协议，主要特点如下：

1）2.4G 全球开放的 ISM 频段，免许可证使用
2）最高工作速率 2Mbps，高效的 GFSK 调制，抗干扰能力强
3）126 个可选的频道，满足多点通信和调频通信的需要
4）6 个数据通道可支持点对多点的通信地址控制
5）低工作电压（1.9~3.6V）
6）硬件 CRC 和自动处理字头
7）可设置自动应答，确保数据可靠传输

由于高速信号是由芯片内部的射频协议处理后进行无线高速通信，对 MCU 的时钟频率要求不高，只需要对 NRF24L01 某些寄存器进行配置即可。芯片与外部 MCU 是通过 SPI 通信接口进行数据通信，并且最大的 SPI 速度可达 10MHz。

这个芯片是 NRF24L01 的升级版。相比 NRF24L01，升级版支持 250k，1M，2M 三种传输速率；支持更多种功率配置，根据不同应用有效节省功耗；稳定性及可靠性更高。

该模块的外形和引脚图如图 49.1.1.1 所示：

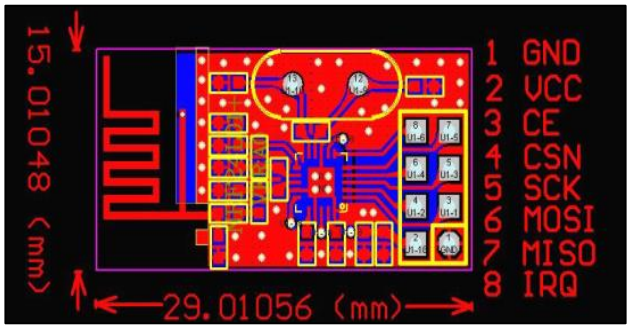


图 49.1.1.1 NRF24L01 无线模块外形和引脚图

模块 VCC 脚的电压范围为 1.9~3.6V，建议不要超过 3.6V，否则可能烧坏模块，一般用 3.3V 电压比较合适。除了 VCC 和 GND 脚，其他引脚都可以和 5V 单片机的 IO 口直连，正是因为其兼容 5V 单片机的 IO，所以使用上具有很大优势。

具体引脚介绍如表 49.1.1.1 所示。

模块引脚	GND	VCC	CE	CSN	SCK	MOSI	MISO	IRQ
功能说明	地线	3.3V 电源线	使能端	片选	时钟	数据输出	数据输入	中断

表 49.1.1.1 引脚介绍表

引脚部分主要分为电源相关的 VCC 和 GND，SPI 通信接口相关的 CSN/SCK/MOSI/MISO，模式选择相关的 CE，中断相关的 IRQ。CE 引脚会与 CONFIG 寄存器共同控制 NRF24L01 进入

某个工作模式。IRQ 引脚会在寄存器的配置下生效，当收到数据、成功发送数据或达到最大重发次数时，IRQ 引脚会变为低电平。

SPI 通信接口将会在 49.1.4 小节进行讲解。

NRF24L01 的 Enhance ShockBurstTM 模式具体表现在自动应答和重发机制，发送端要求接收端在接收到数据后要有应答信号，便于发送端检测有无数据丢失，一旦有数据丢失，则通过重发功能将丢失的数据恢复，这个过程无需 MCU。Enhance ShockBurstTM 模式可以通过 EN_AA 寄存器进行配置。

接下来看一下 Enhanced ShockBurstTM 模式下 NRF24L01 通信图，如图 49.1.1.2 所示：

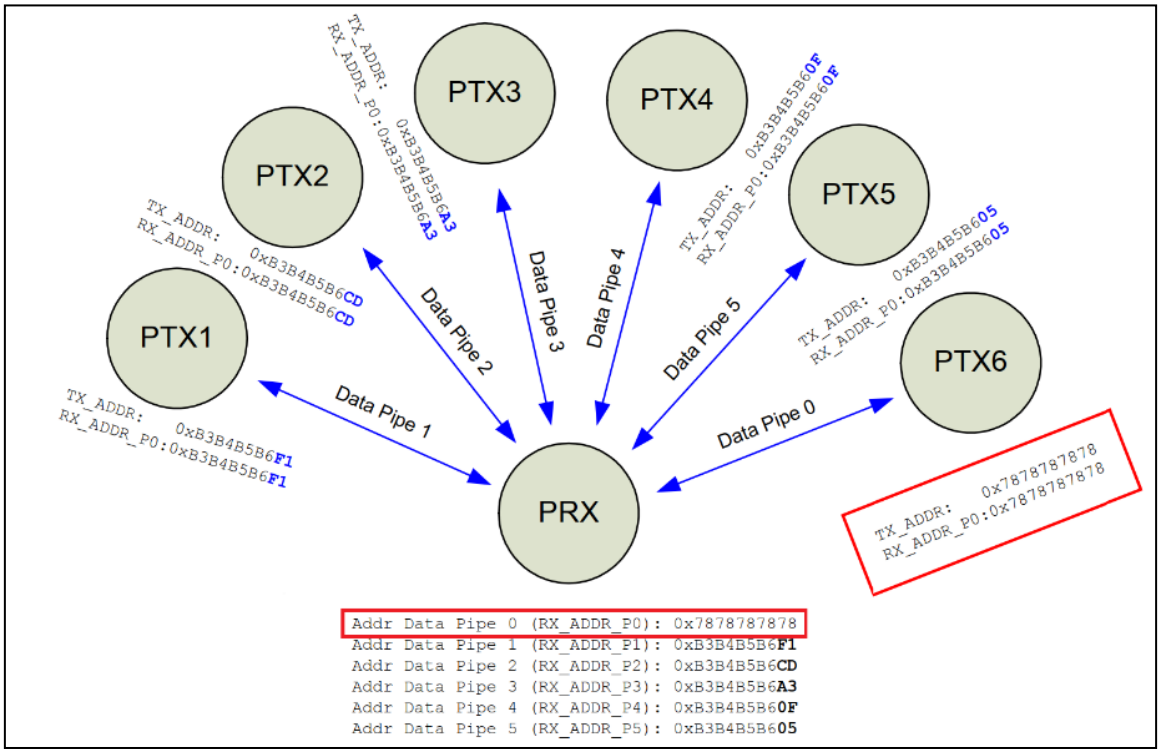


图 49.1.1.2 NRF24L01 通信图

这里我们抽离 PTX6 和 PRX 出来，分析一下通信过程。

PTX6 作为发送端，它就需要设置发送地址，可以看到 TX_ADDR 为 0x7878787878，PRX 作为接收端，它使能接收通道 0 并设置接收通道 0 接收地址 0x7878787878。通信时，发送端发送数据→接收端接收到数据并记录 TX 地址→接收端以 TX 地址为目的地址发送应答信号→发送端会以通道 0 接收应答信号。

NRF24L01 规定：发送端中的数据通道 0 是用来接收接收端发送的应答信号，所以数据通道 0 的接收地址要与发送地址要相同才能确保收到正确的应答信号，这里十分重要，必须要在相关寄存器中配置正确。

49.1.2 NRF24L01 工作模式介绍

NRF24L01 作为无线通信模块，功耗问题十分重要，有数据发送与空闲状态下能耗肯定是需要调整，所以设计者给芯片设计了多种工作模块，如表 49.1.2.1 所示：

24L01 模式	PWR_UP 位	PRIM_RX 位	CE 引脚电平	FIFO 寄存器状态
接收模式	1	1	1	-
发送模式	1	0	1	发送所有 TX FIFO 数据
发送模式	1	0	1（至少 10us）→0	发送一级 TX FIFO 数据
待机模式 II	1	0	1	TX FIFO 为空

待机模式 I	1	-	0	无数据包传输
掉电模式	0	-	-	-

表 49.1.2.1 NRF24L01 工作模式表

NRF24L01 工作模式是由 CE 引脚和 CONFIG 寄存器的 PWR_UP 位和 PRIM_RX 位共同控制。CE 引脚在前面也说到是模式控制线，而 PWR_UP 位是上电位，PRIM_RX 位可以理解为配置身份位(TX or RX)。可以看到发送模式有两种，待机模式也有两种，功耗上各不相同，没有标红的发送模式和待机模式 I 是官方推荐使用，更加节能，但是本实验用到的模式就是上表中标红色部分，因为标红的的模式使用起来更加方便。单看发送模式，使用官方推荐的发送模式，你要发送三级 TX_FIFO 数据需要产生三个边沿信号（CE 从高电平变为低电平）。而我们使用的发送模式，从 CE 引脚的操作上看，只需要拉高，就可以把所有 TX_FIFO 里的数据发送完成。

NRF24L01 的发送和接收都有三级 FIFO，每一级 FIFO 就有 32 个字节。发送和接收都是对 FIFO 进行操作，并且最大操作的数据量就是一级 FIFO 即 32 字节。发送时，只需要把数据存进 TX_FIFO 并按照发送模式下的操作（参考 NRF24L01 工作模式表中的发送模式）即可让 NRF24L01 启动发射，这个发送过程就包括：无线系统上电，启动内部 16MHz 时钟，无线发送数据打包，高速发送数据。接收时，也是通过读取 RX_FIFO 里的内容。

49.1.3 NRF24L01 寄存器

在这里简单介绍一下本实验用到的 NRF24L01 比较重要的寄存器。

- **配置寄存器（CONFIG）**

寄存器地址 0x01，复位值为 0x80，用来配置 NRF24L01 工作状态以及中断相关，描述如表 49.1.3.1 所示：

参数	位	描述
Reserved	7	保留位
MASK_RX_DR	6	可屏蔽中断 RX_RD（1：IRQ 引脚不显示 RX_RD 中断） （0：RX_RD 中断时，IRQ 输出低电平）
MASK_TX_DS	5	可屏蔽中断 TX_RD（1：IRQ 引脚不显示 TX_DS 中断） （0：TX_DS 中断时，IRQ 输出低电平）
MASK_MAX_RT	4	可屏蔽中断 MAX_RT（1：IRQ 引脚不显示 MAX_RT 中断） （0：MAX_RT 中断时，IRQ 输出低电平）
EN_CRC	3	CRC 使能（如果 EN_AA 中任意一位为高则 EN_CRC 强迫为高）
CRCO	2	CRC 模式（1：16 位 CRC 校验　0：8 位 CRC 校验）
PWR_UP	1	上电/掉电模式设置位（1：上电　0：掉电）
PRIM_RX	0	接收/发送模式设置位（1：接收模式　0：发送模式）

表 49.1.3.1 配置寄存器

需要配置成发送模式，可以把该寄存器赋值为 0x0E，如果配置成接收模式，可以把该寄存器赋值为 0x0F。无论是发送模式还是接收模式，都使能 16 位 CRC 以及使能接收中断、发送中断和最大重发次数中断，这里发送端和接收端配置需要一致。

- **自动应答功能寄存器（EN_AA）**

寄存器地址 0x01，复位值为 0x3F，用来设置通道 0~5 的自动应答功能，描述如表 49.1.3.2 所示：

参数	位	描述
Reserved	7:6	保留位
ENAA_P5	5	数据通道 5，自动应答允许
ENAA_P4	4	数据通道 4，自动应答允许
ENAA_P3	3	数据通道 3，自动应答允许

ENAA_P2	2	数据通道 2，自动应答允许
ENAA_P1	1	数据通道 1，自动应答允许
ENAA_P0	0	数据通道 0，自动应答允许

表 49.1.3.2 自动应答功能寄存器

本实验，接收端是以数据通道 0 作为接收通道，并且前面也提及 Enhanced ShockBurstTM 模式的自动应答流程，接收端接收到数据后，需要回复应答信号，通过该寄存器 ENAA_P0 置 1 即可实现。另外，使能自动应答也相当于配置成 Enhanced 模式，所以发送端也需要进行自动应答允许。

- **接收地址允许寄存器（EN_RXADDR）**

寄存器地址 0x02，复位值为 0x03，用于使能接收通道 0~5，描述如表 49.1.3.3 所示：

参数	位	描述
Reserved	7:6	保留位
ERX_P5	5	数据接收通道 5 使能
ERX_P4	4	数据接收通道 4 使能
ERX_P3	3	数据接收通道 3 使能
ERX_P2	2	数据接收通道 2 使能
ERX_P1	1	数据接收通道 1 使能
ERX_P0	0	数据接收通道 0 使能

图 49.1.3.3 接收地址允许寄存器

前面也说到接收端使用的是通道 0 进行接收数据，所以 ERX_P0 需要置 1 处理。同样的，发送端也需要使能数据通道 0 来接收应答信号。

- **地址宽度设置寄存器（SETUP_AW）**

寄存器地址 0x03，复位值为 0x03，对接收/发送地址宽度设置位，描述如表 49.1.3.4 所示：

参数	位	描述
Reserved	7:2	保留位
AW	1:0	RX/TX 地址字段宽度 ‘00’ 非法 ‘01’ 3 字节 ‘10’ 4 字节 ‘11’ 5 字节

表 49.1.3.4 地址宽度设置寄存器

本实验中，无论是发送地址还是接收地址都是使用 5 字节，也就是默认设置便是使用 5 字节宽度的地址。

- **自动重发配置寄存器（SETUP_RETR）**

寄存器地址 0x04，复位值为 0x00，对发送端的自动重发数值和延时进行设置，描述如表 49.1.3.5 所示：

参数	位	描述
ADR	7:4	自动重发延时： 0000~1111→86 us + 250 * (ARD + 1) us
ARC	3:0	自动重发计数 0000~1111→自动重发次数。0 代表禁止

表 49.1.3.5 自动重发配置寄存器图

本实验中，直接对该寄存器写入 0x1A，即自动重发间隔时间为 586us，最大自动重发次数为 10 次。在使能了 MAX_RT 中断时，连续重发 10 次还是发送失败的时候，IRQ 中断引脚就会拉低。

- **射频频率设置寄存器（RF_CH）**

寄存器地址 0x05，复位值为 0x05，对 NRF24L01 的频段进行设置，描述如表 49.1.3.6 所示：

参数	位	描述

Reserved	7	保留位
RF_CH	6:0	0～125，设置 NRF24L01 的射频频率，接收端和发送端需一致

表 49.1.3.6 射频频率设置寄存器图

频率计算公式：2400 + RF_CH（MHz）

本实验中，直接对该寄存器写入 40 即射频频率为 2440MHz。通信双方该寄存器必须配置一样才能通信成功。

- **发射参数设置寄存器（RF_SETUP）**

寄存器地址 0x06，复位值为 0x0E，对 NRF24L01 的发射功率、无线速率进行设置，描述如表 49.1.3.7 所示：

参数	位	描述
CONT_WAVE	7	高电平时，可使载波连续传输
Reserved	6	只允许写‘0’
RF_DR_LOW	5	设置射频数据速率 250kbps（结合 RF_DR_HIGH 查看）
PLL_LOCK	4	PLL_LOCK 允许，仅用于测试模式
RF_DR_HIGH	3	与 RF_DR_LOW 决定传输速率[RF_DR_LOW, RF_DR_HIGH] ‘00’：1Mbps ‘01’：2Mbps ‘10’：250kbps ‘11’：保留
RF_PWR	1:0	设置射频输出功率 ‘00’：-18dBm ‘01’：-12dBm ‘10’：-6dBm ‘11’：0dBm
Obsolete	0	不用关心

表 49.1.3.7 发射参数设置寄存器图

本实验中，直接对该寄存器写入 0x0F 即射频输出功率为 0dBm 增益，传输速率为 2MHz。发送端和接收端该寄存器的配置需一样。功率越小耗电越少，同等条件下，传输距离越小，这里我们设置射频部分功耗为最大，当然大家可以根据实际应用而选择对应的功率配置。

- **状态寄存器（STATUS）**

地址 0x07，复位值为 0x0E，反应 NRF24L01 当前工作状态，描述如表 49.1.3.8 所示：

参数	位	描述
Reserved	7	保留位
RX_DR	6	接收数据标记，收到数据后置 1。写 1 清除中断
TX_DS	5	数据发送完成标记。工作在自动应答模式，必须收到 ACK 才会置 1。写 1 清除中断。
MAX_RT	4	达到最大重发次数标记。写 1 清除中断（如果 MAX_RX 中断产生则必须清除后系统才能进行通信）
RX_P_NO	3:1	接收数据通道： 000~101 数据通道号 110 未使用 111 RX_FIFO 为空
TX_FULL	0	TX_FIFO 寄存器满标记（1：满 0：未满）

表 49.1.3.8 状态寄存器图

该寄存器作为查询作用，作为发送端，发送完数据后，可以查询一下 TX_DS 位状态便知是否成功发送数据，发送数据异常时，也可以通过查询 MAX_RT 位状态获知是否达到最大重发次数。作为接收端，就可以通过查询 RX_OK 位状态获知是否接收到数据。我们查询相关位后都需要将该位置 1 清除中断。

此外，我们还用到设置接收通道 0 地址寄存器 RX_ADDR_P0(0x0A)和发送地址设置寄存器 TX_ADDR(0x10)以及接收通道 0 有效数据看度设置寄存器 RX_PW_P0(0x11)，由于这三个寄存器比较简单，所以这里就不列出来了。

49.1.4 SPI 接口简介

SPI 是英语 Serial Peripheral interface 的缩写，顾名思义就是串行外围设备接口。是 Motorola 首先在其 MC68HCXX 系列处理器上定义的。SPI 接口主要应用在 EEPROM，FLASH，实时时钟，AD 转换器，还有数字信号处理器和数字信号解码器之间。

SPI，是一种高速的，全双工，同步的通信总线，并且在芯片的管脚上只占用四根线，节约了芯片的管脚，同时为 PCB 的布局上节省空间，提供方便，正是出于这种简单易用的特性，现在越来越多的芯片集成了这种通信协议，STM32H743 也有 SPI 接口，其框图如图 49.1.4.1 所示：

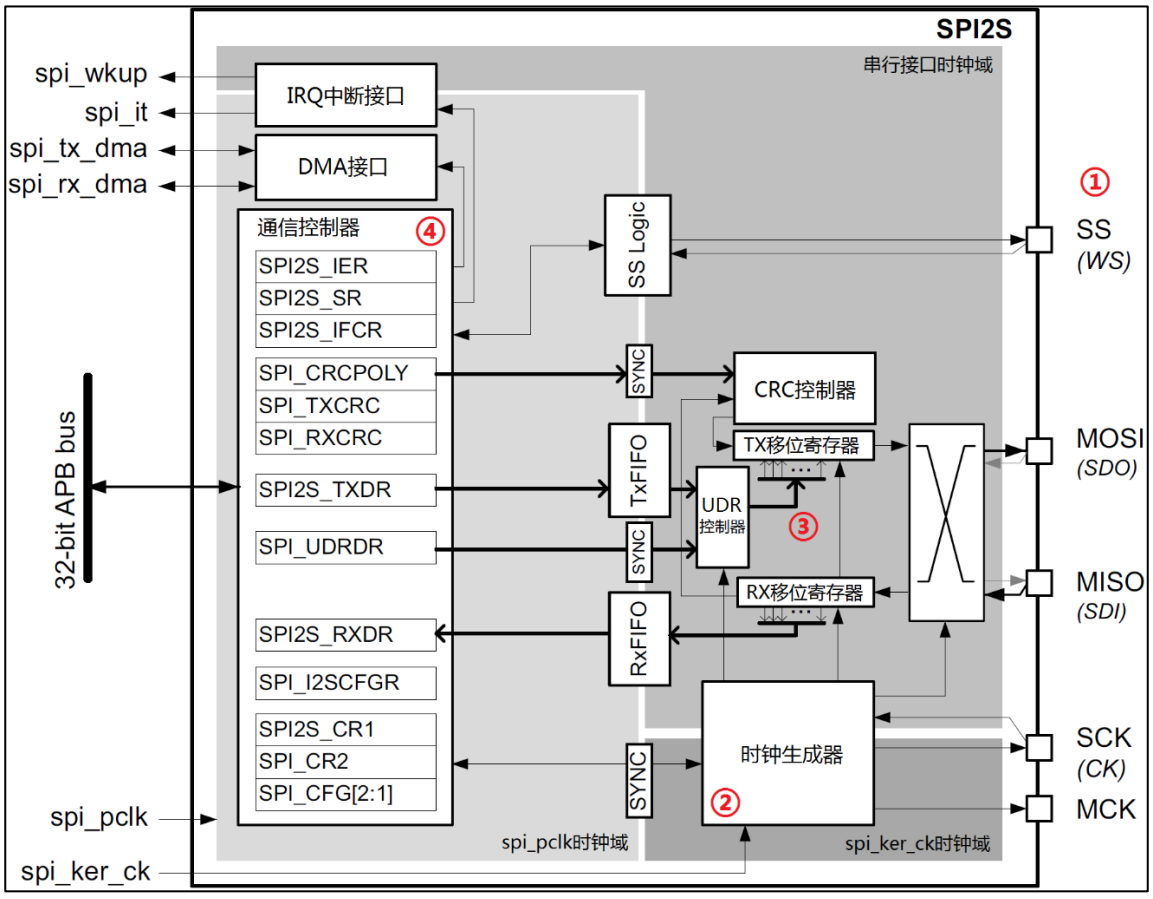


图 49.1.4.1 STM32H743 的 SPI 框图

图中我们挑出了 4 处和我们本章例程相关的位置进行重点介绍，其中①处是 SPI 的对外信号，一般由 4 根信号线组成（MCK 是 I2S 用的，不属于 SPI 信号线）：

MISO（SDI）：主设备数据输入，从设备数据输出。

MOSI（SDO）：主设备数据输出，从设备数据输入。

SCK（CK）：时钟信号，由主设备产生。

SS（WS）：从设备片选信号（也称 CS），由主设备控制。

图中②处是 SPI 的时钟生成器，将来自 spi_ker_ck 的时钟进行分频输出，最终产生 SCK 信号，并驱动 RX/TX 移位寄存器接收/发送数据。SPI1/2/3 的 spi_ker_ck 时通过 RCC_D2CCIP1R 寄存器的 SPI123SEL[2:0]设置，默认为 0，则 spi_ker_ck 来自 pll1_q_ck，为 200Mhz。

图中③处重点由：TX 移位寄存器、RX 移位寄存器等组成，分别用于发送数据和接收数据，TxFIFO 用于发送数据，RxFIFO 用于接收数据，我们可以设置 FIFO 大小为 1，这样写入 TXDR 的数据就可以直接输出到 TX 移位寄存器，在 SCK 时钟的驱动下，输出到 MOSI 脚。同理，当 FIFO 大小为 1 时，RX 移位寄存器接收到的数据，也会直接输出给 RXDR 寄存器。

图中④处是 SPI 的一堆控制寄存器，用于设置 SPI 的各种工作情况（或模式），该区域由 spi_pclk 时钟驱动。一般我们需要合理的设置这些寄存器，SPI 才可以正常使用。

SPI 总线有四种工作模式，如表 49.1.4.1 所示：

SPI 模式	CPOL	CPHA	SCK 空闲电平	数据采样点
0	0	0	低	在 SCK 第一个时钟跳变（上升/下降）沿采集
1	0	1	低	在 SCK 第二个时钟跳变（上升/下降）沿采集
2	1	0	高	在 SCK 第一个时钟跳变（上升/下降）沿采集
3	1	1	高	在 SCK 第二个时钟跳变（上升/下降）沿采集

表 49.1.4.1 SPI 四种工作模式

其中 CPOL 和 CPHA 分别控制 SCK 的时钟极性和相位时钟，可以通过相关寄存器设置。不同时钟/相位下的总线数据传输时序如图 49.1.4.2 所示：

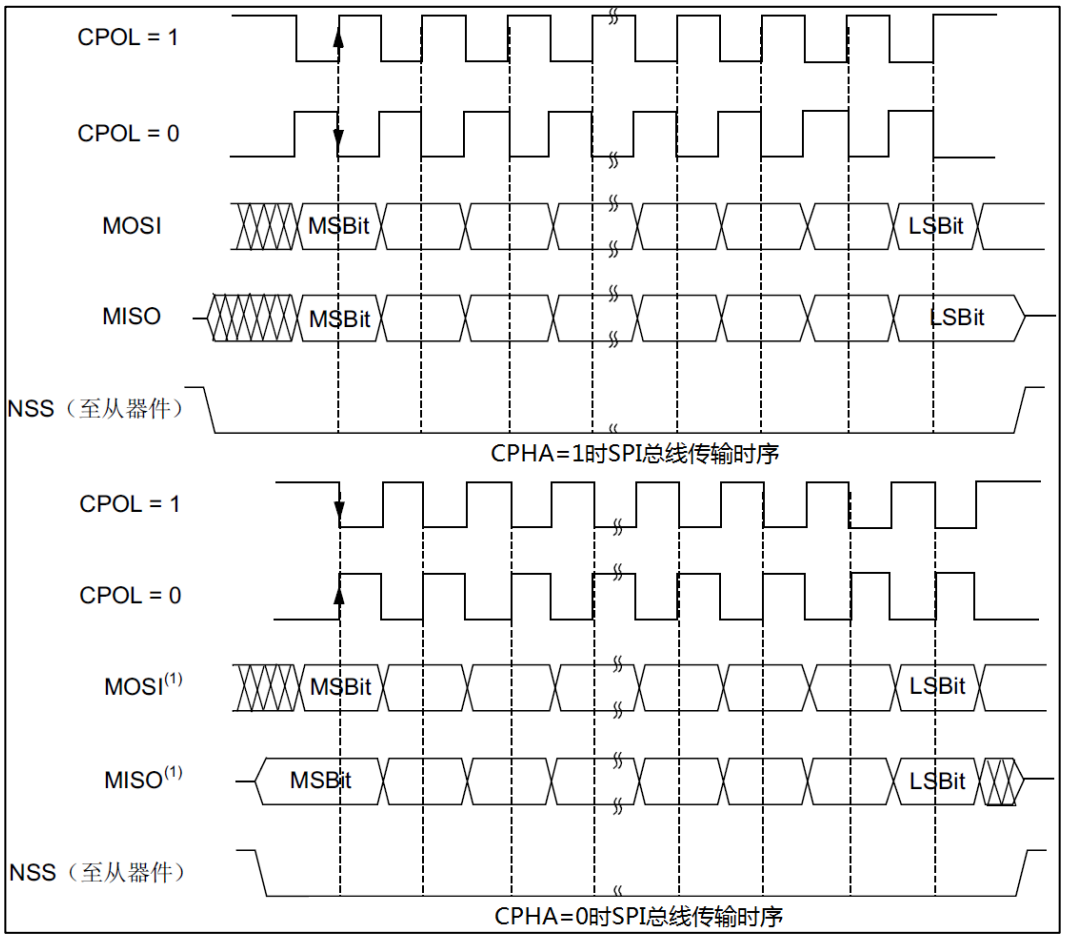


图 49.1.4.2 不同时钟相位下的总线传输时序（CPHA=0/1）

SPI 在驱动不同的器件时，必须注意其支持的工作模式，需要设置合适的 SPI 工作模式，才可以正常通信。

对于 STM32H7 来说，SPI 的 MSB 和 LSB 是可以配置的，通过 SPI_CFG2 的 LSBFIRST 位进行控制，当该位为 1 时，表示 LSB 在前；当该位为 0 时，表示 MSB 在前；

STM32H7 的 SPI 功能很强大，SPI 时钟最高可以到 133Mhz，支持 DMA，可以配置为 SPI 协议或者 I2S 协议（支持全双工 I2S）。

本章，我们将使用 STM32H743 的 SPI 来驱动 NRF24L01 无线模块。这里对 SPI 我们只简单介绍一下 SPI 的使用，STM32H743 的 SPI 详细介绍请参考《STM32H7xx 参考手册_V3（中文版）.pdf》第 2180 页，第 50 章。

SPI 主模式配置步骤

1）配置相关引脚的复用功能，使能 SPI2 时钟。

我们要用 SPI2，第一步就要使能 SPI2 的时钟，SPI2 的时钟通过 APB1LENR 的第 14 位来

设置。其次要设置 SPI2 的相关引脚为复用(AF5)输出，这样才会连接到 SPI2 上。这里我们使用的是 PB13、14、15 这 3 个（SCK.、MISO、MOSI，CS 使用软件管理方式），所以设置这三个为复用 IO，复用功能为 AF5。

2）设置 SPI2 的波特率和数据格式。

这一步通过 SPI2_CFG1 来设置，SPI2 的波特率（就是 SCK 的频率，最大 133Mhz）通过 SPI2_CFG1 寄存器的 MBR[2:0]来设置，可以设置为 spi_ker_ck 的 2~256 分频。而数据格式通过 SPI2_CFG1 寄存器的 DSIZE[4:0]位设置，数据长度等于 DSIZE[4:0]+1，比如，我们要设置为 8 位数据格式，则设置 DSIZE[4:0]=7 即可。

3）设置 SPI2 工作模式。

这一步通过 SPI2_CFG2 来设置，我们设置 SPI2 为主机模式，全双工，然后通过 CPOL 和 CPHA 位来设置 SCK 时钟极性及采样方式。并设置 SPI2 的 SS（即 CS）控制方式为软件控制。

4）使能 SPI2。

通过 SPI2_CR1 的 bit0 来设置，以启动 SPI2，在启动之后，我们就可以开始 SPI 通讯了。

49.2 硬件设计

1. 例程功能

开机的时候先检测 NRF24L01 模块是否存在，在检测到 NRF24L01 模块之后，根据 KEY0 和 KEY1 的设置来决定模块的工作模式。在设定好工作模式之后，就会不停的发送/接收数据，同时在 LCD 上面显示相关信息。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯 LED0 - PB1
2) 独立按键
KEY0 - PH3
KEY1 - PH2
3）2.4G 无线模块 NRF24L01 模块
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
5）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
6）SPI2（连接在 PB13/PB14/PB15 上）

3. 原理图

NRF24L01 模块与 STM32 的连接关系，如下图所示：

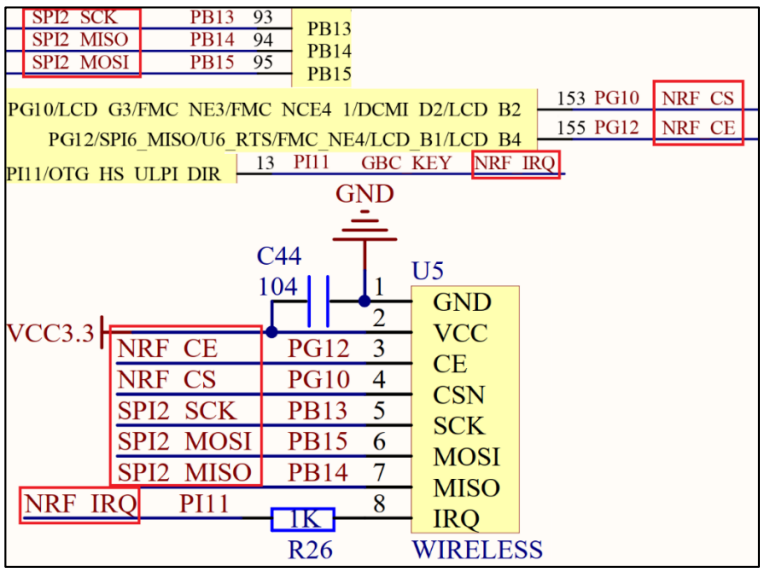


图 49.2.1 NRF24L01 模块接口与 STM32 连接原理图

这里NRF24L01使用的是SPI2，连接在PB13/PB14/PB15上。注意：NRF_IRQ和GBC_LED共用了PC3，所以，他们不能同时使用，需要分时复用。

由于无线通信实验是双向的，所以至少要有两个模块同时能工作，这里我们使用 2 套开发板来向大家演示。

49.3 程序设计

NRF24L01 配置步骤

1）SPI 参数初始化(工作模式、数据时钟极性、时钟相位等)。

HAL 库通过调用 SPI 初始化函数 HAL_SPI_Init 完成对 SPI 参数初始化，详见例程源码。

注意：该函数会调用：HAL_SPI_MspInit 函数来完成对 SPI 底层的初始化，包括：SPI 及 GPIO 时钟使能、GPIO 模式设置等。

2）使能 SPI 时钟和配置相关引脚的复用功能以及 NRF24L01 的其他相关管脚。

本实验用到 SPI2，使用 PB13、PB14 和 PB15 作为 SPI_SCK、SPI_MISO 和 SPI_MOSI，以及 NRF24L01 的 CE、CSN 和 IRQ 分别对应 PG12，PG10 和 PI11，因此需要先使能 SPI2、GPIOG 和 GPIOI 时钟。参考代码如下：

```
__HAL_RCC_GPIOx_CLK_ENABLE();              /* 使能 GPIOx 时钟，x=A……K */
__HAL_RCC_SPI2_CLK_ENABLE ();              /* 使能 SPI 时钟 */
```

GPIO 模式设置通过调用 HAL_GPIO_Init 函数实现，详见本例程源码。

3）使能 SPI

通过__HAL_SPI_ENABLE 函数使能 SPI，便可进行数据传输。

4）SPI 传输数据

通过 HAL_SPI_Transmit 函数进行发送数据。

通过 HAL_SPI_Receive 函数进行接收数据。

也可以通过 HAL_SPI_TransmitReceive 函数进行发送与接收操作。

5）编写 NRF24L01 的读写函数

基于 SPI 的读写函数的基础上，编写 NRF24L01 的读写函数。

6）编写 NRF24L01 接收模式与发送模式函数

通过查看寄存器，编写配置 NRF24L01 接收和发送模式的函数。

49.3.1 程序流程图

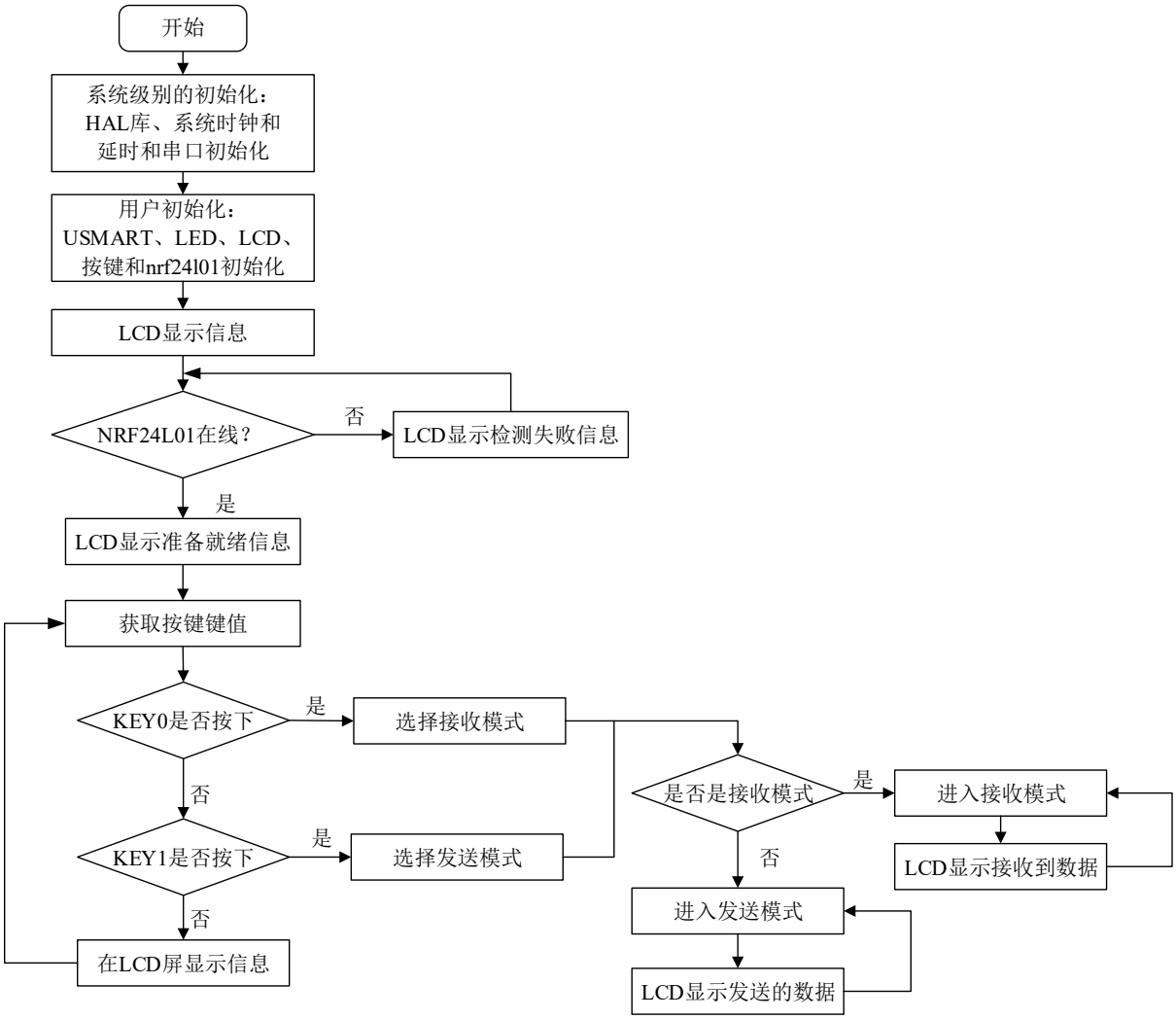


图 49.3.1.1 无线通信实验程序流程图

49.3.2 程序解析

1. SPI 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。SPI 驱动源码包括两个文件：spi.c 和 spi.h。

首先介绍的是 spi.h 文件，具体定义如下：

```
/* SPI2 引脚 定义 */
#define SPI2_SCK_GPIO_PORT              GPIOB
#define SPI2_SCK_GPIO_PIN               GPIO_PIN_13
#define SPI2_SCK_GPIO_CLK_ENABLE()
              do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)    /* PB 口时钟使能 */

#define SPI2_MISO_GPIO_PORT             GPIOB
#define SPI2_MISO_GPIO_PIN              GPIO_PIN_14
#define SPI2_MISO_GPIO_AF               GPIO_AF5_SPI2
#define SPI2_MISO_GPIO_CLK_ENABLE()
              do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)    /* PB 口时钟使能 */

#define SPI2_MOSI_GPIO_PORT             GPIOB
#define SPI2_MOSI_GPIO_PIN              GPIO_PIN_15
```

```
#define SPI2_MOSI_GPIO_AF                GPIO_AF5_SPI2
#define SPI2_MOSI_GPIO_CLK_ENABLE()
              do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)    /* PB 口时钟使能 */

/* SPI2 相关定义 */
#define SPI2_SPI                            SPI2
#define SPI2_SPI_CLK_ENABLE()
              do{ __HAL_RCC_SPI2_CLK_ENABLE(); }while(0)     /* SPI2 时钟使能 */
```

上面的宏定义是 SPI2 的接口（PB13\PB14\PB15）的相关宏定义。

下面介绍 spi.c 文件，首先是 SPI 初始化相关函数，具体如下：

```
SPI_HandleTypeDef g_spi2_handle;  /* SPI 句柄 */
/**
 * @brief       SPI 初始化代码
 * @note        主机模式,8 位数据,禁止硬件片选
 * @param       无
 * @retval      无
 */
void spi2_init(void)
{
    g_spi2_handle.Instance = SPI2;                    /* SP2 */
    /* 设置 SPI 工作模式，设置为主模式 */
    g_spi2_handle.Init.Mode = SPI_MODE_MASTER;
    /* 设置 SPI 单向或者双向的数据模式:SPI 设置为双线模式 */
    g_spi2_handle.Init.Direction = SPI_DIRECTION_2LINES;
    /* 设置 SPI 的数据大小:SPI 发送接收 8 位帧结构 */
    g_spi2_handle.Init.DataSize = SPI_DATASIZE_8BIT;
    /* 串行同步时钟的空闲状态为高电平 */
    g_spi2_handle.Init.CLKPolarity = SPI_POLARITY_HIGH;
    /* 串行同步时钟的第二个跳变沿（上升或下降）数据被采样 */
    g_spi2_handle.Init.CLKPhase = SPI_PHASE_2EDGE;
    /* NSS 信号由硬件（NSS 管脚）还是软件（使用 SSI 位）管理:内部 NSS 信号有 SSI 位控制 */
    g_spi2_handle.Init.NSS = SPI_NSS_SOFT;
    g_spi2_handle.Init.NSSPMode = SPI_NSS_PULSE_DISABLE;  /* NSS 信号脉冲失能 */
    /* SPI 主模式 IO 状态保持使能 */
    g_spi2_handle.Init.MasterKeepIOState = SPI_MASTER_KEEP_IO_STATE_ENABLE;
    /* 定义波特率预分频的值:波特率预分频值为 256 */
    g_spi2_handle.Init.BaudRatePrescaler = SPI_BAUDRATEPRESCALER_256;
    /* 指定数据传输从 MSB 位还是 LSB 位开始:数据传输从 MSB 位开始 */
    g_spi2_handle.Init.FirstBit = SPI_FIRSTBIT_MSB;
    /* 关闭 TI 模式 */
    g_spi2_handle.Init.TIMode = SPI_TIMODE_DISABLE;
    /* 关闭硬件 CRC 校验 */
    g_spi2_handle.Init.CRCCalculation = SPI_CRCCALCULATION_DISABLE;
    g_spi2_handle.Init.CRCPolynomial = 7;             /* CRC 值计算的多项式 */
    HAL_SPI_Init(&g_spi2_handle);

    __HAL_SPI_ENABLE(&g_spi2_handle);                 /* 使能 SPI2 */
    spi2_read_write_byte(0Xff);                       /* 启动传输 */
}

/**
 * @brief       SPI 底层驱动，时钟使能，引脚配置
 * @param       hspi:SPI 句柄
 * @note        此函数会被 HAL_SPI_Init()调用
 * @retval      无
 */
void HAL_SPI_MspInit(SPI_HandleTypeDef *hspi)
{
    GPIO_InitTypeDef gpio_init_struct;
    RCC_PeriphCLKInitTypeDef spi2_clk_init;
```

```
    SPI2_SPI_CLK_ENABLE();              /* SPI2 时钟使能 */
    SPI2_SCK_GPIO_CLK_ENABLE();         /* SPI2_SCK 脚时钟使能 */
    SPI2_MISO_GPIO_CLK_ENABLE();        /* SPI2_MISO 脚时钟使能 */
    SPI2_MOSI_GPIO_CLK_ENABLE();        /* SPI2_MOSI 脚时钟使能 */

    /* 设置 SPI2 的时钟源 */
    spi2_clk_init.PeriphClockSelection = RCC_PERIPHCLK_SPI2;
    /* SPI2 时钟源使用 PLL1Q */
    spi2_clk_init.Spi123ClockSelection = RCC_SPI123CLKSOURCE_PLL;
    HAL_RCCEx_PeriphCLKConfig(&spi2_clk_init);

    gpio_init_struct.Pin = SPI2_SCK_GPIO_PIN|
    SPI2_MISO_GPIO_PIN|SPI2_MOSI_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;                /* 复用推挽 */
    gpio_init_struct.Pull = GPIO_PULLUP;                    /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;     /* 高速 */
    gpio_init_struct.Alternate = GPIO_AF5_SPI2;             /* 复用 */
    HAL_GPIO_Init(SPI2_SCK_GPIO_PORT, &gpio_init_struct); /* 初始化 SCK 引脚 */
}
```

上面介绍的是 SPI2 初始化函数（SPI2_Init）和 SPI 初始化回调函数 HAL_SPI_MspInit。

最后介绍的是 SPI2 速度设置函数和 SPI2 读写一个字节数据函数，具体如下：

```
/**
 * @brief       SPI2 速度设置函数
 * @note        SPI2 时钟选择来自 pll1_q_ck, 为 200Mhz
 *              SPI 速度 = spi_ker_ck / 2^(speed + 1)
 * @param       speed: SPI2 时钟分频系数
 * @retval      无
 */
void spi2_set_speed(unsigned long speed)
{
    assert_param(IS_SPI_BAUDRATE_PRESCALER(speed));/* 判断有效性 */
    __HAL_SPI_DISABLE(&g_spi2_handle);              /* 关闭 SPI */
    g_spi2_handle.Instance->CR1 &= 0XFFC7 ;         /* 位 3-5 清零，用来设置波特率*/
    g_spi2_handle.Instance->CR1 |= speed;           /* 设置 SPI 速度 */
    __HAL_SPI_ENABLE(&g_spi2_handle);               /* 使能 SPI */
}
/**
 * @brief       SPI2 读写一个字节
 * @param       txdata:要写入的字节
 * @retval      返回值:读取到的字节
 */
uint8_t spi2_read_write_byte(uint8_t txdata)
{
    uint8_t rxdata;
    HAL_SPI_TransmitReceive(&g_spi2_handle,&txdata,&rxdata, 1, 1000);
    return rxdata;     /* 返回收到的数据 */
}
```

其中，spi2_set_speed 函数用于设置 SPI2 的传输速度也就是波特率，SPI2 的传输速度是通过 SPI2->CFG1 寄存器的位 3-5 来设置。形参 speed 是 SPI2 时钟分频系数，取值范围是：SPI_BAUDRATEPRESCALER_2~SPI_BAUDRATEPRESCALER_256。

而 spi2_read_write_byte 函数主要是通过调用 HAL 库中 SPI 发送接收函数 HAL_SPI_TransmitReceive 来实现数据的发送和接收。

2. NRF24L01 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。NRF24L01 驱动源码包括两个文件：nrf24l01.c 和 nrf24l01.h。

我们要在 spi.c 文件中封装好的函数的基础上进行调用，实现 nrf24l01 的发送与接收。下面先看一下 nrf24l01.h 文件中定义的信息，其代码如下：

```
/* NRF24L01 操作引脚 定义(不包含 SPI_SCK/MISO/MISO 等三根线) */
#define NRF24L01_CE_GPIO_PORT                GPIOG
#define NRF24L01_CE_GPIO_PIN                 GPIO_PIN_12
#define NRF24L01_CE_GPIO_CLK_ENABLE()
        do{ __HAL_RCC_GPIOG_CLK_ENABLE(); }while(0)    /* PG 口时钟使能 */

#define NRF24L01_CSN_GPIO_PORT               GPIOG
#define NRF24L01_CSN_GPIO_PIN                GPIO_PIN_10
#define NRF24L01_CSN_GPIO_CLK_ENABLE()
        do{ __HAL_RCC_GPIOG_CLK_ENABLE(); }while(0)    /* PG 口时钟使能 */

#define NRF24L01_IRQ_GPIO_PORT               GPIOI
#define NRF24L01_IRQ_GPIO_PIN                GPIO_PIN_11
#define NRF24L01_IRQ_GPIO_CLK_ENABLE()
        do{ __HAL_RCC_GPIOI_CLK_ENABLE(); }while(0)    /* PI 口时钟使能 */
/* 24L01 操作线 */
#define NRF24L01_CE(x)     do{ x ? \
  HAL_GPIO_WritePin(NRF24L01_CE_GPIO_PORT,NRF24L01_CE_GPIO_PIN,GPIO_PIN_SET):\
  HAL_GPIO_WritePin(NRF24L01_CE_GPIO_PORT,NRF24L01_CE_GPIO_PIN,GPIO_PIN_RESET);\
                                  }while(0)       /* 24L01 模式选择信号 */

#define NRF24L01_CSN(x)    do{ x ? \
  HAL_GPIO_WritePin(NRF24L01_CSN_GPIO_PORT,NRF24L01_CSN_GPIO_PIN,GPIO_PIN_SET):\
  HAL_GPIO_WritePin(NRF24L01_CSN_GPIO_PORT,
                                        NRF24L01_CSN_GPIO_PIN,GPIO_PIN_RESET);\
                                  }while(0)       /* 24L01 片选信号 */

#define NRF24L01_IRQ       HAL_GPIO_ReadPin(NRF24L01_IRQ_GPIO_PORT,
                                     NRF24L01_IRQ_GPIO_PIN) /* IRQ 主机数据输入 */
```

以上除了有 NRF24L01 的引脚定义及引脚操作函数，此外还有一些 NRF24L01 寄存器操作命令以及其寄存器地址，由于篇幅太大，所以这里就不列出来了，大家可以去看一下工程文件。

下面看一下 NRF24L01 的初始化函数，其定义如下：

```
/**
 * @brief       针对 NRF24L01 修改 SPI2 驱动
 * @param       无
 * @retval      无
 */
void nrf24l01_spi_init(void)
{
    __HAL_SPI_DISABLE(&g_spi2_handle);    /* 先关闭 SPI2 */
    /* 串行同步时钟的空闲状态为低电平 */
    g_spi2_handle.Init.CLKPolarity = SPI_POLARITY_LOW;
    /* 串行同步时钟的第 1 个跳变沿（上升或下降）数据被采样 */
    g_spi2_handle.Init.CLKPhase = SPI_PHASE_1EDGE;
    HAL_SPI_Init(&g_spi2_handle);
    __HAL_SPI_ENABLE(&g_spi2_handle);     /* 使能 SPI2 */
}

/**
 * @brief       初始化 24L01 的 IO 口
 * @note        将 SPI2 模式改成 SCK 空闲低电平,及 SPI 模式 0
 * @param       无
 * @retval      无
 */
void nrf24l01_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;

    NRF24L01_CE_GPIO_CLK_ENABLE();      /* CE 脚时钟使能 */
    NRF24L01_CSN_GPIO_CLK_ENABLE();     /* CSN 脚时钟使能 */
    NRF24L01_IRQ_GPIO_CLK_ENABLE();     /* IRQ 脚时钟使能 */
```

```
    gpio_init_struct.Pin = NRF24L01_CE_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;                /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;                        /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;              /* 高速 */
    HAL_GPIO_Init(NRF24L01_CE_GPIO_PORT, &gpio_init_struct);    /* 初始化 CE 引脚 */

    gpio_init_struct.Pin = NRF24L01_CSN_GPIO_PIN;
    HAL_GPIO_Init(NRF24L01_CSN_GPIO_PORT, &gpio_init_struct);   /* 初始化 CS 引脚 */

    gpio_init_struct.Pin = NRF24L01_IRQ_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_INPUT;                    /* 输入 */
    gpio_init_struct.Pull = GPIO_PULLUP;                        /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_HIGH;              /* 高速 */
    HAL_GPIO_Init(NRF24L01_IRQ_GPIO_PORT, &gpio_init_struct);   /* 初始化 CE 引脚 */

    spi2_init();                    /* 初始化 SPI2 */
    nrf24l01_spi_init();            /* 针对 NRF 的特点修改 SPI 的设置 */
    NRF24L01_CE(0);                 /* 使能 24L01 */
    NRF24L01_CSN(1);                /* SPI 片选取消 */
}
```

在初始化函数中，我们主要对该模块用到的管脚进行配置以及从初始化工作以及需要调用 spi.c 文件中的 spi2_init 函数对 SPI2 的引脚进行初始化。现在让我们看一下 NRF24L01 的工作时序图见下图 49.3.2.1 所示。

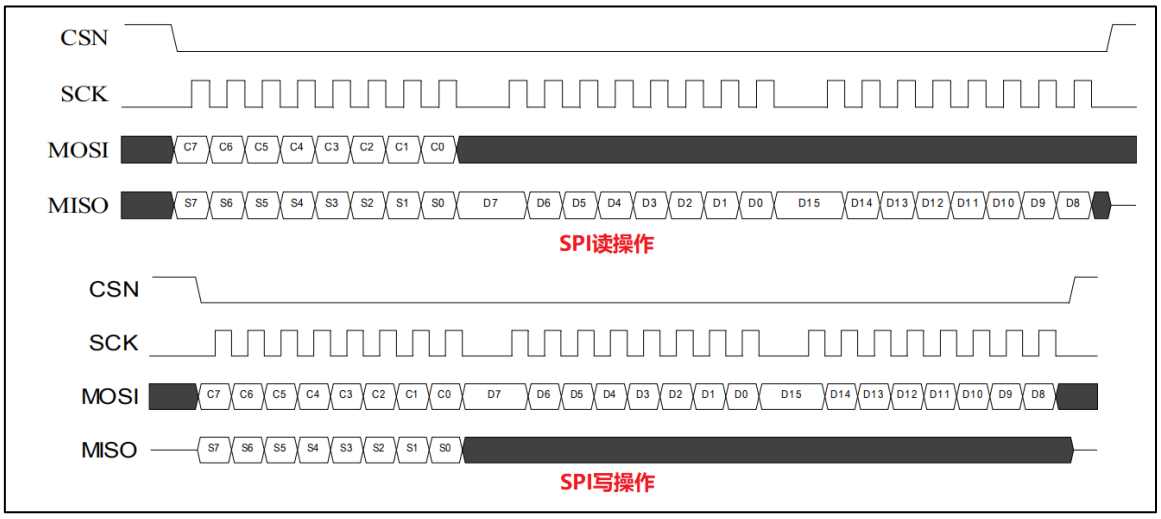


图 49.3.2.1 NRF24L01 的工作时序图

大家可以比对一下前面章节的 SPI 工作时序图，符合工作模式 1 的时序，即在奇数边沿上升沿进行数据的采集。所以我们调用 nrf24l01_spi_init 函数针对 NRF 的特点修改 SPI 的设置。该函数就是将 SPI 的工作模式配置成串行同步时钟空闲状态为低电平，在奇数边沿数据被采集，也就是前面 SPI 实验章节中 SPI 的工作模式 0。看看工作时序图的某些标号意义，Cn 代表指令位，Sn 代表状态寄存器位，Dn 代表数据位。

下面介绍一下 NRF24L01 的读写函数，其代码如下：

```
/**
 * @brief       NRF24L01 写寄存器
 * @param       reg   : 寄存器地址
 * @param       value : 写入寄存器的值
 * @retval      状态寄存器值
 */
static uint8_t nrf24l01_write_reg(uint8_t reg, uint8_t value)
{
    uint8_t status;
    NRF24L01_CSN(0);                        /* 使能 SPI 传输 */
    status = spi2_read_write_byte(reg);     /* 发送寄存器号 */
```

```
    spi2_read_write_byte(value);          /* 写入寄存器的值 */
    NRF24L01_CSN(1);                      /* 禁止 SPI 传输 */
    return status;                        /* 返回状态值 */
}

/**
 * @brief       NRF24L01 读寄存器
 * @param       reg    : 寄存器地址
 * @retval      读取到的寄存器值;
 */
static uint8_t nrf24l01_read_reg(uint8_t reg)
{
    uint8_t reg_val;
    NRF24L01_CSN(0);                                    /* 使能 SPI 传输 */
    spi2_read_write_byte(reg);                          /* 发送寄存器号 */
    reg_val = spi2_read_write_byte(0XFF);               /* 读取寄存器内容 */
    NRF24L01_CSN(1);                                    /* 禁止 SPI 传输 */
    return reg_val;                                     /* 返回状态值 */
}

/**
 * @brief       在指定位置读出指定长度的数据
 * @param       reg    : 寄存器地址
 * @param       pbuf   : 数据指针
 * @param       len    : 数据长度
 * @retval      状态寄存器值
 */
static uint8_t nrf24l01_read_buf(uint8_t reg, uint8_t *pbuf, uint8_t len)
{
    uint8_t status, i;
    NRF24L01_CSN(0);                        /* 使能 SPI 传输 */
    status = spi2_read_write_byte(reg);     /* 发送寄存器值(位置),并读取状态值 */

    for (i = 0; i < len; i++)
    {
        pbuf[i] = spi2_read_write_byte(0XFF);  /* 读出数据 */
    }

    NRF24L01_CSN(1);     /* 关闭 SPI 传输 */
    return status;       /* 返回读到的状态值 */
}

/**
 * @brief       在指定位置写指定长度的数据
 * @param       reg    : 寄存器地址
 * @param       pbuf   : 数据指针
 * @param       len    : 数据长度
 * @retval      状态寄存器值
 */
static uint8_t nrf24l01_write_buf(uint8_t reg, uint8_t *pbuf, uint8_t len)
{
    uint8_t status, i;
    NRF24L01_CSN(0);                       /* 使能 SPI 传输 */
    status = spi2_read_write_byte(reg); /* 发送寄存器值(位置),并读取状态值 */

    for (i = 0; i < len; i++)
    {
        spi2_read_write_byte(*pbuf++); /* 写入数据 */
    }

    NRF24L01_CSN(1);     /* 关闭 SPI 传输 */
    return status;       /* 返回读到的状态值 */
```

```
}
```

以上是 NRF24L01 的写寄存器函数和读寄存器函数，以及扩展的函数：在指定位置写入指定长度的数据函数和指定位置读取指定长度的数据函数。

先讲一下 NRF24L01 读写寄存器函数实现的具体过程：

先拉低片选线→发送寄存器号→发送数据/接收数据→拉高片选线。

这里提及一下 SPI 的相关知识：SPI 是通过移位寄存器进行数据传输，所以发一字节数据就会收到一个字节数据。那么发数据就可以直接发送数据，接收数据只需要发送 0xFF，寄存器会返回要读取的数据。

在指定位置写入指定长度的数据函数和在指定位置读取指定长度的数据函数的实现方式也是通过调用 SPI 的读写一字节函数实现，这里跟写寄存器和读寄存器函数的实现差不多，这里就不做展开了。

下面看一下这两种模式的初始化过程：

Rx 模式初始化过程：

1） 写 Rx 节点的地址
2） 使能通道 x 自动应答
3） 使能通道 x 接收地址
4） 设置通信频率
5） 选择通道 x 的有效数据宽度
6） 配置发射参数（发射功率、无线速率）
7） 配置 NRF24L01 的基本参数以及切换工作模式

其代码如下：

```
/**
 * @brief       NRF24L01 进入接收模式
 * @note        设置 RX 地址,写 RX 数据宽度,选择 RF 频道,波特率和 LNA HCURR
 *              当 CE 变高后,即进入 RX 模式,并可以接收数据了
 * @param       无
 * @retval      无
 */
void nrf24l01_rx_mode(void)
{
    NRF24L01_CE(0);
    nrf24l01_write_buf(NRF_WRITE_REG + RX_ADDR_P0, (uint8_t *)RX_ADDRESS,
                                                    RX_ADR_WIDTH);/* 写 RX 节点地址 */

    nrf24l01_write_reg(NRF_WRITE_REG + EN_AA, 0x01);     /* 使能通道 0 的自动应答 */
    nrf24l01_write_reg(NRF_WRITE_REG + EN_RXADDR, 0x01);/* 使能通道 0 的接收地址 */
    nrf24l01_write_reg(NRF_WRITE_REG + RF_CH, 40);       /* 设置 RF 通信频率 */
    /* 选择通道 0 的有效数据宽度 */
    nrf24l01_write_reg(NRF_WRITE_REG + RX_PW_P0, RX_PLOAD_WIDTH);
    /* 设置 TX 发射参数,0db 增益,2Mbps,低噪声增益开启 */
    nrf24l01_write_reg(NRF_WRITE_REG + RF_SETUP, 0x0f);
    /* 配置基本工作模式的参数;PWR_UP,EN_CRC,16BIT_CRC,接收模式 */
    nrf24l01_write_reg(NRF_WRITE_REG + CONFIG, 0x0f);
    NRF24L01_CE(1); /* CE 为高,进入接收模式 */
}
```

Tx 模式初始化过程：

1） 写 Tx 节点的地址
2） 写 Rx 节点的地址，主要为了使能硬件的自动应答
3） 使能通道 x 的自动应答
4） 使能通道 x 接收地址
5） 配置自动重发次数
6） 配置通信频率
7） 选择通道 x 的有效数据宽度
8） 配置发射参数（发射功率、无线速率）

9） 配置 NRF24L01 的基本参数以及切换工作模式

其代码如下：

```
/**
 * @brief       NRF24L01 进入发送模式
 * @note        设置 TX 地址,写 TX 数据宽度,设置 RX 自动应答的地址,填充 TX 发送数据,选择 RF 频
                道,波特率和 PWR_UP,CRC 使能
 *              当 CE 变高后,即进入 TX 模式,并可以发送数据了, CE 为高大于 10us,则启动发送.
 * @param       无
 * @retval      无
 */
void nrf24l01_tx_mode(void)
{
    NRF24L01_CE(0);
    nrf24l01_write_buf(NRF_WRITE_REG + TX_ADDR, (uint8_t *)TX_ADDRESS,
                                                TX_ADR_WIDTH); /* 写 TX 节点地址 */
    nrf24l01_write_buf(NRF_WRITE_REG + RX_ADDR_P0, (uint8_t *)RX_ADDRESS,
                               RX_ADR_WIDTH);    /* 设置 RX 节点地址,主要为了使能 ACK */

    nrf24l01_write_reg(NRF_WRITE_REG + EN_AA, 0x01);     /* 使能通道 0 的自动应答 */
    nrf24l01_write_reg(NRF_WRITE_REG + EN_RXADDR, 0x01);/* 使能通道 0 的接收地址 */
    /* 设置自动重发间隔时间:500us + 86us;最大自动重发次数:10 次 */
    nrf24l01_write_reg(NRF_WRITE_REG + SETUP_RETR, 0x1a);
    nrf24l01_write_reg(NRF_WRITE_REG + RF_CH, 40);       /* 设置 RF 通道为 40 */
    /* 设置 TX 发射参数,0db 增益,2Mbps,低噪声增益开启 */
    nrf24l01_write_reg(NRF_WRITE_REG + RF_SETUP, 0x0f);
    /* 配置基本工作模式的参数;PWR_UP,EN_CRC,16BIT_CRC,接收模式,开启所有中断 */
    nrf24l01_write_reg(NRF_WRITE_REG + CONFIG, 0x0e);
    NRF24L01_CE(1); /* CE 为高,10us 后启动发送 */
}
```

以上就是两种模式的配置，看过完整代码的，会发现 TX_ADDR 和 RX_ADDR 两个地址是一样的，跟前面说法一致，我们必须保持地址的匹配才能通信成功。以上代码中的发送函数都有一个特点，并不是单纯发送寄存器地址，而是**操作指令+寄存器地址**，这一点需要记得。NRF24L01 的操作指令也有好几个，它是配合寄存器完成特定的操作，其定义如下：

```
/* NRF24L01 寄存器操作命令 */
#define NRF_READ_REG    0x00    /* 读配置寄存器,低 5 位为寄存器地址 */
#define NRF_WRITE_REG   0x20    /* 写配置寄存器,低 5 位为寄存器地址 */
#define RD_RX_PLOAD     0x61    /* 读 RX 有效数据,1~32 字节 */
#define WR_TX_PLOAD     0xA0    /* 写 TX 有效数据,1~32 字节 */
#define FLUSH_TX        0xE1    /* 清除 TX FIFO 寄存器.发射模式下用 */
#define FLUSH_RX        0xE2    /* 清除 RX FIFO 寄存器.接收模式下用 */
#define REUSE_TX_PL     0xE3    /* 重新使用上一包数据,CE 为高,数据包被不断发送. */
#define NOP             0xFF    /* 空操作,可以用来读状态寄存器 */
```

经过上面的发送或者接收模式初始化步骤后，NRF24L01 就可以准备启动发送数据或者等待接收数据了。

下面来看一下启动 NRF24L01 发送一次数据的函数，其定义如下：

```
/**
 * @brief       启动 NRF24L01 发送一次数据(数据长度 = TX_PLOAD_WIDTH)
 * @param       ptxbuf : 待发送数据首地址
 * @retval      发送完成状态
 *   @arg       0     : 发送成功
 *   @arg       1     : 达到最大发送次数,失败
 *   @arg       0XFF  : 其他错误
 */
uint8_t nrf24l01_tx_packet(uint8_t *ptxbuf)
{
    uint8_t sta;
    uint8_t rval = 0XFF;

   NRF24L01_CE(0);
```

```
    /* 写数据到 TX BUF  TX_PLOAD_WIDTH 个字节 */
    nrf24l01_write_buf(WR_TX_PLOAD, ptxbuf, TX_PLOAD_WIDTH);
    NRF24L01_CE(1); /* 启动发送 */

    while (NRF24L01_IRQ != 0);           /* 等待发送完成 */

    sta = nrf24l01_read_reg(STATUS);    /* 读取状态寄存器的值 */
    nrf24l01_write_reg(NRF_WRITE_REG + STATUS,sta);/*清除 TX_DS 或 MAX_RT 中断标志*/

    if (sta & MAX_TX)    /* 达到最大重发次数 */
    {
        nrf24l01_write_reg(FLUSH_TX, 0xff); /* 清除 TX FIFO 寄存器 */
        rval = 1;
    }

    if (sta & TX_OK)    /* 发送完成 */
    {
        rval = 0;       /* 标记发送成功 */
    }
    return rval;        /* 返回结果 */
}
```

在这里启动发送数据函数中，具体实现很简单，拉低片选信号→向发送数据寄存器写入数据→拉高片选信号。这里说明一下，在发送完寄存器号后都会返回一个 status 值，返回的这个值就是前面介绍的 STATUS 寄存器的内容。在这个基础上就可以知道数据是否发送完成以及现在的状态。

然后介绍一下 NRF24L01 接收一次数据函数，其定义如下：

```
/**
 * @brief       启动 NRF24L01 接收一次数据(数据长度 = RX_PLOAD_WIDTH)
 * @param       prxbuf : 接收数据缓冲区首地址
 * @retval      接收完成状态
 *   @arg       0 : 接收成功
 *   @arg       1 : 失败
 */
uint8_t nrf24l01_rx_packet(uint8_t *prxbuf)
{
    uint8_t sta;
    uint8_t rval = 1;

    sta = nrf24l01_read_reg(STATUS); /* 读取状态寄存器的值 */
    nrf24l01_write_reg(NRF_WRITE_REG+STATUS, sta);/*清除 TX_DS 或 MAX_RT 中断标志*/

    if (sta & RX_OK)        /* 接收到数据 */
    {
        nrf24l01_read_buf(RD_RX_PLOAD, prxbuf, RX_PLOAD_WIDTH); /* 读取数据 */
        nrf24l01_write_reg(FLUSH_RX, 0xff); /* 清除 RX FIFO 寄存器 */
        rval = 0;           /* 标记接收完成 */
    }
    return rval;            /* 返回结果 */
}
```

在启动接收的过程中，首先需要判断当前 NRF24L01 的状态，往后才是真正的读取数据，清除接收寄存器的缓冲，完成数据的接收。这里需要注意的是我们通过 RX_PLOAD_WIDTH 和 TX_PLOAD_WIDTH 决定了接收和发送的数据宽度，这也决定每次发送和接收的有效字节数。NRF24L01 每次最多传输 32 个字节，再多的字节传输则需要多次传输。通信双方的发送和接收数据宽度必须一致才能正常通信。

3. main.c 代码

在 main.c 里编写如下代码：

```
int main(void)
```

```
{
    uint8_t key, mode;
    uint16_t t = 0;
    uint8_t tmp_buf[33];

    sys_cache_enable();                             /* 打开 L1-Cache */
    HAL_Init();                                     /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);             /* 设置时钟, 400Mhz */
    delay_init(400);                                /* 延时初始化 */
    usart_init(115200);                             /* 串口初始化为 115200 */
    mpu_memory_protection();                        /* 保护相关存储区域 */
    led_init();                                     /* 初始化 LED */
    sdram_init();                                   /* 初始化 SDRAM */
    lcd_init();                                     /* 初始化 LCD */
    key_init();                                     /* 初始化按键 */
    nrf24l01_init();                                /* 初始化 NRF24L01 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "NRF24L01 TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    while (nrf24l01_check())            /* 检查 NRF24L01 是否在线 */
    {
        lcd_show_string(30, 110, 200, 16, 16, "NRF24L01 Error", RED);
        delay_ms(200);
        lcd_fill(30, 110, 239, 130 + 16, WHITE);
        delay_ms(200);
    }
   lcd_show_string(30, 110, 200, 16, 16, "NRF24L01 OK", RED);

    while (1)   /* 提醒用户选择模式 */
    {
        key = key_scan(0);
        if (key == KEY0_PRES)
        {
            mode = 0;   /* 接收模式 */
            break;
        }
        else if (key == KEY1_PRES)
        {
            mode = 1;   /* 发送模式 */
            break;
        }

        t++;
        if (t == 100)   /* 显示提示信息 */
        {
            lcd_show_string(10, 130, 230, 16, 16,
                            "KEY0:RX_Mode  KEY1:TX_Mode", RED);
        }
        if (t == 200)   /* 关闭提示信息 */
        {
            lcd_fill(10, 130, 230, 150 + 16, WHITE);
            t = 0;
        }
        delay_ms(5);
    }
    lcd_fill(10, 130, 240, 166, WHITE);     /* 清空上面的显示 */

    if (mode == 0)  /* RX 模式 */
    {
        lcd_show_string(30, 130, 200, 16, 16, "NRF24L01 RX_Mode", BLUE);
        lcd_show_string(30, 150, 200, 16, 16, "Received DATA:", BLUE);
```

```
        nrf24l01_rx_mode();     /* 进入 RX 模式 */

        while (1)
        {
            if (nrf24l01_rx_packet(tmp_buf) == 0) /* 一旦接收到信息,则显示出来. */
            {
             tmp_buf[32] = 0;  /* 加入字符串结束符 */
             lcd_show_string(0,170,lcddev.width-1,32,16,(char*)tmp_buf, BLUE);
            }
            else delay_us(100);

            t++;
            if (t == 10000)      /* 大约 1s 钟改变一次状态 */
            {
                t = 0;
                LED0_TOGGLE();
            }
        }
    }
    else    /* TX 模式 */
    {
        lcd_show_string(30, 130, 200, 16, 16, "NRF24L01 TX_Mode", BLUE);
        nrf24l01_tx_mode();    /* 进入 TX 模式 */
        mode = ' ';            /* 从空格键开始发送 */

        while (1)
        {
            if (nrf24l01_tx_packet(tmp_buf) == 0)    /* 发送成功 */
            {
             lcd_show_string(30, 150, 239, 32, 16, "Sended DATA:", BLUE);
             lcd_show_string(0,190,lcddev.width-1,32,16,(char*)tmp_buf, BLUE);
             key = mode;

             for (t = 0; t < 32; t++)
             {
                key++;
                if (key > ('~'))
                    key = ' ';
                    tmp_buf[t] = key;
             }

             mode++;
             if (mode > '~')
                 mode = ' ';
                tmp_buf[32] = 0; /* 加入结束符 */
            }
            else
            {
             lcd_fill(0, 170,lcddev.width, 170 + 16 * 3, WHITE);/* 清空显示 */
             lcd_show_string(30,170,lcddev.width-1,32,16, "Send Failed ",BLUE);
            }

            LED0_TOGGLE();
            delay_ms(200);
        };
    }
}
```

程序运行时先通过 nrf24l01_cheak 函数检测 NRF24L01 是否存在，如果存在，则让用户选择发送模式还是接收模式，在确定模式之后，设置 NRF24L01 的工作模式，然后执行相对应的数据发送/接收处理。

49.4 下载验证

将程序下载到开发板后，可以看到 LCD 显示的内容如图 49.4.1 所示：

图 49.4.1 选择工作模式界面

通过 KEY0 和 KEY1 来选择 NRF24L01 模块所要进入的工作模式，我们两个开发板一个选择发送模式，一个选择接收模式就可以了。

设置好的通信界面如图 49.4.2 和图 49.4.3 所示：

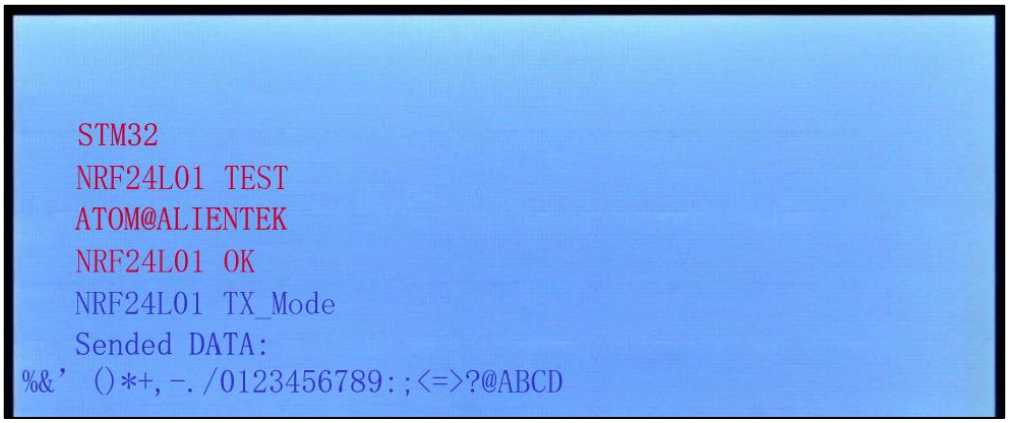


图 49.4.2 开发板 A 通信界面（发送）

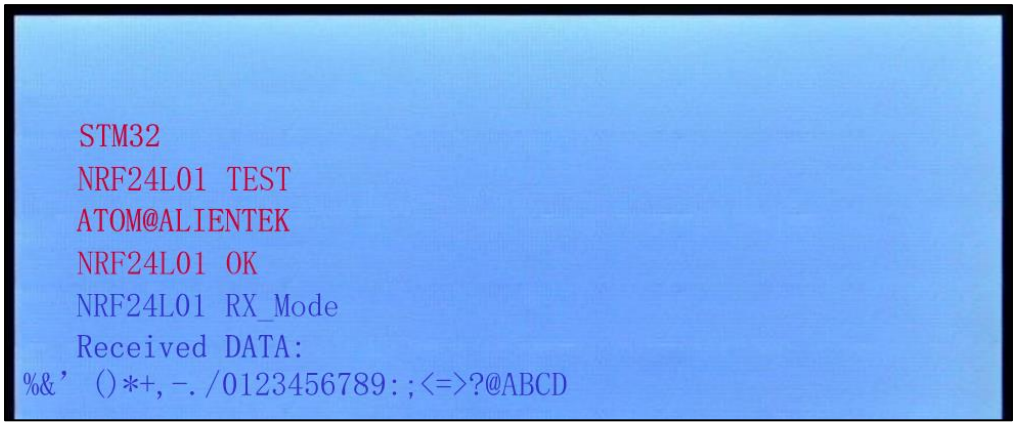


图 49.4.3 开发板 B 通信界面（接收）

图 49.4.2 来自于开发板 A，工作在发送模式。图 49.4.3 来自于开发板 B，工作在接收模式，A 发送，B 接收。发送和接收图片的数据不一样，是因为我们拍照的时间不一样导致的。大家看到收发数据是一致，那就说明实验成功。

第五十章 FLASH 模拟 EEPROM 实验

STM32H743 本身没有自带 EEPROM，但是 STM32H743 具有 IAP（在应用编程）功能，所以我们可以把它的 FLASH 当成 EEPROM 来使用。本章，我们将利用 STM32H743 内部的 FLASH 来实现第四十章实验类似的效果，不过这次我们是将数据直接存放在 STM32H743 内部，而不是存放在 NOR FLASH。

本章分为如下几个小节：

50.1 STM32H743 FLASH 简介
50.2 硬件设计
50.3 程序设计
50.4 下载验证

50.1 STM32H743 FLASH 简介

STM32H743IIT6 的 FLASH 容量为 2048K 字节（2MB），STM32H743IIT6 的闪存模块组织如表 50.1.1 所示：

块	名称	FLASH 起始地址	大小
Bank1	扇区 0	0X0800 0000-0X0801 FFFF	128K
	扇区 1	0X0802 0000-0X0803 FFFF	128K
	扇区 2	0X0804 0000-0X0805 FFFF	128K
	扇区 3	0X0806 0000-0X0807 FFFF	128K
	扇区 4	0X0808 0000-0X0809 FFFF	128K
	扇区 5	0X080A 0000-0X080B FFFF	128K
	扇区 6	0X080C 0000-0X080D FFFF	128K
	扇区 7	0X080E 0000-0X080F FFFF	128K
	系统存储器	0X1FF0 0000-0X1FF1 FFFF	128K
Bank2	扇区 0	0X0810 0000-0X0811 FFFF	128K
	扇区 1	0X0812 0000-0X0813 FFFF	128K
	扇区 2	0X0814 0000-0X0815 FFFF	128K
	扇区 3	0X0816 0000-0X0817 FFFF	128K
	扇区 4	0X0818 0000-0X0819 FFFF	128K
	扇区 5	0X081A 0000-0X081B FFFF	128K
	扇区 6	0X081C 0000-0X081D FFFF	128K
	扇区 7	0X081E 0000-0X081F FFFF	128K
	系统存储器	0X1FF4 0000-0X1FF5 FFFF	128K

表 50.1.1 STM32H743 闪存模块组织

STM32H743IIT6 的 2MB 分为 2 个块：Bank1 和 Bank2，每个块包含 8 个扇区和 1 个 128KB 大小的系统存储器，每个扇区的大小也是 128KB，总共有 18*128=2304KB FLASH 大小。但是，用户可用的 FLASH 只有两个 Bank 里面的扇区 0~7，系统存储器所占用的 FLASH 空间是用来存储 STM32H743 的 bootloader 代码，此代码是出厂的时候就固化在 STM32H743 里面了，专门来给主存储器下载代码的，用户无法使用。因此用户可用的 FLASH 大小是：2*8*128=2048KB，刚好是 2MB。

另外，在 Bank1 内部，还有一个 2KB 字节大小的用户选项字节，可以控制读保护、写保护、看门狗和启动地址等，不过这 2KB FLASH 并没有映射到内存地址上面，只能通过 FLASH 相关寄存器进行访问。关于用户选项字节 FLASH 的详细说明，详见《STM32H7xx 参考手册_V7（英文版）.pdf》第 4.3 小节相关内容。

在执行闪存写操作时，任何对闪存的读操作都会锁住总线，在写操作完成后读操作才能正

确地进行。即在进行写或擦除操作时，不能进行代码或数据的读取操作。

50.1.1 闪存的读取

为了准确读取 Flash 数据，必须根据 ACLK 时钟 (rcc_aclk) 频率和 Vcore 电压范围在 Flash 存取控制寄存器 (FLASH_ACR) 中正确地设置等待周期数 (LATENCY)。Flash 等待周期与 ACLK 时钟频率之间的对应关系，如表 50.1.1.2 所示：

Number of wait states (LATENCY)	Programming delay (WRHIGHFREQ)	AXI Interface clock frequency vs V_{CORE} range			
		VOS3 range 0.95 V - 1.05 V	VOS2 range 1.05 V - 1.15 V	VOS1 range 1.15 V - 1.26 V	VOS0 range 1.26 V - 1.40 V
0 WS (1 FLASH clock cycle)	00	[0;45 MHz]	[0;55 MHz]	[0;70 MHz]	[0;70 MHz]
1 WS (2 FLASH clock cycles)	01	]45 MHz;90 MHz]	]55 MHz;110 MHz]	]70 MHz;140 MHz]	]70 MHz;140 MHz]
2 WS (3 FLASH clock cycles)	01	]90 MHz;135 MHz]	]110 MHz;165 MHz]	]140 MHz;185 MHz]	]140 MHz;185 MHz]
	10	-	-	]185 MHz;210 MHz]	]185 MHz;210 MHz]
3 WS (4 FLASH clock cycles)	10	]135 MHz;180 MHz]	]165 MHz;225 MHz]	]210 MHz;225 MHz]	]210 MHz;225 MHz]
4 WS (5 FLASH clock cycles)	10	]180 MHz;225 MHz]	225 MHz	-	]225 MHz;240 MHz]

表 50.1.1.2 ACLK 时钟（rcc_aclk）频率对应的 FLASH 等待周期表

等待周期通过 FLASH_ACR 寄存器的 LATENCY[2:0]三个位设置。系统复位后，CPU 时钟频率为内部 64M RC 振荡器（HSI），LATENCY 默认是 0，即 1 个等待周期。为了得到更佳的 FLASH 访问性能，我们设置 Vcore 电压范围为 VOS1 级别（1.15V~1.26V）。rcc_aclk 和 rcc_hclk3 的频率是一样的，都是来自 RCC_D1CFGR 的 HPRE[3:0]分频，rcc_hclk3 我们一般设置的是 200Mhz，这样 rcc_aclk 也是 200Mhz 的频率。我们设置等待周期为 4（LATENCY[2:0]=4），否则 FLASH 读写可能出错，导致死机。寄存器的设置和 HAL 库的是不一样的，这个请大家注意一下。这个知识点我们在 11.2.1 小节讲解过，请回顾。

STM32H743 的 FLASH 读取是很简单的。例如，我们要从地址 addr，读取一个字（一个字为 32 位），可以通过如下的语句读取：

```
Data = *(volatile uint32_t *)faddr;
```

将 faddr 强制转换为 volatile uint32_t 指针，然后取该指针所指向的地址的值，即得到了 faddr 地址的值。类似的，将上面的 volatile uint32_t 改为 volatile uint8_t，即可读取指定地址的一个字节。相对 FLASH 读取来说，STM32H743 FLASH 的写就复杂一点了，下面我们介绍 STM32H743 闪存的编程和擦除。

50.1.2 闪存的编程和擦除

在对 STM32H743 的 Flash 执行写入或擦除操作期间，任何读取 Flash 的尝试都会导致总线阻塞。只有在完成编程操作后，才能正确处理读操作。这意味着，写/擦除操作进行期间不能从 Flash 中执行代码或数据获取操作。

STM32H743 用户闪存的编程一般由 9 个 32 位寄存器控制，他们分别是：

- FLASH 访问控制寄存器(FLASH_ACR)
- FLASH 秘钥寄存器 1(FLASH_KEYR1)
- FLASH 秘钥寄存器 2(FLASH_KEYR2)
- FLASH 状态寄存器 1(FLASH_SR1)
- FLASH 状态寄存器 2(FLASH_SR2)
- FLASH 控制寄存器 1(FLASH_CR1)

- FLASH 控制寄存器 2(FLASH_CR2)
- FLASH 清除与控制寄存器 1(FLASH_CCR1)
- FLASH 清除与控制寄存器 2(FLASH_CCR2)

注意：这里的 FLASH_KEYR1/2、FLASH_SR1/2 、FLASH_CR1/2、FLASH_CCR1/2 分别对应 Bank1/Bank2 的相关寄存器，所以单个 Bank 的控制寄存器由：FLASH_KEYR、SR、CR 和 CCR 等四个寄存器控制。下面，我们直接以 FLASH_KEYR、FLASH_CR、FLASH_SR 和 FLASH_CCR 来介绍相关操作。

STM32H743 复位后，FLASH 编程操作是被保护的，不能写入 FLASH_CR 寄存器；通过写入特定的序列（0X45670123 和 0XCDEF89AB）到 FLASH_KEYR 寄存器才可解除写保护，只有在写保护被解除后，我们才能操作相关寄存器。

FLASH_CR 的解锁序列为：

1， 写 0X45670123 到 FLASH_KEYR

2， 写 0XCDEF89AB 到 FLASH_KEYR

通过这两个步骤，即可解锁 FLASH_CR，如果写入错误，那么 FLASH_CR 将被锁定，直到下次复位后才可以再次解锁。

STM32H743 闪存的编程位数固定为 256 位，也就是每次写入数据必须为 8 个字（32 字节），如果不够 8 个字，可以在后面进行补零写入，否则后续的内容将不可预知。而且，写入首地址必须是 32 的整数倍，否则会影响前后数据。

举例来说，假设我们要往：0X0810 0000 这个地址写入一个字节的数据，则写入一个字节数据的同时，也会影响接下来 31 字节的内容，因此，如果你只需要写入一个字节，则可以将后面的 31 字节全部填充成 0，然后组成一个 32 字节数组（256 位），一次写入。而且，如果你写入数据的首地址不是 32 的倍数，比如往 0X0810 0004 这里，写入 1 个字节数据，则会把地址：0X0810 0000 ~ 0X0810 0003 的数据也损坏掉（擦除）。所以，记住 STM32H7 FLASH 写入的规则：**写入首地址必须是 32 的倍数，写入数据长度必须是 32 字节的倍数。**

FLASH 配置步骤

STM32H743 的 FLASH 在编程的时候，也必须要求其写入地址的 FLASH 是被擦除了的（也就是其值必须是 0XFFFFFFFF），否则无法写入。STM32H743 的标准编程步骤如下：

1，检查 FLASH_CR 的 LOCK 是否解锁，如果没有则先解锁

2，检查 FLASH_SR 中的 BSY 位，确保当前未执行任何 FLASH 操作。

3，设置 FLASH_CR 寄存器的 PSIZE[1:0]为 2，按字写入（32 位写入）。

4，将 FLASH_CR 寄存器中的 PG 位置 1，激活 FLASH 编程。

5，在指定的存储器地址，写入数据（一次写入 32 字节，不能超过 32 字节）

6，等待 BSY 位清零，完成一次编程。

按以上六步操作，就可以完成一次 FLASH 编程。不过需要注意：编程前，要确保要写入地址的 FLASH 已经擦除。

在 STM32H743 的 FLASH 编程的时候，要先判断所写地址是否被擦除了，所以，我们有必要再介绍一下 STM32H743 的闪存擦除，STM32H743 的闪存擦除分为两种：扇区擦除和块擦除。

扇区擦除步骤如下：

1，检查 FLASH_CR 的 LOCK 是否解锁，如果没有则先解锁。

2，检查 FLASH_SR 寄存器中的 BSY 位，确保当前未执行任何 FLASH 操作。

3，在 FLASH_CR 寄存器中，将 SER 位置 1，并设置 SNB=0。

4，将 FLASH_CR 寄存器中的 START 位置 1，触发擦除操作。

5，等待 BSY 位清零。

经过以上五步，就可以擦除某个扇区。本章，我们只用到了 STM32H743 的扇区擦除功能。块擦除功能我们在这里就不介绍了，想了解的朋友请看《STM32H7xx 参考手册_V7（英文版）.pdf》的相关内容。

50.1.3 FLASH 寄存器

- **Flash 访问控制寄存器（FLASH_ACR）**

Flash 访问控制寄存器描述如图 50.1.3.1 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WRHIGH FREQ		Res.	LATENCY		
										rw	rw		rw	rw	rw

位 5:4 **WRHIGHFREQ**：Flash 信号延时 (Flash signal delay)

这些位用于控制编程操作期间各 Flash 信号之间的延时。用户需要根据 Flash 接口频率写入正确的值（请参见*表 7*）。不会执行任何检查来验证配置是否正确。

00：≤ 85 MHz
01：≤ 185 MHz
10：≤ 285 MHz
11：≤ 385 MHz

位 2:0 **LATENCY**：读延时 (Read latency)

这些位的值用于指定对两个 Flash 存储区执行读操作期间将会使用的等待状态数。用户需要根据*表 7* 所示的 Flash 接口频率和电源工作模式写入正确的值。不会执行任何检查来验证配置是否正确。

000：从 Flash 中读取字时需要使用 0 个等待状态
001：从 Flash 中读取字时需要使用 1 个等待状态
...
111：从 Flash 中读取字时需要使用 7 个等待状态

图 50.1.3.1 FLASH_ACR 寄存器

WRHIGHFREQ[1:0]位，用于控制 FLASH 编程操作时的延迟，必须根据 FLASH 操作频率（rcc_aclk）进行正确的设置：00，rcc_aclk≤85Mhz；01，rcc_aclk≤185Mhz；10，rcc_aclk≤285Mhz；11，rcc_aclk≤385Mhz；我们的 rcc_aclk 设置的是 200Mhz，设置 WRHIGHFREQ[1:0]=10 即可。

LATENCY[2:0]位，用于控制 FLASH 读延迟，必须根据我们 MCU 内核的工作电压和频率，来进行正确的设置，否则，可能死机，设置规则见表 50.1.1.2。

- **存储区 1 的 FLASH 密钥寄存器（FLASH_KEYR1）**

存储区 1 的 FLASH 密钥寄存器描述如图 50.1.3.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
KEYR1															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
KEYR1															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

位 31:0 **KEYR1**：存储区 1 访问配置解锁密钥 (Bank 1 access configuration unlock key)

FLASH_KEYR1 为只写寄存器。要将 FLASH_CR1 寄存器解锁并允许对其执行编程/擦除操作，必须顺序编程以下值：

a) 第 1 个密钥 = 0x4567 0123
b) 第 2 个密钥 = 0xCDEF 89AB

图 50.1.3.2 FLASH_KEYR1 寄存器

该寄存器主要用来解锁 FLASH_CR，必须在该寄存器写入特定的序列（KEY1 和 KEY2）解锁后，才能对 FLASH_CR 寄存器进行写操作。

- **存储区 1 的 FLASH 控制寄存器（FLASH_CR1）**

存储区 1 的 FLASH 控制寄存器描述如图 50.1.3.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	CRCENDIE1	DBECCERRIE1	SNECCERRIE1	RDSERRIE1	RDPERRIE1	OPERRIE1	INCERRIE1	Res.	STRBERRIE1	PGSERRIE1	WRPERRIE1	EOPIE1
				rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CRC_EN	Res.	Res.	Res.	Res.	SNB1			START1	FW1	PSIZE1		BER1	SER1	PG1	LOCK1
rw					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rs

图 50.1.3.3 FLASH_CR1 寄存器

LOCK 位，该位用于指示 FLASH_CR 寄存器是否被锁住，该位在检测到正确的解锁序列后，硬件将其清零。在一次不成功的解锁操作后，在下次系统复位之前，该位将不再改变。

PG 位，该位用于选择编程操作，在往 FLASH 写数据的时候，该位需要置 1。

SER 位，该位用于选择扇区擦除操作，在扇区擦除的时候，需要将该位置 1。

PSIZE[1:0]位，用于设置编程宽度，我们一般设置 PSIZE =2 即可（32 位）。

SATRT 位，该位用于开始一次擦除操作。在该位写入 1 ，将执行一次擦除操作。

SNB[2:0]位，这 3 个位用于选择要擦除的扇区编号，取值范围为 0~7。

FLASH_CR 的其他位，我们就不在这里介绍了，请大家参考《STM32H7xx 参考手册_V7（英文版）.pdf》 。

- **存储区 1 的 FLASH 状态寄存器（FLASH_SR1）**

存储区 1 的 FLASH 状态寄存器描述如图 50.1.3.4 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	CRCEND1	DBECCERR1	SNECCERR1	RDSERR1	RDPERR1	OPERR1	INCERR1	Res.	STRBERR1	PGSERR1	WRPERR1	EOP1
				r	r	r	r	r	r	r		r	r	r	r

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CRC_BUSY1	QW1	WBNE1	BSY1
												r	r	r	r

图 50.1.3.4 FLASH_SR1 寄存器

BSY 位：表示 BANK 当前正在执行编程操作，必须等待该位为 0，才可以执行其他操作。

WBNE 位：表示 BANK 写 BUFFER 是否为空。当该位为 1 时，表示写 BUFFER 里面还有数据待写入 FLASH，需要等待该位为 0，才表示数据写入全部完成了。

QW 位：表示操作序列里面是否还有编程操作需要执行，需要等待该位为 0，才表示所有的编程操作完成了。

最后，FLASH 清除控制寄存器 FLASH_CCR 用于清除相关错误，这里我们就不做介绍了，详见《STM32H7xx 参考手册_V7（英文版）.pdf》第 3.9.6 节。

50.2 硬件设计

1. 例程功能

按键 KEY1 控制写入 FLASH 的操作，按键 KEY0 控制读出操作，并在 TFTLCD 模块上显示相关信息，还可以借助 USMART 进行读取或者写入操作。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED：LED0 - PB1
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）独立按键：KEY0 - PH3、KEY1 - PH2

50.3 程序设计

50.3.1 FLASH 的 HAL 库驱动

FLASH 在 HAL 库中的驱动代码在 stm32h7xx_hal_flash.c 和 stm32h7xx_hal_flash_ex.c 文件（及其头文件）中。

1. HAL_FLASH_Unlock 函数

解锁闪存控制寄存器访问的函数，其声明如下：

```
HAL_StatusTypeDef HAL_FLASH_Unlock(void);
```

- **函数描述：**

用于解锁闪存控制寄存器的访问，在对 FLASH 进行写操作前必须先解锁，解锁操作也就是必须在 FLASH_KEYR 寄存器写入特定的序列（KEY1 和 KEY2）。

- **函数形参：**无
- **函数返回值：**HAL_StatusTypeDef 枚举类型的值。

2. HAL_FLASH_Lock 函数

锁定闪存控制寄存器访问的函数，其声明如下：

```
HAL_StatusTypeDef HAL_FLASH_Lock (void);
```

- **函数描述：**

用于锁定闪存控制寄存器的访问。

- **函数形参：**无
- **函数返回值：**HAL_StatusTypeDef 枚举类型的值。

3. HAL_FLASHEx_Erase 函数

闪存擦除函数，其声明如下：

```
HAL_StatusTypeDef HAL_FLASHEx_Erase(FLASH_EraseInitTypeDef *pEraseInit,
                                    uint32_t *SectorError);
```

- **函数描述：**

该函数用于大量擦除或擦除指定的闪存扇区。

- **函数形参：**

形参 1 是 FLASH_EraseInitTypeDef 结构体类型指针变量。
形参 2 是 uint32_t 类型指针变量，存放错误码，0xFFFFFFFF 值表示扇区已被正确擦除，其它值表示擦除过程中的错误扇区。。

- **函数返回值：**HAL_StatusTypeDef 枚举类型的值。

4. FLASH_WaitForLastOperation 函数

等待 FLASH 操作完成函数，其声明如下：

```
HAL_StatusTypeDef FLASH_WaitForLastOperation(uint32_t Timeout, uint32_t Bank);
```

- **函数描述：**

该函数用于等待 FLASH 操作完成。

- **函数形参：**
  形参 1 是 FLASH 操作超时时间。
  形参 2 是等待 BANK1 还是 BANK2 操作完成。
- **函数返回值：**
  HAL_StatusTypeDef 枚举类型的值。

50.3.2 程序流程图

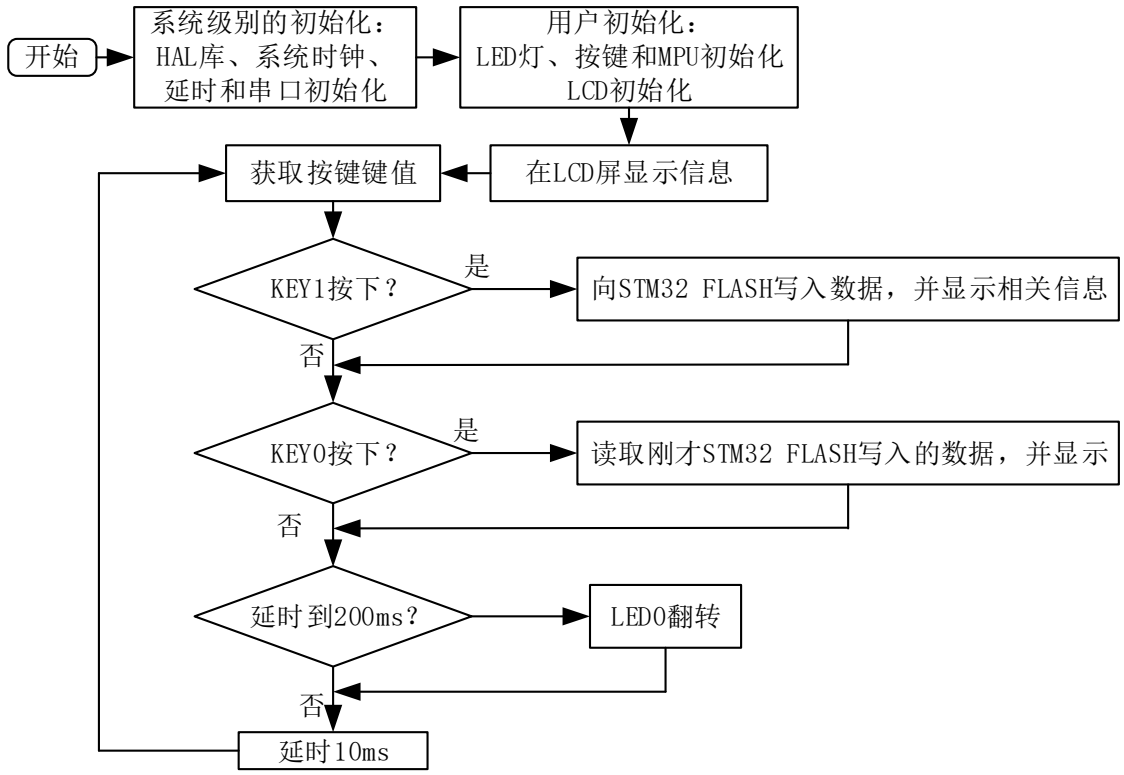


图 50.3.2.1 FLASH 模拟 EEPROM 实验程序流程图

50.3.3 程序解析

1. STMFLASH 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。STMFLASH 驱动源码包括两个文件：stmflash.c 和 stmflash.h。

stmflash.h 头文件做了一些比较重要的宏定义，定义如下：

```
/* FLASH 起始地址 */
#define STM32_FLASH_SIZE        0x0200000           /* STM32 FLASH 总大小 */
#define STM32_FLASH_BASE        0x08000000          /* STM32 FLASH 起始地址 */

#define FLASH_WAITETIME         50000               /* FLASH 等待超时时间 */

/* STM32 H7 FLASH 扇区的起始地址 */
/* BANK1 */
#define ADDR_FLASH_SECTOR_0_BANK1     ((uint32_t )0x08000000)
/* 扇区 0 起始地址, 128 Kbytes */
#define ADDR_FLASH_SECTOR_1_BANK1     ((uint32_t )0x08020000)
/* 扇区 1 起始地址, 128 Kbytes */
#define ADDR_FLASH_SECTOR_2_BANK1     ((uint32_t )0x08040000)
/* 扇区 2 起始地址, 128 Kbytes */
```

```
#define ADDR_FLASH_SECTOR_3_BANK1     ((uint32_t )0x08060000)
/* 扇区 3 起始地址, 128 Kbytes */
#define ADDR_FLASH_SECTOR_4_BANK1     ((uint32_t )0x08080000)
/* 扇区 4 起始地址, 128 Kbytes */
#define ADDR_FLASH_SECTOR_5_BANK1     ((uint32_t )0x080A0000)
/* 扇区 5 起始地址, 128 Kbytes */
#define ADDR_FLASH_SECTOR_6_BANK1     ((uint32_t )0x080C0000)
/* 扇区 6 起始地址, 128 Kbytes */
#define ADDR_FLASH_SECTOR_7_BANK1     ((uint32_t )0x080E0000)
/* 扇区 7 起始地址, 128 Kbytes */

/* BNAK2 */
#define ADDR_FLASH_SECTOR_0_BANK2     ((uint32_t )0x08100000)
/* 扇区 0 起始地址, 128 Kbytes */
#define ADDR_FLASH_SECTOR_1_BANK2     ((uint32_t )0x08120000)
/* 扇区 1 起始地址, 128 Kbytes */
#define ADDR_FLASH_SECTOR_2_BANK2     ((uint32_t )0x08140000)
/* 扇区 2 起始地址, 128 Kbytes */
#define ADDR_FLASH_SECTOR_3_BANK2     ((uint32_t )0x08160000)
/* 扇区 3 起始地址, 128 Kbytes */
#define ADDR_FLASH_SECTOR_4_BANK2     ((uint32_t )0x08180000)
/* 扇区 4 起始地址, 128 Kbytes */
#define ADDR_FLASH_SECTOR_5_BANK2     ((uint32_t )0x081A0000)
/* 扇区 5 起始地址, 128 Kbytes */
#define ADDR_FLASH_SECTOR_6_BANK2     ((uint32_t )0x081C0000)
/* 扇区 6 起始地址, 128 Kbytes */
#define ADDR_FLASH_SECTOR_7_BANK2     ((uint32_t )0x081E0000)
/* 扇区 7 起始地址, 128 Kbytes */
```

比较重要的一个宏定义是 BOOT_FLASH_SIZE，因为这个宏定义需要用户根据自己工程的大小设置的。设置方法如下：首先编译我们的工程，通过查看.map 文件，查询并计算出程序加载到 STM32 内部 FLASH 的指令内存的大小，最后根据这个大小来定义这个宏的值。这里，我们直接编译 HAL 库实验 38 FLASH 模拟 EEPROM 实验的例程，然后如下图操作：

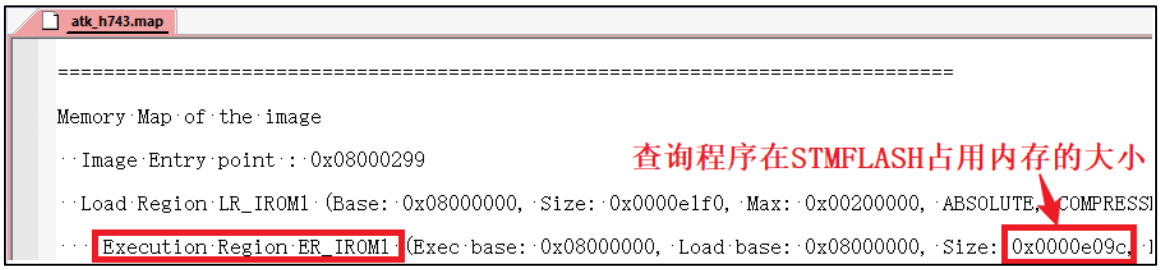


图 50.3.3 查看程序加载占用 FLASH 的大小

首先双击 FLASH 打开.map 文件，然后查询到 Execution Region ER_m_stmflash。如图中所示，得到加载到 STMFLASH 的程序占用的内存大小约为 57.5KB。

FLASH_WAITETIME 是 FLASH 等待超时时间的宏定义。

ADDR_FLASH_SECTOR_0_BANK1 是 Bank1 扇区 0 起始地址。

下面我们开始介绍 stmflash.c 的程序，具体程序源码如下：

```
/**
 * @brief       从指定地址开始写入指定长度的数据
 * @note        特别注意：因为 STM32H7 的扇区实在太大,没办法本地保存扇区数据,所以本函数
 *              写地址如果非 0XFF,那么会先擦除整个扇区且不保存扇区数据.所以
 *              写非 0XFF 的地址,将导致整个扇区数据丢失.建议写之前确保扇区里
 *              没有重要数据,最好是整个扇区先擦除了,然后慢慢往后写
 *              该函数对 OTP 区域也有效!可以用来写 OTP 区!
 *              OTP 区域地址范围:0X1FFF7800~0X1FFF7A0F(注意：最后 16 字节，用于 OTP 数据
                块锁定，别乱写！！)
 * @param       waddr  : 起始地址(此地址必须为 4 的倍数!!)
 * @param       pbuf   : 数据指针
 * @param       length : 字(32 位)数(就是要写入的 32 位数据的个数)
```

```
 * @retval      无
 */
void stmflash_write(uint32_t waddr, uint32_t *pbuf, uint32_t length)
{
    FLASH_EraseInitTypeDef FlashEraseInit;
    HAL_StatusTypeDef FlashStatus = HAL_OK;
    uint32_t SectorError = 0;
    uint32_t  addrx = 0;
    uint32_t  endaddr = 0;
    if (waddr < STM32_FLASH_BASE || waddr % 32 ||
        /* 写入地址小于 STM32_FLASH_BASE, 或不是 32 的整数倍, 非法. */
        waddr > (STM32_FLASH_BASE + STM32_FLASH_SIZE))
        /* 写入地址大于 STM32_FLASH_BASE + STM32_FLASH_SIZE, 非法. */
    {
        return;
    }
    HAL_FLASH_Unlock();             /* 解锁 */
    addrx = waddr;                  /* 写入的起始地址 */
    endaddr = waddr + length * 4; /* 写入的结束地址 */

    if (addrx < 0X1FF00000)
    {
        while (addrx < endaddr)    /* 扫清一切障碍.(对非 FFFFFFFF 的地方,先擦除) */
        {
           if(stmflash_read_word(addrx)!=0XFFFFFFFF)
           /* 有非 0XFFFFFFFF 的地方,要擦除这个扇区 */
            {
                FlashEraseInit.Banks = FLASH_BANK_1;
                FlashEraseInit.TypeErase = FLASH_TYPEERASE_SECTORS;
                /* 擦除类型，扇区擦除 */
                FlashEraseInit.Sector = stmflash_get_flash_sector(addrx);
                /* 要擦除的扇区 */
                FlashEraseInit.NbSectors = 1;
                /* 一次只擦除一个扇区 */
                FlashEraseInit.VoltageRange = FLASH_VOLTAGE_RANGE_3;
                /* 电压范围，VCC=2.7~3.6V 之间!! */
                if (HAL_FLASHEx_Erase(&FlashEraseInit, &SectorError) != HAL_OK)
                {
                    break;     /* 发生错误了 */
                }
            SCB_CleanInvalidateDCache();     /* 清除无效的 D-Cache */
            }
            else
            {
               addrx += 4;
            }
            FLASH_WaitForLastOperation(FLASH_WAITETIME, FLASH_BANK_1);
            /* 等待上次操作完成 */
        }
    }

    FlashStatus = FLASH_WaitForLastOperation(FLASH_WAITETIME, FLASH_BANK_1);
    /* 等待上次操作完成 */
    if (FlashStatus == HAL_OK)
    {
        while (waddr < endaddr)       /* 写数据 */
        {
            if (HAL_FLASH_Program(FLASH_TYPEPROGRAM_FLASHWORD, waddr,
                (uint64_t)pbuf) != HAL_OK)     /* 写入数据 */
            {
                break;  /* 写入异常 */
            }
            waddr += 32;
            pbuf+=8;
```

```
        }
    }
    HAL_FLASH_Lock();           /* 上锁 */
}

/**
 * @brief       从指定地址开始读出指定长度的数据
 * @param       raddr : 起始地址
 * @param       pbuf  : 数据指针
 * @param       length: 要读取的字(32)数,即 4 个字节的整数倍
 * @retval      无
 */
void stmflash_read(uint32_t raddr, uint32_t *pbuf, uint32_t length)
{
    uint32_t i;
    for (i = 0; i < length; i++)
    {
        pbuf [i] = stmflash_read_word(raddr);    /* 读取 4 个字节. */
        raddr+= 4;                               /* 偏移 4 个字节. */
    }
}
```

该部分代码，我们重点介绍一下 stmflash_write 函数，该函数用于在 STM32H743 的指定地址写入指定长度的数据，有几个要注意的点：

1， 写入地址必须是用户代码区以外的地址。

2， 写入地址必须是 4 的倍数。

3， 对 OTP 区域编程也有效

第 1 点比较好理解，如果把用户代码给擦除了，可想而知你运行的程序可能就被废了，从而很可能出现死机的情况。不过，因为 STM32H743 的扇区都比较大，为 128KB，所以本函数不缓存要擦除的扇区内容，也就是如果要擦除，那么就是整个扇区擦除，所以建议大家使用该函数的时候，写入地址定位到用户代码占用扇区以外的扇区，比较保险。

第 2 点则是 3.3V 时，设置 PSIZE=2 所决定的，每次必须写入 32 位，即 4 字节，所以地址必须的 4 的倍数。第 3 点，该函数对 OTP 区域的操作同样有效，所以大家要写 OTP 字节，也可以直接通过该函数写入，不过注意 OTP 是一次写入的，无法擦除，所以，一般不要写 OTP 字节。

另外，在该函数中我们调用了 SCB_CleanInvalidateDCache 函数，用于回写数据到 SRAM，并重新获取 D Cache 数据，如果去掉，将导致死循环。

```
/**
 * @brief       获取某个地址所在的 flash 扇区
 * @param       faddr    : flash 地址
 * @retval      0~7 ,addr 所在的 bank1 扇区编号
 *              8~15,addr 所在的 bank2 扇区编号+8,需要减去 8,才得 bank2 扇区编号
 */
uint8_t stmflash_get_flash_sector(uint32_t  addr)
{
    if (addr < BANK1_FLASH_SECTOR_1) return 0;
    else if (addr < BANK1_FLASH_SECTOR_2) return 1;
    else if (addr < BANK1_FLASH_SECTOR_3) return 2;
    else if (addr < BANK1_FLASH_SECTOR_4) return 3;
    else if (addr < BANK1_FLASH_SECTOR_5) return 4;
    else if (addr < BANK1_FLASH_SECTOR_6) return 5;
    else if (addr < BANK1_FLASH_SECTOR_7) return 6;
    else if (addr < BANK2_FLASH_SECTOR_0) return 7;
    else if (addr < BANK2_FLASH_SECTOR_1) return 8;
    else if (addr < BANK2_FLASH_SECTOR_2) return 9;
    else if (addr < BANK2_FLASH_SECTOR_3) return 10;
    else if (addr < BANK2_FLASH_SECTOR_4) return 11;
    else if (addr < BANK2_FLASH_SECTOR_5) return 12;
    else if (addr < BANK2_FLASH_SECTOR_6) return 13;
    else if (addr < BANK2_FLASH_SECTOR_7) return 14;
    return FLASH_SECTOR_7;
```

```
}
```

该函数比较好理解，根据地址确定其 sector 编号。在头文件 stmflash.h 中，我们定义了从 BANK1_FLASH_SECTOR_0~ BANK2_FLASH_SECTOR_7 等一系列宏定义标识符，实际上这些标识符的值就是对应的 sector 的起始地址值，相信也比较好理解。其他函数我们就不做介绍了。

2. main.c 代码

在 main.c 里面编写如下代码：

```
/* 要写入到 STM32 FLASH 的字符串数组 */
const uint8_t g_text_buf[] = {"STM32 FLASH TEST"};
#define TEXT_LENTH sizeof(g_text_buf)           /* 数组长度 */

/*SIZE 表示字长(4 字节), 大小必须是 4 的整数倍, 如果不是的话, 强制对齐到 4 的整数倍 */
#define SIZE  TEXT_LENTH / 4 + ((TEXT_LENTH % 4) ? 1 : 0)

/* 设置 FLASH 保存地址(必须大于用户代码区地址范围,且为 4 的倍数) */
#define FLASH_SAVE_ADDR     0X08020000

int main(void)
{
    uint8_t key = 0;
    uint16_t i = 0;
    uint8_t datatemp[SIZE];

    sys_cache_enable();                         /* 打开 L1-Cache */
    HAL_Init();                                 /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);         /* 设置时钟, 400Mhz */
    delay_init(400);                            /* 延时初始化 */
    usart_init(115200);                         /* 串口初始化为 115200 */
    usmart_dev.init(200);                       /* 初始化 USMART */
    mpu_memory_protection();                    /* 保护相关存储区域 */
    led_init();                                 /* 初始化 LED */
    key_init();                                 /* 初始化按键 */
    sdram_init();                               /* 初始化 SDRAM */
    lcd_init();                                 /* 初始化 LCD */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "FLASH EEPROM TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY1:Write  KEY0:Read", RED);

    while (1)
    {
        key = key_scan(0);
        if (key == KEY1_PRES)    /* KEY1 按下,写入 STM32 FLASH */
        {
            lcd_fill(0, 150, 239, 319, WHITE);  /* 清除半屏 */
            lcd_show_string(30, 150, 200, 16, 16, "Start Write FLASH....", RED);
            stmflash_write(FLASH_SAVE_ADDR, (uint32_t *)g_text_buf, SIZE);
            /* 提示传送完成 */
            lcd_show_string(30, 150, 200, 16, 16, "FLASH Write Finished!", RED);
        }

        if (key == KEY0_PRES)    /* KEY0 按下,读取字符串并显示 */
        {
            lcd_show_string(30, 150, 200, 16, 16, "Start Read FLASH.... ", RED);
            stmflash_read(FLASH_SAVE_ADDR, (uint32_t *)datatemp, SIZE);
            /* 提示传送完成 */
            lcd_show_string(30, 150, 200, 16, 16, "The Data Readed Is:  ", RED);
            /* 显示读到的字符串 */
```

```
            lcd_show_string(30, 170, 200, 16, 16, (char*)datatemp, BLUE);
        }

        i++;
        delay_ms(10);
        if (i == 20)
        {
            LED0_TOGGLE();  /* 提示系统正在运行 */
            i = 0;
        }
    }
}
```

主函数代码逻辑比较简单，当检测到按键 KEY1 按下后往 FLASH 指定地址开始的连续地址空间写入一段数据，当检测到按键 KEY0 按下后读取 FLASH 指定地址开始的连续空间数据。最后，我们将 stmflash_read_word 和 test_write 函数加入 USMART 控制，这样，我们就可以通过串口调试助手，调用 STM32H743 的 FLASH 读写函数，方便测试。

50.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 50.4.1 所示：

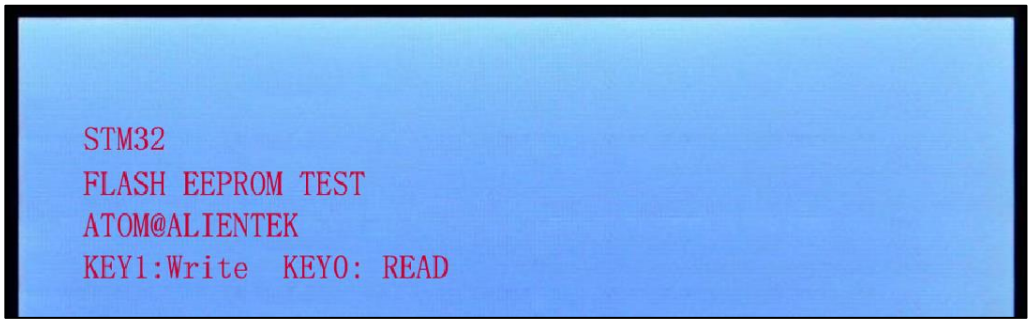


图 50.4.1 程序运行效果图

通过先按 KEY1 按键写入数据，然后按 KEY0 读取数据，得到如图 50.4.2 所示：

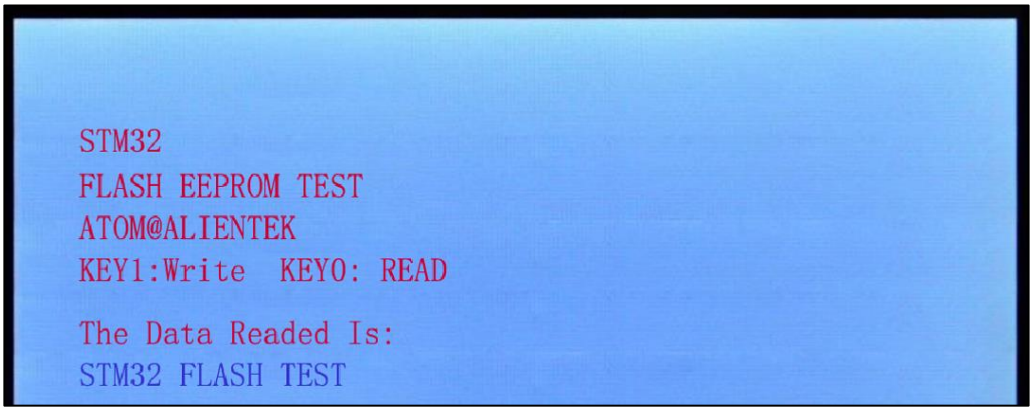


图 50.4.2 操作后的显示效果图

本实验的测试，我们还可以借助 USMART，调用：stmflash_read_word 和 test_write 函数进行测试！

第五十一章 摄像头实验

STM32H743 具有 DCMI 接口，所以我们的阿波罗 STM32H743 开发板板载了一个摄像头接口（P6），该接口可以用来连接正点原子 OV5640/OV2640/OV7725 等摄像头模块。本章，我们将使用 STM32 驱动正点原子 OV5640 摄像头模块，实现摄像头功能。

本章分为如下几个小节：

51.1 OV5640 和 DCMI 简介
51.2 硬件设计
51.3 程序设计
51.4 下载验证

51.1 OV5640 和 DCMI 简介

本节将分为两个部分，分别介绍 OV5640 简介和 STM32H743 DCMI 接口简介。另外，所有 OV5640 的相关资料，都在光盘：A 盘→7，硬件资料→OV5640 资料 文件夹里面。

51.1.1 OV5640 简介

OV5640 是 OV（OmniVision）公司生产的一颗 1/4 寸的 CMOS QSXGA（2592*1944）图像传感器，提供了一个完整的 500W 像素摄像头解决方案，并且集成了自动对焦（AF）功能，具有非常高的性价比。

该传感器体积小、工作电压低，提供单片 QSXGA 摄像头和影像处理器的所有功能。通过 SCCB 总线控制，可以输出整帧、子采样、缩放和取窗口等方式的各种分辨率 8/10 位影像数据。该产品 QSXGA 图像最高达到 15 帧/秒（1080P 图像可达 30 帧，720P 图像可达 60 帧，QVGA 分辨率时可达 120 帧）。用户可以完全控制图像质量、数据格式和传输方式。所有图像处理功能过程包括伽玛曲线、白平衡、对比度、色度等都可以通过 SCCB 接口编程。OmmiVision 图像传感器应用独有的传感器技术，通过减少或消除光学或电子缺陷如固定图案噪声、拖尾、浮散等，提高图像质量，得到清晰稳定的彩色图像。

OV5640 的特点有：

- 采用 1.4μm*1.4μm 像素大小，并且使用 OmniBSI 技术以达到更高性能（高灵敏度、低串扰和低噪声）
- 自动图像控制功能：自动曝光（AEC）、自动白平衡（AWB）、自动消除灯光条纹、自动黑电平校准（ABLC）和自动带通滤波器（ABF）等。
- 支持图像质量控制：色饱和度调节、色调调节、gamma 校准、锐度和镜头校准等
- 标准的 SCCB 接口，兼容 IIC 接口
- 支持 RawRGB、RGB(RGB565/RGB555/RGB444)、CCIR656、YUV(422/420)、YCbCr（422）和压缩图像（JPEG）输出格式
- 支持 QSXGA（500W）图像尺寸输出，以及按比例缩小到其他任何尺寸
- 支持闪光灯
- 支持图像缩放、平移和窗口设置
- 支持图像压缩，即可输出 JPEG 图像数据
- 支持数字视频接口（DVP）和 MIPI 接口
- 支持自动对焦
- 自带嵌入式微处理器

OV5640 的功能框图图如图 51.1.1.1 所示：

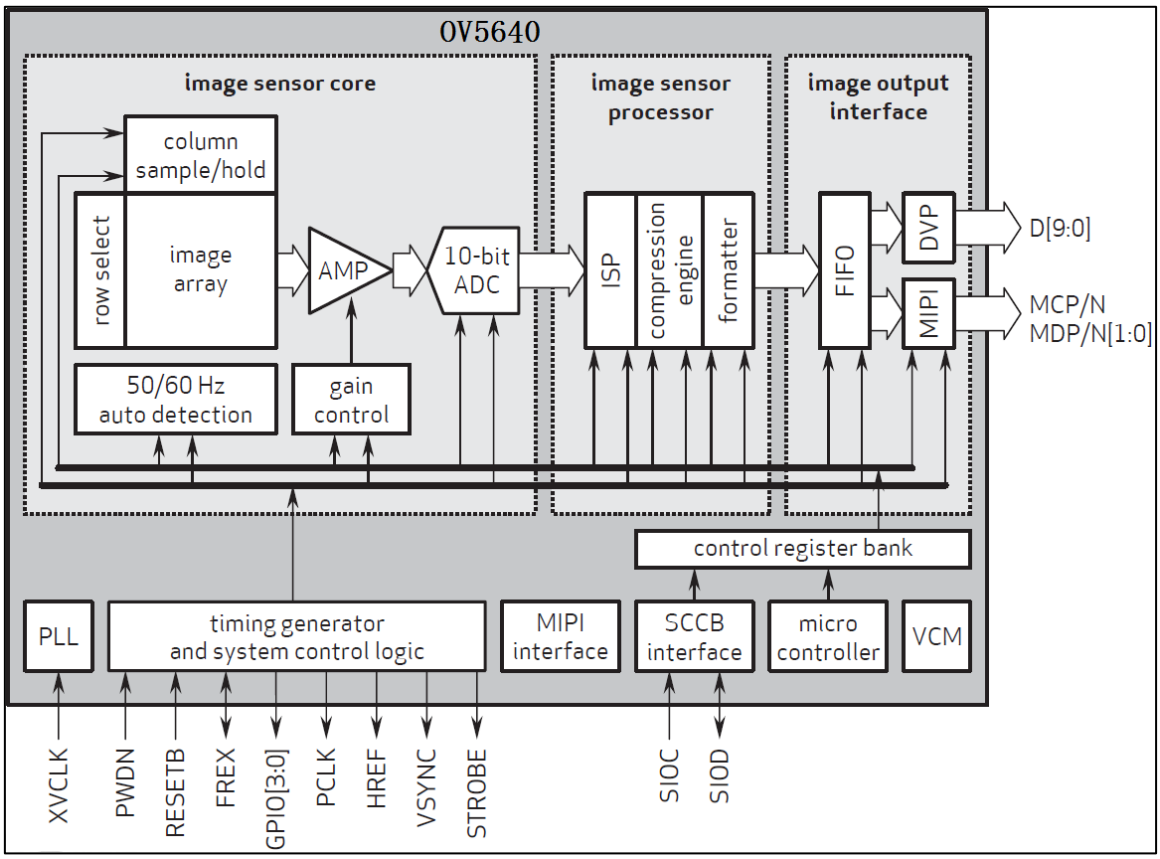


图 51.1.1.1 OV5640 功能框图

其中 image array 部分的尺寸，OV5640 的官方数据并没有给出具体的数字，其最大的有效输出尺寸为：2592*1944，即 500W 像素，我们根据官方提供的一些应用文档，发现其设置的 image array 最大为：2632*1951，所以，在接下来的介绍，我们设定其 image array 最大为 2632*1951。

1、DVP 接口说明

OV5640 支持数字视频接口（DVP）和 MIPI 接口，因为我们的 STM32H743 使用的 DCMI 接口，仅支持 DVP 接口，所以，OV5640 必须使用 DVP 输出接口，才可以连接我们的阿波罗 STM32 开发板。

OV5640 提供一个 10 位 DVP 接口(支持 8 位接法)，其 MSB 和 LSB 可以程序设置先后顺序，正点原子 OV5640 模块采用默认的 8 位连接方式，如图 51.1.1.2 所示：

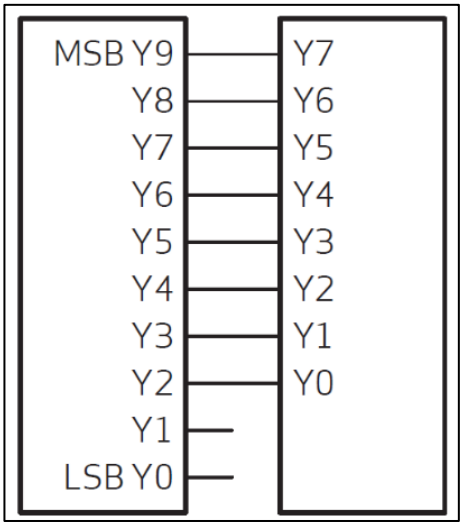


图 51.1.1.2 OV5640 默认 8 位连接方式

OV5640 的寄存器通过 SCCB 时序访问并设置，SCCB 时序和 IIC 时序十分类似。SCCB 与标准的 I2C 协议的区别是它每次传输只能写入或读取一个字节的数据，而 I2C 协议是支持突发读写的，即在一次传输中可以写入多个字节的数据(EEPROM 中的页写入时序即突发写)。在本章我们不做介绍，请大家参考光盘《OmniVision Technologies Seril Camera Control Bus(SCCB) Specification》这个文档。

2、窗口设置说明

接下来，我们介绍一下 OV5640 的：ISP（Image Signal Processor）输入窗口设置、预缩放窗口设置和输出大小窗口设置，这几个设置与我们的正常使用密切相关，有必要了解一下。他们的设置关系，如图 51.1.1.3 所示：

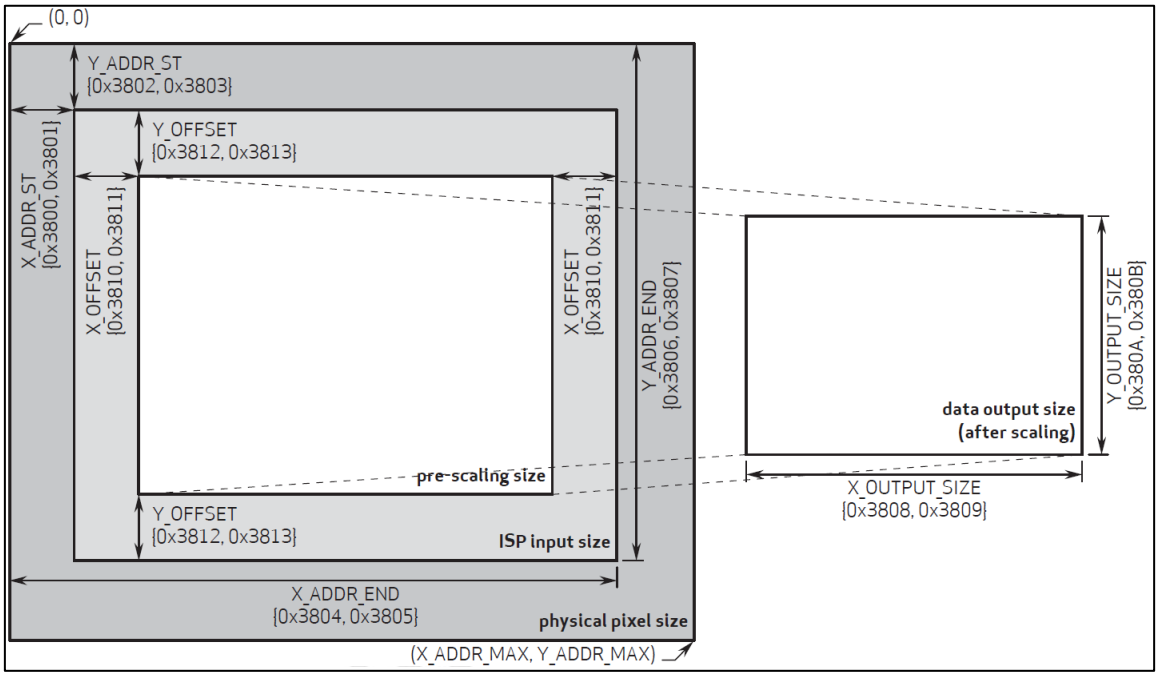


图 51.1.1.3 OV5640 各窗口设置关系

ISP 输入窗口设置（ISP input size）

该设置允许用户设置整个传感器区域（physical pixel size，2632*1951）的感兴趣部分，也就是在传感器里面开窗（X_ADDR_ST、Y_ADDR_ST、X_ADDR_END 和 Y_ADDR_END），开窗范围从 0*0~2632*1951 都可以设置，该窗口所设置的范围，将输入 ISP 进行处理。

ISP 输入窗口，通过：0X3800~0X3807 等 8 个寄存器进行设置，这些寄存器的定义请看：OV5640_CSP3_DS_2.01_Ruisipusheng.pdf 这个文档（下同）。

预缩放窗口设置（pre-scaling size）

该设置允许用户在 ISP 输入窗口的基础上，再次设置将要用于缩放的窗口大小。该设置仅在 ISP 输入窗口内进行 x/y 方向的偏移（X_OFFSET/Y_OFFSET）。通过：0X3810~0X3813 等 4 个寄存器进行设置。

输出大小窗口设置（data output size）

该窗口是以预缩放窗口为原始大小，经过内部 DSP 进行缩放处理后，输出给外部的图像窗口大小。它控制最终的图像输出尺寸（X_OUTPUT_SIZE/Y_OUTPUT_SIZE）。通过：0X3808~0X380B 等 4 个寄存器进行设置。注意：当输出大小窗口与预缩放窗口比例不一致时，图像将进行缩放处理（会变形），仅当两者比例一致时，输出比例才是 1:1（正常）。

图 51.1.1.3 中，右侧 data output size 区域，才是 OV5640 输出给外部的图像尺寸，也就是显示在 LCD 上面的图像大小。输出大小窗口与预缩放窗口比例不一致时，会进行缩放处理，在 LCD 上面看到的图像将会变形。

3、输出时序说明

接下来，我们介绍一下 OV5640 的图像数据输出时序。首先我们简单介绍一些定义：

QSXGA，这里指：分辨率为 2592*1944 的输出格式，类似的还有：QXGA(2048*1536)、

UXGA(1600*1200)、SXGA(1280*1024)、WXGA+(1440*900)、WXGA(1280*800)、XGA(1024*768)、SVGA(800*600)、VGA(640*480)、QVGA(320*240)和QQVGA(160*120)等。

PCLK，即像素时钟，一个 PCLK 时钟，输出一个像素(或半个像素)。

VSYNC，即帧同步信号。

HREF /HSYNC，即行同步信号。

OV5640 的图像数据输出（通过 Y[9:0]）就是在 PCLK，VSYNC 和 HREF/ HSYNC 的控制下进行的。首先看看行输出时序，如图 51.1.1.4 所示：

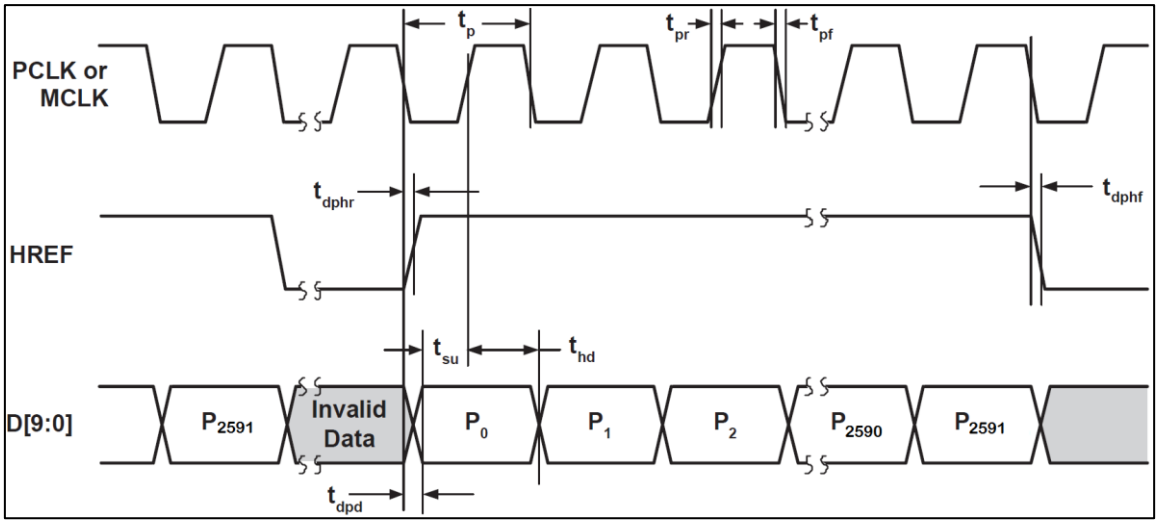


图 51.1.1.4 OV5640 行输出时序

从上图可以看出，图像数据在 HREF 为高的时候输出，当 HREF 变高后，每一个 PCLK 时钟，输出一个 8 位/10 位数据。我们采用 8 位接口，所以每个 PCLK 输出 1 个字节，且在 RGB/YUV 输出格式下，每个 tp=2 个 Tpclk，如果是 Raw 格式，则一个 tp=1 个 Tpclk。比如我们采用 QSXGA 时序，RGB565 格式输出，每 2 个字节组成一个像素的颜色（低字节在前，高字节在后），这样每行输出总共有 2592*2 个 PCLK 周期，输出 2592*2 个字节。

再来看看帧时序（QSXGA 模式），如图 51.1.1.5 所示：

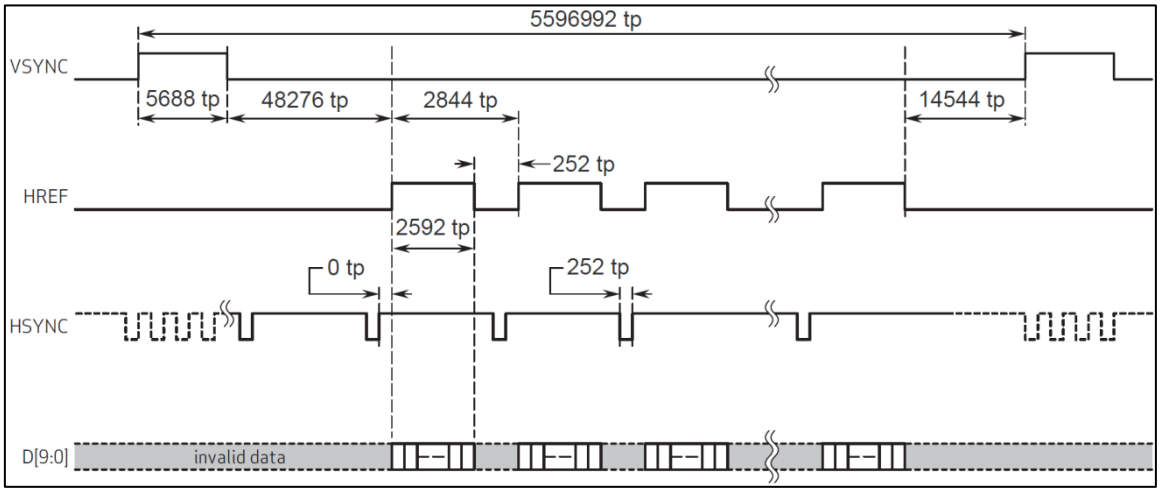


图 51.1.1.5 OV5640 帧时序

上图清楚的表示了 OV5640 在 QSXGA 模式下的数据输出。我们按照这个时序去读取 OV5640 的数据，就可以得到图像数据。

4、自动对焦（Auto Focus）说明

OV5640 由内置微型控制器完成自动对焦，并且 VCM（Voice Coil Motor，即音圈马达）驱动器也已集成在传感器内部。微型控制器的控制固件（firmware）从主机下载。当固件运行后，内置微型控制器从 OV5640 传感器读得自动对焦所需的信息，计算并驱动 VCM 马达带动镜头到达正确的对焦位置。主机可以通过 IIC 命令控制微型控制器的各种功能。

OV5640 的自动对焦命令（通过 SCCB 总线发送），如表 51.1.1.1 所示：

地址	寄存器名	描述	值
0X3022	CMD_MAIN	AF 主命令寄存器	0X03：触发单次自动对焦过程 0X04：启动持续自动对焦过程 0X06：暂停自动对焦过程 0X08：释放马达回到初始状态 0X12：设置对焦区域 0X00：命令完成
0X3023	CMD_ACK	命令确认	0X00：命令完成 0X01：命令执行中
0X3029	FW_STATUS	对焦状态	0X7F：固件下载完成，但未执行，可能是：固件有问题/微控制器关闭 0X7E：固件初始化中 0X70：释放马达，回到初始状态 0X00：正在自动对焦 0X10：自动对焦完成

表 51.1.1.1 OV5640 自动对焦命令

OV5640 内部的微控制器收到自动对焦命令后会自动将 CMD_MAIN（0X3022）寄存器数据清零，当命令完成后会将 CMD_ACK（0X3023）寄存器数据清零。

自动对焦（AF）过程

① 在第一次进入图像预览的时候（图像可以正常输出时），下载固件（firmware）。

② 拍照前，自动对焦，对焦完成后，拍照。

③ 拍照完毕，释放马达到初始状态。

接下来，我们分别说明。

① 下载固件

OV5640 初始化完成后，就可以下载 AF 自动对焦固件了，其操作和下载初始化参数类似，AF 固件下载地址为：0X8000，初始化数组由厂家提供（本例程该数组保存在 ov5640af.h 里面），下载固件完成后，通过检查 0X3029 寄存器的值，来判断固件状态（等于 0X70，说明正常）。

② 自动对焦

OV5640 支持单次自动对焦和持续自动对焦，通过 0X3022 寄存器控制。单次自动对焦过程如下：

1，将 0X3022 寄存器写为 0X03，开始单点对焦过程。

2，读取寄存器 0X3029，如果返回值为 0X10，代表对焦已完成。

3，写寄存器 0X3022 为 0X06，暂停对焦过程，使镜头将保持在此对焦位置。

其中，前两步是必须的，第三步，可以不要，因为单次自动对焦完成以后，就不会继续自动对焦了，镜头也就不会动了。

持续自动对焦过程如下：

1， 将 0X22 寄存器写为 0X08，释放马达到初始位置（对焦无穷远）。

2， 将 0X3022 寄存器写为 0X04，启动持续自动对焦过程。

3， 读取寄存器 0X3023，等待命令完成。

4， 当 OV5640 每次检测到失焦时，就会自动进行对焦（一直检测）。

③ 释放马达，结束自动对焦

最后，在拍照完成，或者需要结束自动对焦的时候，我们对在寄存器 0X3022 写入 0X08，即可释放马达，结束自动对焦。

最后说一下 OV5640 的图像数据格式，我们一般用 2 种输出方式：RGB565 和 JPEG。当输出 RGB565 格式数据的时候，时序完全就是上面两幅图介绍的关系。以满足不同需要。而当输出数据是 JPEG 数据的时候，同样也是这种方式输出（所以数据读取方法一模一样），不过 PCLK 数目大大减少了，且不连续，输出的数据是压缩后的 JPEG 数据，输出的 JPEG 数据以：0XFF,0XD8 开头，以 0XFF,0XD9 结尾，且在 0XFF,0XD8 之前，或者 0XFF,0XD9 之后，会有

不定数量的其他数据存在（一般是 0），这些数据我们直接忽略即可，将得到的 0XFF,0XD8~0XFF,0XD9 之间的数据，保存为.jpg/.jpeg 文件，就可以直接在电脑上打开看到图像了。

OV5640 自带的 JPEG 输出功能，大大减少了图像的数据量，使得其在网络摄像头、无线视频传输等方面具有很大的优势。OV5640 我们就介绍到这，关于 OV5640 更详细的介绍，请大家参考：A 盘→7，硬件资料→OV5640 资料→ OV5640_CSP3_DS_2.01_Ruisipusheng.pdf。

正点原子 OV5640 摄像头模块

本实验，我们将使用阿波罗 STM32H743 开发板的 DCMI 接口连接正点原子 OV5640 摄像头模块，该模块采用 8 位数据输出接口，自带 24M 有源晶振，无需外部提供时钟，模组支持自动对焦功能，且支持闪光灯，整个模块只需提供 3.3V 供电即可正常使用。

正点原子 OV5640 摄像头模块外观如图 51.1.1.6 所示：

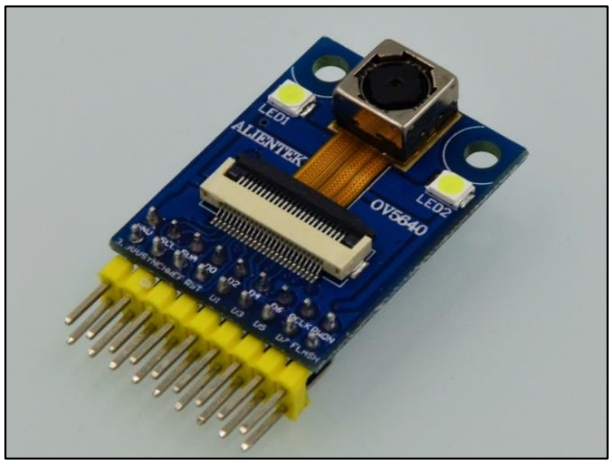


图 51.1.1.6 正点原子 OV5640 摄像头模块外观图

模块原理图如图 51.1.1.7 所示：

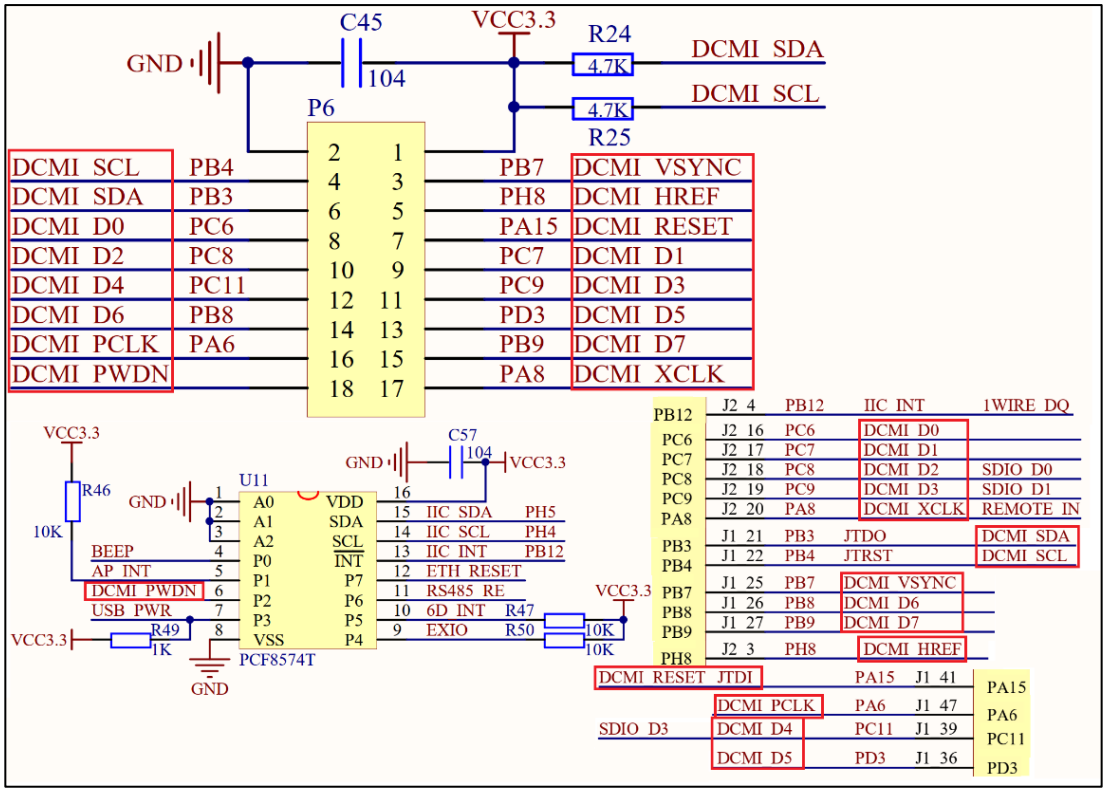


图 51.1.1.7 正点原子 OV5640 摄像头模块原理图

从上图可以看出，正点原子 OV5640 摄像头模块自带了有源晶振，用于产生 24M 时钟作为

OV5640 的 XCLK 输入，模块的闪光灯（LED1&LED2）由 OV5640 的 STROBE 脚控制（可编程控制）。同时自带了稳压芯片，用于提供 OV5640 稳定的 2.8V 和 1.5V 工作电压，模块通过一个 2*9 的双排排针（P1）与外部通信，与外部的通信信号如表 51.1.1.2 所示：

信号	作用描述	信号	作用描述
VCC3.3	模块供电脚，接 3.3V 电源	OV_PCLK	像素时钟输出
GND	模块地线	OV_PWDN	掉电使能(高有效)
OV_SCL	SCCB 通信时钟信号	OV_VSYNC	帧同步信号输出
OV_SDA	SCCB 通信数据信号	OV_HREF	行同步信号输出
OV_D[7:0]	8 位数据输出	OV_RESET	复位信号(低有效)

表 51.1.1.2 OV5640 模块信号及其作用描述

51.1.2 STM32H743 DCMI 接口简介

STM32H743 自带了一个数字摄像头（DCMI）接口，该接口是一个同步并行接口，能够接收外部 8 位、10 位、12 位或 14 位 CMOS 摄像头模块发出的高速数据流。可支持不同的数据格式：YCbCr4:2:2/RGB565 逐行视频和压缩数据 (JPEG)。

STM32H743 DCM 接口特点：

- 8 位、10 位、12 位或 14 位并行接口
- 内嵌码/外部行同步和帧同步
- 连续模式或快照模式
- 裁剪功能
- 支持以下数据格式：
  1，8/10/12/14 位逐行视频：单色或原始拜尔（Bayer）格式
  2，YCbCr 4:2:2 逐行视频
  3，RGB 565 逐行视频
  4，压缩数据：JPEG

DCMI 接口包括如下一些信号：

1， 数据输入（D[0:13]），用于接摄像头的数据输出，接 OV5640 我们只用了 8 位数据。
2， 水平同步（行同步）输入（HSYNC），用于接摄像头的 HSYNC/HREF 信号。
3， 垂直同步（场同步）输入（VSYNC），用于接摄像头的 VSYNC 信号。
4， 像素时钟输入（PIXCLK），用于接摄像头的 PCLK 信号。

DCMI 接口是一个同步并行接口，可接收高速（可达 140 MB/s）数据流。该接口包含多达 14 条数据线(D13-D0)和一条像素时钟线(PIXCLK)。像素时钟的极性可以编程，因此可以在像素时钟的上升沿或下降沿捕获数据。

DCMI 接收到的摄像头数据被放到一个 32 位数据寄存器(DCMI_DR)中，然后通过通用 DMA 进行传输。图像缓冲区由 DMA 管理，而不是由摄像头接口管理。

从摄像头接收的数据可以按行/帧来组织（原始 YUV/RGB/拜尔模式），也可以是一系列 JPEG 图像。要使能 JPEG 图像接收，必须将 JPEG 位（DCMI_CR 寄存器的位 3）置 1。

数据流可由可选的 HSYNC（水平同步）信号和 VSYNC（垂直同步）信号硬件同步，或者通 过数据流中嵌入的同步码同步。

51.1.2.1 DCMI 接口功能概述

STM32H743 DCMI 接口的框图如图 51.1.2.1.1 所示：

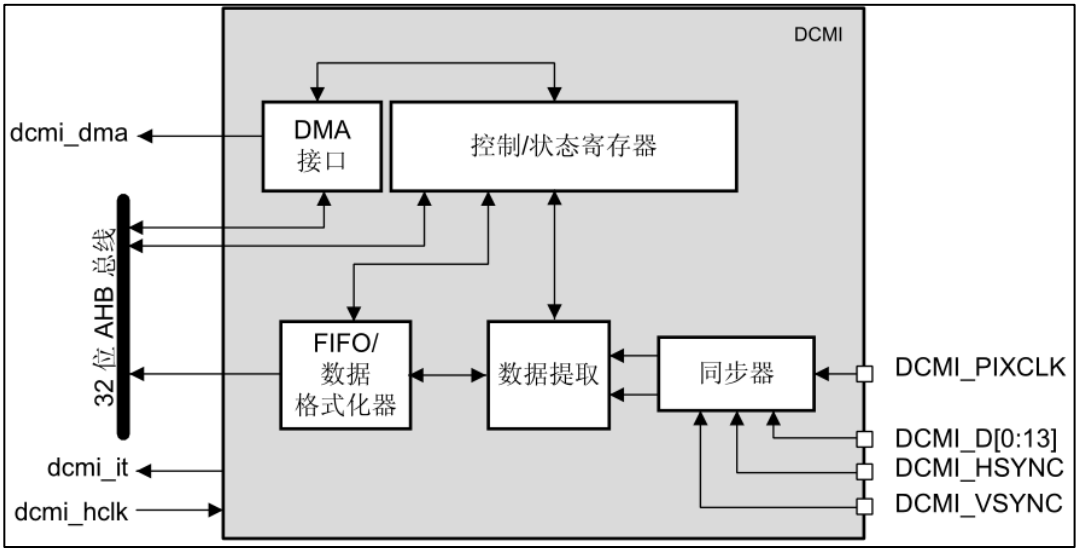


图 51.1.2.1.1 DCMI 接口框图

上图左边可以看到的是 DCMI 的内部信号，如下表所示：

信号名称	信号类型	说明
dcmi_dma	数字输出	DCMI DMA 请求
dcmi_it	数字输出	DCMI 中断请求
dcmi_hclk	数字输入	DCMI 接口时钟

表 51.1.2.1.1 DCMI 内部信号

dcmi_dma，DCMI DMA 请求信号。当 DCMI_CR 寄存器中的 CAPTURE 位置 1 时，激活 DMA 接口。摄像头接口每次在其寄存器中收到一个完整的 32 位数据块时，都将触发一个 DMA 请求。

dcmi_it，DCMI 中断请求信号。DCMI 有五种中断源，所有中断都可通过软件屏蔽。全局中断 (dcmi_it) 是所有单个中断的逻辑或运算所得的结果。

DCMI 的五种中断源，如下表所示：

中断名称	中断事件
IT_LINE	表示行结束
IT_FRAME	表示帧捕获结束
IT_OVR	表示接收的数据发生溢出错误
IT_VSYNC	表示同步帧
IT_ERR	表示内嵌码同步帧检测期间检测到错误
dcmi_it	以上中断的逻辑或结果

表 51.1.2.1.2 DCMI 中断源

dcmi_hclk，DCMI 接口时钟，时钟源自来于 HCLK3（我们设置为 200MHZ）。

上图中间部分就是 DCMI 内核。

上图右边的 DCMI 物理接口，具体如下表所示：

引脚名称	信号类型	说明
8 位： DCMI_D[0..7] 10 位：DCMI_D[0..9] 12 位：DCMI_D[0..11] 14 位：DCMI_D[0..13]	数字输入	DCMI 数据
DCMI_PIXCLK	数字输入	像素时钟
DCMI_HSYNC	数字输入	水平同步/数据有效
DCMI_VSYNC	数字输入	垂直同步

表 51.1.2.1.3 DCMI 物理接口

该接口由 11/13/15/17 个输入信号组成。仅支持从模式。根据 DCMI_CR 寄存器中 EDM[1:0]

位的设置，摄像头接口可以捕获 8 位、10 位、12 位或 14 位数据。如果使用的位数少于 14，则必须将未使用的输入引脚接地。

DCMI 数据与 DCMI_PIXCLK（即 PCLK）保持同步，并根据像素时钟的极性在像素时钟上升沿/下降沿发生变化。

DCMI_HSYNC（HREF）信号指示行的开始/结束。

DCMI_VSYNC 信号指示帧的开始/结束。

DCMI 信号波形，如图 51.1.2.1.2 所示：

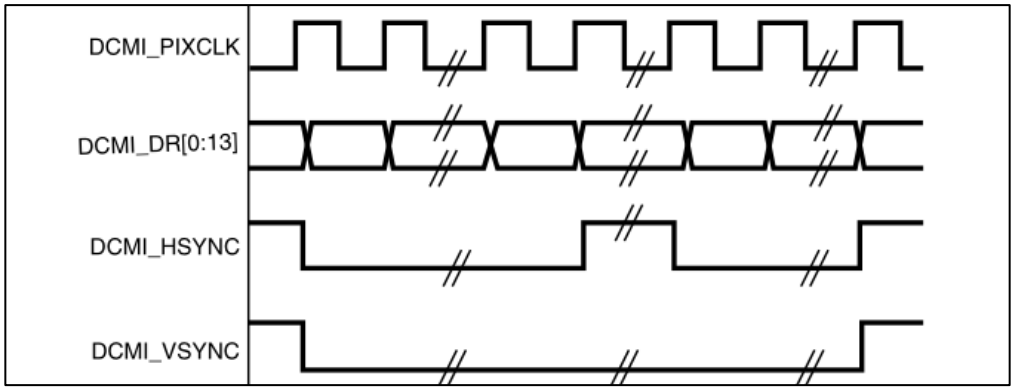


图 51.1.2.1.2 DCMI 信号波形

上图中，对应设置为：DCMI_PIXCLK 的捕获沿为下降沿，DCMI_HSYNC 和 DCMI_VSYNC 的有效状态为 1，注意，这里的有效状态实际上对应的是指示数据在并行接口上无效时，HSYNC/VSYNC 引脚上面的引脚电平。

本实验我们用到 DCMI 的 8 位数据宽度，通过设置 DCMI_CR 中的 EDM[1:0]=00 设置。此时 DCMI_D0~D7 有效，DCMI_D8~D13 上的数据则忽略，这个时候，每次需要 4 个像素时钟来捕获一个 32 位数据。捕获的第一个数据存放在 32 位字的 LSB 位置，第四个数据存放在 32 位字的 MSB 位置，捕获数据字节在 32 位字中的排布如表 51.1.2.1.4 所示：

字节地址	31:24	23:16	15:8	7:0
0	D_{n+3}[7:0]	D_{n+2}[7:0]	D_{n+1}[7:0]	D_n[7:0]
4	D_{n+7}[7:0]	D_{n+6}[7:0]	D_{n+5}[7:0]	D_{n+4}[7:0]

表 51.1.2.1.4 8 位捕获数据在 32 位字中的排布

从表 51.1.2.1.4 可以看出，STM32H743 的 DCMI 接口，接收的数据是低字节在前，高字节在后的，所以，要求摄像头输出数据也是低字节在前，高字节在后才可以，否则就还得程序上处理字节顺序，会比较麻烦。

DCMI 接口支持两种同步方式：内嵌码同步和硬件（HSYNC 和 VSYNC）同步。我们使用硬件同步，硬件同步模式下将使用两个同步信号 (HSYNC/VSYNC)。根据摄像头模块/模式的不同，可能在水平/垂直同步期间内发送数据。由于系统会忽略 HSYNC/VSYNC 信号有效电平期间内接收的所有数据，HSYNC/VSYNC 信号相当于消隐信号。

为了正确地将图像传输到 DMA/RAM 缓冲区，数据传输将与 VSYNC 信号同步。选择硬件同步模式并启用捕获（DCMI_CR 中的 CAPTURE 位置 1）时，数据传输将与 VSYNC 信号的无效电平同步（开始下一帧时）。之后传输便可以连续执行，由 DMA 将连续帧传输到多个连续的缓冲区或一个具有循环特性的缓冲区。为了允许 DMA 管理连续帧，每一帧结束时都将激活 VSIF（垂直同步中断标志，即帧中断），我们可以利用这个帧中断来判断是否有一帧数据采集完成，方便处理数据。

DCMI 接口的捕获模式支持：快照模式和连续采集模式。一般我们使用连续采集模式，通过 DCMI_CR 中的 CM 位设置。另外，DCMI 接口还支持实现了 4 个字深度的 FIFO，配有一个简单的 FIFO 控制器，每次摄像头接口从 AHB 读取数据时读指针递增，每次摄像头接口向 FIFO 写入数据时写指针递增。因为没有溢出保护，如果数据传输率超过 AHB 接口能够承受的速率，FIFO 中的数据就会被覆盖。如果同步信号出错，或者 FIFO 发生溢出，FIFO 将复位，DCMI 接口将等待新的数据帧开始。

关于 DCMI 接口的其他特性，我们这里就不再介绍了，请大家参考《STM32H7xx 参考手册_V7（英文版）.pdf》第 31 章相关内容。

51.1.2.2 DCMI 寄存器

本实验，我们将 OV5640 默认配置为 WXGA 输出，也就是 1280*800 的分辨率，输出信号设置为：VSYNC 高电平有效，HREF 高电平有效，输出数据在 PCLK 的下降沿输出（即上升沿的时候，MCU 才可以采集）。这样，STM32H743 的 DCMI 接口就必须设置为：VSYNC 低电平有效、HSYNC 低电平有效和 PIXCLK 上升沿有效，这些设置都是通过 DCMI_CR 寄存器控制。

- **DCMI 控制寄存器（DCMI_CR）**

DCMI 控制寄存器描述如图 51.1.2.2.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OELS	LSM	OEBS	BSM	
											RW	RW	RW	RW	RW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	ENABLE	Res.	Res.	EDM		FCRC		VSPOL	HSPOL	PCKPOL	ESS	JPEG	CROP	CM	CAPTURE
	rw			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 14 **ENABLE**：DCMI 使能 (DCMI enable)
0：禁止 DCMI　　1：使能 DCMI
注：　使能此位之前，应对 DCMI 配置寄存器进行适当的设置

位 11:10 **EDM[1:0]**：扩展数据模式 (Extended data mode)
00：接口每个像素时钟捕获 8 位数据　　10：接口每个像素时钟捕获 12 位数据
01：接口每个像素时钟捕获 10 位数据　　11：接口每个像素时钟捕获 14 位数据

位 9:8 **FCRC[1:0]**：帧捕获率控制 (Frame capture rate control)
这些位定义了帧捕获频率。仅在连续采集模式下有效。快照模式下将被忽略。
00：捕获所有帧　　10：每隔三帧捕获一次（带宽降低 75%）
01：每隔一帧捕获一次（带宽降低 50%）　　11：保留

位 7 **VSPOL**：垂直同步极性 (Vertical synchronization polarity)
此位指示数据在并行接口上无效时 DCMI_VSYNC 引脚的电平。
0：DCMI_VSYNC 低电平有效　　1：DCMI_VSYNC 高电平有效

位 6 **HSPOL**：水平同步极性 (Horizontal synchronization polarity)
此位指示数据在并行接口上无效时 DCMI_HSYNC 引脚的电平。
0：DCMI_HSYNC 低电平有效　　1：DCMI_HSYNC 高电平有效

位 5 **PCKPOL**：像素时钟极性 (Pixel clock polarity)
此位用来配置像素时钟的捕获沿
0：下降沿有效。　　1：上升沿有效。

位 4 **ESS**：内嵌码同步选择 (Embedded synchronization select)
0：硬件同步，数据捕获（帧/行开始/停止）由 DCMI HSYNC/DCMI VSYNC 信号同步。
1：内嵌码同步，数据捕获由数据流中嵌入的同步码同步。
注：　仅对 8 位并行数据有效。ESS 位置 1 时，将忽略 HSPOL/VSPOL。 JPEG 模式下会禁止此位。

位 2 **CROP**：裁剪功能 (Crop feature)
0：捕获完整图像。这种情况下，图像帧包含的字节总数应该为 4 的倍数。
1：仅捕获剪裁寄存器所指定的窗口中的数据。如果窗口大小超出图片大小，则仅捕获图片大小。

位 1 **CM**：捕获模式 (Capture mode)
0：连续采集模式——收到的数据将通过 DMA 传输到目标存储区。缓冲区位置和模式（线性或循环缓冲区）由系统 DMA 控制。
1：快照模式（单帧）——一旦激活，接口将等待帧开始，然后通过 DMA 传输单帧。帧结束时将自动复位 CAPTURE 位。

位 0 **CAPTURE**：使能捕获 (Capture enable)
0：禁止捕获。　　1：使能捕获。
摄像头接口等待第一帧开始，然后生成一个 DMA 请求以将收到的数据传输到目标存储器中。
在快照模式下，收到的第一帧结束时将自动使 CAPTURE 位清零。
在连续采集模式下，如果在执行捕获操作时通过软件将此位清零，则帧结束后此位的清零才生效。

图 51.1.2.2.1 DCMI_CR 寄存器

ENABLE，该位用于设置是否使能 DCMI，不过，在使能之前，必须将其他配置设置好。

EDM[1:0]，设置扩展数据模式，选择 00：接口每个像素时钟捕获 8 位数据。

FCRC[1:0]，这两个位用于帧率控制，我们捕获所有帧，所以设置为 00 即可。

VSPOL，该位用于设置垂直同步极性，也就是 VSYNC 引脚上面，数据无效时的电平状态，根据前面说所，我们应该设置为 0。

HSPOL，该位用于设置水平同步极性，也就是 HSYNC 引脚上面，数据无效时的电平状态，同样应该设置为 0。

PCKPOL，该位用于设置像素时钟极性，我们用上升沿捕获，所以设置为 1。

ESS，内嵌码同步选择，我们选择硬件同步，默认置 0。

CM，该位用于设置捕获模式，我们用连续采集模式，所以设置为 0 即可。

CAPTURE，该位用于使能捕获，我们设置为 1。该位使能后，将激活 DMA，DCMI 等待第一帧开始，然后生成 DMA 请求将收到的数据传输到目标存储器中。注意：该位必须在 DCMI 的其他配置（包括 DMA）都设置好了之后，才设置!!

DCMI_CR 寄存器的其他位，我们就不介绍了，另外 DCMI 的其他寄存器这里也不再介绍，请大家参考《STM32H7xx 参考手册_V7（英文版）.pdf》第 31.5 小节。关于 DMA 的寄存器，这里就不再赘述，请回顾前面的实验。

51.2 硬件设计

1. 例程功能

1、本实验开机后，初始化摄像头模块（OV5640），如果初始化成功，则提示选择模式：RGB565 模式或者 JPEG 模式。KEY0 用于选择 RGB565 模式，KEY1 用于选择 JPEG 模式。

2、当使用 RGB565 时，输出图像（固定为：WXGA）将经过缩放处理（完全由 OV5640 的 DSP 控制），显示在 LCD 上面（默认开启连续自动对焦）。我们可以通过 KEY_UP 按键选择：1:1 显示，即不缩放，图片不变形，但是显示区域小（液晶分辨率大小），或者缩放显示，即将 1280*800 的图像压缩到液晶分辨率尺寸显示，图片变形，但是显示了整个图片内容。通过 KEY0 按键，可以设置对比度；KEY1 按键，可以启动单次自动对焦；KEY2 按键，可以设置特效。

3、当使用 JPEG 模式时，图像可以设置默认是 QVGA 320*240 尺寸，采集到的 JPEG 数据将先存放到 STM32 的 RAM 内存里面，每当采集到一帧数据，就会关闭 DMA 传输，然后将采集到的数据发送到串口 2（此时可以通过上位机软件（XCAM.exe）接收，并显示图片），之后再重新启动 DMA 传输。通过按键 KEY_UP 设置输出图片的尺寸（QSXGA~QQVGA）、按键 KEY0 可以设置对比度；按键 KEY1 可以启动单次自动对焦；按键 KEY2 可以设置特效。

4、同时可以通过串口 1，借助 USMART 设置/读取 OV5640 的寄存器，方便大家调试。

5、LED0 闪烁，提示程序运行。LED1 用于指示帧中断。。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
LED1：LED1 - PB0
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）独立按键：KEY0 - PH3、KEY1 - PH2
5）串口 2 (PA2/PA3，外接串口线/用杜邦线连接到 CH340 上面)
6）DCMI 接口(用于驱动 OV5640 摄像头模块)
7）定时器 3(用于打印摄像头帧率等信息)
8) PCF8574（控制 OV_PWDN）
9）正点原子 OV5640 摄像头模块，连接关系为：
OV5640 模块 ---------- STM32 开发板

OV_D0~D7 ---------- PC6/PC7/PC8/PC9/PC11/PD3/PB8/PB9
OV_SCL ---------- PB4
OV_SDA ---------- PB3
OV_VSYNC ---------- PB7
OV_HREF ---------- PH8
OV_RESET ---------- PA15
OV_PCLK ---------- PA8
OV_PWDN ---------- PCF8574_P2

3. 原理图

开发板板载的摄像头模块接口与 MCU 的连接关系，如下图所示：

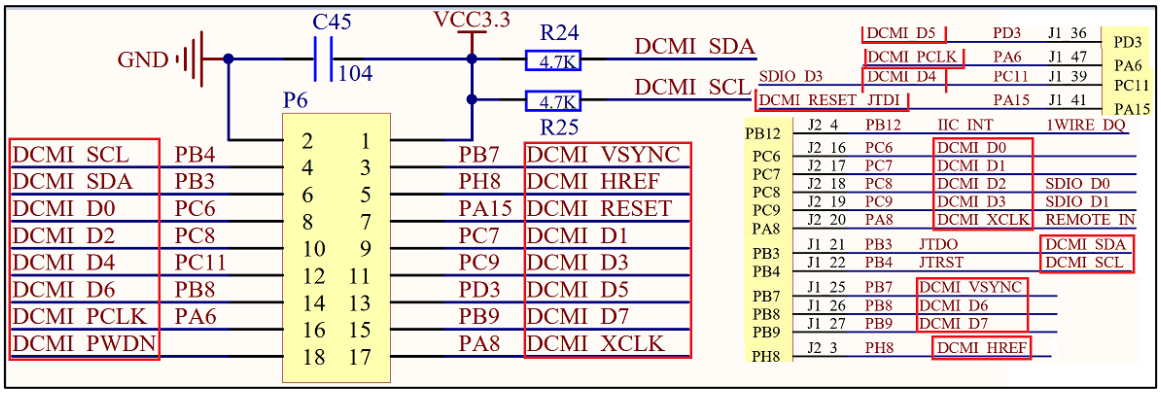


图 51.2.1 摄像头接口与开发板连接示意图

这些 GPIO 口的线都在开发板上连接到 P6 端口，所以我们只需要将 OV5640 摄像头模块插上开发板的连接座子就好了（摄像头模块正面往外插）。

图 51.2.2 OV5640 摄像头模块与开发板的连接座子

51.3 程序设计

51.3.1 DCMI 的 HAL 库驱动

DCMI 在 HAL 库中的驱动代码在 stm32h7xx_hal_dcmi.c 文件（及其头文件）中。

1. HAL_DCMI_Init 函数

DCMI 初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_DCMI_Init(DCMI_HandleTypeDef *hdcmi);
```

- **函数描述：**

  用于初始化 DCMI。

- **函数形参：**

  形参 1 是 DCMI_HandleTypeDef 结构体类型指针变量，其定义如下：

```
typedef struct
{
  DCMI_TypeDef                *Instance;                /* DCMI 外设寄存器基地址 */
  DCMI_InitTypeDef             Init;                    /* DCMI 初始化结构体 */
  HAL_LockTypeDef              Lock;                    /* 锁对象 */
  __IO HAL_DCMI_StateTypeDef   State;                   /* DCMI 工作状态 */
  __IO uint32_t                XferCount;               /* DMA 传输计数器 */
```

```
  __IO uint32_t                 XferSize;                /* DMA 传输数据的大小 */
  uint32_t                      XferTransferNumber;      /* DMA 数据的个数 */
  uint32_t                      pBuffPtr;                /* DMA 输出缓冲区地址 */
  DMA_HandleTypeDef             *DMA_Handle;             /* DMA 配置结构体指针 */
  __IO uint32_t                 ErrorCode;               /* DCMI 错误代码 */
}DCMI_HandleTypeDef;
```

下面重点介绍 DCMI_InitTypeDef 结构体，其定义如下：

```
typedef struct {
 uint32_t SynchroMode;                  /* 同步方式选择硬件同步模式还是内嵌码模式 */
 uint32_t PCKPolarity;                  /* 设置像素时钟的有效边沿 */
 uint32_t VSPolarity;                   /* 设置垂直同步的有效电平 */
 uint32_t HSPolarity;                   /* 设置水平同步的有效边沿 */
 uint32_t CaptureRate;                  /* 设置图像的帧捕获率 */
 uint32_t ExtendedDataMode;             /* 设置数据线的宽度（扩展数据模式） */
 DCMI_CodesInitTypeDef SyncroCode;      /* 分隔符设置 */
 uint32_t JPEGMode;                     /* JPEG 模式选择 */
 uint32_t ByteSelectMode;               /* 配置字节选项模式 */
 uint32_t ByteSelectStart;              /* 字节选择开始 */
 uint32_t LineSelectMode;               /* 行选择模式 */
 uint32_t LineSelectStart;              /* 指定数据行是奇数还是偶数 */
} DCMI_InitTypeDef;
```

1) SynchroMode：用于设置 DCMI 数据的同步模式，可以选择为硬件同步方式或内嵌码方式。如果选择硬件同步值 DCMI_SYNCHRO_HARDWARE，那么数据捕获由 HSYNC/VSYNC 信号同步，如果选择内嵌码同步方式值 DCMI_SYNCHRO_EMBEDDED，那么数据捕获由数据流中嵌入的同步码同步。

2) PCKPolarity：用来设置像素时钟极性为上升沿有效还是下降沿有效。

3) VSPolarity：用于设置 VSYNC 的有效电平，当 VSYNC 信号线表示为有效电平时，表示新的一帧数据传输完成，它可以被设置为高电平有效或低电平有效。

4) HSPolarity：用于设置 HSYNC 的有效电平，当 HSYNC 信号线表示为有效电平时，表示新的一行数据传输完成，它可以被设置为高电平有效或低电平有效。

5) CaptureRate：用于设置帧捕获率，可以设置为全采集、半采集或 1/4 采集。设置的值为 DCMI_CR_ALL_FRAME（表示全帧捕获），设置为 DCMI_CR_ALTERNATE_2_FRAME（表示 2 帧捕获一帧），设置为 DCMI_CR_ALTERNATE_4_FRAME（表示 4 帧捕获一帧）。

6) ExtendedDataMode：用于设置扩展数据模式，可设置为 8/10/12 或者 14 位数据宽度。

7) SyncroCode：用于设置分隔码，包括：帧结束分隔码，行结束分隔码，行开始分隔码以及帧开始分隔码。

8) JPEGMode：用于设置 JPEG 格式使能或禁止。

9）ByteSelectMode 用于设置字节选项模式，也就是接口对接收到的数据每隔多少个字节捕获一个字节，取值为：DCMI_BSM_ALL(捕获所有字节)，DCMI_BSM_OTHER(每隔一个字节进行捕获)，DCMI_BSM_ALTERNATE_4（每四个字节捕获一个字节）和 DCMI_BSM_ALTERNATE_2（每四个字节捕获两个字节）。

10）ByteSelectStart 用于设置奇数偶数字节选择开始，也就是接口从帧/行开始捕获第一个数据同时丢弃第二个字节（DCMI_OEBS_ODD）或者捕获第二个数据同时丢弃第一个字节（DCMI_OEBS_EVEN）。

11）LineSelectMode 用于配置行选择模式，也就是选择接口捕获所有接受到的行（DCMI_LSM_ALL）还是每两行捕获一行（DCMI_LSM_ALTERNATE_2）。

12）LineSelectStart 用于配置奇数偶数行选择开始。也就是接口在帧开始后捕获第一行丢弃第二行（DCMI_OELS_ODD）或者在帧开始后捕获第二行同时丢弃第一行（DCMI_OELS_EVEN）。

- **函数返回值：**

  HAL_StatusTypeDef 枚举类型的值。

以 DMA 方式传输 DCMI 数据配置步骤

1）配置 OV5640 控制引脚，并配置 OV5640 工作模式。

在启动 DCMI 之前，我们先设置好 OV5640。OV5640 通过 OV_SCL 和 OV_SDA 进行寄存器配置，同时还有 OV_PWDN/OV_RESET 等信号，我们也需要配置对应 IO 状态，先设置 OV_PWDN 为 0，退出掉电模式，然后拉低 OV_RESET 复位 OV5640，之后再设置 OV_RESET 为 1，结束复位，然后就是对 OV5640 的寄存器进行配置了。然后，可以根据我们的需要，设置成 RGB565 输出模式，还是 JPEG 输出模式。

2）配置相关引脚的模式和复用功能（AF13），使能时钟。

OV5640 配置好之后，再设置 DCMI 接口与摄像头模块连接的 IO 口，使能 IO 和 DCMI 时钟，然后设置相关 IO 口为复用功能模式，复用功能选择 AF13(DCMI 复用)。

DCMI 时钟使能方法：

```
__HAL_RCC_DCMI_CLK_ENABLE();                    /* 使能 DCMI 时钟 */
```

引脚模式配置就是通过 HAL_GPIO_Init 函数来配置。

3）配置 DCMI 相关设置，初始化 DCMI 接口。

这一步，主要通过 DCMI_CR 寄存器设置，包括 VSPOL/HSPOL/PCKPOL/数据宽度等重要参数，都在这一步设置。HAL 库提供了 DCMI 初始化函数 HAL_DCMI_Init，函数声明如下：

```
HAL_StatusTypeDef HAL_DCMI_Init(DCMI_HandleTypeDef *hdcmi);
```

该结构体第一个成员变量 Instance 用来指向寄存器基地址，设置为 DCMI 即可。

成员变量 XferCount，XferSize，XferTransferNumber，pBuffPtr 和 DMA_Handle 是与 HAL 库中 DMA 处理相关中间变量，由于使用 HAL 库配置的 DCMI DMA 会非常复杂，而且灵活性不高，所以本实验我们是独立配置的 DMA。

同样，HAL 库也提供了 DCMI 接口的 MSP 初始化回调函数：

```
void HAL_DCMI_MspInit(DCMI_HandleTypeDef* hdcmi);
```

一般情况下，该函数内部编写时钟使能，IO 初始化以及 NVIC 相关程序。

4）配置 DMA。

本实验我们采用连续模式采集，并将采集到的数据输出到 LCD（RGB565 模式）或内存（JPEG 模式），所以源地址都是 DCMI_DR，而目的地址可能是 LCD->RAM 或者 SDRAM 的地址。DCMI 的 DMA 传输采用的是 DMA1 数据流 1 的通道 1 来实现的，关于 DMA 的介绍，请大家参考前面的 DMA 实验章节。

5）设置 OV5640 的图像输出大小，使能 DCMI 捕获。

图像输出大小设置，分两种情况：在 RGB565 模式下，我们根据 LCD 的尺寸，设置输出图像大小，以实现全屏显示（图像可能因缩放而变形）；在 JPEG 模式下，我们可以自由设置输出图像大小（可不缩放）；最后，开启 DCMI 捕获，即可正常工作了。

51.3.2 程序流程图

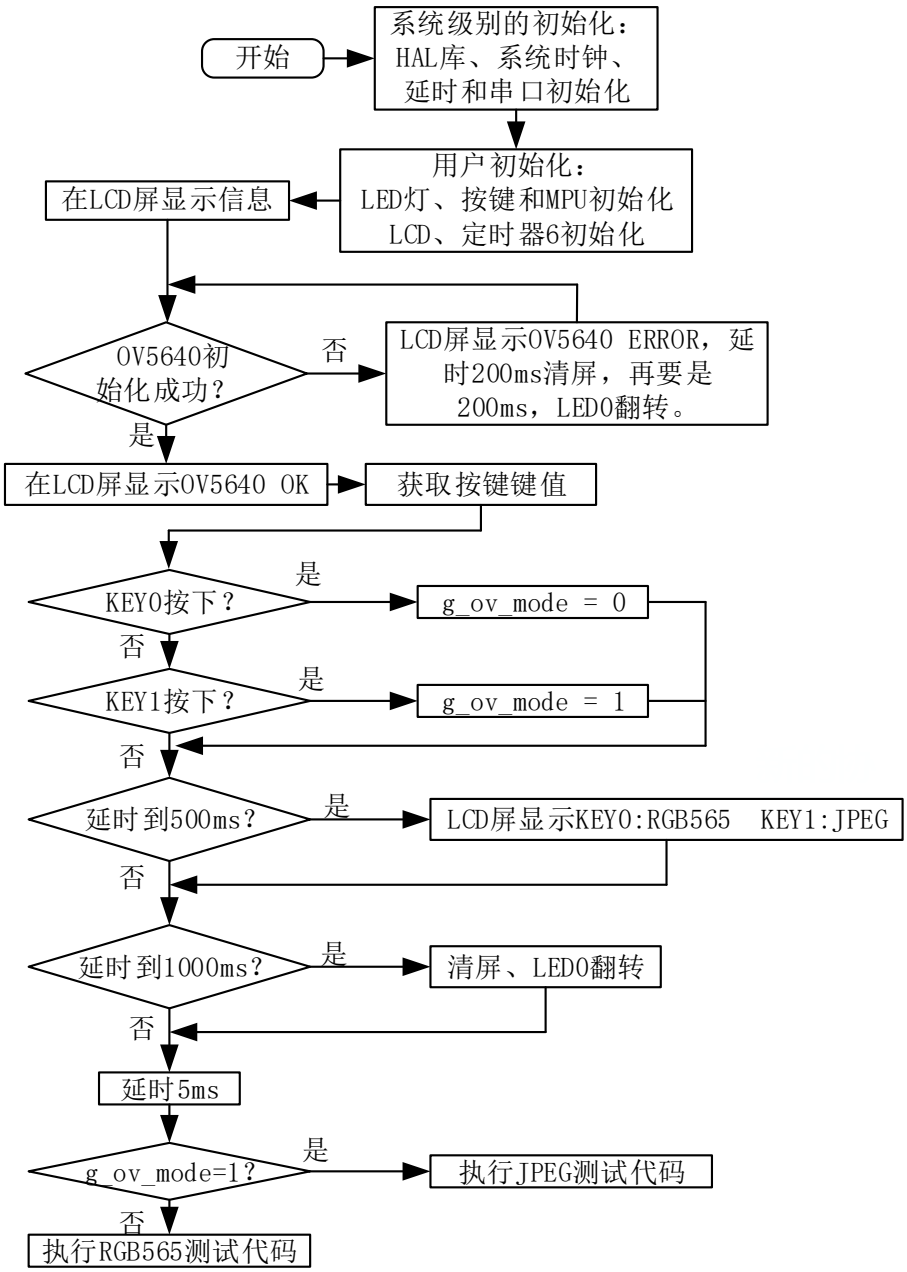


图 51.3.2.1 摄像头实验程序流程图

51.3.3 程序解析

1. DCMI 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。DCMI 驱动源码包括两个文件：dcmi.c 和 dcmi.h。

dcmi.h 头文件只是一些声明，下面直接开始介绍 dcmi.c 文件，首先是 DCMI 初始化函数，其定义如下：

```
/**
 * @brief       DCMI 初始化
 * @note        IO 对应关系如下:
 *              摄像头模块 ------------ STM32 开发板
 *               OV_D0~D7  ------------  PC6/PC7/PC8/PC9/PC11/PD3/PB8/PB9
```

```
 *               OV_SCL     ------------  PB4
 *               OV_SDA     ------------  PB3
 *               OV_VSYNC   ------------  PB7
 *               OV_HREF    ------------  PH8
 *               OV_RESET   ------------  PA15
 *               OV_PCLK    ------------  PA6
 *               OV_XCLK    ------------  PA8
 *              本函数仅初始化 OV_D0~D7/OV_VSYNC/OV_HREF/OV_PCLK 等信号(11 个).
 * @param       无
 * @retval      无
 */
void dcmi_init(void)
{
    g_dcmi_handle.Instance = DCMI;
    g_dcmi_handle.Init.SynchroMode =DCMI_SYNCHRO_HARDWARE;/*硬件同步 HSYNC,VSYNC*/
    g_dcmi_handle.Init.PCKPolarity = DCMI_PCKPOLARITY_RISING;/* PCLK 上升沿有效 */
    g_dcmi_handle.Init.VSPolarity = DCMI_VSPOLARITY_LOW;     /* VSYNC 低电平有效 */
    g_dcmi_handle.Init.HSPolarity = DCMI_HSPOLARITY_LOW;     /* HSYNC 低电平有效 */
    g_dcmi_handle.Init.CaptureRate = DCMI_CR_ALL_FRAME;      /* 全帧捕获 */
    g_dcmi_handle.Init.ExtendedDataMode = DCMI_EXTEND_DATA_8B;/* 8 位数据格式 */
    HAL_DCMI_Init(&g_dcmi_handle);     /* 初始化 DCMI，此函数会开启帧中断 */

    __HAL_DCMI_ENABLE_IT(&g_dcmi_handle, DCMI_IT_FRAME);  /* 开启镜头捕捉中断 */
    __HAL_DCMI_ENABLE(&g_dcmi_handle);
    __HAL_DCMI_DISABLE_IT(&g_dcmi_handle,DCMI_IT_LINE|
                          DCMI_IT_VSYNC|DCMI_IT_ERR|DCMI_IT_OVR);
    HAL_NVIC_SetPriority(DCMI_IRQn, 2, 2);             /* 抢占优先级 2，子优先级 2 */
    HAL_NVIC_EnableIRQ(DCMI_IRQn);                     /* 使能 DCMI 中断 */
}
```

该函数主要对 DCMI_HandleTypeDef 结构体成员赋值并初始化，最后关闭所有中断，只开启帧中断，使能 DCMI。而 DCMI 接口的 GPIO 口的初始化是在 HAL_DCMI_MspInit 回调函数中完成，其定义如下：

```
/**
 * @brief       DCMI 底层驱动，引脚配置，时钟使能，中断配置
 * @param       hdcmi:DCMI 句柄
 * @note        此函数会被 HAL_DCMI_Init()调用
 * @retval      无
 */
void HAL_DCMI_MspInit(DCMI_HandleTypeDef *hdcmi)
{
    GPIO_InitTypeDef gpio_init_struct;
    __HAL_RCC_DCMI_CLK_ENABLE();                    /* 使能 DCMI 时钟 */
    __HAL_RCC_GPIOA_CLK_ENABLE();                   /* 使能 GPIOA 时钟 */
    __HAL_RCC_GPIOB_CLK_ENABLE();                   /* 使能 GPIOB 时钟 */
    __HAL_RCC_GPIOC_CLK_ENABLE();                   /* 使能 GPIOC 时钟 */
    __HAL_RCC_GPIOD_CLK_ENABLE();                   /* 使能 GPIOD 时钟 */
    __HAL_RCC_GPIOH_CLK_ENABLE();                   /* 使能 GPIOH 时钟 */

    gpio_init_struct.Pin = GPIO_PIN_6;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;              /* 推挽复用 */
    gpio_init_struct.Pull = GPIO_PULLUP;                  /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;   /* 高速 */
    gpio_init_struct.Alternate = GPIO_AF13_DCMI;          /* 复用为 DCMI */
    HAL_GPIO_Init(GPIOA, &gpio_init_struct);              /* 初始化 PA6 引脚 */

    gpio_init_struct.Pin = GPIO_PIN_7 | GPIO_PIN_8 | GPIO_PIN_9;
    HAL_GPIO_Init(GPIOB, &gpio_init_struct);          /* 初始化 PB7,8,9 引脚 */

    gpio_init_struct.Pin = GPIO_PIN_6 | GPIO_PIN_7 | GPIO_PIN_8 | GPIO_PIN_9 |
                                                                   GPIO_PIN_11;
    HAL_GPIO_Init(GPIOC, &gpio_init_struct);         /* 初始化 PC6,7,8,9,11 引脚 */
```

```
    gpio_init_struct.Pin = GPIO_PIN_3;
    HAL_GPIO_Init(GPIOD, &gpio_init_struct);         /* 初始化 PD3 引脚 */

    gpio_init_struct.Pin = GPIO_PIN_8;
    HAL_GPIO_Init(GPIOH, &gpio_init_struct);         /* 初始化 PH8 引脚 */
}
```

DCMI 接口的 GPIO 口前面都介绍过了，该函数最后设置了 DCMI 中断抢占优先级为 2，子优先级为 2，并且使能 DCMI 中断。

接下来介绍 DCMI DMA 配置初始化函数，其定义如下：

```
/**
 * @brief       DCMI DMA 配置
 * @param       mem0addr: 存储器地址 0      将要存储摄像头数据的内存地址(也可以是外设地址)
 * @param       mem1addr: 存储器地址 1      当只使用 mem0addr 的时候,该值必须为 0
 * @param       memsize : 存储器长度        0~65535
 * @param       memblen : 存储器位宽        DMA_MDATAALIGN_BYTE,8 位;
                DMA_MDATAALIGN_HALFWORD,16 位;DMA_MDATAALIGN_WORD,32 位
 * @param       meminc:存储器增长方式 DMA_MINC_DISABLE,不增长; DMA_MINC_ENABLE,增长
 * @retval      无
 */
void dcmi_dma_init(uint32_t mem0addr,uint32_t mem1addr,
                   uint16_t memsize,uint32_t memblen,uint32_t meminc)
{
    __HAL_RCC_DMA1_CLK_ENABLE();                                  /* 使能 DMA1 时钟 */
    /* 将 DMA 与 DCMI 联系起来 */
    __HAL_LINKDMA(&g_dcmi_handle, DMA_Handle, g_dma_dcmi_handle);
    /* 先关闭 DMA 传输完成中断(否则在使用 MCU 屏的时候会出现花屏的情况) */
    __HAL_DMA_DISABLE_IT(&g_dma_dcmi_handle, DMA_IT_TC);

    g_dma_dcmi_handle.Instance = DMA1_Stream1;                    /* DMA1 数据流 1 */
    g_dma_dcmi_handle.Init.Request = DMA_REQUEST_DCMI;            /* DCMI 的 DMA 请求 */
    g_dma_dcmi_handle.Init.Direction = DMA_PERIPH_TO_MEMORY;/* 外设到存储器 */
    g_dma_dcmi_handle.Init.PeriphInc = DMA_PINC_DISABLE;      /* 外设非增量模式 */
    g_dma_dcmi_handle.Init.MemInc = meminc;                       /* 存储器增量模式 */
    /* 外设数据长度:32 位 */
    g_dma_dcmi_handle.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD;
    g_dma_dcmi_handle.Init.MemDataAlignment= memblen;/*存储器数据长度:8/16/32 位*/
    g_dma_dcmi_handle.Init.Mode = DMA_CIRCULAR;                /* 使用循环模式 */
    g_dma_dcmi_handle.Init.Priority = DMA_PRIORITY_HIGH;   /* 高优先级 */
    g_dma_dcmi_handle.Init.FIFOMode = DMA_FIFOMODE_ENABLE;/* 使能 FIFO */
    /* 使用 1/2 的 FIFO */
    g_dma_dcmi_handle.Init.FIFOThreshold = DMA_FIFO_THRESHOLD_HALFFULL;
    g_dma_dcmi_handle.Init.MemBurst = DMA_MBURST_SINGLE;     /* 存储器突发传输 */
    g_dma_dcmi_handle.Init.PeriphBurst = DMA_PBURST_SINGLE;/* 外设突发单次传输 */
    HAL_DMA_DeInit(&g_dma_dcmi_handle);                           /* 先清除以前的设置 */
    HAL_DMA_Init(&g_dma_dcmi_handle);                             /* 初始化 DMA */

    /* 在开启 DMA 之前先使用__HAL_UNLOCK()解锁一次 DMA,因为 HAL_DMA_Statrt()
       HAL_DMAEx_MultiBufferStart()这两个函数一开始要先使用__HAL_LOCK()锁定 DMA,
       而函数__HAL_LOCK()会判断当前的 DMA 状态是否为锁定状态，如果是锁定状态的话就直接
       返回 HAL_BUSY，这样会导致函数 HAL_DMA_Statrt()和 HAL_DMAEx_MultiBufferStart()
       后续的 DMA 配置程序直接被跳过！DMA 也就不能正常工作，为了避免这种现象，所以在启动
       DMA 之前先调用__HAL_UNLOC()先解锁一次 DMA。 */

    __HAL_UNLOCK(&g_dma_dcmi_handle);
    if (mem1addr == 0)  /* 开启 DMA，不使用双缓冲 */
    {
        HAL_DMA_Start(&g_dma_dcmi_handle, (uint32_t)&DCMI->DR,
                      mem0addr, memsize);
    }
```

```
    else                    /* 使用双缓冲 */
    {
        HAL_DMAEx_MultiBufferStart(&g_dma_dcmi_handle, (uint32_t)&DCMI->DR,
                                   mem0addr, mem1addr, memsize); /* 开启双缓冲 */
        __HAL_DMA_ENABLE_IT(&g_dma_dcmi_handle, DMA_IT_TC); /* 开启传输完成中断 */
        HAL_NVIC_SetPriority(DMA1_Stream1_IRQn, 2, 3);       /* DMA 中断优先级 */
        HAL_NVIC_EnableIRQ(DMA1_Stream1_IRQn);
    }
}
```

该函数用于配置 DCMI 的 DMA 传输，其外设地址固定为：DCMI->DR，而存储器地址可变（LCD 或者 SRAM）。DMA 被配置为循环模式，一旦开启，DMA 将不停的循环传输数据。

下面介绍的是 DCMI 启动传输和关闭传输函数，它们的定义分别如下：

```
/**
 * @brief       DCMI,启动传输
 * @param       无
 * @retval      无
 */
void dcmi_start(void)
{
    if (lcddev.wramcmd != 0)
    {
        lcd_set_cursor(0, 0);               /* 设置坐标到原点 */
        lcd_write_ram_prepare();            /* 开始写入 GRAM */
    }
    __HAL_DMA_ENABLE(&g_dma_dcmi_handle); /* 使能 DMA */
    DCMI->CR |= DCMI_CR_CAPTURE;            /* DCMI 捕获使能 */
}

/**
 * @brief       DCMI,关闭传输
 * @param       无
 * @retval      无
 */
void dcmi_stop(void)
{
    DCMI->CR &= ~(DCMI_CR_CAPTURE);             /* DCMI 捕获关闭 */
    while (DCMI->CR & 0X01);                    /* 等待传输结束 */
    __HAL_DMA_DISABLE(&g_dma_dcmi_handle);      /* 关闭 DMA */
}
```

下面介绍的是 DCMI 中断服务函数（及其回调函数）、DCMI DMA 接收回调函数和 DMA1 数据流 1 中断服务函数，它们的定义分别如下：

```
/**
 * @brief       DCMI 中断服务函数
 * @param       无
 * @retval      无
 */
void DCMI_IRQHandler(void)
{
    HAL_DCMI_IRQHandler(&g_dcmi_handle);
}

/**
 * @brief       DCMI 中断回调服务函数
 * @param       hdcmi:DCMI 句柄
 * @note        捕获到一帧图像处理
 * @retval      无
 */
void HAL_DCMI_FrameEventCallback(DCMI_HandleTypeDef *hdcmi)
{
    __HAL_DCMI_CLEAR_FLAG(&g_dcmi_handle,DCMI_FLAG_FRAMERI);
    jpeg_data_process();      /* jpeg 数据处理 */
```

```
    LED1_TOGGLE();              /* LED1 闪烁 */
    g_ov_frame++;

    /* 重新使能帧中断,因为 HAL_DCMI_IRQHandler()函数会关闭帧中断 */
    __HAL_DCMI_ENABLE_IT(&g_dcmi_handle, DCMI_IT_FRAME);
}

void (*dcmi_rx_callback)(void); /* DCMI DMA 接收回调函数 */

/**
 * @brief       DMA1 数据流 1 中断服务函数(仅双缓冲模式会用到)
 * @param       无
 * @retval      无
 */
void DMA1_Stream1_IRQHandler(void)
{
    /* DMA 传输完成 */
    if (__HAL_DMA_GET_FLAG(&g_dma_dcmi_handle, DMA_FLAG_TCIF1_5) != RESET)
    {
        /* 清除 DMA 传输完成中断标志位 */
        __HAL_DMA_CLEAR_FLAG(&g_dma_dcmi_handle, DMA_FLAG_TCIF1_5);
        dcmi_rx_callback();     /* 执行摄像头接收回调函数,读取数据等操作在这里面处理 */
    }
}
```

其中：DCMI_IRQHandler 函数，用于处理帧中断，可以实现帧率统计（需要定时器支持）和 JPEG 数据处理等，实际上当捕获到一帧数据后，调用的是 HAL 库回调函数 HAL_DCMI_FrameEventCallback 进行处理。DMA1_Stream1_IRQHandler 函数，仅用于在使用 RGB 屏的时候，双缓冲存储时，数据的搬运处理（通过 dcmi_rx_callback 函数实现）。

最后还定义两个可以通过 usmart 调试、辅助测试使用的函数 dcmi_set_window 和 dcmi_cr_set 函数。dcmi_set_window 函数用于调节屏幕显示的范围，本实验 LCD 的起始坐标要设置为（0,0），LCD 显示范围要设置为屏幕最大像素点范围内。dcmi_cr_set 函数用于设置 pclk/hsync/vsync 这三个信号的有效电平。

DCMI 驱动代码就介绍到这里。

2. OV5640 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。OV5640 驱动源码包括六个文件：ov5640.c、ov5640.h、ov5640af.h、ov5640cfg.h、sccb.c 和 sccb.h。

其中 sccb.c 和 sccb.h 是 SCCB 通信接口的驱动代码。OV5640 的寄存器通过 SCCB 时序访问并设置，SCCB 时序和 IIC 时序十分类似，这里我们也是用软件模拟 SCCB 时序。

下面，首先介绍 sccb.h 头文件中 SCCB 的 IO 口宏定义，其定义情况如下：

```
/* 引脚 定义 */
#define SCCB_SCL_GPIO_PORT                  GPIOB
#define SCCB_SCL_GPIO_PIN                   GPIO_PIN_4
#define SCCB_SCL_GPIO_CLK_ENABLE()
            do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)   /* PB 口时钟使能 */

#define SCCB_SDA_GPIO_PORT                  GPIOB
#define SCCB_SDA_GPIO_PIN                   GPIO_PIN_3
#define SCCB_SDA_GPIO_CLK_ENABLE()
            do{ __HAL_RCC_GPIOB_CLK_ENABLE(); }while(0)   /* PB 口时钟使能 */

/* IO 操作函数 */
#define SCCB_SCL(x)     do{ x ? \
     HAL_GPIO_WritePin(SCCB_SCL_GPIO_PORT, SCCB_SCL_GPIO_PIN, GPIO_PIN_SET) : \
     HAL_GPIO_WritePin(SCCB_SCL_GPIO_PORT, SCCB_SCL_GPIO_PIN, GPIO_PIN_RESET);\
                        }while(0)       /* SCL */

#define SCCB_SDA(x)    do{ x ? \
     HAL_GPIO_WritePin(SCCB_SDA_GPIO_PORT, SCCB_SDA_GPIO_PIN, GPIO_PIN_SET) : \
```

```
    HAL_GPIO_WritePin(SCCB_SDA_GPIO_PORT, SCCB_SDA_GPIO_PIN, GPIO_PIN_RESET);\
                        }while(0)       /* SDA */

#define SCCB_READ_SDA     HAL_GPIO_ReadPin(SCCB_SDA_GPIO_PORT,
                                           SCCB_SDA_GPIO_PIN) /* 读取 SDA */
```

SCCB 时序有两根信号线（SCCB_SCL 和 SCCB_SDA），所以这里定义了两个 IO 口（PB4 和 PB3）来控制。IO 操作函数有三个，包括 SCCB_SCL 用于控制时钟，SCCB_SDA 是写 IO 口输出的值为逻辑 1 或者逻辑 0，SCCB_READ_SDA 是读取 IO 口的值，逻辑 1 或者逻辑 0。

接下来介绍 sccb.c 文件的代码，首先是 SCCB 接口初始化函数，该函数主要就是初始化 PB10 和 PB11 两个 IO 口，其定义如下：

```
/**
 * @brief       初始化 SCCB
 * @param       无
 * @retval      无
 */
void sccb_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;

    SCCB_SCL_GPIO_CLK_ENABLE();      /* SCL 引脚时钟使能 */
    SCCB_SDA_GPIO_CLK_ENABLE();      /* SDA 引脚时钟使能 */

    gpio_init_struct.Pin = SCCB_SCL_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;
    gpio_init_struct.Pull = GPIO_PULLUP;                    /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;     /* 高速 */
    HAL_GPIO_Init(SCCB_SCL_GPIO_PORT, &gpio_init_struct);  /* 初始化 SCL 引脚 */

    gpio_init_struct.Pin = SCCB_SDA_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_OD;
    HAL_GPIO_Init(SCCB_SDA_GPIO_PORT, &gpio_init_struct);    /* 初始化 SDA 引脚 */
}
```

SDA 引脚模式设置为带上拉的开漏输出，这样设置的好处是：SDA 引脚不用再设置 IO 口方向了，因为开漏输出模式下，STM32 的 IO 口可以读取外部信号的高低电平，这个在前面介绍 GPIO 外设的时候已经讲解过。初始化 IO 口后，调用 sccb_stop 函数停止总线上的所有设备，防止误操作。

sccb.c 文件的其他函数，这里就不再介绍了，主要都是 IO 口模拟 SCCB 时序的相关函数，详细请看源码。

ov5640.c、ov5640.h、ov5640af.h 和 ov5640cfg.h 这四个文件主要就是用于 OV5640 摄像头的初始化，当然也要使用到 SCCB 的驱动代码。

ov5640cfg.h 文件存放的是 OV5640 初始化数组，总共有 3 个数组。我们大概了解下数组结构，每个数组条目的第一个字节为寄存器号（也就是寄存器地址），第二个字节为要设置的值，比如{0x4300, 0x30}，就表示在 0x4300 地址，写入 0X30 这个值。

三个数组中，ov5640_init_reg_tbl 数组，用于初始化 OV5640，该数组必须最先进行配置；ov5640_rgb565_reg_tbl 数组，用于设置 OV5640 的输出格式为 RGB565，分辨率为 1280*800，帧率为 15 帧，在 RGB 模式下使用；OV5640_jpeg_reg_tbl 数组用于设置 OV5640 的输出格式为 JPEG，分辨率为 2592*1944，帧率为 7.5 帧，在 JPEG 模式下使用。由于数组的篇幅过长，就不列出来了，请对照源码理解。

ov5640af.h 文件存放的是 OV5640 自动对焦配置数组，前面介绍过。

下面开始介绍 ov5640.h 文件，主要是 OV5640 的 PWDN 和 RESET 引脚定义和控制函数，以及 OV5640 的 ID、访问地址和相关寄存器定义，其定义如下：

```
/* RESET 引脚 定义 */
#define OV_RESET_GPIO_PORT                  GPIOA
#define OV_RESET_GPIO_PIN                   GPIO_PIN_15
#define OV_RESET_GPIO_CLK_ENABLE()
do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)    /* PA 口时钟使能 */
```

```
/* IO 控制函数 */
#define OV5640_RST(x)      do{ x ? \
    HAL_GPIO_WritePin(OV_RESET_GPIO_PORT, OV_RESET_GPIO_PIN, GPIO_PIN_SET) : \
    HAL_GPIO_WritePin(OV_RESET_GPIO_PORT, OV_RESET_GPIO_PIN, GPIO_PIN_RESET); \
                          }while(0)       /* 复位控制信号 */


/* OV5640 的 ID 和访问地址 */
#define OV5640_ID           0X5640         /* OV5640 的芯片 ID */
#define OV5640_ADDR         0X78           /* OV5640 的 IIC 地址 */
/* OV5640 相关寄存器定义 */
#define OV5640_CHIPIDH      0X300A         /* OV5640 芯片 ID 高字节 */
#define OV5640_CHIPIDL      0X300B         /* OV5640 芯片 ID 低字节 */
```

下面开始介绍 ov5640.c 文件，首先是 OV5640 初始化函数，其定义如下：

```
/**
 * @brief       初始化 OV5640
 * @note        配置完以后,默认输出是 1600*1200 尺寸的图片!!
 * @param       无
 * @retval      0,成功
 * @retval      其他,错误代码
 */
uint8_t ov5640_init(void)
{
    GPIO_InitTypeDef gpio_init_struct;

    uint16_t i = 0;
    uint16_t reg;

    OV_RESET_GPIO_CLK_ENABLE();     /* 使能 OV_RESET 脚时钟 */

    gpio_init_struct.Pin = OV_RESET_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;          /* 推挽输出 */
    gpio_init_struct.Pull = GPIO_PULLUP;                  /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_HIGH;             /* 高速 */
    HAL_GPIO_Init(OV_RESET_GPIO_PORT, &gpio_init_struct);/* 初始化 OV_PWDN 引脚 */

    pcf8574_init();
    OV5640_RST(0);      /* 必须先拉低 OV5640 的 RST 脚,再上电 */
    delay_ms(20);
    OV5640_PWDN(0);     /* POWER ON */
    delay_ms(5);
    OV5640_RST(1);      /* 结束复位 */
    delay_ms(20);
    sccb_init();        /* 初始化 SCCB 的 IO 口 */
    delay_ms(5);
    reg = ov5640_read_reg(OV5640_CHIPIDH);  /* 读取 ID 高八位 */
    reg <<= 8;
    reg |= ov5640_read_reg(OV5640_CHIPIDL); /* 读取 ID 低八位 */
    if (reg != OV5640_ID)   /* ID 是否正常? */
    {
        printf("ID:%d\r\n", reg);
        return 1;           /* 失败 */
    }
    ov5640_write_reg(0x3103, 0X11); /* system clock from pad, bit[1] */
    ov5640_write_reg(0X3008, 0X82); /* 软复位 */
    delay_ms(10);

    /* 初始化 OV5640 */
    for (i = 0; i < sizeof(ov5640_init_reg_tbl) / 4; i++)
    {
        ov5640_write_reg(ov5640_init_reg_tbl[i][0], ov5640_init_reg_tbl[i][1]);
    }
```

```
    /* 检查闪光灯是否正常 */
    ov5640_flash_ctrl(1);       /* 打开闪光灯 */
    delay_ms(50);
    ov5640_flash_ctrl(0);       /* 关闭闪光灯 */
    return 0;                   /* ok */
}
```

该函数除了初始化 OV5640 相关的 IO 口，最主要的是完成 OV5640 的寄存器序列初始化。OV5640 的寄存器特多（百几十个），配置特麻烦，幸好厂家有提供参考配置序列，就存放在上面介绍的 ov5640_ init_reg_tbl 这个数组里面。

另外，在 ov5640.c 里面，还有几个函数比较重要，这里不贴代码了，只介绍功能：

ov5640_outsize_set 函数，用于设置图像输出大小；

ov5640_image_window_set 函数，用于设置图像开窗大小(ISP 大小)；

ov5640_focus_init 函数，用于初始化自动对焦功能；

ov5640_focus_single 函数，用于实现一次自动对焦；

ov5640_focus_constant 函数，用于开启持续自动对焦功能；

其中，ov5640_outsize_set 和 ov5640_image_window_set 函数就是前面在 51.1.1 节所介绍的 3 个窗口的设置，他们共同决定了图像的输出。

其他的函数就不列出来了，请大家查看源码。

3. TIMER 驱动代码

TIMER 驱动代码我们主要就是用到定时器 6，在定时器 6 更新中断回调函数中打印帧率，更新中断回调函数定义如下：

```
/**
 * @brief       定时器更新中断回调函数
 * @param       htim:定时器句柄指针
 * @retval      无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim ->Instance== BTIM_TIMX_INT)
    {
        printf("frame:%d\r\n", g_ov_frame);
        g_ov_frame = 0;
    }
}
```

g_ov_frame 变量在 dcmi.c 中被定义，用于存放帧率。TIMER 的其他驱动代码在介绍定时器的时候已经介绍过，请看源码。

4. USART2 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。USART2 驱动源码包括两个文件：usart2.c 和 usart2.h。

usart2.h 头文件对 usart2 的 IO 口做了宏定义，具体如下：

```
/* 串口 2 引脚 定义 */
#define USART2_TX_GPIO_PORT                 GPIOA
#define USART2_TX_GPIO_PIN                  GPIO_PIN_2
#define USART2_TX_GPIO_AF                   GPIO_AF7_USART2   /* AF 功能选择 */
#define USART2_TX_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)     /* PA 口时钟使能 */

#define USART2_RX_GPIO_PORT                 GPIOA
#define USART2_RX_GPIO_PIN                  GPIO_PIN_3
#define USART2_RX_GPIO_AF                   GPIO_AF7_USART2   /* AF 功能选择 */
#define USART2_RX_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOA_CLK_ENABLE(); }while(0)     /* PA 口时钟使能 */
```

PA2 和 PA3 复用为串口 2 的发送和接收引脚。

下面介绍 usart2.c 文件的串口 2 初始化函数，其定义如下：

```
/**
 * @brief       串口 2 初始化函数
 * @param       baudrate: 波特率, 根据自己需要设置波特率值
 * @note        注意: 必须设置正确的时钟源, 否则串口波特率就会设置异常.
 *              这里的 USART 的时钟源在 sys_stm32_clock_init()函数中已经设置过了.
 * @retval      无
 */
void usart2_init(uint32_t baudrate)
{
    GPIO_InitTypeDef gpio_init_struct;

    USART2_UX_CLK_ENABLE();             /* 使能 USART2 时钟 */
    USART2_TX_GPIO_CLK_ENABLE();        /* 使能串口 TX 脚时钟 */
    USART2_RX_GPIO_CLK_ENABLE();        /* 使能串口 RX 脚时钟 */

    gpio_init_struct.Pin = USART2_TX_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;                /* 复用推挽 */
    gpio_init_struct.Pull = GPIO_PULLUP;                    /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;     /* 高速 */
    gpio_init_struct.Alternate = USART2_TX_GPIO_AF;         /* 复用为 USART2 */
    HAL_GPIO_Init(USART2_TX_GPIO_PORT, &gpio_init_struct); /* 初始化串口 TX 引脚 */

    gpio_init_struct.Pin = USART2_RX_GPIO_PIN;
    gpio_init_struct.Alternate = USART2_RX_GPIO_AF;         /* 复用为 USART2 */
    HAL_GPIO_Init(USART2_RX_GPIO_PORT, &gpio_init_struct);/* 初始化串口 RX 引脚 */

    g_uart2_handle.Instance = USART2;                       /* USART2 */
    g_uart2_handle.Init.BaudRate = baudrate;                /* 波特率 */
    g_uart2_handle.Init.WordLength = UART_WORDLENGTH_8B;    /* 字长为 8 位数据格式 */
    g_uart2_handle.Init.StopBits = UART_STOPBITS_1;         /* 一个停止位 */
    g_uart2_handle.Init.Parity = UART_PARITY_NONE;          /* 无奇偶校验位 */
    g_uart2_handle.Init.HwFlowCtl = UART_HWCONTROL_NONE;    /* 无硬件流控 */
    g_uart2_handle.Init.Mode = UART_MODE_TX;                /* 发模式 */
    HAL_UART_Init(&g_uart2_handle);                         /* 使能 USART2 */
}
```

这里需要注意的是，串口 2 的时钟源频率在 sys_stm32_clock_init 函数中已经设置了。其他的代码基本 usart_init 函数是一样的。

5. main.c 代码

main.c 前面定义了一些变量和数组，具体如下：

```
uint8_t g_ov_mode = 0;              /* bit0: 0,RGB565 模式;  1,JPEG 模式 */
uint16_t g_curline = 0;             /* 摄像头输出数据,当前行编号 */
uint16_t g_yoffset = 0;             /* y 方向的偏移量 */

#define jpeg_buf_size   1*1024*1024/* 定义 JPEG 数据缓存 jpeg_buf 的大小*/
#define jpeg_line_size   4*1024     /* 定义 DMA 接收数据时,一行数据的最大值 */

/* JPEG 数据采集,摄像头采用一行一行读取,定义行缓存 */
uint32_t g_dcmi_line_buf[2][jpeg_line_size]
                        __attribute__((at(0XC0000000 + 1280 * 800 * 2)));
uint32_t g_jpeg_data_buf[jpeg_buf_size];/* JPEG 数据缓存 buf,定义在内部 SRAM */

/* buf 中的 JPEG 有效数据长度 */
volatile uint32_t g_jpeg_data_len = 0;

/* JPEG 数据采集完成标志
 * 0,数据没有采集完;
 * 1,数据采集完了,但是还没处理;
```

```
 * 2,数据已经处理完成了,可以开始下一帧接收
 */
volatile uint8_t g_jpeg_data_ok = 0;

/* JPEG 尺寸支持列表 */
const uint16_t jpeg_img_size_tbl[][2] =
{
    160, 120,       /* QQVGA */
    320, 240,       /* QVGA */
    640, 480,       /* VGA */
    800, 600,       /* SVGA */
    1024, 768,      /* XGA */
    1280, 800,      /* WXGA */
    1440, 900,      /* WXGA+ */
    1280, 1024,     /* SXGA */
    1600, 1200,     /* UXGA */
    1920, 1080,     /* 1080P */
    2048, 1536,     /* QXGA */
    2592, 1944,     /* 500W */
    };

const char *EFFECTS_TBL[7] = {"Normal", "Cool", "Warm", "B&W", "Yellowish ",
                              "Inverse", "Greenish"};     /* 7 种特效 */
const char *JPEG_SIZE_TBL[12] =
{"QQVGA","QVGA","VGA","SVGA","XGA","WXGA","WXGA+","SXGA","UXGA","1080P","QXGA",
"500W"}; /* JPEG 图片 12 种尺寸 */
```

其中，定义的 g_jpeg_data_buf 数组大小为 1MB 字节，用来存储 JPEG 数据。g_dcmi_line_buf 数组，我们定义数组的起始地址为 0xC01F4000，由__attribute__关键字，指定数组地址。

在 main.c 里面，总共有 5 个函数，我们接下来分别介绍。首先是处理 JPEG 数据函数，其定义如下：

```
/**
 * @brief       处理 JPEG 数据
 * @ntoe        在 DCMI_IRQHandler 中断服务函数里面被调用
 *              当采集完一帧 JPEG 数据后,调用此函数,切换 JPEG BUF.开始下一帧采集.
 *
 * @param       无
 * @retval      无
 */
void jpeg_data_process(void)
{
    uint16_t i;
    uint16_t rlen;          /* 剩余数据长度 */
    uint32_t *pbuf;
    g_curline = g_yoffset;  /* 行数复位 */
    if (g_ov_mode & 0X01)   /* 只有在 JPEG 格式下,才需要做处理. */
    {
        if (g_jpeg_data_ok == 0)    /* jpeg 数据还未采集完? */
        {
            __HAL_DMA_DISABLE(&g_dma_dcmi_handle);      /* 关闭 DMA */
            /* 得到剩余数据长度 */
            rlen = jpeg_line_size - __HAL_DMA_GET_COUNTER(&g_dma_dcmi_handle);
            pbuf =g_jpeg_data_buf+ g_jpeg_data_len;/* 偏移到有效数据末尾,继续添加 */
            if (DMA1_Stream1->CR & (1 << 19))
            {
                for (i = 0; i < rlen; i++)
                {
                    pbuf[i] = g_dcmi_line_buf[1][i]; /* 读取 buf1 里面的剩余数据 */
                }
            }
            else
            {
```

```
                for (i = 0; i < rlen; i++)
                {
                    pbuf[i] = g_dcmi_line_buf[0][i]; /* 读取 buf0 里面的剩余数据 */
                }
            }
            g_jpeg_data_len += rlen;     /* 加上剩余长度 */
            g_jpeg_data_ok = 1;          /* 标记 JPEG 数据采集完成,等待其他函数处理 */
        }
        if (g_jpeg_data_ok == 2)         /* 上一次的 jpeg 数据已经被处理了 */
        {
            /* 传输长度为 jpeg_buf_size*4 字节 */
            __HAL_DMA_SET_COUNTER(&g_dma_dcmi_handle, jpeg_line_size);
            __HAL_DMA_ENABLE(&g_dma_dcmi_handle);      /* 重新传输 */
            g_jpeg_data_ok = 0;                        /* 标记数据未采集 */
            g_jpeg_data_len = 0;                       /* 数据重新开始 */
        }
    }
    else
    {
      if (lcdltdc.pwidth==0)
      {
        lcd_set_cursor(0, 0);
        lcd_write_ram_prepare();         /* 开始写入 GRAM */
      }
    }
}
```

该函数用于处理 JPEG 数据的接收，在 DCMI_IRQHandler 函数（在 dcmi.c 里面）里面被调用，它与 jpeg_dcmi_rx_callback 函数和 jpeg_test 函数共同控制 JPEG 的数据传送。JPEG 数据的接收，采用 DMA 双缓冲机制，缓冲数组为：dcmi_line_buf（u32 类型，RGB 屏接收 RGB565 数据时，也是用这个数组）；数组大小为：jpeg_line_size，我们定义的是 4*1024，即数组大小为 16K 字节（数组大小不能小于存储摄像头一行输出数据的大小）；JPEG 数据接收处理流程如图 51.3.3.1 所示：

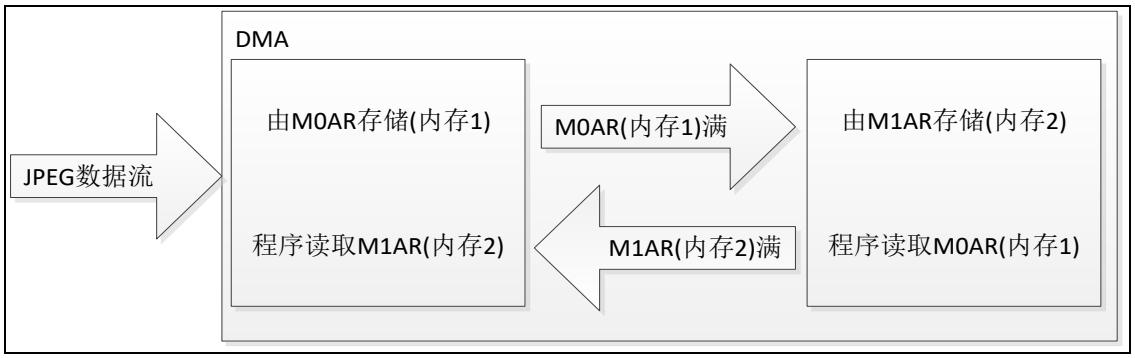


图 51.3.3.1 JPEG 数据流 DMA 双缓冲接收流程

JPEG 数据采集流程：当 JPEG 数据流传输给 MCU 的时候，首先由 M0AR 存储，此时如果 M1AR 有数据，则可以读取 M1AR 里面的数据，当 M0AR 数据满时，由 M1AR 存储，此时程序可以读取 M0AR 里面所存储的数据，当 M1AR 数据满时，由 M0AR 存储……。这个存储数据的操作，绝大部分是由 DMA 传输完成中断服务函数，调用 jpeg_dcmi_rx_callback 函数实现的，当一帧数据传输完成时，会进入 DCMI 帧中断服务函数，调用 jpeg_data_process 函数，对最后的剩余数据进行存储，完成一帧 JPEG 数据的采集。

接下来介绍的是 JPEG 数据接收回调函数，其定义如下：

```
/**
 * @brief       JPEG 数据接收回调函数
 * @ntoe        在 DMA1_Stream1_IRQHandler 中断服务函数里面被调用
 *
 * @param       无
 * @retval      无
 */
```

```
void jpeg_dcmi_rx_callback(void)
{
    uint16_t i;
    uint32_t *pbuf;
    pbuf = g_jpeg_data_buf + g_jpeg_data_len;       /* 偏移到有效数据末尾 */

    if (DMA1_Stream1->CR & (1 << 19))               /* buf0 已满,正常处理 buf1 */
    {
        for (i = 0; i < jpeg_line_size; i++)
        {
            pbuf[i] = g_dcmi_line_buf[0][i];        /* 读取 buf0 里面的数据 */
        }

        g_jpeg_data_len += jpeg_line_size;          /* 偏移 */
    }
    else    /* buf1 已满,正常处理 buf0 */
    {
        for (i = 0; i < jpeg_line_size; i++)
        {
            pbuf[i] = g_dcmi_line_buf[1][i];        /* 读取 buf1 里面的数据 */
        }

        g_jpeg_data_len += jpeg_line_size;          /* 偏移 */
    }
    SCB_CleanInvalidateDCache();                    /* 清除无效化 DCache */
}
```

该函数是 jpeg 数据接收的主要函数，通过判断 DMA1_Stream1->CR 寄存器，读取不同 buf 里面的数据，存储到 g_jpeg_data_buf 里面。该函数由 DMA 的传输完成中断服务函数：DMA1_Stream1_IRQHandler 调用。

接下来介绍的是 JPEG 测试函数，其定义如下：

```
/**
 * @brief       JPEG 测试
 * @ntoe        JPEG 数据,通过串口 2 发送给电脑.
 *
 * @param       无
 * @retval      无
 */
void jpeg_test(void)
{
    uint8_t key;
    uint8_t *p, headok = 0;
    uint32_t i, jpgstart, jpglen;
    uint8_t effect = 0,contrast=2;
    uint8_t msgbuf[15];     /* 消息缓存区 */
    uint8_t size = 1;       /* 默认是 QVGA 320*240 尺寸 */

    lcd_clear(WHITE);
    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "OV5640 JPEG Mode", RED);
    lcd_show_string(30, 100, 200, 16, 16,"KEY0:Contrast", RED);  /* 对比度 */
    lcd_show_string(30, 120, 200, 16, 16,"KEY1: AUTO Focus", RED);/* 自动对焦 */
    lcd_show_string(30, 140, 200, 16, 16, "KEY2:Effects", RED);  /* 分辨率设置 */
    lcd_show_string(30, 160, 200, 16, 16, "KEY_UP:Size", RED);
    sprintf((char *)msgbuf, "JPEG Size:%s", JPEG_SIZE_TBL[size]);
    /* 显示当前 JPEG 分辨率 */
    lcd_show_string(30, 180, 200, 16, 16, (char*)msgbuf, RED);

    /* 自动对焦初始化 */
    ov5640_rgb565_mode();           /* RGB565 模式 */
    ov5640_focus_init();
    ov5640_jpeg_mode();             /* JPEG 模式 */
    ov5640_light_mode(0);           /* 自动模式 */
```

```
    ov5640_color_saturation(3);     /* 色彩饱和度 0 */
    ov5640_brightness(4);           /* 亮度 0 */
    ov5640_contrast(3);             /* 对比度 0 */
    ov5640_sharpness(33);           /* 自动锐度 */
    ov5640_focus_constant();        /* 启动持续对焦 */
    dcmi_init();                    /* DCMI 配置 */
    dcmi_rx_callback = jpeg_dcmi_rx_callback; /* JPEG 接收数据回调函数 */
    dcmi_dma_init((uint32_t)&g_dcmi_line_buf[0], (uint32_t)&g_dcmi_line_buf[1],
       jpeg_line_size, DMA_MDATAALIGN_WORD, DMA_MINC_ENABLE); /* DCMI DMA 配置 */
    ov5640_outsize_set(4, 0, jpeg_img_size_tbl[size][0],
                       jpeg_img_size_tbl[size][1]); /* 设置输出尺寸 */
    dcmi_start();                   /* 启动传输 */
    while (1)
    {
        if (g_jpeg_data_ok == 1)    /* 已经采集完一帧图像了 */
        {
            p = (uint8_t *)g_jpeg_data_buf;
            printf("g_jpeg_data_len:%d\r\n", g_jpeg_data_len * 4); /* 打印帧率 */
            /* 提示正在传输数据 */
            lcd_show_string(30, 210, 210, 16, 16, "Sending JPEG data...", RED);
            jpglen = 0;             /* 设置 jpg 文件大小为 0 */
            headok = 0;             /* 清除 jpg 头标记 */
            /* 查找 0XFF,0XD8 和 0XFF,0XD9,获取 jpg 文件大小 */
            for (i = 0; i < g_jpeg_data_len * 4; i++)
            {
                if ((p[i] == 0XFF) && (p[i + 1] == 0XD8))   /* 找到 FF D8 */
                {
                    jpgstart = i;
                    headok = 1; /* 标记找到 jpg 头(FF D8) */
                }
                /* 找到头以后,再找 FF D9 */
                if ((p[i] == 0XFF) && (p[i + 1] == 0XD9) && headok)
                {
                    jpglen = i - jpgstart + 2;
                    break;
                }
            }
            if (jpglen)             /* 正常的 jpeg 数据 */
            {
                p += jpgstart;  /* 偏移到 0XFF,0XD8 处 */
                for (i = 0; i < jpglen; i++)    /* 发送整个 jpg 文件 */
                {
                    USART2->TDR = p[i];
                    while ((USART2->ISR & 0X40) == 0);  /* 循环发送,直到发送完毕 */
                    key = key_scan(0);
                    if (key)break;
                }
            }
            if (key)    /* 有按键按下,需要处理 */
            {
                lcd_show_string(30, 210, 210, 16, 16, "Quit Sending data   ",
                                RED); /* 提示退出数据传输 */
                switch (key)
                {
                   case KEY0_PRES: /* 对比度设置 */
                    contrast++;
                    if(contrast>6)contrast=0;
                    ov5640_contrast(contrast);
                    sprintf((char *)msgbuf, "Contrast:%d",
                              (signed char)contrast-3);
                         break;
```

```
                    case KEY1_PRES: /* 执行一次自动对焦 */
                        ov5640_focus_single();
                        break;

                    case KEY2_PRES: /* 特效设置 */
                        effect++;
                        if(effect>6)effect=0;
                        ov5640_special_effects(effect);
                        sprintf((char*)msgbuf,"%s",EFFECTS_TBL[effect]);
                        break;

                    case WKUP_PRES: /* JPEG 输出尺寸设置 */
                        size++;
                        /* 最大只支持 WXGA 的 jpeg 数据保存，再大分辨率就不够内存用了 */
                        if (size > 11)size = 0;

                        ov5640_outsize_set(16, 4, jpeg_img_size_tbl[size][0],
                                jpeg_img_size_tbl[size][1]);  /* 设置输出尺寸 */
                        sprintf((char *)msgbuf, "JPEG Size:%s",
                            JPEG_SIZE_TBL[size]);
                        break;
                }
                lcd_fill(30, 180, 239, 190 + 16, WHITE);
                /* 显示提示内容 */
                lcd_show_string(30, 180, 210, 16, 16, (char*)msgbuf, RED);
                delay_ms(800);
            }
            else
            {
            lcd_show_string(30, 210, 210, 16, 16, "Send data complete!!",RED);
            /* 提示传输结束设置 */
            }
            g_jpeg_data_ok = 2; /* 标记 jpeg 数据处理完了,可以让 DMA 去采集下一帧了. */
        }
    }
}
```

该函数将 OV5640 设置为 JPEG 模式，并开启持续自动对焦、实现 OV5640 的 JPEG 数据接收，通过串口 2 发送给上位机软件。

接下来介绍的是 RGB565 测试函数，其定义如下：

```
/**
 * @brief       RGB565 测试
 * @ntoe        RGB 数据直接显示在 LCD 上面
 *
 * @param       无
 * @retval      无
 */
void rgb565_test(void)
{
    uint8_t key;
    float fac = 0;
    uint8_t effect = 0, contrast = 2;
    uint8_t scale = 1;                                      /* 默认是全尺寸缩放 */
    uint8_t msgbuf[15];                                     /* 消息缓存区 */
    uint16_t outputheight = 0;

    lcd_clear(WHITE);
    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "OV5640 RGB565 Mode", RED);
    lcd_show_string(30, 100, 200, 16, 16, "KEY0:Contrast", RED); /* 对比度 */
    lcd_show_string(30, 120, 200, 16,16,"KEY1:AUTO Focus",RED);/* 执行自动对焦 */
    lcd_show_string(30, 140, 200, 16, 16, "KEY2:Effects", RED);/* 特效 */
    lcd_show_string(30, 160, 200, 16, 16, "KEY_UP:FullSize/Scale", RED);
    /* 1:1 尺寸(显示真实尺寸)/全尺寸缩放 */
```

```
    /* 自动对焦初始化 */
    ov5640_rgb565_mode();                                   /* RGB565 模式 */
    ov5640_focus_init();
    ov5640_light_mode(0);                                   /* 自动模式 */
    ov5640_color_saturation(3);                             /* 色彩饱和度 0  */
    ov5640_brightness(4);                                   /* 亮度 0 */
    ov5640_contrast(3);                                     /* 对比度 0 */
    ov5640_sharpness(33);                                   /* 自动锐度 */
    ov5640_focus_constant();                                /* 启动持续对焦 */
    dcmi_init();                                            /* DCMI 配置 */

    if (lcdltdc.pwidth != 0)                                /* RGB 屏 */
    {
        dcmi_rx_callback = rgblcd_dcmi_rx_callback;  /* RGB 屏接收数据回调函数 */
        dcmi_dma_init((uint32_t)g_dcmi_line_buf[0],
        (uint32_t)g_dcmi_line_buf[1], lcddev.width / 2,
        DMA_MDATAALIGN_HALFWORD, DMA_MINC_ENABLE);
        TIM3->CR1&=~(0x01);
        /* 关闭定时器 3,关闭帧率统计，打开的话，RGB 屏，在串口打印的时候，会抖 */
    }
    else                                                    /* MCU 屏 */
    {
        dcmi_dma_init((uint32_t)&LCD->LCD_RAM, 0, 1,
                      DMA_MDATAALIGN_HALFWORD, DMA_MINC_DISABLE);
    }

    if (lcddev.height == 1024)
    {
        g_yoffset = (lcddev.height - 800) / 2;
        outputheight = 800;
        ov5640_write_reg(0x3035, 0X51);         /* 降低输出帧率，否则可能抖动 */
    }
    else if (lcddev.height == 1280)
    {
        g_yoffset = (lcddev.height - 600) / 2;
        outputheight = 600;
        ov5640_write_reg(0x3035, 0X51);         /* 降低输出帧率，否则可能抖动 */
    }
    else
    {
        g_yoffset = 0;
        outputheight = lcddev.height;
    }

    g_curline = g_yoffset;                                  /* 行数复位 */
    ov5640_outsize_set(4, 0, lcddev.width, outputheight);/* 满屏缩放显示 */
    dcmi_start();                                           /* 启动传输 */
    lcd_clear(0xFFFF);
    while(1)
    {
        key = key_scan(0);
        if (key)
        {
            if (key != KEY1_PRES)dcmi_stop(); /* 非 KEY1 按下,停止显示 */
            switch (key)
            {
             case KEY0_PRES:                    /* 对比度设置 */
             contrast++;
             if (contrast > 6)contrast = 0;
             ov5640_contrast(contrast);
             sprintf((char *)msgbuf, "Contrast:%d", (signed char)contrast - 3);
                    break;
```

```
                case KEY1_PRES:                 /* 执行一次自动对焦 */
                    ov5640_focus_single();
                    break;

                case KEY2_PRES:                 /* 特效设置 */
                    effect++;
                    if (effect > 6)effect = 0;
                    ov5640_special_effects(effect); /* 设置特效 */
                    sprintf((char *)msgbuf, "%s", EFFECTS_TBL[effect]);
                    break;

                case WKUP_PRES:                 /* 1:1 尺寸(显示真实尺寸)/缩放 */
                    scale =! scale;
                    if (scale == 0)
                    {
                        fac = 800 / outputheight;           /* 得到比例因子 */
                        ov5640_outsize_set((1280 - fac * lcddev.width) / 2,
                    (800 - fac * outputheight) / 2, lcddev.width, outputheight);
                        sprintf((char *)msgbuf, "Full Size 1:1");
                    }
                    else
                    {
                        ov5640_outsize_set(4, 0, lcddev.width, outputheight);
                        sprintf((char *)msgbuf, "Scale");
                    }
                    break;
            }
            if (key != KEY1_PRES)                       /* 非 KEY1 按下 */
            {
                lcd_show_string(30, 50, 210, 16, 16, (char *)msgbuf, RED);
                /* 显示提示内容  */
                delay_ms(800);
                dcmi_start();                           /* 重新开始传输 */
            }
        }
        delay_ms(10);
    }
}
```

该函数将 OV5640 设置为 RGB565 模式，并将接收到的数据，传送给 LCD。当使用 MCU 屏的时候，完全由硬件 DMA 传输给 LCD，CPU 不用处理。

接下来介绍的是 main 函数，其定义如下：

```
int main(void)
{
    uint8_t key = 0;
    uint16_t t = 0;

    sys_cache_enable();                             /* 打开 L1-Cache */
    HAL_Init();                                     /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);             /* 设置时钟, 400Mhz */
    delay_init(400);                                /* 延时初始化 */
    usart_init(115200);                             /* 串口初始化 */
    usart2_init(921600);                            /* 初始化串口 2 波特率为 921600 */
    usmart_init(200);                               /* 初始化 USMART */
    mpu_memory_protection();                        /* 保护相关存储区域 */
    led_init();                                     /* 初始化 LED */
    key_init();                                     /* 初始化 KEY */
    sdram_init();                                   /* 初始化 SDRAM */
    lcd_init();                                     /* 初始化 LCD */
    btim_timx_int_init(10000 - 1, 20000 - 1);       /* 10KHz 计数频率，1 秒中断一次 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "OV5640 TEST", RED);
```

```
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    while (ov5640_init())           /* 初始化 OV5640 */
    {
        lcd_show_string(30, 130, 240, 16, 16, "OV5640 ERROR", RED);
        delay_ms(200);
        lcd_fill(30, 130, 239, 170, WHITE);
        delay_ms(200);
        LED0_TOGGLE();
    }

    lcd_show_string(30, 130, 200, 16, 16, "OV5640 OK", RED);
    while (1)
    {
        key = key_scan(0);

        if (key == KEY0_PRES)
        {
            g_ov_mode = 0;              /* RGB565 模式 */
            break;
        }
        else if (key == KEY1_PRES)
        {
            g_ov_mode = 1;              /* JPEG 模式 */
            break;
        }
        t++;
        if (t == 100)
        {
         lcd_show_string(30, 150, 230, 16, 16, "KEY0:RGB565  KEY1:JPEG", RED);
         /* 闪烁显示提示信息 */
        }

        if (t == 200)
        {
            lcd_fill(30, 150, 210, 150 + 16, WHITE);
            t = 0;
            LED0_TOGGLE();
        }
        delay_ms(5);
    }

    if (g_ov_mode == 1)
    {
        jpeg_test();
    }
    else
    {
        rgb565_test();
    }
}
```

该函数完成对各相关硬件的初始化，然后检测 OV5640，最后通过按键选择来调用 jpeg_test 还是 rgb565_test，实现 JPEG 测试和 RGB565 测试。

前面提到，我们要用 USMART 来设置摄像头的参数，我们只需要在 usmart_nametab 里面添加 ov5640_write_reg 和 ov5640_read_reg 等相关函数，就可以轻松调试摄像头了。

51.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 51.4.1 所示：

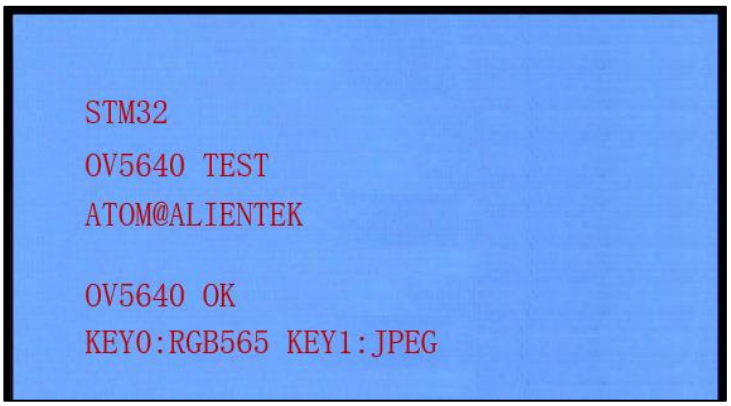


图 51.4.1 摄像头实验程序运行效果图

上图是摄像头已经正确的插上开发板上的运行结果，如果没有插上摄像头，会有错误的报错信息。在 OV5640 初始化成功后，屏幕提示选择测试模式，此时我们可以按 KEY0，进入 RGB565 模式测试，也可以按 KEY1，进入 JPEG 模式测试。

当按 KEY0 后，选择 RGB565 模式，LCD 满屏显示压缩放后的图像（有变形），如图 51.4.1 所示：

图 51.4.1 RGB565 模式测试图片

此时，可以按 KEY_UP 切换为 1:1 显示（不变形）。同时还可以通过 KEY0 按键，设置对比度；KEY1 按键，执行一次自动对焦；KEY2 按键，设置特效。

当按 KEY1 后，选择 JPEG 模式，此时屏幕显示 JPEG 数据传输进程，如图 51.4.2 所示：

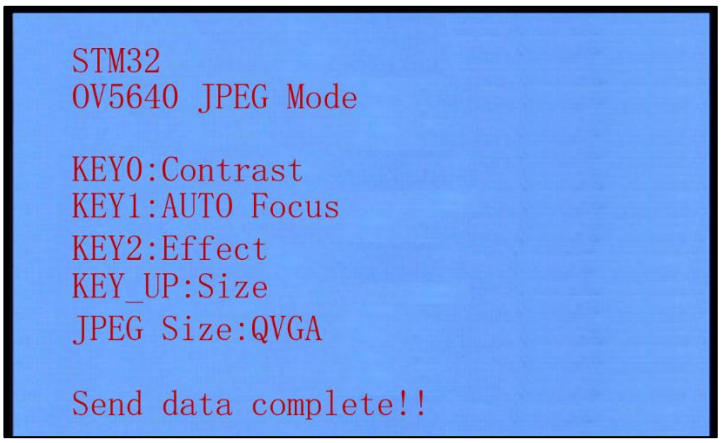


图 51.4.2 JPEG 模式测试图

默认条件下，图像分辨率是 QVGA (320*240)的，硬件上：我们需要用一根杜邦线连接开发板的 PA2 排针到 RXD 排针端。

打开上位机软件：XCAM V1.0.exe（路径：光盘→6，软件资料→1，软件→5，其他软件→串口&网络摄像头软件→ XCAM V1.0.exe），选择正确的串口，然后波特率设置为 921600，打开即可收到下位机传过来的图片了，如图 51.4.3 所示：

图 51.4.3 XCAM V1.0.exe 软件接收并显示 JPEG 图片

我们可以通过 KEY_UP 设置输出图像的尺寸（QQVGA~WXGA）。通过 KEY0 按键，设置对比度；KEY1 按键，执行一次自动对焦；KEY2 按键，设置特效。

同时，你还可以在串口（开发板的串口 1），通过 USMART 调用 ov5640_write_reg 等函数，来设置 OV5640 的各寄存器，达到调试测试 OV5640 的目的，如图 51.4.4 所示：

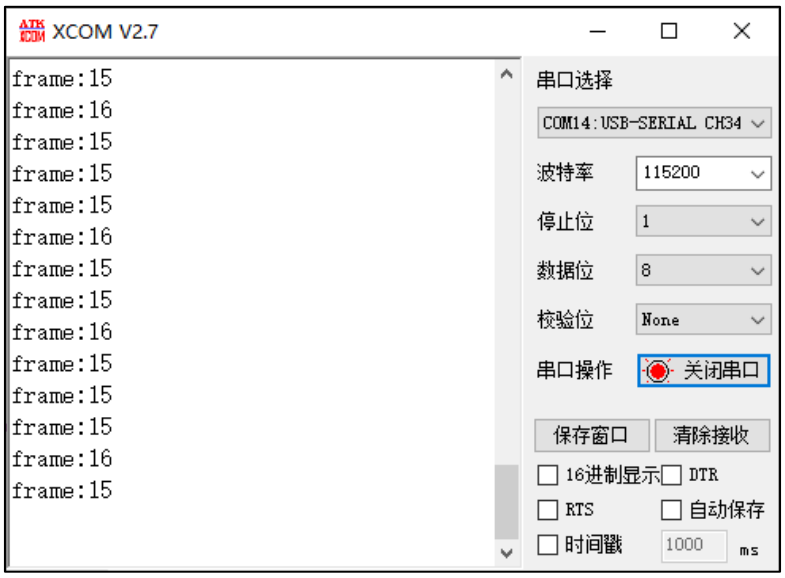


图 51.4.4 USMART 调试 OV5640

从上图还可以看出，帧率为 15/16 帧，每张 JPEG 图片的大小是 225KB 左右（分辨率为：320*240 的时候）。

第三篇 提高篇

现在我们已经学会如何使用 STM32 的一些基础外设了，STM32 我们算是入门了。接下来我们将学习提高篇的内容，本篇大部分内容都和 STM32 硬件底层关系不是很大，主要以各种第三方组件代码为主，比如：内存管理、文件系统、图片解码库、字库、手写识别库、T9 输入法、USB 库、网络协议栈、OS 等。这些内容偏向较复杂应用，掌握起来难度较大，需要大家认真学习，多多练习，做到举一反三。

本篇同样使用一章一实例的方式给大家介绍 STM32 的各种高级应用，通过本篇的学习，我们将可以完成更复杂的 STM32 项目开发。

第五十二章 内存管理实验

如果我们所用的内存都是直接定义一个数组来使用，灵活性会比较差，很多时候不能满足实际使用需求。为了解决这些问题，我们来学习内存管理，实现对内存的动态管理。

本章分为如下几个小节：

52.1 内存管理简介
52.2 硬件设计
52.3 程序设计
52.4 下载验证

52.1 内存管理简介

内存管理，是指软件运行时对计算机内存资源的分配和使用的技术。其最主要的目的是如何高效、快速的分配，并且在适当的时候释放和回收内存资源。内存管理的实现方法有很多种，其实最终都是要实现两个函数：malloc 和 free。malloc 函数用来内存申请，free 函数用于内存释放。

本章，我们介绍一种比较简单的办法来实现：分块式内存管理。下面我们介绍一下该方法的实现原理，如图 52.1.1 所示：

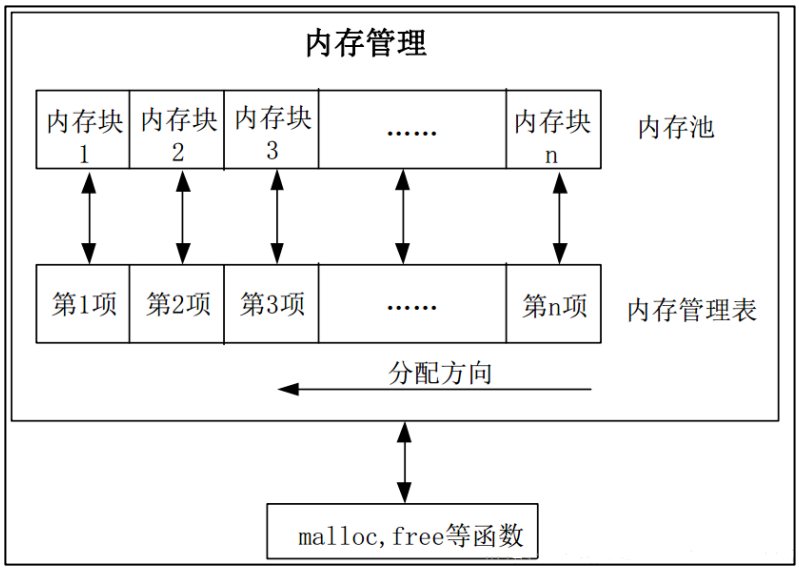


图 52.1.1 分块式内存管理原理

从上图可以看出，分块式内存管理由内存池和内存管理表两部分组成。内存池被等分为了 n 块，对应的内存管理表，大小也为 n，内存管理表的每一个项对应内存池的一块内存。

内存管理表的项值代表的意义为：当该项值为 0 的时候，代表对应的内存块未被占用，当该项值非零的时候，代表该项对应的内存块已经被占用，其数值则代表被连续占用的内存块数。比如某项值为 10，那么说明包括本项对应的内存块在内，总共分配了 10 个内存块给外部的某个指针。

内存分配方向如上图所示，是从顶→底的分配方向。即首先从最末端开始找空内存。当内存管理刚初始化的时候，内存表全部清零，表示没有任何内存块被占用。

分配原理：

当指针 p 调用 malloc 申请内存的时候，先判断 p 要分配的内存块数（m），然后从第 n 开始，向下查找，直到找到 m 块连续的空内存块（即对应内存管理表项为 0），然后将这 m 个内存管理表项的值都设置为m（标记被占用），最后，把最后的这个空内存块的地址返回指针 p，完成一次分配。注意：如果当内存不够的时候（找到最后也没有找到连续 m 块空闲内存），则返回 NULL 给 p，表示分配失败。

释放原理：

当 p 申请的内存用完，需要释放的时候，调用 free 函数实现。free 函数先判断 p 指向的内存地址所对应的内存块，然后找到对应的内存管理表项目，得到 p 所占用的内存块数目 m（内存管理表项目的值就是所分配内存块的数目），将这 m 个内存管理表项目的值都清零，标记释放，完成一次内存释放。

接下来，我们简单介绍一下 STM32H7 的内存分配情况，如表 52.1.1 所示：

地址区间	内存
0X3801 0000 - 0X3FFF FFFF	Reserved
0X3800 0000 - 0X3800 FFFF	SRAM4（ 64 Kbytes ）
0X3004 0000 - 0X3004 7FFF	SRAM3（ 32 Kbytes ）
0X3002 0000 - 0X3003 FFFF	SRAM2（128 Kbytes ）
0X3000 0000 - 0X3001 FFFF	SRAM1（128 Kbytes ）
0X2400 0000 - 0X2407 FFFF	AXI SRAM（512 Kbytes ）
0X2000 0000 - 0X2001 FFFF	DTCM-RAM（128 Kbytes ）
0X0000 0000 - 0X0000 FFFF	ITCM-RAM（ 64 Kbytes ）

表 52.1.1 STM32H7 内存分配表

从上表可以看出，STM32H7 的内存分成：ITCM、DTCM、AXI SRAM、SRAM1~4 等几个部分，其中地址连续的区域分成 5 大块：ITCM（64KB）、DTCM（128KB）、AXI SRAM（512KB）、SRAM1~3（288KB）和 SRAM4（64KB），因为内存管理的内存池，必须是地址连续的内存空间，因此，STM32H7 内部内存需要 5 个内存池来管理。

另外，需要注意：

1、ITCM 和 DTCM 这两个内存块，仅 CPU 和 MDMA 可以直接访问，其他外设不可以直接访问！

2、以太网的 DMA 描述符等必须是定义在 SRAM3 里面才可以正常工作，因此我们一般把 SRAM3（32KB）独立给以太网使用，并不用作内存管理！

52.2 硬件设计

1. 例程功能

开机后，显示提示信息，等待外部输入。KEY0 用于申请内存，每次申请 2K 字节内存，KEY1 用于写数据到申请的内存里面，KEY2 用于释放内存，KEY_UP 用于切换操作内存区（AXI 内存/SDRAM 内存/SRAM12/SRAM4/DTCM/ITCM，总共管理 6 个内存块）。还可以通过 USMART 调试，测试内存管理函数。

LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯

LED0 - PB1

2）独立按键

KEY0 - PH3

KEY1 - PH2

KEY2 - PC13

WK_UP - PA0

3）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)

4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)

52.3 程序设计

52.3.1 程序流程图

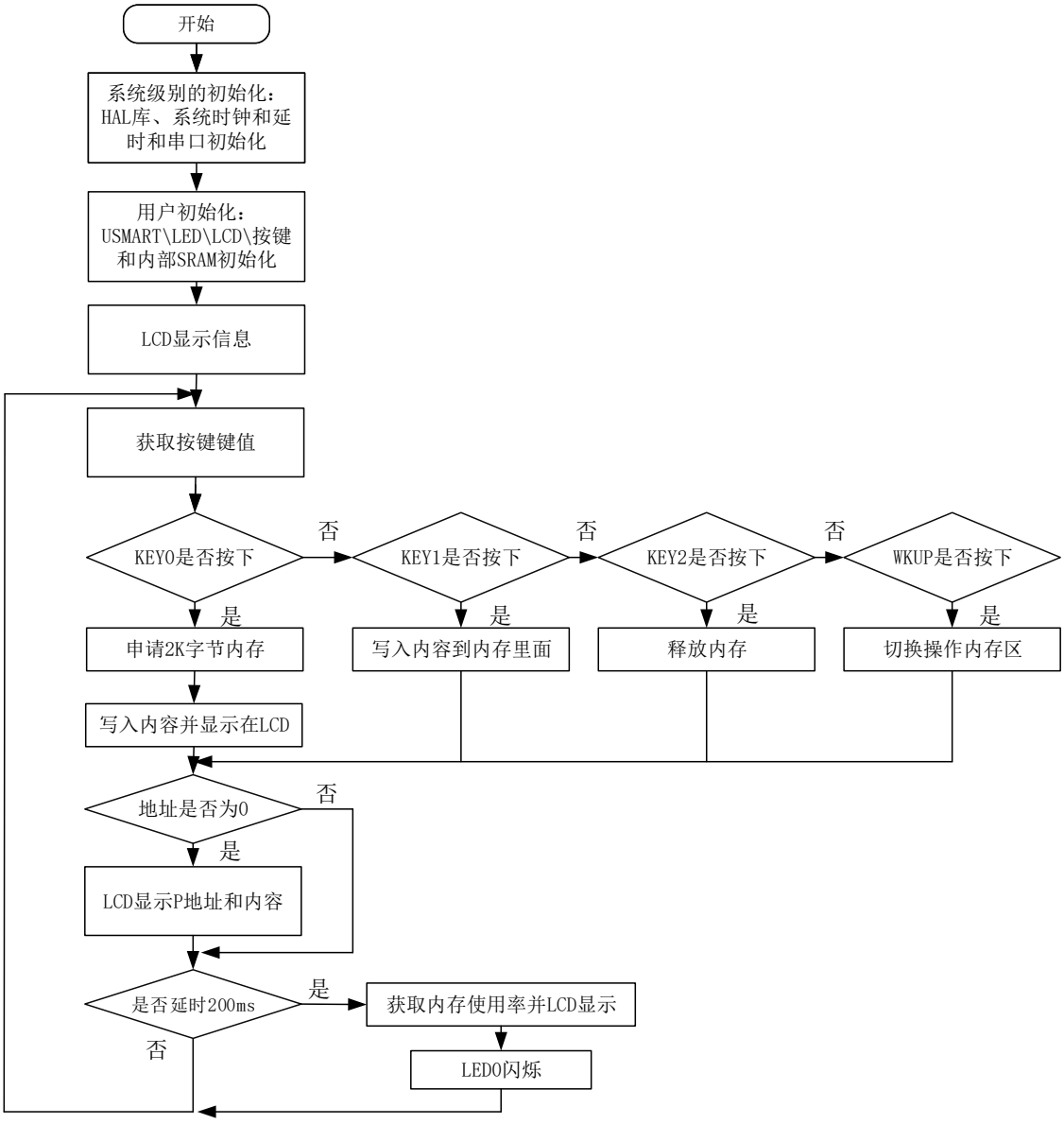


图 52.3.1.1 内存管理实验程序流程图

52.3.2 程序解析

1. MALLOC 代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。MALLOC 驱动源码包括两个文件：malloc.c 和 malloc.h。

下面我们介绍 malloc.h 中比较重要的一些结构体和内存参数宏定义，其定义如下：

```
/* 定义六个内存池 */
#define SRAMIN       0        /* AXI 内存池,AXI 共 512KB */
```

```
#define SRAMEX          1           /* 外部内存池（SDRAM）SDRAM 共 32MB */
#define SRAM12          2           /* SRAM1/2 内存池,SRAM1+SRAM2,共 256KB */
#define SRAM4           3           /* SRAM4 内存池,SRAM4 共 64KB */
/* DTCM 内存池,DTCM 共 128KB,此部分内存仅 CPU 和 MDMA(通过 AHBS)可以访问!!!! */
#define SRAMDTCM        4
/* ITCM 内存池,DTCM 共 64 KB,此部分内存仅 CPU 和 MDMA(通过 AHBS)可以访问!!!! */
#define SRAMITCM        5
#define SRAMBANK        6

/* 定义内存管理表类型,当外扩 SDRAM 的时候，必须使用 uint32_t 类型，
   否则可以定义成 uint16_t，以节省内存占用 */
#define MT_TYPE         uint32_t

/* mem1 内存参数设定.mem1 是 H7 内部的 AXI 内存. */
#define MEM1_BLOCK_SIZE     64              /* 内存块大小为 64 字节 */
#define MEM1_MAX_SIZE       448*1024        /* 最大管理内存 448K,H7 的 AXI 内存总共 512KB */
#define MEM1_ALLOC_TABLE_SIZE   MEM1_MAX_SIZE/MEM1_BLOCK_SIZE   /* 内存表大小 */

/* mem2 内存参数设定.mem2 是 H7 内部的 SRAM1+SRAM2 内存 */
#define MEM2_BLOCK_SIZE     64              /* 内存块大小为 64 字节 */
#define MEM2_MAX_SIZE       28912*1024
/* 最大管理内存 28912K,外扩 SDRAM 总共 32MB，LTDC 占用了 2MB，还剩 30MB */
#define MEM2_ALLOC_TABLE_SIZE   MEM2_MAX_SIZE/MEM2_BLOCK_SIZE   /* 内存表大小 */

/* mem3 内存参数设定.mem3 是 H7 内部的 SRAM1+SRAM2 内存 */
#define MEM3_BLOCK_SIZE     64          /* 内存块大小为 64 字节 */
#define MEM3_MAX_SIZE       240*1024    /* 最大管理内存 240K,H7 的 SRAM1+SRAM2 共 256KB */
#define MEM3_ALLOC_TABLE_SIZE   MEM3_MAX_SIZE/MEM3_BLOCK_SIZE   /* 内存表大小 */

/* mem4 内存参数设定.mem4 是 H7 内部的 SRAM4 内存 */
#define MEM4_BLOCK_SIZE     64          /* 内存块大小为 64 字节 */
#define MEM4_MAX_SIZE       60 *1024    /* 最大管理内存 60K,H7 的 SRAM4 共 64KB */
#define MEM4_ALLOC_TABLE_SIZE   MEM4_MAX_SIZE/MEM4_BLOCK_SIZE   /* 内存表大小 */

/* mem5 内存参数设定.mem5 是 H7 内部的 DTCM 内存,此部分内存仅 CPU 和 MDMA 可以访问!!!!!! */
#define MEM5_BLOCK_SIZE     64          /* 内存块大小为 64 字节 */
#define MEM5_MAX_SIZE       120 *1024   /* 最大管理内存 120K,H7 的 DTCM 共 128KB */
#define MEM5_ALLOC_TABLE_SIZE   MEM5_MAX_SIZE/MEM5_BLOCK_SIZE   /* 内存表大小 */

/* mem6 内存参数设定.mem6 是 H7 内部的 ITCM 内存,此部分内存仅 CPU 和 MDMA 可以访问!!!!!! */
#define MEM6_BLOCK_SIZE     64          /* 内存块大小为 64 字节 */
#define MEM6_MAX_SIZE       60 *1024    /* 最大管理内存 60K,H7 的 ITCM 共 64KB */
#define MEM6_ALLOC_TABLE_SIZE   MEM6_MAX_SIZE/MEM6_BLOCK_SIZE   /* 内存表大小 */

/* 内存管理控制器 */
struct _m_mallco_dev
{
    void (*init)(uint8_t);              /* 初始化 */
    uint16_t (*perused)(uint8_t);       /* 内存使用率 */
    uint8_t *membase[SRAMBANK];         /* 内存池 管理 SRAMBANK 个区域的内存 */
    MT_TYPE *memmap[SRAMBANK];          /* 内存管理状态表 */
    uint8_t  memrdy[SRAMBANK];          /* 内存管理是否就绪 */
};
```

这部分代码，定义了很多关键数据，比如内存块大小的定义：MEM1_BLOCK_SIZE、MEM2_BLOCK_SIZE、MEM3_BLOCK_SIZE、MEM4_BLOCK_SIZE、MEM5_BLOCK_SIZE 和 MEM6_BLOCK_SIZE，都是 64 字节。

MEM1_ALLOC_TABLE_SIZE ~ MEM6_ALLOC_TABLE_SIZE，则分别代表内存池 1 ~ 6 的内存管理表大小。

从这里可以看出，如果内存分块越小，那么内存管理表就越大，当分块为 4 字节 1 个块的

时候，内存管理表就和内存池一样大了（管理表的每项都是 uint32_t 类型）。显然是不合适的，我们这里取 64 字节，比例为 1:16，内存管理表相对就比较小了。

通过这个内存管理控制器_m_malloc_dev 结构体，我们把分块式内存管理的相关信息，其初始化函数、获取使用率、内存池、内存管理表以及内存管理的状态保存下来，实现对内存池的管理控制。

下面介绍 malloc.c 文件，其中，内存池、内存管理表、内存管理参数和内存管理控制器的定义如下：

```
#if !(__ARMCC_VERSION >= 6010050)    /* 不是 AC6 编译器，即使用 AC5 编译器时 */

/* 内存池(64 字节对齐) */
static __align(64) uint8_t mem1base[MEM1_MAX_SIZE];
/* 内部 SRAM 内存池 */
static __align(64) uint8_t mem2base[MEM2_MAX_SIZE]
__attribute__((at(0XC01F4000)));/*外部 SDRAM 内存池,前面 2M 给 LTDC 用了(1280*800*2)*/
static __align(64) uint8_t mem3base[MEM3_MAX_SIZE]
__attribute__((at(0x30000000)));/* 内部 SRAM1+SRAM2 内存池 */
static __align(64) uint8_t mem4base[MEM4_MAX_SIZE]
__attribute__((at(0x38000000)));/* 内部 SRAM4 内存池 */
static __align(64) uint8_t mem5base[MEM5_MAX_SIZE]
__attribute__((at(0x20000000)));/* 内部 DTCM 内存池 */
static __align(64) uint8_t mem6base[MEM6_MAX_SIZE]
__attribute__((at(0x00000000)));/* 内部 ITCM 内存池  */

/* 内存管理表 */
static MT_TYPE mem1mapbase[MEM1_ALLOC_TABLE_SIZE];
/* 内部 SRAM 内存池 MAP */
static MT_TYPE mem2mapbase[MEM2_ALLOC_TABLE_SIZE]
__attribute__((at(0XC01F4000 + MEM2_MAX_SIZE))); /* 外部 SDRAM 内存池 MAP */
static MT_TYPE mem3mapbase[MEM3_ALLOC_TABLE_SIZE]
__attribute__((at(0x30000000 + MEM3_MAX_SIZE)));  /* 内部 SRAM1+SRAM2 内存池 MAP */
static MT_TYPE mem4mapbase[MEM4_ALLOC_TABLE_SIZE]
__attribute__((at(0x38000000 + MEM4_MAX_SIZE)));  /* 内部 SRAM4 内存池 MAP */
static MT_TYPE mem5mapbase[MEM5_ALLOC_TABLE_SIZE]
__attribute__((at(0x20000000 + MEM5_MAX_SIZE)));  /* 内部 DTCM 内存池 MAP */
static MT_TYPE mem6mapbase[MEM6_ALLOC_TABLE_SIZE]
__attribute__((at(0x00000000 + MEM6_MAX_SIZE)));  /* 内部 ITCM 内存池 MAP */

#else       /* 使用 AC6 编译器时 */

/* 内存池(64 字节对齐) */
static __ALIGNED(64) uint8_t mem1base[MEM1_MAX_SIZE];
/* 内部 SRAM 内存池 */
static __ALIGNED(64) uint8_t mem2base[MEM2_MAX_SIZE]
__attribute__((section(".bss.ARM.__at_0XC01F4000")));
/* 外部 SDRAM 内存池,前面 2M 给 LTDC 用了(1280*800*2) */
static __ALIGNED(64) uint8_t mem3base[MEM3_MAX_SIZE]
__attribute__((section(".bss.ARM.__at_0x30000000")));/* 内部 SRAM1+SRAM2 内存池 */
static __ALIGNED(64) uint8_t mem4base[MEM4_MAX_SIZE]
__attribute__((section(".bss.ARM.__at_0x38000000")));/* 内部 SRAM4 内存池 */
static __ALIGNED(64) uint8_t mem5base[MEM5_MAX_SIZE]
__attribute__((section(".bss.ARM.__at_0x20000000")));/* 内部 DTCM 内存池 */
static __ALIGNED(64) uint8_t mem6base[MEM6_MAX_SIZE]
__attribute__((section(".bss.ARM.__at_0x00000000")));/* 内部 ITCM 内存池 */

/* 内存管理表 */
static MT_TYPE mem1mapbase[MEM1_ALLOC_TABLE_SIZE];
/* 内部 SRAM 内存池 MAP */
static MT_TYPE mem2mapbase[MEM2_ALLOC_TABLE_SIZE]
__attribute__((section(".bss.ARM.__at_0XC1E30000"))); /* 内部 CCM 内存池 MAP */
```

```
static MT_TYPE mem3mapbase[MEM3_ALLOC_TABLE_SIZE]
__attribute__((section(".bss.ARM.__at_0X3003C000"))); /* 外部 SRAM 内存池 MAP */
static MT_TYPE mem4mapbase[MEM4_ALLOC_TABLE_SIZE]
__attribute__((section(".bss.ARM.__at_0X3800F000"))); /* 内部 CCM 内存池 MAP */
static MT_TYPE mem5mapbase[MEM5_ALLOC_TABLE_SIZE]
__attribute__((section(".bss.ARM.__at_0X2001E000"))); /* 外部 SRAM 内存池 MAP */
static MT_TYPE mem6mapbase[MEM6_ALLOC_TABLE_SIZE]
__attribute__((section(".bss.ARM.__at_0X0000F000"))); /* 内部 CCM 内存池 MAP */

#endif

/* 内存管理参数 */
const uint32_t memtblsize[SRAMBANK] =
    {MEM1_ALLOC_TABLE_SIZE,MEM2_ALLOC_TABLE_SIZE,
     MEM3_ALLOC_TABLE_SIZE,MEM4_ALLOC_TABLE_SIZE,
     MEM5_ALLOC_TABLE_SIZE,MEM6_ALLOC_TABLE_SIZE
                          };        /* 内存表大小 */

const uint32_t memblksize[SRAMBANK] =
    {MEM1_BLOCK_SIZE, MEM2_BLOCK_SIZE,
     MEM3_BLOCK_SIZE, MEM4_BLOCK_SIZE,
     MEM5_BLOCK_SIZE, MEM6_BLOCK_SIZE
                          };        /* 内存分块大小 */

const uint32_t memsize[SRAMBANK] =
    {MEM1_MAX_SIZE, MEM2_MAX_SIZE,
     MEM3_MAX_SIZE, MEM4_MAX_SIZE,
     MEM5_MAX_SIZE, MEM6_MAX_SIZE
                          };        /* 内存总大小 */

/* 内存管理控制器 */
struct _m_mallco_dev mallco_dev =
{
    my_mem_init,                                                    /* 内存初始化 */
    my_mem_perused,                                                 /* 内存使用率 */
    mem1base, mem2base, mem3base,mem4base,mem5base,mem6base,        /* 内存池 */
    mem1mapbase, mem2mapbase, mem3mapbase, mem4mapbase,
    mem5mapbase, mem6mapbase,       /* 内存管理状态表 */
    0, 0, 0, 0, 0, 0,    /* 内存管理未就绪 */
};
```

我们通过内存管理控制器 mallco_dev 结构体，实现对六个内存池的管理控制。

第一个是内部 SRAM 内存池，定义为：

```
static __align(64) uint8_t mem1base[MEM1_MAX_SIZE];  /* 内部 SRAM 内存池 */
```

第二个是 SDRAM 内存池，定义为：

```
static __align(64) uint8_t mem2base[MEM2_MAX_SIZE]
                    __attribute__((at(0XC01F4000))); /* 外部 SDRAM 内存池 */
```

第三个是 SRAM1+SRAM2 内存池，定义为：

```
static __align(64) uint8_t mem3base[MEM3_MAX_SIZE]
                    __attribute__((at(0x30000000))); /* 内部 SRAM1+SRAM2 内存池 */
```

第四个是 SRAM4 内存池，定义为：

```
static __align(64) uint8_t mem4base[MEM4_MAX_SIZE]
                    __attribute__((at(0x38000000))); /* 内部 SRAM4 内存池 */
```

第五个是 DTCM 内存池，定义为：

```
static __align(64) uint8_t mem5base[MEM5_MAX_SIZE]
                    __attribute__((at(0x20000000))); /* 内部 DTCM 内存池 */
```

第六个是 ITCM 内存池，定义为：

```
static __align(64) uint8_t mem6base[MEM6_MAX_SIZE]
                    __attribute__((at(0x00000000))); /* 内部 ITCM 内存池 */
```

因为 STM32H7 内部有 6 个连续的内存块，因此总共需要 6 个内存池，AXI SRAM 内存池的首地址由编译器指定，其他几个内部内存池的首地址就是相应 SRAM 的首地址。

其中，__align(64)定义内存池为 64 字节对齐，以适应各种不同场合的需求。

这样总共有 6 部分内存，分成 6 个内存池，每个内存池需要一个内存管理表，因此又有 6 个内存管理表：mem1mapbase~ mem6mapbase，内存管理表所占内存，也指定在对应的内存块里面分配。因此：内存池+内存管理表，基本上就占了整个内存块的全部空间了。

下面介绍其他的 malloc 代码，具体如下：

```
/**
 * @brief       复制内存
 * @param       *des : 目的地址
 * @param       *src : 源地址
 * @param        n   : 需要复制的内存长度(字节为单位)
 * @retval      无
 */
void my_mem_copy(void *des, void *src, uint32_t n)
{
    uint8_t *xdes = des;
    uint8_t *xsrc = src;
    while (n--)*xdes++ = *xsrc++;
}

/**
 * @brief       设置内存值
 * @param       *s     : 内存首地址
 * @param       c      : 要设置的值
 * @param       count  : 需要设置的内存大小(字节为单位)
 * @retval      无
 */
void my_mem_set(void *s, uint8_t c, uint32_t count)
{
    uint8_t *xs = s;
    while (count--)*xs++ = c;
}

/**
 * @brief       内存管理初始化
 * @param       memx : 所属内存块
 * @retval      无
 */
void my_mem_init(uint8_t memx)
{
   uint8_t mttsize = sizeof(MT_TYPE);
   /* 内存状态表数据清零 */
   my_mem_set(mallco_dev.memmap[memx], 0, memtblsize[memx]*mttsize);
   /* 内存管理初始化 OK */
   mallco_dev.memrdy[memx] = 1;
}

/**
 * @brief       获取内存使用率
 * @param       memx : 所属内存块
 * @retval      使用率(扩大了 10 倍,0~1000,代表 0.0%~100.0%)
 */
uint16_t my_mem_perused(uint8_t memx)
{
    uint32_t used = 0;
    uint32_t i;

    for (i = 0; i < memtblsize[memx]; i++)
    {
        if (mallco_dev.memmap[memx][i])used++;
    }

    return (used * 1000) / (memtblsize[memx]);
```

```
}

/**
 * @brief       内存分配(内部调用)
 * @param       memx : 所属内存块
 * @param       size : 要分配的内存大小(字节)
 * @retval      内存偏移地址
 *   @arg       0 ~ 0XFFFFFFFE : 有效的内存偏移地址
 *   @arg       0XFFFFFFFF     : 无效的内存偏移地址
 */
uint32_t my_mem_malloc(uint8_t memx, uint32_t size)
{
    signed long offset = 0;
    uint32_t nmemb;       /* 需要的内存块数 */
    uint32_t cmemb = 0;   /* 连续空内存块数 */
    uint32_t i;

    if (!mallco_dev.memrdy[memx])
    {
        mallco_dev.init(memx);           /* 未初始化,先执行初始化 */
    }
    if (size == 0) return 0XFFFFFFFF;    /* 不需要分配 */
    nmemb = size / memblksize[memx];     /* 获取需要分配的连续内存块数 */

    if (size % memblksize[memx]) nmemb++;
   /* 搜索整个内存控制区 */
    for (offset = memtblsize[memx] - 1; offset >= 0; offset--)
    {
        if (!mallco_dev.memmap[memx][offset])
        {
            cmemb++;              /* 连续空内存块数增加 */
        }
        else
        {
            cmemb = 0;           /* 连续内存块清零 */
        }

        if (cmemb == nmemb)      /* 找到了连续 nmemb 个空内存块 */
        {
            for (i = 0; i < nmemb; i++) /* 标注内存块非空 */
            {
                mallco_dev.memmap[memx][offset + i] = nmemb;
            }
            return (offset * memblksize[memx]); /* 返回偏移地址 */
        }
    }
    return 0XFFFFFFFF;  /* 未找到符合分配条件的内存块 */
}

/**
 * @brief       释放内存(内部调用)
 * @param       memx   : 所属内存块
 * @param       offset : 内存地址偏移
 * @retval      释放结果
 *   @arg       0, 释放成功;
 *   @arg       1, 释放失败;
 *   @arg       2, 超区域了(失败);
 */
static uint8_t my_mem_free(uint8_t memx, uint32_t offset)
{
    int i;
```

```
    if (!mallco_dev.memrdy[memx])    /* 未初始化,先执行初始化 */
    {
        mallco_dev.init(memx);
        return 1;                    /* 未初始化 */
    }

    if (offset < memsize[memx])      /* 偏移在内存池内. */
    {
        int index = offset / memblksize[memx];          /* 偏移所在内存块号码 */
        int nmemb = mallco_dev.memmap[memx][index];     /* 内存块数量 */

        for (i = 0; i < nmemb; i++)                     /* 内存块清零 */
        {
            mallco_dev.memmap[memx][index + i] = 0;
        }

        return 0;
    }
    else
    {
        return 2;    /* 偏移超区了. */
    }
}

/**
 * @brief       释放内存(外部调用)
 * @param       memx : 所属内存块
 * @param       ptr  : 内存首地址
 * @retval      无
 */
void myfree(uint8_t memx, void *ptr)
{
    uint32_t offset;

    if (ptr == NULL)return;      /* 地址为 0. */
    offset = (uint32_t)ptr - (uint32_t)mallco_dev.membase[memx];
    my_mem_free(memx, offset); /* 释放内存 */
}

/**
 * @brief       分配内存(外部调用)
 * @param       memx : 所属内存块
 * @param       size : 要分配的内存大小(字节)
 * @retval      分配到的内存首地址.
 */
void *mymalloc(uint8_t memx, uint32_t size)
{
    uint32_t offset;
    offset = my_mem_malloc(memx, size);

    if (offset == 0XFFFFFFFF)    /* 申请出错 */
    {
        return NULL;               /* 返回空(0) */
    }
    else     /* 申请没问题, 返回首地址 */
    {
        return (void *)((uint32_t)mallco_dev.membase[memx] + offset);
    }
}

/**
 * @brief       重新分配内存(外部调用)
 * @param       memx : 所属内存块
```

```
 * @param       *ptr : 旧内存首地址
 * @param       size : 要分配的内存大小(字节)
 * @retval      新分配到的内存首地址.
 */
void *myrealloc(uint8_t memx, void *ptr, uint32_t size)
{
    uint32_t offset;
    offset = my_mem_malloc(memx, size);

    if (offset == 0XFFFFFFFF)    /* 申请出错 */
    {
        return NULL;              /* 返回空(0) */
    }
    else    /* 申请没问题, 返回首地址 */
    {
        my_mem_copy((void *)((uint32_t)mallco_dev.membase[memx] + offset),
                    ptr, size);          /* 拷贝旧内存内容到新内存 */
        myfree(memx, ptr);               /* 释放旧内存 */
        return (void *)((uint32_t)mallco_dev.membase[memx] + offset);
                                         /* 返回新内存首地址 */
    }
}
```

整个 malloc 代码的核心函数：my_mem_malloc 和 my_mem_free，分别用于内存申请和内存释放。思路就是前面 52.1 所介绍的分配内存和释放内存，不过在这里，这两个函数知识内部调用，外部调用我们另外定义了 mymalloc 和 myfree 两个函数，其他函数我们就不多介绍了。

2. main.c 代码

main.c 代码如下：

```
const char *SRAM_NAME_BUF[SRAMBANK] = {"SRAMIN", "SRAMEX","SRAM12", "SRAM4",
                                       "SRAMDTCM", "SRAMITCM"};

int main(void)
{
    uint8_t paddr[20];  /* 存放 P Addr:+p 地址的 ASCII 值 */
    uint16_t memused = 0;
    uint8_t key;
    uint8_t i = 0;
    uint8_t *p = 0;
    uint8_t *tp = 0;
    uint8_t sramx = 0;  /* 默认为内部 sram */

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    my_mem_init(SRAMIN);                    /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                    /* 初始化外部内存池(SDRAM) */
    my_mem_init(SRAM12);                    /* 初始化 SRAM12 内存池(SRAM1+SRAM2) */
    my_mem_init(SRAM4);                     /* 初始化 SRAM4 内存池(SRAM4) */
    my_mem_init(SRAMDTCM);                  /* 初始化 DTCM 内存池(DTCM) */
    my_mem_init(SRAMITCM);                  /* 初始化 ITCM 内存池(ITCM) */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
```

```
    lcd_show_string(30, 70, 200, 16, 16, "MALLOC TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Malloc KEY2: Free", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:SRAMx KEY1:Read", RED);

    lcd_show_string(30, 160, 200, 16, 16, "SRAMIN ", BLUE);
    lcd_show_string(30, 180, 200, 16, 16, "SRAMIN   USED:", BLUE);
    lcd_show_string(30, 200, 200, 16, 16, "SRAMEX   USED:", BLUE);
    lcd_show_string(30, 220, 200, 16, 16, "SRAM12   USED:", BLUE);
    lcd_show_string(30, 240, 200, 16, 16, "SRAM4    USED:", BLUE);
    lcd_show_string(30, 260, 200, 16, 16, "SRAMDTCM USED:", BLUE);
    lcd_show_string(30, 280, 200, 16, 16, "SRAMITCM USED:", BLUE);

    while (1)
    {
        key = key_scan(0);      /* 不支持连按 */
        switch (key)
        {
            case 0:     /* 没有按键按下 */
                break;
            case KEY0_PRES:     /* KEY0 按下 */
                p=mymalloc(sramx, 2048);/*申请 2K 字节,并写入内容,显示在 lcd 屏幕上面*/

                if (p != NULL)
                {
                    /* 向 p 写入一些内容 */
                    sprintf((char *)p, "Memory Malloc Test%03d", i);
                }
                break;
            case KEY1_PRES:             /* KEY1 按下 */
                if(p!=NULL)
                {
                    sprintf((char*)p,"Memory Malloc Test%03d",i);
                    lcd_show_string(30,310,209,16,16,(char*)p,BLUE);
                }       /* 显示 P 的内容 */
                break;

            case KEY2_PRES:             /* KEY2 按下 */
                myfree(sramx, p);       /* 释放内存 */
                p = 0;                  /* 指向空地址 */
                break;

            case WKUP_PRES:         /* KEY UP 按下 */
                sramx++;
                if (sramx > SRAMBANK)sramx = 0;
                lcd_show_string(60, 160, 200, 16, 16,
                                (char *)SRAM_NAME_BUF[sramx], BLUE);
                break;
        }

        if (tp != p && p != NULL)
        {
            tp = p;
            sprintf((char *)paddr, "P Addr:0X%08X", (uint32_t)tp);
            /* 显示 p 的地址 */
            lcd_show_string(30, 310, 209, 16, 16, (char *)paddr, BLUE);

            if (p)
            {
                /* 显示 P 的内容 */
                lcd_show_string(30, 330, 330, 16, 16, (char *)p, BLUE);
            }
            else
            {
```

```
                lcd_fill(30, 310, 300, 296, WHITE); /* p=0,清除显示 */
            }
        }

        delay_ms(10);
        i++;

        if ((i % 20) == 0)  /* DS0 闪烁. */
        {
            memused = my_mem_perused(SRAMIN);
            sprintf((char *)paddr, "%d.%01d%%", memused / 10, memused % 10);
            /* 显示内部内存使用率 */
            lcd_show_string(30 + 112, 180, 200, 16, 16, (char *)paddr, BLUE);

             memused = my_mem_perused(SRAMEX);
            sprintf((char *)paddr, "%d.%01d%%", memused / 10, memused % 10);
            /* 显示 TCM 内存使用率 */
            lcd_show_string(30 + 112, 200, 200, 16, 16, (char *)paddr, BLUE);

            memused = my_mem_perused(SRAM12);
            sprintf((char *)paddr, "%d.%01d%%", memused / 10, memused % 10);
            /* 显示 TCM 内存使用率 */
            lcd_show_string(30 + 112, 220, 200, 16, 16, (char *)paddr, BLUE);

            memused = my_mem_perused(SRAM4);
            sprintf((char *)paddr, "%d.%01d%%", memused / 10, memused % 10);
            /* 显示内部内存使用率 */
            lcd_show_string(30 + 112, 240, 200, 16, 16, (char *)paddr, BLUE);

            memused = my_mem_perused(SRAMDTCM);
            sprintf((char *)paddr, "%d.%01d%%", memused / 10, memused % 10);
            /* 显示外部内存使用率 */
            lcd_show_string(30 + 112, 260, 200, 16, 16, (char *)paddr, BLUE);

            memused = my_mem_perused(SRAMITCM);
            sprintf((char *)paddr, "%d.%01d%%", memused / 10, memused % 10);
            /* 显示 TCM 内存使用率 */
            lcd_show_string(30 + 112, 280, 200, 16, 16, (char *)paddr, BLUE);

            LED0_TOGGLE();  /* LED0 闪烁 */
        }
    }
}
```

该部分代码比较简单，主要是对 mymalloc 和 myfree 的应用。不过这里提醒大家，如果对一个指针进行多次内存申请，而之前的申请又没释放，那么将造成“内存泄露”，这是内存管理所不希望发生的，久而久之，可能导致无内存可用的情况！所以，在使用的时候，请大家一定记得，申请的内存在用完以后，一定要释放。

另外，本章希望利用 USMART 调试内存管理，所以在 USMART 里面添加了 mymalloc 和 myfree 两个函数，用于测试内存分配和内存释放。大家可以通过 USMART 自行测试。

52.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。

刚开始，所有内存的使用率均为 0%，说明还没有任何内存被使用，此时我们按下 KEY0，就可以看到内部 SRAM 内存被使用 0.4%了，如下图所示：

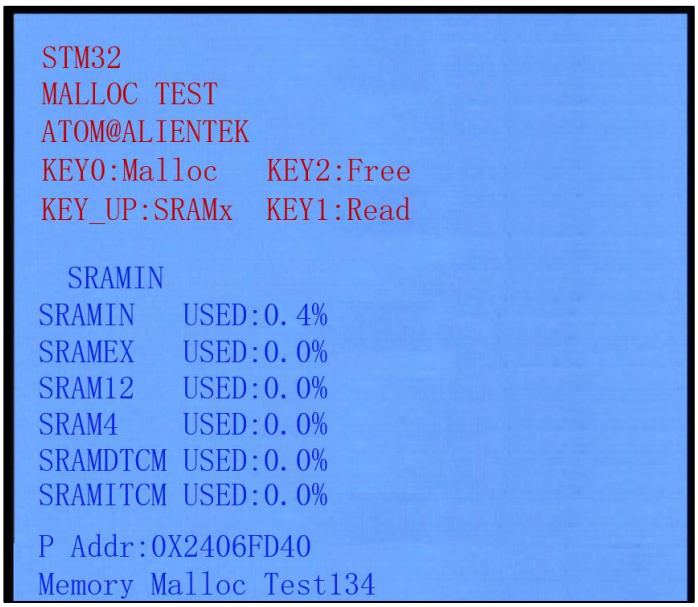


图 52.4.1 内存管理实验测试图

同时看到下面提示了指针 p 所指向的地址（其实就是被分配到的内存地址）和内容。多按几次 KEY0，可以看到内存使用率持续上升（注意对比 p 的值，可以发现是递减的，说明是从顶部开始分配内存！），此时如果按下 KEY2，可以发现内存使用率降低了 0.4%，但是再按 KEY2 将不再降低，说明“内存泄露”了。这就是前面提到的对一个指针多次申请内存，而之前申请的内存又没释放，导致的“内存泄露”。

按 KEY_UP 按键，可以切换当前操作内存（内部 SRAM、外部 SDRAM、内部 SRAM12、内部 SRAM4、内部 DTCM 和内部 ITCM），KEY1 键用于更新指针 p 的内容，更新后的内容将重新显示在 LCD 模块上面。**注意：当使外部 SDRAM 内存的时候，我们需要按很多次 KEY0（15 次），才可以看到内存使用率上升 0.1%，因为按一次只是申请 2K 字节，15 次才申请 30KB，内存总大小为 28912KB，所以，内存使用率为：30/28912=0.001（0.1%）。**

本章，我们还可以借助 USMART，测试内存的分配和释放，有兴趣的朋友可以动手试试。如图 52.4.2 所示：

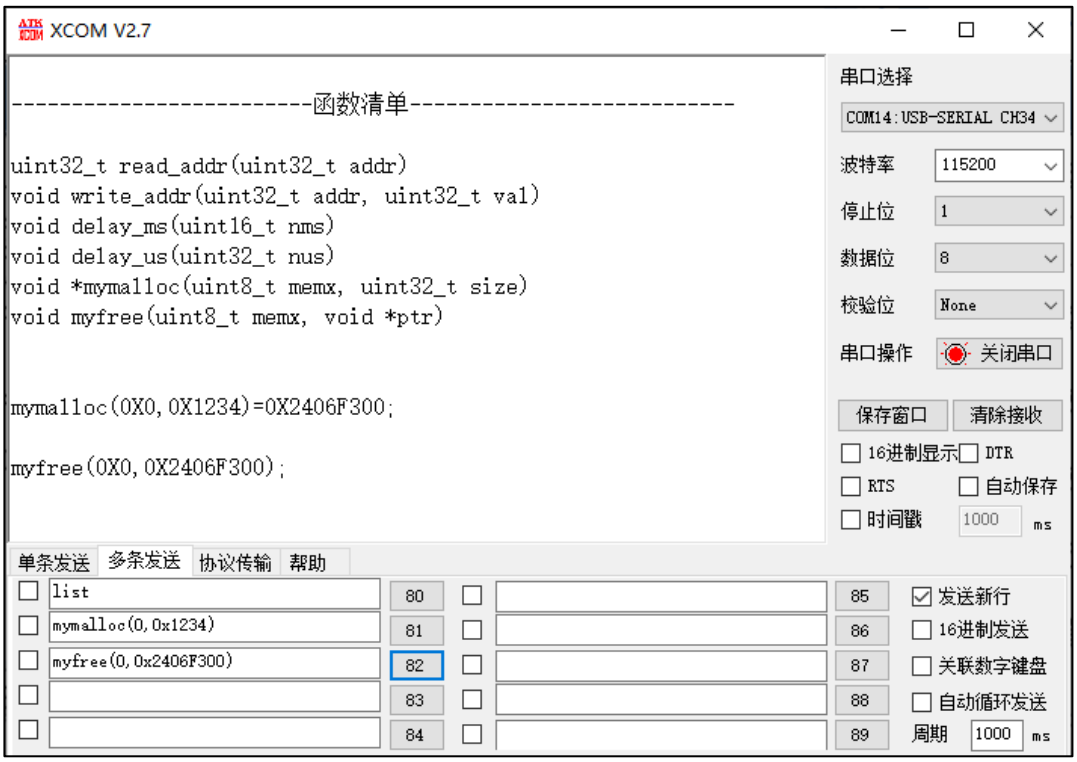


图 52.4.2 USMART 测试内存管理图

图中，我们先申请了 4660 字节的内存，然后得到申请到的内存首地址为：0x2406F300，说明我们申请内存成功（如果不成功，则会收到 0），然后释放内存的时候，参数是指针的地址，即执行：myfree(0x2406F300)，就可以释放我们申请到的内存。其他情况，大家可以自行测试并分析。

第五十三章 SD 卡实验

很多单片机系统都需要大容量存储设备，以存储数据。目前常用的有 U 盘，FLASH 芯片，SD 卡等。他们各有优点，综合比较，最适合单片机系统的莫过于 SD 卡了，它不仅容量可以做到很大（32GB 以上），支持 SPI/SDMMC 驱动，而且有多种体积的尺寸可供选择（标准的 SD 卡尺寸及 Micor SD 卡尺寸等），能满足不同应用的要求。

只需要少数几个 IO 口即可外扩一个高达 32GB 或以上的外部存储器，容量从几十 M 到几十 G 选择范围很大，更换也很方便，编程也简单，是单片机大容量外部存储器的首选。

阿波罗 STM32H743 开发板使用的接口是 Micro SD 卡接口，卡座带自锁功能，使用 STM32H7 自带的 SDMMC 接口驱动，4 位模式，最高通信速度可达 48Mhz（分频器旁路时），最高每秒可传输数据 24M 字节，对于一般应用足够了。在本章中，我们将向大家介绍，如何在开发板上实现 Micor SD 卡的读取。

本章分为如下几个部分：

53.1 SD 卡简介
53.2 SDMMC 接口简介
53.3 SD 卡初始化流程
53.4 硬件设计
53.5 程序设计
53.6 下载验证

53.1 SD 卡简介

53.1.1 SD 物理结构

SD 卡的规范由 SD 卡协会明确，可以访问 https://www.sdcard.org 查阅更多标准。SD 卡主要有 SD、Mini SD 和 microSD(原名 TF 卡，2004 年正式更名为 Micro SD Card，为方便本文用 microSD 表示)三种类型，Mini SD 已经被 microSD 取代，使用得不多，根据最新的 SD 卡规格列出的参数如表 53.1.1.1 所示：

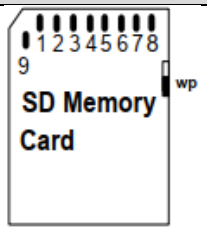
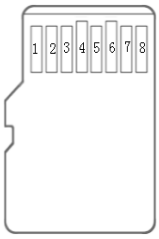
形状		SD	microSD
尺寸		 32 x 24 x 2.1 mm, 32 x 24 x 1.4 mm, 约重 1.2g~2.5g	 11 x 15 x 1.0 mm，约重 0.5g
卡片种类（容量范围）		SD(≤2GB)、SDHC(2GB~32GB)、SDXC(32GB ~2TB)、SDUC(2TB ~128TB)	
硬件规格	脚位数	High Speed and UHS-I : 9 pins	High Speed and UHS-I : 8 pins
		UHS-II and UHS-III: 17 pins	UHS-II and UHS-III: 16 pins
		SD Express 1-lane: 17-19 pins	SD Express 1-lane: 17 pins
		SD Express 2-lane: 25-27 pins	
	电压范围	3.3V 版本: 2.7V - 3.6V	
		1.8V 低电压版: 1.70V-1.95V	
	防写开关	是	否

表 53.1.1.1 SD 卡的主要规格参数

上述表格的“脚位数”，对应于实卡上的“金手指”数，不同类型的卡的触点数量不同，访问的速度也不相同。SD 卡允许了不同的接口来访问它的内部存储单元。最常见的是 SDIO 模式和 SPI 模式，根据这两种接口模式，我们也列出 SD 卡引脚对应于这两种不同的电路模式的引脚功能定义，如表 53.1.1.2 所示。

SD 卡	SD Mode			SPI Mode		
引脚编号	引脚名	引脚类型	功能描述	引脚名	引脚类型	功能描述
1	CD/DAT3	I/O/PP	卡识别/数据线位 3	CS	I3	片选，低电平有效
2	CMD	I/O/PP	命令/响应	DI	I	数据输入
3	VSS1	S	电源地	VSS	S	电源地
4	VDD	S	DC 电源正极	VDD	S	DC 电源正极
5	CLK	I	Clock	SCLK	I	时钟
6	VSS2	S	电源地	VSS2	S	电源地
7	DAT0	I/O/PP	数据线位 0	DO	O/PP	数据输出
8	DAT1	I/O/PP	数据线位 1	RSV		
9	DAT2	I/O/PP	数据线位 2	RSV		

表 53.1.1.2 SD 卡引脚编号（注：S:电源 I：输入 O：推挽输出 PP：推挽）

对比的，我们来看一下 microSD 引脚，可见只比 SD 卡少了一个电源引脚 VSS2，其它的引脚功能类似。

microSD	SD Mode			SPI Mode		
引脚编号	引脚名	引脚类型	功能描述	引脚名	引脚类型	功能描述
1	DAT2	I/O/PP	数据线位 2	RSV		
2	CD/DAT3	I/O/PP	卡识别/数据线位 3	CS	I	片选 低电平有效
3	CMD	PP	命令/响应	DI	I	数据输入
4	VDD	S	DC 电源正极	VDD	S	DC 电源正极
5	CLK	I	时钟	SCLK	I	时钟
6	VSS	S	电源地	VSS	S	电源地
7	DAT0	I/O/PP	数据线位 0	DO	O/PP	数据输出
8	DAT1	I/O/PP	数据线位 1	RSV		

表 53.1.1.3 microSD 卡引脚编号（注：S:电源 I：输入 O：推挽输出 PP：推挽）

SD 卡和 micorSD 只有引脚和形状大小不同，内部结构类似，操作时序完全相同，可以使用完全相同的代码驱动，下面以 9’Pin SD 卡的内部结构为为例，展示 SD 卡的存储结构。

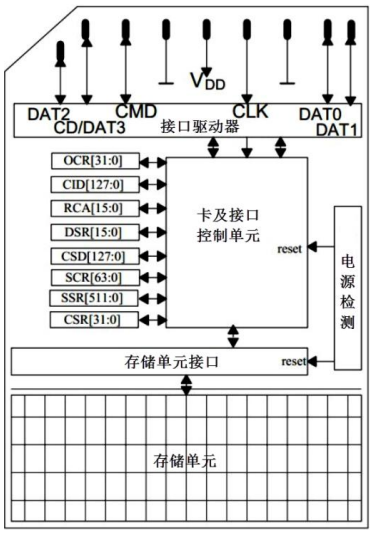


图 53.1.1.1 SD 卡内部物理结构（RCA 寄存器在 SPI 模式下不可访问）

SD 卡有自己的寄存器，但它不能直接进行读写操作，需要通过命令来控制，SDMMC 协议定义了一些命令用于实现某一特定功能，SD 卡根据收到的命令要求对内部寄存器进行修改。图 53.1.1.4 中描述的 SD 卡的寄存器是我们和 SD 卡进行数据通讯的主要通道，如下：

名称	位宽	描述
CID	128	卡标识(Card identification):每个卡都是唯一的
RCA	16	相对地址(Relative card address):卡的本地系统地址，初始化时，动态地由卡建议，经主机核准。
DSR	16	驱动级寄存器(Driver Stage Register):配置卡的输出驱动
CSD	128	卡的特定数据(Card Specific Data):卡的操作条件信息
SCR	64	SD 配置寄存器(SD Configuration Register):SD 卡特殊特性信息
OCR	32	操作条件寄存器(Operation conditions register)：卡电源和状态标识
SSR	512	SD 状态(SD Status):SD 卡专有特征的信息
CSR	32	卡状态(Card Status):卡状态信息

表 53.1.1.4 SD 卡寄存器信息

关于 SD 卡的更多信息和硬件设计规范可以参考 SD 卡协议《**Physical Layer Simplified Specification Version 2.00**》的相关章节(注：因为 STM32 的 SDMMC 匹配的是 SD 协议 2.0 版本，后续版本也兼容此旧协议版本，故本章仍以 2.0 版本为介绍对象)。

53.1.2 命令和响应

一个完整的 SD 卡操作过程是：主机(单片机等)发起“命令”，SD 卡根据命令的内容决定是否发送响应信息及数据等，如果是数据读/写操作，主机还需要发送停止读/写数据的命令来结束本次操作，这意味着主机发起命令指令后，SD 卡可以没有响应、数据等过程，这取决于命令的含义。

这一过程如图 53.1.2.1 所示：

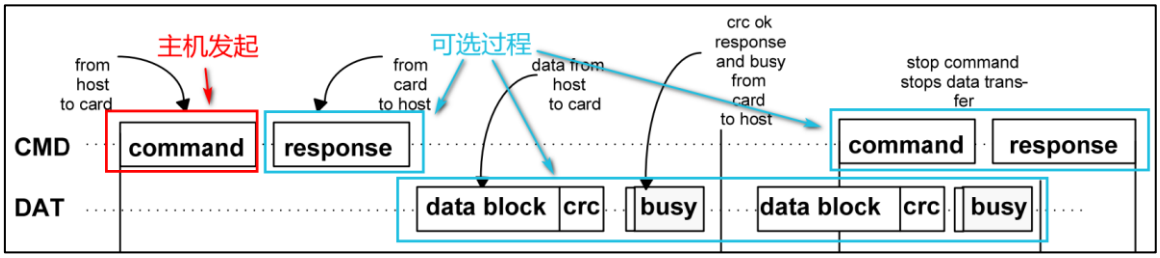


图 53.1.2.1 SD 卡命令格式

SD 卡有多种命令和响应，它们的格式定义及含义在《SD 卡协议 V2.0》的第三和第四章有详细介绍，发送命令时主机只能通过 CMD 引脚发送给 SD 卡，串行逐位发送时先发送最高位(MSB)，然后是次高位这样类推……

接下来，我们看看 SD 卡的命令格式，如表 53.1.2.1 所示：

字节1			字节2--5	字节6	
47	46	45:40	39:8	7:1	0
0	1	command	命令参数	CRC7	1

表 53.1.2.1 SD 卡控制命令格式

SD 卡的命令固定为 48 位，由 6 个字节组成，字节 1 的最高 2 位固定为 01，低 6 位为命令号（比如 CMD16，为 10000B 即 16 进制的 0X10，完整的 CMD16，第一个字节为 01010000，即 0X10+0X40）。字节 2~5 为命令参数，有些命令是没有参数的。字节 6 的高七位为 CRC 值，最低位恒定为 1。

SD 卡的命令总共有 12 类，分为 Class0~Class11，本章，我们仅介绍几个比较重要的命令，如表 53.1.2.2 所示：

命令	参数	响应	描述
CMD0(0X00)	NONE	R1	复位SD卡
CMD8(0X08)	VHS+Checkpattern	R7	发送接口状态命令
CMD9(0X09)	NONE	R1	读取卡特定数据寄存器
CMD10(0X0A)	NONE	R1	读取卡标志数据寄存器
CMD16(0X10)	块大小	R1	设置块大小（字节数）
CMD17(0X11)	地址	R1	读取一个块的数据
CMD24(0X18)	地址	R1	写入一个块的数据
CMD41(0X29)	NONE	R3	发送给主机容量支持信息和激活卡初始化过程
CMD55(0X37)	NONE	R1	告诉SD卡，下一个是特定应用命令
CMD58(0X3A)	NONE	R3	读取OCR寄存器

表 53.1.2.2 SD 卡部分命令

上表中，大部分的命令是初始化的时候用的。表中的 R1、R3 和 R7 等是 SD 卡的应答信号，每个响应也有规定好的格式，如图 53.1.2.2 所示：

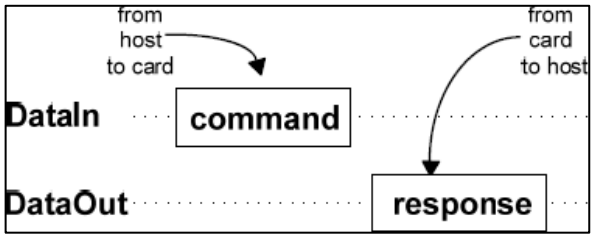


图 53.1.2.2 SD 卡命令传输过程

在规定为有响应的命令下，每发送一个命令，SD 卡都会给出一个应答，以告知主机该命令的执行情况，或者返回主机需要获取的数据，应答可以是 R1~R7，R1 的应答，各位描述如表 53.1.2.3 所示：

	描述	起始位	传输位	命令号	卡状态	CRC7	终止位
R1 响应	Bit	47	46	[45:40]	[39:8]	[7:1]	0
	位宽	1	1	6	32	7	1
	值	"0"	"0"	x	x	x	"1"

表 53.1.2.3 R1 响应

R2~R7 的响应，限于篇幅，我们就不介绍了，但需要注意的是除了 R2 响应是 128 位外，其它的响应都是 48 位，请大家参考 SD 卡 2.0 协议。

53.1.3 卡模式

SD 卡系统(包括主机和 SD 卡)定义了 SD 卡的工作模式，在每个操作模式下，SD 卡都有几种状态，参考表 53.1.3.1，状态之间通过命令控制实现卡状态的切换。

无效模式 (Inactive)	无效状态(Inactive State)
卡识别模 (Card identification mode)	空闲状态(Idle State)
	准备状态(Ready State)
	识别状态(Identification State)
数据传输模式 (Data transfer mode)	待机状态(Stand-by State)
	传输状态(Transfer State)
	发送数据状态(Sending-data State)

	接收数据状态(Receive-data State)
	编程状态(Programming State)
	断开连接状态(Disconnect State)

表 53.1.3.1 SD 卡状态与操作模式

对于我们来说两种有效操作模式：卡识别模式和数据传输模式。在系统复位后，主机处于卡识别模式，寻找总线上可用的 SDIO 设备，对 SD 卡进行数据读写之前需要识别卡的种类：V1.0 标准卡、V2.0 标准卡、V2.0 高容量卡或者不被识别卡；同时，SD 卡也处于卡识别模式，直到被主机识别到，即当 SD 卡在卡识别状态接收到 CMD3 (SEND_RCA)命令后，SD 卡就进入数据传输模式，而主机在总线上所有卡被识别后也进入数据传输模式。

在卡识别模式下，主机会复位所有处于“卡识别模式”的 SD 卡，确认其工作电压范围，识别 SD 卡类型，并且获取 SD 卡的相对地址(卡相对地址较短，便于寻址)。在卡识别过程中，要求 SD 卡工作在识别时钟频率 FOD 的状态下。卡识别模式下 SD 卡状态转换如图 53.1.3.1。

主机上电后，所有卡处于空闲状态，包括当前处于无效状态的卡。主机也可以发送 GO_IDLE_STATE(CMD0)让所有卡软复位从而进入空闲状态，但当前处于无效状态的卡并不会复位。

主机在开始与卡通信前，需要先确定双方在互相支持的电压范围内。SD 卡有一个电压支持范围，主机当前电压必须在该范围可能才能与卡正常通信。SEND_IF_COND(CMD8)命令就是用于验证卡接口操作条件的(主要是电压支持)。卡会根据命令的参数来检测操作条件匹配性，如果卡支持主机电压就产生响应，否则不响应。而主机则根据响应内容确定卡的电压匹配性。CMD8 是 SD 卡标准 V2.0 版本才有的新命令，所以如果主机有接收到响应，可以判断卡为 V2.0 或更高版本 SD 卡。

SD_SEND_OP_COND(ACMD41)命令可以识别或拒绝不匹配它的电压范围的卡。ACMD41 命令的 VDD 电压参数用于设置主机支持电压范围，卡响应会返回卡支持的电压范围。对于对 CMD8 有响应的卡，把 ACMD41 命令的 HCS 位设置为 1，可以测试卡的容量类型，如果卡响应的 CCS 位为 1 说明为高容量 SD 卡，否则为标准卡。卡在响应 ACMD41 之后进入准备状态，不响应 ACMD41 的卡为不可用卡，进入无效状态。ACMD41 是应用特定命令，发送该命令之前必须先发 CMD55。

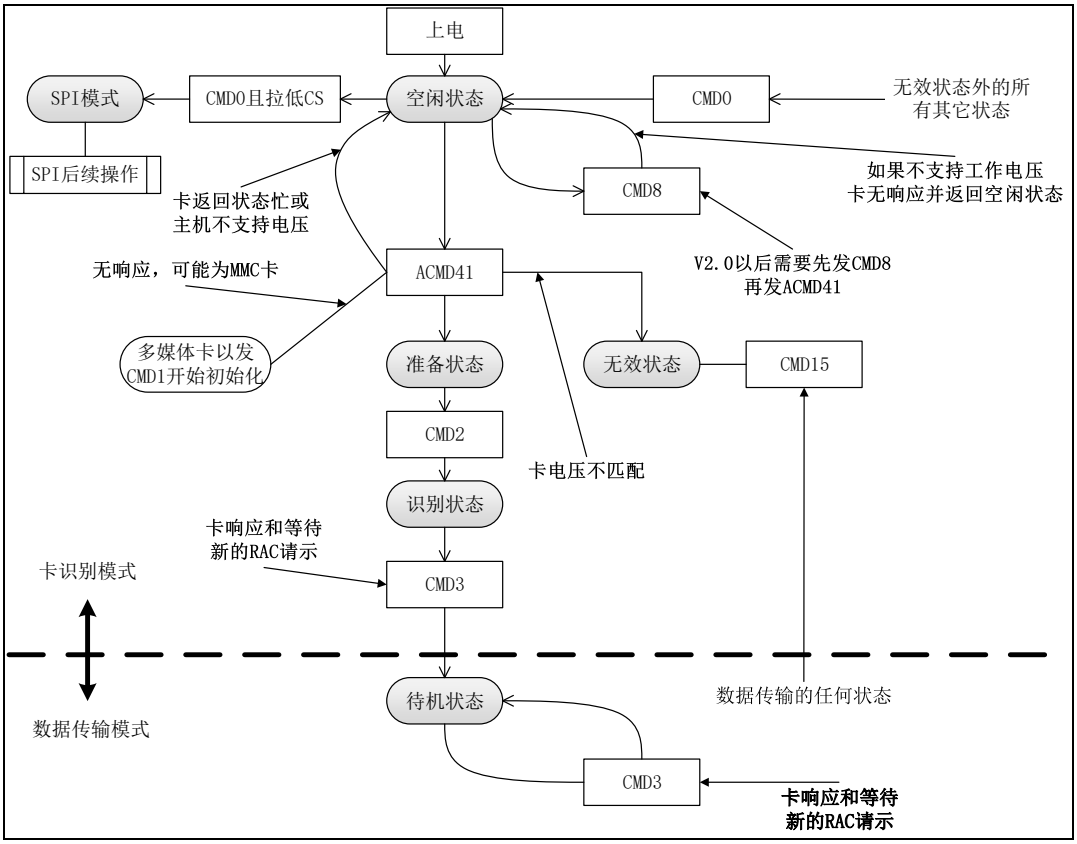


图 53.1.3.2 卡识别模式状态转换图

ALL_SEND_CID(CMD2)用来控制所有卡返回它们的卡识别号(CID)，处于准备状态的卡在发送 CID 之后就进入识别状态。之后主机就发送 SEND_RELATIVE_ADDR(CMD3)命令，让卡自己推荐一个相对地址(RCA)并响应命令。这个 RCA 是 16bit 地址，而 CID 是 128bit 地址，使用 RCA 简化通信。卡在接收到 CMD3 并发出响应后就进入数据传输模式，并处于待机状态，主机在获取所有卡 RCA 之后也进入数据传输模式。

53.1.4 数据模式

在数据模式下我们可以对 SD 卡的存储块进行读写访问操作。SD 卡上电后默认以一位数据总线访问，可以通过指令设置为宽总线模式，可以同时使有 4 位总线并行读写数据，这样对于支持宽总线模式的接口（如：SDIO 和 QSPI 等）都能加快数据操作速度。

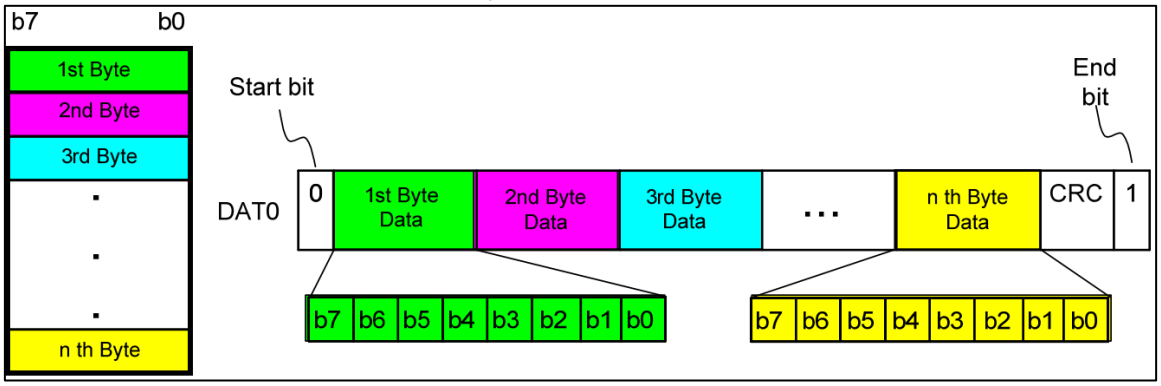


图 53.1.4.1 1 位数据线传输 8bit 的数据流格式

SD 卡有两种数据模式，一种是常规的 8 位宽，即一次按一字节传输，另一种是一次按 512 字节传输，我们只介绍前面一种。当按 8-bit 连续传输时，每次传输从最低字节开始，每字节从最高位(MSB)开始发送，当使用一条数据线时，只能通过 DAT0 进行数据传输，那它的数据传输结构如图 53.1.4.1 1 所示。

当使用 4 线模式传输 8-bit 结构的数据时，数据仍按 MSB 先发送的原则，DAT[3:0]的高位发送高数据位，低位发送低数据位。硬件支持的情况下，使用 4 线传输可以提升传输速率。

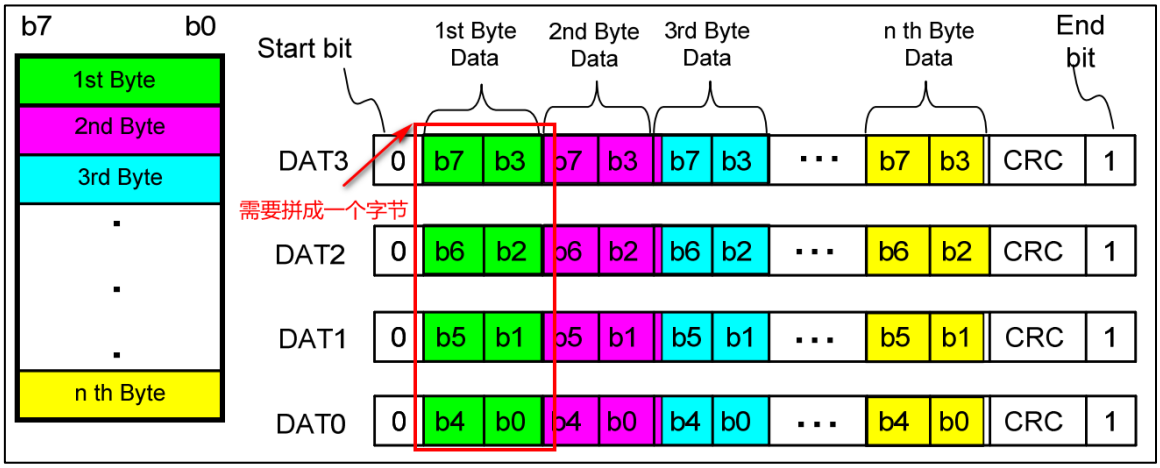


图 53.1.4.2 4 位数据线传输 8bit 格式的数据流格式

只有 SD 卡系统处于数据传输模式下才可以进行数据读写操作。数据传输模式下可以将主机 SD 时钟频率设置为 FPP，默认最高为 25MHz，频率切换可以通过 CMD4 命令来实现。数据传输模式下，SD 卡状态转换过程见图 53.1.4.3。

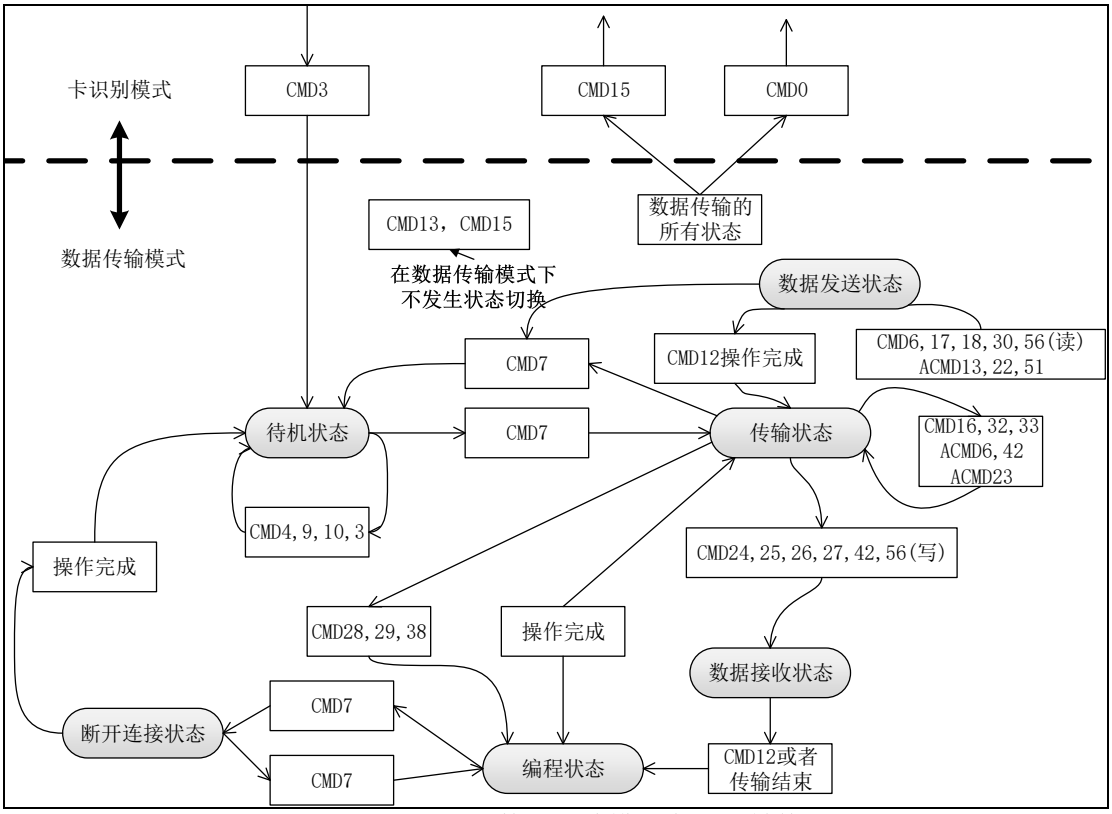


图 53.1.4.3 数据传输模式卡状态转换

CMD7 用来选定和取消指定的卡，卡在待机状态下还不能进行数据通信，因为总线上可能有多个卡都是处于待机状态，必须选择一个 RCA 地址目标卡使其进入传输状态才可以进行数据通信。同时通过 CMD7 命令也可以让已经被选择的目标卡返回到待机状态。

数据传输模式下的数据通信都是主机和目标卡之间通过寻址命令点对点进行的。卡处于传输状态下可以通过命令对卡进行数据读写、擦除。CMD12 可以中断正在进行的数据通信，让卡返回到传输状态。CMD0 和 CMD15 会中止任何数据编程操作，返回卡识别模式，注意谨慎使用，不当操作可能导致卡数据被损坏。

至此，我们已经介绍了 SD 卡操作的一些知识，并知道了 SD 卡操作的命令、响应和数据

传输等状态，接下来我们来分析实际的硬件接口如何向 SD 卡发送我们需要的数据。

53.2 SDMMC 简介

下面简单介绍 STM32H743 的 SDMMC 接口，包括：主要功能及框图、时钟、命令与响应和相关寄存器简介等，最后，我们将介绍 SD 卡的初始化流程。

53.2.1 SDMMC 主要功能及框图

STM32H743 的 SDMMC 控制器支持多媒体卡（MMC 卡）、SD 存储卡、SDIO 卡等设备。SDMMC 的主要功能如下：

- 与多媒体卡系统规格书版本 4.51 全兼容。支持三种不同的数据总线模式：1 位(默认)、4 位和 8 位。
- 与较早的多媒体卡系统规格版本全兼容(向前兼容)。
- 与 SD 存储卡规格版本 4.1 全兼容。
- 与 SD I/O 卡规格版本 4.0 全兼容：支持两种不同的数据总线模式：1 位(默认)和 4 位。
- 8 位总线模式下数据传输速率可达 208MHz（但受限于 IO 口速度）。
- 数据和命令输出使能信号，用于控制外部双向驱动器。

STM32H743 的 SDMMC 控制器包含 4 个部分：AHB 总线接口、SDMMC 适配器模块、内部 DMA 模块（IDMA）和延迟模块（DLYB），其功能框图如图 53.2.1.1 所示：

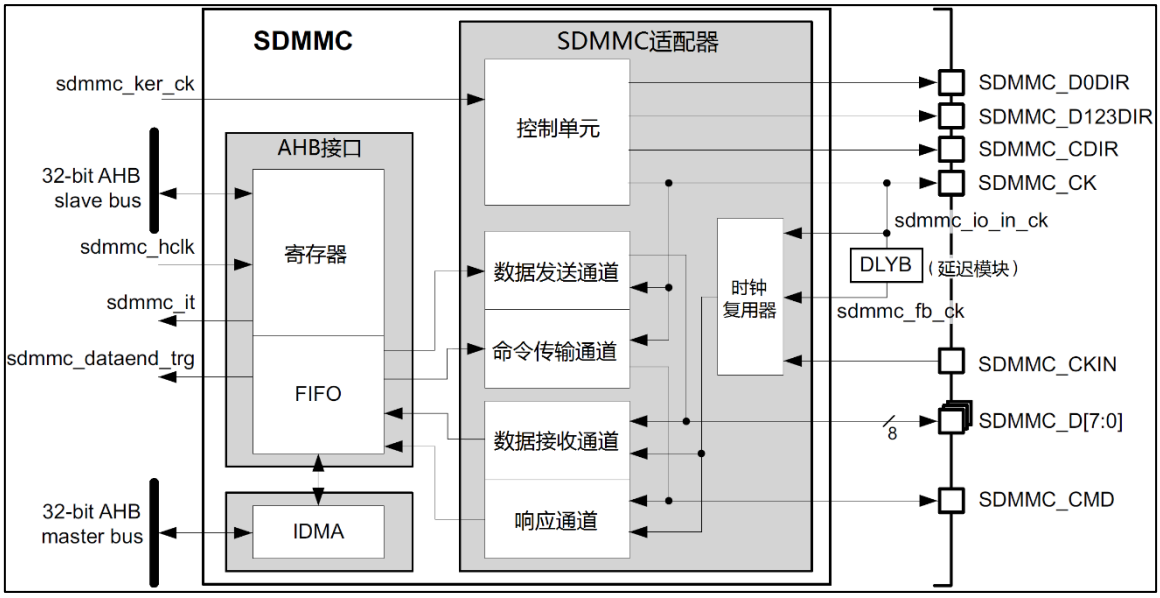


图 53.2.1.1 STM32H743 的 SDMMC 控制器功能框图

图中，右侧的信号为 SDMMC 对外引脚（信号），如表 53.2.1.1 所示：

信号名字	信号类型	说明
SDMMC_D0DIR	输出	SDMMC_D0 方向控制信号
SDMMC_D123DIR	输出	SDMMC_D1/D2/D3 方向控制信号
SDMMC_CDIR	输出	SDMMC_CMD 方向控制信号
SDMMC_CK	输出	时钟输出
SDMMC_CKIN	输入	时钟输入
SDMMC_D[7:0]	输入/输出	数据线
SDMMC_CMD	输入/输出	命令/响应线

表 53.2.1.1 SDMMC 引脚说明

在 SDIO 模式下我们只需要用到：SDMMC_CK、SDMMC_D[3:0]、SDMMC_CMD 这几根线即可正常驱动 SD 卡，复位后默认情况下 SDMMC_D0 用于数据传输。初始化后主机可以改变数据总线的宽度（通过 ACMD6 命令设置）。

如果一个多媒体卡接到了总线上，则 SDMMC_D0、SDMMC_D[3:0]或 SDMMC_D[7:0]可以用于数据传输。MMC 版本 V3.31 和之前版本的协议只支持 1 位数据线，所以只能用 SDMMC_D0（为了通用性考虑，在程序里面我们只要检测到是 MMC 卡就设置为 1 位总线数据）。

如果一个 SD 或 SD I/O 卡接到了总线上，可以通过主机配置数据传输使用 SDMMC_D0 或 SDMMC_D[3:0]。所有的数据线都工作在推挽模式。

SDMMC_CMD 有两种操作模式：

① 用于初始化时的开路模式(仅用于 MMC 版本 V3.31 或之前版本)

② 用于命令传输的推挽模式(SD/SD I/O 卡和 MMC V4.2 在初始化时也使用推挽驱动)

SDMMC 在 SD/SDIO 操作模式下有多种速度模式，如表 53.2.1.2 所示：

SDIO 总线速度模式	最大速度（MB/s）	最大时钟频率（Mhz）	信号电平（V）
DS（默认速度）	12.5	25	3.3
HS（高速）	25	50	3.3
SDR12	12.5	25	1.8
SDR25	25	50	1.8
DDR50	50	50	1.8
SDR50	50	100	1.8
SDR104	104	208	1.8

表 53.2.1.2 SDMMC 在 SD/SDIO 模式下的速度模式

上表中：SDR 表示单倍速率（时钟单边沿采样），DDR 表示双倍速率（时钟双边沿采样），由表可知，STM32H7 支持最高的 SDR104 总线速度模式，理论上传输速度最高可达 104MB/s（实际上由于 IO 口速度无法达到 208Mhz，所以，是达不到 104MB/s 的传输速度的）。

这里的 DS、HS、SDR12、SDR25、DDR50、SDR50、SDR104 等速度模式，实际上是超高速 I 卡（UHS-I，详见光盘→ 7，硬件资料→6，SD 卡资料→ SD4.0 协议标准完整版[中文版].pdf）所支持的一些速度等级，SD 卡需要通过 CMD6 指令来设置相应的速度等级（仅 SDHC 和 SDXC 卡支持 CMD6 命令）。

我们的 SD 卡接口是 3.3V 供电的，所以，仅支持 DS 和 HS 模式（最高 25MB/s 的传输速度）。接下来的 SDMMC 相关介绍，我们也仅以这两个模式为基础进行介绍。

53.2.2 SDMMC 的时钟

从图 53.2.1.1 我们可以看到 SDMMC 总共有 3 个时钟，分别是：

卡时钟（SDMMC_CK）：每个时钟周期在命令和数据线上传输 1 位命令或数据。在 3.3V 信号电平下，该时钟频率最大可以达到 50Mhz。

SDMMC 适配器内核时钟（sdmmc_ker_ck）：该时钟用于驱动 SDMMC 适配器，来自 pll1_q_ck/pll2_r_ck（由 RCC_D1CCIPR 寄存器的 SDMMCSEL 位选择），默认选择来自 pll1_q_ck，其频率一般为 200Mhz，并用于产生 SDMMC_CK 时钟。

AHB3 总线接口时钟（sdmmc_hclk）：该时钟用于驱动 SDMMC 的 AHB 总线接口，来自 AHB3，其频率一般为 200Mhz。

SD 卡时钟（SDMMC_CK），根据卡类型的不同，可能有好几个区间，这就涉及到时钟频率的设置，SDMMC_CK 与 sdmmc_ker_ck 的关系（CLKDIV 不等于 0 时）为：

SDMMC_CK = sdmmc_ker_ck / (2*CLKDIV)

其中，sdmmc_ker_ck 一般来自 pll1_q_ck，为 200Mhz，而 CLKDIV 则是分频系数，可以通过 SDMMC 的 SDMMC_CLKCR 寄存器进行设置（确保 SDMMC_CK 不超过卡的最大操作频率）。注意，以上公式，是时钟分频器不旁路时的计算公式，当 CLKDIV 等于 0 时，SDMMC_CK 直接等于 sdmmc_ker_ck。

这里要提醒大家，在 SD 卡刚刚初始化的时候，其时钟频率（SDMMC_CK）是不能超过 400Khz 的，否则可能无法完成初始化。在初始化以后，就可以设置时钟频率到最大了（但不可超过 SD 卡的最大操作时钟频率）。

53.2.3 SDMMC 的命令与响应

SDMMC 的命令分为应用相关命令（ACMD）和通用命令（CMD）两部分，应用相关命令（ACMD）的发送，必须先发送通用命令（CMD55），然后才能发送应用相关命令（ACMD）。

SDMMC 的所有命令和响应都是通过 SDMMC_CMD 引脚传输的，任何命令的长度都是固定为 48 位，SDMMC 的命令格式如表 53.2.3.1 所示：

位的位置	宽度	值	说明
47	1	0	起始位
46	1	1	传输位
[45:40]	6	-	命令索引
[39:8]	32	-	参数
[7:1]	7	-	CRC7
0	1	1	结束位

表 53.2.3.1 SDMMC 命令格式

所有的命令都是由 STM32H743 发出，其中开始位、传输位、CRC7 和结束位由 SDMMC 硬件控制，我们需要设置的就只有命令索引和参数部分。其中命令索引（如 CMD0，CMD1 之类的）在 SDMMC_CMD 寄存器里面设置，命令参数则由寄存器 SDMMC_ARG 设置。

一般情况下，选中的 SD 卡在接收到命令之后，都会回复一个应答（注意 CMD0 是无应答的），这个应答我们称之为响应，响应也是在 CMD 线上串行传输的。STM32H743 的 SDMMC 控制器支持 2 种响应类型，即：短响应（48 位）和长响应（136 位），这两种响应类型都带 CRC 错误检测（注意不带 CRC 的响应应该忽略 CRC 错误标志，如 CMD1 的响应）。

短响应的格式如表 53.2.3.2 所示：

位的位置	宽度	值	说明
47	1	0	起始位
46	1	0	传输位
[45:40]	6	-	命令索引
[39:8]	32	-	参数
[7:1]	7	-	CRC7（或 1111111）
0	1	1	结束位

表 53.2.3.2 SDMMC 短响应格式

长响应的格式如表 53.2.3.3 所示：

位的位置	宽度	值	说明
135	1	0	起始位
134	1	0	传输位
[133:128]	6	111111	保留
[127:1]	127	-	CID 或 CSD（包括内部 CRC7）
0	1	1	结束位

表 53.2.3.3 SDMMC 长响应格式

同样，硬件为我们滤除了开始位、传输位、CRC7 以及结束位等信息，对于短响应，命令索引存放在 SDMMC_RESPCMD 寄存器，参数则存放在 SDMMC_RESP1 寄存器里面。对于长响应，则仅留 CID/CSD 位域，存放在 SDMMC_RESP1~SDMMC_RESP4 等 4 个寄存器。

SD 存储卡总共有 5 类响应（R1、R2、R3、R6、R7），我们这里以 R1 为例简单介绍一下。R1（普通响应命令）响应输入短响应，其长度为 48 位，R1 响应的格式如表 53.2.3.4 所示：

位的位置	宽度（位）	值	说明
47	1	0	起始位
46	1	0	传输位
[45:40]	6	X	命令索引
[39:8]	32	X	卡状态
[7:1]	7	X	CRC7
0	1	1	结束位

表 53.2.3.4 R1 响应格式

在收到 R1 响应后，我们可以从 SDMMC_RESPCMD 寄存器和 SDMMC_RESP1 寄存器分别读出命令索引和卡状态信息。关于其他响应的介绍，请大家参考光盘:《SD 卡 2.0 协议.pdf》或《STM32H7xx 参考手册_V7（英文版）.pdf》第 54 章。

最后，我们看看数据在 SDMMC 控制器与 SD 卡之间的传输。对于 SDI/SDMMC 存储器，数据是以数据块的形式传输的，而对于 MMC 卡，数据是以数据块或者数据流的形式传输。本节我们只考虑数据块形式的数据传输。

SDMMC（多）数据块读操作，如图 53.2.3.1 所示：

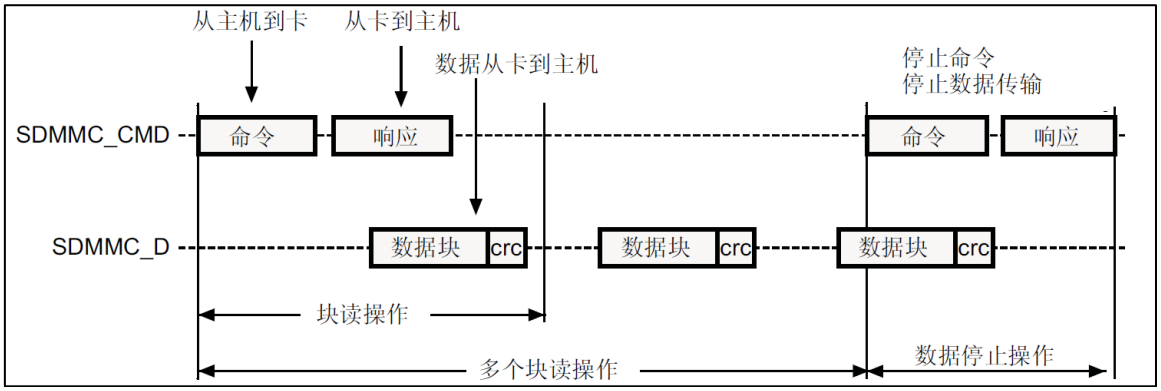


图 53.2.3.1 SDMMC（多）数据块读操作

从上图，我们可以看出，从机在收到主机相关命令后，开始发送数据块给主机，所有数据块都带有 CRC 校验值（CRC 由 SDMMC 硬件自动处理），单个数据块读的时候，在收到 1 个数据块以后即可以停止了，不需要发送停止命令（CMD12）。但是多块数据读的时候，SD 卡将一直发送数据给主机，直到接到主机发送的 STOP 命令（CMD12）。

SDMMC（多）数据块写操作，如图 53.2.3.2 所示：

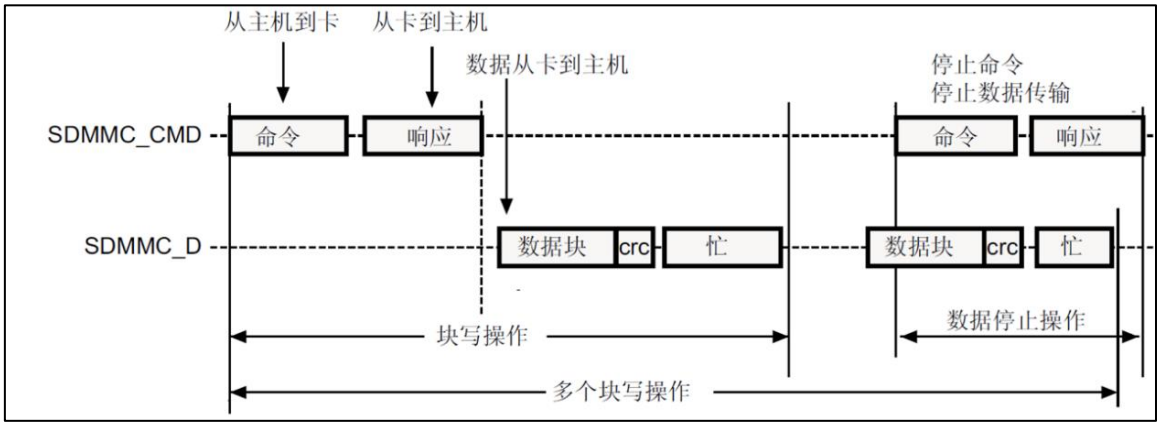


图 53.2.3.2 SDMMC（多）数据块写操作

数据块写操作同数据块读操作基本类似，只是数据块写的时候，多了一个繁忙判断，新的

数据块必须在 SD 卡非繁忙的时候发送。这里的繁忙信号由 SD 卡拉低 SDMMC_D0，以表示繁忙，SDMMC 硬件自动控制，不需要我们软件处理。

SDMMC 的命令与响应就为大家介绍到这里。

53.2.4 SDMMC 寄存器

- **SDMMC 电源控制寄存器 (SDMMC_POWER)**

首先，我们来看 SDMMC 电源控制寄存器（SDMMC_POWER），该寄存器定义如图 53.2.4.1 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DIR POL	V SWITCH EN	V SWITCH	PWRCTRL[1:0]	
											rw	rw	rw	rw	rw

图 53.2.4.1 SDMMC_POWER 寄存器

该寄存器我们只关心 PWRCTRL[1:0]这两个位：00，掉电，停止为卡提供时钟；01，保留；10，上电周期；11，上电，为卡提供时钟。复位后，PWRCTRL[1:0]的值默认为 00，所以 SDMMC 的电源是关闭的，我们要启用 SDMMC，第一步就是要设置该寄存器最低 2 个位均为 1，让 SDMMC 上电，开启卡时钟。

- **SDMMC 时钟控制寄存器 (SDMMC_CLKCR)**

第二个是 SDMMC 时钟控制寄存器（SDMMC_CLKCR），该寄存器主要用于设置 SDMMC_CK 的分配系数，开关等，并可以设置 SDMMC 的数据位宽，该寄存器的定义如图 53.2.4.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SELCLKRX[1:0]		BUS SPEED	DDR	HWFC_EN	NEG EDGE
										rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WIDBUS[1:0]		Res.	PWRSAV	Res.	Res.	CLKDIV[9:0]									
rw	rw		rw			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 53.2.4.2 SDMMC_CLKCR 寄存器

CLKDIV[9:0]位，用于控制时钟分频系数，SDMMC_CK 的时钟频率由这 10 个位控制，其计算公式为：SDMMC_CK = sdmmc_ker_ck / (2*CLKDIV[9:0])。

WIDBUS[1:0]位，用于控制总线宽度。00，1 位宽度；01，4 位宽度；10，8 位宽度。我们一般使用 4 位宽度通信（WIDBUS[1:0]=01）。

DDR 位，用于控制数据采样率。0，表示 SDR 单倍采样（1 个时钟传输 1 个位）；1，表示 DDR 双倍采样（1 个时钟传输 2 个位）。我们一般设置为 0。

BUSSPEED 位，用于设置总线速度模式。0，表示使用 DS/HS/SDR12/SDR25 等总线速度模式；1，表示使用 SDR50/DDR50/SDR104 等总线速度模式。我们一般设置为 0。

该寄存器的其他位，详见《STM32H7xx 参考手册_V7（英文版）.pdf》第 2458 页，第 55.9.2 节。

- **SDMMC 参数寄存器 (SDMMC_ARGR)**

第三个是 SDMMC 参数制寄存器（SDMMC_ARGR），该寄存器的定义如图 53.2.4.3 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CMDARG[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CMDARG[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:0 **CMDARG[31:0]**：命令参数 (Command argument)。
只有在禁止 CPSM (CPSMEN=0) 时才能通过固件写入这些位。
作为命令消息的一部分发送给卡的命令参数。如果命令包含参数，则在将命令写入到命令寄存器之前，必须将参数加载到此寄存器中。

图 53.2.4.3 SDMMC_ARGR 寄存器

该寄存器比较简单，就是一个 32 位寄存器，用于存储命令参数，不过需要注意的是，必须在写命令之前先写这个参数寄存器！

- **SDMMC 命令响应寄存器 (SDMMC_RESPCMDR)**

第四个是 SDMMC 命令响应寄存器（SDMMC_RESPCMDR），该寄存器的定义如图 53.2.4.4 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RESPCMD[5:0]					
										r	r	r	r	r	r

位 5:0 **RESPCMD[5:0]**：响应命令索引 (Response command index)
只读位域。包含接收到的最后一个命令响应的命令索引。

图 53.2.4.4 SDMMC_RESPCMDR 寄存器

该寄存器为 32 位，但只有低 6 位有效，比较简单，用于存储最后收到的命令响应中的命令索引。如果传输的命令响应不包含命令索引，则该寄存器的内容不可预知。

- **SDMMC 响应 1..4 寄存器(SDMMC_RESPxR) (x = 1..4)**

第五个是 SDMMC 响应 1..4 寄存器 (SDMMC_RESPxR) (x = 1..4)，该寄存器的定义如图 53.2.4.5 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CARDSTATUSx[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CARDSTATUSx[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

位 31:0 **CARDSTATUSx[31:0]**：请参见*表 447*。

卡状态为 32 位或 128 位，具体取决于响应类型。

图 53.2.4.5 SDMMC_RESPxR 寄存器

该寄存器组总共由 4 个 32 位寄存器组成，用于存放接收到的卡响应部分信息。如果收到短响应，则数据存放在 SDMMC_RESP1 寄存器里面，其他三个寄存器没有用到。而如果收到长响应，则依次存放在 SDMMC_RESP1~ SDMMC_RESP4 里面，如表 53.2.4.1 所示：

寄存器(1)	短响应	长响应
SDMMC_RESP1R	卡状态 [31:0]	卡状态 [127:96]
SDMMC_RESP2R	全 0	卡状态 [95:64]
SDMMC_RESP3R	全 0	卡状态 [63:32]
SDMMC_RESP4R	全 0	卡状态 [31:0](2)

表 53.2.4.1 响应类型和 SDMMC_RESPx 寄存器

- **SDMMC 命令寄存器 (SDMMC_CMDR)**

第六个是 SDMMC 命令寄存器（SDMMC_CMDR），该寄存器各位定义如图 53.2.4.6 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CMD SUS PEND
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BOOT EN	BOOT MODE	DT HOLD	CPSM EN	WAITP END	WAIT INT	WAITRESP[1:0]		CMD STOP	CMD TRANS	CMDINDEX[5:0]					
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 53.2.4.6 SDMMC_CMDR 寄存器

CMDINDEX[5:0]位，用于表示命令索引号，比如发送 CMD1，其值为 1，索引就设置为 1。

WAITRESP[1:0]位，用于设置等待响应，指示 CPSM 是否需要等待，以及等待类型等。这里的 CPSM，即命令通道状态机，我们就不详细介绍了，请参阅《STM32H7xx 参考手册_V7（英文版）.pdf》，有详细介绍。

WAITINT 位，用于设置是否等待中断，我们一般不用等待中断，因此设置该位为 0。

CPSMEN 位，用于设置是否使能 CPSM 命令通道状态机，我们设置为 1，使能 CPSM。

该寄存器的其他位介绍，详见《STM32H7xx 参考手册_V7（英文版）.pdf》。

- **SDMMC 数据定时器寄存器 (SDMMC_DTIMER)**

第七个是 SDMMC 数据定时器寄存器（SDMMC_DTIMER），寄存器各位定义如下图所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATATIME[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATATIME[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:0 **DATATIME[31:0]**：数据和 R1b 繁忙超时周期 (Data and R1b busy timeout period)
只有在 CPSM 和 DPSM 未激活时（CPSMACT = 0 且 DPSMACT = 0），才能写入该位。
以卡总线时钟周期表示的数据和 R1b 繁忙超时周期。

图 53.2.4.7 SDMMC_DTIMER 寄存器

该寄存器用于存储以卡总线时钟（SDMMC_CK）为周期的数据超时时间，计数器将从 SDMMC_DTIMER 寄存器加载数值，并在数据通道状态机(DPSM)进入 Wait_R 或繁忙状态时进行递减计数，当 DPSM 处在这些状态时，如果计数器减为 0，则设置超时标志。这里的 DPSM，即数据通道状态机，类似 CPSM，详细请参考《STM32H7xx 参考手册_V7（英文版）.pdf》。注意：在写入数据控制寄存器，进行数据传输之前，必须先写入该寄存器（SDMMC_DTIMER）和数据长度寄存器（SDMMC_DLEN）！

- **SDMMC 数据长度寄存器 (SDMMC_DLENR)**

第八个是 SDMMC 数据长度寄存器（SDMMC_DLENR），该寄存器各位定义如图 53.2.4.8 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	DATALENGTH[24:16]								
							rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATALENGTH[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 24:0 **DATALENGTH[24:0]**：数据长度值 (Data length value)
只有在未激活 DPSM (DPSMACT = 0) 时才能通过固件写入此寄存器。
要传输的数据字节数量。
当 DDR = 1 时，DATALENGTH 被截断为 2 的倍数。（不传输最后一个奇数字节）
当 DATALENGTH = 0 时，不会传输任何数据，当通过 CPSMEN 和 CMDTRANS = 1 请求时，也不会传输任何命令。DTEN 和 CPSMEN 均清零。

图 53.2.4.8 SDMMC_DLENR 寄存器

该寄存器低 25 位有效，用于设置需要传输的数据字节长度。对于块数据传输，该寄存器的数值，必须是数据块长度（通过 SDMMC_DCTRL 设置）的倍数。

- **SDMMC 数据控制寄存器 (SDMMC_DCTRL)**

第九个是 SDMMC 数据控制寄存器（SDMMC_DCTRL），该寄存器各位定义如图 53.2.4.9 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	FIFO RST	BOOT ACKEN	SDIO EN	RW MOD	RW STOP	RW START	DBLOCKSIZE[3:0]				DTMODE		DTDIR	DTEN
		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 53.2.4.9 SDMMC_DCTRL 寄存器

DTEN 位，用于使能数据传输。0，禁止数据传输；1，开始数据传输。

DTDIR 位，用于控制数据传输方向。0，数据从主机传到卡；1，数据从卡传到主机。

DTMODE[1:0]位，用于设置数据传输模式；00，指定块数的块传输；01，SDIO 多数据传输；10，eMMC 数据流传输（仅支持 1 位数据线）；11，不限数块传输（由 STOP_TRANSMISSION 命令停止传输）。一般我们使用 00 模式即可（指定块数块传输）。

DBLOCKSIZE[3:0]位，用于设置数据块大小。取值范围为：0~14，表示单个数据块的长度为：2^DBLOCKSIZE[3:0]字节。我们一般设置 DBLOCKSIZE[3:0]=9，表示块大小为 512 字节。

该寄存器的其他位介绍，详见《STM32H7xx 参考手册_V7（英文版）.pdf》。

接下来，我们介绍几个位定义十分类似的寄存器，他们是：状态寄存器（SDMMC_STAR）、清除中断寄存器（SDMMC_ICR）和中断屏蔽寄存器（SDMMC_MASKR），这三个寄存器每个位的定义都相同，只是功能各有不同。所以可以一起介绍，以状态寄存器（SDMMC_STAR）为例，该寄存器各位定义如图 53.2.4.10 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	IDMA BTC	IDMA TE	CK STOP	VSW END	ACK TIME OUT	ACK FAIL	SDIOIT	BUSY D0END	BUSY D0	RX FIFOE	TX FIFOE	RX FIFOF	TX FIFOF
			r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RX FIFO HF	TX FIFO HE	CPSM ACT	DPSM ACT	D ABORT	DBCK END	DHOLD	DATA END	CMD SENT	CMDR END	RX OVERR	TX UNDER R	D TIME OUT	C TIME OUT	DCRC FAIL	CCRC FAIL
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

图 53.2.4.10 SDMMC_STAR 寄存器位定义

CCRCFAIL 位，已接收命令 CRC 检验错误。

DCRCFAIL 位，已发送/接收数据块 CRC 检验错误。

CTIMEOUT 位，命令响应超时。

DTIMEOUT 位，数据超时。

TXUNDERR 位，发送 FIFO 下溢错误。

RXUNDERR 位，接收 FIFO 上溢错误。

CMDREND 位，已接受命令响应。

CMDSENT 位，命令已发送。

DATAEND 位，数据接收。

DHOLD 位，数据发送保持。

DBCKEND 位，已发送/接收数据块。

DABORT 位，被 CMD12 退出数据传输。

DPSMACT 位，数据通道状态机启动。

CPSMACT 位，命令通道状态机启动。

TXFIFOHE 位，发送 FIFO 半空。

RXFIFOHF 位，接收 FIFO 半满。

TXFIFOF 位，发送 FIFO 满。

RXFIFOF 位，接收 FIFO 满。

TXFIFOE 位，发送 FIFO 空。

RXFIFOE 位，接收 FIFO 空。

BUSYD0 位，SDMMC_D0 信号忙。

BUSYD0END 位，SDMMC_D0 忙结束（非忙）。

SDIOIT 位，接收到 SDIO 中断。
ACKFAIL 位，接收启动应答失败。
ACKTIMEOUT 位，接收启动应答超时。
VSWEND 位，电压切换电路切换完成。
CKSTOP 位，在电压切换的时候，SDMMC_CK 停止输出。
IDMATE 位，IDMA 发送错误。
IDMABTC 位：IDMA 缓冲区传输完成。

状态寄存器可以用来查询 SDMMC 控制器的当前状态，以便处理各种事务。比如 SDMMC_STA 的位 2 表示命令响应超时，说明 SDMMC 的命令响应出了问题。我们通过设置 SDMMC_ICR 的位 2 则可以清除这个超时标志，而设置 SDMMC_MASK 的位 2，则可以开启命令响应超时中断，设置为 0 关闭。

状态寄存器（SDMMC_STA）、清除中断寄存器（SDMMC_ICR）和中断屏蔽寄存器（SDMMC_MASK）等三个寄存器的详细介绍，请参考：《STM32H7xx 参考手册_V7（英文版）.pdf》。

最后，我们向大家介绍 SDMMC 的数据 FIFO 寄存器（SDMMC_FIFO），数据 FIFO 寄存器包括接收和发送 FIFO，他们由一组连续的 32 个地址上的 32 个寄存器组成，CPU 可以使用 FIFO 读写多个操作数。例如我们要从 SD 卡读数据，就必须读 SDMMC_FIFO 寄存器，要写数据到 SD 卡，则要写 SDMMC_FIFO 寄存器。SDMMC 将这 32 个地址分为 16 个一组，发送接收各占一半。而我们每次读写的时候，最多就是读取发送 FIFO 或写入接收 FIFO 的一半大小的数据，也就是 8 个字（32 个字节），**这里特别提醒，我们操作 SDMMC_FIFO（不论读出还是写入）必须是以 4 字节对齐的内存进行操作，否则将导致出错！**

至此，SDMMC 的相关寄存器介绍，我们就介绍完了。还有几个不常用的寄存器，我们没有介绍到，请大家参考《STM32H7xx 参考手册_V7（英文版）.pdf》第 55 章相关章节。

53.3 SD 卡初始化流程

53.3.1 SDIO 模式下的 SD 卡初始化

这一节，我们来看看 SD 卡的初始化流程，要实现 SDMMC 驱动 SD 卡，最重要的步骤就是 SD 卡的初始化，只要 SD 卡初始化完成了，那么剩下的（读写操作）就简单了，所以我们这里重点介绍 SD 卡的初始化。从《SD 卡 2.0 协议》（见光盘资料）文档，我们得到 SD 卡初始化流程图如图 53.3.1.1 所示：

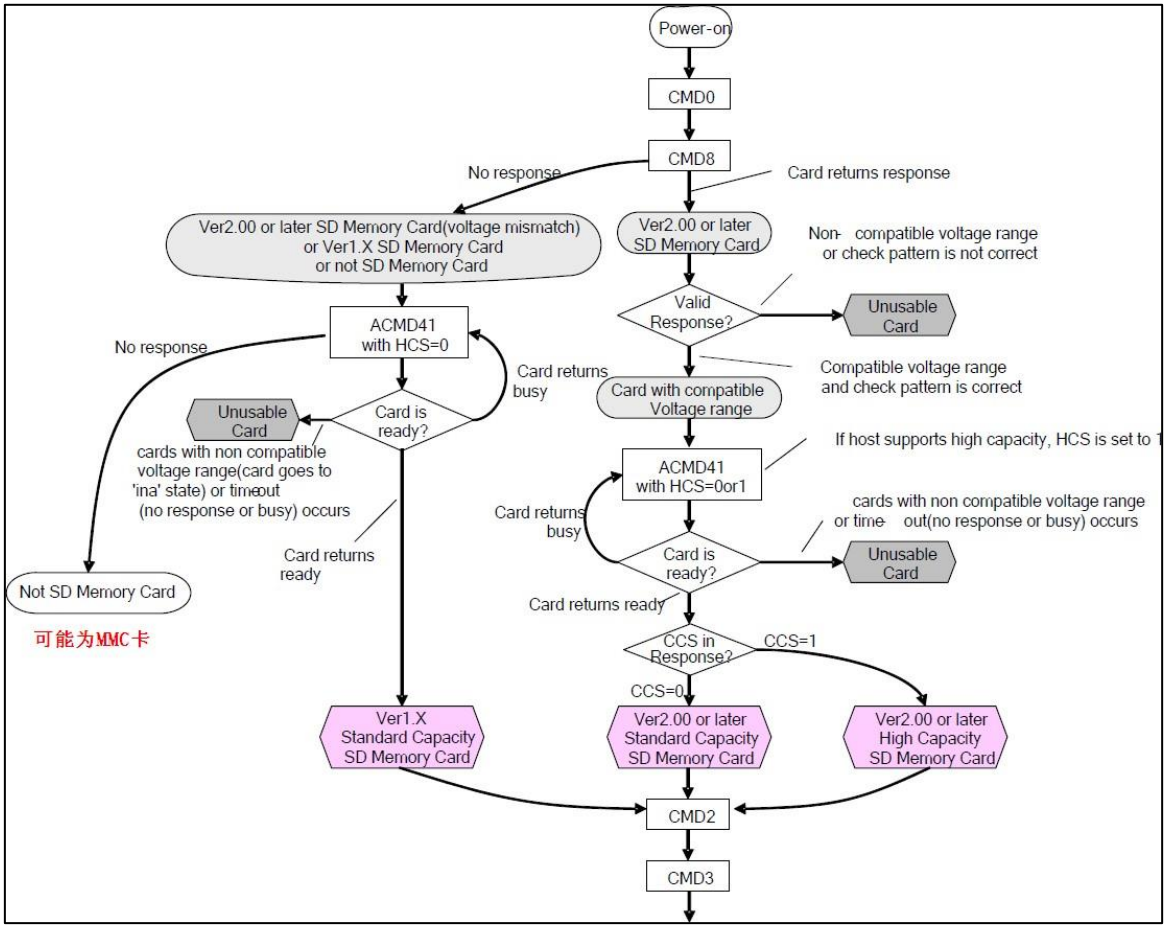


图 53.3.1.1 SD 卡初始化流程（Card Initialization and Identification Flow (SD mode)）

从图中，我们看到，不管什么卡（这里我们将卡分为 4 类：SD2.0 高容量卡（SDHC，最大 32G），SDv2.0 标准容量卡（SDSC，最大 2G），SD1.x 卡和 MMC 卡），首先我们要执行的是卡上电（需要设置 SDIO_POWER[1:0]=11），上电后发送 CMD0，对卡进行软复位，之后发送 CMD8 命令，用于区分 SD 卡 2.0，只有 2.0 及以后的卡才支持 CMD8 命令，MMC 卡和 V1.x 的卡，是不支持该命令的。CMD8 的格式如表 53.3.1.1 所示：

位序	47	46	[45:40]	[39:20]	[19:16]	[15:8]	[7:1]	0
占用位	1	1	6	20	4	8	7	1
命令值	‘0’	‘1’	‘001000’	‘00000h’	x	x	x	‘1’
描述	起始位	传输位	命令索引	保留位	电源(VHS)	校验	CRC7	结束位

表 53.3.1.1 CMD8 命令格式

这里，我们需要在发送 CMD8 的时候，通过其带的参数我们可以设置 VHS 位，以告诉 SD 卡，主机的供电情况，VHS 位定义如表 53.3.1.2 所示：

Voltage Supplied	Value Definition
0000b	Not Defined
0001b	2.7-3.6V
0010b	Reserved for Low Voltage Range
0100b	Reserved
1000b	Reserved
Others	Not Defined

表 53.3.1.2 VHS 位定义

这里我们使用参数 0X1AA，即告诉 SD 卡，主机供电为 2.7~3.6V 之间，如果 SD 卡支持 CMD8，且支持该电压范围，则会通过 CMD8 的响应（R7）将参数部分原本返回给主机，如果

不支持 CMD8，或者不支持这个电压范围，则不响应。

在发送 CMD8 后，发送 ACMD41（注意发送 ACMD41 之前要先发送 CMD55），来进一步确认卡的操作电压范围，并通过 HCS 位来告诉 SD 卡，主机是不是支持高容量卡（SDHC）。ACMD41 的命令格式如表 53.3.5.3 所示：

ACMD INDEX	type	argument	resp	abbreviation	command description
ACMD41	bcr	[31]reserved bit [30]HCS(OCR[30]) [29:24]reserved bits [23:0] V_{DD} Voltage Window(OCR[23:0])	R3	SD_SEND_OP_COND	Sends host capacity support information (HCS) and asks the accessed card to send its operating condition register (OCR) content in the response on the CMD line. HCS is effective when card receives SEND_IF_COND command. Reserved bit shall be set to '0'. CCS bit is assigned to OCR[30].

表 53.3.5.3 ACMD41 命令格式

ACMD41 得到的响应（R3）包含 SD 卡 OCR 寄存器内容，OCR 寄存器内容定义如表 53.3.5.4 所示：

OCR bit position	OCR Fields Definition	
0-6	reserved	
7	Reserved for Low Voltage Range	
8-14	reserved	
15	2.7-2.8	VDD Voltage Window
16	2.8-2.9	
17	2.9-3.0	
18	3.0-3.1	
19	3.1-3.2	
20	3.2-3.3	
21	3.3-3.4	
22	3.4-3.5	
23	3.5-3.6	
24-29	reserved	
30	Card Capacity Status (CCS)[1]	
31	Card power up status bit (busy)[2]	

1) This bit is valid only when the card power up status bit is set.
2) This bit is set to LOW if the card has not finished the power up routine.

表 53.3.5.4 OCR 寄存器定义

对于支持 CMD8 指令的卡，主机通过 ACMD41 的参数设置 HCS 位为 1，来告诉 SD 卡主机支持 SDHC 卡，如果设置为 0，则表示主机不支持 SDHC 卡，SDHC 卡如果接收到 HCS 为 0，则永远不会反回卡就绪状态。对于不支持 CMD8 的卡，HCS 位设置为 0 即可。

SD 卡在接收到 ACMD41 后，返回 OCR 寄存器内容，如果是 2.0 的卡，主机可以通过判断 OCR 的 CCS 位来判断是 SDHC 还是 SDSC；如果是 1.x 的卡，则忽略该位。OCR 寄存器的最后一个位用于告诉主机 SD 卡是否上电完成，如果上电完成，该位将会被置 1。

对于 MMC 卡，则不支持 ACMD41，不响应 CMD55，对 MMC 卡，我们只需要在发送 CMD0 后，在发送 CMD1（作用同 ACMD41），检查 MMC 卡的 OCR 寄存器，实现 MMC 卡的初始化。

至此，我们便实现了对 SD 卡的类型区分，图 53.1.5.1 中，最后发送了 CMD2 和 CMD3 命令，用于获得卡 CID 寄存器数据和卡相对地址（RCA）。

CMD2，用于获得 CID 寄存器的数据，CID 寄存器数据各位定义如表 53.3.5.5 所示：

Name	Field	Width	CID-slice
Manufacturer ID	MID	8	[127:120]
OEM/Application ID	OID	16	[119:104]
Product name	PNM	40	[103:64]
Product revision	PRV	8	[63:56]
Product serial number	PSN	32	[55:24]
reserved	--	4	[23:20]
Manufacturing date	MDT	12	[19:8]
CRC7 checksum	CRC	7	[7:1]
not used, always 1	-	1	[0:0]

表 53.3.5.5 卡 CID 寄存器位定义

SD 卡在收到 CMD2 后，将返回 R2 长响应（136 位），其中包含 128 位有效数据（CID 寄存器内容），存放在 SDMMC_RESP1~4 等 4 个寄存器里面。通过读取这四个寄存器，就可以获得 SD 卡的 CID 信息。

CMD3，用于设置卡相对地址（RCA，必须为非 0），对于 SD 卡（非 MMC 卡），在收到 CMD3 后，将返回一个新的 RCA 给主机，方便主机寻址。RCA 的存在允许一个 SDIO 接口挂多个 SD 卡，通过 RCA 来区分主机要操作的是哪个卡。而对于 MMC 卡，则不是由 SD 卡自动返回 RCA，而是主机主动设置 MMC 卡的 RCA，即通过 CMD3 带参数（高 16 位用于 RCA 设置），实现 RCA 设置。同样 MMC 卡也支持一个 SDIO 接口挂多个 MMC 卡，不同于 SD 卡的是所有的 RCA 都是由主机主动设置的，而 SD 卡的 RCA 则是 SD 卡发给主机的。

在获得卡 RCA 之后，我们便可以发送 CMD9（带 RCA 参数），获得 SD 卡的 CSD 寄存器内容，从 CSD 寄存器，我们可以得到 SD 卡的容量和扇区大小等十分重要的信息。CSD 寄存器我们在这里就不详细介绍了，关于 CSD 寄存器的详细介绍，请大家参考《SD 卡 2.0 协议.pdf》。

至此，我们的 SD 卡初始化基本就结束了，最后通过 CMD7 命令，选中我们要操作的 SD 卡，即可开始对 SD 卡的读写操作了，SD 卡的其他命令和参数，我们这里就不再介绍了，请大家参考《SD 卡 2.0 协议.pdf》，里面有非常详细的介绍。

53.3.2 SPI 模式下的 SD 卡初始化

STM32 的 SDIO 驱动模式和 SPI 模式不兼容，二者使用时需要区分开来。《SD 卡 2.0 协议.pdf》中提供了 SD 卡的 SPI 初始化时序，我们可以按它建议的流程进行 SD 卡的初始化，如图 53.3.2.1 所示。

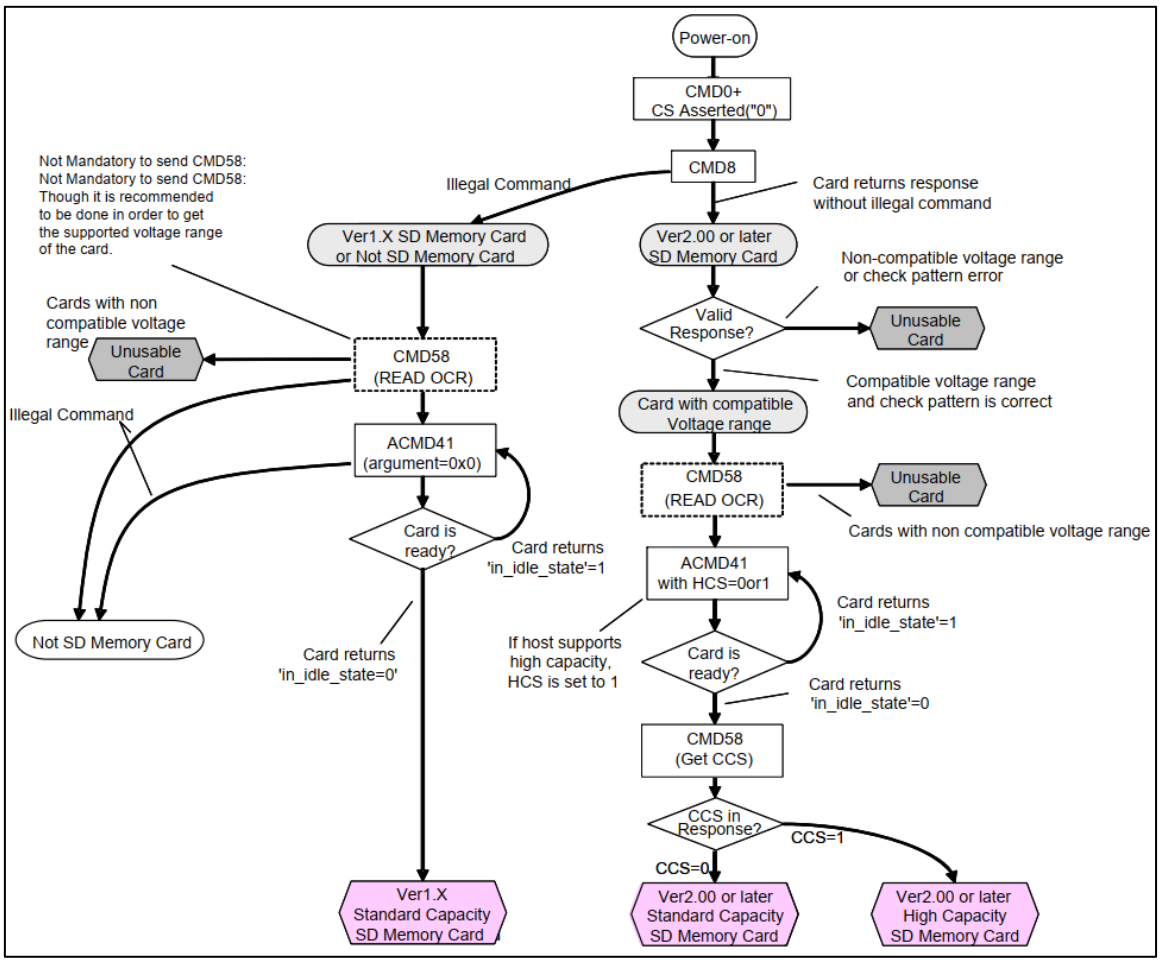


图53.3.2.1 SD卡的SPI初始化流程（SPI Mode Initialization Flow）

要使用 SPI 模式驱动 SD 卡，先得让 SD 卡进入 SPI 模式。方法如下：在 SD 卡收到复位命令（CMD0）时，CS 为有效电平（低电平）则 SPI 模式被启用。不过在发送 CMD0 之前，要发送>74 个时钟，这是因为 SD 卡内部有个供电电压上升时间，大概为 64 个 CLK，剩下的 10 个 CLK 用于 SD 卡同步，之后才能开始 CMD0 的操作，在卡初始化的时候，CLK 时钟最大不能超过 400Khz！

接着我们看看 SD 卡的初始化，由前面 SD 卡的基本介绍，我们知道 SD 卡是先发送数据高位的，SD 卡的典型初始化过程如下：

1、初始化与 SD 卡连接的硬件条件（MCU 的 SPI 配置，IO 口配置）；
2、拉低片选信号，上电延时（>74 个 CLK）；
3、复位卡（CMD0），进入 IDLE 状态；
4、发送 CMD8，检查是否支持 2.0 协议；
5、根据不同协议检查 SD 卡（命令包括：CMD55、ACMD41、CMD58 和 CMD1 等）；
6、取消片选，发多 8 个 CLK，结束初始化

这样我们就完成了对 SD 卡的初始化，注意末尾发送的 8 个 CLK 是提供 SD 卡额外的时钟，完成某些操作。通过 SD 卡初始化，我们可以知道 SD 卡的类型（V1、V2、V2HC 或者 MMC），在完成了初始化之后，就可以开始读写数据了。

SD 卡单扇区读取数据，这里通过 CMD17 来实现，具体过程如下：

1、发送 CMD17；
2、接收卡响应 R1；
3、接收数据起始令牌 0XFE；
4、接收数据；
5、接收 2 个字节的 CRC，如果不使用 CRC，这两个字节在读取后可以丢掉。

6、禁止片选之后，发多 8 个 CLK；

以上就是一个典型的读取 SD 卡数据过程，SD 卡的写与读数据差不多，写数据通过 CMD24 来实现，具体过程如下：

1、发送 CMD24;
2、接收卡响应 R1；
3、发送写数据起始令牌 0XFE；
4、发送数据；
5、发送 2 字节的伪 CRC；
6、禁止片选之后，发多 8 个 CLK；

以上就是一个典型的写 SD 卡过程。关于 SD 卡的介绍，我们就介绍到这里，更详细的介绍请参考光盘资料→7，硬件资料→SD 卡资料→SD 卡 V2.0 协议。

53.4 硬件设计

1. 例程功能

本章实验功能简介：开机的时候先初始化 SD 卡，如果 SD 卡初始化完成，则提示 LCD 初始化成功。按下 KEY0，读取 SD 卡扇区 0 的数据，然后通过串口发送到电脑。如果没初始化通过，则在 LCD 上提示初始化失败。同样用 LED0 来指示程序正在运行。

2. 硬件资源

1）LED 灯
LED：LED0 – PB1
2）独立按键 KEY0 – PH3
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
5）SD 卡，通过 SDMMC1(SDMMC_D0~D4(PC8~PC11),SDMMC_SCK(PC12), SDMMC_CMD(PD2))连接

3. 原理图

前面介绍 SD 卡时我们已经介绍了 SD 卡对外的接口部分，实际上 SD 卡对于我们来说是可以灵活变更的部分，实际使用时，业界常用 SD 卡卡座用于专门放置 SD 卡。

下面我们介绍一下板载的 SD 卡接口和 STM32 的连接关系，如图 53.4.1 所示：

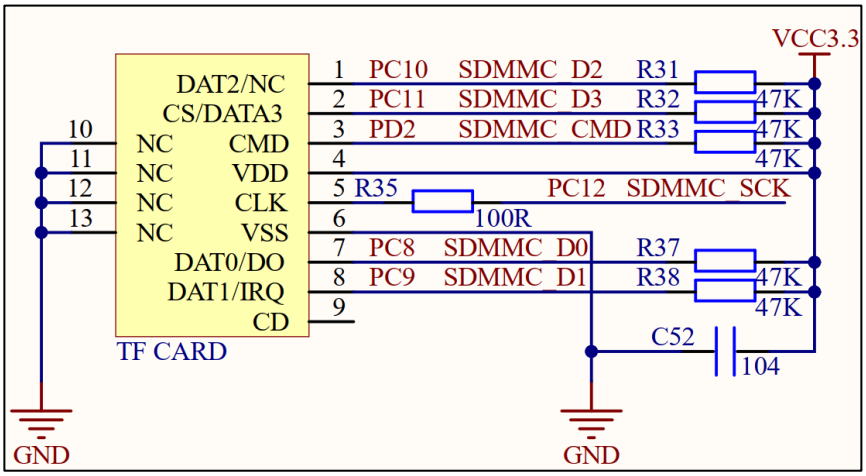


图53.4.1 SD卡接口与STM32连接原理图

microSD 卡座在开发板正面（老版本阿波罗在背面），卡座和 STM32 开发板上是直接连接在一起的，硬件上不需要任何改动。

53.5 程序设计

53.4.1 SD 卡的 HAL 库驱动

STM32 的 HAL 库为 SD 卡操作封装了一些函数，主要存放在 stm32h7xx_hal_sd.c/h 下，下面我们来分析我们主要使用到的几个函数。

1. HAL_SD_Init 函数

要使用一个外设首先要对它进行初始化，所以先看 SD 卡的初始化函数，其声明如下：

```
HAL_StatusTypeDef HAL_SD_Init(SD_HandleTypeDef *hsd)
```

- **函数描述：**

根据 SD 参数，初始化 SDIO 外设以便后续操作 SD 卡。

- **函数形参：**

形参 1 是 SD 卡的句柄，结构体类型是 SD_HandleTypeDef，我们不使用 USE_HAL_SD_REGISTER_CALLBACKS 宏来拓展 SD 卡的自定义函数，精简后其定义如下：

```
/**
  * @brief  SD 操作句柄结构体定义
  */
typedef struct
{
  SD_TypeDef                 *Instance;          /* SD 相关寄存器基地址 */
  SD_InitTypeDef              Init;              /* SDIO 初始化变量 */
  HAL_LockTypeDef             Lock;              /* 互斥锁，用于解决外设访问冲突 */
  uint8_t                    *pTxBuffPtr;        /* SD 发送数据指针 */
  uint32_t                    TxXferSize;        /* SD 发送缓存按字节数的大小 */
  uint8_t                    *pRxBuffPtr;        /* SD 接收数据指针 */
  uint32_t                    RxXferSize;        /* SD 接收缓存按字节数的大小 */
  __IO uint32_t               Context;           /* HAL 库对 SD 卡的操作阶段 */
  __IO HAL_SD_StateTypeDef    State;             /* SD 卡操作状态 */
  __IO uint32_t               ErrorCode;         /* SD 卡错误代码 */
  HAL_SD_CardInfoTypeDef      SdCard;            /* SD 卡信息的 */
  uint32_t                    CSD[4];            /* 保存 SD 卡 CSD 寄存器信息 */
  uint32_t                    CID[4];            /* 保存 SD 卡 CID 寄存器信息 */
}SD_HandleTypeDef;
```

上面的初始化结构体中 HAL_SD_CardInfoTypeDef 用于初始化后提取卡信息，包括卡类型、容量等参数。

```
/**
 * @brief  SD 卡信息结构定义
 */
typedef struct
{
  uint32_t CardType;                    /* 存储卡类型标记：标准卡、高速卡 */
  uint32_t CardVersion;                 /* 存储卡版本 */
  uint32_t Class;                       /* 卡类型 */
  uint32_t RelCardAdd;                  /* 卡相对地址 */
  uint32_t BlockNbr;                    /* 卡存储块数 */
  uint32_t BlockSize;                   /* SD 卡每个存储块大小 */
  uint32_t LogBlockNbr;                 /* 以块表示的卡逻辑容量 */
  uint32_t LogBlockSize;                /* 以字节为单位的逻辑块大小 */
  uint32_t CardSpeed;                   /* 指定卡的速度 */
}HAL_SD_CardInfoTypeDef;
```

- **函数返回值：**

HAL_StatusTypeDef 枚举类型的值，有 4 个，分别是 HAL_OK 表示成功，HAL_ERROR 表示错误，HAL_BUSY 表示忙碌，HAL_TIMEOUT 超时。后续遇到该结构体也是一样的。只有返回 HAL_OK 才是正常的卡初始化状态，遇到其它状态则需要结合硬件分析一下代码。

2. HAL_SD_ConfigWideBusOperation 函数

SD 卡上电后默认使用 1 位数据总线进行数据传输，卡如果允许，可以在初始化完成后重

新设置 SD 卡的数据位宽以加快数据传输过程：

```
HAL_StatusTypeDef HAL_SD_ConfigWideBusOperation(SD_HandleTypeDef *hsd,
                                                uint32_t WideMode);
```

- **函数描述：**

这个函数用于设置数据总线格式的数据宽度，用于加快卡的数据访问速度，当然前提是硬件连接和卡本身能支持这样操作。

- **函数形参：**

形参 1 是 SD 卡的句柄，结构体类型是 SD_HandleTypeDef，此函数需要在 SDMMC 初始化结束后才能使用，我们需要通过使用初始化后的 SDMMC 结构体的句柄访问外设。

形参 2 是总线宽度，根据函数的形参检查规则我们可知它实际上只有三个可选值：

```
#define SDMMC_BUS_WIDE_1B                    ((uint32_t)0x00000000U)
#define SDMMC_BUS_WIDE_4B                    SDMMC_CLKCR_WIDBUS_0
#define SDMMC_BUS_WIDE_8B                    SDMMC_CLKCR_WIDBUS_1
```

- **函数返回值：**

HAL_StatusTypeDef 类型的函数，返回值同样是需要获取到 HAL_OK 表示成功。

3. HAL_SD_ReadBlocks 函数

SD 卡初始化后从 SD 卡的指定扇区读数据：

```
HAL_StatusTypeDef HAL_SD_ReadBlocks (SD_HandleTypeDef *hsd, uint8_t *pData,
                 uint32_t BlockAdd, uint32_t NumberOfBlocks, uint32_t Timeout);
```

这个函数是直接读取，不使用硬件中断。

- **函数描述：**

从 SD 卡的指定扇区读取一定数量的数据。

- **函数形参：**

形参 1 是 SD 卡的句柄，结构体类型是 SD_HandleTypeDef，此函数需要在 SDMMC 初始化结束后才能使用，我们需要通过使用初始化后的 SDMMC 结构体的句柄访问外设。

形参 2 pData 是一个指向 8 位类型的数据指针缓冲，它用于接收我们需要的数据。

形参 3 BlockAdd 指向我们需要访问的数据扇区，对于任意的存储都是类似的，像 SD 卡这样的大存储块也同样是通过位置标识来访问不同的数据。

形参 4 NumberOfBlocks 对应的是我们本次要从指定扇区读取的字节数。

形参 5 Timeout 表示读的超时时间。HAL 库驱动在达到超时时间前还没读到数据会进行重试和等待，达到超时时间后或者本次读取成功才退出本次操作。

- **函数返回值：**

HAL_StatusTypeDef 类型的函数，返回值同样是需要获取到 HAL_OK 表示成功。

类似功能的函数还有，我们的例程没有使用 DMA 和中断方式，故不使用以下两个接口：

```
HAL_StatusTypeDef HAL_SD_ReadBlocks_IT  (SD_HandleTypeDef *hsd, uint8_t *pData,
                                  uint32_t BlockAdd, uint32_t NumberOfBlocks);
HAL_StatusTypeDef HAL_SD_ReadBlocks_DMA (SD_HandleTypeDef *hsd, uint8_t *pData,
                                 uint32_t BlockAdd, uint32_t NumberOfBlocks);
```

它们分别使用了中断方式和 DMA 方式来实现类似的功能，它们的调用非常相似

4. HAL_SD_WriteBlocks 函数

SD 卡初始化后，在 SD 卡的指定扇区写入数据：

```
HAL_StatusTypeDef HAL_SD_WriteBlocks (SD_HandleTypeDef *hsd, uint8_t *pData,
                 uint32_t BlockAdd, uint32_t NumberOfBlocks, uint32_t Timeout);
```

- **函数描述：**

从 SD 卡的指定扇区读取一定数量的数据。

- **函数形参：**

形参 1 是 SD 卡的句柄，结构体类型是 SD_HandleTypeDef，此函数需要在 SDMMC 初始化结束后才能使用，我们需要通过使用初始化后的 SDMMC 结构体的句柄访问外设。

形参 2 pData 是一个指向 8 位类型的数据指针缓冲，它用于接收我们需要的数据。

形参 3 BlockAdd 指向我们需要访问的数据扇区，对于任意的存储都是类似的，像 SD 卡这样的大存储块也同样是通过位置标识来访问不同的数据。

形参 4 NumberOfBlocks 对应的是我们本次要从指定扇区读取的字节数。

形参 5 Timeout 表示写动作的超时时间。HAL 库驱动在达到超时时间前还没读到数据会进

行重试和等待，达到超时时间后或者本次写入成功才退出本次操作。

- **函数返回值：**

HAL_StatusTypeDef 类型的函数，返回值同样是需要获取到 HAL_OK 表示成功。

类似于读函数，写函数同样有中断版本，我们的例程没有使用 DMA 和中断方式，故不使用以下两个接口：

```
HAL_StatusTypeDef HAL_SD_WriteBlocks_IT (SD_HandleTypeDef *hsd, uint8_t *pData,
                                         uint32_t BlockAdd, uint32_t NumberOfBlocks);
HAL_StatusTypeDef HAL_SD_WriteBlocks_DMA(SD_HandleTypeDef *hsd, uint8_t *pData,
                                         uint32_t BlockAdd, uint32_t NumberOfBlocks);
```

它们分别使用了中断方式和 DMA 方式来实现类似的功能，它们的调用非常相似，这里就不重复介绍了，大家查看对应的函数实现即可。

5. HAL_SD_GetCardInfo 函数

SD 卡初始化后，根据设备句柄读 SD 卡的相关状态信息：

```
HAL_StatusTypeDef HAL_SD_GetCardInfo(SD_HandleTypeDef *hsd,
                                     HAL_SD_CardInfoTypeDef *pCardInfo);
```

- **函数描述：**

从 SD 卡的指定扇区读取一定数量的数据。

- **函数形参：**

形参 1 是 SD 卡的句柄，结构体类型是 SD_HandleTypeDef，此函数需要在 SDMMC 初始化结束后才能使用，我们需要通过使用初始化后的 SDMMC 结构体的句柄访问外设。

形参 2 pData 是一个指向 8 位类型的数据指针缓冲，它用于接收我们需要的数据。

形参 3 BlockAdd 指向我们需要访问的数据扇区，对于任意的存储都是类似的，像 SD 卡这样的大存储块也同样是通过位置标识来访问不同的数据。

形参 4 NumberOfBlocks 对应的是我们本次要从指定扇区读取的字节数。

形参 5 Timeout 表示读的超时时间。HAL 库驱动在达到超时时间前还没读到数据会进行重试和等待，达到超时时间后才退出本次操作。

- **函数返回值：**

HAL_StatusTypeDef 类型的函数，返回值同样是需要获取到 HAL_OK 表示成功。类似的函数还有：

```
HAL_StatusTypeDef HAL_SD_SendSDStatus(SD_HandleTypeDef *hsd, uint32_t
                                                              *pSDstatus);
HAL_SD_CardStateTypeDef HAL_SD_GetCardState(SD_HandleTypeDef *hsd);
HAL_StatusTypeDef HAL_SD_GetCardCID(SD_HandleTypeDef *hsd,
                                    HAL_SD_CardCIDTypeDef *pCID);
HAL_StatusTypeDef HAL_SD_GetCardCSD(SD_HandleTypeDef *hsd,
                                    HAL_SD_CardCSDTypeDef *pCSD);
HAL_StatusTypeDef HAL_SD_GetCardStatus(SD_HandleTypeDef *hsd,
                                       HAL_SD_CardStatusTypeDef *pStatus);
HAL_StatusTypeDef HAL_SD_GetCardInfo(SD_HandleTypeDef *hsd,
                                     HAL_SD_CardInfoTypeDef *pCardInfo);
```

它们分别使用了中断方式和 DMA 方式来实现类似的功能，它们的调用非常相似。

SDMMC 驱动 SD 卡配置步骤

1）使能 SDMMC 和相关 GPIO 时钟，并设置好 GPIO 工作模式

我们通过 SDMMC 读写 SD 卡，所以先需要使能 SDMMC 以及相关 GPIO 口的时钟，并设置好 GPIO 的工作模式。

2）初始化 SDMMC

HAL 库通过 SDMMC_Init 完成对 SDMMC 的初始化，不过我们并不直接调用该函数，而是通过： HAL_SD_Init→HAL_SD_InitCard→SDMMC_Init 的调用关系，来完成对 SDMMC 的初始化。我们只需要配置好 SDMMC 相关工作参数，然后调用 HAL_SD_Init 函数即可，详见本例程源码。

3）初始化 SD 卡

HAL 库通过 HAL_SD_InitCard 函数完成对 SD 卡的初始化，如上可知，我们也只需要调用 HAL_SD_Init 函数，即可完成对 SD 卡的初始化。

4）实现 SD 卡读取&写入函数

在初始化 SDMMC 和 SD 卡完成以后，我们就可以访问 SD 卡了，HAL 库提供了两个基本的 SD 卡读写函数：HAL_SD_ReadBlocks 和 HAL_SD_WriteBlocks，用于读取和写入 SD 卡。我们对这两个函数再进行一次封装，以便更好的适配文件系统，再封装后，我们使用：sd_read_disk 来读取 SD 卡，使用：sd_write_disk 来写入 SD 卡，详见本例程源码。

53.4.2 程序流程图

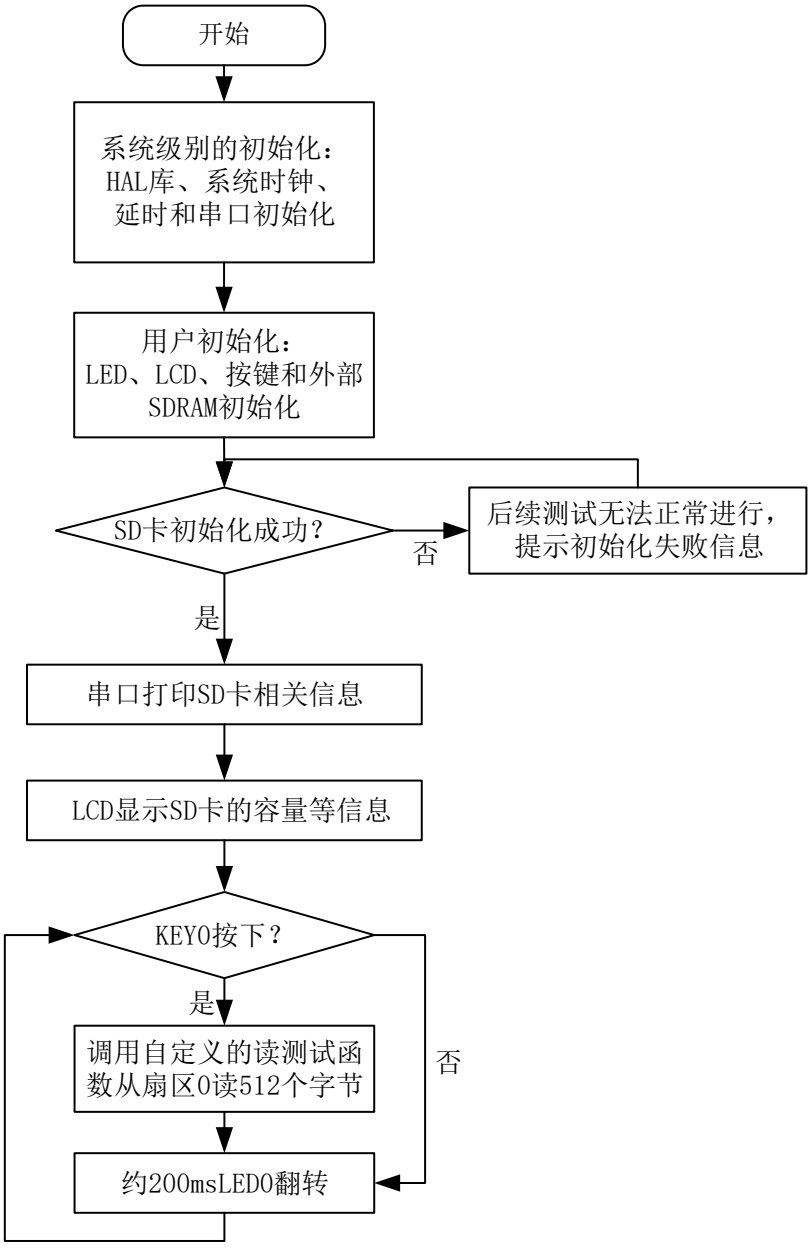


图 53.4.2.1 SD 读写实验程序流程图

53.4.3 程序解析

1. SDMMC 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。SDMMC 驱动源码包括两个文件：sdmmc_sdcard.c 和 sdmmc_sdcard.h。

首先是 sdmmc_sdcard.h 文件，根据我们 STM32 的复用功能和我们的硬件设计，我们把用到的管脚用宏定义，需要更换其它的引脚时也可以通过修改宏实现快速移植，它们列出如下：

```
/* SDMMC1 的信号线: SD1_D0 ~ SD1_D3/SD1_CLK/SD1_CMD 引脚 定义
 * 如果你使用了其他引脚做 SDMMC1 的信号线,修改这里写定义即可适配.
 * 注意, 这里仅支持 SDMMC1, 我们使用的也是 SDMMC1. 不支持 SDMMMC2.
 * 另外, 由于 SDMMC1 的 IO 口复用功能都是 AF12, 这里就不独立再定义每个 IO 口的 AF 功能了
 */
#define SD1_D0_GPIO_PORT                GPIOC
#define SD1_D0_GPIO_PIN                 GPIO_PIN_8
#define SD1_D0_GPIO_CLK_ENABLE()
        do{ __HAL_RCC_GPIOC_CLK_ENABLE(); }while(0)    /* 所在 IO 口时钟使能 */

#define SD1_D1_GPIO_PORT                GPIOC
#define SD1_D1_GPIO_PIN                 GPIO_PIN_9
#define SD1_D1_GPIO_CLK_ENABLE()
        do{ __HAL_RCC_GPIOC_CLK_ENABLE(); }while(0)    /* 所在 IO 口时钟使能 */

#define SD1_D2_GPIO_PORT                GPIOC
#define SD1_D2_GPIO_PIN                 GPIO_PIN_10
#define SD1_D2_GPIO_CLK_ENABLE()
        do{ __HAL_RCC_GPIOC_CLK_ENABLE(); }while(0)    /* 所在 IO 口时钟使能 */

#define SD1_D3_GPIO_PORT                GPIOC
#define SD1_D3_GPIO_PIN                 GPIO_PIN_11
#define SD1_D3_GPIO_CLK_ENABLE()
        do{ __HAL_RCC_GPIOC_CLK_ENABLE(); }while(0)    /* 所在 IO 口时钟使能 */

#define SD1_CLK_GPIO_PORT               GPIOC
#define SD1_CLK_GPIO_PIN                GPIO_PIN_12
#define SD1_CLK_GPIO_CLK_ENABLE()
        do{ __HAL_RCC_GPIOC_CLK_ENABLE(); }while(0)    /* 所在 IO 口时钟使能 */

#define SD1_CMD_GPIO_PORT               GPIOD
#define SD1_CMD_GPIO_PIN                GPIO_PIN_2
#define SD1_CMD_GPIO_CLK_ENABLE()
        do{ __HAL_RCC_GPIOD_CLK_ENABLE(); }while(0)    /* 所在 IO 口时钟使能 */

#define SD_TIMEOUT          ((uint32_t)100000000)      /* 超时时间 */
#define SD_TRANSFER_OK      ((uint8_t)0x00)            /* 传输完成 */
#define SD_TRANSFER_BUSY    ((uint8_t)0x01)            /* 卡正忙 */
```

sdmmc_sdcard.c 我们主要介绍三个函数：sd_init、sd_read_disk 和 sd_write_disk。

1） sd_init 函数

sd_init的设计就比较简单了，我们只需要填充SDMMC结构体的控制句柄，然后使用HAL库的 HAL_SD_Init 初始化函数即可，在此过程中 HAL_SD_Init 会调用 HAL_SD_MspInit 函数回调函数，根据外设的情况，我们可以设置数据总线宽度为 4 位：

```
/**
 * @brief       初始化 SD 卡
 * @param       无
 * @retval      返回值:0 初始化正确；其他值，初始化错误
 */
uint8_t sd_init(void)
{
    uint8_t SD_Error;

    /* 初始化时的时钟不能大于 400KHZ */
    g_sd_handle.Instance = SDMMC1;
    g_sd_handle.Init.ClockEdge = SDMMC_CLOCK_EDGE_RISING;    /* 上升沿 */
    /* 空闲时不关闭时钟电源 */
    g_sd_handle.Init.ClockPowerSave = SDMMC_CLOCK_POWER_SAVE_DISABLE;
    g_sd_handle.Init.BusWide = SDMMC_BUS_WIDE_4B;      /* 4 位数据线 */
    /* 关闭硬件流控 */
```

```
    g_sd_handle.Init.HardwareFlowControl = SDMMC_HARDWARE_FLOW_CONTROL_DISABLE;
    g_sd_handle.Init.ClockDiv=SDMMC_NSpeed_CLK_DIV;/* SD 传输时钟频率最大 25MHZ */

    SD_Error = HAL_SD_Init(&g_sd_handle);
    if (SD_Error != HAL_OK)
    {
        return 1;
    }
    HAL_SD_GetCardInfo(&g_sd_handle, &g_sd_card_info_handle); /* 获取 SD 卡信息 */
    return 0;
}

/**
 * @brief       SDMMC 底层驱动，时钟使能，引脚配置，DMA 配置
 * @param       hsd:SD 卡句柄
 * @note        此函数会被 HAL_SD_Init()调用
 * @retval      无
 */
void HAL_SD_MspInit(SD_HandleTypeDef *hsd)
{
    GPIO_InitTypeDef gpio_init_struct;

    __HAL_RCC_SDMMC1_CLK_ENABLE();  /* 使能 SDMMC1 时钟 */
    SD1_D0_GPIO_CLK_ENABLE();       /* D0 引脚 IO 时钟使能 */
    SD1_D1_GPIO_CLK_ENABLE();       /* D1 引脚 IO 时钟使能 */
    SD1_D2_GPIO_CLK_ENABLE();       /* D2 引脚 IO 时钟使能 */
    SD1_D3_GPIO_CLK_ENABLE();       /* D3 引脚 IO 时钟使能 */
    SD1_CLK_GPIO_CLK_ENABLE();      /* CLK 引脚 IO 时钟使能 */
    SD1_CMD_GPIO_CLK_ENABLE();      /* CMD 引脚 IO 时钟使能 */

    gpio_init_struct.Pin = SD1_D0_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;                /* 推挽复用 */
    gpio_init_struct.Pull = GPIO_PULLUP;                    /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;     /* 高速 */
    gpio_init_struct.Alternate = GPIO_AF12_SDIO1;           /* 复用为 SDIO */
    HAL_GPIO_Init(SD1_D0_GPIO_PORT, &gpio_init_struct);     /* 初始化 D0 引脚 */

    gpio_init_struct.Pin = SD1_D1_GPIO_PIN;
    HAL_GPIO_Init(SD1_D1_GPIO_PORT, &gpio_init_struct);     /* 初始化 D1 引脚 */

    gpio_init_struct.Pin = SD1_D2_GPIO_PIN;
    HAL_GPIO_Init(SD1_D2_GPIO_PORT, &gpio_init_struct);     /* 初始化 D2 引脚 */

    gpio_init_struct.Pin = SD1_D3_GPIO_PIN;
    HAL_GPIO_Init(SD1_D3_GPIO_PORT, &gpio_init_struct);     /* 初始化 D3 引脚 */

    gpio_init_struct.Pin = SD1_CLK_GPIO_PIN;
    HAL_GPIO_Init(SD1_CLK_GPIO_PORT, &gpio_init_struct);    /* 初始化 CLK 引脚 */

    gpio_init_struct.Pin = SD1_CMD_GPIO_PIN;
    HAL_GPIO_Init(SD1_CMD_GPIO_PORT, &gpio_init_struct);    /* 初始化 CMD 引脚 */
}
```

2） sd_read_disk 函数

这个函数比较简单，实际上我们使用它来对 HAL 库的读函数 HAL_SD_ReadBlocks 进行了二次封装，并在最后加入了状态判断以便后续操作（实际上这部分代码也可以省略），直接根据读函数返回值自己作其它处理。为了保护 SD 卡的数据操作，我们在进行操作时暂时关闭了中断以防止数据读过程发生意外。

```
/**
 * @brief       读 SD 卡数据
 * @param       pbuf:读数据缓存区
 * @param       saddr:扇区地址
```

```
 * @param       cnt:扇区个数
 * @retval      返回值:0,正常;其他,错误;
 */
uint8_t sd_read_disk(uint8_t *pbuf, uint32_t saddr, uint32_t cnt)
{
    uint8_t sta = HAL_OK;
    uint32_t timeout = SD_TIMEOUT;
    long long lsector = saddr;
    sys_intx_disable();  /* 关闭总中断(POLLING 模式,严禁中断打断 SDIO 读写操作!!!) */
    sta = HAL_SD_ReadBlocks(&g_sd_handle, (uint8_t*)pbuf,lsector, cnt,
                            SD_TIMEOUT); /* 多个 sector 的读操作 */
    /* 等待 SD 卡读完 */
    while (get_sd_card_state() != SD_TRANSFER_OK)
    {
        if (timeout-- == 0)
        {
            sta = SD_TRANSFER_BUSY;
        }
    }
    sys_intx_enable();  /* 开启总中断 */
    return sta;
}
```

3） sd_write_disk 函数

这个函数比较简单，实际上我们使用它来对 HAL 库的写函数 HAL_SD_WriteBlocks 进行了二次封装，并在最后加入了状态判断以便后续操作（实际上这部分代码也可以省略），直接根据写函数返回值自己作其它处理。为了保护 SD 卡的数据操作，我们在进行操作时暂时关闭了中断以防止数据写过程发生意外。

```
/**
 * @brief       写数据到 SD 卡
 * @param       pbuf:写数据缓存区
 * @param       saddr:扇区地址
 * @param       cnt:扇区个数
 * @retval      返回值:0,正常;其他,错误;
 */
uint8_t sd_write_disk(uint8_t *pbuf, uint32_t saddr, uint32_t cnt)
{
    uint8_t sta = HAL_OK;
    uint32_t timeout = SD_TIMEOUT;
    long long lsector = saddr;
    sys_intx_disable();  /* 关闭总中断(POLLING 模式,严禁中断打断 SDIO 读写操作!!!) */
    sta = HAL_SD_WriteBlocks(&g_sd_handle, (uint8_t*)pbuf, lsector, cnt,
                             SD_TIMEOUT); /* 多个 sector 的写操作 */

    /* 等待 SD 卡写完 */
    while (get_sd_card_state() != SD_TRANSFER_OK)
    {
        if (timeout-- == 0)
        {
            sta = SD_TRANSFER_BUSY;
        }
    }

    sys_intx_enable(); /* 开启总中断 */
    return sta;
}
```

2. main.c 代码

在 main.c 就比较简单了，按照我们的流程图的思路，为了方便测试，我们编写了 sd_test_read()\sd_test_write()\show_sdcard_info ()三个函数分别用于读写测试和卡信息打印，也都是基于对前面 HAL 库的代码进行简单地调用，代码也比较容易看懂，这里就不单独介绍这

几个函数了，大家查看光盘中的源代码即可。

最后，我们在 main.c 编写的程序如下：

```
/**
 * @brief       通过串口打印 SD 卡相关信息
 * @param       无
 * @retval      无
 */
void show_sdcard_info(void)
{
    uint64_t card_capacity;        /* SD 卡容量 */
    HAL_SD_CardCIDTypeDef sd_card_cid;

    HAL_SD_GetCardCID(&g_sd_handle, &sd_card_cid);      /* 获取 CID */
    get_sd_card_info(&g_sd_card_info_handle);           /* 获取 SD 卡信息 */

    switch (g_sd_card_info_handle.CardType)
    {
        case CARD_SDSC:
        {
            if (g_sd_card_info_handle.CardVersion == CARD_V1_X)
            {
                printf("Card Type:SDSC V1\r\n");
            }
            else if (g_sd_card_info_handle.CardVersion == CARD_V2_X)
            {
                printf("Card Type:SDSC V2\r\n");
            }
        }
            break;

        case CARD_SDHC_SDXC:
            printf("Card Type:SDHC\r\n");
            break;
    }

    card_capacity = (uint64_t)(g_sd_card_info_handle.LogBlockNbr) *
              (uint64_t)(g_sd_card_info_handle.LogBlockSize); /* 计算 SD 卡容量 */
    /* 制造商 ID */
    printf("Card ManufacturerID:%d\r\n", sd_card_cid.ManufacturerID);
    /* 卡相对地址 */
    printf("Card RCA:%d\r\n", g_sd_card_info_handle.RelCardAdd);
    printf("LogBlockNbr:%d \r\n",
           (uint32_t)(g_sd_card_info_handle.LogBlockNbr));    /* 显示逻辑块数量 */
    printf("LogBlockSize:%d \r\n",
           (uint32_t)(g_sd_card_info_handle.LogBlockSize));   /* 显示逻辑块大小 */
    /* 显示容量 */
    printf("Card Capacity:%d MB\r\n", (uint32_t)(card_capacity >> 20));
    /* 显示块大小 */
    printf("Card BlockSize:%d\r\n\r\n", g_sd_card_info_handle.BlockSize);
}

/**
 * @brief       测试 SD 卡的读取
 * @note        从 secaddr 地址开始,读取 seccnt 个扇区的数据
 * @param       secaddr : 扇区地址
 * @param       seccnt  : 扇区数
 * @retval      无
 */
void sd_test_read(uint32_t secaddr, uint32_t seccnt)
{
    uint32_t i;
    uint8_t *buf;
    uint8_t sta = 0;
```

```
    buf = mymalloc(SRAMIN, seccnt * 512);       /* 申请内存,从 SDRAM 申请内存 */
    sta = sd_read_disk(buf, secaddr, seccnt);   /* 读取 secaddr 扇区开始的内容 */

    if (sta == 0)
    {
        printf("SECTOR %d DATA:\r\n", secaddr);
        for (i = 0; i < seccnt * 512; i++)
        {
            printf("%x ", buf[i]);  /* 打印 secaddr 开始的扇区数据 */
        }

        printf("\r\nDATA ENDED\r\n");
    }
    else printf("err:%d\r\n", sta);
    myfree(SRAMIN, buf);    /* 释放内存 */
}

/**
 * @brief       测试 SD 卡的写入
 * @note        从 secaddr 地址开始,写入 seccnt 个扇区的数据
 *              慎用!! 最好写全是 0XFF 的扇区,否则可能损坏 SD 卡.
 *
 * @param       secaddr : 扇区地址
 * @param       seccnt  : 扇区数
 * @retval      无
 */
void sd_test_write(uint32_t secaddr, uint32_t seccnt)
{
    uint32_t i;
    uint8_t *buf;
    uint8_t sta = 0;
    buf = mymalloc(SRAMIN, seccnt * 512);   /* 从 SDRAM 申请内存 */

    for (i = 0; i < seccnt * 512; i++)      /* 初始化写入的数据,是 3 的倍数. */
    {
        buf[i] = i * 3;
    }
    /* 从 secaddr 扇区开始写入 seccnt 个扇区内容 */
    sta = sd_write_disk(buf, secaddr, seccnt);

    if (sta == 0)
    {
        printf("Write over!\r\n");
    }
    else printf("err:%d\r\n", sta);
    myfree(SRAMIN, buf);    /* 释放内存 */
}

int main(void)
{
    uint8_t key;
    uint32_t sd_size;
    uint8_t t = 0;
    uint8_t *buf;
    uint64_t card_capacity;                 /* SD 卡容量 */

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    mpu_memory_protection();                /* 保护相关存储区域 */
```

```
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    my_mem_init(SRAMIN);                    /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                    /* 初始化外部内存池 */
    my_mem_init(SRAM12);                    /* 初始化 SRAM12 内存池(SRAM1+SRAM2) */
    my_mem_init(SRAM4);                     /* 初始化 SRAM4 内存池(SRAM4) */
    my_mem_init(SRAMDTCM);                  /* 初始化 DTCM 内存池(DTCM) */
    my_mem_init(SRAMITCM);                  /* 初始化 ITCM 内存池(ITCM) */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "SD  TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Read Sector 0", RED);

    while (sd_init())     /* 检测不到 SD 卡 */
    {
        lcd_show_string(30, 150, 200, 16, 16, "SD Card Error!", RED);
        delay_ms(500);
        lcd_show_string(30, 150, 200, 16, 16, "Please Check! ", RED);
        delay_ms(500);
        LED0_TOGGLE();  /* 红灯闪烁 */
    }

    /* 打印 SD 卡相关信息 */
    show_sdcard_info();

    /* 检测 SD 卡成功 */
    lcd_show_string(30, 150, 200, 16, 16, "SD Card OK    ", BLUE);
    lcd_show_string(30, 170, 200, 16, 16, "SD Card Size:     MB", BLUE);
    card_capacity = (uint64_t)(g_sd_card_info_handle.LogBlockNbr) *
              (uint64_t)(g_sd_card_info_handle.LogBlockSize); /* 计算 SD 卡容量 */
    /* 显示 SD 卡容量 */
    lcd_show_num(30 + 13 * 8, 170, card_capacity >> 20, 5, 16, BLUE);

    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)       /* KEY0 按下了 */
        {
            buf = mymalloc(0, 512); /* 申请内存 */
            key = sd_read_disk(buf, 0, 1);
            if (key == 0)             /* 读取 0 扇区的内容 */
            {
                lcd_show_string(30, 190, 200, 16, 16,
                                "USART1 Sending Data...", BLUE);
                printf("SECTOR 0 DATA:\r\n");
                for (sd_size = 0; sd_size < 512; sd_size++)
                {
                    printf("%x ", buf[sd_size]);    /* 打印 0 扇区数据 */
                }

                printf("\r\nDATA ENDED\r\n");
                lcd_show_string(30, 190, 200, 16, 16,
                                "USART1 Send Data Over!", BLUE);
            }
            else printf("err:%d\r\n", key);
            myfree(0, buf); /* 释放内存 */
        }
        t++;
        delay_ms(10);
        if (t == 20)
```

```
        {
            LED0_TOGGLE();   /* 红灯闪烁 */
            t = 0;
        }
    }
}
```

这里总共 4 个函数：

1、show_sdcard_info 函数

该函数用于从串口输出 SD 卡相关信息，包括：卡类型、制造商 ID、卡相对地址、容量和块大小等信息。

2、sd_test_read

该函数用于测试 SD 卡的读取，通过 USMART 调用，可以指定 SD 卡的任何地址，读取指定个数的扇区数据，将读到的数据，通过串口打印出来，从而验证 SD 卡数据的读取。

3、sd_test_write 函数

该函数用于测试 SD 卡的写入，通过 USMART 调用，可以指定 SD 卡的任何地址，写入指定个数的扇区数据，写入数据自动生成（都是 3 的倍数），写入完成后，在串口打印写入结果。我们可以通过 sd_test_read 函数，来检验写入数据是否正确。**注意：千万别乱写，否则可能把卡写成砖头/数据丢失!! 写之前，先读取该地址的数据，最好全部是 0XFF 才写（全部 0X00 也行），其他情况最好别写！**

4、main 函数函数

该函数，先初化相关外设和 SD 卡，初始化成功，则调用 show_sdcard_info 函数，输出 SD 卡相关信息，并在 LCD 上面显示 SD 卡容量。然后进入死循环，如果有按键 KEY0 按下，则通过 SD_ReadDisk 读取 SD 卡的扇区 0（物理磁盘，扇区 0），并将数据通过串口打印出来。这里，我们对上一章学过的内存管理小试牛刀，稍微用了下，以后我们会尽量使用内存管理来设计。

最后，我们将 sd_test_read 和 sd_test_write 函数加入 USMART 控制，这样，我们就可以通过串口调试助手，测试 SD 卡的读写了，方便测试。

53.6 下载验证

将程序下载到开发板后，我们测试使用的是 16Gb 标有“SDHC”标志的卡，安装方法如图 53.6.1 所示：

图 53.6.1 测试用的 microSD 卡与开发板的连接方式

SD 卡成功初始化后，LCD 显示本程序的一些必要信息，如图 53.6.2：

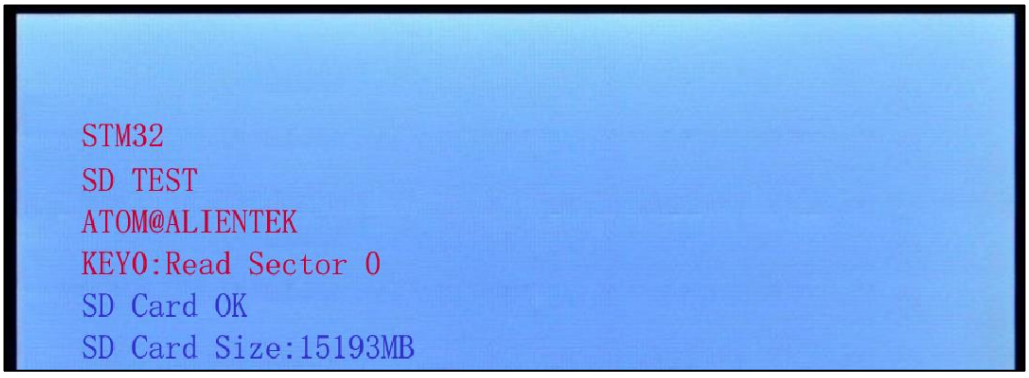


图 53.6.2 程序运行效果图

在进入测试的主循环前，我们如果已经通过 USB 连接开发板的串口 1 和电脑，可以看到

串口端打印出 SD 卡的相关信息（也可以在接好 SD 卡后按 Reset 复位开发板），我们测试使用的是 16Gb 标有“SDHC”标志的卡，SD 卡成功初始化后的信息，如图 53.6.3：

图 53.6.3 测试用的 microSD 卡

可见我们用程序读到的 SD 卡信息与我们使用的 SD 卡一致。伴随 LED0 的不停闪烁，提示程序在运行。此时，我们按下 KEY0，调用我们编写的 SD 卡测试函数，这里我们只用到了读函数，写函数的测试大家可以添加代码进行演示。按下后 LCD 显示按下，信息如图 53.6.4，数量较多的情况我们用串口打印，得到的 SD 卡扇区 0 存储的 512 个字节的信息如图 53.6.5 所示：

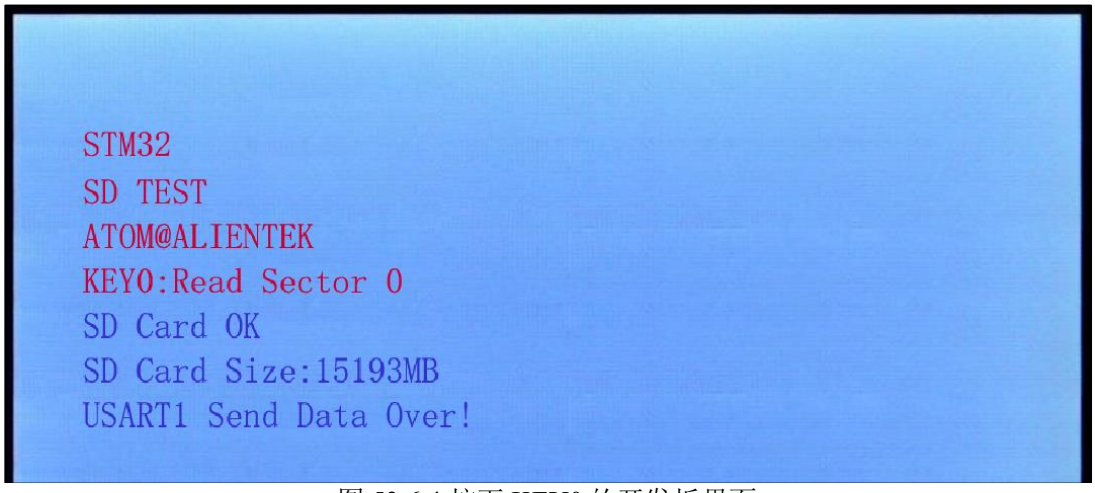


图 53.6.4 按下 KEY0 的开发板界面

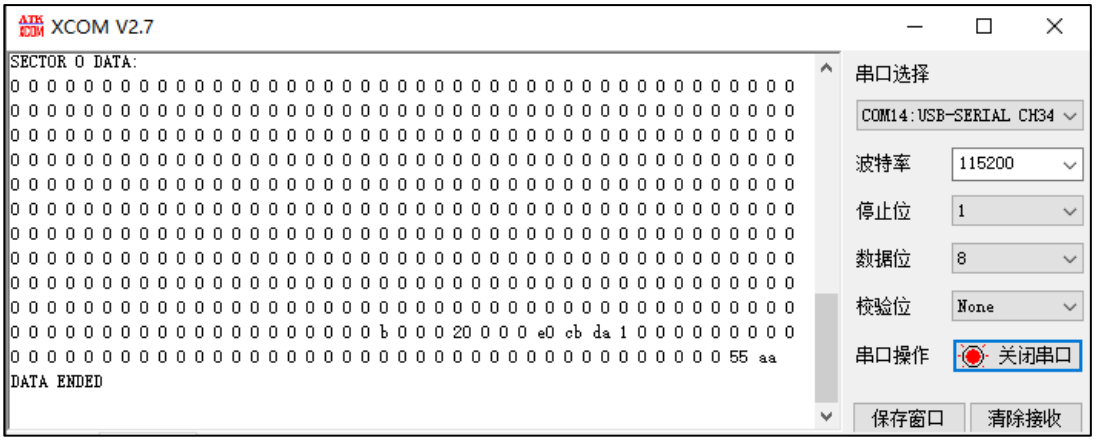


图 53.6.5 串口调试助手显示按下 KEY0 后读取到的信息

对 SD 卡的使用的简单介绍就到这里了，另外，利用 USMART 测试的部分，我们这里就不做介绍了，大家可自行验证下。

第五十四章 NAND FLASH 实验

阿波罗 STM32H743 核心板上面，板载了一颗 512MB 的 NAND FLASH 芯片，支持 MT29F4G08/FSNS8B004G-TWT/IS34ML04G081-TLI 等多种型号，我们可以用它来存储数据，相对于 SPI FLASH（W25Q64）和 SD 卡等存储设备，NAND FLASH 采用 8 位并口访问，具有访问速度快的优势。

本章以 FSNS8B004G 为例介绍驱动流程，其他兼容型号的驱动流程大同小异。接下来我们将完成对 FSNS8B004G 的读写控制。本章主要包含以下内容：

54.1 NAND FLASH 简介
54.2 硬件设计
54.3 程序设计
54.4 下载验证

54.1 NAND FLASH 简介

本章我们将通过 STM32H743 的 FMC 接口，来驱动 FSNS8B004G 这颗 NAND FLASH 芯片（SLC，STM32 仅支持 SLC 类型的 NAND），本节我们将介绍 NAND FLASH 相关知识点，包括：1，NAND FLASH 简介；2，FTL 简介；3，FMC NAND FLASH 接口简介；

54.1.1 NAND FLASH 简介

NAND FLASH 的概念是由东芝公司在 1989 年率先提出，它内部采用非线性宏单元模式，为固态大容量内存的实现提供了廉价有效的解决方案。NAND FLASH 存储器具有容量较大，改写速度快等优点，适用于大量数据的存储，在业界得到了广泛应用，如：SD 卡、TF 卡、U 盘等，一般都是采用 NAND FLASH 作为存储的。关于 NAND FLASH 的基础知识，请大家自行百度学习。接下来，我们介绍 NAND FLASH 的一些重要知识。

（1）NAND FLASH 信号线

信号线	说明
CLE	命令锁存使能，高电平有效，表示写入的是命令
ALE	地址锁存使能，高电平有效，表示写入的是地址
CE#	芯片使能，低电平有效，用于选中 NAND 芯片
RE#	读使能，低电平有效，用于读取数据
WE#	写使能，低电平有效，用于写入数据
WP#	写保护，低电平有效
R/B	就绪/忙，注意用于判断编程/擦除操作是否完成
I/00-7	地址/数据 输入/输出口

表 54.1.1.1 NAND FLASH 信号线

因为 NAND FLASH 地址/数据是共用数据线的，所以必须有 CLE/ALE 信号，告诉 NAND FLASH，发送的数据是命令还是地址。

（2）存储单元

NAND FLASH 存储单元介绍，我们以阿波罗 STM32H743 开发板所使用的 FSNS8B004G（x8，8 位数据）为例进行介绍，FSNS8B004G 的存储单元组织结构如图 54.1.1.1 所示：

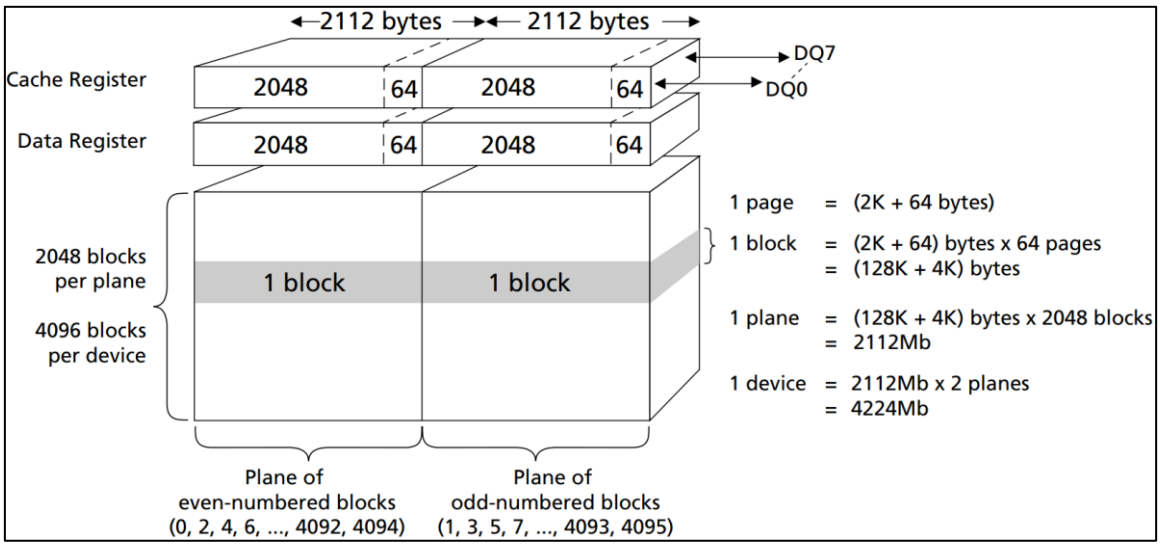


图 54.1.1.1 FSNS8B004G 存储单元组织结构图

由图可知：FSNS8B004G 由 2 个 plane 组成，每个 plane 有 2048 个 block，每个 block 由 64 个 page 组成，每个 page 有 2K+64 字节（2112 字节）的存储容量。所以，FSNS8B004G 的总容量为：2*2048*64*（2K+64）=553648128 字节（512MB）。其中，plane、block、page 等的个数根据 NAND FLASH 型号的不同，会有所区别，大家注意查看对应 NAND FLASH 芯片的数据手册。

NAND FLASH 的最小擦除单位是 block，对 FSNS8B004G 来说，是（128+4）K 字节，NAND FLASH 的写操作具有只可以写 0，不能写 1 的特性，所以，在写数据的时候，必须先擦除 block（擦除后，block 数据全部为 1），才可以写入。

NAND FLASH 的 page 由 2 部分组成：数据存储区（data area）和备用区域（spare area），对 FSNS8B004G 来说，数据存储区大小为 2K 字节，备用区域大小为 64 字节。我们存储的有效数据，一般都是存储在数据存储区（data area）。备用区域（spare area），一般用来存放 ECC（Error Checking and Correcting）校验值，在本章中，我们将利用这个区域，来实现 NAND FLASH 坏块管理和磨损均衡。

NAND FLASH 的的地址分为三类：块地址（Block Address）、页地址（Page Address）和列地址（Column Address）。以 FSNS8B004G 为例，这三个地址，通过 5 个周期发送，如表 54.1.1.2 所示：

周期	I/07	I/06	I/05	I/04	I/03	I/02	I/01	I/00
1	CA7	CA6	CA5	CA4	CA3	CA2	CA1	CA0
2	LOW	LOW	LOW	LOW	CA11	CA10	CA9	CA8
3	BA7	BA6	PA5	PA4	PA3	PA2	PA1	PA0
4	BA15	BA14	BA13	BA12	BA11	BA10	BA9	BA8
5	LOW	LOW	LOW	LOW	LOW	LOW	BA17	BA16

表 54.1.1.2 FSNS8B004G 寻址说明

表中，CA0~CA11 为列地址（Column Address）,用于在一个 Page 内部寻址，FSNS8B004G 的一个 Page 大小为 2112 字节，需要 12 个地址线寻址；PA0~PA5 为页地址（Page Address），用于在一个 Block 内部寻址，FSNS8B004G 一个 Block 大小为 64 个 Page，需要 6 个地址线寻址；BA6~BA17 为块地址（Block Address），用于块寻址，FSNS8B004G 总共有 4096 个 Block，需要 12 根地址寻址。

整个寻址过程，分 5 次发送（5 个周期），首先发送列地址，在发送页地址和块地址。这里提醒一下：块地址和页地址，其实是可以写在一起的，由一个参数传递即可，所以表中的 BA 并不是由 BA0 开始的，大家可以理解为这个地址（PA+BA）为整个 NAND FLASH 的 Page 地址。在完成寻址以后，数据线 I/O0~I/O7 来传输数据了。

（3）控制命令

NAND FLASH 的驱动需要用到一系列命令，这里我们列出常用的一些命令，给大家做一

个简单的介绍，方便大家了解 NAND FLASH 的操作，如表 54.1.1.3 所示：

命令		名称	说明
1#	2#		
0X90		READID	读取 NAND 的 ID 和相关特性，可以此判断 NAND 的容量等信息
0XEF		SET FEATURE	设置 NAND 的相关参数，比如时序模式
0XFF		RESET	复位 NAND
0X70		READ STATUS	读取 NAND 的状态，比如可以判断编程/擦除操作是否完成
0X00	0X30	READ PAGE	该指令由 2 部分组成（分 2 次发），用于读取一个 Page 里面的数据（不能跨页读）
0X80	0X10	WRITE PAGE	该指令由 2 部分组成（分 2 次发），用于写入一个 Page 的数据（不能跨页写）
0X60	0XD0	ERASE BLOCK	该指令由 2 部分组成（分 2 次发），用于擦除一个 Block
0X00	0X35	READ FOR INTERNAL DATA MOVE	这两个指令（分四次发），组成 NAND 的内部数据移动操作，该操作跨页实现拷贝一个 Page 到另外一个 Page（仅限同一 plane 内），且支持拷贝时写入数据，该操作跨页极大的方便数据写入
0X85	0X10	PROGRAM FOR INTERNAL DATA MOVE	

表 54.1.1.3 NNAD FLASH 操作常用指令

表 54.1.1.3 中，我们需要注意两点：1，有的指令一个周期完成传送，有的指令需要分两次传送（2 个周期）；2，指令名称，不同厂家的数据手册里面，标注可能不一样，但是其指令值（HEX 值）一般都是一样的。

上表中，前四条命令相对比较简单，这里我们主要介绍后面五条命令。

1.READ PAGE

该指令用于读取 NAND 的一个 Page（包括 spare 区数据，但不能跨页读），该指令时序如图 54.1.1.2 所示：

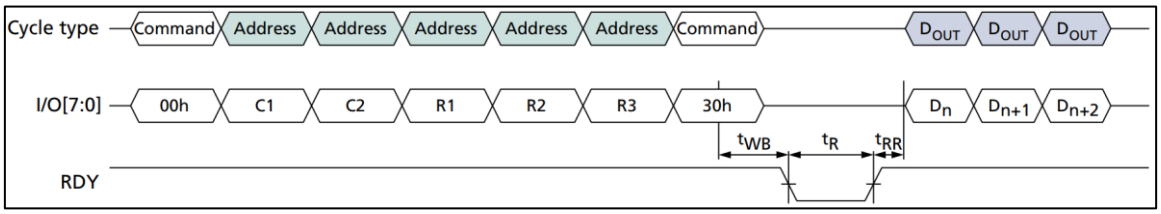


图 54.1.1.2 READ PAGE 指令时序图

由图可知，READ PAGE 的命令分两次发，首先发送 00H 命令，然后发送 5 次地址（Block&Page&Column 地址），指定读取的地址，随后发送 30H 命令，在等待 RDY 后，即可读取 PAGE 里面的数据。注意：不能跨页读，所以最多一次读取一个 PAGE 的数据（包括 spare 区）。

2.WRITE PAGE

该指令用于写一个 Page 的数据（包括 spare 区数据，但不能跨页写），该指令时序如图 54.1.1.3 所示：

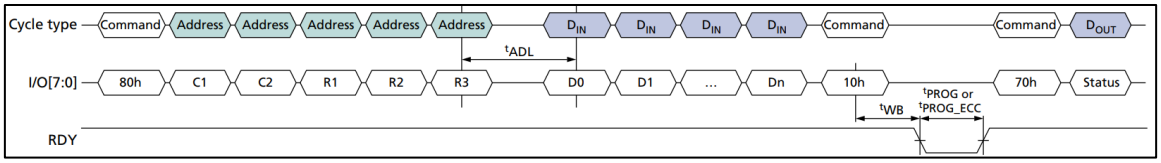


图 54.1.1.3 WRITE PAGE 指令时序图

由图可知，WRITE PAGE 的命令分两次发送，首先发送 80H 命令，然后发送 5 次地址

（Block&Page&Column 地址），指定写入的地址，在地址写入完成后，等待 tADL 时间后，开始发送需要写入的数据，在数据发送完毕后，发送 10H 命令，最后发送 READ STATUS 命令，查询 NAND FLASH 状态，等待状态为 READY 后，完成一次 PAGE 写入操作。

3.ERASE BLOCK

该指令用于擦除 NAND 的一个 BLOCK（NAND 的最小擦除单位），该指令时序如图 54.1.1.4 所示：

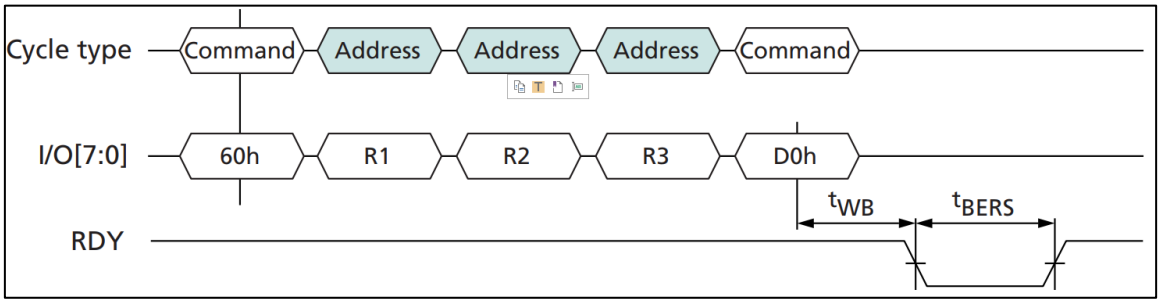


图 54.1.1.4 ERASE BLOCK 指令时序图

由图可知，ERASE BLOCK 的命令分两次发送，首先发送 60H 命令，然后发送 3 次地址（BLOCK 地址），指定要擦除的 BLOCK 地址，随后发送 D0H 命令，在等待 RDY 成功后，完成一个 BLOCK 的擦除。

4.READ FOR INTERNAL DATA MOVE

该指令用于在 NAND 内部进行数据移动时（页对页），指定需要读取的 PAGE 地址，如有必要，可以读取出 PAGE 里面的数据。该指令时序如图 54.1.1.5 所示：

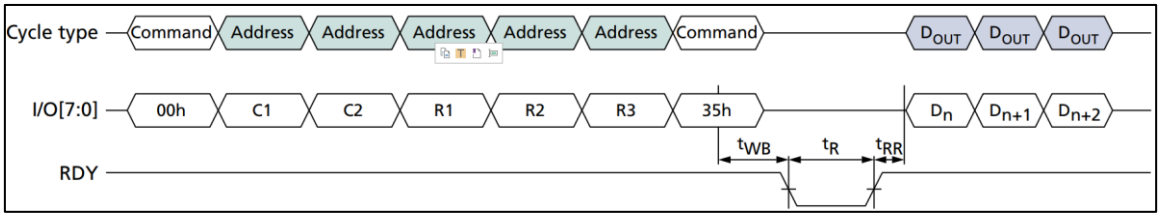


图 54.1.1.5 READ FOR INTERNAL DATA MOVE 指令时序图

由图可知，READ FOR INTERNAL DATA MOVE 的命令分两次发送，首先发送 00H 命令，然后发送 5 次地址（Block&Page&Column 地址），指定读取的地址，随后发送 35H 命令，在等待 RDY 后，可以读取对应 PAGE 里面的数据。在内部数据移动过程中，我们仅用该指令指定需要拷贝的 PAGE 地址（源地址），并不需要读取其数据，所以最后的 Dout 过程，一般都可以省略。

5.PROGRAM FOR INTERNAL DATA MOVE

该指令用于在 NAND 内部进行数据移动时（页对页），指定需要写入的 PAGE 地址（目标地址），如有必要，在拷贝过程中，可以写入新的数据，该指令时序如图 54.1.1.6 所示：

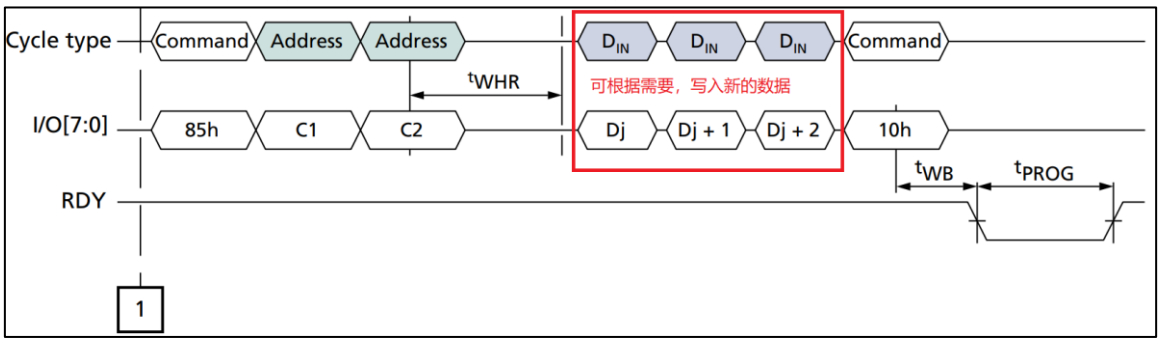


图 54.1.1.6 PROGRAM FOR INTERNAL DATA MOVE 指令时序图

如图 54.1.1.6 所示，该指令，首先发送 85H 命令，然后发送 5 次地址（Block&Page&Column 地址），指定写入的页地址（目标地址），源地址则由 READ FOR INTERNAL DATA MOVE 指令指定。接下来分两种情况：1，要写入新的数据（覆盖源 PAGE 的内容）；2，无需写入新的数据；

对于第 1 种情况，在等待 tWHR（或 tADL）之后，开始写入新的数据，数据在页内的起始地址，由 C1&C2 指定，写入完成后，发送 10H 命令，开始进行页拷贝，在等待 RDY 后，完成一次页对页的数据拷贝（带新数据写入）。

对于第 2 种情况，在发送完 5 次地址后，无需发送新数据，直接发送 10H 命令，开始进行页拷贝，在等待 RDY 后，完成一次页对页的数据拷贝（不带数据写入）。

注意：页对页拷贝，仅支持同一个 plane 里面互相拷贝（源地址和目标地址，必须在同一个 plane 里面），如果不是同一个 plane 里面的页，则不可以执行页对页拷贝。

NAND FLASH 其他命令的介绍，请大家参考 FSNS8B004G 数据手册。

（4）ECC 校验

ECC，英文全称为：Error checking and Correction，是一种对传输数据的错误检测和修正的算法。NAND FLASH 存储单元是串行组织，当读取一个单元的时候，读出放大器所检测到信号强度会被这种串行存储结构削弱，这就降低了所读信号的准确性，导致读数出错（一般只有 1 个 bit 出错）。ECC 就可以检测这种错误，并修正错误的数据位，因此，ECC 在 NAND FLASH 驱动里面，被广泛使用。

ECC 有三种常用的算法：汉明码（Hamming Code）、RS 码（Reed Solomon Code）和 BCH 码。STM32 的 FMC 模块就支持硬件 ECC 计算，使用的就是汉明码，接下来，我们就给大家简单介绍一下汉明码的编码和使用。

1.汉明码编码

汉明码的编码计算比较简单，通过计算块上数据包得到 2 个 ECC 值（ECCo 和 ECCe）。为计算 ECC 值，数据包中的比特数据要先进行分割，如 1/2 组、1/4 组、1/8 组等，直到其精度达到单个比特为止。我们以 8bit 即 1 字节的数据包为例进行说明，如表 54.1.1.4 所示：

数据位	7	6	5	4	3	2	1	0
1/8 偶校验		1		1		0		1
1/4 偶校验			0	1			0	1
1/2 偶校验					0	0	0	1
数据包	0	1	0	1	0	0	0	1
1/2 奇校验	0	1	0	1				
1/4 奇校验	0	1			0	0		
1/8 奇校验	0		0		0		0	

表 54.1.1.4 8bit 数据包校验的数据分割

8 位数据可以按：1/2、1/4 和 1/8 进行分割（1/8 分割时，达到单个比特精度）。简单说一下上表的看法，以 1/2 分割偶校验为例，1/2 分割时，每 4 个 Bit 组成一个新 bit，新的 bit0 等于原来的 bit0~3，新的 bit1 等于原来的 bit4~7，而我们只要偶数位的数据，也就是新 bit0 的数据，实际上就是原来的 bit0~3 的数据，这样就获取了 1/2 偶校验的数据。其他分割以此类推。

表 54.1.1.3 中，数据包上方的三行数据经计算后，得到偶校验值（ECCe），数据包下方的三行数据经计算后，得到奇校验值（ECCo）。1/2 校验值经“异或”操作构成 ECC 校验的最高有效位，1/4 校验值构成 ECC 校验的次高有效位，最低有效位由具体到比特的校验值填补。ECC 校验值（ECCo 和 ECCe）的计算过程，如图 54.1.1.7 所示：

	1/2	1/4	1/8
ECCe=	0^0^0^1	0^1^0^1	1^1^0^1
ECCo=	0^1^0^1	0^1^0^0	0^0^0^0

图 54.1.1.7 计算奇偶 ECC 值

即偶校验值 ECCe 为“101”，奇校验值为“010”。图中所示为只有 1 字节数据的数据包，更大的数据包需要更多的 ECC 值。事实上，每 n bit 的 ECC 数值可满足 2^n bit 数据包的校验要求。不过，汉明码算法要求一对 ECC 数据（奇+偶），所以总共要求 2n bit 的 ECC 校验数据来处理 2^n bit 的数据包。

得到 ECC 值后，我们需要将原数据包和 ECC 数值都写入 NAND 里面。当原数据包从 NAND 中读取时，ECC 值将重新计算，如果新计算的 ECC 不同于先前编入 NAND 器件的 ECC，

那么表明数据在读写过程中发生了错误。例如，原始数据 01010001 中有 1 个单一的比特出现错误，出错后的数据是 01010101。此时，重新计算 ECC 的过程如图 54.1.1.8 所示：

	1/2	1/4	1/8	
nECCe=	0^1^0^1	0^1^0^1	1^1^1^1	=000
nECCo=	0^1^0^1	0^1^0^1	0^0^0^0	=000

图 54.1.1.8 出错时计算的奇偶 ECC 值

可以看到，此时的 nECCo 和 nECCe 都为 000。此时，我们把所有 4 个 ECC 数值进行按位“异或”，就可以判断是否出现了 1 个单一比特的错误或者多比特的错误。如果计算结果全为“0”，说明数据在读写过程中未发生变化。如果计算的结果全为“1”，表明发生了 1 bit 错误，其他情况，则说明至少 2bit 数据出现了错误。不过，汉明码编码算法只能保证更正单一比特错误，如果两个或是更多的比特出错，汉明码就无能为力了（可以检测出错误，但无法修正）。不过，一般情况 SLC NAND 器件（STM32 仅支持 SLC NAND）出现 2bit 及以上的错误非常罕见，所以，我们使用的汉明码基本上够用。

对 4 个 ECC 进行异或的计算方法如下：

ECCe ^ (ECCo ^ (nECCe ^ nECCo)) = 101 ^ (010 ^ (000 ^ 000)) = 111

这样，经过上式计算，4 个 ECC 的“异或”结果全为 1，表示有 1 个 bit 出错了。对于这种 1bit 错误的情况，出错的地址可通过将原有 ECCo 值和新 ECCo 值（nECCo）进行按位“异或”来得到，计算方法如下：

ECCo ^ nECCo = 010 ^ 000 = 010

计算结果为 010（2），表明原数据第 2bit 位出现了问题。然后，对出错后的数据的 bit2 进行取反（该位与 1“异或”即可），就可以得到正确的数据：01010101^00000100=01010001。

一个 8 位数据，需要 6 位 ECC 码，看起来效率不高，但是随着数据的增多，汉明码编码效率将会越来越高。比如，我们一般以 512 字节为单位来计算 ECC 值，只需要 24bit 的 ECC 码即可表示（2^12=4096bit=512 字节，12*2=24bit）。汉明码编码算法原理，我们就给大家介绍到这里。

2.STM32 硬件 ECC

STM32 的 FMC 支持 NAND FLASH 硬件 ECC 计算，采用的就是汉明码计算方法。可以实现 1bit 错误的修正和 2bit 以上错误的检测，支持页大小按 256、512、1024、2048、4096 和 8192 字节为单位进行 ECC 计算。

当 FMC 的硬件 ECC 功能开启后，FMC 模块根据用户设置的参数（计算页大小、数据位宽等）对 NAND FLASH 数据线上传递的（读/写）数据进行 ECC 计算，数据传输结束后，ECC 计算结果自动存放在 FMC_ECCRx(x=2/3)寄存器中。不过 STM32 的硬件 ECC 只负责计算 ECC 值，并不对数据进行修复。错误检测和数据修复，需要用户自己实现。另外，**STM32 的硬件 ECC 支持存储区域 3，其他存储区域不支持！**

STM32 硬件 ECC 计算结果读取过程（以 512 字节页大小为例）：

1. 设置 FMC_PCRx(x=2/3)的 ECCEN 位为 1，使能 ECC 计算
2. 写入/读取 512 字节数据
3. 等待 FMC_SRx(x=2/3)的 FEMPT 位为 1（等待 FIFO 空）
4. 读取 FMC_ECCRx(x=2/3)，得到 ECC 值
5. 设置 FMC_PCRx(x=2/3)的 ECCEN 位为 0，关闭 ECC，以便下一次重新计算

重复以上步骤，就可以在不同时刻进行读/写数据的 ECC 计算。在实际使用的时候，我们在写入/读取数据时，都要开启 STM32 的硬件 ECC 计算。写入的时候，将 STM32 硬件 ECC 计算出来的 ECC 值写入 NAND FLASH 数据所存 Page 的 spare 区。在读取数据的时候，STM32 硬件 ECC 会重新计算一个 ECC 值（ecccl），而从 spare 区对应位置，又可以读取之前写入的 ECC 值（eccrd），当这两个 ECC 值不相等的时候，说明读数有问题，需要进行 ECC 校验，ECC 检查和修正代码如下：

```
/**
 * @brief       获取 ECC 的奇数位/偶数位
 * @param       oe:0,偶数位; 1,奇数位
```

```
 *              eccval:输入的 ecc 值
 * @retval      计算后的 ecc 值(最多 16 位)
 */
uint16_t nand_ecc_get_oe(uint8_t oe, uint32_t eccval)
{
    uint8_t i;
    uint16_t ecctemp = 0;

    for (i = 0; i < 24; i++)
    {
        if ((i % 2) == oe)
        {
            if ((eccval >> i) & 0X01)
            {
                ecctemp += 1 << (i >> 1);
            }
        }
    }
    return ecctemp;
}

/**
 * @brief       ECC 校正函数
 * @param       eccrd:读取出来,原来保存的 ECC 值
 *              ecccl:读取数据时,硬件计算的 ECC 值
 * @retval      0,错误已修正;其他,ECC 错误(有大于 2 个 bit 的错误,无法恢复)
 */
uint8_t nand_ecc_correction(uint8_t* data_buf,
                            uint32_t eccrd, uint32_t ecccl)
{
    uint16_t eccrdo, eccrde, eccclo, ecccle;
    uint16_t eccchk = 0;
    uint16_t errorpos = 0;
    uint32_t bytepos = 0;
    eccrdo = nand_ecc_get_oe(1, eccrd); /* 获取 eccrd 的奇数位 */
    eccrde = nand_ecc_get_oe(0, eccrd); /* 获取 eccrd 的偶数位 */
    eccclo = nand_ecc_get_oe(1, ecccl); /* 获取 ecccl 的奇数位 */
    ecccle = nand_ecc_get_oe(0, ecccl); /* 获取 ecccl 的偶数位 */
    eccchk = eccrdo^eccrde^eccclo^ecccle;

    if (eccchk == 0XFFF)                /* 全 1,说明只有 1bit ECC 错误 */
    {
        errorpos = eccrdo^eccclo;
        printf("errorpos:%d\r\n", errorpos);
        bytepos = errorpos / 8;
        data_buf[bytepos]^=1 << (errorpos % 8);
    }
    else                                /* 不是全 1,说明至少有 2bit ECC 错误,无法修复 */
    {
        printf("2bit ecc error or more\r\n");
        return 1;
    }
    return 0;
}
```

经过以上代码处理，我们就可以利用 STM32 的硬件 ECC，修正 1bit 错误，并报告 2bit 及以上错误。

54.1.2 FTL 简介

因为 NAND FLASH 在使用过程中可能会产生坏块，且每个 BLOCK 的擦除次数是有限制的，超过规定次数后，BLOCK 将无法再擦除（即产生坏块），因此，我们需要这样一段程序，它可以实现：1、坏块管理；2、磨损均衡；从而使应用程序可以很方便的访问 NAND FLASH

（无需关心坏块问题），且最大限度的延长 NAND FLASH 的寿命。

这里给大家介绍 FTL，FTL 是 Flash Translation Layer 的简写，即闪存转换层，它是一个 NAND 闪存芯片与基础文件系统之间的一个转换层，它自带了坏块管理和磨损均衡算法，使得操作系统和文件系统能够像访问硬盘一样访问 NAND 闪存设备，而无需关心坏块和磨损均衡问题。

本章，我们将给大家介绍一个比较简单的 FTL 层算法，它可以支持坏块管理和磨损均衡，提供支持文件系统（如：FATFS）的访问接口，通过这个 FTL，我们可以很容易的实现 NAND FLASH 的文件系统访问。

要做好 NAND FLASH 的坏块管理，我们有以下几点需要实现：

1， 如何识别坏块，标记坏块；

2， 转换表

3， 保留区

1， 如何识别坏块，标记坏块

经过前面的介绍，我们知道 NAND 在使用过程中，会产生坏块，而坏块我们是不能再用来存储数据的，必须对坏块进行识别和标记，并保存这些标记。

NAND FLASH 的坏块识别有几种方式：1，NAND 厂家出厂的时候，会在每个 Block 的第一个 page 和第二个 page 的 spare 区的第一个字节写入非 0XFF 的值来表示，我们可以通过这个判断该块是否为坏块；2，通过给每个 Block 写入数值（0XFF/0X00），然后读取出来，判断写入的数据和读取的数据是否完全一样，来识别坏块；3，通过读取数据时，校验 ECC 错误，来识别坏块。

NAND FLASH 的坏块标记：坏块标记，我们使用每个 Block 的第一个 page 和第二个 page（第二个 page 是备份用的）spare 区的第一个字节来标记，当这个字节的值为 0XFF 时，表示该块为好块，当这个字节的值不等于 0XFF 时，表示该块为坏块。以 FSNS8B004G 为例，坏块表示方法，如表 54.1.2.1 所示：

BLOCK 编号	PAGE 编号	数据区			Spare 区			
		0	……	2047	0	1	……	63
n	0		……		0XAA			
	1		……		0XAA			
	……							
	63		……					

表 54.1.2.1 NAND FLASH 坏块标记说明表

上图中，假设某个 Block 为坏块，那么它的第一个 page 和第二个 page 的 spare 区第一个字节，就不是 0XFF 了（我们改为 0XAA），以表示其是一个坏块。如果是好块，这两个字节，必须都是 0XFF，只要任何一个不是 0XFF，则表示该块是一个坏块。

这样，我们只需要判断每个 Block 的第一个和第二个 page 的 spare 区的第一个字节，就可以判断是否为坏块。达到了标记和保存坏块标记的目的。

2， 转换表

文件系统访问文件的时候，使用的是逻辑地址，是按顺序编号的，它不考虑坏块情况。而 NAND FLASH 存储地址，我们称为物理地址，是有可能存在坏块的。所以，这两个地址之间，必须有一个映射表，将逻辑地址转换为物理地址，且不能指向坏块的物理地址，这个映射表我们称之为逻辑地址-物理地址转换表，简称转换表，如图 54.1.2.1 所示：

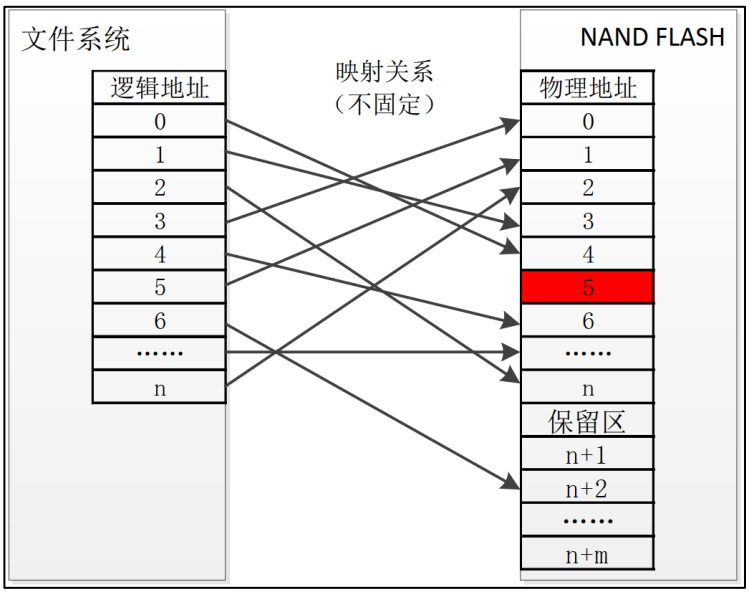


图 54.1.2.1 逻辑-物理地址转换表

图 54.1.2.1 表示某个时刻，逻辑地址与物理地址的对应关系。由图可知，逻辑地址（0~n）到物理地址的映射，映射关系不是一一对应的，而是无序的，这个映射关系是不固定的，随时可能会变化，逻辑地址到物理地址，通过映射表进行映射，所以映射表也是随时需要更新的。

图中，我们假定 NAND 的第 5 个 Block 是坏块，那么映射表一定不能将这个块地址映射给逻辑地址，所以，必须对这个块进行坏块标记，不再作为正常块使用，坏块标记请参考前面的介绍。另外，当产生了一个坏块的时候，我们必须从保留区（下文介绍）提取一个未用过的块（Block），来替代这个坏块，以确保所有逻辑地址，都有正常的物理地址可用。

逻辑地址到物理地址的映射关系，我们采用一个数组来存储，这个数组即映射表（简称：lut 表），同时，这个映射表必须存储到 NAND FLASH 里面，以便上电后重建。这里，我们也是利用每个 Block 的第一个 page 的 spare 区来存储映射表，另外，还需要标记这个 Block 是否被利用了，所以，Block 第一个 page 的 spare 区规划，如表 54.1.2.2 所示：

BLOCK 编号	PAGE 编号	数据区		Spare 区					
		0	…… 2047	0	1	2	3	……	63
n	0		……	0XFF	0XCC	0X01	0X00		
	1		……	0XFF					
	……	……							
	63		……						

表 54.1.2.2 每个 Block 第一个 page 的 spare 区前 4 个字节规划表

如表 54.1.2.2 所示，每个 Block 第一个 page 的 spare 区第一个字节用来表示该块是否为坏块（前面已介绍）；第二个字节用来表示是否被占用（0XFF，表示未占用；0XCC，表示已被占用）；第三和第四个字节，用来存储该块所映射到的逻辑地址，如果为 0XFFFF，则表示该块还未被分配逻辑地址，其他值，则表示该块对应的逻辑地址，FSNS8B004G 有 4096 个 Block，所以这两个字节表示的有效逻辑地址范围，就是 0~4095。因此，我们要想判断一个 Block 是否是被一个未被使用的好块，只需读取这个 Block 第一个 page 的 spare 区前四个字节，如果为 0XFFFFFFFF 则说明是一个未被使用的好块。

上电的时候，重建映射表（lut 表）的过程就是读取 NAND FLASH 每个 Block 第一个 page 的 spare 区前 4 个字节，当这个块是好块（第一个字节为 0XFF），且第三和第四字节组成的 uint16_t 类型数据（逻辑地址，记为：LBNnum），小于 NAND FLASH 的总块数，则这个 Block 地址（物理地址，记为：M）就是映射表里面第 LBNnum 个元素所对应的地址，即：lut[LBNnum]=M。

3.保留区

保留区有两个作用：1，产生坏块的时候，用来替代坏块；2，在复写数据的时候，用来替

代被复写的块，以提高写入速度，并实现磨损均衡处理；

第一个作用，如图 54.1.2.1 所示，当产生坏块后（第 5 个块），使用一个保留区里面的块（n+2），来替代这个坏块，从而保证每个逻辑地址，都有对应的物理地址（好块地址）。

第二个作用，当文件系统要往某个已经被写过数据的块里面写入新数据的时候，由于 NAND 的特性，必须是要先擦除，才能写入新数据。一般的方法：先将整个块数据读出来，然后改写需要写入新数据的部分，再擦除这个块，接着重新写入这个块，这个过程非常耗时，且需要很大的内存（FSNS8B004G 一个 Block 大小为 128K 字节），所以不太实用。

比较好的办法是利用 NAND FLASH 的页拷贝功能，它可以将 NAND FLASH 内部某个 Block 的数据，以页为单位，拷贝到另一个空闲的 Block 里面，而且可以写入新的数据（参见 54.1.1 节的指令介绍），利用这个功能，我们无需读出整个 block 的数据，只需要在页拷贝过程中，在正确的地址写入我们需要写入的新数据即可。这就要求 NAND FLASH 必须有空闲的块，用作页拷贝的目标地址，保留区里面的块，就可以作为空闲块，给页拷贝使用。而且，为了保证不频繁擦除一个块（提高寿命），我们在保留区里面应该预留足够的空闲块，用来均分擦除次数，从而实现简单的磨损均衡处理。

这样，FTL 层的坏块管理和磨损均衡原理，就给大家介绍完了。我们根据这个原理，去设计相应的代码，就可以实现 FTL 层的功能，从而更好的使用 NAND FLASH。

前面提到，我们需要用到 ECC 校验，来确保数据的正确性，一般的软件 ECC 校验都是由 FTL 层实现，我们为了简化代码，并利用 STM32 的硬件 ECC 计算，加快 ECC 计算的速度，我们在 FTL 层并不做 ECC 计算和校验，而是放到 NAND FLASH 的底层驱动去实现。在 54.1.1 节最后，我们给大家介绍了 ECC 原理和纠错方法，每 512 个字节的数据，会生成 3 个字节的 ECC 值，而这个 ECC 值是必须存储在 NAND FLASH 里面，同样我们将 ECC 值存储在每个 page 的 spare 区，而且，为了方便读写，我们用 4 个字节来存储这 3 个字节的 ECC 值，ECC 存储关系如表 54.1.2.3 所示：

BLOCK 编号	PAGE 编号	数据区			Spare 区							
		0	…	2047	0	…	16:19	20:23	24:27	28:31	…	63
n	0		…		0XFF	…	ECC01	ECC02	ECC03	ECC04	…	
	1		…		0XFF	…	ECC11	ECC12	ECC13	ECC14	…	
	2		…			…	ECC21	ECC22	ECC23	ECC24	…	
	…	…										
	63		…			…	ECC631	ECC632	ECC633	ECC634	…	

表 54.1.2.3 每个 page 的 spare 区 ECC 存储关系表

如上表所示，每个 page 的一个数据区有 2048 个字节，而每个 512 字节数据生成一个 ECC 值，用 4 个字节存储，这样，每个 page 需要 16 个字节用于存储 ECC 值。表中的 ECCx1~ECCx4（x=0~63），就是存储的 ECC 值，从每个 page 的 spare 区第 16 个字节（0X10）开始存储 ECC 值，总共占用 16 字节，对应关系为：ECCx1→数据区 0~511 字节；ECCx2→数据区 512~1023 字节；ECCx3→数据区 1024~1535 字节；ECCx4→数据区 1536~2047 字节；

在 page 写入数据的时候，我们将 STM32 硬件计算出的 ECC 值写入 spare 区对应的地址，当读取 page 数据的时候，STM32 硬件 ECC 会计算出一个新的 ECC 值，同时，可以在该 page 的 spare 区读取之前保存的 ECC 值，比较这两个 ECC 值，就可以判断数据是否有误，以及进行数据修复（1bit）。**注意：我们对 ECC 处理是以 512 字节为单位的，如果写入/读取的数据不是 512 的整数倍，则不会进行 ECC 处理。**

54.1.3 FMC NAND FLASH 接口简介

在第二十五和二十六章，我们对 STM32H743 的 FMC 接口进行了简单介绍，并利用 FMC 接口，实现了对 MCU 屏和 SDRAM 的驱动。本章，我们将介绍如何利用 FMC 接口，驱动 NAND FLASH。STM32H743 FMC 接口的 NAND FLASH/PC 控制器，具有如下特点：

- 两个 NAND FLASH 存储区域，可独立配置
- 支持 8 位和 16 位 NAND FLASH
- 支持 16 位 PC 卡兼容设备
- 支持硬件 ECC 计算（汉明码）
- 支持 NAND FLASH 预等待功能

通过 54.1.1 的介绍，我们对 NAND FLASH 已经有了一个比较深入的了解，包括接线、控制命令和读写流程等。

54.1.3.1 FMC NAND FLASH 寄存器

- **PC 卡/ NAND FLASH 控制寄存器（FMC_PCRx）**

NAND FLASH 控制寄存器描述如图 54.1.3.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ECCPS			TAR				TCLR				Res.	Res.	ECCEN	PWID		Res.	PBKEN	PWAITEN	Res.
												rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw			rw	rw	rw		rw	rw	

图 54.1.3.1 FMC_PCRx 寄存器各位描述

该寄存器只有部分位有效，且都需要进行配置：

PWAITEN：该位用于设置等待特性： 0，禁止； 1，使能。这里我们设置为 0，禁止使用控制器自带的等待特性，因为如果使能的话，将导致 RGB 屏抖动（ STM32 硬件 bug）。

PBKEN：该位用于使能存储区域： 0，禁止； 1，使能。我们要正常使用某个存储区域，必须设置该位为 1，所以，这个位要设置为 1。

PWID：这两个位，用于设置数据总线宽度： 00， 8 位宽度； 01， 16 位宽度。我们使用的 FSNS8B004G 为 8 位宽度，所以这里应该设置为： 00。

ECCEN：该位用于使能 STM32 的硬件 ECC 计算逻辑： 0，禁止/复位 ECC； 1，使能 ECC 计算；每次读写数据前，应该设置该位为 1，在数据读写完毕，读取完 ECC 值之后，设置该位为 0，复位 ECC，以便下一次 ECC 计算。

TCLR：这四个位用于设置 CLE 到 RE 的延迟： 0000~1111，表示 1~16 个 fmc_ker_ck 周期。对应 NAND FLASH 数据手册的 tCLR 时间参数，这里设置的 t_clr=(TCLR+SET+2)*Tfmc_ker_ck。TCLR 就是本寄存器的设置值，SET 对应 MEMSET 的值（我只用到 MENSET），Tfmc_ker_ck 对应 fmc_ker_ck 的周期。 FSNS8B004G 的 tCLR 时间最少为 10ns，以 220M 的 fmc_ker_ck 频率计算，一个 fmc_ker_ck=4.5ns（下同），我们设置 TCLR=5，则 t_clr 至少为 7 个 fmc_ker_ck 即 31.5ns。

TAR：这四个位用于设置 ALE 到 RE 的延迟： 0000~1111，表示 1~16 个 HCLK 周期。对应 NAND FLASH 数据手册的 tAR 时间参数，这里设置的 t_ar=(TAR+SET+2)*Tfmc_ker_ck。TAR 就是本寄存器的设置， SET 对应 MEMSET 的值（我只用到 MEMSET）， Tfmc_ker_ck 对应 fmc_ker_ck 的周期。FSNS8B004G 的 tAR 时间最少为 10ns，我们设置 TAR=5，则 t_ar 至少为 7 个 fmc_ker_ck 即 31.5ns。

ECCCPS：这三个位用于设置 ECC 的页大小： 000， 256 字节； 001， 512 字节； 010， 1024 字节； 011， 2048 字节； 100， 4096 字节； 101， 8192 字节。我们需要以 512 字节为单位进行 ECC 计算，所以 ECCCPS 设置为： 001 即可。

- **NAND FLASH 空间时序寄存器（FMC_PMEMx）**

FMC_PMEMx 寄存器各位描述如图 54.1.3.2 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MEMHIZx								MEMHOLDx							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MEMWAITx								MEMSETx							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 54.1.3.2 FMC_PMEMx 寄存器各位描述

该寄存器用于控制 NAND FLASH 的访问时序，非常重要。我们先来了解下 NAND FLASH

控制器的通用存储器访问波形，如图 54.1.3.3 所示：

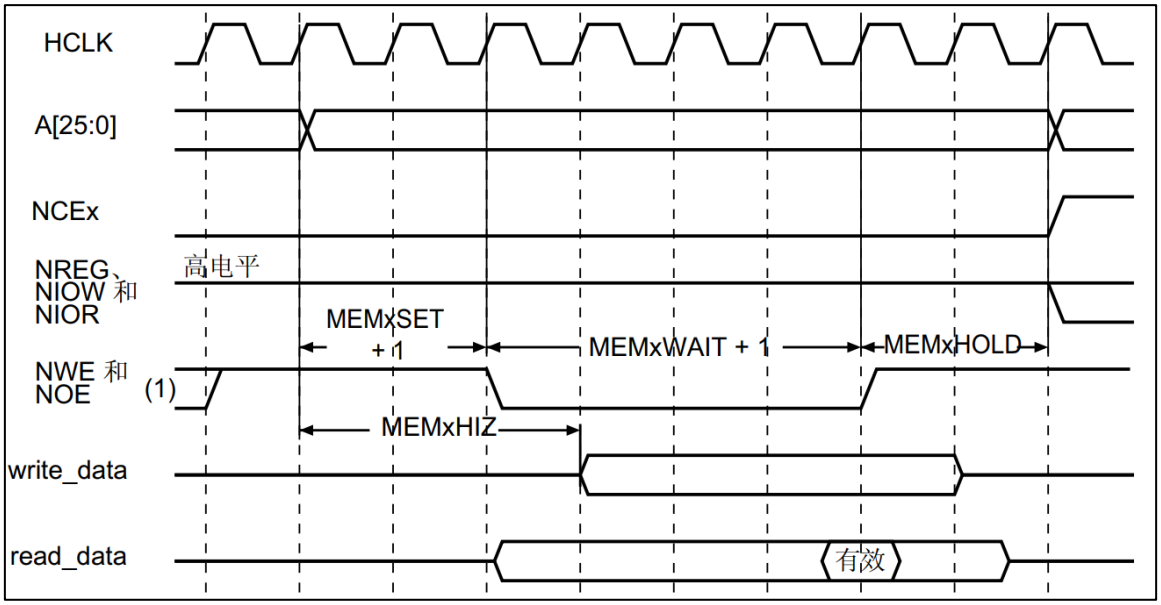


图 54.1.3.3 NAND FLASH 通用存储器访问波形

由图可知，MEMxSET+MEMxHOLD 控制 NWE/NOE 的高电平时间，MENxWAIT 控制 NWE/NOE 的低电平时间，MEMxHIZ 控制写入时数据线高阻态时间。接下来分别介绍这几个参数：

MEMSETx：这八个位定义使能命令（NWE/NOE）前，地址建立所需要的 fmc_ker_ck 时钟周期数，表示 NWE/NOE 的高电平时间，0000 0000~1111 1111 表示 1~256 个 fmc_ker_ck 周期。FSNS8B004G 的 tREH/tWH 最少为 10ns，我们设置 MEMSETx=3，即 4 个 fmc_ker_ck 周期，约 18ns。另外，MEMHOLDx，也可以用于控制 NWE/NOE 的高电平时间，在连续访问的时候，MEMHOLDx 和 MEMSETx 共同构成 NWE/NOE 的高电平脉宽时间。

MEMWAITx：这八个位用于设置使能命令（NWE/NOE）所需的最小 fmc_ker_ck 时钟周期数（使能 NWAIT 将这个时间延长），实际上就是 NWE/NOE 的低电平时间，0000 0000~1111 1111 表示 1~256 个 HCLK 周期。FSNS8B004G 的 tRP/tWP 最少为 10ns，我们设置 MEMWAITx=5，即 6 个 fmc_ker_ck 周期，约 27ns。这里需要设置时间比较长一点，否则可能访问不正常。

MENHOLDx：这八个位用于设置禁止使能命令（NWE/NOE）后，保持地址（和写访问数据）的 fmc_ker_ck 时钟周期数，也可以用于设置一个读写周期内的 NWE/NOE 高电平时间，0000 0000~1111 1111 表示 0~254 个 HCLK 周期。我们设置 MENHOLDx=2，表示 2 个 fmc_ker_ck 周期，加上前面的 MEMSETx，所以 NWE/NOE 高电平时间为 6 个 HCLK，即 27ns 左右。

MEMHIZx：这八个位定义通用存储空间开始执行写访问之后，数据总线保持高阻态所持续的 fmc_ker_ck 时钟周期数。该参数仅对写入事务有效，0000 0000~1111 1111 表示 0~254 个 fmc_ker_ck 周期。我们设置 MENHIZx=3，表示 3 个 fmc_ker_ck 周期，即 13.5ns 左右。

- **ECC 结果寄存器（FMC_ECCRx）**

FMC_ECCRx 寄存器各位描述如图 54.1.3.4 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ECC[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ECC[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

图 54.1.3.4 FMC_ECCRx 寄存器各位描述

该寄存器包含由 ECC 计算逻辑计算所得的结果。根据 ECCPS 位（在 FMC_PCRx 寄存器）的设置，ECCx 的有效位数也有差异，如表 54.1.3.1 所示：

ECCPS[2:0]	以字节为单位的页大小	ECC 位
000	256	ECC[21:0]
001	512	ECC[23:0]
010	1024	ECC[25:0]
011	2048	ECC[27:0]
100	4096	ECC[29:0]
101	8192	ECC[31:0]

表 54.1.3.1 ECC 结果相关位

我们以 512 字节为页大小（ECCPS=001），所以 ECCx 的低 24 位有效，用于存储计算所得的 ECC 值。

- **FIFO 状态和中断寄存器（FMC_SRx）**

FMC_SRx 寄存器各位描述如图 54.1.3.5 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FEMPT	IFEN	ILEN	IREN	IFS	ILS	IRS
									r	rw	rw	rw	rw	rw	rw

图 54.1.3.5 FMC_SRx 寄存器各位描述

该寄存器我们只关心第 6 位：EFMPT 位，该位用于表示 FIFO 的状态。当 EFMPT=0 时，表示 FIFO 非空，表示还有数据在传输；当 EFMPT=1 时，表示 FIFO 为空，表示数据传输完成。在计算 ECC 的时候，我们必须等待 EFMPT=1，再去读取 ECCRx 寄存器的值，确保数据全部传输完毕。

关于 FMC NAND FLASH 控制器的详细介绍，请大家参考《STM32H7xx 参考手册_V3（中文版）.PDF》第 22 节。

54.2 硬件设计

1. 例程功能

本章实验功能简介：开机的时候先检测 NAND FLASH 并初始化 FTL，如果初始化成功，则显示提示信息，然后按下 KEY0 按键，可以通过 FTL 读取扇区 2 的数据；按 KEY1 按键，可以通过 FTL 写入扇区 2 的数据；按 KEY2 按键，则可以恢复扇区 2 的数据（防止损坏文件系统）；LED0 指示程序运行状态。

2. 硬件资源

1）LED 灯
LED：LED0 – PB1
2）正点原子 2.8/3.5/4.3/7 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)
3）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）独立按键
KEY0 – PH3
KEY1 – PH2
KEY2 – PC13
4） NAND FLASH(FSNS8B004G，512MB)

3. 原理图

开发板板载的 FSNS8B004G 芯片挂在 FMC NAND FLASH 的控制器 3 上面（NCE3），其原理图如图 54.2.3.1 所示：

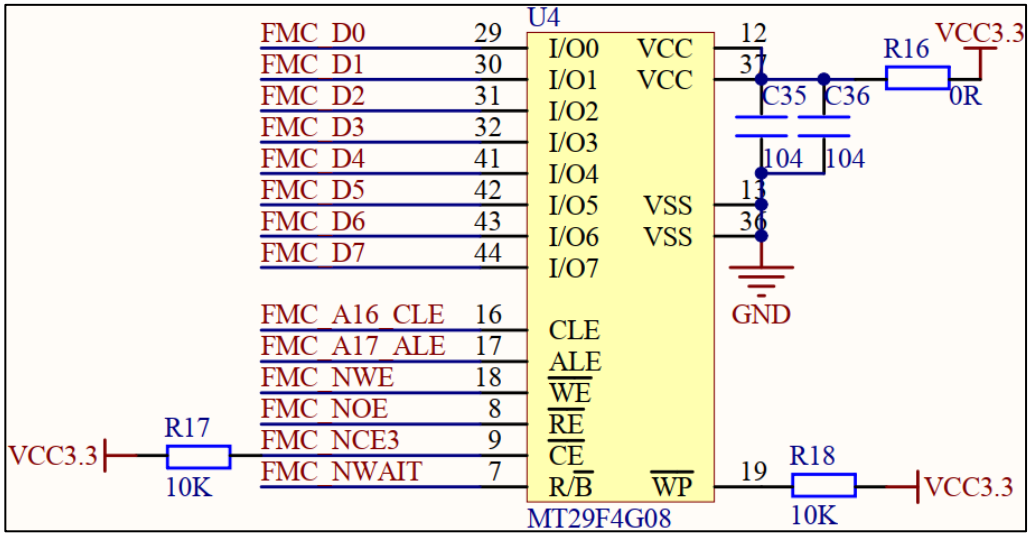


图 54.2.3.1 FSNS8B004G 原理图

从原理图可以看出，MT28F4G08 同开发板的连接关系：

I/O[0:7]接 FMC_D[0:7]
CLE 接 FMC_A16_CLE
ALE 接 FMC_A17_ALE
WE 接 FMC_NWE
RE 接 FMC_NOE
CE 接 FMC_NCE3
R/B 接 FMC_NWAIT

54.3 程序设计

54.3.1 FSNS8B004G 的 HAL 库驱动

FSNS8B004G 的驱动需要引入 stm32xx_II_fmc.c/stm32h7xx_hal_nand.c 源文件以及对应的头文件。具体步骤如下：

1） 使能 FMC 时钟，并配置 FMC 相关的 IO 及其时钟使能

要使用 FMC，当然首先得开启其时钟。然后需要把 FMC_D0~7、FMC_A16_CLE 和 FMC_A17_ALE 等相关 IO 口，全部配置为复用输出，并使能各 IO 组的时钟。使能 FMC 时钟和 IO 口初始化方法前面多次讲解，这里就不介绍了。

2） 初始化 NAND，设置控制参数（设置 FMC_PCR3）和时间参数（设置 FMC_PMEM3）

该步骤通过设置寄存器 FMC_PCR3（因为使用的是 FMC_NAND_BANK3 所以对应寄存器 FMC_PCR3）来设置 NAND FLASH 的相关控制参数，比如数据宽度、CLR/AR 延迟、ECC 页大小等，通过设置寄存器 FMC_PMEM3(因为使用的是 FMC_NAND_BANK3，所以对应寄存器 FMC_PMEM3)来设置 NAND 的相关时间参数，控制 FMC 访问 NAND FLASH 的时序。HAL 库提供了 NAND FLASH 初始化函数 HAL_NAND_Init:

```
HAL_StatusTypeDef  HAL_NAND_Init(NAND_HandleTypeDef *hnand,
                                 FMC_NAND_PCC_TimingTypeDef *ComSpace_Timing,
                                 FMC_NAND_PCC_TimingTypeDef *AttSpace_Timing);
```

该函数有三个入口参数，第一个入口参数 hnand 用来设置 NAND FLASH 的控制参数，第二个入口参数 ComSpace_Timing 用来设置 NAND 通用存储器空间时序，第三个入口参数 AttSpace_Timing 用来设置 NAND 特性存储器空间时序。这里我们着重讲解第一个入口参数 hnand 的定义，该参数为 NAND_HandleTypeDef 结构体类型，该结构体定义为：

```
typedef struct
{
  FMC_NAND_TypeDef                *Instance;
  FMC_NAND_InitTypeDef             Init;
  HAL_LockTypeDef                  Lock;
  __IO HAL_NAND_StateTypeDef       State;
  NAND_DeviceConfigTypeDef         Config;
} NAND_HandleTypeDef;
```

这里我们主要关注成员变量 Init 的含义，该成员变量是 FMC_NAND_InitTypeDef 结构体类型，该结构体定义为：

```
typedef struct
{
  uint32_t NandBank;               /* BANK 编号 */
  uint32_t Waitfeature;            /* 等待特性使能/失能 */
  uint32_t MemoryDataWidth;        /* 数据总线宽度：8 位/16 位 */
  uint32_t EccComputation;         /* ECC 计算逻辑使能/失能 */
  uint32_t ECCPageSize;            /*ECC 页大小：256/512/1024/2048/4096/8192 字节*/
  uint32_t TCLRSetupTime;          /* CLE 到 RE 的延迟 */
  uint32_t TARSetupTime;           /* ALE 到 RE 的延迟 */
} FMC_NAND_InitTypeDef;
```

这些成员变量设置值对应的是 FMC_PCR 寄存器相应位，这些成员变量的含义我们都注释了，如有不了解的地方请参考前面 FMC_PCR 寄存器讲解。

对于 FMC 的时序参数，这里我们就不做过多的讲解，请参考前面讲解。

HAL 库通用为 NAND 初始化提供了 MSP 回调函数 HAL_NAND_MspInit:

```
void HAL_NAND_MspInit(NAND_HandleTypeDef *hnand);
```

该函数内部一般用来使能时钟，初始化 IO 口。

3）配置 FMC_PCR3 寄存器，使能存储区域 3

在 FMC 的配置完成后，设置 FMC_PCR3 寄存器的 PBKEN 位（bit2）为 1，使能存储区域 3。如果使用 HAL 库，那么在函数 HAL_NAND_Init 的尾部有使能存储区域的操作，我们就不需要重复进行此步骤。操作方法为：

```
__FMC_NAND_ENABLE(hnand->Instance, hnand->Init.NandBank);
```

通过以上几个步骤，我们就完成了 FMC 的配置，可以访问 FSNS8B004G 了，最后，我们使用的是 SDRAM 的 BANK3，所以 FSNS8B004G 的访问地址为 0X80000000，而 NAND FLASH 的命令/地址控制由 CLE/ALE 控制，也就是由 FMC_A17_CLE 和 FMC_A16_CLE 控制，因此，发送命令和地址的语句为：

```
*(vu8*)(0X80000000|(1<<17))=CMD;
*(vu8*)(0X80000000|(1<<16))=ADDR;
```

54.3.1 程序流程图

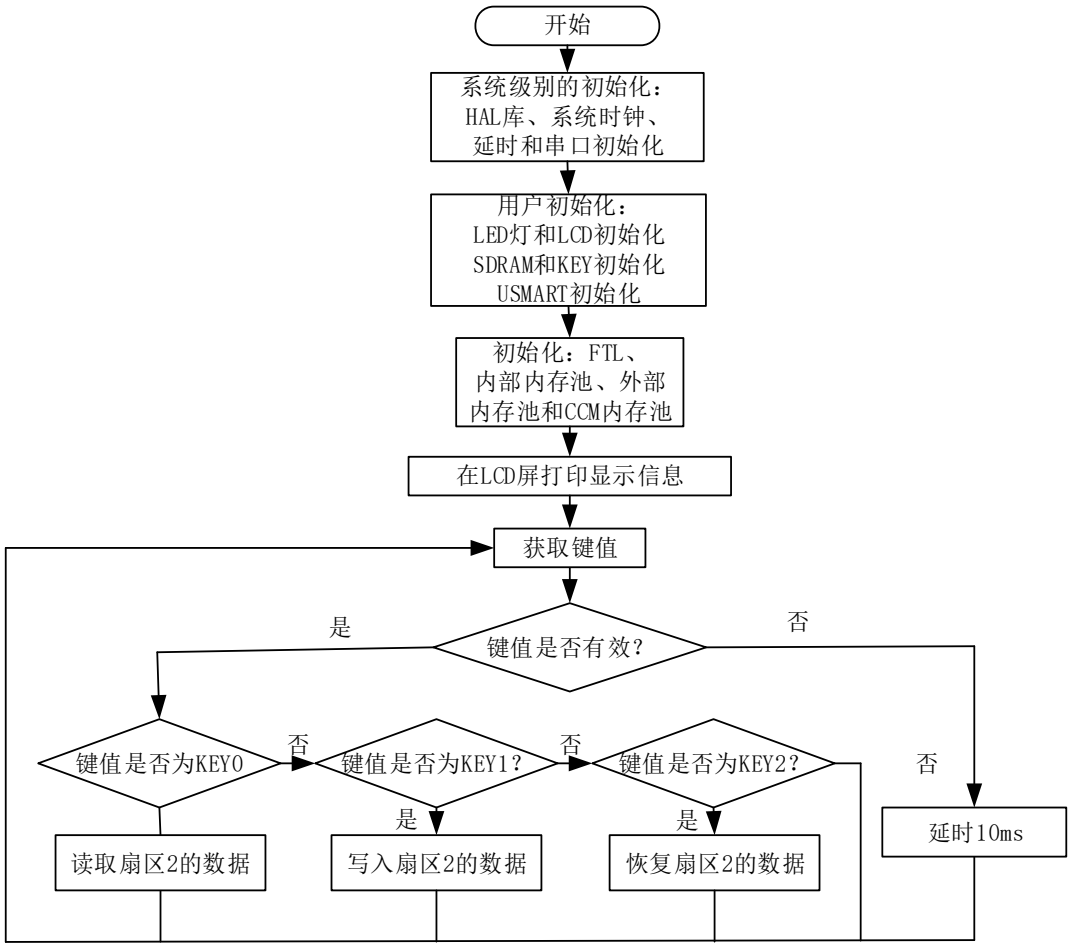


图 54.3.1.1 NAND FLASH 实验程序流程图

54.3.2 程序解析

1. NAND 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。

这里我们需要说明一下，由于 ST 官方 HAL 官方 HAL 库提供的 NAND 相关驱动函数我们在使用过程发现了很多兼容性问题，并且没有提供坏块管理操作，使用起来并不是非常方便。所以我们正点原子团队重写了一套 NAND 操作函数供大家参考。在 nand.c 里面，我们只介绍：nand_init、HAL_NAND_Mspinit、nand_readpage 和 nand_writepage 等三四个函数。

```
/**
 * @brief       初始化 NAND FLASH
 * @param       无
 * @retval      无
 */
uint8_t nand_init(void)
{
    FMC_NAND_PCC_TimingTypeDef comspacetiming, attspacetiming;

    g_nand_handle.Instance = FMC_NAND_DEVICE;
    /* NAND 挂在 BANK3 上 */
    g_nand_handle.Init.NandBank = FMC_NAND_BANK3;
    /* 关闭等待特性 */
    g_nand_handle.Init.Waitfeature=FMC_NAND_PCC_WAIT_FEATURE_DISABLE;
    /* 8 位数据宽度 */
```

```
    g_nand_handle.Init.MemoryDataWidth=FMC_NAND_PCC_MEM_BUS_WIDTH_8;
    /* 不使用 ECC */
    g_nand_handle.Init.EccComputation = FMC_NAND_ECC_DISABLE;
    /* ECC 页大小 2k */
    g_nand_handle.Init.ECCPageSize=FMC_NAND_ECC_PAGE_SIZE_2048BYTE;
    g_nand_handle.Init.TCLRSetupTime=0;
    /* 设置 TCLR(tCLR=CLE 到 RE 的延时)=(TCLR+TSET+2)*THCLK,THCLK=1/180M=5.5ns */
    g_nand_handle.Init.TARSetupTime = 1;
    /* 设置 TAR(tAR=ALE 到 RE 的延时)=(TAR+TSET+2)*THCLK,THCLK=1/180M=5.5n */

    comspacetiming.SetupTime = 2;           /* 建立时间 */
    comspacetiming.WaitSetupTime = 3;       /* 等待时间 */
    comspacetiming.HoldSetupTime = 2;       /* 保持时间 */
    comspacetiming.HiZSetupTime = 1;        /* 高阻态时间 */

    attspacetiming.SetupTime = 2;           /* 建立时间 */
    attspacetiming.WaitSetupTime = 3;       /* 等待时间 */
    attspacetiming.HoldSetupTime = 2;       /* 保持时间 */
    attspacetiming.HiZSetupTime = 1;        /* 高阻态时间 */

    HAL_NAND_Init(&g_nand_handle, &comspacetiming, &attspacetiming);
    nand_reset();                           /* 复位 NAND */
    delay_ms(100);
    nand_dev.id = nand_readid();            /* 读取 ID */
    nand_modeset(4);                        /* 设置为 MODE4,高速模式 */

    if (nand_dev.id == MT29F16G08ABABA)     /* NAND 为 MT29F16G08ABABA */
    {
        nand_dev.page_totalsize = 4320; /* nand 一个 page 的总大小（包括 spare 区）*/
        nand_dev.page_mainsize = 4096;      /* nand 一个 page 的有效数据区大小 */
        nand_dev.page_sparesize = 224;      /* nand 一个 page 的 spare 区大小 */
        nand_dev.block_pagenum = 128;       /* nand 一个 block 所包含的 page 数目 */
        nand_dev.plane_blocknum = 2048;     /* nand 一个 plane 所包含的 block 数目 */
        nand_dev.block_totalnum = 4096;     /* nand 的总 block 数目 */
    }
    else if (nand_dev.id == MT29F4G08ABADA) /* NAND 为 MT29F4G08ABADA */
    {
        nand_dev.page_totalsize = 2112; /* nand 一个 page 的总大小（包括 spare 区）*/
        nand_dev.page_mainsize = 2048;      /* nand 一个 page 的有效数据区大小 */
        nand_dev.page_sparesize = 64;       /* nand 一个 page 的 spare 区大小 */
        nand_dev.block_pagenum = 64;        /* nand 一个 block 所包含的 page 数目 */
        nand_dev.plane_blocknum = 2048;     /* nand 一个 plane 所包含的 block 数目 */
        nand_dev.block_totalnum = 4096;     /* nand 的总 block 数目 */
    }
    else if (nand_dev.id == FSNS8B004G) /* NAND 为 FSNS8B004GABADA */
    {
        nand_dev.page_totalsize = 4160; /* nand 一个 page 的总大小（包括 spare 区）*/
        nand_dev.page_mainsize = 4096;      /* nand 一个 page 的有效数据区大小 */
        nand_dev.page_sparesize = 64;       /* nand 一个 page 的 spare 区大小 */
        nand_dev.block_pagenum = 64;        /* nand 一个 block 所包含的 page 数目 */
        nand_dev.plane_blocknum = 1024;     /* nand 一个 plane 所包含的 block 数目 */
        nand_dev.block_totalnum = 2048;     /* nand 的总 block 数目 */
    }

    else return 1;                          /* 错误，返回 */

    return 0;
}
```

该函数用于初始化NAND FLASH，主要是调用函数HAL_NAND_Init函数初始化NAND，配置相关控制参数和FMC时序，另外，该函数会读取NAND ID，从而判断NAND FLASH的

型号，执行不同的参数初始化。nand_dev是我们在nand.h里面定义的一个NAND属性结构体，存储 NAND FLASH 的一些特性参数，方便驱动。

函数 HAL_NAND_MspInit 内容这里我们就不列出来了，该函数是 NAND 的 MSP 初始化回调函数，用来初始化与 MCU 相关的步骤，包括时钟使能和 IO 初始化。

接下来，我们看 nand_readpage 函数的代码，如下：

```
/**
 * @brief       读取 NAND Flash 的指定页指定列的数据(main 区和 spare 区都可以使用此函数)
 * @param       pagenum: 要读取的页地围:0~(block_pagenum*block_totalnum-1)
 * @param       colnum : 要读取的列开始地址(也就是页内地址),范围:0~(page_totalsize-1)
 * @param       *pbuffer         : 指向数据存储区
 * @param       numbyte_to_read : 读取字节数(不能跨页读)
 * @retval      0,成功; 其他,错误代码
 */
uint8_t nand_readpage(uint32_t pagenum, uint16_t colnum, uint8_t *pbuffer,
                      uint16_t numbyte_to_read)
{
    volatile uint16_t i = 0;
    uint8_t res = 0;
    uint8_t eccnum = 0;
    /* 需要计算的 ECC 个数，每 NAND_ECC_SECTOR_SIZE 字节计算一个 ecc */
    uint8_t eccstart = 0;    /* 第一个 ECC 值所属的地址范围 */
    uint8_t errsta = 0;
    uint8_t *p;

    *(volatile uint8_t*)(NAND_ADDRESS|NAND_CMD) = NAND_AREA_A;
    /* 发送地址 */
    *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)colnum;
    *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)(colnum >> 8);
    *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)pagenum;
    *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)(pagenum >> 8);
    *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)(pagenum >> 16);
    *(volatile uint8_t*)(NAND_ADDRESS | NAND_CMD) = NAND_AREA_TRUE1;

    /* 下面两行代码是等待 R/B 引脚变为低电平，其实主要起延时作用的，等待 NAND 操作 R/B 引脚。
       因为我们是通过将 STM32 的 NWAIT 引脚(NAND 的 R/B 引脚)配置为普通 IO，代码中通过读取
       NWAIT 引脚的电平来判断 NAND 是否准备就绪的。这个也就是模拟的方法，所以在速度很快的时候
       有可能 NAND 还没来得及操作 R/B 引脚来 表示 NAND 的忙闲状态，结果我们就读取了 R/B 引脚，
       这个时候肯定会出错的，事实上确实是会出错!大家也可以将下面两行
       代码换成延时函数,只不过这里我们为了效率所以没有用延时函数。 */
    res = nand_waitrb(0);      /* 等待 RB=0 */
    if (res)
    {
        return NSTA_TIMEOUT;  /* 超时退出 */
    }
    /* 下面 2 行代码是真正判断 NAND 是否准备好的 */
    res = nand_waitrb(1);      /* 等待 RB=1 */
    if (res)
    {
        return NSTA_TIMEOUT;  /* 超时退出 */
    }
    if (numbytetoread % NAND_ECC_SECTOR_SIZE)
    /* 不是 NAND_ECC_SECTOR_SIZE 的整数倍，不进行 ECC 校验 */
    {
        /* 读取 NAND FLASH 中的值 */
        for (i = 0; i < numbyte_to_read; i++)
        {
            *(volatile uint8_t*)pbuffer++ = *(volatile uint8_t*)NAND_ADDRESS;
        }
    }
    else
    {
```

```
        eccnum = numbyte_to_read / NAND_ECC_SECTOR_SIZE;  /* 得到 ecc 计算次数 */
        eccstart = colnum / NAND_ECC_SECTOR_SIZE;
        p = pbuffer;

        for (res = 0; res < eccnum; res++)
        {
            FMC_NAND_DEVICE->PCR |= 1 << 6;  /* 使能 ECC 校验 */
            for (i = 0; i < NAND_ECC_SECTOR_SIZE; i++)
            /* 读取 NAND_ECC_SECTOR_SIZE 个数据 */
            {
              *(volatile uint8_t*)pbuffer++ = *(volatile uint8_t*)NAND_ADDRESS;
            }

            while(!(FMC_NAND_DEVICE -> SR & (1 << 6))); /* 等待 FIFO 空 */
            nand_dev.ecc_hdbuf[res + eccstart] = FMC_NAND_DEVICE -> ECCR;
            /* 读取硬件计算后的 ECC 值 */
            FMC_NAND_DEVICE -> PCR &= ~(1 << 6);  /* 禁止 ECC 校验 */
        }

        i = nand_dev.page_mainsize + 0X10 + eccstart * 4;
        /* 从 spare 区的 0X10 位置开始读取之前存储的 ecc 值 */
        nand_delay(NAND_TRHW_DELAY);    /* 等待 tRHW */
        *(volatile uint8_t*)(NAND_ADDRESS | NAND_CMD) = 0X05; /* 随机读指令 */
        /* 发送地址 */
        *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)i;
        *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)(i >> 8);
        *(volatile uint8_t*)(NAND_ADDRESS | NAND_CMD) = 0XE0;  /* 开始读数据 */

        nand_delay(NAND_TWHR_DELAY);     /* 等待 tWHR */
        pbuffer = (uint8_t *)&nand_dev.ecc_rdbuf[eccstart];

        for (i = 0; i < 4 * eccnum; i++) /* 读取保存的 ECC 值 */
        {
            *(volatile uint8_t *)pbuffer++ = *(volatile uint8_t *)NAND_ADDRESS;
        }

        for (i = 0;i < eccnum; i++)      /* 检验 ECC */
        {
           if(nand_dev.ecc_rdbuf[i + eccstart]!=nand_dev.ecc_hdbuf[i+eccstart])
            /* 不相等,需要校正 */
            {
                printf("err hd,rd:0x%x,0x%x\r\n", nand_dev.ecc_hdbuf[i +
                       eccstart], nand_dev.ecc_rdbuf[i + eccstart]);
                printf("eccnum,eccstart:%d,%d\r\n", eccnum, eccstart);
                printf("PageNum,ColNum:%d,%d\r\n", pagenum, colnum);
                res = nand_ecc_correction(p + NAND_ECC_SECTOR_SIZE *
                       i,nand_dev.ecc_rdbuf[i + eccstart],
                       nand_dev.ecc_hdbuf[i + eccstart]);/* ECC 校验 */
                if (res)
                {
                    errsta = NSTA_ECC2BITERR; /* 标记 2BIT 及以上 ECC 错误 */
                }
                else
                {
                    errsta = NSTA_ECC1BITERR; /* 标记 1BIT ECC 错误 */
                }
            }
        }
    }

    if (nand_wait_for_ready() != NSTA_READY)
    {
        errsta = NSTA_ERROR;     /* 失败 */
    }
```

```
    return errsta;                  /* 成功 */
}
```

该函数用于读取 NAND 里面的数据，通过指定页地址（pagenum）和列地址（colnum），就可以读取 NAND FLASH 里面任何地址的数据，不过该函数读数据时不能跨页读，所以一次最多读取一个 Page 的数据（包括 spare 区数据）。当读取数据长度为 NAND_ECC_SECTOR_SIZE(512 字节)的整数倍时，将执行 ECC 校验，ECC 校验完全是按照 54.1 节介绍的方法来实现，当出现 ECC 错误时，调用 nand_ecc_correction 函数（54.1.1 节已经介绍）进行 ECC 纠错，可以实现 1bit 错误纠正，并报告 2bit 及以上的错误。

接下来，我们看 nand_writepage 函数的代码，如下：

```
/**
 * @brief       在 NAND 一页中写入指定个字节的数据(main 区和 spare 区都可以使用此函数)
 * @param       pagenum: 要写入的页地址,范围:0~(block_pagenum*block_totalnum-1)
 * @param       colnum : 要写入的列开始地址(也就是页内地址),范围:0~(page_totalsize-1)
 * @param       pbuffer: 指向数据存储区
 * @param       numbyte_to_write: 要写入的字节数，该值不能超过该页剩余字节数！！！
 * @retval      0,成功; 其他,错误代码
 */
uint8_t nand_writepage(uint32_t pagenum, uint16_t colnum, uint8_t *pbuffer,
                       uint16_t numbyte_to_write)
{
    volatile uint16_t i = 0;
    uint8_t res = 0;
    uint8_t eccnum = 0;
    /* 需要计算的 ECC 个数，每 NAND_ECC_SECTOR_SIZE 字节计算一个 ecc */
    uint8_t eccstart = 0;    /* 第一个 ECC 值所属的地址范围 */

    *(volatile uint8_t*)(NAND_ADDRESS | NAND_CMD) = NAND_WRITE0;
    /* 发送地址 */
    *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)colnum;
    *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)(colnum >> 8);
    *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)pagenum;
    *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)(pagenum >> 8);
    *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)(pagenum >> 16);
    nand_delay(NAND_TADL_DELAY);                    /* 等待 tADL */
    if (numbyte_to_write % NAND_ECC_SECTOR_SIZE)
    /* 不是 NAND_ECC_SECTOR_SIZE 的整数倍，不进行 ECC 校验 */
    {
        for (i = 0; i < numbyte_to_write; i++)              /* 写入数据 */
        {
            *(volatile uint8_t*)NAND_ADDRESS = *(volatile uint8_t*)pbuffer++;
        }
    }
    else
    {
        eccnum = numbyte_to_write / NAND_ECC_SECTOR_SIZE; /* 得到 ecc 计算次数 */
        eccstart = colnum / NAND_ECC_SECTOR_SIZE;

        for (res = 0; res < eccnum; res++)
        {
            FMC_NAND_DEVICE -> PCR |= 1 << 6;             /* 使能 ECC 校验 */
            for (i = 0; i < NAND_ECC_SECTOR_SIZE; i++)
            /* 写入 NAND_ECC_SECTOR_SIZE 个数据 */
            {
              *(volatile uint8_t*)NAND_ADDRESS = *(volatile uint8_t*)pbuffer++;
            }
            while (!(FMC_NAND_DEVICE -> SR & (1 << 6)));    /* 等待 FIFO 空 */

            nand_dev.ecc_hdbuf[res+eccstart] = FMC_NAND_DEVICE -> ECCR;
            /* 读取硬件计算后的 ECC 值 */

            FMC_NAND_DEVICE -> PCR &= ~(1 << 6);            /* 禁止 ECC 校验 */
```

```
        }
        i = nand_dev.page_mainsize + 0X10 + eccstart * 4;
        /* 计算写入 ECC 的 spare 区地址 */
        nand_delay(NAND_TADL_DELAY);                                /* 等待 tADL */
        *(volatile uint8_t*)(NAND_ADDRESS | NAND_CMD) = 0X85; /* 随机写指令 */

        /* 发送地址 */
        *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)i;
        *(volatile uint8_t*)(NAND_ADDRESS | NAND_ADDR) = (uint8_t)(i >> 8);
        nand_delay(NAND_TADL_DELAY);                                /* 等待 tADL */
        pbuffer = (uint8_t*)&nand_dev.ecc_hdbuf[eccstart];

        for (i = 0; i < eccnum; i++)                                /* 写入 ECC */
        {
            for (res = 0; res < 4; res++)
            {
              *(volatile uint8_t*)NAND_ADDRESS = *(volatile uint8_t*)pbuffer++;
            }
        }
    }
    *(volatile uint8_t*)(NAND_ADDRESS | NAND_CMD) = NAND_WRITE_TURE1;
    delay_us(NAND_TPROG_DELAY);                                     /* 等待 tPROG */

    if (nand_wait_for_ready() != NSTA_READY)
    {
        return NSTA_ERROR;                                          /* 失败 */
    }
    return 0;                                                       /* 成功 */
}
```

该函数用于往 NAND 里面写数据，通过指定页地址（pagenum）和列地址（colnum），就可以往 NAND FLASH 里面任何地址写数据（包括 spare 区），同样，该函数也不支持跨页写。当读取数据长度为 NAND_ECC_SECTOR_SIZE（512 字节）的整数倍时，将执行 ECC 校验，并将 ECC 值写入 spare 区对应的地址，以便读取数据时，进行 ECC 校验。

2. FTL 驱动代码

ftl_init 函数代码如下：

```
/**
 * @brief       FTL 层初始化
 * @param       无
 * @retval      0,正常; 其他,失败
 */
uint8_t ftl_init(void)
{
    uint8_t temp;

    if (nand_init())
    {
        return 1;           /* 初始化 NAND FLASH */
    }
    if (nand_dev.lut)
    {
        myfree(SRAMIN, nand_dev.lut);
    }
    /* 给 LUT 表申请内存 */
    nand_dev.lut = mymalloc(SRAMIN, (nand_dev.block_totalnum) * 2);
    memset(nand_dev.lut, 0, nand_dev.block_totalnum * 2); /* 全部清理 */

    if (!nand_dev.lut)
    {
        return 1;                   /* 内存申请失败  */
    }
```

```
    temp = ftl_create_lut(1);
    if (temp)
    {
        printf("format nand flash...\r\n");
        temp = ftl_format();      /* 格式化 NAND */

        if (temp)
        {
            printf("format failed!\r\n");
            return 2;
        }
    }
    else                          /* 创建 LUT 表成功 */
    {
        printf("total block num: %d\r\n", nand_dev.block_totalnum);
        printf("good block num: %d\r\n",  nand_dev.good_blocknum);
        printf("valid block num: %d\r\n", nand_dev.valid_blocknum);
    }
    return 0;
}
```

该函数用于初始化 FTL，包括：初始化 NAND FLASH、为 lut 表申请内存、创建 lut 表等操作，如果创建 lut 表失败，则会通过 ftl_format 函数初始化 NAND FLASH。

ftl_format 函数代码如下：

```
/**
 * @brief       格式化 NAND 重建 LUT 表
 * @param       无
 * @retval      0,成功;
                其他,失败
 */
uint8_t ftl_format(void)
{
    uint8_t temp;
    uint32_t i, n;
    uint32_t goodblock = 0;
    nand_dev.good_blocknum = 0;

#if FTL_USE_BAD_BLOCK_SEARCH == 1                   /* 使用擦-写-读的方式,检测坏块 */
    /* 搜寻坏块.耗时很久 */
    nand_dev.good_blocknum = FTL_SearchBadBlock();
#else                       /* 直接使用 NAND FLASH 的出厂坏块标志(其他块,默认是好块) */
    for (i = 0; i < nand_dev.block_totalnum; i++)
    {
        temp = ftl_check_badblock(i);               /* 检查一个块是否为坏块 */

        if (temp == 0)                              /* 好块 */
        {
            temp = nand_eraseblock(i);
            if (temp)                               /* 擦除失败,认为坏块 */
            {
                printf("Bad block:%d\r\n", i);
                ftl_badblock_mark(i);               /* 标记是坏块 */
            }
            else
            {
                nand_dev.good_blocknum++;           /* 好块数量加一 */
            }
        }
    }
#endif
    printf("good_blocknum:%d\r\n", nand_dev.good_blocknum);

    if (nand_dev.good_blocknum < 100)
```

```
    {
        return 1;                   /* 如果好块的数量少于 100，则 NAND Flash 报废 */
    }
    goodblock = (nand_dev.good_blocknum * 93) / 100;/* %93 的好块用于存储数据 */
    n = 0;
    for (i = 0; i < nand_dev.block_totalnum; i++)    /* 在好块中标记上逻辑块信息 */
    {
        temp = ftl_check_badblock(i);                /* 检查一个块是否为坏块 */

        if (temp == 0)                               /* 好块 */
        {
            nand_writespare(i * nand_dev.block_pagenum, 2, (uint8_t*)&n, 2);
            /* 写入逻辑块编号 */
            n++;                                     /* 逻辑块编号加 1 */

            if (n == goodblock) break;               /* 全部标记完了 */
        }
    }
    if (ftl_create_lut(1))
    {
        return 2;                                    /* 重建 LUT 表失败 */
    }
    return 0;
}
```

该函数用于格式化 NAND FLASH，执行的操作包括：1，检测/搜索整个 NAND 的坏块，并做标记；2，分割所有好块，93%用作物理地址（并进行逻辑编号），7%用作保留区；3，重新创建 lut 表。此函数，将我们在 54.1.2 节介绍的 FTL 层坏块管理的几个要点（识别坏块并标记、生成转换表、生成保留区），都实现了，从而完成对 NAND FLASH 的格式化（不是文件系统那种格式化，这里的格式化是指针对 FTL 层的初始化设置）。

接下来，我们看 ftl_create_lut 函数，该函数代码如下：

```
/**
 * @brief       重新创建 LUT 表
 * @param       mode:0,仅检查第一个坏块标记
 * @retval      1,两个坏块标记都要检查(备份区也要检查)
 * @retval      0,成功;其他,失败
 */
uint8_t ftl_create_lut(uint8_t mode)
{
    uint32_t i;
    uint8_t buf[4];
    uint32_t lbnnum = 0;                    /* 逻辑块号 */

    for (i = 0; i < nand_dev.block_totalnum; i++)
    /* 复位 LUT 表，初始化为无效值，也就是 0XFFFF */
    {
        nand_dev.lut[i] = 0XFFFF;
    }

    nand_dev.good_blocknum = 0;
    for (i = 0; i < nand_dev.block_totalnum; i++)
    {
        nand_readspare(i * nand_dev.block_pagenum, 0, buf, 4);/* 读取 4 个字节 */

        if (buf[0] == 0XFF && mode)
        {
            nand_readspare(i * nand_dev.block_pagenum + 1, 0, buf, 1);
            /* 好块,且需要检查 2 次坏块标记 */
        }

        if (buf[0] == 0XFF)             /* 是好块 */
        {
            lbnnum = ((uint16_t)buf[3] << 8) + buf[2]; /* 得到逻辑块编号 */
```

```
            if (lbnnum < nand_dev.block_totalnum)/* 逻辑块号肯定小于总的块数量 */
            {
                nand_dev.lut[lbnnum] = i;/* 更新 LUT 表，写 LBNnum 对应的物理块编号 */
            }
            nand_dev.good_blocknum++;
        }
        else
        {
            printf("bad block index:%d\r\n", i);
        }
    }
    /* LUT 表建立完成以后检查有效块个数 */
    for (i = 0; i < nand_dev.block_totalnum; i++)
    {
        if (nand_dev.lut[i] >= nand_dev.block_totalnum)
        {
            nand_dev.valid_blocknum = i;
            break;
        }
    }

    if (nand_dev.valid_blocknum < 100)
    {
        return 2;                   /* 有效块数小于 100,有问题.需要重新格式化 */
    }

    return 0;                       /* LUT 表创建完成 */
}
```

该函数用于重建 lut 表，读取保存在每个 Block 第一个 page 的 spare 区的逻辑编号，存储在 nand_dev.lut 表里，并初始化有效块（nand_dev.valid_blocknum）和好块（nand_dev.good_blocknum）的数量，完成转换表（lut 表）的创建。

接下来，我们看 ftl_lbn_to_pbn 函数，该函数代码如下：

```
/**
 * @brief       逻辑块号转换为物理块号
 * @param       lbnnum:逻辑块编号
 * @retval      物理块编号
 */
uint16_t ftl_lbn_to_pbn(uint32_t lbnnum)
{
    uint16_t pbnno = 0;

    /* 当逻辑块号大于有效块数的时候返回 0XFFFF */
    if (lbnnum > nand_dev.valid_blocknum)
    {
        return 0XFFFF;
    }

    pbnno = nand_dev.lut[lbnnum];

    return pbnno;
}
```

该函数用于将逻辑块地址改为物理块地址，输入参数：lbnnum，表示逻辑块编号，返回值表示 lbnnum 对应的物理块地址。有了该函数，就可以很方便的实现逻辑块地址到物理块地址的映射。

接下来，我们看 ftl_write_sectors 函数，该函数代码如下：

```
/**
 * @brief   写扇区(支持多扇区写)，FATFS 文件系统使用
 * @param   pbuffer    : 要写入的数据
 * @param   sectorno   : 起始扇区号
 * @param   sectorsize:扇区大小(不能大于 NAND_ECC_SECTOR_SIZE 定义的大小,否则会出错!!)
 * @param   sectorcount : 要写入的扇区数量
```

```
 * @retval  0,成功;其他,失败
 */
uint8_t ftl_write_sectors(uint8_t *pbuffer, uint32_t sectorno, uint16_t
                                          sectorsize, uint32_t sectorcount)
{
    uint8_t flag = 0;
    uint16_t temp;
    uint32_t i = 0;
    uint16_t wsecs;                 /* 写页大小 */
    uint32_t wlen;                  /* 写入长度 */
    uint32_t lbnno;                 /* 逻辑块号 */
    uint32_t pbnno;                 /* 物理块号 */
    uint32_t phypageno;             /* 物理页号 */
    uint32_t pageoffset;            /* 页内偏移地址 */
    uint32_t blockoffset;           /* 块内偏移地址 */
    uint32_t markdpbn = 0XFFFFFFFF; /* 标记了的物理块编号 */

    for (i = 0; i < sectorcount; i++)
    {
        lbnno = (sectorno+i) / (nand_dev.block_pagenum *
                (nand_dev.page_mainsize / sectorsize));
       /* 根据逻辑扇区号和扇区大小计算出逻辑块号 */
        pbnno = ftl_lbn_to_pbn(lbnno); /* 将逻辑块转换为物理块 */

        if (pbnno >= nand_dev.block_totalnum)
        {
            return 1; /* 物理块号大于 NAND FLASH 的总块数,则失败. */
        }

        blockoffset =((sectorno + i) % (nand_dev.block_pagenum *
        (nand_dev.page_mainsize / sectorsize))) * sectorsize; /* 计算块内偏移 */
        phypageno = pbnno * nand_dev.block_pagenum + blockoffset /
                    nand_dev.page_mainsize;             /* 计算出物理页号 */
        pageoffset = blockoffset % nand_dev.page_mainsize;
        /* 计算出页内偏移地址  */
        temp = nand_dev.page_mainsize - pageoffset; /* page 内剩余字节数 */
        temp /= sectorsize;       /* 可以连续写入的 sector 数  */
        wsecs = sectorcount - i; /* 还剩多少个 sector 要写 */

        if (wsecs >= temp)
        {
            wsecs = temp; /* 大于可连续写入的 sector 数,则写入 temp 个扇区 */
        }

        wlen = wsecs * sectorsize;     /* 每次写 wsecs 个 sector */

        /* 读出写入大小的内容判断是否全为 0XFF */
       flag = nand_readpagecomp(phypageno, pageoffset,0XFFFFFFFF,wlen/4,&temp);
       /* 读一个 wlen/4 大小个数据,并与 0XFFFFFFFF 对比 */

        if (flag)
        {
            return 2;                   /* 读写错误，坏块 */
        }
        if (temp == (wlen / 4))
        {
            flag = nand_writepage(phypageno, pageoffset, pbuffer, wlen);
            /* 全为 0XFF,可以直接写数据 */
        }
        else
        {
            flag = 1;       /* 不全是 0XFF,则另作处理 */
        }
```

```
        if (flag == 0 && (markdpbn != pbnno))
        /* 全是 0XFF,且写入成功,且标记了的物理块与当前物理块不同 */
        {
            flag = ftl_used_blockmark(pbnno);           /* 标记此块已经使用 */
            markdpbn = pbnno;       /* 标记完成,标记块=当前块,防止重复标记 */
        }
        if (flag)                   /* 不全为 0XFF/标记失败，将数据写到另一个块 */
        {
            temp = ((uint32_t)nand_dev.block_pagenum * nand_dev.page_mainsize
                   - blockoffset) / sectorsize;
            /* 计算整个 block 还剩下多少个 SECTOR 可以写入 */
            wsecs = sectorcount - i;        /* 还剩多少个 sector 要写 */

            if (wsecs >= temp)
            {
                wsecs = temp;/* 大于可连续写入的 sector 数,则写入 temp 个扇区 */
            }

            wlen = wsecs * sectorsize;    /* 每次写 wsecs 个 sector */
            flag = ftl_copy_and_write_to_block(phypageno, pageoffset,
                   pbuffer, wlen);        /* 拷贝到另外一个 block,并写入数据 */
            if (flag)
            {
                return 3;    /* 失败 */
            }
        }

        i += wsecs - 1;
        pbuffer += wlen;      /* 数据缓冲区指针偏移 */
    }
    return 0;

}
```

该函数非常重要，它是 FTL 层对文件系统的接口函数，用于往 NAND FLASH 里面写入数据，用户调用该函数时，无需关心坏块和磨损均衡问题，完全可以把 NAND FLASH 当成一个 SD 卡来访问。该函数输入参数：sectorno 用于指定扇区地址，扇区大小由 sectorsize 指定，一般我们设置 sectorsize=NAND_ECC_SECTOR_SIZE，方便进行 ECC 校验处理。

该函数根据 sectorno 和 sectorsize，首先计算出逻辑块地址（lbnno），然后将逻辑块地址转换为物理块地址（pbnno），然后再计算出块内的页地址（phypageno）和页内的偏移地址（pageoffset），然后计算该页内还可以连续写入的扇区数（如果可以连续写，则可以提高速度），然后判断要写入的区域，数据是否为 0XFF，如果全是 0XFF，则直接写入，写入完成对该物理块进行已被使用标记。如果不是全 0XFF，则需要利用 NAND 拷贝功能，将本页数据拷贝到另外一个 Block，并写入要写入的数据，这个操作由 ftl_copy_and_write_to_block 函数来完成。

最后，我们看 ftl_read_sectors 函数，该函数代码如下：

```
/**
 * @brief       读扇区(支持多扇区读)，FATFS 文件系统使用
 * @param       pbuffer     : 数据缓存区
 * @param       sectorno    : 起始扇区号
 * @param       sectorsize  : 扇区大小
 * @param       sectorcount : 要写入的扇区数量
 * @retval      0,成功;
 *              其他,失败
 */
uint8_t ftl_read_sectors(uint8_t *pbuffer, uint32_t sectorno,
                         uint16_t sectorsize, uint32_t sectorcount)
{
    uint8_t flag = 0;
    uint16_t rsecs;         /* 单次读取页数 */
    uint32_t i = 0;
```

```
    uint32_t lbnno;          /* 逻辑块号 */
    uint32_t pbnno;          /* 物理块号 */
    uint32_t phypageno;      /* 物理页号 */
    uint32_t pageoffset;     /* 页内偏移地址 */
    uint32_t blockoffset;    /* 块内偏移地址 */

    for (i = 0; i < sectorcount; i++)
    {
        /* 根据逻辑扇区号和扇区大小计算出逻辑块号 */
        lbnno = (sectorno + i) / (nand_dev.block_pagenum *
                                  (nand_dev.page_mainsize/sectorsize));
        pbnno = ftl_lbn_to_pbn(lbnno);    /* 将逻辑块转换为物理块 */

        if (pbnno >= nand_dev.block_totalnum)
        {
            return 1;                     /* 物理块号大于 NAND FLASH 的总块数,则失败. */
        }

        /* 计算块内偏移 */
        blockoffset=((sectorno+i)%(nand_dev.block_pagenum*
                    (nand_dev.page_mainsize / sectorsize))) * sectorsize;
        /* 计算出物理页号 */
        phypageno = pbnno * nand_dev.block_pagenum + blockoffset /
                                                     nand_dev.page_mainsize;
        /* 计算出页内偏移地址 */
        pageoffset = blockoffset%nand_dev.page_mainsize;
        rsecs = (nand_dev.page_mainsize - pageoffset) / sectorsize;
        /* 计算一次最多可以读取多少页 */

        if (rsecs > (sectorcount - i))
        {
            rsecs = sectorcount - i;  /* 最多不能超过 SectorCount-i */
        }
        flag=nand_readpage(phypageno, pageoffset, pbuffer, rsecs * sectorsize);
        /* 读取数据 */

        if (flag == NSTA_ECC1BITERR)  /* 对于 1bit ecc 错误,可能为坏块 */
        {
            flag=nand_readpage(phypageno,pageoffset,pbuffer,rsecs* sectorsize);
            /* 重读数据,再次确认 */

            if (flag == NSTA_ECC1BITERR)
            {
                /* 搬运数据 */
                ftl_copy_and_write_to_block(phypageno, pageoffset, pbuffer,
                                                        rsecs * sectorsize);
                flag = ftl_blockcompare(phypageno /
                                        nand_dev.block_pagenum, 0XFFFFFFFF);
                /* 全 1 检查,确认是否为坏块 */
                if (flag == 0)
                {
                    /* 全 0 检查,确认是否为坏块 */
                    flag = ftl_blockcompare(phypageno /
                                            nand_dev.block_pagenum, 0X00);
                    /* 检测完成后,擦除这个块 */
                    nand_eraseblock(phypageno / nand_dev.block_pagenum);
                }

                if (flag)                 /* 全 0/全 1 检查出错,肯定是坏块了. */
                {  /* 标记为坏块 */
                   ftl_badblock_mark(phypageno / nand_dev.block_pagenum);
                   ftl_create_lut(1);  /* 重建 LUT 表 */
                }
```

```
                flag = 0;
            }
        }
        if (flag == NSTA_ECC2BITERR)
        {
            flag = 0;      /* 2bit ecc 错误,不处理(可能是初次写入数据导致的) */
        }
        if (flag)
        {
            return 2;      /* 失败 */
        }
        pbuffer += sectorsize * rsecs;/* 数据缓冲区指针偏移 */
        i += rsecs - 1;
    }
    return 0;
}
```

该函数也是 FTL 层对文件系统的接口函数，用于读取 NAND FLASH 里面的数据，通用，用户在调用该函数时，无需关心坏块管理和磨损均衡问题，可以像访问 SD 卡一样，调用该函数，实现读取 NAND FLASH 的数据。该函数的实现原理，同前面介绍的 ftl_write_sectors 函数基本类似，不过该函数对读数时出现的 ECC 错误，进行了处理，对于读取数据时出现 1bit ECC 错误的 Block，进行两次读取（多次确认，以免误操作），如果两次读取都有 1bitECC 错误，那么该 Block 可能是坏块，当出现此错误后，我们先将该 Block 的数据，拷贝到另外一个 Block（备份现有数据），然后对该 Block 进行擦除和写 0，然后判断擦除/写 0 是否正常，如果正常，则说明这个块不是坏块，还可以继续使用。如果不正常，则说明该块确实是一个坏块，必须进行坏块标记，并重建 lut 表。如果读数时出现 2bit ecc 错误，这个不一定就是出错了，而有可能是读取还未写入过数据的 Block（未写入过数据，那么 ECC 值，肯定也是未写入过，如果进行 ECC 校验的话，必定出错），导致的 ECC 错误，对于此类错误，我们直接不予处理（忽略）就可以了。

ftl.c 里面的其他代码以及 ftl.h 里面的代码，这里就不做介绍了，请大家参考本例程源码。另外， nandtester.c 和 nandtester.h 这两个文件，主要用于 usmart 调试 nand.c 和 ftl.c 里面的相关函数，这里也不做介绍了，请大家参考本例程源码。

3. main.c 代码

最后，我们编写的 main 函数如下：

```
int main(void)
{
    uint8_t key, t = 0;
    uint16_t i;
    uint8_t *buf;
    uint8_t *backbuf;

    sys_cache_enable();                         /* 打开 L1-Cache */
    HAL_Init();                                 /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);         /* 设置时钟,400Mhz */
    delay_init(400);                            /* 延时初始化 */
    usart_init(115200);                         /* 串口初始化为 115200 */
    usmart_dev.init(200);                       /* 初始化 USMART */
    mpu_memory_protection();                    /* 保护相关存储区域 */
    led_init();                                 /* 初始化 LED */
    key_init();                                 /* 初始化按键 */
    sdram_init();                               /* 初始化 SDRAM */
    lcd_init();                                 /* 初始化 LCD */

    my_mem_init(SRAMIN);                        /* 初始化内部内存池 */
    my_mem_init(SRAMEX);                        /* 初始化外部内存池 */
    my_mem_init(SRAM12);                        /* 初始化 SRAM12 内存池 */
    my_mem_init(SRAM4);                         /* 初始化 SRAM4 内存池 */
```

```
    my_mem_init(SRAMDTCM);                          /* 初始化 DTCM 内存池 */
    my_mem_init(SRAMITCM);                          /* 初始化 ITCM 内存池 */

    lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30,  70, 200, 16, 16, "NAND TEST", RED);
    lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:Read Sector 2", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEY1:Write Sector 2", RED);
    lcd_show_string(30, 150, 200, 16, 16, "KEY2:Recover Sector 2", RED);

    while (ftl_init())    /* 检测 NAND FLASH,并初始化 FTL */
    {
        lcd_show_string(30, 170, 200, 16, 16, "NAND Error!", RED);
        delay_ms(500);
        lcd_show_string(30, 170, 200, 16, 16, "Please Check! ", RED);
        delay_ms(500);
        LED0_TOGGLE();  /* 红灯闪烁 */
    }

    backbuf =   mymalloc(SRAMIN, NAND_ECC_SECTOR_SIZE);  /* 申请一个扇区的缓存 */
    buf = mymalloc(SRAMIN, NAND_ECC_SECTOR_SIZE);         /* 申请一个扇区的缓存 */
    sprintf((char *)buf, "NAND Size:%dMB", (nand_dev.block_totalnum
           / 1024) * (nand_dev.page_mainsize / 1024) * nand_dev.block_pagenum);
    lcd_show_string(30, 190, 200, 16, 16, (char *)buf, BLUE); /* 显示 NAND 容量 */
    ftl_read_sectors(backbuf, 2, NAND_ECC_SECTOR_SIZE, 1);
    /* 预先读取扇区 0 到备份区域,防止乱写导致文件系统损坏. */

    while(1)
    {
        key = key_scan(0);
        switch (key)
        {
            case KEY0_PRES:                             /* KEY0 按下,读取 sector */
              key = ftl_read_sectors(buf, 2, NAND_ECC_SECTOR_SIZE, 1);
              /* 读取扇区 */
                if (key == 0)                           /* 读取成功 */
                {
                    lcd_show_string(30, 210, 200, 16, 16,
                                    "USART1 Sending Data...  ", BLUE);
                    printf("Sector 2 data is:\r\n");
                    for (i = 0; i < NAND_ECC_SECTOR_SIZE; i++)
                    {
                        printf("%x ",buf[i]);           /* 输出数据 */
                    }

                    printf("\r\ndata end.\r\n");
                    lcd_show_string(30, 210, 200, 16, 16,
                                    "USART1 Send Data Over!  ", BLUE);
                }
                break;

            case KEY1_PRES:                     /* KEY1 按下,写入 sector */
                 for (i = 0; i < NAND_ECC_SECTOR_SIZE; i++)
                 {
                     buf[i] = i + t;            /* 填充数据(随机的,根据 t 的值来确定) */
                 }

                lcd_show_string(30, 210, 210, 16, 16,
                                "Writing data to sector..", BLUE);
                /* 写入扇区 */
                key = ftl_write_sectors(buf, 2, NAND_ECC_SECTOR_SIZE, 1);

                if (key == 0)
                {
```

```
                    /* 写入成功 */
                    lcd_show_string(30, 210, 200, 16, 16,
                                    "Write data successed", BLUE);
                }
                else
                {
                    /* 写入失败 */
                    lcd_show_string(30, 210, 200, 16, 16,
                                    "Write data failed",BLUE);
                }
                break;

            case KEY2_PRES:
                /* KEY2 按下,恢复 sector 的数据 */
                lcd_show_string(30, 210, 210, 16, 16,
                                "Recovering data...", BLUE);
                /* 写入扇区 */
                key = ftl_write_sectors(backbuf, 2, NAND_ECC_SECTOR_SIZE, 1);

                if (key == 0)
                {
                    /* 恢复成功 */
                    lcd_show_string(30, 210, 200, 16, 16,
                                    "Recovering data OK",BLUE);
                }
                else
                {
                    /* 恢复失败 */
                    lcd_show_string(30, 210, 200, 16, 16,
                                    "Recovering data failed",BLUE);
                }
                break;
        }
        t++;
        delay_ms(10);
        if (t == 20)
        {
            LED0_TOGGLE();  /* 红灯闪烁 */
            t = 0;
        }
    }
}
```

此部分代码比较简单，我们先初始化相关外设，然后初始化 FTL，在 FTL 初始化成功以后，先对扇区 2 的数据进行备份，随后进入死循环，检测按键，可以通过 KEY0/KEY1/KEY2 按键对扇区 2 的数据进行读取、写入和还原操作。同样，LED0 闪烁，用于提示程序正在运行。

最后，我们将 nand_erasechip、nand_eraseblock、ftl_create_lut、ftl_format、test_writepage 和 test_readpage 等函数加入 USMART 控制，这样，我们就可以通过串口调试助手，测试 NAND FLASH 的各种操作了，方便大家测试。

```
/* 函数名列表初始化(用户自己添加)
 * 用户直接在这里输入要执行的函数名及其查找串
 */
struct _m_usmart_nametab usmart_nametab[] =
{
#if USMART_USE_WRFUNS == 1      /* 如果使能了读写操作 */
    (void *)read_addr, "uint32_t read_addr(uint32_t addr)",
    (void *)write_addr, "void write_addr(uint32_t addr,uint32_t val)",
#endif
    (void *)delay_ms, "void delay_ms(uint16_t nms)",
    (void *)delay_us, "void delay_us(uint32_t nus)",

#if USMART_USE_WRFUNS == 1    /* 如果使能了读写操作 */
    (void *)read_addr, "uint32_t read_addr(uint32_t addr)",
    (void *)write_addr, "void write_addr(uint32_t addr,uint32_t val)",
```

```
#endif
    (void *)nand_modeset, "uint8_t nand_modeset(uint8_t mode)",
    (void *)nand_eraseblock, "uint8_t nand_eraseblock(uint32_t blocknum)",
    (void *)nand_erasechip, "void nand_erasechip(void)",
    (void *)nand_copypage_withoutwrite,"uint8_t
   nand_copypage_withoutwrite(uint32_t source_pagenum,uint32_t dest_pagenum)",

    (void *)test_copypageandwrite, "uint8_t test_copypageandwrite(uint32_t
                  spnum,uint32_t dpnum,uint16_t colnum,uint16_t writebytes)",
    (void *)test_readpage,"uint8_t test_readpage(uint32_t pagenum,
                                  uint16_t colnum,uint16_t readbytes)",
    (void *)test_writepage,"uint8_t test_writepage(uint32_t pagenum,
                                 uint16_t colnum, uint16_t writebytes)",
    (void *)test_readspare, "uint8_t test_readspare(uint32_t pagenum,
                                  uint16_t colnum, uint16_t readbytes)",
    (void *)test_readallblockinfo,"void test_readallblockinfo(uint32_tsblock)",
    (void *)test_ftlwritesectors, "uint8_t test_ftlwritesectors
                           (uint32_t secx, uint16_t secsize, uint16_t seccnt)",
    (void *)test_ftlreadsectors, "uint8_t test_ftlreadsectors(
                            uint32_t secx, uint16_t secsize, uint16_t seccnt)",
    (void *)ftl_init, "uint8_t ftl_init(void)",
    (void *)ftl_check_badblock,"uint8_t ftl_check_badblock(uint32_t blocknum)",
    (void *)ftl_used_blockmark,"uint8_t ftl_used_blockmark(uint32_t blocknum)",
    (void *)ftl_find_unused_block, "uint32_t ftl_find_unused_block(
                                             uint32_t sblock, uint8_t flag)",
    (void *)ftl_find_same_plane_unused_block,"uint32_t
                          ftl_find_same_plane_unused_block(uint32_t blocknum)",
    (void *)ftl_lbn_to_pbn, "uint16_t ftl_lbn_to_pbn(uint32_t blocknum)",
    (void *)ftl_create_lut, "uint8_t ftl_create_lut(void)",
    (void *)ftl_format, "uint8_t ftl_format(void)",
};
```

54.4 下载验证

在代码编译成功之后，我们通过下载代码到开发板上，得到如图 54.4.1 所示：

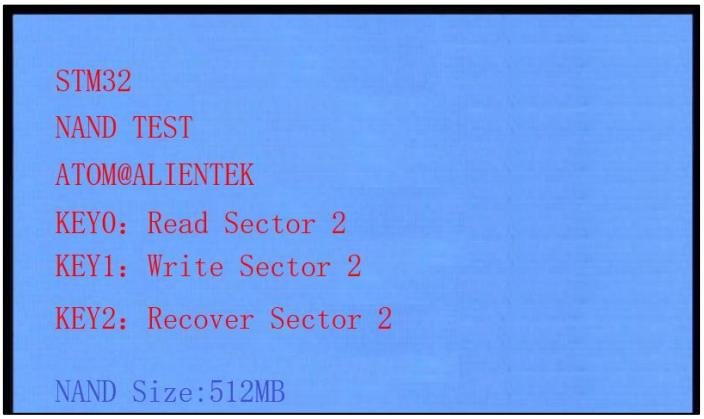


图 54.4.1 程序运行效果图

此时，我们可以按下 KEY0/KEY1/KEY2 等按键进行对应的测试。我们按 KEY0，可以读取扇区 2 里面的数据，通过串口调试助手查看，如图 54.4.2 所示：

图 54.4.2 串口查看扇区 2 里面的数据

另外，我们还可以用 USMART，调用相关函数，执行不同的操作，如图 54.4.3 所示：

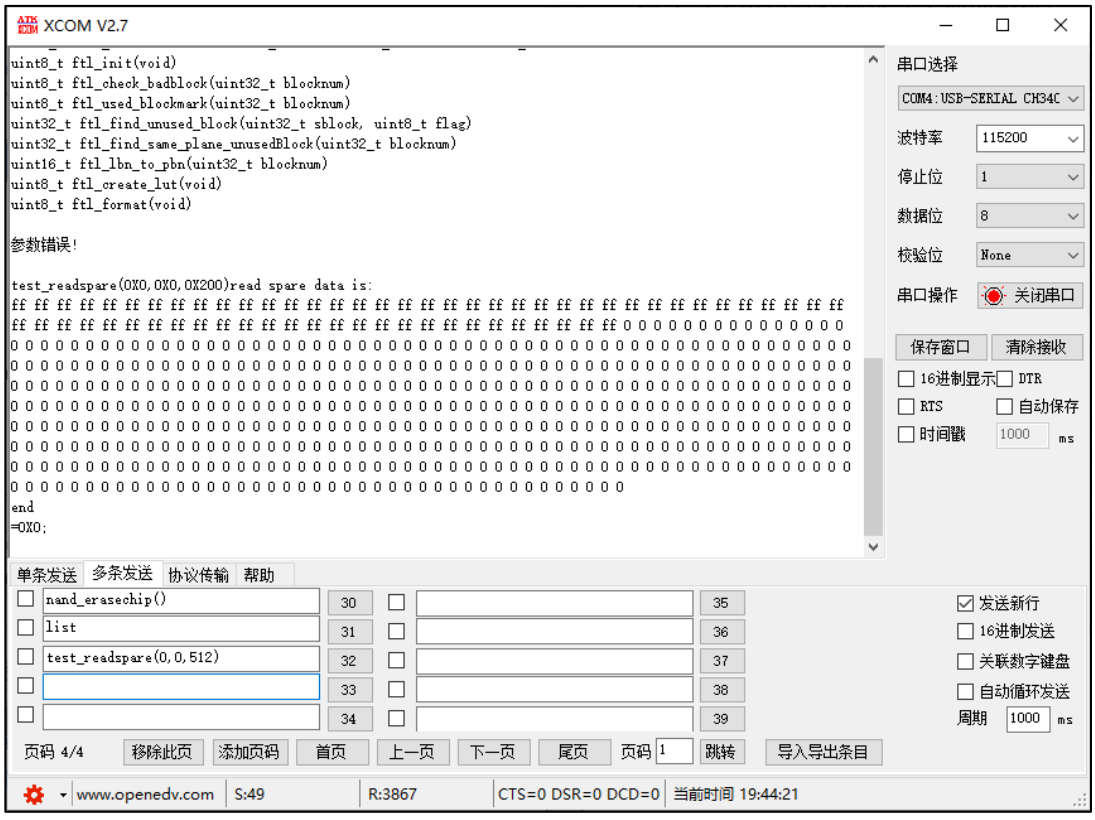


图 54.4.3 USMART 调用相关函数

第五十五章 FATFS 实验

上一章，我们学习了 SD 卡和 NAND FLASH 的使用，并实现了简单的读写扇区功能。在电脑上我们的资料常以文件的形式保存，通过文件名我们可以快速对自己的文件数据等进行分类。对于 SD 卡这种容量可以达到非常大的存储介质，按扇区去管理数据已经变得不方便，我们希望单片机也可以像电脑一样方便地用文件的形式去管理，在需要做数据采集的场合也会更加便利。

本章，我们将介绍 FATFS 这个软件工具，利用它在 STM32 上实现类似电脑上的文件管理功能，方便管理 SD 卡上的数据，并设计例程在 SD 卡上生成文件并对文件实现读写操作。

本章分为如下几个部分：

55.1 FATFS 简介
55.2 硬件设计
55.3 程序设计
55.4 下载验证

55.1 FATFS 简介

FATFS 是一个完全免费开源的 FAT/exFAT 文件系统模块，专门为小型的嵌入式系统而设计。它完全用标准 C 语言（ANSI C C89）编写，所以具有良好的硬件平台独立性，只需做简单的修改就可以移植到 8051、PIC、AVR、ARM、Z80、RX 等系列单片机上。它支持 FATl2、FATl6 和 FAT32，支持多个存储媒介；有独立的缓冲区，可以对多个文件进行读 / 写，并特别对 8 位单片机和 16 位单片机做了优化。FATFS 的特点有：

- Windows/dos 系统兼容的 FAT/exFAT 文件系统
- 独立于硬件平台，方便跨硬件平台移植
- 代码量少、效率高
- 多种配置选项
  - ✧ 支持多卷（物理驱动器或分区，最多 10 个卷）
  - ✧ 多个 ANSI/OEM 代码页包括 DBCS
  - ✧ 支持长文件名、ANSI/OEM 或 Unicode
  - ✧ 支持 RTOS
  - ✧ 支持多种扇区大小
  - ✧ 只读、最小化的 API 和 I/O 缓冲区等
  - ✧ 新版的 exFAT 文件系统，突破了原来 FAT32 对容量管理 32Gb 的上限，可支持更巨大的存储

FATFS 的这些特点，加上免费、开源的原则，使得 FATFS 应用非常广泛。FATFS 模块的层次结构如图 55.1.1 所示：

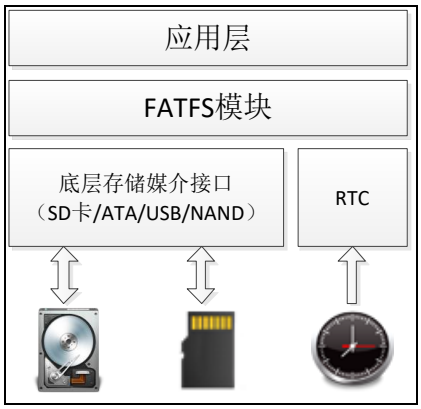


图 55.1.1 FATFS 层次结构图

最顶层是应用层，使用者无需理会 FATFS 的内部结构和复杂的 FAT 协议，只需要调用 FATFS 模块提供给用户的一系列应用接口函数，如 f_open，f_read，f_write 和 f_close 等，就可以像在 PC 上读 / 写文件那样简单。

中间层 FATFS 模块，实现了 FAT 文件读 / 写协议。FATFS 模块提供的是 ff.c 和 ff.h。除非有必要，使用者一般不用修改，使用时将头文件直接包含进去即可。

需要我们编写移植代码的是 FATFS 模块提供的底层接口，它包括存储媒介读 / 写接口（diskI/O）和供给文件创建修改时间的实时时钟。

FATFS 的源码及英文详述，大家可以在：http://elm-chan.org/fsw/ff/00index_e.html 这个网站下载到，教程目前使用的版本为 R0.14。本章我们就使用最新版本的 FATFS 作为介绍，下载最新版本的 FATFS 软件包，解压后可以得到两个文件夹：doc 和 src。doc 里面主要是对 FATFS 的介绍，而 src 里面才是我们需要的源码。source 文件夹详情表，如表 55.1.1.1 所示：

文件名	作用简述	备注
diskio.h	FATFS 和 diskI/O 模块公用的包含文件	与硬件平台无关
ff.c	FATFS 模块	
ff.h	FATFS 和应用模块公用的包含文件	
ffconf.h	FATFS 模块配置文件，宏定义对应的功能，代码中都有说明，具体的配置范围可以见官方配置说明 http://elm-chan.org/fsw/ff/doc/config.html	
ffsystem.c	根据是否有操作系统来修改这个文件	
ffunicode.c	可选，根据 ffconf.h 的配置，进行 Unicode 编码转换	
diskio.c	FATFS 和 diskI/O 模块接口层文件，需要根据硬件修改这部分的代码	硬件平台相关代码

表 55.1.1 FATFS 的源码文件介绍

FATFS 模块在移植的时候，我们一般只需要修改 2 个文件，即 ffconf.h 和 diskio.c。FATFS 模块的所有配置项都是存放在 ffconf.h 里面，我们可以通过配置里面的一些选项，来满足自己的需求。接下来我们介绍几个重要的配置选项。

1） FF_FS_TINY。这个选项在 R0.07 版本中开始出现，之前的版本都是以独立的 C 文件出现（FATFS 和 TinyFATFS），有了这个选项之后，两者整合在一起了，使用起来更方便。我们使用 FATFS，所以把这个选项定义为 0 即可。

2） FF _FS_READONLY。这个用来配置是不是只读，本章我们需要读写都用，所以这里设置为 0 即可。

3） FF _USE_STRFUNC。这个用来设置是否支持字符串类操作，比如 f_putc，f_puts 等，本章我们需要用到，故设置这里为 1。

4） FF _USE_MKFS。用来定时是否使能格式化，本章需要用到，所以设置这里为 1。

5） FF _USE_FASTSEEK。这个用来使能快速定位，我们设置为 1，使能快速定位。

6） FF _USE_LABEL。这个用来设置是否支持磁盘盘符（磁盘名字）读取与设置。我们设置为 1，使能，就可以通过相关函数读取或者设置磁盘的名字了。

7） FF _CODE_PAGE。这个用于设置语言类型，包括很多选项（见 FATFS 官网说明），我们这里设置为 936，即简体中文（GBK 码，同一个文件夹下的 ffunicode.c 根据这个宏选择对应的语言设置）。

8） FF_USE_LFN。该选项用于设置是否支持长文件名（还需要_CODE_PAGE 支持），取值范围为 0~3。0，表示不支持长文件名，1~3 是支持长文件名，但是存储地方不一样，我们选择使用 3，通过 ff_memalloc 函数来动态分配长文件名的存储区域。

9） FF_VOLUMES。用于设置 FATFS 支持的逻辑设备数目，我们设置为 2，即支持 2 个设备。

10）FF_MAX_SS。扇区缓冲的最大值，一般设置为 512。

11）FF_FS_EXFAT。新版本增加的功能，使用 exFAT 文件系统，用于支持超过 32Gb 的超大存储。它们使用的是 exFAT 文件系统，使用它时必须要根据设置 FF_USE_LFN 这个参数的值以决定 exFATs 系统使用的内存来自堆栈还是静态数组。

其他配置项，我们这里就不一一介绍了，FATFS 的说明文档里面有很详细的介绍，大家自己阅读 http://elm-chan.org/fsw/ff/doc/config.html 即可。下面我们来讲讲 FATFS 的移植，FATFS 的移植主要分为 3 步：

① 数据类型：在 integer.h 里面去定义好数据的类型。这里需要了解你用的编译器的数据类型，并根据编译器定义好数据类型。

② 配置：通过 ffconf.h 配置 FATFS 的相关功能，以满足你的需要。

③ 函数编写：打开 diskio.c，进行底层驱动编写，需要编写 5 个接口函数，如下图所示：

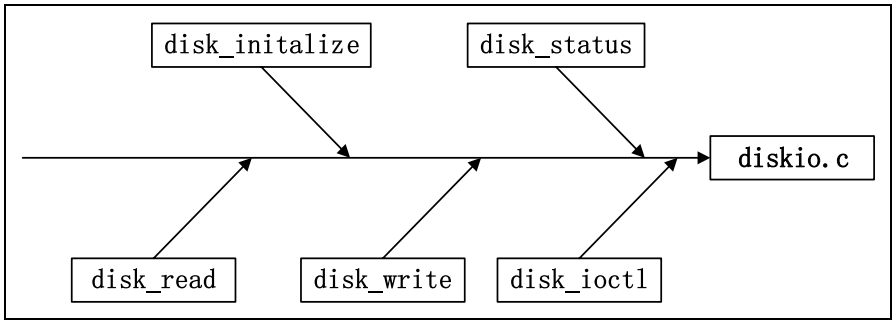


图 55.1.2 diskio 需要实现的函数

通过以下三步，我们即可完成对 FATFS 的移植。

第一步，我们使用的是 MDK5.34 编译器，数据类型和 integer.h 里面定义的一致，所以此步，我们不需要做任何改动。

第二步，关于 ffconf.h 里面的相关配置，我们在前面已经有介绍（之前介绍的 11 个配置），我们将对应配置修改为我们介绍时候的值即可，其他的配置用默认配置。

第三步，因为 FATFS 模块完全与磁盘 I/O 层分开，因此需要下面的函数来实现底层物理磁盘的读写与获取当前时间。底层磁盘 I/O 模块并不是 FATFS 的一部分，并且必须由用户提供。这些函数一般有 5 个，在 diskio.c 里面。

首先是 disk_initialize 函数，该函数介绍如表 55.1.2 所示：

函数名称	disk_initialize
函数原型	DSTATUS disk_initialize(BYTE Drive)
功能描述	初始化磁盘驱动器
函数参数	Drive：指定要初始化的逻辑驱动器号，即盘符，应当取值 0~9
返回值	函数返回一个磁盘状态作为结果，对于磁盘状态的细节信息，请参考 disk_status 函数
所在文件	ff.c
实例	disk_initialize(0); /* 初始化驱动器 0 */
注意事项	disk_initialize 函数初始化一个逻辑驱动器为读/写做准备，函数成功时，返回值的 STA_NOINIT 标志被清零； 应用程序不应调用此函数，否则卷上的 FAT 结构可能会损坏； 如果需要重新初始化文件系统，可使用 f_mount 函数； 在 FatFs 模块上卷注册处理时调用该函数可控制设备的改变； 此函数在 FatFs 挂在卷时调用，应用程序不应该在 FatFs 活动时使用此函数

表 55.1.2 disk_initialize 函数介绍

第二个函数是 disk_status 函数，该函数介绍如表 55.1.3 所示：

函数名称	disk_status
函数原型	DRESULT disk_status (BYTE Drive)
功能描述	返回当前磁盘驱动器的状态
函数参数	Drive：指定要确认的逻辑驱动器号，即盘符，应当取值 0~9
返回值	磁盘状态返回下列标志的组合，FatFs 只使用 STA_NOINIT 和 STA_PROTECTED STA_NOINIT：表明磁盘驱动未初始化，下面列出了产生该标志置位或清零的原因：

	置位：系统复位，磁盘被移除和磁盘初始化函数失败 清零：磁盘初始化函数成功 STA_NODISK：表明驱动器中没有设备，安装磁盘驱动器后总为 0 STA_PROTECTED：表明设备被写保护，不支持写保护的设备总为 0，当 STA_NODISK 置位时非法
所在文件	ff.c
实例	disk_status(0); /* 获取驱动器 0 的状态 */

表 55.1.3 disk_status 函数介绍

第三个函数是 disk_read 函数，该函数介绍如表 55.1.4 所示：

函数名称	disk_read
函数原型	DRESULT disk_read (BYTE Drive, BYTE* Buffer, DWORD SectorNumber, BYTE SectorCount)
功能描述	从磁盘驱动器上读取扇区
函数参数	Drive：指定逻辑驱动器号，即盘符，应当取值 0~9 Buffer：指向存储读取数据字节数组的指针，需要为所读取字节数的大小，扇区统计的扇区大小是需要的 （注：FaFts 指定的内存地址并不总是字对齐的，如果硬件不支持不对齐的数据传输，函数里需要进行处理） SectorNumber：指定起始扇区的逻辑块（LBA）上的地址 SectorCount：指定要读取的扇区数，取值 1~128
返回值	RES_OK(0)：函数成功 RES_ERROR：读操作期间产生了任何错误且不能恢复它 RES_PARERR：非法参数 RES_NOTRDY：磁盘驱动器没有初始化
所在文件	ff.c

表 55.1.4 disk_read 函数介绍

第四个函数是 disk_write 函数，该函数介绍如表 55.1.5 所示：

函数名称	disk_write
函数原型	DRESULT disk_write (BYTE Drive, const BYTE* Buffer, DWORD SectorNumber, BYTE SectorCount)
功能描述	向磁盘写入一个或多个扇区
函数参数	Drive：指定逻辑驱动器号，即盘符，应当取值 0~9 Buffer：指向要写入字节数组的指针 （注：FaFts 指定的内存地址并不总是字对齐的，如果硬件不支持不对齐的数据传输，函数里需要进行处理） SectorNumber：指定起始扇区的逻辑块（LBA）上的地址 SectorCount：指定要写入的扇区数，取值 1~128
返回值	RES_OK(0)：函数成功 RES_ERROR：写操作期间产生了任何错误且不能恢复它 RES_WRPER：媒体被写保护 RES_PARERR：非法参数 RES_NOTRDY：磁盘驱动器没有初始化
所在文件	ff.c
注意事项	只读配置中不需要此函数

表 55.1.5 disk_write 函数介绍

第五个函数是 disk_ioctl 函数，该函数介绍如表 55.1.6 所示：

函数名称	disk_ioctl
函数原型	DRESULT disk_ioctl (BYTE Drive, BYTE Command, void* Buffer)
功能描述	控制设备指定特性和除了读/写外的杂项功能

函数参数	Drive：指定逻辑驱动器号，即盘符，应当取值 0~9 Command：指定命令代码 Buffer：指向参数缓冲区的指针，取决于命令代码，不使用时，指定一个 NULL 指针
返回值	RES_OK(0)：函数成功 RES_ERROR：写操作期间产生了任何错误且不能恢复它 RES_PARERR：非法参数 RES_NOTRDY：磁盘驱动器没有初始化
所在文件	ff.c
注意事项	CTRL_SYNC：确保磁盘驱动器已经完成了写处理，当磁盘 I/O 有一个写回缓存，立即刷新原扇区，只读配置下不适用此命令 GET_SECTOR_SIZE：返回磁盘的扇区大小，只用于 f_mkfs() GET_SECTOR_COUNT：返回可利用的扇区数，_MAX_SS ≥ 1024 时可用 GET_BLOCK_SIZE：获得擦除块大小，只用于 f_mkfs() CTRL_ERASE_SECTOR：强制擦除一块的扇区，_USE_ERASE ＞ 0 时可用

表 55.1.6 disk_ioctl 函数介绍

以上五个函数，我们将在软件设计部分一一实现。通过以上 3 个步骤，我们就完成了对 FATFS 的移植，就可以在我们的代码里面使用 FATFS 了。

FATFS 提供了很多 API 函数，这些函数 FATFS 的自带介绍文件里面都有详细的介绍(包括参考代码)，我们这里就不多说了。这里需要注意的是，在使用 FATFS 的时候，必须先通过 f_mount 函数注册一个工作区，才能开始后续 API 的使用，关于 FATFS 的介绍，我们就介绍到这里。大家可以通过 FATFS 自带的介绍文件进一步了解和熟悉 FATFS 的使用。

55.2 硬件设计

1. 例程功能

本章实验功能简介：开机的时候先初始化 SD 卡，初始化成功之后，注册三个工作区：一个给 SD 卡用，一个给 SPI FLASH 用、一个给 NAND FLASH 用，之所以把 SPI FLASH 也当成磁盘来用，一方面是为了演示大容量的 SPI Flash 也可以用 FATFS 管理，说明 FATFS 的灵活性；另一方面可以展示 FATFS 方式比原来直接按地址管理数据便利性，使板载 SPI Flash 的使用更具灵活性。挂载成功后获取 SD 卡的容量和剩余空间，并显示在 LCD 模块上，最后定义 USMART 输入指令进行各项测试。本实验通过 LED0 指示程序运行状态。

2. 硬件资源

1）LED 灯
LED ：LED0 – PB1
2）正点原子 2.8/3.5/4.3/7 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
3）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）SD 卡，通过 SDMMC1(SDMMC_D0~D4(PC8~PC11)，SDMMC_SCK(PC12)，SDMMC_CMD(PD2))连接
5）norflash(QSPI FLASH 芯片,连接在 QSPI 上)

这几个外设原理图我们在之前的章节已经介绍过了，这里就不重复介绍了，不清楚的话可以查看本文之前章节的描述或对光盘资料提供的开发板原理图。

55.3 程序设计

FATFS 的驱动为一个硬件独立的组件，因此我们把 FATFS 的移植代码放到“Middlewares”文件夹下。

本章，我们在“SD 卡实验”的基础上进行拓展。在“Middlewares”下新建一个 FATFS 的文件夹，然后将 FATFS R0.14 程序包解压到该文件夹下。同时，我们在 FATFS 文件夹里面新

建一个 exfuns 的文件夹，用于存放我们针对 FATFS 做的一些扩展代码（这个我们后面再讲）。操作结果如图 55.3.1 所示：

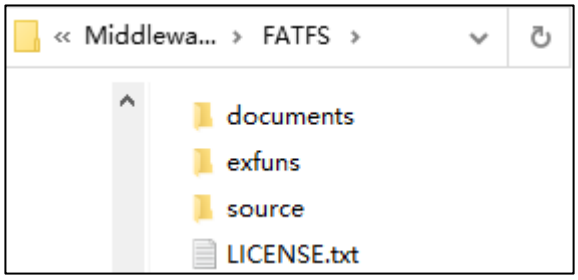


图 55.3.1 FATFS 文件夹子目录

55.3.1 程序流程图

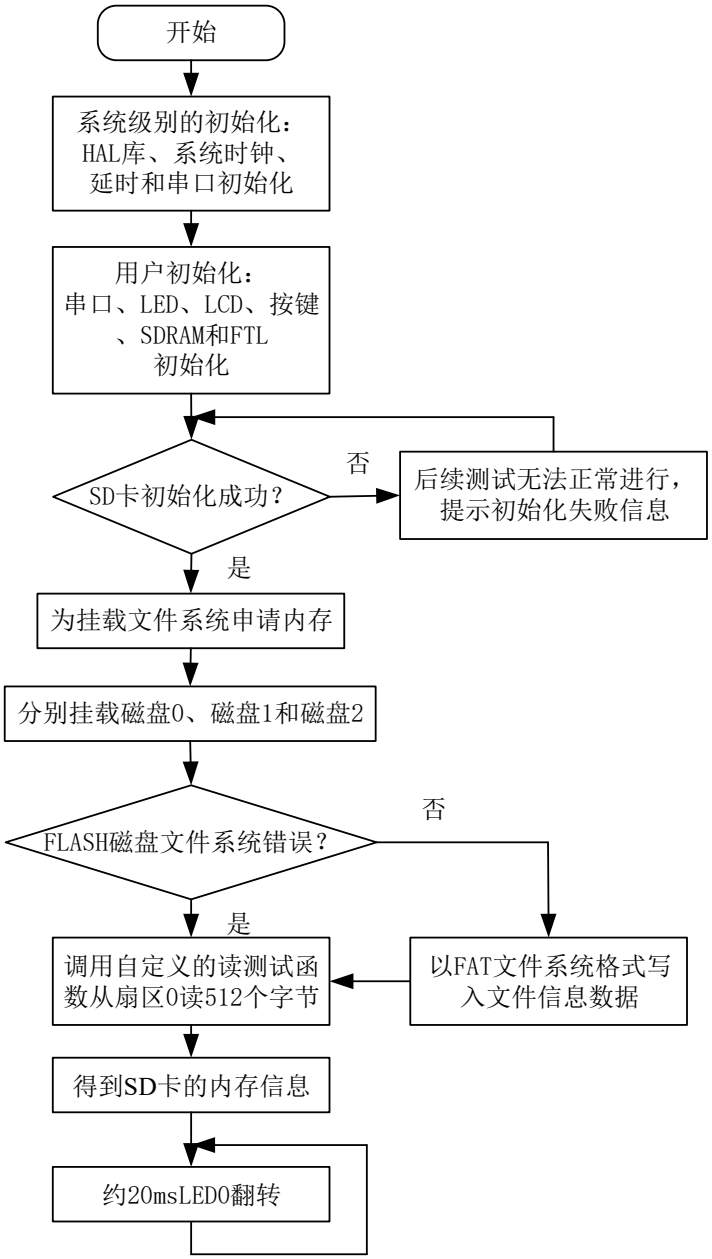


图 55.3.1.1 SD 读写实验程序流程图

55.3.2 程序解析

1. FATFS 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。

diskio.c/.h 为我们提供了规定好的底层驱动接口的返回值。这个函数需要使用到我们的硬件接口，所以需要把使用到的硬件驱动的头文件包进来。

```
#include "./MALLOC/malloc.h"
#include "./BSP/NAND/nand.h"
#include "./SYSTEM/usart/usart.h"
#include "./FATFS/source/diskio.h"
#include "./BSP/SDMMC/sdmmc_sdcard.h"
#include "./BSP/NORFLASH/norflash.h"
```

按照 55.1 的描述，接下来我们来对这几个接口进行补充实现。本章，我们用 FATFS 管理了 3 个磁盘：SD 卡、SPI FLASH 和 NAND FLASH，我们设置 SD_CARD 为 0，EX_FLASH 位为 1，对应到 disk_read/disk_write 函数里面。SD 卡比较好说，但是 SPI FLASH，因为其扇区是 4K 字节大小，我们为了方便设计，强制将其扇区定义为 512 字节，这样带来的好处就是设计使用相对简单，坏处就是擦除次数大增，所以不要随便往 SPI FLASH 里面写数据，非必要最好别写，如果频繁写的话，很容易将 SPI FLASH 写坏。

```
#define SD_CARD      0        /* SD 卡,卷标为 0 */
#define EX_FLASH     1        /* 外部 spi flash,卷标为 1 */
#define EX_NAND      2        /* 外部 NAND FLASH 卷标为 2 */

/**
 * 对于 25Q256 FLASH 芯片，我们规定前 25M 给 FATFS 使用，25M 以后
 * 用于存放字库，字库占用 6.01M，剩余部分，给客户自己用.
 */
#define SPI_FLASH_SECTOR_SIZE    512
#define SPI_FLASH_SECTOR_COUNT   25*1024*2 /* 25Q256，前 25M 字节给 FATFS 占用 */
#define SPI_FLASH_BLOCK_SIZE     8        /* 每个 BLOCK 有 8 个扇区 */
#define SPI_FLASH_FATFS_BASE     0        /* FATFS 在外部 FLASH 的起始地址 从 0 开始 */

/**
 * NAND FLASH 全部归 FATFS 管理
 */
uint32_t NAND_FLASH_SECTOR_COUNT;
uint8_t NAND_FLASH_BLOCK_SIZE;
```

另外，diskio.c 里面的函数，直接决定了磁盘编号（盘符/卷标）所对应的具体设备，比如，以上代码中，我们就通过 switch 来判断，到底要操作 SD 卡，还是 SPI FLASH，还是 NAND FLASH，然后，分别执行对应设备的相关操作。以此实现磁盘编号和磁盘的关联。

1. disk_initialize 函数

要使用 FATFS 管理，首先要对磁盘进行初始化，所以先看磁盘的初始化函数，其声明如下：

```
DSTATUS disk_initialize ( BYTE pdrv)
```

- **函数描述：**

  初始化指定编号的磁盘，磁盘所指定的存储区。使用每个磁盘前进行初始化，那在代码中直接根据编号调用硬件的初始化接口即可，这样也能保证代码的扩展性，硬件的顺序可以根据自己的喜好定义。

- **函数形参：**

  形参 1 是 FATFS 管理的磁盘编号 pdrv：磁盘编号 0~9，我们配置 FF_VOLUMES 为 3 来支持三个磁盘，因此可选值为 0、1 和 2。

  代码实现如下：

```
/**
 * @brief       初始化磁盘
 * @param       pdrv : 磁盘编号 0~9
 * @retval      无
```

```
 */
DSTATUS disk_initialize ( BYTE pdrv)
{
    uint8_t res = 0;
    switch (pdrv)
    {
        case SD_CARD:           /* SD 卡 */
            res = sd_init();    /* SD 卡初始化 */
            break;
        case EX_FLASH:          /* 外部 flash */
            norflash_init();
            break;
        case EX_NAND:           /* NAND flash */
            res=nand_init();
            break;
        default:
            res = 1;
    }

    if (res)
    {
        return  STA_NOINIT;
    }
    else
    {
        return 0; /* 初始化成功*/
    }
}
```

- **函数返回值：**

DSTATUS 枚举类型的值，FATFS 规定了自己的返回值来管理各接口函数的操作结果，方便后续函数的操作和判断,我们看看它的定义:

```
/* Status of Disk Functions */
typedef BYTE    DSTATUS;
/* Disk Status Bits (DSTATUS) */
#define STA_NOINIT      0x01        /* Drive not initialized */
#define STA_NODISK      0x02        /* No medium in the drive */
#define STA_PROTECT     0x04        /* Write protected */
```

定义时也写出了各个参数的含义，根据 ff.c 中的调用实例可知操作返回 0 才是正常的状态，其它情况发生的话就需要结合硬件进行分析了。

2. disk_status 函数

便于我们知道当前磁盘驱动器的状态，所以 FATFS 也有 disk_status 函数提供给我们使用，其声明如下：

```
DSTATUS disk_status (BYTE pdrv)
```

- **函数描述：**

返回当前磁盘驱动器的状态。

- **函数形参：**

形参 1 是 FATFS 管理的磁盘编号 pdrv：磁盘编号 0~9，我们配置 FF_VOLUMES 为 2 来支持两个磁盘，因此可选值为 0 和 1。

为了简单测试，所以我们这里没有加入硬件状态的判断，代码也不贴出来了。

- **函数返回值：**

直接返回 RES_OK。

3. disk_read 函数

disk_read 实现直接从硬件接口读取数据，这个为接口 FATFS 的其它读操作接口函数调用，其声明如下：

```
DRESULT disk_read (BYTE pdrv, BYTE *buff, DWORD sector, UINT count)
```

- **函数描述：**

从磁盘驱动器上读取扇区数据。

- **函数形参：**

形参 1：是 FATFS 管理的磁盘编号 pdrv：磁盘编号 0~9，我们配置 FF_VOLUMES 为 3 来支持三个磁盘，因此可选值为 0、1 和 2。

形参 2：buff 指向要保存数据的内存区域指针，为字节类型。

形参 3：sector 为实际物理操作时要访问的扇区地址。

形参 4：count 为本次要读取的数据量，最长为 unsigned int，读到的数量为字节数。

代码实现如下，同样要根据我们定义的设备标号，在 swich-case 中添加对应硬件的驱动：

```
/**
 * @brief       读扇区
 * @param       pdrv    : 磁盘编号 0~9
 * @param       buff    : 数据接收缓冲首地址
 * @param       sector  : 扇区地址
 * @param       count   : 需要读取的扇区数
 * @retval      无
 */
DRESULT disk_read (
    BYTE pdrv,      /* Physical drive nmuber to identify the drive */
    BYTE *buff,     /* Data buffer to store read data */
    DWORD sector,   /* Sector address in LBA */
    UINT count      /* Number of sectors to read */
)
{
    uint8_t res = 0;
    if (!count)return RES_PARERR;    /* count 不能等于 0，否则返回参数错误 */
    switch (pdrv)
    {
        case SD_CARD:      /* SD 卡 */
            res = sd_read_disk(buff, sector, count);
            while (res)    /* 读出错 */
            {
                sd_init(); /* 重新初始化 SD 卡 */
                res = sd_read_disk(buff, sector, count);
            }

            break;
        case EX_FLASH:/* 外部 flash */
            for (; count > 0; count--)
            {
                norflash_read(buff, SPI_FLASH_FATFS_BASE + sector *
                              SPI_FLASH_SECTOR_SIZE, SPI_FLASH_SECTOR_SIZE);
                sector++;
                buff += SPI_FLASH_SECTOR_SIZE;
            }
            res = 0;
            break;
         case EX_NAND:/* 外部 NAND */
            res =ftl_read_sectors(buff,sector,512,count);
            break;
        default:
            res = 1;
    }

    /* 处理返回值，将返回值转成 ff.c 的返回值 */
    if (res == 0x00)
    {
        return RES_OK;
    }
    else
    {
        return RES_ERROR;
    }
}
```

- **函数返回值：**

DRESULT 为枚举类型，diskio.h 中有其定义，根据返回值的含义确认操作结果即可。在这里也把该结果展示出来，如下所示：

```
/* Results of Disk Functions */
typedef enum
{
    RES_OK = 0,         /* 0: 操作成功 */
    RES_ERROR,          /* 1: 读/写错误 */
    RES_WRPRT,          /* 2: 写保护状态 */
    RES_NOTRDY,         /* 3: 设备忙 */
    RES_PARERR          /* 4: 其它情形 */
} DRESULT;
```

根据返回值的含义确认操作结果即可。

4. disk_write 函数

disk_write 实现直接从硬件接口读取数据，这个接口为 FATFS 的其它写操作接口函数调用，其声明如下：

```
DRESULT disk_write ( BYTE pdrv, const BYTE *buff, DWORD sector, UINT count)
```

- **函数描述：**

  向磁盘驱动器上写入扇区数据。

- **函数形参：**

  形参 1 是 FATFS 管理的磁盘编号 pdrv：磁盘编号 0~9，我们配置 FF_VOLUMES 为 3 来支持两个磁盘，因此可选值为 0、1 和 2。

  形参 2：buff 指向要保存数据的内存区域指针，为字节类型。

  形参 3：sector 为实际物理操作时要访问的扇区地址。

  形参 4：count 为本次要读取的数据量，最长为 unsigned int，读到的数量为字节数。

  代码实现如下，同样要根据我们定义的设备标号，在 swich-case 中添加对应硬件的驱动：

```
/**
 * @brief       写扇区
 * @param       pdrv    : 磁盘编号 0~9
 * @param       buff    : 发送数据缓存区首地址
 * @param       sector  : 扇区地址
 * @param       count   : 需要写入的扇区数
 * @retval      无
 */
DRESULT disk_write (
    BYTE pdrv,              /* Physical drive nmuber to identify the drive */
    const BYTE *buff,   /* Data to be written */
    DWORD sector,           /* Sector address in LBA */
    UINT count              /* Number of sectors to write */
)
{
    uint8_t res = 0;
    if (!count)return RES_PARERR;   /* count 不能等于 0，否则返回参数错误 */
    switch (pdrv)
    {
        case SD_CARD:       /* SD 卡 */
            res = sd_write_disk((uint8_t *)buff, sector, count);
            while (res)     /* 写出错 */
            {
                sd_init();  /* 重新初始化 SD 卡 */
                res = sd_write_disk((uint8_t *)buff, sector, count);
                //printf("sd wr error:%d\r\n", res);
            }
            break;

        case EX_FLASH:      /* 外部 flash */
            for (; count > 0; count--)
            {
                norflash_write((uint8_t *)buff, SPI_FLASH_FATFS_BASE + sector *
                               SPI_FLASH_SECTOR_SIZE, SPI_FLASH_SECTOR_SIZE);
```

```
                sector++;
                buff += SPI_FLASH_SECTOR_SIZE;
            }
            res = 0;
            break;
        case EX_NAND:       /* 外部 NAND */
            res = ftl_write_sectors((uint8_t*)buff,sector,512,count);
            break;
        default:
            res = 1;
    }
    /* 处理返回值，将返回值转成 ff.c 的返回值 */
    if (res == 0x00)
    {
        return RES_OK;
    }
    else
    {
        return RES_ERROR;
    }
}
```

- **函数返回值：**

DRESULT 为枚举类型，diskio.h 中有其定义，编写读函数时已经介绍了，注意要把返回值转成这个枚举类型的参数。

5. disk_ioctl 函数

disk_ioctl 实现一些控制命令，这个接口为 FATFS 提供一些硬件操作信息，其声明如下：

```
DRESULT disk_ioctl (BYTE pdrv, BYTE cmd, void *buff)
```

- **函数描述：**

初始化指定编号的磁盘，磁盘所指定的存储区。

- **函数形参：**

形参 1：是 FATFS 管理的磁盘编号 pdrv :磁盘编号 0~9，我们配置 FF_VOLUMES 为 3 来支持三个磁盘，因此可选值为 0、1 和 2。

形参 2：cmd 是 FATFS 定义好的一些宏，用于访问硬盘设备的一些状态。我们实现几个简单的操作接口，用于获取磁盘容量这些基础信息（diskio.h 中已经定义好了），为方便，我们先只实现几个标准的应用接口，关于 iSDIO/MMC 的一些扩展命令我们再根据需要进行支持。

```
/* Command code for disk_ioctrl fucntion */
/* Generic command (Used by FatFs) */
#define CTRL_SYNC           0   /* 完成挂起的写入过程(当 FF_FS_READONLY == 0) */
#define GET_SECTOR_COUNT    1   /* 获取磁盘扇区数(当 FF_USE_MKFS == 1) */
#define GET_SECTOR_SIZE     2   /* 获取磁盘存储空间大小 (当 FF_MAX_SS != FF_MIN_SS) */
#define GET_BLOCK_SIZE      3   /* 每个扇区块的大小 (当 FF_USE_MKFS == 1) */
#define CTRL_TRIM           4   /* 告知设备哪些块的数据不再使用（当 FF_USE_TRIM==1） */
```

下面是从 http://elm-chan.org/fsw/ff/doc/dioctl.html 得到的参数实现效果，我们也可以参考原有的 disk_ioctl 的实现来理解这几个参数。

命令	说明
CTRL_SYNC	确保设备已完成挂起的写入过程。如果磁盘 I/O 层或存储设备具有回写缓存，则必须立即将缓存数据提交到介质。如果对介质的每个写入操作都正常完成，则此命令无任何动作。
GET_SECTOR_COUNT	返回对应标号的硬盘的可用扇区数。此命令由 f_mkfs\f_fdisk 函数用于确定要创建的卷/分区的大小。使用时需要设置 FF_USE_MKFS 为 1。
GET_SECTOR_SIZE	将用于读/写函数的扇区大小检索到 buff 指向的 WORD 变量中。有效扇区大小为 512、1024、2048 和 4096。只有在 FF_MAX_SS>FF_MIN_SS 执行此命令时。当

	FF_MAX_SS=FF_MIN_SS 时，将永远不会使用此命令，并且读/写函数必须仅在 FF_MAX_SSbytes/扇区中工作。
GET_BLOCK_SIZE	将扇区单位中闪存介质的块大小以 DWORD 指针存到 buff 中。允许的值为 1 到 32768。如果擦除块大小未知或非闪存介质，则返回 1。此命令仅由 f_mkfs 函数使用，并尝试对齐擦除块边界上的数据区域。使用时需要设置 FF_USE_MKFS 为 1。

形参 3： buff 为 void 形指针，根据命令的格式和需要，我们把对应的值转成对应的形式传给它。参考原有的 disk_ioctl 的实现，我们的函数实现如下。

```
/**
 * @brief       获取其他控制参数
 * @param       pdrv    : 磁盘编号 0~9
 * @param       ctrl    : 控制代码
 * @param       buff    : 发送/接收缓冲区指针
 * @retval      无
 */
DRESULT disk_ioctl (
    BYTE pdrv,      /* Physical drive nmuber (0..) */
    BYTE cmd,       /* Control code */
    void *buff      /* Buffer to send/receive control data */
)
{
    DRESULT res;
    if (pdrv == SD_CARD)     /* SD 卡 */
    {
        switch (cmd)
        {
            case CTRL_SYNC:
                res = RES_OK;
                break;

            case GET_SECTOR_SIZE:
                *(DWORD *)buff = 512;
                res = RES_OK;
                break;

            case GET_BLOCK_SIZE:
                *(WORD *)buff = g_sd_card_info_handle.LogBlockSize;
                res = RES_OK;
                break;

            case GET_SECTOR_COUNT:
                *(DWORD *)buff = g_sd_card_info_handle.LogBlockNbr;
                res = RES_OK;
                break;

            default:
                res = RES_PARERR;
                break;
        }
    }
    else if (pdrv == EX_FLASH)  /* 外部 FLASH */
    {
        switch (cmd)
        {
            case CTRL_SYNC:
                res = RES_OK;
                break;

            case GET_SECTOR_SIZE:
                *(WORD *)buff = SPI_FLASH_SECTOR_SIZE;
                res = RES_OK;
```

```
                break;

            case GET_BLOCK_SIZE:
                *(WORD *)buff = SPI_FLASH_BLOCK_SIZE;
                res = RES_OK;
                break;

            case GET_SECTOR_COUNT:
                *(DWORD *)buff = SPI_FLASH_SECTOR_COUNT;
                res = RES_OK;
                break;

            default:
                res = RES_PARERR;
                break;
        }
    }
    else if (pdrv == EX_NAND)               /* 外部 NAND FLASH */
    {
        switch (cmd)
        {
            case CTRL_SYNC:
                res = RES_OK;
                break;

            case GET_SECTOR_SIZE:
                *(WORD *)buff = 512;        /* NAND FLASH 扇区强制为 512 字节大小 */
                res = RES_OK;
                break;

            case GET_BLOCK_SIZE:
                *(WORD *)buff = nand_dev.page_mainsize / 512;
                /* block 大小，定义成一个 page 的大小 */
                res = RES_OK;
                break;

            case GET_SECTOR_COUNT:
                *(DWORD *)buff = nand_dev.valid_blocknum *
                nand_dev.block_pagenum * nand_dev.page_mainsize / 512;
                /* NAND FLASH 的总扇区大小 */
                res = RES_OK;
                break;

            default:
                res = RES_PARERR;
                break;
        }
    }
    else
    {
        res = RES_ERROR;     /* 其他的不支持 */
    }
    return res;
}
```

- **函数返回值：**

DRESULT 为枚举类型，diskio.h 中有其定义，编写读函数时已经介绍了，注意要把返回值转成这个枚举类型的参数。

以上实现了我们 55.1 节提到的 5 个函数，ff.c 中需要实现 get_fattime (void)，同时因为在 ffconf.h 里面设置对长文件名的支持为方法 3，所以必须在 ffsystem.c 中实现 get_fattime、ff_memalloc 和 ff_memfree 这三个函数。这部分比较简单，直接参考我们修改后的 ffsystem.c 的源码。

至此，我们已经可以直接使用 FATFS 下的 ff.c 下的 f_mount 的接口挂载磁盘，然后使用类似标准 C 的文件操作函数，就可以实现文件操作。但 f_mount 还需要一些文件操作的内存，为

了方便操作，我们在 FATFS 文件夹下还新建了一个 exfuns 的文件夹，该文件夹用于保存一些针对 FATFS 的扩展代码，如刚才提到的 FATFS 相关函数的内存申请方法等。

本章，我们编写了 4 个文件，分别是：exfuns.c、exfuns.h、fattester.c 和 fattester.h。其中 exfuns.c 主要定义了一些全局变量，方便 FATFS 的使用，同时实现了磁盘容量获取等函数。而 fattester.c 文件则主要是为了测试 FATFS 用，因为 FATFS 的很多函数无法直接通过 USMART 调用，所以我们在 fattester.c 里面对这些函数进行了一次再封装，使得可以通过 USMART 调用。

这几个文件的代码，大家可以直接使用本例程源码，我们将 exfuns.c/.h 和 fattester.c/.h 加入 FATFS 组下的 exfuns 文件下，直接使用即可。

6. exfuns_init 函数

我们在使用文件操作前，需要用 f_mount 函数挂载磁盘，我们在挂载 SD 卡前需要一些文件系统的内存，为了方便管理，我们定义一个全局的 fs[FF_VOLUMES]指针，定成数组是我们要管理多个磁盘，而 f_mount 也需要一个 FATFS 类型的指针，定义如下：

```
/* 逻辑磁盘工作区(在调用任何 FATFS 相关函数之前,必须先给 fs 申请内存) */
FATFS *fs[FF_VOLUMES];
```

接下来只要用内存管理部分的知识来实现对 fs 指针的内存申请即可。

```
/**
 * @brief       为 exfuns 申请内存
 * @param       无
 * @retval      0, 成功; 1, 失败.
 */
uint8_t exfuns_init(void)
{
    uint8_t i;
    uint8_t res = 0;

    for (i = 0; i < FF_VOLUMES; i++)
    {
       fs[i] = (FATFS *)mymalloc(SRAMIN, sizeof(FATFS));/*为磁盘 i 工作区申请内存 */
       if (!fs[i])break;
    }

#if USE_FATTESTER == 1  /* 如果使能了文件系统测试 */
    res = mf_init();      /* 初始化文件系统测试(申请内存) */
#endif

    if (i == FF_VOLUMES && res == 0)
    {
        return 0;   /* 申请有一个失败,即失败. */
    }
    else
    {
        return 1;
    }
}
```

其它的函数我们暂时没有使用到，这里先不介绍了，大家使用时查阅源码即可。

2. mian.c 代码

最后，我们编写的 main 函数如下：

```
int main(void)
{
    uint32_t total, free;
    uint8_t t = 0;
    uint8_t res = 0;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
```

```
    usart_init(115200);                     /* 串口初始化为 115200 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    my_mem_init(SRAMIN);                    /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                    /* 初始化外部内存池(SDRAM) */
    my_mem_init(SRAM12);                    /* 初始化 SRAM12 内存池(SRAM1+SRAM2) */
    my_mem_init(SRAM4);                     /* 初始化 SRAM4 内存池(SRAM4) */
    my_mem_init(SRAMDTCM);                  /* 初始化 DTCM 内存池(DTCM) */
    my_mem_init(SRAMITCM);                  /* 初始化 ITCM 内存池(ITCM) */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "FATFS TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "Use USMART for test", RED);

    while (sd_init())   /* 检测不到 SD 卡 */
    {
        lcd_show_string(30, 150, 200, 16, 16, "SD Card Error!", RED);
        delay_ms(500);
        lcd_show_string(30, 150, 200, 16, 16, "Please Check! ", RED);
        delay_ms(500);
        LED0_TOGGLE();  /* LED0 闪烁 */
    }
    ftl_init();
    exfuns_init();                          /* 为 fatfs 相关变量申请内存 */
    f_mount(fs[0], "0:", 1);                /* 挂载 SD 卡 */
    res = f_mount(fs[1], "1:", 1);          /* 挂载 FLASH. */
    if (res == 0X0D)    /* FLASH 磁盘,FAT 文件系统错误,重新格式化 FLASH */
    {
        /* 格式化 FLASH */
        lcd_show_string(30, 150, 200, 16, 16, "Flash Disk Formatting...", RED);
        /* 格式化 FLASH,1:,盘符;0,使用默认格式化参数 */
        res = f_mkfs("1:", 0, 0, FF_MAX_SS);

        if (res == 0)
        {
            /* 设置 Flash 磁盘的名字为: ALIENTEK */
            f_setlabel((const TCHAR *)"1:ALIENTEK");
            lcd_show_string(30, 150, 200, 16, 16, "Flash Disk Format Finish",
                            RED); /* 格式化完成 */
        }
        else lcd_show_string(30, 150, 200, 16, 16, "Flash Disk Format Error ",
                             RED);     /* 格式化失败 */
        delay_ms(1000);
    }
    res = f_mount(fs[2], "2:", 1); /* 挂载 FLASH. */
    if (res == 0X0D)    /* FLASH 磁盘,FAT 文件系统错误,重新格式化 FLASH */
    {
        /* 格式化 FLASH */
        lcd_show_string(30, 150, 200, 16, 16, "NAND Disk Formatting...", RED);
        /* 格式化 FLASH,1:,盘符;0,使用默认格式化参数 */
        res = f_mkfs("2:", 0, 0, FF_MAX_SS);

        if (res == 0)
        {
         /* 设置 Flash 磁盘的名字为: ALIENTEK */
         f_setlabel((const TCHAR *)"2:NANDDISK");
```

```
            lcd_show_string(30, 150, 200, 16, 16, "NAND Disk Format Finish",RED);
            /* 格式化完成 */
           }
    else lcd_show_string(30, 150, 200, 16, 16, "Flash Disk Format Error ", RED);
         /* 格式化失败 */
         delay_ms(1000);
     }

     lcd_fill(30, 150, 240, 150 + 16, WHITE);     /* 清除显示 */

     while (exfuns_get_free("0", &total, &free)) /* 得到 SD 卡的总容量和剩余容量 */
     {
         lcd_show_string(30, 150, 200, 16, 16, "SD Card Fatfs Error!", RED);
         delay_ms(200);
         lcd_fill(30, 150, 240, 150 + 16, WHITE);/* 清除显示 */
         delay_ms(200);
         LED0_TOGGLE();  /* LED0 闪烁 */
     }

     lcd_show_string(30, 150, 200, 16, 16, "FATFS OK!", BLUE);
     lcd_show_string(30, 170, 200, 16, 16, "SD Total Size:     MB", BLUE);
     lcd_show_string(30, 190, 200, 16, 16, "SD Free Size:     MB", BLUE);
     /* 显示 SD 卡总容量 MB */
     lcd_show_num(30 + 8 * 14, 170, total >> 10, 5, 16, BLUE);
     /* 显示 SD 卡剩余容量 MB */
     lcd_show_num(30 + 8 * 14, 190, free >> 10, 5, 16, BLUE);

     while (1)
     {
         t++;
         delay_ms(200);
         LED0_TOGGLE();  /* LED0 闪烁 */
     }
}
```

main 函数初始化了 LED 和 LCD 用于显示效果，初始化按键和 ADC 用于辅助显示 ADC。在 usmart_config.c 里面的 usmart_nametab 数组添加如下内容：

```
/* 函数名列表初始化(用户自己添加)
 * 用户直接在这里输入要执行的函数名及其查找串
 */
struct _m_usmart_nametab usmart_nametab[] =
{
#if USMART_USE_WRFUNS == 1      /* 如果使能了读写操作 */
    (void *)read_addr, "uint32_t read_addr(uint32_t addr)",
    (void *)write_addr, "void write_addr(uint32_t addr,uint32_t val)",
#endif
    (void *)delay_ms, "void delay_ms(uint16_t nms)",
    (void *)delay_us, "void delay_us(uint32_t nus)",

    (void *)mf_mount, "uint8_t mf_mount(uint8_t* path,uint8_t mt)",
    (void *)mf_open, "uint8_t mf_open(uint8_t*path,uint8_t mode)",
    (void *)mf_close, "uint8_t mf_close(void)",
    (void *)mf_read, "uint8_t mf_read(uint16_t len)",
    (void *)mf_write, "uint8_t mf_write(uint8_t*dat,uint16_t len)",
    (void *)mf_opendir, "uint8_t mf_opendir(uint8_t* path)",
    (void *)mf_closedir, "uint8_t mf_closedir(void)",
    (void *)mf_readdir, "uint8_t mf_readdir(void)",
    (void *)mf_scan_files, "uint8_t mf_scan_files(uint8_t * path)",
    (void *)mf_showfree, "uint32_t mf_showfree(uint8_t *path)",
    (void *)mf_lseek, "uint8_t mf_lseek(uint32_t offset)",
    (void *)mf_tell, "uint32_t mf_tell(void)",
    (void *)mf_size, "uint32_t mf_size(void)",
    (void *)mf_mkdir, "uint8_t mf_mkdir(uint8_t*path)",
    (void *)mf_fmkfs, "uint8_t mf_fmkfs(uint8_t*path,uint8_t opt,uint16_t au)",
    (void *)mf_unlink, "uint8_t mf_unlink(uint8_t *path)",
```

```
    (void *)mf_rename, "uint8_t mf_rename(uint8_t *oldname,uint8_t* newname)",
    (void *)mf_getlabel, "void mf_getlabel(uint8_t *path)",
    (void *)mf_setlabel, "void mf_setlabel(uint8_t *path)",
    (void *)mf_gets, "void mf_gets(uint16_t size)",
    (void *)mf_putc, "uint8_t mf_putc(uint8_t c)",
    (void *)mf_puts, "uint8_t mf_puts(uint8_t *str)",
    (void *)nand_eraseblock, "uint8_t nand_eraseblock(uint32_t blocknum)",
    (void *)nand_copypage_withoutwrite,"uint8_t nand_copypage_withoutwrite
                              (uint32_t source_pagenum,uint32_t dest_pagenum)",
    (void *)nand_erasechip, "void nand_erasechip(void)",

    (void *)ftl_init, "uint8_t ftl_init(void)",
    (void *)ftl_check_badblock,"uint8_t ftl_check_badblock(uint32_t blocknum)",
    (void *)ftl_used_blockmark,"uint8_t ftl_used_blockmark(uint32_t blocknum)",
    (void *)ftl_find_unused_block,"uint32_t ftl_find_unused_block(uint32_t
                                                    sblock, uint8_t flag)",
    (void *)ftl_find_same_plane_unused_block,"uint32_t
                          ftl_find_same_plane_unused_block(uint32_t sblock)",
    (void *)ftl_lbn_to_pbn, "uint16_t ftl_lbn_to_pbn(uint32_t lbnnum)",
    (void *)ftl_create_lut, "uint8_t ftl_create_lut(uint8_t mode)",
    (void *)ftl_format, "uint8_t ftl_format(void)",
};
```

55.4 下载验证

将程序下载到开发板后，我们使用的是 16Gb 标有“SDHC”标志的 micorSD 卡，可以看到 LCD 显示如图 55.4.1 所示：

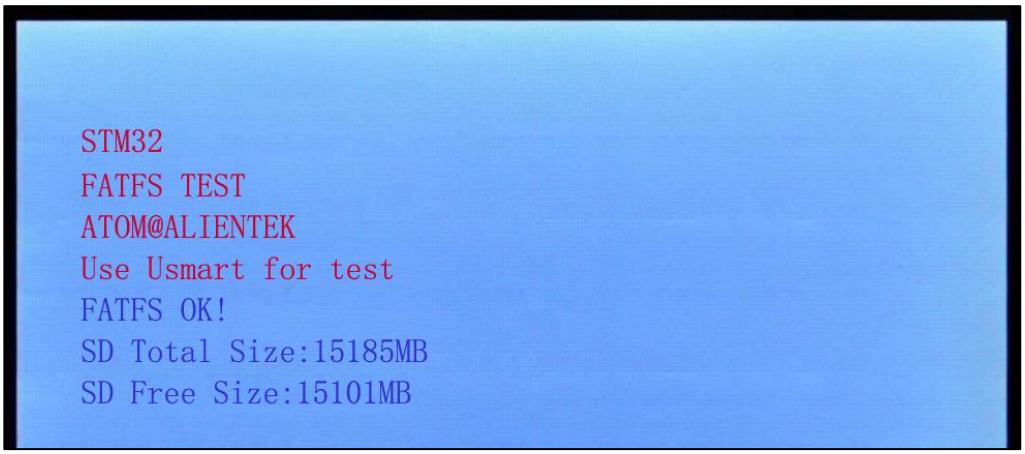


图 55.4.1 程序运行效果图

打开串口调试助手，我们就可以串口调用前面添加的各种 FATFS 测试函数了，比如我们输入 mf_scan_files("0:")，即可扫描 SD 卡根目录的所有文件，如图 55.4.2 所示：

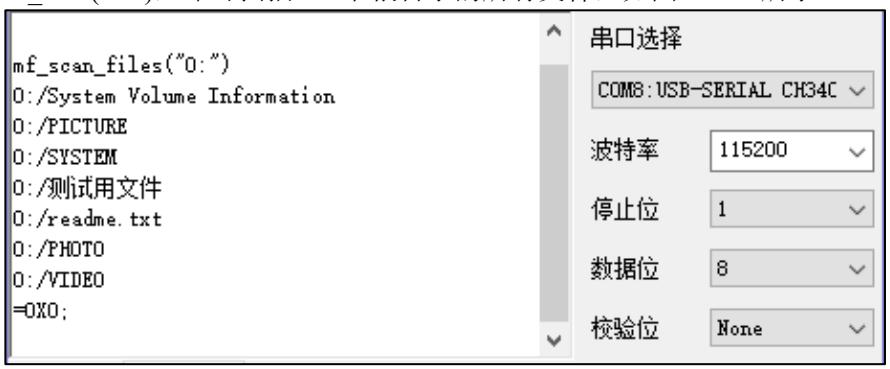


图 55.4.2 扫描 SD 卡根目录所有文件

其他函数的测试，用类似的办法即可实现。注意这里 0 代表 SD 卡，1 代表 SPI FLASH，2 代表 NAND FLASH。另外，提醒大家，mf_unlink 函数，在删除文件夹的时候，必须保证文件夹是空的，才可以正常删除，否则不能删除。

第五十六章 汉字显示实验

本章，我们将介绍如何使用 STM32 控制 LCD 显示汉字。在本章中，我们将使用外部 FLASH 来存储字库，并可以通过 SD 卡更新字库。STM32 读取存在 FLASH 里面的字库，然后将汉字显示在 LCD 上面。本章分为如下几个小节：

56.1 汉字显示介绍
56.2 硬件设计
56.3 程序设计
56.4 下载验证

56.1 汉字显示介绍

56.1.1 字符编码介绍

众所周知，计算机只能识别 0 和 1，所有信息都是以 0 和 1 的形式在计算机里面进行存储，文字信息也是一样。那么如何进行区分文字信息呢？这里就引出了字符编码这个概念，就是对字符进行编码，用规定的 01 数字串来表示特定的文字，最简单的字符编码例子是 ASCII 码。此外还有中文编码，中文编码又可以细分 GB2312，GB13000，GBK，BIG5（繁体）等几种。

1.ASCII 编码

ASCII 编码可以用来表示一些控制字符、英文以及数字，本质上都是二进制数，遵循一定的约定，所以就有了 ASCII 码表。ASCII 码表分为两个部分，第一部分数字编码范围 0~31 是控制字符或者是通讯专用字符，是不具备图形显示但对文本显示有影响的编码；第二部分数字编码范围 32~127 是包含我们日常用到的符号信息，空格、阿拉伯数字、标点符号、大小写英文以及“DEL（删除控制）”，除了 DEL 不具备图形显示外，其他都具备。工程代码中也有 ASCII 的编码表。下面看一下 ASCII 编码表第二部分中的可显示字符如图 56.1.1.1 所示：

二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形
0010 0000	32	20	(space)	0100 0000	64	40	@	0110 0000	96	60	`
0010 0001	33	21	!	0100 0001	65	41	A	0110 0001	97	61	a
0010 0010	34	22	"	0100 0010	66	42	B	0110 0010	98	62	b
0010 0011	35	23	#	0100 0011	67	43	C	0110 0011	99	63	c
0010 0100	36	24	$	0100 0100	68	44	D	0110 0100	100	64	d
0010 0101	37	25	%	0100 0101	69	45	E	0110 0101	101	65	e
0010 0110	38	26	&	0100 0110	70	46	F	0110 0110	102	66	f
0010 0111	39	27	'	0100 0111	71	47	G	0110 0111	103	67	g
0010 1000	40	28	(	0100 1000	72	48	H	0110 1000	104	68	h
0010 1001	41	29	)	0100 1001	73	49	I	0110 1001	105	69	i
0010 1010	42	2A	*	0100 1010	74	4A	J	0110 1010	106	6A	j
0010 1011	43	2B	+	0100 1011	75	4B	K	0110 1011	107	6B	k
0010 1100	44	2C	,	0100 1100	76	4C	L	0110 1100	108	6C	l
0010 1101	45	2D	-	0100 1101	77	4D	M	0110 1101	109	6D	m
0010 1110	46	2E	.	0100 1110	78	4E	N	0110 1110	110	6E	n
0010 1111	47	2F	/	0100 1111	79	4F	O	0110 1111	111	6F	o
0011 0000	48	30	0	0101 0000	80	50	P	0111 0000	112	70	p
0011 0001	49	31	1	0101 0001	81	51	Q	0111 0001	113	71	q

0011 0010	50	32	2	0101 0010	82	52	R	0111 0010	114	72	r
0011 0011	51	33	3	0101 0011	83	53	S	0111 0011	115	73	s
0011 0100	52	34	4	0101 0100	84	54	T	0111 0100	116	74	t
0011 0101	53	35	5	0101 0101	85	55	U	0111 0101	117	75	u
0011 0110	54	36	6	0101 0110	86	56	V	0111 0110	118	76	v
0011 0111	55	37	7	0101 0111	87	57	W	0111 0111	119	77	w
0011 1000	56	38	8	0101 1000	88	58	X	0111 1000	120	78	x
0011 1001	57	39	9	0101 1001	89	59	Y	0111 1001	121	79	y
0011 1010	58	3A	:	0101 1010	90	5A	Z	0111 1010	122	7A	z
0011 1011	59	3B	;	0101 1011	91	5B	[	0111 1011	123	7B	{
0011 1100	60	3C	<	0101 1100	92	5C	\	0111 1100	124	7C	|
0011 1101	61	3D	=	0101 1101	93	5D	]	0111 1101	125	7D	}
0011 1110	62	3E	>	0101 1110	94	5E	^	0111 1110	126	7E	~
0011 1111	63	3F	?	0101 1111	95	5F	_				

图 56.1.1.1 可显示字符

2. 中文编码

由于英文文字是由 26 个字母排列组合而成的，所以 ASCII 表就可以适用于表达英文词典库。在汉字系统中，每一个汉字都是一个独立的个体，但是汉字可以以偏旁以及笔画进行划分，不过也是十分杂乱，毕竟汉字现在已经有 8 万多个了，常用的只有 3500 个。所以中文编码是直接对方块字进行编码的，一个汉字使用一个编码。

汉字库十分庞大，需要使用 2 个字节进行编码，假如和 ASCII 码一样只使用 1 字节去编码，那只能表示 256 个汉字，太少了。在前面提及到 GB2312，GB13000，GBK，BIG5(繁体)等，它们都是一个标准。下面对具有代表性的 GB2312 和 GBK 进行讲解。

GB2312 编码

GB2312 是一个简体中文字符集的中国国家标准，也是我们通常说到的国标码。GB2312 由 6763 个常用汉字和 682 个全角的非汉字字符组成。其中根据汉字使用的频率分为两级。一级汉字 3755 个，二级汉字 3008 个。由于字符数量比较大，GB2312 采用了二维矩阵编码法对所有字符进行编码。构造一个 94 行 94 列的方阵，对每一行称为一个“区”，每一列称为一个“位”，然后把所有的字符都依照“GB2312 字符编码规则”，写入方阵中，这样子所有的字符在方阵中都有一个唯一的位置，这个位置就可以用区号和位号组合表示，称为字符的区位码。

因为 GB2312 与西文的存储存在冲突，所以 GB2312 字符在进行存储时，将原来的每一个字节第 8 位设置为 1 和西文加以区分，如果第 8bit 为 0，则表示西文字符，否则就是 GB2312 字符。在实际存储时，采用将区位码的每个字节分别加上 A0H(160)的方法转换为储存码，计算机存储规则则是此编码的补码，而且是位码在前，区码在后。举个例子：汉字“啊”的区位码为 1601，其存储码为 B0A1H，转换过程为：

区位码	区码转换	位码转换	储存码
1001H	10H+A0H=B0H	01H+A0H=A1H	B0A1H

GBK 编码

GBK 编码即汉字内码扩展规范，完全兼容 GB2312，在 GB2312 的基础上，支持繁体字、人名、古汉语等方面出现的罕用字。GBK 采用的是双字节表示，总体编码范围为 0x8140~0xFEFE，第一个字节在 0x81~0xFE 之间，第二个字节分为两个部分，一是 0x40~0x7E，二是 0x80~0xFE。其中和 GB2312 相同的区域，字完全相同，可表示的汉字数达到了 2 万多个，完全能满足我们的一般应用的要求。如图 56.1.1.2 GBK 码位分布图：

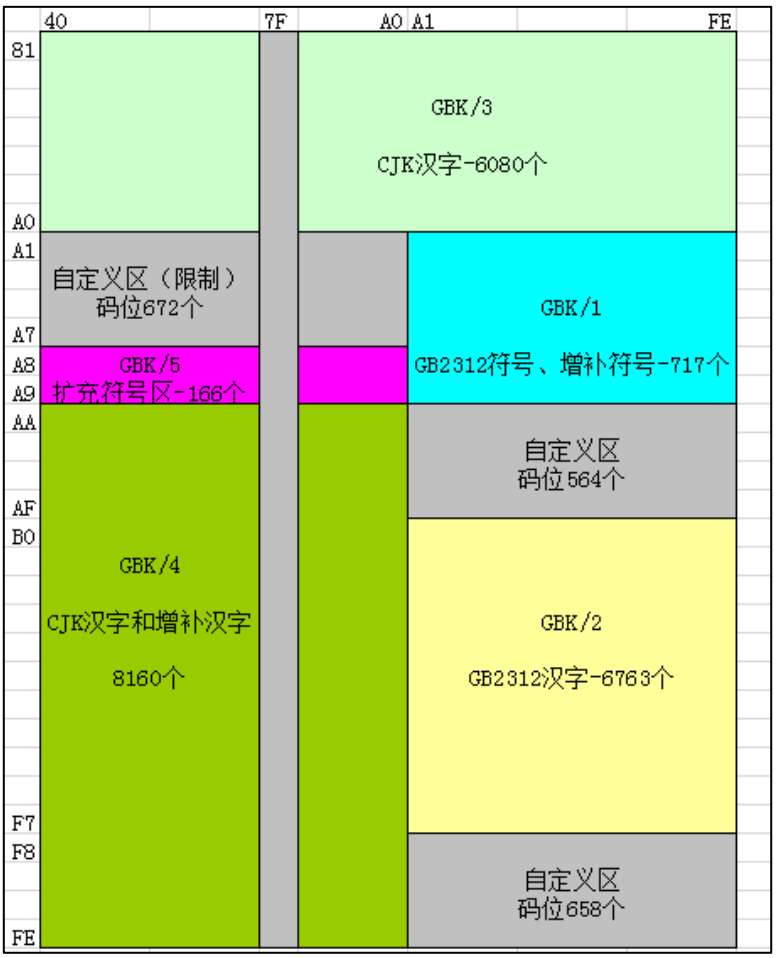


图 56.1.1.2 GBK 码位分布图

在前面 GB2312 也说到，第一字节称为区，那么 GBK 里面总共就有 126 个区（0xFE-0x81+1），第二个字节称为位，我们也可以理解为每个区里面包含的汉字即 190 个汉字（0xFE-0X80+0X7E-0X40+2），GBK 字库总共就 126*190=23940 个汉字。

3.全球统一编码

前一小节讲解的都是中国标准，只有中国使用，并没有表示大多数其他国家的编码。而其他国家又陆续推出各自的编码标准，互不兼容，非常不利于国际交流。所以在后来国际标准化组织(ISO)发行了一个全球统一编码表，把全球各国文字都统一在一个编码标准里，这个全球统一编码表就叫做 Unicode。Unicode 字符集对世界上各国文化使用到的字母和符号进行标号，对每一个字符都分配一个唯一的编号，字符的编号从 0x000000 到 0x10FFFF。

Unicode 没有规定字符对应的二进制码如何存储，只是对每一个字符进行编号。为了解决 Unicode 编码问题，UTF-8、UTF-16 和 UTF-32 的编码方式诞生了。

UTF-8 编码

UTF-8 是一个非常常用的编码方式，漂亮的实现了对 ASCII 码的向后兼容，它是目前互联网上使用最广泛的一种 Unicode 编码方式，它的最大特点就是可变长。它可以使用 1-4 个字节表示一个字符，根据字符的不同变换长度。编码规则如下：

1.对于单字节的字符，第一位设为 0，后面 7 位对应这个字符的 Unicode 码点（即编号）。因此英文中 0~127 字符与 ASCII 码完全相同。这意味着可以使用 UTF-8 编码格式打开用 ASCII 码编写的文档。

2.对于需要使用 N 个字节来表示的字符（N>1），第一个字节的前 N 位都设为 1，第 N+1 位设为 0，剩下的 N-1 个字节的前两位都设为 10，剩下的二进制位则使用这个字符的 Unicode 的码点进行填充。

上面的表述可能比较难接受，下面看一下编码规则表就清晰很多，如表 56.1.1.1 所示：

Unicode 十六进制码点范围	UTF-8 二进制
0000 0000 - 0000 007F	0xxxxxxx
0000 0080 - 0000 07FF	110xxxxx 10xxxxxx
0000 0800 - 0000 FFFF	1110xxxx 10xxxxxx 10xxxxxx
0001 0000 - 0010 FFFF	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

表 56.1.1.1 UTF-8 编码规则表

有了上面这个编码规则表，进行 UTF-8 编码和解码就简单多了。以汉字“汉”为例，它的 Unicode 的码点是 0x6c49(110 1100 0100 1001)，通过查表发现它属于第三行的规则，具体格式：1110xxxx 10xxxxxx 10xxxxxx。接着我们就可以从“汉”的二进制数的最后一位开始，从后向前依次填充对应的格式中的 x，多出的 x 用 0 补充。这样就可以的到“汉”的 UTF-8 编码：11100110 10110001 10001001，转换成十六进制就是 0xE6 0xB1 0x89。

解码的过程也是十分简单，通过判断一个字节的第一位，假如是 0，说明这个字节对应一个字符；假如是 1 则需要往后数有多少个连续不间断的 1，就表示该字符占用多少个字节。

UTF-16

UTF-16 采用的是 2 字节或 4 字节编码方式。对于 Unicode 编码范围在 0x0000~0xFFFF 之间字符，UTF-16 使用两个字节存储，并直接存储 Unicode 编号，不用编码转换。对于 Unicode 编号范围在 0x10000~0x10FFFF 之间的字符，UTF-16 使用四个字节存储，具体来说是：将字符编号的所有比特位分成两部分，较高的一些比特位用一个值介于 0xD800 和 0xDBFF 之间的双字节存储，较低的一些比特位（剩下的比特位），用一个值介于 0xDC00 和 DFFF 之间的双字节存储。

上面表述的可能有点模糊，请看下面的表格 56.1.1.2 所示：

Unicode 编码范围 (Hex)	具体的 Unicode 编号（BIN）	UTF-16 编码	编码后的字节数
0000 0000~0000 FFFF	xxxxxxxx xxxxxxxx	xxxxxxxx xxxxxxxx	2
0001 0000~0010 FFFF	yyyy yyyy yyxx xxxx xxxx	110110yyyyyyyyyy 110111xx xxxxxxxx	4

表 56.1.1.2 UTF-16 编码规则表

举个例子，用 UTF-16 两个字节去表示“汉”字，它的 Unicode 码点是 0x6C49，那么遵行上面的表格规则，那么用 UTF-16 表示的话就是 01101100 01001001（共 16bit，两个字节）。

现在以第二种情况讲解，汉字“？”为例，它的 Unicode 码点为 0x20BB7，该码点显然超出了基本平面的范围(0x0000 – 0xFFFF)，因此需要使用四个字节进行表示，首先用 0x00BB7–0x10000 计算超出部分，然后将其用 20 个二进制位表示(不足的前面补 0)，结果为 0001000010 1110110111。接着将高 10 位映射到 U+D800 到 U+DBFF 之间，低 10 位映射到 U+DC00 到 U+DFFF 即可。那么换算一下即（110110 0000000000 + 0001000010），高位转换一下 16 进制即 0xD842，同理低位也是这样运算，得到 0xDFB7。因此，得出汉字“？”的 UTF-16 编码为 0xD842 0xDFB7。

UTF-32

UTF-32 是固定长度的编码，始终占用 4 个字节，足以容纳所有的 Unicode 字符，所以直接存储 Unicode 编码即可，不需要任何编码转换。浪费了空间，提供了效率。

大小端模式

在了解了前面的各种解码之后，就知道一个字符可能占用多个字节，那么多个字节在计算机中如何进行存储呢？比如字符 0xABCD，它的存储格式到底是 AB CD，还是 CD AB 呢？

实际上两者都有可能，并且分别有不同的名字。如果存储为 AB CD，则称为 Big Endian 即大端；如果存储为 CD AB，则称为 Little Endian 即小端。

Big Endian：高字节在前，低字节在后，详细一点，就是将高位的字节放在低地址表示。

Little Endian：低字节在前，高字节在后，详细一点，就是将高位的字节放在高地址表示。

在了解大小端的意思之后，我们稍微科普一下 BOM。

BOM

BOM 是文档最前面的标记，位于文本文件的开头，一种标记对应一种编码方式。为什么

会需要这个东西呢？因为计算机打开文档时，它首先需要获取一些信息，知道用什么编码方式去解码，才能知道文档写的是什么内容。注意：BOM 是对 Unicode 的几种编码而言的。下面我们看一下 BOM 的标记表，如表 56.1.1.3 所示：

BOM 标记	表示的编码
0xEF 0xBB 0xBF	UTF-8
0xFF 0xFE	UTF-16 小端模式
0xFE 0xFF	UTF-16 大端模式
0xFF 0xFE 0x00 0x00	UTF-32 小端模式
0x00 0x00 0xFE 0xFF	UTF-32 大端模式

表 56.1.1.3 BOM 标记表

56.1.2 字库的生成

有了编码，我们就能可以在计算机上对字符进行操作，但是如果计算机处理完字符后直接以编码的形式输出，我们很难一下子知道究竟是什么图形的字符。因为我们平常看到的都是字符的图形化显示，这里呢，就需要我们提供配套的字符图形。字符图形又称为字模，多个字模组成的文件就叫做字库。当我们为计算机提供了编码和字库的前提下，计算机就可以把字符编码转化成对应的字符图形，我们才能清晰识别。

字模的结构

在前面 OLED 显示实验章节也有说到字模相关知识，这里我们继续展开详细讲解一下，字模是字符的图形结构，字模的实质就是一个个像素点数据。为了方便处理，我们把字模定义成方块形状的像素点阵，像素点只有 0 和 1 两种状态。在单色图像数据中，像素点数值置 1 时，点亮了该像素点，若像素点数值置 0 时，熄灭该像素点。“汉字”字模图，如图 56.1.2.1 所示：

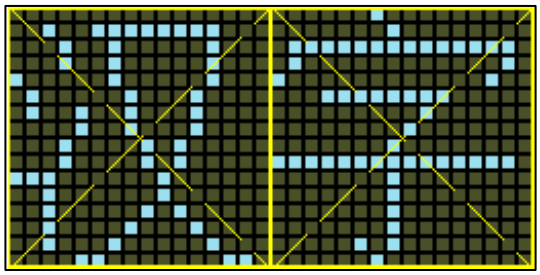


图 56.1.2.1 “汉字”字模图

这两个字模的大小都是 16*16，计算机要表示这样的图形，只需要 16x16 个二进制数据位即可，需要使用 16x16/8=32 个字节保存字模数据。这里存在一个问题，字模数据是从哪里标记为第一个字节呢？我们知道了第一个字节以及字模的走向，我们就可以把生成的点阵数据和字模图像上的小方块进行匹配，更好了解两者的关系。下面我们解密一下，如图 56.1.2.2 取模方式（从上到下，从左到右）进行取模，这里的取模方式也是和前面的 OLED 显示实验章节一样的，可以往前面章节翻翻。

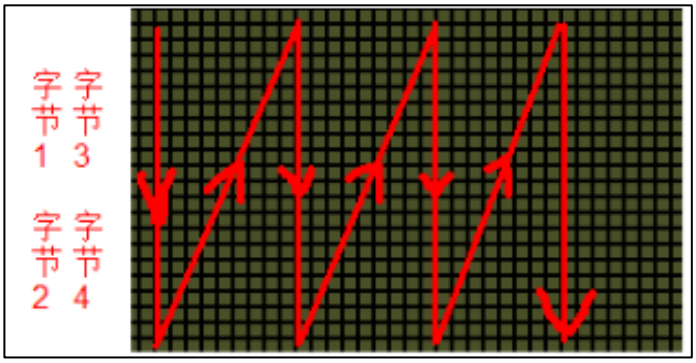


图 56.1.2.2 取模方式

设置好后，生成的数据如下：

```
{0x08,0x20,0x06,0x20,0x40,0x3E,0x30,0xC0,0x03,0x01,0x40,0x01,0x78,0x02,
```

```
0x47,0x04,0x40,0xC8,0x40,0x30,0x40,0xC8,0x47,0x04,0x78,0x02,0x00,0x01,
0x00,0x01,0x00,0x00},/*"汉",0*/
```

便于大家更好地将点阵数据和图形连接起来，请看图 56.1.2.3 所示：

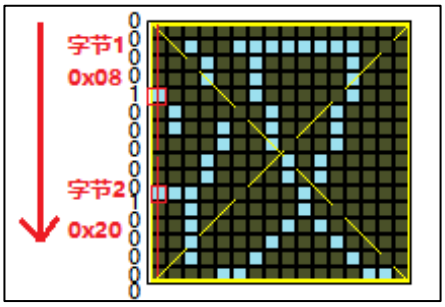


图 56.1.2.3 点阵数据和图形匹配图

了解了字模的结构之后，我们就需要去生成字库了。我们用到一款由正点原子团队开发的软件：ATK_XFONT。该软件可以在 WINDOWS 系统下生成任意点阵大小的 ASCII，GB2312(简体中文)、GBK(简体中文)、BIG5(繁体中文)和 Unicode 等共二十几种编码的字库，不但支持生成二进制文件格式的文件，还支持生成图片功能，并支持横向，纵向等多种扫描方式，扫描方式可以根据用户的需求进行增加。该软件的界面如图 56.1.2.4 所示：

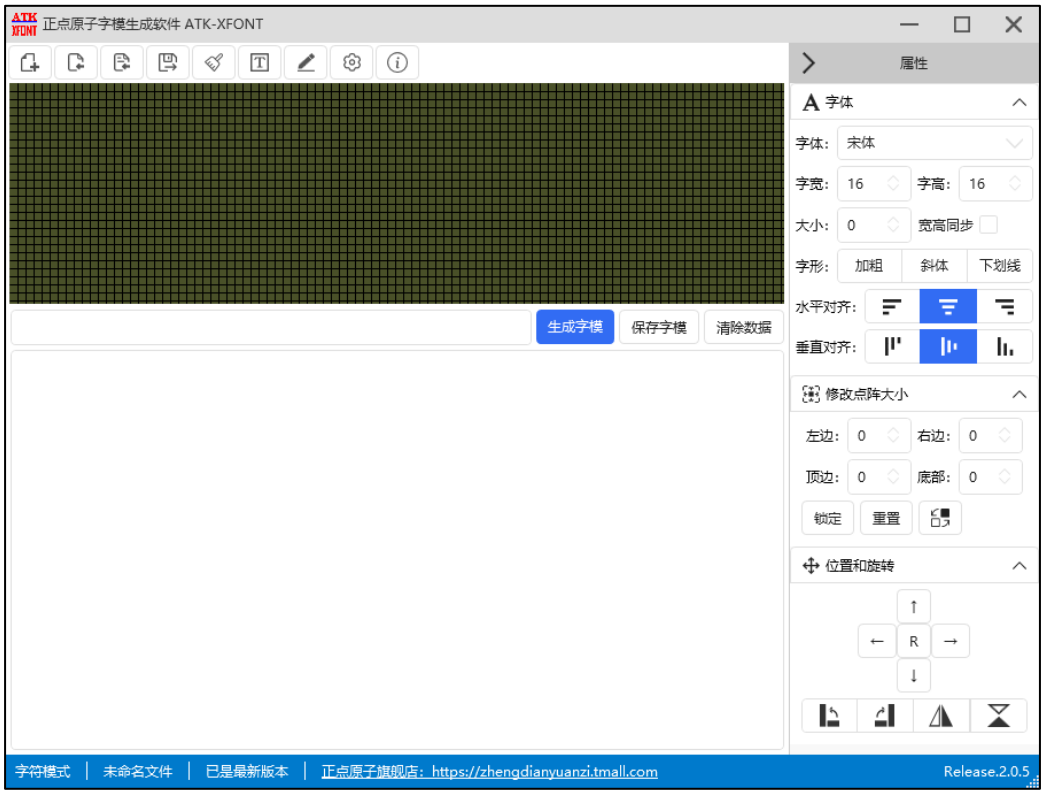


图 56.1.2.4 点阵字库生成器默认界面

本实验，我们总共要生成 4 个字库：12*12 字库、16*16 字库、24*24 字库和 32*32 字库。这里以 16*16 字库为例进行介绍，其他三个字库的制作方法类似。要生成 16*16 的 GBK 字库，则需要选择字体大小、选择编码字库、选择字宽和高度、取模方式、存放路径等操作，最后点击创建，就可以开始生成我们需要的字库（.DZK 文件），具体设置如图 56.1.2.5 所示：

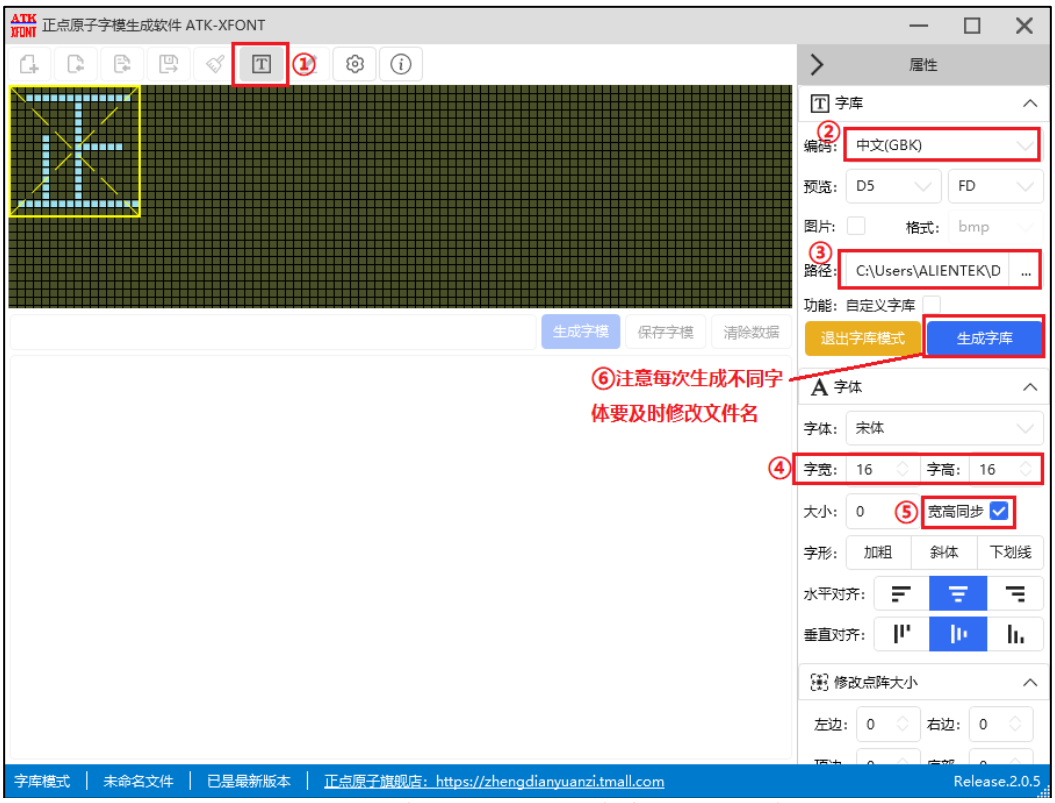


图 56.1.2.5 生成 GBK16*16 字库的设置方法

注意：电脑端的字体大小与我们生成点阵大小的关系为：

$$fsize = dsize * 6 / 8$$

其中，fsize 是电脑端字体的大小，dsize 是点阵大小（12、16、24、32 等）。所以 16*16 点阵大小对应的是 12 号字体。

生成完之后，我们把文件名和后缀改成：GBK16.FON。同样的方法，生成 12*12 的点阵库（GBK12.FON）、24*24 的点阵库（GBK24.FON）和 32*32 的点阵库（GBK32.FON），总共制作 4 个字库。

构建字库完成，那我们怎么把字库跟编码连接起来呢？以 GBK16.FON 为例子，它是根据 GBK 的编码规则从 0X8140 开始，逐一建立。因为 16*16 的点阵大小为 32 字节，所以在 FON 文件中第一个 32 字节的数据就是“ 丐”的字模数据，它的编码是 0x8140，而第二个 32 字节的数据就是“ 丒”的字模数据，它的编码就是 0x8141。在前面讲解 GBK 的时候，也提到了 GBK 具有 126 区，每个区具有 190 个汉字，那么我们就可以通过计算地址偏移，最终在这个字库里面定位汉字了：

当 GBKL<0X7F 时：Hp=((GBKH-0x81)*190+GBKL-0X40)*csize；

当 GBKL>0X80 时：Hp=((GBKH-0x81)*190+GBKL-0X41)*csize；

其中 GBKH、GBKL 分别代表 GBK 的第一个字节和第二个字节(也就是高位和低位)，Hp 为对应汉字点阵数据在字库里面的起始地址(假设是从 0 开始存放)，csize 代表一个汉字点阵所占的字节数。

这样我们只要得到了汉字的 GBK 码，就可以得到该汉字点阵在点阵库里面的位置，从而获取其点阵数据，显示这个汉字了。

56.1.3 汉字显示原理

汉字在液晶上的显示原理与前面 OLED 实验中显示字符的是一样的。汉字在液晶上的显示其实就是一些点的显示与不显示，这就相当于我们的笔一样，有笔经过的地方就画出来，没经过的地方就不画。所以要显示汉字，我们首先要知道汉字的点阵数据，这些数据可以由专门的软件来生成，在前面字库生成已经有讲到如何生成字模与字库了。

在我们前面制作的四个 GBK 字库的基础上，将它们放在 SD 卡里，然后通过 SD 卡，将字

库文件复制到外部 FLASH 芯片 NORFLASH 里，这样，NORFLASH 就相当于一个汉字字库芯片了。

单片机要显示汉字的步骤：汉字内码（GBK/GB2312）→查找点阵库→解析→显示。所以只要我们有了整个汉字库的点阵，就可以把字符图形在单片机上显示出来了。

56.2 硬件设计

1. 例程功能

本实验开机的时候先检测 norflash 中是否已经存在字库，如果存在，则按次序显示汉字(四种字体都显示)。如果没有，则检测 SD 卡和文件系统，并查找 SYSTEM 文件夹下的 FONT 文件夹，在该文件夹内查找 UNIGBK.BIN、GBK12.FON、GBK16.FON、GBK24.FON 和 GBK32.FON（这几个文件的由来，见 STM32H743 开发指南）。在检测到这些文件之后，就开始更新字库，更新完毕才开始显示汉字。通过按按键 KEY0，可以强制更新字库。

LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED ：LED0 – PB1
2）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)
3）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）SD 卡，通过 SDMMC1(SDMMC_D0~D4(PC8~PC11)，SDMMC_SCK(PC12)，SDMMC_CMD(PD2))连接
5）norflash(QSPI FLASH 芯片，连接在 QSPI 上)
6）独立按键
KEY0 – PH3

56.3 程序设计

56.3.1 程序流程图

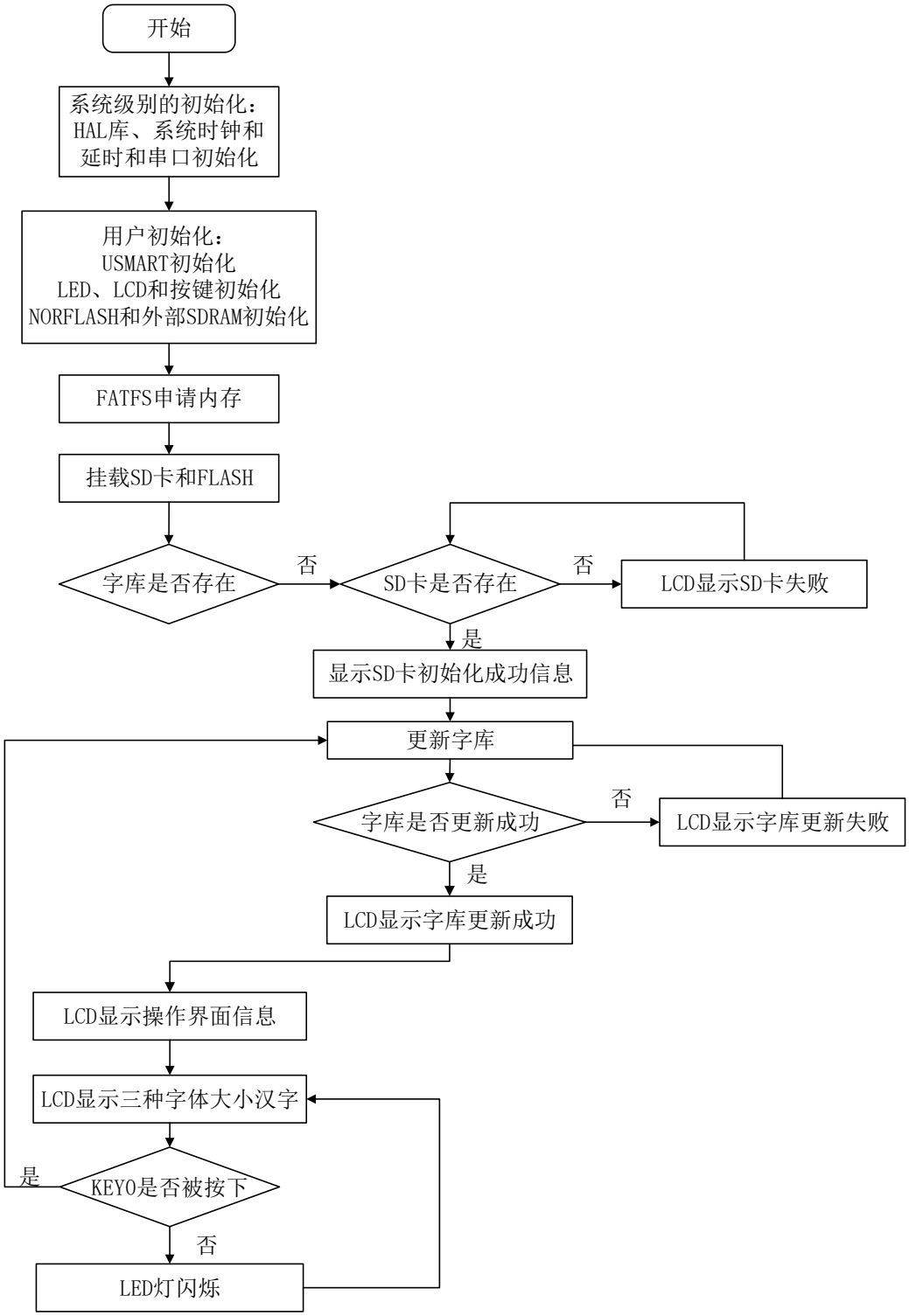


图 56.3.1.1 汉字显示实验程序流程图

56.3.2 程序解析

1. TEXT 代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。TEXT 驱动源码包括四个文件：text.c、text.h、fonts.c 和 fonts.h。

汉字显示实验代码主要分为两部分：一部分就是对字库的更新，另一部分就是对汉字的显示。字库的更新代码放在 font.c 和 font.h 文件中，汉字的显示代码就放在 text.c 和 text.h 中。

下面我们介绍一下有关字库操作的代码，首先我们先看一下 fonts.h 文件中字库信息结构体定义，其代码如下：

```
/* 字库信息结构体定义
 * 用来保存字库基本信息，地址，大小等
 */
__packed typedef struct
{
    uint8_t  fontok;            /* 字库存在标志，0XAA，字库正常；其他，字库不存在 */
    uint32_t ugbkaddr;          /* unigbk 的地址 */
    uint32_t ugbksize;          /* unigbk 的大小 */
    uint32_t f12addr;           /* gbk12 地址 */
    uint32_t gbk12size;         /* gbk12 的大小 */
    uint32_t f16addr;           /* gbk16 地址 */
    uint32_t gbk16size;         /* gbk16 的大小 */
    uint32_t f24addr;           /* gbk24 地址 */
   uint32_t gbk24size;          /* gbk24 的大小 */
    uint32_t f32addr;           /* gbk32 地址 */
    uint32_t gbk32size;         /* gbk32 的大小 */
} _font_info;
```

这个结构体用于记录字库的首地址以及字库大小等信息，总共占用 41 个字节，第一个字节用来标识字库是否 OK，其他的用来记录地址和文件大小。我们将 NORFLASH（W25Q256）的前 25M 字节给 FATFS 管理（用做本地磁盘），随后，紧跟字库结构体、UNIGBK.bin、和四个字库，这部分内容首地址是：（1024*12）*1024，大小约 6.01M，最后 NORFLASH（W25Q256）还剩下约 0.99M 给用户自己用。

下面介绍 font.c 文件几个重要的函数。

字库初始化函数也是利用其存储顺序，进行检查字库，其定义如下：

```
/**
 * @brief       初始化字体
 * @param       无
 * @retval      0, 字库完好; 其他, 字库丢失;
 */
uint8_t fonts_init(void)
{
    uint8_t t = 0;
    while (t < 10)  /* 连续读取 10 次,都是错误,说明确实是有问题,得更新字库了 */
    {
        t++;
        /* 读出 ftinfo 结构体数据 */
        norflash_read((uint8_t *)&ftinfo, FONTINFOADDR, sizeof(ftinfo));

        if (ftinfo.fontok == 0XAA)
        {
            break;
        }

        delay_ms(20);
    }
    if (ftinfo.fontok != 0XAA)
    {
        return 1;
```

```
    }
    return 0;
}
```

该函数就是从 NORFLASH 芯片中，看到 ftinfo 的结构体定义，总共占用 41 个字节，第一个字节 用来标识字库是否 OK，其他的用来记录地址和文件大小

有人会有疑问，ftinfo.fontok 在哪里赋值 AA 呢？肯定是字库更新完毕后，给该标记赋值的，那下面就来看一下是不是这样子，字库更新函数定义如下：

```
/**
 * @brief       更新字体文件
 * @note        所有字库一起更新(UNIGBK,GBK12,GBK16,GBK24,GBK32)
 * @param       x, y    : 提示信息的显示地址
 * @param       size    : 提示信息字体大小
 * @param       src     : 字库来源磁盘
 *   @arg                 "0:", SD 卡;
 *   @Arg                 "1:", SPI FLASH 盘
 *   @arg                 "2:", NAND FLASH 盘
 * @param       color   : 字体颜色
 * @retval      0, 成功; 其他, 错误代码;
 */
uint8_t fonts_update_font(uint16_t x, uint16_t y, uint8_t size,
                          uint8_t *src, uint16_t color)
{
    uint8_t *pname;
    uint32_t *buf;
    uint8_t res = 0;
    uint16_t i, j;
    FIL *fftemp;
    uint8_t rval = 0;
    res = 0XFF;
    ftinfo.fontok = 0XFF;
    pname = mymalloc(SRAMIN, 100);  /* 申请 100 字节内存 */
    buf = mymalloc(SRAMIN, 4096);   /* 申请 4K 字节内存 */
    fftemp = (FIL *)mymalloc(SRAMIN, sizeof(FIL));  /* 分配内存 */

    if (buf == NULL || pname == NULL || fftemp == NULL)
    {
        myfree(SRAMIN, fftemp);
        myfree(SRAMIN, pname);
        myfree(SRAMIN, buf);
        return 5;   /* 内存申请失败 */
    }

    for (i = 0; i < 5; i++)/* 先查找文件 UNIGBK,GBK12,GBK16,GBK24,GBK32 是否正常 */
    {
        strcpy((char *)pname, (char *)src);          /* copy src 内容到 pname */
        strcat((char *)pname, (char *)FONT_GBK_PATH[i]); /* 追加具体文件路径 */
        res = f_open(fftemp, (const TCHAR *)pname, FA_READ);/* 尝试打开 */

        if (res)
        {
            rval |= 1 << 7;   /* 标记打开文件失败 */
            break;            /* 出错了,直接退出 */
        }
    }

    myfree(SRAMIN, fftemp);     /* 释放内存 */

    if (rval == 0)              /* 字库文件都存在. */
    {
        /* 提示正在擦除扇区 */
        lcd_show_string(x, y, 240, 320, size, "Erasing sectors... ", color);
```

```
        for (i = 0; i < FONTSECSIZE; i++)       /* 先擦除字库区域,提高写入速度 */
        {
          fonts_progress_show(x+20*size/2, y, size, FONTSECSIZE, i, color);
          /* 进度显示 */
          norflash_ex_read((uint8_t *)buf,((FONTINFOADDR/4096)+i)*4096, 4096);
            /* 读出整个扇区的内容 */
            for (j = 0; j < 1024; j++)              /* 校验数据 */
            {
                if (buf[j] != 0XFFFFFFFF)break;     /* 需要擦除 */
            }

            if (j != 1024)
            {
                /* 需要擦除的扇区 */
                norflash_ex_erase_sector((FONTINFOADDR / 4096) + i);
            }
        }

        for (i = 0; i < 5; i++) /* 依次更新 UNIGBK,GBK12,GBK16,GBK24,GBK32 */
        {
            lcd_show_string(x, y, 240, 320, size, FONT_UPDATE_REMIND_TBL[i],
                            color);
            strcpy((char *)pname, (char *)src);      /* copy src 内容到 pname */
            strcat((char *)pname, (char *)FONT_GBK_PATH[i]);/* 追加具体文件路径 */
            res = fonts_update_fontx(x + 20 * size / 2, y, size, pname, i,
                                     color);     /* 更新字库 */

            if (res)
            {
                myfree(SRAMIN, buf);
                myfree(SRAMIN, pname);
                return 1 + i;
            }
        }

        /* 全部更新好了 */
        ftinfo.fontok = 0XAA;
       /* 保存字库信息 */
        norflash_ex_write((uint8_t *)&ftinfo, FONTINFOADDR, sizeof(ftinfo));
    }
    myfree(SRAMIN, pname);  /* 释放内存 */
    myfree(SRAMIN, buf);
    return rval;              /* 无错误. */
}
```

函数的实现：动态申请内存→尝试打开文件(UNIGBK、GBK12、GBK16、GBK24 和 GBK32)，确定文件是否存在→擦除字库→依次更新 UNIGBK、GBK12、GBK16、GBK24 和 GBK32→写入 ftinfo 结构体信息。

在字库更新函数中能直接看到的是 ftinfo.fontok 成员被赋值，而其他成员在单个字库更新函数中被赋值，接下来分析一下更新某个字库函数，其代码如下：

```
/**
 * @brief       更新某一个字库
 * @param       x, y    : 提示信息的显示地址
 * @param       size    : 提示信息字体大小
 * @param       fpath   : 字体路径
 * @param       fx      : 更新的内容
 *   @arg                 0, ungbk;
 *   @Arg                 1, gbk12;
 *   @arg                 2, gbk16;
 *   @arg                 3, gbk24;
 *   @arg                 4, gbk32;
 * @param       color   : 字体颜色
 * @retval      0, 成功; 其他, 错误代码;
```

```
 */
static uint8_t fonts_update_fontx(uint16_t x, uint16_t y, uint8_t size,
                                  uint8_t *fpath, uint8_t fx, uint16_t color)
{
    uint32_t flashaddr = 0;
    FIL *fftemp;
    uint8_t *tempbuf;
    uint8_t res;
    uint16_t bread;
    uint32_t offx = 0;
    uint8_t rval = 0;
    fftemp = (FIL *)mymalloc(SRAMIN, sizeof(FIL));  /* 分配内存 */

    if (fftemp == NULL)rval = 1;

    tempbuf = mymalloc(SRAMIN, 4096);               /* 分配 4096 个字节空间 */

    if (tempbuf == NULL)rval = 1;

    res = f_open(fftemp, (const TCHAR *)fpath, FA_READ);

    if (res)rval = 2;   /* 打开文件失败 */

    if (rval == 0)
    {
        switch (fx)
        {
            case 0: /* 更新 UNIGBK.BIN */
            /* 信息头之后，紧跟 UNIGBK 转换码表 */
                ftinfo.ugbkaddr = FONTINFOADDR + sizeof(ftinfo);
                ftinfo.ugbksize = fftemp->obj.objsize;    /* UNIGBK 大小 */
                flashaddr = ftinfo.ugbkaddr;
                break;

            case 1: /* 更新 GBK12.BIN */
            /* UNIGBK 之后，紧跟 GBK12 字库 */
                ftinfo.f12addr = ftinfo.ugbkaddr + ftinfo.ugbksize;
                ftinfo.gbk12size = fftemp->obj.objsize;    /* GBK12 字库大小 */
                flashaddr = ftinfo.f12addr;                /* GBK12 的起始地址 */
                break;

            case 2: /* 更新 GBK16.BIN */
            /* GBK12 之后，紧跟 GBK16 字库 */
                ftinfo.f16addr = ftinfo.f12addr + ftinfo.gbk12size;
                ftinfo.gbk16size = fftemp->obj.objsize;  /* GBK16 字库大小 */
                flashaddr = ftinfo.f16addr;              /* GBK16 的起始地址 */
                break;

            case 3: /* 更新 GBK24.BIN */
            /* GBK16 之后，紧跟 GBK24 字库 */
                ftinfo.f24addr = ftinfo.f16addr + ftinfo.gbk16size;
                ftinfo.gbk24size = fftemp->obj.objsize; /* GBK24 字库大小 */
                flashaddr = ftinfo.f24addr;             /* GBK24 的起始地址 */
                break;

            case 4: /* 更新 GBK32.BIN */
            /* GBK24 之后，紧跟 GBK32 字库 */
                ftinfo.f32addr = ftinfo.f24addr + ftinfo.gbk24size;
                ftinfo.gbk32size = fftemp->obj.objsize; /* GBK32 字库大小 */
                flashaddr = ftinfo.f32addr;             /* GBK32 的起始地址 */
                break;
        }
```

```
        while (res == FR_OK)    /* 死循环执行 */
        {
            res = f_read(fftemp, tempbuf, 4096, (UINT *)&bread);  /* 读取数据 */
            if (res != FR_OK)break;      /* 执行错误 */
            /* 从 0 开始写入 bread 个数据 */
            norflash_ex_write(tempbuf, offx + flashaddr, bread);
            offx += bread;
            /* 进度显示 */
            fonts_progress_show(x, y, size, fftemp->obj.objsize, offx, color);

            if (bread != 4096)break;     /* 读完了. */
        }
        f_close(fftemp);
    }
    myfree(SRAMIN, fftemp);      /* 释放内存 */
    myfree(SRAMIN, tempbuf);     /* 释放内存 */
    return res;
}
```

单个字库更新函数，主要是对把字库从 SD 卡中读取出数据，写入 NORFLASH。同时把字库大小和起始地址保存在 ftinfo 结构体里，在前面的整个字库更新函数中使用函数：

```
norflash_write((uint8_t *)&ftinfo,FONTINFOADDR,sizeof(ftinfo)); /*保存字库信息*/
```

结构体的所有成员一并写入到那 41 个字节。有了这个字库信息结构体，就能很容易进行定位。结合前面的说到的根据地址偏移寻找汉字的点阵数据，我们就可以开始真正把汉字搬上屏幕中去了。

首先我们肯定需要获得汉字的 GBK 码，这里 MDK 已经帮我们实现了。这里用一个例子说明：

```
char* HZ_str = "正点原子";
printf("正点原子的'正'字GBK高位码: %#x \r\n",*HZ_str);
printf("正点原子的'正'字GBK低位码: %#x \r\n",*(HZ_str+1));
```

串口打印

```
正点原子的'正'字GBK高位码: 0xd5
正点原子的'正'字GBK低位码: 0xfd
```

在这里可以看出 MDK 识别汉字的方式是 GBK 码，换句话来说就是 MDK 自动会把汉字看成是两个字节表示的东西。知道了要表示的汉字和其 GBK 码，那么就可以去找对应的点阵数据。在 text.c 文件中，我们就定义了一个获取汉字点阵数据的函数，其定义如下：

```
/**
 * @brief       获取汉字点阵数据
 * @param       code  : 当前汉字编码(GBK 码)
 * @param       mat   : 当前汉字点阵数据存放地址
 * @param       size  : 字体大小
 * @note        size 大小的字体,其点阵数据大小为: (size / 8 + ((size % 8) ? 1 : 0))
                                                  * (size)  字节
 * @retval      无
 */
static void text_get_hz_mat(uint8_t *code, uint8_t *mat, uint8_t size)
{
    uint8_t qh, ql;
    uint8_t i;
    uint64_t foffset;
    /* 得到字体一个字符对应点阵集所占的字节数 */
    uint8_t csize = (size / 8 + ((size % 8) ? 1 : 0)) * (size);
    qh = *code;
    ql = *(++code);

    if (qh < 0x81 || ql < 0x40 || ql == 0xff || qh == 0xff)   /* 非 常用汉字 */
    {
```

```
        for (i = 0; i < csize; i++)
        {
            *mat++ = 0x00;  /* 填充满格 */
        }

        return;     /* 结束访问 */
    }

    if (ql < 0x7f)
    {
        ql -= 0x40; /* 注意! */
    }
    else
    {
        ql -= 0x41;
    }

    qh -= 0x81;
    foffset = ((unsigned long)190 * qh + ql) * csize;  /* 得到字库中的字节偏移量 */

    switch (size)
    {
        case 12:
            norflash_read(mat, foffset + ftinfo.f12addr, csize);
            break;

        case 16:
            norflash_read(mat, foffset + ftinfo.f16addr, csize);
            break;

        case 24:
            norflash_read(mat, foffset + ftinfo.f24addr, csize);
            break;

        case 32:
            norflash_read(mat, foffset + ftinfo.f32addr, csize);
            break;
    }
}
```

函数实现的依据就是前面 56.1.2 小节讲到的两条公式：

当 GBKL<0X7F 时：Hp=((GBKH-0x81)*190+GBKL-0X40)*csize；

当 GBKL>0X80 时：Hp=((GBKH-0x81)*190+GBKL-0X41)*csize；

目标汉字的 GBK 码满足上面两条公式其一，就会得出与一个 GBK 对应的汉字点阵数据的偏移。在这个基础上，通过判断汉字的大小，就可以从对应的字库提取目标汉字点阵数据。

在获取到点阵数据后，接下来就可以进行汉字显示，下面看一下汉字显示函数，其定义如下：

```
/**
 * @brief       显示一个指定大小的汉字
 * @param       x,y   : 汉字的坐标
 * @param       font  : 汉字 GBK 码
 * @param       size  : 字体大小
 * @param       mode  : 显示模式
 * @note              0, 正常显示(不需要显示的点,用 LCD 背景色填充,即 g_back_color)
 * @note              1, 叠加显示(仅显示需要显示的点, 不需要显示的点, 不做处理)
 * @param       color : 字体颜色
 * @retval      无
 */
void text_show_font(uint16_t x, uint16_t y, uint8_t *font, uint8_t size,
                                            uint8_t mode, uint16_t color)
{
    uint8_t temp, t, t1;
```

```
    uint16_t y0 = y;
    uint8_t *dzk;
    /* 得到字体一个字符对应点阵集所占的字节数 */
    uint8_t csize = (size / 8 + ((size % 8) ? 1 : 0)) * (size);
    if (size != 12 && size != 16 && size != 24 && size != 32)
    {
        return;     /* 不支持的 size */
    }

    dzk = mymalloc(SRAMIN, size);           /* 申请内存 */
    if (dzk == 0) return;                   /* 内存不够了 */
    text_get_hz_mat(font, dzk, size);       /* 得到相应大小的点阵数据 */
    for (t = 0; t < csize; t++)
    {
        temp = dzk[t];                      /* 得到点阵数据 */
        for (t1 = 0; t1 < 8; t1++)
        {
            if (temp & 0x80)
            {
                lcd_draw_point(x, y, color);            /* 画需要显示的点 */
            }
            else if (mode == 0)     /* 如果非叠加模式, 不需要显示的点,用背景色填充 */
            {
                lcd_draw_point(x, y, g_back_color);  /* 填充背景色 */
            }

            temp <<= 1;
            y++;
            if ((y - y0) == size)
            {
                y = y0;
                x++;
                break;
            }
        }
    }
    myfree(SRAMIN, dzk);    /* 释放内存 */
}
```

汉字显示函数通过调用获取汉字点阵数据函数 text_get_hz_mat 就获取到点阵数据，使用 lcd 画点函数把点阵数据中“1”的点都画出来，最终会 LCD 显示你所要表示的汉字。

其他函数就不多讲解，大家可以自行消化。

2. main.c 代码

main.c 代码如下：

```
int main(void)
{
    uint32_t fontcnt;
    uint8_t i, j;
    uint8_t fontx[2];   /* GBK 码 */
    uint8_t key, t;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
```

```
    key_init();                             /* 初始化按键 */
    norflash_init();                        /* 初始化 NOR FLASH */
    my_mem_init(SRAMIN);                    /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                    /* 初始化 SRAMEX 内存池 */
    my_mem_init(SRAMDTCM);                  /* 初始化 DTCM 内存池(DTCM) */
    exfuns_init();                          /* 为 fatfs 相关变量申请内存 */
    f_mount(fs[0], "0:", 1);                /* 挂载 SD 卡 */
    f_mount(fs[1], "1:", 1);                /* 挂载 SPI FLASH */
    f_mount(fs[2], "2:", 1);                /* 挂载 NAND FLASH */

    while (fonts_init())                    /* 检查字库 */
    {
    UPD:
        lcd_clear(WHITE);                   /* 清屏 */
        lcd_show_string(30, 30, 200, 16, 16, "STM32", RED);

        while (sd_init())                   /* 检测 SD 卡 */
        {
            lcd_show_string(30, 50, 200, 16, 16, "SD Card Failed!", RED);
            delay_ms(200);
            lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
            delay_ms(200);
        }
        lcd_show_string(30, 50, 200, 16, 16, "SD Card OK", RED);
        lcd_show_string(30, 70, 200, 16, 16, "Font Updating...", RED);
        key = fonts_update_font(20, 90, 16, (uint8_t *)"0:", RED); /* 更新字库 */

        while (key)    /* 更新失败 */
        {
            lcd_show_string(30, 90, 200, 16, 16, "Font Update Failed!", RED);
            delay_ms(200);
            lcd_fill(20, 90, 200 + 20, 90 + 16, WHITE);
            delay_ms(200);
        }

        lcd_show_string(30, 90, 200, 16, 16, "Font Update Success!   ", RED);
        delay_ms(1500);
        lcd_clear(WHITE);/* 清屏 */
    }

    text_show_string(30, 30, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
    text_show_string(30, 50, 200, 16, "GBK 字库测试程序", 16, 0, RED);
    text_show_string(30, 70, 200, 16, "ATOM@ALIENTEK", 16, 0, RED);
    text_show_string(30, 90, 200, 16, "按 KEY0,更新字库", 16, 0, RED);

    text_show_string(30, 110, 200, 16, "内码高字节:", 16, 0, BLUE);
    text_show_string(30, 130, 200, 16, "内码低字节:", 16, 0, BLUE);
    text_show_string(30, 150, 200, 16, "汉字计数器:", 16, 0, BLUE);

    text_show_string(30, 200, 200, 32, "对应汉字为:", 32, 0, BLUE);
    text_show_string(30, 232, 200, 24, "对应汉字为:", 24, 0, BLUE);
    text_show_string(30, 256, 200, 16, "对应汉字(16*16)为:", 16, 0, BLUE);
    text_show_string(30, 272, 200, 12, "对应汉字(12*12)为:", 12, 0, BLUE);
    while (1)
    {
        fontcnt = 0;
        for (i = 0x81; i < 0xff; i++)       /* GBK 内码高字节范围为 0X81~0XFE */
        {
            fontx[0] = i;
            lcd_show_num(118, 110, i, 3, 16, BLUE);  /* 显示内码高字节 */
            /* GBK 内码低字节范围为 0X40~0X7E, 0X80~0XFE) */
            for (j = 0x40; j < 0xfe; j++)
```

```
            {
                if (j == 0x7f)continue;
                fontcnt++;
                lcd_show_num(118, 130, j, 3, 16, BLUE);        /* 显示内码低字节 */
                lcd_show_num(118, 150, fontcnt, 5, 16, BLUE); /* 汉字计数显示 */
                fontx[1] = j;
                text_show_font(30 + 176, 200, fontx, 32, 0, BLUE);
                text_show_font(30 + 132, 232, fontx, 24, 0, BLUE);
                text_show_font(30 + 144, 256, fontx, 16, 0, BLUE);
                text_show_font(30 + 108, 272, fontx, 12, 0, BLUE);
                t = 200;

                while (t--)    /* 延时,同时扫描按键 */
                {
                    delay_ms(1);
                    key = key_scan(0);

                    if (key == KEY0_PRES)
                    {
                        goto UPD;    /* 跳转到 UPD 位置(强制更新字库) */
                    }
                }
                LED0_TOGGLE();
            }
        }
    }
}
```

main 函数实现了我们在硬件设计例程功能所表述的一致，至此整个软件设计就完成了。

56.4 下载验证

本例程支持 12*12、16*16、24*24 和 32*32 四种字体的显示，将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 开始显示四种大小的汉字及内码如图 56.4.1 所示：

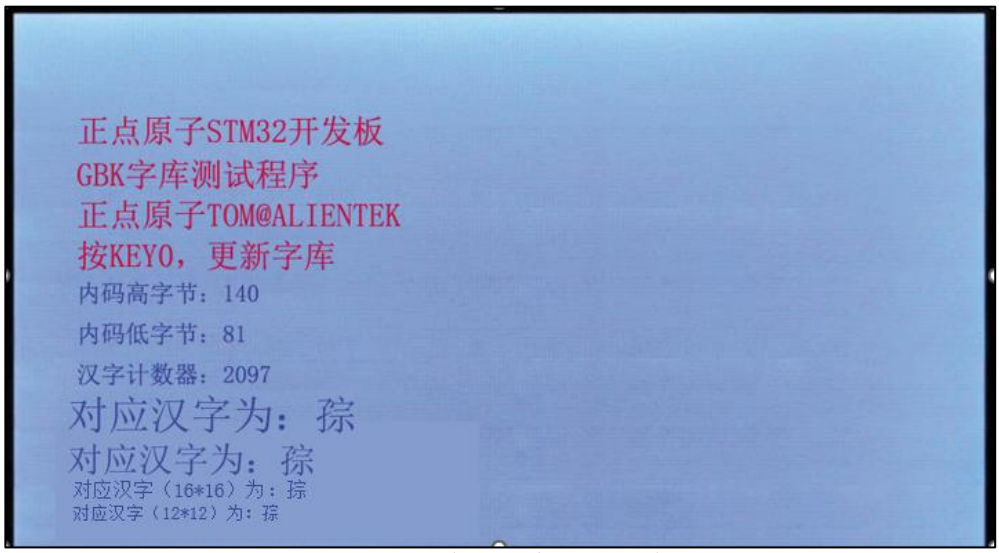


图 56.4.1 汉字显示实验显示效果

一开始就显示汉字，是因为板子在出厂的时候都是测试过的，里面刷了综合测试程序，已经把字库写入到 NORFLASH 里面，所以并不会提示更新字库。如果你想要更新字库，就需要先找一张 SD 卡，把 A 盘资料\5，SD 卡根目录文件 文件夹下面的 SYSTEM 文件夹拷贝到 SD 卡根目录下，插入开发板，并按复位，之后，在显示汉字的时候，按下 KEY0，就可以开始更新字库。字库更新界面如图 56.4.2 所示：

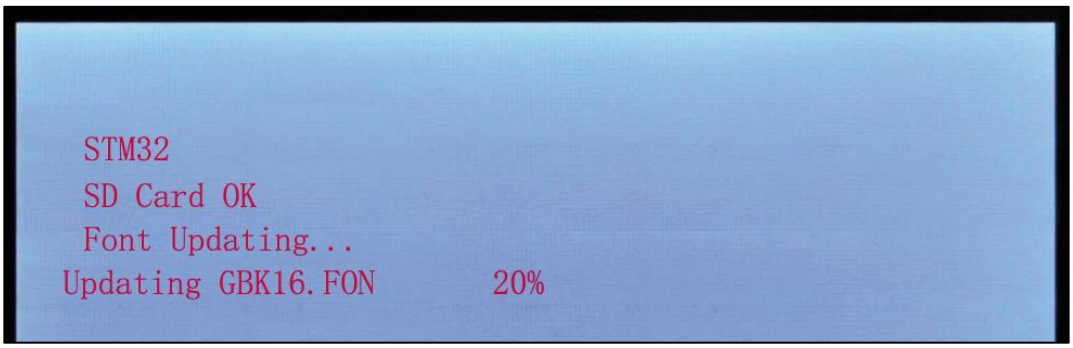


图 56.4.2 汉字字库更新界面

此外我们还可以使用 USMART 来测试该实验。通过 USMART 调用 text_show_string 或者 text_show_string_middle 来实现任意位置显示任何字符串，有兴趣的朋友可以尝试一下。

第五十七章 图片显示实验

在开发产品的时候，很多时候，我们都会用到图片解码，在本章中，我们将向大家介绍如何通过 STM32H7 来解码 BMP/JPG/JPEG/GIF 等图片，并在 LCD 上显示出来。

本章分为如下几个部分：

57.1 图片格式简介
57.2 硬件设计
57.3 程序设计
57.4 下载验证

57.1 图片格式介绍

我们常用的图片格式有很多，一般最常用的有三种：JPEG（或 JPG）、BMP 和 GIF。其中 JPEG（或 JPG）和 BMP 是静态图片，而 GIF 则是可以实现动态图片。下面，我们简单介绍一下这三种图片格式。

57.1.1 BMP 编码简介

我们常用的图片格式有很多，一般最常用的有三种：JPEG（或 JPG）、BMP 和 GIF。其中 JPEG（或 JPG）和 BMP 是静态图片，而 GIF 则是可以实现动态图片。下面，我们简单介绍一下这三种图片格式。

首先，我们来看看 BMP 图片格式。BMP（全称 Bitmap）是 Window 操作系统中的标准图像文件格式，文件后缀名为“.bmp”，使用非常广。它采用位映射存储格式，除了图像深度可选以外，不采用其他任何压缩，因此，BMP 文件所占用的空间很大，但是没有失真。BMP 文件的图像深度可选 1bit、4bit、8bit、16bit、24bit 及 32bit。BMP 文件存储数据时，图像的扫描方式是按从左到右、从下到上的顺序。

典型的 BMP 图像文件由四部分组成：

1、位图头文件数据结构，它包含 BMP 图像文件的类型、显示内容等信息；

2、位图信息数据结构，它包含有 BMP 图像的宽、高、压缩方法，以及定义颜色等信息；

3、调色板，这个部分是可选的，有些位图需要调色板，有些位图，比如真彩色图（24 位的 BMP）就不需要调色板；

4、位图数据，这部分的内容根据 BMP 位图使用的位数不同而不同，在 24 位图中直接使用 RGB，而其他的小于 24 位的使用调色板中颜色索引值。

关于 BMP 的详细介绍，请参考光盘的《BMP 图片文件详解.pdf》。

57.1.2 JPEG 编码简介

JPEG 是 Joint Photographic Experts Group(联合图像专家组)的缩写，文件后辍名为“.jpg”或“.jpeg”，是最常用的图像文件格式，由一个软件开发联合会组织制定，同 BMP 格式不同，JPEG 是一种有损压缩格式，能够将图像压缩在很小的储存空间，图像中重复或不重要的资料会被丢失，因此容易造成图像数据的损伤（BMP 不会，但是 BMP 占用空间大）。尤其是使用过高的压缩比例，将使最终解压缩后恢复的图像质量明显降低，如果追求高品质图像，不宜采用过高压缩比例。但是 JPEG 压缩技术十分先进，它用有损压缩方式去除冗余的图像数据，在获得极高的压缩率的同时能展现十分丰富生动的图像，换句话说，就是可以用最少的磁盘空间得到较好的图像品质。而且 JPEG 是一种很灵活的格式，具有调节图像质量的功能，允许用不同的压缩比例对文件进行压缩，支持多种压缩级别，压缩比率通常在 10:1 到 40:1 之间，压缩比越大，品质就越低；相反地，压缩比越小，品质就越好。比如可以把 1.37Mb 的 BMP 位图文件压缩至 20.3KB。当然也可以在图像质量和文件尺寸之间找到平衡点。JPEG 格式压缩的主要是高频信息，对色彩的信息保留较好，适合应用于互联网，可减少图像的传输时间，可以支持

24bit 真彩色，也普遍应用于需要连续色调的图像。

JPEG/JPG 的解码过程可以简单的概述为如下几个部分：

1、从文件头读出文件的相关信息。

JPEG 文件数据分为文件头和图像数据两大部分，其中文件头记录了图像的版本、长宽、采样因子、量化表、哈夫曼表等重要信息。所以解码前必须将文件头信息读出，以备图像数据解码过程之用。

2、从图像数据流读取一个最小编码单元(MCU)，并提取出里边的各个颜色分量单元。

3、将颜色分量单元从数据流恢复成矩阵数据。

使用文件头给出的哈夫曼表，对分割出来的颜色分量单元进行解码，把其恢复成 8×8 的数据矩阵。

4、8×8 的数据矩阵进一步解码。

此部分解码工作以 8×8 的数据矩阵为单位， 其中包括相邻矩阵的直流系数差分解码、使用文件头给出的量化表反量化数据、反 Zig- zag 编码、隔行正负纠正、反向离散余弦变换等 5 个步骤， 最终输出仍然是一个 8×8 的数据矩阵。

5、颜色系统 YCrCb 向 RGB 转换。

将一个 MCU 的各个颜色分量单元解码结果整合起来，将图像颜色系统从 YCrCb 向 RGB 转换。

6、排列整合各个 MCU 的解码数据。

不断读取数据流中的 MCU 并对其解码，直至读完所有 MCU 为止，将各 MCU 解码后的数据正确排列成完整的图像。

JPEG 的解码本身是比较复杂的，这里 FATFS 的作者，提供了一个轻量级的 JPG/JPEG 解码库：TjpgDec，最少仅需 3KB 的 RAM 和 3.5KB 的 FLASH 即可实现 JPG/JPEG 解码，本例程采用 TjpgDec 作为 JPG/JPEG 的解码库，关于 TjpgDec 的详细使用，请参考光盘：6，软件资料\图片编解码\TjpgDec 技术手册这个文档。

BMP 和 JPEG 这两种图片格式均不支持动态效果，而 GIF 则是可以支持动态效果。最后，我们来看看 GIF 图片格式。

57.1.3 GIF 编码简介

GIF(Graphics Interchange Format)是 CompuServe 公司开发的图像文件存储格式，1987 年开发的 GIF 文件格式版本号是 GIF87a，1989 年进行了扩充，扩充后的版本号定义为 GIF89a。GIF 图像文件以数据块(block)为单位来存储图像的相关信息。一个 GIF 文件由表示图形/图像的数据块、数据子块以及显示图形/图像的控制信息块组成，称为 GIF 数据流(DataStream)。数据流中的所有控制信息块和数据块都必须在文件头(Header)和文件结束块(Trailer)之间。

GIF 文件格式采用了 LZW(Lempel-ZivWalch)压缩算法来存储图像数据，定义了允许用户为图像设置背景的透明(transparency)属性。此外，GIF 文件格式可在一个文件中存放多幅彩色图形/图像。如果在 GIF 文件中存放有多幅图，它们可以像演幻灯片那样显示或者像动画那样演示。

一个 GIF 文件的结构可分为文件头(FileHeader)、GIF 数据流(GIFDataStream)和文件终结器(Trailer)三个部分。文件头包含 GIF 文件署名(Signature)和版本号(Version)；GIF 数据流由控制标识符、图象块(ImageBlock)和其他的一些扩展块组成；文件终结器只有一个值为 0x3B 的字符（';'）表示文件结束。

关于 GIF 的详细介绍，请参考光盘 GIF 解码相关资料。图片格式简介，我们就介绍到这里。

57.2 硬件设计

1. 例程功能

开机的时候先检测字库，然后检测 SD 卡是否存在，如果 SD 卡存在，则开始查找 SD 卡根目录下的 PICTURE 文件夹，如果找到则显示该文件夹下面的图片文件（支持 bmp、jpg、jpeg

或 gif 格式），循环显示，通过按 KEY0 和 KEY1 可以快速浏览下一张和上一张，KEY_UP 按键用于暂停/继续播放，LED1 用于指示当前是否处于暂停状态。如果未找到 PICTURE 文件夹/任何图片文件，则提示错误。

还可以通过 USMART 调用 piclib_ai_load_picfile 和 minibmp_decode 解码任意指定路径的图片。

LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
LED1：LED1 - PB0
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）独立按键：KEY0 - PH3、KEY1 - PH2、WK_UP - PA0
5）SD 卡，通过 SDMMC1(SDMMC_D0~D4(PC8~PC11)，SDMMC_SCK(PC12)，SDMMC_CMD(PD2))连接
6）norflash(QSPI FLASH 芯片，连接在 QSPI 上)

57.3 程序设计

57.3.1 程序流程图

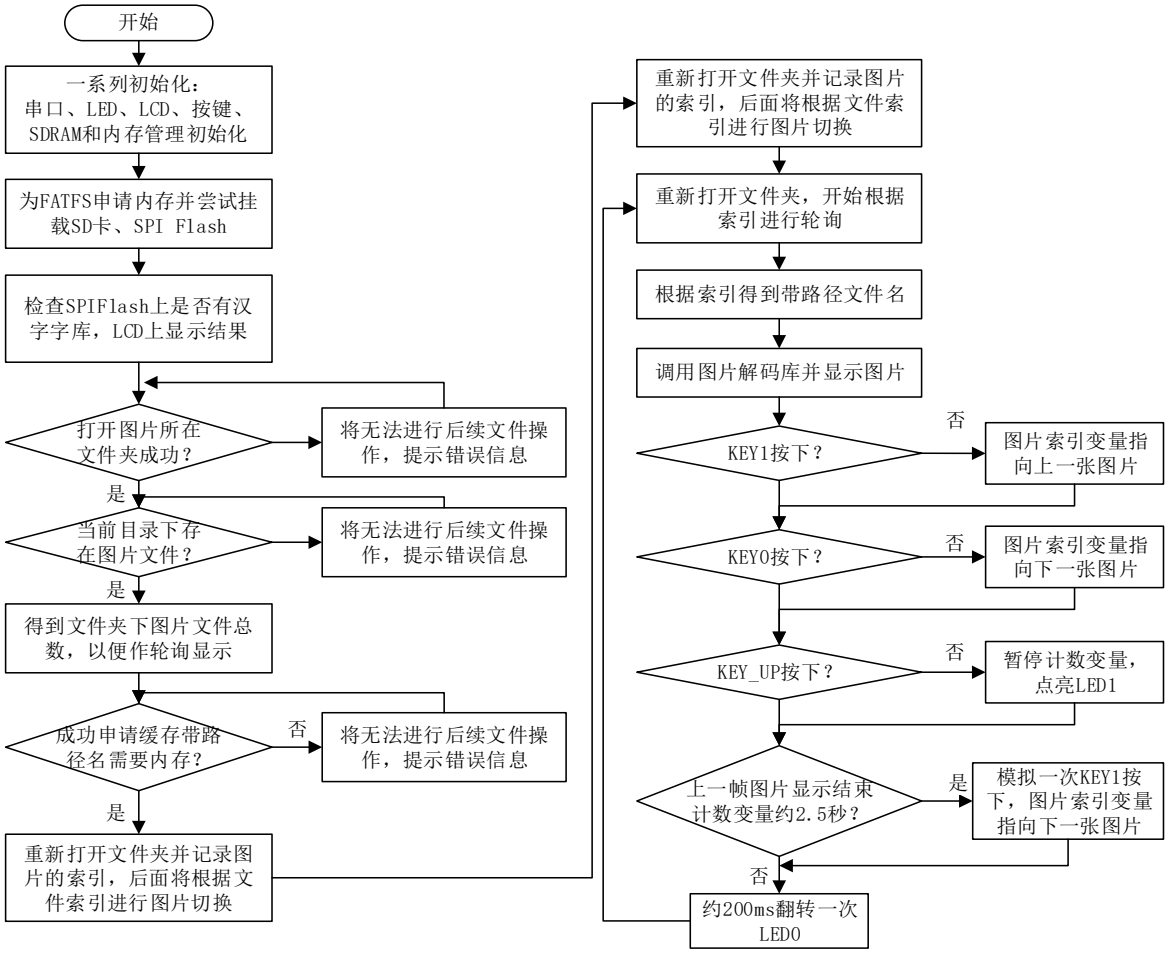


图 57.3.1.1 图片显示实验程序流程图

本程序主要靠文件操作，打开指定位置的图片并调用图片解码库解码后显示不同格式的图片。我们加入了一些交互信息，以控制图片的显示切换等。

57.3.2 程序解析

1. PICTURE 代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。PICTURE 驱动源码包括八个文件：bmp.c、bmp.h、tjpgd.c、tjpgd.h、gif.c、gif.h、piclib.c 和 piclib.h。

其中：

bmp.c 和 bmp.h 用于实现对 bmp 文件的解码；

tjpgd.c 和 tjpgd.h 用于实现对 jpeg/jpg 文件的解码；

gif.c 和 gif.h 用于实现对 gif 文件的解码；

这几个代码太长了，而且也有规定的标准，需要结合各个图片编码的格式来编写，所以我们在这里不贴出来，大家查看光盘中的源码的实现过程即可。下面我们重点讲解这几个解码库对应到我们的 LCD 的显示部分。

1）解码库的控制句柄_pic_phy 和_pic_info

我们使用这个接口，把解码后的图形数据与 LCD 的实际操作对应起来。为了方便去显示图片，我们需要将图片的信息与我们的 LCD 联系上。这里我们定义了_pic_phy 和_pic_info 分别用于定义图片解码库的 LCD 操作和存放解码后的图片尺寸颜色信息。它们的定义如下：

```
/* 在移植的时候,必须由用户自己实现这几个函数 */
typedef struct
{
    /* 读点函数 */
    uint32_t(*read_point)(uint16_t, uint16_t);
    /* 画点函数 */
    void(*draw_point)(uint16_t, uint16_t, uint32_t);
    /* 单色填充函数 */
    void(*fill)(uint16_t, uint16_t, uint16_t, uint16_t, uint32_t);
    /* 画水平线函数 */
    void(*draw_hline)(uint16_t, uint16_t, uint16_t, uint16_t);
    /* 颜色填充 */
    void(*fillcolor)(uint16_t, uint16_t, uint16_t, uint16_t, uint16_t *);
} _pic_phy;

/* 图像信息 */
typedef struct
{
    uint16_t lcdwidth;          /* LCD 的宽度 */
    uint16_t lcdheight;         /* LCD 的高度 */
    uint32_t ImgWidth;          /* 图像的实际宽度和高度 */
    uint32_t ImgHeight;
    uint32_t Div_Fac;           /* 缩放系数 (扩大了 8192 倍的) */
    uint32_t S_Height;          /* 设定的高度和宽度 */
    uint32_t S_Width;
    uint32_t S_XOFF;            /* x 轴和 y 轴的偏移量 */
    uint32_t S_YOFF;
    uint32_t staticx;           /* 当前显示到的 x y 坐标 */
    uint32_t staticy;
} _pic_info;
```

在 piclib.c 文件中，我们用上述类型定义了两个结构体，具体如下：

```
_pic_info picinfo;      /* 图片信息 */
_pic_phy pic_phy;       /* 图片显示物理接口 */
```

2）piclib_init 函数

piclib_init 函数，该函数用于初始化图片解码的相关信息，用于定义解码后的 LCD 操作。具体定义如下：

```
/**
```

```
 * @brief       画图初始化
 * @note        在画图之前,必须先调用此函数, 指定相关函数
 * @param       无
 * @retval      无
 */
void piclib_init(void)
{
    pic_phy.read_point = lcd_read_point;        /* 读点函数实现,仅 BMP 需要 */
    pic_phy.draw_point = lcd_draw_point;        /* 画点函数实现 */
    pic_phy.fill = lcd_fill;                    /* 填充函数实现,仅 GIF 需要 */
    pic_phy.draw_hline = lcd_draw_hline;        /* 画线函数实现,仅 GIF 需要 */
    pic_phy.fillcolor = piclib_fill_color;      /* 颜色填充函数实现,仅 TJPGD 需要 */
    picinfo.lcdwidth = lcddev.width;            /* 得到 LCD 的宽度像素 */
    picinfo.lcdheight = lcddev.height;          /* 得到 LCD 的高度像素 */
    picinfo.ImgWidth = 0;                       /* 初始化宽度为 0 */
    picinfo.ImgHeight = 0;                      /* 初始化高度为 0 */
    picinfo.Div_Fac = 0;                        /* 初始化缩放系数为 0 */
    picinfo.S_Height = 0;                       /* 初始化设定的高度为 0 */
    picinfo.S_Width = 0;                        /* 初始化设定的宽度为 0 */
    picinfo.S_XOFF = 0;                         /* 初始化 x 轴的偏移量为 0 */
    picinfo.S_YOFF = 0;                         /* 初始化 y 轴的偏移量为 0 */
    picinfo.staticx = 0;                        /* 初始化当前显示到的 x 坐标为 0 */
    picinfo.staticy = 0;                        /* 初始化当前显示到的 y 坐标为 0 */
}
```

- **函数描述：**

初始化图片解码的相关信息，这些函数必须由用户在外部实现。我们使用之前 LCD 的操作函数对这个结构体中的绘制操作：画点、画线、画圆等定义与我们的 LCD 操作对应起来。具体这些操作可以查看 TFT LCD 一节的描述。

- **函数形参：**

无。

- **函数返回值：**

无。

3）piclib_alpha_blend 函数

RGB 色彩中，一个标准像素由 32 位组成：透明度(8bit)+R(8bit)+B(8bit)+B(8bit)，8 位的 α 通道(alpha channel)位表示该像素如何产生特技效果，即通常我们说的半透明。alpha 的取值一般为 0 到 255。为 0 时，表示是全透明的，即图片是看不见的。为 255 时，表示图片是显示原始图的。中间值即为半透明状态。计算 alpha blending 时，通常的方法是将源像素的 RGB 值，分别与目标像素（如背景）的 RGB 按比例混合，最后得到一个混合后的 RGB 值。函数定义如下：

```
/**
 * @brief       快速 ALPHA BLENDING 算法
 * @param       src             : 颜色数
 * @param       dst             : 目标颜色
 * @param       alpha           : 透明程度(0~32)
 * @retval      混合后的颜色
 */
uint16_t piclib_alpha_blend(uint16_t src, uint16_t dst, uint8_t alpha)
{
    uint32_t src2;
    uint32_t dst2;
    /* Convert to 32bit |-----GGGGGG-----RRRRR------BBBBB| */
    src2 = ((src << 16) | src) & 0x07E0F81F;
    dst2 = ((dst << 16) | dst) & 0x07E0F81F;
    dst2 = ((((dst2 - src2) * alpha) >> 5) + src2) & 0x07E0F81F;
    return (dst2 >> 16) | dst2;
}
```

- **函数描述：**

piclib_alpha_blend 函数，该函数用于实现半透明效果，在小格式（图片分辨率小于 LCD 分辨率）bmp 解码的时候，可能被用到。

- **函数形参：**

  形参 1 是为 RGB 色彩编号，这里我们使用的是 RGB565 模式，故只有 16 位；

  形参 2 是目标象素，使用时我们一般指背景颜色。

  形参 3 是透明度：有效范围为 0~255，0 表示全透明，255 表不透明。

- **函数返回值：**

  返回计算后的透明度颜色数值。

4）piclib_ai_draw_init 函数

对于给定区域，为了显示更好看，一般会选择图片居中显示，此函数实现此功能，把图片在显示区域中居中。函数定义如下：

```
/**
 * @brief       初始化智能画点
 * @param       无
 * @retval      无
 */
void piclib_ai_draw_init(void)
{
    float temp, temp1;
    temp = (float)picinfo.S_Width / picinfo.ImgWidth;
    temp1 = (float)picinfo.S_Height / picinfo.ImgHeight;
    if (temp < temp1)temp1 = temp;  /* 取较小的那个 */
    if (temp1 > 1)temp1 = 1;
    /* 使图片处于所给区域的中间 */
    picinfo.S_XOFF += (picinfo.S_Width - temp1 * picinfo.ImgWidth) / 2;
    picinfo.S_YOFF += (picinfo.S_Height - temp1 * picinfo.ImgHeight) / 2;
    temp1 *= 8192;  /* 扩大 8192 倍 */
    picinfo.Div_Fac = temp1;
    picinfo.staticx = 0xffff;
    picinfo.staticy = 0xffff;    /* 放到一个不可能的值上面 */
}
```

- **函数描述：**

  piclib_ai_draw_init 函数，该函数使解码后的图片信息处于所给的区域的中间。

- **函数形参：**

  无。

- **函数返回值：**

  无。我们可以在显示实例中测试加与不加此函数的显示效果差异。

5）piclib_is_element_ok 函数

对于给定区域，为了显示更好看，一般会选择图片居中显示，此函数实现此功能，把图片在显示区域中居中。函数定义如下：

```
/**
 * @brief       判断这个像素是否可以显示
 * @param       x, y          : 像素原始坐标
 * @param       chg           : 功能变量
 * @param       无
 * @retval      操作结果
 *   @arg       0   , 不需要显示
 *   @arg       1   , 需要显示
 */
__inline uint8_t piclib_is_element_ok(uint16_t x, uint16_t y, uint8_t chg)
{
    if (x != picinfo.staticx || y != picinfo.staticy)
    {
        if (chg == 1)
        {
            picinfo.staticx = x;
            picinfo.staticy = y;
        }
```

```
        return 1;
    }
    else
    {
        return 0;
    }
}
```

- **函数描述：**

piclib_is_element_ok 函数，该函数用于判断一个点是不是应该显示出来，在图片缩放的时候该函数是必须用到的。这里用__inline 修饰，保证该部分的代码不被优化。

- **函数形参：**

无。

- **函数返回值：**

1：需要显示。

0：不需要显示。

其它函数使用到时，根据此返回值进行判定显示操作。

6）piclib_ai_load_picfile 函数

piclib_ai_load_picfile 帮助我们得到需要显示的图片信息并助于下一步的绘制。本函数需要结合文件系统来操作，图片根据后缀区分并且在文件夹在保存是我们在 PC 下的习分类，也是我们处理和分类图片的最方便的方式。

```
/**
 * @brief       智能画图
 * @note        图片仅在 x,y 和 width, height 限定的区域内显示.
 *
 * @param       filename      : 包含路径的文件名(.bmp/.jpg/.jpeg/.gif 等)
 * @param       x, y          : 起始坐标
 * @param       width, height : 显示区域
 * @param       fast          : 使能快速解码
 *   @arg                       0, 不使能
 *   @arg                       1, 使能
 *   @note                      图片尺寸小于等于液晶分辨率,才支持快速解码
 * @retval      res           : 操作结果 0，成功 其他，错误码
 */
uint8_t piclib_ai_load_picfile(char *filename, uint16_t x, uint16_t y,
                               uint16_t width, uint16_t height, uint8_t fast)
{
    uint8_t res;/* 返回值 */
    uint8_t temp;
    if((x + width) > picinfo.lcdwidth)return PIC_WINDOW_ERR;  /*x 坐标超范围了 */
    if((y + height) > picinfo.lcdheight)return PIC_WINDOW_ERR;/*y 坐标超范围了*/
    /* 得到显示方框大小 */
    if (width == 0 || height == 0)return PIC_WINDOW_ERR;      /* 窗口设定错误 */
    picinfo.S_Height = height;
    picinfo.S_Width = width;
    /* 显示区域无效 */
    if (picinfo.S_Height == 0 || picinfo.S_Width == 0)
    {
        picinfo.S_Height = lcddev.height;
        picinfo.S_Width = lcddev.width;
        return FALSE;
    }
    if (pic_phy.fillcolor == NULL)fast = 0; /* 颜色填充函数未实现,不能快速显示 */
    /* 显示的开始坐标点 */
    picinfo.S_YOFF = y;
    picinfo.S_XOFF = x;
    /* 文件名传递 */
    temp = exfuns_file_type((uint8_t *)filename);   /* 得到文件的类型 */
    switch (temp)
    {
```

```
        case T_BMP:
            res = stdbmp_decode(filename);       /* 解码 bmp */
            break;
        case T_JPG:
        case T_JPEG:
            res = jpg_decode(filename, fast);    /* 解码 JPG/JPEG */
            break;
        case T_GIF:
            res = gif_decode(filename, x, y, width, height);     /* 解码 gif */
            break;
        default:
            res = PIC_FORMAT_ERR;                /* 非图片格式!!! */
            break;
    }
    return res;
}
```

- **函数描述：**

piclib_ai_load_picfile 函数，整个图片显示的对外接口，外部程序，通过调用该函数，可以实现 bmp、jpg/jpeg 和 gif 的显示，该函数根据输入文件的后缀名，判断文件格式，然后交给相应的解码程序（bmp 解码/jpeg 解码/gif 解码），执行解码，完成图片显示。

- **函数形参：**

形参 1 filename 是文件的路径名，具体可以参考 FATFS 一节的描述，为字符口，我们的例程采用的是 SD 卡存图片，故一般为”0:/PICTURE/*.GIF”等类似格式。

形参 2 为画图的起始 x 坐标；

形参 3 为画图的起始 y 坐标；

形参 4 的 width 和**形参 5** 的 height 形成了以 x、y 为起点的(x,y)~(x+width,y+height)的矩形显示区域，对屏幕坐标不理解的可以参考我们的 TFT LCD 一节的描述。

形参 6 根据我们的 LCD 进行适应的一个快速解码的操作，仅 jgp/jpeg 模式下有效。

这里用到的 exfuns_file_type()函数是我们前面 FATFS 一节提到的 FATFS 扩展应用，我们用这个函数来判断文件类型，方便我们进行程序设计。这部分参考文件系统下 exfuns 文件夹下的相关文件。

- **函数返回值：**

0：成功　　其他：错误码

由于图片显示需要用到大内存，我们使用动态内存分配来实现，我们仍使用我们自定的内存管理函数来管理程序内存。申请内存函数 piclib_mem_malloc()和内存释放函数 piclib_mem_free()的实现就比较简单了，大家参考光盘的源码即可。

2. main.c 代码

main.c 函数我们利用 FATFS 的接口来操作和查找图片文件，我们在 microSD/SD 卡的根目录下新建一个 PICTURE 文件夹，然后放置我们准备要显示的 BMP、JPG、GIF 图片，接下来按我们程序流程图设计的思路，先扫描图像文件的数量并切换显示，并加入按键支持图片翻页，主要的代码如下所示：

```
int main(void)
{
    uint8_t res;
    DIR picdir;                 /* 图片目录 */
    FILINFO *picfileinfo;       /* 文件信息 */
    char *pname;                /* 带路径的文件名 */
    uint16_t totpicnum;         /* 图片文件总数 */
    uint16_t curindex;          /* 图片当前索引 */
    uint8_t key;                /* 键值 */
    uint8_t pause = 0;          /* 暂停标记 */
    uint8_t t;
    uint16_t temp;
    uint32_t *picoffsettbl;     /* 图片文件 offset 索引表 */
```

```
    sys_cache_enable();                         /* 打开 L1-Cache */
    HAL_Init();                                 /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);         /* 设置时钟, 400Mhz */
    delay_init(400);                            /* 延时初始化 */
    usart_init(115200);                         /* 串口初始化为 115200 */
    usmart_dev.init(200);                       /* 初始化 USMART */
    mpu_memory_protection();                    /* 保护相关存储区域 */
    led_init();                                 /* 初始化 LED */
    sdram_init();                               /* 初始化 SDRAM */
    lcd_init();                                 /* 初始化 LCD */
    key_init();                                 /* 初始化按键 */
    my_mem_init(SRAMIN);                        /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                        /* 初始化外部内存池(AXI) */
    my_mem_init(SRAM12);                        /* 初始化 SRAM12 内存池(SRAM1+SRAM2) */
    my_mem_init(SRAM4);                         /* 初始化 SRAM4 内存池(SRAM4) */
    my_mem_init(SRAMDTCM);                      /* 初始化 DTCM 内存池(DTCM) */
    my_mem_init(SRAMITCM);                      /* 初始化 ITCM 内存池(ITCM) */
    exfuns_init();                              /* 为 fatfs 相关变量申请内存 */
    f_mount(fs[0], "0:", 1);                    /* 挂载 SD 卡 */
    f_mount(fs[1], "1:", 1);                    /* 挂载 SPI FLASH */
    f_mount(fs[2], "2:", 1);                    /* 挂载 NAND FLASH */
    while (fonts_init())                        /* 检查字库 */
    {
        lcd_show_string(30, 50, 200, 16, 16, "Font Error!", RED);
        delay_ms(200);
        lcd_fill(30, 50, 240, 66, WHITE);     /* 清除显示 */
        delay_ms(200);
    }
    text_show_string(30, 50, 200, 16, "正点原子 STM32", 16, 0, RED);
    text_show_string(30, 70, 200, 16, "图片显示 实验", 16, 0, RED);
    text_show_string(30, 90, 200, 16, "ATOM@ALIENTEK ", 16, 0, RED);
    text_show_string(30, 110, 200, 16, "KEY0:NEXT KEY1:PREV ", 16, 0, RED);
    text_show_string(30, 130, 200, 16, "KEY_UP:PAUSE ", 16, 0, RED);

    while (f_opendir(&picdir, "0:/PICTURE")) /* 打开图片文件夹 */
    {
        text_show_string(30, 150, 240, 16, "PICTURE 文件夹错误!", 16, 0, RED);
        delay_ms(200);
        lcd_fill(30, 150, 240, 186, WHITE);  /* 清除显示 */
        delay_ms(200);
    }

    totpicnum = pic_get_tnum("0:/PICTURE");  /* 得到总有效文件数 */

    while (totpicnum == NULL)    /* 图片文件为 0 */
    {
        text_show_string(30, 150, 240, 16, "没有图片文件!", 16, 0, RED);
        delay_ms(200);
        lcd_fill(30, 150, 240, 186, WHITE);  /* 清除显示 */
        delay_ms(200);
    }

    picfileinfo = (FILINFO *)mymalloc(SRAMIN, sizeof(FILINFO)); /* 申请内存 */
    pname = mymalloc(SRAMIN, FF_MAX_LFN * 2 + 1);    /* 为带路径的文件名分配内存 */
    /* 申请 4*totpicnum 个字节的内存,用于存放图片索引 */
    picoffsettbl = mymalloc(SRAMIN, 4 * totpicnum);

    while (!picfileinfo || !pname || !picoffsettbl) /* 内存分配出错 */
    {
        text_show_string(30, 150, 240, 16, "内存分配失败!", 16, 0, RED);
```

```
        delay_ms(200);
        lcd_fill(30, 150, 240, 186, WHITE);           /* 清除显示 */
        delay_ms(200);
    }

    /* 记录索引 */
    res = f_opendir(&picdir, "0:/PICTURE");          /* 打开目录 */
    if (res == FR_OK)
    {
        curindex = 0;       /* 当前索引为 0 */

        while (1)           /* 全部查询一遍 */
        {
            temp = picdir.dptr; /* 记录当前 dptr 偏移 */
            res = f_readdir(&picdir, picfileinfo);  /* 读取目录下的一个文件 */
            /* 错误了/到末尾了,退出 */
            if (res != FR_OK || picfileinfo->fname[0] == 0)break;

            res = exfuns_file_type(picfileinfo->fname);
            if ((res & 0XF0) == 0X50)    /* 取高四位,看看是不是图片文件 */
            {
                picoffsettbl[curindex] = temp;  /* 记录索引 */
                curindex++;
            }
        }
    }
    text_show_string(30, 150, 240, 16, "开始显示...", 16, 0, RED);
    delay_ms(1500);
    piclib_init();  /* 初始化画图 */
    curindex = 0;   /* 从 0 开始显示 */
    res = f_opendir(&picdir, (const TCHAR *)"0:/PICTURE");  /* 打开目录 */
    while (res == FR_OK)    /* 打开成功 */
    {
        dir_sdi(&picdir, picoffsettbl[curindex]);       /* 改变当前目录索引 */
        res = f_readdir(&picdir, picfileinfo);          /* 读取目录下的一个文件 */
        /* 错误了/到末尾了,退出 */
        if (res != FR_OK || picfileinfo->fname[0] == 0)break;

        strcpy((char *)pname, "0:/PICTURE/");           /* 复制路径(目录) */
        /* 将文件名接在后面 */
        strcat((char *)pname, (const char *)picfileinfo->fname);
        lcd_clear(BLACK);
        /* 显示图片 */
        piclib_ai_load_picfile(pname, 0, 0, lcddev.width, lcddev.height, 1);
        /* 显示图片名字 */
        text_show_string(2, 2, lcddev.width, 16, (char*)pname, 16, 1, RED);
        t = 0;
        while (1)
        {
            key = key_scan(0);          /* 扫描按键 */
            if (t > 250)key = 1;        /* 模拟一次按下 KEY0 */
            if ((t % 20) == 0)
            {
                LED0_TOGGLE();          /* LED0 闪烁,提示程序正在运行. */
            }

            if (key == KEY1_PRES)       /* 上一张 */
            {
                if (curindex)
                {
                    curindex--;
                }
```

```
                else
                {
                    curindex = totpicnum - 1;
                }

                break;
            }
            else if (key == KEY0_PRES)  /* 下一张 */
            {
                curindex++;
                if(curindex >= totpicnum)curindex=0;/* 到末尾的时候,自动从头开始 */
                break;
            }
            else if (key == WKUP_PRES)
            {
                pause = !pause;
                LED1(!pause);       /* 暂停的时候 LED1 亮. */
            }
            if (pause == 0)t++;
            delay_ms(10);
        }
        res = 0;
    }

    myfree(SRAMIN, picfileinfo);        /* 释放内存 */
    myfree(SRAMIN, pname);              /* 释放内存 */
    myfree(SRAMIN, picoffsettbl);       /* 释放内存 */
}
```

可以看到整个设计思路是跟据图片解码库来设计的，piclib_ai_load_picfile()是这套代码的核心，其它的交互是围绕它和图片解码后的图片信息做的显示。大家再仔细对照光盘中的源码进一步了解整个设计思路。另外，我们的程序中只分配了 4 个文件索引，故更多数量的图片无法直接在本程序下演示，大家根据自己的需要再进行修改即可。

57.4 下载验证

将程序下载到开发板后，可以看到 LCD 开始显示图片（假设 SD 卡及文件都准备好了，即：在 SD 卡根目录新建：PICTURE 文件夹，并存放一些图片文件(.bmp/.jpg/.gif)在该文件夹内），如图 57.4.1 所示：

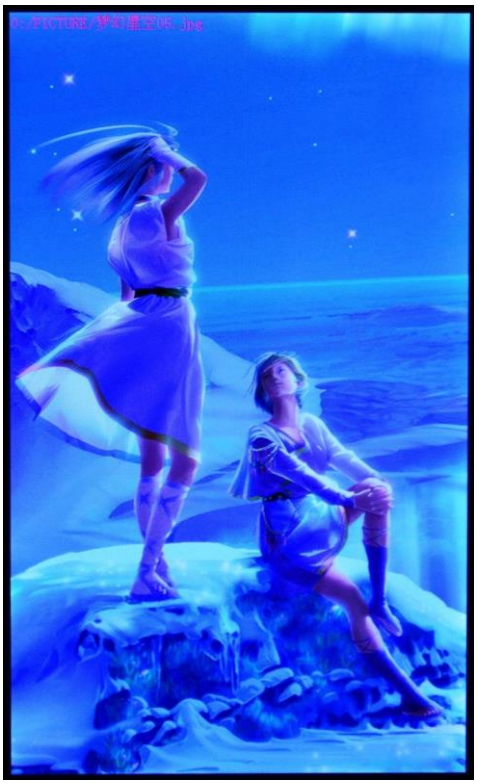


图 57.4.1 图片显示实验显示效果

按 KEY0 和 KEY1 可以快速切换到下一张或上一张，KEY_UP 按键可以暂停自动播放，同时 LED1 亮，指示处于暂停状态，再按一次 KEY_UP 则继续播放。同时，由于我们的代码支持 gif 格式的图片显示（注意尺寸不能超过 LCD 屏幕尺寸），所以可以放一些 gif 图片到 PICTURE 文件夹，来看动画了。

本章，同样可以通过 USMART 来测试该实验，将 piclib_ai_load_picfile 函数加入 USMART 控制（方法前面已经讲了很多次了），就可以通过串口调用该函数，在屏幕上任何区域显示任何你想要显示的图片了！同时，可以发送：runtime1，来开启 USMART 的函数执行时间统计功能，从而获取解码一张图片所需时间，方便验证。

注意：本例程在支持 AC6 时，jpeg 解码库中的函数容易被优化，故如果使用 AC6 进行本实验时，建议单独对其进行优化设置，MDK 也支持对单一文件进行优化等级设置，操作方法如图 57.4.2 所示：

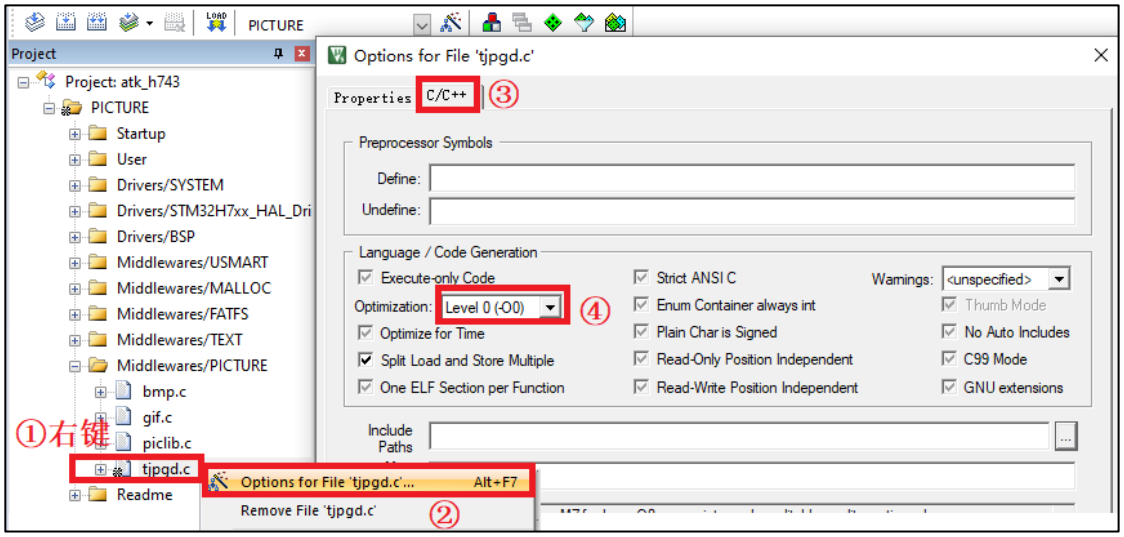


图 57.4.2 对 tjpgd.c 进行单独的优化设置

图片显示实验我们就讲解到这里。

第五十八章 硬件 JPEG 解码实验

上一章，我们学习了图片解码，学会了使用软件解码显示 bmp/jpg/jpeg/gif 等格式的图片，但是软件解码速度都比较慢，本章我们将学习如何使用 STM32H743 自带的硬件 JPEG 编解码器，实现对 JPG/JPEG 图片的硬解码，从而大大提高解码速度。

本章分为如下几个小节：

58.1 硬件 JPEG 编解码器简介
58.2 硬件设计
58.3 程序设计
58.4 下载验证

58.1 硬件 JPEG 编解码器简介

STM32H743 自带了硬件 JPEG 编解码器，可以实现快速 JPG/JPEG 编解码，本章我们仅使用 JPG/JPEG 解码器。STM32H7 的 JPEG 编解码器具有如下特点：

- 支持 JPEG 编码/解码
- 支持 RGB、YCbCr、YCMK 与 BW（灰度）色彩空间
- 单周期解码/编码一个像素
- 支持 JPEG 头数据编解码
- 多达 4 个可编程量化表
- 单周期哈夫曼表编解码
- 完全可编程的哈弗曼表（AC 和 DC 各 2 个）
- 完全可编程的最小编码单元（MCU）
- 单周期哈弗曼编码/解码

58.1.1 JPEG 编解码器框图

STM32H7 的 JPEG 编解码器框图如图 58.1.1.1 所示：

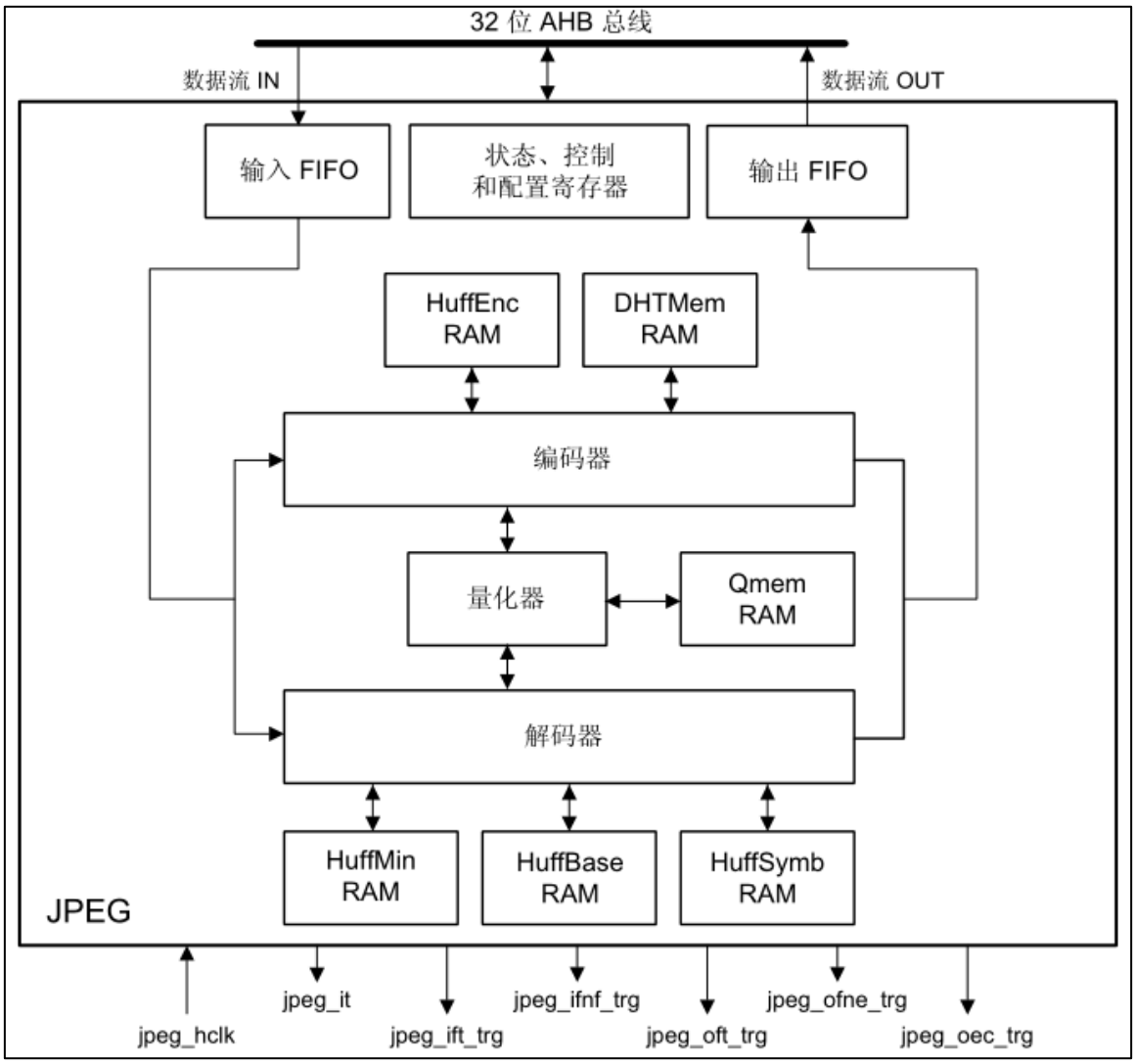


图 58.1.1.1 硬件 JPEG 编解码器框图

图中，硬件 JPEG 编解码器的输入和输出数据流都是通过 32 位 AHB 总线访问，其它的 JPEG 内部信号如下表所示：

信号名称	信号类型	说明
jpeg_hclk	数字输入	JPEG 内核和寄存器接口时钟
jpeg_it	数字输出	JPEG 全局中断
jpeg_ift_trg	数字输出	用于 MDMA 的 JPEG 输入 FIFO 阈值信号
jpeg_ifnf_trg	数字输出	用于 MDMA 的 JPEG 输入 FIFO 未满信号
jpeg_oft_trg	数字输出	用于 MDMA 的 JPEG 输出 FIFO 阈值信号
jpeg_ofne_trg	数字输出	用于 MDMA 的 JPEG 输出 FIFO 非空信号
jpeg_oec_trg	数字输出	用于 MDMA 的 JPEG 转换结束信号

表 58.1.1.1 JPEG 内部信号

58.1.2 JPEG 解码器功能

我们只需要对 JPEG 编解码器的相关寄存器进行设置，然后读写输入/输出 FIFO，即可完成 JPEG 的编解码。本章，我们只介绍如何利用 STM32H7 的硬件 JPEG 解码器实现对 JPG/JPEG 图片的解码。

硬件 JPEG 解码器，支持解码符合 ISO/IEC10918-1 协议规范的 JPEG 数据流，并且支持解码 JPEG 头（可配置），通过输入 FIFO 读取需要解码的 JPEG 数据，通过输出 FIFO 将解码完成

的 YUV 数据传输给外部。

注意：硬件 JPEG 解码器解码完成后是 YUV 格式的数据，并不是 RGB 格式的数据，所以不能直接显示到 LCD 上面，必须经过 YUV→RGB 的转换，才可以显示在 LCD 上面。

硬件 JPEG 解码时 FIFO 数据的处理（读取/写入）有两种方式：

1， 中断方式。

2， DMA 方式（JPEG 编解码内核使用的是 MDMA，下同！）。

为了达到最快的解码速度，我们一般使用 DMA 来处理 FIFO 数据。接下来，我们介绍一下硬件 JPEG 解码的数据处理过程。

1、输入 FIFO MDMA

通过 jpeg_ift_trg 信号（对应 MDMA 数据流 17，见《STM32H7xx 参考手册_V7（英文版）.pdf》第 597 页，Table 100），可以触发 JPEG 输入 FIFO 的 MDMA 请求，当输入 FIFO（总容量为 64 字节）至少半空的时候，将产生一个 MDMA 请求，读取 32 字节数据到输入 FIFO。当 MDMA 不再继续往 JPEG 输入 FIFO 传送数据时，JPEG 解码进程将自动暂停，因此，我们只需要控制 MDMA 的启停，就可以控制 JPEG 的解码进程，这个操作在 MDMA 传输完成，读取下一批 JPEG 数据的时候经常用到。

注意：在当前图片解码完成后，开启下一张图片解码之前，需要先停止 MDMA，然后对输入 FIFO 进行一次清空（设置 JPEG_CR 寄存器的 IFF 位），否则上一张图片的数据会影响到下一张图片的解码。

2、输出 FIFO MDMA

通过 jpeg_oft_trg 信号（对应 MDMA 数据流 19，见《STM32H7xx 参考手册_V7（英文版）.pdf》第 597 页，Table 100），可以触发 JPEG 输出 FIFO 的 MDMA，当输出 FIFO（总容量为 64 字节）至少半满的时候，将产生一个 MDMA 请求，可以从输出 FIFO 读取 32 字节数据。当 MDMA 不再读取 JPEG 输出 FIFO 的数据时，JPEG 输出 FIFO 被暂停，这个操作在 MDMA 传输完成，执行 YUV→RGB 转换的时候经常用到。

注意：当图片解码结束以后，输出 FIFO 里面可能还有数据，此时我们需要手动读取 FIFO 里面的数据，直到 JPEG_SR 寄存器的 OFNEF 位为 0。

3、JPEG 头解码

通过设置 JPEG_CONFR1 寄存器的 HDR 位为 1，可以使能 JPEG 头解码，通过设置 JPEG_CR 寄存器 HPDIE 位为 1，可以使能 JPEG 头解码完成中断。在完成 JPEG 头解码之后，我们可以获取当前 JPEG 图片的很多参数，包括：颜色空间、色度抽样、高度、宽度和 MCU 总数等信息。这些参数对我们后面的解码和颜色转换（YUV→RGB）非常重要。

硬件 JPEG 使用 DMA 实现 JPG/JPEG 图片解码的数据处理流程如图 58.1.2.1 所示：

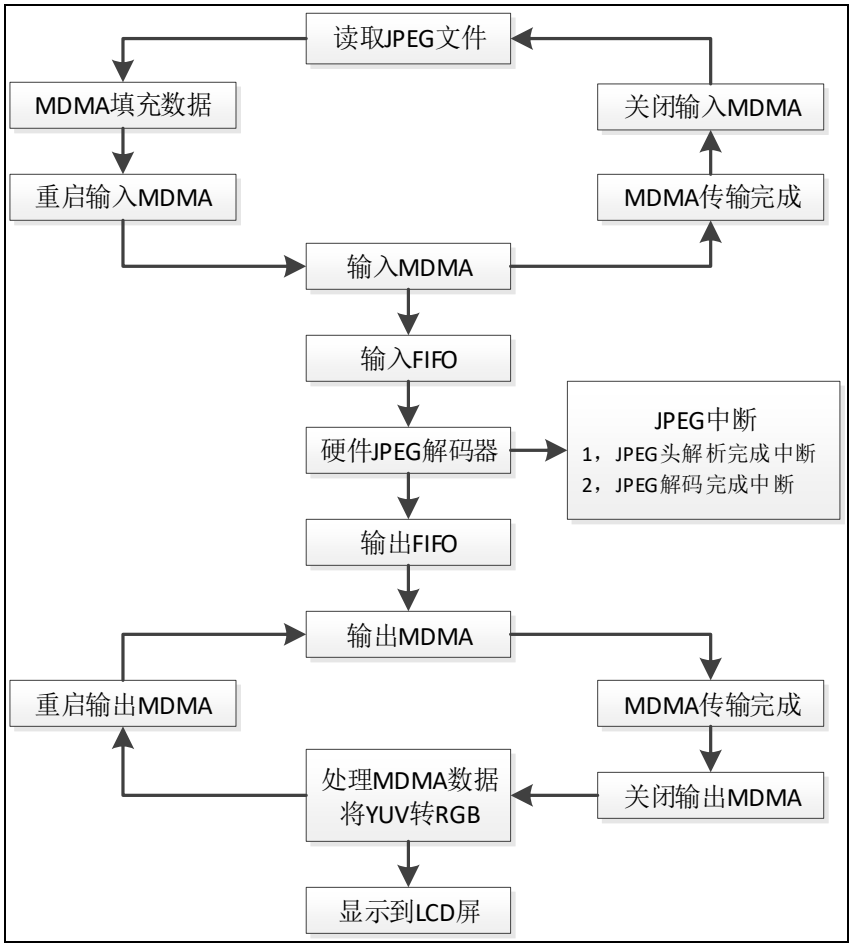


图 58.1.2.1 硬件 JPEG 解码数据处理流程（DMA 方式）

由图可知，数据处理主要由 2 个 MDMA 完成：输入 MDMA 和输出 MDMA，分别处理硬件 JPEG 的输入 FIFO 和输出 FIFO 的数据。通过适当控制输入 MDMA/输出 MDMA 的关闭和重启，从而控制整个数据处理的进程，关闭 MDMA 的时间越少，解码速度就越快。

图中我们还用到了 2 个 JPEG 中断：JPEG 头解析完成中断和 JPEG 解码完成中断，他们共用一个中断服务函数。JPEG 头解析完成中断，在 JPEG 头解码完成后进入，此时我们可以获取 JPG/JPEG 图片的很多重要信息，方便后续解码。JPEG 解码完成中断，在 JPG/JPEG 图片解码完成后进入，标志着整张图片解码完成。

58.1.3 MDMA 简介

MDMA 属于 STM32H7 内部 4 个 DMA 控制器之一，它的主要特点有：

- 支持 3 种传输方式：内存→内存、内存→外设、外设→内存
- 支持多达 16 个通道
- 支持多达 32 个硬件触发源，且支持软件触发
- 所有通道都可以独立设置并连接到 DMA1/DMA2 或相关外设
- 一个 256 级深度 buffer，被分成 2 个独立的 128 级 buffer（先入/先出，即 FIFO）
- 4 个优先级可设置：非常高、高、中、低
- 源和目标的传输位宽可以独立设置（字节、半字、字、双字）
- 源地址/目标地址自增和大小都可以独立设置

我们重点看一下 MDMA 和 DMA1/2 的差异（DMA1/DMA2 的详细介绍请参考第三十章 DMA 实验），MDMA 支持 4 种触发模式（通过 TRGM[1:0]位设置），具体如下：

1、缓冲传输模式（最多一次可以传输 128 字节）

2、块传输模式（最多一次可以传输 64K 字节）

3、重复块传输模式（每次传输后，可配置改变源/目标的起始地址）

4、链表模式（直到各个信道数据传输完毕，比较复杂，一般不用）

而 DMA1/DMA2 只支持单次传输和突发传输，且突发传输仅支持 4、8 和 16 等个传输长度，本实验我们用 MDMA 的缓冲传输模式，因为 JPEG 的输入/输出 FIFO 每次都是读取/输出 32 字节，因此我们用缓冲模式（TRGM[1:0]=00），并设置单次触发传输长度为 32 字节（TLEN[6:0]=32-1），就可以很好的和 JPEG 的输入/输出 FIFO 匹配了。

关于 MDMA 的具体配置，我们在这里就不详细介绍了，MDMA 的配置和 DMA1/DMA2 基本类似，不过稍微复杂一点。关于 MDMA 的详细介绍，请参考《STM32H7xx 参考手册_V7（英文版）.pdf》第 14 章。

58.1.4 YUV 转 RGB 操作

在 STM32F7 系列 MCU 上面，要实现 YUV 到 RGB 的转换，必须使用纯软件的方式来实现，比较耗时间。但是到了 H7 系列 MCU，ST 添加了硬件 YUV 到 RGB 转换的功能，我们通过 DMA2D 可以很方便的实现 YUV 到 RGB 的硬件转换。

H7 的 DMA2D 支持将：YCbCr 4:4:4（YUV444），4:2:2（YUV422）和 4:2:0（YUV420）等三种抽样格式的颜色数据转换成 RGB 数据格式。DMA2D 执行 YCbCr 数据格式转换的时候是以 8*8 的最小编码单元（MCU）为基础的，YCbCr 数据在 MCU 内的排列顺序如下表所示：

抽样方式	@	@ + 64	@ + 128	@+192	@+256	@ + 320
4:4:4	Y_1	Cb_1	Cr_1	Y_2	Cb_2	Cr_2
4:2:2	Y_1	Y_2	Cb_{12}	Cr_{12}	Y_3	Y_4
4:2:0	Y_1	Y_2	Y_3	Y_4	Cb_{1234}	Cr_{1234}

图 58.1.4.1 MCU 中不同抽样方式下 YCbCr 的排列方式

由上表可知：

YCbCr4:4:4（YUV444）抽样，每个 RGB 像素占 3 个 YCbCr 字节

YCbCr4:2:2（YUV422）抽样，每个 RGB 像素占 2 个 YCbCr 字节

YCbCr4:2:0（YUV420）抽样，每个 RGB 像素占 1.5 个 YCbCr 字节

为了保障 DMA2D 能够正确的执行 YCbCr 到 RGB 的转换，当图片采用：YCbCr 4:4:4 的抽样方式时，图片宽度（DMA2D_NLR 寄存器的 PL[13:0]定义）+行偏移（DMA2D_FGOR 寄存器的 LO[15:0]定义）必须是 8 的倍数。当图片采用：YCbCr 4:2:2 或 YCbCr 4:2:0 的抽样方式时，图片宽度（DMA2D_NLR 寄存器的 PL[13:0]定义）+行偏移（DMA2D_FGOR 寄存器的 LO[15:0]定义）必须是 16 的倍数。为了简化处理和节省内存，我们采用单幅图像，多次解码的方式来完成解码，因此规定：**对于硬件 JPEG 解码的图片，其宽度必须是 16 的倍数**。

DMA2D 执行内存到内存的 YCbCr→RGB 转换的简要设置（行偏移为 0）如下：

1，设置 DMA2D_CR 寄存器的 MODE[2:0]=001，DMA2D 工作在带 PFC 的存储器到存储器模式。

2，设置 DMA2D_OPFCCR 的 CM[2:0]=010，设置 PFC 输出图像格式为：RGB565。

3，设置 DMA2D_FGPFCCR 的 CSS[1:0]为正确的抽样方式（YCbCr 4:4:4/4:2:2/4:2:0，根据实际情况设置），并设置 CM[3:0]=1011，设置输入图像格式为：YCbCr。

4，设置 DMA2D_NLR 的 PL[13:0]为图像宽度（必须为 16 的倍数），NL[15:0]为单次转换输出的图像高度（YCbCr 4:4:4 和 YCbCr 4:2:2 每次输出 8 的倍数行，YCbCr 4:2:0 每次输出 16 的倍数行）。

5，设置 DMA2D_OMAR 的 MA[31:0]为输出图像数据的首地址。该地址需根据单次输出的图像高度和宽度，进行变化，以完成整幅图像的输出。

6，设置 DMA2D_FGMAR 的 MA[31:0]为输入图像数据的首地址。另外，单次输入图像数据的大小，是有要求的。输入图像是 YCbCr 数据格式，根据前面的介绍，不同抽样率下，最小输入图像数据的大小为：

单次输出行数*YCbCr 每个像素所占字节数*图像宽度

对于 YCbCr 4:4:4，最小输入图像数据大小为：8*3*图像宽度

对于 YCbCr 4:2:2，最小输入图像数据大小为：8*2*图像宽度

对于 YCbCr 4:2:0，最小输入图像数据大小为：16*1.5*图像宽度

7，最后，设置 DMA2D_CR 寄存器的 START 位为 1，使能 DMA2D 传输。然后等待传输完成就可以完成一次 YCbCr→RGB 的转换。

以上，就是利用 DMA2D 进行 YCbCr 到 RGB 图像数据转换的简要设置，DMA2D 的相关寄存器我们这里就不做介绍，详见《STM32H7xx 参考手册_V7（英文版）.pdf》第 18 章相关章节。

58.1.5 JPEG 编解码器寄存器

下面介绍本实验需要用到的 JPEG 编解码器寄存器。

- **JPEG 编解码器控制寄存器 0（JPEG_CONFR0）**

该寄存器仅最低位（START 位）有效，设置该位为 1，可以启动 JPEG 解码流程。通过设置该位为 0，可以退出当前 JPEG 解码。

- **JPEG 编解码器配置寄存器 1（JPEG_CONFR1）**

JPEG 编解码器配置寄存器 1 描述如图 58.1.5.1 所示：

图 58.1.5.1 JPEG_CONFR1 寄存器

YSIZE[15:0]位，定义 JPEG 图片的高度，读取该寄存器可以获得图片高度（注意：需要在 JPEG 头解析成功以后，才可以读取该寄存器获取图片高度，下同）。

HDR 位，用于设置是否使能 JPEG 头解码，我们一般设置为 1，使能 JPEG 头解码。

DE 位，用于设置硬件 JPEG 工作模式，我们设置为 1，表示使用 JPEG 解码模式。

NF[1:0]位，这两个位用于定义色彩组成：00，表示灰度图片；01，未用到；10，表示 YUV/RGB；11 表示 CYMK。

- **JPEG 编解码器配置寄存器 3（JPEG_CONFR3）**

JPEG 编解码器配置寄存器 3 描述如图 58.1.5.2 所示：

图 58.1.5.2 JPEG_CONFR3 寄存器

该寄存器仅高 16 位（YSIZE[15:0]）有效，定义 JPEG 图片的宽度，读取该寄存器可以获得图片宽度。

另外，还有 JPEG 配置寄存器 4~7：JPEG_CONFR4~7，这四个寄存器 ST 官方数据手册对其解释也不是很清楚，但是我们可以参考 ST 官方提供的参考代码，知道这四个寄存器的 NB[3:0]位用来表示 YUV 的抽样方式（YUV422、YUV420、YUV444），详见本例程源码。

- **JPEG 控制寄存器（JPEG_CR）**

JPEG 控制寄存器描述如图 58.1.5.3 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	OFF	IFF	Res.	Res.	Res.	Res.	Res.	Res.	HPDIE	EOCIE	OFNEIE	OFTIE	IFNFIE	IFTIE	JCEN
	r0	r0							rw	rw	rw	rw	rw	rw	rw

位 14 **OFF**：输出 FIFO 清空 (Output FIFO flush)
此位可清空输出 FIFO。
0：无影响　　1：清空输出 FIFO
读操作将始终返回 0。

位 13 **IFF**：输入 FIFO 清空 (Input FIFO flush)
此位可清空输入 FIFO。
0：无影响　　1：清空输入 FIFO
读操作将始终返回 0。

位 6 **HPDIE**：文件头解析完成中断使能 (Header parsing done interrupt enable)
该位可使能文件头解析操作完成的中断生成。
0：禁止　　1：使能

位 5 **EOCIE**：转换结束中断使能 (End of conversion interrupt enable)
该位可使能转换结束的中断生成。
0：禁止　　1：使能

位 0 **JCEN**：JPEG 内核使能 (JPEG core enable)
该位可使能 JPEG 编解码器内核。
0：禁止（内部寄存器被复位）。　1：使能（内部寄存器可访问）。

图 58.1.5.3 JPEG_CR 寄存器

OFF 位，用于清空输出 FIFO，在启动新图片解码之前，需要对输出 FIFO 进行清空。

IFF 位，用于清空输入 FIFO，在启动新图片解码之前，需要对输入 FIFO 进行清空。

HPDIE 位，用于使能 JPEG 头解码完成中断，我们设置为 1，使能 JPEG 头解码完成中断，在中断服务函数里面读取 JPEG 的相关信息（长宽、颜色空间、色度抽样等），并根据色度抽样方式，获取对应的 YUV→RGB 转换函数。

EOCIE 位，用于使能 JPEG 解码完成中断，我们设置为 1，使能 JPEG 解码完成中断，在中断服务函数里面标记 JPEG 解码完成，以便结束 JPEG 解码流程。

JCEN 位，用于使能硬件 JPEG 内核，我们必须设置此位为 1，以启动硬件 JPEG 内核。

- **JPEG 状态寄存器（JPEG_SR）**

JPEG 状态寄存器描述如图 58.1.5.4 所示：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	COF	HPDF	EOCF	OFNEF	OFTF	IFNFF	IFTF	Res.
								ro	ro	ro	ro	ro	ro	ro	

位 6 **HPDF**：文件头解析完成标志 (Header parsing done flag)
在解码模式下，该位可指示文件头解析和内部寄存器更新是否完成。
0：未完成
1：已完成

位 5 **EOCF**：转换结束标志 (End of conversion flag)
该位可指示编码/解码过程以及向输出 FIFO 传输数据的过程是否完成。
0：未完成
1：已完成

图 58.1.5.4 JPEG_SR 寄存器

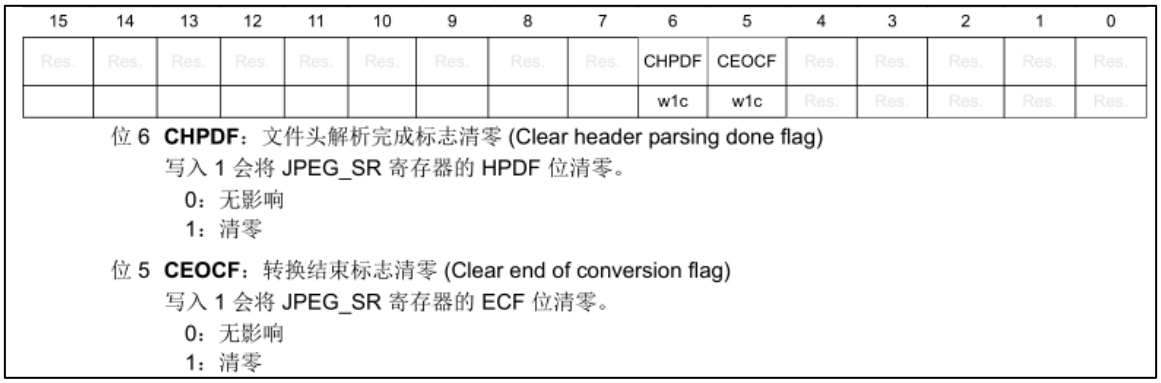
HPDF 位，表示 JPEG 头解码完成的标志，当该位为 1 时，表示 JPEG 头解析成功，我们可以读取相关寄存器，获取 JPEG 图片的长宽、颜色空间和色度抽样等重要信息。向 JPEG_FCR 寄存器的 CHPDF 位写 1，可以清零此位。

EOCF 位，表示 JPEG 解码结束标志，该位为 1 时，表示一张 JPEG 图像解码完成。此时我

们可以从输出 FIFO 读取最后的数据。向 JPEG_FCR 寄存器的 CEOCF 位写 1，可以清零此位。

- **JPEG 清零标志寄存器（JPEG_CFR）**

JPEG 清零标志寄存器描述如图 58.1.5.5 所示：



15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CHPDF	CEOCF	Res.	Res.	Res.	Res.	Res.
									w1c	w1c	Res.	Res.	Res.	Res.	Res.

位 6 **CHPDF**：文件头解析完成标志清零 (Clear header parsing done flag)
写入 1 会将 JPEG_SR 寄存器的 HPDF 位清零。
0：无影响
1：清零

位 5 **CEOCF**：转换结束标志清零 (Clear end of conversion flag)
写入 1 会将 JPEG_SR 寄存器的 ECF 位清零。
0：无影响
1：清零

图 58.1.5.5 JPEG_CFR 寄存器

该寄存器，仅两位有效：CHPDF 位和 CEOCF 位，向这两个位写入 1，可以分别清除 JPEG_SR 寄存器的 HPDF 和 EOCF 位。

最后是 JPEG 数据输入寄存器（JPEG_DIR）和 JPEG 数据输出寄存器（JPEG_DOR），这两个寄存器都是32位有效，前者用于往输入FIFO写入数据。后者用于读取输出FIFO的数据。

58.2 硬件设计

1. 例程功能

1、本实验开机的时候先检测字库，然后检测 SD 卡是否存在，如果 SD 卡存在，则开始查找 SD 卡根目录下的 PICTURE 文件夹，如果找到则显示该文件夹下面的图片文件（支持 bmp、jpg、jpeg 或 gif 格式），循环显示，通过按 KEY0 和 KEY1 可以快速浏览下一张和上一张，KEY_UP 按键用于暂停/继续播放，LED1 用于指示当前是否处于暂停状态。如果未找到 PICTURE 文件夹/任何图片文件，则提示错误。

注意:本例程的实验现象，同上一章(图片显示实验)完全一模一样，唯一的区别，就是 JPEG 解码速度 (要求图片分辨率小于等于 LCD 分辨率) 变快了很多。

对比上一章的图片显示，大家可以利用 USMART 测试同一张 JPEG 图片，软件解码和硬件解码的时间差距。本实验也可以通过 USMART 调用 piclib_ai_load_picfile 和 minibmp_decode 解码任意指定路径的图片。

2、LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
LED1：LED1 - PB0
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
4）独立按键：KEY0 - PH3、KEY1 - PH2、WK_UP - PA0
5）SD 卡，通过 SDMMC1(SDMMC_D0~D4(PC8~PC11)，
SDMMC_SCK(PC12)，SDMMC_CMD(PD2))连接
6）norflash(QSPI FLASH 芯片，连接在 QSPI 上)
7）硬件 JPEG 解码内核(STM32H743 自带)

58.3 程序设计

58.3.1 硬件 JPEG 解码 JPG/JPEG 的简要步骤

为了提高速度，我们直接操作寄存器，没有使用 HAL 库提供的函数。

所以 HAL 库的内容就讲解了，接下来，我们看看在 DMA 模式下，使用 STM32H7 的硬件 JPEG 解码 JPG/JPEG 的简要步骤：

1）初始化硬件 JPEG 内核。

首先，我们通过设置 AHB3ENR 的 bit5 位为 1，使能硬件 JPEG 内核时钟，然后通过 JPEG_CR 寄存器的 JCEN 位，使能硬件 JPEG。通过清零 JPEG_CONFR0 寄存器的 START 位，停止 JPEG 编解码进程。通过设置 JPEG_CONFR1 寄存器的 HDR 位，使能 JPEG 头解码。最后设置 JPEG 中断服务函数的中断优先级，完成初始化硬件 JPEG 内核过程。

2）初始化硬件 JPEG 解码。

在初始化硬件 JPEG 内核以后，我们配置 JPEG 内核工作在 JPEG 解码模式。通过设置 JPEG_CONFR1 寄存器的 DE 位，使能 JPEG 解码模式。然后设置 JPEG_CR 寄存器的 OFF、IFF、HPDIE、EOCIE 等位，清空输出/输入 FIFO，并开启 JPEG 头解码完成和 JPEG 解码完成中断。最后，设置 JPEG_CONFR0 寄存器的 START 位，启动 JPEG 解码进程。

注意：此时我并未开启 JPEG 的输入和输出 MDMA，只要我们不往输入 FIFO 写入数据，JPEG 内核就一直处于等待数据输入状态。

3）配置硬件 JPEG 输入/输出 MDMA。

这一步，我们将配置 JPEG 的输入 MDMA 和输出 MDMA，分别负责 JPEG 输入 FIFO 和输出 FIFO 的数据传输。对于输入 MDMA，目标地址为 JPEG_DIR 寄存器地址，源地址为一片内存区域，利用输入 MDMA 实现 JPEG 输入 FIFO 数据的自动填充。对于输出 MDMA，目标地址为一片内存区域，源地址为 JPEG_DOR 寄存器地址，利用输出 MDMA 实现 JPEG 输出 FIFO 数据自动搬运到对应内存区域。对于输入 MDMA 和输出 MDMA，我们都需要开启传输完成中断，并设置相关中断服务函数。在传输完成中断里面，实现对输入输出数据的处理。

4）编写相关中断服务函数，启动 MDMA。

我们总共开启了 4 个中断：JPEG 头解码完成中断、JPEG 解码完成中断、输入 MDMA 传输完成中断和输出 MDMA 传输完成中断。前两个和后两个中断分别共用一个中断服务函数，所以我们总共只需要编写 2 个中断服务函数。另外，我们采用回调函数的方式，对数据进行处理，总共需要编写 4 个回调函数，分别对应 4 个中断产生时的数据处理。在配置完这些以后，启动 MDMA，开始执行 JPEG 解码。

注意：输出 MDMA 的配置和启动，我们放在 JPEG 头解码完成中断回调函数里的，因为输出 YCbCr 数据的多少和单次输出行数，得根据抽样方式进行不同的设置，因此我们必须先等到解析完 JPEG 头以后，再来配置输出 MDMA。

5）处理 JPEG 数据输出数据，执行 YUV→RGB 转换，并送 LCD 显示。

最后，在主循环里面，根据输入 MDMA 和输出 MDMA 的数据处理情况，持续从源文件读取 JPEG 数据流，并利用 DMA2D，将硬件 JPEG 解码完成的 YCbCr（YUV）数据流转换成 RGB 格式。最后，在完成一张 JPEG 解码之后，将 RGB 数据直接一次性显示到 LCD 屏幕上，实现图片显示。

58.3.2 程序流程图

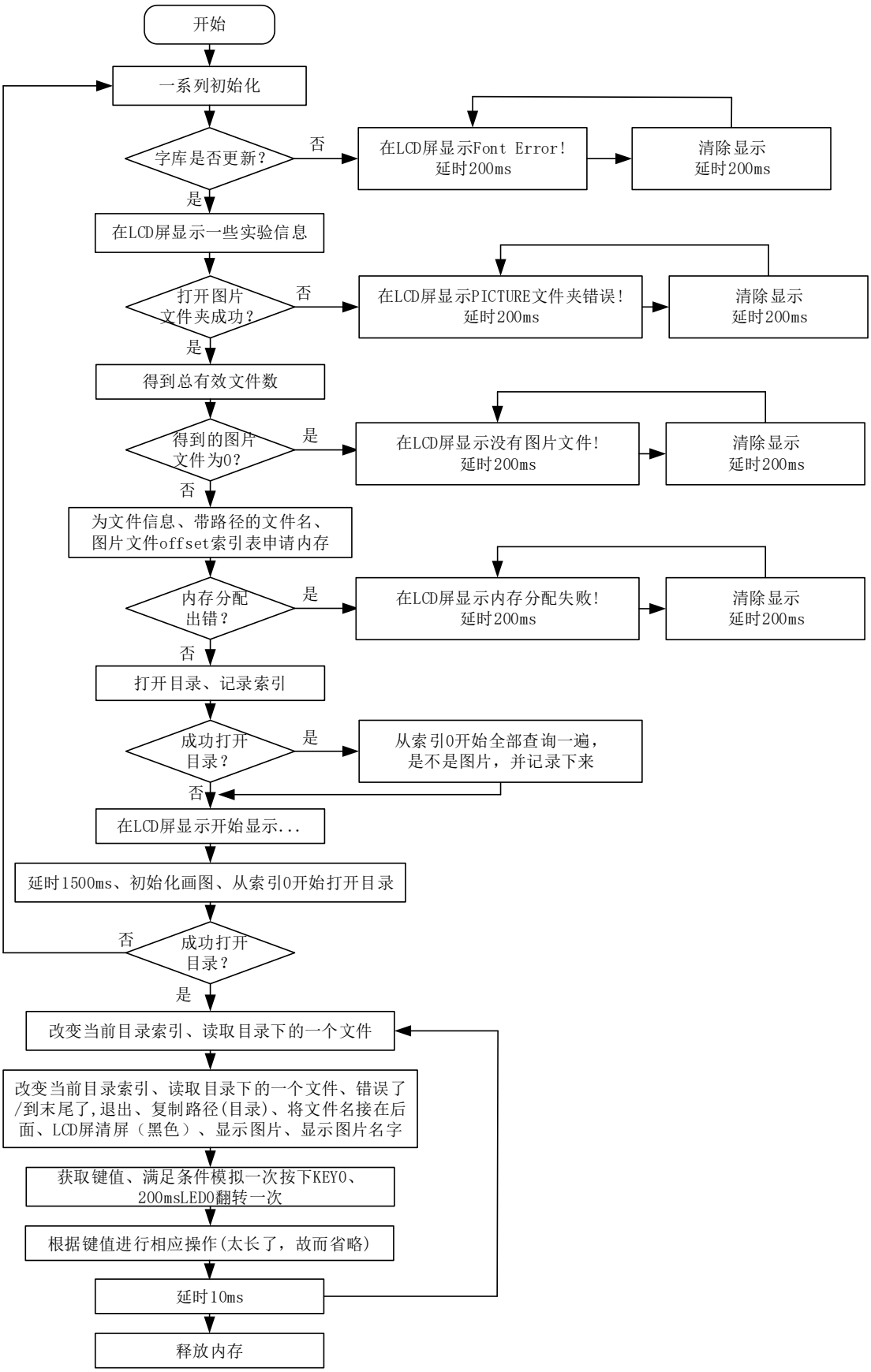


图 58.3.2.1 硬件 JPEG 解码实验程序流程图

58.3.3 程序解析

1. JPEGCODEC 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。JPEGCODEC 驱动源码包括两个文件：jpegcodec.c 和 jpegcodec.h。

jpegcodec.h 头文件定义了两个结构体和一些宏定义，下面重点围绕 jpeg_codec_typedef 这个结构体介绍一下，jpegcodec.h 的相关内容定义如下：

```
#define JPEG_DMA_INBUF_LEN          4096            /* 单个 MDMA IN  BUF 的大小 */
#define JPEG_DMA_INBUF_NB           10              /* MDMA IN  BUF 的个数 */
#define JPEG_DMA_OUTBUF_NB          2               /* MDMA OUT BUF 的个数 */

/* JPEG 数据缓冲结构体 */
typedef struct
{
    uint8_t sta;            /* 状态:0,无数据;1,有数据 */
    char *buf;              /* JPEG 数据缓冲区 */
    uint16_t size;          /* JPEG 数据长度 */
} jpeg_databuf_type;

#define JPEG_STATE_NOHEADER         0               /* HEADER 未读取,初始状态 */
#define JPEG_STATE_HEADEROK         1               /* HEADER 读取成功 */
#define JPEG_STATE_FINISHED         2               /* 解码完成 */
#define JPEG_STATE_ERROR            3               /* 解码错误 */

#define JPEG_YCBCR_COLORSPACE       JPEG_CONFR1_COLORSPACE_0
#define JPEG_CMYK_COLORSPACE        JPEG_CONFR1_COLORSPACE

/* jpeg 编解码控制结构体 */
/typedef struct
{
    JPEG_ConfTypeDef    Conf;                       /* 当前 JPEG 文件相关参数 */
    jpeg_databuf_type inbuf[JPEG_DMA_INBUF_NB];     /* MDMA IN buf */
    jpeg_databuf_type outbuf[JPEG_DMA_OUTBUF_NB];   /* MDMA OUT buf */
    volatile uint8_t inbuf_read_ptr;                /* MDMA IN buf 当前读取位置 */
    volatile uint8_t inbuf_write_ptr;               /* MDMA IN buf 当前写入位置 */
    volatile uint8_t indma_pause;                   /* 输入 MDMA 暂停状态标识 */
    volatile uint8_t outbuf_read_ptr;               /* MDMA OUT buf 当前读取位置 */
    volatile uint8_t outbuf_write_ptr;              /* MDMA OUT buf 当前写入位置 */
    volatile uint8_t outdma_pause;                  /* 输入 MDMA 暂停状态标识 */
    volatile uint8_t state;
   /* 解码状态:0,未识别到 Header;1,识别到了 Header;2,解码完成*/

/* YUV 输出的字节数,使得完成一次 DMA2D YUV2RGB 转换,刚好是图片宽度的整数倍
 * YUV420 图片,每个像素占 1.5 个 YUV 字节,每次输出 16 行,yuvblk_size=图片宽度*16*1.5
 * YUV422 图片,每个像素占 2 个 YUV 字节和 RGB565 一样,每次输出 8 行,yuvblk_size=图片宽度
 * 8*2
 * YUV444 图片,每个像素占 3 个 YUV 字节,每次输出 8 行,yuvblk_size=图片宽度*8*3 */
   uint32_t yuvblk_size;

   /* 每个 YUV 块输出像素的高度,对于 YUV420,为 16,对于 YUV422/YUV444 为 8 */
    uint16_t yuvblk_height;
    uint16_t yuvblk_curheight;                      /* 当前输出高度,0~分辨率高度 */
} jpeg_codec_typedef;
```

该结构体用于控制整个 JPEG 解码，下面分别介绍一下它的成员：

Conf 用于存储当前 JPEG 文件的一些相关信息，其结构体类型定义如下：

```
/* JPEG 文件信息结构体 */
typedef struct
{
```

```
    uint32_t  ColorSpace;    /* 图像的颜色空间: gray-scale/YCBCR/RGB/CMYK */
    /* YCBCR/CMYK 颜色空间的色度抽样情况: 0: 4:4:4;  1: 4:2:2;  2: 4:1:1;  3: 4:2:0 */
    uint32_t  ChromaSubsampling;
    uint32_t ImageHeight;         /* 图像高度 */
    uint32_t ImageWidth;          /* 图像宽度 */
    uint32_t  ImageQuality;       /* 图像编码质量:1~100 */
}JPEG_ConfTypeDef;
```

inbuf 和 outbuf 分别代表输入 MDMA FIFO 和输出 MDMA FIFO，使用 FIFO 来处理 MDMA 数据，可以提高读写效率。注意：这里的输入 MDMA FIFO 和输出 MDMA FIFO 同 JPEG 的输入 FIFO 和输出 FIFO 是不一样的，要注意区分。通过 JPEG_DMA_INBUF_NB 和 JPEG_DMA_OUTBUF_NB 宏定义，我们可以修改输入 DMA FIFO 和输出 DMA FIFO 的深度。

其它，还有输入输出 MDMA FIFO 的读写位置、暂停状态、解码状态、单次 YUV 输出字节数、单次输出图像高度和当前输出高度等信息。

下面开始介绍 jpegcodec.c 文件，首先是 JPEG 规范(ISO/IEC 10918-1 标准)的样本量化表，其定义如下：

```
/* JPEG 规范(ISO/IEC 10918-1 标准)的样本量化表
 * 获取 JPEG 图片质量时需要用到
 */
const uint8_t JPEG_LUM_QuantTable[JPEG_QUANT_TABLE_SIZE] =
{
    16, 11, 10, 16, 24, 40, 51, 61, 12, 12, 14, 19, 26, 58, 60, 55,
    14, 13, 16, 24, 40, 57, 69, 56, 14, 17, 22, 29, 51, 87, 80, 62,
    18, 22, 37, 56, 68, 109, 103, 77, 24, 35, 55, 64, 81, 104, 113, 92,
    49, 64, 78, 87, 103, 121, 120, 101, 72, 92, 95, 98, 112, 100, 103, 99
};

const uint8_t JPEG_ZIGZAG_ORDER[JPEG_QUANT_TABLE_SIZE] =
{
    0, 1, 8, 16, 9, 2, 3, 10, 17, 24, 32, 25, 18, 11, 4, 5,
    12, 19, 26, 33, 40, 48, 41, 34, 27, 20, 13, 6, 7, 14, 21, 28,
    35, 42, 49, 56, 57, 50, 43, 36, 29, 22, 15, 23, 30, 37, 44, 51,
    58, 59, 52, 45, 38, 31, 39, 46, 53, 60, 61, 54, 47, 55, 62, 63
};
```

这两个数组在后面的 jpeg_get_quality 函数，获取 JPEG 图片质量时需要用到。

下面介绍的是 JPEG 硬件解码输入 MDMA 配置函数，其定义如下：

```
/**
 * @brief       JPEG 硬件解码输入 MDMA 配置
 * @param       meminaddr   : JPEG 输入 MDMA 存储器地址
 * @param       meminsize   : 输入 MDMA 数据长度,0~262143,以字节为单位
 * @retval      无
 */
void jpeg_in_dma_init(uint32_t meminaddr, uint32_t meminsize)
{
    uint32_t regval = 0;
    uint32_t addrmask = 0;
    RCC->AHB3ENR |= 1 << 0;             /* 使能 MDMA 时钟 */
    MDMA_Channel7->CCR = 0;             /* 输入 MDMA 清零 */

    while (MDMA_Channel7->CCR & 0X01); /* 等待 MDMA_Channel7 关闭完成 */
    MDMA_Channel7->CIFCR = 0X1F;        /* 中断标志清零 */
    MDMA_Channel7->CCR |= 1 << 2;       /* CTCIE=1,使能通道传输完成中断 */
    MDMA_Channel7->CCR |= 2 << 6;       /* PL[1:0]=2,高优先级 */
    MDMA_Channel7->CBNDTR = meminsize; /* 传输长度为 meminsize */
    MDMA_Channel7->CDAR = (uint32_t)&JPEG->DIR; /* 目标地址为:JPEG->DIR */
    MDMA_Channel7->CSAR = meminaddr;    /* meminaddr 作为源地址 */
    regval = 0 << 28;             /* TRGM[1:0]=0,每个 MDMA 请求触发一次 buffer 传输 */
    regval |= 1 << 25;                  /* PKE=1,打包使能 */
    regval |= (32 - 1) << 18;           /* TLEN[6:0]=31,buffer 传输长度为 32 字节 */
    regval |= 4 << 15;                  /* DBURST[2:0]=4,目标突发传输长度为 16 */
```

```
    regval |= 4 << 12;                  /* SBURST[2:0]=4,源突发传输长度为 16 */
    regval |= 0 << 8;                   /*SINCOS[1:0]=0,源地址变化单位为 8 位(字节)*/
    regval |= 2 << 6;                   /* DSIZE[1:0]=2,目标位宽为 32 位 */
    regval |= 0 << 4;                   /* SSIZE[1:0]=0,源位宽为 8 位 */
    regval |= 0 << 2;                   /* DINC[1:0]=0,目标地址固定 */
    regval |= 2 << 0;                   /* SINC[1:0]=2,源地址自增 */
    MDMA_Channel7->CTCR = regval;       /* 设置 CTCR 寄存器 */
    /* MDMA 的硬件触发通道 17 触发 inmdma,通道 17=JPEG input FIFO threshold
     * 详见< STM32H7xx 参考手册_V7（英文版）.pdf >577 页,table 95 */
    MDMA_Channel7->CTBR = 17 << 0;
    addrmask = meminaddr & 0XFF000000;  /* 获取掩码 */
    /* 使用 AHBS 总线访问 DTCM/ITCM */
    if (addrmask == 0X20000000 || addrmask == 0)MDMA_Channel7->CTBR |= 1 << 16;
    HAL_NVIC_SetPriority(MDMA_IRQn,2,3);/* 设置中断优先级，抢占优先级 2，子优先级 3 */
    HAL_NVIC_EnableIRQ(MDMA_IRQn);      /* 开启 MDMA 中断 */
}
```

该函数用于初始化 JPEG 输入 FIFO 的 MDMA 通道，使用 buffer 传输，单次触发传输 32 字节，满足 JPEG 输入 FIFO 的传输要求。

下面介绍的是 JPEG 硬件解码输出 MDMA 配置函数，其定义如下：

```
/**
 * @brief       JPEG 硬件解码输出 MDMA 配置
 * @param       memoutaddr  : JPEG 输出 MDMA 存储器地址
 * @param       memoutsize  : 输出 MDMA 数据长度,0~262143,以字节为单位
 * @retval      无
 */
void jpeg_out_dma_init(uint32_t memoutaddr, uint32_t memoutsize)
{
    uint32_t regval = 0;
    uint32_t addrmask = 0;
    RCC->AHB3ENR |= 1 << 0;                 /* 使能 MDMA 时钟 */
    MDMA_Channel6->CCR = 0;                 /* 输出 MDMA 清零 */

    while (MDMA_Channel6->CCR & 0X01);      /* 等待 MDMA_Channel6 关闭完成 */

    MDMA_Channel6->CIFCR = 0X1F;            /* 中断标志清零 */
    MDMA_Channel6->CCR |= 3 << 6;           /* PL[1:0]=2,最高优先级 */
    MDMA_Channel6->CCR |= 1 << 2;           /* CTCIE=1,使能通道传输完成中断 */
    MDMA_Channel6->CBNDTR = memoutsize;     /* 传输长度为 meminsize */
    MDMA_Channel6->CDAR = memoutaddr;       /* 目标地址为:memoutaddr */
    MDMA_Channel6->CSAR = (uint32_t)&JPEG->DOR; /* JPEG->DOR 作为源地址 */
    regval = 0 << 28;               /* TRGM[1:0]=0,每个 MDMA 请求触发一次 buffer 传输 */
    regval |= 1 << 25;              /* PKE=1,打包使能 */
    regval |= (32 - 1) << 18;       /* TLEN[6:0]=31,buffer 传输长度为 32 字节 */
    regval |= 4 << 15;              /* DBURST[2:0]=4,目标突发传输长度为 16 */
    regval |= 4 << 12;              /* SBURST[2:0]=4,源突发传输长度为 16 */
    regval |= 0 << 10;              /* DINCOS[1:0]=0,目标地址变化单位为 8 位(字节) */
    regval |= 0 << 6;               /* DSIZE[1:0]=0,目标位宽为 8 位 */
    regval |= 2 << 4;               /* SSIZE[1:0]=2,源位宽为 32 位 */
    regval |= 2 << 2;               /* DINC[1:0]=2,目标地址自增 */
    regval |= 0 << 0;               /* SINC[1:0]=0,源地址固定 */
    MDMA_Channel6->CTCR = regval;/* 设置 CTCR 寄存器 */
    /* MDMA 的硬件触发通道 19 触发 outmdma,通道 19=JPEG output FIFO threshold */
    MDMA_Channel6->CTBR = 19 << 0;

    /* 详见< STM32H7xx 参考手册_V7（英文版）.pdf >577 页,table 95 */
    addrmask = memoutaddr & 0XFF000000; /* 获取掩码 */
    /* 使用 AHBS 总线访问 DTCM/ITCM */
    if (addrmask == 0X20000000 || addrmask == 0)MDMA_Channel6->CTBR |= 1 << 17;
```

```
    HAL_NVIC_SetPriority(MDMA_IRQn,2,3);/* 设置中断优先级，抢占优先级 2，子优先级 3 */
    HAL_NVIC_EnableIRQ(MDMA_IRQn);       /* 开启 MDMA 中断 */
}
```

该函数用于初始化 JPEG 输出 FIFO 的 MDMA 通道，使用 buffer 传输，单次触发传输 32 字节，满足 JPEG 输出 FIFO 的传输要求。

下面介绍一些中断处理函数和回调函数，它们的定义分别如下：

```
void (*jpeg_in_callback)(void);             /* JPEG MDMA 输入回调函数 */
void (*jpeg_out_callback)(void);            /* JPEG MDMA 输出 回调函数 */
void (*jpeg_eoc_callback)(void);            /* JPEG 解码完成 回调函数 */
void (*jpeg_hdp_callback)(void);            /* JPEG Header 解码完成 回调函数 */

/**
 * @brief       MDMA 中断服务函数
 * @note        处理硬件 JPEG 解码时输入/输出数据流
 * @param       无
 * @retval      无
 */
void MDMA_IRQHandler(void)
{
    if (MDMA_Channel7->CISR & (1 << 1))  /* CTCIF,通道 7 传输完成(输入) */
    {
       MDMA_Channel7->CIFCR |= 1 << 1;   /* 清除通道传输完成中断 */
       JPEG->CR &= ~(0X7E);              /* 关闭 JPEG 中断,防止被打断 */
       jpeg_in_callback();               /* 执行输入回调函数,继续读取数据 */
       JPEG->CR |= 3 << 5;               /* 使能 EOC 和 HPD 中断 */
    }
    if (MDMA_Channel6->CISR & (1 << 1))       /* CTCIF,通道 6 传输完成(输出) */
    {
        MDMA_Channel6->CIFCR |= 1 << 1;       /* 清除通道传输完成中断 */
        JPEG->CR &= ~(0X7E);                  /* 关闭 JPEG 中断,防止被打断 */
        jpeg_out_callback();                  /* 执行输出回调函数,将数据转换成 RGB */
        JPEG->CR |= 3 << 5;                   /* 使能 EOC 和 HPD 中断 */
    }
}

/**
 * @brief       JPEG 解码中断服务函数
 * @param       无
 * @retval      无
 */
void JPEG_IRQHandler(void)
{
    if (JPEG->SR & (1 << 6))        /* JPEG Header 解码完成 */
    {
        jpeg_hdp_callback();
        JPEG->CR &= ~(1 << 6);      /* 禁止 Jpeg Header 解码完成中断 */
        JPEG->CFR |= 1 << 6;        /* 清除 HPDF 位(header 解码完成位) */
    }

    if (JPEG->SR & (1 << 5))        /* JPEG 解码完成 */
    {
        jpeg_dma_stop();
        jpeg_eoc_callback();
        JPEG->CFR |= 1 << 5;                /* 清除 EOC 位(解码完成位) */
        MDMA_Channel6->CCR &= ~(1 << 0);    /* 关闭 MDMA 通道 6 */
        MDMA_Channel7->CCR &= ~(1 << 0);    /* 关闭 MDMA 通道 7 */
    }
}
```

MDMA_IRQHandler 中断服务函数，用于处理输入/输出 FIFO MDMA 的传输完成中断，当发生输入 FIFO MDMA 传输完成中断时，调用 jpeg_in_callback 回调函数，处理输入 FIFO

MDMA 传输完成事务。当发生输出 FIFO MDMA 传输完成中断时，调用 jpeg_out_callback 回调函数，处理输出 FIFO MDMA 传输完成事务。

JPEG_IRQHandler 中断服务函数，根据 JPEG_SR 的状态标志位，分别处理 JPEG 头解码完成中断和 JPEG 文件解码完成中断。当 JPEG 头解码完成时，调用 jpeg_hdp_callback 回调函数处理相关事务。当 JPEG 文件解码完成时，调用 jpeg_eoc_callback 回调函数处理相关事务，同时停止 MDMA 传输。

下面介绍的是初始化硬件 JPEG 内核函数，其定义如下：

```
/**
 * @brief       初始化硬件 JPEG 内核
 * @param       tjpeg       : JPEG 编解码控制结构体
 * @retval      0, 成功; 1, 失败;
 */
uint8_t jpeg_core_init(jpeg_codec_typedef *tjpeg)
{
    uint8_t i;
    RCC->AHB3ENR |= 1 << 5;             /* 使能硬件 jpeg 时钟 */

    for (i = 0; i < JPEG_DMA_INBUF_NB; i++)
    {
        tjpeg->inbuf[i].buf = mymalloc(SRAMDTCM, JPEG_DMA_INBUF_LEN);

        if (tjpeg->inbuf[i].buf == NULL)
        {
            jpeg_core_destroy(tjpeg);
            return 1;
        }
    }
    JPEG->CR = 0;                       /* 先清零 */
    JPEG->CR |= 1 << 0;                 /* 使能硬件 JPEG */
    JPEG->CONFR0 &= ~(1 << 0);          /* 停止 JPEG 编解码进程 */
    JPEG->CR |= 1 << 13;                /* 清空输入 fifo */
    JPEG->CR |= 1 << 14;                /* 清空输出 fifo */
    JPEG->CFR = 3 << 5;                 /* 清空标志 */
    HAL_NVIC_SetPriority(JPEG_IRQn,1,3);/* 设置中断优先级，抢占优先级 1，子优先级 3 */
    HAL_NVIC_EnableIRQ(JPEG_IRQn);      /* 开启 JPEG 中断 */
    JPEG->CONFR1 |= 1 << 8;             /* 使能 header 处理 */
    return 0;
}
```

jpeg_core_init 函数用于初始化硬件 JPEG 内核。在该函数里面，对 tjpeg->inbuf[i].buf 数组申请内存（tjpeg->outbuf[i]的内存申请，我们放到了 jpeg_hdrover_callback 函数里面）。tjpeg 是 jpeg_codec_typedef结构体类型变量，用于控制整个 JPEG 解码。jpeg_codec_typedef结构体我们在 jpegcodec.h 里面定义的，前面讲过的。

下面介绍的是关闭硬件 JPEG 内核，并释放内存函数，其定义如下：

```
/**
 * @brief       关闭硬件 JPEG 内核,并释放内存
 * @param       tjpeg       : JPEG 编解码控制结构体
 * @retval      无
 */
void jpeg_core_destroy(jpeg_codec_typedef *tjpeg)
{
    uint8_t i;
    jpeg_dma_stop();    /* 停止 MDMA 传输 */

    for (i = 0; i < JPEG_DMA_INBUF_NB; i++)
    {
        myfree(SRAMDTCM, tjpeg->inbuf[i].buf);  /* 释放内存 */
    }
    for (i = 0; i < JPEG_DMA_OUTBUF_NB; i++)
    {
        myfree(SRAMIN, tjpeg->outbuf[i].buf);   /* 释放内存 */
```

```
    }
}
```

该函数用于关闭 JPEG 处理（停止 MDMA 传输），并释放内存。

下面介绍的是初始化硬件 JPEG 解码器函数，其定义如下：

```
/**
 * @brief       初始化硬件 JPEG 解码器
 * @param       tjpeg       : JPEG 编解码控制结构体
 * @retval      无
 */
void jpeg_decode_init(jpeg_codec_typedef *tjpeg)
{
    uint8_t i;
    tjpeg->inbuf_read_ptr = 0;
    tjpeg->inbuf_write_ptr = 0;
    tjpeg->indma_pause = 0;
    tjpeg->outbuf_read_ptr = 0;
    tjpeg->outbuf_write_ptr = 0;
    tjpeg->outdma_pause = 0;
    tjpeg->state = JPEG_STATE_NOHEADER; /* 图片解码结束标志 */

    for (i = 0; i < JPEG_DMA_INBUF_NB; i++)
    {
        tjpeg->inbuf[i].sta = 0;
        tjpeg->inbuf[i].size = 0;
    }

    for (i = 0; i < JPEG_DMA_OUTBUF_NB; i++)
    {
        tjpeg->outbuf[i].sta = 0;
        tjpeg->outbuf[i].size = 0;
    }

    MDMA_Channel6->CCR = 0;             /* MDMA 通道 6 禁止 */
    MDMA_Channel7->CCR = 0;             /* MDMA 通道 7 禁止 */
    MDMA_Channel6->CIFCR = 0X1F;        /* 中断标志清零 */
    MDMA_Channel7->CIFCR = 0X1F;        /* 中断标志清零 */

    JPEG->CONFR1 |= 1 << 3;             /* 硬件 JPEG 解码模式 */
    JPEG->CONFR0 &= ~(1 << 0);          /* 停止 JPEG 编解码进程 */
    JPEG->CR &= ~(0X3F << 1);           /* 关闭所有中断 */
    JPEG->CR |= 1 << 13;                /* 清空输入 fifo */
    JPEG->CR |= 1 << 14;                /* 清空输出 fifo */
    JPEG->CR |= 1 << 6;                 /* 使能 Jpeg Header 解码完成中断 */
    JPEG->CR |= 1 << 5;                 /* 使能解码完成中断 */
    JPEG->CFR = 3 << 5;                 /* 清空标志 */
    JPEG->CONFR0 |= 1 << 0;             /* 使能 JPEG 编解码进程 */
}
```

该函数用于初始化硬件 JPEG 解码器，同时对输入 MDMA FIFO 和输出 MDMA FIFO 的相关标记进行清理处理，以便开始 JPEG 解码。

下面介绍的是启动输入 MDMA 函数和启动输出 MDMA 函数，它们的定义如下：

```
/**
 * @brief       启动 jpeg in mdma, 开始解码 JPEG
 * @param       无
 * @retval      无
 */
void jpeg_in_dma_start(void)
{
    MDMA_Channel7->CCR |= 1 << 0;    /* 使能 MDMA 通道 7 的传输 */
}

/**
```

```
 * @brief       启动 jpeg out mdma, 开始输出 YUV 数据
 * @param       无
 * @retval      无
 */
void jpeg_out_dma_start(void)
{
    MDMA_Channel6->CCR |= 1 << 0;    /* 使能 MDMA 通道 6 的传输 */
}
```

jpeg_in_dma_start 函数启动 jpeg in mdma，开始解码 JPEG。jpeg_out_dma_start 函数启动 jpeg out mdma，开始输出 YUV 数据。

下面介绍的是停止 JPEG MDMA 解码过程函数，其定义如下：

```
/**
 * @brief       停止 JPEG MDMA 解码过程
 * @param       无
 * @retval      无
 */
void jpeg_dma_stop(void)
{
    JPEG->CONFR0 &= ~(1 << 0);          /* 停止 JPEG 编解码进程 */
    JPEG->CR &= ~(0X3F << 1);           /* 关闭所有中断 */
    JPEG->CFR = 3 << 5;                 /* 清空标志 */
}
```

该函数用于停止 JPEG MDMA 解码过程。

下面介绍的是恢复 MDMA IN 过程函数和恢复 MDMA OUT 过程函数，它们的定义如下：

```
/**
 * @brief       恢复 MDMA IN 过程
 * @param       memaddr     : 存储区首地址
 * @param       memlen      : 要传输数据长度(以字节为单位)
 * @retval      无
 */
void jpeg_in_dma_resume(uint32_t memaddr, uint32_t memlen)
{
    if (memlen % 4)memlen += 4 - memlen % 4;    /* 扩展到 4 的倍数 */

    MDMA_Channel7->CIFCR = 0X1F;        /* 中断标志清零 */
    MDMA_Channel7->CBNDTR = memlen;     /* 传输长度为 memlen */
    MDMA_Channel7->CSAR = memaddr;      /* memaddr 作为源地址 */
    MDMA_Channel7->CCR |= 1 << 0;       /* 使能 MDMA 通道 7 的传输 */
}

/**
 * @brief       恢复 MDMA OUT 过程
 * @param       memaddr     : 存储区首地址
 * @param       memlen      : 要传输数据长度(以字节为单位)
 * @retval      无
 */
void jpeg_out_dma_resume(uint32_t memaddr, uint32_t memlen)
{
    if (memlen % 4)memlen += 4 - memlen % 4;    /* 扩展到 4 的倍数 */

    MDMA_Channel6->CIFCR = 0X1F;        /* 中断标志清零 */
    MDMA_Channel6->CBNDTR = memlen;     /* 传输长度为 memlen */
    MDMA_Channel6->CDAR = memaddr;      /* memaddr 作为源地址 */
    MDMA_Channel6->CCR |= 1 << 0;       /* 使能 MDMA 通道 6 的传输 */
}
```

jpeg_in_dma_resume 函数用于重启启动输入 MDMA。jpeg_out_dma_resume 函数用于重启启动输出 MDMA。

下面介绍的是获取图像信息函数，其定义如下：

```
/**
```

```
 * @brief       获取图像信息
 * @param       tjpeg       : JPEG 编解码控制结构体
 * @retval      无
 */
void jpeg_get_info(jpeg_codec_typedef *tjpeg)
{
    uint32_t yblockNb, cBblockNb, cRblockNb;

    switch (JPEG->CONFR1 & 0X03)
    {
        case 0:/* grayscale,1 color component */
            tjpeg->Conf.ColorSpace = JPEG_GRAYSCALE_COLORSPACE;
            break;

        case 2:/* YUV/RGB,3 color component */
            tjpeg->Conf.ColorSpace = JPEG_YCBCR_COLORSPACE;
            break;

        case 3:/* CMYK,4 color component */
            tjpeg->Conf.ColorSpace = JPEG_CMYK_COLORSPACE;
            break;
    }

    tjpeg->Conf.ImageHeight = (JPEG->CONFR1 & 0XFFFF0000)>>16;/* 获得图像高度 */
    tjpeg->Conf.ImageWidth = (JPEG->CONFR3 & 0XFFFF0000)>>16; /* 获得图像宽度 */

    if ((tjpeg->Conf.ColorSpace == JPEG_YCBCR_COLORSPACE)
        || (tjpeg->Conf.ColorSpace == JPEG_CMYK_COLORSPACE))
    {
        yblockNb  = (JPEG->CONFR4 & (0XF << 4)) >> 4;
        cBblockNb = (JPEG->CONFR5 & (0XF << 4)) >> 4;
        cRblockNb = (JPEG->CONFR6 & (0XF << 4)) >> 4;

        if ((yblockNb == 1) && (cBblockNb == 0) && (cRblockNb == 0))
        {
          tjpeg->Conf.ChromaSubsampling = JPEG_422_SUBSAMPLING;/* 16x8 block */
        }
        else if ((yblockNb == 0) && (cBblockNb == 0) && (cRblockNb == 0))
        {
            tjpeg->Conf.ChromaSubsampling = JPEG_444_SUBSAMPLING;
        }
        else if ((yblockNb == 3) && (cBblockNb == 0) && (cRblockNb == 0))
        {
            tjpeg->Conf.ChromaSubsampling = JPEG_420_SUBSAMPLING;
        }
        else
        {
            tjpeg->Conf.ChromaSubsampling = JPEG_444_SUBSAMPLING;
        }
    }
    else
    {
        tjpeg->Conf.ChromaSubsampling = JPEG_444_SUBSAMPLING;  /* 默认用 4:4:4 */
    }
    /* 图像质量参数在整个图片的最末尾,刚开始的时候,是无法获取的,所以直接设置为 0 */
    tjpeg->Conf.ImageQuality = 0;
}
```

jpeg_get_info 函数，用于获取 JPEG 图像信息，在 JPEG 头解码完成后，被调用。该函数可以获取 JPEG 图片的宽度、高度、颜色空间和色度抽样等重要信息。

下面介绍的是得到 JPEG 图像质量函数，其定义如下：

```
/**
 * @brief       得到 JPEG 图像质量
 * @note        在解码完成后,可以调用并获得正确的结果.
 * @param       无
```

```
 * @retval      图像质量, 0~100
 */
uint8_t jpeg_get_quality(void)
{
    uint32_t quality = 0;
    uint32_t quantRow, quantVal, scale, i, j;
    uint32_t *tableAddress = (uint32_t *)JPEG->QMEM0;
    i = 0;
    while (i < JPEG_QUANT_TABLE_SIZE)
    {
        quantRow = *tableAddress;

        for (j = 0; j < 4; j++)
        {
            quantVal = (quantRow >> (8 * j)) & 0xFF;
            if (quantVal == 1)quality += 100;   /* 100% */
            else
            {
                scale = (quantVal * 100)
                  / ((uint32_t)JPEG_LUM_QuantTable[JPEG_ZIGZAG_ORDER[i + j]]);

                if (scale <= 100)
                {
                    quality += (200 - scale) / 2;
                }
                else
                {
                    quality += 5000 / scale;
                }
            }
        }
        i += 4;
        tableAddress++;
    }
    return (quality / ((uint32_t)64));
}
```

该函数用于获取当前 JPEG 图像的质量，返回值越大，说明图像质量越好，解码所要耗费的时间就越多。该函数我们一般用不到。

最后介绍的是将 YUV 数据转换成 RGB 数据函数，其定义如下：

```
/**
 * @brief       将 YUV 数据转换成 RGB 数据
 * @note        利用 DMA2D, 将 JPEG 解码的 YUV 数据转换成 RGB 数据, 全硬件完成, 速度非常快
 * @param       tjpeg       : JPEG 编解码控制结构体
 * @param       pdst        : 输出数组首地址
 * @retval      0, 成功; 1, 超时,失败;
 */
uint8_t jpeg_dma2d_yuv2rgb_conversion(jpeg_codec_typedef *tjpeg,uint32_t *pdst)
{
    uint32_t regval = 0;
    uint32_t cm = 0;                    /* 采样方式 n */
    uint32_t timeout = 0;

    if (tjpeg->Conf.ChromaSubsampling == JPEG_420_SUBSAMPLING)
    {
        cm = DMA2D_CSS_420;             /* YUV420 转 RGB */
    }
    else if (tjpeg->Conf.ChromaSubsampling == JPEG_422_SUBSAMPLING)
    {
        cm = DMA2D_CSS_422;             /* YUV422 转 RGB */
    }
    else if (tjpeg->Conf.ChromaSubsampling == JPEG_444_SUBSAMPLING)
    {
        cm = DMA2D_NO_CSS;          /* YUV444 转 RGB */
    }
```

```
    RCC->AHB3ENR |= 1 << 4;          /* 使能 DMA2D 时钟 */
    RCC->AHB3RSTR |= 1 << 4;         /* 复位 DMA2D */
    RCC->AHB3RSTR &= ~(1 << 4);      /* 结束复位 */
    DMA2D->CR &= ~(1 << 0);          /* 先停止 DMA2D */
    DMA2D->CR = 1 << 16;             /* MODE[2:0]=001,存储器到存储器,带 PFC 模式 */
    DMA2D->OPFCCR = 2 << 0;          /* CM[2:0]=010,输出为 RGB565 格式 */
    DMA2D->OOR = 0;                  /* 设置行偏移为 0 */
    DMA2D->IFCR |= 1 << 1;           /* 清除传输完成标志 */
    regval = 11 << 0;                /* CM[3:0]=1011,输入数据为 YCbCr 格式 */
    /* CSS[1:0]=cm,Chroma Sub-Sampling:0,4:4:4;1,4:2:2;2,4:2:0 */
    regval |= cm << 18;
    DMA2D->FGPFCCR = regval;         /* 设置 FGPCCR 寄存器 */
    DMA2D->FGOR = 0;                 /* 前景层行偏移为 0 */
    /* 设定行数寄存器 */
    DMA2D->NLR = tjpeg->yuvblk_height | (tjpeg->Conf.ImageWidth << 16);
    DMA2D->OMAR = (uint32_t)pdst;    /* 输出存储器地址 */
    DMA2D->FGMAR = (uint32_t)tjpeg->outbuf[tjpeg->outbuf_read_ptr].buf;
    /*源地址*/
    DMA2D->CR |= 1 << 0;             /* 启动 DMA2D */
    while ((DMA2D->ISR & (1 << 1)) == 0)     /* 等待传输完成 */
    {
        timeout++;

        if (timeout > 0X1FFFFFF)break;       /* 超时退出 */
    }
    /* YUV2RGB 转码结束后,再复位一次 DMA2D */
    RCC->AHB3RSTR |= 1 << 4;      /* 复位 DMA2D */
    RCC->AHB3RSTR &= ~(1 << 4);   /* 结束复位 */
    if (timeout > 0X1FFFFFF)return 1;
    return 0;
}
```

该函数使用硬件 DMA2D 实现 YCbCr（YUV）图像数据到 RGB 图像数据的格式转换，可以快速实现 YUV→RGB 数据的转换。具体的实现原理，我们在前面已经介绍过了。

JPEGCODEC 驱动代码就介绍到这里。

2. PICTURE 驱动代码

PICTURE 驱动代码，本实验我们添加了 hjpgd.c 和 hjpgd.h，用于实现 JPG/JPEG 图片的硬件 JPEG 解码。

hjpgd.h 头文件中只是一些函数声明，下面直接介绍 hjpgd.c 文件的代码，首先是 JPEG 输入数据流回调函数，其定义如下：

```
jpeg_codec_typedef hjpgd;        /* JPEG 硬件解码结构体 */

/**
 * @brief       JPEG 输入数据流回调函数
 * @note        用于获取 JPEG 文件原始数据, 每当 JPEG DMA IN BUF 为空的时候,调用该函数
 * @param       无
 * @retval      无
 */
void jpeg_dma_in_callback(void)
{
    hjpgd.inbuf[hjpgd.inbuf_read_ptr].sta = 0;      /* 此 buf 已经处理完了 */
    hjpgd.inbuf[hjpgd.inbuf_read_ptr].size = 0;     /* 此 buf 已经处理完了 */
    hjpgd.inbuf_read_ptr++;                         /* 指向下一个 buf */
    /* 归零 */
    if (hjpgd.inbuf_read_ptr >= JPEG_DMA_INBUF_NB) hjpgd.inbuf_read_ptr = 0;

    if (hjpgd.inbuf[hjpgd.inbuf_read_ptr].sta == 0)    /* 无有效 buf */
    {
```

```
        hjpgd.indma_pause = 1;  /* 标记暂停 */
    }
    else    /* 有效的 buf */
    {
        /* 继续下一次 DMA 传输 */
        jpeg_in_dma_resume((uint32_t)hjpgd.inbuf[hjpgd.inbuf_read_ptr].buf,
                           hjpgd.inbuf[hjpgd.inbuf_read_ptr].size);
    }
}
```

该函数用于处理 JPEG 输入数据流，当 JPEG 输入 MDMA 传输完成时，调用该函数。对已处理的 buf 标记清零，然后切换到下一个 buf。当 buf 不够时，暂停 JPEG 输入 FIFO 获取数据，并标记暂停；当 buf 足够时，切换到下一个 buf，继续传输。

下面介绍的是 JPEG 输出数据流(YCBCR)回调函数，其定义如下：

```
/**
 * @brief       JPEG 输出数据流(YCBCR)回调函数
 * @note        用于输出 YCbCr 数据流(YUV)
 * @param       无
 * @retval      无
 */
void jpeg_dma_out_callback(void)
{
    uint32_t *pdata = 0;
    hjpgd.outbuf[hjpgd.outbuf_write_ptr].sta = 1;       /* 此 buf 已满 */
    hjpgd.outbuf[hjpgd.outbuf_write_ptr].size = hjpgd.yuvblk_size
                - (MDMA_Channel6->CBNDTR & 0X1FFFF);  /* 此 buf 里面数据的长度 */
    /* 如果文件已经解码完成,需要读取 DOR 最后的数据(<=32 字节) */
    if (hjpgd.state == JPEG_STATE_FINISHED)
    {
        pdata = (uint32_t *)(hjpgd.outbuf[hjpgd.outbuf_write_ptr].buf
              + hjpgd.outbuf[hjpgd.outbuf_write_ptr].size);

        while (JPEG->SR & (1 << 4))
        {
            *pdata = JPEG->DOR;
            pdata++;
            hjpgd.outbuf[hjpgd.outbuf_write_ptr].size += 4;
        }
    }
    hjpgd.outbuf_write_ptr++;   /* 指向下一个 buf */
    /* 归零 */
    if(hjpgd.outbuf_write_ptr >= JPEG_DMA_OUTBUF_NB)hjpgd.outbuf_write_ptr = 0;

    if (hjpgd.outbuf[hjpgd.outbuf_write_ptr].sta == 1)  /* 无有效 buf */
    {
        hjpgd.outdma_pause = 1; /* 标记暂停 */
    }
    else    /* 有效的 buf */
    {
        /* 继续下一次 DMA 传输 */
        jpeg_out_dma_resume((uint32_t)hjpgd.outbuf[hjpgd.outbuf_write_ptr].buf,
                            hjpgd.yuvblk_size);
    }
}
```

该函数用于处理 JPEG 输出数据流，当 JPEG 输出 MDMA 传输完成时，调用该函数。对已满的 buf 标记满，并标记容量，然后切换到下一个 buf。当 buf 不够时，暂停获取 JPEG 输出 FIFO 的数据，并标记暂停；当 buf 足够时，切换到下一个 buf，继续传输。当解码状态结束时，需要手动读取 JPEG_DOR 寄存器的数据。

下面介绍的是 JPEG 整个文件解码完成回调函数，其定义如下：

```
/**
 * @brief       JPEG 整个文件解码完成回调函数
 * @param       无
```

```
 * @retval      无
 */
void jpeg_endofcovert_callback(void)
{
    hjpgd.state = JPEG_STATE_FINISHED;  /* 标记 JPEG 解码完成 */
}
```

该函数在 JPG/JPEG 文件解码结束时调用。该函数处理非常简单，直接将当前解码状态标记为：JPEG 解码完成（JPEG_STATE_FINISHED）即可。

下面介绍的是 JPEG header 解析成功回调函数，其定义如下：

```
/**
 * @brief       JPEG header 解析成功回调函数
 * @param       无
 * @retval      无
 */
void jpeg_hdrover_callback(void)
{
    uint8_t i = 0;
    hjpgd.state=JPEG_STATE_HEADEROK; /* HEADER 获取成功 */
    jpeg_get_info(&hjpgd);           /* 获取 JPEG 相关信息,包括大小,色彩空间,抽样等 */
    picinfo.ImgWidth = hjpgd.Conf.ImageWidth;
    picinfo.ImgHeight = hjpgd.Conf.ImageHeight;

   /* 需要获取 JPEG 基本信息以后,才能根据 jpeg 输出大小和采样方式,来计算输出缓冲大小,
      并启动输出 MDMA */
    switch (hjpgd.Conf.ChromaSubsampling)
    {
        case JPEG_420_SUBSAMPLING:
        /* YUV420,每个 YUV 像素占 1.5 个字节.每次输出 16 行.16*1.5=24 */
            hjpgd.yuvblk_size = 24 * hjpgd.Conf.ImageWidth;
            hjpgd.yuvblk_height = 16;    /* 每次输出 16 行 */
            break;

        case JPEG_422_SUBSAMPLING:
        /* YUV422,每个 YUV 像素占 2 个字节.每次输出 8 行.8*2=16 */
            hjpgd.yuvblk_size = 16 * hjpgd.Conf.ImageWidth;
            hjpgd.yuvblk_height = 8;     /* 每次输出 8 行 */
            break;

        case JPEG_444_SUBSAMPLING:
        /* YUV444,每个 YUV 像素占 3 个字节.每次输出 8 行.8*3=24 */
            hjpgd.yuvblk_size = 24 * hjpgd.Conf.ImageWidth;
            hjpgd.yuvblk_height = 8;     /* 每次输出 8 行 */
            break;
    }
    hjpgd.yuvblk_curheight = 0; /* 当前行计数器清零 */

    for (i = 0; i < JPEG_DMA_OUTBUF_NB; i++)
    {
        /* 有可能会多需要 32 字节内存 */
        hjpgd.outbuf[i].buf = mymalloc(SRAMIN, hjpgd.yuvblk_size + 32);

        if (hjpgd.outbuf[i].buf == NULL)
        {
            hjpgd.state = JPEG_STATE_ERROR; /* HEADER 获取失败 */
        }
    }

    if (hjpgd.outbuf[JPEG_DMA_OUTBUF_NB - 1].buf != NULL) /* 所有 buf 都申请 OK */
    {
        /* 配置输出 DMA */
        jpeg_out_dma_init((uint32_t)hjpgd.outbuf[0].buf, hjpgd.yuvblk_size);
        jpeg_out_dma_start();    /* 启动 DMA OUT 传输,开始接收 JPEG 解码数据流 */
```

```
    }
    piclib_ai_draw_init();
}
```

该函数在 JPEG 头解码成功后调用。该函数先标记状态为 JPEG 头解码成功（JPEG_STATE_HEADEROK），然后调用 jpeg_get_info 函数获取 JPEG 相关信息。在得到 JPEG 文件的抽样方式、图库宽度等信息后，计算：yuvblk_size 和 yuvblk_height 这两个关键参数，然后对申请 outbuf 内存并初始化输出 FIFO MDMA，最后启动输出 FIFO MDMA，开始接收硬件 JPEG 解码后的数据。

这里需要注意：因为 JPEG 输出 FIFO 的 MDMA 是在 JPEG 头解码成功回调函数里面才申请内存并初始化的，因此输入 FIFO 的 MDMA 必须先输入一部分数据给 JPEG 解码内核，才可以执行输出 FIFO MDMA 初始化。但是如果整张图片的大小，都不够一次输入 FIFO MDMA 的传输的话，那么可能会出现解码无法完成的情况，出现这种情况很好解决，就是把输入 FIFO MDMA 的大小改小一点，即 JPEG_DMA_INBUF_LEN 的大小改小一点。

下面介绍的是 JPEG 硬件解码图片函数，其定义如下：

```
/**
 * @brief       JPEG 硬件解码图片
 * @note        注意, 请保证:
 *              1, 待解码图片的分辨率,必须小于等于屏幕的分辨率!
 *              2, 请保证图片的宽度是 16 的倍数,否则解码出错!
 *
 * @param       filename : 包含路径的文件名(.jpeg/jpg)
 * @retval      操作结果
 *   @arg       0    , 成功
 *   @arg       其他, 错误码
 */
uint8_t hjpgd_decode(char *filename)
{
    FIL *ftemp;
    uint16_t *rgb565buf = 0;
    volatile  uint32_t timecnt = 0;
    uint32_t br = 0;
    uint8_t fileover = 0;
    uint8_t i = 0;
    uint8_t res;
    res = jpeg_core_init(&hjpgd);                       /* 初始化 JPEG 内核 */

    if (res)return 1;

    ftemp = (FIL *)mymalloc(SRAMITCM, sizeof(FIL)); /* 申请内存 */

    if (f_open(ftemp, filename, FA_READ) != FR_OK)   /* 打开图片失败 */
    {
        jpeg_core_destroy(&hjpgd);
        myfree(SRAMITCM, ftemp);                        /* 释放内存 */
        return 2;
    }

    rgb565buf = mymalloc(SRAMEX, lcddev.width * lcddev.height * 2);
    /* 申请整帧内存 */
    jpeg_decode_init(&hjpgd);        /* 初始化硬件 JPEG 解码器 */

    for (i = 0; i < JPEG_DMA_INBUF_NB; i++)
    {
        res = f_read(ftemp, hjpgd.inbuf[i].buf, JPEG_DMA_INBUF_LEN, &br);
        /* 填满所有输入数据缓冲区 */

        if (res == FR_OK && br)
        {
            hjpgd.inbuf[i].size = br;    /* 读取 */
            hjpgd.inbuf[i].sta = 1;      /* 标记 buf 满 */
```

```
        }

        if (br == 0)break;
    }

    jpeg_in_dma_init((uint32_t)hjpgd.inbuf[0].buf, hjpgd.inbuf[0].size);
    /* 配置输入 DMA */
    jpeg_in_callback = jpeg_dma_in_callback;       /* JPEG DMA 读取数据回调函数 */
    jpeg_out_callback = jpeg_dma_out_callback;     /* JPEG DMA 输出数据回调函数 */
    jpeg_eoc_callback = jpeg_endofcovert_callback;/* JPEG 解码结束回调函数 */
    jpeg_hdp_callback = jpeg_hdrover_callback;  /* JPEG Header 解码完成回调函数 */
    jpeg_in_dma_start();     /* 启动 DMA IN 传输,开始解码 JPEG 图片 */

    while (1)
    {
        if (hjpgd.inbuf[hjpgd.inbuf_write_ptr].sta == 0 && fileover == 0)
        /* 有 buf 为空 */
        {
            res = f_read(ftemp, hjpgd.inbuf[hjpgd.inbuf_write_ptr].buf,
                  JPEG_DMA_INBUF_LEN, &br);    /* 填满一个缓冲区 */

            if (res == FR_OK && br)
            {
                hjpgd.inbuf[hjpgd.inbuf_write_ptr].size = br;    /* 读取 */
                hjpgd.inbuf[hjpgd.inbuf_write_ptr].sta = 1;      /* buf 满 */
            }
            else if (br == 0)
            {
                timecnt = 0;     /* 清零计时器 */
                fileover = 1;    /* 文件结束了 */
            }

            if (hjpgd.indma_pause == 1 && hjpgd.inbuf
               [hjpgd.inbuf_read_ptr].sta == 1)    /* 之前是暂停的了,继续传输 */
            {
                jpeg_in_dma_resume((uint32_t)hjpgd.inbuf
               [hjpgd.inbuf_read_ptr].buf, hjpgd.inbuf
               [hjpgd.inbuf_read_ptr].size);       /* 继续下一次 DMA 传输 */
                hjpgd.indma_pause = 0;
            }

            hjpgd.inbuf_write_ptr++;

            if (hjpgd.inbuf_write_ptr >=
           JPEG_DMA_INBUF_NB)hjpgd.inbuf_write_ptr = 0;
        }

        if (hjpgd.outbuf[hjpgd.outbuf_read_ptr].sta == 1)
        /* buf 里面有数据要处理 */
        {
            SCB_CleanInvalidateDCache();     /* 清空 D catch */
            jpeg_dma2d_yuv2rgb_conversion(&hjpgd, (uint32_t *)rgb565buf);
            /* 利用 DMA2D,将 YUV 图像转成 RGB565 图像 */

            //SCB_CleanInvalidateDCache();                    /* 清空 D catch */
            hjpgd.outbuf[hjpgd.outbuf_read_ptr].sta = 0;     /* 标记 buf 为空 */
            hjpgd.outbuf[hjpgd.outbuf_read_ptr].size = 0;    /* 数据量清空 */
            hjpgd.outbuf_read_ptr++;

            if (hjpgd.outbuf_read_ptr >= JPEG_DMA_OUTBUF_NB)
                hjpgd.outbuf_read_ptr = 0;  /* 限制范围 */

            if (hjpgd.yuvblk_curheight >= hjpgd.Conf.ImageHeight)break;
```

```
        /* 当前高度等于或者超过图片分辨率的高度,则说明解码完成了,直接退出 */
        }
        else if (hjpgd.outdma_pause == 1 && hjpgd.outbuf
        [hjpgd.outbuf_write_ptr].sta == 0)
        /* out 暂停,且当前 writebuf 已经为空了,则恢复 out 输出 */
        {
            jpeg_out_dma_resume((uint32_t)hjpgd.outbuf
            [hjpgd.outbuf_write_ptr].buf, hjpgd.yuvblk_size);
            /* 继续下一次 DMA 传输 */
            hjpgd.outdma_pause = 0;
        }

        timecnt++;

        if (hjpgd.state == JPEG_STATE_ERROR)     /* 解码出错,直接退出 */
        {
            res = 2;
            break;
        }

        if (fileover)    /* 文件结束后,及时退出,防止死循环 */
        {
            if (hjpgd.state == JPEG_STATE_NOHEADER && hjpgd.indma_pause == 1)
            /* 当前处于暂停状态，且没有解析到 JPEG 头 */
            {
                break;  /* 解码 JPEG 头失败了 */
            }

            if (timecnt > 0X3FFFF)break;     /* 超时退出 */
        }
    }

    if (hjpgd.state == JPEG_STATE_FINISHED) /* 解码完成了, 一次性输出到 LCD */
    {
        pic_phy.fillcolor(picinfo.S_XOFF, picinfo.S_YOFF,
        hjpgd.Conf.ImageWidth, hjpgd.Conf.ImageHeight, rgb565buf);
    }

    f_close(ftemp);              /* 关闭文件 */
    myfree(SRAMITCM, ftemp);     /* 释放申请的内存 */
    myfree(SRAMEX, rgb565buf);   /* 释放内存 */
    jpeg_core_destroy(&hjpgd);   /* 结束 JPEG 解码,释放内存 */
    return res;
}
```

该函数用于解码一张 JPG/JPEG 图片。该函数采用的思路就是按照我们在 58.3.1 节介绍的步骤来解码 JPG/JPEG 图片。请大家参考前面的介绍和源码进行理解。

另外，我们需要将 hjpgd_decode 函数加入到图片解码库里面，修改 piclib_ai_load_picfile 函数代码，具体如下：

```
/**
 * @brief       智能画图
 * @note        图片仅在 x,y 和 width, height 限定的区域内显示.
 *
 * @param       filename        : 包含路径的文件名(.bmp/.jpg/.jpeg/.gif 等)
 * @param       x, y            : 起始坐标
 * @param       width, height   : 显示区域
 * @param       fast            : 使能快速解码
 *   @arg                         0, 不使能
 *   @arg                         1, 使能
 *   @note                        图片尺寸小于等于液晶分辨率,才支持快速解码
 * @retval      无
 */
uint8_t piclib_ai_load_picfile(char *filename, uint16_t x, uint16_t y,
```

```
                                uint16_t width, uint16_t height, uint8_t fast)
{
    uint8_t res;/* 返回值 */
    uint8_t temp;

    if ((x + width) > picinfo.lcdwidth)return PIC_WINDOW_ERR; /* x 坐标超范围了 */
    if ((y + height)> picinfo.lcdheight)return PIC_WINDOW_ERR;/* y 坐标超范围了 */
    /* 得到显示方框大小 */
    if (width == 0 || height == 0)return PIC_WINDOW_ERR;       /* 窗口设定错误 */

    picinfo.S_Height = height;
    picinfo.S_Width = width;

    /* 显示区域无效 */
    if (picinfo.S_Height == 0 || picinfo.S_Width == 0)
    {
        picinfo.S_Height = lcddev.height;
        picinfo.S_Width = lcddev.width;
        return FALSE;
    }

    if (pic_phy.fillcolor == NULL)fast = 0; /* 颜色填充函数未实现,不能快速显示 */
    /* 显示的开始坐标点 */
    picinfo.S_YOFF = y;
    picinfo.S_XOFF = x;
    /* 文件名传递 */
    temp = exfuns_file_type((uint8_t *)filename);    /* 得到文件的类型 */
    switch (temp)
    {
        case T_BMP:
            res = stdbmp_decode(filename);       /* 解码 bmp */
            break;
        case T_JPG:
        case T_JPEG:
            if (fast)    /* 可能需要硬件解码 */
            {
                res = jpg_get_size(filename, &picinfo.ImgWidth,
                                   &picinfo.ImgHeight);

                if (res == 0)
                {
             /* 满足分辨率小于等于屏幕分辨率、满足图片宽度为 16 的整数倍，则可以硬件解码 */
                    if (picinfo.ImgWidth <= lcddev.width && picinfo.ImgHeight
                     <= lcddev.height && picinfo.ImgWidth <= picinfo.S_Width
                     && picinfo.ImgHeight <= picinfo.S_Height &&
                        (picinfo.ImgWidth % 16) == 0)
                    {
                        /* 采用硬解码 JPG/JPEG */
                        res = hjpgd_decode(filename);
                    }
                    else
                    {
                      res = jpg_decode(filename, fast);/* 采用软件解码 JPG/JPEG */
                    }
                }
            }
            else
            {
                res = jpg_decode(filename, fast);    /* 统一采用软件解码 JPG/JPEG */
            }
            break;
        case T_GIF:
            res = gif_decode(filename, x, y, width, height);     /* 解码 gif */
            break;
```

```
        default:
            res = PIC_FORMAT_ERR;                   /* 非图片格式!!! */
            break;
    }
    return res;
}
```

启用快速解码时，当 JPG/JPEG 图片尺寸满足小于等于屏幕分辨率，且图片宽度是 16 的倍数，则我们会通过调用 hjpgd_decode 函数实现硬件 JPEG 解码，从而大大提高速度。

PICTURE 驱动代码就介绍到这里。

3. main.c 代码

pic_get_tnum 函数，我们在上一个实验已将介绍过了，用来得到 path 路径下，所有有效文件（图片文件）的个数。接下来介绍的是 main 函数，其定义如下：

```
int main(void)
{
    uint8_t res;
    DIR picdir;                 /* 图片目录 */
    FILINFO *picfileinfo;       /* 文件信息 */
    char *pname;                /* 带路径的文件名 */
    uint16_t totpicnum;         /* 图片文件总数 */
    uint16_t curindex;          /* 图片当前索引 */
    uint8_t key;                /* 键值 */
    uint8_t pause = 0;          /* 暂停标记 */
    uint8_t t;
    uint16_t temp;
    uint32_t *picoffsettbl;     /* 图片文件 offset 索引表 */

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    usmart_dev.init(200);                   /* 初始化 USMART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    my_mem_init(SRAMIN);                    /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                    /* 初始化外部内存池 */
    my_mem_init(SRAM12);                    /* 初始化 SRAM12 内存池(SRAM1+SRAM2) */
    my_mem_init(SRAM4);                     /* 初始化 SRAM4 内存池(SRAM4) */
    my_mem_init(SRAMDTCM);                  /* 初始化 DTCM 内存池(DTCM) */
    my_mem_init(SRAMITCM);                  /* 初始化 ITCM 内存池(ITCM) */
    exfuns_init();                          /* 为 fatfs 相关变量申请内存 */
    f_mount(fs[0], "0:", 1);                /* 挂载 SD 卡 */
    f_mount(fs[1], "1:", 1);                /* 挂载 SPI FLASH */
    f_mount(fs[2], "2:", 1);                /* 挂载 NAND FLASH */

    while (fonts_init())                    /* 检查字库 */
    {
        lcd_show_string(30, 50, 200, 16, 16, "Font Error!", RED);
        delay_ms(200);
        lcd_fill(30, 50, 240, 66, WHITE);   /* 清除显示 */
        delay_ms(200);
    }
    text_show_string(30, 50, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
    text_show_string(30, 70, 200, 16, "硬件 JPEG 解码 实验", 16, 0, RED);
```

```
    text_show_string(30, 90, 200, 16, " 正点原子@ALIENTEK", 16, 0, RED);
    text_show_string(30, 110, 200, 16, "KEY0:NEXT KEY1:PREV", 16, 0, RED);
    text_show_string(30, 130, 200, 16, "KEY_UP:PAUSE", 16, 0, RED);

    while (f_opendir(&picdir, "0:/PICTURE"))     /* 打开图片文件夹 */
    {
        text_show_string(30, 150, 240, 16, "PICTURE 文件夹错误!", 16, 0, RED);
        delay_ms(200);
        lcd_fill(30, 150, 240, 186, WHITE);      /* 清除显示 */
        delay_ms(200);
    }
    totpicnum = pic_get_tnum("0:/PICTURE");    /* 得到总有效文件数 */
    while (totpicnum == NULL)    /* 图片文件为 0 */
    {
        text_show_string(30, 150, 240, 16, "没有图片文件!", 16, 0, RED);
        delay_ms(200);
        lcd_fill(30, 150, 240, 186, WHITE); /* 清除显示 */
        delay_ms(200);
    }
    picfileinfo = (FILINFO *)mymalloc(SRAMIN, sizeof(FILINFO)); /* 申请内存 */
    pname = mymalloc(SRAMIN, FF_MAX_LFN * 2 + 1);    /* 为带路径的文件名分配内存 */
    /* 申请 4*totpicnum 个字节的内存,用于存放图片索引 */
    picoffsettbl = mymalloc(SRAMIN, 4 * totpicnum);
    while (!picfileinfo || !pname || !picoffsettbl) /* 内存分配出错 */
    {
        text_show_string(30, 150, 240, 16, "内存分配失败!", 16, 0, RED);
        delay_ms(200);
        lcd_fill(30, 150, 240, 186, WHITE);            /* 清除显示 */
        delay_ms(200);
    }
    /* 记录索引 */
    res = f_opendir(&picdir, "0:/PICTURE");            /* 打开目录 */
    if (res == FR_OK)
    {
        curindex = 0;    /* 当前索引为 0 */
        while (1)        /* 全部查询一遍 */
        {
            temp = picdir.dptr; /* 记录当前 dptr 偏移 */
            res = f_readdir(&picdir, picfileinfo);  /* 读取目录下的一个文件 */
            /* 错误了/到末尾了,退出 */
            if (res != FR_OK || picfileinfo->fname[0] == 0)break;
            res = exfuns_file_type((uint8_t *)picfileinfo->fname);
            if ((res & 0XF0) == 0X50)    /* 取高四位,看看是不是图片文件 */
            {
                picoffsettbl[curindex] = temp;  /* 记录索引 */
                curindex++;
            }
        }
    }
    text_show_string(30, 150, 240, 16, "开始显示...", 16, 0, RED);
    delay_ms(1500);
    piclib_init();  /* 初始化画图 */
    curindex = 0;    /* 从 0 开始显示 */
    res = f_opendir(&picdir, (const TCHAR *)"0:/PICTURE");  /* 打开目录 */

    while (res == FR_OK)    /* 打开成功 */
    {
        dir_sdi(&picdir, picoffsettbl[curindex]);      /* 改变当前目录索引 */
        res = f_readdir(&picdir, picfileinfo);        /* 读取目录下的一个文件 */
        /* 错误了/到末尾了,退出 */
        if (res != FR_OK || picfileinfo->fname[0] == 0)break;
        strcpy((char *)pname, "0:/PICTURE/");          /* 复制路径(目录) */
```

```
        /* 将文件名接在后面 */
        strcat((char *)pname, (const char *)picfileinfo->fname);
        lcd_clear(BLACK);
        /* 显示图片 */
        piclib_ai_load_picfile(pname, 0, 0, lcddev.width, lcddev.height, 1);
        /* 显示图片名字 */
        text_show_string(2, 2, lcddev.width, 16, (char*)pname, 16, 1, RED);
        t = 0;

        while (1)
        {
            key = key_scan(0);          /* 扫描按键 */
            if (t > 250)key = 1;        /* 模拟一次按下 KEY0 */
            if ((t % 20) == 0)
            {
                LED0_TOGGLE();          /* LED0 闪烁,提示程序正在运行. */
            }

            if (key == KEY1_PRES)       /* 上一张 */
            {
                if (curindex)
                {
                    curindex--;
                }
                else
                {
                    curindex = totpicnum - 1;
                }

                break;
            }
            else if (key == KEY0_PRES)  /* 下一张 */
            {
                curindex++;
                if(curindex >= totpicnum)curindex=0;/* 到末尾的时候,自动从头开始 */
                break;
            }
            else if (key == WKUP_PRES)
            {
                pause = !pause;
                LED1(!pause);    /* 暂停的时候 LED1 亮. */
            }
            if (pause == 0)t++;
            delay_ms(10);
        }
        res = 0;
    }

    myfree(SRAMIN, picfileinfo);        /* 释放内存 */
    myfree(SRAMIN, pname);              /* 释放内存 */
    myfree(SRAMIN, picoffsettbl);       /* 释放内存 */
}
```

main 函数里面我们通过读/写偏移量（图片文件在 PICTURE 文件夹下的读/写偏移位置，可以看做是一个索引），来查找上一个/下一个图片文件（使用 dir_sdi 函数）。通过 piclib_ai_load_picfile 函数，实现对 JPG/JPEG 图片的解码。这里将 fast 参数设置为 1，当图片文件的分辨率小于等于液晶分辨率的时候，将使用硬件 JPEG 进行解码。

至此本例程代码编写完成。最后，本实验可以通过 USMART 来调用相关函数，以对比性能。将 mf_scan_files、piclib_ai_load_picfile 和 hjpgd_decode 等函数添加到 USMART 管理，即可以通过串口调用这几个函数，测试对比软件 JPEG 解码和硬件 JPEG 解码的速度差别。

58.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示了一些实验信息之后就开始显示图片（假设 SD 卡及文件都准备好了，即：在 SD 卡根目录新建：PICTURE 文件夹，并存放一些图片文件(.bmp/.jpg/.gif)在该文件夹内），如图 58.4.1 所示：

图 58.4.1 硬件 JPEG 解码实验显示效果

按 KEY0 和 KEY1 可以快速切换到下一张或上一张，KEY_UP 按键可以暂停自动播放，同时 LED1 亮，指示处于暂停状态，再按一次 KEY_UP 则继续播放。对比上一章实验，我们可以发现，对于小尺寸的 JPG/JPEG 图片（小于液晶分辨率），本例程解码速度明显提升。

我们通过 USMART 调用 piclib_ai_load_picfile 函数，对比测试同一张图片，使用硬件 JPEG 解码和不使用硬件 JPEG 解码，速度差别明显，如图 58.4.2 所示：

```
XCOM V2.7

Run Time Calculation ON


mf_scan_files("0:")
0:/System Volume Information
0:/PHOTO
0:/RECORDER
0:/NAND测试文件.rar
0:/readme.txt
0:/测试用文件.rar
0:/PICTURE
0:/SYSTEM
0:/MUSIC
0:/VIDEO
=0X0;
Function Run Time:16.3ms

piclib_ai_load_picfile("0:/PICTURE/示例图片.jpg",0X0,0X0,0X1E0,0X320,0X0)=0X0;
Function Run Time:580.9ms

piclib_ai_load_picfile("0:/PICTURE/示例图片.jpg",0X0,0X0,0X1E0,0X320,0X1)=0X0;
Function Run Time:83.7ms
```

图 58.4.2 硬件 JPEG 与软件 JPEG 解码速度对比

上图，是我们使用 4.3 寸 800*480 分辨率的 MCU 屏做的测试，可以看出，对于同一张图片（图片分辨率：800*480），硬件 JPEG 解码，只需要 83.7ms，软件 JPEG 解码，则需要 580.9ms！硬件 JPEG 解码速度是软件 JPEG 解码的 6 倍左右!！可见，硬件 JPEG 解码大大提高了对 JPG/JPEG 图片的解码能力。

第五十九章 照相机实验

上一章，我们学习了如何使用 STM32H743 自带的硬件 JPEG 编解码器，实现对 JPG/JPEG 图片的硬解码，从而大大提高解码速度。本章我们将学习 BMP&JPEG 编码，结合前面的摄像头实验，实现一个简单的照相机。

本章分为如下几个小节：

59.1 BMP&JPEG 编码简介
59.2 硬件设计
59.3 程序设计
59.4 下载验证

59.1 BMP&JPEG 编码简介

我们要实现支持 BMP 图片格式的照片和 JPEG 图片格式的照片的照相机功能，这里简单介绍一下这两种图片格式的编码。这里我们使用 ATK-OV5640-AF 摄像头，来实现拍照。关于 OV5640 的相关知识点，请参考第五十一章。

59.1.1 BMP 编码简介

前面的章节中，我们学习了各种图片格式的解码。本章，我们介绍最简单的图片编码方法：BMP 图片编码。通过前面的了解，我们知道 BMP 文件是由文件头、位图信息头、颜色信息和图形数据等四部分组成。我们先来了解下这几个部分。

1、BMP 文件头（14 字节）：BMP 文件头数据结构含有 BMP 文件的类型、文件大小和位图起始位置等信息。

```
/* BMP 头文件 */
typedef __packed struct
{
    uint16_t  bfType ;       /* 文件标志.只对'BM',用来识别 BMP 位图类型 */
    uint32_t  bfSize ;       /* 文件大小,占四个字节 */
    uint16_t  bfReserved1 ; /* 保留 */
    uint16_t  bfReserved2 ; /* 保留 */
    uint32_t  bfOffBits ;    /* 从文件开始到位图数据(bitmap data)开始之间的的偏移量 */
}BITMAPFILEHEADER ;
```

2、位图信息头（40 字节）：BMP 位图信息头数据用于说明位图的尺寸等信息。

```
/* BMP 信息头 */
typedef __packed struct
{
    uint32_t biSize ;          /* 说明 BITMAPINFOHEADER 结构所需要的字数。 */
    long  biWidth ;            /* 说明图象的宽度，以象素为单位 */
    long  biHeight ;           /* 说明图象的高度，以象素为单位 */
    uint16_t  biPlanes ;       /* 为目标设备说明位面数，其值将总是被设为 1 */
    uint16_t  biBitCount ;     /* 说明比特数/象素，其值为 1、4、8、16、24、或 32 */
    uint32_t biCompression ;  /* 说明图象数据压缩的类型。其值可以是下述值之一
                               * BI_RGB   : 没有压缩
                               * BI_RLE8 : 每个象素 8 比特的 RLE 压缩编码，压缩格式由
                                           2 字节组成(重复象素计数和颜色索引)
                               * BI_RLE4 : 每个象素 4 比特的 RLE 压缩编码，压缩格式由
                                           2 字节组成
                               * BI_BITFIELDS：每个象素的比特由指定的掩码决定
                               */
    uint32_t biSizeImage;/*说明图象的大小，以字节为单位。当用 BI_RGB 格式时，可设置为 0*/
    long  biXPelsPerMeter ; /* 说明水平分辨率，用象素/米表示 */
    long  biYPelsPerMeter ; /* 说明垂直分辨率，用象素/米表示 */
```

```
    uint32_t biClrUsed ;     /* 说明位图实际使用的彩色表中的颜色索引数 */
    /* 说明对图象显示有重要影响的颜色索引的数目，如果是 0，表示都重要 */
    uint32_t biClrImportant ;
}BITMAPINFOHEADER ;
```

3、颜色表：颜色表用于说明位图中的颜色，它有若干个表项，每一个表项是一个 RGBQUAD 类型的结构，定义一种颜色。

```
/* 彩色表  */
typedef __packed struct
{
    uint8_t rgbBlue ;               /* 指定蓝色强度 */
    uint8_t rgbGreen ;              /* 指定绿色强度 */
    uint8_t rgbRed ;                /* 指定红色强度 */
    uint8_t rgbReserved ;           /* 保留，设置为 0 */
}RGBQUAD ;
```

颜色表中 RGBQUAD 结构数据的个数由 biBitCount 来确定：当 biBitCount=1、4、8 时，分别有 2、16、256 个表项；当 biBitCount 大于 8 时，没有颜色表项。

BMP 文件头、位图信息头和颜色表组成位图信息（我们将 BMP 文件头也加进来，方便处理），BITMAPINFO 结构定义如下:

```
/* 位图信息头 */
typedef __packed struct
{
    BITMAPFILEHEADER bmfHeader;
    BITMAPINFOHEADER bmiHeader;
    uint32_t RGB_MASK[3];        /* 调色板用于存放 RGB 掩码 */
    //RGBQUAD bmiColors[256];
}BITMAPINFO;
```

4、位图数据：位图数据记录了位图的每一个像素值，记录顺序是在扫描行内是从左到右，扫描行之间是从下到上。位图的一个像素值所占的字节数：

当 biBitCount=1 时，8 个像素占 1 个字节；
当 biBitCount=4 时，2 个像素占 1 个字节；
当 biBitCount=8 时，1 个像素占 1 个字节；
当 biBitCount=16 时，1 个像素占 2 个字节；
当 biBitCount=24 时，1 个像素占 3 个字节；
当 biBitCount=32 时，1 个像素占 4 个字节；

biBitCount=1 表示位图最多有两种颜色，缺省情况下是黑色和白色，你也可以自己定义这两种颜色。图像信息头装调色板中将有两个调色板项，称为索引 0 和索引 1。图象数据阵列中的每一位表示一个像素。如果一个位是 0，显示时就使用索引 0 的 RGB 值，如果位是 1，则使用索引 1 的 RGB 值。

biBitCount=16 表示位图最多有 65536 种颜色。每个像素用 16 位（2 个字节）表示。这种格式叫作高彩色，或叫增强型 16 位色，或 64K 色。它的情况比较复杂，当 biCompression 成员的值是 BI_RGB 时，它没有调色板。16 位中，最低的 5 位表示蓝色分量，中间的 5 位表示绿色分量，高的 5 位表示红色分量，一共占用了 15 位，最高的一位保留，设为 0。这种格式也被称作 555 16 位位图。如果 biCompression 成员的值是 BI_BITFIELDS，那么情况就复杂了，首先是原来调色板的位置被三个 DWORD 变量占据，称为红、绿、蓝掩码。分别用于描述红、绿、蓝分量在 16 位中所占的位置。在 Windows 95（或 98）中，系统可接受两种格式的位域：555 和 565，在 555 格式下，红、绿、蓝的掩码分别是：0x7C00、0x03E0、0x001F，而在 565 格式下，它们则分别为：0xF800、0x07E0、0x001F。你在读取一个像素之后，可以分别用掩码“与”上像素值，从而提取出想要的颜色分量（当然还要再经过适当的左右移操作）。在 NT 系统中，则没有格式限制，只不过要求掩码之间不能有重叠。（注：这种格式的图像使用起来是比较麻烦的，不过因为它的显示效果接近于真彩，而图像数据又比真彩图像小的多，所以，它更多的被用于游戏软件）。

biBitCount=32 表示位图最多有 4294967296(2 的 32 次方)种颜色。这种位图的结构与 16 位位图结构非常类似，当 biCompression 成员的值是 BI_RGB 时，它也没有调色板，32 位中有 24 位用于存放 RGB 值，顺序是：最高位—保留，红 8 位、绿 8 位、蓝 8 位。这种格式也被成为

888 32 位图。如果 biCompression 成员的值是 BI_BITFIELDS 时，原来调色板的位置将被三个 DWORD 变量占据，成为红、绿、蓝掩码，分别用于描述红、绿、蓝分量在 32 位中所占的位置。在 Windows 95(or 98)中，系统只接受 888 格式，也就是说三个掩码的值将只能是：0xFF0000、0xFF00、0xFF。而 NT 系统，只要注意使掩码之间不产生重叠就行。（注：这种图像格式比较规整，因为它是 DWORD 对齐的，所以在内存中进行图像处理时可进行汇编级的代码优化（简单）。

通过以上了解，我们对 BMP 有了一个比较深入的了解，本章，我们采用 16 位 BMP 编码（因为我们的 LCD 就是 16 位色的，而且 16 位 BMP 编码比 24 位 BMP 编码更省空间），故我们需要设置 biBitCount 的值为 16，这样得到新的位图信息（BITMAPINFO）结构体：

```
/* 位图信息头 */
typedef __packed struct
{
    BITMAPFILEHEADER bmfHeader;
    BITMAPINFOHEADER bmiHeader;
    uint32_t RGB_MASK[3];          /* 调色板用于存放 RGB 掩码 */
}BITMAPINFO;
```

其实就是颜色表由 3 个 RGB 掩码代替。最后，我们来看看将 LCD 的显存保存为 BMP 格式的图片文件的步骤：

1）创建 BMP 位图信息，并初始化各个相关信息

这里，我们要设置 BMP 图片的分辨率为 LCD 分辨率、BMP 图片的大小（整个 BMP 文件大小）、BMP 的像素位数（16 位）和掩码等信息。

2）创建新 BMP 文件，写入 BMP 位图信息

我们要保存 BMP，当然要存放在某个地方（文件），所以需要先创建文件，同时先保存 BMP 位图信息，之后才开始 BMP 数据的写入。

3）保存位图数据。

这里就比较简单了，只需要从 LCD 的 GRAM 里面读取各点的颜色值，依次写入第二步创建的 BMP 文件即可。注意：保存顺序（即读 GRAM 顺序）是从左到右，从下到上。

4）关闭文件。

使用 FATFS，在文件创建之后，必须调用 f_close，文件才会真正体现在文件系统里面，否则是不会写入的！这个要特别注意，写完之后，一定要调用 f_close。

BMP 编码就介绍到这里。

59.1.2 JPEG 编码简介

JPEG（Joint Photographic Experts Group）是一个由 ISO 和 IEC 两个组织机构联合组成的一个专家组，负责制定静态的数字图像数据压缩编码标准，这个专家组开发的算法称为 JPEG 算法，并且成为国际上通用的标准，因此又称为 JPEG 标准。JPEG 是一个适用范围很广的静态图像数据压缩标准，既可用于灰度图像又可用于彩色图像。

JPEG 专家组开发了两种基本的压缩算法，一种是采用以离散余弦变换（Discrete Cosine Transform，DCT）为基础的有损压缩算法，另一种是采用以预测技术为基础的无损压缩算法。使用有损压缩算法时，在压缩比为 25:1 的情况下，压缩后还原得到的图像与原始图像相比较，非图像专家难于找出它们之间的区别，因此得到了广泛的应用。

JPEG 压缩是有损压缩，它利用了人的视角系统的特性，使用量化和无损压缩编码相结合来去掉视角的冗余信息和数据本身的冗余信息。

JPEG 压缩编码分为三个步骤：

1）使用正向离散余弦变换（Forward Discrete Cosine Transform，FDCT）把空间域表示的图变换成频率域表示的图。

2）使用加权函数对 DCT 系数进行量化，这个加权函数对于人的视觉系统是最佳的。

3）使用霍夫曼可变字长编码器对量化系数进行编码。

这里我们不详细介绍 JPEG 压缩的过程了，大家可以自行查找相关资料。我们本实验要实现的 JPEG 拍照，并不需要自己压缩图像，因为我们使用的正点原子 OV5640 摄像头模块，直接就可以输出压缩后的 JPEG 数据，我们完全不需要理会压缩过程，所以本实验我们实现 JPEG

拍照的关键，在于准确接收 OV5640 摄像头模块发送过来的编码数据，然后将这些数据保存为.jpg 文件，就可以实现 JPEG 拍照了。

在第五十一章的摄像头实验中，我们定义了一个很大的数组 jpeg_data_buf（4MB 字节）来存储 JPEG 图像数据。而在本实验中，我们可以使用内存管理来申请内存，无需定义这么大的数组，使用上更加灵活。另外，DCMI 接口使用 DMA 直接传输 JPEG 数据到外部 SDRAM 会出现数据丢失，所以 DMA 接收 JPEG 只能用内部 SRAM，然后再拷贝到外部 SDRAM。所以，我们本实验将使用 DMA 的双缓冲机制来读取，DMA 双缓冲读取 JPEG 数据框图如图 59.1.2.1 所示：

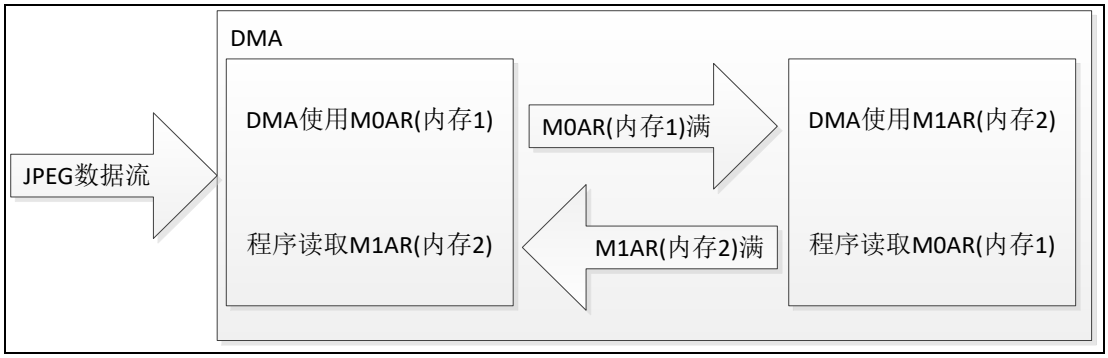


图 59.1.2.1 DMA 双缓冲读取 JPEG 数据原理框图

DMA 接收来自 OV5640 的 JPEG 数据流，首先使用 M0AR（内存 1）来存储，当 M0AR 满了以后，自动切换到 M1AR（内存 2），同时程序读取 M0AR（内存 1）的数据到外部 SDRAM；当 M1AR 满了以后，又切回 M0AR，同时程序读取 M1AR（内存 2）的数据到外部 SDRAM；依次循环（此时的数据处理，是通过 DMA 传输完成中断实现的，在中断里面处理），直到帧中断，结束一帧数据的采集，读取剩余数据到外部 SDRAM，完成一次 JPEG 数据的采集。

这里，M0AR，M1AR 所指向的内存，必须是内部内存，不过由于采用了双缓冲机制，我们就不必定义一个很大的数组，一次性接收所有 JPEG 数据了，而是可以分批次接收，数组可以定义的比较小。

最后，将存储在外部 SDRAM 的 jpeg 数据，保存为.jpg/.jpeg 存放在 SD 卡，就完成了一次 JPEG 拍照。

59.2 硬件设计

1. 例程功能

1、首先是检测字库，然后检测 SD 卡根目录是否存在 PHOTO 文件夹，如果不存在则创建，如果创建失败，则报错（提示拍照功能不可用）。在找到 SD 卡的 PHOTO 文件夹后，开始初始化 OV5640，如果初始化成功，则提示信息：KEY0：拍照(bmp 格式)，KEY1:拍照(jpg 格式)，WK_UP 选择：1:1 显示，即不缩放，图片不变形，但是显示区域小（液晶分辨率大小），或者缩放显示，即将 1280*800 的图像压缩到液晶分辨率尺寸显示，图片变形，但是显示了整个图片内容。可以通过串口 1，借助 USMART 设置/读取 OV5640 的寄存器，方便大家调试。

2、LED0 闪烁，提示程序运行。LED1 用于指示帧中断。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
LED1：LED1 - PB0
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)
4）独立按键：KEY0 - PH3、KEY1 - PH2、WK_UP - PA0
5）SD 卡，通过 SDMMC1(SDMMC_D0~D4(PC8~PC11)，

SDMMC_SCK(PC12)，SDMMC_CMD(PD2))连接

6）norflash(QSPI FLASH 芯片，连接在 QSPI 上)

7）DCMI 接口（用于驱动 OV5640 摄像头模块）

8）正点原子 OV5640 摄像头模块，连接关系为：

OV5640 模块 ---------- STM32 开发板
OV_D0~D7 ---------- PC6/PC7/PC8/PC9/PC11/PD3/PB8/PB9
OV_SCL ---------- PB4
OV_SDA ---------- PB3
OV_VSYNC ---------- PB7
OV_HREF ---------- PH8
OV_RESET ---------- PA15
OV_PCLK ---------- PA6
OV_PWDN ---------- PCF8574_P2

59.3 程序设计

59.3.1 程序流程图

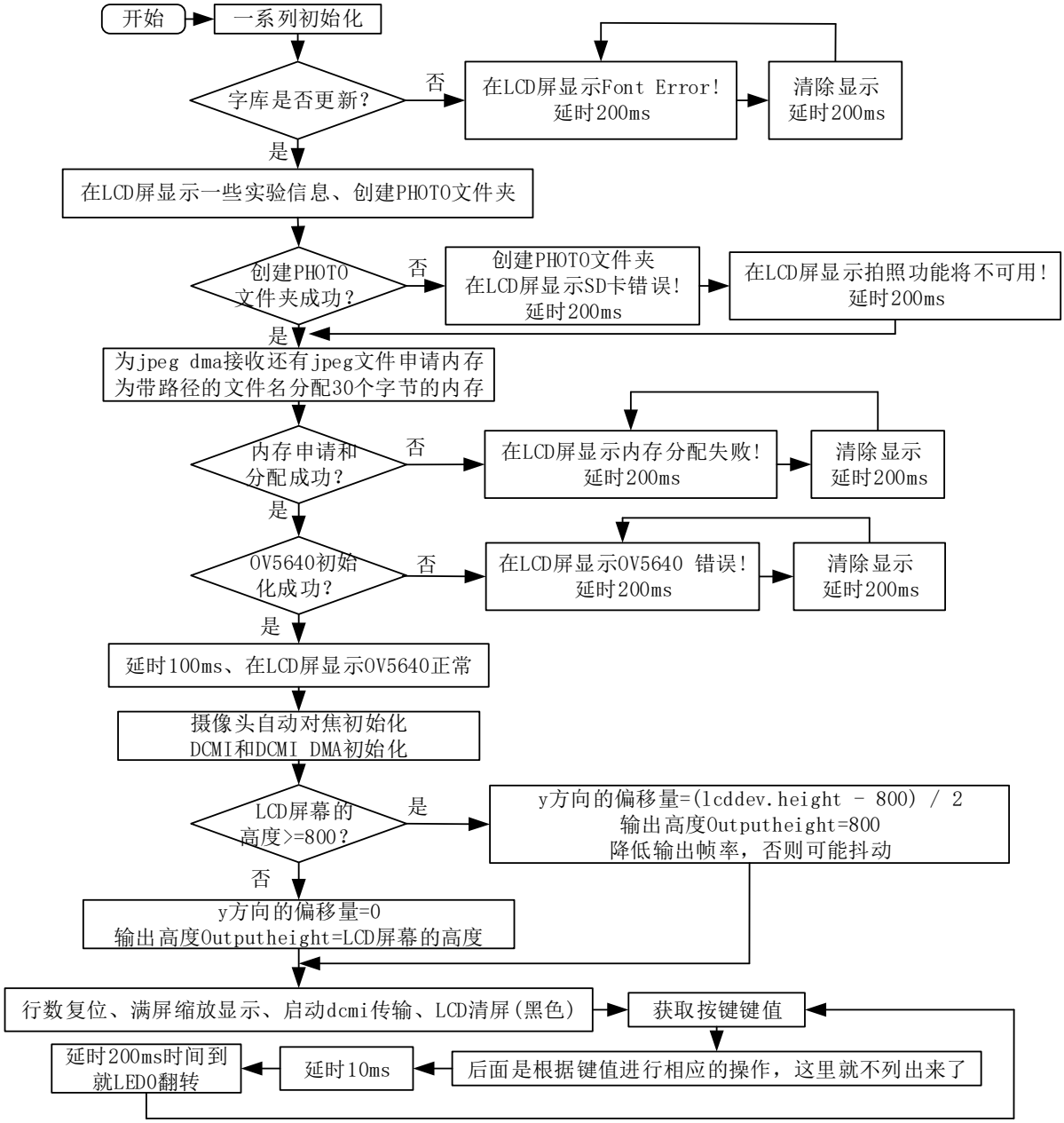


图 59.3.1.1 照相机实验程序流程图

59.3.2 程序解析

1. PICTURE 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。PICTURE 驱动源码包括两个文件：bmp.c 和 bmp.h。

bmp.h 头文件在 59.1.1 小节基本讲过，具体请看源码。下面来看到 bmp.c 文件里面的 bmp 编码函数：bmp_encode，该函数代码如下：

```
/**
 * @brief   BMP 编码函数
```

```
 * @note    将当前 LCD 屏幕的指定区域截图,存为 16 位格式的 BMP 文件 RGB565 格式.
 *          保存为 rgb565 则需要掩码,需要利用原来的调色板位置增加掩码.这里我们已经增加了掩码.
 *          保存为 rgb555 格式则需要颜色转换,耗时间比较久,所以保存为 565 是最快速的办法.
 *
 * @param       filename    : 包含存储路径的文件名(.bmp)
 * @param       x, y        : 起始坐标
 * @param       width,height: 区域大小
 * @param       acolor      : 附加的 alphablend 的颜色(这个仅对 32 位色 bmp 有效!!!)
 * @param       mode        : 保存模式
 *   @arg                   0, 仅仅创建新文件的方式编码;
 *   @arg                   1, 如果之前存在文件,则覆盖之前的文件.如果没有,则创建新的文件;
 * @retval      操作结果
 *   @arg       0   , 成功
 *   @arg       其他, 错误码
 */
uint8_t bmp_encode(uint8_t *filename, uint16_t x, uint16_t y, uint16_t width,
                   uint16_t height, uint8_t mode)
{
    FIL *f_bmp;
    uint32_t bw = 0;
    uint16_t bmpheadsize;   /* bmp 头大小 */
    BITMAPINFO hbmp;        /* bmp 头 */
    uint8_t res = 0;
    uint16_t tx, ty;        /* 图像尺寸 */
    uint16_t *databuf;      /* 数据缓存区地址 */
    uint16_t pixcnt;        /* 像素计数器 */
    uint16_t bi4width;      /* 水平像素字节数 */

    if (width == 0 || height == 0)return PIC_WINDOW_ERR;           /* 区域错误 */
    if ((x + width - 1) > lcddev.width)return PIC_WINDOW_ERR;      /* 区域错误 */
    if ((y + height - 1) > lcddev.height)return PIC_WINDOW_ERR;    /* 区域错误 */

#if BMP_USE_MALLOC == 1     /* 使用 malloc */

   /* 开辟至少 bi4width 大小的字节的内存区域 ,对 240 宽的屏,480 个字节就够了.
     最大支持 1024 宽度的 bmp 编码 */
   databuf = (uint16_t *)piclib_mem_malloc(2048);

    if (databuf == NULL)return PIC_MEM_ERR;             /* 内存申请失败. */
    f_bmp = (FIL *)piclib_mem_malloc(sizeof(FIL));      /* 开辟 FIL 字节的内存区域 */
    if (f_bmp == NULL)      /* 内存申请失败 */
    {
        piclib_mem_free(databuf);
        return PIC_MEM_ERR;
    }
#else
    databuf = (uint16_t *)bmpreadbuf;
    f_bmp = &f_bfile;
#endif
    bmpheadsize = sizeof(hbmp);                         /* 得到 bmp 文件头的大小 */
    my_mem_set((uint8_t *)&hbmp, 0, sizeof(hbmp));      /* 置零空申请到的内存 */
    hbmp.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);   /* 信息头大小 */
    hbmp.bmiHeader.biWidth = width;                     /* bmp 的宽度 */
    hbmp.bmiHeader.biHeight = height;                   /* bmp 的高度 */
    hbmp.bmiHeader.biPlanes = 1;                        /* 恒为 1 */
    hbmp.bmiHeader.biBitCount = 16;                     /* bmp 为 16 位色 bmp */
    hbmp.bmiHeader.biCompression = BI_BITFIELDS;/* 每个象素的比特由指定的掩码决定 */
    hbmp.bmiHeader.biSizeImage = hbmp.bmiHeader.biHeight *
    hbmp.bmiHeader.biWidth * hbmp.bmiHeader.biBitCount / 8;/* bmp 数据区大小 */
    hbmp.bmfHeader.bfType = ((uint16_t)'M' << 8) + 'B';     /* BM 格式标志 */
```

```
    /* 整个 bmp 的大小 */
    hbmp.bmfHeader.bfSize = bmpheadsize + hbmp.bmiHeader.biSizeImage;
    hbmp.bmfHeader.bfOffBits = bmpheadsize; /* 到数据区的偏移 */
    hbmp.RGB_MASK[0] = 0X00F800;            /* 红色掩码 */
    hbmp.RGB_MASK[1] = 0X0007E0;            /* 绿色掩码 */
    hbmp.RGB_MASK[2] = 0X00001F;            /* 蓝色掩码 */

    if (mode == 1)
    {
        /* 尝试打开之前的文件 */
        res = f_open(f_bmp, (const TCHAR *)filename, FA_READ | FA_WRITE);
    }
    if (mode == 0 || res == 0x04)
    {
        /* 模式 0,或者尝试打开失败,则创建新文件 */
        res = f_open(f_bmp, (const TCHAR *)filename, FA_WRITE | FA_CREATE_NEW);
    }
    if ((hbmp.bmiHeader.biWidth * 2) % 4)    /* 水平像素(字节)不为 4 的倍数 */
    {
        /* 实际要写入的宽度像素,必须为 4 的倍数 */
        bi4width = ((hbmp.bmiHeader.biWidth * 2) / 4 + 1) * 4;
    }
    else
    {
        bi4width = hbmp.bmiHeader.biWidth * 2;  /* 刚好为 4 的倍数 */
    }

    if (res == FR_OK)   /* 创建成功 */
    {
        res=f_write(f_bmp, (uint8_t *)&hbmp, bmpheadsize, &bw);/* 写入 BMP 首部*/

        for (ty = y + height - 1; hbmp.bmiHeader.biHeight; ty--)
        {
            pixcnt = 0;
            for (tx = x; pixcnt != (bi4width / 2);)
            {
                if (pixcnt < hbmp.bmiHeader.biWidth)
                {
                   databuf[pixcnt]=pic_phy.read_point(tx, ty);/* 读取坐标点的值 */
                }
                else
                {
                    databuf[pixcnt] = 0Xffff;   /* 补充白色的像素 */
                }
                pixcnt++;
                tx++;
            }
            hbmp.bmiHeader.biHeight--;
            res = f_write(f_bmp, (uint8_t *)databuf, bi4width,&bw);/* 写入数据 */
        }
        f_close(f_bmp);
    }
#if BMP_USE_MALLOC == 1     /* 使用 malloc */
    piclib_mem_free(databuf);
    piclib_mem_free(f_bmp);
#endif
    return res;
}
```

该函数实现了对 LCD 屏幕的任意指定区域进行截屏保存，用到的方法就是 59.1.1 节我们所介绍的方法，该函数实现了将 LCD 任意指定区域的内容，保存为 16 位 BMP 格式，存放在指定位置（由 filename 决定）。注意，代码中的 BMP_USE_MALLOC 是在 bmp.h 定义的一个宏，用于设置是否使用 malloc，本章我们选择使用 malloc。

2. main.c 代码

main.c 前面定义了一些变量和数组，具体如下：

```
/* bmp 拍照请求:0,无 bmp 拍照请求;1,有 bmp 拍照请求,需要在帧中断里面,关闭 DCMI 接口 */
volatile uint8_t g_bmp_request = 0;
uint8_t g_ovx_mode = 0;             /* bit0:0,RGB565 模式;1,JPEG 模式 */
uint16_t g_curline = 0;             /* 摄像头输出数据,当前行编号 */
uint16_t g_yoffset = 0;             /* y 方向的偏移量 */

#define jpeg_buf_size 4*1024*1024   /* 定义 JPEG 数据缓存 jpeg_buf 的大小(440K 字节) */
#define jpeg_line_size  4*1024      /* 定义 DMA 接收数据时,一行数据的最大值 */

uint32_t *p_dcmi_line_buf[2];       /* RGB 屏时,摄像头采用一行一行读取,定义行缓存 */
uint32_t *p_jpeg_data_buf;          /* JPEG 数据缓存 buf */

volatile uint32_t g_jpeg_data_len = 0; /* buf 中的 JPEG 有效数据长度 */

volatile uint8_t g_jpeg_data_ok = 0; /* JPEG 数据采集完成标志
                                      * 0,数据没有采集完;
                                      * 1,数据采集完了,但是还没处理;
                                      * 2,数据已经处理完成了,可以开始下一帧接收
                                      */
```

在 main.c 里面，总共有 7 个函数，我们接下来分别介绍。首先是处理 JPEG 数据函数，其定义如下：

```
/**
 * @brief       处理 JPEG 数据
 * @note        当采集完一帧 JPEG 数据后,调用此函数,切换 JPEG BUF.开始下一帧采集
 * @param       无
 * @retval      无
 */
void jpeg_data_process(void)
{
    uint16_t i,rlen;            /* 剩余数据长度 */
    uint32_t *pbuf;
    g_curline = g_yoffset;      /* 行数复位 */
    if (g_ovx_mode & 0X01)      /* 只有在 JPEG 格式下,才需要做处理 */
    {
        if (g_jpeg_data_ok == 0)     /* jpeg 数据还未采集完 */
        {
            __HAL_DMA_DISABLE(&g_dma_dcmi_handle);    /* 停止当前传输 */
            /* 得到剩余数据长度 */
            while(DMA1_Stream1->CR&0x01)
            rlen=jpeg_line_size-__HAL_DMA_GET_COUNTER(&g_dma_dcmi_handle);
            pbuf=p_jpeg_data_buf + g_jpeg_data_len;/* 偏移到有效数据末尾,继续添加 */
            if (DMA1_Stream1->CR & (1 << 19))
            {
                for (i = 0; i < rlen; i++)
                {
                    pbuf[i] = p_dcmi_line_buf[1][i]; /* 读取 buf1 里面的剩余数据 */
                }
            }
            else
            {
                for (i = 0; i < rlen; i++)
                {
                    pbuf[i] = p_dcmi_line_buf[0][i]; /* 读取 buf0 里面的剩余数据 */
                }
            }
            g_jpeg_data_len += rlen;    /* 加上剩余长度 */
            g_jpeg_data_ok = 1;         /* 标记 JPEG 数据采集完按成,等待其他函数处理 */
        }
```

```
        if (g_jpeg_data_ok == 2)        /* 上一次的 jpeg 数据已经被处理了 */
        {
            /* 传输长度为 jpeg_buf_size*4 字节 */
            __HAL_DMA_SET_COUNTER(&g_dma_dcmi_handle, jpeg_line_size);
            __HAL_DMA_ENABLE(&g_dma_dcmi_handle); /* 重新传输 */
            g_jpeg_data_ok = 0;          /* 标记数据未采集 */
            g_jpeg_data_len = 0;         /* 数据重新开始 */
        }
    }
    else
    {
        if (g_bmp_request == 1)     /* 有 bmp 拍照请求,关闭 DCMI */
        {
            dcmi_stop();            /* 停止 DCMI */
            g_bmp_request = 0;      /* 标记请求处理完成 */
        }
        lcd_set_cursor(0, 0);
        lcd_write_ram_prepare();    /* 开始写入 GRAM */
    }
}
```

该函数用于处理 JPEG 数据的接收，在 DCMI_IRQHandler 函数（在 dcmi.c 里面）里面被调用，它与 jpeg_dcmi_rx_callback 函数和 ov5640_jpg_photo 函数共同控制 JPEG 的数据的采集。JPEG 数据的接收，采用 DMA 双缓冲机制，缓冲数组为：p_dcmi_line_buf（u32 类型，RGB 屏接收 RGB565 数据时，也是用这个数组）；数组大小为：jpeg_line_size，我们定义的是 4*1024，即数组大小为 4K 字节（数组大小不能小于存储摄像头一行输出数据的大小）；JPEG 数据接收处理流程就是按图 59.1.2.1 所示流程来实现的。由 DMA 传输完成中断和 DCMI 帧中断，两个中断服务函数共同完成 jpeg 数据的采集。采集到的 JPEG 数据，全部存储在 p_jpeg_data_buf 数组里面，p_jpeg_data_buf 数组采用内存管理，从外部 SDRAM 申请 4MB 内存作为 JPEG 数据的缓存。

接下来介绍的是 JPEG 数据接收回调函数，其定义如下：

```
/**
 * @brief       JPEG 数据接收回调函数
 * @param       无
 * @retval      无
 */
void jpeg_dcmi_rx_callback(void)
{
    uint16_t i;
    uint32_t *pbuf;
    if (DMA1_Stream1->CR & (1 << 19))               /* buf0 已满,正常处理 buf1 */
    {
        for (i = 0; i < jpeg_line_size; i++)
        {
            pbuf[i] = p_dcmi_line_buf[0][i];        /* 读取 buf0 里面的数据 */
        }
        g_jpeg_data_len += jpeg_line_size;          /* 偏移 */
    }
    else                                            /* buf1 已满,正常处理 buf0 */
    {
        for (i = 0; i < jpeg_line_size; i++)
        {
            pbuf[i] = p_dcmi_line_buf[1][i];        /* 读取 buf1 里面的数据 */
        }
        g_jpeg_data_len += jpeg_line_size;          /* 偏移 */
    }
}
```

这是 jpeg 数据接收的主要函数，通过判断 DMA1_Stream1->CR 寄存器，读取不同 p_dcmi_line_buf 里面的数据，存储到 SDRAM 里面（p_jpeg_data_buf）。该函数由 DMA 的传输完成中断服务函数：DMA1_Stream1_IRQHandler 调用。

接下来介绍的是切换为 OV5640 模式函数，其定义如下：

```
/**
 * @brief       切换为 OV5640 模式
 * @note        切换 PC8/PC9/PC11 为 DCMI 复用功能(AF13)
 * @param       无
 * @retval      无
 */
void sw_ov5640_mode(void)
{
    GPIO_InitTypeDef gpio_init_struct;

    ov5640_write_reg(0X3017, 0XFF); /* 开启 OV5650 输出(可以正常显示) */
    ov5640_write_reg(0X3018, 0XFF);

    /* GPIOC8/9/11 切换为 DCMI 接口 */
    gpio_init_struct.Pin = GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_11;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;              /* 推挽复用 */
    gpio_init_struct.Pull = GPIO_PULLUP;                  /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;   /* 高速 */
    gpio_init_struct.Alternate = GPIO_AF13_DCMI;       /* 复用为 DCMI */
    HAL_GPIO_Init(GPIOC, &gpio_init_struct);           /* 初始化 PC8，9，11 引脚 */
}
```

因为 SD 卡和 OV5640 有几个 IO 共用，所以这几个 IO 需要分时复用。该函数用于切换 GPIO8/9/11 的复用功能为 DCMI 接口，并开启 OV5640，这样摄像头模块，可以开始正常工作。

接下来介绍的是切换为 SD 卡模式函数，其定义如下：

```
/**
 * @brief       切换为 SD 卡模式
 * @note        切换 PC8/PC9/PC11 为 SDMMC 复用功能(AF12)
 * @param       无
 * @retval      无
 */
void sw_sdcard_mode(void)
{
    GPIO_InitTypeDef gpio_init_struct;

    ov5640_write_reg(0X3017, 0X00); /* 关闭 OV5640 全部输出(不影响 SD 卡通信) */
    ov5640_write_reg(0X3018, 0X00);

    /* GPIOC8/9/11 切换为 SDIO 接口 */
    gpio_init_struct.Pin = GPIO_PIN_8 | GPIO_PIN_9 | GPIO_PIN_11;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;              /* 推挽复用 */
    gpio_init_struct.Pull = GPIO_PULLUP;                  /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;   /* 高速 */
    gpio_init_struct.Alternate = GPIO_AF12_SDMMC1;     /* 复用为 SDMMC */
    HAL_GPIO_Init(GPIOC, &gpio_init_struct);           /* 初始化 PC8，9，11 引脚 */
}
```

该数用于切换 GPIO8/9/11 的复用功能为 SDMMC 接口，并关闭 OV5640，这样，SD 卡可以开始正常工作。

接下来介绍的是文件名自增（避免覆盖）函数，其定义如下：

```
/**
 * @brief       文件名自增（避免覆盖）
 * @note        bmp 组合成：形如 "0:PHOTO/PIC13141.bmp" 的文件名
 *              jpg 组合成：形如 "0:PHOTO/PIC13141.jpg" 的文件名
 * @param       pname : 有效的文件名
 * @param       mode  : 0，创建.bmp 文件；  1，创建.jpg 文件；
 * @retval      无
 */
void camera_new_pathname(uint8_t *pname, uint8_t mode)
{
    uint8_t res;
```

```
    uint16_t index = 0;
    FIL *ftemp;
    ftemp = (FIL *)mymalloc(SRAMIN, sizeof(FIL));    /* 开辟 FIL 字节的内存区域 */
    while (index < 0XFFFF)
    {
        if (mode == 0)  /* 创建.bmp 文件名 */
        {
            sprintf((char *)pname, "0:PHOTO/PIC%05d.bmp", index);
        }
        else            /* 创建.jpg 文件名 */
        {
            sprintf((char *)pname, "0:PHOTO/PIC%05d.jpg", index);
        }
        res = f_open(ftemp, (const TCHAR *)pname, FA_READ); /* 尝试打开这个文件 */
        if (res == FR_NO_FILE)break;     /* 该文件名不存在, 正是我们需要的 */
        index++;
    }
    myfree(SRAMIN, ftemp);
}
```

该函数用于生成新的带路径的文件名，且不会重复，防止文件互相覆盖。该函数可以生成.bmp/.jpg 的文件名，方便拍照的时候，保存到 SD 卡里面。

接下来介绍的是 OV5640 拍照 jpg 图片函数，其定义如下：

```
/**
 * @brief       OV5640 拍照 jpg 图片
 * @param       pname : 要创建的 jpg 文件名(含路径)
 * @retval      0, 成功; 其他,错误代码;
 */
uint8_t ov5640_jpg_photo(uint8_t *pname)
{
    FIL *f_jpg;
    uint8_t res = 0, headok = 0;
    uint32_t bwr;
    uint32_t i, jpgstart, jpglen;
    uint8_t *pbuf;

    f_jpg = (FIL *)mymalloc(SRAMIN, sizeof(FIL));    /* 开辟 FIL 字节的内存区域 */
    if (f_jpg == NULL)return 0XFF;                   /* 内存申请失败 */

    g_ovx_mode = 1;
    g_jpeg_data_ok = 0;
    sw_ov5640_mode();                                /* 切换为 OV5640 模式 */
    ov5640_jpeg_mode();                              /* JPEG 模式 */
    ov5640_outsize_set(16, 4, 2592, 1944);           /* 设置输出尺寸(WXGA) */
    dcmi_rx_callback = jpeg_dcmi_rx_callback;        /* JPEG 接收数据回调函数 */
    dcmi_dma_init((uint32_t)p_dcmi_line_buf[0], (uint32_t)p_dcmi_line_buf[1],
       jpeg_line_size, DMA_MDATAALIGN_WORD, DMA_MINC_ENABLE); /* DCMI DMA 配置 */
    dcmi_start();                    /* 启动传输 */
    while (g_jpeg_data_ok != 1);     /* 等待第一帧图片采集完 */
    g_jpeg_data_ok = 2;              /* 忽略本帧图片,启动下一帧采集 */
    while (g_jpeg_data_ok != 1);     /* 等待第二帧图片采集完,第二帧,才保存到 SD 卡去 */

    dcmi_stop();          /* 停止 DMA 搬运 */
    g_ovx_mode = 0;
    sw_sdcard_mode();     /* 切换为 SD 卡模式 */
    printf("jpeg data size:%d\r\n", g_jpeg_data_len *4);/*串口打印 JPEG 文件大小 */
    pbuf = (uint8_t *)p_jpeg_data_buf;
    jpglen = 0;           /* 设置 jpg 文件大小为 0 */
    headok = 0;           /* 清除 jpg 头标记 */
    /* 查找 0XFF,0XD8 和 0XFF,0XD9,获取 jpg 文件大小 */
    for (i = 0; i < g_jpeg_data_len * 4; i++)
    {
```

```
        if ((pbuf[i] == 0XFF) && (pbuf[i + 1] == 0XD8)) /* 找到 FF D8 */
        {
            jpgstart = i;
            headok = 1;          /* 标记找到 jpg 头(FF D8) */
        }
        /* 找到头以后,再找 FF D9 */
        if ((pbuf[i] == 0XFF) && (pbuf[i + 1] == 0XD9) && headok)
        {
            jpglen = i - jpgstart + 2;
            break;
        }
    }
    if (jpglen)                  /* 正常的 jpeg 数据 */
    {
            pbuf += jpgstart;    /* 偏移到 0XFF,0XD8 处 */
            res = f_write(f_jpg, pbuf, jpglen, &bwr);
            if (bwr != jpglen)res = 0XFE;
    }
    else
    {
        res = 0XFD;
    }
    g_jpeg_data_len = 0;
    sw_ov5640_mode();          /* 切换为 OV5640 模式 */
    ov5640_rgb565_mode();      /* RGB565 模式 */
    if (lcdltdc.pwidth != 0)
    {
       dcmi_rx_callback = rgblcd_dcmi_rx_callback;
       dcmi_dma_init((uint32_t)p_dcmi_line_buf[0], (uint32_t)p_dcmi_line_buf[1],
                     lcddev.width/2, DMA_MDATAALIGN_HALFWORD,
                      DMA_MINC_DISABLE); /* DCMI DMA 配置,MCU 屏,竖屏 */
    }
    else
    {
     dcmi_dma_init((uint32_t)&LCD->LCD_RAM, 0, 1, DMA_MDATAALIGN_HALFWORD,
                   DMA_MINC_DISABLE); /* DCMI DMA 配置,MCU 屏,竖屏 */
    }
    myfree(SRAMIN, f_jpg);
    return res;
}
```

该函数实现 OV5640 的 JPEG 图像采集，并保存图像到 SD 卡，完成 JPEG 拍照。该函数首先设置 OV5640 工作在 JPEG 模式，然后，设置输出分辨率为最高的 QSXGA（2592*1944）。然后，开始采集 JPEG 数据，将第二帧 JPEG 数据，保留下来，并写入 SD 卡里面，完成一次 JPEG 拍照。这里，我们丢弃第一帧 JPEG 数据，是防止采集到的图像数据不完整，导致图片错误。

另外，在保存 jpeg 图片的时候，我们将 0XFF，0XD8 和 0XFF，0XD9 之外的数据，进行了剔除，只留下 0XFF，0XD8~0XFF，0XD9 之间的数据，保证图片文件最小，且无其他乱的数据。

注意，在保存图片的时候，必须将 PC8/9/11 切换为 SD 卡模式，并关闭 OV5640 的输出。在图片保存完成以后，切换回 OV5640 模式，并重新使能 OV5640 的输出。

最后介绍的是 main 函数，其定义如下：

```
int main(void)
{
    uint8_t res;
    float fac;
    uint8_t *pname;          /* 带路径的文件名 */
    uint8_t key;             /* 键值 */
    uint8_t i;
    uint8_t sd_ok = 1;       /* 0,sd 卡不正常;1,SD 卡正常 */
    uint8_t scale = 1;       /* 默认是全尺寸缩放 */
    uint8_t msgbuf[15];      /* 消息缓存区 */
```

```
    uint16_t outputheight = 0;

    sys_cache_enable();                         /* 打开 L1-Cache */
    HAL_Init();                                 /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);         /* 设置时钟, 400Mhz */
    delay_init(400);                            /* 延时初始化 */
    usart_init(115200);                         /* 串口初始化为 115200 */
    usmart_dev.init(200);                       /* 初始化 USMART */
    mpu_memory_protection();                    /* 保护相关存储区域 */
    led_init();                                 /* 初始化 LED */
    key_init();                                 /* 初始化 KEY */
    sdram_init();                               /* 初始化 SDRAM */
    lcd_init();                                 /* 初始化 LCD */
    pcf8574_init();                             /* 初始化 PCF8574 */
    ov5640_init();                              /* 初始化 OV5640 */
    sw_sdcard_mode();                           /* 首先切换为 OV5640 模式 */
    norflash_init();                            /* 初始化 norflash */

    my_mem_init(SRAMIN);                        /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                        /* 初始化外部内存池 */
    my_mem_init(SRAM12);                        /* 初始化 SRAM12 内存池(SRAM1+SRAM2) */
    my_mem_init(SRAM4);                         /* 初始化 SRAM4 内存池(SRAM4) */
    my_mem_init(SRAMDTCM);                      /* 初始化 DTCM 内存池(DTCM) */
    my_mem_init(SRAMITCM);                      /* 初始化 ITCM 内存池(ITCM) */

    exfuns_init();                              /* 为 fatfs 相关变量申请内存 */
    f_mount(fs[0], "0:", 1);                    /* 挂载 SD 卡 */
    f_mount(fs[1], "1:", 1);                    /* 挂载 SPI FLASH */
    f_mount(fs[2], "2:", 1);                    /* 挂载 NAND FLASH */

    while (fonts_init())                        /* 检查字库 */
    {
        lcd_show_string(30, 90, 200, 16, 16, "Font Error!", RED);
        delay_ms(200);
        lcd_fill(30, 90, 240, 66, WHITE);     /* 清除显示 */
        delay_ms(200);
    }
    text_show_string(30, 50, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
    text_show_string(30, 70, 200, 16, "照相机 实验", 16, 0, RED);
    text_show_string(30, 90, 200, 16, "KEY0:拍照(bmp 格式)", 16, 0, RED);
    text_show_string(30, 110, 200, 16, "KEY1:拍照(jpg 格式)", 16, 0, RED);
    text_show_string(30, 130, 200, 16, "KEY2: 自动对焦", 16, 0, RED);
    text_show_string(30, 150, 200, 16, "WK_UP:FullSize/Scale", 16, 0, RED);
    res = f_mkdir("0:/PHOTO");  /* 创建 PHOTO 文件夹 */
    if (res != FR_EXIST && res != FR_OK)     /* 发生了错误 */
    {
        res = f_mkdir("0:/PHOTO");           /* 创建 PHOTO 文件夹 */
        text_show_string(30, 150, 240, 16, "SD 卡错误!", 16, 0, RED);
        delay_ms(200);
        text_show_string(30, 150, 240, 16, "拍照功能将不可用!", 16, 0, RED);
        delay_ms(200);
        sd_ok = 0;
    }
    /* 为 jpeg dma 接收申请内存 */
    p_dcmi_line_buf[0] = mymalloc(SRAMIN, jpeg_line_size * 4);
    /* 为 jpeg dma 接收申请内存 */
    p_dcmi_line_buf[1] = mymalloc(SRAMIN, jpeg_line_size * 4);
    p_jpeg_data_buf = mymalloc(SRAMEX, jpeg_buf_size);  /* 为 jpeg 文件申请内存 */
    pname = mymalloc(SRAMIN, 30);  /* 为带路径的文件名分配 30 个字节的内存 */
    while (pname == NULL || !p_dcmi_line_buf[0] || !p_dcmi_line_buf[1]
```

```
        || !p_jpeg_data_buf)      /* 内存分配出错 */
    {
        text_show_string(30, 170, 240, 16, "内存分配失败!", 16, 0, RED);
        delay_ms(200);
        lcd_fill(30, 170, 240, 146, WHITE); /* 清除显示 */
        delay_ms(200);
    }
    while (ov5640_init())    /* 初始化 OV5640 */
    {
        text_show_string(30, 190, 240, 16, "OV5640 错误!", 16, 0, RED);
        delay_ms(200);
        lcd_fill(30, 190, 239, 206, WHITE);
        delay_ms(200);
    }
    text_show_string(30, 170, 230, 16, "OV5640 正常", 16, 0, RED);
    /* 自动对焦初始化 */
    ov5640_rgb565_mode();           /* RGB565 模式 */
    ov5640_focus_init();
    ov5640_light_mode(0);           /* 自动模式 */
    ov5640_color_saturation(3);     /* 色彩饱和度 0 */
    ov5640_brightness(4);           /* 亮度 0 */
    ov5640_contrast(3);             /* 对比度 0 */
    ov5640_sharpness(33);           /* 自动锐度 */
    ov5640_focus_constant();        /* 启动持续对焦 */
    dcmi_init();                    /* DCMI 配置 */
    if(lcdltdc.pwidth!=0)
    {
      dcmi_rx_callback=rgblcd_dcmi_rx_callback;
      dcmi_dma_init((uint32_t)p_dcmi_line_buf[0],(uint32_t)p_dcmi_line_buf[1],
                    lcddev.width/2,DMA_MDATAALIGN_HALFWORD,DMA_MINC_ENABLE);
    }
    else
    {
      dcmi_dma_init((uint32_t)&LCD->LCD_RAM, 0, 1, DMA_MDATAALIGN_HALFWORD,
                     DMA_MINC_DISABLE);        /* DCMI DMA 配置,MCU 屏,竖屏 */
    }
    if (lcddev.height == 1024)
    {
        g_yoffset = (lcddev.height - 800) / 2;
        outputheight = 800;
        ov5640_write_reg(0x3035, 0X51);      /* 降低输出帧率，否则可能抖动 */
    }
    else if(lcddev.height ==1280)
    {
        g_yoffset = (lcddev.height-600)/2;
        outputheight = 600;
       ov5640_write_reg(0x3035,0x51);
    }
    else
    {
        g_yoffset =0;
        outputheight = lcddev.height;
    }

    g_curline = g_yoffset;          /* 行数复位 */
    ov5640_outsize_set(16, 4, lcddev.width, outputheight);  /* 满屏缩放显示 */
    dcmi_start();                   /* 启动传输 */
    lcd_clear(BLACK);
    while (1)
    {
        key = key_scan(0);       /* 不支持连按 */
        if (key)
        {
```

```
        if (key != KEY2_PRES)
        {
          /* 如果是 BMP 拍照,则等待 1 秒钟,去抖动,以获得稳定的 bmp 照片 */
          if (key == KEY0_PRES)
          {
              delay_ms(300);
              g_bmp_request = 1;          /* 请求关闭 DCMI */
              while (g_bmp_request);      /* 等带请求处理完成 */
          }
          else
          {
              dcmi_stop();
          }
        }
          if (key == WKUP_PRES)           /* 缩放处理 */
          {
              scale = !scale;
              if (scale == 0)
              {
                  fac = 800 / outputheight;     /* 得到比例因子 */
                  ov5640_outsize_set((1280 - fac * lcddev.width) / 2, (800
                      - fac * outputheight) / 2, lcddev.width, outputheight);
                  sprintf((char *)msgbuf, "Full Size 1:1");
              }
              else
              {
                  ov5640_outsize_set(16, 4, lcddev.width, outputheight);
                  sprintf((char *)msgbuf, "Scale");
              }
              delay_ms(800);
          }
          if (key == KEY2_PRES)           /* 缩放处理 */
          {
              ov5640_focus_single();
          }

          else if (sd_ok)                 /* SD 卡正常才可以拍照 */
          {
              sw_sdcard_mode();           /* 切换为 SD 卡模式 */
              if (key == KEY0_PRES)       /* BMP 拍照 */
              {
                  camera_new_pathname(pname, 0);      /* 得到文件名 */
                  res = bmp_encode(pname, 0, g_yoffset, lcddev.width,
                                   outputheight, 0);
                  sw_ov5640_mode();                   /* 切换为 OV5640 模式 */
              }
              else if (key == KEY1_PRES)              /* JPG 拍照 */
              {
                  camera_new_pathname(pname, 1);      /* 得到文件名 */
                  res = ov5640_jpg_photo(pname);
                  if (scale == 0)
                  {
                     fac = 800 / outputheight;     /* 得到比例因子 */
                     ov5640_outsize_set((1280 - fac * lcddev.width) / 2, (800
                     - fac * outputheight) / 2, lcddev.width, outputheight);
                  }
                  else
                  {
                      ov5640_outsize_set(16, 4, lcddev.width, outputheight);
                  }
                  /* 降低输出帧率，否则可能抖动 */
                  if (lcddev.height > 800)ov5640_write_reg(0x3035, 0X51);
              }
              if (res)     /* 拍照有误 */
```

```
                {
                 text_show_string(30, 130, 240, 16, "写入文件错误!", 16, 0, RED);
                }
                else
                {
                    text_show_string(30, 130, 240, 16, "拍照成功!", 16, 0, RED);
                    text_show_string(30, 150, 240, 16, "保存为:", 16, 0, RED);
                    text_show_string(30 + 56, 150, 240, 16, (char*)pname,
                                     16, 0, RED);
                    pcf8574_write_bit(BEEP_IO,0);     /* 蜂鸣器短叫，提示拍照完成 */
                    delay_ms(100);
                    pcf8574_write_bit(BEEP_IO,1);     /* 关闭蜂鸣器 */
                }
                delay_ms(1000); /* 等待 1 秒钟 */
                /* 这里先使能 dcmi,然后立即关闭 DCMI,
                   后面再开启 DCMI,可以防止 RGB 屏的侧移问题
                 */
                dcmi_start();
                dcmi_stop();
            }
            else    /* 提示 SD 卡错误 */
            {
                text_show_string(30, 130, 240, 16, "SD 卡错误!", 16, 0, RED);
                text_show_string(30, 150, 240, 16, "拍照功能不可用!", 16, 0, RED);
            }
            if (key!=KEY2_PRES)
            {
              dcmi_start();   /* 开始显示 */
            }
        }
        delay_ms(10);
        i++;
        if (i == 20)     /* DS0 闪烁 */
        {
            i = 0;
            LED0_TOGGLE();
        }
    }
}
```

该函数完成对各相关硬件的初始化，然后检测OV5640，初始化OV5640位RGB565模式，显示采集到的图像到 LCD 上面，实现对图像进行预览。进入主循环以后，按 KEY0 按键，可以实现 BMP 拍照（实际上就是截屏，通过 bmp_encode 函数实现）；按 KEY1 按键，可实现 JPEG 拍照（2592*1944 分辨率，通过 ov5640_jpg_photo 函数实现）；按 KEY2 按键，可以实现自动对焦（单次）；按 KEY_UP 按键，可以实现图像缩放/不缩放预览。main 函数实现了我们在 59.2 节所提到的功能。

至此照相机实验代码编写完成。最后，本实验可以通过 USMART 来设置 OV5640 的相关参数，将 ov5640_contrast、ov5640_color_saturation 和 ov5640_light_mode 等函数添加到 USMART 管理，即可通过串口设置 OV5640 的参数，方便调试。

59.4 下载验证

将程序下载到开发板后，可以看到LCD首先显示一些实验相关的信息，如图59.4.1所示：

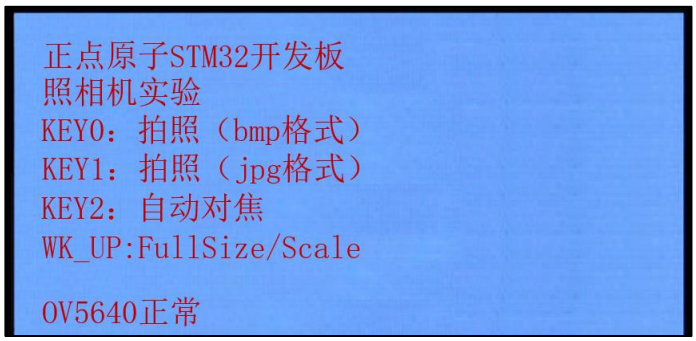


图 59.4.1 显示实验相关信息

显示了上图的信息后，自动进入监控界面。可以看到 LED0 不停的闪烁，提示程序已经在运行了。此外，LED1 不停闪烁，提示进入 DCMI 中断回调服务函数，进行 jpeg 数据处理。此时，我们可以按下 KEY0 和 KEY1，即可进行 bmp/jpg 拍照。拍照得到的照片效果如图 59.4.2 和图 59.4.3 所示：

图 59.4.2 拍照样图（bmp 拍照样图）

图 59.4.3 拍照样图（jpg 拍照样图）

按 KEY_UP 可以实现缩放/不缩放显示。最后，我们还可以通过 USMART 调用 OV5640 的相关控制函数，实现串口控制 OV5640 的在线参数修改，方便调试。

第六十章 音乐播放器实验

正点原子阿波罗 STM32H743 开发板拥有串行音频接口（SAI），支持 SAI、LSB/MSB 对齐、PCM/DSP、TDM 和 AC’97 等协议，且外扩了一颗 HIFI 级 CODEC 芯片：ES8388，支持最高 192K 24BIT 的音频播放，并且支持录音（下一章介绍）本章，我们将利用阿波罗 STM32H743 开发板实现一个简单的音乐播放器（仅支持 WAV 播放）。本章分为如下几个部分：

60.1 WAV&ES8388&SAI 简介
60.2 硬件设计
60.3 软件设计
60.4 下载验证

60.1 WAV&ES8388&SAI 简介

本章知识点比较多，包括：WAV、ES8388 和 SAI 等三个知识点。下面我们将分别向大家介绍。

60.1.1 WAV 简介

WAV 即 WAVE 文件，WAV 是计算机领域最常用的数字化声音文件格式之一，它是微软专门为 Windows 系统定义的波形文件格式（Waveform Audio），由于其扩展名为"*.wav"。它符合 RIFF(Resource Interchange File Format)文件规范，用于保存 Windows 平台的音频信息资源，被 Windows 平台及其应用程序所广泛支持，该格式也支持 MSADPCM，CCITT A LAW 等多种压缩运算法，支持多种音频数字，取样频率和声道，标准格式化的 WAV 文件和 CD 格式一样，也是 44.1K 的取样频率，16 位量化数字，因此在声音文件质量和 CD 相差无几！

WAV 一般采用线性 PCM（脉冲编码调制）编码，本章，我们也主要讨论 PCM 的播放，因为这个最简单。

WAV 文件是由若干个 Chunk 组成的。按照在文件中的出现位置包括：RIFF WAVE Chunk、Format Chunk、 Fact Chunk(可选)和 Data Chunk。每个 Chunk 由块标识符、数据大小和数据三部分组成，如图 60.1.1.1 所示：

图 60.1.1.1 Chunk 组成结构

对于一个基本的 WAVE 文件而言，以下三种 Chunk 是必不可少的：文件中第一个 Chunk 是 RIFF Chunk，然后是 FMT Chunk，最后是 Data Chunk。对于其他的 Chunk，顺序没有严格的限制。使用 WAVE 文件的应用程序必须具有读取以上三种 chunk 信息的能力，如果程序想要复制 WAVE 文件，必须拷贝文件中所有的 chunk。本章，我们主要讨论 PCM，因为这个最简单，它只包含 3 个 Chunk，我们看一下它的文件构成，如图 60.1.2。

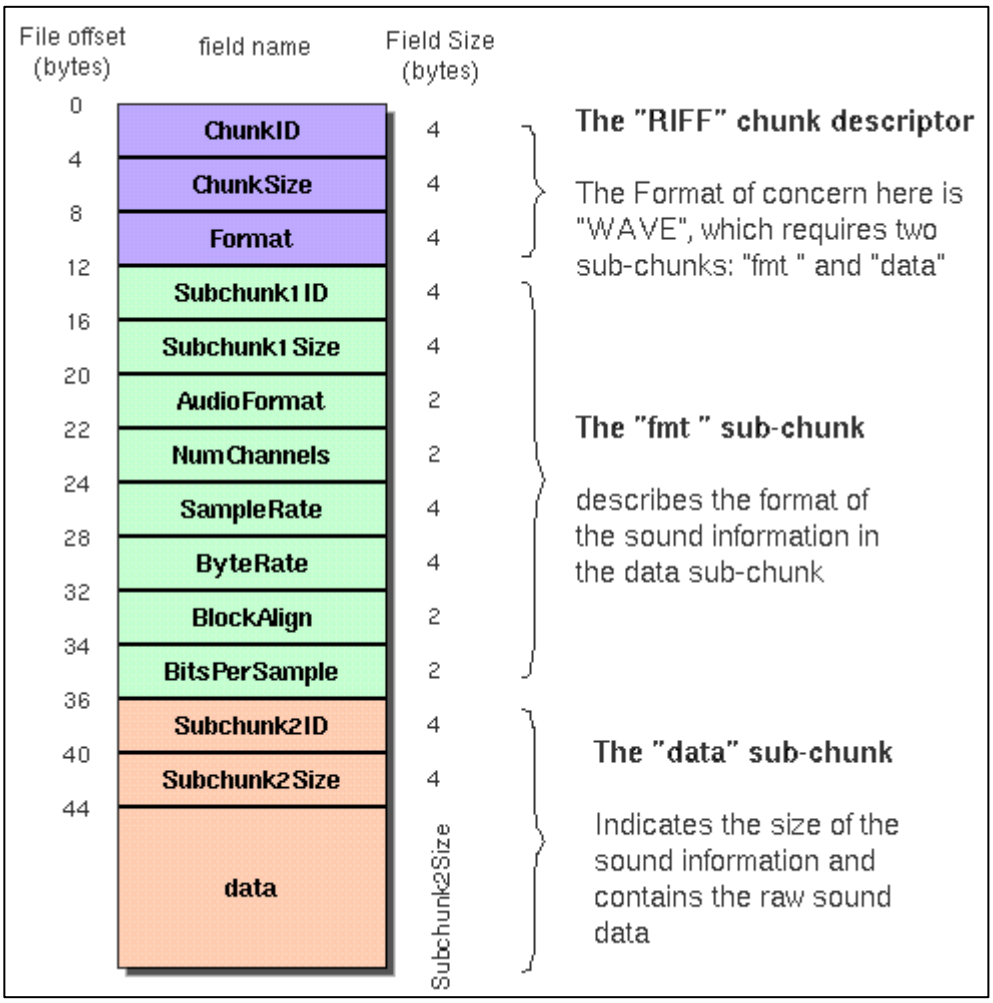


图 60.1.2 PCM 格式的 wav 文件构成

可以看到，不同的 Chunk 有不同的长度，编码文件时，按照 Chunk 的字节和位序排列好之后写入文件头，加上 wav 的后缀，就可以生成一个能被正确解析的 wav 文件了，对于 PCM 结构，我们只需要把获取到的音频数据填充到 Data Chunk 中即可。我们将利用 ES8388 实现 16 位，8Khz 采样率的单声道 WAV 录音(PCM 格式)。

首先，我们来看看 RIFF 块（RIFF WAVE Chunk），该块以“RIFF”作为标示，紧跟 wav 文件大小（该大小是 wav 文件的总大小-8），然后数据段为“WAVE”，表示是 wav 文件。RIFF 块的 Chunk 结构如下：

```
typedef __PACKED_STRUCT
{
    uint32_t ChunkID;       /* chunk id;这里固定为"RIFF",即 0X46464952 */
    uint32_t ChunkSize ;    /* 集合大小;文件总大小-8 */
    uint32_t Format;        /* 格式;WAVE,即 0X45564157 */
}ChunkRIFF;                 /* RIFF 块 */
```

接着，我们看看 Format 块（Format Chunk），该块以“fmt”作为标示（注意有个空格！），一般情况下，该段的大小为 16 个字节，但是有些软件生成的 wav 格式，该部分可能有 18 个字节，含有 2 个字节的附加信息。Format 块的 Chunk 结构如下：

```
typedef __PACKED_STRUCT
{
    uint32_t ChunkID;           /* chunk id;这里固定为"fmt ",即 0X20746D66 */
    uint32_t ChunkSize ;        /* 子集合大小(不包括 ID 和 Size);这里为:20. */
    uint16_t AudioFormat;       /* 音频格式;0X01,表示线性 PCM;0X11 表示 IMA ADPCM */
    uint16_t NumOfChannels;     /* 通道数量;1,表示单声道;2,表示双声道; */
    uint32_t SampleRate;        /* 采样率;0X1F40,表示 8Khz */
    uint32_t ByteRate;          /* /字节速率; */
```

```
    uint16_t BlockAlign;          /* 块对齐(字节); */
    uint16_t BitsPerSample;       /* 单个采样数据大小;4 位 ADPCM,设置为 4 */
}ChunkFMT;                        /* fmt 块 */
```

接下来，我们再看看 Fact 块（Fact Chunk），该块为可选块，以“fact”作为标示，不是每个 WAV 文件都有，在非 PCM 格式的文件中，一般会在 Format 结构后面加入一个 Fact 块，该块 Chunk 结构如下：

```
typedef __PACKED_STRUCT
{
    uint32_t ChunkID;             /* chunk id;这里固定为"fact",即 0X74636166; */
    uint32_t ChunkSize ;          /* 子集合大小(不包括 ID 和 Size);这里为:4. */
    uint32_t NumOfSamples;        /* 采样的数量; */
}ChunkFACT;                       /* fact 块 */
```

DataFactSize 是这个 Chunk 中最重要的数据，如果这是某种压缩格式的声音文件，那么从这里就可以知道他解压缩后的大小。对于解压时的计算会有很大的好处！不过本章我们使用的是 PCM 格式，所以不存在这个块。

最后，我们来看看数据块（Data Chunk），该块是真正保存 wav 数据的地方，以“data”作为该 Chunk 的标示，然后是数据的大小。数据块的 Chunk 结构如下：

```
typedef __PACKED_STRUCT
{
    uint32_t ChunkID;             /* chunk id;这里固定为"data",即 0X5453494C */
    uint32_t ChunkSize ;          /* 子集合大小(不包括 ID 和 Size) */
}ChunkDATA;                       /* data 块 */
```

ChunkSize 后紧接着就是 wav 数据。根据 Format Chunk 中的声道数以及采样 bit 数，wav 数据的 bit 位置可以分成如表 60.1.1.1 所示的几种形式：

单声道	取样 1		取样 2		取样 3	
8 位量化	声道 0	声道 0	声道 0	声道 0	声道 0	声道 0
双声道	取样 1		取样 2		取样 3	
8 位量化	声道 0(左)	声道 1(右)	声道 0(左)	声道 1(右)	声道 0(左)	声道 1(右)
单声道	取样 1		取样 2		取样 3	
16 位量化	声道 0（低字节）	声道 0（高字节）	声道 0（低字节）	声道 0（高字节）	声道 0（低字节）	声道 0（高字节）
双声道	取样 1				取样 2	
16 位量化	声道 0（低字节）	声道 0（高字节）	声道 1（低字节）	声道 1（高字节）	声道 0（低字节）	声道 0（高字节）
单声道	取样 1				取样 2	
24 位量化	声道 0（低字节）	声道 0（中字节）	声道 0（高字节）	声道 0（低字节）	声道 0（中字节）	声道 0（高字节）
双声道	取样 1					
24 位量化	声道 0（低字节）	声道 0（中字节）	声道 0（高字节）	声道 1（低字节）	声道 1（中字节）	声道 1（高字节）

表 60.1.1.1 WAVE 文件数据采样格式

本章，我们播放的音频支持：16 位和 24 位，立体声，所以每个取样为 4/6 个字节，低字节在前，高字节在后。在得到这些 wav 数据以后，通过 SAI 丢给 ES8388，就可以欣赏音乐了。

60.1.2 ES8388 简介

ES8388 是上海顺芯推出的一款高性能、低功耗、高性价比的音频编解码器，有 2 个 ADC 通道和 2 个 DAC 通道，麦克风放大器，耳机放大器，数字音效以及模拟混合和增益功能组成。

ES8388 的主要特性有：

●SAI 接口，支持最高 192K，24bit 音频播放

●DAC 信噪比 96dB；ADC 信噪比 95dB

●支持主机和从机模式
●支持立体声差分输入/麦克风输入
●支持左右声道音量独立调节
●支持 40mW 耳机输出，无爆音

ES8388 的控制通过 I2S 接口（即数字音频接口）同 MCU 进行音频数据传输（支持音频接收和发送），通过两线（CE=0/1，即 IIC 接口）或三线（CE 脚产生一个下降沿,即 SPI 接口）接口进行配置。ES8388 的 SAI 接口，由 4 个引脚组成：

ASDOUT:ADC 数据输出
DSDIN：DAC 数据输入
LRC：数据左/右对齐时钟
SCLK：位时钟，用于同步

ES8388 可作为 SAI 主机，输出 LRC 和 SLCK 时钟，不过我们一般使用 ES8388 作为从机，接收 LRC 和 SLCK。另外，ES8388 的 SAI 接口支持 4 种不同的音频数据模式：左（MSB）对齐标准、右（LSB）对齐标准、飞利浦（SAI）标准、DSP/PCM。本章，我们用飞利浦标准来传输 SAI 数据。

飞利浦（SAI）标准模式，数据在跟随 LRC 传输的 BCLK 的第二个上升沿时传输 MSB，其他位一直到 LSB 按顺序传输。传输依赖于字长、BCLK 频率和采样率，在每个采样的 LSB 和下一个采样的 MSB 之间都应该有未用的 BCLK 周期。飞利浦标准模式的 SAI 数据传输协议如图 60.1.2.1 所示：

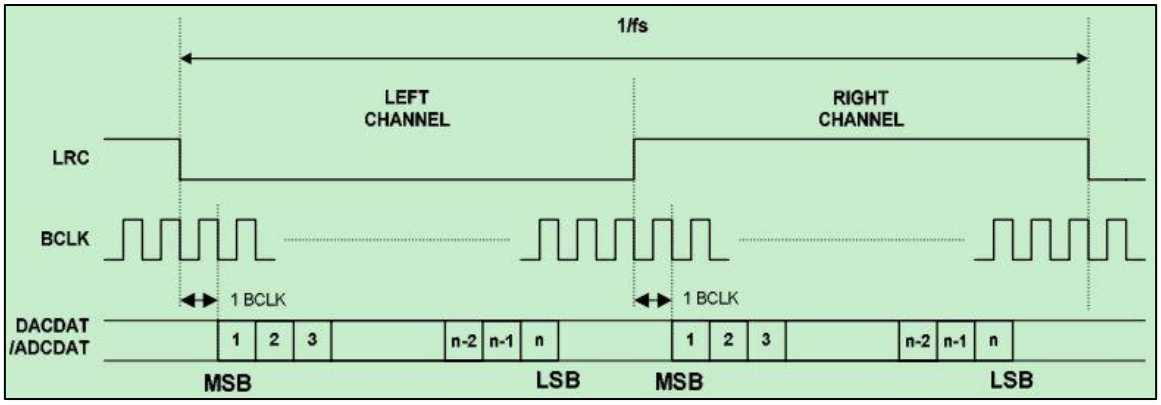


图 60.1.2.1 飞利浦标准模式 SAI 数据传输图

图中，fs 即音频信号的采样率，比如 44.1Khz，因此可以知道，LRC 的频率就是音频信号的采样率。另外，ES8388 还需要一个 MCLK，本章我们采用 STM32H743 为其提供 MCLK 时钟，MCLK 的频率必须等于 256fs，也就是音频采样率的 256 倍。

ES8388 的框图如图 60.1.2.2 所示：

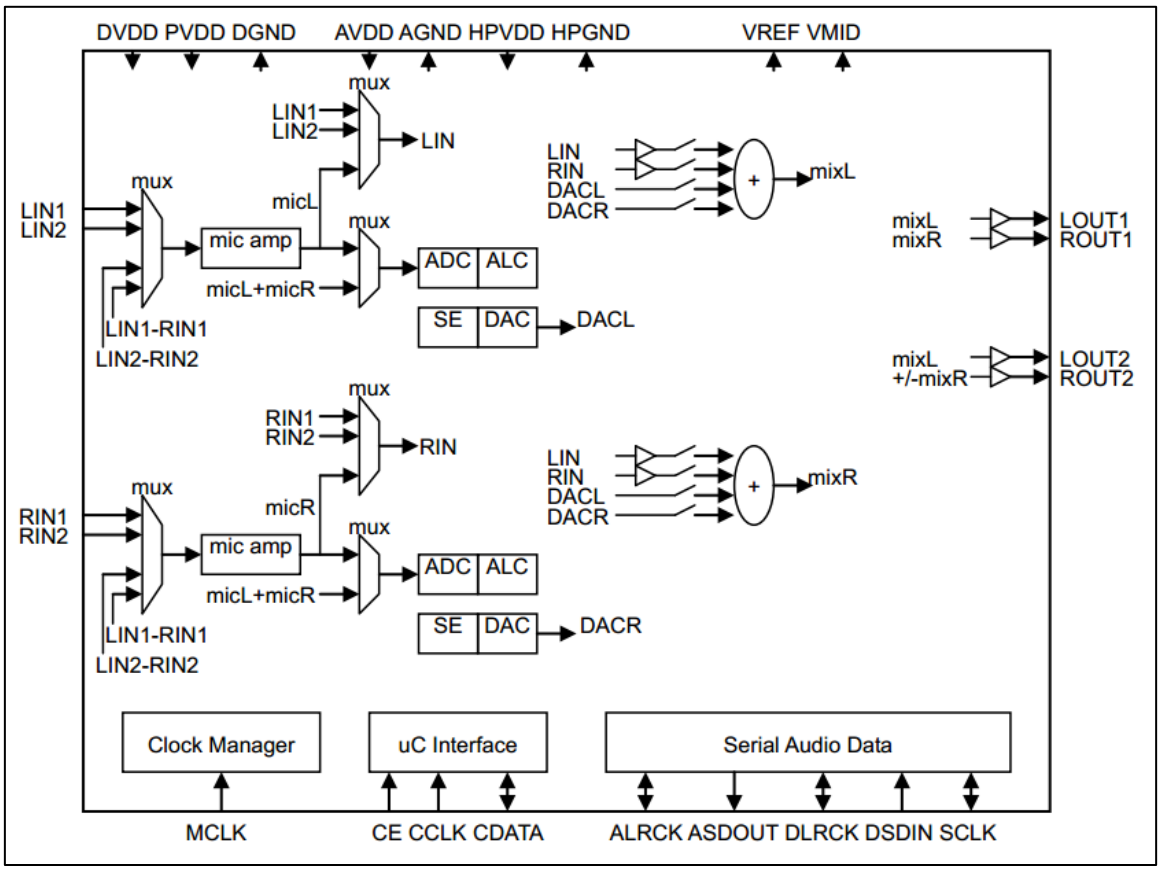


图 60.1.2.2 ES8388 框图

从上图可以看出，ES8388 内部有很多的模拟开关，用来选择通道，同时还有一些运放调节器，用来设置增益和音量。

本章，我们通过 IIC 接口（CE=0）连接 ES8388，ES8388 的 IIC 地址为：0X10。关于 ES8388 的 IIC 详细介绍，请看其数据手册第 10 页 5.2 节。

这里我们简单介绍一下要正常使用 ES8388 来播放音乐，应该执行哪些配置。

1，寄存器 R0（00h），是芯片控制寄存器 1，需要用到的位有：最高位 SCPRese(bit7)用于控制 ES8388 的软复位，写 0X80 到该寄存器地址，即可实现软复位 ES8388，复位后，再写 0X00，ES8388 恢复正常。VMIDSEL[1:0]位用于控制 VMID（校正噪声用），我们一般设置为 10，即用 500KΩ 校正。

2，寄存器 R1（01h），是芯片控制寄存器 2，主要要设置 PdnAna(bit3)，该位设置为 1，模拟部分掉电，相当于复位模拟部分；设置为 0，模拟部分才会工作，才可以听到声音。

3，寄存器 R2（02h），是芯片电源管理控制寄存器，所有位都要用到：adc_DigPDN(bit7)和 dac_DigPDN(bit6)分别用于控制 ADC 和 DAC 的 DSM、DEM、滤波器和数字接口的复位，1 复位，0 正常；adc_stm_rst(bit5)和 dac_stm_rst(bit4)分别用于控制 ADC 和 DAC 的状态机掉电，1 掉电，0 正常；ADCDLL_PDN(bit3)和 DACDLL_PDN(bit2)分别用于控制 ADC 和 DAC 的 DLL 掉电，停止时钟，1 掉电，0 正常；adcVref_PDN(bit1)和 dacVref_PDN(bit0)分别控制 ADC 和 DAC 的模拟参考电压掉电，1 掉电，0 正常；因此想要 ADC 和 DAC 都正常工作，R2 寄存器必须全部设置为 0，否则 ADC 或者 DAC 就会不能正常工作。

4，寄存器 R3（03h），是 ADC 电源管理控制寄存器，需要用到的位有：PdnAINL(bit7)和 PdnAINR(bit6)用于控制左右输入模拟通道的电源，1 掉电，0 正常；PdnADCL(bit5)和 PdnADCR(bit4)用于控制左右通道 ADC 的电源，1 掉电，0 正常；pdnMICB(bit3)用于控制麦克风的偏置电源，1 掉电，0 正常；PdnADCBiasgen(bit2)用于控制偏置电源的产生，1 掉电，0 正常；这里 6 个位，我们全部设置为 0，ADC 部分就可以正常工作了。

5，寄存器 R4（04h），是 DAC 电源管理控制寄存器，需要用到的位有：PdnDACL(bit7)和 PdnDACR(bit6)分别用于左右声道 DAC 的电源控制，1 掉电；0 正常；LOUT1(bit5)和

ROUT1(bit4)分别用于控制通道 1 的左右声道输出是能，1 使能，0 禁止；LOUT2(bit3)和 ROUT2(bit2)分别用于控制通道 2 的左右声道输出是能，1 使能，0 禁止；我们一般设置 PdnDACL 和 PdnDACR 为 0，使能左右声道 DAC，另外，两个输出通道则根据自己的需要设置。

6，寄存器 R8（08h），是主模式控制寄存器，需要用到的位有：MSC(bit7)用于控制接口模式，0 从模式，1 主模式；MCKDIV2(bit6)用于控制 MCLK 的 2 分频，0 不分频，1 二分频；BCLK_INV(bit5)用于控制 BCLK 的反相，0 不反相；1，反相；一般设置这 3 个位都为 0。

7，寄存器 R9（09h），是 ADC 控制寄存器 1，所有位都要用到：MicAmpL(bit7:4)和 MicAmpR(bit3:0)，这两个分别用于控制 MIC 的左右通道增益，从 0 开始，3dB 一个档，最大增益为 24dB，我们一般设置 MicAmpR/L[3:0]=1000，即 24dB。

8，寄存器 R10（0Ah），是 ADC 控制寄存器 2，需要用到的位有：LINSE(bit7:6)和 RINSE(bit5:4)分别选择左右输入通道，0 选择通道 1，1 选择通道 2。

9，寄存器 R12（0Ch），是 ADC 控制寄存器 4，全部位都要用到：DATSEL(bit7:6)用于选择数据格式，一般设置为 01，左右边数据等于左右声道 ADC 数据；ADCLRP（bit5）在 I2S 模式下用于设置数据对其方式，一般设置为 0，正常极性；ADCWL(bit4:2)用于选择数据长度，我们设置 011，选择 16 位数据长度；ADCFORMAT(bit1:0)用于设置 ADC 数据格式，一般设置为 00，选择 I2S 数据格式。

10，寄存器 R13（0Dh），是 ADC 控制寄存器 5，全部位都要用到：ADCFsMode(bit7)用于设置 Fs 模式，0 单速模式，1 双倍速模式，一般设置为 0；ADCFsRatio(bit4:0)用于设置 ADC 的 MCLK 和 FS 的比率，我们设置 00010，即 256 倍关系。

11，寄存器 R16（10h）和 R17（11h），这两个寄存器分别用于控制 ADC 左右声道的音量衰减，LADCVOL(bit7:0)和 RADCVOL(bit7:0)分别控制左声道和右声道 ADC 的衰减，0.5dB 每步，我们一般设置为 0，即不衰减。

12，寄存器 R18（12h），是 ADC 控制寄存器 10，全部位都要用到：ALCSEL(bit7:6)用于控制 ALC，00 表示 ALC 关闭，01 表示 ALC 仅控制左声道，10 表示 ALC 仅控制右声道 11 表示 ALC 立体声控制；我们一般设置为 11。

13，寄存器 R23（17h），是 DAC 控制寄存器 1，需要用到的位有：DACLRSWAP(bit7)用于控制左右声道数据交换，0 正常，1 互换，一般设置为 0；DACLRP(bit6) 在 I2S 模式下用于设置数据对其方式，一般设置为 0，正常极性；DACWL(bit5:3)用于选择数据长度，我们设置 011，选择 16 位数据长度；ADCFORMAT(bit1:0)用于设置 DAC 数据格式，一般设置为 00，选择 I2S 数据格式。

14，寄存器 R24（18h），是 DAC 控制寄存器 2，全部位都要用到：DACFsMode(bit7)用于设置 Fs 模式，0 单速模式，1 双倍速模式，一般设置为 0；DACFsRatio(bit4:0)用于设置 DAC 的 MCLK 和 FS 的比率，我们设置 00010，即 256 倍关系。

15，寄存器 R26（1Ah）和 R27（1Bh），这两个寄存器分别用于控制 DAC 左右声道的音量衰减，LDACVOL(bit7:0)和 RDACVOL(bit7:0)分别控制左声道和右声道 DAC 的衰减，0.5dB 每步，0 表示 0dB 衰减，192 表示 96dB 衰减；通过这两个寄存器可以完成输出音量的调节。

16，寄存器 R29（1Dh），是 DAC 控制寄存器 7，需要用到的位有：ZeroL(bit7)和 ZeroR(bit6)分别控制左右声道的全 0 输出，类似静音，1 输出 0，0 正常；一般设置为 0。Mono(bit5)用于单声道控制，0 立体声，1 单声道；一般设置为 0。SE(bit4:2)用于设置 3D 音效，0~7 表示 3D 效果的强弱，0 表示关闭。

17，寄存器 39（27h）和 42（2Ah），分别控制 DAC 左右通道的混音器，LD2LO(bit7)和 RD2RO(bit7)分别控制左右 DAC 的混音器开关，0 关闭，1 开启，需设置为 1；LI2LO(bit6)和 RI2RO(bit6)分别控制左右输入通道的混音器开关，0 关闭，1 开启，一般设置为 1；LI2LOVOL(bit5:3)和 RI2ROVOL(bit5:3)分别控制左右输入通道的增益，0~7 表示-6 ~ -15dB 的增益调节范围，默认设置为 111，即-15dB。

18，寄存器 43（2Bh），是 DAC 控制寄存器 21，这里我们只关心 slrck(bit7)这个位，用于控制 DACLRC 和 ADCLRC 是否共用，我们设置为 1，表示共用。

以上，就是我们使用 ES8388 时所需要用到的一些寄存器，按照以上所述，对各个寄存器进行相应的配置，即可使用 ES8388 正常播放音乐了。关于 ES8388 更详细的寄存器设置说明，

我们这里就不再介绍了，请大家参考 ES8388 的数据手册自行研究。

60.1.3 SAI 简介

STM32H743 自带了两个串行音频接口（SAI1 和 SAI2），SAI 具有灵活性高、配置多样的特点。可以支持：I2S 标准、LSB 或 MSB 对齐、PCM/DSP、TDM 和 AC 97 等协议，适用于多声道或单声道应用

SAI 通过两个完全独立的音频子模块来实现这种灵活性与可配置性，每个音频子模块与多达 4 个引脚（SD、SCK、FS 和 MCLK）相连。如果将两个子模块声明为同步模块，则其中一些引脚可以共用，从而可释放一些引脚用作通用 I/O。MCLK 引脚是否用作输出引脚取决于实际应用和解码的要求以及音频模块是否配置为主模块。SAI 可以配置为主模式或配置为从模式。音频子模块既可作为接收器，又可作为发送器；既可与另一模块同步，又可以不同步。

STM32H743 自带的 SAI 接口，其特点包括：

●具有两个独立的音频子模块，子模块既可作为接收器，也可作为发送器，并自带 FIFO
●每个音频子模块集成多达 8 个字，每个字 32 位的 FIFO
●两个音频子模块间可以是同步或异步模式
●两个音频子模块的主/从配置相互独立
●当两个音频子模块都配置为主模式时，每个子模块可设置互相独立的采样率
●数据大小可配置：8 位、10 位、16 位、20 位、24 位或 32 位
●支持：I2S、LSB 或 MSB 对齐、PCM/DSP、TDM 和 AC 97 等音频协议
●高达 16 个大小可配置的 Slot，可选择音频帧中的哪些 Slot 有效
●支持 LSB 或 MSB 数据传输
●支持 DMA，有 2 个专业通道，用于处理对每个 SAI 音频子模块的专用集成 FIFO 的访问

STM32H743 的 SAI 框图如图 60.1.3.1 所示：

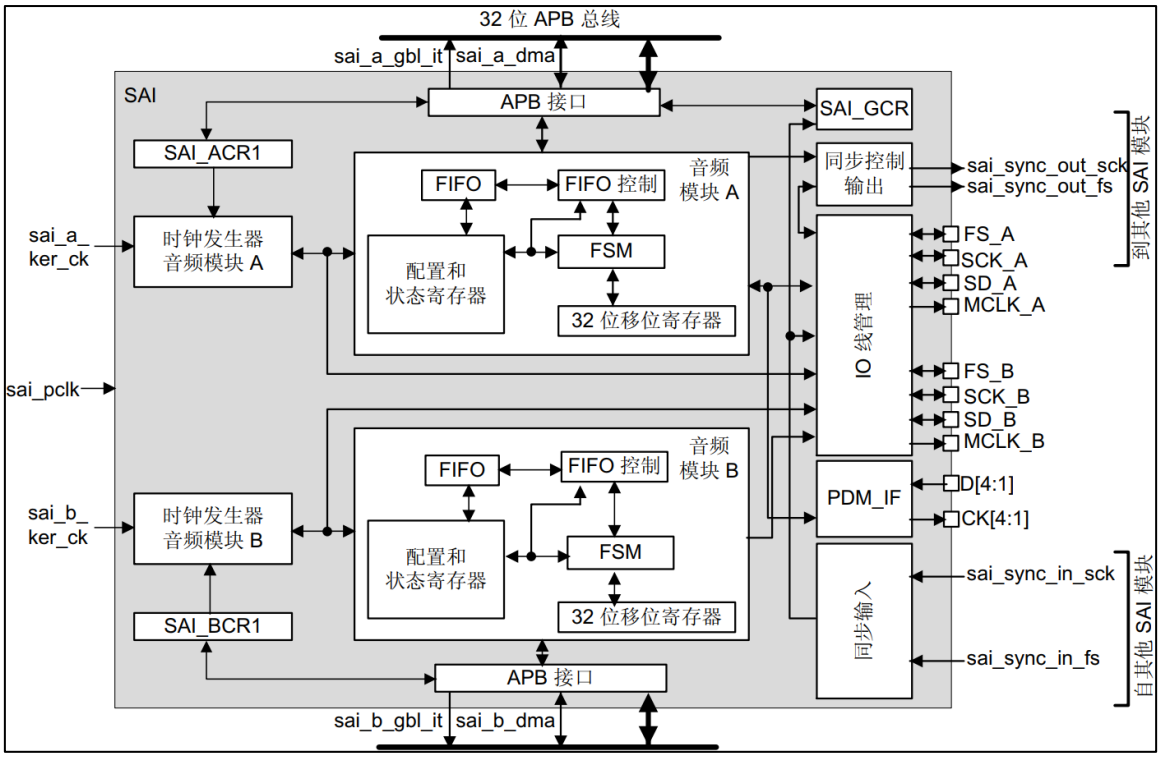


图 60.1.3.1 SAI 框图

本章，我们将用 SAI 接口来驱动 ES8388，而 ES8388 的接口是 I2S 接口的，所以，本章我们只介绍 SAI 支持 I2S 协议使用的方法，其他协议的使用介绍，请看《STM32H7xx 参考手册_V3（中文版）.pdf》第 51 章。

（1）SAI I2S 信号线

SAI 作为 I2S 使用的时候，同 I2S 接口连接的信号线如表 60.1.3.1 所示：

信号线	说明
FS_A/B	通道识别信号，连接 I2S 的左/右对齐时钟信号（LRC）
SCK_A/B	位时钟信号，连接 I2S 的位时钟信号（BLCK）
SD_A/B	数据输入/输出脚，连接 I2S 的数据输出/输入脚（DACDAT/ADCDAT）
MCLK_A/B	主时钟信号，连接 I2S 的主时钟引脚（MCLK）

表 60.1.3.1 SAI 同 I2S 接口连接关系表

表中，A/B 表示 SAI 内部的两个独立的音频子模块，可以独立的连接 I2S，也可以共同连接同一个 I2S（主从同步模式），主从同步模式，常用于全双工 I2S 通信（读/写同时进行），主从同步模式，还可以省略一些信号线（SCK/FS/MCLK 等）。

FS_A/B：连接 I2S 的 LRC 脚，用于切换左右声道的数据，它的频率等于音频信号采样率（fs）。

SCK_A/B：连接 I2S 的 BLCK 脚，用作位时钟，是 I2S 主模式下的串行时钟输出以及从模式下的串行时钟输入。SCK_A/B 频率= FS_A/B 频率（fs）*slot 个数*单个 slot 大小（slot 后面介绍）。

SD_A/B：连接 I2S 的 DACDAT/ADCDAT 脚，是数据输入/输出脚，用于发送或接收数据（单个音频子模块，只能做半双工通信。全双工需要 2 个音频子模块同时工作，使用主从同步模式）。

MCLK_A/B：连接 I2S 的 MCLK 脚，是主时钟输出脚，固定输出频率为 256*fs，fs 即音频信号采样频率（fs）。

（2）SAI slot 简介

slot 是 SAI 音频帧中的基本元素，音频帧中 slot 的数目通过 SAI_xSLOTR 寄存器配置，每个音频帧的 slot 数，最大是 16。在 I2S 模式下，SAI 中 slot 的传输方式如图 60.1.3.2 所示：

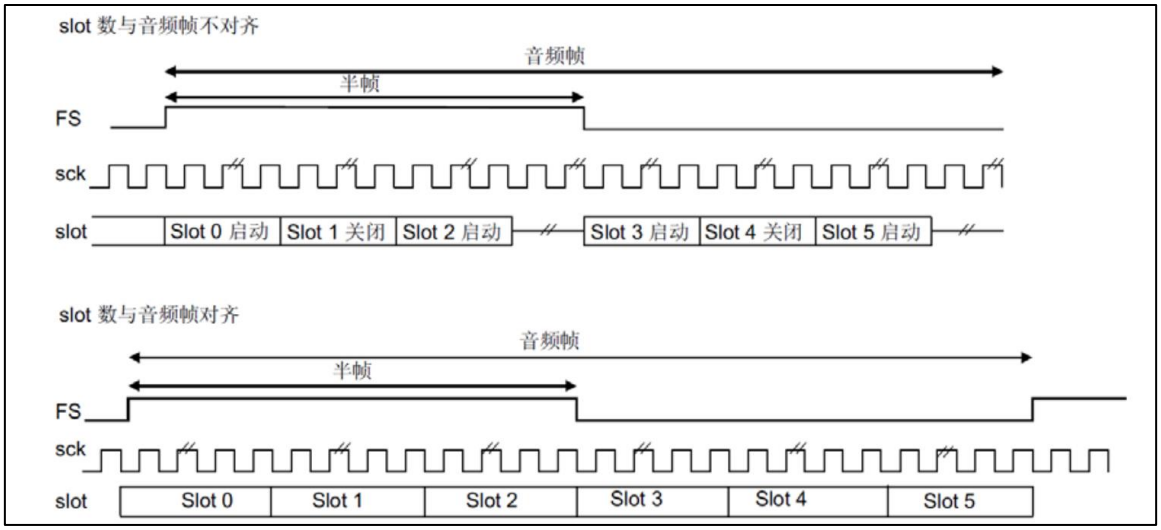


图 60.1.3.2 SAI I2S 模式下 slot 传输示意图

上图中，一个音频帧中，slot 的个数为 6 个，每个半帧有 3 个 slot，根据 slot 数与音频帧的对齐与否分为两种情况，我们一般设计为 slot 数与音频帧对齐，也就是图 60.1.3.2 中下部分图所示的传输方式：一个半帧刚好是 3 个 slot，每个 slot 可以传输一个声道的音频数据，这样，6 个 slot 就可以传输 6 个声道的音频数据。一般音频文件都是立体声，所以只需要 2 个 slot 即可，每个半帧一个 slot。STM32 的 SAI 最多可以实现 16 声道数据传输（16 个 slot）。

每个 slot 的大小是可以配置的，如图 60.1.3.3 所示：

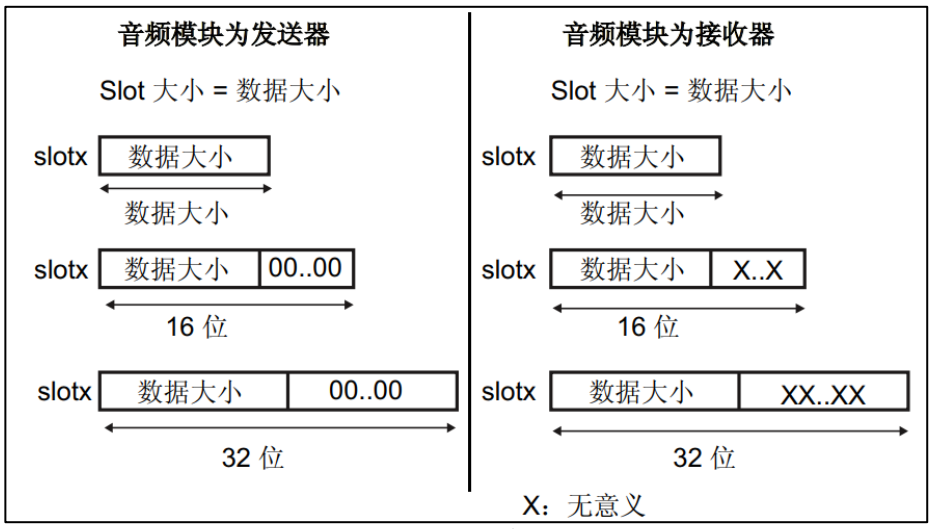


图 60.1.3.3 slot 大小配置

由图可知，数据大小（DS）可以和 slot 相等，也可以不相等（16bit/32bit），当数据大小小于 slot 大小的时候，可以通过 SAI_xSLOTR 寄存器的 FBOFF 位设置数据的偏移。各种设置的约束条件为：

FBOFF ≤（SLOTSZ-DS）
DS ≤ SLOTSZ
NBSLOT * SLOTSZ≤FRL

其中，FBOFF 为数据在 slot 里面的偏移量；SLOTSZ 为单个 slot 的位数；DS 为数据大小位数；NBSLOT 为一帧中 slot 的个数；FRL 为帧长度（位数）。

在 I2S 模式下，我们配置 slot 大小为 32 位，每一帧 slot 个数为 2 个，偏移量为 0，这样就可以支持 16~32 位的立体声音乐播放了。

（3）SAI 时钟发生器

SAI 每个音频子模块都有自己的时钟发生器，这样两个模块完全独立，可以同时工作，并互不干扰。当音频模块定义为主模块时，时钟发生器将产生位时钟（SCK）以及用于外部解码器的主时钟（MCLK），当音频模块定义为从模块时，时钟发生器将关闭（关 SCK 和 MCLK）。SAI 的时钟发生器架构如图 60.1.3.4 所示：

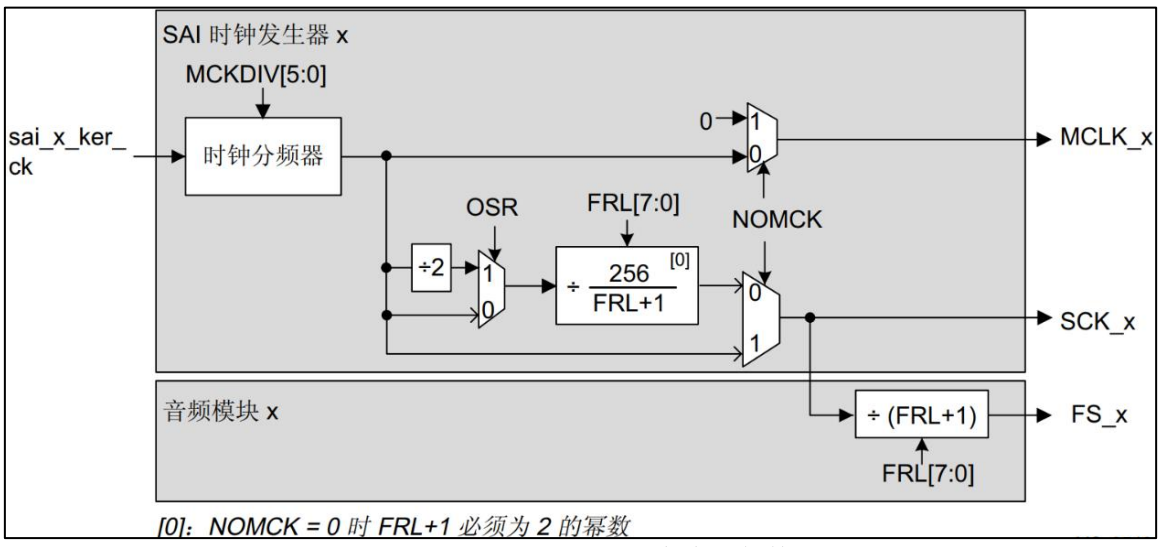


图 60.1.3.4 SAI 时钟发生器架构

图中，NOMCK 可以用于控制是否使能 MCLK 输出。如果设置为 1，则 MCLK 无输出，且 SCK 的时钟直接来自主分频器分频（由 MCKIV[5:0]控制分频系数）。如果设置为 0，则 MCLK 有输出，且 SCK 的时钟来自位时钟分频器（FRL[7:0]）的分频。我们一般设置 NOMCLK=0，使能 MCLK 输出。

SAI1 的 sai_x_ker_ck 时钟来源由：RCC_D2CCIP1R 寄存器的 SAI1SEL[2:0]位选择，本章我们设置 SAI1SEL[2:0]=010，选择 SAI1 的 sai_x_ker_ck 时钟来自 pll3_p_ck。需要注意的是：LTDC 的时钟来自 pll3_r_ck，和 pll3_p_ck 共用一个倍频系数，不同音频采样率下，倍频系数可能不一样，因此可能会导致 LTDC 的频率变化，所以，我们在设置不同音频采样率的时候，需要根据当前倍频系数和 LTDC 的时钟频率，来调整 PLL3 的 r 分频系数。

sai_x_ker_ck 经过主时钟分频器（MCKDIV[5:0]）分频，作为主时钟（MCLK）提供给 ES8388，同时，经主分频器（MCKDIV）分频后，还会经由时钟分频器（FRL[7:0]）分频，作为位时钟（SCK）提供给 ES8388。

OSR 用于设置主时钟的过采样率，当 OSR 设置为 0 时，$F_{MCLK_x}=256*F_{FS_x}$；当 OSR 设置为 1 时，$F_{MCLK_x}=512*F_{FS_x}$；我们一般设置 OSR 为 0，选择 256*fs。

接下来，我们重点介绍当：NOMCK=0 且 OSR=0 时，主时钟（F_{MCLK_x}）、位时钟（F_{SCK_x}）和帧同步时钟（F_{FS_x}，也叫左右时钟/采样时钟）的计算关系：

当 MCKDIV[5:0]=000000 的时候，$F_{MCLKx}=F_{sai_x_ker_ck}$

当 MCKDIV[5:0]!=000000 的时候，$F_{MCLKx}=F_{sai_x_ker_ck}$/MCKDIV[5:0]

位时钟（SCK）计算公式：$F_{SCK_x}=F_{MCLK_x}*(FRL[7:0]+1)/256$

左右时钟计算公式：$F_{FS_x}=F_{MCLKx}/256$

其中：256 是 MCLK 和音频采样率之间的固定比率（当 OSR=0 时，MCLK 恒等于 fs*256）;FRL[7:0]是音频帧中的位时钟-1（在 I2S 协议下，必须是奇数，+1 后是偶数）；因此，我们可以得到音频采样率（fs）的计算公式为：

当 MCKDIV[5:0]==000000 的时候：$fs=F_{SAI_x_ker_ck}/256$

当 MCKDIV[5:0]!=000000 的时候：$fs=F_{SAI_x_ker_x}$/(MCLKDIV[5:0]*256)

其中：$F_{sai_x_ker_ck}$ 我们一般选择来自来自 PLL3 的 P 分频，即：pll3_p_ck，其计算公式为：

```
pll3_p_ck = (HSE / PLL3DIVM) * (PLL3DIVN+1) / (PLL3DIVP + 1)
```

HSE 我们是 25MHz，PLL3DIVM 我们一般设置为 25，这样结合以上公式，可得 fs 的计算公式如下：

MCKDIV[5:0]!=000000 时：fs=1000*(PLL3DIVN+1)/(PLL3DIVP+1)/(MCKDIV[5:0]*256)

MCKDIV[5:0]==000000 时：fs=1000*(PLL3DIVN+1)/(PLL3DIVP+1)/(256)

其中：fs 单位是：KHz；PLL3DIVN 取值范围：3~511；PLL3DIVP 取值范围：0~127；MCKDIV 取值范围：0~63；

根据以上约束条件，我们便可以跟据 fs 来设置各个系数的值了，不过很多时候，并不能取得和 fs 一模一样的频率，只能近似等于 fs，比如 44.1KHz 采样率，我们设置 PLL3DIVN=428，PLL3DIVP=0，MCKDIV=38，得到 fs=44.0995KHz，误差为：0.0011%。晶振频率决定了有时无法通过分频得到我们所要的 fs，所以，某些 fs 如果要实现零误差，大家必须得选用了外部时钟才可以。

如果要通过程序去计算这些系数的值，是比较麻烦的，所以，我们事先计算好常用 fs 对应的系数值，建立一个表，这样用的时候只需要查表取值就可以了，大大简化了代码，常用 fs 对应叙述表如下：

```
const uint16_t SAI_PSC_TBL[][5]=
{
    { 800 , 344, 7, 0, 12 },     /* 8Khz 采样率 */
    { 1102, 429, 2, 18, 2 },     /* 11.025Khz 采样率 */
    { 1600, 344, 7, 0, 6 },      /* 16Khz 采样率 */
    { 2205, 429, 2, 18,1 },      /* 22.05Khz 采样率 */
    { 3200, 344, 7, 0, 3 },      /* 32Khz 采样率 */
    { 4410, 429, 2, 18, 0 },     /* 44.1Khz 采样率 */
    { 4800, 344, 7, 0, 2 },      /* 48Khz 采样率 */
    { 8820, 271, 2, 2, 1 },      /* 88.2Khz 采样率 */
    { 9600, 344, 7, 0, 1 },      /* 96Khz 采样率 */
    { 17640, 271, 2, 2, 0 },     /* 176.4Khz 采样率  */
    { 19200, 344, 7, 0, 0 },     /* 192Khz 采样率 */
};
```

有了上面的 fs-系数对应表，我们可以很方便的完成 SAI 的时钟配置。

（4）SAI 相关寄存器

接下来，我们看看本章需要用到的一些相关寄存器。

首先是 SAI 配置寄存器：SAI_xCR1，该寄存器各位描述如图 60.1.3.5 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	OSR	MCKDIV[5:0]						NOMCK	Res.	DMAEN	SAIEN
					rw	rw	rw	rw	rw	rw	rw	rw		rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	OUTDRIV	MONO	SYNCEN[1:0]		CKSTR	LSBFIRST	DS[2:0]			Res.	PRTCFG[1:0]		MODE[1:0]	
		rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw

图 60.1.3.5 寄存器 SAI_xCR1 各位描述

该寄存器的配置，需要在禁止 SAI 的状况下配置，接下来，看看本章我们需要用到的各位的描述：

MODE[1:0]位：00，主发送器；01，主接收器；10，从发送器；11，从接收器。我们用来播放音乐，设置为 00，就可以了。

PRTCFG[1:0]位：00，自由协议（I2S/LSB/MSB/TDM/PCM/DSP 等）；01：SPDIF 协议；10，AC ’97 协议。我们使用 I2S 协议，需要设置为 00。

DS[2:0]位： 010~111，表示 8/10/16/20/24/32 位数据大小，我们使用的音频一般是 16/24 位，所以设置这三个位为： 100（ 16 位）或 110（ 24 位）。

LSBFIRST 位：控制数据传输时是 MSB 还是 LSB， I2S 为 MSB，我们设置该位为 0。

CKSTR 位：设置时钟选通边沿，我们设置为 1，即数据在时钟的上升沿选通。

SYNCEN[1:0]位： 00，音频模块异步工作； 01，音频模块与另外一个音频模块同步。我们要控制 ES8388 播放音乐，需要设置音频模工作在异步模式，即设置 SYNCEN[1:0]=00。

MONO 位：用于设置单声道/立体声模式，我们设置为 0，工作在立体声模式。

OUTDIV 位： 0，当 SAIEN 置 1 时，驱动音频模块输出； 1，在该位设置为 1 后，立即驱动音频模块输出。这里我们设置为 1。

SAIxEN 位： 0，禁止音频模块； 1，使能音频模块；注意：必须在所有 SAI 配置完成以后，才设置该位为 1。

DMAEN 位： DMA 使能位， 0，禁止 DMA； 1，使能 DMA；我们设置为 1，使能 DMA。

NOMCK 位： 0，使能主时钟和位时钟分频器； 1，禁止主时钟和位时钟分频器；我们一般设置为 0。

MCKDIV[5:0]：主时钟分频器， 当设置为 000000 时，表示 1 分频；其他情况，则分频值为：

MCKDIV[5:0]；我们需要根据音频采样率(fs)的不同来设置不同的值。

OSR 位：主时钟过采样设置位，设置为 0，MCLK=F_{FS}×256；设置为 1 时 MCLK=F_{FS}×512；我们一般设置该位为 0。

第二个是 SAI 帧配置寄存器： SAI_xFRCR，该寄存器各位描述如图 60.1.3.6 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FSOFF	FSPOL	FSDEF
													rw	rw	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	FSALL[6:0]							FRL[7:0]							
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 60.1.3.6 寄存器 SAI_xFRCR 各位描述

FRL[7:0]位：帧长度设置位，它等于音频帧中 SCK 的个数-1。FRL 的最小值为 8，最大值为 256，且 FRL+1 应该为偶数，并且是 2 的指数倍关系。

FSALL[6:0]位：帧同步有效电平长度，用于指定 FS 信号的有效电平长度，即高电平/低电平的宽度，应该等于帧长度的一半。计算方法为： FSALL=(FRL+1)/2-1。

FSDEF 位：帧同步定义， 0， FS 信号为起始帧信号； 1， FS 信号为 SOF 信号+通道识别信号。使用 I2S 协议的时候，我们设置 FS 为 1。

FSPOL 位：帧同步极性设置， 0， FS 低电平有效（下降沿）； 1， FS 高电平有效（上升沿）；我们设置为 FS 低电平有效，即 FSPOL 位为 0。

FSOFF：帧同步偏移， 0，在 slot0 的第一位上使能 FS； 1，在 slot0 的第一位的前一位上使能 FS。使用 I2S 协议时，需要设置 FSOFF 位为 1，以匹配 I2S 协议（见图 60.1.2.1）。

第三个是 SAI slot 寄存器： SAI_xSLOTR，该寄存器各位描述如图 60.1.3.7 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SLOTEN[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	NBSLOT[3:0]				SLOTSZ[1:0]		Res.	FBOFF[4:0]				
				rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw

图 60.1.3.7 寄存器 SAI_xSLOTR 各位描述

FBOFF[4:0]位：设置第一个位的偏移量，用于设置 slot 中第一个数据传输位的位置，它表示一个偏移值。由于前面我们设置了 FSOFF 位，所以，我们设置 FBOFF=0 即可。

SLOTSZ[1:0]位：设置 slot 大小， 00， slot 大小等于数据大小； 01， 16 位； 10， 32 位；我们设置 SLOTSZ 为 10（ 32 位），以支持最高 32 位音频的播放。

NBSLOT[3:0]位：设置音频帧中 slot 的个数（设置值+1）。比如我们用立体声，使用 2 个 slot 就够了，所以设置 NBSLOT=1 即可。

SLOTEN[15:0]位：设置 slot 使能，每个位表示 1 个 slot，最多是 16 个 slot。我们使用 2 个 slot（即 slot0 和 slot1），所以，设置 SLOTEN 的最低 2 位为 1 即可。

第四个是 PLLSAI 配置寄存器： RCC_PLLCKSELR，该寄存器各位描述如图 60.1.3.8 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	DIVM3[5:0]						Res.	Res.	DIVM2[5:4]	
						rw								rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIVM2[3:0]				Res.	Res.	DIVM1[5:0]						Res.	Res.	PLLSRC[1:0]	
rw						rw								rw	

图 60.1.3.8 寄存器 RCC_PLLCKSELR 各位描述

该寄存器用于选择所有 PLL 的输入时钟和 PLL1~3 的预分频系数。对于 PLL3 来说，其预分频系数由：DIVM3[5:0]控制，取值范围为 0~63。因为外部晶振是 25MHz，为了方便计算，我们设置 DIVM3[5:0]=25，可以得 1MHz 的 PLL3 输入时钟。

第五个是 RCC 专用时钟配置寄存器：RCC_PLL3DIVR，该寄存器各位描述如图 60.1.3.9 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	DIVR3[6:0]							Res.	DIVQ3[6:0]						
	rw								rw						
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DIVP3[6:0]								DIVN3[8:0]							
rw								rw							

图 60.1.3.9 寄存器 RCC_PLL3DIVR 各位描述

该寄存器用于配置 PLL3 的 DIVN3[8:0]、DIVQ3[6:0]和 DIVR3[6:0]四个系数。

我们使用 SAI1 来驱动 ES8388，且设置 sai_1_ker_ck 来自 PLL3 的 P 分频（pll3_p_ck），其频率计算公式为：

sai_1_ker_ck=pll3_p_ck=(HSE/DIVM3)×(DIVN3+1)/(DIVP3+1)

HSE 为外部晶振频率，25MHz；DIVM3，我们一般设置为 25；DIVN3[8:0]用于设置 PLL3 的倍频系数，取值范围：3~511，表示 4~512 倍频；DIVP3[6:0]用于设置 PLL3 的 P 分频系数，取值范围为：0~127，表示 1~128 分频。DIVN3 和 DIVP3 需要根据采样率（fs）的不同的值进行合适的设置。

第六个是 SAI 数据寄存器：SAI_xDR，该寄存器各位描述如图 60.1.3.10 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATA[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

位 31:0 **DATA[31:0]**：数据 (Data)
若 FIFO 未满，写入该寄存器可向 FIFO 加载数据。
若 FIFO 非空，读取该寄存器可清空 FIFO 。

图 60.1.3.10 寄存器 SAI_xDR 各位描述

当我们需要向 ES8388 发送音频数据的时候，通过写这个寄存器，就可以实现。不过，我们采用 DMA 来传输，所以直接设置 DMA 的外设地址位 SAI_xDR 即可。

此外，使用 FIFO 的时候，还要用到 SAI_xCR2 寄存器设置 FIFO 阈值和刷新，我们这里就不做多的介绍了，请大家参考《STM32H7xx 参考手册_V3（中文版）.pdf》第 51.5.10 节。

SAI 是相关寄存器，就给大家介绍到这里。

60.2 硬件设计

1. 例程功能

本章实验功能简介：开机后，先初始化各外设，然后检测字库是否存在，如果检测无问题，则开始循环播放 SD 卡 MUSIC 文件夹里面的歌曲（必须在 SD 卡根目录建立一个 MUSIC 文件夹，并存放歌曲在里面），在 TFTLCD 上显示歌曲名字、播放时间、歌曲总时间、歌曲总数目、当前歌曲的编号等信息。KEY0 用于选择下一曲，KEY2 用于选择上一曲，KEY_UP 用来控制暂停/继续播放。LED0 闪烁，提示程序运行状态。

2. 硬件资源

本实验，大家需要准备 1 个 SD 卡（在里面新建一个 MUSIC 文件夹，并存放一些歌曲在 MUSIC 文件夹下）和一个耳机（非必备），分别插入 SD 卡接口和耳机接口，然后下载本实验就可以通过耳机或板载喇叭来听歌了。实验用到的硬件资源如下：

1）LED 灯
LED0 - PB1
LED1 - PB0
2）独立按键
KEY0 - PH3
KEY1 - PH2
KEY2 - PC13
KEY_UP - PA0 (程序中的宏名:WK_UP)
3）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
5）SD 卡，通过 SDMMC 连接
6）NOR FLASH(SPI FLASH 芯片，连接在 SPI5 上)
7）ES8388 音频 CODEC 芯片，通过 SAI 驱动
8）SAI 音频接口

3. 原理图

阿波罗 STM32H743 开发板板载了 ES8388 解码芯片的驱动电路，原理图如图 60.2.1 所示：

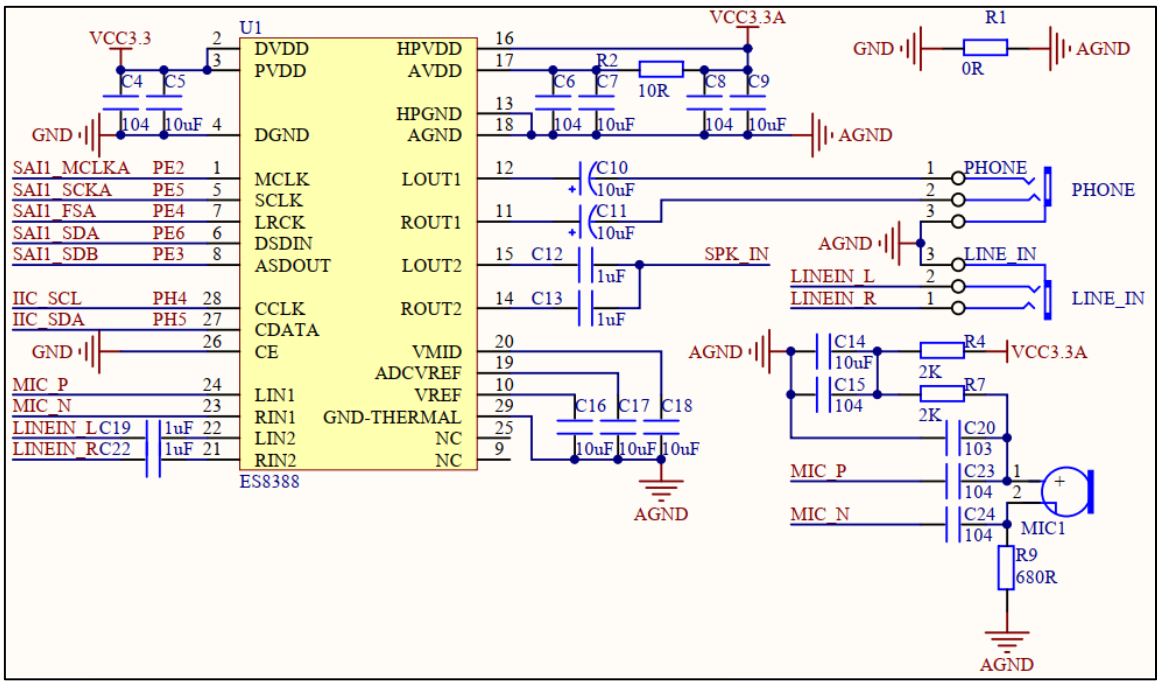


图 60.2.1 ES8388 与 STM32H743 原理图

图中，PHONE 接口可以用来插耳机，SPK+和 SPK-连接了板载的喇叭（在开发板底部）。

60.3 程序设计

60.3.1 程序流程图

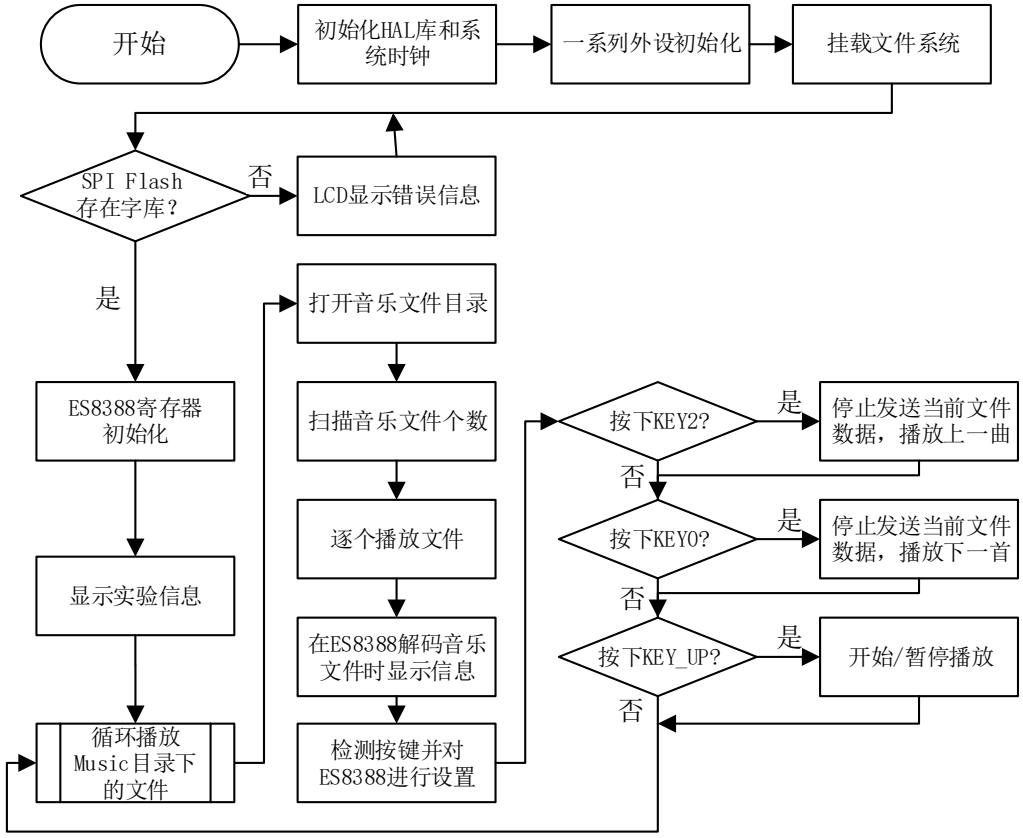


图 60.3.1.1 音乐播放器实验程序流程图

音乐播放我们从 SD 卡的指定目前读取音乐文件，解析格式正确后，通过 SPI 不断向 ES8388 发送文件数据至播放完成，ES8388 解码后通过选择扬声器或直接从耳机输出音乐。为了交互性，我们设置板载的按键用于控制播放的歌曲切换及开始/暂停播放。

60.3.2 程序解析

音乐文件我们要通过 SD 卡来传给单片机，那我们自然要用到文件系统。LCD、按键交互这些我们也需要实现，同样我们为了快速建立工程，复制之前的实验 43 FATFS 实验来修改成我们音乐播放器实验。

由于播放功能涉及到多个外设的配合使用，用文件系统读音频文件，做播放控制等，所以我们把 ES8388 的硬件驱动放到 BSP 目录下，播放功能作为 APP 放到 USER 目录下。

1. SAI 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码，SAI 的驱动主要包括两个文件：sai.c 和 sai.h。

除去 SAI 的管脚，我们需要初始其它 IO 的模式，我们在头文件 sai.h 中定义 SAI 的引脚，方便如果 IO 变更之后作修改：

```
/* 引脚复用为 SAI */
#define SAIA_CLK_GPIO_PORT              GPIOE
#define SAIA_CLK_GPIO_PIN               GPIO_PIN_2
#define SAIA_CLK_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOE_CLK_ENABLE(); }while(0)    /* PE 口时钟使能 */

#define SAIA_SCK_GPIO_PORT              GPIOE
#define SAIA_SCK_GPIO_PIN               GPIO_PIN_5
#define SAIA_SCK_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOE_CLK_ENABLE(); }while(0)    /* PE 口时钟使能 */

#define SAIA_FSA_GPIO_PORT              GPIOE
#define SAIA_FSA_GPIO_PIN               GPIO_PIN_4
#define SAIA_FSA_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOE_CLK_ENABLE(); }while(0)    /* PE 口时钟使能 */

#define SAIA_SDA_GPIO_PORT              GPIOE
#define SAIA_SDA_GPIO_PIN               GPIO_PIN_6
#define SAIA_SDA_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOE_CLK_ENABLE(); }while(0)    /* PE 口时钟使能 */

#define SAIA_SDB_GPIO_PORT              GPIOE
#define SAIA_SDB_GPIO_PIN               GPIO_PIN_3
#define SAIA_SDB_GPIO_CLK_ENABLE()
                do{ __HAL_RCC_GPIOE_CLK_ENABLE(); }while(0)    /* PE 口时钟使能 */

/* SAI1 相关定义 */
#define SAIA_SAI_CLK_ENABLE()
                do{ __HAL_RCC_SAI1_CLK_ENABLE(); }while(0)     /* SAI1 时钟使能 */

#define SAIA_TX_DMASx                   DMA2_Stream3
#define SAIA_TX_DMASx_Channel           DMA_CHANNEL_0
#define SAIA_TX_DMASx_IRQHandler        DMA2_Stream3_IRQHandler
#define SAIA_TX_DMASx_IRQ               DMA2_Stream3_IRQn
#define SAIA_TX_DMASx_FLAG              DMA_FLAG_TCIF3_7
#define SAIA_TX_DMA_CLK_ENABLE()
          do{ __HAL_RCC_DMA2_CLK_ENABLE(); }while(0)    /* SAIA TX DMA 时钟使能 */
```

接下来开始介绍 sai.c，主要是 SAI 的初始化和 DMA 初始化，代码如下：

```
SAI_HandleTypeDef g_sai1_a_handle;          /* SAI1 Block A 句柄 */
DMA_HandleTypeDef g_sai1_tx_dma_handle;     /* DMA 发送句柄 */

/**
```

```
 * @brief       SAI Block A 初始化,I2S,飞利浦标准
 * @param       mode:工作模式,可以设置:
 *                           SAI_MODEMASTER_TX/SAI_MODEMASTER_RX/
 *                           SAI_MODESLAVE_TX/SAI_MODESLAVE_RX
 * @param       cpol:数据在时钟的上升/下降沿选通，可以设置:
                SAI_CLOCKSTROBING_FALLINGEDGE/SAI_CLOCKSTROBING_RISINGEDGE
 * @param       datalen:数据大小,可以设置：SAI_DATASIZE_8/10/16/20/24/32
 * @retval      无
 */
void sai1_saia_init(uint32_t mode, uint32_t cpol, uint32_t datalen)
{
    HAL_SAI_DeInit(&g_sai1_a_handle);                       /* 清除以前的配置 */

    g_sai1_a_handle.Instance = SAI1_Block_A;                /* SAI1 Bock A */
    g_sai1_a_handle.Init.AudioMode = mode;                  /* 设置 SAI1 工作模式 */
    g_sai1_a_handle.Init.Synchro = SAI_ASYNCHRONOUS;        /* 音频模块异步 */
    /*立即驱动音频模块输出 */
    g_sai1_a_handle.Init.OutputDrive=SAI_OUTPUTDRIVE_ENABLE;
    g_sai1_a_handle.Init.NoDivider=SAI_MASTERDIVIDER_ENABLE;
    /*使能主时钟分频器(MCKDIV) */
    g_sai1_a_handle.Init.FIFOThreshold=SAI_FIFOTHRESHOLD_1QF;
    /*设置 FIFO 阈值,1/4FIFO */
    g_sai1_a_handle.Init.ClockSource =SAI_CLKSOURCE_PLLSAI;/*SIA 时钟源为 PLLSAI*/
    g_sai1_a_handle.Init.MonoStereoMode = SAI_STEREOMODE;   /* 立体声模式 */
    g_sai1_a_handle.Init.Protocol = SAI_FREE_PROTOCOL;
    /* 设置 SAI1 协议为:自由协议(支持 I2S/LSB/MSB/TDM/PCM/DSP 等协议) */
    g_sai1_a_handle.Init.DataSize = datalen;                /* 设置数据大小 */
    g_sai1_a_handle.Init.FirstBit = SAI_FIRSTBIT_MSB;       /* 数据 MSB 位优先 */
    g_sai1_a_handle.Init.ClockStrobing = cpol; /* 数据在时钟的上升/下降沿选通 */
    /* 帧设置 */
    g_sai1_a_handle.FrameInit.FrameLength = 64;
    /* 设置帧长度为 64,左通道 32 个 SCK,右通道 32 个 SCK. */
    g_sai1_a_handle.FrameInit.ActiveFrameLength = 32;
     /* 设置帧同步有效电平长度,在 SAI 模式下=1/2 帧长 */
    g_sai1_a_handle.FrameInit.FSDefinition = SAI_FS_CHANNEL_IDENTIFICATION;
    /* FS 信号为 SOF 信号+通道识别信号 */
    /*FS 低电平有效(下降沿)*/
    g_sai1_a_handle.FrameInit.FSPolarity=SAI_FS_ACTIVE_LOW;
    g_sai1_a_handle.FrameInit.FSOffset = SAI_FS_BEFOREFIRSTBIT;
    /* 在 slot0 的第一位的前一位使能 FS,以匹配飞利浦标准 */

    /* SLOT 设置 */
    g_sai1_a_handle.SlotInit.FirstBitOffset = 0;       /* slot 偏移(FBOFF)为 0 */
    g_sai1_a_handle.SlotInit.SlotSize = SAI_SLOTSIZE_32B; /* slot 大小为 32 位 */
    g_sai1_a_handle.SlotInit.SlotNumber = 2;                /* slot 数为 2 个 */
    g_sai1_a_handle.SlotInit.SlotActive = SAI_SLOTACTIVE_0|SAI_SLOTACTIVE_1;
    /* 使能 slot0 和 slot1 */
    HAL_SAI_Init(&g_sai1_a_handle);                                 /* 初始化 SAI */
    __HAL_SAI_ENABLE(&g_sai1_a_handle);                             /* 使能 SAI */
}

/**
 * @brief       SAI 底层驱动，引脚配置，时钟使能
 * @note        此函数会被 HAL_SAI_Init()调用
 * @param       hsdram:SAI 句柄
 * @retval      无
 */
void HAL_SAI_MspInit(SAI_HandleTypeDef *hsai)
{
    GPIO_InitTypeDef gpio_init_struct;
```

```
    SAIA_SAI_CLK_ENABLE();                          /* 使能 SAI1 时钟 */
    SAIA_CLK_GPIO_CLK_ENABLE();                     /* CLK 引脚时钟使能 */
    SAIA_SCK_GPIO_CLK_ENABLE();                     /* SCK 引脚时钟使能 */
    SAIA_FSA_GPIO_CLK_ENABLE();                     /* FSA 引脚时钟使能 */
    SAIA_SDA_GPIO_CLK_ENABLE();                     /* SDA 引脚时钟使能 */
    SAIA_SDB_GPIO_CLK_ENABLE();                     /* SDB 引脚时钟使能 */

    gpio_init_struct.Pin = SAI1_CLK_GPIO_PIN;
    gpio_init_struct.Mode = GPIO_MODE_AF_PP;                    /* 推挽复用 */
    gpio_init_struct.Pull = GPIO_PULLUP;                        /* 上拉 */
    gpio_init_struct.Speed = GPIO_SPEED_HIGH;                   /* 高速 */
    gpio_init_struct.Alternate = GPIO_AF6_SAI1;                 /* 复用为 SAI */
    HAL_GPIO_Init(SAI1_CLK_GPIO_PORT, &gpio_init_struct);       /* 初始化 CLK 引脚 */

    gpio_init_struct.Pin = SAI1_SCK_GPIO_PIN;
    HAL_GPIO_Init(SAI1_SCK_GPIO_PORT,  &gpio_init_struct);      /* 初始化 SCK 引脚 */

    gpio_init_struct.Pin = SAI1_FSA_GPIO_PIN;
    HAL_GPIO_Init(SAI1_FSA_GPIO_PORT,  &gpio_init_struct);      /* 初始化 FSA 引脚 */

    gpio_init_struct.Pin = SAI1_SDA_GPIO_PIN;
    HAL_GPIO_Init(SAI1_SDA_GPIO_PORT,  &gpio_init_struct);      /* 初始化 SDA 引脚 */

    gpio_init_struct.Pin = SAI1_SDB_GPIO_PIN;
    HAL_GPIO_Init(SAI1_SDB_GPIO_PORT,  &gpio_init_struct);      /* 初始化 SDB 引脚 */
}

/* SAI Block A 采样率设置
 * 采样率计算公式(以 NOMCK=0,OSR=0 为前提):
 * Fmclk = 256*Fs = sai_x_ker_ck / MCKDIV[5:0]
 * Fs = sai_x_ker_ck / (256 * MCKDIV[5:0])
 * Fsck = Fmclk * (FRL[7:0]+1) / 256 = Fs * (FRL[7:0] + 1)
 * 其中:
 * sai_x_ker_ck = (HSE / PLL2DIVM) * (PLL2DIVN + 1) / (PLL2DIVP + 1)
 * HSE:一般为 25Mhz
 * PLL2DIVM     :1~63,表示 1~63 分频
 * PLL2DIVN     :3~511,表示 4~512 倍频
 * PLL2DIVP     :0~127,表示 1~128 分频
 * MCKDIV       :0~63,表示 1~63 分频,0 也是 1 分频,推荐设置为偶数
 * SAI A 分频系数表@PLL2DIVM=25,HSE=25Mhz,即 vco 输入频率为 1Mhz
 * 表格式:
 * 采样率|(PLL2DIVN+1)|(PLL2DIVP+1)|MCKDIV
 */
const uint16_t SAI_PSC_TBL[][5]=
{
    { 800 , 344, 7, 0, 12 },    /* 8Khz 采样率 */
    { 1102, 429, 2, 18, 2 },    /* 11.025Khz 采样率 */
    { 1600, 344, 7, 0, 6 },     /* 16Khz 采样率 */
    { 2205, 429, 2, 18,1 },     /* 22.05Khz 采样率 */
    { 3200, 344, 7, 0, 3 },     /* 32Khz 采样率 */
    { 4410, 429, 2, 18, 0 },    /* 44.1Khz 采样率 */
    { 4800, 344, 7, 0, 2 },     /* 48Khz 采样率 */
    { 8820, 271, 2, 2, 1 },     /* 88.2Khz 采样率 */
    { 9600, 344, 7, 0, 1 },     /* 96Khz 采样率 */
    { 17640, 271, 2, 2, 0 },    /* 176.4Khz 采样率  */
    { 19200, 344, 7, 0, 0 },    /* 192Khz 采样率 */
};

/**
 * @brief       开启 SAI 的 DMA 功能,HAL 库没有提供此函数
```

```
 * @note        因此我们需要自己操作寄存器编写一个
 * @param       无
 * @retval      无
 */
void sai1_saia_dma_enable(void)
{
    uint32_t tempreg = 0;
    tempreg = SAI1_Block_A->CR1;            /* 先读出以前的设置 */
    tempreg |= 1 << 17;                     /* 使能 DMA */
    SAI1_Block_A->CR1 = tempreg;            /* 写入 CR1 寄存器中 */
}

/**
 * @brief       设置 SAIA 的采样率(@MCKEN)
 * @param       samplerate:采样率, 单位:Hz
 * @retval      0,设置成功
 *              1,无法设置
 */
uint8_t sai1_samplerate_set(uint32_t samplerate)
{
    uint8_t i = 0;

    RCC_PeriphCLKInitTypeDef rcc_sai1_sture;

    for (i = 0; i < (sizeof(SAI_PSC_TBL) / 10); i++)/* 看看改采样率是否可以支持 */
    {
        if ((samplerate / 10) == SAI_PSC_TBL[i][0])break;
    }

    if (i == (sizeof(SAI_PSC_TBL) / 10))return 1;   /* 搜遍了也找不到 */
    /* 外设时钟源选择*/
    rcc_sai1_sture.PeriphClockSelection = RCC_PERIPHCLK_SAI1;
    rcc_sai1_sture.SaiClockSelection = RCC_SAI1CLKSOURCE_PLL2;
    rcc_sai1_sture.PLL2.PLL2M = 25;                         /* 设置 PLL2M */
    rcc_sai1_sture.PLL2.PLL2N = (uint32_t)SAI_PSC_TBL[i][2];/* 设置 PLL2N*/
    rcc_sai1_sture.PLL2.PLL2P =(uint32_t)SAI_PSC_TBL[i][2]; /* 设置 PLL2P*/
    HAL_RCCEx_PeriphCLKConfig(&rcc_sai1_sture);             /* 设置时钟 */

    __HAL_SAI_DISABLE(&g_sai1_a_handle);                /* 关闭 SAI */
    g_sai1_a_handle.Init.AudioFrequency = samplerate; /* 设置播放频率 */
    HAL_SAI_Init(&g_sai1_a_handle);                     /* 初始化 SAI */
    saia_dma_enable();                                  /* 开启 SAI 的 DMA 功能 */
    __HAL_SAI_ENABLE(&g_sai1_a_handle);                 /* 开启 SAI */

    sai_ltdc_clkset(SAI_PSC_TBL[i][1]);  /* 针对 RGB 屏, 需要重新修改 PLL3R 的值 */
    return 0;
}

/**
 * @brief       SAI1 TX DMA 配置
 * @note        设置为双缓冲模式,并开启 DMA 传输完成中断
 * @param       buf0:M0AR 地址.
 * @param       buf1:M1AR 地址.
 * @param       num:每次传输数据量
 * @param       width:位宽(存储器和外设,同时设置),0,8 位;1,16 位;2,32 位;
 * @retval      0,设置成功
 *              1,无法设置
 */
void sai1_tx_dma_init(uint8_t *buf0,uint8_t *buf1,uint16_t num,uint8_t width)
{
    uint32_t memwidth = 0, perwidth = 0;       /* 外设和存储器位宽 */
```

```
    switch (width)
    {
        case 0:          /* 8 位 */
            memwidth = DMA_MDATAALIGN_BYTE;
            perwidth = DMA_PDATAALIGN_BYTE;
            break;

        case 1:          /* 16 位 */
            memwidth = DMA_MDATAALIGN_HALFWORD;
            perwidth = DMA_PDATAALIGN_HALFWORD;
            break;

        case 2:          /* 32 位 */
            memwidth = DMA_MDATAALIGN_WORD;
            perwidth = DMA_PDATAALIGN_WORD;
            break;
    }
    SAIA_TX_DMA_CLK_ENABLE();                           /* 使能 DMA2 时钟 */
    __HAL_LINKDMA(&g_sai1_a_handle, hdmatx, g_sai1_tx_dma_handle);
    /* 将 DMA 与 SAI 联系起来 */

    g_sai1_tx_dma_handle.Instance = SAIA_TX_DMASx;            /* DMA2 数据流 3 */
    g_sai1_tx_dma_handle.Request=DMA_REQUEST_SAI1_A;          /* SAI1 BlockA */
    g_sai1_tx_dma_handle.Init.Direction=DMA_MEMORY_TO_PERIPH;/* 存储器到外设模式*/
    g_sai1_tx_dma_handle.Init.PeriphInc = DMA_PINC_DISABLE;  /* 外设非增量模式 */
    g_sai1_tx_dma_handle.Init.MemInc = DMA_MINC_ENABLE;       /* 存储器增量模式 */
    /* 外设数据长度:16/32 位 */
    g_sai1_tx_dma_handle.Init.PeriphDataAlignment=perwidth;
    /* 存储器数据长度:16/32 位*/
    g_sai1_tx_dma_handle.Init.MemDataAlignment=memwidth;
    g_sai1_tx_dma_handle.Init.Mode = DMA_CIRCULAR;              /* 使用循环模式 */
    g_sai1_tx_dma_handle.Init.Priority = DMA_PRIORITY_HIGH;     /* 高优先级 */
    g_sai1_tx_dma_handle.Init.FIFOMode = DMA_FIFOMODE_DISABLE; /* 不使用 FIFO */
    g_sai1_tx_dma_handle.Init.MemBurst=DMA_MBURST_SINGLE;/* 存储器单次突发传输 */
    g_sai1_tx_dma_handle.Init.PeriphBurst=DMA_PBURST_SINGLE;/* 外设突发单次传输 */
    HAL_DMA_DeInit(&g_sai1_tx_dma_handle);                  /* 先清除以前的设置 */
    HAL_DMA_Init(&g_sai1_tx_dma_handle);                    /* 初始化 DMA */
    /* 开启双缓冲 */
    HAL_DMAEx_MultiBufferStart(&g_sai1_tx_dma_handle, (uint32_t)buf0,
                           (uint32_t)&SAI1_Block_A->DR, (uint32_t)buf1,num);
    __HAL_DMA_DISABLE(&g_sai1_tx_dma_handle);     /* 先关闭 DMA */
    delay_us(10);                                 /* 10us 延时，防止-O2 优化出问题 */

    __HAL_DMA_ENABLE_IT(&g_sai1_tx_dma_handle, DMA_IT_TC); /* 开启传输完成中断 */
    __HAL_DMA_CLEAR_FLAG(&g_sai1_tx_dma_handle, SAIA_TX_DMASx_FLAG);
    /* 清除 DMA 传输完成中断标志位 */
    HAL_NVIC_SetPriority(SAIA_TX_DMASx_IRQ, 0, 0);       /* DMA 中断优先级 */
    HAL_NVIC_EnableIRQ(SAIA_TX_DMASx_IRQ);
}

/* SAI DMA 回调函数指针 */
void (*sai1_tx_callback)(void);  /* TX 回调函数 */

/**
 * @brief       DMA2_Stream3 中断服务函数
 * @param       无
 * @retval      无
 */
void SAI1_TX_DMASx_IRQHandler(void)
{
    if(__HAL_DMA_GET_FLAG(&g_sai1_tx_dma_handle, SAIA_TX_DMASx_FLAG) != RESET)
    {/* DMA 传输完成 */
```

```
        __HAL_DMA_CLEAR_FLAG(&g_sai1_tx_dma_handle, SAIA_TX_DMASx_FLAG);
        /* 清除 DMA 传输完成中断标志位 */
        sai_tx_callback();  /* 执行回调函数,读取数据等操作在这里面处理 */
    }
}

/**
 * @brief       SAI 开始播放
 * @param       无
 * @retval      无
 */
void sai1_play_start(void)
{
    __HAL_DMA_ENABLE(&g_sai1_tx_dma_handle);    /* 开启 DMA TX 传输 */
}

/**
 * @brief       关闭 SAI 播放
 * @param       无
 * @retval      无
 */
void sai1_play_stop(void)
{
    __HAL_DMA_DISABLE(&g_sai1_tx_dma_handle);  /* 结束播放 */
}
```

函数 sai1_saia_init 完成 SAI 初始化，通过 3 个参数设置 SAI 的详细配置信息。函数 sai1_samplerate_set 则是用前面介绍的查表法，根据采样率来设置 SAI 的时钟。函数 sai1_tx_dma_init 用于设置 SAI1 的 DMA 发送，使用双缓冲模式，发送数据给 ES8388，并开启了发送完成中断。而函数 SAI1_TX_DMASx_IRQHandler 是 DMA 数据流发送完成中断的服务函数，该函数调用 sai_tx_callback 函数（函数指针，使用前需指向特定函数）实现 DMA 数据填充。最后是 sai1_play_start 和 sai1_play_stop 函数，用于开启和关闭 DMA 中断。

2. ES8388 驱动代码

ES8388 主要用来将音频信号转换为数字信号或将数字信号转换为音频信号，接下来，我们开始介绍 ES8388 的几个函数，代码如下：

```
/**
 * @brief       ES8388 初始化
 * @param       无
 * @retval      0,初始化正常
 *              其他,错误代码
 */
uint8_t es8388_init(void)
{
    iic_init();                     /* 初始化 IIC 接口 */

    es8388_write_reg(0, 0x80);      /* 软复位 ES8388 */
    es8388_write_reg(0, 0x00);
    delay_ms(100);                  /* 等待复位 */

    es8388_write_reg(0x01, 0x58);
    es8388_write_reg(0x01, 0x50);
    es8388_write_reg(0x02, 0xF3);
    es8388_write_reg(0x02, 0xF0);

    es8388_write_reg(0x03, 0x09);       /* 麦克风偏置电源关闭 */
    es8388_write_reg(0x00, 0x06);       /* 使能参考 500K 驱动使能 */
    es8388_write_reg(0x04, 0x00);       /* DAC 电源管理，不打开任何通道 */
    es8388_write_reg(0x08, 0x00);       /* MCLK 不分频 */
    es8388_write_reg(0x2B, 0x80);       /* DAC 控制     DACLRC 与 ADCLRC 相同 */
```

```
    es8388_write_reg(0x09, 0x88);       /* ADC L/R PGA 增益配置为+24dB */
    /* ADC  数据选择为 left data = left ADC,
     * right data = left ADC  音频数据为 16bit */

    es8388_write_reg(0x0C, 0x4C);
    es8388_write_reg(0x0D, 0x02);       /* ADC 配置 MCLK/采样率=256 */
    es8388_write_reg(0x10, 0x00);       /* ADC 数字音量控制将信号衰减 L 设置为最小！*/
    es8388_write_reg(0x11, 0x00);       /* ADC 数字音量控制将信号衰减 R 设置为最小！*/

    es8388_write_reg(0x17, 0x18);       /* DAC 音频数据为 16bit */
    es8388_write_reg(0x18, 0x02);       /* DAC 配置 MCLK/采样率=256 */
    es8388_write_reg(0x1A, 0x00);       /* DAC 数字音量控制将信号衰减 L 设置为最小！ */
    es8388_write_reg(0x1B, 0x00);       /* DAC 数字音量控制将信号衰减 R 设置为最小！ */
    es8388_write_reg(0x27, 0xB8);       /* L 混频器 */
    es8388_write_reg(0x2A, 0xB8);       /* R 混频器 */
    delay_ms(100);

    return 0;
}

/**
 * @brief       ES8388 写寄存器
 * @param       reg : 寄存器地址
 * @param       val : 要写入寄存器的值
 * @retval      0,成功
 *              其他,错误代码
 */
uint8_t es8388_write_reg(uint8_t reg, uint8_t val)
{
    iic_start();

    iic_send_byte((ES8388_ADDR << 1)|0);  /* 发送器件地址+写命令 */
    if (iic_wait_ack())
    {
        return 1;                         /* 等待应答(成功?/失败?) */
    }

    iic_send_byte(reg);                   /* 写寄存器地址 */
    if (iic_wait_ack())
    {
        return 2;                         /* 等待应答(成功?/失败?) */
    }

    iic_send_byte(val & 0xFF);            /* 发送数据 */
    if (iic_wait_ack())
    {
        return 3;                         /* 等待应答(成功?/失败?) */
    }

    iic_stop();

    return 0;
}

/**
 * @brief       ES8388 读寄存器
 * @param       reg : 寄存器地址
 * @retval      读取到的数据
 */
uint8_t es8388_read_reg(uint8_t reg)
{
```

```
    uint8_t temp = 0;

    iic_start();

    iic_send_byte((ES8388_ADDR << 1) | 0);     /* 发送器件地址+写命令 */
    if (iic_wait_ack())
    {
        return 1;                              /* 等待应答(成功?/失败?) */
    }

    iic_send_byte(reg);                        /* 写寄存器地址 */
    if (iic_wait_ack())
    {
        return 1;                              /* 等待应答(成功?/失败?) */
    }

    iic_start();
    iic_send_byte((ES8388_ADDR << 1) | 1);     /* 发送器件地址+读命令 */
    if (iic_wait_ack())
    {
        return 1;                              /* 等待应答(成功?/失败?) */
    }
    temp = iic_read_byte(0);
    iic_stop();
    return temp;
}
/**
 * @brief       设置 ES8388 工作模式
 * @param       fmt : 工作模式
 *    @arg      0, 飞利浦标准 I2S;
 *    @arg      1, MSB(左对齐);
 *    @arg      2, LSB(右对齐);
 *    @arg      3, PCM/DSP
 * @param       len : 数据长度
 *    @arg      0, 24bit
 *    @arg      1, 20bit
 *    @arg      2, 18bit
 *    @arg      3, 16bit
 *    @arg      4, 32bit
 * @retval      无
 */
void es8388_sai_cfg(uint8_t fmt, uint8_t len)
{
    fmt &= 0x03;
    len &= 0x07;    /* 限定范围 */
    es8388_write_reg(23, (fmt << 1) | (len << 3));  /* R23,ES8388 工作模式设置 */
}

/**
 * @brief       设置耳机音量
 * @param       voluem : 音量大小(0 ~ 33)
 * @retval      无
 */
void es8388_hpvol_set(uint8_t volume)
{
    if (volume > 33)
    {
        volume = 33;
    }

    es8388_write_reg(0x2E, volume);
    es8388_write_reg(0x2F, volume);
}
```

```
/**
 * @brief       设置喇叭音量
 * @param       volume : 音量大小(0 ~ 33)
 * @retval      无
 */
void es8388_spkvol_set(uint8_t volume)
{
    if (volume > 33)
    {
        volume = 33;
    }

    es8388_write_reg(0x30, volume);
    es8388_write_reg(0x31, volume);
}

/**
 * @brief       设置 3D 环绕声
 * @param       depth : 0 ~ 7(3D 强度,0 关闭,7 最强)
 * @retval      无
 */
void es8388_3d_set(uint8_t depth)
{
    depth &= 0x7;                           /* 限定范围 */
    es8388_write_reg(0x1D, depth << 2);  /* R7,3D 环绕设置 */
}

/**
 * @brief       ES8388 DAC/ADC 配置
 * @param       dacen : dac 使能(1)/关闭(0)
 * @param       adcen : adc 使能(1)/关闭(0)
 * @retval      无
 */
void es8388_adda_cfg(uint8_t dacen, uint8_t adcen)
{
    uint8_t tempreg = 0;

    tempreg |= ((!dacen) << 0);
    tempreg |= ((!adcen) << 1);
    tempreg |= ((!dacen) << 2);
    tempreg |= ((!adcen) << 3);
    es8388_write_reg(0x02, tempreg);
}

/**
 * @brief       ES8388 DAC 输出通道配置
 * @param       o1en : 通道 1 使能(1)/禁止(0)
 * @param       o2en : 通道 2 使能(1)/禁止(0)
 * @retval      无
 */
void es8388_output_cfg(uint8_t o1en, uint8_t o2en)
{
    uint8_t tempreg = 0;
    tempreg |= o1en * (3 << 4);
    tempreg |= o2en * (3 << 2);
    es8388_write_reg(0x04, tempreg);
}

/**
 * @brief       ES8388 MIC 增益设置(MIC PGA 增益)
 * @param       gain : 0~8, 对应 0~24dB  3dB/Step
 * @retval      无
 */
void es8388_mic_gain(uint8_t gain)
```

```
{
    gain &= 0x0F;
    gain |= gain << 4;
    es8388_write_reg(0x09, gain);     /* R9,左右通道 PGA 增益设置 */
}

/**
 * @brief       ES8388 ALC 设置
 * @param       sel
 *   @arg       0,关闭 ALC
 *   @arg       1,右通道 ALC
 *   @arg       2,左通道 ALC
 *   @arg       3,立体声 ALC
 * @param       maxgain : 0~7,对应-6.5~+35.5dB
 * @param       minigain: 0~7,对应-12~+30dB 6dB/STEP
 * @retval      无
 */
void es8388_alc_ctrl(uint8_t sel, uint8_t maxgain, uint8_t mingain)
{
    uint8_t tempreg = 0;
    tempreg = sel << 6;
    tempreg |= (maxgain & 0x07) << 3;
    tempreg |= mingain & 0x07;
    es8388_write_reg(0x12, tempreg);     /* R18,ALC 设置 */
}

/**
 * @brief       ES8388 ADC 输出通道配置
 * @param       in : 输入通道
 *    @arg      0, 通道 1 输入
 *    @arg      1, 通道 2 输入
 * @retval      无
 */
void es8388_input_cfg(uint8_t in)
{
    es8388_write_reg(0x0A, (5 * in) << 4);     /* ADC1 输入通道选择 L/R  INPUT1 */
}
```

以上代码中，es8388_init 函数用于初始化 es8388，这里只是通用配置（ADC&DAC），初始化完成后，并不能正常播放音乐，还需要通过 es8388_adda_cfg 函数使能 DAC，然后通过设置 es8388_output_cfg 选择 DAC 输出，通过 es8388_sai_cfg 配置 SAI 工作模式，最后设置音量才可以接收 SAI 音频数据，实现音乐播放。

3. wavplay 代码

wavpaly 主要用于 wav 格式的音频文件解码，接下来看看 wavplay.c 里面的几个函数，代码如下：

```
/**
 * @brief       WAV 解析初始化
 * @param       fname : 文件路径+文件名
 * @param       wavx  : 信息存放结构体指针
 * @retval      0,打开文件成功
 *              1,打开文件失败
 *              2,非 WAV 文件
 *              3,DATA 区域未找到
 */
uint8_t wav_decode_init(uint8_t *fname, __wavctrl *wavx)
{
    FIL *ftemp;
    uint8_t *buf;
    uint32_t br = 0;
    uint8_t res = 0;
```

```
    ChunkRIFF *riff;
    ChunkFMT *fmt;
    ChunkFACT *fact;
    ChunkDATA *data;

    ftemp = (FIL*)mymalloc(SRAMIN, sizeof(FIL));
    buf = mymalloc(SRAMIN, 512);

    if (ftemp && buf)     /* 内存申请成功 */
    {
        res = f_open(ftemp, (TCHAR*)fname, FA_READ);     /* 打开文件 */

        if (res == FR_OK)
        {
            f_read(ftemp, buf, 512, &br);           /* 读取 512 字节在数据 */
            riff = (ChunkRIFF *)buf;                /* 获取 RIFF 块 */
            if (riff->Format == 0x45564157)         /* 是 WAV 文件 */
            {
                fmt = (ChunkFMT *)(buf + 12);       /* 获取 FMT 块 */
                /* 读取 FACT 块 */
                fact = (ChunkFACT *)(buf + 12 + 8 + fmt->ChunkSize);

                if (fact->ChunkID == 0x74636166 || fact->ChunkID == 0x5453494C)
                {
                    /* 具有 fact/LIST 块的时候(未测试)*/
                    wavx->datastart = 12 + 8 + fmt->ChunkSize+8+fact->ChunkSize;
                }
                else
                {
                    wavx->datastart = 12 + 8 + fmt->ChunkSize;
                }

                data = (ChunkDATA *)(buf + wavx->datastart);   /* 读取 DATA 块 */

                if (data->ChunkID == 0x61746164)            /* 解析成功! */
                {
                    wavx->audioformat = fmt->AudioFormat;   /* 音频格式 */
                    wavx->nchannels = fmt->NumOfChannels;   /* 通道数 */
                    wavx->samplerate = fmt->SampleRate;     /* 采样率 */
                    wavx->bitrate = fmt->ByteRate * 8;      /* 得到位速 */
                    wavx->blockalign = fmt->BlockAlign;     /* 块对齐 */
                    wavx->bps = fmt->BitsPerSample;         /* 位数,16/24/32 位 */

                    wavx->datasize = data->ChunkSize;       /* 数据块大小 */
                    wavx->datastart = wavx->datastart + 8; /* 数据流开始的地方. */

                    printf("wavx->audioformat:%d\r\n", wavx->audioformat);
                    printf("wavx->nchannels:%d\r\n", wavx->nchannels);
                    printf("wavx->samplerate:%d\r\n", wavx->samplerate);
                    printf("wavx->bitrate:%d\r\n", wavx->bitrate);
                    printf("wavx->blockalign:%d\r\n", wavx->blockalign);
                    printf("wavx->bps:%d\r\n", wavx->bps);
                    printf("wavx->datasize:%d\r\n", wavx->datasize);
                    printf("wavx->datastart:%d\r\n", wavx->datastart);
                }
                else
                {
                    res = 3;    /* data 区域未找到. */
                }
            }
            else
            {
```

```
                res = 2;        /* 非 wav 文件 */
            }
        }
        else
        {
            res = 1;            /* 打开文件错误 */
        }
    }

    f_close(ftemp);

    myfree(SRAMIN, ftemp); /* 释放内存 */
    myfree(SRAMIN, buf);

    return 0;
}

/**
 * @brief       填充 buf
 * @param       buf  : 填充区
 * @param       size : 填充数据量
 * @param       bits : 位数(16/24)
 * @retval      读取到的数据长度
 */
uint32_t wav_buffill(uint8_t *buf, uint16_t size, uint8_t bits)
{
    uint16_t readlen = 0;
    uint32_t bread;
    uint16_t i;
    uint32_t *p, *pbuf;

    if (bits == 24)     /* 24bit 音频,需要处理一下 */
    {
        readlen = (size / 4) * 3;     /* 此次要读取的字节数 */
        /* 读取数据 */
        f_read(g_audiodev.file, g_audiodev.tbuf, readlen, (UINT*)&bread);
        pbuf = (uint32_t *)buf;

        for (i = 0; i < size/ 4; i++)
        {
            p = (uint32_t *)(g_audiodev.tbuf + i * 3);
            pbuf[i] = p[0];
        }

        bread = (bread * 4) / 3;     /* 填充后的大小. */
    }
    else
    {
        /* 16bit 音频,直接读取数据 */
        f_read(g_audiodev.file, buf, size, (UINT*)&bread);

        if (bread < size)           /* 不够数据了,补充 0 */
        {
            for (i = bread; i < size; i++)
            {
                buf[i] = 0;
            }
        }
    }

    return bread;
}

/**
```

```
 * @brief       WAV 播放时,SAI DMA 传输回调函数
 * @param       无
 * @retval      无
 */
void wav_sai_dma_tx_callback(void)
{
    uint16_t i;

    if (SAIA_TX_DMASx->CR & (1 << 19))
    {
        wavwitchbuf = 0;

        if ((g_audiodev.status & 0x01) == 0)
        {
            for (i = 0; i < WAV_SAI_TX_DMA_BUFSIZE; i++)  /* 暂停 */
            {
                g_audiodev.saibuf1[i] = 0;                /* 填充 0 */
            }
        }
    }
    else
    {
        wavwitchbuf = 1;
        if ((g_audiodev.status & 0x01) == 0)
        {
            for (i = 0; i < WAV_SAI_TX_DMA_BUFSIZE; i++)  /* 暂停 */
            {
                g_audiodev.saibuf2[i] = 0;                /* 填充 0 */
            }
        }
    }
    wavtransferend = 1;
}

/**
 * @brief       获取当前播放时间
 * @param       fname : 文件指针
 * @param       wavx  : wavx 播放控制器
 * @retval      无
 */
void wav_get_curtime(FIL *fx, __wavctrl *wavx)
{
    long long fpos;
    wavx->totsec = wavx->datasize / (wavx->bitrate / 8); /* 歌曲总长度(单位:秒) */
    fpos = fx->fptr-wavx->datastart;                     /* 得到当前文件播放到的地方 */
    wavx->cursec = fpos*wavx->totsec / wavx->datasize;   /* 当前播放到第多少秒了? */
}

/**
 * @brief       播放某个 wav 文件
 * @param       fname : 文件路径+文件名
 * @retval      KEY0_PRES,错误
 *              KEY1_PRES,打开文件失败
 *              其他,非 WAV 文件
 */
uint8_t wav_play_song(uint8_t *fname)
{
    uint8_t key;
    uint8_t t = 0;
    uint8_t res;
    uint32_t fillnum;

    g_audiodev.file = (FIL*)mymalloc(SRAMIN, sizeof(FIL));
    g_audiodev.saibuf1 = mymalloc(SRAMIN, WAV_SAI_TX_DMA_BUFSIZE);
```

```
    g_audiodev.saibuf2 = mymalloc(SRAMIN, WAV_SAI_TX_DMA_BUFSIZE);
    g_audiodev.tbuf = mymalloc(SRAMIN, WAV_SAI_TX_DMA_BUFSIZE);

    if (g_audiodev.file && g_audiodev.saibuf1 && g_audiodev.saibuf2 &&
                                                        g_audiodev.tbuf)
    {
        res = wav_decode_init(fname, &wavctrl); /* 得到文件的信息 */

        if (res == 0)   /* 解析文件成功 */
        {
            if (wavctrl.bps == 16  )
            {
               es8388_sai_cfg(0, 3);    /* 飞利浦标准,16 位数据长度 */

               sai1_saia_init(SAI_MODEMASTER_TX,
                              SAI_CLOCKSTROBING_RISINGEDGE,SAI_DATASIZE_16);
               sai1_tx_dma_init(g_audiodev.saibuf1, g_audiodev.saibuf2,
                WAV_SAI_TX_DMA_BUFSIZE / 2, 1); /* 配置 TX DMA,16 位 */
            }
            else if (wavctrl.bps == 24)
            {
                es8388_sai_cfg(0, 0);   /* 飞利浦标准,24 位数据长度 */
                sai1_saia_init(SAI_MODEMASTER_TX,
                               SAI_CLOCKSTROBING_RISINGEDGE,SAI_DATASIZE_24);
                /* 配置 TX DMA */
                sai1_tx_dma_init(g_audiodev.saibuf1, g_audiodev.saibuf2,
                                            WAV_SAI_TX_DMA_BUFSIZE / 4, 2);
             }
            sai1_samplerate_set(wavctrl.samplerate);     /* 设置采样率 */
            sai_tx_callback = wav_sai_dma_tx_callback;
            /* 回调函数指 wav_sai_dma_callback */
            audio_stop();

            res = f_open(g_audiodev.file, (TCHAR*)fname, FA_READ); /* 打开文件 */

            if (res == 0)
            {
                f_lseek(g_audiodev.file, wavctrl.datastart);     /* 跳过文件头 */
                fillnum = wav_buffill(g_audiodev.saibuf1,
                                      WAV_SAI_TX_DMA_BUFSIZE,wavctrl.bps);
                fillnum = wav_buffill(g_audiodev.saibuf2,
                                      WAV_SAI_TX_DMA_BUFSIZE, wavctrl.bps);
                audio_start();

                while (res == 0)
                {
                    while(wavtransferend == 0);          /* 等待 wav 传输完成; */
                    wavtransferend = 0;

                    if (fillnum != WAV_SAI_TX_DMA_BUFSIZE)  /* 播放结束? */
                    {
                        res = KEY0_PRES;
                        break;
                    }

                    if (wavwitchbuf)
                    {
                        fillnum = wav_buffill(g_audiodev.saibuf2,
                        WAV_SAI_TX_DMA_BUFSIZE, wavctrl.bps); /* 填充 buf2 */
                    }
                    else
                    {
                        fillnum = wav_buffill(g_audiodev.saibuf1,
                        WAV_SAI_TX_DMA_BUFSIZE, wavctrl.bps); /* 填充 buf1 */
```

```
                    }

                    while (1)
                    {
                        key = key_scan(0);
                        if (key == WKUP_PRES)    /* 暂停 */
                        {
                            if (g_audiodev.status & 0x01)
                            {
                                g_audiodev.status &= ~(1 << 0);
                            }
                            else
                            {
                                g_audiodev.status |= 0x01;
                            }
                        }
                        if(key == KEY2_PRES || key==KEY0_PRES)/* 下一曲/上一曲 */
                        {
                            res = key;
                            break;
                        }

                        wav_get_curtime(g_audiodev.file, &wavctrl);
                        /* 得到总时间和当前播放的时间 */
                        audio_msg_show(wavctrl.totsec, wavctrl.cursec,
                                                        wavctrl.bitrate);

                        t++;
                        if (t == 20)
                        {
                            t = 0 ;
                            LED0_TOGGLE();
                        }

                        if ((g_audiodev.status & 0x01) == 0)
                        {
                            delay_ms(10);
                        }
                        else
                        {
                            break;
                        }
                    }
                }
                audio_stop();
            }
            else
            {
                res = 0xFF;
            }
        }
        else
        {
            res = 0xFF;
        }
    }
    else
    {
        res = 0xFF;
    }
    myfree(SRAMIN, g_audiodev.tbuf);        /* 释放内存 */
    myfree(SRAMIN, g_audiodev.saibuf1);     /* 释放内存 */
    myfree(SRAMIN, g_audiodev.saibuf2);     /* 释放内存 */
    myfree(SRAMIN, g_audiodev.file);        /* 释放内存 */
    return res;
```

```
}
```

以上代码中，wav_decode_init 函数，用来对 wav 音频文件进行解析，得到 wav 的详细信息（音频采样率，位数，数据流起始位置等）；wav_buffill 函数，通过 f_read 读取数据，然后将数据填充到 buf 里面，注意：在读取 24 位音频数据的时候，读出数据需要扩展为 32 位才可填充到 buf；wav_sai_dma_tx_callbac 函数则是 DMA 发送完成的回调函数（sai_tx_callbackja 函数指针指向该函数），这里，我们对数据进行了填 0 处理，而是采用 2 个变量：wavtransferend 和 wavwitchbuf，来告诉 wav_paly_son 函数是否传输完成，以及应该填充哪个数据 buf（saibuf1 或 saibuf2）。

最后，wav_play_song 函数，是播放 WAV 最终执行的函数，该函数解析完 WAV 文件后，设置 ES8388 和 SAI 的参数（采样率，位数等），并开启 DMA，然后不断填充数据，实现 WAV 播放，该函数中还进行了按键检测，实现上下曲切换和暂停/播放等操作。

4. audioplay 代码

这部分我们需要根据 ES8388 推荐的初始化顺序进行配置。我们需要借助 SD 卡和文件系统把我们需要播放的歌曲传给 ES8388 播放。我们在 User 目录下新建一个《APP》文件夹，同时在该目录下新建 audioplay.c 和 audioplay.h 并加入到工程。

首先判断音乐文件类型，符合条件的再把相应的文件数据发送给 ES8388，我们在 FATFS 的扩展文件中已经实现了判断文件类型这个功能，在图片显示实验也演示了这部分代码的使用，我们把这个功能封装成了 audio_get_tnum()函数，这部分参考我们光盘源码即可。接下来我们来分析一下 audio play()和 audio_play_song ()函数，实现播放歌曲的功能，代码如下：

```
/**
 * @brief       播放音乐
 * @param       无
 * @retval      无
 */
void audio_play(void)
{
    uint8_t res;
    DIR wavdir;                 /* 目录 */
    FILINFO *wavfileinfo;       /* 文件信息 */
    uint8_t *pname;             /* 带路径的文件名 */
    uint16_t totwavnum;         /* 音乐文件总数 */
    uint16_t curindex;          /* 当前索引 */
    uint8_t key;                /* 键值 */
    uint32_t temp;
    uint32_t *wavoffsettbl;     /* 音乐 offset 索引表 */

    es8388_adda_cfg(1, 0);      /* 开启 DAC 关闭 ADC */
    es8388_output_cfg(1, 1);    /* DAC 选择通道 1 输出 */

    while (f_opendir(&wavdir, "0:/MUSIC"))      /* 打开音乐文件夹 */
    {
        text_show_string(30, 190, 240, 16, "MUSIC 文件夹错误!", 16, 0, BLUE);
        delay_ms(200);
        lcd_fill(30, 190, 240, 206, WHITE);     /* 清除显示 */
        delay_ms(200);
    }

    totwavnum = audio_get_tnum("0:/MUSIC");     /* 得到总有效文件数 */
    while (totwavnum == NULL)                   /* 音乐文件总数为 0 */
    {
        text_show_string(30, 190, 240, 16, "没有音乐文件!", 16, 0, BLUE);
        delay_ms(200);
        lcd_fill(30, 190, 240, 146, WHITE);     /* 清除显示 */
        delay_ms(200);
    }
```

```
    wavfileinfo = (FILINFO*)mymalloc(SRAMIN, sizeof(FILINFO));/* 申请内存 */
    pname = mymalloc(SRAMIN, FF_MAX_LFN * 2 + 1);     /* 为带路径的文件名分配内存 */
    /* 申请 4*totwavnum 个字节的内存,用于存放音乐文件 off block 索引 */
    wavoffsettbl = mymalloc(SRAMIN, 4 * totwavnum);
    while (!wavfileinfo || !pname || !wavoffsettbl)           /* 内存分配出错 */
    {
        text_show_string(30, 190, 240, 16, "内存分配失败!", 16, 0, BLUE);
        delay_ms(200);
        lcd_fill(30, 190, 240, 146, WHITE);    /* 清除显示 */
        delay_ms(200);
    }

    /* 记录索引 */
    res = f_opendir(&wavdir, "0:/MUSIC");  /* 打开目录 */
    if (res == FR_OK)
    {
        curindex = 0;               /* 当前索引为 0 */
        while (1)                   /* 全部查询一遍 */
        {
            temp = wavdir.dptr;     /* 记录当前 index */

            res = f_readdir(&wavdir, wavfileinfo);     /* 读取目录下的一个文件 */
            if ((res != FR_OK) || (wavfileinfo->fname[0] == 0))
            {
                break;      /* 错误了/到末尾了,退出 */
            }

            res = exfuns_file_type(wavfileinfo->fname);
            if ((res & 0xF0) == 0x40)          /* 取高四位,看看是不是音乐文件 */
            {
                wavoffsettbl[curindex] = temp;/* 记录索引 */
                curindex++;
            }
        }
    }
    curindex = 0;           /* 从 0 开始显示 */
    res = f_opendir(&wavdir, (const TCHAR*)"0:/MUSIC");/* 打开目录 */
    while (res == FR_OK)    /* 打开成功 */
    {
        dir_sdi(&wavdir, wavoffsettbl[curindex]);       /* 改变当前目录索引 */
        res = f_readdir(&wavdir, wavfileinfo);          /* 读取目录下的一个文件 */
        if ((res != FR_OK) || (wavfileinfo->fname[0] == 0))
        {
            break;          /* 错误了/到末尾了,退出 */
        }
        strcpy((char*)pname, "0:/MUSIC/");                    /* 复制路径(目录) */
        strcat((char*)pname,(const char*)wavfileinfo->fname);/* 将文件名接在后面*/
        lcd_fill(30, 190, lcddev.width - 1, 190 + 16, WHITE);/* 清除之前的显示 */
        text_show_string(30, 190, lcddev.width - 60, 16,
                         (char*)wavfileinfo->fname,16,0,BLUE);/* 显示歌曲名字*/
        audio_index_show(curindex + 1, totwavnum);
        key = audio_play_song(pname);     /* 播放这个音频文件 */
        if (key == KEY2_PRES)             /* 上一曲 */
        {
            if (curindex)
            {
                curindex--;
            }
            else
            {
                curindex = totwavnum - 1;
            }
```

```
        }
        else if (key == KEY0_PRES)  /* 下一曲 */
        {
            curindex++;
            if (curindex >= totwavnum)
            {
                curindex = 0;       /* 到末尾的时候,自动从头开始 */
            }
        }
        else
        {
            break;  /* 产生了错误 */
        }
    }
    myfree(SRAMIN, wavfileinfo);    /* 释放内存 */
    myfree(SRAMIN, pname);          /* 释放内存 */
    myfree(SRAMIN, wavoffsettbl);   /* 释放内存 */
}
/**
 * @brief       播放某个音频文件
 * @param       fname : 文件名
 * @retval      按键值
 *   @arg       KEY0_PRES , 下一曲.
 *   @arg       KEY2_PRES , 上一曲.
 *   @arg       其他 , 错误
 */
uint8_t audio_play_song(uint8_t *fname)
{
    uint8_t res;

    res = exfuns_file_type((char *)fname);
    switch (res)
    {
        case T_WAV:
            res = wav_play_song(fname);
            break;

        default:            /* 其他文件,自动跳转到下一曲 */
            printf("can't play:%s\r\n", fname);
            res = KEY0_PRES;
            break;
    }
    return res;
}
```

这里，audio_play 函数在 main 函数里面被调用，该函数首先设置 ES8388 相关配置，然后查找 SD 卡里面的 MUSIC 文件夹，并统计该文件夹里面总共有多少音频文件（统计包括：WAV/MP3/APE/FLAC 等），然后，该函数调用 audio_play_song 函数，按顺序播放这些音频文件。

在 audio_play_song 函数里面，通过判断文件类型，调用不同的解码函数，本章，只支持 WAV 文件，通过 wav_play_song 函数实现 WAV 解码。

4. main.c 代码

解决了音乐播放的问题，main.c 函数实现起来就比较简单了，我们可以按照流程图的设计思路进行编写即可：

```
int main(void)
{
    sys_cache_enable();                         /* 打开 L1-Cache */
    HAL_Init();                                 /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);         /* 设置时钟,400Mhz */
    delay_init(400);                            /* 延时初始化 */
```

```
    usart_init(115200);                             /* 串口初始化为 115200 */
    usmart_init(200);                               /* 初始化 USMART */
    mpu_memory_protection();                        /* 保护相关存储区域 */
    led_init();                                     /* 初始化 LED */
    key_init();                                     /* 初始化按键 */
    sdram_init();                                   /* 初始化 SDRAM */
    lcd_init();                                     /* 初始化 LCD */
    norflash_init();                                /* 初始化 NORFLASH */

    es8388_init();                                  /* ES8388 初始化 */
    es8388_adda_cfg(1, 0);                          /* 开启 DAC 关闭 ADC */
    es8388_output_cfg(1, 1);                        /* DAC 选择通道输出 */
    es8388_hpvol_set(25);                           /* 设置耳机音量 */
    es8388_spkvol_set(25);                          /* 设置喇叭音量 */

    my_mem_init(SRAMIN);                            /* 初始化内部内存池 */
    my_mem_init(SRAMEX);                            /* 初始化外部内存池 */
    my_mem_init(SRAMDTCM);                          /* 初始化 DTCM 内存池 */

    while (sd_init())                               /* 检测 SD 卡 */
    {
        lcd_show_string(30, 50, 200, 16, 16, "SD Card Failed!", RED);
        delay_ms(200);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        delay_ms(200);
    }

    exfuns_init();                                  /* 为 fatfs 相关变量申请内存 */
    f_mount(fs[0], "0:", 1);                        /* 挂载 SD 卡 */
    f_mount(fs[1], "1:", 1);                        /* 挂载 SPI FLASH */
    f_mount(fs[2], "2:", 1);                        /* 挂载 NAND FLASH */

    while (fonts_init())                            /* 检查字库 */
    {
        lcd_show_string(30, 50, 200, 16, 16, "Font Error!", RED);
        delay_ms(200);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        delay_ms(200);
    }

    text_show_string(30, 50, 200, 16, "正点原子 STM32 开发板",16,0, RED);
    text_show_string(30, 70, 200, 16, "音乐播放器 实验", 16, 0, RED);
    text_show_string(30, 90, 200, 16, "正点原子@ALIENTEK", 16, 0, RED);
    text_show_string(30, 110, 200, 16, "KEY0:NEXT   KEY2:PREV", 16, 0, RED);
    text_show_string(30, 130, 200, 16, "KEY_UP:PAUSE/PLAY", 16, 0, RED);

    while (1)
    {
        audio_play();           /* 播放音乐 */
    }
}
```

到这里本实验的代码基本就编写完成了，我们准备好音乐文件放到 SD 卡根目录下的《MUSIC》文件夹下测试本实验的代码，可以把 ES8388 的配置函数加到 USMART 下，这样就可以通过 USMART 来测试和调试 ES8388 了。

60.4 下载验证

在代码编译成功之后，我们下载代码到开发板上，程序先执行字库检测，然后当检测到SD卡根目录的MUSIC文件夹有音频文件（WAV格式音频）的时候，就开始自动播放歌曲了，如图 60.4.1 所示：

图 60.4.1 音乐播放中

从上图可以看出，总共 1 首歌曲，当前正在播放第 1 首歌曲，歌曲名、播放时间、总时长、码率、音量等信息等也都有显示。此时 DS0 会随着音乐的播放而闪烁。

此时我们便可以听到开发板板载喇叭播放出来的音乐了，也可以在开发板的 PHONE 端子插入耳机来听歌。同时，我们可以通过按 KEY0 和 KEY2 来切换下一曲和上一曲，通过 KEY_UP 暂停和继续播放。

至此，我们就完成了一个简单的音乐播放器了，虽然只支持 WAV 文件，但是大家可以在此基础上，增加其他音频格式解码器（可参考综合实验），便可实现其他音频格式解码了。

第六十一章 录音机实验

上一章，我们实现了一个简单的音乐播放器，本章我们将在上一章的基础上，继续用 ES8388 实现一个简单的录音机，录制 WAV 格式的录音。

61.1 SAI 录音简介
61.2 硬件设计
61.3 软件设计
61.4 下载验证

61.1 SAI 录音简介

本章涉及的知识点基本上在上一章都有介绍。本章要实现 WAV 录音，还是和上一章一样，要了解：WAV 文件格式、ES8388 和 SAI 接口。WAV 文件格式，我们在上一章已经做了详细介绍了，这里就不作介绍了。

正点原子阿波罗 STM32H743 开发板将板载的一个 MIC 分别接入到了 ES8388 的 2 个差分输入通道（LIP/LIN 和 RIP/RIN，原理图见：图 60.2.1）。代码上，我们采用立体声 WAV 录音，不过，左右声道的音源都是一样的，录音出来的 WAV 文件，听起来就是个单声道效果。

关于 ES8388 的录音设置（即 ADC 的相关设置），我们在上一章的介绍已经顺带都介绍了，所以本章我们就不再介绍 ES8388 的寄存器了，读者想了解可以参考第 60 章的介绍，或者参考本例程源代码和 ES8388 的 pdf 数据手册理解。

上一章我们向大家介绍了 STM32H743 的 SAI 放音，通过上一章的了解，我们知道：STM32H743 SAI 的全双工通信，需要用到 SAI 的两个子模块（SAI_A 和 SAI_B），一个工作在主模式，产生 FS、SCK 和 MCLK，一个工作在从模式，通过 SD 引脚接收数据。

本章我们必须向 ES8388 提供 WS(FS)，CK(SCK)和 MCK(MCLK)等时钟，同时又要录音，所以只能使用全双工模式。工作在主模式的 SAI 子模块循环发送数据 0X0000，给 ES8388，以产生 CK、WS 和 MCK 等信号，工作在从模式的 SAI 子模块，则接收来自 ES8388 的 ADC 数据（ADCDAT），并保存到 SD 卡，实现录音。

本章我们将同时使用 SAI 的两个子模块，以实现录音功能，SAI 的相关寄存器，我们在上一章已经介绍的差不多了，这里就不再进行寄存器介绍，大家可以参考《STM32H7xx 参考手册_V3（中文版）.pdf》第 51.5 小节。

最后，我们看看要通过 STM32H743 的 SAI，驱动 ES8388 实现 WAV 录音的简要步骤，如下：

1）初始化 ES8388

这个过程就是前面所讲的 ES8388 MIC 录音配置步骤，让 ES8388 的 ADC 以及其模拟部分工作起来。

2）初始化 SAI_A 和 SAI_B

本章要用到 SAI 的全双工模式，所以，SAI_A 和 SAI_B 都需要配置，其中 SAI_A 配置为主模式，SAI_B 设置为从模式，且与 SAI_A 同步。他们的其他配置（协议、时钟电平特性、slot 相关参数）基本一样，只是一个是发送一个是接收，且都要使能 DMA。同时，还需要设置音频采样率，不过这个只需要设置 SAI_A 的即可，还是通过上一章介绍的查表法设置。

3）设置发送和接收 DMA

放音和录音都是采用 DMA 传输数据的，本章放音其实就是个幌子，不过也得设置 DMA（使用 DMA2 数据流 3 的通道 0），配置同上一章一模一样，不过不需要开启 DMA 传输完成中断。对于录音，则使用的是 DMA2 数据流 5 的通道 0 实现的 DMA 数据接收，我们需要配置 DMA2 的数据流 5，本章将 DMA2 数据流 3 设置为：双缓冲循环模式，外设和存储器都是 16 位宽，并开启传输完成中断（方便接收数据）。

4）编写接收通道 DMA 传输完成中断服务函数

为了方便接收音频数据，我们使用 DMA 传输完成中断，每当一个缓冲接数据满了，硬件

自动切换为下一个缓冲，同时进入中断服务函数，将已满缓冲的数据写入 SD 卡的 wav 文件。过程如图 61.1.1 所示：

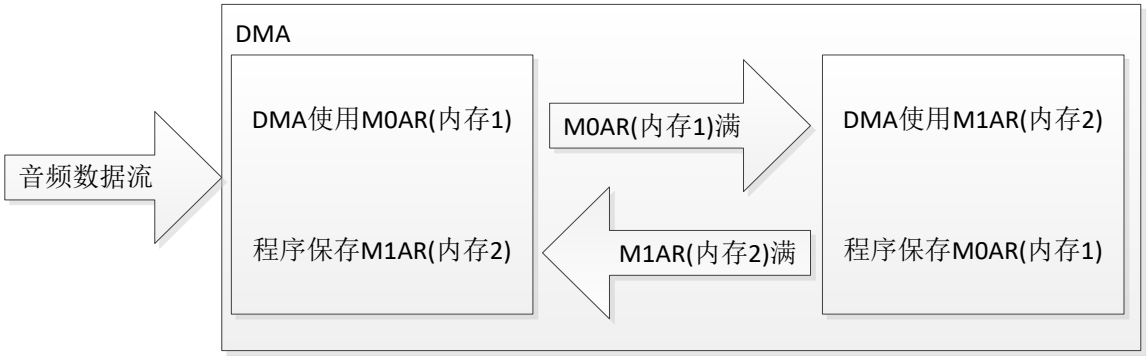


图 61.1.1 DMA 双缓冲接收音频数据流框图

5）创建 WAV 文件，并保存 wav 头

前面 4 步完成，其实就可以开始读取音频数据了，不过在录音之前，我们需要先在创建一个新的文件，并写入 wav 头，然后才能开始写入我们读取到的的 PCM 音频数据。

6）开启 DMA 传输，接收数据

然后，我们就只需要开启 DMA 传输，然后及时将 SAI_B 读到的数据写入到 SD 卡之前新建的 wav 文件里面，就可以实现录音了。

7）计算整个文件大小，重新保存 wav 头并关闭文件

在结束录音的时候，我们必须知道本次录音的大小（数据大小和整个文件大小），然后更新 wav 头，重新写入文件，最后因为 FATFS，在文件创建之后，必须调用 f_close，文件才会真正体现在文件系统里面，否则是不会写入的！所以最后还需要调用 f_close，以保存文件。

61.2 硬件设计

1. 例程功能

本章实验功能简介：开机后，先初始化各外设，然后检测字库是否存在，如果检测无问题，再检测 SD 卡根目录是否存在 RECORDER 文件夹，如果不存在则创建，如果创建失败，则报错。在找到 SD 卡的 RECORDER 文件夹后，即进入录音模式（包括配置 ES8388 和 SAI 等），此时可以在耳机（或喇叭）听到采集到的音频。KEY0 用于开始/暂停录音，KEY2 用于保存并停止录音，KEY_UP 用于播放最近一次的录音。

当我们按下 KEY0 的时候，可以在屏幕上看到录音文件的名字、码率以及录音时间等，然后通过 KEY2 可以保存该文件，同时停止录音（文件名和时间也都将清零），在完成一段录音后，我们可以通过按 KEY_UP 按键，来试听刚刚的录音。LED0 用于提示程序正在运行，LED1 用于提示是否处于暂停录音状态。

2. 硬件资源

本实验，大家需要准备 1 个 microSD/SD 卡（在里面新建一个 MUSIC 文件夹，并存放一些歌曲在 MUSIC 文件夹下）和一个耳机（非必备），分别插入 SD 卡接口和耳机接口，然后下载本实验就可以实现录音机的效果。实验用到的硬件资源如下：

1）LED 灯

LED0 – PB1
LED1 – PB0

2）独立按键

KEY0 - PH3
KEY1 - PH2
KEY2 - PC13
KEY_UP - PA0 (程序中的宏名:WK_UP)

3）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)
5）SD 卡：通过 SDMMC 连接
6）NOR FLASH(SPI FLASH 芯片,连接在 SPI5 上)
7）SAI，驱动 ES8388 芯片
8）开发板板载的咪头或自备麦克风输入
9）喇叭或耳机

录音机实验与上一章(音乐播放器实验)用到的硬件资源基本一样，我们这里就不重复介绍了，有差异的是这次我们用到板载的咪头用于信号输入，也可以通过 3.5mm 的音频接口通过 LINE_IN 接入麦克风输入录音音源。

3. 原理图

本实验相关的原理图同上一章节。

61.3 程序设计

61.3.1 程序流程图

程序的设计流程如下：

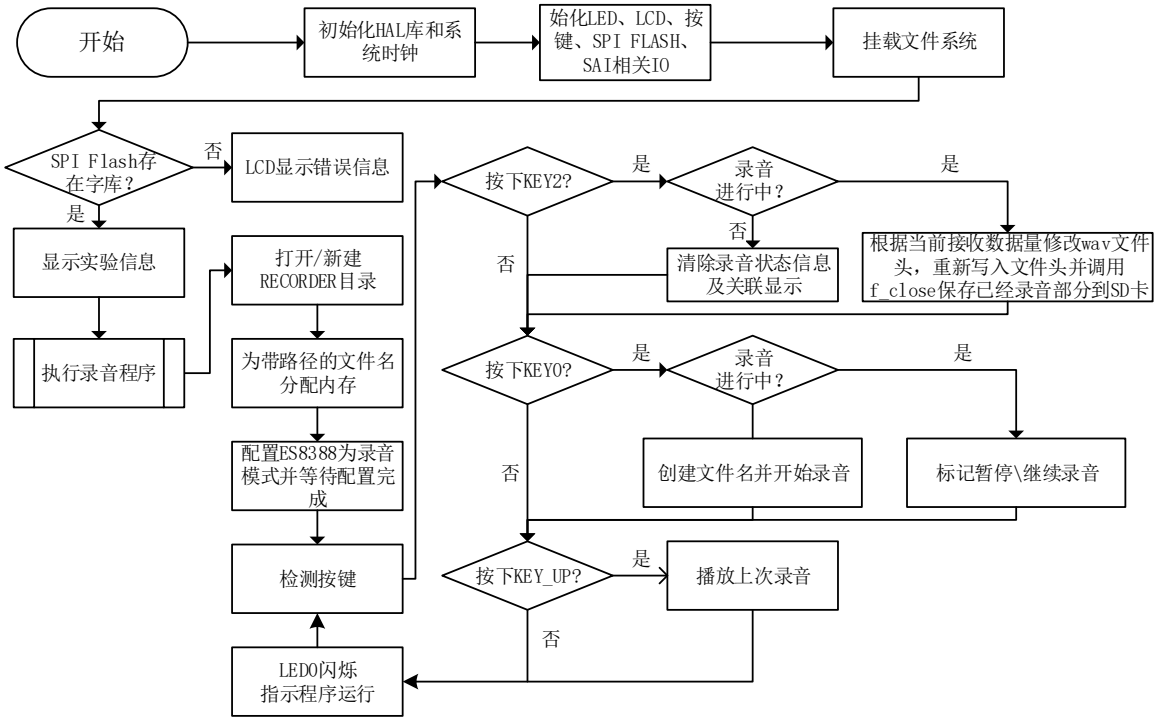


图 61.3.1.1 录音机实验程序流程图

我们通过板载的按键控制录音的开始和停止，检测到录音开始后在录音目录下随机生成一个 wav 后缀的文件名并写入文件头信息，通过 ES8388 的录音模式不断采集声音信息并定入文件，录音结束后，我们保存文件并修改对应的文件头信息以便文件能被解码。最后，我们设计了用 KEY_UP 按键来播放上一次录音的文件，以便查看录音效果。

61.3.2 程序解析

1. recorder 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码，RECORDER 的驱动主要包括两个文件：recorder.c 和 recorder.h。

音乐播放器实验中我们已经学过配置 ES8388 的方法，我们在 recoder.c 编写函数配置 ES8388 工作在 PCM 录音模式，我们编写代码如下：

```
/**
 * @brief       进入 PCM 录音模式
 * @param       无
 * @retval      无
 */
void recoder_enter_rec_mode(void)
{
    SAI1_TX_DMASx->CR &= ~(1 << 4); /* 关闭传输完成中断(这里不用中断送数据) （如果在这
                                       里不关闭 dma 就会卡在清空数据过程）*/
    es8388_adda_cfg(0, 1);          /* 开启 ADC */
    es8388_input_cfg(0);            /* 开启输入通道(通道 1,MIC 所在通道) */
    es8388_mic_gain(8);             /* MIC 增益设置为最大 */
    es8388_alc_ctrl(3, 4, 4);       /* 开启立体声 ALC 控制,以提高录音音量 */
    es8388_output_cfg(0, 0);        /* 关闭通道 1 和 2 的输出 */
    es8388_spkvol_set(0);           /* 关闭喇叭. */
    es8388_sai_cfg(0, 3);           /* 飞利浦标准,16 位数据长度 */

    /* SAI1 Block A,主发送,16 位数据 */
    sai1_saia_init(SAI_MODEMASTER_TX,SAI_CLOCKSTROBING_RISINGEDGE,
                   SAI_DATASIZE_16);
    sai1_saib_init(SAI_MODESLAVE_RX, SAI_CLOCKSTROBING_RISINGEDGE,
                   SAI_DATASIZE_16);   /* SAI1 Block B 从模式接收,16 位 */
    sai1_samplerate_set(REC_SAMPLERATE); /* 设置采样率 */
    /* 配置 TX DMA,16 位 */
    sai1_tx_dma_init((uint8_t*)&SAI_PLAY_BUF[0],
                     (uint8_t *)&SAI_PLAY_BUF[1],1,1);
    /* 关闭传输完成中断(这里不用中断送数据) */
    SAI1_TX_DMASx->CR &= ~(1 << 4);
    sai1_rx_dma_init(p_sai_recbuf1,p_sai_recbuf2,REC_SAI_RX_DMA_BUF_SIZE/2,1);
    /* 配置 RX DMA */
    sai_rx_callback=recoder_sai_dma_rx_callback;
    /* 初始化回调函数指 sai_rx_callback */

    sai1_play_start();              /* 开始 SAI 数据发送(主机) */
    sai1_rec_start();               /* 开始 SAI 数据接收(从机) */

    recoder_remindmsg_show(0);
}
```

该函数就是用我们前面介绍的方法，激活 ES8388 的 PCM 模式，本章，我们使用的是 44.1Khz 采样率，16 位单声道线性 PCM 模式，以及发送与接收 DMA 配置。

由于最后要把录音写入到文件，这里需要准备 wav 的文件头，为方便，我们定义了一个 __WaveHeader 结构体来定义文件头的数据字节，这个结构体包含了前面提到的 wav 文件的数据结构块：

```
typedef __PACKED_STRUCT
{
    ChunkRIFF riff;                 /* riff 块 */
    ChunkFMT fmt;                   /* fmt 块 */
    //ChunkFACT fact;               /* fact 块 线性 PCM,没有这个结构体 */
    ChunkDATA data;                 /* data 块 */
} __WaveHeader;
```

我们定义一个 recoder_wav_init()函数方便初始化文件信息，代码如下：

```
void recoder_wav_init(__WaveHeader *wavhead)
{
    wavhead->riff.ChunkID = 0x46464952;    /* RIFF" */
    wavhead->riff.ChunkSize = 0;           /* 还未确定,最后需要计算 */
    wavhead->riff.Format = 0x45564157;     /* "WAVE" */
    wavhead->fmt.ChunkID = 0x20746D66;     /* "fmt " */
    wavhead->fmt.ChunkSize = 16;           /* 大小为 16 个字节 */
```

```
    wavhead->fmt.AudioFormat = 0x01;        /* 0x01,表示 PCM; 0x00,表示 IMA ADPCM */
    wavhead->fmt.NumOfChannels = 2;         /* 双声道 */
    wavhead->fmt.SampleRate = REC_SAMPLERATE;/* 采样速率 */
    wavhead->fmt.ByteRate = wavhead->fmt.SampleRate * 4;
    /* 字节速率=采样率*通道数*(ADC 位数/8) */
    wavhead->fmt.BlockAlign = 4;            /* 块大小=通道数*(ADC 位数/8) */
    wavhead->fmt.BitsPerSample = 16;        /* 16 位 PCM */
    wavhead->data.ChunkID = 0x61746164;     /* "data" */
    wavhead->data.ChunkSize = 0;            /* 数据大小,还需要计算 */
}
```

录音完成我们还要重新计算录音文件的大小写入文件头，以保证音频文件能正常被解析。我们把这些数据直接按顺序写入文件即可完成录音操作，结合文件操作和按键功能定义，我们用 wav_recoder()函数实现录音过程，代码如下：

```
/**
 * @brief       WAV 录音
 * @param       无
 * @retval      无
 */
void wav_recorder(void)
{
    uint8_t res, i;
    uint8_t key;
    uint8_t rval = 0;
    uint32_t bw;

    __WaveHeader *wavhead = 0;
    DIR recdir;                                 /* 目录 */
    FIL *f_rec = 0;                             /* 录音文件 */

    uint8_t *pdatabuf;                          /* 数据缓存指针 */
    uint8_t *pname = 0;
    uint8_t timecnt = 0;                        /* 计时器 */
    uint32_t recsec = 0;                        /* 录音时间 */

    while (f_opendir(&recdir, "0:/RECORDER"))   /* 打开录音文件夹 */
    {
        lcd_show_string(30, 230, 240, 16, 16, "RECORDER 文件夹错误!", RED);
        delay_ms(200);
        lcd_fill(30, 230, 240, 246, WHITE);     /* 清除显示 */
        delay_ms(200);
        f_mkdir("0:/RECORDER");                 /* 创建该目录 */
    }
    /* 申请内存 */
    for (i = 0; i < REC_SAI_RX_FIFO_SIZE; i++)
    {
        p_sai_recfifo_buf[i] = mymalloc(SRAMIN, REC_SAI_RX_DMA_BUF_SIZE);
        /* SAI 录音 FIFO 内存申请 */

        if (p_sai_recfifo_buf[i] == NULL)
        {
            break;  /* 申请失败 */
        }
    }
    p_sai_recbuf1=mymalloc(SRAMIN,REC_SAI_RX_DMA_BUF_SIZE);/*SAI 录音内存 1 申请 */
    p_sai_recbuf2=mymalloc(SRAMIN,REC_SAI_RX_DMA_BUF_SIZE);/*SAI 录音内存 2 申请 */
    f_rec = (FIL *)mymalloc(SRAMIN, sizeof(FIL));   /* 开辟 FIL 字节的内存区域 */
    wavhead = (__WaveHeader *)mymalloc(SRAMIN, sizeof(__WaveHeader));
    /* 开辟__WaveHeader 字节的内存区域 */
    pname = mymalloc(SRAMIN, 30);
    /*申请 30 个字节内存,文件名类似"0:RECORDER/REC00001.wav"*/
```

```
    if (!p_sai_recbuf2 || !f_rec || !wavhead || !pname)rval = 1;

    if (rval == 0)
    {
        recoder_enter_rec_mode();      /* 进入录音模式 */
        pname[0] = 0;                  /* pname 没有任何文件名 */

        while (rval == 0)
        {
            key = key_scan(0);

            switch (key)
            {
                case KEY2_PRES:       /* STOP&SAVE */
                    if (g_rec_sta & 0x80)    /* 有录音 */
                    {
                        g_rec_sta = 0;       /* 关闭录音 */
                        wavhead -> riff.ChunkSize = g_wav_size + 36;
                        /* 整个文件的大小-8; */
                        wavhead->data.ChunkSize = g_wav_size;  /* 数据大小 */
                        f_lseek(f_rec, 0);                    /* 偏移到文件头. */
                        f_write(f_rec, (const void *)wavhead,
                                sizeof(__WaveHeader), &bw); /* 写入头数据 */
                        f_close(f_rec);
                        g_wav_size = 0;
                    }

                    g_rec_sta = 0;
                    recsec = 0;
                    LED1(1);     /* 关闭 DS1 */
                    lcd_fill(30, 190, lcddev.width, lcddev.height, WHITE);
                    /* 清除显示,清除之前显示的录音文件名 */
                    break;

                case KEY0_PRES: /* REC/PAUSE */
                    if (g_rec_sta & 0x01)          /* 如果是暂停,继续录音 */
                    {
                        g_rec_sta &= 0xFE;         /* 取消暂停 */
                    }
                    else if (g_rec_sta & 0x80)     /* 已经在录音了,暂停 */
                    {
                        g_rec_sta |= 0x01;         /* 暂停 */
                    }
                    else                           /* 还没开始录音 */
                    {
                      recsec = 0;
                      recoder_new_pathname(pname);     /* 得到新的名字 */
                      text_show_string(30,190,lcddev.width,16,"录制:",16,0,RED);
                      text_show_string(30 + 40, 190, lcddev.width, 16, pname+
                                       11, 16, 0, RED);/* 显示当前录音文件名字 */
                      recoder_wav_init(wavhead);         /* 初始化 wav 数据 */
                      res = f_open(f_rec, (const TCHAR *)pname,
                                                FA_CREATE_ALWAYS | FA_WRITE);

                      if (res)              /* 文件创建失败 */
                      {
                          g_rec_sta = 0;  /* 创建文件失败,不能录音 */
                          rval = 0xFE;    /* 提示是否存在 SD 卡 */
                      }
                      else
                      {
                          res = f_write(f_rec, (const void *)wavhead,
                                    sizeof(__WaveHeader), &bw); /* 写入头数据 */
```

```
                    recoder_msg_show(0, 0);
                    g_rec_sta |= 0x80;      /* 开始录音 */
                }
            }

            if (g_rec_sta & 0x01)
            {
                LED1(0);    /* 提示正在暂停 */
            }
            else
            {
                LED1(1);
            }
            break;

        case WKUP_PRES:             /* 播放最近一段录音 */
            if (g_rec_sta != 0x80)  /* 没有在录音 */
            {
                if (pname[0])       /* 如果按键被按下,且 pname 不为空 */
                {
                    text_show_string(30, 190, lcddev.width, 16,
                                     "播放:", 16, 0, RED);
                    /* 显示当播放的文件名字 */
                    text_show_string(30 + 40, 190, lcddev.width, 16,
                                     (char *)pname + 11, 16, 0, RED);
                    recoder_enter_play_mode();  /* 进入播放模式 */
                    audio_play_song(pname);     /* 播放 pname */
                    /* 清除显示,清除之前显示的录音文件名 */
                    lcd_fill(30, 190, lcddev.width, lcddev.height,
                                                        WHITE);
                    recoder_enter_rec_mode();   /* 重新进入录音模式 */
                }
            }
            break;
    }
    /* 读取一次数据, 读到数据了, 写入文件 */
    if (recoder_sai_fifo_read(&pdatabuf))
    {
        /* 写入文件 */
        res = f_write(f_rec, pdatabuf, REC_SAI_RX_DMA_BUF_SIZE, &bw);

        if (res)
        {
            printf("write error:%d\r\n", res);
        }

        g_wav_size += REC_SAI_RX_DMA_BUF_SIZE;  /* WAV 数据大小增加 */
    }
    else
    {
        delay_ms(5);
    }

    timecnt++;

    if ((timecnt % 20) == 0)
    {
        LED0_TOGGLE();      /* LED0 闪烁 */
    }

    if (recsec != (g_wav_size / wavhead->fmt.ByteRate))/* 录音时间显示 */
    {
        LED1_TOGGLE();                                  /* LED1 闪烁 */
        recsec = g_wav_size / wavhead->fmt.ByteRate;    /* 录音时间 */
```

```
                recoder_msg_show(recsec, wavhead->fmt.SampleRate *
                /* 显示码率 */
                wavhead -> fmt.NumOfChannels * wavhead -> fmt.BitsPerSample);
            }
        }
    }

    for (i = 0; i < REC_SAI_RX_FIFO_SIZE; i++)
    {
        myfree(SRAMIN, p_sai_recfifo_buf[i]);    /* 录音 FIFO 内存释放 */
    }

    myfree(SRAMIN, p_sai_recbuf1);               /* 释放内存 */
    myfree(SRAMIN, p_sai_recbuf2);               /* 释放内存 */
    myfree(SRAMIN, f_rec);                       /* 释放内存 */
    myfree(SRAMIN, wavhead);                     /* 释放内存 */
    myfree(SRAMIN, pname);                       /* 释放内存 */
}
```

2. main.c 代码

由于我们把大部分功能已经在 wav_recoder()中实现了，main 函数进行必要的外设初始化，显示相关的数据信息后，调用该接口即可实现我们需要的录音机功能了，最后我们在 main.c 中实现代码如下：

```
int main(void)
{
    sys_cache_enable();                          /* 打开 L1-Cache */
    HAL_Init();                                  /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);          /* 设置时钟,400Mhz */
    delay_init(400);                             /* 延时初始化 */
    usart_init(115200);                          /* 串口初始化为 115200 */
    usmart_init(200);                            /* 初始化 USMART */
    mpu_memory_protection();                     /* 保护相关存储区域 */
    led_init();                                  /* 初始化 LED */
    key_init();                                  /* 初始化按键 */
    sdram_init();                                /* 初始化 SDRAM */
    lcd_init();                                  /* 初始化 LCD */
    norflash_init();                             /* 初始化 NORFLASH */

    es8388_init();                               /* ES8388 初始化 */
    es8388_hpvol_set(25);                        /* 设置耳机音量 */
    es8388_spkvol_set(25);                       /* 设置喇叭音量 */

    my_mem_init(SRAMIN);                         /* 初始化内部内存池 */
    my_mem_init(SRAMEX);                         /* 初始化外部内存池 */
    my_mem_init(SRAMDTCM);                       /* 初始化 DTCM 内存池 */

    exfuns_init();                               /* 为 fatfs 相关变量申请内存 */
    f_mount(fs[0], "0:", 1);                     /* 挂载 SD 卡 */
    f_mount(fs[1], "1:", 1);                     /* 挂载 SPI FLASH */
    f_mount(fs[2], "2:", 1);                     /* 挂载 NAND FLASH */

    while (sd_init())                            /* 检测 SD 卡 */
    {
        lcd_show_string(30, 50, 200, 16, 16, "SD Card Failed!", RED);
        delay_ms(200);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        delay_ms(200);
    }

    while (fonts_init())                         /* 检查字库 */
```

```
    {
        lcd_show_string(30, 50, 200, 16, 16, "Font Error!", RED);
        delay_ms(200);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        delay_ms(200);
    }

    text_show_string(30, 50, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
    text_show_string(30, 70, 200, 16, "WAV 录音机 实验", 16, 0, RED);
    text_show_string(30, 90, 200, 16, "正点原子@ALIENTEK", 16, 0, RED);

    while (1)
    {
        wav_recorder();          /* 录音 */
    }
}
```

可以看到 main 函数与音乐播放器实验十分类似，封装好了 APP，main 函数会精简很多。

61.4 下载验证

在代码编译成功之后，我们下载代码到正点原子阿波罗 STM32H743 开发板上，先初始化各外设，然后检测字库是否存在，如果检测无问题，再检测 SD 卡根目录是否存在 RECORDER 文件夹，如果不存在则创建，如果创建失败，则报错。在找到 SD 卡的 RECORDER 文件夹后，即进入录音模式（包括配置 ES8388 和 SAI 等），此时可以在耳机（或喇叭）听到采集到的音频。KEY0 用于开始/暂停录音，KEY2 用于保存并停止录音，KEY_UP 用于播放最近一次的录音。

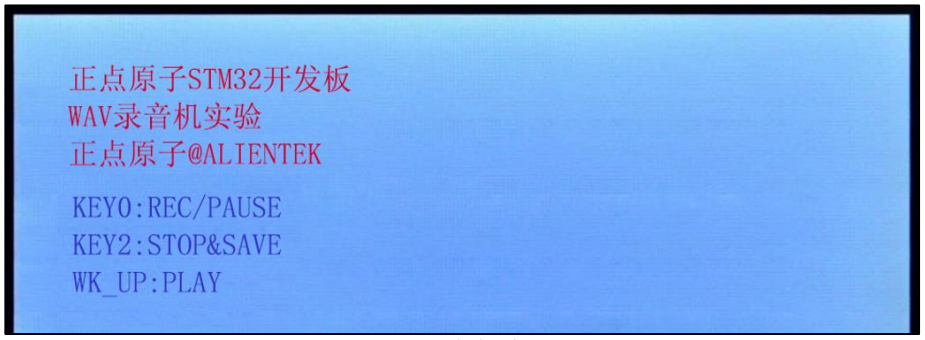


图 61.4.1 录音机实验界面

此时，我们按下 KEY0 就开始录音了，此时看到屏幕显示录音文件的名字以及录音时长，如图 61.4.2 所示：

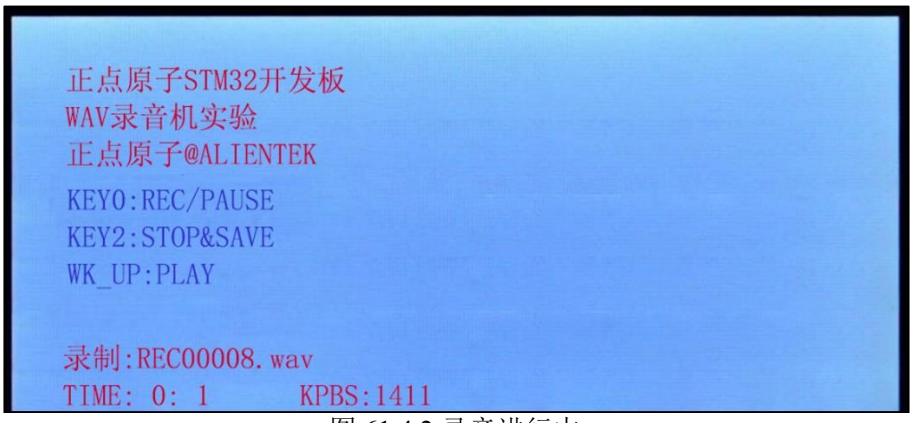


图 61.4.2 录音进行中

在录音的时候按下 KEY0 则执行暂停/继续录音的切换，通过 LED0 指示录音暂停。通过按下 KEY2，可以停止当前录音，并保存录音文件。在完成一次录音文件保存之后，我们可以通过按 KEY_UP 按键，来实现播放这个录音文件（即播放最近一次的录音文件），实现试听。

我们可以把录音完成的 wav 文件放到电脑上，可以通过一些播放软件播放并查看详细的音频编码信息，本例程使用的是 KMPlayer 播放，查看到的信息如图 61.4.3 所示：

图 61.4.3 录音文件属性

这和我们程序设计时的效果一样，通过电脑端的播放器可以直接播放我们所录的音频。经实测，效果还是非常不错的。

第六十二章 SPDIF（光纤音频）实验

前面，我们介绍了 STM32H7 的 SAI 接口，实现了音乐播放器、录音机等功能，本章，我们将介绍 STM32H7 的 SPDIF 接口，结合 SAI 接口和 ES8388 解码器，实现对光纤音频信号的解码。

本章分为如下几个部分：

62.1 SPDIF 简介
62.2 硬件设计
62.3 软件设计
62.4 下载验证

62.1 SPDIF 简介

SPDIF 是 Sony/Philip Digital Interface Format 的缩写，是由索尼和飞利浦公司联合开发的数字音频接口简称，分为 SPDIF 输入（IN）和 SPDIF 输出（OUT）两种，STM32H7 的 SPDIF 接口，仅支持 SPDIF IN，称之为 SPDIF RX。

STM32H7 的 SPDIF 接口的主要特点有：

- 提供多达 4 路输入
- 自动符号率检测
- 最大符号率：12.288MHz
- 支持 8KHz 到 192KHz 的立体声
- 支持 IEC-60958 和 IEC-61937 音频标准，消费类应用
- 奇偶校验位管理
- 使用 DMA 通信进行音频采样
- 支持控制和用户信息 DMA 传输

STM32H7 的 SPDIF RX 接口支持符合 IEC-60958 和 IEC-61937 标准的 SPDIF 数据流。支持高采样率的简单立体声以及压缩的多通道环绕声（Dolby 或 DTS 音频）。SPDIF RX 接口框图如图 62.1.1 所示：

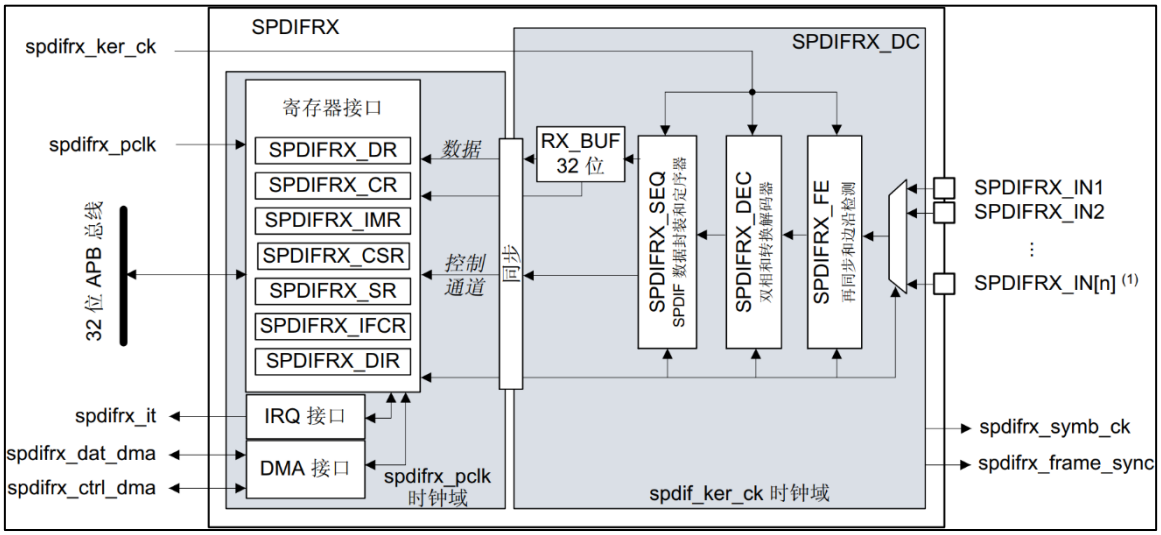


图 62.1.1 SPDIF RX 接口框图

图中 SPDIFX_DC 模块负责解码从 SPDIFX_IN[4:1]输入接收的 SPDIF 数据流。该模块重新采样传入的信号、解码曼彻斯特数据流、并识别帧、子帧和块元素，它传送到寄存器接口、解码数据和相关的状态标志。关于 SPDIFX_DC 模块的详细介绍，请参考《STM32H7xx 参考手册_V3（中文版）.pdf》第 52.3 节。

SPDIF RX 接口通过 APB1 总线完全控制并且能够处理两个 DMA 通道：

1、 专用于传输音频采样的 DMA 通道

2、 专用于传输 IEC-60958 通道状态和用户信息的 DMA 通道

此外，还提供了中断服务，既可用作 DMA 的复用功能，也可用来指示外设的错误或关键状态。接下来，我们简单介绍一下 SPDIF 协议。

1）SPDIF 块

一个 SPDIF 块由 192 个 SPDIF 帧组成，如图 62.1.2 所示：

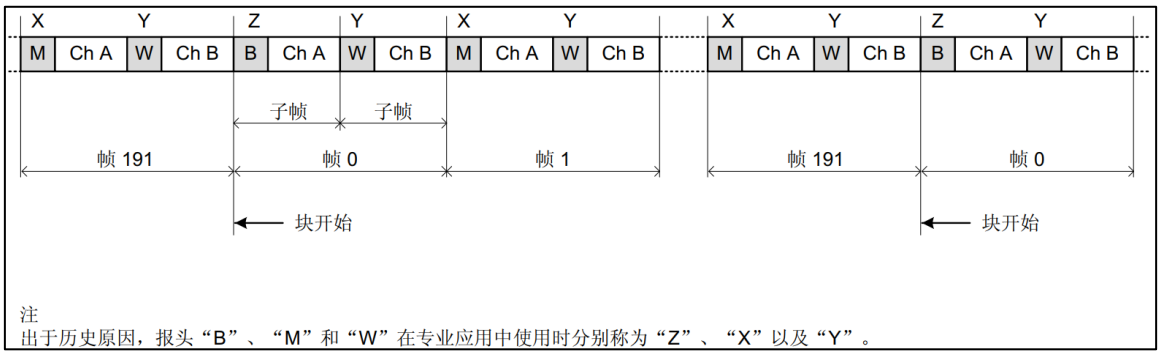


图 62.1.2 SPDIF 块格式

每个 SPDIF 帧又由两个子帧组成，如图 62.1.3 所示：

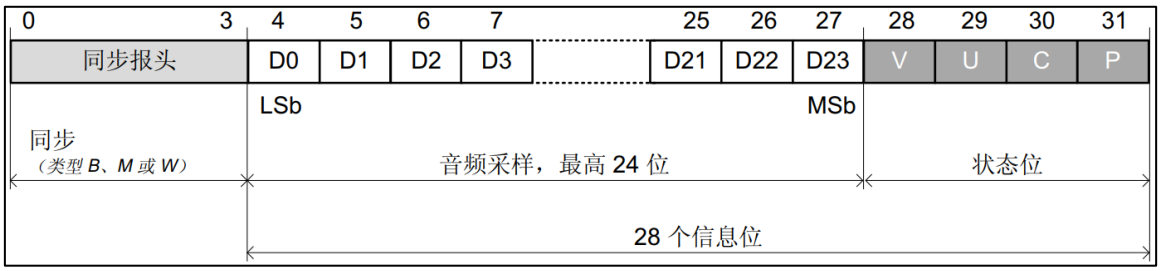


图 62.1.3 SPDIF 子帧格式

每个子帧包含 32 个位，他们的组成如下：

1， 位 0 到 3 包含同步报头之一（B/M/W）；

2， 位 4 到 27 包含以线性 2 的补码表示的音频采样字。最高有效位（MSB）为位 27；

3， 使用 20 位编码范围时，位 8 到 27 包含音频采样字，其中位 8 为 LSB。

4， 位 28（有效性位“V”）表示数据是否有效（例如转换为模拟数据）。

5， 位 29（用户数据位“U”）包含用户数据信息，如光盘的音轨编号。

6， 位 30（通道状态位“C”）包含通道状态信息，如采样率和复制保护。

7， 位 31（奇偶捡验位“P”）包含奇偶校验位，位 4 到 31 将包含偶数个 1 和 0（偶校验）。

对于线性编码音频应用，第一个子帧（立体声操作中的左声道或“A”通道以及单声道操作中的主通道）通常以报头“M”开始。但是，报头每 192 帧切换为报头“B”一次，以识别用于组织通道状态和用户信息的块结构的开始。第二个子帧（立体声操作中的右声道或“B”通道以及单声道操作中的辅助通道）始终以报头“W”开始。

2）同步报头

SPDIF 协议规定，总共有三种同步报头：B、M、W（也可以称为 Z、X、Y）。同步报头总是以和前半个位相反的电平开始的。使能第一个帧的第一个“B”报头的传输前，此前半位值为线路的电平。对于其他报头，此前半位值为之前子帧的奇偶校验位的第二个半位。

SPDIF 三种同步报头的编码方式如图 62.1.4 所示：

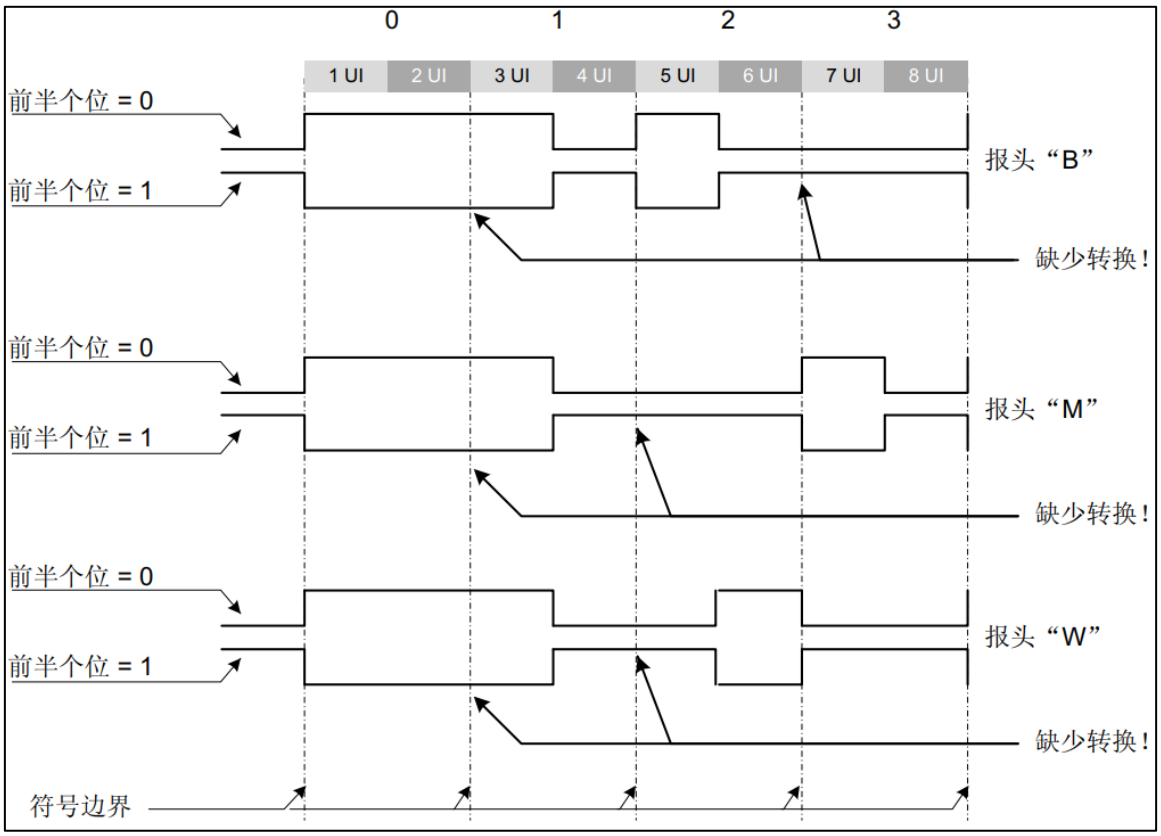


图 62.1.4 SPDIF 同步报头

3）位编码

为最大程度减小传输线路上的直流分量值，并加快时钟从数据流中恢复的速度，从第 4 位开始，到第 31 位，全部采用双相符号编码。

双相符号编码原理：要传输的各个位通过由两个连续二进制状态构成的符号表示。符号的第一个状态始终不同于前一个符号的第二个状态。如果要传输的位为逻辑 0，则符号的第二个状态与第一个状态相同。但如果该位为逻辑 1，则两个状态不同。在 IEC-60958 规范中，这些状态称为“UI”（单位间隔）。

SPDIF 的位编码原理如图 62.1.5 所示：

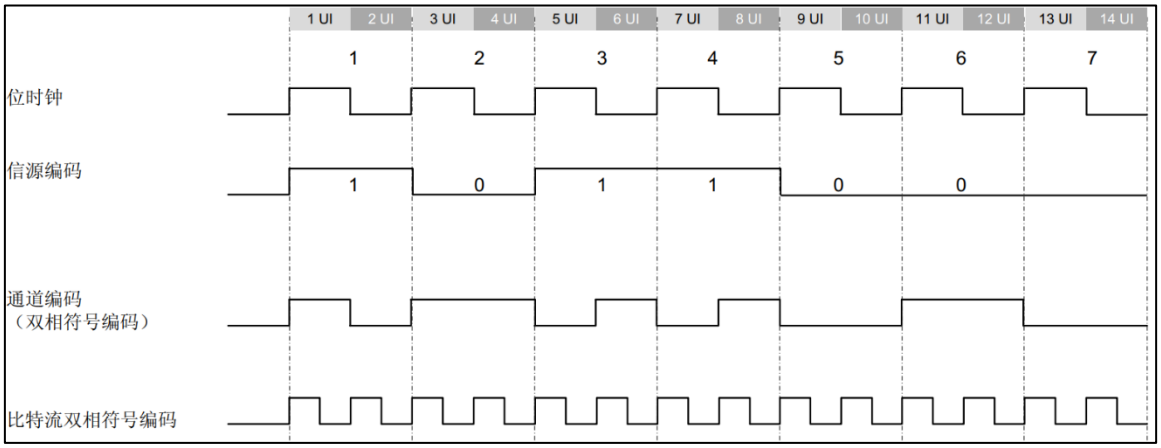


图 62.1.5 通道编码示例

图中比特流实际上就是通道解码时钟，每 2 个时钟解码一个位，在两个时钟内，通道解码状态有变化的（10/01），解码为逻辑 1；在两个解码时钟内，通道解码状态没有变化的（00/11），解码为逻辑 0；

结合通道位编码原理，以及图 62.1.4 所示的同步报头编码方式，我们可以看出，前面 4 个位是不能用这个位编码原理来编码的（因为有 3 个 UI 的状态！！）。

SPDIF 协议我们就介绍到这里，接下来，我们看看 STM32H743 SPDIFRX 的状态流程。如图 62.1.6 所示：

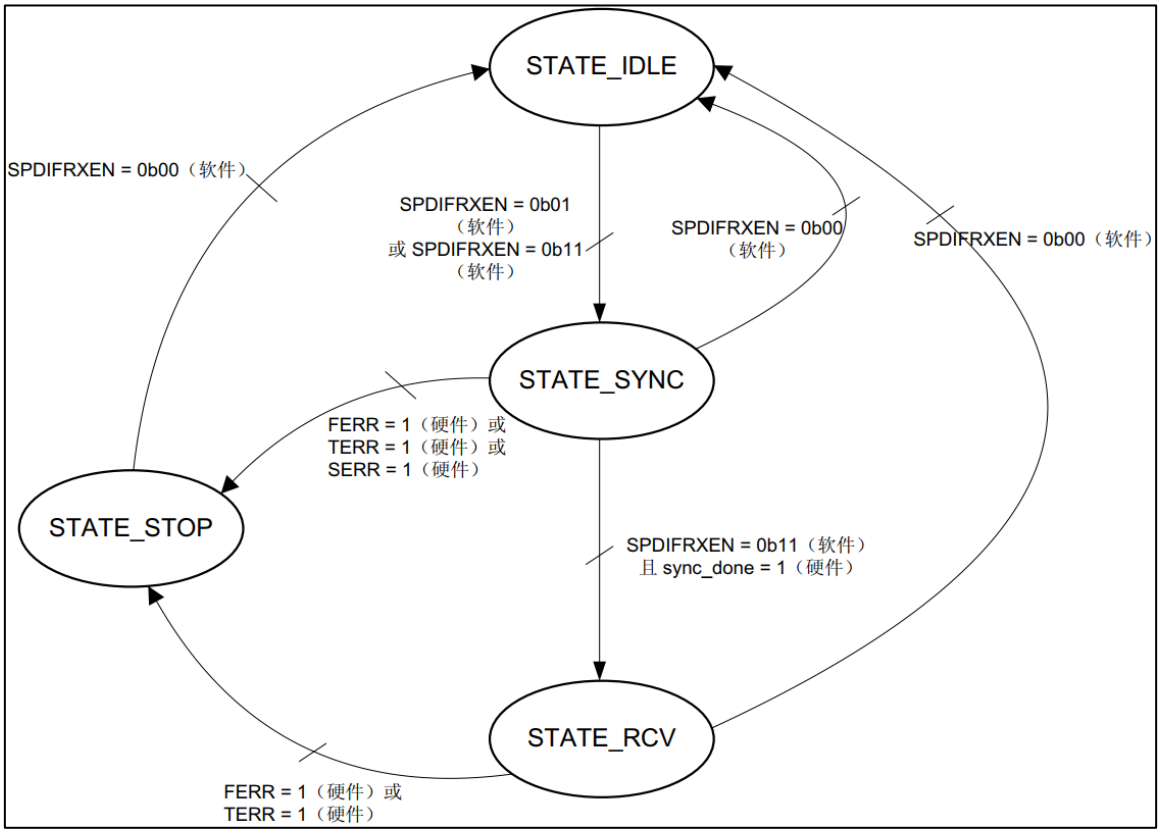


图 62.1.6 STM32H7 SPDIFRX 状态流程

由图可知，STM32H7 的 SPDIFRX 接口总共有四种状态：

1， STATE_IDLE

该状态下，禁止 SPDIF 外设，spdifrx_ker_ck 域复位，spdifrx_pclk 域功能正常。

2， STATE_SYNC

该状态下，SPDIF 将与数据流同步（同步过程请参考《STM32H7xx 参考手册_V3（中文版）》52.3.5 节），阈值定期更新，可通过中断或 DMA 读取用户和通道状态。

3， STATE_RCV

该状态下，SPDIF 将与数据流同步，阈值定期更新，可通过中断或 DMA 通道读取用户、通道状态和音频采样。当检测到“B”报头后，开始保存音频数据。

4， STOP_STATE

该状态下，SPDIF 将不再同步，用户、通道状态和音频数据的接收都将停止，我们可以通过 SPDIF_CR 寄存器的 SPDIFRXEN 字段，控制 SPDIFRX 的状态。

当 SPDIF 处于 STATE_IDLE 时：

通过将 SPDIFRXEN 设置为 01 或 11，将切换到 STATE_SYNC 状态。

当 SPDIFRX 处于 STATE_SYNC 时：

如果同步失败或者接收的数据未正确解码且无法在不进行再同步的情况下恢复（FERR 或 SERR 或 TERR=1），则 SPDIFRX 将进入 STATE_STOP 状态并且等待软件应答。

当同步阶段完成时，如果 SPDIFRXEN=01，将保持该状态。

将 SPDIFRXEN 设置为 0，SPDIFRX 将立刻返回至 STATE_IDLE 状态。

当 SPDIFRXEN=11，且 SYNCD=1，则 SPDIFRX 进入 STATE_RCV 状态。

当 SPDIFRX 处于 STATE_RCV 时：

如果接收的数据未正确解码且无法在不进行再同步的情况下恢复（FERR 或 SERR 或 TERR=1），则 SPDIFRX 将进入 STATE_STOP 状态并且等待软件应答。

将 SPDIFRXEN 设置为 0，SPDIFRX 将立刻返回至 STATE_IDLE 状态。

在此状态下，如果数据接收正确，我们将其通过解码器播放出来，就能实现解码了。

当 SPDIFRX 处于 STATE_STOP 时：

SPDIFRX 停止接收和同步，并等待软件将 SPDIFRXEN 位设置为 0，以清零错误标志。

当 SPDIFRXEN 设置为 0 时，IP 被禁止，这意味着所有状态机被复位，并且 RX_BUF 被刷新。还要注意，标志 FERR、SERR 和 TERR 也会复位。

数据接收

SPDIFRX 为音频采样接收提供了两个 32 位双缓冲区：RX_BUF 和 SPDIF_DR。如果 SPDIF_DR 为空，则包含在 RX_BUF 中的有效数据将立刻传输至 SPDIF_DR。

SPDIFRX 可以使用 DMA 或中断将音频采样传输到存储器中，我们一般使用 DMA 来传输，这样效率比较高。

SPDIFRX 提供多种处理接收数据的方法，用户可以分别处理控制信息流和音频采样流，或将二者一起处理。对于各子帧，数据接收寄存器 SPDIFRX_DR 包含 24 个数据位和可选的 V、U、C、PE 状态位以及 PT，这些可选的位，通过 SPDIFRX_CR 寄存器的 VMSK、CUMSK、PMSK 和 PTMSK 等位控制，我们将在寄存器介绍的时候，给大家详细讲解。

PE 位表示奇偶校验错误位，该位将在解码的子帧中检测到奇偶校验错误时置 1；PT 字段包含报头类型（B、M 或 W）；V、U 和 C 则是从 SPDIF 接口接收的值的直接副本。

通过 SPDIFRX_CR 寄存器的 DRFMT 位，我们可以选择三种音频格式，如图 62.1.7 所示：

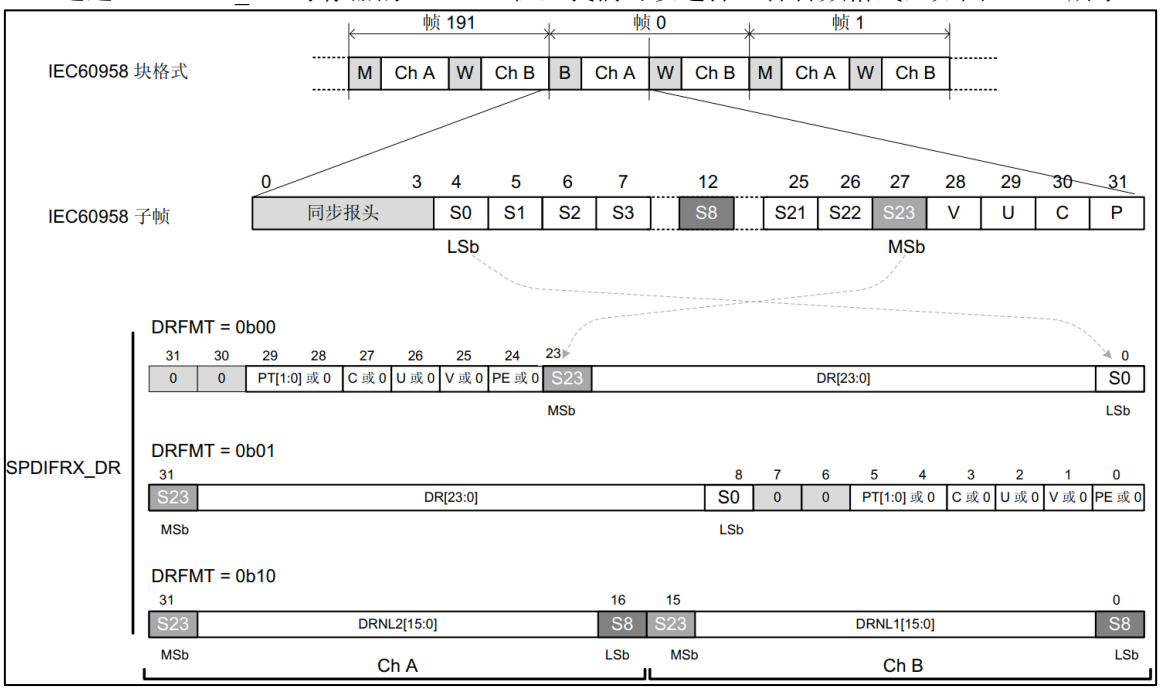


图 62.1.7 SPDIFRX_DR 寄存器格式

将 DRFMT 设置为 00 或 01，可以使数据在 SPDIFRX_DR 寄存器中右对齐或左对齐，此时我们可以根据软件所需的处理方式启用状态信息（V/C/U/PE/PT 等）或将其强制设置为零。

将 DRFMT 设置为 10 得出的格式在非线性模式下相关，因为每个子帧仅使用 16 位。使用此格式，两个连续子帧的数据存储到 SPDIFRX_DR 中，存储器占用量将减半。

本章我们将 DRFMT 设置为 00，使用右对齐格式，且可以支持 24 位音频数据传输。

时钟策略

SPDIFRX 块需要两个不同的时钟：

1， APB1 时钟（spdifrx_pclk），用于寄存器接口，其频率必须至少大于符号率；

2， spdifrx_ker_ck 时钟，主要由 SPDIFRX_DC 部分使用；

为正确解码传入的 SPDIF 数据流，SPDIFRX_DC 应以至少比最大符号率（符号率=采样率*32*2）高 11 倍或者比音频采样率高 704 倍的时钟重新采样接收的数据。

例如，如果用户预期接收到最高 12.288MHz 的符号率（对应音频采样率为 192KHz），则采样率至少为 135.2MHz。

62.1.2.1 SPDIF 寄存器

- **SPDIFRX 控制寄存器（SPDIFRX_CR）**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CKSBKPEN	CKSEN	Res.	INSEL[2:0](1)		
										rw	rw		rw	rw	rw

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	WFA(1)	NBTR[1:0](1)		CHSEL(1)	CBDMAEN(1)	PTMSK(1)	CUMSK(1)	VMSK(1)	PMSK(1)	DRFMT[1:0](1)		RXSTEO(1)	RXDMAEN(1)	SPDIFRXEN[1:0](1)	
	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

图 62.1.2.1 SPDIFRX_CR 寄存器各位描述

该寄存器我们只介绍本章需要用到的一些位（下同）：

INSEL：这三个位，用于选择 SPDIFRX 的输入通道，STM32H7 总共有 4 个输入通道（0~3），我们硬件上连接在通道 1 上面，所以设置 INSEL=001 即可。

NBTR：这两个位用于设置在同步阶段允许的最大重试次数：00，表示不允许重试；01，表示允许重试 3 次；10，表示允许重试 15 次；11，表示允许重试 63 次。本章我们设置为 10，允许重试 15 次。

CHSEL：用于选择通道状态获取路径。我们设置为 0，选择从通道 A 获取通道状态。

CBDMAEN：用于设置通道状态和用户数据是否使能 DMA 接收。我们设置为 0，禁止 DMA 方式接收通道状态和用户数据。

PTMSK：用于设置报头类型屏蔽位。我们设置为 1，禁止将报头数据写入 SPDIFRX_DR。

CUMSK：用于设置通道状态和用户数据屏蔽位。我们设置为 1，禁止将通道状态和用户数据写入 SPDIFRX_DR。

VMSK：用于设置有效性屏蔽位。我们设置为 1，禁止将有效性位写入 SPDIFRX_DR。

PMSK：用于设置奇偶检验屏蔽位。我们设置为 1，禁止将奇偶检验写入 SPDIFRX_DR。

DRFMT：用于设置 SPDIFRX_DR 的数据格式。我们设置为 00，选择右对齐格式（LSB）。

RXSTEO：用于设置是否使能立体声模式。我们设置为 1，使能立体声模式。

RXDMAEN：用于设置是否使能数据流 DMA 接收。我们设置为 1，使能 DMA 接收。

SPDIFRXEN：用于设置 SPDIFRX 的使能。实际上就是控制 SPDIFRX 的工作状态：00，禁止 SPDIFRX；01，使能 SPDIFRX 同步；10，保留；11，使能 SPDIFRX 接收器。关于该寄存器的设置对 SPDIFRX 工作状态的影响，请参考图 62.1.6。

- **SPDIFRX 中断屏蔽寄存器（SPDIFRX_IMR）**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IFEIE	SYNCDIE	SBLKIE	OVRIE	PERRIE	CSRNEIE	RXNEIE
									rw	rw	rw	rw	rw	rw	rw

图 62.1.2.2 SPDIFRX_IMR 寄存器各位描述

IFEIE：串行接口错误中断使能屏蔽位。我们设置为 1，当 SPDIFRX_SR 寄存器中的 SERR=1、TERR=1 或者 FERR=1 时，将产生 SPDIFRX 中断。

PERRIE：奇偶检验错误中断屏蔽位。我们设置为 1，当 SPDIFRX_SR 寄存器中的 PERR=1 时，将产生 SPDIFRX 中断。

- **SPDIFRX 状态寄存器（SPDIFRX_SR）**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	WIDTH5[14:0]														
	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	TERR	SERR	FERR	SYNCD	SBD	OVR	PERR	CSRNE	RXNE
							r	r	r	r	r	r	r	r	r

图 62.1.2.3 SPDIFRX_SR 寄存器各位描述

WIDTH5：使用 spdifrx_ker_ck 计数 5 个符号的持续时间。它表示 5 个连续符号的时间内包含的 spdifrx_ker_ck 时钟周期数。该值可用于估算 SPDIFRX 的音频采样率。其精度受 spdifrx_ker_ck 的频率限制。其估算公式为：

```
Fs = 5 * spdifrx_ker_ck / (WIDTH5 * 64)
```

Fs 为估算的音频采样率。假定 spdifrx_ker_ck 为 84MHz，WIDTH5 为 147。计算可得 Fs 为 44.6KHz，最接近的标准采样率为 44.1KHz，所以，我们基本可以确定采样率为 44.1KHz。

注意，需要在同步完成以后（SYNCD=1），才可以读取 WIDTH5 的值，判断采样率。

TERR：超时错误标志。当该位为 1 时，表示检测到序列错误。

SERR：同步错误标志。当该位为 1 时，表示检测到同步错误。

FERR：帧错误标志。当该位为 1 时，表示检测到曼彻斯特编码错误（帧错误）。

SYNCD：同步完成标志。当该位为 1 时，表示同步完成。

OVR：上溢错误标志。当该位为 1 时，表示检测到上溢错误。

PERR：奇偶校验错误标志。当该位为 1 时，表示检测到奇偶校验错误。

- **SPDIFRX 中断标志清零寄存器（SPDIFRX_IFCR）**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SYNCDCF	SBDCF	OVRCF	PERRCF	Res.	Res.
										w	w	w	w		

图 62.1.2.4 SPDIFRX_IFCR 寄存器各位描述

OVRCF：清零上溢错误标志。向该位写 1，可以清除上溢错误标志。

PERRCF：清零奇偶校验错误标志。向该位写 1，可以清除奇偶校验错误标志。

- **SPDIFRX 数据寄存器（SPDIFRX_DR）**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	PT[1:0]		C	U	V	PE	DR[23:16]							
		r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DR[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

图 62.1.2.5 SPDIFRX_DR 寄存器各位描述

该寄存器有三种数据格式可选（FMT0/FMT1/FMT2，见图 62.1.7），此图为 DRFMT=00 时的格式（FMT0），使用右对齐（LSB）格式的 SPDIFRX_DR 寄存器各位描述。该寄存器里面的 PT/C/U/V/PE 等位，我们都不用，我们只用低 24 位（DR[23:0]），用于读取音频数据。当进入接收模式以后（SPDIFRXEN=11），我们不停的读取 SPDIFRX_DR 的数据，并传送给 SAI 接口，再由 SAI 传输给 ES8388，就可以播放来自 SPDIFRX 接收到的音乐了。不过，我们采用 DMA 来传输，所以直接设置 DMA 的外设地址为 SPDIFRX_DR 即可。

SPDIFRX 的相关寄存器，就给大家介绍到这里，更详细的介绍，请大家参考《STM32H7xx 参考手册_V3（中文版）.pdf》第 52.5 节。

62.2 硬件设计

1. 例程功能

本章实验功能简介：开机后，先初始化各外设，然后检测字库是否存在，如果检测无问题，则初始化 ES8388 的 DAC 工作，并开启 DAC 输出，随后设置 SPDIFRX 的 DMA 和回调函数。然后进入死循环等待，不停的检测 SPDIF 的连接状态；当 SPDIFRX 同步完成，且与 SAI 时钟同步完成后，开启 SPDIFRX 和 SAI 的 DMA 数据传输。此时，在屏幕上会显示当前的音频信号采用率，同时在喇叭/耳机，可以听到音乐。另外，我们可以通过 KEY_UP 和 KEY1 来调节音量，KEY_UP 用于增加音量，KEY1 用于减少音量。

2. 硬件资源

实验用到的硬件资源如下：

1）LED 灯
LED0 – PB1
LED1 – PB0

2）独立按键
KEY0 - PH3
KEY1 - PH2
KEY2 - PC13
KEY_UP - PA0 (程序中的宏名:WK_UP)

3）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)
5）SD 卡：通过 SDMMC 连接
6）NOR FLASH(SPI FLASH 芯片,连接在 SPI5 上)
7）SAI，驱动 ES8388 芯片
8）开发板板载的咪头或自备麦克风输入
9）喇叭或耳机

其中，除了光纤接口，其他资源在之前的学习中都已经介绍过了。

3. 原理图

光纤接口我们使用的是 DLR1150 光纤座，该接口和 STM32H743 的连接，如图 62.2.1 所示：

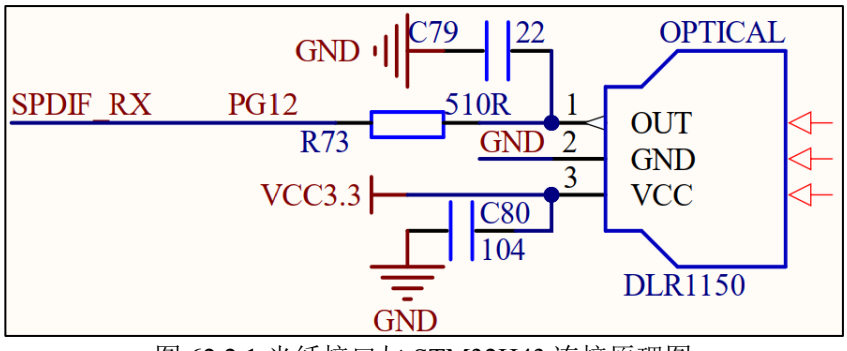


图 62.2.1 光纤接口与 STM32H43 连接原理图

图中 DLR1150 就是我们所使用的光纤座（即光纤接口），它连接在 STM32H743 的 PG12 脚上，是 SPDIFRX 的输入通道 1。

注意：SPDIF_RX 和 NRF_CE 共用 PG12，他们不可以同时使用！

另外，本实验还需要用到一个音频光纤信号发送设备和一条光纤线，这些都需要大家自备。光纤音频发送设备推荐购买正点原子的贝斯（BASE）蓝牙音频接收器，支持光纤输出，购买地址：http://www.openedv.com/thread-86409-1-1.html

62.3 程序设计

我们要通过 STM32H743 的 SPDIFRX 接口接收光纤音频数据，并通过 SAI 驱动 ES8388 播放音乐的简要步骤。HAL 库中 SPDIFRX 相关的库函数定义和声明分布在源文件 stm32h7xx_hal_spdifrx.c 和头文件 stm32h7xx_hal_spdifrx.h 中。

具体配置步骤

1）初始化 ES8388

本实验最终要通过 ES8388 来输出音乐，我们需要先对其进行配置，详细的配置过程，请参考第六十章：

2）初始化 SPDIF

此步需要初始化 SPDIFRX 对应的 IO 口、开启 SPDIFRX 时钟、设置 SPDIFRX 的模式和相关配置，主要通过对 SPDIFRX_CR 寄存器的配置来实现。

HAL 库中初始化 SPDIF 函数为 HAL_SPDIFRX_Init，该函数声明如下：

```
HAL_StatusTypeDef HAL_SPDIFRX_Init(SPDIFRX_HandleTypeDef *hspdif)
```

该函数只有一个 SPDIFRX_HandleTypeDef 结构体类型指针类型入口参数 hspdif，接下来我们看看结构体 SPDIFRX_HandleTypeDef 定义，如下：

```
typedef struct
{
  SPDIFRX_TypeDef             *Instance;
  SPDIFRX_InitTypeDef         Init;
  uint32_t                    *pRxBuffPtr;
  uint32_t                    *pCsBuffPtr;
  __IO uint16_t               RxXferSize;
  __IO uint16_t               RxXferCount;
  __IO uint16_t               CsXferSize;
  __IO uint16_t               CsXferCount;
  DMA_HandleTypeDef           *hdmaCsRx;
  DMA_HandleTypeDef           *hdmaDrRx;
  __IO HAL_LockTypeDef        Lock;
  __IO HAL_SPDIFRX_StateTypeDef  State;
  __IO uint32_t  ErrorCode;
} SPDIFRX_HandleTypeDef;
```

对于 SPDIFRX 初始化，这里我们主要讲解 Init 成员变量，该成员变量用来设置 SPDIFRX 的初始化参数，为 SPDIFRX_InitTypeDef 结构体类型，该结构体定义为：

```
typedef struct
{
  uint32_t InputSelection;            /* 选择 SPDIFRX 的输入通道 */
  uint32_t Retries;                   /* 设置在同步阶段允许的最大重试次数 */
  uint32_t WaitForActivity;           /* 执行同步前是否等待 SPDIFRX_IN 线路上的活动 */
  uint32_t ChannelSelection;          /* 选择通道状态获取路径 */
  uint32_t DataFormat;                /* 设置 SPDIFRX_DR 的数据格式 */
  uint32_t StereoMode;                /* 设置是否使能立体声模式 */
  uint32_t PreambleTypeMask;          /* 设置报头类型屏蔽位 */
  uint32_t ChannelStatusMask;         /* 设置通道状态和用户数据屏蔽位 */
  uint32_t ValidityBitMask;           /* 设置有效性位屏蔽位 */
  uint32_t ParityErrorMask;           /* 设置奇偶校验屏蔽位 */
  FunctionalState SymbolClockGen;
  FunctionalState BackupSymbolClockGen;
} SPDIFRX_InitTypeDef;
```

该结构体各个成员变量的含义都在上面注释了，实际上配置的是 SPDIFRX_CR 寄存器各个位，有不理解的地方可以对照中文参考手册中寄存器位定义描述来理解。

和其他外设接口一样，HAL 库同样提供了 SPDIFRX 初始化回调函数：

```
void HAL_SPDIFRX_MspInit(SPDIFRX_HandleTypeDef *hspdif);
```

3）设置 SPDIFRX 的 DMA

我们通过 DMA 双缓冲模式来接收 SPDIF RX 接收到的数据，从而提高效率。每当一个缓冲区数据接收满以后，硬件自动切换为下一个缓冲区，同时可以将满的缓冲区数据通过 SAI 接

口，传输给 ES8388，从而实现音乐播放。SPDIFRX 使用 DMA 双缓冲接收数据的过程如图 62.3.1 所示：

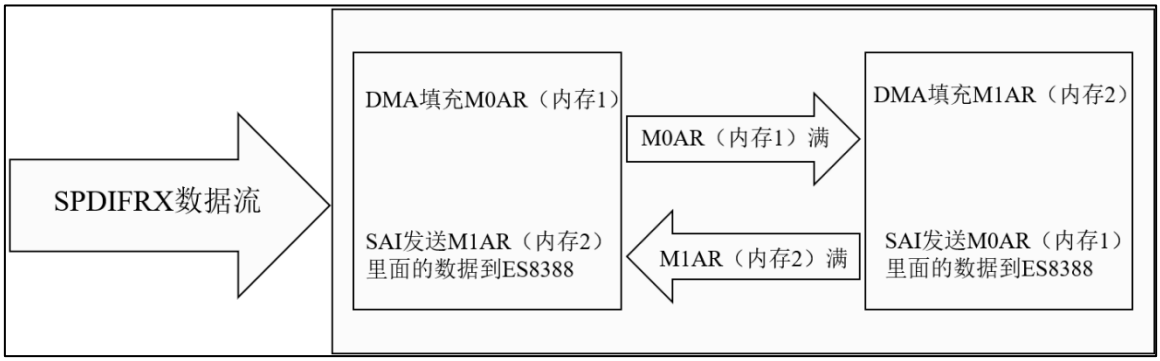


图 62.3.1 DMA 双缓冲发送音频数据流框图

4）配置 SAI

在 SPDIFRX 接口同步完成，且与 SAI 接口完成时钟同步（后续介绍），并成功获取音频数据流的采样率以后，就可以配置 SAI 接口，然后将 SPDIFRX 接收到的音频数据流，传输给 ES8388，实现音频播放。此过程主要配置 SAI 的采样率、工作模式、协议、时钟电平特性、slot 相关参数和 DMA 等。我们同样通过 DMA 双缓冲模式将数据传输给 ES8388。SAI 接口的 DMA 处理流程，请参考第六十章。

5）编写 SPDIFRX 中断服务函数

当 SPDIFRX 传输出现错误的时候（比如突然断开光纤线或采样率发生了变化），需要在 SPDIFRX 中断服务函数里面对其进行处理，当发生不可恢复的错误时，需要重新设置 SPDIFRX 进入 IDLE 状态，以便重新同步。SPDIFRX 中断服务函数为 SPDIF_RX_IRQHandler。

6）开启 DMA 传输

最后，我们就只需要开启 SPDIFRX 和 SAI 的 DMA 传输，就可以实现 SPDIF 接收光纤音频数据，并通过 ES8388 播放出来。此时，就可以在 ES8388 的耳机和喇叭通道听到光纤传输过来的音乐了。

62.3.1 程序流程图

程序的设计流程如下：

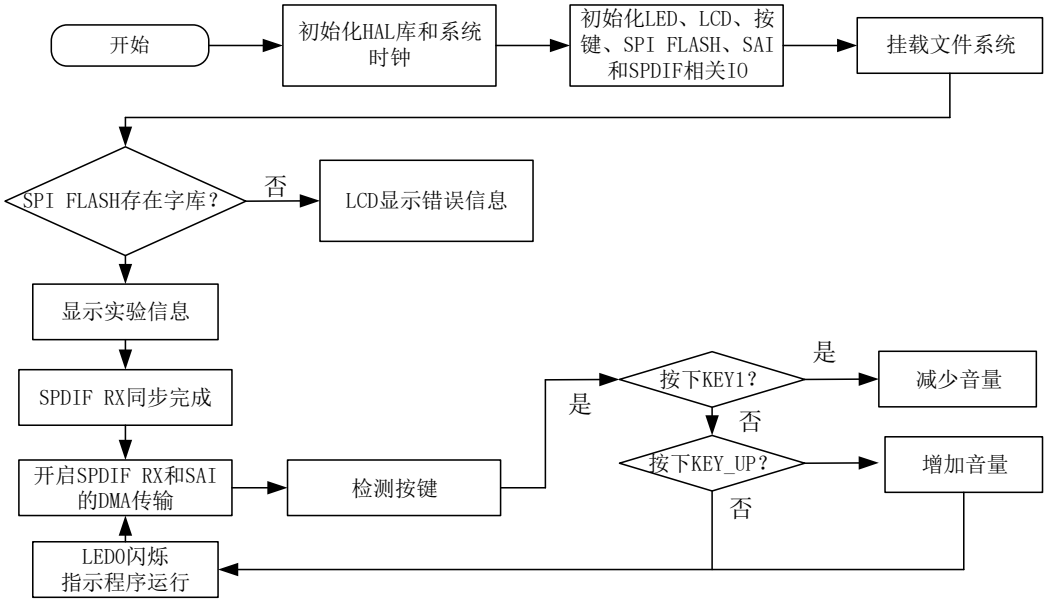


图 62.3.1.1 光纤音频实验程序流程图

62.3.2 程序解析

1. spdifrx 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码，SPDIFRX 的驱动主要包括两个文件：spdifrx.c 和 spdifrx.h。spdifrx.c 我们编写代码如下：

1， spdif_rx_init 函数

```
/**
 * @brief       初始化 SPDIF
 * @param       无
 * @retval      无
 */
void spdif_rx_init(void)
{
    spdif_dev.clock = 200000000;
    /* 设置 SPDIF CLK 的频率,为 200Mhz,要支持 192Khz 采样率必须保证 clock≥135.2Mhz */
    g_spdifin1_handle.Instance = SPDIFRX;
    g_spdifin1_handle.Init.InputSelection = SPDIFRX_INPUT_IN1; /* SPDIF 输入 1 */
    g_spdifin1_handle.Init.Retries=SPDIFRX_MAXRETRIES_15;/* 同步阶段允许重试次数 */
    g_spdifin1_handle.Init.WaitForActivity = SPDIFRX_WAITFORACTIVITY_ON;
    /* 等待同步 */
    g_spdifin1_handle.Init.ChannelSelection = SPDIFRX_CHANNEL_A;
    /* 控制流从通道 A 获取通道状态 */
    g_spdifin1_handle.Init.DataFormat = SPDIFRX_DATAFORMAT_LSB;  /* 右对齐 */
    g_spdifin1_handle.Init.StereoMode = SPDIFRX_STEREOMODE_ENABLE;
    /* 使能立体声模式 */
    g_spdifin1_handle.Init.PreambleTypeMask = SPDIFRX_PREAMBLETYPEMASK_OFF;
    /* 报头类型不复制到 SPDIFRX_DR 中 */
    g_spdifin1_handle.Init.ChannelStatusMask = SPDIFRX_CHANNELSTATUS_OFF;
    /* 通道状态和用户位不复制到 SPDIFRX_DR 中 */
    g_spdifin1_handle.Init.ValidityBitMask = SPDIFRX_VALIDITYMASK_ON;
    /* 有效性位不复制到 SPDIFRX_DR 中 */
    g_spdifin1_handle.Init.ParityErrorMask = SPDIFRX_PARITYERRORMASK_ON;
    /* 奇偶校验错误位不复制到 SPDIFRX_DR 中 */
    HAL_SPDIFRX_Init(&g_spdifin1_handle);
    g_spdifin1_handle.Instance->CR |= SPDIFRX_CR_RXDMAEN;
    /* SPDIF 音频数据使用 DMA 来接收 */

    /* 使能 SPDIF 的上溢错误和奇偶校验错误  */
    __HAL_SPDIFRX_ENABLE_IT(&g_spdifin1_handle,
                            SPDIFRX_IT_IFEIE|SPDIFRX_IT_PERRIE);
}
```

该函数用于初始化 SPDIFRX 接口，包括使能 SPDIFRX 时钟、设置 IO 口复用功能、设置 SPDIFRX 相关参数等，最后开启了奇偶检验错误中断和接口错误中断，用于处理当接收数据出现错误时（包括光纤断开、采样率切换等），重新同步。

该函数还设置了 spdifrx_ker_ck 的时钟频率，spdifrx_ker_ck 通过寄存器 RCC_D2CCIP1R 的 SPDIFSEL[1:0]位选择来源，我们设置为 00，选择来自 pll1_q_ck，因为我们在 sys.c 里面设置了 pll1_q_ck 的时钟频率为 200MHz，因此 spdifrx_ker_ck=200MHz，其频率大于 135.168MHz，因此可以支持最高 192KHz 的音频采样率。

2， HAL_SPDIFRX_MspInit 函数

```
/**
 * @brief       SPDIF 底层 IO 初始化和时钟使能
 * @note        此函数会被 HAL_SPDIF_Init()调用
 * @param       hltdc:SPDIF 句柄
 * @retval      无
 */
void HAL_SPDIFRX_MspInit(SPDIFRX_HandleTypeDef *hspdif)
{
    GPIO_InitTypeDef gpio_initure;
```

```
    __HAL_RCC_SPDIFRX_CLK_ENABLE();                     /* 使能 SPDIF RX 时钟 */
    __HAL_RCC_GPIOG_CLK_ENABLE();                       /* 使能 GPIOG 时钟 */

    /* 初始化 PG12，SPDIF IN 引脚 */
    gpio_initure.Pin = GPIO_PIN_12;                     /* PG12，SPDIF IN 引脚 */
    gpio_initure.Mode = GPIO_MODE_AF_PP;                /* 复用 */
    gpio_initure.Pull = GPIO_NOPULL;                    /* 无上下拉 */
    gpio_initure.Speed = GPIO_SPEED_FREQ_VERY_HIGH;     /* 高速 */
    gpio_initure.Alternate = GPIO_AF8_SPDIF;            /* 复用为 SPDIF RX */
    HAL_GPIO_Init(GPIOG, &gpio_initure);

    HAL_NVIC_SetPriority(SPDIF_RX_IRQn, 1, 0);          /* SPDIF 中断 */
    HAL_NVIC_EnableIRQ(SPDIF_RX_IRQn);
}
```

该函数是 SPDIFRX 的初始化回调函数，主要用来配置时钟使能，IO 口模式和中断分组等。

3，spdifrx_rx_mode 函数

```
/**
 * @brief       设置 SPDIF 的工作模式
 * @param       mode:0,IDLE 模式
 *                   1,RX 同步模式
 *                   2,保留
 *                   3,正常工作模式
 * @retval      无
 */
void spdif_rx_mode(uint8_t mode)
{
    if (mode == 0)
    {
        __HAL_SPDIFRX_IDLE(&g_spdifin1_handle);
    }
    else if (mode == 1)
    {
        __HAL_SPDIFRX_SYNC(&g_spdifin1_handle);
    }
    else if (mode == 3)
    {
        __HAL_SPDIFRX_RCV(&g_spdifin1_handle);
    }
}
```

该函数用于设置 SPDIFRX 的工作状态。可以设置三种状态：IDLE 状态、同步状态和正常工作状态。

4，spdif_rx_dma_init 函数

```
/**
 * @brief       SPDIF RX 数据 DMA 配置
 * @note        设置为双缓冲模式,并开启 DMA 传输完成中断
 * @param       buf0  : M0AR 地址.
 * @param       buf1  : M1AR 地址.
 * @param       num   : 每次传输数据量
 * @param       width : 位宽(存储器和外设,同时设置),0,8 位;1,16 位;2,32 位;
 * @retval      无
 */
void spdif_rx_dma_init(uint32_t *buf0, uint32_t *buf1,
                       uint16_t num, uint8_t width)
{
    uint32_t memwidth = 0, perwidth = 0;        /* 外设和存储器位宽 */
    switch (width)
    {
        case 0:         /* 8 位 */
            memwidth = DMA_MDATAALIGN_BYTE;
            perwidth = DMA_PDATAALIGN_BYTE;
```

```
            break;

        case 1:          /* 16 位 */
            memwidth = DMA_MDATAALIGN_HALFWORD;
            perwidth = DMA_PDATAALIGN_HALFWORD;
            break;

        case 2:          /* 32 位 */
            memwidth = DMA_MDATAALIGN_WORD;
            perwidth = DMA_PDATAALIGN_WORD;
            break;
    }

    __HAL_RCC_DMA1_CLK_ENABLE();                        /* 使能 DMA1 时钟 */
    __HAL_LINKDMA(&g_spdifin1_handle, hdmaDrRx, g_spdif_dtdma_handle);
    /* 将 DMA 与 SPDIF 联系起来 */
    g_spdif_dtdma_handle.Instance = DMA1_Stream1;              /* DMA1 数据流 1 */
    g_spdif_dtdma_handle.Init.Request=DMA_REQUEST_SPDIF_RX_DT;/* SAI1 Bock B */
    g_spdif_dtdma_handle.Init.Direction=DMA_PERIPH_TO_MEMORY;/* 外设到存储器模式*/
    g_spdif_dtdma_handle.Init.PeriphInc = DMA_PINC_DISABLE;  /* 外设非增量模式 */
    g_spdif_dtdma_handle.Init.MemInc = DMA_MINC_ENABLE;      /* 存储器增量模式 */
    g_spdif_dtdma_handle.Init.PeriphDataAlignment = perwidth;
    /* 外设数据长度:16/32 位 */
    g_spdif_dtdma_handle.Init.MemDataAlignment = memwidth;
    /* 存储器数据长度:16/32 位 */
    g_spdif_dtdma_handle.Init.Mode = DMA_CIRCULAR;            /* 使用循环模式  */
    g_spdif_dtdma_handle.Init.Priority = DMA_PRIORITY_MEDIUM;/* 中等优先级 */
    g_spdif_dtdma_handle.Init.FIFOMode = DMA_FIFOMODE_DISABLE;/* 不使用 FIFO */
    g_spdif_dtdma_handle.Init.MemBurst = DMA_MBURST_SINGLE;/* 存储器单次突发传输*/
    g_spdif_dtdma_handle.Init.PeriphBurst = DMA_PBURST_SINGLE;/*外设突发单次传输*/
    HAL_DMA_DeInit(&g_spdif_dtdma_handle);                  /* 先清除以前的设置 */
    HAL_DMA_Init(&g_spdif_dtdma_handle);                    /* 初始化 DMA */

    HAL_DMAEx_MultiBufferStart(&g_spdif_dtdma_handle, (uint32_t)&SPDIFRX->DR,
                               (uint32_t)buf0, (uint32_t)buf1,num);/* 开启双缓冲 */
}
```

该函数用于配置 SPDIFRX 接口的数据接收 DMA，使用双缓冲模式，将 SPDIFRX_DR 的数据，存储到内存里面，实现音频数据接收。

5，spdif_rx_waitsync 函数

```
/**
 * @brief       等待进入同步状态,同步完成以后自动进入接收状态
 * @param       无
 * @retval      返回值:0,未同步;1,已同步
 */
uint8_t spdif_rx_waitsync(void)
{
    uint8_t res = 0;
    uint8_t timeout = 0;
    spdif_rx_mode(SPDIF_RX_SYNC);    /* 设置为同步模式 */

    while(1)
    {
        timeout++;
        delay_ms(2);
        if (timeout > 100)break;
        if (__HAL_SPDIFRX_GET_FLAG(&g_spdifin1_handle, SPDIFRX_FLAG_SYNCD))
        /* 同步完成? */
        {
            res = 1;                        /* 标记同步完成 */
            spdif_rx_mode(SPDIF_RX_RCV);/* 进入接收模式 */
            break;
        }
```

```
    }
    return res;
}
```

该函数用于等待 SPDIFRX 同步完成，SPDIF 必须在等待同步完成以后，才可以获取音频采样率，然后进入接收状态，接收音频数据。

6，spdif_rx_getsamplerate 函数

```
/**
 * @brief       获取 SPDIF RX 收到的音频采样率
 * @param       无
 * @retval      返回值:0,错误的采样率
 *              其他值,音频采样率
 */
uint32_t spdif_rx_getsamplerate(void)
{
    uint16_t spdif_w5;
    uint32_t samplerate;
    spdif_w5 = SPDIFRX->SR >> 16;
    samplerate = (spdif_dev.clock * 5) / (spdif_w5 & 0X7FFF);
    samplerate >>= 6;                       /* 除以 64 */

    if ((8000 - 1500 <= samplerate) && (samplerate <= 8000 + 1500))
    {
        samplerate = 8000;                  /* 8K 的采样率 */
    }
    else if ((11025 - 1500 <= samplerate) && (samplerate <= 11025 + 1500))
    {
        samplerate = 11025;                 /* 11.025K 的采样率 */
    }
    else if ((16000 - 1500 <= samplerate) && (samplerate <= 16000 + 1500))
    {
        samplerate= 16000;                  /* 16K 的采样率 */
    }
    else if ((22050 - 1500 <= samplerate) && (samplerate <= 22050 + 1500))
    {
        samplerate = 22050;                  /* 22.05K 的采样率 */
    }
    else if ((32000 - 1500 <= samplerate) && (samplerate <= 32000 + 1500))
    {
        samplerate = 32000;                  /* 32K 的采样率 */
    }
    else if ((44100 - 1500 <= samplerate) && (samplerate <= 44100 + 1500))
    {
        samplerate = 44100;                  /* 44.1K 的采样率 */
    }
    else if ((48000 - 1500 <= samplerate) && (samplerate <= 48000 + 1500))
    {
        samplerate = 48000;                  /* 48K 的采样率 */
    }
    else if ((88200 - 1500 <= samplerate) && (samplerate <= 88200 + 1500))
    {
        samplerate = 88200;                  /* 88.2K 的采样率 */
    }
    else if ((96000 - 1500 <= samplerate) && (samplerate <= 96000 + 1500))
    {
        samplerate = 96000;                  /* 96K 的采样率 */
    }
    else if ((176400 - 6000 <= samplerate) && (samplerate <= 176400 + 6000))
    {
        samplerate = 176400;                 /* 176.4K 的采 */
    }
    else if ((192000 - 6000 <= samplerate) && (samplerate <= 192000 + 6000))
    {
        samplerate = 192000;                 /* 192K 的采 */
```

```
    }
    else samplerate = 0;

    return samplerate;
}
```

该函数用于获取 SPDIFRX 识别到的音频采样率。该函数首先通过 SPDIFRX_SR 寄存器的 WIDTH5 位，获取大概的音频采样率，然后标准化为最接近的音频采样率。该函数必须在 SPDIF 同步完成以后才可以调用。

7，SPDIF_RX_IRQHandler 函数

```
/**
 * @brief       SPDIF 接收中断服务函数
 * @param       无
 * @retval      无
 */
void SPDIF_RX_IRQHandler(void)
{
    /* 发生超时、同步和帧错误中断,这三个中断一定要处理！ */
    if( __HAL_SPDIFRX_GET_FLAG(&g_spdifin1_handle, SPDIFRX_FLAG_FERR)||\
        __HAL_SPDIFRX_GET_FLAG(&g_spdifin1_handle, SPDIFRX_FLAG_SERR)||\
        __HAL_SPDIFRX_GET_FLAG(&g_spdifin1_handle, SPDIFRX_FLAG_TERR))
    {
        spdif_play_stop();                      /* 发生错误，关闭 SPDIF 播放 */
        spdif_rx_stop_callback();               /* 调用回调函数 */
        spdif_rx_mode(SPDIF_RX_IDLE);
        /* 当发生超时、同步和帧错误的时候要将 SPDIFRXEN 写 0 来清除中断 */
    }
    if(__HAL_SPDIFRX_GET_FLAG(&g_spdifin1_handle,SPDIFRX_FLAG_OVR))
    {/* 上溢错误 */
        __HAL_SPDIFRX_CLEAR_IT(&g_spdifin1_handle, SPDIFRX_FLAG_OVR);
        /* 清除上溢错误中断 */
    }
    if(__HAL_SPDIFRX_GET_FLAG(&g_spdifin1_handle,SPDIFRX_FLAG_PERR))
    {/* 奇偶校验错误 */
        __HAL_SPDIFRX_CLEAR_IT(&g_spdifin1_handle, SPDIFRX_FLAG_PERR);
        /* 清除奇偶校验错误 */
    }
}
```

该函数用于处理 SPDIFRX 接口的错误中断，当发生超时错误、同步错误和帧错误的时候，必须停止 SPDIFRX 接口，然后重新进入 IDLE 状态，以便重新同步。

8，spdif_play_start 和 spdif_play_stop 函数

```
/**
 * @brief       SPDIF 开始播放
 * @param       无
 * @retval      无
 */
void spdif_play_start(void)
{
    spdif_dev.consta = 1;                           /* 标记已经打开 SPDIF */
    __HAL_DMA_ENABLE(&g_spdif_dtdma_handle);        /* 开启 DMA TX 传输 */
}

/**
 * @brief       SPDIF 关闭
 * @param       无
 * @retval      无
 */
void spdif_play_stop(void)
{
    spdif_dev.consta = 0;                           /* 标记已经关闭 SPDIF */
    spdif_dev.saisync = 0;                          /* 清空同步状态 */
    __HAL_DMA_DISABLE(&g_spdif_dtdma_handle);       /* 开启 DMA TX 传输 */
```

```
}
```

这两个函数，前者用于启动 SPDIFRX DMA 传输，在所有配置和状态都正常的情况下，用于启动 SPDIF 音频播放。后者用于关闭 SPDIFRX DMA 传输，停止 SPDIF 播放。其中 spdif_dev 结构体用于控制 SPDIF 接收状态，结构体定义如下：

```
/* SPDIF 控制结构体 */
typedef struct
{
    uint8_t consta;           /* 连接状态，0 未连接;1 连接上 */
    uint8_t saisync;          /* 与 SAI 时钟是否同步,0,未同步;1,已同步. */
    uint32_t samplerate;      /* SPDIF 采样率 */
    uint32_t clock;           /* SPDIF 时钟频率 */
}spdif_struct;
```

consta 表示 SPDIF 连接状态，当 SPDIF 开启 DMA 传输的时候，设置为 1，表示 SPDIF 正在播放。当其为 0 的时候，表示 SPDIF 停止播放。

samplerate 表示 SPDIF 识别到的音频采样率，单位为 Hz。

clock 表示 spdifrx_ker_ck 的时钟频率，单位为 Hz。

2. main.c 代码

最后我们在 main.c 中实现代码如下：

```
int main(void)
{
    uint8_t t;
    uint8_t key;
    uint8_t vol = 45;                       /* 默认音量 */

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化 */
    usmart_init(200);                       /* 初始化 USMART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    key_init();                             /* 初始化 KEY */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    norflash_init();                        /* 初始化 norflash */
    es8388_init();                          /* ES8388 初始化 */
    es8388_hpvol_set(25);                   /* 设置耳机音量 */
    es8388_spkvol_set(25);                  /* 设置喇叭音量 */
    spdif_rx_init();                        /* SPDIF 初始化 */

    my_mem_init(SRAMIN);                    /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                    /* 初始化外部内存池(SDRAM) */
    my_mem_init(SRAM12);                    /* 初始化 SRAM12 内存池(SRAM1+SRAM2) */
    my_mem_init(SRAM4);                     /* 初始化 SRAM4 内存池(SRAM4) */
    my_mem_init(SRAMDTCM);                  /* 初始化 DTCM 内存池(DTCM) */
    my_mem_init(SRAMITCM);                  /* 初始化 ITCM 内存池(ITCM) */

    exfuns_init();                          /* 为 fatfs 相关变量申请内存 */
    f_mount(fs[0], "0:", 1);                /* 挂载 SD 卡 */

    while (fonts_init())                    /* 检查字库 */
    {
        lcd_show_string(30, 50, 200, 16, 16, "Font error!", RED);
        delay_ms(200);
        lcd_fill(30, 50, 200 + 30, 50 + 16, WHITE);
        delay_ms(200);
```

```
}

text_show_string(30, 50, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
text_show_string(30, 70, 200, 16, "SPDIF(光纤音频)实验", 16, 0, RED);
text_show_string(30, 90, 200, 16, "正点原子@ALIENTEK", 16, 0, RED);
text_show_string(30, 110, 200, 16, "KEY_UP:VOL+  KEY1:VOL-", 16, 0, RED);

text_show_string(30, 150, 200, 16, "音量:", 16, 0, RED);
text_show_string(30, 170, 200, 16, "采样率:", 16, 0, RED);
lcd_show_num(30 + 40, 150, vol, 2, 16, BLUE);           /* 显示音量 */
spdif_show_samplerate(0);                               /* 显示采样率 */

es8388_adda_cfg(1, 0);                                  /* 开启 DAC */
es8388_input_cfg(0);                                    /* 关闭输入通道 */
es8388_output_cfg(1, 0);                                /* 开启 DAC 输出 */
spdif_rx_dma_init((uint32_t *)&spdif_audiobuf[0], (uint32_t*)
                  &spdif_audiobuf[1], SPDIF_DBUF_SIZE, 2);
spdif_rx_stop_callback = spdif_rx_stopplay_callback;
/* SPDIF 结束播放时的回调函数 */

while (1)
{
    key = key_scan(1);
    if (key == WKUP_PRES || key == KEY1_PRES)           /* 音量控制 */
    {
        if (key == WKUP_PRES)
        {
            vol++;

            if (vol > 63)vol = 63;
        }
        else
        {
            if (vol > 0)vol--;
        }

        es8388_hpvol_set(vol);                          /* 设置耳机音量设置 */
        es8388_spkvol_set(vol);                         /* 设置喇叭音量设置 */
        lcd_show_num(30 + 40, 150, vol, 2, 16, 0X80);/* 显示音量 */
    }

    if (spdif_dev.consta == 0)                          /* 未连接 */
    {
        if (spdif_rx_waitsync())                        /* 等待同步 */
        {
            spdif_dev.samplerate = spdif_rx_getsamplerate(); /* 获得采样率 */

            if (spdif_dev.saisync == 0)
            {   /* SAI 时钟和 SPDIF 时钟还未同步,则先进行同步 */
                if (spdif_dev.samplerate)           /* 采样率有效,同步 */
                {
                    es8388_sai_cfg(0, 0);           /* 飞利浦标准,24 位数据长度 */
                    sai1_saia_init(SAI_MODEMASTER_TX,
                        SAI_CLOCKSTROBING_RISINGEDGE, SAI_DATASIZE_24);
                   /*设置 SAI,主发送,16 位数据 */
                    sai1_samplerate_set(spdif_dev.samplerate);
                   /* 设置采样率,将修改 plln 的值 */
                    spdif_dev.saisync = 1; /* 标记 SAI 和 SPDIF 时钟同步完成 */
                }
            }
            else if (spdif_dev.samplerate)
            {  /* SAI 与 SPDIF 时钟已经同步了且采样率正常,SPDIF 可以正常工作了 */
```

```
                    spdif_play_start();                 /* 同步完成，打开 SPDIF */
                    sai1_tx_dma_init((uint8_t *)&spdif_audiobuf[0], (uint8_t *)
                     &spdif_audiobuf[1], SPDIF_DBUF_SIZE, 2);
                    /* 配置 TX DMA,32 位 */
                    sai_tx_callback = sai_dma_tx_callback;
                    sai_play_start();                               /* 开启 DMA */
                    spdif_show_samplerate(spdif_dev.samplerate);/* 显示采样率. */
                }
                else spdif_play_stop();  /* 采样率错误,停止 SPDIF 播放 */
            }
            else LED0_TOGGLE();                                     /* LED0 闪烁 */
        }
        delay_ms(20);
        t++;
        if (t > 10)
        {
            t = 0;
            LED0_TOGGLE();
        }
    }
}
```

在 main 函数里面我们初始化 ES8388 和 SPDIFRX 等外设，随后在死循环里面等待 SPDIF 同步完成，在首次同步完成以后，获取音频采样率，然后设置 SAI 接口，设置 SAI 的音频采样率，标记 SAI 与 SPDIF 时钟同步完成，然后重新同步（SPDIF 和光纤信号同步），重新同步完成以后，开启 SPDIF 和 SAI 的 DMA 传输，启动音频播放，此时就可以从喇叭或者耳机听到光纤传输过来的音频信号了。在主循环里面我们还加入了音量调节的代码，可以通过 KEY_UP 和 KEY1 调节音量的大小。

62.4 下载验证

在代码编译成功之后，我们下载代码到正点原子阿波罗 STM32H743 开发板上，先检测字库是否存在，然后进入主循环 ，等待 SPDIF 信号（光纤信号）的输入，屏幕提示：正在识别...，如图 62.4.1 所示：

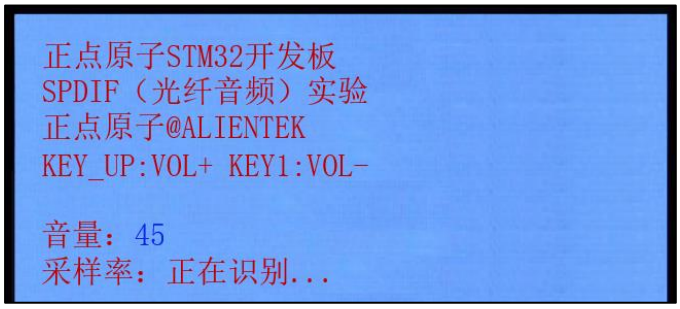


图 62.4.1 等待 SPDIF 信号输入（光纤音频信号）

此时，我们利用光纤线连接开发板的光纤座和光纤音频输出设备（比如贝斯蓝牙音频接收器），然后播放歌曲，就可以看到屏幕显示了音频信号的采样率，然后，通过耳机和板载喇叭就可以欣赏所播放的歌曲了。如图 62.4.2 所示：

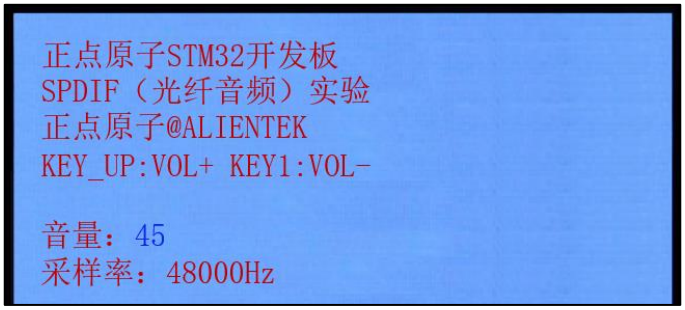


图 62.4.2 识别采样率，播放音乐

此时，我们按下 KEY_UP 按键，可以提高音频，按 KEY1 按键，可以降低音量。STM32H7 自带的 SPDIFRX 功能，可以实现对光纤、同轴等数字音频信号的解码，在 HiFi 解码方面可以有广泛的应用前景。

最后，本例程支持 44.1K、48K、88.2K 和 96KHz 等音频采样率，暂不支持 176.4K 和 192K 等高采样率的音频信号。

第六十三章 视频播放器实验

STM32H743 自带了硬件 JPEG 解码器，完全可以用来播放视频！本章，我们将使用 STM32H7 的硬件 JPEG 解码器来实现播放 AVI 视频（MJPEG 编码），本章我们将实现一个简单的视频播放器，实现 AVI 视频播放。

本章分为如下几个小节：

63.1 AVI 简介
63.2 硬件设计
63.3 程序设计
63.4 下载验证

63.1 AVI 简介

本章，我们使用 STM32H7 的硬件 JPEG 解码器，来实现 MJPG 编码的 AVI 格式视频播放，硬件 JPEG 解码器前面的实验已经介绍过了。接下来给大家简单介绍一下 AVI 格式。

AVI 是音频视频交错(Audio Video Interleaved)的英文缩写，它是微软开发的一种符合 RIFF 文件规范的数字音频与视频文件格式，原先用于 Microsoft Video for Windows (简称 VFW)环境，现在已被多数操作系统直接支持。

AVI 格式允许视频和音频交错在一起同步播放，支持 256 色和 RLE 压缩，但 AVI 文件并未限定压缩标准，AVI 仅仅是一个容器，用不同压缩算法生成的 AVI 文件，必须使用相应的解压缩算法才能播放出来。比如本章，我们使用的 AVI，其音频数据采用 16 位线性 PCM 格式（未压缩），而视频数据，则采用 MJPG 编码方式。

在介绍 AVI 文件前，我们要先来看看 RIFF 文件结构。AVI 文件采用的是 RIFF 文件结构方式，RIFF（Resource Interchange File Format，资源互换文件格式）是微软定义的一种用于管理 WINDOWS 环境中多媒体数据的文件格式，波形音频 WAVE，MIDI 和数字视频 AVI 都采用这种格式存储。构造 RIFF 文件的基本单元叫做数据块（Chunk），每个数据块包含 3 个部分：

1、4 字节的数据块标记（或者叫做数据块的 ID）
2、数据块的大小
3、数据

整个 RIFF 文件可以看成一个数据块，其数据块 ID 为 RIFF，称为 RIFF 块。一个 RIFF 文件中只允许存在一个 RIFF 块。RIFF 块中包含一系列的子块，其中有一种子块的 ID 为"LIST"，称为 LIST 块，LIST 块中可以再包含一系列的子块，但除了 LIST 块外的其他所有的子块都不能再包含子块。

RIFF 和 LIST 块分别比普通的数据块多一个被称为形式类型（Form Type）和列表类型（List Type）的数据域，其组成如下：

1、4 字节的数据块标记（Chunk ID）
2、数据块的大小
3、4 字节的形式类型或者列表类型（ID）
4、数据

下面我们看看 AVI 文件的结构。AVI 文件是目前使用的最复杂的 RIFF 文件，它能同时存储同步表现的音频视频数据。AVI 的 RIFF 块的形式类型（Form Type）是 AVI，它一般包含 3 个子块，如下所述：

1、信息块，一个 ID 为"hdrl"的 LIST 块，定义 AVI 文件的数据格式。
2、数据块，一个 ID 为 "movi"的 LIST 块，包含 AVI 的音视频序列数据。
3、索引块，ID 为"idxl"的子块，定义"movi"LIST 块的索引数据，是可选块（不一定有）。

接下来，我们详细介绍下 AVI 文件的各子块构造，AVI 文件的结构如图 63.1.1 所示：

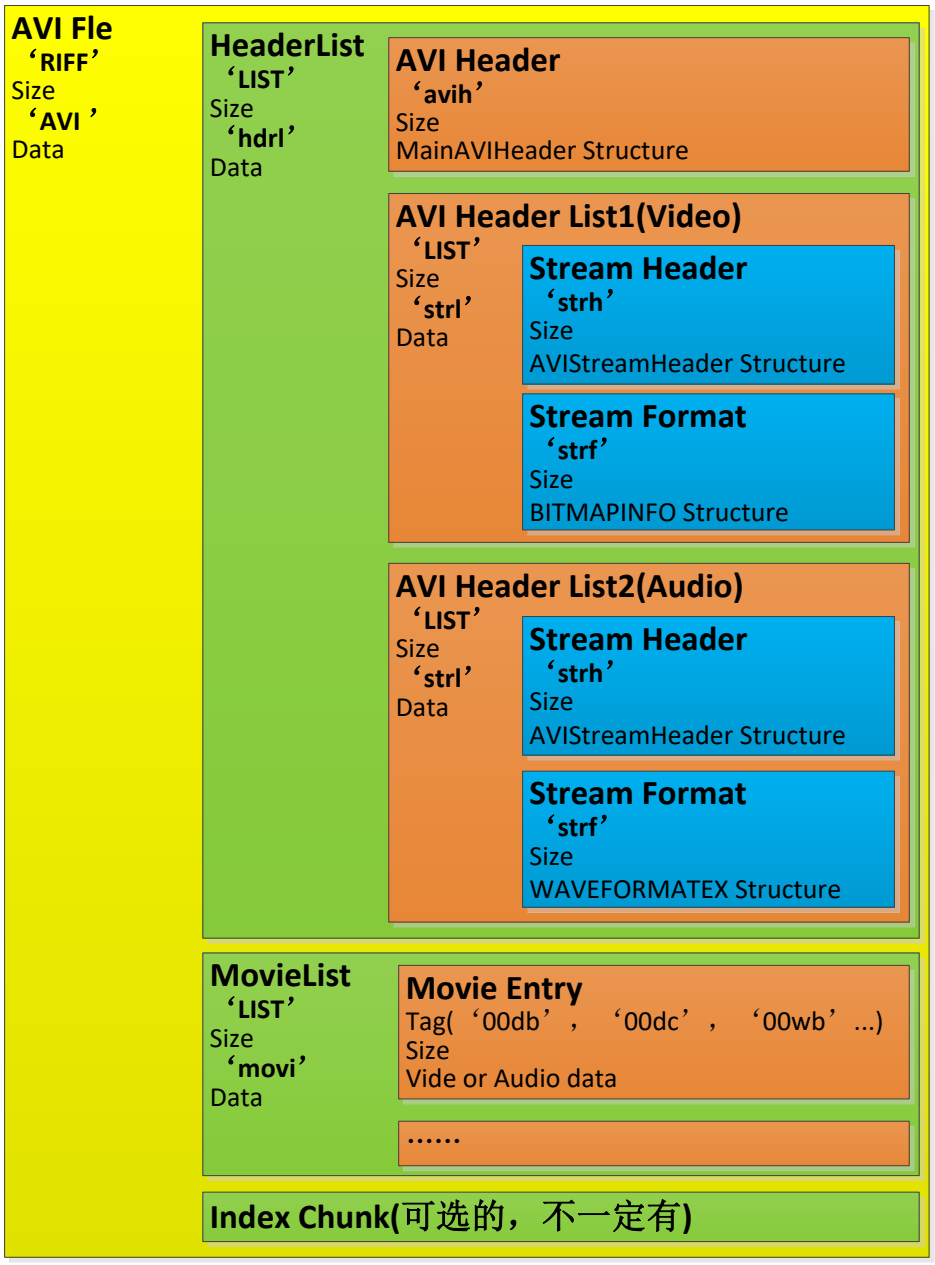


图 63.1.1 AVI 文件结构图

从上图可以看出（注意‘AVI ’，是带了一个空格的），AVI 文件，由：信息块（HeaderList）、数据块（MovieList）和索引块（Index Chunk）等三部分组成，下面，我们分别介绍这几个部分。

1、信息块（HeaderList）

信息块，即 ID 为“hdrl”的 LIST 块，它包含文件的通用信息，定义数据格式，所用的压缩算法等参数等。hdrl 块还包括了一系列的子块，首先是：avih 块，用于记录 AVI 的全局信息，比如数据流的数量，视频图像的宽度和高度等信息，avih 块（结构体都有把 BlockID 和 BlockSize 包含进来，下同）的定义如下：

```
/* avih 子块信息 */
typedef struct
{
    uint32_t BlockID;                   /* 块标志:avih==0X61766968 */
    uint32_t BlockSize;
    /* 块大小(不包含最初 8 字节,也就是 BlockID 和 BlockSize 不计算在内 */
    uint32_t SecPerFrame;               /* 视频帧间隔时间(单位为 us) */
```

```
    uint32_t MaxByteSec;            /* 最大数据传输率,字节/秒 */
    uint32_t PaddingGranularity;    /* 数据填充的粒度 */
    uint32_t Flags;                 /* AVI 文件的全局标记，比如是否含有索引块等 */
    uint32_t TotalFrame;            /* 文件总帧数 */
    uint32_t InitFrames;            /* 为交互格式指定初始帧数（非交互格式应该指定为 0）*/
    uint32_t Streams;               /* 包含的数据流种类个数,通常为 2 */
    uint32_t RefBufSize;
    /* 建议读取本文件的缓存大小（应能容纳最大的块）默认可能是 1M 字节*/
    uint32_t Width;                 /* 图像宽 */
    uint32_t Height;                /* 图像高 */
    uint32_t Reserved[4];           /* 保留 */
} AVIH_HEADER;
```

这里有很多我们要用到的信息，比如 SecPerFrame，通过该参数，我们可以知道每秒钟的帧率，也就知道了每秒钟需要解码多少帧图片，才能正常播放。TotalFrame 告诉我们整个视频有多少帧，结合 SecPerFrame 参数，就可以很方便计算整个视频的时间了。Streams 告诉我们数据流的种类数，一般是 2，即包含视频数据流和音频数据流。

在 avih 块之后，是一个或者多个 strl 子列表，文件中有多少种数据流（即前面的 Streams），就有多少个 strl 子列表。每个 strl 子列表，至少包括一个 strh（Stream Header）块和一个 strf（Stream Format）块，还有一个可选的 strn（Stream Name）块（未列出）。注意：strl 子列表出现的顺序与媒体流的编号（比如：00dc，前面的 00，即媒体流编号 00）是对应的，比如第一个 strl 子列表说明的是第一个流（Stream 0），假设是视频流，则表征视频数据块的四字符码为“00dc”，第二个 strl 子列表说明的是第二个流（Stream 1），假设是音频流，则表征音频数据块的四字符码为“01dw”，以此类推。

先看 strh 子块，该块用于说明这个流的头信息，定义如下：

```
/* strh 流头子块信息(strh∈strl) */
typedef struct
{
   uint32_t BlockID;     /* 块标志:strh==0X73747268 */
   /* 块大小(不包含最初的 8 字节,也就是 BlockID 和 BlockSize 不计算在内) */
   uint32_t BlockSize;
    uint32_t StreamType;/*数据流种类，vids(0X73646976):视频;auds(0X73647561):音频*/
    uint32_t Handler;  /*指定流的处理者，对于音视频来说就是解码器,比如 MJPG/H264 之类的*/
    uint32_t Flags;      /* 标记：是否允许这个流输出？调色板是否变化？ */
    uint16_t Priority;   /* 流的优先级（当有多个相同类型的流时优先级最高的为默认流） */
    uint16_t Language;   /* 音频的语言代号 */
    uint32_t InitFrames;/* 为交互格式指定初始帧数 */
    uint32_t Scale;      /* 数据量，视频每桢的大小或者音频的采样大小 */
    uint32_t Rate;       /* Scale/Rate=每秒采样数 */
    uint32_t Start;      /* 数据流开始播放的位置，单位为 Scale */
    uint32_t Length;     /* 数据流的数据量，单位为 Scale */
    uint32_t RefBufSize;/* 建议使用的缓冲区大小 */
    uint32_t Quality;    /* 解压缩质量参数，值越大，质量越好 */
    uint32_t SampleSize;/* 音频的样本大小 */
    struct               /* 视频帧所占的矩形 */
    {
        short Left;
        short Top;
        short Right;
        short Bottom;
    } Frame;
} STRH_HEADER;
```

这里面，对我们最有用的即 StreamType 和 Handler 这两个参数了，StreamType 用于告诉我们此 strl 描述的是音频流（“auds”），还是视频流（“vids”）。而 Handler 则告诉我们所使用的解码器，比如 MJPG/H264 等（实际以 strf 块为准）。

然后是 strf 子块，不过 strf 子块，需要根据 strh 子块的类型而定。

如果 strh 子块是视频数据流（StreamType=“vids”），则 strf 子块的内容定义如下：

```
/* BMP 结构体 */
typedef struct
{
    uint32_t BmpSize;           /* bmp 结构体大小,包含(BmpSize 在内) */
    long Width;                 /* 图像宽 */
    long Height;                /* 图像高 */
    uint16_t  Planes;           /* 平面数，必须为 1 */
    uint16_t  BitCount;         /* 像素位数,0X0018 表示 24 位 */
    uint32_t  Compression;      /* 压缩类型，比如:MJPG/H264 等 */
    uint32_t  SizeImage;        /* 图像大小 */
    long XpixPerMeter;          /* 水平分辨率 */
    long YpixPerMeter;          /* 垂直分辨率 */
    uint32_t  ClrUsed;          /* 实际使用了调色板中的颜色数,压缩格式中不使用 */
    uint32_t  ClrImportant;     /* 重要的颜色 */
} BMP_HEADER;

/* 颜色表 */
typedef struct
{
    uint8_t  rgbBlue;           /* 蓝色的亮度(值范围为 0-255) */
    uint8_t  rgbGreen;          /* 绿色的亮度(值范围为 0-255) */
    uint8_t  rgbRed;            /* 红色的亮度(值范围为 0-255) */
    uint8_t  rgbReserved;       /* 保留，必须为 0 */
} AVIRGBQUAD;

/* 对于 strh,如果是视频流,strf(流格式)使 STRH_BMPHEADER 块 */
typedef struct
{
    uint32_t BlockID;           /* 块标志,strf==0X73747266 */
   uint32_t BlockSize;          /* 块大小(不包含最初的 8 字节,也就是 BlockID
                                 和本 BlockSize 不计算在内) */
    BMP_HEADER bmiHeader;       /* 位图信息头 */
    AVIRGBQUAD bmColors[1];     /* 颜色表 */
} STRF_BMPHEADER;
```

这里有 3 个结构体，strf 子块完整内容即：STRF_BMPHEADER 结构体，不过对我们有用的信息，都存放在 BMP_HEADER 结构体里面，本结构体对视频数据的解码起决定性的作用，它告诉我们视频的分辨率（Width 和 Height），以及视频所用的编码器（Compression），因此它决定了视频的解码。本章例程仅支持解码视频分辨率小于屏幕分辨率，且编解码器必须是 MJPG 的视频格式。

如果 strh 子块是音频数据流（StreamType="auds"），则 strf 子块的内容定义如下：

```
/* 对于 strh,如果是音频流,strf(流格式)使 STRH_WAVHEADER 块 */
typedef struct
{
    uint32_t BlockID;           /* 块标志,strf==0X73747266 */
    uint32_t BlockSize;         /* 块大小(不包含最初的 8 字节,也就是 BlockID
                                   和本 BlockSize 不计算在内) */
    uint16_t FormatTag;         /* 格式标志:0X0001=PCM,0X0055=MP3 */
    uint16_t Channels;          /* 声道数,一般为 2,表示立体声 */
    uint32_t SampleRate;        /* 音频采样率 */
    uint32_t BaudRate;          /* 波特率 */
    uint16_t BlockAlign;        /* 数据块对齐标志 */
    uint16_t Size;              /* 该结构大小 */
} STRF_WAVHEADER;
```

本结构体对音频数据解码起决定性的作用，他告诉我们音频信号的编码方式（FormatTag）、声道数（Channels）和采样率（SampleRate）等重要信息。本章例程仅支持 PCM 格式（FormatTag=0X0001）的音频数据解码。

2、数据块（MovieList）

信息块，即 ID 为“movi”的 LIST 块，它包含 AVI 的音视频序列数据，是这个 AVI 文件的

主体部分。音视频数据块交错的嵌入在“movi”LIST 块里面，通过标准类型码进行区分，标准类型码有如下 4 种：

1，“##db”（非压缩视频帧）、
2，“##dc”（压缩视频帧）、
3，“##pc”（改用新的调色板）、
4，“##wb”（音频帧）。

其中##是编号，得根据我们的数据流顺序来确定，也就是前面的 strl 块。比如，如果第一个 strl 块是视频数据，那么对于压缩的视频帧，标准类型码就是：00dc。第二个 strl 块是音频数据，那么对于音频帧，标准类型码就是：01wb。

紧跟着标准类型码的是 4 个字节的数据长度（不包含类型码和长度参数本身，也就是总长度必须要加 8 才对），该长度必须是偶数，如果读到为奇数，则加 1 即可。我们读数据的时候，一般一次性要读完一个标准类型码所表征的数据，方便解码。

3、索引块（Index Chunk）

最后，紧跟在‘hdrl’列表和‘movi’列表之后的，就是 AVI 文件可选的索引块。这个索引块为 AVI 文件中每一个媒体数据块进行索引，并且记录它们在文件中的偏移（可能相对于‘movi’列表，也可能相对于 AVI 文件开头）。本章我们用不到索引块，这里就不详细介绍了。

关于 AVI 文件，我们就介绍到这，有兴趣的朋友，可以再看看光盘：6，软件资料→AVI 学习资料 里面的相关文档。

最后，我们看看要实现 avi 视频文件的播放，主要有哪些步骤，如下：

1）初始化各外设

要解码视频，相关外设肯定要先初始化好，比如：SDMMC（驱动 SD 卡用）、SAI、DMA、ES8388、LCD 和按键等。这些具体初始化过程，在前面的例程都有介绍，大同小异，这里就不再细说了。

2）读取 AVI 文件，并解析

要解码，得先读取 avi 文件，读取出音视频关键信息，音频参数：编码方式、采样率、位数和音频流类型码(01wb/00wb)等；视频参数：编码方式、帧间隔、图片尺寸和视频流类型码(00dc/01dc)等；共同的：数据流起始地址。有了这些参数，我们便可以初始化音视频解码，为后续解码做好准备。

3）根据解析结果，设置相关参数

根据第 2 步解析的结果，设置 SAI 的音频采样率和位数，同时要让视频显示在 LCD 中间区域，得根据图片尺寸，设置 LCD 开窗时 x，y 方向的偏移量。

4）读取数据流，开始解码

前面三步完成，就可以正式开始播放视频了。读取视频流数据（movi 块），根据类型码，执行音频/视频解码。对于音频数据(01wb/00wb)，本例程只支持未压缩的 PCM 数据，所以，直接填充到 DMA 缓冲区即可，由 DMA 循环发送给 ES8388，播放音频。对于视频数据(00dc/01dc)，本例程只支持 MJPG，通过硬件 JPEG 解码，硬件 JPEG 解码流程详见第五十八章。然后，利用定时器来控制帧间隔，以正常速度播放视频，从而实现音视频解码。

5）解码完成，释放资源

最后在文件读取完后（或者出错了），需要释放申请的内存、恢复 LCD 窗口、关闭定时器、停止 SAI 播放音乐和关闭文件等一系列操作，等待下一次解码。

63.2 硬件设计

1. 例程功能

1、本实验开机后，先初始化各外设，然后检测字库是否存在，如果检测无问题，则开始播放 SD 卡 VIDEO 文件夹里面的视频（.avi 格式）。

注意：自备 SD 卡一张，并在 SD 卡根目录建立一个 VIDEO 文件夹，存放 AVI 视频（仅支持 MJPG 视频，音频必须是 PCM，且视频分辨率必须小于等于屏幕分辨率）在里面。例程所

需视频，可以通过：狸窝全能视频转换器，转换后得到，具体步骤见 63.4 章节内容。

视频播放时，LCD 上会显示视频名字、当前视频编号、总视频数、声道数、音频采样率、帧率、播放时间和总时间等信息。KEY0 用于选择下一个视频，KEY1 用于选择上一个视频，KEY_UP 可以快进，KEY1 可以快退。

2、LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
LED1：LED1 - PB0
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)
4）独立按键：KEY0 - PH3、KEY1 - PH2、WK_UP - PA0
5）SD 卡，通过 SDMMC1(SDMMC_D0~D4(PC8~PC11)，
SDMMC_SCK(PC12)，SDMMC_CMD(PD2))连接
6）norflash(QSPI FLASH 芯片，连接在 QSPI 上)
7）硬件 JPEG 解码内核(STM32H743 自带)
8）定时器 6、7

63.3 程序设计

63.3.1 程序流程图

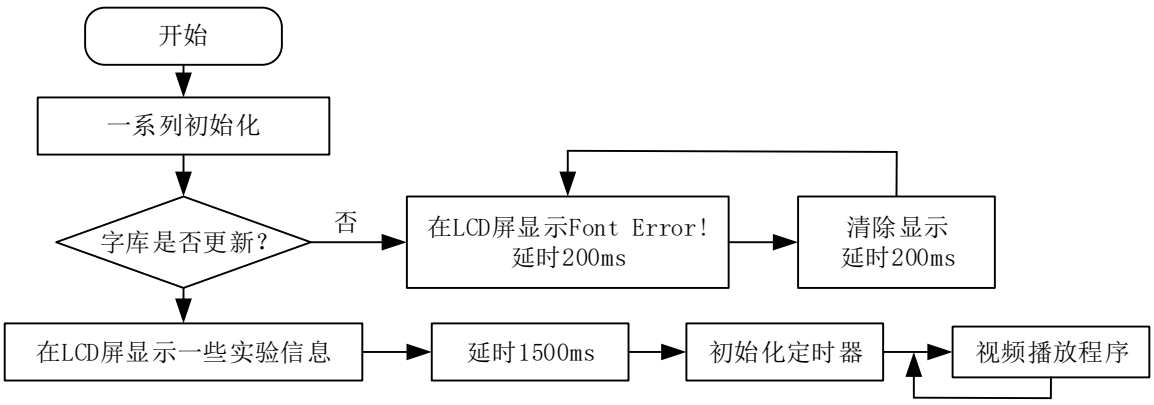


图 63.3.1.1 照相机实验程序流程图

63.3.2 程序解析

1. MJPEG 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。MJPEG 驱动源码包括四个文件：avi.c、avi.h、mjpeg.c 和 mjpeg.h。

avi.h 头文件在 63.1 小节部分讲过，具体请看源码。下面来看到 avi.c 文件，这里总共有三个函数都很重要，首先介绍 AVI 解码初始化函数，该函数定义如下：

```
/* avi 文件相关信息 */
AVI_INFO g_avix;
/* 视频编码标志字符串,00dc/01dc */
char *const AVI_VIDS_FLAG_TBL[2] = {"00dc", "01dc"};
/* 音频编码标志字符串,00wb/01wb */
```

```
char *const AVI_AUDS_FLAG_TBL[2] = {"00wb", "01wb"};

/**
 * @brief       AVI 解码初始化
 * @param       buf  : 输入缓冲区
 * @param       size : 缓冲区大小
 * @retval      执行结果
 *   @arg       AVI_OK, AVI 文件解析成功
 *   @arg       其他  , 错误代码
 */
AVISTATUS avi_init(uint8_t *buf, uint32_t size)
{
    uint16_t offset;
    uint8_t *tbuf;
    AVISTATUS res = AVI_OK;
    AVI_HEADER *aviheader;
    LIST_HEADER *listheader;
    AVIH_HEADER *avihheader;
    STRH_HEADER *strhheader;
    STRF_BMPHEADER *bmpheader;
    STRF_WAVHEADER *wavheader;

    tbuf = buf;
    aviheader = (AVI_HEADER *)buf;
    if (aviheader->RiffID != AVI_RIFF_ID)return AVI_RIFF_ERR; /* RIFF ID 错误 */
    if (aviheader->AviID != AVI_AVI_ID)return AVI_AVI_ERR;    /* AVI ID 错误 */
    buf += sizeof(AVI_HEADER);  /* 偏移 */
    listheader = (LIST_HEADER *)(buf);
    if (listheader->ListID != AVI_LIST_ID)return AVI_LIST_ERR;/* LIST ID 错误 */
    if (listheader->ListType!=AVI_HDRL_ID)return AVI_HDRL_ERR;/* HDRL ID 错误*/
    buf += sizeof(LIST_HEADER); /* 偏移 */
    avihheader = (AVIH_HEADER *)(buf);
    if (avihheader->BlockID!= AVI_AVIH_ID)return AVI_AVIH_ERR;/* AVIH ID 错误 */
    g_avix.SecPerFrame = avihheader->SecPerFrame;  /* 得到帧间隔时间 */
    g_avix.TotalFrame = avihheader->TotalFrame;    /* 得到总帧数 */
    buf += avihheader->BlockSize + 8;              /* 偏移 */
    listheader = (LIST_HEADER *)(buf);
    if (listheader->ListID != AVI_LIST_ID)return AVI_LIST_ERR;/* LIST ID 错误 */
    if (listheader->ListType!= AVI_STRL_ID)return AVI_STRL_ERR;/* STRL ID 错误*/
    strhheader = (STRH_HEADER *)(buf + 12);
    if (strhheader->BlockID!= AVI_STRH_ID)return AVI_STRH_ERR;/* STRH ID 错误 */
    if (strhheader->StreamType == AVI_VIDS_STREAM)             /* 视频帧在前 */
    {
        /* 非 MJPG 视频流,不支持 */
        if (strhheader->Handler != AVI_FORMAT_MJPG)return AVI_FORMAT_ERR;
        g_avix.VideoFLAG=(uint8_t *)AVI_VIDS_FLAG_TBL[0];/* 视频流标记  "00dc" */
        g_avix.AudioFLAG=(uint8_t *)AVI_AUDS_FLAG_TBL[1];/* 音频流标记  "01wb" */
        /* strf */
        bmpheader = (STRF_BMPHEADER *)(buf + 12 + strhheader->BlockSize + 8);
        if(bmpheader->BlockID!= AVI_STRF_ID)return AVI_STRF_ERR;/*STRF ID 错误*/
        g_avix.Width = bmpheader->bmiHeader.Width;
        g_avix.Height = bmpheader->bmiHeader.Height;
        buf += listheader->BlockSize + 8;          /* 偏移 */
        listheader = (LIST_HEADER *)(buf);
        if (listheader->ListID != AVI_LIST_ID)     /* 是不含有音频帧的视频文件 */
        {
            g_avix.SampleRate = 0;      /* 音频采样率 */
            g_avix.Channels = 0;        /* 音频通道数 */
            g_avix.AudioType = 0;       /* 音频格式 */
        }
        else
        {
```

```
            /* STRL ID 错误 */
            if (listheader->ListType != AVI_STRL_ID)return AVI_STRL_ERR;
            strhheader = (STRH_HEADER *)(buf + 12);
            /* STRH ID 错误 */
            if (strhheader->BlockID != AVI_STRH_ID)return AVI_STRH_ERR;
            /* 格式错误 */
            if (strhheader->StreamType != AVI_AUDS_STREAM)
                return AVI_FORMAT_ERR;
            /* strf */
            wavheader = (STRF_WAVHEADER *)(buf + 12 + strhheader->BlockSize+8);
            /* STRF ID 错误 */
            if (wavheader->BlockID != AVI_STRF_ID)return AVI_STRF_ERR;
            g_avix.SampleRate = wavheader->SampleRate;      /* 音频采样率 */
            g_avix.Channels = wavheader->Channels;          /* 音频通道数 */
            g_avix.AudioType = wavheader->FormatTag;        /* 音频格式 */
        }
    }
    else if (strhheader->StreamType == AVI_AUDS_STREAM)     /* 音频帧在前 */
    {
       g_avix.VideoFLAG = (uint8_t *)AVI_VIDS_FLAG_TBL[1];
       /* 视频流标记 "01dc" */
       g_avix.AudioFLAG = (uint8_t *)AVI_AUDS_FLAG_TBL[0];
       /* 音频流标记 "00wb" */
       /* strf */
       wavheader = (STRF_WAVHEADER *)(buf + 12 + strhheader->BlockSize + 8);
       if (wavheader->BlockID != AVI_STRF_ID)return AVI_STRF_ERR;
       /* STRF ID 错误 */
       g_avix.SampleRate = wavheader->SampleRate;           /* 音频采样率 */
       g_avix.Channels = wavheader->Channels;               /* 音频通道数 */
       g_avix.AudioType = wavheader->FormatTag;             /* 音频格式 */
       buf += listheader->BlockSize + 8;                    /* 偏移 */
       listheader = (LIST_HEADER *)(buf);
       if (listheader->ListID != AVI_LIST_ID)return AVI_LIST_ERR;
       /* LIST ID 错误 */
       /* STRL ID 错误 */
       if (listheader->ListType != AVI_STRL_ID)return AVI_STRL_ERR;
       strhheader = (STRH_HEADER *)(buf + 12);/* STRH ID 错误 */
       if (strhheader->BlockID != AVI_STRH_ID)return AVI_STRH_ERR;/* 格式错误 */
       if (strhheader->StreamType != AVI_VIDS_STREAM)return AVI_FORMAT_ERR;
       /* strf */
       bmpheader = (STRF_BMPHEADER *)(buf + 12 + strhheader->BlockSize + 8);
       if (bmpheader->BlockID != AVI_STRF_ID)return AVI_STRF_ERR;
       /* STRF ID 错误 */
       if (bmpheader->bmiHeader.Compression != AVI_FORMAT_MJPG
       return AVI_FORMAT_ERR;                               /* 格式错误 */
       g_avix.Width = bmpheader->bmiHeader.Width;
       g_avix.Height = bmpheader->bmiHeader.Height;
    }
    offset = avi_srarch_id(tbuf, size, (uint8_t*)"movi"); /* 查找 movi ID */
    if (offset == 0)return AVI_MOVI_ERR;                    /* MOVI ID 错误 */
    if (g_avix.SampleRate)                                  /* 有音频流,才查找 */
    {
        tbuf += offset;
        offset = avi_srarch_id(tbuf, size, g_avix.AudioFLAG); /* 查找音频流标记 */
        if (offset == 0)return AVI_STREAM_ERR;          /* 流错误 */
        tbuf += offset + 4;
        g_avix.AudioBufSize = *((uint16_t *)tbuf);      /* 得到音频流 buf 大小 */
    }
    printf("avi init ok\r\n");
    printf("g_avix.SecPerFrame:%d\r\n", g_avix.SecPerFrame);
    printf("g_avix.TotalFrame:%d\r\n", g_avix.TotalFrame);
    printf("g_avix.Width:%d\r\n", g_avix.Width);
```

```
    printf("g_avix.Height:%d\r\n", g_avix.Height);
    printf("g_avix.AudioType:%d\r\n", g_avix.AudioType);
    printf("g_avix.SampleRate:%d\r\n", g_avix.SampleRate);
    printf("g_avix.Channels:%d\r\n", g_avix.Channels);
    printf("g_avix.AudioBufSize:%d\r\n", g_avix.AudioBufSize);
    printf("g_avix.VideoFLAG:%s\r\n", g_avix.VideoFLAG);
    printf("g_avix.AudioFLAG:%s\r\n", g_avix.AudioFLAG);
    return res;
}
```

该函数用于解析 AVI 文件，获取音视频流数据的详细信息，为后续解码做准备。

接下来介绍的是查找 ID 函数，其定义如下：

```
/**
 * @brief       查找 ID
 * @param       buf  : 输入缓冲区
 * @param       size : 缓冲区大小
 * @param       id   : 要查找的 id, 必须是 4 字节长度
 * @retval      执行结果
 *   @arg       0      , 没找到
 *   @arg       其他   , movi ID 偏移量
 */
uint32_t avi_srarch_id(uint8_t *buf, uint32_t size, char *id)
{
    uint32_t i;
    uint32_t idsize = 0;
    size -= 4;
    for (i = 0; i < size; i++)
    {
        if ((buf[i] == id[0]) &&
            (buf[i + 1] == id[1]) &&
            (buf[i + 2] == id[2]) &&
            (buf[i + 3] == id[3]))
        {
            /* 得到帧大小,必须大于 16 字节,才返回,否则不是有效数据 */
            idsize = MAKEDWORD(buf + i + 4);
            if (idsize > 0X10)return i;          /* 找到"id"所在的位置 */
        }
    }
    return 0;
}
```

该函数用于查找某个 ID，可以是 4 个字节长度的 ID，比如 00dc，01wb，movi 之类的，在解析数据以及快进快退的时候，有用到。

接下来介绍的是得到 stream 流信息函数，其定义如下：

```
/**
 * @brief       得到 stream 流信息
 * @param       buf  : 流开始地址(必须是 01wb/00wb/01dc/00dc 开头)
 * @retval      执行结果
 *   @arg       AVI_OK, AVI 文件解析成功
 *   @arg       其他   , 错误代码
 */
AVISTATUS avi_get_streaminfo(uint8_t *buf)
{
    g_avix.StreamID = MAKEWORD(buf + 2);            /* 得到流类型 */
    g_avix.StreamSize = MAKEDWORD(buf + 4);         /* 得到流大小 */
    if (g_avix.StreamSize > AVI_MAX_FRAME_SIZE)     /* 帧大小太大了,直接返回错误 */
    {
        g_avix.StreamSize = 0;
        return AVI_STREAM_ERR;
    }
    /* 奇数加 1(g_avix.StreamSize,必须是偶数) */
    if (g_avix.StreamSize % 2)g_avix.StreamSize++;
    if (g_avix.StreamID == AVI_VIDS_FLAG || g_avix.StreamID == AVI_AUDS_FLAG)
      return AVI_OK;
```

```
    return AVI_STREAM_ERR;
}
```

该函数用来获取当前数据流信息，重点是取得流类型和流大小，方便解码和读取下一个数据流。

mjpeg.h 文件只有一些函数和变量声明，接下来，介绍 mjpeg.c 里面的几个函数，首先是初始化 MJPEG 函数，其定义如下：

```
/**
 * @brief       初始化 MJPEG
 * @param       offx    : 显示图像在 LCD 上 x 方向的偏移量
 * @param       offy    : 显示图像在 LCD 上 y 方向的偏移量
 * @param       width   : 显示图像的宽度
 * @param       height  : 显示图像的高度
 * @retval      执行结果
 *   @arg       0     , 成功
 *   @arg       其他  , 失败
 */
uint8_t mjpeg_init(uint16_t offx, uint16_t offy, uint32_t width,
                   uint32_t height)
{
    uint8_t i;
    uint8_t res;
    res = mjpeg_jpeg_core_init(&mjpeg);    /* 初始化 JPEG 内核,不申请 IN BUF */
    if (res)return 1;
    for (i = 0; i < JPEG_DMA_OUTBUF_NB; i++)
    {
        /* 最大是图片宽度的 24 倍,另外还可能会多需要 32 字节内存 */
        mjpeg.outbuf[i].buf = mymalloc(SRAMIN, width * 24 + 32);
        if (mjpeg.outbuf[i].buf == NULL)return 2;
    }
    p_rgb565buf = 0;                                    /* RGB565 BUF 指针清零 */
    if (lcdltdc.pwidth == 0 || lcddev.dir == 0)
    /* 如果不是 RGB 横屏,则需要 p_rgb565buf 数组 */
    {
        p_rgb565buf = mymalloc(SRAMEX, width * height * 2); /* 申请 RGB 缓存 */
        if (p_rgb565buf == NULL)return  3;
    }
    g_img_offx = offx;
    g_img_offy = offy;
    mjpeg_rgb_framebuf = (uint16_t *)g_ltdc_framebuf[lcdltdc.activelayer];
    /* 指向 RGBLCD 当前显存 */
    return 0;
}
```

该函数用于初始化 jpeg 解码 ，调用 mjpeg_jpeg_core_init 函数，对硬件 JPEG 解码内核进行初始化（类似 jpeg_core_init 函数，详见本例程源码），然后申请内存，确定视频在液晶上面的偏移（让视频显示在 LCD 中央）。

注意：如果是 MCU 屏，我们需要申请内存。如果是 RGB 屏，则可以直接使用 RGB 屏的显存。当使用 RGB 横屏的时候，图片解码后的数据，直接通过 DMA2D 将 YUV 数据转换成 RGB 数据，存放在 RGB 屏的显存位置，这样就可以达到最高的速度！（省去了内存拷贝）

下面介绍的是 MJPEG 释放所有申请的内存函数，其定义如下：

```
/**
 * @brief       MJPEG 释放所有申请的内存
 * @param       无
 * @retval      无
 */
void mjpeg_free(void)
{
    mjpeg_jpeg_core_destroy(&mjpeg);
    myfree(SRAMEX, p_rgb565buf);
}
```

该函数用于释放内存，解码结束后调用。

下面介绍的是 RGB 屏横屏模式函数，其定义如下：

```
/**
 * @brief       对于 RGB 屏,且是横屏模式,则可以利用 DMA2D,直接将 JPEG 解码的 YUV 数据转换成
 *              RGB 数据,并填充到对应的 GRAM 里面,所有操作全硬件完成,速度非常快.
 * @param       sx,sy:起始坐标
 * @param       tjpeg:jpeg 解码结构体
 * @retval      无
 */
void mjpeg_ltdc_dma2d_yuv2rgb_fill(uint16_t sx, uint16_t sy,
                                   jpeg_codec_typedef *tjpeg)
{
    uint32_t timeout = 0;
    uint16_t offline;
    uint32_t addr;
    uint32_t regval = 0;
    uint32_t cm = 0;                            /* 采样方式 */

    if (tjpeg->Conf.ChromaSubsampling == JPEG_420_SUBSAMPLING)
    {
        cm = DMA2D_CSS_420;                     /* YUV420 转 RGB */
    }
    if (tjpeg->Conf.ChromaSubsampling == JPEG_422_SUBSAMPLING)
    {
        cm = DMA2D_CSS_422;                     /* YUV422 转 RGB */
    }
    else if (tjpeg->Conf.ChromaSubsampling == JPEG_444_SUBSAMPLING)
    {
        cm = DMA2D_NO_CSS;                      /* YUV444 转 RGB */
    }

    offline = lcdltdc.pwidth - tjpeg->Conf.ImageWidth;
    addr = ((uint32_t)g_ltdc_framebuf[lcdltdc.activelayer] + lcdltdc.pixsize *
            (lcdltdc.pwidth * sy + sx));
    RCC->AHB3ENR |= 1 << 4;                     /* 使能 DMA2D 时钟 */
    RCC->AHB3RSTR |= 1 << 4;                    /* 复位 DMA2D */
    RCC->AHB3RSTR &= ~(1 << 4);                 /* 结束复位 */
    DMA2D->CR &= ~(1 << 0);                     /* 先停止 DMA2D */
    DMA2D->CR = 1 << 16;                        /* MODE[1:0]=01,存储器到存储器,带 PFC 模式 */
    DMA2D->OPFCCR = 2 << 0;                     /* CM[2:0]=010,输出为 RGB565 格式 */
    DMA2D->OOR = offline;                       /* 设置行偏移 */
    DMA2D->IFCR |= 1 << 1;                      /* 清除传输完成标志 */
    regval = 11 << 0;                           /* CM[3:0]=1011,输入数据为 YCbCr 格式 */
    regval |= cm << 18;
    /* CSS[1:0]=cm,Chroma Sub-Sampling:0,4:4:4;1,4:2:2;2,4:2:0 */
    DMA2D->FGPFCCR = regval;                    /* 设置 FGPCCR 寄存器 */
    DMA2D->FGOR = 0;                            /* 前景层行偏移为 0 */
    DMA2D->NLR = tjpeg->yuvblk_height | (tjpeg->Conf.ImageWidth << 16);
    /* 设定行数寄存器 */
    DMA2D->OMAR = addr;                         /* 输出存储器地址 */
    DMA2D->FGMAR = (uint32_t)tjpeg->outbuf[tjpeg->outbuf_read_ptr].buf;
    /* 源地址 */
    DMA2D->CR |= 1 << 0;                        /* 启动 DMA2D */

    while ((DMA2D->ISR & (1 << 1)) == 0)/* 等待传输完成 */
    {
        timeout++;

        if (timeout > 0X1FFFFFF)break;  /* 超时退出 */
    }

    tjpeg->yuvblk_curheight += tjpeg->yuvblk_height;    /* 偏移到下一个内存地址 */
    /* YUV2RGB 转码结束后,再复位一次 DMA2D */
```

```
    RCC->AHB3RSTR |= 1 << 4;            /* 复位 DMA2D */
    RCC->AHB3RSTR &= ~(1 << 4);         /* 结束复位 */
}
```

该函数在 RGB 横屏的情况下，通过 DMA2D，该函数可以直接将 YUV 数据转换成 RGB 数据并存放到 RGB 屏的显存上面，效率非常高，可以达到最高的视频解码性能。

下面介绍的是填充颜色函数，其定义如下：

```
/**
 * @brief       填充颜色
 * @param       x, y    : 起始坐标
 * @param       width   : 宽度
 * @param       height  : 高度
 * @param       color   : 颜色数组
 * @retval      无
 */
void mjpeg_fill_color(uint16_t x, uint16_t y, uint16_t width, uint16_t height,
                      uint16_t *color)
{
    uint16_t i, j;
    uint32_t param1;
    uint32_t param2;
    uint32_t param3;
    uint16_t *pdata;

    if (lcdltdc.pwidth != 0 && lcddev.dir == 0)
    /* 如果是 RGB 屏,且竖屏,则填充函数不可直接用 */
    {
        param1 = lcdltdc.pixsize * lcdltdc.pwidth *
        (lcdltdc.pheight - x - 1) + lcdltdc.pixsize * y;
         /* 将运算先做完,提高速度 */
        param2 = lcdltdc.pixsize * lcdltdc.pwidth;

        for (i = 0; i < height; i++)
        {
            param3 = i * lcdltdc.pixsize + param1;
            pdata = color + i * width;

            for (j = 0; j < width; j++)
            {
                *(uint16_t *)((uint32_t)mjpeg_rgb_framebuf +
                  param3 - param2 * j) = pdata[j];
            }
        }
    }
    else if (lcdltdc.pwidth == 0)
    {
        lcd_color_fill(x, y, x + width - 1, y + height - 1, color);
        /* 是 MCU 屏(RGB 横屏无需填充!!,在 YUV 转换的时候,直接就填充了) */
    }
}
```

该函数用于解码完成后，将 RGB565 数据填充到液晶屏上，对于 RGB 屏的竖屏模式，不能用 DMA2D 填充，只能打点的方式填充，通过计算参量，提高打点速度。对于 MCU 屏则直接调用 lcd_color_fill 函数进行填充。对于 RGB 横屏，则无需做任何操作，因为在进行 YUV 转换的时候，就通过 mjpeg_ltdc_dma2d_yuv2rgb_fill 完成了填充。

下面介绍的是解码一副 JPEG 图片函数，其定义如下：

```
/**
 * @brief       解码一副 JPEG 图片
 * @note        注意事项:
 *              1, 待解吗图片的分辨率,必须小于等于屏幕的分辨率!
 *              2, 请保证图片的宽度是 16 的倍数,以免左侧出现花纹.
 * @param       buf     : jpeg 数据流数组
 * @param       bsize   : 数组大小
```

```
 * @retval      执行结果
 *   @arg       0     , 成功
 *   @arg       其他  , 失败
 */
uint8_t mjpeg_decode(uint8_t *buf, uint32_t bsize)
{
    volatile uint32_t timecnt = 0;
    if (bsize == 0)return 0;
    jpeg_decode_init(&mjpeg);           /* 初始化硬件 JPEG 解码器 */
    g_mjpeg_remain_size = bsize;        /* 记录当前图片的大小(字节数) */
    mjpeg.inbuf[0].buf =(char*)buf;     /* 指向 jpeg 数据流的首地址 */
    g_mjpeg_fileover = 0;               /* 标记未读完 */
    if (g_mjpeg_remain_size < JPEG_DMA_INBUF_LEN)/* 图片比较小,一次就可以传输完成 */
    {
        mjpeg.inbuf[0].size=g_mjpeg_remain_size;   /* 传输大小等于总大小 */
        g_mjpeg_remain_size = 0;                    /* 一次传输就可以搞完 */
        return 0;   /* 图片尺寸比较小，直接不解码 */
    }
    else    /* 图片比较大,需要分多次传输 */
    {
        mjpeg.inbuf[0].size = JPEG_DMA_INBUF_LEN;   /* 按最大传输长度,分批次传输 */
        g_mjpeg_remain_size -= JPEG_DMA_INBUF_LEN;  /* 剩余长度 */
    }
    /* 配置输入 DMA */
    jpeg_in_dma_init((uint32_t)mjpeg.inbuf[0].buf, mjpeg.inbuf[0].size);
    jpeg_in_callback = mjpeg_dma_in_callback;       /* JPEG DMA 读取数据回调函数 */
    jpeg_out_callback = mjpeg_dma_out_callback;     /* JPEG DMA 输出数据回调函数 */
    jpeg_eoc_callback = mjpeg_endofcovert_callback;/* JPEG 解码结束回调函数 */
    jpeg_hdp_callback = mjpeg_hdrover_callback; /* JPEG Header 解码完成回调函数 */
    jpeg_in_dma_start();        /* 启动 DMA IN 传输,开始解码 JPEG 图片 */
    while (1)
    {
        if(mjpeg.outbuf[mjpeg.outbuf_read_ptr].sta == 1)/* buf 里面有数据要处理 */
        {
            //SCB_CleanInvalidateDCache();                  /* 清空 D catch */
            if (lcdltdc.pwidth == 0 || lcddev.dir == 0)
            /* 非 RGB 横屏,需要先将 YUV 数据解码到 p_rgb565buf,
               然后再从 p_rgb565buf 拷贝到 LCD 的 GRAM */
            {
                jpeg_dma2d_yuv2rgb_conversion(&mjpeg, (uint32_t *)p_rgb565buf);
                /* 利用 DMA2D,将 YUV 图像转成 RGB565 图像 */
            }
            else
            /* RGB 横屏,直接将 YUV 数据解码到 LCD 的 GRAM,省去了拷贝操作,可以达到最佳性能 */
            {
                mjpeg_ltdc_dma2d_yuv2rgb_fill(g_img_offx, g_img_offy +
                                              mjpeg.yuvblk_curheight, &mjpeg);
                /* DMA2D 将 YUV 解码到 LCD GRAM,速度最快 */
            }

            //SCB_CleanInvalidateDCache();                  /* 清空 D catch */
            mjpeg.outbuf[mjpeg.outbuf_read_ptr].sta = 0;    /* 标记 buf 为空 */
            mjpeg.outbuf[mjpeg.outbuf_read_ptr].size = 0;   /* 数据量清空 */
            mjpeg.outbuf_read_ptr++;

            if (mjpeg.outbuf_read_ptr >=
                JPEG_DMA_OUTBUF_NB)mjpeg.outbuf_read_ptr = 0;  /* 限制范围 */

            if (mjpeg.yuvblk_curheight >= mjpeg.Conf.ImageHeight)break;
            /* 当前高度等于或者超过图片分辨率的高度,则说明解码完成了,直接退出. */
        }
```

```
        else if (mjpeg.outdma_pause == 1 &&
                 mjpeg.outbuf[mjpeg.outbuf_write_ptr].sta == 0)
            /* out 暂停,且当前 writebuf 已经为空了,则恢复 out 输出 */
        {
            jpeg_out_dma_resume((uint32_t)mjpeg.outbuf
            [mjpeg.outbuf_write_ptr].buf, mjpeg.yuvblk_size);
            /* 继续下一次 DMA 传输 */
            mjpeg.outdma_pause = 0;
        }

        if (mjpeg.state == JPEG_STATE_ERROR)         /* 解码出错,直接退出 */
        {
            break;
        }

        if (mjpeg.state == JPEG_STATE_FINISHED)     /* 解码结束了,检查是否异常结束 */
        {
            if (mjpeg.yuvblk_curheight < mjpeg.Conf.ImageHeight)
            {
                if (mjpeg.Conf.ImageHeight > (mjpeg.yuvblk_curheight + 16))
                /* 数据异常,直接退出 */
                {
                    mjpeg.state = JPEG_STATE_ERROR;      /* 标记错误 */
                    printf("early finished!\r\n");
                    break;
                }
            }
        }

        if (g_mjpeg_fileover)    /* 文件读完了,及时退出,防止死循环 */
        {
            timecnt++;

            if (mjpeg.state == JPEG_STATE_NOHEADER)break; /* 解码 JPEG 头失败了 */

            if (timecnt > 0X3FFFF)break;                  /* 超时退出 */
        }
    }

    if (mjpeg.state == JPEG_STATE_FINISHED)               /* 解码完成了 */
    {
        mjpeg_fill_color(g_img_offx, g_img_offy, mjpeg.Conf.ImageWidth,
                         mjpeg.Conf.ImageHeight, p_rgb565buf);
    }
    return 0;
}
```

该函数是解码 jpeg 的主要函数，解码步骤参见第五十八章相关内容。解码后利用 DMA2D 将 YUV 转换成 RGB565 数据，存放在 p_rgb565buf（MCU 屏/RGB 竖屏）/RGB 屏显存（RGB 横屏）里面，然后通过 mjpeg_fill_color 函数，将 RGB565 数据显示到 LCD 屏幕上。

2. APP 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。APP 驱动源码包括两个文件：videoplayer.c 和 videoplayer.h。

videoplayer.h 头文件有两个宏定义和函数声明，具体请看源码。下面来看到 videoplayer.c 文件中，播放一个 MJPEG 文件函数，其定义如下：

```
/**
 * @brief       播放一个 MJPEG 文件
 * @param       pname    : 文件名
 * @retval      执行结果
 *   @arg       KEY0_PRES , 下一个视频
 *   @arg       KEY1_PRES , 上一个视频
```

```
 *   @arg       其他       , 错误
 */
uint8_t video_play_mjpeg(uint8_t *pname)
{
    uint8_t *framebuf;      /* 视频解码 buf */
    uint8_t *pbuf;          /* buf 指针 */
    FIL *favi;
    uint8_t  res = 0;
    uint8_t i = 0;
    uint16_t offset = 0;
    uint32_t nr;
    uint8_t key;
    uint8_t saisavebuf;

    for (i = 0; i < AVI_AUDIO_BUF_NUM; i++)
    {
        p_avi_sai_buf[i] = mymalloc(SRAMIN, AVI_AUDIO_BUF_SIZE);
        /* 申请音频内存 */

        if (p_avi_sai_buf[i] == NULL)           /* 申请失败, 直接退出 */
        {
            break;
        }

        memset(p_avi_sai_buf[i], 0, AVI_AUDIO_BUF_SIZE); /* 数据清零 */
    }

    favi = (FIL *)mymalloc(SRAMIN, sizeof(FIL));          /* 申请 favi 内存 */
    framebuf = mymalloc(SRAMIN, AVI_VIDEO_BUF_SIZE) ;     /* 申请视频 buf */

    if (!framebuf)
    /* 只要最后这个视频 buf 申请失败, 前面的申请失不失败都不重要, 总之就是失败了 */
    {
        printf("memory error!\r\n");
        res = 0XFF;
    }

    while (res == 0)
    {
        res = f_open(favi, (char *)pname, FA_READ);

        if (res == 0)
        {
            pbuf = framebuf;
            res = f_read(favi, pbuf, AVI_VIDEO_BUF_SIZE, &nr);  /* 开始读取 */

            if (res)
            {
                printf("fread error:%d\r\n", res);
                break;
            }

            /* 开始 avi 解析 */
            res = avi_init(pbuf, AVI_VIDEO_BUF_SIZE);    /* avi 解析 */


            if (res || g_avix.Width > lcddev.width)
            {
                printf("avi err:%d,width:%d\r\n", res, g_avix.Width);
                res = KEY0_PRES;
                break;
            }

            video_info_show(&g_avix);
            btim_tim7_int_init(g_avix.SecPerFrame / 100 - 1, 20000 - 1);
```

```
        /* 10Khz 计数频率,加 1 是 100us */
        offset = avi_srarch_id(pbuf, AVI_VIDEO_BUF_SIZE, "movi");
        /* 寻找 movi ID */
        avi_get_streaminfo(pbuf + offset + 4);  /* 获取流信息 */
        f_lseek(favi, offset + 12); /* 跳过标志 ID,读地址偏移到流数据开始处 */

        if (lcddev.height <= g_avix.Height)
        {
            res = mjpeg_init((lcddev.width - g_avix.Width) / 2,
            (lcddev.height-g_avix.Height)/2,g_avix.Width,g_avix.Height);
            /* JPG 解码初始化 */
        }
        else
        {
            res = mjpeg_init((lcddev.width - g_avix.Width) / 2, 110 +
            (lcddev.height-110-
            g_avix.Height)/2,g_avix.Width,g_avix.Height);
             /* JPG 解码初始化 */
        }

        if (g_avix.SampleRate)       /* 有音频信息,才初始化 */
        {
            es8388_sai_cfg(0, 3);    /* 飞利浦标准,16 位数据长度 */
            sai1_saia_init(SAI_MODEMASTER_TX, SAI_CLOCKSTROBING_RISINGEDGE,
                           SAI_DATASIZE_16);  /* 设置 SAI,主发送,16 位数据 */
            sai1_samplerate_set(g_avix.SampleRate);          /* 设置采样率 */
            sai1_tx_dma_init(p_avi_sai_buf[0], p_avi_sai_buf[1],
                             g_avix.AudioBufSize / 2, 1);/* TX DMA 16 位宽 */
            sai1_tx_callback = audio_sai_dma_callback;
            /* 回调函数指向 audio_sai_dma_callback */

            g_avi_sai_playbuf = 0;
            saisavebuf = 0;
            sai1_play_start();  /* 开启 SAI 播放 */
        }

        while (1)    /* 播放循环 */
        {
            if (g_avix.StreamID == AVI_VIDS_FLAG)        /* 视频流 */
            {
                pbuf = framebuf;
                f_read(favi, pbuf, g_avix.StreamSize + 8, &nr);
                /* 读入整帧+下一数据流 ID 信息 */
                res = mjpeg_decode(pbuf, g_avix.StreamSize);

                if (res)
                {
                    printf("decode error!\r\n");
                }

                while (g_avi_frameup == 0);
                /* 待时间到达(在 TIM 的中断里面设置为 1) */

                g_avi_frameup = 0;              /* 标志清零 */
                g_avi_frame++;
            }
            else      /* 音频流 */
            {
                video_time_show(favi, &g_avix);    /* 显示当前播放时间 */
                saisavebuf++;

                if (saisavebuf > 3)
                {
```

```
                    saisavebuf = 0;
                }

                do
                {
                    nr = g_avi_sai_playbuf;

                    if (nr)
                    {
                        nr--;
                    }
                    else
                    {
                        nr = 3;
                    }
                } while (saisavebuf == nr); /* 碰撞等待. */

                f_read(favi,p_avi_sai_buf[saisavebuf],g_avix.StreamSize
                       +8, &nr); /* 填充 p_avi_sai_buf */
                pbuf = p_avi_sai_buf[saisavebuf];
            }

            key = key_scan(0);

            if (key == KEY0_PRES || key == KEY2_PRES)
            /* KEY0/KEY2 按下,播放下一个/上一个视频 */
            {
                res = key;
                break;
            }
            else if (key == KEY1_PRES || key == WKUP_PRES)
            {
                sai1_play_stop();   /* 关闭音频 */
                video_seek(favi, &g_avix, framebuf);
                pbuf = framebuf;
                sai1_play_start();  /* 开启 DMA 播放 */
            }

            if (avi_get_streaminfo(pbuf + g_avix.StreamSize))
            /* 读取下一帧 流标志 */
            {
                printf("frame error \r\n");
                res = KEY0_PRES;
                break;
            }
        }

        sai1_play_stop();                                       /* 关闭音频 */
        TIM7->CR1 &= ~(1 << 0);                                 /* 关闭定时器 7 */
        lcd_set_window(0, 0, lcddev.width, lcddev.height);      /* 恢复窗口 */
        mjpeg_free();                                           /* 释放内存 */
        f_close(favi);                                          /* 关闭文件 */
    }
}

/* 释放内存 */
for (i = 0; i < AVI_AUDIO_BUF_NUM; i++)
{
    myfree(SRAMIN, p_avi_sai_buf[i]);
}

myfree(SRAMIN, framebuf);
myfree(SRAMIN, favi);

return res;
```

```
}
```

该函数用来播放一个 avi 视频文件（mjpg 编码），解码过程就是根据前面我们在 63.1 节最后所介绍的步骤进行。不过在这里，我们的音频播放用了 4 个 buf，以提高解码的流畅度。其他代码，我们就不介绍了，请大家参考本例程源码。

3. main.c 代码

下面是 main 函数，其定义如下：

```
int main(void)
{
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化 */
    usmart_init(200);                       /* 初始化 USMART */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    key_init();                             /* 初始化 KEY */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    norflash_init();                        /* 初始化 norflash */
    btim_timx_int_init(10000 - 1,20000 - 1); /* 10Khz 计数,1 秒钟中断一次 */

    my_mem_init(SRAMIN);                    /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                    /* 初始化外部内存池(SDRAM) */
    my_mem_init(SRAM12);                    /* 初始化 SRAM12 内存池(SRAM1+SRAM2) */
    my_mem_init(SRAM4);                     /* 初始化 SRAM4 内存池(SRAM4) */
    my_mem_init(SRAMDTCM);                  /* 初始化 DTCM 内存池(DTCM) */
    my_mem_init(SRAMITCM);                  /* 初始化 ITCM 内存池(ITCM) */

    exfuns_init();                          /* 为 fatfs 相关变量申请内存 */
    f_mount(fs[0], "0:", 1);                /* 挂载 SD 卡 */
    f_mount(fs[1], "1:", 1);                /* 挂载 SPI FLASH */
    f_mount(fs[2], "2:", 1);                /* 挂载 NAND FLASH */

    while (fonts_init())                    /* 检查字库 */
    {
        lcd_show_string(30, 90, 200, 16, 16, "Font error!", RED);
        delay_ms(200);
        lcd_fill(20, 90, 200 + 20, 90 + 16, WHITE);
        delay_ms(200);
    }

    es8388_init();                          /* ES8388 初始化 */
    es8388_adda_cfg(1, 0);                  /* 开启 DAC 关闭 ADC */
    es8388_input_cfg(0);
    es8388_output_cfg(1, 1);                /* DAC 选择通道输出 */
    es8388_hpvol_set(10);                   /* 设置耳机音量 */
    es8388_spkvol_set(25);                  /* 设置喇叭音量 */

    text_show_string(30, 30, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
    text_show_string(30, 50, 200, 16, "视频播放器实验", 16, 0, RED);
    text_show_string(30, 70, 200, 16, "正点原子@ALIENTEK", 16, 0, RED);
    text_show_string(30, 90, 200, 16, "KEY0:NEXT   KEY2:PREV", 16, 0, RED);
    text_show_string(30, 110, 200, 16, "KEY_UP:FF   KEY1: REW", 16, 0, RED);

    delay_ms(1500);

    while (1)
```

```
    {
        video_play();
    }
}
```

该函数代码比较简单，我们就不多说了。最后，为了提高速度，我们对编译器进行设置，选择使用-O2 优化，从而优化代码，提高速度（但调试效果不好，建议调试时设置为-O0），编译器设置如图 63.3.2.1 所示：

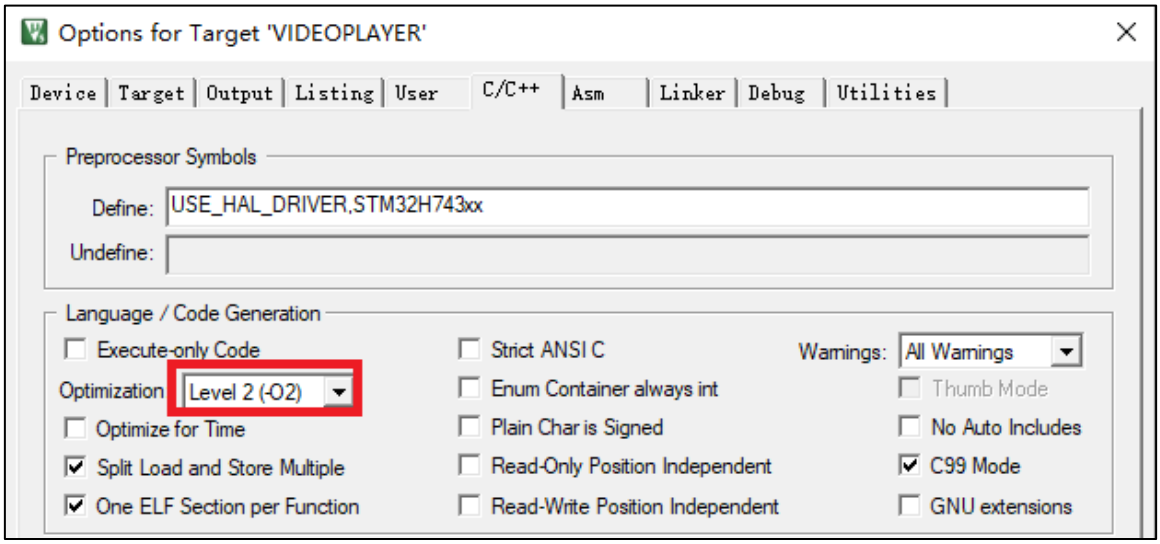


图 63.3.2.1 编译器优化设置

设置完后，重新编译即可。至此，本实验的软件设计部分结束。

63.4 下载验证

本章，我们例程仅支持 MJPG 编码的 avi 格式视频，且音频必须是 PCM 格式，另外视频分辨率不能大于 LCD 分辨率。要满足这些要求，现成的 avi 文件是很难找到的，所以我们需要用软件，将通用视频（任何视频都可以）转换为我们需要的格式，这里我们通过：狸窝全能视频转换器，这款软件来实现（路径：光盘：6，软件资料→软件→视频转换软件→狸窝全能视频转换器.exe）。安装完后，打开，然后进行相关设置，软件设置如图 63.4.1 和 63.4.2 所示：

图 63.4.1 软件启动界面和设置

图 63.4.2 高级设置

首先，如图 63.4.1 所示，点击 1 处，添加视频，找到你要转换的视频，添加进来。有的视频可能有独立字幕，比如我们打开的这个视频就有，所以在 2 处选择下字幕（如果没有的，可以忽略此步）。然后在 3 处，点击▼图标，选择预制方案：AVI-Audio-Video Interleaved(*.avi)，即生成.avi 文件，然后点击 4 处的高级设置按钮，进入 63.4.2 所示的界面，设置详细参数如下：

视频编码器：选择 MJPEG。本例程仅支持 MJPG 视频解码，所以选择这个编码器。

视频尺寸：480x272。这里得根据所用 LCD 分辨率来选择，假设我们用 800*480 的 4.3 寸电容屏模块，则这里最大可以设置：272x480。PS:如果是 2.8 屏，最大宽度只能是 240）。

比特率：1000。这里设置越大，视频质量越好，解码就越慢（可能会卡），我们设置为 1000，可以得到比较好的视频质量，同时也不怎么会卡。

帧率：10。即每秒钟 10 帧。对于 480*272 的视频，本例程最高能播放 30 帧左右的视频，如果要想提高帧率，有几个办法：1，降低分辨率；2，降低比特率；3，降低音频采样率。

音频编码器：PCMS16LE。本例程只支持 PCM 音频，所以选择音频编码器为这个。

采样率：这里设置为 11025，即 11.025Khz 的采样率。这里越高，声音质量越好，不过，转换后的文件就越大，而且视频可能会卡。

其他设置，采用默认的即可。设置完以后，点击确定，即可完成设置。

点击图 63.4.1 的 5 处的文件夹图标，设置转换后视频的输出路径，这里我们设置到了桌面，这样转换后的视频，会保存在桌面。最后，点击图中 6 处的按钮，即可开始转换了，如图 63.4.3 所示：

图 63.4.3 正在转换

等转换完成后，将转换后的.avi 文件，拷贝到 SD 卡→VIDEO 文件夹下，然后插入开发板的 SD 卡接口，就可以开始测试本章例程了。

将程序下载到开发板后，程序先检测字库，只有字库已经更新才可以继续执行后面的程序。字库已更新，就可以看到 LCD 首先显示一些实验相关的信息，如图 63.4.4 所示：

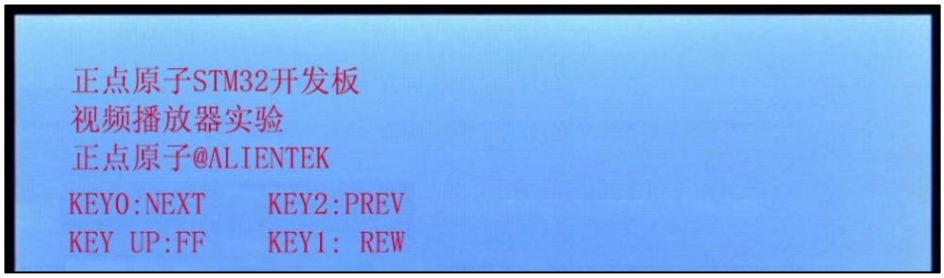


图 63.4.4 显示实验相关信息

显示了上图的信息后，检测 SD 卡的 VIDEO 文件夹，并查找 avi 视频文件，在找到有效视频文件后，便开始播放视频，如图 63.4.5 所示：

图 63.4.5 视频播放中

可以看到，屏幕显示了文件名、索引、声道数、采样率、帧率和播放时间等参数。然后，我们按 KEY0/KEY2，可以切换到下一个/上一个视频，按 KEY_UP/KEY1，可以快进/快退。

至此，本例程介绍就结束了。本实验，我们在开发板上实现了视频播放，体现了 STM32H743 强大的处理能力。

本例程只支持竖屏宽度的分辨率解码（比如 800*480 的屏，最大只支持 480 宽度的视频解码），如果想要支持更大分辨率的视频解码，则必须使用横屏模式，需要在本例程源码的基础上稍作修改（参见综合实验的视频播放器功能）。

附 STM32H743 硬件 JPEG 视频解码性能：

对 480*272 及以下分辨率，可达 30 帧

对 800*480 分辨率，可达 20 帧

对 1024*600 分辨率，可达 10 帧

最后提醒大家，转换的视频分辨率，一定要根据自己的 LCD 设置，不能超过 LCD 的尺寸!!否则无法播放（可能只听到声音，看不到图像）。

第六十四章 FPU 测试(Julia 分形)实验

本章我们将学习如何开启 STM32H743 的硬件 FPU，并对比使用硬件 FPU 和不使用硬件 FPU 的速度差别，以体现硬件 FPU 的优势。

本章分为如下几个小节：

64.1 FPU&Julia 分形简介
64.2 硬件设计
64.3 程序设计
64.4 下载验证

64.1 FPU&Julia 分形简介

本节将分别介绍 STM32H743 的 FPU 和 Julia 分形。

64.1.1 FPU 简介

FPU 即浮点运算单元（Float Point Unit）。浮点运算，对于定点 CPU（没有 FPU 的 CPU）来说必须要按照 IEEE-754 标准的算法来完成运算，是相当耗费时间的。而对于有 FPU 的 CPU 来说，浮点运算则只是几条指令的事情，速度相当快。

STM32H743 属于 Cortex M7 架构，带有 32 位双精度硬件 FPU，支持浮点指令集，相对于 Cortex M0 和 Cortex M3 等，高出数十倍甚至上百倍的运算性能。

STM32H743 硬件上要开启 FPU 是很简单的，通过一个叫：协处理器控制寄存器（CPACR）的寄存器设置即可开启 STM32H743 的硬件 FPU，该寄存器各位描述如图 64.1.1.1 所示：

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved								CP11		CP10		Reserved			
								rw		rw					
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved															

图 64.1.1.1 协处理器控制寄存器（CPACR）各位描述

这里我们就是要设置 CP11 和 CP10 这 4 个位，复位后，这 4 个位的值都为 0，此时禁止访问协处理器（禁止了硬件 FPU），我们将这 4 个位都设置为 1，即可完全访问协处理器（开启硬件 FPU），此时便可以使用 STM32H743 内置的硬件 FPU 了。CPACR 寄存器这 4 个位的设置，我们在 system_stm32h7xx_c 文件里面开启，代码如下：

```
void SystemInit (void)
{
  /* 省略部分代码 */
  /* FPU settings ------------------------------------------------------------*/
  #if (__FPU_PRESENT == 1) && (__FPU_USED == 1)
    SCB->CPACR |= ((3UL << (10*2))|(3UL << (11*2)));
  /* set CP10 and CP11 Full Access */
  #endif
  /* 省略部分代码 */
}
```

此部分代码是系统初始化函数的部分内容，功能就是设置 CPACR 寄存器的 20~23 位为 1，以开启 STM32H7 的硬件 FPU 功能。从程序可以看出，只要我们定义了全局宏定义标识符__FPU_PRESENT 以及__FPU_USED 为 1，那么就可以开启硬件 FPU。其中宏定义标识符__FPU_PRESENT 用来确定处理器是否带 FPU 功能，标识符__FPU_USED 用来确定是否开启 FPU 功能。

实际上，因为 H7 是带 FPU 功能的，所以在我们的 stm32h743xx.h 头文件里面，我们默认是定义了__FPU_PRESENT 为 1。大家可以打开文件搜索即可找到下面一行代码：

```
#define __FPU_PRESENT             1         /* FPU present */
```

但是，仅仅只是说明处理器有 FPU 功能是不够的，我们还需要开启 FPU 功能。开启 FPU

有两种方法，第一种是直接在头文件 stm32h743xx.h 中定义宏定义标识符__FPU_USED 的值为 1。也可以直接在 MDK 编译器上面设置，我们在 MDK5 编译器里面，点击按钮，然后在 Target 选项卡里面，设置 Not Used 为 Use Double Precision，如图 64.1.1.2 所示：

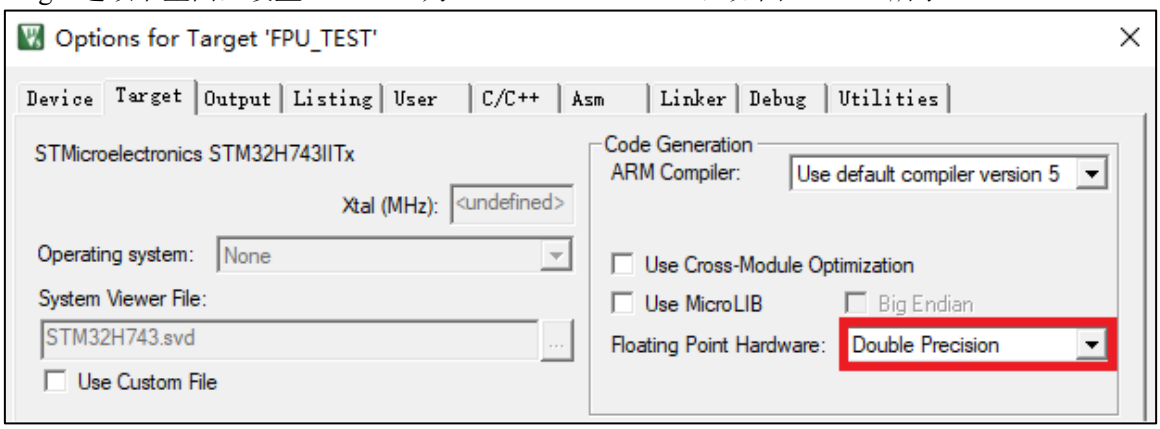


图 64.1.1.2 编译器开启硬件 FPU 选型

经过这个设置，编译器会自动加入标识符__FPU_USED 为 1。这样遇到浮点运算就会使用硬件 FPU 相关指令，执行浮点运算，从而大大减少计算时间。

最后，总结下 STM32H7 硬件 FPU 使用的要点：

1， 设置 CPACR 寄存器 bit20~23 为 1，使能硬件 FPU（参考 SystemInit 函数开头部分）。

2， MDK 编译器 Target 选项卡中 Not Used 选项设置为：Use Double Precision。

经过这两步设置，我们的编写的浮点运算代码，即可使用 STM32H7 的硬件 FPU 了，可以大大加快浮点运算速度。

64.1.2 Julia 分形简介

Julia 分形即 Julia 集，它最早由法国数学家 Gaston Julia 发现，因此命名为 Julia（朱利亚）集。Julia 集合的生成算法非常简单：对于复平面的每个点，我们计算一个定义序列的发散速度。该序列的 Julia 集计算公式为：

$$z_{n}+1 = z_{n}^{2} + c$$

针对复平面的每个 x + i.y 点，我们用 $c = c_x + i.c_y$ 计算该序列：

$$x_{n+1} + i.y_{n+1} = x_{n}^{2} - y_{n}^{2} + 2.i.x_{n}.y_{n} + c_x + i.c_y$$

$$x_{n}+1 = x_{n}^{2} - y_{n}^{2} + c_x \text{ 且 } y_{n+1} = 2.x_{n}.y_{n} + c_y$$

一旦计算出的复值超出给定圆的范围（数值大小大于圆半径），序列便会发散，达到此限值时完成的迭代次数与该点相关。随后将该值转换为颜色，以图形方式显示复平面上各个点的分散速度。

经过给定的迭代次数后，若产生的复值保持在圆范围内，则计算过程停止，并且序列也不发散，本例程生成 Julia 分形图片的代码如下：

```
#define     ITERATION           128             /* 迭代次数 */
#define     REAL_CONSTANT       0.285f          /* 实部常量 */
#define     IMG_CONSTANT        0.01f           /* 虚部常量 */

/**
 * @brief       产生 Julia 分形图形
 * @param       size_x      : 屏幕 x 方向的尺寸
 * @param       size_y      : 屏幕 y 方向的尺寸
 * @param       offset_x    : 屏幕 x 方向的偏移
 * @param       offset_y    : 屏幕 y 方向的偏移
 * @param       zoom        : 缩放因子
 * @retval      无
 */
void julia_generate_fpu(uint16_t size_x, uint16_t size_y, uint16_t offset_x,
                        uint16_t offset_y, uint16_t zoom)
```

```
{
    uint8_t i;
    uint16_t x, y;
    float tmp1, tmp2;
    float num_real, num_img;
    float radius;

    for (y = 0; y < size_y; y++)
    {
        for (x = 0; x < size_x; x++)
        {
            num_real = y - offset_y;
            num_real = num_real / zoom;
            num_img = x - offset_x;
            num_img = num_img / zoom;
            i = 0;
            radius = 0;

            while ((i < ITERATION - 1) && (radius < 4))
            {
                tmp1 = num_real * num_real;
                tmp2 = num_img * num_img;
                num_img = 2 * num_real * num_img + IMG_CONSTANT;
                num_real = tmp1 - tmp2 + REAL_CONSTANT;
                radius = tmp1 + tmp2;
                i++;
            }
            LCD->LCD_RAM = g_color_map[i]; /* 绘制到屏幕 */
        }
    }
}
```

这种算法非常有效地展示了 FPU 的优势：无需修改代码，只需在编译阶段激活或禁止 FPU（在 MDK Code Generation 的 Float Point Hardware 选项里面设置：Double Precision /Not Used），即可测试使用硬件 FPU 和不使用硬件 FPU 的差距。

注意，是该函数将颜色数据填充到 LCD 的时候，根据 MCU 屏还是 RGB 屏，做了不同的处理：MCU 屏可以直接写 LCD_RAM，将颜色显示到 LCD 上面；而 RGB 屏，则需要先缓存到 lcdbuf，然后通过 DMA2D 一次性填充，以提高速度。

64.2 硬件设计

1. 例程功能

1、实验开机后，根据迭代次数生成颜色表（RGB565），然后计算 Julia 分形，并显示到 LCD 上面。同时，程序开启了定时器 6，用于统计一帧所要的时间（ms），在一帧 Julia 分形图片显示完成后，程序会显示运行时间、当前是否使用 FPU 和缩放因子（zoom）等信息，方便观察对比。KEY0/KEY2 用于调节缩放因子，KEY_UP 用于设置自动缩放，还是手动缩放。

2、LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯

LED：LED0 - PB1

2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)

3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)

4）独立按键：KEY0 - PH3、KEY1 - PH2、WK_UP - PA0

5）FPU(浮点计算单元)

64.3 程序设计

64.3.1 程序流程图

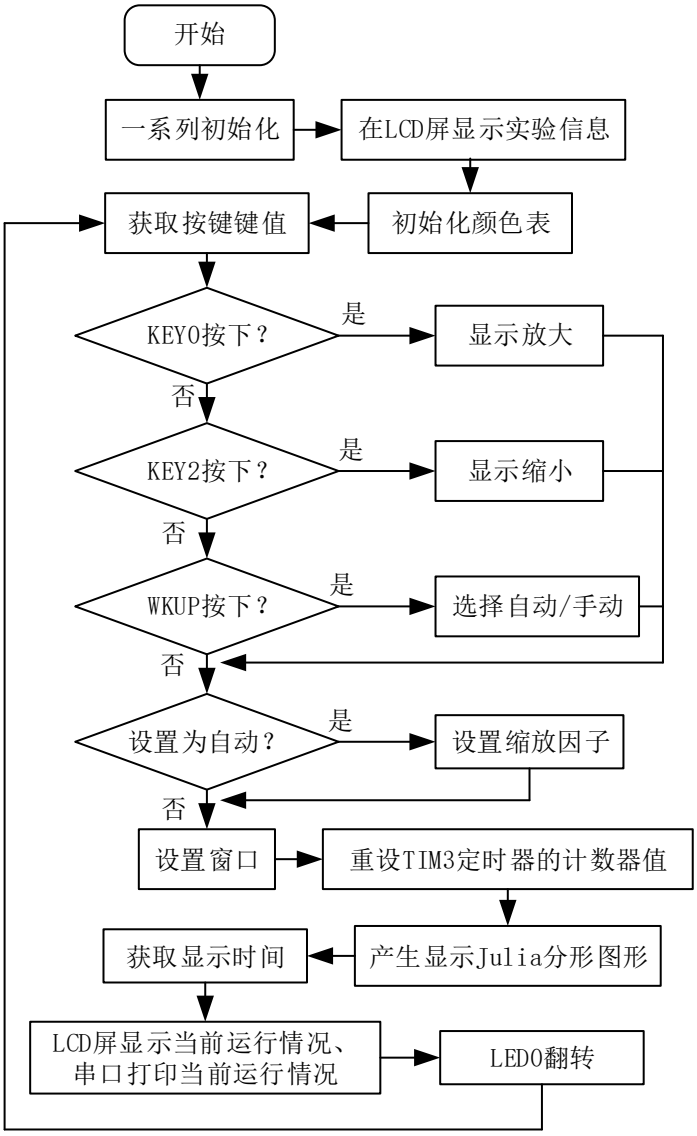


图 64.3.1.1 FPU 测试(Julia 分形)实验程序流程图

64.3.2 程序解析

本实验例程，分成两个工程：

1，实验 52_1 FPU 测试(Julia 分形)实验_开启硬件 FPU

2，实验 52_2 FPU 测试(Julia 分形)实验_关闭硬件 FPU

这两个工程的代码一模一样，只是前者使用硬件 FPU 计算 Julia 分形集（MDK 设置 Double Precision），后者使用 IEEE-754 标准计算 Julia 分形集（MDK 设置 Not Used）。由于两个工程代码一模一样，我们这里仅介绍其中一个：实验 52_1 FPU 测试(Julia 分形)实验_开启硬件 FPU。

1. TIMER 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。由于要统计帧时间，所以我们需要用到 TIMER 驱动代码的 btim.c 和 btim.h 文件。

btim.h 文件不需要做任何修改，只要在 btim.c 文件中，修改定时器更新中断回调函数即可，

具体修改后的代码如下：

```
extern uint8_t g_timeout ;              /* 在 main.c 里面定义 */

/**
 * @brief       定时器更新中断回调函数
 * @param       htim:定时器句柄指针
 * @retval      无
 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim ->Instance==BTIM_TIMX_INT)
    {
        g_timeout++;
    }
}
```

在定时器更新中断回调函数中，我们只是需要让 g_timeout 自加就行，最后把 g_timeout 变量的值*定时器更新中断的周期就可以知道帧时间了。当然在程序上，我们还要声明 g_timeout 变量。

2. main.c 代码

下面是 main.c 的程序，具体如下：

```
/* FPU 模式提示 */
#if __FPU_USED==1
#define SCORE_FPU_MODE                  "FPU On"
#else
#define SCORE_FPU_MODE                  "FPU Off"
#endif

#define     ITERATION           128             /* 迭代次数 */
#define     REAL_CONSTANT       0.285f          /* 实部常量 */
#define     IMG_CONSTANT        0.01f           /* 虚部常量 */

/* 颜色表 */
uint16_t g_color_map[ITERATION];

/* 缩放因子列表 */
const uint16_t zoom_ratio[] =
{
    120, 110, 100, 150, 200, 275, 350, 450,
    600, 800, 1000, 1200, 1500, 2000, 1500,
    1200, 1000, 800, 600, 450, 350, 275, 200,
    150, 100, 110,
};

/**
 * @brief       初始化颜色表
 * @param       clut      : 颜色表指针
 * @retval      无
 */
void julia_clut_init(uint16_t *clut)
{
    uint32_t i = 0x00;
    uint16_t  red = 0, green = 0, blue = 0;
    for (i = 0; i < ITERATION; i++) /* 产生颜色表 */
    {
        /* 产生 RGB 颜色值 */
        red = (i * 8 * 256 / ITERATION) % 256;
        green = (i * 6 * 256 / ITERATION) % 256;
        blue = (i * 4 * 256 / ITERATION) % 256;

        /* 将 RGB888,转换为 RGB565 */
```

```
        red = red >> 3;
        red = red << 11;
        green = green >> 2;
        green = green << 5;
        blue = blue >> 3;
        clut[i] = red + green + blue;
    }
}

/* RGB LCD 缓存*/
uint16_t g_lcdbuf[800];

/**
 * @brief        产生 Julia 分形图形
 * @param       size_x      : 屏幕 x 方向的尺寸
 * @param       size_y      : 屏幕 y 方向的尺寸
 * @param       offset_x    : 屏幕 x 方向的偏移
 * @param       offset_y    : 屏幕 y 方向的偏移
 * @param       zoom        : 缩放因子
 * @retval      无
 */
void julia_generate_fpu(uint16_t size_x, uint16_t size_y, uint16_t offset_x,
                        uint16_t offset_y, uint16_t zoom)
{
    uint8_t i;
    uint16_t x, y;
    float tmp1, tmp2;
    float num_real, num_img;
    float radius;
    for (y = 0; y < size_y; y++)
    {
        for (x = 0; x < size_x; x++)
        {
            num_real = y - offset_y;
            num_real = num_real / zoom;
            num_img = x - offset_x;
            num_img = num_img / zoom;
            i = 0;
            radius = 0;
            while ((i < ITERATION - 1) && (radius < 4))
            {
                tmp1 = num_real * num_real;
                tmp2 = num_img * num_img;
                num_img = 2 * num_real * num_img + IMG_CONSTANT;
                num_real = tmp1 - tmp2 + REAL_CONSTANT;
                radius = tmp1 + tmp2;
                i++;
            }
            if (lcdltdc.pwidth != 0)
            {
               g_lcdbuf[lcddev.width - x - 1] = g_color_map[i];
               /* 保存颜色值到 lcdbuf */
            }
            else
            {
                LCD->LCD_RAM = g_color_map[i];            /* 绘制到屏幕 */
            }
        }
        if (lcdltdc.pwidth != 0)
        {
            ltdc_color_fill(0, y, lcddev.width - 1, y, g_lcdbuf);
            /* DM2D 填充 */
        }
    }
}
```

```
uint8_t g_timeout;

int main(void)
{
    uint8_t key;
    uint8_t i = 0;
    uint8_t autorun = 0;
    float time;
    char buf[50];

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    key_init();                             /* 初始化按键 */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    btim_timx_int_init(65535, 20000 - 1);  /* 10Khz 计数频率,最大计时 6.5 秒超出 */
    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "FPU TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY0:+ KEY1:-", RED);/*显示提示信息*/
    lcd_show_string(30, 130, 200, 16, 16, "KEY_UP:AUTO/MANUL", RED);
    delay_ms(500);
    julia_clut_init(g_color_map);       /* 初始化颜色表 */
    while (1)
    {
        key = key_scan(0);
        switch (key)
        {
            case KEY0_PRES:
                i++;
                if (i > sizeof(zoom_ratio) / 2 - 1)i = 0; /* 限制范围 */
                break;
            case KEY2_PRES:
                if (i)i--;
                else i = sizeof(zoom_ratio) / 2 - 1;
                break;
            case WKUP_PRES:
                autorun = !autorun;  /* 自动/手动 */
                break;
            default:break;
        }
        if (autorun == 1)            /* 自动时,自动设置缩放因子 */
        {
            i++;
            LED1(0);
            if (i > sizeof(zoom_ratio) / 2 - 1)
            {
                i = 0;               /* 限制范围 */
            }
            else
            {
                LED1(1);
            }
        }
        lcd_set_window(0, 0, lcddev.width, lcddev.height);  /* 设置窗口 */
        lcd_write_ram_prepare();
        BTIM_TIMX_INT->CNT = 0;  /* 重设定时器的计数器值 */
        g_timeout = 0;
```

```
        julia_generate_fpu(lcddev.width, lcddev.height, lcddev.width / 2,
                           lcddev.height / 2, zoom_ratio[i]);
        time = BTIM_TIMX_INT->CNT + (uint32_t)g_timeout * 65536;
        sprintf(buf, "%s: zoom:%d  runtime:%0.1fms\r\n", SCORE_FPU_MODE,
                zoom_ratio[i], time / 10);
        lcd_show_string(5, lcddev.height - 5 - 12, lcddev.width - 5,
                        12, 12, buf, RED); /* 显示当前运行情况 */
        printf("%s", buf);                 /* 输出到串口 */
        LED0_TOGGLE();
    }
}
```

这部分程序，总共 3 个函数：julia_clut_init、julia_generate_fpu 和 main 函数。

julia_clut_init 函数，该函数用于初始化颜色表，该函数根据迭代次数（ITERATION）计算出颜色表，这些颜色值将显示在 TFTLCD 上。

julia_generate_fpu 函数，该函数根据给定的条件计算 Julia 分形集，当迭代次数大于等于 ITERATION 或者半径大于等于 4 时，结束迭代，并在 TFTLCD 上面显示迭代次数对应的颜色值，从而得到漂亮的 Julia 分形图。我们可以通过修改 REAL_CONSTANT 和 IMG_CONSTANT 这两个常量的值来得到不同的 Julia 分形图。

main 函数，完成我们在 64.2 节所介绍的实验功能，代码比较简单。这里我们用到一个缩放因子表：zoom_ratio，里面存储了一些不同的缩放因子，方便演示效果。

为了提高速度，我们还可以在 MDK 里面选择使用-O2 优化，优化代码速度，本例程代码就介绍到这里。

再次提醒大家：本例程两个代码（实验 52_1 和实验 52_2）程序是完全一模一样的，他们的区别就是 MDK→Options for Target ‘Target1’ →Target 选项卡→Floating Point Hardware 的设置不一样，当设置 Double Precision 时，使用硬件 FPU；当设置 Not Used 时，不使用硬件 FPU。分别下载这两个代码，通过屏幕显示的 runtime 时间，即可看出速度上的区别。

64.4 下载验证

将实验 52_1 的程序下载到开发板后，可以看到 LCD 首先显示一些实验相关的信息，然后开始显示 Julia 分形图，并显示相关参数，如图 64.4.1 所示：

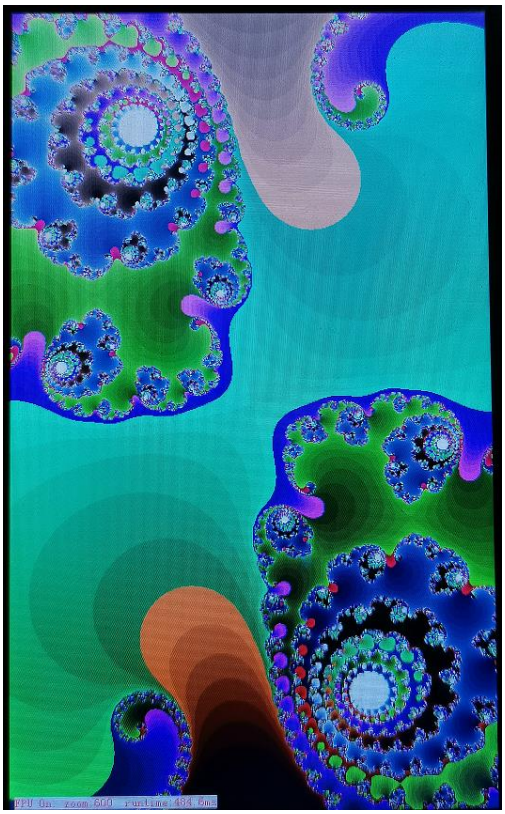


图 64.4.1 Julia 分形显示效果

实验 52_1 是开启了硬件 FPU 的，所以显示 Julia 分形图片速度比较快。除了 LCD 屏幕，还可以通过串口调试助手观察相关信息，如图 64.4.2 所示：

图 64.4.2 Julia 分形显示效果（开启硬件 FPU）

下面是关闭硬件 FPU 的运行情况，如图 64.4.3 所示：

图 64.4.3 Julia 分形显示效果（关闭硬件 FPU）

对比图 64.4.2 和图 64.4.3 知道，关闭硬件 FPU 会比开启硬件 FPU，同等情况下慢 6 倍左右，这充分体现了 STM32H743 硬件 FPU 的优势。

第六十五章 DSP 测试实验

上一章，我们测试了 STM32H743 的硬件 FPU。STM32H743 除了集成硬件 FPU 外，还支持多种 DSP 指令集。同时 ST 还提供了一整套 DSP 库方便我们工程中开发应用。

本章，我们将指导大家入门 STM32H743 的 DSP，手把手教大家搭建 DSP 库测试环境，同时通过对 DSP 库中的几个基本数学功能函数和 FFT 快速傅里叶变换函数的测试，让大家对 STM32H743 的 DSP 库有个基本的了解。

本章分为如下几个小节：

65.1 DSP 简介与环境搭建
65.2 硬件设计
65.3 程序设计
65.4 下载验证

65.1 DSP 简介与环境搭建

本节将分两个部分：1，STM32H743 DSP 简介；2，DSP 库运行环境搭建。

65.1.1 STM32H743 DSP 简介

STM32H743 采用 Cortex-M7 内核，相比 Cortex-M3 系列除了内置硬件 FPU 单元，在数字信号处理方面还增加了 DSP 指令集，支持诸如单周期乘加指令（MAC），优化的单指令多数据指令（SIMD），饱和算数等多种数字信号处理指令集。相比 Cortex-M3，Cortex-M7 在数字信号处理能力方面得到了大大的提升。Cortex-M7 执行所有的 DSP 指令集都可以在单周期内完成，而 Cortex-M3 需要多个指令和多个周期才能完成同样的功能。

接下来来介绍 Cortex-M7 的两个 DSP 指令：MAC 指令（32 位乘法累加）和 SIMD 指令。

32 位乘法累加（MAC）单元包括新的指令集，能够在单周期内完成一个 32×32+ 64→64 的操作或两个 16×16 的操作，其计算能力，如表 65.1.1.1 所示：

计算	指令	周期
16 × 16 = 32	SMULBB , SMULBT , SMULTB , SMULTT	1
16 × 16 + 32 = 32	SMLABB , SMLABT , SMLATB , SMLATT	1
16 × 16 + 64 = 64	SMLALBB , SMLALBT , SMLALTB , SMLALTT	1
16 × 32 = 32	SMULWB , SMULWT	1
(16 × 32) + 32 = 32	SMLAWB , SMLAWT	1
(16 × 16) ± (16 × 16) = 32	SMUAD , SMUADX , SMUSD , SMUSDX	1
(16 × 16) ± (16 × 16) + 32 = 32	SMLAD , SMLADX , SMLSD , SMLSDX	1
(16 × 16) ± (16 × 16) + 64 = 64	SMLALD , SMLALDX , SMLSLD , SMLSLDX	1
32 × 32 = 32	MUL	1
32 ± (32 × 32) = 32	MLA , MLS	1
32 × 32 = 64	SMULL , UMULL	1
(32 × 32) + 64 = 64	SMLAL , UMLAL	1
(32 × 32) + 32 + 32 = 64	UMAAL	1
2 ± (32 × 32) = 32 (上)	SMMLA , SMMLAR , SMMLS , SMMLSR	1
(32 × 32) = 32 (上)	SMMUL , SMMULR	1

图 65.1.1.1 32 位乘法累加（MAC）单元的计算能力

Cortex-M7 支持 SIMD 指令集，这在 Cortex-M3/M0 系列是不可用的。上述表中的指令，有的属于 SIMD 指令。与硬件乘法器一起工作（MAC），使所有这些指令都能在单个周期内执行。

受益于 SIMD 指令的支持，Cortex-M4 处理器能在单周期内完成高达 32×32+64→64 的运算，为其他任务释放处理器的带宽，而不是被乘法和加法消耗运算资源。

比如一个比较复杂的运算：两个 16×16 乘法加上一个 32 位加法，如图 65.1.1.2 所示：

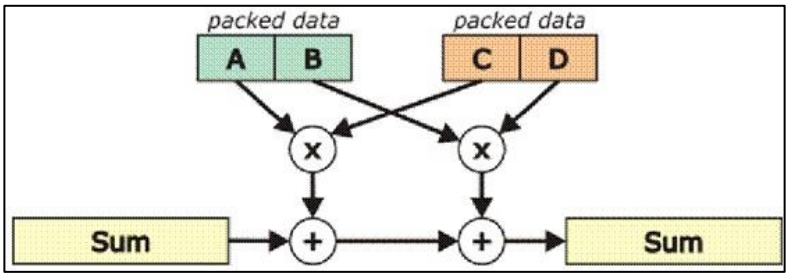


图 65.1.1.2 SUM 运算过程

以上图片所示的运算，即：SUM＝SUM＋（A* C）＋（B *D），在 STM32H743 上面，可以被编译成由一条单周期指令完成。

上面我们简单的介绍了 Cortex-M7 的 DSP 指令，接下来我们来介绍一下 STM32H7 的 DSP 库。STM32H7 的 DSP 库源码和测试实例在 ST 提供的 HAL 库：en.stm32cubeh7.zip 里面就有（该文件可以在 www.st.com 网站下载，搜索 STM32CubeH7 即可找到最新版本），该文件在：光盘→ 8，STM32 参考资料→1，STM32CubeH7 固件包文件夹里面，解压该文件，即可找到 ST 提供的 DSP 库，详细路径为：光盘→8，STM32 参考资料→ 1，STM32CubeH7 固件包→ STM32Cube_FW_H7_V1.9.0→ Drivers→CMSIS→DSP，该文件夹下目录结构如图 65.1.1.3 所示：

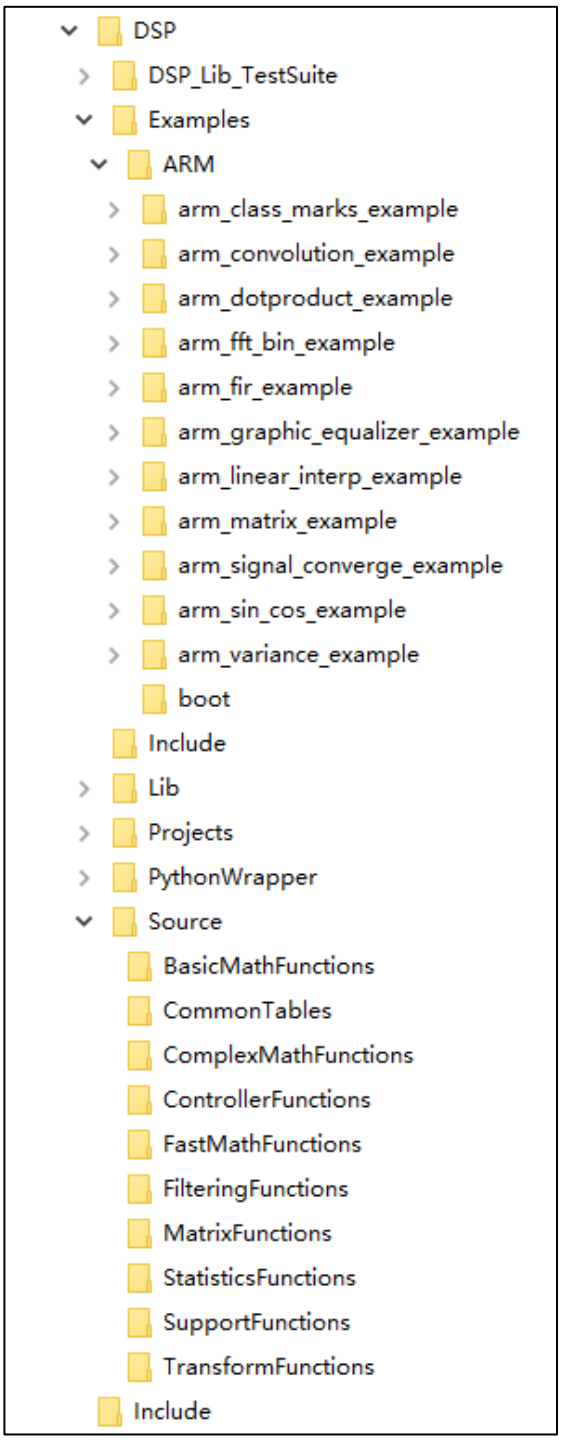


图 64.1.1.3 DSP 目录结构

DSP 源码包的 Source 文件夹是所有 DSP 库的源码，Examples 文件夹是相对应的一些测试实例。这些测试实例都是带 main 函数的，也就是拿到工程中可以直接使用。接下来我们一一讲解一下 Source 源码文件夹下面的子文件夹包含的 DSP 库的功能。

- **BasicMathFunctions**
  基本数学函数：提供浮点数的各种基本运算函数，如向量加减乘除等运算。
- **CommonTables**
  arm_common_tables.c 文件提供位翻转或相关参数表。arm_const_structs.c 文件提供一些常用的常量结构体，方便用户使用。
- **ComplexMathFunctions**
  复数学功能，如向量处理，求模运算。

- **ControllerFunctions**
  控制功能函数。包括正弦余弦，PID 电机控制，矢量 Clarke 变换，矢量 Clarke 逆变换。
- **FastMathFunctions**
  快速数学功能函数。提供了一种快速的近似正弦，余弦和平方根等相比 CMSIS 计算库要快的数学函数。
- **FilteringFunctions**
  滤波函数功能，主要为 FIR 和 LMS（最小均方根）等滤波函数。
- **MatrixFunctions**
  矩阵处理函数。包括矩阵加法、矩阵初始化、矩阵反、矩阵乘法、矩阵规模、矩阵减法、矩阵转置等函数。
- **StatisticsFunctions**
  统计功能函数。如求平均值、最大值、最小值、计算均方根 RMS、计算方差/标准差等。
- **SupportFunctions**
  支持功能函数，如数据拷贝，Q 格式和浮点格式相互转换，Q 任意格式相互转换。
- **TransformFunctions**
  变换功能。包括复数 FFT（CFFT）/复数 FFT 逆运算（CIFFT）、实数 FFT（RFFT）/实数 FFT 逆运算（RIFFT）、和 DCT（离散余弦变换）和配套的初始化函数。

所有这些 DSP 库代码合在一起是比较多的，因此，ST 为我们提了.lib 格式的文件，方便使用。这些.lib 文件就是由 Source 文件夹下的源码编译生成的，如果想看某个函数的源码，大家可以在 Source 文件夹下面查找。.lib 格式文件 HAL 库包路径：Drivers→CMSIS→Lib→ARM，针对 M7 的总共有 6 个.lib 文件，如下：

① arm_cortexM7b_math.lib (Cortex-M7 大端模式)
② arm_cortexM7l_math.lib (Cortex-M7 小端模式)
③ arm_cortexM7bfdp_math.lib (双精度浮点 Cortex-M7 大端模式)
④ arm_cortexM7lfdp_math.lib (双精度浮点 Cortex-M7 小端模式)
⑤ arm_cortexM7bfsp_math.lib (单精度浮点 Cortex-M7 大端模式)
⑥ arm_cortexM7lfsp_math.lib (单精度浮点 Cortex-M7 小端模式)

我们得根据所用 MCU 内核类型以及端模式来选择符合要求的.lib 文件，本章我们所用的 STM32H7 属于 CortexM7F 内核，双精度浮点小端模式，应选择：

arm_cortexM7lfdp_math.lib (双精度浮点 Cortex-M7 小端模式)。

对于 DSP 的子文件夹 Examples 下面存放的文件，是 ST 官方提供的一些 DSP 测试代码，提供简短的测试程序，方便上手，有兴趣的朋友可以根据需要自行测试。

65.1.2 DSP 库运行环境搭建

本节我们将讲解怎么搭建 DSP 库运行环境，只要运行环境搭建好了，使用 DSP 库里面的函数来做相关处理就非常简单了。本节，我们将以上一章例程（实验 52_1）为基础，搭建 DSP 运行环境。

在 MDK 里面搭建 STM32H743 的 DSP 运行环境(使用.lib 方式)是很简单的，分为 3 个步骤：

1、添加文件。

首先，我们在实验 53_1 DSP BasicMath 测试实验\Drivers\CMSIS 目录下新建：DSP 和 Lib 文件夹，然后把官方的相应文件拉到我们的工程的相应位置：arm_cortexM7lfdp_math.lib 和相关头文件，再把没用到的文件删除，如图 65.1.2.1 和图 65.1.2.2 所示：

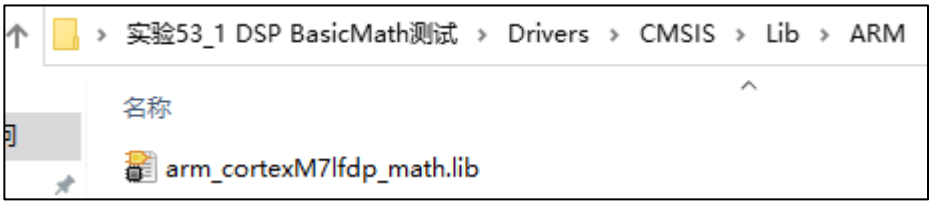


图 65.1.2.1 DSP_LIB 文件夹添加文件

图 65.1.2.2 DSP_LIB 文件夹添加文件

这个步骤具体请参考工程源码文件，arm_cortexM7lfdp_math.lib 文件在 65.1.1 节已经介绍过了。Include 文件夹里面包含了我们可能要用到的相关头文件，所以要添加到工程中。

然后，打开工程，新建 Drivers/CMSIS 分组，并将 arm_cortexM7lfdp_math.lib 添加到工程里面，如图 65.1.2.3 所示：

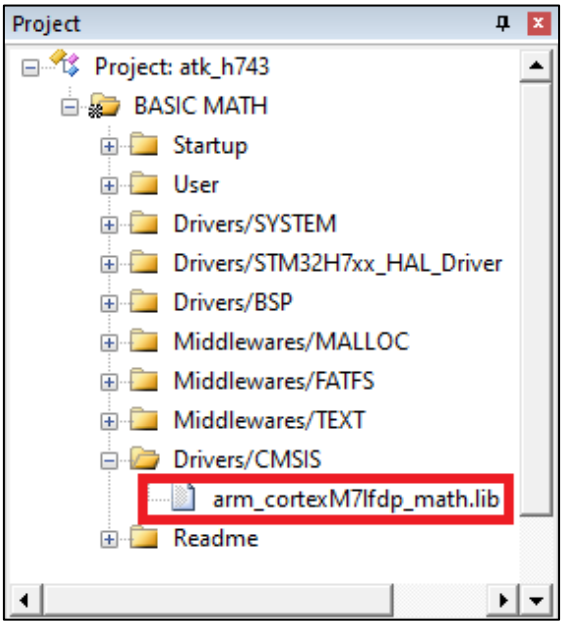


图 65.1.2.3 添加.lib 文件

这样，添加文件就结束了（就添加了一个.lib 文件）。

2、添加头文件包含路径

添加好.lib 文件后，我们要添加头文件包含路径，这个我们工程都统一添加好了，所以不用额外再操作，这里是提醒一下大家要添加头文件包含路径，如图 65.1.2.4 所示：

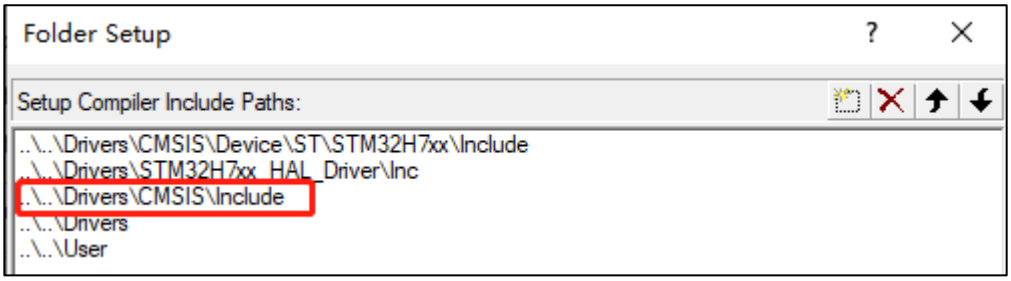


图 65.1.2.4 添加相关头文件包含路径

3、添加全局宏定义

最后，为了使用 DSP 库的所有功能，我们还需要添加几个全局宏定义：

1，__FPU_USED
2，__FPU_PRESENT
3，ARM_MATH_CM7
4，__CC_ARM
5，ARM_MATH_MATRIX_CHECK

6，ARM_MATH_ROUNDING

添加方法：点击→C/C++选项卡，然后在 Define 里面进行设置，如图 65.1.2.5 所示：

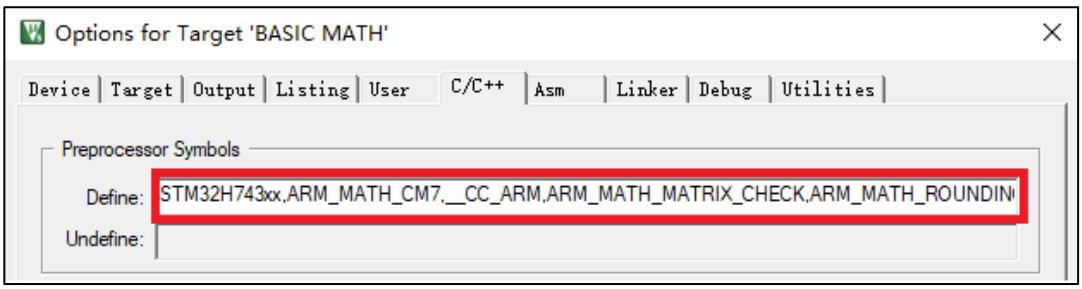


图 65.1.2.5 DSP 库支持全局宏定义设置

这里，两个宏之间用“,”隔开。并且，上面的全局宏里面，我们没有添加__FPU_USED，因为在 Target 选项卡设置 Floating Point Hardware 的时候（上一章有介绍），只要我们选择了：Double Precision（如果没有设置 Double Precision，则必须设置），MDK 就会自动添加这个全局宏，因此不需要我们手动添加了。同时__FPU_PRESENT 全局宏我们 FPU 实验已经讲解，这个宏定义在 stm32h7xx.h 头文件里面已经定义。这样，在 Define 处要输入的所有宏为：STM32H743xx，USE_HAL_DRIVER,ARM_MATH_CM7,

__CC_ARM,ARM_MATH_MATRIX_CHECK,ARM_MATH_ROUNDING 等共 6 个。

至此，STM32H743 的 DSP 库运行环境就搭建完成了。

特别注意，为了方便调试，本章例程我们将 MDK 的优化设置为-O0 优化，以得到最好的调试效果。

65.2 硬件设计

1. 例程功能

本例程包含 2 个源码：**实验 53_1 DSP BasicMath 测试实验**和**实验 53_2 DSP FFT 测试实验**，他们除了 main.c 里面内容不一样外，其他源码完全一模一样（包括 MDK 配置）。

实验 53_1 DSP BasicMath 测试实验功能简介：测试 STM32H743 的 DSP 库基础数学函数：arm_cos_f32 和 arm_sin_f32 和标准库基础数学函数：cosf 和 sinf 的速度差别，并在 LCD 屏幕上面显示两者计算所用时间。LED0 闪烁，提示程序运行。

实验 53_2 DSP FFT 测试实验功能简介：测试 STM32H743 的 DSP 库的 FFT 函数，程序运行后，自动生成 1024 点测试序列，然后，每当 KEY0 按下后，调用 DSP 库的 FFT 算法（基 4 法）执行 FFT 运算，在 LCD 屏幕上面显示运算时间，同时将 FFT 结果输出到串口。LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯

LED ： LED0 – PB1

2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)

3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)

4）独立按键 ： KEY0 – PH3

5）FPU(浮点计算单元)

6）定时器 6

65.3 程序设计

本章代码，分成两个工程：1，实验 53_1 DSP BasicMath 测试；2，实验 53_2 DSP FFT 测试，接下来我们分别介绍。

65.3.1 DSP BasicMath 测试

这是我们使用 STM32H743 的 DSP 库进行基础数学函数测试的一个例程。使用大家耳熟能详的公式进行计算：

$$\sin(x)^2 + \cos(x)^2 = 1$$

这里我们用到的就是 sin 和 cos 函数，不过实现方式不同。MDK 的标准库（math.h）提供我们：sin、cos、sinf 和 cosf 等 4 个函数，带 f 的表示单精度浮点型运算，即 float 型，而不带 f 的表示双精度浮点型，即 double。

STM32H743 的 DSP 库，则提供我们另外两个函数：arm_sin_f32 和 arm_cos_f32（**注意：需要添加：arm_math.h 头文件才可使用**），这两个函数也是单精度浮点型的，用法同 sinf 和 cosf 一模一样。

本例程就是测试：arm_sin_f32& arm_cos_f32 同 sinf&cosf 的速度差别。

因为 65.1.2 节已经搭建好 DSP 库运行环境了，所以我们这里直接只需要修改 main.c 里面的代码即可，main.c 代码如下：

```
#define DELTA   0.00005f         /* 误差值 */

/**
 * @brief       sin cos 测试
 * @param       angle : 起始角度
 * @param       times : 运算次数
 * @param       mode  : 是否使用 DSP 库
 *   @arg       0 , 不使用 DSP 库;
 *   @arg       1 , 使用 DSP 库;
 *
 * @retval      无
 */
uint8_t sin_cos_test(float angle, uint32_t times, uint8_t mode)
{
    float sinx, cosx;
    float result;
    uint32_t i = 0;
    if (mode == 0)
    {
        for (i = 0; i < times; i++)
        {
            cosx = cosf(angle);                 /* 不使用 DSP 优化的 sin，cos 函数 */
            sinx = sinf(angle);
            result = sinx * sinx + cosx * cosx;    /* 计算结果应该等于 1 */
            result = fabsf(result - 1.0f);         /* 对比与 1 的差值 */
            if (result > DELTA)return 0XFF;        /* 判断失败 */
            angle += 0.001f;                       /* 角度自增 */
        }
    }
    else
    {
        for (i = 0; i < times; i++)
        {
            cosx = arm_cos_f32(angle);          /* 使用 DSP 优化的 sin，cos 函数 */
            sinx = arm_sin_f32(angle);
            result = sinx * sinx + cosx * cosx;  /* 计算结果应该等于 1 */
            result = fabsf(result - 1.0f);       /* 对比与 1 的差值 */
            if (result > DELTA)return 0XFF;      /* 判断失败 */
            angle += 0.001f;                     /* 角度自增 */
        }
    }
    return 0;    /* 任务完成 */
}

uint8_t g_timeout;
```

```
int main(void)
{
    float time;
    char buf[50];
    uint8_t res;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    btim_timx_int_init(65535, 20000 - 1);   /* 10Khz 计数频率,最大计时 6.5 秒超出 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "DSP BasicMath TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 120, 200, 16, 16, "No DSP runtime:",RED);/*显示提示信息*/
    lcd_show_string(30, 150, 200, 16, 16, "Use DSP runtime:", RED);
    while (1)
    {
        /* 不使用 DSP 优化 */
        BTIM_TIMX_INT->CNT = 0; /* 重设 TIM3 定时器的计数器值 */
        g_timeout = 0;
        res = sin_cos_test(PI / 6, 1000000, 0);
        time = BTIM_TIMX_INT->CNT + (uint32_t)g_timeout * 65536;
        sprintf(buf, "%0.1fms\r\n", time / 10);

        if (res == 0)
        {
            /* 显示运行时间 */
            lcd_show_string(30 + 16 * 8, 120, 100, 16, 16, buf, BLUE);
        }
        else
        {
            /* 显示当前运行情况 */
            lcd_show_string(30 + 16 * 8, 120, 100, 16, 16, "error! ", BLUE);
        }
        /* 使用 DSP 优化 */
        BTIM_TIMX_INT->CNT = 0; /* 重设 TIM3 定时器的计数器值 */
        g_timeout = 0;
        res = sin_cos_test(PI / 6, 100000, 1);
        time = BTIM_TIMX_INT->CNT + (uint32_t)g_timeout * 65536;
        sprintf(buf, "%0.1fms\r\n", time / 10);
        if (res == 0)
        {
            /* 显示运行时间 */
            lcd_show_string(30 + 16 * 8, 150, 100, 16, 16, buf, BLUE);
        }
        else
        {
            /* 显示错误 */
            lcd_show_string(30 + 16 * 8, 150, 100, 16, 16, "error! ", BLUE);
        }
        LED0_TOGGLE();
    }
}
```

这里包括 2 个函数：sin_cos_test 和 main 函数，sin_cos_test 函数用于根据给定参数，执行 $sin(x)^2+cos(x)^2=1$ 的计算。计算完后，计算结果同给定的误差值（DELTA）对比，如果不大于

误差值，则认为计算成功，否则计算失败。该函数可以根据给定的模式参数(mode)来决定使用哪个基础数学函数执行运算，从而得出对比。

main 函数则比较简单，这里我们通过定时器 6 来统计 sin_cos_test 运行时间，从而得出对比数据。主循环里面，每次循环都会两次调用 sin_cos_test 函数，首先采用不使用 DSP 库方式计算，然后采用使用 DSP 库方式计算，并得出两次计算的时间，显示在 LCD 上面。

DSP 基础数学函数测试的程序设计就讲解到这里。

65.3.2 DSP FFT 测试

这是我们使用 STM32H743 的 DSP 库进行 FFT 函数测试的一个例程。

首先，我们简单介绍下 FFT：FFT 即快速傅里叶变换，可以将一个时域信号变换到频域。因为有些信号在时域上是很难看出什么特征的，但是如果变换到频域之后，就很容易看出特征了，这就是很多信号分析采用 FFT 变换的原因。另外，FFT 可以将一个信号的频谱提取出来，这在频谱分析方面也是经常用的。简而言之，FFT 就是将一个信号从时域变换到频域方便我们分析处理。

在实际应用中，一般的处理过程是先对一个信号在时域进行采集，比如我们通过 ADC，按照一定大小采样频率 F 去采集信号，采集 N 个点，那么通过对这 N 个点进行 FFT 运算，就可以得到这个信号的频谱特性。

这里还涉及到一个采样定理的概念：在进行模拟/数字信号的转换过程中，当采样频率 F 大于信号中最高频率 f_{max} 的 2 倍时($F>2*f_{max}$)，采样之后的数字信号完整地保留了原始信号中的信息，采样定理又称奈奎斯特定理。举个简单的例子：比如我们正常人发声，频率范围一般在 8KHz 以内，那么我们要通过采样之后的数据来恢复声音，我们的采样频率必须为 8KHz 的 2 倍以上，也就是必须大于 16KHz 才行。

模拟信号经过 ADC 采样之后，就变成了数字信号，采样得到的数字信号，就可以做 FFT 变换了。N 个采样点数据，在经过 FFT 之后，就可以得到 N 个点的 FFT 结果。为了方便进行 FFT 运算，通常 N 取 2 的整数次方。

假设采样频率为 F，对一个信号采样，采样点数为 N，那么 FFT 之后结果就是一个 N 点的复数，每一个点就对应着一个频率点（以基波频率为单位递增），这个点的模值（sqrt(实部 ^{2}+虚部 ^{2})）就是该频点频率值下的幅度特性。具体跟原始信号的幅度有什么关系呢？假设原始信号的峰值为 A，那么 FFT 的结果的每个点（除了第一个点直流分量之外）的模值就是 A 的 N/2 倍，而第一个点就是直流分量，它的模值就是直流分量的 N 倍。

这里还有个基波频率，也叫频率分辨率的概念，就是如果我们按照 F 的采样频率去采集一个信号，一共采集 N 个点，那么基波频率（频率分辨率）就是 f_k=F/N。这样，第 n 个点对应信号频率为：F*(n-1)/N；其中 n≥1，当 n=1 时为直流分量。

关于 FFT 我们就介绍到这。

如果我们要自己实现 FFT 算法，对于不懂数字信号处理的朋友来说，是比较难的，不过，ST 提供的 STM32H743 DSP 库里面就有 FFT 函数给我们调用，因此我们只需要知道如何使用这些函数，就可以迅速的完成 FFT 计算，而不需要自己学习数字信号处理，去编写代码了，大大方便了我们的开发。

STM32H743 的 DSP 库里面，提供了定点和浮点 FFT 实现方式，并且有基 4 的也有基 2 的，大家可以根据需要自由选择实现方式。注意：对于基 4 的 FFT 输入点数必须是 4^n，而基 2 的 FFT 输入点数则必须是 2^n，并且基 4 的 FFT 算法要比基 2 的快。

本章我们将采用 DSP 库里面的基 4 浮点 FFT 算法来实现 FFT 变换，并计算每个点的模值，所用到的函数有：

```
/* Deprecated */
arm_status arm_cfft_radix4_init_f32(arm_cfft_radix4_instance_f32 * S,
                  uint16_t fftLen, uint8_t ifftFlag, uint8_t bitReverseFlag);

/* Deprecated */
void arm_cfft_radix4_f32(const arm_cfft_radix4_instance_f32 * S,
                         float32_t * pSrc);

/**
```

```
 * @brief  Floating-point complex magnitude
 * @param[in]  pSrc        points to the complex input vector
 * @param[out] pDst        points to the real output vector
 * @param[in]  numSamples  number of complex samples in the input vector
 */
void arm_cmplx_mag_f32(float32_t * pSrc, float32_t * pDst,uint32_t numSamples);
```

第一个函数 arm_cfft_radix4_init_f32，用于初始化 FFT 运算相关参数，其中：fftLen 用于指定 FFT 长度（16/64/256/1024/4096），本章设置为 1024；ifftFlag 用于指定是傅里叶变换(0)还是反傅里叶变换(1)，本章设置为 0；bitReverseFlag 用于设置是否按位取反，本章设置为 1；最后，所有这些参数存储在一个 arm_cfft_radix4_instance_f32 结构体指针 S 里面。

第二个函数 arm_cfft_radix4_f32 就是执行基 4 浮点 FFT 运算的，pSrc 传入采集到的输入信号数据（实部+虚部形式），同时 FFT 变换后的数据，也按顺序存放在 pSrc 里面，pSrc 必须大于等于 2 倍 fftLen 长度。另外，S 结构体指针参数是先由 arm_cfft_radix4_init_f32 函数设置好，然后传入该函数的。

第三个函数 arm_cmplx_mag_f32 用于计算复数模值，可以对 FFT 变换后的结果数据，执行取模操作。pSrc 为复数输入数组（大小为 2*numSamples）指针，指向 FFT 变换后的结果；pDst 为输出数组（大小为 numSamples）指针，存储取模后的值；numSamples 就是总共有多少个数据需要取模。

通过这三个函数，我们便可以完成 FFT 计算，并取模值。本节例程（实验 53_2 DSP FFT 测试实验）同样是在 65.1.2 节已经搭建好 DSP 库运行环境上面修改代码，只需要修改 main.c 里面的代码即可，本例程 main.c 代码如下：

```
/* FFT 长度，默认是 1024 点 FFT
 * 可选范围为: 16, 64, 256, 1024.
 */
#define FFT_LENGTH      1024

float fft_inputbuf[FFT_LENGTH * 2];     /* FFT 输入数组 */
float fft_outputbuf[FFT_LENGTH];        /* FFT 输出数组 */

uint8_t g_timeout;

int main(void)
{
    float time;
    char buf[50];
    arm_cfft_radix4_instance_f32 scfft;
    uint8_t key, t = 0;
    uint16_t i;

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
    btim_timx_int_init(65535, 20000 - 1);   /* 10Khz 计数频率,最大计时 6.5 秒超出 */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "DSP FFT TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEY0:Run FFT", RED);
    lcd_show_string(30, 160, 200, 16, 16, "FFT runtime:", RED);
    /* 初始化 scfft 结构体，设定 FFT 相关参数 */
    arm_cfft_radix4_init_f32(&scfft, FFT_LENGTH, 0, 1);
```

```
    while (1)
    {
        key = key_scan(0);
        if (key == KEY0_PRES)
        {
            for (i = 0; i < FFT_LENGTH; i++)    /* 生成信号序列 */
            {
                /* 生成输入信号实部 */
                fft_inputbuf[2 * i] = 100 +
                10 * arm_sin_f32(2 * PI * i / FFT_LENGTH) +
                30 * arm_sin_f32(2 * PI * i * 4 / FFT_LENGTH) +
                50 * arm_cos_f32(2 * PI * i * 8 / FFT_LENGTH);
                fft_inputbuf[2 * i + 1] = 0; /* 虚部全部为 0 */
            }
            BTIM_TIMX_INT->CNT = 0;; /* 重设 BTIM_TIMX_INT 定时器的计数器值 */
            g_timeout = 0;

            arm_cfft_radix4_f32(&scfft, fft_inputbuf);       /* FFT 计算（基 4） */
            /* 计算所用时间 */
            time = BTIM_TIMX_INT->CNT + (uint32_t)g_timeout * 65536;
            sprintf((char *)buf, "%0.3fms\r\n", time / 1000);
            /* 显示运行时间 */
            lcd_show_string(30 + 12 * 8, 160, 100, 16, 16, buf, BLUE);
            /* 把运算结果复数求模得幅值 */
            arm_cmplx_mag_f32(fft_inputbuf, fft_outputbuf, FFT_LENGTH);

            printf("\r\n%d point FFT runtime:%0.3fms\r\n", FFT_LENGTH,
                  time / 1000);
            printf("FFT Result:\r\n");

            for (i = 0; i < FFT_LENGTH; i++)
            {
                printf("fft_outputbuf[%d]:%f\r\n", i, fft_outputbuf[i]);
            }
        }
        else
        {
            delay_ms(10);
        }
        t++;
        if ((t % 20) == 0)
        {
            LED0_TOGGLE();
        }
    }
}
```

以上代码只有一个 main 函数，里面通过我们前面介绍的三个函数：arm_cfft_radix4_init_f32、arm_cfft_radix4_f32 和 arm_cmplx_mag_f32 来执行 FFT 变换并取模值。每当按下 KEY0 就会重新生成一个输入信号序列，并执行一次 FFT 计算，将 arm_cfft_radix4_f32 所用时间统计出来，显示在 LCD 屏幕上面，同时将取模后的模值通过串口打印出来。

这里，我们在程序上生成了一个输入信号序列用于测试，输入信号序列表达式：

```
/* 生成输入信号实部 */
fft_inputbuf[2 * i] = 100 +
                      10 * arm_sin_f32(2 * PI * i / FFT_LENGTH) +
                      30 * arm_sin_f32(2 * PI * i * 4 / FFT_LENGTH) +
                      50 * arm_cos_f32(2 * PI * i * 8 / FFT_LENGTH);
```

通过该表达式我们可知，信号的直流分量为 100，外加 2 个正弦信号和一个余弦信号，其幅值分别为 10、30 和 50。

65.4 下载验证

将实验 53_1 DSP BasicMath 测试实验的程序下载到开发板后，可以在屏幕看到两种实现方式的速度差别，如图 65.4.1 所示：

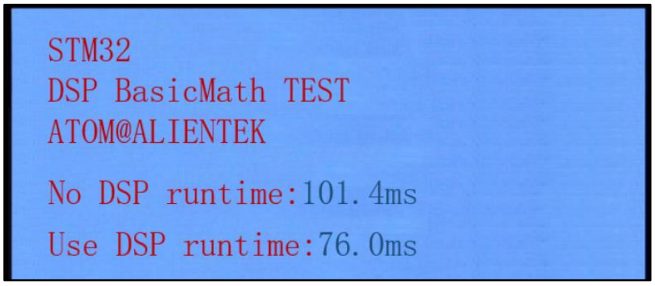


图 65.4.1 使用 DSP 库和不使用 DSP 库的基础数学函数速度对比

从图中可以看出，使用 DSP 库的基础数学函数计算所用时间比不使用 DSP 库的短，使用 STM32H743 的 DSP 库，速度上面比传统的实现方式提升了约 25%。

对于实验 53_2 DSP FFT 测试实验，下载后，屏幕显示提示信息，然后我们按下 KEY0 就可以看到 FFT 运算所耗时间，如图 65.4.2 所示：

图 65.4.2 FFT 测试界面

可以看到，STM32H743 采用基 4 法计算 1024 个浮点数的 FFT，仅用了 0.002ms，速度是非常的快。同时，可以在串口看到 FFT 变换取模后的各频点模值，如图 65.4.3 所示：

图 65.4.3 FFT 变换后个频点模值

查看所有数据，会发现：第 0、1、4、8、1016、1020、1023 这 7 个点的值比较大，其他点的值都很小，接下来我们就简单分析一下这些数据。

由于 FFT 变换后的结果具有对称性，所以，实际上有用的数据，只有前半部分，后半部分和前半部分是对称关系，比如 1 和 1023，4 和 1020，8 和 1016 等，就是对称关系，因此我们只

需要分析前半部分数据即可。这样，就只有第 0、1、4、8 这四个点，比较大，重点分析。

假设我们采样频率为 1024Hz，那么总共采集 1024 个点，频率分辨率就是 1Hz，对应到频谱上面，两个点之间的间隔就是 1Hz。因此，上面我们生成的三个叠加信号：10*sin(2*PI*i/1024)+ 30*sin(2*PI*i*4/1024)+50*cos(2*PI*i*8/1024)，频率分别是：1Hz、4Hz 和 8Hz。

对于上述 4 个值比较大的点，结合 65.3.1 节的知识，很容易分析得出：第 0 点，即直流分量，其 FFT 变换后的模值应该是原始信号幅值的 N 倍，N=1024，所以值是 100*1024=102400，与理论完全一样，然后其他点，模值应该是原始信号幅值的 N/2 倍，即 10*512、30*512、50*512，而我们计算结果是：5119.950684、15359.999023、256000，同理论值非常接近。

DSP 测试实验，我们就讲解到这里，DSP 库的其他测试实例，大家可以自行研究下，我们这里就不再介绍了。

第六十六章 手写识别实验

本章，我们将利用正点原子提供的手写识别库，在开发板上实现一个简单的数字字母手写识别。

本章分为如下几个小节：

66.1 手写识别简介
66.2 硬件设计
66.3 程序设计
66.4 下载验证

66.1 手写识别简介

手写识别，是指对在手写设备上书写时产生的有序轨迹信息进行识别的过程，是人际交互最自然、最方便的手段之一。随着智能手机和平板电脑等移动设备的普及，手写识别的应用也被越来越多的设备采用。

手写识别能够使用户按照最自然、最方便的输入方式进行文字输入，易学易用，可取代键盘或者鼠标。用于手写输入的设备有许多种，比如电磁感应手写板、压感式手写板、触摸屏、触控板、超声波笔等。本实验通过使用 STM32 板子自带得 TFTLCD 触摸屏(2.8/3.5/4.3/7 寸)，可以用来作为手写识别的输入设备。接下来，我们将给大家简单介绍下手写识别的实现过程。

手写识别与其他识别系统如语音识别、图像识别一样分为两个过程：训练学习过程；识别过程。如图 66.1.1 所示：

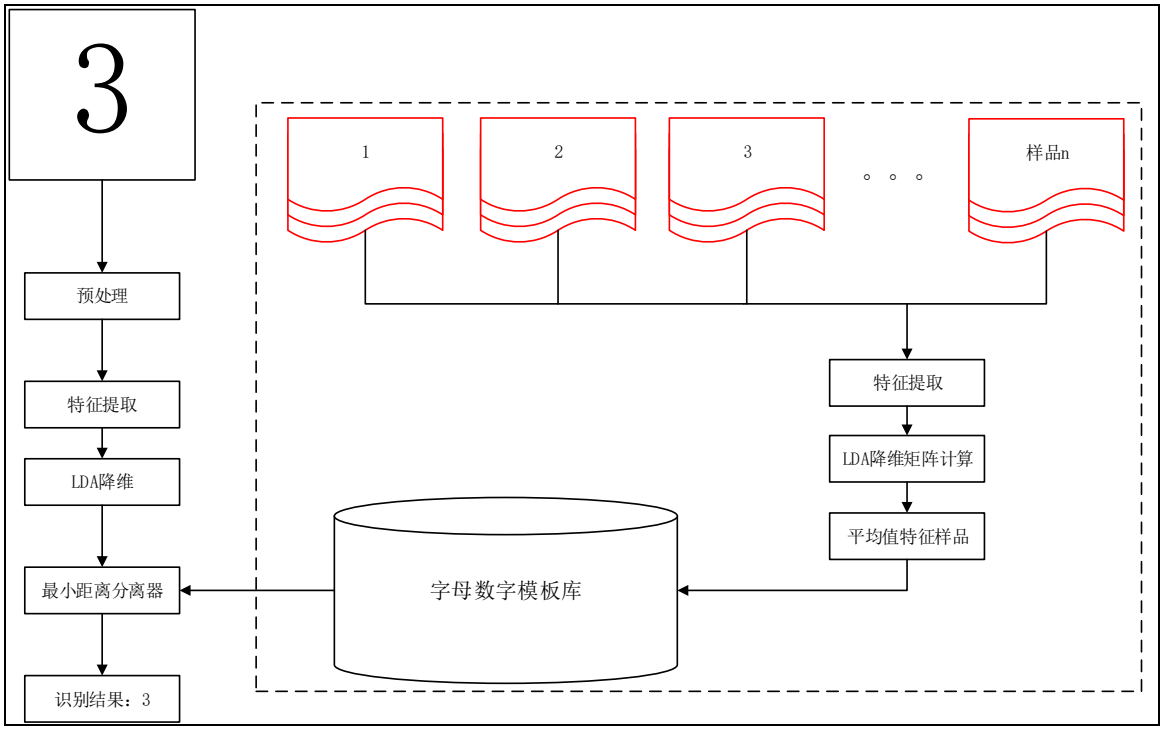


图 66.1.1 字母数字识别系统示意图

上图中虚线部分分为训练学习过程，该过程首先需要使用设备采集大量数据样本，样本类别数目为 0~9，a~z，A~Z 总过 62 类，每个类别 5~10 个样本不等（样本越多识别率就越高）。对这些样本进行传统的八方向特征提取，提取后特征维数为 512 维，这对于 STM32 来说，计算量和模板库的存储量都是难以接受，所以需要运行一些方法进行降维，这里采用 LDA 线性判决分析的方法进行降维。所谓的线性判决分析，即是假设所有样本服从高斯分布（正态分布）对样本进行低维投影，以达到各个样本间的距离最大化。关于 LDA（线性判别分析）的更多

知识可以自行阅读（http://wenku.baidu.com/view/f05c731452d380eb62946d39.html）等参考文档。这里将维度降到 64 维度，然后针对各个样本类别进行平均计算的到该类别的样本模板。

而对于识别过程，首先得到触屏输入的有序轨迹，然后进行一些预处理，预处理主要包括重采样，归一处理。重采样主要是因为不同的输入设备不同的输入处理方式产生的有序轨迹序列有所不同。为了达到更好的识别结果我们需要对训练样本和识别输入的样本进行重采样处理，这里主要应用隔点重采样的方法对输入序列进行重采样；而归一化就是因为不同的书写风格采用分辨率的差异会导致字体太小不同，因此需要对输入轨迹进行归一化。这里把样本进行线性缩放的方法归一化为 64*64 像素。

接下来进行同样的八方向特征提取操作。所谓的八方向特征就是首先将经过预处理后的 64*64 输入进行切分成 8*8 的小方格，每个方格 8*8 个像素；然后对每个 8*8 个小格进行各个方向的点数统计。如某个方格内一共有 10 个点，其中八个方向的点分别为：1、3、5、2、3、4、3、2，那么这个格子得到的八个特征向量为[0.1, 0.3, 0.5, 0.2, 0.3, 0.4, 0.3, 0.2]。总共 64 个格子，于是一个样本最终能得到 64*8=512 维特征，更多八方向特征提取可以参考一下两个文档：

1，http://wenku.baidu.com/view/d37e5a49e518964bcf847ca5.html;

2，http://wenku.baidu.com/view/3e7506254b35eefdc8d333a1.html;

由于训练过程进行了 LDA 降维计算，所以识别过程同样需要对应的 LDA 降维过程得到最终的 64 维特征。这个计算过程就是在训练模板的过程中可以运算得到一个 512*64 维的矩阵，那么我们通过矩阵乘运算可以得到 64 维的最终特征值。

$$[d_1, d_2, \cdots, d_{512}] \times \begin{bmatrix} l & \cdots & l \\ \vdots & \ddots & \vdots \\ l & \cdots & l \end{bmatrix} = \begin{bmatrix} f_1 \\ \vdots \\ f_{64} \end{bmatrix}$$

最后将这 64 维特征分别与模板中的特征进行求距离运算。得到最小的距离为该输入的最佳识别结果输出。

$$output = \arg min_{i \in [1,62]} \{(f_1 - f_1^i)^2 + (f_2 - f_2^i)^2 + \cdots + (f_{64} - f_{64}^i)^2\}$$

关于手写识别原理，我们就介绍到这里。如果想自己实现手写识别，那得花很多时间学习和研究，但是如果只是应用的话，那么就只需要知道怎么用就 OK 了，相对来说，简单得多。

正点原子提供了一个数字字母识别库，我们不需要关心手写识别是如何实现的，只需要知道这个库怎么用，就能实现手写识别。正点原子提供的手写识别库由 4 个文件组成：ATKNCR_M_V2.0.lib、ATKNCR_N_V2.0.lib、atk_ncr.c 和 atk_ncr.h。

ATKNCR_M_V2.0.lib 和 ATKNCR_N_V2.0.lib 是两个识别用的库文件（两个版本），选择其中之一即可。ATKNCR_M_V2.0.lib 用于使用内存管理的情况，用户必须自己实现 alientek_ncr _malloc 和 alientek_ncr_free 两个函数。而 ATKNCR_N_V2.0.lib 用于不使用内存管理的情况，通过全局变量来定义缓存区，缓存区需要提供至少 3K 左右的 RAM。大家根据自己的需要，选择对应的版本。正点原子手写识别库资源需求：FLASH 在 52K 左右，RAM 在 6K 左右。

66.2 硬件设计

1. 例程功能

开机的时候先初始化手写识别器，然后检测字库，之后进入等待输入状态。此时，我们在手写区写数字/字符，在每次写入结束后，自动进入识别状态，进行识别，然后将识别结果输出在 LCD 模块上，同时打印到串口。通过按 KEY0 可以进行模式切换（4 种模式都可以测试），通过按 KEY_UP，可以进入触摸屏校准（仅电阻屏需要校准，如果发现触摸屏不准，请执行此操作）。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯 LED0 - PB1

2）独立按键 KEY0 - PH3、KEY1 - PH2

3）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)

4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)
5）QSPI(PB2/PB6/PF6/PF7/PF8/PF9)
6）norflash(QSPI FLASH 芯片,连接在 QSPI 上)
7）触摸屏(电阻式/电容式)

66.3 程序设计

66.3.1 程序流程图

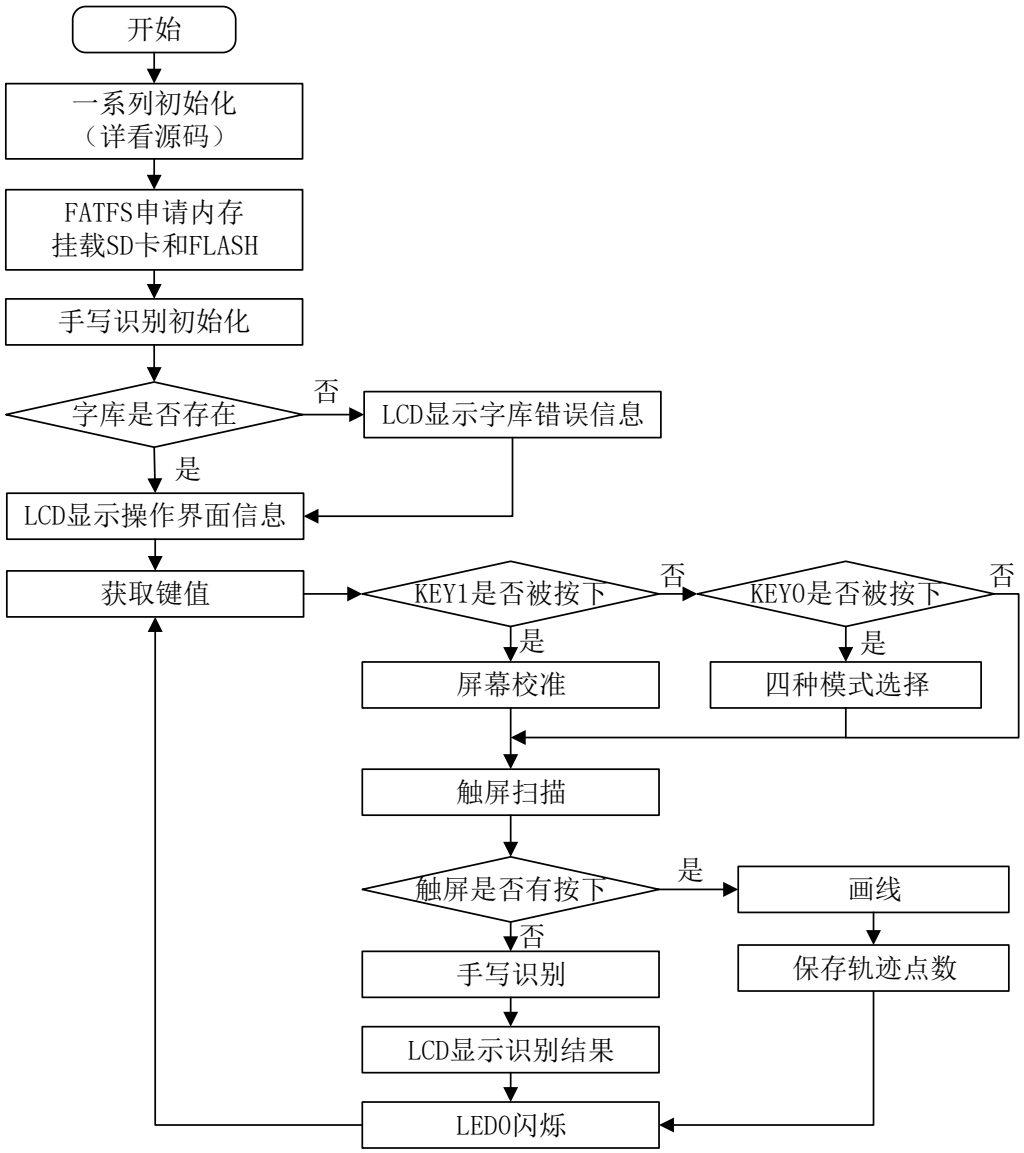


图 66.3.1 手写识别实验程序流程图

66.3.2 程序解析

1. ATKNCR 代码

ATKNCR 代码部分有四个文件组成，前面已经对它们有整体的介绍过。因为工程中我们要使用内存管理，所以我们使用的库是 ATKNCR_M_V2.0.lib。首先我们先看一下 atk_nrc.h 头文件中比较重要部分，其代码如下：

```
/* 输入轨迹坐标类型 */
__packed typedef struct _atk_ncr_point
```

```
{
    short x;            /* x 轴坐标 */
    short y;            /* y 轴坐标 */
}atk_ncr_point;

/* 外部调用函数
 * 初始化识别器
 * 返回值 : 0, 初始化成功
 *          1, 初始化失败
 */
unsigned char alientek_ncr_init(void);

/* 停止识别器 */
void alientek_ncr_stop(void);

/* 识别器识别
 * track   : 输入点阵集合
 * potnum  : 输入点阵的点数,就是 track 的大小
 * charnum : 期望输出的结果数,就是你希望输出多少个匹配结果
 * mode    : 识别模式
 *           1,仅识别数字
 *           2,仅识别大写字母
 *           3,仅识别小写字母
 *           4,混合识别(全部识别)
 *
 * result  : 结果缓存区(至少为:charnum+1 个字节)
 */
void alientek_ncr(atk_ncr_point * track, int potnum, int charnum,
                  unsigned char mode,char*result);
```

在上面的代码中，我们定义了一些外部接口函数以及轨迹结构体等。

alientek_ncr_init 函数用与初始化识别器，该函数在.lib 文件实现，在识别开始之前，我们应该调用该函数。

alientek_ncr_stop 函数用于停止识别器，在识别完成之后（不需要再识别），我们调用该函数，如果一直处于识别状态，则没必要调用。该函数也是在.lib 文件实现。

alientek_ncr 函数就是识别函数了。它有 5 个参数，第一个参数 track，为输入轨迹点的坐标集（最好 200 以内）；第二个参数 potnum，为坐标集点坐标的个数；第三个参数 charnum，为期望输出的结果数，即希望输出多少个匹配结果，识别器按匹配程度排序输出（最佳匹配排第一）；第四个参数 mode，该函数用于设置模式，识别器总共支持 4 种模式：

1，仅识别数字

2，进识别大写字母

3，仅识别小写字母

4，混合识别(全部识别)

最后一个参数是 result，用来输出结果，注意这个结果是 ASCII 码格式的。

下面我们直接来介绍 atk_ncr.c 中内存管理部分，其代码如下：

```
/**
 * @brief       内存设置函数
 * @param       *p    : 内存首地址
 * @param       c     : 要设置的值
 * @param       len   : 需要设置的内存大小(字节为单位)
 * @retval      无
 */
void alientek_ncr_memset(char *p, char c, unsigned long len)
{
    my_mem_set((uint8_t*)p, (uint8_t)c, (uint32_t)len);
}

/**
```

```
 * @brief       分配内存
 * @param       size : 要分配的内存大小(字节)
 * @retval      分配到的内存首地址.
 */
void *alientek_ncr_malloc(unsigned int size)
{
    return mymalloc(SRAMIN,size);
}

/**
 * @brief       释放内存
 * @param       ptr  : 内存首地址
 * @retval      无
 */
void alientek_ncr_free(void *ptr)
{
    myfree(SRAMIN,ptr);
}
```

alientek_ncr_memset、alientek_ncr_free 和 alientek_ncr_free 三个函数的实现主要调用 malloc 中的函数实现。这里就不多讲，忘记这些函数的实现，可以回顾一下内存管理实验章节。

2. main.c 代码

在这里讲一下正点原子提供的手写数字识别库实现数字字母识别的步骤：

1） 调用 alientek_ncr_init 函数，初始化识别程序

该函数用来初始化识别器，在手写识别进行之前，必须调用该函数。

2） 获取输入点阵数据

此步，我们通过触摸屏获取输入轨迹点阵坐标，然后存放到一个缓冲区里面，注意至少要输入 2 个不同坐标的点阵数据，才能正常识别。注意输入点数不要太多，太多的话，需要更多的内存，我们推荐的输入点数范围：100~200 点。

3） 调用 alientek_ncr 函数，得到识别结果

通过调用 alitntek_ncr 函数，我们可以得到输入点阵的识别结果，结果将保存在 result 参数里面，采用 ASCII 码格式存储。

4） 调用 alientek_ncr_stop 函数，终止识别

如果不需要继续识别，则调用 alientek_ncr_stop 函数，终止识别器。如果还需要继续识别，重复步骤 2 和步骤 3 即可。

以上 4 个步骤，就是使用正点原子手写识别库的方法，十分简单。这个操作流程也在主函数中清晰看到，下面看一下 main.c，其代码如下：

```
int main(void)
{
    uint8_t i = 0;
    uint8_t tcnt = 0;
    char sbuf[10];
    uint8_t key;
    uint16_t pcnt = 0;
    uint8_t mode = 4;                       /* 默认是混合模式 */
    uint16_t lastpos[2];                    /* 最后一次的数据 */

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    key_init();                             /* 初始化按键 */
```

```
    norflash_init();                            /* 初始化 norflash */
    tp_dev.init();                              /* 初始化触摸屏 */
    my_mem_init(SRAMIN);                        /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                        /* 初始化外部内存池 */
    my_mem_init(SRAMDTCM);                      /* 初始化 DTCM 内存池(DTCM) */
    alientek_ncr_init();                        /* 初始化手写识别 */
    exfuns_init();                              /* 为 fatfs 相关变量申请内存 */
    f_mount(fs[0], "0:", 1);                    /* 挂载 SD 卡 */
    f_mount(fs[1], "1:", 1);                    /* 挂载 SPI FLASH */
    f_mount(fs[2], "2:", 1);                    /* 挂载 NAND FLASH *

    while (fonts_init())                        /* 检查字库 */
    {
        lcd_show_string(60, 90, 200, 16, 16, "Font Error!", RED);
        delay_ms(200);
        lcd_fill(60, 90, 200+20, 90+16, WHITE); /* 清除显示 */
        delay_ms(200);
    }

RESTART:
    text_show_string(60, 10, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
    text_show_string(60, 30, 200, 16, "手写识别实验", 16, 0, RED);
    text_show_string(60, 50, 200, 16, "正点原子@ALIENTEK", 16, 0, RED);
    text_show_string(60, 70, 200, 16, "KEY0:MODE KEY1:Adjust", 16, 0, RED);
    text_show_string(60, 90, 200, 16, "识别结果:", 16, 0, RED);
    lcd_draw_rectangle(19, 114, lcddev.width - 20, lcddev.height - 5, RED);
    text_show_string(96, 207, 200, 16, "手写区", 16, 0, BLUE);
    tcnt = 100;

    while (1)
    {
        key = key_scan(0);
        if (key == KEY1_PRES && (tp_dev.touchtype & 0X80) == 0)
        {
            tp_adjust();        /* 屏幕校准 */
            lcd_clear(WHITE);
            goto RESTART;       /* 重新加载界面 */
        }
        if (key == KEY0_PRES)
        {
            lcd_fill(20, 115, 219, 314, WHITE); /* 清除当前显示 */
            mode++;
            if (mode > 4)mode = 1;
            switch (mode)
            {
                case 1:
                text_show_string(80, 207, 200, 16, "仅识别数字", 16, 0, BLUE);
                   break;

                case 2:
                text_show_string(64, 207, 200, 16, "仅识别大写字母", 16, 0, BLUE);
                   break;

                case 3:
                text_show_string(64, 207, 200, 16, "仅识别小写字母", 16, 0, BLUE);
                   break;

                case 4:
                text_show_string(88, 207, 200, 16, "全部识别", 16, 0, BLUE);
                   break;
            }
            tcnt = 100;
        }
```

```
        tp_dev.scan(0);         /* 扫描 */
        if (tp_dev.sta & TP_PRES_DOWN)  /* 有按键被按下 */
        {
            delay_ms(1);        /* 必要的延时，否则老认为有按键按下 */
            tcnt = 0;           /* 松开时的计数器清空 */

            if ((tp_dev.x[0] < (lcddev.width - 20 - 2)
               && tp_dev.x[0] >= (20 + 2))
               && (tp_dev.y[0] < (lcddev.height - 5 - 2)
               && tp_dev.y[0] >= (115 + 2)))
            {
                if (lastpos[0] == 0XFFFF)
                {
                    lastpos[0] = tp_dev.x[0];
                    lastpos[1] = tp_dev.y[0];
                }

               lcd_draw_bline(lastpos[0], lastpos[1], tp_dev.x[0], tp_dev.y[0],
                              2, BLUE);  /* 画线 */
                lastpos[0] = tp_dev.x[0];
                lastpos[1] = tp_dev.y[0];

                if (pcnt < 200) /* 总点数少于 200 */
                {
                    if (pcnt)
                    {
                        if ((ncr_input_buf[pcnt - 1].y != tp_dev.y[0])
                         && (ncr_input_buf[pcnt - 1].x
                         != tp_dev.x[0])) /* x,y 不相等 */
                        {
                            ncr_input_buf[pcnt].x = tp_dev.x[0];
                            ncr_input_buf[pcnt].y = tp_dev.y[0];
                            pcnt++;
                        }
                    }
                    else
                    {
                        ncr_input_buf[pcnt].x = tp_dev.x[0];
                        ncr_input_buf[pcnt].y = tp_dev.y[0];
                        pcnt++;
                    }
                }
            }
        }
        else    /* 按键松开了 */
        {
            lastpos[0] = 0XFFFF;
            tcnt++;
            delay_ms(10);
            /* 延时识别 */
            i++;

            if (tcnt == 40)
            {
                if (pcnt)   /* 有有效的输入 */
                {
                    printf("总点数:%d\r\n", pcnt);
                    alientek_ncr(ncr_input_buf, pcnt, 6, mode, sbuf);
                    printf("识别结果:%s\r\n", sbuf);
                    pcnt = 0;
                    lcd_show_string(60 + 72, 90, 200, 16, 16, sbuf, BLUE);
                }

                lcd_fill(20, 115, lcddev.width - 20 - 1,
```

```
                        lcddev.height - 5 - 1, WHITE);
                }
            }

            if (i == 30)
            {
                i = 0;
                LED0_TOGGLE();
            }
        }
}
```

main 函数代码实现手写识别功能的步骤跟前面所说的一致的。其中使用到了 lcd_draw_bline 函数，该函数是用来画粗线的，函数的实现也是通过调用 lcd_fill_circle 实现，这里就不做展开讲解该函数了。在获取触点数据需要注意的是：这里我们采用的都是不重复的点阵（即相邻的坐标不相等）。这样可以避免重复数据，而重复的点阵数对于识别是没有帮助的。

66.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示的内容如图 66.4.1 所示：

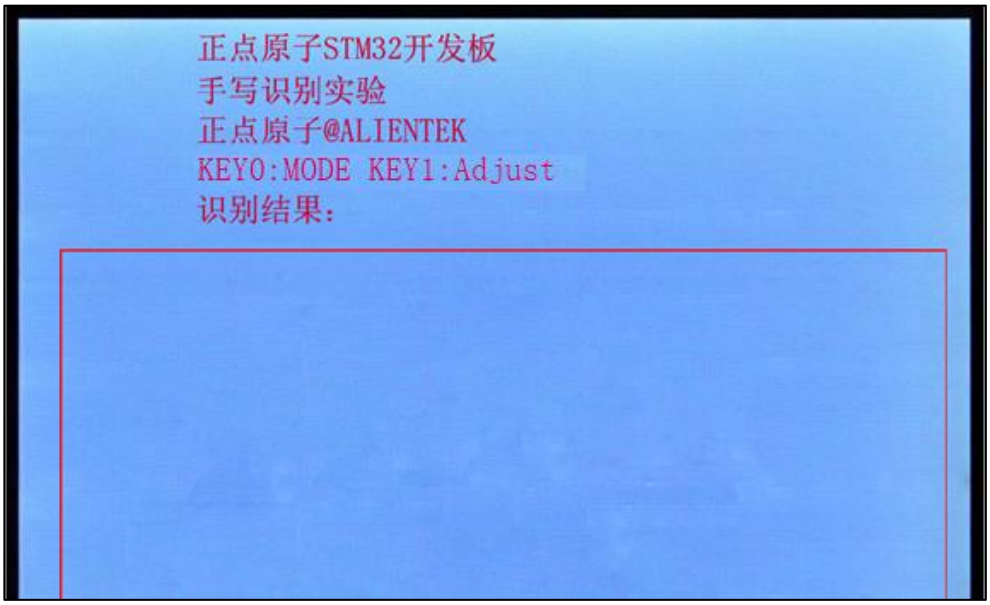


图 66.4.1 手写识别界面图

此时，我们在手写区写数字/字母，即可得到识别结果，如图 66.4.2 所示：

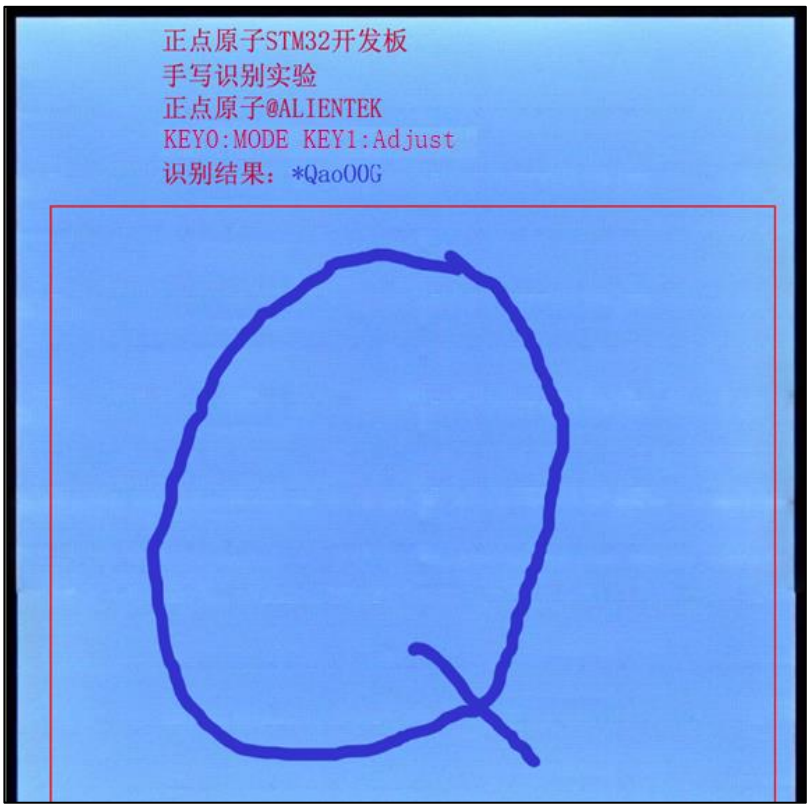


图 66.4.2 手写识别结果图

按下 KEY0 可以切换识别模式，同时在识别区提示当前模式。按下 KEY1 可以对屏幕进行校准（仅限电阻屏，电容屏无需校准）。每次识别结束，会在串口打印本次识别的输入点数和识别结果，大家可以通过串口助手查看。

第六十七章 T9 拼音输入法实验

上一章，我们在阿波罗 STM32 开发板上实现了手写识别输入，但是该方法只能输入数字或者字母，不能输入汉字。本章，我们将介绍如何在 STM32 板子上实现一个简单的 T9 中文拼音输入法。本章分为如下几个小节：

67.1 拼音输入法简介
67.2 硬件设计
67.3 程序设计
67.4 下载验证

67.1 拼音输入法简介

在计算机上汉字的输入法有很多种，比如拼音输入法、五笔输入法、笔画输入法、区位输入法等。其中，又以拼音输入法用的最多。拼音输入法又可以分为很多类，比如全拼输入、双拼输入等。

在手机上，用得最多的就是 T9 拼音输入法了，T9 输入法全名为智能输入法，字库容量九千多字，支持十多种语言。T9 输入法是由美国特捷通讯（Tegic Communications）软件公司开发的，该输入法解决了小型掌上设备的文字输入问题，已经成为全球手机文字输入的标准之一。

一般情况下，手机拼音输入键盘如图 67.1.1 所示：

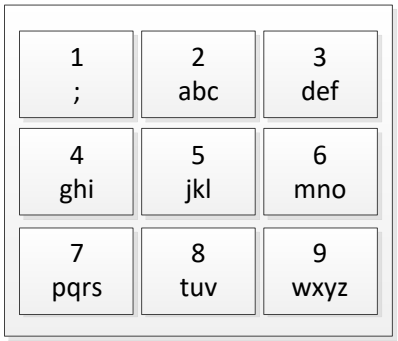


图 67.1.1 手机拼音输入键盘

在这个键盘上，我们对比下传统的输入法和 T9 输入法，输入“中国”两个字需要的按键次数。传统的方法，先按 4 次 9，输入字母 z，再按 2 次 4，输入字母 h，再按 3 次 6，输入字母 o，再按 2 次 6，输入字母 n，最后按 1 次 4，输入字母 g。这样，输入“中”字，要按键 12 次，接着同样的方法，输入“国”字，需要按 6 次，总共就是 18 次按键。

如果是 T9，我们输入“中”字，只需要输入：9、4、6、6、4，即可实现输入“中”字，在选择“中”字之后，T9 会联想出一系列同“中”字组合的词，如：文、国、断、山等。这样输入“国”字，我们直接选择即可，所以输入“国”字按键 0 次，这样使用 T9 输入法总共只需要 5 次按键。

这就是 T9 智能输入法的优越之处。正因为 T9 输入法高效便捷的输入方式得到了众多手机厂商的采用，以至于 T9 成为了使用频率最高知名度最大的手机输入法。

在本实验中，我们实现的 T9 拼音输入法，没有真正的 T9 那么强大，我们这里仅实现输入部分，不支持词组联想。

67.2 硬件设计

1. 例程功能

开机的时候先检测字库，然后显示提示信息和绘制拼音输入表，之后进入等待输入状态。此时用户可以通过屏幕上的拼音输入表输入拼音数字串（通过 DEL 可以实现退格），然后程序

自动检测与之对应的拼音和汉字，并显示在屏幕上（同时输出到串口）。如果有多个匹配的拼音，则通过 KEY0 进行选择。按键 KEY1 用于清除一次输入，按键 WK_UP 用于触摸屏校准。LED0 闪烁用于提示程序正在运行。

2. 硬件资源

1）LED 灯
LED ： LED0 - PB1
2）独立按键
KEY0 - PH3
KEY1 - PH2
WK_UP - PA0
3）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)
5）QSPI(PB2/PB6/PF6/PF7/PF8/PF9)
6）norflash(QSPI FLASH 芯片,连接在 QSPI 上)
7）触摸屏(电阻式/电容式)

67.3 程序设计

67.3.1 程序流程图

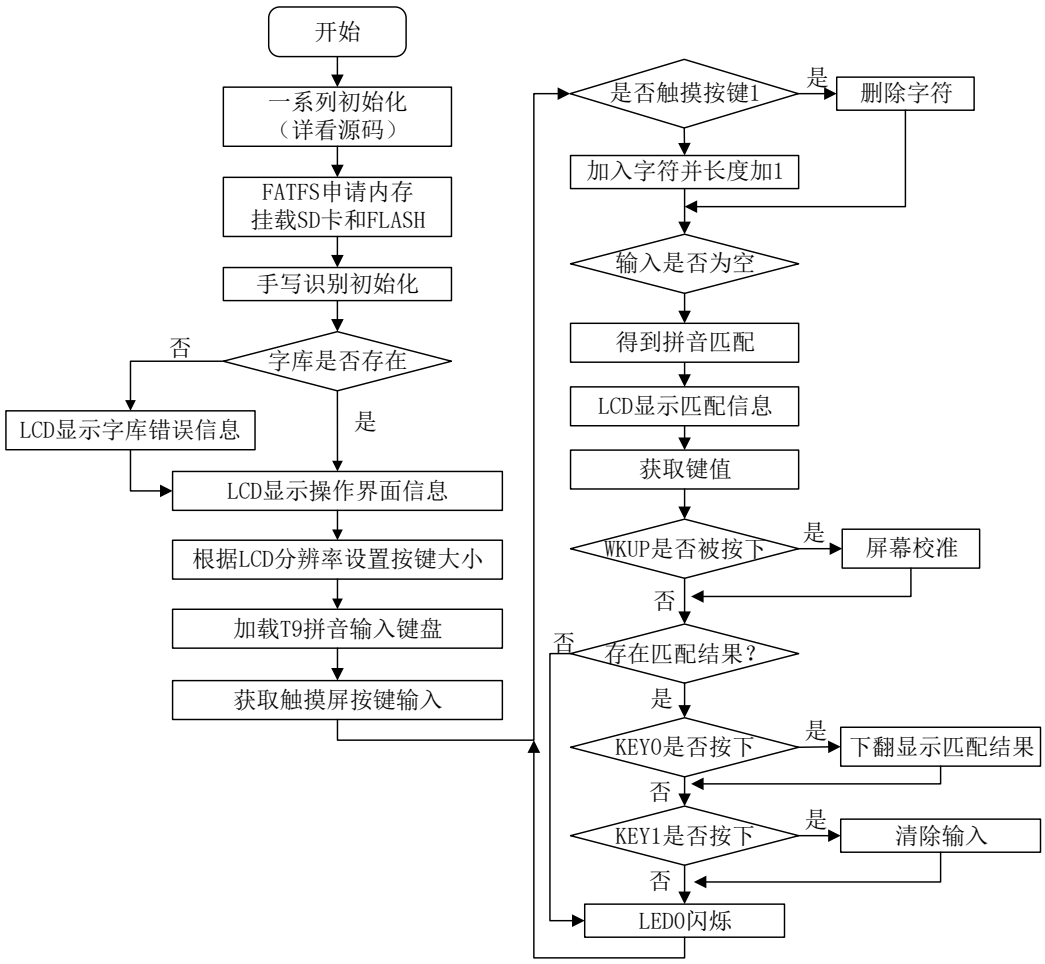


图 67.3.1.1 T9 拼音输入法实验程序流程图

67.3.2 程序解析

1. T9INPUT 代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。T9INPUT 驱动源码包括三个文件：pyinput.c、pyinput.h 和 pymb.h。

首先我们先介绍一下定义在 pymb.h 中的拼音索引。先介绍一下汉字排列表，该表将汉字拼音所有可能的组合都列出来了，如下所示：

```
/* 汉字排列表 */
const uint8_t PY_mb_space []={""};
const uint8_t PY_mb_a     []={"啊阿腌吖锕屙嘎錒呵腌"};
const uint8_t PY_mb_ai    []={"爱埃挨哎唉哀皑癌蔼矮艾碍隘捱嗳嗌嫒瑷暧砹锿霭"};
……此处省略 N 多个组合
const uint8_t PY_mb_zu    []={"足组卒族租祖诅阻俎菹镞"};
const uint8_t PY_mb_zuan  []={"钻攥纂缵躜"};
const uint8_t PY_mb_zui   []={"最罪嘴醉蕞觜"};
const uint8_t PY_mb_zun   []={"尊遵樽鳟撙"};
const uint8_t PY_mb_zuo   []={"左佐做作坐座昨撮唑柞阼琢嘬怍胙祚砟酢"};
```

这里我们只列出了部分组合，我们将这些组合称之为码表，然后将这些码表和其对应的数字串对应起来，组成一个拼音索引表，如下所示：

```
/* 拼音索引表 */
const py_index py_index3[]=
{
{"" ,"",(uint8_t*)PY_mb_space},
{"2","a",(uint8_t*)PY_mb_a},
{"3","e",(uint8_t*)PY_mb_e},
{"6","o",(uint8_t*)PY_mb_o},
{"24","ai",(uint8_t*)PY_mb_ai},
……此处省略 N 多个组合
{"94664","zhong",(uint8_t*)PY_mb_zhong},
{"94824","zhuai",(uint8_t*)PY_mb_zhuai},
{"94826","zhuan",(uint8_t*)PY_mb_zhuan},
{"248264","chuang",(uint8_t*)PY_mb_chuang},
{"748264","shuang",(uint8_t*)PY_mb_shuang},
{"948264","zhuang",(uint8_t*)PY_mb_zhuang},
};
```

其中 py_index 是一个结构体，定义如下：

```
/* 拼音码表与拼音的对应表 */
typedef struct
{
  uint8_t *py_input;    /* 输入的字符串 */
  uint8_t *py;          /* 对应的拼音 */
  uint8_t *pymb;        /* 码表 */
}py_index;
```

其中 py_input，即与拼音对应的数字串，比如“94824”。py，即与 py_input 数字串对应的拼音，如果 py_input = “94824”，那么 py 就是“zhuai”。最后 pymb 就是我们前面说到的码表。注意，一个数字串可以对应多个拼音，也可以对应多个码表。

有了这个拼音索引表（py_index3）之后，我们只需要将输入的数字串和 py_index3 索引表里面所有成员的 py_input 对比，将所有完全匹配的情况记录下来，用户要输入的汉字就被确定了，然后由用户选择可能的拼音组成（假设有多个匹配的项目），再选择对应的汉字，即完成一次汉字输入。

当然还可能是找遍了索引表，也没有发现一个完全符合要求的成员，那么我们会统计匹配数最多的情况，作为最佳结果，反馈给用户。比如，用户输入“323”，找不到完全匹配的情况，那么我们就将能和“32”匹配的结果返回给用户。这样，用户还是可以得到输入结果，同时还可以知道输入有问题，提示用户需要检查输入是否正确。

我们归纳一下一个完整的 T9 拼音输入步骤：

1） 输入拼音数字串

我们用到的 T9 拼音输入法的核心思想就是对比用户输入的拼音数字串，所以必须先由用户输入拼音数字串。

2） 在拼音索引表里面查找和输入字符串匹配的项，并记录

在得到用户输入的拼音数字串之后，在拼音索引表里面查找所有匹配的项目，如果有完全匹配的项目，就全部记录下来，如果没有完全匹配的项目，则记录匹配情况最好的一个项目。

3） 显示匹配清单里面所有可能的文字，供用户选择

将匹配项目的拼音和对应的汉字显示出来，供用户选择。如果有多个匹配项（一个数字串对应多个拼音的情况），则用户还需要选择拼音。

4） 用户选择匹配项，并选择对应的汉字

用户对匹配的拼音和汉字进行选择，选中其真正想输入的拼音和汉字，实现一次拼音输入。

下面介绍一下 pyinput.c 中比较核心的函数 get_matched_pymb，代码如下：

```
/**
 * @brief       获取匹配的拼音码表
 * @param       strin     : 输入的字符串,形如:"726"
 * @param       matchlist : 输出的匹配表
 * @retval      匹配状态
 *              [7]   , 0,表示完全匹配; 1, 表示部分匹配（仅在没有完全匹配的时候才会出现）
 *              [6:0], 完全匹配的时候，表示完全匹配的拼音个数
 *                     部分匹配的时候，表示有效匹配的位数
 */
uint8_t get_matched_pymb(uint8_t *strin, py_index **matchlist)
{
    py_index *bestmatch = 0;     /* 最佳匹配 */
    uint16_t pyindex_len = 0;
    uint16_t i = 0;
    uint8_t temp, mcnt = 0, bmcnt = 0;

    bestmatch = (py_index *)&py_index3[0];  /* 默认为 a 的匹配 */
    pyindex_len = sizeof(py_index3) / sizeof(py_index3[0]);/*得到 py 索引表的大小*/

    for (i = 0; i < pyindex_len; i++)
    {
        temp = str_match(strin, (uint8_t *)py_index3[i].py_input);

        if (temp)
        {
            if (temp == 0XFF)
            {
                matchlist[mcnt++] = (py_index *)&py_index3[i];
            }
            else if (temp > bmcnt)  /* 找最佳匹配 */
            {
                bmcnt = temp;
                bestmatch = (py_index *)&py_index3[i];  /* 最好的匹配 */
            }
        }
    }

    if (mcnt == 0 && bmcnt)      /* 没有完全匹配的结果,但是有部分匹配的结果 */
    {
        matchlist[0] = bestmatch;
        mcnt = bmcnt | 0X80;     /* 返回部分匹配的有效位数 */
    }
    return mcnt;     /* 返回匹配的个数 */
}
```

该函数实现的是将用户输入拼音数字串同拼音索引表里面的各个项对比，找出匹配结果，并将完全匹配的项目存放在 matchlist 里面，同时记录匹配数。对于那些没有完全匹配的输入串，则查找与其最佳匹配的项目，并将匹配的长度返回。

其中该文件还有一个函数 test_py，提供给 usmart 调用，实现串口测试，在串口测试的时候才能用到，如果不使用的话，可以去掉。本实验也是加入 usmart 控制，大家可以通过该函数实现串口调试拼音输入法。

其他两个函数比较简单，这里就不细说了。

前面提及的 matchlist，其定义在 pyinput.h 中，代码如下：

```
/* 拼音输入法 */
typedef struct
{
   uint8_t(*getpymb)(uint8_t *instr);       /* 字符串到码表获取函数 */
   py_index *pymb[MAX_MATCH_PYMB];          /* 码表存放位置 */
}pyinput;
```

该结构体提供了两个成员，一个成员就是字符串到码表获取函数，另外一个成员也就是码表的存放位置。另外一个结构体 py_index，已经在前面讲解了，这里就不作展开。

2. main.c 代码

在 main.c 文件下，除了 main 函数之外，还有 py_load_ui、py_key_staset、py_get_keynum 和 py_show_result 函数。其中，py_load_ui 函数是用于加载输入键盘，在 LCD 上显示我们输入拼音数字串的虚拟键盘。py_key_staset 函数用于与设置虚拟键盘某个按键的状态（按下/松开）。py_get_keynum 函数用于得到触摸屏当前按下的按键键值，通过该函数实现拼音数字串的获取。py_show_result 函数用于显示输入串的匹配结果，并将结果打印到串口。这部分代码就不列出来了，大家可以自行理解。

下面看一下 main 主函数的代码：

```
int main(void)
{
    uint8_t i = 0;
    uint8_t result_num;
    uint8_t cur_index;
    uint8_t key;
    uint8_t inputstr[7];                    /* 最大输入 6 个字符+结束符 */
    uint8_t inputlen;                       /* 输入长度 */

    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    key_init();                             /* 初始化按键 */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    norflash_init();                        /* 初始化 norflash */
    tp_dev.init();                          /* 初始化触摸屏 */
    my_mem_init(SRAMIN);                    /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                    /* 初始化外部内存池 */
    my_mem_init(SRAMDTCM);                  /* 初始化 DTCM 内存池(DTCM) */
    exfuns_init();                          /* 为 fatfs 相关变量申请内存 */
    f_mount(fs[0], "0:", 1);                /* 挂载 SD 卡 */
    f_mount(fs[1], "1:", 1);                /* 挂载 SPI FLASH */
    f_mount(fs[2], "2:", 1);                /* 挂载 SPI FLASH */

    while (fonts_init())                    /* 检查字库 */
    {
        lcd_show_string(30, 90, 200, 16, 16, "Font Error!", RED);
        delay_ms(200);
        lcd_fill(30, 90, 200+20, 90+16, WHITE);    /* 清除显示 */
        delay_ms(200);
```

```
    }

RESTART:
    text_show_string(30, 5, 200, 16, "正点原子 STM32 开发板", 16, 0, RED);
    text_show_string(30, 25, 200, 16, "拼音输入法实验", 16, 0, RED);
    text_show_string(30, 45, 200, 16, "ATOM@ALIENTEK", 16, 0, RED);
    text_show_string(30, 65, 200, 16, "KEY_UP:校准", 16, 0, RED);
    text_show_string(30, 85, 200, 16, "KEY0:翻页  KEY1:清除", 16, 0, RED);
    text_show_string(30, 105, 200, 16, "输入:         匹配:  ", 16, 0, RED);
    text_show_string(30, 125, 200, 16, "拼音:         当前:  ", 16, 0, RED);
    text_show_string(30, 145, 210, 32, "结果:", 16, 0, RED);

    /* 根据 LCD 分辨率设置按键大小 */
    if (lcddev.id == 0X5310)
    {
        kbdxsize = 86;
        kbdysize = 43;
    }
    else if (lcddev.id == 0X5510 || lcdltdc.pheight >= 480)
    {
        kbdxsize = 140;
        kbdysize = 70;
    }
    else
    {
        kbdxsize = 60;
        kbdysize = 40;
    }

    py_load_ui(30, 195);
    my_mem_set(inputstr, 0, 7); /* 全部清零 */
    inputlen = 0;               /* 输入长度为 0 */
    result_num = 0;             /* 总匹配数清零 */
    cur_index = 0;

    while (1)
    {
        i++;
        delay_ms(10);
        key = py_get_keynum(30, 195);
        if (key)
        {
            if (key == 1)                       /* 删除 */
            {
                if (inputlen)inputlen--;
                inputstr[inputlen] = '\0';      /* 添加结束符 */
            }
            else
            {
                inputstr[inputlen] = key + '0'; /* 输入字符 */
                if (inputlen < 7)inputlen++;
            }

            if (inputstr[0] != NULL)
            {
                key = t9.getpymb(inputstr); /* 得到匹配的结果数 */
                if (key)                    /* 有部分匹配/完全匹配的结果 */
                {
                    result_num = key & 0X7F;    /* 总匹配结果 */
                    cur_index = 1;              /* 当前为第一个索引 */
                    if (key & 0X80)             /* 是部分匹配 */
                    {
                        inputlen = key & 0X7F;  /* 有效匹配位数 */
```

```
                        inputstr[inputlen] = '\0';  /* 不匹配的位数去掉 */
                        /* 重新获取完全匹配字符数 */
                        if (inputlen > 1)result_num = t9.getpymb(inputstr);
                    }
                }
                else    /* 没有任何匹配 */
                {
                    inputlen--;
                    inputstr[inputlen] = '\0';
                }
            }
            else
            {
                cur_index = 0;
                result_num = 0;
            }
            /* 清除之前的显示 */
            lcd_fill(30 + 40, 105, 30 + 40 + 48, 110 + 16, WHITE);
            /* 显示匹配的结果数 */
            lcd_show_num(30 + 144, 105, result_num, 1, 16, BLUE);
            text_show_string(30 + 40, 105, 200, 16, (char *)inputstr,
                             16, 0, BLUE); /* 显示有效的数字串 */
            py_show_result(cur_index);    /* 显示第 cur_index 的匹配结果 */
        }
        key = key_scan(0);
        if (key == WKUP_PRES && tp_dev.touchtype == 0)  /* WKUP 按下,且是电阻屏 */
        {
            tp_dev.adjust();
            lcd_clear(WHITE);
            goto RESTART;
        }
        if (result_num) /* 存在匹配的结果 */
        {
            switch (key)
            {
                case KEY0_PRES: /* 下翻 */
                    if (cur_index < result_num)cur_index++;
                    else cur_index = 1;

                    py_show_result(cur_index);  /* 显示第 cur_index 的匹配结果 */
                    break;

                case KEY1_PRES: /* 清除输入 */
                    /* 清除之前的显示 */
                    lcd_fill(30 + 40, 145, lcddev.width - 1, 145 + 48, WHITE);
                    goto RESTART;
            }
        }
        if (i == 30)
        {
            i = 0;
            LED0_TOGGLE();
        }
    }
}
```

在 main 函数里，实现了我们在 67.2.1 小节所说的功能，也是按照它表述的逻辑进行实现。在这里我们并没有实现汉字选择功能，但是由本例程作为基础，再实现汉字选择功能就比较简单了，大家自行实现即可。注意：kbdxsize 和 kbdysize 代表虚拟键盘按键宽度和高度，程序根据 LCD 分辨率不同而自动设置这两个参数，以达到较好的输入效果。

67.4 下载验证

将程序下载到开发板后，可以看到 LED0 不停的闪烁，提示程序已经在运行了。LCD 显示

的内容如图 67.4.1 所示：

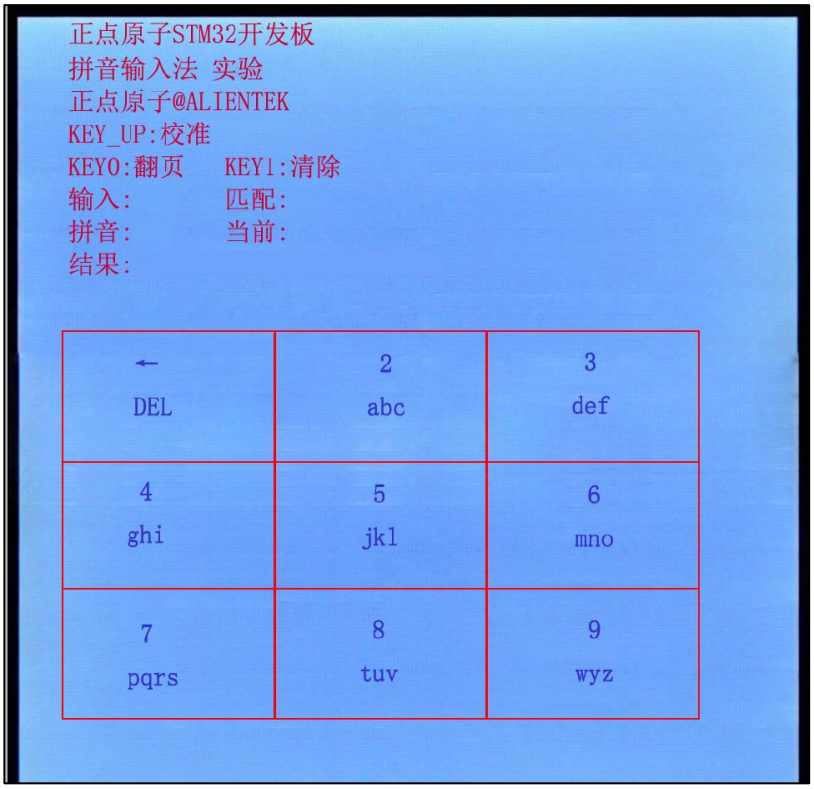


图 67.4.1 汉字输入法界面图

此时，我们在虚拟键盘上输入拼音数字串，即可实现拼音输入，如图 67.4.2 所示：

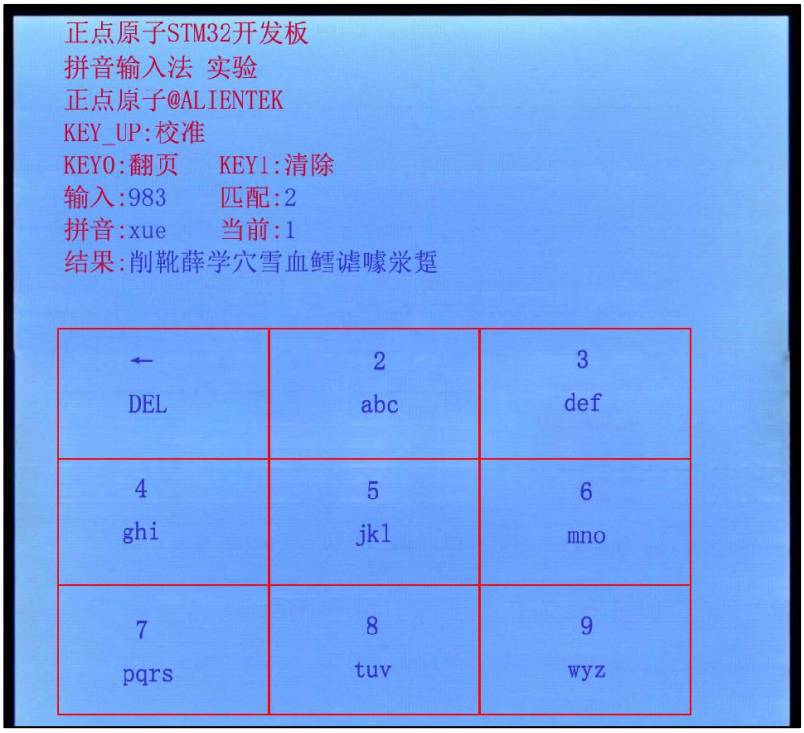


图 67.4.2 实现拼音输入

如果发现输入错误了，可以通过屏幕上的 DEL 按键，来退格。如果有多个匹配的情况（匹配值大于 1），则可以通过 KEY0 来选择拼音。通过按下 KEY1，可以清除当前输入；通过按下 KEY_UP，可以实现触摸屏校准（仅限电阻屏，电容屏无需校准）。

本章，我们还可以借助 USMART 调用 test_py 来实现输入法调试，如图 67.4.3 所示：

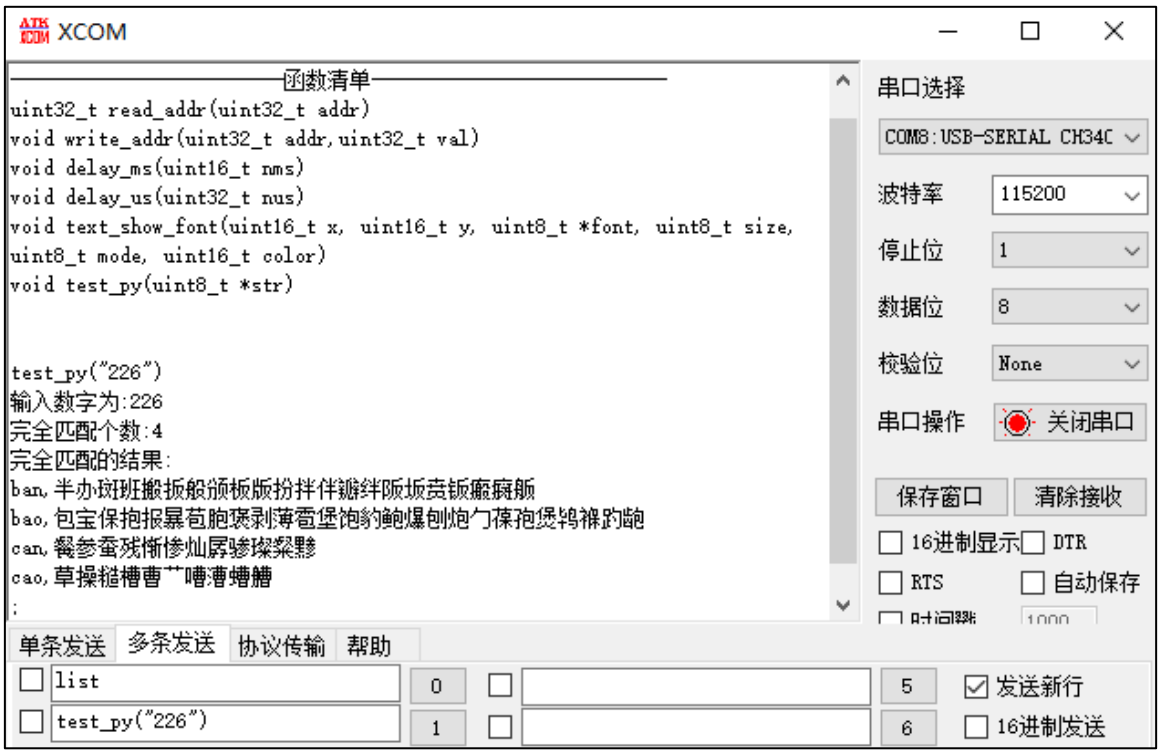


图 67.4.2 USMART 测试 T9 拼音输入法图

第六十八章 串口 IAP 实验

IAP，即在应用编程，通俗地说法就是“程序升级”。产品阶段设计完成后，在脱离实验室的调试环境下，如果想对产品做功能升级或 BUG 修复会十分麻烦，如果硬件支持，在出厂时预留一套升级固件的流程，就可以很好解决这个问题，IAP 技术就是为此而生的。在之前的 FLASH 模拟 EEPROM 实验里面，我们学习了 STM32H743 的 FLASH 自编程，本章我们将结合 FLASH 自编程的知识，通过 STM32H743 的串口实现一个简单的 IAP 功能。

本章分为如下几个小节：

68.1 IAP 简介
68.2 硬件设计
68.3 程序设计
68.4 下载验证

68.1 IAP 简介

IAP（In Application Programming）即在应用编程。IAP 是用户自己的程序在运行过程中对 User Flash 的部分区域进行烧写，目的是为了在产品发布后可以方便地通过预留的通信口对产品中的固件程序进行更新升级。通常实现 IAP 功能时，即用户程序运行中作自身的更新操作，需要在设计固件程序时编写两个项目代码，第一个项目程序不执行正常的功能操作，而只是通过某种通信方式(如 USB、USART)接收程序或数据，执行对第二部分代码的更新；第二个项目代码才是真正的功能代码。这两部分项目代码都同时烧录在 User Flash 中，当芯片上电后，首先是第一个项目代码开始运行，它做如下操作：

1）检查是否需要对第二部分代码进行更新
2）如果不需要更新则转到 4）
3）执行更新操作
4）跳转到第二部分代码执行

第一部分代码必须通过其它手段，如 JTAG 或 ISP 烧入；第二部分代码可以使用第一部分代码 IAP 功能烧入，也可以和第一部分代码一起烧入，以后需要程序更新时再通过第一部分 IAP 代码更新。

我们将第一个项目代码称之为 Bootloader 程序，第二个项目代码称之为 APP 程序，他们存放在 STM32H743 FLASH 的不同地址范围，一般从最低地址区开始存放 Bootloader，紧跟其后的就是 APP 程序（注意，如果 FLASH 容量足够，是可以设计很多 APP 程序的，本章我们只讨论一个 APP 程序的情况）。这样我们就是要实现 2 个程序：Bootloader 和 APP。

STM32H7 的 APP 程序不仅可以放到 FLASH 里面运行，也可以放到 SRAM 里面运行，本章，我们将制作两个 APP，一个用于 FLASH 运行，一个用于内部 SRAM 运行。

我们先来看看 STM32H7 正常的程序运行流程（为了方便说明 IAP 过程，我们先仅考虑代码全部存放在内部 FLASH 的情况），如图 68.1.1 所示：

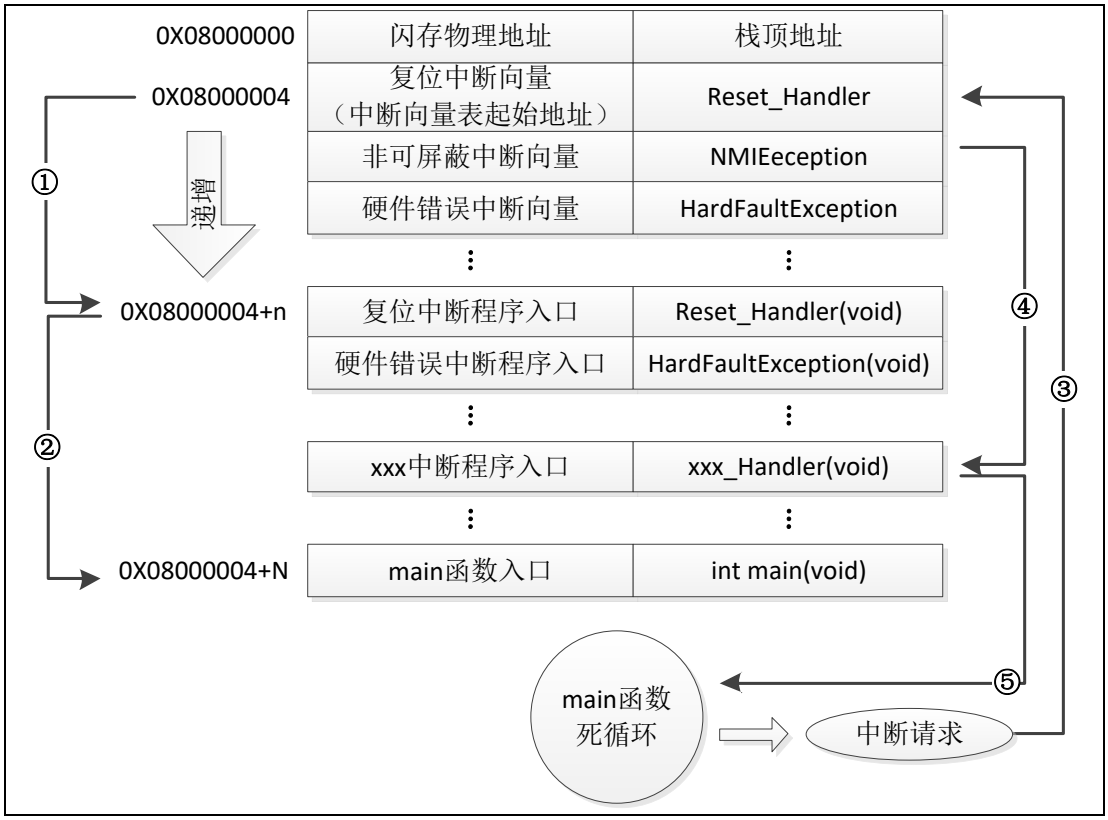


图 68.1.1 STM32H743 正常运行流程图

STM32H7 的内部闪存（FLASH）地址起始于 0X0800 0000，一般情况下，程序文件就从此地址开始写入。此外 STM32H743 是基于 Cortex-M7 内核的微控制器，其内部通过一张“中断向量表”来响应中断，程序启动后，将首先从“中断向量表”取出复位中断向量执行复位中断程序完成启动，而这张“中断向量表”的起始地址是 0x08000004，当中断来临，STM32H743 的内部硬件机制亦会自动将 PC 指针定位到“中断向量表”处，并根据中断源取出对应的中断向量执行中断服务程序。

在图 68.1.1 中，STM32H743 在复位后，先从 0X08000004 地址取出复位中断向量的地址，并跳转到复位中断服务程序，如图标号①所示；在复位中断服务程序执行完之后，会跳转到我们的 main 函数，如图标号②所示；而我们的 main 函数一般都是一个死循环，在 main 函数执行过程中，如果收到中断请求（发生了中断），此时 STM32H743 强制将 PC 指针指回中断向量表处，如图标号③所示；然后，根据中断源进入相应的中断服务程序，如图标号④所示；在执行完中断服务程序以后，程序再次返回 main 函数执行，如图标号⑤所示。

当加入 IAP 程序之后，程序运行流程如图 68.1.2 所示：

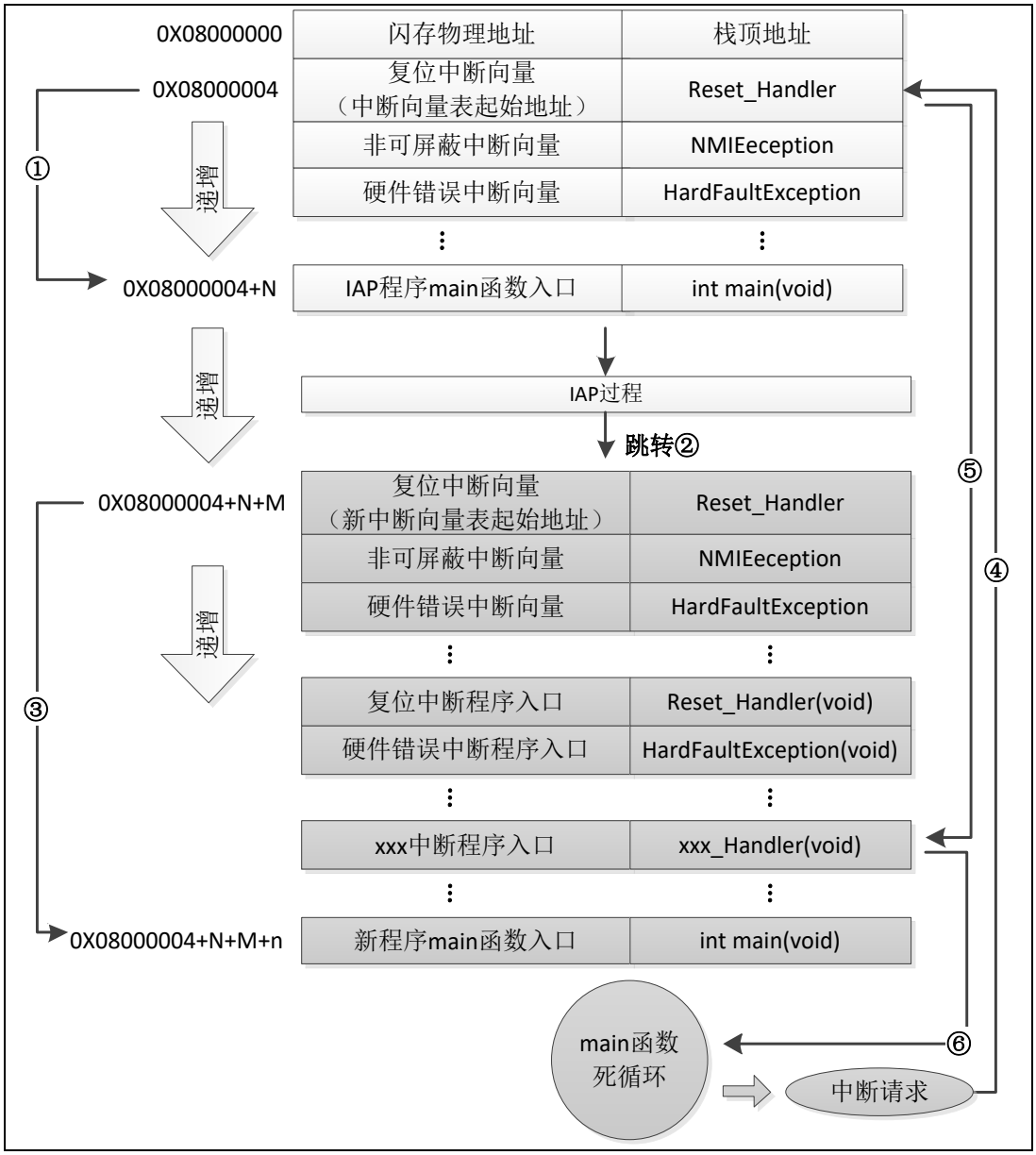


图 68.1.2 加入 IAP 之后程序运行流程图

在图 68.1.2 所示流程中，STM32H743 复位后，还是从 0X08000004 地址取出复位中断向量的地址，并跳转到复位中断服务程序，在运行完复位中断服务程序之后跳转到 IAP 的 main 函数，如图标号①所示，此部分同图 68.1.1 一样；在执行完 IAP 以后（即将新的 APP 代码写入 STM32H743 的 FLASH，灰底部分。新程序的复位中断向量起始地址为 0X08000004+N+M），跳转至新写入程序的复位向量表，取出新程序的复位中断向量的地址，并跳转执行新程序的复位中断服务程序，随后跳转至新程序的 main 函数，如图标号②和③所示，同样 main 函数为一个死循环，并且注意到此时 STM32H743 的 FLASH，在不同位置上，共有两个中断向量表。

在 main 函数执行过程中，如果 CPU 得到一个中断请求，PC 指针仍然会强制跳转到地址 0X08000004 中断向量表处，而不是新程序的中断向量表，如图标号④所示；程序再根据我们设置的中断向量表偏移量，跳转到对应中断源新的中断服务程序中，如图标号⑤所示；在执行完中断服务程序后，程序返回 main 函数继续运行，如图标号⑥所示。

通过以上两个过程的分析，我们知道 IAP 程序必须满足两个要求：

1） 新程序必须在 IAP 程序之后的某个偏移量为 x 的地址开始；

2） 必须将新程序的中断向量表相应的移动，移动的偏移量为 x；

本章，我们有 2 个 APP 程序：

1，FLASH APP 程序，即只运行在内部 FLASH 的 APP 程序。

2，SRAM APP 程序，图 68.1.2 虽然是针对 FLASH APP 来说的，但是在 SRAM 里面运行的过程和 FLASH 基本一致，只是需要设置向量表的地址为 SRAM 的地址。

1.APP 程序起始地址设置方法

我们随便打开一个之前的实例工程，点击 Options for Target→Target 选项卡，如图 68.1.3 所示：

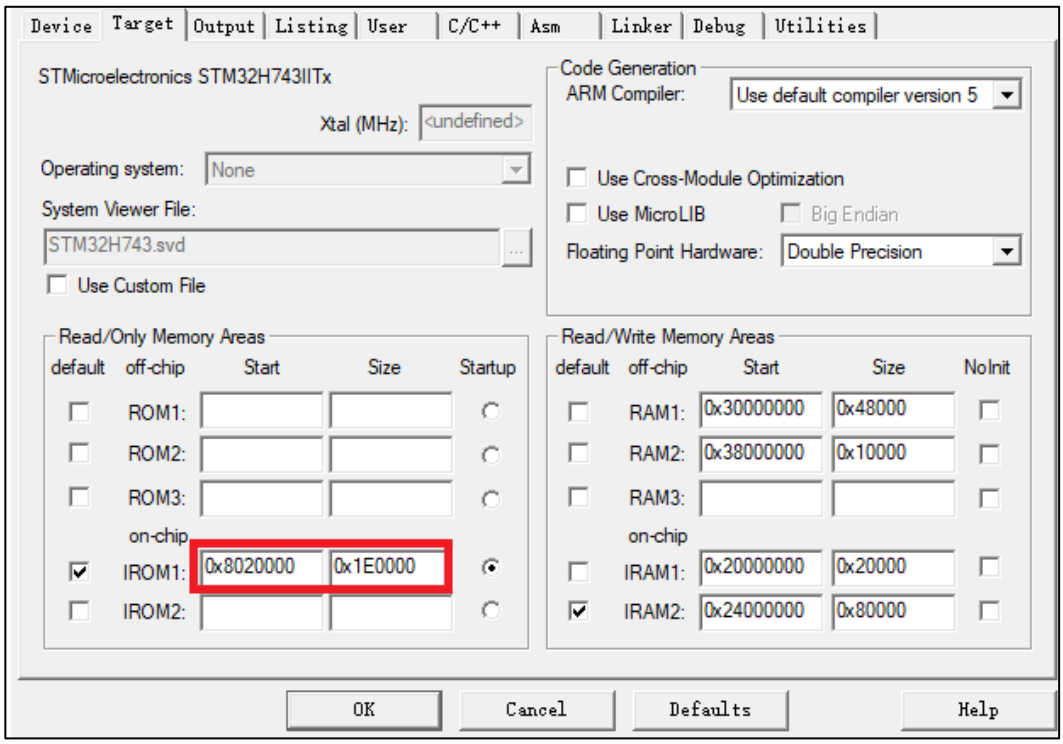


图 68.1.3 FLASH APP Target 选项卡设置

默认的条件下，图中 IROM1 的起始地址（Start）一般为 0X08000000，大小（Size）为 0X200000，即从 0X08000000 开始的 2048K 空间为我们的程序存储区。而图中，我们设置起始地址（Start）为 0X08020000，即偏移量为 0X20000（128K 字节），因而，留给 APP 用的 FLASH 空间（Size）只有 0X200000-0X20000=0X1E0000（1920K 字节）大小了。设置好 Start 和 Size，就完成 APP 程序的起始地址设置。

这里的 128K 字节，一般是需要大家根据 Bootloader 程序大小进行选择，比如我们本章的 Bootloader 程序为 39K 左右，理论上我们只需要确保 APP 起始地址在 Bootloader 之后，并且偏移量为 0X200 的倍数即可（相关知识，请参考：http://www.openedv.com/posts/list/392.htm）。**但是，由于 STM32H7 的扇区大小固定为 128KB 一个，最小擦除单位也是以扇区为单位的，因此，至少分配一个扇区给 Bootloader，否则后续擦写 APP 的时候，可能把 Bootloader 也给擦了！**所以，我们使用 128K（0X20000）字节来存放 Bootloader。

这是针对 FLASH APP 的起始地址设置，如果是 SRAM APP，那么起始地址设置如图 68.1.4 所示：

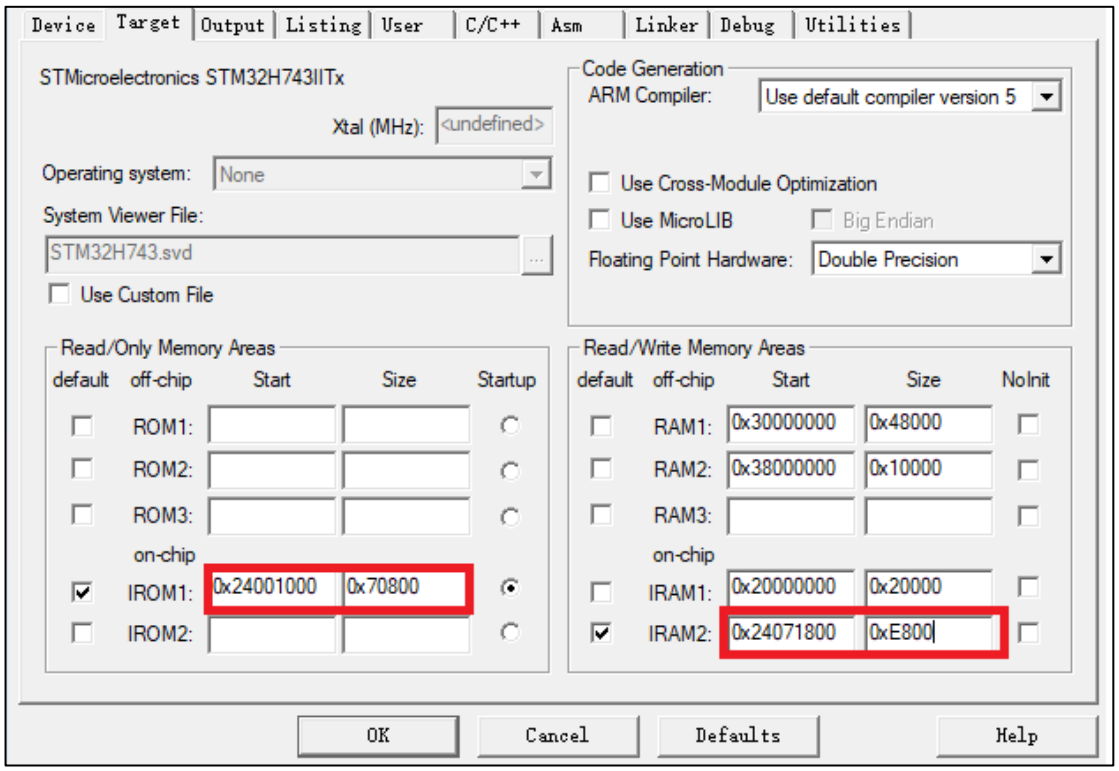


图 68.1.4 SRAM APP Target 选项卡设置

这里我们将 IROM1 的起始地址（Start）定义为：0X24001000，大小为 0X70800（450K 字节），即从地址 0X24000000 偏移 0X1000 开始，存放 APP 代码。这个分配关系大家可以根据自己的实际情况修改，不一定和我们这里的设置一模一样，不过也需要注意，保证偏移量为 0X200 的倍数（我们这里为 0X1000）。IRAM1 的起始地址（Start）定义为：0X24071800，大小为 0XE800（58K 字节）。这样，整个 STM32H743IIT6 的 AXI SRAM 的分配情况为：最开始的 4K 给 Bootloader 程序使用，随后的 450K 存放 APP 程序，最后 58K，用作 APP 程序的内存。

2.中断向量表的偏移量设置方法

前面 11.2.1 小节，我们介绍过 SystemInit 函数，在系统启动的时候，会首先调用 SystemInit 函数初始化时钟系统，同时 SystemInit 还完成了中断向量表的设置，我们可以打开 SystemInit 函数，看到函数体的结尾处有这样几行代码：

```
#ifdef VECT_TAB_SRAM
  SCB->VTOR = D1_AXISRAM_BASE  | VECT_TAB_OFFSET;
                        /* Vector Table Relocation in Internal D1 AXI-RAM */
#else
  SCB->VTOR = FLASH_BANK1_BASE | VECT_TAB_OFFSET;
                        /* Vector Table Relocation in Internal FLASH */
#endif
```

从代码可以理解，VTOR 寄存器存放的是中断向量表的起始地址。默认的情况 VECT_TAB_SRAM 是没有定义，所以执行 SCB->VTOR = FLASH_BANK1_BASE | VECT_TAB_OFFSET；对于 FLASH APP，我们设置为 FLASH_BASE+偏移量 0x10000，所以我们可以在 SystemInit 函数里面修改 SCB->VTOR 的值。为了尽可能不修改系统级别文件，我们也可以在 FLASH APP 的 main 函数最开头处添加如下代码实现中断向量表的起始地址的重设：

```
    SCB->VTOR = FLASH_BASE | 0x20000;
```

以上是 FLASH APP 的情况，当使用 SRAM APP 的时候，我们设置起始地址为：SRAM_BASE+0x1000，同样的方法，我们在 SRAM APP 的 main 函数

```
    SCB->VTOR = FLASH_BASE | 1000;
```

这样，我们就完成了中断向量表偏移量的设置。

通过以上两个步骤的设置，我们就可以生成 APP 程序了，只要 APP 程序的 FLASH 和 SRAM 大小不超过我们的设置即可。不过 MDK 默认生成的文件是.hex 文件，并不方便我们用

作 IAP 更新，我们希望生成的文件是.bin 文件，这样可以方便进行 IAP 升级（至于为什么，请大家自行百度 HEX 和 BIN 文件的区别！）。这里我们通过 MDK 自带的格式转换工具 fromelf.exe，来实现.axf 文件到.bin 文件的转换。该工具在 MDK 的安装目录\ARM\ARMCC\bin 文件夹里面。

fromelf.exe 转换工具的语法格式为：fromelf [options] input_file。其中 options 有很多选项可以设置，详细使用请参考光盘《mdk 如何生成 bin 文件.doc》。

本实验，我们可以通过在 MDK 点击 Options for Target→User 选项卡，在 After Build/Rebuild 一栏中，勾选 Run #1，并写入：fromelf --bin -o ../../Output/atk_h743.bin ../../Output/atk_h743.axf，如图 68.1.5 所示：

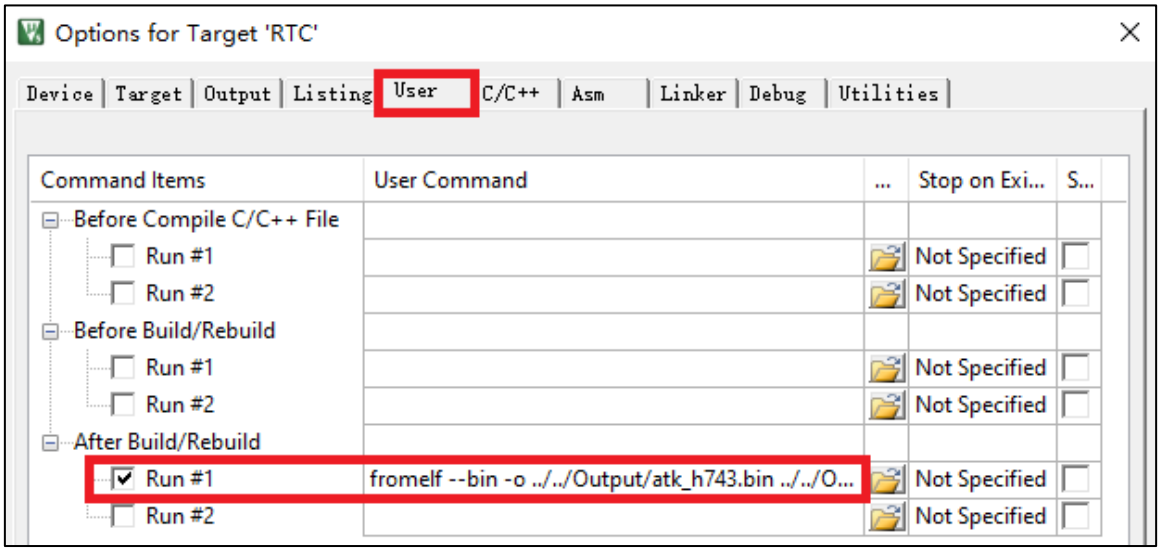


图 68.1.5 MDK 生成.bin 文件设置方法

通过这一步设置，我们就可以在 MDK 编译成功之后，调用 fromelf.exe，根据当前工程的 atk_h743.axf，生成一个 atk_h743.bin 的文件。并存放在 axf 文件相同的目录下，即工程的 Output 文件夹里面。在得到.bin 文件之后，我们只需要将这个 bin 文件传送给单片机，即可执行 IAP 升级。

68.2 硬件设计

1. 例程功能

本实验开机的时候先显示提示信息，然后等待串口输入接收 APP 程序（无校验，一次性接收），在串口接收到 APP 程序之后，即可执行 IAP。

1、对于 SRAM APP，通过按下 KEY0 即可执行这个收到的 SRAM APP 程序。

2、对于 FLASH APP，则需要按下 KEY_UP 按键，将串口接收到的 APP 程序存放到 STM32 的 FLASH，之后再按 KEY2 即可执行这个 FLASH APP 程序。通过 KEY1 按键可以手动清除串口接收到的 APP 程序。

LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
LED1：LED1 - PB0
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)
4）独立按键：KEY0 - PH3、KEY1 - PH2、WK_UP - PA0
5）QSPI(PB2/PB6/PF6/PF7/PF8/PF9)
6）norflash(QSPI FLASH 芯片，连接在 QSPI 上)

68.3 程序设计

APP 程序的生成步骤

1）设置 APP 程序的起始地址和存储空间大小

对于在 FLASH 里面运行的 APP 程序，我们只需要设置 APP 程序的起始地址，和存储空间大小即可。而对于在 SRAM 里面运行的 APP 程序，我们还需要设置 SRAM 的起始地址和大小。无论哪种 APP 程序，都需要确保 APP 程序的大小和所占 SRAM 大小不超过我们的设置范围。

2）设置中断向量表偏移量

此步，通过调用 sys_nvic_set_vector_table 函数，实现对中断向量表偏移量的设置。这个偏移量的大小，其实就等于程序起始地址相对于 0X08000000 或者 0X24000000 的偏移。

3）设置编译后运行 fromelf.exe，生成.bin 文件

通过在 User 选项卡，设置编译后调用 fromelf.exe，根据.axf 文件生成.bin 文件，用于 IAP 更新。

以上 3 个步骤，就可以得到一个.bin 的 APP 程序，通过 Bootlader 程序即可实现更新。

68.3.1 程序流程图

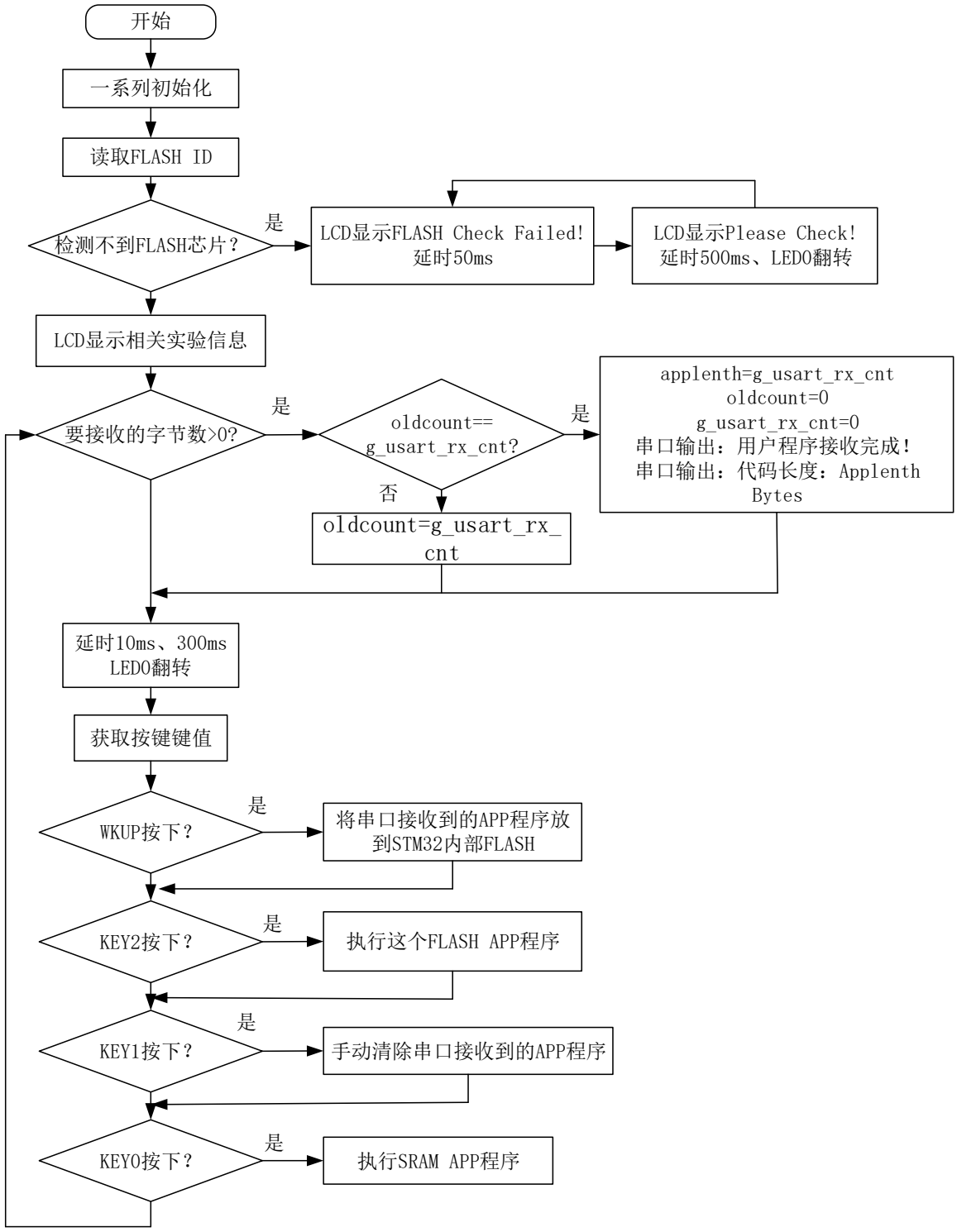


图 68.3.1.1 串口 IAP 实验程序流程图

68.3.2 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。本实验，我们总共需要 3 个程序（1，Bootloader；2，FLASH APP；3，SRAM APP）。其中，我们选择之前做过的 RTC 实验（在第二十九章介绍）来作为 FLASH APP 程序（起始地址为 0X08020000），选择触摸屏实验（在第四十三章介绍）来作为 SRAM APP 程序（起始地址为 0X24001000）。

Bootloader 则是通过 LTDC LCD（RGB 屏）显示实验（在第二十七章介绍）修改得来。

本章关于 APP 程序的生成和修改比较简单，我们就不细说，请大家结合光盘源码，以及 68.1 节的介绍，自行理解。本章程序解析小节仅针对 Bootloader 程序。

1. IAP 程序

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码，IAP 的驱动主要包括两个文件 iap.c 和 iap.h。

通过封装《实验 38 FLASH 模拟 EEPROM 实验》的驱动，我们实现 IAP 驱动的 iap.c 文件，具体的代码如下：

```
iapfun jump2app;
uint32_t g_iapbuf[512];       /* 2K 字节缓存 */

/**
 * @brief       IAP 写入 APP BIN
 * @param       appxaddr : 应用程序的起始地址
 * @param       appbuf   : 应用程序 CODE
 * @param       appsize  : 应用程序大小(字节)
 * @retval      无
 */
void iap_write_appbin(uint32_t appxaddr, uint8_t *appbuf, uint32_t appsize)
{
    uint32_t t;
    uint16_t i = 0;
    uint32_t temp;
    uint32_t fwaddr = appxaddr; /* 当前写入的地址 */
    uint8_t *dfu = appbuf;
    for (t = 0; t < appsize; t += 4)
    {
        temp = (uint32_t)dfu[3] << 24;
        temp |= (uint32_t)dfu[2] << 16;
        temp |= (uint32_t)dfu[1] << 8;
        temp |= (uint32_t)dfu[0];
        dfu += 4;   /* 偏移 4 个字节 */
        g_iapbuf[i++] = temp;
        if (i == 512)
        {
            i = 0;
            stmflash_write(fwaddr, g_iapbuf, 512);
            fwaddr += 2048; /* 偏移 2048  512*4=2048 */
        }
    }
    if (i)
    {
        stmflash_write(fwaddr, g_iapbuf, i);  /* 将最后的一些内容字节写进去 */
    }
}

/**
 * @brief       跳转到应用程序段(执行 APP)
 * @param       appxaddr : 应用程序的起始地址
 * @retval      无
 */
void iap_load_app(uint32_t appxaddr)
{
    if (((*(volatile  uint32_t *)appxaddr) & 0x2FF00000) == 0x24000000)
    /* 检查栈顶地址是否合法. */
    {
        printf("跳转到 APP\r\n");
        /* 用户代码区第二个字为程序开始地址(复位地址) */
        jump2app = (iapfun) * (volatile uint32_t *)(appxaddr + 4);
```

```
        /* 初始化 APP 堆栈指针(用户代码区的第一个字用于存放栈顶地址) */
        sys_msr_msp(*(volatile uint32_t *)appxaddr);

        /* 跳转到 APP */
        jump2app();
    }
}
```

该文件总共只有 2 个函数，其中，iap_write_appbin 函数用于将存放在串口接收 buf 里面的 APP 程序写入到 FLASH。iap_load_app 函数，则用于跳转到 APP 程序运行，其参数 appxaddr 为 APP 程序的起始地址，程序先判断栈顶地址是否合法（因为 FLASH APP 和 SRAM APP 的栈顶地址不一样，所以 2 个都是合法的），在得到合法的栈顶地址后，通过 MSR_MSP 函数（该函数在 sys.c 文件）设置栈顶地址，最后通过一个虚拟的函数（jump2app）跳转到 APP 程序执行代码，实现 IAP→APP 的跳转。

接着打开 iap.h，代码如下：

```
#ifndef __IAP_H
#define __IAP_H

#include "./SYSTEM/sys/sys.h"

typedef void (*iapfun)(void);           /* 定义一个函数类型的参数 */

#define FLASH_APP1_ADDR     0x08010000  /* 第一个应用程序起始地址(存放在内部 FLASH)
                                           保留 0X08000000~0X0800FFFF(64KB) 的空间
                                           为 Bootloader 使用(共 64KB) */
void iap_load_app(uint32_t appxaddr);   /* 跳转到 APP 程序执行 */
void iap_write_appbin(uint32_t appxaddr, uint8_t *appbuf, uint32_t applen);
/* 在指定地址开始,写入 bin */
#endif
```

这部分代码比较简单，本章，我们是通过串口接收 APP 程序的，我们将 usart.c 和 usart.h 稍作修改，在 usart.h 中，我们定义 USART_REC_LEN 为 120K 字节，也就是串口一次最大可以接收 120K 字节的数据，这也是本 Bootloader 程序所能接收的最大 APP 程序大小。然后新增一个 g_usart_rx_cnt 的变量，用于记录接收到的文件大小，而 g_usart_rx_sta 不再使用。在 usart.c 里面，我们修改 USART_UX_IRQHandler 部分代码如下：

```
/**
 * @brief       串口 1 中断服务函数
 * @param       无
 * @retval      无
 */
void USART_UX_IRQHandler(void)
{
    uint8_t res;
#if SYSTEM_SUPPORT_OS       /* 使用 OS */
    OSIntEnter();
#endif
    if ((__HAL_UART_GET_FLAG(&g_uart1_handle, UART_FLAG_RXNE) != RESET))
    {
        HAL_UART_Receive(&g_uart1_handle, &res, 1, 1000);
        if (g_usart_rx_cnt < USART_REC_LEN)
        {
            BTIM_TIMX_INT->CNT = 0;
            BTIM_TIMX_INT->CR1 |= 1 << 0;                   /* 使能定时器 */
            g_usart_rx_buf[g_usart_rx_cnt] = res;
            g_usart_rx_cnt++;
        }
    }
    HAL_UART_IRQHandler(&g_uart1_handle);
#if SYSTEM_SUPPORT_OS       /* 使用 OS */
    OSIntExit();
#endif
}
```

这里，我们指定 g_usart_rx_buf 的地址是从 0x24001000 开始，该地址也就是 SRAM APP 程序的起始地址！然后在 USART_UX_IRQHandler 函数里面，将串口发送过来的数据，全部接收到 g_usart_rx_buf，并通过 g_usart_rx_cnt 计数。代码比较简单，我们就不多说了。

最后，我们介绍 main.c 的程序，具体如下：

```
int main(void)
{
    uint8_t t;
    uint8_t key;
    uint32_t oldcount = 0;                  /* 老的串口接收数据值 */
    uint32_t applenth = 0;                  /* 接收到的 app 代码长度 */
    uint8_t clearflag = 0;
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    mpu_memory_protection();                /* 保护相关存储区域 */
    led_init();                             /* 初始化 LED */
    key_init();                             /* 初始化按键 */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */

    lcd_show_string(30,  50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30,  70, 200, 16, 16, "IAP TEST", RED);
    lcd_show_string(30,  90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY_UP: Copy APP2FLASH!", RED);
    lcd_show_string(30, 130, 200, 16, 16, "KEY0: Run SRAM APP", RED);
    lcd_show_string(30, 150, 200, 16, 16, "KEY1: Erase SRAM APP", RED);
    lcd_show_string(30, 170, 200, 16, 16, "KEY2: Run FLASH APP", RED);

    while (1)
    {
        if (g_usart_rx_cnt)
        {
            if (oldcount == g_usart_rx_cnt)
             {/* 新周期内,没有收到任何数据,认为本次数据接收完成 */
                applenth = g_usart_rx_cnt;
                oldcount = 0;
                g_usart_rx_cnt = 0;
                printf("用户程序接收完成!\r\n");
                printf("代码长度:%dBytes\r\n", applenth);
            }
            else
            {
                oldcount = g_usart_rx_cnt;
            }
       }
        t++;
        delay_ms(10);
        if (t == 30)
        {
            LED0_TOGGLE();
            t = 0;

            if (clearflag)
            {
                clearflag--;

                if (clearflag == 0)
                {
                    lcd_fill(30, 190, 240, 210 + 16, WHITE);     /* 清除显示 */
                }
            }
```

```
    }
    key = key_scan(0);
    if (key == WKUP_PRES)       /* WKUP 按下,更新固件到 FLASH */
    {
        if (applenth)
        {
            printf("开始更新固件...\r\n");
            lcd_show_string(30,190,200, 16,16,"Copying APP2FLASH...",BLUE);
            if(((*(volatile uint32_t*)
                                (0x24001000+4))&0xFF000000)==0x08000000)
             /* 判断是否为 0X08XXXXXX */
             {
               iap_write_appbin(FLASH_APP1_ADDR, g_usart_rx_buf, applenth);
               /* 更新 FLASH 代码 */
               lcd_show_string(30,190,200,16,16,"Copy APP Successed!!"BLUE);
               printf("固件更新完成!\r\n");
             }
             else
             {
                lcd_show_string(30,190,200,16,16,"Illegal FLASH APP!",BLUE);
                printf("非 FLASH 应用程序!\r\n");
             }
        }
        else
        {
            printf("没有可以更新的固件!\r\n");
            lcd_show_string(30, 190, 200, 16, 16, "No APP!", BLUE);
        }

        clearflag = 7;    /* 标志更新了显示,并且设置 7*300ms 后清除显示 */
    }

    if (key == KEY1_PRES)  /* KEY1 按键按下 */
    {
        if (applenth)
        {
            printf("固件清除完成!\r\n");
            lcd_show_string(30,190,200,16,16,"APP Erase Successed!", BLUE);
            applenth = 0;
        }
        else
        {
            printf("没有可以清除的固件!\r\n");
            lcd_show_string(30, 190, 200, 16, 16, "No APP!", BLUE);
        }
        clearflag = 7;      /* 标志更新了显示,并且设置 7*300ms 后清除显示 */
    }

    if (key == KEY2_PRES)     /* KEY2 按下 */
    {
        printf("开始执行 FLASH 用户代码!!\r\n");
        if (((*(volatile uint32_t *)
        (FLASH_APP1_ADDR + 4)) & 0xFF000000) == 0x08000000)
         /* 判断是否为 0X08XXXXXX.*/
        {
            iap_load_app(FLASH_APP1_ADDR); /* 执行 FLASH APP 代码 */
        }
        else
        {
          printf("非 FLASH 应用程序,无法执行!\r\n");
          lcd_show_string(30, 210, 200, 16, 16,"Illegal FLASH APP!", BLUE);
        }
        clearflag = 7;     /* 标志更新了显示,并且设置 7*300ms 后清除显示 */
    }
```

```
        if (key == KEY0_PRES) /* KEY0 按下 */
        {
            printf("开始执行 SRAM 用户代码!!\r\n");
            delay_ms(10);
            if (((*(volatile uint32_t *)
            (0x24001000 + 4)) & 0xFF000000) == 0x24000000)
            /* 判断是否为 0X24XXXXXX */
            {
                iap_load_app(0x24001000);  /* SRAM 地址 */
            }
            else
            {
              printf("非 SRAM 应用程序,无法执行!\r\n");
              lcd_show_string(30, 190, 200, 16, 16, "Illegal SRAM APP!", BLUE);
            }

            clearflag = 7;    /* 标志更新了显示,并且设置 7*300ms 后清除显示 */
        }
    }
}
```

该段代码，实现了串口数据处理，以及 IAP 更新和跳转等各项操作。Bootloader 程序就设计完成了，一般要求 bootloader 程序越小越好（给 APP 省空间嘛），实际应用时，可以根据需要尽量精简代码来得到最小的 IAP。本章例程我们仅作演示用，所以不对代码做任何精简，最后得到工程截图如图 68.3.2.1 所示：

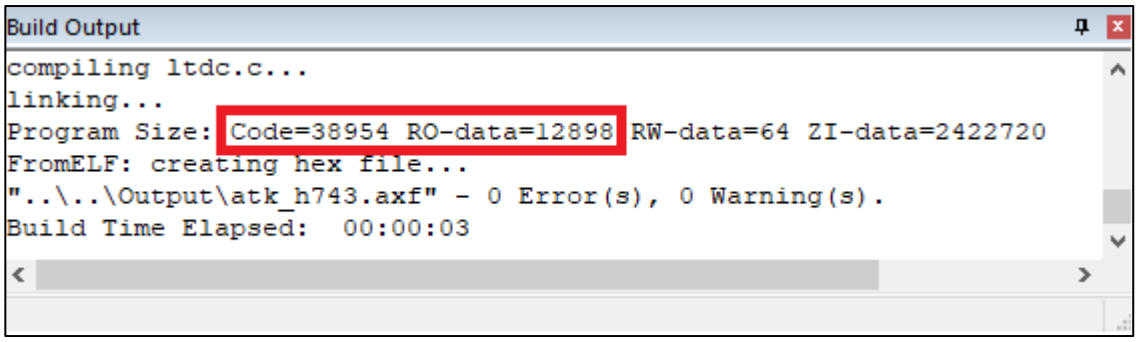


图 68.3.2.1 Bootloader 编译结果

从上图可以看出，Bootloader 大小为 52K 左右，比较大，主要原因是液晶驱动和 printf 占用了比较多的 flash，如果大家想删减代码，可以去掉不用的 LCD 部分代码和 printf 等，不过我们在本章为了演示效果，所以保留了这些代码。至此，本实验的软件设计部分结束。

FLASH APP 和 SRAM APP 两部分代码，根据 68.1 节的介绍，大家自行修改都比较简单，我们这里就不介绍了，不过要提醒大家：FLASH APP 的起始地址必须是 0X08020000，而 SRAM APP 的起始地址必须是 0X24001000。

68.4 下载验证

将程序下载到开发板后，可以看到LCD首先显示一些实验相关的信息，如图68.4.1所示：

图 68.4.1 IAP 程序界面

此时，我们可以通过串口，发送 FLASH APP 或者 SRAM APP 到开发板，如图 68.4.2 所示：

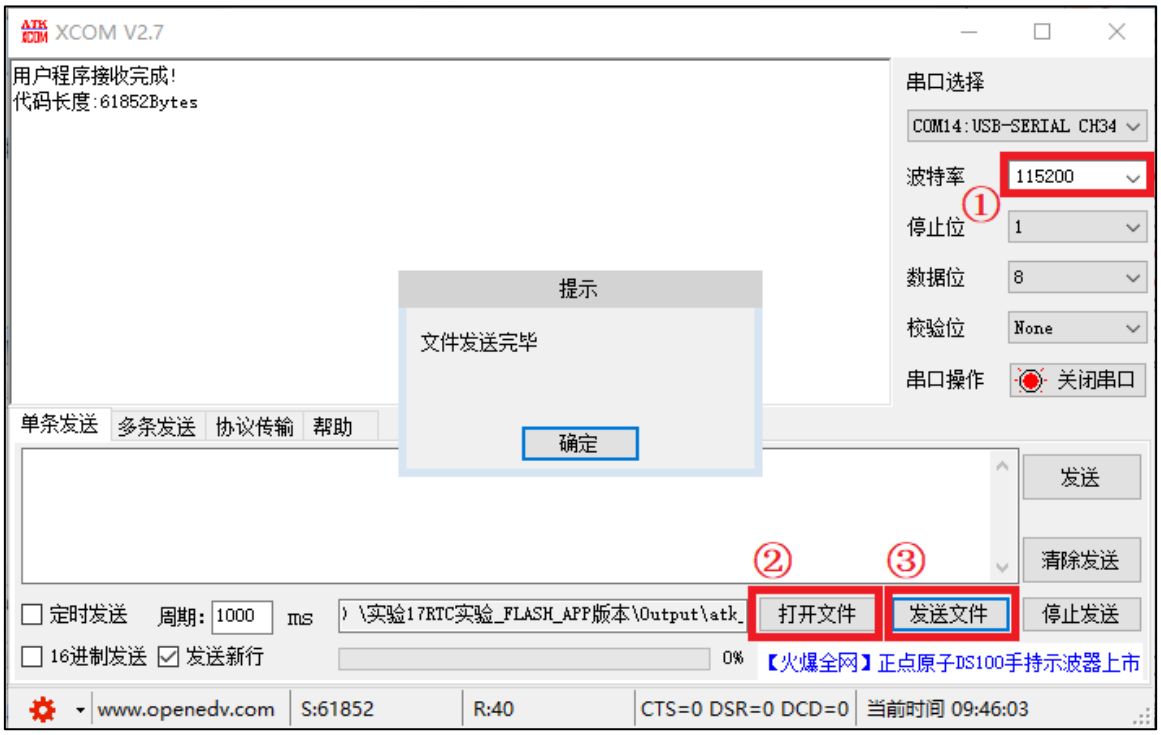


图 68.4.2 串口发送 APP 程序界面

首先找到开发板 USB 转串口的串口号，打开串口（我电脑是 COM14），然后设置波特率为 115200（图中标号 1 所示），然后，点击打开文件按钮（如图标号 2 所示），找到 APP 程序生成的.bin 文件（注意：文件类型要选择所有文件!!默认是只打开 txt 文件的），最后点击发送文件（如图标号 3 所示），将.bin 文件发送给阿波罗 STM32 开发板，发送完成后，XCOM 会提示文件发送完毕。

开发板在收到 APP 程序之后，我们就可以通过 KEY0/KEY2 运行这个 APP 程序了（如果是 FLASH APP，则通过 KEY_UP 将其存入对应 FLASH 区域）。

第六十九章 USB 读卡器(Slave)实验

STM32H743 系列芯片都自带了 USB OTG FS 和 USB OTG HS（HS 需要外扩高速 PHY 芯片实现，速度可达 480Mbps），支持 USB Host 和 USB Device，STM32H743 开发板没有外扩高速 PHY 芯片，所以仅支持 USB OTG FS（FS，即全速，12Mbps），所有 USB 相关例程，均使用 USB OTG FS 实现。

下面，我们将介绍如何使用 USB OTG FS 在阿波罗 STM32H743 开发板上实现一个 USB 读卡器。

本章分为如下几个小节：

69.1 USB 简介
69.2 硬件设计
69.3 程序设计
69.4 下载验证

69.1 USB 简介

USB，即通用串行总线(Universal Serial Bus)，包括 USB 协议和 USB 硬件两个方面，支持热插拔功能。现在日常生活的很多方面都离不开 USB 的应用，如充电和数据传输等方面的应用。

USB 经过多次修改，1996 年确定了初始规范版本 USB1.0，目前由非盈利组织 USB-IF(https://www.usb.org)管理。STM32 自带的 USB 符合 USB2.0 规范，故 2.0 版本仍是本文的重点介绍对象。

69.1.1 USB 简介

USB 本身的知识体系非常复杂，本节只能作一点知识点的引入。本书篇幅有限，不可能在这里详细介绍，想更系统地学习 USB 的知识可以参考《圈圈教你玩 USB》、塞普拉斯提供的《USB 101：通用串行总线 2.0 简介》等文献，下面我们一起来看 USB 的简单特性：

- **USB 的硬件接口**

USB 协议有漫长的发展历程，为的不同的场合和硬件功能而发展出不同的接口：Type-A、Type-B、Type-C，Type-C 规范碰巧是跟着 USB3.1 的规范一起发布的。常见的接口类型列出如图 69.1.1 所示。

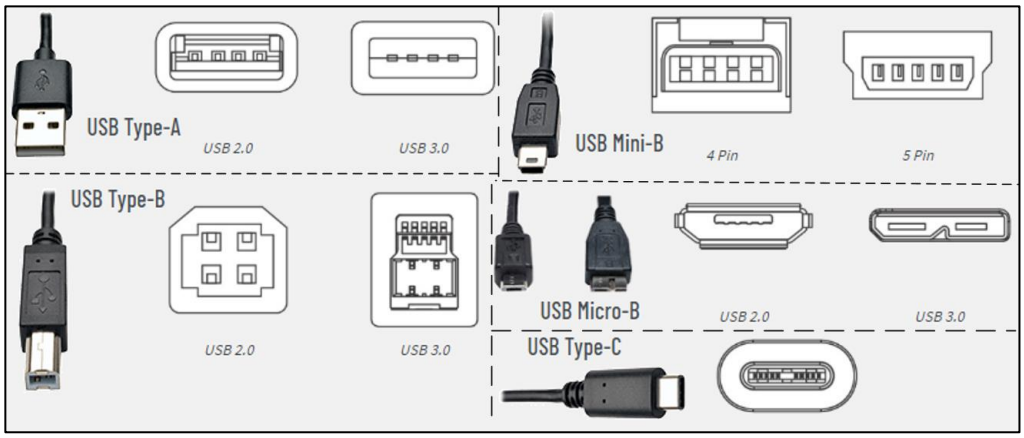


图 69.1.1 常见的 USB 连接器的形状

USB 发展到现在已经有 USB1.0/1.1/2.0/3.x/4 等多个版本。目前用的最多的就是版本 USB1.1 和 USB2.0，USB3.x/USB4 目前也在加速推广。STM32H743 自带的 USB 符合 USB2.0 规范。从图中可以发现不同的版本的 USB 接口内的引脚数量是有差异的。USB3.0 以后为了提高速度，采用了更多数量的通讯线，比如同样的是 Type A 接口，USB2.0 版本内部只有四根线，

采用半双工式广播式通讯，USB3.0 版本则将通讯线提高到了 9 根，并可以支持全双工非广播式的总线，允许两个单向数据管道分别处理一个单向通信。

USB2.0 常使用四根线：VCC（5V）、GND、D+(3.3V)和 D-(3.3V) (注：五线模式多了一个 DI 脚用于支持 OTG 模式，OTG 为 USB 主机+USB 设备双重角色)，其中数据线采用差分电压的方式进行数据传输。在 USB 主机上，D-和 D+都是接了 15K 的电阻到地的，所以在没有设备接入的时候，D+、D-均是低电平。而在 USB 设备中，如果是高速设备，则会在 D+上接一个 1.5K 的电阻到 3.3V，而如果是低速设备，则会在 D-上接一个 1.5K 的电阻到 3.3V。这样当设备接入主机的时候，主机就可以判断是否有设备接入，并能判断设备是高速设备还是低速设备。

关于 USB 硬件还有更多具体的细节规定，硬件设计时需要严格按照 USB 的器件的使用描述和 USB 标准所规定的参数来设计。

- **USB 速度**

USB 规范已经为 USB 系统定义了以下四种速度模式：低速（Low-Speed）、全速（Full-Speed）、高速（Hi-Speed）和超高速（SuperSpeedUSB）。接口的速度上限与设备支持的 USB 协议标准和导线长度、阻抗有关，不同协议版本对硬件的传输线数量、阻抗等要求各不相同，各个版本的能达到的理论速度上限对应如图 69.1.2。

Standard	Also Known As	Logo	Year Introduced	Connector Types	Max. Data Transfer Speed	Cable Length**
USB 1.1	Basic Speed USB		1998	USB-A USB-B	12 Mbps	3 m
USB 2.0	Hi-Speed USB		2000	USB-A USB-B USB Micro A USB Micro B USB Mini A USB Mini B	480 Mbps	5 m
USB 3.2 Gen 1	USB 3.0 USB 3.1 Gen 1 SuperSpeed USB		2008 (USB 3.0) 2013 (USB 3.1)	USB-A USB-B USB Micro B USB-C*	5 Gbps	3 m
USB 3.2 Gen 2	USB 3.1 USB 3.1 Gen 2 SuperSpeed+ SuperSpeed USB 10Gbps		2013 (USB 3.1)	USB-A USB-B USB Micro B USB-C*	10 Gbps	3 m
USB 3.2 Gen 2x2	USB 3.2 SuperSpeed USB 20Gbps		2017 (USB 3.2)	USB-C*	20 Gbps	3 m
Thunderbolt™ 2			2013	Mini DisplayPort	20 Gbps	3 m
Thunderbolt™ 3			2015	USB-C*	20 Gbps (Passive Cable)	2 m
					40 Gbps (Passive Cable)	0.5 m
					40 Gbps (Active Cable)	2 m
USB 4			2019	USB-C*	Up to 40 Gbps	0.8 m
Thunderbolt™ 4			2020	USB-C*	40 Gbps	2 m

图 69.1.2 USB 协议发展与版本对应的速度

USB 端口和连接器有时会标上颜色，以指示 USB 规格及其支持的功能。这些颜色不是 USB 规范所要求的，并且在设备制造商之间不一致。例如，常见的支持 USB3.0 的 U 盘和电脑等设备使用蓝色指示，英特尔使用橙色指示充电端口等。

- **USB 系统**

USB 系统主要包括三个部分：控制器（Host Controller）、集线器 (Hub) 和 USB 设备。

控制器（Host Controller），主机一般可以有一个或多个控制器，主要负责执行由控制器驱动程序发出的命令。控制器驱动程序(Host Controller Driver)在控制器与 USB 设备之间建立通信

信道。

集线器(Hub)连接到 USB 主机的根集线器，可用于拓展主机可访问的 USB 设备的数量。

USB 设备(USB Device)则是我们常用的如 U 盘，USB 鼠标这类受主机控制的设备。

- **USB 通讯**

USB 针对主机、集线器和设备制定了严格的协议。概括来讲，通过检测、令牌、传输控制、数据传输等多种方式，定义了主机和从机在系统中的不同职能。USB 系统通过“管道”进行通讯，有“控制管道”和“数据管道”两种，“控制管道”是双向的，而每个“数据管道”则是单向的，这种关系如图 69.1.3 所示。

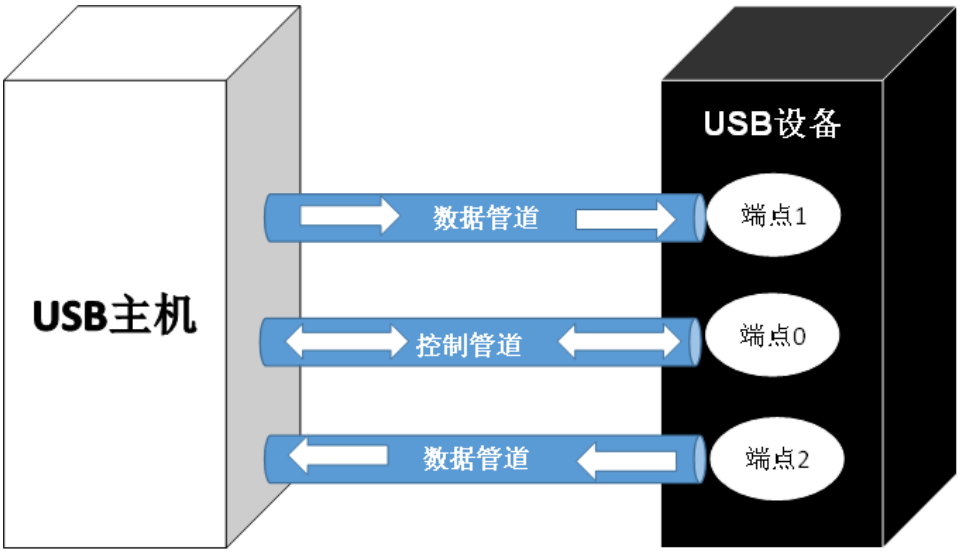


图 69.1.3 USB 管道模型

USB 通讯中的检测和断开总是由主机发起。USB 主机与设备首次进行连接时会交换信息，这一过程叫“USB 枚举”。枚举是设备和主机间进行的信息交换过程，包含用于识别设备的信息。此外，枚举过程主机需要分配设备地址、读取描述符（作为提供有关设备信息的数据结构），并分配和加载设备驱动程序，而从机需要提供相应的描述符使主机知悉如何操作此设备。整个过程需要数秒时间。完成该过程后设备才可以向主机传输数据。数据传输也有规定的三种类型，分别是：IN/读取/上行数据传输、OUT/写入/下行数据传输、控制数据传输。

USB 通过设备端点寻址，在主机和设备间实现信息交流。枚举发生前有一套专用的端点用于与设备进行通信。这些专用的端点统称为控制端点或端点 0，有端点 0 IN 和端点 0 OUT 两个不同的端点，但对开发者来说，它们的构建和运行方式是一样的。每一个 USB 设备都需要支持端点 0。因此，端点 0 不需要使用独立的描述符。除了端点 0 外，特定设备所支持的端点数量将由各自的设计要求决定。简单的设计（如鼠标）可能仅要一个 IN 端点。复杂的设计可能需要多个数据端点。

USB 规定的数据 4 种数据传输方式也是通过管道进行，分别是控制传输(Control Transfer)、中断传输(Interrupt Transfer)、批量传输或叫块传输(Bulk Transfer)、实时传输或叫同步传输(Isochronous Transfer)，每种模式规定了各自通讯时使用的管道类型。

关于 USB 还有很多更详细的时序和要求，像 USB 描述符、VID/PID 的规定、USB 类设备和调试等，因为 USB2.0 和之后的版本有差异，这里就不再为大家列举了，ST 对 USB2.0 也有专门的培训资料，这部分我们也放到“光盘资料 A 盘→8，STM32 参考资料→2，STM32 USB 学习资料”中了，感兴趣的朋友自行去查阅更的 USB 的相关扩展知识，我们对 USB 的简介就到这里。

69.1.2 STM32H7 的 USB 特性

标准 USB 共四根线组成，除 VCC/GND 外，另外为 D+和 D-，这两根数据线采用的是差分电压的方式进行数据传输的。在 USB 主机上，D-和 D+都是接了 15K 的电阻到地的，所以在没有设备接入的时候，D+、D-均是低电平。而在 USB 设备中，如果是高速设备，则会在 D+上

接一个 1.5K 的电阻到 VCC，而如果是低速设备，则会在 D-上接一个 1.5K 的电阻到 VCC。这样当设备接入主机的时候，主机就可以判断是否有设备接入，并能判断设备是高速设备还是低速设备。接下来，我们简单介绍一下 STM32 的 USB 控制器。

STM32H743 系列芯片自带有 2 个 USB OTG，其中 USB1 是高速 USB（USB1 OTG HS）；USB2 是全速 USB（USB2 OTG FS），高速 USB（HS）需要外扩高速 PHY 芯片实现，我们这里不做介绍。

STM32H743 的 USB OTG FS 是一款双角色设备 (DRD) 控制器，同时支持从机功能和主机功能，完全符合 USB 2.0 规范的 On-The-Go 补充标准。此外，该控制器也可配置为“仅主机”模式或“仅从机” 模式，完全符合 USB 2.0 规范。在主机模式下，OTG FS 支持全速（FS，12 Mb/s）和低速（LS，1.5 Mb/s）收发器，而从机模式下则仅支持全速（FS，12 Mb/s）收发器。OTG FS 同时支持 HNP 和 SRP。

STM32H743 的 USB OTG FS 主要特性可分为三类：通用特性、主机模式特性和从机模式特性。

1、通用特性

- 经 USB-IF 认证，符合通用串行总线规范第 2.0 版
- 集成全速 PHY，且完全支持定义在标准规范 OTG 补充第 2.0 版中的 OTG 协议
  1，支持 A-B 器件识别（ID 线）
  2，支持主机协商协议(HNP)和会话请求协议(SRP)
  3，允许主机关闭 VBUS 以在 OTG 应用中节省电池电量
  4，支持通过内部比较器对 VBUS 电平采取监控
  5，支持主机到从机的角色动态切换
- 可通过软件配置为以下角色：
  1， 具有 SRP 功能的 USB FS 从机（B 器件）
  2， 具有 SRP 功能的 USB FS/LS 主机（A 器件）
  3，USB On-The-Go 全速双角色设备
- 支持 FS SOF 和 LS Keep-alive 令牌
  1，SOF 脉冲可通过 PAD 输出
  2，SOF 脉冲从内部连接到定时器 2 (TIM2)
  3，可配置的帧周期
  3， 可配置的帧结束中断
- 具有省电功能，例如在 USB 挂起期间停止系统、关闭数字模块时钟、对 PHY 和 DFIFO 电源加以管理
- 具有采用高级 FIFO 控制的 4KB 专用 RAM
  1，可将 RAM 空间划分为不同 FIFO，以便灵活有效地使用 RAM
  2，每个 FIFO 可存储多个数据包
  3，动态分配存储区
  4，FIFO 大小可配置为非 2 的幂次方值，以便连续使用存储单元
- 一帧之内可以无需要应用程序干预，以达到最大 USB 带宽

2、主机（Host）模式特性

- 通过外部电荷泵生成 VBUS 电压。
- 多达 16 个主机通道（管道）：每个通道都可以动态实现重新配置，可支持任何类型的 USB 传输。
- 内置硬件调度器可：
  1，在周期性硬件队列中存储多达 16 个中断加同步传输请求
  2，在非周期性硬件队列中存储多达 16 个控制加批量传输请求
- 管理一个共享 RX FIFO、一个周期性 TX FIFO 和一个非周期性 TX FIFO，以有效使用 USB 数据 RAM。

3、从机（Slave/Device）模式特性

- 1 个双向控制端点 0
- 8 个 IN 端点 (EP)，可配置为支持批量传输、中断传输或同步传输

- 8 个 OUT 端点(EP)，可配置为支持批量传输、中断传输或同步传输
- 管理一个共享 Rx FIFO 和一个 Tx-OUT FIFO，以高效使用 USB 数据 RAM
- 管理多达 9 个专用 Tx-IN FIFO（分别用于每个使能的 IN EP），降低应用程序负荷支持软断开功能。

STM32H743 USB2 OTG FS 框图如图 69.1.1 所示：

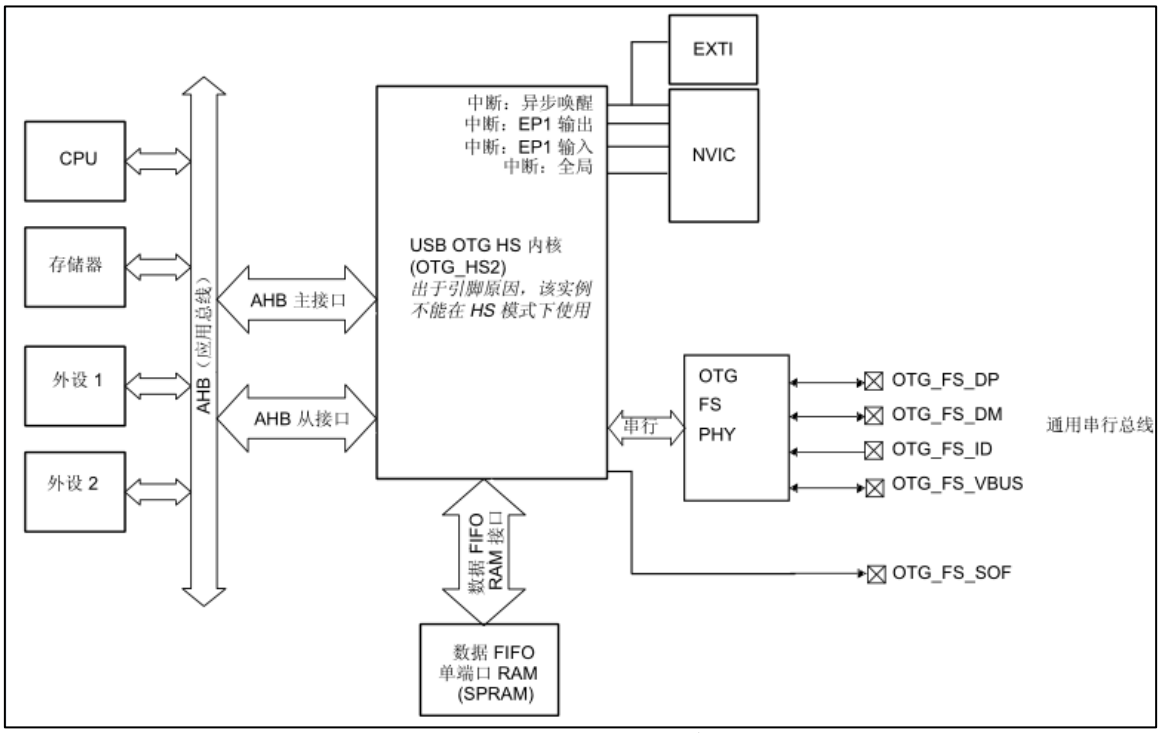


图 69.1.1 USB2 OTG FS 框图

对于 USB OTG FS 功能模块，STM32H743 通过 AHB 总线访问（AHB 频率必须大于 30Mhz），另外，USB OTG 的内核时钟必须是 48Mhz，由 RCC_D2CCIP2R 寄存器的 USBSEL[1:0]位选择：00，禁止 USB 内核时钟；01，USB 内核时钟来自 pll1_q_ck；10，USB 内核时钟来自 pll3_q_ck；11，USB 内核时钟来自 hsi48_ck；因为 pll1_q_ck 和 pll3_q_ck 很有可能被其他外设用作时钟，不方便设置为 48Mhz，因此我们一般使用 hsi48_ck 作为 USB OTG 内核时钟（USBSEL[1:0]=11），这样就不会受到其他外设的影响。

要正常使用 STM32H743 的 USB，就得编写 USB 驱动，而整个 USB 通信的详细过程是很复杂的，本书篇幅有限，不可能在这里详细介绍，有兴趣的朋友可以去看看电脑圈圈的《圈圈教你玩 USB》这本书，该书对 USB 通信有详细讲解。如果要我们自己编写 USB 驱动，那是一件相当困难的事情，尤其对于从没了解过 USB 的人来说，基本上不花个一两年时间学习，是没法搞定的。不过，ST 提供了我们一个完整的 USB OTG 驱动库（包括主机和设备），通过这个库，我们可以很方便的实现我们所要的功能，而不需要详细了解 USB 的整个驱动，大大缩短了我们的开发时间和精力。

STM32H7 的 USB 例程全部是以 HAL 库的形式提供，为了简化开发设计，我们直接使用 ST 提供的 HAL 库版本 USB 驱动库来设计相关例程。

ST 提供的 H7 USB OTG 库和相关参考例程在 en.stm32cubeh7.zip 里面可以找到，该文件可以在 www.st.com 网站搜索：cubeh7 找到。不过，我们已经帮大家下载到开发板光盘：8，STM32 参考资料→ 1，STM32CubeH7 固件包→ en.stm32cubeh7.zip。解压可以得到 STM32H7 的 Cube 固件支持包：STM32Cube_FW_H7_V1.9.0，该文件夹里面包含了 H7 的 USB 主机（Host）和从机（Device）驱动库，并提供了 17 个例程供我们参考，如图 69.1.2 所示：

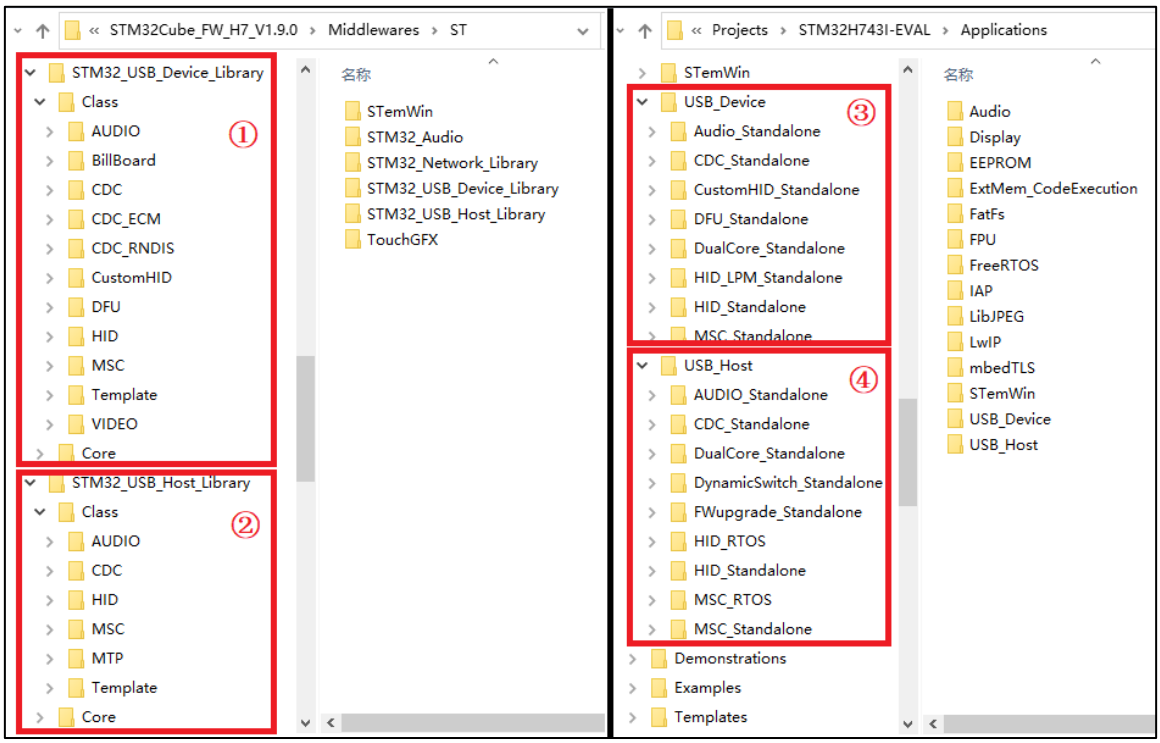


图 69.1.2 ST 提供的 USB OTG 例程

标号 1 是 H7 USB 从机驱动库；

标号 2 是 H7 USB 主机驱动库；

标号 3 是 H7 USB 从机例程（共 8 个）；

标号 4 是 H7 USB 主机例程（共 9 个）；

整个 USB OTG 库的使用和例程说明，可以参考 ST 官方提供的：UM1734（从机）和 UM1720（主机）这两个文档（在光盘：8，STM32 参考资料→2，STM32 USB 学习资料 文件夹），这两个文档详细介绍了 USB OTG 库的各个组成部分以及所提供的例程使用方法，有兴趣学习 USB 的朋友，这个文档是必须仔细看的。

这 17 个例程，虽然都是基于官方 STM32H743IEVAL 板，但是很容易移植到我们的阿波罗 STM32H743 开发板上。

本实验我们移植：STM32Cube_FW_H7_V1.9.0\Projects\STM32H743I_EVAL\Applications\USB_Device\MSC_Standalone 这个例程，以实现 USB 读卡器功能。

69.2 硬件设计

1. 例程功能

本实验代码，开机的时候先检测 SD 卡、SPI FLASH 和 NAND FLASH 是否存在，如果存在则获取其容量，并显示在 LCD 上面（如果不存在，则报错）。之后开始 USB 配置，在配置成功之后就可以在电脑上发现三个可移动磁盘。USB 正在读写会在液晶上显示出来。

LED0 闪烁，提示程序运行。USB 和电脑连接成功后，LED1 常亮。

2. 硬件资源

1）LED 灯

LED0：LED0 - PB1

LED1：LED1 - PB0

2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)

3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)

4）USB_SLAVE 接口(D-/D+连接在 PA11/PA12 上)

5）QSPI(PB2/PB6/PF6/PF7/PF8/PF9)
6）norflash(QSPI FLASH 芯片，连接在 QSPI 上)
7）SD 卡，通过 SDMMC1(SDMMC_D0~D4(PC8~PC11)，SDMMC_SCK(PC12)，SDMMC_CMD(PD2))连接

3. 原理图

开发板采用的是 16PIN 的 Type-C 接口，用来和电脑的 USB 相连接，Type-C 接口与 STM32 的连接电路图，如下图所示：

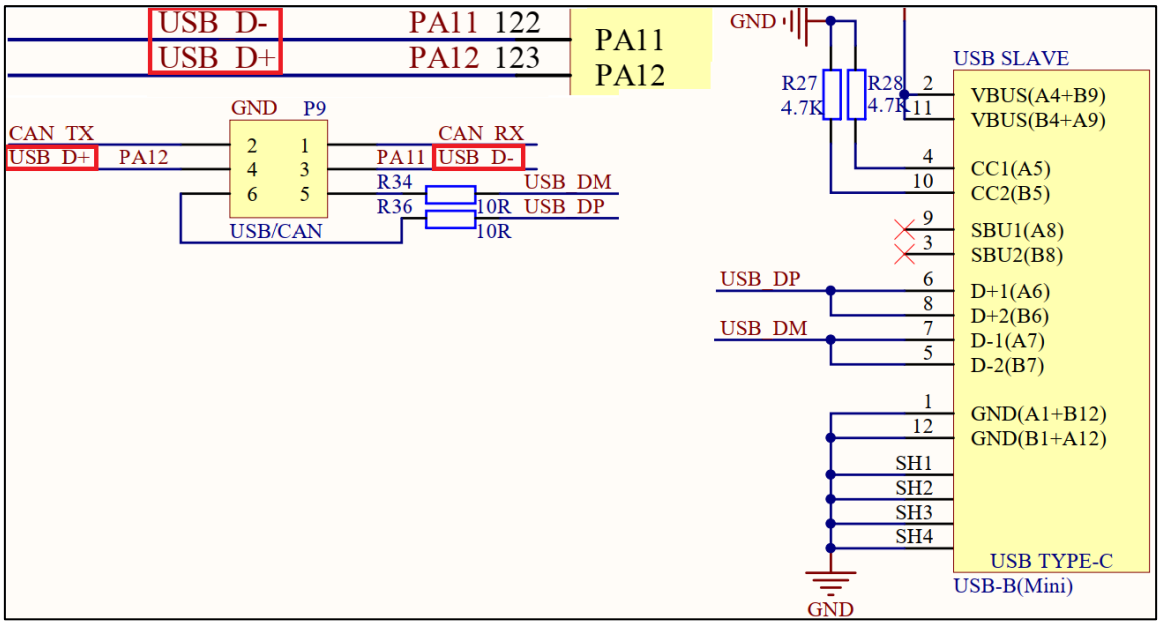


图 69.2.1 Type-C 接口与 STM32 的连接电路图

从上图可以看出，USB Type-C 座是没有直接连接到 STM32H743 上面的，而是通过 P9 转接，所以我们需要通过跳线帽将 PA11 和 PA12 分别接到 D-和 D+，如图 69.2.2 所示：

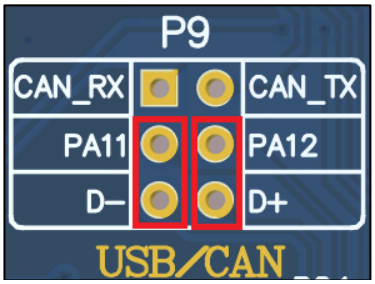


图 69.2.2 硬件连接示意图

需要注意的是：这个 Type-C 座和 USB-B 母座（USB_HOST）是共用 D+和 D-的，所以他们不能同时使用。这个在使用的时候，要特别注意！！本实验测试时，USB_HOST 不能插入任何 USB 设备！另外，如果只有 STM32H743 核心板的，可以利用核心板上面的 MicroUSB 接电脑，同样也实现本例程的功能。

69.3 程序设计

69.3.1 程序流程图

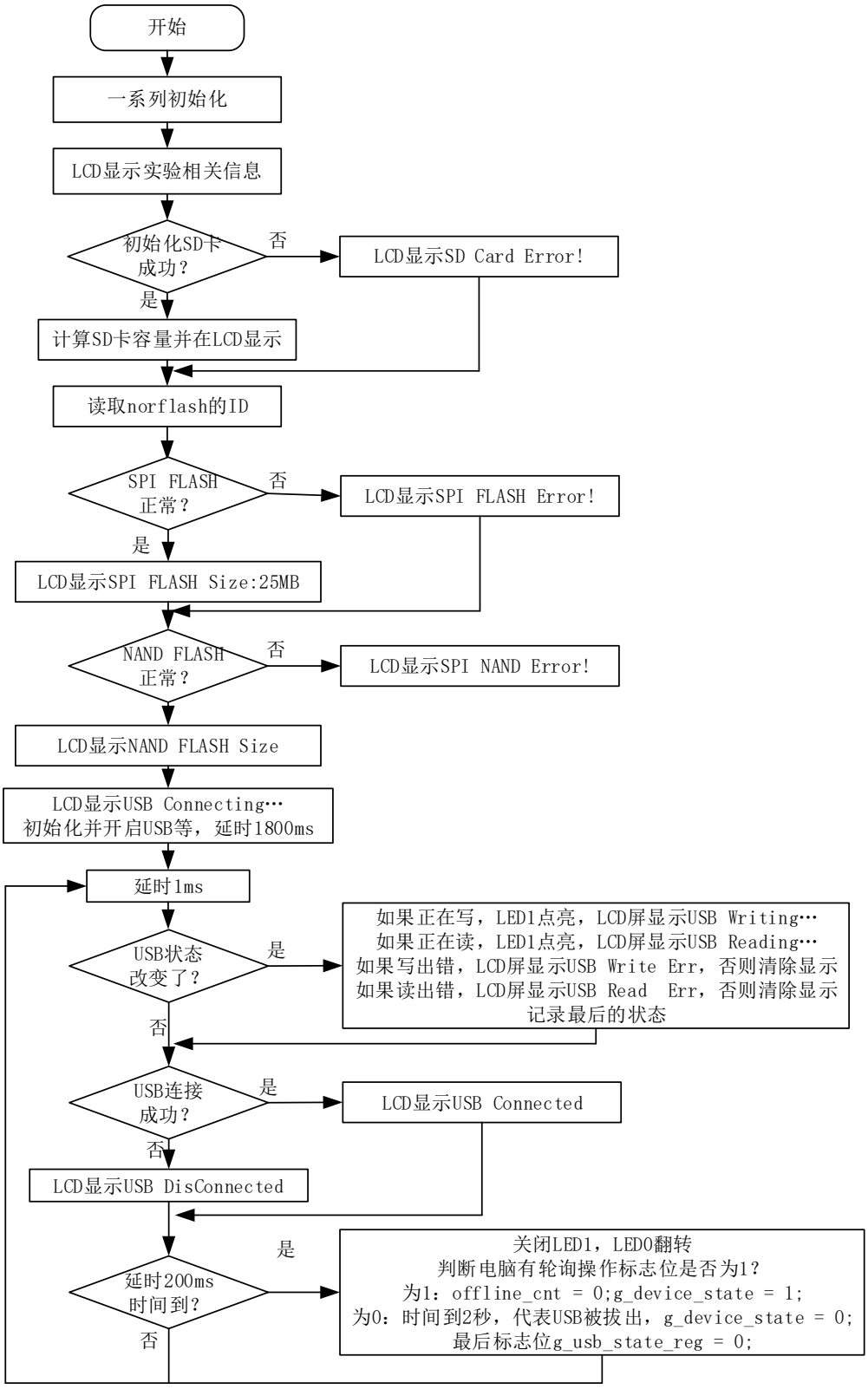


图 69.3.1.1 USB 读卡器(Slave)实验程序流程图

69.3.2 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。

本实验，我们在：实验 41 SD 卡实验的基础上修改，代码移植自 ST 官方例程：STM32Cube_FW_H7_V1.9.0\Projects\STM32H743I_EVAL\Applications\USB_Device\MSC_Standalone。该目录下提供了三种开发环境的工程：IAR、MDK 和 SW4STM32，我们使用的是 MDK，打开 MDK 工程，然后就可以知道和 USB 相关的代码有哪些，如图 69.3.2.1 所示：

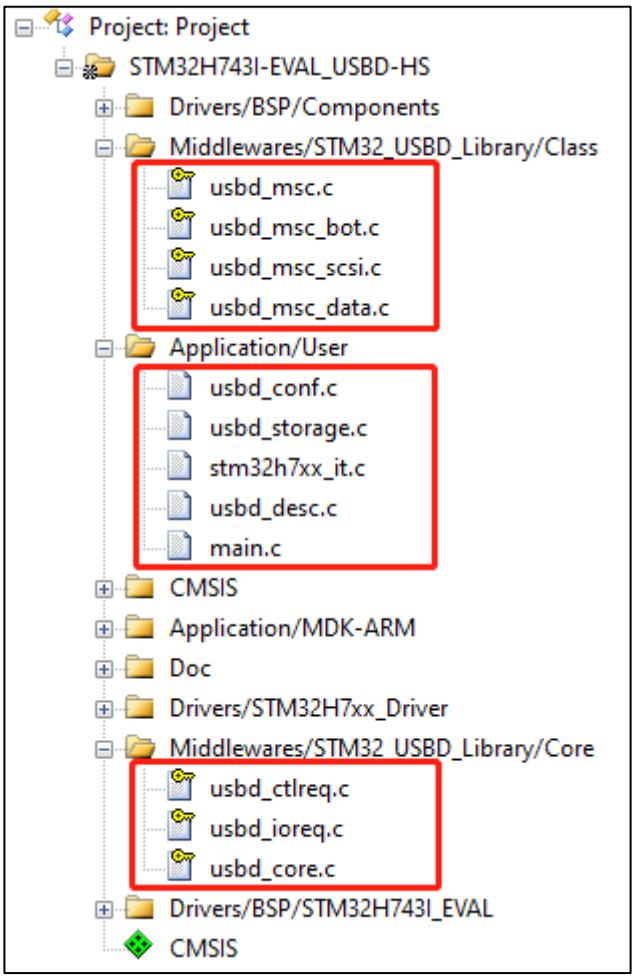


图 69.3.2.1 ST 官方例程 USB 相关代码

有了这个官方例程做指引，我们就知道具体需要哪些文件，从而实现本章例程。

首先，在本例程（即实验 41 SD 卡实验）工程的 Middlewares 文件夹下面，新建一个 USB 文件夹，并拷贝官方 USB 驱动库相关代码到该文件夹下，即拷贝：光盘→ 8，STM32 参考资料→1，STM32CubeH7 固件包→STM32Cube_FW_H7_V1.9.0→Middlewares→ST 文件夹下的：STM32_USB_Device_Library、STM32_USB_HOST_Library 两个文件夹及源码拷贝到该文件夹下面。

然后，在 USB 文件夹下，新建一个 USB_APP 文件夹用于存放 MSC 实现相关代码，即：STM32Cube_FW_H7_V1.9.0→Projects→STM32H743I_EVAL→Applications→USB_Device→MSC_Standalone→Src 下的部分代码：usbd_conf.c、usbd_storage.c 和 usbd_desc.c 等 3 个.c 文件，同时拷贝：STM32Cube_FW_H7_V1.9.0→Projects→STM32H743I_EVAL→Applications→USB_Device→MSC_Standalone→Inc 下面的：usbd_conf.h、usbd_storage.h 和 usbd_desc.h 等三个文件到 USB_APP 文件夹下。最后 USB_APP 文件夹下的文件如图 69.3.2.2 所示：

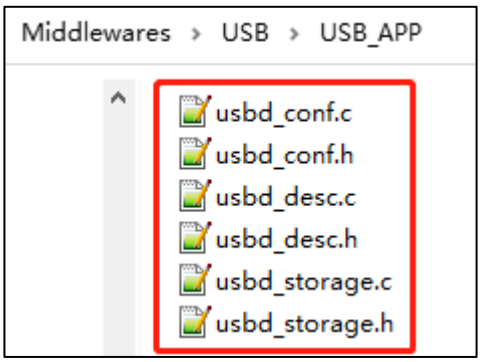


图 69.3.2.2 USB_APP 代码

之后，根据 ST 官方 MSC 例程，在我们本章例程的基础上新建分组添加相关代码，具体细节请参考例程，这里就不详细介绍了，添加好之后，如图 69.3.2.3 所示：

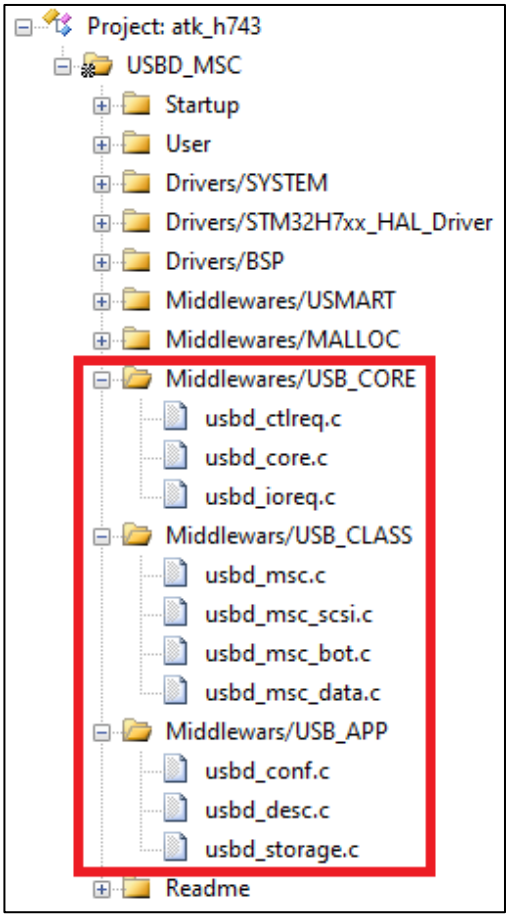


图 69.3.2.3 添加 USB 驱动等相关代码

1. USB 驱动代码

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。接下来我们看看 USB_APP 里面的几个.c 文件：

usbd_conf.c 提供了 USB 设备库（从机库，下同）的回调及 MSP 初始化函数，当 USB 状态机处理完不同事务的时候，会调用这些回调函数，我们通过这些回调函数，就可以知道 USB 当前状态，比如：是否枚举成功了？是否连接上了？是否断开了？等，根据这些状态，用户应用程序可以执行不同操作，完成特定功能。usbd_conf.c 我们重点介绍 3 个函数，首先是 HAL_PCD_MspInit 和 OTG_FS_IRQHandler 函数，它们的定义如下：

```
/**
  * @brief  初始化 PCD MSP.
```

```
  * @param  hpcd: PCD 结构体指针
  * @retval 无
  */
void HAL_PCD_MspInit(PCD_HandleTypeDef *hpcd)
{
    GPIO_InitTypeDef  gpio_init_struct;
    RCC_PeriphCLKInitTypeDef usb_clk_struct;

    usb_clk_struct.PeriphClockSelection = RCC_PERIPHCLK_USB;
    usb_clk_struct.UsbClockSelection = RCC_USBCLKSOURCE_HSI48;
    HAL_RCCEx_PeriphCLKConfig(&usb_clk_struct);

    if (hpcd->Instance == USB2_OTG_FS)
    {
        __HAL_RCC_GPIOA_CLK_ENABLE();                          /* 使能 GPIOA 时钟 */
        __HAL_RCC_GPIOG_CLK_ENABLE();                          /* 使能 GPIOG 时钟 */
        __HAL_RCC_USB2_OTG_FS_CLK_ENABLE();                    /* 使能 OTG FS 时钟 */

        /* 配置 PA11 12 */
        gpio_init_struct.Pin = GPIO_PIN_11 | GPIO_PIN_12;      /* PA11 12 */
        gpio_init_struct.Mode = GPIO_MODE_AF_PP;               /* 复用 */
        gpio_init_struct.Pull = GPIO_NOPULL;                   /* 无上下拉 */
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;    /* 高速 */
        gpio_init_struct.Alternate = GPIO_AF10_OTG1_FS;        /* 复用为 OTG FS */
        HAL_GPIO_Init(GPIOA, &gpio_init_struct);               /* 初始化 */

        gpio_init_struct.Pin = GPIO_PIN_13;
        gpio_init_struct.Mode = GPIO_MODE_OUTPUT_PP;           /* 推挽输出 */
        gpio_init_struct.Pull = GPIO_PULLUP;                   /* 上拉 */
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;    /* 高速 */
        HAL_GPIO_Init(GPIOG, &gpio_init_struct);               /* 初始化 */
        HAL_GPIO_WritePin(GPIOG, GPIO_PIN_13, GPIO_PIN_SET);
        /* 开启 USB HOST 电源供电 C */

        HAL_NVIC_SetPriority(OTG_FS_IRQn, 1, 0);
        HAL_NVIC_EnableIRQ(OTG_FS_IRQn);                       /* 使能 OTG FS 中断 */
    }
    else if (hpcd->Instance == USB1_OTG_HS)
    {
        /* USB1 OTG 本例程没用到,故不做处理 */
    }
}

/**
 * @brief       USB OTG 中断服务函数
 * @note        处理所有 USB 中断
 * @param       无
 * @retval      无
 */
void OTG_FS_IRQHandler(void)
{
    HAL_PCD_IRQHandler(&hpcd);
}
```

HAL_PCD_MspInit 函数，用于使能 USB 时钟，初始化 IO 口，设置中断等。该函数在 HAL_PCD_Init 函数里面被调用。

OTG_FS_IRQHandler 函数，是 USB 的中断服务函数，通过调用 HAL_PCD_IRQHandler 函数，实现对 USB 各种事务的处理。

下面介绍的 USBD 底层初始化函数，其定义如下：

```
/**
 * @brief       USBD 底层初始化函数
 * @param       pdev    : USBD 句柄指针
```

```
 * @retval      USB 状态
 *   @arg       USBD_OK(0)    , 正常;
 *   @arg       USBD_BUSY(1) , 忙;
 *   @arg       USBD_FAIL(2) , 失败;
 */
USBD_StatusTypeDef USBD_LL_Init(USBD_HandleTypeDef *pdev)
{
    if (pdev->id == DEVICE_FS)
    {
        g_pcd_usb_otg_fs.pData = pdev;    /* g_pcd_usb_otg_fs 的 pData 指向 pdev */
        pdev->pData = &g_pcd_usb_otg_fs; /* pdev 的 pData 指向 g_pcd_usb_otg_fs */
        g_pcd_usb_otg_fs.Instance = USB2_OTG_FS;      /* 使用 USB OTG FS */
        g_pcd_usb_otg_fs.Init.dev_endpoints = 8;      /* 端点数为 8 */
        g_pcd_usb_otg_fs.Init.speed = PCD_SPEED_FULL;/* USB 全速(12Mbps) */
        g_pcd_usb_otg_fs.Init.ep0_mps = 0X40;
        /* 设置端点 0 的最大包大小为 0X40(64 字节) */
        g_pcd_usb_otg_fs.Init.phy_itface = PCD_PHY_EMBEDDED;/* 使用内部 PHY */
        g_pcd_usb_otg_fs.Init.Sof_enable = 0;                /* 使能 SOF 中断 */
        g_pcd_usb_otg_fs.Init.low_power_enable = DISABLE;    /* 不使能低功耗模式 */
        g_pcd_usb_otg_fs.Init.lpm_enable = 0;                /* 使能连接电源管理 */
        g_pcd_usb_otg_fs.Init.vbus_sensing_enable=DISABLE;   /* 不使能 VBUS 检测 */
        g_pcd_usb_otg_fs.Init.use_dedicated_ep1 = 0;/* 禁止 EP1 dedicated 中断 */

        g_pcd_usb_otg_fs.pData = pdev;              /* hpcd 的 pData 指向 pdev */
        pdev->pData = &g_pcd_usb_otg_fs;            /* pdev 的 pData 指向 hpcd */

        HAL_PCD_Init(&g_pcd_usb_otg_fs);
        HAL_PCDEx_SetRxFiFo(&g_pcd_usb_otg_fs, 0x80);
        /* 设置接收 FIFO 大小为 0X80(128 字节) */
        HAL_PCDEx_SetTxFiFo(&g_pcd_usb_otg_fs, 0, 0x40);
        /* 设置发送 FIFO 0 的大小为 0X40(64 字节) */
        HAL_PCDEx_SetTxFiFo(&g_pcd_usb_otg_fs, 1, 0x80);
        /* 设置发送 FIFO 1 的大小为 0X80(128 字节) */
    }
    return USBD_OK;
}
```

USBD_LL_Init 函数，用于初始化 USB 底层设置，因为我们定义的是：USE_USB_FS，因此会设置 USB OTG 使用 USB2_OTG_FS，然后完成各种设置，比如，使用内部 PHY，使用全速模式，不使能 VBUS 检测等，详见以上代码。该函数在 USBD_Init 函数里面被调用。

usbd_desc.c 提供了 USB 设备类的描述符，直接决定了 USB 设备的类型、端点、接口、字符串、制造商等重要信息。这个里面的内容，我们一般不用修改，直接用官方的即可。注意，这里：usbd_desc.c 里面的：usbd 即 device 类，同样：usbh 即 host 类，所以通过文件名我们就可以很容易区分该文件是用在 device 还是 host，而只有 usb 字样的那就是 device 和 host 可以共用的。

usbd_storage.c 提供一些磁盘操作函数，包括支持的磁盘个数，以及每个磁盘的初始化和读写等函数。本章我们设置了 3 个磁盘：SD 卡、SPI FLASH 和 NAND FLASH 等。usbd_storage.c 我们重点介绍 3 个函数，首先是初始化存储设备函数，其定义如下：

```
/**
 * @brief       初始化存储设备
 * @param       lun         : 逻辑单元编号
 *   @arg                    0, SPI FLASH
 *   @arg                    1, NAND LASH
 *   @arg                    2, SD 卡
 * @retval      操作结果
 *   @arg       0     , 成功
 *   @arg       其他 , 错误代码
 */
int8_t STORAGE_Init_FS(uint8_t lun)
{
```

```
    uint8_t res = 0;

    switch (lun)
    {
        case 0:                 /* SPI FLASH */
            norflash_init();
            break;
        case 1:                 /* NAND FLASH */
            res = ftl_init();
            break;
        case 2:                 /* SD卡 */
            res = sd_init();
            break;
    }
    return res;
}
```

STORAGE_Init_FS函数，用于初始化存储设备，我们定义了三个存储设备：SPI FLASH、NAND FLASH和SD卡，因此需要根据输入参数（lun），执行不同存储设备的初始化。

下面要介绍的是从存储设备读取数据函数。其定义如下：

```
/**
 * @brief       从存储设备读取数据
 * @param       lun         : 逻辑单元编号
 *   @arg                   0, SPI FLASH
 *   @arg                   1, NAND FLASH
 *   @arg                   2, SD卡
 * @param       buf         : 数据存储区首地址指针
 * @param       blk_addr    : 要读取的地址(扇区地址)
 * @param       blk_len     : 要读取的块数(扇区数)
 * @retval      操作结果
 *   @arg       0     , 成功
 *   @arg       其他  , 错误代码
 */
int8_t STORAGE_Read_FS(uint8_t lun, uint8_t *buf, uint32_t blk_addr,
                       uint16_t blk_len)
{
    int8_t res = 0;
    g_usb_state_reg |= 0X02;    /* 标记正在读数据 */
    switch (lun)
    {
        case 0:             /* SPI FLASH */
        norflash_read(buf,USB_STORAGE_FLASH_BASE + blk_addr*512, blk_len *512);
        break;

        case 1:             /* NAND FLASH */
        res = ftl_read_sectors(buf, blk_addr, 512, blk_len);
        break;

        case 2:             /* SD卡 */
        res = sd_read_disk(buf, blk_addr, blk_len);
        break;
    }

    if (res)
    {
        printf("rerr:%d,%d", lun, res);
        g_usb_state_reg |= 0X08;    /* 读错误! */
    }
    return res;
}
```

STORAGE_Read_FS函数，用于从存储设备读取数据，同样是根据存储设备（lun）的不同，调用不同的读取函数，完成数据读取。

下面要介绍的是向存储设备写数据函数。其定义如下：

```
/**
 * @brief       向存储设备写数据
 * @param       lun         : 逻辑单元编号
 *   @arg                   0, SPI FLASH
 *   @arg                   1, NAND FLASH
 *   @arg                   2, SD 卡
 * @param       buf         : 数据存储区首地址指针
 * @param       blk_addr    : 要写入的地址(扇区地址)
 * @param       blk_len     : 要写入的块数(扇区数)
 * @retval      操作结果
 *   @arg       0     , 成功
 *   @arg       其他  , 错误代码
 */
int8_t STORAGE_Write_FS(uint8_t lun, uint8_t *buf, uint32_t blk_addr, uint16_t
                        blk_len)
{
    int8_t res = 0;
    g_usb_state_reg |= 0X01;    /* 标记正在写数据 */

    switch (lun)
    {
        case 0:                 /* SPI FLASH */
          norflash_write(buf,USB_STORAGE_FLASH_BASE+blk_addr*512,blk_len* 512);
          break;
        case 1:                 /* NAND FLASH */
          res = ftl_write_sectors(buf, blk_addr, 512, blk_len);
          break;
        case 2:                 /* SD 卡 */
          res = sd_write_disk(buf, blk_addr, blk_len);
          break;
    }
    if (res)
    {
        g_usb_state_reg |= 0X04;    /* 写错误! */
        printf("werr:%d,%d", lun, res);
    }
    return res;
}
```

STORAGE_Write_FS 函数，用于往存储设备写入数据，也是根据存储设备（lun）的不同，调用不同的写入函数，完成数据写入。

以上 3 个.c 文件和对应.h 文件的详细代码和修改方法，我们就不详细介绍了，请大家参考光盘本例程源码。

下面提几个重点地方讲解下：

1、要使用 USB OTG FS，必须在 MDK 编译器的全局宏定义里面，添加宏定义：USE_USB_FS，具体添加情况如图 69.3.2.4 所示：

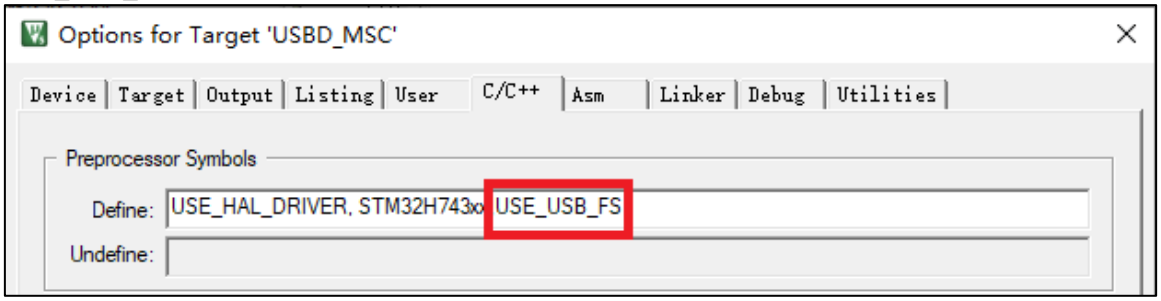


图 69.3.2.4 定义全局宏 USE_USB_FS

2、通过修改 usbd_conf.h 里面的 MSC_MEDIA_PACKET 定义值大小，可以一定程度提高 USB 读写速度（越大越快），本例程我们设置 32*1024，也就是 32KB 大小。另外，我们通过修改：STORAGE_LUN_NBR 宏定义的值为 3，可以支持 3 个磁盘。就本开发板来说是支持 3 个磁盘的。

3、官方例程在 2 个或以上磁盘支持的时候，存在 bug，我们需要修改 usbd_msc.h 里面 USBD_MSC_BOT_HandleTypeDef 结构体的 scsi_blk_nbr 参数，将其改为数组形式：uint32_t scsi_blk_nbr[STORAGE_LUN_NBR];数组大小由 STORAGE_LUN_NBR 指定，本实验我们定义的是 3，因此可以支持最多 3 个磁盘，修改 STORAGE_LUN_NBR 的大小，即可修改支持的最大磁盘个数。修改该参数后，相应的有一些函数要做修改，请大家参考本例程源码。

4、修改 usbd_msc_bot.c 里面修改 MSC_BOT_CBW_Decode 函数，将 hmsc->cbw.bLUN > 1 改为：hmsc->cbw.bLUN > STORAGE_LUN_NBR，以支持多个磁盘。

以上 4 点，就是我们移植的时候需要特别注意的，其他我们就不详细介绍了（USB 相关源码解释，请参考：UM1734(STM32Cube USB device library).pdf 这个文档）。

2. main.c 代码

下面是 main.c 的程序，具体如下：

```
USBD_HandleTypeDef USBD_Device;                 /* USB Device 处理结构体 */
extern PCD_HandleTypeDef g_pcd_usb_otg_fs;      /* PCD_USB_OTG_FS 定义*/
extern volatile uint8_t g_usb_state_reg;        /* USB 状态 */
extern volatile uint8_t g_device_state;         /* USB 连接 情况 */

int main(void)
{
    uint8_t offline_cnt = 0;
    uint8_t tct = 0;
    uint8_t usb_sta;
    uint8_t device_sta;
    uint64_t card_capacity;                             /* SD 卡容量 */

    sys_cache_enable();                                 /* 打开 L1-Cache */
    mpu_memory_protection();                            /* 保护相关存储区域 */
    HAL_Init();                                         /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);                 /* 设置时钟, 400Mhz */
    delay_init(400);                                    /* 延时初始化 */
    usart_init(115200);                                 /* 串口初始化 */
    usmart_init(200);                                   /* 初始化 USMART */
    led_init();                                         /* 初始化 LED */
    key_init();                                         /* 初始化 KEY */
    sdram_init();                                       /* 初始化 SDRAM */
    lcd_init();                                         /* 初始化 LCD */
    norflash_init();                                    /* 初始化 norflash */

    my_mem_init(SRAMIN);                                /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                                /* 初始化外部内存池(SDRAM) */
    my_mem_init(SRAMDTCM);                              /* 初始化 DTCM 内存池(DTCM) */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "USB Card Reader TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);

    if (sd_init())      /* 初始化 SD 卡 */
    {
        lcd_show_string(30, 130, 200, 16, 16, "SD Card Error!", RED);
       /* 检测 SD 卡错误 */
    }
    else                /* SD 卡正常 */
    {
        lcd_show_string(30, 130, 200, 16, 16, "SD Card Size:     MB", RED);
        card_capacity = (uint64_t)(g_sd_card_info_handle.LogBlockNbr)
         *(uint64_t)(g_sd_card_info_handle.LogBlockSize);    /* 计算 SD 卡容量 */
        lcd_show_num(134, 130, card_capacity >>20,5, 16,RED);/* 显示 SD 卡容量 */
    }
```

```
    if (norflash_read_id() != W25Q256)
    {
        lcd_show_string(30, 130, 200, 16, 16, "NorFlash Error!", RED);
    }
    else                /* SPI FLASH 正常 */
    {
        lcd_show_string(30, 150, 200, 16, 16, "SPI FLASH Size:25MB", RED);
    }
    if (ftl_init())  /* 检测 NAND FLASH 错误 */
    {
        lcd_show_string(30, 170, 200, 16, 16, "NAND Error!", RED);
    }
    else                /* NAND FLASH 正常 */
    {
        lcd_show_string(30, 170, 200, 16, 16, "NAND Flash Size:    MB", RED);
        lcd_show_num(158, 170, nand_dev.valid_blocknum * nand_dev.block_pagenum
        * nand_dev.page_mainsize >> 20, 4, 16, RED); /* 显示 SD 卡容量 */
    }

    USBD_Init(&USBD_Device, &MSC_Desc, DEVICE_FS);    /* 初始化 USB */
    USBD_RegisterClass(&USBD_Device, &USBD_MSC);      /* 添加类 */
    USBD_MSC_RegisterStorage(&USBD_Device, &USBD_Storage_Interface_fops_FS);
    /* 为 MSC 类添加回调函数 */
    USBD_Start(&USBD_Device);     /* 开启 USB */
    delay_ms(1800);

    while(1)
    {
        delay_ms(1);

        if (usb_sta != g_usb_state_reg)                    /* 状态改变了 */
        {
            lcd_fill(30, 210, 240, 210 + 16, WHITE);       /* 清除显示 */
            if (g_usb_state_reg & 0x01)                    /* 正在写 */
            {
                LED1(0);
                lcd_show_string(30, 210, 200, 16, 16, "USB Writing...", RED);
                /* 提示 USB 正在写入数据 */
            }
            if (g_usb_state_reg & 0x02)         /* 正在读 */
            {
                LED1(0);
                lcd_show_string(30, 210, 200, 16, 16, "USB Reading...", RED);
                /* 提示 USB 正在读出数据 */
            }
            if (g_usb_state_reg & 0x04)
            {
                lcd_show_string(30, 230, 200, 16, 16, "USB Write Err ", RED);
                /* 提示写入错误 */
            }
            else
            {
                lcd_fill(30, 230, 240, 230 + 16, WHITE);    /* 清除显示 */
            }

            if (g_usb_state_reg & 0x08)
            {
                lcd_show_string(30, 250, 200, 16, 16, "USB Read  Err ", RED);
                /* 提示读出错误 */
            }
            else
            {
                lcd_fill(30, 250, 240, 250 + 16, WHITE);  /* 清除显示 */
```

```
            }

            usb_sta = g_usb_state_reg;      /* 记录最后的状态 */
        }

        if (device_sta != g_device_state)
        {
            if (g_device_state == 1)
            {
                lcd_show_string(30, 190, 200, 16, 16, "USB Connected    ", RED);
                /* 提示 USB 连接已经建立 */
            }
            else
            {
                lcd_show_string(30, 190, 200, 16, 16, "USB DisConnected ",RED);
                /* 提示 USB 被拔出了 */
            }
            device_sta = g_device_state;
        }
        tct++;
        if (tct == 200)
        {
            tct = 0;
            LED1(1);                            /* 关闭 LED1 */
            LED0_TOGGLE();                      /* LED0 闪烁 */

            if (g_usb_state_reg & 0x10)
            {
                offline_cnt = 0;                /* USB 连接了,则清除 offline 计数器 */
                g_device_state = 1;
            }
            else                                /* 没有得到轮询 */
            {
                offline_cnt++;

                if (offline_cnt > 10)
                {
                    g_device_state = 0;     /* 2s 内没收到在线标记,代表 USB 被拔出了 */
                }
            }
            g_usb_state_reg = 0;
        }
    }
}
```

其中，USBD_HandleTypeDef 是一个用于处理 USB 设备类通信处理的结构体类型，它包含了 USB 设备类通信的各种变量、结构体参数、传输状态和端点信息等。凡是 USB 设备类通信，都必须要用定义一个这样的结构体，这里定义成：USBD_Device。

使用 ST 官方提供的 USB 库以后，整个 USB 初始化就变得比较简单了：

1， 调用 USBD_Init 函数，初始化 USB 从机内核；

2， 调用 USBD_RegisterClass 函数，链接 MSC 设备类驱动程序到设备内核；

3， 调用 USBD_MSC_RegisterStorage 函数，为 MSC 设备类驱动添加回调函数；

4， 调用 USBD_Start 函数，启动 USB 通信；

经过以上四步处理，USB 就启动了，所有 USB 事务，都是通过 USB 中断触发，并由 USB 驱动库自动处理。USB 中断服务函数在 usbd_conf.c 里面：

```
/**
 * @brief       USB OTG 中断服务函数
 * @note        处理所有 USB 中断
 * @param       无
 * @retval      无
 */
void OTG_FS_IRQHandler(void)
```

```
{
    HAL_PCD_IRQHandler(&hpcd);
}
```

该函数调用 HAL_PCD_IRQHandler 函数来处理各种 USB 中断请求。因此在 main 函数里面，我们的处理过程就非常简单，main 函数里面通过两个全局状态变量（g_usb_state_reg 和 g_device_state），来判断 USB 状态，并在 LCD 上面显示相关提示信息。

g_usb_state_reg 在 usbd_storage.c 里面定义的一个全局变量，不同的位表示不同状态，用来指示当前 USB 的读写等操作状态。

g_device_state 是在 usbd_conf.c 里面定义的一个全局变量，0 表示 USB 还没有连接；1 表示 USB 已经连接。

69.4 下载验证

将程序下载到开发板后，在 USB 配置成功后（假设已经插入 SD 卡，**注意：USB 数据线，要插在 USB_SLAVE 口！而不是 USB_UART 端口！**），如图 69.4.1 所示：

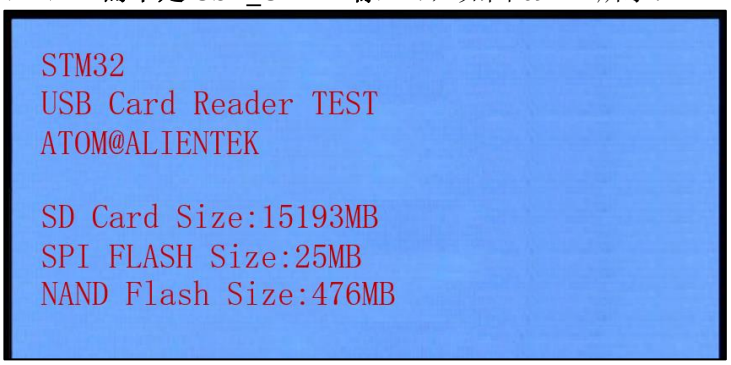


图 69.4.1 USB 连接成功显示界面

此时，电脑提示发现新硬件，并开始自动安装驱动，如图 69.4.2 所示：

图 69.4.2 USB 读卡器被电脑找到

USB 配置成功后，LED1 熄灭，LED0 闪烁，在电脑上可以看到磁盘，如图 69.4.3 所示：

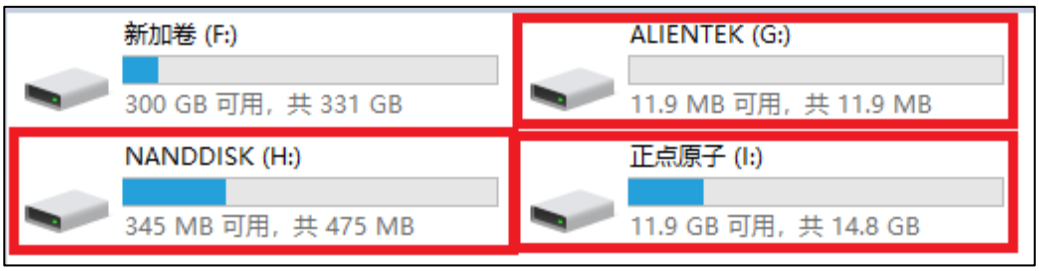


图 69.4.3 电脑找到 USB 读卡器的两个盘符

我们打开设备管理器，在通用串行总线控制器里面可以发现多出了一个 USB 大容量存储设备，同时看到磁盘驱动器里面多了三个磁盘，如图 69.4.4 所示：

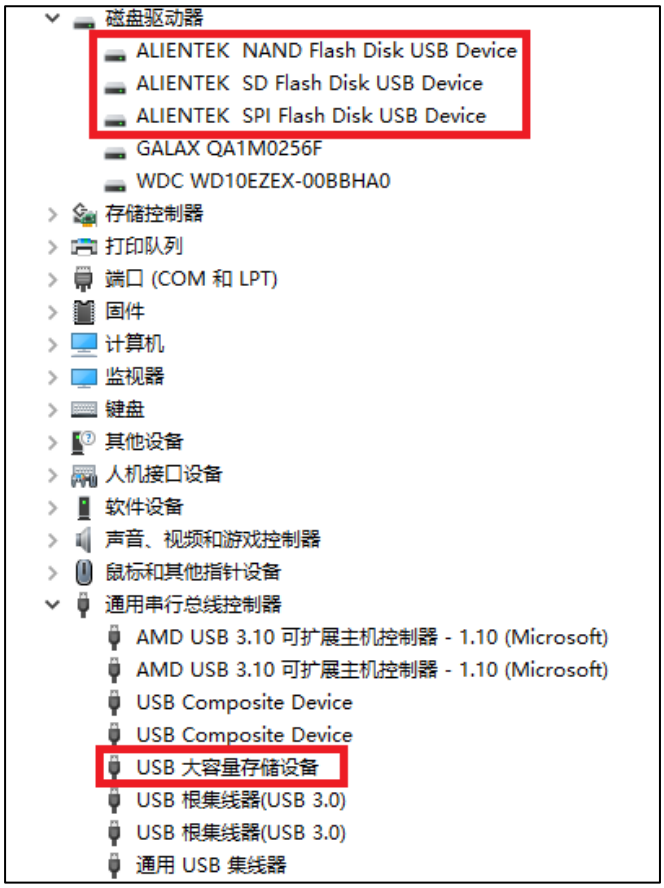


图 69.4.4 通过设备管理器查看磁盘驱动器

此时，我们就可以通过电脑读写 SD 卡、SPI FLASH 和 NAND FLASH 里面的内容了。在执行读写操作的时候，就可以看到 LED1 亮，并且会在液晶上显示当前的读写状态。

注意，在对 SPI FLASH 操作的时候，最好不要频繁的往里面写数据，否则很容易将 SPI FLASH 写爆！

第七十章 USB 声卡(Slave)实验

上一章我们向大家介绍了如何利用 STM32H743 的 USB 接口来做一个 USB 读卡器，本章我们将利用 STM32H743 的 USB 来做一个声卡。本章分为如下几个部分：

70.1 USB 声卡简介
70.2 硬件设计
70.3 软件设计
70.4 下载验证

70.1 USB 声卡简介

USB 声卡依赖于 USB 音频类（USB Audio Class），一个像 USB 通用的数据接口，可以实现数字音频的数据传输。

正点原子阿波罗 STM32H743 开发板板载了一颗高性能 CODEC 芯片：ES8388，我们可以利用 STM32H743 的 SAI 接口，控制 ES8388 播放音乐，同样，如果结合 STM32H743 的 USB 功能，就可以实现一个 USB 声卡。

同上一章一样，我们直接移植官方的 USB AUDIO 例程，官方例程路径：8，STM32 参考资料→1，STM32CubeH7 固件包→STM32Cube_FW_H7_V1.9.0→Projects→STM32H743I_EVAL→Applications→USB_Device→Audio_Standalone，该例程采用 USB 同步传输来传输音频数据流并且支持某些控制命令(比如静音控制)，例程仅支持 USB FS 模式（不支持 HS），同时例程不需要特殊的驱动支持，大多数操作系统直接就可以识别。

70.2 硬件设计

1. 例程功能

本节实验功能简介：先将开发板通过 USB_SLAVE 连接电脑，开发板开机的时候先提示一些信息，然后开始 USB 配置，在配置成功之后就可以在电脑上发现多出一个声卡。我们在 LCD 液晶屏上显示 USB 的连接状态，如果连接成功，此时，我们可以通过耳机或开发板板载喇叭，可以听到来自电脑的音频信号，还可以通过两个按键调节 ES8388 音量：按键 KEY0 可以增大音量，按键 KEY2 可以减少音量。LED0 闪烁，提示程序运行。USB 和电脑连接成功，LED1 常亮，USB 断开连接，LED1 灭。

2. 硬件资源

1）LED 灯
LED0 - PB1
LED1 - PB0
2）独立按键
KEY0 - PH3
KEY1 - PH2
KEY2 - PC13
KEY_UP - PA0 (程序中的宏名:WK_UP)
3）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
4）正点原子 2.8/3.5/4.3/7 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏,都支持)
5）ES8388 音频 CODEC 芯片，通过 SAI 驱动
6）SAI 音频接口
7）USB_SLAVE 接口(D-/D+连接在 PA11/PA12 上)
8）外部 SDRAM 芯片,通过 FMC 驱动

70.3 程序设计

70.3.1 程序流程图

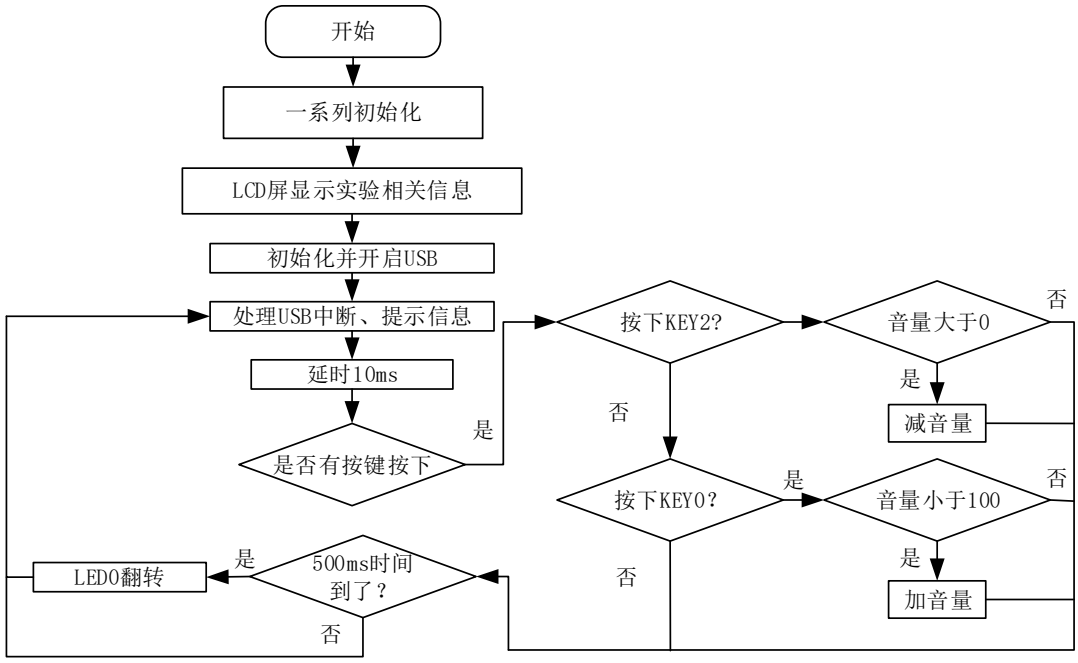


图 70.3.1.1 USB 声卡实验程序流程图

70.3.2 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。

本章，我们在实验 48 音乐播放器实验的基础上修改，把不需要的文件从工程中移除，在 Middlewares 文件夹所在文件夹下新建一个 USB 的文件夹，同上一章一样，对照官方 AUDIO 例子，将相关文件拷贝到 USB 文件夹下。

然后，我们在工程里面去掉一些不必要的代码，并添加 USB 相关代码，最终得到如图 70.3.2.1 所示的工程：

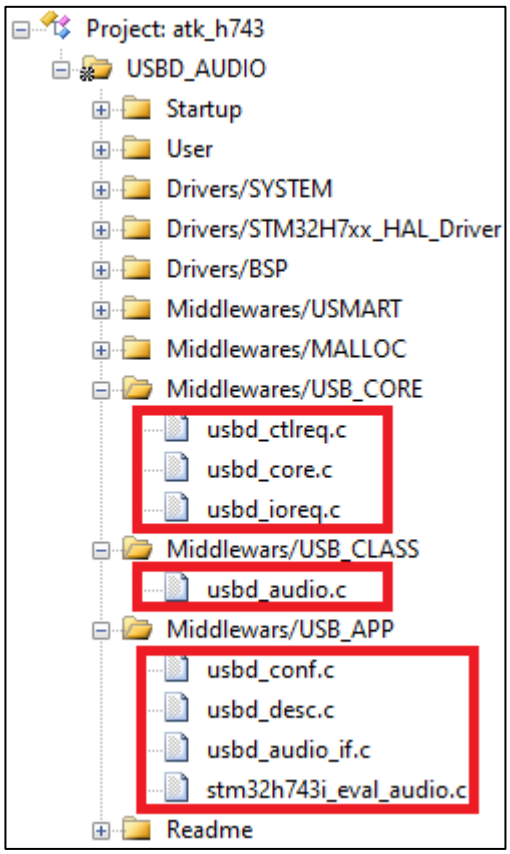


图 70.3.2.1 USB 声卡工程截图

1. USB 驱动代码

可以看到，USB 部分代码，同上一章的在结构上是一模一样的，只是.c 文件稍微有些变化。同样，我们移植需要修改的代码，就是 USB_APP 里面的这四个.c 文件了。

usbd_conf.c 代码，和上一章基本一样，可以用上一章的代码直接替换即可正常使用，但是需要在 **usbd_conf.h** 中定义音频频率，代码如图下。

```
/* AUDIO Class Config */
/* ST USB 库目前只支持这一个采样率，设置其他采样率无法正常工作 */
#define USBD_AUDIO_FREQ                 48000
```

usbd_desc.c/usbd_desc.h 代码，同上一章不一样，上一章描述符是大容量存储设备，本章变成了 USB 声卡了，所以直接用 ST 官方 USB Audio 例程的就行。

usbd_audio_if 代码，是 USB 声卡的接口文件，usbd_audio.c 对外的所有接口函数，都在该.c 文件里面，然后通过 stm32h743i_eval_audio.c 文件里面提供的相关函数，分别实现各个接口函数。该文件不需要做太多修改，这里我们就不做介绍了，详见本例程源码。

stm32h743i_eval_audio.c 代码，是我们重点要修改的，该文件是配合 USB 声卡的 CS43L22 底层驱动相关代码，官方 H743 的板子，用的是 CS43L22，而我们用的是 ES8388，需要修改代码，修改后代码如下：

```
uint8_t g_volume = 0;                   /* 当前音量 */
/* USB 数据输出状态,在 usbd_audio.c 里面定义*/
extern uint8_t usbd_audio_dataout_en;

/**
 * @brief       播放时,SAI DMA 传输回调函数
 * @param       无
 * @retval      无
 */
void audio_sai_dma_callback(void)
{
```

```
    BSP_AUDIO_OUT_TransferComplete_CallBack();       /* 执行回调函数,更新数据 */
}

/**
 * @brief  配置音频接口
 * @param  OutputDevice :输出设备选择,未用到.
 * @param  Volume       :音量大小, 0~100
 * @param  AudioFreq    :音频采样率
 * @retval               操作结果
 *   @arg                0    , 成功
 *   @arg                其他 , 错误代码
 */
uint8_t BSP_AUDIO_OUT_Init(uint16_t OutputDevice, uint8_t Volume,
                                                  uint32_t AudioFreq)
{
    es8388_sai_cfg(0, 3);                                /* 飞利浦标准,16 位数据长度 */
    /* 飞利浦标准,主机发送,时钟低电平有效,16 位扩展帧长度 */
    saia_init(SAI_MODEMASTER_TX, SAI_CLOCKSTROBING_RISINGEDGE,
                                                              SAI_DATASIZE_16);
    saia_samplerate_set(AudioFreq);                      /* 设置采样率 */
    BSP_AUDIO_OUT_SetVolume(Volume);                     /* 设置音量 */
    /* 回调函数指 bsp_audio_sai_dma_callback */
    sai_tx_callback = audio_sai_dma_callback;
    printf("EVAL_AUDIO_Init:%d,%d\r\n", Volume, AudioFreq);

    return 0;
}

/**
 * @brief  开始播放音频数据
 * @param  pBuffer  : 音频数据流首地址指针
 * @param  Size     : 数据流大小(单位:字节)
 * @retval            操作结果
 *   @arg             0    , 成功
 *   @arg             其他 , 错误代码
 */
uint8_t BSP_AUDIO_OUT_Play(uint16_t* pBuffer, uint32_t Size)
{
    saia_tx_dma_init((uint8_t *) pBuffer, 0, Size/2, 1); /* 配置 SAI TX DMA */
    SAIA_TX_DMASx->CR &= ~(1 << 8);                      /* 单次模式 */
    SAIA_TX_DMASx->CR &= ~(1 << 18);                     /* 改成单缓冲模式 */
    sai_play_start();                                    /* 开启 DMA */
    printf("EVAL_AUDIO_Play:%x,%d\r\n", (int)pBuffer, Size);

    return 0;
}

/**
 * @brief  切换音频数据流 buffer,每次 DMA 中断完成会调用该函数
 * @param  pData    : 音频数据流缓存首地址
 * @param  Size     : 音频数据流大小(单位:harf word,也就是 2 个字节)
 * @retval          : 无
  */
void BSP_AUDIO_OUT_ChangeBuffer(uint16_t *pData, uint16_t Size)
{
    if (usbd_audio_dataout_en == 1)        /* USB 有音频数据输出 */
    {
        usbd_audio_dataout_en = 0;         /* 清除标记位 */
    }
    else if (usbd_audio_dataout_en == 0)   /* 没有收到任何数据,全部填充 0 */
    {
        my_mem_set(pData, 0, AUDIO_TOTAL_BUF_SIZE);
```

```
        usbd_audio_dataout_en = 2;          /* 标记已经清零了 buffer,停止数据输出 */
    }

    if (Size != AUDIO_TOTAL_BUF_SIZE)
    {
        // printf("chgbuf:%x,%d\r\n", pData, Size);
        /* 想观察数据,可以 printf,但是会有轻微的影响音质 */
    }

    SAIA_TX_DMASx->CR &= ~(1 << 0);         /* 关闭 DMA 传输 */
    while (SAIA_TX_DMASx->CR & 0X1);        /* 确保 DMA 可以被设置 */
    SAIA_TX_DMASx->NDTR = Size / 2;         /* 设置传输长度 */
    SAIA_TX_DMASx->CR |= 1 << 0;            /* 开启 DMA 传输 */
}

/**
 * @brief  停止播放
 * @retval              操作结果
 *   @arg               0     , 成功
 *   @arg               其他 , 错误代码
 */
uint8_t BSP_AUDIO_OUT_Pause(void)
{
    printf("EVAL_AUDIO_Stop\r\n");
    return 0;
}

/**
 * @brief  停止播放
 * @param  Option    : 未用到
 * @retval              操作结果
 *   @arg               0     , 成功
 *   @arg               其他 , 错误代码
 */
uint8_t BSP_AUDIO_OUT_Stop(uint32_t Option)
{
    printf("EVAL_AUDIO_Stop\r\n");
    return 0;
}

/**
 * @brief  音量设置
 * @param  Volume: 音量大小, 0~100
 * @retval       操作结果
 *   @arg        0     , 成功
 *   @arg        其他 , 错误代码
 */
uint8_t BSP_AUDIO_OUT_SetVolume(uint8_t Volume)
{
    g_volume = Volume;
    es8388_hpvol_set(g_volume * 0.3);
    es8388_spkvol_set(g_volume * 0.3);
    return 0;
}

/**
 * @brief  静音设置
 * @param  Cmd   : 0,不静音; 1,静音
 * @retval         操作结果
 *   @arg          0     , 成功
 *   @arg          其他 , 错误代码
 */
uint8_t BSP_AUDIO_OUT_SetMute(uint32_t Cmd)
```

```
{
    if (Cmd)
    {
        es8388_output_cfg(0, 0);     /* 关闭输出 ,就是静音 */
    }
    else
    {
        es8388_output_cfg(1, 1);     /* 打开输出 ,取消静音 */
    }

    printf("BSP_AUDIO_OUT_SetMute:%d\r\n", Cmd);
    return 0;
}
```

本例程工作的时候，当 USB 连接成功，并完成识别以后，会先调用 BSP_AUDIO_OUT_Init 函数，设置音频接口相关参数，包括：ES8388 的 SAI 配置、音量设置和 SAI DMA 发送完成中断回调函数等。

然后，当上电后首次有音乐输出的时候，会先调用 BSP_AUDIO_OUT_Play 函数，对 SAI 的 DMA 进行配置（使用单次模式，且单缓冲传输），然后启动 SAI DMA 传输，开始播放电脑的声音。然后，当 DMA 传输完成后，会自动停止 DMA（单次模式），并通过 audio_sai_dma_callback 函数调用 BSP_AUDIO_OUT_TransferComplete_CallBack 回调函数，在回调函数里面，经过处理如果有新的音频数据，则会调用 BSP_AUDIO_OUT_ChangeBuffer 函数，重启 DMA 传输，开启下一次数据 DMA 传送，然后 DMA 传输完成又会进入 audio_sai_dma_callback 函数，依次循环。

这样，就实现了 USB 传过来的音频数据，通过 SAI DMA 传输出去，最终通过 ES8388 输出给耳机或开发板板载喇叭，实现 USB 声卡功能。

USB AUDIO 相关代码，就给大家介绍到这里，详细的介绍，请大家参考：UM1734(STM32Cube USB device library).pdf 这个文档。

2. main.c 代码

下面是 main.c 的程序，具体如下：

```
USBD_HandleTypeDef USBD_Device;
extern PCD_HandleTypeDef hpcd;
extern uint8_t g_device_state;          /* USB 连接 情况 */
extern uint8_t g_volume;                /* 音量(可通过按键设置) */

int main(void)
{
    uint8_t key;
    uint8_t t = 0;
    uint8_t device_sta = 0xFF;

    sys_cache_enable();                                 /* 打开 L1-Cache */
    mpu_memory_protection();                            /* 保护相关存储区域 */
    HAL_Init();                                         /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);                 /* 设置时钟, 400Mhz */
    delay_init(400);                                    /* 延时初始化 */
    usart_init(115200);                                 /* 串口初始化 */
    usmart_init(200);                                   /* 初始化 USMART */
    led_init();                                         /* 初始化 LED */
    key_init();                                         /* 初始化 KEY */
    sdram_init();                                       /* 初始化 SDRAM */
    lcd_init();                                         /* 初始化 LCD */
    pcf8574_init();                                     /* 初始化 PCF8574 */

    my_mem_init(SRAMIN);                                /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                                /* 初始化外部内存池(SDRAM) */
    my_mem_init(SRAMDTCM);                              /* 初始化 DTCM 内存池(DTCM) */
```

```
    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "USB Sound Card TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "KEY2:Vol-  KEY0:Vol+", RED);

    lcd_show_string(30, 140, 200, 16, 16, "VOLUME:", BLUE);     /* 提示音量 */
    lcd_show_xnum(30 + 56, 140, AUDIO_DEFAULT_VOLUME, 3, 16, 0x80, BLUE);
    /* 显示音量值 */
    lcd_show_string(30, 180, 200, 16, 16, "USB Connecting...", RED);
    /* 提示正在建立连接 */

    es8388_init();                                      /* ES8388 初始化 */
    es8388_adda_cfg(1, 0);                              /* 开启 DAC 关闭 ADC */
    es8388_output_cfg(1, 1);                            /* DAC 选择通道输出 */
    es8388_hpvol_set(25);                               /* 设置耳机音量 */
    es8388_spkvol_set(15);                              /* 设置喇叭音量 */

    USBD_Init(&USBD_Device, &AUDIO_Desc, 0);            /* 初始化 USB */
    USBD_RegisterClass(&USBD_Device, USBD_AUDIO_CLASS);/* 添加类 */
    USBD_AUDIO_RegisterInterface(&USBD_Device, &USBD_AUDIO_fops);
    /* 为 MSC 类添加回调函数 */
    USBD_Start(&USBD_Device);                             /* 开启 USB */
    delay_ms(1800);
    while (1)
    {
        key = key_scan(1);                  /* 支持连按 */
        if (key)
        {
            if (key == KEY0_PRES)           /* KEY0 按下,音量增加 */
            {
                g_volume++;

                if (g_volume > 100)         /* 范围限定 */
                {
                    g_volume = 100;
                }
            }
            else if (key == KEY2_PRES)      /* KEY2 按下,音量减少 */
            {
                if (g_volume)               /* 范围限定 */
                {
                    g_volume--;
                }
            }

            es8388_hpvol_set(g_volume * 0.3);
            es8388_spkvol_set(g_volume * 0.3);
            lcd_show_xnum(30 + 56, 140, g_volume, 3, 16, 0x80, BLUE);
            /* 显示音量值 */
            delay_ms(20);
        }
        if (device_sta != g_device_state)           /* 状态改变了 */
        {
            if (g_device_state == 1)
            {
              lcd_show_string(30, 180, 200, 16, 16, "USB Connected    ", RED);
              /* 提示 USB 连接已经建立 */
              LED1(0);     /* LED1 亮 */
            }
            else
            {
```

```
                lcd_show_string(30, 180, 200, 16, 16, "USB DisConnected ", RED);
                /* 提示 USB 连接失败 */
                 LED1(1);     /* LED1 灭 */
            }
            device_sta = g_device_state;
        }
        delay_ms(20);
        t++;
        if (t > 10)
        {
            t = 0;
            LED0_TOGGLE();  /* 提示系统在运行 */
        }
    }
}
```

此部分代码比较简单，同上一章一样定义了 USBD_Device 结构体，然后通过 USBD_Init 等函数初始化 USB，不过本章实现的是 USB 声卡功能。本章我们保留了原例程（实验 48）的 USMART 部分，同样可以通过串口 1 设置 ES8388 相关参数。

其他部分我们就不详细介绍了，软件设计部分就为大家介绍到这里。

70.4 下载验证

在代码编译成功之后，我们通过下载代码到阿波罗 STM32H743 开发板上，在 USB 配置成功后（注意：USB 数据线，要插在 USB_SLAVE 端口！不是 USB_UART 端口！另外，USB_HOST 接口不要插任何外设！），LCD 显示效果如图 70.4.1 所示：

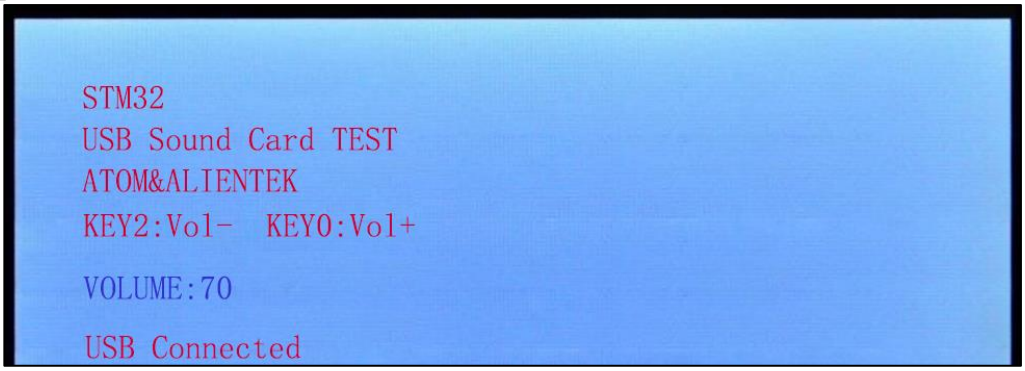


图 70.4.1 USB 连接成功

此时，电脑提示发现新硬件，并自动完成驱动安装。

等 USB 配置成功后，LED1 常亮，LED0 闪烁，并且在设备管理器→声音、视频和游戏控制器里面看到多了设备，如图 70.4.2 所示：

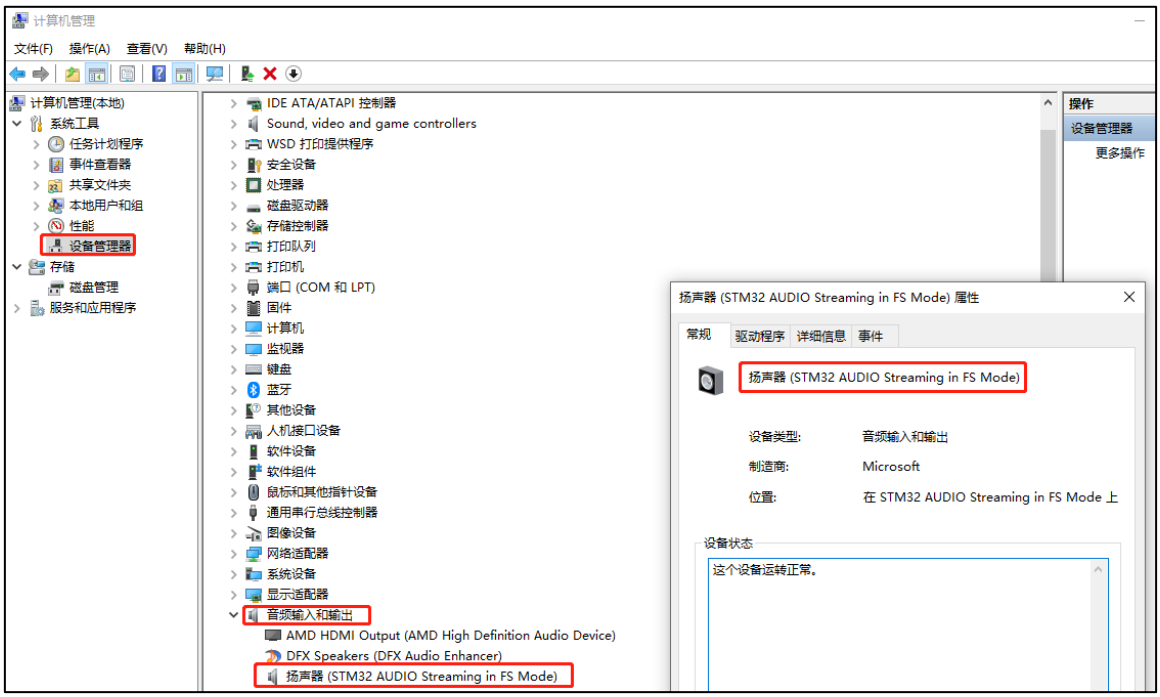


图 70.4.2 设备管理器找到 STM32 USB 声卡

然后，设置 STM32 AUDIO Steaming in FS Mode 为电脑的默认播放设备，则电脑的所有声音都被切换到开发板输出，将耳机插入阿波罗 STM32H743 开发板的 PHONE 端口，即可听到来自电脑的声音，开发板板载喇叭也可以听到来自电脑的声音。通过按键 KEY0/KEY2 可以增大/减少音量，默认音量设置的是 70，大家可以自己调节（范围：0~100）。

第七十一章 USB 虚拟串口(Slave)实验

本章，我们将向大家介绍如何利用 USB 在开发板实现一个 USB 虚拟串口，通过 USB 与电脑数据数据交互。

本章分为如下几个小节：

71.1 USB 虚拟串口简介
71.2 硬件设计
71.3 程序设计
71.4 下载验证

71.1 USB 虚拟串口简介

USB 虚拟串口，简称 VCP，是 Virtual COM Port 的简写，它是利用 USB 的 CDC 类来实现的一种通信接口。

我们可以利用 STM32 自带的 USB 功能，来实现一个 USB 虚拟串口，从而通过 USB，实现电脑与 STM32 的数据互传。上位机无需编写专门的 USB 程序，只需要一个串口调试助手即可调试，非常实用。

同上一章一样，我们直接移植官方的 USB VCP 例程，官方例程路径：：8，STM32 参考资料→1， STM32CubeH7 固件包→ STM32Cube_FW_H7_V1.9.0→Projects→STM32H743I-EVAL→Applications→USB_Device→CDC_Standalone，该例程采用 USB CDC 类来实现，利用 STM32 的 USB 接口，实现一个 USB 转串口的功能。

71.2 硬件设计

1. 例程功能

本实验利用 STM32 自带的 USB 功能，连接电脑 USB，虚拟出一个 USB 串口，实现电脑和开发板的数据通信。本例程功能完全同实验 4（串口通信实验），只不过串口变成了 STM32 的 USB 虚拟串口。当 USB 连接电脑（USB 线插入 USB_SLAVE 接口），开发板将通过 USB 和电脑建立连接，并虚拟出一个串口（注意：需要先安装:光盘\6，软件资料\1，软件\STM32 USB 虚拟串口驱动\VCP_V1.4.0_Setup.exe 这个驱动软件，虚拟串口驱动我们还可以在论坛上下载，链接是：http://www.openedv.com/thread-284178-1-1.html）。

LED0 闪烁，提示程序运行。USB 和电脑连接成功后，LED1 常亮。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
LED1：LED1 - PB0
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)
4）USB_SLAVE 接口(D-/D+连接在 PA11/PA12 上)

71.3 程序设计

71.3.1 程序流程图

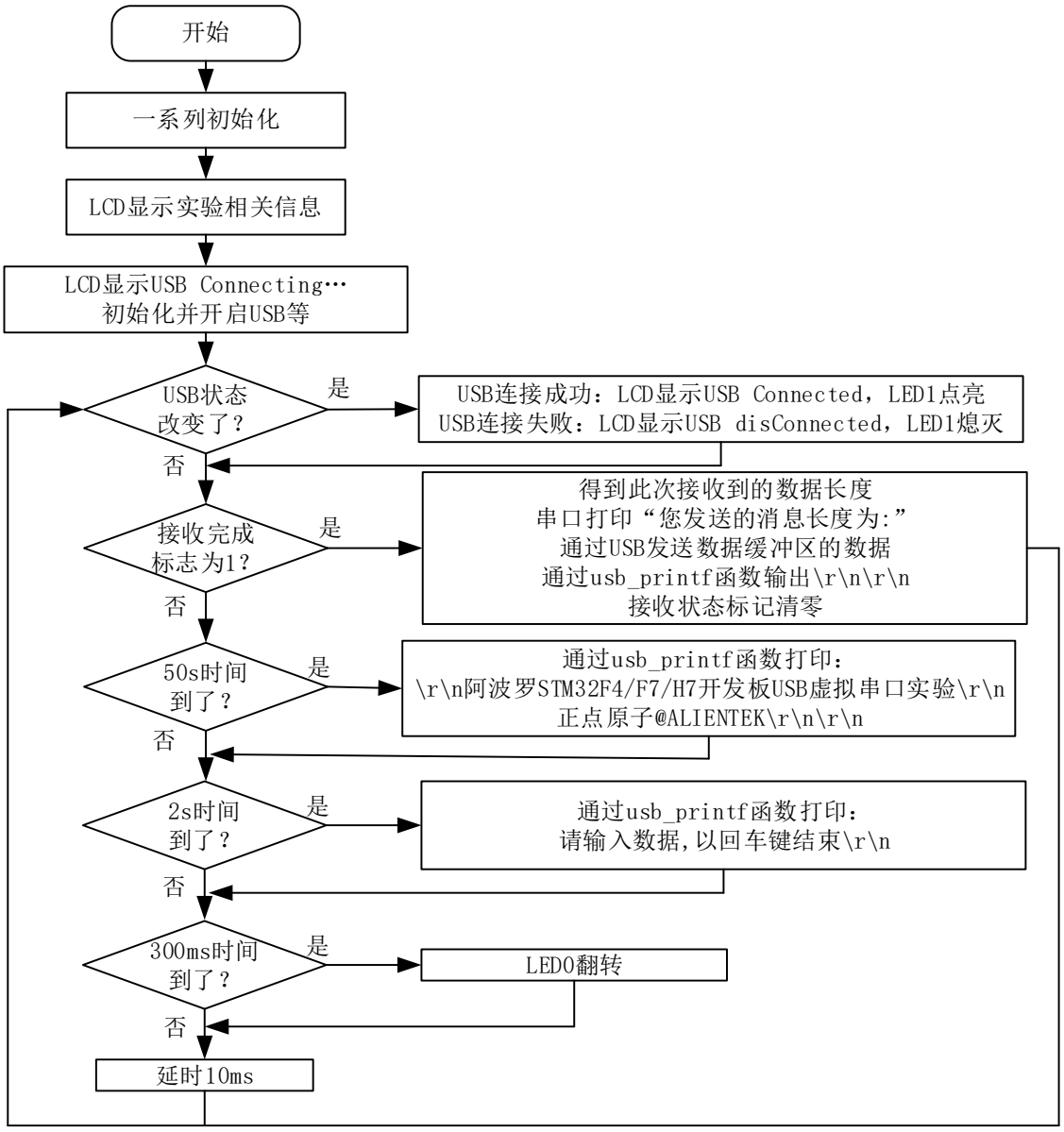


图 71.3.1.1 USB 虚拟串口(Slave)实验程序流程图

71.3.2 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。

本实验，在上一个实验的基础上，把不需要的文件从工程中移除，并对照官方 VCP 例子，将相关文件拷贝到 USB 文件夹下。然后，添加 USB 相关代码到工程中，最终得到如图 71.3.2.1 所示的工程：

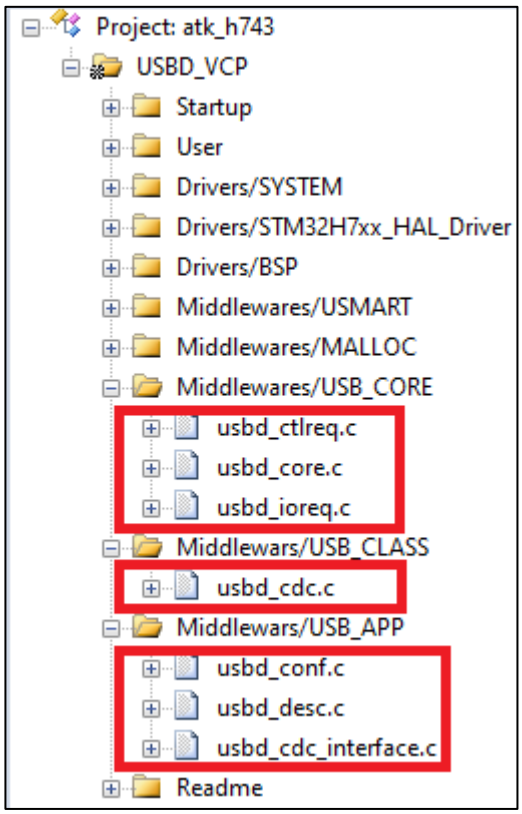


图 71.3.2.1 USB 虚拟串口工程分组

注意：因为 USB 驱动库需要用到内存管理，因此我们保留了相关代码（malloc.c）。

1. USB 驱动代码

可以看到，USB 部分代码，同上一个实验的在结构上是一模一样的，只是.c 文件稍微有些变化。同样，我们移植需要修改的代码，就是 USB_APP 里面的这三个.c 文件了。

usbd_conf.c 代码，和上一个实验一样，不需要修改，可以直接使用上一个实验的代码。

usbd_desc.c 代码，同上一个实验不一样，上一个实验描述符是 USB Audio 设备，本实验变成了 USB 虚拟串口了（CDC），所以直接用 ST 官方的就行。

usbd_cdc_interface.c 代码，是重点要修改的，首先介绍 usbd_cdc_interface.h 文件的相关宏定义，具体如下：

```
#define USB_USART_REC_LEN       200     /* USB 串口接收缓冲区最大字节数 */

/* 轮询周期，最大 65ms，最小 1ms */
#define CDC_POLLING_INTERVAL    1       /* 轮询周期，最大 65ms，最小 1ms */
```

USB_USART_REC_LEN 宏定义是用于定义 USB 串口接收缓冲区最大字节数，这里设置为 200。CDC_POLLING_INTERVAL 宏定义是用于定义 USB 发送数据轮询周期，作为 delay_ms 函数的参数，最大 65ms，最小 1ms，这里设置为最小值即可。

下面重点介绍 usbd_cdc_interface.c 文件，首先是一些结构体变量、数组和变量的定义，具体如下：

```
/* USB 虚拟串口相关配置参数 */
USBD_CDC_LineCodingTypeDef LineCoding =
{
    115200,     /* 波特率 */
    0x00,       /* 停止位,默认 1 位 */
    0x00,       /* 校验位,默认无 */
    0x08        /* 数据位,默认 8 位 */
};

/* usb_printf 发送缓冲区，用于 vsprintf */
```

```
uint8_t g_usb_usart_printf_buffer[USB_USART_REC_LEN];

/* USB 接收的数据缓冲区,最大 USART_REC_LEN 个字节,用于 USBD_CDC_SetRxBuffer 函数 */
uint8_t g_usb_rx_buffer[USB_USART_REC_LEN];

/* 用类似串口 1 接收数据的方法,来处理 USB 虚拟串口接收到的数据 */
uint8_t g_usb_usart_rx_buffer[USB_USART_REC_LEN];
/* 接收缓冲,最大 USART_REC_LEN 个字节 */

/* 接收状态
 * bit15   , 接收完成标志
 * bit14   , 接收到 0x0d
 * bit13~0 , 接收到的有效字节数目
 */
uint16_t g_usb_usart_rx_sta=0;  /* 接收状态标记 */

extern USBD_HandleTypeDef USBD_Device;
static int8_t CDC_Itf_Init(void);
static int8_t CDC_Itf_DeInit(void);
static int8_t CDC_Itf_Control(uint8_t cmd, uint8_t *pbuf, uint16_t length);
static int8_t CDC_Itf_Receive(uint8_t *pbuf, uint32_t *Len);

/* 虚拟串口配置函数(供 USB 内核调用) */
USBD_CDC_ItfTypeDef USBD_CDC_fops =
{
    CDC_Itf_Init,
    CDC_Itf_DeInit,
    CDC_Itf_Control,
    CDC_Itf_Receive
};
```

首先是定义一个 USBD_CDC_LineCodingTypeDef 结构体类型的变量 LineCoding，并赋值。波特率为 115200，停止位和校验位都为 0，数据位，默认 8 位。

g_usb_usart_printf_buffer 是发送缓冲区，大小由 USB_USART_REC_LEN 宏来定义，数组是 uint8_t 类型，所以数字大小为 200 字节。

g_usb_rx_buffer 则是 USB 接收的数据缓冲区，用于 USBD_CDC_SetRxBuffer 函数，大小也是 200 字节。

g_usb_usart_rx_buffer 是用做类似串口 1 接收数据的方法，来处理 USB 虚拟串口接收到的数据，在 cdc_vcp_data_rx 函数中被调用，大小也是 200 字节。

g_usb_usart_rx_sta 变量用于表示接收状态，位 15 表示接收完成标志，位 14 表示接收到 0x0d，位 13~位 0 表示接收到的有效字节数目。

最后定义一个 USBD_CDC_ItfTypeDef 结构体类型的变量 USBD_CDC_fops，供 USB 内核调用，并把四个函数的首地址赋值给其成员。下面会介绍到这几个函数，以及一些其它的函数。

首先是初始化 CDC 函数，其定义如下：

```
/**
 * @brief       初始化 CDC
 * @param       无
 * @retval      USB 状态
 *   @arg       USBD_OK(0)    , 正常;
 *   @arg       USBD_BUSY(1)  , 忙;
 *   @arg       USBD_FAIL(2)  , 失败;
 */
static int8_t CDC_Itf_Init(void)
{
    USBD_CDC_SetRxBuffer(&USBD_Device, g_usb_rx_buffer);
    return USBD_OK;
}
```

CDC_Itf_Init 用于初始化 VCP，在初始化的时候由 USB 内核调用，这里我们调用函数：USBD_CDC_SetRxBuffer，设置 USB 接收数据缓冲区。USB 虚拟串口收到的数据，会先缓存在这个 buf 里面。

下面介绍的是复位 CDC 函数，其定义如下：

```
/**
 * @brief       复位 CDC
 * @param       无
 * @retval      USB 状态
 *   @arg       USBD_OK(0)    , 正常;
 *   @arg       USBD_BUSY(1)  , 忙;
 *   @arg       USBD_FAIL(2)  , 失败;
 */
static int8_t CDC_Itf_DeInit(void)
{
    return USBD_OK;
}
```

CDC_Itf_DeInit 用于复位 VCP，我们用不到，所以直接返回 USBD_OK 即可。

下面介绍的是控制 CDC 的设置函数，其定义如下：

```
/**
 * @brief       控制 CDC 的设置
 * @param       cmd     : 控制命令
 * @param       buf     : 命令数据缓冲区/参数保存缓冲区
 * @param       length  : 数据长度
 * @retval      USB 状态
 *   @arg       USBD_OK(0)    , 正常;
 *   @arg       USBD_BUSY(1)  , 忙;
 *   @arg       USBD_FAIL(2)  , 失败;
 */
static int8_t CDC_Itf_Control(uint8_t cmd, uint8_t *pbuf, uint16_t length)
{
    switch (cmd)
    {
        case CDC_SEND_ENCAPSULATED_COMMAND:
            break;
        case CDC_GET_ENCAPSULATED_RESPONSE:
            break;
        case CDC_SET_COMM_FEATURE:
            break;
        case CDC_GET_COMM_FEATURE:
            break;
        case CDC_CLEAR_COMM_FEATURE:
            break;
        case CDC_SET_LINE_CODING:
            LineCoding.bitrate = (uint32_t) (pbuf[0] | (pbuf[1] << 8) |
                                             (pbuf[2] << 16) | (pbuf[3] << 24));
            LineCoding.format = pbuf[4];
            LineCoding.paritytype = pbuf[5];
            LineCoding.datatype = pbuf[6];
            /* 打印配置参数 */
            printf("linecoding.format:%d\r\n", LineCoding.format);
            printf("linecoding.paritytype:%d\r\n", LineCoding.paritytype);
            printf("linecoding.datatype:%d\r\n", LineCoding.datatype);
            printf("linecoding.bitrate:%d\r\n", LineCoding.bitrate);
            break;
        case CDC_GET_LINE_CODING:
            pbuf[0] = (uint8_t) (LineCoding.bitrate);
            pbuf[1] = (uint8_t) (LineCoding.bitrate >> 8);
            pbuf[2] = (uint8_t) (LineCoding.bitrate >> 16);
            pbuf[3] = (uint8_t) (LineCoding.bitrate >> 24);
            pbuf[4] = LineCoding.format;
            pbuf[5] = LineCoding.paritytype;
            pbuf[6] = LineCoding.datatype;
            break;
        case CDC_SET_CONTROL_LINE_STATE:
            break;
        case CDC_SEND_BREAK:
```

```
            break;
        default:
            break;
    }
    return USBD_OK;
}
```

CDC_Itf_Control 用于控制 VCP 的相关参数，根据 cmd 的不同，执行不同的操作，这里主要用到 CDC_SET_LINE_CODING 命令，该命令用于设置 VCP 的相关参数，比如波特率、数据类型（位数）、校验类型（奇偶校验）等，保存在 linecoding 结构体里面，在需要的时候，应用程序可以读取 LineCoding 结构体里面的参数，以获得当前 VCP 的相关信息。

下面介绍的是 CDC 数据接收函数和处理从 USB 虚拟串口接收到的数据函数，它们的定义如下：

```
/**
 * @brief       CDC 数据接收函数
 * @param       buf     : 接收数据缓冲区
 * @param       len     : 接收到的数据长度
 * @retval      USB 状态
 *   @arg       USBD_OK(0)    , 正常;
 *   @arg       USBD_BUSY(1)  , 忙;
 *   @arg       USBD_FAIL(2)  , 失败;
 */
static int8_t CDC_Itf_Receive(uint8_t *buf, uint32_t *len)
{
    USBD_CDC_ReceivePacket(&USBD_Device);
    cdc_vcp_data_rx(buf, *len);
    return USBD_OK;
}

/**
 * @brief       处理从 USB 虚拟串口接收到的数据
 * @param       buf     : 接收数据缓冲区
 * @param       len     : 接收到的数据长度
 * @retval      无
 */
void cdc_vcp_data_rx (uint8_t *buf, uint32_t len)
{
    uint8_t i;
    uint8_t res;

    for (i = 0; i < len; i++)
    {
        res = buf[i];
        if ((g_usb_usart_rx_sta & 0x8000) == 0)         /* 接收未完成 */
        {
            if (g_usb_usart_rx_sta & 0x4000)            /* 接收到了 0x0d */
            {
                if (res != 0x0a)
                {
                    g_usb_usart_rx_sta = 0;             /* 接收错误,重新开始 */
                }
                else
                {
                    g_usb_usart_rx_sta |= 0x8000;       /* 接收完成了 */
                }
            }
            else    /* 还没收到 0X0D */
            {
                if (res == 0x0d)
                {
                    g_usb_usart_rx_sta |= 0x4000;       /* 标记接收到了 0X0D */
                }
                else
```

```
                {
                    g_usb_usart_rx_buffer[g_usb_usart_rx_sta & 0X3FFF] = res;
                    g_usb_usart_rx_sta++;
                    if (g_usb_usart_rx_sta > (USB_USART_REC_LEN - 1))
                    {
                        g_usb_usart_rx_sta = 0;  /* 接收数据溢出 重新开始接收 */
                    }
                }
            }
        }
    }
}
```

CDC_Itf_Receive 和 cdc_vcp_data_rx，这两个函数一起，用于 VCP 数据接收，当 STM32 的 USB 接收到电脑端串口发送过来的数据时，由 USB 内核程序调用 CDC_Itf_Receive，然后在该函数里面再调用 cdc_vcp_data_rx 函数，实现 VCP 的数据接收，只需要在该函数里面，将接收到的数据，保存起来即可，接收的原理和（实验 4 串口通信实验）完全一样。

下面介绍的是通过 USB 发送数据函数，其定义如下：

```
/**
 * @brief       通过 USB 发送数据
 * @param       buf     : 要发送的数据缓冲区
 * @param       len     : 数据长度
 * @retval      无
 */
void cdc_vcp_data_tx(uint8_t *data, uint32_t len)
{
    USBD_CDC_SetTxBuffer(&USBD_Device, data, len);
    USBD_CDC_TransmitPacket(&USBD_Device);
    delay_ms(CDC_POLLING_INTERVAL);
}
```

cdc_vcp_data_rx 用于发送 len 个字节的数据给 VCP，由 VCP 通过 USB 传输给电脑，实现 VCP 的数据发送。

下面介绍的是通过 USB 格式化输出函数，其定义如下：

```
/**
 * @brief       通过 USB 格式化输出函数
 * @note        通过 USB VCP 实现 printf 输出
 *              确保一次发送数据长度不超 USB_USART_REC_LEN 字节
 * @param       格式化输出
 * @retval      无
 */
void usb_printf(char *fmt, ...)
{
    uint16_t i;
    va_list ap;
    va_start(ap, fmt);
    vsprintf((char *)g_usb_usart_printf_buffer, fmt, ap);
    va_end(ap);
    i = strlen((const char *)g_usb_usart_printf_buffer);  /* 此次发送数据的长度 */
    cdc_vcp_data_tx(g_usb_usart_printf_buffer, i);        /* 发送数据 */
}
```

usb_printf 用于实现和普通串口一样的 printf 操作，该函数将数据格式化输出到 USB VCP，功能完全同 printf，方便大家使用。

USB VCP 相关代码，就给大家介绍到这里，详细的介绍，请大家参考：UM1734(STM32Cube USB device library).pdf 这个文档。

2. main.c 代码

下面是 main.c 的程序，具体如下：

```
USBD_HandleTypeDef USBD_Device;             /* USB Device 处理结构体 */
extern volatile uint8_t g_device_state;     /* USB 连接 情况 */
```

```
int main(void)
{
    uint16_t len;
    uint16_t times = 0;
    uint8_t usbstatus = 0;

    sys_cache_enable();                     /* 打开 L1-Cache */
    mpu_memory_protection();                /* 保护相关存储区域 */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */

    my_mem_init(SRAMIN);                    /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                    /* 初始化外部内存池(SDRAM) */
    my_mem_init(SRAM12);                    /* 初始化 SRAM12 内存池(SRAM1+SRAM2) */
    my_mem_init(SRAM4);                     /* 初始化 SRAM4 内存池(SRAM4) */
    my_mem_init(SRAMDTCM);                  /* 初始化 DTCM 内存池(DTCM) */
    my_mem_init(SRAMITCM);                  /* 初始化 ITCM 内存池(ITCM) */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "USB Virtual USART TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 110, 200, 16, 16, "USB Connecting...", RED);
    /* 提示 USB 开始连接 */

    USBD_Init(&USBD_Device, &VCP_Desc, USE_USB_FS);     /* 初始化 USB */
    USBD_RegisterClass(&USBD_Device, USBD_CDC_CLASS);   /* 添加类 */
    USBD_CDC_RegisterInterface(&USBD_Device, &USBD_CDC_fops);
    /* 为 MSC 类添加回调函数 */
    USBD_Start(&USBD_Device);    /* 开启 USB */
    delay_ms(1800);

    while (1)
    {
        delay_ms(1);
        if (usbstatus != g_device_state)     /* USB 连接状态发生了改变 */
        {
            usbstatus = g_device_state;      /* 记录新的状态 */
            if (usbstatus == 1)
            {
              lcd_show_string(30, 110, 200, 16, 16, "USB Connected    ", BLUE);
              /* 提示 USB 连接成功 */
              LED1(0);         /* 绿灯亮 */
            }
            else
            {
              lcd_show_string(30, 110, 200, 16, 16, "USB disConnected ", BLUE);
              /* 提示 USB 断开 */
              LED1(1);       /* 绿灯灭 */
            }
        }
        if (g_usb_usart_rx_sta & 0x8000)
        {
            len = g_usb_usart_rx_sta & 0x3FFF;  /* 得到此次接收到的数据长度 */
            usb_printf("\r\n 您发送的消息长度为:%d\r\n\r\n", len);
            cdc_vcp_data_tx(g_usb_usart_rx_buffer, len);
            usb_printf("\r\n\r\n");     /* 插入换行 */
            g_usb_usart_rx_sta = 0;
```

```
        }
        else
        {
            times++;
            if (times % 5000 == 0)
            {
                usb_printf("\r\nSTM32 开发板 USB 虚拟串口实验\r\n");
                usb_printf("正点原子@ALIENTEK\r\n\r\n");
            }
            if (times % 200 == 0)usb_printf("请输入数据,以回车键结束\r\n");
            if (times % 30 == 0)
            {
                LED0_TOGGLE();     /* 闪烁 LED,提示系统正在运行 */
            }
            delay_ms(10);
        }
    }
}
```

此部分代码比较简单，首先定义了 USBD_Device 结构体，然后通过 USBD_Init 等函数初始化 USB，不过本章实现的是 USB 虚拟串口的功能。然后在死循环里面轮询 USB 状态并检查是否接收到数据，如果接收到了数据，则通过 VCP_DataTx 将数据通过 VCP 原原本本的返回给电脑端串口调试助手。

71.4 下载验证

本例程的测试，需要在电脑上先安装 ST 提供的 USB 虚拟串口驱动软件，该软件（V1.5.0 版）下载地址：http://www.openedv.com/thread-284178-1-1.html，下载完以后，根据自己电脑的系统，选择合适的驱动安装即可。

将程序下载到开发板后（注意：USB 数据线，要插在 USB_SLAVE 口！而不是 USB_UART 端口！），我们打开设备管理器（我用的是 WIN10），在端口（COM 和 LPT）里面可以发现多出了一个 COM15 的设备，这就是 USB 虚拟的串口设备端口，如图 71.4.1 所示：

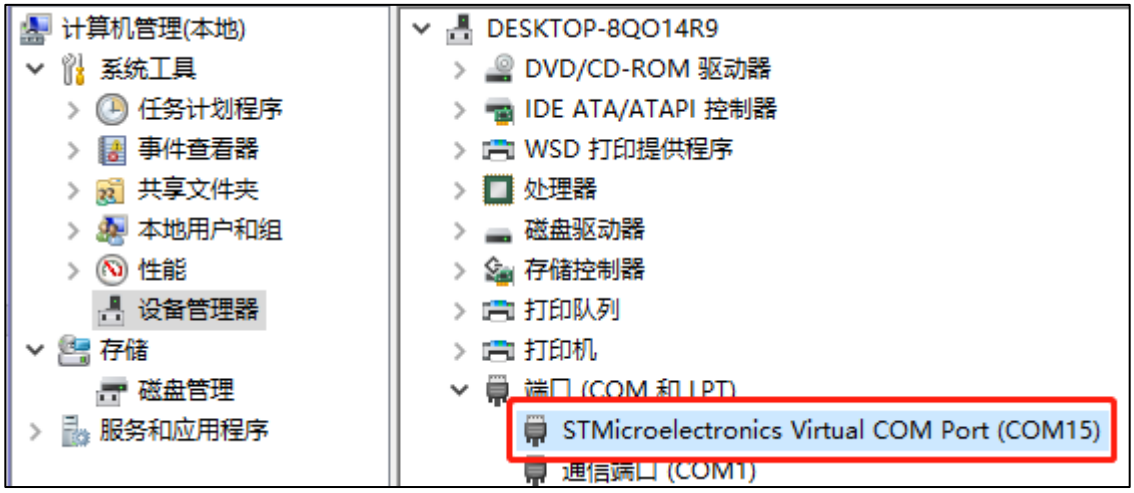


图 71.4.1 通过设备管理器查看 USB 虚拟的串口设备端口

如图 71.4.1，STM32 通过 USB 虚拟的串口，被电脑识别了，端口号为：COM15（可变），字符串名字为：STMicroelectronics Virtual COM Port（COM15）。此时，开发板的 LDE1 常亮，同时，LED0 在闪烁，提示程序运行。开发板的 LCD 显示 USB Connected，如图 71.4.2 所示：

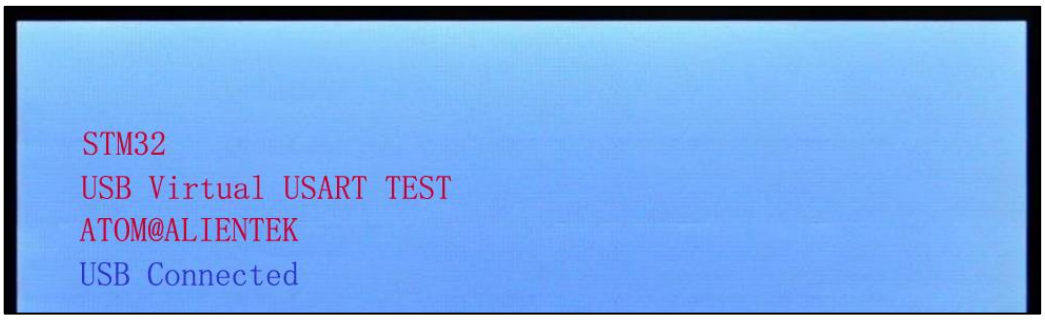


图 71.4.2 USB 虚拟串口连接成功

然后我们打开 XCOM，选择 COM15（需根据自己的电脑识别到的串口号选择），并打开串口（注意：波特率可以随意设置），就可以进行测试了，如图 71.4.3 所示：

图 71.4.3 STM32 虚拟串口通信测试

可以看到，我们的串口调试助手，收到了来自 STM32 开发板的数据，同时，按发送按钮（串口助手必须勾选：发送新行），也可以收到电脑发送给 STM32 的数据（原样返回），说明我们的实验是成功的。实验现象同第十六章完全一样。

至此，USB 虚拟串口实验就完成了，通过本实验，我们就可以利用 STM32 的 USB，直接和电脑进行数据互传了，具有广泛的应用前景。

第七十二章 USB U 盘(Host)实验

本章我们介绍 STM32H743 的 USB HOST 应用，即通过 USB HOST 功能，实现读写 U 盘/读卡器等大容量 USB 存储设备。

本章分为如下几个小节：

72.1 U 盘简介
72.2 硬件设计
72.3 程序设计
72.4 下载验证

72.1 U 盘简介

U 盘，全称 USB 闪存盘，英文名“USB flash disk”。它是一种使用 USB 接口的无需物理驱动器的微型高容量移动存储产品，通过 USB 接口与主机连接，实现即插即用，是最常用的移动存储设备之一。

STM32H743 的 USB OTG FS 支持 U 盘，并且 ST 官方提供了 USB HOST 大容量存储设备（MSC）例程，ST 官方例程路径：光盘→8，STM32 参考资料→1，STM32CubeH7 固件包 → STM32Cube_FW_H7_V1.9.0 → Projects→STM32H743I-EVAL → Applications → USB_Host →MSC_Standalone。本实验，我们就要移植该例程到开发板上，以通过 STM32H743 的 USB HOST 接口，读写 U 盘或 SD 卡读卡器等设备。

72.2 硬件设计

1. 例程功能

本实验代码，开机后，检测字库，然后初始化 USB HOST，并不断轮询。当检测并识别 U 盘后，在 LCD 上面显示 U 盘总容量和剩余容量，此时便可以通过 USMART 调用 FATFS 相关函数，来测试 U 盘数据的读写了，方法同 FATFS 实验一模一样。

LED0 闪烁，提示程序运行。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
LED1：LED1 - PB0
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)
4）USB_HOST 接口(D-/D+连接在 PA11/PA12 上)

3. 原理图

本开发板的 USB HOST 接口采用的是贴片 USB 母座，它和 USB SLAVE 的 Type-C 接头是共用 USB_DM 和 USB_DP 信号的，所以 USB HOST 和 USB SLAVE 功能不能同时使用。USB HOST 和 STM32 的连接原理图，如下图所示：

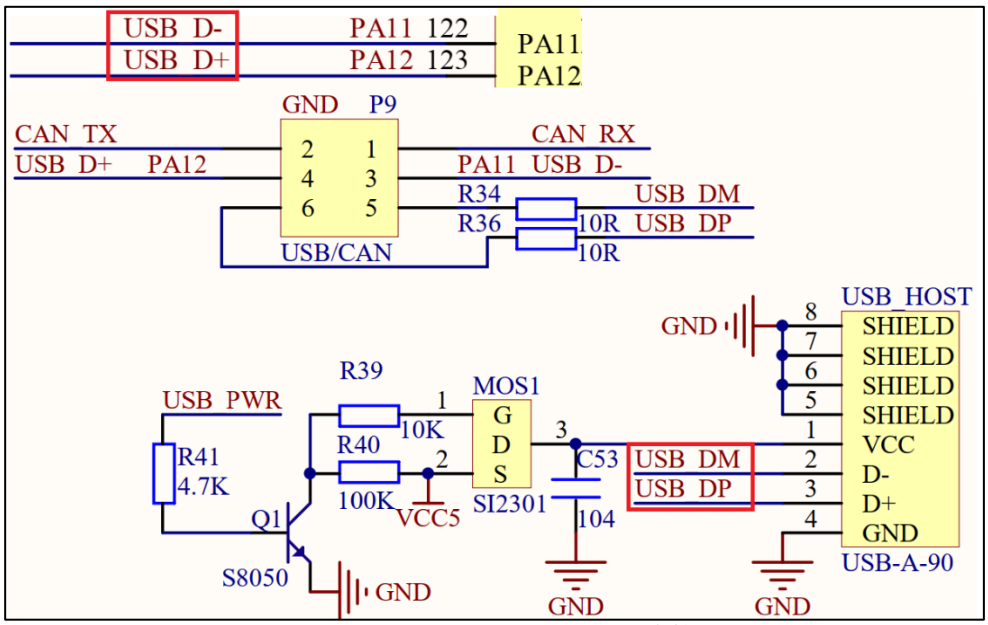


图 72.2.1 USB 母座与 STM32 的连接电路图

从上图可以看出，USB HOST 通过 USB_PWR 控制电源供电，该信号连接在 PCF8574T 的 P3 口上面，通过 PCF8574T 进行间接控制。PCF8574T 的使用说明请见第三十八章 IO 扩展实验。USB_D+和 USB_D-通过跳线帽连接到 STM32H743，所以硬件上不需要我们做什么操作，可直接使用。

需要注意的是：这个 USB 母座（USB_HOST）和 Type-C 接口是共用 D+和 D-的，所以他们不能同时使用。这个在使用的时候，要特别注意！本实验测试时，数据线要用 USB_ UART 接口连接到电脑！

下面，看看开发板上的 USB 母座（USB_HOST）的位置，请将 U 盘插入这个接口上。

图 72.2.2 U 盘与开发板的连接示意图

72.3 程序设计

72.3.1 程序流程图

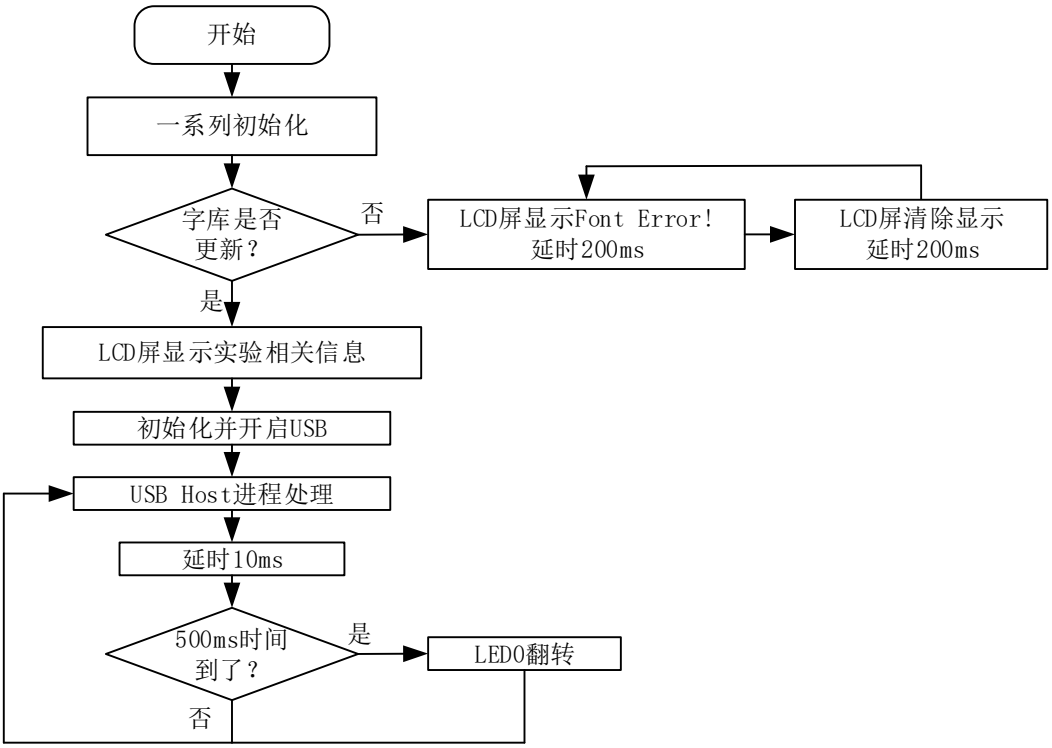


图 72.3.1.1 USB U 盘(Host)实验程序流程图

72.3.2 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。

本实验，我们在：实验 45 图片显示实验的基础上修改，代码移植自 ST 官方例程：STM32Cube_FW_H7_V1.9.0\Projects\STM32H743I_EVAL\Applications\USB_Host\MSC_Standalone。有了这个官方例程做指引，我们就知道具体需要哪些文件，从而实现本实验。

本实验的具体移植步骤，我们这里就不一一介绍了，最终移植好之后的工程分组截图，如图 72.3.2.1 所示：

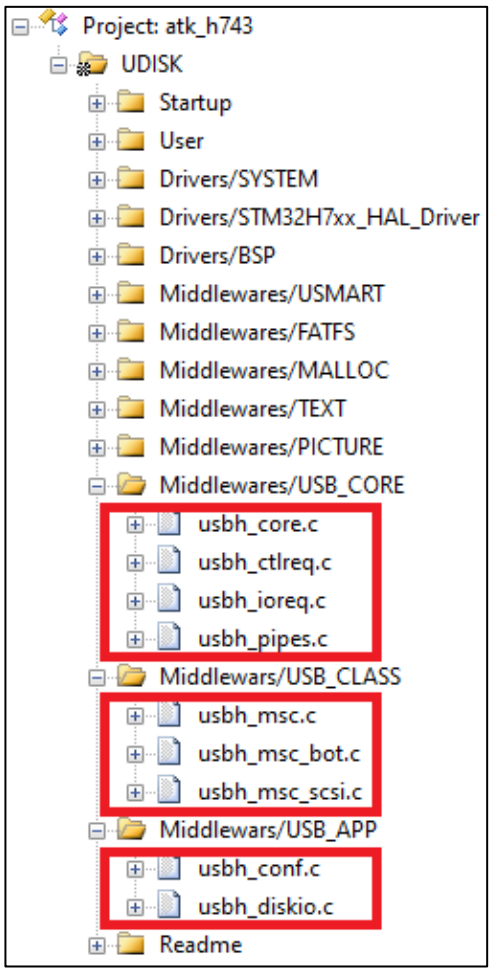


图 72.3.2.1 USB U 盘(Host)实验工程分组

上图工程分组中的 Middlewares/USB_CORE 和 Middlewares/USB_CLASS 分组下的.c 文件，直接拷贝 ST 官方 USB HOST 库，我们重点要修改的是 USB_APP 文件夹下面的代码，接下来重点介绍下。

1. USB 驱动代码

usbh_conf.c 提供了 USB 主机库的回调及 MSP 初始化函数，当 USB 状态机处理完不同事务的时候，会调用这些回调函数，我们通过这些回调函数，就可以知道 USB 当前状态，比如：是否连接上了？是否断开了？等，根据这些状态，用户应用程序可以执行不同操作，完成特定功能。该.c 文件我们重点介绍 3 个函数，首先是初始化 PCD MSP 函数，其定义如下：

```
/**
 * @brief       初始化 PCD MSP
 * @param       hpcd:PCD 结构体指针
 * @retval      无
 */
void HAL_HCD_MspInit(HCD_HandleTypeDef * hhcd)
{
    GPIO_InitTypeDef gpio_init_struct;

    RCC_PeriphCLKInitTypeDef usb_clk_init;

    /* USB 时钟设置，使用 HSI48MHz */
    usb_clk_init.PeriphClockSelection = RCC_PERIPHCLK_USB;
    usb_clk_init.UsbClockSelection = RCC_USBCLKSOURCE_HSI48;
    HAL_RCCEx_PeriphCLKConfig(&usb_clk_init);

    if(hhcd->Instance == USB2_OTG_FS)
```

```
    {
        __HAL_RCC_GPIOA_CLK_ENABLE();                       /* 使能 GPIOA 时钟 */

        gpio_init_struct.Pin = (GPIO_PIN_11 | GPIO_PIN_12);
        gpio_init_struct.Mode = GPIO_MODE_AF_PP;
        gpio_init_struct.Pull = GPIO_NOPULL;
        gpio_init_struct.Speed = GPIO_SPEED_FREQ_VERY_HIGH;
        gpio_init_struct.Alternate = GPIO_AF10_OTG1_FS;
        HAL_GPIO_Init(GPIOA, &gpio_init_struct);
        pcf8574_write_bit(USB_PWR_IO, 1);           /* 开启 USB HOST 电源供电 C */

        __HAL_RCC_USB2_OTG_FS_CLK_ENABLE();
        HAL_NVIC_SetPriority(OTG_FS_IRQn, 1, 0);
        HAL_NVIC_EnableIRQ(OTG_FS_IRQn);
    }
    else if (hhcd->Instance == USB1_OTG_HS)
    {
        /* USB1 OTG 本例程没用到,故不做处理 */
    }
}
```

HAL_HCD_MspInit 函数，用于使能 USB 时钟，设置 USB 使用 HSI 48M 时钟源，选择内核时钟，初始化 IO 口，设置中断等。该函数在 HAL_HCD_Init 函数里面被调用。

接下来介绍的是 USB OTG 中断服务函数，其定义如下：

```
/**
 * @brief       USB OTG 中断服务函数
 * @note        处理所有 USB 中断
 * @param       无
 * @retval      无
 */
void OTG_FS_IRQHandler(void)
{
    HAL_HCD_IRQHandler(&hhcd);
}
```

OTG_FS_IRQHandler 函数，是 USB 的中断服务函数，通过调用 HAL_HCD_IRQHandler 函数，实现对 USB 各种事务的处理。

最后介绍的是 USBH 底层初始化函数，其定义如下：

```
/**
 * @brief       USBH 底层初始化函数
 * @param       phost       : USBH 句柄指针
 * @retval      USB 状态
 *   @arg       USBH_OK(0)    , 正常;
 *   @arg       USBH_BUSY(1)  , 忙;
 *   @arg       其他          , 失败;
 */
USBH_StatusTypeDef USBH_LL_Init(USBH_HandleTypeDef *phost)
{
#ifdef USE_USB_FS
    /* 设置 LL 驱动相关参数 */
    hhcd.Instance = USB2_OTG_FS;                    /* 使用 USB2 OTG */
    hhcd.Init.Host_channels = 11;                   /* 主机通道数为 11 个 */
    hhcd.Init.dma_enable = 0;                       /* 不使用 DMA */
    hhcd.Init.low_power_enable = 0;                 /* 不使能低功耗模式 */
    hhcd.Init.phy_itface = HCD_PHY_EMBEDDED;        /* 使用内部 PHY */
    hhcd.Init.Sof_enable = 0;                       /* 禁止 SOF 中断 */
    hhcd.Init.speed = HCD_SPEED_FULL;               /* USB 全速(12Mbps) */
    hhcd.Init.vbus_sensing_enable = 0;              /* 不使能 VBUS 检测 */
    hhcd.Init.lpm_enable = 0;                       /* 使能连接电源管理 */

    hhcd.pData = phost;                             /* hhcd 的 pData 指向 phost */
    phost->pData = &hhcd;                           /* phost 的 pData 指向 hhcd */
```

```
    HAL_HCD_Init(&hhcd);                                    /* 初始化 LL 驱动 */
#endif

#ifdef USE_USB_HS
    /* 未实现 */
#endif
    USBH_LL_SetTimer(phost, HAL_HCD_GetCurrentFrame(&hhcd));
    return USBH_OK;
}
```

USBH_LL_Init 函数，用于初始化 USB 底层设置，因为我们定义的是：USE_USB_FS，因此会设置 USB OTG 使用 USB2_OTG_FS，然后完成各种设置，比如，使用内部 PHY，使用全速模式，不使能 VBUS 检测等，详见以上代码。该函数在 USBH_Init 函数里面被调用。

usbh_diskio.c 提供 U 盘和 FATFS 文件系统之间的输入/输出接口函数，这里总共有 5 个函数，首先介绍的是初始化 USBH 函数，其定义如下：

```
/**
 * @brief       初始化 USBH
 * @param       无
 * @retval      无
 */
DSTATUS USBH_initialize(void)
{
    return RES_OK;
}
```

USBH_initialize 函数，用于初始化 U 盘，我们不需要做任何事情，直接返回 OK 即可。

下面介绍的是获取 U 盘状态函数，其定义如下：

```
/**
 * @brief       获取 U 盘状态
 * @param       无
 * @retval      无
 */
DSTATUS USBH_status(void)
{
    DRESULT res=RES_ERROR;
    MSC_HandleTypeDef *MSC_Handle=hUSBHost.pActiveClass->pData;
    if(USBH_MSC_UnitIsReady(&hUSBHost,MSC_Handle->current_lun))
    {
        printf("U 盘状态查询成功\r\n");
        res=RES_OK;
    }else
    {
        printf("U 盘状态查询失败\r\n");
        res=RES_ERROR;
    }
    return res;
}
```

USBH_status 函数，用于获取 U 盘状态，本实验暂时用不到。

下面介绍的是 U 盘读扇区操作函数，其定义如下：

```
/**
 * @brief       U 盘读扇区操作
 * @param       buff   : 数据缓冲首地址
 * @param       sector : 扇区地址
 * @param       count  : 需要读取的扇区数
 * @retval      执行结果(详见 DRESULT 定义)
 */
DRESULT USBH_read(BYTE *buff, DWORD sector, UINT count)
{
    DRESULT res = RES_ERROR;
    MSC_LUNTypeDef info;
    MSC_HandleTypeDef *MSC_Handle=hUSBHost.pActiveClass->pData;

    if(USBH_MSC_Read(&hUSBHost,MSC_Handle->current_lun,sector,buff,count)
```

```
                    ==USBH_OK)
    {
        res = RES_OK;
    }else
    {
        printf("U 盘读取失败\r\n");
        USBH_MSC_GetLUNInfo(&hUSBHost,MSC_Handle->current_lun, &info);
        switch (info.sense.asc)
        {
            case SCSI_ASC_LOGICAL_UNIT_NOT_READY:
            case SCSI_ASC_MEDIUM_NOT_PRESENT:
            case SCSI_ASC_NOT_READY_TO_READY_CHANGE:
                USBH_ErrLog("USB Disk is not ready!");
                res = RES_NOTRDY;
                break;
            default:
                res = RES_ERROR;
                break;
        }
    }
    return res;
}
```

USBH_read 函数，用于从 U 盘指定位置，读取指定长度的数据。

下面介绍的是 U 盘写扇区操作函数，其定义如下：

```
/**
 * @brief       U 盘写扇区操作
 * @param       buff   : 数据缓冲首地址
 * @param       sector : 扇区地址
 * @param       count  : 需要写入的扇区数
 * @retval      执行结果(详见 DRESULT 定义)
 */
DRESULT USBH_write(const BYTE *buff,DWORD sector,UINT count)
{
    DRESULT res = RES_ERROR;
    MSC_LUNTypeDef info;
    MSC_HandleTypeDef *MSC_Handle=hUSBHost.pActiveClass->pData;
    if(USBH_MSC_Write(&hUSBHost,MSC_Handle->current_lun,sector,
                      (BYTE *)buff,count) == USBH_OK)
    {
        res=RES_OK;
    }else
    {
        printf("U 盘写入失败\r\n");
        USBH_MSC_GetLUNInfo(&hUSBHost,MSC_Handle->current_lun, &info);
        switch (info.sense.asc)
        {
            case SCSI_ASC_WRITE_PROTECTED:
                USBH_ErrLog("USB Disk is Write protected!");
                res = RES_WRPRT;
                break;
            case SCSI_ASC_LOGICAL_UNIT_NOT_READY:
            case SCSI_ASC_MEDIUM_NOT_PRESENT:
            case SCSI_ASC_NOT_READY_TO_READY_CHANGE:
                USBH_ErrLog("USB Disk is not ready!");
                res = RES_NOTRDY;
                break;
            default:
                res = RES_ERROR;
                break;
        }
    }
    return res;
}
```

USBH_write 函数，用于往 U 盘指定位置，写入指定长度的数据。

最后介绍的是 U 盘 IO 控制操作函数，其定义如下：

```
/**
 * @brief       U 盘 IO 控制操作
 * @param       cmd     : 控制命令
 * @param       buff    : 控制数据
 * @retval      执行结果(详见 DRESULT 定义)
 */
DRESULT USBH_ioctl(BYTE cmd,void *buff)
{
    DRESULT res = RES_ERROR;
    MSC_LUNTypeDef info;
    MSC_HandleTypeDef *MSC_Handle=hUSBHost.pActiveClass->pData;
    switch(cmd)
    {
        case CTRL_SYNC:
            res=RES_OK;
            break;
        case GET_SECTOR_COUNT : /* 获取扇区数量 */
            if(USBH_MSC_GetLUNInfo(&hUSBHost,MSC_Handle->current_lun,
                                   &info)==USBH_OK)
            {
                *(DWORD*)buff=info.capacity.block_nbr;
                res = RES_OK;
                printf("扇区数量:%d\r\n",info.capacity.block_nbr);
            }else
            {
                res = RES_ERROR;
            }
            break;
        case GET_SECTOR_SIZE :/* 获取扇区大小 */
            if(USBH_MSC_GetLUNInfo(&hUSBHost,MSC_Handle->current_lun,&info)
                                   == USBH_OK)
            {
                *(DWORD*)buff=info.capacity.block_size;
                res = RES_OK;
                printf("扇区大小:%d\r\n",info.capacity.block_size);
            }else
            {
                res = RES_ERROR;
            }
            break;
        case GET_BLOCK_SIZE :    /* 获取一个扇区里面擦除块的大小 */
            if(USBH_MSC_GetLUNInfo(&hUSBHost,MSC_Handle->current_lun,&info)
                                   ==USBH_OK)
            {
                *(DWORD*)buff=info.capacity.block_size/USB_DEFAULT_BLOCK_SIZE;
                printf("每个扇区擦除块:%d\r\n",
                     info.capacity.block_size/USB_DEFAULT_BLOCK_SIZE);
                res = RES_OK;
            }else
            {
                res = RES_ERROR;
            }
            break;
        default:
                res = RES_PARERR;
    }
    return res;
}
```

USBH_ioctl 函数，可以用于获取 U 盘扇区数量、扇区大小和块大小等信息。

我们将这 5 个函数在 diskio.c 里面和 FATFS 完成对接，同时需要设置 FATFS 支持 4 个磁盘（SD 卡、SPI FLASH、NAND FLASH 和 U 盘），需要在 ffconf.h 文件里面将 FF_VOLUMES 的宏定义值改成 4，以支持 FATFS 操作 U 盘。

2. main.c 代码

下面是 main.c 的程序，具体如下：

```
USBH_HandleTypeDef g_hUSBHost;                          /* USB Host 处理结构体 */

static void USBH_UserProcess(USBH_HandleTypeDef *phost, uint8_t id)
{
    uint32_t total, free;
    uint8_t res = 0;
    printf("id:%d\r\n", id);

    switch (id)
    {
        case HOST_USER_SELECT_CONFIGURATION:
            break;
        case HOST_USER_DISCONNECTION:
            f_mount(0, "3:", 1);                            /* 卸载 U 盘 */
            text_show_string(30, 140, 200, 16, "设备连接中...", 16, 0, RED);
            LED1(1);
            lcd_fill(30, 160, 239, 220, WHITE);
            break;
        case HOST_USER_CLASS_ACTIVE:
            text_show_string(30, 140, 200, 16, "设备连接成功!", 16, 0, RED);
            f_mount(fs[3], "3:", 1);                        /* 重新挂载 U 盘 */
            LED1(0);
            res = exfuns_get_free((uint8_t *)"3:", &total, &free);
            if (res == 0)
            {
               lcd_show_string(30, 160, 200, 16, 16, "FATFS OK!", BLUE);
               lcd_show_string(30,180,200,16,16,"U Disk Total Size: MB", BLUE);
               lcd_show_string(30,200,200,16,16,"U Disk Free Size: MB", BLUE);
               lcd_show_num(174, 180, total >> 10, 5, 16, BLUE);
               /* 显示 U 盘总容量 MB */
               lcd_show_num(174, 200, free >> 10, 5, 16, BLUE);
            }
            else
            {
                printf("U 盘存储空间获取失败\r\n");
            }
            break;
        case HOST_USER_CONNECTION:
            break;
        default:
            break;
    }
}

int main(void)
{
    uint8_t t = 0;

    sys_cache_enable();                                 /* 打开 L1-Cache */
    mpu_memory_protection();                            /* 保护相关存储区域 */
    HAL_Init();                                         /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);                 /* 设置时钟, 400Mhz */
    delay_init(400);                                    /* 延时初始化 */
    usart_init(115200);                                 /* 串口初始化 */
    usmart_init(200);                                   /* 初始化 USMART */
    led_init();                                         /* 初始化 LED */
    sdram_init();                                       /* 初始化 SDRAM */
    lcd_init();                                         /* 初始化 LCD */
    norflash_init();                                    /* 初始化 norflash */
    pcf8574_init();                                     /* 初始化 PCF8574 */
```

```
    my_mem_init(SRAMIN);                            /* 初始化内部内存池(AXI) */
    my_mem_init(SRAMEX);                            /* 初始化外部内存池(SDRAM) */
    my_mem_init(SRAMDTCM);                          /* 初始化 DTCM 内存池(DTCM) */

    exfuns_init();                                  /* 为 fatfs 相关变量申请内存 */
    f_mount(fs[0], "0:", 1);                        /* 挂载 SD 卡 */
    f_mount(fs[1], "1:", 1);                        /* 挂载 FLASH */
    f_mount(fs[2], "2:", 1);                        /* 挂载 NAND FLASH */

    while (fonts_init())                            /* 检查字库 */
    {
        lcd_show_string(30, 50, 200, 16, 16, "Font Error!", RED);
        delay_ms(200);
        lcd_fill(30, 50, 240, 66, WHITE);           /* 清除显示 */
        delay_ms(200);
    }

    text_show_string(30, 50, 200, 16, "STM32", 16, 0, RED);
    text_show_string(30, 70, 200, 16, "USB U盘 实验", 16, 0, RED);
    text_show_string(30, 90, 200, 16, "ATOM@ALIENTEK", 16, 0, RED);
    text_show_string(30, 120, 200, 16, "设备连接中...", 16, 0, RED);

    USBH_Init(&g_hUSBHost, USBH_UserProcess, 0);
    USBH_RegisterClass(&g_hUSBHost, USBH_MSC_CLASS);
    USBH_Start(&g_hUSBHost);
    HAL_PWREx_EnableUSBVoltageDetector();

    while(1)
    {
        USBH_Process(&g_hUSBHost);
        delay_ms(10);
        t++;
        if (t == 50)
        {
            t = 0;
            LED0_TOGGLE();
        }
    }
}
```

其中，USBH_HandleTypeDef 是一个用于 USB 主机类通信处理的结构体类型，它包含了 USB 主机通信的各种变量、结构体参数、传输状态和管道信息等。凡是 USB 主机类通信，都必须要用定义一个这样的结构体，这里定义成：hUSBHost。

同 USB Device 类通信类似，USB Host 的初始化过程如下：

1， 调用 USBH_Init 函数，初始化 USB 主机内核；

2， 调用 USBH_RegisterClass 函数，链接 MSC 主机类驱动程序到主机内核；

3， 调用 USBH_Start 函数，启动 USB 通信；

经过以上三步处理，USB 主机就启动了，所有 USB 事务，都是通过 USB 中断触发，并由 USB 驱动库自动处理。USB 中断服务函数在 usbh_conf.c 里面：

```
/**
 * @brief       USB OTG 中断服务函数
 * @note        处理所有 USB 中断
 * @param       无
 * @retval      无
 */
void OTG_FS_IRQHandler(void)
{
    HAL_HCD_IRQHandler(&hhcd);
}
```

该函数调用 HAL_HCD_IRQHandler 函数来处理各种 USB 中断请求。

整个 main 函数代码比较简单，不过我们在 main 函数里面，必须不停的调用：USBH_Process 函数，该函数用于实现 USB 主机通信的核心状态机处理，该函数必须在主函数里面被循环调用，而且调用频率得比较快才行（越快越好），以便及时处理各种事务。

USBH_UserProcess 函数是 USB 主机用户处理回调函数，参数 id 表示 USB 主机当前的一些状态，总共有 6 种状态：

```
#define HOST_USER_SELECT_CONFIGURATION   0x01U     /* USB 进入配置状态 */
#define HOST_USER_CLASS_ACTIVE           0x02U     /* USB 初始化配置完成 */
#define HOST_USER_CLASS_SELECTED         0x03U     /* USB 选择了一个类 */
#define HOST_USER_CONNECTION             0x04U     /* USB 连接成功 */
#define HOST_USER_DISCONNECTION          0x05U     /* USB 连接断开 */
#define HOST_USER_UNRECOVERED_ERROR      0x06U     /* USB 发生了不可恢复错误 */
```

本例程我们只用了：HOST_USER_DISCONNECTION 和 HOST_USER_CLASS_ACTIVE 两个状态。程序运行后，如果没有 U 盘插入，则不会执行到 USBH_UserProcess 函数，当插入 U 盘，并成功识别之后，USBH_UserProcess 函数会进入：HOST_USER_CLASS_ACTIVE 状态，我们就可以挂载U盘，并显示容量等信息，表示U盘识别完成。此时，如果我们把U盘拔出，则会进入：HOST_USER_DISCONNECTION 状态，表示 U 盘拔出，我们取消 U 盘的挂载，并显示设备连接中…（表示当前正在连接设备）。

最后，我们需要将 FATFS 相关测试函数(mf_open/ mf_close 等函数)，加入 USMART 管理，这里和 FATFS 实验一模一样，可以参考该实验的方法操作。

72.4 下载验证

将程序下载到开发板后，然后在 USB_HOST 端子插入 U 盘/读卡器（带卡），**注意：此时 USB SLAVE 口不要插 USB 线到电脑，否则会干扰！**

等 U 盘成功识别后，便可以看到 LCD 显示 U 盘容量等信息，如图 72.4.1 所示：

图 72.4.1 U 盘识别成功

此时，我们便可以通过 USMART 来测试 U 盘读写了，如图 72.4.2 和图 72.4.3 所示：

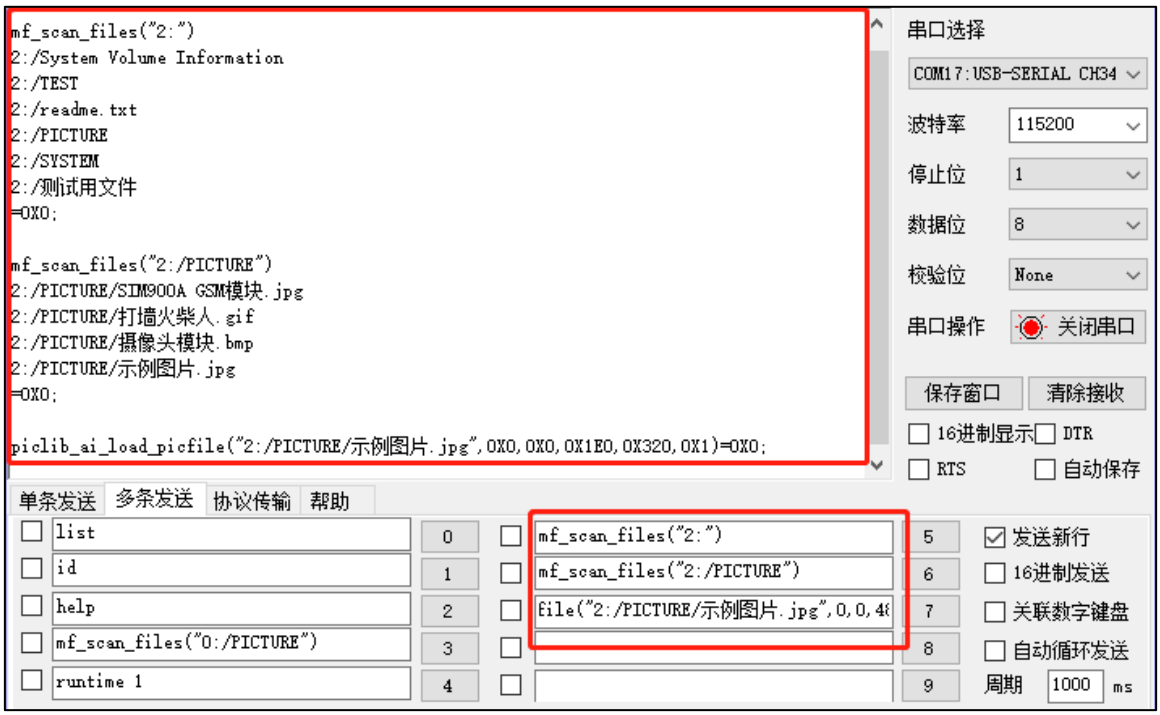


图 72.4.2 测试读取 U 盘读取

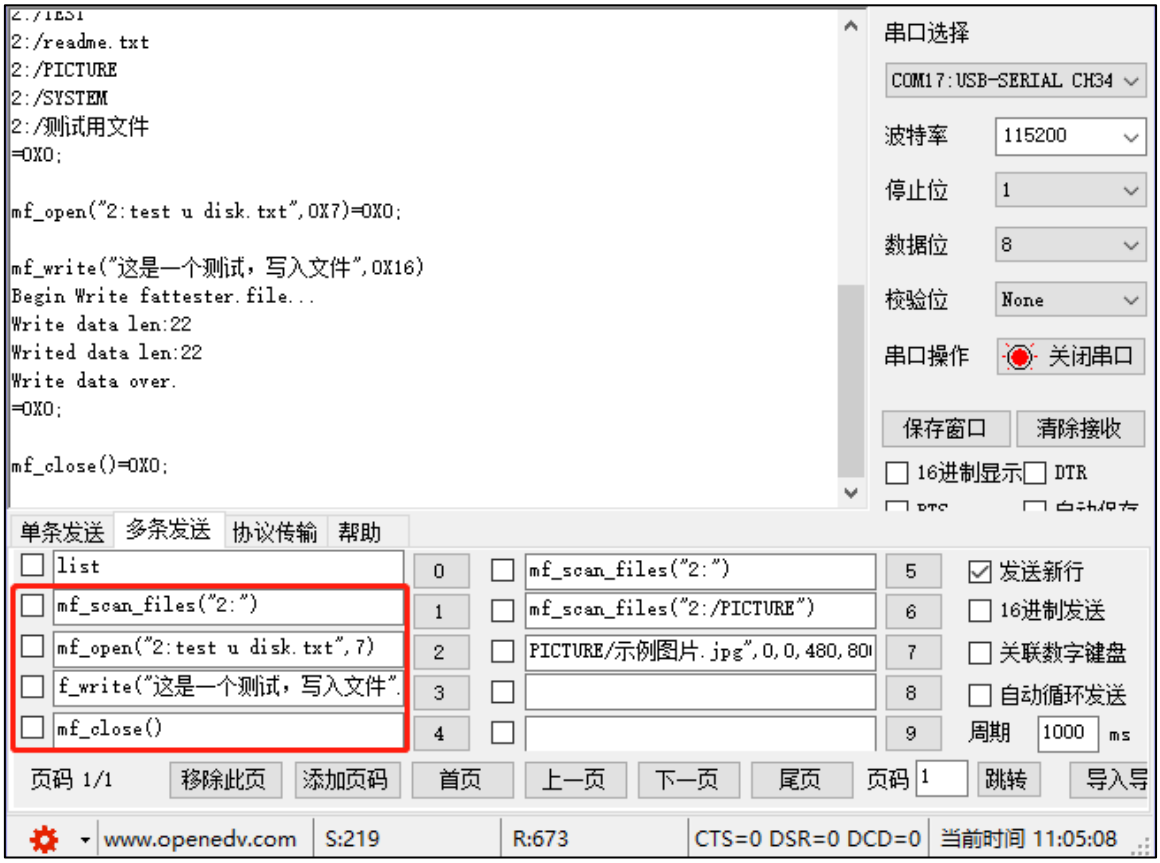


图 72.4.3 测试 U 盘写入

图 72.4.2 分别是通过发送：mf_scan_files("2:")和 mf_scan_files("2:/PICTURE")，扫描 U 盘根目录所有文件和 PICTURE 目录下的所有文件，然后通过 piclib_ai_load_picfile("2:/PICTURE/示例图片.jpg",0,0,480,800,1)，解码图片，并显示在 LCD 上面。说明读 U 盘是没问题的。

图 72.4.3 通过发送：mf_open("2:test u disk.txt",7)，在 U 盘根目录创建 test u disk.txt 这个文

件，然后发送：mf_write("这是一个测试，写入文件",22)，写入“这是一个测试，写入文件”到这个文件里面，然后发送：mf_close()，关闭文件，完成一次文件创建。最后，发送：mf_scan_files("2:")，扫描 U 盘根目录文件，发现比图 72.4.3 所示多出了一个 test u disk.txt 的文件，说明 U 盘写入成功。

这样，就完成了本实验的设计目的：实现U盘的读写操作。最后，大家还可以调用其他函数，实现相关功能测试，这里就不给大家一一演示了。

第七十三章 USB 鼠标键盘(Host)实验

本章我们介绍如何使用 STM32H743 的 USB HOST 来驱动 USB 鼠标/键盘。本章分为如下几个小节：

73.1 USB 鼠标键盘简介
73.2 硬件设计
73.3 程序设计
73.4 下载验证

73.1 USB 鼠标键盘简介

传统的鼠标和键盘是采用 PS/2 接口和电脑通信的，但是现在 PS/2 接口在电脑上逐渐消失，所以现在越来越多的鼠标键盘采用的是 USB 接口，而不是 PS/2 接口的了。

USB 鼠标键盘属于 USB HID 设备。USB HID 即：Human Interface Device（人机交互设备）的缩写，键盘、鼠标与游戏杆等都属于此类设备。不过 HID 设备并不一定要有人机接口，只要符合 HID 类别规范的设备都是 HID 设备。关于 USB HID 的知识，我们这里就不详细介绍了，请大家自行百度学习。

本章，我们同上一章一样，我们直接移植官方的 USB HID 例程，官方例程路径：光盘→8，STM32 参考资料→ 1，STM32CubeH7 固件包→ STM32Cube_FW_H7_V1.9.0→Projects→ STM32H743I-EVAL→Applications→USB_Host→HID_Standalone，该例程支持 USB 鼠标和键盘等 USB HID 设备，本章我们将移植这个例程到我们的开发板上。

73.2 硬件设计

1. 例程功能

本实验代码，开机的时候先显示一些提示信息，然后初始化 USB HOST，并不断轮询。当检测到 USB 鼠标/键盘的插入后，显示设备类型，并显示设备输入数据：

如果是 USB 鼠标：将显示鼠标移动的坐标（X，Y 坐标），滚轮滚动数值（Z 坐标）以及按键（左中右）。

如果是 USB 键盘：将显示键盘输入的数字/字母等内容（不是所有按键都支持，部分按键没有做解码支持，比如 F1~F12）。

2. 硬件资源

1）LED 灯
LED0：LED0 - PB1
LED1：LED1 - PB0
2）串口 1(PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块(包括 MCU 屏和 RGB 屏，都支持)
4）USB_HOST 接口(D-/D+连接在 PA11/PA12 上)

73.3 程序设计

73.3.1 程序流程图

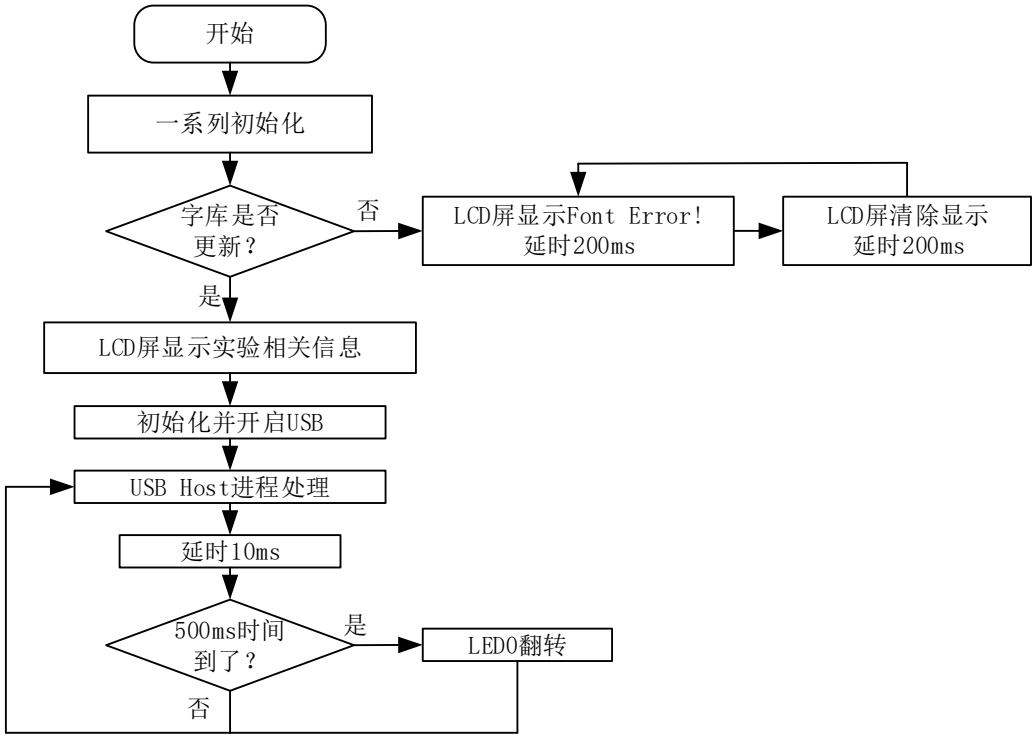


图 73.3.1.1 USB 鼠标键盘实验(Host)程序流程图

73.3.2 程序解析

这里我们只讲解核心代码，详细的源码请大家参考光盘本实验对应源码。

本实验，我们在上一个实验的基础上修改，对照官方 HID 例子，然后，我们在工程里面添加 USB HID 相关代码，如图 73.3.2.1 所示：

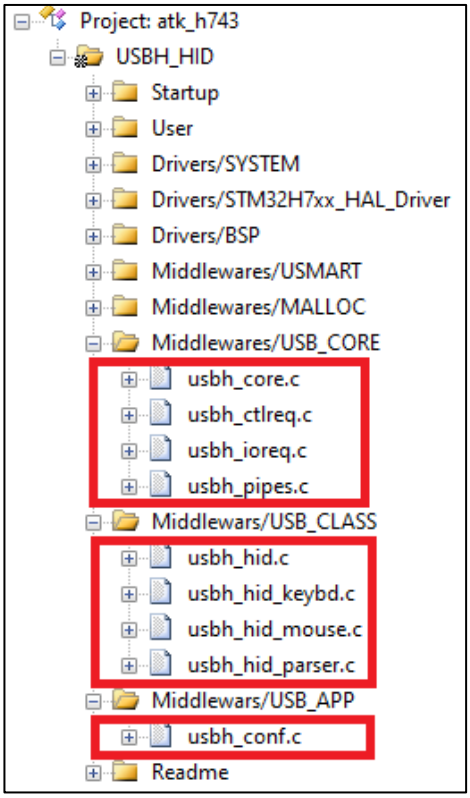


图 73.3.2.1 USB 鼠标键盘工程分组

上图工程分组中的 Middlewares/USB_CORE 和 Middlewares/USB_CLASS 分组下的.c 文件，直接拷贝 ST 官方 USB HOST 库。我们重点介绍下 USB_APP 里面的 usbh_hid_mouse.c 和 usbh_processs 文件，usbh_process 文件是我们自己写的 demo 文件，usbh_hid_mouse.c 文件拷贝自 ST 官方例程，但是 ST 官方例程对鼠标的支持不好（兼容性差，不支持滚轮等），我们重写了该文件，下面介绍这两个文件和 main.c。

1. USB 驱动代码

usbh_hid_mouse.h，首先介绍 usbh_hid_mouse.h 里面定义的一个结构体，具体如下：

```
/* 鼠标信息结构体*/
typedef struct _HID_MOUSE_Info
{
  uint8_t               x;          /* x 轴增量（强制转换成 signed char 后使用）*/
  uint8_t               y;          /* y 轴增量（强制转换成 signed char 后使用）*/
  uint8_t               z;          /* z 轴增量（强制转换成 signed char 后使用）*/
  uint8_t               button;     /* 将 buttons 修改为 button,存储按键状态 */
}
HID_MOUSE_Info_TypeDef;
```

HID_MOUSE_Info_TypeDef 结构体中，成员 x 和 y 表示鼠标的横向/纵向移动的增量值，成员 z 表示鼠标的滚轮的滚动增量值，在使用的时候，需要强制转换成 unsigned char 类型后使用（有±）。button 的 bit0、bit1、bit2 分别表示鼠标的：左键、右键和滚轮键的按下状态。我们通过这个结构体，就可以获得当前鼠标的输入状态。

usbh_hid_mouse.c，总共有三个函数，首先介绍的是 USBH 鼠标初始化函数，其定义如下：

```
/* 鼠标信息(坐标+按钮状态)*/
HID_MOUSE_Info_TypeDef    mouse_info;
/* 鼠标上报数据长度,最多 HID_QUEUE_SIZE 个字节*/
uint8_t mouse_report_data[HID_QUEUE_SIZE];

/**
 * @brief       USBH 鼠标初始化
 * @param       phost       : USBH 句柄指针
 * @retval      USB 状态
 *   @arg       USBH_OK(0)    , 正常;
 *   @arg       USBH_BUSY(1)  , 忙;
 *   @arg       其他          , 失败;
 */
USBH_StatusTypeDef USBH_HID_MouseInit(USBH_HandleTypeDef *phost)
{
    HID_HandleTypeDef *HID_Handle =
                      (HID_HandleTypeDef *) phost->pActiveClass->pData;
    mouse_info.x = 0;
    mouse_info.y = 0;
    mouse_info.button = 0;
    if (HID_Handle->length > sizeof(mouse_report_data))
    {
        HID_Handle->length = sizeof(mouse_report_data);
    }
    HID_Handle->pData = (uint8_t *)mouse_report_data;
    USBH_HID_FifoInit(&HID_Handle->fifo, phost->device.Data, HID_QUEUE_SIZE);
    return USBH_OK;
}
```

USBH_HID_MouseInit 函数，用于初始化鼠标，指定 USB 鼠标输出数据的缓存，初始化 FIFO 等，该函数在 USBH_HID_Process 函数里面被调用。

接下来介绍的是 USBH 获取鼠标信息函数，其定义如下：

```
/**
 * @brief       USBH 获取鼠标信息
 * @param       phost       : USBH 句柄指针
 * @retval      鼠标信息(HID_MOUSE_Info_TypeDef)
 */
```

```
HID_MOUSE_Info_TypeDef *USBH_HID_GetMouseInfo(USBH_HandleTypeDef *phost)
{
    if (USBH_HID_MouseDecode(phost) == USBH_OK)
    {
        return &mouse_info;
    }
    else
    {
        return NULL;
    }
}
```

USBH_HID_GetMouseInfo 函数，用于获取鼠标信息，通过 USBH_HID_MouseDecode 函数获取当前鼠标传回来的数据。该函数在 USB_Demo 函数里面被调用。

最后介绍的是 USBH 鼠标数据解析函数，其定义如下：

```
/**
 * @brief       USBH 鼠标数据解析函数
 * @param       phost       : USBH 句柄指针
 * @retval      USB 状态
 *   @arg       USBH_OK(0)    , 正常;
 *   @arg       USBH_BUSY(1)  , 忙;
 *   @arg       其他           , 失败;
 */
USBH_StatusTypeDef USBH_HID_MouseDecode(USBH_HandleTypeDef *phost)
{
   HID_HandleTypeDef *HID_Handle =
                     (HID_HandleTypeDef *) phost->pActiveClass->pData;

    if (HID_Handle->length == 0)return USBH_FAIL;

   if (USBH_HID_FifoRead(&HID_Handle->fifo, &mouse_report_data,
                      HID_Handle->length) == HID_Handle->length) /* 读取 FIFO*/
    {
        mouse_info.button = mouse_report_data[0];
        mouse_info.x      = mouse_report_data[1];
        mouse_info.y      = mouse_report_data[2];
        mouse_info.z      = mouse_report_data[3];
        return USBH_OK;
    }
    return   USBH_FAIL;
}
```

USBH_HID_MouseDecode 函数，用于解析 USB HOST 获取到的 USB 鼠标数据，并存放到 mouse_info 结构体里面。该函数被 USBH_HID_GetMouseInfo 函数调用。

另外，为了提高对鼠标键盘的识别率和兼容性，对 usbh_hid_core.c 里面的两处代码进行了修改：

1，在 usbh_hid.c 里面，USBH_HID_Process 函数，修改 case HID_IDLE 部分代码（366 行）为：

```
case HID_IDLE:
    status = USBH_HID_GetReport(phost, 0X01U, 0U, HID_Handle->pData, (uint8_t)
                                HID_Handle->length);
    if(status == USBH_OK)
    {
      USBH_HID_Fifo_Write(&HID_Handle->fifo,HID_Handle->pData,
      HID_Handle->length);
      HID_Handle->state = HID_SYNC;
    }else if (status == USBH_BUSY)
    {
      HID_Handle->state = HID_IDLE;
      status = USBH_OK;
    }else if (status == USBH_NOT_SUPPORTED)
    {
      HID_Handle->state = HID_SYNC;
      status = USBH_OK;
```

```
    }else
    {
      HID_Handle->state = HID_ERROR;
      status = USBH_FAIL;
    }
#if (USBH_USE_OS == 1)
    osMessagePut ( phost->os_event, USBH_URB_EVENT, 0);
#endif
    break;
```

2，在 usbh_ctlreq.c 里面，USBH_CtlReq 函数，修改 case CMD_WAIT 代码（552 行）为：

```
case CMD_WAIT:
    status = USBH_HandleControl(phost);
    if (status == USBH_OK)
    {
      /* Commands successfully sent and Response Received  */
      phost->RequestState = CMD_SEND;
      phost->Control.state =CTRL_IDLE;
      status = USBH_OK;
    }else if  (status == USBH_FAIL)
    {
      /* Failure Mode */
      phost->RequestState = CMD_SEND;
      status = USBH_FAIL;
    }else if(status == USBH_NOT_SUPPORTED)
    {
      phost->RequestState = CMD_SEND;
    }
    break;
```

经过以上两处修改，可以显著提高 USB 鼠标的兼容性，基本上各种鼠标都可以正常识别和使用了。

2. main.c 代码

下面是 main.c 的程序，具体如下：

```
USBH_HandleTypeDef hUSBHost;    /* USB Host 处理结构体 */

/* 应用状态结构体类型定义 */
typedef enum
{
    APPLICATION_IDLE = 0,           /* 挂起状态 */
    APPLICATION_DISCONNECT,         /* 断开连接 */
    APPLICATION_START,              /* 应用开始（连接上了） */
    APPLICATION_READY,              /* 准备完成（可以执行相关应用代码了） */
    APPLICATION_RUNNING,            /* 运行中 */
} HID_ApplicationTypeDef;

extern HID_MOUSE_Info_TypeDef mouse_info;
HID_ApplicationTypeDef App_State = APPLICATION_IDLE;

uint8_t g_usb_first_plugin_flag = 0; /* USB 第一次插入标志,如果为 1,说明是第一次插入
*/

/**
 * @brief       USB 信息显示
 * @param       msg    : 信息类型
 *   @arg               0, USB 无连接
 *   @arg               1, USB 键盘
 *   @arg               2, USB 鼠标
 *   @arg               3, 不支持的 USB 设备
 * @retval      无
 */
void usbh_msg_show(uint8_t msgx)
{
```

```
    switch (msgx)
    {
        case 0: /* USB 无连接 */
            lcd_show_string(30, 130, 200, 16, 16, "USB Connecting...", RED);
            lcd_fill(0, 150, lcddev.width, lcddev.height, WHITE);
            break;

        case 1: /* USB 键盘 */
            lcd_show_string(30, 130, 200, 16, 16, "USB Connected    ", RED);
            lcd_show_string(30, 150, 200, 16, 16, "USB KeyBoard", RED);
            lcd_show_string(30, 180, 210, 16, 16, "KEYVAL:", RED);
            lcd_show_string(30, 200, 210, 16, 16, "INPUT STRING:", RED);
            break;

        case 2: /* USB 鼠标 */
            lcd_show_string(30, 130, 200, 16, 16, "USB Connected    ", RED);
            lcd_show_string(30, 150, 200, 16, 16, "USB Mouse", RED);
            lcd_show_string(30, 180, 210, 16, 16, "BUTTON:", RED);
            lcd_show_string(30, 200, 210, 16, 16, "X POS:", RED);
            lcd_show_string(30, 220, 210, 16, 16, "Y POS:", RED);
            lcd_show_string(30, 240, 210, 16, 16, "Z POS:", RED);
            break;

        case 3: /* 不支持的 USB 设备 */
            lcd_show_string(30, 130, 200, 16, 16, "USB Connected    ", RED);
            lcd_show_string(30, 150, 200, 16, 16, "Unknow Device", RED);
            break;
    }
}

/**
 * @brief       完成 USBH 不同用户状态下的数据处理
 * @param       phost : USBH 句柄指针
 * @param       id    : USBH 当前的用户状态
 * @retval      无
 */
void USBH_UserProcess(USBH_HandleTypeDef *phost, uint8_t id)
{
    uint8_t dev_type = 0XFF;     /* 设备类型,1,键盘;2,鼠标;其他,不支持的设备 */
    switch (id)
    {
        case HOST_USER_SELECT_CONFIGURATION:
            break;

        case HOST_USER_DISCONNECTION:
            usbh_msg_show(0);    /* 显示已经断开连接,准备重新连接 */
            App_State = APPLICATION_DISCONNECT;
            break;

        case HOST_USER_CLASS_ACTIVE:
            App_State = APPLICATION_READY;
            dev_type = phost->device.CfgDesc.Itf_Desc[phost->
                       device.current_interface].bInterfaceProtocol;
            if (dev_type == HID_KEYBRD_BOOT_CODE)          /* 键盘设备 */
            {
                g_usb_first_plugin_flag = 1;               /* 标记第一次插入 */
                usbh_msg_show(1);                          /* 显示键盘界面 */
            }
            else if (dev_type == HID_MOUSE_BOOT_CODE)      /* 鼠标设备 */
            {
                g_usb_first_plugin_flag = 1;               /* 标记第一次插入 */
                usbh_msg_show(2);                          /* 显示鼠标界面 */
            }
            else
```

```
            {
                usbh_msg_show(3);    /* 显示不支持 */
            }
            break;

        case HOST_USER_CONNECTION:
            App_State = APPLICATION_START;
            break;

        default:
            break;
    }
}

/* 临时数组,用于存放鼠标坐标/键盘输入内容(4.3 屏,最大可以输入 2016 字节) */
#if !(__ARMCC_VERSION >= 6010050)    /* 不是 AC6 编译器，即使用 AC5 编译器时 */
__align(4) uint8_t g_temp_buffer[2017];
#else       /* 使用 AC6 编译器时 */
__ALIGNED(4) uint8_t g_temp_buffer[2017];
#endif

/**
 * @brief       USB 鼠标数据处理
 * @param       data  : USB 鼠标数据结构体指针
 * @retval      无
 */
void mouse_data_process(HID_MOUSE_Info_TypeDef *data)
{
    static signed short x, y, z;
    if (g_usb_first_plugin_flag)     /* 第一次插入,将数据清零 */
    {
        g_usb_first_plugin_flag = 0;
        x = y = z = 0;
    }
    x += (signed char)data->x;
    if (x > 9999)x = 9999;
    if (x < -9999)x = -9999;
    y += (signed char)data->y;
    if (y > 9999)y = 9999;
    if (y < -9999)y = -9999;
    z += (signed char)data->z;
    if (z > 9999)z = 9999;
    if (z < -9999)z = -9999;
    sprintf((char *)g_temp_buffer, "BUTTON:");
    if (data->button & 0X01)strcat((char *)g_temp_buffer, "LEFT ");
    if ((data->button & 0X03) == 0X02)
    {
        strcat((char *)g_temp_buffer, "RIGHT");
    }
    else
    {
        if ((data->button & 0X03) == 0X03)strcat((char
*)g_temp_buffer,"+RIGHT");
    }
    if ((data->button & 0X07) == 0X04)
    {
        strcat((char *)g_temp_buffer, "MID  ");
    }
    else
    {
        if ((data->button & 0X07) > 0X04)strcat((char *)g_temp_buffer, "+MID");
    }

    lcd_fill(30 + 56, 180, lcddev.width - 1, 180 + 16, WHITE);
    lcd_show_string(30, 180, 210, 16, 16, (char *)g_temp_buffer, BLUE);
```

```
    sprintf((char *)g_temp_buffer, "X POS:%05d", x);
    lcd_show_string(30, 200, 200, 16, 16, (char *)g_temp_buffer, BLUE);
    sprintf((char *)g_temp_buffer, "Y POS:%05d", y);
    lcd_show_string(30, 220, 200, 16, 16, (char *)g_temp_buffer, BLUE);
    sprintf((char *)g_temp_buffer, "Z POS:%05d", z);
    lcd_show_string(30, 240, 200, 16, 16, (char *)g_temp_buffer, BLUE);
   //printf("btn,X,Y,Z:0x%x,%d,%d,%d\r\n",data->button,(signed char)
            data->x,(signed char)data->y,(signed char)data->z);
}
/**
 * @brief       USB 键盘数据处理
 * @param       data  : USB 键盘键值
 * @retval      无
 */
void  keybrd_data_process(uint8_t data)
{
    static uint16_t pos;
    static uint16_t endx, endy;
    static uint16_t maxinputchar;
    uint8_t buf[4];

    if (g_usb_first_plugin_flag)    /* 第一次插入,将数据清零 */
    {
        g_usb_first_plugin_flag = 0;
        endx = ((lcddev.width - 30) / 8) * 8 + 30;      /* 得到 endx 值 */
        endy = ((lcddev.height - 220) / 16) * 16 + 220;/* 得到 endy 值 */
        maxinputchar = ((lcddev.width - 30) / 8);
        maxinputchar *= (lcddev.height - 220)/16;/* 当前 LCD 最大可以显示的字符数 */
        pos = 0;
    }
    sprintf((char *)buf, "%02X", data);
    lcd_show_string(30 + 56, 180, 200, 16, 16, (char *)buf, BLUE);/* 显示键值 */

    if (data >= ' ' && data <= '~')
    {
        g_temp_buffer[pos++] = data;
        g_temp_buffer[pos] = 0; /* 添加结束符 */
        if (pos > maxinputchar)pos = maxinputchar;  /* 最大输入这么多 */
    }
    else if (data == 0X0D)      /* 退格键 */
    {
        if (pos)pos--;
        g_temp_buffer[pos] = 0; /* 添加结束符 */
    }

    if (pos <= maxinputchar)    /* 没有超过显示区 */
    {
        lcd_fill(30, 220, endx, endy, WHITE);
        lcd_show_string(30, 220, endx - 30, endy - 220, 16,
                        (char *)g_temp_buffer, BLUE);
    }
    //printf("KEY Board Value:%02X\r\n",data);
    //printf("KEY Board Char:%c\r\n",data);
}

/**
 * @brief       USB 键盘/鼠标演示 demo 数据处理
 * @param       phost : USBH 句柄指针
 * @retval      无
 */
void usb_demo(USBH_HandleTypeDef *phost)
{
    char c;
    HID_KEYBD_Info_TypeDef *k_pinfo;
```

```
    HID_MOUSE_Info_TypeDef *m_pinfo;
    if (App_State == APPLICATION_READY)
    {
        if (USBH_HID_GetDeviceType(phost) == HID_KEYBOARD)  /* 键盘设备 */
        {
            k_pinfo = USBH_HID_GetKeybdInfo(phost);         /* 获取键盘信息 */
            if (k_pinfo != NULL)
            {
                c = USBH_HID_GetASCIICode(k_pinfo);  /* 转换成 ASCII 码 */
                keybrd_data_process(c);              /* 在 LCD 上显示出键盘字符 */
            }
        }
        else if (USBH_HID_GetDeviceType(phost) == HID_MOUSE)/* 鼠标设备 */
        {
            m_pinfo = USBH_HID_GetMouseInfo(phost);    /* 获取鼠标信息 */
            if (m_pinfo != NULL)
            {
                mouse_data_process(&mouse_info);       /* LCD 上显示鼠标信息 */
            }
        }
    }
}

int main(void)
{
    sys_cache_enable();                     /* 打开 L1-Cache */
    mpu_memory_protection();                /* 保护相关存储区域 */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟, 400Mhz */
    delay_init(400);                        /* 延时初始化 */
    usart_init(115200);                     /* 串口初始化为 115200 */
    led_init();                             /* 初始化 LED */
    sdram_init();                           /* 初始化 SDRAM */
    lcd_init();                             /* 初始化 LCD */
    pcf8574_init();                         /* 初始化 PCF8574 */
    my_mem_init(SRAMIN);                    /* 初始化内部内存池(AXI) */
    my_mem_init(SRAM12);                    /* 初始化 SRAM12 内存池(SRAM1+SRAM2) */
    my_mem_init(SRAM4);                     /* 初始化 SRAM4 内存池(SRAM4) */
    my_mem_init(SRAMDTCM);                  /* 初始化 DTCM 内存池(DTCM) */
    my_mem_init(SRAMITCM);                  /* 初始化 ITCM 内存池(ITCM) */

    lcd_show_string(30, 50, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 70, 200, 16, 16, "USB MOUSE/KEYBOARD TEST", RED);
    lcd_show_string(30, 90, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 130, 200, 16, 16, "USB Connecting...", RED);

    USBH_Init(&hUSBHost, USBH_UserProcess, 0);
    USBH_RegisterClass(&hUSBHost, USBH_HID_CLASS);
    USBH_Start(&hUSBHost);
    HAL_PWREx_EnableUSBVoltageDetector();
    while (1)
    {
        USBH_Process(&hUSBHost);
        usb_demo(&hUSBHost);
    }
}
```

这里总共六个函数：

usbh_msg_show 函数，用于显示一些提示信息，在 USBH_UserProcess 函数里面被调用。

USBH_UserProcess 函数，用于执行 USB 主机不同用户状态下的数据处理，在 usbh_core.c 等相关文件里面被调用，该函数仅针对两个状态进行处理：

1、HOST_USER_DISCONNECTION，表示连接断开了，我们在屏幕显示连接断开，并设

置 App_State 状态为 APPLICATION_DISCONNECT；

2、HOST_USER_CLASS_ACTIVE，表示 USB 主机类激活，即接入 USB 设备已经准备就绪，可以正常工作了，我们设置 App_State 状态为：APPLICATION_READY，并根据接入设备的不同（鼠标/键盘）显示不同的信息。

mouse_data_process 函数，用于处理 USB 鼠标数据，在液晶屏上面显示鼠标的移动坐标值和按键状态等信息，该函数在 USB_Demo 函数里面被调用。

keybrd_data_process 函数，用于处理键盘数据，在液晶屏上面显示键盘的输入内容，最多支持 2017 个字符的输入显示。该函数在 USB_Demo 函数里面被调用。

usb_demo 函数，根据当前用户状态和 USB 接入设备的类型，执行不同的数据处理。该函数在 main 函数里面被调用。

main 函数，初始化各个外设后，通过 USBH_Init 等 3 个函数初始化并启动 USB 主机通信，然后进入死循环，不停的调用 USBH_Process 和 usb_demo，执行各种 USB 处理。

73.4 下载验证

将程序下载到开发板后，然后在 USB_HOST 端子插入 USB 鼠标/键盘，**注意：此时 USB SLAVE 口不要插 USB 线到电脑，否则会干扰！**

等 USB 鼠标/键盘成功识别后，便可以看到 LCD 显示 USB Connected，并显示设备类型：USB Mouse 或者 USB KeyBoard，同时也会显示输入的数据，如图 73.4.1 和图 73.4.2 所示：

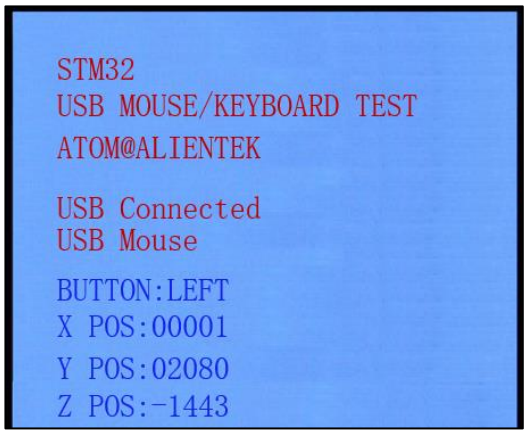


图 73.4.1 USB 鼠标测试

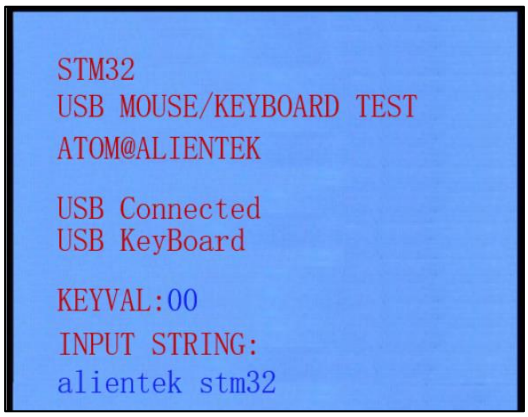


图 73.4.2 USB 键盘测试

其中，图 73.4.1 是 USB 鼠标测试界面，图 73.4.2 是 USB 键盘测试界面。

最后，特别提醒大家，由于例程的 HID 内核，只处理了第一个接口描述符，所以对于 USB 符合设备，只能识别第一个描述符所代表的设备。体现到实际使用中，就是：USB 无线鼠标，一般是无法使用（被识别为键盘），而 USB 无线键盘，可以使用，因为键盘在第一个描述符，鼠标在第二个描述符 。

第七十四章 网络通讯实验

阿波罗开发板板载的以太网芯片相关知识及配套例程，请参考《LWIP 开发指南》。

第七十五章 UCOSII 实验 1-任务调度

前面我们所有的例程都是跑裸机程序，简称裸跑，从本章开始，我们将分 3 个章节向大家介绍 UCOSII（实时多任务操作系统内核）的使用。本章，我们将向大家介绍 UCOSII 最基本也是最重要的应用：任务调度。

本章分为如下几个小节：

75.1 嵌入式实时操作系统介绍
75.2 硬件设计
75.3 程序设计
75.4 下载验证

75.1 嵌入式实时操作系统介绍

75.1.1 裸机系统和多任务系统的区别

在嵌入式设备的开发过程中，我们使用的是两种程序，一是裸机程序，前面所有的实验章节使用的都是使用裸机程序；二是多任务程序，即接下来的三章都是多任务程序。很多人会有疑问用得好好的裸机程序，为什么要用多任务程序呢？

裸机程序最大的特点，就是主函数中会有一个大循环，大循环中就会有很多个小任务的实现。任务间是按照顺序进行执行的，换句话来说它们执行等级是一样的，下一个任务想要执行必须等上一个任务执行完成才能进行。这个运行着的大循环我们称之为后台程序。中断是可以打断系统当前的后台任务优先被执行，等待执行完后，再回到原来后台被打断处继续执行后台程序，中断处理程序称之前台程序。这种使用前后台裸机程序的叫做前后台系统，如下图所示：

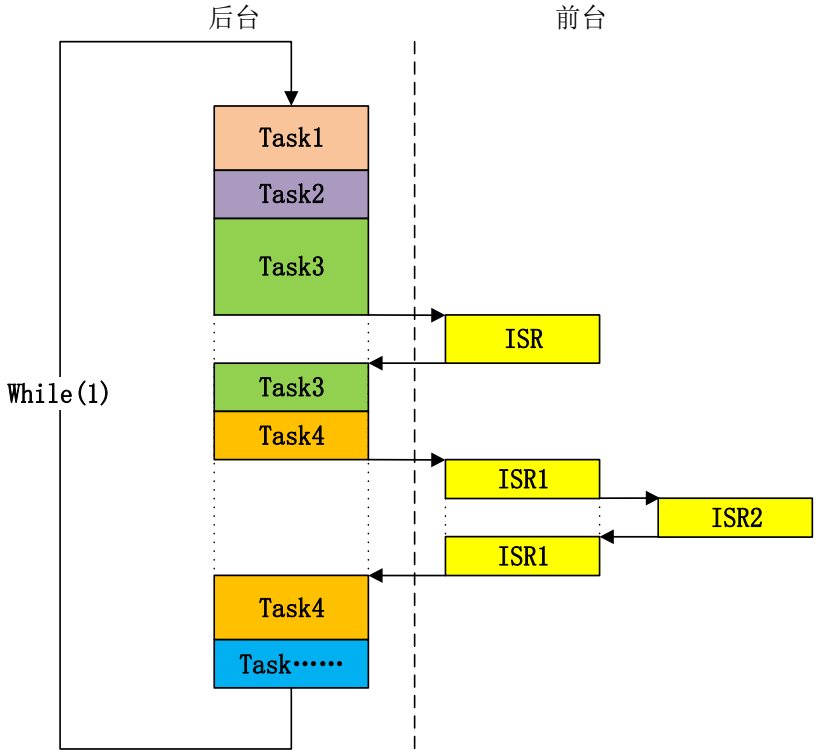


图 75.1.1.1 前后台系统

这样的前后台系统在实时性处理方面存在缺陷，例如 Task1 是重要任务，需要能够得到及时的响应，但是在执行 Task3 的时候，产生中断。现在的情况是执行 Task1 条件满足，理想的处理方式就是 Task1 需要立刻被执行，但是前后台程序中做不到，因为任务是被顺序执行的，

即使 Task1 十万火急，也必须要等待 Task3 处理完毕才能被执行。

前面的情况对于要求实时性比较强的产品来说，是不允许的。所以出现了多任务程序，这种使用多任务程序的系统，叫做嵌入式实时操作系统。它把任务分为不同的优先级，当运行条件被满足时，高优先级任务可以打断低优先级任务优先运行，从而极大地提高了系统的实时性。嵌入式实时操作系统执行任务示意图如图 75.1.1.2 所示：

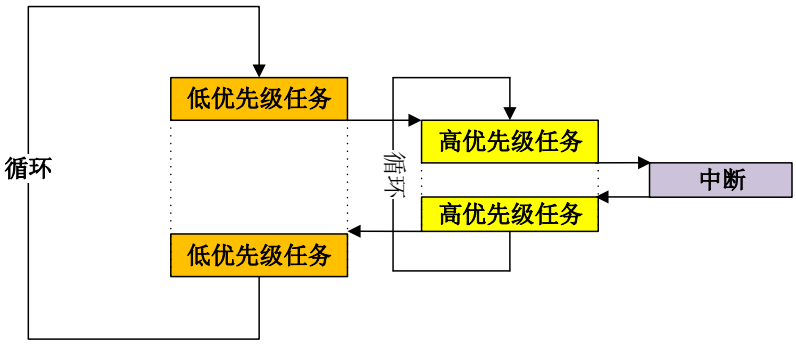


图 75.1.1.2 嵌入式实时操作系统执行任务示意图

嵌入式实时操作系统相比前后台系统明显体现在实时性方面，同时它在多任务管理、任务间通信、内存管理、定时器管理、设备管理等方面也提供了一套完整的机制，极大程度上便利了嵌入式应用程序的开发、管理和维护。

75.1.2 UCOSII 介绍

现在市面上有许多实时操作系统，国外的实时操作系统就有 FreeRTOS，UCOS 和 RTX，国内的实时操作系统就有 RT_Thread、LiteOS 等。其中 FreeRTOS 使用率世界最高，UCOS 发展历史最悠久。在这里我们主要是对 UCOSII 进行学习。

UCOSII，全称是 Micro Control Operation System Two，是由 Micrium 公司提供，是一个可移植、可固化、可裁剪的、占先式多任务实时内核，它适用于多种微处理器，微控制器和数字处理芯片。早在 1992 年就由美国嵌入式专家 Jean J.Labrosse 在《嵌入式系统编程》杂志中提出，并公布源码。UCOSII 只是一个实时操作系统内核，它仅仅包含了任务调度，任务管理，时间管理，内存管理和任务间的通信和同步等基本功能。没有提供输入输出管理，文件系统，网络等额外的服务。该实时系统十分适合初次接触嵌入式实时操作系统的朋友。

本章实验中我们使用的是 UCOSII V2.93.01 版本，它的体系结构如图 75.1.2.1 所示。该版本比早期的 UCOSII(如 V2.52)多了很多功能，比如软件定时器，支持任务数最大达到 255 个等，而且修正了很多已知 BUG。

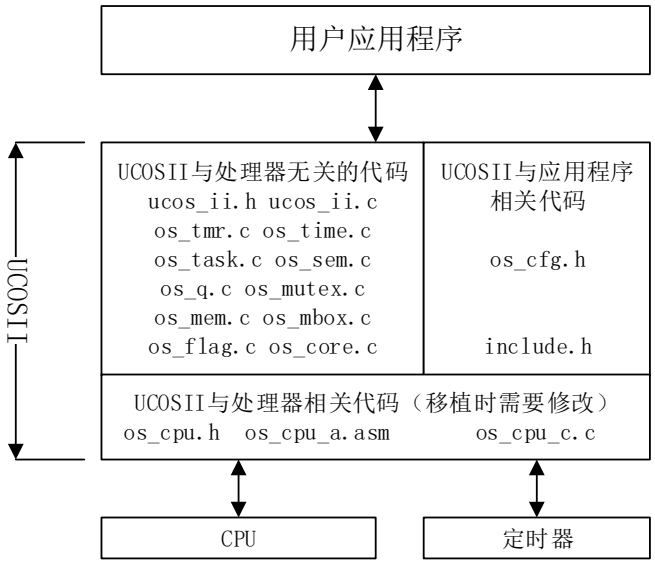


图 75.1.2.1 UCOSII 体系结构图

从上图可以看出，UCOSII 的移植，我们只需要修改：os_cpu.h、os_cpu_a.asm 和 os_cpu.c 三个文件即可，其中 os_cpu.h 是进行数据类型的定义以及处理器相关代码和几个函数原型；os_cpu_a.asm，是移植过程中需要汇编完成的一些函数，主要就是任务切换函数；os_cpu.c 定义一些用户 HOOK 函数。

图中定时器的作用是为 UCOSII 提供系统时钟节拍，实现任务切换和任务延时等功能。这个时钟节拍由 OS_TICKS_PER_SEC(在 os_cfg.h 中定义)设置，一般我们设置 UCOSII 的系统时钟节拍为 1ms~100ms，具体根据你所用处理器和使用需要来设置。本章，我们利用 STM32 的 SYSTICK 定时器来提供 UCOSII 时钟节拍。

75.1.3 任务定义

在前面也说到有任务，在前面的多任务系统中，我们根据功能的不同，把整个系统分成一个个独立且无法返回的函数，这些函数称为任务。而 UCOSII 就是一个能对这些任务的运行进行管理和调度的多任务操作系统。UCOSII 最大支持的任务数达到了 255 个，但是对于我们来说一般 64 个任务已经足够。

任务类型有两种：一种是系统任务，另一种是用户任务。由系统提供的任务叫系统任务，由用户编写的任务叫用户任务。系统任务是为应用程序提供某种服务或为系统本身服务的。UCOSII 具有 2 个系统任务，即空闲任务和统计任务，占用最低 2 个优先级。空闲任务是 UCOSII 优先级最低的任务，当所有其他任务均没有使用 CPU 时，空闲任务就会占用 CPU。统计任务是 UCOSII 优先级倒数第二低的任务，用于统计 CPU 的使用率和各个任务的堆栈使用情况。

相对于系统任务而言，我们开发者用得多的就是用户任务。用户任务需要注意的是：用户任务对应的函数是一个带有无限循环体的函数，没有返回值；每一个用户任务具有唯一的优先级号。

实时操作系统为了更好的调度任务，给每一个任务都定义了一个任务控制块 TCB（Task Control Block）。这个任务控制块就相当于任务在系统里的身份证，存放着任务的所有信息，比如任务函数指针，任务堆栈指针，任务优先级等。

由于 CPU 只有一个，所以一个时刻只会有一个任务占用 CPU 处于运行状态，而其他任务只能处于其他状态。UCOSII 系统中的任务具有 5 种，系统运行起来的时候，每一个任务都处在以下 5 种状态之一的状态下，这 5 种状态分别是睡眠状态、就绪状态、运行状态、等待状态和中断服务状态。

睡眠状态，任务在没有被配备任务控制块或被剥夺了任务控制块时的状态。

就绪状态，系统为任务配备了任务控制块且在任务就绪表中进行了就绪登记，任务已经准备好了，但由于该任务的优先级比正在运行的任务的优先级低，还暂时不能运行，这时任务的状态叫做就绪状态。

运行状态，该任务获得 CPU 使用权，并正在运行中，此时的任务状态叫做运行状态。

等待状态，正在运行的任务，需要等待一段时间或需要等待一个事件发生再运行时，该任务就会把 CPU 的使用权让给别的任务而使任务进入等待状态。

中断服务状态，一个正在运行的任务一旦响应中断申请就会中止运行而去执行中断服务程序，这时任务的状态叫做中断服务状态。

UCOSII 任务的 5 个状态转换关系如图 75.1.3.1 所示：

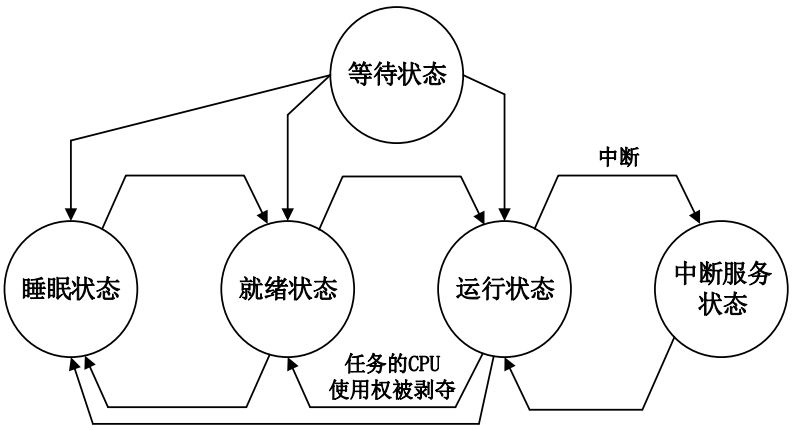


图 75.1.3.1 UCOSII 任务转换关系

75.1.4 任务调度

UCOSII 的任务调度思想是：“近似每时每刻让优先级最高的就绪任务处于运行状态”。在具体做法上，它在系统或者用户任务调用系统函数及执行中断服务程序结束时来调用调度器，以确定应该运行的任务并运行它。

在多任务系统中，令 CPU 中止当前正在运行的任务转而去运行另一个任务的工作叫任务切换，而按照某种规则进行任务切换的工作叫做任务调度。

在 UCOSII 中，任务调度是由任务调度器来完成。任务调度器的主要工作就有两个，①在任务就绪表中查找具有最高优先级别的就绪任务 ②实现任务切换

75.2 硬件设计

1. 例程功能

本实验我们在 UCOSII 里面创建 3 个任务：开始任务、LED0 任务和 LED1 任务，开始任务用于创建其他（LED0 和 LED1）任务，之后挂起；LED0 任务用于控制 LED0 的亮灭，LED0 每秒钟亮 80ms；LED1 任务用于控制 LED1 的亮灭，LED1 亮 300ms，灭 300ms，依次循环。

2. 硬件资源

1）LED 灯

LED0 ：LED0 - PB1

LED1 ：LED1 - PB0

75.3 程序设计

75.3.1 UCOSII 驱动函数

在这里主要对本实验用到的 UCOSII 驱动函数进行介绍。

1. OSTaskCreateExt 函数

创建任务函数，该函数是 OSTaskCreate 函数的扩展，并提供了一些附加功能。OSTaskCreateExt 函数创建任务更加灵活，不过会增加一些额外的开销。其声明如下：

```
INT8U  OSTaskCreateExt (void   (*task)(void *p_arg),
                        void    *p_arg,
                        OS_STK  *ptos,
                        INT8U    prio,
                        INT16U   id,
                        OS_STK  *pbos,
                        INT32U   stk_size,
                        void    *pext,
                        INT16U   opt)
```

- **函数描述：**
  用于创建一个任务
- **函数形参：**
  OSTaskCreatEx 函数的 9 个形参介绍，如表 75.3.1.1 所示：

参数	描述	
(*task)(void *p_arg)	指向任务的指针	
*p_arg	传递给任务的参数	
*ptos	指向任务堆栈栈顶的指针	
prio	任务的优先级	
id	任务的标识	
*pbos	任务堆栈栈底的指针	
stk_size	任务堆栈的容量	
*pext	指向附加数据域的指针	
opt	用于设定操作选项	
	OS_OPT_TASK_NONE	表示没有任何选项。
	OS_OPT_TASK_STK_CHK	指定是否允许检测该任务的堆栈。
	OS_OPT_TASK_STK_CLR	指定是否清除该任务的堆栈。
	OS_OPT_TASK_SAVE_FP	指定是否存储浮点寄存器，CPU 需要有浮点运算。

表 75.3.1.1 OSTaskCreatEx 函数的 9 个形参介绍

- **函数返回值：**
  OS_ERR_NONE：函数调用成功
  OS_ERR_PRIO_EXIST：具有该优先级的任务已经存在
  OS_ERR_PRIO_INVALID：参数指定的优先级大于最大优先级
  OS_ERR_TASK_CREATE_ISR：在 ISR 中创建任务
  OS_ERR_ILLEGAL_CREATE_RUN_TIME：尝试在安全关键操作启动后创建任务
- **注意事项：**
  1、任务必须被创建在多任务开始之前或者运行的任务中
  2、任务不能由 ISR 创建
  3、任务必须在死循环中，并且不能返回

2. OSTaskSuspend 函数

任务挂起函数，其声明如下：

```
INT8U  OSTaskSuspend (INT8U prio)
```

- **函数描述：**
  用于将任务挂起
- **函数形参：**
  Prio：要挂起任务的优先级。
- **函数返回值：**
  OS_ERR_NONE：函数调用成功
  OS_ERR_TASK_SUSPEND_IDLE：挂起空闲任务
  OS_ERR_PRIO_INVALID：参数指定的优先级大于最大优先级
  OS_ERR_TASK_SUSPEND_PRIO：需要挂起的任务不存在
  OS_ERR_TASK_NOT_EXITS：任务被分配到一个互斥执行

3. OSTaskDel 函数

删除任务函数，其声明如下：

```
INT8U  OSTaskDel (INT8U prio)
```

- **函数描述：**
  用于删除任务
- **函数形参：**
  Prio：要删除任务的优先级。如果任务不知道自己优先级，还可以传递参数

OS_PRIO_SELF。被删除的任务将回到休眠状态。

- **函数返回值：**

  OS_ERR_NONE：函数调用成功
  OS_ERR_TASK_DEL_IDLE：删除空闲任务
  OS_ERR_PRIO_INVALID：参数指定的优先级大于最大优先级
  OS_ERR_TASK_DEL：任务被分配给互斥量执行
  OS_ERR_TASK_NOT_EXIST：要删除的任务不存在
  OS_ERR_TASK_DEL_ISR：在中断处理函数中删除任务

4.OSInit 函数

UCOSII 系统初始化函数，其声明如下：

```
void  OSInit (void)
```

- **函数描述：**

  用于初始化 UCOSII 内部

- **函数形参：**无
- **函数返回值：**无

5. OSStart 函数

多任务启动函数，其声明如下：

```
void  OSStart (void)
```

- **函数描述：**

  用于用于启动多任务

- **函数形参：**无
- **函数返回值：**无
- **注意事项：**

  多任务的的启动是通过调用 OSStart 实现的，而在启动 UCOSII 之前至少需要建立一个应用任务。

75.3.2 程序流程图

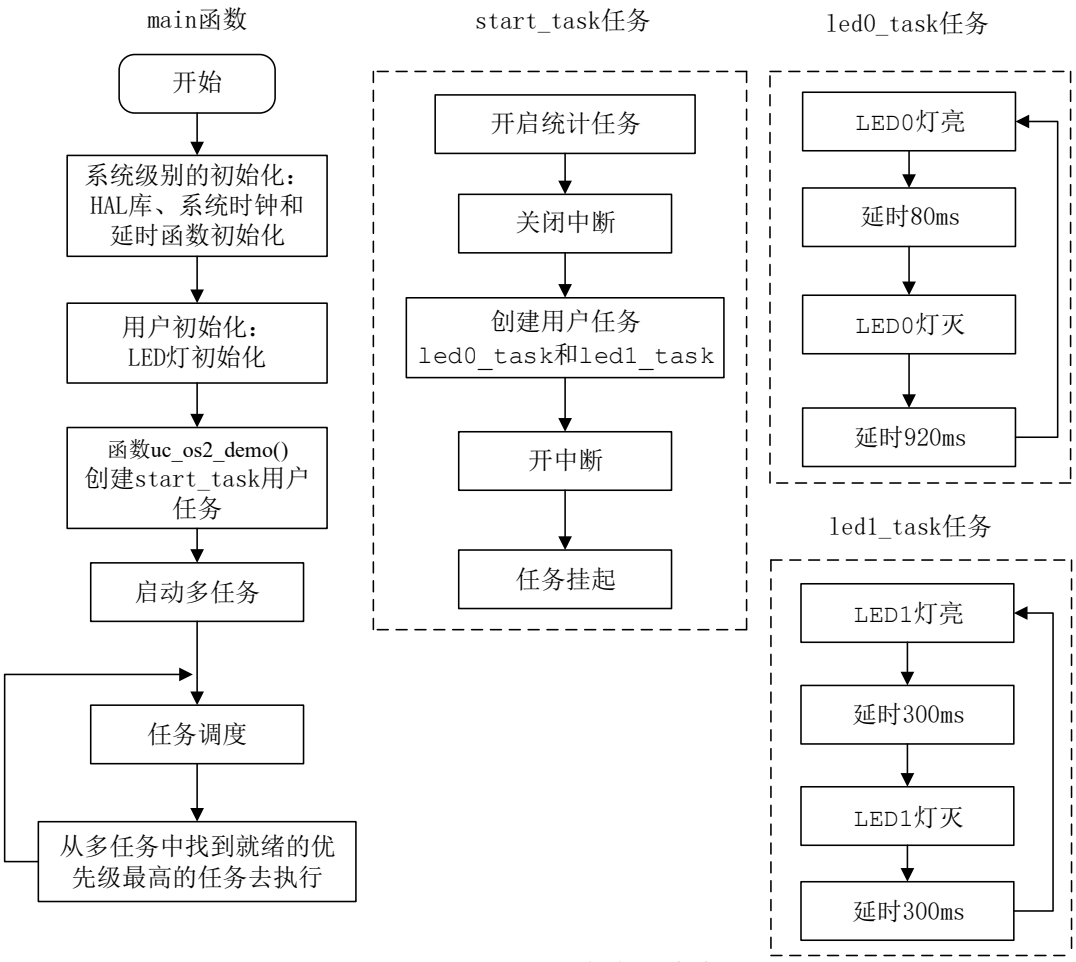


图 75.3.2.1 UCOSII 任务调度实验

我们在 main 函数中进行外设初始化后，然后调用函数 uc_os2_demo()，运行 ucosii 的例程，首先创建起始任务，并在起始任务创建 LED0 和 LED1 任务。任务初始化完成后，程序不断在 LED0 和 LED1 任务间切换。

75.3.3 程序解析

在 STM32 上运行 UCOSII 的步骤：

1、 移植 UCOSII

要使得 UCOSII 在 STM32 上正常运行，首先需要移植 UCOSII，这部分我们已经为大家做好了。（要学习更详细的 UCOS 移植方法，可以参考我们关于 UCOS 的专门教程）

这里我们需要注意的一个地方，SYSTEM 文件夹里面的系统函数直接支持 UCOSII，只需要在 sys.h 文件里将：SYSTEM_SUPPORT_UCOS 宏定义改为 1，即可通过 delay_init 函数初始化 UCOSII 的系统时钟节拍，为 UCOSII 提供时钟节拍。

2、 编写任务函数并设置其堆栈大小和优先级等参数

编写任务函数，以便 UCOSII 调用。

设置函数堆栈大小，这个需要根据函数的需求来设置，如果任务函数的局部变量多，嵌套层数多，那么对应的堆栈就得大一些，如果堆栈设置小了，很可能出现的结果就是 CPU 进入 HardFault，遇到这种情况，你就必须把堆栈设置大一点了。另外，有些地方还需要注意堆栈字节对齐的问题，如果任务运行出现莫名其妙的错误（比如用到 sprintf 出错），请考虑是不是字节对齐的问题。

设置任务优先级，这个需要大家根据任务的重要性和实时性设置，记住高优先级的任务有

优先使用 CPU 的权力。

3、 初始化 UCOSII，并在 UCOSII 中创建任务

调用 OSInit，初始化 UCOSII，通过调用 OSTaskCreate 函数创建我们的任务。

4、 启动 UCOSII

调用 OSStart，启动 UCOSII。

通过以上 4 个步骤，UCOSII 就开始在 STM32 上面运行了，这里还需要注意我们必须对 os_cfg.h 进行部分配置，以满足我们的需求。

main.c 代码

在 main.c 文件下，只初始化一些外设，然后调用 ucosii 的例程入口函数 uc_os2_demo()，如下代码所示：

```
#include "./SYSTEM/sys/sys.h"
#include "./SYSTEM/usart/usart.h"
#include "./SYSTEM/delay/delay.h"
#include "./BSP/LED/led.h"
#include "uc-os2_demo.h"

int main(void)
{
    sys_cache_enable();                     /* 打开 L1-Cache */
    HAL_Init();                             /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);     /* 设置时钟,400Mhz */
    delay_init(400);                        /* 延时初始化 */
    led_init();                             /* 初始化 LED */

    uc_os2_demo();                          /* 运行 uC/OS-II 例程 */
}
```

可以看到，在 main.c 文件中只包含了一个 main()函数，main()函数主要就是完成了一些外设的初始化，如串口、LED 等，并在最后调用了函数 uc_os2_demo()。

下面看一下 uc-os2_demo.c 的代码：

```
/* UCOSII 任务设置 */

/* START 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define START_TASK_PRIO                 10      /* 开始任务的优先级设置为最低 */
#define START_STK_SIZE                  128     /* 堆栈大小 */

OS_STK START_TASK_STK[START_STK_SIZE];          /* 任务堆栈 */
void start_task(void *pdata);                   /* 任务函数 */


/* LED0 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define LED0_TASK_PRIO                  7       /* 开始任务的优先级设置为最低 */
#define LED0_STK_SIZE                   128     /* 堆栈大小 */
OS_STK LED0_TASK_STK[LED0_STK_SIZE];            /* 任务堆栈 */
void led0_task(void *pdata);                    /* 任务函数 */

/* LED1 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define LED1_TASK_PRIO                  6       /* 开始任务的优先级设置为最低 */
#define LED1_STK_SIZE                   128     /* 堆栈大小 */
OS_STK LED1_TASK_STK[LED0_STK_SIZE];            /* 任务堆栈 */
void led1_task(void *pdata);                    /* 任务函数 */
```

```
/*********************************************************/
/**
 * @brief       uC/OS-II 例程入口函数
 * @param       无
 * @retval      无
 */
void uc_os2_demo(void)
{
    OSInit();                                                   /* UCOS 初始化 */
    OSTaskCreateExt((void(*)(void *) )start_task,               /* 任务函数 */
                    (void *           )0,                       /* 传递给任务函数的参数 */
                    (OS_STK *         )&START_TASK_STK[START_STK_SIZE - 1],
                    /* 任务堆栈栈顶 */
                    (INT8U            )START_TASK_PRIO,         /* 任务优先级 */
                    (INT16U           )START_TASK_PRIO,
                    /* 任务 ID，这里设置为和优先级一样 */
                    (OS_STK *         )&START_TASK_STK[0],/* 任务堆栈栈底 */
                    (INT32U           )START_STK_SIZE,          /* 任务堆栈大小 */
                    (void *           )0,                       /* 用户补充的存储区 */
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                      | OS_TASK_OPT_SAVE_FP);
                    /* 任务选项,为了保险起见，所有任务都保存浮点寄存器的值 */
    OSStart();  /* 开始任务 */

    for (;;)
    {
        /* 不会进入这里 */
    }
}

/**
 * @brief       开始任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void start_task(void *p_arg)
{
    OS_CPU_SR cpu_sr = 0;
    CPU_INT32U cnts;

    OSStatInit();                       /* 开启统计任务 */

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OS_TICKS_PER_SEC);
    OS_CPU_SysTickInit(cnts);
    OS_ENTER_CRITICAL();                /* 进入临界区(关闭中断) */

    OSTaskCreateExt((void(*)(void *) )led0_task,
                    (void *           )0,
                    (OS_STK *         )&LED0_TASK_STK[LED0_STK_SIZE - 1],
                    (INT8U            )LED0_TASK_PRIO,
                    (INT16U           )LED0_TASK_PRIO,
                    (OS_STK *         )&LED0_TASK_STK[0],
                    (INT32U           )LED0_STK_SIZE,
                    (void *           )0,
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                      | OS_TASK_OPT_SAVE_FP);
    /* LED1 任务 */
    OSTaskCreateExt((void(*)(void *) )led1_task,
                    (void *           )0,
                    (OS_STK *         )&LED1_TASK_STK[LED1_STK_SIZE - 1],
                    (INT8U            )LED1_TASK_PRIO,
```

```
                    (INT16U           )LED1_TASK_PRIO,
                    (OS_STK *         )&LED1_TASK_STK[0],
                    (INT32U           )LED1_STK_SIZE,
                    (void *           )0,
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                      | OS_TASK_OPT_SAVE_FP);

    OS_EXIT_CRITICAL();             /* 退出临界区(开中断) */
    OSTaskSuspend(START_TASK_PRIO); /* 挂起开始任务 */
}
/**
 * @brief       LED0 任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void led0_task(void *p_arg)
{
    while (1)
    {
        LED0(0);
        OSTimeDly(80);
        LED0(1);
        OSTimeDly(920);
    }
}

/**
 * @brief       LED1 任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void led1_task(void *p_arg)
{
    while (1)
    {
        LED1(0);
        OSTimeDly(300);
        LED1(1);
        OSTimeDly(300);
    }
}
```

对于 uc-os2_demo.c 文件，这里先简单介绍一下这个文件的代码结构，这个文件的代码结构可分为五个部分，分别是包含头文件、μC/OS-II 相关配置、应用程序入口函数、开始任务入口函数、其他任务入口函数，接下来分别地介绍以上一个部分的代码。

① 包含的头文件分成两个部分，分别为 μC/OS-II 相关的头文件和其他头文件，如下所示：

```
#include "uc-os2_demo.h"
#include "./SYSTEM/usart/usart.h"
#include "./BSP/LED/led.h"
/*uC/OS-III*********************************************************/
#include "os.h"
#include "cpu.h"

/*******************************************************************/
```

② μC/OS-II 的配置主要包括所创建 μC/OS-II 任务的相关定义（任务优先级、任务栈大小、任务栈、任务函数）以及定义应用于全局的 μC/OS-II 相关变量（信号量、事件标志、消息队列、软件定时器等），如下所示：

```
/* START 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define START_TASK_PRIO                 10      /* 开始任务的优先级设置为最低 */
#define START_STK_SIZE                  128     /* 堆栈大小 */
```

```
OS_STK START_TASK_STK[START_STK_SIZE];          /* 任务堆栈 */
void start_task(void *pdata);                   /* 任务函数 */


/* LED0 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define LED0_TASK_PRIO                  7       /* 开始任务的优先级设置为最低 */
#define LED0_STK_SIZE                   128     /* 堆栈大小 */
OS_STK LED0_TASK_STK[LED0_STK_SIZE];            /* 任务堆栈 */
void led0_task(void *pdata);                    /* 任务函数 */

/* LED1 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define LED1_TASK_PRIO                  6       /* 开始任务的优先级设置为最低 */
#define LED1_STK_SIZE                   128     /* 堆栈大小 */
OS_STK LED1_TASK_STK[LED0_STK_SIZE];            /* 任务堆栈 */
void led1_task(void *pdata);                    /* 任务函数 */
```

③ 这部分就是函数 uc_os2_demo()，函数 uc_os2_demo()一开始就是对 μC/OS-II 内核进行初始化，接着创建一个开始任务，最后开启 μC/OS-II 内核的任务调度，如下所示：

```
/**
 * @brief       uC/OS-II 例程入口函数
 * @param       无
 * @retval      无
 */
void uc_os2_demo(void)
{
    OSInit();                                                   /* UCOS 初始化 */
    OSTaskCreateExt((void(*)(void *) )start_task,               /* 任务函数 */
                    (void *           )0,                       /* 传递给任务函数的参数 */
                    (OS_STK *         )&START_TASK_STK[START_STK_SIZE - 1],
                    /* 任务堆栈栈顶 */
                    (INT8U            )START_TASK_PRIO,         /* 任务优先级 */
                    (INT16U           )START_TASK_PRIO,
                    /* 任务 ID，这里设置为和优先级一样 */
                    (OS_STK *         )&START_TASK_STK[0],/* 任务堆栈栈底 */
                    (INT32U           )START_STK_SIZE,          /* 任务堆栈大小 */
                    (void *           )0,                       /* 用户补充的存储区 */
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                      | OS_TASK_OPT_SAVE_FP);
                    /* 任务选项,为了保险起见，所有任务都保存浮点寄存器的值 */
    OSStart();  /* 开始任务 */

    for (;;)
    {
        /* 不会进入这里 */
    }
}
```

④ 这部分就是开始任务的入口函数，开始任务主要用于初始化 μC/CPU 组件、配置 μC/OS-II 内核节拍的时基定时器、并且还会创建其他用于实验演示的任务，如下所示：

```
/**
 * @brief       开始任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void start_task(void *p_arg)
{
    OS_CPU_SR cpu_sr = 0;
    CPU_INT32U cnts;
```

```
    OSStatInit();                       /* 开启统计任务 */

    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OS_TICKS_PER_SEC);
    OS_CPU_SysTickInit(cnts);
    OS_ENTER_CRITICAL();                /* 进入临界区(关闭中断) */

    OSTaskCreateExt((void(*)(void *) )led0_task,
                    (void *           )0,
                    (OS_STK *         )&LED0_TASK_STK[LED0_STK_SIZE - 1],
                    (INT8U            )LED0_TASK_PRIO,
                    (INT16U           )LED0_TASK_PRIO,
                    (OS_STK *         )&LED0_TASK_STK[0],
                    (INT32U           )LED0_STK_SIZE,
                    (void *           )0,
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                      | OS_TASK_OPT_SAVE_FP);
    /* LED1 任务 */
    OSTaskCreateExt((void(*)(void *) )led1_task,
                    (void *           )0,
                    (OS_STK *         )&LED1_TASK_STK[LED1_STK_SIZE - 1],
                    (INT8U            )LED1_TASK_PRIO,
                    (INT16U           )LED1_TASK_PRIO,
                    (OS_STK *         )&LED1_TASK_STK[0],
                    (INT32U           )LED1_STK_SIZE,
                    (void *           )0,
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                      | OS_TASK_OPT_SAVE_FP);

    OS_EXIT_CRITICAL();                 /* 退出临界区(开中断) */
    OSTaskSuspend(START_TASK_PRIO); /* 挂起开始任务 */
}
```

从上面的代码可以看到 start_task 函数中创建了 led0_task 和 led1_task 两个任务，创建这两个任务后，将自己挂起。

我们单独创建 start_task 的目的是为了提供一个单一任务，实现应用程序开始之前的准备工作，比如外设初始化，创建任务，初始化统计任务，以及后面讲到的创建信号量、创建邮箱、创建消息队列、创建信号量集等。

在应用程序中经常有一些代码段必须不受任何干扰地连续运行，这样的代码段叫做临界段（或临界区）。因此，为了使临界段在运行时不受中断所打断，在临界段代码前必须用关中断指令使 CPU 屏蔽中断请求，而在临界段代码后必须用开中断指令接触屏蔽使得 CPU 可以响应中断请求。因为临界段代码不能被中断打断，将严重影响系统的实时性，所以临界段代码越短越好!

在 start_task 任务中，我们在创建 led0_task 和 led1_task 的时候，不希望中断打断，故使用了临界区。

⑤ 这部分包含了实验中用于演示的任务入口函数，如下所示：

```
/**
 * @brief       LED0 任务
 * @param       pdata : 传入参数(未用到)
 * @retval      无
 */
void led0_task(void *pdata)
{
    while (1)
    {
        LED0(0);
        delay_ms(80);
        LED0(1);
        delay_ms(920);
    }
}
```

```
/**
 * @brief       LED1 任务
 * @param       pdata : 传入参数(未用到)
 * @retval      无
 */
void led1_task(void *pdata)
{
    while (1)
    {
        LED1(0);
        delay_ms(300);
        LED1(1);
        delay_ms(300);
    }
}
```

上述两个任务较为简单，分别控制 led0 与 led1 不同频率的翻转，所以就不多做介绍。

以上就是 uc_os2_demo.c 文件的代码结构，本教程配套的 3 个 ucosii 的实验例程都会按照这个代码结构进行实验代码的编写，建议读者先熟悉以下这个代码结构。

另外，一个任务里面一般是必须有延时函数的，以释放 CPU 使用权，否则可能导致低优先级的任务因高优先级的任务不释放 CPU 使用权而一直无法得到 CPU 使用权，从而无法运行。

软件设计部分就为大家介绍到这里。

75.4 下载验证

将程序下载到开发板后，可以看到 LED0 一秒钟闪一次，而 LED1 则以固定的频率闪烁。说明两个任务（led0_task 和 led1_task）都已经正常运行，符合我们预期的设计。

第七十六章 UCOSII 实验 2-信号量和邮箱

上一章，我们学习了如何使用 UCOSII，学习了 UCOSII 的任务调度，但是并没有用到任务间的同步与通信，本章我们将学习两个最基本的任务间通讯方式：信号量和邮箱。

本章分为如下几个小节：

76.1 UCOSII 信号量和邮箱简介
76.2 硬件设计
76.3 程序设计
76.4 下载验证

76.1 UCOSII 信号量和邮箱简介

系统中的多个任务在运行时，经常需要互相无冲突地访问同一个共享资源，或者需要互相支持和依赖，甚至有时还要互相加以必要的限制和制约，才保证任务的顺利运行。因此，操作系统必须具有对任务的运行进行协调的能力，从而使任务之间可以无冲突、流畅地同步运行，而不导致灾难性的后果。

例如，任务 A 和任务 B 共享一台打印机，如果系统已经把打印机分配给了任务 A，则任务 B 因不能获得打印机的使用权而应该处于等待状态，只有当任务 A 把打印机释放后，系统才能唤醒任务 B，使其获得打印机的使用权。如果这两个任务不这样做，那么会造成极大的混乱。

任务间的同步依赖于任务间的通信。在 UCOSII 中，是使用信号量、邮箱（消息邮箱）和消息队列，这些被称作事件的中间环节来实现任务之间的通信的。这里我们仅介绍信号量和邮箱，消息队列将会在下一章介绍。

事件

两个任务通过事件进行通讯的示意图如图 76.1.1 所示：

图 76.1.1 两个任务使用事件进行通信的示意图

在上图中任务 1 是发信方，任务 2 是收信方。任务 1 负责把信息发送到事件上，这项操作叫做发送事件。任务 2 通过读取事件操作对事件进行查询：如果有信息则读取，否则等待。读事件操作叫做请求事件。

为了把描述事件的数据结构统一起来，UCOSII 使用叫做事件控制块(ECB)的数据结构来描述诸如信号量、邮箱（消息邮箱）和消息队列这些事件。事件控制块中包含包括等待任务表在内的所有有关事件的数据，事件控制块结构体定义如下：

```
typedef struct os_event {
    INT8U    OSEventType;                       /* 事件的类型 */
    void    *OSEventPtr;                        /* 消息或消息队列的指针 */
    INT16U   OSEventCnt;                        /* 信号量计数器 */
    OS_PRIO  OSEventGrp;                        /* 等待事件的任务组 */
    OS_PRIO  OSEventTbl[OS_EVENT_TBL_SIZE];     /* 任务等待表 */
#if OS_EVENT_NAME_EN > 0u
    INT8U   *OSEventName;                       /* 事件名 */
#endif
} OS_EVENT;
```

信号量

使用信号量的最初目的，是为了给共享资源设立一个标志，该标志表示该共享资源的占用情况。这样，当一个任务在访问共享资源之前，就可以先对这个标志进行查询，从而在了解资源被占用的情况之后，再来决定自己的行为。

信号量的实质是一个全局计数器的实现机制，释放信号量的任务使得该计数器的值加 1，请求到信号量的任务使得该计数器的值减 1。如果计数器得值为 0，则请求该信号量得任务将挂起等待，直到别的任务释放该信号量。通过这种方式，使得释放信号量的任务可以控制请求信号量的任务的运行。

信号量的工作原理如图 76.1.2 所示：

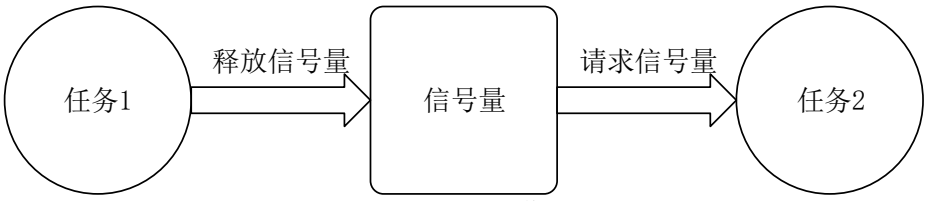


图 76.1.2 信号量工作原理图

信号量可以分为两种：一种是二值型信号量，另外一种是 N 值信号量。

UCOSII 将二值型信号量称之为也叫互斥型信号量，将 N 值信号量称之为计数型信号量，也就是普通的信号量。

信号量相关的主要操作有：创建信号量 OSSemCreate、请求信号量 OSSemPend、释放信号量 OSSemPost 和删除信号量 OSSemDel。后面再对这几个函数进行讲解。

邮箱

在多任务操作系统中，常常需要在任务与任务之间通过传递一个数据（这种数据叫做“消息”）的方式来进行通信。为了达到这个目的，可以在内存中创建一个存储空间作为该数据的缓冲区。如果把这个缓冲区称之为消息缓冲区，这样在任务间传递数据（消息）的最简单办法就是传递消息缓冲区的指针。我们把用来传递消息缓冲区指针的数据结构叫做邮箱（消息邮箱）。消息邮箱的工作情况如图 76.1.3 所示：

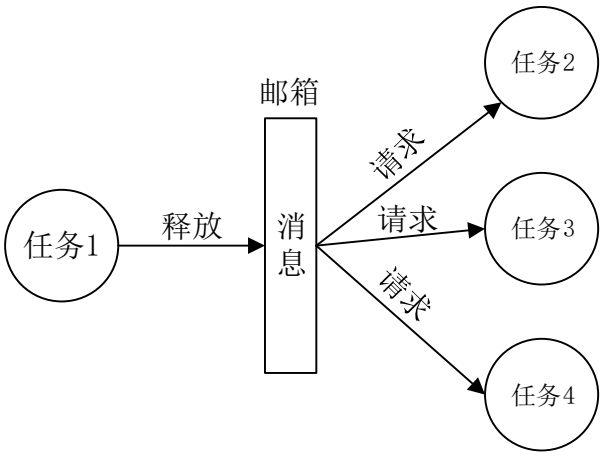


图 76.1.3 消息邮箱工作情况图

从上图可知道，只有任务才能请求消息，邮箱里仅能存放一条消息，如果释放消息的速度比请求消息的速度快，则释放的消息将会丢失。可以通过广播的方式，使得释放的消息传递给所有请求该消息邮箱的任务。如果当前邮箱为空，且有某一任务 2 正在请求邮箱，则当另一任务 1 向邮箱中释放消息时，释放的消息将直接发送给任务 2，而不用经过邮箱中转。

在 UCOSII 中，我们通过事件控制块的 OSEventPrt 来传递消息缓冲区指针，同时使事件控制块的成员 OSEventType 为常数 OS_EVENT_TYPE_MBOX，则该事件控制块就叫做消息邮箱。

与消息邮箱相关的主要操作有：创建邮箱函数 OSMboxCreate、向邮箱发送消息函数 OSMboxPost、请求邮箱函数 OSMboxPend、查询邮箱状态函数 OSMboxQuery 和删除邮箱函数 OSMboxDel。后面再对这几个进行函数进行讲解。

76.2 硬件设计

1. 例程功能

本实验我们在 UCOSII 里面创建 6 个任务：开始任务、LED 任务、触摸屏任务、蜂鸣器任

务、按键扫描任务和主任务，开始任务用于创建信号量、创建邮箱、初始化统计任务以及其他任务的创建，之后挂起；LED 任务用于 LED0 控制，提示程序运行状况；蜂鸣器任务用于测试信号量，是请求信号量函数，每得到一个信号量，蜂鸣器就叫一次；触摸屏任务用于在屏幕上画图，可以用于测试 CPU 使用率；按键扫描任务用于按键扫描，优先级最高，将得到的键值通过消息邮箱发送出去；主任务则通过查询消息邮箱获得键值，并根据键值执行 LED1 控制、信号量发送（蜂鸣器控制）、触摸区域清屏和触摸屏校准等控制。

通过按 KEY0，可以控制 KEY1 的亮灭。

通过按 KEY1，则可以控制蜂鸣器的发声（连续按下多次后，可以看到蜂鸣每隔 1 秒叫一次）。

通过按 KEY_UP，可以进入校准程序，进行触摸屏校准（注意，电容触摸屏不需要校准，所以如果是电容屏，

按 KEY_UP，就相当于清屏一次的效果，不会进行校准）

电阻屏校准方法：

1，下载本代码。

2，按 KEY_UP。

3，依次点击屏幕上的 5 个十字架（十字+小圈圈）的最中央（最好用笔尖）

4，直到屏幕提示："Touch Screen Adjust OK!"字样。

5，校准完成。

2. 硬件资源

1）LED 灯

LED0：LED0 – PB1

LED1：LED1 – PB0

2）独立按键

KEY0 – PH3

KEY1 – PH2

WK_UP – PA0

3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块（包括 MCU 屏和 RGB 屏，都支持）

4）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)

5）触摸屏(电阻式/电容式)

76.3 程序设计

76.3.1 UCOSII 相关函数介绍

信号量函数

在这里对本实验用到的 UCOSII 信号量函数进行介绍，相关代码存放在 os_sem.c 中。

1. OSSemCreate 函数

创建信号量函数，其声明如下：

```
OS_EVENT  *OSSemCreate (INT16U cnt)
```

- **函数描述：**

  用于创建一个信号量

- **函数形参：**

  cnt 是信号量计数器（OSEventCnt）的初始值

- **函数返回值：**

  已创建的信号量的指针

2. OSSemPend 函数

请求信号量函数，其声明如下：

```
void  OSSemPend (OS_EVENT *pevent, INT32U timeout, INT8U *perr)
```

- **函数描述：**

请求信号量

- **函数形参：**

  pevent：被请求信号量的指针

  timeout：等待时限

  perr：错误信息

  OS_ERR_NONE：调用成功，信号量不为零

  OS_ERR_TIMEOUT：信号量没有在指定数目的时钟周期内被设置

  OS_ERR_PEND_ABOUT：取消对信号量的等待

  OS_ERR_EVENT_TYPE：没有传递信号量的指针

  OS_ERR_PEND_ISR：从中断调用该函数时错误

  OS_ERR_PEVENT_NULL：pevent 是一个空指针

  OS_ERR_PEND_LOCKED：调度器上锁了

- **函数返回值：**

  无

- **注意事项：**

  为了防止任务因得不到信号量而处于长期的等待状态，函数 OSSemPend 允许用参数 timeout 设置一个等待时间的限制，当任务等待的时间超过 timeout 时可以结束等待状态而进入就绪状态。如果参数 timeout 被设置为 0，则表明任务的等待时间为无限长。

3. OSSemPost 函数

发送信号量函数，其声明如下：

```
INT8U  OSSemPost (OS_EVENT *pevent)
```

- **函数描述：**

  用于发送信号量或者称为释放信号量

- **函数形参：**

  pevent：被请求信号量的指针

- **函数返回值：**

  OS_ERR_NONE：函数调用成功，信号量被成功地设置

  OS_ERR_SEM_OVF：信号量的值溢出

  OS_ERR_EVENT_TYPE：pevent 不是指向信号量的指针

  OS_ERR_PEVENT_NULL：pevent 是一个空指针

4.OSSemDel 函数

删除信号量函数，其声明如下：

```
OS_EVENT  *OSSemDel (OS_EVENT *pevent, INT8U opt, INT8U *perr)
```

- **函数描述：**

  用于删除信号量并准备挂起所有任务

- **函数形参：**

  pevent：要删除的信号量指针

  opt：删除条件选项

  OS_DEL_NO_PEND：在没有任务挂起时删除信号量

  OS_DEL_ALWAYS：删除信号量，即使任务正在等待。

  perr：错误信息

  OS_ERR_NONE：函数调用成功，成功删除信号量

  OS_ERR_DEL_ISR：尝试在中断中删除信号量

  OS_ERR_INVALID_OPT：指向一个无效的选项

  OS_ERR_TASK_WAITING：一个或多个任务在等待这个信号量

  OS_ERR_EVENT_TYPE：没有传递一个指向信号量的指针

  OS_ERR_PEVENT_NULL：pevent 是一个空指针

- **函数返回值：**

  pevent：存在错误　　(OS_EVENT *)0：该信号量被成功删除。

消息邮箱函数

在这里对本实验用到的 UCOSII 消息邮箱函数进行介绍，相关代码存放在 os_mbox.c 中

1. OSMboxCreate 函数

创建邮箱函数，其声明如下：

```
OS_EVENT  *OSMboxCreate (void *pmsg)
```

- **函数描述：**
  用于创建邮箱函数
- **函数形参：**
  pmsg：消息的指针
- **函数返回值：**
  消息邮箱的指针
- **注意事项：**
  调用 OSMboxCreate 前，需先定义 msg 的初始值。在一般情况下，这个初始值为 NULL。但也可以事先定义一个邮箱，然后把这个邮箱的指针作为参数传递到函数 OSMboxCreate 中，使得一开始就指向一个邮箱。

2. OSMboxPost 函数

向邮箱发送消息函数，其声明如下：

```
INT8U  OSMboxPost (OS_EVENT  *pevent, void *pmsg)
```

- **函数描述：**
  用于向消息邮箱发送消息
- **函数形参：**
  pevent：消息邮箱的指针
  pmsg：消息指针
- **函数返回值：**
  OS_ERR_NONE：消息发送成功
  OS_ERR_MBOX_FULL：不能向满邮箱再发送消息
  OS_ERR_EVENT_TYPE：指定的事件不是消息邮箱类型
  OS_ERR_PEVENT_NULL：不能向不存在的消息邮箱发送消息
  OS_ERR_POST_NULL_PTR：消息缓冲区不能为空

3. OSMboxPend 函数

请求邮箱函数，其声明如下：

```
void  *OSMboxPend (OS_EVENT  *pevent, INT32U timeout, INT8U *perr)
```

- **函数描述：**
  请求消息邮箱，就是等待一个消息传送到消息邮箱或取得一个消息数据
- **函数形参：**
  pevent：消息邮箱的指针
  timeout：等待时限
  perr：错误信息
  OS_ERR_NONE：函数调用成功，接收到消息
  OS_ERR_TIMEOUT：未在“超时”时间内接收到消息
  OS_ERR_PEND_ABORT：终止邮箱的等待
  OS_ERR_EVENT_TYPE：无效事件类型
  OS_ERR_PEND_ISR：从 ISR 中调用该函数，导致任务挂起
  OS_ERR_PEVENT_NULL：pevent 是一个空指针
  OS_ERR_PEND_LOCKED：调度器上锁了
- **函数返回值：**
  NULL：未得到消息　　！NULL：预期消息的指针

4. OSMboxQuery 函数

查询邮箱状态函数，其声明如下：

```
UINT8U  *OSMboxQuery (OS_EVENT  *pevent, OS_MBOX_DATA *p_mbox_data)
```

- **函数描述：**
  获取消息邮箱的相关信息
- **函数形参：**
  pevent：消息邮箱的指针
  p_mbox_data：存放邮箱信息的结构
- **函数返回值：**
  OS_ ERR_NONE：调用成功
  OS_ERR_EVENT_TYPE：pevent 不是指向消息邮箱的指针
  OS_ERR_PEVENT_NULL：不能向不存在的消息邮箱发送消息
  OS_ERR_PDATA_NULL：p_mbox_data 是一个空指针
- **注意事项：**
  必须先建立消息邮箱，然后使用

5.OSMboxDel 函数

删除邮箱函数，其声明如下：

```
OS_EVENT  *OSMboxDel (OS_EVENT *pevent, INT8U opt, INT8U *perr)
```

- **函数描述：**
  对一个不再使用的消息邮箱及时删除以释放资源
- **函数形参：**
  pevent：要删除的邮箱指针
  opt：删除条件选项
  OS_DEL_NO_PEND：在没有任务挂起时删除邮箱
  OS_DEL_ALWAYS：无条件删除邮箱，所有等待该事件的任务转到就绪态。
  perr：错误信息
  OS_ERR_NONE：函数调用成功，成功删除邮箱
  OS_ERR_DEL_ISR：不支持在中断中删除邮箱
  OS_ERR_INVALID_OPT：指向一个无效的选项
  OS_ERR_TASK_WAITING：一个或多个任务在等待这个信号量
  OS_ERR_EVENT_TYPE：没有传递一个指向邮箱的指针
  OS_ERR_PEVENT_NULL：pevent 是一个空指针
- **函数返回值：**
  pevent ：存在错误　　(OS_EVENT *)0 : 该邮箱被成功删除。

76.3.2 程序流程图

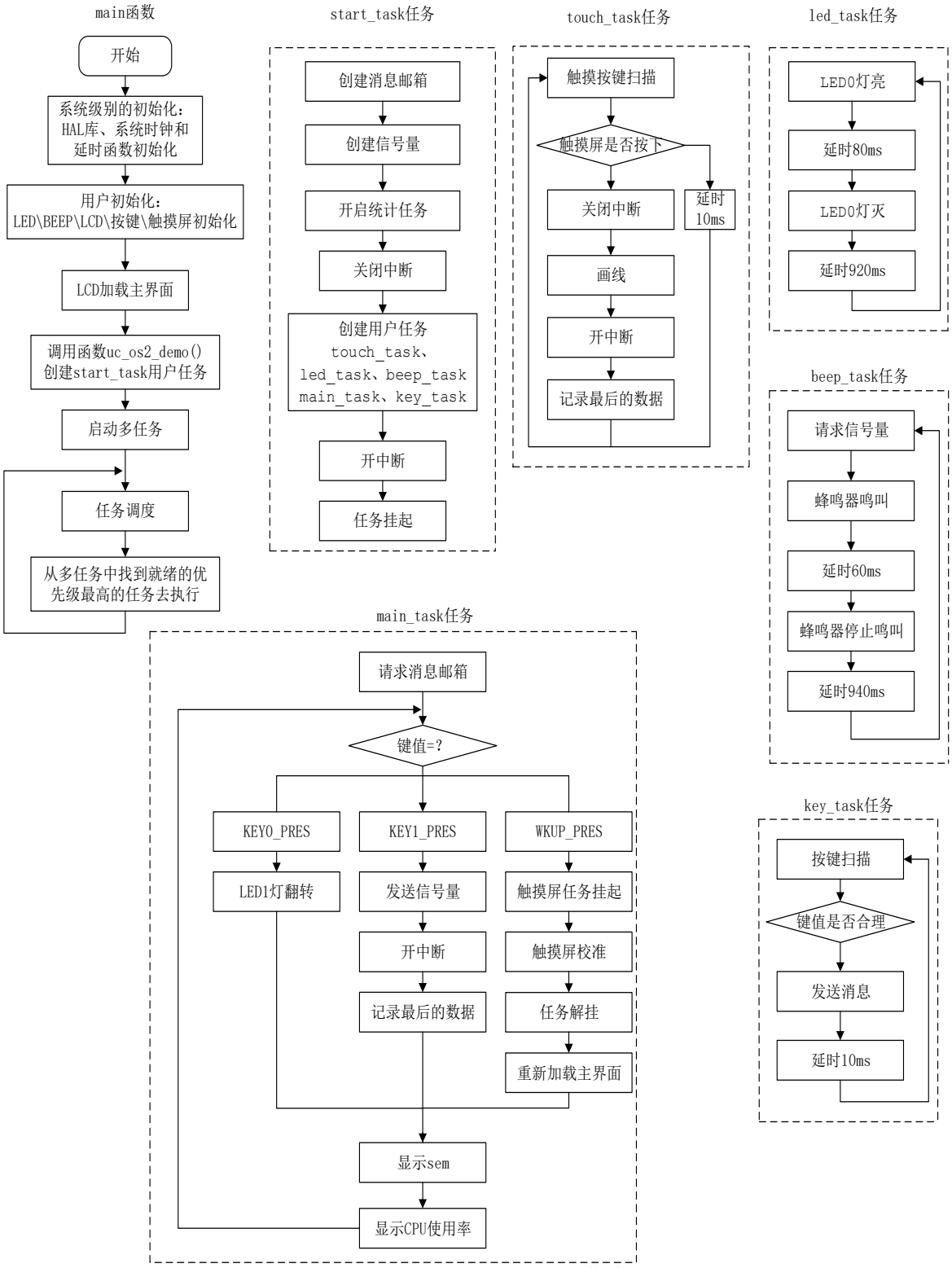


图 76.3.2.1 UCOSII 信号量和邮箱实验

程序我们按流程图的设计来实现本节的功能代码。我们通过 start_task 创建其他任务，包括：

LED 任务，成功请求信号量时 LED 灯闪烁一次，否则阻塞；

蜂鸣器任务，成功请求信号量时蜂鸣器响一次，否则阻塞；

按键扫描函数，根据按键发送不同的消息；

main_task 用于创建定时器任务，并根据获取的队列消息得到按键值并投递给其他任务，控制部分任务运行或挂起，重绘界面等。

76.3.3 程序解析

main.c 代码

在 main.c 文件下，只初始化一些外设，然后调用 ucosii 的例程入口函数 uc_os2_demo()，如下代码所示：

```
int main(void)
{
    sys_cache_enable();                             /* 打开 L1-Cache */
    HAL_Init();                                     /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);             /* 设置时钟,400Mhz */
    delay_init(400);                                /* 延时初始化 */
    usart_init(115200);                             /* 串口初始化 */
    mpu_memory_protection();                        /* 保护相关存储区域 */
    led_init();                                     /* 初始化 LED */
    sdram_init();                                   /* 初始化 SDRAM */
    lcd_init();                                     /* 初始化 LCD */
    key_init();                                     /* 初始化按键 */
    pcf8574_init();                                 /* 初始化 PCF8574 */
    tp_dev.init();                                  /* 触摸屏初始化 */

    uc_os2_demo();                                  /* 运行 uC/OS-II 例程 */
}
```

可以看到，在 main.c 文件中只包含了一个 main()函数，main()函数主要就是完成了一些外设的初始化，如串口、LED、LCD 等，并在最后调用了函数 uc_os2_demo()。

下面看一下 uc-os2_demo.c 的代码：

```
/* UCOSII 任务设置 */

/* START 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define START_TASK_PRIO                 10      /* 开始任务的优先级设置为最低 */
#define START_STK_SIZE                  128     /* 堆栈大小 */

OS_STK START_TASK_STK[START_STK_SIZE];          /* 任务堆栈 */
void start_task(void *pdata);                   /* 任务函数 */


/* 触摸屏任务 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define TOUCH_TASK_PRIO                 7       /* 优先级设置(越小优先级越高) */
#define TOUCH_STK_SIZE                  128     /* 堆栈大小 */

OS_STK TOUCH_TASK_STK[TOUCH_STK_SIZE];          /* 任务堆栈 */
void touch_task(void *pdata);                   /* 任务函数 */


/* LED 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define LED_TASK_PRIO                   6       /* 优先级设置(越小优先级越高) */
#define LED_STK_SIZE                    128     /* 堆栈大小 */
```

```
OS_STK LED_TASK_STK[LED_STK_SIZE];              /* 任务堆栈 */
void led_task(void *pdata);                     /* 任务函数 */


/* 蜂鸣器 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define BEEP_TASK_PRIO                  5       /* 优先级设置(越小优先级越高) */
#define BEEP_STK_SIZE                   128     /* 堆栈大小 */

OS_STK BEEP_TASK_STK[BEEP_STK_SIZE];            /* 任务堆栈 */
void beep_task(void *pdata);                    /* 任务函数 */


/* 主 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define MAIN_TASK_PRIO                  4       /* 优先级设置(越小优先级越高) */
#define MAIN_STK_SIZE                   512     /* 堆栈大小 */

OS_STK MAIN_TASK_STK[MAIN_STK_SIZE];            /* 任务堆栈 */
void main_task(void *pdata);                    /* 任务函数 */


/* 按键扫描 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define KEY_TASK_PRIO                   3       /* 优先级设置(越小优先级越高) */
#define KEY_STK_SIZE                    128     /* 堆栈大小 */

OS_STK KEY_TASK_STK[KEY_STK_SIZE];              /* 任务堆栈 */
void key_task(void *pdata);                     /* 任务函数 */


/*********************************************************************/
OS_EVENT *msg_key;      /* 按键邮箱事件块指针 */
OS_EVENT *sem_beep;     /* 蜂鸣器信号量指针 */
void ucos_load_main_ui(void);
void lcd_draw_bline(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2,
                    uint8_t size, uint16_t color);

/**
 * @brief       uC/OS-II 例程入口函数
 * @param       无
 * @retval      无
 */
void uc_os2_demo(void)
{
    ucos_load_main_ui();                        /* 加载主界面 */

    OSInit();                                   /* UCOS 初始化 */
    OSTaskCreateExt((void(*)(void *) )start_task,     /* 任务函数 */
                    (void *           )0,             /* 传递给任务函数的参数 */
                    (OS_STK *         )&START_TASK_STK[START_STK_SIZE - 1],
                    /* 任务堆栈栈顶 */
                    (INT8U            )START_TASK_PRIO,     /* 任务优先级 */
                    (INT16U           )START_TASK_PRIO,
                    /* 任务 ID，这里设置为和优先级一样 */
                    (OS_STK *         )&START_TASK_STK[0],  /* 任务堆栈栈底 */
                    (INT32U           )START_STK_SIZE,      /* 任务堆栈大小 */
                    (void *           )0,                   /* 用户补充的存储区 */
```

```
                    (INT16U          )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                     | OS_TASK_OPT_SAVE_FP);
                    /* 任务选项,为了保险起见，所有任务都保存浮点寄存器的值 */
    OSStart();  /* 开始任务 */

    for (;;)
    {
        /* 不会进入这里 */
    }
}

/**
 * @brief       start_task
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void start_task(void *p_arg)
{
    OS_CPU_SR cpu_sr = 0;
    CPU_INT32U cnts;

    msg_key = OSMboxCreate((void *)0);  /* 创建消息邮箱 */
    sem_beep = OSSemCreate(0);          /* 创建信号量 */

    OSStatInit();                       /* 开启统计任务 */
    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OS_TICKS_PER_SEC);
    OS_CPU_SysTickInit(cnts);

    OS_ENTER_CRITICAL();                /* 进入临界区(关闭中断) */

    /* 触摸任务 */
    OSTaskCreateExt((void(*)(void *) )touch_task,
                    (void *          )0,
                    (OS_STK *        )&TOUCH_TASK_STK[TOUCH_STK_SIZE - 1],
                    (INT8U           )TOUCH_TASK_PRIO,
                    (INT16U          )TOUCH_TASK_PRIO,
                    (OS_STK *        )&TOUCH_TASK_STK[0],
                    (INT32U          )TOUCH_STK_SIZE,
                    (void *          )0,
                    (INT16U          )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                     | OS_TASK_OPT_SAVE_FP);
    /* LED 任务 */
    OSTaskCreateExt((void(*)(void *) )led_task,
                    (void *          )0,
                    (OS_STK *        )&LED_TASK_STK[LED_STK_SIZE - 1],
                    (INT8U           )LED_TASK_PRIO,
                    (INT16U          )LED_TASK_PRIO,
                    (OS_STK *        )&LED_TASK_STK[0],
                    (INT32U          )LED_STK_SIZE,
                    (void *          )0,
                    (INT16U          )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                     | OS_TASK_OPT_SAVE_FP);
    /* 蜂鸣器任务 */
    OSTaskCreateExt((void(*)(void *) )beep_task,
                    (void *          )0,
                    (OS_STK *        )&BEEP_TASK_STK[BEEP_STK_SIZE - 1],
                    (INT8U           )BEEP_TASK_PRIO,
                    (INT16U          )BEEP_TASK_PRIO,
                    (OS_STK *        )&BEEP_TASK_STK[0],
                    (INT32U          )BEEP_STK_SIZE,
                    (void *          )0,
                    (INT16U          )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                     | OS_TASK_OPT_SAVE_FP);
```

```
    /* 主任务 */
    OSTaskCreateExt((void(*)(void *) )main_task,
                    (void *           )0,
                    (OS_STK *         )&MAIN_TASK_STK[MAIN_STK_SIZE - 1],
                    (INT8U            )MAIN_TASK_PRIO,
                    (INT16U           )MAIN_TASK_PRIO,
                    (OS_STK *         )&MAIN_TASK_STK[0],
                    (INT32U           )MAIN_STK_SIZE,
                    (void *           )0,
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                       | OS_TASK_OPT_SAVE_FP);
    /* 按键任务 */
    OSTaskCreateExt((void(*)(void *) )key_task,
                    (void *           )0,
                    (OS_STK *         )&KEY_TASK_STK[KEY_STK_SIZE - 1],
                    (INT8U            )KEY_TASK_PRIO,
                    (INT16U           )KEY_TASK_PRIO,
                    (OS_STK *         )&KEY_TASK_STK[0],
                    (INT32U           )KEY_STK_SIZE,
                    (void *           )0,
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                       | OS_TASK_OPT_SAVE_FP);
    OS_EXIT_CRITICAL();              /* 退出临界区(开中断) */
    OSTaskSuspend(START_TASK_PRIO); /* 挂起开始任务 */
}
/**
 * @brief       LED0 任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void led_task(void *p_arg)
{
    uint8_t t;

    while (1)
    {
        t++;
        OSTimeDly(10);

        if (t == 8)LED0(1); /* LED0 灭 */

        if (t == 100)       /* LED0 亮 */
        {
            t = 0;
            LED0(0);
        }
    }
}

/**
 * @brief       蜂鸣器任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void beep_task(void *p_arg)
{
    uint8_t err;

    while (1)
    {
        OSSemPend(sem_beep, 0, &err);  /* 请求信号量 */
        pcf8574_write_bit(BEEP_IO, 0); /* 打开蜂鸣器 */
        OSTimeDly(60);
        pcf8574_write_bit(BEEP_IO, 1); /* 关闭蜂鸣器 */
```

```
        OSTimeDly(940);
    }
}

/**
 * @brief       触摸屏任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void touch_task(void *p_arg)
{
    uint32_t cpu_sr;
    uint16_t lastpos[2];     /* 最后一次的数据 */

    while (1)
    {
        tp_dev.scan(0);

        if (tp_dev.sta & TP_PRES_DOWN)  /* 触摸屏被按下 */
        {
            if (tp_dev.x[0] < lcddev.width && tp_dev.y[0] < lcddev.height
                        tp_dev.y[0] > 120)
            {
                if (lastpos[0] == 0XFFFF)
                {
                    lastpos[0] = tp_dev.x[0];
                    lastpos[1] = tp_dev.y[0];
                }

                OS_ENTER_CRITICAL();
               /* 进入临界段,防止其他任务,打断 LCD 操作,导致液晶乱序 */
                lcd_draw_bline(lastpos[0],lastpos[1],tp_dev.x[0],tp_dev.y[0],
                               2, RED); /* 画线 */
                OS_EXIT_CRITICAL();
                lastpos[0] = tp_dev.x[0];
                lastpos[1] = tp_dev.y[0];
            }
        }
        else
        {
            lastpos[0] = 0XFFFF;
            OSTimeDly(10);    /* 没有按键按下的时候 */
        }
    }
}

/**
 * @brief       主任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void main_task(void *p_arg)
{
    uint32_t key = 0;
    uint8_t err;
    uint8_t semmask = 0;
    uint8_t tcnt = 0;

    while (1)
    {
        key = (uint32_t)OSMboxPend(msg_key, 10, &err);

        switch (key)
        {
            case KEY0_PRES: /* 控制 DS1,并清除触摸区域 */
```

```
                LED1_TOGGLE();
                lcd_fill(0, 121, lcddev.width - 1, lcddev.height - 1, WHITE);
                break;

            case KEY1_PRES: /* 发送信号量 */
                semmask = 1;
                OSSemPost(sem_beep);
                break;

            case WKUP_PRES:                         /* 校准 */
                OSTaskSuspend(TOUCH_TASK_PRIO); /* 挂起触摸屏任务 */

                if ((tp_dev.touchtype & 0X80) == 0)
                {
                    tp_adjust();
                }

                OSTaskResume(TOUCH_TASK_PRIO); /* 解挂 */
                ucos_load_main_ui();            /* 重新加载主界面 */
                break;
            }

            if (semmask || sem_beep->OSEventCnt) /* 需要显示 sem */
            {
                lcd_show_xnum(192, 50, sem_beep->OSEventCnt, 3, 16, 0X80,
                 BLUE);  /* 显示信号量的值 */

                if (sem_beep->OSEventCnt == 0)
                    semmask = 0; /* 停止更新 */
            }

            if (tcnt == 10) /* 0.6 秒更新一次 CPU 使用率 */
            {
                tcnt = 0;
                lcd_show_xnum(192, 30, OSCPUUsage, 3, 16, 0, BLUE);
               /* 显示 CPU 使用率 */
            }

        tcnt++;
        OSTimeDly(10);
    }
}

/**
 * @brief       按键扫描任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void key_task(void *p_arg)
{
    uint32_t key;

    while (1)
    {
        key = key_scan(0);

        if (key)OSMboxPost(msg_key, (void *)key); /* 发送消息 */

        OSTimeDly(10);
    }
}

/**
 * @brief       加载主界面
```

```
 * @param       无
 * @retval      无
 */
void ucos_load_main_ui(void)
{
    lcd_clear(WHITE);   /* 清屏 */
    lcd_show_string(30, 10, 200, 16, 16, "STM32", RED);
    lcd_show_string(30, 30, 200, 16, 16, "UCOSII TEST2", RED);
    lcd_show_string(30, 50, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(30, 75, 200, 16, 16, "KEY0:DS1 AND CLEAR", RED);
    lcd_show_string(30, 95, 200, 16, 16, "KEY1:BEEP  KEY_UP:ADJUST", RED);
    lcd_show_string(80, 210, 200, 16, 16, "Touch Area", RED);
    lcd_draw_line(0, 120, lcddev.width - 1, 120, RED);
    lcd_draw_line(0, 70, lcddev.width - 1, 70, RED);
    lcd_draw_line(150, 0, 150, 70, RED);

    lcd_show_string(160, 30, 200, 16, 16, "CPU:   %", BLUE);
    lcd_show_string(160, 50, 200, 16, 16, "SEM:000", BLUE);
}

/**
 * @brief       画粗线
 * @param       x1,y1: 起点坐标
 * @param       x2,y2: 终点坐标
 * @param       size : 线条粗细程度
 * @param       color: 线的颜色
 * @retval      无
 */
void lcd_draw_bline(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2,
                    uint8_t  size, uint16_t color)
{
    uint16_t t;
    int xerr = 0, yerr = 0, delta_x, delta_y, distance;
    int incx, incy, row, col;

    if (x1 < size || x2 < size || y1 < size || y2 < size)return;

    delta_x = x2 - x1;  /* 计算坐标增量 */
    delta_y = y2 - y1;
    row = x1;
    col = y1;

    if (delta_x > 0)incx = 1;       /* 设置单步方向 */
    else if (delta_x == 0)incx = 0; /* 垂直线 */
    else
    {
        incx = -1;
        delta_x = -delta_x;
    }

    if (delta_y > 0)
    {
        incy = 1;
    }
    else if (delta_y == 0)
    {
        incy = 0;   /* 水平线 */
    }
    else
    {
        incy = -1;
        delta_y = -delta_y;
    }
```

```
    if ( delta_x > delta_y)distance = delta_x; /* 选取基本增量坐标轴 */
    else distance = delta_y;

    for (t = 0; t <= distance + 1; t++ )     /* 画线输出 */
    {
        lcd_fill_circle(row, col, size, color); /* 画点 */
        xerr += delta_x ;
        yerr += delta_y ;

        if (xerr > distance)
        {
            xerr -= distance;
            row += incx;
        }

        if (yerr > distance)
        {
            yerr -= distance;
            col += incy;
        }
    }
}
```

上面就是对创建的 start_task、led0_task、touch_task、led1_task、main_task 和 key_task 等 6 个任务的参数进行配置，例如优先级、堆栈大小和任务函数的声明及定义。这 6 个任务做的事情在前面程序流程图里面已经有说明，这里就不多说了。

这一章的的运行流程比上一章复杂了一些，我们创建了消息邮箱 msg_key，用于按键任务和主任务之间的数据传输（传递键值）。另外创建了信号量 sem_led1，用于 LED 1 任务和主任务之间的通信。

在代码中，我们使用了 UCOSII 提供的 CPU 统计服务，通过 OSStatInit 初始化 CPU 统计任务，然后在主任务中显示 CPU 使用率。

另外，在主任务中，我们用到了任务的挂起和恢复函数，在执行触摸校准的时候，我们必须先将触摸屏任务挂起，待校准完成之后，再恢复触摸屏任务。这是因为触摸屏校准和触摸屏任务都用到了触摸屏和 TFTLCD，而这两个东西是不支持多任务占用的，所有必须采用独占的方式使用，否则可能导致数据错乱。

76.4 下载验证

将程序下载到开发板后，可以看到 LCD 显示界面如图 76.4.1 所示：

图 76.4.1 初始化界面

默认情况下，CPU 使用率仅为 1%。此时通过在触摸区域(Touch Area)画图，可以看到 CPU 使用率飙升，这说明触摸屏任务是一个很占 CPU 的任务；通过按 KEY0，可以控制 LED1 的亮灭；同时，可以在 LCD 上看到信号量的当前值；通过按下 KEY1 可以清屏；通过按下 WK_UP 可以进入校准程序，对触摸屏进行校准。

第七十七章 UCOSII 实验 3-消息队列、信号量集和软件定时器

上一章，我们学习了如何使用 UCOSII 的信号量和邮箱的使用，本章，我们将学习消息队列、信号量集和软件定时器的使用。

本章分为如下几个小节：

77.1 UCOSII 消息队列、信号量集和软件定时器简介
77.2 硬件设计
77.3 程序设计
77.4 下载验证

77.1 UCOSII 消息队列、信号量集和软件定时器简介

77.1.1 消息队列

消息队列可以视为消息邮箱的数组形式，消息邮箱一次传递一则消息，而消息队列可以在任务之间传递多条消息。消息队列的工作情况如图 77.1.1 所示：

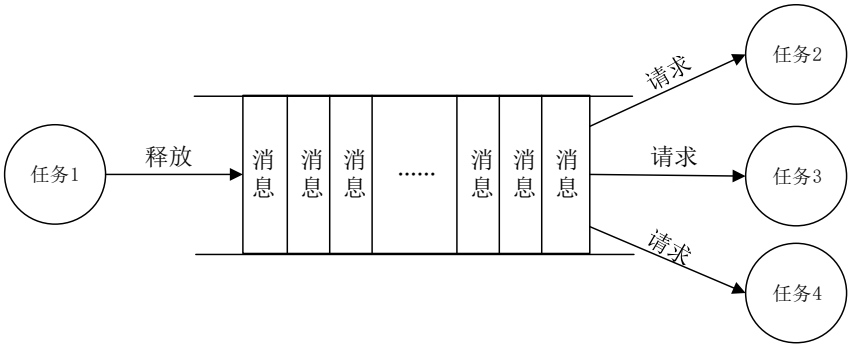


图 77.1.1.1 消息队列的工作情况图

从上图可知，任务可以向消息队列中释放消息，只有任务才能从消息队列中请求消息，任务可以始终请求消息，也可以周期性地请求消息。消息队列具有一定的长度，其长度可包含的消息个数，如果向队列中释放消息的速度大于从队列中请求消息的速度，那么消息队列将会溢出。消息队列的数据结构如图 77.1.2 所示：

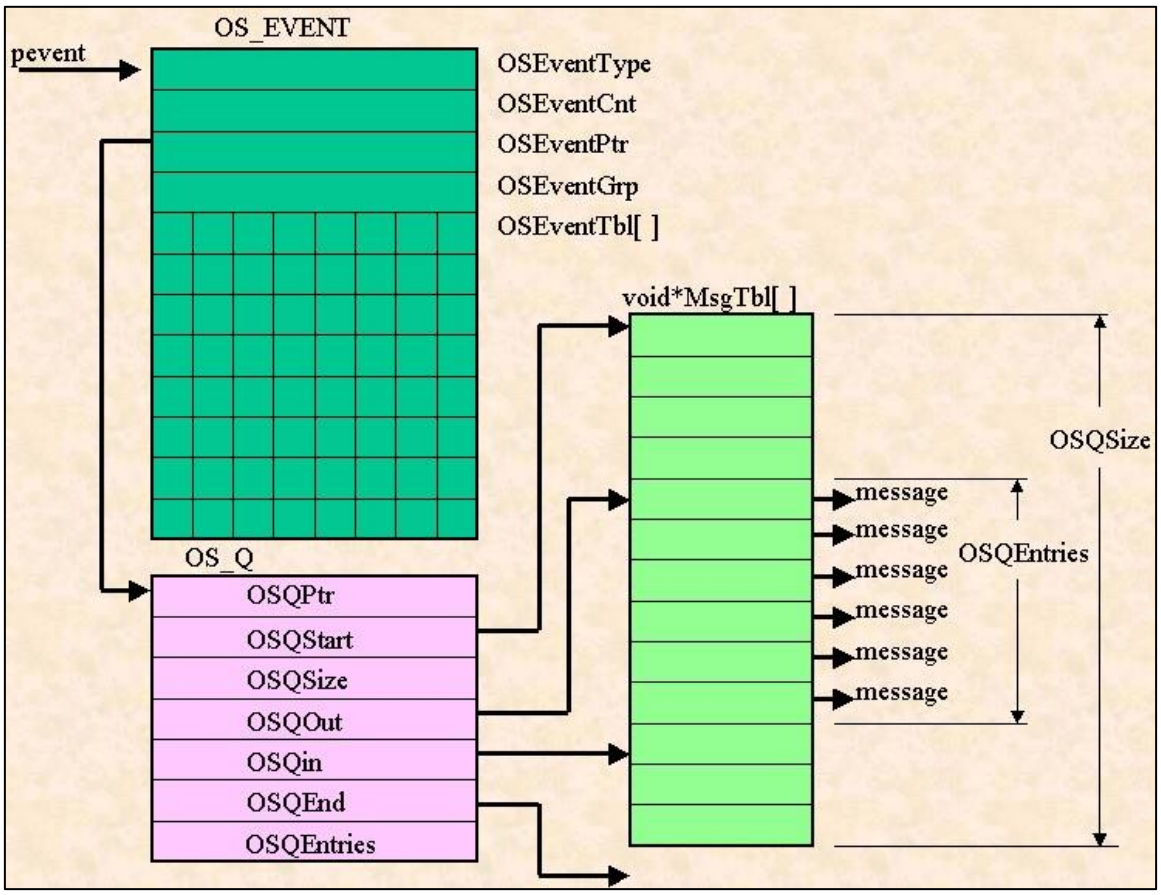


图 77.1.1.2 消息队列的数据结构图

消息队列由三个部分组成：事件控制块、消息队列和消息。当把事件控制块成员 OSEventType 的值设置为 OS_EVENT_TYPE_Q 时，该事件控制块描述的就是一个消息队列。

从上图可以知道，消息队列相当于一个任务等待列表的消息邮箱数组，事件控制块成员 OSEventPtr 指向了一个叫做队列控制块（OS_Q）的结构，该结构管理了一个数组 MsgTbl[]，该数组中的元素都是一些指向消息的指针。

队列控制块（OS_Q）的结构定义：

```
typedef struct os_q {                    /* 队列控制块 */
    struct os_q   *OSQPtr;               /* 指向下一个空的队列控制块 */
    void          **OSQStart;            /* 指向消息指针数组的起始地址 */
    void          **OSQEnd;              /* 指向消息指针数组结束单元的下一个单元 */
    void          **OSQIn;               /* 指向插入一条消息的位置 */
    void          **OSQOut;              /* 指向被取出消息的位置 */
    INT16U          OSQSize;             /* 数组的长度 */
    INT16U          OSQEntries;          /* 已存放消息指针的元素数目 */
} OS_Q;
```

其中，可以移动的指针为 OSQIn 和 OSQOut，而指针 OSQStart 和 OSQEnd 只是一个标志（常指针）。当可移动的指针 OSQIn 或 OSQOut 移动到数组末尾，也就是与 OSQEnd 相等时，可移动的指针将会被调整到数组的起始位置 OSQStart。也就是说，从效果上看，指针 OSQEnd 与 OSQStart 等值。于是，这个由消息指针构成的数组就头尾衔接起来形成了一个循环队列，如图 77.1.3 所示：

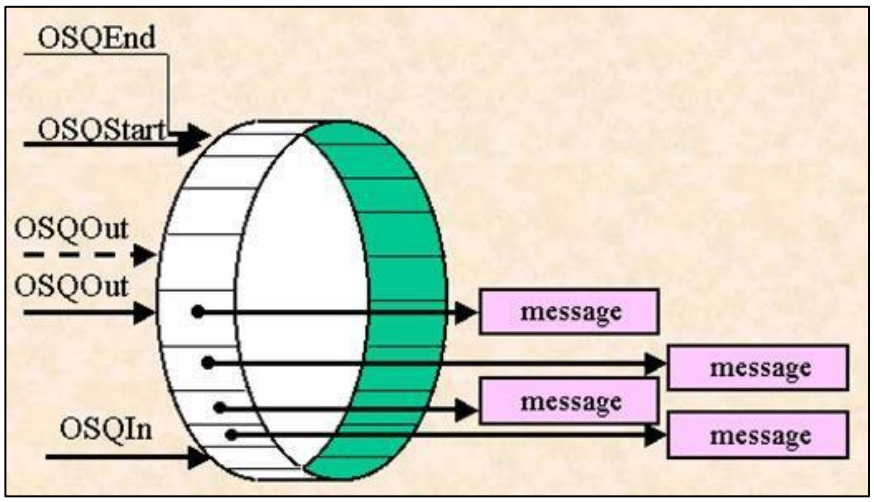


图 77.1.1.3 消息指针数组构成的环形数据缓冲区

在 UCOSII 初始化时，系统将按 os_cfg.h 文件中的 OS_MAX_QS 的数值定义 OS_MAX_QS 个队列控制块，并用队列控制块中的指针 OSQptr 将所有队列控制块链接为链表。由于这时候还没有使用它们，所以这个链表叫做空队列控制块链表。

消息队列相关的主要操作有：创建消息队列函数 OSQCreate、请求消息队列函数 OSQPend 和向消息队列发送消息函数 OSQPost。后面再对这几个函数进行讲解。

消息到这里就介绍完成了，想了解更多的朋友可以参考《嵌入式实时操作系统 UCOSII 原理及应用》第五章。

77.1.2 信号量集

在实际应用中，任务常常需要与多个事件同步，即要根据多个信号量组合作用的结果来决定任务的运行方式。UCOSII 为了实现多个信号量组合的功能定义了一种特殊的数据结构——信号量集。

信号量集所能管理的信号量都是一些二值信号，所有信号量集实质上是一种可以对多个输入的逻辑信号进行基本逻辑运算的组合逻辑，其示意图如图 77.1.2.1 所示：

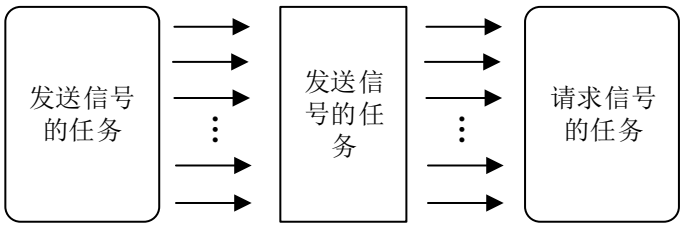


图 77.1.2.1 信号量集示意图

不同于信号量、消息邮箱、消息队列等事件，UCOSII 不使用事件控制块来描述信号量集，而使用了一个叫做标志组的结构 OS_FLAG_GRP 来描述。OS_FLAG_GRP 结构如下：

```
typedef struct os_flag_grp {                    /* 标志组 */
    INT8U          OSFlagType;                  /* 信号量集的标志 */
    void          *OSFlagWaitList;              /* 指向等待任务链表的指针 */
    OS_FLAGS       OSFlagFlags;                 /* 所有信号列表 */
#if OS_FLAG_NAME_EN > 0u
    INT8U         *OSFlagName;
#endif
} OS_FLAG_GRP;
```

成员 OSFlagFlags 是一个指针，当一个信号量集被创建后，这个指针指向了这个信号量集的等待任务链表。

与其他前面介绍过的事件不同，信号量集用一个双向链表来组织等待任务，每一个等待任务都是该链表中的一个节点（node）。标志组 OS_FLAG_GRP 的成员 OSFlagWaitList 就指向了信号量集的这个等待任务链表。等待任务链表节点 OS_FLAG_NODE 的结构如下：

```
typedef struct os_flag_node {                    /*  等待任务链表节点  */
    void          *OSFlagNodeNext;               /*  指向下一个节点的指针 */
    void          *OSFlagNodePrev;               /*  指向前一个节点的指针 */
    void          *OSFlagNodeTCB;                /*  指向对应任务控制块的指针 */
    void          *OSFlagNodeFlagGrp;            /*  反向指向信号量集的指针 */
    OS_FLAGS       OSFlagNodeFlags;              /*  信号过滤器 */
    INT8U          OSFlagNodeWaitType;           /*  定义逻辑运算关系的数据 */
} OS_FLAG_NODE;
```

其中 OSFlagNodeWaitType 是定义逻辑运算关系的一个常数（根据需要设置），其可选值和对应的逻辑关系如表 77.1.2.1 所示：

常数	信号有效状态	等待任务的就绪条件
WAIT_CLR_ALL 或 WAIT_CLR_AND	0	信号全部有效（全 0）
WAIT_CLR_ANY 或 WAIT_CLR_OR	0	信号有一个或一个以上有效（有 0）
WAIT_SET_ALL 或 WAIT_SET_AND	1	信号全部有效（全 1）
WAIT_SET_ANY 或 WAIT_SET_OR	1	信号有一个或者一个以上有效（有 1）

表 77.1.2.1 OSFlagNodeWaitType 可选值及其意义

OSFlagFlags、OSFlagNodeFlags、OSFlagNodeWaitType 三者的关系如图 77.1.2.2 所示：

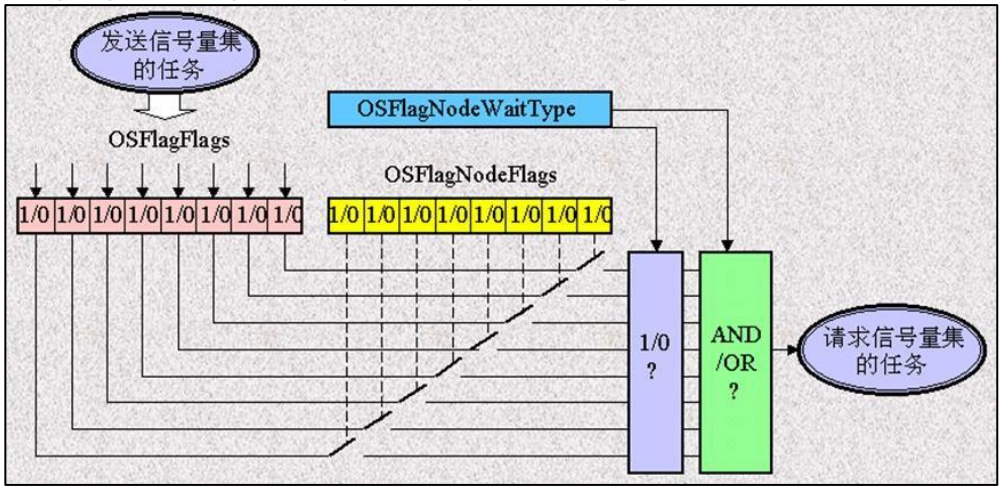


图 77.1.2.2 标志组与等待任务共同完成信号量集的逻辑运算及控制

为了方便说明，我们将 OSFlagFlags 定义为 8 位，但是 UCOSII 支持 8 位/16 位/32 位定义，这个通过修改 OS_FLAGS 的类型来确定（UCOSII 默认设置 OS_FLAGS 为 16 位）。

上图清楚表达了信号量集各成员的关系：OSFlagFlags 位信号量表，通过发送信号量集任务设置；OSFlagNodeFlags 为信号滤波器，由请求信号量集的任务设置，用于选择性的挑选 OSFlagFlags 中的部分（或全部）位作为有效信号；OSFlagNodeWaitType 定义有效信号的逻辑运算关系，也是由请求信号量集的任务设置，用于选择有效的组合方式（0/1？与/或？）。

举个简单的例子，假设请求信号量集的任务设置 OSFlagNodeFlags 的值为 0x0F，同时设置 OSFlagNodeWaitType 的值为 WAIT_SET_ANY，那么只要 OSFlagFlags 的低四位的任何一位为 1，请求信号量集的任务将得到有效的请求，从而执行相关操作；如果第四位都为 0，那么请求信号量集的任务将得到无效的请求。

信号量集相关的主要操作有：创建一个信号量集函数 OSFlagCreate，请求一个信号量集函数 OSFlagPend，向信号量集发送信号函数 OSFlagPost。后面再对这几个函数进行讲解。

信号量集就介绍到这里，更详细的介绍，请参考《嵌入式实时操作系统 UCOSII 原理及应用》第六章。

77.1.3 软件定时器

UCOSII 从 V2.83 版本以后，加入了软件定时器，这使得 UCOSII 的功能更加完善，在其上的应用程序开发与移植也更加方便。在实时操作系统中一个好的软件定时器实现要求有较高的精度、较小的处理器开销，且占用较少的储存器资源。

通过前面的学习，我们知道 UCOSII 通过 OSTimTick 函数对时钟节拍进行加 1 操作，同时遍历任务控制块，以判断任务延时是否到时。软件定时器同样由 OSTimTick 提供时钟，但是软件定时器的时钟还受 OS_TMR_CFG_TICKS_PER_SEC 设置的控制，也就是在 UCOSII 的时钟节拍上面在做了一次“分频”，软件定时器的最快时钟节拍就等于 UCOSII 的系统时钟节拍。这也决定了软件定时器的精度。

软件定时器定义了一个单独的计数器 OSTmrTime，用于软件定时器的计时，UCOSII 并不在 OSTimTick 中进行软件定时器的到时判断与处理，而是创建了一个高于应用程序中所有其他任务优先级的定时器管理任务 OSTmr_Task，在这个任务中进行定时器的到时判断和处理。时钟节拍函数通过信号量给这个高优先级任务发信号。这种方法缩短了中断服务程序的执行时间，但也使得定时器到时处理函数的响应收到中断推出时恢复现场和任务切换的影响。

UCOSII 中软件定时器的实现方法是，将定时器按定时事件分组，使得每次时钟节拍到来时只对部分定时器及逆行比较操作，缩短了每次处理的时间。但这就需要动态地维护一个定时器组。定时器组的维护只是在每次定时器到时的时候才发生，而且定时器从组中移除和再插入操作不需要排序。这是一种比较高效的算法，减少了维护所需的操作时间。

UCOSII 软件定时器实现了 3 类链表的维护：

```
OS_EXT  OS_TMR            OSTmrTbl[OS_TMR_CFG_MAX]; /* 定时器控制块数组 OS_EXT */
OS_EXT  OS_TMR           *OSTmrFreeList;            /* 空闲定时器控制块链表指针  */
OS_EXT  OS_TMR_WHEEL     OSTmrWheelTbl[OS_TMR_CFG_WHEEL_SIZE]; /* 定时器轮 */
```

其中 OS_TMR 为定时器控制块，定时器控制块是软件定时器管理的基本单元，包含软件定时器的名称、定时时间、在链表中的位置、使用状态、使用方式，以及到时回调函数及其参数等基本信息。

OSTmrTbl[OS_TMR_CFG_MAX]：以数组的形式静态分配定时器控制块所需的 RAM 空间，并存储所有已建立的定时器控制块，OS_TMR_CFG_MAX 为最大软件定时器的个数。

OSTmrFreeList：为空闲定时器控制块链表头指针。空闲态的定时器控制块(OS_TMR)中，OSTmrnext 和 OSTmrPrev 两个指针分别指向空闲控制块的前一个和后一个，组织了空闲控制块双向链表。建立定时器时，从这个链表中搜索空闲定时器控制块。

OSTmrWheelTbl[OS_TMR_CFG_WHEEL_SIZE]：该数组的每个元素都是已开启定时器的一个分组，元素中记录了指向该分组中第一个定时器控制块的指针，以及定时器控制块的个数。运行态的定时器控制块（OS_TMR）中，OSTmrnext 和 OSTmrPre 两个指针同样也组织了所在分组中定时器控制块的双向链表。软件定时器管理所需的数据结构示意图如图 77.1.3.1 所示：

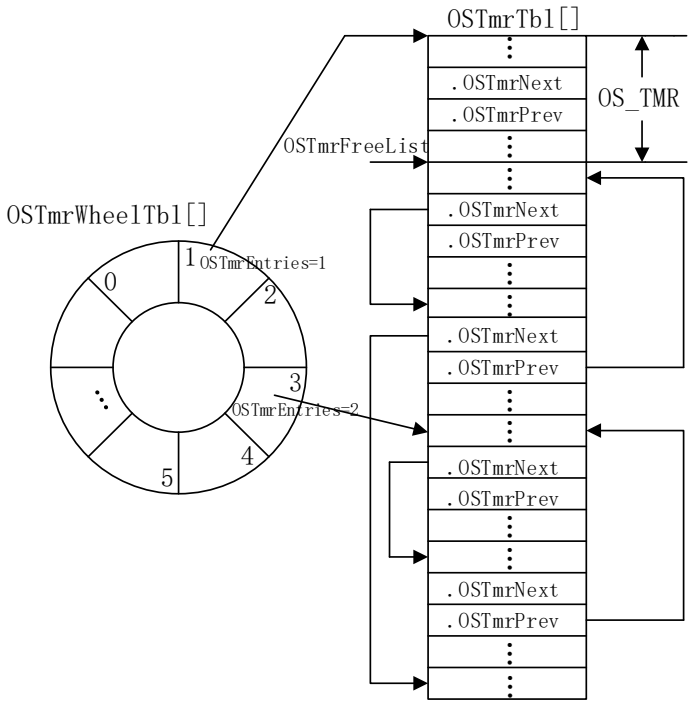


图 77.1.3.1 软件定时器管理所需的数据结构示意图

OS_TMR_CFG_WHEEL_SIZE 定义了 OSTmrWheelTbl 的大小，同时这个值也是定时器分组的依据。按照定时器到时值与 OS_TMR_CFG_WHEEL_SIZE 相除的余数进行分组：不同余数的定时器放在不同分组中；相同余数的定时器处在同一组中，由双向链表连接。这样，在余数值为 0～OS_TMR_CFG_WHEEL_SIZE - 1 的不同定时器控制块，正好分别对应了数组元素 OSTmrWheelTbl[0]～OSTmrWheelTbl[OS_TMR_CFGWHEEL_SIZE - 1]的不同分组。每次时钟节拍到来时，时钟数 OSTmrTime 值加 1，然后也进行求余操作，只有余数相同的那组定时器才有可能到时，所以只对该组定时器进行判断。这种方法比循环判断所有定时器更高效。随着时钟数的累加，处理的分组也由 0～OS_TMR_CFG_WHEEL_SIZE - 1 循环。这里，我们推荐 OS_TMR_CFG_WHEEL_SIZE 的取值为 2 的 N 次方，以便采用移位操作计算余数，缩短处理时间。

信号量唤醒定时器管理任务，计算出当前所要处理的分组后，程序遍历该分组中所有控制块，将当前 OSTmrTime 值与定时器控制块中的到时值（OSTmrMatch）相比较。若相等（即到时），则调用该定时器到时回调函数；若不相等，则判断该组中下一个定时器控制块。如此操作，直到该分组链表的结尾。软件定时器管理任务的流程如图 77.1.3.2 所示：

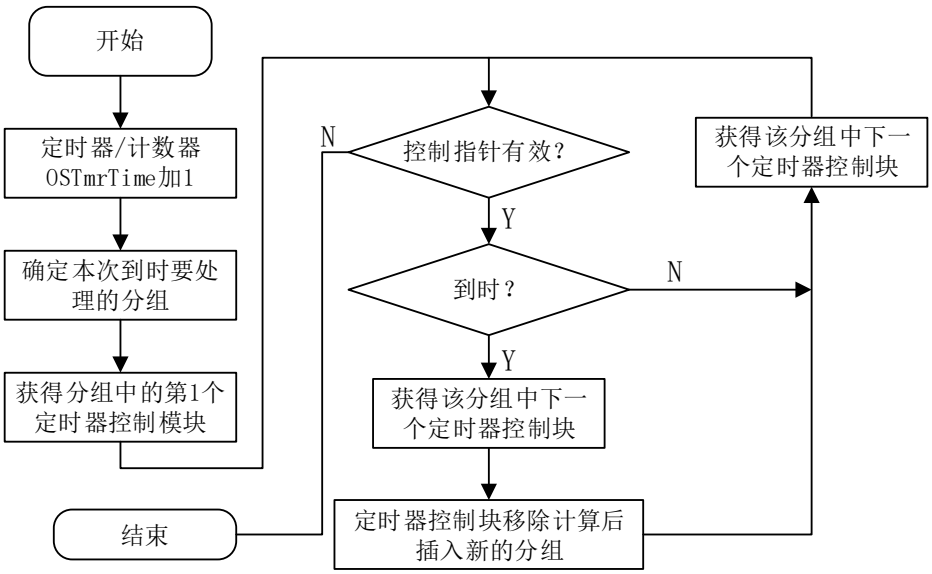


图 77.1.3.2 软件定时器管理任务流程

当运行完软件定时器的到时处理函数之后，需要进行该定时器控制块在链表中的移除和再插入操作。插入前需要重新计算定时器下次到时时所处的分组。计算公式如下：

定时器下次到时的OSTmrTime值(OSTmrMatch) = 定时器定时值 + 当前OSTmrTime值

新分组 = 定时器下次到时的 OSTmrTime 值(OSTmrMatch) % OS_TMR_CFG_WHEEL_SIZE

软件定时器相关的主要操作有：创建软件定时器函数 OSTmrCreate，开启软件定时器函数 OSTmrStart，停止软件定时器函数 OSTmrStop。

77.2 硬件设计

1. 例程功能

本实验我们在 UCOSII 里面创建 7 个任务：开始任务、LED 任务、触摸屏任务、队列消息显示任务、信号量集任务、按键扫描任务和主任务，开始任务用于创建邮箱、消息队列、信号量集以及其他任务，之后挂起；触摸屏任务用于在屏幕上画图，测试 CPU 使用率；队列消息显示任务请求消息队列，在得到消息后显示收到的消息数据；信号量集任务用于测试信号量集，采用 OS_FLAG_WAIT_SET_ANY 的方法，任何按键按下（包括 TPAD），该任务都会控制蜂鸣器发出“滴”的一声；按键扫描任务用于按键扫描，优先级最高，将得到的键值通过消息邮箱发送出去；主任务创建 3 个软件定时器（定时器 1，100ms 溢出一次，显示 CPU 和内存使用率；定时 2，200ms 溢出一次，在固定区域不停的显示不同颜色；定时 3，100ms 溢出一次，用于自动发送消息到消息队列），并通过查询消息邮箱获得键值，根据键值执行 LED1 控制、控制软件定时器 3 的开关、触摸区域清屏、触摸屏校和软件定时器 2 的开关控制等。

通过按 KEY0，可以启动/停止 tmr2，从而控制屏幕的刷新，还会清除 TOUCH 区域的输入。

通过按 KEY1 则可以启动 tmr3 控制消息队列发送，可以在 LCD 上面看到 Q 和 MEM 的值慢慢变大（说明队列消息在增多，占用内存也随着消息增多而增大），在 QUEUE MSG 区，开始显示队列消息，再按一次 KEY1 停止 tmr3，此时可以看到 Q 和 MEM 逐渐减小。当 Q 值变为 0 的时候，QUEUE MSG 也停止显示（队列为空）。

通过 KEY_UP 按键，可以进行触摸屏校准(仅电阻屏有效)。

2. 硬件资源

1）LED 灯

LED0 - PB1

LED1 - PB0

2）独立按键
KEY0 - PH3
KEY1 - PH2
WK_UP - PA0
3）正点原子 2.8/3.5/4.3/7/10 寸 TFTLCD 模块（包括 MCU 屏和 RGB 屏，都支持）
4）串口 1 (PA9/PA10 连接在板载 USB 转串口芯片 CH340 上面)
5）触摸屏(电阻式/电容式)

77.3 程序设计

77.3.1 UCOSII 相关函数介绍

消息队列函数

在这里对本实验用到的 UCOSII 消息队列函数进行介绍，相关代码存放在 os_q.c 中。

1. OSQCreate 函数

创建消息队列函数，其声明如下：

```
OS_EVENT  *OSQCreate (void **start, INT16U size)
```

- **函数描述：**
  用于创建一个消息队列
- **函数形参：**
  start：存放消息缓冲区指针数组的地址
  size：该数组的大小
- **函数返回值：**
  返回一个指向消息队列控制块的指针。如果没有空闲的控制块，返回空指针。

2. OSQPend 函数

请求消息队列函数，其声明如下：

```
void  *OSQPend (OS_EVENT  *pevent, INT32U timeout, INT8U *perr)
```

- **函数描述：**
  用于向消息队列请求消息。
- **函数形参：**
  pevent：要访问的消息队列事件控制块的指针
  timeout：等待时限
  perr：错误信息
  OS_ERR_NONE：调用成功，消息被正确地接受
  OS_ERR_TIMEOUT：消息没有在指定数目的时钟周期内被接收
  OS_ERR_PEND_ABOUT：取消队列等待
  OS_ERR_EVENT_TYPE：没有传递一个指向队列的指针
  OS_ERR_PEVENT_NULL：pevent 是一个空指针
  OS_ERR_PEND_ISR：从中断调用该函数，导致任务挂起
  OS_ERR_PEND_LOCKED：调度器上锁了
- **函数返回值：**
  无
- **注意事项：**
  如果队列中有消息，该消息传递给任务，并从队列中清除该消息；如果队列中没有消息，则调用该函数的任务被挂起等待，直到有消息或者等待超时。当有多个任务请求到同一消息队列时，UCOSII 进行任务调度，当前任务和所有请求该消息队列的任务中最高优先级的任务得到运行。

3. OSQPost 函数

向消息队列发送消息函数，其声明如下：

```
INT8U  OSQPost (OS_EVENT  *pevent, void *pmsg)
```

- **函数描述：**
  用于向消息队列发送消息
- **函数形参：**
  pevent：消息队列指针
  pmsg：消息指针
- **函数返回值：**
  OS_ERR_NONE：函数调用成功，成功发送消息到消息队列
  OS_ERR_Q_FULL：队列已满
  OS_ERR_EVENT_TYPE：pevent 不是指向消息队列的指针
  OS_ERR_PEVENT_NULL：pevent 是一个空指针

信号量集函数

在这里对本实验用到的 UCOSII 信号量集函数进行介绍，相关代码存放在 os_flag.c 中。

1. OSFlagCreate 函数

创建信号量集函数，其声明如下：

```
OS_FLAG_GRP  *OSFlagCreate (OS_FLAGS  flags, INT8U    *perr)
```

- **函数描述：**
  用于创建一个信号量集
- **函数形参：**
  flags：信号的初始值
  perr：错误信息
  OS_ERR_NONE：函数调用成功
  OS_ERR_CREATE_ISR：在中断中创建信号量集
  OS_ERR_FLAG_GRP_DEPLETED：没有更多的事件标志组
- **函数返回值：**
  这个信号量集的标志组的指针，应用程序可以用这个指针对信号量集进行相对应的操作。

2. OSFlagPend 函数

请求信号量集函数，其声明如下：

```
OS_FLAGS  OSFlagPend (OS_FLAG_GRP  *pgrp,
                      OS_FLAGS      flags,
                      INT8U         wait_type,
                      INT32U        timeout,
                      INT8U        *perr)
```

- **函数描述：**
  用于请求一个信号量集
- **函数形参：**
  pgrp：消息邮箱的指针
  flags ：消息指针
  wait_type：逻辑运算类型
  OS_FLAG_WAIT_CLR_ALL：等待“mask”中的所有位被清除
  OS_FLAG_WAIT_SET_ALL：等待“mask”中的所有位被设置
  OS_FLAG_WAIT_CLR_ANY：等待“mask”中的任何位被清除
  OS_FLAG_WAIT_SET_ANY：等待“mask”中的任何位被设置
  timeout：等待时延
  perr：错误信息
  OS_ERR_NONE：指定比特数已经被设置“超时”
  OS_ERR_PEND_ISR：从中断挂起
  OS_ERR_FLAG_INVALID_PGRP：pgrp 是一个空指针
  OS_ERR_TIMEOUT：指定比特数未设置“超时”
  OS_ERR_PEND_ABORT：取消标志等待
  OS_ERR_FLAG_WAIT_TYPE：没有指定正确的“wait_type”参数

- **函数返回值：**
  返回标志组成员 OSFlagFlags 的值

3. OSFlagPost 函数

向信号量集发送信号函数，其声明如下：

```
OS_FLAGS  OSFlagPost (OS_FLAG_GRP  *pgrp,
                      OS_FLAGS      flags,
                      INT8U         opt,
                      INT8U        *perr)
```

- **函数描述：**
  用于向信号量集发送信号
- **函数形参：**
  pgrp：信号量集指针
  flags：选择所要发送的信号
  opt：信号有效的选项
  perr：错误信息
    OS_ERR_NONE：函数调用成功
    OS_ERR_FLAG_INVALID_PGRP：pgrp 是一个空指针
    OS_ERR_EVENT_TYPE：没有指向时间标志组
    OS_ERR_FLAG_INVALID_OPT：使用无效选项
- **函数返回值：**
  返回标志组成员 OSFlagFlags 的值
- **注意事项：**
  所谓任务向信号量集发信号，就是对信号量集标志组中的信号进行置“1”(置位)或者置“0”(复位)的操作。至于对信号量几种的哪些信号进行操作，用函数中的参数 flags 来指定；对指定的信号是置“1”还是置“0”，用函数中的参数 opt 来指定(opt = OS_FLAG_SET 为置“1”操作；opt = OS_FLAG_CLR 为置“0”操作)。

软件定时器函数

在这里对本实验用到的 UCOSII 软件定时器函数进行介绍，相关代码存放在 os_tmr.c 中。

1. OSTmrCreate 函数

创建软件定时器函数，其声明如下：

```
OS_TMR  *OSTmrCreate (INT32U            dly,
                      INT32U            period,
                      INT8U             opt,
                      OS_TMR_CALLBACK   callback,
                      void             *callback_arg,
                      INT8U            *pname,
                      INT8U            *perr)
```

- **函数描述：**
  用于创建软件定时器
- **函数形参：**
  dly：用于初始化定时时间
  period：软件定时器的周期溢出时间
  opt：设置软件定时器工作模式
    OS_TMR_OPT_ONE_SHOT：计时器只倒计时一次
    OS_TMR_OPT_PERIODIC：定时器倒计时，然后重新加载
  callback_arg：回调函数的参数
  pname：软件定时器的名字
  perr：错误信息
    OS_ERR_NONE：函数调用成功
    OS_ERR_TMR_INVALID_DLY：执行无效延时操作
    OS_ERR_TMR_INVALID_PERIOD：执行操作时输入了不合法的时间

OS_ERR_TMR_INVALID_OPT：使用无效选项
OS_ERR_TMR_ISR：在中断中创建
OS_ERR_TMR_NON_AVAIL：定时器池中没有空闲的定时器

- **函数返回值：**
  返回定时器控制块
- **注意事项：**
  软件定时器的回调函数有固定格式，我们必须按照这个格式编写，软件定时器的回调函数格式为：void (*OS_TMR_CALLBACK)(void *ptmr, void *parg)。其中，函数名我们可以自己随意设置，而 ptmr 这个参数，软件定时器用来传递当前定时器的控制块指针，所以我们一般设置其类型为 OS_TMR*类型，第二个参数 parg 为回调函数的参数，这个就可以根据自己需要设置了，你也可以不用，但是必须有这个参数。

2. OSTmrStart 函数

开启软件定时器函数，其声明如下：

```
BOOLEAN  OSTmrStart (OS_TMR   *ptmr, INT8U  *perr)
```

- **函数描述：**
  用于开启某个软件定时器
- **函数形参：**
  ptmr：要开启的软件定时器指针
  perr：错误信息
  OS_ERR_NONE：函数调用成功
  OS_ERR_TMR_INVALID：定时器无效
  OS_ERR_TMR_INVALID_TYPE：ptmr 不是指向 OS_TMR
  OS_ERR_TMR_ISR：从中断中创建
  OS_ERR_TMR_INACTIVE：定时器没有被创建
  OS_ERR_TMR_INVALID_STATE：定时器处于无效状态
- **函数返回值：**
  返回布尔值 OS_TRUE：软件定时器开启　OS_FALSE：错误发生

3. OSTmrStop 函数

停止软件定时器函数，其声明如下：

```
BOOLEAN  OSTmrStop (OS_TMR  *ptmr,
                    INT8U    opt,
                    void    *callback_arg,
                    INT8U   *perr)
```

- **函数描述：**
  用于停止某个软件定时器
- **函数形参：**
  ptmr：要停止的软件定时器指针
  opt：停止选项
  OS_TMR_OPT_NONE，直接停止，不做任何其他处理
  OS_TMR_OPT_CALLBACK，停止，用初始化的参数执行一次回调函数
  OS_TMR_OPT_CALLBACK_ARG，停止，用新的参数执行一次回调函数
  callback_arg：新的回调函数参数
  perr：错误信息
  OS_ERR_NONE：函数调用成功
  OS_ERR_TMR_INVALID：定时器无效
  OS_ERR_TMR_INVALID_TYPE：ptmr 不是指向 OS_TMR
  OS_ERR_TMR_ISR：从中断中创建
  OS_ERR_TMR_INACTIVE：定时器没有被创建
  OS_ERR_TMR_INVALID_OPT：操作时使用无效选项
  OS_ERR_TMR_STOPPED：定时器已经停止
  OS_ERR_TMR_INVALID_STATE：定时器处于无效状态

OS_ERR_TMR_NO_CALLBACK：定时器没有定义回调函数

- **函数返回值：**
  返回布尔值 OS_TRUE：软件定时器停止　OS_FALSE：软件定时器停止失败

77.3.2 程序流程图

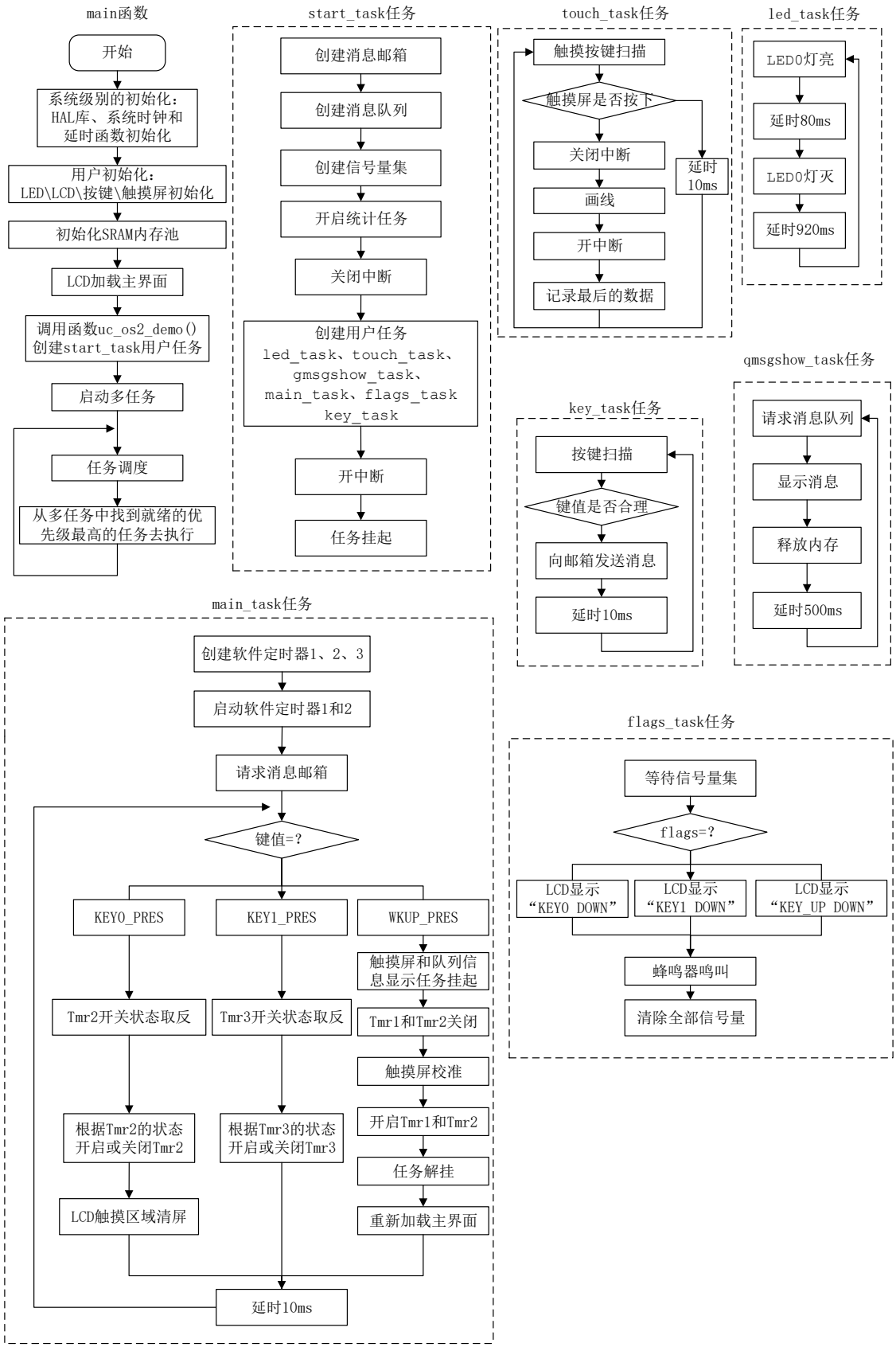


图 77.3.2.1 UCOSII 消息队列、信号量集和软件定时器实验

程序我们按流程图的设计来实现本节的功能代码。我们通过 start_task 创建其他任务，包括：

LED 任务，用于定时闪烁 LED 灯；
触摸任务，用于实现触摸画线功能；
按键扫描函数，根据按键发送不同的消息；
qmsgshow_task，请求并显示收到的队列消息
flag_task，用于请求和显示按键信号量集，获取完的信号于一段时间后从队列中清除；
main_task 用于创建定时器任务，并根据获取的队列消息得到按键值并投递给 flag_task 进行显示，根据按键值设置软件定时器 1，2，3 的状态，控制部分任务运行或挂起，重绘界面等。

77.3.3 程序解析

main.c 代码

在 main.c 文件下，只初始化一些外设，然后调用 ucosii 的例程入口函数 uc_os2_demo()，如下代码所示：

```
#include "./SYSTEM/sys/sys.h"
#include "./SYSTEM/usart/usart.h"
#include "./SYSTEM/delay/delay.h"
#include "./BSP/LED/led.h"
#include "./BSP/LCD/lcd.h"
#include "./BSP/KEY/key.h"
#include "./BSP/SDRAM/sdram.h"
#include "./BSP/MPU/mpu.h"
#include "./BSP/LCD/ltdc.h"
#include "./MALLOC/malloc.h"
#include "./BSP/TOUCH/touch.h"
#include "./BSP/PCF8574/pcf8574.h"
#include "uc-os2_demo.h"


int main(void)
{
    sys_cache_enable();                         /* 打开 L1-Cache */
    HAL_Init();                                 /* 初始化 HAL 库 */
    sys_stm32_clock_init(160, 5, 2, 4);         /* 设置时钟,400Mhz */
    usart_init(115200);                         /* 串口初始化 */
    delay_init(400);                            /* 延时初始化 */
    mpu_memory_protection();                    /* 保护相关存储区域 */
    sdram_init();                               /* 初始化 SDRAM */
    led_init();                                 /* 初始化 LED */
    lcd_init();                                 /* 初始化 LCD */
    key_init();                                 /* 初始化按键 */
    pcf8574_init();                             /* 初始化 PCF8574 */
    tp_dev.init();                              /* 触摸屏初始化 */

    my_mem_init(SRAMIN);                        /* 初始化内部 SRAM 内存池 */
    my_mem_init(SRAMEX);                        /* 初始化外部 SRAM 内存池 */
    my_mem_init(SRAM12);                        /* 初始化 SRAM12 内存池 */
    my_mem_init(SRAM4);                         /* 初始化 SRAM4 内存池 */
    my_mem_init(SRAMDTCM);                      /* 初始化 DTCM 内存池 */
    my_mem_init(SRAMITCM);                      /* 初始化 ITCM 内存池 */

    uc_os2_demo();                              /* 运行 uC/OS-II 例程 */
}
```

可以看到，在 main.c 文件中只包含了一个 main()函数，main()函数主要就是完成了一些外设的初始化，如串口、LED、LCD 等，并在最后调用了函数 uc_os2_demo()。

下面看一下 uc-os2_demo.c 的代码：

```
/* UCOSII 任务设置 */

/* START 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define START_TASK_PRIO                 10          /* 开始任务的优先级设置为最低 */
#define START_STK_SIZE                  128         /* 堆栈大小 */

OS_STK START_TASK_STK[START_STK_SIZE];              /* 任务堆栈 */
void start_task(void *pdata);                       /* 任务函数 */


/* 触摸屏任务 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define TOUCH_TASK_PRIO                 7           /* 优先级设置(越小优先级越高) */
#define TOUCH_STK_SIZE                  128         /* 堆栈大小 */

OS_STK TOUCH_TASK_STK[TOUCH_STK_SIZE];              /* 任务堆栈 */
void touch_task(void *pdata);                       /* 任务函数 */


/* LED 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define LED_TASK_PRIO                   6           /* 优先级设置(越小优先级越高) */
#define LED_STK_SIZE                    128         /* 堆栈大小 */

OS_STK LED_TASK_STK[LED_STK_SIZE];                  /* 任务堆栈 */
void led_task(void *pdata);                         /* 任务函数 */


/* 队列消息显示 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define QMSGSHOW_TASK_PRIO              5           /* 优先级设置(越小优先级越高) */
#define QMSGSHOW_STK_SIZE               128         /* 堆栈大小 */

OS_STK QMSGSHOW_TASK_STK[QMSGSHOW_STK_SIZE];        /* 任务堆栈 */
void qmsgshow_task(void *pdata);                    /* 任务函数 */


/* 主 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define MAIN_TASK_PRIO                  4           /* 优先级设置(越小优先级越高) */
#define MAIN_STK_SIZE                   512         /* 堆栈大小 */

OS_STK MAIN_TASK_STK[MAIN_STK_SIZE];                /* 任务堆栈 */
void main_task(void *pdata);                        /* 任务函数 */


/* 信号量集 任务 配置
 * 包括: 任务优先级 堆栈大小 等
 */
#define FLAGS_TASK_PRIO                 3           /* 优先级设置(越小优先级越高) */
#define FLAGS_STK_SIZE                  512         /* 堆栈大小 */

OS_STK FLAGS_TASK_STK[FLAGS_STK_SIZE];              /* 任务堆栈 */
void flags_task(void *pdata);                       /* 任务函数 */
```

```
/* 按键扫描 任务 配置
 * 包括：任务优先级 堆栈大小 等
 */
#define KEY_TASK_PRIO                   2       /* 优先级设置(越小优先级越高) */
#define KEY_STK_SIZE                    512     /* 堆栈大小 */

OS_STK KEY_TASK_STK[KEY_STK_SIZE];              /* 任务堆栈 */
void key_task(void *pdata);                     /* 任务函数 */


/*********************************************************************/
OS_EVENT *msg_key;          /* 按键邮箱事件块 */
OS_EVENT *q_msg;            /* 消息队列 */
OS_TMR    *tmr1;            /* 软件定时器 1 */
OS_TMR    *tmr2;            /* 软件定时器 2 */
OS_TMR    *tmr3;            /* 软件定时器 3 */
OS_FLAG_GRP *flags_key;     /* 按键信号量集 */

void *MsgGrp[256];          /* 消息队列存储地址,最大支持 256 个消息 */

/* 这些函数在 main 函数后面实现 */
void tmr1_callback(OS_TMR *ptmr, void *p_arg);
void tmr2_callback(OS_TMR *ptmr, void *p_arg);
void tmr3_callback(OS_TMR *ptmr, void *p_arg);

void ucos_load_main_ui(void);
void lcd_draw_bline(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2,
                    uint8_t size, uint16_t color);

/**
 * @brief       uC/OS-II 例程入口函数
 * @param       无
 * @retval      无
 */
void uc_os2_demo(void)
{
    ucos_load_main_ui();                                    /* 加载主界面 */

    OSInit();                                               /* UCOS 初始化 */
    OSTaskCreateExt((void(*)(void *) )start_task,           /* 任务函数 */
                    (void *           )0,                   /* 传递给任务函数的参数 */
                    (OS_STK *         )&START_TASK_STK[START_STK_SIZE - 1],
                    /* 任务堆栈栈顶 */
                    (INT8U            )START_TASK_PRIO,     /* 任务优先级 */
                    (INT16U           )START_TASK_PRIO,
                   /* 任务 ID，这里设置为和优先级一样 */
                    (OS_STK *         )&START_TASK_STK[0],/* 任务堆栈栈底 */
                    (INT32U           )START_STK_SIZE,      /* 任务堆栈大小 */
                    (void *           )0,                   /* 用户补充的存储区 */
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                      | OS_TASK_OPT_SAVE_FP);
                    /* 任务选项,为了保险起见，所有任务都保存浮点寄存器的值 */
    OSStart();  /* 开始任务 */

    for (;;)
    {
        /* 不会进入这里 */
    }
}

/**
```

```
 * @brief       开始任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void start_task(void *p_arg)
{
    uint8_t err;
    OS_CPU_SR cpu_sr = 0;
    CPU_INT32U cnts;
    /* 根据配置的节拍频率配置 SysTick */
    cnts = (CPU_INT32U)(HAL_RCC_GetSysClockFreq() / OS_TICKS_PER_SEC);
    OS_CPU_SysTickInit(cnts);

    msg_key = OSMboxCreate((void *)0);  /* 创建消息邮箱 */
    q_msg = OSQCreate(&MsgGrp[0], 256); /* 创建消息队列 */
    flags_key = OSFlagCreate(0, &err);  /* 创建信号量集 */
    OSStatInit();                       /* 开启统计任务 */
    OS_ENTER_CRITICAL();                /* 进入临界区(关闭中断) */

    /* LED 任务 */
    OSTaskCreateExt((void(*)(void *) )led_task,
                    (void *           )0,
                    (OS_STK *         )&LED_TASK_STK[LED_STK_SIZE - 1],
                    (INT8U            )LED_TASK_PRIO,
                    (INT16U           )LED_TASK_PRIO,
                    (OS_STK *         )&LED_TASK_STK[0],
                    (INT32U           )LED_STK_SIZE,
                    (void *           )0,
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                      | OS_TASK_OPT_SAVE_FP);
    /* 触摸任务 */
    OSTaskCreateExt((void(*)(void *) )touch_task,
                    (void *           )0,
                    (OS_STK *         )&TOUCH_TASK_STK[TOUCH_STK_SIZE - 1],
                    (INT8U            )TOUCH_TASK_PRIO,
                    (INT16U           )TOUCH_TASK_PRIO,
                    (OS_STK *         )&TOUCH_TASK_STK[0],
                    (INT32U           )TOUCH_STK_SIZE,
                    (void *           )0,
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                      | OS_TASK_OPT_SAVE_FP);
    /* 消息队列显示任务 */
    OSTaskCreateExt((void(*)(void *) )qmsgshow_task,
                    (void *           )0,
                    (OS_STK *         )&QMSGSHOW_TASK_STK[QMSGSHOW_STK_SIZE-1],
                    (INT8U            )QMSGSHOW_TASK_PRIO,
                    (INT16U           )QMSGSHOW_TASK_PRIO,
                    (OS_STK *         )&QMSGSHOW_TASK_STK[0],
                    (INT32U           )QMSGSHOW_STK_SIZE,
                    (void *           )0,
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                      | OS_TASK_OPT_SAVE_FP);
    /* 主任务 */
    OSTaskCreateExt((void(*)(void *) )main_task,
                    (void *           )0,
                    (OS_STK *         )&MAIN_TASK_STK[MAIN_STK_SIZE - 1],
                    (INT8U            )MAIN_TASK_PRIO,
                    (INT16U           )MAIN_TASK_PRIO,
                    (OS_STK *         )&MAIN_TASK_STK[0],
                    (INT32U           )MAIN_STK_SIZE,
                    (void *           )0,
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                      | OS_TASK_OPT_SAVE_FP);
    /* 信号量集任务 */
```

```
    OSTaskCreateExt((void(*)(void *) )flags_task,
                    (void *           )0,
                    (OS_STK *         )&FLAGS_TASK_STK[FLAGS_STK_SIZE - 1],
                    (INT8U            )FLAGS_TASK_PRIO,
                    (INT16U           )FLAGS_TASK_PRIO,
                    (OS_STK *         )&FLAGS_TASK_STK[0],
                    (INT32U           )FLAGS_STK_SIZE,
                    (void *           )0,
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                       | OS_TASK_OPT_SAVE_FP);
    /* 按键任务 */
    OSTaskCreateExt((void(*)(void *) )key_task,
                    (void *           )0,
                    (OS_STK *         )&KEY_TASK_STK[KEY_STK_SIZE - 1],
                    (INT8U            )KEY_TASK_PRIO,
                    (INT16U           )KEY_TASK_PRIO,
                    (OS_STK *         )&KEY_TASK_STK[0],
                    (INT32U           )KEY_STK_SIZE,
                    (void *           )0,
                    (INT16U           )OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR
                                       | OS_TASK_OPT_SAVE_FP);
    OS_EXIT_CRITICAL();              /* 退出临界区(开中断) */
    OSTaskSuspend(START_TASK_PRIO); /* 挂起开始任务 */
}
/**
 * @brief       LED0 任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void led_task(void *p_arg)
{
    uint8_t t;

    while (1)
    {
        t++;
        OSTimeDly(10);

        if (t == 8)LED0(1); /* LED0 灭 */

        if (t == 100)       /* LED0 亮 */
        {
            t = 0;
            LED0(0);
        }
    }
}


/**
 * @brief       触摸屏任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void touch_task(void *p_arg)
{
    uint32_t cpu_sr;
    uint16_t lastpos[2]; /* 最后一次的数据 */

    while (1)
    {
        tp_dev.scan(0);

        if (tp_dev.sta & TP_PRES_DOWN)  /* 触摸屏被按下 */
```

```
        {
            if (tp_dev.x[0] < (130 - 1) && tp_dev.y[0] < lcddev.height &&
                tp_dev.y[0] > (220 + 1))
            {
                if (lastpos[0] == 0XFFFF)
                {
                    lastpos[0] = tp_dev.x[0];
                    lastpos[1] = tp_dev.y[0];
                }

                OS_ENTER_CRITICAL();
                /* 进入临界段,防止其他任务,打断 LCD 操作,导致液晶乱序 */
                lcd_draw_bline(lastpos[0], lastpos[1], tp_dev.x[0],tp_dev.y[0],
                               2, RED);    /* 画线 */
                OS_EXIT_CRITICAL();
                lastpos[0] = tp_dev.x[0];
                lastpos[1] = tp_dev.y[0];
            }
        }
        else
        {
            lastpos[0] = 0XFFFF;
            OSTimeDly(10);    /* 没有按键按下的时候 */
        }
    }
}

/**
 * @brief       队列消息显示任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void qmsgshow_task(void *p_arg)
{
    char *p;
    uint8_t err;

    while (1)
    {
        p = OSQPend(q_msg, 0, &err);                    /* 请求消息队列 */
        lcd_show_string(5, 170, 240, 16, 16, p, RED); /* 显示消息 */
        myfree(SRAMIN, p);
        OSTimeDly(500);
    }
}

/**
 * @brief       主任务
 * @param       p_arg : 传入参数(未用到)
 * @retval      无
 */
void main_task(void *p_arg)
{
    uint32_t key = 0;
    uint8_t err;
    uint8_t tmr2sta = 1;     /* 软件定时器 2 开关状态 */
    uint8_t tmr3sta = 0;     /* 软件定时器 3 开关状态 */
    uint8_t flagsclrt = 0;   /* 信号量集显示清零倒计时 */
   tmr1 = OSTmrCreate(10, 10, OS_TMR_OPT_PERIODIC,
   (OS_TMR_CALLBACK)tmr1_callback, 0, (unsigned char*)"tmr1", &err);
    /* 100ms 执行一次 */
    tmr2 = OSTmrCreate(10, 20, OS_TMR_OPT_PERIODIC,
    (OS_TMR_CALLBACK)tmr2_callback, 0, (unsigned char*)"tmr2", &err);
    /* 200ms 执行一次 */
```

```
    tmr3 = OSTmrCreate(10, 10, OS_TMR_OPT_PERIODIC,
    (OS_TMR_CALLBACK)tmr3_callback, 0, (unsigned char*)"tmr3", &err);
    /* 100ms 执行一次 */
    OSTmrStart(tmr1, &err); /* 启动软件定时器 1 */
    OSTmrStart(tmr2, &err); /* 启动软件定时器 2 */

    while (1)
    {
        key = (uint32_t)OSMboxPend(msg_key, 10, &err);

        if (key)
        {
            flagsclrt = 51; /* 500ms 后清除 */
            OSFlagPost(flags_key, 1 << (key - 1), OS_FLAG_SET, &err);
            /* 设置对应的信号量为 1 */
        }

        if (flagsclrt)   /* 倒计时 */
        {
            flagsclrt--;

            if (flagsclrt == 1)lcd_fill(140, 162, 239, 162 + 16, WHITE);
            /* 清除显示 */
        }

        switch (key)
        {
            case KEY0_PRES:/* 软件定时器 2 开关,并清屏 */
                tmr2sta = !tmr2sta;

                if (tmr2sta)
                {
                    OSTmrStart(tmr2, &err); /* 开启软件定时器 2 */
                }
                else
                {
                    OSTmrStop(tmr2, OS_TMR_OPT_NONE, 0, &err);
                    /* 关闭软件定时器 2 */
                    lcd_show_string(148, 262, 240, 16, 16, "TMR2 STOP", RED);
                    /* 提示定时器 2 关闭了 */
                }

                lcd_fill(0, 221, 129, lcddev.height - 1, WHITE); /* 顺便清屏 */
                break;

            case KEY1_PRES:/* 控制软件定时器 3 */
                tmr3sta = !tmr3sta;

                if (tmr3sta)
                {
                    OSTmrStart(tmr3, &err);
                }
                else
                {
                    OSTmrStop(tmr3, OS_TMR_OPT_NONE,0,&err);/* 关闭软件定时器 3 */
                }

                break;

            case WKUP_PRES:/* 校准 */
                OSTaskSuspend(TOUCH_TASK_PRIO);         /* 挂起触摸屏任务 */
                OSTaskSuspend(QMSGSHOW_TASK_PRIO);      /* 挂起队列信息显示任务 */
                OSTmrStop(tmr1, OS_TMR_OPT_NONE, 0, &err);  /* 关闭软件定时器 1 */
```

```
                if (tmr2sta)OSTmrStop(tmr2, OS_TMR_OPT_NONE, 0, &err);
                /* 关闭软件定时器 2 */

                if ((tp_dev.touchtype & 0X80) == 0)
                {
                    tp_adjust();
                }

                OSTmrStart(tmr1, &err);                 /* 重新开启软件定时器 1 */

                if (tmr2sta)OSTmrStart(tmr2, &err);     /* 重新开启软件定时器 2 */

                OSTaskResume(TOUCH_TASK_PRIO);          /* 解挂 */
                OSTaskResume(QMSGSHOW_TASK_PRIO);       /* 解挂 */
                ucos_load_main_ui();                    /* 重新加载主界面 */
                break;
            }

            OSTimeDly(10);
        }
    }

    /**
     * @brief       信号量集处理任务
     * @param       p_arg : 传入参数(未用到)
     * @retval      无
     */
    void flags_task(void *p_arg)
    {
        uint16_t flags;
        uint8_t err;

        while (1)
        {
            flags = OSFlagPend(flags_key, 0X0007, OS_FLAG_WAIT_SET_ANY, 0, &err);
            /* 等待信号量 */

            if (flags&0X0001)lcd_show_string(140, 162, 240,16,16,"KEY0 DOWN", RED);

            if (flags&0X0002)lcd_show_string(140, 162, 240,16,16,"KEY1 DOWN", RED);

            if (flags&0X0004)lcd_show_string(140,162,240,16,16,"KEY_2 DOWN", RED);

            pcf8574_write_bit(BEEP_IO, 0); /* 打开蜂鸣器 */
            OSTimeDly(50);
            pcf8574_write_bit(BEEP_IO, 1); /* 关闭蜂鸣器 */
            OSFlagPost(flags_key, 0X0007, OS_FLAG_CLR, &err); /* 全部信号量清零 */
        }
    }

    /**
     * @brief       按键扫描任务
     * @param       p_arg : 传入参数(未用到)
     * @retval      无
     */
    void key_task(void *p_arg)
    {
        uint32_t key;

        while (1)
        {
            key = key_scan(0);

            if (key)OSMboxPost(msg_key, (void *)key); /* 发送消息 */
```

```
        OSTimeDly(10);
    }
}

/**
 * @brief       软件定时器 1 的回调函数
 * @note        每 100ms 执行一次,用于显示 CPU 使用率和内存使用率
 * @param       ptmr : 软件定时器指针
 * @param       p_arg: 参数指针(未用到)
 * @retval      无
 */
void tmr1_callback(OS_TMR *ptmr, void *p_arg)
{
    static uint16_t cpuusage = 0;
    static uint8_t tcnt = 0;

    if (tcnt == 5)
    {
        lcd_show_xnum(202, 10, cpuusage / 5, 3, 16, 0, BLUE);/* 显示 CPU 使用率 */
        cpuusage = 0;
        tcnt = 0;
    }

    cpuusage += OSCPUUsage;
    tcnt++;
    lcd_show_xnum(202, 30, my_mem_perused(SRAMIN) / 10, 3, 16, 0, BLUE);
     /* 显示内存使用率 */
    lcd_show_xnum(202, 50, ((OS_Q *)(q_msg->OSEventPtr))->OSQEntries, 3, 16,
                  0X80, BLUE); /* 显示队列当前的大小 */
}

/**
 * @brief       软件定时器 2 的回调函数
 * @note        每 200ms 执行一次
 * @param       ptmr : 软件定时器指针
 * @param       p_arg: 参数指针(未用到)
 * @retval      无
 */
void tmr2_callback(OS_TMR *ptmr, void *p_arg)
{
    static uint8_t sta = 0;

    switch (sta)
    {
        case 0:
            lcd_fill(131, 221, lcddev.width - 1, lcddev.height - 1, RED);
            break;

        case 1:
            lcd_fill(131, 221, lcddev.width - 1, lcddev.height - 1, GREEN);
            break;

        case 2:
            lcd_fill(131, 221, lcddev.width - 1, lcddev.height - 1, BLUE);
            break;

        case 3:
            lcd_fill(131, 221, lcddev.width - 1, lcddev.height - 1, MAGENTA);
            break;

        case 4:
            lcd_fill(131, 221, lcddev.width - 1, lcddev.height - 1, CYAN);
            break;
```

```
        case 5:
            lcd_fill(131, 221, lcddev.width - 1, lcddev.height - 1, YELLOW);
            break;

        case 6:
            lcd_fill(131, 221, lcddev.width - 1, lcddev.height - 1, BRRED);
            break;
    }

    sta++;

    if (sta > 6)sta = 0;
}

/**
 * @brief       软件定时器 3 的回调函数
 * @note        每 300ms 执行一次
 * @param       ptmr : 软件定时器指针
 * @param       p_arg: 参数指针(未用到)
 * @retval      无
 */
void tmr3_callback(OS_TMR *ptmr, void *p_arg)
{
    uint8_t *p;
    uint8_t err;
    static uint8_t msg_cnt = 0; /* msg 编号 */
    p = mymalloc(SRAMIN, 13);   /* 申请 13 个字节的内存 */

    if (p)
    {
        sprintf((char *)p, "ALIENTEK %03d", msg_cnt);
        msg_cnt++;
        err = OSQPost(q_msg, p);/* 发送队列 */

        if (err != OS_ERR_NONE) /* 发送失败 */
        {
            myfree(SRAMIN, p);  /* 释放内存 */
            OSTmrStop(tmr3, OS_TMR_OPT_NONE, 0, &err);  /* 关闭软件定时器 3 */
        }
    }
}

/**
 * @brief       加载主界面
 * @param       无
 * @retval      无
 */
void ucos_load_main_ui(void)
{
    lcd_clear(WHITE);   /* 清屏 */
    lcd_show_string(10, 10, 200, 16, 16, "STM32", RED);
    lcd_show_string(10, 30, 200, 16, 16, "UCOSII TEST3", RED);
    lcd_show_string(10, 50, 200, 16, 16, "ATOM@ALIENTEK", RED);
    lcd_show_string(10, 75, 240, 16, 16, "KEY0:TMR2 SW & CLR KEY1:Q SW", RED);
    lcd_show_string(10, 95, 240, 16, 16, "KEY_UP:ADJUST", RED);
    lcd_draw_line(0, 70, lcddev.width - 1, 70, RED);
    lcd_draw_line(150, 0, 150, 70, RED);

    lcd_draw_line(0, 120, lcddev.width - 1, 120, RED);
    lcd_draw_line(0, 220, lcddev.width - 1, 220, RED);
    lcd_draw_line(130, 120, 130, lcddev.height - 1, RED);

    lcd_show_string(5, 125, 240, 16, 16, "QUEUE MSG", RED);     /* 队列消息 */
```

```
    lcd_show_string(5, 150, 240, 16, 16, "Message:", RED);
    lcd_show_string(5 + 130, 125, 240, 16, 16, "FLAGS", RED);    /* 信号量集 */
    lcd_show_string(5, 225, 240, 16, 16, "TOUCH", RED);          /* 触摸屏 */
    lcd_show_string(5 + 130, 225, 240, 16, 16, "TMR2", RED);     /* 队列消息 */

    lcd_show_string(170, 10, 200, 16, 16, "CPU:   %", BLUE);
    lcd_show_string(170, 30, 200, 16, 16, "MEM:   %", BLUE);
    lcd_show_string(170, 50, 200, 16, 16, " Q :000", BLUE);

    OSTimeDly(300);
}

/**
 * @brief       画粗线
 * @param       x1,y1: 起点坐标
 * @param       x2,y2: 终点坐标
 * @param       size : 线条粗细程度
 * @param       color: 线的颜色
 * @retval      无
 */
void lcd_draw_bline(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t y2,
                    uint8_t size, uint16_t color)
{
    uint16_t t;
    int xerr = 0, yerr = 0, delta_x, delta_y, distance;
    int incx, incy, row, col;

    if (x1 < size || x2 < size || y1 < size || y2 < size)return;

    delta_x = x2 - x1;  /* 计算坐标增量 */
    delta_y = y2 - y1;
    row = x1;
    col = y1;

    if (delta_x > 0)incx = 1;       /* 设置单步方向 */
    else if (delta_x == 0)incx = 0; /* 垂直线 */
    else
    {
        incx = -1;
        delta_x = -delta_x;
    }

    if (delta_y > 0)
    {
        incy = 1;
    }
    else if (delta_y == 0)
    {
        incy = 0;   /* 水平线 */
    }
    else
    {
        incy = -1;
        delta_y = -delta_y;
    }

    if ( delta_x > delta_y)distance = delta_x;  /* 选取基本增量坐标轴 */
    else distance = delta_y;

    for (t = 0; t <= distance + 1; t++ )        /* 画线输出 */
    {
        lcd_fill_circle(row, col, size, color); /* 画点 */
        xerr += delta_x ;
        yerr += delta_y ;
```

```
        if (xerr > distance)
        {
            xerr -= distance;
            row += incx;
        }
        if (yerr > distance)
        {
            yerr -= distance;
            col += incy;
        }
    }
}
```

上面就是对创建的 start_task、led_task、touch_task、qmsgshow_task、main_task、flags_task 和 key_task 等 7 个任务的参数进行配置，例如优先级、堆栈大小和任务函数的声明及定义。此外还有 3 个软件定时器及其回调函数。

软件定时器 tmr1、tmr2 和 tmr3，tmr1 用于显示 CPU 使用率和内存使用率，每 100ms 执行一次；tmr2 用于在 LCD 的右下角区域不停的显示各种颜色，每 200ms 执行一次；tmr3 用于定时器向队列发送消息（用到了对台内存申请），每 100ms 发送一次。

在本实验中，我们还是使用消息邮箱 msg_key 在按键任务和主任务之间传递键值数据，我们创建信号量集 flags_key，在主任务里面将按键键值通过信号量集传递给信号量集处理任务 flags_task，实现按键信息的显示以及 LED1 的提示性闪灯。

此外我们还创建了一个大小为 256 的消息队列 q_msg，通过软件定时器 tmr3 的回调函数向消息队列发送消息，然后在消息队列显示任务 qmsgshow_task 里面请求消息队列，并在 LCD 上面显示得到的消息。消息队列还用到了动态内存管理。

在主任务 main_task 里面，我们实现了前面介绍的功能：KEY0 控制软件定时器 3 的开关，间接控制消息队列的发送；KEY1 控制软件定时器 2 的开关，同时清除 LCD 触摸屏区域的数据；WK_UP 用于触摸屏校准，在校准的时候，要先挂起触摸屏任务、队列消息显示任务，并停止软件定时器 tmr1 和 tmr2，否则可能对校准时的 LCD 显示造成干扰；

其他任务做的事情在前面程序流程图里面已经有说明，这里就不多说了。

77.4 下载验证

将程序下载到开发板后，可以看到 LCD 显示界面如图 77.4.1 所示：

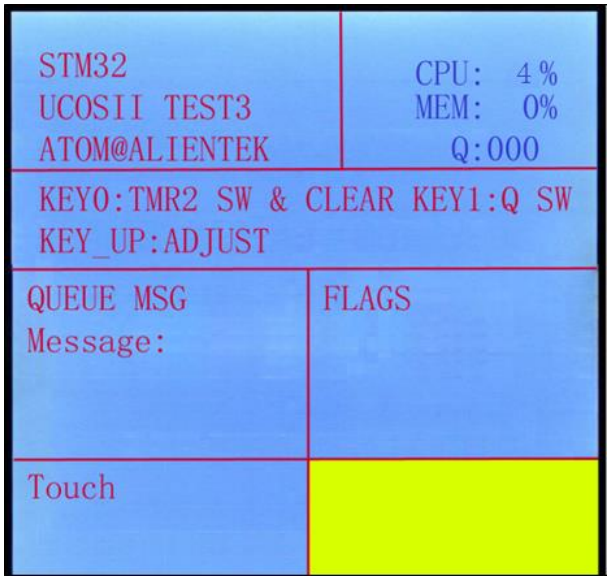


图 77.4.1 初始化界面

从图中可以看出，默认状态下，CPU 使用率为 4%左右。比上一章多出一些，这主要是 key_task 里面增加不停的刷屏（tmr2）操作导致的。

通过按 KEY0，可以启动/停止 tmr2，从而控制屏幕的刷新，同时清除 LCD 触摸屏区域数

据。

通过按 KEY1 则可以启动 tmr3 控制消息队列发送，可以在 LCD 上面看到 Q 和 MEM 的值慢慢变大（说明队列消息在增多，占用内存也随着消息增多而增大），在 QUEUE MSG 区，开始显示队列消息，再按一次 KEY1 停止 tmr3，此时可以看到 Q 和 MEM 逐渐减小。当 Q 值变为 0 的时候，QUEUE MSG 也停止显示（队列为空）。

通过 KEY_UP 按键，可以进行触摸屏校准(仅电阻屏，电容屏无需校准)。

在 TOUCH 区域，可以输入手写内容。

任何按键按下，蜂鸣器都会发出“滴”的一声，提示按键被按下，同时在 FLAGS 区域显示按键信息。

第七十八章 综合测试实验

为了方便大家使用和验证综合例程，本章内容是综合例程的使用介绍。目的是展示 STM32H7 的强大处理能力，并且可以测试开发板的大部分功能。本实验代码只提供寄存器版本，存放的路径是：阿波罗 V2 STM32H743 开发板资料盘(A 盘)→4，程序源码→1，标准例程-寄存器版本→实验 66 综合测试实验，代码非常多，这里不讲代码，只讲功能。

本章将分为如下 2 个小节：

78.1 综合测试实验简介

78.2 综合测试实验详解

78.1 综合测试实验简介

阿波罗 V2 是正点原子阿波罗 V1 开发板的升级版本，板载资源更合理，设计更精湛，软件功能也更强大。

开发板的硬件介绍请参考《阿波罗 V2 STM32H743 硬件参考手册_V1.0.pdf》。为了展示 STM32H7 系列强大的处理能力，综合例程实现了 27 个界面功能，分别是：电子图书、数码相框、音乐播放、视频播放、时钟、系统设置、FC 游戏机、记事本、运行器、手写画笔、照相机、录音机、USB 连接、网络通信、无线传书、计算器、二维码、IP 摄像头、人脸识别、9D 测试、水平仪、蜂鸣器、按键测试、LED 测试、拨号、应用中心和电压表。

电子图书：支持.txt/.c/.h/.lrc 等 4 种格式的文件阅读。

数码相框：支持.bmp/.jpeg/.jpb/.gif 等 4 种格式的图片文件播放。

音乐播放，支持.mp3/.wav/.ape/.flac 等 4 种常见音频文件的播放，全部软解码实现。

视频播放，支持.avi 格式（MJPEG 编码）的视频播放(带音频)，也是软解码实现。

时钟：支持温度、时间、日期、星期的显示，同时具有指针式时钟显示。

系统设置：整个综合实验的设置。

FC 游戏机：支持绝大部分 NES 游戏(.nes)，支持 USB 手柄/键盘控制，带声音，超 Info NES。

记事本：可以实现文本（.txt/.c/.h/.lrc）记录编辑等功能，支持中英文输入，手写识别。

运行器：即 SRAM IAP 功能，支持.bin 文件的运行（文件大小+SRAM 大小≤120K）。

手写画笔：可以作画/对 bmp 图片进行编辑，支持画笔颜色/尺寸设置。

照相机：可以拍照（.bmp/.jpg 格式，需摄像头模块支持），支持成像效果设置。

录音机：支持 wav 文件格式的录音（8~48KHz/16 位立体声录音），支持 AGC 设置。

USB 连接：支持和电脑连接读写 SD 卡/SPI FLASH 的内容。

网络通信：LWIP，支持 10/100M 自适应，支持 DHCP，支持 UDP/TCP/Web Server 测试。

无线传书：通过无线模块，实现两个开发板之间的无线通信。

计算器：一个科学计算器，支持各种运算，精度为 12 位，支持科学计数法表示。

二维码，支持二维码生成&二维码/条形码识别（需摄像头支持）。

IP 摄像头，一个网络摄像头应用，配合上位机软件，可以实现网络视频监控（局域网）。

人脸识别，一个简单的图像识别应用，可以实现人脸识别（需摄像头支持）。

9D 测试，利用 QMI8658A 的姿态库实现姿态解算，实时显示俯仰角、航向角和横滚角。

水平仪，利用 QMI8658A，实现一个类似 IPHONE 里面的的水平仪功能。

蜂鸣器，一个蜂鸣器测试应用，可以测试蜂鸣器的好坏。

按键测试，一个按键测试应用，可以测试板载 4 个按键的好坏。

LED 测试，一个 LED 测试应用，可以测试板载 2 个 LED 的好坏。

拨号：支持拨打电话（需要 4G 模块支持）。

应用中心：可扩展 16 个应用程序，我们实现了其中 2 个（红外遥控&指南针），其他预留。

电压表：用于测量 0~3.3V 的直流电压源。

以上，就是综合实验的 27 个功能简介，涉及到的内容包括：GUI（正点原子编写，非

ucGUI）、UCOSII、内存管理、图片解码、音频解码、视频解码、文件系统、USB(主机&从机)、IAP、LWIP(TCP/UDP/Web Server)、磁力计(ST480MC)、六轴传感器（QMI8658A）、NES 模拟器、手写识别、人脸识别、二维码识别、汉字输入等非常多的内容。下面，我们将详细介绍这 27 个功能。

78.2 综合测试实验详解

要测试阿波罗 V2 STM32H743 开发板综合测试实验的全部功能，大家得自备 1 个 TF 卡、1 个 U 盘、1 根网线、1 个耳机（非必须）、1 个 DS18B20 温度传感器（非必须）、1 个 4G 模块、1 个正点原子 OV5640 摄像头模块等。不过，就算没有这些东西，综合实验还是可以正常运行的，只是有些限制而已，比如：不能保存新建的记事本、不能保存新建的画图、不能使用录音机功能、不能使用摄像头功能、不能拨号等。除了这几个，其他功能基本都可以正常运行。

预备知识：

1，系统支持：所有液晶屏模块（包括电阻/电容屏、包括 MCU/RGB 屏），全部自动识别。

2，系统针对不同分辨率的屏幕，不同界面，会采用不同的字体和图标，以达到最佳效果。

3，系统测试有可能需要比较大电流（4.3 屏、7 寸屏、网络、喇叭）供电，强烈建议使用外部电源供电。

4，系统要用到 USB 通信（接 U 盘/USB 手柄/USB 键盘），请将开发板 P9 端子的 D+和 D-分别连接到 PA12 和 PA11。

有了以上预备知识，我们先来看看阿波罗 V2 STM32H743 开发板综合测试实验的启动界面，启动界面如图 78.2.1 所示：

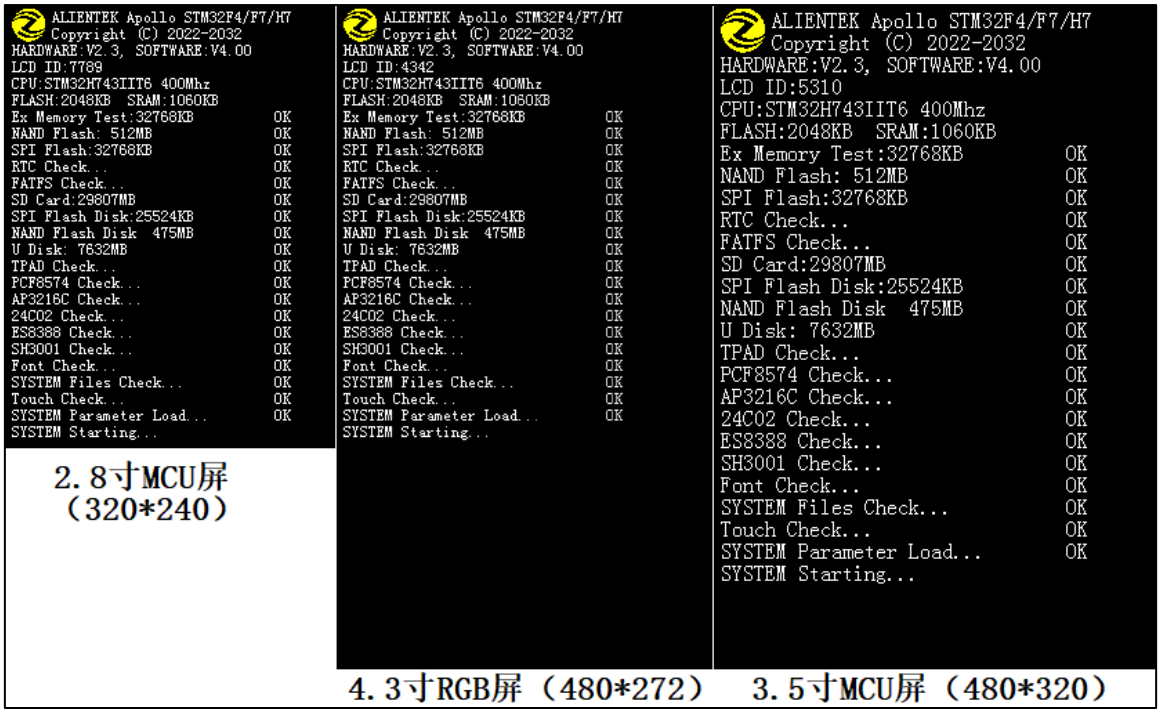


图 78.2.1 综合实验启动界面

注意：综合实验支持屏幕截图（通过 USMART 控制，波特率为 115200），本章所有图片均来自屏幕截图！

图 78.2.1 总共有 3 个截图拼成，分别代表 2.8 寸 MCU 屏、4.3 寸 RGB 屏和 3.5 寸 MCU 屏模块，显示内容都一样，但是图标大小和文字大小各不相同。**注意：4.3 寸 MCU 屏和 7 寸 RGB 屏的启动界面和图 78.2.1 类似，这里未列出**。图片显示了综合实验的详细启动过程，首先显示了版权信息，软硬件版本，接着显示了 LCD 驱动器的型号（LCD ID），然后显示 CPU 和内存信息，之后显示 SPI FLASH 和 NAND FLASH 的大小，接着开始初始化 RTC 和文件系统（FATFS），然后显示 SD 卡容量、FLASH Disk 容量（注意 FLASH Disk 就是指 SPI FLASH，因为我们划分了 25MB 空间给 FATFS 管理，所以 FLASH Disk 的容量显示为 25524KB）和 U 盘

容量。

接着，就是硬件检测，完了之后检测字库和系统文件，再初始化触摸屏，加载系统参数（参数保存在 24C02 里面），最后启动系统。在加载过程中，任何一个地方出错，都会显示相应的提示信息，请在检查无误后，按复位重启。

这里有几个注意的地方：

① 如果没插入 TF 卡/U 盘，其容量显示 0，并提示 ERROR，不过系统还是会继续启动，因为就算没有 TF 卡/U 盘系统还是可以启动的（前提是 SPI FLASH（25Q256）里面的系统文件和字库文件都是正常的）。

② 系统文件和字库文件都是存在 SPI FLASH(25Q256)里面的，如这些文件被破坏了，在启动的时候，会提示 **Font Error / SYSTEM File Error**。解决方法如下：

准备一个 TF/U 盘卡，并拷贝 SYSTEM 文件夹（**注意：这个 SYSTEM 文件夹不是开发板例程里的 SYSTEM 文件夹，而是光盘根目录→SD 卡根目录文件→SYSTEM** 文件夹）到 TF 卡根目录，然后 TF 卡插入开发板，按复位重启，然后开发板会自动更新文件。

③ SPI Flash Disk 是从 SPI FLASH（25Q256）里面分割 25MB 空间出来实现的，强制将 4K 字节的扇区改为 512 字节使用，所以在写操作的时候擦除次数会明显提升（8 倍以上），因此，如非必要，请不要往 SPI Flash Disk 里面写文件。频繁的写操作，很容易将 SPI Flash Disk 写挂掉。

④ 在系统启动时，一直按着 KEY0 不放（加载到 Touch Check 时），可以进入**强制校准**（仅电阻屏支持）。当你发现触摸屏不准的时候，可以使用这个办法强制校准。

⑤ 在系统启动时，一直按着 KEY1 不放（加载到 Font Check 时），可以强制更新字库。

⑥ 在系统启动时，一直按着 WK_UP 不放（加载到 FLASH 容量时），可以选择是否擦除所有文件（清空 SPI FLASH），当需要重新更新的时候，建议先用此方法擦除，再更新。

⑦ 本系统用到触摸按键 TPAD 做返回（类似手机的 HOME 键，**TPAD 在开发板右下角，‘Z’图标丝印，该区域是触摸按键区域!! 手指轻轻一摸，即可返回**），所以请确保多功能端口 P10 的 ADC 和 TPAD 用跳线帽短接！

在 SYSTEM Starting…之后，系统启动 UCOSII，并加载 SPB 界面，在加载成功之后，来到主界面，主界面如图 78.2.2~78.2.6 所示：

图 78.2.2 综合实验系统主界面(2.8 寸屏版本)

图 78.2.3 综合实验系统主界面(3.5 寸屏版本)

图 78.2.4 综合实验系统主界面(4.3 寸 RGB 屏版本)

图 78.2.5 综合实验系统主界面(4.3 寸 MCU 屏版本)

图 78.2.6 综合实验系统主界面(7 寸 RGB 屏版本)

从上面五张图可以看出，主界面有 3 个页面（滑动切换），总共是 27 个图标。每个图标代表一大功能，主界面顶部具有状态栏，显示 4G 模块信号质量、运营商、TF 卡状态、CPU 使用率和时间等信息。

注意：4G 模块信号质量和运营商，必须是接了 ATK-MW510 或者是 ATK-MW196 4G 模块后，才可能正常显示的，否则信号质量显示灰色，运营商显示：无移动网。

这里推荐大家使用性价比更高些的 ATK-MW196 4G 模块，模块与开发板的连接方式如下：

用杜邦线将开发板的 PB11(RX)接 4G 模块的 TXD 脚，将开发板的 PB10(TX)接 4G 模块的 RXD 脚。最后共地，开发板上随便找一个 GND 用杜邦线和 4G 模块的 GND 连接起来。

只有 4G 模块连接好，SIM 卡正常，且长按 4G 模块的 POWER 键开机后，开发板才会显

示如图 78.2.6 所示的信号质量和运营商，才可以进行拨号功能测试！！如果不显示信号质量和运营商，请检查是否有 4G 模块，或者 4G 模块是否工作正常！

回到主界面，主界面默认是简体中文的，我们可以在系统设置里面设置语言，综合测试实验支持 3 种语言选择：简体中文、繁体中文和英文。

在进入主界面之后，开发板上的 LED0 开始有规律的短亮（每 2.5 秒左右亮 100ms），提示系统运行正常，我们可以通过 LED0 判断系统的运行状况。另外，如果运行过程中，出现 HardFault 的情况，系统则会进入 HardFault 中断服务函数，此时 LED0 和 LED1 都会闪烁，提示系统故障。同时在串口打印故障信息。通过串口，系统会打印其他很多信息，最常打印的是内存使用率，然后我们还可以通过 USMART 对系统进行调试。

我们可以通过点击任何一个图标，选中，然后再次点击，即可进入该图标的功能。接下来，我们主要以 4.3 寸屏为例，给大家讲解综合实验。2.8、3.5、7 寸屏操作基本一模一样，下面就不再分别贴图了。

在任何界面下，都可以通过按 TPAD 返回上一级，直至返回到主界面。

上面已经介绍完系统启动，下面开始介绍各个功能。

78.2.1 电子图书

双击主界面的电子图书图标，进入如图 78.2.1.1 所示的文件浏览界面：

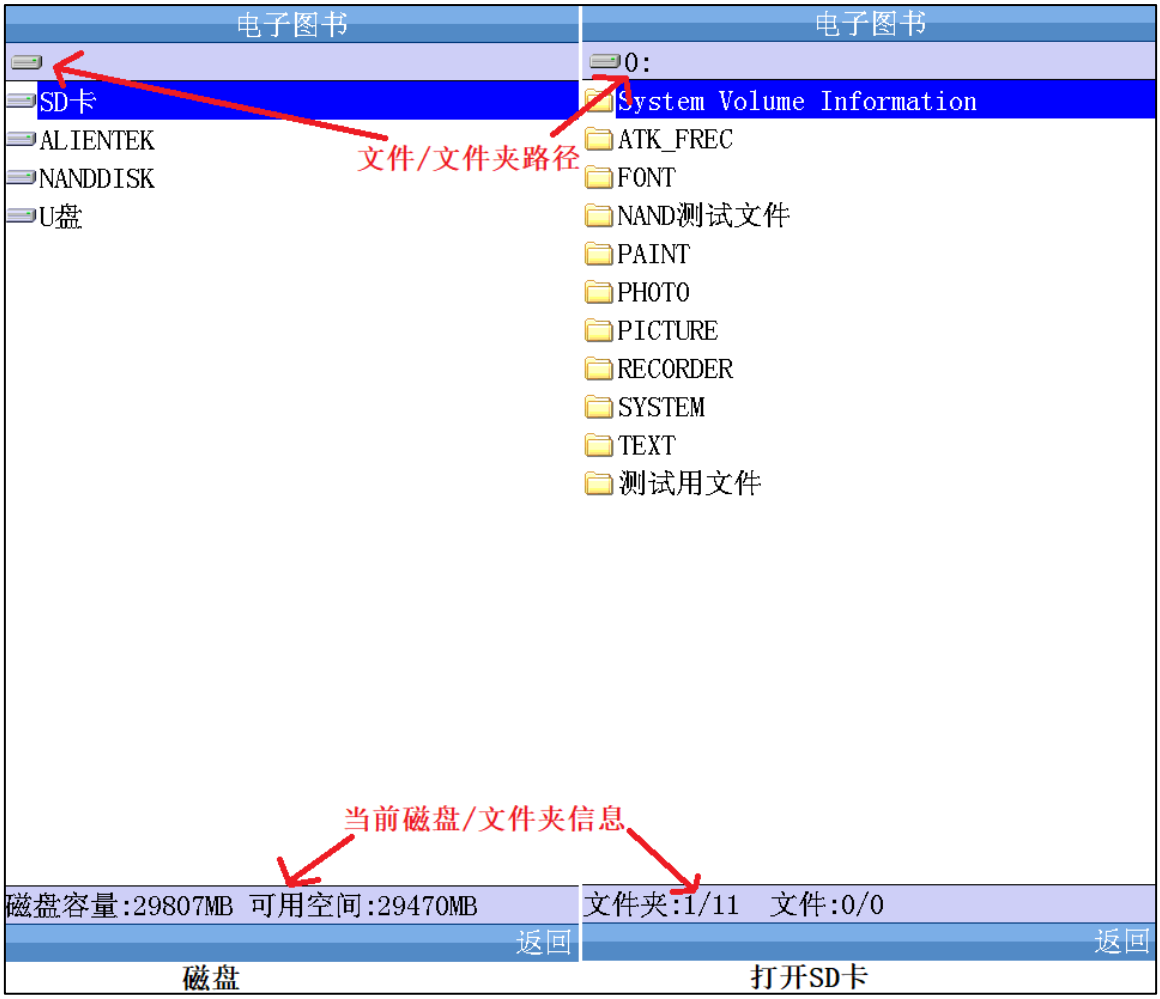


图 78.2.1.1 文件浏览界面

图 78.2.1.1 中，左侧的图是我们刚刚进入的时候看到的界面（类似在 WIN7/WIN10 上打开我的电脑），可以看到我们有 4 个盘，磁盘名字分别是：SD 卡、ALIENTEK、NANDDISK 和 U 盘。SD 卡是我们 TF 卡的卷标（即磁盘名字），ALIENTEK 是板载 SPI FLASH 磁盘的卷标，NANDDISK 是板载 NAND FLASH 磁盘的卷标，U 盘是我们插入的一个 U 盘的卷标。注意：如

果没有插入 TF 卡和 U 盘，则只会显示 ALIENTEK 这一个卷标。我们可以选择任何一个磁盘打开，并浏览里面的内容。

界面的上方，显示文件/文件夹的路径。如果当前路径是磁盘/磁盘根目录则显示磁盘图标，如果是文件夹，则显示文件夹图标，另外，如果路径太深，则只显示部分路径（其余用...代替）。

界面的下方，显示磁盘信息/当前文件夹信息。对磁盘，则显示当前选中磁盘的总容量和可用空间，对文件夹，则显示当前路径下文件夹总数和文件总数，并显示你当前选中的是第几个文件夹/文件。

双击图 78.2.1.1 中的“SD 卡”，打开 TF 卡，得到图 78.2.1.1 右侧图片所示的界面，选中 TEXT 文件夹，双击打开得到如图 78.2.1.2 所示界面：

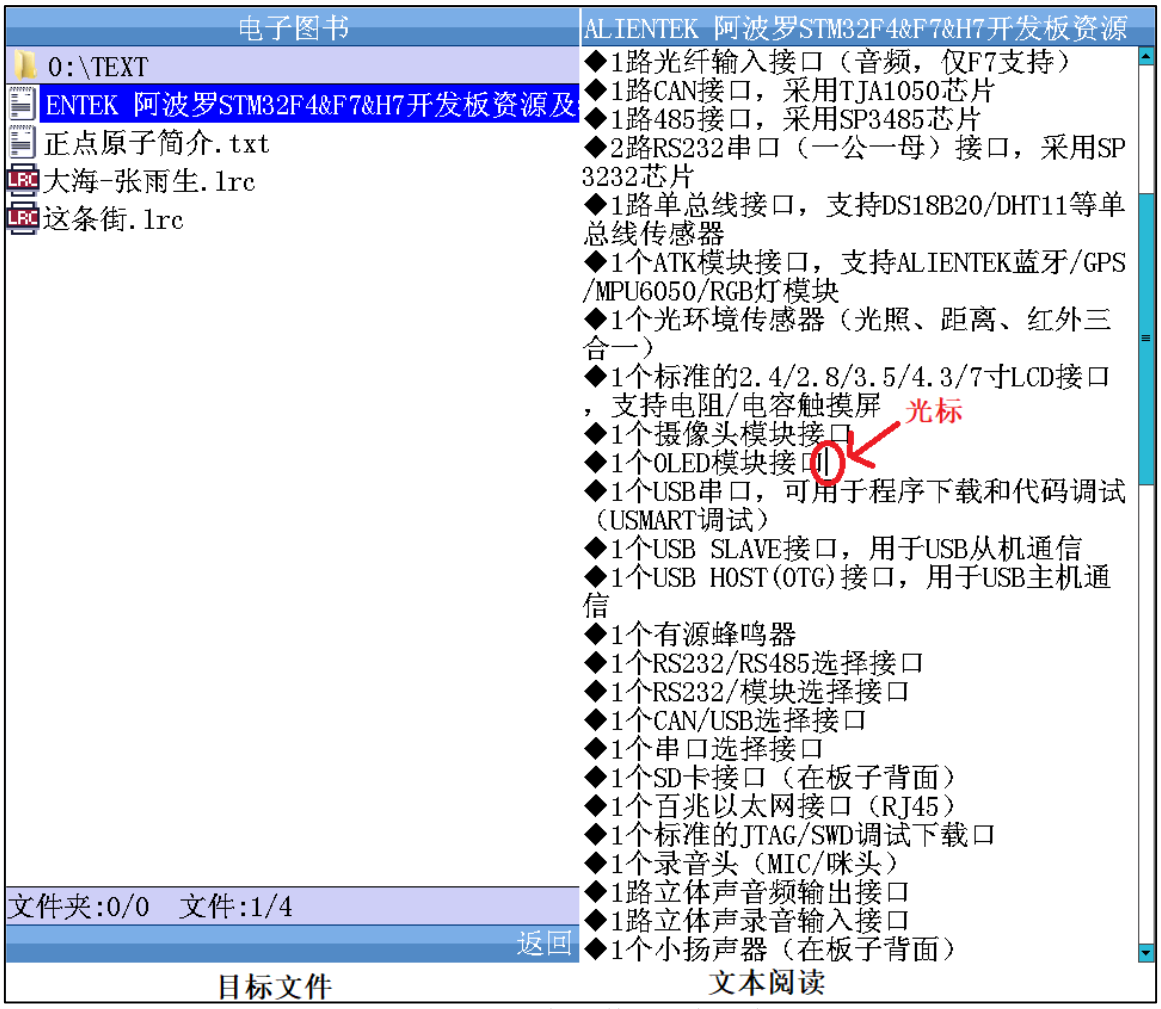


图 78.2.1.2 目标文件和文本阅读

图 78.2.1.2 左侧显示了当前文件夹下面的目标文件（即电子图书支持的文件，包括.txt/.h/.c/.lrc 等格式，其中.txt/.h/.c 文件共用 1 个图标，.lrc 文件单独一个图标）。另外，如果文件名太长，在我们选中该文件名后，系统会以走字的形式，显示整个文件名。

我们打开一个 txt 文件，开始文本阅读，如图 78.2.1.2 右侧的图片所示，同样我们可以通过滚动条/拖动的方式来浏览，图中我们还看到有一个光标，触摸屏点到哪，它就在哪里闪烁，可以方便大家阅读。

文本阅读是将整个文本文件加载到外部内存里面来实现的，所以文本文件最大不能超过外部内存总大小，即 28912KB（这里仅指受内存管理的部分，不是整个外部 SRAM 的大小）。

当我们想退出文本阅读的时候，通过按 TPAD 触摸按键实现，按一下 TPAD，则又回到查找目标文件状态（左侧图），按返回按钮可以返回上一层目录，如果再按一次 TPAD 则直接返回主界面。

78.2.2 数码相框

双击主界面的数码相框图标，进入文件浏览界面，这个和 78.2.1 节差不多，我们找到存放图片的文件夹，如图 78.2.2.1 所示：

图 78.2.2.1 文件浏览和图片播放

左侧是文件浏览的界面，可以看到在 PICTURE 文件夹下总共有 27 个文件，包括 gif/jpg/bmp 等，这些都是数码相框功能所支持的格式。右侧图片显示了一个正在播放的 GIF 图片，并在其左上角显示当前图片的名字。当然，我们也可以播放 bmp 和 jpg 文件，如图 78.2.2.2 所示：

图 78.2.2.2 bmp 和 jpg 图片播放

对于 bmp 和 jpg 文件，基本没有尺寸限制（但图片越大，解码时间越久），但是对于 gif 文件，则只支持尺寸在 LCD 分辨率以内的文件（因为 gif 图片我们不好做尺寸压缩处理），超过这个尺寸的 gif 图片将无法显示!!

我们可以通过点击屏幕的上方（1/3 屏幕）区域切换到上一张图片浏览；通过点击屏幕的下方（1/3 屏幕）区域切换到下一章图片；通过单击屏幕的中间（1/3 屏幕）区域可以暂停自动播放，同时 LED1 亮，提示正在暂停状态，同样，通过按 TPAD 按钮，可以返回文件浏览状态。

图片浏览支持两种自动播放模式：循环播放/随即播放。大家可以在系统设置里面设置图片播放模式。系统默认是循环播放模式，在该模式下，每隔 4 秒左右自动播放下一张图片，依次播放所有图片。而随机播放模式，也是每隔 4 秒左右自动播放下一张图片，但是不是顺序播放，而是随机的播放下一张图片。

78.2.3 音乐播放

综合测试实验的音乐播放器性能非常强悍，可作为 HIFI 播放器使用。支持常见的无损音乐(wav/flac/ape)播放，具体性能如下：

wav 文件：支持最高 192K@24bit 播放

mp3 文件：全码率支持

flac 文件：支持最高 192K@24bit 播放

ape 文件：最高支持 192K@16bit 播放

注意：如果是播放 U 盘的音乐，建议不要播放采样率太高的音频，否则可能导致播放不正常（建议在 48K 及以下比较好）。

双击主界面的音乐播放图标，进入文件浏览界面，这个和 78.2.1 节差不多，只是这里我们

浏览的文件变为了.wav/.mp3/.flac/.ape 等音频文件，我们找到存放音频文件的文件夹，如图 78.2.3.1 所示：

图 78.2.3.1 文件浏览和 wav 格式播放

左侧是文件浏览的界面，可以看到在音乐文件 文件夹下总共有 33 个音频文件，包括 wav/mp3/flac/ape 等格式，这些都是播放器所支持的格式。右侧图片则是我们播放器的主界面，该界面显示了当前播放歌曲的名字、播放进度、播放时长、总时长、采样率、位数、码率、音量、当前文件编号、总文件数、歌词等信息。下方的 5 个按键分别是：目录、上一曲、暂停/播放、下一曲、返回。点击播放进度条，可以直接设置歌曲播放位置（注意：ape 格式不支持），点击声音进度条，可以设置音量。上图为正在播放 wav 文件，当然我们还可以播放其他音频格式，如图 78.2.3.2 所示：

图 78.2.3.2 mp3 格式播放和 flac 格式播放

图 78.2.3.2 中，分别展示了播放 mp3 格式和 flac 格式的音频文件。

播放器还可以设置音效和播放模式（均在系统设置里面设置）。音效包括 3D 效果。播放模式有 3 种：全部循环、随机播放、单曲循环，默认为全部循环。

另外，关于歌词显示。歌词必须和歌曲在同一个文件夹里面，且名字必须相同（当然后缀是不同的，歌词后缀为.lrc）,这样才能正常显示歌词。对于没有歌词文件的歌曲，则直接播放，不显示歌词。歌词采用多行显示，中间为当前正在演唱的歌词（粉红色字体显示），上下分别有预览歌词（白色字体显示），如果正在演唱的歌词太长，则会采用走字的形式来显示，走字时间由系统自动确定。

我们可以通过按目录按键，来选择其他音频文件；按返回按键（或 TPAD）则可以返回主界面，不过此时正在播放的歌曲还是会继续播放（后台播放），如果想关闭音乐播放器，则需要先按暂停，然后返回主界面，即可关闭音频播放器，否则音频播放器将一直播放音乐。

本音乐播放器支持多种无损音频格式播放，前面介绍了 wav 和 flac。wav 和 flac 是支持 24bit 播放的，不过 ape 则只支持 16bit 播放。最后，看看 ape 文件的播放，如图 78.2.3.3 所示：

图 78.2.3.3 ape 格式播放

注意，需要外接耳机（插入 PHONE 端子）或者喇叭（接 SPK）才可以欣赏音乐哦!!! 耳机和喇叭需自备。

78.2.4 视频播放

开发板的综合实验支持视频播放（带声音），软解码 avi 文件，实现视频播放。支持的视频格式为：.avi，视频必须使用 MJPEG 压缩，音频采用线性 PCM 编码（无压缩）。视频分辨率必须小于等于屏幕分辨率，对于综合实验来说：2.8 寸屏，最大支持 240*164 分辨率的视频；3.5 寸屏，最大支持 320*296 分辨率的视频；4.3 寸屏，最大支持 480*550 分辨率的视频。**特别提醒：一般网络下载的视频文件（.avi/.rmvb/.mkv/.mp4 等），本播放器不支持，必须通过软件转换（狸窝全能视频转换器），才可以，详细转换方法，见第六十三章实验。**

双击主界面的视频播放图标，进入文件浏览界面，这个和 78.2.1 节差不多，只是这里我们浏览的文件变为了.avi 的视频文件，我们找到存放视频文件的文件夹，如图 78.2.4.1 所示：

图 78.2.4.1 文件浏览和 avi 视频播放

左侧是文件浏览的界面，可以看到在视频文件夹下总共有 21 个视频文件。右侧图片则是我们视频播放器的主界面，该界面显示了当前播放视频的名字、播放进度、播放时长、总时长、音频采样率、视频帧率、视频分辨率、音量、当前文件编号、总文件数等信息。下方的 5 个按键分别是：目录、上一个视频、暂停/播放、下一个视频、返回。点击视频播放进度条，可以直接设置视频播放位置，点击声音进度条，可以设置音量。

视频播放器还可以设置音效和播放模式（均在系统设置里面设置）。音效包括 3D 效果等设置。播放模式有 3 种：全部循环、随机播放、单曲循环，默认为全部循环。

我们可以通过按目录按钮，来选择其他视频文件；按返回按键（或 TPAD）则可以返回主界面。视频播放不支持后台播放，所以一旦退出到文件浏览或者主界面，则停止视频播放。

在图 78.2.4.1 中，右侧图片播放的是 480*272 的视频，帧率为 10 帧/秒。对于小分辨率的视频，帧率可以更快一些，比如 320*240 的可以去到 25 帧，240*160 的可以去到 30 帧，如图

78.2.4.2 所示：

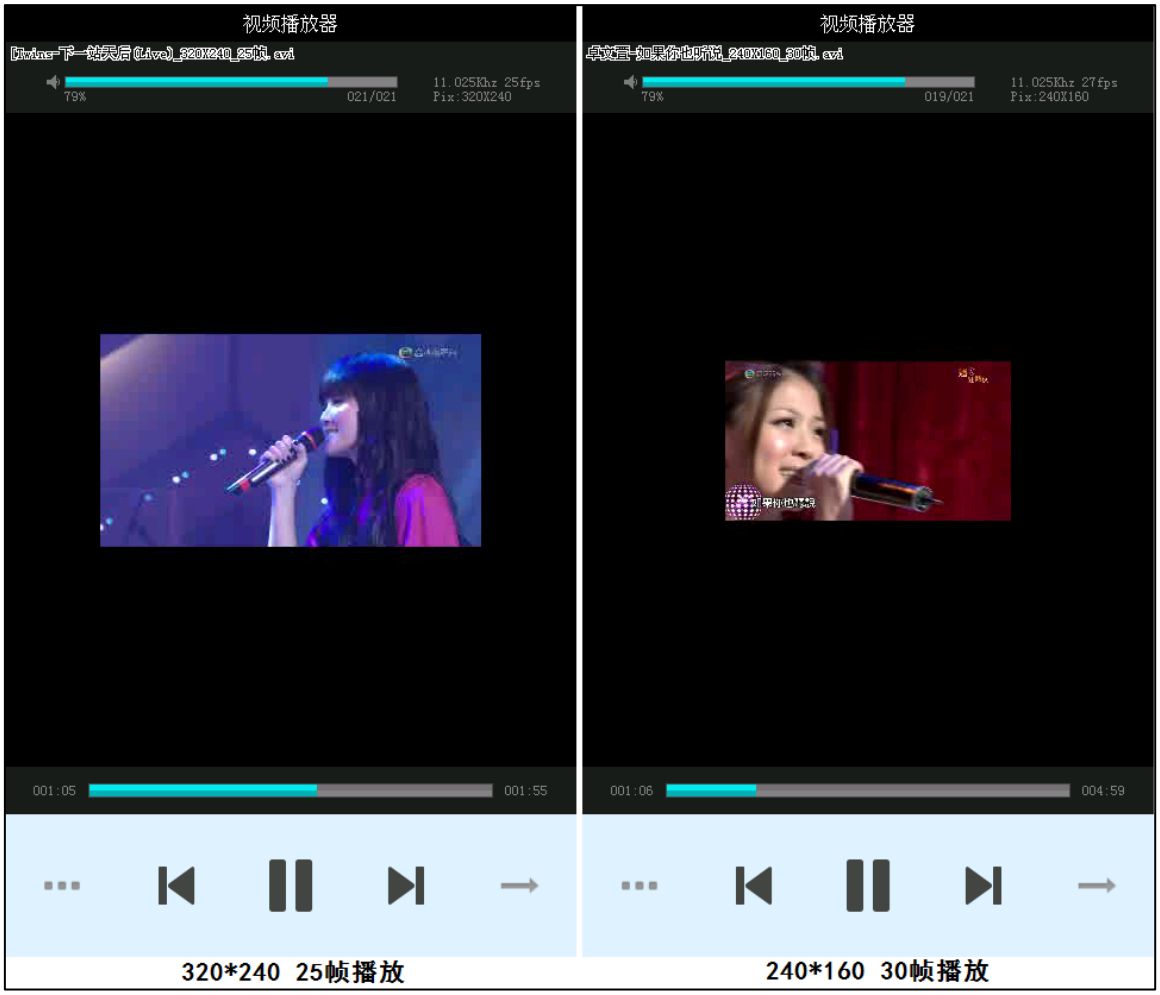


图 78.2.4.2 视频播放

注意，需要外接耳机（插入 PHONE 端子）或者喇叭（接 SPK）才可以听到视频的声音哦!!! 耳机和喇叭需自备。

78.2.5 时钟

双击主界面的时钟图标，进入时钟界面，如图 78.2.5.1 所示：

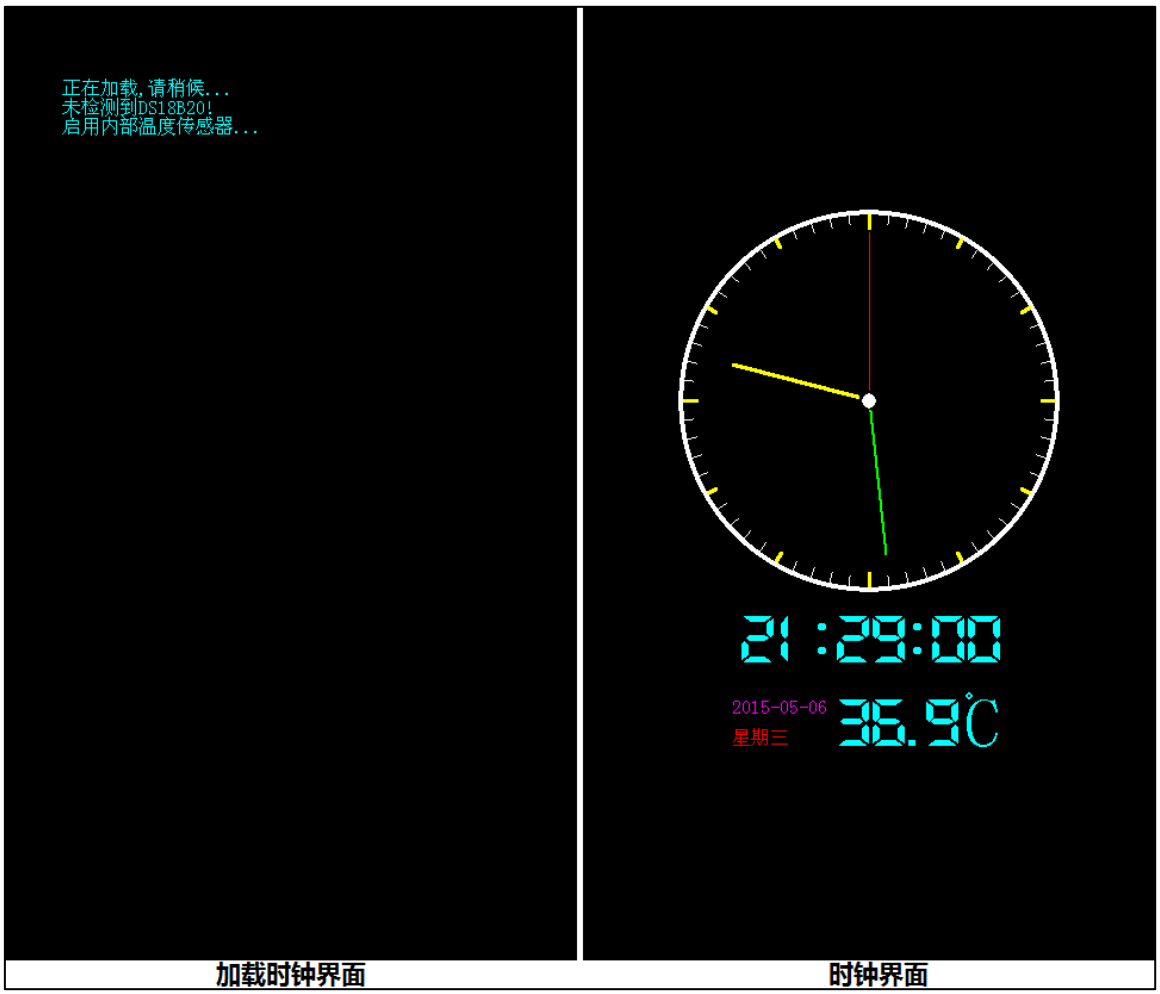


图 78.2.5.1 时钟界面

图 78.2.5.1 的左侧图片为加载时钟界面时的提示界面，表明没有检测到 DS18B20，启用 STM32H743 的内部温度传感器，之后进入时钟主界面，如右侧图片所示。在时钟界面，我们显示了日期、时间、温度、星期等信息，并且在屏幕上方区域，有一个指针式时钟显示。我们可以在系统设置里面设置时间和日期，并且还可以设置闹钟和闹铃，这个我们后面再介绍。

同样，按 TPAD 可以返回主界面。

78.2.6 系统设置

双击主界面的系统设置图标，进入系统设置界面，如图 78.2.6.1 所示：

图 78.2.6.1 系统设置主界面和时间设置界面

上图中左侧的图片为系统设置主界面，在系统设置里面，总共有 23 个项目：时间设置、日期设置、闹钟时间设置、闹钟开关设置、闹钟铃声设置、语言设置、数码相框设置、音乐播放器模式设置、视频播放器模式设置、板载喇叭开关设置、ES8388 音量&3D 设置、ES8388 音效 1 设置、ES8388 音效 2 设置、ES8388 音效 3 设置、ES8388 音效 4 设置、ES8388 音效 5 设置、背光设置、屏幕校准、系统文件更新、恢复默认设置、系统信息、系统状态、关于。通过这 23 个项目，我们可以设置和查看各种系统参数。下面我们将一一介绍这些设置。

首先是时间设置，如图 78.2.6.1 右侧图片所示，双击时间设置，就会弹出一个时间是指对话框，通过这个对话框，我们就可以设置开发板的时间了。设置好之后点击确定回到系统设置主界面，如果想放弃设置，则直接点击取消（或 TPAD）。

再来看看日期设置和闹钟时间设置，如图 78.2.6.2 所示：

图 78.2.6.2 日期设置和闹钟时间设置

上图中，左侧的对话框用来设置系统日期，右侧的对话框用来设置闹钟时间。操作上同前面介绍的时间设置的方法一模一样。关于闹钟，我们等下再详细介绍，先看闹钟开关设置和闹钟铃声设置两个界面，如图 78.2.6.3 所示：

图 78.2.6.3 闹钟开关设置和闹钟铃声设置

上图中，左侧对话框用来设置闹钟开关，右侧对话框用来设置闹钟铃声。这里，我们来介绍一下本系统的闹钟，本系统的闹钟以星期为周期，以时间为点实现闹钟，比如判断一个闹钟是否应该响铃的标准是：先判断星期的条件是否满足，比如上图我们设置是周一到周五闹铃，今天（2023 年 6 月 19 号）是周一，所以满足星期条件，接着看时间是否相等，如果两个条件都满足，则闹铃。假定当前时间是 5:59，我们设置的闹钟时间是 6:00，所以时间还不相等，故不闹铃，当时间来到 6:00 的时候，系统将会闹铃。闹铃铃声有 4 种，如上图右侧图片所示，铃声由蜂鸣器产生，铃声 1 对应“滴”，铃声 2 对应“滴、滴”，铃声 3 和 4 依此类推。当闹钟时间到来的时候，产生闹铃，如图 78.2.6.4 所示：

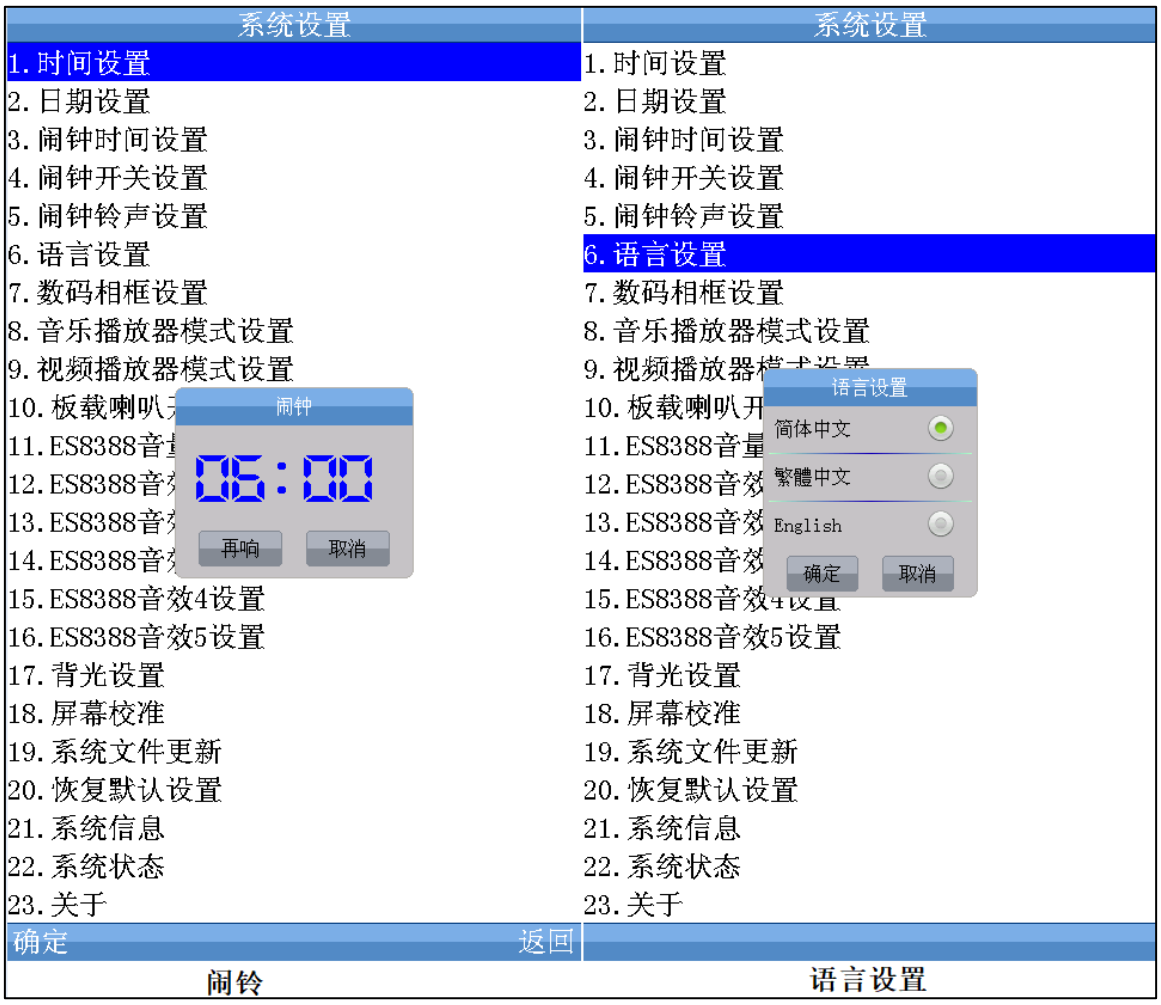


图 78.2.6.4 闹铃和语言设置

上图中，左侧的图片显示正在闹铃。此时会弹出一个闹钟的对话框，并显示当前时间，同时蜂鸣器发出“滴、滴、滴、滴”的闹铃声（铃声 4）。按取消（或 TPAD）可以关闭闹钟，按再响，则 5 分钟后（6:05）继续闹铃。右侧的图片为语言设置界面，系统支持 3 种语言设置，默认为简体中文，设置为繁体中文/English 之后如图 78.2.6.5 所示：

系統設置	SYSTEM SET
1.時間設置	1.TIME SET
2.日期設置	2.DATE SET
3.鬧鐘時間設置	3.ALARM TIME SET
4.鬧鐘開關設置	4.ALARM ON/OFF SET
5.鬧鐘鈴聲設置	5.ALARM RING SET
6.語言設置	6.LANGUAGE SET
7.數碼相框設置	7.DIGITAL PHOTO FRAME SET
8.音樂播放器模式設置	8.AUDIO PLAYER MODE SET
9.視頻播放器模式設置	9.VIDEO PLAYER MODE SET
10.板載喇叭開關設置	10.ONBOARD SPEAKER SET
11.ES8388音量&3D設置	11.ES8388 VOL&3D SET
12.ES8388音效1設置	12.ES8388 EQ1 SET
13.ES8388音效2設置	13.ES8388 EQ2 SET
14.ES8388音效3設置	14.ES8388 EQ3 SET
15.ES8388音效4設置	15.ES8388 EQ4 SET
16.ES8388音效5設置	16.ES8388 EQ5 SET
17.背光設置	17.BACKLIGHT SET
18.屏幕校準	18.TOUCH SCREEN ADJUST
19.系統文件更新	19.SYSTEM FILE UPDATE
20.恢復默認設置	20.RESTORE DEFAULT SET
21.系統信息	21.SYSTEM INFORMATION
22.系統狀態	22.SYSTEM STATUS
23.關於	23.ABOUT
確定 返回	OK BACK
繁体中文	English

图 78.2.6.5 繁体中文和 English

上图显示了繁体中文和 English 的设置，不过本章我们还是以简体中文为例进行介绍。下面，我们来看看数码相框设置和音乐播放器模式设置，如图 78.2.6.6 所示：

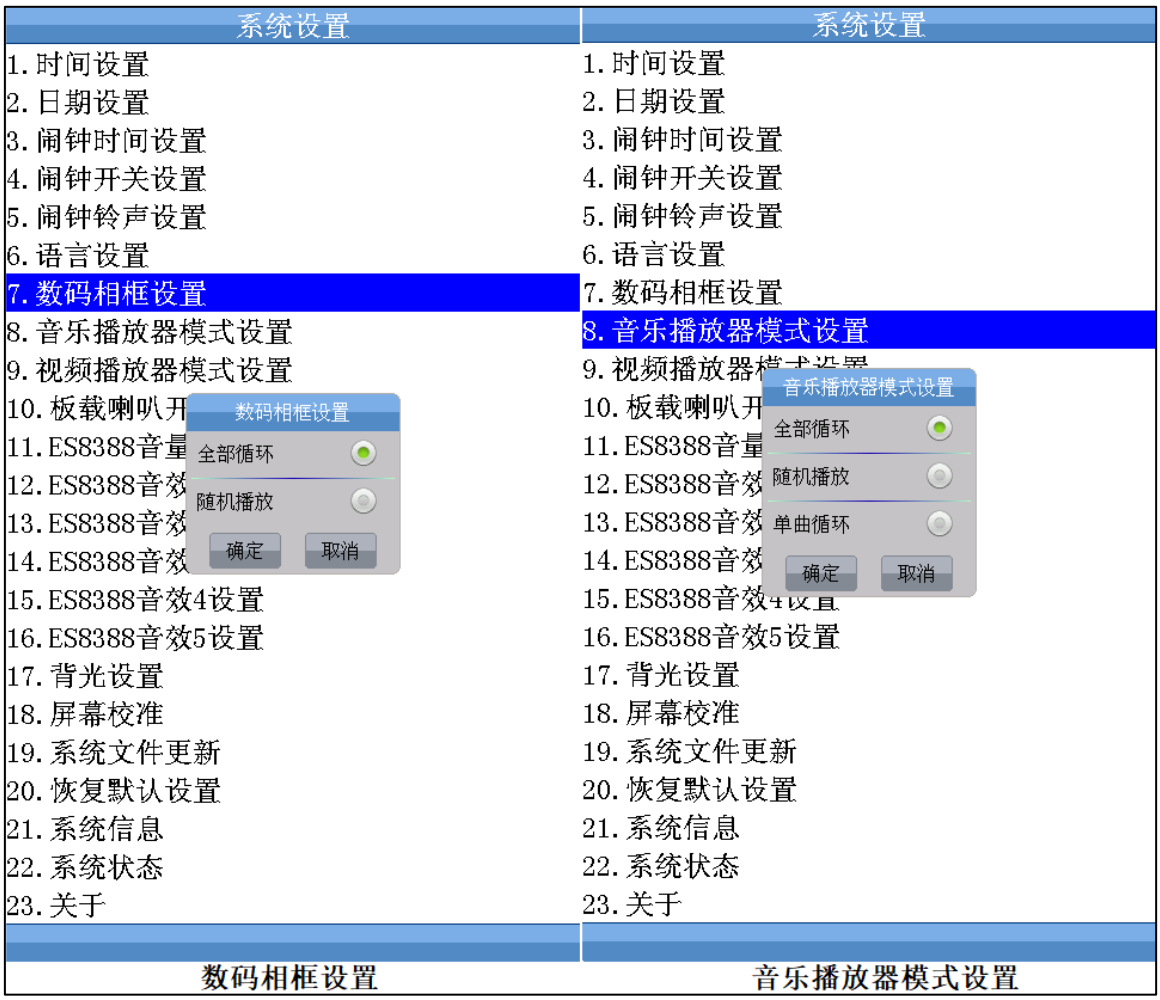


图 78.2.6.6 数码相框设置和音乐播放器模式设置

前面提到数码相框支持全部循环播放和随机播放两种模式，就是通过上图左侧的界面设置的。而音乐播放器的三个播放模式，则通过右侧的界面进行设置。接下来看看视频播放器模式和板载喇叭开关设置，如图 78.2.6.7 所示：

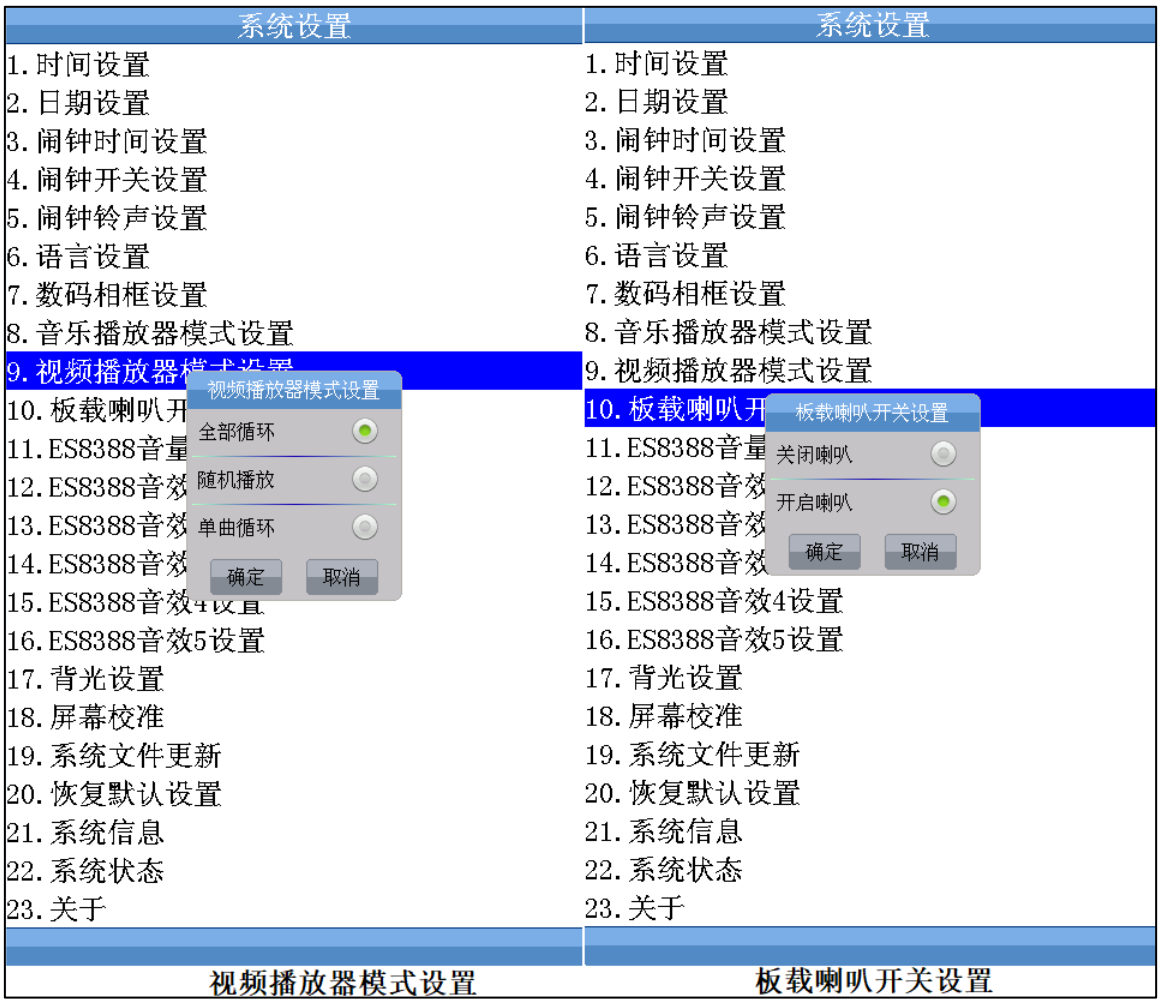


图 78.2.6.7 视频播放器模式设置和板载喇叭开关设置

上图中，左侧的界面可以设置视频播放器的模式：全部循环、随机播放、单曲循环，默认是全部循环模式，这个设置和音乐播放器的模式一模一样。右侧的界面可以设置板载喇叭是否开启，阿波罗 V2 板载了一个喇叭，在开发板的背面，可以通过这个设置来选择是否开启。默认是开启板载喇叭的，如果设置为关闭喇叭，则板载的喇叭不会发声，需要插耳机到 PHONE 端口，才可以听到开发板的声音。

下面我们看看 ES8388 音量&3D 设置和背光设置，如图 78.2.6.8 所示：

图 78.2.6.8 ES8388 音量&3D 设置和 ES8388 音效设置

上图左侧是 ES8388 音量&3D 设置，可以设置 ES8388 的音量（在此处修改后，将写入 EEPROM 保存，而如果在音频播放的时候修改音量，是不会保存在 EEPROM 的），同时还可以设置 3D 效果，值越大，3D 效果越强，默认是 0，即关闭 3D 效果。

右侧的界面用于设置 ES8388 的音效 1，ES8388 总共支持 5 段 EQ 设置，分别对应设置界面的：音效 1~音效 5 设置，这里我们仅以音效 1 设置为例，其他的类似。图中的中心频率，即增益对应的频点，默认是 80Hz，可以通过中心频率滚动条设置（80Hz~175Hz），而增益默认设置的是 0，也可以通过滚动条设置（范围：-12dB~+12dB）。大家可以根据自己的喜好，来设置音效以达到最好的音乐享受。**注意：系统设置里面对 ES8388 的设置，都是立即生效的，并且会保存在 EEPROM 里面，当这里设置好之后，其他用到 ES8388 来播放音乐的程序（音乐播放、视频播放、NES 和 SMS 游戏等），都会共用这些设置。**

接下来，我们看看背光设置和系统文件更新，如图 78.2.6.9 所示：

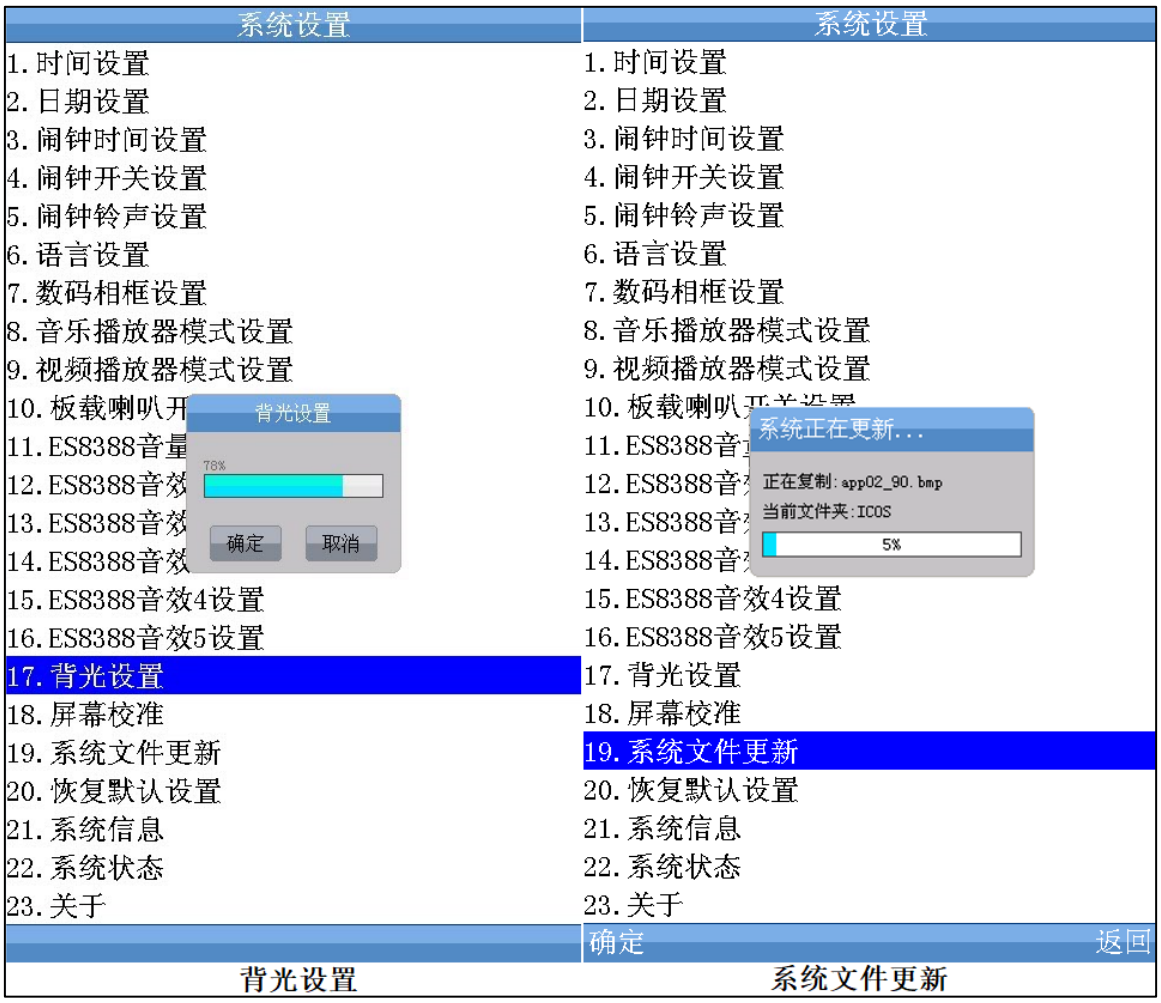


图 78.2.6.9 背景设置和系统文件更新

左侧的界面用于设置 LCD 背光，背光通过 PWM 控制。当设置为 0 的时候，启动自动（auto）背光控制，其他值则是固定的背光亮度，值越大越亮。自动背光的时候，通过板载的光环境传感器（AP3216C）采集环境光强，自动调整背光。

第 18 项，屏幕校准，这里因为我们用的是 4.3 寸电容触摸屏为例讲解的，电容屏不需要校准，所以这个设置对 4.3 屏模块无效。如果是电阻屏，点击该项则可以进入屏幕校准，根据提示完成校准即可。

上图中，右侧是双击系统文件更新提示，这里的系统文件是指 SYSTEM 文件夹里面除字库文件外的所有内容。综合测试实验之所以可以没有 TF 卡也能正常运行，主要是将 SYSTEM 文件夹（注意这个不是源码里面的 SYSTEM 文件夹!!）拷贝到了 SPI Flash Disk（即 25Q256）里面，这样，我们所有的系统资源都可以从 25Q256 里面获得，从而正常启动。

SYSTEM 文件夹目前是包含 209 个文件，总大小为 11MB。这些文件一般不要修改，如果你想自己 DIY 的话，那可以修改这些文件，以达到你要的效果，不过建议修改之前备份一下，搞坏了还可以还原。

如果点击了确定更新系统文件，则会执行系统文件更新，将 TF 卡/U 盘的 SYSTEM 文件夹（拷贝自光盘：5，SD 卡根目录文件 里面的 SYSTEM 文件夹），拷贝到 FLASH Disk 里面。这里有个前提，就是你的 TF 卡/U 盘里面必须有这个 SYSTEM 文件夹！更新时界面如图 78.2.6.9 右侧图片所示，该界面显示了当前更新的文件夹以及文件和进度等信息。

接下来，我们看看恢复默认设置和系统信息，如图 78.2.6.10 所示：

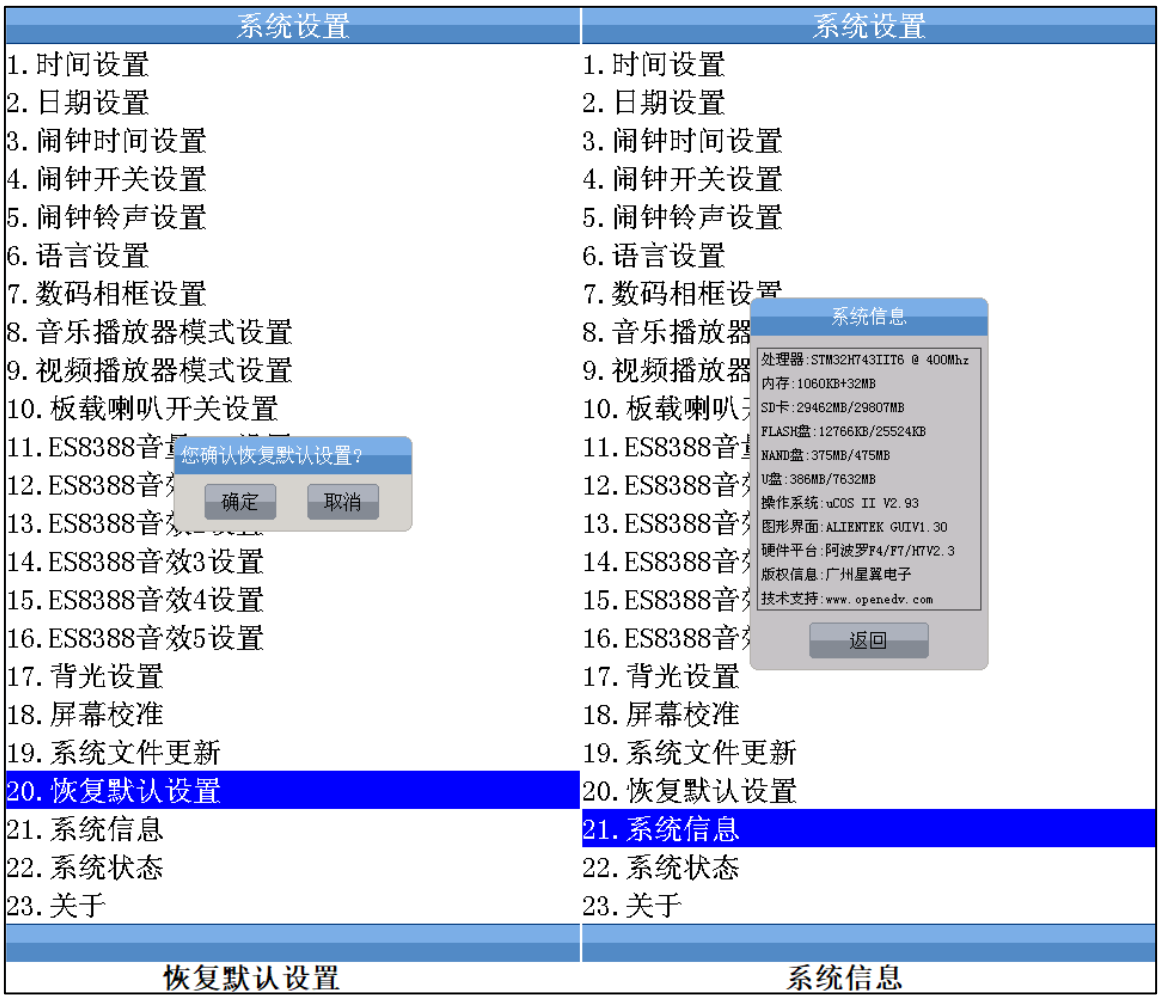


图 78.2.6.10 恢复默认设置和系统信息

上图左侧图片为恢复默认设置功能确认界面，当选择确定后，系统将恢复默认设置，除了 RTC 时间日期以外的所有设置，都将恢复默认值，方便大家在设置乱以后，恢复正常。

上图右侧图片为系统信息界面，通过该界面，可以看到软硬件的详细信息。

最后，我们来看看系统状态和关于界面，如图 78.2.6.11 所示：

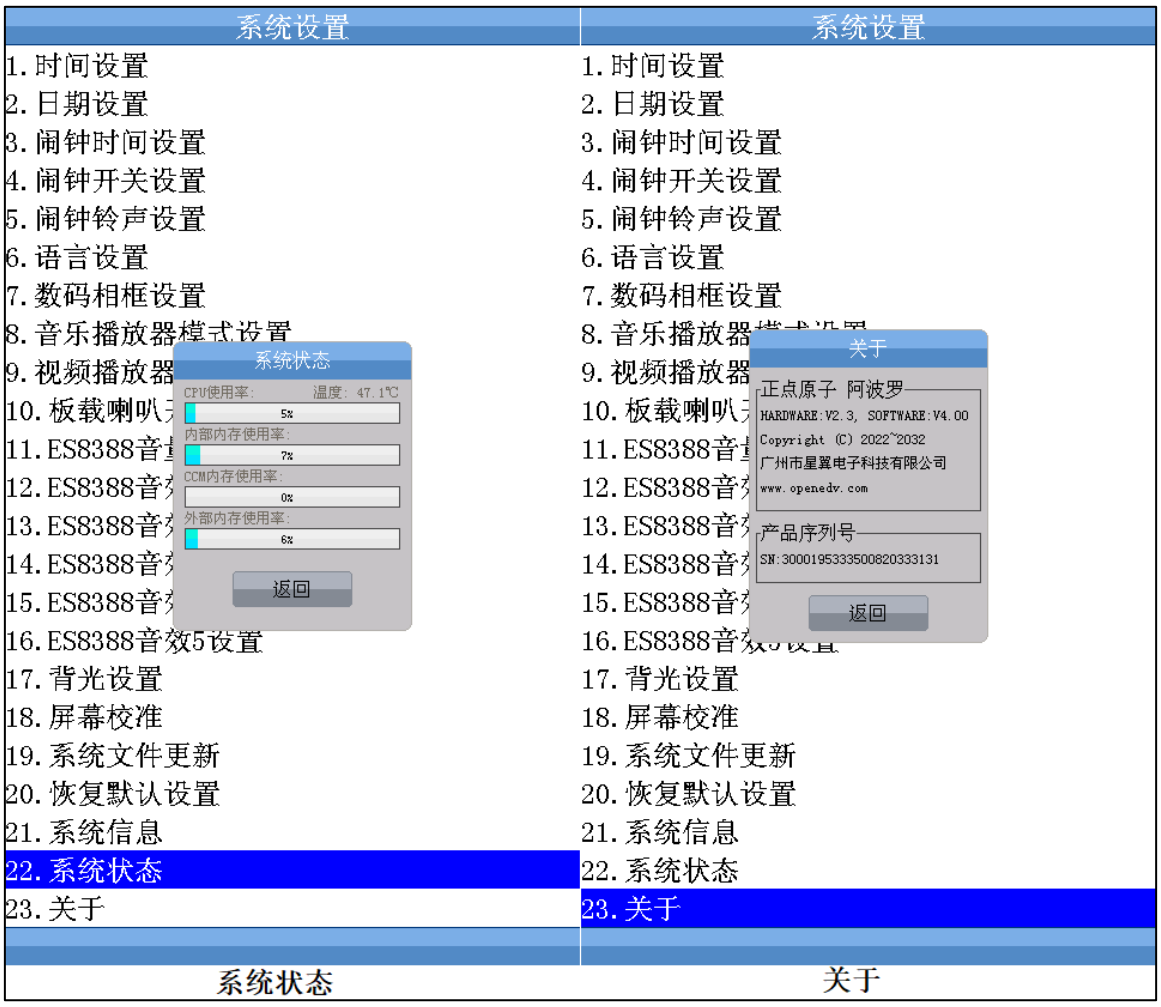


图 78.2.6.11 系统状态和关于界面

上图中，左侧的界面显示了当前系统资源状况，显示了当前 CPU 使用率，CPU 温度以及内存使用率。

右侧的图片显示了阿波罗 STM32H743 开发板的软硬件版本以及产品序列号，这个序列号是全球唯一的，每个开发板都不一样。

78.2.7 FC 游戏机

综合测试实验移植了一个非常强大的 NES 模拟器，核心部分采用汇编实现，效率极高，支持音频输出，支持 MAP，支持绝大部分 NES 游戏的运行。综合性能超过 infoNES。该模拟器由开源电子网（http://www.openedv.com/forum.php）论坛网友：ye781205 编写，然后正点原子移植到本开发板上，即 FC 游戏机。

该 FC 游戏机特点如下：

1， 支持 MAP，可运行绝大部分小于 960K 的 NES 游戏。

2， 支持 USB 手柄（目前已测试了迪龙 PU201、PU401、PU701、蓝觉 L600、酷孩 USB 无振动手柄等）。

3， 支持 USB 键盘输入，可双人游戏。键盘按键与手柄功能对应关系如表 78.2.7.1 所示。

4， 支持声音输出（音效远超 infoNES）。

5， 支持全速运行（60 帧），在 4.3 寸屏会放大 4 倍处理（480*480 分辨率）。

双击主界面的 FC 游戏机图标，屏幕会弹出提示对话框，如图 78.2.7.1 所示：

标准 FC 手柄功能	A	A 连击	B	B 连击	上	下	左	右	Select	Start
对应 USB 键盘按键										

手柄 1（主）	K	I	J	U	W	S	A	D	空格	回车
手柄 2（从）	3	6	2	5	↑	↓	←	→		

表 78.2.7.1 USB 键盘按键与手柄功能对应关系

上表中的 3、6、2、5 等数字，是指小键盘的数字按键。另外，需要**注意：玩 NES 游戏的时候，USB_SLAVE 不要插电脑，否则无法识别 USB 键盘/USB 手柄！！！**

双击主界面的系统设置图标，如果当时插着 U 盘，屏幕将提示：请先拔掉 U 盘。然后，会提示：请插入 USB 手柄/键盘。然后进入 nes 文件浏览界面，如图 78.2.7.1 所示：

图 78.2.7.2 文件浏览和超级玛丽游戏

在检测到 USB 手柄/键盘插入后，屏幕会提示：检测到 USB 手柄/键盘。这个检测是一直开启的，只要插入 USB 手柄/键盘，就会被系统检测到。

上图中，左侧为 nes 文件浏览界面，我们随便选择一个打开即可开始游戏了，记得插上手柄哦！右侧的图片为经典的超级玛丽游戏界面，当然还可以玩很多其他经典游戏，如下面的图片所示：

图 78.2.7.3 冒险岛和魂斗罗

图 78.2.7.4 三木童子和双截龙

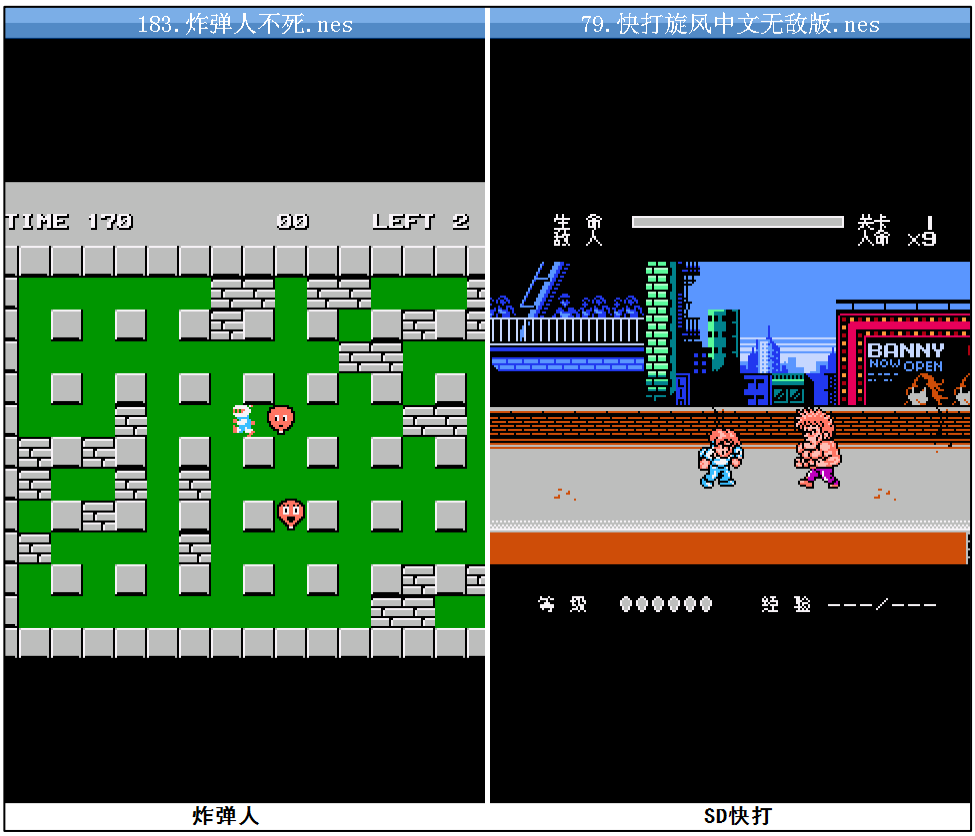


图 78.2.7.5 炸弹人和 SD 快打

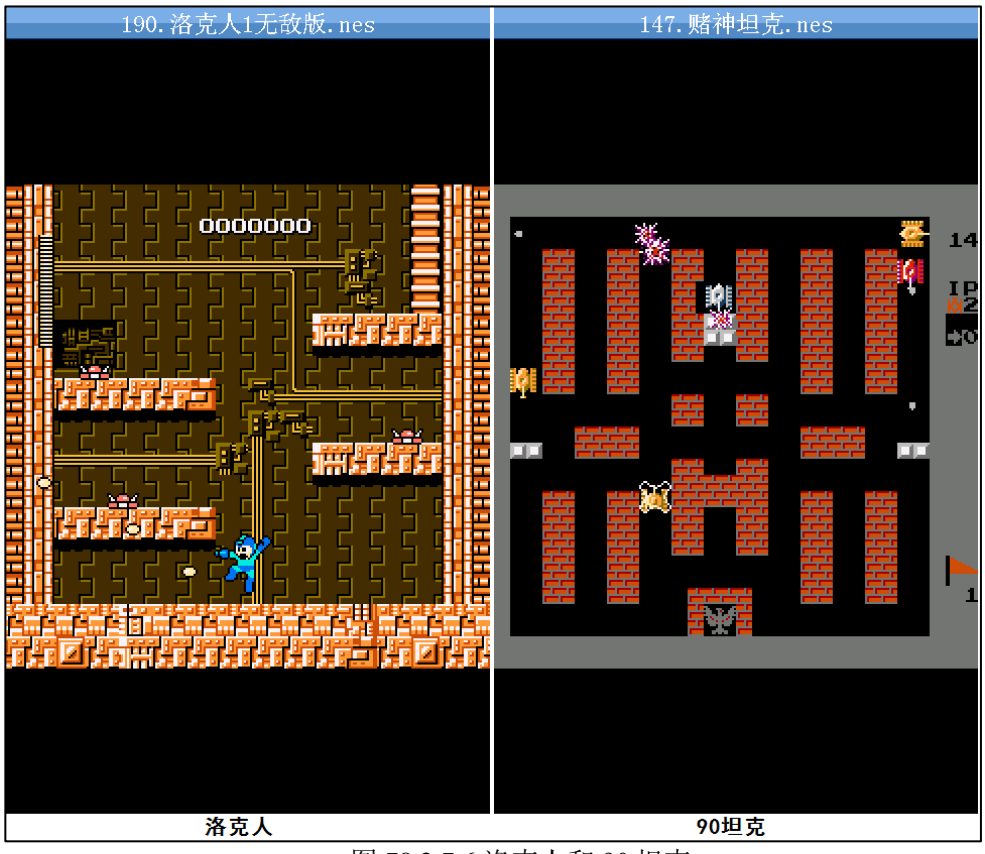


图 78.2.7.6 洛克人和 90 坦克

这里，我们仅列出了几种游戏，这都是 80 后童年时玩的经典游戏，如今，在阿波罗 V2 开发板上，大家可以回味一下当年的经典了。

78.2.8 记事本

双击主界面的记事本图标，首先弹出模式选择对话框，如图 78.2.8.1 所示：

图 78.2.8.1 模式选择和新建文本文件

记事本支持 2 种模式：1，新建文本文件，这种方式完全新建一个文本文件（以当前系统时间命名），用来输入信息。2，打开已有文件，这种方式可以对已有的文件进行编辑。

上图中，右侧的界面为我们选择新建文本文件后的界面，此时出现一个空白编辑区和一个闪烁的光标，我们通过下方的键盘输入信息即可，这个输入键盘和我们的手机键盘十分类似，输入方法也是一模一样，支持中文、字母、数字和手写识别输入等几种输入方式，如图 78.2.8.2 和图 78.2.8.3 所示：

图 78.2.8.2 中文输入和标点符号输入

图 78.2.8.3 英文输入和手写识别输入

其中，中文输入就是我们前面 T9 拼音输入法实验的具体运用，而手写识别的输入界面，我们也是用到前面手写识别实验的知识实现的。

只要新建文本文件有被编辑过，那么在返回（按 TPAD 返回）的时候，系统会提示是否保存，如图 78.2.8.4 所示：

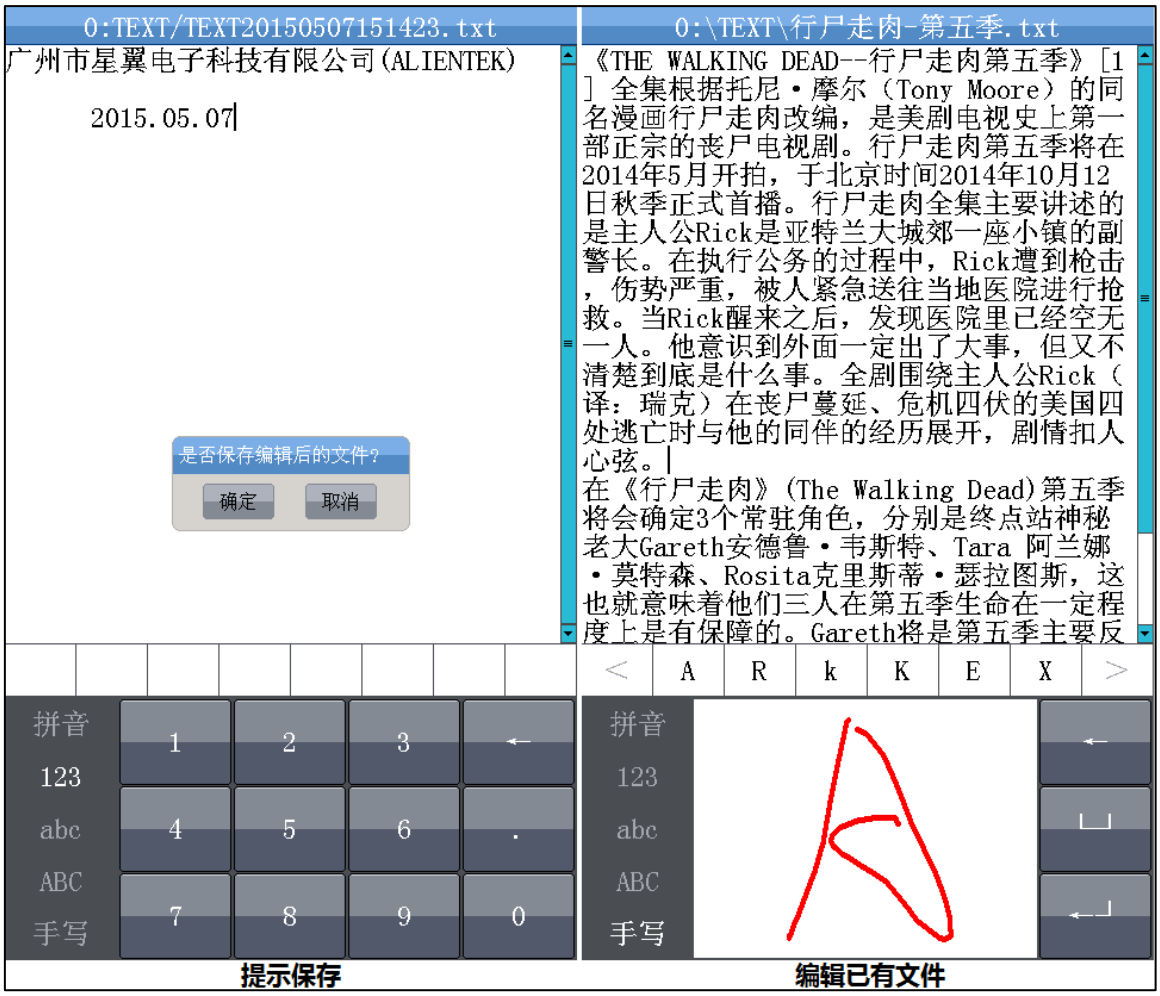


图 78.2.8.4 保存提示和编辑已有文件

上图中，左侧图片为提示保存界面，如果选择确定，该文件将被保存在 TF 卡根目录的 TEXT 文件夹里面。右侧图片为打开已有文件进行编辑的界面，这样我们就可以在开发板上编辑.txt/.h/.c/.lrc 文件了。

78.2.9 运行器

双击主界面的运行器图标，首先进入文件浏览界面，如图 78.2.9.1 所示：

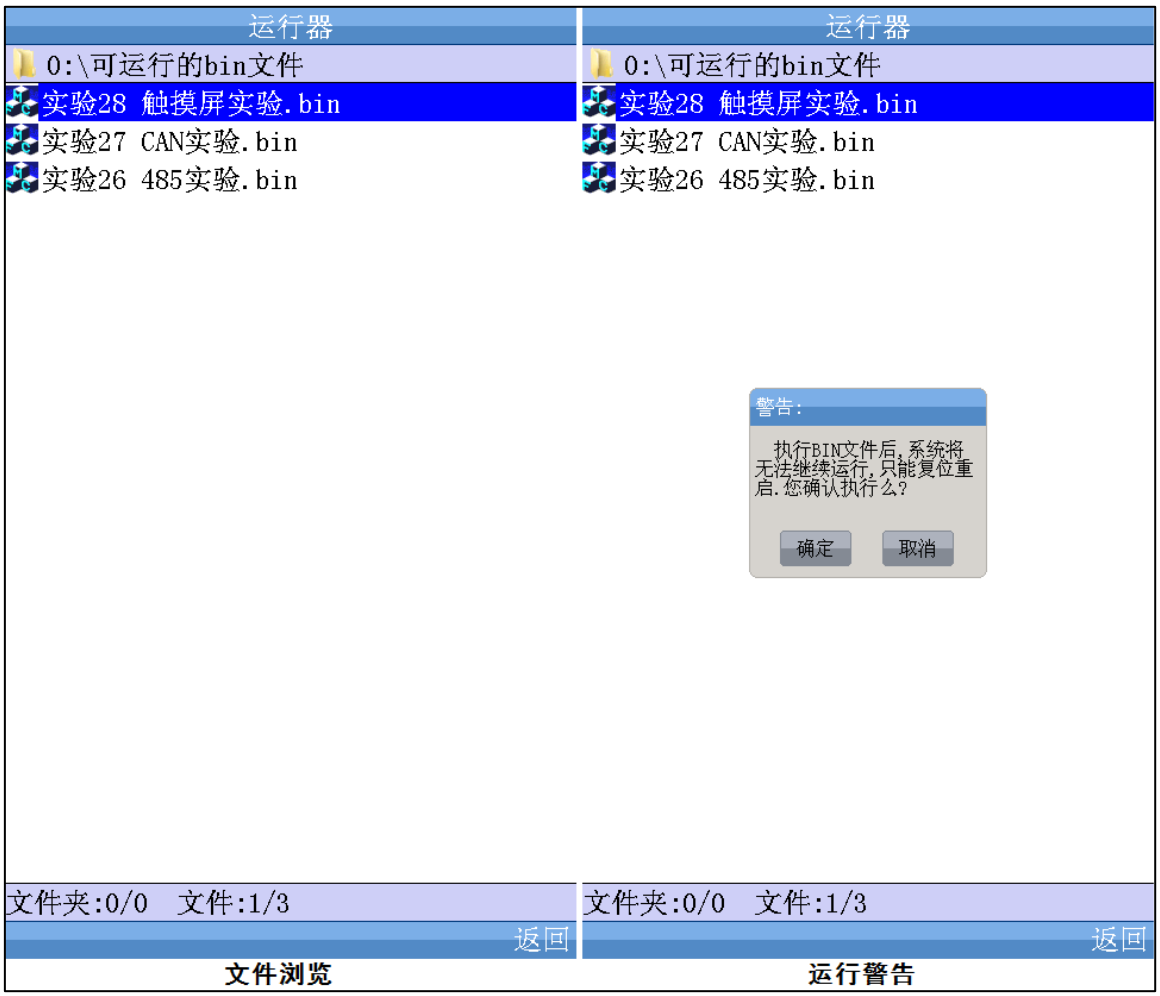


图 78.2.9.1 文件浏览和运行警告

上图中，左侧为文件浏览界面，图中显示了可运行的 bin 文件有 3 个，这些全部来自我们的标准例程对应的实验。本运行器支持 146K 字节以内的程序运行（FLASH+SRAM 总共不超过 192K），很多例程都可以生成 SRAM 版本的 bin 文件，在运行器里面运行。**SRAM 版本.bin 文件的生成办法，请参考串口 IAP 实验这个章节，里面有详细介绍。**通过运行器，大家可以直接运行我们大部分例程，而不用再去刷代码了，方便大家测试和验证我们的实验。

右侧的图片是运行前的警告界面，因为一旦执行.bin 文件，我们的系统将无法恢复，只能靠复位重启。点击确定之后，STM32 就开始运行你所选择的.bin 文件了，实验现象和对应实验所描述的现象一模一样。

78.2.10 手写画笔

双击主界面的手写画笔图标，首先弹出模式选择对话框，如图 78.2.10.1 所示：

图 78.2.10.1 模式选择和新建画板

上图中，左侧图片为我们双击手写画笔后，弹出的模式选择界面，我们可以选择新建画笔，建立一个新的文件；也可以选择打开一个已有的位图进行编辑。右侧的图片为我们新建画笔后输入的内容，默认画笔为最小尺寸，颜色为红色。画笔的颜色和尺寸是可以设置的，按 KEY_UP 按键，则弹出画笔设置对话框，然后，可以对画笔颜色和画笔尺寸进行设置。如图 78.2.10.2 和图 78.2.10.3 所示：

图 78.2.10.2 画笔设置和画笔颜色设置

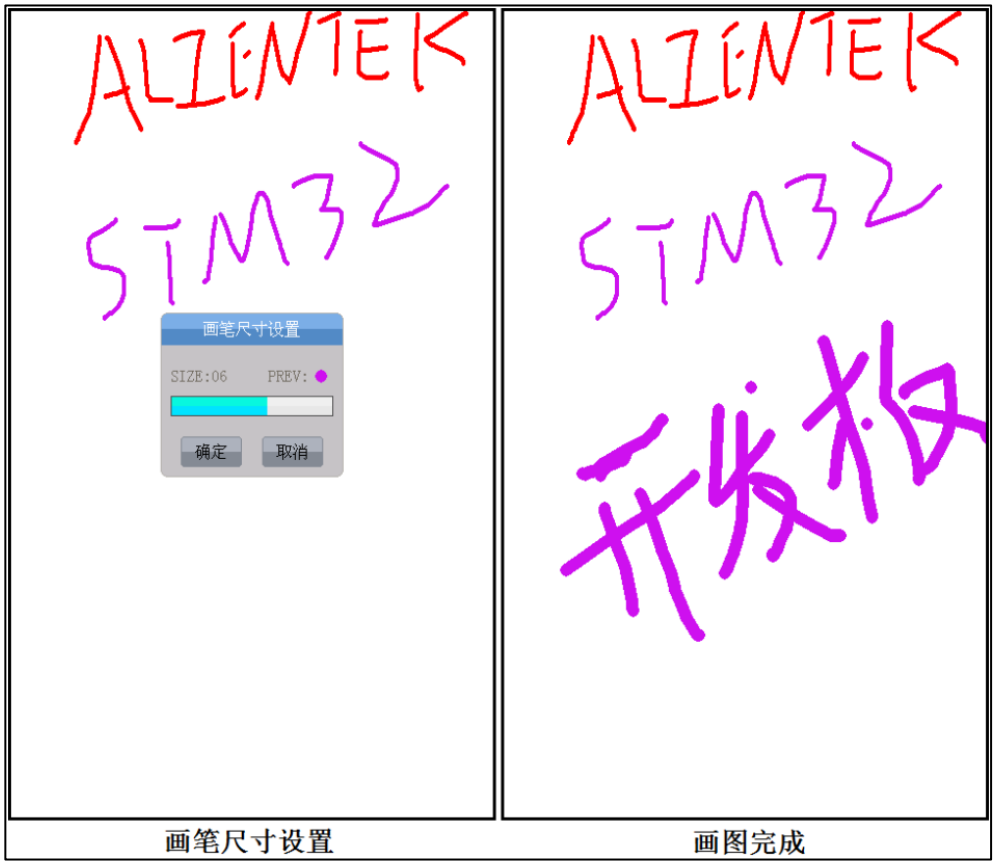


图 78.2.10.3 画笔尺寸设置和完成后的画图

图 78.2.10.2 中，左侧的图片为按 KEY_UP 按键后弹出的画笔设置对话框，我们可以选择

对画笔颜色和画笔尺寸进行设置。右侧的图片为画笔颜色设置对话框，在该对话框里面，我们可以直接在颜色条快速输入要设置的颜色，也可以通过下方的三个滚动条进行精确设置，左侧的正方形区域为预览区。

图 78.2.10.3 中，左侧为画笔尺寸设置界面，我们可以通过滚动条设置画笔尺寸，对话框显示了画笔尺寸和对应的预览图。右侧的图片为我们完成的画图文件，在返回主界面（按 TPAD）的时候，会提示保存，如图 78.2.10.4 所示：

图 78.2.10.4 保存画图和编辑已有位图

上图中，左侧为我们退出时弹出的提示保存对话框，如果选择确定，则新的画图文件将会被保存在 TF 卡的 PAINT 文件夹里面，命名方式是以当前系统的时间命名的，如 PAINT20220620193550.bmp。

右侧的图片为对打开的位图进行编辑的界面，通过这个功能，我们可以在开发板上实现对一些相片（bmp 格式）进行涂鸦。

78.2.11 照相机

本照相机支持正点原子 OV5640 这款 500W 像素的 CMOS 摄像头模块，本照相机的特点有：

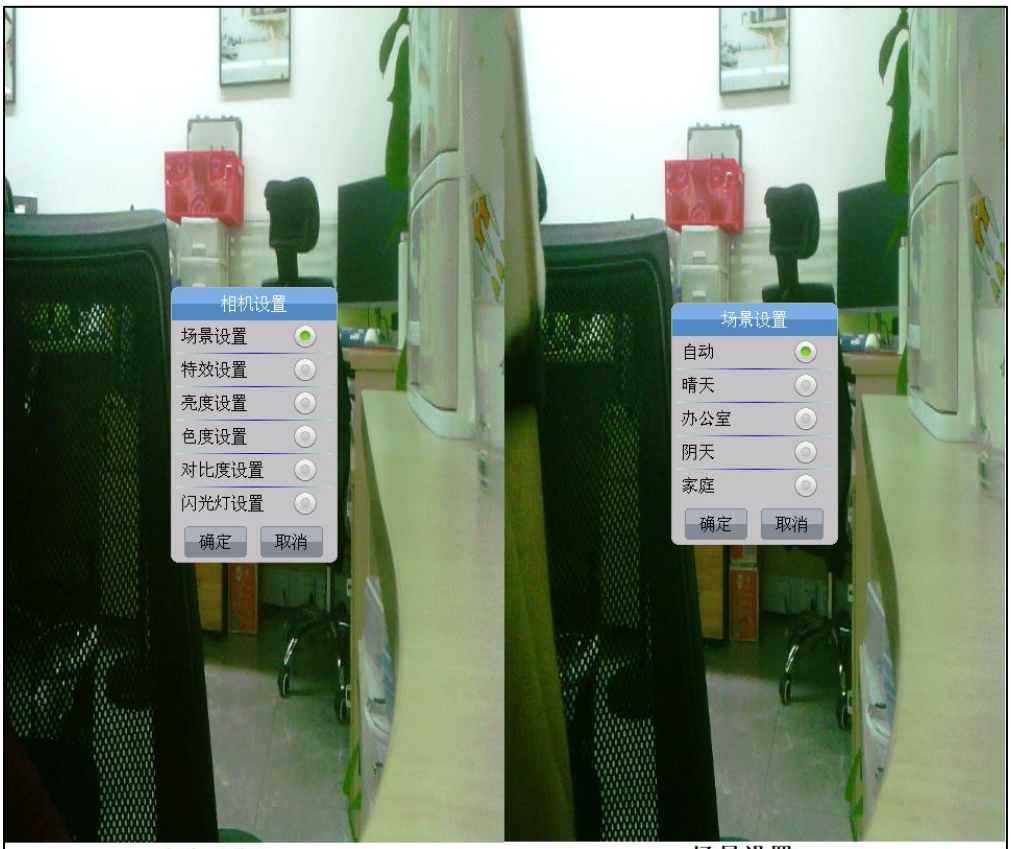
1， 支持自动对焦（轻触触摸屏，即可进行一次自动对焦）。
2， 支持闪光灯（可以设置开启/关闭）。
3， 支持 BMP 拍照（拍下的 bmp 分辨率为 LCD 分辨率），按 KEY2 拍 BMP 照片。
4， 支持 JPG 拍照（默认是 2592*1944 像素(500W 像素)，按 KEY0 拍 JPG 照片。
5， 屏幕显示可以是全景（缩放）或者 1:1 显示（无缩放），默认是全景，通过 KEY1 切换。
6， 支持各种参数设置，包括：场景、特效、亮度、色度、对比度和闪光灯等设置，按

KEY_UP 进入设置。

双击主界面的照相机图标，首先初始化 OV5640 摄像头模块，如图 78.2.11.1 所示：

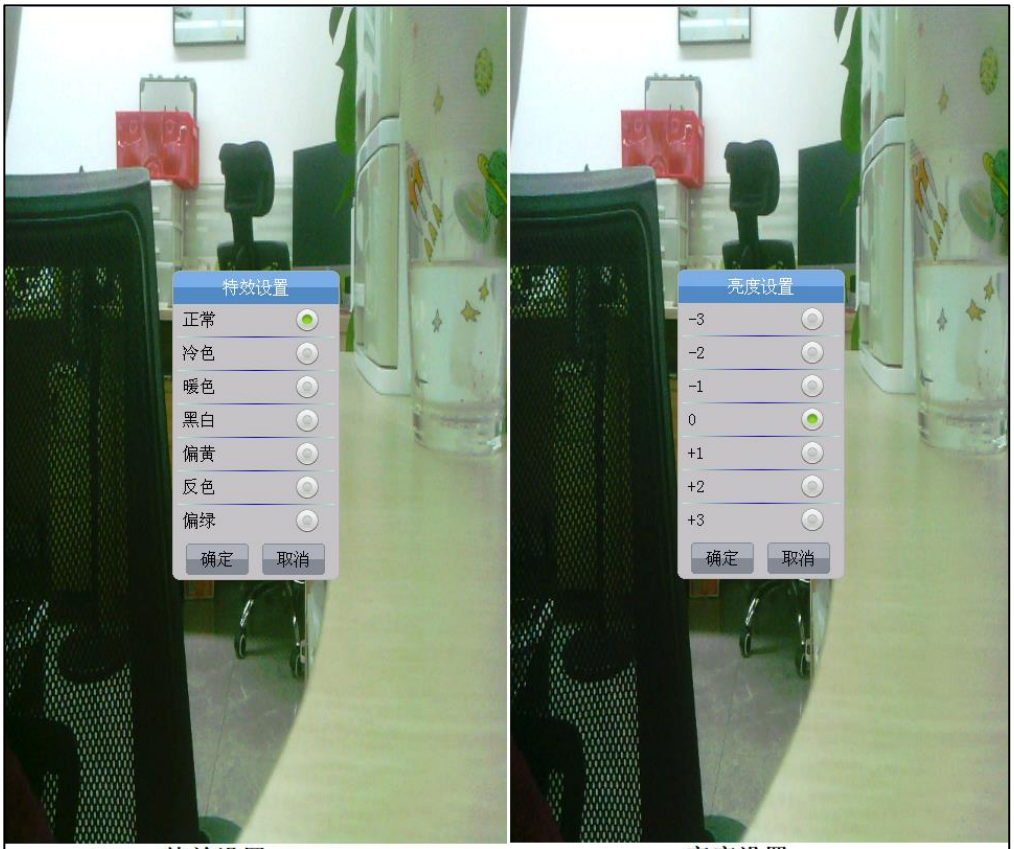
图 78.2.11.1 初始化 OV5640 和等待拍照

在初始化 OV5640 之后，进入等待拍照模式，此时我们可以通过按 KEY_UP 按键，弹出相机设置对话框，对摄像头的参数进行设置，如图 78.2.11.2~78.2.11.5 所示：



相机设置　　场景设置

图 78.2.11.2 相机设置和场景设置



特效设置　　亮度设置

图 78.2.11.3 特效设置和亮度设置

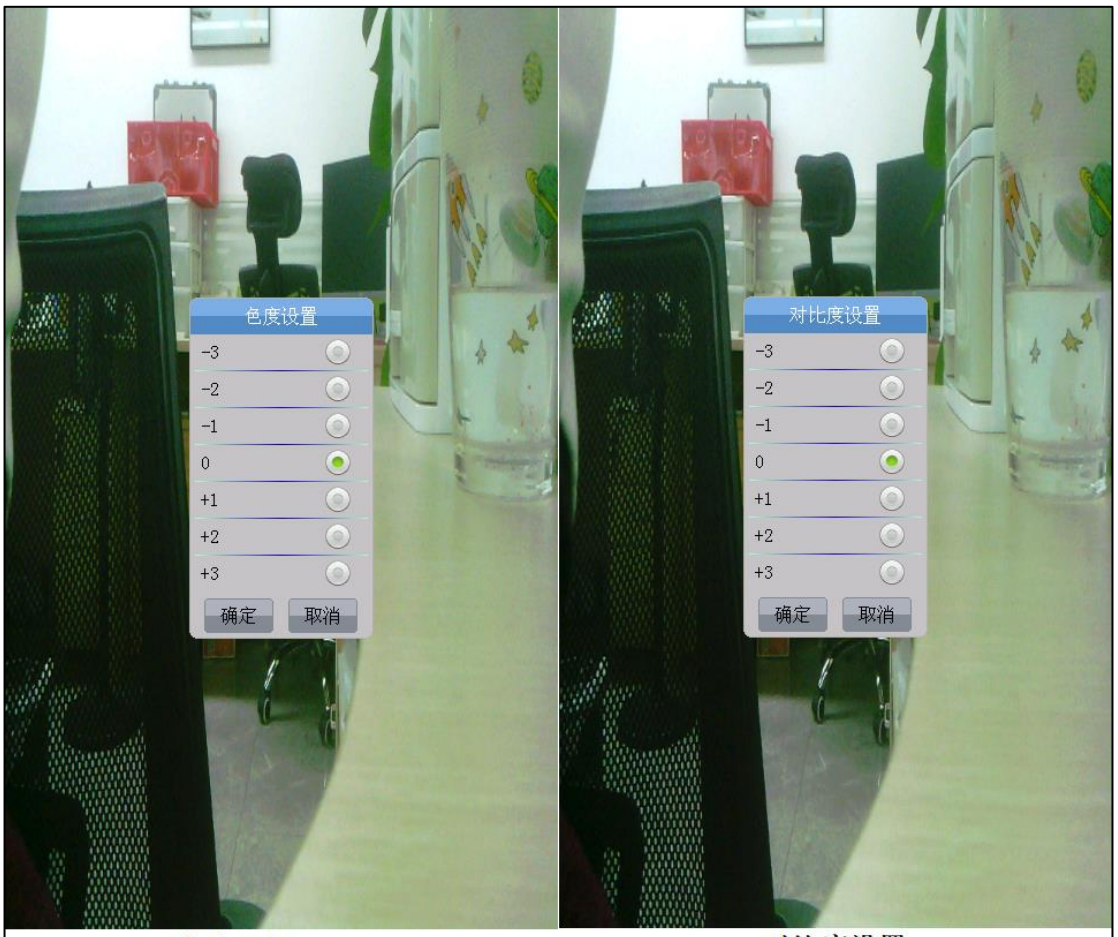


图 78.2.11.4 色度设置和对比度设置

图 78.2.11.5 拍照

以上设置，和手机类似，这里就不一一和大家解释了。我们按 KEY0 按键，即可实现 JPG 拍照，JPG 照片的尺寸，默认是 1600*1200 分辨率，可通过 KEY_UP 设置其他分辨率。按 KEY2 按键即可实现 BMP 拍照，BMP 照片的尺寸就是 LCD 的分辨率。在照片保存期间 LED1 亮，保存完后蜂鸣器发出“滴”的一声，提示拍照成功，同时弹出拍照成功对话框，如上图右侧图片所示。

从上图可以看出，照片文件的命名还是以当前时间为名字命名的。我们将所有的照片都保存在 TF 卡/U 盘的 PHOTO 文件夹。如果你没有插入 TF 卡/U 盘，拍照时会提示“创建文件失败，请检查!”的提示信息。

按 KEY1 按键，可以实现 1:1 显示（显示区域小，但是图像无压缩，不变形），或者全尺寸显示（显示整个摄像头拍照区域，但是图像有压缩，会变形）。另外，如果你觉得照片模糊，可以手动调节摄像头模块的镜头，进行调焦，以达到最佳效果。

78.2.12 录音机

综合测试实验带了录音机功能，可以实现通过 MIC（咪头）录音，并将录音文件保存在 TF 卡/U 盘。录音文件为 WAV 文件，格式为：立体声（但是左声道数据=右声道数据）、16 位、8Khz~48Khz 采样率可设置。

双击主界面的录音机图标，进入录音机主界面，如图 78.2.12.1 左侧图片所示，该界面显示了当前录音时间以及信号电平等，在该界面有两个按钮：左边的按钮用于开始/暂停录音，右边的按钮用于停止录音，并保存当前录音文件。

录音机功能可以设置采样率和 MIC（咪头，这里称之为麦克风）增益，通过点击左下角的选项按钮，弹出录音设置对话框，可以设置采样率和 MIC 增益，如图 78.2.12.1 右侧图片所示。

图 78.2.12.1 录音机主界面和麦克风增益设置

采样率设置，可以设置录音时的采样率，支持：8Khz、16Khz、32Khz、44.1Khz 和 48Khz 等几种采样率。默认是 16Khz。**注意：采样率必须在没有开始录音之前进行设置才行，否则会提示：请先停止录音。**

MIC 增益设置，支持从-12dB~32.25dB 的设置范围，以 0.75dB 步进。默认是 22.5dB 增益，大家可以根据自己需要设置合适的 MIC 增益，以达到最好效果。注意：如果外接了喇叭或者音箱，当增益太大的时候，会产生啸叫，这个时候，请将增益设置小一点。

采样率设置和 MIC 增益设置，如图 78.2.12.2 所示：

图 78.2.12.2 采样率设置和麦克风增益设置

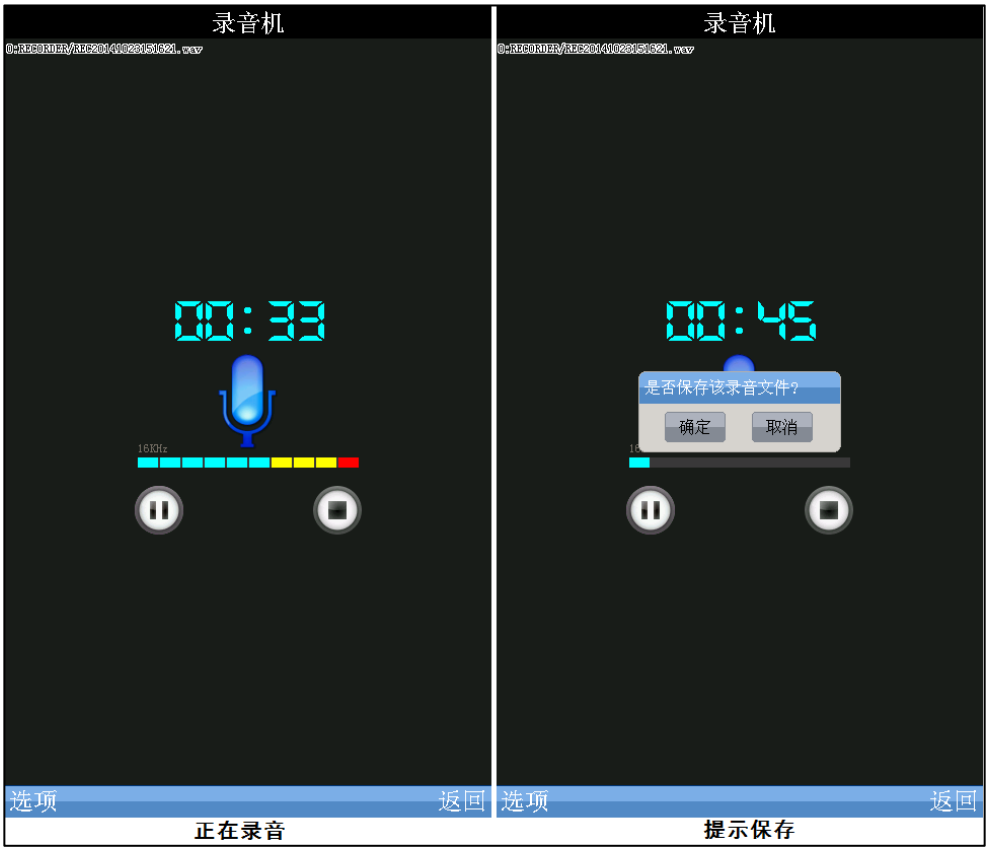


图 78.2.12.3 录音进行中和提示保存

在图 78.2.12.3 中，左侧的图片为正在录音的界面，此时我们可以按暂停/停止，按停止则

自动保存当前录音文件，录音文件同样是以时间命名（见图中上方白字），所有录音文件都是被保存在 TF 卡/U 盘根目录的 RECORDER 文件夹里面的。在录音的时候，按下 TPAD，会提示是否保存，如上图右侧图片所示，我们可以根据需要选择。

78.2.13 USB 连接

双击主界面的 USB 连接图标，如果开发板的 USB 端口没有连接电脑，则显示无连接，如图 78.2.13.1 所示：

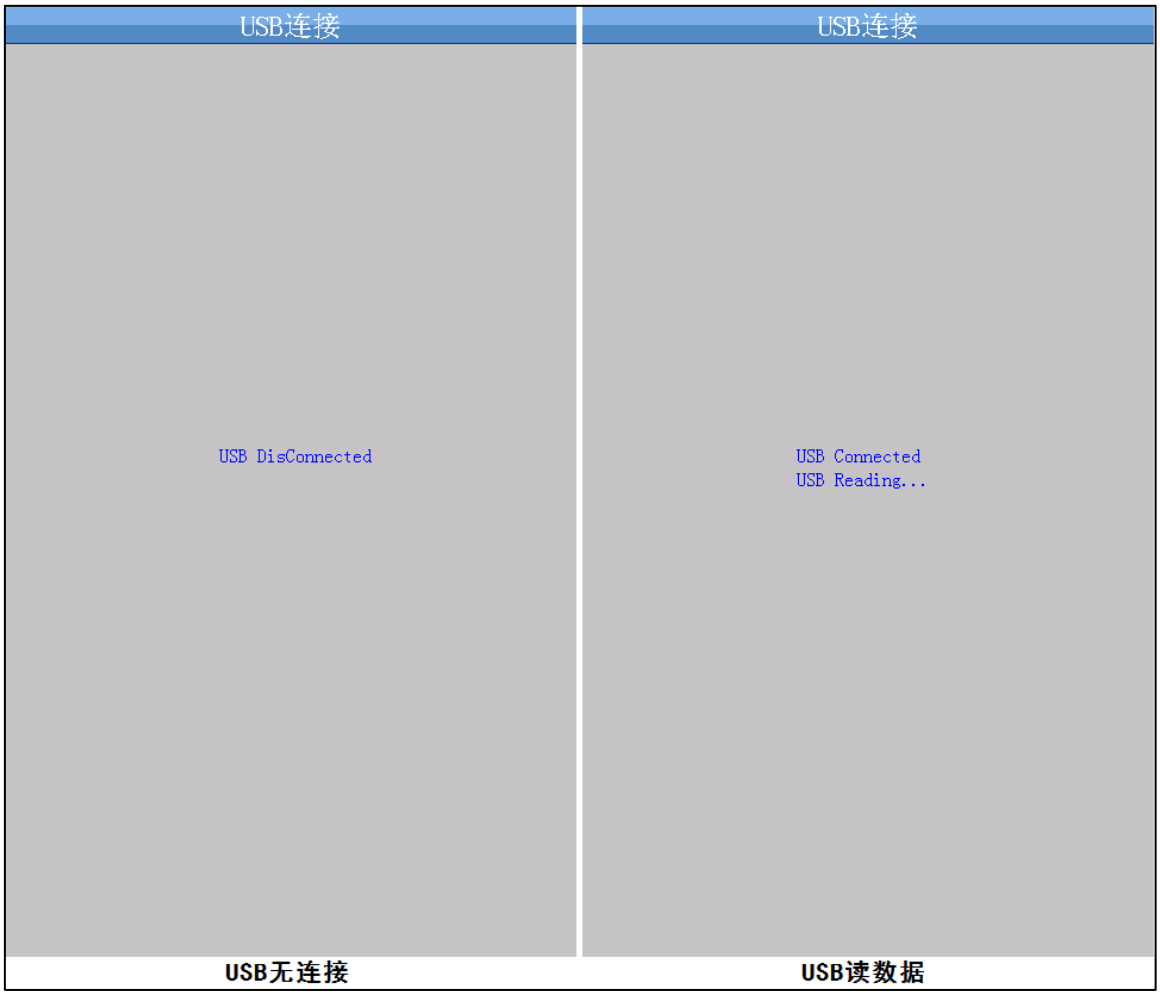


图 78.2.13.1 USB 无连接和 USB 读数据

上图中，左侧的图片显示开发板没有和电脑连接上，此时，我们找一根 USB 线，连接开发板的 USB_Slave 端口和电脑的 USB，然后，可以看到开发板提示 USB 已连接，并显示 USB 正在读数据，同时我们在电脑上面，可以看到右下角提示发现新硬件，并自动安装驱动（如果是第一次连接的话），如图 78.2.13.2 所示：

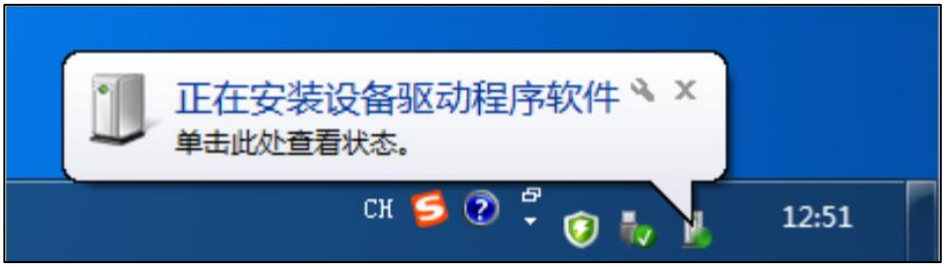


图 78.2.13.2 电脑发现新硬件

此时，我们打开我的电脑，即可找到可移动磁盘，如果有 TF 卡插入，那么会显示 3 个磁盘：ALIENTEK 磁盘、NANDDISK 磁盘和 TF 卡磁盘。如果 TF 卡没插入，则只显示

ALIENTEK 磁盘和 NANDDISK 磁盘。

这样，我们就实现了开发板和电脑的 USB 连接，可以直接从电脑拷贝文件到开发板的 TF 卡、SPI FLASH Disk（即 W25Q256）和 NAND FLASH（FSNS8B004G）。

这里再次提醒大家，如非必要，不要往 SPI Flash Disk 写入数据！否则容易写坏 SPI FLASH。

78.2.14 网络通信

开发板板载了一个 10M/100M 自适应以太网接口，可以实现网络通信。本系统的网络通信，就是给大家演示开发板的网络通信功能。

本系统自带的网络通信具有如下特点：

1， 使用 LWIP 作为 TCP/IP 协议栈。

2， 支持 DHCP，当 DHCP 失败时，使用静态 IP（静态 IP 地址为：192.168.1.30）。

3， 自适应网线（支持交叉和直连网线）。

4， 支持 Web Server 测试，可通过浏览器，控制开发板的 LED 和蜂鸣器等。

5， 支持 TCP Client、TCP Server 和 UDP 等测试。

特别注意：本测试，必须在开发板网口接入网线，并连接正常后，才可以进行测试。也就是必须用网线连接开发板和电脑/路由器，才可以进行测试，否则，系统会提示网卡初始化失败，从而退出测试!!!

双击主界面的网络通信图标，开始网卡初始化，在网卡初始化成功后，开始 DHCP 获取 IP 地址，在 DHCP 成功后，进入网络通信主界面，如图 78.2.14.1 所示：

图 78.2.14.1 初始化网卡和网络通信主界面

上图中，左侧图片显示正在初始化网卡，此时，我们需要在开发板的网口插入网线，并连接电脑或者路由器。初始化网卡成功后，则开始 DHCP 获取 IP 地址：

1，开发板连接到路由器，此种方式，DHCP 一般可以成功获取 IP 地址。

2，如果是直接连接电脑，那么 DHCP 肯定失败，最终会使用静态 IP 地址：192.168.1.30，此时需要设置电脑 IP 地址为 192.168.1.XXX，其中 XXX 可以由用户自己随意设置（但是不能是 1 和 30）。

这里我们以连接路由器为例，DHCP 获取成功后，进入网络通信主界面，如图 78.2.14.1 右侧图片所示，该界面显示了开发板网卡的详细设置和参数，包括：本机 MAC 地址、本机 IP 地址（DHCP 获取）、子网掩码、网关和网速等。

能进入网络通信主界面，说明开发板与路由器的连接已经正常了，可以在电脑端 ping 屏幕显示的 IP 地址，即可查看网络是否连接正常，如图 78.2.14.2 所示：

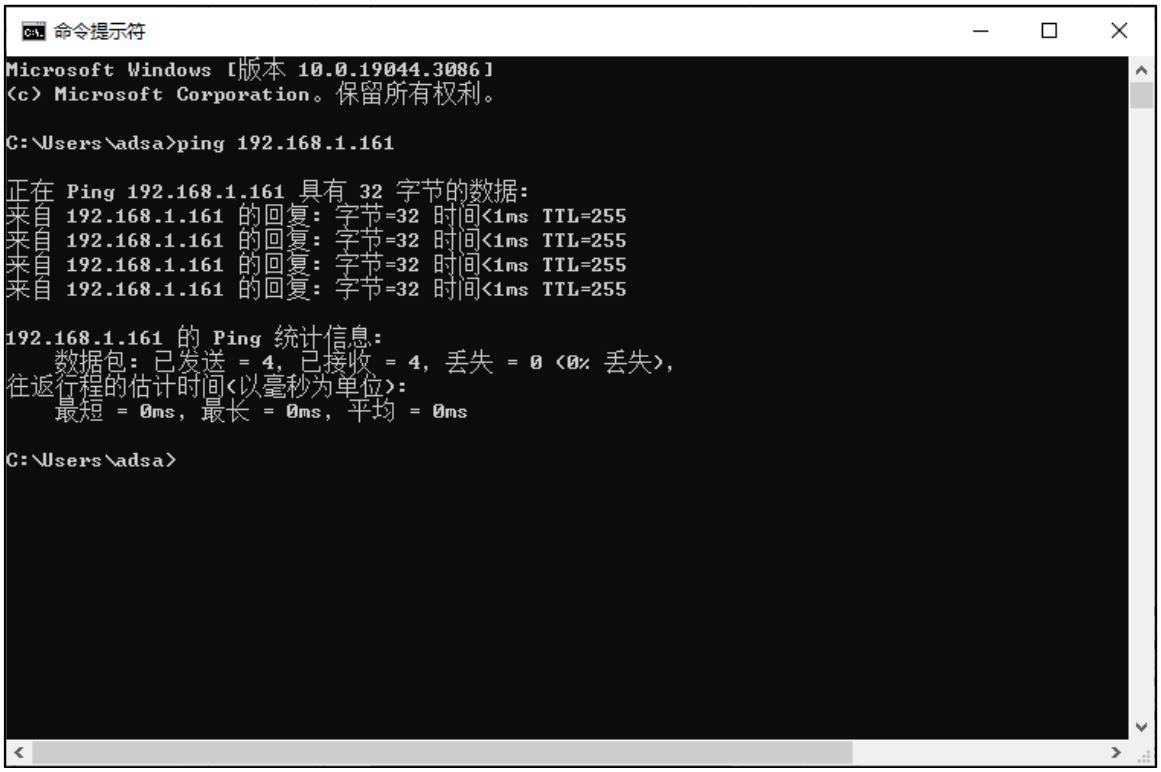


图 78.2.14.2 ping 192.168.1.161

从上图可以看出，ping 开发板 IP 地址是成功的，说明网络连接正常了。然后在浏览器输入开发板的 IP 地址，即可登录 Web Server 的主界面，如图 78.2.14.3 所示：

图 78.2.14.3 Web Server 主界面

在上图所示的 Web Server 主界面，可以实现对开发板 LED 和蜂鸣器的控制，以及读取 ADC1 通道 1 的值、温度传感器值、RTC 时间和日期等。**特别注意：ADC1_CH1 所在通道刚好是 TPAD 的输入通道，所以，本测试读取 ADC1_CH1 的电压值，是不正确的，因为 IO 被 TPAD 占用了!!! 如需要正确读取，请参考 LWIP 开发指南.pdf。**

点击图 78.2.14.1 右侧图片所示的开始测试，即可进入：TCP Server、TCP Client 和 UDP 的测试界面，如图 78.2.14.4 所示：

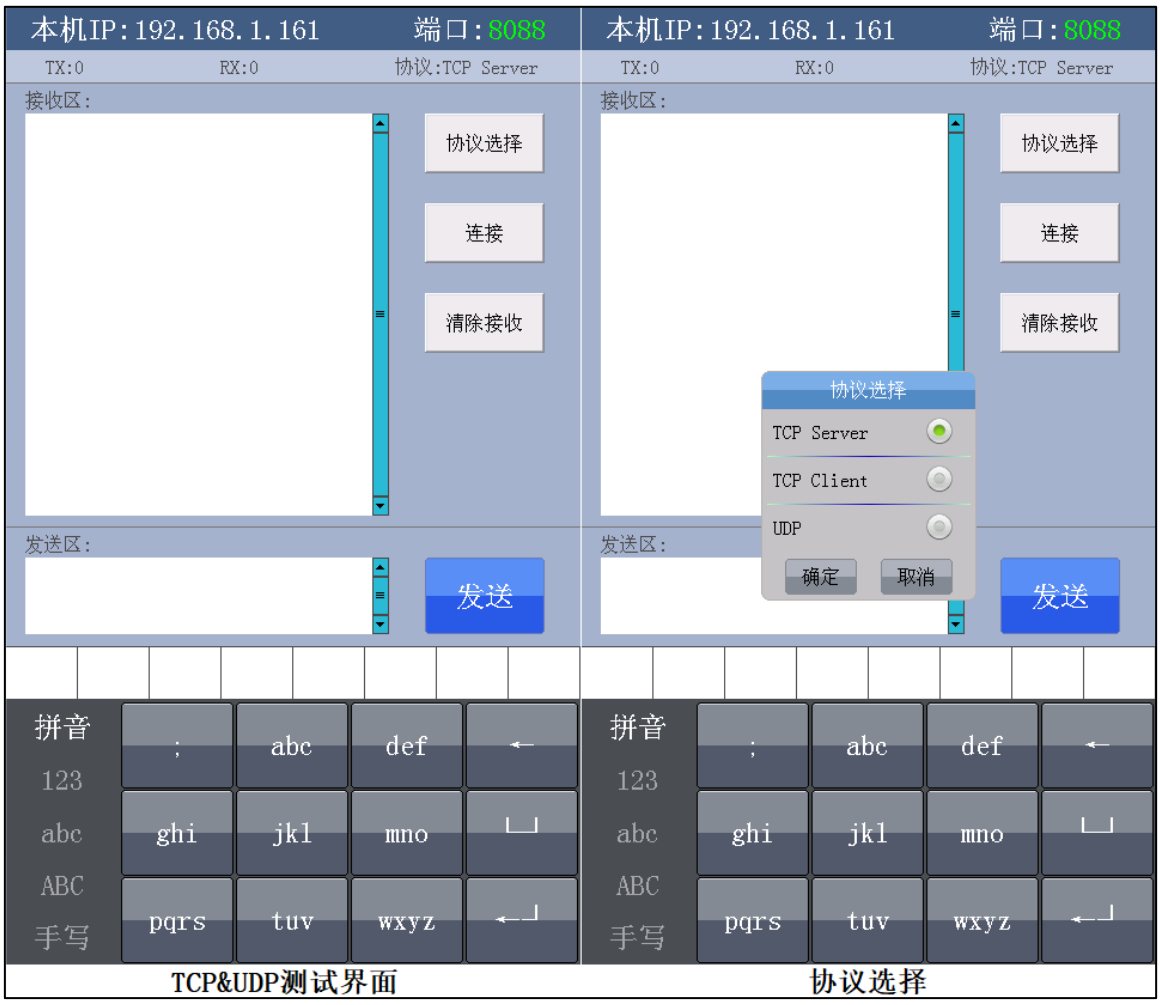


图 78.2.14.4 TCP&UDP 测试界面和协议选择

从图 78.2.14.4 左侧图片可以看出：最顶部，显示了 TCP Server 的本机 IP 地址和端口号，其中 IP 地址是白色，表示不可以设置；端口号是绿色，表示可以设置，设办法：触摸点击该区域，看到光标闪烁后，即可输入数字进行设置。随后，显示 TX，RX，和协议等三个信息，分别代表发送接收的数据量，和当前所使用的协议类型。

然后就是一个接收区和发送区，分别用于显示接收到的数据，和发送的数据。这个同电脑端的网络调试助手一样。右边的协议选择按钮，可以选择不同协议（如图 78.2.14.4 右侧图片所示），该选择按钮只有在连接断开的时候，才有效。连接按钮，用于启动连接（TCP Server/TCP Client 和 UDP 等），连接结果会有对话框提示（成功或者失败）。清除接收按钮，则可以清除接收区的所有数据，同时清除 RX 和 TX 计数器。发送按钮，则用于发送数据，每按一次，发送区的数据就发送一次。

进入 TCP&UDP 测试后，协议默认选择的是 TCP Server，可以通过协议选择按钮，选择不同协议（TCP Server、TCP Client 和 UDP），如上图右侧图片所示。

我们使用默认的 TCP Server 端口号（8088），然后点击链接，提示：连接成功后，即可开启开发板的 TCP Server 服务，然后在电脑端，打开网络调试助手，设置正确的网络参数后（**如不懂设置方法，请参考《LWIP 开发指南》对应测试部分，下同!!**），即可连接上开发板的 TCP Server，并互相通信，如图 78.2.14.5 所示：

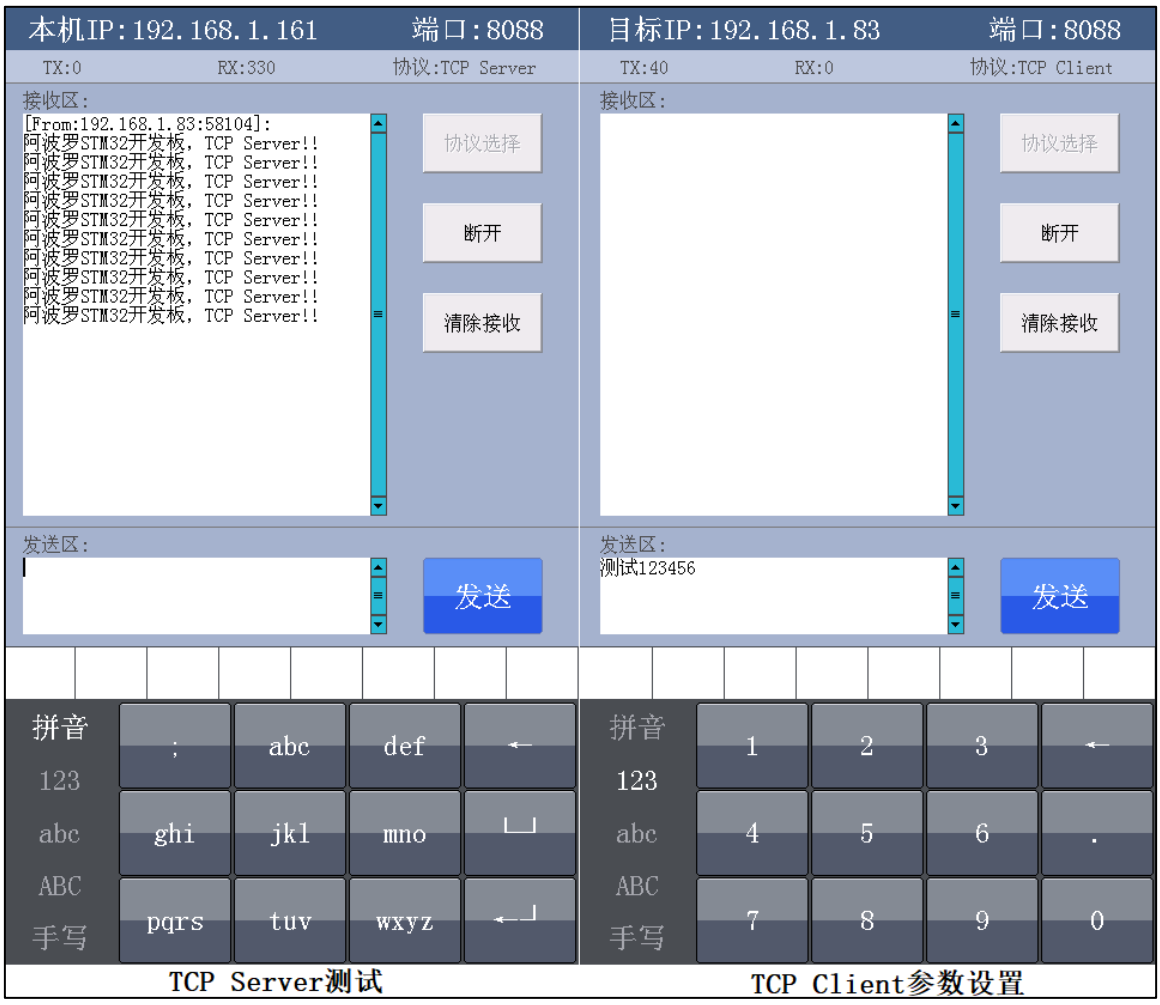


图 78.2.14.5 TCP Server 测试和 TCP Client 参数设置

上图中，左侧图片为 TCP Server 测试界面，可以看到，我们收到来自电脑的数据，接收数据时，首先会在接收区提示收到的数据来自何处（即电脑端的 IP 地址，本例可以看出，电脑 IP 地址为：192.168.1.83），随后才是电脑端发送过来的数据。同时，我们也发送了一些数据给电脑（电脑端网络调试助手可以看到，这里就不截图出来了）。此时，协议选择按钮变为了灰色，处于无效状态。只有在断开连接后，才可以选择新的协议。

TCP Client 的测试如图 78.2.14.5 中右侧图片所示，我们选择了 TCP Client 协议之后，顶部 IP 变为了目标 IP，也就是 TCP Client 要连接的 IP 地址，我们可以根据自己需要设置对应的 IP 地址和端口号。这里，我们设置为：192.168.1.83，端口为：8088。注意：这里的 IP 和端口号，要根据自己电脑的实际情况修改。随后需要在电脑端开启网络调试助手，并设置正确的网络参数（主要是端口号和 IP 地址），然后开启 TCP Server 服务。

TCP Client 测试必须等电脑端 TCP Server 服务开启后，我们才可以在开发板点击连接，并连接成功，否则肯定是连接失败的。连接成功后，TCP Client 的测试如图 78.2.14.6 所示：

图 78.2.14.6 TCP Client 测试和 UDP 测试

上图中，左侧为 TCP Client 测试界面，右侧为 UDP 测试界面，这两个测试界面和 TCP Server 差不多，请参考前面的介绍。

UDP 的测试，同 TCP Client 基本一样，也是先设置目标 IP 和端口号，然后按连接，进行测试。UDP 测试的时候，一般需要开发板先发送一次数据给电脑端的网络调试助手，随后才可以实现数据互发。UDP 测试界面如图 78.2.14.6 右侧图片所示。 注意：UDP 不是基于可靠连接的通信，所以程序提示连接成功的时候，仅仅是个提示作用，并不是说就一定连接上了目标 IP 和端口号，因此，不一定能成功发送数据给对方，这个在使用的时候，要注意。

网络通信就为大家介绍到这里。

78.2.15 无线传书

该功能用来实现两个开发板之间的无线数据传输，在开发板 A 输入的内容，会在开发板 B 上完整的“复制”一份，该功能需要 2 个阿波罗 V2 开发板（也可以一个阿波罗板与战舰板或精英板或 Mini 板搭配用，不过都要刷综合实验!!）和 2 个 NRF24L01 无线模块。

双击主界面的无线传书图标（假定开发板已插上 NRF24L01 无线模块），会先弹出模式选择对话框，如图 78.2.15.1 所示：

图 78.2.15.1 模式选择和发送模式

从左侧的图片可以看出，模式设置，我们可以设置为发送模式或接收模式。右侧的图片则是选择发送模式后进入的界面。我们在另外一块开发板（开发板 B）设置模式为接收模式，然后在本开发板（开发板 A）手写输入一些内容，就可以看到在另外一个开发板也出现了同样的内容，如图 78.2.15.2 所示：

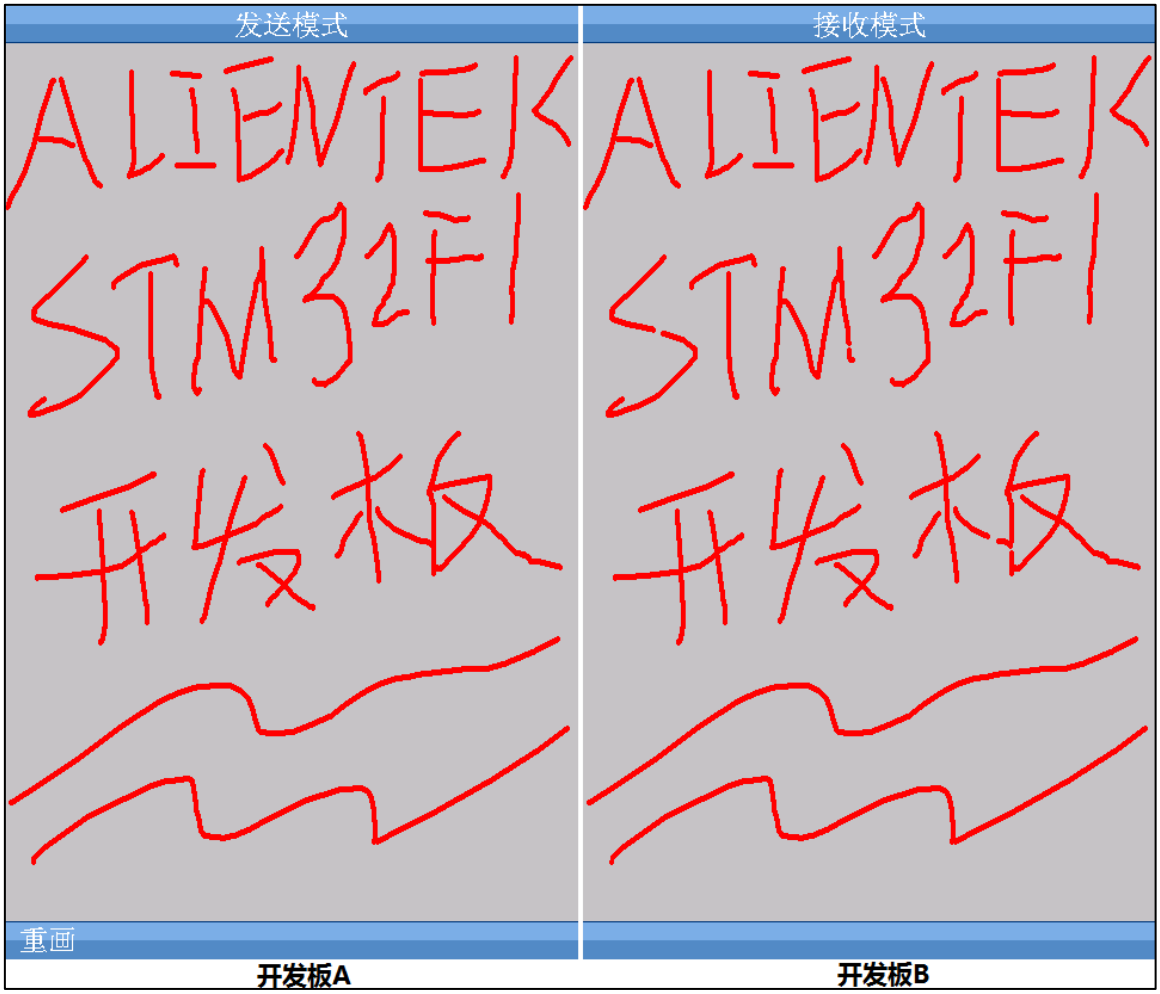


图 78.2.15.2 在开发板 A 输入的内容完整的显示在开发板 B 上

从上图可以看出，在开发板 A 上输入的内容，被完整的复制到开发板 B 上了。这就是无线传书功能。

78.2.16 计算器

本开发板实现了一个简单的科学计算器，可以计算加减乘除、开方、平方、M^N 次方、正弦、余弦、正切、对数、倒数、格式转换等一些常见的计算器功能，精度为 12 位，支持科学计数法表示。双击主界面的计算器图标，进入计算器主界面，如图 78.2.16.1 所示：

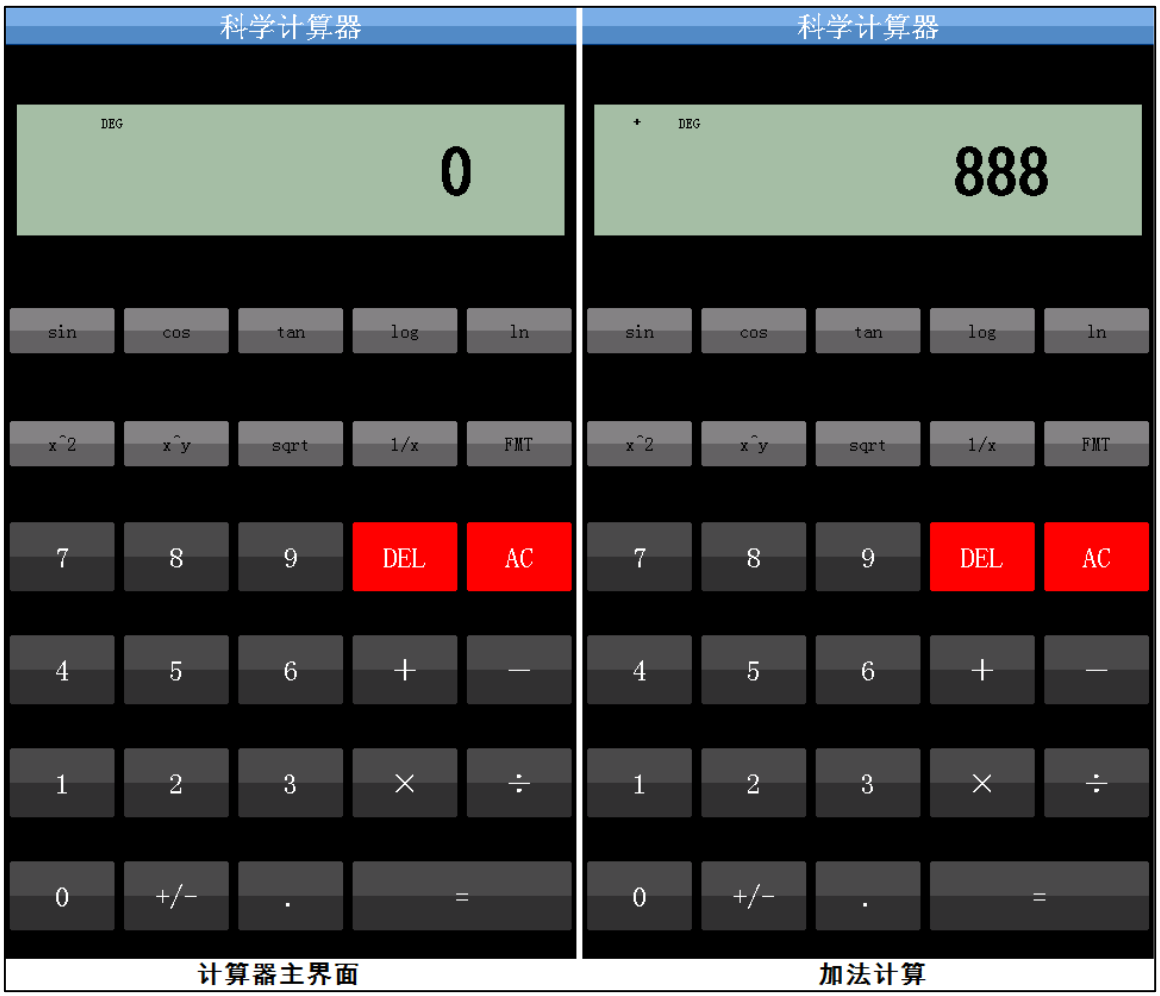


图 78.2.16.1 计算器主界面和加法计算

上图中，左侧的图片为科学计算器的主界面，和我们手机用的计算器基本一样，使用上非常简单，我们就不详细介绍了。右侧的图片为加法计算，支持累加功能。

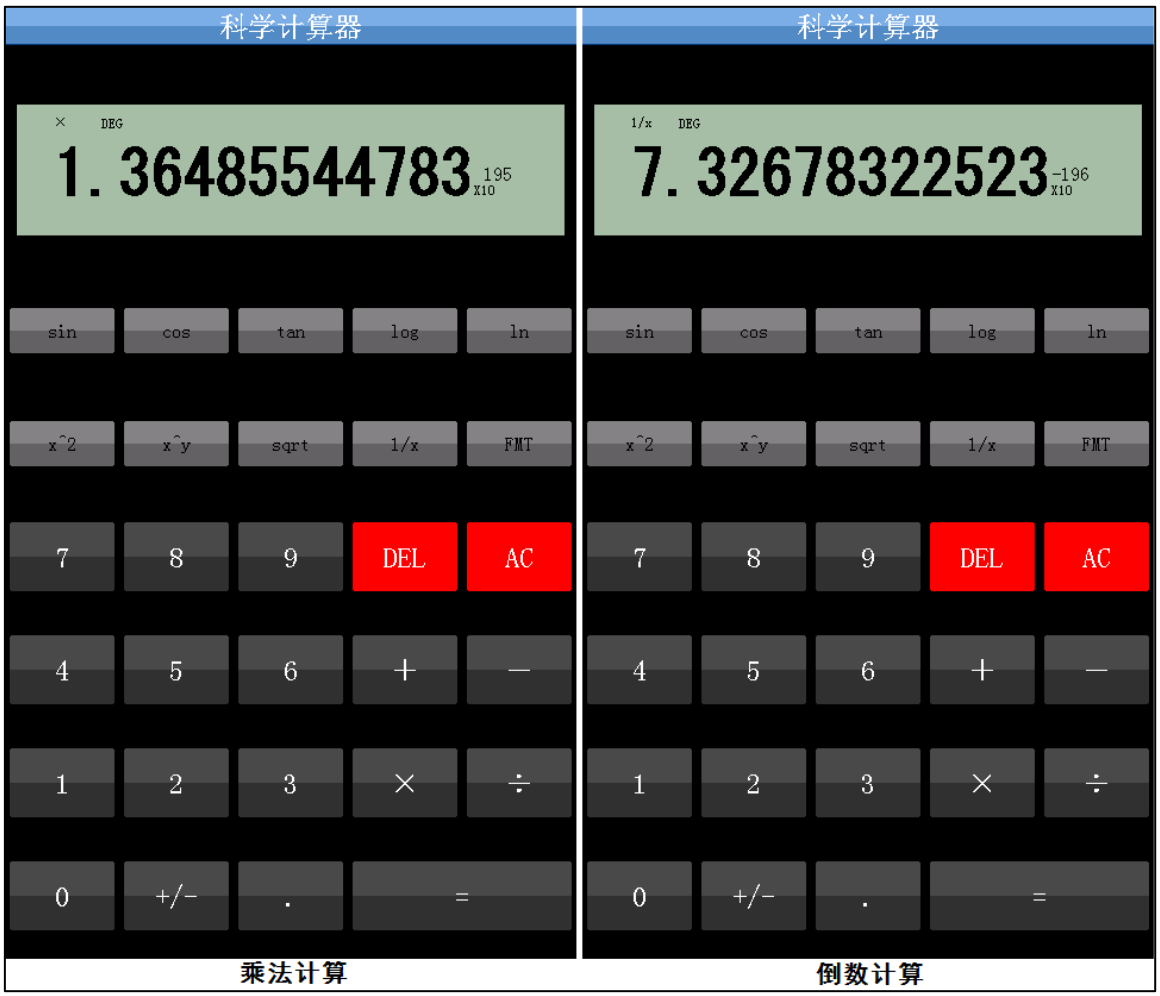


图 78.2.16.2 计算器主界面和加法计算

上图为乘法计算和倒数计算，可以看到，结果是以科学计数法表示的，最大支持 200 位指数表示，超过范围直接显示错误（E）。

该计算器还支持格式转换（按 FMT 键），可以将十进制数据（最大为 65535，超过部分将被丢弃）转换为 16 进制/二进制数据表示，如图 78.2.16.3 所示：

图 78.2.16.3 格式转换

上图显示我们将十进制的 65535 转换为 16 进制/二进制后的表示。计算器的其他功能，我们就再列举了，感兴趣的朋友可以慢慢摸索，当然也可以在这个基础上进行改进。通过按 TPAD 可以返回主界面。

78.2.17 二维码

阿波罗 STM32H743 开发板可以用作二维码识别和二维码生成，我们在综合实验集成了这两个功能。

二维码识别：由正点原子提供一个二维码识别库（ATK_QR_V1.4.lib），它可以实现二维码和条形码的识别。其特点包括：

1， 支持 UTF8 和 GBK 两种编码方式的二维码识别。

2， 支持 CODE128、CODE39、I25 和 EAN13 等四种编码方式的条码识别。

3， 支持 UTF8-OEM 转换输出(需要客户自己提供转换码表)。

二维码生成：支持 GBK 编码方式的二维码生成（不支持 UTF8 编码）。

双击二维码图标，首先进行模式选择，如图 78.2.17.1 所示：

图 78.2.17.1 模式选择和待识别的二维码&条形码图案

模式选择可以选择：识别二维码或者生成二维码，首先我们看识别二维码，上图右侧有一个待识别的二维码和条形码图案。选择识别二维码后，首先会初始化 OV5640，如果没有 OV5640 则直接退出，无法进行识别。在初始化 OV5640 成功后，进入识别状态，此时屏幕中央会显示摄像头拍摄到的图案，将二维码/条形码放到摄像头区域，就会自动进行识别，识别完成，会有“滴”的一声提示，同时屏幕弹出对话框，显示识别结果。如图 78.2.17.2 和图 78.2.17.3 所示。

特别注意：

1)，二维码识别需要 ATK-OV5640 摄像头模块的支持，如果没有，则无法进入此功能。

2)，OV5640 默认开启了自动对焦，如果发现失焦，我们可以触摸屏幕（按住一段时间再松开），手动触发一次对焦，以获得清晰的图片。

3)，要识别的图像要全部在扫描区内，才可以识别。

4)，对于条形码，最好大于等于 7 个字符/数字长度，否则可能无法识别。

5)，二维码一次最多识别约 512 汉字大小的编码图片。

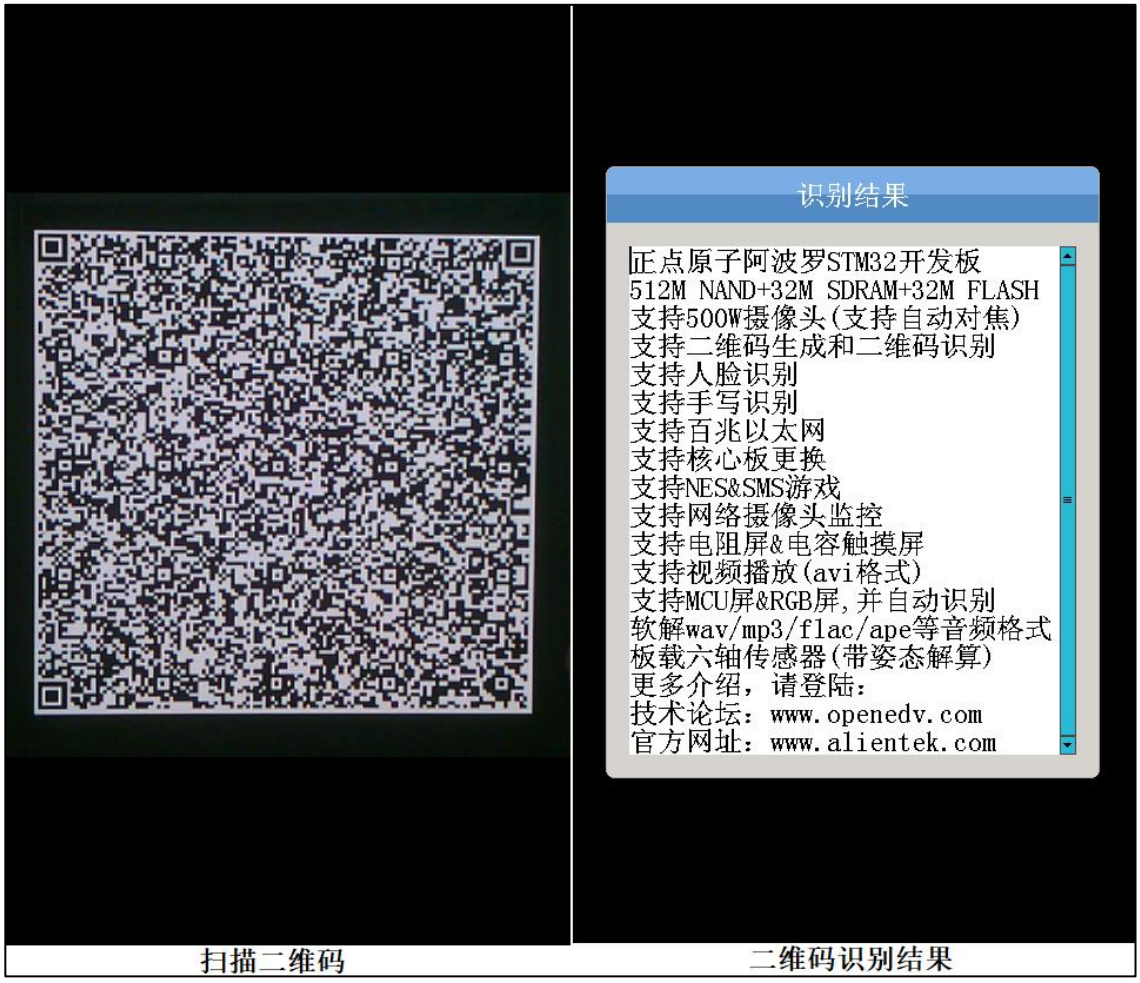


图 78.2.17.2 二维码扫描和二维码识别

图 78.2.17.3 条形码扫描和条形码识别

我们按 TPAD，可以返回主界面，结束二维码识别。接下来，我们看二维码编码，在模式选择界面，选择生成二维码，然后如图 78.2.17.4 所示：

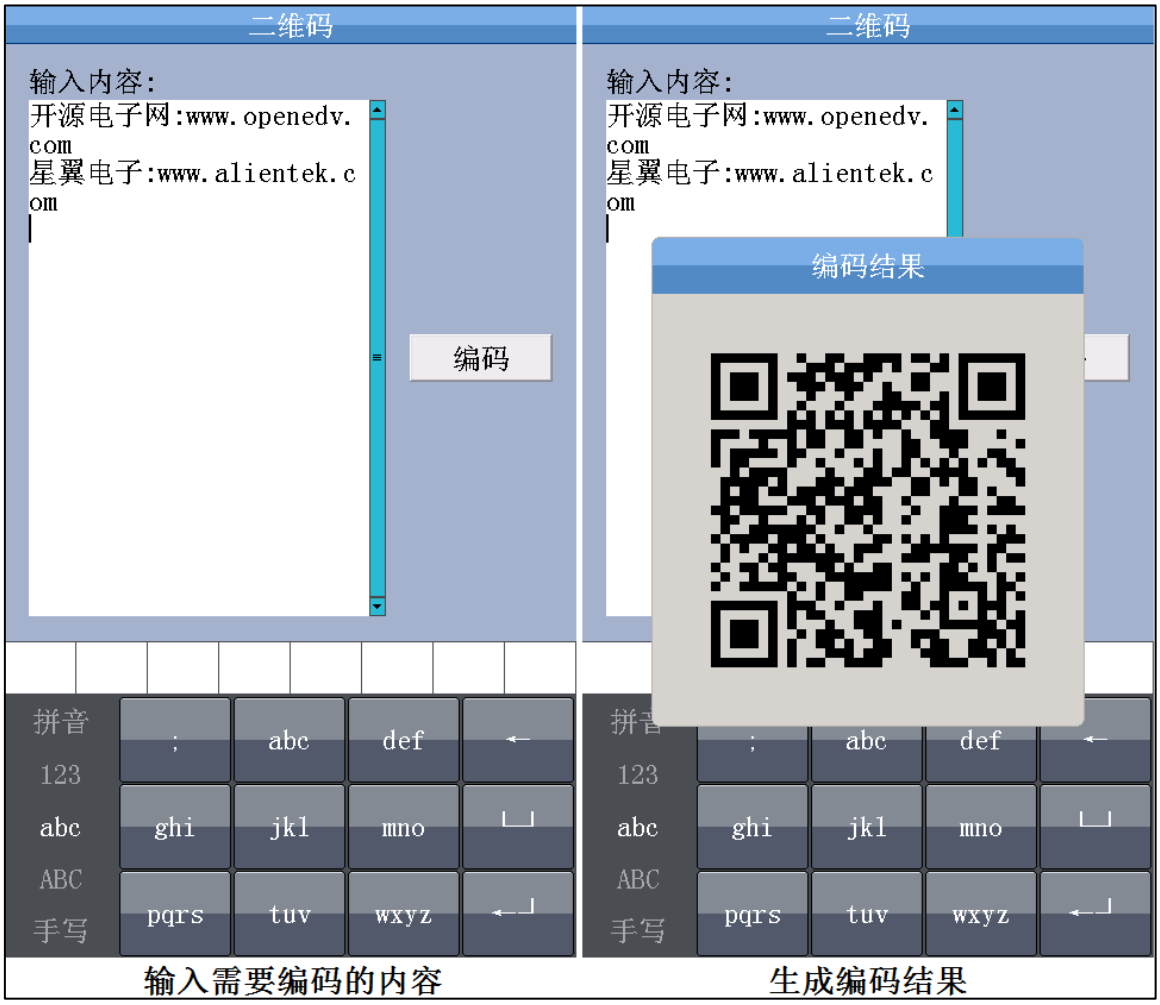


图 78.2.17.4 输入编码内容和生成编码结果

首先，我们输入需要编码的内容，如图 78.2.17.4 左侧图片所示，在输入完成后，点击“编码”按钮，即可生成编码结果，如图 78.2.17.4 右侧图片所示。然后，我们可以用手机扫描这个二维码，来验证是否编码正常。按 TPAD 可以清除编码结果，重新回到输入编码内容界面，我们可以输入新的内容，然后重新编码。在编码界面，按 TPAD，可以返回主界面，结束二维码生成应用。

注意：这里的二维码编码采用的是 GBK 码，并不是 UTF8 编码，所以有的扫描软件，可能无法识别其中的汉字。遇到这种情况，请用微信自带的扫一扫功能，来扫描。

78.2.18 IP 摄像头

双击主界面的 IP 摄像头图标，即可进入本应用。IP 摄像头实际上就是一个网络摄像头，可以用作网络视频监控，该功能需要上位机软件的配合（ATK-XCAM.exe，软件路径：光盘 A 盘→6，软件资料→1，软件→串口&网络摄像头软件）。该网络摄像头应用的功能如下：

1， 采用 JPEG 图像传输，图像质量好，帧率高。
2， 支持图像分辨率设置，支持的分辨率有：320*240、640*480、800*600、**960*640（默认）**、1024*768 和 1280*800。
3， 支持自动对焦，并可以手动触发对焦。
4， 支持最高 15 帧的图像传输（分辨率小于等于 960*640 时才可以 15 帧，否则为 7.5 帧）。
5， 支持端口设置。

特别注意：本测试，**必须在开发板网口接入网线**，并连接正常后，才可以进行测试。也

就是必须用网线连接开发板和电脑/路由器，才可以进行测试，否则，系统会提示网卡初始化失败，从而退出测试!!另外，需要有 ATK-OV5640 摄像头模块才可以开启本应用!!!

双击 IP 摄像头图标，首先进行 OV5640 摄像头的初始化，如图 78.2.18.1 所示：

图 78.2.18.1 初始化 OV5640&网络摄像头主界面

在 ATK-OV5640 摄像头模块初始化成功后，开始网卡初始化，在网卡初始化成功后，开始 DHCP 获取 IP 地址，在 DHCP 成功后，进入网络摄像头主界面，如上图右侧图片所示。

右侧图片，显示了我们网络摄像头功能所使用的协议（TCP 协议），开发板做 TCP Server，开发板的 IP 地址为：192.168.1.161（由 DHCP 分配），并且端口号默认是 8088（可以设置为其他端口号）。此时，摄像头的分辨率是 960*640，帧率是 15 帧，因为还没有和电脑进行数据传输，所以速度是 0KB/S。

图中端口号 8088 是蓝色，并且有光标闪烁，说明可以设置，我们可以通过下方的键盘进行设置。设置完成后，点击“开启”按钮，即可开启 TCP Server，此时端口号变成白色，表示不可设置。注意：关闭 TCP Server 后，可以重新设置端口号。

当开发板开启 TCP Server 以后，我们需要在电脑端打开 ATK-XCAM.exe 这个软件（路径：A 盘→6，软件资料→1，软件→串口&网络摄像头软件），然后选择网络，设置为 TCP 客户端模式，Server IP 为 192.168.1.161（即主界面显示的 IP 地址，根据实际情况填写），端口号为 8088，设置完成以后，点击“开启”按钮，启动数据传输，如图 78.2.18.2 所示：

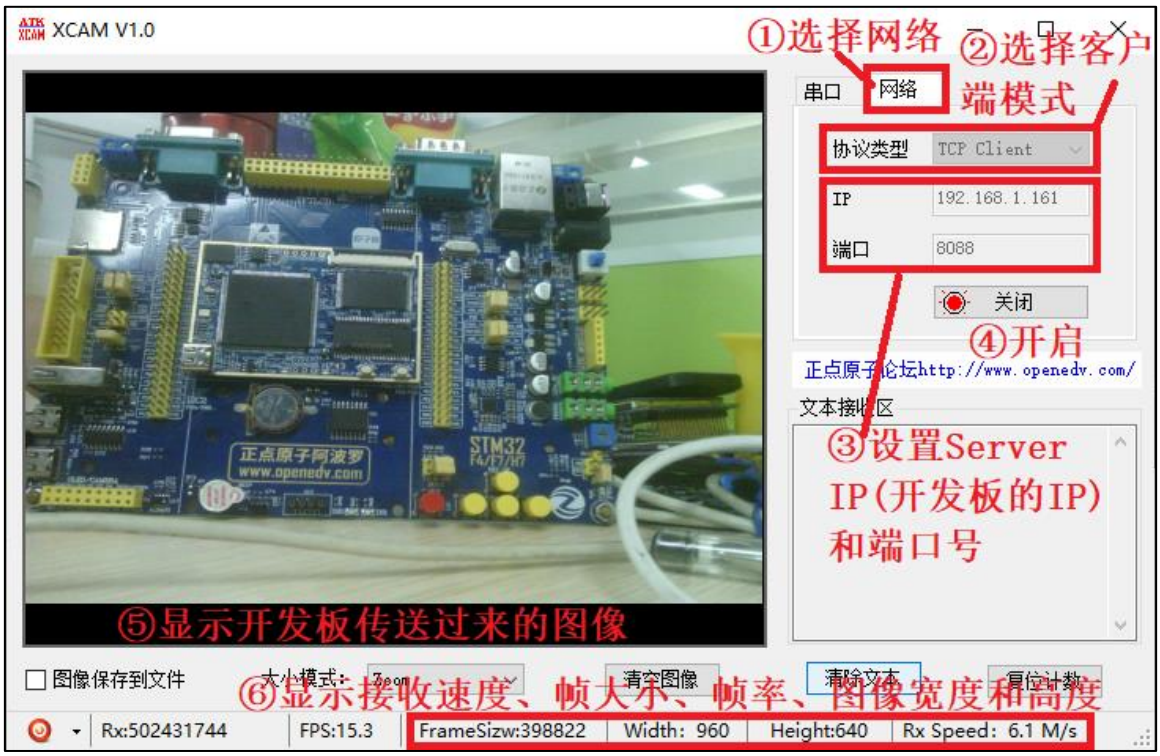


图 78.2.18.2 上位机软件（ATK-XCAM.exe）启动网络连接

在启动完成以后，上位机软件就开始显示开发板传递过来的图像数据，图片默认是 960*640 分辨率，帧率是 15 帧的。点击分辨率，可以修改分辨率大小，如图 78.2.18.3 所示：

图 78.2.18.3 放大图片显示

上图中，图片分辨率为 1280*800，帧率为 7.5 帧，单个图片为 3000KB 左右。图片分辨率通过主界面的“分辨率”按钮进行切换（切换范围：320*240、640*480、800*600、**960*640（默认）**、1024*768 和 1280*800），按一次切换到下一个分辨率，如图 78.2.18.4 所示：

图 78.2.18.4 图像传输和切换分辨率

上图左侧的图片，显示分辨率为 960*640 的图像正在传输中，网络传输速度为 2MB/S，帧率为 9 帧（实际是 15.3 帧）。我们可以按“分辨率”按钮，切换不同的分辨率，如上图右侧图片所示，显示分辨率为 1280*800，网络传输速度为 1.2MB/S，帧率为 4 帧（实际上是 7.5 帧）。

当发现图像失焦以后，我们可以按“对焦”按钮，手动触发一次自动对焦，从而获得清晰的图像。

按 TPAD 可以退出网络摄像头功能，回到主界面。

78.2.19 人脸识别

双击主界面的人脸识别图标，即可进入本应用。本应用需要 ATK-OV5640 摄像头模块的支持，如果没有摄像头，则无法打开本应用，本应用采用正点原子提供的人脸识别 LIB（ATKFREC.lib）来实现人脸识别，本人脸识别 LIB 的特点有：

1， 占用内存少，20 张人脸，仅需约 560KB 内存。

2， 使用简单，仅需修改 6 个函数，即可实现移植。

3， 识别速度快，20 张人脸里面，识别一张人脸，耗时约 0.5 秒（STM32H743@216Mhz）。

不过，由于本识别库以 M3/M4 为目标处理器，内存有限，算法上进行了大量阉割。所以，很多功能不太完善，效果也不是很好，且没有做识别有效检查(输入错误的人脸,也会有结果输出)，所以，此 LIB，仅供大家参考用。

双击主界面的人脸识别图标后，首先会初始化 OV5640 摄像头模块，在初始化成功以后，进入人脸扫描界面，如图 78.2.19.1 所示：

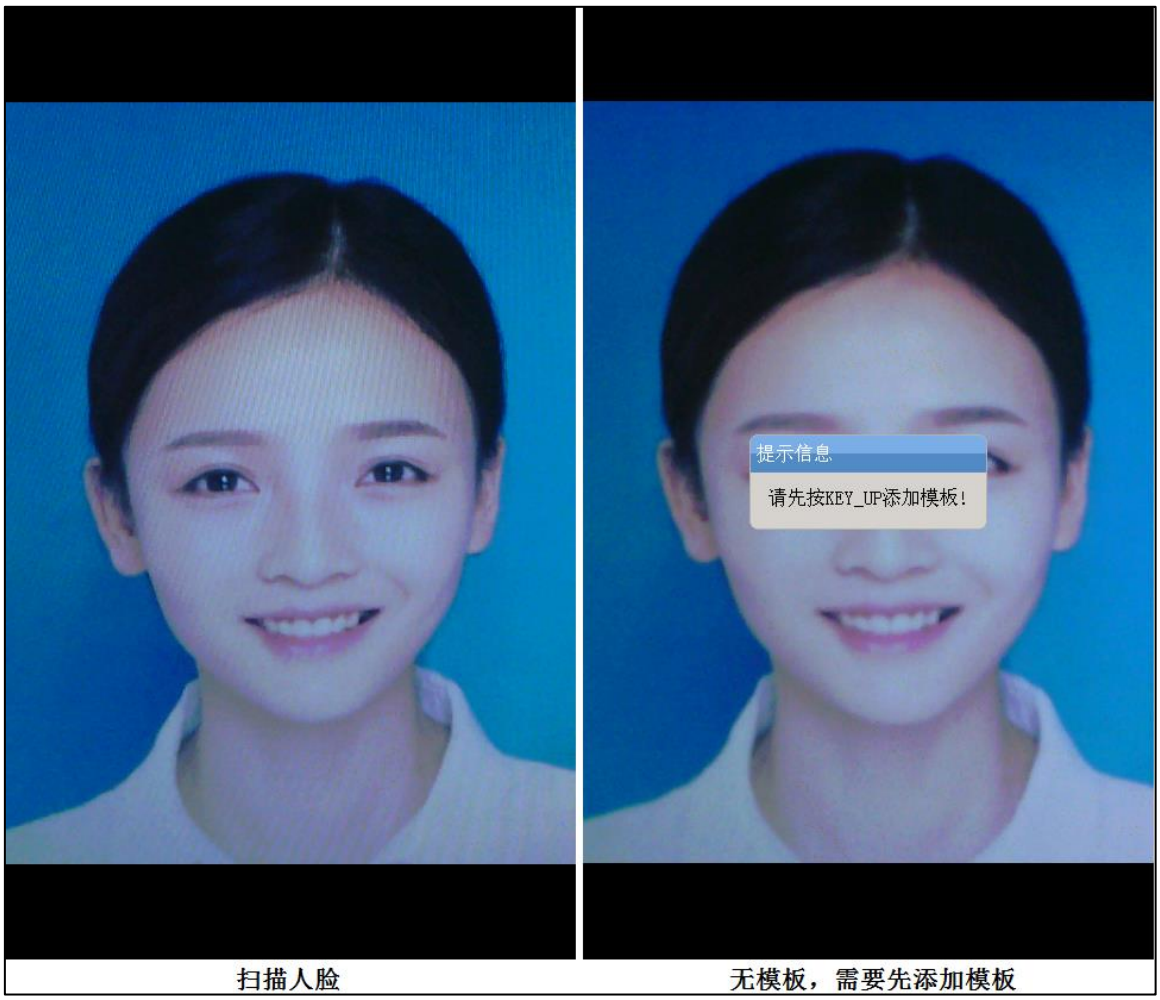


图 78.2.19.1 扫描人脸和模板添加提示

上图左侧图片显示识别程序正在扫描人脸，我们可以调节摄像头，将人脸放到屏幕正中央，以便程序识别，当发现人脸模糊，对焦不清的时候，轻触屏幕触摸屏，即可手动触发一次自动对焦，以获得清晰的图像。

当调整好人脸位置以后，按 KEY0 按键，可以对当前扫描的人脸进行识别，如上图右侧图片所示，不过由于我们默认是没有模板的，所以无法进行识别（至少需要有一个模板，才可以进行人脸识别），屏幕提示要按：KEY_UP 按键添加模板，所以，我们按 KEY_UP 按键，先添加一些模板，如图 78.2.19.2 所示：

图 78.2.19.2 添加人脸和查看人脸列表

上图左侧图片就是添加人脸的界面，我们可以对当前人脸的：姓名、电话和性别等信息进行编辑，编辑完成后，点击保存，即可将该人脸数据保存到 NAND FLASH 磁盘里面（NANDDISK 磁盘根目录：ATK_FREC 文件夹内），以便后续识别使用。

综合实验，我们最多可以添加 20 个人脸，当添加人脸完成以后，按 KEY2 按键，可以弹出人脸列表，如上图右侧图片所示，由图可知，总共有 20 个人脸，我们可以双击某个人的名字，对这个人的信息进行编辑，也可以按左下角的“选项”按钮，弹出对话框，选择“编辑”，对该人脸进行编辑，如图 78.2.19.3 所示：

图 78.2.19.3 选项对话框和编辑人脸

上图中，左侧图片表示我们按下“选项”按钮后，弹出一个选择操作的对话框，可以选择编辑和删除，如果选择编辑，则和双击人脸列表的姓名一样，进入编辑人脸信息界面，如上图右侧图片所示。在编辑人脸信息界面，我们可以对当前人脸的姓名、电话和性别等信息进行修改，修改完成后，点击保存即可完成编辑。

删除人脸方法：在人脸列表里面，我们选中某个人脸（单击），然后按“选项”按钮，弹出选择对话框（如图 78.2.19.3 左侧图片所示），选择“删除”，然后点击确定，即可完成对这个人脸的删除，删除完成会有提示对话框，如图 78.2.19.4 所示：

图 78.2.19.4 删除人脸和识别结果

上图右侧图片显示人脸删除成功。在编辑/删除人脸完成以后，我们按 TPAD 或“返回”按钮，回到人脸扫描界面，然后调节摄像头（轻触屏幕的触摸屏，可以触发一次自动对焦，以获得清晰的图像），让待识别的人脸处于屏幕正中央，调整好位置以后，按 KEY0 按键，即可对当前图像进行识别，识别完成后，显示识别结果，如上图右侧图片所示。

在识别结果界面，上半部分的大图，是摄像头拍摄到的需要识别的图片，下面右侧的小图片，则是在添加模板的时候，保存的预览图片。可以看到，两个图片基本一样，说明识别是成功的，同时，还显示了这个人的姓名、电话和性别等，整个识别过程，耗时 570ms（20 张人脸），还是比较快的。在识别结果界面，按 TPAD，可以返回人脸扫描，继续识别其他人脸。

在人脸扫描界面，再按 TPAD 即可返回主界面，结束人脸识别。

78.2.20 9D 测试

阿波罗 STM32H743 开发板底板板载了 QMI8658A 六轴传感器（三轴加速度和三轴陀螺），可以直接输出姿态解算数据。9D 测试就是利用 QMI8658A，结合 IMU 库，完成 6 轴姿态解算，输出横滚角、俯仰角和航向角，并显示在液晶屏上。

双击主界面的 9D 测试图标，首先初始化 QMI8658A，如图 78.2.20.1 所示：

图 78.2.20.1 9D 测试主界面

在初始化 QMI8658A 成功以后，就会进入 9D 测试主界面，如上图右侧图片所示。在 9D 测试主界面，显示了六轴姿态解算后的俯仰角、横滚角和航向角。此时，我们调整开发板的位置，就可以得到不同的俯仰角、横滚角和航向角。

按 TPAD，可以返回主界面，结束 9D 测试。

78.2.21 水平仪

本应用，同样是利用板载 QMI8658A 六轴传感器，输出姿态解算后的横滚角和俯仰角，利用俯仰角和横滚角计算出当前倾角，来判断开发板是否处于水平位置，实现一个类似 IPHONE 指南针里面的水平仪效果，可以用来测量倾角。

双击主界面的水平仪图标，首先初始化 QMI8658A，如图 78.2.21.1 所示：

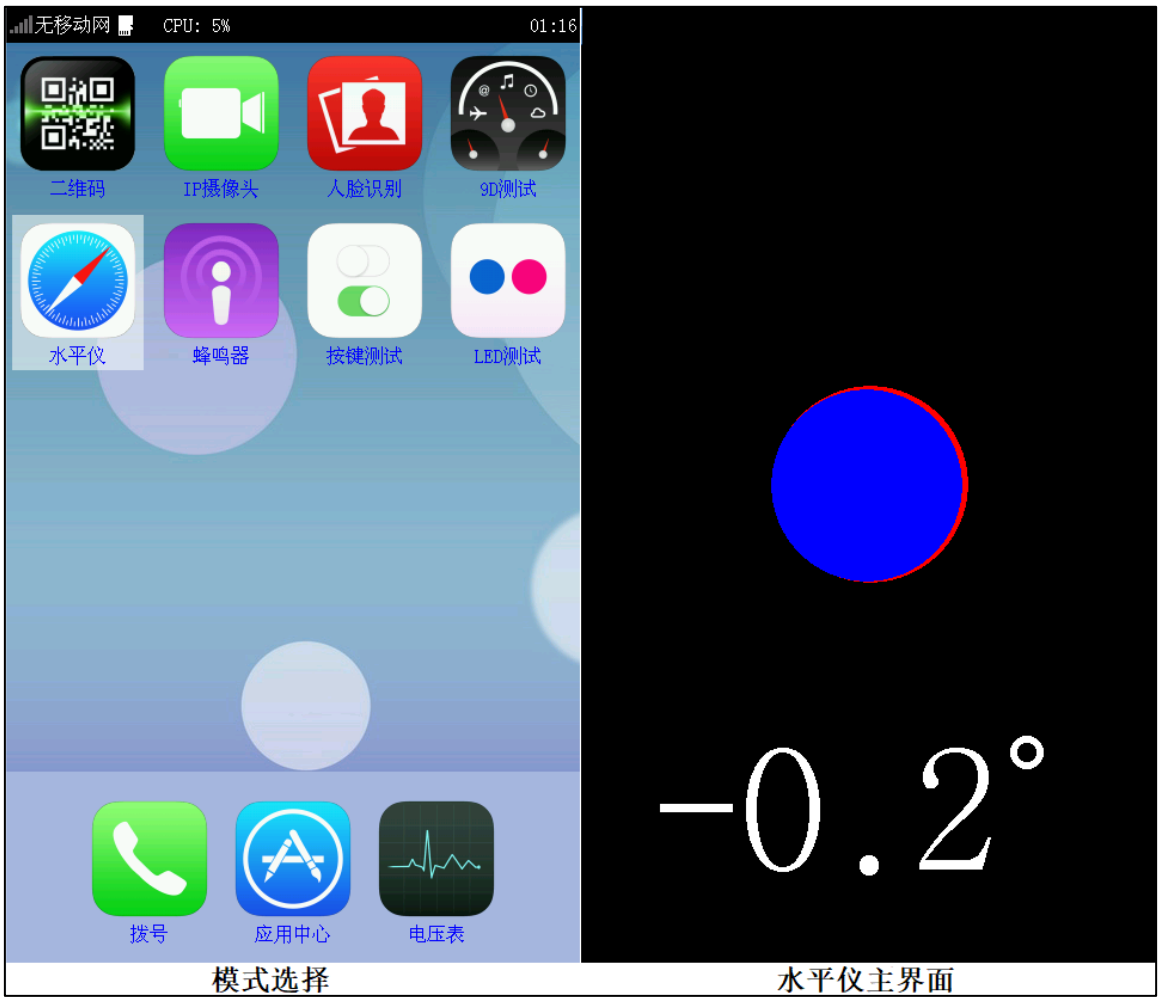


图 78.2.21.1 水平仪主界面

在初始化 QMI8658A 成功以后，就会进入水平仪主界面，如上图右侧图片所示。水平仪主界面显示了 2 个小实心圆和当前的倾角，调整开发板的位置，当两个小实心圆完全重叠时（倾角为 0° 或者接近 0° ），表明当前位置是水平的，此时背景色变为绿色，如图 78.2.21.2 所示：

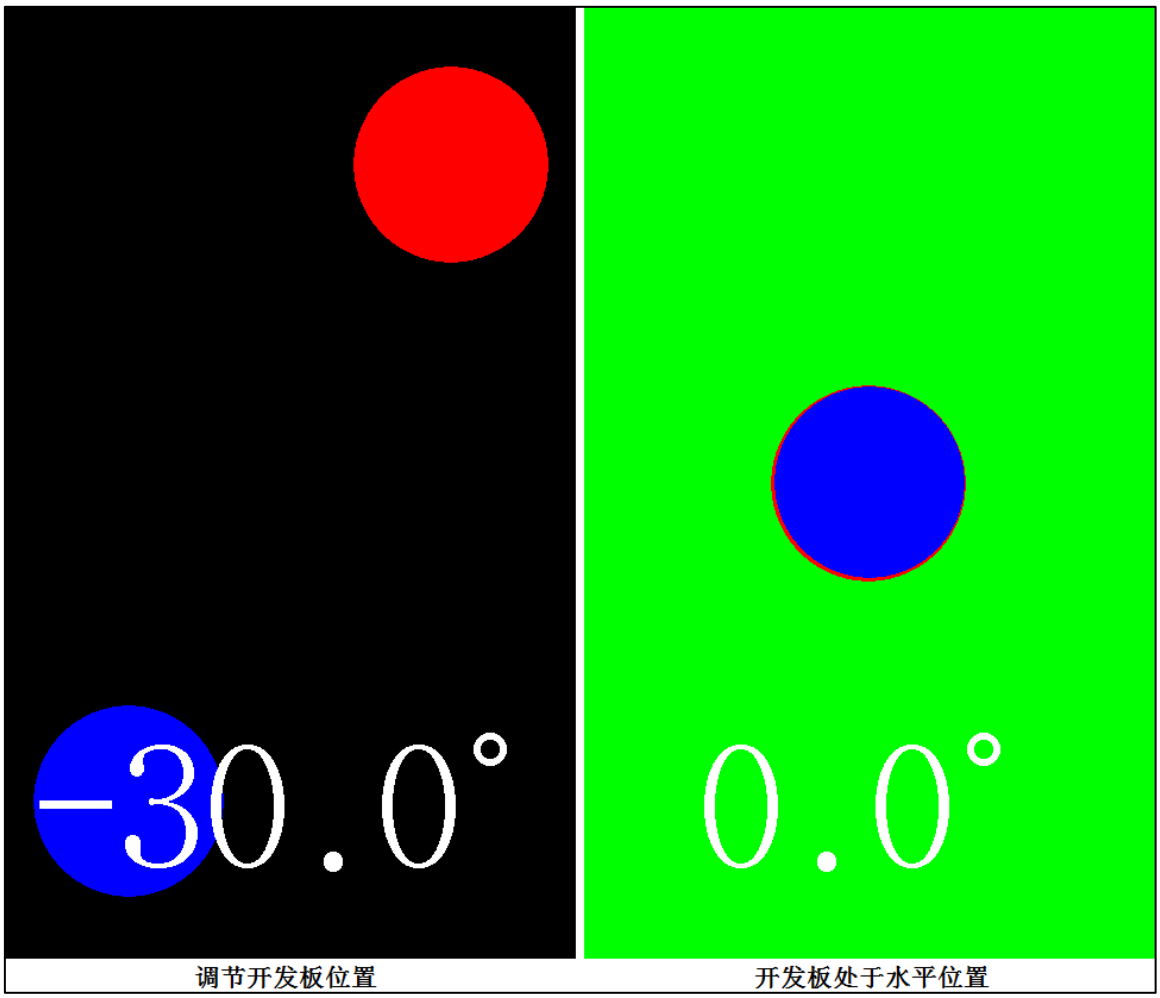


图 78.2.21.2 调节开发板位置和水平状态

按 TPAD，可以返回主界面，结束水平仪功能。

78.2.22 蜂鸣器

本应用，可以用于测试板载蜂鸣器的好坏，双击蜂鸣器图标，进入蜂鸣器测试，测试界面如图 78.2.22.1 所示：

图 78.2.22.1 蜂鸣器关闭和蜂鸣器打开

上图左侧为默认界面，默认蜂鸣器是关闭的，我们可以按“打开”按钮，将蜂鸣器开启，如上图右侧图片所示，当蜂鸣器打开后，可以听到蜂鸣器持续发出“滴”…的声音，同时按钮名称变为关闭，我们点击“关闭”按钮，既可以关闭蜂鸣器，回到上图左侧图片所示的界面。

按 TPAD，可以返回主界面，结束蜂鸣器测试。

78.2.23 按键测试

本应用，可以用于测试板载按键（KEY0、KEY1、KEY2 和 KEY_UP）的好坏，双击按键测试图标，进入按键测试，测试界面如图 78.2.23.1 所示：

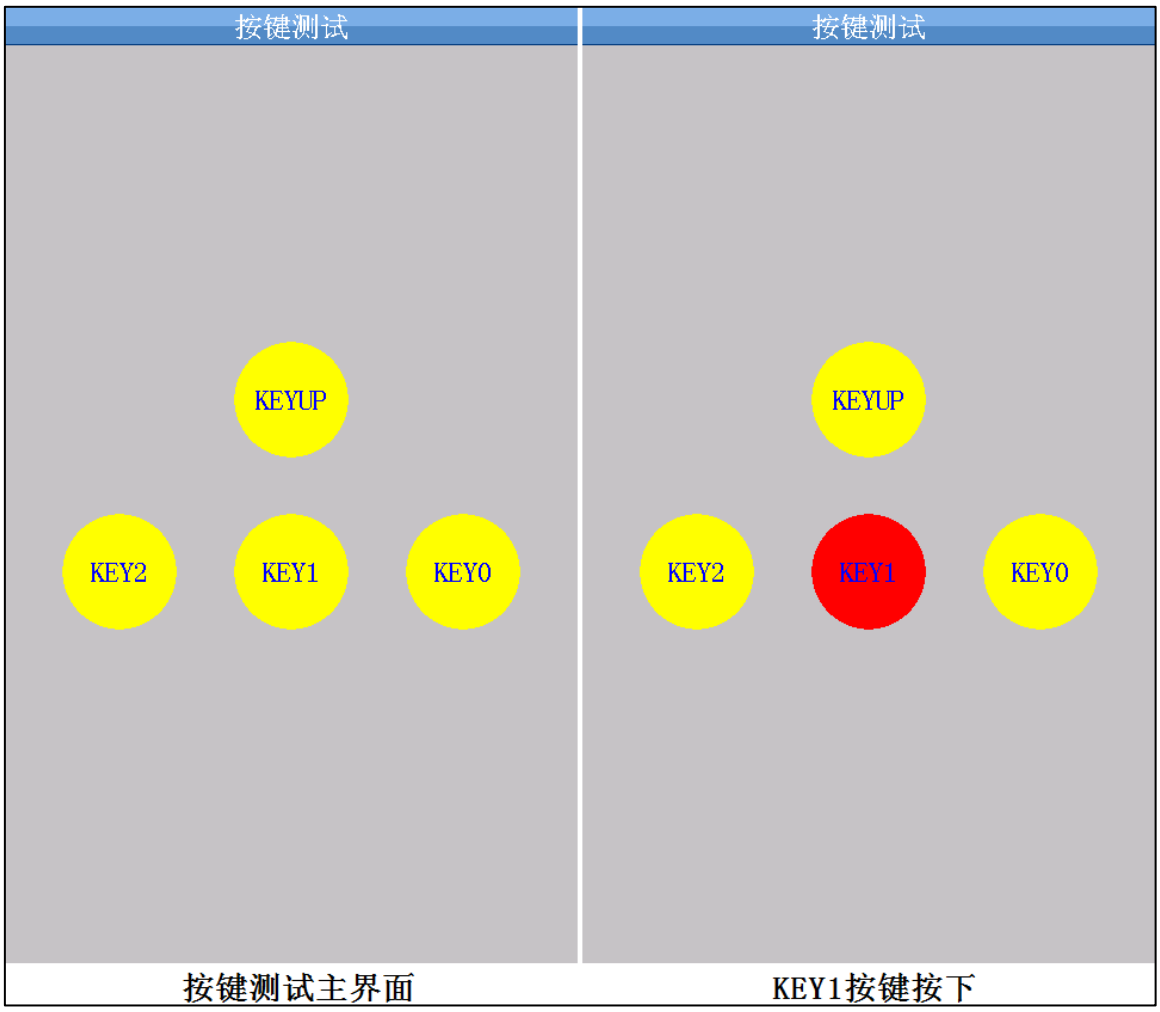


图 78.2.23.1 按键测试主界面和 KEY1 按下

图中左侧为按键测试主界面，默认四个按键都是黄色的底色，当检测到某个按键按下的时候，该按键的底色变成红色，如上图右侧图片所示，当按键松开后，该按键的底色又恢复黄色。这样，我们就可以测试开发板板载的四个按键的好坏。

注意：按键检测不支持同时按下多个按键，当多个按键同时按下时，最先按下的那个按键有效。

按 TPAD，可以返回主界面，结束按键测试。

78.2.24 LED 测试

本应用，可以用于测试板载 LED（DS0 和 DS1）的好坏，双击 LED 测试图标，进入 LED 测试，测试界面如图 78.2.24.1 所示：

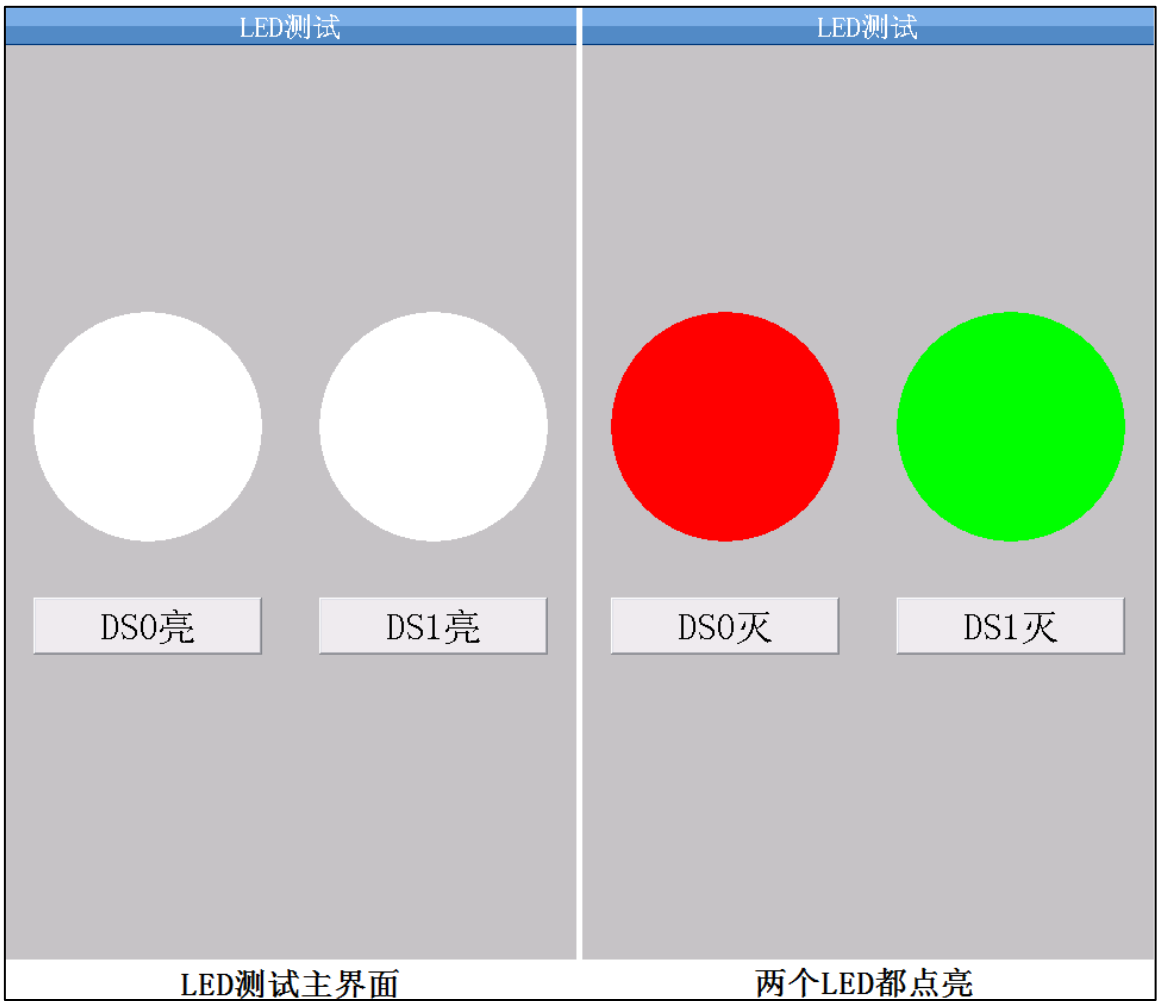


图 78.2.24.1 LED 测试主界面和两个 LED 都点亮

上图左侧为默认界面，默认两个 LED（DS0 和 DS1）都是关闭的，我们可以按“DS0 亮”和“DS1 亮”这两个按钮，分别点亮 DS0 和 DS1。当对应的 LED 点亮后，显示对应颜色的实心圆，同时按钮名称变为“DS0 灭”或“DS1 灭”，此时按钮的作用变为关闭 LED，如上图右侧图片所示。

按 TPAD，可以返回主界面，结束 LED 测试。

78.2.25 拨号

本综合实验支持拨打和接听电话，不过需要正点原子 ATK-MW510/ATK- MW196 模块的支持，所以本功能的测试，请先确保有 4G 模块，并连接成功（详见 78.2 节开头部分）。

双击主界面的拨号图标，进入拨号界面，如图 78.2.25.1 所示：

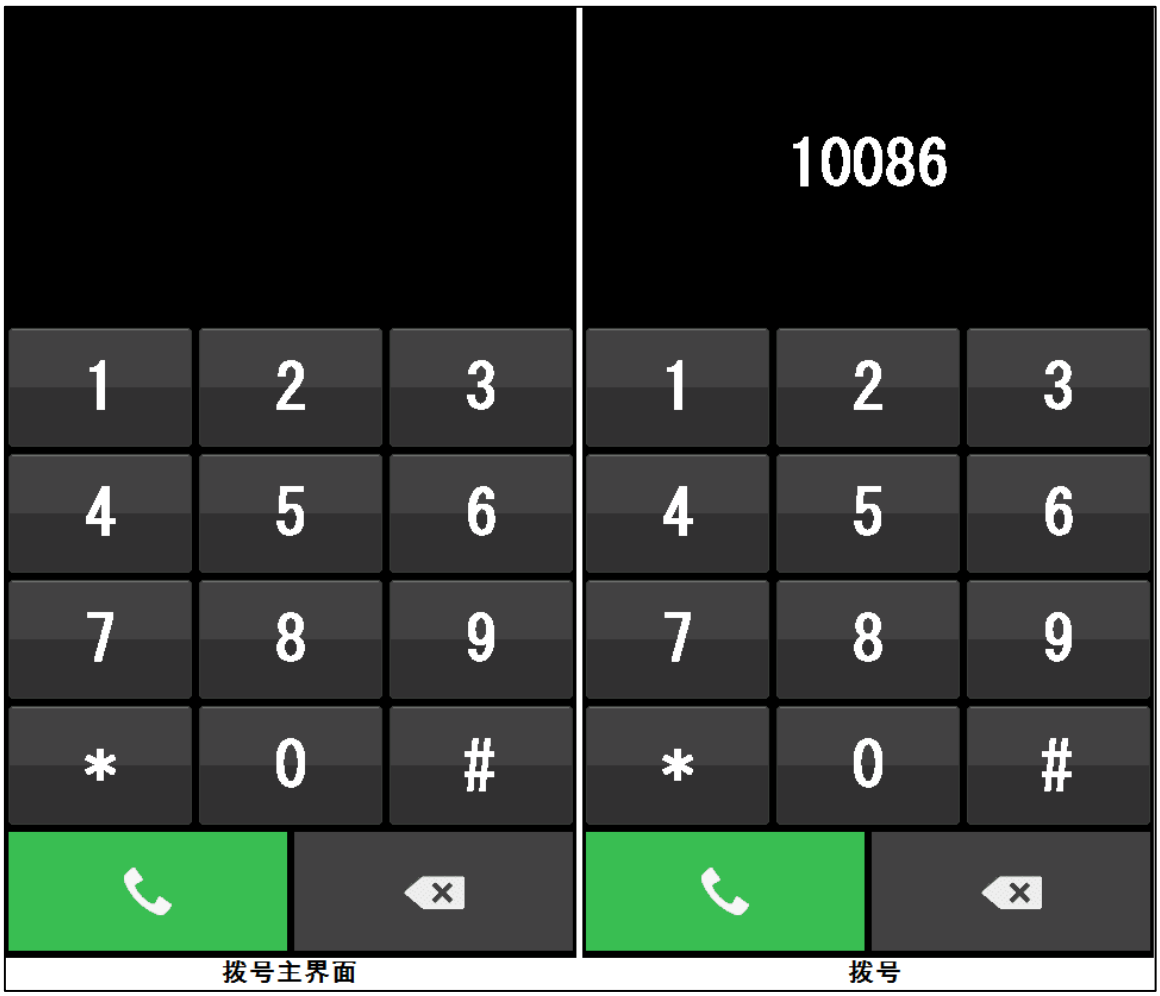


图 78.2.25.1 拨号主界面和拨号

上图中左侧图片就是拨号主界面，这个和手机拨号是一样的。右侧是我们输入的拨号号码，点击拨号图标，即可进行拨号。如图 78.2.25.2 所示。

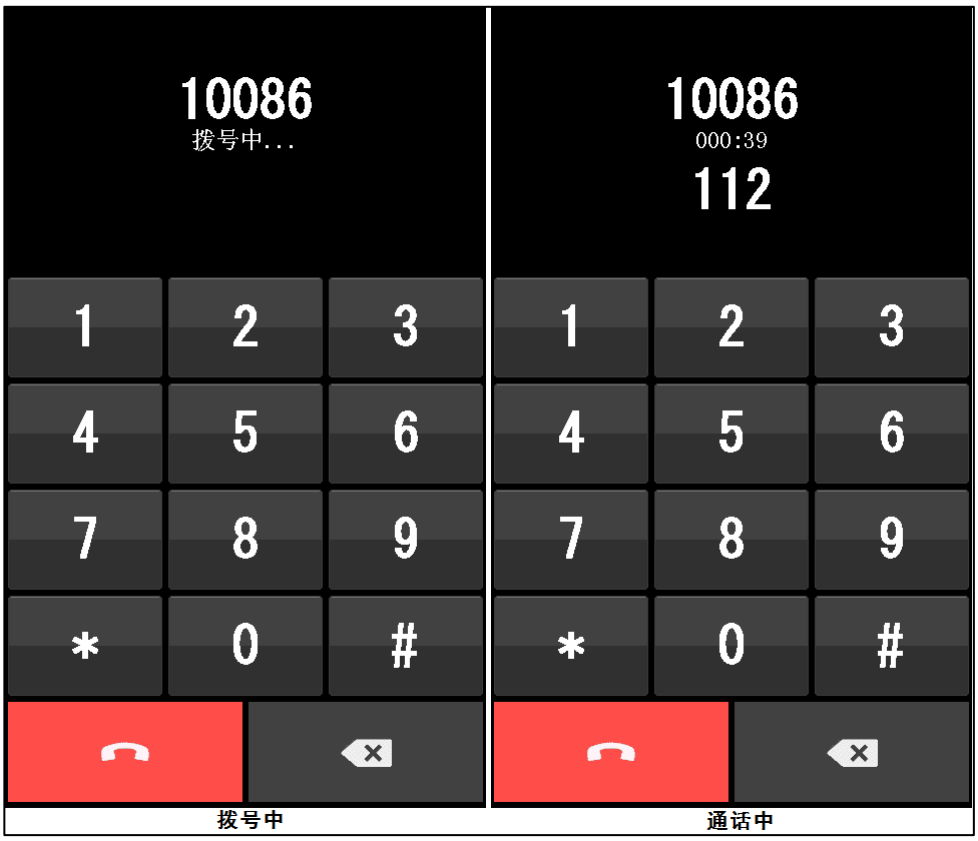


图 78.2.25.2 拨号中和通话中

图 78.2.25.3 来电和接听来电

图 78.2.25.2 为拨号和拨通后的通话界面。图 78.2.25.3 为来电和接听来电后的通话界面，

此时蜂鸣器会发出“滴、滴”的提示声，提示有电话呼入。其他的操作和我们智能手机基本一模一样，就无需多说了。

注意，在通话状态，如果按 TPAD，则会挂断电话，结束通话。

78.2.26 应用中心

双击主界面的应用中心图标，进入应用中心界面，如图 78.2.26.1 所示：

图 78.2.26.1 应用中心和红外遥控测试

左侧图片是我们刚进入应用中心看到的界面，在该界面下总共有 16 个图标，我们仅实现了前三个功能：红外遥控、光纤接收和指南针。其他都没有实现，大家可以自由发挥，添加属于自己的东西。

双击第一个图标，会弹出一个红外遥控的小窗口，用于接收红外信号，此时，我们将红外遥控对准开发板的红外接收头，并按下按钮，就可以在红外遥控窗口里面显示键值、按键次数、符号等信息。如图 78.2.26.1 右侧图片所示。

按 TPAD 可以退出红外遥控功能，返回应用中心主界面，然后按第二个图标，即可进入光纤接收测试界面。

进入光纤接收功能后，利用 STM32H7 的 SPDIF RX 接口接收来自光纤接口的音频信号，并通过 ES8388 播放出来，光纤接收界面如图 78.2.26.2 所示：

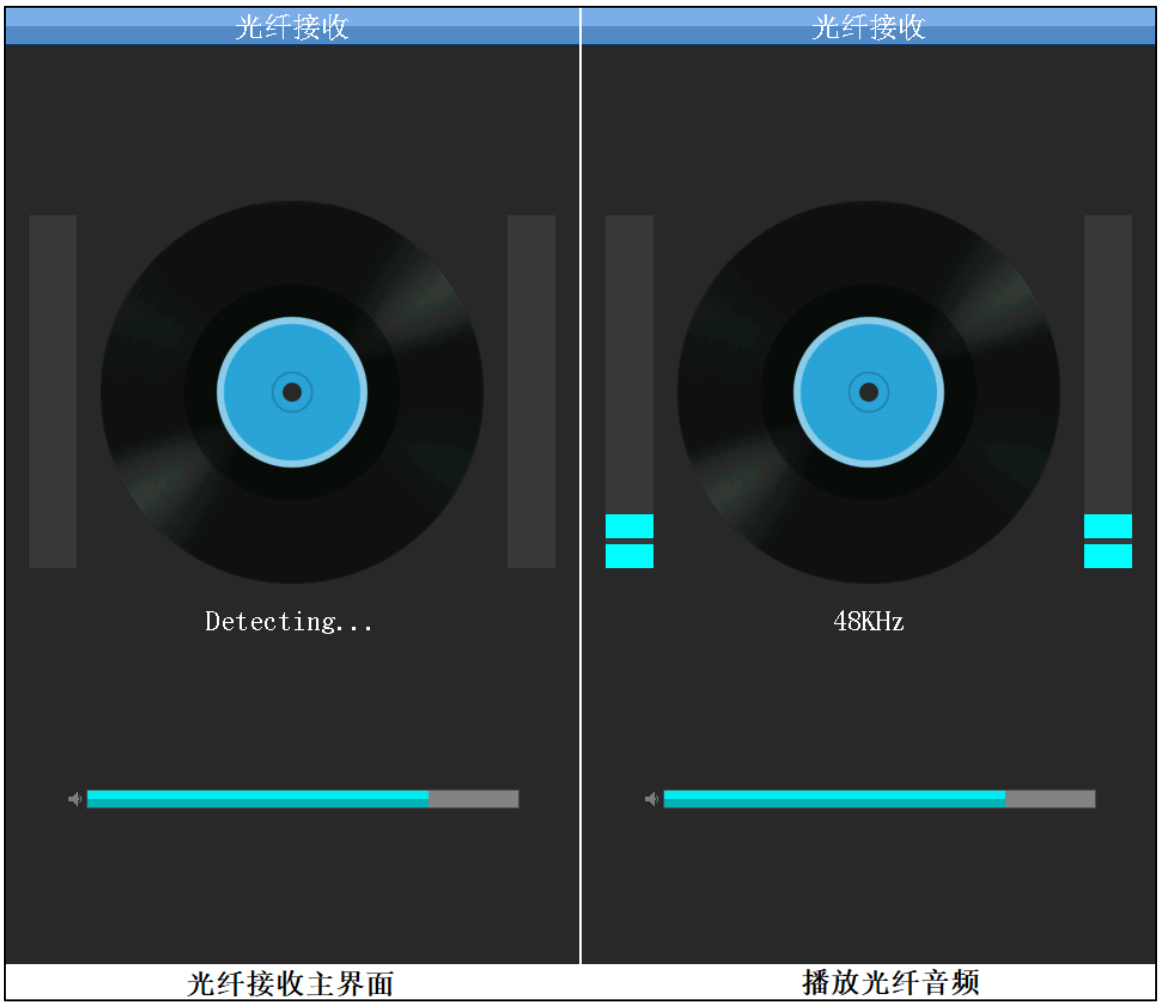


图 78.2.26.2 光纤接收和播放光纤音频

上图中，左侧图片为没有接入光纤信号时，屏幕显示 Detecting，表示正在检测音频信号。当检测到有音频信号输入的时候，如右侧图片所示，会显示音频信号的采样率，同时显示左右声道的柱状信号强度，此时我们可以通过开发板的喇叭或耳机欣赏音乐。通过下方的音量条可以调节音量大小。

光纤接收支持：44.1Khz、48Khz、88.2Khz 和 96Khz 等几种采样率的音频信号。

注意：光纤接收功能的测试，需要大家自备光纤音频发送设备一台以及光纤线一条。光纤音频发送设备，大家可以选择 ALIENTEK 的贝斯蓝牙音频接收器，它可以通过蓝牙和手机连接，也可以连接电脑 USB，做 USB 声卡，将蓝牙/USB 信号转换为光纤音频信号输出，贝斯蓝牙音频接收器购买地址：http://www.openedv.com/thread-86409-1-1.html**。**

按 TPAD 返回应用中心主界面，再按 TPAD 返回系统主界面。

然后双击进入指南针窗口后，此时，显示屏上显示一个电子罗盘并显示当前指向，如图 78.2.26.3 所示：

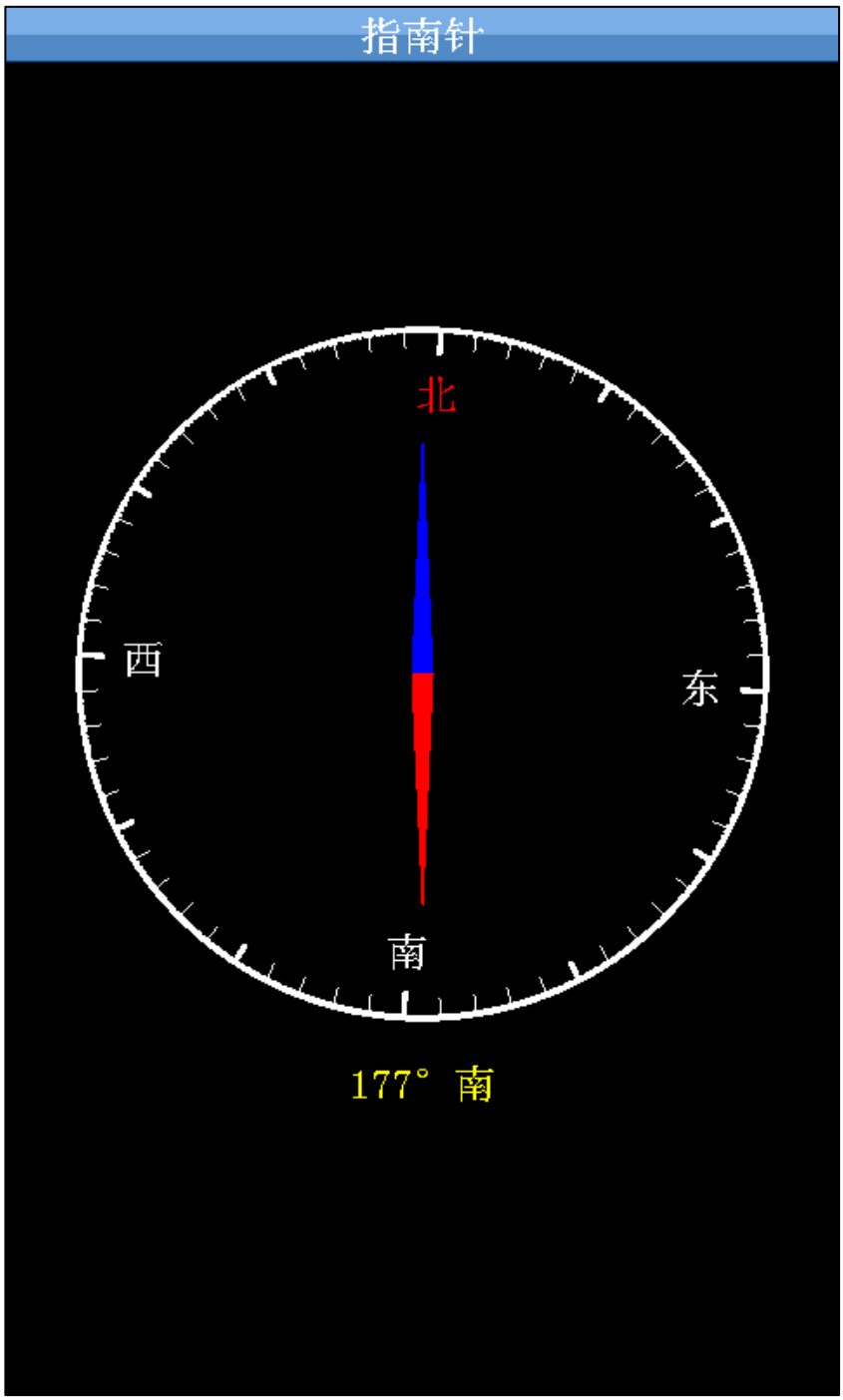


图 75.2.26.3 指南针

这里的指南针就是通过开发板板载的 ST480MC 传感器实现。按下 KEY_UP 按键可进行校准，校准后偏差会比较小。

按 TPAD，则会退出指南针功能，回到应用中心主界面，再按一次 TPAD 就可以返回主界面。

78.2.27 电压表

电压表功能用于测量 0~3.3V 的直流电压源，可以是开发板上的电压源，也可以是外部的，但是电压的范围一定要是 0~3.3V，否则容易烧毁 STM32 芯片。

下面我们一起来测量板载的可调电位器的电压，双击主界面的电压表图标，屏幕会弹出提示对话框，如图 78.2.27.1 所示：

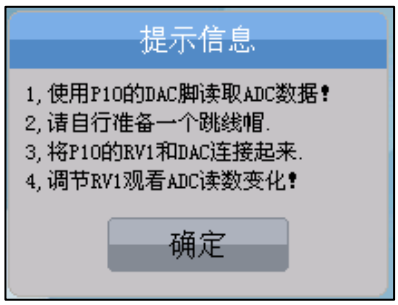


图 78.2.27.1 双击电压表后提示界面

提示信息告诉我们，需要使用短路帽将多功能端口 P10 的 DAC 和 RV1 排针连接。

进入电压表功能后，就可以通过调节可调点位旋钮改变接入 ADC 的电压，从而在屏幕上显示不同的电压值，如图 78.2.27.2 左侧图所示：

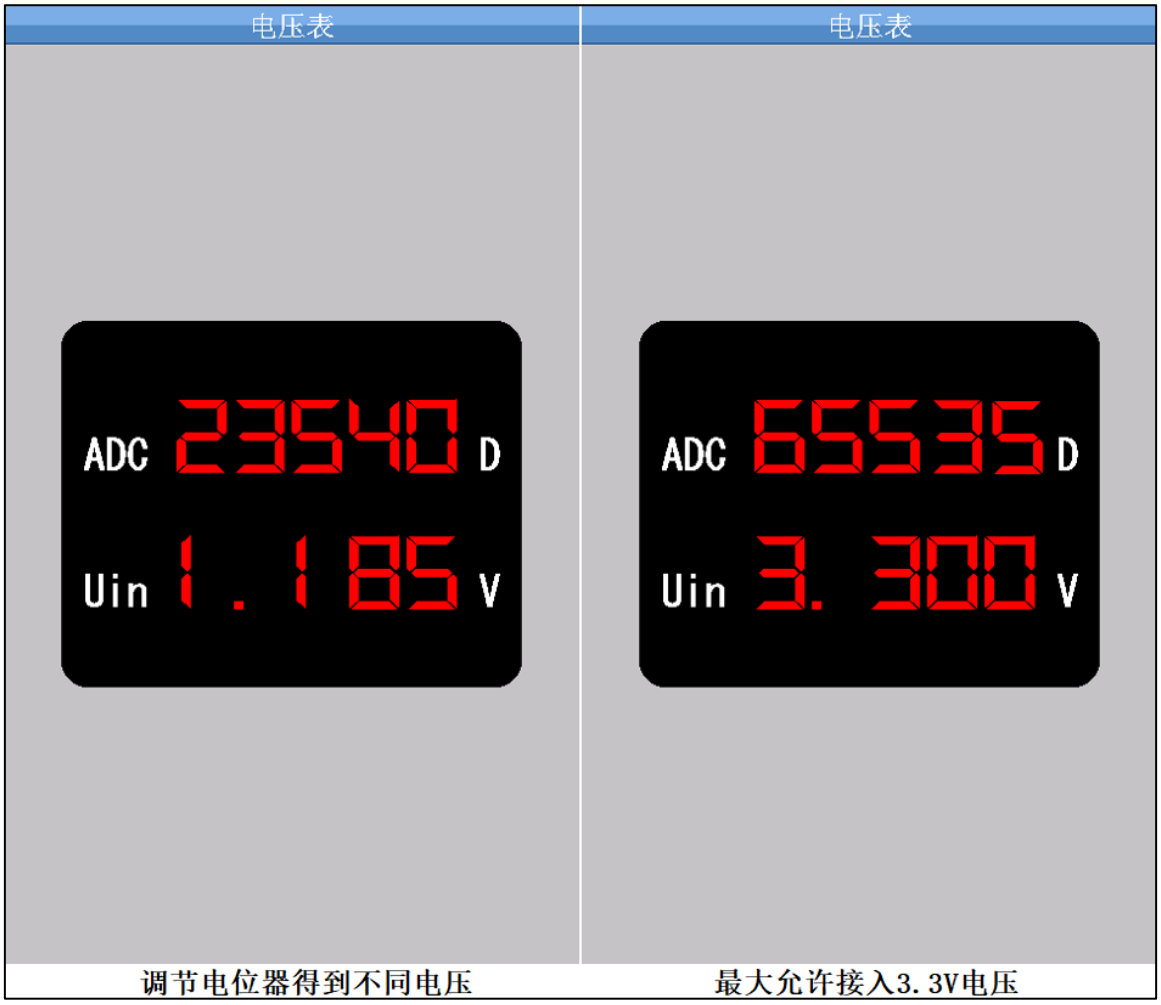


图 78.2.27.2 调节电位器得到不同电压和最大允许接入 3.3V 电压

上图中，ADC 值是电压的数字值，Uin 值是经过转换后的实际电压。由于 STM32H743 的 ADC 最大分辨率是 16 位，所以电压的数字值范围是 0~65535。

当我们把可调电压值调至最大，就可以得到 3.3V 的最大输入电压，如图 78.2.27.2 右侧图所示。电压的数字值是 65535，对应的实际电压是 3.3V。

至此，开发板的综合测试实验就介绍完了。其中，参考了不少网友的代码，对这些网友表示衷心的感谢，同时我也希望我们的这个代码，可以让大家有所受益，能开发出更强更好的产品。

综合实验整个代码编译后大小为 637K 左右（-O2 优化后），代码量是非常的大，希望大家慢慢理解，各个攻破，最后祝大家身体健康、学习进步！